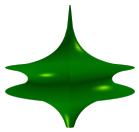


XlCalcNet Documentation

Release 1.0.0

Dietrich Hadler

October 23, 2026

Welcome to the documentation of **XlCalcNet**, an addin for MS E**x**ce**l** and LibreOffice **Calc**, based on Python**Net**. This edition of the documentation describes release 1.0.0.

XlCalcNet is free software released under the Mozilla Public License 2.0 (see *License*).

The git repository is https://github.com/duhadler/xlcalcnet)

This documentation is available online at https://duhadler.github.io/XlCalcNetDocsHTML/.

The pdf can be downloaded from https://github.com/duhadler/DocsXlCalcNet/raw/master/pdf/xlcalcnet.pdf.

Copyright © 2020-2026 Dietrich Hadler

PREFACE

Spreadheet formulas (as used in MS Excel and LibreOffice Calc) are probably the most widely used programming language in the world. In addition to the built-in formulas, there is also the option to write user-defined functions (UDFs) in the Basic dialects of these programs.

XlCalcNet extends these options by providing an additional set of functions which are accessible through the function dialog (MS Excel) or function wizard (LibreOffice Calc), and expect as first argument a string containing code written in Python. This code can call any function that is available in the underlying Python installation (currently supported are versions 3.8 - 3.15), which the user can populate with additional packages as needed.

Contents

Preface i

I Getting started 1

1 Setup and general usage 2

1.1 Setting up XlCalcNet . . . 3
1.1.1 Installing XlCalcNet . . . 3
1.1.2 Installing and using Python.NET: Calling C# from Python . . . 5
1.1.3 Installing and using the Tiny IDE as a Python application . . . 6
1.1.4 Installing and using Numpy, Matplotlib, Pandas, Scipy, Seaborn, Statsmodels and Scikit-learn . . . 20
1.1.5 Calling Python from C:#: socket server vs in-process calls . . . 34
1.1.6 Reasons for using multiprecision arithmetic . . . 39
1.1.7 Rebuilding the .dll files of XlCalcNet and XlCalcNet2 from source code . . . 41
1.2 Using XlCalcNet within MS Excel . . . 42
1.2.1 Preparing MS Excel for using XlCalcNet . . . 42
1.2.2 Using the Python standard library functions within spreadsheet formulas . . . 43
1.2.3 Using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet within spreadsheet formulas . . . 45
1.2.4 Using dynamic arrrays in MS Excel . . . 51
1.2.5 Managing procedures instead of functions . . . 53
1.2.6 Uninstalling . . . 54
1.3 Using XlCalcNet within LibreOffice Calc . . . 55
1.3.1 Preparing LibreOffice Calc for using XlCalcNet . . . 55
1.3.2 Using the Python standard library functions within spreadsheet formulas . . . 55
1.3.3 Using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet within spreadsheet formulas . . . 58
1.3.4 Using links to external `*.csv`, `*.xlsx` and `*.svg` files . . . 64
1.3.5 Managing procedures instead of functions . . . 71
1.3.6 Exporting and uninstalling . . . 72
1.4 General user interface and functions . . . 74
1.4.1 The Tiny Data Viewer . . . 74
1.4.2 The Output Monitor . . . 76
1.4.3 The Gallery of Plots . . . 78
1.4.4 Interactive 3D wpf plots . . . 83
1.4.5 Functions related to folders . . . 91
1.4.6 Information about installation status of supporting python packages . . . 92
1.4.7 Information about context lists, and setting global precision . . . 93
1.4.8 Support for running interactive Matplotlib output in a separate process . . . 94
1.5 Mathematical functions based on Mpmath, Gmpy2 and Python-Flint (only Python) . . . 95
1.5.1 Overview . . . 95
1.5.2 Double-precision arithmetic: `fpm` . . . 98
1.5.3 Binary floating-point with arbitrary-precision and exponent: `mpm` . . . 102
1.5.4 Interval arithmetic with arbitrary-precision and exponent:`ipm` . . . 106
1.5.5 Decimal floating-point in arbitrary-precision with limited exponent: `dpm` . . . 110

1.5.6 Rational numbers: qpm . . . 115
1.5.7 Binary floating-point in arbitrary-precision with limited exponent: gpm . . . 117
1.5.8 Binary balls in arbitrary-precision and with arbitrary exponent: apm . . . 122
1.5.9 Numpy support for multiprecision data types: npm . . . 126
1.6 Mathematical functions in fixed precision (C#, can be called from Python) . . . 127
1.6.1 C# based context groups . . . 127
1.6.2 Scalar functions with Double and Complex: math53 and cmath53 . . . 128
1.6.3 Binary floating point, single precision: sreal and scplx . . . 134
1.6.4 Binary floating point, double precision: dreal and dcplx . . . 140
1.6.5 Binary floating point, extended precision: ereal and ecplx . . . 146
1.6.6 Binary floating point, quadruple precision: qreal and qcplx . . . 152
1.6.7 Binary floating point, octuple precision: oreal and ocplx . . . 158
1.6.8 Use of Numpy with scalar functions written in C#: np53, np53c . . . 163
1.6.9 Building the underlying C#, C/C++ and Pascal libraries . . . 164
1.7 Mathematical functions based on XlCalcNet2 (C#, can be called from Python) . . . 165
1.7.1 Binary floating point, multiple precision: mreal, mcplx . . . 165
1.7.2 Binary arbitrary precision ball: aflint and aflintc . . . 171
1.7.3 Binary floating point, single precision: sflint and sflintc . . . 176
1.7.4 Binary floating point, double precision: dflint and dflintc . . . 180
1.7.5 Binary floating point, extended precision: eflint and eflintc . . . 184
1.7.6 Binary floating point, quadruple precision: qflint and qflintc . . . 188
1.7.7 Binary floating point, octuple precision: oflint and oflintc . . . 192
1.7.8 Binary floating point, multiple precision: mflint, mflintc . . . 196
1.7.9 Building the underlying C# and C/C++ libraries . . . 199
1.8 Building user libraries and documenting them . . . 200
1.8.1 Building a library of user defined Python functions . . . 200
1.8.2 Building a library of user defined functions in C# . . . 200
1.8.3 Sphinx: building documentation for software with Python (html and pdf) . . . 200

2 Basic floating point functions **201**
2.1 Operator overloading, general real functions . . . 201
2.1.1 Operator overloading: scope and restrictions . . . 201
2.1.2 The cost of creating and destroying objects . . . 201
2.1.3 Raw addition of two floating point numbers . . . 201
2.1.4 Raw subtraction of two floating point numbers . . . 201
2.1.5 Raw multiplication of two floating point numbers . . . 202
2.1.6 Raw division of two floating point numbers . . . 202
2.1.7 Raw addition of a floating point number and a signed 32 bit integer . . . 202
2.1.8 Raw subtraction of a floating point number and a signed 32 bit integer . . . 202
2.1.9 Raw multiplication of a floating point number and a signed 32 bit integer . . . 203
2.1.10 Raw division of a floating point number and a signed 32 bit integer . . . 203
2.1.11 Fused multiplication and addition: $\mathrm{fma}(x, y, z)$ . . . 203
2.1.12 Maximum of two floating point numbers: $\mathrm{fmax}(x, y)$ . . . 204
2.1.13 Minimum of two floating point numbers: $\mathrm{fmin}(x, y)$ . . . 204
2.2 Machine constants, general . . . 205
2.2.1 Zero, positive zero . . . 205
2.2.2 Negative zero . . . 205
2.2.3 One . . . 205
2.2.4 Imaginary One . . . 206
2.2.5 Positive infinity . . . 206
2.2.6 Negative infinity . . . 206
2.2.7 NaN (Not a Number) . . . 207
2.3 Properties of numbers . . . 208
2.3.1 Test for signbit . . . 208

2.3.2 Test for a finite number . . . 208
2.3.3 Test for infinity . . . 208
2.3.4 Test for positive infinity . . . 208
2.3.5 Test for negative infinity . . . 209
2.3.6 Test for NaN . . . 209
2.3.7 Test for zero . . . 209
2.3.8 Test for one . . . 209
2.3.9 Test if a number is an integer . . . 210
2.3.10 Test if a number is a simple number (not an infinity or NaN) . . . 210
2.3.11 Test if a number is a regular number (not zero, an infinity or NaN) . . . 210
2.3.12 Test if a number is a normal number (not subnormal, zero, an infinity or NaN) . . . 210
2.3.13 Test if 2 numbers are unordered . . . 211
2.3.14 Test if a number fits in a signed 32 bit integer . . . 211
2.3.15 Test if a number fits in a signed 64 bit integer . . . 211
2.4 Integer related functions . . . 212
2.4.1 Integral value nearest x (rounding to nearest): $\mathrm{nearbyint}(x), \mathrm{rint}(x)$ . . . 212
2.4.2 32 bit integer nearest x (rounding to nearest): $\mathrm{lrint}(x)$ . . . 212
2.4.3 64 bit integer nearest x (rounding to nearest): $\mathrm{llrint}(x)$ . . . 212
2.4.4 Smallest integral value not less than x : $\mathrm{ceil}(x)$ . . . 213
2.4.5 Largest integral value not greater than x : $\mathrm{floor}(x)$ . . . 213
2.4.6 Nearest integral value not greater in magnitude than x : $\mathrm{trunc}(x)$ . . . 213
2.4.7 Nearest integral value, rounding halfway cases away from zero: $\mathrm{round}(x)$ . . . 214
2.4.8 Nearest 32 bit integer, rounding halfway cases away from zero: $\mathrm{lround}(x)$ . . . 214
2.4.9 Nearest 64 bit integer, rounding halfway cases away from zero: $\mathrm{llround}(x)$ . . . 214
2.5 Floating point functions for real numbers . . . 215
2.5.1 Efficient computation of $|x| \cdot \mathrm{sign}(y)$: $\mathrm{copysign}(x, y)$ . . . 215
2.5.2 Decomposition of x as $x = m \cdot 2^e; 0.5 < m < 1$: $\mathrm{frexp}(x)$ . . . 215
2.5.3 Integral value e, from the decomposition of x as $x = m \cdot 2^e; 1 < m < 2$: $\mathrm{logb}(x)$ . . . 216
2.5.4 32 bit integer e, from the decomposition of x as $x = m \cdot 2^e; 1 < m < 2$: $\mathrm{ilogb}(x)$ . . . 216
2.5.5 Efficient computation of $x \cdot 2^e$: $\mathrm{ldexp}(x, e), \mathrm{scalbn}(x, e), \mathrm{scalbln}(x, e)$ . . . 217
2.5.6 Positive difference between x and y: $\mathrm{fdim}(x, y)$ . . . 217
2.6 Fraction and remainder related functions . . . 219
2.6.1 Integral and fractional part of a floating point number: $\mathrm{modf}(x)$ . . . 219
2.6.2 Floating point remainder: $\mathrm{fmod}(x, y)$ . . . 219
2.6.3 IEEE floating point remainder: $\mathrm{remainder}(x, y)$ . . . 220
2.6.4 IEEE floating point remainder and part of quotient: $\mathrm{remquo}(x, y)$ . . . 220
2.7 Functions related to mantissa width and exponent range . . . 222
2.7.1 Machine Epsilon . . . 222
2.7.2 Unit in the last place . . . 222
2.7.3 Largest representable number . . . 222
2.7.4 Lowest representable number . . . 223
2.7.5 Smallest representable positive number . . . 223
2.7.6 Next representable floating point number: $\mathrm{nexttowards}(x, y)$ . . . 223
2.7.7 Next representable floating point number: $\mathrm{nextabove}(x)$ . . . 224
2.7.8 Next representable floating point number: $\mathrm{nextbelow}(x)$ . . . 224
2.8 Mathematical Constants . . . 225
2.8.1 Degree . . . 225
2.8.2 Golden ratio phi . . . 225
2.8.3 Natural logarithm of 2 . . . 225
2.8.4 Natural logarithm of 10 . . . 226
2.8.5 Pi (π) . . . 226
2.8.6 Euler e . . . 226
2.8.7 Euler-Mascheroni constant γ . . . 227
2.8.8 Apéry's constant . . . 227

2.8.9 Catalan's constant . . . 227
2.8.10 Glaisher's constant . . . 228
2.8.11 Khinchin's constant . . . 228

3 Elementary scalar functions (real and complex) 229
3.1 Complex components . . . 229
3.1.1 Absolute value of a real or complex number . . . 229
3.1.2 Sign of a real or complex number . . . 229
3.1.3 Real part of a real or complex number . . . 230
3.1.4 Imaginary part of a real or complex number . . . 230
3.1.5 Phase (or argument) of a real or complex number . . . 230
3.1.6 Conjugate of a real or complex number . . . 230
3.1.7 Polar representation of a real or complex number . . . 231
3.1.8 Rectangular coordinates calculated from the polar representation of a real or complex number . . . 231
3.2 Roots and quadratic, cubic, and quartic equations . . . 232
3.2.1 Square root, $\mathrm{sqrt}(x) = \sqrt{x}$ . . . 232
3.2.2 Reciprocal square root, $\mathrm{rsqrt}(x) = 1/\sqrt{x}$ . . . 233
3.2.3 Auxiliary function $\mathrm{sqrt1pm1}(x) = \sqrt{1+x} - 1$ . . . 234
3.2.4 Cube root, $\mathrm{cbrt}(x) = \sqrt[3]{x}$ . . . 235
3.2.5 Nth root, $\mathrm{nroot}(x, n) = \sqrt[n]{x}$ . . . 236
3.2.6 Unit root, $\mathrm{unitroot}(n)$ . . . 237
3.3 Exponential and related functions . . . 238
3.3.1 Exponential function $\exp(x) = e^x$ . . . 238
3.3.2 Auxiliary function $\mathrm{expj}(x) = e^{ix}$ . . . 239
3.3.3 Auxiliary function $\mathrm{expjpi}(x) = e^{i\pi x} = (-1)^x$ . . . 240
3.3.4 Exponential function with base 10, $\mathrm{exp10}(x) = 10^z$ . . . 240
3.3.5 Exponential function with base 2, $\mathrm{exp2}(x) = 2^x$ . . . 241
3.3.6 Auxiliary function $\mathrm{expm1}(x) = e^x - 1$ . . . 242
3.3.7 Auxiliary function $\mathrm{exp10m1}(x) = 10^x - 1$ . . . 243
3.3.8 Auxiliary function $\mathrm{exp2m1}(x) = 2^x - 1$ . . . 244
3.4 Logarithms and related functions . . . 245
3.4.1 Logarithm $\log(x[, base])$ . . . 245
3.4.2 Logarithm with base 10, $\mathrm{log10}(x) = \log_{10}(x)$ . . . 246
3.4.3 Logarithm with base 2, $\mathrm{log2}(x) = \log_2(x)$ . . . 247
3.4.4 Auxiliary function $\mathrm{log1p}(x) = \log(x+1)$ . . . 248
3.4.5 Auxiliary function $\mathrm{log10p1}(x) = \log_{10}(1+x)$ . . . 249
3.4.6 Auxiliary function $\mathrm{log2p1}(x) = \log_2(1+x)$ . . . 249
3.5 Power functions . . . 251
3.5.1 Square, x^2 . . . 251
3.5.2 Cube, x^3 . . . 251
3.5.3 Auxiliary function $\mathrm{powi}(x, n) = x^n$ . . . 252
3.5.4 Auxiliary function $\mathrm{hypot}(x, y) = \sqrt{x^2 + y^2}$ . . . 253
3.5.5 Power function, $\mathrm{pow}(x, y) = x^y$ . . . 253
3.5.6 Auxiliary function, $\mathrm{powm1}(x, y) = x^y - 1$ . . . 254
3.5.7 Auxiliary function, $\mathrm{pow1p}(x, y) = (1+x)^y$ . . . 255
3.5.8 Auxiliary function, $\mathrm{pow1pm1}(x, y) = (1+x)^y - 1$ . . . 256
3.6 Trigonometric functions, in radians . . . 258
3.6.1 Sine, $\sin(x)$ . . . 258
3.6.2 Cosine, $\cos(x)$ . . . 260
3.6.3 Tangent, $\tan(x)$ . . . 261
3.6.4 Cotangent, $\cot(x)$ . . . 262
3.6.5 Cosecant, $\csc(x)$ . . . 264
3.6.6 Secant, $\sec(x)$ . . . 265
3.6.7 Cardinal sine, $\mathrm{sinc}(x)$ . . . 267

3.7 Trigonometric functions, in multiples of π . . . 268
3.7.1 Auxiliary function $\mathrm{sinpi}(x) = \sin(\pi x)$ . . . 268
3.7.2 Auxiliary function $\mathrm{cospi}(x) = \cos(\pi x)$ . . . 268
3.7.3 Auxiliary function $\mathrm{tanpi}(x) = \tan(\pi x)$ . . . 269
3.7.4 Auxiliary function $\mathrm{cotpi}(x) = \cot(\pi x)$ . . . 270
3.7.5 Auxiliary function $\mathrm{cscpi}(x) = \csc(\pi x)$ . . . 270
3.7.6 Auxiliary function $\mathrm{secpi}(x) = \sec(\pi x)$ . . . 270
3.7.7 Auxiliary function $\mathrm{sincpi}(x) = \mathrm{sinc}(\pi x)$ . . . 270
3.8 Hyperbolic functions . . . 272
3.8.1 Hyperbolic sine, $\sinh(x)$ . . . 272
3.8.2 Hyperbolic cosine, $\cosh(x)$ . . . 273
3.8.3 Hyperbolic tangent, $\tanh(x)$ . . . 275
3.8.4 Hyperbolic cotangent, $\coth(x)$ . . . 276
3.8.5 Hyperbolic cosecant, $\mathrm{csch}(x)$ . . . 277
3.8.6 Hyperbolic secant, $\mathrm{sech}(x)$ . . . 279
3.9 Inverse trigonometric functions, in radians . . . 281
3.9.1 Inverse sine, $\mathrm{asin}(x)$ . . . 281
3.9.2 Inverse cosine, $\mathrm{acos}(x)$ . . . 282
3.9.3 Inverse tangent, $\mathrm{atan}(x)$ . . . 284
3.9.4 Inverse tangent, 2 arguments, $\mathrm{atan2}(y, x)$ . . . 285
3.9.5 Inverse cotangent, $\mathrm{acot}(x)$ . . . 286
3.9.6 Inverse cosecant, $\mathrm{acsc}(x)$ . . . 287
3.9.7 Inverse secant, $\mathrm{asec}(x)$ . . . 289
3.10 Inverse hyperbolic functions . . . 291
3.10.1 Inverse hyperbolic sine, $\mathrm{asinh}(x)$ . . . 291
3.10.2 Inverse hyperbolic cosine, $\mathrm{acosh}(x)$ . . . 292
3.10.3 Inverse hyperbolic tangent, $\mathrm{atanh}(x)$ . . . 293
3.10.4 Inverse hyperbolic cotangent, $\mathrm{acoth}(x)$ . . . 295
3.10.5 Inverse hyperbolic secant, $\mathrm{asech}(x)$ . . . 296
3.10.6 Inverse Hyperbolic Cosecant, $\mathrm{acsch}(x)$ . . . 297
3.11 Factorials, Gamma and related functions . . . 299
3.11.1 Gamma function, $\Gamma(x)$ . . . 299
3.11.2 Auxiliary function $\Gamma(x+1) - 1$ . . . 300
3.11.3 Log-gamma function, $\log\Gamma(x)$ . . . 301
3.11.4 Reciprocal Gamma function, $1/\Gamma(x)$ . . . 303
3.11.5 Factorial, $x!$ . . . 305
3.11.6 Double factorial, $x!!$ . . . 306
3.11.7 Rising factorial $a^{\overline{n}} = (a)_n$ . . . 307
3.11.8 Falling factorial, $(a)^{\underline{n}} = (a-n+1)^{\overline{n}}$ . . . 309
3.11.9 Ratio of gamma functions, $\Gamma(a)/\Gamma(b)$ . . . 309
3.11.10 Gamma-delta ratio, $\Gamma(a)/\Gamma(a+\delta)$ . . . 309
3.11.11 Beta function, $B(a,b) = \Gamma(a)\Gamma(b)/\Gamma(a+b)$ . . . 310
3.11.12 Binomial coefficient, ${}_nC_k = (k \cdot B(k, n-k+1))^{-1}$ . . . 311
3.12 Miscellaneous functions . . . 312
3.12.1 Lambert W function, $W_0(x)$ . . . 312
3.12.2 Lambert W function, $W_{-1}(x)$ . . . 313
3.12.3 Lambert W (general case) . . . 313
3.12.4 Arithmetic-geometric mean . . . 314

4 Statistical Distributions **316**
4.1 Introduction to random variables and distributions . . . 316
4.1.1 Elementary symbolic algebra of random variables . . . 316
4.1.2 Linear transformations of random variables . . . 318
4.1.3 Random variables having mixture distributions . . . 319

4.1.4 Linear combinations of independent random variables . . . 320
4.1.5 The log-transform of continuous random variables . . . 321
4.2 Base class for univariate distributions . . . 322
4.2.1 Boost: Cumulative distribution function . . . 322
4.2.2 Boost: Survival function . . . 322
4.2.3 Hazard function . . . 323
4.2.4 Cumulative Hazard Function . . . 323
4.2.5 Boost: Quantile function . . . 323
4.2.6 Boost: Inverse survival function . . . 324
4.2.7 Boost: Mode . . . 324
4.2.8 Boost: Expected value (mean) . . . 324
4.2.9 Boost: Median . . . 324
4.2.10 Boost: Variance . . . 325
4.2.11 Boost: Standard deviation . . . 325
4.2.12 Boost: Skewness . . . 325
4.2.13 Boost: Kurtosis . . . 325
4.2.14 Boost: Kurtosis excess . . . 326
4.2.15 Boost: Support, lower endpoint . . . 326
4.2.16 Boost: Support, upper endpoint . . . 326
4.2.17 Boost: Range, lower endpoint . . . 327
4.2.18 Boost: Range, upper endpoint . . . 327
4.2.19 Characteristic function . . . 327
4.2.20 Moment generating function . . . 328
4.2.21 Cumulant generating function . . . 328
4.2.22 Raw Moments . . . 329
4.2.23 Central Moments . . . 329
4.2.24 Cumulants . . . 330
4.3 Base class for continuous univariate distributions . . . 331
4.3.1 Probability density function . . . 331
4.3.2 Probability sparsity function . . . 331
4.3.3 Differerential entropy . . . 332
4.3.4 Negative loglikelihood function . . . 332
4.3.5 Logarithm of the probability density function . . . 332
4.4 Base class for discrete univariate distributions . . . 333
4.4.1 Probability mass function . . . 333
4.4.2 Probability generating function . . . 333
4.4.3 Vector containing all values of the pmf . . . 334
4.4.4 Entropy . . . 334
4.4.5 Logarithm of the probability mass function . . . 335
4.5 Closed form distributions, based on elementary functions . . . 336
4.5.1 Boost: Arcsine Distribution . . . 336
4.5.2 Boost: Cauchy distribution . . . 340
4.5.3 Boost: Exponential distribution . . . 343
4.5.4 Boost: Gumbel (Generalized Extreme Value distribution Type-I) distribution . . . 347
4.5.5 Boost: Hyperexponential Distribution . . . 351
4.5.6 !!!Boost: Kumaraswamy distribution . . . 355
4.5.7 Boost: Laplace distribution . . . 359
4.5.8 Boost: Logistic distribution . . . 363
4.5.9 Boost: Pareto distribution . . . 367
4.5.10 Boost: Rayleigh distribution . . . 371
4.5.11 Boost: Triangular Distribution . . . 375
4.5.12 Boost: Uniform distribution . . . 379
4.5.13 Boost: Weibull (Minimum-Type-III) distribution . . . 383
4.5.14 Dagum (Burr Type III) distribution . . . 387

4.5.15 Fisk (log-logistic) distribution . . . 390
4.5.16 Fréchet (Maximum/Minimum-Type-II or Inverse Weibull) distribution . . . 393
4.5.17 Generalized Extreme Value (Maximum) or GEV distribution . . . 396
4.5.18 Generalized Pareto distribution . . . 399
4.5.19 Gompertz-Makeham distribution . . . 402
4.5.20 Lomax distribution . . . 405
4.5.21 Shifted Gompertz distribution . . . 408
4.5.22 Singh-Maddala (Burr Type XII) distribution . . . 411
4.6 Closed form distributions, based on the error function . . . 414
4.6.1 !!!Boost: Lévy distribution . . . 415
4.6.2 Boost: Lognormal (Johnson S_L) distribution . . . 418
4.6.3 !!!Boost: Moyal Distribution . . . 422
4.6.4 Boost: Normal (Johnson S_N) distribution . . . 426
4.6.5 Boost: Skew normal Distribution . . . 430
4.6.6 Boost: Wald (or Inverse Gaussian) distribution . . . 434
4.6.7 Birnbaum-Saunders Distribution . . . 437
4.6.8 Exponentially Modified Gaussian (EMG) distribution . . . 441
4.6.9 Folded normal distribution . . . 444
4.6.10 Half-normal distribution . . . 447
4.6.11 Johnson S_B distribution . . . 450
4.6.12 Johnson S_U distribution . . . 453
4.6.13 Normal maximum distribution, $\rho_{ij,i\neq j} = 0$ . . . 456
4.6.14 Normal maximum modulus distribution, $\rho_{ij,i\neq j} = 0$ . . . 459
4.6.15 Sinh-arcsinh normal distribution . . . 462
4.6.16 Truncated normal distribution . . . 465
4.7 Closed form distributions, based on the incomplete gamma function . . . 468
4.7.1 !!!Boost: Chi Distribution . . . 468
4.7.2 Boost: Chi2 distribution . . . 472
4.7.3 Boost: Gamma (Pearson Type III, Erlang) distribution . . . 477
4.7.4 Boost: Inverse chisquared distribution . . . 481
4.7.5 Boost: Inverse Gamma (Pearson Type V) distribution . . . 485
4.7.6 !!!Boost: Maxwell Distribution . . . 489
4.7.7 !!!Boost: Nakagami distribution . . . 493
4.7.8 Amoroso distribution . . . 497
4.7.9 Distribution of the logarithm of a χ^2 random variable . . . 500
4.7.10 Hypoexponential (Generalized Erlang) Distribution . . . 503
4.7.11 Lindley distribution (generalized) . . . 507
4.7.12 Skew exponential power distribution . . . 510
4.7.13 Stacy (generalized gamma) distribution . . . 513
4.8 Closed form distributions, based on the incomplete beta function . . . 516
4.8.1 Boost: Beta (Pearson Type I and II) distribution . . . 516
4.8.2 Boost: Central Fisher F distribution . . . 521
4.8.3 Boost: Student t (Pearson Type VII) distribution . . . 526
4.8.4 Distribution of the negative logarithm of a beta variable . . . 530
4.8.5 Beta-prime (Pearson Type VI) distribution . . . 533
4.8.6 Generalized Beta (Type 1) distribution . . . 536
4.8.7 Generalized Beta (Type 2) distribution . . . 539
4.8.8 Generalized logistic distribution (JKB Types I - IV) . . . 542
4.8.9 Generalized beta-exponential distribution . . . 545
4.8.10 Feller-Pareto distribution . . . 548
4.8.11 Fisher z distribution . . . 551
4.8.12 Skew t-distribution (Jones) . . . 555
4.8.13 Pearson's rho distribution (under H_0) . . . 558
4.9 Noncentral distributions . . . 561

4.9.1 Boost: Noncentral χ^2 distribution . . . 561
4.9.2 Boost: Noncentral Student t distribution . . . 566
4.9.3 Boost: Noncentral Fisher F distribution . . . 570
4.9.4 Boost: Noncentral Beta Type I distribution . . . 575
4.9.5 Noncentral Chi distribution . . . 580
4.9.6 Rice (Nakagami-n) distribution . . . 583
4.9.7 Noncentral distribution of the sample correlation coefficient . . . 586
4.9.8 Distribution of the logarithm of a noncentral Beta Type II variable . . . 590
4.9.9 Noncentral distribution (Type I) of Fisher's R^2 . . . 594
4.9.10 Distribution of the logarithm of a noncentral Fisher $1 - R^2$ variable . . . 599
4.9.11 Doubly non-central Student t distribution . . . 603
4.9.12 Doubly non-central Fisher F distribution . . . 606
4.10 Distributions related to multiple comparisons of means . . . 611
4.10.1 Overview and literature . . . 611
4.10.2 Normal maximum distribution, $\rho_{ij,i\neq j} = \rho$ . . . 612
4.10.3 Normal maximum modulus distribution, $\rho_{ij,i\neq j} = \rho$ . . . 615
4.10.4 Normal maximum distribution, $\rho_{ij,i\neq j} = \lambda_i\lambda_j$ . . . 618
4.10.5 Normal maximum modulus distribution, $\rho_{ij,i\neq j} = \lambda_i\lambda_j$ . . . 621
4.10.6 Normal range distribution . . . 624
4.10.7 Studentized maximum distribution . . . 627
4.10.8 Studentized maximum modulus distribution . . . 630
4.10.9 Distribution of Dunnett's t, one-sided . . . 633
4.10.10 Distribution of Dunnett's t, two-sided . . . 636
4.10.11 Nair's t-distribution . . . 639
4.10.12 Halperin's t-distribution . . . 642
4.10.13 Nelson's h-distribution . . . 645
4.10.14 Studentized range distribution . . . 648
4.11 Distributions related to multivariate statistical analysis . . . 651
4.11.1 Distribution of the sum of the negative logarithms of independent beta variables . . . 651
4.11.2 Distribution of the product of independent beta variables . . . 655
4.11.3 Distribution of Wilks' Λ . . . 660
4.11.4 Distribution of Wilks' L_{vc} . . . 664
4.11.5 Distribution of Wilks' L_{vcm} . . . 668
4.11.6 Distribution of Wilks' test of independence of p variates . . . 672
4.11.7 Distribution of Wilks' test of independence of k groups of variates . . . 676
4.11.8 Distribution of Mauchly's test of sphericity vs general structure . . . 680
4.11.9 Distribution of Box's test of equality of covariance matrices, equal sample sizes . . . 684
4.11.10 Distribution of Box's test of equality of k covariance matrices, unequal sample sizes . . . 688
4.11.11 Distribution of Box's test for same multivariate normal distributions, unequal sample sizes . . . 693
4.11.12 Distribution of the modified likelihood ratio test (LRT) for a given covariance matrix . . . 697
4.11.13 Distribution of the modified likelihood ratio test (LRT) for a given covariance matrix and mean vector . . . 702
4.11.14 Central distribution of Roy's largest root . . . 706
4.11.15 Central distribution of Pillai's V . . . 709
4.11.16 Central distribution of Hotelling's T^2 . . . 712
4.11.17 Noncentral Distribution of Wilks' Λ: MANOVA . . . 715
4.11.18 Noncentral Distribution of Wilks' Λ: Canonical Correlation . . . 719
4.12 Miscellaneous continuous distributions . . . 723
4.12.1 Boost: Kolmogorov-Smirnov distribution (limiting form) . . . 723
4.12.2 Boost: Landau Distribution . . . 728
4.12.3 Boost: Holtsmark distribution . . . 731
4.12.4 Boost: Map-Airy distribution . . . 736
4.12.5 Boost: Saspoint5 distribution . . . 741
4.13 Elementary discrete (lattice) distributions . . . 746
4.13.1 Boost: Bernoulli distribution . . . 746

4.13.2 Boost: Geometric distribution . . . 750
4.13.3 Boost: Poisson distribution . . . 755
4.13.4 Boost: Binomial distribution . . . 760
4.13.5 Boost: Negative binomial distribution . . . 765
4.13.6 Boost: Classical hypergeometric distribution . . . 770
4.14 Discrete (lattice) distributions related to (stratified) rank tests . . . 775
4.14.1 Wilcoxon signed rank T distribution, continuous data . . . 775
4.14.2 Noncentral Wilcoxon signed rank T distribution, Bennett alternatives . . . 780
4.14.3 Mann-Whitney U distribution, continuous data . . . 783
4.14.4 Noncentral Mann-Whitney U distribution, Lehmann alternatives . . . 788
4.14.5 Noncentral Mann-Whitney U distribution, Milton alternatives . . . 791
4.14.6 Kendall's tau distribution, continuous data . . . 794
4.14.7 Jonckheere-Terpsta T distribution, continuous data . . . 799
4.14.8 Generalized Page L distribution, continuous data . . . 804
4.14.9 Noncentral generalized Page L distribution, Milton alternatives . . . 807
4.15 Discrete (non-lattice) distributions related to rank tests . . . 810
4.15.1 Cochran-Friedman-Quade distribution . . . 810
4.15.2 Kruskal-Wallis distribution . . . 813

5 Numerical calculus **816**
5.1 Introduction . . . 816
5.1.1 Delegates, lambda expressions, callback functions: an example . . . 816
5.1.2 Usage in Python . . . 817
5.1.3 Usage in CSharp . . . 817
5.2 DAMath: Numerical Rootfinding and Minimization . . . 819
5.2.1 Quadratic Equation, Real Coefficients . . . 819
5.2.2 Cubic Equation, Real Coefficients . . . 820
5.2.3 Brent's algorithm for finding a local minimum . . . 821
5.2.4 Modified Brent's algorithm for finding a local minimum . . . 824
5.2.5 Rootfinding: Brent/Dekker algorithm . . . 825
5.2.6 Rootfinding: Simplified Brent/Dekker algorithm . . . 826
5.3 Boost/Math: Root Finding and Minimization Algorithms . . . 827
5.3.1 Root Finding Without Derivatives (TOM 748 algorithm) . . . 827
5.3.2 Root Finding With Derivatives: Newton-Raphson . . . 828
5.3.3 Root Finding With Derivatives: Halley . . . 829
5.3.4 Root Finding With Derivatives: Schröder . . . 830
5.3.5 Locating Function Minima using Brent's algorithm . . . 831
5.4 Mpmath: Rootfinding and optimization . . . 832
5.4.1 General root-finding interface . . . 832
5.4.2 Bisection algorithm . . . 832
5.4.3 Secant algorithm . . . 833
5.4.4 Illinois, Pegasus, Anderson algorithms . . . 834
5.4.5 Muller algorithm . . . 835
5.4.6 Ridder algorithm . . . 835
5.4.7 Brent algorithm (to be implemented from Scipy brent) . . . 836
5.4.8 Newton algorithm . . . 836
5.4.9 Newton-Steffenson algorithm (ANewton) . . . 836
5.4.10 Modified Newton algorithm (MNewton) . . . 837
5.4.11 Halley algorithm . . . 838
5.4.12 Roots of a vector function (MDNewton, to be implemented) . . . 838
5.4.13 Levenberg-Marquardt algorithm . . . 839
5.4.14 BFGS algorithm. . . . 839
5.4.15 L-BFGS algorithm. . . . 839
5.4.16 Conjugate gradient. . . . 839

5.5 DAMath: Numerical Quadrature . . . 840
5.5.1 General error codes for Amath integration functions, type 1 . . . 840
5.5.2 General error codes for Amath integration functions, type 2 . . . 840
5.5.3 Global adaptive quadrature by Forsythe, Malcolm, Moler (quanc8) . . . 841
5.5.4 21-point Gauss-Kronrod rule, finite interval (qags) . . . 842
5.5.5 15-point Gauss-Kronrod rule, infinite interval (qagi) . . . 843
5.5.6 Cauchy principal value, finite interval (qawc) . . . 844
5.5.7 Double Exponential (DE) transformation, finite interval (intde) . . . 845
5.5.8 DE transformation, infinite interval, no oscillatory factor (intdei) . . . 846
5.5.9 DE transformation, infinite interval, oscillatory factor (intdeo) . . . 847
5.6 Boost/Math: Numerical integration . . . 848
5.6.1 Trapezoidal Quadrature . . . 848
5.6.2 Gauss-Legendre quadrature . . . 849
5.6.3 Gauss-Kronrod Quadrature . . . 850
5.6.4 Double-exponential quadrature: tanh_sinh . . . 851
5.6.5 Double-exponential quadrature: sinh_sinh . . . 852
5.6.6 Double-exponential quadrature: exp_sinh . . . 853
5.6.7 Fourier Integral, Cosine . . . 854
5.6.8 Fourier Integral, Sine . . . 855
5.6.9 Calculating the pdf from the characteristic function . . . 855
5.6.10 Calculating the cdf from the characteristic function . . . 855
5.7 Mpmath: Numerical integration . . . 858
5.7.1 General quadrature interface . . . 858
5.7.2 Quadrature with subdivision . . . 861
5.7.3 Quadrature of oscillatory functions (Filon) . . . 862
5.7.4 Verified numerical integration (Okayama) . . . 864
5.7.5 Specialized Gauss quadrature rules . . . 866
5.8 Mpmath: Numerical inverse Laplace transform . . . 870
5.8.1 General inverse Laplace transform interface . . . 870
5.8.2 Talbot method . . . 872
5.8.3 Stehfest method . . . 873
5.8.4 de Hoog, Knight, and Stokes method . . . 874
5.8.5 Cohen method . . . 875
5.9 Boost/Odeint: Ordinary differential equations . . . 876
5.9.1 Runge-Kutta 4 method, constant stepper . . . 877
5.9.2 Cash-Karp method, constant stepper . . . 878
5.9.3 Dormand-Prince 5 method, constant stepper . . . 879
5.9.4 Fehlberg 78 method, constant stepper . . . 880
5.9.5 Adams-Bashforth-Moulton, constant stepper . . . 881
5.9.6 Cash-Karp method, adaptive stepper . . . 882
5.9.7 Dormand-Prince 5 method, adaptive stepper . . . 883
5.9.8 Fehlberg 78 method, adaptive stepper . . . 884
5.9.9 Bulirsch-Stoer method, adaptive stepper . . . 885
5.9.10 Dormand-Prince 5 method, dense output stepper . . . 886
5.9.11 Bulirsch-Stoer method, dense output stepper . . . 887
5.10 Mpmath: Numerical differentiation . . . 888
5.10.1 First derivative, using the complex-step derivative approximation . . . 888
5.10.2 Second derivative, using the complex-step derivative approximation . . . 888
5.10.3 Gradient, using the complex-step derivative approximation . . . 888
5.10.4 Jacobi matrix, using the complex-step derivative approximation . . . 888
5.10.5 Nth numerical (partial) derivative, using finite differences or numerical quadrature . . . 889
5.10.6 Function object which evaluates the nth derivative of a given function . . . 891
5.10.7 Forward difference, based on a given sequence . . . 891
5.10.8 Generating a sequence of derivatives . . . 891

5.10.9 Composition of derivatives . . . 892
5.10.10 Composition of exponential of derivatives . . . 892
5.10.11 Fractional derivatives / differintegration . . . 893
5.10.12 Taylor series . . . 894
5.10.13 Solving an ODE using high-order Taylor series . . . 894
5.11 Mpmath: Asymptotic expansions . . . 897
5.11.1 Edgeworth expansion: general approximation of the pdf, cdf and sf . . . 897
5.11.2 Cornish-Fisher expansion: general approximation of the qtf and isf . . . 898
5.11.3 Sheppard correction, cumulants . . . 899
5.11.4 The Sheppard correction, cumulant generating function . . . 899
5.11.5 Luggannini-Rice expansion: general approximation of the pdf, cdf, and sf . . . 900
5.11.6 Jensen expansion: general approximation of the qtf and isf . . . 901
5.11.7 Box-Davis expansion: general approximation of the pdf, cdf and sf . . . 901
5.11.8 Box-Davis expansion: general approximation of the qtf and isf (for $\omega_1 = 0$) . . . 902
5.12 Mpmath: Function approximation . . . 904
5.12.1 Pade approximation . . . 904
5.12.2 Chebyshev approximation . . . 904
5.12.3 Fourier series . . . 905
5.12.4 Fourier series evaluation . . . 906
5.13 Mpmath: Sums, products, limits and extrapolation . . . 907
5.13.1 Summation of infinite series . . . 907
5.13.2 Summation using the Euler-Maclaurin formula . . . 913
5.13.3 Summation using the Abel-Plana formula . . . 914
5.13.4 Products . . . 914
5.13.5 Limits, general . . . 916
5.13.6 Richardson extrapolation . . . 918
5.13.7 Shanks extrapolation . . . 918
5.13.8 Levin extrapolation . . . 920
5.13.9 Cohan alternating extrapolation . . . 923
5.14 Mpmath: Polynomials . . . 924
5.14.1 Polynomial evaluation . . . 925
5.14.2 Polynomial roots . . . 925
5.15 Eigen/MinPack: non linear optimization . . . 927
5.15.1 Multidimensional Rootfinding: Powell Hybrid . . . 927
5.15.2 Nonlinear LeastSquares: Levenberg-Marquardt . . . 929
5.16 Eigen/CppOptLib: multidimensional optimization . . . 930
5.16.1 Nelder-Mead Solver . . . 932
5.16.2 CMAes solver . . . 934
5.16.3 BFGS Solver . . . 936
5.16.4 L-BFGS Solver . . . 938
5.16.5 Gradient descent solver . . . 940
5.16.6 Conjugate gradient descent solver . . . 942
5.16.7 Newton descent solver (description needs correction) . . . 944

6 Eigen: Dense and Sparse Matrices **946**
6.1 Creating scalars and matrices . . . 946
6.1.1 Creating a matrix, and converting from compatible data types . . . 946
6.1.2 Creating a matrix of zeros . . . 946
6.1.3 Creating a matrix of ones . . . 947
6.1.4 Creating an identity matrix . . . 947
6.1.5 Creating a general random matrix . . . 947
6.1.6 Creating a symmetric random matrix . . . 947
6.1.7 Creating a self-adjoint random matrix . . . 948
6.1.8 Creating a positive definite self-adjoint matrix . . . 948

6.1.9 Creating a matrix with linearly increasing values . . . 948
6.2 Read-only properties: information about a matrix . . . 949
6.2.1 Rows of a matrix . . . 949
6.2.2 Columns of a matrix . . . 949
6.2.3 Size of a matrix . . . 949
6.3 Accessing and setting parts of a matrix . . . 950
6.3.1 Getting and setting a matrix coefficient . . . 950
6.3.2 Getting and setting a block . . . 950
6.3.3 Getting and setting a matrix row . . . 951
6.3.4 Getting and setting a matrix column . . . 951
6.3.5 Getting and setting a diagonal . . . 952
6.3.6 Getting and setting a triangular view . . . 953
6.4 Changing the shape of a matrix and/or the order of coefficients . . . 954
6.4.1 Sorting a whole matrix . . . 954
6.4.2 Sorting the rows of a matrix by a given column . . . 954
6.4.3 Resize a matrix . . . 955
6.4.4 Resize a matrix like another matrix . . . 955
6.4.5 Conservatively resize a matrix . . . 956
6.4.6 Convert vector to diagonal matrix . . . 956
6.4.7 Adjoint of a matrix . . . 957
6.4.8 Conjugate of a matrix . . . 957
6.4.9 Transpose of a matrix . . . 958
6.4.10 Full Reverse of a matrix . . . 959
6.4.11 Row-wise Reverse of a matrix . . . 959
6.4.12 Column-wise Reverse of a matrix . . . 959
6.4.13 Full Replication of a matrix . . . 960
6.4.14 Row-wise Replication of a matrix . . . 960
6.4.15 Column-wise Replication of a matrix . . . 961
6.4.16 Horizontal concatenation of two matrices . . . 961
6.4.17 Vertical concatenation of two matrices . . . 962
6.5 Basic arithmetic operations . . . 963
6.5.1 Matrix deep copy (unary plus) . . . 963
6.5.2 Matrix negation (unary minus) . . . 963
6.5.3 General matrix addition . . . 964
6.5.4 Matrix addition of a vector as diagonal matrix . . . 964
6.5.5 Matrix: addition of a scalar . . . 965
6.5.6 General matrix subtraction . . . 965
6.5.7 Matrix: subtraction of a scalar . . . 966
6.5.8 General matrix multiplication (“gemm”) . . . 966
6.5.9 Arithmetic comparisons with a scalar or a matrix . . . 968
6.6 Descriptive Statistics . . . 970
6.6.1 Partial Sum . . . 970
6.6.2 Partial Product . . . 970
6.6.3 Partial Arithmetic Mean . . . 970
6.6.4 Partial Minimal Coefficient . . . 970
6.6.5 Partial Maximal Coefficient . . . 970
6.6.6 Trace . . . 970
6.6.7 Partial Squared Norm . . . 971
6.6.8 Partial Norm . . . 971
6.6.9 Partial Stable Norm . . . 971
6.6.10 Covariance matrix . . . 971
6.6.11 Correlation matrix . . . 971
6.7 Standard decompositions and linear solving . . . 972
6.7.1 Cholesky Decomposition with Pivoting . . . 972

6.7.2 LU Decomposition with partial Pivoting . . . 978
6.7.3 LU Decomposition with full Pivoting . . . 983
6.7.4 Cholesky Decomposition without Pivoting . . . 988
6.7.5 QR Decomposition without Pivoting . . . 992
6.7.6 QR Decomposition with column Pivoting . . . 996
6.7.7 QR Decomposition with full Pivoting . . . 1002
6.7.8 Complete orthogonal decomposition (COD) . . . 1008
6.8 Singular Value and Eigen (selfadjoint) decompositions . . . 1015
6.8.1 Singular Value Decomposition, only singular values . . . 1015
6.8.2 Singular Value Decomposition, singular values and thin singular vectors . . . 1018
6.8.3 Singular Value Decomposition, singular values and full singular vectors . . . 1023
6.8.4 Symmetric/Hermitian Eigensystem, only eigen values . . . 1029
6.8.5 Symmetric/Hermitian Eigensystem, eigenvalues and eigenvectors . . . 1032
6.8.6 Generalized Selfadjoint Eigensystem, only eigenvalues . . . 1037
6.8.7 Generalized Selfadjoint Eigensystem: eigenvalues and eigenvectors . . . 1040
6.8.8 Tridiagonalization . . . 1045
6.9 Eigen decompositions of general square matrices . . . 1049
6.9.1 Matrix balancing and eigenvalue/eigenvector computation . . . 1049
6.9.2 Hessenberg Decomposition . . . 1050
6.9.3 Schur Decomposition . . . 1053
6.9.4 Eigensystem of a general square matrix: only eigenvalues . . . 1055
6.9.5 Eigensystem of a general square matrix: eigenvalues and eigenvectors . . . 1058
6.9.6 Real QZ Decomposition . . . 1062
6.9.7 PseudoEigenSystem . . . 1064
6.9.8 Real Generalized Nonsymmetric Eigenvalues . . . 1066
6.9.9 Real Generalized Nonsymmetric Eigensystem . . . 1068
6.10 Eigen: Polynomials . . . 1070
6.10.1 Building a polynomial from its roots . . . 1070
6.10.2 Polynomial evaluation . . . 1070
6.10.3 Polynomial roots . . . 1070
6.11 Eigen: Fast Fourier Transform . . . 1071
6.11.1 Forward transform . . . 1071
6.11.2 Inverse transform . . . 1071
6.12 Eigen and Numpy (Multiprecision): Functions of matrix argument . . . 1074
6.12.1 Matrix Exponential . . . 1074
6.12.2 Matrix Sine . . . 1075
6.12.3 Matrix Cosine . . . 1076
6.12.4 Matrix Hyperbolic Sine . . . 1077
6.12.5 Matrix Hyperbolic Cosine . . . 1078
6.12.6 Matrix Square Root . . . 1080
6.12.7 Matrix Logarithm . . . 1082
6.12.8 Matrix power . . . 1084

7 Numpy: use with multiprecision data types 1085
7.1 Numpy array creation from shape or value . . . 1085
7.1.1 Conversion from integer, float or complex array . . . 1085
7.1.2 Ones on the diagonal and zeros elsewhere: numpy.eye . . . 1088
7.1.3 Identity array: numpy.identity . . . 1088
7.1.4 Array of ones: numpy.ones . . . 1089
7.1.5 Array of ones: numpy.ones_like . . . 1089
7.1.6 Array of zeros: numpy.zeros . . . 1090
7.1.7 Array of zeros: numpy.zeros_like . . . 1090
7.1.8 Array of constant: numpy.full . . . 1091
7.1.9 Array of constant: numpy.full_like . . . 1092

7.1.10 Array with ones at and below the given diagonal and zeros elsewhere: numpy.tri . . . 1093
7.2 Numpy array creation from existing data . . . 1094
7.2.1 Array from object: numpy.array . . . 1094
7.2.2 Array from object: numpy.asarray . . . 1095
7.2.3 Array from object: numpy.asanyarray . . . 1096
7.2.4 Array from data in a text or binary file: numpy.fromfile . . . 1097
7.2.5 Array from a function: numpy.fromfunction . . . 1097
7.2.6 Array from an iterable object: numpy.fromiter . . . 1098
7.2.7 Array from text data in a string: numpy.fromstring . . . 1099
7.2.8 Array from text data in a formatted file: numpy.loadtxt . . . 1100
7.2.9 Array from text data in a text file (can handle missing values): numpy.genfromtxt . . . 1102
7.3 Building special arrays for numerical work . . . 1104
7.3.1 Evenly spaced values within a given interval: numpy.arange . . . 1104
7.3.2 Evenly spaced values within a given interval: numpy.linspace . . . 1105
7.3.3 Evenly spaced values on a log scale: numpy.logspace . . . 1106
7.3.4 Evenly spaced values on a log scale (a geometric progression): numpy.geomspace . . . 1107
7.3.5 Extract a diagonal or construct a diagonal array: numpy.diag . . . 1108
7.3.6 Extract specified diagonals: numpy.diagonal . . . 1110
7.3.7 Two-dimensional array with the flattened input as a diagonal: numpy.diagflat . . . 1111
7.3.8 Lower triangle of an array: numpy.tril . . . 1111
7.3.9 Upper triangle of an array: numpy.triu . . . 1112
7.3.10 Vandermonde matrix.: numpy.vander . . . 1112
7.4 Numpy indexing . . . 1114
7.4.1 Single element indexing . . . 1114
7.4.2 Slicing and striding . . . 1114
7.4.3 Dimensional indexing tools . . . 1116
7.4.4 Broadcasting Overview . . . 1116
7.4.5 General Broadcasting Rules . . . 1117
7.4.6 Broadcastable arrays . . . 1117
7.4.7 Assigning values to indexed arrays . . . 1119
7.5 Numpy basic array manipulation routines . . . 1120
7.5.1 Copy values from one array to another: numpy.copyto . . . 1120
7.5.2 Get the shape of an array: numpy.shape . . . 1120
7.5.3 Reshape an array without changing its data: numpy.reshape . . . 1120
7.5.4 Get a contiguous flattened array: numpy.ravel . . . 1121
7.5.5 1-D iterator over the array: ndarry.flat . . . 1122
7.5.6 Get a copy of the array collapsed into one dimension: ndarry.flatten . . . 1123
7.6 Numpy array manipulation: Transpose-like operations . . . 1124
7.6.1 Move axes of an array to new positions: numpy.flatten . . . 1124
7.6.2 Interchange two axes of an array: numpy.swapaxes . . . 1124
7.6.3 Get an array with axes transposed: numpy.transpose . . . 1124
7.7 Numpy array manipulation: Changing number of dimensions . . . 1126
7.7.1 Convert inputs to arrays with at least one dimension: numpy.atleast_1d . . . 1126
7.7.2 Convert inputs to arrays with at least two dimensions: numpy.atleast_2d . . . 1126
7.7.3 Convert inputs to arrays with at least two dimensions: numpy.atleast_3d . . . 1126
7.7.4 Broadcast an array to a new shape: numpy.broadcast_to . . . 1127
7.7.5 Broadcast any number of arrays against each other: numpy.broadcast_arrays . . . 1127
7.7.6 Expand the shape of an array: numpy.expand_dims . . . 1127
7.7.7 Remove axes of length one from an array: numpy.squeeze . . . 1128
7.8 Numpy array manipulation: Joining arrays . . . 1130
7.8.1 Join a sequence of arrays along an existing axis: numpy.concatenate . . . 1130
7.8.2 Join a sequence of arrays along an existing axis: numpy.stack . . . 1130
7.8.3 Assemble an nd-array from nested lists of blocks: numpy.block . . . 1131
7.8.4 Stack arrays in sequence vertically (row wise): numpy.vstack . . . 1132

7.8.5 Stack arrays in sequence horizontally (column wise): numpy.hstack . . . 1133
7.8.6 Stack arrays in sequence depth wise (along third axis): numpy.dstack . . . 1133
7.8.7 Stack 1-D arrays as columns into a 2-D array: numpy.column_stack . . . 1134
7.9 Numpy array manipulation: Splitting and tiling arrays . . . 1135
7.9.1 Split an array into multiple sub-arrays as views: numpy.split . . . 1135
7.9.2 Split an array into multiple sub-arrays: numpy.array_split . . . 1135
7.9.3 Split array into multiple sub-arrays along the 3rd axis (depth): numpy.dsplit . . . 1135
7.9.4 Split an array into multiple sub-arrays horizontally (column-wise): numpy.hsplit . . . 1136
7.9.5 Split an array into multiple sub-arrays vertically (row-wise): numpy.vsplit . . . 1137
7.9.6 Construct an array by repeating a given array: numpy.tile . . . 1137
7.9.7 Repeat each element of an array after themselves: numpy.repeat . . . 1138
7.10 Numpy array manipulation: Adding and removing elements . . . 1139
7.10.1 Pad an array: numpy.pad . . . 1139
7.10.2 Return a new array with sub-arrays along an axis deleted: numpy.delete . . . 1140
7.10.3 Insert values along the given axis before the given indices: numpy.insert . . . 1140
7.10.4 Append values to the end of an array: numpy.append . . . 1141
7.10.5 Trim zeros from a 1-D array: numpy.trim_zeros . . . 1142
7.10.6 Find the unique elements of an array: numpy.unique . . . 1142
7.11 Numpy array manipulation: Rearranging elements . . . 1144
7.11.1 Reverse the order of elements in an array along the given axis: numpy.flip . . . 1144
7.11.2 Reverse the order of elements along axis 1 (left/right): numpy.fliplr . . . 1144
7.11.3 Reverse the order of elements along axis 0 (up/down): numpy.flipud . . . 1145
7.11.4 Roll array elements along a given axis: numpy.roll . . . 1145
7.11.5 Rotate an array by 90 degrees: numpy.rot90 . . . 1146
7.12 Numpy array manipulation: Sorting . . . 1147
7.12.1 Sorted copy of an array: numpy.sort . . . 1147
7.12.2 Indirect stable sort using a sequence of keys: numpy.lexsort . . . 1148
7.12.3 Indices that would sort an array: numpy.argsort . . . 1149
7.12.4 Matching 1d index and data slices: numpy.take_along_axis . . . 1150
7.12.5 Sorting a complex array: numpy.sort_complex . . . 1150
7.13 Numpy array manipulation: Searching . . . 1151
7.13.1 Indices of the maximum values along an axis: numpy.argmax . . . 1151
7.13.2 Get indices of the minimum values along an axis: numpy.argmin . . . 1151
7.13.3 Find the indices of array elements that are non-zero, grouped by element: numpy.argwhere . . . 1152
7.13.4 Get the indices of array elements that are non-zero: numpy.nonzero . . . 1153
7.13.5 Return elements depending on a condition: numpy.where . . . 1153
7.13.6 Return an array drawn from elements in choicelist, depending on a condition: numpy.select . . . 1154
7.14 Numpy mathematical functions: Sums, products, differences . . . 1155
7.14.1 Product of array elements: numpy.prod . . . 1155
7.14.2 Sum of array elements: numpy.sum . . . 1155
7.14.3 Cumulative product of array elements: numpy.cumprod . . . 1156
7.14.4 Cumulative sum of array elements: numpy.cumsum . . . 1156
7.14.5 N-th discrete difference: numpy.diff . . . 1157
7.14.6 Differences between consecutive elements: numpy.ediff1d . . . 1158
7.14.7 Gradient of an N-dimensional array: numpy.gradient . . . 1158
7.14.8 Trapezoidal rule: numpy.trapezoid . . . 1159
7.15 Numpy mathematical functions: Extrema Finding . . . 1161
7.15.1 Element-wise maximum of array elements: numpy.maximum . . . 1161
7.15.2 Element-wise minimum of array elements: numpy.minimum . . . 1161
7.15.3 Maximum of an array or maximum along an axis: numpy.max . . . 1161
7.15.4 Minimum of an array or maximum along an axis: numpy.min . . . 1162
7.16 Numpy mathematical functions: Arithmetic operations, elementwise . . . 1164
7.16.1 Numerical positive, element-wise: numpy.positive . . . 1164
7.16.2 Numerical negative, element-wise: numpy.negative . . . 1164

7.16.3 Add arguments element-wise: numpy.add . . . 1164
7.16.4 Subtract arguments element-wise: numpy.subtract . . . 1165
7.16.5 Multiply arguments element-wise: numpy.multiply . . . 1165
7.16.6 Divide arguments element-wise: numpy.divide . . . 1165
7.16.7 Floor-divide arguments element-wise: numpy.floor_divide . . . 1166
7.16.8 Element-wise remainder of division: numpy.remainder . . . 1166
7.16.9 Element-wise square: numpy.square . . . 1167
7.16.10 Element-wise reciprocal: numpy.reciprocal . . . 1167
7.17 Numpy mathematical functions: Averages and variances . . . 1168
7.17.1 Median of the array elements.: numpy.median . . . 1168
7.17.2 Average of the array elements.: numpy.average . . . 1168
7.17.3 Arithmetic mean: numpy.mean . . . 1169
7.17.4 Variance: numpy.var . . . 1170
7.18 Numpy mathematical functions: Matrix and vector products . . . 1171
7.18.1 Dot product: numpy.dot . . . 1171
7.18.2 Dot product of two vectors: numpy.vdot . . . 1171
7.18.3 Inner product of two arrays: numpy.inner . . . 1172
7.18.4 Outer product of two vectors: numpy.outer . . . 1173
7.18.5 Matrix product of two arrays: numpy.matmul . . . 1174
7.18.6 Tensor dot product: numpy.matmul . . . 1175
7.18.7 Einstein summation: numpy.einsum . . . 1176
7.18.8 Kronecker product: numpy.kron . . . 1177
7.18.9 Discrete, linear convolution: numpy.convolve . . . 1178
7.19 Numpy logical functions: Truth value testing . . . 1180
7.19.1 Test whether all array elements evaluate to True.: numpy.all, ctx.all . . . 1180
7.19.2 Test whether any array element evaluates to True: numpy.any, ctx.any . . . 1180
7.19.3 Logical AND: numpy.logical_and, ctx.logical_and . . . 1181
7.19.4 Logical OR: numpy.logical_or, ctx.logical_or . . . 1181
7.19.5 Logical XOR: numpy.logical_xor, ctx.logical_xor . . . 1182
7.19.6 Testing array equality: numpy.array_equal, ctx.array_equal . . . 1182
7.19.7 Testing array array_equivalence: numpy.array_equiv, ctx.array_equiv . . . 1183
7.19.8 Truth value of (x1 > x2) element-wise: numpy.greater, ctx.greater . . . 1183
7.19.9 Truth value of (x1 >= x2) element-wise: numpy.greater_equal, ctx.greater_equal . . . 1183
7.19.10 Truth value of (x1 < x2) element-wise: numpy.less, ctx.less . . . 1184
7.19.11 Truth value of (x1 <= x2) element-wise: numpy.less_equal, ctx.less_equal . . . 1184
7.19.12 Truth value of (x1 == x2) element-wise: numpy.equal, ctx.equal . . . 1184
7.19.13 Truth value of (x1 != x2) element-wise: numpy.equal, ctx.equal . . . 1185
7.20 Numpy mathematical functions: Integer and fractional . . . 1186
7.20.1 Floor of the input, element-wise.: numpy.floor, ctx.floor . . . 1186
7.20.2 Ceiling of the input, element-wise.: numpy.ceil, ctx.ceil . . . 1186
7.20.3 Truncated value of the input, element-wise.: numpy.trunc, ctx.trunc . . . 1187
7.20.4 Fix value of the input, element-wise.: numpy.fix, ctx.fix . . . 1187
7.21 Numpy mathematical functions: Miscellaneous . . . 1188
7.21.1 Random Generator . . . 1188
7.21.2 Vectorization of a function: numpy.vectorize . . . 1188
7.22 Summary and examples: Numpy utility functions . . . 1190
7.22.1 Exact mathematical constants . . . 1190
7.22.2 Complex components . . . 1191
7.22.3 Array contents . . . 1195
7.22.4 Array type testing . . . 1200
7.22.5 Using tolerances . . . 1203
7.22.6 Clipping (limiting) the values in an array. . . . 1205
7.23 Arithmetic operations with scalars and iterables . . . 1207
7.23.1 fadd: Addition using a custom precision and rounding mode . . . 1207

7.23.2 fsub: Subtraction using a custom precision and rounding mode . . . 1209
7.23.3 fneg: Negation of a number using a custom precision and rounding mode . . . 1209
7.23.4 fmul: Multiplication using a custom precision and rounding mode . . . 1210
7.23.5 fdiv: Division using a custom precision and rounding mode . . . 1211
7.23.6 fsum: Sum of a finite number of terms . . . 1212
7.23.7 fprod: Product of a finite number of factors . . . 1212
7.23.8 fdot: Dot product . . . 1212
7.24 Numerical transformations and descriptive statistics . . . 1214
7.24.1 Centering of a matrix . . . 1214
7.24.2 Standardization of a matrix . . . 1214
7.24.3 Trace . . . 1215
7.24.4 Squared Norm . . . 1215
7.24.5 Euclidian Norm . . . 1216
7.24.6 Covariance matrix . . . 1216
7.24.7 Correlation matrix . . . 1217
7.24.8 Summary statistic . . . 1217
7.24.9 Vector norm of a matrix . . . 1218
7.24.10 Matrix norm . . . 1219
7.25 Standard decompositions and linear solving . . . 1220
7.25.1 Cholesky decomposition . . . 1220
7.25.2 Cholesky decomposition, solve . . . 1221
7.25.3 Matrix LU factorization . . . 1221
7.25.4 Determinant of a matrix, using LU decomposition . . . 1221
7.25.5 Inverse of a matrix, using the LU factorization . . . 1221
7.25.6 Linear equations: LU solve . . . 1222
7.25.7 Linear equations: residual of LU solve . . . 1222
7.25.8 ??? LU improve solution . . . 1222
7.25.9 mpmath: LU condition number . . . 1222
7.25.10 QR factorization . . . 1223
7.25.11 QR solve . . . 1224
7.26 Singular Value and Eigen decompositions . . . 1225
7.26.1 Real singular value decomposition of a matrix A . . . 1225
7.26.2 Complex singular value decomposition of a matrix A . . . 1226
7.26.3 mpmath: Singular value decomposition of a matrix A (real or complex) . . . 1227
7.26.4 Eigenvalue problem for a real symmetric square matrix A . . . 1228
7.26.5 Eigenvalue problem for a complex hermitian square matrix A . . . 1229
7.26.6 mpmath: Eigenvalue problem for a selfadjoint square matrix A . . . 1230
7.26.7 mpmath: tridiag_sym . . . 1231
7.26.8 mpmath tridiag_her . . . 1232
7.26.9 mpmath: tridiag_eigen_sym . . . 1232
7.26.10 Eigensystem decomposition of a matrix A (real or complex) . . . 1233
7.26.11 Sorting Eigenvalues . . . 1234
7.26.12 mpmath: Hessenberg decomposition of a matrix A (real or complex) . . . 1234
7.26.13 Schur decomposition of a matrix A (real or complex) . . . 1235

II Special Functions 1237

8 Elliptic functions and related 1238
8.1 Carlson symmetric elliptic integrals . . . 1239
8.1.1 Carlson symmetric elliptic integral of the first kind, $R_F(x, y, z)$ . . . 1239
8.1.2 Carlson completely symmetric elliptic integral of the second kind, $R_G(x, y, z)$ . . . 1239
8.1.3 Carlson symmetric elliptic integral of the third kind, $R_J(x, y, z, p)$ . . . 1240
8.1.4 Carlson symmetric elliptic integral of the second kind, $R_D(x, y, z)$ . . . 1241

8.1.5 Carlson degenerate symmetric elliptic integral of the first kind, $R_C(x, y)$ . . . 1241
8.2 Legendre elliptic integrals (elliptic parameter m) . . . 1243
8.2.1 Legendre complete elliptic integral of the first kind, $K(m)$ . . . 1243
8.2.2 Legendre complete elliptic integral of the second kind, $E(m)$ . . . 1243
8.2.3 Legendre complete elliptic integral of the third kind, $\Pi(n, m)$ . . . 1244
8.2.4 Legendre incomplete elliptic integral of the first kind, $F(\phi, m)$ . . . 1245
8.2.5 Legendre incomplete elliptic integral of the second kind, $E(\phi, m)$ . . . 1246
8.2.6 Legendre incomplete elliptic integral of the third kind, $\Pi(n, \phi, m)$ . . . 1247
8.3 Legendre elliptic integrals (elliptic modulus k), and related functions . . . 1249
8.3.1 Legendre complete elliptic integral of the first kind, $K(k)$ . . . 1249
8.3.2 Legendre complete elliptic integral of the second kind, $E(k)$ . . . 1250
8.3.3 Legendre complete elliptic integral of the third kind, $\Pi(n; k)$ . . . 1251
8.3.4 Legendre incomplete elliptic integral of the first kind, $F(\phi, k)$ . . . 1252
8.3.5 Legendre incomplete elliptic integral of the second kind, $E(\phi, k)$ . . . 1252
8.3.6 Legendre incomplete elliptic integral of the third kind, $\Pi(n, \phi, k)$ . . . 1253
8.4 Jacobi elliptic functions . . . 1255
8.4.1 Jacobi elliptic function $\mathrm{sn}(x, k)$ . . . 1255
8.4.2 Jacobi elliptic function $\mathrm{cn}(x, k)$ . . . 1257
8.4.3 Jacobi elliptic function $\mathrm{dn}(x, k)$ . . . 1259
8.4.4 Jacobi elliptic function $\mathrm{nc}(x, k)$ . . . 1261
8.4.5 Jacobi elliptic function $\mathrm{sc}(x, k)$ . . . 1262
8.4.6 Jacobi elliptic function $\mathrm{dc}(x, k)$ . . . 1263
8.4.7 Jacobi elliptic function $\mathrm{nd}(x, k)$ . . . 1264
8.4.8 Jacobi elliptic function $\mathrm{sd}(x, k)$ . . . 1265
8.4.9 Jacobi elliptic function $\mathrm{cd}(x, k)$ . . . 1266
8.4.10 Jacobi elliptic function $\mathrm{ns}(x, k)$ . . . 1267
8.4.11 Jacobi elliptic function $\mathrm{cs}(x, k)$ . . . 1268
8.4.12 Jacobi elliptic function $\mathrm{ds}(x, k)$ . . . 1269
8.5 Jacobi theta functions and related functions . . . 1270
8.5.1 Jacobi theta function $\theta_1(z, q)$ . . . 1270
8.5.2 Jacobi theta function $\theta_2(z, q)$ . . . 1272
8.5.3 Jacobi theta function $\theta_3(z, q)$ . . . 1274
8.5.4 Jacobi theta function $\theta_4(z, q)$ . . . 1276
8.6 Conversions of parameters of Weierstrass $\wp$ . . . 1278
8.6.1 Elliptic lattice invariants g_2, g_3 from lattice roots e_1, e_2, e_3 . . . 1278
8.6.2 Elliptic lattice invariants g_2, g_3 from (ω, τ) . . . 1280
8.6.3 Elliptic lattice roots e_1, e_2, e_3 from (ω, τ) . . . 1281
8.7 Weierstrass elliptic functions, in terms of elliptic period ratio τ . . . 1283
8.7.1 Weierstrass function $\wp(z|\tau)$ . . . 1283
8.7.2 Weierstrass function, first derivative: $\wp' z|\tau)$ . . . 1283
8.7.3 Inverse Weierstrass function $\wp^{-1}(z|\tau)$ . . . 1284
8.7.4 Weierstrass Zeta function $\zeta(z, \tau)$ . . . 1285
8.7.5 Weierstrass Sigma function $\sigma(z|\tau)$ . . . 1286
8.8 Modular forms, in terms of half-period ω_1 and elliptic period ratio τ . . . 1287
8.8.1 Dedekind eta function $\eta(\tau)$ . . . 1287
8.8.2 Elliptic modular lambda function $\lambda(\tau)$ (also DAMath) . . . 1287
8.8.3 Elliptic modular delta function $\Delta(\omega, \tau)$ . . . 1288
8.8.4 Klein j-invariant $j(\tau)$ (also DAMath) . . . 1289

9 Lerch's phi and related **1290**
9.1 Lerch's transcendent and Lerch's zeta . . . 1291
9.1.1 Lerch's transcendent, $\Phi(x, s, a)$ . . . 1291
9.1.2 Lerch's zeta . . . 1294
9.2 Polygamma and related functions . . . 1295

9.2.1 Polygamma function, $\psi^{(n)}(x)$ . . . 1295
9.2.2 TriGamma function, $\psi'(x)$ . . . 1297
9.2.3 DiGamma function $\psi(x)$ . . . 1298
9.2.4 Harmonic number function, H_x . . . 1300
9.3 Polylogarithm and related functions . . . 1301
9.3.1 Polylogarithm, $\mathrm{Li}_s(z)$ . . . 1301
9.3.2 Trilogarithm Function, $\mathrm{Li}_3(z)$ . . . 1303
9.3.3 Dilogarithm Function, $\mathrm{Li}_2(z)$ . . . 1304
9.3.4 Generalized Clausen sine function . . . 1306
9.3.5 Generalized Clausen cosine function . . . 1308
9.3.6 Classical Clausen function, $\mathrm{Cl}_2(x)$ . . . 1309
9.3.7 Bose-Einstein integrals, $G_s(x)$ . . . 1310
9.3.8 Fermi-Dirac integrals, $F_s(x)$ . . . 1311
9.3.9 Legendre's Chi function, $\chi_s(x)$ . . . 1312
9.3.10 Generalized inverse tangent integral . . . 1314
9.4 Hurwitz zeta and related functions . . . 1315
9.4.1 Hurwitz zeta function, $\zeta(s, a)$ . . . 1315
9.4.2 Generalized harmonic number function, $H_x^{(r)}$ . . . 1317
9.4.3 Bernoulli numbers, B_n . . . 1318
9.4.4 Bernoulli polynomials, $B_n(x)$ . . . 1319
9.4.5 Euler numbers . . . 1320
9.4.6 Euler polynomials, $E_n(x)$ . . . 1321
9.4.7 Barnes G-function . . . 1322
9.4.8 Logarithm of Barnes G function . . . 1323
9.4.9 Hyperfactorial . . . 1324
9.4.10 Superfactorial . . . 1325
9.5 Riemann zeta function, and related functions . . . 1326
9.5.1 Riemann zeta function, $\zeta(s)$ . . . 1326
9.5.2 Riemann $\zeta(s) - 1$ . . . 1328
9.5.3 Hardy (or Riemann-Siegel) theta function . . . 1329
9.5.4 Hardy (or Riemann-Siegel) Z function . . . 1330
9.5.5 Riemann (Landau) function $\xi(s)$ . . . 1332
9.5.6 Dirichlet eta function, $\eta(s)$ . . . 1333
9.5.7 Dirichlet $\eta(s) - 1$ . . . 1334
9.5.8 Dirichlet beta function, $\beta(s)$ . . . 1335
9.5.9 Dirichlet lambda function, $\lambda(s)$ . . . 1336
9.5.10 Zeros of the Riemann zeta function . . . 1337

10 Hypergeometric function ${}_0F_1$ and related **1338**
10.1 Hypergeometric Limit Function ${}_0F_1$ . . . 1338
10.1.1 Confluent Hypergeometric Limit Function ${}_0F_1(b, x)$ . . . 1338
10.1.2 Regularized Confluent Hypergeometric Limit Function ${}_0\widetilde{F}_1(b; x)$ . . . 1340
10.2 Bessel functions . . . 1341
10.2.1 Bessel function of the 1st kind, $J_\nu(x)$ . . . 1341
10.2.2 Bessel function of the 2nd kind, $Y_\nu(x)$ . . . 1343
10.2.3 First derivative of the Bessel function of the first kind, $J'_\nu(x)$ . . . 1345
10.2.4 First derivative of the Bessel function of the second kind, $Y'_\nu(x)$ . . . 1346
10.2.5 Boost: Zeros $j_{\nu,m}$ of the Bessel function of the first kind: $J_\nu(j_{\nu,m}) = 0$ . . . 1347
10.2.6 Boost: Zeros $y_{\nu,m}$ of the Bessel function of the first kind: $Y_\nu(y_{\nu,m}) = 0$ . . . 1348
10.3 Modified Bessel functions . . . 1349
10.3.1 Modified Bessel function of the 1st kind, $I_\nu(x)$ . . . 1349
10.3.2 Modified Bessel function of the 2nd kind, $K_\nu(x)$ . . . 1351
10.3.3 First derivative of the modified Bessel function of the first kind $I'_\nu(x)$ . . . 1353
10.3.4 First derivative of the modified Bessel function of the second kind $K'_\nu(x)$ . . . 1354

10.3.5 Marcum Q function, $Q_m(a,b)$ (up to octuple precision) . . . 1355
10.4 Spherical Bessel functions . . . 1356
10.4.1 Spherical Bessel function of the first kind, $j_n(x)$ . . . 1356
10.4.2 Spherical Bessel function of the second kind, $y_n(x)$ . . . 1357
10.4.3 First derivative of the spherical Bessel function of the first kind, $j_n'(x)$ . . . 1358
10.4.4 First derivative of the spherical Bessel function of the second kind, $y_n'(x)$ . . . 1359
10.4.5 Boost: Zeros $a_{n,m}$ of the spherical Bessel function of the first kind: $j_n(a_{n,m}) = 0$ . . . 1360
10.4.6 Boost: Zeros $b_{n,m}$ of the spherical Bessel function of the second kind: $y_n(b_{n,m}) = 0$ . . . 1361
10.5 Modified spherical Bessel functions . . . 1362
10.5.1 Modified spherical Bessel function of the first kind, $i_n(x)$ . . . 1362
10.5.2 Modified Spherical Bessel function of the second kind, $k_n(x)$ . . . 1364
10.5.3 First derivative of the modified spherical Bessel function of the first kind, $i_n'(x)$ . . . 1365
10.5.4 First derivative of the modified spherical Bessel function of the second kind, $k_n'(x)$ . . . 1366
10.5.5 Bessel polynomials $y_n(x)$ . . . 1367
10.5.6 Reverse Bessel polynomials $\theta_n(x)$ . . . 1368
10.6 Hankel functions . . . 1369
10.6.1 Hankel function of the first kind $H_\nu^{(1)}(x)$ . . . 1369
10.6.2 Hankel function of the second kind $H_\nu^{(2)}(x)$ . . . 1370
10.6.3 Spherical Hankel function of the first kind, $h_\nu^{(1)}(x)$ . . . 1371
10.6.4 Spherical Hankel function of the second kind, $h_\nu^{(2)}(x)$ . . . 1372
10.7 Airy functions . . . 1373
10.7.1 Airy function $\mathrm{Ai}(x)$ . . . 1373
10.7.2 Airy function $\mathrm{Bi}(x)$ . . . 1375
10.7.3 First derivative of the Airy function Ai, $\mathrm{Ai}'(x)$ . . . 1377
10.7.4 First derivative of the Airy function Bi, $\mathrm{Bi}'(x)$ . . . 1378
10.7.5 Real zeros a_k of the Airy function Ai, $\mathrm{Ai}(a_k) = 0$ . . . 1379
10.7.6 Real zeros b_k of the Airy function Bi, $\mathrm{Bi}(b_k) = 0$ . . . 1380
10.8 Kelvin functions . . . 1381
10.8.1 Kelvin function $\mathrm{ber}(\nu, x)$ . . . 1381
10.8.2 Kelvin function $\mathrm{bei}(\nu, x)$ . . . 1382
10.8.3 Kelvin function $\mathrm{ker}(\nu, x)$ . . . 1383
10.8.4 Kelvin function $\mathrm{kei}(\nu, x)$ . . . 1384
10.8.5 First derivative of the Kelvin function $\mathrm{ber}(\nu, x)$, $\mathrm{ber}'(\nu, x)$ . . . 1385
10.8.6 First derivative of the Kelvin function $\mathrm{bei}(\nu, x)$, $\mathrm{bei}'(\nu, x)$ . . . 1386
10.8.7 First derivative of the Kelvin function $\mathrm{ker}(\nu, x)$, $\mathrm{ker}'(\nu, x)$ . . . 1387
10.8.8 First derivative of the Kelvin function $\mathrm{kei}(\nu, x)$, $\mathrm{kei}'(\nu, x)$ . . . 1388

11 Hypergeometric function ${}_1F_1$ and related **1389**
11.1 Hypergeometric Functions ${}_1F_1$ (Kummer) and U (Tricomi) . . . 1389
11.1.1 Kummer's Confluent Hypergeometric Function ${}_1F_1(a,b;x)$ . . . 1389
11.1.2 Regularized Kummer Confluent Hypergeometric Function, ${}_1\widetilde{F}_1(a,b;x)$ . . . 1391
11.1.3 Tricomi's Confluent Hypergeometric Function, $U(a,b;x)$ . . . 1392
11.1.4 Generalized Laguerre polynomials, $L_n^{(a)}(x)$ . . . 1394
11.1.5 Hermite polynomial (physicist), $H_n(x)$ . . . 1395
11.1.6 Hermite polynomials (probabilist) $\mathrm{He}_n(x)$ . . . 1396
11.2 Incomplete gamma functions . . . 1397
11.2.1 Lower normalized incomplete gamma function, $P(a,x)$ . . . 1397
11.2.2 Upper normalized incomplete gamma functions , $Q(a,x)$ . . . 1398
11.2.3 Lower non-normalized incomplete gamma function, $\gamma(a,x)$ . . . 1400
11.2.4 Upper non-normalized incomplete gamma function, $\Gamma(a,x)$ . . . 1401
11.2.5 Derivative of the incomplete gamma function . . . 1402
11.2.6 Inverse of the real lower normalised incomplete gamma function, $P^{-1}(a,q)$ . . . 1402
11.2.7 Inverse of the real upper normalised incomplete gamma function, $Q^{-1}(a,q)$ . . . 1403

11.2.8 Inverse (on parameter a) of the real lower normalised incomplete gamma function . . . 1403
11.2.9 Inverse (on parameter a) of the real upper normalised incomplete gamma function . . . 1403
11.3 Coulomb, Whittaker and parabolic cylinder function . . . 1405
11.3.1 Regular Coulomb wave function $F_\ell(\eta, x)$ . . . 1405
11.3.2 Irregular Coulomb wave function $G_\ell(\eta, z)$ . . . 1406
11.3.3 Irregular Coulomb wave function $H_\ell^{(+)}(\eta, x)$ . . . 1407
11.3.4 Irregular Coulomb wave function $H_\ell^{(-)}(\eta, x)$ . . . 1408
11.3.5 Whittaker function $M_{\kappa,\mu}(x)$ . . . 1409
11.3.6 Whittaker function $W_{\kappa,\mu}(x)$ . . . 1410
11.3.7 Parabolic cylinder function $D_\nu(x)$ . . . 1411
11.3.8 Parabolic cylinder function $U(a, x)$ . . . 1412
11.3.9 Parabolic cylinder function $V(a, x)$ . . . 1413
11.3.10 Parabolic cylinder function $W(a, x)$ . . . 1414
11.4 Error function and related functions . . . 1415
11.4.1 Error function, $\mathrm{erf}(x)$ . . . 1415
11.4.2 Complementary error function, $\mathrm{erfc}(x)$ . . . 1417
11.4.3 Inverse of the real error function, $\mathrm{erf}^{-1}(x)$ . . . 1419
11.4.4 Inverse of the real complementory error function, $\mathrm{erfc}^{-1}(x)$ . . . 1420
11.4.5 Standard normal density function $\phi(x)$ . . . 1421
11.4.6 Standard normal cumulative distribution function $\Phi(x)$ . . . 1422
11.4.7 Standard normal quantile function $\Phi^{-1}(q)$ . . . 1422
11.4.8 Imaginary error function, $\mathrm{erfi}(x)$ . . . 1423
11.4.9 Dawson integral, $F(x)$ . . . 1424
11.4.10 Faddeeva function, $w(z)$ . . . 1425
11.4.11 Fresnel sine integral, $S(x)$ . . . 1426
11.4.12 Fresnel cosine integral, $C(x)$ . . . 1428
11.4.13 Owen's T function, $T(h, a)$ . . . 1430
11.5 Exponential integrals, and related functions . . . 1431
11.5.1 Exponential integral $E_1(x)$ . . . 1431
11.5.2 Exponential integral $\mathrm{Ei}(x)$ . . . 1432
11.5.3 Logarithmic integral $\mathrm{li}(x)$ . . . 1434
11.5.4 Hyperbolic sine integral $\mathrm{Shi}(x)$ . . . 1436
11.5.5 Hyperbolic cosine integral $\mathrm{Chi}(x)$ . . . 1437
11.5.6 Generalized exponential integral $E_n(x)$ . . . 1438
11.5.7 Sine integral $\mathrm{Si}(x)$ . . . 1439
11.5.8 Cosine integral $\mathrm{Ci}(x)$ . . . 1441

12 Hypergeometric functions ${}_2F_1$ and ${}_1F_2$ (and related functions) 1443
12.1 Gauss Hypergeometric Function ${}_2F_1$ . . . 1443
12.1.1 Gauss Hypergeometric Function, ${}_2F_1(a, b; c; x)$ . . . 1443
12.1.2 Regularized Hypergeometric Function, ${}_2\widetilde{F}_1(a, b; c; x)$ . . . 1445
12.2 Chebyshev, Gegenbauer and Jacobi polynomials . . . 1446
12.2.1 Chebyshev polynomial (or function) of the first kind, $T_n(x)$ . . . 1446
12.2.2 Chebyshev polynomial of the second kind, $U_n(x)$ . . . 1447
12.2.3 Chebyshev polynomials of the third kind, $V_n(x)$ . . . 1448
12.2.4 Chebyshev polynomials of the fourth kind, $W_n(x)$ . . . 1449
12.2.5 Gegenbauer (ultraspherical) polynomial, $C_n^\alpha(x)$ . . . 1450
12.2.6 Jacobi polynomials, $P_n^{(a,b)}(x)$ . . . 1451
12.2.7 Zernike radial polynomials $R_n^m(r)$ . . . 1453
12.3 Legendre polynomials and related . . . 1454
12.3.1 Legendre polynomial (or function) of the first kind, $P_n(x)$ . . . 1454
12.3.2 Associated Legendre function of the first kind, $P_l^m(x)$ . . . 1455
12.3.3 Legendre function of the second kind, $Q_l(x)$ . . . 1456

12.3.4 Associated Legendre function of the second kind, $Q_l^m(x)$ . . . 1457
12.3.5 Spherical harmonics, $Y_n^m(\theta,\phi)$ . . . 1458
12.3.6 Toroidal harmonics $P^m_{l-1/2}(x)$ . . . 1458
12.3.7 Toroidal harmonics $Q^m_{l-1/2}(x)$ . . . 1459
12.4 Incomplete beta functions . . . 1460
12.4.1 Non-normalized lower incomplete beta function, $B_{\text{lower}}(a,b;x)$ . . . 1460
12.4.2 Normalized incomplete beta function, $I_x(a,b)$ . . . 1461
12.4.3 Derivative of the incomplete beta function . . . 1463
12.4.4 Real non-normalized upper incomplete beta function, $B_{\text{upper}}(a,b;x)$ . . . 1464
12.4.5 Real normalized complementory incomplete beta function, $I_{1-x}(a,b)$ . . . 1465
12.4.6 Inverse of the real normalised incomplete beta function . . . 1467
12.4.7 Inverse of the real normalised complementary incomplete beta function . . . 1468
12.4.8 Inverse (on parameter a) of the real normalised incomplete beta function . . . 1469
12.4.9 Inverse (on parameter a) of the real normalised complementary incomplete beta function . . . 1470
12.4.10 Inverse (on parameter b) of the real normalised incomplete beta function . . . 1471
12.4.11 Inverse (on parameter b) of the real normalised complementary incomplete beta function . . . 1472
12.5 Hypergeometric function ${}_1F_2$ . . . 1473
12.5.1 Non-regularized hypergeometric function ${}_1F_2$ . . . 1473
12.5.2 Regularized hypergeometric function ${}_1\widetilde{F}_2$ . . . 1474
12.6 Scorer functions . . . 1475
12.6.1 Scorer function $\mathrm{Gi}(x)$ . . . 1475
12.6.2 Scorer function $\mathrm{Hi}(x)$ . . . 1476
12.6.3 First derivative of the Scorer function $\mathrm{Gi}(x)$, $\mathrm{Gi}'(x)$ . . . 1477
12.6.4 First derivative of the Scorer function $\mathrm{Hi}(x)$, $\mathrm{Hi}'(x)$ . . . 1478
12.7 Struve functions . . . 1479
12.7.1 Struve function $\mathbf{H}_n(x)$ . . . 1479
12.7.2 Struve function $\mathbf{L}_\nu(x)$ . . . 1480
12.7.3 Struve function $\mathbf{K}_\nu(x)$ . . . 1481
12.7.4 Struve function $\mathbf{M}_\nu(x)$ . . . 1482
12.8 Anger, Weber and Lommel functions . . . 1483
12.8.1 Anger function $\mathbf{J}_\nu(x)$ . . . 1483
12.8.2 Weber function $\mathbf{E}_\nu(x)$ . . . 1484
12.8.3 Lommel function $s_{\mu,\nu}(x) = s^{(1)}_{\mu,\nu}(x)$ . . . 1485
12.8.4 Lommel function $S_{\mu,\nu}(x) = s^{(2)}_{\mu,\nu}(x)$ . . . 1486

III Supporting Functions **1487**

13 Algebra with random variables **1488**
13.1 Probability density function (pdf) . . . 1488
13.1.1 Calculating the pdf from the cdf . . . 1488
13.1.2 Calculating the pdf from the characteristic function . . . 1488
13.2 Probability mass function (pmf) . . . 1489
13.2.1 Calculating the pmf from the cdf . . . 1489
13.2.2 Calculating the pmf from the characteristic function . . . 1489
13.2.3 Calculating the pmf from the factorial moments . . . 1489
13.2.4 Approximating the pmf with asymptotic expansions . . . 1489
13.3 Cumulative distribution function (cdf) . . . 1490
13.3.1 Calculating the cdf from the pdf . . . 1490
13.3.2 Calculating the cdf from the pmf . . . 1490
13.3.3 Calculating the cdf from the characteristic function . . . 1490
13.3.4 Calculating the cdf and sf from the characteristic function (lattice distribution) . . . 1492
13.3.5 Calculating the cdf from the factorial moments (lattice distributions) . . . 1492
13.4 Quantile function . . . 1493

13.4.1 Calculating the quantile function from the cdf . . . 1493
13.4.2 Approximating the quantile function from the cumulant generating function . . . 1493
13.4.3 Approximating the quantile function from the cumulants . . . 1493
13.5 Characteristic function . . . 1494
13.5.1 Calculating the characteristic function from the pdf (continuous distribution) . . . 1494
13.5.2 Calculating the characteristic function from the pmf (lattice distribution) . . . 1494
13.5.3 Calculating the characteristic function from the quantile function . . . 1494
13.5.4 Calculating the characteristic function from the raw moments . . . 1494
13.6 Moment generating function . . . 1495
13.6.1 Calculating the moment-generating function from the pdf . . . 1495
13.6.2 Calculating the moment-generating function from the characteristic function . . . 1495
13.6.3 Calculating the moment-generating function from the cumulant-generating function . . . 1495
13.6.4 Calculating the moment-generating function from the probability-generating function . . . 1495
13.6.5 Calculating the moment-generating function from the raw moments . . . 1495
13.7 Cumulant generating function . . . 1496
13.7.1 Calculating the cumulant-generating function from the characteristic function . . . 1496
13.7.2 Calculating the cumulant-generating function from the moment-generating function . . . 1496
13.7.3 Calculating the cumulant-generating function from the probability-generating function . . . 1496
13.7.4 Calculating the cumulant-generating function from the cumulants . . . 1496
13.8 Probability generating function . . . 1497
13.8.1 Calculating the probability-generating function from the pmf vector . . . 1497
13.8.2 Calculating the probability-generating function from the moment-generating function . . . 1497
13.9 Factorial Moments . . . 1498
13.9.1 Calculating the factorial moments from the raw moments . . . 1498
13.9.2 Calculating the factorial moments from the cumulants . . . 1498
13.10 Raw Moments . . . 1499
13.10.1 Calculating the raw moments from the pdf . . . 1499
13.10.2 Calculating the raw moments from the pmf . . . 1499
13.10.3 Calculating the raw moments from the factorial moments . . . 1499
13.10.4 Calculating the raw moments from the central moments . . . 1499
13.10.5 Calculating the raw moments from the cumulants . . . 1500
13.10.6 Calculating the raw moments from the moment-generating function . . . 1500
13.10.7 Calculating the raw moments from the characteristic function . . . 1500
13.10.8 Calculating the raw moments from the probability-generating function . . . 1500
13.11 Central Moments . . . 1501
13.11.1 Calculating the central moments from the factorial moments . . . 1501
13.11.2 Calculating the central moments from the raw moments . . . 1501
13.11.3 Calculating the central moments from the cumulants . . . 1501
13.12 Cumulants . . . 1502
13.12.1 Calculating the cumulants from the pmf vector . . . 1502
13.12.2 Calculating the cumulants from the factorial moments . . . 1502
13.12.3 Calculating the cumulants from the raw moments . . . 1502
13.12.4 Calculating the cumulants from the central moments . . . 1502
13.12.5 Calculating the cumulants from the cumulant-generating function . . . 1502

14 Series and integrals **1504**
14.1 Finite series algorithms for selected distributions . . . 1504
14.1.1 Central χ^2 distribution, cdf (integer degrees of freedom) . . . 1504
14.1.2 Central Student t distribution, cdf (integer degrees of freedom) . . . 1504
14.1.3 Central Fisher F distribution, cdf (integer degrees of freedom) . . . 1505
14.1.4 Central Beta distribution, cdf ($2a$ an integer, $2b$ an integer) . . . 1506
14.1.5 Noncentral χ^2 distribution, cdf (integer degrees of freedom) . . . 1506
14.1.6 Noncentral Student t distribution, cdf (integer degrees of freedom) . . . 1507
14.1.7 Noncentral Fisher F distribution, cdf (m an even integer) . . . 1507

14.1.8 Noncentral Beta distribution, cdf (b an integer) . . . 1508
14.1.9 Pearson's ρ distribution, pdf (integer N) . . . 1508
14.1.10 Pearson's ρ distribution, cdf (integer N) . . . 1509
14.1.11 Fisher's R^2 distribution, cdf (finite sum for $N - p$ even) . . . 1510
14.1.12 Roy's largest root distribution, pdf, cdf and sf . . . 1511
14.2 Infinite series algorithms for selected functions and distributions . . . 1513
14.2.1 Incomplete gamma function, continued fractions (Peizer) . . . 1513
14.2.2 Incomplete gamma function, asymptotic expansion (Paris) . . . 1514
14.2.3 Incomplete beta function, continued fractions (Peizer) . . . 1514
14.2.4 Noncentral χ^2 distribution, pdf, cdf and sf (Boost) . . . 1515
14.2.5 Noncentral Student t distribution, pdf, cdf and sf (Boost) . . . 1516
14.2.6 Noncentral Beta distribution, pdf, cdf and sf (Boost) . . . 1517
14.2.7 Noncentral F distribution, pdf, cdf and sf (Benton) . . . 1517
14.2.8 Pearson's ρ distribution, cdf and sf (Hotelling's series) . . . 1518
14.2.9 Pearson's ρ distribution, cdf and sf (Guenther's series) . . . 1519
14.2.10 Fisher's R^2 distribution, cdf and sf (Boost, Benton) . . . 1520
14.3 Finite series for lattice distributions, based on factorial moments . . . 1521
14.3.1 Fréchet's formula for calculating the pmf from the factorial moments . . . 1521
14.3.2 Laurent's formula for calculating the cdf from the factorial moments . . . 1521
14.3.3 Binomial distribution, pmf, cdf . . . 1522
14.3.4 Classical hypergeometric distribution, pmf, cdf . . . 1522
14.3.5 Wilcoxon distribution, pmf, cdf . . . 1523
14.3.6 Mann-Whitney distribution, pmf, cdf . . . 1524
14.3.7 Kendall distribution, pmf, cdf . . . 1525
14.3.8 Jonckheere-Terpsta T distribution, pmf, cdf . . . 1525
14.4 Efficient integration of bell-shaped functions . . . 1527
14.4.1 Introduction . . . 1527
14.4.2 Error function . . . 1527
14.4.3 Lower non-normalised incomplete gamma function, $\gamma(a, x)$ (Continued fractions) . . . 1527
14.4.4 Real upper non-normalised incomplete gamma function, $\Gamma(a, x)$ (Quadrature) . . . 1527
14.4.5 Normalised incomplete beta function, $I_x(a, b)$ . . . 1528
14.4.6 Non-central chi-square cdf and sf (Chow) . . . 1528
14.4.7 Marcum Q function . . . 1528
14.4.8 Owen's T function . . . 1529
14.5 Verified numerical integration . . . 1531
14.5.1 Error function . . . 1531
14.5.2 Lower non-normalised incomplete gamma function, $\gamma(a, x)$ (Continued fractions) . . . 1531
14.5.3 Real upper non-normalised incomplete gamma function, $\Gamma(a, x)$ (Quadrature) . . . 1531
14.5.4 Normalised incomplete beta function, $I_x(a, b)$ . . . 1531
14.5.5 Non-central chi-square cdf and sf (Chow) . . . 1532
14.5.6 Marcum Q function . . . 1532
14.5.7 Owen's T function . . . 1533

15 Pmf vectors **1535**
15.1 Basic discrete (lattice) distribution functions . . . 1535
15.1.1 Poisson distribution, pmf vector . . . 1535
15.1.2 Binomial distribution, pmf vector . . . 1535
15.1.3 Negative binomial distribution, pmf vector . . . 1536
15.1.4 Hypergeometric distribution, pmf vector . . . 1536
15.1.5 Noncentral hypergeometric distribution (Fisher), pmf vector . . . 1536
15.2 Discrete (lattice) distribution functions related to (stratified) rank tests . . . 1538
15.2.1 Sign test distribution (under H_0), pmf vector . . . 1538
15.2.2 Wilcoxon T distribution (under H_0), pmf vector . . . 1538
15.2.3 Wilcoxon T distribution (under Bennett alternatives), pmf vector . . . 1538

15.2.4 Kendall S (or tau) distribution (under H_0), pmf vector . . . 1538
15.2.5 Mann-Whitney U distribution (under H_0), pmf vector . . . 1539
15.2.6 Jonckheere-Terpsta S distribution (under H_0), pmf vector . . . 1539
15.2.7 Spearman ρ distribution (under H_0), pmf vector . . . 1539
15.2.8 Page L distribution (under H_0), pmf vector . . . 1540
15.2.9 Quade L distribution (under H_0), pmf vector . . . 1540
15.2.10 Mann-Whitney U distribution (under Lehmann alternatives), pmf vector . . . 1540
15.2.11 Mann-Whitney U distribution (under Milton alternatives), pmf vector . . . 1541
15.3 Discrete (non-lattice) distribution functions related to rank tests . . . 1542
15.3.1 Cochran S distribution (under H_0), pmf vector . . . 1542
15.3.2 Friedman S distribution (under H_0), pmf vector . . . 1542
15.3.3 Quade S distribution (under H_0), pmf vector . . . 1542
15.3.4 Kruskal-Wallis H distribution (under H_0), pmf vector . . . 1543

16 Fast approximations **1544**
16.1 Approximations based on the normal distribution . . . 1544
16.1.1 Non-central chi-squared distribution: cdf and sf (Penev) . . . 1544
16.1.2 (Non-central) chi-squared distribution: qtf and isf (Canal) . . . 1544
16.1.3 Gamma distribution: qtf and isf (Canal) . . . 1545
16.1.4 F distribution: qtf and isf (Davis) . . . 1546
16.1.5 Beta distribution: qtf and isf (Davis) . . . 1546
16.1.6 Pearson's rho distribution: pdf (Winterbottom) . . . 1547
16.1.7 Pearson's rho distribution: cdf and sf (Winterbottom) . . . 1547
16.1.8 Pearson's rho distribution: qtf and isf (Winterbottom) . . . 1547
16.1.9 Pearson's rho distribution: confidence limit for ρ (Winterbottom) . . . 1548
16.1.10 Singly noncentral t: pdf (Broda) . . . 1549
16.1.11 Singly noncentral t: cdf, sf (Broda) . . . 1549
16.1.12 Singly noncentral t: qtf, isf (Harley) . . . 1550
16.1.13 Singly noncentral t: confidence limit for δ (Akahira) . . . 1550
16.1.14 Doubly noncentral t: cdf, sf (Broda) . . . 1551
16.1.15 Doubly noncentral t: qtf, isf (Broda) . . . 1552
16.1.16 Spearman's rho, first 8 cumulants (David) . . . 1552
16.1.17 Mann-Whitney U distribution: general alternatives specified by rank order probabilities (Sundrum) . . . 1553
16.1.18 First 4 moments of Kendalls τ in the general case (Sundrum) . . . 1555
16.2 Approximations based on the chi-squared distribution . . . 1557
16.2.1 Non-Central chi-squared : cdf, sf (Patnaik) . . . 1557
16.2.2 Non-Central chi-squared: qtf, isf (Patnaik) . . . 1557
16.2.3 Non-Central chi-squared: confidence limit for λ (Winterbottom) . . . 1557
16.2.4 Roy's largest root θ: pdf (Chiani) . . . 1558
16.2.5 Roy's largest root: cdf and sf (Chiani) . . . 1559
16.2.6 Roy's largest root: qtf and isf (Chiani) . . . 1560
16.3 Approximations based on the central t, F or beta distribution . . . 1561
16.3.1 Dunn-Šidák percentage points . . . 1561
16.3.2 Singly non-central Fisher F distribution: cdf, sf (Patnaik) . . . 1561
16.3.3 Singly non-central F distribution: qtf, isf (Patnaik) . . . 1562
16.3.4 Singly non-central F: confidence interval for the noncentrality parameter λ . . . 1562
16.3.5 Doubly non-central F distribution: cdf, sf (Patnaik) . . . 1562
16.3.6 Doubly non-central F distribution: qtf, isf (Patnaik) . . . 1563
16.3.7 Multiple correlation coefficient: cdf, sf (Lee and Gurland) . . . 1564
16.3.8 Multiple correlation coefficient: qtf, isf (Lee and Gurland) . . . 1564
16.3.9 Fisher R^2,: confidence limit for ρ^2 . . . 1564
16.3.10 Central Wilks' Lambda: cdf, sf (Rao) . . . 1566
16.3.11 Central Wilks' Lambda: qtf, isf (Rao) . . . 1566
16.3.12 Central Hotelling's T^2: cdf, sf (Pillai and Young) . . . 1567

16.3.13 Central Hotelling's T^2: qtf, isf (Pillai and Young) . . . 1567
16.3.14 Central Pillai's V: cdf, sf (Ginzberg) . . . 1568
16.3.15 Central Pillai's V: qtf, isf (Ginzberg) . . . 1568
16.3.16 Product of independent beta variables: cdf, sf (Nagarsenker) . . . 1569
16.3.17 Product of independent beta variables: qtf, isf (Nagarsenker) . . . 1569
16.4 Approximations based on the noncentral chi-squared distribution . . . 1571
16.4.1 Non-central Wilks' Lambda (GLM): cdf and sf (Fujikoshi) . . . 1571
16.4.2 Non-central Wilks' Lambda (independence): cdf and sf (Lee) . . . 1573
16.4.3 Non-central Pillai's V (GLM): cdf and sf Fujikoshi . . . 1574
16.4.4 Non-central Pillai's V (independence): cdf and sf (Lee) . . . 1575
16.4.5 Non-central Hotelling T^2 (GLM): cdf and sf (Fujikoshi) . . . 1576
16.4.6 Non-central Hotelling T^2 (independence): cdf and sf (Lee) . . . 1577
16.5 Approximations based on the noncentral F or beta distribution . . . 1579
16.5.1 Multiple correlation coefficient (Lee and Gurland) . . . 1579
16.5.2 Noncentral Wilks' Lambda under the GLM alternative . . . 1579
16.5.3 Noncentral Wilks' Lambda under the independence alternative . . . 1579
16.5.4 Noncentral Hotelling's T under the GLM alternative . . . 1580
16.5.5 Noncentral Hotelling's T under the independence alternative . . . 1580
16.5.6 Noncentral Pillai's V under the GLM alternative . . . 1581
16.5.7 Noncentral Pillai's V under the independence alternative . . . 1581
16.5.8 Noncentral Roy's largest root under the GLM alternative . . . 1581
16.5.9 Noncentral Roy's largest root under the independence alternative . . . 1582
16.6 Approximations based on hypergeometric functions of scalar argument . . . 1583
16.6.1 Hypergeometric function ${}_1F_1$ for matrix argument (Butler's approximation) . . . 1583
16.6.2 Comparisons with MATLAB: some results . . . 1583
16.6.3 Hypergeometric function ${}_2F_1$ for matrix argument (Butler's approximation) . . . 1584
16.6.4 Comparison with MATLAB: some results . . . 1584

IV Gallery of Plots **1585**

17 Visualisation of datasets **1586**
17.1 Bar Charts . . . 1587
17.1.1 Bar chart, part 1 . . . 1587
17.1.2 Bar chart, part 2 . . . 1588
17.1.3 Stacked Bar Plot . . . 1589
17.1.4 Grouped Bar Plot . . . 1590
17.1.5 Grouped StackedBar Plot . . . 1591
17.1.6 Barplot with error bar . . . 1592
17.1.7 Waterfall Plot . . . 1593
17.1.8 Mirror Histogram . . . 1594
17.1.9 Horizontal Bar . . . 1595
17.1.10 Horizontal Error Bar . . . 1596
17.1.11 Stacked horizontal bar with labels . . . 1597
17.1.12 Stacked and grouped horizontal bar . . . 1598
17.1.13 Gantt Chart . . . 1599
17.1.14 Marketing Funnel . . . 1600
17.1.15 Population Pyramid . . . 1601
17.1.16 Mirror Bar Plot (horizontal) . . . 1602
17.1.17 Tornado Chart . . . 1603
17.1.18 Circular Barplot . . . 1604
17.1.19 Barplot with color . . . 1605
17.2 Line and lollipop charts . . . 1606
17.2.1 Line chart (Seaborn) . . . 1606

17.2.2 Slope Chart . . . 1607
17.2.3 Line chart with error Bar . . . 1608
17.2.4 Confidence band (RandomWalk) . . . 1609
17.2.5 Plotting Parallel (Pandas) . . . 1610
17.2.6 Andrew Curves (Pandas) . . . 1611
17.2.7 Highlighted LinePlot . . . 1612
17.2.8 Multiple Lines And Panels . . . 1613
17.2.9 Line chart with fancy formatting . . . 1614
17.2.10 Lolllipop 2 Groups . . . 1615
17.2.11 Highlight Group In Lollipop . . . 1616
17.2.12 Lolllipop Conditional Color . . . 1617
17.2.13 Lollipop chart with fancy formatting . . . 1618
17.2.14 Circular Lolllipop . . . 1619
17.3 Area Plots . . . 1621
17.3.1 Percentage Stacked Area Chart . . . 1621
17.3.2 StackedAreaChart1 . . . 1622
17.3.3 Steam graph . . . 1624
17.3.4 Area Chart . . . 1625
17.3.5 Density Chart . . . 1626
17.3.6 Ridgetline Plot . . . 1627
17.3.7 Kde Plot, 3 versions . . . 1628
17.4 Boxplot, Violinplot and Raincloud plot . . . 1631
17.4.1 Boxplot, simple . . . 1631
17.4.2 Boxplot, multiple groups . . . 1632
17.4.3 ViolinPlot . . . 1633
17.4.4 Boxplot vs violinplot . . . 1634
17.4.5 Boxenplot . . . 1635
17.4.6 Swarmplot . . . 1636
17.4.7 Stripplot . . . 1637
17.4.8 Catplot . . . 1638
17.4.9 Raincloud Plot . . . 1639
17.5 Correlation and regression . . . 1640
17.5.1 Linear regression with confidence Band . . . 1640
17.5.2 Anscombe example . . . 1641
17.5.3 Faceted Scatter Plots: all in one plot . . . 1642
17.5.4 Faceted Scatter Plots: separate plots . . . 1643
17.5.5 Faceted Scatter Plots: further subgrouping . . . 1644
17.5.6 Bubbleplot . . . 1645
17.5.7 Scatterplot with fancy formatting . . . 1646
17.5.8 Scatterplots with positive, negative and weak correlation: confidence ellipses . . . 1647
17.5.9 Scatterplot with confidence confidence ellipses at different standard deviations . . . 1648
17.5.10 Hexagonal binned plot and marginal histograms (Seaborn) . . . 1649
17.5.11 Joint and marginal Kde (Seaborn) . . . 1650
17.5.12 Marginal ticks . . . 1651
17.5.13 Regression_marginals (Seaborn) . . . 1652
17.5.14 Pair grid with kde (Seaborn) . . . 1653
17.5.15 Scatterplot matrix (Seaborn) . . . 1654
17.6 Financial plots (requires mplfinance) . . . 1655
17.6.1 HiLoPlot . . . 1655
17.6.2 CandlePlot . . . 1656
17.6.3 LinePlot . . . 1657
17.6.4 RenkoPlot . . . 1658
17.6.5 PnfPlot . . . 1659
17.6.6 OhlcMav4Plot . . . 1660

17.6.7 CandleMav369Plot . . . 1661
17.6.8 CandleMav369VolPlot . . . 1662
17.6.9 CandleMav369VolNonTradPlot . . . 1663
17.7 Geographic data (requires cartopy) . . . 1664
17.7.1 Antarctis . . . 1664
17.7.2 Australia . . . 1665
17.7.3 Africa . . . 1666
17.7.4 World Flight . . . 1667
17.7.5 World Night Shading . . . 1668
17.7.6 World2projections . . . 1669
17.7.7 Country population estimates . . . 1670
17.7.8 Arctis and Antarctis . . . 1671
17.7.9 Geographic Mapping, with longitude and altitude on 4 axes crs=ccrs.PlateCarree . . . 1672
17.7.10 Geographic Mapping, World, projection=ccrs.InterruptedGoodeHomolosine . . . 1673
17.7.11 Geographic Mapping, World, projection=ccrs.PlateCarree . . . 1674
17.7.12 Labelling projection axes: PlateCarree and Mercator . . . 1675
17.8 Geographic data (requires geopandas) . . . 1676
17.8.1 World Happiness Report 2019 . . . 1676
17.8.2 World Happiness Report 2019: Europe . . . 1677
17.8.3 US state populations . . . 1678
17.8.4 California . . . 1679
17.8.5 Berlin, Esda . . . 1680
17.8.6 New York, Collisions . . . 1681
17.8.7 New York, Tickets . . . 1682
17.8.8 New York, Factors . . . 1683
17.8.9 Europe CO2 Emissions . . . 1684
17.8.10 US, Salary . . . 1685
17.8.11 Belgium, unemployment . . . 1686
17.8.12 Earthquakes . . . 1687
17.9 Parts of a whole . . . 1688
17.9.1 Waffle Chart . . . 1688
17.9.2 Pie Chart . . . 1689
17.9.3 Donut Chart . . . 1690
17.9.4 Radar Chart . . . 1691
17.9.5 Venn Diagram . . . 1692
17.9.6 Treemap . . . 1693
17.10 Dendrograms, heatmaps and clustermaps . . . 1694
17.10.1 Dendrogram . . . 1694
17.10.2 Heatmap, simple . . . 1697
17.10.3 Heatmap, complex . . . 1698
17.10.4 Matshow . . . 1699
17.10.5 Many pairwise correlations (Seaborn) . . . 1700
17.10.6 Structured heatmap (Seaborn) . . . 1701
17.10.7 Clustermap (Seaborn) . . . 1702
17.11 PlotTable . . . 1703
17.11.1 PlotTable1 . . . 1703
17.11.2 PlotTable2 . . . 1704
17.11.3 PlotTable3 . . . 1705
17.11.4 PlotTable4 . . . 1706
17.11.5 PlotTable5 . . . 1707
17.12 Flow and connections . . . 1708
17.12.1 Sankey Default . . . 1708
17.12.2 Sankey 2 Systems . . . 1709
17.12.3 Sankey Rankine . . . 1710

17.12.4 Sankey One 2 one . . . 1711
17.12.5 Sankey Flow . . . 1712
17.12.6 Arcplot . . . 1713
17.12.7 Funnel Chart . . . 1714
17.13 Circular plots and flows . . . 1715
17.13.1 Circular track plotting . . . 1715
17.13.2 Circular heatmap . . . 1716
17.13.3 Chord diagram (matrix 3x6) . . . 1717
17.13.4 Chord diagram (matrix10x10) . . . 1718
17.13.5 Chord diagram (from-to table) . . . 1719
17.13.6 Circos plot: genom of a phage . . . 1720
17.13.7 Circos plot: genom of homo sapiens . . . 1721
17.13.8 Circos plot: chromosome links . . . 1722
17.13.9 Philogenetic tree: bars . . . 1723
17.13.10 Philogenetic tree: heatmap . . . 1724
17.14 Network data (requires networkx) . . . 1725
17.14.1 Multipartite_Layout . . . 1725
17.14.2 Rainbow_Coloring . . . 1726
17.14.3 Random_Geometry . . . 1727
17.14.4 Travelling_Salesman . . . 1728
17.14.5 NapoleonRussianCampaign . . . 1729
17.14.6 Community Visualisation . . . 1730
17.14.7 Connected Components . . . 1731

18 Visualisation of functions and curves (2D) 1732
18.1 Introduction to 2D functions plots . . . 1732
18.1.1 Simple plot of (well-behaved) functions in 2D . . . 1732
18.1.2 Function plots on different scales . . . 1733
18.1.3 Stepplot . . . 1734
18.1.4 Stemplot . . . 1735
18.1.5 Histograms and densities . . . 1736
18.1.6 Cumulative histogram and CDF . . . 1737
18.1.7 Plots of continuous distribution functions . . . 1738
18.1.8 Plots of discrete distribution functions . . . 1745
18.1.9 Mpmath-inspired functions plots in 2D . . . 1752
18.2 Basic curves . . . 1758
18.2.1 Regular convex polygon . . . 1758
18.2.2 Circle . . . 1759
18.2.3 Ellipse . . . 1760
18.2.4 Parabola . . . 1761
18.2.5 Hyperbola . . . 1762
18.2.6 Cycloid . . . 1763
18.2.7 Trochoid . . . 1764
18.2.8 Cardioid . . . 1765
18.2.9 Limaçon curve . . . 1766
18.2.10 Conchoid of de Sluze . . . 1767
18.2.11 Freeth's Nephroid . . . 1768
18.2.12 Strophoid . . . 1769
18.2.13 Cycloid of Ceva . . . 1770
18.2.14 Lemniscate of Gerono . . . 1771
18.2.15 Lemniscate of Bernoulli . . . 1772
18.3 Spirals . . . 1773
18.3.1 Archimedes' spiral . . . 1773
18.3.2 Fermat's spiral . . . 1774

18.3.3 Hyperbolic Spiral . . . 1775
18.3.4 Lituus . . . 1776
18.3.5 Logarithmic Spiral . . . 1777
18.3.6 Poinsot's Spiral . . . 1778
18.3.7 Cotes's spiral . . . 1779
18.3.8 Tanh Spiral . . . 1780
18.3.9 Nielsens Spiral . . . 1781
18.3.10 Cornu Spiral . . . 1782
18.4 Decorative curves . . . 1783
18.4.1 Fish curve . . . 1783
18.4.2 Heart curve . . . 1784
18.4.3 Chrysanthemum curve . . . 1785
18.4.4 Butterfly curve . . . 1786
18.5 General curve families . . . 1787
18.5.1 Regular star polygon . . . 1787
18.5.2 Regular compound polygon (polygram) . . . 1788
18.5.3 Epicycloid . . . 1789
18.5.4 Hypocycloid . . . 1790
18.5.5 Epitrochoid . . . 1791
18.5.6 Hypotrochoid . . . 1792
18.5.7 Rose curve . . . 1793
18.5.8 Lissajous curves . . . 1794
18.5.9 Superellipses (Lamé curves) . . . 1795
18.5.10 Epispirals . . . 1796
18.5.11 Nodal curves (stereographic projections of the clelia) . . . 1797
18.5.12 Cyclic-harmonic curves . . . 1798
18.5.13 Rational circular cubics . . . 1799
18.6 Field lines . . . 1800
18.6.1 Colored electric field lines: 1 positive vs 1 negative charge . . . 1800
18.6.2 Colored electric field lines: 3 positive vs 1 negative charge(s) . . . 1801
18.6.3 Colored electric field lines: 3 positive vs 1 positive charge(s) . . . 1802
18.6.4 Colored electric field lines: 3 positive and 2 negative vs 1 positive charge(s) . . . 1803
18.7 Contours . . . 1804
18.7.1 Open Contourplot . . . 1804
18.7.2 Filled Contourplot . . . 1805
18.7.3 Electric dipole: field lines and open contours . . . 1806
18.7.4 Electric quadrupole: field lines and open contours . . . 1807
18.7.5 Electric dipoles: field lines and filled contours . . . 1808
18.7.6 Electric quadrupole: field lines and filled contours . . . 1809
18.7.7 Orthogonal electric dipoles: field lines and filled contours . . . 1810
18.7.8 Electric field lines around a point charge with grounded sphere . . . 1811
18.8 Complex functions rendered as contours . . . 1812
18.8.1 Complex square function . . . 1812
18.8.2 Complex sqrt function . . . 1813
18.8.3 Complex exp function . . . 1814
18.8.4 Complex log function . . . 1815
18.8.5 Complex sine function . . . 1816
18.8.6 Complex asin function . . . 1817
18.8.7 Complex tan function . . . 1818
18.8.8 Complex atan function . . . 1819
18.9 Streams, barbs and quivers (some require cartopy) . . . 1820
18.9.1 Streams, Electric Dipole: vector field . . . 1820
18.9.2 Streams, Electric Quadrupole: vector field . . . 1821
18.9.3 Streams, Electric Octopole: vector field . . . 1822

18.9.4 Streams, Electric Hexapole: vector field . . . 1823
18.9.5 Streams, North Atlantic Ocean and Europe (requires cartopy) . . . 1824
18.9.6 Quivers, North Atlantic Ocean and Europe (requires cartopy) . . . 1825
18.9.7 Quivers, plain and regridded (requires cartopy) . . . 1826
18.9.8 Barbs, North Atlantic Ocean and Europe (requires cartopy) . . . 1827

19 Bitmaps **1828**
19.1 Fractals: introduction . . . 1828
19.1.1 Barnsley's Fern . . . 1829
19.1.2 Dragon curve . . . 1830
19.1.3 Sierpiński carpet . . . 1831
19.2 Fractals related to the Mandelbrot set . . . 1832
19.2.1 Mandelbrot set, example 1, classical set: $z_{n+1} = z_n^2 + c$ . . . 1832
19.2.2 Mandelbrot set, example 2: higher powers . . . 1833
19.2.3 Mandelbrot set, example 3: $z_{n+1} = z_n^5 + \frac{c}{z_n}$ . . . 1834
19.2.4 Mandelbrot set, example 4: $z_{n+1} = z_n^5 + \frac{c}{z_n^5}$ . . . 1835
19.2.5 Mandelbrot set, example 5, "Inverse Mandelbrot" . . . 1836
19.2.6 Mandelbrot set, example 6, "Marek Dragon Fractal": . . . 1837
19.2.7 Mandelbrot set, example 7, "Burning Ship Fractal" . . . 1838
19.3 Fractals related to the Julia set . . . 1839
19.3.1 Julia sets, example 1: Classical set . . . 1839
19.3.2 Julia sets, example 2: Glynn fractals . . . 1840
19.3.3 Julia sets, example 3: Spiral Septagon fractal . . . 1841
19.3.4 Julia sets, example 4: Reuleaux Triangle fractal . . . 1842
19.3.5 Julia sets, example 5: Phoenix Julia fractals . . . 1843
19.3.6 Julia sets, example 6: Cubic Julia fractals . . . 1844
19.3.7 Julia sets, example 7: Spiral Julia fractal (uses $z_{n+1} = \tan(z_n^2 + c)$) . . . 1845
19.3.8 Julia sets, example 8: Julia set of $\sin(z)$ . . . 1846
19.3.9 Julia sets, example 9: Julia09 fractals . . . 1847
19.3.10 Julia sets, example 10: Julia10 fractals . . . 1848
19.3.11 Julia sets, example 11: Julia11 fractals . . . 1849
19.4 Newton Fractals . . . 1850
19.4.1 Newton, zeros of $z^3 - 1$ . . . 1850
19.4.2 Newton, zeros of $\cos(z)$ . . . 1851
19.4.3 Modified Newton (containing an error), zeros of $z^3 - 1$ . . . 1852
19.4.4 Modified Newton (containing an error), not converging . . . 1853
19.4.5 Halley, zeros of $z^3 - 1$ . . . 1854
19.4.6 Halley, zeros of $\cos(z)$ . . . 1855
19.4.7 Modified Halley (containing an error), zeros of $z^3 - 1$ . . . 1856
19.5 Domain coloring . . . 1857
19.5.1 Domain coloring without contours: $f(z) = (z^3 - 1)/z$ . . . 1858
19.5.2 Domain coloring with contours of the phase: $f(z) = (z^3 - 1)/z$ . . . 1859
19.5.3 Domain coloring with contours of the modulus: $f(z) = (z^3 - 1)/z$ . . . 1860
19.5.4 Domain coloring with contours of phase and modulus: $f(z) = (z^3 - 1)/z$ . . . 1861
19.5.5 Domain coloring with contours of phase and modulus: $f(z) = z^5 + z$ . . . 1862
19.5.6 Domain coloring with contours of phase and modulus: $f(z) = (z^6 - 1)/(z^{12} + 1)$ . . . 1863
19.5.7 Domain coloring with contours of phase and modulus: $f(z) = (z - 0.5(1 + i))/z^2$ . . . 1864
19.5.8 Domain coloring with contours of phase and modulus: $f(z) = \exp(1/z)$ . . . 1865
19.5.9 Domain coloring with contours of phase and modulus: $f(z) = z\exp(1/z)$ . . . 1866
19.5.10 Domain coloring with contours of phase and modulus: $f(z) = \exp(1/z^2)$ . . . 1867
19.5.11 Domain coloring with contours of phase and modulus: $f(z) = \sin(z)$ . . . 1868
19.5.12 Domain coloring with contours of phase and modulus: $f(z) = z\sin(1/z)$ . . . 1869
19.5.13 Domain coloring with contours of phase and modulus: $f(z) = \sin(z)/(z - i)^2$ . . . 1870
19.5.14 Domain coloring with contours of phase and modulus: $f(z) = \tan(z)$ . . . 1871

20 Matplotlib 3D Graphics **1872**
20.1 Introduction . . . 1872
20.1.1 Overview . . . 1872
20.2 Matplotlib: 3D Graphics (from Matplotlib tutorials) . . . 1873
20.2.1 Klein bottle . . . 1873
20.2.2 Lorentz attractor . . . 1874
20.2.3 Parametric curve . . . 1875
20.2.4 Projected filled contour . . . 1876
20.2.5 Projected open contour . . . 1877
20.2.6 Scatterplot . . . 1878
20.2.7 Surface colormap . . . 1879
20.2.8 Surface hillshading . . . 1880
20.2.9 Surface solid . . . 1881
20.2.10 Surface triangular . . . 1882
20.3 S3dlib: Basic geometric figures . . . 1883
20.3.1 Base 3D class surface . . . 1883
20.3.2 Hypercube . . . 1884
20.3.3 Surface elements . . . 1885
20.3.4 Hyperboloid 3D . . . 1886
20.3.5 Conic sections . . . 1887
20.3.6 Base 3D class surface, part 2 . . . 1888
20.3.7 Rhombicuboctahedron . . . 1889
20.3.8 Rhombicuboctahedron, bubbles . . . 1890
20.4 S3dlib: Real functions . . . 1891
20.4.1 Surface plot of a real function, basic example . . . 1891
20.4.2 Surface plot of a real function, grid . . . 1892
20.4.3 Surface plot of a real function, peaks example . . . 1893
20.4.4 Function plots: Ackley, Himmelblau, Rosenbrock, Rastrigin . . . 1894
20.4.5 McCormick function . . . 1895
20.4.6 Surface plot combined with contour . . . 1896
20.4.7 Sliced polar surface . . . 1897
20.5 S3dlib: Parametric Surfaces . . . 1898
20.5.1 Color representation vs geometric representation, part 1 . . . 1898
20.5.2 Color representation vs geometric representation, part 2 . . . 1899
20.5.3 Color maps: Möbius function example . . . 1900
20.5.4 Shading: Möbius function example . . . 1901
20.5.5 Color maps: Boy, Dini, Bour, Roman, Klein, and Figure8 examples . . . 1902
20.5.6 Colored knot in 3D . . . 1903
20.5.7 Multiple geometric maps . . . 1904
20.5.8 Example from Mayavi . . . 1905
20.5.9 S3Dlib Logo . . . 1906
20.5.10 Spherical harmonics, 3D representation . . . 1907
20.5.11 Boy surface . . . 1908
20.5.12 Dini surface . . . 1909
20.5.13 Bour surface . . . 1910
20.5.14 Roman surface . . . 1911
20.5.15 Klein bottle . . . 1912
20.5.16 Klein bottle8 . . . 1913
20.6 S3dlib: Image mapping and clipping . . . 1914
20.6.1 Image HSV value clipping . . . 1914
20.6.2 Spherical image mapping . . . 1915
20.6.3 Earth in 3D . . . 1916
20.6.4 Cylindrical image mapping . . . 1917
20.6.5 Imaginary Earth, part A . . . 1918

20.6.6 Imaginary Earth, part B . . . 1919
20.6.7 Earth, projected on Klein bottle . . . 1920
20.6.8 Earth, projected on planar, screw, knot and swirl surfaces . . . 1921
20.7 S3dlib: Edge-to-edge surface coloring . . . 1922
20.7.1 Platonic solid, with surface edges filled, part 1 . . . 1922
20.7.2 Platonic solid, with surface edges filled, part 2 . . . 1923
20.7.3 Spherical contour surface, filled . . . 1924
20.7.4 Edge-to-edge surface, part 1 . . . 1925
20.7.5 Spherical edge surface, filled . . . 1926
20.7.6 Edge-to-edge surface, part 2 . . . 1927
20.7.7 Line colormap sequence . . . 1928
20.7.8 Sequential line color, part 1 . . . 1929
20.7.9 Sequential line color, part 2 . . . 1930
20.8 S3dlib: Decorative parametric surfaces . . . 1931
20.8.1 Shell surface with colormap . . . 1931
20.8.2 Escher Knot . . . 1932
20.8.3 Escher sphere spirals . . . 1933
20.8.4 Escher spirals . . . 1934
20.9 S3dlib: Geometric and color datagrid mapping (requires scipy) . . . 1935
20.9.1 Mandelbrot contours . . . 1935
20.9.2 Geometric and color datagrid mapping . . . 1936
20.9.3 MRI 3D . . . 1937
20.9.4 Jacksboro fault . . . 1938
20.9.5 Mandelbrot 3D . . . 1939
20.9.6 MRI regions . . . 1940
20.10 S3dlib: Implicit surfaces (requires scikit-image) . . . 1941
20.10.1 Sphere (implicit definition), part 1 . . . 1941
20.10.2 Sphere (implicit definition), part 2 . . . 1942
20.10.3 Goursat surface . . . 1943
20.10.4 Schwarz P surface . . . 1944
20.10.5 Nodal surface . . . 1945
20.10.6 Spirals surface . . . 1946
20.10.7 Iso surface . . . 1947
20.10.8 Implicit surface . . . 1948

21 Plotly **1949**
21.1 General surface plots in 3D . . . 1949
21.1.1 Mount Bruno . . . 1949
21.1.2 Ring Cyclide . . . 1950
21.1.3 Torus . . . 1951
21.1.4 Möbius surface . . . 1952
21.1.5 Boy surface . . . 1953
21.1.6 Overlaid function . . . 1954
21.1.7 Iris data . . . 1955
21.2 Geographical plots . . . 1956
21.2.1 Movable projection of the world . . . 1956
21.2.2 Rotatable globe . . . 1957
21.2.3 Choropleth of the US with Alaska and Hawai . . . 1958
21.2.4 Voting results embedded in map . . . 1959
21.3 Sankey plots . . . 1960
21.3.1 Sankey plot 1 . . . 1960
21.3.2 Sankey plot 2 . . . 1961
21.3.3 Sankey plot 3 . . . 1962

V Interactive 3D Wpf Plots 1963

22 Wpf: Altitude surfaces in 3D, real and complex functions 1964
22.1 Special techniques for height surfaces, real and complex functions 1964
22.1.1 Wpf figures as standard display of complex functions 1964
22.1.2 Truncation vs loglog transformation 1965
22.1.3 Displaying branch cuts 1965
22.2 Scatterplots and building 3D scenes 1966
22.2.1 Scatter plot of bivariate normal 1966
22.2.2 Scatter plot of surfaces 1967
22.3 Height plots of general bivariate real functions 1968
22.3.1 Hyperbolic paraboloid 1968
22.3.2 Array Surface, general 1969
22.3.3 Fractal Landscape surface 1970
22.3.4 Sombrero function 1971
22.3.5 General 3D Wave function 1972
22.3.6 Bivariate normal function 1972

23 Wpf: Parametric surfaces 1973
23.1 Surfaces of translation 1973
23.1.1 Fermat spiral as generator 1973
23.1.2 Lemniscate as generator 1974
23.1.3 Tanh spiral as generator 1975
23.1.4 Rose curve as generator 1976
23.1.5 Hypocycloid as generator 1977
23.1.6 Epicycloid as generator 1978
23.2 Surfaces of revolution 1979
23.2.1 One-Sheeted Hyperboloid 1979
23.2.2 Sphere 1980
23.2.3 Prolate Spheroid 1981
23.2.4 Oblate Spheroid 1982
23.2.5 Drop surface 1983
23.2.6 Egg surface 1984
23.2.7 Apple surface 1985
23.2.8 Spinning top 1986
23.2.9 Pseudosphere 1987
23.2.10 Wave sphere Surface 1988
23.2.11 Torus 1989
23.2.12 Horn Torus 1990
23.2.13 Spindle Torus 1991
23.2.14 Elliptic Torus 1992
23.3 Generalisations of common surfaces 1993
23.3.1 Ellipsoid 1993
23.3.2 Superellipsoid 1994
23.3.3 Hexaedron 1995
23.3.4 Super Toroid 1996
23.3.5 Elliptic Helicoid 1997
23.3.6 Hyperbolic Helicoid 1998
23.3.7 Lemniscate 1999
23.3.8 Bohemian Dome 2000
23.3.9 Dupin1 surface 2001
23.3.10 Dupin2 surface 2002
23.3.11 Dinis Surface (twisted pseudosphere) 2003
23.3.12 Plueckers conoid 2004

23.3.13 Umbilic Torus . . . 2005
23.3.14 Skidian's ruled surface . . . 2006
23.3.15 Umbrella surface . . . 2007
23.3.16 Cyclic surfaces (generalized torus) . . . 2008
23.3.17 Goursat surfaces . . . 2010
23.3.18 Cyclides triples . . . 2012
23.3.19 Ship Lamé . . . 2014
23.4 Minimal surfaces . . . 2015
23.4.1 Catenoid . . . 2015
23.4.2 Helicoid . . . 2016
23.4.3 Bours minimal surface . . . 2017
23.4.4 Catalan minimal surface . . . 2018
23.4.5 Ennepers first minimal surface . . . 2019
23.4.6 Enneper second minimal surface . . . 2020
23.4.7 Hennebergs minimal surface . . . 2021
23.4.8 Scherk's first minimal surface . . . 2022
23.4.9 Scherk's second minimal surface . . . 2023
23.4.10 Richmond minimal surface . . . 2024
23.4.11 Generalized Ennneper surfaces . . . 2025
23.5 Nonorientable (one-sided) Surfaces . . . 2026
23.5.1 Moebius Strip . . . 2026
23.5.2 Cross-Cap Surface . . . 2027
23.5.3 Pseudo Cross-Cap Surface . . . 2028
23.5.4 Roman Surface (or Steiner Surface) . . . 2029
23.5.5 Klein bagel . . . 2030
23.5.6 Klein bottle . . . 2031
23.5.7 3D Boy Surface, version 1 . . . 2032
23.5.8 3D Boy Surface, version 2 . . . 2033
23.5.9 Morin Surface . . . 2034
23.6 Decorative parametric surfaces . . . 2035
23.6.1 Bourke Seashell . . . 2035
23.6.2 Seashell (mathworld.wolfram) . . . 2036
23.6.3 3D Apple . . . 2037
23.6.4 Bow curve . . . 2038
23.6.5 Fish Surface . . . 2039
23.6.6 Bourke Horn . . . 2040
23.6.7 Hexa torus . . . 2041
23.6.8 3D Breather Surface . . . 2042
23.6.9 Kuen surface . . . 2043
23.6.10 Tranguloid trefoil . . . 2044
23.6.11 Triaxial teardrop . . . 2045
23.6.12 Gray Bottle . . . 2046
23.6.13 Surfaces mimicking snail shells . . . 2047

24 Wpf: Path surfaces in 3D **2050**
24.1 Introduction to path surfaces . . . 2050
24.1.1 Borromean rings, A, B, C . . . 2050
24.1.2 Ellipses, A, B, C . . . 2052
24.1.3 Trefoil 2 . . . 2053
24.1.4 Formatting options for path surfaces, Trefoil 5 . . . 2054
24.2 Functions with real input and complex results . . . 2055
24.2.1 Expj (Cis) function . . . 2055
24.2.2 Hankel function of the first kind $H_{1,\nu}(x)$ . . . 2057
24.2.3 Hankel function of the second kind $H_{2,\nu}(x)$ . . . 2058

24.2.4 Kelvin functions ber and bei . . . 2059
24.2.5 Kelvin functions ker and kei . . . 2060
24.3 Characteristic functions of statistical distributions . . . 2061
24.3.1 Uniform distribution . . . 2061
24.3.2 Normal distribution . . . 2062
24.3.3 Chi-squared distribution . . . 2063
24.3.4 Beta distribution . . . 2064
24.3.5 F distribution . . . 2065
24.3.6 Non-central Chi-squared distribution . . . 2066
24.3.7 Binomial distribution . . . 2067
24.3.8 Hypergeometric distribution . . . 2068
24.4 Helices and related curves traced on cylinders, cones and spheres . . . 2069
24.4.1 Cylindrical helix . . . 2070
24.4.2 Conical helix based on Archimedes spiral (spiral of Pappus) . . . 2071
24.4.3 Conical helix based on Fermat's spiral . . . 2072
24.4.4 Conical helix based on the logarithmic spiral (Concho-Spiral) . . . 2073
24.4.5 Rhumb line of the sphere . . . 2074
24.4.6 Clelia . . . 2075
24.4.7 Spherical helix . . . 2076
24.4.8 Satellite curve . . . 2078
24.5 (Coil) springs . . . 2080
24.5.1 Spring 1 . . . 2080
24.5.2 Spring 2 . . . 2081
24.5.3 Spring 3 . . . 2082
24.5.4 Spring 4 . . . 2083
24.5.5 Spring 5 . . . 2084
24.5.6 Spring 6 . . . 2085
24.5.7 Spring 7 . . . 2086
24.5.8 Spring 8 . . . 2087
24.6 General knots . . . 2088
24.6.1 Trefoil A . . . 2088
24.6.2 Trefoil B . . . 2089
24.6.3 Square knot . . . 2090
24.6.4 Granny knot . . . 2091
24.6.5 Cinquefoil Knot . . . 2092
24.6.6 74 Knot . . . 2093
24.6.7 Figure-eight knot . . . 2094
24.7 Torus knots . . . 2095
24.7.1 Torus knot 3:2 . . . 2095
24.7.2 Torus knot 5:2 . . . 2096
24.7.3 Torus knot 7:2 . . . 2097
24.7.4 Torus knot 7:3 . . . 2098
24.7.5 Torus knot 15:2 . . . 2099
24.8 Lissajous knots . . . 2100
24.8.1 Lissajous knot 1-1-1 . . . 2101
24.8.2 Lissajous knot 1-2-1 . . . 2102
24.8.3 Lissajous knot 1-5-3 . . . 2103
24.8.4 Lissajous knot 3-5-2 . . . 2104
24.8.5 Lissajous knot 3-5-7 . . . 2105
24.9 Polynomial knots . . . 2106
24.9.1 Polynomial knot 3-1 . . . 2106
24.9.2 Polynomial knot 4-1 . . . 2107
24.9.3 Polynomial knot 5-1 . . . 2108
24.9.4 Polynomial knot 6-2 . . . 2109

24.9.5 Polynomial knot 7-4 . . . 2110

25 Wpf: Built-in 3D objects **2111**
25.1 Builtin solids with support for textures . . . 2112
25.1.1 Sphere . . . 2112
25.1.2 Prolate Spheroid . . . 2114
25.1.3 Oblate Spheroid . . . 2115
25.1.4 Ellipsoid . . . 2117
25.1.5 Torus . . . 2118
25.1.6 Rectangular cuboid . . . 2119
25.1.7 Rhombohedron . . . 2120
25.1.8 Parallelepiped . . . 2121
25.2 Builtin solids without support for textures . . . 2122
25.2.1 Triangular Prism . . . 2122
25.2.2 Square prism . . . 2123
25.2.3 Hexagonal prism . . . 2124
25.2.4 Octagonal prism . . . 2125
25.2.5 Cylinder . . . 2126
25.2.6 Cylinder, truncated by an inclined plane . . . 2127
25.2.7 Pyramid . . . 2128
25.2.8 Pyramid frustum . . . 2129
25.2.9 Pyramid, truncated by an inclined plane . . . 2130
25.2.10 Cone . . . 2131
25.2.11 Cone Frustum . . . 2132
25.2.12 Cone, truncated by an inclined plane . . . 2133
25.3 Platonic solids, and related solids . . . 2134
25.3.1 Tetrahedron . . . 2134
25.3.2 Cube . . . 2135
25.3.3 Octahedron . . . 2136
25.3.4 Dodecahedron . . . 2137
25.3.5 Icosahedron . . . 2138
25.3.6 Geodesic sphere . . . 2139
25.3.7 Augmented Octahedron . . . 2140
25.3.8 Augmented Dodecahedron . . . 2141
25.3.9 Augmented Icosahedron . . . 2142
25.3.10 Augmented Geodesic Sphere . . . 2143
25.3.11 Stella Octangula . . . 2144
25.3.12 Small stellated Dodecahedron . . . 2145
25.3.13 Great Dodecahedron . . . 2146
25.3.14 Great stellated Dodecahedron . . . 2147

VI Additional functions in double precision **2148**

26 Real elementary functions **2149**
26.1 Root, exponential, logarithmic and power functions . . . 2150
26.1.1 Auxiliary function $\mathrm{sqrt1pmx}(x) = \sqrt{1+x^2} - x$ . . . 2150
26.1.2 Bring radical . . . 2150
26.1.3 Auxiliary function $\mathrm{expmx2h}(x) = \exp(-x^2/2)$ . . . 2150
26.1.4 Relative error exponential $\mathrm{exprel}(x) = (\exp(x) - 1)/x$ . . . 2150
26.1.5 Auxiliary function $\mathrm{expx2}(x) = \exp(x \cdot |x|)$ . . . 2151
26.1.6 Auxiliary function $\mathrm{logistic}(x) = 1/(1 + \exp(-x))$ . . . 2151
26.1.7 Einstein functions . . . 2151
26.1.8 Auxiliary function $\mathrm{log1mexp}(x) = \log(1 - \exp(x))$ . . . 2152

26.1.9 Auxiliary function $\mathrm{log1pexp}(x) = \log(1 + \exp(x))$ . . . 2152
26.1.10 Auxiliary function $\mathrm{log1pmx}(x) = \log(1 + x) - x$ . . . 2152
26.1.11 Auxiliary function $\mathrm{logaddexp}(x, y) = \log[\exp(x) + \exp(y)]$ . . . 2152
26.1.12 Auxiliary function $\mathrm{logsubexp}(x, y) = \log[\exp(x) - \exp(y)]$ . . . 2153
26.1.13 Auxiliary function $\mathrm{logit}(x) = \log(x/(1.0 - x))$ . . . 2153
26.1.14 Wright ω function . . . 2153
26.1.15 Auxiliary function $\mathrm{compound}(x, n) = (1 + x)^n$ . . . 2154
26.1.16 Auxiliary function $\mathrm{comprel}(x, n) = (1 + x)^n - 1$ . . . 2154
26.1.17 Auxiliary function $\mathrm{hypot3}(x, y, z) = \sqrt{x^2 + y^2 + z^2}$ . . . 2154
26.1.18 Fibonacci polynomials, $\mathrm{fibpoly}(n, x)$ . . . 2154
26.1.19 Lucas polynomials, $\mathrm{lucpoly}(n, x)$ . . . 2155
26.1.20 Fibonacci function, $F_\nu(x)$, of real index ν . . . 2155
26.2 Trigonometric functions . . . 2156
26.2.1 Sine, x in degrees, $\mathrm{sind}(x)$ . . . 2156
26.2.2 Inverse sine, input in degrees, $\mathrm{asind}(x)$ . . . 2156
26.2.3 Cosine, x in degrees, $\mathrm{cosd}(x)$ . . . 2156
26.2.4 Inverse cosine, input in degrees, $\mathrm{acosd}(x)$ . . . 2156
26.2.5 Tangent, with x in degrees, $\mathrm{tand}(x)$ . . . 2157
26.2.6 Inverse tangent, input in degrees, $\mathrm{atand}(x)$ . . . 2157
26.2.7 Cotangent, with x in degrees, $\mathrm{cotd}(x)$ . . . 2157
26.2.8 Inverse cotangent, input in degrees, $\mathrm{acotd}(x)$ . . . 2158
26.2.9 Auxiliary function, $\mathrm{acos}(1 - x)$ . . . 2158
26.2.10 Haversine function $\mathrm{hav}(x) = (1 - \cos(x))/2$ . . . 2158
26.2.11 Inverse haversine function $\mathrm{archav}(x) = \mathrm{acos}(1 - 2x)$ . . . 2158
26.2.12 Versine function $\mathrm{vers}(x) = 1 - \cos(x)$ . . . 2159
26.2.13 Coversine, $\mathrm{covers}(x) = 1 - \sin(x)$ . . . 2159
26.2.14 Versint function $\mathrm{versint}(x) = x - \sin(x)$ . . . 2159
26.2.15 Integral of cos powers, $\mathrm{cosint}(n, x)$ . . . 2159
26.2.16 Integral of sin powers, $\mathrm{sinint}(n, x)$ . . . 2160
26.3 Hyperbolic functions . . . 2161
26.3.1 Cardinal hyperbolic sine, $\mathrm{sinhc}(x) = \sinh(x)/x$ . . . 2161
26.3.2 Auxiliary function, $\mathrm{sinhmx}(x) = \sinh(x) - x$ . . . 2161
26.3.3 Auxiliary function, $\mathrm{coshm1}(x) = \cosh(x) - 1$ . . . 2161
26.3.4 Auxiliary function $\mathrm{acosh}(1 + x)$ . . . 2161
26.3.5 Auxiliary function $\log(\cosh(x))$ . . . 2162
26.3.6 Auxiliary function $\log(\sinh(x))$ . . . 2162
26.3.7 Gudermannian function $\mathrm{gd}(x) = \mathrm{asin}(\tanh(x))$ . . . 2162
26.3.8 Inverse Gudermannian function $\mathrm{arcgd}(x) = \mathrm{atanh}(\sin(x))$ . . . 2162
26.3.9 Langevin function, $L(x)$ . . . 2163
26.3.10 Inverse Langevin function, $L^{-1}(x)$ . . . 2163
26.3.11 Solutions of Kepler's equation, $\mathrm{kepler}(M, e)$ . . . 2163
26.4 Gamma functions . . . 2165
26.4.1 Sign of the gamma function . . . 2165
26.4.2 Logarithm and sign of the gamma function . . . 2165
26.4.3 Logarithm of $\Gamma(1 + x)$ . . . 2165
26.4.4 Logarithm of factorials: $\log(x!)$ . . . 2165
26.4.5 Logarithm of the binomial coefficient . . . 2166
26.4.6 Log-Beta function . . . 2166
26.4.7 Inverse of the gamma function, $\Gamma^{-1}(y)$ . . . 2166
26.4.8 Inverse of the logarithm of the gamma function, $\log \Gamma^{-1}(y)$ . . . 2167
26.4.9 Temme's regulated gamma function, $\Gamma^*(x)$ . . . 2167
26.4.10 Relative Pochhammer symbol, $((a)_x - 1)/x$ . . . 2167
26.4.11 Catalan function $C(x)$ . . . 2167
26.4.12 Bessel Lambda function, $\Lambda(\nu, x)$ . . . 2168

27 Real elliptic functions **2169**
27.1 Conversions of parameters of elliptic functions . . . 2169
27.1.1 Elliptic nome $q(k)$ (DAMath) . . . 2169
27.1.2 Elliptic modulus $k(q)$ (DAMath) . . . 2169
27.1.3 Jacobi amplitude, $\mathrm{am}(x,k)$ (DAMath) . . . 2170
27.2 Elliptic integrals . . . 2171
27.2.1 Legendre complete elliptic integral $B(k)$ . . . 2171
27.2.2 Legendre complete elliptic integral $D(k)$ . . . 2171
27.2.3 Legendre incomplete elliptic integral $B(\phi,k)$ . . . 2171
27.2.4 Legendre incomplete elliptic integral $D(\phi,k)$ . . . 2172
27.2.5 Heuman's Lambda function, $\Lambda_0(\phi,k)$ . . . 2172
27.2.6 Jacobi Zeta function, $Z(\phi,k)$ . . . 2172
27.3 Bulirsch elliptic integrals . . . 2173
27.3.1 Complete elliptic integral of the 1st kind $\mathrm{cel1}(k_c)$ . . . 2173
27.3.2 Complete elliptic integral of the 2nd kind $\mathrm{cel2}(k_c,a,b)$ . . . 2173
27.3.3 General complete elliptic integral $\mathrm{cel}(k_c,p,a,b)$ . . . 2174
27.3.4 Incomplete elliptic integral of the 1st kind $\mathrm{el1}(x,k_c)$ . . . 2174
27.3.5 Incomplete elliptic integral of the 2nd kind $\mathrm{el2}(x,k_c,a,b)$ . . . 2174
27.3.6 Incomplete elliptic integral of the 3rd kind $\mathrm{el3}(x,k_c,p)$ . . . 2175
27.4 Maple style elliptic integrals . . . 2176
27.4.1 Complete integral of the 1st kind, $\mathrm{EllipticK}(k)$ . . . 2176
27.4.2 Complete integral of the 1st kind for imaginary modulus, $\mathrm{EllipticKim}(k)$ . . . 2176
27.4.3 Complementary complete integral of the 1st kind, $\mathrm{EllipticCK}(k)=K'(k)$ . . . 2176
27.4.4 Complete integral of the 2nd kind, $\mathrm{EllipticEC}(k)$ . . . 2177
27.4.5 Complete integral of the 2nd kind for imaginary modulus, $\mathrm{EllipticECim}(k)$ . . . 2177
27.4.6 Complementary complete integral of the 2nd kind, $\mathrm{EllipticCE}(k)$ . . . 2177
27.4.7 Complete integral of the 3rd kind, $\mathrm{EllipticPiC}(\nu,k)$ . . . 2178
27.4.8 Complete integral of the 3rd kind for imaginary modulus, $\mathrm{EllipticPiCim}(\nu,k)$ . . . 2178
27.4.9 Complementary complete integral of the 3rd kind, $\mathrm{EllipticCPi}(\nu,k)$ . . . 2178
27.4.10 Incomplete integral of the 1st kind, $\mathrm{EllipticF}(z,k)$ . . . 2179
27.4.11 Incomplete integral of the 2nd kind, $\mathrm{EllipticE}(z,k)$ . . . 2179
27.4.12 Incomplete integral of the 3rd kind, $\mathrm{EllipticPi}(z,\nu,k)$ . . . 2179
27.5 Jacobi theta functions at $x=0$ for $0\le q<1$ . . . 2180
27.5.1 Jacobi theta function $\theta_1'(q)$ . . . 2180
27.5.2 Jacobi theta function $\theta_2(q)$ . . . 2180
27.5.3 Jacobi theta function $\theta_3(q)$ . . . 2180
27.5.4 Jacobi theta function $\theta_4(q)$ . . . 2180
27.6 Inverse Jacobi elliptic functions . . . 2182
27.6.1 Inverse Jacobi elliptic function $\mathrm{arcsn}(x,k)$ . . . 2182
27.6.2 Inverse Jacobi elliptic function $\mathrm{arccn}(x,k)$ . . . 2182
27.6.3 Inverse Jacobi elliptic function $\mathrm{arcdn}(x,k)$ . . . 2182
27.6.4 Inverse Jacobi elliptic function $\mathrm{arccd}(x,k)$ . . . 2183
27.6.5 Inverse Jacobi elliptic function $\mathrm{arcsd}(x,k)$ . . . 2183
27.6.6 Inverse Jacobi elliptic function $\mathrm{arcnd}(x,k)$ . . . 2183
27.6.7 Inverse Jacobi elliptic function $\mathrm{arcdc}(x,k)$ . . . 2184
27.6.8 Inverse Jacobi elliptic function $\mathrm{arcnc}(x,k)$ . . . 2184
27.6.9 Inverse Jacobi elliptic function $\mathrm{arcsc}(x,k)$ . . . 2184
27.6.10 Inverse Jacobi elliptic function $\mathrm{arcns}(x,k)$ . . . 2185
27.6.11 Inverse Jacobi elliptic function $\mathrm{arcds}(x,k)$ . . . 2185
27.6.12 Inverse Jacobi elliptic function $\mathrm{arccs}(x,k)$ . . . 2185
27.7 Lemniscate functions . . . 2187
27.7.1 Lemniscate sine function, $\mathrm{sinlemn}(x)$ . . . 2187
27.7.2 Lemniscate cosine function, $\mathrm{coslemn}(x)$ . . . 2187
27.7.3 Inverse lemniscate sine function, $\mathrm{arcsl}(x)$ . . . 2187

27.7.4 Inverse lemniscate cosine function, $\operatorname{arccl}(x)$ . . . 2188
27.8 Neville theta functions . . . 2189
27.8.1 Neville $\theta_c(x,k)$ . . . 2189
27.8.2 Neville $\theta_d(x,k)$ . . . 2189
27.8.3 Neville $\theta_n(x,k)$ . . . 2189
27.8.4 Neville $\theta_s(x,k)$ . . . 2190

28 Real functions related to Lerchs phi **2191**
28.1 Polygamma, and related functions . . . 2191
28.1.1 Pentagamma function $\psi'''(x)$ . . . 2191
28.1.2 Tetragamma function, $\psi''(x)$ . . . 2191
28.1.3 Auxiliary function $\psi^*(x)$ . . . 2191
28.1.4 Inverse digamma function $\psi^{-1}(y)$ . . . 2192
28.1.5 Bateman function $G(x)$ . . . 2192
28.2 Polylogarithm, and related functions . . . 2193
28.2.1 Polylogarithm of integer order, $\operatorname{Li}_n(x)$ . . . 2193
28.2.2 Fermi-Dirac integrals of integer order, $F_n(x)$ . . . 2193
28.2.3 Fermi-Dirac integral $F_{-1/2}(x)$ . . . 2194
28.2.4 Fermi-Dirac integral $F_{1/2}(x)$ . . . 2194
28.2.5 Fermi-Dirac integral $F_{3/2}(x)$ . . . 2194
28.2.6 Fermi-Dirac integral $F_{5/2}(x)$ . . . 2194
28.2.7 Inverse tangent integral, $\operatorname{Ti}_2(x)$ . . . 2195
28.2.8 Lobachevsky's log-cos integral, $L(x)$ . . . 2195
28.2.9 Lobachevsky's log-sin integral, $\Lambda(x)$ . . . 2195
28.2.10 Debye functions, $\operatorname{D}_n(x)$ . . . 2195
28.2.11 Transport integral, $\operatorname{J}_n(x)$ . . . 2196
28.3 Riemann zeta, and related functions . . . 2197
28.3.1 Riemann $\zeta(n)$ for integer arguments . . . 2197
28.3.2 Riemann $\zeta(1+x)$ . . . 2197
28.3.3 Dirichlet eta function for integer argument, $\eta(n)$ . . . 2197
28.3.4 Prime zeta function . . . 2198
28.3.5 Riemann prime counting function, $R(x)$ . . . 2198
28.3.6 Inverse of the Riemann prime counting function, $R^{-1}(x)$ . . . 2198
28.3.7 Rogers-Ramanujan continued fraction . . . 2198
28.3.8 Moebius function, $\mu(n)$ . . . 2199
28.3.9 q-Pochhammer Euler function, $(q)_\infty$ . . . 2199
28.4 Miscellaneous functions . . . 2200
28.4.1 Log-series distribution, pmf . . . 2200
28.4.2 Log-series distribution, cdf . . . 2200
28.4.3 Zeta distribution, pmf . . . 2200
28.4.4 Zeta distribution, cdf . . . 2201

29 Real functions related to ${}_0F_1$ **2202**
29.1 Bessel functions of integer order . . . 2202
29.1.1 Bessel function of the 1st kind, order 0, $J_0(x)$ . . . 2202
29.1.2 Bessel function of the 1st kind, order 1, $J_1(x)$ . . . 2202
29.1.3 Bessel function of the 1st kind, integer order n, $J_n(x)$ . . . 2202
29.1.4 Bessel function of the 2nd kind, order 0, $Y_0(x)$ . . . 2203
29.1.5 Bessel function of the 2nd kind, order 1, $Y_1(x)$ . . . 2203
29.1.6 Bessel function of the 2nd kind, integer order n, $Y_n(x)$ . . . 2203
29.2 Modified Bessel functions of integer order . . . 2205
29.2.1 Modified Bessel function of the 1st kind, order 0, $I_0(x)$ . . . 2205
29.2.2 Modified Bessel function of the 1st kind, order 1, $I_1(x)$ . . . 2205
29.2.3 Modified Bessel function of the 1st kind, integer order n, $I_n(x)$ . . . 2205

29.2.4 Modified Bessel function of the 2nd kind, order 0, $K_0(x)$ . . . 2205
29.2.5 Modified Bessel function of the 2nd kind, order 1, $K_1(x)$ . . . 2206
29.2.6 Modified Bessel function of the 2nd kind, integer order n, $K_n(x)$ . . . 2206
29.2.7 Exponentially scaled modified Bessel function of the 1st kind, order 0, $I_{0,e}(x)$ . . . 2206
29.2.8 Exponentially scaled modified Bessel function of the 1st kind, order 1, $I_{1,e}(x)$ . . . 2207
29.2.9 Exponentially scaled modified Bessel function of the 2nd kind, order 0, $K_{0,e}(x)$ . . . 2207
29.2.10 Exponentially scaled modified Bessel function of the 2nd kind, order 1, $K_{1,e}(x)$ . . . 2207
29.3 Integrals of zero-order Bessel functions . . . 2208
29.3.1 Integral of J_0 . . . 2208
29.3.2 Integral of Y_0 . . . 2208
29.3.3 Integral of I_0 . . . 2208
29.3.4 Integral of K_0 . . . 2208
29.4 Kelvin functions of order 0 . . . 2210
29.4.1 Kelvin function $\mathrm{ber0}(x)$ . . . 2210
29.4.2 Kelvin function $\mathrm{bei0}(x)$ . . . 2210
29.4.3 Kelvin function $\mathrm{ker0}(x)$ . . . 2210
29.4.4 Kelvin function $\mathrm{kei0}(x)$ . . . 2211
29.4.5 First derivative of the Kelvin function $\mathrm{ber0}, \mathrm{ber0}'(x)$ . . . 2211
29.4.6 First derivative of the Kelvin function $\mathrm{bei0}, \mathrm{bei0}'(x)$ . . . 2211
29.4.7 First derivative of the Kelvin function $\mathrm{ker0}, \mathrm{ker0}'(\nu, x)$ . . . 2212
29.4.8 First derivative of the Kelvin function $\mathrm{kei0}, \mathrm{kei0}'(x)$ . . . 2212
29.5 Synchrotron functions . . . 2213
29.5.1 First synchrotron function $F(x)$ . . . 2213
29.5.2 Second synchrotron function $G(x)$ . . . 2213

30 Real functions related to ${}_1F_1$ 2214
30.1 Incomplete gamma functions . . . 2215
30.1.1 Tricomi's entire incomplete gamma function: $\gamma^*(a, x)$ . . . 2215
30.1.2 Truncated exponential function, $e_n(x)$ . . . 2216
30.1.3 Relative exponential, $\mathrm{exprel}_n(x)$ . . . 2216
30.2 Coulomb wave functions . . . 2217
30.2.1 Normalizing Gamow constant for Coulomb wave functions . . . 2217
30.2.2 Coulomb phase shift $\sigma_L(\eta)$ . . . 2218
30.2.3 Regular Coulomb wave functions $F_l(\eta, x), F_l'(\eta, x)$ . . . 2219
30.2.4 Irregular Coulomb wave functions $G_l(\eta, x), G_l'(\eta, x)$ . . . 2220
30.3 Error function, and related functions . . . 2221
30.3.1 Exponentially scaled complementary error function, $\mathrm{erfcx}(x)$ . . . 2221
30.3.2 Difference of error functions, $\mathrm{erfh}(x, h) = \mathrm{erf}(x + h) - \mathrm{erf}(x - h)$ . . . 2221
30.3.3 Difference of error functions, $\mathrm{erf2}(x_1, x_2) = \mathrm{erf}(x_2) - \mathrm{erf}(x_1)$ . . . 2221
30.3.4 Probability function $Q(x) = \Phi(-x)$ . . . 2221
30.3.5 Inverse of the exponentially scaled complementary error function, $\mathrm{erfcx}^{-1}(x)$ . . . 2222
30.3.6 Inverse of the imaginary error function, $\mathrm{erfi}^{-1}(x)$ . . . 2222
30.3.7 Generalized Dawson integral, $F(p, x)$ . . . 2222
30.3.8 Generalized error function, $\mathrm{erfg}(p, x)$ . . . 2222
30.3.9 Expint3, $\mathrm{erfg}(3, x)$ . . . 2223
30.3.10 Scaled repeated integrals of the complementary error function, $i^n\mathrm{erfc}(x)$ . . . 2223
30.3.11 Fresnel auxiliary function $\mathrm{f}(x)$ . . . 2224
30.3.12 Fresnel auxiliary function $\mathrm{g}(x)$ . . . 2224
30.3.13 Goodwin-Staton integral, $G(x)$ . . . 2224
30.3.14 Voigt function U . . . 2225
30.3.15 Voigt function V . . . 2225
30.3.16 Voigt function H . . . 2225
30.3.17 Voigt Profile distribution, pdf . . . 2226
30.4 Exponential integrals, and related functions . . . 2227

30.4.1 Entire cosine integral $\mathrm{Cin}(x)$ . . . 2227
30.4.2 Entire hyperbolic cosine integral $\mathrm{Cinh}(x)$ . . . 2227
30.4.3 Entire exponential integral $\mathrm{Ein}(x)$ . . . 2227
30.4.4 Scaled exponential integral $\mathrm{e1s}(x)$ . . . 2227
30.4.5 Scaled exponential integral $\mathrm{eis}(x)$ . . . 2228
30.4.6 Scaled exponential integral $\mathrm{eisx2}(x)$ . . . 2228
30.4.7 Inverse of the exponential integral, $\mathrm{Ei}^{-1}(x)$ . . . 2228
30.4.8 Generalized exponential integral $E_p(p, x)$ . . . 2228
30.4.9 Exponential integral $\beta_n(x)$ . . . 2229
30.4.10 Inverse of the logarithmic integral, $\mathrm{li}^{-1}(x)$ . . . 2229
30.4.11 Shifted sine integral $\mathrm{si}(x) = \mathrm{Si}(x) - \pi/2$ . . . 2229
30.5 Struve functions . . . 2230
30.5.1 Struve function $\mathbf{H}_0(x)$ . . . 2230
30.5.2 Struve function $\mathbf{H}_1(x)$ . . . 2230
30.5.3 Struve function $\mathbf{L}_0(x)$ . . . 2230
30.5.4 Struve function $\mathbf{L}_1(x)$ . . . 2230

31 Real and complex functions **2232**
31.1 Additional elementary complex functions . . . 2232
31.1.1 Auxiliary function $\sqrt{1-z^2}$ . . . 2232
31.1.2 Cube root, $\mathrm{cuberoot}(x) = \sqrt[3]{x} = y$, with $\arg(y)$ closest to $\arg(x)$ . . . 2232
31.1.3 Nth root, $\mathrm{surd}(x, n) = \sqrt[n]{x} = y$, with $\arg(y)$ closest to $\arg(x)$ . . . 2232
31.1.4 Continuous inverse cotangent, $\mathrm{acotc}(x)$ . . . 2233
31.1.5 Continuous inverse hyperbolic cotangent, $\mathrm{acothc}(x)$ . . . 2233
31.2 Weierstrass elliptic functions, in terms of (real) lattice invariants g_2, g_3 . . . 2234
31.2.1 Weierstrass function $\wp_g(z, g_2, g_3)$ . . . 2234
31.2.2 Weierstrass function, first derivative: $\wp'_g(z, g_2, g_3)$ . . . 2236
31.2.3 Weierstrass Zeta function $\zeta_g(z, g_2, g_3)$ . . . 2238
31.2.4 Weierstrass Sigma function $\sigma_g(z, g_2, g_3)$ . . . 2239
31.2.5 Inverse Weierstrass function $\wp_g^{-1}(z, g_2, g_3)$ . . . 2240
31.3 Weierstrass elliptic functions, in terms of (real) lattice roots e_1, e_2 . . . 2241
31.3.1 Weierstrass function $\wp_e(x, e_1, e_2)$ . . . 2241
31.3.2 Weierstrass function $\wp'_e(x, e_1, e_2)$ . . . 2241
31.3.3 Weierstrass Zeta function $\zeta_e(z, e_1, e_2)$ . . . 2241
31.3.4 Weierstrass Sigma function $\sigma_e(z, e_1, e_2)$ . . . 2242
31.3.5 Inverse Weierstrass function $\wp_e^{-1}(y, e_1, e_2)$ . . . 2242

VII User library: numerical **2243**

32 Additional Classes **2244**
32.1 User defined functions based on multiple precision arithmetic (Python) . . . 2245
32.1.1 Overview . . . 2245
32.1.2 Double-precision arithmetic (`fpmlib`) . . . 2245
32.1.3 Binary floating-point in arbitrary-precision and with arbitrary exponent (`mpmlib`) . . . 2245
32.1.4 Interval arithmetic in arbitrary-precision and with arbitrary exponent (`ipmlib`) . . . 2245
32.1.5 Decimal floating-point in arbitrary-precision with limited exponent (`dpmlib`) . . . 2245
32.1.6 Rational numbers (quotients) in arbitrary-precision (`qpmlib`) . . . 2245
32.1.7 Binary floating-point in arbitrary-precision with limited exponent (`gpmlib`) . . . 2246
32.1.8 Binary balls in arbitrary-precision and with arbitrary exponent (`apmlib`) . . . 2246
32.2 User defined functions based on fixed precision arithmetic (C#) . . . 2247
32.2.1 Scalar functions with `Double` and `Complex`: `m53lib` and `m53libc` . . . 2248
32.2.2 Binary floating point, single precision: `slib` and `slibc` . . . 2249
32.2.3 Binary floating point, double precision: `dlib` and `dlibc` . . . 2250

32.2.4 Binary floating point, extended precision: elib and elibc . . . 2251
32.2.5 Binary floating point, quadruple precision: qlib and qlib . . . 2252
32.2.6 Binary floating point, octuple precision: olib and olibc . . . 2253
32.3 User defined functions based on arbitrary precision arithmetic (C#) . . . 2254
32.3.1 Binary floating point, multiple precision: mlib, mlibc . . . 2255
32.3.2 Binary floating point, single precision: sflib and sflibc . . . 2255
32.3.3 Binary floating point, double precision: dflib and dflibc . . . 2256
32.3.4 Binary floating point, extended precision: eflib and eflibc . . . 2257
32.3.5 Binary floating point, quadruple precision: qflib and qflibc . . . 2258
32.3.6 Binary floating point, octuple precision: oflib and oflibc . . . 2259
32.3.7 Binary floating point, multiple precision: mflib, mflibc . . . 2260
32.3.8 Binary arbitrary precision ball: aflib and aflibc . . . 2261
32.4 Scalar functions . . . 2262
32.4.1 Real and complex sine . . . 2262
32.4.2 Quadratic equation . . . 2263
32.4.3 Monic cubic equation . . . 2264
32.4.4 Cubic equation . . . 2265
32.4.5 Quartic equation . . . 2266
32.4.6 Transscribed from Julia: complex elliptic functions in double precision . . . 2267
32.4.7 Transscribed from Julia: complex hurwitz function, and related, in double precision . . . 2268
32.4.8 Speeedups for iterative algorithms which require an initial guess . . . 2269
32.4.9 Examples for matrix functions . . . 2270

33 Distribution functions **2271**
33.1 Distributions related to multiple comparisons of means . . . 2271
33.1.1 Normal maximum distribution, $\rho_{ij,i\neq j} = \rho$: pdf, cdf, qtf, boost class . . . 2271
33.1.2 Normal maximum modulus distribution, $\rho_{ij,i\neq j} = \rho$: pdf, cdf, qtf, boost class . . . 2273
33.1.3 Normal maximum distribution, $\rho_{ij,i\neq j} = \lambda_i\lambda_j$: pdf, cdf, qtf, boost class . . . 2274
33.1.4 Normal maximum modulus distribution, $\rho_{ij,i\neq j} = \lambda_i\lambda_j$: pdf, cdf, qtf, boost class . . . 2275
33.1.5 Normal range distribution: pdf, cdf, qtf, boost class . . . 2277
33.1.6 Studentized maximum distribution: pdf, cdf, qtf, boost class . . . 2278
33.1.7 Studentized maximum modulus distribution: pdf, cdf, qtf, boost class . . . 2279
33.1.8 Dunnett t-distribution, 1-sided: pdf, cdf, qtf, boost class . . . 2280
33.1.9 Dunnett t-distribution, 2-sided: pdf, cdf, qtf, boost class . . . 2281
33.1.10 Nair t-distribution: pdf, cdf, qtf, boost class . . . 2282
33.1.11 Halperin t-distribution: pdf, cdf, qtf, boost class . . . 2283
33.1.12 Studentized range distribution: pdf, cdf, qtf, boost class . . . 2284
33.2 Discrete (lattice) distribution functions related to (stratified) rank tests . . . 2285
33.2.1 Kendall S (or tau) distribution (under H_0), pmf vector . . . 2285
33.2.2 Mann-Whitney U distribution (under H_0), pmf vector . . . 2285
33.2.3 Jonckheere-Terpsta S distribution (under H_0), pmf vector . . . 2285
33.2.4 Spearman ρ distribution (under H_0), pmf vector . . . 2285
33.2.5 Sign test distribution (under H_0), pmf vector . . . 2286
33.2.6 Wilcoxon T distribution (under H_0), pmf vector . . . 2286
33.2.7 Page L distribution (under H_0), pmf vector . . . 2286
33.2.8 Quade L distribution (under H_0), pmf vector . . . 2287
33.3 Discrete (non-lattice) distribution functions related to rank tests . . . 2288
33.3.1 Cochran S distribution (under H_0), pmf vector . . . 2288
33.3.2 Friedman S distribution (under H_0), pmf vector . . . 2288
33.3.3 Quade S distribution (under H_0), pmf vector . . . 2288

34 Inferential statistics **2289**
34.1 Basic classical statistical tests (stratified) . . . 2289
34.1.1 Student t-test for 1 sample: tests (p-values) and confidence intervals . . . 2289

34.1.2 Student t-test for 1 sample: power calculations . . . 2292
34.1.3 Student t-test for 1 sample: sample size calculation . . . 2294
34.1.4 Chi-squared-test for the variance of 1 sample: tests (p-values) . . . 2295
34.1.5 Chi-squared-test for the variance of 1 sample: confidence intervals . . . 2296
34.1.6 Chi-squared-test for the variance of 1 sample: power . . . 2297
34.1.7 Chi-squared-test for the variance of 1 sample: sample size . . . 2298
34.2 Basic classical statistical tests for 2 independent samples (stratified) . . . 2299
34.2.1 Student t-test for 2 independent samples: tests (p-values) . . . 2299
34.2.2 Student t-test for 2 independent samples: confidence intervals . . . 2301
34.2.3 Student t-test for 2 independent samples: power . . . 2303
34.2.4 Student t-test for 2 independent samples: sample size calculation . . . 2305
34.2.5 Student t-test for 2 independent samples: power and sample size: failure to stratify . . . 2307
34.2.6 Student t-test for 2 independent samples: equivalence and non-inferiority . . . 2308
34.2.7 F-test for the variances of 2 independent samples: tests (p-values) . . . 2309
34.2.8 F-test for the variances of 2 independent samples: confidence intervals . . . 2311
34.2.9 F-test for the variances of 2 independent samples: power . . . 2313
34.2.10 F-test for the variances of 2 independent samples: sample size . . . 2315
34.3 Basic classical statistical tests for 2 correlated samples (stratified) . . . 2317
34.3.1 Student t-test for 2 correlated samples: tests (p-values) . . . 2317
34.3.2 Student t-test for 2 correlated samples: confidence intervals . . . 2319
34.3.3 Student t-test for 2 correlated samples: power . . . 2321
34.3.4 Student t-test for 2 correlated samples: sample size calculation . . . 2323
34.3.5 Morgan-Pitman test for the variances of 2 correlated samples: tests (p-values) . . . 2325
34.3.6 Morgan-Pitman test for the variances of 2 correlated samples: confidence intervals . . . 2327
34.3.7 Morgan-Pitman test for the variances of 2 correlated samples: power . . . 2329
34.3.8 Morgan-Pitman test for the variances of 2 correlated samples: sample size . . . 2331
34.3.9 Pearson's rho, 2 correlated samples: tests (p-values) . . . 2333
34.3.10 Pearson's rho, 2 correlated samples: confidence intervals . . . 2335
34.3.11 Pearson's rho, 2 correlated samples:: power . . . 2337
34.3.12 Pearson's rho, 2 correlated samples:: sample size calculation . . . 2339

VIII User library: graphics 2341

35 Distribution functions 2342
35.1 Introduction to 2D functions plots . . . 2342
35.1.1 Plots of continuous distribution functions . . . 2342
35.1.2 Plots of discrete distribution functions . . . 2345

36 Complex functions 2347
36.1 Additional complex functions . . . 2348
36.1.1 Elliptic K . . . 2348

IX Back matter 2349

37 License and History 2350
37.1 Mozilla Public License Version 2.0 . . . 2350
37.2 History . . . 2356

Bibliography 2357

Index 2431

Part I

Getting started

CHAPTER

ONE

SETUP AND GENERAL USAGE

XlCalcNet is a numerical library with parts written in Python and other parts written in C#, C++, C and Pascal, with focus on numerical calculations in multiple precision and data visualisation.

Since one of the main goals is to give access to software written in Python (or via PythonNet software written in C#) from within spreadsheet formulas, a dedicated CPython installation is strongly recommend, to make it easier to configure the interaction with Microsoft Excel and/or LibreOffice Calc, without disturbing existing Python installations.

The interaction with LibreOffice Calc is based on the fact that LibreOffice has built-in support for writing spreadsheet functions (which appear in the Function Wizard) in Python.

The interaction with Microsoft Excel is achieved by running a socket server written in Python, which is called from spreadsheet formulas using the functionality provided by Excel.Dna.

On the Python side XlCalcNet uses Mpmath 4.0 to provide a rich set of functions in arbitrary precision, using not only Mpmath's binary and interval data types, but also Python's built-in Decimal and Fraction data types. If GMP2 is installed, its data types can be used in many cases instead of Mpmath's binary data types, being much faster. Likewise, if Python-Flint is installed, its data types can be used in many cases instead of Mpmath's interval data types, being much faster, and often also more accurate.

On the C/C++ side, XlCalcNet uses DAMath, Boost Math, Boost Multiprecision and Eigen to provide numerical functions in single, double, extended, quadruple and octuple precision, which are available to the user both from C# and Python.

The XlCalcNet2 library, which is licensed under the LGPL-3.0 and is therefore provided as a separate project, is based on Boost Math, Boost Multiprecision, Eigen, GMP, MPFR, MPC and Flint and provides additional functions for the same fixed-precision data types as XlCalcNet and also in arbitrary precision.

XlCalcNet is intended to be used together with existing Python libraries like NumPy, Matplotlib, Pandas, SciPy. It can also be used from recent versions of RStudio and R, using the reticulate package.

1.1 Setting up XlCalcNet

1.1.1 Installing XlCalcNet

1.1.1.1 Installing a version of Python which is dedicated to MS Excel and/or LibreOffice Calc

The XlCalcNet package is compatible with Python versions 3.8 - 3.15. It is recommended to use a version of CPython which is both mature and supported. In this manual, we will use Python version 3.13.15.

> **Important**
>
> However, if we want to use XlCalcNet with LibreOffice (version 7.0 or later, 64 bit), we need to make sure that the version which we install is compatible with the main Python core installation of LibreOffice, which means that the first 2 numbers of the version number `x.y.z` must match.

To find out which version this is, follow these steps: In the folder `C:\Program Files\LibreOffice\program` double-click on `python.exe`. This opens the python console, which in the first line states which Python version is used. At the time of writing (October 2026), the most current version of LibreOffice is version 26.8, which uses Python version 3.13.15. Earlier version of LibreOffice use different versions of Python.

Python can be installed in different ways. It is recommended to install Python as a dedicated version for use with MS Excel and/or LibreOffice Calc, which does not require uninstalling any previous versions of Python, does not interfere with other Python installations, and does not require administrative privileges.

As installation folder any folder can be used for which the user has read and write access. In the following we will assume that the installation folder is `C:\Python313`. We assume that the path to this folder is NOT added to the PATH environment variable.

As an example of the installation process, we download the windows installer of version 3.13.15 (64 bit) from https://www.python.org/downloads/release/python-31315/. We will install this version into the folder `C:\Temp`, so we make sure beforehand that this folder exists and is empty.

In the Download folder, double-click on the downloaded file `python-3.13.15-amd64.exe` to start the installation. Click on "Customize installation".

In the next dialog page "Optional Features" unselect "Documentation", "Python test suite", "py launcher" and "for all users". Click on "Next".

In the next dialog page "Advanced Options" unselect all items, including "Create shortcuts". In the line below "Customize install location" enter `C:\Temp`. Click on `Install`. Once the installation has been completed, the last dialog page "Setup was successful" appears. Click on `Close`.

In the Windows explorer, navigate to the folder `C:\Temp`. Open another Windows explorer instance, navigate to `C:\`, and create the folder `C:\Python313`. Copy (DO NOT move!) the content of `C:\Temp` into `C:\Python313`.

Click the Start button and open **Settings** (or press `Windows Key + I`). Click on **Apps** on the left menu. Select **Installed apps** to view and manage all programs installed on your PC. Navigate to "Python 3.13.15 (64-bit)" and click on the `...` symbol. Select `Uninstall` and confirm. Once Uninstall has been completed, the last dialog page "Uninstall was successful" appears. Click on `Close`.

Now the folder `C:\Temp` can be deleted. The folder `C:\Python313` contains the ("freestanding") Python version that we will work with in this manual.

1.1.1.2 First steps with XlCalcNet

Since the path to `C:\Python313` is not added to the PATH environment variable, it is convenient to prepare, within the `C:\Python313` folder, batch files for the installation of the Python packages which one wishes to install and possibly update later.

The first batch file will ensure that our version of pip is up to date. Right-click over any free space in the `C:\Python313` folder, select `New -> Text Document`. Rename this document to `A01upgradepip.bat`, ignoring the warning. Open this file for editing in the

Windows standard editor. Type `python -m pip install -U --no-warn-script-location pip` as install command, begin a new line, type `pause`, save the file, and close the editor.

Now double-click on `A01upgradepip.bat`. This will update pip as needed.

In the same way prepare a batch file named `A02installxlcalcnet.bat`, using `python -m pip install -U xlcalcnet` as install command. Double-click on `A02installxlcalcnet.bat`. This will install the latest version of the XlCalcNet package.

To check that this installation was successful, double-click on `python.exe`, which will open the python console:

```
>>> from xlcalcnet import mpm
>>> mpm.dps = 40
>>> mpm.sqrt(2)
    mpf('1.4142135623730950488016887242096980785696 62')
```

1.1.1.3 The 3 folders which are used by XlCalcNet

XlCalcNet uses the following 3 folders:

- The installation folder (in our example `C:\Python313\Lib\site-packages\xlcalcnet`), which contains the program code of the package (which should not be changed by the user during normal use). This folder is automatically created during the installation of XlCalcNet.
- The `DataXlCalcNet` folder in the Documents folder, which contains code examples and data which are directly manipulated by the user. This folder needs to created manually by the user (see below).
- The `XlCalcNetIDE` folder in the AppDataLocal folder, which contains data or executable binaries which are generated as a result of running a Python script or C# program. This folder is automatically created while using XlCalcNet.

Important

It is crucial for the correct functioning of XlCalcNetThat that the `DataXlCalcNet` folder is located in the Documents folder. The `DataXlCalcNet` folder has already been downloaded during the installation of XlCalcNet; it is located in the installation folder (in our example `C:\Python313\Lib\site-packages\xlcalcnet`). **The user needs to copy it manually from the installation folder into the Documents folder**.

1.1.1.4 Preparing the use of additional installation batch files for installing Python packages

The `DataXlCalcNet` folder contains a number of batch files for installing additional Python packages which can be used with XlCalcNet. These batch files are located in the subfolder `\PythonBatchFiles\BatchFiles`. The user can copy these batch files into the `C:\Python313` folder, and double-click on them to install the corresponding Python packages. The user can also create new batch files for installing additional Python packages, using the same procedure as described above for creating the batch files `A01upgradepip.bat` and `A02installxlcalcnet.bat`.

1.1.2 Installing and using Python.NET: Calling C# from Python

Python.NET is a package that gives Python programmers nearly seamless integration with the .NET Common Language Runtime (CLR) and provides a powerful application scripting tool for .NET developers. It allows Python code to interact with the CLR, and may also be used to embed Python into a .NET application.

See https://github.com/pythonnet/pythonnet

Assuming that the Python batch files have been installed as described *here*, Python.NET can be installed by double-clicking on the batch file `A03installpythonnet.bat` in the `C:\Python313` folder.

To check that this installation was successful, double-click on `python.exe`, which will open the python console. With Python.NET installed, we can access the C# module `qreal`, which provides functions in quadruple precision:

```
>>> from xlcalcnet import qreal
>>> qreal.sqrt(2)
    qreal('1.41421356237309504880168872420969')
```

Soem background information:

.NET Framework is part of the Microsoft Windows operating system since Windows Vista; .NET Framework 4.x can be installed since Windows XP. Recent versions of Windows (Windows 7 - Windows 11), which have been kept fully maintained, have .NET Framework 4.8 installed, which is the latest version of .NET Framework and will continue to be distributed with future releases of Windows. As long as it is installed on a supported version of Windows, .NET Framework 4.8 will continue to also be supported (see https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-framework).

As part of the .NET Framework 4.x runtime, 3 different compilers are provided: csc.exe (C#), vbc.exe (Visual Basic), and jsc.exe (JScript).

The 32 bit targeting versions are located in `C:\Windows\Microsoft.NET\Framework\v4.0.30319` and those for 64 bit in `C:\Windows\Microsoft.NET\Framework64\v4.0.30319`.

It is important to note that these compilers are not part of Visual Studio but part of Windows; they are, in a way, the closest to a compiler as part of the operating system that Windows has ever come up with. On the other hand, these compilers have been tugged away with the rest of the .NET Framework 4.x runtime; in that sense, they are "hidden" compilers.

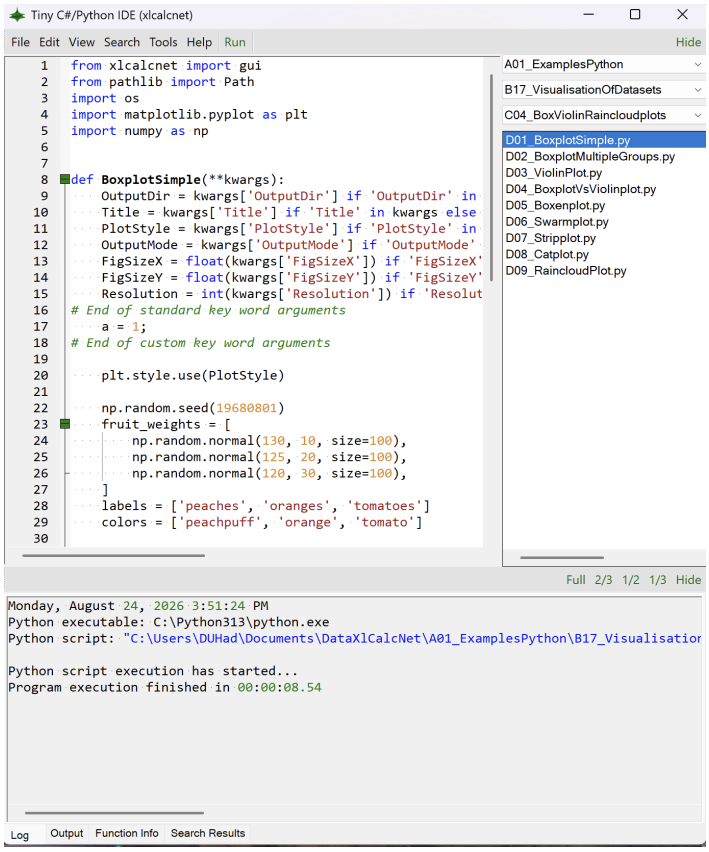
In the following, we will ignore the Visual Basic and JScript compilers, but focus only on C#. The C# compiler supports only language versions up to C# 5. Luckily, this still provides us with all language features which we need for our purposes.

In terms of usability, the .NET Framework 4.x runtime does not include an IDE; we therefore include a Tiny IDE as described below.

1.1.3 Installing and using the Tiny IDE as a Python application

1.1.3.1 Installation

With the `DataXlCalcNet` folder in the Documents folder and Python.NET installed, we can start using the "Tiny C#/Python IDE", which makes it easy to edit and run small programs written in Python or C#. The Tiny IDE is a Python program (it is started with `pythonw.exe`) which loads a UserControl written in C#. This is made possible by Python.NET.



Follow these steps to make the Tiny IDE available:

- In the Python installation folder, rightclick on `pythonw.exe`.
- Select `Create shortcut` -> Result: `pythonw.exe-Shortcut`. (On Windows 11: Select `Show more options` -> `Create shortcut` -> Result: `pythonw.exe-Shortcut`.)
- Rightclick on `pythonw.exe-shortcut`; Select Properties.
- In the dialogue Properties, select "Target", and type:`C:\Python313\pythonw.exe C:\Users\DUHad\Documents\DataXlCalcNet\A01_ExamplesPython\B01_GeneralUsage\C01_Setup\D03_ShowEditor.py`. Here the path to the documents folder (in our case `C:\Users\DUHad\Documents`) needs to be changed to meet the settings of your system. Then save.
- Rename `pythonw.exe-Shortcut` to `TinyIDE_Python313`
- Doubleclick on `TinyIDE_Python313`
- In the task-bar, right-click on the appearing Python symbol, and select "Pin to taskbar"

From now on, you can start the Tiny IDE by clicking on this Python symbol on the taskbar. To start an additional instance of the Tiny IDE when the Tiny IDE is already running, click in the main menu on `Tools` -> `Tiny IDE (external)`. Note that the Python source code for starting the Tiny IDE is also the Python code example which is shown after starting the Tiny IDE (in `A01_ExamplesPython\B01_GeneralUsage\C01_Setup\D03_ShowEditor.py`), so clicking on `Run` will also start an additional instance. However, now

the program does not return until it is closed, as it is not running in an additional instance of Python; running the program like this is useful for debugging and testing purposes, like modifying title, width and height of the window, which can be changed on lines 67-68.

1.1.3.2 Overview

The Tiny IDE is designed around the structure of the `DataXlCalcNet` folder, making it easy to navigate and manage existing code examples, as well a new code written by the user. The Tiny IDE can be used only to edit and run Python or C# code examples which are located in the `DataXlCalcNet` folder and are located in subfolders beginning with `A0`. The code examples are organized in a tree structure, which is shown in the top right panel (the "folder panel") of the Tiny IDE.

The top left panel (the "editor panel") of the Tiny IDE shows the code of the selected file, which can be edited and run. The Tiny IDE also provides features like syntax highlighting, autocompletion, bookmarks, code folding, and error highlighting to make coding easier.

The output of the program is shown in the bottom panel (the "output panel") of the Tiny IDE, which has 4 tabs: the "Log" tab, which documents the progress of compiling and running the code (and also displays warning and error messages); the "Output" tab, which shows the text output of the program (if any); the "Function Info" tab, which provides information about the functions (if they belong to a module from XlCalcNet); and the "Search Results" tab, which shows the search results.

Each file in the `DataXlCalcNet\A0*` folders can be opened in the Tiny IDE by clicking on it. Per Tiny IDE instance, only one file can be opened, edited and run at a time (to open an additional file, you need to start a new instance of the Tiny IDE: `Tools -> Tiny IDE (external)`). The Tiny IDE will load this file into the editor, and the code can be edited and run. To run the code, click on the "Run" button in the main menu. The output of the program (if any) is shown in the output tab of the output panel.

The ordering and naming of the code examples in the subfolders `A01_ExamplesPython` and `A02_ExamplesCSharp` matches the ordering and naming of the corresponding chapters, sections and subsections in this manual, making it easy to find the code example which corresponds to a specific section of the manual. In addition, the "section help" feature (`Help -> Section Help`), which is available in the main menu of the Tiny IDE and opens the corresponding section of this manual in a web browser, depends on this ordering and naming.

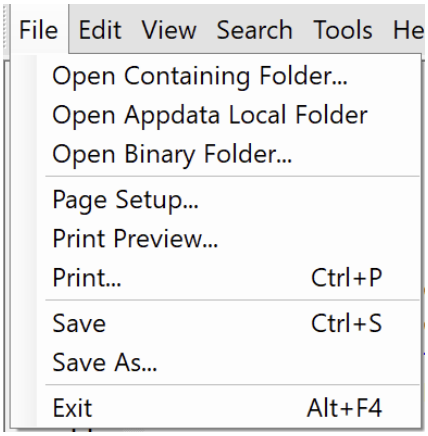
Also, in Python, the feature that code examples can be called like modules, using statements like `from A01_ExamplesPython.B17_VisualisationOfDatasets.C01_BarCharts import D01_BarChartPart1`, depends on keeping this ordering and naming intact.

Therefore the names of these folders and files should not be changed, and new code examples should be added to the appropriate folders, either `A03_UserlibPython` or `A04_UserlibCSharp` for user-defined libraries, or `A05_UserExamplesPython` or `A06_UserExamplesCSharp` for code examples which demonstrate the use of existing or user-defined libraries. The Tiny IDE will automatically detect new files and folders, and will update the tree structure accordingly.

1.1.3.3 The menu bar of the Editor

The Menubar of the Tiny IDE contains the following menus: `File`, `Edit`, `View`, `Search`, `Tools`, and `Help`. Each menu contains a number of items, which are described below.

There are also two buttons (with green foreground color) in the menu bar: the "Run" button, which runs the code of the current file, and the "Show/Hide" button, which toggles the visibility of the folder panel.



In the `File` menu, two standard items are missing: "New" and "Open".

The "New" item is missing because the Tiny IDE is designed to work within the `DataXlCalcNet` folder, and therefore new files which are not in the current folder should be created in the appropriate subfolders of `DataXlCalcNet`, which often have to be created manually first using the Windows Explorer.

There is, however, a `Save As` item in the file menu, which allows to save the current file under a new name in the current folder. This is useful for creating a copy of an existing file, and using this as a starting point for a new file.

The "Open" item (which would typically open a dialog box with the functionally of the Windows Explorer) is missing because the Tiny IDE is limited to the `DataXlCalcNet` folder. Instead the folder panel allows to navigate through the `DataXlCalcNet` folder and open any file in it.

Renaming or deleting files and folders is possible, but should be done with care, using the Windows Explorer, and not using the Tiny IDE. After these changes, when returning to the Tiny IDE, the tree structure will not be updated automatically, but has to be updated manually by clicking on the appropriate parent folder in the tree structure.

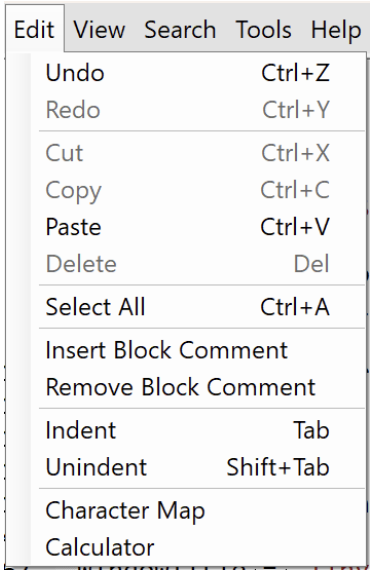
`Open Containing Folder` opens the Windows Explorer in the folder of the current file. This is useful for quickly accessing other files in the same folder, or for deleting files and folders.

`Open Appdata Local Folder` opens the Windows Explorer in the `AppData\Local\XlCalcNetIDE` folder. This is useful for quickly accessing files and folders which are generated by running Python or C# code examples in the Tiny IDE.

`Open Binary Folder` opens the Windows Explorer in the `Bin` subfolder of the `XlCalcNet` installation. This is useful mostly for debugging purposes.

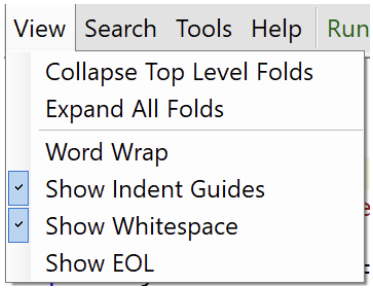
`Page Setup` opens the standard Windows dialog for setting up the page layout for printing. `Print Preview` opens a preview of the current file, which can be printed from there. `Print` opens the standard Windows dialog for printing the current file.

`Save` will just save the current file without any further action. `Save As` has been descibed above. `Exit` will close the Tiny IDE.



In the `Edit` menu, `Undo`, `Redo`, `Cut`, `Copy`, `Paste`, `Delete`, and `Select All` are standard items. `Insert BlockComment` and `Remove BlockComment` allows to comment or uncomment a block of code, which is useful for testing purposes (these 2 items are available in the context menu as well). `Indent` and `Unindent` allow to indent or unindent a block of code, which is useful for formatting purposes.

`Character Map` opens the standard Windows dialog for selecting special characters, which can be inserted into the code. `Calculator` opens the standard Windows calculator, which can be used for quick calculations.



In the `View` menu, `Collapse Top Level Folds` allows to collapse all top level folds in the code, which is useful for quickly navigating through the code.

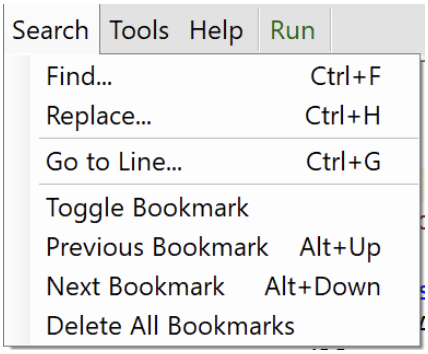
`Expand All Folds` allow to collapse or expand all folds in the code, which will make all code visible

`Show Indentation Guides` toggles the visibility of indentation guides in the editor panel.

`Word Wrap` toggles word wrap in the editor panel.

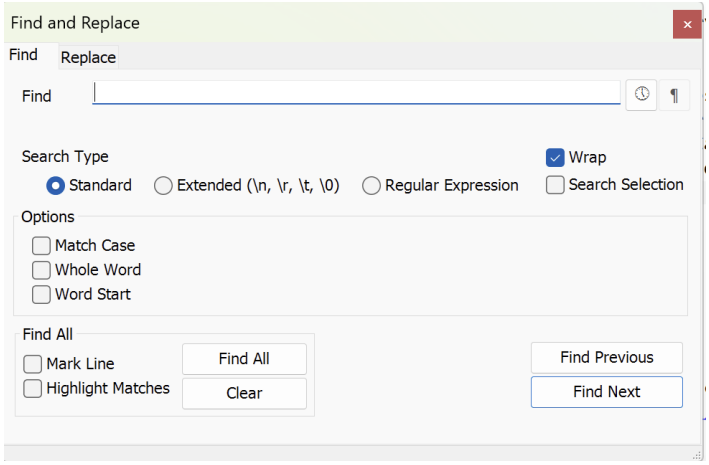
`Show Whitespace` toggles the visibility of whitespace characters in the editor panel.

`Show EOL` toggles the visibility of end of line markers in the editor panel.

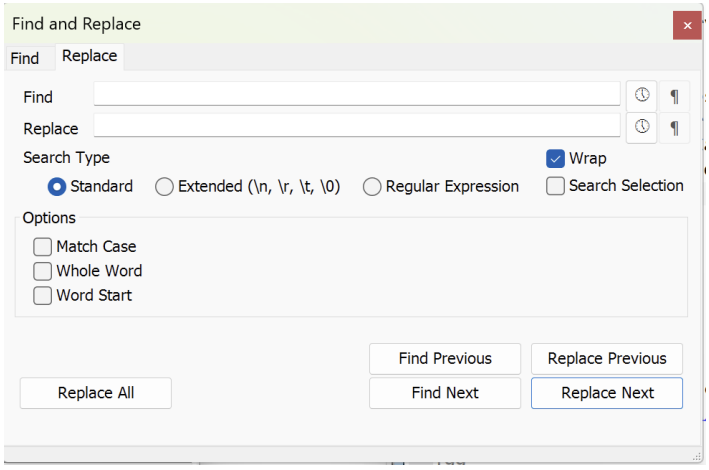


In the `Search` menu, `Find` starts the search for a string in the code, `Replace` allows to replace a string in the code, and `Goto` allows to jump to a specific line number in the code.

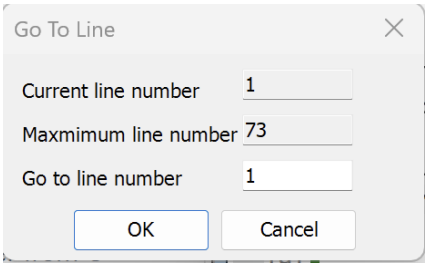
The corresponding dialog boxes are shown below:



If the string is found, the first occurrence is highlighted in the code. If the string is not found, a message box appears which informs that the string was not found. The search can be continued by clicking on `Find Next` or `Find Previous`. The search can be limited to the current selection, and it can be case sensitive or insensitive. The search can also be done using regular expressions.

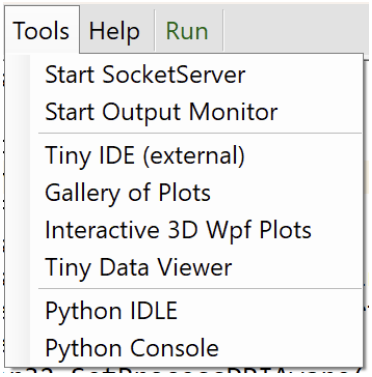


If the string is found, the first occurrence is highlighted in the code. If the string is not found, a message box appears which informs that the string was not found. The search can be continued by clicking on `Find Next` or `Find Previous`. `Replace Next`, `Replace Previous` or `Replace All` can be used to decide what to do with the found string. The search can be limited to the current selection, and it can be case sensitive or insensitive. The search can also be done using regular expressions.



if the line number is valid, the cursor jumps to the specified line number. If the line number is invalid, a message box appears which informs that the line number is invalid.

`Toggle Bookmark` allows to set or delete a bookmark in the code, which can be used to quickly navigate through the code. `Delete All Bookmarks` deletes all bookmarks in the code. `Next Bookmark` jumps to the next bookmark in the code. `Previous Bookmark` jumps to the previous bookmark in the code.



In the `Tools` menu, `Start Socket Server` starts the socket server, see *here* for details.

`Start Output Monitor` starts the output monitor application, see *here* for details.

`Tiny IDE (external)` starts an additional instance of the Tiny IDE, which can be used to edit and run another file.

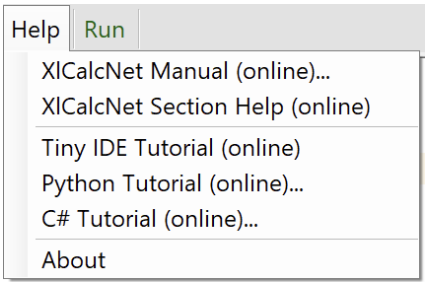
`Gallery of Plots` starts the gallery of plots application, see *here* for details.

`Interactive 3D Wpf Plots` starts the Interactive 3D Wpf Plots application, see *here* for details.

`Tiny Data Viewer` starts the Tiny Data Viewer application, see *here* for details.

`Python IDLE` starts the Python IDLE application.

`Python Console` starts the Python console.



In the `Help` menu, `XlCalcNet Manual (online)...` displays the start page of the XlCalcNet online manual in a web browser.

`XlCalcNet Section Help (online)...` displays the section of the XlCalcNet.online manual which corresponds to the current file in a web browser.

`Tiny IDE Tutorial (online)...` displays the section of the XlCalcNet.online manual which explains the use of the Tiny IDE in a web browser.

`Python Tutorial (online)...` displays the online tutorial for Python, which is available on the official Python website in a web browser.

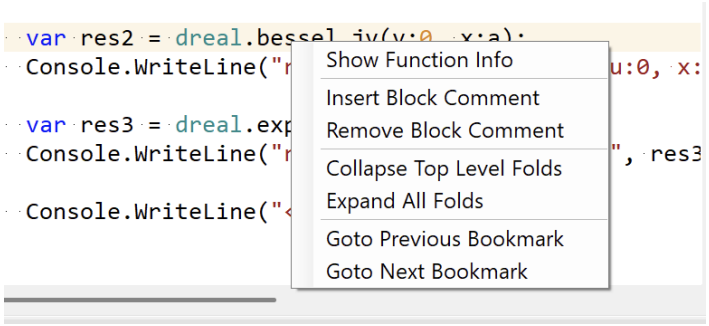
`C\# Tutorial (online)...` displays the online tutorial for C#, which is available on the official C# website in a web browser.

`About XlCalcNet...` displays information about the XlCalcNet package, including version number and copyright information.

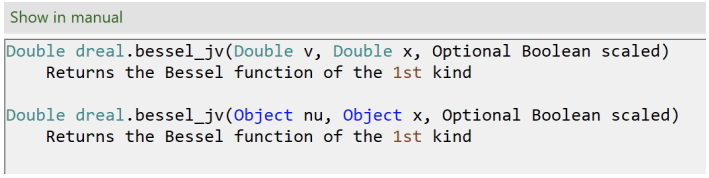
1.1.3.4 The context menu of the editor

Right-clicking in the editor panel opens a context menu, which provides quick access to common actions like `Insert Block`, `Remove Block Comment`, `Collapse Top Level Folds`, `Expand All Folds`, `Goto Previous Bookmark`, `Goto Next Bookmark`, all of which are also available in the main menu.

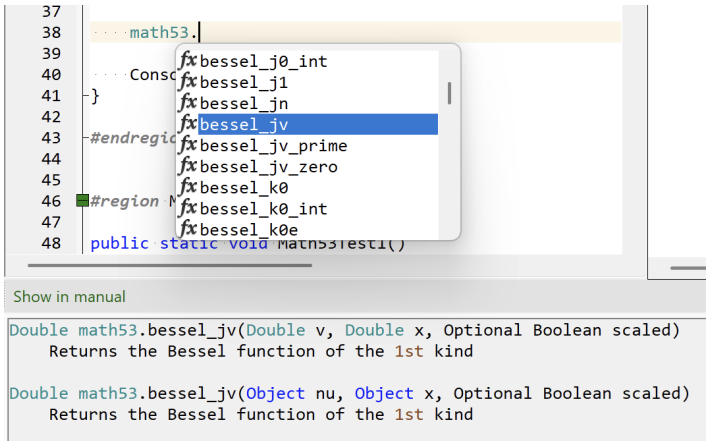
New in this context menu is the item `Show Function Info`, which for functions which belong to a module from XlCalcNet, displays information about the function in the "Function Info" tab of the output panel.



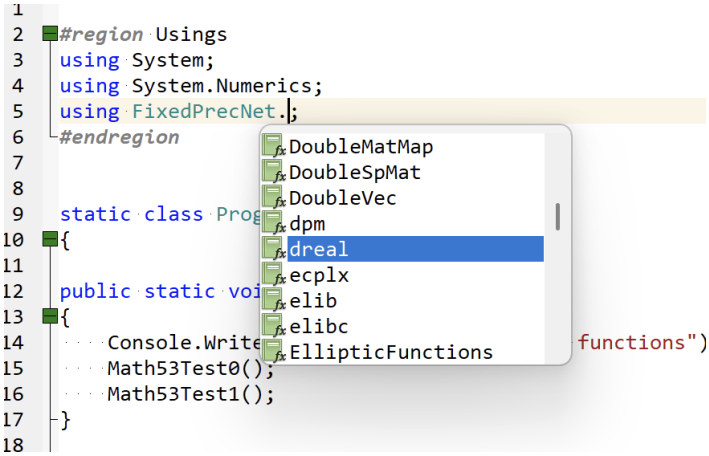
In the screenshot above, the context menu is shown after right-clicking on the function `dreal.bessel_jv` (with the curser to the right of the dot). The item `Show Function Info` is highlighted, and the corresponding information about this function is shown in the "Function Info" tab of the output panel below.



Just to the upper left of the `Show Function Info` item, the button `Show in Manual` is visible. Clicking on this button opens the corresponding section of the XlCalcNet online manual in a web browser.



This works in combination with the autocomplete feature, which is activated by pressing `Ctrl + Space`. The autocomplete feature shows a list of functions which belong to a module from XlCalcNet, and for each function, the corresponding information is shown in the "Function Info" tab of the output panel. This allows to quickly find the function which is needed, and to see how it can be used.



The autocomplete feature also works for user-defined libraries, however in these cases no additional information is shown in the "Function Info" tab of the output panel.

1.1.3.5 The Python development cycle in the Tiny IDE

Here we describe the Python development cycle in the Tiny IDE for producing and running python programs. The process for producing a Python library is similar but different, and is described *here*.

Overall, the Python development cycle in the Tiny IDE consists of the following steps: The file is opened in the editor panel, and the code is edited as needed. The code can be run by clicking on the "Run" button in the main menu bar. If there are any errors or warnings, they are shown in the "Log" tab of the output panel. When clicking on the error messages in the "Log" tab, they are also highlighted in the code. After correcting the errors, the "Run" button can be clicked again. If there are no errors, the code is is executed. Any text output of the program is shown in the "Output" tab of the output panel after the program has finished running.

These steps are illustrated in the following screenshots, which show the Python development cycle in the Tiny IDE for producing and running python programs.

```
  1
  2  # See: https://python-graph-gallery.
  3
  4  from xlcalcnet import gui
  5  from pathlib import Path
  6  import os, time
  7  import seaborn as sns
  8  import pandas as pd
  9
 10
 11 def KdePlot3(**kwargs):
 56 try:
 64 except Exception:
```

Directly after loading the Python code example, the code is folded, and the "Log" tab of the output panel is empty. The "Output" tab of the output panel is also empty, since the program has not been run yet. The unfolded code looks like this:

```
# See: https://python-graph-gallery.com/80-contour-plot-with-seaborn/

from xlcalcnet import gui
from pathlib import Path
import os, time
import seaborn as sns
import pandas as pd

def KdePlot3(**kwargs):
    OutputDir = kwargs['OutputDir'] if 'OutputDir' in kwargs else 'OutputMonitor'
    Title = kwargs['Title'] if 'Title' in kwargs else 'KdePlot3'
    PlotStyle = kwargs['PlotStyle'] if 'PlotStyle' in kwargs else 'default'
    OutputMode = kwargs['OutputMode'] if 'OutputMode' in kwargs else 'svg'
    FigSizeX = float(kwargs['FigSizeX']) if 'FigSizeX' in kwargs else 4
    FigSizeY = float(kwargs['FigSizeY']) if 'FigSizeY' in kwargs else 4
    Resolution = int(kwargs['Resolution']) if 'Resolution' in kwargs else 300
# End of standard key word arguments

    # Choose 'Agg' as backend unless we want to display the graphics
    if (OutputMode != 'gui'):
        import matplotlib
        matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.style.use(PlotStyle)

    fn = gui.get_dataxlcalcnet()
    fn += r'\DataExamples\MainExamples\Workbooks\Datasets.xlsx'
    datasets = pd.ExcelFile(fn)
    df = pd.read_excel(datasets, 'iris')

    sns.set_style('white')
    sns.kdeplot(x=df.sepal_width, y=df.sepal_length,
                cmap='Blues', fill=True, bw_adjust=0.5)

    fig = plt.gcf()
    fig.tight_layout()


# Start of output choices
    if (OutputMode == 'plt'):
        plt.show()
    elif (OutputMode == 'gui'):
        gui.plot(fig, __file__, Title)
    else:
```

(continues on next page)

(continued from previous page)

```
        FName = 'Temp'
        if OutputDir != 'Temp': FName = (Path(__file__).stem)
        LocalDir = gui.get_local_appdata_xlcalcnet()
        FullPath = os.sep.join([LocalDir, OutputDir, FName + '.' + OutputMode])
        plt.savefig(FullPath,  bbox_inches='tight')
        if OutputDir != 'Temp': print('Graphics written to: ', FullPath)
    plt.close('all')

try:
    if __name__ == '__main__':
        start0 = time.time()
        KdePlot3(OutputMode = 'gui')
        end0 = time.time()
        print('Elapsed time:', format(end0 - start0, '.4g'), 'seconds' )


except Exception:
    import traceback
    print(traceback.format_exc())
```

Folding in Python works on code blocks which are defined by indentation. At the top of the file are the import statements, followed by a `def funcname(**kwargs)` statement, where "funcname" in this case is `KdePlot3`. The body of this function is the place where "the real work" is done.

This function also has two standardized parts (for Matplotlib graphic code examples), one at the beginning, which takes care of the keyword arguments and ensures that the 'Agg' backend is used unless one wants to display the graphics (this is necessary to ensure this function can be called from older versions of Libreoffice like 7.4); and another at the end ("# Start of output choices"), which makes sure that interactive use, output to the OutputMonitor folder, or output to another folder is possible in verious formats (like jpg, png, svg, pdf).

At the end of the file, the `try ... except` block makes sure that exceptions are dealt with properly, that the time for actually running the code (as opposed to the time required to load and initialize the imported libraries) is printed, and that we have the chance to call the function with a specific set of keyword arguments (in this case `OutputMode = 'gui'`).

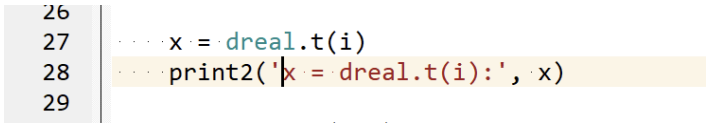
Python files with this overall structure are well suited for calls from within spreadsheet functions.

```
  1
  2 <H1 Title="General assignments and conversions">
 17 <H1 Title="Functions with argument conversion">
 32 <H1 Title="Arithmetic operators with dreal">
 45 <H1 Title="Arithmetic operators with floats">
 57 <H1 Title="Arithmetic operators with (long) integers">
 70 <H1 Title="Arithmetic operators with (short) integers">
 81 <H1 Title="Arithmetic comparisons with dreal">
 98 <H1 Title="Arithmetic comparisons with floats">
114 <H1 Title="Arithmetic comparisons with (long) integers">
131 <H1 Title="Arithmetic comparisons with (short) integers">
147 <H1 Title="Assignments to matrices">
149
```

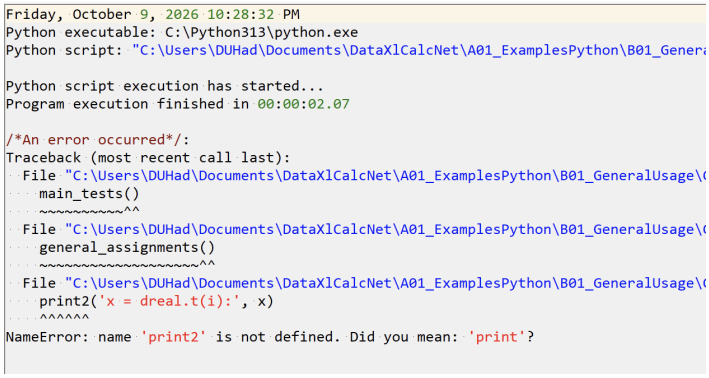
The output of the program is shown in the "Output" tab of the output panel after the program has finished running. Lengthy text output can be folded and unfolded, and the context menu allows to copy the output to the clipboard, clear the output, or save the output to a file.

```
Saturday, October 10, 2026 7:49:52 AM
Python executable: C:\Python313\python.exe
Python script: "C:\Users\DUHad\Documents\DataXlCalcNet\

Python script execution has started...
Program execution finished in 00:00:07.33
```

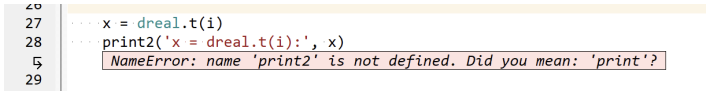
The "Log" tab of the output panel contains additional information after successfully running the program.



The snapshot above shows a code snippet which contains an error (`print` is misspelled as print2``)



This error is detected by the Python interpreter, and is reported in the Log tab of the output panel.



After double-clicking in the Log tab on the first line in red color, the error is also indicated in the editor, with a full error message below the line where the error occured.

1.1.3.6 The CSharp development cycle in the Tiny IDE

Here we describe the C# development cycle in the Tiny IDE for producing `*.exe` files which are stored in the `AppData\Local\XlCalcNetIDE\Bin` folder, and which can be run immediately. The process for producing `*.dll` files which can be called from Python is similar but different, and is described *here*.

Overall, the C# development cycle in the Tiny IDE consists of the following steps: The file is opened in the editor panel, and the code is edited as needed. The code can be compiled by clicking on the "Run" button in the main menu bar. If there are any errors or warnings, they are shown in the "Log" tab of the output panel. When clicking on the error messages in the "Log" tab, they are also highlighted in the code. After correcting the errors, the "Run" button can be clicked again to compile the code. If there are no errors, the code is compiled into a `*.exe` file, which is stored in the `AppData\Local\XlCalcNetIDE\Bin` folder and which is executed automatically after successful compilation. Any text output of the program is shown in the "Output" tab of the output panel after the program has finished running.

These steps are illustrated in the following screenshots, which show the C# development cycle in the Tiny IDE for producing `*.exe` files:

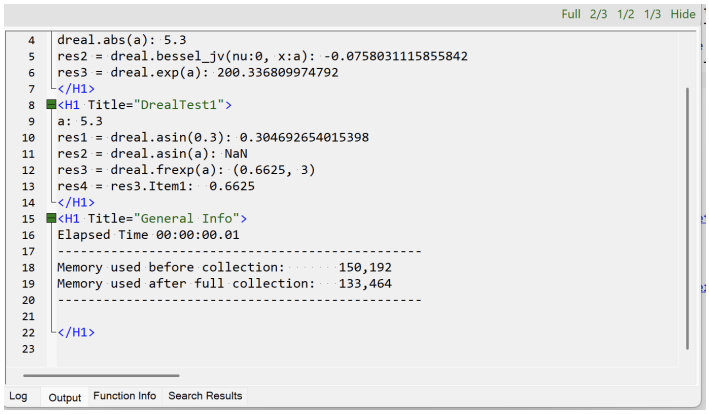
```
1
2   #region Usings
7
8
9   static class Program
10  {
11
12  public static void MainTests()
13  {
14      Console.WriteLine("Demo of some scplx functions");
15      ScplxTest0();
16      ScplxTest1();
17  }
18
19
20  #region ScplxTest0
39
40
41  #region ScplxTest1
60
61
62  /* This region contains the program entry point. Do not change */
63  #region Main
185
186
187 }
188
189  /* Do not remove. Do not add anything after this */
190 #region EOF
193
```

Directly after loading the C# code example, the code is folded, and the "Log" tab of the output panel is empty. The "Output" tab of the output panel is also empty, since the program has not been run yet.

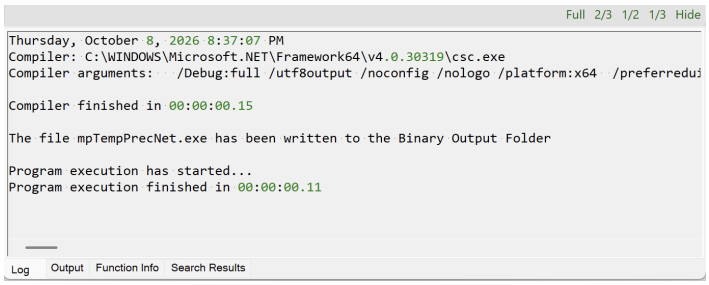
Folding in C# works both on regions and on code blocks. Overall, the folding of regions is more important than the folding of code blocks, since it allows to hide the implementation of functions and classes, and to focus on the overall structure of the code. The folding of code blocks is useful for hiding the implementation of loops and conditional statements, and to focus on the overall structure of the code.

The screenshot above shows the overall structure of the C# code examples in XlCalcNet, which are organized into regions. The first region contains the using statements, followed (outside of any region) by a `static class Program` statement, and the first function in this class, which always has the form `public static void MainTests()`. The body of this function contains calls to the functions where "the real work" is done.

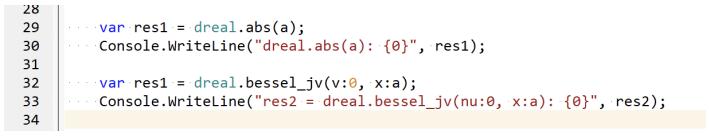
The implementations of these functions are located in the following regions (in this example the regions "ScplxTest0" and "ScplxTest1"), which are organized according to the functionality of the functions. The next region is called "Main", and contains the implementation of the `Main` function, which is the entry point of the program, and contains a lot of necessary boilerplate code, which makes sure that the program can run without a project file (which Visual Studio would require). The last region is called "EOF", and makes sure that code folding is working properly (adressing a small bug in the underlying ScintillaNet library). These last two regions should never be changed by the user during normal use.



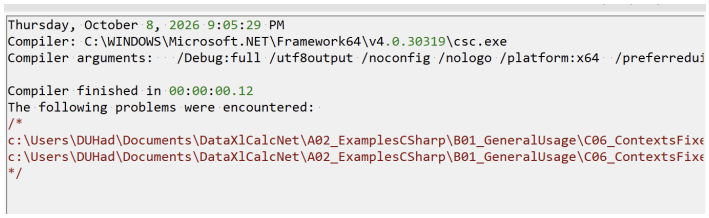
The output of the program is shown in the "Output" tab of the output panel after the program has finished running. The output can be folded and unfolded, and the context menu allows to copy the output to the clipboard, clear the output, or save the output to a file.



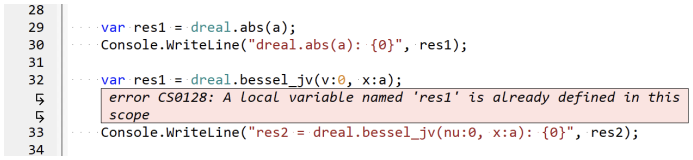
The "Log" tab of the output panel contains additional information after successfully compiling and running the program.



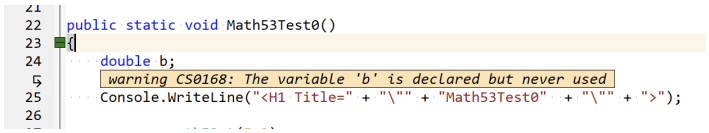
The snapshot above shows a code snippet which contains an error (the variable `res1` is defined twice)



This error is detected by the compiler, and is reported in the Log tab of the output panel.



After double-clicking in the Log tab on the first line in red color, the error is also indicated in the editor, with a full error message below the line where the error occured.

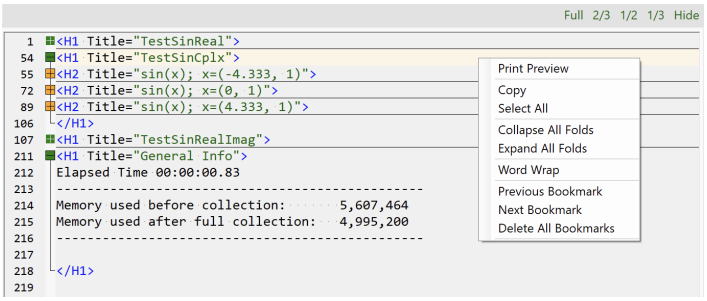


Warnings are treated in a way similar to errors: in this case the program is compiled (in the absence of errors), but is not run. Instead the warning is shown in the Log tab, and after double-clicking on it the warning is also indicated in the editor, with a full error message below the line where the warning was triggered.

1.1.3.7 The Output tab in the Output Panel

Describe the toolbar on top of the Output tab.

Note that the Output tab in the Output Panel can be used to display any text output of a Python or C# program. The output can be folded and unfolded, and the context menu allows to copy the output to the clipboard, clear the output, or save the output to a file.



1.1.3.8 Source code

The Python source code for starting the IDE can be found here: https://github.com/duhadler/XlCalcNet/blob/master/xlcalcnet/ShowEditor.py

The C# source code for the IDE can be found here: https://github.com/duhadler/XlCalcNet/tree/master/xlcalcnet/Addin/NET48/Source/TinyEditor

1.1.4 Installing and using Numpy, Matplotlib, Pandas, Scipy, Seaborn, Statsmodels and Scikit-learn

1.1.4.1 Numpy

Numpy is "The fundamental package for scientific computing with Python". For more information, see https://numpy.org/ and https://en.wikipedia.org/wiki/NumPy.

Assuming that the Python batch files have been installed as described *here*, Numpy can be installed by double-clicking on the batch file `A04installnumpy.bat` in the `C:\Python313` folder.

The following example shows how Numpy can be used with the Decimal type to calculate the mean:

```
import numpy as np
from xlcalcnet import dpm

def demo_mp():
    r = 8
    c = 2
    dpm.dps = 30
    R = np.ndarray((r,c),dtype=dpm.realtype)
    d1 = dpm.t(1.0)
    for i in range(r):
        for j in range(c):
            R[i,j] = 10*(i+1) + d1/(j+7)
    print(R)
    print()
    res = np.mean(R)
    print("res = np.mean(R): \n", res, type(res))
    print()
    res = np.mean(R, axis=0)
    print("res = np.mean(R, axis=0): \n", res, type(res))
    print()
    res = np.mean(R, axis=1)
    print("res = np.mean(R, axis=1): \n", res, type(res))

def demo_all():
    demo_mp()

demo_all()
```

The output looks like this:

```
[[Decimal('10.1428571428571428571428571429') Decimal('10.125')]
 [Decimal('20.1428571428571428571428571429') Decimal('20.125')]
 [Decimal('30.1428571428571428571428571429') Decimal('30.125')]
 [Decimal('40.1428571428571428571428571429') Decimal('40.125')]
 [Decimal('50.1428571428571428571428571429') Decimal('50.125')]
 [Decimal('60.1428571428571428571428571429') Decimal('60.125')]
 [Decimal('70.1428571428571428571428571429') Decimal('70.125')]
 [Decimal('80.1428571428571428571428571429') Decimal('80.125')]]

res = np.mean(R):
 45.133928571428571428571428571 <class 'decimal.Decimal'>

res = np.mean(R, axis=0):
 [Decimal('45.142857142857142857142857143') Decimal('45.125')] <class 'numpy.ndarray'>

res = np.mean(R, axis=1):
 [Decimal('10.133928571428571428571428571')
 Decimal('20.133928571428571428571428571')
 Decimal('30.133928571428571428571428571')
 Decimal('40.133928571428571428571428571')
 Decimal('50.133928571428571428571428571')
 Decimal('60.133928571428571428571428571')
```

(continues on next page)

(continued from previous page)

```
Decimal('70.133928571428571428571428571')
Decimal('80.133928571428571428571428571')] <class 'numpy.ndarray'>
```

1.1.4.2 Matplotlib

Matplotlib is "a comprehensive library for creating static, animated, and interactive visualizations in Python". For more information, see https://matplotlib.org/ and https://en.wikipedia.org/wiki/Matplotlib.

Assuming that the Python batch files have been installed as described *here*, Matplotlib can be installed by double-clicking on the batch file `A05installmatplotlib.bat` in the `C:\Python313` folder.

The following example shows how Matplotlib can be used to produce a boxplot. This code example shows how parameters can be passed:

The Python code for the example below can also be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04b_Matplotlib3D.py.

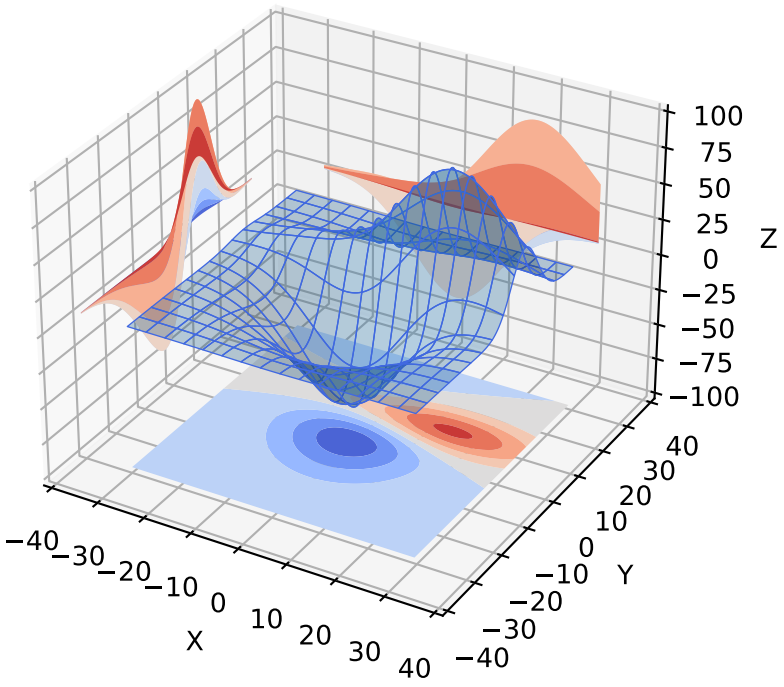
```
from xlcalcnet import gui
import os, re
import numpy as np
import mpl_toolkits.mplot3d.axes3d as axes3d
import matplotlib.pyplot as plt

def ProjectFilledContour(**kwargs):
    OutputDir = kwargs['OutputDir'] if 'OutputDir' in kwargs else 'OutputMonitor'
    Title = kwargs['Title'] if 'Title' in kwargs else 'ProjectFilledContour'
    PlotStyle = kwargs['PlotStyle'] if 'PlotStyle' in kwargs else 'default'
    OutputMode = kwargs['OutputMode'] if 'OutputMode' in kwargs else 'gui'
    FigSizeX = float(kwargs['FigSizeX']) if 'FigSizeX' in kwargs else 4
    FigSizeY = float(kwargs['FigSizeY']) if 'FigSizeY' in kwargs else 4
    Resolution = int(kwargs['Resolution']) if 'Resolution' in kwargs else 300
# End of standard key word arguments

    plt.style.use(PlotStyle)

    fig = plt.figure()
    ax = fig.add_subplot(projection='3d')
    X, Y, Z = axes3d.get_test_data(0.05)

    # Plot the 3D surface
    ax.plot_surface(X, Y, Z, edgecolor='royalblue', lw=0.5, rstride=8, cstride=8,
                    alpha=0.3)

    # Plot projections of the contours for each dimension.  By choosing offsets
    # that match the appropriate axes limits, the projected contours will sit on
    # the 'walls' of the graph
    ax.contourf(X, Y, Z, zdir='z', offset=-100, cmap='coolwarm')
    ax.contourf(X, Y, Z, zdir='x', offset=-40, cmap='coolwarm')
    ax.contourf(X, Y, Z, zdir='y', offset=40, cmap='coolwarm')

    ax.set(xlim=(-40, 40), ylim=(-40, 40), zlim=(-100, 100),
           xlabel='X', ylabel='Y', zlabel='Z')
    fig.tight_layout()

# Start of output choices
    if (OutputMode == 'gui'):
        gui.plot(fig, __file__, Title)
    else:
        FName = 'Temp'
        if OutputDir != 'Temp': FName = re.sub('[^a-zA-Z0-9]', '', Title)
        LocalDir = gui.get_local_appdata_xlcalcnet()
        FullPath = os.sep.join([LocalDir, OutputDir, FName])
        plt.savefig(FullPath + '.' + OutputMode,  bbox_inches='tight')
    plt.close('all')

try:
    if __name__ == '__main__':
        ProjectFilledContour()
```

(continues on next page)

(continued from previous page)

```
except Exception:
    import traceback
    print(traceback.format_exc())
```

This produces the following output:



1.1.4.3 Pandas

Pandas is "a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language". For more information, see https://pandas.pydata.org/ and https://en.wikipedia.org/wiki/Pandas_(software)

Assuming that the Python batch files have been installed as described *here*, Pandas can be installed by double-clicking on the batch file `A06installpandas.bat` in the `C:\Python313` folder.

The following example shows how Pandas can be used to access data in an Excel workbook:

```
import time
import pandas as pd

def main_tests():
    get_irisdata()

def get_documents_folder():
    """Returns the documents folder."""
    import ctypes, ctypes.wintypes
    buf = ctypes.create_unicode_buffer(ctypes.wintypes.MAX_PATH)
    ctypes.windll.shell32.SHGetFolderPathW(None, 0x0005, None, 0, buf)
    return str(buf.value)

def get_irisdata():
    fn = get_documents_folder()
    fn += r'\DataXlCalcNet\DataExamples\MainExamples\Workbooks\Datasets.xlsx'
    datasets = pd.ExcelFile(fn)
    df = pd.read_excel(datasets, 'iris')
    print(df)
    print(df.head(8))
    print(df.dtypes)

try:
    if __name__ == '__main__':
        start0 = time.time()
        main_tests()
        end0 = time.time()
        print('Elapsed time:', format(end0 - start0, '.4g'), 'seconds' )

except Exception:
    import traceback
    print(traceback.format_exc())
```

The output looks like this:

```
     Sepal.Length  Sepal.Width  Petal.Length  Petal.Width    Species
0             5.1          3.5           1.4          0.2     setosa
1             4.9          3.0           1.4          0.2     setosa
2             4.7          3.2           1.3          0.2     setosa
3             4.6          3.1           1.5          0.2     setosa
4             5.0          3.6           1.4          0.2     setosa
..            ...          ...           ...          ...        ...
145           6.7          3.0           5.2          2.3  virginica
146           6.3          2.5           5.0          1.9  virginica
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica

[150 rows x 5 columns]
   Sepal.Length  Sepal.Width  Petal.Length  Petal.Width Species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
5           5.4          3.9           1.7          0.4  setosa
```

(continues on next page)

(continued from previous page)

```
6           4.6          3.4           1.4          0.3  setosa
7           5.0          3.4           1.5          0.2  setosa
Sepal.Length    float64
Sepal.Width     float64
Petal.Length    float64
Petal.Width     float64
Species          object
dtype: object
Elapsed time: 0.4529 seconds
```

1.1.4.4 Scipy

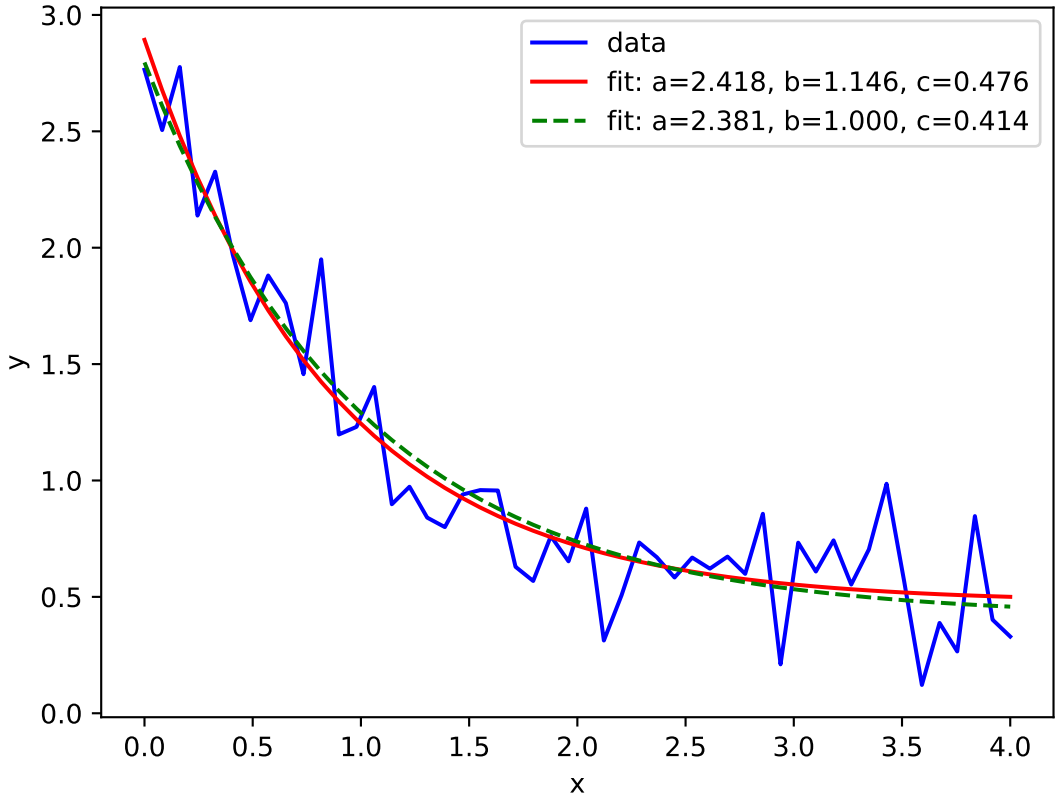
Scipy is "a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language". For more information, see https://scipy.org/ and https://en.wikipedia.org/wiki/SciPy.

Assuming that the Python batch files have been installed as described *here*, Scipy can be installed by double-clicking on the batch file `A07installscipy.bat` in the `C:\Python313` folder.

The following example shows how Scipy can be used for fitting data:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def func(x, a, b, c):
    return a * np.exp(-b * x) + c

def test_curvefit():
    # See https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html
    print('Hello from test_curvefit!')

    xdata = np.linspace(0, 4, 50)
    y = func(xdata, 2.5, 1.3, 0.5)
    rng = np.random.default_rng()
    y_noise = 0.2 * rng.normal(size=xdata.size)
    ydata = y + y_noise
    plt.plot(xdata, ydata, 'b-', label='data')

    popt, pcov = curve_fit(func, xdata, ydata)
    print(popt)
    plt.plot(xdata, func(xdata, *popt), 'r-',
             label='fit: a=%5.3f, b=%5.3f, c=%5.3f' % tuple(popt))

    popt, pcov = curve_fit(func, xdata, ydata, bounds=(0, [3., 1., 0.5]))
    print(popt)
    plt.plot(xdata, func(xdata, *popt), 'g--',
             label='fit: a=%5.3f, b=%5.3f, c=%5.3f' % tuple(popt))

    plt.xlabel('x')
    plt.ylabel('y')
    plt.legend()
    plt.show()

try:
    print()
    test_curvefit()

except Exception:
    import traceback
    print(traceback.format_exc())
```

This produces the following output:


data
fit: a=2.418, b=1.146, c=0.476
fit: a=2.381, b=1.000, c=0.414
y
x
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

1.1.4.5 Seaborn

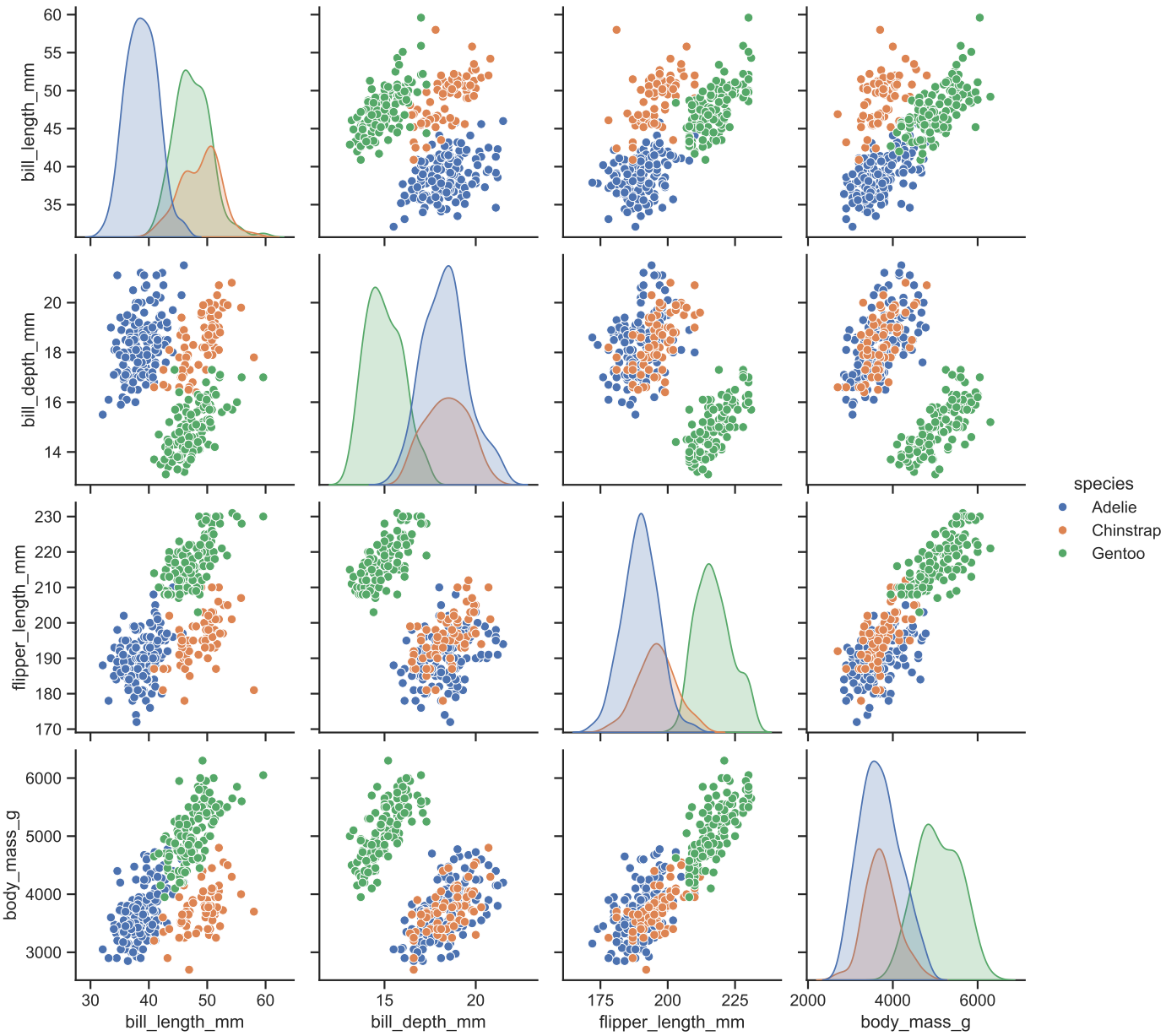
Seaborn is "a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics". For more information, see https://seaborn.pydata.org/.

Assuming that the Python batch files have been installed as described *here*, Seaborn can be installed by double-clicking on the batch file `A08installseaborn.bat` in the `C:\Python313` folder.

The following example shows how Seaborn can be used to produce a scatterplot matrix:

```
from xlcalcnet import gui
from pathlib import Path
import os
import seaborn as sns
import matplotlib.pyplot as plt

# See also: https://seaborn.pydata.org/examples/scatterplot_matrix.html

def ScatterplotMatrix(**kwargs):
    OutputDir = kwargs['OutputDir'] if 'OutputDir' in kwargs else 'OutputMonitor'
    Title = kwargs['Title'] if 'Title' in kwargs else 'ScatterplotMatrix'
    PlotStyle = kwargs['PlotStyle'] if 'PlotStyle' in kwargs else 'default'
    OutputMode = kwargs['OutputMode'] if 'OutputMode' in kwargs else 'svg'
    FigSizeX = float(kwargs['FigSizeX']) if 'FigSizeX' in kwargs else 4
    FigSizeY = float(kwargs['FigSizeY']) if 'FigSizeY' in kwargs else 4
    Resolution = int(kwargs['Resolution']) if 'Resolution' in kwargs else 300
# End of standard key word arguments

    plt.style.use(PlotStyle)

    sns.set_theme(style='ticks')
    df = sns.load_dataset('penguins')
    sns.pairplot(df, hue='species')
    fig = plt.gcf()

# Start of output choices
    if (OutputMode == 'plt'):
        plt.show()
    elif (OutputMode == 'gui'):
        gui.plot(fig, __file__, Title)
    else:
        FName = 'Temp'
        if OutputDir != 'Temp': FName = (Path(__file__).stem)
        LocalDir = gui.get_local_appdata_xlcalcnet()
        FullPath = os.sep.join([LocalDir, OutputDir, FName + '.' + OutputMode])
        plt.savefig(FullPath,  bbox_inches='tight')
        if OutputDir != 'Temp': print('Graphics written to: ', FullPath)
    plt.close('all')

try:
    if __name__ == '__main__':
        ScatterplotMatrix(OutputMode='gui')

except Exception:
    import traceback
    print(traceback.format_exc())
```

This produces the following output:


bill_length_mm
bill_depth_mm
flipper_length_mm
body_mass_g
species
Adelie
Chinstrap
Gentoo

1.1.4.6 Statsmodels

Statsmodels is a Python package for statistical computations. For more information, see https://www.statsmodels.org/stable/index.html.

Assuming that the Python batch files have been installed as described *here*, Statsmodels can be installed by double-clicking on the batch file `A09installstatsmodels.bat` in the `C:\Python313` folder.

The following example shows how Statsmodels can be used for ordinary least squares calculations:

```
import time
import numpy as np
import pandas as pd
import statsmodels.api as sm

# See also: https://www.statsmodels.org/stable/examples/notebooks/generated/ols.html


def OLS(**kwargs):

# End of standard key word arguments

    nsample = 100
    x = np.linspace(0, 10, 100)
    X = np.column_stack((x, x**2))
    beta = np.array([1, 0.1, 10])
    e = np.random.normal(size=nsample)

    X = sm.add_constant(X)
    y = np.dot(X, beta) + e

    model = sm.OLS(y, X)
    results = model.fit()
    print(results.summary())


try:
    if __name__ == '__main__':
        start0 = time.time()
        OLS()
        end0 = time.time()
        print('Elapsed time:', format(end0 - start0, '.4g'), 'seconds' )

except Exception:
    import traceback
    print(traceback.format_exc())
```

The output looks like this

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       1.000
Model:                            OLS   Adj. R-squared:                  1.000
Method:                 Least Squares   F-statistic:                4.027e+06
Date:                Sun, 11 Oct 2026   Prob (F-statistic):          2.61e-239
Time:                        08:43:02   Log-Likelihood:                -146.41
No. Observations:                 100   AIC:                             298.8
Df Residuals:                      97   BIC:                             306.6
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          1.6938      0.312      5.422      0.000       1.074       2.314
x1            -0.1856      0.144     -1.286      0.202      -0.472       0.101
x2            10.0215      0.014    717.320      0.000       9.994      10.049
```

(continues on next page)

(continued from previous page)

```
==============================================================================
Omnibus:                        2.046   Durbin-Watson:                   2.207
Prob(Omnibus):                  0.359   Jarque-Bera (JB):                1.489
Skew:                          -0.065   Prob(JB):                        0.475
Kurtosis:                       2.416   Cond. No.                         144.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
Elapsed time: 0.005406 seconds
```

1.1.4.7 Scikit-learn

Scikit-learn is a Python package for machine learning. For more information, see https://scikit-learn.org/stable/index.html.

Assuming that the Python batch files have been installed as described *here*, Scikit-learn can be installed by double-clicking on the batch file `A10install_scikit_learn.bat` in the `C:\Python313` folder.

The following example shows how Scikit-learn can be used for a Principal Component Analysis (PCA) on the Iris dataset.:

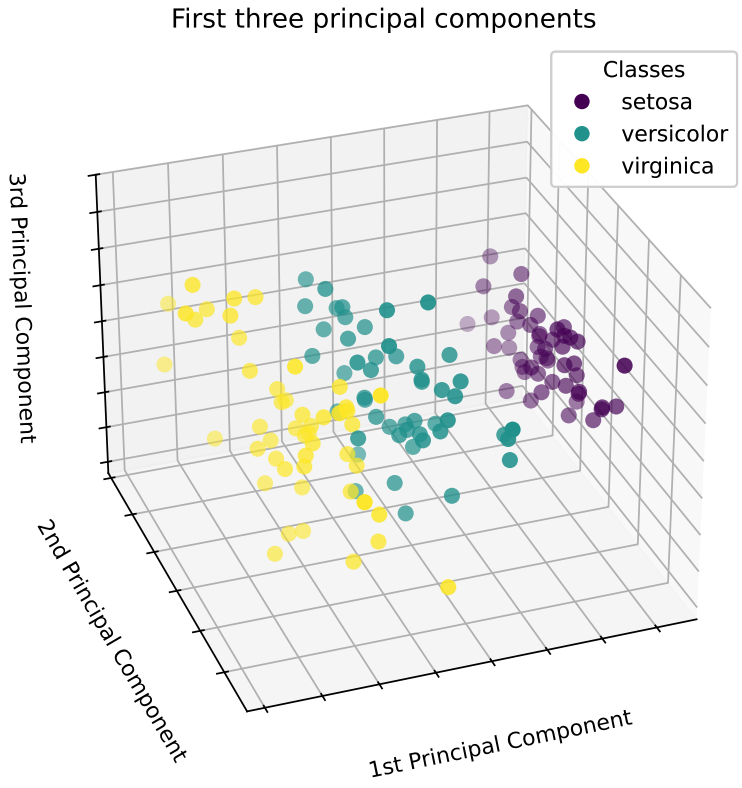
```
from xlcalcnet import gui
from pathlib import Path
import os, time
import numpy as np
from sklearn.decomposition import PCA

# see https://scikit-learn.org/stable/auto_examples/decomposition/plot_pca_iris.html

def PlotPCA(**kwargs):
    OutputDir = kwargs['OutputDir'] if 'OutputDir' in kwargs else 'OutputMonitor'
    Title = kwargs['Title'] if 'Title' in kwargs else 'First three principal components'
    PlotStyle = kwargs['PlotStyle'] if 'PlotStyle' in kwargs else 'default'
    OutputMode = kwargs['OutputMode'] if 'OutputMode' in kwargs else 'svg'
    FigSizeX = float(kwargs['FigSizeX']) if 'FigSizeX' in kwargs else 8
    FigSizeY = float(kwargs['FigSizeY']) if 'FigSizeY' in kwargs else 6
    Resolution = int(kwargs['Resolution']) if 'Resolution' in kwargs else 300
# End of standard key word arguments

    # Choose 'Agg' as backend unless we want to display the graphics
    if (OutputMode != 'gui'):
        import matplotlib
        matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.style.use(PlotStyle)

    from sklearn.datasets import load_iris
    iris = load_iris(as_frame=True)
    #print(iris.keys())
    fig = plt.figure(1, figsize=(FigSizeX, FigSizeY))
    ax = fig.add_subplot(111, projection='3d', elev=-150, azim=110)

    X_reduced = PCA(n_components=3).fit_transform(iris.data)
    scatter = ax.scatter(
        X_reduced[:, 0],
        X_reduced[:, 1],
        X_reduced[:, 2],
        c=iris.target,
        s=40,
    )
    ax.set(
        title=Title,
        xlabel='1st Principal Component',
        ylabel='2nd Principal Component',
        zlabel='3rd Principal Component',
    )
    ax.xaxis.set_ticklabels([])
    ax.yaxis.set_ticklabels([])
    ax.zaxis.set_ticklabels([])

    # Add a legend
    legend1 = ax.legend(
        scatter.legend_elements()[0],
        iris.target_names.tolist(),
        loc='upper right',
        title='Classes',
    )
    ax.add_artist(legend1)
```

(continues on next page)

(continued from previous page)

```
# Start of output choices
    if (OutputMode == 'plt'):
        plt.show()
    elif (OutputMode == 'gui'):
        gui.plot(fig, __file__, Title)
    else:
        FName = 'Temp'
        if OutputDir != 'Temp': FName = (Path(__file__).stem)
        LocalDir = gui.get_local_appdata_xlcalcnet()
        FullPath = os.sep.join([LocalDir, OutputDir, FName + '.' + OutputMode])
        plt.savefig(FullPath,  bbox_inches='tight')
        if OutputDir != 'Temp': print('Graphics written to: ', FullPath)
    plt.close('all')

try:
    if __name__ == '__main__':
        start0 = time.time()
        PlotPCA(OutputMode = 'gui')
        end0 = time.time()
        print('Elapsed time:', format(end0 - start0, '.4g'), 'seconds' )

except Exception:
    import traceback
    print(traceback.format_exc())
```

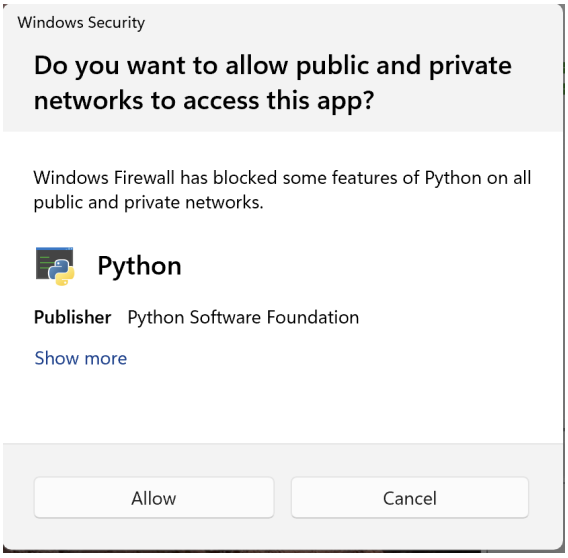
The output looks like this:



1.1.5 Calling Python from C:#: socket server vs in-process calls

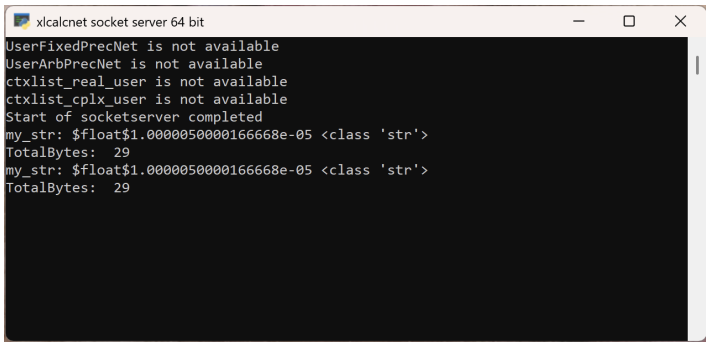
A running socket server is critical for the use of XlCalcNet from Microsoft Excel.

When starting the socket server for the first time from a specific installation of python.exe, a dialog will appear to allow access of this version of Python to networks.



Confirm, since this is required for the socket server to work properly. If you have multiple installations of Python, you may have to do this for each installation.

The socketserver is started from the TinyIDE: in the main menu, click on `Tools` -> `Start SocketServer`.



1.1.5.1 Calling the socketserver from C#

The full C# source code of the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05a_CallSocketServer.cs.

The following example illustrates the use:

```
#region Usings
using MpFunLabClient;
using System;
using System.Diagnostics;
using System.Globalization;
using System.Threading;
using System.Numerics;
using FixedPrecNet;
#endregion
```

(continues on next page)

(continued from previous page)

```
static class Program
{

public static void MainTests()
{
    Console.WriteLine("Demo of call socket server");
    for (int i = 0; i < 1; i++)
        TestSocketServer();
}

public static void TestSocketServer()
{
    Console.WriteLine("Hello TestSocketServer!");
    bool Transpose = false;
    bool ShowShape = true;

    string Code2 = "from A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C02_BasicCurves import D02_Circle;";
    Code2 += "D02_Circle.CircleXY(); result = 'Done'";

    dynamic ResultFinal = MpFunLabSocketClientClass.CallSocketServer0(Code2, Transpose, ShowShape);
    Console.WriteLine();
    Console.WriteLine("Returned:");
    Console.WriteLine("{0}, {1}", ResultFinal.ToString(), ResultFinal.GetType());

    try
    {
        int U0 = ResultFinal.GetUpperBound(0);
        int U1 = ResultFinal.GetUpperBound(1);
        Console.WriteLine("U0: {0}, U1: {1}", U0, U1);
        for (int i = 0; i <= U0; i++)
        {
            for (int j = 0; j <= U1; j++)
            {
                Console.WriteLine("{0}, {1}", ResultFinal[i, j], ResultFinal[i, j].GetType());
            }
        }
    }
    catch (Exception)
    {
    }
    Console.WriteLine();
}
```

The Python source code for the socketserver itself can be found here: https://github.com/duhadler/XlCalcNet/blob/master/xlcalcnet/Addin/NET48/Bin/socketspy.py

The C# source code for the socket client which is called by the above program can be found here: https://github.com/duhadler/XlCalcNet/tree/master/xlcalcnet/Addin/NET48/Source/ClientServer

1.1.5.2 Calling Python from C# in-process

With Pythonnet, it is possible to call python from C# in-process. For the purposes of XlCalcNet this method is slower than calling the socket server. However, in other use cases, it might be the preferred method.

The full C# source code of the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05b_TestPythonNet.cs.

The following example illustrates the use. Note that this requires including a `using Python.Runtime;` statement (line 6), passing the full path of the Python root directory(line 61), the Python DLL path (line 62) and setting a number of environment variables (lines 87-90). The Python engine must be initialized before any Python code can be executed (line 22):

```
1 #region Usings
2 using System;
```

(continues on next page)

(continued from previous page)

```
using System.Collections.Generic;
using System.Numerics;
using FixedPrecNet;
using Python.Runtime;
#endregion

static class Program
{

public static void MainTests()
{
    TestPythonNet0();
}

#region TestPythonNet0

public static void TestPythonNet0()
{
    Console.WriteLine("Hello TestPythonNet1");
    PythonEngine.Initialize();
    using (Py.GIL())
    {
        dynamic np = Py.Import("numpy");
        Console.WriteLine(np.cos(np.pi * 2));

        dynamic sin = np.sin;
        Console.WriteLine(sin(5));

        double c = (double)(np.cos(5) + sin(5));
        Console.WriteLine(c);

        dynamic a = np.array(new List<float> { 1, 2, 3 });
        Console.WriteLine(a.dtype);

        dynamic b = np.array(new List<float> { 6, 5, 4 }, dtype: np.int32);
        Console.WriteLine(b.dtype);

        Console.WriteLine(a * b);
    }
}

#endregion



/* This region contains the program entry point. Do not change */
#region Main

public static void Main(string[] args)
{
    if (args.Length < 2)
    {
        Console.WriteLine(
            "This application needs to be started with 2 arguments;");
        Console.WriteLine("See the manual of xlcalcnet for details.");
    }
    else
    {
        _PythonRootDir = args[0];
        _PythonNetPyDll = args[1];
        _LocalAppDataDir = Environment.GetFolderPath(Environment.SpecialFolder
            .LocalApplicationData);
        AppDomain currentDomain = AppDomain.CurrentDomain;
        currentDomain.AssemblyResolve +=
            new ResolveEventHandler(LoadFromXlCalcNet);
        currentDomain.AssemblyResolve +=
```

(continues on next page)

(continued from previous page)

```
            new ResolveEventHandler(LoadFromXlCalcNet2);
        currentDomain.AssemblyResolve +=
            new ResolveEventHandler(LoadFromPythonNet);
        currentDomain.AssemblyResolve +=
            new ResolveEventHandler(LoadFromAppLocal);
        System.Threading.Thread.CurrentThread.CurrentCulture =
            new System.Globalization.CultureInfo("en-US");
        System.Threading.Thread.CurrentThread.CurrentUICulture =
            new System.Globalization.CultureInfo("en-US");
        var ci = (System.Globalization.CultureInfo)System.Threading.Thread
            .CurrentThread.CurrentCulture.Clone();
        ci.NumberFormat.NegativeInfinitySymbol = "-Inf";
        ci.NumberFormat.PositiveInfinitySymbol = "+Inf";
        System.Threading.Thread.CurrentThread.CurrentCulture = ci;
        var stopWatch = new System.Diagnostics.Stopwatch();
        stopWatch.Start();
        try
        {
            Environment.SetEnvironmentVariable("PYTHONHOME", _PythonRootDir);
            Environment.SetEnvironmentVariable("PYTHONPATH", _PythonRootDir);
            Environment.SetEnvironmentVariable("PYTHONNET_PYDLL",
                _PythonNetPyDll);
            MainTests();
        }
        catch (Exception Ex)
        {
            Console.Error.WriteLine(Ex.Message);
            Console.Error.WriteLine("$+$");
            Console.Error.WriteLine(Ex.StackTrace);
            Console.Error.WriteLine("$+$");
        }
        stopWatch.Stop();
        var ts = stopWatch.Elapsed;
        string elapsedTime = string.Format("{0:00}:{1:00}:{2:00}.{3:00}",
            ts.Hours, ts.Minutes, ts.Seconds, ts.Milliseconds / 10);
        Console.WriteLine("<H1 Title=" + "\"" + "General Info" + "\"" + ">");
        Console.WriteLine("Elapsed Time " + elapsedTime);
        Console.WriteLine("-------------------------------------------------");
        Console.WriteLine("Memory used before collection:       {0:N0}",
            GC.GetTotalMemory(false));
        GC.Collect();
        Console.WriteLine("Memory used after full collection:   {0:N0}",
            GC.GetTotalMemory(true));
        Console.WriteLine("-------------------------------------------------");
        Console.WriteLine("");
        Console.WriteLine("</H1>");
    }

}

private static string _PythonRootDir;
private static string _PythonNetPyDll;
private static string _LocalAppDataDir;

/*
    The code for LoadFromXlCalcNet, LoadFromXlCalcNet2, LoadFromPythonNet
    and LoadFromAppLocal is not shown here
*/


#endregion


}
```

Running the program produces the following output:

```
Hello TestPythonNet1
1.0
-0.9589242746631385
-0.675262089199912
float64
int32
[ 6. 10. 12.]
<H1 Title="General Info">
Elapsed Time 00:00:00.58
-------------------------------------------------
Memory used before collection:       5,854,328
Memory used after full collection:   1,431,368
-------------------------------------------------

</H1>
```

In other words, it took about 0.58 seconds to run the program. This is much slower than calling the socket server, which takes about 0.02 seconds. The reason for this is that Pythonnet has to load the entire Python interpreter into the C# process, which takes a lot of time and memory. In contrast, the socket server runs in a separate process and can be started once and then used for multiple calls from C#.

1.1.6 Reasons for using multiprecision arithmetic

An introduction to the problems of rounding errors and catastrophic cancellation can be found in Goldberg [351]. Excellent reference texts are Higham [370] and Higham [371].

In the following sections we will give a few examples of how the use of double precision without special precaution can give wrong results.

Example: Sums

Sums are often calculated exactly if all summands have an exact representation. If this is not the case, results can be unpredictable. In MS Excel, the formula

```
=SUM(10000000000,-16000000000,6000000000)
```

will give the correct result 0, but the analogous formula

```
=SUM(1E+40,-1.6E+40,6E+39)
```

returns $1.20893E+24$ instead of the correct result 0.

Example: Standard Deviation

Like sums, variances and standard deviations are often calculated exactly if all arguments have an exact representation. If this is not the case, results can again be unpredictable. In MS Excel, the formula

```
=VAR(1E+30,1E+30,1E+30)
```

returns $2.97106E+28$ instead of the correct result 0, which should be the obvious results since all arguments are the same.

Example: Overflow and underflow

In many situations where the final result is representable in double precision, some of the interim results cause overflow or underflow. A popular example is the function $f(x,y) = \sqrt{x^2+y^2}$. With $x = 3 \cdot 10^{300}$ and $y = 4 \cdot 10^{300}$ the result $f(x,y) = 5 \cdot 10^{300}$ is representable in double precision, but the (naive) calculation will overflow.

Example: Polynomials

Consider the following example from Cuyt *et al.* [190]:

For $a = 77617$ and $b = 33096$, calculate

$$Y = 333.75b^6 + a^2(11a^2b^2 - b^6 - 121b^4 - 2) + 5.5b^8 + \frac{a}{2b}$$

The correct result is $Y = -54767/66192 = -8.27396\ldots \cdot 10^{-1}$

Example: Trigonometric Functions

Trigonometric functions are sensitive to small perturbations.

In double precision and binary floating point arithmetic, the tangent of $x = 1.57079632679489$ is calculated as $\tan(x) = 1.48752 \cdot 10^{14}$, whereas the correct result is $\tan(x) = 1.51075 \cdot 10^{14}$. This amounts to an absolute error of $2.32287 \cdot 10^{12}$ and a relative error of 1.54%.

There are also limits on the range of arguments, e.g. $\sin(10^8)$ returns the value $-9.31639 \cdot 10^{-1}$ (with an relative error of $-6.22776 \cdot 10^{-13}$), whereas $\sin(10^9)$ returns an invalid result (the exact result is $5.45843 \cdot 10^{-1}$)

Example: Logarithms and Exponential Functions

Consider the following example from Kaveh R. Ghazi and Vincent Lefèvre and Philippe Théveny and Paul Zimmermann [1503]:

Determine 10 decimal digits of the constant

$$Y = 173746a + 94228b - 78487c, \quad \text{where}$$

$$a = \sin(10^{22}), b = \log(17.1), c = \exp(0.42).$$

The expected result is $Y = -1.341818958 \cdot 10^{-12}$.

Example: Linear Algebra

The following example is from W. Hofschuster and W. Krämer [1504]:

We want to solve the (ill-conditioned) system of linear equations $Ax = b$ with

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} = \begin{pmatrix} 64919121 & -159018721 \\ 41869520.5 & -102558961 \end{pmatrix}, b = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

The correct solution is $x_1 = 205117922$, $x_2 = 83739041$.

To solve this 2×2 system numerically we first use the well known formulas

$$x_1 = \frac{a_{22}}{a_{11}a_{22} - a_{12}a_{21}}, \quad x_2 = \frac{-a_{21}}{a_{11}a_{22} - a_{12}a_{21}},$$

Calculating this directly in double precision gives the following wrong result:

$x_1 = 102558961$, $x_2 = 41869520.5$

Example: Eigenvalues

The following example is from Brown *et al.* [161]:

The behaviour and stability of many physical systems are connected with the spectral properties of non-self-adjoint operators. However, numerical approximations of eigenvalues of non-selfadjoint operators (even matrices) may fail dramatically. For example, the non-normal 7×7 matrix

$$A = \begin{pmatrix} 289 & 2054 & 326 & 128 & 70 & 32 & 6 \\ 1152 & 30 & 1312 & 512 & 288 & 128 & 32 \\ -29 & -1990 & 766 & 384 & 1018 & 224 & 58 \\ 512 & 128 & 640 & 0 & 640 & 512 & 128 \\ 1053 & 2246 & -514 & -384 & -766 & 800 & 198 \\ -287 & -6 & 1722 & -128 & 1978 & -30 & -2042 \\ -2176 & -285 & -1563 & -512 & -539 & -1152 & -287 \end{pmatrix}$$

has the eigenvalues $-2, -4, 0, 1, 1, 2, 4$. Calculations in double precision yield a set of complex eigenvalues, such as $8.57 \pm 3.73i; 2.29 \pm 8.33i; -5.43 \pm 6.56i; -8.85$ with imaginary parts as large as 8.33, which are nowhere near the true eigenvalues. The reason for this is that owing to the nonnormality of the matrix, its eigenvalues are highly sensitive to perturbations, and therefore unavoidable rounding errors render the numerical eigenvalue computations unreliable.

1.1.7 Rebuilding the .dll files of XlCalcNet and XlCalcNet2 from source code

The XlCalcNet and XlCalcNet2 python packages contain both precompiled .dll files and their source code.

This section descibes how to rebuild the .dll files of XlCalcNet and XlCalcNet2 from source code, either completely or only in part. The building process itself is not particularly difficult, but it requires the installation of MSYS2 (version 3.4.9.x86_64 or later: about 4 GB in size), Free Pascal (version 2.6.4 or later: about 320 MB in size), and Visual Studio Community (version 2019 or later: about 4.6 GB in size).

In the following it is assumed that the user has installed MSYS2, Free Pascal and Visual Studio Community and is comfortable using them.

MSYS2 is required to build the file `FixedPrecGCC64K8.dll` in the XlCalcNet `Bin` folder. The details of the building process are described in *FixedPrec*.

MSYS2 is also required to build the file `ArbPrecNetGCCK8.dll` in the XlCalcNet2 `Bin` folder. The details of the building process are described in *ArbPrec*.

Free Pascal is required to build the file `libwe64d.dll` in the XlCalcNet `Bin` folder. The details of the building process are described in *FixedPrec*.

Erverything else can be done in Visual Studio Community:

The details of building the file `FixedPrecNet.dll` in the XlCalcNet `Bin` folder are described in *FixedPrec*.

The details of building the file `ArbPrecNet.dll` in the XlCalcNet2 `Bin` folder are described in *ArbPrec*.

The details of building the files `MpFunLabClient.dll` and `MpFunLabAddin64.dll` in the XlCalcNet `Bin` folder are described in *ClientServer*.

The details of building the file `TinyOutputMonitorUserCtrl.dll` in the XlCalcNet `Bin` folder are described in *OutputMonitor*.

The details of building the file `TinyIDEUserCtrl.dll` and `FindReplaceSD.dll` in the XlCalcNet `Bin` folder are described in *TinyIde*.

The details of building the file `TinyPlot2DUserCtrl.dll` in the XlCalcNet `Bin` folder are described in *GalleryOfPlots*.

The details of building the file `TinyPlot3DUserCtrl.dll` in the XlCalcNet `Bin` folder are described in *Wpf3D*.

The details of building the file `TinyDataViewerUserCtrl.dll` in the XlCalcNet `Bin` folder are described in *DataViewer*.

All other .dll files in the XlCalcNet `Bin` folder have been aquired via https://www.nuget.org/ from the supporting libraries and should probably not be changed.

1.2 Using XlCalcNet within MS Excel

Compared to Microsofts "Python in Excel" (see https://support.microsoft.com/en-us/excel/python/introduction-to-python-in-excel), the following features of XlCalcNet can be highlighted:

- Everything is open source and can be modified by the user.
- The user can use any (locally installed) Python version from 3.8 to 3.15, and any Python package, including packages which are not supported by Microsoft.
- MS Excel 2010 - 2024 are supported.
- Everything stays local. No data and no code leave the computer of the user. An internet connection is not necessary.
- Using Python.NET, code written in C# can also be used from Python, and therefore from MS Excel.
- The user can use any Python IDE, including the Tiny IDE which is part of XlCalcNet.
- The user can write functions of arbitrary length and complexity in Python using an external editor, and call them in a short Python script in a spreadsheet cell.
- Such functions can also be called as procedures from VBA.
- There is extensive support for calculations in multiprecision, including arbitrary precision.

1.2.1 Preparing MS Excel for using XlCalcNet

Important

Before beginning to install the MS Excel addins, make sure that the socket server is up and running as expected (see the section above). When calling a function, which calls the socket server, for the first time, MS Excel will seem to freeze if the socket server does not respond quickly. However, subsequent calls will return quickly when the socket server is not up and running (there seems to be a learning effect).

To enable the use of XlCalcNet in MS Excel, two add-ins need to be installed: the first, named `Mpfunlablocal.xlam`, contains functions written in Visual Basic for Applications (VBA), including functions populating the XlCalcNet Navigator dialog, and is intended to be modified by the user during normal use. This add-in is located in a subfolder of the `DataXlCalcNet` folder in the `Documents` folder: `DataXlCalcNet\SpreadsheetAddins\MSExcel`.

The second, named `Mpfunlab.xll`, contains the functionality behind the spreadsheet functions `CPY_0` - `CPY_9` and `ASDOUBLE` and the XlCalcNet Navigator dialog container. The files comprising the whole add-in (`Mpfunlab.xll`, `ExcelDna.Integration.dll`, `MpFunLabAddin64.dll`, `MpFunLabClient.dll`, `MpFunLab.Dna`) have been compiled using Visual Studio and are NOT intended to be modified by the user during normal use. This add-in is located in the `Bin` folder in the `XlCalcNet` package: `PathToPython\Lib\site-packages\xlcalcnet\Addin\NET48\Bin`, where `PathToPython` is the path to the Python installation (in our example `C:\Python313`).

1.2.1.1 Installing Mpfunlablocal.xlam and Mpfunlab.xll

To install these two add-ins properly, follow these steps:

Again, make sure that the socket server is up and running.

Within MS Excel, on an Excel worksheet, open the Add-ins dialog from the main menu/ribbon: `Developer` -> `Excel Add-ins`. In the Add-ins dialog, click on `Browse...` and in the `Documents` folder navigate to the folder `DataXlCalcNet\SpreadsheetAddins\MSExcel`. Double-click on `Mpfunlablocal.xlam`, which will then appear in the Add-ins dialog under "Add-ins available". Still in the Add-ins dialog, click on `Browse...` again, and navigate to the folder `PathToPython\Lib\site-packages\xlcalcnet\Addin\NET48\Bin` (see above). Double-click on `Mpfunlab.xll`, which will then appear in the Add-ins dialog under "Add-ins available". Still in the Add-ins dialog, click on `OK`. Exit MS Excel.

When starting MS Excel the next time, both add-ins will be loaded.

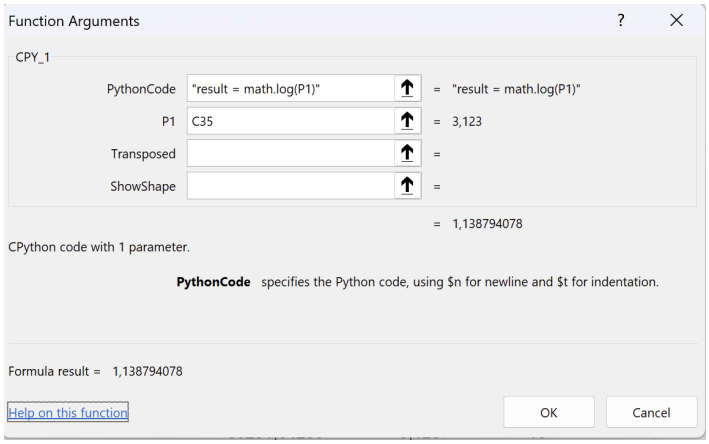
1.2.2 Using the Python standard library functions within spreadsheet formulas

> **Note**
>
> In the following subsections, we will work with the file `XlDemoCPYstd.xlsx`, which contains examples for using the Python standard library. More advanced examples using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet start *here*, and examples for using dynamic arrays in MS Excel with XlCalcNet start *here*.

Within MS Excel, open from the main menu `File -> Open` in the Documents folder the MS Excel workbook `DataXlCalcNet\DataExamples\MainExamples\Workbooks\XlDemoCPYstd.xlsx`.

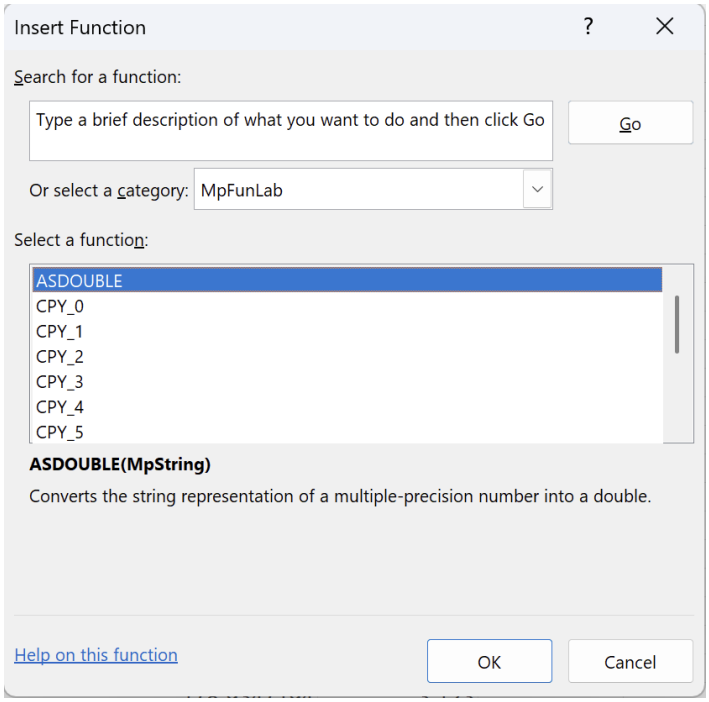
This file contains several worksheets which demonstrate different possibilities of using the Python standard library within MS Excel spreadsheet formulas.

Before we start with that we will briefly explore the use of these functions with the Function Dialog of MS Excel: In the worksheet "Math", select cell `B12` and click on the icon of the Function Dialog in the Formula Bar. This opens the Function Dialog:



We see that the formula in the spreadsheeet cell is `=CPY_1("result = math.ceil(P1)",C12)`. The function `CPY_1` has a required string parameter, `Formula`, which contains a Python script. This python script can contain several Python statement. The last statement is always expected to assign a value to the variable result, in this case a Python float, which is coverted to a floating point number in double precision (a "Double") in LibreOffice. The next optional parameter, `Param1`, can be a string, a Double, a Boolean value or a reference. In this case, it is a reference (`C12`) which points to a Double with the value `3.123`. This optional parameter, `Param1`, is referenced in the Python formula given above as `P1`. The last but one parameter, `Transposed`, and the last parameter, `ShowShape`, are only relevant when arrays are returned; this is discussed *here*.

In the worksheet "Math", select an emptz cell and click on the icon of the Function Dialog in the Formula Bar. This opens the Function Insert Dialog:



We see that the function `CPY_1` is in the Category "MpFunLab", and that there are 10 functions, `CPY_0` to `CPY_9`, which differ only by the number of parameters `Param1` to `Param9` which they support (The function `CPY_0` does not have a parameter `Param0`). There is also a function `ASDOUBLE`, which is used to convert string representations of a Python Fraction or Decimal into a Double; this is discussed *here*.

1.2.2.1 The worksheet "GeneralInfo"

The worksheet "GeneralInfo" contains calls to the Python modules `os`, `platform` and `sys`.

For example, the cell `B4` contains the formula `=CPY_0("result = platform.processor()")`. The function result depends on the hardware, e.g. `Intel64 Family 6 Model 165 Stepping 5, GenuineIntel`.

The cell `B16` contains the formula `=CPY_0("result = os.getcwd()")`. The function result depends on the LibreOffice installation, e.g. `C:\Program Files\LibreOffice\program`.

The cell `B33` contains the formula `=CPY_0("result = str(sys.float_info.epsilon)")`, which return the machine epsilon in double precision. The function result is: `2.220446049250313e-16`.

1.2.2.2 The worksheet "Math"

The worksheet "Math" demonstrates the use of the Python module `math` and the use of parameters in functions.

For example, the cell `B33` contains the formula `=CPY_1("result = math.exp(P1)-1",C33)`, which calculates $\exp(\mathrm{P1})-1$ naively. The function result is `1.00000500000696E-05`, with the cell `C33` containing the value `0.00001`.

The cell `B34` contains the formula `=CPY_1("result = math.expm1(P1)",C34)`, which calculates $\exp(\mathrm{P1}) - 1$ using the `expm1` function. The function result is `1.00000500001667E-05`, with the cell `C34` again containing the value `0.00001`.

The floating point values `NaN`, `+inf` and `-inf` are not supported in spreadsheet programs, which only return `#NUM!` in these cases. Use the string representation instead. This is shown in the cells `A4:B9` on this worksheet.

The cell `B36` contains the formula `=CPY_2("result = math.log(P1, P2)",C36, D36)`, with the cell `C36` containing the value `3.123` and the cell `D36` containing the value `10`. The function result is: `0.494571984230199`.

1.2.2.3 The worksheet "Arrays"

The worksheet "Arrays" demonstrates the use of arrays in functions. The Python equivalent of arrays are lists.

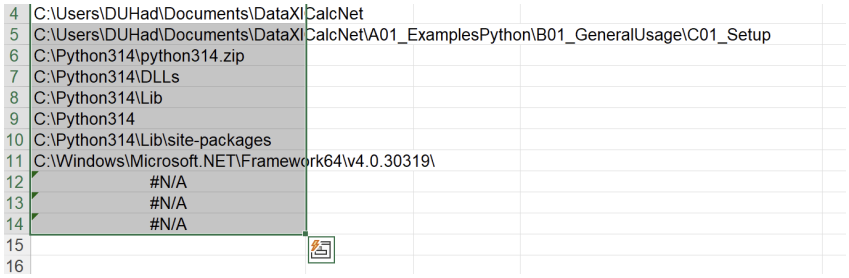
For example, the cell `B3` contains the formula `=CPY_0("result = str(sys.path)")`. The function `sys.path` returns a list of strings (containing the entries of the Python path), which is converted into one string by writing `str(sys.path)`. This makes sure that the result can be displayed in one cell, but the result string is quite long and hard to read.

We can use an array formula to improve readability. We recall that in addition to the usual parameters `P1` - `P9` in the functions `CPY_0` to `CPY_9` there are two more: The last but one paramter, named `Transposed`, which is optional with default value 0, where a non-zero value means that the returned array should be transposed; and the last paramter, named `ShowShape`, which is optional with default value 0, where a non-zero value means that the shape of the returned array should be indicated like `R7C3` for an array with 7 rows and 3 columns.

As an example, the cell `B6` contains the formula `=CPY_0("result = sys.path",1,1)` and returns the shape information `R12C1` followed by the separator `|`, followed by the first entry of the transposed array, i.e. in this case `R12xC1| C:\Users\DUHad\Documents\DataXlCalcNet`.

We can build the corresponding array formula by first selecting a range like `R12C1`, then typing in the input line of the formula bar `=CPY_0("result = sys.path",1,0)`, and finally, while keeping the shift and control keys pressed down, pressing the enter key.

This is what has been done in cells `B9:B20`. The formula is now displayed as `{=CPY_0("result = sys.path",1,0)}`, with curly braces, to indicate that it is an array formula. We can change the range which is covered by the array formula by dragging the small blue rectangle in the lower right corner in the selected range (see the screenshot below).



1.2.2.4 The worksheet "Data"

The worksheet "Data " demonstrates the use of named ranges in MS Excel spreadsheet formulas.

Currently there a two small datasets with predefined as named ranges: `matA` defined as the range $Data.$A$3:$A$7 and `matB` defined as the range $Data.$A$3:$A$7.

These ranges are used in the workbook "Arrays".

1.2.3 Using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet within spreadsheet formulas

> **Note**
>
> In the following subsections we will work with the file `XlDemoCPYadv.xlsx`, which contains more advanced examples using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet. Examples for using MS Excel with the Python standard library start *here*, and an example of using dynamic arrays in MS Excel starts *here*.

Within MS Excel, open from the main menu `File -> Open` in the Documents folder the MS Excel workbook `DataXlCalcNet\DataExamples\MainExamples\Workbooks\XlDemoCPYadv.xlsx`.

This file contains several worksheets which demonstrate different possibilities of using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet within MS Excel spreadsheet formulas.

Some of these functions write their output into the `XlCalcNetIDE\OutputMonitor` folder in the AppDataLocal folder. In order to see it, we need to open the `Output Monitor` application: start the `Tiny IDE` by clicking on its icon in the task bar, and in the main menu, click on `Tools` -> `Start Output Monitor`.

1.2.3.1 The worksheet "FractionsInput"

The worksheet "FractionsInput " demonstrates the use of the Python data type `Fraction` in MS Excel spreadsheet formulas.

MpInputFractions | 123456789012345678 / 901234567890

	A	B	C	D
1				
2		**Formatted as Double**		**Original Data**
3		1,36986300138694E+05		123456789012345678 / 901234567890
4		9,99999999900000E-12		123456789 / 01234567890123456789O
5		2,17395555782604E-03		12345678901234 / 5678901234567890
6		1,38696253953018E+03		12345678901234567 / 8901234567890
7		5,26315809944599E+11		123456789012345678901 / 234567890
8		1,56470034579900E+01		1234567890123456 / 78901234567890
9		3,57142968723526E-07		123456789012 / 345678901234567890
10		2,17395603043452E+17		1234567890123456789012345 / 67890
11		1,81847937116985E-01		123456789012345 / 678901234567890
12				
13				
14	**Prod:**	4,80254411022073E+17		109488241064252366394927246517420416158540118291
15	**Sum:**	2,17396129359400E+17		48848206021544887892701015659858329178996133656
16				
17	**Median:**	1,56470034579900E+01		29394473574368/1878600823045
18	**Average:**	2,41551254843778E+16		48848206021544887892701015659858329178996133656
19	**Mean:**	2,41551254843778E+16		48848206021544887892701015659858329178996133656
20	**Variance:**	4,66773527093429E+33		19089073868930015117999458972414586128722220685
21				
22	**Min:**	9,99999999900000E-12		1/100000000010
23	**Max:**	2,17395603043452E+17		617283945061728394506 17/283945
24				

If we try to enter a number like `123456789012345678/901234567890`, i.e. a number with more than 16 digits into a spreadsheet cell, it will automatically be shortened to `1.23456789012346E+029`. In order to be able to enter such numbers into spreadsheet cells, the cells must first be formatted as text, and thereafter these numbers can be entered as text. This is mostly useful when working with Python Fractions and Decimals.

However, this kind of input is hard to read; it is also not directly usable for numerical spreadsheet functions. This can be changed by using the function `ASDOUBLE`, a shown in column `B`

As an example, worksheet "FractionsInput" contains a named range called "MpInputFractions" in column `A`, which can be used as input for descriptive statistics or linear algebra routines.

The module `npm` of XlCalcNet can be used to calculate desciptive statistics, like `prod`, `sum`, `mean`, `median`, `variance` with fractions. The function `npm.mean` can be called from a spreadsheet cell as follows:

```
=CPY_1("from xlcalcnet import npm, qpm $n x = npm.array(P1, dtype=qpm); result=str(npm.prod(x))",MpInputFractions)
```

Às before, we are using `$n` for newline and `$t` for indentation. The result is itself a `Fraction` (converted to a string), and is exact. The corresponding Python code with conventional formatting would look like this (where `P1` is the input range):

```
from xlcalcnet import npm, qpm
x = npm.array(P1, dtype=qpm)
result=str(npm.prod(x))
```

Note that the input range is converted into a Numpy array with data type `qpm`, which is the data type for rational numbers in XlCalcNet. The result is then converted into a string, so that it can be displayed in a spreadsheet cell.

Since the string output is quite long, we can use `result=float(npm.prod(x))` instead of `result=str(npm.prod(x))` to get a floating point number in double precision, which is more readable. However, this will not be exact anymore.

1.2.3.2 The worksheet "DecimalsInput"

The worksheet "DecimalsInput " demonstrates the use of the Python data type `Decimal` in MS Excel spreadsheet formulas.

MpInputDecimals | 123456789012345678.901234567890

	A	B	C	D	E
1					
2	**Dps:**	50			
3					
4		**Formatted as Double**		**Original Data**	
5		1,23456789012346E+17		123456789012345678.901234567890	
6		1,23456789012346E+08		123456789.012345678901234567890	
7		1,23456789012346E+15		1234567890123456.78901234567890	
8		1,23456789012346E+13		12345678901234.5678901234567890	
9		1,23456789012346E+20		1234567890123456789012.34567890	
10		1,23456789012346E+15		1234567890123456.78901234567890	
11		1,23456789012346E+11		123456789012.34567890123456789O	
12		1,23456789012346E+23		1234567890123456789012345.67890	
13		1,23456789012346E+14		123456789012345.678901234567890	
14					
15					
16	**Prod:**	6,66246276571616E+136		6.66246276571615849418609358097213134465746545108 69E+136	
17	**Sum:**	1,23580371863209E+23		123580371863208865332109.886533087408281457890	
18					
19	**Median:**	1,23456789012346E+15		1234567890123456.78901234567890	
20	**Average:**	1,37311524292454E+22		1373115242924542948 1208231423842 10	
21	**Mean:**	1,37311524292454E+22		1373115242924542948 1208231423842 10	
22	**Variance:**	1,50496589683514E+45		150496589683514229980943223153006330220129796 7.884	
23					
24	**Min:**	1,23456789012346E+08		123456789.012345678901234567890	
25	**Max:**	1,23456789012346E+23		123456789012345678901234.567890	
26					

If we try to enter a number like `123456789012345678.901234567890`, i.e. a number with more than 16 digits into a spreadsheet cell, it will automatically be shortened to `1.23456789012346E+017`. In order to be able to enter such numbers into spreadsheet cells, the cells must first be formatted as text, and thereafter these numbers can be entered as text. This is mostly useful when working with Python Fractions and Decimals.

However, this kind of input is hard to read; it is also not directly usable for numerical spreadsheet functions. This can be changed by using the function `ASDOUBLE`, a shown in column `B`

As an example, worksheet "DecimalsInput" contains a named range called "MpInputDecimals" in column `C`, which can be used as input for descriptive statistics or linear algebra routines.

The module `npm` of XlCalcNet can be used to calculate desciptive statistics, like `prod`, `sum`, `mean`, `median`, `variance` with decimals. The function `npm.mean` can be called from a spreadsheet cell as follows:

```
=CPY_2("from xlcalcnet import npm, dpm $n dpm.dps=int(P2); x = npm.array(P1, dtype=dpm); result=str(npm.mean(x))",
↪MpInputDecimals, $B$2)
```

Às before, we are using `$n` for newline and `$t` for indentation. The result is itself a `Decimal` (converted to a string), and is correct to the number of digits specified. The corresponding Python code with conventional formatting would look like this (where `P1` is the input range and `P2` is the number of digits):

```
from xlcalcnet import npm, dpm
dpm.dps=int(P2)
x = npm.array(P1, dtype=dpm)
result=str(npm.mean(x))
```

Note that the input range is converted into a Numpy array with data type `dpm`, which is the data type for decimal numbers in XlCalcNet. The result is then converted into a string, so that it can be displayed in a spreadsheet cell.

Since the string output is quite long, we can use `result=float(npm.prod(x))` instead of `result=str(npm.prod(x))` to get a floating point number in double precision, which is more readable. However, this will then be correct only to the precision of a double-precision floating-point number.

1.2.3.3 The worksheet "Programming"

The worksheet "Programming" demonstrates the use of small python scripts in MS Excel spreadsheet formulas.

	A	B	C	D
1				
2				
3	**Python standard library: simple loop**			
4	**n:**	45		
5	**Result:**	1980		
6				
7				
8				
9				
10	**Scipy: Numerical quadrature; f(x) = a*x**2 + b**			
11	**Lower limit of integration:**	-1,4		
12	**Upper limit of integration:**	1,5		
13	**Parameter a:**	2,1		
14	**Parameter b:**	1,1		
15	**Result (integral, error estimate):**	(7.473300000000001, 8.297029729931183e-14)		
16				
17				

The first example in this worksheeet shows how to program a simple loop. The formula in the cell `B4`. is

```
=CPY_1("temp = 0 $n for i in range(int(P1)): $n$t temp += i $n result = 2 * temp", B4)
```

The cell `B4` contains the value `45`; the result is `1980`. The corresponding Python code with conventional formatting would look like this(where `P1` is `n`, the number of iterations):

```
temp = 0
for i in range(int(P1)):
    temp += i
result = 2 * temp
```

The second example in this worksheet shows how to perform numerical integration.The formula in the cell `B15`. is

```
=CPY_4("from scipy.integrate import quad $n def integrand(x, a, b): return a*x**2 + b $n a = float(P3); b = float(P4);  $n I␣
↪= quad(integrand, float(P1), float(P2), args=(a,b))  $n result = str(I) ", B11, B12, B13, B14)
```

This formula calls the function `quad` from the module `scipy.integrate` in the package `scipy`. The parameters of this function are taken from the cells `B11` to `B14`. The result is a tuple containing the intergral and an error estimate. The corresponding Python code with conventional formatting would look like this (with P1 is the lower limit of integration, P2 is the upper limit, P3 is the coefficient of x^2, and P4 is the constant term):

```
from scipy.integrate import quad
def integrand(x, a, b): return a*x**2 + b
a = float(P3); b = float(P4)
I = quad(integrand, float(P1), float(P2), args=(a,b))
result = str(I)
```

The cells B16 to B17 show the same result as an array formula, with B16 containing the integral and B17 the error estimate.

1.2.3.4 The worksheet "OutputMonitorCSV"

This is a snapshot of the worksheet "OutputMonitorCSV" with the relevant formula in cell B1. The output of the formula in cell B1 is a CSV file named TTest1External.csv which is written to the Output Monitor folder. The Output Monitor application must be running in order to see the output. The Output Monitor can be started from the main menu of the Tiny IDE by clicking on Tools -> Start Output Monitor.

	A	B	C	D	E	F	G	H
1	**Python Script:**	Students t-test, 1 sample						
2								
3	**n:**	[10, 20, 30]						
4	**mu0:**	1.0						
5	**mean:**	[4.5,4.7]						
6	**std:**	[1,2,3,4]						
7	**alpha:**	0.05						
8	**Options:**	,I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv'						
9								
10								
11								
12	**Note:**	The result is written to the Output Monitor folder (start the Output Monitor application first!)						
13								

The formula in the cell B1. is

```
=CPY_0(TEXTJOIN(" ",1,"from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_
↪PValues $n importlib.reload(D01_StudentT_PValues) $n D01_StudentT_PValues.stats_student_t_1sample_test(n=",B3,", mu0=",B4,
↪", mean=",B5,", std=",B6,", alpha=", B7, B8, ")"," $n result='Students t-test, 1 sample'"))
```

This formula calls the function stats_student_t_1sample_test from the module D01_StudentT_PValues in the package A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample. The parameters of this function are taken from the cells B3 to B8. The result is a CSV file named TTest1External.csv which is written to the Output Monitor folder. The corresponding Python code with conventional formatting and substitution of the referenced cell values would look like this:

```
from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_PValues
importlib.reload(D01_StudentT_PValues)
D01_StudentT_PValues.stats_student_t_1sample_test(n=[10, 20, 30], mu0=1.0, mean=[4.5,4.7], std=[1,2,3,14], alpha=0.015,␣
↪I=True, D=True, T=True, C=True, Onesided=True, Twosided=True, OutputMode='csv')
result='Students t-test, 1 sample'
```

Note

Here we are using importlib.reload to make sure that any changes in the code of the module D01_StudentT_PValues which may have occured since the last use of this spreadsheet function become effective without restarting the Python interpreter.

The cell A3 contains ="n:", and the cell B3 contains ="[10, 20, 30]". This passes the list [10, 20, 30] as the parameter n to the function stats_student_t_1sample_test.

The cell A4 contains ="mu0:", and the cell B4 contains ="1.0". This passes the value 1.0 as the parameter mu0 to the function stats_student_t_1sample_test.

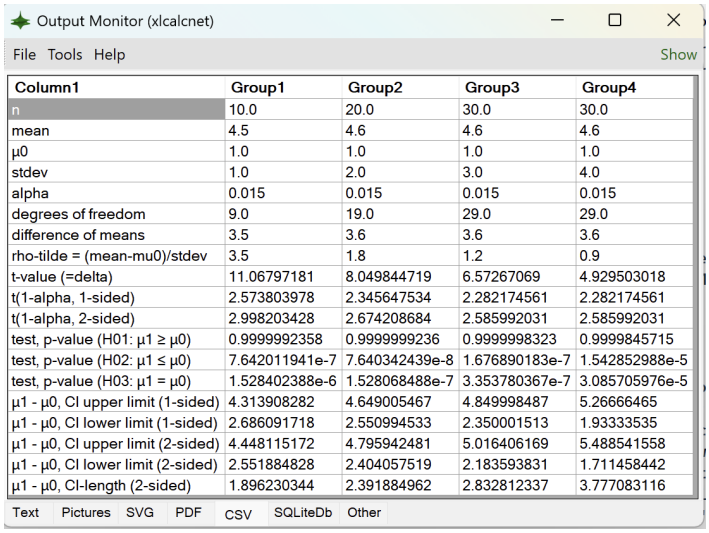
The cell A5 contains ="mean:", and the cell B5 contains ="[4.5,4.7]". This passes the list [4.5,4.7] as the parameter mean to the function stats_student_t_1sample_test.

The cell A6 contains ="std:", and the cell B6 contains ="[1,2,3,14]". This passes the list [1,2,3,14] as the parameter std to the function stats_student_t_1sample_test.

The cell A7 contains ="alpha:", and the cell B7 contains ="0.015". This passes the value 0.015 as the parameter alpha to the function stats_student_t_1sample_test.

The cell A8 contains ="Options:", and the cell B8 contains =",I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv'". This passes the options I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv' as keyword arguments to the function stats_student_t_1sample_test.

And this is the Output Monitor showing the result (with the project panel hidden)



Column1	Group1	Group2	Group3	Group4
n	10.0	20.0	30.0	30.0
mean	4.5	4.6	4.6	4.6
μ0	1.0	1.0	1.0	1.0
stdev	1.0	2.0	3.0	4.0
alpha	0.015	0.015	0.015	0.015
degrees of freedom	9.0	19.0	29.0	29.0
difference of means	3.5	3.6	3.6	3.6
rho-tilde = (mean-mu0)/stdev	3.5	1.8	1.2	0.9
t-value (=delta)	11.06797181	8.049844719	6.57267069	4.929503018
t(1-alpha, 1-sided)	2.573803978	2.345647534	2.282174561	2.282174561
t(1-alpha, 2-sided)	2.998203428	2.674208684	2.585992031	2.585992031
test, p-value (H01: μ1 ≥ μ0)	0.9999992358	0.9999999236	0.9999998323	0.9999845715
test, p-value (H02: μ1 ≤ μ0)	7.642011941e-7	7.640342439e-8	1.676890183e-7	1.542852988e-5
test, p-value (H03: μ1 = μ0)	1.528402388e-6	1.528068488e-7	3.353780367e-7	3.085705976e-5
μ1 - μ0, CI upper limit (1-sided)	4.313908282	4.649005467	4.849998487	5.26666465
μ1 - μ0, CI lower limit (1-sided)	2.686091718	2.550994533	2.350001513	1.93333535
μ1 - μ0, CI upper limit (2-sided)	4.448115172	4.795942481	5.016406169	5.488541558
μ1 - μ0, CI lower limit (2-sided)	2.551884828	2.404057519	2.183593831	1.711458442
μ1 - μ0, CI-length (2-sided)	1.896230344	2.391884962	2.832812337	3.777083116

Text Pictures SVG PDF CSV SQLiteDb Other

1.2.3.5 The worksheet "OutputMonitorSVG"

This is a snapshot of the worksheet "OutputMonitorSVG" with the relevant formula in cell B1. The output of the formula in cell B1 is a SVG file named D07_DistPlotContinuous.svg which is written to the Output Monitor folder. The Output Monitor application must be running in order to see the output. The Output Monitor can be started from the main menu of the Tiny IDE by clicking on Tools -> Start Output Monitor.

	A	B	C	D	E	F	G	H	I
1	**Python Script:**	Beta distribution							
2									
3	**target:**	'pdf'	# 'pdf', 'cdf', 'sf', 'hf', 'chf', 'qtf', 'isf'						
4	**a:**	[5, 10.0, 20.5]							
5	**b:**	[20.5, 10.0, 5]							
6	**Title:**	'Beta distribution'							
7									
8									
9									
10	**Note:**	The result is written to the Output Monitor folder (start the Output Monitor application first!)							
11									

The formula in the cell B1. is

```
==CPY_0(TEXTJOIN(" ",1,"from A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C01_Intro_Parametric2D import D07_
↪DistPlotContinuous $n importlib.reload(D07_DistPlotContinuous) $n  D07_DistPlotContinuous.DistPlotBeta(target=",B3,", a=",
↪B4,", b=",B5,", Title=",B6,", OutputMode = 'svg'",  ")"," $n result='Beta distribution'"))
```

This formula calls the function DistPlotBeta from the module D07_DistPlotContinuous in the package A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C01_Intro_Parametric2D. The parameters of this function are taken from the cells B3 to B8. The result is a CSV file named TTest1External.csv which is written to the Output Monitor folder. The corresponding Python code with conventional formatting and substitution of the referenced cell values would look like this:

```
from A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C01_Intro_Parametric2D import D07_DistPlotContinuous
importlib.reload(D07_DistPlotContinuous)
```

(continues on next page)

(continued from previous page)

```
D07_DistPlotContinuous.DistPlotBeta(target='pdf', a=[5, 10.0, 20.5], b=[20.5, 10.0, 5], Title='Beta distribution',␣
↪OutputMode = 'svg')
result='Beta distribution'
```

Note

Here we are using `importlib.reload` to make sure that any changes in the code of the module `D07_DistPlotContinuous` which may have occured since the last use of this spreadsheet function become effective without restarting the Python interpreter.

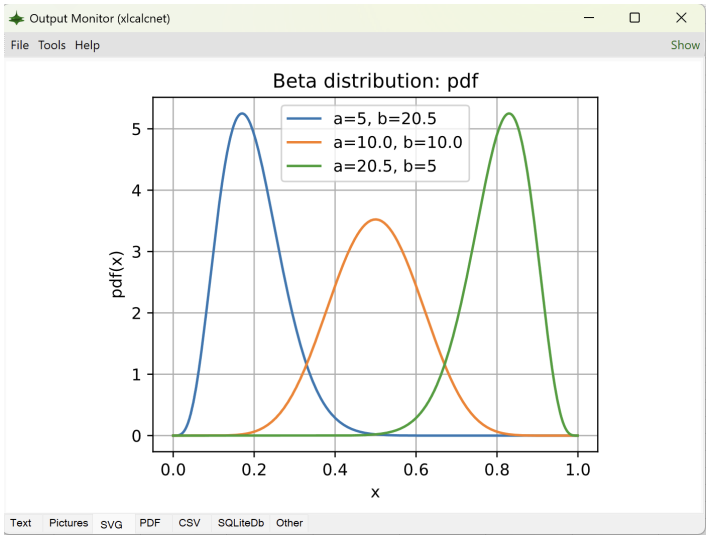
The cell `A3` contains `="target:"`, and the cell `B3` contains `="pdf"`. This passes the string `"pdf"` as the parameter `target` to the function `DistPlotBeta`.

The cell `A4` contains `="a:"`, and the cell `B4` contains `="[5, 10.0, 20.5]"`. This passes This passes the list `[5, 10.0, 20.5]` as the parameter `a` to the function `DistPlotBeta`.

The cell `A5` contains `="b:"`, and the cell `B5` contains `="[20.5, 10.0, 5]"`. This passes the list `[20.5, 10.0, 5]` as the parameter `b` to the function `DistPlotBeta`.

The cell `A6` contains `="Title:"`, and the cell `B6` contains `="Beta distribution"`. This passes the string `"Beta distribution"` as the keyword argument `Title` to the function `DistPlotBeta`.

And this is the Output Monitor showing the result (with the project panel hidden)



1.2.4 Using dynamic arrrays in MS Excel

Note

In the following subsection we will work with the file `XlDemoCPYdyn.xlsx`, which contains an example of using dynamic arrays in MS Excel with XlCalcNet. Examples for using MS Excel with the Python standard library start *here*, and more advanced examples using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet start *here*.

1.2.4.1 The worksheet “Student_t_1sample_test”

Dynamic arrays are only supported in MS Excel 365 and MS Excel 2021 or later. In these versions, the output of a function can be an array, which is then automatically “spilled” into the cells below and to the right of the cell containing the formula. The size of the output array can be changed by changing the input parameters of the function.

The worksheet "Student_t_1sample_test" demonstrates the use of dynamic arrays in Excel with XlCalcNet.

This is a snapshot of the worksheet "Student_t_1sample_test" with the formula in cell `A11`, which is also where the output is starting. The main difference to the example with `*.csv` output is that now in "Options" we have `OutputMode='list'` and not `OutputMode='csv'`.

	A	B	C	D	E	F
1	**Students t-test, 1 sample**					
2						
3	**n:**	[10, 20, 30]				
4	**mu0:**	1.1				
5	**mean:**	[4.5,4.7]				
6	**std:**	[1,2,3,4]				
7	**alpha:**	0.05				
8	**Options:**	,I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='list'				
9						
10						
11	**Item**	**Group1**	**Group2**	**Group3**	**Group4**	
12	**n**	10.0	20.0	30.0	30.0	
13	**mean**	4.5	4.7	4.7	4.7	
14	**μ0**	1.1	1.1	1.1	1.1	
15	**stdev**	1.0	2.0	3.0	4.0	
16	**alpha**	0.05	0.05	0.05	0.05	
17	**degrees of freedom**	9.0	19.0	29.0	29.0	
18	**difference of means**	3.4	3.6	3.6	3.6	
19	**rho-tilde = (mean-mu0)/stdev**	3.4	1.8	1.2	0.9	
20	**t-value (=delta)**	10.75174404	8.049844719	6.57267069	4.929503018	
21	**t(1-alpha, 1-sided)**	1.833112933	1.729132812	1.699127027	1.699127027	
22	**t(1-alpha, 2-sided)**	2.262157163	2.093024054	2.045229642	2.045229642	
23	**test, p-value (H01: μ1 ≥ μ0)**	0.9999990244	0.9999999236	0.9999998323	0.9999845715	
24	**test, p-value (H02: μ1 ≤ μ0)**	9.755717864e-7	7.640342437e-8	1.676890183e-7	1.542852988e-5	
25	**test, p-value (H03: μ1 = μ0)**	1.951143573e-6	1.528068487e-7	3.353780366e-7	3.085705976e-5	
26	**μ1 - μ0, CI upper limit (1-sided)**	3.979681208	4.373291702	4.5306502	4.840866934	
27	**μ1 - μ0, CI lower limit (1-sided)**	2.820318792	2.826708298	2.6693498	2.359133066	
28	**μ1 - μ0, CI upper limit (2-sided)**	4.115356906	4.536028813	4.72021841	5.093624547	
29	**μ1 - μ0, CI lower limit (2-sided)**	2.684643094	2.663971187	2.47978159	2.106375453	
30	**μ1 - μ0, CI-length (2-sided)**	1.430713812	1.872057626	2.24043682	2.987249094	
31						

The formula in the cell `A11` is

```
=CPY_0(TEXTJOIN(" ",1,"from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_
↪PValues $n importlib.reload(D01_StudentT_PValues) $n  D01_StudentT_PValues.stats_student_t_1sample_test(n=",B3,", mu0=",B4,
↪", mean=",B5,", std=",B6,", alpha=", B7, B8, ")"," $n result='Students t-test, 1 sample'"))
```

This formula calls the function `stats_student_t_1sample_test` from the module `D01_StudentT_PValues` in the package `A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample`. The parameters of this function are taken from the cells `B3` to `B8`. The result is a Python list, which is displayed as a dynamic array. The corresponding Python code with conventional formatting and substitution of the referenced cell values would look like this:

```
from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_PValues
importlib.reload(D01_StudentT_PValues)
D01_StudentT_PValues.stats_student_t_1sample_test(n=[10, 20, 30], mu0=1.0, mean=[4.5,4.7], std=[1,2,3,14], alpha=0.015,␣
↪I=True, D=True, T=True, C=True, Onesided=True, Twosided=True, OutputMode='list')
result='Students t-test, 1 sample'
```

Note

Here we are using `importlib.reload` to make sure that any changes in the code of the module `D01_StudentT_PValues` which may have occured since the last use of this spreadsheet function become effective without restarting the Python interpreter.

The cell `A3` contains `="n:"`, and the cell `B3` contains `="[10, 20, 30]"`. This passes the list `[10, 20, 30]` as the parameter `n` to the function `stats_student_t_1sample_test`.

The cell `A4` contains `="mu0:"`, and the cell `B4` contains `="1.0"`. This passes the value `1.0` as the parameter `mu0` to the function `stats_student_t_1sample_test`.

The cell `A5` contains `="mean:"`, and the cell `B5` contains `="[4.5,4.7]"`. This passes the list `[4.5,4.7]` as the parameter `mean` to the function `stats_student_t_1sample_test`.

The cell `A6` contains `="std:"`, and the cell `B6` contains `="[1,2,3,14]"`. This passes the list `[1,2,3,14]` as the parameter `std` to the function `stats_student_t_1sample_test`.

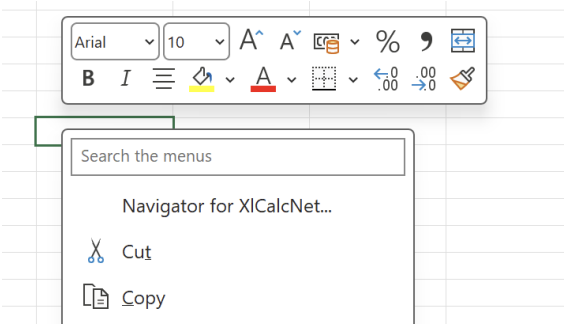
The cell `A7` contains `="alpha:"`, and the cell `B7` contains `="0.015"`. This passes the value `0.015` as the parameter `alpha` to the function `stats_student_t_1sample_test`.

The cell `A8` contains `="Options:"`, and the cell `B8` contains `=",I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv'"`. This passes the options `I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='list'` as keyword arguments to the function `stats_student_t_1sample_test`.

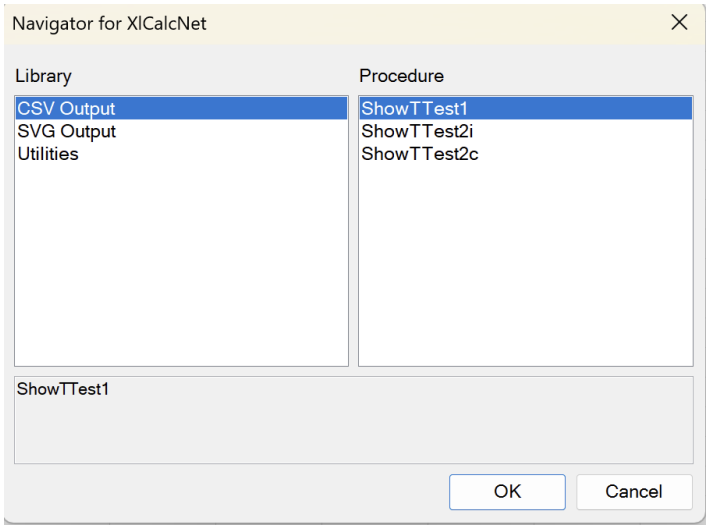
1.2.5 Managing procedures instead of functions

It is possible to call the spreadsheet functions `CPY_0` to `CPY_9` from LibreOffice Basic and to use this to start procedures instead of functions. A few simple examples have already been prepared to demonstrate the technique.

To access the relevant dialog, click on the XlCalcNet logo (in orange) on the main menu bar:



The following dialog box with the title **Navigator for XlCalcNet** will appear:



The first example, with `CSV Output` in "Category" and `ShowTTest1` in "Subroutine", will write a CSV file into the `XlCalcNetIDE\OutputMonitor` folder in the AppDataLocal folder. In order to see it, we need to open the `Output Monitor` application: start the `Tiny IDE` by clicking on its icon in the task bar, and in the main menu, click on `Tools -> Start Output Monitor`. With the `Output Monitor` application running, click `OK`. The CSV file will then immediately appear in the `Output Monitor`.

The corresponding code in the module `MpFunlabLocal -> BasicAndDialogs` is shown below:

```
Sub ShowTTest1()
    Rem Output (a .csv file) is shown in output monitor
    Code = "from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_PValues;"
    Code = Code + "D01_StudentT_PValues.demo_stats_student_t_1sample_test(); result='Done';"
    Result = Application.Run("CPY_0", Code)
    showResult (Result)
End Sub
```

The second example, with `SVG Output` in "Category" and `ShowPolygon` in "Subroutine", will write a SVG file into the `XlCalcNetIDE\OutputMonitor` folder in the AppDataLocal folder. With the `Output Monitor` application running (see above), click `OK`. The SVG file will then immediately appear in the `Output Monitor`.

The corresponding code in the module `MpFunlabLocal` -> `BasicAndDialogs` is shown below:

```
Sub ShowPolygon()
    Rem Graphics is shown in output monitor
    Code = "from A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C02_BasicCurves import D01_RegularConvexPolygon;"
    Code = Code + "D01_RegularConvexPolygon.RegularConvexPolygon(OutputMode='svg'); result='Done';"
    Result = Application.Run("CPY_0", Code)
    showResult (Result)
End Sub
```

1.2.6 Uninstalling

To uninstall these two Add-ins properly, follow these steps: Within MS Excel, on an Excel worksheet, open the Add-ins dialog from the main menu/ribbon: `Developer` -> `Excel Add-ins`. In the Add-ins dialog, unselect `MpFunlab` and `Mpfunlablocal`, and click on `OK`. Close MS Excel. When Excel is started the next time these Add-ins will not be loaded, however, they will still appear (unselected) in the Add-ins dialog (this seems to be a decade-old bug/feature). To remove them also from the Add-ins dialog, you need to rename them in their original location, start Excel again, and start the Add-ins dialog again. When you then re-select these unselected items, a dialog stating that the add-in cannot be found will appear, followed by "Delete from list?". Click on Yes, and the add-in will be removed from the list.

1.3 Using XlCalcNet within LibreOffice Calc

1.3.1 Preparing LibreOffice Calc for using XlCalcNet

To enable the use of XlCalcNet in LibreOffice Calc, two extensions need to be installed: the first, named `MpfunlabLocal.oxt`, contains functions and dialogs written in LibreOffice Basic and is intended to be modified by the user during normal use. The second, named `Mpfunlab.oxt`, contains the functionality behind the spreadsheet functions `APY0` - `APY9` and `ASDOUBLE`. It has been compiled using the SDK of LibreOfffice 7.4 and is NOT intended to be modified by the user during normal use. Both extensions are located in a subfolder of the `DataXlCalcNet` folder in the `Documents` folder: `DataXlCalcNet\SpreadsheetAddins\LibreOfficeCalc`.

1.3.1.1 Installing MpfunlabLocal.oxt and Mpfunlab.oxt

To install these two extensions properly, following the correct order is important. First install `MpfunlabLocal.oxt` within a normally launched LibreOffice Calc (i.e. without administrator privileges): Open the Extension dialog from the main menu:`Tools -> Extensions...`. In the Extension dialog click the `Add` button and in the `Documents` folder navigate to the folder `DataXlCalcNet\SpreadsheetAddins\LibreOfficeCalc`. Double- click on `MpfunlabLocal.oxt`. Then click on the `Close` button in the Extension dialog. In the dialog `Restart LibreOffice`, click on `Restart Now`. This will close LibreOffice Calc and will open the general LibreOffice desktop. Exit LibreOffice.

Now start LibreOffice with administrator privileges: Right-click on the desktop icon of LibreOffice and select `Run as administrator`. In the following dialog, confirm that you want to proceed. In the LibreOffice desktop, open a `Calc Spreadsheet`. Open the Extension dialog from the main menu:`Tools -> Extensions...`. In the Extension dialog click the `Add` button and in the `Documents` folder navigate to the folder `DataXlCalcNet\SpreadsheetAddins\LibreOfficeCalc`. Double- click on `Mpfunlab.oxt`. In the following dialog box "For whom do you want to install the extension?", click on `For all users`. In the following "License Agreement" dialog, click on `Accept`. Then click on the `Close` button in the Extension dialog. In the dialog `Restart LibreOffice`, click on `Restart Now`. This will close LibreOffice Calc and will open the general LibreOffice desktop. Exit LibreOffice.

Within a normally launched LibreOffice Calc, open the Macro Editor from the main menu: `Tools -> Macros -> Edit Macros...` In the Object Catalog of the Macro Editor, select `My Macros & Dialogs -> MpFunlabLocal -> CallsFromPython`: In the function `GetCPythonExeDirLocal` set the path of the external Python installation which matches the main internal Python version, e.g. `GetCPythonExeDirLocal ="C:\Python313"`. Then close the Macro Editor.

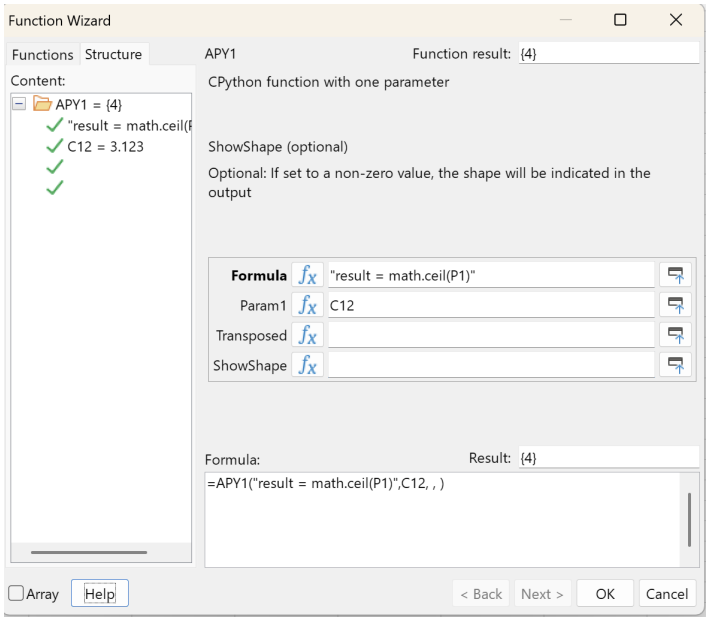
1.3.2 Using the Python standard library functions within spreadsheet formulas

> **Note**
>
> In the following subsections, we will work with the file `LoDemoAPYstd.ods`, which contains examples for using the Python standard library. More advanced examples using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet start *here*.

Within LibreOffice Calc, open from the main menu `File -> Open` in the Documents folder the LibreOffice Calc workbook `DataXlCalcNet\DataExamples\MainExamples\Workbooks\LoDemoAPYstd.ods`.

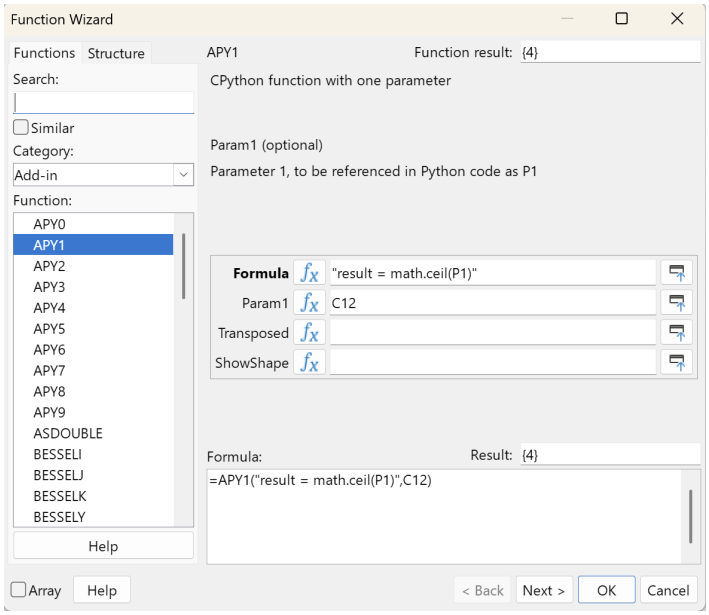
This file contains several worksheets which demonstrate different possibilities of using the Python standard library within Libreoffice Calc spreadsheet formulas.

Before we start with that we will briefly explore the use of these functions with the Function Wizard of LibreOffice Calc: In the worksheet "Math", select cell `B12` and click on the icon of the Function Wizard in the Formula Bar. This opens the Function Wizard with the Structure tab selected:



We see that the formula in the spreadsheeet cell is `=APY1("result = math.ceil(P1)",C12)`. The function `APY1` has a required string parameter, `Formula`, which contains a Python script. This python script can contain several Python statement. The last statement is always expected to assign a value to the variable result, in this case a Python float, which is coverted to a floating point number in double precision (a "Double") in LibreOffice. The next optional parameter, `Param1`, can be a string, a Double, a Boolean value or a reference. In this case, it is a reference (`C12`) which points to a Double with the value `3.123`. This optional parameter, `Param1`, is referenced in the Python formula given above as `P1`. The last but one parameter, `Transposed`, and the last parameter, `ShowShape`, are only relevant when arrays are returned; this is discussed *here*.

If we open the Functions tab, the Functions wizard looks like this:



We see that the function `APY1` is in the Category "Add-in", and that there are 10 functions, `APY0` to `APY9`, which differ only by the number of parameters `Param1` to `Param9` which they support (The function `APY0` does not have a parameter `Param0`). There is also a function `ASDOUBLE`, which is used to convert string representations of a Python Fraction or Decimal into a Double; this is discussed *here*.

1.3.2.1 The worksheet "GeneralInfo"

The worksheet "GeneralInfo" contains calls to the Python modules `os`, `platform` and `sys`.

For example, the cell `B4` contains the formula `=APY0("result = platform.processor()")`. The function result depends on the hardware, e.g. `Intel64 Family 6 Model 165 Stepping 5, GenuineIntel`.

The cell `B16` contains the formula `=APY0("result = os.getcwd()")`. The function result depends on the LibreOffice installation, e.g. `C:\Program Files\LibreOffice\program`.

The cell `B33` contains the formula `=APY0("result = str(sys.float_info.epsilon)")`, which return the machine epsilon in double precision. The function result is: `2.220446049250313e-16`.

1.3.2.2 The worksheet "Math"

The worksheet "Math" demonstrates the use of the Python module `math` and the use of parameters in functions.

For example, the cell `B33` contains the formula `=APY1("result = math.exp(P1)-1",C33)`, which calculates $\exp(\mathrm{P1}) - 1$ naively. The function result is `1.00000500000696E-05`, with the cell `C33` containing the value `0.00001`.

The cell `B34` contains the formula `=APY1("result = math.expm1(P1)",C34)`, which calculates $\exp(\mathrm{P1}) - 1$ using the `expm1` function. The function result is `1.00000500001667E-05`, with the cell `C34` again containing the value `0.00001`.

The floating point values `NaN`, `+inf` and `-inf` are not supported in spreadsheet programs, which only return `#NUM!` in these cases. Use the string representation instead. This is shown in the cells `A4:B9` on this worksheet.

The cell `B36` contains the formula `=APY2("result = math.log(P1, P2)",C36, D36)`, with the cell `C36` containing the value `3.123` and the cell `D36` containing the value `10`. The function result is: `0.494571984230199`.

1.3.2.3 The worksheet "Arrays"

The worksheet "Arrays" demonstrates the use of arrays in functions. The Python equivalent of arrays are lists.

For example, the cell `B3` contains the formula `=APY0("result = str(sys.path)")`. The function `sys.path` returns a list of strings (containing the entries of the Python path), which is converted into one string by writing `str(sys.path)`. This makes sure that the result can be displayed in one cell, but the result string is quite long and hard to read.

We can use an array formula to improve readability. We recall that in addition to the usual parameters `P1` - `P9` in the functions `APY0` to `APY9` there are two more: The last but one paramter, named `Transposed`, which is optional with default value 0, where a non-zero value means that the returned array should be transposed; and the last paramter, named `ShowShape`, which is optional with default value 0, where a non-zero value means that the shape of the returned array should be indicated like `R7C3` for an array with 7 rows and 3 columns.

As an example, the cell `B6` contains the formula `=APY0("result = sys.path",1,1)` and returns the shape information `R12C1` followed by the separator `|`, followed by the first entry of the transposed array, i.e. in this case `R12xC1| C:\Users\DUHad\Documents\DataXlCalcNet`.

We can build the corresponding array formula by first selecting a range like `R12C1`, then typing in the input line of the formula bar `=APY0("result = sys.path",1,0)`, and finally, while keeping the shift and control keys pressed down, pressing the enter key.

This is what has been done in cells `B9:B20`. The formula is now displayed as `{=APY0("result = sys.path",1,0)}`, with curly braces, to indicate that it is an array formula. We can change the range which is covered by the array formula by dragging the small blue rectangle in the lower right corner in the selected range (see the screenshot below).

4	C:\Users\DUHad\Documents\DataXlCalcNet
5	C:\Python313\Lib\site-packages
6	C:\Python313\Lib
7	C:\Python313\DLLs
8	C:\Python313
9	C:\Program Files\LibreOffice\program\python-core-3.13.15\lib
10	C:\Program Files\LibreOffice\program\python-core-3.13.15\lib\site-packages
11	C:\Program Files\LibreOffice\program
12	C:\Program Files\LibreOffice\program\python313.zip
13	C:\Program Files\LibreOffice\program\Lib
14	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\
15	C:\Program Files\LibreOffice\share\extensions\dict-en\pythonpath
16	

1.3.2.4 The worksheet "Data"

The worksheet "Data " demonstrates the use of named ranges in LibreOffice Calc spreadsheet formulas.

Currently there a two small datasets with predefined as named ranges: `matA` defined as the range $Data.$A$3:$A$7 and `matB` defined as the range $Data.$A$3:$A$7.

These ranges are used in the workbook "Arrays".

1.3.3 Using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet within spreadsheet formulas

> **Note**
>
> In the following subsections. we will work with the file `LoDemoAPYadv.ods`, which contains more advanced examples using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet. Examples for using LibreOffice Calc with the Python standard library start *here*.

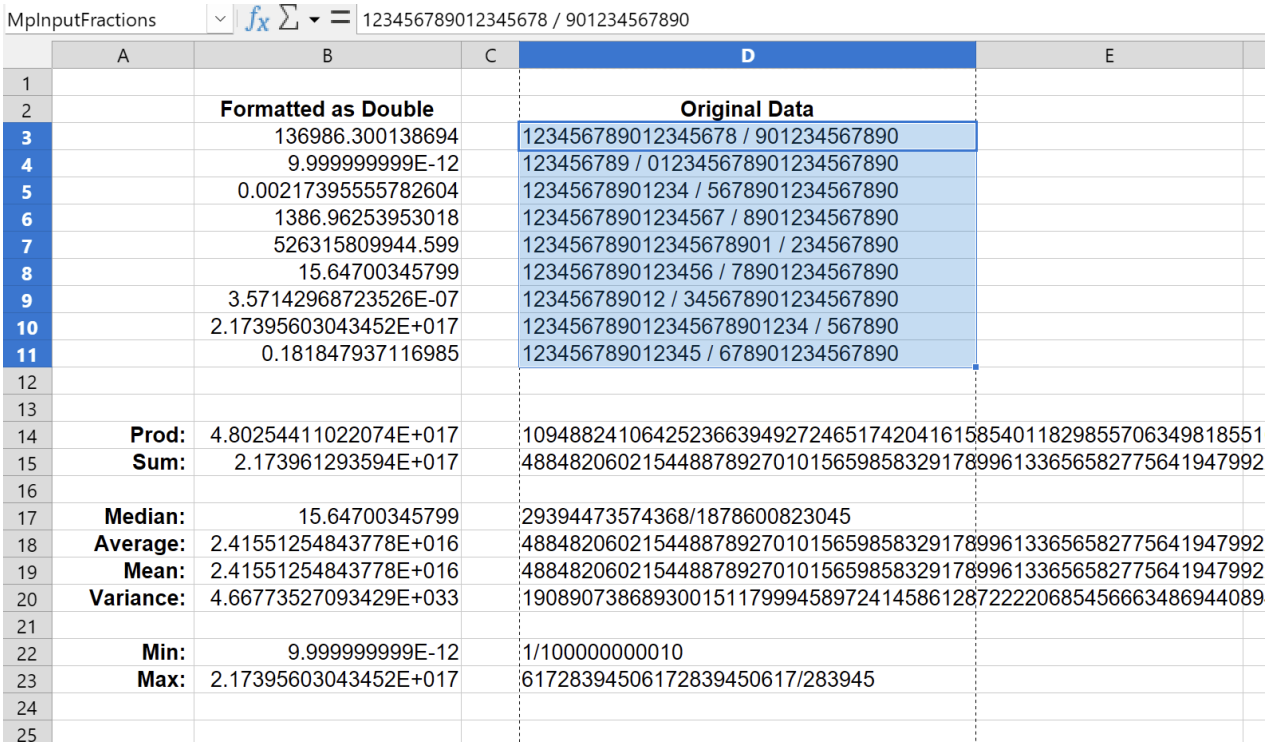
Within LibreOffice Calc, open from the main menu `File -> Open` in the Documents folder the LibreOffice Calc workbook `DataXlCalcNet\DataExamples\MainExamples\Workbooks\LoDemoAPYadv.ods`.

This file contains several worksheets which demonstrate different possibilities of using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet within Libreoffice Calc spreadsheet formulas.

Some of these functions write their output into the `XlCalcNetIDE\OutputMonitor` folder in the AppDataLocal folder. In order to see it, we need to open the `Output Monitor` application: start the `Tiny IDE` by clicking on its icon in the task bar, and in the main menu, click on `Tools -> Start Output Monitor`.

1.3.3.1 The worksheet "FractionsInput"

The worksheet "FractionsInput " demonstrates the use of the Python data type `Fraction` in LibreOffice Calc spreadsheet formulas.



	A	B	C	D	E
1					
2		**Formatted as Double**		**Original Data**	
3		136986.300138694		123456789012345678 / 901234567890	
4		9.999999999E-12		123456789 / 01234567890123456789O	
5		0.00217395555782604		12345678901234 / 5678901234567890	
6		1386.96253953018		12345678901234567 / 8901234567890	
7		526315809944.599		123456789012345678901 / 234567890	
8		15.64700345799		1234567890123456 / 78901234567890	
9		3.57142968723526E-07		123456789012 / 345678901234567890	
10		2.17395603043452E+017		1234567890123456789012345 / 567890	
11		0.181847937116985		123456789012345 / 678901234567890	
12					
13					
14	**Prod:**	4.80254411022074E+017		109488241064252366394927246517420416158540118298557063498185512	
15	**Sum:**	2.173961293594E+017		488482060215448878927010156598583291789961336565827756419479920	
16					
17	**Median:**	15.64700345799		29394473574368/1878600823045	
18	**Average:**	2.41551254843778E+016		4884820602154488789270101565985832917899613365658277564194799200	
19	**Mean:**	2.41551254843778E+016		4884820602154488789270101565985832917899613365658277564194799200	
20	**Variance:**	4.66773527093429E+033		190890738689300151179994589724145861287222206854566634869440890	
21					
22	**Min:**	9.999999999E-12		1/100000000010	
23	**Max:**	2.17395603043452E+017		617283945061728394506 17/283945	
24					
25					

If we try to enter a number like `123456789012345678/901234567890`, i.e. a number with more than 16 digits into a spreadsheet cell, it will automatically be shortened to `1.23456789012346E+029`. In order to be able to enter such numbers into spreadsheet cells, the cells must first be formatted as text, and thereafter these numbers can be entered as text. This is mostly useful when working with Python Fractions and Decimals.

However, this kind of input is hard to read; it is also not directly usable for numerical spreadsheet functions. This can be changed by using the function `ASDOUBLE`, a shown in column `B`

As an example, worksheet "FractionsInput" contains a named range called "MpInputFractions" in column `A`, which can be used as input for descriptive statistics or linear algebra routines.

The module `npm` of XlCalcNet can be used to calculate desciptive statistics, like `prod`, `sum`, `mean`, `median`, `variance` with fractions. The function `npm.mean` can be called from a spreadsheet cell as follows:

```
=APY1("from xlcalcnet import npm, qpm $n x = npm.array(P1, dtype=qpm); result=str(npm.prod(x))",MpInputFractions)
```

Às before, we are using `$n` for newline and `$t` for indentation. The result is itself a `Fraction` (converted to a string), and is exact. The corresponding Python code with conventional formatting would look like this (where `P1` is the input range):

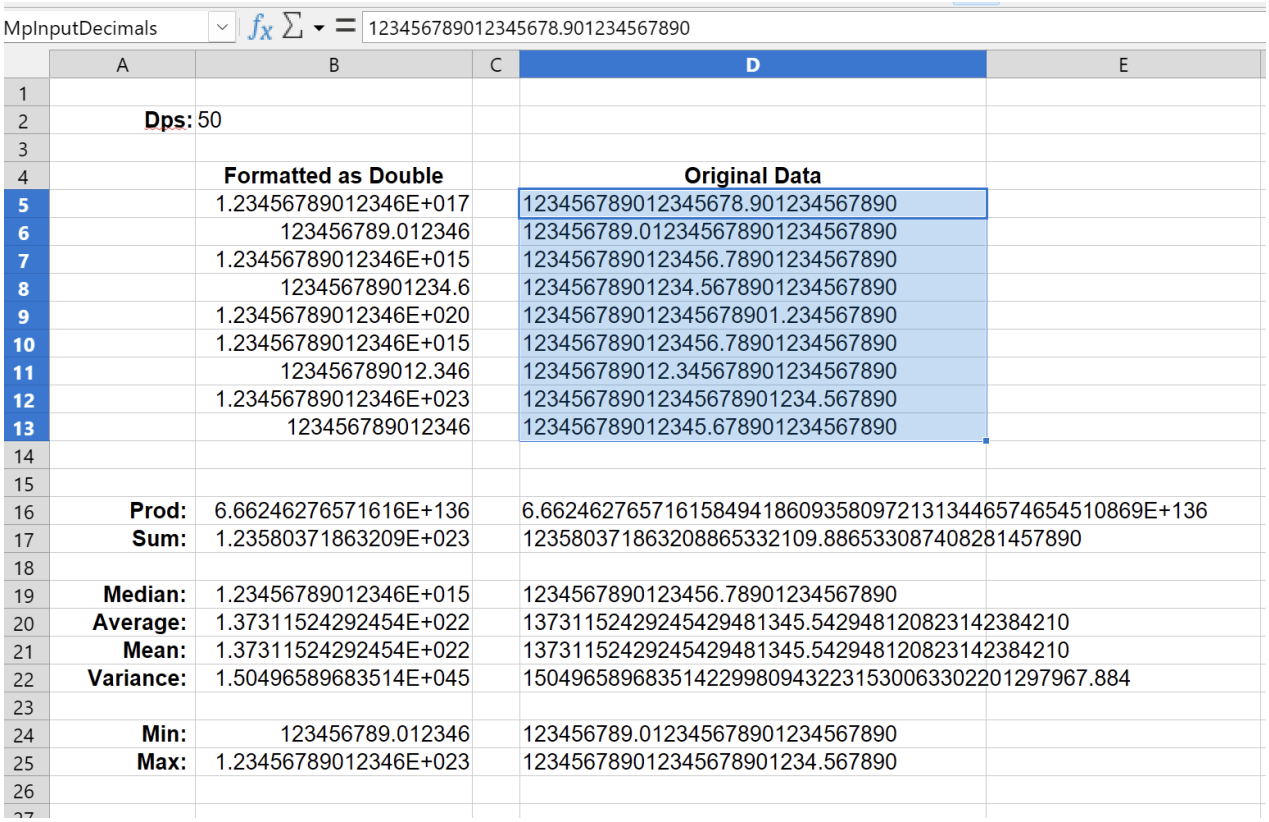
```
from xlcalcnet import npm, qpm
x = npm.array(P1, dtype=qpm)
result=str(npm.prod(x))
```

Note that the input range is converted into a Numpy array with data type `qpm`, which is the data type for rational numbers in XlCalcNet. The result is then converted into a string, so that it can be displayed in a spreadsheet cell.

Since the string output is quite long, we can use `result=float(npm.prod(x))` instead of `result=str(npm.prod(x))` to get a floating point number in double precision, which is more readable. However, this will not be exact anymore.

1.3.3.2 The worksheet "DecimalsInput"

The worksheet "DecimalsInput " demonstrates the use of the Python data type `Decimal` in LibreOffice Calc spreadsheet formulas.



	A	B	C	D	E
1					
2	**Dps:**	50			
3					
4		**Formatted as Double**		**Original Data**	
5		1.23456789012346E+017		12345678901234567.901234567890	
6		123456789.012346		123456789.01234567890123456789	
7		1.23456789012346E+015		1234567890123456.78901234567890	
8		12345678901234.6		12345678901234.5678901234567890	
9		1.23456789012346E+020		123456789012345678901.234567890	
10		1.23456789012346E+015		1234567890123456.78901234567890	
11		123456789012.346		123456789012.345678901234567890	
12		1.23456789012346E+023		123456789012345678901234.567890	
13		123456789012346		123456789012345.678901234567890	
14					
15					
16	**Prod:**	6.66246276571616E+136		6.66246276571615849418609358097213134465746545108 69E+136	
17	**Sum:**	1.23580371863209E+023		123580371863208865332109.886533087408281457890	
18					
19	**Median:**	1.23456789012346E+015		1234567890123456.78901234567890	
20	**Average:**	1.37311524292454E+022		13731152429245429481345.542948120823142384210	
21	**Mean:**	1.37311524292454E+022		13731152429245429481345.542948120823142384210	
22	**Variance:**	1.50496589683514E+045		150496589683514229980943223153006330220129796 7.884	
23					
24	**Min:**	123456789.012346		123456789.01234567890123456789	
25	**Max:**	1.23456789012346E+023		123456789012345678901234.567890	
26					

If we try to enter a number like `12345678901234567.901234567890`, i.e. a number with more than 16 digits into a spreadsheet cell, it will automatically be shortened to `1.23456789012346E+017`. In order to be able to enter such numbers into spreadsheet cells, the cells must first be formatted as text, and thereafter these numbers can be entered as text. This is mostly useful when working with Python Fractions and Decimals.

However, this kind of input is hard to read; it is also not directly usable for numerical spreadsheet functions. This can be changed by using the function `ASDOUBLE`, a shown in column `B`

As an example, worksheet "DecimalsInput" contains a named range called "MpInputDecimals" in column `C`, which can be used as input for descriptive statistics or linear algebra routines.

The module `npm` of XlCalcNet can be used to calculate desciptive statistics, like `prod`, `sum`, `mean`, `median`, `variance` with decimals. The function `npm.mean` can be called from a spreadsheet cell as follows:

```
=APY2("from xlcalcnet import npm, dpm $n dpm.dps=int(P2); x = npm.array(P1, dtype=dpm); result=str(npm.mean(x))",
↪MpInputDecimals, $B$2)
```

As before, we are using `$n` for newline and `$t` for indentation. The result is itself a `Decimal` (converted to a string), and is correct to the number of digits specified. The corresponding Python code with conventional formatting would look like this (where `P1` is the input range and `P2` is the number of digits):

```
from xlcalcnet import npm, dpm
dpm.dps=int(P2)
x = npm.array(P1, dtype=dpm)
result=str(npm.mean(x))
```

Note that the input range is converted into a Numpy array with data type `dpm`, which is the data type for decimal numbers in XlCalcNet. The result is then converted into a string, so that it can be displayed in a spreadsheet cell.

Since the string output is quite long, we can use `result=float(npm.prod(x))` instead of `result=str(npm.prod(x))` to get a floating point number in double precision, which is more readable. However, this will then be correct only to the precision of a double-precision floating-point number.

1.3.3.3 The worksheet "Programming"

The worksheet "Programming" demonstrates the use of small python scripts in LibreOffice Calc spreadsheet formulas.

	A	B	C	D
1				
2				
3	**Python standard library: simple loop**			
4	**n:**	45		
5	**Result:**	1980		
6				
7				
8				
9				
10	**Scipy: Numerical quadrature; f(x) = a*x**2 + b**			
11	**Integral start:**	-1.4		
12	**Integral end:**	1.5		
13	**Parameter a:**	2.1		
14	**Parameter b:**	1.1		
15	**Result (integral, error estimate):**	(7.473300000000001, 8.297029729931183e-14)		
16	**Result (integral only):**	7.4733		
17	**Result (error estimate only):**	8.29702973E-14		
18				
19				

The first example in this worksheeet shows how to program a simple loop. The formula in the cell `B4`. is

```
=APY1("temp = 0 $n for i in range(int(P1)): $n$t temp += i $n result = 2 * temp", B4)
```

The cell `B4` contains the value `45`; the result is `1980`. The corresponding Python code with conventional formatting would look like this(where `P1` is `n`, the number of iterations):

```
temp = 0
for i in range(int(P1)):
    temp += i
result = 2 * temp
```

The second example in this worksheet shows how to perform numerical integration.The formula in the cell `B15`. is

```
=APY4("from scipy.integrate import quad $n def integrand(x, a, b): return a*x**2 + b $n a = float(P3); b = float(P4);  $n I␣
↪= quad(integrand, float(P1), float(P2), args=(a,b))   $n result = str(I) ", B11, B12, B13, B14)
```

This formula calls the function `quad` from the module `scipy.integrate` in the package `scipy`. The parameters of this function are taken from the cells `B11` to `B14`. The result is a tuple containing the intergral and an error estimate. The corresponding Python code with conventional formatting would look like this (with P1 is the lower limit of integration, P2 is the upper limit, P3 is the coefficient of x^2, and P4 is the constant term):

```
from scipy.integrate import quad
def integrand(x, a, b): return a*x**2 + b
a = float(P3); b = float(P4)
I = quad(integrand, float(P1), float(P2), args=(a,b))
result = str(I)
```

The cells `B16` to `B17` show the same result as an array formula, with `B16` containing the integral and `B17` the error estimate.

1.3.3.4 The worksheet "OutputMonitorCSV"

This is a snapshot of the worksheet "OutputMonitorCSV" with the relevant formula in cell `B1`. The output of the formula in cell `B1` is a CSV file named `TTest1External.csv` which is written to the Output Monitor folder. The Output Monitor application must be running in order to see the output. The Output Monitor can be started from the main menu of the Tiny IDE by clicking on `Tools -> Start Output Monitor`.

	A	B	C	D	E	F	G	H
1	**Python Script:**	Students t-test, 1 sample						
2								
3	**n:**	[10, 20, 30]						
4	**mu0:**	1.0						
5	**mean:**	[4.5,4.7]						
6	**std:**	[1,2,3,4]						
7	**alpha:**	0.05						
8	**Options:**	,I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv'						
9								
10								
11								
12	**Note:**	The result is written to the Output Monitor folder (start the Output Monitor application first!)						
13								

The formula in the cell `B1`. is

```
=APY0(TEXTJOIN(" ",1,"from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_
↪PValues $n importlib.reload(D01_StudentT_PValues) $n D01_StudentT_PValues.stats_student_t_1sample_test(n=",B3,", mu0=",B4,
↪", mean=",B5,", std=",B6,", alpha=", B7, B8, ")"," $n result='Students t-test, 1 sample'"))
```

This formula calls the function `stats_student_t_1sample_test` from the module `D01_StudentT_PValues` in the package `A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample`. The parameters of this function are taken from the cells `B3` to `B8`. The result is a CSV file named `TTest1External.csv` which is written to the Output Monitor folder. The corresponding Python code with conventional formatting and substitution of the referenced cell values would look like this:

```
from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_PValues
importlib.reload(D01_StudentT_PValues)
D01_StudentT_PValues.stats_student_t_1sample_test(n=[10, 20, 30], mu0=1.0, mean=[4.5,4.7], std=[1,2,3,14], alpha=0.015,␣
↪I=True, D=True, T=True, C=True, Onesided=True, Twosided=True, OutputMode='csv')
result='Students t-test, 1 sample'
```

> **Note**
>
> Here we are using `importlib.reload` to make sure that any changes in the code of the module `D01_StudentT_PValues` which may have occured since the last use of this spreadsheet function become effective without restarting the Python interpreter.

The cell `A3` contains `="n:"`, and the cell `B3` contains `="[10, 20, 30]"`. This passes the list `[10, 20, 30]` as the parameter `n` to the function `stats_student_t_1sample_test`.

The cell `A4` contains `="mu0:"`, and the cell `B4` contains `="1.0"`. This passes the value `1.0` as the parameter `mu0` to the function `stats_student_t_1sample_test`.

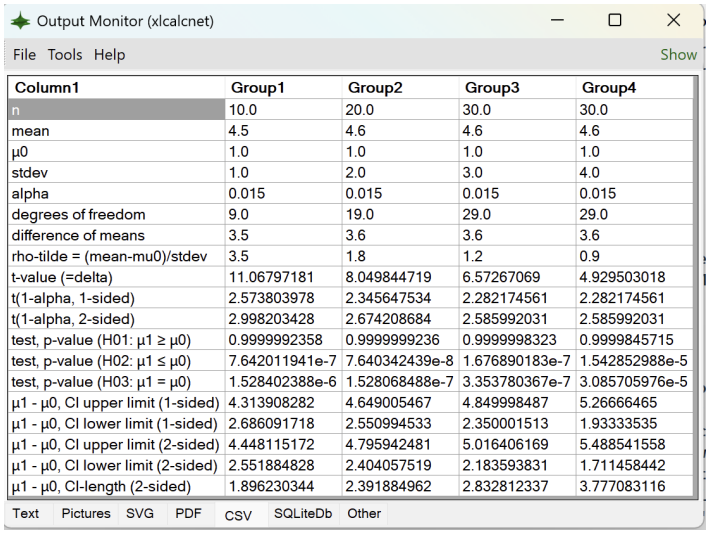
The cell `A5` contains `="mean:"`, and the cell `B5` contains `="[4.5,4.7]"`. This passes the list `[4.5,4.7]` as the parameter `mean` to the function `stats_student_t_1sample_test`.

The cell `A6` contains `="std:"`, and the cell `B6` contains `="[1,2,3,14]"`. This passes the list `[1,2,3,14]` as the parameter `std` to the function `stats_student_t_1sample_test`.

The cell `A7` contains `="alpha:"`, and the cell `B7` contains `="0.015"`. This passes the value `0.015` as the parameter `alpha` to the function `stats_student_t_1sample_test`.

The cell `A8` contains `="Options:"`, and the cell `B8` contains `=",I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv'"`. This passes the options `I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv'` as keyword arguments to the function `stats_student_t_1sample_test`.

And this is the Output Monitor showing the result (with the project panel hidden)



Column1	Group1	Group2	Group3	Group4
n	10.0	20.0	30.0	30.0
mean	4.5	4.6	4.6	4.6
μ0	1.0	1.0	1.0	1.0
stdev	1.0	2.0	3.0	4.0
alpha	0.015	0.015	0.015	0.015
degrees of freedom	9.0	19.0	29.0	29.0
difference of means	3.5	3.6	3.6	3.6
rho-tilde = (mean-mu0)/stdev	3.5	1.8	1.2	0.9
t-value (=delta)	11.06797181	8.049844719	6.57267069	4.929503018
t(1-alpha, 1-sided)	2.573803978	2.345647534	2.282174561	2.282174561
t(1-alpha, 2-sided)	2.998203428	2.674208684	2.585992031	2.585992031
test, p-value (H01: μ1 ≥ μ0)	0.9999992358	0.9999999236	0.9999998323	0.9999845715
test, p-value (H02: μ1 ≤ μ0)	7.642011941e-7	7.640342439e-8	1.676890183e-7	1.542852988e-5
test, p-value (H03: μ1 = μ0)	1.528402388e-6	1.528068488e-7	3.353780367e-7	3.085705976e-5
μ1 - μ0, CI upper limit (1-sided)	4.313908282	4.649005467	4.849998487	5.26666465
μ1 - μ0, CI lower limit (1-sided)	2.686091718	2.550994533	2.350001513	1.93333535
μ1 - μ0, CI upper limit (2-sided)	4.448115172	4.795942481	5.016406169	5.488541558
μ1 - μ0, CI lower limit (2-sided)	2.551884828	2.404057519	2.183593831	1.711458442
μ1 - μ0, CI-length (2-sided)	1.896230344	2.391884962	2.832812337	3.777083116

1.3.3.5 The worksheet "OutputMonitorSVG"

This is a snapshot of the worksheet "OutputMonitorSVG" with the relevant formula in cell `B1`. The output of the formula in cell `B1` is a SVG file named `D07_DistPlotContinuous.svg` which is written to the Output Monitor folder. The Output Monitor application must be running in order to see the output. The Output Monitor can be started from the main menu of the Tiny IDE by clicking on `Tools -> Start Output Monitor`.

	A	B	C	D	E	F	G
1	**Python Script:**	Beta distribution					
2							
3	**target:**	'pdf'	# 'pdf', 'cdf', 'sf', 'hf', 'chf', 'qtf', 'isf'				
4	**a:**	[5, 10.0, 20.5]					
5	**b:**	[20.5, 10.0, 5]					
6	**Title:**	'Beta distribution'					
7							
8							
9							
10	**Note:**	The result is written to the Output Monitor folder (start the Output Monitor application first!)					
11							

The formula in the cell `B1`. is

```
==APY0(TEXTJOIN(" ",1,"from A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C01_Intro_Parametric2D import D07_
↪DistPlotContinuous $n importlib.reload(D07_DistPlotContinuous) $n  D07_DistPlotContinuous.DistPlotBeta(target=",B3,", a=",
↪B4,", b=",B5,", Title=",B6,", OutputMode = 'svg'",  ")"," $n result='Beta distribution'"))
```

This formula calls the function `DistPlotBeta` from the module `D07_DistPlotContinuous` in the package `A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C01_Intro_Parametric2D`. The parameters of this function are taken from the cells `B3` to `B8`. The result is a CSV file named `TTest1External.csv` which is written to the Output Monitor folder. The corresponding Python code with conventional formatting and substitution of the referenced cell values would look like this:

```
from A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C01_Intro_Parametric2D import D07_DistPlotContinuous
importlib.reload(D07_DistPlotContinuous)
D07_DistPlotContinuous.DistPlotBeta(target='pdf', a=[5, 10.0, 20.5], b=[20.5, 10.0, 5], Title='Beta distribution',␣
↪OutputMode = 'svg')
result='Beta distribution'
```

> **Note**
>
> Here we are using `importlib.reload` to make sure that any changes in the code of the module `D07_DistPlotContinuous` which may have occured since the last use of this spreadsheet function become effective without restarting the Python interpreter.

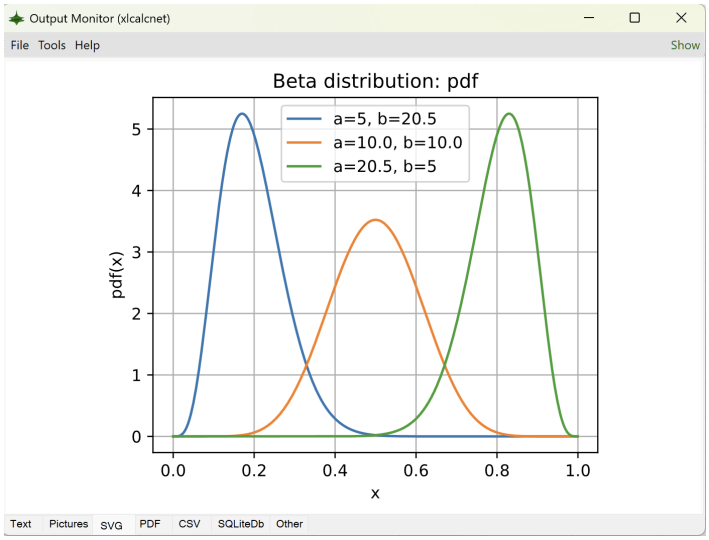
The cell A3 contains ="target:", and the cell B3 contains ="pdf". This passes the string "pdf" as the parameter target to the function DistPlotBeta.

The cell A4 contains ="a:", and the cell B4 contains ="[5, 10.0, 20.5]". This passes This passes the list [5, 10.0, 20.5] as the parameter a to the function DistPlotBeta.

The cell A5 contains ="b:", and the cell B5 contains ="[20.5, 10.0, 5]". This passes the list [20.5, 10.0, 5] as the parameter b to the function DistPlotBeta.

The cell A6 contains ="Title:", and the cell B6 contains ="Beta distribution". This passes the string "Beta distribution" as the keyword argument Title to the function DistPlotBeta.

And this is the Output Monitor showing the result (with the project panel hidden)



1.3.4 Using links to external *.csv, *.xlsx and *.svg files

Note

In the following subsections. we will work with the file LoDemoAPYext.ods, which contains more advanced examples using Numpy, Matplotlib, Pandas, Scipy, Seaborn and XlCalcNet. Examples for using LibreOffice Calc with the Python standard library start *here*.

Within LibreOffice Calc, open from the main menu File -> Open in the Documents folder the LibreOffice Calc workbook \DataXlCalcNet\DataExamples\MainExamples\Workbooks\LoDemoAPYext.ods.

This file contains several worksheets which demonstrate the use of links to external files in such a way that the output is displayed in the spreadsheet. This is demonstrated in the worksheets “ExternalCSV”, “ExternalXlsx” and “ExternalSVG”.

Using links to external files poses a security risk, because it allows the execution of arbitrary code on the local computer. This can be adressed by using “Trusted Locations”: In the main menu, click on Tools -> Options -> LibreOffice -> Security -> Macro Security.... In the dialog “Macro Security”, select the tab “Trusted Sources” and in the section “Trusted File Locations”, click “Add”, and add the path to the folder of the file **containing the links** to the external files (NOT the paths to the external files themselves!). In our case this is the file LoDemoAPYext.ods and the path to the folder containing this file is DataXlCalcNet\DataExamples\MainExamples\Workbooks). Confirm with OK.

Still in the Options dialog, go to LibreOffice Calc -> General. In the section “Update links when opening”, select the item “Always (from trusted locations)” and confirm with OK.

1.3.4.1 The worksheet “ExternalCSV”

The worksheet “ExternalCSV” demonstrates the use of a link to an external `*.csv` file in such a way that the output is displayed in the spreadsheet and is updated automatically when the input data changes. This mimics, to some extend, the use of dynamic arrays in Excel, which are not available in LibreOffice Calc at the time of writing (October 2026).

This is a snapshot of the worksheet “ExternalCSV” with the formula in cell `B1` and the output starting at cell `A10`. The output of the formula in cell `B1` is a CSV file named `TTest1External.csv` which is written to the Output Monitor folder.

	A	B	C	D	E	F
1	**Python Script:**	Students t-test, 1 sample				
2						
3	**n:**	[10, 20, 30]				
4	**mu0:**	1.0				
5	**mean:**	[4.5,4.7]				
6	**std:**	[1,2,3,14]				
7	**alpha:**	0.015				
8	**Options:**	,I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv'				
9						
10	Items	Group1	Group2	Group3	Group4	
11	n	10	20	30	30	
12	mean	4.5	4.7	4.7	4.7	
13	μ0	1	1	1	1	
14	stdev	1	2	3	4	
15	alpha	0.05	0.05	0.05	0.05	
16	degrees of freedom	9	19	29	29	
17	difference of means	3.5	3.7	3.7	3.7	
18	rho-tilde = (mean-mu0)/stdev	3.5	1.85	1.233333333	0.925	
19	t-value (=delta)	11.06797181	8.273451517	6.755244876	5.066433657	
20	t(1-alpha, 1-sided)	1.833112933	1.729132812	1.699127027	1.699127027	
21	t(1-alpha, 2-sided)	2.262157163	2.093024054	2.045229642	2.045229642	
22	test, p-value (H01: μ1 ≥ μ0)	0.9999992358	0.9999999493	0.9999998973	0.9999894555	
23	test, p-value (H02: μ1 ≤ μ0)	7.642011941E-07	5.068466643E-08	1.026891697E-07	1.0544453E-05	
24	test, p-value (H03: μ1 = μ0)	1.528402388E-06	1.013693329E-07	2.053783395E-07	2.1088906E-05	
25	μ1 - μ0, CI upper limit (1-sided)	4.079681208	4.473291702	4.630650201	4.940866934	
26	μ1 - μ0, CI lower limit (1-sided)	2.920318792	2.926708298	2.7693498	2.459133066	
27	μ1 - μ0, CI upper limit (2-sided)	4.215356906	4.636028813	4.82021841	5.193624547	
28	μ1 - μ0, CI lower limit (2-sided)	2.784643094	2.763971187	2.57978159	2.206375453	
29	μ1 - μ0, CI-length (2-sided)	1.430713812	1.872057626	2.240436821	2.987249094	
30						
31						

The formula in the cell `B1`. is

```
=APY0(TEXTJOIN(" ",1,"from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_
↪PValues $n importlib.reload(D01_StudentT_PValues) $n D01_StudentT_PValues.stats_student_t_1sample_test(n=",B3,", mu0=",B4,
↪", mean=",B5,", std=",B6,", alpha=", B7, B8, ")"," $n result='Students t-test, 1 sample'"))
```

This formula calls the function `stats_student_t_1sample_test` from the module `D01_StudentT_PValues` in the package `A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample`. The parameters of this function are taken from the cells `B3` to `B8`. The result is a CSV file named `TTest1External.csv` which is written to the Output Monitor folder. The corresponding Python code with conventional formatting and substitution of the referenced cell values would look like this:

```
from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_PValues
importlib.reload(D01_StudentT_PValues)
D01_StudentT_PValues.stats_student_t_1sample_test(n=[10, 20, 30], mu0=1.0, mean=[4.5,4.7], std=[1,2,3,14], alpha=0.015,␣
↪I=True, D=True, T=True, C=True, Onesided=True, Twosided=True, OutputMode='csv')
result='Students t-test, 1 sample'
```

> **Note**
>
> Here we are using `importlib.reload` to make sure that any changes in the code of the module `D01_StudentT_PValues` which may have occured since the last use of this spreadsheet function become effective without restarting the Python interpreter.

The cell `A3` contains `="n:"`, and the cell `B3` contains `="[10, 20, 30]"`. This passes the list `[10, 20, 30]` as the parameter `n` to the function `stats_student_t_1sample_test`.

The cell `A4` contains `="mu0:"`, and the cell `B4` contains `="1.0"`. This passes the value `1.0` as the parameter `mu0` to the function `stats_student_t_1sample_test`.

The cell A5 contains ="mean:", and the cell B5 contains ="[4.5,4.7]". This passes the list [4.5,4.7] as the parameter mean to the function stats_student_t_1sample_test.

The cell A6 contains ="std:", and the cell B6 contains ="[1,2,3,14]". This passes the list [1,2,3,14] as the parameter std to the function stats_student_t_1sample_test.

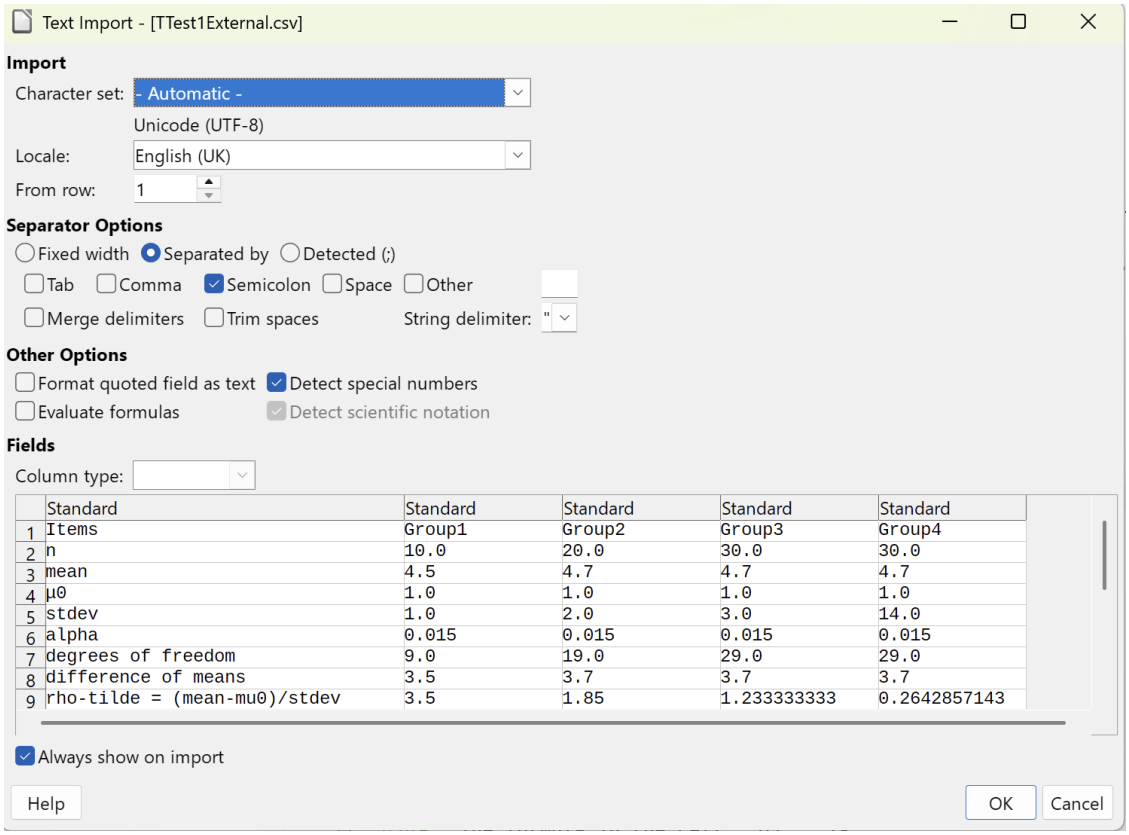
The cell A7 contains ="alpha:", and the cell B7 contains ="0.015". This passes the value 0.015 as the parameter alpha to the function stats_student_t_1sample_test.

The cell A8 contains ="Options:", and the cell B8 contains =",I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv'". This passes the options I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv' as keyword arguments to the function stats_student_t_1sample_test.

To have the output displayed in the spreadsheet, follow these steps:

Select the cell where the output should start to be displayed (in this case, cell A10). In the main menu, click on Sheet -> External Links.... In the dialog "External Data", click on Browse.... In the dialog "Insert", select the file TTest1External.csv in the Output Monitor folder and click on Open.

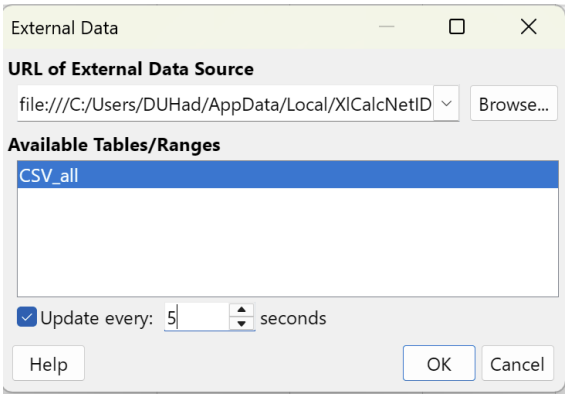
This will open the dialog "Text Import - [TTest1External.csv]".



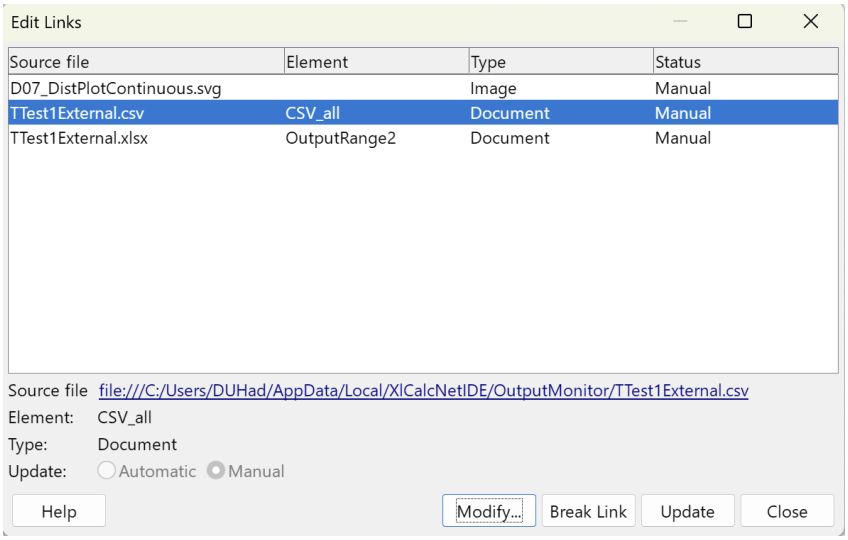
	Standard	Standard	Standard	Standard	Standard
1	Items	Group1	Group2	Group3	Group4
2	n	10.0	20.0	30.0	30.0
3	mean	4.5	4.7	4.7	4.7
4	μ0	1.0	1.0	1.0	1.0
5	stdev	1.0	2.0	3.0	14.0
6	alpha	0.015	0.015	0.015	0.015
7	degrees of freedom	9.0	19.0	29.0	29.0
8	difference of means	3.5	3.7	3.7	3.7
9	rho-tilde = (mean-mu0)/stdev	3.5	1.85	1.233333333	0.2642857143

Make sure that the option "Separated by" is selected, and that the option "Semicolon" is checked. Then click on OK.

Back in the dialog "External Data", check the box "Update every" and and set the time to 5 seconds. The output of the function stats_student_t_1sample_test will now be displayed in the spreadsheet, starting at the cell which was selected when opening the dialog "External Data". And because the option "Update every" is checked, the output will be updated every 5 seconds. If you change any of the parameters in cells B3 to B8, the output will be updated accordingly.



If later on you want to change the settings of the external link, click in the main menu on `Edit -> Links to External Files...`. In the dialog "Edit Links", select the link to the file `TTest1External.csv` and click on `Modify...`. This will open the dialog "External Data" again, where you can change the settings.



1.3.4.2 The worksheet "ExternalXlsx"

The worksheet "ExternalXlsx" demonstrates the use of a link to an external `*.xlsx` file in such a way that the output is displayed in the spreadsheet and is updated automatically when the input data changes. This mimics, to some extend, the use of dynamic arrays in Excel, which are not available in LibreOffice Calc at the time of writing (October 2026).

This is a snapshot of the worksheet "ExternalXlsx" with the formula in cell `B1` and the output starting at cell `A10`. The output of the formula in cell `B1` is an Excel file named `TTest1External.xlsx` which is written to the Output Monitor folder.

	A	B	C	D	E
1	**Python Script:**	Students t-test, 1 sample			
2					
3	**n:**	[10, 20, 30]			
4	**mu0:**	1.0			
5	**mean:**	[4.5,4.7]			
6	**std:**	[1,2,3,4]			
7	**alpha:**	0.015			
8	**Options:**	,I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='xlsx			
9					
10	**Item**	**Group1**	**Group2**	**Group3**	**Group4**
11	**n**	10.0	20.0	30.0	30.0
12	**mean**	4.5	4.7	4.7	4.7
13	**μ0**	1.0	1.0	1.0	1.0
14	**stdev**	1.0	2.0	3.0	4.0
15	**alpha**	0.015	0.015	0.015	0.015
16	**degrees of freedom**	9.0	19.0	29.0	29.0
17	**difference of means**	3.5	3.7	3.7	3.7
18	**rho-tilde = (mean-mu0)/stdev**	3.5	1.85	1.233333333	0.925
19	**t-value (=delta)**	11.06797181	8.273451517	6.755244876	5.066433657
20	**t(1-alpha, 1-sided)**	2.573803978	2.345647534	2.282174561	2.282174561
21	**t(1-alpha, 2-sided)**	2.998203428	2.674208684	2.585992031	2.585992031
22	**test, p-value (H01: μ1 ≥ μ0)**	0.9999992358	0.9999999493	0.9999998973	0.9999894555
23	**test, p-value (H02: μ1 ≤ μ0)**	7.642011941e-7	5.068466643e-8	1.026891697e-7	1.05444532e-5
24	**test, p-value (H03: μ1 = μ0)**	1.528402388e-6	1.013693329e-7	2.053783395e-7	2.108890639e-5
25	**μ1 - μ0, CI upper limit (1-sided)**	4.313908282	4.749005467	4.949998487	5.36666465
26	**μ1 - μ0, CI lower limit (1-sided)**	2.686091718	2.650994533	2.450001513	2.03333535
27	**μ1 - μ0, CI upper limit (2-sided)**	4.448115172	4.895942481	5.116406169	5.588541558
28	**μ1 - μ0, CI lower limit (2-sided)**	2.551884828	2.504057519	2.283593831	1.811458442
29	**μ1 - μ0, CI-length (2-sided)**	1.896230344	2.391884962	2.832812337	3.777083116
30					

The formula in the cell `B1`. is

```
=APY0(TEXTJOIN(" ",1,"from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_
↪PValues $n importlib.reload(D01_StudentT_PValues) $n D01_StudentT_PValues.stats_student_t_1sample_test(n=",B3,", mu0=",B4,
↪", mean=",B5,", std=",B6,", alpha=", B7, B8, ")"," $n result='Students t-test, 1 sample'"))
```

This formula calls the function `stats_student_t_1sample_test` from the module `D01_StudentT_PValues` in the package `A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample`. The parameters of this function are taken from the cells `B3` to `B8`. The result is a CSV file named `TTest1External.csv` which is written to the Output Monitor folder. The corresponding Python code with conventional formatting and substitution of the referenced cell values would look like this:

```
from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_PValues
importlib.reload(D01_StudentT_PValues)
D01_StudentT_PValues.stats_student_t_1sample_test(n=[10, 20, 30], mu0=1.0, mean=[4.5,4.7], std=[1,2,3,14], alpha=0.015,␣
↪I=True, D=True, T=True, C=True, Onesided=True, Twosided=True, OutputMode='xlsx')
result='Students t-test, 1 sample'
```

Note

Here we are using `importlib.reload` to make sure that any changes in the code of the module `D01_StudentT_PValues` which may have occured since the last use of this spreadsheet function become effective without restarting the Python interpreter.

The cell `A3` contains `="n:"`, and the cell `B3` contains `="[10, 20, 30]"`. This passes the list `[10, 20, 30]` as the parameter `n` to the function `stats_student_t_1sample_test`.

The cell `A4` contains `="mu0:"`, and the cell `B4` contains `="1.0"`. This passes the value `1.0` as the parameter `mu0` to the function `stats_student_t_1sample_test`.

The cell `A5` contains `="mean:"`, and the cell `B5` contains `="[4.5,4.7]"`. This passes the list `[4.5,4.7]` as the parameter `mean` to the function `stats_student_t_1sample_test`.

The cell `A6` contains `="std:"`, and the cell `B6` contains `="[1,2,3,14]"`. This passes the list `[1,2,3,14]` as the parameter `std` to the function `stats_student_t_1sample_test`.

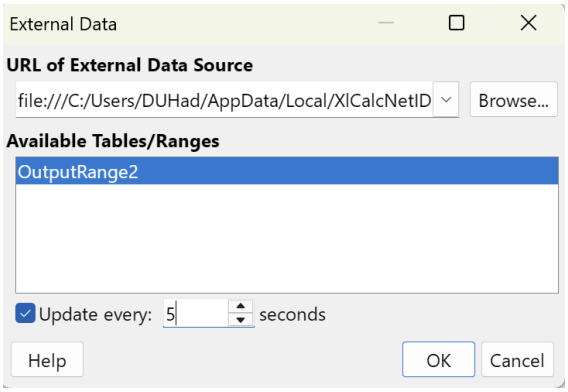
The cell `A7` contains `="alpha:"`, and the cell `B7` contains `="0.015"`. This passes the value `0.015` as the parameter `alpha` to the function `stats_student_t_1sample_test`.

The cell `A8` contains `="Options:"`, and the cell `B8` contains `=",I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='csv'"`. This passes the options `I=True, D=True, T=True, C=True, Onesided=True, Twosided = True, OutputMode='xlsx'` as keyword arguments to the function `stats_student_t_1sample_test`.

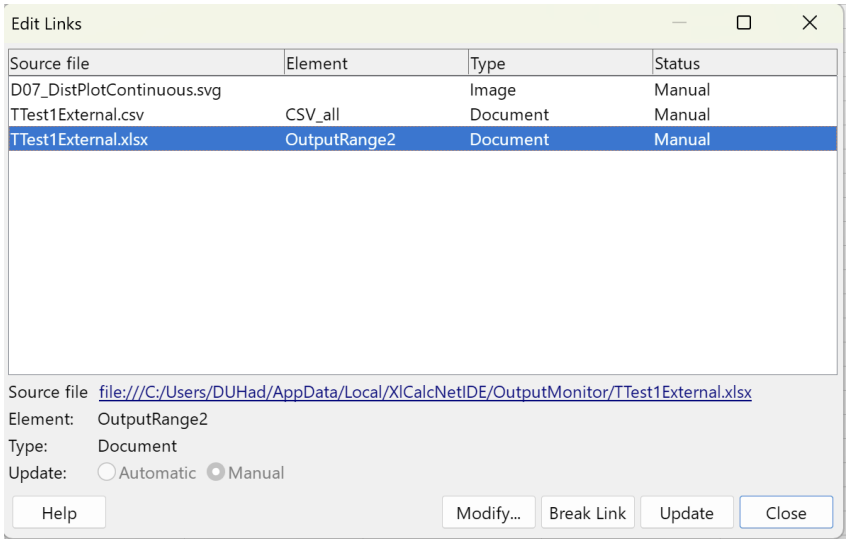
To have the output displayed in the spreadsheet, follow these steps:

Select the cell where the output should start to be displayed (in this case, cell `A10`). In the main menu, click on `Sheet -> External Links...`. In the dialog "External Data", click on `Browse...`. In the dialog "Insert", select the file `TTest1External.xlsx` in the Output Monitor folder and click on `Open`.

Back in the dialog "External Data", check the box "Update every" and and set the time to 5 seconds. The output of the function `stats_student_t_1sample_test` will now be displayed in the spreadsheet, starting at the cell which was selected when opening the dialog "External Data". And because the option "Update every" is checked, the output will be updated every 5 seconds. If you change any of the parameters in cells `B3` to `B8`, the output will be updated accordingly.



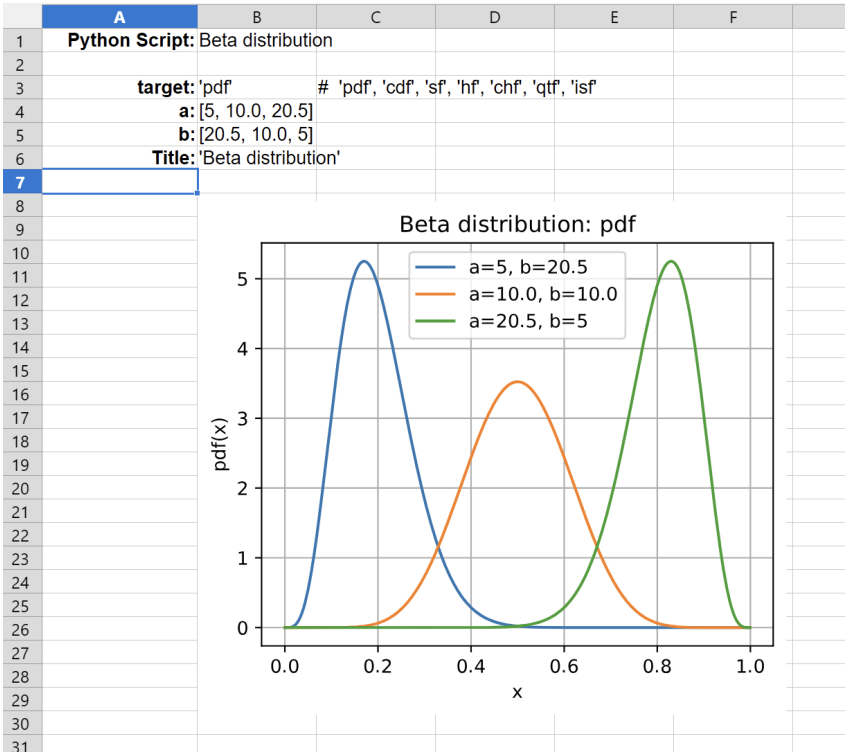
If later on you want to change the settings of the external link, click in the main menu on `Edit -> Links to External Files...`. In the dialog "Edit Links", select the link to the file `TTest1External.xlsx` and click on `Modify...`. This will open the dialog "External Data" again, where you can change the settings.



1.3.4.3 The worksheet "ExternalSVG"

The worksheet "ExternalSVG" demonstrates the use of a link to an external `*.svg` file in such a way that the output is displayed in the spreadsheet and is updated manually when the input data changes.

This is a snapshot of the worksheet "ExternalSVG" with the formula in cell `B1` and the output starting at cell `A10`. The output of the formula in cell `B1` is an SVG file named `D07_DistPlotContinuous.svg` which is written to the Output Monitor folder.



The formula in the cell `B1.` is

```
==APY0(TEXTJOIN(" ",1,"from A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C01_Intro_Parametric2D import D07_
↪DistPlotContinuous $n importlib.reload(D07_DistPlotContinuous) $n D07_DistPlotContinuous.DistPlotBeta(target=",B3,", a=",
↪B4,", b=",B5,", Title=",B6,", OutputMode = 'svg'",  ")"," $n result='Beta distribution'"))
```

This formula calls the function `DistPlotBeta` from the module `D07_DistPlotContinuous` in the package `A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C01_Intro_Parametric2D`. The parameters of this function are taken from the cells `B3` to `B8`. The result is a CSV file named `TTest1External.csv` which is written to the Output Monitor folder. The corresponding Python code with conventional formatting and substitution of the referenced cell values would look like this:

```
from A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C01_Intro_Parametric2D import D07_DistPlotContinuous
importlib.reload(D07_DistPlotContinuous)
D07_DistPlotContinuous.DistPlotBeta(target='pdf', a=[5, 10.0, 20.5], b=[20.5, 10.0, 5], Title='Beta distribution',␣
↪OutputMode = 'svg')
result='Beta distribution'
```

Note

Here we are using `importlib.reload` to make sure that any changes in the code of the module `D07_DistPlotContinuous` which may have occured since the last use of this spreadsheet function become effective without restarting the Python interpreter.

The cell `A3` contains `="target:"`, and the cell `B3` contains `="pdf"`. This passes the string `"pdf"` as the parameter `target` to the function `DistPlotBeta`.

The cell `A4` contains `="a:"`, and the cell `B4` contains `="[5, 10.0, 20.5]"`. This passes This passes the list `[5, 10.0, 20.5]` as the parameter `a` to the function `DistPlotBeta`.

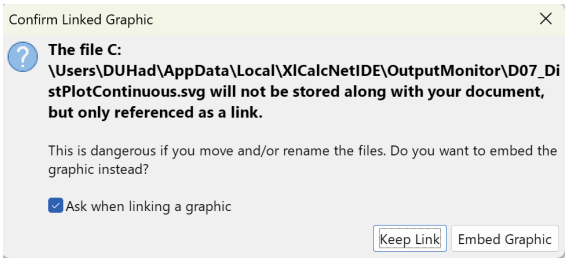
The cell `A5` contains `="b:"`, and the cell `B5` contains `="[20.5, 10.0, 5]"`. This passes the list `[20.5, 10.0, 5]` as the parameter `b` to the function `DistPlotBeta`.

The cell `A6` contains `="Title:"`, and the cell `B6` contains `="Beta distribution"`. This passes the string `"Beta distribution"`

as the keyword argument `Title` to the function `DistPlotBeta`.

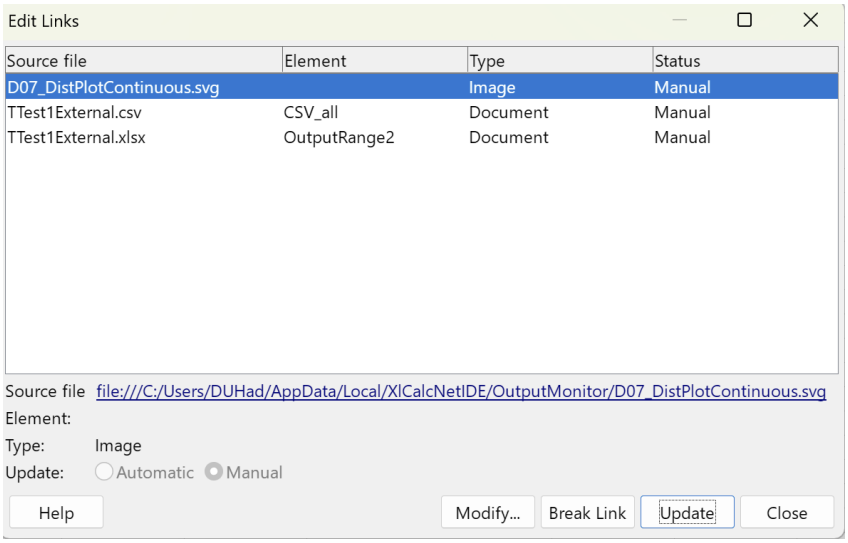
To have the output displayed in the spreadsheet, follow these steps:

In the main menu, click on `Insert -> Image...`. In the dialog "Insert Image", select `Anchor: To page` and then select the file `D07_DistPlotContinuous.svg` in the Output Monitor folder and click on `Open`.



The "Confirm Linked Graphic" dialog will appear. Click on `Keep Link`.

Unfortunately, LibreOffice Calc does not allow to update the linked graphic automatically when the input parameters have changed and the modified `*.svg` file has been written to the Output Monitor folder.

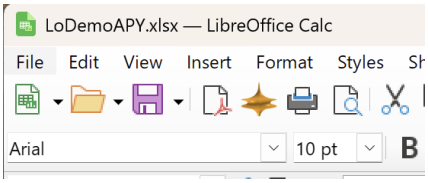


To manually update the linked graphic, click in the main menu on `Edit -> Links to External Files...`. In the dialog "Edit Links", select the link to the file `D07_DistPlotContinuous.svg` and click on `Update`. This will immediately display the updated graphic.

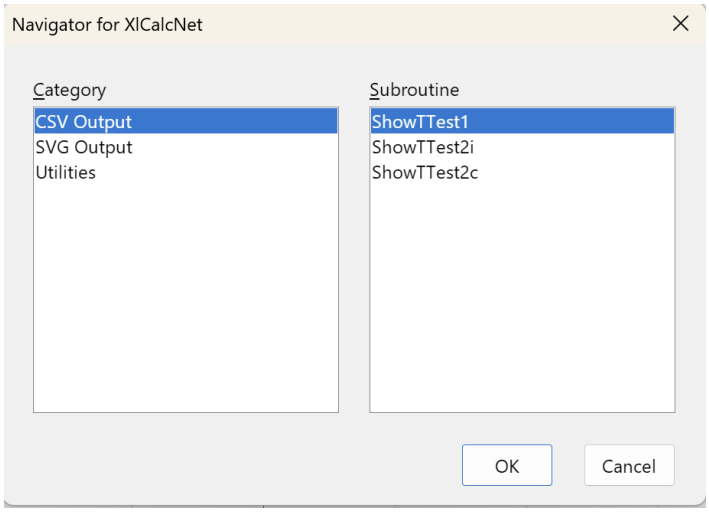
1.3.5 Managing procedures instead of functions

It is possible to call the spreadsheet functions `APY0` to `APY9` from LibreOffice Basic and to use this to start procedures instead of functions. A few simple examples have already been prepared to demonstrate the technique.

To access the relevant dialog, click on the XlCalcNet logo (in orange) on the main menu bar:



The following dialog box with the title **Navigator for XlCalcNet** will appear:



The first example, with `CSV Output` in "Category" and `ShowTTest1` in "Subroutine", will write a CSV file into the `XlCalcNetIDE\OutputMonitor` folder in the AppDataLocal folder. In order to see it, we need to open the `Output Monitor` application: start the `Tiny IDE` by clicking on its icon in the task bar, and in the main menu, click on `Tools -> Start Output Monitor`. With the `Output Monitor` application running, click `OK`. The CSV file will then immediately appear in the `Output Monitor`.

The corresponding code in the module `MpFunlabLocal -> BasicAndDialogs` is shown below:

```
Sub ShowTTest1()
    REM Output (a .csv file) is shown in output monitor
    fa = createUnoService("com.sun.star.sheet.FunctionAccess")
    Code = "from A06_UserlibExamplesPython.B29_InferentialStatistics.C01_BasicTests1Sample import D01_StudentT_PValues;"
    Code = Code + "D01_StudentT_PValues.demo_stats_student_t_1sample_test(); result='Done';"
    ResultArray = fa.callFunction("APY0", Array(Code))
    result = ResultArray(0)(0)
End Sub
```

The second example, with `SVG Output` in "Category" and `ShowPolygon` in "Subroutine", will write a SVG file into the `XlCalcNetIDE\OutputMonitor` folder in the AppDataLocal folder. With the `Output Monitor` application running (see above), click `OK`. The SVG file will then immediately appear in the `Output Monitor`.

The corresponding code in the module `MpFunlabLocal -> BasicAndDialogs` is shown below:

```
Sub ShowPolygon()
    REM Graphics is shown in output monitor
    fa = createUnoService("com.sun.star.sheet.FunctionAccess")
    Code = "from A01_ExamplesPython.B18_FunctionsAndCurvesPlots.C02_BasicCurves import D01_RegularConvexPolygon;"
    Code = Code + "D01_RegularConvexPolygon.RegularConvexPolygon(OutputMode='svg'); result='Done';"
    ResultArray = fa.callFunction("APY0", Array(Code))
    result = ResultArray(0)(0)
End Sub
```

1.3.6 Exporting and uninstalling

1.3.6.1 Exporting changes in MpfunlabLocal to MpfunlabLocal.oxt

Changes which are made in the MpfunlabLocal library are stored in the script folders of LibreOffice and are not automatically saved to MpfunlabLocal.oxt. However, they can be exported to MpfunlabLocal.oxt, so that they are available for installation (re-installation at the current LibreOffice installation or new installation for another LibreOffice installation).

To export the MpfunlabLocal library to MpfunlabLocal.oxt, follow these steps: open the Macro Editor from the main menu of LibreOffice Calc: `Tools -> Macros -> Edit Macros...`. In the main menu of the Macro Editor, select `Dialogs -> Organize Dialogs`. In the dialog "Basic Macro Organizer" select the tab "Libraries". "Location" should be set to "My Macros & Dialogs" and "Library: Name" should be "MpFunlabLocal". Click on `Export...`. In the following dialog box select "Export as extension", and

click on OK. In the dialog "Export library as extension" navigate to the documents folder, and within this folder to DataXlCalcNet\SpreadsheetAddins\LibreOfficeCalc. Select MpfunlabLocal.oxt and click on Save. Confirm to replace. In the dialog "Basic Macro Organizer" click on Close. Then close the Macro Editor.

1.3.6.2 Uninstalling MpfunlabLocal.oxt and Mpfunlab.oxt

To uninstall these two extensions properly, following the correct order is important.

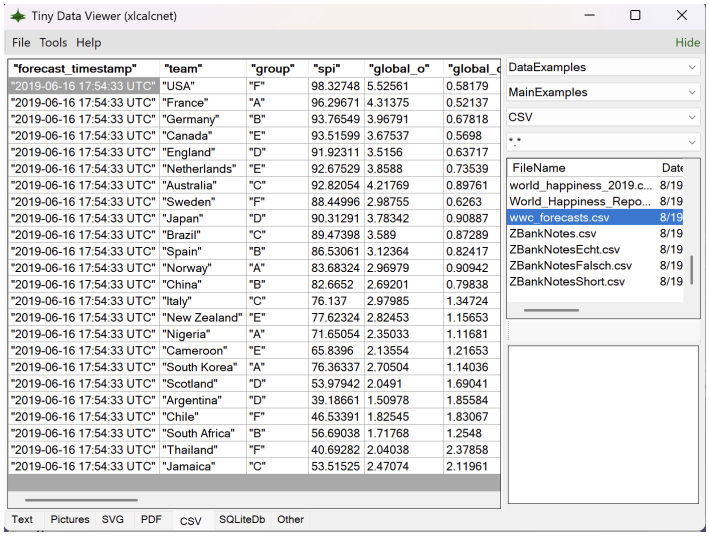
First start LibreOffice with administrator privileges: Right-click on the desktop icon of LibreOffice and select Run as administrator. In the following dialog, confirm that you want to proceed. In the LibreOffice desktop, open a Calc Spreadsheet. Open the Extension dialog from the main menu:Tools -> Extensions.... In the Extension dialog select MpFunlab.oxt, click the Remove button and click on OK in the confirmation dialog. Then click on the Close button in the Extension dialog. In the dialog Restart LibreOffice, click on Restart Now. This will close LibreOffice Calc and will open the general LibreOffice desktop. Exit LibreOffice.

Now uninstall MpfunlabLocal.oxt within a normally launched LibreOffice Calc (i.e. without administrator privileges): Open the Extension dialog from the main menu: Tools -> Extensions.... In the Extension dialog select MpfunlabLocal.oxt, click the Remove button and click on OK in the confirmation dialog. Then click on the Close button in the Extension dialog. In the dialog Restart LibreOffice, click on Restart Now. This will close LibreOffice Calc and will open the general LibreOffice desktop. Exit LibreOffice.

1.4 General user interface and functions

1.4.1 The Tiny Data Viewer

The "Tiny Data Viewer" application is started from the Tiny IDE: in the main menu, click on `Tools -> Tiny Data Viewer`. The screenshot below shows an example



The "Tiny Data Viewer" application is designed around the structure of the `DataXlCalcNet` folder, making it easy to navigate and manage existing data sources, as well a new data provided by the user. The Tiny IDE can be used only to view data sources which are located in the `DataXlCalcNet` folder and are located in subfolders beginning with `Data`. The data source examples are organized in a tree structure, which is shown in the right panel (the "folder panel") of the "Tiny Data Viewer" application.

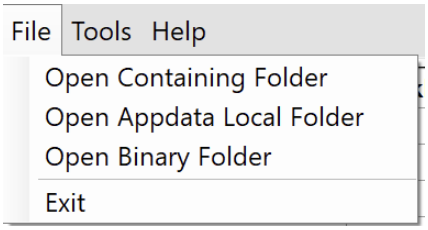
The left panel (the "data panel") of the "Gallery of Plots" application displays the data of the selected file. This panel has 7 tabs: "Text", "Pictures", "SVG", "PDF", "CSV", "SQLiteDb", "Other".

Each file in the `DataXlCalcNet\Data*` folders can be opened in the "Tiny Data Viewer" application by clicking on it. Per application instance, only one file can be opened and displayed at a time (to open an additional file, you need to start a new instance of the application: `Tools -> Tiny Data Viewer (external)`).

1.4.1.1 The menu bar of the application

The Menubar of the "Tiny Data Viewer" application contains the following menus: `File`, `Tools`, and `Help`. Each menu contains a number of items, which are described below.

There are also a button (with green foreground color) in the menu bar: the "Show/Hide" button, which toggles the visibility of the folder panel.



In the `File` menu, two standard items are missing: "New" and "Open".

The "New" item is missing because the application is designed to view existing data, not to create new data.

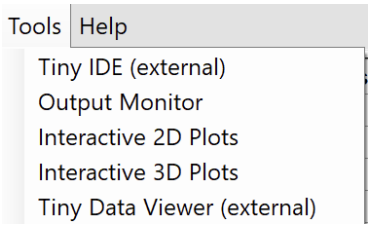
The "Open" item (which would typically open a dialog box with the functionally of the Windows Explorer) is missing because the "Tiny Data Viewer" application is limited to the `DataXlCalcNet` folder. Instead the folder panel allows to navigate through the `DataXlCalcNet` folder and open any file in it.

Renaming or deleting files and folders is possible, but should be done with care, using the Windows Explorer. After these changes, when returning to the Tiny IDE, the tree structure will not be updated automatically, but has to be updated manually by clicking on the appropriate parent folder in the tree structure.

`Open Containing Folder` opens the Windows Explorer in the folder of the current file. This is useful for quickly accessing other files in the same folder, or for deleting files and folders.

`Open Appdata Local Folder` opens the Windows Explorer in the `AppData\Local\XlCalcNetIDE` folder. This is useful for quickly accessing files and folders which are generated by running Python or C# code examples in the Tiny IDE.

`Open Binary Folder` opens the Windows Explorer in the `Bin` subfolder of the `XlCalcNet` installation. This is useful mostly for debugging purposes.



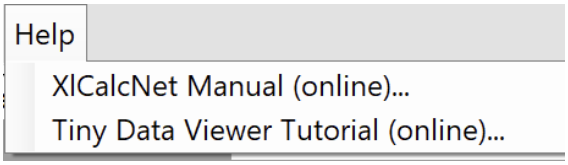
`Tiny IDE (external)` starts the Tiny IDE application, see *here* for details.

`Output Monitor` starts the "Output Monitor" application, see *here* for details.

`Gallery of Plots` starts the "Gallery of Plots" application, see *here* for details.

`Interactive 3D Wpf Plots` starts an additional the Interactive 3D Wpf Plots application, see *here* for details.

`Tiny Data Viewer (external)` starts an additional instance of the Tiny Data Viewer application, see *here* for details.



In the `Help` menu, `XlCalcNet Manual (online)...` displays the start page of the XlCalcNet online manual in a web browser.

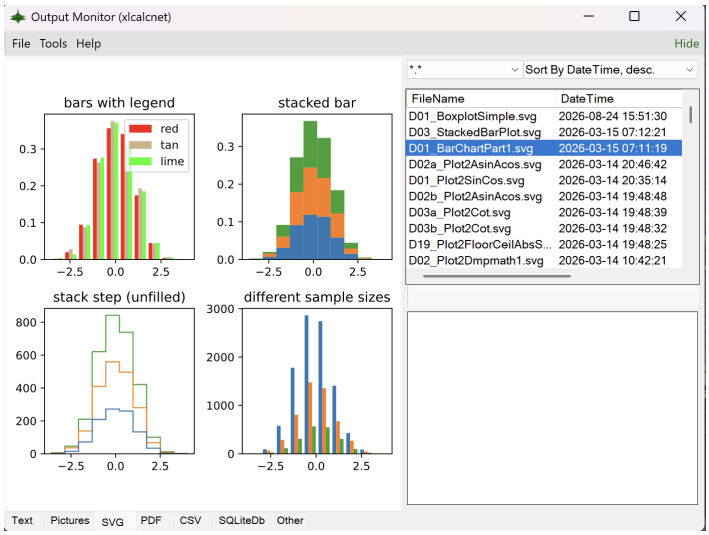
`Tiny Data Viewer Tutorial (online)...` displays the section of the XlCalcNet.online manual which explains the use of the Tiny Data Viewer application in a web browser.

1.4.1.2 Source code

The C# source code for the data viewer can be found here: https://github.com/duhadler/XlCalcNet/tree/master/xlcalcnet/Addin/NET48/Source/DataViewer

1.4.2 The Output Monitor

The "Output Monitor" application is started from the Tiny IDE: in the main menu, click on `Tools -> Output Monitor`. The screenshot below shows an example



The "Output Monitor" application is designed to be used together with spreadsheet functions, when the output is a table or a graphics file, and it it not convenient to incorporate the output into the spreadsheet right away.

The folder which is monitored is `PathToLocalAppDara\\XlCalcNetIDE\OutputMonitor`. Whenever a file changes or is newly created in this folder, the "Output Monitor" application is updated.

The Tiny IDE can be used only to view data which are located in the `OutputMonitor` folder. The files in this folder are organized in a tree structure, which is shown in the right panel (the "folder panel") of the "Output Monitor" application.

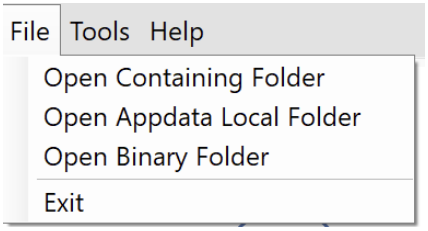
The left panel (the "data panel") of the "Gallery of Plots" application displays the data of the selected file. This panel has 7 tabs: "Text", "Pictures", "SVG", "PDF", "CSV", "SQLiteDb", "Other".

Each file in the `DataXlCalcNet\Data*` folders can be opened in the "Output Monitor" application by clicking on it. Per application instance, only one file can be opened and displayed at a time (to open an additional file, you need to start a new instance of the application: `Tools -> Output Monitor (external)`).

1.4.2.1 The menu bar of the application

The Menubar of the "Output Monitor" application contains the following menus: `File`, `Tools`, and `Help`. Each menu contains a number of items, which are described below.

There are also a button (with green foreground color) in the menu bar: the "Show/Hide" button, which toggles the visibility of the folder panel.



In the `File` menu, two standard items are missing: "New" and "Open".

The "New" item is missing because the application is designed to view existing data, not to create new data.

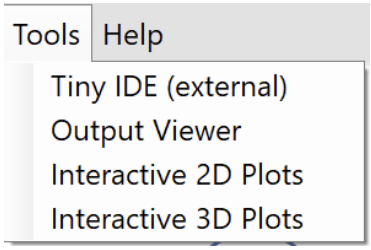
The "Open" item (which would typically open a dialog box with the functionally of the Windows Explorer) is missing because the "Output Monitor" application limited to the `OutputMonitor` folder. Instead the folder panel allows to navigate through the `DataXlCalcNet` folder and open any file in it.

Renaming or deleting files and folders is possible, but should be done with care, using the Windows Explorer. After these changes, when returning to the "Output Monitor" application, the "Output Monitor" application is updated automatically.

`Open Containing Folder` opens the Windows Explorer in the folder of the current file. This is useful for quickly accessing other files in the same folder, or for deleting files.

`Open Appdata Local Folder` opens the Windows Explorer in the `AppData\Local\XlCalcNetIDE` folder. This is useful for quickly accessing files and folders which are generated by running Python or C# code examples in the Tiny IDE.

`Open Binary Folder` opens the Windows Explorer in the `Bin` subfolder of the `XlCalcNet` installation. This is useful mostly for debugging purposes.



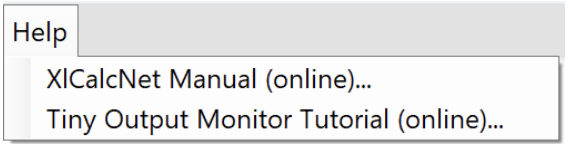
`Tiny IDE` starts the Tiny IDE application, see *here* for details.

`Output Monitor (external)` starts an additional instance of the the "Output Monitor" application, see *here* for details.

`Gallery of Plots` starts the gallery of plots application, see *here* for details.

`Interactive 3D Wpf Plots` starts an additional the Interactive 3D Wpf Plots application, see *here* for details.

`Tiny Data Viewer` starts the Tiny Data Viewer application, see *here* for details.



In the `Help` menu, `XlCalcNet Manual (online)...` displays the start page of the XlCalcNet online manual in a web browser.

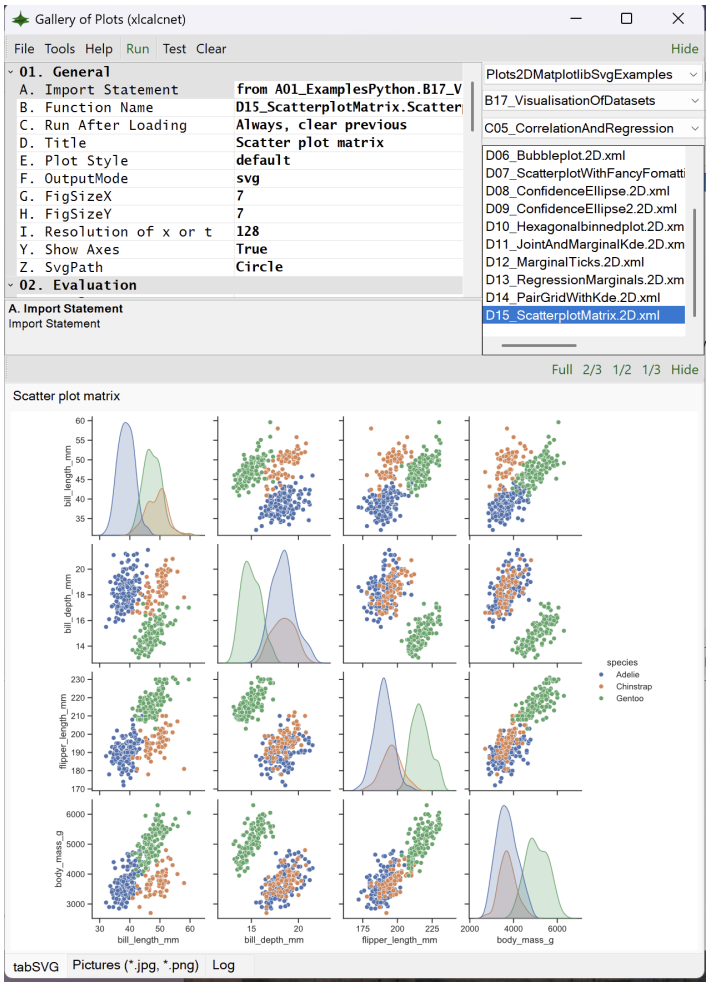
`Tiny Data Viewer Tutorial (online)...` displays the section of the XlCalcNet.online manual which explains the use of the Tiny Data Viewer application in a web browser.

1.4.2.2 Source code

The C# source code for the output monitor can be found here: https://github.com/duhadler/XlCalcNet/tree/master/xlcalcnet/Addin/NET48/Source/OutputMonitor

1.4.3 The Gallery of Plots

The "Gallery of Plots" application is started from the Tiny IDE: in the main menu, click on `Tools -> Gallery of Plots`. The screenshot below shows an example



The "Gallery of Plots" application is designed around the structure of the `DataXlCalcNet` folder, making it easy to navigate and manage existing plot definition examples, as well a new plot definitions written by the user. This application can be used only to edit and run plot definition examples which are located in the `DataXlCalcNet` folder and are located in subfolders beginning with `GalleryOfPlots`. The plot definition examples are organized in a tree structure, which is shown in the top right panel (the "folder panel") of the "Gallery of Plots" application.

The top left panel (the "plot definition editor panel") of the application shows the plot definition of the selected file, which can be edited and run. The plot definition is stored as an *.xml* file, but is edited in a property grid. This allows to structure the definition and to use of drop down boxes to make the definition of the plot easier.

The output of the program is shown in the bottom panel (the "output panel") of the application, which has 3 tabs: the "SVG" tab, the "Pictures" tab, the "Log" tab.

Each file in the `DataXlCalcNet\GalleryOfPlots*` folders can be opened in the application by clicking on it. Per application instance, only one file can be opened, edited and run at a time (to open an additional file, you need to start a new instance of the "Gallery of Plots" application: `Tools -> "Gallery of Plots" application (external)`). The application will load this file into the editor, and the plot definition can be edited and run. To run the plot definition, click on the "Run" button in the main menu. The output is shown in the output panel.

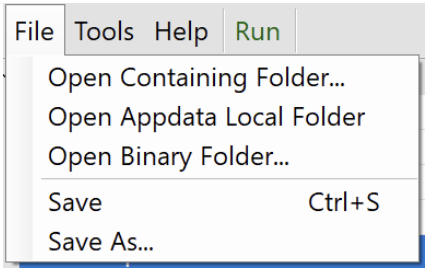
The ordering and naming of the code examples in the subfolder `Plots3DWpfExamples` match the ordering and naming of the corre-

sponding chapters, sections and subsections in this manual, making it easy to find the code example which corresponds to a specific section of the manual. The "section help" feature (`Help -> Section Help`), which is available in the main menu of the "Gallery of Plots" application and opens the corresponding section of this manual in a web browser, depends on this ordering and naming.

1.4.3.1 The menu bar of the application

The Menubar of the "Gallery of Plots" application contains the following menus: `File`, `Tools`, and `Help`. Each menu contains a number of items, which are described below.

There are also two buttons (with green foreground color) in the menu bar: the "Run" button, which runs the code of the current file, and the "Show/Hide" button, which toggles the visibility of the folder panel.



In the `File` menu, two standard items are missing: "New" and "Open".

The "New" item is missing because the Tiny IDE is designed to work within the `DataXlCalcNet` folder, and therefore new files which are not in the current folder should be created in the appropriate subfolders of `DataXlCalcNet`, which often have to be created manually first using the Windows Explorer.

There is, however, a `Save As` item in the file menu, which allows to save the current file under a new name in the current folder. This is useful for creating a copy of an existing file, and using this as a starting point for a new file.

The "Open" item (which would typically open a dialog box with the functionally of the Windows Explorer) is missing because the Tiny IDE is limited to the `DataXlCalcNet` folder. Instead the folder panel allows to navigate through the `DataXlCalcNet` folder and open any file in it.

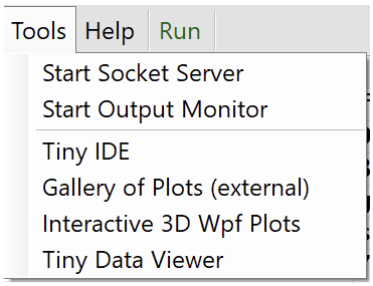
Renaming or deleting files and folders is possible, but should be done with care, using the Windows Explorer, and not using the Tiny IDE. After these changes, when returning to the Tiny IDE, the tree structure will not be updated automatically, but has to be updated manually by clicking on the appropriate parent folder in the tree structure.

`Open Containing Folder` opens the Windows Explorer in the folder of the current file. This is useful for quickly accessing other files in the same folder, or for deleting files and folders.

`Open Appdata Local Folder` opens the Windows Explorer in the `AppData\Local\XlCalcNetIDE` folder. This is useful for quickly accessing files and folders which are generated by running Python or C# code examples in the Tiny IDE.

`Open Binary Folder` opens the Windows Explorer in the `Bin` subfolder of the `XlCalcNet` installation. This is useful mostly for debugging purposes.

`Save` will just save the current file without any further action. `Save As` has been descibed above. `Exit` will close the Tiny IDE.



`Start Socket Server` starts the socket server, see *here* for details.

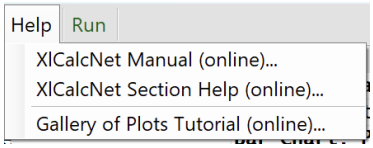
`Start Output Monitor` starts the output monitor application, see *here* for details.

`Tiny IDE` starts the Tiny IDE application, see *here* for details.

`Gallery of Plots` starts the gallery of plots application, see *here* for details.

`Interactive 3D Wpf Plots` starts the Interactive 3D Wpf Plots application, see *here* for details.

`Tiny Data Viewer` starts the Tiny Data Viewer application, see *here* for details.



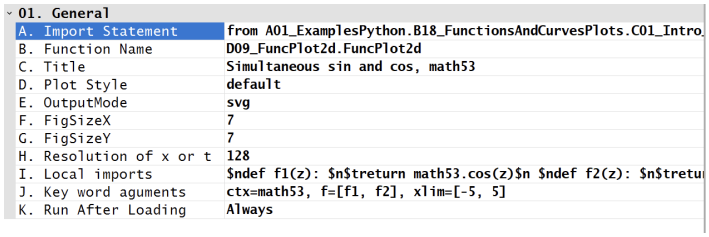
In the `Help` menu, `XlCalcNet Manual (online)...` displays the start page of the XlCalcNet online manual in a web browser.

`XlCalcNet Section Help (online)...` displays the section of the XlCalcNet.online manual which corresponds to the current file in a web browser.

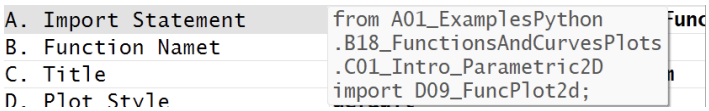
`Interactive 3D plots Tutorial (online)...` displays the section of the XlCalcNet.online manual which explains the use of the Interactive 3D plots application in a web browser.

1.4.3.2 The General section in the edtior property grit

As described above, the plot definition is edited in a property grid:



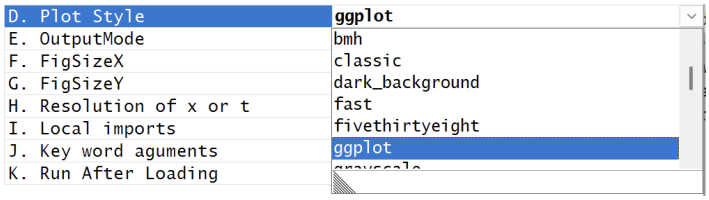
The example above shows the plot definition of the file `\GalleryOfPlotsExamples\B18_FunctionsAndCurvesPlots\C01_Intro_Parametric2D\D09b_SinCosMath53.2D.xml`.



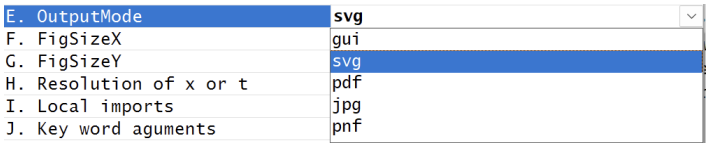
A. Import Statement contains the import statement.

B. Function Name contains the function name.

C. Title contains the function title.



D. PlotStyle contains the plotstyle (see above).

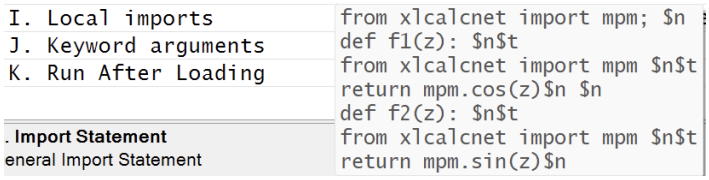


E. OutputMode contains the Output Mode (see above).

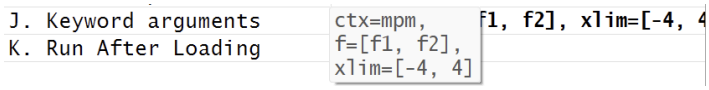
G. FigSizeX contains the size of the figure in x (horizontal) direction.

F. FigSizeY contains the size of the figure in y (vertical) direction.

H. Resolution of x or t contains the resolution of x or t.

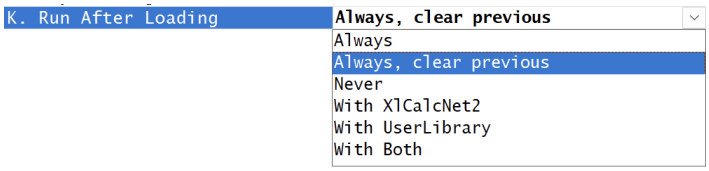


I. Local imports contains the local imports and definitions before the function call.



J. Keyword arguments contains the which passed to function.

Some text



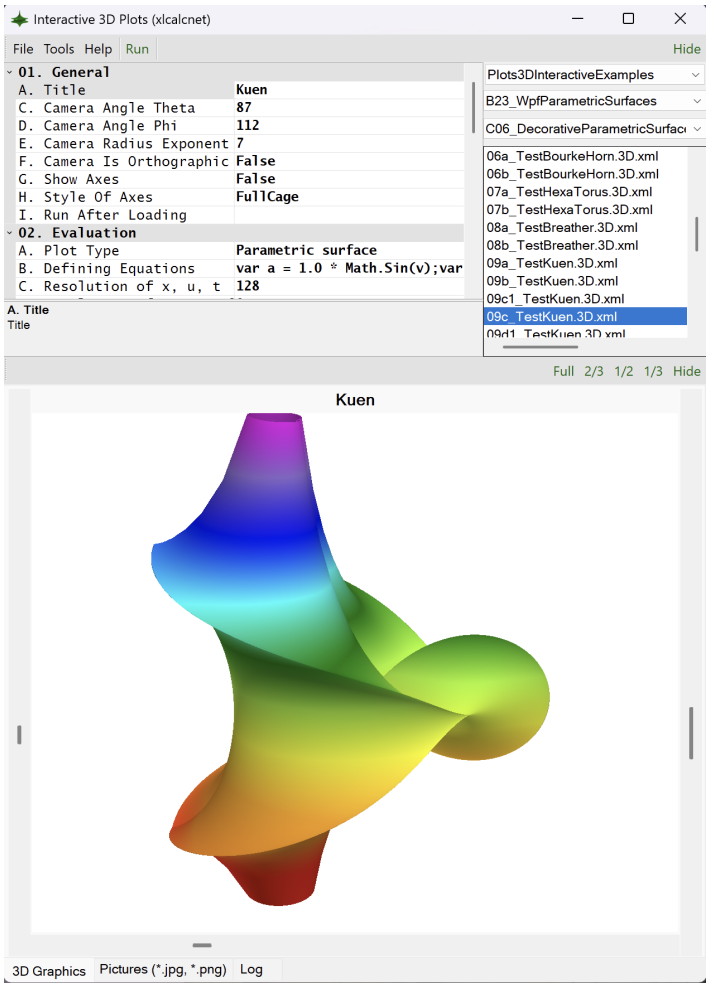
K. Run After loading determines whether and how the plot definition is run after loading (see above).

1.4.3.3 Source code

The C# source code for the gallery of plots can be found here: https://github.com/duhadler/XlCalcNet/tree/master/xlcalcnet/Addin/NET48/Source/TinyPlot2D

1.4.4 Interactive 3D wpf plots

The "Interactive 3D Wpf Plots" application is started from the Tiny IDE: in the main menu, click on `Tools -> Interactive 3D Wpf Plots`. The screenshot below shows an example



The "Interactive 3D Wpf Plots" application is designed around the structure of the `DataXlCalcNet` folder, making it easy to navigate and manage existing 3D plot definition examples, as well a new definitions written by the user. This application can be used only to edit and run 3D plot definition examples which are located in the `DataXlCalcNet` folder and are located in subfolders beginning with `Plots3D`. The code examples are organized in a tree structure, which is shown in the top right panel (the "folder panel") of the application.

The top left panel (the "3D plot definition editor panel") of the application shows the 3D plot definition of the selected file, which can be edited and run. The 3D plot definition is stored as an *.xml* file, but is edited in a property grid. This allows to structure the definition in the sections `0.1 General`, `02. Evaluation`, `03. Reshaping`, `Formatting`, and to use drop down menus to make the definition of the 3D plot easier.

The output of the program is shown in the bottom panel (the "output panel") of the application, which has 3 tabs: the "3D graphics" tab, which displays the interactive 3D plot; the "Pictures" tab, which shows available pictures which can be used as textures; and the "Log" tab, which documents the progress of compiling and running the code containing the "Defining Equations" and the "Expression Evaluator" (and also displays warning and error messages).

Each file in the `DataXlCalcNet\Plots3D*` folders can be opened in the application by clicking on it. Per application instance, only one file can be opened, edited and run at a time (to open an additional file, you need to start a new instance of the application: `Tools -> Interactive 3D Wpf Plots`). The application will load this file into the 3D plot definition editor, and the 3D plot definition can be edited and run. To run the 3D plot definition, click on the "Run" button in the main menu. The output is shown in the output

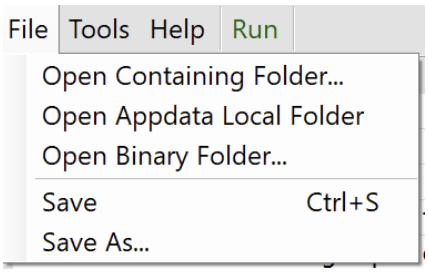
panel.

The ordering and naming of the 3D plot definition examples in the subfolder `Plots3DWpfExamples` match the ordering and naming of the corresponding chapters, sections and subsections in this manual, making it easy to find the 3D plot definition example which corresponds to a specific section of the manual. The "section help" feature (`Help -> Section Help`), which is available in the main menu of the application and opens the corresponding section of this manual in a web browser, depends on this ordering and naming.

1.4.4.1 The menu bar of the application

The Menubar of the "Interactive 3D Wpf Plots" application contains the following menus: `File`, `Tools`, and `Help`. Each menu contains a number of items, which are described below.

There are also two buttons (with green foreground color) in the menu bar: the "Run" button, which runs the code of the current file, and the "Show/Hide" button, which toggles the visibility of the folder panel.



In the `File` menu, two standard items are missing: "New" and "Open".

The "New" item is missing because the Tiny IDE is designed to work within the `DataXlCalcNet` folder, and therefore new files which are not in the current folder should be created in the appropriate subfolders of `DataXlCalcNet`, which often have to be created manually first using the Windows Explorer.

There is, however, a `Save As` item in the file menu, which allows to save the current file under a new name in the current folder. This is useful for creating a copy of an existing file, and using this as a starting point for a new file.

The "Open" item (which would typically open a dialog box with the functionally of the Windows Explorer) is missing because the Tiny IDE is limited to the `DataXlCalcNet` folder. Instead the folder panel allows to navigate through the `DataXlCalcNet` folder and open any file in it.

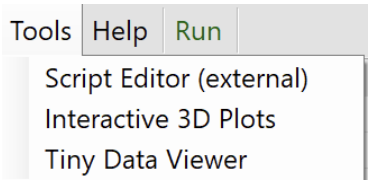
Renaming or deleting files and folders is possible, but should be done with care, using the Windows Explorer, and not using the Tiny IDE. After these changes, when returning to the Tiny IDE, the tree structure will not be updated automatically, but has to be updated manually by clicking on the appropriate parent folder in the tree structure.

`Open Containing Folder` opens the Windows Explorer in the folder of the current file. This is useful for quickly accessing other files in the same folder, or for deleting files and folders.

`Open Appdata Local Folder` opens the Windows Explorer in the `AppData\Local\XlCalcNetIDE` folder. This is useful for quickly accessing files and folders which are generated by running Python or C# code examples in the Tiny IDE.

`Open Binary Folder` opens the Windows Explorer in the `Bin` subfolder of the `XlCalcNet` installation. This is useful mostly for debugging purposes.

`Save` will just save the current file without any further action. `Save As` has been descibed above. `Exit` will close the Tiny IDE.



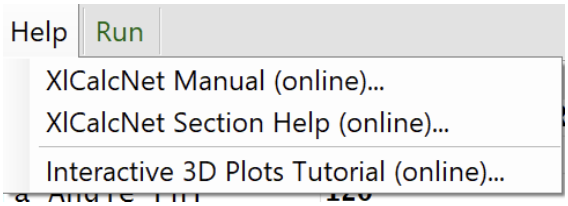
`Tiny IDE` starts the Tiny IDE application, see *here* for details.

`Output Monitor (external)` starts an additional instance of the the "Output Monitor" application, see *here* for details.

`Gallery of Plots` starts the gallery of plots application, see *here* for details.

`Interactive 3D Wpf Plots (external)` starts an additional the Interactive 3D Wpf Plots application, see *here* for details.

`Tiny Data Viewer` starts the Tiny Data Viewer application, see *here* for details.

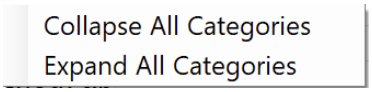


In the `Help` menu, `XlCalcNet Manual (online)...` displays the start page of the XlCalcNet online manual in a web browser.

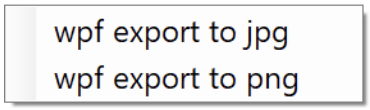
`XlCalcNet Section Help (online)...` displays the section of the XlCalcNet.online manual which corresponds to the current file in a web browser.

`Interactive 3D plots Tutorial (online)...` displays the section of the XlCalcNet.online manual which explains the use of the Interactive 3D plots application in a web browser.

1.4.4.2 The context menus of the 3D plot definition editor



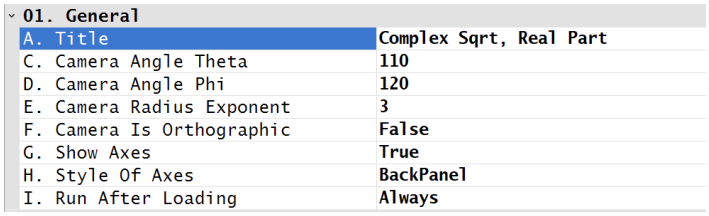
Right-clicking in the 3D plot definition editor opens a context menu, which provides quick access to



Right-clicking in the 3D Graphics tab in the output panel opens a context menu, which provides quick access to

1.4.4.3 The General section in the edtior property grit

The General section in the edtior property grit contains the following items:



A. Title contains the title of the graphics

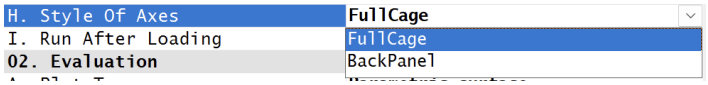
C. Camera Angle Theta contains the camera angle theta.

D. Camera Angle Phi contains the camera angle phi.

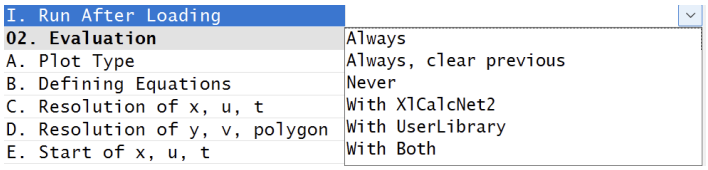
E. Camera Radius Exponent contains the camera radius exponent.

F. Camera Is Orthographic determines if the camera is orthographic or perspective.

G. Show Axes determines if axes are shown in the graphics.



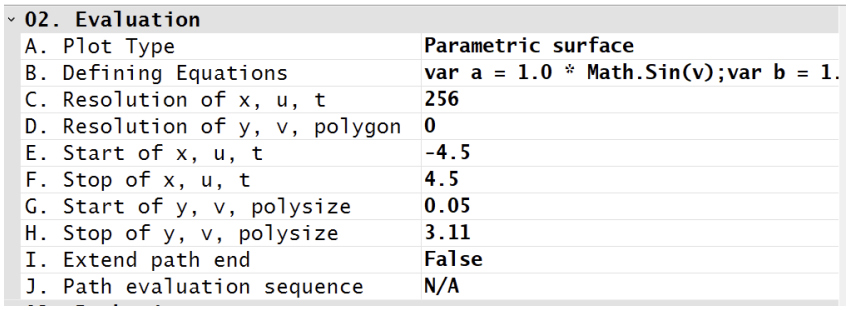
H. Style Of Axes determines the style axes (if any).

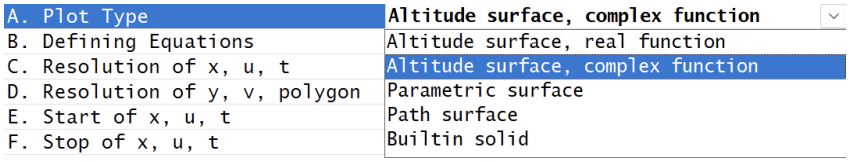


I. Run After Loading determines the action which occurs after loading the plot definition.

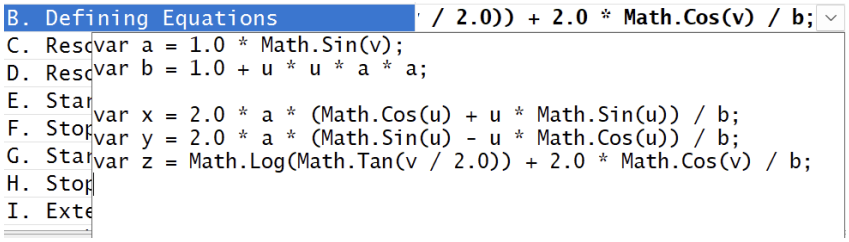
1.4.4.4 The Evaluation section in the edtior property grit

The Evaluation section in the edtior property grit contains the following items:

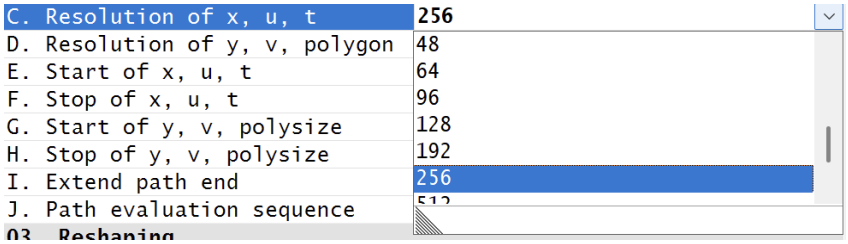




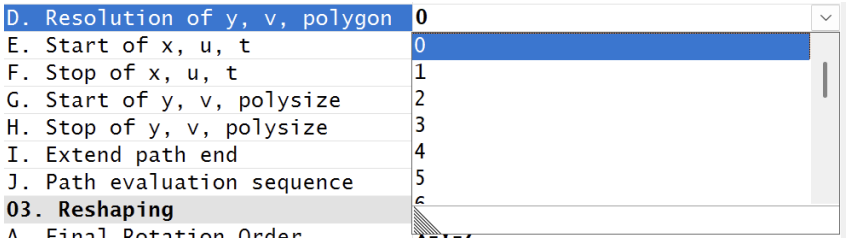
A. Plot Type determines the plot type.



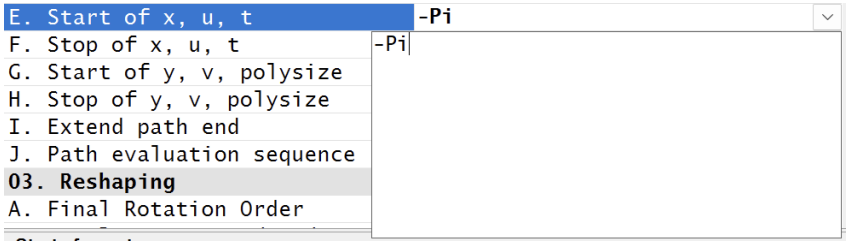
B. Defining equations contains the C# code for the equations which define the graphics.



C. Resolution of x, u, t determines the resolution of the variable, x ,u or t, depending on the plot type.



D. Resolution of y, v, polygon determines the resolution of the variable, y ,v or polygon, depending on the plot type.



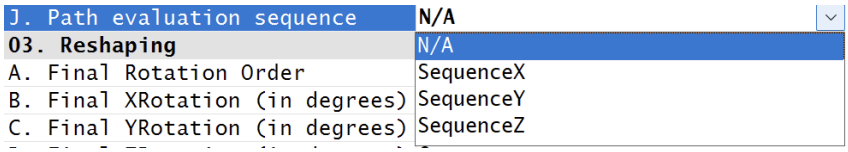
E. Start of x, u, t determines the start of the variable, x ,u or t, depending on the plot type.

F. Stop of x, u, t determines the stop of the variable, x ,u or t, depending on the plot type.

G. Start of y, v, polygon determines the start of the variable, y ,v or polygon, depending on the plot type.

H. Stop of y, v, polygon determines the stop of the variable, y ,v or polygon, depending on the plot type.

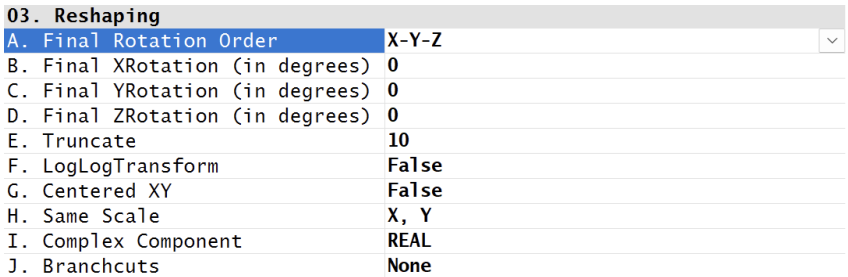
I. Extend path end determines if the path end is extended to close the path. Only relevant for path surfaces.

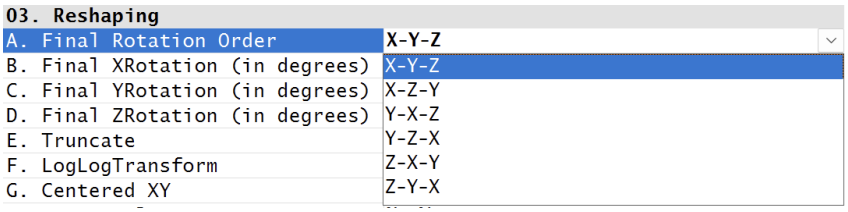


J. Path evaluation sequence determines the path sequence which is used for optimal results. Only relevant for path surfaces.

1.4.4.5 The Reshaping section in the edtior property grit

The Evaluation section in the edtior property grit contains the following items:





A. Final Rotation Order Sets the order for the final rotation (if any).

B. Final XRotation (in degrees) Rotation around the X axis (in degrees).

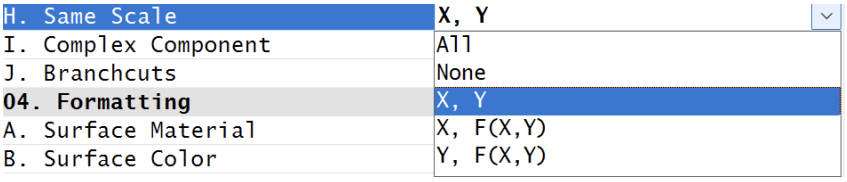
C. Final YRotation (in degrees) Rotation around the Y axis (in degrees).

D. Final ZRotation (in degrees) Rotation around the Z axis (in degrees).

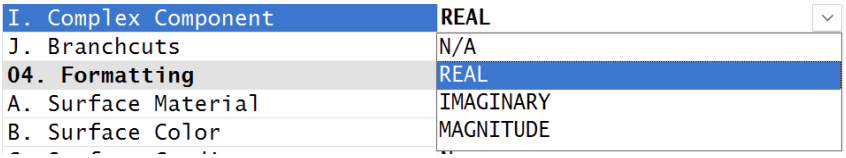
E. Truncate Sets the truncation threshold for the Z axis.

F. LogLogTransform determines if the LogLog transform is applied.

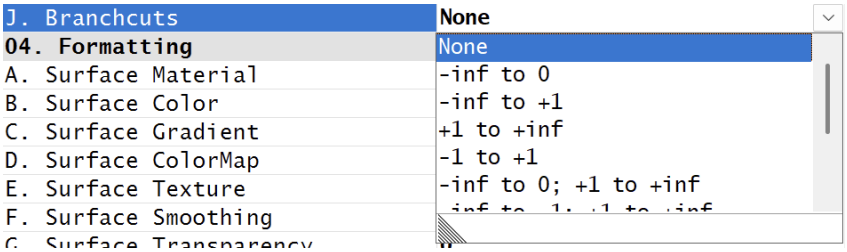
G. Centered XY determines if x,y are centered or not.



H. Same Scale determines which axes are using the same scale.



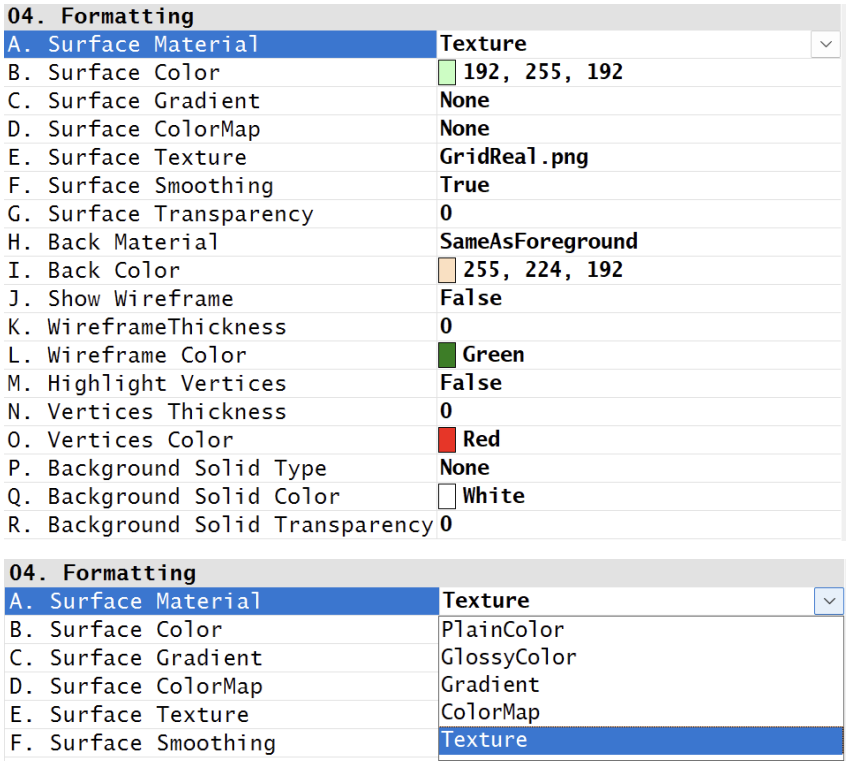
I. Complex Component determines which complex component is used. Only relevant if the plot type is Altitude surface, complex function.



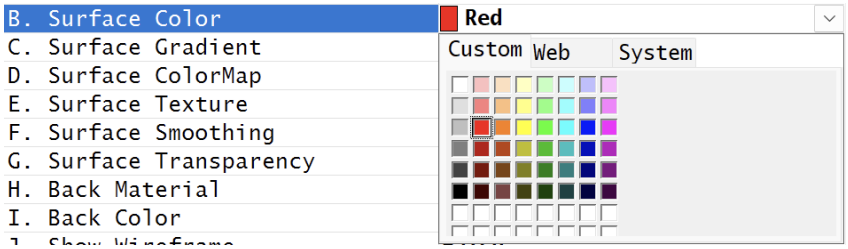
J. Branchcuts determines which brachcuts. Only relevant if the plot type is Altitude surface, complex function.

1.4.4.6 The Formatting section in the edtior property grit

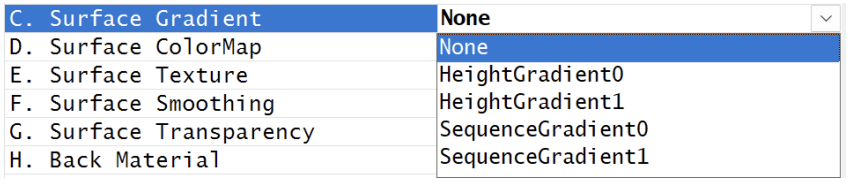
The Formatting section in the edtior property grit contains the following items:



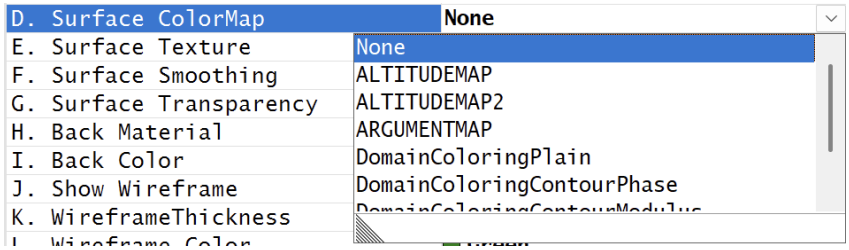
A. Surface Material determines which surface material is used.



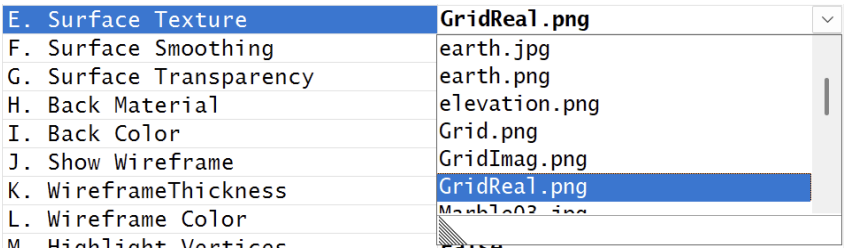
B. Surface Color determines which surface color is used. Note that there are 3 different tabs which can be used to choose the color. Only relevant the surface material is Plain Color or Glossy Color.



C. Surface Gradient determines which surface gradient is used.. Only relevant if the surface material is Gradient.



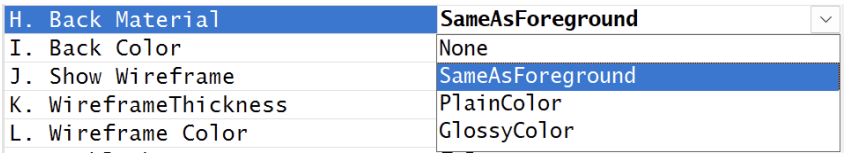
D. Surface ColorMap determines which surface colormap is used.. Only relevant if the surface material is ColorMap.



E. Surface Texture determines which texture is used.. Only relevant if the surface material is Texture.

F. Surface Smoothing determines if surface smoothing is applied.

G. Surface Transparency determines the amount of surface transparency (from o to 255).



H. Back Material determines the background material.

I. Back Color determines the background color.

J. Show Wireframe determines if a wireframe is shown.

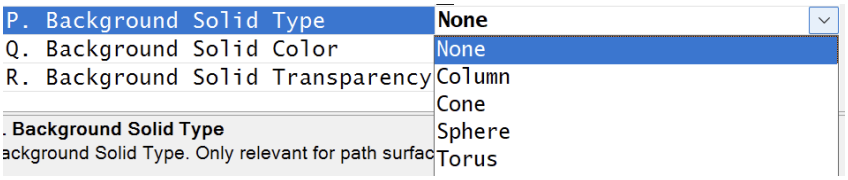
K. Wireframe Thickness determines the wireframe thickness.

L. Wireframe Color determines the wireframe color.

M. Highlight Vertices determines if vertices are highlighted.

N. Vertices Thickness determines the thickness of the vertices.

O. Vertices Color determines the color of the vertices.



P. Background Solid Type determines the type of the background, if any.

Q. Background Solid color determines the color of the background, if any.

R. Background Solid Transparency determines the amount of surface transparency (from o to 255).

1.4.4.7 Source code

The Python source code for starting the IDE can be found here: https://github.com/duhadler/XlCalcNet/blob/master/xlcalcnet/ShowEditor.py

The C# source code for the gallery of plots can be found here: https://github.com/duhadler/XlCalcNet/tree/master/xlcalcnet/Addin/NET48/Source/TinyPlot3D

Much of the C# code of this application is based on Stephens [506].

1.4.5 Functions related to folders

There are several functions related to folders

gui.**get_local_appdata**()

Return the current user's local Application Data folder

```
>>> from xlcalcnet import gui
>>> gui.get_local_appdata()
```

gui.**get_local_appdata_xlcalcnet**()

Return the current user's local AppData/XlCalcNetIDE folder

```
>>> from xlcalcnet import gui
>>> gui.get_local_appdata_xlcalcnet()
```

gui.**get_my_documents**()

Return the current user's local AppData/XlCalcNetIDE folder

```
>>> from xlcalcnet import gui
>>> gui.get_my_documents()
```

1.4.6 Information about installation status of supporting python packages

gui.**info**()

Support for Matplotlib output in a separate process

```
>>> from xlcalcnet import gui
>>> gui.info()
```

gui.**has_gpm**()

Returns True if gmpy2 is installed

```
>>> from xlcalcnet import gui
>>> gui.has_gpm()
```

gui.**has_apm**()

Returns True if python-flint is installed

```
>>> from xlcalcnet import gui
>>> gui.has_apm()
```

gui.**has_xlcalcnet2**()

Returns True if xlcalcnet2 is installed

```
>>> from xlcalcnet import gui
>>> gui.has_xlcalcnet2()
```

1.4.7 Information about context lists, and setting global precision

property gui.**ctxlistreal**

Returns a list of all available numerical contexts supporting real data types

```
>>> from xlcalcnet import gui
>>> gui.ctxlistreal
```

property gui.**ctxlistcplx**

Returns a list of all available numerical contexts supporting complex data types

```
>>> from xlcalcnet import gui
>>> gui.ctxlistcomplex
```

gui.**setdps**(*dps*)

Sets the decimal precision for all multiprecision data types

```
>>> from xlcalcnet import gui
>>> dps = 50
>>> gui.setdps(dps)
```

1.4.8 Support for running interactive Matplotlib output in a separate process

There is support for Matplotlib output in a separate process

See also: https://matplotlib.org/stable/users/explain/figure/interactive.html#default-ui

property gui.**plot**(*fig*, *file*, *fname*)

Support for Matplotlib output in a separate process

```
>>> from xlcalcnet import gui
>>> gui.plot(fig, file, fname)
```

1.5 Mathematical functions based on Mpmath, Gmpy2 and Python-Flint (only Python)

1.5.1 Overview

1.5.1.1 Contexts

The following group of contexts are available in Python only:

- Context group `ctx_pm`: this context group includes `fpm` (see *fpm*), `mpm` (see *mpm*), `dpm` (see *dpm*), `ipm` (see *ipm*), `gpm` (see *gpm*), `apm` (see *apm*). The `gpm` context is only available if Gmpy2 is installed, and the `apm` context is only available if Python-FLINT is installed.

XlCalcNet provides the following contexts from mpmath:

- Double-precision binary floating point arithmetic using Python's builtin `float` and `complex` types (`fp`)
- Arbitrary-precision binary floating point arithmetic (`mp`)
- Arbitrary-precision binary interval arithmetic (`iv`)

and in addition

- Arbitrary-precision decimal arithmetic (`dp`)
- Arbitrary-precision binary floating point arithmetic based on gmpy2 (`gp`): requires gmpy2
- Arbitrary-precision ball arithmetic based on ARB (`ap`): requires xlcalcnet libraries

The implementation of contexts in xlcalcnet extents and sometimes modifies the features of the contexts provided by mpmath; in this manual, changes to features already existing in mpmath will be pointed out explicitly.

By and large, xlcalcnet tries to be compatible with mpmath conventions as much as possible. The main difference is that in xlcalcnet the context must always explicitly be stated, whereas in mpmath the `mp` context is assumed as the default if the context is missing. In mpmath, we can write:

```
>>> from xlcalcnet import *
>>> sqrt(2)
mpf('1.4142135623730951')
```

This does not work in xlcalcnet, where also the `mp` context has to be given explicitly:

```
>>> from xlcalcnet import mp
>>> mp.sqrt(2)
mpf('1.4142135623730951')
```

The need to import xlcalcnet and mpmath into the same module should rarely arise (importing them separately in different modules of the same project is of course fine). If both are imported, care should be taken that the imported components do not shadow each other, e.g.:

```
>>> from xlcalcnet import mp
>>> from xlcalcnet.mpmath import mp as mpm   # importing the mpmath version of xlcalcnet

>>> mp.sqrt(2) # calling xlcalcnet
mpf('1.4142135623730951')
>>> mpm.sqrt(2) # calling mpmath
mpf('1.4142135623730951')
```

1.5.1.2 Conversion of sclars

This function converts scalars into each other

`ctx.t`(*x*, *strings=True*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Converts *x* to an `ctx.mpf` or `ctx.mpc`. If *x* is of type `ctx.mpf`, `ctx.mpc`, `int`, `float`, `complex`, the conversion will be performed losslessly, except in the following cases:

- conversion of a double to a decimal: cutoff at 14 digits
- conversion of an int to a mpfr: cutoff at current precision

If *x* is a string, the result will be rounded to the present working precision. Strings representing fractions or complex numbers are permitted.

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall:
>>> .... print(ctx.name)
>>> .... ctx.dps = 10; print([ctx.t(3.5), ctx.t(2+3j)])
>>> .... ctx.dps = 10; print([ctx.t('3.1'), ctx.t('3.1 + 4.6j')])
fpm
[3.5, (2+3j)]
[3.1, (3.1+4.6j)]

mpm
[mpf('3.5'), mpc(real='2.0', imag='3.0')]
[mpf('3.100000000006'), mpc(real='3.100000000006', imag='4.599999999977')]

ipm
[mpi('3.5', '3.5'), iv.mpc(mpi('2.0', '2.0'), mpi('3.0', '3.0'))]
[mpi('3.099999999977', '3.100000000006'),
iv.mpc(mpi('3.099999999977', '3.100000000006'), mpi('4.599999999977', '4.600000000035'))]

dec
:cite:t:`Decimal('3.5'), DecCplx('2 + 3.0j')]
:cite:t:`Decimal('3.1'), DecCplx('3.1 + 4.6j')]

gmp
[mpfr('3.5',37), mpc('2.0+3.0j',(37,37))]
[mpfr('3.100000000006',37), mpc('3.100000000006+4.599999999977j',(37,37))]

apm
[arb('3.50'), acb('2.00 + 3.00j')]
[arb('[3.09999999998 +/- 3.24e-11]'),
acb('[3.09999999998 +/- 3.24e-11] + [4.59999999998 +/- 6.15e-11]j')]
```

1.5.1.3 Precision in bits and digits

`ctx.prec` holds the current precision (in bits):

The default precision (in bits) for all contexts after starting xlcalcnet is `ctx.prec = 53`.

```
>>> fp.prec, mp.prec, iv.prec, dp.prec, gp.prec, ap.prec
(53, 53, 53, 53, 53, 53)
```

`ctx.dps` holds the current decimal precision (in digits):

The default decimal precision (in digits) for all contexts after starting xlcalcnet is `ctx.dps = 15`.

```
>>> fp.dps, mp.dps, iv.dps, dp.dps, gp.dps, ap.dps
(15, 15, 15, 15, 15, 15)
```

Like mpmath, xlcalcnet expects every devision of a *normal* number by zero to raise a `DivisionByZero`, and not to return `+inf` or `-inf`. This is the default behaviour for the `fp`, `mp`, `iv` and `dp` contexts anyway, and has been changed to follow this convention for the `gp` and `ap` context.

A number of algorithms use this exception to trigger a temporary increase of precision.

1.5.2 Double-precision arithmetic: `fpm`

The Python module `fpm` provides support for real and complex floating point numbers in double precision (see https://en.wikipedia.org/wiki/Double-precision_floating-point_format). The corresponding real and complex data types, `float` and `complex`, are implemented in hardware (and are therefore quite fast).

Note: automatic conversion from `Double` in .Net Framework to `float`

Note: no conversion from `Complex` in .Net Framework to `complex` in Python. Explain workarounds.

1.5.2.1 Examples for `fpm`, real input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
```

(continues on next page)

(continued from previous page)

```
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02a_FpmReal.py.

1.5.2.2 Examples for fpm, complex input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3984573


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
```

(continues on next page)

(continued from previous page)

```
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02b_FpmCplx.py.

1.5.2.3 Implementation in Python

The Python source code for this module can be found here: ctx_fpm.py.

1.5.3 Binary floating-point with arbitrary-precision and exponent: mpm

The Python module mpm provides support for real and complex floating point numbers with arbitrary-precision and exponent. The corresponding real and complex data types, float and complex, are implemented in software.

1.5.3.1 Examples for mpm, real input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
```

(continues on next page)

(continued from previous page)

```
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03a_MpmReal.py.

1.5.3.2 Examples for `mpm`, complex input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
```

(continues on next page)

(continued from previous page)

```
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3

def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
```

(continues on next page)

(continued from previous page)

```
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03b_MpmCplx.py.

1.5.3.3 Implementation in Python

The Python source code for this module can be found here: ctx_mpm.py.

1.5.4 Interval arithmetic with arbitrary-precision and exponent:ipm

The Python module ipm provides support for real and complex intervals with arbitrary-precision and exponent. The corresponding real and complex data types, float and complex, are implemented in software.

1.5.4.1 Examples for ipm, real input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
```

(continues on next page)

(continued from previous page)

```
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04a_IpmReal.py.

1.5.4.2 Examples for `ipm`, complex input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
```

(continues on next page)

(continued from previous page)

```
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3

def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
```

(continues on next page)

(continued from previous page)

```
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04b_IpmCplx.py.

1.5.4.3 Implementation in Python

The Python source code for this module can be found here: ctx_ipm.py.

1.5.5 Decimal floating-point in arbitrary-precision with limited exponent: dpm

The Python module dpm provides support for real and complex decimal floating-point in arbitrary-precision with limited exponent. The corresponding real and complex data types, float and complex, are implemented in software.

Additional contexts are used in xlcalcnet to implement its functions for the mpmath data types, and the Decimal data type, which is part of Python.

Both real numbers (mpf) and complex numbers (mpc) are implemented.

In CPython, the decimal module provides support for fast correctly-rounded decimal floating point arithmetic. See https://docs.python.org/3/library/decimal.html for a decription of the Decimal data type.

The decimal module includes only a few transcendental functions: sqrt, log, exp.

The dec context gives access to many more:

1.5.5.1 Examples for dpm, real input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
```

(continues on next page)

(continued from previous page)

```
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):         ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):     ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
```

(continues on next page)

(continued from previous page)

```
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04a_IpmReal.py.

1.5.5.2 Examples for `dpm`, complex input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
```

(continues on next page)

(continued from previous page)

```
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):         ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):     ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05b_DpmCplx.py.

1.5.5.3 Implementation in Python

The Python source code for this module can be found here: ctx_dpm.py.

1.5.6 Rational numbers: qpm

The qpm data type is mostly useful in the context of linear algebra, where it can provide exact results.

Only real numbers (mpf) are implemented.

The internal representation dependes on what else is installed on the system:

If apm is available, the fmpq data type is used; otherwise, if gpm is available, the mpq data type is used; otherwise, Python's built in Fraction data type is used.

In CPython, the fractions module provides support for rational number arithmetic. See https://docs.python.org/3/library/fractions.html for a decription of the Fraction data type.

1.5.6.1 Examples for qpm, real input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06a_QpmReal.py.

1.5.6.2 Implementation in Python

The Python source code for this module can be found here: ctx_dpm.py.

1.5.7 Binary floating-point in arbitrary-precision with limited exponent: gpm

The Python module gpm provides support for real and complex Binary floating-point in arbitrary-precision with limited exponent. The corresponding real and complex data types, float and complex, are implemented in software.

gmpy2 is a C-coded Python extension module that supports multiple-precision arithmetic.

https://gmpy2.readthedocs.io/en/latest/

Assuming that the Python batch files have been installed as described *here*, gmpy2 can be installed by double-clicking on the batch file A11installgmpy2.bat in the C:\Python313 folder.

1.5.7.1 Examples for gpm, real input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734638475398457398457 3)
    print('x0 = math53T(232945639845394856394563936482734638475398457398457 3', x0)
    x1 = ctx.t("232945639845394856394563936482734638475398457398457 3")
    print('x1 = ctx.t("232945639845394856394563936482734638475398457398457 3"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
```

(continues on next page)

(continued from previous page)

```
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):         ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):     ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
```

(continues on next page)

(continued from previous page)

```
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_GpmReal.py.

1.5.7.2 Examples for `gpm`, complex input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(23294563984539485639456393648273463847539845739845 7)
    print('x0 = math53T(23294563984539485639456393648273463847539845739845 7', x0)
    x1 = ctx.t("23294563984539485639456393648273463847539845739845 7")
    print('x1 = ctx.t("23294563984539485639456393648273463847539845739845 7"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
```

(continues on next page)

(continued from previous page)

```
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_GpmCplx.py.

1.5.7.3 Implementation in Python

The Python source code for this module can be found here: ctx_gpm.py.

1.5.8 Binary balls in arbitrary-precision and with arbitrary exponent: apm

The Python module apm provides support for real and complex binary balls in arbitrary-precision and with arbitrary exponent. The corresponding real and complex data types, float and complex, are implemented in software.

pythonflint is a C-coded Python extension module that supports multiple-precision arithmetic.

https://python-flint.readthedocs.io/en/latest/

Assuming that the Python batch files have been installed as described *here*, Python-flint can be installed by double-clicking on the batch file A12installpythonflint.bat in the C:\Python313 folder.

1.5.8.1 Examples for apm, real input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08a_ApmReal.py.

1.5.8.2 Examples for apm, complex input

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3984573


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734638475398457398457 3984573)
    print('x0 = math53T(232945639845394856394563936482734638475398457398457 3984573', x0)
    x1 = ctx.t("232945639845394856394563936482734638475398457398457 3984573")
    print('x1 = ctx.t("232945639845394856394563936482734638475398457398457 3984573"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
```

(continues on next page)

(continued from previous page)

```
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08b_ApmCplx.py.

1.5.8.3 Implementation in Python

The Python source code for this module can be found here: ctx_apm.py.

1.5.9 Numpy support for multiprecision data types: `npm`

1.5.9.1 Overview

Numpy supports general objects as elements, and this can be used to use numpy functions for the above data types.

1.6 Mathematical functions in fixed precision (C#, can be called from Python)

1.6.1 C# based context groups

Various additional numerical contexts support C# and Python. It is convenient to divide them into groups to make referencing them per function description more efficient.

The following groups of contexts are available:

1.6.1.1 Context group `ctx53`

This context group includes `math53` and `cmath53` (see *math53/cmath53*). Almost all functions are from the DAMath library, with a few functions from Boost Math, Scipy, Julia and other libraries available on Github.

1.6.1.2 Context group `ctxcpp`

This context group includes contexts which contain functions of the C++ standard library, the Eigen library, and functions which can easily be derived from them. They include contexts in fixed precision: `sreal` and `scplx` (see *sreal/scplx*), `dreal` and `dcplx` (see *dreal/dcplx*), `ereal` and `ecplx` (see *ereal/ecplx*), `qreal` and `qcplx` (see *qreal/cplx*), `oreal` and `ocplx` (see *oreal/ocplx*), ; and contexts in arbitrary precision, which are part of XlCalcNet2 (which hence needs to be installed): `mreal` and `mcplx` (see *mreal/mcplx*).

1.6.1.3 Context group `ctxboost`

This context group includes contexts which contain functions of the the Boost Math/Multiprecision/Odeint libraries, and functions which can easily be derived from them. They include contexts in fixed precision: `sreal` (see *sreal*), `dreal` (see *dreal*), `ereal` (see *ereal*), `qreal` (see *qreal*), `oreal` (see *oreal*), ; and one context in arbitrary precision, which is part of XlCalcNet2 (which hence needs to be installed): `mreal` (see *mreal*).

1.6.1.4 Context group `ctxflint`

This context group includes contexts which contain functions of the Flint 3.4 library, the Eigen library, and functions which can easily be derived from them. These are contexts in fixed and in arbitrary precision, which are part of XlCalcNet2 (which hence needs to be installed). They include `sflint` and `sflintc` (see *sflint/sflintc*), `dflint` and `dflintc` (see *dflint/dflintc*), `eflint` and `eflintc` (see *eflint/eflintc*), `qflint` and `qflintc` (see *qflint/qflint*), `oflint` and `oflintc` (see *oflint/oflintc*), `mflint` and `mflintc` (see *mflint/mflintc*), `aflint` and `aflintc` (see *aflint/aflint*).

1.6.2 Scalar functions with Double and Complex: math53 and cmath53

The C# modules math53 and cmath53 provide support for real and complex floating point numbers in double precision (see https://en.wikipedia.org/wiki/Double-precision_floating-point_format). The corresponding real and complex data types, Double and Complex, are implemented in hardware and are part of .Net Framework (and are therefore quite fast).

Note: automatic conversion from Double in .Net Framework to float

Note: no conversion from Complex in .Net Framework to complex in Python. Explain workarounds.

DAMath library.

1.6.2.1 Examples in C#, math53

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some math53 functions");
    SrealTest0();
    SrealTest1();
}
```

General test code for C# can be found here

```
public static void Math53Test0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "ScplxTest0"  + "\"" + ">");

    var a = cmath53.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = cmath53.abs(a);
    Console.WriteLine("res1 = cmath53.abs(a): {0}", res1);

    var res2 = cmath53.exp(a);
    Console.WriteLine("res2 = cmath53.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void Math53Test1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "ScplxTest1"  + "\"" + ">");

    var a = cmath53.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = cmath53.asin(a);
    Console.WriteLine("res1 = cmath53.asin(a) {0}", res1);

    var res2 = cmath53.sin(a);
    Console.WriteLine("res2 = cmath53.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02a_math53.cs.

1.6.2.2 Examples in C#, `cmath53`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some math53 functions");
    SrealTest0();
    SrealTest1();
}
```

General test code for C# can be found here

```
public static void ScplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "ScplxTest0"  + "\"" + ">");

    var a = cmath53.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = cmath53.abs(a);
    Console.WriteLine("res1 = cmath53.abs(a): {0}", res1);

    var res2 = cmath53.exp(a);
    Console.WriteLine("res2 = cmath53.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void ScplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "ScplxTest1"  + "\"" + ">");

    var a = cmath53.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = cmath53.asin(a);
    Console.WriteLine("res1 = cmath53.asin(a) {0}", res1);

    var res2 = cmath53.sin(a);
    Console.WriteLine("res2 = cmath53.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02b_Cmath53.cs.

1.6.2.3 Examples in Python, `math53`

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3984573


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734638475398457398457 3984573)
    print('x0 = math53T(232945639845394856394563936482734638475398457398457 3984573', x0)
    x1 = ctx.t("232945639845394856394563936482734638475398457398457 3984573")
    print('x1 = ctx.t("232945639845394856394563936482734638475398457398457 3984573"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
```

(continues on next page)

(continued from previous page)

```
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02a_math53.py.

1.6.2.4 Examples in Python, `cmath53`

General test code for Python can be found here

```
import math
from xlcalcnet import math53
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
```

(continues on next page)

(continued from previous page)

```
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_math53()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ctx.t(i)
    print('x = ctx.t(i):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)

    x = ctx.t(5.7)
    print('x = math53T(5.7):', x)
    x0 = ctx.t(232945639845394856394563936482734 6)
    print('x0 = math53T(232945639845394856394563936482734 6', x0)
    x1 = ctx.t("232945639845394856394563936482734 6")
    print('x1 = ctx.t("232945639845394856394563936482734 6"):', x1)
    x = ctx.t("5.5")
    print('x = math53T("5.5"):', x)

    print()
    x = ctx.t(55)
    print('x = ctx.t(5):', x)
    y = ctx.exp(x)
    print('y = ctx.exp(x):', y)

    z = ctx.exp(5.5)
    print('z = ctx.exp(5.5):', z)
    z = ctx.exp(5)
    print('z = ctx.exp(5):', z)
    z = ctx.exp("5.5")
    print('z = ctx.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ctx.exp(dec)
    print('z = ctx.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ctx.exp(frac)
    print('z = ctx.exp(frac):', z)

    print()
    x = ctx.t(5.5)
    print('x = ctx.t(55):', x)
    y = ctx.t(3.3)
    print('y = ctx.t(33):', y)
    z = ctx.pow(x, y)
    print('z = ctx.pow(x, y):        ', z)
    z = ctx.pow(5.5, 3.3)
    print('z = ctx.pow(5.5, 3.3):    ', z)
    z = ctx.pow("5.5", "3.3")
    print('z = ctx.pow("5.5", "3.3"):', z)
    z = ctx.pow(5, 3)
```

(continues on next page)

(continued from previous page)

```
    print('z = ctx.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_math53():
    print()
    print('<H1 Title="Arithmetic operators with math53">')

    x = ctx.t(5.0)
    y = ctx.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02b_Cmath53.py.

1.6.2.5 Notes regarding the implementation in C# and C++

The C# source code for the module `math53` can be found here: ctx.cs.

The C# source code for the module `cmath53` can be found here: cmath53.cs.

The Pascal source code which is called from `ctx.cs` and `cmath53.cs` can be found here: libwe64d.pas.

The C++ source code which is called from `ctx.cs` and `cmath53.cs` can be found here: XSF.cpp.

The C# source code (transcribed from the Julia project) which is called from `ctx.cs` and `cmath53.cs` can be found here: fromjulia.cs.

1.6.3 Binary floating point, single precision: `sreal` and `scplx`

The C# modules `sreal` and `scplx` provide support for real and complex floating point numbers in single precision (see https://en.wikipedia.org/wiki/Single-precision_floating-point_format). The corresponding real data type, `Single`, is implemented in hardware and is part of .Net Framework (and is therefore quite fast), the corresponding complex data type, `SingleC`, is also implemented in hardware, but has been added as an object in XlCalcNet (and has therefore the overhead of objects).

The C# modules `sreal` and `scplx` fully support Boost.Math and Eigen.

TODO: python code showing larget, lowest, machine epsilon etc.

1.6.3.1 Examples in C#, `sreal`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some sreal functions");
    SrealTest0();
    SrealTest1();
}
```

General test code for C# can be found here

```
public static void ScplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "ScplxTest0"  + "\"" + ">");

    var a = scplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = scplx.abs(a);
    Console.WriteLine("res1 = scplx.abs(a): {0}", res1);

    var res2 = scplx.exp(a);
    Console.WriteLine("res2 = scplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void ScplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "ScplxTest1"  + "\"" + ">");

    var a = scplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = scplx.asin(a);
    Console.WriteLine("res1 = scplx.asin(a) {0}", res1);

    var res2 = scplx.sin(a);
    Console.WriteLine("res2 = scplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02a_Sreal.cs.

1.6.3.2 Examples in C#, `scplx`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some sreal functions");
    SrealTest0();
    SrealTest1();
}
```

General test code for C# can be found here

```
public static void ScplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "ScplxTest0"  + "\"" + ">");

    var a = scplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = scplx.abs(a);
    Console.WriteLine("res1 = scplx.abs(a): {0}", res1);

    var res2 = scplx.exp(a);
    Console.WriteLine("res2 = scplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void ScplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "ScplxTest1"  + "\"" + ">");

    var a = scplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = scplx.asin(a);
    Console.WriteLine("res1 = scplx.asin(a) {0}", res1);

    var res2 = scplx.sin(a);
    Console.WriteLine("res2 = scplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03b_Scplx.cs.

1.6.3.3 Examples in Python, sreal

General test code for Python can be found here

```
import math
from xlcalcnet import sreal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3

def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_sreal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = sreal.t(i)
    print('x = sreal.t(i):', x)

    x = sreal.t(5.7)
    print('x = srealT(5.7):', x)

    x = sreal.t(5.7)
    print('x = srealT(5.7):', x)
    x0 = sreal.t(232945639845394856394563936482734 6)
    print('x0 = srealT(232945639845394856394563936482734 6', x0)
    x1 = sreal.t("232945639845394856394563936482734 6")
    print('x1 = sreal.t("232945639845394856394563936482734 6"):', x1)
    x = sreal.t("5.5")
    print('x = srealT("5.5"):', x)

    print()
    x = sreal.t(55)
    print('x = sreal.t(5):', x)
    y = sreal.exp(x)
    print('y = sreal.exp(x):', y)

    z = sreal.exp(5.5)
    print('z = sreal.exp(5.5):', z)
    z = sreal.exp(5)
    print('z = sreal.exp(5):', z)
    z = sreal.exp("5.5")
    print('z = sreal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = sreal.exp(dec)
    print('z = sreal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = sreal.exp(frac)
    print('z = sreal.exp(frac):', z)

    print()
```

(continues on next page)

(continued from previous page)

```
    x = sreal.t(5.5)
    print('x = sreal.t(55):', x)
    y = sreal.t(3.3)
    print('y = sreal.t(33):', y)
    z = sreal.pow(x, y)
    print('z = sreal.pow(x, y):        ', z)
    z = sreal.pow(5.5, 3.3)
    print('z = sreal.pow(5.5, 3.3):    ', z)
    z = sreal.pow("5.5", "3.3")
    print('z = sreal.pow("5.5", "3.3"):', z)
    z = sreal.pow(5, 3)
    print('z = sreal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_sreal():
    print()
    print('<H1 Title="Arithmetic operators with sreal">')

    x = sreal.t(5.0)
    y = sreal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03a_SReal.py.

1.6.3.4 Examples in Python, `scplx`

General test code for Python can be found here

```
import math
from xlcalcnet import sreal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
```

(continues on next page)

(continued from previous page)

```
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_sreal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = sreal.t(i)
    print('x = sreal.t(i):', x)

    x = sreal.t(5.7)
    print('x = srealT(5.7):', x)

    x = sreal.t(5.7)
    print('x = srealT(5.7):', x)
    x0 = sreal.t(232945639845394856394563936482734 6)
    print('x0 = srealT(232945639845394856394563936482734 6', x0)
    x1 = sreal.t("232945639845394856394563936482734 6")
    print('x1 = sreal.t("232945639845394856394563936482734 6"):', x1)
    x = sreal.t("5.5")
    print('x = srealT("5.5"):', x)

    print()
    x = sreal.t(55)
    print('x = sreal.t(5):', x)
    y = sreal.exp(x)
    print('y = sreal.exp(x):', y)

    z = sreal.exp(5.5)
    print('z = sreal.exp(5.5):', z)
    z = sreal.exp(5)
    print('z = sreal.exp(5):', z)
    z = sreal.exp("5.5")
    print('z = sreal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = sreal.exp(dec)
    print('z = sreal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = sreal.exp(frac)
    print('z = sreal.exp(frac):', z)

    print()
    x = sreal.t(5.5)
    print('x = sreal.t(55):', x)
    y = sreal.t(3.3)
    print('y = sreal.t(33):', y)
    z = sreal.pow(x, y)
    print('z = sreal.pow(x, y):        ', z)
    z = sreal.pow(5.5, 3.3)
    print('z = sreal.pow(5.5, 3.3):    ', z)
    z = sreal.pow("5.5", "3.3")
    print('z = sreal.pow("5.5", "3.3"):', z)
    z = sreal.pow(5, 3)
```

(continues on next page)

(continued from previous page)

```
    print('z = sreal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_sreal():
    print()
    print('<H1 Title="Arithmetic operators with sreal">')

    x = sreal.t(5.0)
    y = sreal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03b_SCplx.py.

1.6.3.5 Notes regarding the implementation in C# and C++

The C# source code for the module `sreal` can be found here: sreal.cs.

The C# source code for the module `scplx` can be found here: scplx.cs.

The C++ source code supporting scalars in general which is called from `sreal.cs` and `scplx.cs` can be found here: UseSReal.cpp.

The C++ source code which interacts directly with Boost can be found here: BoostSReal.cpp.

The C++ source code supporting Eigen which is called from `sreal.cs` and `scplx.cs` can be found here: UseEigenSReal.cpp.

The C++ source code which interacts directly with Eigen can be found in the folder: BoostEigenMath.cpp.

1.6.4 Binary floating point, double precision: `dreal` and `dcplx`

The C# modules `dreal` and `dcplx` provide support for real and complex floating point numbers in double precision (see https://en.wikipedia.org/wiki/Double-precision_floating-point_format). The corresponding real and complex data types, `Double` and `Complex`, are implemented in hardware and are part of .Net Framework (and are therefore quite fast).

The C# modules `dreal` and `dcplx` fully support Boost.Math and Eigen.

Note: automatic conversion from `Double` in .Net Framework to `float`

Note: no conversion from `Complex` in .Net Framework to `complex` in Python. Explain workarounds.

Note: issues with multiplication scalar * matrix in complex

1.6.4.1 Examples in C#, `dreal`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some dreal functions");
    DrealTest0();
    DrealTest1();
}
```

General test code for C# can be found here

```
public static void DcplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "DcplxTest0"  + "\"" + ">");

    var a = dcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = dcplx.abs(a);
    Console.WriteLine("res1 = dcplx.abs(a): {0}", res1);

    var res2 = dcplx.exp(a);
    Console.WriteLine("res2 = dcplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void DcplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "DcplxTest1"  + "\"" + ">");

    var a = dcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = dcplx.asin(a);
    Console.WriteLine("res1 = dcplx.asin(a) {0}", res1);

    var res2 = dcplx.sin(a);
    Console.WriteLine("res2 = dcplx.sin(a) {0}", res2);
```

(continues on next page)

(continued from previous page)

```
    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04a_Dreal.cs.

1.6.4.2 Examples in C#, `dcplx`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some dreal functions");
    DrealTest0();
    DrealTest1();
}
```

General test code for C# can be found here

```
public static void DcplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "DcplxTest0"  + "\"" + ">");

    var a = dcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = dcplx.abs(a);
    Console.WriteLine("res1 = dcplx.abs(a): {0}", res1);

    var res2 = dcplx.exp(a);
    Console.WriteLine("res2 = dcplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void DcplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "DcplxTest1"  + "\"" + ">");

    var a = dcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = dcplx.asin(a);
    Console.WriteLine("res1 = dcplx.asin(a) {0}", res1);

    var res2 = dcplx.sin(a);
    Console.WriteLine("res2 = dcplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04b_Dcplx.cs.

1.6.4.3 Examples in Python, dreal

General test code for Python can be found here

```
import math
from xlcalcnet import dreal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3

def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_dreal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = dreal.t(i)
    print('x = dreal.t(i):', x)

    x = dreal.t(5.7)
    print('x = drealT(5.7):', x)

    x = dreal.t(5.7)
    print('x = drealT(5.7):', x)
    x0 = dreal.t(232945639845394856394563936482734 6)
    print('x0 = drealT(232945639845394856394563936482734 6', x0)
    x1 = dreal.t("232945639845394856394563936482734 6")
    print('x1 = dreal.t("232945639845394856394563936482734 6"):', x1)
    x = dreal.t("5.5")
    print('x = drealT("5.5"):', x)

    print()
    x = dreal.t(55)
    print('x = dreal.t(5):', x)
    y = dreal.exp(x)
    print('y = dreal.exp(x):', y)

    z = dreal.exp(5.5)
    print('z = dreal.exp(5.5):', z)
    z = dreal.exp(5)
    print('z = dreal.exp(5):', z)
    z = dreal.exp("5.5")
    print('z = dreal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = dreal.exp(dec)
    print('z = dreal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = dreal.exp(frac)
    print('z = dreal.exp(frac):', z)

    print()
```

(continues on next page)

(continued from previous page)

```
    x = dreal.t(5.5)
    print('x = dreal.t(55):', x)
    y = dreal.t(3.3)
    print('y = dreal.t(33):', y)
    z = dreal.pow(x, y)
    print('z = dreal.pow(x, y):        ', z)
    z = dreal.pow(5.5, 3.3)
    print('z = dreal.pow(5.5, 3.3):    ', z)
    z = dreal.pow("5.5", "3.3")
    print('z = dreal.pow("5.5", "3.3"):', z)
    z = dreal.pow(5, 3)
    print('z = dreal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_dreal():
    print()
    print('<H1 Title="Arithmetic operators with dreal">')

    x = dreal.t(5.0)
    y = dreal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04a_DReal.py.

1.6.4.4 Examples in Python, `dcplx`

General test code for Python can be found here

```
import math
from xlcalcnet import dreal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
```

(continues on next page)

(continued from previous page)

```
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_dreal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = dreal.t(i)
    print('x = dreal.t(i):', x)

    x = dreal.t(5.7)
    print('x = drealT(5.7):', x)

    x = dreal.t(5.7)
    print('x = drealT(5.7):', x)
    x0 = dreal.t(232945639845394856394563936482734 6)
    print('x0 = drealT(232945639845394856394563936482734 6', x0)
    x1 = dreal.t("232945639845394856394563936482734 6")
    print('x1 = dreal.t("232945639845394856394563936482734 6"):', x1)
    x = dreal.t("5.5")
    print('x = drealT("5.5"):', x)

    print()
    x = dreal.t(55)
    print('x = dreal.t(5):', x)
    y = dreal.exp(x)
    print('y = dreal.exp(x):', y)

    z = dreal.exp(5.5)
    print('z = dreal.exp(5.5):', z)
    z = dreal.exp(5)
    print('z = dreal.exp(5):', z)
    z = dreal.exp("5.5")
    print('z = dreal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = dreal.exp(dec)
    print('z = dreal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = dreal.exp(frac)
    print('z = dreal.exp(frac):', z)

    print()
    x = dreal.t(5.5)
    print('x = dreal.t(55):', x)
    y = dreal.t(3.3)
    print('y = dreal.t(33):', y)
    z = dreal.pow(x, y)
    print('z = dreal.pow(x, y):        ', z)
    z = dreal.pow(5.5, 3.3)
    print('z = dreal.pow(5.5, 3.3):    ', z)
    z = dreal.pow("5.5", "3.3")
    print('z = dreal.pow("5.5", "3.3"):', z)
    z = dreal.pow(5, 3)
```

(continues on next page)

(continued from previous page)

```
    print('z = dreal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_dreal():
    print()
    print('<H1 Title="Arithmetic operators with dreal">')

    x = dreal.t(5.0)
    y = dreal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04b_DCplx.py.

1.6.4.5 Notes regarding the implementation in C# and C++

The C# source code for the module `dreal` can be found here: dreal.cs.

The C# source code for the module `dcplx` can be found here: dcplx.cs.

The C++ source code supporting scalars in general which is called from `dreal.cs` and `dcplx.cs` can be found here: UseFReal.cpp.

The C++ source code which interacts directly with Boost can be found here: BoostSReal.cpp, and BoostDouble.cpp.

The C++ source code supporting Eigen which is called from `dreal.cs` and `dcplx.cs` can be found here: UseEigenFReal.cpp.

The C++ source code which interacts directly with Eigen can be found in the folder: BoostEigenMath.cpp.

1.6.5 Binary floating point, extended precision: `ereal` and `ecplx`

The C# modules `ereal` and `ecplx` provide support for real and complex floating point numbers in extended precision (see https://en.wikipedia.org/wiki/Extended_precision). The corresponding real and complex data types, `Extended` and `ExtendedC`, have been added as objects in XlCalcNet (and have therefore the overhead of objects). They are, however, implemented in hardware, and are supported as native types in C++ (when using GCC). This means that the routines in extended precision provided in Boost and Eigen are much faster than the corresponding routines in quadruple precision.

The C# modules `ereal` and `ecplx` fully support Boost.Math and Eigen.

1.6.5.1 Examples in C#, `ereal`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some ereal functions");
    ErealTest0();
    ErealTest1();
}
```

General test code for C# can be found here

```
public static void EcplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "EcplxTest0"  + "\"" + ">");

    var a = ecplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = ecplx.abs(a);
    Console.WriteLine("res1 = ecplx.abs(a): {0}", res1);

    var res2 = ecplx.exp(a);
    Console.WriteLine("res2 = ecplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void EcplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "EcplxTest1"  + "\"" + ">");

    var a = ecplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = ecplx.asin(a);
    Console.WriteLine("res1 = ecplx.asin(a) {0}", res1);

    var res2 = ecplx.sin(a);
    Console.WriteLine("res2 = ecplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05a_Ereal.cs.

1.6.5.2 Examples in C#, `ecplx`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some ereal functions");
    ErealTest0();
    ErealTest1();
}
```

General test code for C# can be found here

```
public static void EcplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "EcplxTest0"  + "\"" + ">");

    var a = ecplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = ecplx.abs(a);
    Console.WriteLine("res1 = ecplx.abs(a): {0}", res1);

    var res2 = ecplx.exp(a);
    Console.WriteLine("res2 = ecplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void EcplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "EcplxTest1"  + "\"" + ">");

    var a = ecplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = ecplx.asin(a);
    Console.WriteLine("res1 = ecplx.asin(a) {0}", res1);

    var res2 = ecplx.sin(a);
    Console.WriteLine("res2 = ecplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05b_Ecplx.cs.

1.6.5.3 Examples in Python, ereal

General test code for Python can be found here

```
import math
from xlcalcnet import ereal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_ereal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ereal.t(i)
    print('x = ereal.t(i):', x)

    x = ereal.t(5.7)
    print('x = erealT(5.7):', x)

    x = ereal.t(5.7)
    print('x = erealT(5.7):', x)
    x0 = ereal.t(232945639845394856394563936482734 6)
    print('x0 = erealT(232945639845394856394563936482734 6', x0)
    x1 = ereal.t("232945639845394856394563936482734 6")
    print('x1 = ereal.t("232945639845394856394563936482734 6"):', x1)
    x = ereal.t("5.5")
    print('x = erealT("5.5"):', x)

    print()
    x = ereal.t(55)
    print('x = ereal.t(5):', x)
    y = ereal.exp(x)
    print('y = ereal.exp(x):', y)

    z = ereal.exp(5.5)
    print('z = ereal.exp(5.5):', z)
    z = ereal.exp(5)
    print('z = ereal.exp(5):', z)
    z = ereal.exp("5.5")
    print('z = ereal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ereal.exp(dec)
    print('z = ereal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ereal.exp(frac)
    print('z = ereal.exp(frac):', z)

    print()
```

(continues on next page)

(continued from previous page)

```
    x = ereal.t(5.5)
    print('x = ereal.t(55):', x)
    y = ereal.t(3.3)
    print('y = ereal.t(33):', y)
    z = ereal.pow(x, y)
    print('z = ereal.pow(x, y):        ', z)
    z = ereal.pow(5.5, 3.3)
    print('z = ereal.pow(5.5, 3.3):    ', z)
    z = ereal.pow("5.5", "3.3")
    print('z = ereal.pow("5.5", "3.3"):', z)
    z = ereal.pow(5, 3)
    print('z = ereal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_ereal():
    print()
    print('<H1 Title="Arithmetic operators with ereal">')

    x = ereal.t(5.0)
    y = ereal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05a_EReal.py.

1.6.5.4 Examples in Python, `ecplx`

General test code for Python can be found here

```
import math
from xlcalcnet import ereal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
```

(continues on next page)

(continued from previous page)

```
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_ereal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = ereal.t(i)
    print('x = ereal.t(i):', x)

    x = ereal.t(5.7)
    print('x = erealT(5.7):', x)

    x = ereal.t(5.7)
    print('x = erealT(5.7):', x)
    x0 = ereal.t(232945639845394856394563936482734 6)
    print('x0 = erealT(232945639845394856394563936482734 6', x0)
    x1 = ereal.t("232945639845394856394563936482734 6")
    print('x1 = ereal.t("232945639845394856394563936482734 6"):', x1)
    x = ereal.t("5.5")
    print('x = erealT("5.5"):', x)

    print()
    x = ereal.t(55)
    print('x = ereal.t(5):', x)
    y = ereal.exp(x)
    print('y = ereal.exp(x):', y)

    z = ereal.exp(5.5)
    print('z = ereal.exp(5.5):', z)
    z = ereal.exp(5)
    print('z = ereal.exp(5):', z)
    z = ereal.exp("5.5")
    print('z = ereal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = ereal.exp(dec)
    print('z = ereal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = ereal.exp(frac)
    print('z = ereal.exp(frac):', z)

    print()
    x = ereal.t(5.5)
    print('x = ereal.t(55):', x)
    y = ereal.t(3.3)
    print('y = ereal.t(33):', y)
    z = ereal.pow(x, y)
    print('z = ereal.pow(x, y):        ', z)
    z = ereal.pow(5.5, 3.3)
    print('z = ereal.pow(5.5, 3.3):    ', z)
    z = ereal.pow("5.5", "3.3")
    print('z = ereal.pow("5.5", "3.3"):', z)
    z = ereal.pow(5, 3)
```

(continues on next page)

(continued from previous page)

```
    print('z = ereal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_ereal():
    print()
    print('<H1 Title="Arithmetic operators with ereal">')

    x = ereal.t(5.0)
    y = ereal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05b_ECplx.py.

1.6.5.5 Notes regarding the implementation in C# and C++

The C# source code for the module `ereal` can be found here: ereal.cs.

The C# source code for the module `ecplx` can be found here: ecplx.cs.

The C++ source code supporting scalars in general which is called from `ereal.cs` and `ecplx.cs` can be found here: UseXReal.cpp.

The C++ source code which interacts directly with Boost can be found here: BoostXReal.cpp.

The C++ source code supporting Eigen which is called from `ereal.cs` and `ecplx.cs` can be found here: UseEigenXReal.cpp.

The C++ source code which interacts directly with Eigen can be found in the folder: BoostEigenMath.cpp.

1.6.6 Binary floating point, quadruple precision: `qreal` and `qcplx`

The C# modules `qreal` and `qcplx` provide support for real and complex floating point numbers in quadruple precision (see https://en.wikipedia.org/wiki/Quadruple-precision_floating-point_format). The corresponding real and complex data types, `Quadruple` and `QuadrupleC`, have been added as objects in XlCalcNet (and have therefore the overhead of objects). They are implemented in software, using the `float128` data type of Boost Multiprecision. This means that the routines in quadruple precision provided in Boost and Eigen are much slower than the corresponding routines in extended precision.

The C# modules `qreal` and `qcplx` fully support Boost.Math and Eigen.

1.6.6.1 Examples in C#, `qreal`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some qreal functions");
    QrealTest0();
    QrealTest1();
}
```

General test code for C# can be found here

```
public static void QcplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "QcplxTest0"  + "\"" + ">");

    var a = qcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qcplx.abs(a);
    Console.WriteLine("res1 = qcplx.abs(a): {0}", res1);

    var res2 = qcplx.exp(a);
    Console.WriteLine("res2 = qcplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void QcplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "QcplxTest1"  + "\"" + ">");

    var a = qcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qcplx.asin(a);
    Console.WriteLine("res1 = qcplx.asin(a) {0}", res1);

    var res2 = qcplx.sin(a);
    Console.WriteLine("res2 = qcplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06a_Qreal.cs.

1.6.6.2 Examples in C#, `qcplx`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some qreal functions");
    QrealTest0();
    QrealTest1();
}
```

General test code for C# can be found here

```
public static void QcplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "QcplxTest0"  + "\"" + ">");

    var a = qcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qcplx.abs(a);
    Console.WriteLine("res1 = qcplx.abs(a): {0}", res1);

    var res2 = qcplx.exp(a);
    Console.WriteLine("res2 = qcplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void QcplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "QcplxTest1"  + "\"" + ">");

    var a = qcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qcplx.asin(a);
    Console.WriteLine("res1 = qcplx.asin(a) {0}", res1);

    var res2 = qcplx.sin(a);
    Console.WriteLine("res2 = qcplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02b_Qcplx.cs.

1.6.6.3 Examples in Python, qreal

General test code for Python can be found here

```
import math
from xlcalcnet import qreal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_qreal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = qreal.t(i)
    print('x = qreal.t(i):', x)

    x = qreal.t(5.7)
    print('x = qrealT(5.7):', x)

    x = qreal.t(5.7)
    print('x = qrealT(5.7):', x)
    x0 = qreal.t(232945639845394856394563936482734 6)
    print('x0 = qrealT(232945639845394856394563936482734 6', x0)
    x1 = qreal.t("232945639845394856394563936482734 6")
    print('x1 = qreal.t("232945639845394856394563936482734 6"):', x1)
    x = qreal.t("5.5")
    print('x = qrealT("5.5"):', x)

    print()
    x = qreal.t(55)
    print('x = qreal.t(5):', x)
    y = qreal.exp(x)
    print('y = qreal.exp(x):', y)

    z = qreal.exp(5.5)
    print('z = qreal.exp(5.5):', z)
    z = qreal.exp(5)
    print('z = qreal.exp(5):', z)
    z = qreal.exp("5.5")
    print('z = qreal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = qreal.exp(dec)
    print('z = qreal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = qreal.exp(frac)
    print('z = qreal.exp(frac):', z)

    print()
```

(continues on next page)

(continued from previous page)

```
    x = qreal.t(5.5)
    print('x = qreal.t(55):', x)
    y = qreal.t(3.3)
    print('y = qreal.t(33):', y)
    z = qreal.pow(x, y)
    print('z = qreal.pow(x, y):        ', z)
    z = qreal.pow(5.5, 3.3)
    print('z = qreal.pow(5.5, 3.3):    ', z)
    z = qreal.pow("5.5", "3.3")
    print('z = qreal.pow("5.5", "3.3"):', z)
    z = qreal.pow(5, 3)
    print('z = qreal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_qreal():
    print()
    print('<H1 Title="Arithmetic operators with qreal">')

    x = qreal.t(5.0)
    y = qreal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06a_QReal.py.

1.6.6.4 Examples in Python, `qcplx`

General test code for Python can be found here

```
import math
from xlcalcnet import qreal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
```

(continues on next page)

(continued from previous page)

```
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_qreal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = qreal.t(i)
    print('x = qreal.t(i):', x)

    x = qreal.t(5.7)
    print('x = qrealT(5.7):', x)

    x = qreal.t(5.7)
    print('x = qrealT(5.7):', x)
    x0 = qreal.t(232945639845394856394563936482734 6)
    print('x0 = qrealT(232945639845394856394563936482734 6', x0)
    x1 = qreal.t("232945639845394856394563936482734 6")
    print('x1 = qreal.t("232945639845394856394563936482734 6"):', x1)
    x = qreal.t("5.5")
    print('x = qrealT("5.5"):', x)

    print()
    x = qreal.t(55)
    print('x = qreal.t(5):', x)
    y = qreal.exp(x)
    print('y = qreal.exp(x):', y)

    z = qreal.exp(5.5)
    print('z = qreal.exp(5.5):', z)
    z = qreal.exp(5)
    print('z = qreal.exp(5):', z)
    z = qreal.exp("5.5")
    print('z = qreal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = qreal.exp(dec)
    print('z = qreal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = qreal.exp(frac)
    print('z = qreal.exp(frac):', z)

    print()
    x = qreal.t(5.5)
    print('x = qreal.t(55):', x)
    y = qreal.t(3.3)
    print('y = qreal.t(33):', y)
    z = qreal.pow(x, y)
    print('z = qreal.pow(x, y):        ', z)
    z = qreal.pow(5.5, 3.3)
    print('z = qreal.pow(5.5, 3.3):    ', z)
    z = qreal.pow("5.5", "3.3")
    print('z = qreal.pow("5.5", "3.3"):', z)
    z = qreal.pow(5, 3)
```

(continues on next page)

(continued from previous page)

```
    print('z = qreal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_qreal():
    print()
    print('<H1 Title="Arithmetic operators with qreal">')

    x = qreal.t(5.0)
    y = qreal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06b_QCplx.py.

1.6.6.5 Notes regarding the implementation in C# and C++

The C# source code for the module `qreal` can be found here: qreal.cs.

The C# source code for the module `qcplx` can be found here: qcplx.cs.

The C++ source code supporting scalars in general which is called from `qreal.cs` and `qcplx.cs` can be found here: UseQReal.cpp.

The C++ source code which interacts directly with Boost can be found here: BoostQReal.cpp.

The C++ source code supporting Eigen which is called from `qreal.cs` and `qcplx.cs` can be found here: UseEigenQReal.cpp.

The C++ source code which interacts directly with Eigen can be found in the folder: BoostEigenMath.cpp.

1.6.7 Binary floating point, octuple precision: `oreal` and `ocplx`

The C# modules `oreal` and `ocplx` provide support for real and complex floating point numbers in octuple precision (see https://en.wikipedia.org/wiki/Octuple-precision_floating-point_format). The corresponding real and complex data types, `Octuple` and `OctupleC`, have been added as objects in XlCalcNet (and have therefore the overhead of objects). They are implemented in software, using the `cpp_bin_float_oct` data type of Boost Multiprecision. The routines in octuple precision provided in Boost and Eigen are typically much slower than the corresponding routines in quadruple precision.

The C# modules `oreal` and `ocplx` fully support Boost.Math and Eigen.

1.6.7.1 Examples in C#, `oreal`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some oreal functions");
    OrealTest0();
    OrealTest1();
}
```

General test code for C# can be found here

```
public static void OcplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "OcplxTest0"  + "\"" + ">");

    var a = ocplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = ocplx.abs(a);
    Console.WriteLine("res1 = ocplx.abs(a): {0}", res1);

    var res2 = ocplx.exp(a);
    Console.WriteLine("res2 = ocplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void OcplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "OcplxTest1"  + "\"" + ">");

    var a = ocplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = ocplx.asin(a);
    Console.WriteLine("res1 = ocplx.asin(a) {0}", res1);

    var res2 = ocplx.sin(a);
    Console.WriteLine("res2 = ocplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_Oreal.cs.

1.6.7.2 Examples in C#, `ocplx`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some oreal functions");
    OrealTest0();
    OrealTest1();
}
```

General test code for C# can be found here

```
public static void OcplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "OcplxTest0"  + "\"" + ">");

    var a = ocplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = ocplx.abs(a);
    Console.WriteLine("res1 = ocplx.abs(a): {0}", res1);

    var res2 = ocplx.exp(a);
    Console.WriteLine("res2 = ocplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void OcplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "OcplxTest1"  + "\"" + ">");

    var a = ocplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = ocplx.asin(a);
    Console.WriteLine("res1 = ocplx.asin(a) {0}", res1);

    var res2 = ocplx.sin(a);
    Console.WriteLine("res2 = ocplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_Ocplx.cs.

1.6.7.3 Examples in Python, oreal

General test code for Python can be found here

```
import math
from xlcalcnet import oreal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_oreal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = oreal.t(i)
    print('x = oreal.t(i):', x)

    x = oreal.t(5.7)
    print('x = orealT(5.7):', x)

    x = oreal.t(5.7)
    print('x = orealT(5.7):', x)
    x0 = oreal.t(232945639845394856394563936482734 6)
    print('x0 = orealT(232945639845394856394563936482734 6', x0)
    x1 = oreal.t("232945639845394856394563936482734 6")
    print('x1 = oreal.t("232945639845394856394563936482734 6"):', x1)
    x = oreal.t("5.5")
    print('x = orealT("5.5"):', x)

    print()
    x = oreal.t(55)
    print('x = oreal.t(5):', x)
    y = oreal.exp(x)
    print('y = oreal.exp(x):', y)

    z = oreal.exp(5.5)
    print('z = oreal.exp(5.5):', z)
    z = oreal.exp(5)
    print('z = oreal.exp(5):', z)
    z = oreal.exp("5.5")
    print('z = oreal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = oreal.exp(dec)
    print('z = oreal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = oreal.exp(frac)
    print('z = oreal.exp(frac):', z)

    print()
```

(continues on next page)

(continued from previous page)

```
    x = oreal.t(5.5)
    print('x = oreal.t(55):', x)
    y = oreal.t(3.3)
    print('y = oreal.t(33):', y)
    z = oreal.pow(x, y)
    print('z = oreal.pow(x, y):        ', z)
    z = oreal.pow(5.5, 3.3)
    print('z = oreal.pow(5.5, 3.3):    ', z)
    z = oreal.pow("5.5", "3.3")
    print('z = oreal.pow("5.5", "3.3"):', z)
    z = oreal.pow(5, 3)
    print('z = oreal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_oreal():
    print()
    print('<H1 Title="Arithmetic operators with oreal">')

    x = oreal.t(5.0)
    y = oreal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_OReal.py.

1.6.7.4 Examples in Python, `ocplx`

General test code for Python can be found here

```
import math
from xlcalcnet import oreal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
```

(continues on next page)

(continued from previous page)

```
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_oreal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = oreal.t(i)
    print('x = oreal.t(i):', x)

    x = oreal.t(5.7)
    print('x = orealT(5.7):', x)

    x = oreal.t(5.7)
    print('x = orealT(5.7):', x)
    x0 = oreal.t(232945639845394856394563936482734 6)
    print('x0 = orealT(232945639845394856394563936482734 6', x0)
    x1 = oreal.t("232945639845394856394563936482734 6")
    print('x1 = oreal.t("232945639845394856394563936482734 6"):', x1)
    x = oreal.t("5.5")
    print('x = orealT("5.5"):', x)

    print()
    x = oreal.t(55)
    print('x = oreal.t(5):', x)
    y = oreal.exp(x)
    print('y = oreal.exp(x):', y)

    z = oreal.exp(5.5)
    print('z = oreal.exp(5.5):', z)
    z = oreal.exp(5)
    print('z = oreal.exp(5):', z)
    z = oreal.exp("5.5")
    print('z = oreal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = oreal.exp(dec)
    print('z = oreal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = oreal.exp(frac)
    print('z = oreal.exp(frac):', z)

    print()
    x = oreal.t(5.5)
    print('x = oreal.t(55):', x)
    y = oreal.t(3.3)
    print('y = oreal.t(33):', y)
    z = oreal.pow(x, y)
    print('z = oreal.pow(x, y):        ', z)
    z = oreal.pow(5.5, 3.3)
    print('z = oreal.pow(5.5, 3.3):    ', z)
    z = oreal.pow("5.5", "3.3")
    print('z = oreal.pow("5.5", "3.3"):', z)
    z = oreal.pow(5, 3)
```

(continues on next page)

(continued from previous page)

```
    print('z = oreal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_oreal():
    print()
    print('<H1 Title="Arithmetic operators with oreal">')

    x = oreal.t(5.0)
    y = oreal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_OCplx.py.

1.6.7.5 Notes regarding the implementation in C# and C++

The C# source code for the module `oreal` can be found here: oreal.cs.

The C# source code for the module `ocplx` can be found here: ocplx.cs.

The C++ source code supporting scalars in general which is called from `oreal.cs` and `ocplx.cs` can be found here: UseOReal.cpp.

The C++ source code which interacts directly with Boost can be found here: BoostSReal.cpp.

The C++ source code supporting Eigen which is called from `oreal.cs` and `ocplx.cs` can be found here: UseEigenSReal.cpp.

The C++ source code which interacts directly with Eigen can be found in the folder: BoostEigenMath.cpp.

1.6.8 Use of Numpy with scalar functions written in C#: `np53`, `np53c`

Some text

1.6.9 Building the underlying C#, C/C++ and Pascal libraries

On the Pascal/C/C++ side, XlCalcNet uses DAMath, Boost Math, Boost Multiprecision, Boost Odeint and Eigen to provide numerical functions in single, double, extended, quadruple and octuple precision, which are available to the user both from C# and Python.

The SourceOfBasicLibraries repository provides copies of the source code of the underlying numerical libraries, which are required to build the XlCalcNet library. These copies also include small patches, as required. They are provided to make it easier to reproduce the compilation results, as distributed as part of the XlCalcNet repository.

1.6.9.1 Installing MSYS2

Text descibing MSYS2

1.6.9.2 Eigen, (version 5.0.1)

Text descibing Eigen

1.6.9.3 Boost, (version 1.90.0)

Text descibing Boost

1.6.9.4 DAMath (version 2.27)

Text descibing DAMath

1.6.9.5 FixedPreNet (version 1.0.0)

Text descibing FixedPreNet

1.7 Mathematical functions based on XlCalcNet2 (C#, can be called from Python)

Assuming that the Python batch files have been installed as described *here*, XlCalcNet2 can be installed by double-clicking on the batch file `A13installxlcalcnet2.bat` in the `C:\Python313` folder.

1.7.1 Binary floating point, multiple precision: `mreal`, `mcplx`

The C# modules `mreal` and `mcplx` provide support for real and complex floating point numbers with an arbitrary precision significand and a fixed size exponent. The corresponding real and complex data types, `Mpfr` and `MpfrC`, have been added as objects in XlCalcNet2 (and have therefore the overhead of objects). They are implemented in software, using the `mpfr_t` data type of the MPFR library (see https://www.mpfr.org/) and the `mpc_t` data type of the MPC library (see https://www.multiprecision.org/mpc/).

The C# modules `mreal` and `mcplx` fully support Boost.Math and Eigen.

1.7.1.1 Examples in C#, `mreal`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some mreal functions");
    MrealTest0();
    MrealTest1();
}
```

General test code for C# can be found here

```
public static void McplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "McplxTest0"  + "\"" + ">");

    var a = mcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = mcplx.abs(a);
    Console.WriteLine("res1 = mcplx.abs(a): {0}", res1);

    var res2 = mcplx.exp(a);
    Console.WriteLine("res2 = mcplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void McplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "McplxTest1"  + "\"" + ">");

    var a = mcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = mcplx.asin(a);
    Console.WriteLine("res1 = mcplx.asin(a) {0}", res1);
```

(continues on next page)

(continued from previous page)

```
    var res2 = mcplx.sin(a);
    Console.WriteLine("res2 = mcplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01a_Mreal.cs.

1.7.1.2 Examples in C#, `mcplx`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some mreal functions");
    MrealTest0();
    MrealTest1();
}
```

General test code for C# can be found here

```
public static void McplxTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "McplxTest0"  + "\"" + ">");

    var a = mcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = mcplx.abs(a);
    Console.WriteLine("res1 = mcplx.abs(a): {0}", res1);

    var res2 = mcplx.exp(a);
    Console.WriteLine("res2 = mcplx.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void McplxTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "McplxTest1"  + "\"" + ">");

    var a = mcplx.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = mcplx.asin(a);
    Console.WriteLine("res1 = mcplx.asin(a) {0}", res1);

    var res2 = mcplx.sin(a);
    Console.WriteLine("res2 = mcplx.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01b_Mcplx.cs.

1.7.1.3 Examples in Python, `mreal`

General test code for Python can be found here

```
import math
from xlcalcnet import mreal
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3

def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_mreal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = mreal.t(i)
    print('x = mreal.t(i):', x)

    x = mreal.t(5.7)
    print('x = mrealT(5.7):', x)

    x = mreal.t(5.7)
    print('x = mrealT(5.7):', x)
    x0 = mreal.t(23294563984539485639456393648273)
    print('x0 = mrealT(23294563984539485639456393648273', x0)
    x1 = mreal.t("23294563984539485639456393648273")
    print('x1 = mreal.t("23294563984539485639456393648273"):', x1)
    x = mreal.t("5.5")
    print('x = mrealT("5.5"):', x)

    print()
    x = mreal.t(55)
    print('x = mreal.t(5):', x)
    y = mreal.exp(x)
    print('y = mreal.exp(x):', y)

    z = mreal.exp(5.5)
    print('z = mreal.exp(5.5):', z)
    z = mreal.exp(5)
    print('z = mreal.exp(5):', z)
    z = mreal.exp("5.5")
    print('z = mreal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = mreal.exp(dec)
    print('z = mreal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
```

(continues on next page)

(continued from previous page)

```
    z = mreal.exp(frac)
    print('z = mreal.exp(frac):', z)

    print()
    x = mreal.t(5.5)
    print('x = mreal.t(55):', x)
    y = mreal.t(3.3)
    print('y = mreal.t(33):', y)
    z = mreal.pow(x, y)
    print('z = mreal.pow(x, y):        ', z)
    z = mreal.pow(5.5, 3.3)
    print('z = mreal.pow(5.5, 3.3):    ', z)
    z = mreal.pow("5.5", "3.3")
    print('z = mreal.pow("5.5", "3.3"):', z)
    z = mreal.pow(5, 3)
    print('z = mreal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_mreal():
    print()
    print('<H1 Title="Arithmetic operators with mreal">')

    x = mreal.t(5.0)
    y = mreal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01a_MReal.py.

1.7.1.4 Examples in Python, `mcplx`

General test code for Python can be found here

```
import math
from xlcalcnet import mreal
from decimal import Decimal
from fractions import Fraction
```

(continues on next page)

(continued from previous page)

```
i = 232945639845394856394563936482734638475398457398457398457 3

def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_mreal()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = mreal.t(i)
    print('x = mreal.t(i):', x)

    x = mreal.t(5.7)
    print('x = mrealT(5.7):', x)

    x = mreal.t(5.7)
    print('x = mrealT(5.7):', x)
    x0 = mreal.t(232945639845394856394563936482734 6)
    print('x0 = mrealT(232945639845394856394563936482734 6', x0)
    x1 = mreal.t("232945639845394856394563936482734 6")
    print('x1 = mreal.t("232945639845394856394563936482734 6"):', x1)
    x = mreal.t("5.5")
    print('x = mrealT("5.5"):', x)

    print()
    x = mreal.t(55)
    print('x = mreal.t(5):', x)
    y = mreal.exp(x)
    print('y = mreal.exp(x):', y)

    z = mreal.exp(5.5)
    print('z = mreal.exp(5.5):', z)
    z = mreal.exp(5)
    print('z = mreal.exp(5):', z)
    z = mreal.exp("5.5")
    print('z = mreal.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = mreal.exp(dec)
    print('z = mreal.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = mreal.exp(frac)
    print('z = mreal.exp(frac):', z)

    print()
    x = mreal.t(5.5)
    print('x = mreal.t(55):', x)
    y = mreal.t(3.3)
    print('y = mreal.t(33):', y)
    z = mreal.pow(x, y)
    print('z = mreal.pow(x, y):        ', z)
    z = mreal.pow(5.5, 3.3)
```

(continues on next page)

(continued from previous page)

```
    print('z = mreal.pow(5.5, 3.3):    ', z)
    z = mreal.pow("5.5", "3.3")
    print('z = mreal.pow("5.5", "3.3"):', z)
    z = mreal.pow(5, 3)
    print('z = mreal.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_mreal():
    print()
    print('<H1 Title="Arithmetic operators with mreal">')

    x = mreal.t(5.0)
    y = mreal.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02b_TestMCplx.py.

1.7.1.5 Notes regarding the implementation in C# and C++

The C# source code for the module `mreal` can be found here: mreal.cs.

The C# source code for the module `mcplx` can be found here: mcplx.cs.

The C++ source code supporting scalars in general which is called from `mreal.cs` and `mcplx.cs` can be found here: UseMpfr.cpp.

The C++ source code which interacts directly with Boost can be found here: BoostMpfr.cpp.

The C++ source code supporting Eigen which is called from `mreal.cs` and `mcplx.cs` can be found here: UseAnyEigenMat.cpp.

The C++ source code which interacts directly with Eigen can be found in the folder: BoostEigenMath.cpp.

1.7.2 Binary arbitrary precision ball: `aflint` and `aflintc`

The C# modules `aflint` and `aflintc` provide support for real and complex ball arithmetic with an arbitrary precision significand and an arbitrary size exponent. The corresponding real and complex data types, `Arb` and `ArbC`, have been added as objects in XlCalcNet2 (and have therefore the overhead of objects). They are implemented in software, using the `arb_t` and and `acb_t` data types of the FLINT library (see https://flintlib.org/).

The C# modules `mreal` and `mcplx` partially support Eigen.

Note: Exponent soft maximum and minimum

1.7.2.1 Examples in C#, `aflint`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some aflint functions");
    AflintTest0();
    AflintTest1();
}
```

General test code for C# can be found here

```
public static void AflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "AflintcTest0"  + "\"" + ">");

    var a = aflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = aflintc.abs(a);
    Console.WriteLine("res1 = aflintc.abs(a): {0}", res1);

    var res2 = aflintc.exp(a);
    Console.WriteLine("res2 = aflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void AflintcTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "AflintcTest1"  + "\"" + ">");

    var a = aflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = aflintc.asin(a);
    Console.WriteLine("res1 = aflintc.asin(a) {0}", res1);

    var res2 = aflintc.sin(a);
    Console.WriteLine("res2 = aflintc.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the DataXlCalcNet repository or in the corresponding local DataXlCalcNet folder in the file D02a_Aflint.cs.

1.7.2.2 Examples in C#, aflintc

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some aflint functions");
    AflintTest0();
    AflintTest1();
}
```

General test code for C# can be found here

```
public static void AflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "AflintcTest0"  + "\"" + ">");

    var a = aflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = aflintc.abs(a);
    Console.WriteLine("res1 = aflintc.abs(a): {0}", res1);

    var res2 = aflintc.exp(a);
    Console.WriteLine("res2 = aflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

General test code for C# can be found here

```
public static void AflintcTest1()
{
    Console.WriteLine("<H1 Title=" + "\"" + "AflintcTest1"  + "\"" + ">");

    var a = aflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = aflintc.asin(a);
    Console.WriteLine("res1 = aflintc.asin(a) {0}", res1);

    var res2 = aflintc.sin(a);
    Console.WriteLine("res2 = aflintc.sin(a) {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the DataXlCalcNet repository or in the corresponding local DataXlCalcNet folder in the file D02b_Aflintc.cs.

1.7.2.3 Examples in Python, `aflint`

General test code for Python can be found here

```
import math
from xlcalcnet import aflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_aflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = aflint.t(i)
    print('x = aflint.t(i):', x)

    x = aflint.t(5.7)
    print('x = aflintT(5.7):', x)

    x = aflint.t(5.7)
    print('x = aflintT(5.7):', x)
    x0 = aflint.t(232945639845394856394563936482734 6)
    print('x0 = aflintT(232945639845394856394563936482734 6', x0)
    x1 = aflint.t("232945639845394856394563936482734 6")
    print('x1 = aflint.t("232945639845394856394563936482734 6"):', x1)
    x = aflint.t("5.5")
    print('x = aflintT("5.5"):', x)

    print()
    x = aflint.t(55)
    print('x = aflint.t(5):', x)
    y = aflint.exp(x)
    print('y = aflint.exp(x):', y)

    z = aflint.exp(5.5)
    print('z = aflint.exp(5.5):', z)
    z = aflint.exp(5)
    print('z = aflint.exp(5):', z)
    z = aflint.exp("5.5")
    print('z = aflint.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = aflint.exp(dec)
    print('z = aflint.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = aflint.exp(frac)
    print('z = aflint.exp(frac):', z)

    print()
```

(continues on next page)

(continued from previous page)

```
    x = aflint.t(5.5)
    print('x = aflint.t(55):', x)
    y = aflint.t(3.3)
    print('y = aflint.t(33):', y)
    z = aflint.pow(x, y)
    print('z = aflint.pow(x, y):        ', z)
    z = aflint.pow(5.5, 3.3)
    print('z = aflint.pow(5.5, 3.3):    ', z)
    z = aflint.pow("5.5", "3.3")
    print('z = aflint.pow("5.5", "3.3"):', z)
    z = aflint.pow(5, 3)
    print('z = aflint.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_aflint():
    print()
    print('<H1 Title="Arithmetic operators with aflint">')

    x = aflint.t(5.0)
    y = aflint.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02a_AFlint.py.

1.7.2.4 Examples in Python, `aflintc`

General test code for Python can be found here

```
import math
from xlcalcnet import aflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
```

(continues on next page)

(continued from previous page)

```
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_aflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = aflint.t(i)
    print('x = aflint.t(i):', x)

    x = aflint.t(5.7)
    print('x = aflintT(5.7):', x)

    x = aflint.t(5.7)
    print('x = aflintT(5.7):', x)
    x0 = aflint.t(232945639845394856394563936482 7346)
    print('x0 = aflintT(232945639845394856394563936482 7346', x0)
    x1 = aflint.t("232945639845394856394563936482 7346")
    print('x1 = aflint.t("232945639845394856394563936482 7346"):', x1)
    x = aflint.t("5.5")
    print('x = aflintT("5.5"):', x)

    print()
    x = aflint.t(55)
    print('x = aflint.t(5):', x)
    y = aflint.exp(x)
    print('y = aflint.exp(x):', y)

    z = aflint.exp(5.5)
    print('z = aflint.exp(5.5):', z)
    z = aflint.exp(5)
    print('z = aflint.exp(5):', z)
    z = aflint.exp("5.5")
    print('z = aflint.exp("5.5"):', z)
    print('</H1>')
```

General test code can be found here

```
def functions_with_argument_conversion():
    print()
    print('<H1 Title="Functions with argument conversion">')
    dec = Decimal(1) / Decimal(7)
    print('dec = Decimal(1) / Decimal(7):', dec)
    z = aflint.exp(dec)
    print('z = aflint.exp(dec):', z)
    frac = Fraction("-3/7")
    print('frac = Fraction("-3/7:")', frac)
    z = aflint.exp(frac)
    print('z = aflint.exp(frac):', z)

    print()
    x = aflint.t(5.5)
    print('x = aflint.t(55):', x)
    y = aflint.t(3.3)
    print('y = aflint.t(33):', y)
    z = aflint.pow(x, y)
    print('z = aflint.pow(x, y):        ', z)
    z = aflint.pow(5.5, 3.3)
    print('z = aflint.pow(5.5, 3.3):    ', z)
    z = aflint.pow("5.5", "3.3")
    print('z = aflint.pow("5.5", "3.3"):', z)
    z = aflint.pow(5, 3)
```

(continues on next page)

(continued from previous page)

```
    print('z = aflint.pow(5, 3):', z)

    t = z + 3
    print('t = z + 3:', t)
    print('</H1>')
```

General test code can be found here

```
def arithmetic_operators_with_aflint():
    print()
    print('<H1 Title="Arithmetic operators with aflint">')

    x = aflint.t(5.0)
    y = aflint.t(2.5)
    print('x: ', x)
    print('y: ', y)

    res = x + y
    print('res = x + y:', res)
    res = y + x
    print('res = y + x:', res)

    res = x - y
    print('res = x - y:', res)
    res = y - x
    print('res = y - x:', res)

    res = x * y
    print('res = x * y:', res)
    res = y * x
    print('res = y * x:', res)

    res = x / y
    print('res = x / y:', res)
    res = y / x
    print('res = y / x:', res)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02b_AFlintc.py.

1.7.2.5 Notes regarding the implementation in C# and C++

The C# source code for the module `aflint` can be found here: aflint.cs.

The C# source code for the module `aflintc` can be found here: aflintc.cs.

The C++ source code supporting scalars in general which is called from `aflint.cs` and `aflintc.cs` can be found here: UseArb.cpp.

The C++ source code supporting Eigen which is called from `aflint.cs` and `aflintc.cs` can be found here: UseAnyEigenMat.cpp.

The C++ source code which interacts directly with Eigen can be found in the folder: BoostEigenMath.cpp.

1.7.3 Binary floating point, single precision: `sflint` and `sflintc`

The C# modules `sflint` and `sflintc` provide a rich set of elementary and special real and complex scalar functions for the `Single` and `Single` data types, with the actual calculations done using the `Arb` and `ArbC` data types, which allows control of the absolute and relative error of the result.

1.7.3.1 Examples in C#, sflint

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some sflint functions");
    SflintTest0();
    SflintTest1();
}
```

General test code for C# can be found here

```
public static void SflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "SflintcTest0"  + "\"" + ">");

    var a = qflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qflintc.abs(a);
    Console.WriteLine("res1 = qflintc.abs(a): {0}", res1);

    var res2 = qflintc.exp(a);
    Console.WriteLine("res2 = qflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03a_Sflint.cs.

1.7.3.2 Examples in C#, sflintc

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some sflint functions");
    SflintTest0();
    SflintTest1();
}
```

General test code for C# can be found here

```
public static void SflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "SflintcTest0"  + "\"" + ">");

    var a = qflintc.t(3.5, 6.3);
```

(continues on next page)

(continued from previous page)

```
    Console.WriteLine("a: {0}", a);

    var res1 = qflintc.abs(a);
    Console.WriteLine("res1 = qflintc.abs(a): {0}", res1);

    var res2 = qflintc.exp(a);
    Console.WriteLine("res2 = qflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03b_Sflintc.cs.

1.7.3.3 Examples in Python, `sflint`

General test code for Python can be found here

```
import math
from xlcalcnet import sflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_sflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = sflint.t(i)
    print('x = sflint.t(i):', x)

    x = sflint.t(5.7)
    print('x = sflintT(5.7):', x)

    x = sflint.t(5.7)
    print('x = sflintT(5.7):', x)
    x0 = sflint.t(232945639845394856394563936482734 6)
    print('x0 = sflintT(232945639845394856394563936482734 6', x0)
    x1 = sflint.t("232945639845394856394563936482734 6")
    print('x1 = sflint.t("232945639845394856394563936482734 6"):', x1)
    x = sflint.t("5.5")
    print('x = sflintT("5.5"):', x)

    print()
    x = sflint.t(55)
    print('x = sflint.t(5):', x)
    y = sflint.exp(x)
    print('y = sflint.exp(x):', y)

    z = sflint.exp(5.5)
    print('z = sflint.exp(5.5):', z)
    z = sflint.exp(5)
    print('z = sflint.exp(5):', z)
    z = sflint.exp("5.5")
    print('z = sflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03a_SFlint.py.

1.7.3.4 Examples in Python, `sflintc`

General test code for Python can be found here

```
import math
from xlcalcnet import sflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3

def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_sflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = sflint.t(i)
    print('x = sflint.t(i):', x)

    x = sflint.t(5.7)
    print('x = sflintT(5.7):', x)

    x = sflint.t(5.7)
    print('x = sflintT(5.7):', x)
    x0 = sflint.t(23294563984539485639456393648273 46)
    print('x0 = sflintT(23294563984539485639456393648273 46', x0)
    x1 = sflint.t("23294563984539485639456393648273 46")
    print('x1 = sflint.t("23294563984539485639456393648273 46"):', x1)
    x = sflint.t("5.5")
    print('x = sflintT("5.5"):', x)

    print()
    x = sflint.t(55)
    print('x = sflint.t(5):', x)
    y = sflint.exp(x)
    print('y = sflint.exp(x):', y)

    z = sflint.exp(5.5)
    print('z = sflint.exp(5.5):', z)
    z = sflint.exp(5)
    print('z = sflint.exp(5):', z)
    z = sflint.exp("5.5")
    print('z = sflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03b_SFlintc.py.

1.7.3.5 Notes regarding the implementation in C# and C++

The C# source code for the modules `sflint` and `sflintc` can be found here: srealcplxflint.cs.

The C++ source code which is called from `srealcplxflint.cs` can be found here: UseSReal.cpp.

1.7.4 Binary floating point, double precision: dflint and dflintc

The C# modules dflint and dflintc provide a rich set of elementary and special real and complex scalar functions for the Double and Complex data types, with the actual calculations done using the Arb and ArbC data types, which allows control of the absolute and relative error of the result.

1.7.4.1 Examples in C#, dflint

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some sflint functions");
    DflintTest0();
    DflintTest1();
}
```

General test code for C# can be found here

```
public static void DflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "DflintcTest0"  + "\"" + ">");

    var a = qflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qflintc.abs(a);
    Console.WriteLine("res1 = qflintc.abs(a): {0}", res1);

    var res2 = qflintc.exp(a);
    Console.WriteLine("res2 = qflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the DataXlCalcNet repository or in the corresponding local DataXlCalcNet folder in the file D04a_Dflint.cs.

1.7.4.2 Examples in C#, dflintc

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some sflint functions");
    DflintTest0();
    DflintTest1();
}
```

General test code for C# can be found here

```
public static void DflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "DflintcTest0"  + "\"" + ">");

    var a = qflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qflintc.abs(a);
    Console.WriteLine("res1 = qflintc.abs(a): {0}", res1);

    var res2 = qflintc.exp(a);
    Console.WriteLine("res2 = qflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04b_Dflintc.cs.

1.7.4.3 Examples in Python, `dflint`

General test code for Python can be found here

```
import math
from xlcalcnet import sflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3984573


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_sflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = sflint.t(i)
    print('x = sflint.t(i):', x)

    x = sflint.t(5.7)
    print('x = sflintT(5.7):', x)

    x = sflint.t(5.7)
    print('x = sflintT(5.7):', x)
    x0 = sflint.t(23294563984539485639456393648273463847539845739845 73)
    print('x0 = sflintT(23294563984539485639456393648273463847539845739845 73', x0)
    x1 = sflint.t("23294563984539485639456393648273463847539845739845 73")
    print('x1 = sflint.t("23294563984539485639456393648273463847539845739845 73"):', x1)
    x = sflint.t("5.5")
    print('x = sflintT("5.5"):', x)

    print()
    x = sflint.t(55)
    print('x = sflint.t(5):', x)
    y = sflint.exp(x)
    print('y = sflint.exp(x):', y)

    z = sflint.exp(5.5)
    print('z = sflint.exp(5.5):', z)
    z = sflint.exp(5)
```

(continues on next page)

(continued from previous page)

```
    print('z = sflint.exp(5):', z)
    z = sflint.exp("5.5")
    print('z = sflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04a_DFlint.py.

1.7.4.4 Examples in Python, `dflintc`

General test code for Python can be found here

```
import math
from xlcalcnet import sflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_sflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = sflint.t(i)
    print('x = sflint.t(i):', x)

    x = sflint.t(5.7)
    print('x = sflintT(5.7):', x)

    x = sflint.t(5.7)
    print('x = sflintT(5.7):', x)
    x0 = sflint.t(23294563984539485639456393648273463847539845739845 73)
    print('x0 = sflintT(23294563984539485639456393648273463847539845739845 73', x0)
    x1 = sflint.t("23294563984539485639456393648273463847539845739845 73")
    print('x1 = sflint.t("23294563984539485639456393648273463847539845739845 73"):', x1)
    x = sflint.t("5.5")
    print('x = sflintT("5.5"):', x)

    print()
    x = sflint.t(55)
    print('x = sflint.t(5):', x)
    y = sflint.exp(x)
    print('y = sflint.exp(x):', y)

    z = sflint.exp(5.5)
    print('z = sflint.exp(5.5):', z)
    z = sflint.exp(5)
    print('z = sflint.exp(5):', z)
    z = sflint.exp("5.5")
    print('z = sflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04b_DFlintc.py.

1.7.4.5 Notes regarding the implementation in C# and C++

The C# source code for the modules `dflint` and `dflintc` can be found here: drealcplxflint.cs.

The C++ source code which is called from `drealcplxflint.cs` can be found here: UseFReal.cpp.

1.7.5 Binary floating point, extended precision: eflint and eflintc

The C# modules eflint and eflintc provide a rich set of elementary and special real and complex scalar functions for the Extended and ExtendedC data types, with the actual calculations done using the Arb and ArbC data types, which allows control of the absolute and relative error of the result.

1.7.5.1 Examples in C#, eflint

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some eflint functions");
    EflintTest0();
    EflintTest1();
}
```

General test code for C# can be found here

```
public static void EflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "EflintcTest0"  + "\"" + ">");

    var a = eflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = eflintc.abs(a);
    Console.WriteLine("res1 = eflintc.abs(a): {0}", res1);

    var res2 = eflintc.exp(a);
    Console.WriteLine("res2 = eflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the DataXlCalcNet repository or in the corresponding local DataXlCalcNet folder in the file D05a_Eflint.cs.

1.7.5.2 Examples in C#, eflintc

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some eflint functions");
    EflintTest0();
    EflintTest1();
}
```

General test code for C# can be found here

```
public static void EflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "EflintcTest0"  + "\"" + ">");

    var a = eflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = eflintc.abs(a);
    Console.WriteLine("res1 = eflintc.abs(a): {0}", res1);

    var res2 = eflintc.exp(a);
    Console.WriteLine("res2 = eflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05b_Eflintc.cs.

1.7.5.3 Examples in Python, `eflint`

General test code for Python can be found here

```
import math
from xlcalcnet import eflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3984573


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_eflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = eflint.t(i)
    print('x = eflint.t(i):', x)

    x = eflint.t(5.7)
    print('x = eflintT(5.7):', x)

    x = eflint.t(5.7)
    print('x = eflintT(5.7):', x)
    x0 = eflint.t(23294563984539485639456393648273463847539845739845 73)
    print('x0 = eflintT(23294563984539485639456393648273463847539845739845 73', x0)
    x1 = eflint.t("23294563984539485639456393648273463847539845739845 73")
    print('x1 = eflint.t("23294563984539485639456393648273463847539845739845 73"):', x1)
    x = eflint.t("5.5")
    print('x = eflintT("5.5"):', x)

    print()
    x = eflint.t(55)
    print('x = eflint.t(5):', x)
    y = eflint.exp(x)
    print('y = eflint.exp(x):', y)

    z = eflint.exp(5.5)
    print('z = eflint.exp(5.5):', z)
    z = eflint.exp(5)
```

(continues on next page)

(continued from previous page)

```
    print('z = eflint.exp(5):', z)
    z = eflint.exp("5.5")
    print('z = eflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05a_EFlint.py.

1.7.5.4 Examples in Python, `eflintc`

General test code for Python can be found here

```
import math
from xlcalcnet import eflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_eflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = eflint.t(i)
    print('x = eflint.t(i):', x)

    x = eflint.t(5.7)
    print('x = eflintT(5.7):', x)

    x = eflint.t(5.7)
    print('x = eflintT(5.7):', x)
    x0 = eflint.t(23294563984539485639456393648273)
    print('x0 = eflintT(23294563984539485639456393648273', x0)
    x1 = eflint.t("23294563984539485639456393648273")
    print('x1 = eflint.t("23294563984539485639456393648273"):', x1)
    x = eflint.t("5.5")
    print('x = eflintT("5.5"):', x)

    print()
    x = eflint.t(55)
    print('x = eflint.t(5):', x)
    y = eflint.exp(x)
    print('y = eflint.exp(x):', y)

    z = eflint.exp(5.5)
    print('z = eflint.exp(5.5):', z)
    z = eflint.exp(5)
    print('z = eflint.exp(5):', z)
    z = eflint.exp("5.5")
    print('z = eflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05b_EFlintc.py.

1.7.5.5 Notes regarding the implementation in C# and C++

The C# source code for the modules `eflint` and `eflintc` can be found here: erealcplxflint.cs.

The C++ source code which is called from `erealcplxflint.cs` can be found here: UseXReal.cpp.

1.7.6 Binary floating point, quadruple precision: `qflint` and `qflintc`

The C# modules `qflint` and `qflintc` provide a rich set of elementary and special real and complex scalar functions for the `Quadruple` and `QuadrupleC` data types, with the actual calculations done using the `Arb` and `ArbC` data types, which allows control of the absolute and relative error of the result.

1.7.6.1 Examples in C#, `qflint`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some qflint functions");
    SflintTest0();
    SflintTest1();
}
```

General test code for C# can be found here

```
public static void QflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "QflintcTest0"  + "\"" + ">");

    var a = qflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qflintc.abs(a);
    Console.WriteLine("res1 = qflintc.abs(a): {0}", res1);

    var res2 = qflintc.exp(a);
    Console.WriteLine("res2 = qflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06a_Qflint.cs.

1.7.6.2 Examples in C#, `qflintc`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some qflint functions");
    SflintTest0();
    SflintTest1();
}
```

General test code for C# can be found here

```
public static void QflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "QflintcTest0"  + "\"" + ">");

    var a = qflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qflintc.abs(a);
    Console.WriteLine("res1 = qflintc.abs(a): {0}", res1);

    var res2 = qflintc.exp(a);
    Console.WriteLine("res2 = qflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06b_Qflintc.cs.

1.7.6.3 Examples in Python, `qflint`

General test code for Python can be found here

```
import math
from xlcalcnet import qflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3984573


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_qflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = qflint.t(i)
    print('x = qflint.t(i):', x)

    x = qflint.t(5.7)
    print('x = qflintT(5.7):', x)

    x = qflint.t(5.7)
    print('x = qflintT(5.7):', x)
    x0 = qflint.t(23294563984539485639456393648273463847539845739845 73984573)
    print('x0 = qflintT(23294563984539485639456393648273463847539845739845 73984573', x0)
    x1 = qflint.t("23294563984539485639456393648273463847539845739845 73984573")
    print('x1 = qflint.t("23294563984539485639456393648273463847539845739845 73984573"):', x1)
    x = qflint.t("5.5")
    print('x = qflintT("5.5"):', x)

    print()
    x = qflint.t(55)
    print('x = qflint.t(5):', x)
    y = qflint.exp(x)
    print('y = qflint.exp(x):', y)

    z = qflint.exp(5.5)
    print('z = qflint.exp(5.5):', z)
    z = qflint.exp(5)
```

(continues on next page)

(continued from previous page)

```
    print('z = qflint.exp(5):', z)
    z = qflint.exp("5.5")
    print('z = qflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06a_QFlint.py.

1.7.6.4 Examples in Python, `qflintc`

General test code for Python can be found here

```
import math
from xlcalcnet import qflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_qflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = qflint.t(i)
    print('x = qflint.t(i):', x)

    x = qflint.t(5.7)
    print('x = qflintT(5.7):', x)

    x = qflint.t(5.7)
    print('x = qflintT(5.7):', x)
    x0 = qflint.t(232945639845394856394563936482734 6)
    print('x0 = qflintT(232945639845394856394563936482734 6', x0)
    x1 = qflint.t("232945639845394856394563936482734 6")
    print('x1 = qflint.t("232945639845394856394563936482734 6"):', x1)
    x = qflint.t("5.5")
    print('x = qflintT("5.5"):', x)

    print()
    x = qflint.t(55)
    print('x = qflint.t(5):', x)
    y = qflint.exp(x)
    print('y = qflint.exp(x):', y)

    z = qflint.exp(5.5)
    print('z = qflint.exp(5.5):', z)
    z = qflint.exp(5)
    print('z = qflint.exp(5):', z)
    z = qflint.exp("5.5")
    print('z = qflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06b_QFlintc.py.

1.7.6.5 Notes regarding the implementation in C# and C++

The C# source code for the modules `qflint` and `qflintc` can be found here: qrealcplxflint.cs.

The C++ source code which is called from `qrealcplxflint.cs` can be found here: UseQReal.cpp.

1.7.7 Binary floating point, octuple precision: `oflint` and `oflintc`

The C# modules `oflint` and `oflintc` provide a rich set of elementary and special real and complex scalar functions for the `Octuple` and `OctupleC` data types, with the actual calculations done using the `Arb` and `ArbC` data types, which allows control of the absolute and relative error of the result.

1.7.7.1 Examples in C#, `oflint`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some oflint functions");
    OflintTest0();
    OflintTest1();
}
```

General test code for C# can be found here

```
public static void OflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "OflintcTest0"  + "\"" + ">");

    var a = qflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qflintc.abs(a);
    Console.WriteLine("res1 = qflintc.abs(a): {0}", res1);

    var res2 = qflintc.exp(a);
    Console.WriteLine("res2 = qflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_Oflint.cs.

1.7.7.2 Examples in C#, `oflintc`

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some oflint functions");
    OflintTest0();
    OflintTest1();
}
```

General test code for C# can be found here

```
public static void OflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "OflintcTest0"  + "\"" + ">");

    var a = qflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qflintc.abs(a);
    Console.WriteLine("res1 = qflintc.abs(a): {0}", res1);

    var res2 = qflintc.exp(a);
    Console.WriteLine("res2 = qflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_Oflintc.cs.

1.7.7.3 Examples in Python, `oflint`

General test code for Python can be found here

```
import math
from xlcalcnet import oflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3984573


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_oflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = oflint.t(i)
    print('x = oflint.t(i):', x)

    x = oflint.t(5.7)
    print('x = oflintT(5.7):', x)

    x = oflint.t(5.7)
    print('x = oflintT(5.7):', x)
    x0 = oflint.t(23294563984539485639456393648273463847539845739845 73984573)
    print('x0 = oflintT(23294563984539485639456393648273463847539845739845 73984573', x0)
    x1 = oflint.t("23294563984539485639456393648273463847539845739845 73984573")
    print('x1 = oflint.t("23294563984539485639456393648273463847539845739845 73984573"):', x1)
    x = oflint.t("5.5")
    print('x = oflintT("5.5"):', x)

    print()
    x = oflint.t(55)
    print('x = oflint.t(5):', x)
    y = oflint.exp(x)
    print('y = oflint.exp(x):', y)

    z = oflint.exp(5.5)
    print('z = oflint.exp(5.5):', z)
    z = oflint.exp(5)
```

(continues on next page)

(continued from previous page)

```
    print('z = oflint.exp(5):', z)
    z = oflint.exp("5.5")
    print('z = oflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_OFlint.py.

1.7.7.4 Examples in Python, `oflintc`

General test code for Python can be found here

```
import math
from xlcalcnet import oflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_oflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = oflint.t(i)
    print('x = oflint.t(i):', x)

    x = oflint.t(5.7)
    print('x = oflintT(5.7):', x)

    x = oflint.t(5.7)
    print('x = oflintT(5.7):', x)
    x0 = oflint.t(23294563984539485639456393648273)
    print('x0 = oflintT(23294563984539485639456393648273', x0)
    x1 = oflint.t("23294563984539485639456393648273")
    print('x1 = oflint.t("23294563984539485639456393648273"):', x1)
    x = oflint.t("5.5")
    print('x = oflintT("5.5"):', x)

    print()
    x = oflint.t(55)
    print('x = oflint.t(5):', x)
    y = oflint.exp(x)
    print('y = oflint.exp(x):', y)

    z = oflint.exp(5.5)
    print('z = oflint.exp(5.5):', z)
    z = oflint.exp(5)
    print('z = oflint.exp(5):', z)
    z = oflint.exp("5.5")
    print('z = oflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_OFlintc.py.

1.7.7.5 Notes regarding the implementation in C# and C++

The C# source code for the modules `oflint` and `oflintc` can be found here: orealcplxflint.cs.

The C++ source code which is called from `orealcplxflint.cs` can be found here: UseOReal.cpp.

1.7.8 Binary floating point, multiple precision: mflint, mflintc

The C# modules mflint and mflintc provide a rich set of elementary and special real and complex scalar functions for the Octuple and OctupleC data types, with the actual calculations done using the Arb and ArbC data types, which allows control of the absolute and relative error of the result.

1.7.8.1 Examples in C#, mflint

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some mflint functions");
    MflintTest0();
    MflintTest1();
}
```

General test code for C# can be found here

```
public static void MflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "MflintcTest0"  + "\"" + ">");

    var a = qflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qflintc.abs(a);
    Console.WriteLine("res1 = qflintc.abs(a): {0}", res1);

    var res2 = qflintc.exp(a);
    Console.WriteLine("res2 = qflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the DataXlCalcNet repository or in the corresponding local DataXlCalcNet folder in the file D08a_Mflint.cs.

1.7.8.2 Examples in C#, mflintc

General test code for C# can be found here

```
#region Usings
using System;
using System.Numerics;
using FixedPrecNet;
#endregion


public static void MainTests()
{
    Console.WriteLine("Demo of some mflint functions");
    MflintTest0();
    MflintTest1();
}
```

General test code for C# can be found here

```
public static void MflintcTest0()
{
    Console.WriteLine("<H1 Title=" + "\"" + "MflintcTest0"  + "\"" + ">");

    var a = qflintc.t(3.5, 6.3);
    Console.WriteLine("a: {0}", a);

    var res1 = qflintc.abs(a);
    Console.WriteLine("res1 = qflintc.abs(a): {0}", res1);

    var res2 = qflintc.exp(a);
    Console.WriteLine("res2 = qflintc.exp(a): {0}", res2);

    Console.WriteLine("</H1>");
}
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08b_Mflintc.cs.

1.7.8.3 Examples in Python, `mflint`

General test code for Python can be found here

```
import math
from xlcalcnet import mflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3984573


def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_mflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = mflint.t(i)
    print('x = mflint.t(i):', x)

    x = mflint.t(5.7)
    print('x = mflintT(5.7):', x)

    x = mflint.t(5.7)
    print('x = mflintT(5.7):', x)
    x0 = mflint.t(23294563984539485639456393648273463847539845739845 73984573)
    print('x0 = mflintT(23294563984539485639456393648273463847539845739845 73984573', x0)
    x1 = mflint.t("23294563984539485639456393648273463847539845739845 73984573")
    print('x1 = mflint.t("23294563984539485639456393648273463847539845739845 73984573"):', x1)
    x = mflint.t("5.5")
    print('x = mflintT("5.5"):', x)

    print()
    x = mflint.t(55)
    print('x = mflint.t(5):', x)
    y = mflint.exp(x)
    print('y = mflint.exp(x):', y)

    z = mflint.exp(5.5)
    print('z = mflint.exp(5.5):', z)
    z = mflint.exp(5)
```

(continues on next page)

(continued from previous page)

```
    print('z = mflint.exp(5):', z)
    z = mflint.exp("5.5")
    print('z = mflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08a_MFlint.py.

1.7.8.4 Examples in Python, `mflintc`

General test code for Python can be found here

```
import math
from xlcalcnet import mflint
from decimal import Decimal
from fractions import Fraction
i = 232945639845394856394563936482734638475398457398457 3
```

```
def main_tests():
    general_assignments()
    functions_with_argument_conversion()
    arithmetic_operators_with_mflint()
```

General test code can be found here

```
def general_assignments():
    print()
    print('<H1 Title="General assignments and conversions">')

    x = mflint.t(i)
    print('x = mflint.t(i):', x)

    x = mflint.t(5.7)
    print('x = mflintT(5.7):', x)

    x = mflint.t(5.7)
    print('x = mflintT(5.7):', x)
    x0 = mflint.t(23294563984539485639456393648273)
    print('x0 = mflintT(23294563984539485639456393648273', x0)
    x1 = mflint.t("23294563984539485639456393648273")
    print('x1 = mflint.t("23294563984539485639456393648273"):', x1)
    x = mflint.t("5.5")
    print('x = mflintT("5.5"):', x)

    print()
    x = mflint.t(55)
    print('x = mflint.t(5):', x)
    y = mflint.exp(x)
    print('y = mflint.exp(x):', y)

    z = mflint.exp(5.5)
    print('z = mflint.exp(5.5):', z)
    z = mflint.exp(5)
    print('z = mflint.exp(5):', z)
    z = mflint.exp("5.5")
    print('z = mflint.exp("5.5"):', z)
    print('</H1>')
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08b_MFlintc.py.

1.7.8.5 Notes regarding the implementation in C# and C++

The C# source code for the modules `mflint` and `mflintc` can be found here: mrealcplxflint.cs.

The C++ source code which is called from `mrealcplxflint.cs` can be found here: UseMpfr.cpp.

1.7.9 Building the underlying C# and C/C++ libraries

The XlCalcNet2 library, which is licensed under the LGPL-3.0 and is therefore provided in a separate repository, is based on Boost Math, Boost Multiprecision, Boost Odeint, Eigen, GMP, MPFR, MPC and FLINT and provides functions for the same data types as XlCalcNet and also in arbitrary precision, which are available to the user both from C# and Python.

The SourceOfBasicLibraries2 repository provides copies of the source code of the underlying numerical libraries, which are required (in addition to Eigen and Boost, see SourceOfBasicLibraries) to build the XlCalcNet2 library.. These copies also include small patches and .sh files, as required. They are provided to make it easier to reproduce the compilation results, as distributed as part of the XlCalcNet2 repository.

1.7.9.1 GMP (version 6.3.0)

Text descibing GMP, including config files

1.7.9.2 MPFR (version 4.2.2)

Text descibing MPFR, including config files

1.7.9.3 MPC (version 1.3.1)

Text descibing MPC, including config files

1.7.9.4 FLINT (version 3.4.0)

Text descibing FLINT, including config files

1.7.9.5 ArbPreNet (version 1.0.0)

Text descibing ArbPreNet

1.8 Building user libraries and documenting them

XlCalcNet allows the user to build libraries of user defined functions in Python and C#, which can be used from spreadsheet formulas in Microsoft Excel and LibreOffice Calc, as well as from Python scripts and C# programs. The user libraries can be documented using Sphinx, which allows to build documentation in html and pdf format.

Basic examples of building a user library are provided for Python in the subfolders of the `A03_UserlibPython` folder, and for C# in the subfolders of the `A04_UserlibCSharp` folder, which can be used as templates for building more complex user libraries.

The user library can be documented using Sphinx, which allows to build documentation in html and pdf format. The documentation can be built starting with examples in the `A05_UserlibDocs` folder, which can be used as a template for documenting more complex user libraries. The documentation for the already included starting user libraray is part of this manual. It is written in the same style as this manual and comprises two parts: User library: numerical; and User library: graphics.

Examples of using the user libraries are provided in the subfolders of the `A06_UserlibExamplesPython` folder for Python and in the `A07_UserlibExamplesCSharp` folder for C#.

1.8.1 Building a library of user defined Python functions

In Python, the user library is built in the subfolders of the `A03_UserlibPython` folder, which contains Python modules with the user defined functions. The modules can be imported in Python scripts and used from spreadsheet formulas in Microsoft Excel and LibreOffice Calc. The functions are decribed here.

These functions can be tested in the subfolders of the `A06_UserlibExamplesPython` folder.

1.8.2 Building a library of user defined functions in C#

The library of user defined functions in C# is built in form of 2 dynamic link libraries (DLLs), which are called `UserFixedPrecNet.dll` and `UserArbPrecNet.dll` and which are written into the Binary Output Folder. These functions are described here.

There is also a third DLL called `UserMpPrecNet.dll`, which is used by the Python user library. This DLL is also written into the Binary Output Folder.

1.8.3 Sphinx: building documentation for software with Python (html and pdf)

This manual is written in reStructuredText (reST) format, which is a lightweight markup language. The manual is built using Sphinx, which is a tool that makes it easy to create intelligent and beautiful documentation for Python projects (or other software projects). Sphinx uses reStructuredText as its markup language and can generate output in various formats, including HTML and PDF.

To build the documentation, you need to have Sphinx installed.

See also: https://www.sphinx-doc.org/en/master/

You also need to install 3 additional Sphinx extensions that are used in this manual: the Sphinx Book Theme, the Sphinx Copybutton extension, and the Sphinx Contrib Bibtex extension

Assuming that the Python batch files have been installed as described *here*, these four packages can be installed by double-clicking on the batch files `A14installSphinx.bat`, `A15install_sphinx_book_theme.bat`, `A16install_sphinx_copybutton.bat`, `A17install_sphinxcontribute_bibtex.bat`, in the `C:\Python313` folder.

With these packages installed, you can build the documentation of the user library by opening in the Tiny IDE the file `A05_UserlibDocs\B01_Lib01\_static\builddoc.py`. In the Tiny IDE main menu, click on `Run`. The HTML documentation will then be built in the subfolder `html` in the `XlCalcNetIDE` folder in the AppDataLocal folder. If built has succeded, the documentation will be opened automatically. The html documentation can be viewed by opening the file `index.html` in a web browser.

CHAPTER

TWO

BASIC FLOATING POINT FUNCTIONS

The source code for the C# tests in Visual Studio can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B02_BasicFloatingPoint.cs.

2.1 Operator overloading, general real functions

2.1.1 Operator overloading: scope and restrictions

Compatible operators

2.1.2 The cost of creating and destroying objects

Some text

2.1.3 Raw addition of two floating point numbers

`ctx.`**`rawadd`**(*res*, *x*, *y*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Sets `res` to the sum of the floating point number `x` and `y`.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Abs(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Abs('0.1')
ecplx('5.351847902755998475 4E-1')
```

2.1.4 Raw subtraction of two floating point numbers

`ctx.`**`rawsub`**(*res*, *x*, *y*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Sets `res` to the difference of the floating point numbers `x` and `y`.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Abs(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Abs('0.1')
ecplx('5.351847902755998475 4E-1')
```

2.1.5 Raw multiplication of two floating point numbers

`ctx.rawmul`(*res*, *x*, *y*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Sets `res` to the product of the floating point numbers `x` and `y`.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Abs(0.5)
ecplx('5.235987755982988 7307E-1')
>>> ecplx.Abs('0.1')
ecplx('5.351847902755998 4754E-1')
```

2.1.6 Raw division of two floating point numbers

`ctx.rawdiv`(*res*, *x*, *y*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Sets `res` to the quotient of the floating point numbers `x` and `y`.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Abs(0.5)
ecplx('5.235987755982988 7307E-1')
>>> ecplx.Abs('0.1')
ecplx('5.351847902755998 4754E-1')
```

2.1.7 Raw addition of a floating point number and a signed 32 bit integer

`ctx.rawaddint32`(*res*, *x*, *i*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Sets `res` to the sum of the floating point number `x` and the signed 32 bit integer `i`.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Abs(0.5)
ecplx('5.235987755982988 7307E-1')
>>> ecplx.Abs('0.1')
ecplx('5.351847902755998 4754E-1')
```

2.1.8 Raw subtraction of a floating point number and a signed 32 bit integer

`ctx.rawsubint32`(*res*, *x*, *i*)

`ctx.rawint32sub`(*res*, *x*, *i*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Sets `res` to the difference of the floating point number `x` and the signed 32 bit integer `i` (`SubInt`) or to the difference of the signed 32 bit integer `i` the floating point number `x` (`IntSub`).

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Abs(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Abs('0.1')
ecplx('5.351847902755998475 4E-1')
```

2.1.9 Raw multiplication of a floating point number and a signed 32 bit integer

`ctx.`**`rawmulint32`**(*res*, *x*, *i*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Sets `res` to the product of the floating point number `x` and the signed 32 bit integer `i`.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Abs(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Abs('0.1')
ecplx('5.351847902755998475 4E-1')
```

2.1.10 Raw division of a floating point number and a signed 32 bit integer

`ctx.`**`rawdivint32`**(*res*, *x*, *i*)

`ctx.`**`rawint32div`**(*res*, *i*, *x*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Sets `res` to the quotient of the floating point number `x` and the signed 32 bit integer `i` (`DivInt`) or to the quotient of the signed 32 bit integer `i` the floating point number `x` (`IntDiv`).

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Abs(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Abs('0.1')
ecplx('5.351847902755998475 4E-1')
```

2.1.11 Fused multiplication and addition: $\mathrm{fma}(x, y, z)$

`ctx.`**`fma`**(*x*, *y*, *z*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

If successful, returns the value of x * y + z as if calculated to infinite precision and rounded once to fit the result type (or, alternatively, calculated as a single ternary floating-point operation).

See also: https://en.cppreference.com/w/cpp/numeric/math/fma

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Copysign(0.5, 2)
ereal('5.235987755982988730 7E-1')
>>> math53.Copysign('0.51', 2)
ereal('5.351847902755998475 4E-1')
```

2.1.12 Maximum of two floating point numbers: $\mathrm{fmax}(x, y)$

`ctx.`**`fmax`**(x, e)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the maximum of two floating point numbers; $x, y \neq$ NAN.

Returns the larger of two floating point arguments, treating NaNs as missing data (between a NaN and a numeric value, the numeric value is chosen). Only if both arguments are NaN, NaN is returned.

This function is not required to be sensitive to the sign of zero, although some implementations additionally enforce that if one argument is +0 and the other is -0, then +0 is returned.

See also: https://en.cppreference.com/w/cpp/numeric/math/fmax

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Max(0.5, 2)
ereal('5.235987755982988730 7E-1')
>>> math53.Max('0.51', 2)
ereal('5.351847902755998475 4E-1')
```

2.1.13 Minimum of two floating point numbers: $\mathrm{fmin}(x, y)$

`ctx.`**`fmin`**(x, y)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the minimum of two floating point numbers; $x, y \neq$ NAN.

Returns the smaller of two floating point arguments, treating NaNs as missing data (between a NaN and a numeric value, the numeric value is chosen). Only if both arguments are NaN, NaN is returned.

This function is not required to be sensitive to the sign of zero, although some implementations additionally enforce that if one argument is +0 and the other is -0, then -0 is returned.

See also: https://en.cppreference.com/w/cpp/numeric/math/fmin

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Min(0.5, 2)
ereal('5.235987755982988730 7E-1')
>>> math53.Min('0.51', 2)
ereal('5.351847902755998475 4E-1')
```

2.2 Machine constants, general

2.2.1 Zero, positive zero

property ctx.**zero**

where ctx is math53, ctxboost or ctxflint.

Returns $+0$.

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.Zero()
ereal('0.000000000000000000')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.Zero()
0.0
```

2.2.2 Negative zero

property ctx.**negzero**

where ctx is math53, ctxboost or ctxflint.

Returns -0.

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.Zero()
ereal('0.000000000000000000')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.Zero()
0.0
```

2.2.3 One

property Cxt.**one**

where ctx is math53, ctxboost or ctxflint.

Returns 1

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.One()
ereal('1.000000000000000000')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.One()
1.0
```

2.2.4 Imaginary One

property ctx.**onej**

where ctx is math53, ctxboost or ctxflint.

Returns the imaginary unit.

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstDegree()
ereal('1.74532925199433295769E-2')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstDegree()
0.0174532925199433
```

2.2.5 Positive infinity

ctx.**inf**()

where ctx is math53, ctxboost or ctxflint.

Returns $+\infty$

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.PositiveInfinity()
ereal('+Inf')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.PositiveInfinity()
+Inf
```

2.2.6 Negative infinity

ctx.**neginf**()

where ctx is math53, ctxboost or ctxflint.

Returns the representation of $-\infty$

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.NegativeInfinity()
ereal('-Inf')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.NegativeInfinity()
-Inf
```

2.2.7 NaN (Not a Number)

`ctx.nan()`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `NaN` (Not a Number).

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.Nan()
ereal('Nan')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.Nan()
Nan
```

2.3 Properties of numbers

2.3.1 Test for signbit

`ctx.signbit`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if the sign bit of x is set.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsInfinity(0.5)
False
>>> math53.IsInfinity('inf')
True
```

2.3.2 Test for a finite number

`ctx.isfinite`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is neither $+\infty$ nor $-\infty$ nor `NaN`.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsFinite(0.5)
False
>>> math53.IsFinite('inf')
True
```

2.3.3 Test for infinity

`ctx.isinf`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is either $+\infty$ or $-\infty$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsInfinity(0.5)
False
>>> math53.IsInfinity('inf')
True
```

2.3.4 Test for positive infinity

`ctx.isposinf`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is $+\infty$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsPositiveInfinity(0.5)
False
```

(continues on next page)

(continued from previous page)

```
>>> math53.IsPositiveInfinity('inf')
True
```

2.3.5 Test for negative infinity

`ctx.`**`isneginf`**(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is $-\infty$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNegativeInfinity(0.5)
False
>>> math53.IsNegativeInfinity('inf')
True
```

2.3.6 Test for NaN

`ctx.`**`isnan`**(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is Nan (Not a number).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNan(0.5)
False
>>> math53.IsNan('inf')
True
```

2.3.7 Test for zero

`ctx.`**`iszero`**(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is 0 (+0 or -0).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNan(0.5)
False
>>> math53.IsNan('inf')
True
```

2.3.8 Test for one

`ctx.`**`isone`**(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is 1.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNan(0.5)
False
>>> math53.IsNan('inf')
True
```

2.3.9 Test if a number is an integer

`ctx.isinteger`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is an integer.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNan(0.5)
False
>>> math53.IsNan('inf')
True
```

2.3.10 Test if a number is a simple number (not an infinity or NaN)

`ctx.isnumber`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is an integer.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNan(0.5)
False
>>> math53.IsNan('inf')
True
```

2.3.11 Test if a number is a regular number (not zero, an infinity or NaN)

`ctx.isregular`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is a regular number (not zero, an infinity or NaN).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNan(0.5)
False
>>> math53.IsNan('inf')
True
```

2.3.12 Test if a number is a normal number (not subnormal, zero, an infinity or NaN)

`ctx.isnormal`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x is a normal number (not subnormal, zero, an infinity or NaN)

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNan(0.5)
False
>>> math53.IsNan('inf')
True
```

2.3.13 Test if 2 numbers are unordered

`ctx.`**`isunordered`**(x, y)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns `True` if x and y are unordered.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNan(0.5)
False
>>> math53.IsNan('inf')
True
```

2.3.14 Test if a number fits in a signed 32 bit integer

`ctx.`**`fitsint32`**(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Test if a number fits in a signed 32 bit integer

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNan(0.5)
False
>>> math53.IsNan('inf')
True
```

2.3.15 Test if a number fits in a signed 64 bit integer

`ctx.`**`fitsint64`**(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Test if a number fits in a signed 64 bit integer

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IsNan(0.5)
False
>>> math53.IsNan('inf')
True
```

2.4 Integer related functions

2.4.1 Integral value nearest x (rounding to nearest): $\mathrm{nearbyint}(x), \mathrm{rint}(x)$

`ctx.rint(`x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the integral value (in floating-point format) nearest x, using the `rounding-to-nearest` rounding mode.

For the std::rint function:

- If num is $\pm\infty$, it is returned, unmodified.
- If num is ± 0, it is returned, unmodified.
- If num is NaN, NaN is returned.

See also: https://en.cppreference.com/w/cpp/numeric/math/nearbyint

See also: https://en.cppreference.com/w/cpp/numeric/math/rint

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Rint(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Rint('0.51')
ereal('5.351847902755998475 4E-1')
```

`ctx.nearbyint(`x`)`

Is an alias for `ctx.rint(x)` in this implementation. The only difference between `std::rint` and `std::nearbyint` is that `std::nearbyint` never raises `FE_INEXACT`.

2.4.2 32 bit integer nearest x (rounding to nearest): $\mathrm{lrint}(x)$

`ctx.lrint(`x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns an 32 bit signed integer nearest x, using the `rounding-to-nearest` rounding mode.

If the result is outside the range representable by an 32 bit signed integer, a domain error or a range error may occur.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Rint(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Rint('0.51')
ereal('5.351847902755998475 4E-1')
```

2.4.3 64 bit integer nearest x (rounding to nearest): $\mathrm{llrint}(x)$

`ctx.llrint(`x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns an 64 bit signed integer nearest x, using the `rounding-to-nearest` rounding mode.

If the result is outside the range representable by an 64 bit signed integer, a domain error or a range error may occur.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Rint(0.5)
ereal('5.23598775598298873077E-1')
>>> math53.Rint('0.51')
ereal('5.35184790275599847540E-1')
```

2.4.4 Smallest integral value not less than x : ceil(x)

`ctx.ceil(`x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the smallest integer $> x$; $|x| \leq$ MaxLongint.

See also: https://en.cppreference.com/w/cpp/numeric/math/ceil

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Ceil(0.5)
ereal('5.23598775598298873077E-1')
>>> math53.Ceil('0.51')
ereal('5.35184790275599847540E-1')
```

2.4.5 Largest integral value not greater than x : floor(x)

`ctx.floor(`x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the largest integer $< x$; $|x| \leq$ MaxLongint.

See also: https://en.cppreference.com/w/cpp/numeric/math/floor

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Floor(0.5)
ereal('5.23598775598298873077E-1')
>>> math53.Floor('0.51')
ereal('5.35184790275599847540E-1')
```

2.4.6 Nearest integral value not greater in magnitude than x : trunc(x)

`ctx.trunc(`x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the truncated value of x.

See also: https://en.cppreference.com/w/cpp/numeric/math/trunc

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Truncate(0.5)
ereal('5.23598775598298873077E-1')
>>> math53.Truncate('0.51')
ereal('5.35184790275599847540E-1')
```

2.4.7 Nearest integral value, rounding halfway cases away from zero: $\mathrm{round}(x)$

`ctx.round`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Computes the nearest integer (in floating-point format), rounding halfway cases away from zero, regardless of the current rounding mode.

See also: https://en.cppreference.com/w/cpp/numeric/math/round

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Round(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Round('0.51')
ereal('5.351847902755998475 4E-1')
```

2.4.8 Nearest 32 bit integer, rounding halfway cases away from zero: $\mathrm{lround}(x)$

`ctx.lround`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the nearest 32 bit signed integer, rounding halfway cases away from zero.

See also: https://en.cppreference.com/w/cpp/numeric/math/round

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Round(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Round('0.51')
ereal('5.351847902755998475 4E-1')
```

2.4.9 Nearest 64 bit integer, rounding halfway cases away from zero: $\mathrm{llround}(x)$

`ctx.llround`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the nearest 64 bit signed integer, rounding halfway cases away from zero.

See also: https://en.cppreference.com/w/cpp/numeric/math/round

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Round(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Round('0.51')
ereal('5.351847902755998475 4E-1')
```

2.5 Floating point functions for real numbers

2.5.1 Efficient computation of $|x| \cdot \text{sign}(y) : \text{copysign}(x, y)$

`ctx.copysign`(*mag*, *sgn*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $|x| \cdot \text{sign}(y)$.

See also: https://en.cppreference.com/w/cpp/numeric/math/copysign

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Copysign(0.5, 2)
ereal('5.2359877559829887307E-1')
>>> math53.Copysign('0.51', 2)
ereal('5.351847902755998475 4E-1')
```

2.5.2 Decomposition of x as $x = m \cdot 2^e; 0.5 < m < 1 : \text{frexp}(x)$

`ctx.frexp`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`. See also Mpmath [684].

Returns (as a tuple) the mantissa m and exponent e of x with $x = m \cdot 2^e, 0.5 < m < 1$.

If x is 0, $\pm\infty$ or NaN, returns $m = x, e = 0$.

See also: https://en.cppreference.com/w/cpp/numeric/math/frexp

See also: https://en.cppreference.com/w/cpp/numeric/math/ilogb

See also: https://en.cppreference.com/w/cpp/numeric/math/logb

Ilogb: Returns base 2 exponent of x. For finite x ilogb = floor(log2($|x|$)), otherwise -MaxLongint for x = 0 or MaxLongint if x = +-INF or Nan.

Extracts the value of the unbiased exponent from the floating-point argument num, and returns it as a signed integer value. The value of the exponent returned by `std::ilogb` is always 1 less than the exponent returned by `std::frexp` because of the different normalization requirements: for the exponent e returned by `std::ilogb`, $|num * 2^{-e}|$ is between 1 and 2, but for the exponent e returned by std::frexp, $|num * 2^{-e}|$ is between 0.5 and 1.

If the implementation supports IEEE floating-point arithmetic (IEC 60559),

- If num is ± 0, $-\inf$ is returned and FE_DIVBYZERO is raised.
- If num is $\pm\inf$, $+\inf$ is returned.
- If num is NaN, NaN is returned.

In all other cases, the result is exact (FE_INEXACT is never raised) and the current rounding mode is ignored.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Frexp(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Frexp('0.51')
ereal('5.351847902755998475 4E-1')
```

2.5.3 Integral value e, from the decomposition of x as $x = m \cdot 2^e; 1 < m < 2 : \log b(x)$

`ctx.logb`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns (as a tuple) the mantissa m and exponent e of x with $x = m \cdot 2^e, 0.5 < m < 1$.

If x is 0, $\pm\infty$ or NaN, returns $m = x, e = 0$.

See also: https://en.cppreference.com/w/cpp/numeric/math/frexp

See also: https://en.cppreference.com/w/cpp/numeric/math/ilogb

See also: https://en.cppreference.com/w/cpp/numeric/math/logb

Logb: Returns base 2 exponent of x. For finite x logb = floor(log2($|x|$)), otherwise -MaxLongint for x = 0 or MaxLongint if x = +-INF or Nan.

Extracts the value of the unbiased exponent from the floating-point argument num, and returns it as a signed integer value. The value of the exponent returned by `std::ilogb` is always 1 less than the exponent returned by `std::frexp` because of the different normalization requirements: for the exponent e returned by `std::ilogb`, $|num * 2^{-e}|$ is between 1 and 2, but for the exponent e returned by std::frexp, $|num * 2^{-e}|$ is between 0.5 and 1.

If the implementation supports IEEE floating-point arithmetic (IEC 60559),

- If num is ± 0, $-\inf$ is returned and FE_DIVBYZERO is raised.
- If num is $\pm\inf$, $+\inf$ is returned.
- If num is NaN, NaN is returned.

In all other cases, the result is exact (FE_INEXACT is never raised) and the current rounding mode is ignored.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Frexp(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Frexp('0.51')
ereal('5.3518479027559984754E-1')
```

2.5.4 32 bit integer e, from the decomposition of x as $x = m \cdot 2^e; 1 < m < 2 : \text{ilogb}(x)$

`ctx.ilogb`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns (as a tuple) the mantissa m and exponent e of x with $x = m \cdot 2^e, 0.5 < m < 1$.

If x is 0, $\pm\infty$ or NaN, returns $m = x, e = 0$.

See also: https://en.cppreference.com/w/cpp/numeric/math/frexp

See also: https://en.cppreference.com/w/cpp/numeric/math/ilogb

See also: https://en.cppreference.com/w/cpp/numeric/math/logb

Ilogb: Returns base 2 exponent of x. For finite x ilogb = floor(log2($|x|$)), otherwise -MaxLongint for x = 0 or MaxLongint if x = +-INF or Nan.

Extracts the value of the unbiased exponent from the floating-point argument num, and returns it as a signed integer value. The value of the exponent returned by `std::ilogb` is always 1 less than the exponent returned by `std::frexp` because of the different normalization requirements: for the exponent e returned by `std::ilogb`, $|num * 2^{-e}|$ is between 1 and 2, but for the exponent e returned by std::frexp, $|num * 2^{-e}|$ is between 0.5 and 1.

If the implementation supports IEEE floating-point arithmetic (IEC 60559),

- If num is ± 0, $-\inf$ is returned and FE_DIVBYZERO is raised.
- If num is $\pm\inf$, $+\inf$ is returned.
- If num is NaN, NaN is returned.

In all other cases, the result is exact (FE_INEXACT is never raised) and the current rounding mode is ignored.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Frexp(0.5)
ereal('5.23598775598298873077E-1')
>>> math53.Frexp('0.51')
ereal('5.35184790275599847540E-1')
```

2.5.5 Efficient computation of $x \cdot 2^e$: $\mathrm{ldexp}(x, e), \mathrm{scalbn}(x, e), \mathrm{scalbln}(x, e)$

`ctx.`**`ldexp`**(x, e)

where `ctx` is `math53`, `ctxboost` or `ctxflint`. See also Mpmath [688].

Returns $x \cdot 2^e$

On binary systems (where `FLT_RADIX` is 2), `std::scalbn` is equivalent to std::ldexp.

Although `std::scalbn` and `std::scalbln` are specified to perform the operation efficiently, on many implementations they are less efficient than multiplication or division by a power of two using arithmetic operators.

The function name stands for "new scalb", where scalb was an older non-standard function whose second argument had floating-point type.

The `std::scalbln` function is provided because the factor required to scale from the smallest positive floating-point value to the largest finite one may be greater than 32767, the standard-guaranteed INT_MAX. In particular, for the 80-bit long double, the factor is 32828.

See also: https://en.cppreference.com/w/cpp/numeric/math/ldexp

See also: https://en.cppreference.com/w/cpp/numeric/math/scalbn

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Ldexp(0.5, 2)
ereal('5.23598775598298873077E-1')
>>> math53.Ldexp('0.51', 2)
ereal('5.35184790275599847540E-1')
```

`ctx.`**`scalbn`**(x, e)

where `ctx` is `math53`, `ctxboost` or `ctxflint`. This is an alias of `ctx.ldexp`.

`ctx.`**`scalbln`**(x, e)

where `ctx` is `math53`, `ctxboost` or `ctxflint`. This is an alias of `ctx.ldexp`.

2.5.6 Positive difference between x and y: $\mathrm{fdim}(x, y)$

`ctx.`**`fdim`**(x, y)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the positive difference between x and y, that is, if $x > y$, returns $x - y$, otherwise (i.e. if $x \leq y$) returns +0.

See also: https://en.cppreference.com/w/cpp/numeric/math/fdim

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Copysign(0.5, 2)
ereal('5.2359877559829887307E-1')
>>> math53.Copysign('0.51', 2)
ereal('5.351847902755998475 4E-1')
```

2.6 Fraction and remainder related functions

2.6.1 Integral and fractional part of a floating point number: $\mathrm{modf}(x)$

`ctx.modf`(x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns (as a tuple) frac(x) and trunc(x) in ip, $|x| <$ MaxLongint

Decomposes given floating point value num into integral and fractional parts, each having the same type and sign as num. The integral part (in floating-point format) is stored in the object pointed to by iptr.

If the implementation supports IEEE floating-point arithmetic (IEC 60559),

- If num is ± 0, ± 0 is returned, and ± 0 is stored in `*iptr`.
- If num is $\pm \inf$, ± 0 is returned, and $\pm \inf$ is stored in `*iptr`.
- If num is NaN, NaN is returned, and NaN is stored in `*iptr`.

The returned value is exact, the current rounding mode is ignored.

See also: https://en.cppreference.com/w/cpp/numeric/math/modf

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Modf(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Modf('0.51')
ereal('5.351847902755998475 4E-1')
```

2.6.2 Floating point remainder: $\mathrm{fmod}(x, y)$

`ctx.fmod`(x, y)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`. See also Mpmath [674].

!!! TypeError: unsupported operand type(s) for %: 'ivmpf' and 'ivmpf' !!!

!!! TypeError: unsupported operand type(s) for %: 'arb3_t' and 'arb3_t' !!!

Converts x and y to mpmath numbers and returns $x \mod y$. For mpmath numbers, this is equivalent to `x % y`.

```
>>> from xlcalcnet import fp, mp, iv, dp, gp, ap
>>> for ctx in [fp, mp, dp, gp]: ctx.dps = 15;  print(repr(ctx.fmod(100, +ctx.pi)))
2.6106277387164134
mpf('2.6106277387164134')
Decimal('2.61062773871651')
mpfr('2.6106277387164134')
```

You can use *`fmod()`* to compute fractional parts of numbers:

```
>>> for ctx in [fp, mp, dp, gp]: ctx.dps = 15;  print(repr(ctx.fmod(10.25, 1)), end=', ')
0.25, mpf('0.25'), Decimal('0.25'), mpfr('0.25'),
```

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $x \mod y$, $y \neq 0$, sign(result) = sign(x).

The floating-point remainder of the division operation `x / y` calculated by this function is exactly the value `x - rem * y`, where `rem` is `x / y` with its fractional part truncated.

The returned value has the same sign as x and is less than y in magnitude.

If the implementation supports IEEE floating-point arithmetic (IEC 60559),

- If x is ± 0 and y is not zero, ± 0 is returned.
- If x is $\pm \inf$ and y is not NaN, NaN is returned and FE_INVALID is raised.
- If y is ± 0 and x is not NaN, NaN is returned and FE_INVALID is raised.
- If y is $\pm \inf$ and x is finite, x is returned.
- If either argument is NaN, NaN is returned.

See also: https://en.cppreference.com/w/cpp/numeric/math/fmod

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Fmod(0.5, 2)
ereal('5.235987755982988730 7E-1')
>>> math53.Fmod('0.51', 2)
ereal('5.351847902755998475 4E-1')
```

2.6.3 IEEE floating point remainder: $\mathrm{remainder}(x, y)$

`ctx.`**`remainder`**`(`x, y`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the IEEE754 remainder x REM y = x - rmNearest(x/y)*y.

The IEEE floating-point remainder of the division operation `x / y` calculated by this function is exactly the value `x - quo * y`, where the value `quo` is the integral value nearest the exact value `x / y`. When `|quo - x / y| = 0.5`, the value `quo` is chosen to be even.

In contrast to std::fmod, the returned value is not guaranteed to have the same sign as x.

If the returned value is zero, it will have the same sign as x.

If the implementation supports IEEE floating-point arithmetic (IEC 60559),

- The current rounding mode has no effect.
- FE_INEXACT is never raised, the result is always exact.
- If x is $\pm \inf$ and y is not NaN, NaN is returned and FE_INVALID is raised.
- If y is ± 0 and x is not NaN, NaN is returned and FE_INVALID is raised.
- If either argument is NaN, NaN is returned.

See also: https://en.cppreference.com/w/cpp/numeric/math/remainder

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Remainder(0.5, 2)
ereal('5.235987755982988730 7E-1')
>>> math53.Remainder('0.51', 2)
ereal('5.351847902755998475 4E-1')
```

2.6.4 IEEE floating point remainder and part of quotient: $\mathrm{remquo}(x, y)$

`ctx.remquo(`x, y`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the IEEE754 remainder x REM y = x - rmNearest(x/y)*y.

Computes the floating-point remainder of the division operation x/y as the `std::remainder` function does. Additionally, the sign and at least three of the last bits of x/y will be stored in quo, sufficient to determine the octant of the result within a period.(formally, stores a value whose sign is the sign of x/y and whose magnitude is congruent modulo 2^n to the magnitude of the integral quotient of x/y, where n is an implementation-defined integer greater than or equal to 3).

If the implementation supports IEEE floating-point arithmetic (IEC 60559),

The current rounding mode has no effect.

FE_INEXACT is never raised.

If x is $\pm\inf$ and y is not NaN, NaN is returned and FE_INVALID is raised.

If y is ± 0 and x is not NaN, NaN is returned and FE_INVALID is raised.

If either x or y is NaN, NaN is returned.

See also: https://en.cppreference.com/w/cpp/numeric/math/remquo

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Remainder(0.5, 2)
ereal('5.235987755982988730 7E-1')
>>> math53.Remainder('0.51', 2)
ereal('5.351847902755998475 4E-1')
```

2.7 Functions related to mantissa width and exponent range

2.7.1 Machine Epsilon

`ctx.epsilon()`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the machine epsilon for the given data type. See also Wikipedia [1263].

See also: https://docs.python.org/3/library/sys.html#sys.float_info

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.MachineEps()
ereal('1.0842021724855044340E-19')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.MachineEps()
2.22044604925031e-16
```

2.7.2 Unit in the last place

`ctx.ulp(`x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the "unit in the last place" for a specified floating-point value x, i.e.the smallest representable positive number u with $x + u > x$.

See also: https://docs.python.org/3/library/math.html#math.ulp

See also: https://en.wikipedia.org/wiki/Unit_in_the_last_place

See also: https://www.boost.org/doc/libs/1_85_0/libs/math/doc/html/math_toolkit/next_float/ulp.html

```
>>> from xlcalcnet import math53
>>> math53.MachineEps()
ereal('1.0842021724855044340E-19')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.MachineEps()
2.22044604925031e-16
```

2.7.3 Largest representable number

`ctx.maxvalue()`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the largest representable number for the given data type.

See also: https://docs.python.org/3/library/sys.html#sys.float_info

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.MaxValue()
ereal('1.1897314953572317650E+4932')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.MaxValue()
1.79769313486232E+308
```

2.7.4 Lowest representable number

`ctx.lowestvalue()`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the lowest representable number for the given data type.

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.MaxValue()
ereal('-1.189731495357231765085E+4932')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.MaxValue()
-1.79769313486232E+308
```

2.7.5 Smallest representable positive number

`ctx.minposvalue()`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

See also: https://docs.python.org/3/library/sys.html#sys.float_info

Returns the smallest representable positive number for the given data type.

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.MinValue()
ereal('3.362103143112093506 3E-4932')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.MinValue()
2.22507385850720e-308
```

2.7.6 Next representable floating point number: $\mathrm{nexttowards}(x, y)$

`ctx.nexttowards(`x, y`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

See also: https://docs.python.org/3/library/math.html#math.nextafter

Returns the next representable floating point number before x in the direction of y.

See also: https://en.cppreference.com/w/cpp/numeric/math/nextafter

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Succ(0.5)
ereal('5.23598775598298873077E-1')
>>> math53.Succ('0.51')
ereal('5.35184790275599847547E-1')
```

2.7.7 Next representable floating point number: nextabove(x)

`ctx.`**`nextabove`**`(`x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Note: ctx.pred(x), ctx.succ(x).

Nextabove: Returns the next representable floating point number before x in the direction of `+Inf`.

See also: https://en.cppreference.com/w/cpp/numeric/math/nextafter

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Succ(0.5)
ereal('5.23598775598298873077E-1')
>>> math53.Succ('0.51')
ereal('5.35184790275599847547E-1')
```

2.7.8 Next representable floating point number: nextbelow(x)

`ctx.`**`nextbelow`**`(`x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Note: ctx.pred(x), ctx.succ(x).

Nextbelow: Returns the next representable floating point number before x in the direction of `-Inf`.

See also: https://en.cppreference.com/w/cpp/numeric/math/nextafter

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Succ(0.5)
ereal('5.23598775598298873077E-1')
>>> math53.Succ('0.51')
ereal('5.35184790275599847547E-1')
```

2.8 Mathematical Constants

2.8.1 Degree

property ctx.**degree**

where ctx is math53, ctxboost or ctxflint.

Returns one degree of angle, $1° = \pi/180$. See also Wikipedia [1274], MathWorld [902], Mpmath [605], Mpmath [711], Mpmath [704].

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstDegree()
ereal('1.7453292519943295769E-2')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstDegree()
0.0174532925199433
```

2.8.2 Golden ratio phi

property ctx.**phi**

where ctx is math53, ctxboost or ctxflint.

Returns the golden ratio $\phi = (1 + \sqrt{5})/2$. See also Wikipedia [1281], MathWorld [905], Mpmath [612].

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstPhi()
ereal('1.6180339887498948482')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstPhi()
1.61803398874989
```

2.8.3 Natural logarithm of 2

property ctx.**ln2**

where ctx is math53, ctxboost or ctxflint.

Returns the logarithm of 2. See also Wikipedia [1265], MathWorld [910].

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstLog2()
ereal('6.9314718055994530943E-1')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstLog2()
0.693147180559945
```

2.8.4 Natural logarithm of 10

property ctx.**ln10**

where ctx is math53, ctxboost or ctxflint.

Returns the logarithm of 10. See also MathWorld [909].

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstLog10()
ereal('2.3025850929940456840')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstLog10()
2.30258509299405
```

2.8.5 Pi (π)

property ctx.**pi**

where ctx is math53, ctxboost or ctxflint.

Returns the constant pi. See also Wikipedia [1266], MathWorld [911], Mpmath [617].

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.Pi()
ereal('3.1415926535897932385')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.Pi()
3.14159265358979
```

2.8.6 Euler e

property ctx.**e**

where ctx is math53, ctxboost or ctxflint.

Returns the constant const_e. See also Wikipedia [1270], MathWorld [912], Mpmath [602].

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstE()
ereal('3.1415926535897932385')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstE()
2.71828182845905
```

2.8.7 Euler-Mascheroni constant γ

property `ctx.egamma`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Euler gamma constant. See also Wikipedia [1279], MathWorld [903], Mpmath [610].

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstEulerGamma()
ereal('5.772156649015328606 2E-1')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstEulerGamma()
0.577215664901533
```

2.8.8 Apéry's constant

property `ctx.apery`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Represents Apery's constant. See also Wikipedia [1271], MathWorld [897], Mpmath [600].

It is an irrational number approximately equal to 1.2020569 given by

$$\zeta(3) = \sum_{k=1}^{\infty} \frac{1}{k^3}.$$

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstApery()
ereal('1.2020569031595942854')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstApery()
1.20205690315959
```

2.8.9 Catalan's constant

property `ctx.catalan`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Catalan constant. See also Wikipedia [1282], MathWorld [898], Mpmath [604].

Catalan's constant K = 0.91596559… is given by the infinite series

$$K = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)^2}.$$

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstCatalan()
ereal('9.159655941772190150 5E-1')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstCatalan()
0.915965594177219
```

2.8.10 Glaisher's constant

property ctx.**glaisher**

where ctx is math53, ctxboost or ctxflint.

Returns Glaisher's constant. See also Wikipedia [1280], MathWorld [904], Mpmath [611].

The constant is defined as $A = \exp(1/12 - \zeta'(-1))$ where $\zeta'(s)$ denotes the derivative of the Riemann zeta function.

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstGlaisher()
ereal('1.2824271291006226369')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstGlaisher()
1.28242712910062
```

2.8.11 Khinchin's constant

property ctx.**khinchin**

where ctx is math53, ctxboost or ctxflint.

Returns Khinchin's constant. See also Wikipedia [1287], MathWorld [906], Mpmath [616].

In extended precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstKhinchin()
ereal('2.6854520010653064454')
```

In double precision (64 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.ConstKhinchin()
2.68545200106531
```

CHAPTER

THREE

ELEMENTARY SCALAR FUNCTIONS (REAL AND COMPLEX)

The functions in this chapter comprise the usual elementary functions, as included in the C++ standard library, and a few specializations.

The source code for the C# tests in Visual Studio is divided by real and complex modules:

The source code for the C# tests in Visual Studio using real modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B03a_RealElementaryFunctions.cs,

The source code for the C# tests in Visual Studio using complex modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B03b_CplxElementaryFunctions.cs,

3.1 Complex components

3.1.1 Absolute value of a real or complex number

`ctx.abs`(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the absolute value of x, $|x|$.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Abs(0.5)
ecplx('5.2359877559829887307E-1')
>>> ecplx.Abs('0.1')
ecplx('5.351847902755998475 4E-1')
```

`ctx.fabs`(x)

where `ctx` is `math53`, `mathc53` or `ctxcpp`. This is an alias of `ctx.abs(x)`.

3.1.2 Sign of a real or complex number

`ctx.sign`(x)

where `ctx` is `math53`, `mathc53` or `ctxcpp`.

Returns the sign of x, defined as $\text{sign}(x) = x/|x|$ (with the special case $\text{sign}(0) = 0$):

See also: https://en.wikipedia.org/wiki/Sign_function

Note that the sign function is also defined for complex numbers, for which it gives the projection onto the unit circle:

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Sign(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Sign('0.1')
ecplx('5.351847902755998475 4E-1')
```

3.1.3 Real part of a real or complex number

`ctx.real`(x)

where `ctx` is `math53`, `mathc53` or `ctxcpp`.

Returns the real part of x, $\Re(x)$.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Real(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Real('0.1')
ecplx('5.351847902755998475 4E-1')
```

3.1.4 Imaginary part of a real or complex number

`ctx.imag`(x)

where `ctx` is `math53`, `mathc53` or `ctxcpp`.

Returns the imaginary part of x, $\Im(x)$.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Imaginary(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Imaginary('0.1')
ecplx('5.351847902755998475 4E-1')
```

3.1.5 Phase (or argument) of a real or complex number

`ctx.phase`(x)

where `ctx` is `math53`, `mathc53` or `ctxcpp`.

Computes the complex argument (phase) of x, defined as the signed angle between the positive real axis and x in the complex plane: The angle is defined to satisfy $-\pi < \arg(x) \leq \pi$ and with the sign convention that a nonnegative imaginary part results in a nonnegative argument.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Phase(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Phase('0.1')
ecplx('5.351847902755998475 4E-1')
```

3.1.6 Conjugate of a real or complex number

`ctx.conj`(x)

where `ctx` is `math53`, `mathc53` or `ctxcpp`.

Returns the complex conjugate of x, $\overline{x}$.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Conj(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Conj('0.1')
ecplx('5.351847902755998475 4E-1')
```

3.1.7 Polar representation of a real or complex number

`ctx.`**`polar`**`(`z`)`

where `ctx` is `math53`, `mathc53` or `ctxcpp`.

Returns the polar representation of the complex number z as a pair (r, ϕ) such that $z = re^{i\phi}$:

See also Mpmath [677].

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Polar(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Polar('0.1')
ecplx('5.351847902755998475 4E-1')
```

3.1.8 Rectangular coordinates calculated from the polar representation of a real or complex number

`ctx.`**`rect`**`(`r, *phi*`)`

where `ctx` is `math53`, `mathc53` or `ctxcpp`.

Returns the complex number represented by polar coordinates (r, ϕ):

See also Mpmath [669].

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Rect(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Rect('0.1')
ecplx('5.351847902755998475 4E-1')
```

3.2 Roots and quadratic, cubic, and quartic equations

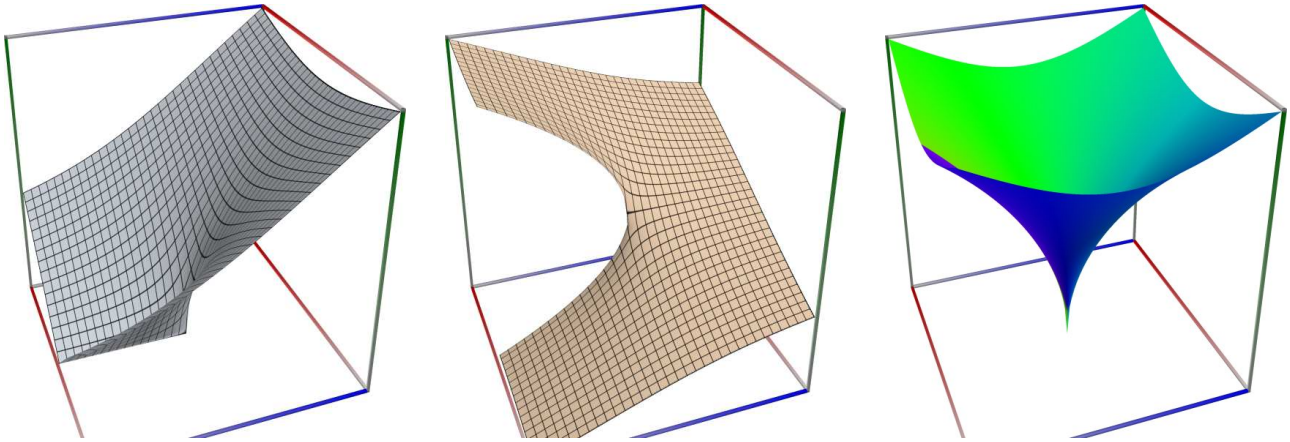
3.2.1 Square root, $\mathrm{sqrt}(x) = \sqrt{x}$

`ctx.sqrt(x)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the principal square root of x, $\sqrt{x}$. See also Wikipedia [1246], MathWorld [857], Flint [745], Ehrhardt [297] (4.1.19), Mpmath [558].

For positive real numbers, the principal root is simply the positive square root. For arbitrary complex numbers, the principal square root is defined to satisfy $\sqrt{x} = \exp(\log(x)/2)$. The function thus has a branch cut along the negative half real axis.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Sqrt(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Sqrt('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '10.7'
>>> \mathrm{d}x = dec.sqrt(x); mx = mpm.sqrt(x); ix = ipm.sqrt(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  3.2710854467592252121535649236034010853 26E+0
mpm:  3.2710854467592252121535649236034010853 26e+0
ipm:  3.2710854467592252121535649236034010853 26e+0 (7.019e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10.7'
>>> fx = fpm.sqrt(x); gx = gmp.sqrt(x); ax = apm.sqrt(x)
>>> mpm.show([fx, gx, ax])
fpm:  3.27108544675922E+00
gmp:  3.2710854467592252121535649236034010853 26E+00
apm:  3.2710854467592252121535649236034010853 26e+0 (7.019e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> \mathrm{d}z = dec.sqrt(z); mz = mpm.sqrt(z); iz = ipm.sqrt(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 3.1937447478943743524E+0              + 2.34834045674587046 92E-3j
mpm: 3.1937447478943743524e+0              + 2.34834045674587046 92e-3j
ipm: 3.1937447478943743524e+0 (5.304e-20%) + 2.34834045674587046 92e-3 (2.113e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> fz = fpm.sqrt(z); gz = gmp.sqrt(z); az = apm.sqrt(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 3.19374474789437E+00                  + 2.34834045674587E-03j
gmp: 3.1937447478943743524E+00             + 2.34834045674587046 92E-03j
apm: 3.1937447478943743524e+0 (1.061e-19%) + 2.34834045674587046 92e-3 (1.409e-19%)j
```

3.2.2 Reciprocal square root, $\mathrm{rsqrt}(x) = 1/\sqrt{x}$

`ctx.`**`rsqrt`**(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the reciprocal of the principal square root of x, $1/\sqrt{x}$. See also Wikipedia [1246], MathWorld [857], Flint [745], Ehrhardt [297] (4.1.19), Mpmath [558].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Rsqrt(0.5)
ereal('5.23598775598298873 07E-1')
>>> math53.Rsqrt('0.51')
ereal('5.35184790275599847 54E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '10.7'
>>> \mathrm{d}x = dec.RSqrt(x); mx = mpm.RSqrt(x); ix = ipm.RSqrt(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  3.2710854467592252121535649236034010853 26E+0
mpm:  3.2710854467592252121535649236034010853 26e+0
ipm:  3.2710854467592252121535649236034010853 26e+0 (7.019e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10.7'
>>> fx = fpm.RSqrt(x); gx = gmp.RSqrt(x); ax = apm.RSqrt(x)
>>> mpm.show([fx, gx, ax])
fpm:  3.27108544675922E+00
gmp:  3.2710854467592252121535649236034010853 26E+00
apm:  3.2710854467592252121535649236034010853 26e+0 (7.019e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> \mathrm{d}z = dec.RSqrt(z); mz = mpm.RSqrt(z); iz = ipm.RSqrt(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 3.1937447478943743524E+0              + 2.34834045674587046 92E-3j
mpm: 3.1937447478943743524e+0              + 2.34834045674587046 92e-3j
ipm: 3.1937447478943743524e+0 (5.304e-20%) + 2.34834045674587046 92e-3 (2.113e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> fz = fpm.RSqrt(z); gz = gmp.RSqrt(z); az = apm.RSqrt(z)
```

(continues on next page)

(continued from previous page)

```
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 3.19374474789437E+00                  + 2.34834045674587E-03j
gmp: 3.193744747894374 3524E+00            + 2.34834045674587 04692E-03j
apm: 3.193744747894374 3524e+0 (1.061e-19%) + 2.34834045674587 04692e-3 (1.409e-19%)j
```

3.2.3 Auxiliary function $\mathrm{sqrt1pm1}(x) = \sqrt{1+x} - 1$

`ctx.`**`sqrt1pm1`**`(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` , `ctxboost` or `ctxflint`.

Returns $\sqrt{1+x} - 1$, accurate also for x near 0. See also Wikipedia [1246], MathWorld [857], BoostMath [92].

This is calculated as $\mathrm{sqrt1pm1}(v) = \mathrm{expm1}(\mathrm{logp1}(v)/2)$. See *log1p()* and *expm1()*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Sqrt1pm1(0.5)
ereal('5.235987755982988 7307E-1')
>>> math53.Sqrt1pm1('0.51')
ereal('5.351847902755998 4754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.0E-10'
>>> \mathrm{d}x = dec.sqrt1pm1(x); mx = mpm.sqrt1pm1(x); ix = ipm.sqrt1pm1(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  4.9999999998750000000062499999996093750 00E-11
mpm:  4.9999999998750000000062499999996093750 00e-11
ipm:  4.9999999998750000000062499999996095978 67e-11 (1.121e-32%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.0E-10'
>>> fx = fpm.sqrt1pm1(x); gx = gmp.sqrt1pm1(x); ax = apm.sqrt1pm1(x)
>>> mpm.show([fx, gx, ax])
fpm:  4.99999999987500E-11
gmp:  4.9999999998750000000062499999996093750 00E-11
apm:  4.9999999998750000000062499999996093750 00e-11 (1.336e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.0E-10+ 1.5E-12j'
>>> \mathrm{d}z = dec.sqrt1pm1(z); mz = mpm.sqrt1pm1(z); iz = ipm.sqrt1pm1(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 4.99999999987502812 50E-11              + 7.49999999962500000 00E-13j
mpm: 4.99999999987502812 50e-11              + 7.49999999962500000 00e-13j
ipm: 4.99999999987503153 83e-11 (8.272e-13%) + 7.49999999962500000 00e-13 (1.541e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.0E-10+ 1.5E-12j'
>>> fz = fpm.sqrt1pm1(z); gz = gmp.sqrt1pm1(z); az = apm.sqrt1pm1(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 4.99999999987503E-11                    + 7.49999999962500E-13j
gmp: 4.99999999987502812 50E-11              + 7.49999999962500000 00E-13j
apm: 4.99999999987502812 50e-11 (1.972e-19%) + 7.49999999962500000 00e-13 (2.054e-19%)j
```

3.2.4 Cube root, $\mathrm{cbrt}(x) = \sqrt[3]{x}$

`ctx.cbrt(x)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxboost`, `ctxflint`.

Returns the cube root of x, $x^{1/3}$. See also Wikipedia [1233], MathWorld [842], BoostMath [97], Ehrhardt [297] (4.2.17), Mpmath [542].

This function is faster and more accurate than raising to a floating-point fraction.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Cbrt(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Cbrt('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '10.7'
>>> \mathrm{d}x = dec.cbrt(x); mx = mpm.cbrt(x); ix = ipm.cbrt(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  2.2035754532216254713516735131427483239 63E+0
mpm:  2.2035754532216254713516735131427483239 63e+0
ipm:  2.2035754532216254713516735131427483239 63e+0 (1.042e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10.7'
>>> fx = fpm.cbrt(x); gx = gmp.cbrt(x); ax = apm.cbrt(x)
>>> mpm.show([fx, gx, ax])
fpm:  2.20357545322163E+00
gmp:  2.2035754532216254713516735131427483239 63E+00
apm:  2.2035754532216254713516735131427483239 63e+0 (1.042e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> \mathrm{d}z = dec.cbrt(z); mz = mpm.cbrt(z); iz = ipm.cbrt(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 2.1687034063721566890E+0              + 1.0630892238793233640E-3j
mpm: 2.1687034063721566890e+0              + 1.0630892238793233640e-3j
ipm: 2.1687034063721566890e+0 (7.811e-20%) + 1.0630892238793233640e-3 (3.112e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> fz = fpm.cbrt(z); gz = gmp.cbrt(z); az = apm.cbrt(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 2.16870340637216E+00                  + 1.06308922387932E-03j
gmp: 2.1687034063721566890E+00             + 1.0630892238793233640E-03j
apm: 2.1687034063721566890e+0 (7.811e-20%) + 1.0630892238793233640e-3 (2.334e-19%)j
```

Every nonzero complex number has three cube roots. This function returns the cube root defined by $\exp(\log(x)/3)$ where the principal branch of the natural logarithm is used. Note that this does not give a real cube root for negative real numbers:

```
>>> mp.pretty = True
>>> mp.cbrt(-1)
(0.5 + 0.866025403784439j)
```

If you want the real cube root for negative real numbers, use the function `cuberoot` instead.

3.2.5 Nth root, $\mathrm{nroot}(x, n) = \sqrt[n]{x}$

`ctx.root_si`(*x*, *n*, *k=0*)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the nth root of x. For real negative x, n needs to be an odd integer for a real result. See also Wikipedia [1283], MathWorld [938], Flint [745], Flint [735], Ehrhardt [297] (4.2.47), Mpmath [620].

See also: https://dlmf.nist.gov/1.11#iv

The roots of $z^n = a + ib$ are

$$\sqrt[n]{R}\left(\cos\left(\frac{\alpha + 2k\pi}{n}\right) + i\sin\left(\frac{\alpha + 2k\pi}{n}\right)\right),$$

where $R = \sqrt{a^2 + b^2}$, $\alpha = \mathrm{ph}(a + ib)$, with the principal value phase, and $k = 0, 1, \ldots, n - 1$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Nroot(0.5, 4)
ereal('5.235987755982988730 7E-1')
>>> math53.Nroot('0.51', 4)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '125'; n = '3'
>>> \mathrm{d}x = dec.nthroot(x, n); mx = mpm.nthroot(x, n); ix = ipm.nthroot(x, n)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  5.000000000000000000000000000000000000000E+0
mpm:  5.000000000000000000000000000000000000000e+0
ipm:  5.000000000000000000000000000000000000000e+0 (2.755e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '125'; n = '3'
>>> fx = fpm.nthroot(x, n); gx = gmp.nthroot(x, n); ax = apm.nthroot(x, n)
>>> mpm.show([fx, gx, ax])
fpm:  5.00000000000000E+00
gmp:  5.000000000000000000000000000000000000000E+00
apm:  5.000000000000000000000000000000000000002e+0 (3.673e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '20.2 + 1.5E-2j'; n = '3'
>>> \mathrm{d}z = dec.nthroot(z, n); mz = mpm.nthroot(z, n); iz = ipm.nthroot(z, n)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 2.723435848429670208 0E+0              + 6.741176741256368021 6E-4j
mpm: 2.723435848429670208 0e+0              + 6.741176741256368021 6e-4j
ipm: 2.723435848429670208 0e+0 (1.866e-19%) + 6.741176741256368021 6e-4 (4.295e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '20.2 + 1.5E-2j'; n = '3'
>>> fz = fpm.nthroot(z, n); gz = gmp.nthroot(z, n); az = apm.nthroot(z, n)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 2.72343584842967E+00                  + 6.74117674125637E-04j
gmp: 2.723435848429670208 0E+00            + 6.741176741256368021 6E-04j
apm: 2.723435848429670208 1e+0 (4.354e-19%) + 6.741176741256368021 6e-4 (6.135e-19%)j
```

3.2.6 Unit root, unitroot(n)

`ctx.unitroot(`n`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the nth root of z.

See also Wikipedia [1291], MathWorld [933], Mpmath [619].

> **Caution**
>
> This still needs to be implemented

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Unitroots(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Unitroots('0.1')
ecplx('5.351847902755998475 4E-1')
```

3.3 Exponential and related functions

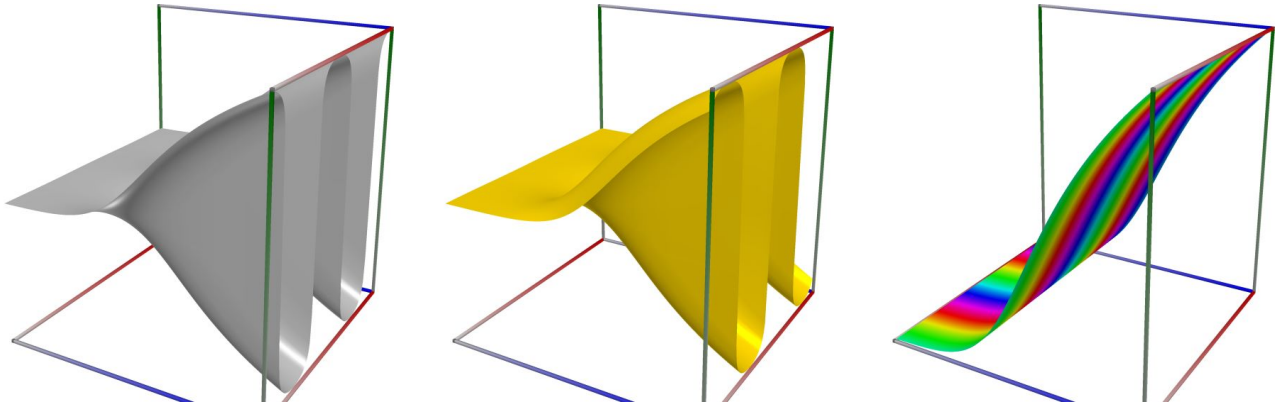
3.3.1 Exponential function $\exp(x) = e^x$

`ctx.exp(z)`

where `ctx` is `ctx_pm` (see *Python contexts* for details), `ctx53`, `ctxcpp`, `ctxflint` (see *.NET contexts* for details).

Returns $\exp(x)$, the exponential function of x. See also Wikipedia [1236], MathWorld [846], NIST [490], Ehrhardt [297] (4.2.34), Flint [738], Flint [728], Mpmath [547].

For complex numbers, the exponential function satisfies $\exp(x+iy) = e^x(\cos y + i \sin y)$.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x+iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example with real input:

```
>>> from xlcalcnet import *
>>> x = -5.1; dps = 90
>>> for ctx in ctxlistreal: print(ctx.fmtname + ': ' + ctx.fmt(ctx.exp(x)))
    fpm:  0.00609674656551564
    mpm:  0.006096746565515636107134564785424901789060042080668097308393715695956976367739348325760136 56
    dpm:  0.006096746565515636107134564785424901789060042080668097308393715695956976367739348325760136 56
    ipm: [0.006096746565515636107134564785424901789060042080668097308393715695956976367739348325760136 5613434,
          0.006096746565515636107134564785424901789060042080668097308393715695956976367739348325760136 5690139]
    gpm:  0.006096746565515636107134564785424901789060042080668097308393715695956976367739348325760136 5623
    apm: [0.006096746565515636107134564785424901789060042080668097308393715695956976367739348325760136 57 +/- 8.90e-93]
  dreal:  0.00609674656551564
  sreal:  0.006096747
  dreal:  0.00609674656551564
  ereal:  0.006096746565515636 1078
  qreal:  0.006096746565515636107134564785424 9
  oreal:  0.006096746565515636107134564785424901789060042080668097308393715695956976 3
  mreal:  0.006096746565515636107134564785424901789060042080668097308393715695956976367739348325760136 57
 sflint:  0.006096747
 dflint:  0.00609674656551564
 eflint:  0.006096746565515636 1078
 qflint:  0.006096746565515636107134564785424 9
 oflint:  0.006096746565515636107134564785424901789060042080668097308393715695956976 4
 mflint:  0.006096746565515636107134564785424901789060042080668097308393715695956976367739348325760136 57
 aflint: [0.006096746565515636107134564785424901789060042080668097308393715695956976367739348325760136 6 +/- 4.16e-92]
```

An example with complex input, using C#-style formatting for complex numbers for the result

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(50)
>>> for ctx in [fpm, mpm, cmath53, qcplx]: print(ctx.fmtname + ': ' + ctx.fmt(ctx.exp(x)))
```

(continues on next page)

(continued from previous page)

```
    fpm:  (-0.00253714179646899, 0.00554375596403168)
    mpm:  (-0.0025371417964689871432868572987332349386956711932343, 0.
↪00554375596403168031558493172233256766312015818728 07)
cmath53:  (-0.00253714179646899, 0.00554375596403168)
  qcplx:  (-0.0025371417964689871432868572987332 4, 0.00554375596403168031558493172233257)
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder for Python (real input), Python (complex input), and C# (real or complex input).

3.3.2 Auxiliary function $\mathrm{expj}(x) = e^{ix}$

`ctx.expj(z)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns $e^{iz} = \cos(z) + i\sin(z)$. See also Wikipedia [1374], MathWorld [1019], Mpmath [705].

An example with real input:

```
>>> from xlcalcnet import *
>>> x = -5.1; dps = 50
>>> for ctx in gui.ctxlist_real: print(ctx.fmtname + ': ' + ctx.cplxctx.fmt(ctx.expj(x)))
    fpm:  (0.37797774271298, 0.925814682327732)
    mpm:  (0.377977742712980563320575552928981670898641576134 2730157, 0.925814682327732296946146247544863312509403016355 1)
    dpm:  (0.377977742712980563320575552928981670898641576134 28730, 0.925814682327732296946146247544863312509403016355 6)
    ipm: ([0.377977742712980563320575552928981670898641576134 2730157, 0.
↪377977742712980563320575552928981670898641576134 2837068],
          [0.925814682327732296946146247544863312509403016355 6011089, 0.
↪925814682327732296946146247544863312509403016355 6064544])
    gpm:  (0.377977742712980563320575552928981670898641576134 2737, 0.
↪92581468232773229694614624754486331250940301635560 51)
    apm: ([0.377977742712980563320575552928981670898641576134 3 +/- 3.78e-50], [0.
↪925814682327732296946146247544863312509403016355 6 +/- 1.06e-50])
  dreal:  (0.37797774271298, 0.925814682327732)
  sreal:  (0.3779777, 0.9258147)
  dreal:  (0.37797774271298, 0.925814682327732)
  ereal:  (0.37797774271298056325, 0.92581468232773229699)
  qreal:  (0.377977742712980563320575552928981, 0.92581468232773229694614624754486 3)
  oreal:  (0.377977742712980563320575552928981670898641576134277570541766913010950 88, 0.
↪925814682327732296946146247544863312509403016355603948797054551932281 86)
  mreal:  (0.377977742712980563320575552928981670898641576134 29, 0.925814682327732296946146247544863312509403016355 59)
 sflint:  (0.3779777, 0.9258147)
 dflint:  (0.37797774271298, 0.925814682327732)
 eflint:  (0.37797774271298056325, 0.92581468232773229699)
 qflint:  (0.377977742712980563320575552928981, 0.92581468232773229694614624754486 3)
 oflint:  (0.377977742712980563320575552928981670898641576134277570541766913010950 88, 0.
↪925814682327732296946146247544863312509403016355603948797054551932281 85)
 mflint:  (0.377977742712980563320575552928981670898641576134 29, 0.925814682327732296946146247544863312509403016355 59)
 aflint: ([0.377977742712980563320575552928981670898641576134 3 +/- 2.92e-50], [0.
↪925814682327732296946146247544863312509403016355 6 +/- 2.44e-50])
```

An example with complex input, using C#-style formatting for complex numbers for the result

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(50)
>>> for ctx in [fpm, mpm, cmath53, qcplx]: print(ctx.fmtname + ': ' + ctx.fmt(ctx.expj(x)))
    fpm:  (0.0511537248671967, 0.125295392257438)
    mpm:  (0.0511537248671967434898253949927467610442454662229, 0.125295392257438253335999479318014631952742902 8572)
cmath53:  (0.0511537248671967, 0.125295392257438)
  qcplx:  (0.0511537248671967434898253949927467, 0.125295392257438253335999479318014)
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder for Python (real input), Python (complex input), and C# (real or complex input).

3.3.3 Auxiliary function $\mathrm{expjpi}(x) = e^{i\pi x} = (-1)^x$

`ctx.expjpi(z)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $e^{i\pi z} = \cos(\pi z) + i\sin(\pi z)$. See also Wikipedia [1374], MathWorld [1019], Flint [728], Mpmath [706].

Evaluation is accurate near zeros (see also *cospi()* and *sinpi()*):

An example with real input:

```
>>> from xlcalcnet import *
>>> x = -5.1; dps = 50
>>> for ctx in gui.ctxlist_real: print(ctx.fmtname + ': ' + ctx.cplxctx.fmt(ctx.expjpi(x)))
    fpm:  (-0.951056516295154, 0.309016994374946)
    mpm:  (-0.95105651629515357211643933337938214340569863412575, 0.30901699437494742410229341718281905886015458990287)
    dpm:  (-0.95105651629515357211643933337938214340569863412575, 0.30901699437494742410229341718281905886015458990288)
    ipm: ([-0.95105651629515357211643933337938214340569863412576538, -0.
↪95105651629515357211643933337938214340569863412573697 4], [0.309016994374947424102293417182819058860154589902837818 8,
↪ 0.309016994374947424102293417182819058860154589902920006 3])
    gpm:  (-0.95105651629515357211643933337938214340569863412575 17, 0.
↪30901699437494742410229341718281905886015458990278 6)
    apm: ([-0.95105651629515357211643933337938214340569863412575 8 +/- 5.79e-50], [0.
↪30901699437494742410229341718281905886015458990290 29 +/- 6.59e-50])
  dreal:  (-0.951056516295154, 0.309016994374946)
  sreal:  (-0.9510566, 0.3090167)
  dreal:  (-0.951056516295154, 0.309016994374946)
  ereal:  (-0.95105651629515357222, 0.30901699437494742386)
  qreal:  (-0.95105651629515357211643933337938214340569863412575022244730564443015319, 0.30901699437494742410229341718281905886015458990288)
  oreal:  (-0.95105651629515357211643933337938214340569863412575022244730564443015319, 0.
↪30901699437494742410229341718281905886015458990288143106772431135263019)
  mreal:  (-0.95105651629515357211643933337938214340569863412573, 0.30901699437494742410229341718281905886015458990293)
 sflint:  (-0.9510566, 0.3090167)
 dflint:  (-0.951056516295154, 0.309016994374946)
 eflint:  (-0.95105651629515357222, 0.30901699437494742383)
 qflint:  (-0.95105651629515357211643933337938214340569863412575022244730564443015318, 0.30901699437494742410229341718281905886015458990288)
 oflint:  (-0.95105651629515357211643933337938214340569863412575022244730564443015318, 0.
↪30901699437494742410229341718281905886015458990288143106772431135263019)
 mflint:  (-0.95105651629515357211643933337938214340569863412573, 0.30901699437494742410229341718281905886015458990292)
 aflint: ([-0.95105651629515357211643933337938214340569863412575 7 +/- 6.44e-50], [0.
↪30901699437494742410229341718281905886015458990290 29 +/- 8.93e-50])
```

An example with complex input, using C#-style formatting for complex numbers for the result

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(50)
>>> for ctx in [fpm, mpm, cmath53, qcplx]: print(ctx.fmtname + ': ' + ctx.fmt(ctx.expjpi(x)))
    fpm:  (-0.00177604357879891, 0.00057707154011974)
    mpm:  (-0.00177604357879890496420546318525410367559608804475 26, 0.
↪00057707154011974403113330884851121775451896357682577)
cmath53:  (-0.00177604357725158, 0.00057707154070385 5)
  qcplx:  (-0.00177604357879890496420546318529556, 0.00057707154011974403113330884851709 4)
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder for Python (real input), Python (complex input), and C# (real or complex input).

3.3.4 Exponential function with base 10, $\mathrm{exp10}(x) = 10^z$

`ctx.exp10(x)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $\mathrm{exp10}(x) = 10^z = \exp(x \cdot \log(10))$, the base-10 exponential function of z. See also Wikipedia [1242], MathWorld [846], NIST [490], Ehrhardt [297] (4.2.36), Mpmath [553].

An example with real input:

```
>>> from xlcalcnet import *
>>> x = -5.1; dps = 90
>>>     for ctx in gui.ctxlist_real: print(ctx.fmtname + ': ' + ctx.fmt(ctx.exp10(x))
    fpm:  7.94328234724282E-06
    mpm:  0.000007943282347242815020659182828363879325889606317554843320923239293169556971914875374970952 51
    dpm:  0.000007943282347242815020659182828363879325889606317554843320923239293169556971914875374970952 52
    ipm: [0.000007943282347242815020659182828363879325889606317554843320923239293169556971914875374970952 4789891,
          0.000007943282347242815020659182828363879325889606317554843320923239293169556971914875374970952 5445325]
    gpm:  7.943282347242815020659182828363879325889606317554843320923239293169556971914875374970952 5146e-06
    apm: [7.943282347242815020659182828363879325889606317554843320923239293169556971914875374970953e-6 +/- 5.13e-94]
  dreal:  7.94328234724282E-06
  sreal:  7.943281E-06
  dreal:  7.94328234724281E-06
  ereal:  7.943282347242815 0204e-06
  qreal:  7.943282347242815020659182828 36389e-06
  oreal:  7.943282347242815020659182828363879325889606317554843320923239293169556 9e-06
  mreal:  7.943282347242815020659182828363879325889606317554843320923239293169556971914875374970952 53E-06
 sflint:  7.943284E-06
 dflint:  7.94328234724282E-06
 eflint:  7.943282347242815 0221e-06
 qflint:  7.943282347242815020659182828 36388e-06
 oflint:  7.943282347242815020659182828363879325889606317554843320923239293169556 72e-06
 mflint:  7.943282347242815020659182828363879325889606317554843320923239293169556971914875374970952 53E-06
 aflint: [7.943282347242815020659182828363879325889606317554843320923239293169556971914875374970953e-6 +/- 5.60e-94]
```

An example with complex input, using C#-style formatting for complex numbers for the result

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(50)
>>> for ctx in [fpm, mpm, cmath53, qcplx]: print(ctx.fmtname + ': ' + ctx.fmt(ctx.exp10(x)))
    fpm:  (-8.50038314869339E-07, -7.89766859973711E-06)
    mpm:  (-0.000000850038314869338609239805360338337681680955604224 9, -0.
↪0000078976685997371034342479584101601100277547297776704)
cmath53:  (-8.50038314869336E-07, -7.89766859973711E-06)
  qcplx:  (-8.500383148693386092398053603383396816809556042249e-07, -7.897668599737103434247958410160 12e-06)
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder for Python (real input), Python (complex input), and C# (real or complex input).

3.3.5 Exponential function with base 2, $\exp2(x) = 2^x$

`ctx.exp2(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $\exp2(x) = 2^x = \exp(x \cdot \log(2))$, the base-2 exponential function of x. See also Wikipedia [1243], MathWorld [846], NIST [490], Ehrhardt [297] (4.2.35).

An example with real input:

```
>>> from xlcalcnet import *
>>> x = -5.1; dps = 90
>>> for ctx in gui.ctxlist_real: print(ctx.fmtname + ': ' + ctx.fmt(ctx.exp2(x)))
    fpm:  0.0291572809855252
    mpm:  0.029157280985525231749416977067185692719600936385984188715640543296418656884705110862615858 5
    dpm:  0.029157280985525231749416977067185692719600936385984188715640543296418656884705110862615858 5
    ipm: [0.029157280985525231749416977067185692719600936385984188715640543296418656884705110862615858 5305,
          0.029157280985525231749416977067185692719600936385984188715640543296418656884705110862615858 591864]
    gpm:  0.029157280985525231749416977067185692719600936385984188715640543296418656884705110862615858 538
    apm: [0.029157280985525231749416977067185692719600936385984188715640543296418656884705110862615858 6 +/- 8.98e-92]
  dreal:  0.0291572809855252
  sreal:  0.02915728
  dreal:  0.0291572809855252
  ereal:  0.029157280985525231751
  qreal:  0.029157280985525231749416977067185 7
```

(continues on next page)

(continued from previous page)

```
 oreal:  0.02915728098552523174941697706718569271960093638598418871564054329641865688470511086261585 86
 mreal:  0.02915728098552523174941697706718569271960093638598418871564054329641865688470511086261585 86
sflint:  0.02915728
dflint:  0.0291572809855252
eflint:  0.02915728098552523175 1
qflint:  0.02915728098552523174941697706718 57
oflint:  0.02915728098552523174941697706718569271960093638598418871564054329641865688470511086261585 7
mflint:  0.02915728098552523174941697706718569271960093638598418871564054329641865688470511086261585 86
aflint: [0.02915728098552523174941697706718569271960093638598418871564054329641865688470511086261585 9 +/- 5.38e-91]
```

An example with complex input, using C#-style formatting for complex numbers for the result

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(50)
>>> for ctx in [fpm, mpm, cmath53, qcplx]: print(ctx.fmtname + ': ' + ctx.fmt(ctx.exp2(x)))
    fpm:  (0.00534910656134485, 0.0286624160437366)
    mpm:  (0.00534910656134485266603100402957978954990 4770752861, 0.
↪028662416043736589994269252987760995841700161519681)
cmath53:  (0.00534910656134486, 0.0286624160437366)
  qcplx:  (0.00534910656134485266603100402957979, 0.028662416043736589994269252987761)
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder for Python (real input), Python (complex input), and C# (real or complex input).

3.3.6 Auxiliary function $\mathrm{expm1}(x) = e^x - 1$

`ctx.`**`expm1`**`(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $\mathrm{expm1}(x) = \exp(x) - 1 = e^x - 1$, computed accurately also for small x. See also Wikipedia [1235], MathWorld [846], NIST [490], BoostMath [75], Ehrhardt [297] (4.2.37), Flint [738], Flint [728], Mpmath [548].

An example with real input:

```
>>> from xlcalcnet import *
>>> x = -5.1; dps = 90
>>> for ctx in gui.ctxlist_real: print(ctx.fmtname + ': ' + ctx.fmt(ctx.expm1(x)))
    fpm:  -0.993903253434484
    mpm:  -0.99390325343448436389286543521457509821093957919331902691606284304043023632260651674239863
    dpm:  -0.99390325343448436389286543521457509821093957919331902691606284304043023632260651674239863
    ipm: [-0.99390325343448436389286543521457509821093957919331902691606284304043023632260651674239863 86,
          -0.99390325343448436389286543521457509821093957919331902691606284304043023632260651674239863 40222]
    gpm:  -0.99390325343448436389286543521457509821093957919331902691606284304043023632260651674239863 4
    apm: [-0.99390325343448436389286543521457509821093957919331902691606284304043023632260651674239863 +/- 5.31e-91]
  dreal:  -0.993903253434484
  sreal:  -0.9939033
  dreal:  -0.993903253434484
  ereal:  -0.99390325343448436389
  qreal:  -0.99390325343448436389286543521457 5
  oreal:  -0.99390325343448436389286543521457509821093957919331902691606284304043 2
  mreal:  -0.99390325343448436389286543521457509821093957919331902691606284304043023632260651674239864
 sflint:  -0.9939033
 dflint:  -0.993903253434484
 eflint:  -0.99390325343448436389
 qflint:  -0.99390325343448436389286543521457 5
 oflint:  -0.99390325343448436389286543521457509821093957919331902691606284304043 2
 mflint:  -0.99390325343448436389286543521457509821093957919331902691606284304043023632260651674239864
 aflint: [-0.99390325343448436389286543521457509821093957919331902691606284304043023632260651674239863 +/- 6.53e-91]
```

An example with complex input, using C#-style formatting for complex numbers for the result

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(50)
>>> for ctx in [fpm, mpm, cmath53, qcplx]: print(ctx.fmtname + ': ' + ctx.fmt(ctx.expm1(x)))
    fpm:  (-1.00253714179647, 0.00554375596403168)
    mpm:  (-1.0025371417964689871432868572987332349386956711932, 0.
↪00554375596403168031558493172233256766312015818728 07)
cmath53:  (-1.00253714179647, 0.00554375596403168)
  qcplx:  (-1.0025371417964689871432868572987 3, 0.00554375596403168031558493172233257)
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder for Python (real input), Python (complex input), and C# (real or complex input).

3.3.7 Auxiliary function $\mathrm{exp10m1}(x) = 10^x - 1$

`ctx.`**`exp10m1`**`(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $10^x - 1 = \mathrm{expm1}(x \cdot \log(10))$. See also *expm1()*.

An example with real input:

```
>>> from xlcalcnet import *
>>> x = -5.1; dps = 90
>>> for ctx in gui.ctxlist_real: print(ctx.fmtname + ': ' + ctx.fmt(ctx.exp10m1(x)))
    fpm:  -0.999992056717653
    mpm:  -0.999992056717652757184979340817171636120674110393682445156679076760706830443028085124625029
    dpm:  -0.999992056717652757184979340817171636120674110393682445156679076760706830443028085124625029
    ipm: [-0.999992056717652757184979340817171636120674110393682445156679076760706830443028085124625502915426,
          -0.999992056717652757184979340817171636120674110393682445156679076760706830443028085124625502903153]
    gpm:  -0.999992056717652757184979340817171636120674110393682445156679076760706830443028085124625502903
    apm: [-0.999992056717652757184979340817171636120674110393682445156679076760706830443028085124625029 +/- 1.55e-91]
  dreal:  -0.999992056717653
  sreal:  -0.9999921
  dreal:  -0.999992056717653
  ereal:  -0.9999920567176527519
  qreal:  -0.999992056717652757184979340817172
  oreal:  -0.999992056717652757184979340817171636120674110393682445156679076760706830
  mreal:  -0.999992056717652757184979340817171636120674110393682445156679076760706830443028085124625029
 sflint:  -0.9999921
 dflint:  -0.999992056717653
 eflint:  -0.9999920567176527519
 qflint:  -0.999992056717652757184979340817172
 oflint:  -0.999992056717652757184979340817171636120674110393682445156679076760706830
 mflint:  -0.999992056717652757184979340817171636120674110393682445156679076760706830443028085124625029
 aflint: [-0.999992056717652757184979340817171636120674110393682445156679076760706830443028085124625029 +/- 7.06e-91]
```

An example with complex input, using C#-style formatting for complex numbers for the result

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(50)
>>> for ctx in [fpm, mpm, cmath53, qcplx]: print(ctx.fmtname + ': ' + ctx.fmt(ctx.exp10m1(x)))
    fpm:  (-1.00000085003831, -7.8976685997371E-06)
    mpm:  (-1.0000008500383148693386092398053603383376816809556, -0.
↪00000789766859973710343424795841016011002775472977 6704)
cmath53:  (-1.00000085003831, -7.8976685997371E-06)
  qcplx:  (-1.0000008500383148693386092398 536, -7.897668599737103434247958410160 12e-06)
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder for Python (real input), Python (complex input), and C# (real or complex input).

3.3.8 Auxiliary function $\mathrm{exp2m1}(x) = 2^x - 1$

`ctx.exp2m1(x)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $2^x - 1 = \mathrm{expm1}(x \cdot \log(2))$. See also *expm1()*.

An example with real input:

```
>>> from xlcalcnet import *
>>> x = -5.1; dps = 90
>>> for ctx in gui.ctxlist_real: print(ctx.fmtname + ': ' + ctx.fmt(ctx.exp2m1(x)))
    fpm:  -0.970842719014475
    mpm:  -0.97084271901447476825058302293281430728039906361401581128435945670358134311529488913738414 1
    dpm:  -0.97084271901447476825058302293281430728039906361401581128435945670358134311529488913738414 1
    ipm: [-0.97084271901447476825058302293281430728039906361401581128435945670358134311529488913738414 15347,
          -0.97084271901447476825058302293281430728039906361401581128435945670358134311529488913738414 128924]
    gpm:  -0.97084271901447476825058302293281430728039906361401581128435945670358134311529488913738414 141
    apm: [-0.97084271901447476825058302293281430728039906361401581128435945670358134311529488913738414 1 +/- 5.79e-91]
  dreal:  -0.970842719014475
  sreal:  -0.9708427
  dreal:  -0.970842719014475
  ereal:  -0.97084271901447476827
  qreal:  -0.97084271901447476825058302293281 4
  oreal:  -0.97084271901447476825058302293281430728039906361401581128435945670358134
  mreal:  -0.97084271901447476825058302293281430728039906361401581128435945670358134311529488913738414 2
 sflint:  -0.9708427
 dflint:  -0.970842719014475
 eflint:  -0.97084271901447476827
 qflint:  -0.97084271901447476825058302293281 4
 oflint:  -0.97084271901447476825058302293281430728039906361401581128435945670358134
 mflint:  -0.97084271901447476825058302293281430728039906361401581128435945670358134311529488913738414 2
 aflint: [-0.97084271901447476825058302293281430728039906361401581128435945670358134311529488913738414 1 +/- 7.68e-91]
```

An example with complex input, using C#-style formatting for complex numbers for the result

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(50)
>>> for ctx in [fpm, mpm, cmath53, qcplx]: print(ctx.fmtname + ': ' + ctx.fmt(ctx.exp2m1(x)))
    fpm:  (-0.994650893438655, 0.0286624160437366)
    mpm:  (-0.99465089343865514733396899597042021045009152292471, 0.
↪028662416043736589994269252987760995841700161519681)
cmath53:  (-0.994650893438655, 0.0286624160437366)
  qcplx:  (-0.99465089343865514733396899597042, 0.028662416043736589994269252987761)
```

The above and additional examples can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder for Python (real input), Python (complex input), and C# (real or complex input).

3.4 Logarithms and related functions

3.4.1 Logarithm $\log(x[, base])$

`ctx.log`(*x*[, *base*])

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

With one argument, returns the natural logarithm of x (to base e).

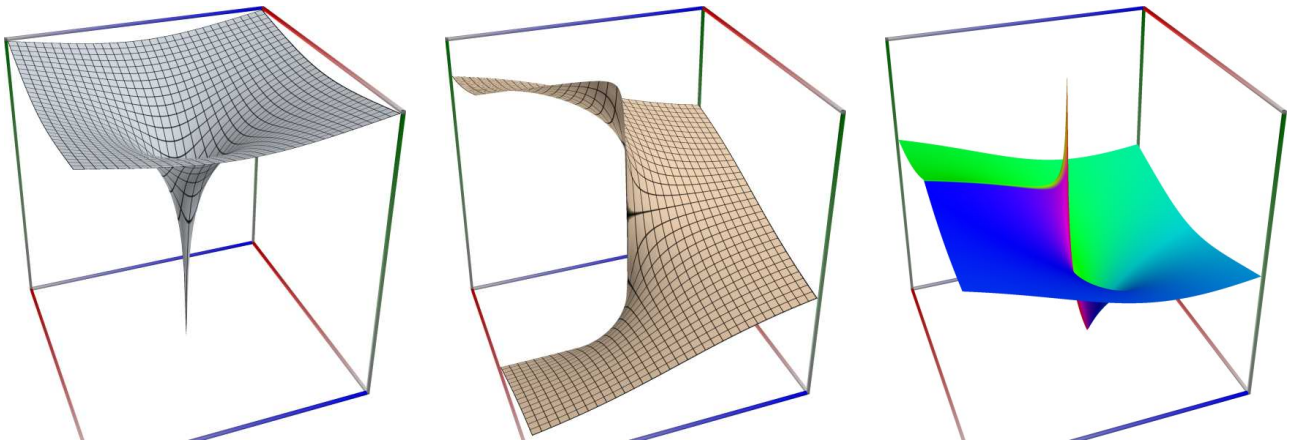
With two arguments, returns the logarithm of x to the given base, calculated as $\log(x)/\log(base)$.

See also Wikipedia [1232], MathWorld [852], NIST [491], Ehrhardt [297] (4.2.41), Flint [738], Flint [728], Mpmath [554].

$$\log(x) = \int_1^x \frac{1}{t}\,\mathrm{d}t.$$

If x is less than 1, then this area is considered to be negative.

The principal branch of the complex logarithm is used, meaning that $\Im(\log(z)) = -\pi < \arg(z) \leq \pi$.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Log(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Log('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.0E-100'
>>> \mathrm{d}x = dec.Log(x); mx = mpm.Log(x); ix = ipm.Log(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  -2.3025850929940456840179914546843642076 01E+2
mpm:  -2.3025850929940456840179914546843642076 01e+2
ipm:  -2.3025850929940456840179914546843642076 01e+2 (-6.381e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.0E-100'
>>> fx = fpm.Log(x); gx = gmp.Log(x); ax = apm.Log(x)
>>> mpm.show([fx, gx, ax])
```

(continues on next page)

(continued from previous page)

```
fpm:  -2.30258509299405E+02
gmp:  -2.3025850929940456840179914546843642076O1E+02
apm:  -2.3025850929940456840179914546843642076O1e+2 (-1.276e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.0E-100 + 1.57079632679489j'
>>> \mathrm{d}z = dec.Log(z); mz = mpm.Log(z); iz = ipm.Log(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 4.51582705289451E-01                + 1.57079632679490E+00j
mpm: 4.515827052894506579E-01            + 1.570796326794896619E+00j
ipm: 4.515827052894506579e-01 (1.876e-19%) + 1.570796326794896619e+00 (5.392e-20%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.0E-100 + 1.57079632679489j'
>>> fz = fpm.Log(z); gz = gmp.Log(z); az = apm.Log(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 4.51582705289451E-01                + 1.57079632679490E+00j
gmp: 4.515827052894506579E-01            + 1.570796326794896619E+00j
apm: 4.515827052894506579e-1 (1.407e-19%) + 1.570796326794896619e+0 (1.078e-19%)j
```

3.4.2 Logarithm with base 10, $\log10(x) = \log_{10}(x)$

`ctx.log10(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the base-10 logarithm of x, $\log_{10}(x) = \log(x)/\log(10)$. See also. Wikipedia [1230], MathWorld [841], NIST [491], Ehrhardt [297] (4.2.44), Mpmath [553].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Log10(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Log10('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.0E-100'
>>> \mathrm{d}x = dec.log10(x); mx = mpm.log10(x); ix = ipm.log10(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  -1.000000000000000000000000000000000000000E+2
mpm:  -1.000000000000000000000000000000000000000e+2
ipm:  -1.000000000000000000000000000000000000000e+2 (-1.469e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.0E-100'
>>> fx = fpm.log10(x); gx = gmp.log10(x); ax = apm.log10(x)
>>> mpm.show([fx, gx, ax])
fpm:  -1.00000000000000E+02
gmp:  -1.000000000000000000000000000000000000000E+02
apm:  -1.000000000000000000000000000000000000000e+2 (-1.469e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.0E-100 + 1.57079632679489j'
>>> \mathrm{d}z = dec.log10(z); mz = mpm.log10(z); iz = ipm.log10(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
```

(continues on next page)

(continued from previous page)

```
dec: 1.96119877030150829O5E-1             + 6.821881769209206737 4E-1j
mpm: 1.96119877030150829O5e-1             + 6.821881769209206737 4e-1j
ipm: 1.96119877030150829O5e-1 (1.62e-19%) + 6.821881769209206737 4e-1 (6.208e-20%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.0E-100 + 1.570796326794 89j'
>>> fz = fpm.log10(z); gz = gmp.log10(z); az = apm.log10(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.96119877030151E-01                  + 6.82188176920921E-01j
gmp: 1.96119877030150829O5E-01             + 6.821881769209206737 4E-01j
apm: 1.96119877030150829O5e-1 (2.159e-19%) + 6.821881769209206737 4e-1 (6.208e-20%)j
```

3.4.3 Logarithm with base 2, $\log2(x) = \log_2(x)$

`ctx.log2`(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the base-2 logarithm of x, $\log_2(x) = \log(x)/\log(2)$. See also Wikipedia :cite:p: $WikipediaFun19$, MathWorld [840], NIST [491].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Log2(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Log2('0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.0E-100'
>>> \mathrm{d}x = dec.log2(x); mx = mpm.log2(x); ix = ipm.log2(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  -3.3219280948873623478703194294893901758 65E+2
mpm:  -3.3219280948873623478703194294893901758 65e+2
ipm:  -3.3219280948873623478703194294893901758 65e+2 (-8.846e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.0E-100'
>>> fx = fpm.log2(x); gx = gmp.log2(x); ax = apm.log2(x)
>>> mpm.show([fx, gx, ax])
fpm:  -3.32192809488736E+02
gmp:  -3.3219280948873623478703194294893901758 65E+02
apm:  -3.3219280948873623478703194294893901758 65e+2 (-8.846e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.0E-100 + 1.570796326794 89j'
>>> \mathrm{d}z = dec.log2(z); mz = mpm.log2(z); iz = ipm.log2(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.96119877030150829O5E-1             + 6.821881769209206737 4E-1j
mpm: 1.96119877030150829O5e-1             + 6.821881769209206737 4e-1j
ipm: 1.96119877030150829O5e-1 (1.62e-19%) + 6.821881769209206737 4e-1 (6.208e-20%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.0E-100 + 1.570796326794 89j'
>>> fz = fpm.log2(z); gz = gmp.log2(z); az = apm.log2(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.96119877030151E-01                  + 6.82188176920921E-01j
gmp: 1.96119877030150829O5E-01             + 6.821881769209206737 4E-01j
apm: 1.96119877030150829O5e-1 (2.159e-19%) + 6.821881769209206737 4e-1 (6.208e-20%)j
```

3.4.4 Auxiliary function $\mathrm{log1p}(x) = \log(x+1)$

`ctx.log1p(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns $\log(1+x) = \log(1+x)$, accurately for small x. See also Wikipedia [1241], Wikipedia [1240], MathWorld [851], NIST [491], BoostMath [107], Ehrhardt [297] (4.2.34), Flint [738], Flint [728], Mpmath [552].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Log1p(0.5)
ereal('5.23598775598298873077E-1')
>>> math53.Log1p('0.51')
ereal('5.35184790275599847540E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.0E-10'
>>> \mathrm{d}x = dec.log1p(x); mx = mpm.log1p(x); ix = ipm.log1p(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  9.9999999995000000000333333333308333333334E-11
mpm:  9.9999999995000000000333333333308333333334e-11
ipm:  9.9999999995000000000333333333308337790670e-11 (1.121e-32%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.0E-10'
>>> fx = fpm.log1p(x); gx = gmp.log1p(x); ax = apm.log1p(x)
>>> mpm.show([fx, gx, ax])
fpm:  9.99999999950000E-11
gmp:  9.9999999995000000000333333333308333333334E-11
apm:  9.9999999995000000000333333333308333333330e-11 (1.336e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.0E-10 + 1.57079632679489j'
>>> \mathrm{d}z = dec.log1p(z); mz = mpm.log1p(z); iz = ipm.log1p(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 6.2170267547320030388E-1              + 1.0038848218085834701E+0j
mpm: 6.2170267547320030388e-1              + 1.0038848218085834701e+0j
ipm: 6.2170267547320030388e-1 (1.362e-19%) + 1.0038848218085834701e+0 (1.688e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.0E-10 + 1.57079632679489j'
>>> fz = fpm.log1p(z); gz = gmp.log1p(z); az = apm.log1p(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 6.21702675473200E-01                  + 1.00388482180858E+00j
gmp: 6.2170267547320030388E-01             + 1.0038848218085834701E+00j
apm: 6.2170267547320030388e-1 (1.362e-19%) + 1.0038848218085834701e+0 (1.688e-19%)j
```

From mpmath:

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> mp.dps = 15; mp.pretty = True
>>> log(1+1e-10); print(mp.log1p(1e-10))
1.00000008269037e-10
9.9999999995e-11
>>> mp.log1p(1e-100j)
(5.0e-201 + 1.0e-100j)
>>> mp.log1p(0)
0.0
```

3.4.5 Auxiliary function $\mathrm{log10p1}(x) = \log_{10}(1+x)$

`ctx.log10p1`(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $\mathrm{log10p1}(x)) = \log_{10}(1+x) = \mathrm{log1p}(x)/\log(10)$. See also *log1p()*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Log10p1(0.5)
ereal('5.23598775598298873077E-1')
>>> math53.Log10p1('0.51')
ereal('5.35184790275599847547E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.0E-10'
>>> \mathrm{d}x = dec.log10p1(x); mx = mpm.log10p1(x); ix = ipm.log10p1(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  4.3429448188153710355741397580695090215 36E-11
mpm:  4.3429448188153710355741397580695090215 36e-11
ipm:  4.3429448188153710355741397580695092151 16e-11 (1.121e-32%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.0E-10'
>>> fx = fpm.log10p1(x); gx = gmp.log10p1(x); ax = apm.log10p1(x)
>>> mpm.show([fx, gx, ax])
fpm:  4.34294481881537E-11
gmp:  4.3429448188153710355741397580695090215 36E-11
apm:  4.3429448188153710355741397580695090215 36e-11 (3.077e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.0E-10 + 1.57079632679489j'
>>> \mathrm{d}z = dec.log10p1(z); mz = mpm.log10p1(z); iz = ipm.log10p1(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 2.70002041342499033 26E-1              + 4.35981638577897039 55E-1j
mpm: 2.70002041342499033 26e-1              + 4.35981638577897039 55e-1j
ipm: 2.70002041342499033 26e-1 (4.706e-19%) + 4.35981638577897039 55e-1 (4.371e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.0E-10 + 1.57079632679489j'
>>> fz = fpm.log10p1(z); gz = gmp.log10p1(z); az = apm.log10p1(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 2.70002041342499E-01                  + 4.35981638577897E-01j
gmp: 2.70002041342499033 26E-01            + 4.35981638577897039 55E-01j
apm: 2.70002041342499033 26e-1 (6.274e-19%) + 4.35981638577897039 55e-1 (6.8e-19%)j
```

3.4.6 Auxiliary function $\mathrm{log2p1}(x) = \log_{2}(1+x)$

`ctx.log2p1`(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $\mathrm{log2p1}(x)) = \log_{2}(1+x) = \mathrm{log1p}(x)/\log(2)$. See also *log1p()*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Log2p1(0.5)
ereal('5.23598775598298873077E-1')
```

(continues on next page)

(continued from previous page)

```
>>> math53.Log2p1('0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.0E-10'
>>> \mathrm{d}x = dec.log2p1(x); mx = mpm.log2p1(x); ix = ipm.log2p1(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.4426950408168286553202854941031651076 41E-10
mpm:  1.4426950408168286553202854941031651076 41e-10
ipm:  1.4426950408168286553202854941031651719 47e-10 (1.121e-32%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.0E-10'
>>> fx = fpm.log2p1(x); gx = gmp.log2p1(x); ax = apm.log2p1(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.44269504081683E-10
gmp:  1.4426950408168286553202854941031651076 41E-10
apm:  1.4426950408168286553202854941031651076 41e-10 (3.705e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.0E-10 + 1.57079632679489j'
>>> \mathrm{d}z = dec.log2p1(z); mz = mpm.log2p1(z); iz = ipm.log2p1(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 8.9692736681258666009E-1              + 1.4482996540469440736E+0j
mpm: 8.9692736681258666009e-1              + 1.4482996540469440736e+0j
ipm: 8.9692736681258666009e-1 (3.305e-19%) + 1.4482996540469440736e+0 (4.679e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.0E-10 + 1.57079632679489j'
>>> fz = fpm.log2p1(z); gz = gmp.log2p1(z); az = apm.log2p1(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 8.96927366812587E-01                  + 1.44829965404694E+00j
gmp: 8.9692736681258666009E-01             + 1.4482996540469440736E+00j
apm: 8.9692736681258666009e-1 (5.666e-19%) + 1.4482996540469440736e+0 (6.433e-19%)j
```

3.5 Power functions

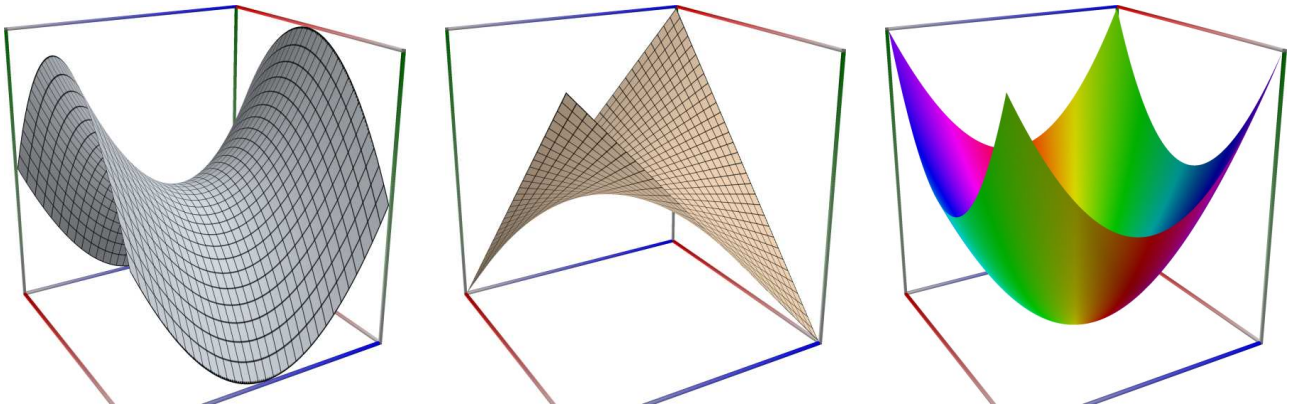
3.5.1 Square, x^2

`ctx.sqr`(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the square function of x, x^2. See also Wikipedia [1245], MathWorld [856], Ehrhardt [297] (4.1.18).

Returns the square function of x, x^2. See also Wikipedia [1245], MathWorld [856], Ehrhardt [297] (4.1.18).



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

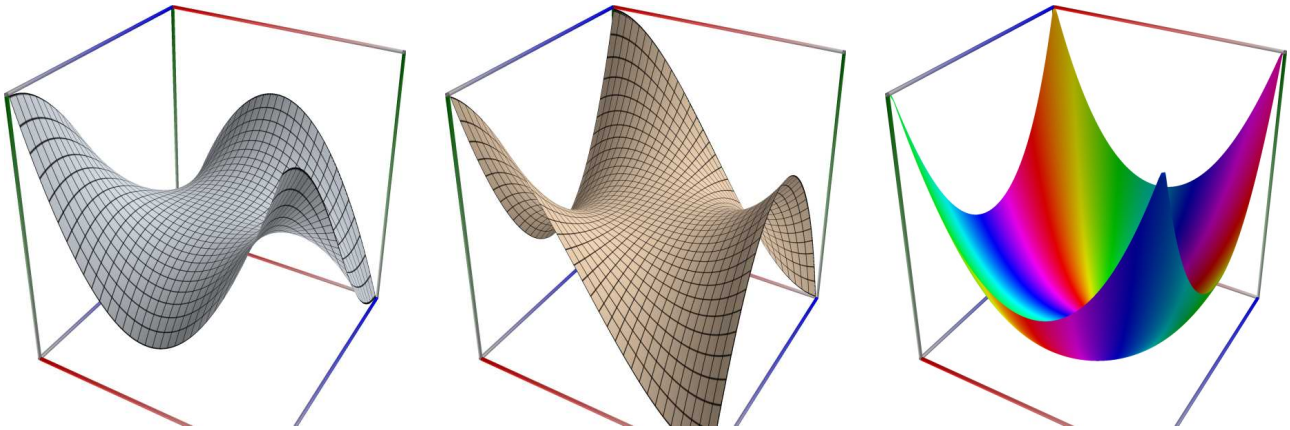
```
>>> from xlcalcnet import math53
>>> math53.Square(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Square('0.51')
ereal('5.3518479027559984754E-1')
```

3.5.2 Cube, x^3

`ctx.cube`(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the cube function of x, x^3. See also Wikipedia [1245], MathWorld [856], Ehrhardt [297] (4.1.18).



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Square(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Square('0.51')
ereal('5.351847902755998475 4E-1')
```

3.5.3 Auxiliary function $\mathrm{powi}(x, n) = x^n$

`ctx.`**`pow_si`**(x, n)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Note: ctx.intpower(x, n)

Returns the integer power function of x, x^n. See also Wikipedia [1245], MathWorld [856].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Intpower(0.5, 2)
ereal('5.2359877559829887307E-1')
>>> math53.Intpower('0.51', 2)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '10.5'
>>> \mathrm{d}x = dec.square(x); mx = mpm.square(x); ix = ipm.square(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.1025000000000000000000000000000000000E+2
mpm:  1.1025000000000000000000000000000000000e+2
ipm:  1.1025000000000000000000000000000000000e+2 (0.0%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10.5'
>>> fx = fpm.square(x); gx = gmp.square(x); ax = apm.square(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.10250000000000E+02
gmp:  1.1025000000000000000000000000000000000E+02
apm:  1.1025000000000000000000000000000000000e+2 (0.0%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> \mathrm{d}z = dec.square(z); mz = mpm.square(z); iz = ipm.square(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.0403977500000000000E+2              + 3.0600000000000000000E-1j
mpm: 1.0403977500000000000e+2              + 3.0600000000000000000e-1j
ipm: 1.0403977500000000000e+2 (2.084e-19%) + 3.0600000000000000000e-1 (2.076e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> fz = fpm.square(z); gz = gmp.square(z); az = apm.square(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.04039775000000E+02                  + 3.06000000000000E-01j
gmp: 1.0403977500000000000E+02             + 3.0600000000000000000E-01j
apm: 1.0403977500000000000e+2 (2.084e-19%) + 3.0600000000000000000e-1 (2.076e-19%)j
```

3.5.4 Auxiliary function $\mathrm{hypot}(x, y) = \sqrt{x^2 + y^2}$

`ctx.`**`hypot`**(x, y)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $\mathrm{hypot}(x, y) = \sqrt{x^2 + y^2}$ in a way which avoids undue underflow and overflow. See also Wikipedia [1286], BoostMath [122], Mpmath [614].

The function is even and symmetric in x and y, so we assume $x, y > 0$ and $x > y$ (we can permute the arguments if this is not the case). Then the result is calculated as:

$$\mathrm{hypot}(x, y) = x\sqrt{1 + \left(\frac{y}{x}\right)^2}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Hypot(0.5, 2)
ereal('5.235987755982988730 7E-1')
>>> math53.Hypot(0.5, 2)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; a = '20.4'; b = '10.4'
>>> \mathrm{d}x = dec.hypot(a, b); mx = mpm.hypot(a, b); ix = ipm.hypot(a, b)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  2.2898034850178737554414370473513406543 58E+1
mpm:  2.2898034850178737554414370473513406543 58e+1
ipm:  2.2898034850178737554414370473513406543 58e+1 (2.406e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; a = '20.4'; b = '10.4'
>>> fx = fpm.hypot(a, b); gx = gmp.hypot(a, b); ax = apm.hypot(a, b)
>>> mpm.show([fx, gx, ax])
fpm:  2.28980348501787E+01
gmp:  2.2898034850178737554414370473513406543 58E+01
apm:  2.2898034850178737554414370473513406543 58e+1 (2.326e-38%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; a = '20.2 + 1.5E+2j'; b = '10.7 + 2.3E+1j'
>>> \mathrm{d}z = dec.hypot(a, b); mz = mpm.hypot(a, b); iz = ipm.hypot(a, b)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 2.1614348471624799140E+1              + 1.5157061080517169075E+2j
mpm: 2.1614348471624799140e+1              + 1.5157061080517169075e+2j
ipm: 2.1614348471624799140e+1 (3.762e-19%) + 1.5157061080517169075e+2 (1.431e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; a = '20.2 + 1.5E+2j'; b = '10.7 + 2.3E+1j'
>>> fz = fpm.hypot(a, b); gz = gmp.hypot(a, b); az = apm.hypot(a, b)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 2.16143484716248E+01                  + 1.51570610805172E+02j
gmp: 2.1614348471624799140E+01             + 1.5157061080517169075E+02j
apm: 2.1614348471624799140e+1 (2.508e-18%) + 1.5157061080517169075e+2 (3.29e-18%)j
```

3.5.5 Power function, $\mathrm{pow}(x, y) = x^y$

`ctx.`**`pow`**(x, y)

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns $x^y = \exp(y \log(x))$. See also Wikipedia [1250], Wikipedia [1237], MathWorld [853], NIST [489], Ehrhardt [297] (4.2.34), Flint [745], Flint [735], Mpmath [555].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Pow(0.5, 3)
ereal('5.235987755982988730 7E-1')
>>> math53.Pow('0.51', 3)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; a = '20.4'; b = '10.4'
>>> \mathrm{d}x = dec.power(a, b); mx = mpm.power(a, b); ix = ipm.power(a, b)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  4.17016982299024709890148265522949408331 1E+13
mpm:  4.17016982299024709890148265522949408331 0e+13
ipm:  4.17016982299024709890148265522949408331 2e+13 (3.874e-38%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; a = '20.4'; b = '10.4'
>>> fx = fpm.power(a, b); gx = gmp.power(a, b); ax = apm.power(a, b)
>>> mpm.show([fx, gx, ax])
fpm:  4.17016982299025E+13
gmp:  4.17016982299024709890148265522949408331 0E+13
apm:  4.17016982299024709890148265522949408331 2e+13 (3.874e-38%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; a = '20.2 + 1.5E+2j'; b = '10.7 + 2.3E+1j'
>>> \mathrm{d}z = dec.power(a, b); mz = mpm.power(a, b); iz = ipm.power(a, b)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 4.088253633998730 3539E+8               - 8.454558465387471 0182E+8j
mpm: 4.088253633998730 3540e+8               - 8.454558465387471 0183e+8j
ipm: 4.088253633998730 3538e+8 (5.951e-18%) - 8.454558465387471 0180e+8 (-4.195e-18%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; a = '20.2 + 1.5E+2j'; b = '10.7 + 2.3E+1j'
>>> fz = fpm.power(a, b); gz = gmp.power(a, b); az = apm.power(a, b)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 4.08825363399861E+08                    - 8.45455846538744E+08j
gmp: 4.088253633998730 3540E+08              - 8.454558465387471 0183E+08j
apm: 4.088253633998730 3538e+8 (5.951e-18%) - 8.454558465387471 0180e+8 (-4.195e-18%)j
```

3.5.6 Auxiliary function, $\mathrm{powm1}(x, y) = x^y - 1$

`ctx.powm1`(a, b)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $a^b - 1$, computed accurately also when a^b is very close to 1. This is calculated as $\mathrm{powm1}(a, b) = \mathrm{expm1}(b \log(a))$. See also Wikipedia [1237], MathWorld [853], NIST [489], BoostMath [91], Mpmath [618].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Powm1(0.5, 3)
ereal('5.235987755982988730 7E-1')
>>> math53.Powm1(0.5, 3)
ereal('5.351847902755998475 4E-1')
```

From mpmath:

```
>>> from xlcalcnet import mp
>>> mp.dps = 15; mp.pretty = True
>>> power(0.99999995, 1e-10) - 1
0.0
>>> powm1(0.99999995, 1e-10)
-5.00000012791934e-18
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; a = '0.99999995'; b = '1e-10'
>>> \mathrm{d}x = dec.powm1(a, b); mx = mpm.powm1(a, b); ix = ipm.powm1(a, b)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  -5.0000001250000041541668222916728880419 26E-18
mpm:  -5.0000001250000041541668222916728879195 13e-18
ipm:  -5.0000001250000041541668222916728884934 85e-18 (-1.148e-32%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; a = '0.99999995'; b = '1e-10'
>>> fx = fpm.powm1(a, b); gx = gmp.powm1(a, b); ax = apm.powm1(a, b)
>>> mpm.show([fx, gx, ax])
fpm:  -5.00000012791934E-18
gmp:  -5.0000001250000041541668222916728879195 13E-18
apm:  -5.0000001250000041541668222916728880450 69e-18 (-1.168e-32%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; a = '0.99999995 + 1.5E-12j'; b = '1e-10 + 2.3E-10j'
>>> \mathrm{d}z = dec.powm1(a, b); mz = mpm.powm1(a, b); iz = ipm.powm1(a, b)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: -5.0003451250172541087E-18              - 1.1499850287492509267E-17j
mpm: -5.0003451250172909401e-18              - 1.1499850287492593979e-17j
ipm: -5.0003451250172485884e-18 (-8.47e-13%) - 1.1499850287492496570e-17 (-8.47e-13%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; a = '0.99999995 + 1.5E-12j'; b = '1e-10 + 2.3E-10j'
>>> fz = fpm.powm1(a, b); gz = gmp.powm1(a, b); az = apm.powm1(a, b)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: -5.00034512793659E-18                    - 1.14998502942070E-17j
gmp: -5.0003451250172909401E-18               - 1.1499850287492593979E-17j
apm: -5.0003451250172552059e-18 (-8.899e-13%) - 1.1499850287492511790e-17 (-8.783e-13%)j
```

3.5.7 Auxiliary function, $\mathrm{pow1p}(x, y) = (1 + x)^y$

`ctx.pow1p`(x, y)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $(1 + x)^y, x > -1$, with dbl2 arithmetic for critical values.

Returns $(1 + x)^y$. This is calculated as $\mathrm{pow1p}(x, y) = \exp(y\mathrm{logp1}(x))$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Pow1p(0.5, 3)
ereal('5.2359877559829887307E-1')
>>> math53.Pow1p(0.5, 3)
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; a = '0.000005'; b = '1e-5'
>>> \mathrm{d}x = dec.pow1p(a, b); mx = mpm.pow1p(a, b); ix = ipm.pow1p(a, b)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.0000000000499998750016666588542223955210835211422 82E-11
mpm:  1.0000000000499998750016666588542223955210835211422 82e-11
ipm:  1.0000000000499998750016666588542223955210835211422 75e-11 (2.272e-37%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; a = '0.000005'; b = '1e-5'
>>> fx = fpm.pow1p(a, b); gx = gmp.pow1p(a, b); ax = apm.pow1p(a, b)
>>> mpm.show([fx, gx, ax])
fpm:  1.00000000005000E+00
gmp:  1.0000000000499998750016666588542223955210835211422 82E-11
apm:  1.0000000000499998750016666588542223955210835211422 82e-11 (4.592e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; a = '0.000005 + 1.5E-12j'; b = '1e-5 + 2.3E-10j'
>>> \mathrm{d}z = dec.pow1p(a, b); mz = mpm.pow1p(a, b); iz = ipm.pow1p(a, b)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.00000000004999987 50E+0             + 1.16499705006820800 27E-15j
mpm: 1.00000000004999987 50e+0             + 1.16499705006820800 27e-15j
ipm: 1.00000000004999987 50e+0 (8.47e-20%) + 1.16499705006820800 26e-15 (1.653e-17%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; a = '0.000005 + 1.5E-12j'; b = '1e-5 + 2.3E-10j'
>>> fz = fpm.pow1p(a, b); gz = gmp.pow1p(a, b); az = apm.pow1p(a, b)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.00000000005000E+00                  + 1.16499705006821E-15j
gmp: 1.00000000004999987 50E+00            + 1.16499705006820800 27E-15j
apm: 1.00000000004999987 50e+0 (1.694e-19%) + 1.16499705006820800 27e-15 (5.166e-19%)j
```

3.5.8 Auxiliary function, $\mathrm{pow1pm1}(x, y) = (1 + x)^y - 1$

ctx.pow1pm1(x, y)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $(1 + x)^y - 1, x > -1$, special code for small x, y.

This is calculated as $\mathrm{pow1pm1}(x, y) = \mathrm{expm1}(y \cdot \mathrm{logp1}(x))$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Sqrt(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Sqrt('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; a = '0.000005'; b = '1e-5'
>>> \mathrm{d}x = dec.pow1pm1(a, b); mx = mpm.pow1pm1(a, b); ix = ipm.pow1pm1(a, b)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  4.9999875001666658854222395520835211422 82E-11
mpm:  4.9999875001666658854222395520835211422 82e-11
ipm:  4.9999875001666658854222395520835211422 75e-11 (2.272e-37%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; a = '0.000005'; b = '1e-5'
```

(continues on next page)

(continued from previous page)

```
>>> fx = fpm.pow1pm1(a, b); gx = gmp.pow1pm1(a, b); ax = apm.pow1pm1(a, b)
>>> mpm.show([fx, gx, ax])
fpm:  4.99998750016667E-11
gmp:  4.99998750016666588542223955208352114228 2E-11
apm:  4.99998750016666588542223955208352114228 2e-11 (5.346e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; a = '0.000005 + 1.5E-12j'; b = '1e-5 + 2.3E-10j'
>>> \mathrm{d}z = dec.pow1pm1(a, b); mz = mpm.pow1pm1(a, b); iz = ipm.pow1pm1(a, b)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 4.99998750013216605 90E-11              + 1.16499705006820800 27E-15j
mpm: 4.99998750013216605 90e-11              + 1.16499705006820800 27e-15j
ipm: 4.99998750013216605 85e-11 (1.676e-17%) + 1.16499705006820800 26e-15 (1.653e-17%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; a = '0.000005 + 1.5E-12j'; b = '1e-5 + 2.3E-10j'
>>> fz = fpm.pow1pm1(a, b); gz = gmp.pow1pm1(a, b); az = apm.pow1pm1(a, b)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 4.99998750013217E-11                    + 1.16499705006821E-15j
gmp: 4.99998750013216605 90E-11              + 1.16499705006820800 27E-15j
apm: 4.99998750013216605 89e-11 (3.944e-19%) + 1.16499705006820800 27e-15 (5.166e-19%)j
```

3.6 Trigonometric functions, in radians

For a general introduction to trigonometric functions, see Wikipedia [1247], NIST [492].

3.6.1 Sine, $\sin(x)$

`ctx.sin(`x`)`

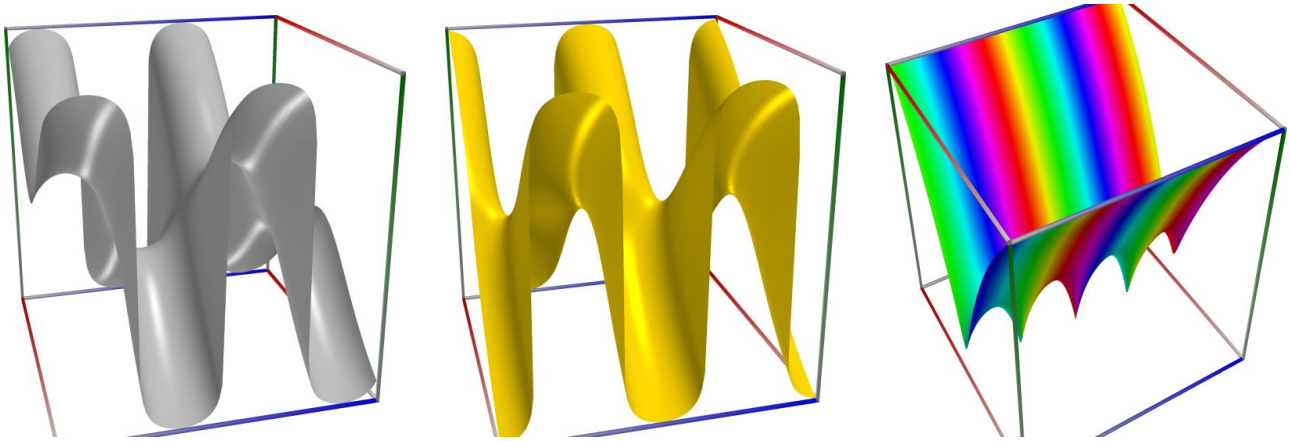
where `ctx` is `ctx_pm` (see *Python contexts* for details), `ctx53`, `ctxcpp`, `ctxflint` (see *.NET contexts* for details). The corresponding `ctx` python lists are `ctxlistreal` and `ctxlistcplx`.

Returns the sine of x, $\sin(x)$. See also Wikipedia [1244], MathWorld [874], NIST [492], Ehrhardt [297] (4.2.55), Flint [746], Flint [736], Mpmath [557].

The sine can be defined as $\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}$.

The complex sine can be expressed in terms of related real functions:

$$\sin(z) = \sin(x+iy) = \frac{e^{iz} - e^{-iz}}{2i} = \sin(x)\cosh(y) + i\cos(x)\sinh(y).$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x+iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example with real input:

```
>>> from xlcalcnet import *
>>> x = -5.1; dps = 90
>>> for ctx in ctxlistreal: print(ctx.fmtname + ': ' + ctx.fmt(ctx.sin(x)))
    fpm:  0.925814682327732
    mpm:  0.92581468232773229694614624754486331250940301635560394879705455193228184723656688943690637 5
    dpm:  0.92581468232773229694614624754486331250940301635560394879705455193228184723656688943690637 5
    ipm: [0.92581468232773229694614624754486331250940301635560394879705455193228184723656688943690637503659,
          0.92581468232773229694614624754486331250940301635560394879705455193228184723656688943690637755275]
    gpm:  0.92581468232773229694614624754486331250940301635560394879705455193228184723656688943690637504
    apm: [0.92581468232773229694614624754486331250940301635560394879705455193228184723656688943690638 +/- 5.10e-90]
 math53:  0.925814682327732
  sreal:  0.9258147
  dreal:  0.925814682327732
  ereal:  0.92581468232773229699
  qreal:  0.92581468232773229694614624754486 3
  oreal:  0.92581468232773229694614624754486331250940301635560394879705455193228186
  mreal:  0.92581468232773229694614624754486331250940301635560394879705455193228184723656688943690637 6
 sflint:  0.9258147
```

(continues on next page)

(continued from previous page)

```
 dflint:  0.925814682327732
 eflint:  0.92581468232773229699
 qflint:  0.925814682327732296946146247544863
 oflint:  0.9258146823277322969461462475448633125094030163556039487970545519322818472365668894369063 76
 mflint:  0.9258146823277322969461462475448633125094030163556039487970545519322818472365668894369063 76
 aflint: [0.9258146823277322969461462475448633125094030163556039487970545519322818472365668894369063 75 +/- 9.57e-91]
```

An example with complex input, using C#-style formatting for complex numbers for the result

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(50)
>>> for ctx in [fpm, mpm, cmath53, qcplx]: print(ctx.fmtname + ': ' + ctx.fmt(ctx.sin(x)))
    fpm:  (3.48309600859536, 1.37087251009309)
    mpm:  (3.4830960085953555305395191320496018742602081571329906610356518811477911668012326827103078133, 1.3708725100930962119067139658774871514235754109441)
cmath53:  (3.48309600859536, 1.37087251009309)
  qcplx:  (3.4830960085953555305395191320496, 1.370872510093096211906713965877487151423575410944129163310605310528913627609479426503523 6)
```

An example with the same complex input as above, showing only the real part of the result:

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(90)
>>> for ctx in gui.ctxlistcplx: print(ctx.fmtname + ': ' +  ctx.realctx.fmt(ctx.real(ctx.sin(x))))
    fpm:  3.48309600859536
    mpm:  3.4830960085953555305395191320496018742602081571329906610356518811477911668012326827103078 1
    dpm:  3.4830960085953555305395191320496018742602081571329906610356518811477911668012326827103078 1
    ipm: [3.4830960085953555305395191320496018742602081571329906610356518811477911668012326827103078127848,
          3.4830960085953555305395191320496018742602081571329906610356518811477911668012326827103078147484]
    gpm:  3.4830960085953555305395191320496018742602081571329906610356518811477911668012326827103078133
    apm: [3.4830960085953555305395191320496018742602081571329906610356518811477911668012326827103078 1 +/- 6.64e-90]
cmath53:  3.48309600859536
  scplx:  3.483096
  dcplx:  3.48309600859536
  ecplx:  3.4830960085953555309
  qcplx:  3.483096008595355530539519132049601874260208157132990661035651881147791166801232682710307 81
  ocplx:  3.4830960085953555305395191320496018742602081571329906610356518811477912
  mcplx:  3.483096008595355530539519132049601874260208157132990661035651881147791166801232682710307 82
sflintc:  3.483096
dflintc:  3.48309600859536
eflintc:  3.4830960085953555307
qflintc:  3.483096008595355530539519132049 6
oflintc:  3.4830960085953555305395191320496018742602081571329906610356518811477912
mflintc:  3.483096008595355530539519132049601874260208157132990661035651881147791166801232682710307 82
aflintc: [3.483096008595355530539519132049601874260208157132990661035651881147791166801232682710307 81 +/- 8.22e-90]
```

An example with the same complex input as above, showing only the imaginary part of the result:

```
>>> from xlcalcnet import *
>>> x = -5.1+2j; gui.setdps(90)
>>> for ctx in gui.ctxlistcplx: print(ctx.fmtname + ': ' +  ctx.realctx.fmt(ctx.imag(ctx.sin(x))))
    fpm:  1.37087251009309
    mpm:  1.3708725100930962119067139658774871514235754109441291633106053105289136276094794265035223 6
    dpm:  1.3708725100930962119067139658774871514235754109441291633106053105289136276094794265035223 6
    ipm: [1.3708725100930962119067139658774871514235754109441291633106053105289136276094794265035223541435,
          1.3708725100930962119067139658774871514235754109441291633106053105289136276094794265035223575799]
    gpm:  1.3708725100930962119067139658774871514235754109441291633106053105289136276094794265035223576
    apm: [1.3708725100930962119067139658774871514235754109441291633106053105289136276094794265035223 5 +/- 7.68e-90]
cmath53:  1.37087251009309
  scplx:  1.370872
  dcplx:  1.37087251009309
  ecplx:  1.3708725100930962117
  qcplx:  1.370872510093096211906713965877487151423575410944129163310605310528913627609479426503522 35
  ocplx:  1.3708725100930962119067139658774871514235754109441291633106053105289136
  mcplx:  1.370872510093096211906713965877487151423575410944129163310605310528913627609479426503522 35
sflintc:  1.370872
```

(continues on next page)

(continued from previous page)

```
dflintc:  1.37087251009309
eflintc:  1.3708725100930962116
qflintc:  1.370872510093096211906713965877487151423575410944129163310605310528913627609479426503522 35
oflintc:  1.370872510093096211906713965877487151423575410944129163310605310528913627609479426503522 35
mflintc:  1.370872510093096211906713965877487151423575410944129163310605310528913627609479426503522 35
aflintc: [1.370872510093096211906713965877487151423575410944129163310605310528913627609479426503522 35 +/- 7.29e-90]
```

3.6.2 Cosine, $\cos(x)$

`ctx.cos(`x`)`

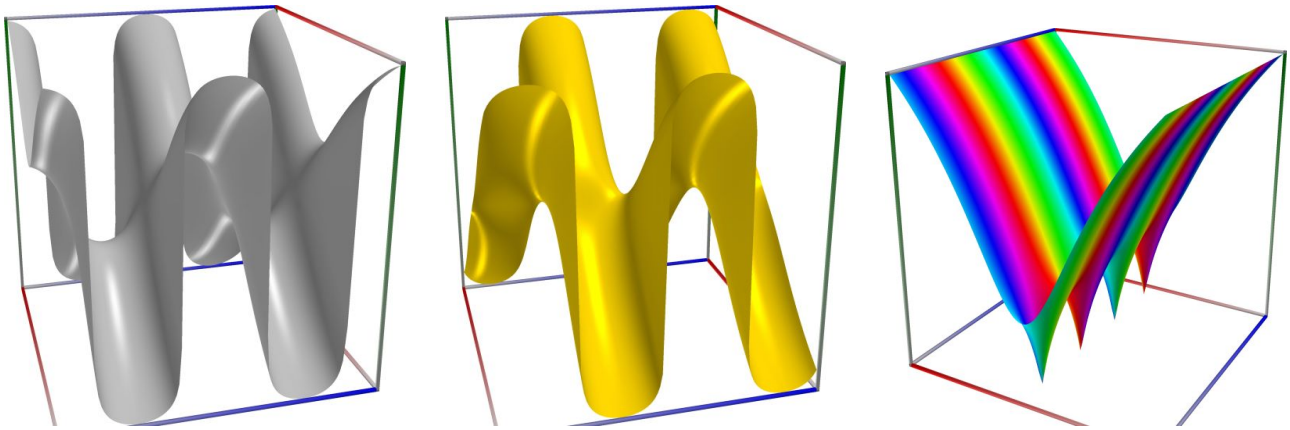
where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the cosine of x, $\cos(x)$. See also Wikipedia [1247], MathWorld [859], NIST [492], Ehrhardt [297] (4.2.19), Flint [746], Flint [736], Mpmath [541].

The cosine can be defined as $\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n}$.

The complex cosine can be expressed in terms of related real functions:

$$\cos(z) = \cos(x + iy) = \frac{e^{ix} + e^{-ix}}{2} = \cos(x)\cosh(y) - i\sin(x)\sinh(y).$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Cos(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Cos('0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> \mathrm{d}x = dec.cos(x); mx = mpm.cos(x); ix = ipm.cos(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  6.619231321691639751442098584651351473054E-15
mpm:  6.619231321691639751442098575708073666164e-15
ipm:  6.619231321691639751442098587187510685913e-15 (1.734e-25%)
```

(continues on next page)

(continued from previous page)

```
>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> fx = fpm.cos(x); gx = gmp.cos(x); ax = apm.cos(x)
>>> mpm.show([fx, gx, ax])
fpm:  6.72257048770831E-15
gmp:  6.619231321691639751442098575708073666164E-15
apm:  6.619231321691639751442098584676383837843e-15 (1.768e-25%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '3.14159265358979 + 1.5E+2j'
>>> \mathrm{d}z = dec.cos(z); mz = mpm.cos(z); iz = ipm.cos(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: -6.9685479033318984866E+64              - 2.2567382063567230055E+50j
mpm: -6.9685479033318984866e+64              - 2.2567375654278714043e+50j
ipm: -6.9685479033318984866e+64 (-6.4e-20%) - 2.2567387459458051327e+50 (-5.231e-5%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '3.14159265358979 + 1.5E+2j'
>>> fz = fpm.cos(z); gz = gmp.cos(z); az = apm.cos(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: -6.96854790333190E+64                   - 2.25159995138029E+50j
gmp: -6.9685479033318984866E+64              - 2.2567375654278714043E+50j
apm: -6.9685479033318984866e+64 (-6.4e-20%) - 2.2567382294692091265e+50 (-5.406e-5%)j
```

3.6.3 Tangent, $\tan(x)$

`ctx.tan(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

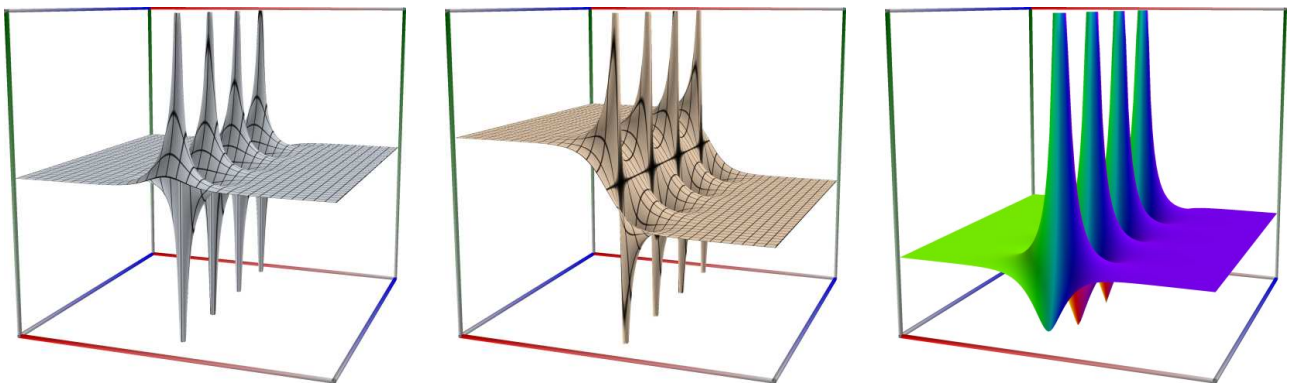
Returns the tangent of x, $\tan(x)$. See also Wikipedia [1247], MathWorld [875], NIST [492], Ehrhardt [297] (4.2.61), Flint [746], Flint [736], Mpmath [580].

The tangent can be defined in terms of related functions as $\tan(x) = \dfrac{\sin(x)}{\cos(x)}$.

The tangent function is singular at $x = (n + 1/2)\pi$, but `tan(x)` always returns a finite result since $(n + 1/2)\pi$ cannot be represented exactly using floating-point arithmetic.

The complex tangent can be expressed in terms of related real functions:

$$\tan(z) = \tan(x + iy) = \frac{\sin(z)}{\cos(z)} = \frac{\sin(2x) + i\sinh(2y)}{\cos(2x) + i\cosh(2y)}$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Tan(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Tan('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> \mathrm{d}x = dec.tan(x); mx = mpm.tan(x); ix = ipm.tan(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.5107494381151731974847433363776779549 45E+14
mpm:  1.5107494381151731974847433384185936484 5e+14
ipm:  1.5107494381151731974847433357988341406 73e+14 (1.734e-25%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> fx = fpm.tan(x); gx = gmp.tan(x); ax = apm.tan(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.48752623989354E+14
gmp:  1.5107494381151731974847433384185936484 5E+14
apm:  1.5107494381151731974847433363719646550 59e+14 (1.767e-25%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.57079632679489 + 0.001j'
>>> \mathrm{d}z = dec.tan(z); mz = mpm.tan(z); iz = ipm.tan(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 6.6192291152816404696E-9            + 1.0000003333333111111E+3j
mpm: 6.6192285676675710077e-9            + 1.0000003333333111111e+3j
ipm: 6.6192294147002359176e-9 (1.28e-5%) + 1.0000003333333111111e+3 (6.505e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.57079632679489 + 0.001j'
>>> fz = fpm.tan(z); gz = gmp.tan(z); az = apm.tan(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 6.72256824685193E-09                + 1.00000033333331E+03j
gmp: 6.6192285676675710076E-09           + 1.0000003333333111111E+03j
apm: 6.6192291235327573548e-9 (1.32e-5%) + 1.0000003333333111111e+3 (1.301e-19%)j
```

From mpmath:

```
>>> from xlcalcnet import  mp
>>> iv.tan([0,2])  # Interval includes a singularity
[-inf, +inf]
```

3.6.4 Cotangent, $\cot(x)$

`ctx.cot(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

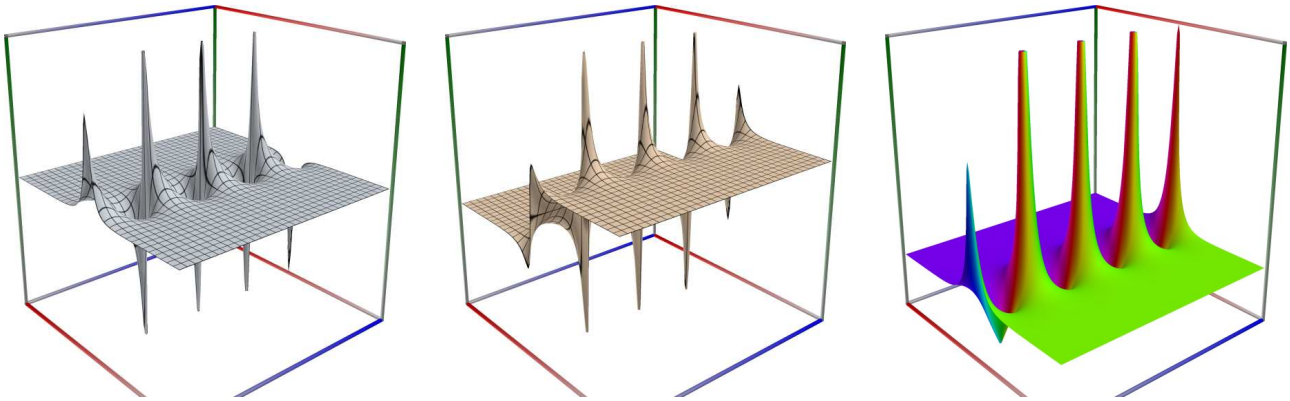
Returns the cotangent of x, $\cot(x)$. See also Wikipedia [1247], MathWorld [860], NIST [492], Ehrhardt [297] (4.2.21), Flint [746], Flint [736], Mpmath [560].

The cotangent can be expressed in terms of related functions as $\cot(x) = \dfrac{\cos(x)}{\sin(x)}$.

This cotangent function is singular at $x = n\pi$, but with the exception of the point $x = 0$, `cot(x)` returns a finite result since $n\pi$ cannot be represented exactly using floating-point arithmetic.

The complex cotangent can be expressed in terms of related real functions:

$$\cot(z) = \cot(x + iy) = \frac{\cos(z)}{\sin(z)} = \frac{\cos(2x) + i\cosh(2y)}{\sin(2x) + i\sinh(2y)}$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Cot(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Cot('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> \mathrm{d}x = dec.cot(x); mx = mpm.cot(x); ix = ipm.cot(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  6.619231321691639751442098584796359712623E-15
mpm:  6.619231321691639751442098575853081905733e-15
ipm:  6.619231321691639751442098587332518925482e-15 (1.734e-25%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> fx = fpm.cot(x); gx = gmp.cot(x); ax = apm.cot(x)
>>> mpm.show([fx, gx, ax])
fpm:  6.72257048770831E-15
gmp:  6.619231321691639751442098575853081905733E-15
apm:  6.619231321691639751442098584821392051071e-15 (1.768e-25%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.57079632679489 + 0.001j'
>>> \mathrm{d}z = dec.cot(z); mz = mpm.cot(z); iz = ipm.cot(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 6.619224702464730878 2E-15             - 9.999996666667999999 5E-4j
mpm: 6.619224154851026492 1e-15             - 9.999996666667999999 5e-4j
ipm: 6.619225001883126714 0e-15 (1.28e-5%) - 9.999996666667999999 5e-4 (-4.963e-19%)j
```

(continues on next page)

(continued from previous page)

```
>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.57079632679489 + 0.001j'
>>> fz = fpm.cot(z); gz = gmp.cot(z); az = apm.cot(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 6.72256376514230E-15                 - 9.99999666666800E-04j
gmp: 6.619224154851026492 1E-15           - 9.99999666666799999995E-04j
apm: 6.6192247107158422627e-15 (1.32e-5%) - 9.9999966666679999995e-4 (-1.654e-19%)j
```

3.6.5 Cosecant, $\csc(x)$

`ctx.csc(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

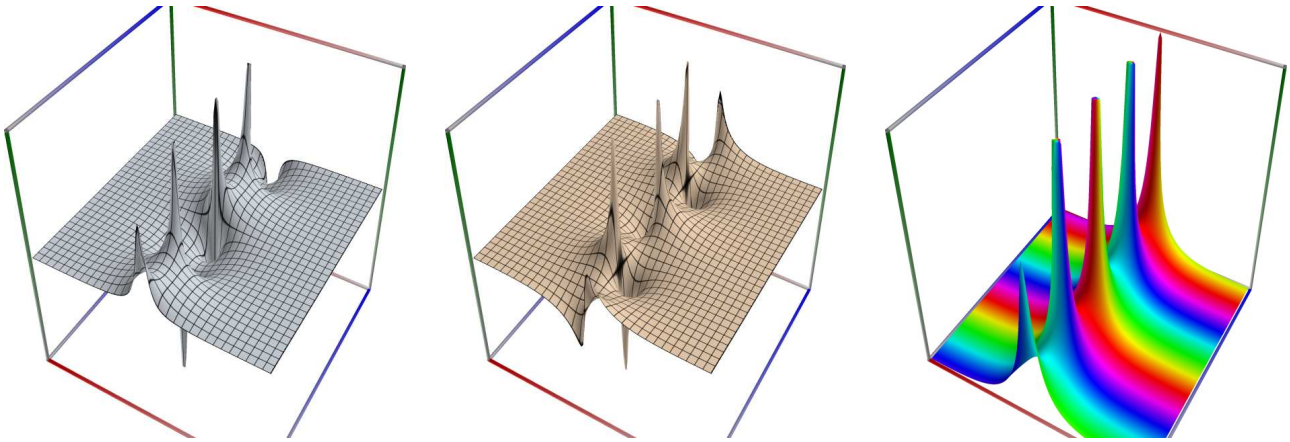
Returns the cosecant of x, $\csc(x)$. See also Wikipedia [1247], MathWorld [858], NIST [492], Ehrhardt [297] (4.2.23), Flint [746], Flint [736], Mpmath [559].

The cosecant can be expressed in terms of related functions as $\csc(x) = \dfrac{1}{\sin(x)}$.

The cosecant function is singular at $x = n\pi$, but with the exception of the point $x = 0$, `csc(x)` returns a finite result since $n\pi$ cannot be represented exactly using floating-point arithmetic.

The complex cosecant can be expressed in terms of related real functions:

$$\csc(z) = \csc(x + iy) = \frac{1}{\sin(z)} = \frac{2i}{e^{ix} - e^{-ix}} = \frac{1}{\sin(x)\cosh(y) + i\cos(x)\sinh(y)}.$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Csc(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Csc('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '3.14159265358979'
>>> \mathrm{d}x = dec.csc(x); mx = mpm.csc(x); ix = ipm.csc(x)
```

(continues on next page)

(continued from previous page)

```
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  3.0878849322012935747865859567471663097 34E+14
mpm:  3.0878849322012935747865859406877951547 00e+14
ipm:  3.0878849322012935747865859625791581295 74e+14 (7.089e-25%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '3.14159265358979'
>>> fx = fpm.csc(x); gx = gmp.csc(x); ax = apm.csc(x)
>>> mpm.show([fx, gx, ax])
fpm:  3.09493162808962E+14
gmp:  3.0878849322012935747865859406877951547 00E+14
apm:  3.0878849322012935747865859564221223489 0e+14 (7.222e-25%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.57079632679489 + 1.5E+2j'
>>> \mathrm{d}z = dec.csc(z); mz = mpm.csc(z); iz = ipm.csc(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.4350191946328820840E-65               - 9.4987240003426845058E-80j
mpm: 1.4350191946328820840e-65               - 9.4987232145057216312e-80j
ipm: 1.4350191946328820840e-65 (5.605e-20%) - 9.4987244300142594276e-80 (-1.28e-5%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.57079632679489 + 1.5E+2j'
>>> fz = fpm.csc(z); gz = gmp.csc(z); az = apm.csc(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.43501919463288E-65                    - 9.64701768713396E-80j
gmp: 1.4350191946328820840E-65               - 9.4987232145057216313E-80j
apm: 1.4350191946328820840e-65 (5.605e-20%) - 9.4987240121831995601e-80 (-1.326e-5%)j
```

3.6.6 Secant, $\sec(x)$

`ctx.sec(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

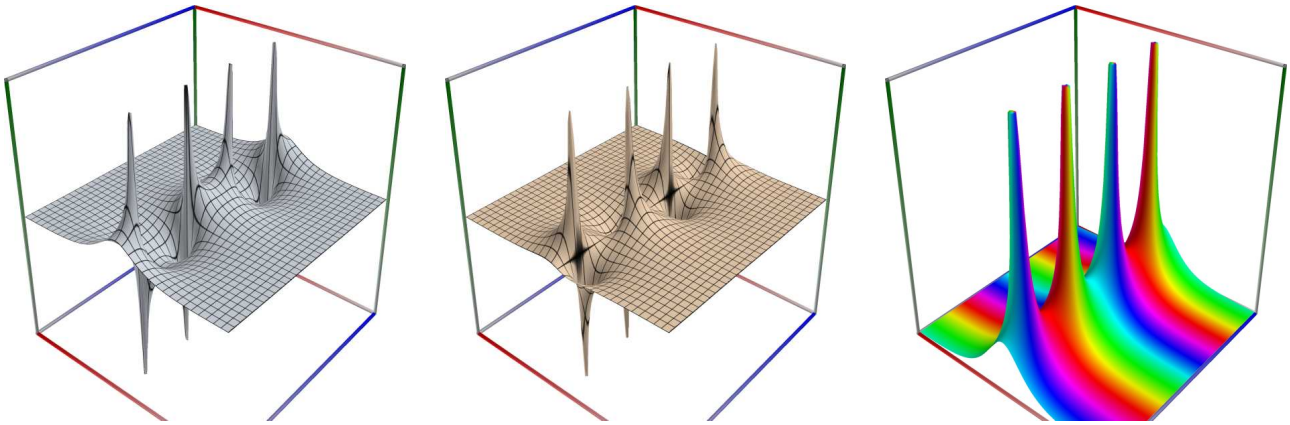
Returns the secant of x, $\sec(x)$. See also Wikipedia [1247], MathWorld [873], NIST [492], Ehrhardt [297] (4.2.53), Flint [746], Flint [736], Mpmath [579].

The secant can be expressed in terms of related functions as $\sec(x) = \dfrac{1}{\cos(x)}$.

The secant function is singular at $x = (n + 1/2)\pi$, but `sec(x)` always returns a finite result since $(n + 1/2)\pi$ cannot be represented exactly using floating-point arithmetic.

The complex secant can be expressed in terms of related real functions:

$$\sec(z) = \sec(x + iy) = \frac{1}{\cos(z)} = \frac{2}{e^{ix} + e^{-ix}} = \frac{1}{\cos(x)\cosh(y) - i\sin(x)\sinh(y)}.$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Sec(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Sec('0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> \mathrm{d}x = dec.sec(x); mx = mpm.sec(x); ix = ipm.sec(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.5107494381151731974847433364107741115 54E+14
mpm:  1.5107494381151731974847433338451955521454e+14
ipm:  1.5107494381151731974847433358319302972 81e+14 (1.734e-25%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> fx = fpm.sec(x); gx = gmp.sec(x); ax = apm.sec(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.48752623989354E+14
gmp:  1.5107494381151731974847433384519555214 54E+14
apm:  1.5107494381151731974847433364050608072 89e+14 (1.767e-25%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '3.14159265358979 + 1.5E+2j'
>>> \mathrm{d}z = dec.sec(z); mz = mpm.sec(z); iz = ipm.sec(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: -1.4350191946328820840E-65               + 4.6472560543565481719E-80j
mpm: -1.4350191946328820840e-65               + 4.6472547345045256263e-80j
ipm: -1.4350191946328820840e-65 (-5.605e-20%) + 4.6472571655216012191e-80 (5.231e-5%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '3.14159265358979 + 1.5E+2j'
>>> fz = fpm.sec(z); gz = gmp.sec(z); az = apm.sec(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: -1.435019194632 88E-65                   + 4.636674948191 55E-80j
gmp: -1.4350191946328820840E-65               + 4.6472547345045256263E-80j
apm: -1.4350191946328820840e-65 (-5.605e-20%) + 4.6472561019516306473e-80 (5.401e-5%)j
```

3.6.7 Cardinal sine, $\mathrm{sinc}(x)$

`ctx.sinc`(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the cardinal sine of x. See also Wikipedia [1292], MathWorld [934], BoostMath [127], Flint [746], Flint [736], Mpmath [603].

`sinc(x)` computes the unnormalized sinc function, defined as $\mathrm{sinc}(x) = \begin{cases} \sin(x)/x, & \text{if } x \neq 0 \\ 1, & \text{if } x = 0. \end{cases}$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Sinc(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Sinc('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '0.0000001'
>>> \mathrm{d}x = dec.sinc(x); mx = mpm.sinc(x); ix = ipm.sinc(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  9.99999999999998333333333333333341666666667E-1
mpm:  9.99999999999998333333333333333341666666667e-1
ipm:  9.99999999999998333333333333333341666666667e-1 (2.87e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.0000001'
>>> fx = fpm.sinc(x); gx = gmp.sinc(x); ax = apm.sinc(x)
>>> mpm.show([fx, gx, ax])
fpm:  9.99999999999998E-01
gmp:  9.99999999999998333333333333333341666666667E-01
apm:  9.99999999999998333333333333333341666666667e-1 (5.74e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '0.0000001 + 0.001j'
>>> \mathrm{d}z = dec.sinc(z); mz = mpm.sinc(z); iz = ipm.sinc(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.0000001666666733333E+0              - 3.333333666666752381E-11j
mpm: 1.0000001666666733333e+0              - 3.333333666666752360e-11j
ipm: 1.0000001666666733333e+0 (5.082e-19%) - 3.333333666666640784e-11 (-1.199e-12%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '0.0000001 + 0.001j'
>>> fz = fpm.sinc(z); gz = gmp.sinc(z); az = apm.sinc(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.00000016666667E+00                - 3.33333366590436E-11j
gmp: 1.0000001666666733333E+00           - 3.333333666666773989E-11j
apm: 1.0000001666666733333e+0 (8.47e-20%) - 3.333333666666751169e-11 (-2.592e-12%)j
```

3.7 Trigonometric functions, in multiples of π

3.7.1 Auxiliary function $\mathrm{sinpi}(x) = \sin(\pi x)$

`ctx.sinpi(x)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the sine of $x \cdot \pi$. See also BoostMath [111], Ehrhardt [297] (4.2.57), Flint [746], Flint [736], Mpmath [556].

The function can also be expressed as

$$\sin(\pi z) = \frac{\pi}{\Gamma(z)\Gamma(1-z)}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SinPi(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.SinPi('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1'
>>> \mathrm{d}x = dec.sinpi(x); mx = mpm.sinpi(x); ix = ipm.sinpi(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.693993751058209749445923078164062862090E-40
mpm:  0.0e+0
ipm:  1.611255246984498999948306365093859981582e-40 (14.25%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1'
>>> fx = fpm.sinpi(x); gx = gmp.sinpi(x); ax = apm.sinpi(x)
>>> mpm.show([fx, gx, ax])
fpm:  -0.000000000000000E+00
gmp:  4.134064219652797647299380610320968938829E-43
apm:  0.0e+0 (0.0%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1 + 0.001j'
>>> \mathrm{d}z = dec.sinpi(z); mz = mpm.sinpi(z); iz = ipm.sinpi(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: -3.7356801068163132819E-20           - 3.1415978213051234531E-3j
mpm: 0.0e+0                               - 3.1415978213051234531e-3j
ipm: -3.8307303903313326019e-20 (-4.422%) - 3.1415978213051234531e-3 (-2.633e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1 + 0.001j'
>>> fz = fpm.sinpi(z); gz = gmp.sinpi(z); az = apm.sinpi(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: -0.000000000000000E+00     - 3.14159782130512E-03j
gmp: 6.564039477850253644 3E-22 - 3.1415978213051234531E-03j
apm: 0.0e+0 (0.0%)              - 3.1415978213051234531e-3 (-1.58e-19%)j
```

3.7.2 Auxiliary function $\mathrm{cospi}(x) = \cos(\pi x)$

`ctx.cospi(x)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns $\cos(\pi x)$. See also BoostMath [98], Flint [746], Flint [736], Mpmath [540].

The function is calculated as

$$\cos(\pi z) = \frac{\pi}{\Gamma\left(\frac{1}{2} + z\right)\Gamma\left(\frac{1}{2} - z\right)}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CosPi(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.CosPi('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '0.5'
>>> \mathrm{d}x = dec.cospi(x); mx = mpm.cospi(x); ix = ipm.cospi(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  5.8469968755291048747229615390820314310 45E-40
mpm:  6.3661977236758134307553505349005744813 78e-1
ipm:  8.0562762349224949997415318254692999079 08e-41 (14.25%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.5'
>>> fx = fpm.cospi(x); gx = gmp.cospi(x); ax = apm.cospi(x)
>>> mpm.show([fx, gx, ax])
fpm:  -0.0000000000000000E+00
gmp:  2.0670321098263988236496903051604844694 15E-43
apm:  0.0e+0 (0.0%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '0.5 + 0.001j'
>>> \mathrm{d}z = dec.cospi(z); mz = mpm.cospi(z); iz = ipm.cospi(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 3.1321846206231396497E-20             - 3.1415978213051234531E-3j
mpm: 6.3662036748134886471e-1              - 1.2732407349626977294e-3j
ipm: -1.9153651951656663009e-20 (-4.422%) - 3.1415978213051234531e-3 (-2.633e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '0.5 + 0.001j'
>>> fz = fpm.cospi(z); gz = gmp.cospi(z); az = apm.cospi(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: -0.0000000000000000E+00     - 3.14159782130512E-03j
gmp: 3.2820197389251268222E-22 - 3.1415978213051234531E-03j
apm: 0.0e+0 (0.0%)             - 3.1415978213051234531e-3 (-1.58e-19%)j
```

3.7.3 Auxiliary function $\mathrm{tanpi}(x) = \tan(\pi x)$

`ctx.tanpi(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns $\tan(\pi x)$. See also BoostMath [98], Flint [746], Flint [736].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.TanPi(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.TanPi('0.51')
ereal('5.351847902755998475 4E-1')
```

3.7.4 Auxiliary function $\mathrm{cotpi}(x) = \cot(\pi x)$

`ctx.cotpi(`x`)`

Returns $\cot(\pi x)$. See also BoostMath [98], Flint [746], Flint [736].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.TanPi(0.5)
ereal('5.23598775598298873 07E-1')
>>> math53.TanPi('0.51')
ereal('5.35184790275599847 54E-1')
```

3.7.5 Auxiliary function $\mathrm{cscpi}(x) = \csc(\pi x)$

`ctx.cscpi(`x`)`

Returns $\mathrm{cscpi}(x)$. See also BoostMath [98], Flint [746], Flint [736].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.TanPi(0.5)
ereal('5.23598775598298873 07E-1')
>>> math53.TanPi('0.51')
ereal('5.35184790275599847 54E-1')
```

3.7.6 Auxiliary function $\mathrm{secpi}(x) = \sec(\pi x)$

`ctx.secpi(`x`)`

Returns $\mathrm{secpi}(x)$. See also BoostMath [98], Flint [746], Flint [736].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.TanPi(0.5)
ereal('5.23598775598298873 07E-1')
>>> math53.TanPi('0.51')
ereal('5.35184790275599847 54E-1')
```

3.7.7 Auxiliary function $\mathrm{sincpi}(x) = \mathrm{sinc}(\pi x)$

`ctx.sincpi(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the cardinal sine of x. See also Wikipedia [1292], MathWorld [934], BoostMath [127], Flint [746], Flint [736], Mpmath [538].

`sincpi(x)` computes the normalized sinc function, defined as $\mathrm{sinc}_\pi(x) = \begin{cases} \sin(\pi x)/(\pi x), & \text{if } x \neq 0 \\ 1, & \text{if } x = 0. \end{cases}$

We also have

$$\mathrm{sinc}(\pi z) = \frac{1}{\Gamma(1+z)\,\Gamma(1-z)}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SincPi(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.SincPi('0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1'
>>> \mathrm{d}x = dec.sincpi(x); mx = mpm.sincpi(x); ix = ipm.sincpi(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  5.3921495809539137379799049791268736155 91E-41
mpm:  0.0e+0
ipm:  5.1287847428067140430269195189123357055 33e-41 (14.25%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1'
>>> fx = fpm.sincpi(x); gx = gmp.sincpi(x); ax = apm.sincpi(x)
>>> mpm.show([fx, gx, ax])
fpm:  -0.00000000000000E+00
gmp:  1.3159135112341634178031786073740547071 37E-43
apm:  0.0e+0 (0.0%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1 + 0.001j'
>>> \mathrm{d}z = dec.sincpi(z); mz = mpm.sincpi(z); iz = ipm.sincpi(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: -1.0000006449342455476E-6                - 1.000000644934233656 6E-3j
mpm: -1.0000006449342336566e-6                - 1.000000644934233656 6e-3j
ipm: -1.0000006449342458502e-6 (-5.392e-14%) - 1.000000644934233656 6e-3 (-5.79e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1 + 0.001j'
>>> fz = fpm.sincpi(z); gz = gmp.sincpi(z); az = apm.sincpi(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: -1.00000064493423E-06                    - 1.00000064493423E-03j
gmp: -1.0000006449342334477E-06               - 1.000000644934233656 6E-03j
apm: -1.0000006449342336566e-6 (-4.362e-18%) - 1.000000644934233656 6e-3 (-4.301e-18%)j
```

3.8 Hyperbolic functions

For a general introduction into hyperbolic functions, see Wikipedia [1238], NIST [485].

3.8.1 Hyperbolic sine, $\sinh(x)$

`ctx.sinh(x)`

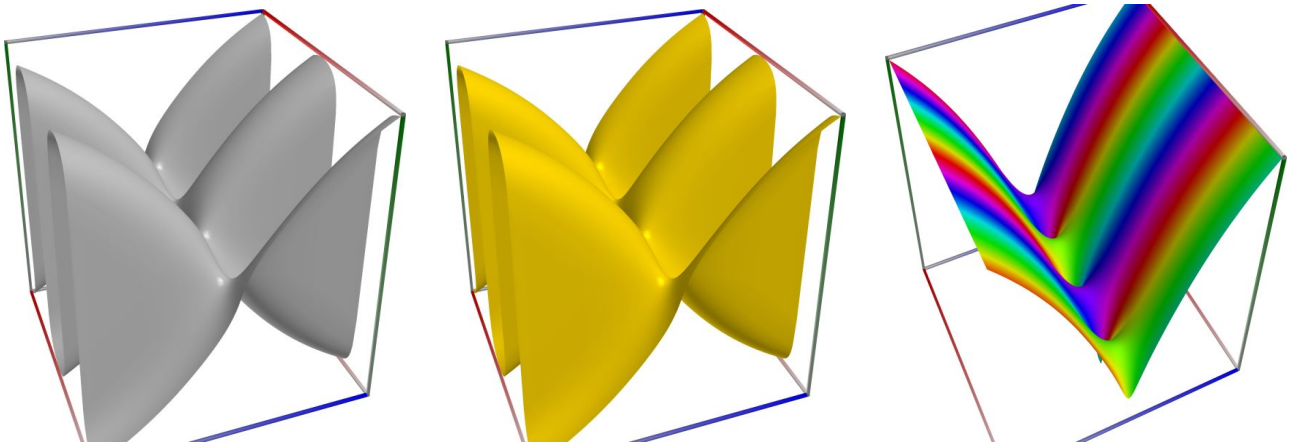
where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the hyperbolic sine of x, $\sinh(x)$. See also Wikipedia [1238], MathWorld [865], NIST [485], Ehrhardt [297] (4.2.56), Flint [740], Flint [730], Mpmath [565].

The hyperbolic sine can be expressed in terms of the exponential function as $\sinh(x) = \dfrac{e^x - e^{-x}}{2} = \dfrac{e^{2x-1}}{2e^x} = \dfrac{1 - e^{-2x}}{2e^x}$.

The complex hyperbolic sine can be expressed in terms of related functions:

$$\sinh(z) = \frac{e^z - e^{-z}}{2} = \frac{e^{2z-1}}{2e^z} = \frac{1 - e^{-2z}}{2e^z} = -i\sin(iz)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Sinh(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Sinh('0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '3.14159265358979'
>>> \mathrm{d}x = dec.sinh(x); mx = mpm.sinh(x); ix = ipm.sinh(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.1548739357257710837869687694559984769 91E+1
mpm:  1.1548739357257710837869687694559984769 91e+1
ipm:  1.1548739357257710837869687694559984769 91e+1 (3.181e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '3.14159265358979'
>>> fx = fpm.sinh(x); gx = gmp.sinh(x); ax = apm.sinh(x)
```

(continues on next page)

(continued from previous page)

```
>>> mpm.show([fx, gx, ax])
fpm:  1.15487393572577E+01
gmp:  1.154873935725771083786968769455998476991E+01
apm:  1.154873935725771083786968769455998476991e+1 (3.181e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.5E+2 - 1.57079632679489j'
>>> \mathrm{d}z = dec.sinh(z); mz = mpm.sinh(z); iz = ipm.sinh(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 4.6126430548443107461E+50             - 6.9685479033318984866E+64j
mpm: 4.6126426732366959851e+50             - 6.9685479033318984866e+64j
ipm: 4.6126432634956628493e+50 (1.28e-5%) - 6.9685479033318984866e+64 (-6.4e-20%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.5E+2 - 1.57079632679489j'
>>> fz = fpm.sinh(z); gz = gmp.sinh(z); az = apm.sinh(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 4.68465544771206E+50                  - 6.96854790333190E+64j
gmp: 4.6126426732366959851E+50             - 6.9685479033318984866E+64j
apm: 4.6126430605941429898e+50 (1.32e-5%) - 6.9685479033318984866e+64 (-6.4e-20%)j
```

From mpmath:

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> sinh(2+3j)
(-3.59056458998577995201256 5 + 0.530921086248519805267040 1j)
>>> j*sin(3-2j)
(-3.59056458998577995201256 5 + 0.530921086248519805267040 1j)
```

3.8.2 Hyperbolic cosine, $\cosh(x)$

`ctx.cosh`(x)

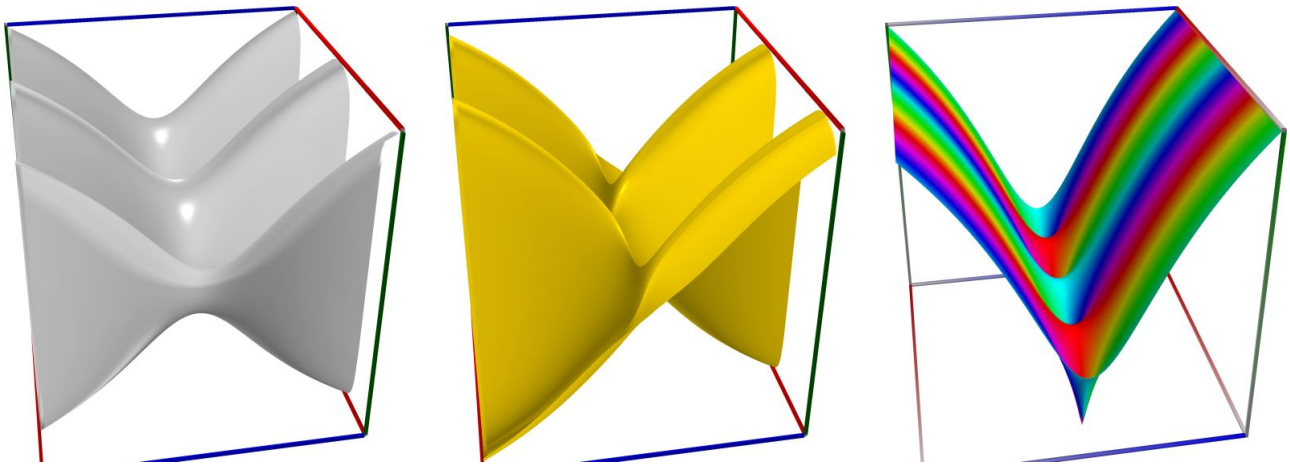
where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the hyperbolic cosine of x, $\cosh(x)$. See also Wikipedia [1238], MathWorld [862], NIST [485], Ehrhardt [297] (4.2.20), Flint [740], Flint [730], Mpmath [562].

The hyperbolic cosine can be expressed in terms of the exponential function as $\cosh(x) = \dfrac{e^x + e^{-x}}{2} = \dfrac{e^{2x+1}}{2e^x} = \dfrac{1 + e^{-2x}}{2e^x}$.

The complex hyperbolic cosine can be expressed in terms of related functions:

$$\cosh(z) = \frac{e^z + e^{-z}}{2} = \frac{e^{2z+1}}{2e^z} = \frac{1 + e^{-2z}}{2e^z} = \cos(iz)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Cosh(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Cosh('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> \mathrm{d}x = dec.cosh(x); mx = mpm.cosh(x); ix = ipm.cosh(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  6.619231321691639751442098584651351473054E-15
mpm:  6.619231321691639751442098575708073666164e-15
ipm:  6.619231321691639751442098587187510685913e-15 (1.734e-25%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> fx = fpm.cosh(x); gx = gmp.cosh(x); ax = apm.cosh(x)
>>> mpm.show([fx, gx, ax])
fpm:  6.72257048770831E-15
gmp:  6.619231321691639751442098575708073666164E-15
apm:  6.619231321691639751442098584676383837843e-15 (1.768e-25%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.5E+2 - 3.14159265358979j'
>>> \mathrm{d}z = dec.cosh(z); mz = mpm.cosh(z); iz = ipm.cosh(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: -6.9685479033318984866E+64              - 2.2567382063567230055E+50j
mpm: -6.9685479033318984866e+64              - 2.2567375654278714043e+50j
ipm: -6.9685479033318984866e+64 (-6.4e-20%) - 2.2567387459458051327e+50 (-5.231e-5%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.5E+2 - 3.14159265358979j'
>>> fz = fpm.cosh(z); gz = gmp.cosh(z); az = apm.cosh(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: -6.96854790333190E+64                   - 2.25159995138029E+50j
gmp: -6.9685479033318984866E+64              - 2.2567375654278714043E+50j
apm: -6.9685479033318984866e+64 (-6.4e-20%) - 2.2567382294692091265e+50 (-5.406e-5%)j
```

Generalized to complex numbers, the hyperbolic cosine is equivalent to a cosine with the argument rotated in the imaginary direction, or $\cosh x = \cos ix$:

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> cosh(2+3j)
(-3.724545504915322565473971 + 0.5118225699873846088344638j)
>>> cos(3-2j)
(-3.724545504915322565473971 + 0.5118225699873846088344638j)
```

3.8.3 Hyperbolic tangent, $\tanh(x)$

`ctx.tanh(`x`)`

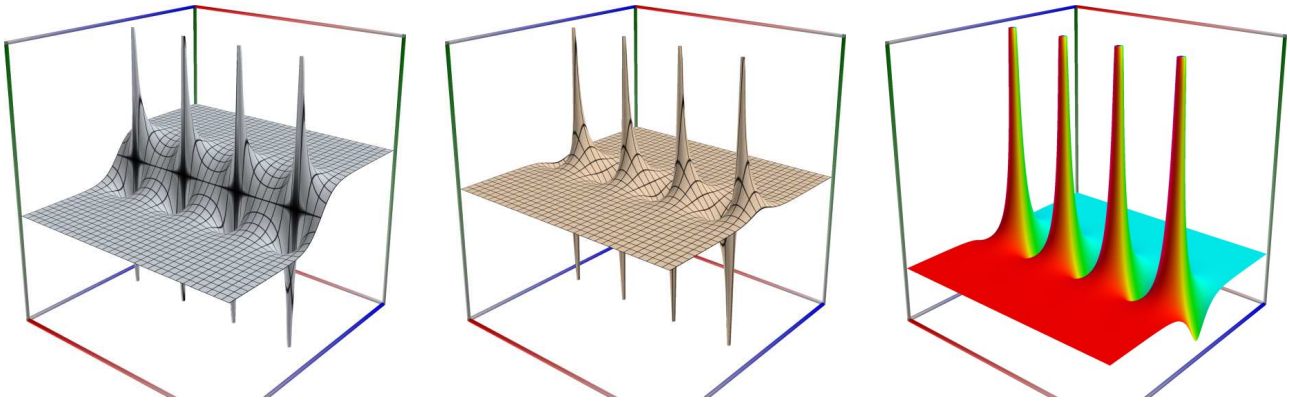
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the hyperbolic tangent of x, $\tanh(x)$. See also Wikipedia [1238], MathWorld [866], NIST [485], Ehrhardt [297] (4.2.62), Flint [740], Flint [730], Mpmath [566].

The hyperbolic tangent can be expressed in terms of the exponential function as $\tanh(x) = \dfrac{\sinh(x)}{\cosh(x)} = \dfrac{e^x - e^{-x}}{e^x + e^{-x}} = \dfrac{e^{2x-1}}{e^{2x+1}}$.

The complex hyperbolic tangent can be expressed in terms of related functions:

$$\tanh(z) = \frac{\sinh(z)}{\cosh(z)} = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \frac{e^{2z-1}}{e^{2z+1}} = -i\tan(iz)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Tanh(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Tanh('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> \mathrm{d}x = dec.tanh(x); mx = mpm.tanh(x); ix = ipm.tanh(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  9.171523356672732950300364697768401207429E-1
mpm:  9.171523356672732950300364697768401207429e-1
```

(continues on next page)

(continued from previous page)

```
ipm:  9.17152335667273295030036469776840120742 9e-1 (8.136e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> fx = fpm.tanh(x); gx = gmp.tanh(x); ax = apm.tanh(x)
>>> mpm.show([fx, gx, ax])
fpm:  9.17152335667273E-01
gmp:  9.17152335667273295030036469776840120742 9E-01
apm:  9.17152335667273295030036469776840120742 9e-1 (1.252e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '0.001 - 1.57079632679489j'
>>> \mathrm{d}z = dec.tanh(z); mz = mpm.tanh(z); iz = ipm.tanh(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.0000003333333111111E+3              - 6.619229115281640469 6E-9j
mpm: 1.0000003333333111111e+3              - 6.619228567667571007 7e-9j
ipm: 1.0000003333333111111e+3 (6.505e-19%) - 6.619229414700235917 6e-9 (-1.28e-5%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '0.001 - 1.57079632679489j'
>>> fz = fpm.tanh(z); gz = gmp.tanh(z); az = apm.tanh(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.00000033333331E+03                  - 6.72256824685193E-09j
gmp: 1.0000003333333111111E+03             - 6.619228567667571007 6E-09j
apm: 1.0000003333333111111e+3 (1.301e-19%) - 6.619229123532757354 8e-9 (-1.32e-5%)j
```

3.8.4 Hyperbolic cotangent, $\coth(x)$

ctx.coth(x)

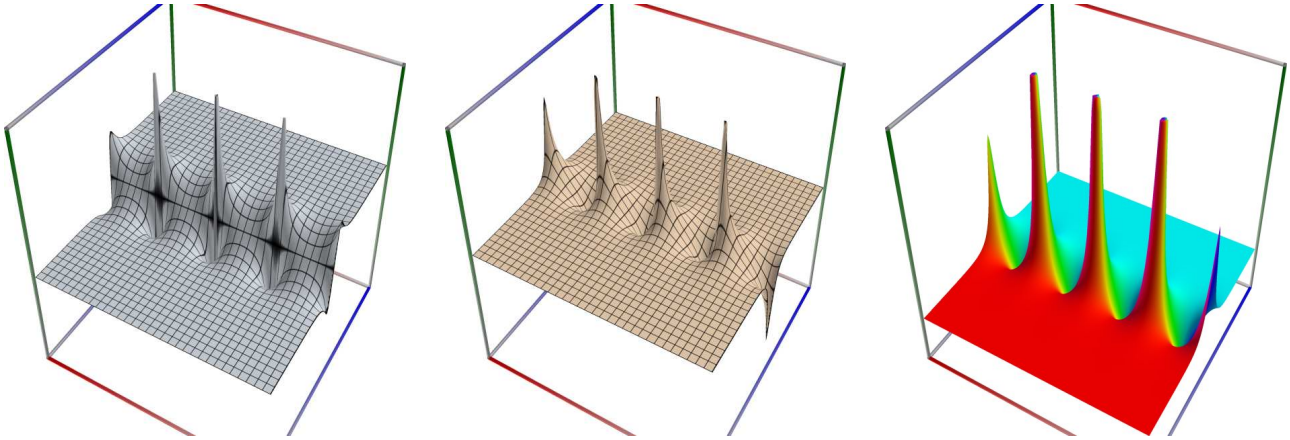
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the hyperbolic cotangent of x, $\coth(x)$. See also Wikipedia [1238], MathWorld [863], NIST [485], Ehrhardt [297] (4.2.22), Flint [740], Flint [730], Mpmath [563].

The hyperbolic cotangent can be expressed in terms of related functions as $\coth(x) = \dfrac{\cosh(x)}{\sinh(x)} = \dfrac{e^x + e^{-x}}{e^x - e^{-x}} = \dfrac{e^{2x+1}}{e^{2x-1}}$.

The complex hyperbolic cotangent can be expressed in terms of related functions:

$$\coth(z) = \frac{\cosh(z)}{\sinh(z)} = \frac{e^z + e^{-z}}{e^z - e^{-z}} = \frac{e^{2z+1}}{e^{2z-1}} = i \cdot \cot(iz)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Coth(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Coth('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> \mathrm{d}x = dec.coth(x); mx = mpm.coth(x); ix = ipm.coth(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.0903314107273694798903827835820272491 53E+0
mpm:  1.0903314107273694798903827835820272491 53e+0
ipm:  1.0903314107273694798903827835820272491 53e+0 (9.476e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> fx = fpm.coth(x); gx = gmp.coth(x); ax = apm.coth(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.09033141072737E+00
gmp:  1.0903314107273694798903827835820272491 53E+00
apm:  1.0903314107273694798903827835820272491 53e+0 (1.053e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '0.001 - 1.57079632679489j'
>>> \mathrm{d}z = dec.coth(z); mz = mpm.coth(z); iz = ipm.coth(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 9.9999966666679999995E-4              + 6.619224702464730 8782E-15j
mpm: 9.9999966666679999995e-4              + 6.619224154851026 4921e-15j
ipm: 9.9999966666679999995e-4 (4.963e-19%) + 6.619225001883126 7140e-15 (1.28e-5%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '0.001 - 1.57079632679489j'
>>> fz = fpm.coth(z); gz = gmp.coth(z); az = apm.coth(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 9.99999666666800E-04                  + 6.72256376514230E-15j
gmp: 9.9999966666679999995E-04             + 6.619224154851026 4921E-15j
apm: 9.9999966666679999995e-4 (1.654e-19%) + 6.619224710715842 2627e-15 (1.32e-5%)j
```

3.8.5 Hyperbolic cosecant, $\operatorname{csch}(x)$

`ctx.csch`(x)

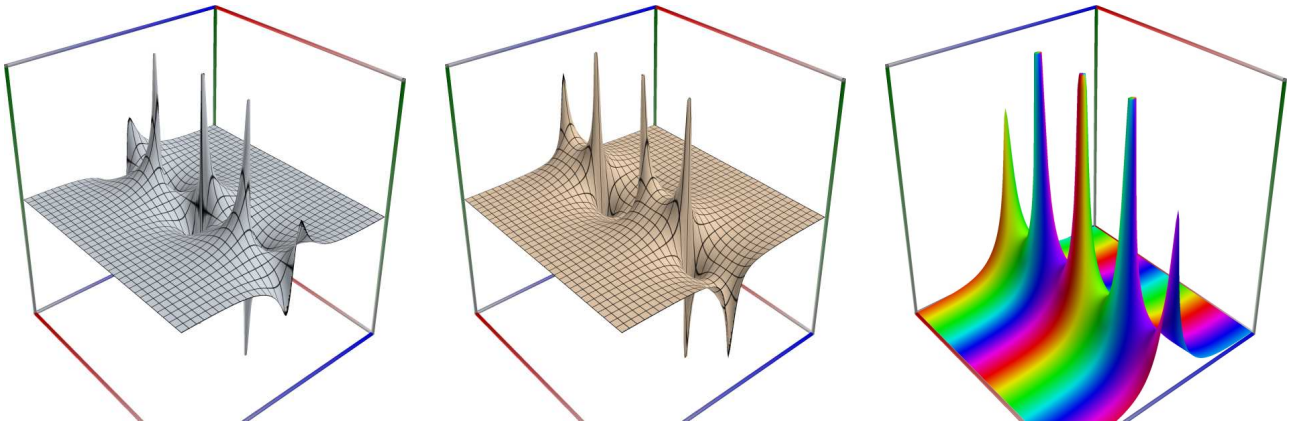
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the hyperbolic cosecant of x, $\operatorname{csch}(x)$. See also Wikipedia [1238], MathWorld [861], NIST [485], Ehrhardt [297] (4.2.24), Flint [740], Flint [730], Mpmath [561].

The hyperbolic cosecant can be expressed in terms of the exponential function as $\operatorname{csch}(x) = \dfrac{1}{\sinh(x)} = \dfrac{2}{e^x - e^{-x}} = \dfrac{2e^x}{1 - e^{-2x}}$.

The complex hyperbolic cosecant can be expressed in terms of related functions:

$$\operatorname{csch}(z) = \frac{1}{\sinh(z)} = \frac{2}{e^z - e^{-z}} = \frac{2e^z}{1 - e^{-2z}} = i \cdot \operatorname{csc}(iz)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Csch(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Csch('0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '3.14159265358979'
>>> \mathrm{d}x = dec.csch(x); mx = mpm.csch(x); ix = ipm.csch(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  8.6589537530047223294727159904650354673 80E-2
mpm:  8.6589537530047223294727159904650354673 81e-2
ipm:  8.6589537530047223294727159904650354673 80e-2 (7.457e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '3.14159265358979'
>>> fx = fpm.csch(x); gx = gmp.csch(x); ax = apm.csch(x)
>>> mpm.show([fx, gx, ax])
fpm:  8.65895375300472E-02
gmp:  8.6589537530047223294727159904650354673 81E-02
apm:  8.6589537530047223294727159904650354673 80e-2 (3.314e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.5E+2 - 1.570796326794 89j'
>>> \mathrm{d}z = dec.csch(z); mz = mpm.csch(z); iz = ipm.csch(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 9.4987240003426845058E-80            + 1.4350191946328820840E-65j
mpm: 9.4987232145057216312e-80            + 1.4350191946328820840e-65j
ipm: 9.4987244300142594276e-80 (1.28e-5%) + 1.4350191946328820840e-65 (5.605e-20%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.5E+2 - 1.570796326794 89j'
>>> fz = fpm.csch(z); gz = gmp.csch(z); az = apm.csch(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 9.64701768713396E-80                  + 1.43501919463288E-65j
gmp: 9.4987232145057216313E-80             + 1.4350191946328820840E-65j
apm: 9.4987240121831995601e-80 (1.326e-5%) + 1.4350191946328820840e-65 (5.605e-20%)j
```

3.8.6 Hyperbolic secant, $\mathrm{sech}(x)$

`ctx.sech`(x)

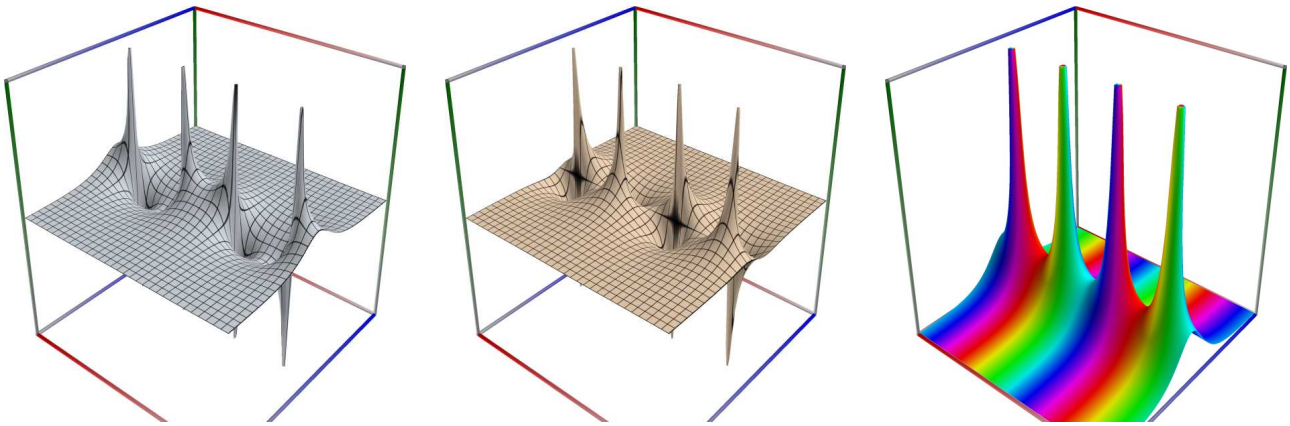
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the hyperbolic secant of x, $\mathrm{sech}(x)$. See also Wikipedia [1238], MathWorld [864], NIST [485], Ehrhardt [297] (4.2.54), Mpmath [564].

The hyperbolic secant can be expressed in terms of the exponential function as $\mathrm{sech}(x) = \dfrac{1}{\cosh(x)} = \dfrac{2}{e^x + e^{-x}} = \dfrac{2e^x}{e^{2x}+1}$.

The complex hyperbolic secant can be expressed in terms of related functions:

$$\mathrm{sech}(z) = \frac{1}{\cosh(z)} = \frac{2}{e^z + e^{-z}} = \frac{2e^z}{e^{2z}+1} = \sec(iz)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Sech(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Sech('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> \mathrm{d}x = dec.sech(x); mx = mpm.sech(x); ix = ipm.sech(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  3.9853681533838909988824430189041636662 17E-1
mpm:  3.9853681533838909988824430189041636662 17e-1
ipm:  3.9853681533838909988824430189041636662 17e-1 (4.321e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.57079632679489'
>>> fx = fpm.sech(x); gx = gmp.sech(x); ax = apm.sech(x)
>>> mpm.show([fx, gx, ax])
fpm:  3.98536815338389E-01
gmp:  3.9853681533838909988824430189041636662 17E-01
apm:  3.9853681533838909988824430189041636662 17e-1 (2.16e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1.5E+2 - 3.14159265358979j'
>>> \mathrm{d}z = dec.sech(z); mz = mpm.sech(z); iz = ipm.sech(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: -1.435019194632882084OE-65               + 4.64725605435654817 19E-80j
mpm: -1.435019194632882084e-65               + 4.647254734504525626 3e-80j
ipm: -1.435019194632882084e-65 (-5.605e-20%) + 4.647257165521601219 1e-80 (5.231e-5%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1.5E+2 - 3.14159265358979j'
>>> fz = fpm.sech(z); gz = gmp.sech(z); az = apm.sech(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: -1.43501919463288E-65                   + 4.63667494819155E-80j
gmp: -1.435019194632882084OE-65              + 4.647254734504525626 3E-80j
apm: -1.435019194632882084e-65 (-5.605e-20%) + 4.647256101951630647 3e-80 (5.401e-5%)j
```

3.9 Inverse trigonometric functions, in radians

For a general introduction to inverse trigonometric functions, see Wikipedia [1260], NIST [487].

3.9.1 Inverse sine, $\mathrm{asin}(x)$

`ctx.asin(x)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

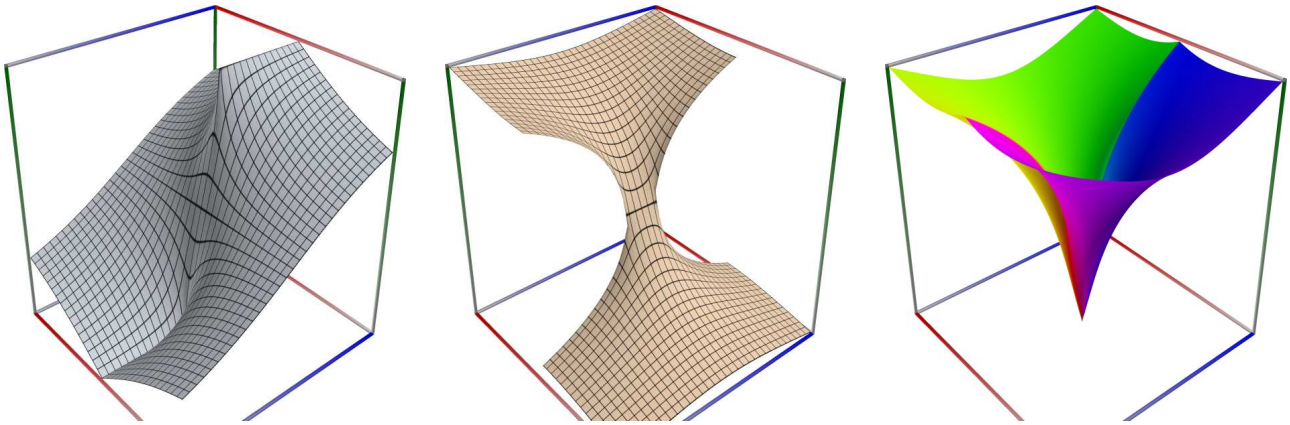
Returns the inverse sine of x, $\mathrm{asin}(x)$. See also Wikipedia [1260], MathWorld [871], NIST [487], Ehrhardt [297] (4.2.13), Mpmath [577].

The inverse sine can be expressed in terms of the inverse tangent as $\mathrm{asin}(x) = \mathrm{atan}\left(\frac{x}{\sqrt{1-x^2}}\right)$. The domain is the open interval $(-1, 1)$. We have $\sin(\mathrm{asin}(x)) = x$ for all x, but $\mathrm{asin}(\sin(x)) = x$ only for $-\pi/2 < x < \pi/2$.

The inverse sine can be expressed in terms of related functions (with the principal-branch log and square root):

$$\mathrm{asin}(z) = -i \log\left(iz + \sqrt{1-z^2}\right)$$

The inverse sine has two branch points: $x = \pm 1$.The branch cuts are placed along the line segments $(-\infty, -1)$ and $(+1, +\infty)$. Since $-1 \le \sin(x) \le 1$ for real x, the inverse sine is real-valued only for $-1 \le x \le 1$. On this interval, it is defined to be a monotonically increasing function assuming values between $-\pi/2$ and $\pi/2$.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Asin(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Asin('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1'
>>> \mathrm{d}x = dec.asin(x); mx = mpm.asin(x); ix = ipm.asin(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.5707963267948966192313216916397514420 99E+0
mpm:  1.5707963267948966192313216916397514420 99e+0
```

(continues on next page)

(continued from previous page)

```
ipm:  1.57079632679489661923132169163975144209 9e+0 (7.308e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1'
>>> fx = fpm.asin(x); gx = gmp.asin(x); ax = apm.asin(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.57079632679490E+00
gmp:  1.57079632679489661923132169163975144209 9E+00
apm:  1.57079632679489661923132169163975144209 9e+0 (1.462e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1 + 1.5E-2j'
>>> \mathrm{d}z = dec.asin(z); mz = mpm.asin(z); iz = ipm.asin(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.44847544713555477567E+0              + 1.22627061264029 98997E-1j
mpm: 1.44847544713555477567e+0              + 1.22627061264029 98997e-1j
ipm: 1.44847544713555477567e+0 (1.754e-19%) + 1.22627061264029 98997e-1 (1.295e-18%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1 + 1.5E-2j'
>>> fz = fpm.asin(z); gz = gmp.asin(z); az = apm.asin(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.44847544713555E+00                   + 1.22627061264030E-01j
gmp: 1.44847544713555477567E+00             + 1.22627061264029 98997E-01j
apm: 1.44847544713555477567e+0 (5.848e-20%) + 1.22627061264029 98997e-1 (1.727e-19%)j
```

$\mathrm{asin}(z)$ is defined so as to be a proper inverse function of $\sin(\theta)$ for $-\pi/2 < \theta < \pi/2$. We have $\sin(\sin^{-1}(x)) = x$ for all x, but $\sin^{-1}(\sin(x)) = x$ only for $-\pi/2 < \Re[x] < \pi/2$:

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> for x in [1, 10, -1, 1+3j, -2+3j]:
...     print("%s %s" % (chop(sin(asin(x))), asin(sin(x))))
...
1.0 1.0
10.0 -0.57522203923062028461206 98
-1.0 -1.0
(1.0 + 3.0j) (1.0 + 3.0j)
(-2.0 + 3.0j) (-1.14159265358979323846264 3 - 3.0j)
```

3.9.2 Inverse cosine, $\mathrm{acos}(x)$

ctx.acos(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the inverse cosine of x, $\mathrm{acos}(x)$. See also Wikipedia [1260], MathWorld [868], NIST [487], Ehrhardt [297] (4.2.3), Flint [743], Flint [733], Mpmath [568].

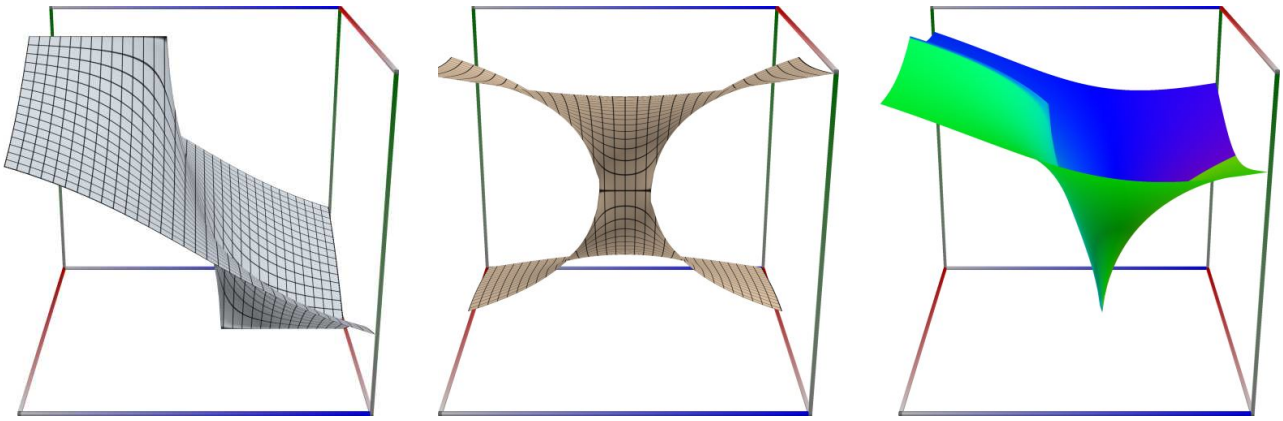
The inverse cosine can be expressed in terms of the inverse tangent as $\mathrm{acos}(x) = \mathrm{atan}\left(\frac{\sqrt{1-x^2}}{x}\right)$. The domain is the open interval $(-1, 1)$. We have $\cos(\mathrm{acos}(x)) = x$ for all x, but $\mathrm{acos}(\cos(x)) = x$ only for $0 \le x < \pi$.

The inverse cosine can be expressed in terms of related functions (with the principal-branch log and square root):

$$\mathrm{acos}(z) = \frac{\pi}{2} + i \log\left(iz + \sqrt{1-z^2}\right)$$

The inverse cosine has two branch points: $x = \pm 1$.The branch cuts are placed along the line segments $(-\infty, -1)$ and $(+1, +\infty)$.

Since $-1 \leq \cos(x) \leq 1$ for real x, the inverse cosine is real-valued only for $-1 \leq x \leq 1$, where it is a monotonically decreasing function assuming values between $+\pi$ and 0.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Acos(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Acos('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '0.5'
>>> \mathrm{d}x = dec.acos(x); mx = mpm.acos(x); ix = ipm.acos(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.0471975511965977461542144610931676280 66E+0
mpm:  1.0471975511965977461542144610931676280 66e+0
ipm:  1.0471975511965977461542144610931676280 66e+0 (1.096e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.5'
>>> fx = fpm.acos(x); gx = gmp.acos(x); ax = apm.acos(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.04719755119660E+00
gmp:  1.0471975511965977461542144610931676280 66E+00
apm:  1.0471975511965977461542144610931676280 66e+0 (1.096e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '0.5 + 1.5E-2j'
>>> \mathrm{d}z = dec.acos(z); mz = mpm.acos(z); iz = ipm.acos(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.0472841234408009862E+0              - 1.7318776682711839316E-2j
mpm: 1.0472841234408009862e+0              - 1.7318776682711839316e-2j
ipm: 1.0472841234408009862e+0 (8.088e-20%) - 1.7318776682711839316e-2 (-4.967e-18%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '0.5 + 1.5E-2j'
>>> fz = fpm.acos(z); gz = gmp.acos(z); az = apm.acos(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.04728412344080E+00                  - 1.73187766827118E-02j
```

(continues on next page)

(continued from previous page)

```
gmp: 1.0472841234408009862E+00             - 1.7318776682711839316E-02j
apm: 1.0472841234408009862e+0 (1.618e-19%) - 1.7318776682711839316e-2 (-3.821e-19%)j
```

$\operatorname{acos}(z)$ is defined so as to be a proper inverse function of $\cos(\theta)$ for $0 \le \theta < \pi$. We have $\cos(\cos^{-1}(x)) = z$ for all z, but $\cos^{-1}(\cos(z)) = z$ only for $0 \le \Re[x] < \pi$:

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> for x in [1, 10, -1, 2+3j, 10+3j]:
...     print("%s %s" % (cos(acos(x)), acos(cos(x))))
...
1.0 1.0
(10.0 + 0.0j) 2.566370614359172953850574
-1.0 1.0
(2.0 + 3.0j) (2.0 + 3.0j)
(10.0 + 3.0j) (2.566370614359172953850574 - 3.0j)
```

3.9.3 Inverse tangent, $\operatorname{atan}(x)$

`ctx.atan(`x`)`

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

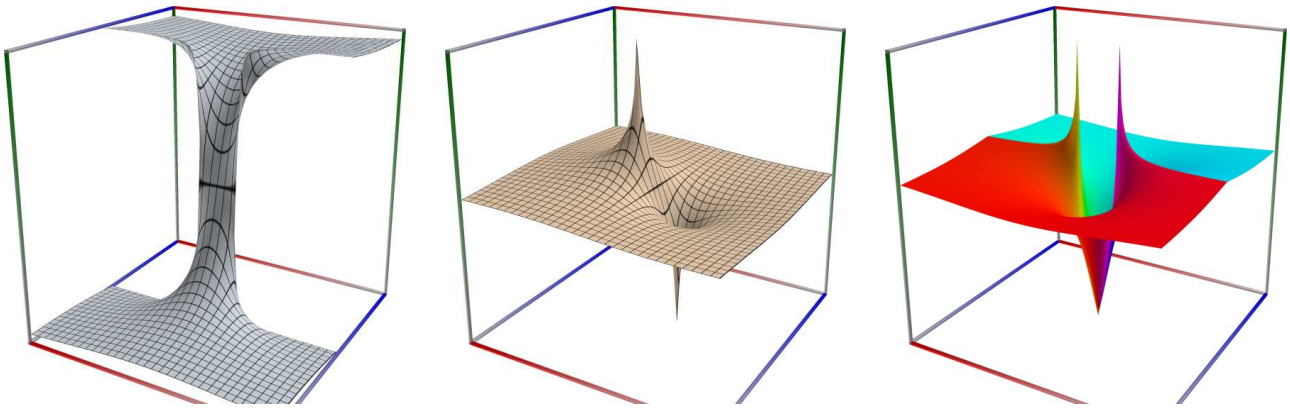
Returns the inverse tangent of x, $\operatorname{atan}(x)$. See also Wikipedia [1260], MathWorld [872], NIST [487], Ehrhardt [297] (4.2.15), Flint [743], Flint [733], Mpmath [578].

The inverse tangent can be defined as $\operatorname{atan}(x) = \int_0^x \frac{1}{t^2+1} dt$. This is a real-valued function for all real x, with range $(-\pi/2, \pi/2)$. We have $\tan(\operatorname{atan}(x)) = x$ for all x, but $\operatorname{atan}(\tan(x)) = x$ only for $-\pi/2 < x < \pi/2$.

The inverse tangent can be expressed in terms of related functions (with the principal-branch log and square root):

$$\operatorname{atan}(z) = \frac{i}{2}\left(\log(1 - iz) - \log(1 + iz)\right)$$

The inverse tangent has two branch points: $x = \pm i$.The branch cuts are placed along the line segments $(-i\infty, -i)$ and $(+i, +i\infty)$.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Atan(0.5)
ereal('5.23598775598298873O7E-1')
>>> math53.Atan('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '0.5'
>>> \mathrm{d}x = dec.atan(x); mx = mpm.atan(x); ix = ipm.atan(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  4.6364760900080611621425623146121440202 85E-1
mpm:  4.6364760900080611621425623146121440202 85e-1
ipm:  4.6364760900080611621425623146121440202 85e-1 (6.19e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.5'
>>> fx = fpm.atan(x); gx = gmp.atan(x); ax = apm.atan(x)
>>> mpm.show([fx, gx, ax])
fpm:  4.63647609000806E-01
gmp:  4.6364760900080611621425623146121440202 85E-01
apm:  4.6364760900080611621425623146121440202 85e-1 (6.19e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '0.5 + 1.5E-2j'
>>> \mathrm{d}z = dec.atan(z); mz = mpm.atan(z); iz = ipm.atan(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 4.6371961677714818965E-1             + 1.2000143940891281738E-2j
mpm: 4.6371961677714818965e-1             + 1.2000143940891281738e-2j
ipm: 4.6371961677714818965e-1 (1.37e-19%) + 1.2000143940891281738e-2 (4.853e-18%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '0.5 + 1.5E-2j'
>>> fz = fpm.atan(z); gz = gmp.atan(z); az = apm.atan(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 4.63719616777148E-01                  + 1.20001439408913E-02j
gmp: 4.6371961677714818965E-01             + 1.2000143940891281738E-02j
apm: 4.6371961677714818965e-1 (4.567e-20%) + 1.2000143940891281738e-2 (3.309e-19%)j
```

$\mathrm{atan}(z)$ is defined so as to be a proper inverse function of $\tan(\theta)$ for $-\pi/2 < \theta < \pi/2$. We have $\tan(\tan^{-1}(x)) = x$ for all x, but $\tan^{-1}(\tan(x)) = x$ only for $-\pi/2 < \Re[x] < \pi/2$:

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> mp.dps = 25
>>> for x in [1, 10, -1, 1+3j, -2+3j]:
...     print("%s %s" % (tan(atan(x)), atan(tan(x))))
...
1.0 1.0
10.0 0.5752220392306202846120698
-1.0 -1.0
(1.0 + 3.0j) (1.000000000000000000000001 + 3.0j)
(-2.0 + 3.0j) (1.1415926535897932384626 44 + 3.0j)
```

3.9.4 Inverse tangent, 2 arguments, $\mathrm{atan2}(y, x)$

`ctx.atan2`(y, x)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the inverse tangent of x and y. See also Wikipedia [1273], MathWorld [872], Flint [743], Flint [733], Mpmath [615].

Computes the two-argument arctangent, $\mathrm{atan2}(y, x)$, giving the signed angle between the positive x-axis and the point (x, y) in the 2D plane. This function is defined for real x and y only.

The two-argument arctangent essentially computes $\mathrm{atan}(y/x)$, but accounts for the signs of both x and y to give the angle for the correct quadrant. The following examples illustrate the difference:

An example in Python

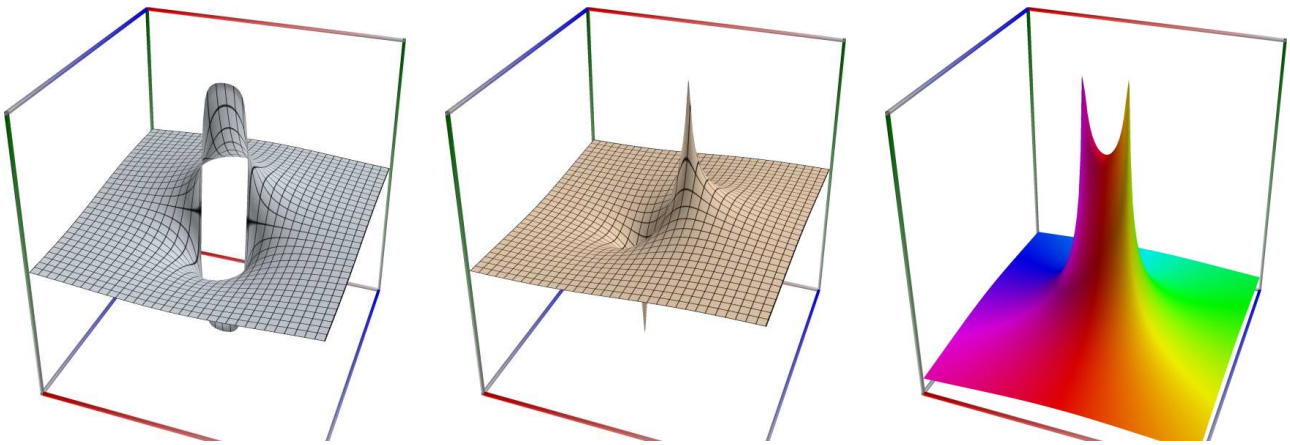
```
>>> from xlcalcnet import math53
>>> math53.Atan2(1,1), ereal.Atan(1/1.)
(0.785398163397448, 0.785398163397448)
>>> math53.Atan2(1,-1), ereal.Atan(1/-1.)
(2.35619449019234, -0.785398163397448)
>>> math53.Atan2(-1,1), ereal.Atan(-1/1.)
(-0.785398163397448, -0.785398163397448)
>>> math53.Atan2(-1,-1), ereal.Atan(-1/-1.)
(-2.35619449019234, 0.785398163397448)
```

The angle convention is the same as that used for the complex argument.

3.9.5 Inverse cotangent, $\mathrm{acot}(x)$

`ctx.acot(`x`)`

Returns the inverse cotangent of x, $\mathrm{acot}(x)$. See also Wikipedia [1260], MathWorld [869], NIST [487], Ehrhardt [297] (4.2.5).



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

The inverse cotangent can be expressed in terms of the inverse tangent as $\mathrm{acot}(x) = \mathrm{atan}\left(\frac{1}{x}\right)$. This is a real-valued function for all real x, with range $(0, \pi)$.

The inverse cotangent can be expressed in terms of related functions (with the principal-branch log and square root):

$$\mathrm{acot}(z) = \frac{i}{2}\left[\log\left(1 - \frac{i}{z}\right) - \log\left(1 + \frac{i}{z}\right)\right] = \mathrm{atan}\left(\frac{1}{z}\right)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Acot(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Acot('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '2.0'
>>> \mathrm{d}x = dec.asec(x); mx = mpm.asec(x); ix = ipm.asec(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.047197551196597746154214461093167628066E+0
mpm:  1.047197551196597746154214461093167628066e+0
ipm:  1.047197551196597746154214461093167628066e+0 (1.096e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '2.0'
>>> fx = fpm.asec(x); gx = gmp.asec(x); ax = apm.asec(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.04719755119660E+00
gmp:  1.047197551196597746154214461093167628066E+00
apm:  1.047197551196597746154214461093167628066e+0 (1.096e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '2.0 + 1.5E-2j'
>>> \mathrm{d}z = dec.asec(z); mz = mpm.asec(z); iz = ipm.asec(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.0472354363409255922E+0              + 4.3297752322606827325E-3j
mpm: 1.0472354363409255922e+0              + 4.3297752322606827325e-3j
ipm: 1.0472354363409255922e+0 (2.426e-19%) + 4.3297752322606827323e-3 (3.508e-17%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '2.0 + 1.5E-2j'
>>> fz = fpm.asec(z); gz = gmp.asec(z); az = apm.asec(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.04723543634093E+00                  + 4.32977523226068E-03j
gmp: 1.0472354363409255922E+00             + 4.329775232260682732 5E-03j
apm: 1.0472354363409255922e+0 (8.088e-20%) + 4.3297752322606827324e-3 (1.146e-18%)j
```

3.9.6 Inverse cosecant, $\operatorname{acsc}(x)$

ctx.acsc(x)

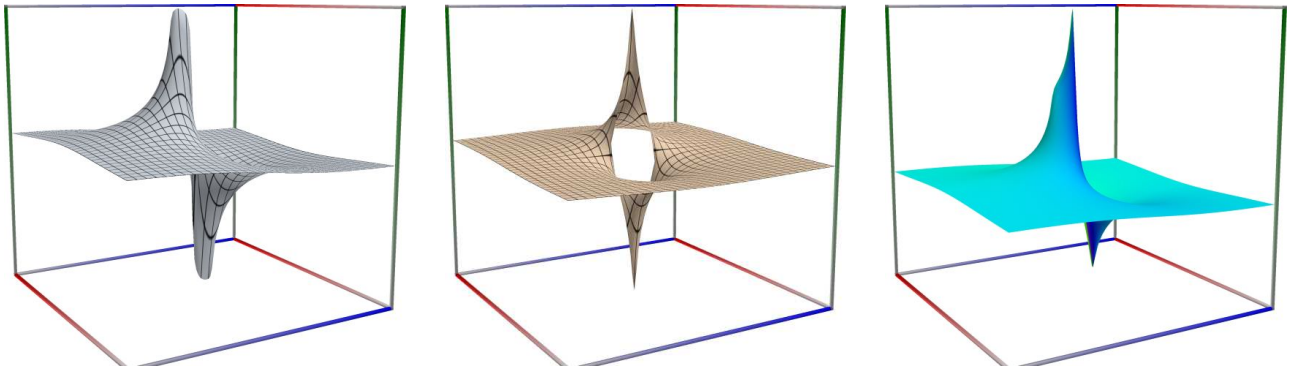
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the inverse cosecant of x, $\operatorname{acsc}(x)$. See also Wikipedia [1260], MathWorld [867], NIST [487], Ehrhardt [297] (4.2.9), Mpmath [567].

The inverse cosecant can be expressed in terms of the inverse sine as $\operatorname{acsc}(x) = \operatorname{asin}\left(\frac{1}{x}\right)$. The domain is $\mathbb{R} \setminus (-1, 1)$, i.e. $|x| > 0$.

The inverse cosecant can be expressed in terms of related functions (with the principal-branch log and square root):

$$\operatorname{acsc}(z) = -i \log\left(\sqrt{1 - \frac{1}{z^2}} + \frac{i}{z}\right) = \operatorname{asin}\left(\frac{1}{z}\right)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Acsc(0.5)
ereal('5.235987755982988730771072305465838140329E-1')
>>> math53.Acsc('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '2.0'
>>> \mathrm{d}x = dec.acsc(x); mx = mpm.acsc(x); ix = ipm.acsc(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  5.235987755982988730771072305465838140329E-1
mpm:  5.235987755982988730771072305465838140329e-1
ipm:  5.235987755982988730771072305465838140329e-1 (2.192e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '2.0'
>>> fx = fpm.acsc(x); gx = gmp.acsc(x); ax = apm.acsc(x)
>>> mpm.show([fx, gx, ax])
fpm:  5.2359877559829 9E-01
gmp:  5.235987755982988730771072305465838140329E-01
apm:  5.235987755982988730771072305465838140329e-1 (1.096e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '2.0 + 1.5E-2j'
>>> \mathrm{d}z = dec.acsc(z); mz = mpm.acsc(z); iz = ipm.acsc(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 5.2356089045397102702E-1              - 4.3297752322606827325E-3j
mpm: 5.2356089045397102702e-1              - 4.3297752322606827325e-3j
ipm: 5.2356089045397102702e-1 (4.045e-19%) - 4.3297752322606827323e-3 (-3.508e-17%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '2.0 + 1.5E-2j'
>>> fz = fpm.acsc(z); gz = gmp.acsc(z); az = apm.acsc(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 5.23560890453971E-01                  - 4.32977523226068E-03j
gmp: 5.2356089045397102702E-01             - 4.3297752322606827325E-03j
apm: 5.2356089045397102702e-1 (8.089e-20%) - 4.3297752322606827324e-3 (-1.146e-18%)j
```

3.9.7 Inverse secant, $\operatorname{asec}(x)$

`ctx.asec(`x`)`

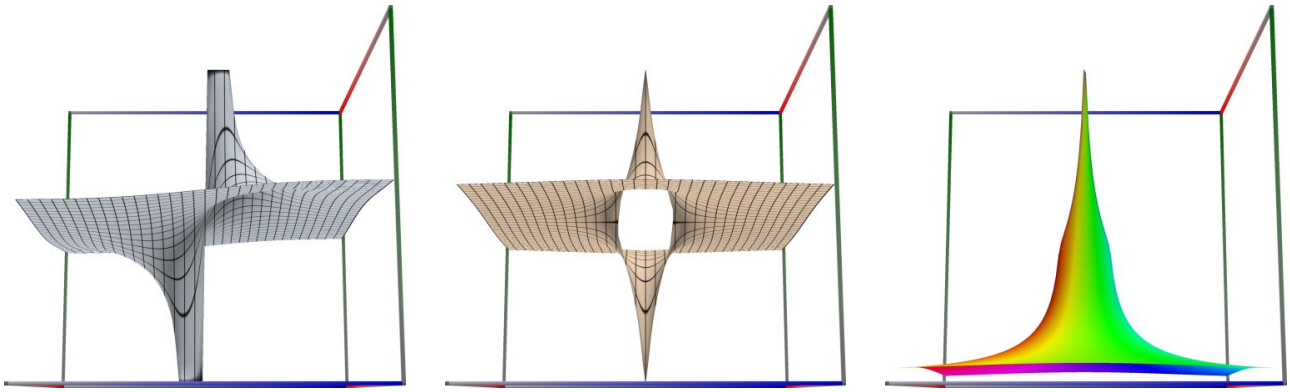
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the inverse secant of x, $\operatorname{asec}(x)$. See also Wikipedia [1260], MathWorld [870], NIST [487], Ehrhardt [297] (4.2.11), Mpmath [576].

The inverse secant can be expressed in terms of the inverse cosine as $\operatorname{asec}(x) = \operatorname{acos}\left(\frac{1}{x}\right)$. The domain is $\mathbb{R} \setminus (-1, 1)$, i.e. $|x| > 0$.

The inverse secant can be expressed in terms of related functions (with the principal-branch log and square root):

$$\operatorname{asec}(z) = -i \log\left(\sqrt{\frac{1}{z^2} - 1} + \frac{1}{z}\right) = \operatorname{acos}\left(\frac{1}{z}\right)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Asec(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Asec('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '2.0'
>>> \mathrm{d}x = dec.asec(x); mx = mpm.asec(x); ix = ipm.asec(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.0471975511965977461542144610931676280 66E+0
mpm:  1.0471975511965977461542144610931676280 66e+0
ipm:  1.0471975511965977461542144610931676280 66e+0 (1.096e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '2.0'
>>> fx = fpm.asec(x); gx = gmp.asec(x); ax = apm.asec(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.04719755119660E+00
gmp:  1.0471975511965977461542144610931676280 66E+00
apm:  1.0471975511965977461542144610931676280 66e+0 (1.096e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '2.0 + 1.5E-2j'
>>> \mathrm{d}z = dec.asec(z); mz = mpm.asec(z); iz = ipm.asec(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.0472354363409255922E+0              + 4.3297752322606827325E-3j
mpm: 1.0472354363409255922e+0              + 4.3297752322606827325e-3j
ipm: 1.0472354363409255922e+0 (2.426e-19%) + 4.3297752322606827323e-3 (3.508e-17%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '2.0 + 1.5E-2j'
>>> fz = fpm.asec(z); gz = gmp.asec(z); az = apm.asec(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.04723543634093E+00                  + 4.32977523226068E-03j
gmp: 1.0472354363409255922E+00             + 4.3297752322606827325E-03j
apm: 1.0472354363409255922e+0 (8.088e-20%) + 4.3297752322606827324e-3 (1.146e-18%)j
```

3.10 Inverse hyperbolic functions

For a general introduction to inverse hyperbolic functions, see Wikipedia [1261], NIST [486].

3.10.1 Inverse hyperbolic sine, $\operatorname{asinh}(x)$

`ctx.asinh(x)`

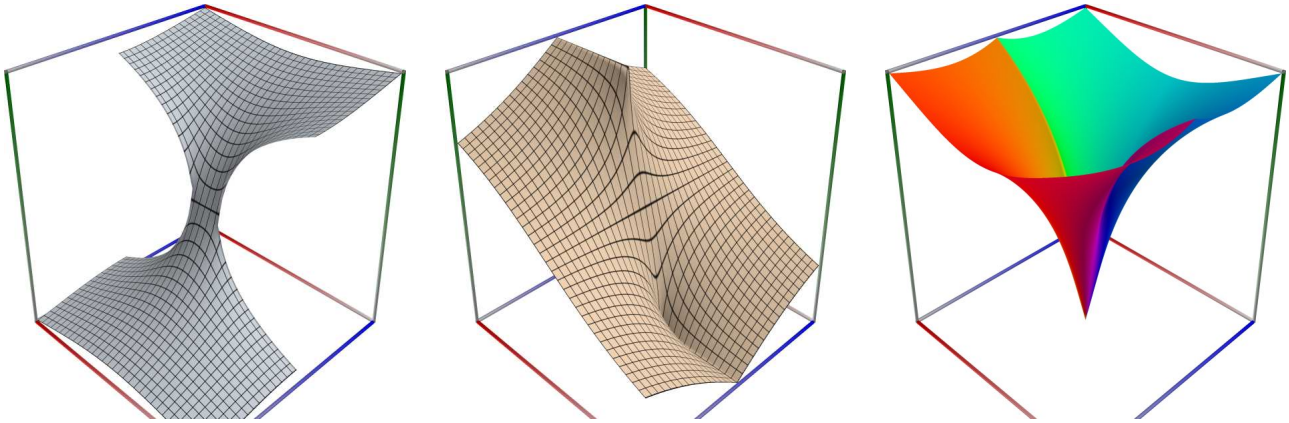
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the inverse hyperbolic sine of x, $\operatorname{asinh}(x)$. See also Wikipedia [1261], MathWorld [889], NIST [486], Ehrhardt [297] (4.2.14), Flint [742], Flint [732], Mpmath [574].

The inverse hyperbolic sine can be defined as $\operatorname{asinh}(x) = \log\left(x + \sqrt{1+x^2}\right)$. The domain is the whole real line.

The inverse hyperbolic sine can be expressed in terms of related functions (with the principal-branch log and square root):

$$\operatorname{asinh}(z) = \log\left(z + \sqrt{1+z^2}\right) = \frac{1}{i}\operatorname{asin}(iz)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Asinh(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Asinh('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1'
>>> \mathrm{d}x = dec.asinh(x); mx = mpm.asinh(x); ix = ipm.asinh(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  8.8137358701954302523260932497979230902 82E-1
mpm:  8.8137358701954302523260932497979230902 82e-1
ipm:  8.8137358701954302523260932497979230902 82e-1 (3.907e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1'
>>> fx = fpm.asinh(x); gx = gmp.asinh(x); ax = apm.asinh(x)
>>> mpm.show([fx, gx, ax])
```

(continues on next page)

(continued from previous page)

```
fpm:  8.81373587019543E-01
gmp:  8.8137358701954302523260932497979230902 82E-01
apm:  8.8137358701954302523260932497979230902 82e-1 (6.512e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '1 + 1.5E+2j'
>>> \mathrm{d}z = dec.asinh(z); mz = mpm.asinh(z); iz = ipm.asinh(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 5.7037935865697639286E+0              + 1.5641296107511107312E+0j
mpm: 5.7037935865697639286e+0              + 1.5641296107511107312e+0j
ipm: 5.7037935865697639120e+0 (5.704e-16%) + 1.5641296107511107313e+0 (3.016e-17%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '1 + 1.5E+2j'
>>> fz = fpm.asinh(z); gz = gmp.asinh(z); az = apm.asinh(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 5.70379358656976E+00                  + 1.56412961075111E+00j
gmp: 5.7037935865697639286E+00             + 1.5641296107511107312E+00j
apm: 5.7037935865697639283e+0 (1.936e-17%) + 1.5641296107511107312e+0 (1.083e-18%)j
```

3.10.2 Inverse hyperbolic cosine, $\operatorname{acosh}(x)$

`ctx.acosh`(x)

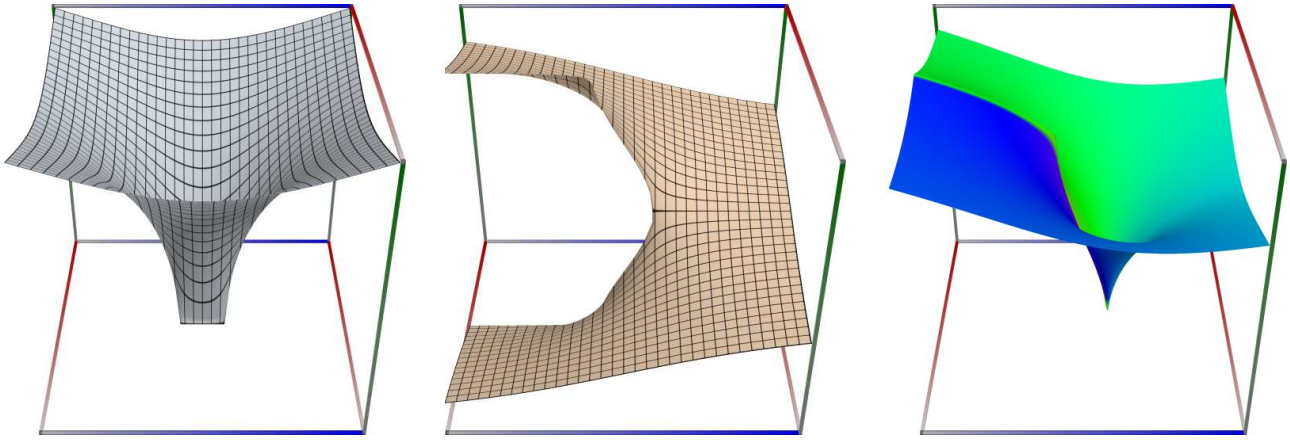
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the inverse hyperbolic cosine of x, $\operatorname{acosh}(x)$. See also Wikipedia [1261], MathWorld [886], NIST [486], Ehrhardt [297] (4.2.4), Flint [742], Flint [732], Mpmath [571].

The inverse hyperbolic cosine can be defined as $\operatorname{acosh}(x) = \log\left(x + \sqrt{x^2 - 1}\right)$. The domain is the closed interval $[1, +\infty)$.

The inverse hyperbolic cosine can be expressed in terms of related functions (with the principal-branch log and square root):

$$\operatorname{acosh}(z) = \log\left(z + \sqrt{z+1}\sqrt{z-1}\right) = \frac{\sqrt{z-1}}{\sqrt{1-z}}\operatorname{acos}(z)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Acosh(0.5)
ereal('5.23598775598298873O7E-1')
>>> math53.Acosh('0.51')
ereal('5.35184790275599847 54E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '2'
>>> \mathrm{d}x = dec.acosh(x); mx = mpm.acosh(x); ix = ipm.acosh(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.3169578969248167086250463473079684440 27E+0
mpm:  1.3169578969248167086250463473079684440 27e+0
ipm:  1.3169578969248167086250463473079684440 27e+0 (8.717e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '2'
>>> fx = fpm.acosh(x); gx = gmp.acosh(x); ax = apm.acosh(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.31695789692482E+00
gmp:  1.3169578969248167086250463473079684440 27E+00
apm:  1.3169578969248167086250463473079684440 27e+0 (8.717e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '2 + 1.5E+2j'
>>> \mathrm{d}z = dec.acosh(z); mz = mpm.acosh(z); iz = ipm.acosh(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 5.7038824606468806598E+0              + 1.5574640796818581268E+0j
mpm: 5.7038824606468806598e+0              + 1.5574640796818581268e+0j
ipm: 5.7038824606468806598e+0 (1.188e-19%) + 1.5574640796818581268e+0 (5.439e-20%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '2 + 1.5E+2j'
>>> fz = fpm.acosh(z); gz = gmp.acosh(z); az = apm.acosh(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 5.703882460 64688E+00                 + 1.55746407968186E+00j
gmp: 5.7038824606468806598E+00             + 1.5574640796818581268E+00j
apm: 5.7038824606468806598e+0 (5.94e-20%) + 1.5574640796818581268e+0 (5.439e-20%)j
```

3.10.3 Inverse hyperbolic tangent, $\operatorname{atanh}(x)$

`ctx.atanh`(x)

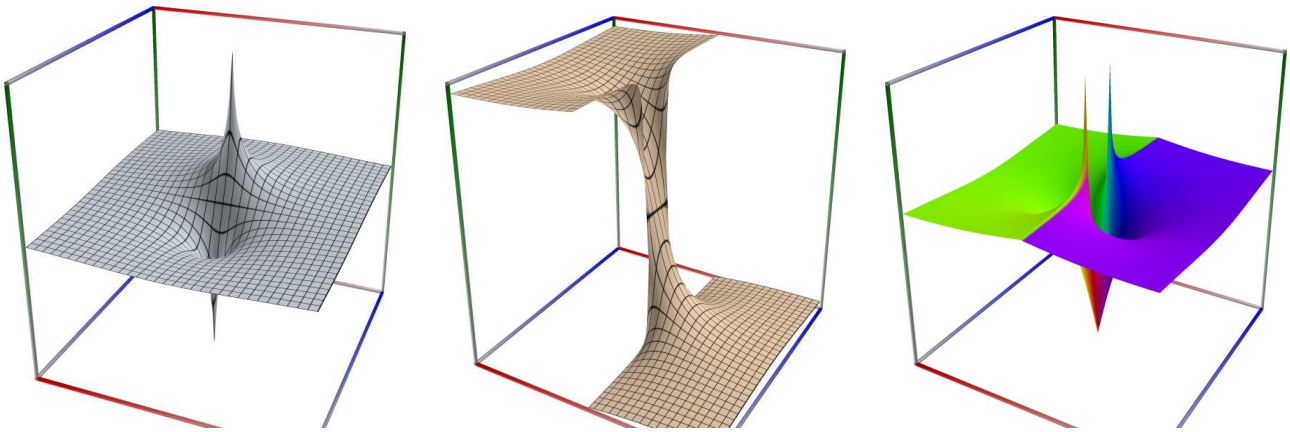
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the inverse hyperbolic tangent of x, $\operatorname{atanh}(x)$. See also Wikipedia [1261], MathWorld [890], NIST [486], Ehrhardt [297] (4.2.16), Flint [742], Flint [732], Mpmath [575].

The inverse hyperbolic tangent can be defined as $\operatorname{atanh}(x) = \frac{1}{2}\log\left(\frac{1+x}{1-x}\right)$. The domain is the open interval $(-1, 1)$.

The inverse hyperbolic tangent can be expressed in terms of related functions (with the principal-branch log and square root):

$$\operatorname{atanh}(z) = \frac{1}{2}\log\left(\frac{1+z}{1-z}\right) = \frac{1}{2}\left[\log(1+z) - \log(1-z)\right] = \frac{1}{i}\operatorname{atan}(iz)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Atanh(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Atanh('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '0.5'
>>> \mathrm{d}x = dec.atanh(x); mx = mpm.atanh(x); ix = ipm.atanh(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  5.4930614433405484569762261846126285232 37E-1
mpm:  5.4930614433405484569762261846126285232 37e-1
ipm:  5.4930614433405484569762261846126285232 37e-1 (2.09e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.5'
>>> fx = fpm.atanh(x); gx = gmp.atanh(x); ax = apm.atanh(x)
>>> mpm.show([fx, gx, ax])
fpm:  5.49306144334055E-01
gmp:  5.4930614433405484569762261846126285232 37E-01
apm:  5.4930614433405484569762261846126285232 37e-1 (2.09e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '2 + 1.5E+2j'
>>> \mathrm{d}z = dec.atanh(z); mz = mpm.atanh(z); iz = ipm.atanh(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 8.8869141126887266889E-5              + 1.5641309437602554673E+0j
mpm: 8.8869141126887266889e-5              + 1.5641309437602554673e+0j
ipm: 8.8869141126887268292e-5 (3.812e-15%) + 1.5641309437602554673e+0 (5.415e-20%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '2 + 1.5E+2j'
>>> fz = fpm.atanh(z); gz = gmp.atanh(z); az = apm.atanh(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 8.88691411268873E-05                 + 1.56413094376026E+00j
gmp: 8.8869141126887266889E-05            + 1.5641309437602554673E+00j
apm: 8.8869141126887266883e-5 (4.73e-17%) + 1.5641309437602554673e+0 (1.083e-19%)j
```

3.10.4 Inverse hyperbolic cotangent, $\operatorname{acoth}(x)$

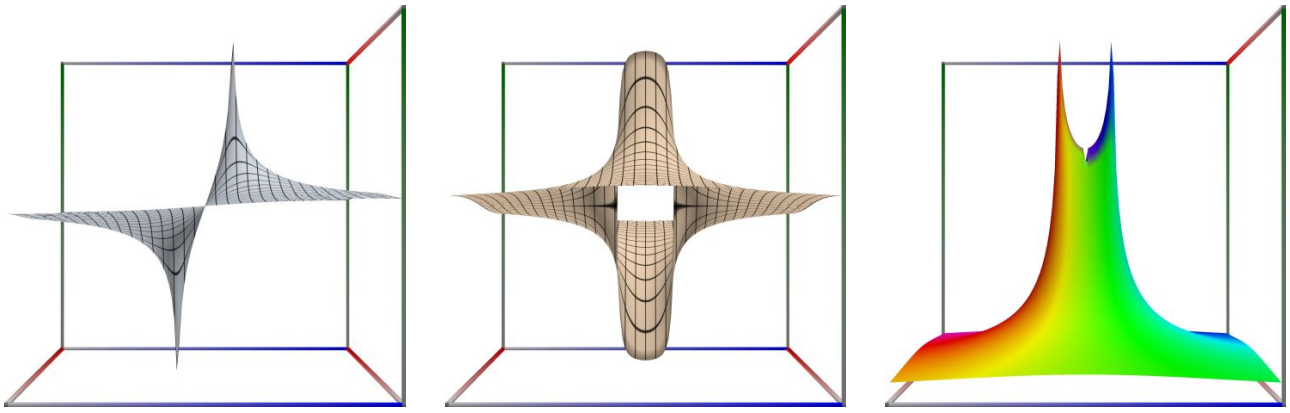
`ctx.acoth(x)`

Returns the inverse hyperbolic cotangent of x, $\operatorname{acoth}(x)$. See also Wikipedia [1261], MathWorld [887], NIST [486], Ehrhardt [297] (4.2.7), Mpmath [572].

The real inverse hyperbolic cotangent can be defined as $\operatorname{acoth}(x) = \frac{1}{2}\log\left(\frac{x+1}{x-1}\right)$. The domain is the union of the open intervals $(-\infty, -1)$ and $(1, +\infty)$.

The complex inverse hyperbolic cotangent can be expressed in terms of related functions (with the principal-branch log and square root):

$$\operatorname{acoth}(z) = \frac{1}{2}\log\left(\frac{z+1}{z-1}\right) = \frac{1}{2}\left[\log\left(1+\frac{1}{z}\right) - \log\left(1-\frac{1}{z}\right)\right] = \frac{1}{i}\operatorname{acot}(-iz).$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Acoth(0.5)
ereal('5.23598775598298873076E-1')
>>> math53.Acoth('0.51')
ereal('5.35184790275599847540E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.5'
>>> \mathrm{d}x = dec.acoth(x); mx = mpm.acoth(x); ix = ipm.acoth(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  8.0471895621705018730037966661309381976280E-1
mpm:  8.0471895621705018730037966661309381976280E-1
ipm:  8.0471895621705018730037966661309381976280E-1 (2.853e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.5'
>>> fx = fpm.acoth(x); gx = gmp.acoth(x); ax = apm.acoth(x)
>>> mpm.show([fx, gx, ax])
fpm:  8.04718956217050E-01
gmp:  8.0471895621705018730037966661309381976280E-01
apm:  8.0471895621705018730037966661309381976280E-1 (7.133e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '2 + 1.5E+2j'
>>> \mathrm{d}z = dec.acoth(z); mz = mpm.acoth(z); iz = ipm.acoth(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 8.8869141126887266889E-5              - 6.6653830346411519558E-3j
mpm: 8.8869141126887266889e-5              - 6.6653830346411519558e-3j
ipm: 8.8869141126887267011e-5 (7.15e-16%) - 6.6653830346411519558e-3 (-1.489e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '2 + 1.5E+2j'
>>> fz = fpm.acoth(z); gz = gmp.acoth(z); az = apm.acoth(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 8.88691411268873E-05                  - 6.66538303464115E-03j
gmp: 8.8869141126887266889E-05             - 6.6653830346411519558E-03j
apm: 8.8869141126887266888e-5 (1.28e-18%) - 6.6653830346411519558e-3 (-1.986e-19%)j
```

3.10.5 Inverse hyperbolic secant, $\operatorname{asech}(x)$

`ctx.asech(`x`)`

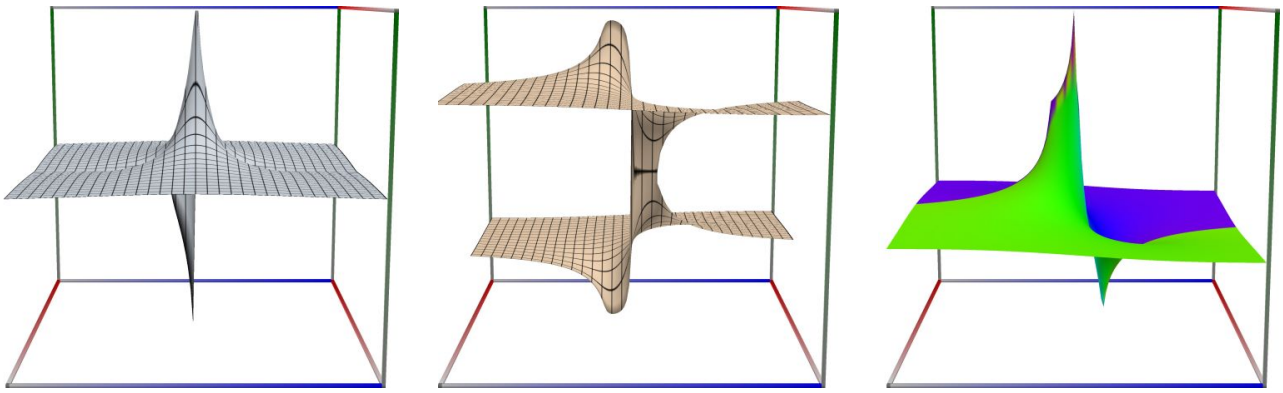
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the inverse hyperbolic secant of x, $\operatorname{asech}(x)$. See also Wikipedia [1261], MathWorld [888], NIST [486], Ehrhardt [297] (4.2.12), Mpmath [573].

The inverse hyperbolic secant can be defined as $\operatorname{asech}(x) = \log\left(\frac{1}{x} + \sqrt{\frac{1}{x^2} - 1}\right)$. The domain is the semi-open interval $(0, 1]$.

The inverse hyperbolic secant can be expressed in terms of related functions (with the principal-branch log and square root):

$$\operatorname{asech}(z) = \log\left(\frac{1}{z} + \sqrt{\frac{1}{z} + 1}\sqrt{\frac{1}{z} - 1}\right)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Asech(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Asech('0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '0.5'
>>> \mathrm{d}x = dec.asech(x); mx = mpm.asech(x); ix = ipm.asech(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.31695789692481670862504634730796844402 7E+0
mpm:  1.31695789692481670862504634730796844402 7e+0
ipm:  1.31695789692481670862504634730796844402 7e+0 (8.717e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.5'
>>> fx = fpm.asech(x); gx = gmp.asech(x); ax = apm.asech(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.31695789692482E+00
gmp:  1.31695789692481670862504634730796844402 7E+00
apm:  1.31695789692481670862504634730796844402 7e+0 (8.717e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '2 + 1.5E+2j'
>>> \mathrm{d}z = dec.asech(z); mz = mpm.asech(z); iz = ipm.asech(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 6.6654323630640601125E-3              - 1.5707074556797408039E+0j
mpm: 6.6654323630640601125e-3              - 1.5707074556797408039e+0j
ipm: 6.6654323630640601131e-3 (5.053e-17%) - 1.5707074556797408039e+0 (-5.393e-20%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '2 + 1.5E+2j'
>>> fz = fpm.asech(z); gz = gmp.asech(z); az = apm.asech(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 6.66543236306406E-03                  - 1.57070745567974E+00j
gmp: 6.6654323630640601125E-03             - 1.5707074556797408039E+00j
apm: 6.6654323630640601124e-3 (1.787e-18%) - 1.5707074556797408039e+0 (-1.079e-19%)j
```

3.10.6 Inverse Hyperbolic Cosecant, $\operatorname{acsch}(x)$

`ctx.acsch`(x)

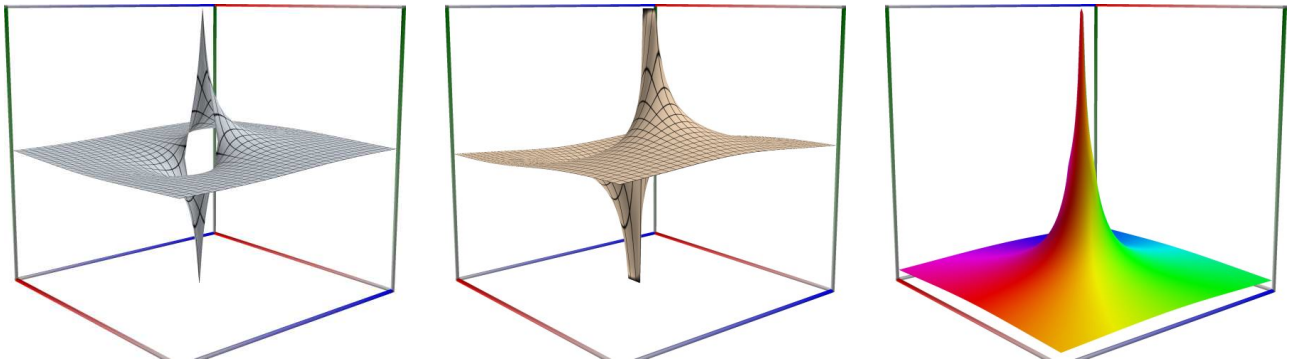
where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the inverse hyperbolic cosecant of x, $\operatorname{acsch}(x)$. See also Wikipedia [1261], MathWorld [885], NIST [440], Ehrhardt [297] (4.2.10), Mpmath [570].

The real inverse hyperbolic cosecant can be defined as $\operatorname{acsch}(x) = \log\left(\frac{1}{x} + \sqrt{\frac{1}{x^2} + 1}\right)$. The domain is the real line with 0 removed.

The complex inverse hyperbolic cosecant can be expressed in terms of related functions (with the principal-branch log and square root):

$$\operatorname{acsch}(z) = \log\left(\frac{1}{z} + \sqrt{\frac{1}{z^2} + 1}\right)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Acsch(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Acsch('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '1.5'
>>> \mathrm{d}x = dec.acsch(x); mx = mpm.acsch(x); ix = ipm.acsch(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  6.2514511725041668763425167322610240703 42E-1
mpm:  6.2514511725041668763425167322610240703 42e-1
ipm:  6.2514511725041668763425167322610240703 42e-1 (1.01e-38%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.5'
>>> fx = fpm.acsch(x); gx = gmp.acsch(x); ax = apm.acsch(x)
>>> mpm.show([fx, gx, ax])
fpm:  6.25145117250417E-01
gmp:  6.2514511725041668763425167322610240703 42E-01
apm:  6.2514511725041668763425167322610240703 42e-1 (9.181e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '2 + 1.5E+2j'
>>> \mathrm{d}z = dec.acsch(z); mz = mpm.acsch(z); iz = ipm.acsch(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 6.6654323630640601125E-3              - 1.5707074556797408039E+0j
mpm: 6.6654323630640601125e-3              - 1.5707074556797408039e+0j
ipm: 6.6654323630640601131e-3 (5.053e-17%) - 1.5707074556797408039e+0 (-5.393e-20%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '2 + 1.5E+2j'
>>> fz = fpm.acsch(z); gz = gmp.acsch(z); az = apm.acsch(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 6.66543236306406E-03                  - 1.57070745567974E+00j
gmp: 6.6654323630640601125E-03             - 1.5707074556797408039E+00j
apm: 6.6654323630640601124e-3 (1.787e-18%) - 1.5707074556797408039e+0 (-1.079e-19%)j
```

3.11 Factorials, Gamma and related functions

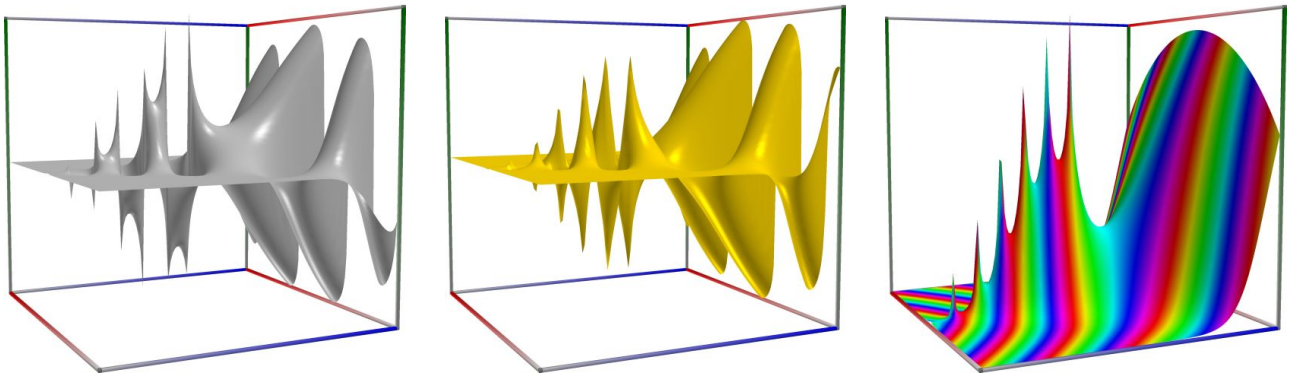
3.11.1 Gamma function, $\Gamma(x)$

ctx.gamma(x)

where ctx is math53, mathc53, ctxboost, ctxflint.

Returns the gamma function $\Gamma(x) = \int_0^\infty t^{x-1} e^{-t}\,\mathrm{d}t$, for any real or complex x with $\Re(x) > 0$ and for $\Re(x) < 0$ by analytic continuation.

See also Wikipedia [1258], MathWorld [884], NIST [22], BoostMath [101], Ehrhardt [297] (3.5.1.1), Ehrhardt [297] (4.2.38), Flint [739], Flint [729], Mpmath [589].



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Gamma(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Gamma('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '10.5'
>>> \mathrm{d}x = dec.gamma(x); mx = mpm.gamma(x); ix = ipm.gamma(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.133278388948785567334574165588892475560E+6
mpm:  1.133278388948785567334574165588892475560e+6
ipm:  1.133278388948785567334574165588892475560e+6 (1.062e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10.5'
>>> fx = fpm.gamma(x); gx = gmp.gamma(x); ax = apm.gamma(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.1332783894879E+06
gmp:  1.133278388948785567334574165588892475560E+06
apm:  1.133278388948785567334574165588892475560e+6 (1.062e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
```

(continues on next page)

(continued from previous page)

```
>>> \mathrm{d}z = dec.gamma(z); mz = mpm.gamma(z); iz = ipm.gamma(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 5.7016098526432799845E+5              + 1.9443478604345155482E+4j
mpm: 5.7016098526432799845e+5              + 1.9443478604345155482e+4j
ipm: 5.7016098526432799844e+5 (1.558e-18%) + 1.9443478604345155482e+4 (1.713e-18%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> fz = fpm.gamma(z); gz = gmp.gamma(z); az = apm.gamma(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 5.701609852643l6E+05                  + 1.94434786043447E+04j
gmp: 5.7016098526432799845E+05             + 1.9443478604345155482E+04j
apm: 5.7016098526432799845e+5 (4.284e-18%) + 1.9443478604345155482e+4 (8.065e-18%)j
```

Arguments can also be large. Note that the gamma function grows very quickly:

```
>>> from xlcalcnet import mp
>>> mp.dps = 25; mp.pretty = True
>>> mp.dps = 15
>>> gamma(10**20)
1.9328495143101e+1956570551809674817225
```

3.11.2 Auxiliary function $\Gamma(x+1)-1$

`ctx.gamma1pm1(`z`)`

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns $\Gamma(1+x)-1$, accurate also for x near 0.

See also Wikipedia [1258], MathWorld [884], NIST [22], BoostMath [101], Ehrhardt [297] (3.5.1.2)., Flint [739], Flint [729].

3.11.3 Log-gamma function, $\log\Gamma(x)$

`ctx.lgamma`(*x*)

where `ctx` is `math53`, `mathc53`, `ctxboost`, `ctxflint`.

Returns the principal branch of the log-gamma function, $\log|\Gamma(x)|$ for $x \neq 0, -1, -2, \ldots$. If $x < 0$, the logarithmic form of the reflection formula is used.

See also: https://en.wikipedia.org/wiki/Gamma_function#Log-gamma_function

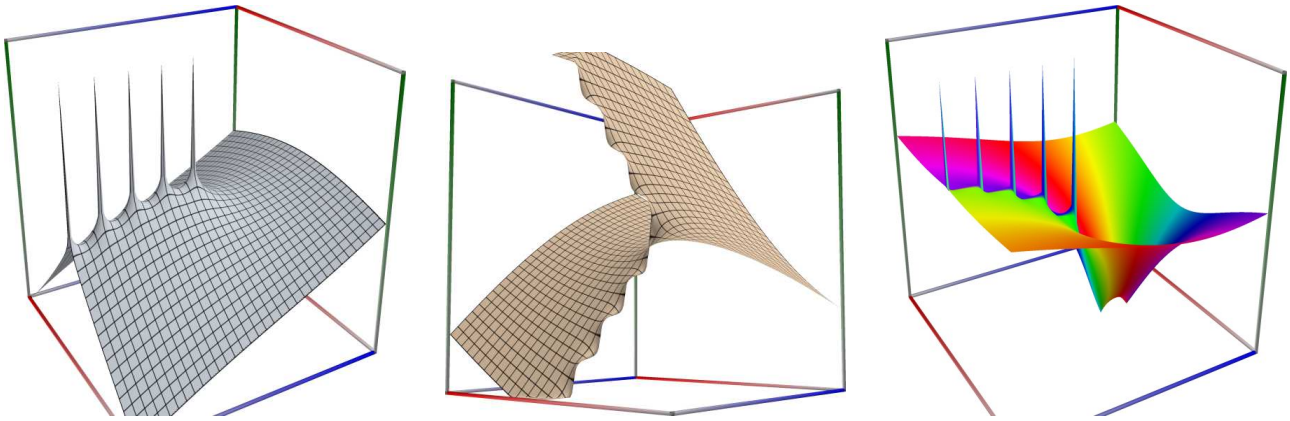
See also: https://mathworld.wolfram.com/LogGammaFunction.html

See also Wikipedia [1269], MathWorld [892], BoostMath [102], Ehrhardt [297] (3.5.1.5), Ehrhardt [297] (4.2.43), Flint [739], Flint [729], Mpmath [595].

Unlike $\log(\Gamma(z))$, which has infinitely many complex branch cuts, the principal log-gamma function only has a single branch cut along the negative half-axis. The principal branch continuously matches the asymptotic Stirling expansion

$$\log\Gamma(z) \sim \frac{\log(2\pi)}{2} + \left(z - \frac{1}{2}\right)\log(z) - z + O(z^{-1}).$$

The real parts of both functions agree, but their imaginary parts generally differ by $2n\pi$ for some $n \in \mathbb{Z}$. They coincide for $z \in \mathbb{R}, z > 0$.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LogGamma(1.5)
ereal('5.235987755982988730 7E-1')
>>> math53.LogGamma('1.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '10.5'
>>> \mathrm{d}x = dec.loggamma(x); mx = mpm.loggamma(x); ix = ipm.loggamma(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  1.3940625219403763633161237887971849479 80E+1
mpm:  1.3940625219403763633161237887971849479 80e+1
ipm:  1.3940625219403763633161237887971849479 80e+1 (6.588e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
```

(continues on next page)

(continued from previous page)

```
>>> mpm.dps = 40; x = '10.5'
>>> fx = fpm.loggamma(x); gx = gmp.loggamma(x); ax = apm.loggamma(x)
>>> mpm.show([fx, gx, ax])
fpm:  1.39406252194038E+01
gmp:  1.3940625219403763633161237887971849479 80E+01
apm:  1.3940625219403763633161237887971849479 80e+1 (1.318e-39%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> \mathrm{d}z = dec.loggamma(z); mz = mpm.loggamma(z); iz = ipm.loggamma(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.3254255156536136589E+1              + 3.4088524535204731293E-2j
mpm: 1.3254255156536136589e+1              + 3.4088524535204731293e-2j
ipm: 1.3254255156536136589e+1 (2.045e-19%) + 3.4088524535204731293e-2 (1.553e-19%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> fz = fpm.loggamma(z); gz = gmp.loggamma(z); az = apm.loggamma(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.32542551565361E+01                 + 3.40885245352047E-02j
gmp: 1.3254255156536136589E+01            + 3.4088524535204731293E-02j
apm: 1.3254255156536136589e+1 (4.09e-19%) + 3.4088524535204731293e-2 (6.989e-19%)j
```

Note the imaginary parts for negative arguments:

```
>>> from xlcalcnet import mpm
>>> mpm.dps = 25; mpm.pretty = True
>>> mpm.loggamma(-0.5); mpm.loggamma(-1.5); mpm.loggamma(-2.5)
(1.2655121234846453964889 46 - 3.1415926535897932384626 43j)
(0.8600470153764810145109327 - 6.2831853071795864769252 87j)
(-0.0562437164976740506725 9453 - 9.4247779607693797153879 3j)
```

Huge arguments are permitted:

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> loggamma('1e3000')
6.9067552789821370520539 74e+3003
>>> loggamma('1e100000000000000000000')
2.3025850929940456840079 91e+100000000000000000020
>>> loggamma('1e300j')
(-1.5707963267948966192313 22e+300 + 6.8977552789821370520539 74e+302j)
>>> loggamma('1e3000j')
(-1.5707963267948966192313 22e+3000 + 6.9067552789821370520539 74e+3003j)
```

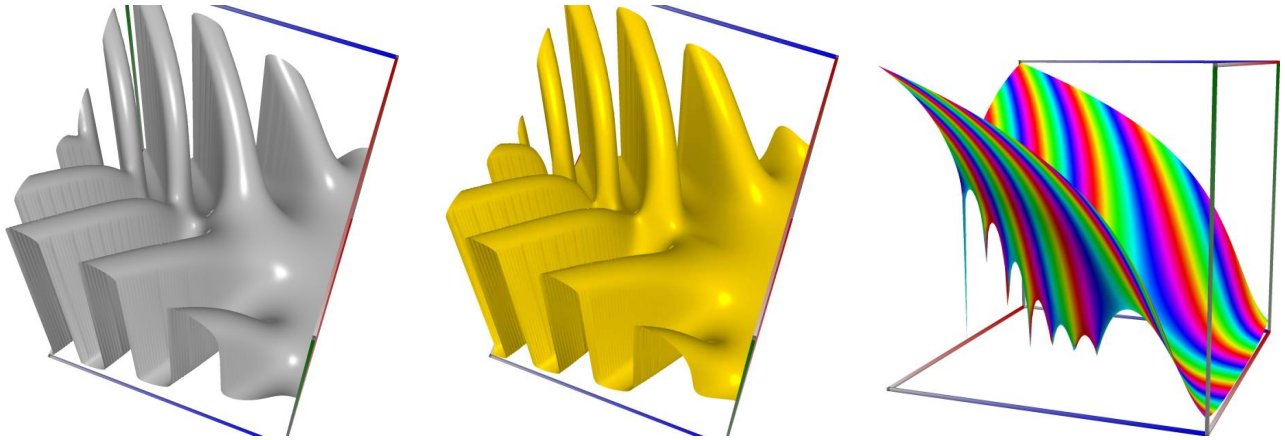
3.11.4 Reciprocal Gamma function, $1/\Gamma(x)$

`ctx.rgamma(x)`

where `ctx` is `math53`, `mathc53`, `ctxboost`, `ctxflint`.

Returns the reciprocal of gamma function x, $1/\Gamma(x)$, which is an entire function with simple zeros at the points $x = 0$ and the negative integers.

See also Wikipedia [1268], MathWorld [894], Ehrhardt [297] (3.5.1.8), Ehrhardt [297] (4.2.51), Flint [739], Flint [729].



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.RGamma(1)
1.0
>>> math53.RGamma(4)
0.16666666666666666666666667
>>> math53.RGamma(0); ereal.RGamma(-1)
0.0
0.0
>>> math53.RGamma(1000)
2.4851681432667848627835 96e-2565
>>> math53.RGamma('inf')
0.0
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '10.5'
>>> \mathrm{d}x = dec.rgamma(x); mx = mpm.rgamma(x); ix = ipm.rgamma(x)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  8.8239572002038009055094026242 56928377655E-7
mpm:  8.8239572002038009055094026242 56928377655e-7
ipm:  8.8239572002038009055094026242 56928377655e-7 (1.861e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10.5'
>>> fx = fpm.rgamma(x); gx = gmp.rgamma(x); ax = apm.rgamma(x)
>>> mpm.show([fx, gx, ax])
fpm:  8.8239572002038 0E-07
gmp:  8.8239572002038009055094026242 56928377655E-07
apm:  8.8239572002038009055094026242 56928377655e-7 (6.203e-40%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> \mathrm{d}z = dec.rgamma(z); mz = mpm.rgamma(z); iz = ipm.rgamma(z)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 1.7518533332561467304E-6              - 5.9741237445991417084E-8j
mpm: 1.7518533332561467304e-6              - 5.9741237445991417084e-8j
ipm: 1.7518533332561467304e-6 (4.749e-18%) - 5.9741237445991417085e-8 (-4.902e-18%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '10.2 + 1.5E-2j'
>>> fz = fpm.rgamma(z); gz = gmp.rgamma(z); az = apm.rgamma(z)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 1.75185333325618E-06                  - 5.97412374459925E-08j
gmp: 1.7518533332561467304E-06             - 5.9741237445991417084E-08j
apm: 1.7518533332561467304e-6 (4.242e-18%) - 5.9741237445991417084e-8 (-7.521e-18%)j
```

This function evaluates to zero at the poles of the gamma function, $z = 0, -1, -2, \ldots$.

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> rgamma(1)
1.0
>>> rgamma(4)
0.1666666666666666666666667
>>> rgamma(0); rgamma(-1)
0.0
0.0
>>> rgamma(1000)
2.4851681432667848627835960e-2565
>>> rgamma(inf)
0.0
```

3.11.5 Factorial, $x!$

`ctx.factorial(`x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $x!$, the factorial of x. For integers $x \geq 0$, we have $x! = 1 \cdot 2 \cdots (x-1) \cdot x$ and for real or complex x we have $x! = \Gamma(x+1)$.

See also Wikipedia [1256], MathWorld [882], BoostMath [99], Ehrhardt [297] (3.5.4.1), Flint [739], Flint [729], Mpmath [587].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Factorial(3)
ereal('5.235987755982988730 7E-1')
>>> math53.Factorial('0.51')
ereal('5.351847902755998475 4E-1')
```

3.11.6 Double factorial, $x!!$

`ctx.double_factorial(x)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $x!!$, the double factorial of x.

$$n!! = \begin{cases} 1 \cdot 3 \cdot 5 \cdots n & \text{if } n \text{ is odd.} \\ 2 \cdot 4 \cdot 6 \cdots n & \text{if } n \text{ is even.} \end{cases}$$

and more generally by

$$x!! = 2^{x/2} \left(\frac{\pi}{2}\right)^{(\cos(\pi x)-1)/4} \Gamma\left(\frac{x}{2}+1\right).$$

See also Wikipedia [1276], MathWorld [918], BoostMath [118], Ehrhardt [297] (3.5.4.2), Flint [739], Flint [729], Mpmath [607].

See also http://dlmf.nist.gov/5.4.E2

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.DFactorial(3)
ereal('5.235987755982988730 7E-1')
>>> math53.DFactorial('0.51')
ereal('5.3518479027559984754E-1')
```

3.11.7 Rising factorial $a^{\overline{n}} = (a)_n$

`ctx.rising_factorial(a, n)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the rising factorial , $a^{\overline{n}} = (a)_n = a(a+1)\cdots(a+n-1) = \dfrac{\Gamma(a+n)}{\Gamma(a)}$, where the rightmost expression is valid for nonintegral n. By convention $(a)_0 = 1$. Note that in Wikipedia [1257], the Pochhammer symbol $(a)_n$ is used for the falling factorial (as is common in combinatorics), whereas in this manual we follow the convention in MathWorld [895], Abramowitz and Stegun. [4], BoostMath [110], Ehrhardt [297], and the literature of special functions (in particular the hypergeometric functions), using it for the rising factorial.

If a or $a+n$ are negative integers or zero special care must be taken: If only a is a negative integer then the result is zero. If $a+n$ is also a negative integer then the Pochhammer symbol is computed from the limiting form of the Γ reflection formula $(a)_n = (-1)^n \dfrac{\Gamma(1-a)}{\Gamma(1-a-n)}$, and otherwise the function is undefined.

See also Wikipedia [1257], MathWorld [895], BoostMath [110], Ehrhardt [297] (3.5.4.6), Flint [739], Flint [729], Mpmath [599].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Pochhammer(13, 7)
ereal('5.2359877559829887307E-1')
>>> math53.RisingFactorial(12.6, '4.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '20.4'; n = '10.4'
>>> \mathrm{d}x = dec.rf(x, n); mx = mpm.rf(x, n); ix = ipm.rf(x, n)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  3.34337297397901888955468000837171771979653E+14
mpm:  3.34337297397901888955468000837171771979652e+14
ipm:  3.34337297397901888955468000837171771979655e+14 (1.836e-37%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '20.4'; n = '10.4'
>>> fx = fpm.rf(x, n); gx = gmp.rf(x, n); ax = apm.rf(x, n)
>>> mpm.show([fx, gx, ax])
fpm:  3.343372973979 01E+14
gmp:  3.34337297397901888955468000837171771979652E+14
apm:  3.34337297397901888955468000837171771979654e+14 (3.73e-37%)
```

The following example with complex input shows that the relative error can be high in double precision:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 20; z = '20.2 + 1.5E-2j'; n = '10.7 + 2.3E-1j'
>>> \mathrm{d}z = dec.rf(z, n); mz = mpm.rf(z, n); iz = ipm.rf(z, n)
>>> mpm.show([\mathrm{d}z, mz, iz], aligned=True)
dec: 5.9977312704618499868E+14              + 6.0755212828478581691E+14j
mpm: 5.9977312704618499868e+14              + 6.0755212828478581692e+14j
ipm: 5.9977312704618499868e+14 (2.178e-17%) + 6.0755212828478581692e+14 (2.292e-17%)j

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 20; z = '20.2 + 1.5E-2j'; n = '10.7 + 2.3E-1j'
>>> fz = fpm.rf(z, n); gz = gmp.rf(z, n); az = apm.rf(z, n)
>>> mpm.show([fz, gz, az], aligned=True)
fpm: 5.99773127046176E+14                   + 6.07552128284776E+14j
gmp: 5.9977312704618499868E+14              + 6.0755212828478581692E+14j
apm: 5.9977312704618499868e+14 (4.245e-17%) + 6.0755212828478581692e+14 (4.442e-17%)j
```

Evaluation is supported for arbitrary arguments:

```
>>> from xlcalcnet import mp
>>> mp.dps = 25; mp.pretty = True
>>> mp.rf(2+3j, 5.5)
(-7202.03920483347 - 3777.58810701527j)
```

3.11.8 Falling factorial, $(a)^{\underline{n}} = (a-n+1)^{\overline{n}}$

`ctx.falling_factorial`(a, n)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the falling factorial of a and n, $a^{\underline{n}} = a(a-1)\cdots(a-n+1) = \dfrac{\Gamma(a+1)}{\Gamma(a+1-n)}$, where the rightmost expression is valid for nonintegral n.

The falling factorial $(a)^{\underline{n}}$ is related to the rising factorial $(a)^{\overline{n}}$ by $(a)^{\underline{n}} = (a-n+1)^{\overline{n}}$.

See also Wikipedia [1257], MathWorld [883], BoostMath [100], Mpmath [588].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.FallingFactorial(13, 7)
ereal('5.235987755982988730 7E-1')
>>> math53.FallingFactorial(12.6, '4.51')
ereal('5.351847902755998475 4E-1')
```

3.11.9 Ratio of gamma functions, $\Gamma(a)/\Gamma(b)$

`ctx.gamma_ratio`(a, b)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

This functions returns the ratio of gamma functions in the form

$$\frac{\Gamma(a)}{\Gamma(b)}$$

See also BoostMath [126], Ehrhardt [297] (3.5.5).

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; a = '20.4'; b = '10.4'
>>> \mathrm{d}x = dec.gamma_ratio(a, b); mx = mpm.gamma_ratio(a, b); ix = ipm.gamma_ratio(a, b)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  4.4500053833434933493760000000000000000000E+11
mpm:  4.4500053833434933493760000000000000000000e+11
ipm:  4.4500053833434933493760000000000000000001e+11 (7.871e-38%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; a = '20.4'; b = '10.4'
>>> fx = fpm.gamma_ratio(a, b); gx = gmp.gamma_ratio(a, b); ax = apm.gamma_ratio(a, b)
>>> mpm.show([fx, gx, ax])
fpm:  4.45000538334346E+11
gmp:  4.4500053833434933493760000000000000000000E+11
apm:  4.4500053833434933493760000000000000000000e+11 (2.049e-37%)
```

3.11.10 Gamma-delta ratio, $\Gamma(a)/\Gamma(a+\delta)$

`ctx.gamma_delta_ratio`(x, *delta*)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the tgamma_ratio function of z and m. See also BoostMath [126].

Returns $\Gamma(x)/\Gamma(x+d)$, accurate even for $|d| << |x|$.

This functions returns the ratio of gamma functions in the form

$$\frac{\Gamma(a)}{\Gamma(a+\delta)}$$

Note that the result is calculated accurately even when δ is small compared to a: indeed even if $a + \delta \approx a$. The function is typically used when a is large and δ is very small.

Note: ctxboost.TgammaDeltaRatio(x, d)

Returns $\dfrac{\Gamma(x)}{\Gamma(x+d)}$, accurate also for $|d| \ll |x|$.

See also BoostMath [126], Ehrhardt [297] (3.5.5).

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; a = '2000.4'; d = '0.0004'
>>> \mathrm{d}z = dec.gamma_delta_ratio(a, d); mz = mpm.gamma_delta_ratio(a, d)
>>> iz = ipm.gamma_delta_ratio(a, d); fz = fpm.gamma_delta_ratio(a, d)
>>> gz = gmp.gamma_delta_ratio(a, d); az = apm.gamma_delta_ratio(a, d)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  9.969642761288672131226497458238654348190E-1
mpm:  9.969642761288672131226497458238654347429e-1
ipm:  9.969642761288672131226497458238654348320e-1 (2.681e-35%)
fpm:  9.96964276128867E-01
gmp:  9.969642761288672131226497458238654347429E-01
apm:  9.969642761288672131226497458238654348405e-1 (3.693e-35%)
```

3.11.11 Beta function, $B(a,b) = \Gamma(a)\Gamma(b)/\Gamma(a+b)$

`ctx.beta`(a, b)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the beta function $B(x,y) = \dfrac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)} = \displaystyle\int_0^1 t^{x-1}(1-t)^{y-1}\,\mathrm{d}t$.

See also Wikipedia [1252], MathWorld [880], NIST [21], BoostMath [95], Ehrhardt [297] (3.5.3.1), Mpmath [585].

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; a = '20.4'; b = '10.4'
>>> \mathrm{d}x = dec.beta(a, b); mx = mpm.beta(a, b); ix = ipm.beta(a, b)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  2.693713532046140908251383587041524146973E-9
mpm:  2.693713532046140908251383587041524146974e-9
ipm:  2.693713532046140908251383587041524146972e-9 (2.048e-37%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; a = '20.4'; b = '10.4'
>>> fx = fpm.beta(a, b); gx = gmp.beta(a, b); ax = apm.beta(a, b)
>>> mpm.show([fx, gx, ax])
fpm:  2.69371353204614E-09
gmp:  2.693713532046140908251383587041524146974E-09
apm:  2.693713532046140908251383587041524146975e-9 (6.549e-37%)
```

3.11.12 Binomial coefficient, ${}_nC_k = (k \cdot B(k, n-k+1))^{-1}$

`ctx.binomial`(n, k)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the binomial coefficient of n and k, ${}_nC_k = \binom{n}{k} = \frac{n!}{k!(n-k)!}$, for $k \geq 0$. More generally, the binomial coefficient is a well-defined function of arbitrary real or complex n and k, via the gamma function.

$$ {}_nC_k = \binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{\Gamma(n+1)}{\Gamma(k+1)\Gamma(n-k+1)} = \frac{1}{k \cdot B(k, n-k+1)}. $$

See also Wikipedia [1253], MathWorld [881], NIST [484], BoostMath [96], Ehrhardt [297] (3.5.4.4), Flint [739], Flint [729], Mpmath [586].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Binomial(13, 7)
ereal('5.2359877559829887307E-1')
>>> math53.Binomial(12.6, '4.51')
ereal('5.3518479027559984754E-1')
```

3.12 Miscellaneous functions

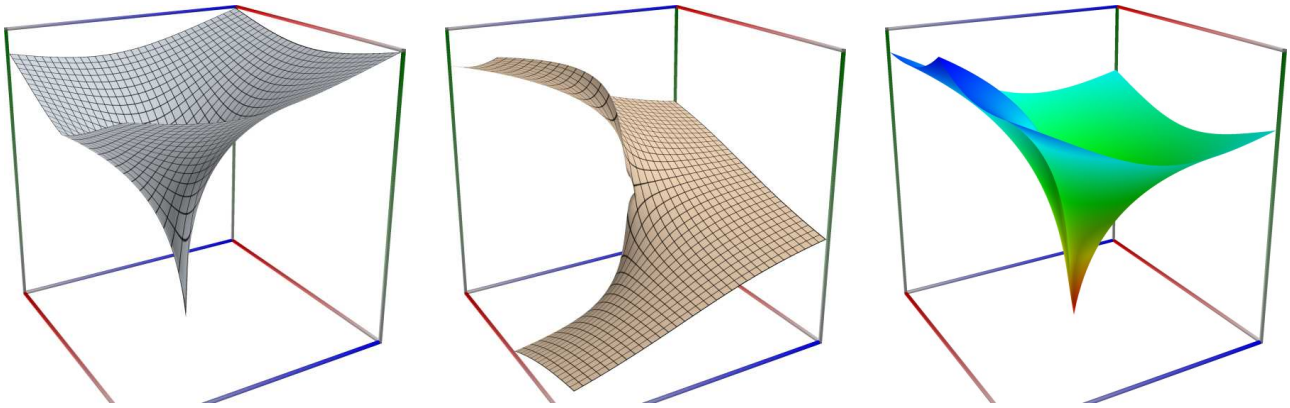
3.12.1 Lambert W function, $W_0(x)$

`ctx.lambert_w0(x)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $W_0(x)$, the real-valued principal branch of the Lambert W function for $x > -1/e$, with $W_0(x) \geq -1$ for $x < 0$. The Lambert W function is defined as the solution of $x = W(x)\exp(W(x))$.

See also Wikipedia [1262], MathWorld [891], NIST [488], BoostMath [106], Corless *et al.* [185], Ehrhardt [297] (3.10.15), Flint [744], Flint [734], Mpmath [594].



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LambertW(3.4)
ereal('5.235987755982988730 7E-1')
>>> math53.LambertW(13.4)
ereal('5.351847902755998475 4E-1')
```

Basic examples

The Lambert W function is the inverse of $w\exp(w)$:

```
>>> from xlcalcnet import *
>>> mp.dps = 25; mp.pretty = True
>>> w = lambertw(1)
>>> w
0.5671432904097838729999687
>>> w*exp(w)
1.0
```

Any branch gives a valid inverse:

```
>>> w = lambertw(1, k=3)
>>> w
(-2.8535817554090378072068 19 + 17.113535539412145912607 83j)
>>> w = lambertw(1, k=25)
>>> w
(-5.0470204642215697093786 86 + 155.4763860949415867162066j)
>>> chop(w*exp(w))
1.0
```

3.12.2 Lambert W function, $W_{-1}(x)$

`ctx.lambert_wm1(x)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $W_{-1}(x)$, the second real-valued, non-principal, branch of the Lambert W function for $-1/e < x < 0$, with $W_{-1}(x) \leq -1$ for $x < 0$. The Lambert W function is defined as the solution of $x = W(x)\exp(W(x))$, Flint [744], Flint [734].

See also Wikipedia [1262], MathWorld [891], NIST [488], BoostMath [106], Ehrhardt [297] (3.10.15), Mpmath [594].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LambertW1(0.2)
ereal('5.235987755982988730 7E-1')
>>> math53.LambertW1(0.21)
ereal('5.351847902755998475 4E-1')
```

Basic examples

The Lambert W function is the inverse of $w\exp(w)$:

```
>>> from xlcalcnet import *
>>> mp.dps = 25; mp.pretty = True
>>> w = lambertw(1)
>>> w
0.56714329040978387299997
>>> w*exp(w)
1.0
```

Any branch gives a valid inverse:

```
>>> w = lambertw(1, k=3)
>>> w
(-2.85358175540903780720 6819 + 17.113535539412145912607 83j)
>>> w = lambertw(1, k=25)
>>> w
(-5.04702046422156970937 8686 + 155.47638609494158671620 66j)
>>> chop(w*exp(w))
1.0
```

3.12.3 Lambert W (general case)

`ctx.lambert_wk(z, k=0)`

where `ctx` is `math53`, `ctxboost`, `ctxflint`.

Returns the Lambertw function of z.

See also BoostMath [106], Wikipedia [1262], MathWorld [891], NIST [488], Corless *et al.* [185], Ehrhardt [297] (4.2.39), Flint [744], Flint [734], Mpmath [594].

The Lambert W function $W(z)$ is defined as the inverse function of $w\exp(w)$. In other words, the value of $W(z)$ is such that $z = W(z)\exp(W(z))$ for any complex number z.

The Lambert W function is a multivalued function with infinitely many branches $W_k(z)$, indexed by $k \in \mathbb{Z}$. Each branch gives a different solution w of the equation $z = w\exp(w)$. All branches are supported:

- `lambertw(z)` gives the principal solution (branch 0)
- `lambertw(z, k)` gives the solution on branch k

The Lambert W function has two partially real branches: the principal branch ($k = 0$) is real for real $z > -1/e$, and the $k = -1$ branch is real for $-1/e < z < 0$. All branches except $k = 0$ have a logarithmic singularity at $z = 0$.

The definition, implementation and choice of branches is based on Corless *et al.* [185].

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.LambertW(0.5, 2)
ecplx('5.2359877559829887307E-1')
>>> ecplx.LambertW('0.1')
ecplx('5.3518479027559984754E-1')
```

Basic examples

The Lambert W function is the inverse of $w \exp(w)$:

```
>>> from xlcalcnet import *
>>> mp.dps = 25; mp.pretty = True
>>> w = lambertw(1)
>>> w
0.56714329040978387299996866
>>> w*exp(w)
1.0
```

Any branch gives a valid inverse:

```
>>> w = lambertw(1, k=3)
>>> w
(-2.8535817554090378072068189 + 17.113535539412145912607834j)
>>> w = lambertw(1, k=25)
>>> w
(-5.0470204642215697093786863 + 155.47638609494158671620662j)
>>> chop(w*exp(w))
1.0
```

3.12.4 Arithmetic-geometric mean

`ctx.agm`(*x*, *y=1*)

where `ctx` is `math53`, `mathc53`, `ctxboost` or `ctxflint`.

Returns the arithmetic-geometric mean of $|x|$ and $|y|$. See also Wikipedia [1272], MathWorld [913], Ehrhardt [297] (4.2.1), Ehrhardt [297] (4.2.2), Mpmath [601].

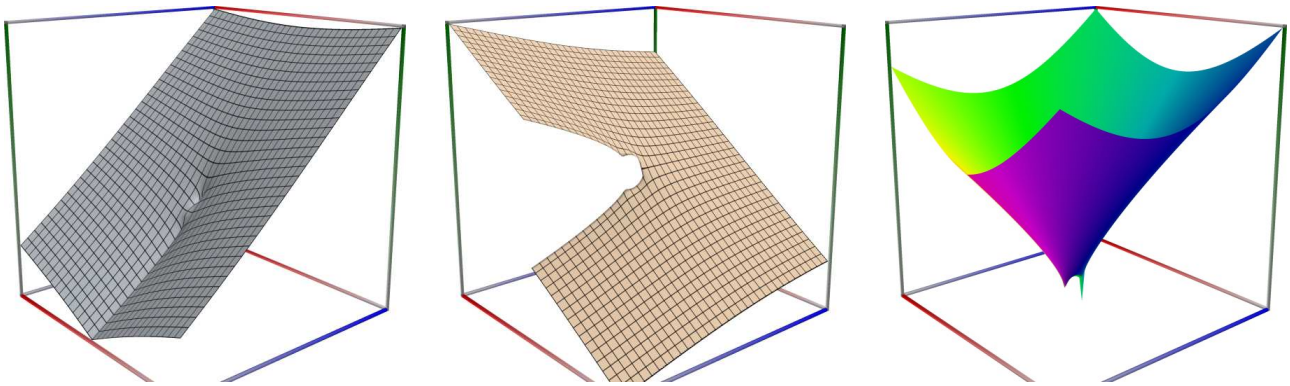
With $a_0 = \max(|x|, |y|)$, $b_0 = \min(|x|, |y|)$ the AGM is calculated using the recurrence formulas

$$a_{n+1} = \tfrac{1}{2}(a_n + b_n), \quad b_{n+1} = \sqrt{a_n b_n}.$$

The sequences converge quadratically to a common limit and $a_n \geq b_n$. The iteration is terminated if $a_n - b_n \leq \epsilon a_n$, where ϵ is less than the square root of the machine epsilon. The result is $(a_n + b_n)/2$.

The function $\mathrm{agm}(a, b)$ can also be expressed in closed form in terms of the complete elliptic integral of the first kind $K(k)$ as

$$\mathrm{agm}(a, b) = \frac{(a+b)\pi}{4K((a-b)/(a+b))}.$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Agm(0.5, 2)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Agm('0.1')
ecplx('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = '0.6'; b = '0.3'
>>> \mathrm{d}x = dec.agm(a, b); mx = mpm.agm(a, b); gx = gmp.agm(a, b)
>>> fx = fpm.agm(a, b); ax = apm.agm(a, b)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.3703730931407206075592971497952459249 22E-1
mpm:  4.3703730931407206075592971497952459249 22e-1
gmp:  4.3703730931407206075592971497952459249 22E-01
fpm:  4.37037309314072E-01
apm:  4.3703730931407206075592971497952459249 22e-1 (1.97e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '5.0 + 2.0j'; b = '7.0 + 3.0j'
>>> \mathrm{d}z = dec.agm(a, b); mz = mpm.agm(a, b); gz = gmp.agm(a, b)
>>> fz = fpm.agm(a, b); az = apm.agm(a, b)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 5.9582264514273988109E+0              + 2.4753102663993183285E+0j
mpm: 5.9582264514273988109e+0              + 2.4753102663993183285e+0j
gmp: 5.9582264514273988109E+00             + 2.4753102663993183285E+00j
fpm: 5.95822645142740E+00                  + 2.47531026639932E+00j
apm: 5.9582264514273988109e+0 (1.137e-19%) + 2.4753102663993183285e+0 (1.369e-19%)j
```

CHAPTER

FOUR

STATISTICAL DISTRIBUTIONS

For a general introduction to probability distributions, see: Wikipedia [1136], Wikipedia [1128], Wikipedia [1126], Wikipedia [1143].

The design of this library has been influenced by Bristow *et al.* [159], Witkovsky [1458].

See also Wikipedia [1123] and Abramowitz and Stegun. [4] (from which the following introductory text has been taken, with small edits).

A real-valued function $F(x)$ is termed a (univariate) cumulative distribution function (cdf) or simply distribution function if

1. $F(x)$ is non-decreasing, i.e. $F(x_1) \leq F(x_2)$ for $x_1 \leq x_2$.
2. $F(x)$ is everywhere continuous from the right, i.e. $F(x) = \lim_{\epsilon \to 0+} F(x + \epsilon)$
3. $F(-\infty) = 0, F(\infty) = 1$.

The function $F(x)$ signifies the probability of the event "$X \leq x$", i.e. Pr $\{X \leq x\} = F(x)$, where X is a random variable, and thus describes the cdf of X. The two principle types of distribution functions are termed *discrete* and *continuous*.

Discrete distributions: Discrete distributions are characterized by the random variable X taking on an enumerable number of values $\ldots x_{-1}, x_0, x_1, \ldots$ with point probabilities $p_n = \text{Pr}\{X = x_n\} \geq 0$, which only need to be subject to the restriction $\sum_n p_n = 1$. The corresponding distribution function can then be written as

$$F(x) = \text{Pr}\{X \leq x\} = \sum_{x_n \leq x} p_n,$$

where the summation is over all values of x for which $x_n \leq x$. The set $\{x_n\}$ of values for which $p_n > 0$ is termed the domain of he random variable X. A discrete distribution of a random variable is called a *lattice distribution* if there exists numbers a and $b \neq 0$ such that every possible value of X can be represented in the form $a + bn$ where n takes only integral values.

Continuous distributions: Continuous distributions are characterized by $F(x)$ being continuous. If $F(x)$ is absolutely continuous then $F(x)$ possesses a derivative $F'(x) = f(x)$ and the cdf can be written as

$$F(x) = \text{Pr}\{X \leq x\} = \int_{-\infty}^{x} f(t)\text{d}t.$$

The derivative $f(x)$ is termed the probability density function (pdf), and the values of x for which $f(x) > 0$ make up the domain of the variable X.

The source code for the C# tests in Visual Studio can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B04_Distributions.cs.

4.1 Introduction to random variables and distributions

4.1.1 Elementary symbolic algebra of random variables

See also https://en.wikipedia.org/wiki/Algebra_of_random_variables.

Considering two random variables X and Y the following algebraic operations are possible:

Addition: $Z = X + Y = Y + X$.

Subtraction: $Z = X - Y = -Y + X$.

Multiplication: $Z = XY = YX$.

Division: $Z = X/Y = X \cdot (1/Y) = (1/Y) \cdot X$.

In all cases, the variable Z resulting from each operation is also a random variable. All commutative and associative properties of conventional algebraic operations are also valid for random variables. If any of the random variables is replaced by a deterministic variable or by a constant value, all the previous properties remain valid.

Simple tranformations, results for pdf and cdf: see Rinne [480], page 217.

This is a summary of some transformations:

Table 1: Some transformations

Transformation $Y = g(X)$	pdf $f_Y(y)$	cdf $F_Y(y)$
$Y = a + bX; \quad a, x \in \mathbb{R}; b \neq 0.$	$= \frac{1}{\lvert b \rvert} f_X\left(\frac{y-a}{b}\right).$	$= \begin{cases} F_X\left(\frac{y-a}{b}\right) & \text{for } b > 0, \\ 1 - F_X\left(\frac{y-a}{b}\right) & \text{for } b < 0. \end{cases}$
$Y = X^r; \quad r \in \mathbb{N}, x \in \mathbb{R}.$	$= \frac{1}{r} y^{\frac{1}{r}-1} f_X\left(\sqrt[r]{y}\right).$	$= F_X\left(\sqrt[r]{y}\right).$
$Y = aX^k; \quad a, k \in \mathbb{R} \setminus \{0\}; \quad x > 0.$	$= \frac{1}{\lvert ka \rvert} \left(\frac{y}{a}\right)^{\frac{1}{k}-1} f_X\left(\sqrt[k]{\frac{y}{a}}\right)$	$= \begin{cases} F_X\left(\sqrt[k]{\frac{y}{a}}\right) & \text{for } a > 0, \\ 1 - F_X\left(\sqrt[k]{\frac{y}{a}}\right) & \text{for } a < 0. \end{cases}$
$Y = a\sqrt[k]{X}; \quad a, k \in \mathbb{R} \setminus \{0\}; \quad x > 0.$	$= \frac{k}{\lvert a \rvert} \left(\frac{y}{a}\right)^{k-1} f_X\left(\left[\frac{y}{a}\right]^k\right)$	$= \begin{cases} F_X\left(\left[\frac{y}{a}\right]^k\right) & \text{for } a > 0, \\ 1 - F_X\left(\left[\frac{y}{a}\right]^k\right) & \text{for } a < 0. \end{cases}$
$Y = a/X; \quad a \in \mathbb{R} \setminus \{0\}; \quad x > 0.$	$= \frac{\lvert a \rvert}{y^2} f_X\left(\frac{a}{y}\right)$	$= \begin{cases} F_X\left(\frac{a}{y}\right) & \text{for } a > 0, \\ 1 - F_X\left(\frac{a}{y}\right) & \text{for } a < 0. \end{cases}$

Please see the above Table. *The linear transformation*.

For more detailed result: see Rinne [480], p. 218-219.

4.1.2 Linear transformations of random variables

All distributions have location (L) and Scale (S) parameters along with any other parameters needed. The distributions are given in standard forms where $L = 0$ and $S = 1$. The nonstandard forms can be obtained for the various functions using the following transformations:

Table 2: The linear transformation

Function	Transformation
random variates	$\text{rnd}_X(L, S) = L + S \cdot \text{rnd}X(0, 1)$
cumulative distribution function	$\text{cdf}_X(x; L, S) = \text{cdf}_X\left(\frac{x-L}{S}; 0, 1\right)$
probability density function	$\text{pdf}_X(x; L, S) = \frac{1}{S} \cdot \text{pdf}_X\left(\frac{x-L}{S}; 0, 1\right)$
quantile function	$\text{qtf}_X(q; L, S) = L + S \cdot \text{qtf}_X(q; 0, 1)$
probability sparsity function	$\text{psf}_X(q; L, S) = S \cdot \text{psf}_X(q; 0, 1)$
hazard function	$\text{hf}_X(x; L, S) = \frac{1}{S} \cdot \text{hf}_X\left(\frac{x-L}{S}; 0, 1\right)$
cumulative hazard function	$\text{chf}_X(x; L, S) = \text{chf}_X\left(\frac{x-L}{S}; 0, 1\right)$
survival function	$\text{sf}_X(x; L, S) = \text{sf}_X\left(\frac{x-L}{S}; 0, 1\right)$
inverse survival function	$\text{isf}_X(p; L, S) = L + S \cdot \text{isf}_X(p; 0, 1)$
moment generating function	$M_X(t; L, S) = e^{t \cdot L} \cdot M_X(t \cdot S; 0, 1)$
characteristic function	$C_X(t; L, S) = e^{it \cdot L} \cdot C_X(t \cdot S; 0, 1)$
cumulant generating function	$K_X(t; L, S) = t \cdot L + K_X(t \cdot S; 0, 1)$
differential entropy	$\text{h}_X(L, S) = \text{h}_X(0, 1) + \log(S)$
mean	$\mu_X(1; L, S) = L + S \cdot \mu_X(1; 0, 1)$
central moments	$\mu_X(r; L, S) = S^r \cdot \mu_X(r; 0, 1); r > 1$
cumulants	$\kappa_X(r; L, S) = S^r \cdot \kappa_X(r; 0, 1); r > 1$

Please see the above Table. *The linear transformation*.

For more detailed result: see Rinne [480], p. 218-219.

Examples

- Standard normal distribution
- Ch-squared distribution with different scales

4.1.3 Random variables having mixture distributions

A random variable follows a mixture distribution if its cumulative distribution function (and the probability density function if it exists) can be expressed as a convex combination (i.e. a weighted sum, with non-negative weights w_i that sum to 1) of other distribution functions and density functions. The individual distributions that are combined to form the mixture distribution are called the mixture components, and the probabilities (or weights) associated with each component are called the mixture weights w_i. See also Wikipedia [1160].

Table 3: The mixture distribution

Function	Transformation
random variates	see text below
cumulative distribution function	$\text{cdf}_Z(x) = \sum_{i=1}^{k} w_i \cdot \text{cdf}_{X_i}(x)$
probability density function	$\text{pdf}_Z(x) = \sum_{i=1}^{k} w_i \cdot \text{pdf}_{X_i}(x)$
probability mass function	$\text{pmf}_Z(x) = \sum_{i=1}^{k} w_i \cdot \text{pmf}_{X_i}(x)$
survival function	$\text{sf}_Z(x) = \sum_{i=1}^{k} w_i \cdot \text{sf}_{X_i}(x)$
moment generating function	$M_Z(t) = \sum_{i=1}^{k} w_i \cdot M_{X_i}(t)$
characteristic function	$C_Z(t) = \sum_{i=1}^{k} w_i \cdot C_{X_i}(t)$
raw moments	$\mu'_Z(r) = \sum_{i=1}^{k} w_i \cdot \mu'_{X_i}(x)$

Please see the above Table. *The mixture distribution*.

The cumulants and the central moments can be calculated from the raw moments.

The cumulant generating function can be calculated from the moment generating function.

Examples

- Skellam: poisson with same rate back to back
- Laplace: exponential with same rate back to back
- Asymmetric Laplace: exponential with different rates back to back
- Hyperexponential distribution

4.1.4 Linear combinations of independent random variables

Suppose Z is the sum of n independent random variables $X_1, \ldots, X_n$ each with probability mass functions $f_{X_i}(x)$. Then $Z = \sum_{i=1}^{n} X_i$ is distributed as the convolution of the distributions of the X_i.

See also: Wikipedia [1161].

In cases where there are no known expressions in terms of other, readily computed distributions, the distribution of Z can be calculated using the characteristic functions of the X_i, as shown in the table below.

Table 4: The sums of random variables

Function	Transformation
random variates	$\text{rnd}_Z() = \sum_{i=1}^{k} a_i \cdot \text{rnd}_{X_i}()$
moment generating function	$M_Z(t) = \prod_{i=1}^{k} M_{X_i}(a_i \cdot t)$
characteristic function	$C_Z(t) = \prod_{i=1}^{k} C_{X_i}(a_i \cdot t)$
cumulant generating function	$K_Z(t) = \sum_{i=1}^{k} K_{X_i}(a_i \cdot t)$
cumulants	$\kappa_Z(r) = \sum_{i=1}^{k} a_i^r \cdot \kappa_{X_i}(r)$

Please see the above Table. *The sums of random variables*.

The raw moments and central moments are calculated from the cumulants.

The pmf, pdf and cdf are calculated by methods 1-3.

The qtf is started via the cumulants (Cornish-Fisher), followed by Newton iterations.

The pmf-vector is calculated via discrete convolution.

4.1.5 The log-transform of continuous random variables

A random variable X has a log-transform Y (which also is a random variable), if $\text{cdf}_X(x) = \text{cdf}_Y(\log(x))$, in other words if $\log(X) \sim Y$ (where $\sim$ means "is distributed as"). The transformation is only useful if the domain of X is (a, b) with $0 \leq a < b \leq \infty$ (otherwise the domain of Y would be extended into the complex plane). It is only implemented for continuous distributions.

The pdf, cdf, sf, qtf and isf of X and Y can be expressed in terms of each other, as shown below, together with the remarkable fact that the characteristic function of Y can be expressed in terms of the raw moments of X (this assumes that the raw moments of X are available in a form which allows to replace the discrete index variable r with the continuous complex variable it).

Table 5: The logarithmic transformation of a variable

Function	Transformation
random variates	$\text{rnd}_Y() = \log(\text{rnd}_X())$
	$\text{rnd}_X() = \exp(\text{rnd}_Y())$
probability density function	$\text{pdf}_Y(t) = \text{pdf}_X(\exp(t)) \cdot \exp(t)$
	$\text{pdf}_X(u) = \text{pdf}_Y(\log(u)) \cdot \frac{1}{u}$
cumulative distribution function	$\text{cdf}_Y(t) = \text{cdf}_X(\exp(t))$
	$\text{cdf}_X(u) = \text{cdf}_Y(\log(u))$
survival function	$\text{sf}_Y(t) = \text{sf}_X(\exp(t))$
	$\text{cdf}_X(u) = \text{cdf}_Y(\log(u))$
quantile function	$\text{qtf}_Y(q) = \exp(\text{qtf}_X)$
	$\text{qtf}_X(q) = \log(\text{qtf}_Y)$
inverse survival function	$\text{isf}_Y(q) = \exp(\text{isf}_X)$
	$\text{isf}_X(q) = \log(\text{isf}_Y)$
characteristic function of Y and raw moments of X	$C_Y(t) = \mu'_X(it)$

Please see the above Table. *The logarithmic transformation of a variable*.

One of the major features of the log-transform is its ability to express the distribution of the product of variables X_i as the sum of their log-transforms Y_i.

Examples

- Lognormal distribution

X: Lognormal

Y: Standard normal

- Beta distribution

4.2 Base class for univariate distributions

class rv_base(*rv_base*)

This is the base class, from which `rv_continuous` and `rv_discrete` inherit. There are no parameters and the support interval is $(-\infty, \infty)$.

The constructor has the following form:

```
class rv_continuous(rv_base):
    __a = -mp.inf
    __b = +mp.inf

    def __init__(self, df):
        pass
```

4.2.1 Boost: Cumulative distribution function

rv_base.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X:

See also Wikipedia [1123].

There are various ways to calculate a cdf. Ideally, it is available in closed from and can be calculated directly, or there is are dedicated functions to calculate it. If this is not the case, then one of the options given below can be used. There is no method which is always best, and the choice depends not only on the function, but also on the values of the argument and the parameters.

rv_base.**set_cdf_method**(*cdf_method*)

Sets the method to be used for the calculation of the cdf. `cdf_method` can take the following values:

1. "direct": this is the default if a closed form of the cdf exists, otherwise returns `not implemented`.
2. "from_pdf": only for continuous functions for which the pdf is available in closed form, otherwise returns `not implemented`.
3. "from_pmf": only for discrete functions, for which the pmf vector is available, otherwise returns `not implemented`.
4. "from_charfunc": only for functions for which the characteristic is available in closed form, otherwise returns `not implemented`.
5. "from_edgeworth": only if at least the first 2 cumulants exist, otherwise returns `not implemented`.
6. "from_lug_rice": only if the cumulant generating function exists, otherwise returns `not implemented`.

There may be additional options for any particular function, which are documented in the decription of the function.

4.2.2 Boost: Survival function

rv_base.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X.

See also Wikipedia [1142].

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x) = \int_x^\infty \text{pdf}_X(x)\text{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145561356E+00
```

4.2.3 Hazard function

`rv_continuous.`**`hazard`**`(`x`)`

Returns $\text{hazard}_X(x)$, the hazard function of a random variable X, defined as the ratio of the probability density function to the survival function.

See also Wikipedia [1127], BoostMath [46].

$$\text{hazard}_X(x) = \frac{\text{pdf}_X(x)}{\text{sf}_X(x)} = \frac{\text{pdf}_X(x)}{1 - \text{cdf}_X(x)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

4.2.4 Cumulative Hazard Function

`rv_continuous.`**`chf`**`(`x`)`

Returns $\text{chf}_X(x)$, the cumulative function of a random variable X, defined as the ratio of the probability density function to the survival function.

See also Wikipedia [1127], BoostMath [39].

$$\text{chf}_X(x) == \int_{-\infty}^{x} \text{hazard}_X(x)\text{d}t = -\log(1 - \text{cdf}_X(x)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

4.2.5 Boost: Quantile function

`rv_base.`**`qtf`**`(`x`)`

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X.

$$\text{qtf}_X(q) = \text{cdf}_X^{-1}(q).$$

See also Wikipedia [1137].

There are various ways to calculate the quantile function. Ideally, it is available in closed from and can be calculated directly, or there is are dedicated functions to calculate it. If this is not the case, then one of the options given below can be used. There is no method which is always best, and the choice depends not only on the function, but also on the values of the argument and the parameters.

`rv_base.`**`set_qtf_method`**`(`*qtf_method*`)`

Sets the method to be used for the calculation of the cdf. `qtf_method` can take the following values:

1. "direct": this is the default if a closed form of the quantile function exists.
2. "from_cdf": this is the default if a closed form of the qtf does not exist.
3. "from_cornish_fisher": only if at least the first 2 cumulants exist, otherwise returns `not implemented`.
4. "from_jensen": only if the cumulant generating function exists, otherwise returns `not implemented`.

There may be additional options for any particular function, which are documented in the decription of the function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

4.2.6 Boost: Inverse survival function

`rv_base.isf(x)`

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X.

See also Wikipedia [1137]

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

4.2.7 Boost: Mode

`rv_base.mode()`

Returns the mode of the function.

See also Wikipedia [1132]

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

4.2.8 Boost: Expected value (mean)

`rv_base.mean()`

Returns the expected value (mean) of the function.

See also Wikipedia [1125].

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

4.2.9 Boost: Median

`rv_base.median()`

Returns the median of the function.

See also Wikipedia [1131]

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

4.2.10 Boost: Variance

`rv_base.variance()`

Returns the variance of the function.

See also Wikipedia [1144].

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

4.2.11 Boost: Standard deviation

`rv_base.stdev()`

Returns the Standard deviation of the function.

See also Wikipedia [1140].

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

4.2.12 Boost: Skewness

`rv_base.skewness()`

Returns the skewness of the function.

See also Wikipedia [1139].

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

4.2.13 Boost: Kurtosis

`rv_base.kurtosis()`

Returns the 'proper' kurtosis (normalized fourth moment) of the distribution dist.kurtosis μ_4/μ_2^2, where μ_i is the i'th central moment of the distribution, and in particular μ_2 is the variance of the distribution.

See also Wikipedia [1130].

The kurtosis is a measure of the "peakedness" of a distribution.

Note that the literature definition of kurtosis is confusing. The definition used here is that used by for example Wolfram MathWorld (that includes a table of formulae for kurtosis excess for various distributions) but NOT the definition of kurtosis used by Wikipedia which treats "kurtosis" and "kurtosis excess" as the same quantity.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456l356E+00
```

4.2.14 Boost: Kurtosis excess

`rv_base.`**`excess_kurtosis`**`()`

Returns the kurtosis excess of the distribution dist. kurtosis excess = kurtosis - 3.

See also Wikipedia [1124].

The kurtosis excess is a measure of the "peakedness" of a distribution, and is more widely used than the "kurtosis proper". It is defined so that the kurtosis excess of a normal distribution is zero.

This function may return a domain_error if the distribution does not have a defined kurtosis excess.

Kurtosis excess can have a value from -2 to + infinity.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456l356E+00
```

4.2.15 Boost: Support, lower endpoint

`Dist.`**`support_lower_endpoint`**`()`

Returns the lower endpoint of the support of the function.

See also Wikipedia [1141].

See also https://en.wikipedia.org/wiki/Interval_(mathematics)

The distribution is said to be 'supported' over a range that is "the smallest closed set whose complement has probability zero". Non-mathematicians might say it means the 'interesting' smallest range of random variate x that has the cdf going from zero to unity. Outside are uninteresting zones where the pdf is zero, and the cdf zero or unity.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456l356E+00
```

4.2.16 Boost: Support, upper endpoint

`Dist.`**`support_upper_endpoint`**`()`

Returns the upper endpoint of the support of the function.

See also Wikipedia [1141].

See also https://en.wikipedia.org/wiki/Interval_(mathematics)

The distribution is said to be 'supported' over a range that is "the smallest closed set whose complement has probability zero". Non-mathematicians might say it means the 'interesting' smallest range of random variate x that has the cdf going from zero to unity. Outside are uninteresting zones where the pdf is zero, and the cdf zero or unity.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

4.2.17 Boost: Range, lower endpoint

`Dist.`**`range_lower_endpoint`**`()`

Returns the lower endpoint of the valid range of the random variable.

See also Wikipedia [1141].

See also https://en.wikipedia.org/wiki/Interval_(mathematics)

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

4.2.18 Boost: Range, upper endpoint

`Dist.`**`range_upper_endpoint`**`()`

Returns the upper endpoint of the valid range of the random variable.

See also Wikipedia [1141].

See also https://en.wikipedia.org/wiki/Interval_(mathematics)

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

4.2.19 Characteristic function

`rv_base.`**`c_x`**`(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X.

See also Wikipedia [1121], Wikipedia [1125].

For a scalar random variable X the characteristic function is defined as the expected value of e^{itX}, where i is the imaginary unit, and $t \in \mathbb{R}$ is the argument of the characteristic function:

$$C_X(t) = \mathrm{E}\left[e^{itX}\right] = \int_{-\infty}^{\infty} e^{itx} d\,\mathrm{cdf}_X(x) = \int_{-\infty}^{\infty} e^{itx} \mathrm{pdf}_X(x) \mathrm{d}x = \int_0^1 e^{it\ \mathrm{qtf}_X(p)} dp = M_X(it).$$

Here $\mathrm{cdf}_X(x)$ is the cumulative distribution function of X, and the integral is of the Riemann–Stieltjes kind. If a random variable X has a probability density function $\mathrm{pdf}_X(x)$, then the characteristic function is its Fourier transform with sign reversal in the complex exponential. $\mathrm{qtf}_X(p)$ is the quantile function of X, also known as the inverse cumulative distribution function of X or the quantile function of X. $M_X(t)$ is the moment generating function of X. Note that the characteristic function of a distribution always exists, even when the probability density function or moment-generating function do not.

There are various ways to calculate the characteristic function. Ideally, it is available in closed from and can be calculated directly, or there is are dedicated functions to calculate it. If this is not the case, then one of the options given below can be used. There is no method which is always best, and the choice depends not only on the function, but also on the values of the argument and the parameters.

rv_base.**set_cf_method**(*qtf_method*)

Sets the method to be used for the calculation of the cdf. `qtf_method` can take the following values:

1. "direct": this is the default if a closed form of the characteristic function exists.
2. "from_pdf": only if the pdf is available in closed form, otherwise returns `not implemented`.
3. "from_qtf": only if the qtf is available in closed form, otherwise returns `not implemented`.
4. "from_moments": only if the raw moments are available in closed form, otherwise returns `not implemented`.

There may be additional options for any particular function, which are documented in the decription of the function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

4.2.20 Moment generating function

rv_base.**m_x**(x)

Returns $M_X(t)$, the moment generating function of a random variable X.

See also Wikipedia [1134].

There are various ways to calculate the moment generating function. Ideally, it is available in closed from and can be calculated directly, or there is are dedicated functions to calculate it. If this is not the case, then one of the options given below can be used. There is no method which is always best, and the choice depends not only on the function, but also on the values of the argument and the parameters.

rv_base.**set_mgf_method**(*mgf_method*)

Sets the method to be used for the calculation of the mgf. `mgf_method` can take the following values:

1. "direct": this is the default if a closed form of the moment generating function exists.
2. "from_charfunc": only if the characteristic is available in closed form, otherwise returns `not implemented`.
3. "from_cgf": only if the moment generating function is available in closed form, otherwise returns `not implemented`.
4. "from_pgf": only for discrete distributions and only if the probability generating function is available in closed form, otherwise returns `not implemented`.
5. "from_moments": only if the raw moments are available in closed form, otherwise returns `not implemented`.

There may be additional options for any particular function, which are documented in the decription of the function.

Related to the moment-generating function are a number of other transforms that are common in probability theory:

4.2.21 Cumulant generating function

rv_base.**k_x**(x)

Returns $K_X(t)$, the cumulant generating function of a random variable X.

See also Wikipedia [1122].

There are various ways to calculate the cumulant generating function. Ideally, it is available in closed from and can be calculated directly, or there is are dedicated functions to calculate it. If this is not the case, then one of the options given below can be used. There is no method which is always best, and the choice depends not only on the function, but also on the values of the argument and the parameters.

rv_base.**set_cgf_method**(*cgf_method*)

Sets the method to be used for the calculation of the cgf. `cgf_method` can take the following values:

1. "direct": this is the default if a closed form of the moment generating function exists.
2. "from_charfunc": only if the characteristic function is available in closed form, otherwise returns `not implemented`.
3. "from_mgf": only if the moment generating function is available in closed form, otherwise returns `not implemented`.
4. "from_pgf": only for discrete distributions and only if the probability generating function is available in closed form, otherwise returns `not implemented`.
5. "from_cumulants": only if the cumulants are available in closed form, otherwise returns `not implemented`.

There may be additional options for any particular function, which are documented in the decription of the function.

4.2.22 Raw Moments

rv_base.**raw_moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \dots k$, of a random variable X.

See also Wikipedia [1133]

There are various ways to calculate the raw moments. Ideally, they are available in closed from and can be calculated directly, or there is are dedicated functions to calculate them. If this is not the case, then one of the options given below can be used. There is no method which is always best.

rv_base.**set_moments_method**(*moments_method*)

Sets the method to be used for the calculation of the raw moments. `moments_method` can take the following values:

1. "direct": this is the default if a closed form of the cdf exists, otherwise returns `not implemented`.
2. "from_pdf": only for continuous functions for which the pdf is available in closed form, otherwise returns `not implemented`.
3. "from_pmf": only for discrete functions, for which the pmf vector is available, otherwise returns `not implemented`.
4. "from_cumulants": only if the cumulants are available in closed form, otherwise returns `not implemented`.
5. "from_mgf": only for functions for which the moment generating function is available in closed form, otherwise returns `not implemented`.
6. "from_charfunc": only for functions for which the characteristic function is available in closed form, otherwise returns `not implemented`.
7. "from_pgf": only for discrete distributions and only for functions for which the moment generating function is available in closed form, otherwise returns `not implemented`.

There may be additional options for any particular function, which are documented in the decription of the function.

4.2.23 Central Moments

rv_base.**central_moments**(*k*)

Returns the first j central moments, $\mu_j, j = 1 \dots k$, of a random variable X.

See also Wikipedia [1122]

There are various ways to calculate the central moments. Ideally, they are available in closed from and can be calculated directly, or there is are dedicated functions to calculate them. If this is not the case, then one of the options given below can be used. There is no method which is always best.

rv_base.**set_central_moments_method**(*central_moments_method*)

Sets the method to be used for the calculation of the central_moments_method. `central_moments_method` can take the following values:

1. "direct": this is the default if a closed form of the cdf exists, otherwise returns `not implemented`.
2. "from_raw_moments": only if the raw moments are available in closed form, otherwise returns `not implemented`.
3. "from_cumulants": only if the cumulants are available in closed form, otherwise returns `not implemented`.
4. "from_pdf": calculates the raw moments from the pdf, the converts the raw moments to the central moments.

There may be additional options for any particular function, which are documented in the decription of the function.

4.2.24 Cumulants

rv_base.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \dots k$, of a random variable X.

See also Wikipedia [1122]

There are various ways to calculate the cumulants. Ideally, they are available in closed from and can be calculated directly, or there is are dedicated functions to calculate them. If this is not the case, then one of the options given below can be used. There is no method which is always best.

rv_base.**set_cumulants_method**(*cumulants_method*)

Sets the method to be used for the calculation of the cumulants. `cumulants_method` can take the following values:

1. "direct": this is the default if a closed form of the cdf exists, otherwise returns `not implemented`.
2. "from_moments": only if the moments are available in closed form, otherwise returns `not implemented`.
3. "from_cgf": only for functions for which the moment generating function is available in closed form, otherwise returns `not implemented`.

There may be additional options for any particular function, which are documented in the decription of the function.

4.3 Base class for continuous univariate distributions

class rv_continuous(*rv_base*)

The base continuous probability distribution class, which inherits from `rv_base`. There are no parameters and the support interval is $(-\infty, \infty)$.

The constructor has the following form:

```
class rv_continuous(rv_base):
    __a = -mp.inf
    __b = +mp.inf

    def __init__(self, df):
        pass
```

4.3.1 Probability density function

`rv_continuous.`**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X:

See also Wikipedia [1135].

There are various ways to calculate a pdf. Ideally, it is available in closed from and can be calculated directly, or there is are dedicated functions to calculate it. If this is not the case, then one of the options given below can be used. There is no method which is always best, and the choice depends not only on the function, but also on the values of the argument and the parameters.

`rv_base.`**set_pdf_method**(*pdf_method*)

Sets the method to be used for the calculation of the pdf. `pdf_method` can take the following values:

1. "direct": this is the default if a closed form of the cdf exists, otherwise returns `not implemented`.
2. "from_cdf": only for continuous functions for which the pdf is available in closed form, otherwise returns `not implemented`.
3. "from_charfunc": only for functions for which the characteristic is available in closed form, otherwise returns `not implemented`.
4. "from_edgeworth": only if at least the first 2 cumulants exist, otherwise returns `not implemented`.
5. "from_lug_rice": only if the cumulant generating function exists, otherwise returns `not implemented`.

There may be additional options for any particular function, which are documented in the decription of the function.

4.3.2 Probability sparsity function

`rv_continuous.`**psf**(x)

Returns $\text{psf}_X(x)$, the probability sparsity function (psf), which is the derivative of the quantile function (qtf).

See also Wikipedia [1137], Wikipedia [1129].

The probability sparsity function can be calculated using the probability density function (pdf) and the qtf:

$$\text{psf}_X(q) = \frac{\mathrm{d}}{\mathrm{d}q}\text{qtf}_X(q) = \frac{1}{\text{pdf}_X(\text{qtf}_X(q))}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

4.3.3 Differerential entropy

rv_continuous.**entropy**(x)

Returns entropy$_X(x)$, the differential entropy of a random variable X.

See also Wikipedia [1149]

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

4.3.4 Negative loglikelihood function

See also Wikipedia [1154]

rv_continuous.**nnlf**(x)

Returns nnlf$_X(x)$, the negative loglikelihood function of a random variable X:

$$\text{nnlf}_X(x) = -\sum(\log \text{pdf}_X(x, \theta), axis = 0),$$

where theta are the parameters (including loc and scale).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

4.3.5 Logarithm of the probability density function

rv_continuous.**logpdf**(x)

Returns logpdf$_X(x)$, the logarithm of the probability density function (pdf) of a random variable X.

See also Wikipedia [1135]

$$\text{logpdf}_X(x) = \log(\text{pdf}_X(x))$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

4.4 Base class for discrete univariate distributions

class rv_discrete(*rv_base*)

The base discrete probability distribution class, which inherits from `rv_base`. There are no parameters and the support interval is $(-\infty, \infty)$.

The constructor has the following form:

```
class rv_continuous(rv_base):
    __a = -mp.inf
    __b = +mp.inf

    def __init__(self, df):
        pass
```

4.4.1 Probability mass function

`rv_discrete.`**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X.

See also Wikipedia [1169].

There are various ways to calculate a pmf. Ideally, it is available in closed from and can be calculated directly, or there is are dedicated functions to calculate it. If this is not the case, then one of the options given below can be used. There is no method which is always best, and the choice depends not only on the function, but also on the values of the argument and the parameters.

`rv_base.`**set_pmf_method**(*pmf_method*)

Sets the method to be used for the calculation of the pmf. `pmf_method` can take the following values:

1. "direct": this is the default if a closed form of the cdf exists, otherwise returns `not implemented`.
2. "from_pmf_vector": only for discrete functions for which the pmf vector is available in polynomial time, otherwise returns `not implemented`.
3. "from_charfunc": only for functions for which the characteristic is available in closed form, otherwise returns `not implemented`.
4. "from_edgeworth": only if at least the first 2 cumulants exist, otherwise returns `not implemented`.
5. "from_lug_rice": only if the cumulant generating function exists, otherwise returns `not implemented`.

There may be additional options for any particular function, which are documented in the decription of the function.

4.4.2 Probability generating function

See also Wikipedia [1170].

`rv_discrete.`**pgf**(x)

Returns $\text{pgf}_X(x)$, the probability generating function (pgf) of a discrete random variable X.

See also Wikipedia [1170].

It is a power series representation (the generating function) of the probability mass function of the random variable.

If X is a discrete random variable taking values in the non-negative integers $\{0, 1, ...\}$, then the probability generating function of X is defined as

$$G(z) = \mathrm{E}(z^X) = \sum_{x=0}^{\infty} p(x) z^x,$$

where p is the probability mass function of X. Note that the subscripted notations G_X and p_X are often used to emphasize that these pertain to a particular random variable X, and to its distribution. The power series converges absolutely at least for all complex numbers z with $|z| \leq 1$; in many examples the radius of convergence is larger.

The probability-generating function is related to the moment-generating function by

$$G_X(e^t) = M_X(t); \quad G(t) = M_X(\log(t)).$$

There are various ways to calculate a pgf. Ideally, it is available in closed from and can be calculated directly, or there is are dedicated functions to calculate it. If this is not the case, then one of the options given below can be used. There is no method which is always best, and the choice depends not only on the function, but also on the values of the argument and the parameters.

`rv_base.`**`set_pgf_method`**(*pgf_method*)

Sets the method to be used for the calculation of the pgf. `pgf_method` can take the following values:

1. "direct": this is the default if a closed form of the cdf exists, otherwise returns `not implemented`.
2. "from_pmf_vector": only for discrete functions for which the pmf vector is available in polynomial time, otherwise returns `not implemented`.
3. "from_charfunc": only for functions for which the characteristic function is available in closed form, otherwise returns `not implemented`.
4. "from_mgf": only for functions for which the moment generating function is available in closed form, otherwise returns `not implemented`.
5. "from_mgf": only for functions for which the moment generating function is available in closed form, otherwise returns `not implemented`.

There may be additional options for any particular function, which are documented in the decription of the function.

4.4.3 Vector containing all values of the pmf

`rv_discrete.`**`pmfvec`**(x)

Returns a vector containing all values of the pmf. This is useful is such a vector is cheap to compute, and further operations (like convolutions) are required.

See also Wikipedia [1169].

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145614356E+00
```

4.4.4 Entropy

`rv_discrete.`**`entropy`**(x)

Returns entropy$_X(x)$, the entropy of a random variable X.

See also Wikipedia [1149].

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145614356E+00
```

4.4.5 Logarithm of the probability mass function

rv_discrete.**logpmf**(x)

Returns $\text{logpmf}_X(x)$, the logarithm of the probability mass function (pmf) of a random variable X.

See also Wikipedia [1169].

$$\text{logpmf}_X(x) = \log(\text{pmf}_X(x))$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

4.5 Closed form distributions, based on elementary functions

4.5.1 Boost: Arcsine Distribution

The following functions return the pdf, cdf, qtf or boost class of the arcsine distribution with endpoints $a \in \mathbb{R}$ and $b \in \mathbb{R}$, and the support interval (a, b).

See also Wikipedia [1145], BoostMath [50].

`math53.`**`arcsine_pdf`**(a, b, x)

Returns $\text{pdf}(x)$, the value of the probability density function (*Pdf*) of the arcsine distribution:

$$\text{pdf}(x) = \frac{1}{\pi\sqrt{(x-a)(b-x)}}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("ArcsinePdf(x, a, b): ", ArcsinePdf(x, a, b))
>>> print ("dist_arcsine(a, b).pdf(x): ", dist_arcsine(a, b).pdf(x))
6.356352346256452561561561561456 1356E+00
```

`math53.`**`arcsine_cdf`**(a, b, x)

Returns $\text{cdf}(x)$, the value of the cumulative distribution function (*Cdf*) of the arcsine distribution:

$$\text{cdf}(x) = \frac{2}{\pi}\arcsin\left(\sqrt{\frac{x-a}{b-a}}\right)$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("ArcsineCdf(x, a, b): ", ArcsineCdf(x, a, b))
>>> print ("dist_arcsine(a, b).cdf(x): ", dist_arcsine(a, b).cdf(x))
6.356352346256452561561561561456 1356E+00
```

`math53.`**`arcsine_qtf`**(a, b, q)

where `Ctx` is `Math53` or `CtxBoost`.

Returns $\text{qtf}(q)$, the value of the quantile function (*Qtf*) of the arcsine distribution:

$$\text{qtf}(q) = a + (b-a)\left(\sin\left(\tfrac{1}{2}\pi \cdot q\right)\right)^2$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("ArcsineQtf(q, a, b): ", ArcsineQtf(q, a, b))
>>> print ("dist_arcsine(a, b).qtf(q): ", dist_arcsine(a, b).qtf(q))
6.356352346256452561561561561456 1356E+00
```

class ctx.**dist_arcsine**(a, b)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The arcsine distribution is a continuous probability distribution with endpoints $a \in \mathbb{R}$ and $b \in \mathbb{R}$, $a < b$, and the support interval (a, b).

See also Wikipedia [1145], MathWorld [243], BoostMath [50], Witkovský [1462].

dist_arcsine.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an arcsine distribution:

$$\mathrm{pdf}_X(x) = \frac{1}{\pi\sqrt{(x-a)(b-x)}}.$$

An example:

```
>>> from xlcalcnet import mpm
>>> mpm.dps = 30; a = '0'; b = '1'; x = '0.25'
>>> print ("pdf: ", mpm.dist_arcsine(a, b).pdf(x))
6.356352346256452561561561561456 1356E-20
```

dist_arcsine.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an arcsine distribution:

$$\mathrm{cdf}_X(x) = \frac{2}{\pi}\arcsin\left(\sqrt{\frac{x-a}{b-a}}\right).$$

An example:

```
>>> from xlcalcnet import mpm
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mpm.arcsine(mu, sigma).pdf(x))
6.356352346256452561561561456 1356E-20
```

dist_arcsine.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following an arcsine distribution:

$$\mathrm{sf}_X(x) = \frac{2}{\pi}\arccos\left(\sqrt{\frac{x-a}{b-a}}\right).$$

An example:

```
>>> from xlcalcnet import mpm
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mpm.arcsine(mu, sigma).pdf(x))
sf: 6.356352346256452561561561456 1356E-20
```

dist_arcsine.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an arcsine distribution:

$$\mathrm{qtf}_X(q) = a + (b-a)\left(\sin\left(\tfrac{1}{2}\pi \cdot q\right)\right)^2$$

```
>>> from xlcalcnet import mpm
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mpm.arcsine(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456 1356E+00
```

dist_arcsine.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an arcsine distribution:

$$\text{isf}_X(q) = a + (b-a)\left(\cos\left(\tfrac{1}{2}\pi \cdot q\right)\right)^2$$

```
>>> from xlcalcnet import mpm
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mpm.arcsine(mu, sigma).isf(q))
6.356352346256452561561561561456 1356E+00
```

dist_arcsine.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an arcsine distribution:

$$C_X(t) = {}_1F_1\left(\frac{1}{2}; 1; it\right).$$

```
>>> from xlcalcnet import mpm
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mpm.arcsine(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_arcsine.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an arcsine distribution:

$$M_X(t) = {}_1F_1\left(\frac{1}{2}; 1; t\right).$$

```
>>> from xlcalcnet import mpm
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", mpm.arcsine(mu, sigma).m_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_arcsine.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an arcsine distribution:

$$K_X(t) = \log\left[{}_1F_1\left(\frac{1}{2}; 1; t\right)\right].$$

```
>>> from xlcalcnet import mpm
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("k_x: ", mpm.arcsine(mu, sigma).k_x(t, k))
6.356352346256452561561561561456 1356E+00
```

dist_arcsine.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an arcsine distribution:

$$\mu'_X(r) = \prod_{j=0}^{n-1} \frac{2j+1}{2j+2}.$$

```
>>> from xlcalcnet import mpm
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("moments: ", mpm.arcsine(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_arcsine.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an arcsine distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import mpm
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("cumulants: ", mpm.arcsine(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.5.2 Boost: Cauchy distribution

The following functions return the pdf, cdf, qtf or boost class of the Cauchy distribution with parameters $a \in \mathbb{R}$ (location), $b > 0$ (scale), and the support interval $(-\infty, +\infty)$.

See also Wikipedia [1147], MathWorld [817], BoostMath [52], Ehrhardt [297] (3.9.4).

math53.cauchy_pdf(a, b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the Cauchy distribution:

$$\text{pdf}(x) = \frac{1}{\pi b(1 + ((x-a)/b)^2)}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("CauchyPdf(x, a, b): ", CauchyPdf(x, a, b))
>>> print ("dist_cauchy(a, b).pdf(x): ", dist_cauchy(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

math53.cauchy_cdf(a, b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Cauchy distribution:

$$\text{cdf}(x) = \frac{1}{2} + \frac{1}{\pi} \arctan\left(\frac{x-a}{b}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("CauchyCdf(x, a, b): ", CauchyCdf(x, a, b))
>>> print ("dist_cauchy(a, b).cdf(x): ", dist_cauchy(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

math53.cauchy_qtf(a, b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Cauchy distribution:

$$\text{qtf}(q) = \begin{cases} a - b/\tan(\pi q), & q < 0.5, \\ a, & q = 0.5, \\ a - b/\tan(\pi(1-q)) & q > 0.5. \end{cases}$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("CauchyQtf(q, a, b): ", CauchyQtf(q, a, b))
>>> print ("dist_cauchy(a, b).qtf(q): ", dist_cauchy(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_cauchy**(a, b)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Cauchy distribution is a continuous probability distribution with parameters $a \in \mathbb{R}$ (location), $b > 0$ (scale), and the support interval $(-\infty, +\infty)$. See also Wikipedia [1147], MathWorld [817], BoostMath [52], R (Statistical System) [520].

`dist_cauchy.pdf(`x`)`

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Cauchy distribution:

$$\mathrm{pdf}_X(x) = \frac{1}{\pi(1 + ((x-a)/b)^2)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", cauchy(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

`dist_cauchy.cdf(`x`)`

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Cauchy distribution:

$$\mathrm{cdf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\arctan\left(\frac{x-a}{b}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", cauchy(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

`dist_cauchy.sf(`x`)`

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Cauchy distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\arctan\left(\frac{x-a}{b}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", cauchy(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

`dist_cauchy.qtf(`q`)`

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Cauchy distribution:

$$\mathrm{qtf}_X(q) = \begin{cases} a - b/\tan(\pi q), & q < 0.5, \\ a, & q = 0.5, \\ a - b/\tan(\pi(1-q)) & q > 0.5. \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", cauchy(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

dist_cauchy.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Cauchy distribution:

$$\text{isf}_X(q) = \begin{cases} a + b/\tan(\pi q), & q < 0.5, \\ a, & q = 0.5, \\ a + b/\tan(\pi(1-q)) & q > 0.5. \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", cauchy(mu, sigma).isf(q))
6.356352346256452561561561561456 1356E+00
```

dist_cauchy.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Cauchy distribution:

$$C_X(t) = \exp(a \cdot it - b|t|).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", cauchy(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_cauchy.**m_x**(t)

Returns NaN, since the moment generating function does not exist.

dist_cauchy.**k_x**(t, *k=0*)

Returns NaN, since the cumulant generating function does not exist.

dist_cauchy.**moments**(k)

Returns NaN, since moments do not exist.

dist_cauchy.**cumulants**(k)

Returns NaN, since cumulants do not exist.

4.5.3 Boost: Exponential distribution

The following functions return the pdf, cdf, qtf or boost class of the distribution with rate parameter $\lambda_1 > 0$ and the support interval $(0, +\infty)$.

See also Wikipedia [1150], MathWorld [819], BoostMath [54], Ehrhardt [297] (3.9.7).

`math53.`**`exponential_pdf`**(*lambda1*, *x*)

Returns pdf(x), the value of the probability density function (*Pdf*) of the exponential distribution:

$$\text{pdf}(x) = \lambda_1 \exp(-\lambda_1 x).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("ExponentialPdf(x, a, b): ", ExponentialPdf(x, a, b))
>>> print ("dist_exponential(a, b).pdf(x): ", dist_exponential(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`exponential_cdf`**(*lambda1*, *x*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the exponential distribution:

$$\text{cdf}(x) = 1 - \exp(-\lambda_1 x) = -\text{expm1}(-\lambda_1 x).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("ExponentialCdf(x, a, b): ", ExponentialCdf(x, a, b))
>>> print ("dist_exponential(a, b).cdf(x): ", dist_exponential(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`exponential_qtf`**(*lambda1*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the exponential distribution:

$$\text{qtf}(q) = -\text{log1p}(-q)/\lambda_1.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("ExponentialQtf(q, a, b): ", ExponentialQtf(q, a, b))
>>> print ("dist_exponential(a, b).qtf(q): ", dist_exponential(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

`class` `ctx.`**`dist_exponential`**(*lambda1*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The exponential distribution is a continuous probability distribution with rate parameter $\lambda_1 > 0$, and the support interval $[0, +\infty)$. See also Wikipedia [1150], MathWorld [819], BoostMath [54], Witkovský [1465], R (Statistical System) [522].

dist_exponential.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an exponential distribution:

$$\text{pdf}_X(x) = \lambda_1 \exp(-\lambda_1 x).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", exponential(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_exponential.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an exponential distribution:

$$\text{cdf}_X(x) = 1 - \exp(-\lambda_1 x) = -\text{expm1}(-\lambda_1 x).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", exponential(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_exponential.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an exponential distribution:

$$\text{sf}_X(x) = \exp(-\lambda_1 x).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", exponential(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_exponential.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an exponential distribution:

$$\text{qtf}_X(q) = -\text{log1p}(-q)/\lambda_1.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", exponential(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_exponential.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an exponential distribution:

$$\text{isf}_X(q) = -\log(q)/\lambda_1.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", exponential(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_exponential.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an exponential distribution:

$$C_X(t) = \frac{\lambda}{\lambda - it}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", exponential(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_exponential.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an exponential distribution:

$$M_X(t) = \frac{\lambda}{\lambda - t}, \quad \text{for } t < \lambda.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", exponential(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_exponential.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an exponential distribution:

$$K_X(t) = \log\left(\frac{\lambda}{\lambda - t}\right), \quad \text{for } t < \lambda.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", exponential(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_exponential.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an exponential distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(n) = \frac{n!}{\lambda^n}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", exponential(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_exponential.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an exponential distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", exponential(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.5.4 Boost: Gumbel (Generalized Extreme Value distribution Type-I) distribution

The following functions return the pdf, cdf, qtf or boost class of the Gumbel distribution with parameters $a \in \mathbb{R}$ (location), $b > 0$ (scale), and the support interval $(-\infty, +\infty)$.

See also Wikipedia [1153], MathWorld [822], BoostMath [55], Ehrhardt [297] (3.9.8).

math53.**gumbel_pdf**(a, b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the Gumbel distribution:

$$\text{pdf}(x) = \frac{e^{-(x-a)/b}}{b} e^{-e^{-(x-a)/b}}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("GumbelPdf(x, a, b): ", GumbelPdf(x, a, b))
>>> print ("dist_gumbel(a, b).pdf(x): ", dist_gumbel(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**gumbel_cdf**(a, b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Gumbel distribution:

$$\text{cdf}(x) = e^{-e^{-(x-a)/b}}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("GumbelCdf(x, a, b): ", GumbelCdf(x, a, b))
>>> print ("dist_gumbel(a, b).cdf(x): ", dist_gumbel(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**gumbel_qtf**(a, b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Gumbel distribution:

$$\text{qtf}(q) = a - b\log(-\log(q)).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("GumbelQtf(q, a, b): ", GumbelQtf(q, a, b))
>>> print ("dist_gumbel(a, b).qtf(q): ", dist_gumbel(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_gumbel**(a, b)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Gumbel (or Extreme Value Type I)distribution is a continuous probability distribution with parameters $a \in \mathbb{R}$ (location), $b > 0$ (scale), and the support interval $(-\infty, +\infty)$.

See also Wikipedia [1153], MathWorld [822], BoostMath [55], Witkovský [1468].

Note: In MathWorld/Mathematica, the Gumbel (Maximum) distribution is called the ExtremeValueDistribution, and the Gumbel (Minimum) distribution is called the GumbelDistribution.

dist_gumbel.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Gumbel distribution:

$$\text{pdf}_X(x) = \frac{e^{-(x-a)/b}}{b} e^{e^{-(x-a)/b}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", gumbel(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_gumbel.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Gumbel distribution:

$$\text{cdf}_X(x) = e^{e^{-(x-a)/b}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", gumbel(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_gumbel.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an Gumbel distribution:

$$\text{sf}_X(x) = 1 - e^{e^{-(x-a)/b}} = -\text{expm1}(-e^{-(x-a)/b}).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", gumbel(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_gumbel.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an Gumbel distribution:

$$\text{qtf}_X(q) = a - b\log(-\log(q)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", gumbel(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_gumbel.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an Gumbel distribution:

$$\text{isf}_X(q) = a - b\log(-\log 1\text{p}(-q)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", gumbel(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_gumbel.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Gumbel distribution:

$$C_X(t) = \Gamma(1 - ibt)e^{iat}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", gumbel(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_gumbel.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Gumbel distribution:

$$M_X(t) = \Gamma(1 - bt)e^{at}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", gumbel(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_gumbel.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Gumbel distribution:

$$K_X(t) = at + \log(\Gamma(1 - bt)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", gumbel(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_gumbel.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Gumbel distribution. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", gumbel(mu, sigma).moments(k))
6.356352346256452561561561561456l356E+00
```

dist_gumbel.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Gumbel distribution:

$$\kappa'_X(n) = (n-1)!\zeta(n).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", gumbel(mu, sigma).cumulants(k))
6.356352346256452561561561561456l356E+00
```

4.5.5 Boost: Hyperexponential Distribution

The following functions return pdf, cdf, qtf or boost class of the hyperexponential distribution with weights $w_j > 0$, rate parameters $\lambda_j > 0$, $j = 1 \ldots k$, and the support interval $(0, +\infty)$. The weights are interpreted as relative weights, to ensure that $\sum_{j=1}^{k} w_j = 1$.

See also Wikipedia [1178], MathWorld [249], BoostMath [77].

`math53.`**`hyperexponential_pdf`**(*lambdaj*, *wj*, *x*)

Returns pdf(x), the value of the probability density function (*Pdf*) of the hyperexponential distribution:

$$\text{pdf}(x) = \sum_{j=1}^{k} w_j \cdot \lambda_j e^{-\lambda_j x}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("HyperexpPdf(x, a, b): ", HyperexpPdf(x, a, b))
>>> print ("dist_hyperexponential(a, b).pdf(x): ", dist_hyperexponential(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`hyperexponential_cdf`**(*lambdaj*, *wj*, *x*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the hyperexponential distribution:

$$\text{cdf}(x) = \sum_{j=1}^{k} w_j(1 - e^{-\lambda_j x}) = -\sum_{j=1}^{k} w_j \cdot \text{expm1}(-\lambda_j x).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("HyperexpCdf(x, a, b): ", HyperexpCdf(x, a, b))
>>> print ("dist_hyperexponential(a, b).cdf(x): ", dist_hyperexponential(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`hyperexponential_qtf`**(*lambdaj*, *wj*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the hyperexponential distribution:

There is no known closed form for qtf(q). The function is computed with the starting value $x_0 = -\text{log1p}(-q) \sum_{j=1}^{k}(w_j/\lambda_j)$ using Newton iterations.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("HyperexpQtf(q, a, b): ", HyperexpQtf(q, a, b))
>>> print ("dist_hyperexponential(a, b).qtf(q): ", dist_hyperexponential(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

`class` `ctx.`**`dist_hyperexponential`**(*wj*, *lambdaj*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The hyperexponential distribution is a continuous probability mixture distribution with weights $w_j > 0$, rate parameters $\lambda_j > 0$, $j = 1 \ldots k$, and the support interval $(0, +\infty)$. The weights are interpreted as relative weights, to ensure that $\sum_{j=1}^{k} w_j = 1$.

It is called the hyperexponential distribution as it has a coefficient of variation greater than one, compared to the hypoexponential distribution which has coefficient of variation less than one and the exponential distribution which has coefficient of variation of one.

See also Wikipedia [1178], MathWorld [249], BoostMath [77].

`dist_hyperexponential.pdf(`x`)`

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a hyperexponential distribution:

$$\text{pdf}_X(x) = \sum_{j=1}^{k} w_j \cdot \lambda_j e^{-\lambda_j x}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_hyperexponential(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

`dist_hyperexponential.cdf(`x`)`

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a hyperexponential distribution:

$$\text{cdf}_X(x) = \sum_{j=1}^{k} w_j(1 - e^{-\lambda_j x}) = -\sum_{j=1}^{k} w_j \cdot \text{expm1}(-\lambda_j x).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_hyperexponential(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

`dist_hyperexponential.sf(`x`)`

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following a hyperexponential distribution:

$$\text{sf}_X(x) = 1 - \sum_{j=1}^{k} w_j(1 - e^{-\lambda_j x}) = \sum_{j=1}^{k} w_j \cdot \exp(-\lambda_j x).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_hyperexponential(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

`dist_hyperexponential.qtf(`q`)`

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a hyperexponential distribution.

There is no known closed form for $\text{qtf}_X(q)$. The function is computed with the starting value $x_0 = -\text{log1p}(-q)\sum_{j=1}^{k}(w_j/\lambda_j)$ using Newton iterations.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_hyperexponential(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_hyperexponential.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a hyperexponential distribution.

There is no known closed form for $\text{isf}_X(q)$. The function is computed with the starting value $x_0 = -\log(q)\sum_{j=1}^{k}(w_j/\lambda_j)$ using Newton iterations.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_hyperexponential(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_hyperexponential.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a hyperexponential distribution:

$$C_X(t) = \sum_{j=1}^{k} w_j \frac{\lambda_j}{\lambda_j - it}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_hyperexponential(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_hyperexponential.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a hyperexponential distribution:

$$M_X(t) = \sum_{j=1}^{k} w_j \frac{\lambda_j}{\lambda_j - t}, \quad \text{for } \min(\lambda_j) > t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_hyperexponential(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_hyperexponential.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a hyperexponential distribution:

$$K_X(t) = \log\left[\sum_{j=1}^{k} w_j \frac{\lambda_j}{\lambda_j - t}\right], \quad \text{for } \min(\lambda_j) > t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", mp_hyperexponential(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_hyperexponential.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a hyperexponential distribution:

$$\mu_X(r) = r! \sum_{j=1}^{k} \frac{w_j}{\lambda_j^r}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_hyperexponential(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_hyperexponential.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a hyperexponential distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_hyperexponential(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.5.6 !!!Boost: Kumaraswamy distribution

The following functions return pdf, cdf, qtf or boost class of the Kumaraswamy distribution with shape parameters $a > 0$, $b > 0$, and the support interval $(0, 1)$.

See also Wikipedia [1183], MathWorld [252], Ehrhardt [297] (3.9.13).

See also: https://stats.stackexchange.com/questions/171952/parameter-estimation-for-kumaraswamy-distribution

See also: https://www.johndcook.com/blog/2009/11/24/kumaraswamy-distribution/

`math53.`**`kumaraswamy_pdf`**(a, b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the Kumaraswamy distribution:

$$\text{pdf}(x) = abx^{a-1}(1-x^a)^{b-1} = abx^{a-1} \cdot (-\text{powm1}(x, a))^{b-1}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("KumaraswamyPdf(x, a, b): ", KumaraswamyPdf(x, a, b))
>>> print ("dist_kumaraswamy(a, b).pdf(x): ", dist_kumaraswamy(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`kumaraswamy_cdf`**(a, b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Kumaraswamy distribution:

$$\text{cdf}(x) = 1 - (1-x^a)^b = -\text{powm1}(-\text{powm1}(x, a), b).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("KumaraswamyCdf(x, a, b): ", KumaraswamyCdf(x, a, b))
>>> print ("dist_kumaraswamy(a, b).cdf(x): ", dist_kumaraswamy(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`kumaraswamy_qtf`**(a, b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Kumaraswamy distribution:

$$\text{qtf}(q) = (1-(1-q)^{\frac{1}{b}})^{\frac{1}{a}} = \text{pow}(-\text{pow1pm1}(-q, 1/b), 1/a).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("KumaraswamyQtf(q, a, b): ", KumaraswamyQtf(q, a, b))
>>> print ("dist_kumaraswamy(a, b).qtf(q): ", dist_kumaraswamy(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

`class` `ctx.`**`dist_kumaraswamy`**(a, b)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The Kumaraswamy distribution is a continuous probability distribution with shape parameters $a > 0$, $b > 0$, and the support interval $(0, 1)$. See also Wikipedia [1183], MathWorld [252].

dist_kumaraswamy.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Kumaraswamy distribution:

$$\text{pdf}_X(x) = abx^{a-1}(1-x^a)^{b-1} = abx^{a-1} \cdot (-\text{powm1}(x,a))^{b-1}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_kumaraswamy(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_kumaraswamy.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Kumaraswamy distribution:

$$\text{cdf}_X(x) = 1-(1-x^a)^b = -\text{powm1}(-\text{powm1}(x,a),b).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_kumaraswamy(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_kumaraswamy.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Kumaraswamy distribution:

$$\text{sf}_X(x) = (1-x^a)^b = \text{pow}(-\text{powm1}(x,a),b).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_kumaraswamy(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_kumaraswamy.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Kumaraswamy distribution:

$$\text{qtf}_X(q) = (1-(1-q)^{\frac{1}{b}})^{\frac{1}{a}} = \text{pow}(-\text{pow1pm1}(-q,1/b),1/a).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_kumaraswamy(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_kumaraswamy.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Kumaraswamy distribution:

$$\text{isf}_X(q) = (1-q^{\frac{1}{b}})^{\frac{1}{a}} = \text{pow}(-\text{powm1}(q,1/b),1/a).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_kumaraswamy(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_kumaraswamy.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Kumaraswamy distribution:

$$C_X(t) = tbd$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_kumaraswamy(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_kumaraswamy.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Kumaraswamy distribution:

$$M_X(t) = tbd$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", mp_kumaraswamy(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_kumaraswamy.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Kumaraswamy distribution:

$$K_X(t) = tbd$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", mp_kumaraswamy(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_kumaraswamy.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Kumaraswamy distribution (Wikipedia). The central moments are calculated from the raw moments.

$$\mu_X(r) = \frac{b\Gamma(1+n/a)\Gamma(b)}{\Gamma(1+b+n/a)} = bB(1+n/a, b).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_kumaraswamy(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_kumaraswamy.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Kumaraswamy distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_kumaraswamy(mu, sigma).cumulants(k))
6.35635234625645256156156145561356E+00
```

4.5.7 Boost: Laplace distribution

The following functions return the pdf, cdf, qtf or boost class of the Laplace distribution with parameters $a \in \mathbb{R}$ (location), $b > 0$ (scale), and the support interval $(-\infty, +\infty)$.

See also Wikipedia [1157], MathWorld [826], BoostMath [60], Ehrhardt [297] (3.9.15).

`math53.laplace_pdf`(a, b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the Laplace distribution:

$$\text{pdf}(x) = \exp(-|x - a|/b)/(2b).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("LaplacePdf(x, a, b): ", LaplacePdf(x, a, b))
>>> print ("dist_laplace(a, b).pdf(x): ", dist_laplace(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.laplace_cdf`(a, b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Laplace distribution:

$$\text{cdf}(x) = \begin{cases} \frac{1}{2} - \frac{1}{2}\text{expm1}\left(-\frac{x-a}{b}\right) & x \geq a \\ \frac{1}{2}\exp\left(-\frac{x-a}{b}\right) & x < a. \end{cases}$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("LaplaceCdf(x, a, b): ", LaplaceCdf(x, a, b))
>>> print ("dist_laplace(a, b).cdf(x): ", dist_laplace(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.laplace_qtf`(a, b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Laplace distribution:

$$\text{qtf}(q) = \begin{cases} a + b\ \log(2q), & q \leq 0.5, \\ a - b\ \log(2(1-q)) & q > 0.5. \end{cases}$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("LaplaceQtf(q, a, b): ", LaplaceQtf(q, a, b))
>>> print ("dist_laplace(a, b).qtf(q): ", dist_laplace(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class `ctx.dist_laplace`(a, b)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The Laplace distribution is a continuous probability distribution with parameters $a \in \mathbb{R}$ (location), $b > 0$ (scale), and the support interval $(-\infty, +\infty)$. See also Wikipedia [1157], MathWorld [826], BoostMath [60], Witkovský [1471].

dist_laplace.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Laplace distribution:

$$\text{pdf}_X(x) = \exp(-|x-a|/b)/(2b).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", laplace(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_laplace.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Laplace distribution:

$$\text{cdf}_X(x) = \begin{cases} \frac{1}{2} - \frac{1}{2}\text{expm1}\left(-\frac{x-a}{b}\right) & x \geq a \\ \frac{1}{2}\exp\left(-\frac{x-a}{b}\right) & x < a. \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", laplace(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_laplace.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an Laplace distribution:

$$\text{sf}_X(x) = \begin{cases} \frac{1}{2} + \frac{1}{2}\text{expm1}\left(-\frac{x-a}{b}\right) & x \geq a \\ 1 - \frac{1}{2}\exp\left(-\frac{x-a}{b}\right) & x < a. \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", laplace(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_laplace.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an Laplace distribution:

$$\text{qtf}_X(q) = \begin{cases} a + b\ \log(2q), & q \leq 0.5, \\ a - b\ \log(2(1-q)) & q > 0.5. \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", laplace(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_laplace.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an Laplace distribution:

$$\text{isf}_X(q) = \begin{cases} a - b\ \log(2q), & q \leq 0.5, \\ a + b\ \log(2(1-q)) & q > 0.5. \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", laplace(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_laplace.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Laplace distribution:

$$C_X(t) = \frac{e^{iat}}{1 - b^2t^2}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", laplace(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_laplace.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Laplace distribution:

$$M_X(t) = \frac{e^{at}}{1 - b^2t^2}.$$

```

```

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", laplace(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_laplace.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Laplace distribution:

$$K_X(t) = at - \log(1 - b^2t^2.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("k_x: ", laplace(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_laplace.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Laplace distribution. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", laplace(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_laplace.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Laplace distribution. The cumulants are calculated from the moments.

$$\kappa_X(r) = \begin{cases} a & r = 1 \\ 0 & r = 2k + 1; k = 0, 1, \ldots \\ b^r 2(r-1)! & r = 2k; k = 0, 1, \ldots \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", laplace(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.5.8 Boost: Logistic distribution

The following functions return the pdf, cdf, qtf or boost class of the logistic distribution with parameters $a \in \mathbb{R}$ (location), $b > 0$ (scale), and the support interval $(-\infty, +\infty)$.

See also Wikipedia [1159], MathWorld [829], BoostMath [62], Ehrhardt [297] (3.9.18).

`math53.logistic_pdf`(a, b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the logistic distribution:

$$\text{pdf}(x) = \frac{1}{b} \frac{\exp\left(-\frac{x-a}{b}\right)}{\left(1+\exp\left(-\frac{x-a}{b}\right)\right)^2}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("LogisticPdf(x, a, b): ", LogisticPdf(x, a, b))
>>> print ("dist_logistic(a, b).pdf(x): ", dist_logistic(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.logistic_cdf`(a, b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the logistic distribution:

$$\text{cdf}(x) = \frac{1}{1+\exp\left(-\frac{x-a}{b}\right)}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("LogisticCdf(x, a, b): ", LogisticCdf(x, a, b))
>>> print ("dist_logistic(a, b).cdf(x): ", dist_logistic(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.logistic_qtf`(a, b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the logistic distribution:

$$\text{qtf}(q) = a + b \log\left(\frac{q}{1-q}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("LogisticQtf(q, a, b): ", LogisticQtf(q, a, b))
>>> print ("dist_logistic(a, b).qtf(q): ", dist_logistic(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class `ctx.dist_logistic`(a, b)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The logistic distribution is a continuous probability distribution with parameters $a \in \mathbb{R}$ (location), $b > 0$ (scale), and the support interval $(-\infty, +\infty)$. See also Wikipedia [1159], MathWorld [829], BoostMath [62].

dist_logistic.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an logistic distribution:

$$\text{pdf}_X(x) = \frac{1}{b} \frac{\exp\left(-\frac{x-a}{b}\right)}{\left(1+\exp\left(-\frac{x-a}{b}\right)\right)^2}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", logistic(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

dist_logistic.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an logistic distribution:

$$\text{cdf}_X(x) = \frac{1}{1+\exp\left(-\frac{x-a}{b}\right)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", logistic(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

dist_logistic.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an logistic distribution:

$$\text{sf}_X(x) = \frac{1}{1+\exp\left(\frac{x-a}{b}\right)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", logistic(mu, sigma).pdf(x))
sf: 6.356352346256452561561561456135 6E-20
```

dist_logistic.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an logistic distribution:

$$\text{qtf}_X(q) = a + b\ \log\left(\frac{q}{1-q}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", logistic(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561456135 6E+00
```

dist_logistic.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an logistic distribution:

$$\text{isf}_X(q) = a - b \log\left(\frac{q}{1-q}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", logistic(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_logistic.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an logistic distribution:

$$C_X(t) = e^{ita}\frac{\pi bt}{\sinh(\pi bt)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", logistic(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_logistic.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an logistic distribution:

$$M_X(t) = e^{ta}\frac{\pi bt}{\sinh(\pi bt)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", logistic(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_logistic.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an logistic distribution:

$$K_X(t) = ta + \log(\pi bt) - \log(\sinh(\pi bt)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", logistic(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_logistic.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an logistic distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(n) = b^n \pi^n (2^n - 2) \cdot |B_n|.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", logistic(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_logistic.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an logistic distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", logistic(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

4.5.9 Boost: Pareto distribution

The following functions return pdf, cdf, qtf or boost class of the Pareto distribution with minimum (real) value $k > 0$, shape $a > 0$, and the support interval $(k, +\infty)$.

See also Wikipedia [1167], MathWorld [834], BoostMath [68], Ehrhardt [297] (3.9.25).

`math53.pareto_pdf`(k, a, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the Pareto distribution:

$$\text{pdf}(x) = \frac{a}{x}\left(\frac{k}{x}\right)^a.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("ParetoPdf(x, a, b): ", ParetoPdf(x, a, b))
>>> print ("dist_pareto(a, b).pdf(x): ", dist_pareto(a, b).pdf(x))
6.356352346256452561561561561456135 6E+00
```

`math53.pareto_cdf`(k, a, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Pareto distribution:

$$\text{cdf}(x) = 1 - \left(\frac{k}{x}\right)^a = -\text{powm1}(k/x, a).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("ParetoCdf(x, a, b): ", ParetoCdf(x, a, b))
>>> print ("dist_pareto(a, b).cdf(x): ", dist_pareto(a, b).cdf(x))
6.356352346256452561561561561456135 6E+00
```

`math53.pareto_qtf`(k, a, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Pareto distribution:

$$\text{qtf}(q) = \frac{k}{(1-q)^{1/a}}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("ParetoQtf(q, a, b): ", ParetoQtf(q, a, b))
>>> print ("dist_pareto(a, b).qtf(q): ", dist_pareto(a, b).qtf(q))
6.356352346256452561561561561456135 6E+00
```

class `ctx.dist_pareto`(k, a)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The Pareto distribution is a continuous probability distribution with minimum (real) value $k > 0$, shape $a > 0$, and the support interval $(k, +\infty)$. See also Wikipedia [1167], MathWorld [834], BoostMath [68], Witkovský [1476].

dist_pareto.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Pareto distribution:

$$\text{pdf}_X(x) = \frac{a}{x}\left(\frac{k}{x}\right)^a.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", pareto(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_pareto.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Pareto distribution:

$$\text{cdf}_X(x) = 1 - \left(\frac{k}{x}\right)^a = -\text{powm1}(k/x, a).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", pareto(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_pareto.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an Pareto distribution:

$$\text{sf}_X(x) = \left(\frac{k}{x}\right)^a = \text{pow}(k/x, a).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", pareto(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_pareto.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an Pareto distribution:

$$\text{qtf}_X(q) = \frac{k}{(1-q)^{1/a}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", pareto(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

dist_pareto.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an Pareto distribution:

$$\text{isf}_X(q) = \frac{k}{q^{1/a}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", pareto(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_pareto.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Pareto distribution:

$$C_X(t) = a(-ikt)^a \Gamma(-a, -ikt).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", pareto(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_pareto.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Pareto distribution:

$$M_X(t) = a(-kt)^a \Gamma(-a, -kt), \quad \text{for } t < 0.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", pareto(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_pareto.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Pareto distribution:

$$K_X(t) = K_X(t) = \log(M_X(t)), \quad \text{for } t < 0.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", pareto(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_pareto.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Pareto distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = \frac{ak}{a-n} \quad \text{for } a > n.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", pareto(mu, sigma).moments(k))
6.356352346256452561561561456l356E+00
```

dist_pareto.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Pareto distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", pareto(mu, sigma).cumulants(k))
6.356352346256452561561561456l356E+00
```

4.5.10 Boost: Rayleigh distribution

The following functions return the pdf, cdf, qtf or boost class of the Rayleigh distribution with scale $b > 0$ and the support interval $(0, +\infty)$.

See also Wikipedia [1190], MathWorld [836], BoostMath [85], Ehrhardt [297] (3.9.27).

`math53.`**`rayleigh_pdf`**(b, x)

Returns $\text{pdf}(x)$, the value of the probability density function (*Pdf*) of the Rayleigh distribution:

$$\text{pdf}(x) = \frac{x}{b^2} \exp\left(-\frac{x^2}{2b^2}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("RayleighPdf(x, a, b): ", RayleighPdf(x, a, b))
>>> print ("dist_rayleigh(a, b).pdf(x): ", dist_rayleigh(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`rayleigh_cdf`**(b, x)

Returns $\text{cdf}(x)$, the value of the cumulative distribution function (*Cdf*) of the Rayleigh distribution:

$$\text{cdf}(x) = 1 - \exp\left(-\frac{x^2}{2b^2}\right) = -\text{expm1}\left(-\frac{x^2}{2b^2}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("RayleighCdf(x, a, b): ", RayleighCdf(x, a, b))
>>> print ("dist_rayleigh(a, b).cdf(x): ", dist_rayleigh(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`rayleigh_qtf`**(b, q)

Returns $\text{qtf}(q)$, the value of the quantile function (*Qtf*) of the Rayleigh distribution:

$$\text{qtf}(q) = b\sqrt{-2 \cdot \text{log1p}(-q)}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("RayleighQtf(q, a, b): ", RayleighQtf(q, a, b))
>>> print ("dist_rayleigh(a, b).qtf(q): ", dist_rayleigh(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

`class` `ctx.`**`dist_rayleigh`**(b)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The Rayleigh distribution is a continuous probability distribution with scale $b > 0$ and the support interval $(0, +\infty)$. See also Wikipedia [1190], MathWorld [836], BoostMath [85], Witkovský [1477].

dist_rayleigh.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Rayleigh distribution:

$$\text{pdf}_X(x) = \frac{x}{b^2} \exp\left(-\frac{x^2}{2b^2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", rayleigh(mu, sigma).pdf(x))
6.356352346256452561561561561456 1356E-20
```

dist_rayleigh.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Rayleigh distribution:

$$\text{cdf}_X(x) = 1 - \exp\left(-\frac{x^2}{2b^2}\right) = -\text{expm1}\left(-\frac{x^2}{2b^2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", rayleigh(mu, sigma).pdf(x))
6.356352346256452561561561561456 1356E-20
```

dist_rayleigh.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an Rayleigh distribution:

$$\text{sf}_X(x) = \exp\left(-\frac{x^2}{2b^2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", rayleigh(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456 1356E-20
```

dist_rayleigh.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an Rayleigh distribution:

$$\text{qtf}_X(q) = b\sqrt{-2 \cdot \text{log1p}(-q)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", rayleigh(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456 1356E+00
```

dist_rayleigh.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an Rayleigh distribution:

$$\text{isf}_X(q) = b\sqrt{-2 \cdot \log(q)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", rayleigh(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_rayleigh.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Rayleigh distribution:

$$C_X(t) = 1 + bte^{b^2t^2/2}\sqrt{\frac{\pi}{2}}\left(\text{erfi}\left(\frac{bt}{\sqrt{2}}\right) - i\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", rayleigh(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_rayleigh.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Rayleigh distribution:

$$M_X(t) = 1 + bte^{b^2t^2/2}\sqrt{\frac{\pi}{2}}\left(\text{erf}\left(\frac{bt}{\sqrt{2}}\right) + 1\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", rayleigh(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_rayleigh.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Rayleigh distribution:

$$K_X(t) = K_X(t) = \log(M_X(t)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", rayleigh(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_rayleigh.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Rayleigh distribution.

$$\mu'_X(j) = b^j 2^{j/2}\Gamma(1 + j/2).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", rayleigh(mu, sigma).moments(k))
6.3563523462564525615615614561356E+00
```

dist_rayleigh.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Rayleigh distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", rayleigh(mu, sigma).cumulants(k))
6.3563523462564525615615614561356E+00
```

4.5.11 Boost: Triangular Distribution

The following functions return the pdf, cdf, qtf or boost class of the triangular distribution, with finite $a < b$, mode $c, a \le c \le b$, and the support interval $[a, b]$.

See also Wikipedia [1192], MathWorld [837], BoostMath [88], Ehrhardt [297] (3.9.30).

math53.**triangular_pdf**(*lower*, *mode*, *upper*, *x*)

Returns pdf(x), the value of the probability density function (*Pdf*) of the triangular distribution:

$$\text{pdf}(x) = \begin{cases} 0 & x < a \\ \frac{2(x-a)}{(b-a)(c-a)} & a \le x < c \\ \frac{2}{b-a} & x = c \\ \frac{2(b-x)}{(b-a)(b-c)} & c < x \le b \\ 0 & x > b \end{cases}$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("TriangularPdf(x, a, b): ", TriangularPdf(x, a, b))
>>> print ("dist_triangular(a, b).pdf(x): ", dist_triangular(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**triangular_cdf**(*lower*, *mode*, *upper*, *x*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the triangular distribution:

$$\text{cdf}(x) = \begin{cases} 0 & x < a \\ \frac{(x-a)^2}{(b-a)(c-a)} & a \le x < c \\ \frac{c-a}{b-a} & x = c \\ 1 - \frac{(b-x)^2}{(b-a)(b-c)} & c < x \le b \\ 1 & x > b \end{cases}$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("TriangularCdf(x, a, b): ", TriangularCdf(x, a, b))
>>> print ("dist_triangular(a, b).cdf(x): ", dist_triangular(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**triangular_qtf**(*lower*, *mode*, *upper*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the triangular distribution:

$$\text{qtf}(q) = \begin{cases} a + \sqrt{(b-a)(c-a)y} & y < (c-a)/(b-a) \\ c & y = (c-a)/(b-a) \\ b - \sqrt{(b-a)(b-c)(1-y)} & y > (c-a)/(b-a) \end{cases},$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("TriangularQtf(q, a, b): ", TriangularQtf(q, a, b))
>>> print ("dist_triangular(a, b).qtf(q): ", dist_triangular(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_triangular**(*lower*, *mode*, *upper*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The triangular distribution is a continuous probability distribution with finite $a < b$, mode c, $a \leq c \leq b$, and the support interval $[a, b]$. See also Wikipedia [1192], MathWorld [837], BoostMath [88], Witkovský [1480].

dist_triangular.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a triangular distribution:

$$\mathrm{pdf}_X(x) = \begin{cases} 0 & x < a \\ \frac{2(x-a)}{(b-a)(c-a)} & a \leq x < c \\ \frac{2}{b-a} & x = c \\ \frac{2(b-x)}{(b-a)(b-c)} & c < x \leq b \\ 0 & x > b \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", triangular(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_triangular.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a triangular distribution:

$$\mathrm{cdf}_X(x) = \begin{cases} 0 & x < a \\ \frac{(x-a)^2}{(b-a)(c-a)} & a \leq x < c \\ \frac{c-a}{b-a} & x = c \\ 1 - \frac{(b-x)^2}{(b-a)(b-c)} & c < x \leq b \\ 1 & x > b \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", triangular(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_triangular.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a triangular distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^\infty \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", triangular(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_triangular.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a triangular distribution:

$$\text{qtf}_X(q) = \begin{cases} a + \sqrt{(b-a)(c-a)y} & y < (c-a)/(b-a) \\ c & y = (c-a)/(b-a) \\ b - \sqrt{(b-a)(b-c)(1-y)} & y > (c-a)/(b-a) \end{cases},$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", triangular(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_triangular.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a triangular distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", triangular(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_triangular.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a triangular distribution:

$$C_X(t) = 2\frac{(b-c)e^{iat} - (b-a)e^{ict} + (c-a)e^{ibt}}{(b-a)(c-a)(b-c)t^2}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", triangular(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_triangular.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a triangular distribution:

$$M_X(t) = 2\frac{(b-c)e^{at} - (b-a)e^{ct} + (c-a)e^{bt}}{(b-a)(c-a)(b-c)t^2}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", triangular(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_triangular.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating functionof a random variable X, following a triangular distribution:

$$K_X(t) = \log\left[2\frac{(b-c)e^{at} - (b-a)e^{ct} + (c-a)e^{bt}}{(b-a)(c-a)(b-c)t^2}\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", triangular(mu, sigma).k_x(t, k))
6.356352346256452561561561561456135 6E+00
```

dist_triangular.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a triangular distribution:

$$\mu'_{n+1} = \sum_{i=0}^{k} \binom{k}{i} (b-a)^i a^{k-1} \frac{2(1-\theta^{i+1})}{(i+1)(i+2)(1-\theta)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", triangular(mu, sigma).moments(k))
6.356352346256452561561561561456135 6E+00
```

dist_triangular.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a triangular distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", triangular(mu, sigma).cumulants(k))
6.356352346256452561561561561456135 6E+00
```

References:

Kotz Triangular (moments.)

4.5.12 Boost: Uniform distribution

Returns the pdf, cdf, qtf or boost class of a random variable X, following a uniform distribution on the support interval $[a, b]$ with finite $a < b$.

See also Wikipedia [1193], MathWorld [838], BoostMath [89], Team [536], Ehrhardt [297] (3.9.31).

`math53.`**`uniform_pdf`**(a, b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the uniform distribution:

$$\text{pdf}(x) = \frac{1}{b-a}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("UniformPdf(x, a, b): ", UniformPdf(x, a, b))
>>> print ("dist_uniform(a, b).pdf(x): ", dist_uniform(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`uniform_cdf`**(a, b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the uniform distribution:

$$\text{cdf}(x) = \frac{x-a}{b-a}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("UniformCdf(x, a, b): ", UniformCdf(x, a, b))
>>> print ("dist_uniform(a, b).cdf(x): ", dist_uniform(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`uniform_qtf`**(a, b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the uniform distribution:

$$\text{qtf}(q) = a + q(b-a).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("UniformQtf(q, a, b): ", UniformQtf(q, a, b))
>>> print ("dist_uniform(a, b).qtf(q): ", dist_uniform(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

`class` `ctx.`**`dist_uniform`**$(lower, upper)$

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The uniform distribution is a continuous probability distribution on the support interval $[a, b]$ with finite $a < b$. See also Wikipedia [1193], MathWorld [838], BoostMath [89], Witkovský [1478], R (Statistical System) [536].

dist_uniform.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a uniform distribution:

$$\text{pdf}_X(x) = \frac{1}{b-a}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", uniform(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_uniform.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a uniform distribution:

$$\text{cdf}_X(x) = \frac{1}{b-a}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", uniform(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_uniform.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a uniform distribution:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x) = \int_x^{\infty} \text{pdf}_X(x)\text{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", uniform(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_uniform.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a uniform distribution:

$$\text{qtf}_X(q) = \frac{x-a}{b-a}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", uniform(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_uniform.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a uniform distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", uniform(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_uniform.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following a uniform distribution:

$$C_X(t) = \frac{e^{itb} - e^{ita}}{it(b-a)}, \quad \text{for } t \neq 0, 0 \text{ otherwise.}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", uniform(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_uniform.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following a uniform distribution:

$$M_X(t) = \frac{e^{tb} - e^{ta}}{t(b-a)}, \quad \text{for } t \neq 0, 1 \text{ otherwise.}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", uniform(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_uniform.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a uniform distribution:

$$K_X(t) = K_X(t) = \log(M_X(t)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", uniform(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_uniform.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a uniform distribution.

$$\mu'_X(n) = \frac{1}{n+1} \sum_{k=0}^{n} a^k b^{n-k}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", uniform(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_uniform.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a uniform distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", uniform(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.5.13 Boost: Weibull (Minimum-Type-III) distribution

The following functions return the pdf, cdf, qtf or boost class of the Weibull distribution with shape $a > 0$, scale $b > 0$, and the support interval $(0, +\infty)$.

See also Wikipedia [1194], MathWorld [839], BoostMath [90], Ehrhardt [297] (3.9.33).

`math53.weibull_pdf`(a, b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the Weibull distribution:

$$\text{pdf}(x) = \frac{a}{x}\left(-\frac{x}{b}\right)^a \exp(-(x/b)^a).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("WeibullPdf(x, a, b): ", WeibullPdf(x, a, b))
>>> print ("dist_weibull(a, b).pdf(x): ", dist_weibull(a, b).pdf(x))
6.356352346256452561561561561456 1356E+00
```

`math53.weibull_cdf`(a, b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Weibull distribution:

$$\text{cdf}(x) = 1 - \exp\left(-(x/b)^a\right) = -\text{expm1}\left(-(x/b)^a\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("WeibullCdf(x, a, b): ", WeibullCdf(x, a, b))
>>> print ("dist_weibull(a, b).cdf(x): ", dist_weibull(a, b).cdf(x))
6.356352346256452561561561561456 1356E+00
```

`math53.weibull_qtf`(a, b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Weibull distribution:

$$\text{qtf}(q) = b \cdot (-\text{log1p}(-q))^{1/a}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("WeibullQtf(q, a, b): ", WeibullQtf(q, a, b))
>>> print ("dist_weibull(a, b).qtf(q): ", dist_weibull(a, b).qtf(q))
6.356352346256452561561561561456 1356E+00
```

class `ctx.dist_weibull`(a, b)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The Weibull distribution is a continuous probability distribution with shape $a > 0$, scale $b > 0$, and the support interval $(0, +\infty)$. See also Wikipedia [1194], MathWorld [839], BoostMath [90], Witkovský [1481], R (Statistical System) [537].

dist_weibull.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Weibull distribution:

$$\text{pdf}_X(x) = \frac{a}{x}\left(-\frac{x}{b}\right)^a \exp(-(x/b)^a).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", weibull(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_weibull.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Weibull distribution:

$$\text{cdf}_X(x) = 1 - \exp\left(-(x/b)^a\right) = -\text{expm1}\left(-(x/b)^a\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", weibull(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_weibull.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a Weibull distribution:

$$\text{sf}_X(x) = \exp\left(-(x/b)^a\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", weibull(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_weibull.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Weibull distribution:

$$\text{qtf}_X(q) = b \cdot \left(-\text{log1p}(-q)\right)^{1/a}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", weibull(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_weibull.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Weibull distribution:

$$\text{isf}_X(q) = b \cdot \left(-\log(q)\right)^{1/a}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", weibull(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_weibull.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Weibull distribution:

$$C_X(t) = \sum_{n=0}^{\infty} \frac{(it)^n \lambda^n}{n!} \Gamma(1 + n/k).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", weibull(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_weibull.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Weibull distribution:

$$M_X(t) = \sum_{n=0}^{\infty} \frac{t^n \lambda^n}{n!} \Gamma(1 + n/k), k \geq 1.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", weibull(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_weibull.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Weibull distribution:

$$K_X(t) = K_X(t) = \log(M_X(t)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", weibull(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_weibull.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Weibull distribution.

$$\mu'_X(r) = \sum_{j=0}^{r} \binom{r}{j} \Gamma\left(\frac{r-j}{k} + 1\right) \lambda^{r-j}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", weibull(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_weibull.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Weibull distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", weibull(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.5.14 Dagum (Burr Type III) distribution

class ctx.**dist_dagum**(*n1*, *n2*, *lambda*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

These functions return PDF, CDF, and ICDF of the Dagum distribution with location $a > 0$, scale $b > 0$, and the support interval $(0, +\infty)$.

See also: Wikipedia [1198], Kleiber and Kotz [393], Dagum [194].

dist_dagum.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Dagum distribution:

$$\mathrm{pdf}_X(x) = f(x; a, b, p) = \frac{ap}{x} \frac{(x/b)^{ap}}{((x/b)^a + 1)^{p+1}}.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_dagum(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_dagum.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Dagum distribution:

$$\mathrm{cdf}_X(x) = \left[1 + \left(\frac{x}{b}\right)^{-a}\right]^{-p} \quad \text{for } x > 0, \text{ where } a, b, p > 0$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_dagum(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_dagum.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Dagum distribution:

$$\mathrm{sf}_X(x) = 1 - \left[1 + \left(\frac{x}{b}\right)^{-a}\right]^{-p}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_dagum(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_dagum.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Dagum distribution:

$$\mathrm{qtf}_X(q) = b(q^{-1/p} - 1)^{-1/a}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_dagum(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456 1356E+00
```

dist_dagum.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Dagum distribution:

$$\text{isf}_X(q) = b((1-q)^{-1/p} - 1)^{-1/a}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_dagum(mu, sigma).isf(q))
6.356352346256452561561561456 1356E+00
```

dist_dagum.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Dagum distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_dagum(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

dist_dagum.**m_x**(*t*)

Returns None, since the moment generating function does not exist.

dist_dagum.**k_x**(*t*, *k=0*)

Returns None, since the cumulant generating function does not exist.

dist_dagum.**moments**(*k*)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Dagum distribution (see Kleiber_2007_Dagum_moments). The kth moment exists for $-ap < k < a$ and equals

$$\mu_k = \frac{b^k \Gamma(p + k/a)\Gamma(1 - k/a)}{\Gamma(p)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_dagum(mu, sigma).moments(k))
6.356352346256452561561561456 1356E+00
```

dist_dagum.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Dagum distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_dagum(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

4.5.15 Fisk (log-logistic) distribution

class ctx.**dist_fisk**(*n1*, *n2*, *lambda*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Fisk distribution is a continuous probability distribution with scale parameter $a > 0$, shape parameter $b > 0$, and the support interval $[0, +\infty)$.

See also: Wikipedia [1214], Kleiber and Kotz [393].

dist_fisk.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Fisk distribution:

$$\mathrm{pdf}_X(x) = f(x;\alpha,\beta) = \frac{(\beta/\alpha)(x/\alpha)^{\beta-1}}{(1+(x/\alpha)^{\beta})^2}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_fisk(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_fisk.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Fisk distribution:

$$\mathrm{cdf}_X(x) = \frac{1}{1+(x/\alpha)^{-\beta}} = \frac{x^{\beta}}{\alpha^{\beta}+x^{\beta}}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_fisk(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_fisk.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Fisk distribution:

$$\mathrm{sf}_X(x) = 1 - \frac{1}{1+(x/\alpha)^{-\beta}} = 1 - \frac{x^{\beta}}{\alpha^{\beta}+x^{\beta}}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_fisk(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_fisk.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Fisk distribution:

$$\mathrm{qtf}_X(q) = \alpha\left(\frac{p}{1-p}\right)^{1/\beta}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_fisk(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456 1356E+00
```

dist_fisk.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Fisk distribution:

$$\text{isf}_X(q) = \alpha \left(\frac{q}{1-q} \right)^{1/\beta}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_fisk(mu, sigma).isf(q))
6.356352346256452561561561456 1356E+00
```

dist_fisk.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Fisk distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \text{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_fisk(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

dist_fisk.**m_x**(t)

Returns None, since the moment generating function does not exist.

dist_fisk.**k_x**(t, *k=0*)

Returns None, since the cumulant generating function does not exist.

dist_fisk.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Fisk distribution (see Kleiber_2007_Dagum_moments). The kth raw moment exists only when $k < \beta$, when it is given by

$$\mu_k = \alpha^k \frac{k\pi/\beta}{\sin(k\pi/\beta)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_fisk(mu, sigma).moments(k))
6.356352346256452561561561456 1356E+00
```

dist_fisk.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Fisk distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_fisk(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

4.5.16 Fréchet (Maximum/Minimum-Type-II or Inverse Weibull) distribution

class ctx.**dist_frechet**(a, b=*1*)

The Fréchet distribution is a continuous probability distribution with shape parameters a and b, and the support interval $(0, +\infty)$. If $b = 1$ this is the classical Fréchet distribution. For $b \neq 1$ this is also called reciprocal Weibull distribution or the generalized inverse Weibull distribution ([DeGusmao2011`).

See also: Wikipedia [1204], De Gusmao *et al.* [205], Kleiber and Kotz [393],

TODO: remove m and s from these formulas, use formulas from De Gusmao *et al.* [205].

Note: α in Wikipedia corresponds to $-\beta$ in De Gusmao *et al.* [205].

dist_frechet.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Fréchet distribution:

$$\text{pdf}_X(x) = \frac{\alpha}{s}\left(\frac{x-m}{s}\right)^{-1-\alpha} e^{-(\frac{x-m}{s})^{-\alpha}}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_frechet(mu, sigma).pdf(x))
6.356352346256452561561561456l356E-20
```

dist_frechet.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Fréchet distribution:

$$\text{cdf}_X(x) = e^{-(\frac{x-m}{s})^{-\alpha}}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_frechet(mu, sigma).pdf(x))
6.356352346256452561561561456l356E-20
```

dist_frechet.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Fréchet distribution:

$$\text{sf}_X(x) = 1 - e^{-(\frac{x-m}{s})^{-\alpha}}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_frechet(mu, sigma).pdf(x))
sf: 6.356352346256452561561561456l356E-20
```

dist_frechet.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Fréchet distribution:

$$\text{qtf}_X(q) = m + s\left(-\log(q)\right)^{-1/a}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_frechet(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_frechet.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Fréchet distribution:

$$\text{isf}_X(q) = m + s\left(-\log(1-q)\right)^{-1/a}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_frechet(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_frechet.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Fréchet distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_frechet(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_frechet.**m_x**(t)

Returns None, since the moment generating function does not exist.

dist_frechet.**k_x**(t, *k=0*)

Returns None, since the cumulant generating function does not exist.

dist_frechet.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Fréchet distribution. Moments exist only for $a > k$.

$$\mu_k = a^k \Gamma(1 - kb^{-1})$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_frechet(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_frechet.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Fréchet distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_frechet(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

4.5.17 Generalized Extreme Value (Maximum) or GEV distribution

class ctx.**dist_gev**(*n1*, *n2*, *lambda*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

These functions return PDF, CDF, and ICDF of the GEV distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$:

See also: Wikipedia [1205], Kleiber and Kotz [393], Kotz and Nadarajah [398], Coles [184].

Extreme value theory studies the statistical behaviour of $M_n = \max\{X_1, \ldots, X_n\}$, or $M_n = \min\{X_1, \ldots, X_n\}$, where $\{X_1, \ldots, X_n\}$ is a sequence of independent random variables having a common distribution function F.

If there exist sequences of constants $\{a_n\}$ and $\{b_n\}$ such that $\lim_{n\to\infty} P\{(M_n - b_n)/a_n \le z\} = G(z)$, where G is a non-degenerate distribution function (this assumption is called the stability postulate), then G belongs to the Gumbel, Fréchet or Weibull family of distributions.

The following table summarizes the forms of limiting distributions for maxima and minima for seven widely used continuous distributions:

Initial Distribution	Limiting distribution for maxima	Limiting distribution for minima
Exponential	Type 1 (Gumbel)	Type 3 (Weibull)
Gamma	Type 1 (Gumbel)	Type 3 (Weibull)
Normal	Type 1 (Gumbel)	Type 1 (Gumbel)
Log-Normal	Type 1 (Gumbel)	Type 1 (Gumbel)
Uniform	Type 3 (Weibull)	Type 3 (Weibull)
Pareto	Type 2 (Fréchet)	Type 3 (Weibull)
Cauchy	Type 2 (Fréchet)	Type 2 (Fréchet)

dist_gev.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an GEV distribution:

$$\mathrm{pdf}_X(x) = \mathrm{cdf}_X(x) \times \begin{cases} \left(1 - c\dfrac{x-a}{b}\right)^{1/c-1} & \text{for } c \neq 0 \\ \exp\left(-\dfrac{x-a}{b}\right) & \text{for } c = 0 \end{cases}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_gev(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

`dist_gev.cdf(`x`)`

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an GEV distribution:

$$\mathrm{cdf}_X(x) = \begin{cases} \exp\left(1 - c\dfrac{x-a}{b}\right)^{1/c} & \text{for } c \neq 0 \\ \exp\left(-\exp\left(-\dfrac{x-a}{b}\right)\right) & \text{for } c = 0 \end{cases}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_gev(mu, sigma).pdf(x))
6.356352346256452561561561561456 1356E-20
```

`dist_gev.sf(`x`)`

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an GEV distribution:

$$\mathrm{sf}_X(x) = \begin{cases} 1 - \exp\left(1 - c\dfrac{x-a}{b}\right)^{1/c} & \text{for } c \neq 0 \\ 1 - \exp\left(-\exp\left(-\dfrac{x-a}{b}\right)\right) & \text{for } c = 0 \end{cases}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_gev(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456 1356E-20
```

`dist_gev.qtf(`q`)`

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an GEV distribution:

$$\mathrm{qtf}_X(x) = \begin{cases} a + \dfrac{b}{c}\left(1 - \log(q)\right)^c) & \text{for } c \neq 0 \\ a - b\log(-\log(q)) & \text{for } c = 0 \end{cases}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_gev(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456 1356E+00
```

`dist_gev.isf(`q`)`

Returns $\mathrm{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an GEV distribution:

$$\mathrm{isf}_X(x) = \begin{cases} a + \dfrac{b}{c}\left(1 - \log(1-q)\right)^c) & \text{for } c \neq 0 \\ a - b\log(-\log(1-q)) & \text{for } c = 0 \end{cases}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_gev(mu, sigma).isf(q))
6.356352346256452561561561561456 1356E+00
```

dist_gev.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an GEV distribution:

$$C_X(t) = \int_0^\infty e^{itx} \mathrm{pdf}_X(x) \mathrm{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_gev(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_gev.**m_x**(t)

Returns None, since the moment generating function does not exist.

dist_gev.**k_x**(t, *k=0*)

Returns None, since the cumulant generating function does not exist.

dist_gev.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an GEV distribution (see Kleiber_2007_Dagum_moments). The kth moment exists for $-ap < k < a$ and equals

$$\mu_k = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_gev(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_gev.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an GEV distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_gev(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

4.5.18 Generalized Pareto distribution

class ctx.**dist_genpareto**(*n1*, *n2*, *lambda*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

These functions return PDF, CDF, and ICDF of the Generalized Pareto distribution with parameters mu in mathbb R, sigma >0`, and $c \in \mathbb{R}$, where the support of $x \geq \mu$ when $c \geq 0$, and $\mu \leq x \leq \mu - \sigma/c$ when $c < 0$.

See also: Wikipedia [1206], Pires *et al.* [472], Kleiber and Kotz [393],

dist_genpareto.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Generalized Pareto distribution:

$$\mathrm{pdf}_X(x) = f_c(z) = \begin{cases} (1+cz)^{-\frac{c+1}{c}} & \text{for } c \neq 0, \\ e^{-z} & \text{for } c = 0. \end{cases}, \quad \text{where } z = \frac{x-\mu}{\sigma}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_genpareto(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_genpareto.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Generalized Pareto distribution:

$$\mathrm{cdf}_X(x) = \begin{cases} 1-(1+cz)^{-1/c} & \text{for } c \neq 0, \\ 1-\exp(-z) & \text{for } c = 0, \end{cases}, \quad \text{where } z = \frac{x-\mu}{\sigma}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_genpareto(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_genpareto.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following a Generalized Pareto distribution:

$$\mathrm{sf}_X(x) = \begin{cases} (1+cz)^{-1/c} & \text{for } c \neq 0, \\ \exp(-z) & \text{for } c = 0, \end{cases}, \quad \text{where } z = \frac{x-\mu}{\sigma}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_genpareto(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_genpareto.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a Generalized Pareto distribution:

$$\mathrm{qtf}_X(q) = \mu + \sigma \times \begin{cases} \dfrac{(1-q)^{-c}-1}{c} & \text{for } c \neq 0, \\ -\log(1-q) & \text{for } c = 0, \end{cases}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_genpareto(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456 1356E+00
```

`dist_genpareto.`**`isf`**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a Generalized Pareto distribution:

$$\text{isf}_X(q) = \mu + \sigma \times \begin{cases} \dfrac{(q)^{-c} - 1}{c} & \text{for } c \neq 0, \\ -\log(q) & \text{for } c = 0, \end{cases}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_genpareto(mu, sigma).isf(q))
6.356352346256452561561561456 1356E+00
```

`dist_genpareto.`**`c_x`**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Generalized Pareto distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_genpareto(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

`dist_genpareto.`**`m_x`**(t)

Returns None, since the moment generating function does not exist.

`dist_genpareto.`**`k_x`**$(t,$ *k=0*$)$

Returns None, since the cumulant generating function does not exist.

`dist_genpareto.`**`moments`**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Generalized Pareto distribution (see Kleiber_2007_Dagum_moments). The rth moment exists for $c < 1/r$ and equals

$$\mu_k = \frac{r! \sigma!}{\prod_{i=1}^{r}(1 - ic)}.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_genpareto(mu, sigma).moments(k))
6.356352346256452561561561456 1356E+00
```

dist_genpareto.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Generalized Pareto distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_genpareto(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

4.5.19 Gompertz-Makeham distribution

class ctx.**dist_gompertz**(*a*, *b*, *lambda1=0*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Gompertz-Makeham distribution is a continuous probability distribution with parameters $a > 0, b > 0, \lambda \geq 0$, and the support interval $(0, +\infty)$. The hazard function has the form $\mathrm{hf}_X(x) = ae^{bx} + \lambda$.

See also: Wikipedia [1207], MathWorld [248], Jodrá [385], Riffi [479].

dist_gompertz.**pdf**(*x*)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Gompertz-Makeham distribution:

$$\mathrm{pdf}_X(x) = \mathrm{hf}_X(x) \cdot \mathrm{sf}_X(x) = (ae^{bx} + \lambda) \cdot \exp\left[-\lambda x - \frac{a}{b}\left(e^{bx} - 1\right)\right]$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_gompertz(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_gompertz.**cdf**(*x*)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Gompertz-Makeham distribution:

$$\mathrm{cdf}_X(x) = 1 - \exp\left[-\lambda x - \frac{a}{b}\left(e^{bx} - 1\right)\right] = -\mathrm{expm1}\left[-\lambda x - \frac{a}{b}\left(\mathrm{expm1}(bx)\right)\right]$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_gompertz(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_gompertz.**sf**(*x*)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Gompertz-Makeham distribution:

$$\mathrm{sf}_X(x) = \exp\left[-\lambda x - \frac{a}{b}\left(e^{bx} - 1\right)\right] = \exp\left[-\lambda x - \frac{a}{b}\left(\mathrm{expm1}(bx)\right)\right]$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_gompertz(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_gompertz.**qtf**(*q*)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Gompertz-Makeham distribution:

$$\mathrm{qtf}_X(q) = \begin{cases} \dfrac{1}{b}\mathrm{log1p}\left(-\dfrac{b}{a}\,\mathrm{log1p}(-q)\right) & \text{for } \lambda = 0, \\ \dfrac{a}{b\lambda} - \dfrac{1}{\lambda}\mathrm{log1p}(-q) - \dfrac{1}{b}W_0\left(\dfrac{a}{\lambda}e^{a/\lambda} \cdot \mathrm{pow1p}(-q, -b/\lambda)\right) & \text{for } \lambda > 0, \end{cases}$$

where W_0 denotes the principal branch of the Lambert W function (see ...).

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_gompertz(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

`dist_gompertz.isf(q)`

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Gompertz-Makeham distribution:

$$\text{isf}_X(q) = \begin{cases} \frac{1}{b}\text{log1p}\left(-\frac{b}{a}\log(q)\right) & \text{for } \lambda = 0, \\ \frac{a}{b\lambda} - \frac{1}{\lambda}\log(q) - \frac{1}{b}W_0\left(\frac{a}{\lambda}e^{a/\lambda}\cdot q^{-b/\lambda}\right) & \text{for } \lambda > 0, \end{cases}$$

where W_0 denotes the principal branch of the Lambert W function (see ...).

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_gompertz(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

`dist_gompertz.c_x(t)`

Returns $C_X(t)$, the characteristic function of a random variable X, following an Gompertz-Makeham distribution:

$$C_X(t) = e^d\left(d\cdot E_{p(it)}(d) + \frac{\lambda}{b}E_{1+p(it)}(d)\right), \quad \text{where } p(it) = \frac{\lambda - it}{b} \text{and } d = \frac{a}{b}.$$

Here $E_p(z)$ denotes the exponential integral, defined as $E_p(z) = z^{p-1}\Gamma(1-p,z) = z^{p-1}e^{-z}U(p,p,z)$. $\Gamma(\cdot)$ denotes the incomplete gamma function (see ...), and $U(\cdot)$ denotes Tricomi's hypergeometric function (see ...).

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_gompertz(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_gompertz.m_x(t)`

Returns $M_X(t)$, the moment generating function of a random variable X, following an Gompertz-Makeham distribution:

$$M_X(t) = e^d\left(d\cdot E_{p(t)}(d) + \frac{\lambda}{b}E_{1+p(t)}(d)\right), \quad \text{where } p(t) = \frac{\lambda - t}{b} \text{and } d = \frac{a}{b}.$$

Here $E_p(z)$ denotes the exponential integral, defined as $E_p(z) = z^{p-1}\Gamma(1-p,z) = z^{p-1}e^{-z}U(p,p,z)$. $\Gamma(\cdot)$ denotes the incomplete gamma function (see ...), and $U(\cdot)$ denotes Tricomi's hypergeometric function (see ...).

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_gompertz(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_gompertz.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Gompertz-Makeham distribution:

$$K_X(t) = \log(M_X(t))$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_gompertz(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_gompertz.**moments**(*k*)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Gompertz-Makeham distribution. The moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_0^1 x^r \text{pdf}_X(x) \text{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_gompertz(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

dist_gompertz.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Gompertz-Makeham distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_gompertz(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

4.5.20 Lomax distribution

class ctx.**dist_lomax**(*n1*, *n2*, *lambda*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

These functions return PDF, CDF, and ICDF of the Lomax distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$:

See also: Wikipedia [1215], Kleiber and Kotz [393],

dist_lomax.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Lomax distribution:

$$\mathrm{pdf}_X(x) = \frac{a}{b}\left(1+\frac{x}{b}\right)^{-(a+1)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_lomax(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_lomax.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Lomax distribution:

$$\mathrm{cdf}_X(x) = 1 - \left(1+\frac{x}{b}\right)^{-a}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_lomax(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_lomax.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Lomax distribution:

$$\mathrm{sf}_X(x) = \left(1+\frac{x}{b}\right)^{-a}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_lomax(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_lomax.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Lomax distribution:

$$\mathrm{qtf}_X(q) = b\left((1-p)^{-\frac{1}{a}} - 1\right)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_lomax(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_lomax.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Lomax distribution:

$$\text{isf}_X(q) = b\left((1-q)^{-\frac{1}{a}} - 1\right)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_lomax(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_lomax.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Lomax distribution:

$$C_X(t) = \int_0^\infty e^{itx}\text{pdf}_X(x)\mathrm{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_lomax(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_lomax.**m_x**(*t*)

Returns None, since the moment generating function does not exist.

dist_lomax.**k_x**(*t*, *k=0*)

Returns None, since the cumulant generating function does not exist.

dist_lomax.**moments**(*k*)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Lomax distribution (see Kleiber_2007_Dagum_moments). The kth moment exists for $-k < a$ and equals

$$\mu_k = \frac{b^k\Gamma(a-k)\Gamma(1+k)}{\Gamma(a)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_lomax(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_lomax.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Lomax distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_lomax(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

4.5.21 Shifted Gompertz distribution

class ctx.**dist_shifted_gompertz**(*a*, *b*, *lambda1=0*, ***kwargs*)

Shifted Gompertz distribution is a continuous probability distribution with parameters $a > 0, b > 0$, and the support interval $[0, +\infty)$.

See also: Wikipedia [1217], MathWorld [255], Jiménez and Pedro Jodrá [381], Jiménez Torres [382].

dist_shifted_gompertz.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Shifted Gompertz distribution:

$$\mathrm{pdf}_X(x) = be^{-(bx+ae^{-bx})}\left(1 + a\left(1 - e^{-bx}\right)\right), \quad x > 0.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_shifted_gompertz(mu, sigma).pdf(x))
6.356352346256452561561561561456 1356E-20
```

dist_shifted_gompertz.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Shifted Gompertz distribution:

$$\mathrm{cdf}_X(x) = \left(1 - e^{-bx}\right)e^{-ae^{-bx}}, \quad x > 0.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_shifted_gompertz(mu, sigma).pdf(x))
6.356352346256452561561561561456 1356E-20
```

dist_shifted_gompertz.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Shifted Gompertz distribution:

$$\mathrm{sf}_X(x) = 1 - \left(1 - e^{-bx}\right)e^{-ae^{-bx}}, \quad x > 0.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_shifted_gompertz(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456 1356E-20
```

dist_shifted_gompertz.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Shifted Gompertz distribution:

$$\mathrm{qtf}_X(q) = \frac{1}{b}\log\left(1 - \frac{W_0(ae^a q)}{a}\right)$$

where W_0 denotes the principal branch of the Lambert W function (see ...).

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_shifted_gompertz(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

`dist_shifted_gompertz.isf(`q`)`

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Shifted Gompertz distribution:

$$\text{isf}_X(q) = \frac{1}{b} \log\left(1 - \frac{W_0(ae^a(1-q))}{a}\right)$$

where W_0 denotes the principal branch of the Lambert W function (see ...).

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_shifted_gompertz(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

`dist_shifted_gompertz.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following an Shifted Gompertz distribution:

$$C_X(t) = a^{it/(b-1)}(a + it/b)\Gamma(1 - it/b, a) + e^{-a}.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_shifted_gompertz(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_shifted_gompertz.m_x(`t`)`

Returns $M_X(t)$, the moment generating function of a random variable X, following an Shifted Gompertz distribution:

$$M_X(t) = a^{t/(b-1)}(a + t/b)\Gamma(1 - t/b, a) + e^{-a}.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_shifted_gompertz(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_shifted_gompertz.k_x(`t`, k=0)`

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Shifted Gompertz distribution:

$$K_X(t) = \log(M_X(t))$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_shifted_gompertz(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_shifted_gompertz.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Shifted Gompertz distribution. The moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_0^1 x^r \text{pdf}_X(x) \text{d}x$$

There is an explicit expression for the mean:

$$\mu'_X(1) = E(X) = \frac{1}{b}\left(\gamma + \log(a) + \frac{1 - e^{-a}}{a} + E_1(a)\right).$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_shifted_gompertz(mu, sigma).moments(k))
6.3563523462564525615615615614561356E+00
```

dist_shifted_gompertz.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Shifted Gompertz distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_shifted_gompertz(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

4.5.22 Singh-Maddala (Burr Type XII) distribution

class ctx.**dist_singh_maddala**(*n1*, *n2*, *lambda*, ***kwargs*)

These functions return PDF, CDF, and ICDF of the Singh-Maddala distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$:

See also: Wikipedia [1197], Rodriguez [483], Kleiber and Kotz [393] (page 198), Kumar [402],

dist_singh_maddala.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Singh-Maddala distribution:

$$\text{pdf}_X(x) = f(x; c, d) = \frac{adx^{a-1}}{b^a(1 + (x/b)^a)^{1+d}}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_singh_maddala(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_singh_maddala.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Singh-Maddala distribution:

$$\text{cdf}_X(x) = 1 - (1 + (x/b)^a)^{-d}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_singh_maddala(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_singh_maddala.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Singh-Maddala distribution:

$$\text{sf}_X(x) = (1 + (x/b)^a)^{-d}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_singh_maddala(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_singh_maddala.**qtf**(d)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Singh-Maddala distribution:

$$\text{qtf}_X(q) = b[(1 - q)^{-1/d} - 1]^{1/a}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; d = 0.3;
>>> print ("qtf: ", dist_singh_maddala(mu, sigma).qtf(d))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_singh_maddala.**isf**(d)

Returns $\text{isf}_X(d)$, the inverse survival function function (isf) of a random variable X, following an Singh-Maddala distribution:

$$\text{isf}_X(d) = b[(1-q)^{-1/d} - 1]^{1/a}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; d = 0.3;
>>> print ("isf: ", dist_singh_maddala(mu, sigma).isf(d))
6.35635234625645256156156145613 56E+00
```

dist_singh_maddala.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Singh-Maddala distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_singh_maddala(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_singh_maddala.**m_x**(t)

Returns None, since the moment generating function does not exist.

dist_singh_maddala.**k_x**(t, *k=0*)

Returns None, since the cumulant generating function does not exist.

dist_singh_maddala.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Singh-Maddala distribution. The kth moment exists for $-a < k < aq$; it equals (see Kleiber(2003), page 201)

$$\mu_n = \frac{b^k \Gamma(1 + k/a)\Gamma(d - k/a)}{\Gamma(d)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_singh_maddala(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_singh_maddala.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Singh-Maddala distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_singh_maddala(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

4.6 Closed form distributions, based on the error function

4.6.1 !!!Boost: Lévy distribution

The following functions return the pdf, cdf, qtf or boost class of the Lévy distribution with location $a \in \mathbb{R}$, scale $b > 0$ and the support interval $(a, +\infty)$.

See also Wikipedia [1184], Ehrhardt [297] (3.9.16).

`math53.levy_pdf`(a, b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the Lévy distribution:

$$\text{pdf}(x) = \sqrt{\frac{b}{2\pi}} \frac{e^{-\frac{b}{2(x-a)}}}{(x-a)^{3/2}}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("LevyPdf(x, a, b): ", LevyPdf(x, a, b))
>>> print ("dist_levy(a, b).pdf(x): ", dist_levy(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.levy_cdf`(a, b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Lévy distribution:

$$\text{cdf}(x) = \text{erfc}\left(\sqrt{\frac{b}{2(x-a)}}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("LevyCdf(x, a, b): ", LevyCdf(x, a, b))
>>> print ("dist_levy(a, b).cdf(x): ", dist_levy(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.levy_qtf`(a, b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Lévy distribution:

$$\text{qtf}(q) = a + \frac{b}{2(\text{erfc}^{-1}(q))^2}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("LevyQtf(q, a, b): ", LevyQtf(q, a, b))
>>> print ("dist_levy(a, b).qtf(q): ", dist_levy(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class `ctx.dist_levy`(a, b)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The Lévy distribution is a continuous probability distribution with location $a \in \mathbb{R}$, scale $b > 0$ and the support interval $(a, +\infty)$. See also Wikipedia [1184], MathWorld [830].

dist_levy.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Lévy distribution:

$$\text{pdf}_X(x) = \sqrt{\frac{b}{2\pi}} \frac{e^{-\frac{b}{2(x-a)}}}{(x-a)^{3/2}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", levy(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

dist_levy.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Lévy distribution:

$$\text{cdf}_X(x) = \text{erfc}\left(\sqrt{\frac{b}{2(x-a)}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", levy(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

dist_levy.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following a Lévy distribution:

$$\text{sf}_X(x) = \text{erf}\left(\sqrt{\frac{b}{2(x-a)}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", levy(mu, sigma).pdf(x))
sf: 6.356352346256452561561561456135 6E-20
```

dist_levy.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a Lévy distribution:

$$\text{qtf}_X(q) = a + \frac{b}{2(\text{erfc}^{-1}(q))^2}.$$

where $\text{erfc}(\cdot)$ and $\text{erfc}^{-1}(\cdot)$ denote the complementary error function and its functional inverse, respectively.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", levy(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561456135 6E+00
```

dist_levy.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a Lévy distribution:

$$\text{isf}_X(q) = a + \frac{b}{2(\text{erf}^{-1}(q))^2}.$$

where $\text{erfc}(\cdot)$ and $\text{erfc}^{-1}(\cdot)$ denote the complementary error function and its functional inverse, respectively.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", levy(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_levy.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Lévy distribution:

$$C_X(t) = e^{i\mu t - \sqrt{-2ict}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", levy(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_levy.**m_x**(t)

Returns NaN, since the moment generating function does not exist.

dist_levy.**k_x**(t, *k=0*)

Returns NaN, since the cumulant generating function does not exist.

dist_levy.**moments**(k)

Returns NaN, since moments do not exist.

dist_levy.**cumulants**(k)

Returns NaN, since cumulants do not exist.

4.6.2 Boost: Lognormal (Johnson S_L) distribution

The following functions return the pdf, cdf, qtf or boost class of the lognormal distribution with location $a \in \mathbb{R}$, scale $b > 0$, and the support interval $(0, +\infty)$.

See also Wikipedia [1158], MathWorld [827], BoostMath [61], Ehrhardt [297] (3.9.19).

`math53.lognormal_pdf`(a, b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the lognormal distribution:

$$\text{pdf}(x) = \frac{1}{bx\sqrt{2\pi}} \exp\left(-\frac{(\log(x) - a)^2}{2b^2}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("LognormalPdf(x, a, b): ", LognormalPdf(x, a, b))
>>> print ("dist_lognormal(a, b).pdf(x): ", dist_lognormal(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.lognormal_cdf`(a, b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the lognormal distribution:

$$\text{cdf}(x) = \frac{1}{2}\text{erfc}\left(-\frac{\log(x) - a}{b\sqrt{2}}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("LognormalCdf(x, a, b): ", LognormalCdf(x, a, b))
>>> print ("dist_lognormal(a, b).cdf(x): ", dist_lognormal(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.lognormal_qtf`(a, b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the lognormal distribution:

$$\text{qtf}(q) = \exp\left(a - b\sqrt{2} \cdot \text{erfc}^{-1}(2q)\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("LognormalQtf(q, a, b): ", LognormalQtf(q, a, b))
>>> print ("dist_lognormal(a, b).qtf(q): ", dist_lognormal(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class `ctx.dist_lognormal`(a, b)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The lognormal distribution is a continuous probability distribution with location $a \in \mathbb{R}$, scale $b > 0$, and the support interval $(0, +\infty)$. See also Wikipedia [1158], MathWorld [827], BoostMath [61], Witkovský [1472], R (Statistical System) [525].

dist_lognormal.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an lognormal distribution:

$$\text{pdf}_X(x) = \frac{1}{bx\sqrt{2\pi}} \exp\left(-\frac{(\log(x)-a)^2}{2b^2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", lognormal(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_lognormal.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an lognormal distribution:

$$\text{cdf}_X(x) = \frac{1}{2}\text{erfc}\left(-\frac{\log(x)-a}{b\sqrt{2}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", lognormal(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_lognormal.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an lognormal distribution:

$$\text{sf}_X(x) = \frac{1}{2}\text{erfc}\left(\frac{\log(x)-a}{b\sqrt{2}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", lognormal(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_lognormal.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an lognormal distribution:

$$\text{qtf}_X(q) = \exp\left(a - b\sqrt{2}\cdot\text{erfc}^{-1}(2q)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", lognormal(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_lognormal.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an lognormal distribution:

$$\text{isf}_X(q) = \exp\left(a + b\sqrt{2} \cdot \text{erfc}^{-1}(2q)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", lognormal(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_lognormal.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an lognormal distribution. The characteristic function is defined for real values of t, but is not defined for any complex value of t that has a negative imaginary part, and hence the characteristic function is not analytic at the origin. Consequently, the characteristic function of the log-normal distribution cannot be represented as an infinite convergent series. A closed-form formula for the characteristic function in the domain of convergence is not known. A relatively simple approximating formula is available in closed form, and is given by

$$C_X(t) = \frac{\exp(-\frac{V^2+2V}{2\sigma^2})}{\sqrt{1+V}}, \quad \text{where } V = W(-it\sigma^2 e^{\mu}).$$

where W is the Lambert W function. This approximation is derived via an asymptotic method, but it stays sharp all over the domain of convergence.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", lognormal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_lognormal.**m_x**(t)

The moment generating function does not exist.

dist_lognormal.**k_x**(t, *k=0*)

The cumulant generating function does not exist.

dist_lognormal.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an lognormal distribution. All moments of the log-normal distribution exist.

$$\mu'_X(r) = \exp(r\mu + r^2\sigma^2/2).$$

However, the log-normal distribution is not determined by its moments. This implies that it cannot have a defined moment generating function in a neighborhood of zero.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", lognormal(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_lognormal.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an lognormal distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", lognormal(mu, sigma).cumulants(k))
6.35635234625645256156156145561356E+00
```

4.6.3 !!!Boost: Moyal Distribution

The following functions return the pdf, cdf, qtf or boost class of the Moyal distribution with location $a \in \mathbb{R}$, scale $b > 0$, and the support interval $(-\infty, +\infty)$.

See also MathWorld [253], G. [341], Walck [790], Ehrhardt [297] (3.9.21).

`math53.moyal_pdf(a, b, x)`

Returns pdf(x), the value of the probability density function (*Pdf*) of the Moyal distribution:

$$\text{pdf}(x) = \frac{1}{\sqrt{2\pi}b} \exp\left(-\frac{x-a}{2b} - \frac{1}{2}e^{-\frac{x-a}{b}}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("MoyalPdf(x, a, b): ", MoyalPdf(x, a, b))
>>> print ("dist_moyal(a, b).pdf(x): ", dist_moyal(a, b).pdf(x))
6.356352346256452561561561561456 1356E+00
```

`math53.moyal_cdf(a, b, x)`

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Moyal distribution:

$$\text{cdf}(x) = 1 - P\left(\frac{1}{2}, \frac{e^{-x}}{2}\right) = \text{erfc}\left(\frac{e^{-\frac{x-a}{2b}}}{\sqrt{2}}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("MoyalCdf(x, a, b): ", MoyalCdf(x, a, b))
>>> print ("dist_moyal(a, b).cdf(x): ", dist_moyal(a, b).cdf(x))
6.356352346256452561561561561456 1356E+00
```

`math53.moyal_qtf(a, b, q)`

Returns qtf(q), the value of the quantile function (*Qtf*) of the Moyal distribution:

$$\text{qtf}(q) = a - b\log\left(2(\text{erfc}^{-1}(q))^2\right)$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("MoyalQtf(q, a, b): ", MoyalQtf(q, a, b))
>>> print ("dist_moyal(a, b).qtf(q): ", dist_moyal(a, b).qtf(q))
6.356352346256452561561561561456 1356E+00
```

class `ctx.dist_moyal(a, b)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The Moyal distribution is a continuous probability distribution with location $a \in \mathbb{R}$, scale $b > 0$, and the support interval $(-\infty, +\infty)$. See also MathWorld [253], G. [341], Walck [790].

Other References: Amath: inversion, Walck: moments and char func.

dist_moyal.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Moyal distribution:

$$\mathrm{pdf}_X(x) = \frac{1}{\sqrt{2\pi}b} \exp\left(-\frac{x-a}{2b} - \frac{1}{2}e^{-\frac{x-a}{b}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", moyal(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_moyal.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Moyal distribution:

$$\mathrm{cdf}_X(x) = 1 - P\left(\frac{1}{2}, \frac{e^{-x}}{2}\right) = \mathrm{erfc}\left(\frac{e^{-\frac{x-a}{2b}}}{\sqrt{2}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", moyal(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_moyal.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Moyal distribution:

$$\mathrm{sf}_X(x) = P\left(\frac{1}{2}, \frac{e^{-x}}{2}\right) = \mathrm{erf}\left(\frac{e^{-\frac{x-a}{2b}}}{\sqrt{2}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", moyal(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_moyal.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Moyal distribution:

$$\mathrm{qtf}_X(q) = a - b\log\left(2(\mathrm{erfc}^{-1}(q))^2\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", moyal(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

dist_moyal.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Moyal distribution:

$$\text{isf}_X(q) = a - b \log\left(2(\text{erf}^{-1}(q))^2\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", moyal(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_moyal.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Moyal distribution:

$$C_X(t) = \frac{2^{it}}{\sqrt{\pi}}\Gamma\left(\tfrac{1}{2} - it\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", moyal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_moyal.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Moyal distribution:

$$M_X(t) = \frac{2^{t}}{\sqrt{\pi}}\Gamma\left(\tfrac{1}{2} - t\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", moyal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_moyal.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Moyal distribution:

$$K_X(t) = \log\left[\frac{2^{t}}{\sqrt{\pi}}\Gamma\left(\tfrac{1}{2} - t\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", moyal(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_moyal.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Moyal distribution. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", moyal(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_moyal.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \dots k$, of a random variable X, following an Moyal distribution. The cumulants are calculated from the moments.

$$\kappa_X(1) = \log(2) + \gamma, \qquad \kappa_X(n) = (n-1)!(2^n - 1)\zeta(n), \quad n \geq 2.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", moyal(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.6.4 Boost: Normal (Johnson S_N) distribution

The following functions return the pdf, cdf, qtf or boost class of the normal distribution with mean $\mu \in \mathbb{R}$, standard deviation $\sigma > 0$, and the support interval $(-\infty, +\infty)$.

See also Wikipedia [1166], MathWorld [833], BoostMath [67], Ehrhardt [297] (3.9.24).

`math53.`**`normal_pdf`**(*mu=0*, *sigma=1*, *x*)

Returns pdf(x), the value of the probability density function (*Pdf*) of the normal distribution:

$$\text{pdf}(x) = \phi(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("NormalPdf(x, a, b): ", NormalPdf(x, a, b))
>>> print ("dist_normal(a, b).pdf(x): ", dist_normal(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`normal_cdf`**(*mu=0*, *sigma=1*, *x*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the normal distribution:

$$\text{cdf}(x) = \Phi(x) = \frac{1}{2}\text{erfc}\left(-\frac{x-\mu}{\sigma\sqrt{2}}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("NormalCdf(x, a, b): ", NormalCdf(x, a, b))
>>> print ("dist_normal(a, b).cdf(x): ", dist_normal(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`normal_qtf`**(*mu=0*, *sigma=1*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the normal distribution:

$$\text{qtf}(q) = \mu - \sigma\sqrt{2} \cdot \text{erfc}^{-1}(2q).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("NormalQtf(q, a, b): ", NormalQtf(q, a, b))
>>> print ("dist_normal(a, b).qtf(q): ", dist_normal(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

`class` `ctx.`**`dist_normal`**(*mu*, *sigma*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The normal distribution is a continuous probability distribution with mean $\mu \in \mathbb{R}$, standard deviation $\sigma > 0$, and the support interval $(-\infty, +\infty)$. See also Wikipedia [1166], MathWorld [833], BoostMath [67], R (Statistical System) [526], Mpmath [543], Mpmath [539].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nrdtrimn.html#scipy.special.nrdtrimn

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nrdtrisd.html#scipy.special.nrdtrisd

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.ndtr.html#scipy.special.ndtr

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.log_ndtr.html#scipy.special.log_ndtr

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.ndtri.html#scipy.special.ndtri

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.ndtri_exp.html#scipy.special.ndtri_exp

`dist_normal.`**`pdf`**`(`x`)`

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a normal distribution:

$$\text{pdf}_X(x) = \phi(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", normal(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

`dist_normal.`**`cdf`**`(`x`)`

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a normal distribution:

$$\text{cdf}_X(x) = \Phi(x) = \frac{1}{2}\text{erfc}\left(-\frac{x-\mu}{\sigma\sqrt{2}}\right).$$

Here $\text{erf}(\cdot)$ and $\text{erf}^{-1}(\cdot)$ are the error function and its functional inverse, respectively.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", normal(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

`dist_normal.`**`sf`**`(`x`)`

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a normal distribution:

$$\text{sf}_X(x) = 1 - \Phi(x) = \frac{1}{2}\text{erfc}\left(\frac{x-\mu}{\sigma\sqrt{2}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", normal(mu, sigma).pdf(x))
sf: 6.356352346256452561561561456135 6E-20
```

`dist_normal.`**`qtf`**`(`q`)`

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a normal distribution:

$$\text{qtf}_X(q) = \mu - \sigma\sqrt{2} \cdot \text{erfc}^{-1}(2q).$$

Here $\text{erf}(\cdot)$ and $\text{erf}^{-1}(\cdot)$ are the error function and its functional inverse, respectively.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", normal(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_normal.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a normal distribution:

$$\text{isf}_X(q) = \mu + \sigma\sqrt{2} \cdot \text{erfc}^{-1}(2q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", normal(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_normal.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a normal distribution:

$$C_X(t) = \exp\left(i\mu t - \tfrac{1}{2}\sigma^2 t^2\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", normal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_normal.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a normal distribution:

$$M_X(t) = \exp\left(\mu t + \tfrac{1}{2}\sigma^2 t^2\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", normal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_normal.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a normal distribution:

$$K_X(t) = \mu t + \tfrac{1}{2}\sigma^2 t^2,$$

$$K_X^{(1)}(t) = \mu + \sigma^2 t, \quad K_X^{(2)}(t) = \sigma^2, \quad K_X^{(j)}(t) = 0, \quad j > 2.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", normal(mu, sigma).k_x(t, k))
6.3563523462564525615615615614561356E+00
```

dist_normal.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \dots k$, of a random variable X, following a normal distribution: the moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", normal(mu, sigma).moments(k))
6.3563523462564525615615615614561356E+00
```

dist_normal.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \dots k$, of a random variable X, following a normal distribution:

$$\kappa_1 = \mu, \quad \kappa_2 = \sigma^2, \quad \kappa_j = 0, \quad j > 2.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", normal(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

4.6.5 Boost: Skew normal Distribution

The following functions return the pdf, cdf, qtf or boost class of the skew normal distribution with location $a \in \mathbb{R}$, scale $b > 0$, shape $c \in \mathbb{R}$, and the support interval $(-\infty, +\infty)$.

See also Wikipedia [1191], MathWorld [256], BoostMath [86], Haas [360].

`math53.skewnormal_pdf(`a, b, c, x`)`

Returns pdf(x), the value of the probability density function (*Pdf*) of the skew normal distribution:

$$\text{pdf}(x) = \frac{2}{b}\phi\left(\frac{x-a}{b}\right)\Phi\left(c\left(\frac{x-a}{b}\right)\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("SkewnormalPdf(x, a, b): ", SkewnormalPdf(x, a, b))
>>> print ("dist_skewnormal(a, b).pdf(x): ", dist_skewnormal(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.skewnormal_cdf(`a, b, c, x`)`

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the skew normal distribution:

$$\text{cdf}(x) = \Phi\left(\frac{x-a}{b}\right) - 2T\left(\frac{x-a}{b}, c\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("SkewnormalCdf(x, a, b): ", SkewnormalCdf(x, a, b))
>>> print ("dist_skewnormal(a, b).cdf(x): ", dist_skewnormal(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.skewnormal_qtf(`a, b, c, q`)`

Returns qtf(q), the value of the quantile function (*Qtf*) of the skew normal distribution:

There is no known closed form for qtf(q): it computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("SkewnormalQtf(q, a, b): ", SkewnormalQtf(q, a, b))
>>> print ("dist_skewnormal(a, b).qtf(q): ", dist_skewnormal(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

`class ctx.dist_skewnormal(`a, b, c`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The skew normal distribution is a continuous probability distribution with location $a \in \mathbb{R}$, scale $b > 0$, shape $c \in \mathbb{R}$, and the support interval $(-\infty, +\infty)$. See also Wikipedia [1191], MathWorld [256], BoostMath [86], Witkovský [1494], Haas [360].

dist_skewnormal.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a skew normal distribution:

$$\text{pdf}_X(x) = \frac{2}{b}\phi\left(\frac{x-a}{b}\right)\Phi\left(c\left(\frac{x-a}{b}\right)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_skewnormal(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_skewnormal.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a skew normal distribution:

$$\text{cdf}_X(x) = \Phi\left(\frac{x-a}{b}\right) - 2T\left(\frac{x-a}{b}, c\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_skewnormal(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_skewnormal.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following a skew normal distribution:

$$\text{sf}_X(x) = \Phi\left(-\frac{x-a}{b}\right) + 2T\left(\frac{x-a}{b}, c\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_skewnormal(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_skewnormal.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a skew normal distribution:

There is no known closed form for $\text{qtf}_X(q)$ or $\text{isf}_X(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_skewnormal(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_skewnormal.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a skew normal distribution:

There is no known closed form for $\text{qtf}_X(q)$ or $\text{isf}_X(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_skewnormal(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_skewnormal.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following a skew normal distribution:

$$C_X(t) = 2\exp\left(ita - \frac{b^2t^2}{2}\right)\Phi(itbd).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_skewnormal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_skewnormal.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following a skew normal distribution:

$$M_X(t) = 2\exp\left(ta + \frac{b^2t^2}{2}\right)\Phi(tbd).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_skewnormal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_skewnormal.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a skew normal distribution:

$$K_X(t) = ta + \frac{b^2t^2}{2} + \log\left(2\Phi(tbd)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", mp_skewnormal(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_skewnormal.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a skew normal distribution:

$$\mu'_{2r+1} = \sqrt{\frac{2}{\pi}}\frac{(2r+1)!}{2^r r!}\sum_{j=0}^{r}(-1)^j\binom{r}{j}\frac{d^{2j+1}}{2j+1}$$

The even moments are equal to those of the standard normal. Hass 2012: Odd Moments

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_skewnormal(mu, sigma).moments(k))
6.356352346256452561561561456l356E+00
```

dist_skewnormal.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a skew normal distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_skewnormal(mu, sigma).cumulants(k))
6.356352346256452561561561456l356E+00
```

4.6.6 Boost: Wald (or Inverse Gaussian) distribution

The following functions return the pdf, cdf, qtf or boost class of the Wald distribution with mean $\mu > 0$, scale $b > 0$, and the support interval $(0, +\infty)$.

See also Wikipedia [1155], MathWorld [825], BoostMath [59], Ehrhardt [297] (3.9.32).

math53.**wald_pdf**(*mu*, *b*, *x*)

Returns pdf(x), the value of the probability density function (*Pdf*) of the Wald distribution:

$$\text{pdf}(x) = \sqrt{\frac{b}{2\pi x^3}} \exp\left(\frac{-b(x-\mu)^2}{2\mu^2 x}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("WaldPdf(x, a, b): ", WaldPdf(x, a, b))
>>> print ("dist_wald(a, b).pdf(x): ", dist_wald(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**wald_cdf**(*mu*, *b*, *x*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Wald distribution:

$$\text{cdf}(x) = \Phi\left(\sqrt{\frac{b}{x}}\left(\frac{x}{\mu} - 1\right)\right) + \exp\left(\frac{2b}{\mu}\right)\Phi\left(-\sqrt{\frac{b}{x}}\left(\frac{x}{\mu} + 1\right)\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("WaldCdf(x, a, b): ", WaldCdf(x, a, b))
>>> print ("dist_wald(a, b).cdf(x): ", dist_wald(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**wald_qtf**(*mu*, *b*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Wald distribution:

There is no known closed form for qtf(q): it is computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("WaldQtf(q, a, b): ", WaldQtf(q, a, b))
>>> print ("dist_wald(a, b).qtf(q): ", dist_wald(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_wald**(*mu*, *b*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The Wald (or inverse Gaussian) distribution is a continuous probability distribution with mean $\mu > 0$, scale $b > 0$, and the support interval $(0, +\infty)$. See also Wikipedia [1155], MathWorld [825], BoostMath [59], Witkovský [1470].

dist_wald.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Wald distribution:

$$\text{pdf}_X(x) = \sqrt{\frac{b}{2\pi x^3}} \exp\left(\frac{-b(x-\mu)^2}{2\mu^2 x}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", wald(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_wald.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Wald distribution:

$$\text{cdf}_X(x) = \Phi\left(\sqrt{\frac{b}{x}}\left(\frac{x}{\mu} - 1\right)\right) + \exp\left(\frac{2b}{\mu}\right)\Phi\left(-\sqrt{\frac{b}{x}}\left(\frac{x}{\mu} + 1\right)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", wald(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_wald.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a Wald distribution:

$$\text{sf}_X(x) = \Phi\left(-\sqrt{\frac{b}{x}}\left(\frac{x}{\mu} - 1\right)\right) - \exp\left(\frac{2b}{\mu}\right)\Phi\left(-\sqrt{\frac{b}{x}}\left(\frac{x}{\mu} + 1\right)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", wald(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_wald.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Wald distribution:

There is no known closed form for $\text{qtf}_X(q)$ or $\text{isf}_X(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", wald(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_wald.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Wald distribution:

There is no known closed form for $\text{qtf}_X(q)$ or $\text{isf}_X(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", wald(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_wald.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Wald distribution:

$$C_X(t) = \exp\left[\frac{\lambda}{\mu}\left(1 - \sqrt{1 - \frac{2\mu^2 it}{\lambda}}\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wald(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wald.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Wald distribution:

$$M_X(t) = \exp\left[\frac{\lambda}{\mu}\left(1 - \sqrt{1 - \frac{2\mu^2 t}{\lambda}}\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", wald(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wald.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a Wald distribution:

$$K_X(t) = \frac{\lambda}{\mu}\left(1 - \sqrt{1 - \frac{2\mu^2 t}{\lambda}}\right).$$

$$K_X^{(j)}(t) = tbd.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", wald(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_wald.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Wald distribution. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wald(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_wald.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Wald distribution.

$$\kappa_X(n+1) = \kappa_{X,n+1} = \frac{(2n)!}{2^n n! \mu^{2n+1} \lambda^n}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wald(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.6.7 Birnbaum-Saunders Distribution

class ctx.**dist_birnb_saunders**(*mu*, *sigma*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Birnbaum-Saunders distribution is a continuous probability distribution with mean $\mu \in \mathbb{R}$, standard deviation $\sigma > 0$, and the support interval $(-\infty, +\infty)$. See also Wikipedia [1174], MathWorld [246], Balakrishnan and Kundi [25].

dist_birnb_saunders.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Birnbaum-Saunders distribution:

$$\text{pdf}_X(x) = \frac{1}{2\sqrt{2\pi}\alpha\beta}\left[\sqrt{\frac{t}{\beta}} + \sqrt{\frac{\beta^3}{t^3}}\right] \exp\left[-\frac{1}{2\alpha^2}\left(\frac{t}{\beta} + \frac{\beta}{t} - 2\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_birnb_saunders(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_birnb_saunders.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Birnbaum-Saunders distribution:

$$\text{cdf}_X(x) = \Phi\left[\frac{1}{\alpha}\left(\sqrt{\frac{t}{\beta}} - \sqrt{\frac{\beta}{t}}\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_birnb_saunders(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_birnb_saunders.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Birnbaum-Saunders distribution:

$$\mathrm{sf}_X(x) = 1 - \Phi\left[\frac{1}{\alpha}\left(\sqrt{\frac{t}{\beta}} - \sqrt{\frac{\beta}{t}}\right)\right].$$

```

```

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_birnb_saunders(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_birnb_saunders.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Birnbaum-Saunders distribution:

$$\mathrm{qtf}_X(q) = \frac{\beta}{4}\left[\alpha Z + \sqrt{(\alpha Z)^2 + 4}\right]^2, \quad Z = \Phi^{-1}(q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_birnb_saunders(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

dist_birnb_saunders.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Birnbaum-Saunders distribution:

$$\mathrm{isf}_X(q) = \frac{\beta}{4}\left[\alpha Z + \sqrt{(\alpha Z)^2 + 4}\right]^2, \quad Z = \Phi^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_birnb_saunders(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_birnb_saunders.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Birnbaum-Saunders distribution:

$$C_X(t) = \frac{1}{2}\exp\left[\frac{\beta}{\alpha} - \sqrt{\frac{\beta^2}{\alpha^2} - 2it\beta}\right]\left(1 + \sqrt{\frac{1}{1 - 2it\alpha^2/\beta}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_birnb_saunders(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_birnb_saunders.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Birnbaum-Saunders distribution:

$$M_X(t) = \frac{1}{2}\exp\left[\frac{\beta}{\alpha} - \sqrt{\frac{\beta^2}{\alpha^2} - 2t\beta}\right]\left(1 + \sqrt{\frac{1}{1 - 2t\alpha^2/\beta}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", mp_birnb_saunders(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_birnb_saunders.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Birnbaum-Saunders distribution:

$$K_X(t) = \frac{\beta}{\alpha} - \sqrt{\frac{\beta^2}{\alpha^2} - 2t\beta} + \log\left(1 + \sqrt{\frac{1}{1 - 2t\alpha^2/\beta}}\right) - \log(2).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", mp_birnb_saunders(mu, sigma).k_x(t, k))
6.356352346256452561561561561456 1356E+00
```

dist_birnb_saunders.**moments**(*k*)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Birnbaum-Saunders distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(r) = \beta^r \sum_{j=0}^{r} \binom{2r}{2j} \sum_{i=0}^{j} \binom{i}{j} \frac{(2r - 2j + 2i)!}{2^{r-j+i}(r - j + i)!}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_birnb_saunders(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

dist_birnb_saunders.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Birnbaum-Saunders distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_birnb_saunders(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.6.8 Exponentially Modified Gaussian (EMG) distribution

class ctx.**dist_emg**(*n1*, *n2*, *lambda*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

These functions return PDF, CDF, and ICDF of the exponentially modified Gaussian distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$:

See also: Wikipedia [1201].

dist_emg.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an exponentially modified Gaussian distribution:

$$\mathrm{pdf}_X(x) = f(x;\mu,\sigma,\lambda) = \frac{\lambda}{2} e^{\frac{\lambda}{2}(2\mu+\lambda\sigma^2-2x)} \operatorname{erfc}\left(\frac{\mu+\lambda\sigma^2-x}{\sqrt{2}\sigma}\right)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_emg(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_emg.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an exponentially modified Gaussian distribution:

$$\mathrm{cdf}_X(x) = \Phi(x,\mu,\sigma) - \tfrac{1}{2} e^{\frac{\lambda}{2}(2\mu+\lambda\sigma^2-2x)} \operatorname{erfc}\left(\frac{\mu+\lambda\sigma^2-x}{\sqrt{2}\sigma}\right)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_emg(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_emg.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an exponentially modified Gaussian distribution:

$$\mathrm{sf}_X(x) = \Phi(-x,\mu,\sigma) + \tfrac{1}{2} e^{\frac{\lambda}{2}(2\mu+\lambda\sigma^2-2x)} \operatorname{erfc}\left(\frac{\mu+\lambda\sigma^2-x}{\sqrt{2}\sigma}\right)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_emg(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_emg.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an exponentially modified Gaussian distribution:

$$\text{qtf}_X(q) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_emg(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_emg.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an exponentially modified Gaussian distribution:

$$\text{isf}_X(q) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_emg(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_emg.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an exponentially modified Gaussian distribution:

$$C_X(t) = \left(1 - \frac{it}{\lambda}\right)^{-1} \exp\left(i\mu t - \tfrac{1}{2}\sigma^2 t^2\right)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_emg(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_emg.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an exponentially modified Gaussian distribution:

$$M_X(t) = \left(1 - \frac{t}{\lambda}\right)^{-1} \exp\left(\mu t + \tfrac{1}{2}\sigma^2 t^2\right)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", dist_emg(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_emg.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an exponentially modified Gaussian distribution:

$$K_X(t) = \log(M_X(t))$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", dist_emg(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_emg.**moments**(*k*)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an exponentially modified Gaussian distribution. The moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_0^1 x^r \text{pdf}_X(x) \text{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_emg(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_emg.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an exponentially modified Gaussian distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_emg(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.6.9 Folded normal distribution

class ctx.**dist_folded_normal**(*n1*, *n2*, *lambda*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

These functions return PDF, CDF, and ICDF of the folded normal distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$:

See also: Wikipedia [1203], Tsagris *et al.* [788], Reig *et al.* [475].

dist_folded_normal.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an folded normal distribution:

$$\mathrm{pdf}_X(x) = f_Y(x;\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}}\, e^{-\frac{(x-\mu)^2}{2\sigma^2}} + \frac{1}{\sqrt{2\pi\sigma^2}}\, e^{-\frac{(x+\mu)^2}{2\sigma^2}}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_folded_normal(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_folded_normal.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an folded normal distribution:

$$\mathrm{cdf}_X(x) = F_Y(x;\mu,\sigma^2) = \frac{1}{2}\left[\mathrm{erf}\left(\frac{x+\mu}{\sqrt{2\sigma^2}}\right) + \mathrm{erf}\left(\frac{x-\mu}{\sqrt{2\sigma^2}}\right)\right]$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_folded_normal(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_folded_normal.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an folded normal distribution:

$$\mathrm{sf}_X(x) = 1 - F_Y(x;\mu,\sigma^2) = 1 - \frac{1}{2}\left[\mathrm{erf}\left(\frac{x+\mu}{\sqrt{2\sigma^2}}\right) + \mathrm{erf}\left(\frac{x-\mu}{\sqrt{2\sigma^2}}\right)\right]$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_folded_normal(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_folded_normal.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an folded normal distribution:

$$\mathrm{qtf}_X(q) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_folded_normal(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456 1356E+00
```

`dist_folded_normal.`**`isf`**`(`q`)`

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an folded normal distribution:

$$\text{isf}_X(q) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_folded_normal(mu, sigma).isf(q))
6.356352346256452561561561561456 1356E+00
```

`dist_folded_normal.`**`c_x`**`(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following an folded normal distribution:

$$C_X(t) = e^{\frac{-\sigma^2 t^2}{2} + i\mu t}\Phi\left(\frac{\mu}{\sigma} + i\sigma t\right) + e^{-\frac{\sigma^2 t^2}{2} - i\mu t}\Phi\left(-\frac{\mu}{\sigma} + i\sigma t\right).$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_folded_normal(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

`dist_folded_normal.`**`m_x`**`(`t`)`

Returns $M_X(t)$, the moment generating function of a random variable X, following an folded normal distribution:

$$M_X(t) = e^{\frac{\sigma^2 t^2}{2} + \mu t}\Phi\left(\frac{\mu}{\sigma} + \sigma t\right) + e^{\frac{\sigma^2 t^2}{2} - \mu t}\Phi\left(-\frac{\mu}{\sigma} + \sigma t\right).$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", dist_folded_normal(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

`dist_folded_normal.`**`k_x`**`(`t`, `*`k=0`*`)`

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an folded normal distribution:

$$K_X(t) = \left(\frac{\sigma^2 t^2}{2} + \mu t\right) + \log\left\{1 - \Phi\left(-\frac{\mu}{\sigma} - \sigma t\right) + e^{\frac{\sigma^2 t^2}{2} - \mu t}\left[1 - \Phi\left(\frac{\mu}{\sigma} - \sigma t\right)\right]\right\}.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", dist_folded_normal(mu, sigma).k_x(t, k))
6.356352346256452561561561561456 1356E+00
```

dist_folded_normal.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an folded normal distribution. The moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_0^1 x^r \text{pdf}_X(x) \text{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_folded_normal(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

dist_folded_normal.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an folded normal distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_folded_normal(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

4.6.10 Half-normal distribution

class ctx.**dist_half_normal**(*sigma*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The half-normal distribution is a continuous probability distribution with standard deviation $\sigma > 0$, and the support interval $(0, +\infty)$. See also Wikipedia [1177], MathWorld [823], Witkovský [1486], .

dist_half_normal.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an half-normal distribution:

$$\mathrm{pdf}_X(x) = \frac{\sqrt{2}}{\sigma\sqrt{\pi}} \exp\left(\frac{x^2}{2\sigma^2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", half_normal(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_half_normal.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an half-normal distribution:

$$\mathrm{cdf}_X(x) = \mathrm{erf}\left(\frac{x}{\sigma\sqrt{2}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", half_normal(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_half_normal.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following an half-normal distribution:

$$\mathrm{sf}_X(x) = \mathrm{erfc}\left(\frac{x}{\sigma\sqrt{2}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", half_normal(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_half_normal.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an half-normal distribution:

$$\mathrm{qtf}_X(q) = \sigma\sqrt{2} \cdot \mathrm{erf}^{-1}(q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", half_normal(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

`dist_half_normal.isf(`q`)`

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an half-normal distribution:

$$\text{isf}_X(q) = \sigma\sqrt{2} \cdot \text{erfc}^{-1}(q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", half_normal(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

`dist_half_normal.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following an half-normal distribution:

$$C_X(t) = M\left(\frac{3}{2}, \frac{1}{2}\frac{-t^2}{2}\right) + \frac{2\sqrt{2}\,it}{\sqrt{\pi}} M\left(2, \frac{3}{2}, \frac{-t^2}{2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", half_normal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_half_normal.m_x(`t`)`

Returns $M_X(t)$, the moment generating function of a random variable X, following an half-normal distribution:

$$M_X(t) = M\left(\frac{3}{2}, \frac{1}{2}\frac{t^2}{2}\right) + \frac{2\sqrt{2}\,t}{\sqrt{\pi}} M\left(2, \frac{3}{2}, \frac{t^2}{2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", half_normal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_half_normal.k_x(`t`, k=0)`

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an half-normal distribution:

$$K_X(t) = \log\left[M\left(\frac{3}{2}, \frac{1}{2}\frac{t^2}{2}\right) + \frac{2\sqrt{2}\,t}{\sqrt{\pi}} M\left(2, \frac{3}{2}, \frac{t^2}{2}\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", half_normal(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_half_normal.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \dots k$, of a random variable X, following an half-normal distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(r) = \begin{cases} \sqrt{\frac{2}{\pi}} k! \alpha^{2k-1} & \text{for } n = 2k-1, \\ (n+1)!! \alpha^n & \text{for } n \text{ even}, \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", half_normal(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_half_normal.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \dots k$, of a random variable X, following an half-normal distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", half_normal(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.6.11 Johnson S_B distribution

class ctx.**dist_johnson_sb**(*n1*, *n2*, *lambda*, ***kwargs*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

These functions return PDF, CDF, and ICDF of the Johnson S_B distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$:

See also: Wikipedia [1213], Jodrá [384], Hill *et al.* [372].

dist_johnson_sb.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Johnson S_B distribution:

$$\mathrm{pdf}_X(x) = f(x; a, b) = \frac{b}{u(1-u)}\phi\left(a + b\log\left(\frac{u}{1-u}\right)\right), \quad \text{where } u = (x-\xi)/\lambda, a = \gamma, b = \delta.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_johnson_sb(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_johnson_sb.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Johnson S_B distribution:

$$\mathrm{cdf}_X(x) = \Phi\left(a + b\log\left(\frac{x}{1-x}\right)\right), \quad \text{where } u = (x-\xi)/\lambda, a = \gamma, b = \delta$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_johnson_sb(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_johnson_sb.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Johnson S_B distribution:

$$\mathrm{sf}_X(x) = \Phi\left(-a - b\log\left(\frac{x}{1-x}\right)\right), \quad \text{where } u = (x-\xi)/\lambda, a = \gamma, b = \delta$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_johnson_sb(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_johnson_sb.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Johnson S_B distribution:

$$\mathrm{qtf}_X(q) = \lambda \cdot u + \xi, u = \frac{1}{1 + \exp\left(-\frac{1}{b}\left(\Phi^{-1}(q) - a\right)\right)}, \quad \text{where } a = \gamma, b = \delta.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_johnson_sb(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456 1356E+00
```

`dist_johnson_sb.isf(`q`)`

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Johnson S_B distribution:

$$\text{isf}_X(q) = \lambda \cdot u + \xi, u = \frac{1}{1 + \exp\left(-\frac{1}{b}\left(\Phi^{-1}(1-q) - a\right)\right)}, \quad \text{where } a = \gamma, b = \delta.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_johnson_sb(mu, sigma).isf(q))
6.356352346256452561561561456 1356E+00
```

`dist_johnson_sb.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following an Johnson S_B distribution:

$$C_X(t) = \int_0^1 e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_johnson_sb(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

`dist_johnson_sb.m_x(`t`)`

Returns $M_X(t)$, the moment generating function of a random variable X, following an Johnson S_B distribution:

$$M_X(t) = \int_0^1 e^{tx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", dist_johnson_sb(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

`dist_johnson_sb.k_x(`t`, `*k=0*`)`

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Johnson S_B distribution:

$$K_X(t) = \log(M_X(t))$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", dist_johnson_sb(mu, sigma).k_x(t, k))
6.356352346256452561561561456 1356E+00
```

dist_johnson_sb.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Johnson S_B distribution. The moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_0^1 x^r \text{pdf}_X(x) \text{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_johnson_sb(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

dist_johnson_sb.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Johnson S_B distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_johnson_sb(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

4.6.12 Johnson S_U distribution

class ctx.**dist_johnson_su**(*n1*, *n2*, *lambda*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

These functions return PDF, CDF, and ICDF of the Johnson S_U distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$:

See also: Wikipedia [1213], Jodrá [384], Hill *et al.* [372].

dist_johnson_su.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Johnson S_U distribution:

$$\mathrm{pdf}_X(x) = \frac{b}{\sqrt{x^2+1}}\phi\left(a + b\log\left(x + \sqrt{x^2+1}\right)\right), \quad \text{where } u = (x-\xi)/\lambda, a = \gamma, b = \delta.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_johnson_su(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_johnson_su.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Johnson S_U distribution:

$$\mathrm{cdf}_X(x) = \Phi\left(a + b\log\left(x + \sqrt{x^2+1}\right)\right), \quad \text{where } u = (x-\xi)/\lambda, a = \gamma, b = \delta$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_johnson_su(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_johnson_su.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Johnson S_U distribution:

$$\mathrm{sf}_X(x) = \Phi\left(-a - b\log\left(x + \sqrt{x^2+1}\right)\right), \quad \text{where } u = (x-\xi)/\lambda, a = \gamma, b = \delta$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_johnson_su(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_johnson_su.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Johnson S_U distribution:

$$\mathrm{qtf}_X(q) = \sinh\left(\frac{\Phi^{-1}(q) - a}{b}\right), \quad \text{where } u = (x-\xi)/\lambda, a = \gamma, b = \delta$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_johnson_su(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

`dist_johnson_su.isf(`q`)`

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Johnson S_U distribution:

$$\text{isf}_X(q) = \sinh\left(\frac{\Phi^{-1}(1-q) - a}{b}\right), \quad \text{where } u = (x-\xi)/\lambda, a = \gamma, b = \delta$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_johnson_su(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

`dist_johnson_su.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following an Johnson S_U distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \text{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_johnson_su(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_johnson_su.m_x(`t`)`

Returns $M_X(t)$, the moment generating function of a random variable X, following an Johnson S_U distribution:

$$M_X(t) = \int_0^1 e^{tx} \text{pdf}_X(x) \text{d}x$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", dist_johnson_su(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_johnson_su.k_x(`t`, `*`k=0`*`)`

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Johnson S_U distribution:

$$K_X(t) = \log(M_X(t))$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", dist_johnson_su(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_johnson_su.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Johnson S_U distribution (See Johnson1949, page 163)

If r is even:

$$\mu'_r = 2^{-(r-1)} \sum_{s=0}^{\frac{1}{2}r-1} (-1)^s \binom{r}{s} e^{\frac{1}{2}(r-2s)^2\delta^{-2}} \cosh[(r-2s)(\gamma/\delta)] + (-1)^{\frac{1}{2}r} \frac{1}{2} \binom{r}{\frac{1}{2}r}$$

If r is odd:

$$\mu'_r = 2^{-(r-1)} \sum_{s=0}^{\frac{1}{2}r-1} (-1)^{s+1} \binom{r}{s} e^{\frac{1}{2}(r-2s)^2\delta^{-2}} \sinh[(r-2s)(\gamma/\delta)]$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_johnson_su(mu, sigma).moments(k))
6.356352346256452561561561561456l356E+00
```

dist_johnson_su.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Johnson S_U distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_johnson_su(mu, sigma).cumulants(k))
6.356352346256452561561561561456l356E+00
```

4.6.13 Normal maximum distribution, $\rho_{ij,i\neq j} = 0$

class ctx.**dist_nmax_0**(k)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The normal maximum distribution (the distribution of the maximum of $k \geq 1$ independent standard normal variates) is a continuous probability distribution with the support interval $(-\infty, +\infty)$.

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom. Then

$$Q_1 = \frac{\max X_j}{s}, \quad j = 1, \ldots, k$$

follows a studentized maximum distribution with k and n degrees of freedom, and

$$Q_2 = \frac{\max|X_j|}{s}, \quad j = 1, \ldots, k$$

follows a Studentized Maximum Modulus distribution with k and n degrees of freedom.

See also Stoline and Ury [507], Hochberg and Tamhane [373], Narula [422].

For tables see Bechhofer and Dunnett [26], Stoline and Ury [507], and Hahn and Hendrickson [361].

dist_nmax_0.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a normal maximum distribution:

$$\text{pdf}_X(x) = f_{\text{nmax0}}(x, k) = k \cdot \Phi(x)^{k-1} \cdot \phi(x).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_smm(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_nmax_0.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a normal maximum distribution:

$$\text{cdf}_X(x) = F_{\text{nmax0}}(x, k) = [\Phi(x)]^k.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_smm(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_nmax_0.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following a normal maximum distribution:

$$\text{sf}_X(x) = 1 - [\Phi(x)]^k.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_smm(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

`dist_nmax_0.qtf(`q`)`

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a normal maximum distribution:

$$\text{qtf}_X(x) = \Phi^{-1}\left(q^{1/k}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_smm(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

`dist_nmax_0.isf(`q`)`

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a normal maximum distribution:

$$\text{isf}_X(x) = \Phi^{-1}\left((1-q)^{1/k}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_smm(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

`dist_nmax_0.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following a normal maximum distribution:

$$C_X(t) = \int_0^\infty e^{itx}\text{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_smm(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

`dist_nmax_0.m_x(`t`)`

Returns $M_X(t)$, the moment generating function of a random variable X, following a normal maximum distribution:

$$M_X(t) = \int_0^\infty e^{tx}\text{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_smm(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_nmax_0.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a normal maximum distribution:

$$K_X(t) = \log\left(M_X(t)\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_smm(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_nmax_0.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a normal maximum distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = \int_0^\infty x^r \mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_smm(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_nmax_0.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a central Studentized Maximum Modulus distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_smm(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.6.14 Normal maximum modulus distribution, $\rho_{ij,i\neq j} = 0$

class ctx.**dist_nmm_0**(k)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The normal maximum modulus distribution (the distribution of the maximum of the absolute value of $k \geq 1$ independent standard normal variates) is a continuous probability distribution with the support interval $(0, +\infty)$.

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom. Then

$$Q_1 = \frac{\max X_j}{s}, \quad j = 1, \ldots, k$$

follows a studentized maximum distribution with k and n degrees of freedom, and

$$Q_2 = \frac{\max|X_j|}{s}, \quad j = 1, \ldots, k$$

follows a Studentized Maximum Modulus distribution with k and n degrees of freedom.

See also Stoline and Ury [507], Hochberg and Tamhane [373], Narula [422].

For tables see Bechhofer and Dunnett [26], Stoline and Ury [507], and Hahn and Hendrickson [361].

dist_nmm_0.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a normal maximum modulus distribution:

$$\text{pdf}_X(x) = f_{\text{nmm0}}(x, k) = 2k \cdot (2\Phi(x) - 1)^{k-1} \cdot \phi(x), \quad x > 0.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_smm(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

dist_nmm_0.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a normal maximum modulus distribution:

$$\text{cdf}_X(x) = F_{\text{nmm0}}(x, k) = [\Phi(x) - \Phi(-x)]^k = [2\Phi(x) - 1]^k, \quad x > 0.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_smm(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

dist_nmm_0.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following a normal maximum modulus distribution:

$$\text{sf}_X(x) = 1 - [\Phi(x) - \Phi(-x)]^k = 1 - [2\Phi(x) - 1]^k, \quad x > 0.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_smm(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

`dist_nmm_0.qtf`(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a normal maximum modulus distribution:

$$\text{qtf}_X(x) = \Phi^{-1}\left(\tfrac{1}{2} + \tfrac{1}{2}q^{1/k}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_smm(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561456135 6E+00
```

`dist_nmm_0.isf`(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a normal maximum modulus distribution:

$$\text{isf}_X(x) = \Phi^{-1}\left(\tfrac{1}{2} + \tfrac{1}{2}(1-q)^{1/k}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_smm(mu, sigma).isf(q))
6.356352346256452561561561456135 6E+00
```

`dist_nmm_0.c_x`(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a normal maximum modulus distribution:

$$C_X(t) = \int_0^\infty e^{itx}\text{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_smm(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

`dist_nmm_0.m_x`(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a normal maximum modulus distribution:

$$M_X(t) = \int_0^\infty e^{tx}\text{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_smm(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

`dist_nmm_0.k_x`(t, k=*0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a normal maximum modulus distribution:

$$K_X(t) = \log\left(M_X(t)\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_smm(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

`dist_nmm_0.moments`(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a normal maximum modulus distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = \int_0^\infty x^r \mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_smm(mu, sigma).moments(k))
6.3563523462564525615615615614561356E+00
```

`dist_nmm_0.cumulants`(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a central Studentized Maximum Modulus distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_smm(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

4.6.15 Sinh-arcsinh normal distribution

class ctx.**dist_sasnormal**(a, b)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The sinh-arcsinh normal distribution distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, and the support interval $(0, +\infty)$.

See also: Jones and Faddy [391].

dist_sasnormal.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a sinh-arcsinh normal distribution distribution:

$$\text{pdf}_X(x) = \frac{1}{\sqrt{2\pi}} \frac{\delta C_{\epsilon,\delta}(x)}{\sqrt{1+x^2}} \exp\left(-\tfrac{1}{2} S^2_{\epsilon,\delta}(x)\right), \quad \text{where } S_{\epsilon,\delta}(x) = \sinh\left(\epsilon + \delta \sinh^{-1}(x)\right), \text{and } C_{\epsilon,\delta}(x) = \sqrt{1 + S^2_{\epsilon,\delta}(x)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_sasnormal.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a sinh-arcsinh normal distribution distribution:

$$\text{cdf}_X(x) = \Phi\left(S_{\epsilon,\delta}(x)\right), \quad \text{where } S_{\epsilon,\delta}(x) = \sinh\left(\epsilon + \delta \sinh^{-1}(x)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_sasnormal.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a sinh-arcsinh normal distribution distribution:

$$\text{sf}_X(x) = 1 - \Phi\left(S_{\epsilon,\delta}(x)\right), \quad \text{where } S_{\epsilon,\delta}(x) = \sinh\left(\epsilon + \delta \sinh^{-1}(x)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_sasnormal.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a sinh-arcsinh normal distribution distribution:

$$\text{qtf}_X(q) = S_{-\epsilon/\delta,1/\delta}\left(\Phi^{-1}(q)\right), \quad \text{where } S_{\epsilon,\delta}(x) = \sinh\left(\epsilon + \delta \sinh^{-1}(x)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561356E+00
```

dist_sasnormal.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a sinh-arcsinh normal distribution distribution:

$$\text{isf}_X(q) = S_{-\epsilon/\delta,1/\delta}\left(\Phi^{-1}(1-q)\right), \quad \text{where } S_{\epsilon,\delta}(x) = \sinh\left(\epsilon + \delta \sinh^{-1}(x)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561356E+00
```

dist_sasnormal.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a sinh-arcsinh normal distribution distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.356352346256452561561561561356E+00
```

dist_sasnormal.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a sinh-arcsinh normal distribution distribution:

$$M_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.356352346256452561561561561356E+00
```

dist_sasnormal.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a sinh-arcsinh normal distribution distribution:

$$K_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_sasnormal.`**`moments`**`(`k`)`

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a sinh-arcsinh normal distribution distribution. All moments exist and are given by

$$\mu'_X(r) = \frac{1}{2^r} \sum_{j=0}^{r} (-1)^j \exp\left((r-2j)(\epsilon/\delta)\right) P_{(r-2j)/\delta}, \quad \text{where } P_q = \frac{e^{1/4}}{\sqrt{8\pi}} \left(K_{(q+1)/2}(1/4) + K_{(q-1)/2}(1/4)\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

`dist_sasnormal.`**`cumulants`**`(`k`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a sinh-arcsinh normal distribution distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.6.16 Truncated normal distribution

class ctx.**dist_truncated_normal**(*n1*, *n2*, *lambda*, ***kwargs*)

These functions return PDF, CDF, and ICDF of the truncated normal distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$:

See also: Wikipedia [1219], Orjebin [452], Burkardt [164],

John Burkardt: The Truncated Normal Distribution (has closed form qtf)

dist_truncated_normal.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an truncated normal distribution:

$$\text{pdf}_X(x) = f(x; \mu, \sigma, a, b) = \frac{1}{\sigma} \frac{\phi(\frac{x-\mu}{\sigma})}{\Phi\left(\frac{b-\mu}{\sigma}\right) - \Phi\left(\frac{a-\mu}{\sigma}\right)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_truncated_normal(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_truncated_normal.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an truncated normal distribution:

$$\text{cdf}_X(x) = \frac{\Phi(\xi) - \Phi(\alpha)}{\Phi(\beta) - \Phi(\alpha)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_truncated_normal(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_truncated_normal.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following an truncated normal distribution:

$$\text{sf}_X(x) = 1 - \frac{\Phi(\xi) - \Phi(\alpha)}{\Phi(\beta) - \Phi(\alpha)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_truncated_normal(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_truncated_normal.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an truncated normal distribution:

$$\text{qtf}_X(q) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_truncated_normal(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_truncated_normal.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an truncated normal distribution:

$$\text{isf}_X(q) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_truncated_normal(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_truncated_normal.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an truncated normal distribution:

$$C_X(t) = e^{\mu it - \sigma^2 t^2/2} \cdot \frac{\Phi(\beta - \sigma it) - \Phi(\alpha - \sigma it)}{\Phi(\beta) - \Phi(\alpha)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_truncated_normal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_truncated_normal.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an truncated normal distribution:

$$M_X(t) = e^{\mu t + \sigma^2 t^2/2} \cdot \frac{\Phi(\beta - \sigma t) - \Phi(\alpha - \sigma t)}{\Phi(\beta) - \Phi(\alpha)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", dist_truncated_normal(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_truncated_normal.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an truncated normal distribution:

$$K_X(t) = \log(M_X(t))$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", dist_truncated_normal(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_truncated_normal.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an truncated normal distribution. The moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_0^1 x^r \mathrm{pdf}_X(x) \mathrm{d}x$$

A recursive formula for μ_k (based on $\mu_{-1} = 0$ and $\mu_0 = 1$) is:

$$\mu'_X(r) = m_k = (k-1)\sigma^2 m_{k-2} + \mu m_{k-1} - \sigma \frac{b^{k-1}\phi\left(\frac{b-\mu}{\sigma}\right) - a^{k-1}\phi\left(\frac{a-\mu}{\sigma}\right)}{\Phi\left(\frac{b-\mu}{\sigma}\right) - \Phi\left(\frac{a-\mu}{\sigma}\right)}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_truncated_normal(mu, sigma).moments(k))
6.35635234625645256156156145613356E+00
```

dist_truncated_normal.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an truncated normal distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_truncated_normal(mu, sigma).cumulants(k))
6.35635234625645256156156145613356E+00
```

4.7 Closed form distributions, based on the incomplete gamma function

4.7.1 !!!Boost: Chi Distribution

The following functions return the pdf, cdf, qtf or boost class of the chi distribution with $n > 0$ degrees of freedom and the support interval $(0, +\infty)$.

See also Wikipedia [1175], MathWorld [818], Ehrhardt [297] (3.9.5)

math53.**chi_pdf**(n, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the chi distribution:

$$\text{pdf}(x) = 2x \times f_{\chi^2}\left(x^2, n\right).$$

Here $f_{\chi^2}(x, n)$ denotes the probability density function of a random variable following an chi-squared distribution with n degress of freedom.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("ChiPdf(x, a, b): ", ChiPdf(x, a, b))
>>> print ("dist_chi(a, b).pdf(x): ", dist_chi(a, b).pdf(x))
6.356352346256452561561561561456 1356E+00
```

math53.**chi_cdf**(n, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the chi distribution:

$$\text{cdf}(x) = P(n/2, x^2/2).$$

Here $P(\cdot)$ denotes the lower regularized incomplete gamma function (*RealGammaP*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("ChiCdf(x, a, b): ", ChiCdf(x, a, b))
>>> print ("dist_chi(a, b).cdf(x): ", dist_chi(a, b).cdf(x))
6.356352346256452561561561561456 1356E+00
```

math53.**chi_qtf**(n, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the chi distribution:

$$\text{qtf}(q) = \sqrt{2P^{-1}(n/2, q)}.$$

Here $P^{-1}(\cdot)$ denotes the inverse of the lower regularized incomplete gamma function (*RealGammaPInv*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("ChiQtf(q, a, b): ", ChiQtf(q, a, b))
>>> print ("dist_chi(a, b).qtf(q): ", dist_chi(a, b).qtf(q))
6.356352346256452561561561561456 1356E+00
```

class ctx.**dist_chi**(n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The chi distribution is a continuous probability distribution with $n > 0$ degrees of freedom and the support interval $(0, +\infty)$. See also Wikipedia [1175], MathWorld [818], Witkovský [1484].

dist_chi.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an chi distribution:

$$\text{pdf}_X(x) = 2x \times f_{\chi^2}\left(x^2, n\right).$$

Here $f_{\chi^2}(x, n)$ denotes the probability density function of a random variable following an chi-squared distribution with n degress of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", chi(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_chi.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an chi distribution:

$$\text{cdf}_X(x) = P(n/2, x^2/2).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", chi(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_chi.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an chi distribution:

$$\text{sf}_X(x) = Q(n/2, x^2/2).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", chi(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_chi.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an chi distribution:

$$\text{qtf}_X(q) = \sqrt{2P^{-1}(n/2, q)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", chi(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_chi.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an chi distribution:

$$\text{isf}_X(q) = \sqrt{2Q^{-1}(n/2, q)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", chi(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_chi.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an chi distribution:

$$C_X(t) = M\left(\frac{k}{2}, \frac{1}{2}, \frac{-t^2}{2}\right) + it\sqrt{2}\frac{\Gamma\left((k+1)/2\right)}{\Gamma(k/2)} M\left(\frac{k+1}{2}, \frac{3}{2}, \frac{-t^2}{2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_chi.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an chi distribution:

$$M_X(t) = M\left(\frac{k}{2}, \frac{1}{2}, \frac{t^2}{2}\right) + t\sqrt{2}\frac{\Gamma\left((k+1)/2\right)}{\Gamma(k/2)} M\left(\frac{k+1}{2}, \frac{3}{2}, \frac{t^2}{2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", chi(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_chi.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an chi distribution:

$$K_X(t) = \log\left[M\left(\frac{k}{2}, \frac{1}{2}, \frac{t^2}{2}\right) + t\sqrt{2}\frac{\Gamma\left((k+1)/2\right)}{\Gamma(k/2)} M\left(\frac{k+1}{2}, \frac{3}{2}, \frac{t^2}{2}\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", chi(mu, sigma).k_x(t, k))
6.3563523462564525615615615614561356E+00
```

dist_chi.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an chi distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(j) = 2^{2j} \frac{\Gamma\left((k+j)/2\right)}{\Gamma(k/2)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", chi(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_chi.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an chi distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", chi(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.7.2 Boost: Chi2 distribution

The following functions return the pdf, cdf, qtf or boost class of the chi-squared distribution with $n > 0$ degrees of freedom and the support interval $(0, +\infty)$.

See also Wikipedia [1148], MathWorld [796], BoostMath [53], Ehrhardt [297] (3.9.6)

`math53.chi2_pdf(`n, x`)`

Returns pdf(x), the value of the probability density function (*Pdf*) of the chi-squared distribution:

$$\text{pdf}(x) = f_{\chi^2}(x, n) = \frac{1}{2^{n/2}\Gamma(n/2)} x^{(n-2)/2} e^{-x/2}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("ChiSquaredPdf(x, a, b): ", ChiSquaredPdf(x, a, b))
>>> print ("dist_chi_squared(a, b).pdf(x): ", dist_chi_squared(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.chi2_cdf(`n, x`)`

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the chi-squared distribution:

$$\text{cdf}(x) = F_{\chi^2}(x, n) = P(n/2, x/2).$$

Here $P(\cdot)$ denotes the lower regularized incomplete gamma function (*RealGammaP*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("ChiSquaredCdf(x, a, b): ", ChiSquaredCdf(x, a, b))
>>> print ("dist_chi_squared(a, b).cdf(x): ", dist_chi_squared(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.chi2_qtf(`n, q`)`

Returns qtf(q), the value of the quantile function (*Qtf*) of the chi-squared distribution:

$$\text{qtf}(q) = 2P^{-1}(n/2, q).$$

Here $P^{-1}(\cdot)$ denotes the inverse of the lower regularized incomplete gamma function (*RealGammaPInv*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("ChiSquaredQtf(q, a, b): ", ChiSquaredQtf(q, a, b))
>>> print ("dist_chi_squared(a, b).qtf(q): ", dist_chi_squared(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

`class ctx.dist_chi2(`n`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The chi-squared distribution is a continuous probability distribution with $n > 0$ degrees of freedom and the support interval $(0, +\infty)$. See also Wikipedia [1148], MathWorld [796], BoostMath [53], Witkovský [1464], R (Statistical System) [521].

dist_chi_squared.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a chi-squared distribution:

$$\text{pdf}_X(x) = f_{\chi^2}(n, x) = \frac{1}{2^{n/2}\Gamma(n/2)} x^{(n-2)/2} e^{-x/2}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", chi_squared(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_chi_squared.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a chi-squared distribution:

$$\text{cdf}_X(x) = F_{\chi^2}(x, n) = P(n/2, x/2).$$

Here $P(\cdot)$ and $P^{-1}(\cdot)$ are the regularized gamma function and its functional inverse, respectively.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", chi_squared(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_chi_squared.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a chi-squared distribution:

$$\text{sf}_X(x) = 1 - F_{\chi^2}(x, n) = Q(n/2, x/2).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", chi_squared(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_chi_squared.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a chi-squared distribution:

$$\text{qtf}_X(q) = 2P^{-1}(n/2, q).$$

Here $P(\cdot)$ and $P^{-1}(\cdot)$ are the regularized gamma function and its functional inverse, respectively.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", chi_squared(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_chi_squared.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a chi-squared distribution:

$$\text{isf}_X(q) = 2Q^{-1}(n/2, q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", chi_squared(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_chi_squared.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a chi-squared distribution:

$$C_X(t) = (1 - 2it)^{-n/2}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_chi_squared.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a chi-squared distribution:

$$M_X(t) = (1 - 2t)^{-n/2}, \quad t \in \left(-\infty, \tfrac{1}{2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_chi_squared.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a chi-squared distribution:

$$K_X(t) = -\frac{n}{2}\log(1 - 2t), \quad t \in \left(-\infty, \tfrac{1}{2}\right),$$

$$K_X^{(j)}(t) = \frac{2^{j-1}(j-1)!}{(1-2t)^j}n.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", chi_squared(mu, sigma).k_x(t, k))
6.3563523462564525615615615614561356E+00
```

dist_chi_squared.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a chi-squared distribution: the moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", chi_squared(mu, sigma).moments(k))
6.35635234625645256156156145614561356E+00
```

dist_chi_squared.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a chi-squared distribution:

$$\kappa_{r+1} = 2^r r! n.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", chi_squared(mu, sigma).cumulants(k))
6.35635234625645256156156145614561356E+00
```

Recurrences: Central Chi-square

ctx.**chi_squared_recurrence**(x, n, *lambda*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The following recurrence relations hold for the pdf and CDF:

$$f_{\chi^2}(n+2, x) = \frac{x}{n} f_{\chi^2}(n, x)$$

$$F_{\chi^2}(n, x) - F_{\chi^2}(n+2, x) = 2 f_{\chi^2}(n+2, x)$$

Approximations

ctx.**chi_squared_gp**(x, n, *results='cdf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the pdf, cdf and sf from the characteristic function using the procedure of Gil-Pelaez (see *gil_pelaez_pdf()* and *gil_pelaez_cdf()*).

ctx.**chi_squared_ecf**(x, f, *results='cdf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

ctx.**chi_squared_ecf_inv**(q, f, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**chi_squared_spa**(x, n, *results='c'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

The solution $\hat{s}(x)$ of the saddlepoint equation $K_X^{(1)}(\hat{s}(x)) = x$, of a random variable X, following a non-central chi-squared distribution is given by:

$$\hat{s}(x) = -\frac{1}{4x}\left[n - 2x + n\right], \quad x > 0$$

ctx.**chi_squared_spa_inv**(*x*, *n*, *results='qtf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.7.3 Boost: Gamma (Pearson Type III, Erlang) distribution

The following functions return the pdf, cdf, qtf or boost class of the gamma distribution with shape $a > 0$, scale $b > 0$, and the support interval $(0, +\infty)$.

See also Wikipedia [1152], MathWorld [820], BoostMath [57], Ehrhardt [297] (3.9.10).

`math53.gamma_pdf`(a, b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the gamma distribution:

$$\text{pdf}(x) = \frac{x^{a-1}e^{-x/b}}{\Gamma(a)b^a}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("GammaPdf(x, a, b): ", GammaPdf(x, a, b))
>>> print ("dist_gamma(a, b).pdf(x): ", dist_gamma(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.gamma_cdf`(a, b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the gamma distribution:

$$\text{cdf}(x) = P(a, x/b).$$

Here $P(\cdot)$ denotes the lower regularized incomplete gamma function (*RealGammaP*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("GammaCdf(x, a, b): ", GammaCdf(x, a, b))
>>> print ("dist_gamma(a, b).cdf(x): ", dist_gamma(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.gamma_qtf`(a, b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the gamma distribution:

$$\text{qtf}(q) = b \cdot P^{-1}(a, q).$$

Here $P^{-1}(\cdot)$ denotes the inverse of the lower regularized incomplete gamma function (*RealGammaPInv*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("GammaQtf(q, a, b): ", GammaQtf(q, a, b))
>>> print ("dist_gamma(a, b).qtf(q): ", dist_gamma(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_gamma**(a, b)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The gamma distribution is a continuous probability distribution with shape $a > 0$, scale $b > 0$, and the support interval $(0, +\infty)$. See also Wikipedia [1152], MathWorld [820], BoostMath [57], Witkovský [1467], R (Statistical System) [524].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.gdtr.html#scipy.special.gdtr

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.gdtrc.html#scipy.special.gdtrc

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.gdtrib.html#scipy.special.gdtrib

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.gdtrix.html#scipy.special.gdtrix

dist_gamma.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a gamma distribution:

$$\text{pdf}_X(x) = \frac{x^{a-1}e^{-x/b}}{\Gamma(a)b^a}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", gamma(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_gamma.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a gamma distribution:

$$\text{cdf}_X(x) = P(a, x/b).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", gamma(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_gamma.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a gamma distribution:

$$\text{sf}_X(x) = Q(a, x/b).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", gamma(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_gamma.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a gamma distribution:

$$\text{qtf}_X(q) = b \cdot P^{-1}(a, q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", gamma(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

`dist_gamma.isf(q)`

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a gamma distribution:

$$\text{isf}_X(q) = b \cdot Q^{-1}(a, q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", gamma(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

`dist_gamma.c_x(t)`

Returns $C_X(t)$, the characteristic function of a random variable X, following a gamma distribution:

$$C_X(t) = \left(\frac{b^{-1}}{(b^{-1} - it)}\right)^a.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", gamma(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_gamma.m_x(t)`

Returns $M_X(t)$, the moment generating function of a random variable X, following a gamma distribution:

$$M_X(t) = \left(\frac{b^{-1}}{(b^{-1} - t)}\right)^a.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", gamma(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_gamma.k_x(t, k=0)`

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a gamma distribution:

$$K_X(t) = a \log\left(\frac{b^{-1}}{(b^{-1} - t)}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", gamma(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_gamma.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an exponential distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = b^r \frac{\Gamma(a+r)}{\Gamma(a)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", gamma(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

dist_gamma.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an exponential distribution. The cumulants are calculated from the moments.

$$\kappa'_X(r) = a \cdot b \cdot \Gamma(r).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", gamma(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

4.7.4 Boost: Inverse chisquared distribution

Returns the pdf, cdf, qtf or boost class of a random variable X, following an inverse chi-squared distribution with shape $a > 0$, scale $b > 0$, and the support interval $(0, +\infty)$.

See also Wikipedia [1156], BoostMath [58], Witkovský [1459].

`math53.`**`inverse_chi2_pdf`**(a, b, x)

Returns $\text{pdf}(x)$, the value of the probability density function (*Pdf*) of the inverse chis-quared distribution:

$$\text{pdf}(x) = \left(\frac{b}{x}\right)^a \frac{e^{-\frac{b}{x}}}{x\Gamma(a)}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("InvchisquaredPdf(x, a, b): ", InvchisquaredPdf(x, a, b))
>>> print ("dist_invchisquared(a, b).pdf(x): ", dist_invchisquared(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`inverse_chi2_cdf`**(a, b, x)

Returns $\text{cdf}(x)$, the value of the cumulative distribution function (*Cdf*) of the inverse chis-quared distribution:

$$\text{cdf}(x) = Q\left(a, -\frac{b}{x}\right).$$

Here $Q(\cdot)$ denotes the upper regularized incomplete gamma function (*RealGammaQ*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("InvchisquaredCdf(x, a, b): ", InvchisquaredCdf(x, a, b))
>>> print ("dist_invchisquared(a, b).cdf(x): ", dist_invchisquared(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`inverse_chi2_qtf`**(a, b, q)

Returns $\text{qtf}(q)$, the value of the quantile function (*Qtf*) of the inverse chis-quared distribution:

$$\text{qtf}(q) = \frac{b}{Q^{-1}(a, q)}.$$

Here $Q^{-1}(\cdot)$ denotes the inverse of the upper regularized incomplete gamma function (*RealGammaQInv*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("InvchisquaredQtf(q, a, b): ", InvchisquaredQtf(q, a, b))
>>> print ("dist_invchisquared(a, b).qtf(q): ", dist_invchisquared(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_inverse_chi2**(a, b)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The inverse chisquared distribution is a continuous probability distribution with shape $a > 0$, scale $b > 0$, and the support interval $(0, +\infty)$. See also Wikipedia [1180], MathWorld [251], BoostMath [82].

dist_invchisquared.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an inverse chisquared distribution:

$$\text{pdf}_X(x) = \left(\frac{b}{x}\right)^a \frac{e^{-\frac{b}{x}}}{x\Gamma(a)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_invchisquared(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_invchisquared.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an inverse chisquared distribution:

$$\text{cdf}_X(x) = Q\left(a, -\frac{b}{x}\right).$$

where $Q(\cdot)$ and $Q^{-1}(\cdot)$ denote the regularized gamma function and its functional inverse.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_invchisquared(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_invchisquared.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following an inverse chisquared distribution:

$$\text{sf}_X(x) = P\left(a, -\frac{b}{x}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_invchisquared(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_invchisquared.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an inverse chisquared distribution:

$$\text{qtf}_X(q) = \frac{b}{Q^{-1}(a, q)}.$$

where $Q(\cdot)$ and $Q^{-1}(\cdot)$ denote the regularized gamma function and its functional inverse.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_invchisquared(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

dist_invchisquared.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an inverse chisquared distribution:

$$\text{isf}_X(q) = \frac{b}{P^{-1}(a,q)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_invchisquared(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_invchisquared.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an inverse chisquared distribution:

$$C_X(t) = \frac{2(-i\beta t)^{\alpha/2}}{\Gamma(\alpha)} K_\alpha(\sqrt{-4i\beta t}),$$

where $K_n(\cdot)$ denotes the modified Bessel function of the second kind.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_invchisquared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_invchisquared.**m_x**(t)

The moment generating function does not exist.

dist_invchisquared.**k_x**(t, *k=0*)

The cumulant generating function does not exist.

dist_invchisquared.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an inverse chisquared distribution. The rth moments only exists for $\alpha > r$.

$$\mu'_X(r) = \frac{\beta^n}{(\alpha-1)\cdots(\alpha-n)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_invchisquared(mu, sigma).moments(k))
6.356352346256452561561561561456135 6E+00
```

dist_invchisquared.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an inverse chisquared distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_invchisquared(mu, sigma).cumulants(k))
6.35635234625645256156156145561356E+00
```

4.7.5 Boost: Inverse Gamma (Pearson Type V) distribution

Returns the pdf, cdf, qtf or boost class of a random variable X, following an inverse gamma distribution with shape $a > 0$, scale $b > 0$, and the support interval $(0, +\infty)$.

See also Wikipedia [1156], BoostMath [58], Witkovský [1459], Ehrhardt [297] (3.9.12).

`math53.inverse_gamma_pdf`(a, b, x)

Returns $\text{pdf}(x)$, the value of the probability density function (*Pdf*) of the inverse gamma distribution:

$$\text{pdf}(x) = \left(\frac{b}{x}\right)^a \frac{e^{-\frac{b}{x}}}{x\Gamma(a)}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("InvgammaPdf(x, a, b): ", InvgammaPdf(x, a, b))
>>> print ("dist_invgamma(a, b).pdf(x): ", dist_invgamma(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.inverse_gamma_cdf`(a, b, x)

Returns $\text{cdf}(x)$, the value of the cumulative distribution function (*Cdf*) of the inverse gamma distribution:

$$\text{cdf}(x) = Q\left(a, -\frac{b}{x}\right).$$

Here $Q(\cdot)$ denotes the upper regularized incomplete gamma function (*RealGammaQ*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("InvgammaCdf(x, a, b): ", InvgammaCdf(x, a, b))
>>> print ("dist_invgamma(a, b).cdf(x): ", dist_invgamma(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.inverse_gamma_qtf`(a, b, q)

Returns $\text{qtf}(q)$, the value of the quantile function (*Qtf*) of the inverse gamma distribution:

$$\text{qtf}(q) = \frac{b}{Q^{-1}(a, q)}.$$

Here $Q^{-1}(\cdot)$ denotes the inverse of the upper regularized incomplete gamma function (*RealGammaQInv*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("InvgammaQtf(q, a, b): ", InvgammaQtf(q, a, b))
>>> print ("dist_invgamma(a, b).qtf(q): ", dist_invgamma(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_inverse_gamma**(a, b)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The inverse gamma distribution is a continuous probability distribution with shape $a > 0$, scale $b > 0$, and the support interval $(0, +\infty)$. See also Wikipedia [1156], MathWorld [244], BoostMath [58], Witkovský [1469], Witkovský [1459].

dist_invgamma.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an inverse gamma distribution:

$$\text{pdf}_X(x) = \left(\frac{b}{x}\right)^a \frac{e^{-\frac{b}{x}}}{x\Gamma(a)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", invgamma(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_invgamma.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an inverse gamma distribution:

$$\text{cdf}_X(x) = Q\left(a, -\frac{b}{x}\right).$$

where $Q(\cdot)$ and $Q^{-1}(\cdot)$ denote the regularized gamma function and its functional inverse.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", invgamma(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_invgamma.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an inverse gamma distribution:

$$\text{sf}_X(x) = P\left(a, -\frac{b}{x}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", invgamma(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_invgamma.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an inverse gamma distribution:

$$\text{qtf}_X(q) = \frac{b}{Q^{-1}(a, q)}.$$

where $Q(\cdot)$ and $Q^{-1}(\cdot)$ denote the regularized gamma function and its functional inverse.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", invgamma(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_invgamma.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an inverse gamma distribution:

$$\text{isf}_X(q) = \frac{b}{P^{-1}(a, q)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", invgamma(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_invgamma.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an inverse gamma distribution:

$$C_X(t) = \frac{2(-i\beta t)^{\alpha/2}}{\Gamma(\alpha)} K_\alpha(\sqrt{-4i\beta t}),$$

where $K_n(\cdot)$ denotes the modified Bessel function of the second kind.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", invgamma(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_invgamma.**m_x**(t)

The moment generating function does not exist.

dist_invgamma.**k_x**(t, *k=0*)

The cumulant generating function does not exist.

dist_invgamma.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \dots k$, of a random variable X, following an inverse gamma distribution. The rth moments only exists for $\alpha > r$.

$$\mu'_X(r) = \frac{\beta^n}{(\alpha - 1) \cdots (\alpha - n)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", invgamma(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_invgamma.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an inverse gamma distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", invgamma(mu, sigma).cumulants(k))
6.35635234625645256156156145561356E+00
```

4.7.6 !!!Boost: Maxwell Distribution

Returns the pdf, cdf, qtf or boost class of a random variable X, following a Maxwell distribution with scale $b > 0$, and the support interval $(0, +\infty)$.

See also Wikipedia [1186], MathWorld [831], Ehrhardt [297] (3.9.20).

math53.**maxwell_pdf**(b, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the Maxwell distribution:

$$\text{pdf}(x) = \sqrt{\frac{2}{\pi}} \frac{x^2}{b^3} \exp\left(-\frac{x^2}{2b^2}\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("MaxwellPdf(x, a, b): ", MaxwellPdf(x, a, b))
>>> print ("dist_maxwell(a, b).pdf(x): ", dist_maxwell(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**maxwell_cdf**(b, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Maxwell distribution:

$$\text{cdf}(x) = P\left(\frac{3}{2}, \frac{x^2}{2b^2}\right).$$

Here $P(\cdot)$ denotes the lower regularized incomplete gamma function (*RealGammaP*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("MaxwellCdf(x, a, b): ", MaxwellCdf(x, a, b))
>>> print ("dist_maxwell(a, b).cdf(x): ", dist_maxwell(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**maxwell_qtf**(b, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Maxwell distribution:

$$\text{qtf}(q) = b\sqrt{2P^{-1}\left(\frac{3}{2}, q\right)}.$$

Here $P^{-1}(\cdot)$ denotes the inverse of the lower regularized incomplete gamma function (*RealGammaPInv*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("MaxwellQtf(q, a, b): ", MaxwellQtf(q, a, b))
>>> print ("dist_maxwell(a, b).qtf(q): ", dist_maxwell(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_maxwell**(b)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Maxwell distribution is a continuous probability distribution with scale $b > 0$, and the support interval $(0, +\infty)$. See also Wikipedia [1186], MathWorld [831], Witkovský [1487].

dist_maxwell.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Maxwell distribution:

$$\text{pdf}_X(x) = \sqrt{\frac{2}{\pi}} \frac{x^2}{b^3} \exp\left(-\frac{x^2}{2b^2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", maxwell(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_maxwell.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Maxwell distribution:

$$\text{cdf}_X(x) = P\left(\frac{3}{2}, \frac{x^2}{2b^2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", maxwell(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_maxwell.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an Maxwell distribution:

$$\text{sf}_X(x) = Q\left(\frac{3}{2}, \frac{x^2}{2b^2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", maxwell(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_maxwell.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an Maxwell distribution:

$$\text{qtf}_X(q) = b\sqrt{2P^{-1}\left(\frac{3}{2}, q\right)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", maxwell(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145614356E+00
```

dist_maxwell.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an Maxwell distribution:

$$\text{isf}_X(q) = b\sqrt{2Q^{-1}\left(\frac{3}{2}, q\right)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", maxwell(mu, sigma).isf(q))
6.35635234625645256156156145614356E+00
```

dist_maxwell.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Maxwell distribution:

$$C_X(t) = M\left(\frac{3}{2}, \frac{1}{2}\frac{-t^2}{2}\right) + \frac{2\sqrt{2}\,it}{\sqrt{\pi}} M\left(2, \frac{3}{2}, \frac{-t^2}{2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", maxwell(mu, sigma).c_x(t))
6.35635234625645256156156145614356E+00
```

dist_maxwell.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Maxwell distribution:

$$M_X(t) = M\left(\frac{3}{2}, \frac{1}{2}\frac{t^2}{2}\right) + \frac{2\sqrt{2}\,t}{\sqrt{\pi}} M\left(2, \frac{3}{2}, \frac{t^2}{2}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", maxwell(mu, sigma).c_x(t))
6.35635234625645256156156145614356E+00
```

dist_maxwell.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Maxwell distribution:

$$K_X(t) = \log\left[M\left(\frac{3}{2}, \frac{1}{2}\frac{t^2}{2}\right) + \frac{2\sqrt{2}\,t}{\sqrt{\pi}} M\left(2, \frac{3}{2}, \frac{t^2}{2}\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", maxwell(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_maxwell.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \dots k$, of a random variable X, following an Maxwell distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(r) = \begin{cases} \sqrt{\frac{2}{\pi}} k! \alpha^{2k-1} & \text{for } n = 2k-1, \\ (n+1)!! \alpha^n & \text{for } n \text{ even}, \end{cases}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", maxwell(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_maxwell.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \dots k$, of a random variable X, following an Maxwell distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", maxwell(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.7.7 !!!Boost: Nakagami distribution

The following functions return the pdf, cdf, qtf or boost class of the Nakagami distribution with shape $m > 0$, scale $\omega > 0$, and the support interval $(0, +\infty)$.

See also Wikipedia [1187], MathWorld [254], Dharmawansa P and Ahmed [241], Hauberg [365], Ehrhardt [297] (3.9.22).

See also: https://reference.wolfram.com/language/ref/NakagamiDistribution.html

See also: https://mathworld.wolfram.com/PochhammerSymbol.html

math53.**nakagami_pdf**(*m*, *omega*, *x*)

Returns pdf(x), the value of the probability density function (*Pdf*) of the Nakagami distribution:

$$\text{pdf}(x) = \frac{2m^m x^{2m-1}}{\omega^m \Gamma(m)} \exp\left(-\frac{m}{\omega}x^2\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("NakagamiPdf(x, a, b): ", NakagamiPdf(x, a, b))
>>> print ("dist_nakagami(a, b).pdf(x): ", dist_nakagami(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**nakagami_cdf**(*m*, *omega*, *x*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Nakagami distribution:

$$\text{cdf}(x) = P\left(\frac{m}{\omega}x^2\right).$$

Here $P(\cdot)$ denotes the lower regularized incomplete gamma function (*RealGammaP*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("NakagamiCdf(x, a, b): ", NakagamiCdf(x, a, b))
>>> print ("dist_nakagami(a, b).cdf(x): ", dist_nakagami(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**nakagami_qtf**(*m*, *omega*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Nakagami distribution:

$$\text{qtf}(q) = \sqrt{\frac{\omega}{m}P^{-1}(m, q)}.$$

Here $P^{-1}(\cdot)$ denotes the inverse of the lower regularized incomplete gamma function (*RealGammaPInv*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("NakagamiQtf(q, a, b): ", NakagamiQtf(q, a, b))
>>> print ("dist_nakagami(a, b).qtf(q): ", dist_nakagami(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_nakagami**(*m*, *omega*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Nakagami distribution is a continuous probability distribution with shape $m > 0$, scale $\omega > 0$, and the support interval $(0, +\infty)$. See also Wikipedia [1187], MathWorld [254], Witkovský [1488], Dharmawansa P and Ahmed [241], Hauberg [365].

Dharmawansa 2007: Characteristic function Hauberg 2005: Moments

dist_nakagami.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Nakagami distribution:

$$\text{pdf}_X(x) = \frac{2m^m x^{2m-1}}{\omega^m \Gamma(m)} \exp\left(-\frac{m}{\omega}x^2\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", nakagami(mu, sigma).pdf(x))
6.35635234625645256156156145613356E-20
```

dist_nakagami.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Nakagami distribution:

$$\text{cdf}_X(x) = P\left(\frac{m}{\omega}x^2\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", nakagami(mu, sigma).pdf(x))
6.35635234625645256156156145613356E-20
```

dist_nakagami.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an Nakagami distribution:

$$\text{sf}_X(x) = Q\left(\frac{m}{\omega}x^2\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", nakagami(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613356E-20
```

dist_nakagami.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an Nakagami distribution:

$$\text{qtf}_X(q) = \sqrt{\frac{\omega}{m}P^{-1}(m, q)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", nakagami(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456 1356E+00
```

dist_nakagami.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an Nakagami distribution:

$$\text{isf}_X(q) = \sqrt{\frac{\omega}{m} Q^{-1}(m, q)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", nakagami(mu, sigma).isf(q))
6.356352346256452561561561456 1356E+00
```

dist_nakagami.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Nakagami distribution:

$$C_X(t) = \frac{\Gamma(2m)}{2^{m-1}\Gamma(m)} \exp\left(-\frac{\omega}{8m}t^2\right) D_{-2m}\left(-it\sqrt{\frac{\omega}{2m}}\right).$$

Dharmawansa 2007: Characteristic function

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", nakagami(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

dist_nakagami.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Nakagami distribution:

$$M_X(t) = \frac{\Gamma(2m)}{2^{m-1}\Gamma(m)} \exp\left(-\frac{\omega}{8m}t^2\right) D_{-2m}\left(-t\sqrt{\frac{\omega}{2m}}\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", nakagami(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

dist_nakagami.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Nakagami distribution:

$$K_X(t) = \log\left[\frac{\Gamma(2m)}{2^{m-1}\Gamma(m)} \exp\left(-\frac{\omega}{8m}t^2\right) D_{-2m}\left(-t\sqrt{\frac{\omega}{2m}}\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", nakagami(mu, sigma).k_x(t, k))
6.356352346256452561561561456135 6E+00
```

dist_nakagami.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \dots k$, of a random variable X, following an Nakagami distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(n) = tbd.$$

Hauberg 2005: Moments

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", nakagami(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_nakagami.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \dots k$, of a random variable X, following an Nakagami distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", nakagami(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

4.7.8 Amoroso distribution

class ctx.**dist_amoroso**(a, b)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The Amoroso distribution distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, and the support interval $(0, +\infty)$.

See also Crooks [188], Crooks [187].

dist_amoroso.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Amoroso distribution:

$$\mathrm{pdf}_X(x) = \frac{|\beta|}{\Gamma(\alpha)|\theta|}\left(\frac{x-a}{\theta}\right)^{\alpha\beta-1}\exp\left(-\left(\frac{x-a}{\theta}\right)^{\beta}\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_amoroso.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Amoroso distribution:

$$\mathrm{cdf}_X(x) = P(\alpha, z) \quad \text{for } \beta/\theta > 0, \quad \text{where } z = \left(\frac{x-a}{\theta}\right)^{\beta}$$

$$\mathrm{cdf}_X(x) = Q(\alpha, z) \quad \text{for } \beta/\theta < 0, \quad \text{where } z = \left(\frac{x-a}{\theta}\right)^{\beta}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_amoroso.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Amoroso distribution:

$$\mathrm{sf}_X(x) = Q(\alpha, z) \quad \text{for } \beta/\theta < 0 \quad \text{where } z = \left(\frac{x-a}{\theta}\right)^{\beta}$$

$$\mathrm{sf}_X(x) = P(\alpha, z) \quad \text{for } \beta/\theta > 0, \quad \text{where } z = \left(\frac{x-a}{\theta}\right)^{\beta}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_amoroso.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Amoroso distribution:

$$\text{qtf}_X(q) = a + \theta \cdot z^{1/\beta}; z = P^{-1}(\alpha, q) \quad \text{for } \beta/\theta > 0$$

$$\text{qtf}_X(q) = a + \theta \cdot z^{1/\beta}; z = Q^{-1}(\alpha, q) \quad \text{for } \beta/\theta < 0$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_amoroso.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Amoroso distribution:

$$\text{isf}_X(q) = a + \theta \cdot z^{1/\beta}; z = Q^{-1}(\alpha, q) \quad \text{for } \beta/\theta > 0$$

$$\text{isf}_X(q) = a + \theta \cdot z^{1/\beta}; z = P^{-1}(\alpha, q) \quad \text{for } \beta/\theta < 0$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_amoroso.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Amoroso distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_amoroso.**m_x**(t)

Returns `NaN`, since the moment generating function does not exist.

dist_amoroso.**k_x**(t, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_amoroso.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Amoroso distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = ??$$

Standard moments:

$$\mu'_X(r) = \frac{\Gamma(\alpha + r/\beta)}{\Gamma(\alpha)}, \quad \alpha + r/\beta) > 0$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256156156145613356E+00
```

dist_amoroso.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Amoroso distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.35635234625645256156156145613356E+00
```

4.7.9 Distribution of the logarithm of a χ^2 random variable

class ctx.**dist_logrv_chisquared**(n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of the logarithm of a χ^2 random variable with n degrees of freedom is a continuous probability distribution with the support interval $(-\infty, +\infty)$. See also Witkovský [1464].

dist_logrv_chisquared.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the logarithm of a χ^2 random variable:

$$\text{pdf}_X(x) = e^x f_{\chi^2}\left(e^x, n\right),$$

Here $f_{\chi^2}(\cdot, n)$ denotes the pdf of a random variable following an chi-squared distribution with n degress of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", logrv_chisquared(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_logrv_chisquared.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the logarithm of a χ^2 random variable:

$$\text{cdf}_X(x) = F_{\chi^2}\left(e^x, n\right) = P(n/2, e^{x/2}),$$

Here $F_{\chi^2}(\cdot, n)$ denotes the cdf of a random variable following an chi-squared distribution with n degress of freedom, and $P(\cdot)$ is the lower regularized gamma function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", logrv_chisquared(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_logrv_chisquared.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the logarithm of a χ^2 random variable:

$$\text{sf}_X(x) = 1 - F_{\chi^2}\left(e^x, n\right) = Q(n/2, e^{x/2}).$$

Here $1 - F_{\chi^2}(\cdot, n)$ denotes the sf of a random variable following an chi-squared distribution with n degress of freedom, and $Q(\cdot)$ is the upper regularized gamma function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", logrv_chisquared(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_logrv_chisquared.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of the logarithm of a χ^2 random variable:

$$\text{qtf}_X(q) = \log\left(2P^{-1}(n/2, q)\right).$$

Here $P^{-1}(\cdot)$ denotes the inverse of the lower regularized incomplete gamma function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", logrv_chisquared(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_logrv_chisquared.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the logarithm of a χ^2 random variable:

$$\text{qtf}_X(q) = \log\left(2Q^{-1}(n/2, q)\right).$$

Here $Q^{-1}(\cdot)$ denotes the inverse of the upper regularized incomplete gamma function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", logrv_chisquared(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_logrv_chisquared.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the logarithm of a χ^2 random variable:

$$C_X(t) = 2^{it}\frac{\Gamma(n/2 + it)}{\Gamma(n/2)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", logrv_chisquared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_logrv_chisquared.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of the logarithm of a χ^2 random variable:

$$M_X(t) = 2^{t}\frac{\Gamma(n/2 + t)}{\Gamma(n/2)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", logrv_chisquared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_logrv_chisquared.k_x(t, k=0)`

Returns $K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following the distribution of the logarithm of a χ^2 random variable:

$$K_X(t) = t \cdot \log(2) + \log\left(\Gamma(n/2 + t)\right) - \log\left(\Gamma(n/2)\right)$$

$$K_X^{(1)}(t) = \log(2) + \psi^{(0)}(n/2 + t),$$

$$K_X^{(j)}(t) = \psi^{(j-1)}(n/2 + t), \quad j \geq 2.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", logrv_chisquared(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

`dist_logrv_chisquared.moments(k)`

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the logarithm of a χ^2 random variable: the moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", logrv_chisquared(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

`dist_logrv_chisquared.cumulants(k)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of the logarithm of a χ^2 random variable:

$$\kappa_1 = \log(2) + \psi^{(0)}(n/2),$$

$$\kappa_r = \psi^{(r-1)}(n/2), \quad r \geq 2,$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", logrv_chisquared(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Approximations

`ctx.logrv_chisquared_ecf(x, f, results='cdf')`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

`ctx.logrv_chisquared_ecf_inv(q, f, results='qtf')`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

4.7.10 Hypoexponential (Generalized Erlang) Distribution

class ctx.**dist_hypoexponential**(*n*, *lambdaj*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The hypoexponential distribution is a continuous probability distribution with n rate parameters $\lambda_j > 0, j = 1 \dots n$, and the support interval $(0, +\infty)$. It is the distribution of the sum of n independent exponential variables with parameters λ_j. It is called the hypoexponential distribution as it has a coefficient of variation less than one, compared to the hyperexponential distribution which has coefficient of variation greater than one and the exponential distribution which has coefficient of variation of one. If all parameters are equal ($\lambda_j = \lambda$), the hypoexponential distribution is identical with the Erlang distribution (which is a special case of the gamma distribution wherein the shape parameter of the distribution is an integer).

See also: Wikipedia [1212], Wikipedia [1199], MathWorld [250], Chesneau [176], Smaili K. [502].

dist_hypoexponential.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a hypoexponential distribution:

$$\text{pdf}_X(x) = \begin{cases} \dfrac{\lambda^n x^{n-1} e^{-\lambda x}}{(n-1)!} & \text{for all parameters equal: } \lambda_j = \lambda, \\ \displaystyle\sum_{j=1}^{n} \frac{\lambda_j}{d_j} e^{-\lambda_j x} & \text{for all parameters distinct: } \lambda_j \neq \lambda_k \text{ for } j \neq k, \\ -\boldsymbol{\alpha} e^{x\Theta} \Theta \mathbf{1} & \text{for general parameters: } \lambda_j > 0, \end{cases}$$

where $\boldsymbol{\alpha} = (1, 0, \dots, 0)$, $\mathbf{1}$ is a column vector of ones of the size n, e^A is the matrix exponential of A, and

$$d_j = \prod_{k=1, k\neq j}^{n} \left(1 - \frac{\lambda_j}{\lambda_k}\right), \quad \Theta = \begin{bmatrix} -\lambda_1 & \lambda_1 & 0 & \dots & 0 & 0 \\ 0 & -\lambda_2 & \lambda_2 & \ddots & 0 & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \ddots & -\lambda_{n-2} & \lambda_{n-2} & 0 \\ 0 & 0 & \dots & 0 & -\lambda_{n-1} & \lambda_{n-1} \\ 0 & 0 & \dots & 0 & 0 & -\lambda_n \end{bmatrix}. \tag{4.1}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_hypoexponential(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_hypoexponential.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a hypoexponential distribution:

$$\text{cdf}_X(x) = \begin{cases} P(n, \lambda x) & \text{for all parameters equal: } \lambda_j = \lambda, \\ 1 - \displaystyle\sum_{j=1}^{n} \frac{1}{d_j} e^{-\lambda_j x} & \text{for all parameters distinct: } \lambda_j \neq \lambda_k \text{ for } j \neq k, \\ 1 - \boldsymbol{\alpha} e^{x\Theta} \mathbf{1} & \text{for general parameters: } \lambda_j > 0, \end{cases}$$

where $P(\cdot)$ is the lower regularized gamma function, d_j and Θ are defined in equation (4.1), $\boldsymbol{\alpha} = (1, 0, \dots, 0)$, $\mathbf{1}$ is a column vector of ones of the size n, and e^A is the matrix exponential of A.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_hypoexponential(mu, sigma).pdf(x))
6.356352346256452561561561561456 1356E-20
```

`dist_hypoexponential.sf(x)`

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following a hypoexponential distribution:

$$\mathrm{sf}_X(x) = \begin{cases} Q(n, \lambda x) & \text{for all parameters equal: } \lambda_j = \lambda, \\ \sum_{j=1}^{n} \frac{1}{d_j} e^{-\lambda_j x} & \text{for all parameters distinct: } \lambda_j \neq \lambda_k \text{ for } j \neq k, \\ \boldsymbol{\alpha} e^{x\Theta} \mathbf{1} & \text{for general parameters: } \lambda_j > 0, \end{cases}$$

where $Q(\cdot)$ is the upper regularized gamma function, d_j and Θ are defined in equation (4.1), $\boldsymbol{\alpha} = (1, 0, \ldots, 0)$, $\mathbf{1}$ is a column vector of ones of the size n, and e^A is the matrix exponential of A.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_hypoexponential(mu, sigma).pdf(x))
sf: 6.356352346256452561561561456 1356E-20
```

`dist_hypoexponential.qtf(q)`

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a hypoexponential distribution.

There is no known closed form for $\mathrm{qtf}_X(q)$ or $\mathrm{isf}_X(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_hypoexponential(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561456 1356E+00
```

`dist_hypoexponential.isf(q)`

Returns $\mathrm{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a hypoexponential distribution.

There is no known closed form for $\mathrm{qtf}_X(q)$ or $\mathrm{isf}_X(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_hypoexponential(mu, sigma).isf(q))
6.356352346256452561561561456 1356E+00
```

`dist_hypoexponential.c_x(t)`

Returns $C_X(t)$, the characteristic function of a random variable X, following a hypoexponential distribution:

$$C_X(t) = \prod_{j=1}^{n} \frac{\lambda_j}{\lambda_j - it}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_hypoexponential(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_hypoexponential.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a hypoexponential distribution:

$$M_X(t) = \prod_{j=1}^{n} \frac{\lambda_j}{\lambda_j - t}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_hypoexponential(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_hypoexponential.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a hypoexponential distribution:

$$K_X(t) = \sum_{j=1}^{n} \log\left(\frac{\lambda_j}{\lambda_j - t}\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", mp_hypoexponential(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_hypoexponential.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a hypoexponential distribution. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_hypoexponential(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_hypoexponential.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a hypoexponential distribution. The cumulants are given by

$$\kappa_X(r) = (r-1)! \sum_{j=1}^{n} \frac{1}{\lambda_j^r}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_hypoexponential(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.7.11 Lindley distribution (generalized)

class ctx.**dist_lindley**(b)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Lindley distribution is a continuous probability distribution with scale $b > 0$, and the support interval $(0, +\infty)$.

See also: Al-Babtain *et al.* [7], Zakerzadeh and Dolati [1499], Lindley [409].

dist_lindley.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Lindley distribution:

$$\mathrm{pdf}_X(x) = \frac{\theta^2}{\eta + \theta k}\left(\frac{k(\theta x)^{\alpha-1}}{\Gamma(\alpha)} + \frac{\eta(\theta x)^{\beta-1}}{\theta\Gamma(\beta)}\right)e^{-\theta x}, \quad x > 0.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", lindley(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_lindley.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Lindley distribution:

$$\mathrm{cdf}_X(x) = \frac{1}{\eta + \theta k}\left(\theta k P(\alpha, \theta x) + \eta P(\beta, \theta x)\right), \quad x > 0.$$

$$\mathrm{cdf}_X(x) = w \cdot P(\alpha, \theta x) + (1 - w) \cdot P(\beta, \theta x), \quad w = \frac{\theta k}{\eta + \theta k}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", lindley(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_lindley.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following an Lindley distribution:

$$\mathrm{sf}_X(x) = 1 - \frac{1}{\eta + \theta k}\left(\theta k P(\alpha, \theta x) + \eta P(\beta, \theta x)\right), \quad x > 0.$$

$$\mathrm{sf}_X(x) = w \cdot Q(\alpha, \theta x) + (1 - w) \cdot Q(\beta, \theta x), \quad w = \frac{\theta k}{\eta + \theta k}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", lindley(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_lindley.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an Lindley distribution:

$$\text{qtf}_X(q) = \frac{1}{\theta}\left(w \cdot P^{-1}(\alpha, q) + (1-w) \cdot P^{-1}(\beta, q)\right), \quad w = \frac{\theta k}{\eta + \theta k}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", lindley(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_lindley.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an Lindley distribution:

$$\text{isf}_X(q) = \frac{1}{\theta}\left(w \cdot Q^{-1}(\alpha, q) + (1-w) \cdot Q^{-1}(\beta, q)\right), \quad w = \frac{\theta k}{\eta + \theta k}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", lindley(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_lindley.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Lindley distribution:

$$C_X(t) = \frac{1}{\eta + \theta k}\left[\theta k \left(1 - \frac{it}{\theta}\right)^{-\alpha} + \eta \left(1 - \frac{it}{\theta}\right)^{-\beta}\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", lindley(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_lindley.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Lindley distribution:

$$M_X(t) = \frac{1}{\eta + \theta k}\left[\theta k \left(1 - \frac{t}{\theta}\right)^{-\alpha} + \eta \left(1 - \frac{t}{\theta}\right)^{-\beta}\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", lindley(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_lindley.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Lindley distribution:

$$K_X(t) = \log\left(M(t)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", lindley(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_lindley.**moments**(k)

Returns the first j raw moments, $\mu'_j, j = 1 \ldots k$, of a random variable X, following an Lindley distribution (Wikipedia). The raw moments are given by

$$\mu'_X(k) = \frac{\alpha(\alpha+1)\cdots(\alpha+r-1)\theta k + \beta(\beta+1)\cdots(\beta+r-1)\eta}{\theta^r(\eta+\theta k)}.$$

$$\mu'_X(k) = \frac{w}{\theta^r}\frac{\Gamma(\alpha+r)}{\Gamma(\alpha)} + \frac{1-w}{\theta^r}\frac{\Gamma(\beta+r)}{\Gamma(\beta)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", lindley(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_lindley.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Lindley distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", lindley(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.7.12 Skew exponential power distribution

class ctx.**skewexponpower**(a, b)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The skew exponential power distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, and the support interval $(0, +\infty)$.

See also: Hutson [377], Kleiber and Kotz [393] (p. 131), Wikipedia [1211], MathWorld [247].

skewexponpower.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a skew exponential power distribution:

$$\mathrm{pdf}_X(x) = \frac{1}{k\sigma} \exp\left(-\tfrac{1}{2}\left(|z| + (2\alpha - 1)z\right)\right)^{1/c}, \quad \text{where } c = \frac{1+\beta}{2}, \quad k = \frac{4\alpha(1-\alpha)}{\Gamma(1+c)2^{1+c}}, \quad z = \frac{x-\theta}{\sigma}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

skewexponpower.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a skew exponential power distribution:

$$\mathrm{cdf}_X(x) = \begin{cases} \alpha P\left(c, 2^{(1/c)-1}((\alpha-1)z)^{1/c}\right) & \text{if } z \le 0 \\ 1 - (1-\alpha)P\left(c, 2^{(1/c)-1}(\alpha z)^{1/c}\right) & \text{if } z > 0 \end{cases}, \quad \text{where } c = \frac{1+\beta}{2}, \quad z = \frac{x-\theta}{\sigma}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

skewexponpower.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a skew exponential power distribution:

$$\mathrm{sf}_X(x) = \begin{cases} 1 - \alpha P\left(c, 2^{(1/c)-1}((\alpha-1)z)^{1/c}\right) & \text{if } z \le 0 \\ (1-\alpha)P\left(c, 2^{(1/c)-1}(\alpha z)^{1/c}\right) & \text{if } z > 0 \end{cases}, \quad \text{where } c = \frac{1+\beta}{2}, \quad z = \frac{x-\theta}{\sigma}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

skewexponpower.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a skew exponential power distribution:

$$\text{qtf}_X(q) = \begin{cases} \theta + \frac{\sigma}{1-\alpha}\left(2^{((1/c)-1)/2}\sqrt{P^{-1}\left(c, q/\alpha\right)}\right)^{\beta+1} & \text{if } q \leq \alpha \\ \theta + \frac{\sigma}{\alpha}\left(2^{((1/c)-1)/2}\sqrt{P^{-1}\left(c, (1-q)/(1-\alpha)\right)}\right)^{\beta+1} & \text{if } q > \alpha \end{cases}, \quad \text{where } c = \frac{1+\beta}{2}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

skewexponpower.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a skew exponential power distribution:

$$\text{isf}_X(q) = \begin{cases} \theta + \frac{\sigma}{1-\alpha}\left(2^{((1/c)-1)/2}\sqrt{P^{-1}\left(c, q/\alpha\right)}\right)^{\beta+1} & \text{if } q \leq \alpha \\ \theta + \frac{\sigma}{\alpha}\left(2^{((1/c)-1)/2}\sqrt{P^{-1}\left(c, (1-q)/(1-\alpha)\right)}\right)^{\beta+1} & \text{if } q > \alpha \end{cases}, \quad \text{where } c = \frac{1+\beta}{2}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

skewexponpower.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a skew exponential power distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

skewexponpower.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a skew exponential power distribution:

$$M_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

skewexponpower.**k_x**(t, k=*0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a skew exponential power distribution:

$$K_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

skewexponpower.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a skew exponential power distribution. All moments exist. The moments of $Z = (X - \theta)/\sigma$ are given by

$$\mu'_Z(r) = \frac{(\beta+1) \cdot \left((-2)^r \alpha^{r+1} + (1-\alpha)(2(1-\alpha))^r\right) \cdot 2^{(\beta-3)r/2-1} \cdot \Gamma((\beta+1)(r+1)/2)}{(1-\alpha)^r \alpha^r \Gamma((\beta+3)/2)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.3563523462564525615615615614561356E+00
```

skewexponpower.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a skew exponential power distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

4.7.13 Stacy (generalized gamma) distribution

class ctx.**dist_stacy**(a, b)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The Stacy distribution distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, and the support interval $(0, +\infty)$.

See also Wikipedia [1209], Crooks [187] (p.87).

dist_stacy.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Stacy distribution:

$$\mathrm{pdf}_X(x) = \frac{|\beta|}{\Gamma(\alpha)|\theta|}\left(\frac{x}{\theta}\right)^{\alpha\beta-1}\exp\left(-\left(\frac{x}{\theta}\right)^{\beta}\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613356E-20
```

dist_stacy.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Stacy distribution:

$$\mathrm{cdf}_X(x) = P(\alpha, z) \quad \text{for } \beta/\theta > 0, \quad \text{where } z = \left(\frac{x}{\theta}\right)^{\beta}$$

$$\mathrm{cdf}_X(x) = Q(\alpha, z) \quad \text{for } \beta/\theta < 0, \quad \text{where } z = \left(\frac{x}{\theta}\right)^{\beta}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613356E-20
```

dist_stacy.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Stacy distribution:

$$\mathrm{sf}_X(x) = Q(\alpha, z) \quad \text{for } \beta/\theta < 0 \quad \text{where } z = \left(\frac{x}{\theta}\right)^{\beta}$$

$$\mathrm{sf}_X(x) = P(\alpha, z) \quad \text{for } \beta/\theta > 0, \quad \text{where } z = \left(\frac{x}{\theta}\right)^{\beta}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613356E-20
```

dist_stacy.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Stacy distribution:

$$\text{qtf}_X(q) = \theta \cdot z^{1/\beta}; z = P^{-1}(\alpha, q) \quad \text{for } \beta/\theta > 0$$

$$\text{qtf}_X(q) = \theta \cdot z^{1/\beta}; z = Q^{-1}(\alpha, q) \quad \text{for } \beta/\theta < 0$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_stacy.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Stacy distribution:

$$\text{isf}_X(q) = \theta \cdot z^{1/\beta}; z = Q^{-1}(\alpha, q) \quad \text{for } \beta/\theta > 0$$

$$\text{isf}_X(q) = \theta \cdot z^{1/\beta}; z = P^{-1}(\alpha, q) \quad \text{for } \beta/\theta < 0$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_stacy.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Stacy distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_stacy.**m_x**(t)

Returns `NaN`, since the moment generating function does not exist.

dist_stacy.**k_x**(t, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_stacy.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Stacy distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = ??$$

Standard moments:

$$\mu'_X(r) = \frac{\Gamma(\alpha + r/\beta)}{\Gamma(\alpha)}, \quad \alpha + r/\beta) > 0$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256156156145613356E+00
```

`dist_stacy.`**`cumulants`**`(`k`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Stacy distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.35635234625645256156156145613356E+00
```

4.8 Closed form distributions, based on the incomplete beta function

4.8.1 Boost: Beta (Pearson Type I and II) distribution

Returns the pdf, cdf, qtf or boost class of a random variable X, following a beta distribution with parameters $a > 0$, $b > 0$, and the support interval $(0, 1)$.

See also Wikipedia [1146], MathWorld [795], BoostMath [51], Ehrhardt [297] (3.9.2).

math53.**beta_pdf**(a, b, x)

Returns $\text{pdf}(x)$, the value of the probability density function (*Pdf*) of the beta distribution:

$$\text{pdf}(x) = f_{\text{Beta}}(a, b, x) = \frac{1}{B(a, b)} x^{a-1}(1 - x)^{b-1}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("BetaPdf(x, a, b): ", BetaPdf(x, a, b))
>>> print ("dist_beta(a, b).pdf(x): ", dist_beta(a, b).pdf(x))
6.35635234625645256156156145561356E+00
```

math53.**beta_cdf**(a, b, x)

Returns $\text{cdf}(x)$, the value of the cumulative distribution function (*Cdf*) of the beta distribution:

$$\text{cdf}(x) = I_x(a, b) = \text{ibeta}(a, b, x).$$

Here $\text{ibeta}(\cdot)$ denotes the real normalised incomplete beta function (*RealIBeta*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("BetaCdf(x, a, b): ", BetaCdf(x, a, b))
>>> print ("dist_beta(a, b).cdf(x): ", dist_beta(a, b).cdf(x))
6.35635234625645256156156145561356E+00
```

math53.**beta_qtf**(a, b, q)

Returns $\text{qtf}(q)$, the value of the quantile function (*Qtf*) of the beta distribution:

$$\text{qtf}(q) = \text{ibeta_inv}(a, b, q).$$

Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function (*RealIBetaInv*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("BetaQtf(q, a, b): ", BetaQtf(q, a, b))
>>> print ("dist_beta(a, b).qtf(q): ", dist_beta(a, b).qtf(q))
6.35635234625645256156156145561356E+00
```

class ctx.**dist_beta**(a, b)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The beta distribution is a continuous probability distribution with parameters $a > 0$, $b > 0$, and the support interval $(0, 1)$. See also Wikipedia [1146], MathWorld [795], BoostMath [51], Witkovský [1463], R (Statistical System) [519].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.betainc.html#scipy.special.betainc

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.betaincinv.html#scipy.special.betaincinv

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.btdtria.html#scipy.special.btdtria

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.btdtrib.html#scipy.special.btdtrib

dist_beta.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a beta distribution:

$$\mathrm{pdf}_X(x) = f_{\mathrm{Beta}}(a, b, x) = \frac{1}{B(a, b)} x^{a-1}(1-x)^{b-1}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", beta(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_beta.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a beta distribution:

$$\mathrm{cdf}_X(x) = I_x(a, b) = \mathrm{ibeta}(a, b, x).$$

Here $\mathrm{ibeta}(\cdot)$ denotes the real normalised incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", beta(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_beta.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a beta distribution:

$$\mathrm{sf}_X(x) = 1 - I_x(a, b) = \mathrm{ibetac}(a, b, x).$$

Here $\mathrm{ibetac}(\cdot)$ denotes the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", beta(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_beta.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a beta distribution:

$$\text{qtf}_X(q) = \text{ibeta_inv}(a, b, q).$$

Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", beta(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_beta.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a beta distribution:

$$\text{isf}_X(q) = \text{ibetac_inv}(a, b, q).$$

Here $\text{ibetac_inv}(\cdot)$ denotes the inverse of the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", beta(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_beta.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a beta distribution:

$$C_X(t) = {}_1F_1(a, a + b; it).$$

where ${}_1F_1()$ is Kummer's confluent hypergeometric function (of the first kind).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", beta(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_beta.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a beta distribution:

$$M_X(t) = {}_1F_1(a, a + b; t).$$

where ${}_1F_1()$ is Kummer's confluent hypergeometric function (of the first kind).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", beta(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_beta.k_x(t, k=0)`

Returns $K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a beta distribution:

$$K_X(t) = \log({}_1F_1(a, a+b; t)),$$

$$K_X^{(j)}(t) = tbd.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", beta(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

`dist_beta.moments(k)`

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a beta distribution: the moments are calculated from the cumulants.

$$\mu'_h = \frac{\Gamma(a+h)\Gamma(a+b)}{\Gamma(a)\Gamma(a+b+h)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", beta(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

`dist_beta.cumulants(k)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a beta distribution: The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", beta(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Recurrences: Central Beta

`ctx.beta_recurrence(x, n, lambda)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The following recurrence relations hold for the pdf and CDF:

$$I(a,b;x) = 1 - I(b,a;1-x) \tag{4.2}$$

$$I(a,b;x) = \binom{n}{a} x^a (1-x)^{b-1} + I(a+1, b-1; x) \tag{4.3}$$

$$I(a,b;x) = \binom{n}{a} x^a (1-x)^{b} + I(a+1, b; x) \tag{4.4}$$

$$I(a,b+1;x) = \binom{n}{a} x^a (1-x)^{b} + I(a, b; x) \tag{4.5}$$

$$I(a,b;x) = \binom{n}{a+b} x^a (1-x)^b \frac{a}{a+b-x} + I(a+1, b+1; x) \tag{4.6}$$

$$I(a,b;x) = F\left(2a, 2b, \frac{nx}{m-mx}\right) \tag{4.7}$$

4.8.2 Boost: Central Fisher F distribution

The following functions return the pdf, cdf, qtf or boost class of the Fisher F distribution with $m > 0$ and $n > 0$ degrees of freedom, and the support interval $(0, +\infty)$.

See also Wikipedia [1151], MathWorld [799], BoostMath [56], Ehrhardt [297] (3.9.9).

`math53.fisher_f_pdf(m, n, x)`

Returns pdf(x), the value of the probability density function (*Pdf*) of the Fisher F distribution:

$$\text{pdf}(x) = \frac{m^{m/2} n^{n/2}}{B(m/2, n/2)} x^{(m-2)/2} (n + mx)^{-(m+n)/2}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("FisherFPdf(x, a, b): ", FisherFPdf(x, a, b))
>>> print ("dist_fisher_f(a, b).pdf(x): ", dist_fisher_f(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.fisher_f_cdf(m, n, x)`

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Fisher F distribution:

$$\text{cdf}(x) = \begin{cases} \text{ibetac}(n/2, m/2, n/(n + mx)), & mx > n, \\ \text{ibeta}(m/2, n/2, mx/(n + mx)) & mx \le n. \end{cases}$$

Here ibeta$(\cdot)$ denotes the real normalised incomplete beta function (*RealIBeta*), and ibetac$(\cdot)$ denotes the real normalised complementary incomplete beta function (*RealIBetac*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("FisherFCdf(x, a, b): ", FisherFCdf(x, a, b))
>>> print ("dist_fisher_f(a, b).cdf(x): ", dist_fisher_f(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.fisher_f_qtf(m, n, q)`

Returns qtf(q), the value of the quantile function (*Qtf*) of the Fisher F distribution:

$$\text{qtf}(q) = \frac{nx}{m(1 - x)}, \quad \text{where } x = \text{ibeta_inv}(m/2, n/2, q).$$

Here ibeta_inv$(\cdot)$ denotes the inverse of the real normalised incomplete beta function (*RealIBetaInv*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("FisherFQtf(q, a, b): ", FisherFQtf(q, a, b))
>>> print ("dist_fisher_f(a, b).qtf(q): ", dist_fisher_f(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_fisher_f**(m, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Fisher F-distribution is a continuous probability distribution with $m > 0$ and $n > 0$ degrees of freedom, and the support interval $(0, +\infty)$. See also Wikipedia [1151], MathWorld [799], BoostMath [56], Witkovský [1466], R (Statistical System) [523], Abramowitz and Stegun. [3], Butler and Paolella [166], Chattamvelli and Jones [175], Witkovský [1460].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.fdtr.html#scipy.special.fdtr

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.fdtrc.html#scipy.special.fdtrc

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.fdtri.html#scipy.special.fdtri

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.fdtridfd.html#scipy.special.fdtridfd

dist_fisher_f.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a central Fisher F distribution:

$$\mathrm{pdf}_X(x) = \frac{m^{m/2} n^{n/2}}{B(m/2, n/2)} x^{(m-2)/2} (n + mx)^{-(m+n)/2},$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_fisher_f.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a central Fisher F distribution:

$$\mathrm{cdf}_X(x) = \begin{cases} \mathrm{ibetac}(n/2, m/2, n/(n+mx)), & mx > n, \\ \mathrm{ibeta}(m/2, n/2, mx/(n+mx)) & mx \le n. \end{cases}$$

Here $\mathrm{ibeta}(\cdot)$ denotes the real normalised incomplete beta function, and $\mathrm{ibetac}(\cdot)$ denotes the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_fisher_f.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a central Fisher F distribution:

$$\mathrm{sf}_X(x) = \begin{cases} \mathrm{ibeta}(n/2, m/2, n/(n+mx)), & mx > n, \\ \mathrm{ibetac}(m/2, n/2, mx/(n+mx)) & mx \le n. \end{cases}$$

Here $\mathrm{ibeta}(\cdot)$ denotes the real normalised incomplete beta function, and $\mathrm{ibetac}(\cdot)$ denotes the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_fisher_f.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a central Fisher F distribution:

$$\text{qtf}_X(q) = \frac{nx}{m(1-x)}, \quad \text{where } x = \text{ibeta_inv}(m/2, n/2, q).$$

Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145614561356E+00
```

dist_fisher_f.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a central Fisher F distribution:

$$\text{isf}_X(q) = \frac{nx}{m(1-x)}, \quad \text{where } x = \text{ibetac_inv}(m/2, n/2, q).$$

Here $\text{ibetac_inv}(\cdot)$ denotes the inverse of the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145614561356E+00
```

dist_fisher_f.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a central Fisher F distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x = \frac{\Gamma(m/2+n/2)}{\Gamma(n/2)} U\left(\frac{m}{2}, 1-\frac{n}{2}, -\frac{n}{m}it\right)$$

where $U(\cdot)$ denotes the confluent hypergeometric function of the second kind.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.35635234625645256156156145614561356E+00
```

dist_fisher_f.**m_x**(t)

Returns `NaN`, since the moment generating function does not exist.

dist_fisher_f.**k_x**(t, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_fisher_f.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a central Fisher F distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = \mu'_F(r) = \frac{\Gamma(\frac{1}{2}n_1 + r) - \Gamma(\frac{1}{2}n_2 - r)}{\Gamma(\frac{1}{2}n_2)}, \quad \text{for } n_2 > 2r.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_fisher_f.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a central Fisher F distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

Additional methods: Recurrence relations

Let the density $g_{m,n}$ be that of m/n times an $F_{m,n}$ random variable. Let $G_{m,n}(y)$ be its distribution function.

Then the following recurrence relations hold (see Chattamvelli, 1995)

dist_fisher_f.**recurrence_pdf**(*x*, *lambda*, *start_n1*, *start_n2*, *target_n1*, *target_n2*)

Applies a recurrence relation to calculate the cdf for different degrees of freedom for a given value of x. This is mostly useful when using asymptotic methods.

$$n\left[G_{m,n+2}(y) - G_{m-2,n+2}(y)\right] = -2g_{m,n}(y)$$

$$m(1+y)]g_{m+2,n}(y) + y(m+n)g_{m,n}(y).$$

$$n(1+y)g_{m,n+2}(y) = (m+n)g_{m,n}(y).$$

$$mg_{m+2,n-2}(y) = (n-2)yg_{m,n}(y).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", fisher_f(mu, sigma).k_x(t, k))
6.356352346256452561561561456135 6E+00
```

dist_fisher_f.**recurrence_cdf**(*x*, *lambda*, *start_n1*, *start_n2*, *target_n1*, *target_n2*)

Applies a recurrence relation to calculate the cdf for different degrees of freedom for a given value of x. This is mostly useful when using asymptotic methods.

From these equations we obtain

$$\begin{aligned}[(m+2)(1+y)]G_{m+4,n}(y) &= [(m+2)(1+y) + y(m+n]G_{m+2,n}(y) \\ &- y(m+n)G_{m,n}(y)\end{aligned} \tag{4.8}$$

$$n(1+y) : cite : t : `G_{m,n+2}(y) - G_{m+2,n+2}(y)] = (m+n) : cite : t : `G_{m,n}(y) - G_{m+2,n}(y)]$$

$$(m+2) : cite : t : `G_{m+2,n}(y) - G_{m+4,n-2}(y)] = (n-2) : cite : t : `G_{m,n}(y) - G_{m+2,n}(y)]$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", fisher_f(mu, sigma).k_x(t, k))
6.356352346256452561561561561456l356E+00
```

4.8.3 Boost: Student t (Pearson Type VII) distribution

The following functions return the pdf, cdf, qtf or boost class of the Student t distribution with $n > 0$ degrees of freedom and the support interval $(-\infty, +\infty)$.

See also Wikipedia [1171], MathWorld [813], BoostMath [69], Ehrhardt [297] (3.9.29).

`math53.student_t_pdf`(n, x)

Returns $\text{pdf}(x)$, the value of the probability density function (*Pdf*) of the Student t distribution:

$$\text{pdf}(x) = \frac{\Gamma((n+1)/2)}{\sqrt{n\pi}\Gamma(n/2)} \left(\frac{n}{n+t^2}\right)^{(n+1)/2}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("StudentTPdf(x, a, b): ", StudentTPdf(x, a, b))
>>> print ("dist_student_t(a, b).pdf(x): ", dist_student_t(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.student_t_cdf`(n, x)

Returns $\text{cdf}(x)$, the value of the cumulative distribution function (*Cdf*) of the Student t distribution:

$$\text{cdf}(x) = \begin{cases} 1-p, & x > 0, \\ p & x \le 0. \end{cases}, \quad \text{where } p = \begin{cases} \text{ibeta}(n/2, 1/2, n/(n+x^2))/2, & n < 2x^2, \\ \text{ibetac}(1/2, n/2, x^2/(n+x^2))/2, & n \ge 2x^2. \end{cases}$$

Here $\text{ibeta}(\cdot)$ denotes the real normalised incomplete beta function (*RealIBeta*), and $\text{ibetac}(\cdot)$ denotes the real normalised complementary incomplete beta function (*RealIBetac*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("StudentTCdf(x, a, b): ", StudentTCdf(x, a, b))
>>> print ("dist_student_t(a, b).cdf(x): ", dist_student_t(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.student_t_qtf`(n, q)

where `Ctx` is `Math53` or `CtxBoost`.

Returns $\text{qtf}(q)$, the value of the quantile function (*Qtf*) of the Student t distribution:

$$\text{qtf}(q) = t_{\nu,\alpha} = \text{sign}(q-0.5)\sqrt{n(1-x)/x}, \quad \text{where } x = \text{ibeta_inv}\left(\tfrac{1}{2}n, \tfrac{1}{2}, 2\cdot\min(q, 1-q)\right).$$

Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function (*RealIBetaInv*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("StudentTQtf(q, a, b): ", StudentTQtf(q, a, b))
>>> print ("dist_student_t(a, b).qtf(q): ", dist_student_t(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_student_t**(n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Student t distribution is a continuous probability distribution with $n > 0$ degrees of freedom and the support interval $(-\infty, +\infty)$. See also Wikipedia [1171], MathWorld [813], BoostMath [69], Broda and Paolella [160], Witkovský [1460], Witkovský [1479], R (Statistical System) [527].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.t.html#scipy.stats.t

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.stdtr.html#scipy.special.stdtr

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.stdtridf.html#scipy.special.stdtridf

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.stdtrit.html#scipy.special.stdtrit

dist_student_t.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a central Student t distribution:

$$\mathrm{pdf}_X(x) = \frac{\Gamma((n+1)/2)}{\sqrt{n\pi}\Gamma(n/2)} \left(\frac{n}{n+t^2}\right)^{(n+1)/2}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", student_t(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_student_t.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a central Student t distribution:

$$\mathrm{cdf}_X(x) = \begin{cases} 1-p, & x > 0, \\ p & x \le 0. \end{cases}, \quad \text{where } p = \begin{cases} \mathrm{ibeta}(n/2, 1/2, n/(n+x^2))/2, & n < 2x^2, \\ \mathrm{ibetac}(1/2, n/2, x^2/(n+x^2))/2, & n \ge 2x^2. \end{cases}$$

Here $\mathrm{ibeta}(\cdot)$ denotes the real normalised incomplete beta function, and $\mathrm{ibetac}(\cdot)$ denotes the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", student_t(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_student_t.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a central Student t distribution:

$$\mathrm{sf}_X(x) = \mathrm{cdf}_X(-x)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", student_t(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_student_t.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a central Student t distribution:

$$\text{qtf}_X(q) = \text{sign}(q-0.5)\sqrt{n(1-x)/x}, \quad \text{where } x = \text{ibeta_inv}\left(\tfrac{1}{2}n, \tfrac{1}{2}, 2\cdot\min(q, 1-q)\right).$$

Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", student_t(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_student_t.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a central Student t distribution:

$$\text{isf}_X(q) = -\text{qtf}_X(q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", student_t(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_student_t.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a central Student t distribution:

$$C_X(t) = \frac{K_{n/2}(\sqrt{n}|t|)^{n/2}}{\Gamma(n/2)2^{n/2-1}},$$

where $K_n(\cdot)$ denotes the modified Bessel function of the second kind.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", student_t(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_student_t.**m_x**(t)

Returns `NaN`, since the moment generating function does not exist.

dist_student_t.**k_x**(t, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_student_t.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a central Student t distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = \mu'_t(r) = \left(\tfrac{1}{2}n\right)^{r/2} \frac{\Gamma\left(\frac{1}{2}(n-r)\right)}{\Gamma\left(\frac{1}{2}n\right)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", student_t(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_student_t.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a central Student t distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", student_t(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.8.4 Distribution of the negative logarithm of a beta variable

class ctx.**dist_logrv_beta**(a, b)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of the negative logarithm of a beta variable with parameters $a > 0$ and $b > 0$ is a continuous probability distribution with the support interval $(0, +\infty)$. See also Wikipedia [1146], MathWorld [795], BoostMath [51], Witkovský [1463], R (Statistical System) [519].

dist_logrv_beta.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the negative logarithm of a beta variable:

$$\mathrm{pdf}_X(x) = e^{-x} f_{\mathrm{Beta}}(e^{-x}; a, b),$$

where $f_{\mathrm{Beta}}(\cdot, a, b)$ is the pdf of the Beta distribution with parameters a and b.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_logrv_beta.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the negative logarithm of a beta variable:

$$\mathrm{cdf}_X(x) = 1 - F_{\mathrm{Beta}}(e^{-x}; a, b),$$

where $F_{\mathrm{Beta}}(\cdot, a, b)$ is the cdf of the Beta distribution with parameters a and b.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_logrv_beta.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the negative logarithm of a beta variable:

$$\mathrm{sf}_X(x) = F_{\mathrm{Beta}}(e^{-x}; a, b),$$

where $F_{\mathrm{Beta}}(\cdot, a, b)$ is the cdf of the Beta distribution with parameters a and b.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_logrv_beta.**qtf**(q)

Returns $\text{qtf}_X(q)$, the quantile function (qtf) of a random variable X, following the distribution of the negative logarithm of a beta variable:

$$\text{qtf}_X(q) = -\log\left(F^{-1}_{\text{Beta}}(1-q;a,b)\right),$$

where $F^{-1}_{\text{Beta}}(\cdot,a,b)$ is the qtf of the Beta distribution with parameters a and b.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_logrv_beta.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the negative logarithm of a beta variable:

$$\text{isf}_X(q) = -\log\left(F^{-1}_{\text{Beta}}(q;a,b)\right),$$

where $F^{-1}_{\text{Beta}}(\cdot,a,b)$ is the qtf of the Beta distribution with parameters a and b.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_logrv_beta.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the negative logarithm of a beta variable:

$$C_X(t) = \frac{\Gamma\left((a-it)\right)\Gamma\left((a+b)\right)}{\Gamma\left(a\right)\Gamma\left(a+b-it\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_logrv_beta.**m_x**(t)

Returns the moment generating function of a random variable X, following the distribution of the negative logarithm of a beta variable.

$$M_X(t) = \frac{\Gamma\left((a-t)\right)\Gamma\left((a+b)\right)}{\Gamma\left(a\right)\Gamma\left(a+b-t\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_logrv_beta.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of a beta variable.

$$K_X(t) = \log\left(\Gamma(a-t)\right) - \log\left(\Gamma(a+b-t)\right) + \log\left(\Gamma(a+b)\right) - \log\left(\Gamma(a)\right).$$

$$K_X^{(j)}(t) = (-1)^j \left(\psi^{(j-1)}(a-t) - \psi^{(j-1)}(a+b-t)\right)$$

where $\psi^{(j)}(\cdot)$ is the polygamma function of order j.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_logrv_beta.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of a beta variable. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_logrv_beta.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of a beta variable.

$$\kappa_j = (-1)^j \left(\psi^{(j-1)}(a) - \psi^{(j-1)}(a+b)\right)$$

where $\psi^{(j)}(\cdot)$ is the polygamma function of order j.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

Approximations

ctx.**logrv_beta_ecf_pdf**(*x*, *a*, *b*, *results='cdf'*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

ctx.**logrv_beta_ecf_qtf**(*q*, *a*, *b*, *results='cdf'*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

4.8.5 Beta-prime (Pearson Type VI) distribution

class ctx.**dist_beta_prime**(a, b)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The beta-prime distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, and the support interval $(0, +\infty)$.

See also Wikipedia [1196], Becker [27].

Pearson Type I: Beta

Pearson Type II: Symmetric Beta

Pearson Type III: Gamma

Pearson Type IV: Extra

Pearson Type V: Inverse Gamma

Pearson Type VI: Beta Prime

Pearson Type VII: Student's t

dist_beta_prime.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a beta-prime distribution:

$$\mathrm{pdf}_X(x) = f(x) = \frac{x^{\alpha-1}(1+x)^{-\alpha-\beta}}{B(\alpha,\beta)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_beta_prime.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a beta-prime distribution:

$$\mathrm{cdf}_X(x) = I_{\frac{x}{1+x}}(a,b)$$

Here ibeta$(\cdot)$ denotes the real normalised incomplete beta function, and ibetac$(\cdot)$ denotes the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_beta_prime.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a beta-prime distribution:

$$\mathrm{sf}_X(x) = 1 - I_{\frac{x}{1+x}}(a,b)$$

Here ibeta$(\cdot)$ denotes the real normalised incomplete beta function, and ibetac$(\cdot)$ denotes the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

`dist_beta_prime.qtf(`q`)`

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a beta-prime distribution:

$$\text{qtf}_X(q) = ??, \quad \text{where } x = \text{ibeta_inv}(m/2, n/2, q).$$

Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

`dist_beta_prime.isf(`q`)`

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a beta-prime distribution:

$$\text{isf}_X(q) = ??, \quad \text{where } x = \text{ibetac_inv}(m/2, n/2, q).$$

Here $\text{ibetac_inv}(\cdot)$ denotes the inverse of the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

`dist_beta_prime.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following a beta-prime distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

`dist_beta_prime.m_x(`t`)`

Returns `NaN`, since the moment generating function does not exist.

`dist_beta_prime.k_x(`t`, `*k=0*`)`

Returns `NaN`, since the cumulant generating function does not exist.

dist_beta_prime.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \dots k$, of a random variable X, following a beta-prime distribution. For $-\alpha < k < \beta$, the k-th moment is given by

$$\mu'_X(r) = \frac{B(\alpha + k, \beta - k)}{B(\alpha, \beta)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

dist_beta_prime.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \dots k$, of a random variable X, following a beta-prime distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

4.8.6 Generalized Beta (Type 1) distribution

class ctx.**dist_genbeta1**(a, b, p, q)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The Generalized Beta (Type 1) distribution distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, $p > 0$, $q > 0$, and the support interval $(0, +\infty)$.

See also: Kleiber and Kotz [393] (page 230), Wikipedia [1208].

dist_genbeta1.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Generalized Beta (Type 1) distribution:

$$\mathrm{pdf}_X(x) = \frac{a x^{ap-1}[1-(x/b)^a]^{q-1}}{b^{ap}B(p,q)}, \quad 0 \le x \le b.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_genbeta1.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Generalized Beta (Type 1) distribution:

$$\mathrm{cdf}_X(x) = I_z(p,q), \quad \text{where } z = \left(\frac{y}{b}\right)^a, y = \left(\frac{x^a}{1+x^a}\right)^{1/a} \quad x > 0, \quad 0 \le x \le b.$$

See also: Kleiber, page 231

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_genbeta1.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Generalized Beta (Type 1) distribution:

$$\mathrm{sf}_X(x) = 1 - I_z(p,q) = I_{1-z}(p,q), \quad \text{where } z = \left(\frac{y}{b}\right)^a, y = \left(\frac{x^a}{1+x^a}\right)^{1/a} \quad x > 0, \quad 0 \le x \le b.$$

Here ibeta($\cdot$) denotes the real normalised incomplete beta function, and ibetac($\cdot$) denotes the real normalised complementary incomplete beta function. See Kleiber, page 188.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_genbeta1.**qtf**(*q*)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Generalized Beta (Type 1) distribution:

$$\text{qtf}_X(\text{prob}) = b \cdot z^{1/a}, \quad \text{where } z = \text{ibeta_inv}(p, q, \text{prob}).$$

Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function.

MISSING: transformatio from x to y.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_genbeta1.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Generalized Beta (Type 1) distribution:

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Generalized Beta (Type 2) distribution:

$$\text{isf}_X(\text{prob}) = b \cdot z^{1/a}, \quad \text{where } z = \text{ibetac_inv}(p, q, \text{prob}).$$

MISSING: transformatio from x to y.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_genbeta1.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Generalized Beta (Type 1) distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_genbeta1.**m_x**(*t*)

Returns NaN, since the moment generating function does not exist.

dist_genbeta1.**k_x**(*t*, *k=0*)

Returns NaN, since the cumulant generating function does not exist.

dist_genbeta1.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Generalized Beta (Type 1) distribution. The rth moments only exists for $-ap < k < \infty$.

$$\mu'_X(k) = \frac{b^k B(p + k/a, q)}{B(p, q)} = \frac{b^k \Gamma(p + k/a)\Gamma(p + q)}{\Gamma(p + q + k/a)\Gamma(p)},$$

See Kleiber, page 231.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_genbeta1.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Generalized Beta (Type 1) distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.8.7 Generalized Beta (Type 2) distribution

class ctx.**dist_genbeta2**(a, b, p, q)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The Generalized Beta (Type 2) distribution distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, $p > 0$, $q > 0$, and the support interval $(0, +\infty)$.

See also: Kleiber and Kotz [393] (page 184), Wikipedia [1208].

dist_genbeta2.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Generalized Beta (Type 2) distribution:

$$\mathrm{pdf}_X(x) = \frac{a x^{ap-1}}{b^{ap} B(p,q)[1+(x/b)^a]^{p+q}}, \quad x > 0.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256256256245623 56E-20
```

dist_genbeta2.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Generalized Beta (Type 2) distribution:

$$\mathrm{cdf}_X(x) = I_z(p,q), \quad \text{where } z = \left(\frac{x}{b}\right)^a, \quad x > 0.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256256256245623 56E-20
```

dist_genbeta2.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Generalized Beta (Type 2) distribution:

$$\mathrm{sf}_X(x) = 1 - I_z(p,q) = I_{1-z}(p,q), \quad \text{where } z = \left(\frac{x}{b}\right)^a, \quad x > 0.$$

Here ibeta($\cdot$) denotes the real normalised incomplete beta function, and ibetac($\cdot$) denotes the real normalised complementary incomplete beta function. See Kleiber, page 188.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.35635234625645256256256245623 56E-20
```

dist_genbeta2.**qtf**(*q*)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Generalized Beta (Type 2) distribution:

$$\text{qtf}_X(\text{prob}) = b \cdot z^{1/a}, \quad \text{where } z = \text{ibeta_inv}(p, q, \text{prob}).$$

Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.356352346256452562562562562456235 6E+00
```

dist_genbeta2.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Generalized Beta (Type 2) distribution:

$$\text{isf}_X(\text{prob}) = b \cdot z^{1/a}, \quad \text{where } z = \text{ibetac_inv}(p, q, \text{prob}).$$

Here $\text{ibetac_inv}(\cdot)$ denotes the inverse of the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452562562562562456235 6E+00
```

dist_genbeta2.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Generalized Beta (Type 2) distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.356352346256452562562562562456235 6E+00
```

dist_genbeta2.**m_x**(*t*)

Returns `NaN`, since the moment generating function does not exist.

dist_genbeta2.**k_x**(*t*, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_genbeta2.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 2 \ldots k$, of a random variable X, following a Generalized Beta (Type 2) distribution. The rth moments only exists for $-ap < k < aq$.

$$\mu'_X(k) = \frac{b^k B(p + k/a, q - k/a)}{B(p, q)} = \frac{b^k \Gamma(p + k/a)\Gamma(q - k/a)}{\Gamma(p)\Gamma(q)},$$

See Kleiber, page 188.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.356352346256452562562562562456235 6E+00
```

`dist_genbeta2.`**`cumulants`**(k)

Returns the first j cumulants, $\kappa_j, j = 2 \ldots k$, of a random variable X, following a Generalized Beta (Type 2) distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.356352346256452562562562562456235 6E+00
```

4.8.8 Generalized logistic distribution (JKB Types I - IV)

class ctx.**dist_genlogistic**(a, b)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The generalized logistic distribution distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, and the support interval $(0, +\infty)$.

See also: Johnson *et al.* [387], Johnson *et al.* [388], Wikipedia [1210].

Type IV is the most general form of the distribution. The Type III distribution can be obtained from Type IV by setting $\beta = \alpha$. The Type II distribution can be obtained from Type IV by setting $\alpha = 1$. The Type I distribution can be obtained from Type IV by setting $\beta = 1$. The logistic distribution is obtained by setting $\alpha = \beta = 1$.

For Type I and Type II, cdf, sf, qtf and isf are availabe in closed form in terms of elementary functions.

For Type III and Type IV, we use the fact that the variable $Z = (\beta/\alpha)\exp(-X)$ has a central F-distribution with $2\alpha, 2\beta$ degrees of freedom (see JKB vol2 2, p.142).

dist_genlogistic.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a generalized logistic (Type IV) distribution:

$$\mathrm{pdf}_X(x) = \frac{1}{B(\alpha,\beta)}\frac{e^{-\beta x}}{(1+e^{-x})^{\alpha+\beta}}, \quad \alpha,\beta > 0.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256256256256245623 56E-20
```

dist_genlogistic.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a generalized logistic distribution:

$$\mathrm{cdf}_X(x) = \frac{1}{(1+e^{-x})^{\alpha}} \quad \text{for Type I,} \quad \text{and } \mathrm{cdf}_X(x) = 1 - \frac{e^{-\beta x}}{(1+e^{-x})^{\alpha}} \quad \text{for Type II.}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256256256256245623 56E-20
```

dist_genlogistic.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a generalized logistic distribution:

The variable $Z = (\beta/\alpha)\exp(-x)$ has a central F-distribution with $2\alpha, 2\beta$ degrees of freedom (see JKB vol2 2, p.142)

$$\mathrm{sf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525625625625624562356E-20
```

dist_genlogistic.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a generalized logistic distribution:

The variable $Z = (\beta/\alpha)\exp(-x)$ has a central F-distribution with $2\alpha, 2\beta$ degrees of freedom (see JKB vol2 2, p.142)

$$\text{qtf}_X(q) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.3563523462564525625625625624562356E+00
```

dist_genlogistic.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a generalized logistic distribution:

The variable $Z = (\beta/\alpha)\exp(-x)$ has a central F-distribution with $2\alpha, 2\beta$ degrees of freedom (see JKB vol2 2, p.142)

$$\text{isf}_X(q) = ??$$

Here $\text{ibetac_inv}(\cdot)$ denotes the inverse of the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525625625625624562356E+00
```

dist_genlogistic.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a generalized logistic distribution:

$$C_X(t) = \frac{\Gamma(\beta - it)\Gamma(\alpha + it)}{\Gamma(\alpha)\Gamma(\beta)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.3563523462564525625625625624562356E+00
```

dist_genlogistic.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a generalized logistic distribution:

$$M_X(t) == \frac{\Gamma(\beta - t)\Gamma(\alpha + t)}{\Gamma(\alpha)\Gamma(\beta)}, \quad -\alpha < t < \beta.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256256256256245623 56E+00
```

dist_genlogistic.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a generalized logistic distribution:

$$K_X(t) = \log(\Gamma(\beta - t)) + \log(\Gamma(\alpha + t)) - \log(\Gamma(\alpha)) - \log(\Gamma(\beta)), \quad -\alpha < t < \beta.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256256256256245623 56E+00
```

dist_genlogistic.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 2 \ldots k$, of a random variable X, following a generalized logistic distribution. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256256256256245623 56E+00
```

dist_genlogistic.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 2 \ldots k$, of a random variable X, following a generalized logistic distribution. The cumulants are given by

$$\kappa_X(r) = \psi^{r-1}(\alpha) + (-1)^r \psi^{r-1}(\beta)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.35635234625645256256256256245623 56E+00
```

4.8.9 Generalized beta-exponential distribution

class ctx.**dist_gen_beta_exp**(a, b)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The generalized beta-exponential distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, and the support interval $(0, +\infty)$.

See also: Wikipedia [1200], Kleiber and Kotz [393] page 184, equ. 6.6, distribution of Y = log(X), Ristić *et al.* [481].

This is the distribution of the generalized logistic distribution, for b = a = 1.

dist_gen_beta_exp.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a generalized beta-exponential distribution:

$$\mathrm{pdf}_X(x) = \frac{a e^{ap(x-\log(b))}}{B(p,q)[1+e^{a(x-\log(b))}]^{p+q}}, \quad -\infty < x < \infty.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_gen_beta_exp.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a generalized beta-exponential distribution:

$$\mathrm{cdf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_gen_beta_exp.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a generalized beta-exponential distribution:

$$\mathrm{sf}_X(x) = ??$$

Here ibeta($\cdot$) denotes the real normalised incomplete beta function, and ibetac($\cdot$) denotes the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_gen_beta_exp.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a generalized beta-exponential distribution:

$$\text{qtf}_X(q) = ??$$

Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_gen_beta_exp.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a generalized beta-exponential distribution:

$$\text{isf}_X(q) = ??$$

Here $\text{ibetac_inv}(\cdot)$ denotes the inverse of the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_gen_beta_exp.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a generalized beta-exponential distribution:

$$C_X(t) = \frac{\Gamma(\beta - it)\Gamma(\alpha + it)}{\Gamma(\alpha)\Gamma(\beta)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_gen_beta_exp.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a generalized beta-exponential distribution:

$$M_X(t) = b^t \frac{\Gamma(p + t/a)\Gamma(q - t/a)}{\Gamma(p)\Gamma(q)}, \quad -p < t < q.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256256256256245623 56E+00
```

dist_gen_beta_exp.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a generalized beta-exponential distribution:

$$K_X(t) = \log(\Gamma(\beta - t)) + \log(\Gamma(\alpha + t)) - \log(\Gamma(\alpha)) - \log(\Gamma(\beta)), \quad -\alpha < t < \beta.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 2; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256256256245623 56E+00
```

dist_gen_beta_exp.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 2 \ldots k$, of a random variable X, following a generalized beta-exponential distribution. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_gen_beta_exp.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 2 \ldots k$, of a random variable X, following a generalized beta-exponential distribution. The cumulants are given by

$$\kappa_X(r) = \psi^{r-1}(\alpha) + (-1)^r \psi^{r-1}(\beta)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.8.10 Feller-Pareto distribution

class ctx.**dist_feller_pareto**(a, b)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The Feller-Pareto distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, and the support interval $(0, +\infty)$.

The Feller-Pareto distribution is defined as the distribution of a random variable X with

$$X = \mu + \theta \left(\frac{U}{V}\right)^{1/\gamma}, \quad \gamma, \theta > 0, \mu \in \mathbb{R},$$

where U and V are two independent gamma distributions with shape parameter $\tau > 0$ and $\alpha > 0$, respectively, and common scale parameter 1.

See also: Wikipedia [1202], Dutang *et al.* [295].

For $\mu = \theta, \gamma = \tau = 1$ we obtain the Pareto Type I distribution.

For $\gamma = \tau = 1$ we obtain the Pareto Type II distribution. When $\mu = 0$, we obtain what is generally simply called the Pareto distribution.

For $\alpha = \tau = 1$, we obtain the Pareto Type III distribution.

For $\tau = 1$ we obtain the Pareto Type IV distribution.

For $\mu = 0$, this reduces to the four parameter generalized beta of the second kind distribution (see GB2), with $\text{pdf}_X(x) = f(a, b, p, q)$ (McDonald 1984), where $a = \theta, b = \gamma, p = \tau, q = \alpha$.

dist_feller_pareto.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Feller-Pareto distribution:

$$\text{pdf}_X(x) = \frac{\gamma \cdot ((x-\mu)/\theta)^{\gamma\tau - 1}}{\theta B(\alpha, \tau)[1 + ((x-\mu)/\theta)^{\gamma}]^{\alpha+\tau}} = \frac{\gamma u^{\tau}(1-u)^{\alpha}}{(x-\mu)B(\alpha,\tau)}, \quad \text{where } u = \frac{y}{1+y}, y = \left(\frac{x-\mu}{\theta}\right)^{\gamma}, \quad x \geq \mu.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613356E-20
```

dist_feller_pareto.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Feller-Pareto distribution:

$$\text{cdf}_X(x) = I(\tau, \alpha; u), \quad \text{where } u = \frac{y}{1+y}, y = \left(\frac{x-\mu}{\theta}\right)^{\gamma}, \quad x \geq \mu.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613356E-20
```

dist_feller_pareto.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Feller-Pareto distribution:

$$\mathrm{sf}_X(x) = 1 - I(\tau, \alpha; u), \quad \text{where } u = \frac{y}{1+y}, y = \left(\frac{x-\mu}{\theta}\right)^{\gamma}, \quad x \geq \mu.$$

Here $\mathrm{ibeta}(\cdot)$ denotes the real normalised incomplete beta function, and $\mathrm{ibetac}(\cdot)$ denotes the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_feller_pareto.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Feller-Pareto distribution:

$$\mathrm{qtf}_X(q) = ??$$

Here $\mathrm{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_feller_pareto.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Feller-Pareto distribution:

$$\mathrm{isf}_X(q) = ??$$

Here $\mathrm{ibetac_inv}(\cdot)$ denotes the inverse of the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_feller_pareto.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Feller-Pareto distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_feller_pareto.**m_x**(*t*)

Returns NaN, since the moment generating function does not exist.

dist_feller_pareto.**k_x**(*t*, *k=0*)

Returns NaN, since the cumulant generating function does not exist.

dist_feller_pareto.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \dots k$, of a random variable X, following a Feller-Pareto distribution. The kth moments only exists for $k < \alpha\gamma$.

$$\mu'_X(k) = \sum_{j=0}^{k} \binom{k}{j} \frac{\mu^{k-j}\theta^j \Gamma(\tau + j/\gamma)\Gamma(\alpha - j/\gamma)}{\Gamma(\alpha)\Gamma(\tau)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_feller_pareto.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \dots k$, of a random variable X, following a Feller-Pareto distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

4.8.11 Fisher z distribution

class ctx.**dist_fisher_z**(m, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Fisher z-distribution is a continuous probability distribution with $m > 0$ and $n > 0$ degrees of freedom, and the support interval $(-\infty, +\infty)$. A random variable X follows a Fisher z-distribution with m and n degrees of freedom if it is defined as $X = \frac{1}{2}\log(Y)$, where Y follows a Fisher F-distribution with m and n degrees of freedom. The Fisher z-distribution is always unimodal, asymmetrical if $m \neq n$, and symmetrical if $m = n$. Interchanging m and n is the same as replacing z with $-z$. The mode is at 0.

See also Wikipedia [1151], MathWorld [799], BoostMath [56], Witkovský [1466], R (Statistical System) [523], Abramowitz and Stegun. [3], Butler and Paolella [166], Chattamvelli and Jones [175], Witkovský [1460].

See also: https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-12/issue-4/A-Study-of-R-A-Fishers-z-Distribution-and-the/10.1214/aoms/1177731681.full

A Study of R. A. Fisher's z Distribution and the Related F Distribution. Leo A. Aroian. Ann. Math. Statist. 12(4): 429-448 (December, 1941). DOI: 10.1214/aoms/1177731681

dist_fisher_z.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Fisher z distribution:

$$\text{pdf}_X(x) = 2e^{2x} f_{\text{FisherF}}(e^{2x}; m, n),$$

where $f_{\text{FisherF}}(\cdot, m, n)$ is the pdf of the Fisher F-distribution with m and n degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_fisher_z.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Fisher z distribution:

$$\text{cdf}_X(x) = F_{\text{FisherF}}(e^{2x}; m, n),$$

where $F_{\text{FisherF}}(\cdot, m, n)$ is the cdf of the Fisher F-distribution with m and n degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_fisher_z.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a Fisher z distribution:

$$\text{sf}_X(x) = F_{\text{FisherF}}(e^{-2x}; n, m),$$

where $F_{\text{FisherF}}(\cdot, n, m)$ is the cdf of the Fisher F-distribution with n and m degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

`dist_fisher_z.qtf(`q`)`

Returns $\text{qtf}_X(q)$, the quantile function (qtf) of a random variable X, following a Fisher z distribution:

$$\text{qtf}_X(q) = \tfrac{1}{2}\log\left(F^{-1}_{\text{FisherF}}(q;m,n)\right),$$

where $F^{-1}_{\text{FisherF}}(\cdot,m,n)$ is the qtf of the Fisher F-distribution with m and n degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

`dist_fisher_z.isf(`q`)`

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Fisher z distribution:

$$\text{isf}_X(q) = -\tfrac{1}{2}\log\left(F^{-1}_{\text{FisherF}}(q;n,m)\right),$$

where $F^{-1}_{\text{FisherF}}(\cdot,n,m)$ is the qtf of the Fisher F-distribution with n and m degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

`dist_fisher_z.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following a Fisher z distribution:

$$C_X(t) = \left(\frac{n}{m}\right)^{it/2} \frac{\Gamma\left(\frac{1}{2}(n-it)\right)\Gamma\left(\frac{1}{2}(m+it)\right)}{\Gamma\left(\frac{1}{2}n\right)\Gamma\left(\frac{1}{2}m\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

`dist_fisher_z.m_x(`t`)`

Returns the moment generating function of a random variable X, following a Fisher z distribution.

$$M_X(t) = \left(\frac{n}{m}\right)^{t/2} \frac{\Gamma\left(\frac{1}{2}(n-t)\right)\Gamma\left(\frac{1}{2}(m+t)\right)}{\Gamma\left(\frac{1}{2}n\right)\Gamma\left(\frac{1}{2}m\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561456135 6E+00
```

dist_fisher_z.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a Fisher z distribution.

$$K_X(t) = \log\left(\Gamma\left(\tfrac{1}{2}(n-t)\right)\right) + \log\left(\Gamma\left(\tfrac{1}{2}(m+t)\right)\right) + \tfrac{1}{2}t\left(\log(n) - \log(m)\right) - \log\left(\Gamma\left(\tfrac{1}{2}n\right)\right) - \log\left(\Gamma\left(\tfrac{1}{2}m\right)\right).$$

$$K_X^{(1)}(t) = \tfrac{1}{2}\left(-\psi^{(0)}\left(\tfrac{1}{2}n - t\right) + \psi^{(0)}\left(\tfrac{1}{2}m + t\right)\right) + \tfrac{1}{2}\left(\log(n) - \log(m)\right),$$

$$K_X^{(j)}(t) = 2^{-j}\left((-1)^j\psi^{(j-1)}\left(\tfrac{1}{2}n - t\right) + \psi^{(j-1)}\left(\tfrac{1}{2}m + t\right)\right), \quad j \geq 2,$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561456135 6E+00
```

dist_fisher_z.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Fisher z distribution. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561456135 6E+00
```

dist_fisher_z.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Fisher z distribution.

$$\kappa_1 = \tfrac{1}{2}\left(-\psi^{(0)}\left(\tfrac{1}{2}n\right) + \psi^{(0)}\left(\tfrac{1}{2}m\right)\right) + \tfrac{1}{2}\left(\log(n) - \log(m)\right),$$

$$\kappa_r = 2^{-r}\left((-1)^r\psi^{(r-1)}\left(\tfrac{1}{2}n\right) + \psi^{(r-1)}\left(\tfrac{1}{2}m\right)\right), \quad r \geq 2,$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561456135 6E+00
```

Aroian gives the following approximations for the cumulants, for $n \geq 2, m \geq 2$:

$$\kappa_1 \approx \frac{1}{2}\left(\frac{1}{n} - \frac{1}{m}\right) + \frac{1}{6}\left(\frac{1}{n^2} - \frac{1}{m^2}\right) - \frac{1}{15}\left(\frac{1}{n^4} - \frac{1}{m^4}\right) + \frac{8}{63}\left(\frac{1}{n^6} - \frac{1}{m^6}\right),$$

$$\kappa_r \approx \frac{(r-2)!}{2}\left(\frac{n+r-1}{n^r}+(-1)^r\frac{m+r-1}{m^r}\right)+\frac{r!}{6}\left(\frac{1}{n^{r+1}}+\frac{(-1)^r}{m^{r+1}}\right)-\frac{(r+2)!}{90}\left(\frac{1}{n^{r+3}}+\frac{(-1)^r}{m^{r+3}}\right), \quad r \geq 2.$$

Approximations

`ctx.`**`fisher_z_ecf`**`(`*x*, *m*, *n*, *results='cdf'*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

`ctx.`**`fisher_z_ecf_inv`**`(`*q*, *m*, *n*, *results='qtf'*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

`ctx.`**`fisher_z_spa`**`(`*x*, *n*, *results='c'*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

`ctx.`**`fisher_z_spa_inv`**`(`*x*, *n*, *results='qtf'*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.8.12 Skew t-distribution (Jones)

class ctx.**dist_skewt**(a, b)

where ctx is dec, mpm, ipm, fpm, gmp or arb.

The skew-t-distribution is a continuous probability distribution with parameters $a > 0$ and $b > 0$, and the support interval $(-\infty, +\infty)$.

See also: Jones and Faddy [390].

Other skew t-distributions: see downloads.

dist_skewt.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a skew-t-distribution:

$$\mathrm{pdf}_X(x) = \frac{1}{2^{a+b-1}B(a,b)\sqrt{a+b}} \times \left(1 + \frac{x}{\sqrt{a+b+x^2}}\right)^{a+1/2} \times \left(1 - \frac{x}{\sqrt{a+b+x^2}}\right)^{b+1/2}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_skewt.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a skew-t-distribution:

$$\mathrm{cdf}_X(x) = I_z(a,b), \quad \text{where } z = \frac{1}{2}\left(1 + \frac{x}{\sqrt{a+b+x^2}}\right)$$

Here ibeta$(\cdot)$ denotes the real normalised incomplete beta function, and ibetac$(\cdot)$ denotes the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_skewt.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a skew-t-distribution:

$$\mathrm{sf}_X(x) = 1 - I_z(a,b), \quad \text{where } z = \frac{1}{2}\left(1 + \frac{x}{\sqrt{a+b+x^2}}\right)$$

Here ibeta$(\cdot)$ denotes the real normalised incomplete beta function, and ibetac$(\cdot)$ denotes the real normalised complementary incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_skewt.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a skew-t-distribution:

$$\text{qtf}_X(q) = \frac{\sqrt{a+b}(2Y-1)}{2\sqrt{Y(1-Y)}},$$

where Y is the quantile of the beta distribution. Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_skewt.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a skew-t-distribution:

$$\text{isf}_X(q) = \frac{\sqrt{a+b}(2Y-1)}{2\sqrt{Y(1-Y)}},$$

where Y is the quantile of the beta distribution. Here $\text{ibeta_inv}(\cdot)$ denotes the inverse of the real normalised incomplete beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_skewt.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a skew-t-distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_skewt.**m_x**(t)

Returns NaN, since the moment generating function does not exist.

dist_skewt.**k_x**(t, *k=0*)

Returns NaN, since the cumulant generating function does not exist.

dist_skewt.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a skew-t-distribution. The rth moments only exists for $a > r/2$ and $b > r/2$.

$$\mu'_X(r) = \frac{(a+b)^{r/2}}{B(a,b)} \sum_{i=0}^{r} \binom{r}{i} 2^{-i} (-1)^i B\left(a + \frac{r}{2} - i, b - \frac{r}{2}\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_skewt.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a skew-t-distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.8.13 Pearson's rho distribution (under H_0)

class ctx.**dist_pearson_rho**(N)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Pearson's rho (under H_0) with sample size $N \geq 3$, ie. the distribution of the sample correlation coefficient when $\rho = 0$), is a continuous distribution with the support interval $(-1, +1)$.

See also: Wikipedia [1226], Johnson *et al.* [388] page 550.

See also Johnson II, page 550, for characteristic function and mgf.

dist_pearson_rho.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Pearson's rho (under H_0):

$$\text{pdf}_X(x) = \frac{(1-r^2)^{\frac{N-4}{2}}}{B\left(\frac{1}{2}, \frac{N-2}{2}\right)},$$

where $B(a, b)$ is the beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", pearson_rho(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_pearson_rho.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of Pearson's rho (under H_0):

$$\text{cdf}_X(r) = F_{\text{StudentT}}(t, N-2), \quad \text{where } t = r\sqrt{\frac{N-2}{1-r^2}},$$

and $F_{\text{StudentT}}(t, N-2,)$ is the cdf of the t-distribution with $N-2$ degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", pearson_rho(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_pearson_rho.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of Pearson's rho (under H_0):

$$\text{sf}_X(r) = F_{\text{StudentT}}(-t, N-2), \quad \text{where } t = r\sqrt{\frac{N-2}{1-r^2}},$$

and $F_{\text{StudentT}}(t, N-2,)$ is the cdf of the t-distribution with $N-2$ degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", pearson_rho(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

`dist_pearson_rho.qtf(q)`

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of Pearson's rho (under H_0):

$$\text{qtf}_X(q) = \frac{t_{\alpha,N-2}}{\sqrt{N-2+t_{\alpha,N-2}^2}},$$

where $t_{\alpha,N-2}$ is the quantile function of the central t-distribution with $N-2$ degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", pearson_rho(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

`dist_pearson_rho.isf(q)`

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of Pearson's rho (under H_0):

$$\text{isf}_X(q) = \frac{-t_{\alpha,N-2}}{\sqrt{N-2+t_{\alpha,N-2}^2}},$$

where $t_{\alpha,N-2}$ is the quantile function of the central t-distribution with $N-2$ degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", pearson_rho(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

`dist_pearson_rho.c_x(t)`

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of Pearson's rho (under H_0):

$$C_X(t) = \Gamma(\tfrac{1}{2}(N-1)) \cdot 2^{(N-3)/2} \cdot t^{-(N-3)/2} \cdot J_{(N-3)/2}(t),$$

where $J_\nu(t)$ is the Bessel function of the first kind of order ν.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", pearson_rho(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_pearson_rho.m_x(t)`

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of Pearson's rho (under H_0):

$$M_X(t) = \Gamma(\tfrac{1}{2}(N-1)) \cdot 2^{(N-3)/2} \cdot t^{-(N-3)/2} \cdot I_{(N-3)/2}(t),$$

where $I_\nu(t)$ is the modified Bessel function of the second kind of order ν.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", pearson_rho(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_pearson_rho.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following the distribution of Pearson's rho (under H_0):

$$K_X(t) = \log\left(\Gamma(\tfrac{1}{2}(N-1)) \cdot 2^{(N-3)/2} \cdot t^{-(N-3)/2} \cdot I_{(N-3)/2}(t)\right),$$

where $I_\nu(t)$ is the modified Bessel function of the second kind of order ν.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", pearson_rho(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_pearson_rho.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of Pearson's rho (under H_0):

All odd moments are zero. For the even moments we have

$$\mu_X(r) = \tfrac{1}{2} B\left(\tfrac{1}{2}(r+1), \tfrac{1}{2}(N-2)\right)$$

where $B(a, b)$ is the beta function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", pearson_rho(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_pearson_rho.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of Pearson's rho (under H_0). The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", pearson_rho(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.9 Noncentral distributions

4.9.1 Boost: Noncentral χ^2 distribution

The following functions return the pdf, cdf, qtf or boost class of the noncentral chi-squared distribution with degrees of freedom $n > 0$, noncentrality parameter λ_1, and support interval $(0, +\infty)$.

See also Wikipedia [1164], MathWorld [808], Patnaik [464], Penev and Raykov [466], Wang and Gray [793], Winterbottom [1454], BoostMath [64], Johansson [386].

`math53.chi2_nc_pdf(`*n*, *lambda1*, *x*`)`

Returns pdf(x), the value of the probability density function (*Pdf*) of the noncentral chi-squared distribution:

$$\text{pdf}(x) = f_{\chi^2}(n, x; \lambda_1) = \frac{1}{2} e^{-(x+\lambda)/2} \left(\frac{x}{\lambda}\right)^{k/4-1/2} I_{k/2-1}(\sqrt{\lambda x}) \tag{4.9}$$

where $I_k(y)$ is a modified Bessel function of the first kind of order k.

Alternatively, the pdf can be written in a form which shows the relationship to the central distribution more clearly:

$$\text{pdf}(x) = f_{\chi^2}(n, x; \lambda_1) = f_{\chi^2}(x, n) \times e^{-\lambda_1/2} \times {}_0F_1\left(-; \frac{n}{2}; \frac{x\lambda_1}{4}\right). \tag{4.10}$$

Here $f_{\chi^2}(\cdot)$ is the PDF of the central chi-square distribution, and ${}_0F_1(\cdot)$ is the confluent hypergeometric limit function.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("Chi2NcPdf(x, a, b): ", Chi2NcPdf(x, a, b))
>>> print ("dist_chi_squared_nc(a, b).pdf(x): ", dist_chi_squared_nc(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.chi2_nc_cdf(`*n*, *lambda1*, *x*`)`

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the noncentral chi-squared distribution:

$$\text{cdf}(x) = F_{\chi^2}(n, x; \lambda) = \int_0^x f_{\chi^2}(n, x; \lambda_1)\, \mathrm{d}t.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("Chi2NcCdf(x, a, b): ", Chi2NcCdf(x, a, b))
>>> print ("dist_chi_squared_nc(a, b).cdf(x): ", dist_chi_squared_nc(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.chi2_nc_qtf(`*n*, *lambda1*, *q*`)`

where `Ctx` is `Math53` or `CtxBoost`.

Returns qtf(q), the value of the quantile function (*Qtf*) of the noncentral chi-squared distribution:

There is no known closed exact form for qtf(q). The default method is to call the function provided by Boost.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("Chi2NcQtf(q, a, b): ", Chi2NcQtf(q, a, b))
>>> print ("dist_chi_squared_nc(a, b).qtf(q): ", dist_chi_squared_nc(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_chi2_nc**(*n*, *lambda1*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The noncentral chi-square distribution is a continuous probability distribution with degrees of freedom $n > 0$, noncentrality parameter λ_1, and support interval $(0, \infty)$. See also Wikipedia [1164], MathWorld [808], Patnaik [464], Penev and Raykov [466], Wang and Gray [793], Winterbottom [1454], BoostMath [64], Witkovský [1474], Johansson [386], R (Statistical System) [516], Yu [1498].

dist_chi_squared_nc.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a non-central chi-squared distribution:

$$\text{pdf}_X(x) = f_{\chi^2}(n, x; \lambda) = e^{-\lambda/2} f_{\chi^2}(n, x) {}_0F_1\left(-; \frac{n}{2}; \frac{x\lambda}{4}\right),$$

Here $f_{\chi^2}(\cdot)$ and $F_{\chi^2}(\cdot)$ are the PDF and CDF, respectively, of the central chi-square distribution, and ${}_0F_1(\cdot)$ is the confluent hypergeometric limit function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", chi_squared_nc(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_chi_squared_nc.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a non-central chi-squared distribution:

$$\text{cdf}_X(x) = \int_0^x \text{pdf}_X(x)\text{d}t = F_{\chi^2}(n, x; \lambda) = e^{-\lambda/2} \sum_{j=0}^{\infty} \frac{(\lambda/2)^j}{j!} F_{\chi^2}(n + 2 + j, x),$$

Here $f_{\chi^2}(\cdot)$ and $F_{\chi^2}(\cdot)$ are the PDF and CDF, respectively, of the central chi-square distribution, and ${}_0F_1(\cdot)$ is the confluent hypergeometric limit function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", chi_squared_nc(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_chi_squared_nc.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a non-central chi-squared distribution:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x) = \int_x^{\infty} \text{pdf}_X(x)\text{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", chi_squared_nc(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_chi_squared_nc.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a non-central chi-squared distribution:

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", chi_squared_nc(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_chi_squared_nc.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a non-central chi-squared distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", chi_squared_nc(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_chi_squared_nc.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a non-central chi-squared distribution:

$$C_X(t) = \exp\left(\frac{i\lambda t}{1-2it}\right)(1-2it)^{-n/2}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared_nc(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_chi_squared_nc.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a non-central chi-squared distribution:

$$M_X(t) = \exp\left(\frac{\lambda t}{1-2t}\right)(1-2t)^{-n/2}, \quad t \in \left(-\infty, \tfrac{1}{2}\right),$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared_nc(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_chi_squared_nc.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a non-central chi-squared distribution:

$$K_X(t) = -\frac{n}{2}\log(1-2t) + \frac{\lambda t}{1-2t}, \quad t \in \left(-\infty, \tfrac{1}{2}\right),$$

$$K_X^{(j)}(t) = \frac{2^{j-1}(j-1)!}{(1-2t)^j}\left[n + \frac{\lambda j}{1-2t}\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", chi_squared_nc(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_chi_squared_nc.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a non-central chi-squared distribution: the moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", chi_squared_nc(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_chi_squared_nc.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a non-central chi-squared distribution:

$$\kappa_r = 2^{r-1}(r-1)!(n + r\lambda)$$

```
def MakeNoncentralChiSquaredCumulants(self):
    df = iv.mpf(2000)
    lambda_ = 33
    k = 10
    kappa = iv.matrix(k+1, 1)
    kappa[0] = 1
    kappa[1] = df + lambda_
    for i in range(2, k+1):
        kappa[i] = kappa[i - 1] * 2 * (i - 1) * (1 + lambda_ / (df + (i - 1) * lambda_))
    return kappa
```

Recurrences: Non-central Chi-square

ctx.**chi_squared_nc_recurrence**(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The following recurrence relations hold for the pdf and CDF (see Cohen [183]):

$$f_{\chi^2}(n+4, x; \lambda) = \frac{x \cdot f_{\chi^2}(n, x; \lambda) - n \cdot f_{\chi^2}(n+2, x; \lambda)}{\lambda}$$

$$F_{\chi^2}(n, x; \lambda) - F_{\chi^2}(n+2, x; \lambda) = 2 f_{\chi^2}(n+2, x; \lambda)$$

$$F_{\chi^2}(n, x; \lambda) - F_{\chi^2}(n-2, x; \lambda) = 2\frac{\partial}{\partial \lambda} F_{\chi^2}(n-2, x; \lambda)$$

Ref:

Pav 2015: Moments of log noncentral chisquare

Yu 2011: mode of noncentral chi-square

Approximations

`ctx.chi_squared_nc_gp`(*x*, *n*, *lambda*, *results='cdf'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the pdf, cdf and sf from the characteristic function using the procedure of Gil-Pelaez (see *gil_pelaez_pdf()* and *gil_pelaez_cdf()*).

`ctx.chi_squared_nc_ecf`(*x*, *f*, *lambda1*, *results='cdf'*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

`ctx.chi_squared_nc_ecf_inv`(*q*, *f*, *lambda1*, *results='qtf'*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

`ctx.chi_squared_nc_spa`(*x*, *n*, *lambda*, *results='cdf'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf.

The solution $\hat{s}(x)$ of the saddlepoint equation $K_X^{(1)}(\hat{s}(x)) = x$, of a random variable X, following a non-central chi-squared distribution is given by:

$$\hat{s}(x) = -\frac{1}{4x}\left[n - 2x + \sqrt{n^2 + 4x\lambda}\right], \quad x > 0$$

```
def NonCentralChi2_SPA2(self, n0, x0, lambda0_):
    n = iv.mpf(n0)
    x = iv.mpf(x0)
    lambda_ = iv.mpf(lambda0_)
    s = -(1 / (4 * x)) * (n - 2 * x + iv.sqrt(n * n + 4 * x * lambda_))
    Order = 18
    kderiv = iv.matrix(Order+2, 1)
    for j in range(0, Order+1):
        kderiv[j] = self.NonCentralChi2_CGF_Derivative(s, n, lambda_, j)

    LeftTail, RightTail = self.LugannaniRice(Order, kderiv, s)
    return LeftTail, RightTail
```

The following code tests the procedure

```
def LugannaniRiceDemo(self):
    nu = 40.0
    x = 61.0
    nc = 70.0
    LeftTail, RightTail = self.NonCentralChi2_SPA2(nu, x, nc)
    print("LeftTail:  ", LeftTail)
    print("RightTail: ", RightTail)
```

`ctx.chi_squared_nc_spa_inv`(*x*, *n*, *lambda*, *results='qtf'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.9.2 Boost: Noncentral Student t distribution

The following functions return the pdf, cdf, qtf or boost class of the non-central Student t distribution with $n > 0$ degrees of freedom, noncentrality parameter δ, and support interval $(-\infty, +\infty)$.

See also Wikipedia [1165], MathWorld [810], BoostMath [66], Benton and Krishnamoorthy [30], Broda and Paolella [160], Owen [454], Wang and Gray [793], Witkovský [1461], Johansson [386].

`math53.student_t_nc_pdf`(*n*, *delta*, *x*)

Returns pdf(x), the value of the probability density function (*Pdf*) of the non-central Student t distribution:

$$\text{pdf}(x) = f_{\text{StudentT}}(n, x, \delta) = \int_0^\infty \phi\left(x\sqrt{\frac{y}{n}} - \delta\right) f_{\chi^2}(y, n)\sqrt{\frac{y}{n}}\,\mathrm{d}y, \tag{4.11}$$

where $\phi(\cdot)$ denotes the pdf of the standard normal distribution and $f_{\chi^2}(\cdot, n)$ denotes the pdf of the χ^2 distribution with n degrees of freedom (see *chi_squared_pdf()*).

Alternatively, the pdf can be written in a form which shows the relationship to the central distribution more clearly:

$$\text{pdf}(x) = f_{t'}(x; n, \delta) = \frac{n^m \Gamma(n+1)\exp(-\frac{1}{2}\delta^2)}{2^n a^m \Gamma(m)}\left[\frac{\sqrt{2}\delta x \cdot {}_1F_1(m+1; \frac{3}{2}; y^2)}{a\Gamma(m+\frac{1}{2})} - \frac{{}_1F_1(m+\frac{1}{2}; \frac{1}{2}; y^2)}{\sqrt{a}\Gamma(m+1)}\right], \tag{4.12}$$

$$m = \frac{n}{2}, \quad a = n + x^2, \quad y^2 = \frac{\delta^2 x^2}{2a},$$

and ${}_1F_1(\cdot)$ is the confluent hypergeometric function.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("StudentTNcPdf(x, a, b): ", StudentTNcPdf(x, a, b))
>>> print ("dist_student_t_nc(a, b).pdf(x): ", dist_student_t_nc(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.student_t_nc_cdf`(*n*, *delta*, *x*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the non-central Student t distribution:

$$\text{cdf}(x) = F_{\text{StudentT}}(x, n, \delta) = \int_0^\infty \Phi\left(x\sqrt{\frac{y}{n}} - \delta\right) f_{\chi^2}(y, n)\,\mathrm{d}y, \tag{4.13}$$

for $x < 0$ by $F_{\text{StudentT}}(n, x, \delta) = 1 - F_{\text{StudentT}}(n, -x, -\delta)$, and for $x = 0$ by $F_{\text{StudentT}}(n, x, \delta) = \Phi(-\delta)$.

Here $\Phi(\cdot)$ denotes the cdf of the standard normal distribution and $f_{\chi^2}(\cdot, n)$ denotes the pdf of the χ^2 distribution with n degrees of freedom.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("StudentTNcCdf(x, a, b): ", StudentTNcCdf(x, a, b))
>>> print ("dist_student_t_nc(a, b).cdf(x): ", dist_student_t_nc(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**student_t_nc_qtf**(*n*, *delta*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the non-central Student t distribution:

There is no known closed exact form for qtf(q). For `fpm.` the default method is to call the function provided by Boost.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("StudentTNcQtf(q, a, b): ", StudentTNcQtf(q, a, b))
>>> print ("dist_student_t_nc(a, b).qtf(q): ", dist_student_t_nc(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_student_t_nc**(*n*, *lambda1*)

The noncentral t distribution is a continuous probability distribution with degrees of freedom $n > 0$, noncentrality parameter δ, and support interval $(-\infty, +\infty)$. See also Wikipedia [1165], MathWorld [810], BoostMath [66], Benton and Krishnamoorthy [30], Broda and Paolella [160], Owen [454], Wang and Gray [793], Witkovský [1461], Johansson [386], R (Statistical System) [518].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nctdtr.html#scipy.special.nctdtr

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nctdtridf.html#scipy.special.nctdtridf

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nctdtrit.html#scipy.special.nctdtrit

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nctdtrinc.html#scipy.special.nctdtrinc

Ref: Kim (2007), CI noncentrality parameters

Application: Scholz.

Young, R package tolerance intervals

dist_student_t_nc.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a non-central Student t distribution:

$$\text{pdf}_X(x) = f_{t'}(x; n, \delta) = \frac{n^m \Gamma(n+1) \exp(-\frac{1}{2}\delta^2)}{2^n a^m \Gamma(m)} \left[\frac{\sqrt{2}\delta x \cdot {}_1F_1(m+1; \frac{3}{2}; y^2)}{a\Gamma(m+\frac{1}{2})} - \frac{{}_1F_1(m+\frac{1}{2}; \frac{1}{2}; y^2)}{\sqrt{a}\Gamma(m+1)} \right],$$

$$m = \frac{n}{2}, \quad a = n + x^2, \quad y^2 = \frac{\delta^2 x^2}{2a},$$

and ${}_1F_1(\cdot)$ is the confluent hypergeometric function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", student_t_nc(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_student_t_nc.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a non-central Student t distribution:

$$\text{cdf}_X(x) = F_{t'}(x; n, \delta) = \int_0^x f_t(x; n, \delta) \mathrm{d}t = \Phi(-\delta) + \frac{1}{2} \sum_{i=0}^{\infty} P_i I_z\left(i + \tfrac{1}{2}, m\right) + \frac{\delta}{\sqrt{2}} Q_i I_z\left(i + 1, m\right),$$

$$P_i = \frac{e^{-\lambda}\lambda^i}{i!}, \quad Q_i = \frac{e^{-\lambda}\lambda^i}{\Gamma(i+3/2)}; \quad z = \frac{t^2}{n+t^2},$$

and $I_x(\cdot,\cdot)$ denotes the (normalized) incomplete beta function , and $\Phi(\cdot)$ denotes the CDF of the normal distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", student_t_nc(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

`dist_student_t_nc.`**`sf`**`(`x`)`

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a non-central Student t distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^\infty \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", student_t_nc(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

`dist_student_t_nc.`**`qtf`**`(`q`)`

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a non-central Student t distribution:

There is no known explicit form for the quantile function $\mathrm{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a normal approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", student_t_nc(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

`dist_student_t_nc.`**`isf`**`(`q`)`

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a non-central Student t distribution:

$$\mathrm{isf}_X(q) = \mathrm{qtf}_X(1-q) = \mathrm{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", student_t_nc(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

`dist_student_t_nc.`**`c_x`**`(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following a non-central Student t distribution:

$$C_X(t) = \int_{-\infty}^{\infty} e^{itx}\mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", student_t_nc(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_student_t_nc.**m_x**(*t*)

Returns NaN, since the moment generating function does not exist.

dist_student_t_nc.**k_x**(*t*, *k=0*)

Returns NaN, since the cumulant generating function does not exist.

dist_student_t_nc.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a non-central Student t distribution. The rth moment only exists for $n > r$. and is given by

$$\mu'_X(r) = \left(\tfrac{1}{2}n\right)^{r/2} \frac{\Gamma\left(\frac{1}{2}(n-r)\right)}{\Gamma\left(\frac{1}{2}n\right)} \times \sum_{i=0}^{\lfloor r/2 \rfloor} \binom{r}{2i} \frac{(2i)!}{2^i i!} \delta^{r-2i},$$

See also: Paoella 2, page 381-382

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", student_t_nc(mu, sigma).moments(k))
6.3563523462564525615615615614561356E+00
```

dist_student_t_nc.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a non-central Student t distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", student_t_nc(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

Approximations

ctx.**student_t_nc_ecf**(*x*, *n*, *delta*, *results='cdf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Edgeworth approximation to the pdf, cdf and sf. See also: Paolella [456], page 381-382.

ctx.**student_t_nc_ecf_inv**(*q*, *n*, *delta*, *theta*, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

4.9.3 Boost: Noncentral Fisher F distribution

The following functions return the pdf, cdf, qtf or boost class of the non-central Fisher F distribution with $m > 0$ and $n > 0$ degrees of freedom, noncentrality parameter λ_1 , and the support interval $(0, +\infty)$.

See also Wikipedia [1162], MathWorld [809], BoostMath [65], Benton and Krishnamoorthy [30], Butler and Paolella [166], Chou [179], Chattamvelli and Jones [175], Wang and Gray [793], Johansson [386].

`math53.fisher_f_nc_pdf(m, n, lambda1, x)`

Returns pdf(x), the value of the probability density function (*Pdf*) of the non-central Fisher F distribution:

$$\text{pdf}(x) = f_{\text{FisherF}}(x; m, n; \lambda_1) = \frac{m}{n} \int_0^\infty y \cdot f_{\chi^2}\left(x \cdot y \cdot m/n, m, \lambda_1\right) \cdot f_{\chi^2}(y, n)\, \mathrm{d}y \tag{4.14}$$

Here $f_{\chi^2}(\cdot, m, \lambda_1)$ denotes the pdf of the noncentral χ^2 distribution with m degrees of freedom and noncentrality parameter λ_1 (see *chi2_nc_pdf()*), and $f_{\chi^2}(\cdot, n)$ denotes the pdf of the central χ^2 distribution with n degrees of freedom (see *chi_squared_pdf()*).

Alternatively, the pdf can be written in a form which shows the relationship to the central distribution more clearly:

$$\text{pdf}(x) = f_{F'}(x; n_1, n_2, \lambda_1) = e^{-\lambda_1/2} f_F(x; n_1, n_2) {}_1F_1\left(\tfrac{1}{2}(m+n), \tfrac{1}{2}n, \tfrac{nx\lambda_1}{2(m+nx)}\right), \tag{4.15}$$

Here $f_F(\cdot)$ denotes the pdf of the central F-distribution, and ${}_1F_1(\cdot)$ is the confluent hypergeometric function.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("FisherFNcPdf(x, a, b): ", FisherFNcPdf(x, a, b))
>>> print ("dist_fisher_f_nc(a, b).pdf(x): ", dist_fisher_f_nc(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.fisher_f_nc_cdf(m, n, lambda1, x)`

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the non-central Fisher F distribution:

$$\text{cdf}(x) = F_{\text{FisherF}}(x; m, n; \lambda_1) = \frac{mx}{n} \int_0^\infty f_{\chi^2}\left(x \cdot y \cdot m/n, m; \lambda_1\right) \cdot \left[1 - F_{\chi^2}(y, n)\right] \mathrm{d}y \tag{4.16}$$

Here $f_{\chi^2}(\cdot, m, \lambda_1)$ denotes the pdf of the noncentral χ^2 distribution with m degrees of freedom and noncentrality parameter λ_1 (see *chi2_nc_pdf()*), and $F_{\chi^2}(\cdot, n)$ denotes the cdf of the central χ^2 distribution with n degrees of freedom (see *chi_squared_cdf()*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("FisherFNcCdf(x, a, b): ", FisherFNcCdf(x, a, b))
>>> print ("dist_fisher_f_nc(a, b).cdf(x): ", dist_fisher_f_nc(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.fisher_f_nc_qtf(m, n, lambda1, q)`

Returns qtf(q), the value of the quantile function (*Qtf*) of the non-central Fisher F distribution:

There is no known closed exact form for qtf(q). For `fpm.` the default method is to call the function provided by Boost.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("FisherFNcQtf(q, a, b): ", FisherFNcQtf(q, a, b))
>>> print ("dist_fisher_f_nc(a, b).qtf(q): ", dist_fisher_f_nc(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_fisher_f_nc**(*m*, *n*, *lambda1*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The non-central Fisher F distribution is a continuous probability distribution with $m > 0$ and $n > 0$ degrees of freedom, noncentrality parameter λ_1 , and the support interval $(0, +\infty)$. See also Wikipedia [1162], MathWorld [809], BoostMath [65], Benton and Krishnamoorthy [30], Butler and Paolella [166], Chou [179], Chattamvelli and Jones [175], Wang and Gray [793], Witkovský [1475], Johansson [386], R (Statistical System) [517]. Also doubly-noncentral: Yin [1497].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.ncfdtr.html#scipy.special.ncfdtr

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.ncfdtridfd.html#scipy.special.ncfdtridfd

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.ncfdtridfn.html#scipy.special.ncfdtridfn

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.ncfdtri.html#scipy.special.ncfdtri

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.ncfdtrinc.html#scipy.special.ncfdtrinc

dist_fisher_f_nc.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a non-central Fisher F distribution:

$$\text{pdf}_X(x) = f_{F'}(x; n_1, n_2, \lambda) = e^{-\lambda/2} f_F(x; n_1, n_2)\, {}_1F_1\left(\tfrac{1}{2}(m+n), \tfrac{1}{2}n, \tfrac{nx\lambda}{2(m+nx)}\right),$$

Here $f_F(\cdot)$ denotes the pdf of the central F-distribution, and ${}_1F_1(\cdot)$ is the confluent hypergeometric function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f_nc(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_fisher_f_nc.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a non-central Fisher F distribution:

$$\text{cdf}_X(x) = F_{F'}(x; m, n, \lambda) = e^{-\lambda} \sum_{j=0}^{\infty} \frac{(\lambda/2)^j}{j!} F_F(x; m+2j, n)$$

Here $F_F(\cdot)$ denotes the cdf of the central F-distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f_nc(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_fisher_f_nc.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a non-central Fisher F distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^\infty \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f_nc(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_fisher_f_nc.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a non-central Fisher F distribution:

There is no known explicit form for the quantile function $\mathrm{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f_nc(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_fisher_f_nc.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a non-central Fisher F distribution:

$$\mathrm{isf}_X(q) = \mathrm{qtf}_X(1-q) = \mathrm{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f_nc(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_fisher_f_nc.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a non-central Fisher F distribution:

$$C_X(t) = \int_0^\infty e^{itx}\mathrm{pdf}_X(x)\mathrm{d}x$$

$$\phi(t) = e^{-\lambda/2}\sum_{k=0}^{\infty}\frac{(\lambda/2)^k}{k!}{}_1F_1\left(\frac{\nu_1}{2}+k, -\frac{\nu_2}{2}, -\frac{\nu_2}{\nu_1}it\right).$$

CRC: page 122: arcsin, page 146 half-normal

See also: http://www.stat.uchicago.edu/~yibi/teaching/stat222/2017/Lectures/C05.pdf

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f_nc(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_fisher_f_nc.**m_x**(*t*)

Returns NaN, since the moment generating function does not exist.

dist_fisher_f_nc.**k_x**(*t*, *k=0*)

Returns NaN, since the cumulant generating function does not exist.

dist_fisher_f_nc.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \dots k$, of a random variable X, following a non-central Fisher F distribution. The rth moment only exists for $n_2 > 2r$ and is given by

$$\mu'_X(r) = \left(\frac{n_2}{n_1}\right)^r \frac{\Gamma(\frac{1}{2}n_1 + r)\Gamma(\frac{1}{2}n_2 - r)}{\Gamma(\frac{1}{2}n_2)} {}_1\widetilde{F}_1(-r; \tfrac{1}{2}n_1; -\tfrac{1}{2}\lambda_1),$$

where ${}_1\widetilde{F}_1(a, b; z)$ denotes Kummer's regularized confluent hypergeometric function.

See also: https://mathworld.wolfram.com/NoncentralF-Distribution.html

See also: Paoella 2, page 358-360

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f_nc(mu, sigma).moments(k))
6.35635234625645256156156145561356E+00
```

dist_fisher_f_nc.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \dots k$, of a random variable X, following a non-central Fisher F distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f_nc(mu, sigma).cumulants(k))
6.35635234625645256156156145561356E+00
```

Additional methods: Confidence intervals and sample size estimates

dist_fisher_f_nc.**nc_ci**(*alpha*, *beta*)

Returns a confidence interval for the noncentrality parameter *lambda*

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", fisher_f_nc(mu, sigma).k_x(t, k))
6.35635234625645256156156145561356E+00
```

Recurrences: Non-central Fisher F, recurrence pdf

ctx.**fisher_f_nc_pdf_recurrence**(*x*, *lambda*, *start_n1*, *start_n2*, *target_n1*, *target_n2*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The following recurrence relations hold for the pdf:

Let the density $g_{m,n}$ be that of m/n times an $F_{m,n}$ random variable. Let $G_{m,n}(y)$ be its distribution function, and let $g^{\lambda}_{m,n}$ and $G^{\lambda}_{m,n}(y)$ be the density and distribution function of its (singly) noncentral version (the distribution of $\chi^2_m(\lambda)/\chi^2_n(0)$). Then the following recurrence relations hold (see Chattamvelli and Jones [175])

Applies a recurrence relation to calculate the cdf for different degrees of freedom for a given value of *x*. This is mostly useful when using asymptotic methods.

$$n\left[G^{\lambda}_{m,n+2}(y) - G^{\lambda}_{m-2,n+2}(y)\right] = -2g^{\lambda}_{m,n}(y)$$

$$\lambda(1+y)g^{\lambda}_{m+4,n}(y) = [\lambda y - m(1+y)]g^{\lambda}_{m+2,n}(y) + y(m+n)g^{\lambda}_{m,n}(y).$$

$$n(1+y)g^{\lambda}_{m,n+2}(y) = (m+n)g^{\lambda}_{m,n}(y) + \lambda g^{\lambda}_{m+2,n}(y).$$

$$\lambda g^{\lambda}_{m+4,n-2}(y) + m g^{\lambda}_{m+2,n-2}(y) = (n-2)y g^{\lambda}_{m,n}(y).$$

Recurrences: Non-central Fisher F, recurrence cdf

`ctx.`**`fisher_f_nc_cdf_recurrence`**(*x*, *lambda*, *start_n1*, *start_n2*, *target_n1*, *target_n2*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Applies a recurrence relation to calculate the cdf for different degrees of freedom for a given value of *x*. This is mostly useful when using asymptotic methods.

From these equations we obtain

$$\begin{aligned} \lambda(1+y)G^{\lambda}_{m+6,n}(y) &= [\lambda y - (m+2-\lambda)(1+y)]G^{\lambda}_{m+4,n}(y) \\ &+ [(m+2)(1+y) + y(m+n-\lambda)]G^{\lambda}_{m+2,n}(y) \\ &- y(m+n)G^{\lambda}_{m,n}(y) \end{aligned} \tag{4.17}$$

$$\begin{aligned} n(1+y) : cite : t : `G^{\lambda}_{m,n+2}(y) - G^{\lambda}_{m+2,n+2}(y)] &= (m+n)G^{\lambda}_{m,n}(y) \\ &+ (\lambda - m - n)G^{\lambda}_{m+2,n}(y) \\ &- \lambda G^{\lambda}_{m+4,n}(y) \end{aligned} \tag{4.18}$$

$$\begin{aligned} (n-2)y : cite : t : `G^{\lambda}_{m,n}(y) - G^{\lambda}_{m+2,n}(y)] &= (m+2)G^{\lambda}_{m+2,n-2}(y) \\ &+ (\lambda - m - 2)G^{\lambda}_{m+4,n-2}(y) \\ &- \lambda G^{\lambda}_{m+6,n-2}(y) \end{aligned} \tag{4.19}$$

Approximations

`ctx.`**`fisher_f_nc_ecf`**(*x*, *m*, *n*, *lambda1*, *results='cdf'*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf. See also: MathWorld [809], Paolella [456], page 358-360.

`ctx.`**`fisher_f_nc_ecf_inv`**(*q*, *m*, *n*, *lambda1 results='qtf'*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

4.9.4 Boost: Noncentral Beta Type I distribution

The following functions return pdf, cdf, qtf or boost class of the noncentral beta distribution with shape parameters a and b, noncentrality parameter λ_1 and the support interval $(0, 1)$.

See also Wikipedia [1163], BoostMath [63], Wang and Gray [793], Johansson [386].

`math53.beta_nc_pdf`(a, b, *lambda1*, x)

Returns pdf(x), the value of the probability density function (*Pdf*) of the noncentral beta distribution:

$$\text{pdf}(x) = f_{\text{Beta}}(x; a, b; \lambda_1) = \frac{a}{b} \int_0^\infty y \cdot f_{\chi^2}\left(x \cdot y \cdot a/b, 2a, \lambda_1\right) \cdot f_{\chi^2}(y, n)\, \mathrm{d}y \tag{4.20}$$

Here $f_{\chi^2}(\cdot, m, \lambda_1)$ denotes the pdf of the noncentral χ^2 distribution with m degrees of freedom and noncentrality parameter λ_1 (see *chi2_nc_pdf()*), and $f_{\chi^2}(\cdot, n)$ denotes the pdf of the central χ^2 distribution with n degrees of freedom (see *chi_squared_pdf()*).

Alternatively, the pdf can be written in a form which shows the relationship to the central distribution more clearly:

$$\text{pdf}(x) = f_{\text{Beta}'}(x; a, b, \lambda_1) = e^{-\lambda_1/2} f_{\text{Beta}}(x; a, b) \times {}_1F_1\left((a+b), b, \tfrac{nx\lambda_1}{2(m+nx)}\right) \tag{4.21}$$

and $f_{\text{Beta}}(\cdot)$ denotes the PDF of the central Beta-distribution, and ${}_1F_1(\cdot)$ is the confluent hypergeometric function.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("BetaNcPdf(x, a, b): ", BetaNcPdf(x, a, b))
>>> print ("dist_beta_nc(a, b).pdf(x): ", dist_beta_nc(a, b).pdf(x))
6.35635234625645256156156145613356E+00
```

`math53.beta_nc_cdf`(a, b, *lambda1*, x)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the noncentral beta distribution:

$$\text{cdf}(x) = F_{\text{Beta}}(x; a, b; \lambda_1) = \frac{az}{b} \int_0^\infty f_{\chi^2}\left(z \cdot y \cdot a/b, 2a; \lambda_1\right) \cdot [1 - F_{\chi^2}(y, 2b)]\, \mathrm{d}y, \quad z = \frac{bx}{a(1-x)} \tag{4.22}$$

Here $f_{\chi^2}(\cdot, m, \lambda_1)$ denotes the pdf of the noncentral χ^2 distribution with m degrees of freedom and noncentrality parameter λ_1 (see *chi2_nc_pdf()*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("BetaNcCdf(x, a, b): ", BetaNcCdf(x, a, b))
>>> print ("dist_beta_nc(a, b).cdf(x): ", dist_beta_nc(a, b).cdf(x))
6.35635234625645256156156145613356E+00
```

`math53.beta_nc_qtf`(a, b, *lambda1*, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the noncentral beta distribution:

There is no known closed exact form for qtf(q). For `fpm.` the default method is to call the function provided by Boost.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("BetaNcQtf(q, a, b): ", BetaNcQtf(q, a, b))
>>> print ("dist_beta_nc(a, b).qtf(q): ", dist_beta_nc(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_beta_nc_type_I**(*a*, *b*, *lambda1*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The noncentral Beta Type I distribution is a continuous probability distribution with shape parameters a and b, noncentrality parameter λ_1 and the support interval $(0, 1)$. See also Wikipedia [1163], MathWorld [245], BoostMath [63], Wang and Gray [793], Witkovský [1473], Johansson [386], R (Statistical System) [515].

A random variable X follows a Type I noncentral beta distribution with shape parameters a and b and noncentrality parameter λ, if it is defined as $X = U_1/(U_1 + U_2)$ where U_1 and U_2 are independent with $U_1 \sim \chi^2(n_1, \lambda)$, and $U_2 \sim \chi^2(n_2)$ $n_1 = 2a$ and $n_2 = 2b$ are the degrees of freedom, and λ is the noncentrality parameter of the noncentral χ^2 distribution. The random variable $Y = 1 - X$ is said to follow a Type II noncentral beta distribution with shape parameters a and b and noncentrality parameter λ.

dist_beta_nc_type_I.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Type I noncentral beta distribution:

$$\mathrm{pdf}_X(x) = f_{\mathrm{Beta}'}(x; a, b, \lambda) = e^{-\lambda/2} f_{\mathrm{Beta}}(x; a, b) \times {}_1F_1\left((a+b), b, \tfrac{nx\lambda}{2(m+nx)}\right)$$

and $f_{\mathrm{Beta}}(\cdot)$ and $F_{\mathrm{Beta}}(\cdot)$ denote the PDF and CDF, respectively, of the central Beta-distribution, and ${}_1F_1(\cdot)$ is the confluent hypergeometric function. There is no known explicit form for the quantile function $\mathrm{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", beta_nc_type_I(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_beta_nc_type_I.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Type I noncentral beta distribution:

$$\mathrm{cdf}_X(x) = F_{\mathrm{Beta}'}(x; a, b, \lambda) = \int_0^x f_{\mathrm{Beta}'}(x; a, b, \lambda)\mathrm{d}t = e^{-\lambda/2} \sum_{j=0}^{\infty} \frac{(\lambda/2)^j}{j!} F_{\mathrm{Beta}}(x; a+j, b)$$

and $f_{\mathrm{Beta}}(\cdot)$ and $F_{\mathrm{Beta}}(\cdot)$ denote the PDF and CDF, respectively, of the central Beta-distribution, and ${}_1F_1(\cdot)$ is the confluent hypergeometric function. There is no known explicit form for the quantile function $\mathrm{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", beta_nc_type_I(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_beta_nc_type_I.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Type I noncentral beta distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^1 \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", beta_nc_type_I(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456 1356E-20
```

dist_beta_nc_type_I.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Type I noncentral beta distribution:

There is no known closed form for the quantile function $\mathrm{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a approximation by Winterbottom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", beta_nc_type_I(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561456 1356E+00
```

dist_beta_nc_type_I.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Type I noncentral beta distribution:

$$\mathrm{isf}_X(q) = \mathrm{qtf}_X(1-q) = \mathrm{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", beta_nc_type_I(mu, sigma).isf(q))
6.356352346256452561561561456 1356E+00
```

dist_beta_nc_type_I.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Type I noncentral beta distribution:

$$C_X(t) = \int_0^1 e^{itx}\mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", beta_nc_type_I(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

dist_beta_nc_type_I.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Type I noncentral beta distribution:

$$M_X(t) = \int_0^1 e^{tx}\mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", beta_nc_type_I(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_beta_nc_type_I.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function:

$$K_X(t) = \log(M_X(t))$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", beta_nc_type_I(mu, sigma).k_x(t, k))
6.3563523462564525615615615614561356E+00
```

dist_beta_nc_type_I.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Type I noncentral beta distribution: the moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_0^1 x^r \text{pdf}_X(x)\text{d}x$$

Returns the raw moments of non-central beta function. See Walck for details.

Algebraic moments of the non-central Beta-distribution are given as (see cite{walck_2007}, p. 109)

$$\text{E}(x^k) = e^{-\lambda/2} \sum_{r=0}^{\infty} \frac{(\lambda/2)^r}{r!} \frac{B(p+r+k, q)}{B(p+r, q)}$$

$$\text{E}(x^k) = \frac{\Gamma(p+k)\Gamma(p+q)}{\Gamma(p)\Gamma(p+q+k)} \times e^{-\lambda/2}{}_2F_2 1(p+q, p+k; p, p+q+k; \lambda/2)$$

Lower order moments are given by

$$\text{E}(Y) = \frac{a}{a+b} \times e^{-\lambda/2}{}_2F_2(a+1, a+b; a, a+b+1; \lambda/2)$$

$$\text{E}(Y^2) = \frac{a(a+1)}{a+b(a+b+1)} \times e^{-\lambda/2}{}_2F_2(a+1, a+b; a, a+b+1; \lambda/2)$$

The moments of $Y = 1 - X$ are given by:

$$\mu'_Y(r) = \frac{\Gamma(n/2+r)\Gamma((n+m)/2)}{\Gamma(n/2)\Gamma((n+m)/2+r)} \times {}_1F_1\left(r; \frac{n+m}{2} + r; -\frac{1}{2}\lambda\right),$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", beta_nc_type_I(mu, sigma).moments(k))
6.3563523462564525615615615614561356E+00
```

dist_beta_nc_type_I.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Type I noncentral beta distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", beta_nc_type_I(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

4.9.5 Noncentral Chi distribution

class ctx.**dist_chi_nc**(*n1*, *n2*, *lambda*, ***kwargs*)

These functions return PDF, CDF, and ICDF of the noncentral chi distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$:

See also: Wikipedia [1216], Witkovský [1491].

dist_chi_nc.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an noncentral chi distribution:

$$\text{pdf}_X(x) = \frac{e^{-(x^2+\lambda^2)/2} x^k \lambda}{(\lambda x)^{k/2}} I_{k/2-1}(\lambda x)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_chi_nc(mu, sigma).pdf(x))
6.356352346256452561561561456l356E-20
```

dist_chi_nc.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an noncentral chi distribution:

$$\text{cdf}_X(x) = 1 - Q_{\frac{k}{2}}(\lambda, x)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_chi_nc(mu, sigma).pdf(x))
6.356352346256452561561561456l356E-20
```

dist_chi_nc.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following an noncentral chi distribution:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x).$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_chi_nc(mu, sigma).pdf(x))
sf: 6.356352346256452561561561456l356E-20
```

dist_chi_nc.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an noncentral chi distribution:

$$\text{qtf}_X(q) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_chi_nc(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_chi_nc.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an noncentral chi distribution:

$$\text{isf}_X(q) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_chi_nc(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_chi_nc.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an noncentral chi distribution:

$$C_X(t) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_chi_nc(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_chi_nc.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following an noncentral chi distribution:

$$M_X(t) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", dist_chi_nc(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_chi_nc.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an noncentral chi distribution:

$$K_X(t) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", dist_chi_nc(mu, sigma).k_x(t, k))
6.3563523462564525615615615614561356E+00
```

dist_chi_nc.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an noncentral chi distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(k) = 2^{k/2}\Gamma(1+k/2)L_{k/2}^{((\nu-2)/2)}\left(-\tfrac{1}{2}\lambda^2\right)$$

$$L_n^{(\alpha)}(x) = \frac{\Gamma(\alpha+n+1)}{\Gamma(\alpha+1)\Gamma(n+1)}{}_1F_1(-n;\alpha+1;x)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_chi_nc(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_chi_nc.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an noncentral chi distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_chi_nc(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.9.6 Rice (Nakagami-n) distribution

class ctx.**dist_rice**(*n1*, *n2*, *lambda*, ***kwargs*)

These functions return PDF, CDF, and ICDF of the Rice distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$.

See also: Wikipedia [1227], Witkovský [1493].

dist_rice.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Rice distribution:

$$\mathrm{pdf}_X(x) = \frac{x}{\sigma^2} \exp\left(\frac{-(x^2+\nu^2)}{2\sigma^2}\right) I_0\left(\frac{x\nu}{\sigma^2}\right)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_rice(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

dist_rice.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Rice distribution:

$$\mathrm{cdf}_X(x) = 1 - Q_1\left(\frac{\nu}{\sigma}, \frac{x}{\sigma}\right)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_rice(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

dist_rice.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Rice distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x).$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_rice(mu, sigma).pdf(x))
sf: 6.356352346256452561561561456135 6E-20
```

dist_rice.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Rice distribution:

$$\mathrm{qtf}_X(q) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_rice(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_rice.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Rice distribution:

$$\text{isf}_X(q) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_rice(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_rice.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Rice distribution:

$$C_X(t) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_rice(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_rice.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following an Rice distribution:

$$M_X(t) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", dist_rice(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_rice.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an Rice distribution:

$$K_X(t) = ??$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", dist_rice(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_rice.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an Rice distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(r) = \sigma^k 2^{k/2}\,\Gamma(1+k/2)\,L_{k/2}(-\nu^2/2\sigma^2),$$

$$L_q(x) = L_q^{(0)}(x) = M(-q, 1, x) = {}_1F_1(-q; 1; x)$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_rice(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_rice.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an Rice distribution. The cumulants are calculated from the moments.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dist_rice(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.9.7 Noncentral distribution of the sample correlation coefficient

class ctx.**dist_pearson_rho_nc**(*N*, *rho*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The noncentral distribution of Pearson's rho (the distribution of the sample correlation coefficient), with sample size $N \geq 3$, noncentrality parameter $\rho \in (-1, +1)$ is a continuous distribution with the support interval $(-1, +1)$. See also Wikipedia [1226], Hotelling [375], Guenther [355], Winterbottom [1454], Winterbottom [1455], Odeh [426], Ruben [493], Subrahmaniam and Subrahmaniam [508].

The correlation coefficient r in samples of size $N > 2$ from a non-singular bivariate normal population with correlation coefficent ρ can be represented in the form

$$\tilde{r} = (z + \tilde{\rho}\chi_{N-1})/\chi_{N-2}$$

where $\tilde{r} = r/\sqrt{1-r^2}$, $\tilde{\rho} = \rho/\sqrt{1-\rho^2}$, z is a standardized normal variate and z , χ_{N-1}, and χ_{N-2} are independent.

A random variable X follows the distribution of Pearson's correlation coefficient with sample size N and noncentrality parameter ρ, if $Y = X/(1-X)$ has the representation given above.

dist_pearson_rho_nc.**pdf**(*x*)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the noncentral distribution of Pearson's rho:

$$\text{pdf}_X(x) = f_R(r, N; \rho) = \frac{(N-2)\Gamma(N-1)}{\sqrt{2\pi}\Gamma\left(N-\frac{1}{2}\right)} A^{N-1} C^{N-4} (1-x)^{\frac{3}{2}-N} {}_2F_1\left(\tfrac{1}{2}, \tfrac{1}{2}; N-\tfrac{1}{2}; \tfrac{1}{2}+\tfrac{1}{2}\rho r\right),$$

where $A = \sqrt{1-\rho^2}$, $\quad C = \sqrt{1-r^2}$, and ${}_2F_1(\cdot)$ is the Gaussian hypergeometric function.

In the special case when $\rho = 0$, the exact density function $f(r)$ can be written as:

$$f(r) = \frac{(1-r^2)^{\frac{n-4}{2}}}{\mathbf{B}\left(\frac{1}{2}, \frac{n-2}{2}\right)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", pearson_rho_nc(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_pearson_rho_nc.**cdf**(*x*)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the noncentral distribution of Pearson's rho:

$$\text{cdf}_X(x) = F_{R^2}(x; p, N, \rho^2) = \int_{-1}^{x} f_{R^2}(x; p, N, \rho^2)\mathrm{d}t$$

Two additional algorithms by Hotelling(1953) are also used to cover a broader range of parameters.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", pearson_rho_nc(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_pearson_rho_nc.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following the noncentral distribution of Pearson's rho:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^1 \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", pearson_rho_nc(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_pearson_rho_nc.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the noncentral distribution of Pearson's rho:

There is no known closed form for the quantile function $\mathrm{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a approximation by Winterbottom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", pearson_rho_nc(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_pearson_rho_nc.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the noncentral distribution of Pearson's rho:

$$\mathrm{isf}_X(q) = \mathrm{qtf}_X(1-q) = \mathrm{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", pearson_rho_nc(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_pearson_rho_nc.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the noncentral distribution of Pearson's rho:

$$C_X(t) = \int_{-1}^1 e^{itx}\mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", pearson_rho_nc(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_pearson_rho_nc.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following the noncentral distribution of Pearson's rho:

$$M_X(t) = \int_{-1}^{1} e^{tx} \text{pdf}_X(x) \text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", pearson_rho_nc(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_pearson_rho_nc.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function:

$$K_X(t) = \log(M_X(t))$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", pearson_rho_nc(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_pearson_rho_nc.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the noncentral distribution of Pearson's rho: the moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_{-1}^{1} x^r \text{pdf}_X(x) \text{d}x$$

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of Pearson's rho:

Johnson *et al.* [388], 2nd vol, page 555, gives the following formulas:

$$\mu_{2k+1} = \frac{B((k+1)/2), (n-2)/2}{B(1/2, (n-2)/2)} (1-\rho^2)^{(n-1)/2} \times {}_3F_2\left(\frac{k+1}{2}, \frac{n-1}{2}, \frac{n-1}{2}, \frac{n+k-1}{2}, \frac{1}{2}; \rho^2\right)$$

$$\mu_{2k} = \frac{(n-2)B((k+2)/2), (n-2)/2}{B(1/2, (n-2)/2)} (1-\rho^2)^{(n-1)/2} \times {}_3F_2\left(\frac{k+2}{2}, \frac{n}{2}, \frac{n}{2}, \frac{n+k}{2}, \frac{3}{2}; \rho^2\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", pearson_rho_nc(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_pearson_rho_nc.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the noncentral distribution of Pearson's rho. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", pearson_rho_nc(mu, sigma).cumulants(k))
6.35635234625645256156156145613356E+00
```

Additional methods: Results based on Fisher's z-transformation

The generating functions and cumulants all exist, but are complicated and not useful for numerical work. Asymptotic expansions typically rely on the Fisher z-transform $Z(a) = \text{atanh}(a)$ and its inverse $Z^{-1}(a) = \tanh(a)$.

Let $m = N - 1$ and let $u_\alpha = \Phi^{-1}(\alpha)$ be the lower 100α percentage point of the standard normal distribution. The first 4 cumulants of $Z(X)$ are given by

$$\kappa_1 = \tfrac{1}{2}\log\left(\frac{1+\rho}{1-\rho}\right) + \frac{\rho}{2m} + \frac{5+\rho^2}{4m^2} + \frac{11+2\rho^2+3\rho^4}{8m^3} + O(m^{-4}),$$

$$\kappa_2 = \frac{1}{m} + \frac{4-\rho^2}{2m^2} + \frac{22-6\rho^2-3\rho^4}{6m^3} + O(m^{-4}), \quad \kappa_3 = \frac{\rho^3}{m^3} + O(m^{-4}), \quad \kappa_4 = \frac{2}{m^3} + O(m^{-4}).$$

4.9.8 Distribution of the logarithm of a noncentral Beta Type II variable

class ctx.**dist_logrv_beta_nc_type_II**(*a*, *b*, *lambda1*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The noncentral Beta Type II distribution is a continuous probability distribution with shape parameters a and b, noncentrality parameter λ_1 and the support interval $(0, 1)$. See also Wikipedia [1163], MathWorld [245], BoostMath [63], Wang and Gray [793], Witkovský [1473], Johansson [386], R (Statistical System) [515].

A random variable X follows a Type I noncentral beta distribution with shape parameters a and b and noncentrality parameter λ, if it is defined as $X = U_1/(U_1 + U_2)$ where U_1 and U_2 are independent with $U_1 \sim \chi^2(n_1, \lambda)$, and $U_2 \sim \chi^2(n_2)$ $n_1 = 2a$ and $n_2 = 2b$ are the degrees of freedom, and λ is the noncentrality parameter of the noncentral χ^2 distribution. The random variable $Y = 1 - X$ is said to follow a Type II noncentral beta distribution with shape parameters a and b and noncentrality parameter λ.

dist_logrv_beta_nc_type_II.**pdf**(*x*)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Type I noncentral beta distribution:

$$\mathrm{pdf}_X(x) = f_{\mathrm{Beta}'}(x; a, b, \lambda) = e^{-\lambda/2} f_{\mathrm{Beta}}(x; a, b) \times {}_1F_1\left((a+b), b, \tfrac{nx\lambda}{2(m+nx)}\right)$$

and $f_{\mathrm{Beta}}(\cdot)$ and $F_{\mathrm{Beta}}(\cdot)$ denote the PDF and CDF, respectively, of the central Beta-distribution, and ${}_1F_1(\cdot)$ is the confluent hypergeometric function. There is no known explicit form for the quantile function $\mathrm{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", beta_nc_type_II(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_logrv_beta_nc_type_II.**cdf**(*x*)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Type I noncentral beta distribution:

$$\mathrm{cdf}_X(x) = F_{\mathrm{Beta}'}(x; a, b, \lambda) = \int_0^x f_{\mathrm{Beta}'}(x; a, b, \lambda)\mathrm{d}t = e^{-\lambda/2} \sum_{j=0}^{\infty} \frac{(\lambda/2)^j}{j!} F_{\mathrm{Beta}}(x; a+j, b)$$

and $f_{\mathrm{Beta}}(\cdot)$ and $F_{\mathrm{Beta}}(\cdot)$ denote the PDF and CDF, respectively, of the central Beta-distribution, and ${}_1F_1(\cdot)$ is the confluent hypergeometric function. There is no known explicit form for the quantile function $\mathrm{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", beta_nc_type_II(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_logrv_beta_nc_type_II.**sf**(*x*)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Type I noncentral beta distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^1 \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", beta_nc_type_II(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_logrv_beta_nc_type_II.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Type I noncentral beta distribution:

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a approximation by Winterbottom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", beta_nc_type_II(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_logrv_beta_nc_type_II.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Type I noncentral beta distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", beta_nc_type_II(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

It is more convenient to work with Y rather than X to obtain generating functions and moments: the characteristic function, $C_Y(t)$, moment generating function, $M_Y(t)$, cumulant generating function and its j^{th} derivative, $K_Y(t)$ and $K_Y^{(j)}(t)$, and the r^{th} raw moment, $\mu'_Y(r)$, of Y are given below:

dist_logrv_beta_nc_type_II.**c_y**(t)

Returns $C_Y(s)$, the characteristic function of a random variable Y, as defined above:

$$C_Y(t) = \frac{\Gamma(n/2 - it)\Gamma((n+m)/2)}{\Gamma(n/2)\Gamma((n+m)/2 - it)} \times {}_1F_1\left(-it; \frac{n+m}{2} - it; -\frac{1}{2}\lambda\right),$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f_nc_type_II(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_logrv_beta_nc_type_II.**m_y**(t)

Returns $M_Y(s)$, the moment generating function of a random variable Y, as defined above:

$$M_Y(t) = \frac{\Gamma(n/2 + t)\Gamma((n+m)/2)}{\Gamma(n/2)\Gamma((n+m)/2 + t)} \times {}_1F_1\left(t, \tfrac{1}{2}(n+m) + t, -\tfrac{1}{2}\lambda\right),$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f_nc_type_II(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_logrv_beta_nc_type_II.**k_y**(t, *k=0*)

Returns $K_Y(s)$, the cumulant generating function, and its j^{th} derivatives, $K_Y^{(j)}(s), j = 1 \ldots k$, of a random variable Y, as defined above:

$$K_Y(t) = \log\left[\frac{\Gamma(n/2+t)\Gamma((n+m)/2)}{\Gamma(n/2)\Gamma((n+m)/2+t)} \times {}_1F_1\left(t, \tfrac{1}{2}(n+m)+t, -\tfrac{1}{2}\lambda\right)\right],$$

$$\begin{aligned} K_Y^{(j)}(t) &= \psi^{(j-1)}\left(\tfrac{1}{2}n + t - \tfrac{1}{2}(i-1)\right) - \psi^{(j-1)}\left(\tfrac{1}{2}(n+m) + t - \tfrac{1}{2}(i-1)\right) && (4.23) \\ &+ \frac{\partial}{\partial t}\log\left[{}_1F_1\left(t, \tfrac{1}{2}(n+m)+t, -\tfrac{1}{2}\lambda\right)\right], && (4.24) \end{aligned}$$

where $\psi^{(j-1)}(\cdot)$ denotes the polygamma function and ${}_1F_1(\cdot)$ is the confluent hypergeometric function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", fisher_f_nc_type_II(mu, sigma).k_x(t, k))
6.356352346256452561561561456135 6E+00
```

dist_logrv_beta_nc_type_II.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the square of the multiple correlation coefficient: the moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_0^1 x^r \text{pdf}_X(x)\text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_r_square_type_II(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_logrv_beta_nc_type_II.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of the square of the multiple correlation coefficient. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_r_square_type_II(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

Approximations

ctx.**log1mbeta_nc_gp**(*t*, *n*, *m*, *lambda1*)

where ctx is dec, mpm, fpm, gmp or apm.

Calculates the pdf, cdf and sf from the characteristic function using the procedure of Gil-Pelaez (see *gil_pelaez_pdf()* and *gil_pelaez_cdf()*).

This uses $\log(1-X)$, X noncentral beta

ctx.**beta_nc_spa**(*x*, *n*, *results='c'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

This uses $2\log(1-X)$ of the non-central beta X

Returns $K_Y(s)$, the cumulant generating function, and its j^{th} derivatives, $K_Y^{(j)}(s), j = 1 \ldots k$, of a random variable Y, as defined above:

$$K_Y(t) = \log\left[\frac{\Gamma(n/2+t)\Gamma((n+m)/2)}{\Gamma(n/2)\Gamma((n+m)/2+t)} \times {}_1F_1\left(t, \tfrac{1}{2}(n+m)+t, -\tfrac{1}{2}\lambda\right)\right],$$

$$\begin{aligned} K_Y^{(j)}(t) &= \psi^{(j-1)}\left(\tfrac{1}{2}n + t - \tfrac{1}{2}(i-1)\right) - \psi^{(j-1)}\left(\tfrac{1}{2}(n+m) + t - \tfrac{1}{2}(i-1)\right) && (4.25)\\ &+ \frac{\partial}{\partial t}\log\left[{}_1F_1\left(t, \tfrac{1}{2}(n+m)+t, -\tfrac{1}{2}\lambda\right)\right], && (4.26)\end{aligned}$$

where $\psi^{(j-1)}(\cdot)$ denotes the polygamma function and ${}_1F_1(\cdot)$ is the confluent hypergeometric function.

ctx.**beta_nc_spa_inv**(*x*, *n*, *results='qtf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

This uses $2\log(1-X)$ of the non-central beta X

4.9.9 Noncentral distribution (Type I) of Fisher's R^2

class ctx.**dist_fisher_r2_type_I**(*p*, *N*, *rho2*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Fisher's R^2 Type I (the distribution of the square of the sample multiple correlation coefficient), with $p \geq 1$ predictor variables, sample size $N \geq p+2$ and noncentrality parameter $\rho^2 \in (0,1)$ is a continuous probability distribution with the support interval $(0,1)$. See also Lee [407], Lee [408], Gurland [357], Gurland and Milton [359], Gurland and Asiribo [358], Muirhead [418], Benton and Krishnamoorthy [30], Fisher [336], Gatsonis and Sampson [343], Kelley [392].

A random variable X follows the distribution of Fisher's R^2 (the square of the multiple correlation coefficient) with p variables, sample size N and noncentrality parameter ρ^2, if $G = X/(1-X)$ has the representation

$$G = \frac{(\tilde{\rho}\chi_{N-1} + z)^2 + \chi^2_{p-1}}{\chi^2_{N-1-p}},$$

where $\tilde{\rho} = \sqrt{\rho^2/(1-\rho^2)}$, z is a standard normal variate, χ_f and χ^2_f are chi and chi-square variates on f degrees of freedom, and the variates figuring in this relation are independently distributed.

dist_fisher_r2_type_I.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the square of the multiple correlation coefficient:

$$\mathrm{pdf}_X(x) = f_{R^2}(x; p, N, \rho^2) = f_{\text{Beta}}\left(x; \tfrac{1}{2}(p-1), \tfrac{1}{2}(N-p)\right) \times (1-\rho^2)^{n/2} \times {}_2F_1(\tfrac{1}{2}N, \tfrac{1}{2}N, \tfrac{1}{2}p; \rho^2 x),$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_r_square_type_I(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_fisher_r2_type_I.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the square of the multiple correlation coefficient:

$$\begin{aligned} \mathrm{cdf}_X(x) &= F_{R^2}(x; p, N, \rho^2) = \int_0^x f_{R^2}(x; p, N, \rho^2)\mathrm{d}t \\ &= \sum_{i=0}^{\infty} f_{\text{NegBin}}\left((N-1)/2, i; 1-\rho^2\right) \times F_{\text{Beta}}\left(x; \tfrac{1}{2}(p-1)+i, \tfrac{1}{2}(N-p)\right), \end{aligned} \tag{4.27}$$

and $f_{\text{Beta}}(\cdot)$ and $F_{\text{Beta}}(\cdot)$ denote the PDF and CDF, respectively, of the central Beta-distribution, $f_{\text{NegBin}}(\cdot)$ denotes the PMF of the negative binomial distribution and and ${}_2F_1(\cdot)$ is the Gaussian hypergeometric function. There is no known explicit form for the quantile function $\mathrm{cdf}^{-1}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_r_square_type_I(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_fisher_r2_type_I.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the square of the multiple correlation coefficient:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^1 \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_r_square_type_I(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_fisher_r2_type_I.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of the square of the multiple correlation coefficient:

There is no known explicit form for the quantile function $\mathrm{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_r_square_type_I(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_fisher_r2_type_I.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the square of the multiple correlation coefficient:

$$\mathrm{isf}_X(q) = \mathrm{qtf}_X(1-q) = \mathrm{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_r_square_type_I(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_fisher_r2_type_I.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the square of the multiple correlation coefficient:

$$C_X(t) = \int_0^1 e^{itx}\mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_r_square_type_I(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_fisher_r2_type_I.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of the square of the multiple correlation coefficient:

$$M_X(t) = \int_0^1 e^{tx} \text{pdf}_X(x) \text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_r_square_type_I(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_fisher_r2_type_I.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function:

$$K_X(t) = \log(M_X(t))$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", fisher_r_square_type_I(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_fisher_r2_type_I.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the square of the multiple correlation coefficient: the moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_0^1 x^r \text{pdf}_X(x) \text{d}x$$

The moments of $Y = 1 - X$ are given by:

$$\mu'_Y(r) = \frac{\Gamma(n/2 + r)\Gamma((n + m)/2)}{\Gamma(n/2)\Gamma((n + m)/2 + r)} \times {}_2F_1\left(r; \frac{n + m}{2} + r; -\tfrac{1}{2}\rho^2\right),$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_r_square_type_I(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_fisher_r2_type_I.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of the square of the multiple correlation coefficient. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_r_square_type_I(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Additional information: Moments of Fisher's `R^2`

[Muirhead1982` p. 178, gives the following formula (see also Johnson *et al.* [388] p. 621)

$$\mathrm{E}[(1-R^2)^h] = \frac{\left[\frac{1}{2}(n-m+1)\right]_h}{\left(\frac{1}{2}n\right)_h}(1-\bar{R}^2)^h \times {}_2F_1(h,h,\tfrac{1}{2}n+h;\bar{R}^2).$$

where $(a)_k$ is the Pochammer symbol and ${}_2F_1(\cdot)$ is the Gaussian hypergeometric function.

The lower order moments are given by

$$\mu_1' = 1 - \frac{(n-m+1)}{n}(1-\bar{R}^2)^h \times {}_2F_1(1,1,\tfrac{1}{2}n+1;\bar{R}^2).$$

$$\mathrm{E}[(1-R^2)^2] = \frac{\left[\frac{1}{2}(n-m+1)\right]_2}{\left(\frac{1}{2}n\right)_2}(1-\bar{R}^2)^2 \times {}_2F_1(2,2,\tfrac{1}{2}n+2;\bar{R}^2).$$

Additional information: Random number generation for Fisher's `R^2`

A notable result concerning the distribution of R^2 is that $\tilde{R}^2 = R^2/(1-R^2)$ has the representation

$$\tilde{R}^2 = \frac{(\tilde{\rho}\chi_n + z)^2 + \chi^2_{p-1}}{\chi^2_{n-p}}$$

where $\tilde{\rho}^2 = \rho^2/(1-\rho^2)$, n is the sample size less one, z is a standard normal variate, χ_f and χ^2_f are chi and chi-square variates on f degrees of freedem; the variates figuring in this relation are independently distributed and $\tilde{\rho}$ is taken to be positive (see Lee [407]).

Additional information: Recurrence relations for the pdf

For the particular cases of $p = 3$ and $p = 5$, we have for the pdf (see Lee [407]):

$$f_{R^2}(n,3;\rho^2) = \frac{(n-3)\sqrt{1-\rho^2}\,[f_R(n-1,R;\rho) - f_R(n-1,-R;\rho)]}{2(n-2)\rho\sqrt{1-R^2}}$$

$$\begin{aligned} f_{R^2}(n,5;\rho^2) &= \frac{(n-5)(1-\rho^2)R\,[f_R(n-2,R;\rho) + f_R(n-2,-R;\rho)]}{2(n-2)\rho^2(1-R^2)} \\ &- \frac{2(1-\rho^2)(1-R^2)f_{R^2}(n-2,3;\rho^2)}{2(n-2)\rho^2(1-R^2)} \end{aligned} \tag{4.28}$$

where $f_R(\cdot)$ denotes the pdf of the distribution of the sample correlation coeffcient. Starting with these formulas, the pdf can be calculated for all odd p, using the following recurrence relation, which is valid for all integer values of $p \geq 6$:

$$\begin{aligned} (n-2)\rho^2(1-R^2)f_{R^2}(n,p;\rho^2) &= (n-p)R^2(1-\rho^2)f_{R^2}(n-2,p-4;\rho^2) \\ &- (p-4)(1-\rho^2)(1-R^2)f_{R^2}(n-2,p-2;\rho^2) \end{aligned} \tag{4.29}$$

Additional information: Recurrence relations for the cdf

For the particular cases of $p = 3$ and $p = 5$, we have for the CDF Lee [407]:

$$F_{R^2}(n,3;\rho^2) = F_{R^2}(n,1;\rho^2) - \frac{\sqrt{(1-\rho^2)(1-R^2)}\,[f_R(n-1,R;\rho) - f_R(n-1,-R;\rho)]}{(n-2)\rho}$$

$$\begin{aligned} F_{R^2}(n,5;\rho^2) &= F_{R^2}(n,3;\rho^2) - \frac{(1-\rho^2)R\,[f_R(n-2,R;\rho) - f_R(n-2,-R;\rho)]}{(n-2)\rho} \\ &- \frac{(1-\rho^2)\left[F_{R^2}(n-2,3;\rho^2) - F_{R^2}(n-2,1;\rho^2)\right]}{(n-2)\rho} \end{aligned} \tag{4.30}$$

where $F_R(\cdot)$ denotes the CDF of the distribution of the sample correlation coeffcient. Starting with these formulas, the CDF can be calculated for all odd p, using the following recurrence relations for the cdf, which is valid for all integer values of $p \geq 6$:

$$\begin{aligned}(n-2)\rho^2(1-R^2)F_{R^2}(n,p;\rho^2) &= (n-p)\rho^2 F_{R^2}(n,p-2;\rho^2) \\ &- (p-4)(1-\rho^2)\left[F_{R^2}(n-2,p-2;\rho^2) - F_{R^2}(n-2,p-4;\rho^2)\right] \\ &- 2R^2(1-\rho^2)f_{R^2}(n-2,p-4;\rho^2)\end{aligned} \tag{4.31}$$

4.9.10 Distribution of the logarithm of a noncentral Fisher $1 - R^2$ variable

class ctx.**dist_logrv_fisher_1mr2**(*p*, *N*, *rho2*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Fisher's R^2 Type II with $p \geq 1$ predictor variables, sample size $N \geq p + 2$ and noncentrality parameter $\rho^2 \in (0, 1)$ is a continuous probability distribution with the support interval $(0, 1)$. See also Lee [407], Lee [408], Gurland [357], Gurland and Milton [359], Gurland and Asiribo [358], Muirhead [418], Benton and Krishnamoorthy [30], Fisher [336], Gatsonis and Sampson [343].

A random variable X follows the distribution of Fisher's R^2 (the square of the multiple correlation coefficient) with p variables, sample size N and noncentrality parameter ρ^2, if $G = X/(1 - X)$ has the representation

$$G = \frac{(\tilde{\rho}\chi_{N-1} + z)^2 + \chi^2_{p-1}}{\chi^2_{N-1-p}},$$

where $\tilde{\rho} = \sqrt{\rho^2/(1 - \rho^2)}$, z is a standard normal variate, χ_f and χ^2_f are chi and chi-square variates on f degrees of freedom, and the variates figuring in this relation are independently distributed.

dist_logrv_fisher_1mr2.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the square of the multiple correlation coefficient:

$$\mathrm{pdf}_X(x) = f_{R^2}(x; p, N, \rho^2) = f_{\text{Beta}}\left(x; \tfrac{1}{2}(p-1), \tfrac{1}{2}(N-p)\right) \times (1-\rho^2)^{n/2} \times {}_2F_1(\tfrac{1}{2}N, \tfrac{1}{2}N, \tfrac{1}{2}p; \rho^2 x),$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_r_square_type_II(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_logrv_fisher_1mr2.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the square of the multiple correlation coefficient:

$$\begin{aligned} \mathrm{cdf}_X(x) &= F_{R^2}(x; p, N, \rho^2) = \int_0^x f_{R^2}(x; p, N, \rho^2)\mathrm{d}t \\ &= \sum_{i=0}^{\infty} f_{\text{NegBin}}\left((N-1)/2, i; 1-\rho^2\right) \times F_{\text{Beta}}\left(x; \tfrac{1}{2}(p-1)+i, \tfrac{1}{2}(N-p)\right), \end{aligned} \tag{4.32}$$

and $f_{\text{Beta}}(\cdot)$ and $F_{\text{Beta}}(\cdot)$ denote the PDF and CDF, respectively, of the central Beta-distribution, $f_{\text{NegBin}}(\cdot)$ denotes the PMF of the negative binomial distribution and and ${}_2F_1(\cdot)$ is the Gaussian hypergeometric function. There is no known explicit form for the quantile function $\mathrm{cdf}^{-1}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_r_square_type_II(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_logrv_fisher_1mr2.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the square of the multiple correlation coefficient:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^1 \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_r_square_type_II(mu, sigma).pdf(x))
sf: 6.35635234625645256156156156145613 56E-20
```

dist_logrv_fisher_1mr2.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of the square of the multiple correlation coefficient:

There is no known explicit form for the quantile function $\mathrm{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_r_square_type_II(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156156145613 56E+00
```

dist_logrv_fisher_1mr2.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the square of the multiple correlation coefficient:

$$\mathrm{isf}_X(q) = \mathrm{qtf}_X(1-q) = \mathrm{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_r_square_type_II(mu, sigma).isf(q))
6.35635234625645256156156156145613 56E+00
```

It is more convenient to work with $Y = 1 - X$ rather than X to obtain generating functions and moments: the characteristic function, $C_Y(t)$, moment generating function, $M_Y(t)$, cumulant generating function and its j^{th} derivative, $K_Y(t)$ and $K_Y^{(j)}(t)$, and the r^{th} raw moment, $\mu'_Y(r)$, of Y are given below:

dist_logrv_fisher_1mr2.**c_y**(t)

Returns $C_Y(s)$, the characteristic function of a random variable Y (as defined above):

$$C_Y(t) = \frac{\Gamma(n/2 - it)\Gamma((n+m)/2)}{\Gamma(n/2)\Gamma((n+m)/2 - it)} \times {}_2F_1\left(-it; \frac{n+m}{2} - it; -\frac{1}{2}\rho^2\right),$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f_nc(mu, sigma).c_x(t))
6.35635234625645256156156156145613 56E+00
```

dist_logrv_fisher_1mr2.**m_y**(t)

Returns $M_Y(s)$, the moment generating function of a random variable Y (as defined above):

$$M_Y(t) = \frac{\Gamma(n/2+t)\Gamma((n+m)/2)}{\Gamma(n/2)\Gamma((n+m)/2+t)} \times {}_2F_1\left(t, \tfrac{1}{2}(n+m)+t, -\tfrac{1}{2}\rho^2\right),$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f_nc(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_logrv_fisher_1mr2.**k_y**(t, *k=0*)

Returns $K_Y(s)$, the cumulant generating function, and its j^{th} derivatives, $K_Y^{(j)}(s), j = 1 \ldots k$, of a random variable Y (as defined above):

$$K_Y(t) = \log\left[\frac{\Gamma(n/2+t)\Gamma((n+m)/2)}{\Gamma(n/2)\Gamma((n+m)/2+t)} \times {}_2F_1\left(t, \tfrac{1}{2}(n+m)+t, -\tfrac{1}{2}\rho^2\right)\right],$$

$$K_Y^{(j)}(t) = \psi^{(j-1)}\left(\tfrac{1}{2}n + t - \tfrac{1}{2}(i-1)\right) - \psi^{(j-1)}\left(\tfrac{1}{2}(n+m) + t - \tfrac{1}{2}(i-1)\right) \tag{4.33}$$

$$+ \frac{\partial}{\partial t}\log\left[{}_2F_1\left(t, \tfrac{1}{2}(n+m)+t, -\tfrac{1}{2}\rho^2\right)\right], \tag{4.34}$$

where $\psi^{(j-1)}(\cdot)$ denotes the polygamma function and ${}_2F_1(\cdot)$ is the Gaussian hypergeometric function. The solution $\hat{s}(y)$ of the saddlepoint equation $K_Y^{(1)}(\hat{s}(y)) = y$ needs to be determined numerically.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", fisher_f_nc(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_logrv_fisher_1mr2.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the square of the multiple correlation coefficient: the moments are calculated from their definition:

$$\mu'_X(r) = E(X^r) = \int_0^1 x^r \text{pdf}_X(x)\text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_r_square_type_II(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_logrv_fisher_1mr2.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of the square of the multiple correlation coefficient. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_r_square_type_II(mu, sigma).cumulants(k))
6.35635234625645256156156145561356E+00
```

Approximations

`ctx.`**`fisher_log1mr2_gp`**(*t*, *p*, *N*, *rho2*)

where `ctx` is `dec`, `mpm`, `fpm`, `gmp` or `apm`.

Calculates the pdf, cdf and sf from the characteristic function using the procedure of Gil-Pelaez (see *gil_pelaez_pdf()* and *gil_pelaez_cdf()*).

This uses $Y = \log(1 - R^2)$

`ctx.`**`fisher_r2_spa`**(*x*, *n*, *results='c'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

This uses $Y = \log(1 - R^2)$

`ctx.`**`fisher_r2_spa_inv`**(*x*, *n*, *results='qtf'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

This uses $Y = \log(1 - R^2)$

4.9.11 Doubly non-central Student t distribution

class ctx.**dist_student_t_2nc**(*n*, *delta*, *theta*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The doubly non-central Student t distribution is a continuous probability distribution with $n > 0$ degrees of freedom, noncentrality parameters δ and θ, and support interval $(-\infty, +\infty)$. See also Broda and Paolella [160], Kocherlakota and Kocherlakota [394], Paolella [455], Paolella [456], Gessner [348],

dist_student_t_2nc.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a doubly non-central Student t distribution:

$$\mathrm{pdf}_X(x) = f_{t''}(t; n; \mu, \theta) = \sum_{i=0}^{\infty} \omega_{i,\theta} s_{i,n} f_{t'}(s_{i,n} x; n + 2i, \mu),$$

where $f_{t'}(\cdot)$ denotes the PDF of the singly noncentral t-distribution, and

$$\omega_{i,\theta} = \frac{\exp(-\theta/2)(\theta/2)^i}{i!} \quad \text{and} \quad s_{i,n} = \sqrt{\frac{n+2i}{n}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", student_t_2nc(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_student_t_2nc.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a doubly non-central Student t distribution:

$$\mathrm{cdf}_X(x) = F_{t''}(x; n; \mu, \theta) = \int_0^x f_{t''}(x; n; \mu, \theta)\mathrm{d}t = \sum_{i=0}^{\infty} \omega_{i,\theta} s_{i,n} F_{t'}(s_{i,n} x; n + 2i, \mu),$$

where $F_{t'}(\cdot)$ denotes the CDF of the singly noncentral t-distribution, and

$$\omega_{i,\theta} = \frac{\exp(-\theta/2)(\theta/2)^i}{i!} \quad \text{and} \quad s_{i,n} = \sqrt{\frac{n+2i}{n}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", student_t_2nc(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_student_t_2nc.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a doubly non-central Student t distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^{\infty} \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", student_t_2nc(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_student_t_2nc.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a doubly non-central Student t distribution:

There is no known explicit form for the quantile function $\text{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a singly noncentral t approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", student_t_2nc(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_student_t_2nc.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a doubly non-central Student t distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", student_t_2nc(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_student_t_2nc.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a doubly non-central Student t distribution:

$$C_X(t) = \int_{-\infty}^{\infty} e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", student_t_2nc(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_student_t_2nc.**m_x**(t)

Returns `NaN`, since the moment generating function does not exist.

dist_student_t_2nc.**k_x**(t, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

`dist_student_t_2nc.`**`moments`**`(`k`)`

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a doubly non-central Student t distribution. The rth moment only exists for $n > r$ and is given by

$$\mu'_X(r) = \left(\tfrac{1}{2}n\right)^{r/2} \frac{\Gamma\left(\frac{1}{2}(n-r)\right)}{\Gamma\left(\frac{1}{2}n\right)} \times {}_1F_1\left(\tfrac{1}{2}r, \tfrac{1}{2}n, -\tfrac{1}{2}\theta\right) \times \sum_{i=0}^{\lfloor r/2 \rfloor} \binom{r}{2i} \frac{(2i)!}{2^i i!} \delta^{r-2i},$$

where ${}_1F_1(\cdot)$ denotes Kummer's confluent hypergeometric function.

See also: Paoella 2, page 381-382

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", student_t_2nc(mu, sigma).moments(k))
6.35635234625645256156156145613356E+00
```

`dist_student_t_2nc.`**`cumulants`**`(`k`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a doubly non-central Student t distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", student_t_2nc(mu, sigma).cumulants(k))
6.35635234625645256156156145613356E+00
```

Approximations

`ctx.`**`student_t_nc2_ecf`**`(`*x*, *n*, *delta*, *theta*, *results='cdf'*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf. See also: Paolella [456], page 381-382.

`ctx.`**`student_t_nc2_ecf_inv`**`(`*q*, *n*, *delta*, *theta*, *results='qtf'*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

4.9.12 Doubly non-central Fisher F distribution

class ctx.**dist_fisher_f_2nc**(*m*, *n*, *lambda1*, *lambda2*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The doubly non-central Fisher F distribution is a continuous probability distribution with $m > 0$ and $n > 0$ degrees of freedom, noncentrality parameters $\lambda_1 \geq 0$ and $\lambda_2 \geq 0$, and the support interval $(0, +\infty)$. See also Butler and Paolella [166], Chou [179], Chattamvelli and Jones [175].

dist_fisher_f_2nc.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a doubly non-central Fisher F distribution:

$$\mathrm{pdf}_X(x) = f_{F''}(x; n_1, n_2; \lambda_1, \lambda_2) = \sum_{i=0}^{\infty} \omega_{i,\lambda_2} s_{i,n} f_{F'}(s_{i,n}x; n + 2i, \lambda_1),$$

where $f_{F'}(\cdot)$ denotes the PDF of the singly noncentral F-distribution, and

$$\omega_{i,\lambda_2} = \frac{\exp(-\lambda_2/2)(\lambda_2/2)^i}{i!} \quad \text{and} \quad s_{i,n} = \sqrt{\frac{n+2i}{n}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f_2nc(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_fisher_f_2nc.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a doubly non-central Fisher F distribution:

$$\mathrm{cdf}_X(x) = F_{F''}(x; n_1, n_2; \lambda_1, \lambda_2) = \sum_{i=0}^{\infty} \omega_{i,\theta} s_{i,n} F_{F'}(s_{i,n_2}x; n_1 + 2i, n_2; \lambda_1),$$

where $F_{F'}(\cdot)$ denotes the CDF of the singly noncentral F-distribution, and

$$\omega_{i,\lambda_2} = \frac{\exp(-\lambda_2/2)(\lambda_2/2)^i}{i!} \quad \text{and} \quad s_{i,n} = \sqrt{\frac{n+2i}{n}}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f_2nc(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_fisher_f_2nc.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a doubly non-central Fisher F distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^{\infty} \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f_2nc(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_fisher_f_2nc.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a doubly non-central Fisher F distribution:

There is no known explicit form for the quantile function $\text{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f_2nc(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_fisher_f_2nc.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a doubly non-central Fisher F distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f_2nc(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_fisher_f_2nc.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a doubly non-central Fisher F distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", fisher_f_2nc(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_fisher_f_2nc.**m_x**(t)

Returns `NaN`, since the moment generating function does not exist.

dist_fisher_f_2nc.**k_x**(t, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_fisher_f_2nc.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a doubly non-central Fisher F distribution. The rth moment only exists for $n_2 > 2r$, and is given by

$$\mu'_X(r) = \left(\frac{n_2}{n_1}\right)^r \Gamma(\tfrac{1}{2}n_1 + r)\Gamma(\tfrac{1}{2}n_2 - r) \times {}_1\widetilde{F}_1(-r; \tfrac{1}{2}n_1; -\tfrac{1}{2}\lambda_1) \times {}_1\widetilde{F}_1(r; \tfrac{1}{2}n_2; -\tfrac{1}{2}\lambda_2),$$

where ${}_1\widetilde{F}_1(a, b; z)$ denotes Kummer's regularized confluent hypergeometric function.

See also: https://mathworld.wolfram.com/NoncentralF-Distribution.html

See also: Paoella 2, page 358-360

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f_2nc(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_fisher_f_2nc.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a doubly non-central Fisher F distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", fisher_f_2nc(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Additional methods: confidence intervals and sample size estimates

dist_fisher_f_2nc.**nc_ci**(*alpha*, *beta*)

Returns a confidence interval for the noncentrality parameter *lambda*

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", fisher_f_2nc(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

Recurrences: Doubly non-central Fisher F, recurrence pdf

ctx.**fisher_f_nc2_pdf_recurrence**(*x*, *lambda*, *start_n1*, *start_n2*, *target_n1*, *target_n2*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Applies a recurrence relation to calculate the cdf for different degrees of freedom for a given value of x. This is mostly useful when using asymptotic methods.

Let the density $g_{m,n}$ be that of m/n times an $F_{m,n}$ random variable. Let $G_{m,n}(y)$ be its distribution function, and let $g^{\lambda_1,\lambda_2}_{m,n}$ and $G^{\lambda_1,\lambda_2}_{m,n}(y)$ be the density and distribution function of its doubly noncentral version (the distribution of $\chi^2_m(\lambda_1)/\chi^2_n(\lambda_2)$). Then the following recurrence relations hold (see Chattamvelli and Jones [175])

$$n\left[G^{\lambda}_{m,n}(y) - G^{\lambda}_{m,n}(y)\right] = -2g^{\lambda}_{m,n}(y)$$

$$\lambda_1(1+y)g^{\lambda_1,\lambda_2}_{m+4,n}(y) = [\lambda_1 y - m(1+y)]g^{\lambda_1,\lambda_2}_{m+2,n}(y) + y(m+n)g^{\lambda_1,\lambda_2}_{m,n}(y) + \lambda_2 g^{\lambda_1,\lambda_2}_{m,n+2}(y).$$

$$\lambda_2(1+y)g^{\lambda_1,\lambda_2}_{m,n+4}(y) = [\lambda_2 y - n(1+y)]g^{\lambda_1,\lambda_2}_{m+2,n}(y) + (m+n)g^{\lambda_1,\lambda_2}_{m,n}(y) + \lambda_1 g^{\lambda_1,\lambda_2}_{m+2,n}(y).$$

$$\lambda_1 g^{\lambda_1,\lambda_2}_{m+4,n-2}(y) = -mg^{\lambda_1,\lambda_2}_{m+2,n}(y) + nyg^{\lambda_1,\lambda_2}_{m,n+2}(y) + \lambda_2 g^{\lambda_1,\lambda_2}_{m+2,n}(y).$$

Recurrences: Doubly non-central Fisher F, recurrence cdf

`ctx.fisher_f_nc2_cdf_recurrence`(*x*, *lambda*, *start_n1*, *start_n2*, *target_n1*, *target_n2*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Applies a recurrence relation to calculate the cdf for different degrees of freedom for a given value of *x*. This is mostly useful when using asymptotic methods.

$$\begin{aligned}\lambda_1(1+y)G^{\lambda_1,\lambda_2}_{m+6,n}(y) &= [\lambda_1 y-(m+2-\lambda_1)(1+y)]G^{\lambda_1,\lambda_2}_{m+4,n}(y)\\ &+ [(m+2)(1+y)+y(m+n-\lambda_1)]G^{\lambda_1,\lambda_2}_{m+2,n}(y)\\ &- y(m+n)G^{\lambda_1,\lambda_2}_{m,n}(y)+\lambda_2 y:cite:t:`G^{\lambda_1,\lambda_2}_{m+2,n+2}(y)-G^{\lambda_1,\lambda_2}_{m,n+2}(y)]\end{aligned} \tag{4.35}$$

$$\begin{aligned}[n(1+y)-\lambda_2]:cite:t:`G^{\lambda_1,\lambda_2}_{m,n+2}(y)-G^{\lambda_1,\lambda_2}_{m+2,n+2}(y) &= (m+n):cite:t:`G^{\lambda_1,\lambda_2}_{m+2,n}(y)-G^{\lambda_1,\lambda_2}_{m,n}(y)]\\ &+ \lambda_2(1+y)[G^{\lambda_1,\lambda_2}_{m+2,n}(y)-G^{\lambda_1,\lambda_2}_{m+2,n}(y)]\\ &- \lambda_1:cite:t:`G^{\lambda_1,\lambda_2}_{m+2,n}(y)-G^{\lambda_1,\lambda_2}_{m+4,n}(y)]\end{aligned} \tag{4.36}$$

$$\begin{aligned}(m+2):cite:t:`G^{\lambda_1,\lambda_2}_{m+2,n}(y) &= (\lambda_1-m-2)G^{\lambda_1,\lambda_2}_{m+4,n}(y)-\lambda_1 G^{\lambda_1,\lambda_2}_{m+6,n}(y)\\ &+ y[nG^{\lambda_1,\lambda_2}_{m,n}(y)-nG^{\lambda_1,\lambda_2}_{m+2,n}(y)\\ &- \lambda_2 G^{\lambda_1,\lambda_2}_{m,n}(y)-\lambda_2 G^{\lambda_1,\lambda_2}_{m+2,n}(y)]\end{aligned} \tag{4.37}$$

Approximations

`ctx.fisher_f_nc2_ecf`(*x*, *m*, *n*, *lambda1*, *lambda2*, *results='cdf'*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf. See also: MathWorld [809], Paolella [456], page 358-360.

`ctx.fisher_f_nc2_ecf_inv`(*q*, *m*, *n*, *lambda1*, *lambda2*, *results='qtf'*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

`ctx.fisher_f_nc2_spa`(*x*, *m*, *n*, *lambda1*, *lambda2*, *results='cdf'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

A random variable X follows a doubly noncentral F-distribution with n_1 and n_2 degrees of freedom and noncentrality parameters λ_1 and λ_2, if it is defined as $X = (U_1/n_1)/(U_2/n_2)$, where U_1 and U_2 are independent with $U_i \sim \chi^2(n_i, \lambda_i)$, the n_i are the degrees of freedom, and the λ_i are the noncentrality parameters of the noncentral χ^2 distributions.

The moment generating function and the cumulant generating function of X do not exist. However, the CDF of X can also be computed by writing

$$\text{cdf}_X(x) = \Pr(X \le x) = \Pr\left(\frac{n_2}{n_1}U_1 - xU_2 \le 0\right) = \Pr\left(Y_x \le 0\right),$$

where Y_x is the so-defined linear combination of U_1 and U_2, and then using the inversion methods for weighted sums. In the following equations, we use these definitions:

$l_1 = n_2/n_1$, $l_2 = -x$, $v_1 = 1/(1-2sl_1)$, $v_2 = 1/(1-2sl_2)$, $g_1 = l_1 v_1$, $g_2 = l_2 v_2$, $c_j = 1$ for $j = 1$ and $c_j = 2(j-1)c_{j-1}$ for $j > 1$.

$K_{Y_x}(s)$, the cumulant generating function, and its j^{th} derivatives, $K^{(j)}_{Y_x}(s), j = 1 \ldots k$, of a random variable Y_x, are defined as:

$$K_{Y_x}(s) = \tfrac{1}{2}n_1\log(v_1) + \tfrac{1}{2}n_2\log(v_2) + s(\lambda_1 g_1 + \lambda_2 g_2)$$

$$K_{Y_x}^{(j)}(s) = c_j \left(g_1^j \left(n_1 + j\lambda_1 v_1 \right) + g_2^j \left(n_2 + j\lambda_2 v_2 \right) \right).$$

saddlepoint central

Returns the solution $\hat{s}$ of the saddlepoint equation $K_{Y_x}^{(1)}(\hat{s}) = 0$, which can be obtained in closed form

$$\hat{s_0} = \frac{n_1(f-1)}{2f(n_1+n_2}$$

saddlepoint noncentral

Returns the solution $\hat{s}$ of the saddlepoint equation $K_{Y_x}^{(1)}(\hat{s}) = 0$, which can be obtained in closed form

$$\hat{s_1} = \left(xn_1(n_1 + 2n_2 + \lambda) - n_1 n_2 - \sqrt{n_1 a_2} \right) / a_1, \quad \text{where } a_1 = 4n_2 x(n1 + n2),$$

$$a_2 = x^2 n_1^3 + 2x^2 n_1^2 \lambda + 2n_1^2 x n_2 + 4x^2 n_1 n_2 \lambda + n_1 \lambda^2 x^2 + 2n_1 \lambda x n_2 + n_2^2 n_1 + 4x n_2^2 \lambda.$$

saddlepoint doubly noncentral

The solution $\hat{s}$ of the saddlepoint equation $K_{Y_x}^{(1)}(\hat{s}) = 0$, which can be obtained in closed form

$$\hat{s_2} = -2p \cos\left(\tfrac{1}{3} \arccos(-\tfrac{1}{2} q p^{-3}) + \tfrac{1}{3}\pi \right) - a_2, \quad \text{where}$$

$p^2 = |3a_2^2 - a_1|/3$, $q = a_2(2a_2^2 - a_1) + a_0$, $a = 8x^2 n_2^2 (n_1 + n_2)$, $a_0 = (x\lambda_2 n_1^2 - (1-x) n_1^2 n_2 - n_1 n_2 \lambda_1)/a$,

$a_1 = (2(n_2^2 n_1 + n_1^2 n_2 x^2) - 4x n_1 n_2 (n_1 + n_2 + \lambda_1 + \lambda_2))/a$,

$a_2 = (8x(1-x) n_1 n_2^2 + 4x(n_2^3 + \lambda_2 n_2^2 - n_1^2 n_2 x - n_1 n_2 \lambda_1 x))/(3a)$.

While the PDF of X cannot be obtained directly from the above, it can be obtained from the CDF a follows: Let the density $g_{m,n}$ be that of m/n times an $F_{m,n}$ random variable. Let $G_{m,n}(y)$ be its distribution function, and let $g_{m,n}^{\lambda_1,\lambda_2}$ and $G_{m,n}^{\lambda_1,\lambda_2}(y)$ be the density and distribution function of its doubly noncentral version (the distribution of $\chi_m^2(\lambda_1)/\chi_n^2(\lambda_2)$). Then the following recurrence relations hold citep{Chattamvelli_1995}

$$n \left[G_{m,n+2}^{\lambda}(y) - G_{m-2,n+2}^{\lambda}(y) \right] = -2 g_{m,n}^{\lambda}(y)$$

`ctx.`**`fisher_f_nc2_spa_inv`**`(`*q*, *m*, *n*, *lambda1*, *lambda2*, *results='cdf'*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.10 Distributions related to multiple comparisons of means

4.10.1 Overview and literature

See also: Hahn and Hendrickson [361], Narula [422], Stoline and Ury [507], Tong [785], Genz *et al.* [347].

See also: Bechhofer and Dunnett [26]

See also: Genz and Bretz [346]

See also: Genz *et al.* [347]

See also: Hahn and Hendrickson [361]

See also: Narula [422]

See also: Stoline and Ury [507]

See also: Tong [785]

For tables see Soong [504].

For tables see David [197].

For tables see David *et al.* [198].

An n-dimensional random variable $\mathbf{X}$ with mean vector $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$ is said to have a nonsingular multivariate normal distribution, in symbols $\boldsymbol{X} \sim \mathcal{N}_n(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, if $\boldsymbol{\Sigma}$ is positive definite, and the density function of $\mathbf{X}$ is of the form (see Tong 1990):

$$f(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{n/2}|\boldsymbol{\Sigma}|^{1/2}} e^{-Q_n(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\Sigma})/2}, \quad \boldsymbol{x} \in \Re^n$$

where

$$Q_n(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = (\boldsymbol{x} - \boldsymbol{\mu})'\boldsymbol{\Sigma}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}).$$

The notion of cumulative distribution function (cdf) in one dimension can be extended to the multidimensional case, based on rectangular regions. We define the cdf $F(\mathbf{x})$ of a random vector $\mathbf{X}$ as the probability that all components $\mathbf{X}$ are less than or equal to the corresponding values in the vector $\mathbf{x}$:

$$F(\mathbf{x}) = \mathbb{P}(\mathbf{X} \leq \mathbf{x}), \quad \text{where } \mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}).$$

See also: Genz(2009), Bretz(2003).

Let $\boldsymbol{R} = (\rho_{ij})$ be an $n \times n$ symmetric matrix such that it is either positive definite or positive semidefinite and $\rho_{ii} = 1 (i = 1, \ldots, n)$. Let $\mathbf{Z} = (Z_1, \ldots, Z_n)'$ have an $\mathcal{N}_n(\mathbf{0}, \boldsymbol{R})$ distribution, and let the univariate variable S be such that S is independent of $\boldsymbol{Z}$, and νS^2 has a $\chi^2(\nu)$ distribution. Then a natural generalization of the Student's t variable is

$$\boldsymbol{t} = (t_1, \ldots, t_n)' = \left(\frac{Z_1}{S}, \ldots, \frac{Z_n}{S}\right)'.$$

If $\boldsymbol{R}$ is positive definite, then the density of $\boldsymbol{t}$ (with correlation matrix $\boldsymbol{R}$ and degrees of freedom ν) is given by citep{Tong_1990}:

$$h(\boldsymbol{t}; \boldsymbol{R}, \nu) = \frac{\Gamma((n+\nu)/2)}{(\nu\pi)^{n/2}\Gamma(\nu/2)|\boldsymbol{R}|^{1/2}} \left(1 + \frac{1}{\nu}\boldsymbol{t}'\boldsymbol{R}^{-1}\boldsymbol{t}\right)^{-(n+\nu)/2}, \quad \boldsymbol{t} \in \Re^n.$$

The notion of cumulative distribution function (cdf) in one dimension can be extended to the multidimensional case, based on rectangular regions. We define the cdf $F(\mathbf{x})$ of a random vector $\mathbf{X}$ as the probability that all components $\mathbf{X}$ are less than or equal to the corresponding values in the vector $\mathbf{x}$:

$$F(\mathbf{x}) = \mathbb{P}(\mathbf{X} \leq \mathbf{x}), \quad \text{where } \mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}).$$

See also: Genz(2009), Bretz(2003).

4.10.2 Normal maximum distribution, $\rho_{ij,i\neq j} = \rho$

class ctx.**dist_nmax_rho**(*rho*, *k*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of the maximum of $k \geq 2$ correlated standard normal variates, with common correlation $0 \leq \rho \leq 1$ is a continuous distribution with support interval $(-\infty, +\infty)$. See also Dunnett [292], Bechhofer and Dunnett [26], Grubbs [354], Bechhofer and Dunnett [26], Stoline and Ury [507], and Hahn and Hendrickson [361].

dist_nmax_rho.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$\text{pdf}_X(x) = f_{\text{nmax_corr}}(x, \rho, k) = \frac{k}{b} \int_{-\infty}^{\infty} (\Phi(z_1))^{k-1} \, \phi(z_1) \, \phi(y) \, \mathrm{d}y, \quad \text{where}$$

$$z_1 = \frac{x + a}{b}, \quad a = y\sqrt{|\rho|}, \quad b = \sqrt{1 - \rho}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_dunnett_t(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_nmax_rho.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$\text{cdf}_X(x) = F_{\text{nmax_corr}}(x, \rho, k) = \int_{-\infty}^{\infty} \left[\Phi\left(\frac{x + \sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)\right]^n \phi(y)\mathrm{d}y$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_dunnett_t(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_nmax_rho.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$\text{sf}_X(x) = 1 - \int_{-\infty}^{\infty} \left[\Phi\left(\frac{x + \sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)\right]^n \phi(y)\mathrm{d}y$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_dunnett_t(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_nmax_rho.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

There is no known explicit form for the quantile function $\text{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_dunnett_t(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_nmax_rho.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_dunnett_t(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_nmax_rho.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_dunnett_t(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_nmax_rho.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$M_X(t) = \int_0^\infty e^{tx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_studentized_range(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_nmax_rho.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$K_X(t) = \log\left(M_X(t)\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_studentized_range(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_nmax_rho.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \text{pdf}_X(x)\text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_dunnett_t(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_nmax_rho.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_dunnett_t(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

4.10.3 Normal maximum modulus distribution, $\rho_{ij,i\neq j} = \rho$

class ctx.**dist_nmm_rho**(*rho*, *k*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variates, with common correlation $0 \leq \rho \leq 1$ is a continuous distribution with support interval $(-\infty, +\infty)$. See also Dunnett [292], Bechhofer and Dunnett [26], Grubbs [354], Bechhofer and Dunnett [26], Stoline and Ury [507], and Hahn and Hendrickson [361].

dist_nmm_rho.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$\text{pdf}_X(x) = \frac{k}{b} \int_{-\infty}^{\infty} \left(\Phi(z_1) - \Phi(z_2)\right)^{k-1} \left(\phi(z_1) + \phi(z_1)\right) \phi(y)\, \mathrm{d}y, \quad \text{where}$$

$$z_1 = \frac{x+a}{b}, z_2 = \frac{-x+a}{b}, \quad a = y\sqrt{|\rho|}, \quad b = \sqrt{1-\rho}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_dunnett_t(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_nmm_rho.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$\text{cdf}_X(x) = \int_{-\infty}^{\infty} \left[\Phi\left(\frac{x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right) - \Phi\left(\frac{-x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)\right]^k \phi(y)\mathrm{d}y$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_dunnett_t(mu, sigma).pdf(x))
6.356352346256452561561561456135 6E-20
```

dist_nmm_rho.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$\text{sf}_X(x) = 1 - \int_{-\infty}^{\infty} \left[\Phi\left(\frac{x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right) - \Phi\left(\frac{-x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)\right]^k \phi(y)\mathrm{d}y$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_dunnett_t(mu, sigma).pdf(x))
sf: 6.356352346256452561561561456135 6E-20
```

dist_nmm_rho.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

There is no known explicit form for the quantile function $\text{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_dunnett_t(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_nmm_rho.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_dunnett_t(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_nmm_rho.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_dunnett_t(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_nmm_rho.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$M_X(t) = \int_0^\infty e^{tx} \text{pdf}_X(x) \mathrm{d}x$$

dist_nmm_rho.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$K_X(t) = \log\left(M_X(t)\right)$$

dist_nmm_rho.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_dunnett_t(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_nmm_rho.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_dunnett_t(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.10.4 Normal maximum distribution, $\rho_{ij,i\neq j} = \lambda_i \lambda_j$

class ctx.**dist_nmax_rhoij**(*rho*, *k*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of the maximum of $k \geq 2$ correlated standard normal variates, with common correlation $0 \leq \rho \leq 1$ is a continuous distribution with support interval $(-\infty, +\infty)$. See also Dunnett [292], Bechhofer and Dunnett [26], Grubbs [354], Bechhofer and Dunnett [26], Stoline and Ury [507], and Hahn and Hendrickson [361].

dist_nmax_rhoij.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$\text{pdf}_X(x) = f_{\text{nmaxrhoij}}(x, \rho, k) = \frac{k}{b} \int_{-\infty}^{\infty} (\Phi(z_1))^{k-1} \, \phi(z_1) \, \phi(y) \, \mathrm{d}y, \quad \text{where}$$

$$z_1 = \frac{x + a}{b}, \quad a = y\sqrt{|\rho|}, \quad b = \sqrt{1 - \rho}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_dunnett_t(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_nmax_rhoij.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$\text{cdf}_X(x) = F_{\text{nmaxrhoij}}(x, \rho, k) = \int_{-\infty}^{\infty} \left[\Phi\left(\frac{x + \sqrt{|\rho|}y}{\sqrt{1-\rho}} \right) \right]^n \phi(y) \mathrm{d}y$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_dunnett_t(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_nmax_rhoij.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$\text{sf}_X(x) = 1 - \int_{-\infty}^{\infty} \left[\Phi\left(\frac{x + \sqrt{|\rho|}y}{\sqrt{1-\rho}} \right) \right]^n \phi(y) \mathrm{d}y$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_dunnett_t(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_nmax_rhoij.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

There is no known explicit form for the quantile function $\text{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_dunnett_t(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_nmax_rhoij.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_dunnett_t(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_nmax_rhoij.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_dunnett_t(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_nmax_rhoij.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$M_X(t) = \int_0^\infty e^{tx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_studentized_range(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_nmax_rhoij.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables:

$$K_X(t) = \log\left(M_X(t)\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_studentized_range(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_nmax_rhoij.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_dunnett_t(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_nmax_rhoij.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variables. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_dunnett_t(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.10.5 Normal maximum modulus distribution, $\rho_{ij,i\neq j} = \lambda_i\lambda_j$

class ctx.**dist_nmm_rhoij**(*rho*, *k*, ***kwargs*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variates, with common correlation $0 \leq \rho \leq 1$ is a continuous distribution with support interval $(-\infty, +\infty)$. See also Dunnett [292], Bechhofer and Dunnett [26], Grubbs [354], Bechhofer and Dunnett [26], Stoline and Ury [507], and Hahn and Hendrickson [361].

dist_nmm_rhoij.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$\text{pdf}_X(x) = \frac{k}{b}\int_{-\infty}^{\infty}\left(\Phi(z_1)-\Phi(z_2)\right)^{k-1}\left(\phi(z_1)+\phi(z_1)\right)\phi(y)\,\mathrm{d}y, \quad \text{where}$$

$$z_1 = \frac{x+a}{b}, z_2 = \frac{-x+a}{b}, \quad a = y\sqrt{|\rho|}, \quad b = \sqrt{1-\rho}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_dunnett_t(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_nmm_rhoij.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$\text{cdf}_X(x) = \int_{-\infty}^{\infty}\left[\Phi\left(\frac{x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)-\Phi\left(\frac{-x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)\right]^k\phi(y)\mathrm{d}y$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_dunnett_t(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_nmm_rhoij.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$\text{sf}_X(x) = 1 - \int_{-\infty}^{\infty}\left[\Phi\left(\frac{x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)-\Phi\left(\frac{-x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)\right]^k\phi(y)\mathrm{d}y$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_dunnett_t(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_nmm_rhoij.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

There is no known explicit form for the quantile function $\text{cdf}_X^{-1}(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_dunnett_t(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_nmm_rhoij.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_dunnett_t(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_nmm_rhoij.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_dunnett_t(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_nmm_rhoij.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$M_X(t) = \int_0^\infty e^{tx} \text{pdf}_X(x) \mathrm{d}x$$

dist_nmm_rhoij.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables:

$$K_X(t) = \log\left(M_X(t)\right)$$

dist_nmm_rhoij.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \text{pdf}_X(x) \text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_dunnett_t(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_nmm_rhoij.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variables. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_dunnett_t(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.10.6 Normal range distribution

class ctx.**dist_normal_range**(k)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The normal range distribution is a continuous probability distribution with $k \geq 2$ groups, and the support interval $(0, +\infty)$. See also Wikipedia [1218], Harter [364], R (Statistical System) [535], and *dist_normal_range()*.

dist_normal_range.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a central normal range distribution:

$$\text{pdf}_X(x) = k(k-1)\int_{-\infty}^{\infty} (\Phi(y) - \Phi(y-x))^{k-2}\,\phi(y)\phi(y-x)\mathrm{d}y.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_normal_range(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_normal_range.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a central normal range distribution:

$$\text{cdf}_X(x) = k\int_{-\infty}^{\infty} \phi(y)\,(\Phi(y) - \Phi(y-x))^{k-1}\,\mathrm{d}y.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_normal_range(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_normal_range.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function function (sf) of a random variable X, following a central normal range distribution:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x) = k\int_{-\infty}^{\infty} \phi(y)\left(L_1^k - : cite : t : `L_1 - L_2\right]^{k-1})\,\mathrm{d}y, \quad \text{where}$$

$$L_1 = \Phi(y), \quad L_2 = \Phi(y-x), \quad L_2 \to 0 \text{ for } x \to \infty$$

$$L_1^k - (L_1 - L_2)^k = L_1^k\left(1 - \left(1 - \frac{L_2}{L_1}\right)^k\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_normal_range(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_normal_range.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a central normal range distribution:

There is no known explicit form for the quantile function $\text{qtf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_normal_range(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_normal_range.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a central normal range distribution:

There is no known explicit form for the quantile function $\text{isf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_normal_range(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_normal_range.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a central normal range distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_normal_range(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_normal_range.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a central normal range distribution:

$$M_X(t) = \int_0^\infty e^{tx} \text{pdf}_X(x) \text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_normal_range(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_normal_range.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a central normal range distribution:

$$K_X(t) = \log\left(M_X(t)\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_normal_range(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_normal_range.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a central normal range distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_normal_range(mu, sigma).moments(k))
6.3563523462564525615615615614561356E+00
```

dist_normal_range.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a central normal range distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_normal_range(mu, sigma).cumulants(k))
6.3563523462564525615615615614561356E+00
```

Additional information

$$Q(x) = k \int_{-\infty}^{\infty} \phi(y) \left(\Phi(y) - \Phi(y - x)\right)^{k-1} \mathrm{d}y$$

$$P(x) = 1 - Q(x) = k \int_{-\infty}^{\infty} \phi(y) \left(L_1^k - : cite : t : `L_1 - L_2\right]^{k-1}) \mathrm{d}y, \quad \text{where}$$

$$L_1 = \Phi(y), \quad L_2 = \Phi(y - x), \quad L_2 \to 0 \text{ for } x \to \infty$$

$$L_1^k - (L_1 - L_2)^k = L_1^k \left(1 - \left(1 - \frac{L_2}{L_1}\right)^k\right)$$

4.10.7 Studentized maximum distribution

class ctx.**dist_smax**(*k*, *nu*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The studentized maximum distribution with $k \geq 1$ groups and ν error degrees of freedom is a continuous distribution with the support interval $(-\infty, +\infty)$. See also Stoline and Ury [507], Hochberg and Tamhane [373], Narula [422], Bechhofer and Dunnett [26], and Hahn and Hendrickson [361].

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom. Then

$$Q = \frac{\max X_j}{s}, \quad j = 1, \ldots, k$$

has a Studentized Maximum distribution with k and n degrees of freedom.

dist_smax.**pdf**(*x*)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a studentized maximum distribution:

$$\mathrm{pdf}_X(x) = \int_0^\infty f_{\mathrm{nnax}}(sx, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $f_{\mathrm{nnax}}(\cdot, k)$ is the pdf of the normal maximum with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", smax(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_smax.**cdf**(*x*)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a studentized maximum distribution:

$$\mathrm{cdf}_X(x) = \int_0^\infty F_{\mathrm{nnax}}(sx, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nnax}}(\cdot, k)$ is the cdf of the normal maximum with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", smax(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_smax.**sf**(*x*)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following a studentized maximum distribution:

$$\mathrm{sf}_X(x) = 1 - \int_0^\infty F_{\mathrm{nnax}}(sx, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nnax}}(\cdot, k)$ is the cdf of the normal maximum with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", smax(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_smax.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a studentized maximum distribution:

There is no known explicit form for the quantile function $\text{qtf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", smax(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

dist_smax.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a studentized maximum distribution:

There is no known explicit form for the quantile function $\text{isf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", smax(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_smax.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a studentized maximum distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", smax(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_smax.**m_x**(t)

Returns `NaN`, since the moment generating function does not exist.

dist_smax.**k_x**(t, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_smax.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a studentized maximum distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", smax(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_smax.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a studentized maximum distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", smax(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.10.8 Studentized maximum modulus distribution

class ctx.**dist_smm**(k, *nu*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The studentized maximum modulus distribution with $k \geq 1$ groups and ν error degrees of freedom is a continuous distribution with the support interval $(-\infty, +\infty)$. See also Stoline and Ury [507], Hochberg and Tamhane [373], Narula [422], Bechhofer and Dunnett [26], and Hahn and Hendrickson [361].

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom. Then

$$Q = \frac{\max|X_j|}{s}, \quad j = 1, \ldots, k$$

has a Studentized Maximum Modulus distribution with k and n degrees of freedom.

dist_smm.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a studentized maximum modulus distribution:

$$\mathrm{pdf}_X(x) = \int_0^\infty f_{\mathrm{nmm}}(sx, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $f_{\mathrm{nmm}}(\cdot, k)$ is the pdf of the normal maximum modulus with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", smm(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_smm.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a studentized maximum modulus distribution:

$$\mathrm{cdf}_X(x) = \int_0^\infty F_{\mathrm{nmm}}(sx, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmm}}(\cdot, k)$ is the cdf of the normal maximum modulus with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", smm(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_smm.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following a studentized maximum modulus distribution:

$$\mathrm{sf}_X(x) = 1 - \int_0^\infty F_{\mathrm{nmm}}(sx, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmm}}(\cdot, k)$ is the cdf of the normal maximum modulus with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", smm(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_smm.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a studentized maximum modulus distribution:

There is no known explicit form for the quantile function $\text{qtf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", smm(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

dist_smm.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a studentized maximum modulus distribution:

There is no known explicit form for the quantile function $\text{isf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", smm(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_smm.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a studentized maximum modulus distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", smm(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_smm.**m_x**(t)

Returns `NaN`, since the moment generating function does not exist.

dist_smm.**k_x**(t, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_smm.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a studentized maximum modulus distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \text{pdf}_X(x) \text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", smm(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_smm.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a studentized maximum modulus distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", smm(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.10.9 Distribution of Dunnett's t, one-sided

class ctx.**dist_dunnett1_t**(*rho*, *k*, *nu*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The 1-sided Dunnett t-distribution with common correlation ρ, $k \geq 2$ groups (including control group) and error degrees of freedom ν is a continuous distribution with the support interval $(-\infty, +\infty)$. See also Dunnett [292], Bechhofer and Dunnett [26].

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom. Then

$$Q = \frac{\max(X_1 - X_j)}{s}, \quad 1 < j < k$$

has a onesided Dunnett's t-distribution with k and n degrees of freedom.

For Dunnett's test:

$$\lambda_i = \frac{1}{\sqrt{1 + n_0/n_i}}$$

dist_dunnett1_t.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a 1-sided Dunnett t-distribution:

$$\mathrm{pdf}_X(x) = \int_0^\infty f_{\mathrm{nmax_corr}}(sx, \rho, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $f_{\mathrm{nmax_corr}}(\cdot, \rho, k)$ is the pdf of the normal maximum (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dunnett1_t(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_dunnett1_t.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a 1-sided Dunnett t-distribution:

$$\mathrm{cdf}_X(x) = \int_0^\infty F_{\mathrm{nmax_corr}}(sx, \rho, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmax_corr}}(\cdot, \rho, k)$ is the cdf of the normal maximum (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dunnett1_t(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_dunnett1_t.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following a 1-sided Dunnett t-distribution:

$$\mathrm{sf}_X(x) = 1 - \int_0^\infty F_{\mathrm{nmax_corr}}(sx, \rho, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmax_corr}}(\cdot, \rho, k)$ is the cdf of the normal maximum (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dunnett1_t(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_dunnett1_t.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a 1-sided Dunnett t-distribution:

There is no known explicit form for the quantile function $\mathrm{qtf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dunnett1_t(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

dist_dunnett1_t.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a 1-sided Dunnett t-distribution:

There is no known explicit form for the quantile function $\mathrm{isf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dunnett1_t(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_dunnett1_t.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a 1-sided Dunnett t-distribution:

$$C_X(t) = \int_0^\infty e^{itx} \mathrm{pdf}_X(x) \mathrm{d}x$$

where $U(\cdot)$ denotes the confluent hypergeometric function of the second kind.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dunnett1_t(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_dunnett1_t.**m_x**(t)

Returns NaN, since the moment generating function does not exist.

dist_dunnett1_t.**k_x**(t, *k=0*)

Returns NaN, since the cumulant generating function does not exist.

dist_dunnett1_t.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a 1-sided Dunnett t-distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dunnett1_t(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_dunnett1_t.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a 1-sided Dunnett t-distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dunnett1_t(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.10.10 Distribution of Dunnett's t, two-sided

class ctx.**dist_dunnett2_t**(*rho*, *k*, *nu*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The 2-sided Dunnett t-distribution with common correlation ρ, $k \geq 2$ groups (including control group) and error degrees of freedom ν is a continuous distribution with the support interval $(0, +\infty)$. See also Dunnett [292], Bechhofer and Dunnett [26].

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom. Then

$$Q = \frac{\max|X_1 - X_j|}{s}, \quad 1 < j < k$$

has a twosided Dunnett's $|t|$-distribution with k and n degrees of freedom.

For Dunnett's test:

$$\lambda_i = \frac{1}{\sqrt{1 + n_0/n_i}}$$

dist_dunnett2_t.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a 2-sided Dunnett t-distribution:

$$\mathrm{pdf}_X(x) = \int_0^\infty f_{\mathrm{nmm_corr}}(sx, \rho, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $f_{\mathrm{nmm_corr}}(\cdot, \rho, k)$ is the pdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dunnett2_t(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_dunnett2_t.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a 2-sided Dunnett t-distribution:

$$\mathrm{cdf}_X(x) = \int_0^\infty F_{\mathrm{nmm_corr}}(sx, \rho, k)\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmm_corr}}(\cdot, \rho, k)$ is the cdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dunnett2_t(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_dunnett2_t.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following a 2-sided Dunnett t-distribution:

$$\mathrm{sf}_X(x) = 1 - \int_0^\infty F_{\mathrm{nmm_corr}}(sx, \rho, k)\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmm_corr}}(\cdot, \rho, k)$ is the cdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dunnett2_t(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_dunnett2_t.**qtf**(*q*)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a 2-sided Dunnett t-distribution:

There is no known explicit form for the quantile function $\text{qtf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dunnett2_t(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_dunnett2_t.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a 2-sided Dunnett t-distribution:

There is no known explicit form for the quantile function $\text{isf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dunnett2_t(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_dunnett2_t.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following a 2-sided Dunnett t-distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

where $U(\cdot)$ denotes the confluent hypergeometric function of the second kind.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dunnett2_t(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_dunnett2_t.**m_x**(*t*)

Returns `NaN`, since the moment generating function does not exist.

dist_dunnett2_t.**k_x**(*t*, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_dunnett2_t.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a 2-sided Dunnett t-distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \text{pdf}_X(x)\text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dunnett2_t(mu, sigma).moments(k))
6.35635234625645256156156145614561356E+00
```

dist_dunnett2_t.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a 2-sided Dunnett t-distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", dunnett2_t(mu, sigma).cumulants(k))
6.35635234625645256156156145614561356E+00
```

4.10.11 Nair's t-distribution

class ctx.**dist_nair_t**(*rho*, *k*, *nu*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Nair t-distribution with common correlation ρ, $k \geq 2$ groups (including control group) and error degrees of freedom ν is a continuous distribution with the support interval $(-\infty, +\infty)$. See also Dunnett [292], Bechhofer and Dunnett [26].

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom. Then

$$Q = \frac{\max(X_1 - X_j)}{s}, \quad 1 < j < k$$

has a Nair t-distribution with k and n degrees of freedom.

For Dunnett's test:

$$\lambda_i = \frac{1}{\sqrt{1 + n_0/n_i}}$$

dist_nair_t.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Nair t-distribution:

$$\mathrm{pdf}_X(x) = \int_0^\infty f_{\mathrm{nmax_corr}}(sx, \rho, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $f_{\mathrm{nmax_corr}}(\cdot, \rho, k)$ is the pdf of the normal maximum (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", nair_t(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_nair_t.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Nair t-distribution:

$$\mathrm{cdf}_X(x) = \int_0^\infty F_{\mathrm{nmax_corr}}(sx, \rho, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmax_corr}}(\cdot, \rho, k)$ is the cdf of the normal maximum (equicorrelated case) with common correlation ρ and k groups , and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", nair_t(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_nair_t.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following a Nair t-distribution:

$$\mathrm{sf}_X(x) = 1 - \int_0^\infty F_{\mathrm{nmax_corr}}(sx, \rho, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmax_corr}}(\cdot, \rho, k)$ is the cdf of the normal maximum (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", nair_t(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_nair_t.**qtf**(*q*)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a Nair t-distribution:

There is no known explicit form for the quantile function $\text{qtf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", nair_t(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_nair_t.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a Nair t-distribution:

There is no known explicit form for the quantile function $\text{isf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", nair_t(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_nair_t.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Nair t-distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

where $U(\cdot)$ denotes the confluent hypergeometric function of the second kind.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", nair_t(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_nair_t.**m_x**(*t*)

Returns NaN, since the moment generating function does not exist.

dist_nair_t.**k_x**(*t*, *k=0*)

Returns NaN, since the cumulant generating function does not exist.

dist_nair_t.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Nair t-distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \text{pdf}_X(x) \text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", nair_t(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_nair_t.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Nair t-distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", nair_t(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.10.12 Halperin's t-distribution

class ctx.**dist_halperin_t**(*rho*, *k*, *nu*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Halperin t-distribution with common correlation ρ, $k \geq 2$ groups (including control group) and error degrees of freedom ν is a continuous distribution with the support interval $(0, +\infty)$. See also Dunnett [292], Bechhofer and Dunnett [26].

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom. Then

$$Q = \frac{\max|X_1 - X_j|}{s}, \quad 1 < j < k$$

has a Halperin $|t|$-distribution with k and n degrees of freedom.

For Dunnett's test:

$$\lambda_i = \frac{1}{\sqrt{1 + n_0/n_i}}$$

dist_halperin_t.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Halperin t-distribution:

$$\mathrm{pdf}_X(x) = \int_0^\infty f_{\mathrm{nmm_corr}}(sx, \rho, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $f_{\mathrm{nmm_corr}}(\cdot, \rho, k)$ is the pdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", halperin_t(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_halperin_t.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Halperin t-distribution:

$$\mathrm{cdf}_X(x) = \int_0^\infty F_{\mathrm{nmm_corr}}(sx, \rho, k)\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmm_corr}}(\cdot, \rho, k)$ is the cdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", halperin_t(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_halperin_t.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following a Halperin t-distribution:

$$\mathrm{sf}_X(x) = 1 - \int_0^\infty F_{\mathrm{nmm_corr}}(sx, \rho, k)\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmm_corr}}(\cdot, \rho, k)$ is the cdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", halperin_t(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_halperin_t.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a Halperin t-distribution:

There is no known explicit form for the quantile function $\text{qtf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", halperin_t(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561456135 6E+00
```

dist_halperin_t.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a Halperin t-distribution:

There is no known explicit form for the quantile function $\text{isf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", halperin_t(mu, sigma).isf(q))
6.356352346256452561561561456135 6E+00
```

dist_halperin_t.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Halperin t-distribution:

$$C_X(t) = \int_0^{\infty} e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

where $U(\cdot)$ denotes the confluent hypergeometric function of the second kind.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", halperin_t(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_halperin_t.**m_x**(t)

Returns `NaN`, since the moment generating function does not exist.

dist_halperin_t.**k_x**(t, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_halperin_t.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Halperin t-distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \text{pdf}_X(x) \text{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", halperin_t(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_halperin_t.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Halperin t-distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", halperin_t(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.10.13 Nelson's h-distribution

class ctx.**dist_nelson_h**(*rho*, *k*, *nu*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Halperin t-distribution with common correlation ρ, $k \geq 2$ groups (including control group) and error degrees of freedom ν is a continuous distribution with the support interval $(0, +\infty)$. See also Dunnett [292], Bechhofer and Dunnett [26].

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom. Then

$$Q = \frac{\max|X_1 - X_j|}{s}, \quad 1 < j < k$$

has a Halperin $|t|$-distribution with k and n degrees of freedom.

For Dunnett's test:

$$\lambda_i = \frac{1}{\sqrt{1 + n_0/n_i}}$$

dist_nelson_h.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a Halperin t-distribution:

$$\mathrm{pdf}_X(x) = \int_0^\infty f_{\mathrm{nmm_corr}}(sx, \rho, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $f_{\mathrm{nmm_corr}}(\cdot, \rho, k)$ is the pdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", nelson_h(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_nelson_h.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Halperin t-distribution:

$$\mathrm{cdf}_X(x) = \int_0^\infty F_{\mathrm{nmm_corr}}(sx, \rho, k)\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmm_corr}}(\cdot, \rho, k)$ is the cdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", nelson_h(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_nelson_h.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following a Halperin t-distribution:

$$\mathrm{sf}_X(x) = 1 - \int_0^\infty F_{\mathrm{nmm_corr}}(sx, \rho, k)\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nmm_corr}}(\cdot, \rho, k)$ is the cdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", nelson_h(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_nelson_h.**qtf**(*q*)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a Halperin t-distribution:

There is no known explicit form for the quantile function $\text{qtf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", nelson_h(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_nelson_h.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a Halperin t-distribution:

There is no known explicit form for the quantile function $\text{isf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", nelson_h(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_nelson_h.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Halperin t-distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

where $U(\cdot)$ denotes the confluent hypergeometric function of the second kind.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", nelson_h(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_nelson_h.**m_x**(*t*)

Returns `NaN`, since the moment generating function does not exist.

dist_nelson_h.**k_x**(*t*, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_nelson_h.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Halperin t-distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", nelson_h(mu, sigma).moments(k))
6.356352346256452561561561561456l356E+00
```

dist_nelson_h.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Halperin t-distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", nelson_h(mu, sigma).cumulants(k))
6.356352346256452561561561561456l356E+00
```

4.10.14 Studentized range distribution

class ctx.**dist_studentized_range**(*k*, *nu*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The studentized range distribution with $k \geq 2$ groups and error degrees of freedom ν is a continuous distribution with the support interval $(0, +\infty)$. See also Wikipedia [1218], Harter [364], R (Statistical System) [535],

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom. Then

$$Q = \frac{\max|X_i - X_j|}{s}, \quad 1 < i < j < k$$

has a Studentized Range distribution with k and n degrees of freedom.

Let $X_1, \ldots, X_k$ be a random sample of size k from a $\mathcal{N}(0, \sigma^2)$ distribution. Let s^2 be an independent mean square estimate of σ with n degrees of freedom.

Then $Q_n = \max|X_i - X_j|, 1 < i < j < k$, has a Normal Range distribution with k degrees of freedom, and $Q_t = Q_n/s$ has a Studentized Range distribution with k and n degrees of freedom.

dist_studentized_range.**pdf**(*x*)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following a central studentized range distribution:

$$\text{pdf}_X(x) = \int_0^\infty f_{\text{nrange}}(sx, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $f_{\text{nrange}}(\cdot, k)$ is the pdf of the normal range with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", mp_studentized_range(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_studentized_range.**cdf**(*x*)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a central studentized range distribution:

$$\text{cdf}_X(x) = \int_0^\infty F_{\text{nrange}}(sx, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\text{nrange}}(\cdot, k)$ is the cdf of the normal range with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mp_studentized_range(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_studentized_range.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following a central studentized range distribution:

$$\mathrm{sf}_X(x) = 1 - \int_0^\infty F_{\mathrm{nrange}}(sx, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\, \mathrm{d}s$$

where $F_{\mathrm{nrange}}(\cdot, k)$ is the cdf of the normal range with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mp_studentized_range(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_studentized_range.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following a central studentized range distribution:

There is no known explicit form for the quantile function $\mathrm{qtf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mp_studentized_range(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_studentized_range.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following a central studentized range distribution:

There is no known explicit form for the quantile function $\mathrm{isf}_X(x)$: It is computed using Newton iterations with starting values from a central F approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mp_studentized_range(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_studentized_range.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a central studentized range distribution:

$$C_X(t) = \int_0^\infty e^{itx} \mathrm{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mp_studentized_range(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_studentized_range.**m_x**(*t*)

Returns NaN, since the moment generating function does not exist.

dist_studentized_range.**k_x**(*t*, *k=0*)

Returns NaN, since the cumulant generating function does not exist.

dist_studentized_range.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a central studentized range distribution. The rth moments only exists for $n_2 > 2r$.

$$\mu'_X(r) = E(X^r) = \int_0^\infty x^r \mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_studentized_range(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_studentized_range.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a central studentized range distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mp_studentized_range(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.11 Distributions related to multivariate statistical analysis

4.11.1 Distribution of the sum of the negative logarithms of independent beta variables

class ctx.**dist_logrv_beta_sum**(p, bi, ci)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

A random variable X follows the distribution of the negative logarithm of the product of p beta variables with parameters a_i and b_i if it is defined as $X = -\log(Y)$, where Y follows a beta product distribution with parameters a_i and b_i. The support interval of X is $(0, +\infty)$.

See also Anderson [9], Muirhead [418], Butler [165], Ginzberg [349], pages 92-105, Marques *et al.* [411], Tang and Gupta [513].

dist_logrv_beta_sum.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\text{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_logrv_beta_sum.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\text{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_logrv_beta_sum.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\mathrm{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

`dist_logrv_beta_sum.qtf(q)`

Returns $\mathrm{qtf}_X(q)$, the quantile function (qtf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{qtf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

`dist_logrv_beta_sum.isf(q)`

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{isf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

`dist_logrv_beta_sum.c_x(t)`

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

$$C_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - it)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - it\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_logrv_beta_sum.**m_x**(t)

Returns the moment generating function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$M_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - t)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - t\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_logrv_beta_sum.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its r^{th} derivatives, $K_X^{(r)}(t), r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$K_X(t) = \sum_{j=1}^{p} \log\left(\Gamma(a_j - t)\right) - \log\left(\Gamma(a_j + b_j - t)\right) + \log\left(\Gamma(a_j + b_j)\right) - \log\left(\Gamma(a_j)\right).$$

$$K_X^{(r)}(t) = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j - t) - \psi^{(r-1)}(a_j + b_j - t)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_logrv_beta_sum.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_logrv_beta_sum.**cumulants**(k)

Returns the first j cumulants, $\kappa_r, r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$\kappa_r = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j) - \psi^{(r-1)}(a_j + b_j)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145561356E+00
```

Approximations

`ctx.`**`beta_product_ecf`**(*x*, *ai*, *bi*, *results='cdf'*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

`ctx.`**`beta_product_ecf_inv`**(*q*, *ai*, *bi*, *results='qtf'*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

4.11.2 Distribution of the product of independent beta variables

class ctx.**dist_beta_product**(*p*, *bi*, *ci*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The betaproduct distribution is a continuous probability distribution, defined as the product of $p \geq 1$ independent beta variables, which have a beta distribution with b_j and $c_j - b_j$ degrees of freedom, and the support interval $(0, 1)$. See also Anderson [9], Muirhead [418], Butler [165], Ginzberg [349], pages 92-105, Marques *et al.* [411], Tang and Gupta [513].

Text explaining the concepts.

See also Olkin and Press [432], Wilks [1453], Geisser [345], Marques *et al.* [411], Mauchly [412], Coelho *et al.* [180].

dist_beta_product.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the product of independent beta variables. The pdf can expressed as a Meijer G function as follows (PhamGia_2008, p. 1702, Ginzberg_2013, p. 97 + 98):

$$\text{pdf}_X(x) = \frac{Ke^{-x/2}}{2\alpha} G^{m,0}_{m,m}\left(\begin{matrix} a_1, \ldots, a_n; a_{n+1} \ldots a_p \\ b_1, \ldots, b_m; b_{m+1} \ldots b_q \end{matrix} \middle| \; e^{\frac{-x}{2\alpha}}\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", beta_product(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_beta_product.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the product of independent beta variables. The cdf can expressed as a Meijer G function as follows (PhamGia_2008, p. 1702, Ginzberg_2013, p. 97 + 98):

$$\text{cdf}_X(x) == Ke^{-x/2} G^{m+1,0}_{m+1,m+1}\left(\begin{matrix} a_1, \ldots, a_n; a_{n+1} \ldots a_p \\ b_1, \ldots, b_m; b_{m+1} \ldots b_q \end{matrix} \middle| \; e^{\frac{-x}{2\alpha}}\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", beta_product(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_beta_product.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the product of independent beta variables. The cdf can expressed as a Meijer G function as follows (PhamGia_2008, p. 1702, Ginzberg_2013, p. 97 + 98):

$$\text{sf}_X(x) == 1 - Ke^{-x/2} G^{m+1,0}_{m+1,m+1}\left(\begin{matrix} a_1, \ldots, a_n; a_{n+1} \ldots a_p \\ b_1, \ldots, b_m; b_{m+1} \ldots b_q \end{matrix} \middle| \; e^{\frac{-x}{2\alpha}}\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", beta_product(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_beta_product.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of the product of independent beta variables:

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", beta_product(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_beta_product.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the product of independent beta variables:

There is no known closed form for the quantile function $\text{isf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", beta_product(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_beta_product.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the product of independent beta variables:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", beta_product(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_beta_product.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of the product of independent beta variables:

$$M_X(t) = \int_0^\infty e^{tx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", beta_product(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_beta_product.k_x(`s, k=*0*`)`

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following the distribution of the product of independent beta variables:

$$K_X(s) = \log\left(M_X(t)\right)$$

where $\Gamma_p(\cdot)$ is the multivariate gamma function and $\psi(\cdot)$ is the digamma function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", beta_product(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

`dist_beta_product.moments(`k`)`

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the product of independent beta variables:

$$\mu'_X(r) = \prod_{j=1}^{p} \frac{\Gamma(c_j)\Gamma(b_j + h)}{\Gamma(b_j)\Gamma(c_j + h)}, .$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", beta_product(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

`dist_beta_product.cumulants(`k`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of the product of independent beta variables. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", beta_product(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Additional information

`ctx.log_beta_prod_gp(`k`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the pdf, cdf and sf from the characteristic function using the procedure of Gil-Pelaez (see *gil_pelaez_pdf()* and *gil_pelaez_cdf()*).

This uses $U = \log 2W$.

For a large subset of the random variables W defined in in the previous section there exists a power transformation $U = W^{2/N}$, such that

$$\mathrm{E}[U^h] = \prod_{j=1}^{p} \frac{\Gamma(c_j)\Gamma(b_j + h)}{\Gamma(b_j)\Gamma(c_j + h)},$$

i.e. U is distributed as the product of p independent random variables which have a beta distribution with b_j and $c_j - b_j$ degrees of freedom. This allows to specify these distributions in a simpler form, just giving the b_j and c_j. Its density defined on $(0, 1)$

These are functions for which the l.r.t. $M/2$ is distributed as the product of p independent random variables which have a beta distribution with b_j and $c_j - b_j$ degrees of freedom.

The characteristic function has the general form:

$$\phi_{M/2}(t) = \prod_{j=1}^{p} \frac{\Gamma(c_j)\Gamma(b_j - it)}{\Gamma(b_j)\Gamma(c_j - it)}$$

The distribution of the product of independent beta random variables can also be expressed as a Box-Davis distribution of the Box type with the following parameter choices for a, b, x_k, ξ_k, y_j and η_j:

$$a = b = p, \quad x_k = y_j = N/2, \quad \xi_k = b_k - N/2, \quad \eta_j = c_j - N/2, \quad j, k = 1 \ldots p,$$

`ctx.beta_prod_spa(`*x*, *n*, *results='c'*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

This uses $2 \log W$.

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following the distribution of`2log W` of central Wilks W:

$$K_X(s) = \log \left[\frac{\Gamma_p(n/2 + s)\Gamma_p((n+m)/2)}{\Gamma_p(n/2)\Gamma_p((n+m)/2 + s)} \right].$$

$$K'(s) = \sum_{i=1}^{p} \left[\psi\left(\tfrac{1}{2}n + s - \tfrac{1}{2}(i-1)\right) - \psi\left(\tfrac{1}{2}(n+m) + s - \tfrac{1}{2}(i-1)\right) \right]$$

where $\Gamma_p(\cdot)$ is the multivariate gamma function and $\psi(\cdot)$ is the digamma function.

`ctx.beta_prod_spa_inv(`*x*, *n*, *results='qtf'*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

This uses $2 \log W$.

`ctx.beta_product_bd(`*x*, *f*, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Box-Davis approximation to the pdf, cdf and sf.

$$f = -2 \sum_{j=1}^{p} (b_j - c_j); \quad \theta = \frac{N}{2};$$

$$\rho = 1 - \frac{1}{f \cdot \theta} \sum_{j=1}^{p} \big(B_2(b_j - \theta) - B_2(c_j - \theta) \big)$$

$$\omega_r = \frac{(-1)^{r+1}}{r(r+1)(\rho\theta)^r} \sum_{j=1}^{p} \big(B_{r+1}(b_j - \rho\theta) - B_{r+1}(c_j - \rho\theta) \big)$$

`ctx.beta_product_bd_inv(`*q*, *f*, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Box-Davis approximation to the qtf and isf.

ctx.**beta_product_bd_cdf_old**(*x*, *f*, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Box-Davis approximation to the pdf, cdf and sf.

For the product-of-independent-beta-variates distributions, the parameters of the Box-Davis expansion are given in equations ....

The distribution of the product of independent beta random variables can also be expressed as a Box-Davis distribution of the Box type with the following parameter choices for a, b, x_k, ξ_k, y_j and η_j:

$$a = b = p, \quad x_k = y_j = N/2, \quad \xi_k = b_k - N/2, \quad \eta_j = c_j - N/2, \quad j, k = 1 \ldots p,$$

$$\theta = \min_{i,j}(x_i, y_j),$$

$$f = -2\left[\sum_{k=1}^{a} \xi_k - \sum_{j=1}^{b} \eta_j - \tfrac{1}{2}(a-b)\right],$$

$$\rho = 1 - \frac{1}{f}\left[\sum_{k=1}^{a} x_k^{-1}\left(\xi_k^2 - \xi_k + \tfrac{1}{6}\right) - \sum_{j=1}^{b} y_j^{-1}\left(\eta_j^2 - \eta_j + \tfrac{1}{6}\right)\right],$$

$$\omega_r = \frac{(-1)^{r+1}}{r(r+1)}\left[\sum_{k=1}^{a} \frac{B_{r+1}((1-\rho)x_k + \xi_k)}{(\rho x_k)^r} - \sum_{j=1}^{b} \frac{B_{r+1}((1-\rho)y_j + \eta_j)}{(\rho y_j)^r}\right],$$

$$\begin{aligned} \log(K_B) &= \log(K) + \frac{1}{2}(m-k)\log(2\pi) - \frac{f}{2}\log(\rho) \\ &\quad + \sum_{i=1}^{m}\left(x_i + \xi_i - \frac{1}{2}\right)\log(x_i) - \sum_{i=1}^{k}\left(y_i + \eta_i - \frac{1}{2}\right)\log(x_i) \end{aligned} \tag{4.38}$$

4.11.3 Distribution of Wilks' Λ

class ctx.**dist_wilks_lambda**(p, m, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Wilks' Λ distribution is a continuous probability distribution with $p \geq 1$ predictor variables, error degress of freedom $m \geq 1$ and $n \geq 1$, and the support interval $(0, 1)$. See also: Wilks [1451], Anderson [9], Muirhead [418], Butler [165], Pham-Gia [469], Witkovský [1496], Witkovský [1495].

dist_wilks_lambda.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Wilks' Lambda: The pdf is computed by numerical inversion of the characteristic function or cumulant generating function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", wilks_lambda(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_wilks_lambda.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of Wilks' Lambda: The cdf is computed by numerical inversion of the characteristic function or cumulant generating function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", wilks_lambda(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_wilks_lambda.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of Wilks' Lambda:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x) = \int_x^\infty \text{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", wilks_lambda(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_wilks_lambda.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of Wilks' Lambda:

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", wilks_lambda(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_wilks_lambda.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of Wilks' Lambda:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", wilks_lambda(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of Wilks' Lambda:

$$C_X(t) = \frac{\Gamma_p(n/2 - it)\Gamma_p((n+m)/2)}{\Gamma_p(n/2)\Gamma_p((n+m)/2 - it)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wilks_lambda(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of Wilks' Lambda:

$$M_X(t) = \frac{\Gamma_p(n/2 + s)\Gamma_p((n+m)/2)}{\Gamma_p(n/2)\Gamma_p((n+m)/2 + s)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wilks_lambda(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda.**k_x**(*s*, *k=0*)

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following the distribution of`2log W` of central Wilks W:

$$K_X(s) = \log\left[\frac{\Gamma_p(n/2 + s)\Gamma_p((n+m)/2)}{\Gamma_p(n/2)\Gamma_p((n+m)/2 + s)}\right].$$

$$K'(s) = \sum_{i=1}^{p}\left[\psi\left(\tfrac{1}{2}n + s - \tfrac{1}{2}(i-1)\right) - \psi\left(\tfrac{1}{2}(n+m) + s - \tfrac{1}{2}(i-1)\right)\right]$$

where $\Gamma_p(\cdot)$ is the multivariate gamma function and $\psi(\cdot)$ is the digamma function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", wilks_lambda(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of Wilks' Lambda:

$$\mu'_X(r) = \frac{\Gamma_p(n/2+s)\Gamma_p((n+m)/2)}{\Gamma_p(n/2)\Gamma_p((n+m)/2+s)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wilks_lambda(mu, sigma).moments(k))
6.35635234625645256156156145561356E+00
```

dist_wilks_lambda.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of Wilks' Lambda The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wilks_lambda(mu, sigma).cumulants(k))
6.35635234625645256156156145561356E+00
```

Additional information

Anderson, 2003, p. 651-656

p: # of variables

m = q1 = # of groups

M: n-p+1; N=n+q; n = N-q = degrees of freedom (error)

Tables: Renscher 2002, p.566 - 573

Approximations

ctx.**wilks_lambda_gp**(*x*, *p*, *m*, *k*, *results='cdf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the pdf, cdf and sf from the characteristic function using the procedure of Gil-Pelaez (see *gil_pelaez_pdf()* and *gil_pelaez_cdf()*).

This uses $U = \log 2W$.

ctx.**wilks_lambda_ecf**(*x*, *p*, *m*, *n*, *results='cdf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

ctx.**wilks_lambda_ecf_inv**(*q*, *p*, *m*, *n*, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**wilks_lambda_spa**(*(x*, *p*, *n1*, *n2*, *results='c'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

This uses $2 \log W$.

ctx.**wilks_lambda_spa_inv**(*x*, *n*, *results='qtf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

This uses $2 \log W$.

ctx.**wilks_lambda_bd**(*x*, *f*, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Box-Davis approximation to the pdf.

For the Wilks' Lambda distribution, the parameters of the Box-Davis expansion are given vy

$$f = pq; \quad \rho = 1 - \frac{p+q+1}{2(N-1)},$$

$$\omega_r = \frac{(-2)^r}{r(r+1)\mu^r} \sum_{j=0}^{q-1} B_{r+1}((\beta - j)/2) - B_{r+1}((\beta - p - j)/2), \quad \text{where}$$

$$\beta = \frac{p+q+1}{2}; \quad \mu = N - \frac{p+q+3}{2}.$$

wilks_lambda_bd_inv(*q*, *f*, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Box-Davis approximation to the qtf and isf.

4.11.4 Distribution of Wilks' L_{vc}

class ctx.**dist_wilks_cs**(*p*, *bi*, *ci*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

A random variable X follows the distribution of the negative logarithm of the product of p beta variables with parameters a_i and b_i if it is defined as $X = -\log(Y)$, where Y follows a beta product distribution with parameters a_i and b_i. The support interval of X is $(0, +\infty)$.

See also Wilks [1453], Anderson [9], Muirhead [418], Butler [165], Ginzberg [349], pages 92-105, Marques *et al.* [411], Tang and Gupta [513].

dist_wilks_cs.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\text{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\,\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_wilks_cs.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\text{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\,\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_wilks_cs.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function,

only. In particular,

$$\mathrm{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_wilks_cs.**qtf**(q)

Returns $\mathrm{qtf}_X(q)$, the quantile function (qtf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{qtf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

dist_wilks_cs.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{isf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_wilks_cs.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

$$C_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - it)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - it\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_wilks_cs.**m_x**(*t*)

Returns the moment generating function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$M_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - t)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - t\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_wilks_cs.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its r^{th} derivatives, $K_X^{(r)}(t), r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$K_X(t) = \sum_{j=1}^{p} \log\left(\Gamma(a_j - t)\right) - \log\left(\Gamma(a_j + b_j - t)\right) + \log\left(\Gamma(a_j + b_j)\right) - \log\left(\Gamma(a_j)\right).$$

$$K_X^{(r)}(t) = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j - t) - \psi^{(r-1)}(a_j + b_j - t)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_wilks_cs.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_wilks_cs.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_r, r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$\kappa_r = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j) - \psi^{(r-1)}(a_j + b_j)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

4.11.5 Distribution of Wilks' L_{vcm}

class ctx.**dist_wilks_lcm**(*p*, *bi*, *ci*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

A random variable X follows the distribution of the negative logarithm of the product of p beta variables with parameters a_i and b_i if it is defined as $X = -\log(Y)$, where Y follows a beta product distribution with parameters a_i and b_i. The support interval of X is $(0, +\infty)$.

See also Wilks [1453], Anderson [9], Muirhead [418], Butler [165], Ginzberg [349], pages 92-105, Marques *et al.* [411], Tang and Gupta [513].

dist_wilks_lcm.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\text{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\,\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_wilks_lcm.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\text{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\,\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_wilks_lcm.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function,

only. In particular,

$$\mathrm{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145614561356E-20
```

dist_wilks_lcm.**qtf**(q)

Returns $\mathrm{qtf}_X(q)$, the quantile function (qtf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{qtf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145614561356E+00
```

dist_wilks_lcm.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{isf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145614561356E+00
```

dist_wilks_lcm.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

$$C_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - it)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - it\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145614561356E+00
```

dist_wilks_lcm.**m_x**(t)

Returns the moment generating function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$M_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - t)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - t\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_wilks_lcm.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its r^{th} derivatives, $K_X^{(r)}(t), r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$K_X(t) = \sum_{j=1}^{p} \log\left(\Gamma(a_j - t)\right) - \log\left(\Gamma(a_j + b_j - t)\right) + \log\left(\Gamma(a_j + b_j)\right) - \log\left(\Gamma(a_j)\right).$$

$$K_X^{(r)}(t) = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j - t) - \psi^{(r-1)}(a_j + b_j - t)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_wilks_lcm.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_wilks_lcm.**cumulants**(k)

Returns the first j cumulants, $\kappa_r, r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$\kappa_r = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j) - \psi^{(r-1)}(a_j + b_j)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

4.11.6 Distribution of Wilks' test of independence of p variates

class ctx.**dist_wilks_ip**(*p*, *bi*, *ci*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

A random variable X follows the distribution of the negative logarithm of the product of p beta variables with parameters a_i and b_i if it is defined as $X = -\log(Y)$, where Y follows a beta product distribution with parameters a_i and b_i. The support interval of X is $(0, +\infty)$.

See also Wilks [1452], Anderson [9], Muirhead [418], Butler [165], Ginzberg [349], pages 92-105, Marques *et al.* [411], Tang and Gupta [513].

dist_wilks_ip.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\text{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\,\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_wilks_ip.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\text{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\,\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_wilks_ip.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function,

only. In particular,

$$\mathrm{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

`dist_wilks_ip.qtf(`q`)`

Returns $\mathrm{qtf}_X(q)$, the quantile function (qtf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{qtf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

`dist_wilks_ip.isf(`q`)`

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{isf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

`dist_wilks_ip.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

$$C_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - it)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - it\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_wilks_ip.**m_x**(*t*)

Returns the moment generating function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$M_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - t)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - t\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_wilks_ip.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its r^{th} derivatives, $K_X^{(r)}(t), r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$K_X(t) = \sum_{j=1}^{p} \log\left(\Gamma(a_j - t)\right) - \log\left(\Gamma(a_j + b_j - t)\right) + \log\left(\Gamma(a_j + b_j)\right) - \log\left(\Gamma(a_j)\right).$$

$$K_X^{(r)}(t) = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j - t) - \psi^{(r-1)}(a_j + b_j - t)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_wilks_ip.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_wilks_ip.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_r, r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$\kappa_r = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j) - \psi^{(r-1)}(a_j + b_j)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145561356E+00
```

4.11.7 Distribution of Wilks' test of independence of k groups of variates

class ctx.**dist_wilks_iblocks**(*p*, *bi*, *ci*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

A random variable X follows the distribution of the negative logarithm of the product of p beta variables with parameters a_i and b_i if it is defined as $X = -\log(Y)$, where Y follows a beta product distribution with parameters a_i and b_i. The support interval of X is $(0, +\infty)$.

See also Wilks [1452], Anderson [9], Muirhead [418], Butler [165], Ginzberg [349], pages 92-105, Marques *et al.* [411], Tang and Gupta [513].

dist_wilks_iblocks.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\mathrm{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_wilks_iblocks.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\mathrm{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_wilks_iblocks.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function,

only. In particular,

$$\mathrm{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145614561356E-20
```

dist_wilks_iblocks.**qtf**(q)

Returns $\mathrm{qtf}_X(q)$, the quantile function (qtf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{qtf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145614561356E+00
```

dist_wilks_iblocks.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{isf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145614561356E+00
```

dist_wilks_iblocks.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

$$C_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - it)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - it\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145614561356E+00
```

dist_wilks_iblocks.**m_x**(t)

Returns the moment generating function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$M_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - t)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - t\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_wilks_iblocks.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its r^{th} derivatives, $K_X^{(r)}(t), r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$K_X(t) = \sum_{j=1}^{p} \log\left(\Gamma(a_j - t)\right) - \log\left(\Gamma(a_j + b_j - t)\right) + \log\left(\Gamma(a_j + b_j)\right) - \log\left(\Gamma(a_j)\right).$$

$$K_X^{(r)}(t) = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j - t) - \psi^{(r-1)}(a_j + b_j - t)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_wilks_iblocks.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_wilks_iblocks.**cumulants**(k)

Returns the first j cumulants, $\kappa_r, r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$\kappa_r = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j) - \psi^{(r-1)}(a_j + b_j)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

Approximations

`ctx.`**`wilks_iblocks_bd`**`(`x, f, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Given N observationsfrom a p variate normal population, suppose that the variates are partitioned into k groupes of sizes $p_i (i = 1, \ldots, k; \sum p_i = p)$, and it required to test the indepence of the groups

$$\omega = \frac{(-1)^{r+1}}{r(r+1)(r+2)\mu^r} \sum_{s=0}^{r+1} \binom{s+1}{r+2} 2^s \delta_s(pl) \left[\beta^{r+1-s} - \left(\beta - \sum_{n=1}^{l-1} p_n \right)^{r+1-s} \right]$$

$$f = \frac{1}{2} S_2; \quad \rho = 1 - \beta / v; \quad \beta = \frac{2S_3 + 3S_2}{12f}; \quad \mu = v\rho; \quad S_i (\sum_l p_l)^i - \sum_l (p_l)^i.$$

Under the null hypothesis, U is distributed as $U = \prod_{i=2}^{q} \prod_{j=1}^{p(i)} X_{ij}$, where the X_{ij} are independent and X_{ij} has the density $\beta[x|, (n - pp_i + 1 - j)/2, p_i/2]$.

Lit.: Anderson 1984, p.383 and 386

`ctx.`**`wilks_iblocks_bd_inv`**`(`q, f, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Box-Davis approximation to the qtf and isf.

4.11.8 Distribution of Mauchly's test of sphericity vs general structure

class ctx.**dist_mauchley**(*p*, *bi*, *ci*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

A random variable X follows the distribution of the negative logarithm of the product of p beta variables with parameters a_i and b_i if it is defined as $X = -\log(Y)$, where Y follows a beta product distribution with parameters a_i and b_i. The support interval of X is $(0, +\infty)$.

See also Mauchly [412], Anderson [9], Muirhead [418], Butler [165], Ginzberg [349], pages 92-105, Marques *et al.* [411], Tang and Gupta [513].

dist_mauchley.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\text{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\,\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_mauchley.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\text{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\,\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_mauchley.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function,

only. In particular,

$$\mathrm{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145614561356E-20
```

`dist_mauchley.qtf(`q`)`

Returns $\mathrm{qtf}_X(q)$, the quantile function (qtf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{qtf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145614561356E+00
```

`dist_mauchley.isf(`q`)`

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{isf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145614561356E+00
```

`dist_mauchley.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

$$C_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - it)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - it\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.35635234625645256156156145614561356E+00
```

dist_mauchley.**m_x**(*t*)

Returns the moment generating function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$M_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - t)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - t\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_mauchley.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its r^{th} derivatives, $K_X^{(r)}(t), r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$K_X(t) = \sum_{j=1}^{p} \log\left(\Gamma(a_j - t)\right) - \log\left(\Gamma(a_j + b_j - t)\right) + \log\left(\Gamma(a_j + b_j)\right) - \log\left(\Gamma(a_j)\right).$$

$$K_X^{(r)}(t) = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j - t) - \psi^{(r-1)}(a_j + b_j - t)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_mauchley.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_mauchley.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_r, r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$\kappa_r = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j) - \psi^{(r-1)}(a_j + b_j)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

Approximations

`ctx.`**`mauchley_bd`**(*x*, *f*, *rho*, *omega*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

The likelihood criterion for testing the hypothesis that a sample of size N is drawn from a p-variate normal population whose covaraince matrix is proportional to a given matrix Σ_0 is a power of

$$W = |S\Sigma_0^{-1}|^{\frac{1}{2}n} \left(tr(S\Sigma_0^{-1})/p\right)^{\frac{1}{2}pn}$$

where $n = N - 1$ and S is the sample covariance matrix. We have

$$f = \tfrac{1}{2}(p-1)(p+2); \quad \rho = 1 - \frac{2p^2+p+2}{6pn}$$

$$\omega_r = \frac{2(-1)^r}{r(r+1)(r+2)\rho^r} \sum_{s=1}^{r+2} \binom{r+2}{s+1} (1-\rho)^{r+1-s} \frac{\delta_s + \frac{1}{2}(s+1)B_s/p^{s-1}}{(\frac{1}{2}^{s-1})},$$

where B_s are the Bernoulli numbers.

`ctx.`**`mauchley_bd_inv`**(*q*, *f*, *rho*, *omega*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Box-Davis approximation to the qtf and isf.

4.11.9 Distribution of Box's test of equality of covariance matrices, equal sample sizes

class ctx.**dist_box_nsame_cov**(*p*, *bi*, *ci*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

A random variable X follows the distribution of the negative logarithm of the product of p beta variables with parameters a_i and b_i if it is defined as $X = -\log(Y)$, where Y follows a beta product distribution with parameters a_i and b_i. The support interval of X is $(0, +\infty)$.

See also Anderson [9], Muirhead [418], Butler [165], Ginzberg [349], pages 92-105, Marques *et al.* [411], Tang and Gupta [513].

dist_box_nsame_cov.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\text{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\,\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_box_nsame_cov.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\text{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\,\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", fisher_f(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_box_nsame_cov.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function,

only. In particular,

$$\mathrm{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", fisher_f(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

`dist_box_nsame_cov.qtf(`q`)`

Returns $\mathrm{qtf}_X(q)$, the quantile function (qtf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{qtf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", fisher_f(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

`dist_box_nsame_cov.isf(`q`)`

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

There is no known closed exact form for $\mathrm{isf}_X(q)$ or $\mathrm{isf}_X(q)$. It is computed with Newton iterations where the starting values are from Nagarsenker's approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

`dist_box_nsame_cov.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables:

$$C_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - it)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - it\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

dist_box_nsame_cov.**m_x**(t)

Returns the moment generating function of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$M_X(t) = \prod_{j=1}^{p} \frac{\Gamma\left((a_j - t)\right)\Gamma\left((a_j + b_j)\right)}{\Gamma\left(a_j\right)\Gamma\left(a_j + b_j - t\right)}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_box_nsame_cov.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function, and its r^{th} derivatives, $K_X^{(r)}(t), r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$K_X(t) = \sum_{j=1}^{p} \log\left(\Gamma(a_j - t)\right) - \log\left(\Gamma(a_j + b_j - t)\right) + \log\left(\Gamma(a_j + b_j)\right) - \log\left(\Gamma(a_j)\right).$$

$$K_X^{(r)}(t) = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j - t) - \psi^{(r-1)}(a_j + b_j - t)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_box_nsame_cov.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_box_nsame_cov.**cumulants**(k)

Returns the first j cumulants, $\kappa_r, r = 1 \ldots k$, of a random variable X, following the distribution of the negative logarithm of the product of independent beta variables.

$$\kappa_r = (-1)^r \sum_{j=1}^{p} \left(\psi^{(r-1)}(a_j) - \psi^{(r-1)}(a_j + b_j)\right)$$

where $\psi^{(r)}(\cdot)$ is the polygamma function of order r.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", fisher_f(mu, sigma).isf(q))
6.356352346256452561561561561456l356E+00
```

Approximations

`ctx.box_spa`(*x*, *p*, *n1*, *n2*, *results='c'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

This uses $2 \log W$.

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following a Box-Davis distribution:

$$K_X(s) = \log \left[\frac{\Gamma_p(n/2 + s)\Gamma_p((n+m)/2)}{\Gamma_p(n/2)\Gamma_p((n+m)/2 + s)} \right].$$

$$K'(s) = \sum_{i=1}^{p} \left[\psi\left(\tfrac{1}{2}n + s - \tfrac{1}{2}(i-1)\right) - \psi\left(\tfrac{1}{2}(n+m) + s - \tfrac{1}{2}(i-1)\right) \right]$$

where $\Gamma_p(\cdot)$ is the multivariate gamma function and $\psi(\cdot)$ is the digamma function.

`ctx.box_spa_inv`(*x*, *n*, *results='qtf'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

This uses $2 \log W$.

4.11.10 Distribution of Box's test of equality of k covariance matrices, unequal sample sizes

class ctx.**dist_box_cov**(p, k, ni)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Box's test of equality of k covariance matrices is a continuous probability distribution with $p \geq 1$ predictor variables, error degress of freedom $n_i \geq 1$, and the support interval $(0, 1)$. See also Anderson [9], Muirhead [418], Butler [165], Box [35].

dist_box_cov.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Box's LRTs.

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\text{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\,\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", box_cov(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_box_cov.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of Box's LRTs.

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\text{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\,\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", box_cov(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_box_cov.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of Box's LRTs.

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\text{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\,\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", box_cov(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

`dist_box_cov.qtf(`q`)`

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of Box's LRTs:

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", box_cov(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

`dist_box_cov.isf(`q`)`

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of Box's LRTs:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", box_cov(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

`dist_box_cov.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of Box's LRTs:

The characteristic function $C_X(t)$ of $M = -2\log(W)$ is given by

$$C_X(t) = K\left(\frac{\prod_{j=1}^{k} 2y_j^{2y_j}}{\prod_{l=1}^{m} 2x_l^{2x_l}}\right)^{-2it} \frac{\prod_{l=1}^{m}\Gamma[x_l(1-2it)+\xi_l]}{\prod_{j=1}^{k}\Gamma[y_j(1-2it)+\eta_j]}, \text{where}$$

$$K = \frac{\prod_{j=1}^{b}\Gamma(y_j+\eta_j)}{\prod_{k=1}^{a}\Gamma(x_k+\xi_k)}, \quad \text{and} \quad \sum_{k=1}^{a} x_k = \sum_{j=1}^{b} y_j.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", box_cov(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

`dist_box_cov.m_x(`t`)`

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of Box's LRTs:

$$M_X(t) = K\left(\frac{\prod_{j=1}^{k} 2y_j^{2y_j}}{\prod_{l=1}^{m} 2x_l^{2x_l}}\right)^{-2t} \frac{\prod_{l=1}^{m}\Gamma[x_l(1-2t)+\xi_l]}{\prod_{j=1}^{k}\Gamma[y_j(1-2t)+\eta_j]}, \text{where}$$

$$K = \frac{\prod_{j=1}^{b} \Gamma(y_j + \eta_j)}{\prod_{k=1}^{a} \Gamma(x_k + \xi_k)}, \quad \text{and} \quad \sum_{k=1}^{a} x_k = \sum_{j=1}^{b} y_j.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", box_cov(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

`dist_box_cov.`**`k_x`**`(`*s*, *k=0*`)`

Returns $K_X(t)$, the cumulant generating function, and its j^{th} derivative, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs:

The cumulant generating function and the j^{th} derivative of the cumulant generating function of $M = -2\log(W)$ are given by

$$K_X(t) = 2\left(\sum_{l=1}^{m} x_l \log(x_l) - \sum_{r=1}^{k} y_r \log(y_r) - \sum_{l=1}^{m} x_l \psi(x_l(1-2t)+\xi_l) + \sum_{r=1}^{k} y_r \psi(y_r(1-2t)+\eta_r)\right), \quad \text{and}$$

$$K_X^{(j)}(t) = \sum_{l=1}^{m} (-2x_l)^j \psi^{(j-1)}(x_l(1-2t)+\xi_l) - \sum_{r=1}^{k} (-2y_r)^j \psi^{(j-1)}(y_r(1-2t)+\eta_r), \quad j \geq 2,$$

where $\psi(\cdot)$ is the digamma function, and its derivatives $\psi^{(j)}(\cdot)$ are polygamma functions.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", box_cov(mu, sigma).k_x(t, k))
6.356352346256452561561561561456 1356E+00
```

`dist_box_cov.`**`moments`**`(`*k*`)`

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", box_cov(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

`dist_box_cov.`**`cumulants`**`(`*k*`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs

The cumulants κ_j of $M = -2\log(W)$ are given by

$$\kappa_1 = 2\left(\sum_{l=1}^{m} x_l \log(x_l) - \sum_{r=1}^{k} y_r \log(y_r) - \sum_{l=1}^{m} x_l \psi(x_l+\xi_l) + \sum_{r=1}^{k} y_r \psi(y_r+\eta_r)\right), \quad \text{and}$$

$$\kappa_j = \sum_{l=1}^{m} (-2x_l)^j \psi^{(j-1)}(x_l+\xi_l) - \sum_{r=1}^{k} (-2y_r)^j \psi^{(j-1)}(y_r+\eta_r), \quad j \geq 2,$$

where $\psi(\cdot)$ is the digamma function, and its derivatives $\psi^{(j)}(\cdot)$ are polygamma functions.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", box_cov(mu, sigma).cumulants(k))
6.356352346256452561561561561456135 6E+00
```

Additional information

`ctx.log_box_cov_gp(k)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the pdf, cdf and sf from the characteristic function using the procedure of Gil-Pelaez (see *gil_pelaez_pdf()* and *gil_pelaez_cdf()*).

This uses $U = \log 2W$.

In this section we discuss a general class of distributions functions of random variables $M = -2\log(W)$, when the moments of W are products and ratios of gamma functions. W is typically a LRT statistic. M is a special case of the Box-Davis statistic discussed in the previous section.

A random variable $W(0 \le W \le 1)$ is said to be of Box type if its h th moment is given by

$$\mathrm{E}[W^h] = K\left(\frac{\prod_{j=1}^{b} y_j^{y_j}}{\prod_{k=1}^{a} x_k^{x_k}}\right)^h \frac{\prod_{k=1}^{a}\Gamma[x_k(1+h)+\xi_k]}{\prod_{j=1}^{b}\Gamma[y_j(1+h)+\eta_j]}, \quad h = 0, 1, \ldots,$$

where $\Gamma(\cdot)$ denotes the gamma function, K is a constant such that $\mathrm{E}[W^0] = 1$, i.e.

$$K = \frac{\prod_{j=1}^{b}\Gamma(y_j+\eta_j)}{\prod_{k=1}^{a}\Gamma(x_k+\xi_k)}, \quad \text{and} \quad \sum_{k=1}^{a} x_k = \sum_{j=1}^{b} y_j.$$

The l.r.t. statistic λ_6 to test the null hypothesis $H_{04} : \Sigma_1 = \ldots = \Sigma_q$, i.e. the equality of q covariance matrices from p-multivariate normal or elliptically contoured distributions is given by

$$\lambda_6 = \frac{n^{np}}{\prod_{k=1}^{q} n_k^{n_k p}} \frac{\prod_{k=1}^{q}|A_k|^{n_k}}{|A|^n}$$

where N_i the sample size in group i, $n_i = N_i - 1, n = \sum_i n_i$, A_k is equal to n_k times the m.l.e. of $\Sigma_k (k = 1, \ldots, q)$, and $A = A_1 + \ldots + A_q$.

The distribution of M can be expressed as a Box-Davis distribution of the Box type with the following parameter choices for a, b, x_k, ξ_k, y_j and η_j:

$$a = pq, \quad b = p, \quad y_j = n/2, \quad \eta_j = (1-j)/2, \quad j = 1 \ldots b$$

$$x_k = n_g/2, \quad \xi_k = (1-i)/2, \quad k = 1 \ldots a = (g-1)p + i, \quad \text{where } g = 1 \ldots q, \quad i = 1 \ldots p$$

For further details, see Anderson_book_2003, pages 419-420.

Now, let $X_k \sim N_p(\mu_k, \Sigma_k)$, for $k = 1, ..., q$. We want to test the hypothesis that the q normal populations are equally distributed. That is, we want to test that the mean vectors μ_k are equal for all $k = 1, ..., q$, as well as the covariance matrices Σ_k are equal for all $k = 1, ..., q$. Then, the null hypothesis is given as

$H_0 : \mu_1 = ... = \mu_q$ and $\Sigma_1 = ... = \Sigma_k$.

Here, the null hypothesis H_0 and the LRT statistic can be decomposed: $\Lambda = \Lambda_{Means} \times \Lambda_{Covariances}$, where (first) $\Lambda_{Covariances}$ represents the LRT for testing equality of covariance matrices of given q normal populations, and (second) Λ_{Means} represents (conditionally) the LRT for testing equality of means of given q normal populations. Under null hypothesis, distributions of $\Lambda_{Covariances}$ and Λ_{Means} are independent, and the distribution of the test statistic Lambda is $\Lambda = \Lambda_{Means} \times \Lambda_{Covariances}$

Let p be the number of variables, q the number of groups, N_i the sample size in group i, $N = \sum_i N_i$. Then the distribution of M can be expressed as a Box-Davis distribution of the Box type with the following parameter choices for a, b, x_k, ξ_k, y_j and η_j:

$$a = pq, \quad b = p \quad y_j = N/2, \quad \eta_j = -j/2, \quad j = 1 \dots b$$

$$x_k = N_g/2, \quad \xi_k = -i/2, \quad k = 1 \dots a = (g-1)p + i, \quad \text{where } g = 1 \dots q, \quad i = 1 \dots p$$

If the sample sizes are the same, the distribution of the test statistic Lambda is

$$\Lambda = \Lambda_{Means} \times \Lambda_{Covariances}, \sim (\prod_{k=1}^{q} \prod_{j=1}^{p} (B_{jk})^{n/2}) \times (\prod_{j=1}^{p} (B_j)^{nq/2})$$

where the B_{jk} and B_j are mutually independent beta distributed random variables. Here we assume that n is equal sample size for each sample, $k = 1, ..., q, n > p$. For further details, see Anderson_book_2003, pages 420-421.

Approximations

`ctx.box_davis_ecf(`*x*, *p*, *k*, *ni*, *results='cdf'*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

`ctx.box_davis_ecf_inv(`*q*, *p*, *k*, *ni*, *results='cdf'*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

`ctx.box_cov_bd(`*x*, *f*, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Box-Davis approximation to the pdf, cdf and sf.

For Box-type distributions, the parameters of the Box-Davis expansion are given by

Let p be the number of variables, k the number of groups, v_i the sample size in groups i, $N = \sum_i v_i$.

$$\omega = \frac{(-1)^r k}{r(r+1)(r+2)\mu^r} \sum_{s=1}^{r+1} \binom{s+1}{r+2} 2^s \delta_s \gamma_s \beta^{r+1-s}$$

$$\gamma_s = \frac{1}{k} \sum_{i=1}^{k} \left(\frac{v}{v_i}\right)^{s-1} - \frac{1}{k^s}; \quad \text{for equal } v_i : \gamma_s = 1 - \frac{1}{k^s}$$

$$f = \frac{p(p+1)(k-1)}{2}; \quad \rho = \frac{2p^2 + 3p - 1}{6(p+1)(k-1)} \left(-\frac{1}{N} + \sum_{i=1}^{k} \frac{1}{n_i}\right);$$

$$v = \frac{N}{k}; \quad \mu + \rho v = \frac{N\rho}{k}; \quad \beta = (1-\rho)v.$$

Korin (1969), Anderson 1984, p.420

`ctx.box_cov_bd_inv(`*q*, *f*, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Box-Davis approximation to the qtf and isf.

4.11.11 Distribution of Box's test for same multivariate normal distributions, unequal sample sizes

class ctx.**dist_box_mvn**(p, k, ni)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Box's test for same multivariate normal distributions is a continuous probability distribution with $p \geq 1$ predictor variables, error degress of freedom $n_i \geq 1$, and the support interval $(0, 1)$. See also Anderson [9], Muirhead [418], Butler [165], Box [35].

dist_box_mvn.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Box's LRTs.

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\mathrm{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", box_mvn(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_box_mvn.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of Box's LRTs.

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\mathrm{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", box_mvn(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_box_mvn.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of Box's LRTs.

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\mathrm{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", box_mvn(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

`dist_box_mvn.qtf(`q`)`

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of Box's LRTs:

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", box_mvn(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

`dist_box_mvn.isf(`q`)`

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of Box's LRTs:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", box_mvn(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

`dist_box_mvn.c_x(`t`)`

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of Box's LRTs:

The characteristic function $C_X(t)$ of $M = -2\log(W)$ is given by

$$C_X(t) = K\left(\frac{\prod_{j=1}^{k} 2y_j^{2y_j}}{\prod_{l=1}^{m} 2x_l^{2x_l}}\right)^{-2it} \frac{\prod_{l=1}^{m}\Gamma[x_l(1-2it)+\xi_l]}{\prod_{j=1}^{k}\Gamma[y_j(1-2it)+\eta_j]}, \text{where}$$

$$K = \frac{\prod_{j=1}^{b}\Gamma(y_j+\eta_j)}{\prod_{k=1}^{a}\Gamma(x_k+\xi_k)}, \quad \text{and} \quad \sum_{k=1}^{a} x_k = \sum_{j=1}^{b} y_j.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", box_mvn(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

`dist_box_mvn.m_x(`t`)`

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of Box's LRTs:

$$M_X(t) = K\left(\frac{\prod_{j=1}^{k} 2y_j^{2y_j}}{\prod_{l=1}^{m} 2x_l^{2x_l}}\right)^{-2t} \frac{\prod_{l=1}^{m}\Gamma[x_l(1-2t)+\xi_l]}{\prod_{j=1}^{k}\Gamma[y_j(1-2t)+\eta_j]}, \text{where}$$

$$K = \frac{\prod_{j=1}^{b} \Gamma(y_j + \eta_j)}{\prod_{k=1}^{a} \Gamma(x_k + \xi_k)}, \quad \text{and} \quad \sum_{k=1}^{a} x_k = \sum_{j=1}^{b} y_j.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", box_mvn(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

`dist_box_mvn.k_x(`*s*, *k=0*`)`

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs:

The cumulant generating function and the j th derivative of the cumulant generating function ` $M = -2\log(W)$ are given by

$$K_X(t) = 2\left(\sum_{i=1}^{m} x_i \log(x_i) - \sum_{r=1}^{k} y_r \log(y_r) - \sum_{l=1}^{m} x_l \psi(x_l(1-2t)+\xi_l) + \sum_{r=1}^{k} y_r \psi(y_r+\eta_r)\right), \quad \text{and}$$

$$K_X^{(j)}(t) = \sum_{i=1}^{m} (-2x_l)^j \psi^{(j-1)}(x_l(1-2t)+\xi_l) - \sum_{r=1}^{k} (-2y_r(1-2t))^j \psi^{(j-1)}(y_r(1-2t)+\eta_r), \quad j \geq 2,$$

where $\psi(\cdot)$ is the digamma function, and its derivatives $\psi^{(j)}(\cdot)$ are polygamma functions.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", box_mvn(mu, sigma).k_x(t, k))
6.356352346256452561561561456135 6E+00
```

`dist_box_mvn.moments(`*k*`)`

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", box_mvn(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

`dist_box_mvn.cumulants(`*k*`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs

The cumulants κ_j of $M = -2\log(W)$ are given by

$$\kappa_1 = 2\left(\sum_{i=1}^{m} x_i \log(x_i) - \sum_{r=1}^{k} y_r \log(y_r) - \sum_{i=1}^{m} x_i \psi(x_i+\xi_i) + \sum_{r=1}^{k} y_r \psi(y_r+\eta_r)\right), \quad \text{and}$$

$$\kappa_j = \sum_{i=1}^{m} (-2x_i)^j \psi^{(j-1)}(x_i+\xi_i) - \sum_{r=1}^{k} (-2y_r)^j \psi^{(j-1)}(y_r+\eta_r), \quad j \geq 2,$$

where $\psi(\cdot)$ is the digamma function, and its derivatives $\psi^{(j)}(\cdot)$ are polygamma functions.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", box_mvn(mu, sigma).cumulants(k))
6.356352346256452561561561561456135 6E+00
```

Additional information

In this section we discuss a general class of distributions functions of random variables $M = -2\log(W)$, when the moments of W are products and ratios of gamma functions. W is typically a LRT statistic. M is a special case of the Box-Davis statistic discussed in the previous section.

A random variable $W(0 \leq W \leq 1)$ is said to be of Box type if its h th moment is given by

$$\mathrm{E}[W^h] = K\left(\frac{\prod_{j=1}^{b} y_j^{y_j}}{\prod_{k=1}^{a} x_k^{x_k}}\right)^h \frac{\prod_{k=1}^{a}\Gamma[x_k(1+h)+\xi_k]}{\prod_{j=1}^{b}\Gamma[y_j(1+h)+\eta_j]}, \quad h = 0, 1, \ldots,$$

where $\Gamma(\cdot)$ denotes the gamma function, K is a constant such that $\mathrm{E}[W^0] = 1$, i.e.

$$K = \frac{\prod_{j=1}^{b}\Gamma(y_j+\eta_j)}{\prod_{k=1}^{a}\Gamma(x_k+\xi_k)}, \quad \text{and} \quad \sum_{k=1}^{a} x_k = \sum_{j=1}^{b} y_j.$$

The l.r.t. statistic λ_6 to test the null hypothesis $H_{04} : \Sigma_1 = \ldots = \Sigma_q$, i.e. the equality of q covariance matrices from p-multivariate normal or elliptically contoured distributions is given by

$$\lambda_6 = \frac{n^{np}}{\prod_{k=1}^{q} n_k^{n_k p}} \frac{\prod_{k=1}^{q}|A_k|^{n_k}}{|A|^n}$$

where N_i the sample size in group i, $n_i = N_i - 1, n = \sum_i n_i$, A_k is equal to n_k times the m.l.e. of $\Sigma_k (k = 1, \ldots, q)$, and $A = A_1 + \ldots + A_q$.

The distribution of M can be expressed as a Box-Davis distribution of the Box type with the following parameter choices for a, b, x_k, ξ_k, y_j and η_j:

$$a = pq, \quad b = p, \quad y_j = n/2, \quad \eta_j = (1-j)/2, \quad j = 1 \ldots b$$

$$x_k = n_g/2, \quad \xi_k = (1-i)/2, \quad k = 1 \ldots a = (g-1)p + i, \quad \text{where } g = 1 \ldots q, \quad i = 1 \ldots p$$

For further details, see Anderson_book_2003, pages 419-420.

Now, let $X_k \sim N_p(\mu_k, \Sigma_k)$, for $k = 1, ..., q$. We want to test the hypothesis that the q normal populations are equally distributed. That is, we want to test that the mean vectors μ_k are equal for all $k = 1, ..., q$, as well as the covariance matrices Σ_k are equal for all $k = 1, ..., q$. Then, the null hypothesis is given as

$H_0 : \mu_1 = ... = \mu_q$ and $\Sigma_1 = ... = \Sigma_k$.

Here, the null hypothesis H_0 and the LRT statistic can be decomposed: $\Lambda = \Lambda_{Means} \times \Lambda_{Covariances}$, where (first) $\Lambda_{Covariances}$ represents the LRT for testing equality of covariance matrices of given q normal populations, and (second) Λ_{Means} represents (conditionally) the LRT for testing equality of means of given q normal populations. Under null hypothesis, distributions of $\Lambda_{Covariances}$ and Λ_{Means} are independent, and the distribution of the test statistic Lambda is $\Lambda = \Lambda_{Means} \times \Lambda_{Covariances}$

Let p be the number of variables, q the number of groups, N_i the sample size in group i, $N = \sum_i N_i$. Then the distribution of M can be expressed as a Box-Davis distribution of the Box type with the following parameter choices for a, b, x_k, ξ_k, y_j and η_j:

$$a = pq, \quad b = p \quad y_j = N/2, \quad \eta_j = -j/2, \quad j = 1 \ldots b$$

$$x_k = N_g/2, \quad \xi_k = -i/2, \quad k = 1 \ldots a = (g-1)p + i, \quad \text{where } g = 1 \ldots q, \quad i = 1 \ldots p$$

If the sample sizes are the same, the distribution of the test statistic Lambda is

$$\Lambda = \Lambda_{Means} \times \Lambda_{Covariances}, \sim (\prod_{k=1}^{q}\prod_{j=1}^{p}(B_{jk})^{n/2}) \times (\prod_{j=1}^{p}(B_j)^{nq/2})$$

where the B_{jk} and B_j are mutually independent beta distributed random variables. Here we assume that n is equal sample size for each sample, $k = 1, ..., q, n > p$. For further details, see Anderson_book_2003, pages 420-421.

Approximations

`ctx.box_means_cov_bd(`x, f, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Box-Davis approximation to the pdf, cdf and sf.

For Box-type distributions, the parameters of the Box-Davis expansion are given by

$$`\theta = \min_{i,j}(x_i, y_j)`,$$

$$f = -2\left[\sum_{k=1}^{a}\xi_k - \sum_{j=1}^{b}\eta_j - \tfrac{1}{2}(a-b)\right],$$

$$\rho = 1 - \frac{1}{f}\left[\sum_{k=1}^{a}x_k^{-1}\left(\xi_k^2 - \xi_k + \tfrac{1}{6}\right) - \sum_{j=1}^{b}y_j^{-1}\left(\eta_j^2 - \eta_j + \tfrac{1}{6}\right)\right],$$

$$\omega_r = \frac{(-1)^{r+1}}{r(r+1)}\left[\sum_{k=1}^{a}\frac{B_{r+1}((1-\rho)x_k + \xi_k)}{(\rho x_k)^r} - \sum_{j=1}^{b}\frac{B_{r+1}((1-\rho)y_j + \eta_j)}{(\rho y_j)^r}\right],$$

$$\begin{aligned}\log(K_B) &= \log(K) + \frac{1}{2}(m-k)\log(2\pi) - \frac{f}{2}\log(\rho) \\ &\quad + \sum_{i=1}^{m}\left(x_i + \xi_i - \frac{1}{2}\right)\log(x_i) - \sum_{i=1}^{k}\left(y_i + \eta_i - \frac{1}{2}\right)\log(x_i)\end{aligned} \tag{4.39}$$

See Siotani p. 356

`ctx.box_means_cov_bd_inv(`q, f, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Box-Davis approximation to the qtf and isf.

4.11.12 Distribution of the modified likelihood ratio test (LRT) for a given covariance matrix

`class ctx.dist_lrt_s0(`p, n`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The distribution of the modified (LRT) for a given covariance matrix is a continuous probability distribution with $p \geq 1$ predictor variables, error degress of freedom $n \geq 1$, and the support interval $(0, 1)$. See also Anderson [9], Davis [204].

The modified likelihood criterion λ_1^* for testing the hypothesis that a sample of size N is drawn from a p-variate normal population with a given covariance matrix $\boldsymbol{\Sigma}_0$ is given by

$$\lambda_1^* = e^{\frac{1}{2}pn}\left|\boldsymbol{S}\boldsymbol{\Sigma}_0^{-1}\right|^{\frac{1}{2}n}\exp\left(-\tfrac{1}{2}n\,\mathrm{tr}\left(\boldsymbol{S}\boldsymbol{\Sigma}_0^{-1}\right)\right)$$

where $n = N - 1$ and $\boldsymbol{S}$ is the sample covariance matrix.

`dist_lrt_s0.pdf(x)`

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Box's LRTs.

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\text{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\,\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", lrt_s0(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

`dist_lrt_s0.cdf(x)`

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of Box's LRTs.

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\text{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\,\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", lrt_s0(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

`dist_lrt_s0.sf(x)`

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of Box's LRTs.

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\text{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\,\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", lrt_s0(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

`dist_lrt_s0.qtf(q)`

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of Box's LRTs:

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", lrt_s0(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

`dist_lrt_s0.isf(q)`

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of Box's LRTs:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", lrt_s0(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

`dist_lrt_s0.c_x(t)`

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of Box's LRTs:

The characteristic function of $M = -2\log(\lambda_1^*)$ has the form (see Anderson 2003, page 441, equation 16)

$$C_X(t) = \left(\frac{2e}{n}\right)^{-it\cdot pn} (1-2it)^{-\frac{1}{2}pn(1-2it)} \prod_{j=1}^{p} \frac{\Gamma\left[\frac{1}{2}(n+1-j) - it\cdot n\right]}{\Gamma\left[\frac{1}{2}(n+1-j)\right]}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", lrt_s0(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_lrt_s0.m_x(t)`

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of Box's LRTs:

$$M_X(t) = \left(\frac{2e}{n}\right)^{-t\cdot pn} (1-2t)^{-\frac{1}{2}pn(1-2t)} \prod_{j=1}^{p} \frac{\Gamma\left[\frac{1}{2}(n+1-j) - t\cdot n\right]}{\Gamma\left[\frac{1}{2}(n+1-j)\right]}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", lrt_s0(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_lrt_s0.**k_x**(*s*, *k=0*)

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs:

$$K_X(t) = \log(M_X(t))$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", lrt_s0(mu, sigma).k_x(t, k))
6.356352346256452561561561561456 1356E+00
```

dist_lrt_s0.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", lrt_s0(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

dist_lrt_s0.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs

The cumulants κ_j of $M = -2\log(W)$ are given by

$$\kappa_1 = tbd$$

$$\kappa_j = tbd$$

where $\psi(\cdot)$ is the digamma function, and its derivatives $\psi^{(j)}(\cdot)$ are polygamma functions.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", lrt_s0(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

Additional information

We have for the Box-Davis expansion (see Davis 1971, equations 20-21):

$$f = \tfrac{1}{2}p(p+1); \quad \rho = 1 - \frac{2p^2+3p-1}{6n(p+1)}$$

$$\omega_r = \frac{2(-1)^r}{r(r+1)(r+2)\rho^r} \sum_{s=1}^{r+1} \binom{r+2}{s+1} (1-\rho)^{r+1-s} \frac{\delta_s}{(\frac{1}{2}n)^{s-1}},$$

where δ is defined in Box (1949). For tables see Korin 1968 and Davis 1971.

Approximations

ctx.**lrt_vc0_bd**(q, f, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Box-Davis approximation to the qtf and isf.

The modified likelihood criterion λ_1^* for testing the hypothesis that a sample of size N is drawn from a p-variate normal population with a given covariance matrix $\mathbf{\Sigma}_0$ is given by

$$\lambda_1^* = e^{\frac{1}{2}pn} \left|\boldsymbol{S}\mathbf{\Sigma}_0^{-1}\right|^{\frac{1}{2}n} \exp\left(-\tfrac{1}{2}n \operatorname{tr}\left(\boldsymbol{S}\mathbf{\Sigma}_0^{-1}\right)\right)$$

where $n = N - 1$ and $\boldsymbol{S}$ is the sample covariance matrix.

We have for the Box-Davis expansion (see Davis 1971, equations 20-21):

$$f = \tfrac{1}{2}p(p+1); \quad \rho = 1 - \frac{2p^2 + 3p - 1}{6n(p+1)}; \quad \delta_{s,p} = -\frac{s+1}{2}\sum_{j=0}^{p-1} B_s(-j/2)$$

$$\omega_r = \frac{2(-1)^r}{r(r+1)(r+2)\rho^r} \sum_{s=1}^{r+1} \binom{r+2}{s+1} (1-\rho)^{r+1-s} \frac{\delta_{s,p}}{(\frac{1}{2}n)^{s-1}},$$

where $B_s(\cdot)$ is the Bernoulli polynomial of degree s. For tables see Korin 1968 and Davis 1971.

ctx.**lrt_vc0_bd_inv**(q, f, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Box-Davis approximation to the qtf and isf.

4.11.13 Distribution of the modified likelihood ratio test (LRT) for a given covariance matrix and mean vector

class ctx.**dist_lrt_x0_s0**(p, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of the modified (LRT) for a given covariance matrix and mean vector is a continuous probability distribution with $p \geq 1$ predictor variables, error degress of freedom $n \geq 1$, and the support interval $(0, 1)$. See also Anderson [9], Davis [204]. For tables see Nagarsenker 1974

The likelihood criterion λ_2 for testing the hypothesis that a sample of size N is drawn from a p-variate normal population with a given covariance matrix $\boldsymbol{\Sigma}_0$ and mean vector $\boldsymbol{\mu}_0$ is given by

$$\lambda_2 = \left(\frac{e}{N}\right)^{\frac{1}{2}pN} \left|\boldsymbol{S}\boldsymbol{\Sigma}_0^{-1}\right|^{\frac{1}{2}N} \exp\left(-\tfrac{1}{2}\left[\operatorname{tr}\left(\boldsymbol{S}\boldsymbol{\Sigma}_0^{-1}\right) + N(\bar{\boldsymbol{x}} - \boldsymbol{\mu}_0)'\boldsymbol{\Sigma}_0(\bar{\boldsymbol{x}} - \boldsymbol{\mu}_0)\right]\right)$$

where $\boldsymbol{S}$ is the sample covariance matrix and $\bar{\boldsymbol{x}}$ is the sample mean vector.

dist_lrt_x0_s0.**pdf**(x)

Returns $\operatorname{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Box's LRTs.

The pdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\operatorname{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", lrt_x0_s0(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_lrt_x0_s0.**cdf**(x)

Returns $\operatorname{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of Box's LRTs.

The cdf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\operatorname{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", lrt_x0_s0(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

`dist_lrt_x0_s0.sf`(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of Box's LRTs.

The sf can be calculated (in principle in arbitrary precision) by numerical inversion of the characteristic function, using the algorithm by Gil-Pelaez. Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function, only. In particular,

$$\mathrm{sf}_X(x) = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", lrt_x0_s0(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

`dist_lrt_x0_s0.qtf`(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of Box's LRTs:

There is no known closed form for the quantile function $\mathrm{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", lrt_x0_s0(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

`dist_lrt_x0_s0.isf`(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of Box's LRTs:

$$\mathrm{isf}_X(q) = \mathrm{qtf}_X(1-q) = \mathrm{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", lrt_x0_s0(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

`dist_lrt_x0_s0.c_x`(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of Box's LRTs:

The characteristic function of $M = -2\log(\lambda_2)$ has the form (Nagarsenker 1974, equation 2.3)

$$C_X(t) = \frac{(2e/N)^{-it\cdot pN}}{(1-2it)^{-\frac{1}{2}pN(1-2it)}}\prod_{j=1}^{p}\frac{\Gamma\left[\frac{1}{2}N(1-2it)-j/2\right]}{\Gamma\left[\frac{1}{2}(N-j)\right]}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", lrt_x0_s0(mu, sigma).c_x(t))
6.3563523462564525615615615614561356E+00
```

dist_lrt_x0_s0.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of Box's LRTs:

$$M_X(t) = \frac{(2e/N)^{-t \cdot pN}}{(1-2t)^{-\frac{1}{2}pN(1-2t)}} \prod_{j=1}^{p} \frac{\Gamma\left[\frac{1}{2}N(1-2t) - j/2\right]}{\Gamma\left[\frac{1}{2}(N-j)\right]}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", lrt_x0_s0(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_lrt_x0_s0.**k_x**(s, *k=0*)

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs:

$$M_X(t) = \log(M_X(t))$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", lrt_x0_s0(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_lrt_x0_s0.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", lrt_x0_s0(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_lrt_x0_s0.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of Box's LRTs

The cumulants κ_j of $M = -2\log(W)$ are given by

$$\kappa_1 = tbd$$

$$\kappa_j = tbd$$

where $\psi(\cdot)$ is the digamma function, and its derivatives $\psi^{(j)}(\cdot)$ are polygamma functions.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", lrt_x0_s0(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Additional information

We have for the Box-Davis expansion (see Davis 1971, equations 23-24):

$$f = \tfrac{1}{2}p(p+3); \quad \rho = 1 - \frac{2p^2+9p+11}{6N(p+3)}$$

$$\omega_r = \frac{2(-1)^r}{r(r+1)(r+2)\rho^r}\sum_{s=1}^{r+1}\binom{r+2}{s+1}(1-\rho-N^{-1})^{r+1-s}\frac{\delta_s + p(s+1)/2^{s+1}}{(\frac{1}{2}N)^{s-1}},$$

where δ is defined in Box (1949). For tables see Nagarsenker 1974

Approximations

`ctx.`**`lrt_x0_vc0_bd`**`(`q, f, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

We have for the Box-Davis expansion (see Davis 1971, equations 23-24):

$$f = \tfrac{1}{2}p(p+3); \quad \rho = 1 - \frac{2p^2+9p+11}{6N(p+3)}; \quad \delta_{s,p} = -\frac{s+1}{2}\sum_{j=0}^{p-1}B_s(-j/2)$$

$$\omega_r = \frac{2(-1)^r}{r(r+1)(r+2)\rho^r}\sum_{s=1}^{r+1}\binom{r+2}{s+1}(1-\rho-N^{-1})^{r+1-s}\frac{\delta_{s,p} + p(s+1)/2^{s+1}}{(\frac{1}{2}N)^{s-1}},$$

where $B_s(\cdot)$ is the Bernoulli polynomial of degree s. . For tables see Nagarsenker 1974

`ctx.`**`lrt_x0_vc0_bd_inv`**`(`q, f, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Box-Davis approximation to the qtf and isf.

4.11.14 Central distribution of Roy's largest root

class ctx.**dist_roy**(p, m, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Roy's largest root, with parameters p, m and n, and the support interval $(0, 1)$. See also Anderson [9], Muirhead [418], Butler [165], Chiani [177], Chiani [178].

dist_roy.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Roy's largest root:

$$\text{pdf}_X(x) = \tfrac{1}{2} C \sqrt{|A(\theta_1)|}\ \text{tr}\left(A(t)^{-1} B\right).$$

A and C are defined as in *roy_pdf_cdf_sf()*.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", roy_largest_root(mu, sigma).pdf(x))
6.356352346256452561561561561456 1356E-20
```

dist_roy.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of Roy's largest root:

$$\text{cdf}_X(x) = C \sqrt{|A(\theta_1)|}.$$

A and C are defined as in *roy_pdf_cdf_sf()*.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", roy_largest_root(mu, sigma).pdf(x))
6.356352346256452561561561561456 1356E-20
```

dist_roy.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of Roy's largest root:

$$\text{sf}_X(x) = 1 - C \sqrt{|A(\theta_1)|}.$$

A and C are defined as in *roy_pdf_cdf_sf()*.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", roy_largest_root(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456 1356E-20
```

dist_roy.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of Roy's largest root:

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation by Chiani.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", roy_largest_root(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_roy.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of Roy's largest root:

There is no known closed form for the quantile function $\text{isf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation by Chiani.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", roy_largest_root(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_roy.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of Roy's largest root:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", roy_largest_root(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_roy.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of Roy's largest root:

$$M_X(t) = \int_0^\infty e^{tx} \text{pdf}_X(x) \mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", roy_largest_root(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_roy.**k_x**(s, *k=0*)

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following the distribution of Roy's largest root:

$$K_X(t) = \log\left(M_X(t)\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", roy_largest_root(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_roy.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of Roy's largest root:

$$\mu'_X(r) = \int_0^\infty x^r \mathrm{pdf}_X(x)\mathrm{d}x$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", roy_largest_root(mu, sigma).moments(k))
6.356352346256452561561561561456135 6E+00
```

dist_roy.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of Roy's largest root. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", roy_largest_root(mu, sigma).cumulants(k))
6.356352346256452561561561561456135 6E+00
```

4.11.15 Central distribution of Pillai's V

class ctx.**dist_pillai_v**(p, m, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Pillai V distribution is a continuous probability distribution with $p \geq 1$ predictor variables, error degress of freedom $m \geq 1$ and $n \geq 1$, and the support interval $(0, 1)$. See also Anderson [9], Muirhead [418], Butler [165], Davis [202], Davis [203], Davis [204].

dist_pillai_v.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Pillai's V:

The pdf is computed by numerical inversion of the characteristic function or cumulant generating function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", pillai_v(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_pillai_v.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of Pillai's V:

The cdf is computed by numerical inversion of the characteristic function or cumulant generating function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", pillai_v(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_pillai_v.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of Pillai's V:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x) = \int_x^\infty \text{pdf}_X(x)\text{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", pillai_v(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_pillai_v.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of Pillai's V:

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", pillai_v(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_pillai_v.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of Pillai's V:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", pillai_v(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_pillai_v.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of Pillai's V:

```

```

TO BE CALCULATED VIA RAW MOMENTS

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", pillai_v(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_pillai_v.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of Pillai's V:

```
TO BE CALCULATED VIA RAW MOMENTS
```

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", pillai_v(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_pillai_v.**k_x**(s, *k=0*)

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following the distribution of Pillai's V:

```

```

TO BE CALCULATED VIA RAW MOMENTS

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", pillai_v(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_pillai_v.**moments**(k)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of Pillai's V:

```
TO BE CALCULATED AS IN DAVIS
```

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", pillai_v(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_pillai_v.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of Pillai's V. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", pillai_v(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Approximations

ctx.**pillai_v_ecf**(x, p, m, n, *results='cdf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

ctx.**pillai_v_ecf_inv**(q, p, m, n, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**pillai_v_bd**(x, f, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Box-Davis approximation to the pdf, cdf and sf.

For Pillai's V distribution, the parameters of the Box-Davis expansion are given by

Let $s = 1, \quad k = m + 1, \quad a = 2k + n_1$. Then

$$f = pq; \quad \rho = 1.$$

$$\omega_1 = mn_1k/(2n_2), \quad 2r\omega_r = 2(r-1)\omega_r - 1 - s(1-k/n_2)c_{1,r}, \quad r = 2, 3, \ldots$$

$$c_{0,0} = 1; \quad c_{0,r} = c_{r,0} = 0; \quad (r = 1, 2, \ldots); \quad c_{r,1} = 0 (r = 2, 3, \ldots)$$

$$\begin{aligned} jc_{j,r} &= [(m-j+1)(n_1-j+1)]c_{j-1,r-1} + [(j(2m+n_1-2j+2)+2(r-1))/n2]c_{j,r-1} \\ &+ [(j+1)/n_2 - (j+1)(m-j+1)/n_2^2]c_{j+1,r-1} - [(mn_1+2(r-2)/n_2]c_{j,r-2} \\ &+ (2/n_2)\sum_{i=1}^{r-2} i\omega_i(c_{j,r-i-1} - c_{j,r-i-2}) \end{aligned}$$

ctx.**pillai_v_bd_inv**(q, f, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Box-Davis approximation to the qtf and isf.

4.11.16 Central distribution of Hotelling's T^2

class ctx.**dist_hotelling_t2**(p, m, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Hotelling T^2 distribution is a continuous probability distribution with $p \geq 1$ predictor variables, error degress of freedom $m \geq 1$ and $n \geq 1$, and the support interval $(0, 1)$. See also Anderson [9], Muirhead [418], Butler [165], Davis [202], Davis [203], Davis [204].

dist_hotelling_t2.**pdf**(x)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Hotelling's T^2:

The pdf is computed by numerical inversion of the characteristic function or cumulant generating function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", hotelling_t2(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_hotelling_t2.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of Hotelling's T^2:

The cdf is computed by numerical inversion of the characteristic function or cumulant generating function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", hotelling_t2(mu, sigma).pdf(x))
6.35635234625645256156156145613 56E-20
```

dist_hotelling_t2.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of Hotelling's T^2:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x) = \int_x^\infty \text{pdf}_X(x)\text{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", hotelling_t2(mu, sigma).pdf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_hotelling_t2.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of Hotelling's T^2:

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", hotelling_t2(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_hotelling_t2.**isf**(*q*)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of Hotelling's T^2:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", hotelling_t2(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_hotelling_t2.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of Hotelling's T^2:

```
TO BE CALCULATED VIA RAW MOMENTS
```

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", hotelling_t2(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_hotelling_t2.**m_x**(*t*)

Does not exist

dist_hotelling_t2.**k_x**(*s*, *k=0*)

Does not exist

dist_hotelling_t2.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \dots k$, of a random variable X, following the distribution of Hotelling's T^2:

Returns the first k moments of the Lawley-Hotelling generalized T_0^2 statistic. The moments of Hotelling's T_0^2 exist up to the j^{th}, where j is the largest integer such that $j < \frac{1}{2}(n_2 - m + 1)$. The raw moments are determined as follows (see Davis [202]):

$$E(T^r) = (-1)^r r!(n_1+n_2)! \sum_{k=0}^{m} \frac{l_{kr}}{(m+n_2-k)!}, \quad \text{where}$$

$$\boldsymbol{l_i} = (l_{0i}, \dots, l_{mi})', \quad a_i = \tfrac{1}{2}(m-i)(n_2-i),$$

$$\boldsymbol{l_0} = \frac{n_2!}{(n_1+n_2)!}(0, \dots, 0, 1)',$$

$$\boldsymbol{l_r} = \text{diag}\left(\frac{1}{(r-a_0)}, \dots, \frac{1}{(r-a_m)}\right) \sum_{s=0}^{r-1} \boldsymbol{l_s},$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", hotelling_t2(mu, sigma).moments(k))
6.356352346256452561561561561456135 6E+00
```

dist_hotelling_t2.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of Hotelling's T^2. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", hotelling_t2(mu, sigma).cumulants(k))
6.356352346256452561561561561456135 6E+00
```

Approximations

ctx.**hotelling_t2_ecf**(*x*, *p*, *m*, *n*, *results='cdf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

ctx.**hotelling_t2_ecf_inv**(*q*, *p*, *m*, *n*, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**hotelling_t2_bd**(*x*, *f*, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Box-Davis approximation to the pdf, cdf and sf.

For Hotelling's T^2 distribution, the parameters of the Box-Davis expansion are given vy

The coefficients for the Box-Davis expansion are calculated as follows:

Let $s = -1, \quad k = -n_1, \quad a = 2n_1 + m + 1$. Then

$$f = pq; \quad \rho = 1.$$

$$\omega_1 = mn_1k/(2n_2), \quad 2r\omega_r = 2(r-1)\omega_r - 1 - s(1-k/n_2)c_{1,r}, \quad r = 2, 3, \ldots$$

$$c_{0,0} = 1; \quad c_{0,r} = c_{r,0} = 0; \quad (r = 1, 2, \ldots); \quad c_{r,1} = 0 (r = 2, 3, \ldots)$$

$$\begin{aligned} jc_{j,r} &= [(m-j+1)(n_1-j+1)]c_{j-1,r-1} + [(j(2m+n_1-2j+2)+2(r-1))/n2]c_{j,r-1} \\ &+ [(j+1)/n_2 - (j+1)(m-j+1)/n_2^2]c_{j+1,r-1} - [(mn_1+2(r-2)/n_2]c_{j,r-2} \\ &+ (2/n_2)\sum_{i=1}^{r-2} i\omega_i(c_{j,r-i-1} - c_{j,r-i-2}) \end{aligned}$$

ctx.**hotelling_t2_bd_inv**(*q*, *f*, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Box-Davis approximation to the qtf and isf.

4.11.17 Noncentral Distribution of Wilks' Λ: MANOVA

class ctx.**dist_wilks_lambda_glm**(*p*, *m*, *n*, *omega*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The noncentral Wilks' Λ distribution under the GLM alternative, with $p \geq 1$ predictor variables, error degress of freedom $m \geq 1$ and $n \geq 1$, noncentrality parameter Ω with diagonal entries $\omega_{jj} \in (0, \infty)$ is a continuous probability distribution with the support interval $(0, 1)$. See also Anderson [9], Muirhead [418], Butler [165], Butler and Wood [167], Butler and Wood [168], Fujikoshi [338], Fujikoshi [339], Fujikoshi [340], Lee [405], Lee [406], Walster and Tretter [791], Witkovský [1490].

dist_wilks_lambda_glm.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the noncentral distribution of Wilks' Lambda (GLM): The pdf is computed by numerical inversion of the characteristic function or cumulant generating function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", wilks_lambda_glm(mu, sigma).pdf(x))
6.356352346256452561561561561456 1356E-20
```

dist_wilks_lambda_glm.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the noncentral distribution of Wilks' Lambda (GLM): The cdf is computed by numerical inversion of the characteristic function or cumulant generating function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", wilks_lambda_glm(mu, sigma).pdf(x))
6.356352346256452561561561561456 1356E-20
```

dist_wilks_lambda_glm.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following the noncentral distribution of Wilks' Lambda (GLM):

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^\infty \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", wilks_lambda_glm(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456 1356E-20
```

dist_wilks_lambda_glm.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the noncentral distribution of Wilks' Lambda (GLM):

There is no known closed form for the quantile function $\mathrm{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", wilks_lambda_glm(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda_glm.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the noncentral distribution of Wilks' Lambda (GLM):

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", wilks_lambda_glm(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda_glm.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the noncentral distribution of Wilks' Lambda (GLM):

$$C_X(t) = \frac{\Gamma_p(n/2-it)\Gamma_p((n+m)/2)}{\Gamma_p(n/2)\Gamma_p((n+m)/2-it)} {}_1F_1\left(-it; \frac{n+m}{2} + -it; -\frac{1}{2}\Omega\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wilks_lambda_glm(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda_glm.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the noncentral distribution of Wilks' Lambda (GLM):

$$M_X(t) = \left[\frac{\Gamma_p(n/2+s)\Gamma_p((n+m)/2)}{\Gamma_p(n/2)\Gamma_p((n+m)/2+s)} {}_1F_1\left(s, \tfrac{1}{2}(n+m)+s, -\tfrac{1}{2}\Omega\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wilks_lambda_glm(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda_glm.**k_x**(s, *k=0*)

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following the noncentral distribution of Wilks' Lambda (GLM):

$$K_X(s) = \log\left[\frac{\Gamma_p(n/2+s)\Gamma_p((n+m)/2)}{\Gamma_p(n/2)\Gamma_p((n+m)/2+s)} {}_1F_1\left(s, \tfrac{1}{2}(n+m)+s, -\tfrac{1}{2}\Omega\right)\right].$$

$$K'(s) = \sum_{i=1}^{p} \left[\psi\left(\tfrac{1}{2}n + s - \tfrac{1}{2}(i-1)\right) - \psi\left(\tfrac{1}{2}(n+m) + s - \tfrac{1}{2}(i-1)\right)\right] + \frac{\partial}{\partial s} \log\left[{}_1F_1\left(s, \tfrac{1}{2}(n+m) + s, -\tfrac{1}{2}\Omega\right)\right],$$

where $\Gamma_p(\cdot)$ is the multivariate gamma function , $\psi(\cdot)$ is the digamma function , and ${}_1F_1(\cdot, \cdot, X)$ is the confluent hypergeometric function of matrix argument .

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", wilks_lambda_glm(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

`dist_wilks_lambda_glm.`**`moments`**`(`*k*`)`

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the noncentral distribution of Wilks' Lambda (GLM):

$$\mu'_X(r) = \frac{\Gamma_p(n/2 + s)\Gamma_p((n+m)/2)}{\Gamma_p(n/2)\Gamma_p((n+m)/2 + s)} {}_1F_1\left(s; \frac{n+m}{2} + s; -\frac{1}{2}\Omega\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wilks_lambda_glm(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

`dist_wilks_lambda_glm.`**`cumulants`**`(`*k*`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the noncentral distribution of Wilks' Lambda (GLM) The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wilks_lambda_glm(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Approximations

`ctx.`**`wilks_lambda_glm_gp`**`(`*x*, *p*, *m*, *n*, *Omega*, *cdf=True*, *method='default'*`)`

where `ctx` is `dec`, `mpm`, `fpm`, `gmp` or `apm`.

Calculates the pdf, cdf and sf from the characteristic function using the procedure of Gil-Pelaez (see *gil_pelaez_pdf()* and *gil_pelaez_cdf()*).

This uses $U = \log 2W$.

`ctx.`**`wilks_lambda_glm_spa`**`(`*x*, *n*, *results='c'*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

This uses $2 \log W$ of non-central Wilks W

ctx.**wilks_lambda_glm_inv**(*x*, *n*, *results='qtf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

This uses $2 \log W$ of non-central Wilks W

4.11.18 Noncentral Distribution of Wilks' Λ: Canonical Correlation

class ctx.**dist_wilks_lambda_corr**(*p1*, *p2*, *n*, *Rho2*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The noncentral Wilks' Λ distribution under the independence alternative, with $p_1 \geq 1$ and $p_2 \geq 1$ groups of variables, error degress of freedom $n \geq 1$, noncentrality parameter P^2 with diagonal entries $\rho_{jj}^2 \in (0, 1)$ is a continuous probability distribution with the support interval $(0, 1)$. See also Anderson [9], Muirhead [418], Butler [165], Butler and Wood [167], Butler and Wood [168], Fujikoshi [338], Fujikoshi [339], Lee [405], Lee [406], Walster and Tretter [791], Witkovský [1483].

dist_wilks_lambda_corr.**pdf**(*x*, *kwargs*)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the noncentral distribution of Wilks' Lambda (CORR): The pdf is computed by numerical inversion of the characteristic function or cumulant generating function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", wilks_lambda_corr(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_wilks_lambda_corr.**cdf**(*x*, *kwargs*)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the noncentral distribution of Wilks' Lambda (CORR): The cdf is computed by numerical inversion of the characteristic function or cumulant generating function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", wilks_lambda_corr(mu, sigma).pdf(x))
6.3563523462564525615615615614561356E-20
```

dist_wilks_lambda_corr.**sf**(*x*)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following the noncentral distribution of Wilks' Lambda (CORR):

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x) = \int_x^\infty \text{pdf}_X(x)\text{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", wilks_lambda_corr(mu, sigma).pdf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_wilks_lambda_corr.**qtf**(*q*)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the noncentral distribution of Wilks' Lambda (CORR):

There is no known closed form for the quantile function $\text{cdf}_X^{-1}(q)$: It is computed with Newton iterations where the starting values are from a central chi-square approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", wilks_lambda_corr(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda_corr.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the noncentral distribution of Wilks' Lambda (CORR):

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", wilks_lambda_corr(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda_corr.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following the noncentral distribution of Wilks' Lambda (CORR):

$$C_X(t) = \frac{\Gamma_{p_1}(n/2)\Gamma_{p_1}((n-p_2)/2 - it)}{\Gamma_{p_1}(n/2 - it)\Gamma_{p_1}((n-p_2)/2)} \times |I_{p_1} - P^2|^{n/2} {}_2F_1\left(\frac{n}{2}, \frac{n}{2}; \frac{n}{2} - it; P^2\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wilks_lambda_corr(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda_corr.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following the noncentral distribution of Wilks' Lambda (CORR):

$$M_X(t) = \left[\frac{\Gamma_{p_1}(n/2)\Gamma_{p_1}((n-p_2)/2 + s)}{\Gamma_{p_1}(n/2 + s)\Gamma_{p_1}((n-p_2)/2)} \times |I_{p_1} - P^2|^{n/2} {}_2F_1\left(\frac{n}{2}, \frac{n}{2}; \frac{n}{2} + s; P^2\right)\right].$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wilks_lambda_corr(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wilks_lambda_corr.**k_x**(s, *k=0*)

Returns $K_X(s)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(s), j = 1 \ldots k$, of a random variable X, following the noncentral distribution of Wilks' Lambda (CORR):

$$K_X(s) = \log\left[\frac{\Gamma_{p_1}(n/2)\Gamma_{p_1}((n-p_2)/2 + s)}{\Gamma_{p_1}(n/2 + s)\Gamma_{p_1}((n-p_2)/2)} \times |I_{p_1} - P^2|^{n/2} {}_2F_1\left(\frac{n}{2}, \frac{n}{2}; \frac{n}{2} + s; P^2\right)\right].$$

$$K'(s) = \sum_{i=1}^{p} \left[\psi\left(\tfrac{1}{2}n + s - \tfrac{1}{2}(i-1)\right) - \psi\left(\tfrac{1}{2}(n+m) + s - \tfrac{1}{2}(i-1)\right)\right] + \frac{\partial}{\partial s} \log\left[|I_{p_1} - P^2|^{n/2}\, {}_2F_1\left(\frac{n}{2}, \frac{n}{2}; \frac{n}{2} + s; P^2\right)\right],$$

where $\Gamma_p(\cdot)$ is the multivariate gamma function , $\psi(\cdot)$ is the digamma function , and ${}_2F_1(\cdot, \cdot, X)$ is the Gauss hypergeometric function of matrix argument . The saddlepoint equation needs to be evaluated numerically. Also, the computation of $K''(s)$ is performed using a numerical derivative of $K'(s)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", wilks_lambda_corr(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

`dist_wilks_lambda_corr.`**`moments`**`(`*k*`)`

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the noncentral distribution of Wilks' Lambda (CORR):

$$\mu'_X(r) = \frac{\Gamma_{p_1}(n/2)\Gamma_{p_1}((n-p_2)/2+s)}{\Gamma_{p_1}(n/2+s)\Gamma_{p_1}((n-p_2)/2)} \times |I_{p_1} - P^2|^{n/2}\, {}_2F_1\left(\frac{n}{2}, \frac{n}{2}; \frac{n}{2} + s; P^2\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wilks_lambda_corr(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

`dist_wilks_lambda_corr.`**`cumulants`**`(`*k*`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the noncentral distribution of Wilks' Lambda (CORR) The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wilks_lambda_corr(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Approximations

`ctx.`**`wilks_lambda_ind_gp`**`(`*x*, *p1*, *p2*, *n*, *Rho2*, *method='default'*`)`

where `ctx` is `dec`, `mpm`, `fpm`, `gmp` or `apm`.

Calculates the pdf, cdf and sf from the characteristic function using the procedure of Gil-Pelaez (see *gil_pelaez_pdf()* and *gil_pelaez_cdf()*).

This uses $U = \log 2W$.

`ctx.`**`wilks_lambda_ind_spa`**`(`*x*, *n*, *results='c'*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

This uses $2 \log W$ of non-central Wilks W

ctx.**wilks_lambda_ind_spa_inv**(*x*, *n*, *results='qtf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

This uses $2 \log W$ of non-central Wilks W

4.12 Miscellaneous continuous distributions

4.12.1 Boost: Kolmogorov-Smirnov distribution (limiting form)

Returns the pdf, cdf, qtf or boost class of a random variable X, following the limiting form of the Kolmogorov distribution with parameters $n > 0$ and the support interval $[0, \infty)$.

!!! The following references need to be updated: !!!

See also Wikipedia [1146], MathWorld [795], BoostMath [51], Ehrhardt [297] (3.9.2).

`math53.`**`kolmogorov_smirnov_pdf`**(n, x)

Returns the pdf of the limiting form of the Kolmogorov distribution, for $x \in [0, \infty)$.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("BetaPdf(x, a, b): ", BetaPdf(x, a, b))
>>> print ("dist_beta(a, b).pdf(x): ", dist_beta(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`kolmogorov_smirnov_cdf`**(n, x)

Returns $\mathrm{KF}(x) = 1 - 2\sum_{k=1}^{\infty}(-1)^k e^{-2k^2x^2}$, the CDF of the limiting form of the Kolmogorov distribution, for $x \in [0, \infty)$.

See also Ehrhardt [297] (3.9.14).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("BetaCdf(x, a, b): ", BetaCdf(x, a, b))
>>> print ("dist_beta(a, b).cdf(x): ", dist_beta(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`kolmogorov_smirnov_qtf`**(n, q)

Returns $\mathrm{KF}^{-1}(x)$, the functional inverse of the CDF of the Kolmogorov distribution (limiting form), ie $\mathrm{KF}(\mathrm{KF}^{-1}(x)) = x$.

See also Ehrhardt [297] (3.9.14).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("BetaQtf(q, a, b): ", BetaQtf(q, a, b))
>>> print ("dist_beta(a, b).qtf(q): ", dist_beta(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

`CtxBoost.`**`dist_kolmogorov_smirnov`**(a, b)

Returns an `dist_kolmogorov_smirnov` object, which gives access to the functions descibed below:

```
>>> from xlcalcnet import SReal, FReal, ereal, QReal, CReal, OReal
>>> a = 0; b = 1;
>>> Ctx = SReal
>>> dist_beta = Ctx.dist_beta(a, b)
>>> print ("Dist.qtf(q=0.5): ", Dist.qtf(q=0.5))
6.35635234625645256156156145613 56E+00
```

dist_kolmogorov_smirnov.**pdf**(x)

Returns pdf(x), the value of the probability density function of the Kolmogorov distribution (limiting form). See *Ctx.kolmogorov_smirnov_pdf* for formulas and examples.

dist_kolmogorov_smirnov.**cdf**(x)

Returns cdf(x), the value of the cumulative distribution function of the Kolmogorov distribution (limiting form). See *Ctx.kolmogorov_smirnov_cdf* for formulas and examples.

dist_kolmogorov_smirnov.**qtf**(q)

Returns qtf(q), the value of the quantile function of the Kolmogorov distribution (limiting form). See *Ctx.kolmogorov_smirnov_qtf* for formulas and examples.

dist_kolmogorov_smirnov.**sf**(x)

Returns sf(x), the value of the survival function (*Sf*) of the Kolmogorov distribution (limiting form).

```
>>> # continued from above
>>> print ("Dist.sf(x=0.5): ", Dist.qtf(x=0.5))
6.35635234625645256156156145613 56E+00
```

dist_kolmogorov_smirnov.**isf**(q)

Returns isf(q), the value of the inverse survival function (*Isf*) of the Kolmogorov distribution (limiting form).

```
>>> # continued from above
>>> print ("Dist.isf(x=0.5): ", Dist.isf(x=0.5))
6.35635234625645256156156145613 56E+00
```

dist_kolmogorov_smirnov.**hf**(x)

Returns hazard(x), the value of the hazard function (*Hf*) of the Kolmogorov distribution (limiting form).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_kolmogorov_smirnov.**chf**(x)

Returns chf(x), the value of the cumulative hazard function (*Chf*) of the Kolmogorov distribution (limiting form).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_kolmogorov_smirnov.**mode**()

Returns the *mode* of the Kolmogorov distribution (limiting form). Since there is not one unique mode, Nan is returned.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_kolmogorov_smirnov.**median**()

Returns the *median* of the Kolmogorov distribution (limiting form). Calculated as $\frac{1}{2}(a+b)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_kolmogorov_smirnov.**mean**()

Returns the *mean* (expected value) of the Kolmogorov distribution (limiting form). Calculated as $\frac{1}{2}(a+b)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_kolmogorov_smirnov.**variance**()

Returns the *variance* of the Kolmogorov distribution (limiting form). Calculated as $\frac{1}{8}(b-a)^2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_kolmogorov_smirnov.**stdev**()

Returns the *standard deviation* of the Kolmogorov distribution (limiting form). Calculated as $\sqrt{\frac{1}{8}(b-a)^2}$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_kolmogorov_smirnov.**skewness**()

Returns the *skewness* of the Kolmogorov distribution (limiting form). Calculated as 0.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
```

(continues on next page)

(continued from previous page)

```
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_kolmogorov_smirnov.**kurtosis**()

Returns the 'proper' *kurtosis* (normalized fourth moment) of the Kolmogorov distribution (limiting form). Calculated as $3/2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_kolmogorov_smirnov.**kurtosis_excess**()

Returns the *kurtosis excess* of the Kolmogorov distribution (limiting form). Calculated as $-3/2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_kolmogorov_smirnov.**support_lower_endpoint**()

Returns the *support* of the Kolmogorov distribution (limiting form) as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_kolmogorov_smirnov.**support_upper_endpoint**()

Returns the *support* of the Kolmogorov distribution (limiting form) as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_kolmogorov_smirnov.**range_lower_endpoint**()

Returns the valid *range* of the Kolmogorov distribution (limiting form) as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_kolmogorov_smirnov.**range_upper_endpoint**()

Returns the valid *range* of the Kolmogorov distribution (limiting form) as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

4.12.2 Boost: Landau Distribution

The Landau distribution is a stable distribution (see Wikipedia [1228]) with the shape parameters $\alpha = 1, \beta = 1$. For simplicity of numerical computation, this paper evaluates as follows assuming location parameter $\mu = 0$, scale parameter $c = 1/\pi/2$, i.e. $p(x) = p(x; \alpha = 1, \beta = 1, \mu = 0, c = \pi/2)$. For other choices of the location parameter μ and scale parameter c there is the relationship

$$p(x_1; \mu_1, c_1) = p\left(x := \frac{x_1 - \mu_1}{c_1} - \frac{2}{\pi}\log(c); \mu := 0, c := 1\right) \cdot \frac{1}{c_1}.$$

The support interval is $(-\infty, +\infty)$.

!!! The following references need to be updated: !!!

See also: https://www.boost.org/doc/libs/1_89_0/libs/math/doc/html/math_toolkit/dist_ref/dists/landau_dist.html

See also: https://en.wikipedia.org/wiki/Landau_distribution

`math53.landau_pdf(mu, c, x)`

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Landau distribution, with $\mu \in \mathbb{R}$, $c > 0$:

$$\mathrm{pdf}_X(x) = \frac{1}{\pi c}\int_0^\infty e^{-t}\cos\left(t\left(\frac{x-\mu}{c}\right) + \frac{2t}{\pi}\log\left(\frac{t}{c}\right)\right)\mathrm{d}t.$$

For $\mu = 0, c = \pi/2$ this simplifies to:

$$\mathrm{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \frac{\exp(-xt)\sin(\pi t)}{t^t}\mathrm{d}t.$$

`math53.landau_cdf(mu, c, x)`

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Landau distribution.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("StudentTCdf(x, a, b): ", StudentTCdf(x, a, b))
>>> print ("dist_student_t(a, b).cdf(x): ", dist_student_t(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.landau_qtf(mu, c, q)`

where `Ctx` is `Math53` or `CtxBoost`.

Returns $\mathrm{qtf}(q)$, the value of the quantile function (*Qtf*) of the Landau distribution.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("StudentTQtf(q, a, b): ", StudentTQtf(q, a, b))
>>> print ("dist_student_t(a, b).qtf(q): ", dist_student_t(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_landau**(*n1*, *n2*, *lambda*, ***kwargs*)

In probability theory, the Landau distribution is a probability distribution named after Lev Landau. Because of the distribution's "fat" tail, the moments of the distribution, like mean or variance, are undefined. The distribution is a particular case of stable distribution.

These functions return PDF, CDF, and ICDF of the Landau distribution with location a, scale $b > 0$, and the support interval $(-\infty, +\infty)$:

See also: Wikipedia [1224].

dist_landau.**pdf**(x)

Returns $\mathrm{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Landau distribution:

$$\mathrm{pdf}_X(x) = \frac{1}{\pi}\int_0^\infty \Re\left(e^{-itx}C_X(t)\right)\mathrm{d}t. \tag{4.40}$$

$$= \frac{1}{\pi c}\int_0^\infty e^{-t}\cos\left(t\left(\frac{x-\mu}{c}\right)+\frac{2t}{\pi}\log\left(\frac{t}{c}\right)\right)\mathrm{d}t., \tag{4.41}$$

where $\Re(z)$ denotes the real part of z.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pdf: ", dist_landau(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_landau.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Landau distribution:

$$\mathrm{cdf}_X(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t,$$

where $\Im(z)$ denotes the imaginary part of z.

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", dist_landau(mu, sigma).pdf(x))
6.356352346256452561561561561456135 6E-20
```

dist_landau.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function function (sf) of a random variable X, following an Landau distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x) = \int_x^\infty \mathrm{pdf}_X(x)\mathrm{d}t.$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", dist_landau(mu, sigma).pdf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_landau.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function function (qtf) of a random variable X, following an Landau distribution:

$$\text{qtf}_X(q) = \text{no closed form.}$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", dist_landau(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_landau.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function function (isf) of a random variable X, following an Landau distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q) = \text{cdf}_X^{-1}(1-q).$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", dist_landau(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_landau.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an Landau distribution:

$$C_X(t) = \exp\left(it\mu - \frac{2ict}{\pi}\log|t| - c|t|\right).$$

```
>>> from mpdistrib import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", dist_landau(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_landau.**m_x**(t)

Returns `NaN`, since the moment generating function does not exist.

dist_landau.**k_x**(t, *k=0*)

Returns `NaN`, since the cumulant generating function does not exist.

dist_landau.**moments**(k)

Returns `NaN`, since moments do not exist.

dist_landau.**cumulants**(k)

Returns `NaN`, since cumulants do not exist.

4.12.3 Boost: Holtsmark distribution

The Holtsmark distribution is a stable distribution (see Wikipedia [1228]) with the shape parameters $\alpha = 3/2, \beta = 0$. The Holtsmark distribution is linear with respect to the location parameter μ and scale parameter c.

$$p(x_1; \mu_1, c_1) = p\left(x := \frac{x_1 - \mu_1}{c_1}; \mu := 0, c := 1\right) \cdot \frac{1}{c_1}.$$

The support interval is $(-\infty, +\infty)$.

!!! The following references need to be updated: !!!

See also: https://www.boost.org/doc/libs/1_89_0/libs/math/doc/html/math_toolkit/dist_ref/dists/holtsmark_dist.html

See also: https://en.wikipedia.org/wiki/Holtsmark_distribution

`math53.`**`holtsmark_pdf`**(*mu*, *c*, *x*)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Holtsmark distribution

$$\text{pdf}_X(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp\left(it\mu - |ct|^{3/2}\right) e^{ixt} \mathrm{d}t,$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("FisherFPdf(x, a, b): ", FisherFPdf(x, a, b))
>>> print ("dist_fisher_f(a, b).pdf(x): ", dist_fisher_f(a, b).pdf(x))
6.356352346256452561561561456l356E+00
```

`math53.`**`holtsmark_cdf`**(*mu*, *c*, *x*)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Holtsmark distribution.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("FisherFCdf(x, a, b): ", FisherFCdf(x, a, b))
>>> print ("dist_fisher_f(a, b).cdf(x): ", dist_fisher_f(a, b).cdf(x))
6.356352346256452561561561456l356E+00
```

`math53.`**`holtsmark_qtf`**(*mu*, *c*, *q*)

Returns $\text{qtf}(q)$, the value of the quantile function (*Qtf*) of the Holtsmark distribution.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("FisherFQtf(q, a, b): ", FisherFQtf(q, a, b))
>>> print ("dist_fisher_f(a, b).qtf(q): ", dist_fisher_f(a, b).qtf(q))
6.356352346256452561561561456l356E+00
```

`CtxBoost.`**`dist_holtsmark`**(*m*, *n*)

Returns an `dist_holtsmark` object, which gives access to the functions descibed below:

```
>>> from xlcalcnet import SReal, FReal, ereal, QReal, CReal, OReal
>>> a = 0; b = 1;
>>> Ctx = SReal
>>> dist_fisher_f = Ctx.dist_fisher_f(a, b)
>>> print ("Dist.qtf(q=0.5): ", Dist.qtf(q=0.5))
6.35635234625645256156156145613 56E+00
```

dist_holtsmark.**pdf**(x)

Returns pdf(x), the value of the probability density function of the Holtsmark distribution. See *Ctx.holtsmark_pdf* for formulas and examples.

dist_holtsmark.**cdf**(x)

Returns cdf(x), the value of the cumulative distribution function of the Holtsmark distribution. See *Ctx.holtsmark_cdf* for formulas and examples.

dist_holtsmark.**qtf**(q)

Returns qtf(q), the value of the quantile function of the Holtsmark distribution. See *Ctx.holtsmark_qtf* for formulas and examples.

dist_holtsmark.**sf**(x)

Returns sf(x), the value of the survival function (*Sf*) of the Holtsmark distribution.

$$\mathrm{sf}(x) = \begin{cases} \mathrm{ibeta}(n/2, m/2, n/(n+mx)), & mx > n, \\ \mathrm{ibetac}(m/2, n/2, mx/(n+mx)) & mx \le n. \end{cases}$$

Here ibeta($\cdot$) denotes the real normalised incomplete beta function (*RealIBeta*), and ibetac($\cdot$) denotes the real normalised complementary incomplete beta function (*RealIBetac*).

```
>>> # continued from above
>>> print ("Dist.sf(x=0.5): ", Dist.qtf(x=0.5))
6.35635234625645256156156145613 56E+00
```

dist_holtsmark.**isf**(q)

Returns isf(q), the value of the inverse survival function (*Isf*) of the Holtsmark distribution.

$$\mathrm{isf}(q) = \frac{nx}{m(1-x)}, \quad \text{where } x = \mathrm{ibetac_inv}(m/2, n/2, q).$$

Here $\mathrm{ibetac_inv}(\cdot)$ denotes the inverse of the real normalised complementary incomplete beta function (*RealIBetacInv*).

```
>>> # continued from above
>>> print ("Dist.isf(x=0.5): ", Dist.isf(x=0.5))
6.35635234625645256156156145613 56E+00
```

dist_holtsmark.**hf**(x)

Returns hazard(x), the value of the hazard function (*Hf*) of the Holtsmark distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_holtsmark.**chf**(x)

Returns chf(x), the value of the cumulative hazard function (*Chf*) of the Holtsmark distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_holtsmark.**mode**()

Returns the *mode* of the Holtsmark distribution. Since there is not one unique mode, `Nan` is returned.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_holtsmark.**median**()

Returns the *median* of the Holtsmark distribution. Calculated as $\frac{1}{2}(a+b)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_holtsmark.**mean**()

Returns the *mean* (expected value) of the Holtsmark distribution. Calculated as $\frac{1}{2}(a+b)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_holtsmark.**variance**()

Returns the *variance* of the Holtsmark distribution. Calculated as $\frac{1}{8}(b-a)^2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_holtsmark.**stdev**()

Returns the *standard deviation* of the Holtsmark distribution. Calculated as $\sqrt{\frac{1}{8}(b-a)^2}$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
```

(continues on next page)

(continued from previous page)

```
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_holtsmark.**skewness**()

Returns the *skewness* of the Holtsmark distribution. Calculated as 0.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_holtsmark.**kurtosis**()

Returns the 'proper' *kurtosis* (normalized fourth moment) of the Holtsmark distribution. Calculated as $3/2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_holtsmark.**kurtosis_excess**()

Returns the *kurtosis excess* of the Holtsmark distribution. Calculated as $-3/2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_holtsmark.**support_lower_endpoint**()

Returns the *support* of the Holtsmark distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_holtsmark.**support_upper_endpoint**()

Returns the *support* of the Holtsmark distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_holtsmark.**range_lower_endpoint**()

Returns the valid *range* of the Holtsmark distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_holtsmark.**range_upper_endpoint**()

Returns the valid *range* of the Holtsmark distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

4.12.4 Boost: Map-Airy distribution

The Map-Airy distribution (or Airy distribution of the 'Map'-type) is a stable distribution (see Wikipedia [1228]) with the shape parameters $\alpha = 3/2, \beta = 1$, which describes the probability distribution of the area under a Brownian excursion over a unit interval. For simplicity of numerical computation, this paper evaluates as follows assuming location parameter $\mu = 0$, scale parameter $c = 1/\sqrt[3]{18}$, i.e. $p(x) = p(x; \alpha = 3/2, \beta = 1, \mu = 0, c = 1/\sqrt[3]{18})$. The Map-Airy distribution is linear with respect to the location parameter μ and scale parameter c.

$$p(x_1; \mu_1, c_1) = p\left(x := \frac{x_1 - \mu_1}{c_1}; \mu := 0, c := 1\right) \cdot \frac{1}{c_1}.$$

The support interval is $(-\infty, +\infty)$:

!!! The following references need to be updated: !!!

See also: https://www.boost.org/doc/libs/1_89_0/libs/math/doc/html/math_toolkit/dist_ref/dists/mapairy_dist.html

See also: https://mathworld.wolfram.com/Map-AiryDistribution.html

math53.**mapairy_pdf**(*mu*, *c*, *x*)

Returns pdf(x), the value of the probability density function (*Pdf*) of the MapAiry distribution:

Boost:

$$\text{pdf}(x) = 2e^{-2x^3/3}\left(x\text{Ai}(x^2) - \text{Ai}'(x^2)\right).$$

Wolfram:

$$\text{pdf}(x) = 2e^{2x^3/3}\left(-x\text{Ai}(x^2) - \text{Ai}'(x^2)\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("StudentTPdf(x, a, b): ", StudentTPdf(x, a, b))
>>> print ("dist_student_t(a, b).pdf(x): ", dist_student_t(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**mapairy_cdf**(*mu*, *c*, *x*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the MapAiry distribution:

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("StudentTCdf(x, a, b): ", StudentTCdf(x, a, b))
>>> print ("dist_student_t(a, b).cdf(x): ", dist_student_t(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**mapairy_qtf**(*mu*, *c*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the MapAiry distribution:

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("StudentTQtf(q, a, b): ", StudentTQtf(q, a, b))
>>> print ("dist_student_t(a, b).qtf(q): ", dist_student_t(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

CtxBoost.**dist_mapairy**(a, b)

Returns an dist_mapairy object, which gives access to the functions descibed below:

```
>>> from xlcalcnet import SReal, FReal, ereal, QReal, CReal, OReal
>>> a = 0; b = 1;
>>> Ctx = SReal
>>> dist_student_t = Ctx.dist_student_t(a, b)
>>> print ("Dist.qtf(q=0.5): ", Dist.qtf(q=0.5))
6.35635234625645256156156145613 56E+00
```

dist_mapairy.**pdf**(x)

Returns pdf(x), the value of the probability density function of the Map-Airy distribution. See *Ctx.mapairy_pdf* for formulas and examples.

dist_mapairy.**cdf**(x)

Returns cdf(x), the value of the cumulative distribution function of the Map-Airy distribution. See *Ctx.mapairy_cdf* for formulas and examples.

dist_mapairy.**qtf**(q)

Returns qtf(q), the value of the quantile function of the Map-Airy distribution. See *Ctx.mapairy_qtf* for formulas and examples.

dist_mapairy.**sf**(x)

Returns sf(x), the value of the survival function (*Sf*) of the Map-Airy distribution.

$$\mathrm{sf}(x) = \mathrm{cdf}_X(-x)$$

```
>>> # continued from above
>>> print ("Dist.sf(x=0.5): ", Dist.qtf(x=0.5))
6.35635234625645256156156145613 56E+00
```

dist_mapairy.**isf**(q)

Returns isf(q), the value of the inverse survival function (*Isf*) of the Map-Airy distribution.

$$\mathrm{isf}(q) = -\mathrm{qtf}(q).$$

```
>>> # continued from above
>>> print ("Dist.isf(x=0.5): ", Dist.isf(x=0.5))
6.35635234625645256156156145613 56E+00
```

dist_mapairy.**hf**(x)

Returns hazard(x), the value of the hazard function (*Hf*) of the Map-Airy distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mapairy.**chf**(x)

Returns chf(x), the value of the cumulative hazard function (*Chf*) of the Map-Airy distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_mapairy.**mode**()

Returns the *mode* of the Map-Airy distribution. Since there is not one unique mode, Nan is returned.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_mapairy.**median**()

Returns the *median* of the Map-Airy distribution. Calculated as $\frac{1}{2}(a+b)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_mapairy.**mean**()

Returns the *mean* (expected value) of the Map-Airy distribution. Calculated as $\frac{1}{2}(a+b)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_mapairy.**variance**()

Returns the *variance* of the Map-Airy distribution. Calculated as $\frac{1}{8}(b-a)^2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_mapairy.**stdev**()

Returns the *standard deviation* of the Map-Airy distribution. Calculated as $\sqrt{\frac{1}{8}(b-a)^2}$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_mapairy.**skewness**()

Returns the *skewness* of the Map-Airy distribution. Calculated as 0.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mapairy.**kurtosis**()

Returns the 'proper' *kurtosis* (normalized fourth moment) of the Map-Airy distribution. Calculated as $3/2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mapairy.**kurtosis_excess**()

Returns the *kurtosis excess* of the Map-Airy distribution. Calculated as $-3/2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mapairy.**support_lower_endpoint**()

Returns the *support* of the Map-Airy distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mapairy.**support_upper_endpoint**()

Returns the *support* of the Map-Airy distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mapairy.**range_lower_endpoint**()

Returns the valid *range* of the Map-Airy distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mapairy.**range_upper_endpoint**()

Returns the valid *range* of the Map-Airy distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

4.12.5 Boost: Saspoint5 distribution

The SaS point5 distribution is a stable distribution (see Wikipedia [1228]) with the shape parameters $\alpha = 1/2, \beta = 0$. The Saspoint5 distribution is linear with respect to the location parameter μ and scale parameter c.

$$p(x_1; \mu_1, c_1) = p\left(x := \frac{x_1 - \mu_1}{c_1}; \mu := 0, c := 1\right) \cdot \frac{1}{c_1}.$$

The support interval is $(-\infty, +\infty)$.

!!! The following references need to be updated: !!!

See also: https://www.boost.org/doc/libs/1_89_0/libs/math/doc/html/math_toolkit/dist_ref/dists/saspoint5_dist.html

See also: https://en.wikipedia.org/wiki/Stable_distribution#Other_analytic_cases

math53.**saspoint5_pdf**(*mu*, *c*, *x*)

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following an Saspoint5 distribution. Let $S(x)$ and $C(x)$ denote the Fresnel integrals. Then:

$$\text{pdf}_X(x) = \frac{1}{\sqrt{2\pi|x|^3}}\left(\sin\left(\tfrac{1}{4|x|}\right)\left[\frac{1}{2} - S\left(\tfrac{1}{\sqrt{2\pi|x|}}\right)\right] + \cos\left(\tfrac{1}{4|x|}\right)\left[\frac{1}{2} - C\left(\tfrac{1}{\sqrt{2\pi|x|}}\right)\right]\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("StudentTPdf(x, a, b): ", StudentTPdf(x, a, b))
>>> print ("dist_student_t(a, b).pdf(x): ", dist_student_t(a, b).pdf(x))
6.35635234625645256156156145613356E+00
```

math53.**saspoint5_cdf**(*mu*, *c*, *x*)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an Saspoint5 distribution.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("StudentTCdf(x, a, b): ", StudentTCdf(x, a, b))
>>> print ("dist_student_t(a, b).cdf(x): ", dist_student_t(a, b).cdf(x))
6.35635234625645256156156145613356E+00
```

math53.**saspoint5_qtf**(*mu*, *c*, *q*)

Returns $\text{qtf}(q)$, the value of the quantile function (*Qtf*) of the Saspoint5 distribution:

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("StudentTQtf(q, a, b): ", StudentTQtf(q, a, b))
>>> print ("dist_student_t(a, b).qtf(q): ", dist_student_t(a, b).qtf(q))
6.35635234625645256156156145613356E+00
```

CtxBoost.**dist_saspoint5**(a, b)

Returns an dist_saspoint5 object, which gives access to the functions descibed below:

```
>>> from xlcalcnet import SReal, FReal, ereal, QReal, CReal, OReal
>>> a = 0; b = 1;
>>> Ctx = SReal
>>> dist_student_t = Ctx.dist_student_t(a, b)
>>> print ("Dist.qtf(q=0.5): ", Dist.qtf(q=0.5))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**pdf**(x)

Returns pdf(x), the value of the probability density function of the Saspoint5 distribution. See *Ctx.saspoint5_pdf* for formulas and examples.

dist_saspoint5.**cdf**(x)

Returns cdf(x), the value of the cumulative distribution function of the Saspoint5 distribution. See *Ctx.saspoint5_cdf* for formulas and examples.

dist_saspoint5.**qtf**(q)

Returns qtf(q), the value of the quantile function of the Saspoint5 distribution. See *Ctx.saspoint5_qtf* for formulas and examples.

dist_saspoint5.**sf**(x)

Returns sf(x), the value of the survival function (*Sf*) of the Saspoint5 distribution.

$$\mathrm{sf}(x) = \mathrm{cdf}_X(-x)$$

```
>>> # continued from above
>>> print ("Dist.sf(x=0.5): ", Dist.qtf(x=0.5))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**isf**(q)

Returns isf(q), the value of the inverse survival function (*Isf*) of the Saspoint5 distribution.

$$\mathrm{isf}(q) = -\mathrm{qtf}(q).$$

```
>>> # continued from above
>>> print ("Dist.isf(x=0.5): ", Dist.isf(x=0.5))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**hf**(x)

Returns hazard(x), the value of the hazard function (*Hf*) of the Saspoint5 distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**chf**(x)

Returns chf(x), the value of the cumulative hazard function (*Chf*) of the Saspoint5 distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**mode**()

Returns the *mode* of the Saspoint5 distribution. Since there is not one unique mode, Nan is returned.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**median**()

Returns the *median* of the Saspoint5 distribution. Calculated as $\frac{1}{2}(a + b)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**mean**()

Returns the *mean* (expected value) of the Saspoint5 distribution. Calculated as $\frac{1}{2}(a + b)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**variance**()

Returns the *variance* of the Saspoint5 distribution. Calculated as $\frac{1}{8}(b - a)^2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**stdev**()

Returns the *standard deviation* of the Saspoint5 distribution. Calculated as $\sqrt{\frac{1}{8}(b - a)^2}$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**skewness**()

Returns the *skewness* of the Saspoint5 distribution. Calculated as 0.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**kurtosis**()

Returns the 'proper' *kurtosis* (normalized fourth moment) of the Saspoint5 distribution. Calculated as $3/2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**kurtosis_excess**()

Returns the *kurtosis excess* of the Saspoint5 distribution. Calculated as $-3/2$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**support_lower_endpoint**()

Returns the *support* of the Saspoint5 distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**support_upper_endpoint**()

Returns the *support* of the Saspoint5 distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_saspoint5.**range_lower_endpoint**()

Returns the valid *range* of the Saspoint5 distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_saspoint5.**range_upper_endpoint**()

Returns the valid *range* of the Saspoint5 distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

4.13 Elementary discrete (lattice) distributions

4.13.1 Boost: Bernoulli distribution

The following functions return the pmf, cdf, qtf or boost class of the Bernoulli distribution

The Bernoulli distribution is a discrete distribution of the outcome of a single trial with only two results, 0 (failure) or 1 (success), with a probability of success p.

The Bernoulli distribution is the simplest building block on which other discrete distributions of sequences of independent Bernoulli trials can be based.

The Bernoulli is the binomial distribution (k = 1, p) with only one trial.

The domain of the random variable is 0 and 1, and the useful supported range is only 0 or 1.

The support interval for k is $0, 1$, and $0 \leq q \leq 1$.

The Bernoulli distribution is implemented here as a strict discrete distribution. If a generalised version, allowing k to be any real, is required then the binomial distribution with a single trial should be used, for example:

binomial_distribution(1, 0.25)

!!! The following references need to be updated: !!!

See also Wikipedia [1176], MathWorld [821], BoostMath [76] .

math53.**bernoulli_pmf**(*p*, *k*)

Returns pmf(x), the value of the probability mass function (*Pmf*) of the Bernoulli distribution. It is calculated using the relation pmf(k) $= 1 - p$ for $k = 0$, else p.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("BernoulliPdf(x, a, b): ", BernoulliPdf(x, a, b))
>>> print ("dist_bernoulli(a, b).pdf(x): ", dist_bernoulli(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**bernoulli_cdf**(*p*, *k*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Bernoulli distribution. It is calculated using the relation cdf(k) $= 1 - p$ for $k = 0$, else 1.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("BernoulliCdf(x, a, b): ", BernoulliCdf(x, a, b))
>>> print ("dist_bernoulli(a, b).cdf(x): ", dist_bernoulli(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**bernoulli_qtf**(*p*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Bernoulli distribution. It is calculated using the relation qtf(k) $= 0$ for $k \leq (1 - p)$, else 1.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("BernoulliQtf(q, a, b): ", BernoulliQtf(q, a, b))
>>> print ("dist_bernoulli(a, b).qtf(q): ", dist_bernoulli(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

CtxBoost.**dist_bernoulli**(p)

Returns an dist_bernoulli object, which gives access to the functions descibed below:

```
>>> from xlcalcnet import SReal, FReal, ereal, QReal, CReal, OReal
>>> a = 0; b = 1;
>>> Ctx = SReal
>>> dist_bernoulli = Ctx.dist_bernoulli(a, b)
>>> print ("Dist.qtf(q=0.5): ", Dist.qtf(q=0.5))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**pmf**(x)

Returns pdf(x), the value of the probability density function of the Bernoulli distribution. See *Ctx.BernoulliPmf* for formulas and examples.

dist_bernoulli.**cdf**(x)

Returns cdf(x), the value of the cumulative distribution function of the Bernoulli distribution. See *Ctx.BernoulliCdf* for formulas and examples.

dist_bernoulli.**qtf**(q)

Returns qtf(q), the value of the quantile function of the Bernoulli distribution. See *Ctx.BernoulliQtf* for formulas and examples.

dist_bernoulli.**sf**(x)

Returns sf(x), the value of the survival function (*Sf*) of the Bernoulli distribution. It is calculated using the relation $\text{sf}(k) = p$ for $k = 0$, else 0.

```
>>> # continued from above
>>> print ("Dist.sf(x=0.5): ", Dist.qtf(x=0.5))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**isf**(q)

Returns isf(q), the value of the inverse survival function (*Isf*) of the Bernoulli distribution. It is calculated using the relation $\text{qtf}(k) = 1$ for $k \leq (1 - p)$, else 0.

```
>>> # continued from above
>>> print ("Dist.isf(x=0.5): ", Dist.isf(x=0.5))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**hf**(x)

Returns hazard(x), the value of the hazard function (*Hf*) of the Bernoulli distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**chf**(x)

Returns chf(x), the value of the cumulative hazard function (*Chf*) of the Bernoulli distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_bernoulli.**mode**()

Returns the *mode* of the Bernoulli distribution, which is 0 if $(p < 0.5)$ else 1.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_bernoulli.**median**()

Returns the *median* of the Bernoulli distribution. Calculated as *tbd*.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_bernoulli.**mean**()

Returns the *mean* (expected value) of the Bernoulli distribution, which is p.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_bernoulli.**variance**()

Returns the *variance* of the Bernoulli distribution. Calculated as $p(1-p)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_bernoulli.**stdev**()

Returns the *standard deviation* of the Bernoulli distribution. Calculated as $\sqrt{p(1-p)}$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
```

(continues on next page)

(continued from previous page)

```
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**skewness**()

Returns the *skewness* of the Bernoulli distribution. Calculated as $(1 - 2p)/\sqrt{p(1 - p)}$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**kurtosis**()

Returns the 'proper' *kurtosis* (normalized fourth moment) of the Bernoulli distribution. Calculated as $3+(1-6p(1-p))/(p(1-p))$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**kurtosis_excess**()

Returns the *kurtosis excess* of the Bernoulli distribution. Calculated as $(1 - 6p(1 - p))/(p(1 - p))$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**support_lower_endpoint**()

Returns the *support* of the Bernoulli distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**support_upper_endpoint**()

Returns the *support* of the Bernoulli distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**range_lower_endpoint**()

Returns the valid *range* of the Bernoulli distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_bernoulli.**range_upper_endpoint**()

Returns the valid *range* of the Bernoulli distribution as a tuple (left, right).

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

4.13.2 Boost: Geometric distribution

The following functions return the pmf, cdf, qtf or boost class of the geometric distribution with parameter $p, 0 \leq p \leq 1$ (p is the probability that any one trial will be successful, it is also known as "success fraction"). The function returns the probability of obtaining exactly k failures from k trials with success fraction p. For this implementation, the set of trials includes zero (unlike another definition where the set of trials starts at one, sometimes named shifted), so the support interval for k is $(0, \infty)$, and $0 \leq q \leq 1$.

See also Wikipedia [1176], MathWorld [821], BoostMath [76] .

|newpage|

math53.**geometric_pmf**(*p*, *k*)

Returns pmf(x), the value of the probability mass function (*Pmf*) of the geometric distribution:

$$\text{pmf}_X(k) = p(1-p)^k = p \cdot \exp\left(k\text{log1p}(-p)\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("GeometricPdf(x, a, b): ", GeometricPdf(x, a, b))
>>> print ("dist_geometric(a, b).pdf(x): ", dist_geometric(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

|newpage|

math53.**geometric_cdf**(*p*, *k*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the geometric distribution. The cumulative distribution function returns the probability of obtaining k failures or fewer from k trials with success fraction p and success on the last trial.

$$\text{cdf}(x) = 1 - (1-p)^{k+1} = -\text{expm1}\left(\text{log1p}(-p)(k+1)\right).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("GeometricCdf(x, a, b): ", GeometricCdf(x, a, b))
>>> print ("dist_geometric(a, b).cdf(x): ", dist_geometric(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

|newpage|

`math53.`**`geometric_qtf`**(p, q)

Returns qtf(q), the value of the quantile function (*Qtf*) of the geometric distribution. The quantile function returns the greatest number of failures k expected to be observed from k trials with success fraction p, at probability q. Note that the value returned is a real-number, and not an integer. Depending on the use case you may want to take either the floor or ceiling of the real result.

$$\text{qtf}(q) = \frac{\log1\text{p}(-q)}{\log1\text{p}(-p)} - 1.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("GeometricQtf(q, a, b): ", GeometricQtf(q, a, b))
>>> print ("dist_geometric(a, b).qtf(q): ", dist_geometric(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

|newpage|

`class` `ctx.`**`dist_geometric`**(p)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The geometric distribution is a (discrete or continuous) probability distribution with parameter $p, 0 \leq p \leq 1$ (p is the probability that any one trial will be successful, it is also known as "success fraction"). The function returns the probability of obtaining exactly k failures from k trials with success fraction p. For this implementation, the set of trials includes zero (unlike another definition where the set of trials starts at one, sometimes named shifted), so the support interval for k is $(0, \infty)$. See also Wikipedia [1176], MathWorld [821], BoostMath [76] , and Witkovský [1485], R (Statistical System) [531].

|newpage|

`dist_geometric.`**`pmf`**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following an geometric distribution:

$$\text{pmf}_X(k) = p(1-p)^k = p \cdot \exp\left(k\log1\text{p}(-p)\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", geometric(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

|newpage|

dist_geometric.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an geometric distribution:

$$\text{cdf}_X(x) = 1 - (1-p)^{k+1} = -\text{expm1}\left(\text{log1p}(-p)(k+1)\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", geometric(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

|newpage|

dist_geometric.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an geometric distribution:

$$\text{sf}_X(x) = (1-p)^{k+1} = \exp\left(\text{log1p}(-p)(k+1)\right)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", geometric(mu, sigma).pmf(x))
sf: 6.3563523462564525615615615614561356E-20
```

|newpage|

dist_geometric.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an geometric distribution:

$$\text{qtf}_X(q) = \frac{\text{log1p}(-q)}{\text{log1p}(-p)} - 1$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", geometric(mu, sigma).qtf(q))
qtf: 6.3563523462564525615615615614561356E+00
```

|newpage|

dist_geometric.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an geometric distribution:

$$\text{isf}_X(q) = \frac{\log(q)}{\text{log1p}(-p)} - 1$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", geometric(mu, sigma).isf(q))
6.3563523462564525615615615614561356E+00
```

|newpage|

dist_geometric.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following an geometric distribution:

$$C_X(t) = ??.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", geometric(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

|newpage|

dist_geometric.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following an geometric distribution:

$$M_X(t) = ??.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", geometric(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

|newpage|

dist_geometric.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an geometric distribution:

$$K_X(t) = ??.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", geometric(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

|newpage|

dist_geometric.**moments**(*k*)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an geometric distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(n) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", geometric(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

|newpage|

dist_geometric.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an geometric distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", geometric(mu, sigma).cumulants(k))
6.35635234625645256156156145614356E+00
```

The geometric distribution is used when there are exactly two mutually exclusive outcomes of a Bernoulli trial: these outcomes are labelled "success" and "failure". For Bernoulli trials each with success fraction p, the geometric distribution gives the probability of observing k trials (failures, events, occurrences, or arrivals) before the first success.

The pmf of a variable following a geometric distribution with parameter p is given by

$$f(k;p) = p(1-p)^k$$

The CDF of a variable following a geometric distribution with parameter p is given by

$$F(k;p) = 1 - (1-p)^{k+1}$$

The ICDF of a variable following a geometric distribution with parameter p is given by

$$F^{-1}(x;p) = \frac{\log1\mathrm{p}(-x)}{\log1\mathrm{p}(-p)} - 1$$

4.13.3 Boost: Poisson distribution

The following functions return the pmf, cdf, qtf or boost class of the Poisson distribution with mean mu and the support interval $(0, +\infty)$, and $0 \leq q \leq 1$.

See also Wikipedia [1189], MathWorld [835], BoostMath [84], Ehrhardt [297] (3.9.26).

`math53.poisson_pmf`(*mu*, *x*)

Returns pmf(x), the value of the probability mass function (*Pmf*) of the Poisson distribution:

$$\text{pmf}(x) = \frac{\mu^k}{k!} e^{-\mu}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("PoissonPdf(x, a, b): ", PoissonPdf(x, a, b))
>>> print ("dist_poisson(a, b).pdf(x): ", dist_poisson(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.poisson_cdf`(*mu*, *x*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the Poisson distribution:

$$\text{cdf}(x) = e^{-\mu} \sum_{i=0}^{k} \frac{\mu^i}{i!} = Q(1 + k, \mu).$$

Here $Q(\cdot)$ denotes the upper regularized incomplete gamma function (*RealGammaQ*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("PoissonCdf(x, a, b): ", PoissonCdf(x, a, b))
>>> print ("dist_poisson(a, b).cdf(x): ", dist_poisson(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.poisson_qtf`(*mu*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the Poisson distribution:

$$\text{qtf}(q) = \text{gammaq_inva}(\mu, p) - 1.$$

Here $\text{gammaq_inva}(\cdot)$ denotes the inverse (on parameter a) of the real upper normalised incomplete gamma function (*RealGammaQInva*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("PoissonQtf(q, a, b): ", PoissonQtf(q, a, b))
>>> print ("dist_poisson(a, b).qtf(q): ", dist_poisson(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_poisson**(*lambda1*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The Poisson distribution is a discrete (lattice) probability distribution with mean mu and the support interval $(0, +\infty)$. See also Wikipedia [1189], MathWorld [835], BoostMath [84], and Witkovský [1492], R (Statistical System) [534].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.pdtr.html#scipy.special.pdtr

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.pdtrc.html#scipy.special.pdtrc

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.pdtri.html#scipy.special.pdtri

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.pdtrik.html#scipy.special.pdtrik

dist_poisson.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Poisson distribution:

$$\text{pmf}_X(x) = \frac{\lambda_1^k}{k!} e^{-\lambda_1}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", poisson(mu, sigma).pmf(x))
6.356352346256452561561561456135 6E-20
```

dist_poisson.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Poisson distribution:

$$\text{cdf}_X(x) = e^{-\lambda_1} \sum_{i=0}^{k} \frac{\lambda_1^i}{i!} = Q(1+k, \lambda_1).$$

Here $Q(\cdot)$ denotes the upper regularized incomplete gamma functions.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", poisson(mu, sigma).pmf(x))
6.356352346256452561561561456135 6E-20
```

dist_poisson.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a Poisson distribution:

$$\text{sf}_X(x) = 1 - e^{-\lambda_1} \sum_{i=0}^{k} \frac{\lambda_1^i}{i!} = P(1+k, \lambda_1).$$

Here $P(\cdot)$ denotes the lower regularized incomplete gamma functions.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", poisson(mu, sigma).pmf(x))
sf: 6.356352346256452561561561456135 6E-20
```

dist_poisson.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Poisson distribution.

$$\mathrm{qtf}_X(q) = \mathrm{gamma_q_inva}(\lambda_1, p) - 1.$$

Here $\mathrm{gamma_q_inva}(\cdot)$ denotes the inverse (on parameter a) of the real normalised incomplete gamma function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", poisson(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_poisson.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Poisson distribution:

$$\mathrm{isf}_X(q) = \mathrm{gamma_p_inva}(\lambda_1, q) - 1.$$

Here $\mathrm{gamma_p_inva}(\cdot)$ denotes the inverse (on parameter a) of the real normalised complementary incomplete gamma function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", poisson(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_poisson.**g_x**(t)

Returns $G_X(t)$, the probability generating function of a random variable X, following a Poisson distribution:

$$G_X(t) = \exp(\lambda(t-1)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", poisson(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_poisson.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Poisson distribution:

$$C_X(t) = \exp(\lambda(e^{it}-1)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", poisson(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_poisson.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Poisson distribution:

$$M_X(t) = \exp(\lambda(e^t - 1)).$$

$$L_X(t) = \exp(\lambda(e^{-t} - 1)).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", poisson(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_poisson.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Poisson distribution:

$$K_X(t) = \lambda(e^t - 1).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", poisson(mu, sigma).k_x(t, k))
6.356352346256452561561561456135 6E+00
```

dist_poisson.**moments**(*k*)

Returns the first j moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Poisson distribution (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", poisson(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_poisson.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Poisson distribution:

$$\kappa_r = \lambda, \quad r = 1, 2, ...$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", poisson(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

Approximations

ctx.**poisson_ecf**(*x*, *f*, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

ctx.**poisson_ecf_inv**(*x*, *f*, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**poisson_spa**(*x*, *n*, *results='c'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

The solution $\hat{s}(x)$ of the saddlepoint equation $K_X^{(1)}(\hat{s}(x)) = x$, of a random variable X, following a Poisson distribution is given by:

$$\hat{s}(x) = \log(k/\lambda).$$

ctx.**poisson_spa_inv**(*x*, *n*, *results='qtf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.13.4 Boost: Binomial distribution

The following functions return the pmf, cdf, qtf or boost class of the binomial distribution with number of trials $n \geq 0$ and success probability $0 \leq p \leq 1$, and $0 \leq q \leq 1$.

See also Wikipedia [1173], MathWorld [816], BoostMath [71], Ehrhardt [297] (3.9.3).

math53.**binomial_pmf**(*n*, *p*, *k*)

Returns pmf(x), the value of the probability density function (*Pmf*) of the binomial distribution:

$$\text{pmf}(x) = \binom{n}{k} p^k (1-p)^{n-k} = f_{\text{Beta}}(k+1, n-k+1, p)/(n+1).$$

Here $f_{\text{Beta}}(\cdot)$ denotes the PDF of the central beta distribution.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("BinomialPdf(x, a, b): ", BinomialPdf(x, a, b))
>>> print ("dist_binomial(a, b).pdf(x): ", dist_binomial(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**binomial_cdf**(*n*, *p*, *k*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the binomial distribution:

$$\text{cdf}(x) = \sum_{j=0}^{k} \text{pmf}_X(j) = \text{ibetac}(k+1, n-k, p).$$

Here ibetac$(\cdot)$ denotes the real normalised complementary incomplete beta function (*RealIBetac*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("BinomialCdf(x, a, b): ", BinomialCdf(x, a, b))
>>> print ("dist_binomial(a, b).cdf(x): ", dist_binomial(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

math53.**binomial_qtf**(*n*, *p*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the binomial distribution:

There is no known closed form for qtf(q): it is computed with Newton iterations where the starting values are from the corresponding Boost functions.

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("BinomialQtf(q, a, b): ", BinomialQtf(q, a, b))
>>> print ("dist_binomial(a, b).qtf(q): ", dist_binomial(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_binomial**(n, p)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The binomial distribution is a discrete (lattice) probability distribution with number of trials $n \geq 0$ and success probability $0 \leq p \leq 1$. See also Wikipedia [1173], MathWorld [816], BoostMath [71], Witkovský [1482], R (Statistical System) [530].

A special case is the Bernoulli distribution, see See also Wikipedia [1172], MathWorld [815], and BoostMath [70].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.bdtr.html#scipy.special.bdtr

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.bdtrc.html#scipy.special.bdtrc

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.bdtri.html#scipy.special.bdtri

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.bdtrik.html#scipy.special.bdtrik

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.bdtrin.html#scipy.special.bdtrin

dist_binomial.**pmf**(x)

Returns $\mathrm{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following an binomial distribution:

$$\mathrm{pmf}_X(x) = \binom{n}{k} p^k (1-p)^{n-k} = f_{\mathrm{Beta}}(k+1, n-k+1, p)/(n+1).$$

and $f_{\mathrm{Beta}}(\cdot)$ and $F_{\mathrm{Beta}}(\cdot)$ denote the PDF and CDF, respectively, of the central beta distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", binomial(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

dist_binomial.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an binomial distribution:

$$\mathrm{cdf}_X(x) = \sum_{j=0}^{k} \mathrm{pmf}_X(j) = F_{\mathrm{Beta}}(1-p; n-k, k+1).$$

and $f_{\mathrm{Beta}}(\cdot)$ and $F_{\mathrm{Beta}}(\cdot)$ denote the PDF and CDF, respectively, of the central beta distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", binomial(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

dist_binomial.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following an binomial distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", binomial(mu, sigma).pmf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_binomial.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an binomial distribution. There is no closed form for the qtf: It is computed with Newton iterations where the starting values are from Boost.

$$\text{qtf}_X(q) = tbd.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", binomial(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_binomial.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an binomial distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", binomial(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_binomial.**g_x**(t)

Returns $G_X(t)$, the probability generating function of a random variable X, following an binomial distribution:

$$G_X(t) = (Pt + Q)^n.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", binomial(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_binomial.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an binomial distribution:

$$C_X(t) = \left(Pe^{it} + Q\right)^n.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", binomial(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_binomial.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following an binomial distribution:

$$M_X(t) = \left(Pe^t + Q\right)^n.$$

$$L_X(t) = \left(Pe^{-t} + Q\right)^n.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", binomial(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_binomial.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an binomial distribution:

$$K_X(t) = K_X(t) = n \log\left(Pe^t + Q\right).$$

$K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a binomial distribution, are defined as

$$K_X(t) = n \log\left(p \cdot e^t + 1 - p\right),$$

$$K_X^{(1)}(t) = \frac{n \cdot p \cdot e^x}{p(e^x - 1) + 1},$$

$$K_X^{(2)}(t) = -\frac{n(p-1)p \cdot e^x}{(p(e^x - 1) + 1)^2},$$

$$K_X^{(3)}(t) = \frac{n(p-1)p \cdot e^x (p \cdot e^x + p - 1)}{(p(e^x - 1) + 1)^3},$$

and for $j \geq 4$ the derivatives are calculated by numerically differentiating $K_X^{(3)}(t)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", binomial(mu, sigma).k_x(t, k))
6.356352346256452561561561561456 1356E+00
```

dist_binomial.**moments**(*k*)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an binomial distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(r) = nPQ \sum_{i=0}^{r-2} \binom{r-1}{i} \mu_i - P \sum_{i=0}^{r-2} \binom{r-1}{i} \mu_{i+1}, \quad r > 2; \quad \mu_1 = nP, \quad \mu_2 = nPQ.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", binomial(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

dist_binomial.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an binomial distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", binomial(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

Approximations

ctx.**hypergeo_ft**(x, n, *results='cdf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the pdf, cdf and sf from the characteristic function (see *pmf_from_cf_lattice()* and *cdf_from_cf_lattice()*).

ctx.**binomial_ft**(x, n, *results='cdf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the pdf, cdf and sf from the characteristic function (see *pmf_from_cf_lattice()* and *cdf_from_cf_lattice()*).

ctx.**binomial_ecf**(k, n, p, *results='cdf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Edgeworth approximation to the pdf, cdf and sf, support $k \in \{0, \cdots n\}$.

ctx.**binomial_ecf_inv**(q, n, p, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**binomial_spa**(k, n, p, *results='cdf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

The saddlepoint is given by

$$\hat{s}(x) = \log\left(\frac{kQ}{(n-k)P}\right).$$

ctx.**binomial_spa_inv**(q, n, p, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.13.5 Boost: Negative binomial distribution

The following functions return the pmf, cdf, qtf or boost class of the negative binomial distribution with target for number of successful trials $r > 0$ and success probability $0 \le p \le 1$, and $0 \le q \le 1$.

See also Wikipedia [1188], MathWorld [832], BoostMath [83], Ehrhardt [297] (3.9.23).

`math53.`**`negbinomial_pmf`**(r, p, k)

Returns $\text{pmf}(x)$, the value of the probability mass function (*Pmf*) of the negative binomial distribution:

$$\text{pmf}(x) = \frac{\Gamma(k+r)}{k!\Gamma(r)} p^r (1-p)^k.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("NegbinomPdf(x, a, b): ", NegbinomPdf(x, a, b))
>>> print ("dist_negbinomial(a, b).pdf(x): ", dist_negbinomial(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`negbinomial_cdf`**(r, p, k)

where `Ctx` is `Math53` or `CtxBoost`.

Returns $\text{cdf}(x)$, the value of the cumulative distribution function (*Cdf*) of the negative binomial distribution:

$$\text{cdf}(x) = \sum_{j=0}^{k} \text{pmf}_X(j) = \text{ibeta}(r, k+1, p).$$

Here $\text{ibeta}(\cdot)$ denotes the real normalised incomplete beta function (*RealIBeta*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("NegbinomCdf(x, a, b): ", NegbinomCdf(x, a, b))
>>> print ("dist_negbinomial(a, b).cdf(x): ", dist_negbinomial(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.`**`negbinomial_qtf`**(r, p, q)

Returns $\text{qtf}(q)$, the value of the quantile function (*Qtf*) of the negative binomial distribution:

$$\text{qtf}(q) = \text{ibeta_invb}(r, p, q) - 1.$$

Here $\text{ibeta_invb}(\cdot)$ denotes the inverse (on parameter b) of the real normalised incomplete beta function (*RealIBetaInvb*).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("NegbinomQtf(q, a, b): ", NegbinomQtf(q, a, b))
>>> print ("dist_negbinomial(a, b).qtf(q): ", dist_negbinomial(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_negbinom**(r, p)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The negative binomial distribution is a discrete (lattice) probability distribution with target for number of successful trials $r > 0$ and success probability $0 \leq p \leq 1$. See also Wikipedia [1188], MathWorld [832], BoostMath [83] , and Witkovský [1489], R (Statistical System) [533].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nbdtr.html#scipy.special.nbdtr

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nbdtrc.html#scipy.special.nbdtrc

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nbdtri.html#scipy.special.nbdtri

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nbdtrik.html#scipy.special.nbdtrik

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.nbdtrin.html#scipy.special.nbdtrin

dist_negbinom.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following an negative binomial distribution:

$$\text{pmf}_X(x) = \frac{\Gamma(n+k)}{K!\Gamma(n)} P^n (1-P)^k.$$

and $f_{\text{Beta}}(\cdot)$ and $F_{\text{Beta}}(\cdot)$ denote the PDF and CDF, respectively, of the central beta distribution. The following recursions are used for the PMF:

$$\Pr(X = k+1|n) = \frac{(n+k)(1-P)}{(k+1)} \Pr(X = k|n)$$

$$\Pr(X = k-1|n) = \frac{k}{(n-k+1)(1-P)} \Pr(X = k|n)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", negative_binomial(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

dist_negbinom.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an negative binomial distribution:

$$\text{cdf}_X(x) = \sum_{j=0}^{k} \text{pmf}_X(j) = F_{\text{Beta}}(1-p; r, k+1).$$

and $f_{\text{Beta}}(\cdot)$ and $F_{\text{Beta}}(\cdot)$ denote the PDF and CDF, respectively, of the central beta distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", negative_binomial(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

dist_negbinom.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following an negative binomial distribution:

$$\mathrm{sf}_X(x) = 1 - \mathrm{cdf}_X(x).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", negative_binomial(mu, sigma).pmf(x))
sf: 6.356352346256452561561561561456 1356E-20
```

dist_negbinom.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an negative binomial distribution:

$$\mathrm{qtf}_X(q) = a - b \ln\left((1-y)/y\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", negative_binomial(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456 1356E+00
```

dist_negbinom.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an negative binomial distribution:

$$\mathrm{isf}_X(q) = \mathrm{qtf}_X(1-q) = \mathrm{cdf}_X^{-1}(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", negative_binomial(mu, sigma).isf(q))
6.356352346256452561561561561456 1356E+00
```

dist_negbinom.**g_x**(t)

Returns $G_X(t)$, the probability generating function of a random variable X, following an negative binomial distribution:

$$G_X(t) = \left(\frac{1}{P} - \frac{1-P}{P}t\right)^{-n}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", negative_binomial(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_negbinom.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an negative binomial distribution:

$$C_X(t) = \left(\frac{1}{P} - \frac{1-P}{P}e^{it}\right)^{-n}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", negative_binomial(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_negbinom.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following an negative binomial distribution:

$$M_X(t) = \left(\frac{1}{P} - \frac{1-P}{P}e^t\right)^{-n}.$$

$$L_X(t) = \left(\frac{1}{P} - \frac{1-P}{P}e^t\right)^{-n}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", negative_binomial(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

dist_negbinom.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an negative binomial distribution:

$$K_X(t) = -n\log\left(\frac{1}{P} - \frac{1-P}{P}e^t\right).$$

$K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1\ldots k$, of a random variable X, following a negative binomial distribution, are defined as

$$K_X(t) = r\log\left(\frac{1-p}{1-p\cdot e^t}\right), \quad \text{for } t < -\log(p)$$

$$K_X^{(1)}(t) = \frac{p\cdot r\cdot e^x}{1-p\cdot e^x},$$

$$K_X^{(2)}(t) = \frac{p\cdot r\cdot e^x}{(1-p\cdot e^x)^2},$$

$$K_X^{(3)}(t) = \frac{p\cdot r\cdot e^x(p\cdot e^x+1)}{(1-p\cdot e^x)^3},$$

and for $j \geq 4$ the derivatives are calculated by numerically differentiating $K_X^{(3)}(t)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", negative_binomial(mu, sigma).k_x(t, k))
6.356352346256452561561561561456 1356E+00
```

dist_negbinom.**moments**(k)

Returns the first j central moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an negative binomial distribution (Wikipedia). The raw moments are calculated from the central moments.

$$\mu_X(r) = \mu_2 \sum_{i=0}^{r-2} \binom{r-1}{i} \mu_i - \frac{1-P}{P} \sum_{i=0}^{r-2} \binom{r-1}{i} \mu_{i+1}, \quad r > 2; \quad \mu_1 = n\frac{1-P}{P}, \quad \mu_2 = \frac{\mu_1}{P}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", negative_binomial(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_negbinom.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an negative binomial distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", negative_binomial(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Approximations

ctx.**negbinom_ecf**(k, r, p, *results='cdf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Edgeworth approximation to the pdf, cdf and sf

ctx.**negbinom_ecf_inv**(q, r, p, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**negbinom_spa**(k, r, p, *results='c'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

The saddlepoint is given by:

$$\hat{s}(x) = \log\left(\frac{k(1-P)}{(n-k)P}\right).$$

ctx.**negbinom_spa_inv**(q, r, p, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.13.6 Boost: Classical hypergeometric distribution

The following functions return pmf, cdf, qtf or boost class of the hypergeometric distribution, with k successes (random draws for which the object drawn has a specified feature) in $n \in \{0, 1, 2, \ldots, N\}$ draws, without replacement, from a finite population of size $N \in \{0, 1, 2, \ldots\}$‘ that contains exactly $K \in \{0, 1, 2, \ldots, N\}$ objects with that feature, wherein each draw is either a success or a failure, and the support interval $(\max(0, n + K - N), \min(K, n))$, and $0 \leq q \leq 1$.

See also Wikipedia [1179], MathWorld [824], BoostMath [78], Team [532], Berkopec [31], Johnson *et al.* [389] page 251, Ehrhardt [297] (3.9.11).

`math53.hypergeometric_pmf`(*r*, *n*, *N*, *k*)

Returns pmf(k), the value of the probability mass function (*Pmf*) of the hypergeometric distribution.

$$\text{pmf}_X(k) = \frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}.$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("HypergeoPdf(x, a, b): ", HypergeoPdf(x, a, b))
>>> print ("dist_hypergeometric(a, b).pdf(x): ", dist_hypergeometric(a, b).pdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.hypergeometric_cdf`(*r*, *n*, *N*, *k*)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the hypergeometric distribution:

$$\text{cdf}_X(k) = \sum_{j=\max(0,n+K-N)}^{k} \text{pmf}_X(j).$$

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; x = 0.6;
>>> print ("HypergeoCdf(x, a, b): ", HypergeoCdf(x, a, b))
>>> print ("dist_hypergeometric(a, b).cdf(x): ", dist_hypergeometric(a, b).cdf(x))
6.35635234625645256156156145613 56E+00
```

`math53.hypergeometric_qtf`(*r*, *n*, *N*, *q*)

Returns qtf(q), the value of the quantile function (*Qtf*) of the hypergeometric distribution:

There is no known closed exact form for qtf(q) or isf(q).

The following example shows both forms of the syntax:

```
>>> from xlcalcnet import *
>>> a = 0; b = 1; t = 0.3; q = 0.6;
>>> print ("HypergeoQtf(q, a, b): ", HypergeoQtf(q, a, b))
>>> print ("dist_hypergeometric(a, b).qtf(q): ", dist_hypergeometric(a, b).qtf(q))
6.35635234625645256156156145613 56E+00
```

class ctx.**dist_hypergeometric**(n, K, N)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The hypergeometric distribution is a discrete (lattice) probability distribution with k successes (random draws for which the object drawn has a specified feature) in $n \in \{0, 1, 2, \ldots, N\}$ draws, without replacement, from a finite population of size $N \in \{0, 1, 2, \ldots\}$‘ that contains exactly $K \in \{0, 1, 2, \ldots, N\}$ objects with that feature, wherein each draw is either a success or a failure, and the support interval $(\max(0, n+K-N), \min(K, n))$. See also Wikipedia [1179], MathWorld [824], BoostMath [78], R (Statistical System) [532], Berkopec [31], Johnson *et al.* [389] page 251.

dist_hypergeometric.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following an hypergeometric distribution:

$$\text{pmf}_X(x) = \binom{K}{k}\binom{N-K}{n-k}\Big/\binom{N}{n}.$$

The following recursions are used for the PMF:

$$f(k+1) = \frac{(n_1-k)(n-k)}{(k+1)(n_2-n+k+1}f(k)$$

$$f(k-1) = \frac{k(n_2-n+k)}{(n_1-k+1)(n-k+1}f(k)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", hypergeometric(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_hypergeometric.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following an hypergeometric distribution:

$$\text{cdf}_X(k) = \sum_{j=\max(0,n+K-N)}^{k} \text{pmf}_X(j) = 1 - \text{pmf}_X(k+1) \times {}_3F_2(1, k+1-K, k+1-n; k+2, N+k+2-K-n; 1),$$

where ${}_3F_2(\cdot)$ is a generalized hypergeometric function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", hypergeometric(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_hypergeometric.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following an hypergeometric distribution:

$$\text{sf}_X(k) = \sum_{j=k+1}^{\min(K,n)} \text{pmf}_X(j) = \text{pmf}_X(k+1) \times {}_3F_2(1, k+1-K, k+1-n; k+2, N+k+2-K-n; 1),$$

where ${}_3F_2(\cdot)$ is a generalized hypergeometric function.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", hypergeometric(mu, sigma).pmf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_hypergeometric.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following an hypergeometric distribution.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", hypergeometric(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

dist_hypergeometric.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following an hypergeometric distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", hypergeometric(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_hypergeometric.**g_x**(t)

Returns $G_X(t)$, the probability generating function of a random variable X, following an hypergeometric distribution:

$$G_X(t) = {}_2F_1(-n, -K; N-K-n+1; t)\binom{N-K}{n} \Big/ \binom{N}{n}.$$

where ${}_2F_1(\cdot)$ is the Gauss hypergeometric function (see *hyp2f1()*.)

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", hypergeometric(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_hypergeometric.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following an hypergeometric distribution:

$$C_X(t) = {}_2F_1(-n, -K; N-K-n+1; e^{it})\binom{N-K}{n} \Big/ \binom{N}{n}.$$

where ${}_2F_1(\cdot)$ is the Gauss hypergeometric function (see *hyp2f1()*.)

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", hypergeometric(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

`dist_hypergeometric.m_x(t)`

Returns $M_X(t)$, the moment generating function of a random variable X, following an hypergeometric distribution:

$$M_X(t) = {}_2F_1(-n, -K; N-K-n+1; e^t)\binom{N-K}{n}\Big/\binom{N}{n}.$$

$$L_X(t) = {}_2F_1(-n, -K; N-K-n+1; e^{-t})\binom{N-K}{n}\Big/\binom{N}{n}.$$

where ${}_2F_1(\cdot)$ is the Gauss hypergeometric function (see *hyp2f1()*.)

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", hypergeometric(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_hypergeometric.k_x(t, k=0)`

Returns $K_X(t)$, the cumulant generating function of a random variable X, following an hypergeometric distribution:

$$K_X(t) = \log\left[({}_2F_1(-n, -K; N-K-n+1; e^t)\binom{N-K}{n}\Big/\binom{N}{n}\right].$$

where ${}_2F_1(\cdot)$ is the Gauss hypergeometric function (see *hyp2f1()*.)

$K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a hypergeometric distribution, are defined as

$$K_X(t) = \log\left(T \cdot {}_2F_1(a, b; c; e^t)\right), \quad \text{where } a = -n, b = -K, c = N-K-n+1, T = \binom{N-K}{n}\Big/\binom{N}{n}.$$

$$K_X^{(1)}(t) = \frac{a \cdot b \cdot e^x \cdot G_1(x)}{c \cdot G_0(x)}, \quad \text{where } G_i(x) = {}_2F_1(a+i, b+i; c+i; e^t),$$

$$K_X^{(2)}(t) = -\frac{a^2 \cdot b^2 \cdot e^{2x} \cdot (G_1(x))^2}{c^2 \cdot (G_0(x))^2} + \frac{a \cdot b \cdot e^x \cdot G_1(x)}{c \cdot G_0(x)} + \frac{a(a+1) \cdot b(b+1) \cdot e^{2x} \cdot G_2(x)}{c(c+1) \cdot G_0(x)},$$

$$K_X^{(3)}(t) = \frac{2a^3 \cdot b^3 \cdot e^{3x} \cdot (G_1(x))^3}{c^3 \cdot (G_0(x))^3} - \frac{3a^2 \cdot b^2 \cdot e^{2x} \cdot (G_1(x))^2}{c^2 \cdot (G_0(x))^2} - \frac{3a^2(a+1) \cdot b^2(b+1) \cdot e^{3x} \cdot G_2(x) \cdot G_1(x)}{c(c+1) \cdot G_0(x)}$$
$$+ \frac{a \cdot b \cdot e^x \cdot G_1(x)}{c \cdot G_0(x)} + \frac{3a(a+1) \cdot b(b+1) \cdot e^{2x} \cdot G_2(x)}{c(c+1) \cdot G_0(x)} + \frac{a(a+1)(a+2) \cdot b(b+1)(b+2) \cdot e^{3x} \cdot G_3(x)}{c(c+1)(c+2) \cdot G_0(x)}$$

and for $j \geq 4$ the derivatives are calculated by numerically differentiating $K_X^{(3)}(t)$. We also have

$$\frac{\mathrm{d}}{\mathrm{d}z}\left({}_2F_1(a, b, c; z)\right) = \frac{ab}{c} \cdot {}_2F_1(a+1, b+1, c+1; z), \quad \text{and}$$

$$z(1-z) \cdot \frac{\mathrm{d}^2}{\mathrm{d}z^2}\left({}_2F_1(a, b, c; z)\right) + (c - (a+b+1)z) \cdot \frac{\mathrm{d}}{\mathrm{d}z}\left({}_2F_1(a, b, c; z)\right) - ab \cdot {}_2F_1(a, b, c; z) = 0.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", hypergeometric(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_hypergeometric.**moments**(*k*)

Returns the first j raw moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following an hypergeometric distribution (Wikipedia).

$$\mu'_X(r) = \sum_{j=1}^{j_{\max}} S_{r,j} T_j, \quad \text{where } j_{\max} = \min(r, n, M, N),$$

$$S_{r,1} = S_{r,r} = 1, \quad S_{r+1,j} = jS_{r,j} + S_{r,j-1}, \quad \text{and}$$

$$T_1 = nM/N, \quad T_{j+1} = T_j(n-j)(M-j)/(N-j).$$

$$\mu'_{[r]} = \frac{n!K!(K+(N-K)-r)!}{(n-r)!(K-r)!(K+(N-K))!}$$

$$\mu'_{[r]} = \frac{n!K!(N-r)!}{(n-r)!(K-r)!N!}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", hypergeometric(mu, sigma).moments(k))
6.35635234625645256156156145614356E+00
```

dist_hypergeometric.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following an hypergeometric distribution. The cumulants are calculated from the moments.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", hypergeometric(mu, sigma).cumulants(k))
6.35635234625645256156156145614356E+00
```

Approximations

ctx.**hypergeo_ecf**(*k*, *n*, *K*, *N*, *results='cdf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

ctx.**hypergeo_ecf_inv**(*q*, *n*, *K*, *N*, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**hypergeo_spa**(*k*, *n*, *K*, *N*, *results='cdf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

ctx.**hypergeo_spa_inv**(*q*, *n*, *K*, *N*, *results='qtf'*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.14 Discrete (lattice) distributions related to (stratified) rank tests

4.14.1 Wilcoxon signed rank T distribution, continuous data

class ctx.**dist_wilcoxon**(N)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Wilcoxon's signed rank test is a discrete (lattice) probability distribution with sample size $N > 2$ and the support interval $(0, N(N+1))$. See also Wikipedia [1195], R (Statistical System) [529], Fellingham and Stoker [300], vandeWiel [789], Bennett [29] and Zimmermann [1501].

We consider N continuously distributed random variables $D_i, i = 1 \ldots N$, with common pdf h_0. In a sample $(d_1, \ldots, d_N)$ of size N let r_i be the rank of d_i in the ordered sample.

The test criterion of Wilcoxon's Signed Rank is $T_N = \sum_{i=1}^{N} S(d_i) r_i$, where $S(d_i) = 1$ for $x > 0$ and $S(d_i) = 0$ for $x < 0$. T_N can assume values between 0 and $\frac{1}{2}N(N+1)$ in steps of 1.

dist_wilcoxon.**pmf**(x)

Returns $\mathrm{pmf}_X(k)$, the probability mass function (pmf) of a random variable X, following the distribution of Wilcoxon's signed rank test, with sample size $N > 2$ and the support interval $(0, N(N+1)/2)$.

$$\mathrm{pmf}_X(x) = \sum_{j=x}^{N(N+1)/2} (-1)^{x+j} \binom{j}{x} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The factorial moments are calculated from the cumulants (see *factorial_moments_from_cumulants()*).

The pmf can also be calculated from the characteristic function $C_X(t)$:

$$\mathrm{pmf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re\left(e^{-itx} C_X(t)\right) \mathrm{d}t,$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", wilcoxon_continuous(mu, sigma).pmf(x))
6.35635234625645256156156145561356E-20
```

dist_wilcoxon.**cdf**(x)

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Wilcoxon signed rank T distribution.

$$\mathrm{cdf}_X(x) = \sum_{j=x}^{N(N+1)/2} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The cdf can also be calculated from the characteristic function $C_X(t)$:

$$\mathrm{cdf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re\left(C_X(t) \sum_{z=0}^{x} e^{-itz}\right) \mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", wilcoxon_continuous(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

dist_wilcoxon.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Wilcoxon signed rank T distribution:

$$\mathrm{sf}_X(x) = 1 - \sum_{j=x}^{N(N+1)/2} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The cdf can also be calculated from the characteristic function $C_X(t)$:

$$\mathrm{sf}(x) = 1 - \mathrm{cdf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re \left(C_X(t) \sum_{z=x+1}^{N(N+1)/2} e^{-itz} \right) \mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", wilcoxon_continuous(mu, sigma).pmf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_wilcoxon.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Wilcoxon signed rank T distribution. There is no closed form for the qtf: It is computed using the Brent algorithm with starting values from a Cornish-Fisher or Jensen approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", wilcoxon_continuous(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_wilcoxon.**isf**(q)

Returns $\mathrm{isf}_X(x)$, the inverse survival function (isf) of a random variable X, following a Wilcoxon signed rank T distribution. There is no closed form for the isf: It is computed using the Brent algorithm with starting values from a Cornish-Fisher or Jensen approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", wilcoxon_continuous(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_wilcoxon.**g_x**(t)

Returns $G_X(x)$, the probability generating function of a random variable X, following a Wilcoxon signed rank T distribution:

$$G_X(x) = \frac{1}{2^N} \prod_{h=1}^{N} (1 + x^h)$$

See also: v.d.Wiel, p. 17, equ. 2.15

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wilcoxon_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wilcoxon.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Wilcoxon signed rank T distribution:

$$C_X(t) = \exp\left(\tfrac{1}{4}N(N+1)it\right) \prod_{h=1}^{N} \cosh\left(\tfrac{h}{2}it\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wilcoxon_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wilcoxon.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Wilcoxon signed rank T distribution:

$$M_X(t) = \exp\left(\tfrac{1}{4}N(N+1)t\right) \prod_{h=1}^{N} \cosh\left(\tfrac{h}{2}t\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", wilcoxon_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_wilcoxon.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Wilcoxon signed rank T distribution:

$$K_X(t) = \tfrac{1}{4}N(N+1)t + \sum_{h=1}^{N} \log\left(\cosh\left(\tfrac{h}{2}t\right)\right).$$

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Wilcoxon signed rank T distribution:

$$K_X(t) = \tfrac{1}{2}n\log(4p(1-p)) + \tfrac{1}{4}n(n+1)t + \sum_{h=1}^{n} \log\left(\cosh\left(\tfrac{1}{2}(\log(p/(1-p)) + h\cdot t)\right)\right).$$

$K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1\ldots k$, of a random variable X, following a Wilcoxon signed rank T distribution, are defined as:

$$K_X(t) = \tfrac{1}{4}n(n+1)t + \sum_{h=1}^{n} \log\left(\cosh\left(\tfrac{h}{2}t\right)\right),$$

$$K_X^{(1)}(t) = \sum_{h=1}^{n} \frac{h}{2}\left(1 - \frac{2}{\exp(h\cdot s)+1}\right),$$

See also `Bennett1` in `DistCiornishArb.vb`.

$$K_X^{(j)}(t) = \sum_{h=1}^{n} h^j \cdot \sum_{k=1}^{j} c(j-2,k)\left(\exp(h\cdot s)+1\right)^{-k}, \quad j \geq 2.$$

where the coefficients $c(i,j)$ are calculated recursively, with $c(0,1) = c(0,2) = c(i,1) = 1$, and

$$c(i,j) = (j-1)\cdot c(i-1,j-1) + j\cdot c(i-1,j), \quad j \geq 2.$$

The saddlepoint s is determined numerically using Newton iterations, with a starting value of $s = 0$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", wilcoxon_continuous(mu, sigma).k_x(t, k))
6.356352346256452561561561561456 1356E+00
```

dist_wilcoxon.**moments**(*k*)

Returns the first j moments, $\mu_j, j = 1\ldots k$, of a random variable X, following a Wilcoxon signed rank T distribution. The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wilcoxon_continuous(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

dist_wilcoxon.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1\ldots k$, of a random variable X, following a Wilcoxon signed rank T distribution. The cumulants of W are given by:

$$\kappa_{2j} = \frac{2^{2j}(2^{2j}-1)B_{2j}}{2j}\sum_{i=1}^{N} r_i^{2j} = \frac{2^{2j}(2^{2j}-1)B_{2j}}{2j}\frac{B_{2j+1}(N+1) - B_{2j+1}}{2j+1},$$

where $\kappa_1 = N(N+1)/4$), $\kappa_{2j+1} = 0$ for $j \geq 1$, and B_{2j} and $B_{2j}(x)$ are the Bernoulli numbers and polynomials, respectively, of degree $2j$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wilcoxon_continuous(mu, sigma).cumulants(k))
6.35635234625645256156156156145613 56E+00
```

Approximations

`ctx.`**`wilcoxon_ft`**(*x*, *n*, *results='cdf'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the pdf, cdf and sf from the characteristic function (see *pmf_from_cf_lattice()* and *cdf_from_cf_lattice()*).

`ctx.`**`wilcoxon_ecf`**(*x*, *f*, *rho*, *omega*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

`ctx.`**`wilcoxon_ecf_inv`**(*q*, *f*, *rho*, *omega*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

`ctx.`**`wilcoxon_spa`**(*x*, *n*, *results='c'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

The saddlepoint s is determined numerically using Newton iterations, with a starting value of $s = 0$.

`ctx.`**`wilcoxon_spa_inv`**(*q*, *n*, *results='qtf'*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.14.2 Noncentral Wilcoxon signed rank T distribution, Bennett alternatives

class ctx.**dist_bennett**(N)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Wilcoxon's signed rank test is a discrete (lattice) probability distribution with sample size $N > 2$ and the support interval $(0, N(N+1))$. See also Wikipedia [1195], R (Statistical System) [529], Fellingham and Stoker [300], vandeWiel [789], Bennett [29] and Zimmermann [1501].

We consider N continuously distributed random variables $D_i, i = 1 \ldots N$, with common pdf h_0. In a sample $(d_1, \ldots, d_N)$ of size N let r_i be the rank of d_i in the ordered sample.

The test criterion of Wilcoxon's Signed Rank is $T_N = \sum_{i=1}^{N} S(d_i) r_i$, where $S(d_i) = 1$ for $x > 0$ and $S(d_i) = 0$ for $x < 0$. T_N can assume values between 0 and $\frac{1}{2}N(N+1)$ in steps of 1.

dist_bennett.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Wilcoxon signed rank T distribution. The null distribution can be calculated as follows: Let $p_N(w)$ denote the probability $\Pr[W_N = w]$ in a sample of size N. Then the following recurrence relation holds (Zimmermann_1985_dependent) :

$$p_N(w) = \tfrac{1}{2}\left(p_{N-1}(w) + p_{N-1}(w - N)\right).$$

$$\text{pmf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", wilcoxon_continuous(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

dist_bennett.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Wilcoxon signed rank T distribution.

$$\text{cdf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", wilcoxon_continuous(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

dist_bennett.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a Wilcoxon signed rank T distribution:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", wilcoxon_continuous(mu, sigma).pmf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_bennett.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Wilcoxon signed rank T distribution. There is no closed form for the qtf: It is computed with Newton iterations where the starting values are from Boost.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", wilcoxon_continuous(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145614356E+00
```

dist_bennett.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Wilcoxon signed rank T distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", wilcoxon_continuous(mu, sigma).isf(q))
6.35635234625645256156156145614356E+00
```

dist_bennett.**g_x**(t)

Returns $G_X(t)$, the probability generating function of a random variable X, following a Wilcoxon signed rank T distribution:

$$G_X(t) = \prod_{h=1}^{n} \left(pt^h + (1-p)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wilcoxon_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145614356E+00
```

dist_bennett.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Wilcoxon signed rank T distribution:

$$C_X(t) = \exp\left(\tfrac{1}{4}n(n+1)it\right) \prod_{h=1}^{n} \cosh\left(\tfrac{h}{2}it\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", wilcoxon_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145614356E+00
```

dist_bennett.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Wilcoxon signed rank T distribution:

$$M_X(t) = (4p(1-p))^{\frac{1}{2}n} \exp\left(\tfrac{1}{4}n(n+1)t\right) \prod_{h=1}^{n} \cosh\left(\tfrac{1}{2}(\log(p/(1-p)) + h \cdot t)\right), \quad a =$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", wilcoxon_continuous(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_bennett.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Wilcoxon signed rank T distribution:

$$K_X(t) = \tfrac{1}{2}n\log(4p(1-p)) + \tfrac{1}{4}n(n+1)t + \sum_{h=1}^{n}\log\left(\cosh\left(\tfrac{1}{2}(\log(p/(1-p)) + h\cdot t)\right)\right).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", wilcoxon_continuous(mu, sigma).k_x(t, k))
6.356352346256452561561561456135 6E+00
```

dist_bennett.**moments**(*k*)

Returns the first j moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Wilcoxon signed rank T distribution (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wilcoxon_continuous(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_bennett.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Wilcoxon signed rank T distribution. The cumulants of W are given by (Fellingham_1964) :

Central case:

$$\kappa_{2j}(W_N) = \frac{2^{2j}(2^{2j}-1)B_{2j}}{2j}\sum_{i=1}^{N} r_i^{2j} = \frac{2^{2j}(2^{2j}-1)B_{2j}}{2j}\,\frac{B_{2j+1}(N+1) - B_{2j+1}}{2j+1}, \quad \text{and}$$

$$\kappa_{2j+1}(W_N) = 0, \quad \text{for } j \geq 0.$$

Noncentral case:

$$\kappa_1 = \tfrac{1}{4}n(n+1)(1+\tanh(0.5a)) = \tfrac{1}{2}n(n+1)p$$

$$\kappa_2 = \tfrac{1}{6}n(n+1)(2n+1)p(1-p)$$

$$\kappa_3 = \tfrac{1}{4}n^2(n+1)^2pq(q-p)$$

$$\kappa_4 = \tfrac{1}{30}n(n+1)(2n+1)(3n^2+3n-1)pq(1-6pq)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", wilcoxon_continuous(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

4.14.3 Mann-Whitney U distribution, continuous data

class ctx.**dist_mann_whitney_u**(*N1*, *N2*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Mann-Whitney's U is a discrete (lattice) probability distribution with sample sizes $N_1 \geq 1$ and $N_2 \geq 1$ and the support interval $(0, N_1 \cdot N_2))$. See also Wikipedia [1185], R (Statistical System) [528], Murakami and Kamakura [420], vandeWiel [789], Robillard [482] and Zimmermann [1502].

Let $x_1, \ldots, x_{N_1}$ and $y1, \ldots, y_{N_2}$ be two sets of measurements, which we denote by X and Y. The test criterion U of the Mann-Whitney test is then

$$U = \sum_{j=1}^{N_1} \sum_{k=1}^{N_2} \operatorname{sgn}(x_k - y_k)$$

dist_mann_whitney_u.**pmf**(x)

Returns $\operatorname{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Mann-Whitney U distribution.

$$\operatorname{pmf}_X(x) = \sum_{j=x}^{N_1 \cdot N_2} (-1)^{x+j} \binom{j}{x} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The factorial moments are calculated from the cumulants (see *factorial_moments_from_cumulants()*).

The pmf can also be calculated from the characteristic function $C_X(t)$:

$$\operatorname{pmf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re\left(e^{-itx} C_X(t)\right) \mathrm{d}t,$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", mann_whitney_u_continuous(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_mann_whitney_u.**cdf**(x)

Returns $\operatorname{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Mann-Whitney U distribution.

$$\operatorname{cdf}_X(x) = \sum_{j=x}^{N_1 \cdot N_2} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The cdf can also be calculated from the characteristic function $C_X(t)$:

$$\operatorname{cdf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re\left(C_X(t) \sum_{z=0}^{x} e^{-itz}\right) \mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mann_whitney_u_continuous(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_mann_whitney_u.**sf**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Mann-Whitney U distribution:

$$\mathrm{sf}_X(x) = 1 - \sum_{j=x}^{N_1 \cdot N_2} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The cdf can also be calculated from the characteristic function $C_X(t)$:

$$\mathrm{sf}(x) = 1 - \mathrm{cdf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re \left(C_X(t) \sum_{z=x+1}^{N_1 \cdot N_2} e^{-itz} \right) \mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mann_whitney_u_continuous(mu, sigma).pmf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_mann_whitney_u.**qtf**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Mann-Whitney U distribution. There is no closed form for the qtf: It is computed using the Brent algorithm with starting values from a Cornish-Fisher or Jensen approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mann_whitney_u_continuous(mu, sigma).qtf(q))
qtf: 6.356352346256452561561561561456135 6E+00
```

dist_mann_whitney_u.**isf**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Mann-Whitney U distribution. There is no closed form for the isf: It is computed using the Brent algorithm with starting values from a Cornish-Fisher or Jensen approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mann_whitney_u_continuous(mu, sigma).isf(q))
6.356352346256452561561561561456135 6E+00
```

dist_mann_whitney_u.**g_x**(t)

Returns $G_X(t)$, the probability generating function of a random variable X, following a Mann-Whitney U distribution:

$$G_X(t) = \frac{1}{\binom{N_1+N_2}{N_1}} \frac{\prod_{r=N_1+1}^{N_1+N_2} (1-x^r)}{\prod_{r=1}^{N_2} (1-x^r)}.$$

See also: v.d.Wiel, p. 14, equ. 2.5

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mann_whitney_u_continuous(mu, sigma).c_x(t))
6.356352346256452561561561561456135 6E+00
```

dist_mann_whitney_u.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Mann-Whitney U distribution:

$$C_X(t) = \prod_{r=1}^{N_2} \frac{r}{N_2 + r} \frac{1 - \exp((N_2 + r)it)}{1 - \exp(itr)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mann_whitney_u_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mann_whitney_u.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Mann-Whitney U distribution:

$$M_X(t) = \prod_{r=1}^{N_2} \frac{r}{N_2 + r} \frac{1 - \exp((N_2 + r)t)}{1 - \exp(tr)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", mann_whitney_u_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mann_whitney_u.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Mann-Whitney U distribution:

$$K_X(t) = \sum_{r=1}^{N_2} \log\left(\frac{r}{N_2 + r} \frac{1 - \exp((N_2 + r)t)}{1 - \exp(tr)}\right).$$

$K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a Mann-Whitney U distribution, are defined as

$$K_X(t) = \sum_{r=1}^{m} \log\left(\frac{r}{n + r} \frac{1 - \exp((n + r)t)}{1 - \exp(t \cdot r)}\right),$$

$$K_X^{(1)}(s) = \sum_{r=1}^{m} \left(\frac{n + r}{1 - \exp((n + r)s)} - \frac{n(\exp(r \cdot s) + n + r)}{1 - \exp(r \cdot s)}\right),$$

See also `Murakami1_new` in `DistCiornishArb.vb`.

$$K_X^{(j)}(s) = (-1)^j \sum_{r=1}^{m} \sum_{i=0}^{1} (-1)^i \cdot t_i^j \cdot \sum_{k=1}^{j} c(j-2, k) \cdot z_i^j, \quad j \geq 2, \quad \text{where}$$

$$z_i = \frac{1}{1 - \exp(t_i \cdot s)}, \quad t_i = \begin{cases} n + r, & i = 0, \\ r & i = 1, \end{cases}$$

and the coefficients $c(i, j)$ are calculated recursively, with $c(0, 1) = c(0, 2) = c(i, 1) = 1$, and

$$c(i, j) = (j - 1) \cdot c(i - 1, j - 1) + j \cdot c(i - 1, j), \quad j \geq 2.$$

See also `Murakami2_deriv2` in `DistCiornishArb.vb`.

The saddlepoint s is determined numerically using Newton iterations, with a starting value of $s = 0.1$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", mann_whitney_u_continuous(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

`dist_mann_whitney_u.`**`moments`**`(`k`)`

Returns the first j moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Mann-Whitney U distribution (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mann_whitney_u_continuous(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

`dist_mann_whitney_u.`**`cumulants`**`(`k`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Mann-Whitney U distribution. The cumulants of U are given by (Robillard1972):

$$\begin{aligned}\kappa_{2j} &= \frac{B_{2j}}{2j}\left[\sum_{s=N_1+1}^{N_1+N_2} s^{2j} - \sum_{s=1}^{N_2} s^{2j}\right] \\ &= \frac{B_{2j}}{2j(2j+1)}\left[B_{2j+1}(N_2+N_1+1) + B_{2j+1} - B_{2j+1}(N_1+1) - B_{2j+1}(N_2+1)\right]\end{aligned}$$

and $\kappa_{2j+1} = 0$, $j \geq 1$, and B_{2j} and $B_{2j}(x)$ are the Bernoulli numbers and polynomials, respectively, of degree $2j$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mann_whitney_u_continuous(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Approximations

`ctx.`**`mannwhitney_ft`**`(`x, n, *results='cdf'*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the pdf, cdf and sf from the characteristic function (see *pmf_from_cf_lattice()* and *cdf_from_cf_lattice()*).

`ctx.`**`mannwhitney_ecf`**`(`x, f, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

`ctx.`**`mannwhitney_ecf_inv`**`(`q, f, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**mannwhitney_spa**(*x*, *m*, *n*, *results='c'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

The saddlepoint s is determined numerically using Newton iterations, with a starting value of $s = 0.1$.

ctx.**mannwhitney_spa_inv**(*q*, *m*, *n*, *results='qtf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.14.4 Noncentral Mann-Whitney U distribution, Lehmann alternatives

class ctx.**dist_mann_whitney_u_lehmann**(m, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Mann-Whitney's U is a discrete (lattice) probability distribution with sample sizes $m \geq 1$ and $n \geq 1$ and the support interval $(0, nm))$. See also Wikipedia [1185], R (Statistical System) [528], Murakami and Kamakura [420], Robillard [482], vandeWiel [789] and Zimmermann [1502].

Let $x1, \ldots, x_m$ and $y1, \ldots, y_n$ be two sets of measurements, which we denote by X and Y. The test criterion U of the Mann-Whitney test is then

$$U = \sum_{i=1}^{m} \sum_{j=1}^{n} \text{sgn}(x_i - y_j)$$

dist_mann_whitney_u_lehmann.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives.

$$\text{pmf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", mann_whitney_u_continuous(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_mann_whitney_u_lehmann.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives.

$$\text{cdf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mann_whitney_u_continuous(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_mann_whitney_u_lehmann.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mann_whitney_u_continuous(mu, sigma).pmf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_mann_whitney_u_lehmann.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives. There is no closed form for the qtf: It is computed with Newton iterations where the starting values are from Boost.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mann_whitney_u_continuous(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145614561356E+00
```

dist_mann_whitney_u_lehmann.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives:

$$\text{isf}_X(q) = \text{qtf}_X(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mann_whitney_u_continuous(mu, sigma).isf(q))
6.35635234625645256156156145614561356E+00
```

dist_mann_whitney_u_normal.**g_x**(t)

Returns $G_X(t)$, the probability generating function of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives:

$$G_X(t) = ??$$

dist_mann_whitney_u_lehmann.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mann_whitney_u_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145614561356E+00
```

dist_mann_whitney_u_lehmann.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives:

$$M_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", mann_whitney_u_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mann_whitney_u_lehmann.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives:

$$K_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", mann_whitney_u_continuous(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

dist_mann_whitney_u_lehmann.**moments**(*k*)

Returns the first j moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mann_whitney_u_continuous(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

dist_mann_whitney_u_lehmann.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Mann-Whitney U distribution under Lehmann alternatives.

kappa_{j} = ??

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mann_whitney_u_continuous(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

4.14.5 Noncentral Mann-Whitney U distribution, Milton alternatives

class ctx.**dist_mann_whitney_u_milton**(m, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Mann-Whitney's U is a discrete (lattice) probability distribution with sample sizes $m \geq 1$ and $n \geq 1$ and the support interval $(0, nm))$. See also Wikipedia [1185], R (Statistical System) [528], Murakami and Kamakura [420], Robillard [482], vandeWiel [789] and Zimmermann [1502].

Let $x1, \ldots, x_m$ and $y1, \ldots, y_n$ be two sets of measurements, which we denote by X and Y. The test criterion U of the Mann-Whitney test is then

$$U = \sum_{i=1}^{m} \sum_{j=1}^{n} \operatorname{sgn}(x_i - y_j)$$

dist_mann_whitney_u_milton.**pmf**(x)

Returns $\operatorname{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Mann-Whitney U distribution under Milton alternatives.

$$\operatorname{pmf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", mann_whitney_u_continuous(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

dist_mann_whitney_u_milton.**cdf**(x)

Returns $\operatorname{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Mann-Whitney U distribution under Milton alternatives.

$$\operatorname{cdf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", mann_whitney_u_continuous(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

dist_mann_whitney_u_milton.**sf**(x)

Returns $\operatorname{sf}_X(x)$, the survival function (sf) of a random variable X, following a Mann-Whitney U distribution under Milton alternatives:

$$\operatorname{sf}_X(x) = 1 - \operatorname{cdf}_X(x)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", mann_whitney_u_continuous(mu, sigma).pmf(x))
sf: 6.35635234625645256156156145613 56E-20
```

dist_mann_whitney_u_milton.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Mann-Whitney U distribution under Milton alternatives. There is no closed form for the qtf: It is computed with Newton iterations where the starting values are from Boost.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", mann_whitney_u_continuous(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_mann_whitney_u_milton.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Mann-Whitney U distribution under Milton alternatives:

$$\text{isf}_X(q) = \text{qtf}_X(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", mann_whitney_u_continuous(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_mann_whitney_u_milton.**g_x**(t)

Returns $G_X(t)$, the probability generating function of a random variable X, following a Mann-Whitney U distribution under Milton alternatives:

$$G_X(t) = ??$$

dist_mann_whitney_u_milton.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Mann-Whitney U distribution under Milton alternatives:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", mann_whitney_u_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_mann_whitney_u_milton.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Mann-Whitney U distribution under Milton alternatives:

$$M_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", mann_whitney_u_continuous(mu, sigma).c_x(t))
6.356352346256452561561561456135 6E+00
```

dist_mann_whitney_u_milton.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Mann-Whitney U distribution under Milton alternatives:

$$K_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", mann_whitney_u_continuous(mu, sigma).k_x(t, k))
6.356352346256452561561561456135 6E+00
```

dist_mann_whitney_u_milton.**moments**(*k*)

Returns the first j moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Mann-Whitney U distribution under Milton alternatives (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mann_whitney_u_continuous(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_mann_whitney_u_milton.**cumulants**(*k*)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Mann-Whitney U distribution under Milton alternatives.

kappa_{j} = ??

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", mann_whitney_u_continuous(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

4.14.6 Kendall's tau distribution, continuous data

class ctx.**dist_kendall_tau**(N)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of $K_N = \frac{1}{4}N(N-1)(1-\tau)$ is a discrete (lattice) probability distribution with sample size $N \geq 2$ and the support interval $(0, N(N-1)/2)$. See also Wikipedia [1182], Noether [424], vandeWiel [789], Robillard [482].

Let $(X_1, Y_1), ..., (X_N, Y_N)$ be a sample of N pairs of observations. The Kendall rank correlation coefficient τ is defined as

$$\tau = 1 - \frac{2K_N}{N(N-1)/2}$$

where K_N is the number of inversions: the number of pairs $\{(X_i, Y_i), (X_j, Y_j)\}$ such that $X_i < X_j$ and $Y_i > Y_j$ for $i < j$, where $i = 1, \ldots n-1$ and $j = 2, \ldots, n$. K_N can assume values between 0 and $N(N-1)/2$, whereas τ can assume values between -1 and $+1$.

dist_kendall_tau.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following the distribution of Kendall's K_N.

$$\text{pmf}_X(x) = \sum_{j=x}^{N(N-1)/2} (-1)^{x+j} \binom{j}{x} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The factorial moments are calculated from the cumulants (see *factorial_moments_from_cumulants()*).

The pmf can also be calculated from the characteristic function $C_X(t)$:

$$\text{pmf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re\left(e^{-itx} C_X(t)\right) \mathrm{d}t,$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", kendall_tau_continuous(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

dist_kendall_tau.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following the distribution of Kendall's K_N:

$$\text{cdf}_X(x) = \sum_{j=x}^{N(N-1)/2} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The cdf can also be calculated from the characteristic function $C_X(t)$:

$$\text{cdf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re\left(C_X(t) \sum_{z=0}^{x} e^{-itz}\right) \mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", kendall_tau_continuous(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

`dist_kendall_tau.`**`sf`**(x)

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following the distribution of Kendall's K_N:

$$\mathrm{sf}_X(x) = 1 - \sum_{j=x}^{N(N-1)/2} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The cdf can also be calculated from the characteristic function $C_X(t)$:

$$\mathrm{sf}(x) = 1 - \mathrm{cdf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re \left(C_X(t) \sum_{z=x+1}^{N(N-1)/2} e^{-itz} \right) \mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", kendall_tau_continuous(mu, sigma).pmf(x))
sf: 6.35635234625645256156156145613 56E-20
```

`dist_kendall_tau.`**`qtf`**(q)

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following the distribution of Kendall's K_N. There is no closed form for the qtf: It is computed using the Brent algorithm with starting values from a Cornish-Fisher or Jensen approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", kendall_tau_continuous(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

`dist_kendall_tau.`**`isf`**(q)

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following the distribution of Kendall's K_N. There is no closed form for the isf: It is computed using the Brent algorithm with starting values from a Cornish-Fisher or Jensen approximation

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", kendall_tau_continuous(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_kendall_tau.**g_x**(*t*)

Returns $G_X(t)$, the probability generating function of a random variable X, following the distribution of Kendall's K_N:

$$G_X(t) = \frac{1}{N!} \prod_{k=1}^{N} \frac{(t^k - 1)}{(t - 1)}.$$

See also: v.d.Wiel, p. 16, equ. 2.1

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", kendall_tau_continuous(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

dist_kendall_tau.**c_x**(*t*)

Returns $C_X(t)$, the characteristic function of a random variable X, following the distribution of Kendall's K_N:

$$C_X(t) = \frac{1}{N!} \prod_{k=1}^{N} \frac{\exp(it \cdot k) - 1}{\exp(it) - 1}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", kendall_tau_continuous(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

dist_kendall_tau.**m_x**(*t*)

Returns $M_X(t)$, the moment generating function of a random variable X, following the distribution of Kendall's K_N:

$$M_X(t) = \frac{1}{N!} \prod_{k=1}^{N} \frac{\exp(t \cdot k) - 1}{\exp(t) - 1}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", kendall_tau_continuous(mu, sigma).c_x(t))
6.356352346256452561561561456 1356E+00
```

dist_kendall_tau.**k_x**(*t*, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following the distribution of Kendall's K_N:

$$K_X(t) = -\log(N!) + \sum_{k=1}^{N} \log\left(\frac{\exp(t \cdot k) - 1}{\exp(t) - 1}\right).$$

$K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a Kendall tau distribution, are defined as

$$K_X(t) = \sum_{j=2}^{k} \log\left(\frac{1}{N_j + 1} \frac{1 - \exp((N_j + 1)t)}{1 - \exp(t)}\right).$$

$$K_X^{(1)}(s) = \sum_{r=1}^{m} \left(\frac{n+r}{1-\exp((n+r)s)} - \frac{n(\exp(r\cdot s)+n+r)}{1-\exp(r\cdot s)} \right),$$

$$K_X^{(j)}(s) = (-1)^j \sum_{r=1}^{m} \sum_{i=0}^{1} (-1)^i \cdot t_i^j \cdot \sum_{k=1}^{j} c(j-2,k) \cdot z_i^j, \quad j \geq 2, \quad \text{where}$$

$$z_i = \frac{1}{1-\exp(t_i \cdot s)}, \quad t_i = \begin{cases} n+r, & i=0, \\ r & i=1, \end{cases}$$

and the coefficients $c(i,j)$ are calculated recursively, with $c(0,1) = c(0,2) = c(i,1) = 1$, and

$$c(i,j) = (j-1)\cdot c(i-1,j-1) + j \cdot c(i-1,j), \quad j \geq 2.$$

The saddlepoint s is determined numerically using Newton iterations, with a starting value of $s = 0.1$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", kendall_tau_continuous(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

`dist_kendall_tau.`**`moments`**`(`*`k`*`)`

Returns the first j moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following the distribution of Kendall's K_N (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", kendall_tau_continuous(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

`dist_kendall_tau.`**`cumulants`**`(`*`k`*`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following the distribution of Kendall's K_N. The cumulants of T_N are given by

$$\kappa_{2j}(T_N) = \frac{B_{2j}}{2j} \sum_{s=1}^{N} s^{2j} = \frac{B_{2j}}{2j} \left[\frac{B_{2j+1}(N+1) - B_{2j+1}}{2j+1} - N \right], \quad \text{and}$$

$$\kappa_{2j+1}(W_N) = 0, \quad \text{for } j \geq 1.$$

In particular, $\kappa_1(T_N) = N(N-1)/4$, and $\kappa_2(T_N) = N(N-1)(2N+5)/72$, and B_{2j} and $B_{2j}(x)$ are the Bernoulli numbers and polynomials, respectively, of degree $2j$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", kendall_tau_continuous(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Approximations

`ctx.`**`kendall_ft`**`(`*`x`*`, `*`n`*`, `*`results='cdf'`*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the pdf, cdf and sf from the characteristic function (see *pmf_from_cf_lattice()* and *cdf_from_cf_lattice()*).

ctx.**kendall_tau_ecf**(*x*, *f*, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Edgeworth approximation to the pdf.

ctx.**kendall_tau_ecf_inv**(*q*, *f*, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**kendall_tau_spa**(*x*, *n*, *results='c'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the Lugannini-Rice saddlepoint approximation of the pdf, cdf and sf.

The saddlepoint s is determined numerically using Newton iterations, with a starting value of $s = 0.1$.

ctx.**kendall_tau_spa_inv**(*x*, *n*, *results='qtf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.14.7 Jonckheere-Terpsta T distribution, continuous data

class ctx.**dist_jterpsta_s**(k, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Jonckheere-Terpsta's T is a discrete (lattice) probability distribution with k samples of size $n_1 \geq 1, \ldots, n_k \geq 1$ and the support interval $[0, M]$). See also Wikipedia [1181], Murakami and Kamakura [420], vandeWiel [789], Robillard [482]. and Skillings [501].

Consider k independent groups X_i of sizes $n_i, i = 1 \ldots k$, and define $N_i = \sum_{j=1}^{i-1} n_j$ and $M = \sum_{i=2}^{k} n_i N_i$. The Jonckheere-Terpsta statistic is defined as

$$J_T = \sum_{i<j}^{c} U_{ij} = \sum_{i=1}^{c-1} \sum_{j=i+1}^{c} U_{ij}$$

where U_{ij} is Mann-Whitney's U calculated for groups X_i and X_j.

Let S_i denote the combined samples $X_1, ..., X_i$, and let T_i be the Mann-Whitney T statistic calculated for groups S_{i-1} and X_i, $i = 2...k$. Then $T = \sum_{i=2}^{k} T_i$ is related to J_T by $J_T = 2T - M$.

dist_jterpsta_s.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution.

$$\text{pmf}_X(x) = \sum_{j=x}^{M} (-1)^{x+j} \binom{j}{x} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The factorial moments are calculated from the cumulants (see *factorial_moments_from_cumulants()*).

The pmf can also be calculated from the characteristic function $C_X(t)$:

$$\text{pmf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re \left(e^{-itx} C_X(t) \right) \mathrm{d}t,$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", jonckheere_terpsta_s_continuous(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_jterpsta_s.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Jonckheere-Terpsta S distribution.

$$\text{cdf}_X(x) = \sum_{j=x}^{M} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The cdf can also be calculated from the characteristic function $C_X(t)$:

$$\text{cdf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re \left(C_X(t) \sum_{z=0}^{x} e^{-itz} \right) \mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", jonckheere_terpsta_s_continuous(mu, sigma).pmf(x))
6.35635234625645256156156145613 56E-20
```

`dist_jterpsta_s.sf(x)`

Returns $\mathrm{sf}_X(x)$, the survival function (sf) of a random variable X, following a Jonckheere-Terpsta S distribution:

$$\mathrm{sf}_X(x) = 1 - \sum_{j=x}^{M} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!},$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

The cdf can also be calculated from the characteristic function $C_X(t)$:

$$\mathrm{sf}(x) = 1 - \mathrm{cdf}(x) = \frac{1}{\pi} \int_0^{\pi} \Re \left(C_X(t) \sum_{z=x+1}^{M} e^{-itz} \right) \mathrm{d}t.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", jonckheere_terpsta_s_continuous(mu, sigma).pmf(x))
sf: 6.35635234625645256156156145613 56E-20
```

`dist_jterpsta_s.qtf(q)`

Returns $\mathrm{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Jonckheere-Terpsta S distribution. There is no closed form for the qtf: It is computed using the Brent algorithm with starting values from a Cornish-Fisher or Jensen approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", jonckheere_terpsta_s_continuous(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

`dist_jterpsta_s.isf(q)`

Returns $\mathrm{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Jonckheere-Terpsta S distribution. There is no closed form for the isf: It is computed using the Brent algorithm with starting values from a Cornish-Fisher or Jensen approximation.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", jonckheere_terpsta_s_continuous(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

`dist_jterpsta_s.g_x(t)`

Returns $G_X(t)$, the characteristic function of a random variable X, following a Jonckheere-Terpsta S distribution:

$$G_X(t) = \prod_{i=2}^{k} \frac{1}{\binom{n_i+N_i}{n_i}} \frac{\prod_{l=N_i+1}^{n_i+N_i} (1-x^l)}{\prod_{l=1}^{n_i} (1-x^l)}.$$

See also: v.d.Wiel, p. 15, equ. 2.9

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", jonckheere_terpsta_s_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_jterpsta_s.c_x(t)`

Returns $C_X(t)$, the characteristic function of a random variable X, following a Jonckheere-Terpsta S distribution:

$$C_X(t) = \prod_{j=2}^{k} \prod_{r=1}^{n_j} \frac{r}{N_j + r} \frac{1 - \exp((N_j + r)it)}{1 - \exp(itr)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", jonckheere_terpsta_s_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_jterpsta_s.m_x(t)`

Returns $M_X(t)$, the moment generating function of a random variable X, following a Jonckheere-Terpsta S distribution:

$$M_X(t) = \prod_{j=2}^{k} \prod_{r=1}^{n_j} \frac{r}{N_j + r} \frac{1 - \exp((N_j + r)t)}{1 - \exp(tr)}.$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", jonckheere_terpsta_s_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

`dist_jterpsta_s.k_x(t, k=0)`

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Jonckheere-Terpsta S distribution:

$$K_X(t) = \sum_{j=2}^{k} \sum_{r=1}^{n_j} \log\left(\frac{r}{N_j + r} \frac{1 - \exp((N_j + r)t)}{1 - \exp(tr)} \right).$$

$K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a Jonckheere-Terpsta S distribution, are defined as

$$K_X(t) = \sum_{j=2}^{k} \sum_{r=1}^{n_j} \log\left(\frac{r}{N_j + r} \frac{1 - \exp((N_j + r)t)}{1 - \exp(tr)} \right).$$

$$K_X^{(1)}(s) = \sum_{r=1}^{m} \left(\frac{n + r}{1 - \exp((n + r)s)} - \frac{n(\exp(r \cdot s) + n + r)}{1 - \exp(r \cdot s)} \right),$$

$$K_X^{(j)}(s) = (-1)^j \sum_{r=1}^{m} \sum_{i=0}^{1} (-1)^i \cdot t_i^j \cdot \sum_{k=1}^{j} c(j-2, k) \cdot z_i^j, \quad j \geq 2, \quad \text{where}$$

$$z_i = \frac{1}{1-\exp(t_i \cdot s)}, \quad t_i = \begin{cases} n+r, & i=0, \\ r & i=1, \end{cases}$$

and the coefficients $c(i,j)$ are calculated recursively, with $c(0,1) = c(0,2) = c(i,1) = 1$, and

$$c(i,j) = (j-1) \cdot c(i-1, j-1) + j \cdot c(i-1, j), \quad j \geq 2.$$

The saddlepoint s is determined numerically using Newton iterations, with a starting value of $s = 0.1$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", jonckheere_terpsta_s_continuous(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

`dist_jterpsta_s.`**`moments`**`(`*k*`)`

Returns the first j moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Jonckheere-Terpsta S distribution (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", jonckheere_terpsta_s_continuous(mu, sigma).moments(k))
6.35635234625645256156156145613 56E+00
```

`dist_jterpsta_s.`**`cumulants`**`(`*k*`)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Jonckheere-Terpsta S distribution. The cumulants of J_N are given by (Robillard1972):

$$\begin{aligned} \kappa_{2j} &= \frac{B_{2j}}{2j}\left[\sum_{s=1}^{N} s^{2j} - \sum_{i=1}^{k}\sum_{s=1}^{n_i} s^{2j}\right] \\ &= \frac{B_{2j}}{2j(2j+1)}\left[B_{2j+1}(N+1) + (k-1)B_{2j+1} - \sum_{i=1}^{k} B_{2j+1}(n_i+1)\right] \end{aligned}$$

and $\kappa_{2j+1} = 0$, $j \geq 1$, and B_{2j} and $B_{2j}(x)$ are the Bernoulli numbers and polynomials, respectively, of degree $2j$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", jonckheere_terpsta_s_continuous(mu, sigma).cumulants(k))
6.35635234625645256156156145613 56E+00
```

Approximations

`ctx.`**`jterpsta_ft`**`(`*x*, *n*, *results='cdf'*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the pdf, cdf and sf from the characteristic function (see *pmf_from_cf_lattice()* and *cdf_from_cf_lattice()*).

`ctx.`**`jterpsta_ecf`**`(`*x*, *f*, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

ctx.**jterpsta_ecf_inv**(*q*, *f*, *rho*, *omega*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the Cornish-Fisher approximation to the qtf and isf.

ctx.**jterpsta_spa**(*x*, *n*, *results='c'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

The saddlepoint s is determined numerically using Newton iterations, with a starting value of $s = 0.1$.

ctx.**jterpsta_spa_inv**(*x*, *n*, *results='qtf'*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.14.8 Generalized Page L distribution, continuous data

class ctx.**dist_page_l**(k, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Page's L is a discrete (lattice) probability distribution with k samples of size $n_1 \geq 1, \ldots, n_k \geq 1$ and the support interval $(0, nm)$). See also Wikipedia [1181], Noether [424], vandeWiel [789] .

Consider k independent groups X_i of sizes $n_i, i = 1 \ldots k$. The Page L statistic is defined as

$$L = ??$$

dist_page_l.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a generalized Page L distribution.

$$\text{pmf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", page_l_continuous(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_page_l.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a generalized Page L distribution.

$$\text{cdf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", page_l_continuous(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_page_l.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a generalized Page L distribution:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", page_l_continuous(mu, sigma).pmf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_page_l.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a generalized Page L distribution. There is no closed form for the qtf: It is computed with Newton iterations where the starting values are from Boost.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", page_l_continuous(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_page_l.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a generalized Page L distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", page_l_continuous(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_page_l.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a generalized Page L distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", page_l_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_page_l.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a generalized Page L distribution:

$$M_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", page_l_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_page_l.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a generalized Page L distribution:

$$K_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", page_l_continuous(mu, sigma).k_x(t, k))
6.35635234625645256156156145613 56E+00
```

`dist_page_l.moments(k)`

Returns the first j moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a generalized Page L distribution (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", page_l_continuous(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

`dist_page_l.cumulants(k)`

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a generalized Page L distribution. The cumulants of J_N are given by :

$$\kappa_{2j} = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", page_l_continuous(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

Approximations

`ctx.page_ecf(x, f, rho, omega)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth approximation to the pdf, cdf and sf.

For the Page L distribution, the cumulants are calculated from the pmf

`ctx.page_ecf_inv(q, f, rho, omega)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation to the qtf and isf.

For the Page L distribution, the cumulants are calculated from the pmf

`ctx.page_spa(x, n, results='c')`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf, cdf and sf.

$K_X(t)$, the cumulant generating function, and its j^{th} derivatives, $K_X^{(j)}(t), j = 1 \ldots k$, of a random variable X, following a Jonckheere-Terpsta S distribution, are calculated from the pgf. The saddlepoint s is determined numerically using Newton iterations, with a starting value of $s = 0.1$.

`ctx.page_spa_inv(x, n, results='qtf')`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the inverse Jensen saddlepoint approximation of the qtf and isf.

4.14.9 Noncentral generalized Page L distribution, Milton alternatives

class ctx.**dist_page_l_nc_milton**(k, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of Page's L is a discrete (lattice) probability distribution with k samples of size $n_1 \geq 1, \ldots, n_k \geq 1$ and the support interval $(0, nm))$. See also Wikipedia [1181], Noether [424], vandeWiel [789] .

Consider k independent groups X_i of sizes $n_i, i = 1 \ldots k$. The Page L statistic is defined as

$$L = ??$$

dist_page_l_nc_milton.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Page L distribution under Milton alternatives.

$$\text{pmf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", page_l_nc_milton_continuous(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_page_l_nc_milton.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Page L distribution under Milton alternatives.

$$\text{cdf}_X(x) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", page_l_nc_milton_continuous(mu, sigma).pmf(x))
6.3563523462564525615615615614561356E-20
```

dist_page_l_nc_milton.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a Page L distribution under Milton alternatives:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", page_l_nc_milton_continuous(mu, sigma).pmf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_page_l_nc_milton.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Page L distribution under Milton alternatives. There is no closed form for the qtf: It is computed with Newton iterations where the starting values are from Boost.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", page_l_nc_milton_continuous(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_page_l_nc_milton.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Page L distribution under Milton alternatives:

$$\text{isf}_X(q) = \text{qtf}_X(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", page_l_nc_milton_continuous(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_page_l_nc_milton.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Page L distribution under Milton alternatives:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", page_l_nc_milton_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_page_l_nc_milton.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Page L distribution under Milton alternatives:

$$M_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", page_l_nc_milton_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_page_l_nc_milton.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Page L distribution under Milton alternatives:

$$K_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", page_l_nc_milton_continuous(mu, sigma).k_x(t, k))
6.356352346256452561561561561456 1356E+00
```

dist_page_l_nc_milton.**moments**(k)

Returns the first j moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Page L distribution under Milton alternatives (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", page_l_nc_milton_continuous(mu, sigma).moments(k))
6.356352346256452561561561561456 1356E+00
```

dist_page_l_nc_milton.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Page L distribution under Milton alternatives. The cumulants of J_N are given by :

$$\kappa_{2j} = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", page_l_nc_milton_continuous(mu, sigma).cumulants(k))
6.356352346256452561561561561456 1356E+00
```

4.15 Discrete (non-lattice) distributions related to rank tests

4.15.1 Cochran-Friedman-Quade distribution

class ctx.**dist_friedman**(k, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of the Cochran-Friedman-Quade tests is a discrete (non-lattice) probability distribution with k samples of size $n_1 \geq 1, \ldots, n_k \geq 1$ and the support interval $(0, nm))$. See also Wikipedia [1181], Noether [424], vandeWiel [789] .

Consider k independent groups X_i of sizes $n_i, i = 1 \ldots k$. The Page L statistic is defined as

$$L = ??$$

dist_friedman.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Page L distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = ??$$

where the sum is over all x with positive $p(\cdot)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", friedman_continuous(mu, sigma).pmf(x))
6.356352346256452561561561561456135 6E-20
```

dist_friedman.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Page L distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = ??$$

where the sum is over all x with positive $p(\cdot)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", friedman_continuous(mu, sigma).pmf(x))
6.356352346256452561561561561456135 6E-20
```

dist_friedman.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a Page L distribution:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", friedman_continuous(mu, sigma).pmf(x))
sf: 6.356352346256452561561561561456135 6E-20
```

dist_friedman.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Page L distribution. There is no closed form for the qtf: It is computed with Newton iterations where the starting values are from Boost.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", friedman_continuous(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_friedman.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Page L distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", friedman_continuous(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_friedman.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Page L distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", friedman_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_friedman.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Page L distribution:

$$M_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", friedman_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_friedman.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Page L distribution:

$$K_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", friedman_continuous(mu, sigma).k_x(t, k))
6.356352346256452561561561456135 6E+00
```

dist_friedman.**moments**(k)

Returns the first j moments, $\mu_j, j = 1 \ldots k$, of a random variable X, following a Page L distribution (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", friedman_continuous(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_friedman.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \ldots k$, of a random variable X, following a Page L distribution. The cumulants of J_N are given by :

$$\kappa_{2j} = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", friedman_continuous(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

4.15.2 Kruskal-Wallis distribution

class ctx.**dist_kruskal_wallis**(k, n)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

The distribution of the Kruskal-Wallis test is a discrete (non-lattice) probability distribution with k samples of size $n_1 \geq 1, \ldots, n_k \geq 1$ and the support interval $(0, nm))$. See also Wikipedia [1181], Noether [424], vandeWiel [789] .

Consider k independent groups X_i of sizes $n_i, i = 1 \ldots k$. The Kruskal_Wallis statistic is defined as

$$L = ??$$

dist_kruskal_wallis.**pmf**(x)

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Kruskal-Wallis distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = ??$$

where the sum is over all x with positive $p(\cdot)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("pmf: ", kruskal_wallis_continuous(mu, sigma).pmf(x))
6.3563523462564525615615614561356E-20
```

dist_kruskal_wallis.**cdf**(x)

Returns $\text{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X, following a Kruskal-Wallis distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = ??$$

where the sum is over all x with positive $p(\cdot)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print ("cdf: ", kruskal_wallis_continuous(mu, sigma).pmf(x))
6.3563523462564525615615614561356E-20
```

dist_kruskal_wallis.**sf**(x)

Returns $\text{sf}_X(x)$, the survival function (sf) of a random variable X, following a Kruskal-Wallis distribution:

$$\text{sf}_X(x) = 1 - \text{cdf}_X(x)$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; x = 3;
>>> print (" sf: ", kruskal_wallis_continuous(mu, sigma).pmf(x))
sf: 6.3563523462564525615615615614561356E-20
```

dist_kruskal_wallis.**qtf**(q)

Returns $\text{qtf}_X(x)$, the quantile function (qtf) of a random variable X, following a Kruskal-Wallis distribution. There is no closed form for the qtf: It is computed with Newton iterations where the starting values are from Boost.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("qtf: ", kruskal_wallis_continuous(mu, sigma).qtf(q))
qtf: 6.35635234625645256156156145613 56E+00
```

dist_kruskal_wallis.**isf**(q)

Returns $\text{isf}_X(q)$, the inverse survival function (isf) of a random variable X, following a Kruskal-Wallis distribution:

$$\text{isf}_X(q) = \text{qtf}_X(1-q).$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; q = 0.3;
>>> print ("isf: ", kruskal_wallis_continuous(mu, sigma).isf(q))
6.35635234625645256156156145613 56E+00
```

dist_kruskal_wallis.**c_x**(t)

Returns $C_X(t)$, the characteristic function of a random variable X, following a Kruskal-Wallis distribution:

$$C_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", kruskal_wallis_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_kruskal_wallis.**m_x**(t)

Returns $M_X(t)$, the moment generating function of a random variable X, following a Kruskal-Wallis distribution:

$$M_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("m_x: ", kruskal_wallis_continuous(mu, sigma).c_x(t))
6.35635234625645256156156145613 56E+00
```

dist_kruskal_wallis.**k_x**(t, *k=0*)

Returns $K_X(t)$, the cumulant generating function of a random variable X, following a Kruskal-Wallis distribution:

$$K_X(t) = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3; k = 6;
>>> print ("c_x: ", kruskal_wallis_continuous(mu, sigma).k_x(t, k))
6.356352346256452561561561456135 6E+00
```

dist_kruskal_wallis.**moments**(k)

Returns the first j moments, $\mu_j, j = 1 \dots k$, of a random variable X, following a Kruskal-Wallis distribution (Wikipedia). The moments are calculated from the cumulants.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", kruskal_wallis_continuous(mu, sigma).moments(k))
6.356352346256452561561561456135 6E+00
```

dist_kruskal_wallis.**cumulants**(k)

Returns the first j cumulants, $\kappa_j, j = 1 \dots k$, of a random variable X, following a Kruskal-Wallis distribution. The cumulants of J_N are given by :

$$\kappa_{2j} = ??$$

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; k = 6;
>>> print ("saddlepoint: ", kruskal_wallis_continuous(mu, sigma).cumulants(k))
6.356352346256452561561561456135 6E+00
```

CHAPTER

FIVE

NUMERICAL CALCULUS

The source code for the C# tests in Visual Studio is divided by in 2 parts:

Part 1 contains procedures from DAMath, Boost Math, Boost Odeint and Eigen. The source code for the C# tests in Visual Studio can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B05a_NumericalCalculus.cs.

Part 2 is devoted to numerical integration in ball arithmetic. The source code for the C# tests in Visual Studio can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B05b_FlintNumericalCalculus.cs.

5.1 Introduction

5.1.1 Delegates, lambda expressions, callback functions: an example

Returns a local minimum of the function f in the interval (a, b), implementing Brent's algorithm.

See also: Wikipedia [1086], Zhang [1500], Brent [36]

x is the approximate minimum abscissa, fx=f(x). eps and t define a tolerance tol =eps*|x|+t. f is never evaluated for 2 points closer together than tol. eps shall not be less than 2*eps_x, preferably not smaller than sqrt(eps_x). ic is the iteration count, -1 if a=b, 0 if max. count = 5000 exceeded. The algorithm combines golden section search and successive parabolic interpolation using only function (not derivative) evaluations.

Parameters:

f
: the function for which a local minimum is to be found.

a
: left border of the interval (a, b).

b
: right border of the interval (a, b).

eps
: defines a tolerance tol = $\epsilon|x|+$ t.

tol
: defines a tolerance tol = $\epsilon|x|+$ t.

Results:

x
: the approximate minimum abscissae.

fx
: fx = f(x).

ic
: the iteration count, -1 if a=b, 0 if max. count = 5000 exceeded.

5.1.2 Usage in Python

The routine in Python, using a local function definition. This form allows monitoring of the calls into the function (by un-commenting the `print` statement), but variables and functions local to the calling function cannot be accessed:

```
>>> from xlcalcnet import math53
>>> a = -10.0; b = 20.0; eps = 1E-8; tol = 1E-8
>>> f = lambda x: -ereal.Exp(-x * x)
>>> res = ereal.LocalMin(f, a, b, eps, tol)
>>> print('res: (x0, fx0, ic1):', res)
```

The same routine in Python, using a local function definition. This form allows monitoring of the calls into the function (by un-commenting the `print` statement). Variables and functions local to the calling function can be accessed:

```
a = -10.0; b = 20.0; eps = 1E-8; tol = 1E-8
f = lambda x: -ereal.Exp(-x * x)
print('ereal.LocalMin')
res3 = ereal.LocalMin(f, a, b, eps, tol)
print('res3: (x0, fx0, ic1):', res3)
print()
print()
```

The same routine in Python, using a lambda expression. Variables and functions local to the calling function can be accessed:

```
a = -10.0; b = 20.0; eps = 1E-8; tol = 1E-8
f = lambda x: -ereal.Exp(-x * x)
print('ereal.LocalMin')
res3 = ereal.LocalMin(f, a, b, eps, tol)
print('res3: (x0, fx0, ic1):', res3)
print()
print()
```

The same routine in Python, inserting the lambda expression into `ereal.LocalMin`. Variables and functions local to the calling function can be accessed from within the lambda expression:

```
a = -10.0; b = 20.0; eps = 1E-8; tol = 1E-8
f = lambda x: -ereal.Exp(-x * x)
print('ereal.LocalMin')
res3 = ereal.LocalMin(lambda x: -ereal.Exp(-x * x), a, b, eps, tol)
print('res3: (x0, fx0, ic1):', res3)
print()
print()
```

5.1.3 Usage in CSharp

The same routine in CSharp, using a callback function definition outside of the scope of the calling function. This form allows the callback function to be called from other functions as well, and it supports monitoring of the calls into the function (by un-commenting the `Console.WriteLine` statement), but variables, functions and subroutines local to the calling function cannot be accessed:

```
public static ereal XF1(ereal x)
{
    var y = -ereal.Exp(-x * x);
    //Console.WriteLine("x: {0},   y: {1}", x, y)
    return y;
}

public static void DemoLocalMin()
{
    var a = -10; var b = 20; var eps = 1E-10; var tol = 1E-10;
    var Res1 = ereal.LocalMin(XF1, a, b, eps, tol);
    Console.WriteLine("Res1:(x, fx, ic) {0}", Res1);
}
```

The same routine in CSharp, using a multi-line lambda expression to define the callback function. Note that in contrast to Python, in CSharp a lambda expression can contain not only expressions but also statements, including a `return` statement. This is functionally equivalent to defining a local function without the need to create a new class. Variables, functions and subroutines local to the calling function can be accessed from within the lambda expression. The function which is defined by the lambda expression can be called locally. In this form the lambda expression allows monitoring of the calls into the callback function (by un-commenting the `Console.WriteLine` statement):

```
public static void DemoLocalMin2()
{
    var a = -10; var b = 20; var eps = 1E-10; var tol = 1E-10;
    var F2 = ereal (ereal x) =>
    {
        var y = -ereal.Exp(-x * x);
        //Console.WriteLine("x: {0},   y: {1}", x, y)
        return y;
    };
    var Res1 = ereal.LocalMin(F2, a, b, eps, tol);
    Console.WriteLine("Res1:(x, fx, ic) {0}", Res1);
}
```

The same routine in CSharp, using a single-line lambda expression to define the callback function. Note that there is no `return` statement, and the return type of the function is not specified but inferred from the function expression. Variables, functions and subroutines local to the calling function can be accessed from within the lambda expression. The function which is defined by the lambda expression can be called locally. Monitoring of calls into the callback function is not supported:

```
public static void DemoLocalMin3()
{
    var a = -10; var b = 20; var eps = 1E-10; var tol = 1E-10;
    var F3 = (ereal x) => -ereal.Exp(-x * x);
    var Res1 = ereal.LocalMin(F3, a, b, eps, tol);
    Console.WriteLine("Res1:(x, fx, ic) {0}", Res1);
}
```

The same routine in CSharp, using an anonymous lambda expression, which is like a single line lambda expression inserted directly into `ereal.LocalMin`. Variables and functions local to the calling function can be accessed from within the lambda expression. Since an anonymous lambda expression does not have a name, it cannot be called locally. Monitoring of calls into the callback function is not supported:

```
public static void DemoLocalMin4()
{
    var a = -10; var b = 20; var eps = 1E-10; var tol = 1E-10;
    var Res1 = ereal.LocalMin((ereal x) => -ereal.Exp(-x * x), a, b, eps, tol);
    Console.WriteLine("Res1:(x, fx, ic) {0}", Res1);
}
```

All of the above CSharp routines produce the following output:

```
Res1:(x, fx, ic) (-1.000175554142801716OE-10, -1.0000000000000000000, 14)
```

5.2 DAMath: Numerical Rootfinding and Minimization

5.2.1 Quadratic Equation, Real Coefficients

`ctx.squadx`(a, b, c)

where `ctx` is `math53`. Solves the quadratic equation $ax^2 + bx + c = 0$.

Parameters

- **a** – real coeffient in the quadratic equation.
- **b** – real coeffient in the quadratic equation.
- **c** – real coeffient in the quadratic equation.

Returns

a tuple (*z1, z2*), where *z1* is the first complex root.and *z2* is the second complex root.

See also: Wikipedia [1097], MathWorld [811].

```
>>> from xlcalcnet import math53
>>> a = -13;  b = 4;  c = 5
>>> ic, x1, y1, x2, y2 = ctx.squadx(a,b,c)
>>> print("ic: ", ic,"x1: ", x1,"y1: ", y1,"x2: ", x2,"y2: ", y2)
```

```
Quadratic
Return Code:  2
x1:  -0.4851249125321596 y1:  0.0
x2:  0.7928172202244673 y2:  0.0
```

5.2.2 Cubic Equation, Real Coefficients

`ctx.cubsolve`(a, b, c, d)

where `ctx` is `math53`. Solves the cubic equation $ax^3 + bx^2 + cx + d = 0$.

Parameters

- **a** – real coeffient in the cubic equation.
- **b** – real coeffient in the cubic equation.
- **c** – real coeffient in the cubic equation.
- **d** – real coeffient in the cubic equation.

Returns

a tuple (*x, z1, z2*), where *x* is the real root, *z1* is the first complex root and *z2* is the second complex root.

See also: Wikipedia [1088], MathWorld [798].

```
>>> from xlcalcnet import math53
>>> a = -13; b = 4; c = 5; d = 1
>>> x0, x1, y1, x2, y2 = ctx.cubsolve(a,b,c,d)
>>> print("x0: ", x0,"x1: ", x1,"y1: ", y1,"x2: ", x2,"y2: ", y2)
```

```
Cubic
x:  0.8593905490844492
x1:  -0.27584912069607076 y1:  -0.11582803026356575
x2:  -0.27584912069607076 y2:  0.11582803026356575
```

5.2.3 Brent's algorithm for finding a local minimum

`ctx.localmin`(*f*, *a*, *b*, *eps*, *t*)

where `ctx` is `math53`. Returns a local minimum of the function *f* in the interval (a, b), implementing Brent's algorithm.

Parameters

- **f** – the function for which a local minimum is to be found.
- **a** – the left border of the search interval.
- **b** – the right border of the search interval.
- **eps** – defines a tolerance eps*|x| + tol; eps should not be less than 2*eps_x, preferably not smaller than sqrt(eps_x).
- **tol** – defines a tolerance eps*|x| + tol.

Returns

a tuple (*localmin, fx, ic*), where *localmin* is an approximation to a local minimum; *fx* = f(localmin); and *ic* is the iteration count, -1 if a=b, 0 if max.=5000 exceeded.

See also: Wikipedia [1086], Zhang [1500], Brent [36]

x is the approximate minimum abscissa, fx=f(x). eps and t define a tolerance tol =eps*|x|+t. f is never evaluated for 2 points closer together than tol. eps shall not be less than 2*eps_x, preferably not smaller than sqrt(eps_x). ic is the iteration count, -1 if a=b, 0 if max. count = 5000 exceeded. The algorithm combines golden section search and successive parabolic interpolation using only function (not derivative) evaluations.

Examples in Python

The routine in Python, using a local function definition. This form allows monitoring of the calls into the function (by un-commenting the `print` statement), but variables and functions local to the calling function cannot be accessed:

```
>>> from xlcalcnet import math53
>>> a = -10.0; b = 20.0; eps = 1E-8; tol = 1E-8
>>> f = lambda x: -ereal.Exp(-x * x)
>>> res = ereal.LocalMin(f, a, b, eps, tol)
>>> print('res: (x0, fx0, ic1):', res)
```

The same routine in Python, using a local function definition. This form allows monitoring of the calls into the function (by un-commenting the `print` statement). Variables and functions local to the calling function can be accessed:

```
a = -10.0; b = 20.0; eps = 1E-8; tol = 1E-8
f = lambda x: -ereal.Exp(-x * x)
print('ereal.LocalMin')
res3 = ereal.LocalMin(f, a, b, eps, tol)
print('res3: (x0, fx0, ic1):', res3)
print()
print()
```

The same routine in Python, using a lambda expression. Variables and functions local to the calling function can be accessed:

```
a = -10.0; b = 20.0; eps = 1E-8; tol = 1E-8
f = lambda x: -ereal.Exp(-x * x)
print('ereal.LocalMin')
res3 = ereal.LocalMin(f, a, b, eps, tol)
print('res3: (x0, fx0, ic1):', res3)
print()
print()
```

The same routine in Python, inserting the lambda expression into `ereal.LocalMin`. Variables and functions local to the calling function can be accessed from within the lambda expression:

```
a = -10.0; b = 20.0; eps = 1E-8; tol = 1E-8
f = lambda x: -ereal.Exp(-x * x)
print('ereal.LocalMin')
res3 = ereal.LocalMin(lambda x: -ereal.Exp(-x * x), a, b, eps, tol)
print('res3: (x0, fx0, ic1):', res3)
print()
print()
```

Examples in CSharp

The same routine in CSharp, using a callback function definition outside of the scope of the calling function. This form allows the callback function to be called from other functions as well, and it supports monitoring of the calls into the function (by un-commenting the `Console.WriteLine` statement), but variables, functions and subroutines local to the calling function cannot be accessed:

```
public static ereal XF1(ereal x)
{
    var y = -ereal.Exp(-x * x);
    //Console.WriteLine("x: {0},   y: {1}", x, y)
    return y;
}

public static void DemoLocalMin()
{
    var a = -10; var b = 20; var eps = 1E-10; var tol = 1E-10;
    var Res1 = ereal.LocalMin(XF1, a, b, eps, tol);
    Console.WriteLine("Res1:(x, fx, ic) {0}", Res1);
}
```

The same routine in CSharp, using a multi-line lambda expression to define the callback function. Note that in contrast to Python, in CSharp a lambda expression can contain not only expressions but also statements, including a `return` statement. This is functionally equivalent to defining a local function without the need to create a new class. Variables, functions and subroutines local to the calling function can be accessed from within the lambda expression. The function which is defined by the lambda expression can be called locally. In this form the lambda expression allows monitoring of the calls into the callback function (by un-commenting the `Console.WriteLine` statement):

```
public static void DemoLocalMin2()
{
    var a = -10; var b = 20; var eps = 1E-10; var tol = 1E-10;
    var F2 = ereal (ereal x) =>
    {
        var y = -ereal.Exp(-x * x);
        //Console.WriteLine("x: {0},   y: {1}", x, y)
        return y;
    };
    var Res1 = ereal.LocalMin(F2, a, b, eps, tol);
    Console.WriteLine("Res1:(x, fx, ic) {0}", Res1);
}
```

The same routine in CSharp, using a single-line lambda expression to define the callback function. Note that there is no `return` statement, and the return type of the function is not specified but inferred from the function expression. Variables, functions and subroutines local to the calling function can be accessed from within the lambda expression. The function which is defined by the lambda expression can be called locally. Monitoring of calls into the callback function is not supported:

```
public static void DemoLocalMin3()
{
    var a = -10; var b = 20; var eps = 1E-10; var tol = 1E-10;
    var F3 = (ereal x) => -ereal.Exp(-x * x);
    var Res1 = ereal.LocalMin(F3, a, b, eps, tol);
    Console.WriteLine("Res1:(x, fx, ic) {0}", Res1);
}
```

The same routine in CSharp, using an anonymous lambda expression, which is like a single line lambda expression inserted

directly into `ereal.LocalMin`. Variables and functions local to the calling function can be accessed from within the lambda expression. Since an anonymous lambda expression does not have a name, it cannot be called locally. Monitoring of calls into the callback function is not supported:

```
public static void DemoLocalMin4()
{
    var a = -10; var b = 20; var eps = 1E-10; var tol = 1E-10;
    var Res1 = ereal.LocalMin((ereal x) => -ereal.Exp(-x * x), a, b, eps, tol);
    Console.WriteLine("Res1:(x, fx, ic) {0}", Res1);
}
```

All of the above CSharp routines produce the following output:

```
Res1:(x, fx, ic) (-1.000175554142801716 0E-10, -1.000000000000000000 0, 14)
```

5.2.4 Modified Brent's algorithm for finding a local minimum

`ctx.mbrent`(*f*, *a*, *b*, *t*)

where `ctx` is `math53`. Returns a local minimum of the function *f* in the interval (a, b), implementing a modified (simplified) version of Brent's algorithm as in procedure localmin with fixed eps=0.5*sqrt(eps_x).

Parameters

- **f** – the function for which a local minimum is to be found.
- **a** – the left border of the search interval.
- **b** – the right border of the search interval.
- **tol** – defines a tolerance eps*|x| + tol.

Returns

a tuple (*localmin, fx, ic*), where *localmin* is an approximation to a local minimum; *fx* = f(localmin); and *ic* is the iteration count, -1 if a=b, 0 if max.=5000 exceeded.

See also: Wikipedia [1086], Zhang [1500], Brent [36], BoostMath [47].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> a = -10.0; b = 20.0; tol = 1E-8
>>> f = lambda x: -ereal.Exp(-x * x)
>>> res = ereal.MBrent(f, a, b, tol)
>>> print('res4: (x0, fx0, ic1):', res4)
```

5.2.5 Rootfinding: Brent/Dekker algorithm

`ctx.zbrent`(f, a, b, t)

where `ctx` is `math53`. Performs the Brent/Dekker algorithm with guaranteed convergence for finding a zero of a function,; assumes that f(a) and f(b) have different signs.

Parameters

- **f** – the function for which a zero is sought.
- **a** – the left border of the search interval.
- **b** – the right border of the search interval.
- **tol** – defines a tolerance 6 * eps_x * $|x|$ +2*tol.

Returns

a tuple (*root, ic, err*), where *root* is an approximation to a zero (root) of the function; *ic* is the iteration count, -1 if a=b, 0 if max.=5000 exceeded; and *err* is the error code (0: no error, -1: if f(a) and f(b) have the same sign, -2: max. iteration count exceeded).

Brent/Dekker algorithm with guaranteed convergence for finding a zero of a function: Return a zero x of the function f in the interval [a,b] to within a tolerance 6*eps_x*|x|+2*t, where t is a positive tolerance; assumes that f(a) and f(b) have different signs. ic is the iteration count; err is an error code (0: no error, -1: if f(a) and f(b) have the same sign, -2: max. iteration count exceeded). The algorithm is based on a combination of successive interpolations and bisection.

See also: Wikipedia [1086], Zhang [1500], Brent [37], BoostMath [47].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> a = -20.0; b = 20.0; tol = 1E-8
>>> f = lambda x: ereal.Exp(x) - 10
>>> res = ereal.ZBrent(f, a, b, tol)
>>> print('res5: (x0, ic1, err):', res5)
```

5.2.6 Rootfinding: Simplified Brent/Dekker algorithm

`ctx.zeroin`(f, a, b, t)

where `ctx` is `math53`. Performs a simplified version of the Brent/Dekker algorithm for finding a zero of a function; assumes that f(a) and f(b) have different signs.

Parameters

- **f** – the function for which a zero is sought.
- **a** – the left border of the search interval.
- **b** – the right border of the search interval.
- **tol** – defines a tolerance 6 * eps_x * $|x|$ +2*tol.

Returns

an approximation to a zero (root) of the function.

See also: Wikipedia [1086], Zhang [1500], Brent [37], BoostMath [47].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> a = -20.0; b = 20.0; tol = 1E-8
>>> f = lambda x: ereal.Exp(x) - 10
>>> x0 = ereal.ZeroIn(f, a, b, tol)
>>> print('x0: ', x0)
```

5.3 Boost/Math: Root Finding and Minimization Algorithms

5.3.1 Root Finding Without Derivatives (TOM 748 algorithm)

ctxboost.**BracketRoot**(*f*, *guess*, *factor*, *is_rising*, *get_digits*, *maxit*)

where ctx is ctxboost.

Parameters

- **f** – the function for which a root is determined.
- **guess** – an intial guess.
- **factor** – a multiplication factor used when searching for the bracket.
- **is_rising** – an indicator whether the function is rising.
- **get_digits** – the targeted number of accurate digits.
- **maxit** – the maximal number of iterations.

Returns

a tuple (*root, error, iter*), where *root* is an approximation to a root of the function *f*, using the TOM 748 algorithm for finding a zero; *error* is an estimate of the modulus of the error of the approximation; and *iter* is the actual number of iterations.

See also: BoostMath [38], Alefeld *et al.* [8].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> f = lambda x: x * x * x - 28
>>> guess = 2.33;  factor = 2.0; is_rising = True; get_digits = 64;  maxit = 20
>>> res = ereal.BracketRoot(f, guess, factor, is_rising, get_digits, maxit)
>>> print("res (x0, error, iter): ", res)
```

```
boost.bracket_root(f, guess, factor, is_rising, get_digits, maxit
result:  3.036588971875662
```

5.3.2 Root Finding With Derivatives: Newton-Raphson

`ctxboost.`**`NewtonRaphson`**(*f*, *df*, *guess*, *xmin*, *xmax*, *get_digits*, *maxit*)

where `ctx` is `ctxboost`.

Parameters

- **`f`** – the function for which a root is determined.
- **`df`** – the first derivative of the function *f*.
- **`guess`** – an intial guess.
- **`xmin`** – the left border of the search interval.
- **`xmax`** – the right border of the search interval.
- **`get_digits`** – the targeted number of accurate digits.
- **`maxit`** – the maximal number of iterations.

Returns

a tuple (*root, iter*), where *root* is an approximation to a root of the function *f* in the interval (*xmin, xmax*), using the Newton algorithm for finding a zero; and *iter* is the actual number of iterations.

See also: Wikipedia [1094], BoostMath [48], MathWorld [807].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> f = lambda x: x * x * x - 28
>>> df1 = lambda x: 3 * x * x
>>> guess = 2.33; xmin = 1.0; xmax = 4.0; get_digits = 64;  maxit = 20
>>> result3 = ereal.NewtonRaphson(f, df1, guess, xmin, xmax, get_digits, maxit)
>>> print("res2 (x0, iter): ", result3)
```

```
boost.newton_raphson(f, df, guess, xmin, xmax, get_digits, maxit)
result:  3.0365889718756627
```

5.3.3 Root Finding With Derivatives: Halley

ctxboost.**Halley**(*f*, *df*, *d2f*, *guess*, *xmin*, *xmax*, *get_digits*, *maxit*)

where ctx is ctxboost.

Parameters

- **f** – the function for which a root is determined.
- **df** – the first derivative of the function *f*.
- **df2** – the second derivative of the function *f*.
- **guess** – an intial guess.
- **xmin** – the left border of the search interval.
- **xmax** – the right border of the search interval.
- **get_digits** – the targeted number of accurate digits.
- **maxit** – the maximal number of iterations.

Returns

a tuple (*root, iter*), where *root* is an approximation to a root of the function *f* in the interval (*xmin, xmax*), using the Halley algorithm for finding a zero; and *iter* is the actual number of iterations.

See also: Wikipedia [1091], BoostMath [48], MathWorld [802].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> f = lambda x: x * x * x - 28
>>> df1 = lambda x: 3 * x * x
>>> df2 = lambda x: 6 * x
>>> guess = 2.33; xmin = 1.0; xmax = 4.0; get_digits = 64;  maxit = 20
>>> res = ereal.Halley(f, df1, df2, guess, xmin, xmax, get_digits, maxit)
>>> print("res (x0, iter): ", res)
```

```
boost.halley(f, df, df2, guess, xmin, xmax, get_digits, maxit)
result:  3.0365889718756627
```

5.3.4 Root Finding With Derivatives: Schröder

`ctxboost.`**`Schroeder`**(*f*, *df*, *d2f*, *guess*, *xmin*, *xmax*, *get_digits*, *maxit*)

where `ctx` is `ctxboost`.

Parameters

- **`f`** – the function for which a root is determined.
- **`df`** – the first derivative of the function *f*.
- **`df2`** – the second derivative of the function *f*.
- **`guess`** – an intial guess.
- **`xmin`** – the left border of the search interval.
- **`xmax`** – the right border of the search interval.
- **`get_digits`** – the targeted number of accurate digits.
- **`maxit`** – the maximal number of iterations.

Returns

a tuple (*root, iter*), where *root* is an approximation to a root of the function *f* in the interval (*xmin, xmax*), using the Schröder algorithm for finding a zero; and *iter* is the actual number of iterations.

See also: BoostMath [48], MathWorld [812], Schröder [495].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> f = lambda x: x * x * x - 28
>>> df1 = lambda x: 3 * x * x
>>> df2 = lambda x: 6 * x
>>> guess = 2.33; xmin = 1.0; xmax = 4.0; get_digits = 64;  maxit = 20
>>> res = ereal.Schroeder(f, df1, df2, guess, xmin, xmax, get_digits, maxit)
>>> print("res (x0, iter): ", res)
```

```
boost.schroder(f, df, df2, guess, xmin, xmax, get_digits, maxit)
result:  3.0365889718756627
```

5.3.5 Locating Function Minima using Brent's algorithm

`ctx.BrentMinimum`(*f*, *a*, *b*, *get_digits*, *maxit*)

where `ctx` is `ctxboost`.

Parameters

- **f** – the function for which a local minimum is determined.
- **a** – the left border of the search interval.
- **b** – the right border of the search interval.
- **get_digits** – the targeted number of accurate digits.
- **maxit** – the maximal number of iterations.

Returns

a tuple (*localmin, fx0, iter*), where *localmin* is an approximation to a local minimum of the function f in the interval (*a,b*), using Brent's algorithm; *fx0* is the value of *f(localmin)*; and *iter* is the number of iterations.

See also: Wikipedia [1086], Zhang [1500], Brent [36], BoostMath [47].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> f = lambda x: (x + 3) * (x - 1) * (x - 1)
>>> bracket_min = 0.5; bracket_max = 1.5;  bits = 125;  maxit = 20
>>> res = ereal.Brent_Minimum(f, bracket_min, bracket_max, bits, maxit)
>>> print("res5 (x0, fx0, iter): ", result3)
```

```
boost.brent_minimum(f3, a, b, get_digits, maxit)
x0:  2.9086877250246185e-09
```

5.4 Mpmath: Rootfinding and optimization

5.4.1 General root-finding interface

`ctx.findroot`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Find a solution to $f(x) = 0$, using *x0* as starting point or interval for *x*.

Multidimensional overdetermined systems are supported. You can specify them using a function or a list of functions.

If the found root does not satisfy $|f(x)|^2 \leq \mathrm{tol}$, an exception is raised (this can be disabled with *verify=False*).

Arguments

f
one dimensional function

x0
starting point, several starting points or interval (depends on solver)

tol
the returned solution has an error smaller than this

verbose
print additional information for each iteration if true

verify
verify the solution and raise a ValueError if $|f(x)|^2 > \mathrm{tol}$

solver
a generator for f and *x0* returning approximative solution and error

maxsteps
after how many steps the solver will cancel

df
first derivative of f (used by some solvers)

d2f
second derivative of f (used by some solvers)

multidimensional
force multidimensional solving

J
Jacobian matrix of f (used by multidimensional solvers)

norm
used vector norm (used by multidimensional solvers)

solver has to be callable with `(f, x0, **kwargs)` and return an generator yielding pairs of approximative solution and estimated error (which is expected to be positive). You can use the following string aliases: 'secant', 'mnewton', 'halley', 'muller', 'illinois', 'pegasus', 'anderson', 'ridder', 'anewton', 'bisect'

5.4.2 Bisection algorithm

`ctx.bisection`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Find a solution to $f(x) = 0$, using *x0* as starting point or interval for *x*.

1d-solver generating pairs of approximative root and error. See also: Wikipedia [1085].

Uses bisection method to find a root of f in [a, b]. Might fail for multiple roots (needs sign change).

Pro:

- robust and reliable

Contra:

- converges slowly
- needs sign change

5.4.3 Secant algorithm

`ctx.secant`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Find a solution to $f(x) = 0$, using *x0* as starting point or interval for x.

1d-solver generating pairs of approximative root and error. See also: Wikipedia [1100].

Needs starting points x0 and x1 close to the root. x1 defaults to x0 + 0.25.

Pro:

- converges fast

Contra:

- converges slowly for multiple roots

The function `findroot()` locates a root of a given function using the secant method by default. A simple example use of the secant method is to compute π as the root of $\sin x$ closest to $x_0 = 3$:

```
>>> from xlcalcnet import *
>>> mp.dps = 30; mp.pretty = True
>>> findroot(sin, 3)
3.14159265358979323846264338328
```

The secant method can be used to find complex roots of analytic functions, although it must in that case generally be given a nonreal starting value (or else it will never leave the real line):

```
>>> mp.dps = 15
>>> findroot(lambda x: x**3 + 2*x + 1, j)
(0.226698825758202 + 1.46771150871022j)
```

A nice application is to compute nontrivial roots of the Riemann zeta function with many digits (good initial values are needed for convergence):

```
>>> mp.dps = 30
>>> findroot(zeta, 0.5+14j)
(0.5 + 14.1347251417346937904572519836j)
```

The secant method can also be used as an optimization algorithm, by passing it a derivative of a function. The following example locates the positive minimum of the gamma function:

```
>>> mp.dps = 20
>>> findroot(lambda x: diff(gamma, x), 1)
1.4616321449683623413
```

5.4.4 Illinois, Pegasus, Anderson algorithms

`ctx.illinois`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Find a solution to $f(x) = 0$, using *x0* as starting point or interval for x.

1d-solver generating pairs of approximative root and error. See also: Wikipedia [1092], Dowell and Jarratt [288], Wikipedia [1083].

Uses Illinois method or similar to find a root of f in [a, b]. Might fail for multiple roots (needs sign change). Combines bisect with secant (improved regula falsi).

The only difference between the methods is the scaling factor m, which is used to ensure convergence (you can choose one using the 'method' keyword):

Illinois method ('illinois'):
m = 0.5

Pegasus method ('pegasus'):
m = fb/(fb + fz)

Anderson-Bjoerk method ('anderson'):
m = 1 - fz/fb if positive else 0.5

Pro:

- converges very fast

Contra:

- has problems with multiple roots
- needs sign change

Intersection methods

When you need to find a root in a known interval, it's highly recommended to use an intersection-based solver like `'anderson'` or `'ridder'`. Usually they converge faster and more reliable. They have however problems with multiple roots and usually need a sign change to find a root:

```
>>> findroot(lambda x: x**3, (-1, 1), solver='anderson')
0.0
```

Be careful with symmetric functions:

```
>>> findroot(lambda x: x**2, (-1, 1), solver='anderson')
Traceback (most recent call last):
    ...
ZeroDivisionError
```

It fails even for better starting points, because there is no sign change:

```
>>> findroot(lambda x: x**2, (-1, .5), solver='anderson')
Traceback (most recent call last):
    ...
ValueError: Could not find root within given tolerance. (1.0 > 2.1684043449710088680 1e-19)
Try another starting point or tweak arguments.
```

5.4.5 Muller algorithm

`ctx.muller`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Find a solution to $f(x) = 0$, using *x0* as starting point or interval for *x*.

1d-solver generating pairs of approximative root and error. See also: Wikipedia [1093].

Needs starting points x0, x1 and x2 close to the root. x1 defaults to x0 + 0.25; x2 to x1 + 0.25. Uses Muller's method that converges towards complex roots.

Pro:

- converges fast (somewhat faster than secant)
- can find complex roots

Contra:

- converges slowly for multiple roots
- may have complex values for real starting points and real roots

Complex roots

For complex roots it's recommended to use Muller's method as it converges even for real starting points very fast:

```
>>> findroot(lambda x: x**4 + x + 1, (0, 1, 2), solver='muller')
(0.727136084491197 + 0.934099289460529j)
```

5.4.6 Ridder algorithm

`ctx.ridder`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Find a solution to $f(x) = 0$, using *x0* as starting point or interval for *x*.

1d-solver generating pairs of approximative root and error. See also: Wikipedia [1099].

Ridders' method to find a root of f in [a, b]. Is told to perform as well as Brent's method (see Wikipedia [1086]) while being simpler.

Pro:

- very fast
- simpler than Brent's method

Contra:

- two function evaluations per step
- has problems with multiple roots
- needs sign change

Finally, a useful application is to compute inverse functions, such as the Lambert W function which is the inverse of we^w, given the first term of the solution's asymptotic expansion as the initial value. In basic cases, this gives identical results to mpmath's built-in `lambertw` function:

```
>>> def lambert(x):
...     return findroot(lambda w: w*exp(w) - x, log(1+x))
...
>>> mp.dps = 15
```

(continues on next page)

(continued from previous page)

```
>>> lambert(1); lambertw(1)
0.567143290409784
0.567143290409784
>>> lambert(1000); lambert(1000)
5.2496028524016
5.2496028524016
```

5.4.7 Brent algorithm (to be implemented from Scipy brent)

`ctx.brent`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Find a solution to $f(x) = 0$, using *x0* as starting point or interval for x.

5.4.8 Newton algorithm

`ctx.newton_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

1d-solver generating pairs of approximative root and error. See also Wikipedia [1094], MathWorld [807].

Needs starting points x0 close to the root.

Pro:

- converges fast
- sometimes more robust than secant with bad second starting point

Contra:

- converges slowly for multiple roots
- needs first derivative
- 2 function evaluations per iteration

Multiple roots

For multiple roots all methods of the Newtonian family (including secant) converge slowly. Consider this example:

```
>>> f = lambda x: (x - 1)**99
>>> findroot(f, 0.9, verify=False)
0.918073542444929
```

Even for a very close starting point the secant method converges very slowly. Use `verbose=True` to illustrate this.

5.4.9 Newton-Steffenson algorithm (ANewton)

`ctx.newton_steffenson_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

EXPERIMENTAL 1d-solver generating pairs of approximative root and error.

Uses Newton's method modified to use Steffensens method when convergence is slow. (I.e. for multiple roots.)

Steffensen's method for quadratic convergence of a linear converging sequence. See also Wikipedia [1101]. Don not use it for higher rates of convergence. It may even work for divergent sequences.

Definition: F(x) = (x*f(f(x)) - f(x)**2) / (f(f(x)) - 2*f(x) + x)

Multiple roots

For multiple roots all methods of the Newtonian family (including secant) converge slowly. Consider this example:

```
>>> f = lambda x: (x - 1)**99
>>> findroot(f, 0.9, verify=False)
0.918073542444929
```

Even for a very close starting point the secant method converges very slowly. Use `verbose=True` to illustrate this.

Alternatively you can use an experimental Newtonian solver that keeps track of the speed of convergence and accelerates it using Steffensen's method if necessary:

```
>>> findroot(f, -10, solver='anewton', verbose=True)
x:     -9.88888888888888889
error: 0.11111111111111111111
converging slowly
x:     -9.77890011223344556678
error: 0.10998877665544332211
converging slowly
x:     -9.67002233332199662166
error: 0.10887778911448945119
converging slowly
accelerating convergence
x:     -9.562244329955107669
error: 0.107778003366888854764
converging slowly
x:     0.99999999999999999214
error: 10.562244329955107759
x:     1.0
error: 7.8598304758094664213e-18
ZeroDivisionError: canceled with x = 1.0
1.0
```

5.4.10 Modified Newton algorithm (MNewton)

`ctx.mnewton_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

1d-solver generating pairs of approximative root and error. See also: Wikipedia [1094].

Needs starting point x0 close to the root. Uses modified Newton's method that converges fast regardless of the multiplicity of the root.

Pro:

- converges fast for multiple roots

Contra:

- needs first and second derivative of f
- 3 function evaluations per iteration

Multiple roots

For multiple roots all methods of the Newtonian family (including secant) converge slowly. Consider this example:

```
>>> f = lambda x: (x - 1)**99
>>> findroot(f, 0.9, verify=False)
0.918073542444929
```

Even for a very close starting point the secant method converges very slowly. Use `verbose=True` to illustrate this.

It is possible to modify Newton's method to make it converge regardless of the root's multiplicity:

```
>>> findroot(f, -10, solver='mnewton')
1.0
```

5.4.11 Halley algorithm

ctx.**halley_c**(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where ctx is dec, mpm, or gmp.

1d-solver generating pairs of approximative root and error. See also: Wikipedia [1091], MathWorld [802].

Needs a starting point x0 close to the root. Uses Halley's method with cubic convergence rate.

Pro:

- converges even faster the Newton's method
- useful when computing with *many* digits

Contra:

- needs first and second derivative of f
- 3 function evaluations per iteration
- converges slowly for multiple roots

5.4.12 Roots of a vector function (MDNewton, to be implemented)

ctx.**md_newton_c**(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where ctx is dec, mpm, or gmp.

Find the root of a vector function numerically using Newton's method.

f is a vector function representing a nonlinear equation system.

x0 is the starting point close to the root.

J is a function returning the Jacobian matrix for a point.

Supports overdetermined systems.

Use the 'norm' keyword to specify which norm to use. Defaults to max-norm. The function to calculate the Jacobian matrix can be given using the keyword 'J'. Otherwise it will be calculated numerically.

Please note that this method converges only locally. Especially for high- dimensional systems it is not trivial to find a good starting point being close enough to the root.

Multidimensional functions are also supported:

```
>>> f = [lambda x1, x2: x1**2 + x2,
...      lambda x1, x2: 5*x1**2 - 3*x1 + 2*x2 - 3]
>>> findroot(f, (0, 0))
[-0.618033988749895]
[-0.381966011250105]
>>> findroot(f, (10, 10))
[ 1.61803398874989]
[-2.61803398874989]
```

You can verify this by solving the system manually.

Please note that the following (more general) syntax also works:

```
>>> def f(x1, x2):
...     return x1**2 + x2, 5*x1**2 - 3*x1 + 2*x2 - 3
...
>>> findroot(f, (0, 0))
[-0.618033988749895]
[-0.381966011250105]
```

See also: https://numerary.readthedocs.io/en/latest/newton-method.html

See also: https://en.wikipedia.org/wiki/Newton%27s_method#Systems_of_equations

See also powel hybrid method

See also: https://github.com/mateuv/MetodosNumericos/blob/master/python/NumericalMethodsInEngineeringWithPython/goldSearch.py

See also: https://github.com/mateuv/MetodosNumericos/blob/master/python/NumericalMethodsInEngineeringWithPython/powell.py

5.4.13 Levenberg-Marquardt algorithm

`ctx.levenberg_marquardt_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: https://lmfit.github.io/lmfit-py/

See also: https://github.com/lmfit/lmfit-py/tree/master

5.4.14 BFGS algorithm.

`ctx.bfgs_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: https://medium.com/@tru11631/bfgs-and-l-bfgs-with-code-2fcaeee7b225

5.4.15 L-BFGS algorithm.

`ctx.lbfgs_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: https://medium.com/@tru11631/bfgs-and-l-bfgs-with-code-2fcaeee7b225

5.4.16 Conjugate gradient.

`ctx.conj_grad_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: https://gregorygundersen.com/blog/2022/03/20/conjugate-gradient-descent/

See also: https://indrag49.github.io/Numerical-Optimization/conjugate-gradient-methods-1.html

See also: https://indrag49.github.io/Numerical-Optimization/conjugate-gradient-methods-1.html#nonlinear-conjugate-gradient-algorithm

See also: https://en.wikipedia.org/wiki/Nonlinear_conjugate_gradient_method

5.5 DAMath: Numerical Quadrature

For a general overview, see: Wikipedia [1095], Wikipedia [1102], MathWorld [801], Wikipedia [1082].

5.5.1 General error codes for Amath integration functions, type 1

Amath integration functions which are NOT using the double exponential transformation return an error code `ier`, which has the following meaning:

`ier = 0`: Normal and reliable termination of the routine. It is assumed that the requested accuracy has been achieved.

`ier > 0`: Abnormal termination of the routine the estimates for integral and error are less reliable. It is assumed that the requested accuracy has not been achieved.

`ier = 1`: Maximum number of subdivisions allowed has been achieved. One can allow more subdivisions by increasing the value of limit (and taking the according dimension adjustments into account). However, if this yields no improvement it is advised to analyze the integrand in order to determine the integration difficulties. If the position of a local difficulty can be determined (e.g. singularity, discontinuity within the interval) one will probably gain from splitting up the interval at this point and calling the integrator on the subranges. If possible, an appropriate special-purpose integrator should be used, which is designed for handling the type of difficulty involved.

`ier = 2`: The occurrence of roundoff error is detected, which prevents the requested tolerance from being achieved. The error may be under-estimated.

`ier = 3`: Extremely bad integrand behaviour occurs at some points of the integration interval.

`ier = 4`: The algorithm does not converge. Roundoff error is detected in the extrapolation table. It is presumed that the requested tolerance cannot be achieved, and that the returned result is the best which can be obtained.

`ier = 5`: The integral is probably divergent, or slowly convergent. It must be noted that divergence can occur with any other value of ier.

`ier = 6`: The input is invalid, because epsabs <= 0 and epsrel < 50*eps_d. result, abserr, last are set to zero.

`ier = 7`: The input is invalid, limit < 0 or limit > QMAXLIM. result, abserr, last are set to zero.

`ier = 8`: Dynamic list vectors cannot be allocated. result, abserr, last are set to zero.

`ier = 9`: At least one limit a or b is NaN, or infinite a=b. result, abserr, last are set to NaN_d.

5.5.2 General error codes for Amath integration functions, type 2

Amath integration functions which are using the double exponential transformation return an error code `ier`, which has the following meaning:

`ier = 0`: Normal and reliable termination of the routine. It is assumed that the requested accuracy has been achieved.

`ier > 0`: Abnormal termination of the routine the estimates for integral and error are less reliable. It is assumed that the requested accuracy has not been achieved.

`ier = 1`: eps < 8*eps_x.

`ier = 2`: roundoff problems.

`ier = 3`: max. iterations, result/abserr have values.

5.5.3 Global adaptive quadrature by Forsythe, Malcolm, Moler (quanc8)

`ctx.quanc8`(*f*, *a*, *b*, *epsabs*, *epsrel*, *limit*)

where `ctx` is `math53`. Performs a global adaptive quadrature of f over (a,b) based on a Fortran subroutine by Forsythe, Malcolm, Moler.

Parameters

- **f** – the function for which the integral is determined.
- **a** – the left border of the integration interval.
- **b** – the right border of the integration interval.
- **abserr** – the absolute accuracy requested.
- **relerr** – the relative accuracy requested.

Returns

a tuple (*integral, err, flag, neval*), where *integral* is an approximation to the integral; *abserr* is an estimate of of the modulus of the absolute error; *neval* is the number of integrand evaluations; and *ier* is the error code (success=0, error > 0: see *General error codes for Amath integration functions, type 1*).

Include reference to Forsythe, Malcolm, Moler.

The routine in Python:

```
>>> from xlcalcnet import math53
>>> a = 0.0; b = 2.0; epsabs = 1E-8; epsrel = 0.0; limit = 0
>>> f = lambda x: ereal.Exp(-x * x / 2)
>>> result, abserr, neval, ier = ereal.Qags(f, a, b, epsabs, epsrel, limit)
>>> print("result: ", result, "abserr: ", abserr)
>>> print("neval: ", neval, "ier: ", ier)
```

5.5.4 21-point Gauss-Kronrod rule, finite interval (qags)

`ctx.qags`(*f*, *a*, *b*, *epsabs*, *epsrel*, *limit*)

where `ctx` is `math53`. Performs a global adaptive quadrature of f over (a,b) based on 21-point Gauss-Kronrod rule for the subintervals, with acceleration by Wynn's epsilon algorithm.

Parameters

- **f** – the function for which the integral is determined.
- **a** – the left border of the integration interval.
- **b** – the right border of the integration interval.
- **epsabs** – the absolute accuracy requested.
- **epsrel** – the relative accuracy requested.
- **limit** – upperbound on the no. of subintervals, 0: use DefLimit.

Returns

a tuple (*integral, abserr, neval, ier*), where *integral* is an approximation to the integral; *abserr* is an estimate of of the modulus of the absolute error; *neval* is the number of integrand evaluations; and *ier* is the error code (success=0, error > 0: see *General error codes for Amath integration functions, type 1*).

See also: Piessens *et al.* [470], Wikipedia [1096], Wikipedia [1110].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> a = 0.0; b = 2.0; epsabs = 1E-8; epsrel = 0.0; limit = 0
>>> f = lambda x: ereal.Exp(-x * x / 2)
>>> result, abserr, neval, ier = ereal.Qags(f, a, b, epsabs, epsrel, limit)
>>> print("result: ", result, "abserr: ", abserr)
>>> print("neval: ", neval, "ier: ", ier)
```

5.5.5 15-point Gauss-Kronrod rule, infinite interval (qagi)

`ctx.qagi`(*f*, *bound*, *inf*, *epsabs*, *epsrel*, *limit*)

where `ctx` is `math53`. Performs a global adaptive quadrature of f over an infinite interval based on a transformed 15-point Gauss-Kronrod for the subintervals, with acceleration by Wynn's epsilon algorithm.

Parameters

- **f** – the function for which the integral is determined.
- **bound** – finite bound of integration range (if any).
- **inf** – indicating the kind of integration range involved: 1 corresponds to (bound, +infinity), -1 to (-infinity, bound), 2 to (-infinity, +infinity).
- **epsabs** – the absolute accuracy requested.
- **epsrel** – the relative accuracy requested.
- **limit** – upperbound on the no. of subintervals, 0: use DefLimit.

Returns

a tuple (*integral, abserr, neval, ier*), where *integral* is an approximation to the integral; *abserr* is an estimate of of the modulus of the absolute error; *neval* is the number of integrand evaluations; and *ier* is the error code (success=0, error > 0: see *General error codes for Amath integration functions, type 1*).

See also: Piessens *et al.* [470], Wikipedia [1096].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> bound = 0.0; infcode = 2; epsabs = 1E-8; epsrel = 0.0; limit = 0
>>> f = lambda x: ereal.Exp(-x * x / 2)
>>> result, abserr, neval, ier = ereal.Qagi(f, bound, infcode, epsabs, epsrel, limit)
>>> print("result: ", result, "abserr: ", abserr)
>>> print("neval: ", neval, "ier: ", ier)
```

5.5.6 Cauchy principal value, finite interval (qawc)

`ctx.qawc`(*f*, *a*, *b*, *c*, *epsabs*, *epsrel*, *limit*)

where `ctx` is `math53`. Performs an adaptive quadrature of $f(x)/(x-c)$ over the finite interval (a, b) with the singularity at c and c not equal a or b. The routine calculates an approximation result to the Cauchy principal value.

Parameters

- **f** – the function for which the integral is determined.
- **a** – the left border of the integration interval.
- **b** – the right border of the integration interval.
- **c** – singularity, ier=6 if $c = a$ or $c = b$.
- **epsabs** – the absolute accuracy requested.
- **epsrel** – the relative accuracy requested.
- **limit** – upperbound on the no. of subintervals, 0: use DefLimit.

Returns

a tuple (*integral, abserr, neval, ier*), where *integral* is an approximation to the integral; *abserr* is an estimate of of the modulus of the absolute error; *neval* is the number of integrand evaluations; and *ier* is the error code (success=0, error > 0: see *General error codes for Amath integration functions, type 1*).

See also: https://maxima.sourceforge.io/docs/manual/maxima_100.html

See also: Piessens *et al.* [470], Wikipedia [1096], Wikipedia [1105].

The routine in Python:

```
# Qawc Example is missing
bound = 0.0; infcode = 2; epsabs = 1E-8; epsrel = 0.0; limit = 0
f = lambda x: ereal.Exp(-x * x / 2)
result, abserr, neval, ier = ereal.Qagi(f, bound, infcode, epsabs, epsrel, limit)
print("ereal.Qagi")
print("result: ", result, "abserr: ", abserr)
print("neval: ", neval, "ier: ", ier)
print()
```

5.5.7 Double Exponential (DE) transformation, finite interval (intde)

`ctx.intde`(*f*, *a*, *b*, *eps*)

where `ctx` is `math53`. Performs an automatic quadrature of f(x) over the finite interval (a,b)} using Double Exponential (DE) transformation.

Parameters

- **f** – the function for which the integral is determined.
- **a** – the left border of the integration interval.
- **b** – the right border of the integration interval.
- **eps** – the relative accuracy requested.

Returns

a tuple (*integral, abserr, neval, ier*), where *integral* is an approximation to the integral, if ier=0 or 3; *abserr* is an estimate of of the modulus of the absolute error, if ier=0 or 3; *neval* is the number of integrand evaluations; and *ier* is the error code (success=0, error > 0: see *General error codes for Amath integration functions, type 2*).

See also: Wikipedia [1119], Takahasi and Mori [511], Ooura and M. [449], Mori [416].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> a = 0.0; b = 2.0; eps = 1E-8
>>> f = lambda x: ereal.Exp(-x * x / 2)
>>> result, abserr, neval, ier = ereal.Intde(f, a, b, eps)
>>> print("result: ", result, "abserr: ", abserr)
>>> print("neval: ", neval, "ier: ", ier)
```

5.5.8 DE transformation, infinite interval, no oscillatory factor (intdei)

`ctx.intdei`(*f*, *a*, *eps*)

where `ctx` is `math53`. Performs an automatic quadrature of f(x) over (a,INF) using Double Exponential transformation when f(x) has no oscillatory factor.

Parameters

- **f** – the function for which the integral is determined.
- **a** – the left border of the integration interval.
- **eps** – the relative accuracy requested.

Returns

a tuple (*integral, abserr, neval, ier*), where *integral* is an approximation to the integral, if ier=0 or 3; *abserr* is an estimate of of the modulus of the absolute error, if ier=0 or 3; *neval* is the number of integrand evaluations; and *ier* is the error code (success=0, error > 0: see *General error codes for Amath integration functions, type 2*).

See also: Wikipedia [1119], Takahasi and Mori [511], Ooura and M. [449], Mori [416].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> a = 0.0; eps = 1E-8
>>> f = lambda x: ereal.Exp(-x * x / 2)
>>> result, abserr, neval, ier = ereal.Intdei(f, a, eps)
>>> print("result: ", result, "abserr: ", abserr)
>>> print("neval: ", neval, "ier: ", ier)
```

5.5.9 DE transformation, infinite interval, oscillatory factor (intdeo)

`ctx.intdeo`(*f*, *a*, *omega*, *eps*)

where `ctx` is `math53`. Performs an automatic quadrature of f(x) over (a,INF) using Double Exponential transformation when f(x) has an oscillatory factor.

Parameters

- **f** – the function for which the integral is determined.
- **a** – the left border of the integration interval.
- **omega** – oscillatory factor.
- **eps** – the relative accuracy requested.

Returns

a tuple (*integral, abserr, neval, ier*), where *integral* is an approximation to the integral, if ier=0 or 3; *abserr* is an estimate of of the modulus of the absolute error, if ier=0 or 3; *neval* is the number of integrand evaluations; and *ier* is the error code (success=0, error > 0: see *General error codes for Amath integration functions, type 2*).

See also: Wikipedia [1119], Takahasi and Mori [511], Ooura and M. [449], Mori [416].

The routine in Python:

```
# Intdeo example is missing
a = 0.0; eps = 1E-8
f = lambda x: ereal.Exp(-x * x / 2)
result, abserr, neval, ier = ereal.Intdei(f, a, eps)
print("ereal.Intdei")
print("result: ", result, "abserr: ", abserr)
print("neval: ", neval, "ier: ", ier)
return "0"
```

5.6 Boost/Math: Numerical integration

For a general overview, see: Wikipedia [1095], Wikipedia [1102].

5.6.1 Trapezoidal Quadrature

`ctxboost.Trapezoidal`(*f*, *a*, *b*, *tol*, *max_refinements*)

where `ctx` is `ctxboost`.

Parameters

- **`f`** – the function for which the integral is determined.
- **`a`** – the left border of the integration interval.
- **`b`** – the right border of the integration interval.
- **`tol`** – the requested tolerance.
- **`max_refinements`** – the maximal number of refinements. The default is 12.

Returns

a tuple (*integral, error, L1*), where *integral* is an approximation to the integral, using a trapezoidal quadrature; *error* is an estimate of the error; and *L1* is an estimate of the L1 value.

See also: BoostMath [49], Wikipedia [1120].

If we assume only that the integrand is twice continuously differentiable, we can prove that the error of the composite trapezoidal rule is O(h2). Hence halving the interval only cuts the error by about a fourth, which in turn implies that we must evaluate the function many times before an acceptable accuracy can be achieved.

However, the trapezoidal rule has an astonishing property: If the integrand is periodic, and we integrate it over a period, then the trapezoidal rule converges faster than any power of the step size h. This can be seen by examination of the Euler-Maclaurin summation formula, which relates a definite integral to its trapezoidal sum and error terms proportional to the derivatives of the function at the endpoints and the Bernoulli numbers. If the derivatives at the endpoints are the same or vanish, then the error very nearly vanishes. Hence the trapezoidal rule is essentially optimal for periodic integrands.

Other classes of integrands which are integrated efficiently by this method are the bump functions and bell-shaped integrals over the infinite interval. For details, see Trefethen's SIAM review.

The routine in Python:

```
>>> from xlcalcnet import math53
>>> f = lambda x: 1 / (5 - 4 * ereal.Cos(x))
>>> result2 = ereal.Trapezoidal(f, a, b)
>>> print("res: ", res)
```

```
boost.trapezoidal(f6, a, b, tol)
result:  1.3639170485190606
```

5.6.2 Gauss-Legendre quadrature

`ctxboost.GaussLegendre`(*f*, *a*, *b*)

where `ctx` is `ctxboost`.

Parameters

- **f** – the function for which the integral is determined.
- **a** – the left border of the integration interval.
- **b** – the right border of the integration interval.

Returns

a tuple (*integral, L1*), where *integral* is an approximation to the integral, using a non-adaptive Gauss-Legendre quadrature; and *L1* is an estimate of the L1 value.

See also: Wikipedia [1112], BoostMath [45].

This is intentionally a very simple quadrature routine, it obtains no estimate of the error, and is not adaptive, but is very efficient in simple cases that involve integrating smooth "bell like" functions and functions with rapidly convergent power series.

The Gaussian quadrature routine support both real and complex-valued quadrature.

Internally class gauss has pre-computed tables of abscissa and weights for 7, 15, 20, 25 and 30 points at up to 100-decimal digit precision. That means that using for example, gauss<double, 30>::integrate incurs absolutely zero set-up overhead from computing the abscissa/weight pairs. When using multiprecision types with less than 100 digits of precision, then there is a small initial one time cost, while the abscissa/weight pairs are constructed from strings.

However, for types with higher precision, or numbers of points other than those given above, the abscissa/weight pairs are computed when first needed and then cached for future use, which does incur a noticeable overhead.

The routine in Python:

```
>>> from xlcalcnet import math53
>>> a = 0.0; b = 1.0; tol = 1.0E-9
>>> f = lambda x: x * x * ereal.Atan(x)
>>> res = ereal.GaussLegendre(f, a, b)
>>> print("res: ", res)
```

```
boost.gauss_legendre(f6, a, b, tol)
result:  14.0
```

5.6.3 Gauss-Kronrod Quadrature

ctxboost.**GaussKronrod**(*f*, *a*, *b*, *max_depth*, *tol*)

where ctx is ctxboost.

Parameters

- **f** – the function for which the integral is determined.
- **a** – the left border of the integration interval.
- **b** – the right border of the integration interval.
- **max_depth** – the maximal number of levels.
- **tol** – the requested tolerance.

Returns

a tuple (*integral, error, L1*), where *integral* is an approximation to the integral, using a Gauss-Kronrod quadrature; *error* is an estimate of the error; and *L1* is an estimate of the L1 value.

See also: Wikipedia [1110], BoostMath [44].

The idea behind Gaussian quadrature is to choose n nodes and weights in such a way that polynomials of order 2n-1 are integrated exactly. However, integration of polynomials is trivial, so it is rarely done via numerical methods. Instead, transcendental and numerically defined functions are integrated via Gaussian quadrature, and the defining problem becomes how to estimate the remainder. Gaussian quadrature alone (without some form of interval splitting) cannot answer this question.

It is possible to compute a Gaussian quadrature of order n and another of order (say) 2n+1, and use the difference as an error estimate. However, this is not optimal, as the zeros of the Legendre polynomials (nodes of the Gaussian quadrature) are never the same for different orders, so 3n+1 function evaluations must be performed. Kronrod considered the problem of how to interleave nodes into a Gaussian quadrature in such a way that all previous function evaluations can be reused, while increasing the order of polynomials that can be integrated exactly. This allows an a posteriori error estimate to be provided while still preserving exponential convergence. Kronrod discovered that by adding n+1 nodes (computed from the zeros of the Legendre-Stieltjes polynomials) to a Gaussian quadrature of order n, he could integrate polynomials of order 3n+1.

The integration routines provided here will perform either adaptive or non-adaptive quadrature, they should be chosen for the integration of smooth functions with no end point singularities.

The number of points specified in the Points template parameter must be an odd number: giving a (N-1)/2 Gauss quadrature as the comparison for error estimation.

Internally class gauss_kronrod has pre-computed tables of abscissa and weights for 15, 31, 41, 51 and 61 Gauss-Kronrod points at up to 100-decimal digit precision. That means that using for example, gauss_kronrod<double, 31>::integrate incurs absolutely zero set-up overhead from computing the abscissa/weight pairs. When using multiprecision types with less than 100 digits of precision, then there is a small initial one time cost, while the abscissa/weight pairs are constructed from strings.

The Gauss-Kronrod quadrature support integrands defined on the real line and returning complex values. In this case, the template argument is the real type, and the complex type is deduced via the return type of the function.

The routine in Python:

```
>>> from xlcalcnet import math53
>>> a = 0.0; b = float('+inf')
>>> f = lambda x: ereal.Exp(-x * x / 2)
>>> result3 = ereal.GaussKronrod(f, a, b)
>>> print("result3: ", result3)
```

```
boost.gauss_kronrod(f6, a, b, max_depth, tol)
result:  14.000000000000002
```

5.6.4 Double-exponential quadrature: tanh_sinh

ctxboost.**TanhSinh**(*f*, *a*, *b*, *tol*, *max_refinements*)

where ctx is ctxboost.

Parameters

- **f** – the function for which the integral is determined.
- **a** – the left border of the integration interval.
- **b** – the right border of the integration interval.
- **tol** – the requested tolerance.
- **max_refinements** – the maximal number of refinements. The default is 12.

Returns

a tuple (*integral, error, L1, levels*), where *integral* is an approximation to the integral, using a tanh-sinh quadrature; *error* is an estimate of the error; *L1* is an estimate of the L1 value; and *levels* is the number of levels used.

See also: Wikipedia [1119], BoostMath [42], Okayama *et al.* [430], Okayama [431].

The tanh-sinh quadrature routine provided by boost is a rapidly convergent numerical integration scheme for holomorphic integrands. By this we mean that the integrand is the restriction to the real line of a complex-differentiable function which is bounded on the interior of the unit disk $|z| < 1$, so that it lies within the so-called Hardy space. If your integrand obeys these conditions, it can be shown that tanh-sinh integration is optimal, in the sense that it requires the fewest function evaluations for a given accuracy of any quadrature algorithm for a random element from the Hardy space.

Complex integrands are supported.

The routine in Python:

```
>>> from xlcalcnet import math53
>>> a = -1.0; b = 1.0
>>> f = lambda x: 5 * x + 7
>>> res = ereal.TanhSinh(f, a, b)
>>> print("res: ", res)
```

```
boost.tanh_sinh(f6, a, b)
result:  14.0
```

5.6.5 Double-exponential quadrature: sinh_sinh

ctxboost.**SinhSinh**(*f*, *tol*, *max_refinements*)

where ctx is ctxboost.

Parameters

- **f** – the function for which the integral is determined.
- **tol** – the requested tolerance.
- **max_refinements** – the maximal number of refinements. The default is 9.

Returns

a tuple (*integral, error, L1, levels*), where *integral* is an approximation to the integral, using a sinh-sinh quadrature over the limits of integration $(-\infty, \infty)$; *error* is an estimate of the error; *L1* is an estimate of the L1 value; and *levels* is the number of levels used.

See also: BoostMath [41], Okayama *et al.* [430], Okayama [431].

Returns the value of the integral of f over the limits of integration $(-\infty, \infty)$. Complex integrands are supported.

The routine in Python:

```
>>> from xlcalcnet import math53
>>> f = lambda x: math.exp(-x * x)
>>> res = ereal.SinhSinh(f)
>>> print("res: ", res)
```

```
boost.sinh_sinh(f7)
result:  1.7724538509055168
```

5.6.6 Double-exponential quadrature: exp_sinh

ctxboost.**ExpSinh**(*f*, *tol*, *max_refinements*, *a*, *b*)

where ctx is ctxboost.

Parameters

- **f** – the function for which the integral is determined.
- **tol** – the requested tolerance.
- **max_refinements** – the maximal number of refinements. The default is 12.
- **a** – the left border of the integration interval. The default is 0.
- **b** – the right border of the integration interval. The default is $+\infty$. Note that *a* and *b* cannot both be finite.

Returns

a tuple (*integral, error, L1, levels*), where *integral* is an approximation to the integral, using a tanh-sinh quadrature; *error* is an estimate of the error; *L1* is an estimate of the L1 value; and *levels* is the number of levels used.

Returns the value of the integral of f over the limits of integration $(0, \infty)$. Complex integrands are supported.

See also: BoostMath [40], Okayama *et al.* [430], Okayama [431].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> f = lambda x: math.exp(-3 * x)
>>> result3 = ereal.ExpSinh(f)
>>> print("result3: ", result3)
```

```
boost.exp_sinh(f8)
result:  0.33333333333333337
```

5.6.7 Fourier Integral, Cosine

ctxboost.**OouraCos**(*f*)

where ctx is ctxboost.

Parameters
f – the function for which the integral is determined.

Returns
a tuple (*integral, error*), where *integral* is an approximation to the integral, using a Ooura Cos quadrature of the function f in the interval (0, +inf); *error* is an estimate of the error.

See also: BoostMath [43], Ooura and M. [450], Ooura [451].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> f = lambda x: 1 / (x * x + 1)
>>> result3 = ereal.Ooura_Cos(f)
>>> print("result3: ", result3)
```

```
boost.OouraCos(f3, a, b, get_digits, maxit)
x0:  2.9086877250246185e-09
```

5.6.8 Fourier Integral, Sine

`ctxboost.OouraSin(`*f*`)`

where `ctx` is `ctxboost`.

Parameters
f – the function for which the integral is determined.

Returns
a tuple (*integral, error*), where *integral* is an approximation to the integral, using a Ooura Sin quadrature of the function f in the interval (0, +inf); *error* is an estimate of the error.

See also: BoostMath [43], Ooura and M. [450], Ooura [451].

The routine in Python:

```
>>> from xlcalcnet import math53
>>> f = lambda x: 1 / x
>>> result3 = ereal.Ooura_Sin(f)
>>> print("result3: ", result3)
```

```
boost.OouraCos(f3, a, b, get_digits, maxit)
x0:  2.9086877250246185e-09
```

5.6.9 Calculating the pdf from the characteristic function

`ctx.pdf_from_cf_boost(`*x*, *cf*`)`

where `ctx` is `dec`, `mpm`, or `gmp`.

Calculates the pdf as the inverse Fourier transform of its characteristic function.

The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\mathrm{pdf}_X(x) = \frac{1}{2\pi}\int_{-\infty}^{\infty} e^{ity} C_X(t)\mathrm{d}y = \frac{1}{\pi}\int_0^{\infty} \Re\left(e^{-itx} C_X(t)\right)\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z. We also have

$$\int_0^{\infty} \Re\left(e^{-itx} C_X(t)\right)\mathrm{d}t = \int_0^{\infty} \Re\left(C_X(t)\right)\cos(tx)\mathrm{d}t + \int_0^{\infty} \Im\left(C_X(t)\right)\sin(tx)\mathrm{d}t.$$

Using the right-hand side of this equation allows for efficient use of the quadrature formula of Fillon, with the half-period $\omega = \pi/x$.

5.6.10 Calculating the cdf from the characteristic function

`ctx.cdf_from_cf_continuous_boost(`*x*, *cf*`)`

where `ctx` is `dec`, `mpm`, or `gmp`.

Calculates the cdf from the characteristic function using the procedure of Gil-Pelaez.

Assuming that the characteristic function is absolutely integrable over $(-\infty, \infty)$, Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function only. In particular,

$$\mathrm{cdf}_X(x) = \frac{1}{2} - \frac{1}{2\pi}\int_{-\infty}^{\infty} \frac{e^{-itx}C_X(t) - e^{itx}C_X(t)}{it}\mathrm{d}t = \frac{1}{2} - \frac{1}{\pi}\int_0^{\infty} \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z. We also have

$$\int_0^{\infty} \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t = \int_0^{\infty} \Im\left(\frac{C_X(t)}{t}\right)\cos(tx)\mathrm{d}t - \int_0^{\infty} \Re\left(\frac{C_X(t)}{t}\right)\sin(tx)\mathrm{d}t.$$

Using the right-hand side of this equation allows for efficient use of the quadrature formula of Fillon, with the half-period $\omega = \pi/x$.

The python code is currently in Charfun.py

The following code provides a test-suite for the numerical inversion of the characteristic function:

```
class tests_charfunc(rv_cont):

    def __init__(self, rv2, x = 5, a = 0, b = 2):

        cdf_value = rv2.cdf(x)
        print ("rv2.cdf(x): ", cdf_value)

        rv2.set_x(x)
        plot(rv2.gil_pelaez_imag, [a, b], points=200)
        print

        rv2.set_x(x)
        plot(rv2.gil_pelaez_cos, [a, b], points=200)
        print

        rv2.set_x(x)
        plot(rv2.gil_pelaez_sin, [a, b], points=200)
        print

        rv2.set_x(x)
        I0 = quad(rv2.gil_pelaez_imag, [0, +inf])
        print("Integral: ", I0)
        result0 = 0.5 - I0/pi
        print("result0:", result0 )
        print("diff0:", result0 - cdf_value)


        rv2.set_x(x)
        I1 =quadosc(rv2.gil_pelaez_cos, [0, inf], period=1*pi/x) # half period
        print("I1:", I1 )

        rv2.set_x(x)
        I2 =quadosc(rv2.gil_pelaez_sin, [0, inf], period=1*pi/x) # half period
        print("I2:", I2 )

        I3 = I1 + I2
        print("I3:", I3 )
        print("Int diff:", I3 - I0)
        result3 = 0.5 - I3/pi
        print("result3:", result3 )
        print("diff3:", result3 - cdf_value)
```

Example: non-central chi-squared distribution

The following code shows the difference between using the generic integration (error: diff0: -6.1218939724378892896e-6) and quadrature for oscillatory functions (error: diff3: 8.4703294725430033907e-22).

```
mp.dps = 20
print()
print ("Hello mpDistributions local ! ")
print()


a = 0.0
b = 2

n = mpf("5")
x = mpf("10")
rv2 = mpr().chisquare(n)
```

(continues on next page)

(continued from previous page)

```
tests_charfunc(rv2, x, a, b)
```

This produces the following output (plots are to be added):

```
Hello mpDistributions local !

rv2.cdf(x):  0.92476475385348782128

Integral:  -1.334418597712864291
result0: 0.92475863195951538339
diff0: -6.1218939724378892896e-6
I1: 0.11817924829245123086
I2: -1.4526170785024453884
I3: -1.3344378302099941575
Int diff: -0.00001923249712986651 1475
result3: 0.92476475385348782128
diff3: 8.4703294725430033907e-22
```

5.7 Mpmath: Numerical integration

5.7.1 General quadrature interface

`ctx.quad`(*f*, **points*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: Wikipedia [1119]

See also: MathWorld [801].

Computes a single, double or triple integral over a given 1D interval, 2D rectangle, or 3D cuboid. A basic example:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> quad(sin, [0, pi])
2.0
```

A basic 2D integral:

```
>>> f = lambda x, y: cos(x+y/2)
>>> quad(f, [-pi/2, pi/2], [0, pi])
4.0
```

Interval format

The integration range for each dimension may be specified using a list or tuple. Arguments are interpreted as follows:

`quad(f, [x1, x2])` – calculates $\int_{x_1}^{x_2} f(x)\,\mathrm{d}x$

`quad(f, [x1, x2], [y1, y2])` – calculates $\int_{x_1}^{x_2} \int_{y_1}^{y_2} f(x,y)\,\mathrm{d}y\,\mathrm{d}x$

`quad(f, [x1, x2], [y1, y2], [z1, z2])` – calculates $\int_{x_1}^{x_2} \int_{y_1}^{y_2} \int_{z_1}^{z_2} f(x,y,z)\,dz\,\mathrm{d}y\,\mathrm{d}x$

Endpoints may be finite or infinite. An interval descriptor may also contain more than two points. In this case, the integration is split into subintervals, between each pair of consecutive points. This is useful for dealing with mid-interval discontinuities, or integrating over large intervals where the function is irregular or oscillates.

Options

`quad()` recognizes the following keyword arguments:

method
Chooses integration algorithm (described below).

error
If set to true, `quad()` returns (v, e) where v is the integral and e is the estimated error.

maxdegree
Maximum degree of the quadrature rule to try before quitting.

verbose
Print details about progress.

Algorithms

Mpmath presently implements two integration algorithms: tanh-sinh quadrature and Gauss-Legendre quadrature. These can be selected using *method='tanh-sinh'* or *method='gauss-legendre'* or by passing the classes *method=TanhSinh*, *method=GaussLegendre*. The functions `quadts()` and `quadgl()` are also available as shortcuts.

Both algorithms have the property that doubling the number of evaluation points roughly doubles the accuracy, so both are ideal for high precision quadrature (hundreds or thousands of digits).

At high precision, computing the nodes and weights for the integration can be expensive (more expensive than computing the function values). To make repeated integrations fast, nodes are automatically cached.

The advantages of the tanh-sinh algorithm are that it tends to handle endpoint singularities well, and that the nodes are cheap to compute on the first run. For these reasons, it is used by `quad()` as the default algorithm.

Gauss-Legendre quadrature often requires fewer function evaluations, and is therefore often faster for repeated use, but the algorithm does not handle endpoint singularities as well and the nodes are more expensive to compute. Gauss-Legendre quadrature can be a better choice if the integrand is smooth and repeated integrations are required (e.g. for multiple integrals).

See the documentation for `TanhSinh` and `GaussLegendre` for additional details.

Examples of 1D integrals

Intervals may be infinite or half-infinite. The following two examples evaluate the limits of the inverse tangent function ($\int 1/(1+x^2) = \tan^{-1} x$), and the Gaussian integral $\int_{\infty}^{\infty} \exp(-x^2)\,dx = \sqrt{\pi}$:

```
>>> mp.dps = 15
>>> quad(lambda x: 2/(x**2+1), [0, inf])
3.14159265358979
>>> quad(lambda x: exp(-x**2), [-inf, inf])**2
3.14159265358979
```

Integrals can typically be resolved to high precision. The following computes 50 digits of π by integrating the area of the half-circle defined by $x^2 + y^2 \le 1$, $-1 \le x \le 1$, $y \ge 0$:

```
>>> mp.dps = 50
>>> 2*quad(lambda x: sqrt(1-x**2), [-1, 1])
3.1415926535897932384626433832795028841971693993751
```

One can just as well compute 1000 digits (output truncated):

```
>>> mp.dps = 1000
>>> 2*quad(lambda x: sqrt(1-x**2), [-1, 1])
3.14159265358979323846264338327950288419...216420199
```

Complex integrals are supported. The following computes a residue at $z = 0$ by integrating counterclockwise along the diamond-shaped path from 1 to $+i$ to -1 to $-i$ to 1:

```
>>> mp.dps = 15
>>> chop(quad(lambda z: 1/z, [1,j,-1,-j,1]))
(0.0 + 6.28318530717959j)
```

Examples of 2D and 3D integrals

Here are several nice examples of analytically solvable 2D integrals (taken from MathWorld [1]) that can be evaluated to high precision fairly rapidly by `quad()`:

```
>>> mp.dps = 30
>>> f = lambda x, y: (x-1)/((1-x*y)*log(x*y))
>>> quad(f, [0, 1], [0, 1])
0.577215664901532860606512090082
>>> +euler
0.577215664901532860606512090082

>>> f = lambda x, y: 1/sqrt(1+x**2+y**2)
>>> quad(f, [-1, 1], [-1, 1])
3.17343648530607134219175646705
>>> 4*log(2+sqrt(3))-2*pi/3
3.17343648530607134219175646705

>>> f = lambda x, y: 1/(1-x**2 * y**2)
>>> quad(f, [0, 1], [0, 1])
```

(continues on next page)

(continued from previous page)

```
1.2337005501361698273543113749 8
>>> pi**2 / 8
1.2337005501361698273543113749 8

>>> quad(lambda x, y: 1/(1-x*y), [0, 1], [0, 1])
1.6449340668482264364724151666 5
>>> pi**2 / 6
1.6449340668482264364724151666 5
```

Multiple integrals may be done over infinite ranges:

```
>>> mp.dps = 15
>>> print(quad(lambda x,y: exp(-x-y), [0, inf], [1, inf]))
0.367879441171442
>>> print(1/e)
0.367879441171442
```

For nonrectangular areas, one can call `quad()` recursively. For example, we can replicate the earlier example of calculating π by integrating over the unit-circle, and actually use double quadrature to actually measure the area circle:

```
>>> f = lambda x: quad(lambda y: 1, [-sqrt(1-x**2), sqrt(1-x**2)])
>>> quad(f, [-1, 1])
3.14159265358979
```

Here is a simple triple integral:

```
>>> mp.dps = 15
>>> f = lambda x,y,z: x*y/(1+z)
>>> quad(f, [0,1], [0,1], [1,2], method='gauss-legendre')
0.101366277027041
>>> (log(3)-log(2))/4
0.101366277027041
```

Singularities

Both tanh-sinh and Gauss-Legendre quadrature are designed to integrate smooth (infinitely differentiable) functions. Neither algorithm copes well with mid-interval singularities (such as mid-interval discontinuities in $f(x)$ or $f'(x)$). The best solution is to split the integral into parts:

```
>>> mp.dps = 15
>>> quad(lambda x: abs(sin(x)), [0, 2*pi])   # Bad
3.99900894176779
>>> quad(lambda x: abs(sin(x)), [0, pi, 2*pi])  # Good
4.0
```

The tanh-sinh rule often works well for integrands having a singularity at one or both endpoints:

```
>>> mp.dps = 15
>>> quad(log, [0, 1], method='tanh-sinh')  # Good
-1.0
>>> quad(log, [0, 1], method='gauss-legendre')  # Bad
-0.999932197413801
```

However, the result may still be inaccurate for some functions:

```
>>> quad(lambda x: 1/sqrt(x), [0, 1], method='tanh-sinh')
1.99999999946942
```

This problem is not due to the quadrature rule per se, but to numerical amplification of errors in the nodes. The problem can be circumvented by temporarily increasing the precision:

```
>>> mp.dps = 30
>>> a = quad(lambda x: 1/sqrt(x), [0, 1], method='tanh-sinh')
>>> mp.dps = 15
>>> +a
2.0
```

Highly variable functions

For functions that are smooth (in the sense of being infinitely differentiable) but contain sharp mid-interval peaks or many "bumps", `quad()` may fail to provide full accuracy. For example, with default settings, `quad()` is able to integrate $\sin(x)$ accurately over an interval of length 100 but not over length 1000:

```
>>> quad(sin, [0, 100]); 1-cos(100)   # Good
0.137681127712316
0.137681127712316
>>> quad(sin, [0, 1000]); 1-cos(1000)   # Bad
-37.8587612408485
0.437620923709297
```

One solution is to break the integration into 10 intervals of length 100:

```
>>> quad(sin, linspace(0, 1000, 10))   # Good
0.437620923709297
```

Another is to increase the degree of the quadrature:

```
>>> quad(sin, [0, 1000], maxdegree=10)   # Also good
0.437620923709297
```

Whether splitting the interval or increasing the degree is more efficient differs from case to case. Another example is the function $1/(1+x^2)$, which has a sharp peak centered around $x = 0$:

```
>>> f = lambda x: 1/(1+x**2)
>>> quad(f, [-100, 100])   # Bad
3.64804647105268
>>> quad(f, [-100, 100], maxdegree=10)   # Good
3.12159332021646
>>> quad(f, [-100, 0, 100])   # Also good
3.12159332021646
```

5.7.2 Quadrature with subdivision

`ctx.quadsubdiv`(*f*, *interval*, *tol=None*, *maxintervals=None*, ***kwargs*)

where `ctx` is `mpm` or `dec`.

Computes the integral of f over the interval or path specified by interval, using quad() together with adaptive subdivision of the interval.

This function gives an accurate answer for some integrals where quad() fails:

```
>>> ctx.dps = 15; ctx.pretty = True
>>> ctx.quad(lambda x: abs(ctx.sin(x)), [0, 2*ctx.pi])
3.99900894176779
>>> ctx.quadsubdiv(lambda x: abs(ctx.sin(x)), [0, 2*ctx.pi])
4.0
>>> ctx.quadsubdiv(ctx.sin, [0, 1000])
0.437620923709297
>>> ctx.quadsubdiv(lambda x: 1/(1+x**2), [-100, 100])
3.12159332021646
>>> ctx.quadsubdiv(lambda x: ctx.ceil(x), [0, 100])
5050.0
```

(continues on next page)

(continued from previous page)

```
>>> ctx.quadsubdiv(lambda x: ctx.sin(x+ctx.exp(x)), [0,8])
0.347400172657248
```

The argument *maxintervals* can be set to limit the permissible subdivision:

```
>>> ctx.quadsubdiv(lambda x: ctx.sin(x**2), [0,100], maxintervals=5, error=True)
(-5.40487904307774, 5.011)
>>> ctx.quadsubdiv(lambda x: ctx.sin(x**2), [0,100], maxintervals=100, error=True)
(0.631417921866934, 1.10101120134116e-17)
```

Subdivision does not guarantee a correct answer, since the error estimate on subintervals may be inaccurate:

```
>>> ctx.quadsubdiv(lambda x: ctx.sech(10*x-2)**2 + ctx.sech(100*x-40)**4 \
... + ctx.sech(1000*x-600)**6, [0,1], error=True)
(0.210802735500549, 1.0001111101e-17)
>>> ctx.dps = 20
>>> ctx.quadsubdiv(lambda x: ctx.sech(10*x-2)**2 + ctx.sech(100*x-40)**4 \
... + ctx.sech(1000*x-600)**6, [0,1], error=True)
(0.21080273550054927738, 2.200000001e-24)
```

The second answer is correct. We can get an accurate result at lower precision by forcing a finer initial subdivision:

```
>>> ctx.dps = 15
>>> ctx.quadsubdiv(lambda x: ctx.sech(10*x-2)**2 + ctx.sech(100*x-40)**4 \
... + ctx.sech(1000*x-600)**6, ctx.linspace(0,1,5))
0.210802735500549
```

The following integral is too oscillatory for convergence, but we can get a reasonable estimate:

```
>>> v, err = ctx.quadsubdiv(lambda x: ctx.sin(1/x), [0,1], error=True)
>>> round(v, 6), round(err, 6)
(0.504067, 1e-06)
>>> ctx.sin(1) - ctx.ci(1)
0.504067061906928
```

5.7.3 Quadrature of oscillatory functions (Filon)

`ctx.quadosc`(*f*, *interval*, *omega=None*, *period=None*, *zeros=None*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also Filon's rule: MathWorld [800]

Calculates

$$I = \int_a^b f(x)\mathrm{d}x$$

where at least one of a and b is infinite and where $f(x) = g(x)\cos(\omega x + \phi)$ for some slowly decreasing function $g(x)$. With proper input, `quadosc()` can also handle oscillatory integrals where the oscillation rate is different from a pure sine or cosine wave.

In the standard case when $|a| < \infty, b = \infty$, `quadosc()` works by evaluating the infinite series

$$I = \int_a^{x_1} f(x)\mathrm{d}x + \sum_{k=1}^{\infty} \int_{x_k}^{x_{k+1}} f(x)\mathrm{d}x$$

where x_k are consecutive zeros (alternatively some other periodic reference point) of $f(x)$. Accordingly, `quadosc()` requires information about the zeros of $f(x)$. For a periodic function, you can specify the zeros by either providing the angular frequency ω (*omega*) or the *period* $2\pi/\omega$. In general, you can specify the n-th zero by providing the *zeros* arguments. Below is an example of each:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> f = lambda x: sin(3*x)/(x**2+1)
>>> quadosc(f, [0,inf], omega=3)
0.37833007080198
>>> quadosc(f, [0,inf], period=2*pi/3)
0.37833007080198
>>> quadosc(f, [0,inf], zeros=lambda n: pi*n/3)
0.37833007080198
>>> (ei(3)*exp(-3)-exp(3)*ei(-3))/2  # Computed by Mathematica
0.37833007080198
```

Note that *zeros* was specified to multiply n by the *half-period*, not the full period. In theory, it does not matter whether each partial integral is done over a half period or a full period. However, if done over half-periods, the infinite series passed to `nsum()` becomes an *alternating series* and this typically makes the extrapolation much more efficient.

Here is an example of an integration over the entire real line, and a half-infinite integration starting at $-\infty$:

```
>>> quadosc(lambda x: cos(x)/(1+x**2), [-inf, inf], omega=1)
1.15572734979092
>>> pi/e
1.15572734979092
>>> quadosc(lambda x: cos(x)/x**2, [-inf, -1], period=2*pi)
-0.0844109505595739
>>> cos(1)+si(1)-pi/2
-0.0844109505595738
```

Of course, the integrand may contain a complex exponential just as well as a real sine or cosine:

```
>>> quadosc(lambda x: exp(3*j*x)/(1+x**2), [-inf,inf], omega=3)
(0.156410688228254 + 0.0j)
>>> pi/e**3
0.156410688228254
>>> quadosc(lambda x: exp(3*j*x)/(2+x+x**2), [-inf,inf], omega=3)
(0.00317486988463794 - 0.0447701735209082j)
>>> 2*pi/sqrt(7)/exp(3*(j+sqrt(7))/2)
(0.00317486988463794 - 0.0447701735209082j)
```

Non-periodic functions

If $f(x) = g(x)h(x)$ for some function $h(x)$ that is not strictly periodic, *omega* or *period* might not work, and it might be necessary to use *zeros*.

A notable exception can be made for Bessel functions which, though not periodic, are "asymptotically periodic" in a sufficiently strong sense that the sum extrapolation will work out:

```
>>> quadosc(j0, [0, inf], period=2*pi)
1.0
>>> quadosc(j1, [0, inf], period=2*pi)
1.0
```

More properly, one should provide the exact Bessel function zeros:

```
>>> j0zero = lambda n: findroot(j0, pi*(n-0.25))
>>> quadosc(j0, [0, inf], zeros=j0zero)
1.0
```

For an example where *zeros* becomes necessary, consider the complete Fresnel integrals

$$\int_0^\infty \cos x^2 \, dx = \int_0^\infty \sin x^2 \, dx = \sqrt{\frac{\pi}{8}}.$$

Although the integrands do not decrease in magnitude as $x \to \infty$, the integrals are convergent since the oscillation rate increases (causing consecutive periods to asymptotically cancel out). These integrals are virtually impossible to calculate to any kind of

accuracy using standard quadrature rules. However, if one provides the correct asymptotic distribution of zeros ($x_n \sim \sqrt{n}$), `quadosc()` works:

```
>>> mp.dps = 30
>>> f = lambda x: cos(x**2)
>>> quadosc(f, [0,inf], zeros=lambda n:sqrt(pi*n))
0.626657068657750125603941321203
>>> f = lambda x: sin(x**2)
>>> quadosc(f, [0,inf], zeros=lambda n:sqrt(pi*n))
0.626657068657750125603941321203
>>> sqrt(pi/8)
0.626657068657750125603941321203
```

(Interestingly, these integrals can still be evaluated if one places some other constant than π in the square root sign.)

In general, if $f(x) \sim g(x)\cos(h(x))$, the zeros follow the inverse-function distribution $h^{-1}(x)$:

```
>>> mp.dps = 15
>>> f = lambda x: sin(exp(x))
>>> quadosc(f, [1,inf], zeros=lambda n: log(n))
-0.25024394235267
>>> pi/2-si(e)
-0.250243942352671
```

Non-alternating functions

If the integrand oscillates around a positive value, without alternating signs, the extrapolation might fail. A simple trick that sometimes works is to multiply or divide the frequency by 2:

```
>>> f = lambda x: 1/x**2+sin(x)/x**4
>>> quadosc(f, [1,inf], omega=1)  # Bad
1.28642190869861
>>> quadosc(f, [1,inf], omega=0.5)  # Perfect
1.28652953559617
>>> 1+(cos(1)+ci(1)+sin(1))/6
1.28652953559617
```

Fast decay

`quadosc()` is primarily useful for slowly decaying integrands. If the integrand decreases exponentially or faster, `quad()` will likely handle it without trouble (and generally be much faster than `quadosc()`):

```
>>> quadosc(lambda x: cos(x)/exp(x), [0, inf], omega=1)
0.5
>>> quad(lambda x: cos(x)/exp(x), [0, inf])
0.5
```

5.7.4 Verified numerical integration (Okayama)

quad_verified(*f*, **points*, ***kwargs*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Computes a single integral over a given 1D interval, providing a guaranteed error bound.

This function implements an efficient algorithm for 1-dimensional numerical real integration with rigorous error bounds. See Petras [467], Petras [468] for an overview. The algorithms used in this section have been described in detail in a series of papers by Okayama *et al.* [430], Okayama [429] and Okayama [431], where explicit error bounds for the double exponential formulas proposed by Takahasi and Mori [511] are provided.

See also https://github.com/fredrik-johansson/arb/issues/196 (DH comment on integration, contains better description).

The type of integrals considered is as follows:

$$I = \int_a^b f(x)\mathrm{d}x = \int_a^b \frac{g(x)}{(x-a)^{1-\alpha}(b-x)^{1-\beta}}\mathrm{d}x, \quad \text{where}$$

$$g(x) = f(x)(x-a)^{\alpha-1}(b-x)^{\beta-1}$$

and α and β are positive constants. We assume that the integrand $f(x)$ is a function on an open interval (a, b) and may have an integrable algebraic singularity of the type shown above at the end-points $x = a$ and/or $x = b$. The function f (or the function g) needs to be analytic on the following complex domain $\mathcal{D}$:

$$\mathcal{D} = \left\{ z \in \mathbb{C} : \left| \arg\left[\frac{1}{\pi}\log\left(\frac{z-a}{b-z}\right) + \sqrt{1 + \left(\frac{1}{\pi}\log\left(\frac{z-a}{b-z}\right)\right)^2} \right] \right| < d \right\}$$

where $0 < d < \pi/2$, and satisfy, for some positive constant K, for all $z \in \mathcal{D}$ the condition

$$|f(z)| \leq\ K|z-a|^{\alpha-1}|b-z|^{\beta-1}, \quad \text{or, equivalently,}$$

$$|g(z)| \leq\ K$$

A rigorous upper bound for K, given d (which can be freely chosen observing the condition $0 < d < \pi/2$) can be computed using complex interval arithmetic; details will be discussed below.

For a given choice of d and a positive constant ϵ_{abs}, we have the following key result, which establishes an explicit rigorous bound for the absolute error of the approximation to the integral, provided that all calculations are carried out in real interval arithmetic:

$$\frac{1}{C_1}\left| \int_a^b f(x)\mathrm{d}x - h\sum_{k=-M}^{N} g(\phi(kh))P(kh) \right| \leq \epsilon_{abs}, \quad \text{where}$$

$$\phi(t) = \tanh\left(\frac{\pi}{2}\sinh(t)\right), \quad \text{and}$$

$$P(u) = \frac{\pi}{2}\left(\frac{b-a}{2}\right)^{\alpha+\beta-1}\cosh(u)(1+\phi(u))^{\alpha}(1-\phi(u))^{\beta}$$

Note that in formula ref{eq:KeyIntegrationformula} and in the subsequent formulas C_1, C_2, h, M, N only depend on d and ϵ_{abs} and the function parameters α and β. The number of intervals and the interval width h are automatically chosen in a way which balances the discretization error and the truncation error, resulting in a "nearly optimal" number of function evaluations. C_1, C_2, h, M, N are calculated as follows:

$$C_1 = \frac{2K(b-a)^{\alpha+\beta-1}}{\min(\alpha,\beta)}$$

$$C_2 = \frac{2}{\cos^{\alpha+\beta}\left(\frac{\pi}{2}\sin(d)\right)\cos(d)}$$

$$h = \frac{2\pi d}{\log\left(1+\frac{2C_2}{\epsilon_{abs}}\right)}$$

$$n = \left\lceil \frac{1}{h}\log\left(\frac{2}{\pi\min(\alpha,\beta)}\log\left(\frac{2e^{\max(\alpha,\beta)\pi/2}}{\epsilon_{abs}}\right)\right)\right\rceil$$

$$\begin{cases} M = n, & N = n - \lfloor \log(\beta/\alpha)/h \rfloor, \quad \text{for } \alpha \leq \beta \\ N = n, & M = n - \lfloor \log(\alpha/\beta)/h \rfloor, \quad \text{for } \alpha > \beta \end{cases}$$

The main challenge in implementing the algorithm is to determine a rigorous upper bound for K. We are going to achieve this by using the complex interval arithmetic which is implemented in mpmath. Since the domain $\mathcal{D}$ is not of rectangular shape, we need to compute a rectangular domain, say $\mathcal{D}_{\text{Rect}}$, which includes $\mathcal{D}$: $\mathcal{D} \subset \mathcal{D}_{\text{Rect}}$.

The rectangular domain $\mathcal{D}_{\text{Rect}}$ corresponds to a complex interval, say z_1. The function g is then evaluated in complex interval arithmetic with z_1 as argument: $z_2 = g(z_1)$. The absolute value of z_2 is a real interval, say x_1. The supremum of x_1 is a rigorous upper bound of K (see equation ref{eq:BoundFor_K_G}).

The choice of d has a great impact on $M + N + 1$, the total number of function evaluations, which are required to achieve the desired absolute error. To be able to select a near-optimal value of d resulting in the lowest number of function evaluations, we calculate n for a number of values for d, ranging from $d = 1.5$ to $d = 0.1$, using pre-computed values for the matching rectangular domains $\mathcal{D}_{\text{Rect}}$, and choose the value for d which results in the smallest n.

If equation ref{eq:gFormula} does not give a useful alternative representation of f, we set $\alpha = \beta = 1$, so that $f(x) = g(x)$. However, taking a, b, $\alpha = \frac{1}{2}$ and $\beta = 1$ let us consider the integral

$$I = \int_0^1 \frac{\sin(\exp(x))}{\sqrt{x}}\mathrm{d}x = \int_0^1 f(x)\mathrm{d}x = \int_0^1 \frac{g(x)}{\sqrt{x}}\mathrm{d}x, \text{ i.e}$$

$$f = \frac{\sin(\exp(x))}{\sqrt{x}} \text{ and } g = \sin(\exp(x))$$

In this situation, equation ref{eq:BoundFor_K_F} cannot be used to determine K because $f(z)$ has a singularity at $z = 0$; on the other hand, equation ref{eq:BoundFor_K_G} can still be used because $g(z)$ has no singularities in the interval $(0, 1)$.

It is possible to generalize this methodology to integrals for which $a = -\infty$ and/or $b = +\infty$; details can be found in Okayama (2014). Unfortunately, the resulting domains are of a shape which makes it impossible to include them in a rectangular domain, unless this domain spans the entire complex plane. Therefore, except for very few, extremely well behaved functions, the constant K cannot be determined using complex interval arithmetic (but can possibly be determined by analytical methods). The usual transformation formulas, which are widely used when performing non-verified numerical integration, are not applicable, because they will (almost always) produce a singularity at one of the endpoints, which will prevent the determination of the constant K .

5.7.5 Specialized Gauss quadrature rules

`ctx.gauss_quad_rules`(*ctx*, *n*, *qtype='legendre'*, *alpha=0*, *beta=0*)

where `ctx` is `dec`, `mpm`, or `gmp`.

This routine calulates Gaussian quadrature rules for different families of orthogonal polynomials. Let (a, b) be an interval, W(x) a positive weight function and n a positive integer.

Then the purpose of this routine is to calculate pairs (x_k, w_k) for $k = 0, 1, 2, ...(n-1)$ which give

$$\int_a^b W(x) * F(x)\mathrm{d}x = \sum_0^{n-1} w_k * F(x_k)$$

exact for all polynomials F(x) of degree (strictly) less than 2*n. For all integrable functions F(x) the sum is (more or less) good approximation to the integral. The x_k are called nodes (which are the zeros of the related orthogonal polynomials) and the w_k are called the weights.

```
parameters
    n        (input) The degree of the quadrature rule, i.e. its number of
             nodes.

    qtype    (input) The family of orthogonal polynmomials for which to
             compute the quadrature rule. See the list below.

    alpha    (input) real number, used as parameter for some orthogonal
             polynomials
```

(continues on next page)

(continued from previous page)

```
    beta     (input) real number, used as parameter for some orthogonal
             polynomials.

return value

    (X, W)    a pair of two real arrays where x_k = X[k] and w_k = W[k].


orthogonal polynomials:

    qtype           polynomial
    -----           ----------

    "legendre"      Legendre polynomials, W(x)=1 on the interval (-1, +1)
    "legendre01"    shifted Legendre polynomials, W(x)=1 on the interval (0, +1)
    "hermite"       Hermite polynomials, W(x)=exp(-x*x) on (-infinity,+infinity)
    "laguerre"      Laguerre polynomials, W(x)=exp(-x) on (0,+infinity)
    "glaguerre"     generalized Laguerre polynomials, W(x)=exp(-x)*x**alpha on (0, +infinity)
    "chebyshev1"    Chebyshev polynomials of the first kind, W(x)=1/sqrt(1-x*x) on (-1, +1)
    "chebyshev2"    Chebyshev polynomials of the second kind, W(x)=sqrt(1-x*x) on (-1, +1)
    "jacobi"        Jacobi polynomials, W(x)=(1-x)**alpha * (1+x)**beta on (-1, +1)
                    with alpha>-1 and beta>-1

references:
  - golub and welsch, "calculations of gaussian quadrature rules", mathematics of
    computation 23, p. 221-230 (1969)
  - golub, "some modified matrix eigenvalue problems", siam review 15, p. 318-334 (1973)
  - stroud and secrest, "gaussian quadrature formulas", prentice-hall (1966)

See also the routine gaussq.f in netlog.org or ACM Transactions on
Mathematical Software algorithm 726.
```

Gauss-Chebyshev quadrature, first and second kind

See also: MathWorld [805], Wikipedia [1111], Gatteschi [344], Petras [467], Petras [468].

See also: https://en.wikipedia.org/wiki/Chebyshev%E2%80%93Gauss_quadrature

See also: https://mathworld.wolfram.com/Chebyshev-GaussQuadrature.html

See also: Abramowitz, p. 383

For the first kind:

$$\int_{-1}^{1} \frac{f(x)}{\sqrt{1-x^2}}\mathrm{d}x = \sum_{i=1}^{n} w_i f(x_i) + R_n, \quad \text{where}$$

$$x_i = \cos\left(\frac{2i-1}{2n}\pi\right), w_i = \frac{\pi}{n}, \quad R_n = \frac{f^{(2n)}(\xi)\pi}{(2n)!2^{2n-1}} \text{ for } -1 < \xi < 1.$$

For the second kind:

$$\int_{-1}^{1} \sqrt{1-x^2} g(x)\mathrm{d}x = \sum_{i=1}^{n} w_i g(x_i) + R_n, \quad \text{where}$$

$$x_i = \cos\left(\frac{i}{n+1}\pi\right), w_i = \frac{\pi}{n+1}\sin^2\left(\frac{i}{n+1}\pi\right), \quad R_n = \frac{g^{(2n)}(\xi)\pi}{(2n)!2^{2n+1}} \text{ for } -1 < \xi < 1.$$

```
def demo_gauss_quadrature_chebyshev(ctx):
    # orthogonality of the chebyshev polynomials:
    f = lambda x: ctx.chebyt(3, x) * ctx.chebyt(2, x) # problem with gpm.chebyt
```

(continues on next page)

(continued from previous page)

```
    X, W = gauss_quadrature(ctx, 3, "chebyshev1")
    A = ctx.fdot([(f(x), w) for x, w in zip(X, W)])
    print("A:", A)
    print(ctx.chop(A, tol = 1e-10))
```

Gauss-Legendre quadrature

See also: MathWorld [806], Wikipedia [1112], Petras [467], Petras [468].

$$\int_a^b f(x)\mathrm{d}x = \frac{b-a}{2}\sum_{i=1}^{n} w_i f\left(\tfrac{1}{2}(b-a)(x_i+1)\right) + R_n, \quad \text{where}$$

$$w_i = \frac{2}{((1-x^2)P_n'(x_i))^2}, \quad R_n = \frac{f^{(2n)}(\xi)(b-a)^{2n+1}(n!)^4}{((2n+1)(2n)!)^3} \text{ for } a < \xi < b,$$

P_n are the Legendre polynomials of degree n, and x_i is the i^{th} zero of P_n.

```
def demo_gauss_quadrature_laguerre_mp():
    from xlcalcnet.mpmath import mp
    f = lambda x: x**5 - 2 * x**4 + 3 * x**3 - 5 * x**2 + 7 * x - 11
    X, W = mp.gauss_quadrature(3, "laguerre")
    A = mp.fdot([(f(x), w) for x, w in zip(X, W)])
    B = 76
    C = mp.quad(lambda x: mp.exp(-x) * f(x), [0, +mp.inf])
    print("A:", A)
    print("B:", B)
    print("C:", C)
    print(mp.chop(A-B, tol = 1e-10), mp.chop(A-C, tol = 1e-10))
```

Gauss-Jacobi quadrature

See also: MathWorld [804], Wikipedia [1109], Petras [467], Petras [468].

$$\int_a^b f(x)(1-x)^\alpha(1+x)^\beta\mathrm{d}x = \frac{b-a}{2}\sum_{i=1}^{n} w_i f\left(\tfrac{1}{2}(b-a)(x_i+1)\right) + R_n, \quad \text{where } \alpha,\beta > -1,$$

$$w_i = -\frac{2n+\alpha+\beta+2}{n+\alpha+\beta+1}\frac{\Gamma(n+\alpha+1)\Gamma(n+\beta+1)}{\Gamma(n+\alpha+\beta+1)(n+1)!}\frac{2^{\alpha+\beta}}{P_n^{(\alpha,\beta)\prime}(x_i)P_{n+1}^{(\alpha,\beta)}(x_i)},$$

$$R_n = \frac{\Gamma(n+\alpha+1)\Gamma(n+\beta+1)\Gamma(n+\alpha+\beta+1)}{(2n+\alpha+\beta+1)[\Gamma(2n+\alpha+\beta+1)]^2}\frac{2^{2+\alpha+\beta+1}}{(2n)!}f^{(2n)}(\xi), \text{ for } a < \xi < b,$$

$P_n^{(\alpha,\beta)}$ are the Jacobi polynomials of degree n, and x_i is the i^{th} zero of $P_n^{(\alpha,\beta)}$.

Gauss-Legendre quadrature is a special case of Gauss-Jacobi quadrature with $\alpha = \beta = 0$. Similarly, the Chebyshev-Gauss quadrature of the first (second) kind arises when one takes $\alpha = \beta = -0.5(+0.5)$. The special case $\alpha = \beta$ turns Jacobi polynomials into Gegenbauer polynomials, in which case the technique is sometimes called Gauss-Gegenbauer quadrature. Gauss-Jacobi quadrature can be used to approximate integrals with singularities at the end points.

```
def demo_gauss_quadrature_laguerre_mp():
    from xlcalcnet.mpmath import mp
    f = lambda x: x**5 - 2 * x**4 + 3 * x**3 - 5 * x**2 + 7 * x - 11
    X, W = mp.gauss_quadrature(3, "laguerre")
    A = mp.fdot([(f(x), w) for x, w in zip(X, W)])
    B = 76
    C = mp.quad(lambda x: mp.exp(-x) * f(x), [0, +mp.inf])
    print("A:", A)
    print("B:", B)
    print("C:", C)
    print(mp.chop(A-B, tol = 1e-10), mp.chop(A-C, tol = 1e-10))
```

Gauss-Laguerre quadrature, classic and generalized

See also: MathWorld [805], Wikipedia [1111], Gatteschi [344], Petras [467], Petras [468].

$$\int_0^\infty e^{-ax} f(x)\mathrm{d}x = \frac{1}{a}\sum_{i=1}^{n} w_i f\left(\frac{x_i}{a}\right) + R_n, \quad \text{where}$$

$$w_i = \frac{x_i}{((n+1)L_{n+1}(x_i))^2}, \quad R_n = \frac{f^{(2n)}(\xi)(n!)^2}{(2n)!} \text{ for } 0 < \xi < \infty,$$

L_n are the Laguerre polynomials of degree n, and x_i is the i^{th} zero of L_n.

```
def demo_gauss_quadrature_laguerre_mp():
    from xlcalcnet.mpmath import mp
    f = lambda x: x**5 - 2 * x**4 + 3 * x**3 - 5 * x**2 + 7 * x - 11
    X, W = mp.gauss_quadrature(3, "laguerre")
    A = mp.fdot([(f(x), w) for x, w in zip(X, W)])
    B = 76
    C = mp.quad(lambda x: mp.exp(-x) * f(x), [0, +mp.inf])
    print("A:", A)
    print("B:", B)
    print("C:", C)
    print(mp.chop(A-B, tol = 1e-10), mp.chop(A-C, tol = 1e-10))
```

Gauss-Hermite quadrature

See also: MathWorld [803], Wikipedia [1108], Townsend *et al.* [786], Petras [467], Petras [468].

$$\int_{-\infty}^\infty e^{-ax^2} f(x)\mathrm{d}x = \frac{1}{\sqrt{a}}\sum_{i=1}^{n} w_i f\left(\frac{x_i}{\sqrt{a}}\right) + R_n, \quad \text{where}$$

$$w_i = \frac{2^{n-1}n!\sqrt{\pi}}{(nH_{n-1}(x_i))^2}, \quad R_n = \frac{f^{(2n)}(\xi)n!\sqrt{\pi}}{2^n(2n)!} \text{ for } -\infty < \xi < \infty,$$

H_n are the Hermite polynomials of degree n, and x_i is the i^{th} zero of H_n.

```
def demo_gauss_quadrature_hermite_mp():
    from xlcalcnet.mpmath import mp
    f = lambda x: x**8 + 2 * x**6 - 3 * x**4 + 5 * x**2 - 7
    X, W = mp.gauss_quadrature(5, "hermite")
    A = mp.fdot([(f(x), w) for x, w in zip(X, W)])
    B = mp.sqrt(mp.pi) * 57 / 16
    C = mp.quad(lambda x: mp.exp(- x * x) * f(x), [-mp.inf, +mp.inf])
    print("A:", A)
    print("B:", B)
    print("C:", C)
    print(mp.chop(A-B, tol = 1e-10), mp.chop(A-C, tol = 1e-10))
```

5.8 Mpmath: Numerical inverse Laplace transform

5.8.1 General inverse Laplace transform interface

`ctx.invertlaplace`(*f*, *t*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: Cohen [181], Duffy [291], Bellman *et al.* [28], Davies and Martin [201], Duffy [290], Kuhlman [400].

Computes the numerical inverse Laplace transform for a Laplace-space function at a given time. The function being evaluated is assumed to be a real-valued function of time.

The user must supply a Laplace-space function $\bar{f}(p)$, and a desired time at which to estimate the time-domain solution $f(t)$.

Options

`invertlaplace()` recognizes the following optional keywords valid for all methods:

method
Chooses numerical inverse Laplace transform algorithm (described below).

degree
Number of terms used in the approximation

Basic examples

A few basic examples of Laplace-space functions with known inverses (see Abate and Valko [1], Talbot [512]) :

$$\mathcal{L}\left\{f(t)\right\} = \bar{f}(p)$$

$$\mathcal{L}^{-1}\left\{\bar{f}(p)\right\} = f(t)$$

$$\bar{f}(p) = \frac{1}{(p+1)^2}$$

$$f(t) = te^{-t}$$

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> tt = [0.001, 0.01, 0.1, 1, 10]
>>> fp = lambda p: 1/(p+1)**2
>>> ft = lambda t: t*exp(-t)
>>> ft(tt[0]),ft(tt[0])-invertlaplace(fp,tt[0],method='talbot')
(0.000999000499833375, 8.57923043561212e-20)
>>> ft(tt[1]),ft(tt[1])-invertlaplace(fp,tt[1],method='talbot')
(0.00990049833749168, 3.27007646698047e-19)
>>> ft(tt[2]),ft(tt[2])-invertlaplace(fp,tt[2],method='talbot')
(0.090483741803596, -1.75215800052168e-18)
>>> ft(tt[3]),ft(tt[3])-invertlaplace(fp,tt[3],method='talbot')
(0.367879441171442, 1.2428864009344e-17)
>>> ft(tt[4]),ft(tt[4])-invertlaplace(fp,tt[4],method='talbot')
(0.000453999297624849, 4.04513489306658e-20)
```

The methods also work for higher precision:

```
>>> mp.dps = 100; mp.pretty = True
>>> nstr(ft(tt[0]),15),nstr(ft(tt[0])-invertlaplace(fp,tt[0],method='talbot'),15)
('0.000999000499833375', '-4.96868310693356e-105')
>>> nstr(ft(tt[1]),15),nstr(ft(tt[1])-invertlaplace(fp,tt[1],method='talbot'),15)
('0.00990049833749168', '1.23032291513122e-104')
```

$$\bar{f}(p) = \frac{1}{p^2+1}$$

$$f(t) = \mathrm{J}_0(t)$$

```
>>> mp.dps = 15; mp.pretty = True
>>> fp = lambda p: 1/sqrt(p*p + 1)
>>> ft = lambda t: besselj(0,t)
>>> ft(tt[0]),ft(tt[0])-invertlaplace(fp,tt[0])
(0.999999750000016, -8.2477943034014e-18)
>>> ft(tt[1]),ft(tt[1])-invertlaplace(fp,tt[1])
(0.99997500015625, -3.69810144898872e-17)
```

$$\bar{f}(p) = \frac{\log p}{p}$$

$$f(t) = -\gamma - \log t$$

```
>>> mp.dps = 15; mp.pretty = True
>>> fp = lambda p: log(p)/p
>>> ft = lambda t: -euler-log(t)
>>> ft(tt[0]),ft(tt[0])-invertlaplace(fp,tt[0],method='stehfest')
(6.3305396140806, -1.92126634837863e-16)
>>> ft(tt[1]),ft(tt[1])-invertlaplace(fp,tt[1],method='stehfest')
(4.02795452108656, -4.81486093200704e-16)
```

Relationship to distribution functions

The concept of Laplace transforms also finds use in the context of statistical distribution functions, although here often the moment generating function $M_X(t)$ is given instead of the Laplace transform $\mathcal{L}\{f(t)\}$. They are related by $\mathcal{L}\{f(t)\} = M_X(-t)$. Note that in the case of statistical distribution functions, neither $M_X(t)$ nor $\mathcal{L}\{f(t)\}$ are guaranteed to exist. If they do, however, they can be used to calculate the pdf and cdf, particularly if the domain of X is $X > 0$.

The Laplace transform (of the pdf)of a nonnegative RV uniquely determines the distribution. Applying the inverse Laplace transform to the Laplace transform of the distribution gives the pdf.

These two transformations, $X \longrightarrow e^{-sX}$ provides the definition of a Laplace transformation:

$$\mathcal{L}[f](s) := \mathrm{E}^{\mathbb{P}}\left[e^{-sX}\right] = \int_{\mathbb{R}} e^{-sx} f_X(x)\mathrm{d}x.$$

Therefore, given a (nonnegative) random variable X and its associated probability density (PDF) $f_X(x)$, we always have the Laplace transform of that density $\mathcal{L}[f_X](s)$ defined at s . But note that this also requires a transformation of the parameter x in to $-s$.

The Laplace transform finds the CDF in the transformed variable s easily:

$$F_X(x) = \mathcal{L}^{-1}\left[\frac{1}{s}\,\mathrm{E}^{\mathbb{P}}\left[e^{-sX}\right]\right](x).$$

Singularities

All numerical inverse Laplace transform methods have problems at large time when the Laplace-space function has poles, singularities, or branch cuts to the right of the origin in the complex plane. For simple poles in $\bar{f}(p)$ at the p-plane origin, the time function is constant in time (e.g., $\mathcal{L}\{1\} = 1/p$ has a pole at $p = 0$). A pole in $\bar{f}(p)$ to the left of the origin is a decreasing function of time (e.g., $\mathcal{L}\left\{e^{-t/2}\right\} = 1/(p + 1/2)$ has a pole at $p = -1/2$), and a pole to the right of the origin leads to an increasing function in time (e.g., $\mathcal{L}\left\{te^{t/4}\right\} = 1/(p - 1/4)^2$ has a pole at $p = 1/4$). When singularities occur off the real p axis, the time-domain function is oscillatory. For example $\mathcal{L}\{\mathrm{J}_0(t)\} = 1/\sqrt{p^2 + 1}$ has a branch cut starting at $p = j = \sqrt{-1}$ and is a decaying oscillatory function, This range of behaviors is illustrated in Duffy [3] Figure 4.10.4, p. 228.

In general as $p \to \infty$ $t \to 0$ and vice-versa. All numerical inverse Laplace transform methods require their abscissa to shift closer to the origin for larger times. If the abscissa shift left of the rightmost singularity in the Laplace domain, the answer will be completely wrong (the effect of singularities to the right of the Bromwich contour are not included in the results).

For example, the following exponentially growing function has a pole at $p = 3$:

$$\bar{f}(p) = \frac{1}{p^2 - 9}$$

$$f(t) = \frac{1}{3}\sinh 3t$$

```
>>> mp.dps = 15; mp.pretty = True
>>> fp = lambda p: 1/(p*p-9)
>>> ft = lambda t: sinh(3*t)/3
>>> tt = [0.01,0.1,1.0,10.0]
>>> ft(tt[0]),invertlaplace(fp,tt[0],method='talbot')
(0.0100015000675014, 0.0100015000675014)
>>> ft(tt[1]),invertlaplace(fp,tt[1],method='talbot')
(0.101506764482381, 0.101506764482381)
>>> ft(tt[2]),invertlaplace(fp,tt[2],method='talbot')
(3.33929164246997, 3.33929164246997)
>>> ft(tt[3]),invertlaplace(fp,tt[3],method='talbot')
(1781079096920.74, -1.61331069624091e-14)
```

Algorithms

Mpmath implements three numerical inverse Laplace transform algorithms, attributed to: Talbot, Stehfest, and de Hoog, Knight and Stokes. These can be selected by using *method='talbot'*, *method='stehfest'*, *method='dehoog'*, or *method='cohen'* The functions `mpmath.invlaptalbot`, `mpmath.invlapstehfest`, and `mpmath.invlapdehoog` and `mpmath.invlapcohen` are also available as shortcuts.

All four algorithms implement a heuristic balance between the requested precision and the precision used internally for the calculations. This has been tuned for a typical exponentially decaying function and precision up to few hundred decimal digits.

The Laplace transform converts the variable time (i.e., along a line) into a parameter given by the right half of the complex p-plane. Singularities, poles, and branch cuts in the complex p-plane contain all the information regarding the time behavior of the corresponding function. Any numerical method must therefore sample p-plane "close enough" to the singularities to accurately characterize them, while not getting too close to have catastrophic cancellation, overflow, or underflow issues. Most significantly, if one or more of the singularities in the p-plane is not on the left side of the Bromwich contour, its effects will be left out of the computed solution, and the answer will be completely wrong.

5.8.2 Talbot method

`ctx.`**`invlaptalbot`**(*f*, *t*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: Abate and Valko [1], Talbot [512].

The fixed Talbot method is high accuracy and fast, but the method can catastrophically fail for certain classes of time-domain behavior, including a Heaviside step function for positive time (e.g., $H(t-2)$), or some oscillatory behaviors. The Talbot method usually has adjustable parameters, but the "fixed" variety implemented here does not. This method deforms the Bromwich integral contour in the shape of a parabola towards $-\infty$, which leads to problems when the solution has a decaying exponential in it (e.g., a Heaviside step function is equivalent to multiplying by a decaying exponential in Laplace space).

`def calc_laplace_parameter(self,t,**kwargs)`

The "fixed" Talbot method deforms the Bromwich contour towards $-\infty$ in the shape of a parabola. Traditionally the Talbot algorithm has adjustable parameters, but the "fixed" version does not. The r parameter could be passed in as a parameter, if you want to override the default given by (Abate & Valko, 2004).

The Laplace parameter is sampled along a parabola opening along the negative imaginary axis, with the base of the parabola along the real axis at $p = \frac{r}{t_{\max}}$. As the number of terms used in the approximation (degree) grows, the abscissa required for function evaluation tend towards $-\infty$, requiring high precision to prevent overflow. If any poles, branch cuts or other singularities exist such that the deformed Bromwich contour lies to the left of the singularity, the method will fail.

Optional arguments

`calc_laplace_parameter` recognizes the following keywords

tmax
: maximum time associated with vector of times (typically just the time requested)

degree
integer order of approximation (M = number of terms)

r
abscissa for p_0 (otherwise computed using rule of thumb $2M/5$)

The working precision will be increased according to a rule of thumb. If 'degree' is not specified, the working precision and degree are chosen to hopefully achieve the dps of the calling context. If 'degree' is specified, the working precision is chosen to achieve maximum resulting precision for the specified degree.

$$p_0 = \frac{r}{t}$$

$$p_i = \frac{ir\pi}{Mt_{\max}}\left[\cot\left(\frac{i\pi}{M}\right) + j\right] \qquad 1 \le i < M$$

where $j = \sqrt{-1}$, $r = 2M/5$, and $t_{\max}$ is the maximum specified time.

`def calc_time_domain_solution(self,fp,t,manual_prec=False)`

The fixed Talbot time-domain solution is computed from the Laplace-space function evaluations using

$$f(t,M) = \frac{2}{5t}\sum_{k=0}^{M-1}\Re\left[\gamma_k \bar{f}(p_k)\right]$$

where

$$\gamma_0 = \frac{1}{2}e^r \bar{f}(p_0)$$

$$\gamma_k = e^{tp_k}\left\{1 + \frac{jk\pi}{M}\left[1 + \cot\left(\frac{k\pi}{M}\right)^2\right] - j\cot\left(\frac{k\pi}{M}\right)\right\} \qquad 1 \le k < M.$$

Again, $j = \sqrt{-1}$.

Before calling this function, call `calc_laplace_parameter` to set the parameters and compute the required coefficients.

5.8.3 Stehfest method

`ctx.invlapstehfest(`*f*, *t*, ***kwargs*`)`

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: Stehfest [505], Widder [1080].

The Stehfest algorithm only uses abscissa along the real axis of the complex p-plane to estimate the time-domain function. Oscillatory time-domain functions have poles away from the real axis, so this method does not work well with oscillatory functions, especially high-frequency ones. This method also depends on summation of terms in a series that grows very large, and will have catastrophic cancellation during summation if the working precision is too low.

`def calc_laplace_parameter(self,t,**kwargs)`

The Gaver-Stehfest method is a discrete approximation of the Widder-Post inversion algorithm, rather than a direct approximation of the Bromwich contour integral.

The method abscissa along the real axis, and therefore has issues inverting oscillatory functions (which have poles in pairs away from the real axis).

The working precision will be increased according to a rule of thumb. If 'degree' is not specified, the working precision and degree are chosen to hopefully achieve the dps of the calling context. If 'degree' is specified, the working precision is chosen to achieve maximum resulting precision for the specified degree.

$$p_k = \frac{k\log 2}{t} \qquad 1 \le k \le M$$

`def calc_time_domain_solution(self,fp,t,manual_prec=False)`

Compute time-domain Stehfest algorithm solution.

$$f(t,M) = \frac{\log 2}{t} \sum_{k=1}^{M} V_k \bar{f}\left(p_k\right)$$

where

$$V_k = (-1)^{k+N/2} \sum_{i=\lfloor (k+1)/2 \rfloor}^{\min(k,N/2)} \frac{i^{\frac{N}{2}}(2i)!}{\left(\frac{N}{2}-i\right)!\, i!\, (i-1)!\, (k-i)!\, (2i-k)!}$$

As the degree increases, the abscissa (p_k) only increase linearly towards ∞, but the Stehfest coefficients (V_k) alternate in sign and increase rapidly in sign, requiring high precision to prevent overflow or loss of significance when evaluating the sum.

5.8.4 de Hoog, Knight, and Stokes method

`ctx.invlapdehoog`(f, t, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: Davies [200], [deHoog1982`.

The de Hoog, Knight, and Stokes method is essentially a Fourier-series quadrature-type approximation to the Bromwich contour integral, with non-linear series acceleration and an analytical expression for the remainder term. This method is typically the most robust and is therefore the default method. This method also involves the greatest amount of overhead, so it is typically the slowest of the three methods at high precision.

`calc_laplace_parameter(self,t,**kwargs)`

The de Hoog, Knight & Stokes algorithm is an accelerated form of the Fourier series numerical inverse Laplace transform algorithms.

$$p_k = \gamma + \frac{jk}{T} \qquad 0 \le k < 2M+1$$

where

$$\gamma = \alpha - \frac{\log \mathrm{tol}}{2T},$$

$j = \sqrt{-1}$, $T = 2t_{\max}$ is a scaled time, $\alpha = 10^{-\mathrm{dps_goal}}$ is the real part of the rightmost pole or singularity, which is chosen based on the desired accuracy (assuming the rightmost singularity is 0), and $\mathrm{tol} = 10\alpha$ is the desired tolerance, which is chosen in relation to α.`

When increasing the degree, the abscissa increase towards $j\infty$, but more slowly than the fixed Talbot algorithm. The de Hoog et al. algorithm typically does better with oscillatory functions of time, and less well-behaved functions. The method tends to be slower than the Talbot and Stehfest algorithsm, especially so at very high precision (e.g., > 500 digits precision).

`def calc_time_domain_solution(self,fp,t,manual_prec=False)` Calculate time-domain solution for de Hoog, Knight & Stokes algorithm.

The un-accelerated Fourier series approach is:

$$f(t,2M+1) = \frac{e^{\gamma t}}{T} \sum_{k=0}^{2M}{}' \Re\left[\bar{f}\left(p_k\right) e^{i\pi t/T}\right],$$

where the prime on the summation indicates the first term is halved.

This simplistic approach requires so many function evaluations that it is not practical. Non-linear acceleration is accomplished via Pade-approximation and an analytic expression for the remainder of the continued fraction. See the original paper (reference 2 below) a detailed description of the numerical approach.

5.8.5 Cohen method

`ctx.invlapcohen`(*f*, *t*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also Cohen *et al.* [182], Glasserman and Ruiz-Mata [350].

The Cohen method is a trapezoidal rule approximation to the Bromwich contour integral, with linear acceleration for alternating series. This method is as robust as the de Hoog et al method and the fastest of the four methods at high precision, and is therefore the default method.

The Cohen algorithm accelerates the convergence of the nearly alternating series resulting from the application of the trapezoidal rule to the Bromwich contour inversion integral.

$$p_k = \frac{\gamma}{2t} + \frac{\pi i k}{t} \qquad 0 \le k < M$$

where

$$\gamma = \frac{2}{3}(d + \log(10) + \log(2t)),$$

$d = \mathrm{dps_goal}$, which is chosen based on the desired accuracy using the method developed in [1] to improve numerical stability. The Cohen algorithm shows robustness similar to the de Hoog et al. algorithm, but it is faster than the fixed Talbot algorithm.

Optional arguments

degree
: integer order of the approximation (M = number of terms)

alpha
: abscissa for p_0 (controls the discretization error)

The working precision will be increased according to a rule of thumb. If 'degree' is not specified, the working precision and degree are chosen to hopefully achieve the dps of the calling context. If 'degree' is specified, the working precision is chosen to achieve maximum resulting precision for the specified degree.

Calculate time-domain solution for Cohen algorithm.

The accelerated nearly alternating series is:

$$f(t, M) = \frac{e^{\gamma/2}}{t}\left[\frac{1}{2}\Re\left(\bar{f}\left(\frac{\gamma}{2t}\right)\right) - \sum_{k=0}^{M-1}\frac{c_{M,k}}{d_M}\Re\left(\bar{f}\left(\frac{\gamma + 2(k+1)\pi i}{2t}\right)\right)\right],$$

where coefficients $\frac{c_{M,k}}{d_M}$ are described in Cohen *et al.* [182].

5.9 Boost/Odeint: Ordinary differential equations

Boost Odeint is a library for solving initial value problems (IVP) of ordinary differential equations. Mathematically, these problems are formulated as follows:

$$x'(t) = f(x,t), x(0) = x0.$$

x and f can be vectors and the solution is some function x(t) fulfilling both equations above. In the following we will refer to x'(t) also dxdt which is also our notation for the derivative in the source code.

Ordinary differential equations occur nearly everywhere in natural sciences. For example, the whole Newtonian mechanics are described by second order differential equations. Be sure, you will find them in every discipline. They also occur if partial differential equations (PDEs) are discretized. Then, a system of coupled ordinary differential occurs, sometimes also referred as lattices ODEs.

odeint provides a quite large number of different steppers such that the user is left with the question of which stepper fits his needs. Our personal recommendations are:

runge_kutta_dopri5 is maybe the best default stepper. It has step size control as well as dense-output functionality. Simple create a dense-output stepper by make_dense_output(1.0e-6 , 1.0e-5 , runge_kutta_dopri5< state_type >()).

runge_kutta4 is a good stepper for constant step sizes. It is widely used and very well known. If you need to create artificial time series this stepper should be the first choice.

'runge_kutta_fehlberg78' is similar to the 'runge_kutta4' with the advantage that it has higher precision. It can also be used with step size control.

adams_bashforth_moulton is very well suited for ODEs where the r.h.s. is expensive (in terms of computation time). It will calculate the system function only once during each step.

See also: BoostMath [5], Wikipedia [1113].

5.9.1 Runge-Kutta 4 method, constant stepper

ctxboost.**RungeKutta4Const**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, *mathrm{d}t*)

Solves Ordinary differential equations using the Runge-Kutta 4 method with a constant stepper.

See also: Wikipedia [1115], Rump [494], Kutta [403].

In numerical analysis, the Runge-Kutta methods are an important family of implicit and explicit iterative methods, which are used in temporal discretization for the approximation of solutions of ordinary differential equations. These techniques were developed around 1900 by the German mathematicians C. Runge and M. W. Kutta.

See the article on numerical methods for ordinary differential equations for more background and other methods. See also List of Runge-Kutta methods.

One member of the family of Runge-Kutta methods is often referred to as "RK4", "classical Runge-Kutta method" or simply as "the Runge-Kutta method".

Let an initial value problem be specified as follows.

Here, y is an unknown function (scalar or vector) of time t which we would like to approximate; we are told that , the rate at which y changes, is a function of t and of y itself. At the initial time the corresponding y-value is . The function f and the data , are given.

Example:

```
Public Sub matLorenz(x As dbl_mat_t, dxdt As dbl_mat_t)
    Dim sigma, R, b As Double
    sigma = 10: R = 28: b = 8 / 3
    dxdt(0) = sigma * ( x(1) - x(0) )
    dxdt(1) = R * x(0) - x(1) - x(0) * x(2)
    dxdt(2) = -b * x(2) + x(0) * x(1)
End Sub

Sub DemoLorenzOdeintLocal()
    Console.WriteLine( "DemoLorenzOdeintLocal(matX, StartTime, EndTime, \mathrm{d}t)")
    Dim matX3 As New dbl_mat_t()
    matX3.resize(3,1)
    matX3(0) = 10.0
    matX3(1) = 10.0
    matX3(2) = 10.0
    Dim StartTime = 0.0
    Dim EndTime = 10.0
    Dim \mathrm{d}t = 0.1
    matX3.print("matX at StartTime", 15)
    Dim ODE As New Odeint(AddressOf matLorenz, matX3, StartTime, EndTime, \mathrm{d}t)
    ODE.Integrate()
    matX3.print("matX at EndTime", 15)
End Sub
```

This produces the following output:

```
DemoLorenzOdeintLocal(matX, StartTime, EndTime, \mathrm{d}t)
matX at StartTime
10,
10,
10,

matX at EndTime
-2.43203453419697,
0.560815431795122,
25.4697563106702,
```

5.9.2 Cash-Karp method, constant stepper

ctxboost.**CashKarpConst**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, \mathrm{d}t)

Solves Ordinary differential equations using the Cash-Karp method, using a constant stepper

See also: Wikipedia [1104], Cash and Karp [174].

In numerical analysis, the Cash-Karp method is a method for solving ordinary differential equations (ODEs). It was proposed by Professor Jeff R. Cash [1] from Imperial College London and Alan H. Karp from IBM Scientific Center. The method is a member of the Runge-Kutta family of ODE solvers. More specifically, it uses six function evaluations to calculate fourth- and fifth-order accurate solutions. The difference between these solutions is then taken to be the error of the (fourth order) solution. This error estimate is very convenient for adaptive stepsize integration algorithms. Other similar integration methods are Fehlberg (RKF) and Dormand-Prince (RKDP).

5.9.3 Dormand-Prince 5 method, constant stepper

ctxboost.**DormandPrince5Const**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, \mathrm{d}t, *epsabs*, *epsrel*)

Solves Ordinary differential equations using the Dormand-Prince method, using a constant stepper

See also: Wikipedia [1106], Dormand and Prince [287].

In numerical analysis, the Dormand-Prince method, or DOPRI method, is an explicit method for solving ordinary differential equations citep{Dormand1980}. The method is a member of the Runge-Kutta family of ODE solvers. More specifically, it uses six function evaluations to calculate fourth- and fifth-order accurate solutions. The difference between these solutions is then taken to be the error of the (fourth-order) solution. This error estimate is very convenient for adaptive stepsize integration algorithms. Other similar integration methods are Fehlberg (RKF) and Cash-Karp (RKCK).

The Dormand-Prince method has seven stages, but it uses only six function evaluations per step because it has the FSAL (First Same As Last) property: the last stage is evaluated at the same point as the first stage of the next step. Dormand and Prince choose the coefficients of their method to minimize the error of the fifth-order solution. This is the main difference with the Fehlberg method, which was constructed so that the fourth-order solution has a small error. For this reason, the Dormand-Prince method is more suitable when the higher-order solution is used to continue the integration, a practice known as local extrapolation (Shampine 1986; Hairer, Nørsett & Wanner 2008, pp. 178-179).

5.9.4 Fehlberg 78 method, constant stepper

ctxboost.**Fehlberg78Const**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, *mathrm{d}t*, *epsabs*, *epsrel*)

Solves Ordinary differential equations using the Fehlberg 78 method, using a constant stepper

See also: Wikipedia [1116], Fehlberg [299].

In mathematics, the Runge-Kutta-Fehlberg method (or Fehlberg method) is an algorithm in numerical analysis for the numerical solution of ordinary differential equations. It was developed by the German mathematician Erwin Fehlberg and is based on the large class of Runge-Kutta methods.

The novelty of Fehlberg's method is that it is an embedded method from the Runge-Kutta family, meaning that identical function evaluations are used in conjunction with each other to create methods of varying order and similar error constants. The method presented in Fehlberg's 1969 paper has been dubbed the RKF45 method, and is a method of order O(h4) with an error estimator of order O(h5).[1] By performing one extra calculation, the error in the solution can be estimated and controlled by using the higher-order embedded method that allows for an adaptive stepsize to be determined automatically.

5.9.5 Adams-Bashforth-Moulton, constant stepper

ctxboost.**AdamsBashforthMoultonConst**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, \mathrm{d}t, *epsabs*, *epsrel*)

Solves Ordinary differential equations using the Adams-Bashforth-Moulton method, using a constant stepper

See also: Wikipedia [1107].

The methods of Euler, Heun, Taylor and Runge-Kutta are called single-step methods because they use only the information from one previous point to compute the successive point, that is, only the initial point is used to compute and in general is needed to compute . After several points have been found it is feasible to use several prior points in the calculation. The Adams-Bashforth-Moulton method uses in the calculation of . This method is not self-starting; four initial points , , , and must be given in advance in order to generate the points .

A desirable feature of a multistep method is that the local truncation error (L. T. E.) can be determined and a correction term can be included, which improves the accuracy of the answer at each step. Also, it is possible to determine if the step size is small enough to obtain an accurate value for , yet large enough so that unnecessary and time-consuming calculations are eliminated. If the code for the subroutine is fine-tuned, then the combination of a predictor and corrector requires only two function evaluations of f(t,y) per step.

5.9.6 Cash-Karp method, adaptive stepper

ctxboost.**CashKarpAdaptive**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, \mathrm{d}t, *epsabs*, *epsrel*)

Solves Ordinary differential equations using the Cash-Karp method method with an adaptive stepper.

See also: Wikipedia [1104], Cash and Karp [174].

In numerical analysis, the Cash-Karp method is a method for solving ordinary differential equations (ODEs). It was proposed by Professor Jeff R. Cash [1] from Imperial College London and Alan H. Karp from IBM Scientific Center. The method is a member of the Runge-Kutta family of ODE solvers. More specifically, it uses six function evaluations to calculate fourth- and fifth-order accurate solutions. The difference between these solutions is then taken to be the error of the (fourth order) solution. This error estimate is very convenient for adaptive stepsize integration algorithms. Other similar integration methods are Fehlberg (RKF) and Dormand-Prince (RKDP).

5.9.7 Dormand-Prince 5 method, adaptive stepper

ctxboost.**DormandPrince5Adaptive**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, *mathrm{d}t*, *epsabs*, *epsrel*)

Solves Ordinary differential equations using the Dormand-Prince 5 method with an adaptive stepper.

See also: Wikipedia [1106], Dormand and Prince [287].

In numerical analysis, the Dormand-Prince method, or DOPRI method, is an explicit method for solving ordinary differential equations citep{Dormand1980}. The method is a member of the Runge-Kutta family of ODE solvers. More specifically, it uses six function evaluations to calculate fourth- and fifth-order accurate solutions. The difference between these solutions is then taken to be the error of the (fourth-order) solution. This error estimate is very convenient for adaptive stepsize integration algorithms. Other similar integration methods are Fehlberg (RKF) and Cash-Karp (RKCK).

The Dormand-Prince method has seven stages, but it uses only six function evaluations per step because it has the FSAL (First Same As Last) property: the last stage is evaluated at the same point as the first stage of the next step. Dormand and Prince choose the coefficients of their method to minimize the error of the fifth-order solution. This is the main difference with the Fehlberg method, which was constructed so that the fourth-order solution has a small error. For this reason, the Dormand-Prince method is more suitable when the higher-order solution is used to continue the integration, a practice known as local extrapolation (Shampine 1986; Hairer, Nørsett & Wanner 2008, pp. 178-179).

5.9.8 Fehlberg 78 method, adaptive stepper

ctxboost.**Fehlberg78Adaptive**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, \mathrm{d}t, *epsabs*, *epsrel*)

Solves Ordinary differential equations using the Fehlberg 78 method with an adaptive stepper.

See also: Wikipedia [1116], Fehlberg [299].

In mathematics, the Runge-Kutta-Fehlberg method (or Fehlberg method) is an algorithm in numerical analysis for the numerical solution of ordinary differential equations. It was developed by the German mathematician Erwin Fehlberg and is based on the large class of Runge-Kutta methods.

The novelty of Fehlberg's method is that it is an embedded method from the Runge-Kutta family, meaning that identical function evaluations are used in conjunction with each other to create methods of varying order and similar error constants. The method presented in Fehlberg's 1969 paper has been dubbed the RKF45 method, and is a method of order O(h4) with an error estimator of order O(h5).[1] By performing one extra calculation, the error in the solution can be estimated and controlled by using the higher-order embedded method that allows for an adaptive stepsize to be determined automatically.

5.9.9 Bulirsch-Stoer method, adaptive stepper

ctxboost.**BulirschStoerAdaptive**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, *mathrm{d}t*, *epsabs*, *epsrel*)

Solves Ordinary differential equations using the Bulirsch-Stoer method, with an adaptive stepper

See also: Wikipedia [1103], Hairer *et al.* [362].

In numerical analysis, the Bulirsch-Stoer algorithm is a method for the numerical solution of ordinary differential equations which combines three powerful ideas: Richardson extrapolation, the use of rational function extrapolation in Richardson-type applications, and the modified midpoint method, to obtain numerical solutions to ordinary differential equations (ODEs) with high accuracy and comparatively little computational effort. It is named after Roland Bulirsch and Josef Stoer. It is sometimes called the Gragg-Bulirsch-Stoer (GBS) algorithm because of the importance of a result about the error function of the modified midpoint method, due to William B. Gragg.

5.9.10 Dormand-Prince 5 method, dense output stepper

ctxboost.**DormandPrince5DenseOutput**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, \mathrm{d}t, *epsabs*, *epsrel*)

Solves Ordinary differential equations using the Dormand-Prince 5 method, with a dense output stepper.

See also: Wikipedia [1106], Dormand and Prince [287].

In numerical analysis, the Dormand-Prince method, or DOPRI method, is an explicit method for solving ordinary differential equations citep{Dormand1980}. The method is a member of the Runge-Kutta family of ODE solvers. More specifically, it uses six function evaluations to calculate fourth- and fifth-order accurate solutions. The difference between these solutions is then taken to be the error of the (fourth-order) solution. This error estimate is very convenient for adaptive stepsize integration algorithms. Other similar integration methods are Fehlberg (RKF) and Cash-Karp (RKCK).

The Dormand-Prince method has seven stages, but it uses only six function evaluations per step because it has the FSAL (First Same As Last) property: the last stage is evaluated at the same point as the first stage of the next step. Dormand and Prince choose the coefficients of their method to minimize the error of the fifth-order solution. This is the main difference with the Fehlberg method, which was constructed so that the fourth-order solution has a small error. For this reason, the Dormand-Prince method is more suitable when the higher-order solution is used to continue the integration, a practice known as local extrapolation (Shampine 1986; Hairer, Nørsett & Wanner 2008, pp. 178-179).

5.9.11 Bulirsch-Stoer method, dense output stepper

`ctxboost.`**`BulirschStoerDenseOutput`**(*F1*, *F2*, *matInput*, *StartTime*, *EndTime*, *mathrm{d}t*, *epsabs*, *epsrel*)

Solves Ordinary differential equations using the Bulirsch-Stoer method, with a dense output stepper.

See also: Wikipedia [1103], Hairer *et al.* [362].

In numerical analysis, the Bulirsch-Stoer algorithm is a method for the numerical solution of ordinary differential equations which combines three powerful ideas: Richardson extrapolation, the use of rational function extrapolation in Richardson-type applications, and the modified midpoint method, to obtain numerical solutions to ordinary differential equations (ODEs) with high accuracy and comparatively little computational effort. It is named after Roland Bulirsch and Josef Stoer. It is sometimes called the Gragg-Bulirsch-Stoer (GBS) algorithm because of the importance of a result about the error function of the modified midpoint method, due to William B. Gragg.

5.10 Mpmath: Numerical differentiation

5.10.1 First derivative, using the complex-step derivative approximation

`ctx.deriv1_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: https://nhigham.com/2020/10/06/what-is-the-complex-step-approximation/

See Lai and Crassidis

The complex-step derivative approximation can be derived by approximating a nonlinear function with a complex variable using a Taylor's series expansion:

$$f(x+ih) = f(x) + ihf'(x) - \frac{h^2}{2!}f''(x) - i\frac{h^3}{3!}f^{(3)}(x) + \frac{h^4}{4!}f^{(4)}(x) + \cdots$$

Taking only the imaginary parts of both sides, dividing by h and rearranging gives

$$f'(x) = \frac{\Im\left(f(x+ih)\right)}{h} + O(h^2)$$

Terms with order h^2 or higher can be ignored since the interval h can be chosen up to machine precision. Thus, to within first order the complex-step derivative approximation is given by

$$f'(x) = \frac{\Im\left(f(x+ih)\right)}{h}, E_{\text{trunc}}(h) = \frac{h^2}{6}f^{(3)}(x)$$

where $E_{\text{trunc}}(h)$ denotes the truncation error. Note that this solution is not a function of differences, which ultimately provides better roundoff characteristics than a standard finite difference.

5.10.2 Second derivative, using the complex-step derivative approximation

`ctx.deriv2_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: https://nhigham.com/2020/10/06/what-is-the-complex-step-approximation/

See Lai and Crassidis

5.10.3 Gradient, using the complex-step derivative approximation

`ctx.gradient_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: https://nhigham.com/2020/10/06/what-is-the-complex-step-approximation/

The gradient of a vector function is a simple extension of the scalar case.

5.10.4 Jacobi matrix, using the complex-step derivative approximation

`ctx.jacobi_c`(*f*, *x0*, *solver='secant'*, *tol=None*, *verbose=False*, *verify=True*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: https://nhigham.com/2020/10/06/what-is-the-complex-step-approximation/

The Jacobian of a vector function is a simple extension of the scalar case. This Jacobian is defined by

$$F_x = \begin{pmatrix} \frac{\partial f_1(\boldsymbol{x})}{\partial x_1} & \frac{\partial f_1(\boldsymbol{x})}{\partial x_2} & \cdots & \frac{\partial f_1(\boldsymbol{x})}{\partial x_p} & \cdots & \frac{\partial f_1(\boldsymbol{x})}{\partial x_n} \\ \frac{\partial f_2(\boldsymbol{x})}{\partial x_1} & \frac{\partial f_2(\boldsymbol{x})}{\partial x_2} & \cdots & \frac{\partial f_2(\boldsymbol{x})}{\partial x_p} & \cdots & \frac{\partial f_2(\boldsymbol{x})}{\partial x_n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_q(\boldsymbol{x})}{\partial x_1} & \frac{\partial f_q(\boldsymbol{x})}{\partial x_2} & \cdots & \frac{\partial f_q(\boldsymbol{x})}{\partial x_p} & \cdots & \frac{\partial f_q(\boldsymbol{x})}{\partial x_n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \frac{\partial f_m(\boldsymbol{x})}{\partial x_1} & \frac{\partial f_m(\boldsymbol{x})}{\partial x_2} & \cdots & \frac{\partial f_m(\boldsymbol{x})}{\partial x_p} & \cdots & \frac{\partial f_m(\boldsymbol{x})}{\partial x_n} \end{pmatrix}$$

The complex-step approximation of the Jacobian is defined by

$$F_x = \frac{1}{h}\Im \begin{pmatrix} f_1(\boldsymbol{x}+ih\boldsymbol{e}_1) & f_1(\boldsymbol{x}+ih\boldsymbol{e}_2) & \cdots & f_1(\boldsymbol{x}+ih\boldsymbol{e}_q) & \cdots & f_1(\boldsymbol{x}+ih\boldsymbol{e}_n) \\ f_2(\boldsymbol{x}+ih\boldsymbol{e}_1) & f_2(\boldsymbol{x}+ih\boldsymbol{e}_2) & \cdots & f_2(\boldsymbol{x}+ih\boldsymbol{e}_q) & \cdots & f_2(\boldsymbol{x}+ih\boldsymbol{e}_n) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ f_q(\boldsymbol{x}+ih\boldsymbol{e}_1) & f_q(\boldsymbol{x}+ih\boldsymbol{e}_2) & \cdots & f_q(\boldsymbol{x}+ih\boldsymbol{e}_q) & \cdots & f_q(\boldsymbol{x}+ih\boldsymbol{e}_n) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ f_m(\boldsymbol{x}+ih\boldsymbol{e}_1) & f_m(\boldsymbol{x}+ih\boldsymbol{e}_2) & \cdots & f_m(\boldsymbol{x}+ih\boldsymbol{e}_q) & \cdots & f_m(\boldsymbol{x}+ih\boldsymbol{e}_n) \end{pmatrix}$$

where $\boldsymbol{e}_p$ is the p^{th} column of an n^{th}-order identity matrix and f_q is the q^{th} equation of $\boldsymbol{f}(\boldsymbol{x})$.

!!! CHECK CODE IN MPMATH !!!

5.10.5 Nth numerical (partial) derivative, using finite differences or numerical quadrature

`ctx.diff`(*f*, *x*, *n=1*, ***options*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Numerically computes the derivative of f, $f'(x)$, or generally for an integer $n \geq 0$, the n-th derivative $f^{(n)}(x)$. A few basic examples are:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> diff(lambda x: x**2 + x, 1.0)
3.0
>>> diff(lambda x: x**2 + x, 1.0, 2)
2.0
>>> diff(lambda x: x**2 + x, 1.0, 3)
0.0
>>> nprint([diff(exp, 3, n) for n in range(5)])   # exp'(x) = exp(x)
[20.0855, 20.0855, 20.0855, 20.0855, 20.0855]
```

Even more generally, given a tuple of arguments $(x_1, \ldots, x_k)$ and order $(n_1, \ldots, n_k)$, the partial derivative $f^{(n_1,\ldots,n_k)}(x_1, \ldots, x_k)$ is evaluated. For example:

```
>>> diff(lambda x,y: 3*x*y + 2*y - x, (0.25, 0.5), (0,1))
2.75
>>> diff(lambda x,y: 3*x*y + 2*y - x, (0.25, 0.5), (1,1))
3.0
```

Options

The following optional keyword arguments are recognized:

`method`
Supported methods are `'step'` or `'quad'`: derivatives may be computed using either a finite difference with a small step size h (default), or numerical quadrature.

direction
Direction of finite difference: can be -1 for a left difference, 0 for a central difference (default), or +1 for a right difference; more generally can be any complex number.

addprec
Extra precision for h used to account for the function's sensitivity to perturbations (default = 10).

relative
Choose h relative to the magnitude of x, rather than an absolute value; useful for large or tiny x (default = False).

h
As an alternative to `addprec` and `relative`, manually select the step size h.

singular
If True, evaluation exactly at the point x is avoided; this is useful for differentiating functions with removable singularities. Default = False.

radius
Radius of integration contour (with `method = 'quad'`). Default = 0.25. A larger radius typically is faster and more accurate, but it must be chosen so that f has no singularities within the radius from the evaluation point.

A finite difference requires $n+1$ function evaluations and must be performed at $(n+1)$ times the target precision. Accordingly, f must support fast evaluation at high precision.

With integration, a larger number of function evaluations is required, but not much extra precision is required. For high order derivatives, this method may thus be faster if f is very expensive to evaluate at high precision.

Further examples

The direction option is useful for computing left- or right-sided derivatives of nonsmooth functions:

```
>>> diff(abs, 0, direction=0)
0.0
>>> diff(abs, 0, direction=1)
1.0
>>> diff(abs, 0, direction=-1)
-1.0
```

More generally, if the direction is nonzero, a right difference is computed where the step size is multiplied by sign(direction). For example, with direction=+j, the derivative from the positive imaginary direction will be computed:

```
>>> diff(abs, 0, direction=j)
(0.0 - 1.0j)
```

With integration, the result may have a small imaginary part even even if the result is purely real:

```
>>> diff(sqrt, 1, method='quad')
(0.5 - 4.59...e-26j)
>>> chop(_)
0.5
```

Adding precision to obtain an accurate value:

```
>>> diff(cos, 1e-30)
0.0
>>> diff(cos, 1e-30, h=0.0001)
-9.99999998328279e-31
>>> diff(cos, 1e-30, addprec=100)
-1.0e-30
```

5.10.6 Function object which evaluates the nth derivative of a given function

`ctx.diffun`(*f*, *n=1*, ***options*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Given a function f, returns a function $g(x)$ that evaluates the nth derivative $f^{(n)}(x)$:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> cos2 = diffun(sin)
>>> sin2 = diffun(sin, 4)
>>> cos(1.3), cos2(1.3)
(0.267498828624587, 0.267498828624587)
>>> sin(1.3), sin2(1.3)
(0.963558185417193, 0.963558185417193)
```

The function f must support arbitrary precision evaluation. See `diff()` for additional details and supported keyword options.

5.10.7 Forward difference, based on a given sequence

`ctx.difference`(*s*, *n*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Given a sequence (s_k) containing at least $n+1$ items, returns the n-th forward difference,

$$\Delta^n = \sum_{k=0}^{\infty} (-1)^{k+n} \binom{n}{k} s_k.$$

EXAMPLE !!!!!

5.10.8 Generating a sequence of derivatives

`ctx.diffs`(*f*, *x*, *n=None*, ***options*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns a generator that yields the sequence of derivatives

$$f(x), f'(x), f''(x), \ldots, f^{(k)}(x), \ldots$$

With `method='step'`, `diffs()` uses only $O(k)$ function evaluations to generate the first k derivatives, rather than the roughly $O(k^2)$ evaluations required if one calls `diff()` k separate times.

With $n < \infty$, the generator stops as soon as the n-th derivative has been generated. If the exact number of needed derivatives is known in advance, this is further slightly more efficient.

Options are the same as for `diff()`.

Examples

```
>>> from xlcalcnet import *
>>> mp.dps = 15
>>> nprint(list(diffs(cos, 1, 5)))
[0.540302, -0.841471, -0.540302, 0.841471, 0.540302, -0.841471]
>>> for i, d in zip(range(6), diffs(cos, 1)):
...     print("%s %s" % (i, d))
...
0 0.54030230586814
1 -0.841470984807897
```

(continues on next page)

(continued from previous page)

```
2 -0.54030230586814
3 0.841470984807897
4 0.54030230586814
5 -0.841470984807897
```

5.10.9 Composition of derivatives

`ctx.diffs_prod`(*factors*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Given a list of N iterables or generators yielding $f_k(x), f_k'(x), f_k''(x), \ldots$ for $k = 1, \ldots, N$, generate $g(x), g'(x), g''(x), \ldots$ where $g(x) = f_1(x) f_2(x) \cdots f_N(x)$.

At high precision and for large orders, this is typically more efficient than numerical differentiation if the derivatives of each $f_k(x)$ admit direct computation.

Note: This function does not increase the working precision internally, so guard digits may have to be added externally for full accuracy.

Examples

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> f = lambda x: exp(x)*cos(x)*sin(x)
>>> u = diffs(f, 1)
>>> v = mp.diffs_prod([diffs(exp,1), diffs(cos,1), diffs(sin,1)])
>>> next(u); next(v)
1.23586333600241
1.23586333600241
>>> next(u); next(v)
0.104658952245596
0.104658952245596
>>> next(u); next(v)
-5.96999877552086
-5.96999877552086
>>> next(u); next(v)
-12.4632923122697
-12.4632923122697
```

5.10.10 Composition of exponential of derivatives

`ctx.diffs_exp`(*fdiffs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Given an iterable or generator yielding $f(x), f'(x), f''(x), \ldots$ generate $g(x), g'(x), g''(x), \ldots$ where $g(x) = \exp(f(x))$.

At high precision and for large orders, this is typically more efficient than numerical differentiation if the derivatives of $f(x)$ admit direct computation.

Note: This function does not increase the working precision internally, so guard digits may have to be added externally for full accuracy.

Examples

The derivatives of the gamma function can be computed using logarithmic differentiation:

```
.. code-block:: pycon
```

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>>
>>> def diffs_loggamma(x):
...     yield loggamma(x)
...     i = 0
...     while 1:
...         yield psi(i,x)
...         i += 1
...
>>> u = diffs_exp(diffs_loggamma(3))
>>> v = diffs(gamma, 3)
>>> next(u); next(v)
2.0
2.0
>>> next(u); next(v)
1.84556867019693
1.84556867019693
>>> next(u); next(v)
2.49292999190269
2.49292999190269
>>> next(u); next(v)
3.44996501352367
3.44996501352367
```

5.10.11 Fractional derivatives / differintegration

`ctx.differint`(*f*, *x*, *n=1*, *x0=0*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Calculates the Riemann-Liouville differintegral, or fractional derivative, defined by

$$ {}_{x_0}\mathbb{D}_x^n f(x) = \frac{1}{\Gamma(m-n)} \frac{\mathrm{d}^m}{\mathrm{d}x^m} \int_{x_0}^{x} (x-t)^{m-n-1} f(t) \mathrm{d}t $$

where f is a given (presumably well-behaved) function, x is the evaluation point, n is the order, and x_0 is the reference point of integration (m is an arbitrary parameter selected automatically).

With $n = 1$, this is just the standard derivative $f'(x)$; with $n = 2$, the second derivative $f''(x)$, etc. With $n = -1$, it gives $\int_{x_0}^{x} f(t) \mathrm{d}t$, with $n = -2$ it gives $\int_{x_0}^{x} \left(\int_{x_0}^{t} f(u) du \right) \mathrm{d}t$, etc.

As n is permitted to be any number, this operator generalizes iterated differentiation and iterated integration to a single operator with a continuous order parameter.

Examples

There is an exact formula for the fractional derivative of a monomial x^p, which may be used as a reference. For example, the following gives a half-derivative (order 0.5):

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> x = mpf(3); p = 2; n = 0.5
>>> differint(lambda t: t**p, x, n)
7.81764019044672
>>> gamma(p+1)/gamma(p-n+1) * x**(p-n)
7.81764019044672
```

Another useful test function is the exponential function, whose integration / differentiation formula easy generalizes to arbitrary order. Here we first compute a third derivative, and then a triply nested integral. (The reference point x_0 is set to $-\infty$ to avoid nonzero endpoint terms.):

```
>>> differint(lambda x: exp(pi*x), -1.5, 3)
0.278538406900792
>>> exp(pi*-1.5) * pi**3
0.278538406900792
>>> differint(lambda x: exp(pi*x), 3.5, -3, -inf)
1922.50563031149
>>> exp(pi*3.5) / pi**3
1922.50563031149
```

However, for noninteger n, the differentiation formula for the exponential function must be modified to give the same result as the Riemann-Liouville differintegral:

```
>>> x = mpf(3.5)
>>> c = pi
>>> n = 1+2*j
>>> differint(lambda x: exp(c*x), x, n)
(-123295.005390743 + 140955.117867654j)
>>> x**(-n) * exp(c)**x * (x*c)**n * gammainc(-n, 0, x*c) / gamma(-n)
(-123295.005390743 + 140955.117867654j)
```

5.10.12 Taylor series

`ctx.taylor`(*f*, *x*, *n*, ***options*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Produces a degree-n Taylor polynomial around the point x of the given function f. The coefficients are returned as a list.

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> nprint(chop(taylor(sin, 0, 5)))
[0.0, 1.0, 0.0, -0.166667, 0.0, 0.00833333]
```

The coefficients are computed using high-order numerical differentiation. The function must be possible to evaluate to arbitrary precision. See `diff()` for additional details and supported keyword options.

Note that to evaluate the Taylor polynomial as an approximation of f, e.g. with `polyval()`, the coefficients must be reversed, and the point of the Taylor expansion must be subtracted from the argument:

```
>>> p = taylor(exp, 2.0, 10)
>>> polyval(p[::-1], 2.5 - 2.0)
12.1824939606092
>>> exp(2.5)
12.1824939607035
```

5.10.13 Solving an ODE using high-order Taylor series

`ctx.odefun`(*F*, *x0*, *y0*, *tol=None*, *degree=None*, *method='taylor'*, *verbose=False*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns a function $y(x) = [y_0(x), y_1(x), \ldots, y_n(x)]$ that is a numerical solution of the $n+1$-dimensional first-order ordinary differential equation (ODE) system

$$
\begin{aligned}
y_0'(x) &= F_0(x, [y_0(x), y_1(x), \ldots, y_n(x)]) \\
y_1'(x) &= F_1(x, [y_0(x), y_1(x), \ldots, y_n(x)]) \\
&\vdots \\
y_n'(x) &= F_n(x, [y_0(x), y_1(x), \ldots, y_n(x)])
\end{aligned}
$$

The derivatives are specified by the vector-valued function *F* that evaluates $[y_0', \ldots, y_n'] = F(x, [y_0, \ldots, y_n])$. The initial point x_0 is specified by the scalar argument *x0*, and the initial value $y(x_0) = [y_0(x_0), \ldots, y_n(x_0)]$ is specified by the vector argument *y0*.

For convenience, if the system is one-dimensional, you may optionally provide just a scalar value for *y0*. In this case, *F* should accept a scalar *y* argument and return a scalar. The solution function *y* will return scalar values instead of length-1 vectors.

Evaluation of the solution function $y(x)$ is permitted for any $x \geq x_0$.

A high-order ODE can be solved by transforming it into first-order vector form. This transformation is described in standard texts on ODEs. Examples will also be given below.

Options, speed and accuracy

By default, `odefun()` uses a high-order Taylor series method. For reasonably well-behaved problems, the solution will be fully accurate to within the working precision. Note that *F* must be possible to evaluate to very high precision for the generation of Taylor series to work.

To get a faster but less accurate solution, you can set a large value for *tol* (which defaults roughly to *eps*). If you just want to plot the solution or perform a basic simulation, *tol = 0.01* is likely sufficient.

The *degree* argument controls the degree of the solver (with *method='taylor'*, this is the degree of the Taylor series expansion). A higher degree means that a longer step can be taken before a new local solution must be generated from *F*, meaning that fewer steps are required to get from x_0 to a given x_1. On the other hand, a higher degree also means that each local solution becomes more expensive (i.e., more evaluations of *F* are required per step, and at higher precision).

The optimal setting therefore involves a tradeoff. Generally, decreasing the *degree* for Taylor series is likely to give faster solution at low precision, while increasing is likely to be better at higher precision.

The function object returned by `odefun()` caches the solutions at all step points and uses polynomial interpolation between step points. Therefore, once $y(x_1)$ has been evaluated for some x_1, $y(x)$ can be evaluated very quickly for any $x_0 \leq x \leq x_1$. and continuing the evaluation up to $x_2 > x_1$ is also fast.

Examples of first-order ODEs

We will solve the standard test problem $y'(x) = y(x), y(0) = 1$ which has explicit solution $y(x) = \exp(x)$:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> f = odefun(lambda x, y: y, 0, 1)
>>> for x in [0, 1, 2.5]:
...     print((f(x), exp(x)))
...
(1.0, 1.0)
(2.71828182845905, 2.71828182845905)
(12.1824939607035, 12.1824939607035)
```

The solution with high precision:

```
>>> mp.dps = 50
>>> f = odefun(lambda x, y: y, 0, 1)
>>> f(1)
2.7182818284590452353602874713526624977572470937
>>> exp(1)
2.7182818284590452353602874713526624977572470937
```

Using the more general vectorized form, the test problem can be input as (note that *f* returns a 1-element vector):

```
>>> mp.dps = 15
>>> f = odefun(lambda x, y: [y[0]], 0, [1])
>>> f(1)
[2.71828182845905]
```

`odefun()` can solve nonlinear ODEs, which are generally impossible (and at best difficult) to solve analytically. As an example of a nonlinear ODE, we will solve $y'(x) = x\sin(y(x))$ for $y(0) = \pi/2$. An exact solution happens to be known for this problem, and is given by $y(x) = 2\tan^{-1}\left(\exp\left(x^2/2\right)\right)$:

```
>>> f = odefun(lambda x, y: x*sin(y), 0, pi/2)
>>> for x in [2, 5, 10]:
...     print((f(x), 2*atan(exp(mpf(x)**2/2))))
...
(2.87255666284091, 2.87255666284091)
(3.14158520028345, 3.14158520028345)
(3.14159265358979, 3.14159265358979)
```

If F is independent of y, an ODE can be solved using direct integration. We can therefore obtain a reference solution with `quad()`:

```
>>> f = lambda x: (1+x**2)/(1+x**3)
>>> g = odefun(lambda x, y: f(x), pi, 0)
>>> g(2*pi)
0.72128263801696
>>> quad(f, [pi, 2*pi])
0.72128263801696
```

Examples of second-order ODEs

We will solve the harmonic oscillator equation $y''(x)+y(x) = 0$. To do this, we introduce the helper functions $y_0 = y, y_1 = y_0'$ whereby the original equation can be written as $y_1' + y_0' = 0$. Put together, we get the first-order, two-dimensional vector ODE

$$\begin{cases} y_0' = y_1 \\ y_1' = -y_0 \end{cases}$$

To get a well-defined IVP, we need two initial values. With $y(0) = y_0(0) = 1$ and $-y'(0) = y_1(0) = 0$, the problem will of course be solved by $y(x) = y_0(x) = \cos(x)$ and $-y'(x) = y_1(x) = \sin(x)$. We check this:

```
>>> f = odefun(lambda x, y: [-y[1], y[0]], 0, [1, 0])
>>> for x in [0, 1, 2.5, 10]:
...     nprint(f(x), 15)
...     nprint([cos(x), sin(x)], 15)
...     print("---")
...
[1.0, 0.0]
[1.0, 0.0]
---
[0.54030230586814, 0.841470984807897]
[0.54030230586814, 0.841470984807897]
---
[-0.801143615546934, 0.598472144103957]
[-0.801143615546934, 0.598472144103957]
---
[-0.839071529076452, -0.54402111088937]
[-0.839071529076452, -0.54402111088937]
---
```

Note that we get both the sine and the cosine solutions simultaneously.

5.11 Mpmath: Asymptotic expansions

5.11.1 Edgeworth expansion: general approximation of the pdf, cdf and sf

`ctx.edgeworth`(*x*, *cumulants*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Edgeworth expansion approximations of the pdf, cdf and sf by using the cumulants of the distribution.

See also Wikipedia [1090], Blinnikov and Moessner [32], Wikipedia [1084], Wikipedia [1307].

Using the first 4 cumulants, we have

$$\Pr\left[\frac{u-\mu}{\sigma} \le x\right] = \Phi(x) + \phi(x)\left[\frac{\gamma_1}{6}(1-x^2) + \frac{\gamma_2}{24}(3x-x^3) + \frac{\gamma_1^2}{72}(15x-10x^3+x^5)\right] + O(N^{-3/2})$$

Withers and Nadarajah [1456] and Withers and Nadarajah [1457] have shown that the Edgeworth expansion can also be expressed in terms of standardized cumulants $\alpha_r = \kappa_r/\kappa_2^{r/2}$ (with $\alpha_1 = \alpha_2 = 0$), partial exponential Bell polynomials $B_{r,k}(\boldsymbol{\alpha})$, $\boldsymbol{\alpha} = (\alpha_1, \cdots, \alpha_r)$, and Hermite polynomials $H_r(x)$:

$$f(x) = \phi(x) + \phi(x)\sum_{j=1}^{\infty} h_{1j}(x) n^{-j/2}, \quad \text{where}$$

$$h_{1j}(x) = \sum_{k=1}^{j} B_{r,k}(\boldsymbol{\alpha}) H_r(x)/r!, \quad \text{with } r = j+2k.$$

$$F(x) = \Phi(x) - \phi(x)\sum_{j=1}^{\infty} h_{0j}(x) n^{-j/2},$$

$$1 - F(x) = \Phi(-x) + \phi(x)\sum_{j=1}^{\infty} h_{0j}(x) n^{-j/2}, \quad \text{where}$$

$$h_{0j}(x) = \sum_{k=1}^{j} B_{r,k}(\boldsymbol{\alpha}) H_{r-1}(x)/r!, \quad \text{with } r = j+2k.$$

The partial Bell polynomials can be computed efficiently by a recurrence relation:

$$B_{r,k}(\boldsymbol{\alpha}) = \sum_{i=1}^{r-k+1} \binom{r-1}{i-1} \alpha_i B_{r-i,k-1}(\boldsymbol{\alpha}), \quad \text{where}$$

$$B_{0,0} = 1; \quad B_{r,0} = 0 \text{ (for } r \ge 1); \quad B_{0,k} = 0 \text{ (for } k \ge 1); \quad B_{r,1}(\boldsymbol{\alpha}) = \alpha_r.$$

The Hermite polynomials are defined by

$$H_0(x) = 1, \quad H_1(x) = x, \quad H_{r+1}(x) = xH_r(x) - rH_{r-1}(x)$$

The following code shows how to call the Edgeworth expansion for the central cdf.

```
def EdgeworthWhithersDemo(self):
    Order = 30
    F = 100
    X = mp.mpf(110)

    kappa = mp.matrix(Order+1, 1)
    kappa[1] = F
    for i in range(2, Order+1):
        kappa[i] = kappa[i - 1] * 2 * (i - 1)

    LeftTail1, RightTail1 = self.EdgeworthWhithers(X, Order, kappa)
    print("EdgeworthWhithers LeftTail1: ", LeftTail1, " RightTail1: ",  RightTail1)
```

This gives the following results:

```
Hello!
X:  110.0 , Z:  0.707106781186547524400844362104849039284 8 ,
mean:  100.0 , sigma:  14.142135623730950488016887242096980785 7
s3:  0.760249938906523268841373326945982264368 2 ,
s4:  0.310696560376927744870908891228809194554 7

j:  1 , s2:  0.007323188157795373284114366290203815819688
j:  2 , s2:  0.000244106271926512442803812209673460527323
j:  3 , s2:  -0.000024275012597136515145490214184194130217 11
j:  4 , s2:  0.000002526386902276474842073713864467689855687
j:  5 , s2:  -0.000000284396453393571901162750504664042110277 5
j:  6 , s2:  0.000000020873385359592273002246472679643569714 42
j:  7 , s2:  -0.000000001797321695705282603398930576647079128807
j:  8 , s2:  0.000000000110571628345128221545086895055796849600 9

LeftTail1:  0.767795219500732193188205545658764614828 51
RightTail1:  0.232204780499267806811794453412335851714 9
```

5.11.2 Cornish-Fisher expansion: general approximation of the qtf and isf

`ctx.cornish_fisher`(*x*, *cumulants*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Cornish-Fisher approximation of the quantile by using the cumulants of the distribution.

The quantiles of many distribution functions can be approximated by the classical Cornish and Fisher expansion Cornish and Fisher [186], Fisher and Cornish [337]), provided that the cumulants κ_r of the distribution are known (see also Wikipedia [1087]):

$$x = z + \sum_{k=1}^{\infty} n^{-k/2} \xi_k(z)$$

where $\xi_k(z)$ is the collection of all the terms that belong to the k-th power of $n^{-1/2}$. It seems to be easier to describe algorithm AS269 (see Lee and Lin [404]), which is used to calculate the k-th adjustment term $\xi_k(z)$, by giving a program which implements it, rather than to describe it using mathematical formulas:

```
def CornishFisherLee(self, r, x0, ac, adj):
    a = iv.matrix(r+3, 1); d = iv.matrix(r+3, 1)
    h = iv.matrix(3 * (r+3), 1)
    p = iv.matrix((3 * (r+3)) * ((r+3) + 1) // 2, 1)
    x = iv.mpf(x0); cc = -1
    for j in range(1, r+1):
        a[j] = cc * ac[j] / ((j + 1) * (j + 2))
        cc = -cc
    h[1] = -x; h[2] = x * x - 1
    for j in range(3, (3 * r)+1):
        h[j] = -(x * h[j - 1] + (j - 1) * h[j - 2])
    d[1] = -a[1] * h[2]; adj[1] = d[1]; p[1] = d[1]
    p[3] = a[1]; fac = iv.mpf(1); ja = 0
    for j in range(2, r+1):
        fac = fac * j
        bc = iv.mpf(1)
        ja = ja + 3 * (j - 1)
        jb = ja
        for k in range(1, j):
            bcd = bc * d[k]
            bca = bc * a[k]
            jb -= 3 * (j - k)
            for m in range(1, (3 * (j - k))+1):
                jbl = jb + m; jal = ja + m
                p[jal + 1] += bcd * p[jbl]
```

(continues on next page)

(continued from previous page)

```
            p[jal + k + 2] += bca * p[jbl]
        bc = (bc * (j - k)) / k
    p[ja + j + 2] += a[j]; d[j] = 0
    for m in range(2, (3 * j)+1):
        d[j] -= p[ja + m] * h[m - 1]
    p[ja + 1] = d[j]
    adj[j] = d[j] / fac
```

However, if one prefers mathematical formulas, Jaschke [379] gives the following description of the algorithm: "Lee developed a recurrence formula for the k-th adjustment term $\xi_k(z)$ in the Cornish-Fisher expansion, which is implemented in the algorithm AS269 :

$$\xi_k(H) = a_k H^{*(k+1)} - \sum_{j=1}^{k-1} \frac{j}{k} \left(\xi_{k-j}(H) - \xi_{k-j}\right) * \left(\xi_j - a_j H^{*(j+1)}\right) * H, \quad \text{where } a_k = \frac{\kappa_{k+2}}{(k+2)!}.$$

$\xi_k(H)$ is a formal polynomial expression in H with the usual algebraic relations between the summation "$+$" and the "multiplication" "$*$". Once $\xi_k(H)$ is multiplied out in $*$-powers of H, each H^{*k} is to be interpreted as the Hermite polynomial H_k and then the whole term becomes a polynomial in z with the "normal" multiplication "$\cdot$". ξ_k denotes the scalar that results when the "normal" polynomial $\xi_k(H)$ is evaluated at the fixed quantile z, while $\xi_k(H)$ denotes the expression in the $(+, *)$-algebra. In this formula the adjustment of a given order is calculated recursively, the values of lower order adjustments being used in each stage of calculation."

5.11.3 Sheppard correction, cumulants

`ctx.sheppard`(*cumulants*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Returns the Sheppard correction term for a vector of cumulants.

The Sheppard correction is typically applied to the cumulants computed from normally distributed data which have been binned. If κ_r^* is the r-th cumulant of an ungrouped distribution and κ_r the r-th cumulant of an grouped distribution with class interval c, the corrected cumulants (under rather restrictive conditions) are

$$\kappa_r^* = \begin{cases} \kappa_r & \text{for } r \text{ odd} \\ \kappa_r - \frac{B_r}{r} c^r & \text{for } r \text{ even.} \end{cases}$$

where B_r is the r-th Bernoulli number. The same correction can be applied to the cumulants of the continuity-corrected test statistic of a random variable following a lattice distribution with step c. The (continuous) Edgeworth and Cornish-Fisher expansions using these corrected cumulants will then achieve the usual $o(n^{1-r/2})$ error terms of these expansions (see Kolassa and McCullagh [396]), despite the fact that the approximated CDF is discontinuous with jumps of order $O(n^{-1/2})$.

A similar correction can be applied to the cumulant generating function (and its derivatives), as it is needed by the Lugannani-Rice saddle point approximation.

5.11.4 The Sheppard correction, cumulant generating function

`ctx.sheppard_per_cgf`(*cgf*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Returns the Sheppard correction term for cumulant generating function of a discrete ("lattice") random variable.

The Sheppard correction is typically applied to the cumulants computed from normally distributed data which have been binned. If κ_r^* is the r-th cumulant of an ungrouped distribution and κ_r the r-th cumulant of an grouped distribution with class interval c, the corrected cumulants (under rather restrictive conditions) are

$$\kappa_r^* = \begin{cases} \kappa_r & \text{for } r \text{ odd} \\ \kappa_r - \frac{B_r}{r} c^r & \text{for } r \text{ even.} \end{cases}$$

where B_r is the r-th Bernoulli number. The same correction can be applied to the cumulants of the continuity-corrected test statistic of a random variable following a lattice distribution with step c. The (continuous) Edgeworth and Cornish-Fisher expansions using these corrected cumulants will then achieve the usual $o(n^{1-r/2})$ error terms of these expansions (see Kolassa and McCullagh [396]), despite the fact that the approximated CDF is discontinuous with jumps of order $O(n^{-1/2})$.

A similar correction can be applied to the cumulant generating function (and its derivatives), as it is needed by the Lugannani-Rice saddlepoint approximation.

5.11.5 Lugannini-Rice expansion: general approximation of the pdf, cdf, and sf

`ctx.luggannini_rice`(*x*, *s*, *poly_t cgf_series*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Luggannini-Rice saddlepoint approximation of the pdf by using the cumulant generating function and its derivatives.

See also Butler [165], Daniels [195], Daniels [196], Lugannani and Rice [410], Wikipedia [1122], Wikipedia [1138].

Suppose the moment generating function of the random variable X is analytic and given by $M(t)$ for t in some open neighbourhood of zero. Let $K(t) = \log(M(t))$ be the cumulant generating function of X, and denote by $K^{(j)}(t)$ the j^{th} derivative of $K(t)$.

Let $F(x)$ and $f(x)$ denote the CDF and pdf of X, respectively, and $\Phi(\cdot)$ and $\phi(\cdot)$ denote the CDF and pdf of the normal distribution, respectively. Let s be the solution to the saddlepoint equation $K^{(1)}(s) = x$, which in general needs to be solved numerically. Then

$$f(x) \approx \phi(w) \sum_{k=0}^{\infty} C_k,$$

$$F(x) \approx \Phi(w) + \phi(w) \sum_{k=0}^{\infty} (A_k - B_k), \quad \text{and}$$

$$1 - F(x) \approx \Phi(-w) - \phi(w) \sum_{k=0}^{\infty} (A_k - B_k), \quad \text{where}$$

$$w = \operatorname{sgn}(s)\sqrt{2(sK^{(1)}(s) - K(s))}; \quad t = \sqrt{K^{(2)}(s)}; \quad u = st.$$

The coefficients A_k, B_k and C_k are calculated as follows:

$$A_0 = \frac{1}{u}, \quad A_j = A_0 \sum_{n=0}^{2j} \left(\frac{1}{u}\right)^{2j-n} \sum_{m=0}^{n} d_{m,n} \, (-2)^{m+j} \left(\tfrac{1}{2}\right)_{m+j},$$

where $d_{m,n}$ is computed (with $d_{0,0} = 1$ and $d_{0,n} = 0$ for $n > 0$) from

$$d_{m+1,n+1} = \frac{1}{n+1} \sum_{j=1}^{n-m+1} \frac{j}{(j+2)!} \frac{K^{(j+2)}(s)}{t^{j+2}} \, d_{m,n-j+1}, \quad 0 \le m \le n.$$

$$B_0 = \frac{1}{w}, \quad B_j = \frac{-2(j - \frac{1}{2})B_{j-1}}{w^2}.$$

$$C_0 = \frac{1}{t}, \quad C_j = \sum_{m=1}^{2j} d_{m,2j} \, (-2)^{m+j} \left(\tfrac{1}{2}\right)_{m+j}.$$

5.11.6 Jensen expansion: general approximation of the qtf and isf

`ctx.jensen_inv`(*q*, *s*, *x0*, *poly_t cgf_series*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Jensen approximation of the qtf and isf

Using the same notation as above, Jensen [380] derived the following approximation (see also Wang [792]):

$$F(x) \approx \Phi(r(s)), \quad \text{where}$$

$$w(s) = \operatorname{sgn}(s)\sqrt{2(sK'(s) - K(s))},$$

$$u(s) = s\sqrt{K''(s)},$$

$$r(s) = w(s) + \frac{v(s)}{w(s)}, \quad \text{where } v(s) = \log\frac{u(s)}{w(s)}.$$

To obtain an approximation to the α-quantile of $F(x)$, we start with a reasonable estimate, say, x_0, and convert this to the corresponding saddlepoint, say s_0, such that $K'(s_0) = x_0$.

Let z_α be the α-quantile of the standard normal distribution. We wish to solve for s in the equation

$$h(s) = w(s) + \frac{v(s)}{w(s)} - z_\alpha = 0.$$

Starting with $n = 0$, we compute successive approximations using Newton iterations as

$$s_{n+1} = s_n - \frac{h(s_n)}{h'(s_n)}, \quad \text{where}$$

$$h'(s) = w'(s) + \frac{w(s)u'(s) - u(s)w'(s)\,(v(s)+1)}{u(s)w(s)^2},$$

$$w'(s) = \operatorname{sgn}(s)\frac{sK''(s)}{\sqrt{2sK'(s) - 2K(s)}} = \operatorname{sgn}(s)\frac{sK''(s)}{w(s)},$$

$$u'(s) = \frac{sK^{(3)}(s) + 2K''(s)}{2\sqrt{K''(s)}}.$$

The final estimate x_{n+1} of the α-quantile of $F(x)$ is then obtained as $x_{n+1} = K'(s_{n+1})$.

5.11.7 Box-Davis expansion: general approximation of the pdf, cdf and sf

`ctx.box_davis_expansion`(*x*, *f*, *rho*, *omega*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the Box-Davis approximation of the pdf, cdf and sf

The Box-Davis expansion supports a general class of random variables, say X, with absolutely continuous distribution functions, for which the logarithm of the characteristic function, say $\log[C_X(t)]$, may be validly represented by an asymptotic series

$$\log[C_X(t)] \sim -\frac{1}{2}f\log(1-2it) + \sum_{r=1}^{\infty}\omega_r(\rho)\left[(1-2it)^{-r} - 1\right],$$

corresponding to a limiting chi-squared distribution with f degrees of freedom, where ρ is chosen in such a way that $\omega_1 = 0$, whenever this is possible.

The first comprehensive review of this type of expansion for the CDF is due to Box (see Box [35]). The corresponding expansion for the inverse CDF is due to Davis (see Davis [204]), who also gave algorithms for variables with characteristic functions

different from those considered by Box. We refer therefore to this system of asymptotic expansions as the Box-Davis expansion. Distributions covered by this expansion include the beta distribution, the distribution of the product of independent beta variables, Wilks' Lambda, Mauchley's test for sphericity, Bartletts test for equality of independent covariance matrices, Hotelling's T^2 and Pillai's V. For more information, see Anderson [9] and Ginzberg [349].

Let θ be a suitably chosen constant (which will usually be proportional to sample size). We define

$$a_0 = 1; \quad a_j = \frac{1}{j}\sum_{l=1}^{j} l\omega_l a_{j-l}, \quad j > 0.$$

$$\log(K_B) = -\sum_{j=1}^{r} \omega_j + O(\theta^{-r-1}),$$

Then X follows asymptotically (with $\theta \to \infty$) a χ^2-distribution with f degrees of freedom, and the pdf, CDF of M can be developed into asymptotic expansions including terms of order $O(\theta^{-r})$, depending only on f, ρ, K_B and a_j:

$$\text{pdf}_X(x)) = \rho K_B \sum_{j=0}^{r} a_j f_{\chi^2_{f+2j}}(\rho x) + O(\theta^{-r-1})$$

$$\text{cdf}_X(x) = K_B \sum_{j=0}^{r} a_j F_{\chi^2_{f+2j}}(\rho x) + O(\theta^{-r-1})$$

$$1 - \text{cdf}_X(x) = K_B \sum_{j=0}^{r} a_j \left[1 - F_{\chi^2_{f+2j}}(\rho x)\right] + O(\theta^{-r-1})$$

Gupta and Tang (see Gupta and Tang [356]) show that the above series are convergent as $r \to \infty$ for fixed $x < 4\pi\theta$.

When calling the following functions, only f, ρ and ω_j need to be supplied by the user. K_B and a_j are calculated internally.

5.11.8 Box-Davis expansion: general approximation of the qtf and isf (for $\omega_1 = 0$)

`ctx.box_davis_expansion_inv(`x, f, *rho*, *omega*`)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

The Box-Davis expansion can be inverted efficiently in a two-step procedure. In the first step, an initial estimate is obtained, using the procedure given below. In the second step, the initial estimate is used as starting point of a Newton iteration, using the functions of the previous section.

Assuming $\omega_1 = 0$, an approximation of the quantile function of X, $\text{qtf}_X(q))$, in terms of the quantiles of a χ^2-distribution with f degrees of freedom, say $u = \chi^2_{f,\alpha}$, can be given as $\text{qtf}_X(q)) = (u + 2S)/\rho + O(\theta^{-8})$, where S is a polynomial in u, whose coefficients depend only on f and ω_j (see Davis [204]).

$$\begin{aligned} S &= \omega_2 P_2(u) \\ &+ \omega_3 P_3(u) \\ &+ \omega_4 P_4(u) + \tfrac{1}{2}\omega_2^2 P_{2,2}(u) \\ &+ \omega_5 P_5(u) + \omega_3\omega_2 P_{3,2}(u) \\ &+ \omega_6 P_6(u) + \omega_4\omega_2 P_{4,2}(u) + \tfrac{1}{2}\omega_3^2 P_{3,3}(u) + \tfrac{1}{6}\omega_2^3 P_{2,2,2}(u) \\ &+ \omega_7 P_7(u) + \omega_5\omega_2 P_{5,2}(u) + \omega_4\omega_3 P_{4,3}(u) + \tfrac{1}{2}\omega_3\omega_2^2 P_{3,2,2}(u), \end{aligned}$$

$$P_r(u) = \frac{u}{f_1} + \ldots + \frac{u^r}{f_r}, \quad f_r = f(f+2)\ldots(f+2r-2),$$

$$P_{2,2} = -\frac{8u^4(f+3)}{(f_2 f_4)} + \frac{8u^3}{(f_2 f_3)} + \frac{6u^2}{(f f_2)} + \frac{2u}{f^2}$$

$$P_{3,2} = -\frac{12u^5(f+4)}{(f_2f_5)} - \frac{2u^4(f-6)}{(f_2f_4)} + \frac{2u^3(3f+10)}{(f_2f_3)} + \frac{6u^2}{(ff_2)} + \frac{2u}{f^2}$$

$$P_{4,2}(u) = -\frac{16u^6(f+5)}{f_2f_6} - \frac{4u^5(f-4}{f_2f_5}) + \frac{2u^4(3f+14)}{f_2f_4} + \frac{2u^3(3f+10)}{f_2f_3} + \frac{6u^2}{ff_2} + \frac{2u}{f^2},$$

$$P_{3,3}(u) = -\frac{6u^6(3f^2+30f+80)}{f_3f_6} - \frac{6u^5(f_2+2f-16)}{f_3f_5} + \frac{4u^4(f+12)}{f_2f_4} + \frac{4u^3(3f+8)}{f_2f_3} + \frac{6u^2}{ff_2} + \frac{2u}{f^2},$$

$$P_{2,2,2}(u) = \frac{32u^6(7f^2+62f+120)}{f_2^2f_6} - \frac{32u^5(2f^2+37f+96)}{f_2^2f_5} - \frac{8u^4(23f^2+124f+132)}{f_2^2f_4} - \frac{8u^3(f-10)}{ff_2f_3} + \frac{28u^2}{f^2f_2} + \frac{4u}{f^3},$$

$$P_{5,2} = -\frac{20u^7(f+6)}{(f_2f_7)} - \frac{2u^6(3f-10)}{(f_2f_6)} + \frac{2u}{f^2} + 2\sum_{r=2}^{5}\frac{u^r(3f+4r-2)}{f_2f_r}$$

$$P_{4,3}(u) = -\frac{24u^7(f_2+12f+40)}{f_3f_7} - \frac{2u^6(5f_2+18f-80)}{f_3f_6} + \frac{2u^5(f_2+42f+176)}{f_3f_5} + \frac{4u^4(3f+16)}{f_2f_4} + \frac{4u^3(3f+8)}{f_2f_3} + \frac{6u^2}{ff_2} + \frac{2u}{f^2},$$

$$P_{3,2,2}(u) = \frac{192u^7(2f^3+31f^2+154f+240)}{f_2f_3f_7} - \frac{16u^6(4f^3+153f^2+1106f+2160)}{f_2f_3f_6} - \frac{8u^5(35f^3420f^2+1540f+1632)}{f_2f_3f_5} - \frac{4u^4(25f^2+80f+12)}{f_2^2f_4} + \frac{4u^3(7f+38)}{ff_2f_3} + \frac{28u^2}{f^2f_3} + \frac{4u}{f^3}.$$

When calling the following functions, only f, ρ and ω_j need to be supplied by the user. K_B and a_j are calculated internally.

5.12 Mpmath: Function approximation

5.12.1 Pade approximation

`ctx.pade`(a, L, M)

where `ctx` is `dec`, `mpm`, or `gmp`.

Computes a Pade approximation of degree (L, M) to a function. Given at least $L + M + 1$ Taylor coefficients a approximating a function $A(x)$, `pade()` returns coefficients of polynomials P, Q satisfying

$$P = \sum_{k=0}^{L} p_k x^k$$
$$Q = \sum_{k=0}^{M} q_k x^k$$
$$Q_0 = 1$$
$$A(x)Q(x) = P(x) + O(x^{L+M+1})$$

$P(x)/Q(x)$ can provide a good approximation to an analytic function beyond the radius of convergence of its Taylor series (example from G.A. Baker 'Essentials of Pade Approximants' Academic Press, Ch.1A):

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> one = mpf(1)
>>> def f(x):
...     return sqrt((one + 2*x)/(one + x))
...
>>> a = taylor(f, 0, 6)
>>> p, q = pade(a, 3, 3)
>>> x = 10
>>> polyval(p[::-1], x)/polyval(q[::-1], x)
1.38169105566806
>>> f(x)
1.38169855941551
```

5.12.2 Chebyshev approximation

`ctx.chebyfit`(f, *interval*, N, *error=False*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Computes a polynomial of degree $N - 1$ that approximates the given function f on the interval $[a, b]$. With `error=True`, `chebyfit()` also returns an accurate estimate of the maximum absolute error; that is, the maximum value of $|f(x) - P(x)|$ for $x \in [a, b]$.

`chebyfit()` uses the Chebyshev approximation formula, which gives a nearly optimal solution: that is, the maximum error of the approximating polynomial is very close to the smallest possible for any polynomial of the same degree.

Chebyshev approximation is very useful if one needs repeated evaluation of an expensive function, such as function defined implicitly by an integral or a differential equation. (For example, it could be used to turn a slow mpmath function into a fast machine-precision version of the same.)

Examples

Here we use `chebyfit()` to generate a low-degree approximation of $f(x) = \cos(x)$, valid on the interval $[1, 2]$:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> poly, err = chebyfit(cos, [1, 2], 5, error=True)
```

(continues on next page)

(continued from previous page)

```
>>> nprint(poly)
[0.00291682, 0.146166, -0.732491, 0.174141, 0.949553]
>>> nprint(err, 12)
1.61351758081e-5
```

The polynomial can be evaluated using `polyval`:

```
>>> nprint(polyval(poly, 1.6), 12)
-0.0291858904138
>>> nprint(cos(1.6), 12)
-0.0291995223013
```

Sampling the true error at 1000 points shows that the error estimate generated by `chebyfit` is remarkably good:

```
>>> error = lambda x: abs(cos(x) - polyval(poly, x))
>>> nprint(max([error(1+n/1000.) for n in range(1000)]), 12)
1.61349954245e-5
```

Choice of degree

The degree N can be set arbitrarily high, to obtain an arbitrarily good approximation. As a rule of thumb, an N-term Chebyshev approximation is good to $N/(b-a)$ decimal places on a unit interval (although this depends on how well-behaved f is). The cost grows accordingly: `chebyfit` evaluates the function $(N^2)/2$ times to compute the coefficients and an additional N times to estimate the error.

Possible issues

One should be careful to use a sufficiently high working precision both when calling `chebyfit` and when evaluating the resulting polynomial, as the polynomial is sometimes ill-conditioned. It is for example difficult to reach 15-digit accuracy when evaluating the polynomial using machine precision floats, no matter the theoretical accuracy of the polynomial. (The option to return the coefficients in Chebyshev form should be made available in the future.)

It is important to note the Chebyshev approximation works poorly if f is not smooth. A function containing singularities, rapid oscillation, etc can be approximated more effectively by multiplying it by a weight function that cancels out the nonsmooth features, or by dividing the interval into several segments.

5.12.3 Fourier series

`ctx.fourier`(*f*, *interval*, *N*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Computes the Fourier series of degree N of the given function on the interval $[a, b]$. More precisely, `fourier()` returns two lists (c, s) of coefficients (the cosine series and sine series, respectively), such that

$$f(x) \sim \sum_{k=0}^{N} c_k \cos(kmx) + s_k \sin(kmx)$$

where $m = 2\pi/(b-a)$.

Note that many texts define the first coefficient as $2c_0$ instead of c_0. The easiest way to evaluate the computed series correctly is to pass it to `fourierval()`.

Examples

The function $f(x) = x$ has a simple Fourier series on the standard interval $[-\pi, \pi]$. The cosine coefficients are all zero (because the function has odd symmetry), and the sine coefficients are rational numbers:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> c, s = fourier(lambda x: x, [-pi, pi], 5)
>>> nprint(c)
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
>>> nprint(s)
[0.0, 2.0, -1.0, 0.666667, -0.5, 0.4]
```

This computes a Fourier series of a nonsymmetric function on a nonstandard interval:

```
>>> I = [-1, 1.5]
>>> f = lambda x: x**2 - 4*x + 1
>>> cs = fourier(f, I, 4)
>>> nprint(cs[0])
[0.583333, 1.12479, -1.27552, 0.904708, -0.441296]
>>> nprint(cs[1])
[0.0, -2.6255, 0.580905, 0.219974, -0.540057]
```

It is instructive to plot a function along with its truncated Fourier series:

```
>>> plot([f, lambda x: fourierval(cs, I, x)], I)
```

Fourier series generally converge slowly (and may not converge pointwise). For example, if $f(x) = \cosh(x)$, a 10-term Fourier series gives an L^2 error corresponding to 2-digit accuracy:

```
>>> I = [-1, 1]
>>> cs = fourier(cosh, I, 9)
>>> g = lambda x: (cosh(x) - fourierval(cs, I, x))**2
>>> nprint(sqrt(quad(g, I)))
0.00467963
```

`fourier()` uses numerical quadrature. For nonsmooth functions, the accuracy (and speed) can be improved by including all singular points in the interval specification:

```
>>> nprint(fourier(abs, [-1, 1], 0), 10)
([0.5000441648], [0.0])
>>> nprint(fourier(abs, [-1, 0, 1], 0), 10)
([0.5], [0.0])
```

5.12.4 Fourier series evaluation

`ctx.fourierval`(*series*, *interval*, *x*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Evaluates a Fourier series (in the format computed by by `fourier()` for the given interval) at the point x.

The series should be a pair (c, s) where c is the cosine series and s is the sine series. The two lists need not have the same length.

5.13 Mpmath: Sums, products, limits and extrapolation

5.13.1 Summation of infinite series

`ctx.nsum`(*f*, **intervals*, ***options*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: Wikipedia [1117].

Computes the sum

$$S = \sum_{k=a}^{b} f(k)$$

where (a, b) = *interval*, and where $a = -\infty$ and/or $b = \infty$ are allowed, or more generally

$$S = \sum_{k_1=a_1}^{b_1} \cdots \sum_{k_n=a_n}^{b_n} f(k_1, \ldots, k_n)$$

if multiple intervals are given.

Two examples of infinite series that can be summed by `nsum()`, where the first converges rapidly and the second converges slowly, are:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> nsum(lambda n: 1/fac(n), [0, inf])
2.71828182845905
>>> nsum(lambda n: 1/n**2, [1, inf])
1.64493406684823
```

When appropriate, `nsum()` applies convergence acceleration to accurately estimate the sums of slowly convergent series. If the series is finite, `nsum()` currently does not attempt to perform any extrapolation, and simply calls `fsum()`.

Multidimensional infinite series are reduced to a single-dimensional series over expanding hypercubes; if both infinite and finite dimensions are present, the finite ranges are moved innermost. For more advanced control over the summation order, use nested calls to `nsum()`, or manually rewrite the sum as a single-dimensional series.

Options

tol
Desired maximum final error. Defaults roughly to the epsilon of the working precision.

method
Which summation algorithm to use (described below). Default: `'richardson+shanks'`.

maxterms
Cancel after at most this many terms. Default: 10*dps.

steps
An iterable giving the number of terms to add between each extrapolation attempt. The default sequence is [10, 20, 30, 40, ...]. For example, if you know that approximately 100 terms will be required, efficiency might be improved by setting this to [100, 10]. Then the first extrapolation will be performed after 100 terms, the second after 110, etc.

verbose
Print details about progress.

ignore
If enabled, any term that raises `ArithmeticError` or `ValueError` (e.g. through division by zero) is replaced by a zero. This is convenient for lattice sums with a singular term near the origin.

Methods

Unfortunately, an algorithm that can efficiently sum any infinite series does not exist. `nsum()` implements several different algorithms that each work well in different cases. The *method* keyword argument selects a method.

The default method is `'r+s'`, i.e. both Richardson extrapolation and Shanks transformation is attempted. A slower method that handles more cases is `'r+s+e'`. For very high precision summation, or if the summation needs to be fast (for example if multiple sums need to be evaluated), it is a good idea to investigate which one method works best and only use that.

`'richardson'` / `'r'`:
Uses Richardson extrapolation. Provides useful extrapolation when $f(k) \sim P(k)/Q(k)$ or when $f(k) \sim (-1)^k P(k)/Q(k)$ for polynomials P and Q. See `richardson()` for additional information.

`'shanks'` / `'s'`:
Uses Shanks transformation. Typically provides useful extrapolation when $f(k) \sim c^k$ or when successive terms alternate signs. Is able to sum some divergent series. See `shanks()` for additional information.

`'levin'` / `'l'`:
Uses the Levin transformation. It performs better than the Shanks transformation for logarithmic convergent or alternating divergent series. The `'levin_variant'`-keyword selects the variant. Valid choices are "u", "t", "v" and "all" whereby "all" uses all three u,t and v simultanously (This is good for performance comparison in conjunction with "verbose=True"). Instead of the Levin transform one can also use the Sidi-S transform by selecting the method `'sidi'`. See `levin()` for additional details.

`'alternating'` / `'a'`:
This is the convergence acceleration of alternating series developped by Cohen, Villegras and Zagier. See `cohen_alt()` for additional details.

`'euler-maclaurin'` / `'e'`:
Uses the Euler-Maclaurin summation formula to approximate the remainder sum by an integral. This requires high-order numerical derivatives and numerical integration. The advantage of this algorithm is that it works regardless of the decay rate of f, as long as f is sufficiently smooth. See `sumem()` for additional information.

`'direct'` / `'d'`:
Does not perform any extrapolation. This can be used (and should only be used for) rapidly convergent series. The summation automatically stops when the terms decrease below the target tolerance.

Basic examples

A finite sum:

```
>>> nsum(lambda k: 1/k, [1, 6])
2.45
```

Summation of a series going to negative infinity and a doubly infinite series:

```
>>> nsum(lambda k: 1/k**2, [-inf, -1])
1.64493406684823
>>> nsum(lambda k: 1/(1+k**2), [-inf, inf])
3.15334809493716
```

`nsum()` handles sums of complex numbers:

```
>>> nsum(lambda k: (0.5+0.25j)**k, [0, inf])
(1.6 + 0.8j)
```

The following sum converges very rapidly, so it is most efficient to sum it by disabling convergence acceleration:

```
>>> mp.dps = 1000
>>> a = nsum(lambda k: -(-1)**k * k**2 / fac(2*k), [1, inf],
...     method='direct')
```

(continues on next page)

(continued from previous page)

```
>>> b = (cos(1)+sin(1))/4
>>> abs(a-b) < mpf('1e-998')
True
```

Examples with Richardson extrapolation

Richardson extrapolation works well for sums over rational functions, as well as their alternating counterparts:

```
>>> mp.dps = 50
>>> nsum(lambda k: 1 / k**3, [1, inf],
...     method='richardson')
1.2020569031595942853997381615114499907649862923405
>>> zeta(3)
1.2020569031595942853997381615114499907649862923405

>>> nsum(lambda n: (n + 3)/(n**3 + n**2), [1, inf],
...     method='richardson')
2.9348022005446793094172454999380755676568497036204
>>> pi**2/2-2
2.9348022005446793094172454999380755676568497036204

>>> nsum(lambda k: (-1)**k / k**3, [1, inf],
...     method='richardson')
-0.90154267736969571404980362113358749307373971925537
>>> -3*zeta(3)/4
-0.90154267736969571404980362113358749307373971925538
```

Examples with Shanks transformation

The Shanks transformation works well for geometric series and typically provides excellent acceleration for Taylor series near the border of their disk of convergence. Here we apply it to a series for $\log(2)$, which can be seen as the Taylor series for $\log(1+x)$ with $x = 1$:

```
>>> nsum(lambda k: -(-1)**k/k, [1, inf],
...     method='shanks')
0.69314718055994530941723212145817656807550013436025
>>> log(2)
0.69314718055994530941723212145817656807550013436025
```

Here we apply it to a slowly convergent geometric series:

```
>>> nsum(lambda k: mpf('0.995')**k, [0, inf],
...     method='shanks')
200.0
```

Finally, Shanks' method works very well for alternating series where $f(k) = (-1)^k g(k)$, and often does so regardless of the exact decay rate of $g(k)$:

```
>>> mp.dps = 15
>>> nsum(lambda k: (-1)**(k+1) / k**1.5, [1, inf],
...     method='shanks')
0.765147024625408
>>> (2-sqrt(2))*zeta(1.5)/2
0.765147024625408
```

The following slowly convergent alternating series has no known closed-form value. Evaluating the sum a second time at higher precision indicates that the value is probably correct:

```
>>> nsum(lambda k: (-1)**k / log(k), [2, inf],
...     method='shanks')
0.924299897222939
>>> mp.dps = 30
```

(continues on next page)

(continued from previous page)

```
>>> nsum(lambda k: (-1)**k / log(k), [2, inf],
...     method='shanks')
0.92429989722293885595957018136
```

Examples with Levin transformation

The following example calculates Euler's constant as the constant term in the Laurent expansion of zeta(s) at s=1. This sum converges extremly slow because of the logarithmic convergence behaviour of the Dirichlet series for zeta.

```
>>> mp.dps = 30
>>> z = mp.mpf(10) ** (-10)
>>> a = mp.nsum(lambda n: n**(-(1+z)), [1, mp.inf], method = "levin") - 1 / z
>>> print(mp.chop(a - mp.euler, tol = 1e-10))
0.0
```

Now we sum the zeta function outside its range of convergence (attention: This does not work at the negative integers!):

```
>>> mp.dps = 15
>>> w = mp.nsum(lambda n: n ** (2 + 3j), [1, mp.inf], method = "levin", levin_variant = "v")
>>> print(mp.chop(w - mp.zeta(-2-3j)))
0.0
```

The next example resummates an asymptotic series expansion of an integral related to the exponential integral.

```
>>> mp.dps = 15
>>> z = mp.mpf(10)
>>> # exact = mp.quad(lambda x: mp.exp(-x)/(1+x/z),[0,mp.inf])
>>> exact = z * mp.exp(z) * mp.expint(1,z) # this is the symbolic expression for the integral
>>> w = mp.nsum(lambda n: (-1) ** n * mp.fac(n) * z ** (-n), [0, mp.inf], \
...   method = "sidi", levin_variant = "t")
>>> print(mp.chop(w - exact))
0.0
```

Following highly divergent asymptotic expansion needs some care. Firstly we need copious amount of working precision. Secondly the stepsize must not be chosen to large, otherwise nsum may miss the point where the Levin transform converges and reach the point where only numerical garbage is produced due to numerical cancellation.

```
>>> mp.dps = 15
>>> z = mp.mpf(2)
>>> # exact = mp.quad(lambda x: mp.exp( -x * x / 2 - z * x ** 4), [0,mp.inf]) * 2 / mp.sqrt(2 * mp.pi)
>>>  # the following is the symbolic expression for the integral
>>> exact = mp.exp(mp.one / (32 * z)) * mp.besselk(mp.one / 4, \
...   mp.one / (32 * z)) / (4 * mp.sqrt(z * mp.pi))
>>> w = mp.nsum(lambda n: (-z)**n * mp.fac(4 * n) / (mp.fac(n) * mp.fac(2 * n) * (4 ** n)),
...   [0, mp.inf], method = "levin", levin_variant = "t", workprec = 8*mp.prec,
...   steps = [2] + [1 for x in xrange(1000)])
>>> print(mp.chop(w - exact))
0.0
```

The hypergeometric function can also be summed outside its range of convergence:

```
>>> mp.dps = 15
>>> z = 2 + 1j
>>> exact = mp.hyp2f1(2 / mp.mpf(3), 4 / mp.mpf(3), 1 / mp.mpf(3), z)
>>> f = lambda n: mp.rf(2 / mp.mpf(3), n) * mp.rf(4 / mp.mpf(3), n) * z**n \
...   / (mp.rf(1 / mp.mpf(3), n) * mp.fac(n))
>>> v = mp.nsum(f, [0, mp.inf], method = "levin", steps = [10 for x in xrange(1000)])
>>> print(mp.chop(exact-v))
0.0
```

Examples with Cohen's alternating series resummation

The next example sums the alternating zeta function:

```
>>> v = mp.nsum(lambda n: (-1)**(n-1) / n, [1, mp.inf], method = "a")
>>> print(mp.chop(v - mp.log(2)))
0.0
```

The derivate of the alternating zeta function outside its range of convergence:

```
>>> v = mp.nsum(lambda n: (-1)**n * mp.log(n) * n, [1, mp.inf], method = "a")
>>> print(mp.chop(v - mp.diff(lambda s: mp.altzeta(s), -1)))
0.0
```

Examples with Euler-Maclaurin summation

The sum in the following example has the wrong rate of convergence for either Richardson or Shanks to be effective.

```
>>> f = lambda k: log(k)/k**2.5
>>> mp.dps = 15
>>> nsum(f, [1, inf], method='euler-maclaurin')
0.38734195032621
>>> -diff(zeta, 2.5)
0.38734195032621
```

Increasing `steps` improves speed at higher precision:

```
>>> mp.dps = 50
>>> nsum(f, [1, inf], method='euler-maclaurin', steps=[250])
0.38734195032620997271199237593105101319948228874688
>>> -diff(zeta, 2.5)
0.38734195032620997271199237593105101319948228874688
```

Divergent series

The Shanks transformation is able to sum some *divergent* series. In particular, it is often able to sum Taylor series beyond their radius of convergence (this is due to a relation between the Shanks transformation and Pade approximations; see `pade()` for an alternative way to evaluate divergent Taylor series). Furthermore the Levin-transform examples above contain some divergent series resummation.

Here we apply it to $\log(1+x)$ far outside the region of convergence:

```
>>> mp.dps = 50
>>> nsum(lambda k: -(-9)**k/k, [1, inf],
...     method='shanks')
2.3025850929940456840179914546843642076011014886288
>>> log(10)
2.3025850929940456840179914546843642076011014886288
```

A particular type of divergent series that can be summed using the Shanks transformation is geometric series. The result is the same as using the closed-form formula for an infinite geometric series:

```
>>> mp.dps = 15
>>> for n in range(-8, 8):
...     if n == 1:
...         continue
...     print("%s %s %s" % (mpf(n), mpf(1)/(1-n),
...         nsum(lambda k: n**k, [0, inf], method='shanks')))
...
-8.0 0.111111111111111 0.111111111111111
-7.0 0.125 0.125
-6.0 0.142857142857143 0.142857142857143
-5.0 0.166666666666667 0.166666666666667
-4.0 0.2 0.2
-3.0 0.25 0.25
-2.0 0.333333333333333 0.333333333333333
-1.0 0.5 0.5
```

(continues on next page)

(continued from previous page)

```
0.0 1.0 1.0
2.0 -1.0 -1.0
3.0 -0.5 -0.5
4.0 -0.333333333333333 -0.333333333333333
5.0 -0.25 -0.25
6.0 -0.2 -0.2
7.0 -0.166666666666667 -0.166666666666667
```

Multidimensional sums

Any combination of finite and infinite ranges is allowed for the summation indices:

```
>>> mp.dps = 15
>>> nsum(lambda x,y: x+y, [2,3], [4,5])
28.0
>>> nsum(lambda x,y: x/2**y, [1,3], [1,inf])
6.0
>>> nsum(lambda x,y: y/2**x, [1,inf], [1,3])
6.0
>>> nsum(lambda x,y,z: z/(2**x*2**y), [1,inf], [1,inf], [3,4])
7.0
>>> nsum(lambda x,y,z: y/(2**x*2**z), [1,inf], [3,4], [1,inf])
7.0
>>> nsum(lambda x,y,z: x/(2**z*2**y), [3,4], [1,inf], [1,inf])
7.0
```

Some nice examples of double series with analytic solutions or reductions to single-dimensional series (see MathWorld [1030]):

```
>>> nsum(lambda m, n: 1/2**(m*n), [1,inf], [1,inf])
1.60669515241529
>>> nsum(lambda n: 1/(2**n-1), [1,inf])
1.60669515241529

>>> nsum(lambda i,j: (-1)**(i+j)/(i**2+j**2), [1,inf], [1,inf])
0.278070510848213
>>> pi*(pi-3*ln2)/12
0.278070510848213

>>> nsum(lambda i,j: (-1)**(i+j)/(i+j)**2, [1,inf], [1,inf])
0.129319852864168
>>> altzeta(2) - altzeta(1)
0.129319852864168

>>> nsum(lambda i,j: (-1)**(i+j)/(i+j)**3, [1,inf], [1,inf])
0.0790756439455825
>>> altzeta(3) - altzeta(2)
0.0790756439455825

>>> nsum(lambda m,n: m**2*n/(3**m*(n*3**m+m*3**n)),
...     [1,inf], [1,inf])
0.28125
>>> mpf(9)/32
0.28125

>>> nsum(lambda i,j: fac(i-1)*fac(j-1)/fac(i+j),
...     [1,inf], [1,inf], workprec=400)
1.64493406684823
>>> zeta(2)
1.64493406684823
```

A hard example of a multidimensional sum is the Madelung constant in three dimensions (see MathWorld [1034]). The defining sum converges very slowly and only conditionally, so `nsum()` is lucky to obtain an accurate value through convergence acceleration. The second evaluation below uses a much more efficient, rapidly convergent 2D sum:

```
>>> nsum(lambda x,y,z: (-1)**(x+y+z)/(x*x+y*y+z*z)**0.5,
...     [-inf,inf], [-inf,inf], [-inf,inf], ignore=True)
-1.74756459463318
>>> nsum(lambda x,y: -12*pi*sech(0.5*pi * \
...     sqrt((2*x+1)**2+(2*y+1)**2))**2, [0,inf], [0,inf])
-1.74756459463318
```

Another example of a lattice sum in 2D:

```
>>> nsum(lambda x,y: (-1)**(x+y) / (x**2+y**2), [-inf,inf],
...     [-inf,inf], ignore=True)
-2.1775860903036
>>> -pi*ln2
-2.1775860903036
```

An example of an Eisenstein series:

```
>>> nsum(lambda m,n: (m+n*1j)**(-4), [-inf,inf], [-inf,inf],
...     ignore=True)
(3.1512120021539 + 0.0j)
```

5.13.2 Summation using the Euler-Maclaurin formula

ctx.sumem(*f*, *interval*, *tol=None*, *reject=10*, *integral=None*, *adiffs=None*, *bdiffs=None*, *verbose=False*, *error=False*, *_fast_abort=False*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Uses the Euler-Maclaurin formula to compute an approximation accurate to within `tol` (which defaults to the present epsilon) of the sum

$$S = \sum_{k=a}^{b} f(k)$$

where (a, b) are given by `interval` and a or b may be infinite. The approximation is

$$S \sim \int_a^b f(x)\,\mathrm{d}x + \frac{f(a)+f(b)}{2} + \sum_{k=1}^{\infty} \frac{B_{2k}}{(2k)!}\left(f^{(2k-1)}(b) - f^{(2k-1)}(a)\right).$$

The last sum in the Euler-Maclaurin formula is not generally convergent (a notable exception is if f is a polynomial, in which case Euler-Maclaurin actually gives an exact result).

The summation is stopped as soon as the quotient between two consecutive terms falls below *reject*. That is, by default (*reject* = 10), the summation is continued as long as each term adds at least one decimal.

Although not convergent, convergence to a given tolerance can often be "forced" if $b = \infty$ by summing up to $a + N$ and then applying the Euler-Maclaurin formula to the sum over the range $(a + N + 1, \ldots, \infty)$. This procedure is implemented by `nsum()`.

By default numerical quadrature and differentiation is used. If the symbolic values of the integral and endpoint derivatives are known, it is more efficient to pass the value of the integral explicitly as `integral` and the derivatives explicitly as `adiffs` and `bdiffs`. The derivatives should be given as iterables that yield $f(a), f'(a), f''(a), \ldots$ (and the equivalent for b).

Examples

Summation of an infinite series, with automatic and symbolic integral and derivative values (the second should be much faster):

```
>>> from xlcalcnet import *
>>> mp.dps = 50; mp.pretty = True
>>> sumem(lambda n: 1/n**2, [32, inf])
```

(continues on next page)

(continued from previous page)

```
0.031743366520302090126581680438741427141321886413417
>>> I = mpf(1)/32
>>> D = adiffs=((-1)**n*fac(n+1)*32**(-2-n) for n in range(999))
>>> sumem(lambda n: 1/n**2, [32, inf], integral=I, adiffs=D)
0.031743366520302090126581680438741427141321886413417
```

An exact evaluation of a finite polynomial sum:

```
>>> sumem(lambda n: n**5-12*n**2+3*n, [-100000, 200000])
10500155000624963999742499550000.0
>>> print(sum(n**5-12*n**2+3*n for n in range(-100000, 200001)))
10500155000624963999742499550000
```

5.13.3 Summation using the Abel-Plana formula

`ctx.sumap`(*f*, *interval*, *integral=None*, *error=False*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Evaluates an infinite series of an analytic summand f using the Abel-Plana formula

$$\sum_{k=0}^{\infty} f(k) = \int_0^{\infty} f(t)\mathrm{d}t + \frac{1}{2}f(0) + i\int_0^{\infty} \frac{f(it) - f(-it)}{e^{2\pi t} - 1}\mathrm{d}t.$$

Unlike the Euler-Maclaurin formula (see `sumem()`), the Abel-Plana formula does not require derivatives. However, it only works when $|f(it) - f(-it)|$ does not increase too rapidly with t.

Examples

The Abel-Plana formula is particularly useful when the summand decreases like a power of k; for example when the sum is a pure zeta function:

```
>>> from xlcalcnet import *
>>> mp.dps = 25; mp.pretty = True
>>> sumap(lambda k: 1/k**2.5, [1,inf])
1.3414872572509171797567 7
>>> zeta(2.5)
1.3414872572509171797567 7
>>> sumap(lambda k: 1/(k+1j)**(2.5+2.5j), [1,inf])
(-3.3853610685464733422860 84 - 0.7432082105196321803869551j)
>>> zeta(2.5+2.5j, 1+1j)
(-3.3853610685464733422860 84 - 0.7432082105196321803869551j)
```

If the series is alternating, numerical quadrature along the real line is likely to give poor results, so it is better to evaluate the first term symbolically whenever possible:

```
>>> n=3; z=-0.75
>>> I = expint(n,-log(z))
>>> chop(sumap(lambda k: z**k / k**n, [1,inf], integral=I))
-0.691703603690459451014144 8
>>> polylog(n,z)
-0.691703603690459451014144 8
```

5.13.4 Products

`ctx.nprod`(*f*, *interval*, *nsum=False*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Computes the product

$$P = \prod_{k=a}^{b} f(k)$$

where (a, b) = *interval*, and where $a = -\infty$ and/or $b = \infty$ are allowed.

By default, `nprod()` uses the same extrapolation methods as `nsum()`, except applied to the partial products rather than partial sums, and the same keyword options as for `nsum()` are supported. If `nsum=True`, the product is instead computed via `nsum()` as

$$P = \exp\left(\sum_{k=a}^{b} \log(f(k))\right).$$

This is slower, but can sometimes yield better results. It is also required (and used automatically) when Euler-Maclaurin summation is requested.

Examples

A simple finite product:

```
>>> from xlcalcnet import *
>>> mp.dps = 25; mp.pretty = True
>>> nprod(lambda k: k, [1, 4])
24.0
```

A large number of infinite products have known exact values, and can therefore be used as a reference. Most of the following examples are taken from MathWorld (see MathWorld [1032]).

A few infinite products with simple values are:

```
>>> 2*nprod(lambda k: (4*k**2)/(4*k**2-1), [1, inf])
3.14159265358979323846264
>>> nprod(lambda k: (1+1/k)**2/(1+2/k), [1, inf])
2.0
>>> nprod(lambda k: (k**3-1)/(k**3+1), [2, inf])
0.666666666666666666666667
>>> nprod(lambda k: (1-1/k**2), [2, inf])
0.5
```

Next, several more infinite products with more complicated values:

```
>>> nprod(lambda k: exp(1/k**2), [1, inf]); exp(pi**2/6)
5.18066831789711574841662
5.18066831789711574841662

>>> nprod(lambda k: (k**2-1)/(k**2+1), [2, inf]); pi*csch(pi)
0.272029054982133162950236
0.272029054982133162950236

>>> nprod(lambda k: (k**4-1)/(k**4+1), [2, inf])
0.848054049352900392129650
>>> pi*sinh(pi)/(cosh(sqrt(2)*pi)-cos(sqrt(2)*pi))
0.848054049352900392129650

>>> nprod(lambda k: (1+1/k+1/k**2)**2/(1+2/k+3/k**2), [1, inf])
1.84893618285824448522493
>>> 3*sqrt(2)*cosh(pi*sqrt(3)/2)**2*csch(pi*sqrt(2))/pi
1.84893618285824448522493

>>> nprod(lambda k: (1-1/k**4), [2, inf]); sinh(pi)/(4*pi)
0.919019477593744430173924
0.919019477593744430173924
```

(continues on next page)

(continued from previous page)

```
>>> nprod(lambda k: (1-1/k**6), [2, inf])
0.98268427774219251832447 59
>>> (1+cosh(pi*sqrt(3)))/(12*pi**2)
0.98268427774219251832447 59

>>> nprod(lambda k: (1+1/k**2), [2, inf]); sinh(pi)/(2*pi)
1.83803895518748886034784 9
1.83803895518748886034784 9

>>> nprod(lambda n: (1+1/n)**n * exp(1/(2*n)-1), [1, inf])
1.44725592689036529895913 8
>>> exp(1+euler/2)/sqrt(2*pi)
1.44725592689036529895913 8
```

The following two products are equivalent and can be evaluated in terms of a Jacobi theta function. Pi can be replaced by any value (as long as convergence is preserved):

```
>>> nprod(lambda k: (1-pi**-k)/(1+pi**-k), [1, inf])
0.38384512074816724047786 86
>>> nprod(lambda k: tanh(k*log(pi)/2), [1, inf])
0.38384512074816724047786 86
>>> jtheta(4,0,1/pi)
0.38384512074816724047786 86
```

This product does not have a known closed form value:

```
>>> nprod(lambda k: (1-1/2**k), [1, inf])
0.28878809508660242127889 97
```

A product taken from $-\infty$:

```
>>> nprod(lambda k: 1-k**(-3), [-inf,-2])
0.80939659736629010957868 05
>>> cosh(pi*sqrt(3)/2)/(3*pi)
0.80939659736629010957868 05
```

A doubly infinite product:

```
>>> nprod(lambda k: exp(1/(1+k**2)), [-inf, inf])
23.414326882318643374200 35
>>> exp(pi/tanh(pi))
23.414326882318643374200 35
```

A product requiring the use of Euler-Maclaurin summation to compute an accurate value:

```
>>> nprod(lambda k: (1-1/k**2.5), [2, inf], method='e')
0.69615511133623105289812 5
```

5.13.5 Limits, general

`ctx.`**`limit`**(*f*, *x*, *direction=1*, *exp=False*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Computes an estimate of the limit

$$\lim_{t \to x} f(t)$$

where x may be finite or infinite.

For finite x, `limit()` evaluates $f(x + d/n)$ for consecutive integer values of n, where the approach direction d may be specified using the *direction* keyword argument. For infinite x, `limit()` evaluates values of $f(\mathrm{sign}(x) \cdot n)$.

If the approach to the limit is not sufficiently fast to give an accurate estimate directly, `limit()` attempts to find the limit using Richardson extrapolation or the Shanks transformation. You can select between these methods using the *method* keyword (see documentation of `nsum()` for more information).

Options

The following options are available with essentially the same meaning as for `nsum()`: *tol*, *method*, *maxterms*, *steps*, *verbose*.

If the option *exp=True* is set, f will be sampled at exponentially spaced points $n = 2^1, 2^2, 2^3, \ldots$ instead of the linearly spaced points $n = 1, 2, 3, \ldots$. This can sometimes improve the rate of convergence so that `limit()` may return a more accurate answer (and faster). However, do note that this can only be used if f supports fast and accurate evaluation for arguments that are extremely close to the limit point (or if infinite, very large arguments).

Examples

A basic evaluation of a removable singularity:

```
>>> from xlcalcnet import *
>>> mp.dps = 30; mp.pretty = True
>>> limit(lambda x: (x-sin(x))/x**3, 0)
0.16666666666666666666666666667
```

Computing the exponential function using its limit definition:

```
>>> limit(lambda n: (1+3/n)**n, inf)
20.085536923187667740928529654
>>> exp(3)
20.085536923187667740928529654
```

A limit for π:

```
>>> f = lambda n: 2**(4*n+1)*fac(n)**4/(2*n+1)/fac(2*n)**2
>>> limit(f, inf)
3.1415926535897932384626433832
```

Calculating the coefficient in Stirling's formula:

```
>>> limit(lambda n: fac(n) / (sqrt(n)*(n/e)**n), inf)
2.5066282746310005024157652848
>>> sqrt(2*pi)
2.5066282746310005024157652848
```

Evaluating Euler's constant γ using the limit representation

$$\gamma = \lim_{n \to \infty} \left[\left(\sum_{k=1}^{n} \frac{1}{k} \right) - \log(n) \right]$$

(which converges notoriously slowly):

```
>>> f = lambda n: sum([mpf(1)/k for k in range(1,int(n)+1)]) - log(n)
>>> limit(f, inf)
0.57721566490153286060651209008
>>> +euler
0.57721566490153286060651209008
```

With default settings, the following limit converges too slowly to be evaluated accurately. Changing to exponential sampling however gives a perfect result:

```
>>> f = lambda x: sqrt(x**3+x**2)/(sqrt(x**3)+x)
>>> limit(f, inf)
0.99283115855833028112924968649
>>> limit(f, inf, exp=True)
1.0
```

5.13.6 Richardson extrapolation

`ctx.richardson`(*seq*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: Wikipedia [1114] and C.M. and Orszag [169], pp. 375-376.

Given a list `seq` of the first N elements of a slowly convergent infinite sequence, `richardson()` computes the N-term Richardson extrapolate for the limit.

`richardson()` returns (v, c) where v is the estimated limit and c is the magnitude of the largest weight used during the computation. The weight provides an estimate of the precision lost to cancellation. Due to cancellation effects, the sequence must be typically be computed at a much higher precision than the target accuracy of the extrapolation.

Applicability and issues

Richardson extrapolation only works for a specific type of sequence, namely one converging like partial sums of $P(1)/Q(1) + P(2)/Q(2) + \ldots$ where P and Q are polynomials. When the sequence does not convergence at such a rate `richardson()` generally produces garbage.

Richardson extrapolation has the advantage of being fast: the N-term extrapolate requires only $O(N)$ arithmetic operations, and usually produces an estimate that is accurate to $O(N)$ digits. Contrast with the Shanks transformation (see `shanks()`), which requires $O(N^2)$ operations.

`richardson()` is unable to produce an estimate for the approximation error. One way to estimate the error is to perform two extrapolations with slightly different N and comparing the results.

Richardson extrapolation does not work for oscillating sequences. As a simple workaround, `richardson()` detects if the last three elements do not differ monotonically, and in that case applies extrapolation only to the even-index elements.

Examples

Applying Richardson extrapolation to the Leibniz series for π:

```
>>> from xlcalcnet import *
>>> mp.dps = 30; mp.pretty = True
>>> S = [4*sum(mpf(-1)**n/(2*n+1) for n in range(m))
...     for m in range(1,30)]
>>> v, c = richardson(S[:10])
>>> v
3.2126984126984126984126984127
>>> nprint([v-pi, c])
[0.0711058, 2.0]

>>> v, c = richardson(S[:30])
>>> v
3.1415926546862405282995420622 6
>>> nprint([v-pi, c])
[1.09645e-9, 20833.3]
```

5.13.7 Shanks extrapolation

`ctx.shanks`(*seq*, *table=None*, *randomized=False*)

where `ctx` is `dec`, `mpm`, or `gmp`.

See also: Wikipedia [1118], MathWorld [814].

Given a list `seq` of the first N elements of a slowly convergent infinite sequence (A_k), `shanks()` computes the iterated Shanks transformation $S(A), S(S(A)), \ldots, S^{N/2}(A)$. The Shanks transformation often provides strong convergence acceleration, especially if the sequence is oscillating.

The iterated Shanks transformation is computed using the Wynn epsilon algorithm (see [1]). `shanks()` returns the full epsilon table generated by Wynn's algorithm, which can be read off as follows:

- The table is a list of lists forming a lower triangular matrix, where higher row and column indices correspond to more accurate values.
- The columns with even index hold dummy entries (required for the computation) and the columns with odd index hold the actual extrapolates.
- The last element in the last row is typically the most accurate estimate of the limit.
- The difference to the third last element in the last row provides an estimate of the approximation error.
- The magnitude of the second last element provides an estimate of the numerical accuracy lost to cancellation.

For convenience, so the extrapolation is stopped at an odd index so that `shanks(seq)[-1][-1]` always gives an estimate of the limit.

Optionally, an existing table can be passed to `shanks()`. This can be used to efficiently extend a previous computation after new elements have been appended to the sequence. The table will then be updated in-place.

The Shanks transformation

The Shanks transformation is defined as follows (see C.M. and Orszag [169], pp. 368-375): given the input sequence $(A_0, A_1, \ldots)$, the transformed sequence is given by

$$S(A_k) = \frac{A_{k+1}A_{k-1} - A_k^2}{A_{k+1} + A_{k-1} - 2A_k}$$

The Shanks transformation gives the exact limit A_∞ in a single step if $A_k = A + aq^k$. Note in particular that it extrapolates the exact sum of a geometric series in a single step.

Applying the Shanks transformation once often improves convergence substantially for an arbitrary sequence, but the optimal effect is obtained by applying it iteratively: $S(S(A_k)), S(S(S(A_k))), \ldots$.

Wynn's epsilon algorithm provides an efficient way to generate the table of iterated Shanks transformations. It reduces the computation of each element to essentially a single division, at the cost of requiring dummy elements in the table. See Graves-Morris *et al.* [352] for details.

Precision issues

Due to cancellation effects, the sequence must be typically be computed at a much higher precision than the target accuracy of the extrapolation.

If the Shanks transformation converges to the exact limit (such as if the sequence is a geometric series), then a division by zero occurs. By default, `shanks()` handles this case by terminating the iteration and returning the table it has generated so far. With *randomized=True*, it will instead replace the zero by a pseudorandom number close to zero. (TODO: find a better solution to this problem.)

Examples

We illustrate by applying Shanks transformation to the Leibniz series for π:

```
>>> from xlcalcnet import *
>>> mp.dps = 50
>>> S = [4*sum(mpf(-1)**n/(2*n+1) for n in range(m))
...     for m in range(1,30)]
>>>
>>> T = shanks(S[:7])
>>> for row in T:
...     nprint(row)
...
[-0.75]
[1.25, 3.16667]
[-1.75, 3.13333, -28.75]
[2.25, 3.14524, 82.25, 3.14234]
[-2.75, 3.13968, -177.75, 3.14139, -969.937]
[3.25, 3.14271, 327.25, 3.14166, 3515.06, 3.14161]
```

The extrapolated accuracy is about 4 digits, and about 4 digits may have been lost due to cancellation:

```
>>> L = T[-1]
>>> nprint([abs(L[-1] - pi), abs(L[-1] - L[-3]), abs(L[-2])])
[2.22532e-5, 4.78309e-5, 3515.06]
```

Now we extend the computation:

```
>>> T = shanks(S[:25], T)
>>> L = T[-1]
>>> nprint([abs(L[-1] - pi), abs(L[-1] - L[-3]), abs(L[-2])])
[3.75527e-19, 1.48478e-19, 2.96014e+17]
```

The value for pi is now accurate to 18 digits. About 18 digits may also have been lost to cancellation.

Here is an example with a geometric series, where the convergence is immediate (the sum is exactly 1):

```
>>> mp.dps = 15
>>> for row in shanks([0.5, 0.75, 0.875, 0.9375, 0.96875]):
...     nprint(row)
[4.0]
[8.0, 1.0]
```

5.13.8 Levin extrapolation

`ctx.levin`(*seq*, *table=None*, *randomized=False*)

where `ctx` is `dec`, `mpm`, or `gmp`.

This interface implements Levin's (nonlinear) sequence transformation for convergence acceleration and summation of divergent series. It performs better than the Shanks/Wynn-epsilon algorithm for logarithmic convergent or alternating divergent series.

See also: Weniger [1078], Sidi [500], Homeier [374].

Let *A* be the series we want to sum:

$$A = \sum_{k=0}^{\infty} a_k$$

Attention: all a_k must be non-zero!

Let s_n be the partial sums of this series:

$$s_n = \sum_{k=0}^{n} a_k.$$

Methods

Calling `levin` returns an object with the following methods.

`update(...)` works with the list of individual terms a_k of *A*, and `update_step(...)` works with the list of partial sums s_k of *A*:

```
v, e = ...update([a_0, a_1,..., a_k])
v, e = ...update_psum([s_0, s_1,..., s_k])
```

`step(...)` works with the individual terms a_k and `step_psum(...)` works with the partial sums s_k:

```
v, e = ...step(a_k)
v, e = ...step_psum(s_k)
```

v is the current estimate for A, and e is an error estimate which is simply the difference between the current estimate and the last estimate. One should not mix `update`, `update_psum`, `step` and `step_psum`.

A word of caution

One can only hope for good results (i.e. convergence acceleration or resummation) if the s_n have some well defind asymptotic behavior for large n and are not erratic or random. Furthermore one usually needs very high working precision because of the numerical cancellation. If the working precision is insufficient, levin may produce silently numerical garbage. Furthermore even if the Levin-transformation converges, in the general case there is no proof that the result is mathematically sound. Only for very special classes of problems one can prove that the Levin-transformation converges to the expected result (for example Stieltjes-type integrals). Furthermore the Levin-transform is quite expensive (i.e. slow) in comparison to Shanks/Wynn-epsilon, Richardson & co. In summary one can say that the Levin-transformation is powerful but unreliable and that it may need a copious amount of working precision.

The Levin transform has several variants differing in the choice of weights. Some variants are better suited for the possible flavours of convergence behaviour of A than other variants:

```
convergence behaviour   levin-u   levin-t   levin-v   shanks/wynn-epsilon

logarithmic               +         -         +          -
linear                    +         +         +          +
alternating divergent     +         +         +          +

    "+" means the variant is suitable,"-" means the variant is not suitable;
    for comparison the Shanks/Wynn-epsilon transform is listed, too.
```

The variant is controlled though the variant keyword (i.e. `variant="u"`, `variant="t"` or `variant="v"`). Overall "u" is probably the best choice.

Finally it is possible to use the Sidi-S transform instead of the Levin transform by using the keyword `method='sidi'`. The Sidi-S transform works better than the Levin transformation for some divergent series (see the examples).

Parameters:

```
method      "levin" or "sidi" chooses either the Levin or the Sidi-S transformation
variant     "u","t" or "v" chooses the weight variant.
```

The Levin transform is also accessible through the nsum interface. `method="l"` or `method="levin"` select the normal Levin transform while `method="sidi"` selects the Sidi-S transform. The variant is in both cases selected through the levin_variant keyword. The stepsize in `nsum()` must not be chosen too large, otherwise it will miss the point where the Levin transform converges resulting in numerical overflow/garbage. For highly divergent series a copious amount of working precision must be chosen.

Examples

First we sum the zeta function:

```
>>> from mpmath import mp
>>> mp.prec = 53
>>> eps = mp.mpf(mp.eps)
>>> with mp.extraprec(2 * mp.prec): # levin needs a high working precision
...     L = mp.levin(method = "levin", variant = "u")
...     S, s, n = [], 0, 1
...     while 1:
...         s += mp.one / (n * n)
...         n += 1
...         S.append(s)
...         v, e = L.update_psum(S)
...         if e < eps:
...             break
...         if n > 1000: raise RuntimeError("iteration limit exceeded")
>>> print(mp.chop(v - mp.pi ** 2 / 6))
0.0
```

(continues on next page)

(continued from previous page)

```
>>> w = mp.nsum(lambda n: 1 / (n*n), [1, mp.inf], method = "levin", levin_variant = "u")
>>> print(mp.chop(v - w))
0.0
```

Now we sum the zeta function outside its range of convergence (attention: This does not work at the negative integers!):

```
>>> eps = mp.mpf(mp.eps)
>>> with mp.extraprec(2 * mp.prec): # levin needs a high working precision
...     L = mp.levin(method = "levin", variant = "v")
...     A, n = [], 1
...     while 1:
...         s = mp.mpf(n) ** (2 + 3j)
...         n += 1
...         A.append(s)
...         v, e = L.update(A)
...         if e < eps:
...             break
...         if n > 1000: raise RuntimeError("iteration limit exceeded")
>>> print(mp.chop(v - mp.zeta(-2-3j)))
0.0
>>> w = mp.nsum(lambda n: n ** (2 + 3j), [1, mp.inf], method = "levin", levin_variant = "v")
>>> print(mp.chop(v - w))
0.0
```

Now we sum the divergent asymptotic expansion of an integral related to the exponential integral (see also Sidi [500] p.373). The Sidi-S transform works best here:

```
>>> z = mp.mpf(10)
>>> exact = mp.quad(lambda x: mp.exp(-x)/(1+x/z),[0,mp.inf])
>>> # exact = z * mp.exp(z) * mp.expint(1,z) # this is the symbolic expression for the integral
>>> eps = mp.mpf(mp.eps)
>>> with mp.extraprec(2 * mp.prec): # high working precisions are mandatory for divergent resummation
...     L = mp.levin(method = "sidi", variant = "t")
...     n = 0
...     while 1:
...         s = (-1)**n * mp.fac(n) * z ** (-n)
...         v, e = L.step(s)
...         n += 1
...         if e < eps:
...             break
...         if n > 1000: raise RuntimeError("iteration limit exceeded")
>>> print(mp.chop(v - exact))
0.0
>>> w = mp.nsum(lambda n: (-1) ** n * mp.fac(n) * z ** (-n), [0, mp.inf], \
...   method = "sidi", levin_variant = "t")
>>> print(mp.chop(v - w))
0.0
```

Another highly divergent integral is also summable:

```
>>> z = mp.mpf(2)
>>> eps = mp.mpf(mp.eps)
>>> exact = mp.quad(lambda x: mp.exp( -x * x / 2 - z * x ** 4), [0,mp.inf]) * 2 / mp.sqrt(2 * mp.pi)
>>> # exact = mp.exp(mp.one / (32 * z)) * mp.besselk(mp.one / 4, mp.one / (32 * z))
>>> # / (4 * mp.sqrt(z * mp.pi)) # this is the symbolic expression for the integral
>>> with mp.extraprec(7 * mp.prec):
>>> # we need copious amount of precision to sum this highly divergent series
...     L = mp.levin(method = "levin", variant = "t")
...     n, s = 0, 0
...     while 1:
...         s += (-z)**n * mp.fac(4 * n) / (mp.fac(n) * mp.fac(2 * n) * (4 ** n))
...         n += 1
...         v, e = L.step_psum(s)
...         if e < eps:
```

(continues on next page)

(continued from previous page)

```
...             break
...         if n > 1000: raise RuntimeError("iteration limit exceeded")
>>> print(mp.chop(v - exact))
0.0
>>> w = mp.nsum(lambda n: (-z)**n * mp.fac(4 * n) / (mp.fac(n) * mp.fac(2 * n) * (4 ** n)),
...   [0, mp.inf], method = "levin", levin_variant = "t", workprec = 8*mp.prec, \
...   steps = [2] + [1 for x in xrange(1000)])
>>> print(mp.chop(v - w))
0.0
```

These examples run with 15-20 decimal digits precision. For higher precision the working precision must be raised.

Examples for nsum

Here we calculate Euler's constant as the constant term in the Laurent expansion of $\zeta(s)$ at $s = 1$. This sum converges extremly slowly because of the logarithmic convergence behaviour of the Dirichlet series for zeta:

```
>>> mp.dps = 30
>>> z = mp.mpf(10) ** (-10)
>>> a = mp.nsum(lambda n: n**(-(1+z)), [1, mp.inf], method = "l") - 1 / z
>>> print(mp.chop(a - mp.euler, tol = 1e-10))
0.0
```

The Sidi-S transform performs excellently for the alternating series of $\log(2)$:

```
>>> a = mp.nsum(lambda n: (-1)**(n-1) / n, [1, mp.inf], method = "sidi")
>>> print(mp.chop(a - mp.log(2)))
0.0
```

Hypergeometric series can also be summed outside their range of convergence. The stepsize in `nsum()` must not be chosen too large, otherwise it will miss the point where the Levin transform converges resulting in numerical overflow/garbage:

```
>>> z = 2 + 1j
>>> exact = mp.hyp2f1(2 / mp.mpf(3), 4 / mp.mpf(3), 1 / mp.mpf(3), z)
>>> f = lambda n: mp.rf(2 / mp.mpf(3), n) * mp.rf(4 / mp.mpf(3), n) * z**n \
...   / (mp.rf(1 / mp.mpf(3), n) * mp.fac(n))
>>> v = mp.nsum(f, [0, mp.inf], method = "levin", steps = [10 for x in xrange(1000)])
>>> print(mp.chop(exact-v))
0.0
```

5.13.9 Cohan alternating extrapolation

`ctx.cohen_alt`(*seq*, *table=None*, *randomized=False*)

where `ctx` is `dec`, `mpm`, or `gmp`.

This interface implements the convergence acceleration of alternating series as described in Cohen *et al.* [182].

This series transformation works only well if the individual terms of the series have an alternating sign. It belongs to the class of linear series transformations (in contrast to the Shanks/Wynn-epsilon or Levin transform). This series transformation is also able to sum some types of divergent series. See the paper under which conditions this resummation is mathematical sound.

Let A be the series we want to sum:

$$A = \sum_{k=0}^{\infty} a_k$$

Let s_n be the partial sums of this series:

$$s_n = \sum_{k=0}^{n} a_k.$$

Interface

Calling `cohen_alt` returns an object with the following methods.

Then `update(...)` works with the list of individual terms a_k and `update_psum(...)` works with the list of partial sums s_k:

```
v, e = ...update([a_0, a_1,..., a_k])
v, e = ...update_psum([s_0, s_1,..., s_k])
```

v is the current estimate for A, and e is an error estimate which is simply the difference between the current estimate and the last estimate.

Examples

Here we compute the alternating zeta function using `update_psum`:

```
>>> from mpmath import mp
>>> AC = mp.cohen_alt()
>>> S, s, n = [], 0, 1
>>> while 1:
...     s += -((-1) ** n) * mp.one / (n * n)
...     n += 1
...     S.append(s)
...     v, e = AC.update_psum(S)
...     if e < mp.eps:
...         break
...     if n > 1000: raise RuntimeError("iteration limit exceeded")
>>> print(mp.chop(v - mp.pi ** 2 / 12))
0.0
```

Here we compute the product $\prod_{n=1}^{\infty} \Gamma(1+1/(2n-1))/\Gamma(1+1/(2n))$:

```
>>> A = []
>>> AC = mp.cohen_alt()
>>> n = 1
>>> while 1:
...     A.append( mp.loggamma(1 + mp.one / (2 * n - 1)))
...     A.append(-mp.loggamma(1 + mp.one / (2 * n)))
...     n += 1
...     v, e = AC.update(A)
...     if e < mp.eps:
...         break
...     if n > 1000: raise RuntimeError("iteration limit exceeded")
>>> v = mp.exp(v)
>>> print(mp.chop(v - 1.06215090557106, tol = 1e-12))
0.0
```

`cohen_alt` is also accessible through the `nsum()` interface:

```
>>> v = mp.nsum(lambda n: (-1)**(n-1) / n, [1, mp.inf], method = "a")
>>> print(mp.chop(v - mp.log(2)))
0.0
>>> v = mp.nsum(lambda n: (-1)**n / (2 * n + 1), [0, mp.inf], method = "a")
>>> print(mp.chop(v - mp.pi / 4))
0.0
>>> v = mp.nsum(lambda n: (-1)**n * mp.log(n) * n, [1, mp.inf], method = "a")
>>> print(mp.chop(v - mp.diff(lambda s: mp.altzeta(s), -1)))
0.0
```

5.14 Mpmath: Polynomials

Python code for `gmp` and `dec` is available in `Test_Python.numpy.py`.

5.14.1 Polynomial evaluation

`ctx.polyval`(*coeffs*, *x*, *derivative=False*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Given coefficients $[c_n, \ldots, c_2, c_1, c_0]$ and a number x, `polyval()` evaluates the polynomial

$$P(x) = c_n x^n + \ldots + c_2 x^2 + c_1 x + c_0.$$

If *derivative=True* is set, `polyval()` simultaneously evaluates $P(x)$ with the derivative, $P'(x)$, and returns the tuple $(P(x), P'(x))$.

```
>>> from xlcalcnet import *
>>> mp.pretty = True
>>> polyval([3, 0, 2], 0.5)
2.75
>>> polyval([3, 0, 2], 0.5, derivative=True)
(2.75, 3.0)
```

The coefficients and the evaluation point may be any combination of real or complex numbers.

5.14.2 Polynomial roots

`ctx.polyroots`(*coeffs*, *maxsteps=50*, *cleanup=True*, *extraprec=10*, *error=False*, *roots_init=None*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Computes all roots (real or complex) of a given polynomial.

The roots are returned as a sorted list, where real roots appear first followed by complex conjugate roots as adjacent elements. The polynomial should be given as a list of coefficients, in the format used by `polyval()`. The leading coefficient must be nonzero.

With *error=True*, `polyroots()` returns a tuple *(roots, err)* where *err* is an estimate of the maximum error among the computed roots.

Examples

Finding the three real roots of $x^3 - x^2 - 14x + 24$:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> nprint(polyroots([1,-1,-14,24]), 4)
[-4.0, 2.0, 3.0]
```

Finding the two complex conjugate roots of $4x^2 + 3x + 2$, with an error estimate:

```
>>> roots, err = polyroots([4,3,2], error=True)
>>> for r in roots:
...     print(r)
...
(-0.375 + 0.599478940414 09j)
(-0.375 - 0.599478940414 09j)
>>>
>>> err
2.22044604925031e-16
>>>
>>> polyval([4,3,2], roots[0])
(2.22044604925031e-16 + 0.0j)
>>> polyval([4,3,2], roots[1])
(2.22044604925031e-16 + 0.0j)
```

The following example computes all the 5th roots of unity; that is, the roots of $x^5 - 1$:

```
>>> mp.dps = 20
>>> for r in polyroots([1, 0, 0, 0, 0, -1]):
...     print(r)
...
1.0
(-0.8090169943749474241 + 0.58778525229247312917j)
(-0.8090169943749474241 - 0.58778525229247312917j)
(0.3090169943749474241 + 0.95105651629515357212j)
(0.3090169943749474241 - 0.95105651629515357212j)
```

Precision and conditioning

The roots are computed to the current working precision accuracy. If this accuracy cannot be achieved in `maxsteps` steps, then a `NoConvergence` exception is raised. The algorithm internally is using the current working precision extended by `extraprec`. If `NoConvergence` was raised, that is caused either by not having enough extra precision to achieve convergence (in which case increasing `extraprec` should fix the problem) or too low `maxsteps` (in which case increasing `maxsteps` should fix the problem), or a combination of both.

The user should always do a convergence study with regards to `extraprec` to ensure accurate results. It is possible to get convergence to a wrong answer with too low `extraprec`.

Provided there are no repeated roots, `polyroots()` can typically compute all roots of an arbitrary polynomial to high precision:

```
>>> mp.dps = 60
>>> for r in polyroots([1, 0, -10, 0, 1]):
...     print(r)
...
-3.14626436994197234232913506571557044551247712918732870123249
-0.31783724519578224472575761729617428837313337843343255487912 7
0.31783724519578224472575761729617428837313337843343255487912 7
3.14626436994197234232913506571557044551247712918732870123249
>>>
>>> sqrt(3) + sqrt(2)
3.14626436994197234232913506571557044551247712918732870123249
>>> sqrt(3) - sqrt(2)
0.31783724519578224472575761729617428837313337843343255487912 7
```

Algorithm

`polyroots()` implements the Durand-Kerner method (see Wikipedia [1089]), which uses complex arithmetic to locate all roots simultaneously. The Durand-Kerner method can be viewed as approximately performing simultaneous Newton iteration for all the roots. In particular, the convergence to simple roots is quadratic, just like Newton's method.

Although all roots are internally calculated using complex arithmetic, any root found to have an imaginary part smaller than the estimated numerical error is truncated to a real number (small real parts are also chopped). Real roots are placed first in the returned list, sorted by value. The remaining complex roots are sorted by their real parts so that conjugate roots end up next to each other.

5.15 Eigen/MinPack: non linear optimization

5.15.1 Multidimensional Rootfinding: Powell Hybrid

ctxboost.**PowellHybrid**(*f*, *fjac*, *guess*)

This is a modified version of Powell's Hybrid method as implemented in the hybrj algorithm in minpack. The Hybrid algorithm retains the fast convergence of Newton's method but will also reduce the residual when Newton's method is unreliable. The algorithm uses a generalized trust region to keep each step under control.

See also: Moré *et al.* [417], Eigen [216], Wikipedia [1433], Powell [473].

Parameters:

f
a callback matrix function defining the system.

fjac
a callback matrix function defining the Jacobian of the system system.

guess
a matrix containing the initial guess for the solution

Results:

x1
a matrix containing the solution.

Example:

This is an example from the original manual of MINPACK (Moré *et al.* [417]).

The routine in Python:

```
def demo_PowellHybridCtx(ctx):

    # The function to optimize
    def XmatHybrd(x, fvec):
        n = x.size
        for k in range(n):
            temp = (3.0 - 2.0 * x[k]) * x[k]
            temp1 = ctx.zero()
            if (k!=0): temp1 = x[k-1]
            temp2 = ctx.zero()
            if (k != n-1): temp2 = x[k+1]
            fvec[k] = temp - temp1 - 2.0*temp2 + 1.0

    # The Jacobi function
    def XmatHybrdJ(x, fjac):
        n = x.size
        for k in range(n):
            for j in range(n):
                fjac[k, j] = ctx.zero()
            fjac[k, k] = 3.0 - 4.0 * x[k]
            if (k != 0): fjac[k, k - 1] = -1
            if (k != n - 1): fjac[k, k + 1] = -2

    # This defines the start vector
    n = 9
    matInput = ctx.matZeros(n,1)
    matInput[0] = 1
    matInput[1] = 2  # entries 2 .. 8 are 0.
    print('matInput: \n', matInput)

    # This executes the PowellHybrd solver
    matRes = ctx.powellHybrd(XmatHybrd, XmatHybrdJ, matInput)
```

(continues on next page)

(continued from previous page)

```
    # Check the result
    Y = ctx.matZeros(n,1)
    XmatHybrd(matRes, Y)
    print('matRes: \n', matRes)
    print('Y: \n', Y)
```

The output of these routines:

```
Hello DemoPowellHybrdClassDbl()

X (solution):
-0.570654511600659,
-0.681628342291231,
-0.701732452563471,
-0.704212940083752,
-0.701369047627289,
-0.691865643379914,
-0.665792012154689,
-0.596034201280817,
-0.416412062998472,

matEval =  F(X=solution):
6.56011067690088E-09,
-4.17547307840493E-09,
-5.19316567526573E-09,
-2.39601338769546E-09,
2.02249372804886E-09,
4.81791939677123E-09,
2.57950016901987E-09,
-3.88373844195655E-09,
-1.35886191188206E-10,
```

5.15.2 Nonlinear LeastSquares: Levenberg-Marquardt

`ctxboost.Levenberg`(*f*, *fjac*, *matInput*)

This is a robust and efficient version of the Levenberg-Marquardt algorithm as implemented in the scaled lmder routine in minpack. The algorithm uses a generalized trust region to keep each step under control.

See also: Moré *et al.* [417], Eigen [216], Wikipedia [1427].

Parameters:

f
a callback matrix function defining the system.

fjac
a callback matrix function defining the Jacobian of the system system.

guess
a matrix containing the initial guess for the solution

Results:

x1
a matrix containing the solution.

Example:

This is an example from the original manual of MINPACK (Moré *et al.* [417]).

The routine in Python:

```
def demo_LevenbergCtx(ctx):

    # The function to optimize
    def XmatLM(x, fvec):
        m = 15
        y = ctx.matZeros(m,1)
        y[0] = 1.4e-1
        y[1] = 1.8e-1
        y[2] = 2.2e-1
        y[3] = 2.5e-1
        y[4] = 2.9e-1
        y[5] = 3.2e-1
        y[6] = 3.5e-1
        y[7] = 3.9e-1
        y[8] = 3.7e-1
        y[9] = 5.8e-1
        y[10] = 7.3e-1
        y[11] = 9.6e-1
        y[12] = 1.34e0
        y[13] = 2.1e0
        y[14] = 4.39e0
        for i in range(m):
            tmp1 = i + 1
            tmp2 = m - i
            tmp3 = tmp1
            if (i >= 8): tmp3 = tmp2
            fvec[i] = y[i] - (x[0] + tmp1/(x[1]*tmp2 + x[2]*tmp3))

    # The Jacobi function
    def XmatLMJ(x, fjac):
        m = 15
        for i in range(m):
            tmp1 = i + 1
            tmp2 = m - i
            tmp3 = tmp1
            if (i >= 8): tmp3 = tmp2
```

(continues on next page)

(continued from previous page)

```
            tmp4 = (x[1] * tmp2 + x[2] * tmp3)
            tmp4 = tmp4 * tmp4
            fjac[i, 0] = -1
            fjac[i, 1] = tmp1 * tmp2 / tmp4
            fjac[i, 2] = tmp1 * tmp3 / tmp4

    # This defines the start vector
    n = 3; m = 15
    matInput = ctx.matZeros(n,1)
    matInput[0] = 1
    matInput[1] = 2  # entries 2 .. 8 are 0.
    print('matInput: \n', matInput)
    # This executes the Levenberg solver
    matRes = ctx.levenberg(XmatLM, XmatLMJ, matInput)
    # Check the result
    Y = ctx.matZeros(m,1)
    XmatLM(matRes, Y)
    print('matRes: \n', matRes)
    print('Y: \n', Y)
```

This produces the following output:

```
Hello DemoLevenbergClassDbl()

X (solution):
0.0824105765758334,
1.1330366534715,
2.34369463894115,

matEval =  F(X=solution):
0.00588109515673704,
0.000265360346254795,
-0.000274673051589042,
-0.00654152299741256,
0.000823003778696318,
0.00129950005693674,
0.00446310734534455,
0.0199629386905548,
-0.0822160569476201,
0.0182119488681469,
0.0148111570102206,
0.0147099692233311,
0.0112079895785155,
0.00420403028888439,
-0.00680784758001085,
```

5.16 Eigen/CppOptLib: multidimensional optimization

The Homepage is https://github.com/PatWie/CppNumericalSolvers.

To use another solver, simply replace BfgsSolver by another name. Supported solvers are:

gradient descent solver (GradientDescentSolver)

conjugate gradient descent solver (ConjugatedGradientDescentSolver)

BFGS solver (BfgsSolver)

L-BFGS solver (LbfgsSolver) bfgsbSolver)

CMAes solver (CMAesSolver)

Nelder-Mead solver (NelderMeadSolver)

These solvers are tested on the Rosenbrock function from multiple difficult starting points by unit tests using the Google Testing Framework. And yes, you can use them directly in MATLAB. Additional benchmark functions are Beale, GoldsteinPrice, Booth, Matyas, Levi. Note, not all solvers are equivalently good at all problems.

For checking your gradient this library uses high-order central difference. Study the examples for more information about including box-constraints and gradient-information.

5.16.1 Nelder-Mead Solver

ctxboost.**NelderMead**(*f*, *fjac*, *matInput*)

The Nelder-Mead method (or downhill simplex method) is a commonly applied numerical method used to find the minimum or maximum of an objective function in a multidimensional space.

See also: Wieschollek [1081], Wikipedia [1432].

Parameters:

f
a callback matrix function defining the system.

fjac
a callback matrix function defining the Jacobian of the system system.

guess
a matrix containing the initial guess for the solution

Results:

x1
a matrix containing the solution.

Example:

The routine in Python:

```
def demo_NelderMeadCtx(ctx):

    # The function to optimize
    def matF1(X, Y):
        t1 = 1 - X[0]
        t2 = X[1] - X[0] * X[0]
        Y[0] = t1 * t1 + 100 * t2 * t2

    # The Jacobi function
    def matF2(X, Y):
        Y[0] = -2 * (1 - X[0]) + 200 * (X[1] - X[0] * X[0]) * (-2 * X[0])
        Y[1] = 200 * (X[1] - X[0] * X[0])

    # This defines the start vector
    r = 2
    c = 1
    X = ctx.matZeros(r,c)
    X[0] = 1
    X[1] = 2
    print('X: \n', X)

    # This executes the NelderMead solver
    matRes = ctx.nelderMeadSolver(matF1, matF2, X)

    # Check the result
    Y = ctx.matZeros(r,c)
    matF1(matRes, Y)
    print('matRes: \n', matRes)
    print('Y: \n', Y)
    nrm = ctx.sqrt( Y[0]*Y[0] + Y[1]*Y[1]) # result is correct
    print('Norm: \n', nrm)
```

This produces the following output:

```
Hello DemoNelderMeadSolverDbl()
matRes:
    1.000014;
```

(continues on next page)

(continued from previous page)

```
    1.000029;

Y = F(matRes):
    0.000000;
    0.000000;

Norm: 2.96378497762668E-10
```

5.16.2 CMAes solver

ctxboost.**CMAes**(*f*, *fjac*, *matInput*)

CMA-ES stands for covariance matrix adaptation evolution strategy. Evolution strategies (ES) are stochastic, derivative-free methods for numerical optimization of non-linear or non-convex continuous optimization problems.

See also: Wieschollek [1081], Wikipedia [1423].

Parameters:

f
a callback matrix function defining the system.

fjac
a callback matrix function defining the Jacobian of the system system.

guess
a matrix containing the initial guess for the solution

Results:

x1
a matrix containing the solution.

Example:

The routine in Python:

```
def demo_CMAesCtx(ctx):

    # The function to optimize
    def matF1(X, Y):
        t1 = 1 - X[0]
        t2 = X[1] - X[0] * X[0]
        Y[0] = t1 * t1 + 100 * t2 * t2

    # The Jacobi function
    def matF2(X, Y):
        Y[0] = -2 * (1 - X[0]) + 200 * (X[1] - X[0] * X[0]) * (-2 * X[0])
        Y[1] = 200 * (X[1] - X[0] * X[0])

    # This defines the start vector
    r = 2
    c = 1
    X = ctx.matZeros(r,c)
    X[0] = 1
    X[1] = 2
    print('X: \n', X)

    # This executes the CMAes solver
    matRes = ctx.cMAesSolver(matF1, matF2, X)

    # Check the result
    Y = ctx.matZeros(r,c)
    matF1(matRes, Y)
    print('matRes: \n', matRes)
    print('Y: \n', Y)
    nrm = ctx.sqrt( Y[0]*Y[0] + Y[1]*Y[1]) # result is correct
    print('Norm: \n', nrm)
```

This produces the following output:

```
Hello DemoCMAesSolverDbl()
matRes:
    0.999140;
```

(continues on next page)

(continued from previous page)

```
    0.998260;

Y = F(matRes):
    0.000001;
    0.000000;

Norm: 7.83903245823684E-07
```

5.16.3 BFGS Solver

ctxboost.**BFGS**(*f*, *fjac*, *matInput*)

The Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm is an iterative method for solving unconstrained nonlinear optimization problems.

See also: Wieschollek [1081], Wikipedia [1422].

Parameters:

f
a callback matrix function defining the system.

fjac
a callback matrix function defining the Jacobian of the system system.

guess
a matrix containing the initial guess for the solution

Results:

x1
a matrix containing the solution.

Example:

The routine in Python:

```
def demo_BfgsCtx(ctx):

    # The function to optimize
    def matF1(X, Y):
        t1 = 1 - X[0]
        t2 = X[1] - X[0] * X[0]
        Y[0] = t1 * t1 + 100 * t2 * t2

    # The Jacobi function
    def matF2(X, Y):
        Y[0] = -2 * (1 - X[0]) + 200 * (X[1] - X[0] * X[0]) * (-2 * X[0])
        Y[1] = 200 * (X[1] - X[0] * X[0])

    # This defines the start vector
    r = 2
    c = 1
    X = ctx.matZeros(r,c)
    X[0] = 1
    X[1] = 2
    print('X: \n', X)

    # This executes the Bfgs solver
    matRes = ctx.bfgsSolver(matF1, matF2, X)

    # Check the result
    Y = ctx.matZeros(r,c)
    matF1(matRes, Y)
    print('matRes: \n', matRes)
    print('Y: \n', Y)
    nrm = ctx.sqrt( Y[0]*Y[0] + Y[1]*Y[1]) # result is correct
    print('Norm: \n', nrm)
```

This produces the following output:

```
Hello DemoBfgsSolverClassDbl()
matRes:
0.999999951430365,
```

(continues on next page)

(continued from previous page)

```
0.999999899650162,

Y = F(matRes):
3.389785983959666E-15,
0,

Norm: 3.389785983959666E-15
```

5.16.4 L-BFGS Solver

`ctxboost.LBFGS`(*f*, *fjac*, *matInput*)

Limited-memory BFGS (L-BFGS or LM-BFGS) is an optimization algorithm in the family of quasi-Newton methods that approximates the Broyden-Fletcher-Goldfarb-Shanno algorithm (BFGS) using a limited amount of computer memory.

See also: Wieschollek [1081], Wikipedia [1428].

Parameters:

f
a callback matrix function defining the system.

fjac
a callback matrix function defining the Jacobian of the system system.

guess
a matrix containing the initial guess for the solution

Results:

x1
a matrix containing the solution.

Example:

The routine in Python:

```
def demo_LBfgsCtx(ctx):

    # The function to optimize
    def matF1(X, Y):
        t1 = 1 - X[0]
        t2 = X[1] - X[0] * X[0]
        Y[0] = t1 * t1 + 100 * t2 * t2

    # The Jacobi function
    def matF2(X, Y):
        Y[0] = -2 * (1 - X[0]) + 200 * (X[1] - X[0] * X[0]) * (-2 * X[0])
        Y[1] = 200 * (X[1] - X[0] * X[0])

    # This defines the start vector
    r = 2
    c = 1
    X = ctx.matZeros(r,c)
    X[0] = 1
    X[1] = 2
    print('X: \n', X)

    # This executes the LBfgs solver
    matRes = ctx.lbfgsBSolver(matF1, matF2, X)

    # Check the result
    Y = ctx.matZeros(r,c)
    matF1(matRes, Y)
    print('matRes: \n', matRes)
    print('Y: \n', Y)
    nrm = ctx.sqrt( Y[0]*Y[0] + Y[1]*Y[1]) # result is correct
    print('Norm: \n', nrm)
```

This produces the following output:

```
Hello DemoLbfgsSolverDbl()
matRes:
    1.000000;
```

(continues on next page)

(continued from previous page)

```
    1.000000;

Y = F(matRes):
    0.000000;
    0.000000;

Norm: 7.41089272169439E-14
```

5.16.5 Gradient descent solver

ctxboost.**GradientDescent**(*f*, *fjac*, *matInput*)

Gradient descent is a first-order iterative optimization algorithm for finding a local minimum of a differentiable function.

See also: Wieschollek [1081], Wikipedia [1422]. and https://en.wikipedia.org/wiki/Gradient_descent.

Parameters:

f
a callback matrix function defining the system.

fjac
a callback matrix function defining the Jacobian of the system system.

guess
a matrix containing the initial guess for the solution

Results:

x1
a matrix containing the solution.

Example:

The routine in Python:

```
def demo_GradientDescentCtx(ctx):

    # The function to optimize
    def matF1(X, Y):
        t1 = 1 - X[0]
        t2 = X[1] - X[0] * X[0]
        Y[0] = t1 * t1 + 100 * t2 * t2

    # The Jacobi function
    def matF2(X, Y):
        Y[0] = -2 * (1 - X[0]) + 200 * (X[1] - X[0] * X[0]) * (-2 * X[0])
        Y[1] = 200 * (X[1] - X[0] * X[0])

    # This defines the start vector
    r = 2
    c = 1
    X = ctx.matZeros(r,c)
    X[0] = 1
    X[1] = 2
    print('X: \n', X)

    # This executes the GradientDescentSolver solver
    matRes = ctx.gradientDescentSolver(matF1, matF2, X)

    # Check the result
    Y = ctx.matZeros(r,c)
    matF1(matRes, Y)
    print('matRes: \n', matRes)
    print('Y: \n', Y)
    nrm = ctx.sqrt( Y[0]*Y[0] + Y[1]*Y[1]) # result is correct
    print('Norm: \n', nrm)
```

This produces the following output:

```
Hello DemoBfgsSolverClassDbl()
matRes:
0.999999951430365,
0.999999899650162,
```

(continues on next page)

(continued from previous page)

```
Y = F(matRes):
3.38978598395966E-15,
0,

Norm: 3.38978598395966E-15
```

5.16.6 Conjugate gradient descent solver

`ctxboost.ConjugateGradientDescent`(*f*, *fjac*, *matInput*)

The conjugate gradient method is an algorithm for the numerical solution of particular systems of linear equations, namely those whose matrix is symmetric and positive-definite. The conjugate gradient method can also be used to solve unconstrained optimization problems such as energy minimization.

See also: Wieschollek [1081], Wikipedia [1422]. and https://en.wikipedia.org/wiki/Conjugate_gradient_method.

Parameters:

f
a callback matrix function defining the system.

fjac
a callback matrix function defining the Jacobian of the system system.

guess
a matrix containing the initial guess for the solution

Results:

x1
a matrix containing the solution.

Example:

The routine in Python:

```
def demo_GradientDescentCtx(ctx):

    # The function to optimize
    def matF1(X, Y):
        t1 = 1 - X[0]
        t2 = X[1] - X[0] * X[0]
        Y[0] = t1 * t1 + 100 * t2 * t2

    # The Jacobi function
    def matF2(X, Y):
        Y[0] = -2 * (1 - X[0]) + 200 * (X[1] - X[0] * X[0]) * (-2 * X[0])
        Y[1] = 200 * (X[1] - X[0] * X[0])

    # This defines the start vector
    r = 2
    c = 1
    X = ctx.matZeros(r,c)
    X[0] = 1
    X[1] = 2
    print('X: \n', X)

    # This executes the GradientDescentSolver solver
    matRes = ctx.gradientDescentSolver(matF1, matF2, X)

    # Check the result
    Y = ctx.matZeros(r,c)
    matF1(matRes, Y)
    print('matRes: \n', matRes)
    print('Y: \n', Y)
    nrm = ctx.sqrt( Y[0]*Y[0] + Y[1]*Y[1]) # result is correct
    print('Norm: \n', nrm)
```

Example:

```
Public Sub matF1(x As dbl_mat_t, y As dbl_mat_t)
    Dim t1 As Double = 1 - x(0)
```

(continues on next page)

(continued from previous page)

```
    Dim t2 As Double = x(1) - x(0) * x(0)
    y(0) = t1 * t1 + 100 * t2 * t2
End Sub

Public Sub matF2(x As dbl_mat_t, y As dbl_mat_t)
    y(0) = -2 * (1 - x(0)) + 200 * (x(1) - x(0) * x(0)) * (-2 * x(0))
    y(1) = 200 * (x(1) - x(0) * x(0))
End Sub

Sub DemoConjugatedGradientDescentSolverDbl()
    Console.WriteLine("Hello DemoConjugatedGradientDescentSolverDbl() ")
    Dim X, Y As New dbl_mat_t()
    X.resize(2, 1)
    Y.resize(X.rows, 1)
    X(0) = 1
    X(1) = 2

    Dim OptLib As New ConjugatedGradientDescentSolverDbl(AddressOf matF1, AddressOf matF2, X)
    Dim matRes = OptLib.Solve()
    matRes.print("matRes: ", 15)
    matF1(matRes, Y)
    Y.print("Y = F(matRes): ", 15)

    Dim nrm = Math.Sqrt( Y(0)*Y(0) + Y(1)*Y(1))
    Console.WriteLine("Norm: {0}", nrm)
    Console.WriteLine("")
End Sub
```

This produces the following output:

```
Hello DemoConjugatedGradientDescentSolverDbl()
matRes:
    0.999964;
    0.999928;

Y = F(matRes):
    0.000000;
    0.000000;

Norm: 1.3055097726625E-09
```

5.16.7 Newton descent solver (description needs correction)

ctxboost.**NewtonDescent**(*f*, *fjac*, *matInput*)

The conjugate gradient method is an algorithm for the numerical solution of particular systems of linear equations, namely those whose matrix is symmetric and positive-definite. The conjugate gradient method can also be used to solve unconstrained optimization problems such as energy minimization.

See also: Wieschollek [1081], Wikipedia [1422]. and https://en.wikipedia.org/wiki/Conjugate_gradient_method.

Parameters:

f
a callback matrix function defining the system.

fjac
a callback matrix function defining the Jacobian of the system system.

guess
a matrix containing the initial guess for the solution

Results:

x1
a matrix containing the solution.

Example:

The routine in Python:

```
def demo_GradientDescentCtx(ctx):

    # The function to optimize
    def matF1(X, Y):
        t1 = 1 - X[0]
        t2 = X[1] - X[0] * X[0]
        Y[0] = t1 * t1 + 100 * t2 * t2

    # The Jacobi function
    def matF2(X, Y):
        Y[0] = -2 * (1 - X[0]) + 200 * (X[1] - X[0] * X[0]) * (-2 * X[0])
        Y[1] = 200 * (X[1] - X[0] * X[0])

    # This defines the start vector
    r = 2
    c = 1
    X = ctx.matZeros(r,c)
    X[0] = 1
    X[1] = 2
    print('X: \n', X)

    # This executes the GradientDescentSolver solver
    matRes = ctx.gradientDescentSolver(matF1, matF2, X)

    # Check the result
    Y = ctx.matZeros(r,c)
    matF1(matRes, Y)
    print('matRes: \n', matRes)
    print('Y: \n', Y)
    nrm = ctx.sqrt( Y[0]*Y[0] + Y[1]*Y[1]) # result is correct
    print('Norm: \n', nrm)
```

Example:

```
Public Sub matF1(x As dbl_mat_t, y As dbl_mat_t)
    Dim t1 As Double = 1 - x(0)
```

(continues on next page)

(continued from previous page)

```
    Dim t2 As Double = x(1) - x(0) * x(0)
    y(0) = t1 * t1 + 100 * t2 * t2
End Sub

Public Sub matF2(x As dbl_mat_t, y As dbl_mat_t)
    y(0) = -2 * (1 - x(0)) + 200 * (x(1) - x(0) * x(0)) * (-2 * x(0))
    y(1) = 200 * (x(1) - x(0) * x(0))
End Sub

Sub DemoConjugatedGradientDescentSolverDbl()
    Console.WriteLine("Hello DemoConjugatedGradientDescentSolverDbl() ")
    Dim X, Y As New dbl_mat_t()
    X.resize(2, 1)
    Y.resize(X.rows, 1)
    X(0) = 1
    X(1) = 2

    Dim OptLib As New ConjugatedGradientDescentSolverDbl(AddressOf matF1, AddressOf matF2, X)
    Dim matRes = OptLib.Solve()
    matRes.print("matRes: ", 15)
    matF1(matRes, Y)
    Y.print("Y = F(matRes): ", 15)

    Dim nrm = Math.Sqrt( Y(0)*Y(0) + Y(1)*Y(1))
    Console.WriteLine("Norm: {0}", nrm)
    Console.WriteLine("")
End Sub
```

This produces the following output:

```
Hello DemoConjugatedGradientDescentSolverDbl()
matRes:
    0.999964;
    0.999928;

Y = F(matRes):
    0.000000;
    0.000000;

Norm: 1.3055097726625E-09
```

CHAPTER

SIX

EIGEN: DENSE AND SPARSE MATRICES

The source code for the C# tests in Visual Studio is divided in 4 parts:

Part 1 contains the source code for test procedures which work for real and complex dense matrices, and which work for floating point and ball arithmetic. It can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B06a_MatAll.cs.

Part 2 contains the source code for test procedures which work for real and complex dense matrices, but which work only for floating point arithmetic. It can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B06b_MatFp.cs.

Part 3 contains the source code for test procedures which work only for real dense matrices, and only for floating point arithmetic. It can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B06c_MatFpReal.cs.

Part 4 contains the source code for test procedures which work only for real and complex sparse matrices, and only for floating point arithmetic. It can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B06d_SpMat.cs.

6.1 Creating scalars and matrices

6.1.1 Creating a matrix, and converting from compatible data types

`ctx.mat_t(`x`)`

Converts from a real scalar of type dreal

6.1.2 Creating a matrix of zeros

`ctx.mat_zeros(`n, m`)`

Creates a $n \times m$ matrix of the indicated type and sets all entries to zero.

```
>>> from xlcalcnet import *
>>> ctx.dps = 15; n = 4; m = 4
>>> matA = ctx.mat_zeros(n, m, eigen=True)
>>> matA.show("matA, mat_zeros(n, m) :")
matA, mat_zeros(n, m) :
 0,  0,  0,  0,
 0,  0,  0,  0,
 0,  0,  0,  0,
 0,  0,  0,  0,
```

6.1.3 Creating a matrix of ones

`ctx.mat_ones(n, m)`

Creates a $n \times m$ matrix of the indicated type and sets all entries to one.

```
>>> from xlcalcnet import *
>>> ctx.dps = 15; n = 4; m = 4
>>> matA = ctx.mat_ones(n, m)
>>> matA.show("matA, mat_ones(n, m) :")
matA, mat_ones(n, m) :
 1,  1,  1,  1,
 1,  1,  1,  1,
 1,  1,  1,  1,
 1,  1,  1,  1,
```

6.1.4 Creating an identity matrix

`ctx.mat_identity(n, m)`

Creates a $n \times m$ identity matrix of the indicated type.

```
>>> from xlcalcnet import *
>>> ctx.dps = 15; n = 4; m = 4
>>> matA = ctx.mat_identity(n, m)
>>> matA.show("matA, mat_identity(n, m) :")
matA, mat_identity(n, m) :
 1,  0,  0,  0,
 0,  1,  0,  0,
 0,  0,  1,  0,
 0,  0,  0,  1,
```

6.1.5 Creating a general random matrix

`ctx.mat_random(n, m)`

Creates a $n \times m$ matrix of the indicated type and sets all entries to random values.

```
>>> from xlcalcnet import *
>>> ctx.dps = 15; n = 4; m = 4
>>> matA = ctx.mat_random(n, m)
>>> matA.show("matA, mat_random(n, m) :")
matA, mat_random(n, m) :
 0.35029145176550,  0.17410809656056,  0.30399487289041,  0.14731284524064,
 0.89596240119633,  0.85894344920194,  0.014984588152715,  0.16589861751152,
 0.82284005249184,  0.71050141911069,  0.091402935880612,  0.98852504043703,
 0.74660481582080,  0.51353495895260,  0.36445204016236,  0.44569231238746,
```

6.1.6 Creating a symmetric random matrix

`ctx.mat_random_symmetric(n)`

Creates a $n \times n$ symmetric matrix of the indicated type and sets all entries to random values.

```
>>> from xlcalcnet import *
>>> ctx.dps = 15; n = 4; m = 4
>>> matA = ctx.mat_random_symmetric(n)
>>> matA.show("matA, mat_random_symmetric(n) :")
matA, mat_random_symmetric(n) :
 0.23816644795068,  0.53633228553118,  0.17514572588275,  0.43491927854244,
 0.53633228553118,  1.1423688467055,  1.2648091067232,  1.2148503067110,
 0.17514572588275,  1.2648091067232,  0.90157780693990,  1.1354411450545,
 0.43491927854244,  1.2148503067110,  1.1354411450545,  1.6052125614185,
```

6.1.7 Creating a self-adjoint random matrix

`ctx.`**`mat_random_selfadjoint`**(*n*, *complex=False*, *eigen=False*)

Creates a $n \times n$ selfadjoint matrix of the indicated type and sets all entries to random values.

```
>>> from xlcalcnet import *
>>> ctx.dps = 15; n = 4; m = 4
>>> matA = ctx.mat_random_sa(n)
>>> matA.show("matA, mat_random_sa(n) :")
matA, mat_random_sym(n) :
 0.23816644795068,  0.53633228553118,  0.17514572588275,  0.43491927854244,
 0.53633228553118,  1.1423688467055,  1.2648091067232,  1.2148503067110,
 0.17514572588275,  1.2648091067232,  0.90157780693990,  1.1354411450545,
 0.43491927854244,  1.2148503067110,  1.1354411450545,  1.6052125614185,
```

6.1.8 Creating a positive definite self-adjoint matrix

`ctx.`**`mat_random_selfadjoint_posdef`**(*n*, *complex=False*, *eigen=False*)

Creates a $n \times n$ positive definite random matrix of the indicated type.

```
>>> from xlcalcnet import *
>>> ctx.dps = 15; n = 4; m = 4
>>> matA = ctx.mat_random_sa_posdef(n)
>>> matA.show("matA, mat_random_sa_posdef(n)")
matA, mat_random_sa_posdef(n)
 1.6144949159812,  0.37328104299787,  0.64671931578811,  1.0725029543798,
 0.37328104299787,  0.23220359390837,  0.27432857744706,  0.35913470470614,
 0.64671931578811,  0.27432857744706,  0.55785829596540,  0.52838216584663,
 1.0725029543798,  0.35913470470614,  0.52838216584663,  1.4188647561723,
```

6.1.9 Creating a matrix with linearly increasing values

`ctx.`**`mat_fill_linear`**(*n*, *m*, *x*)

Creates a $n \times m$ matrix of the indicated type and sets all entries to increasing values.

```
>>> from xlcalcnet import *
>>> ctx.dps = 15; n = 4; m = 4
>>> matA = ctx.mat_linspace(n, m, x)
>>> matA.show("matA, mat_linspace(n, m, x) :")
matA, mat_linspace(n, m, x) :
 0,  0,  0,  0,
 0,  0,  0,  0,
 0,  0,  0,  0,
 0,  0,  0,  0,
```

6.2 Read-only properties: information about a matrix

6.2.1 Rows of a matrix

`ctx.`**`mat_rows`**`(`*matA*`)`

Returns the number of rows of the matrix. Writing `matA.rows` has the same effect.

```
>>> from xlcalcnet import *
>>> mpm.dps = 15;
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", "")
>>> print("A.rows: ", A.rows)
A.rows: 6
```

6.2.2 Columns of a matrix

`ctx.`**`mat_cols`**`(`*matA*`)`

Returns the number of columns of the matrix. Writing `matA.cols` has the same effect,

```
>>> from xlcalcnet import *
>>> mpm.dps = 15;
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", "")
>>> print("A.cols: ", A.cols)
A.cols: 4
```

6.2.3 Size of a matrix

`ctx.`**`mat_size`**`(`*matA*`)`

Returns the size of the matrix. Writing `matA.size` has the same effect.

```
>>> from xlcalcnet import *
>>> mpm.dps = 15;
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", "")
>>> print("A.size: ", A.size)
A.cols: 24
```

6.3 Accessing and setting parts of a matrix

6.3.1 Getting and setting a matrix coefficient

Individual coefficients of a matrix A are accessed using the `A[row,col]` syntax. Note that indexing starts at 0, so that the coefficient in the 4^{th} row and 2^{nd} column is accessed as `A[3,1]`. Examples:

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11,  12,  13,  14,
21,  22,  23,  24,
31,  32,  33,  34,
41,  42,  43,  44,
51,  52,  53,  54,
61,  62,  63,  64,

>>> # gets the coefficient in row number 3 and column number 1
>>> print("A[3,1]: ", A[3,1])
A[3,1]:  42

>>> # sets the coefficient in row number 1 and column number 2 to the value of 99.
>>> A[1,2] = 99; A.show("A")
matA:
11,  12,  13,  14,
21,  22,  99,  24,
31,  32,  33,  34,
41,  42,  43,  44,
51,  52,  53,  54,
61,  62,  63,  64,
```

6.3.2 Getting and setting a block

ctx.mat_get_block(*matA*, *i*, *j*, *p*, *q*)

Gets a block of the matrix, starting at row i and column j, with p row elements and q column elements. Writing `matA.get_Block(i, j, p, q)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11,  12,  13,  14,
21,  22,  23,  24,
31,  32,  33,  34,
41,  42,  43,  44,
51,  52,  53,  54,
61,  62,  63,  64,

>>> # gets a block, beginning at A[1, 1], stretching to A[1+3, 1+2]
>>> b1 = A.block(1, 1, 3, 2); b1.show("b1")
b1:
22,  23,
32,  33,
42,  43,
```

ctx.mat_set_block(*matA*, *i*, *j*, *p*, *q*, *matB*)

Sets a block of the matrix to *matB*, starting at row i and column j, with p row elements and q column elements. Writing `matA.set_Block(i, j, p, q, matB)` has the same effect

```
>>> # continued from above
>>> # sets b1 into a block, beginning at A[0, 0], stretching to A[0+3, 0+2]
```

(continues on next page)

(continued from previous page)

```
>>> A.set_Block(0, 0, 3, 2, b1); ; A.show("A")
A:
22,  23,  13,  14,
32,  33,  23,  24,
42,  43,  33,  34,
41,  42,  43,  44,
51,  52,  53,  54,
61,  62,  63,  64,
```

6.3.3 Getting and setting a matrix row

ctx.mat_get_row(*matA*, *i*)

Gets the i^{th} row. Writing `matA.get_Row(i)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11,  12,  13,  14,
21,  22,  23,  24,
31,  32,  33,  34,
41,  42,  43,  44,
51,  52,  53,  54,
61,  62,  63,  64,

>>> # gets row number 2 (i.e. the 3rd row from the top)
>>> r1 = A.row(2); r1.show("r1")
r1:
31,  32,  33,  34,
```

ctx.mat_set_row(*matA*, *i*, *matB*)

Sets the i^{th} row to *matB*. Writing `matA.set_Row(i, matB)` has the same effect

```
>>> # continued from above
>>> # sets the content of row number 5 to the content of row number 2
>>> A.set_Row(5, A.get_Row(2)); A.show("A")
A:
11,  12,  13,  14,
21,  22,  23,  24,
31,  32,  33,  34,
41,  42,  43,  44,
51,  52,  53,  54,
31,  32,  33,  34,
```

6.3.4 Getting and setting a matrix column

ctx.mat_get_col(*matA*, *j*)

Gets the j^{th} column. Writing `matA.get_Col(j)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11,  12,  13,  14,
21,  22,  23,  24,
31,  32,  33,  34,
41,  42,  43,  44,
51,  52,  53,  54,
61,  62,  63,  64,
```

(continues on next page)

(continued from previous page)

```
>>> # gets column number 1 (i.e. the 2nd column from the left)
>>> c1 = A.col(1); c1.show("c1")
c1:
12,
22,
32,
42,
52,
62,
```

`ctx.`**`mat_set_col`**`(`*matA*, *j*, *matB*`)`

Sets the j^{th} column to *matB*. Writing `matA.set_Col(j, matB)` has the same effect

```
>>> # continued from above
>>> # sets the content of column number 3 to the content of column number 1
>>> A.set_Col(3, A.get_Col(1)); A.show("A")
A:
11,  12,  13,  12,
21,  22,  23,  22,
31,  32,  33,  32,
41,  42,  43,  42,
51,  52,  53,  52,
61,  62,  63,  62,
```

6.3.5 Getting and setting a diagonal

`ctx.`**`mat_get_diagonal`**`(`*matA*, *q=0*`)`

Gets the diagonal or a subdiagonal of the matrix. Writing `matA.get_Diagonal(q=0)` has the same effect.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11,  12,  13,  14,
21,  22,  23,  24,
31,  32,  33,  34,
41,  42,  43,  44,
51,  52,  53,  54,
61,  62,  63,  64,

>>> # gets the first lower subdiagonal
>>> b1 = A.diagonal(-1); b1.show("b1")
b1 = A.diagonal(-1):
21,
32,
43,
54,
```

`ctx.`**`mat_set_diagonal`**`(`*matA*, *q*, *matB*`)`

Sets the diagonal or a subdiagonal of the matrix to *matB*. Writing `matA.set_Diagonal(q, matB)` has the same effect.

```
>>> # continued from above
>>> # sets the diagonal to the first lower subdiagonal
>>> A.set_Diagonal(0, b1); A.show("A")
A:
21,  12,  13,  14,
21,  32,  23,  24,
31,  32,  43,  34,
41,  42,  43,  54,
51,  52,  53,  54,
61,  62,  63,  64,
```

6.3.6 Getting and setting a triangular view

`ctx.`**`mat_get_triangular_view`**(*matA*, *view=1*)

Gets the triangular view of the matrix. Writing `matA.get_TriangularView(View = 1)` has the same effect.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11,  12,  13,  14,
21,  22,  23,  24,
31,  32,  33,  34,
41,  42,  43,  44,
51,  52,  53,  54,
61,  62,  63,  64,

>>> # gets the first lower subdiagonal
>>> b1 = A.diagonal(-1); b1.show("b1")
b1 = A.diagonal(-1):
21,
32,
43,
54,
```

`ctx.`**`mat_set_triangular_view`**(*matA*, *view*, *matB*)

Sets the triangular view of the matrix to *matB*. Writing `matA.set_TriangularView(View, matB)` has the same effect.

```
>>> # continued from above
>>> # sets the diagonal to the first lower subdiagonal
>>> A.set_Diagonal(0, b1); A.show("A")
A:
21,  12,  13,  14,
21,  32,  23,  24,
31,  32,  43,  34,
41,  42,  43,  54,
51,  52,  53,  54,
61,  62,  63,  64,
```

6.4 Changing the shape of a matrix and/or the order of coefficients

6.4.1 Sorting a whole matrix

`ctx.mat_sort(`*matA*, *sort_order=0*, *sort_criterion=1*`)`

Returns a sorted version of the matrix, sorted according to SortOrder and SortCriterion.

Writing `matA.Sort(sort_order=0, sort_criterion=1)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # returns a copy of the whole matrix in ascending order (SortOrder=0)
>>> B = A.sorted(SortOrder=0); B.show("B")
B:
11, 23, 41, 53,
12, 24, 42, 54,
13, 31, 43, 61,
14, 32, 44, 62,
21, 33, 51, 63,
22, 34, 52, 64,

>>> # sorts the whole matrix in descending order (SortOrder=1)
>>> C = A.sorted(SortOrder=1); C.show("C")
C:
64, 52, 34, 22,
63, 51, 33, 21,
62, 44, 32, 14,
61, 43, 31, 13,
54, 42, 24, 12,
53, 41, 23, 11,
```

6.4.2 Sorting the rows of a matrix by a given column

`ctx.mat_sort_rows_by_col(`*matA*, *column_to_sort_by=0*, *sort_order=0*, *sort_criterion=1*`)`

Returns a sorted version of the matrix, sorted by column ColumnToSortBy, according to SortOrder and SortCriterion.

Writing `matA.SortRowsByCol(column_to_sort_by=0, sort_order=0, sort_criterion=1)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # sorts the whole matrix by column 0 in descending order (SortOrder=1)
>>> B = A.sorted_rows_by_col(ColumnToSortBy=0, SortOrder=1); B.show("B")
B:
61, 62, 63, 64,
51, 52, 53, 54,
```

(continues on next page)

(continued from previous page)

```
41, 42, 43, 44,
31, 32, 33, 34,
21, 22, 23, 24,
11, 12, 13, 14,

>>> # sorts the whole matrix in descending order (SortOrder=0)
>>> C = A.sorted_rows_by_col(ColumnToSortBy=0, SortOrder=0); C.show("C")
C:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,
```

6.4.3 Resize a matrix

`ctx.`**`mat_resize`**(*matA*, *r*, *c*)

Resizes the matrix inplace, setting values of coefficients to zero.

Writing `matA.Resize(r, c)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # resizes the matrix to 3 rows and 2 columns, setting values of coefficients to zero.
>>> A.resize(3,2); A.show("A")
A:
0, 0,
0, 0,
0, 0,
```

6.4.4 Resize a matrix like another matrix

`ctx.`**`mat_resize_like`**(*matA*, *matB*)

Resizes the matrix inplace, setting values of coefficients to zero.

Writing `matA.ResizeLike(matB)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # resizes the matrix to 3 rows and 2 columns, setting values of coefficients to zero.
>>> A.resize(3,2); A.show("A")
A:
```

(continues on next page)

(continued from previous page)

```
0, 0,
0, 0,
0, 0,
```

6.4.5 Conservatively resize a matrix

`ctx.`**`mat_conservative_resize`**`(`*matA*, *r*, *c*`)`

Resizes the matrix inplace, retaining values of existing coefficients.

Writing `matA.ConservativeResize(r, c)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # resizes the matrix to 3 rows and 6 columns, retaining values of existing coefficients.
>>> A.conservative_resize(3,6); A.show("A")
A:
11, 12, 13, 14,  0,  0,
21, 22, 23, 24,  0,  0,
31, 32, 33, 34,  0,  0,
```

6.4.6 Convert vector to diagonal matrix

`ctx.`**`mat_as_diagonal`**`(`*matA*`)`

Returns the first column vector of a matrix as a diagonal matrix.

Writing `matA.AsDiagonal()` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> # read the first column from the matrix
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", "").col(0); A.show("A")
A:
11,
21,
31,
41,
51,
61,

>>> # resizes the matrix to 3 rows and 6 columns, retaining values of existing coefficients.
>>> B = A.as_diagonal(); B.show("B")
B:
11,  0,  0,  0,  0,  0,
 0, 21,  0,  0,  0,  0,
 0,  0, 31,  0,  0,  0,
 0,  0,  0, 41,  0,  0,
 0,  0,  0,  0, 51,  0,
 0,  0,  0,  0,  0, 61,
```

6.4.7 Adjoint of a matrix

`ctx.mat_adjoint(matA)`

Returns the adjoint matrix of the matrix. Writing `matA.Adjoint()` has the same effect

```
>>> from xlcalcnet import *

>>> # example with a real matrix
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,


>>> # returns the adjoint matrix of a real matrix, i.e. the transpose
>>> B = A.adjoint(); B.show("B")
B:
11, 21, 31, 41, 51, 61,
12, 22, 32, 42, 52, 62,
13, 23, 33, 43, 53, 63,
14, 24, 34, 44, 54, 64,
15, 25, 35, 45, 55, 65,
16, 26, 36, 46, 56, 66,


>>> # example with a complex matrix
>>> ctx = mp14.dcf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableA6x6", ""); A.show("A")
A:
11 + 31j, 12 + 32j, 13 + 33j, 14 + 34j, 15 + 35j, 16 + 36j,
21 + 41j, 22 + 42j, 23 + 43j, 24 + 44j, 25 + 45j, 26 + 46j,
31 + 51j, 32 + 52j, 33 + 53j, 34 + 54j, 35 + 55j, 36 + 56j,
41 + 61j, 42 + 62j, 43 + 63j, 44 + 64j, 45 + 65j, 46 + 66j,
51 + 71j, 52 + 72j, 53 + 73j, 54 + 74j, 55 + 75j, 56 + 76j,
61 + 81j, 62 + 82j, 63 + 83j, 64 + 84j, 65 + 85j, 66 + 86j,


>>> # returns the adjoint matrix of a complex matrix, i.e. the conjugate of the transpose
>>> B = A.adjoint(); B.show("B")
B:
11 + -31j, 21 + -41j, 31 + -51j, 41 + -61j, 51 + -71j, 61 + -81j,
12 + -32j, 22 + -42j, 32 + -52j, 42 + -62j, 52 + -72j, 62 + -82j,
13 + -33j, 23 + -43j, 33 + -53j, 43 + -63j, 53 + -73j, 63 + -83j,
14 + -34j, 24 + -44j, 34 + -54j, 44 + -64j, 54 + -74j, 64 + -84j,
15 + -35j, 25 + -45j, 35 + -55j, 45 + -65j, 55 + -75j, 65 + -85j,
16 + -36j, 26 + -46j, 36 + -56j, 46 + -66j, 56 + -76j, 66 + -86j,
```

6.4.8 Conjugate of a matrix

`ctx.mat_conjugate(matA)`

Returns the conjugate matrix of matrix matA. Writing `matA.Conjugate()` has the same effect

```
>>> from xlcalcnet import *

>>> # example with a real matrix
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
```

(continues on next page)

(continued from previous page)

```
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,


>>> # returns the conjugate matrix of a real matrix, i.e. the matrix itself
>>> B = A.conjugate(); B.show("B")
B:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,


>>> # example with a complex matrix
>>> ctx = mp14.dcf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableA6x6", ""); A.show("A")
A:
11 + 31j, 12 + 32j, 13 + 33j, 14 + 34j, 15 + 35j, 16 + 36j,
21 + 41j, 22 + 42j, 23 + 43j, 24 + 44j, 25 + 45j, 26 + 46j,
31 + 51j, 32 + 52j, 33 + 53j, 34 + 54j, 35 + 55j, 36 + 56j,
41 + 61j, 42 + 62j, 43 + 63j, 44 + 64j, 45 + 65j, 46 + 66j,
51 + 71j, 52 + 72j, 53 + 73j, 54 + 74j, 55 + 75j, 56 + 76j,
61 + 81j, 62 + 82j, 63 + 83j, 64 + 84j, 65 + 85j, 66 + 86j,


>>> # returns the conjugate matrix of a complex matrix
>>> B = A.conjugate(); B.show("B")
B:
11 + -31j, 12 + -32j, 13 + -33j, 14 + -34j, 15 + -35j, 16 + -36j,
21 + -41j, 22 + -42j, 23 + -43j, 24 + -44j, 25 + -45j, 26 + -46j,
31 + -51j, 32 + -52j, 33 + -53j, 34 + -54j, 35 + -55j, 36 + -56j,
41 + -61j, 42 + -62j, 43 + -63j, 44 + -64j, 45 + -65j, 46 + -66j,
51 + -71j, 52 + -72j, 53 + -73j, 54 + -74j, 55 + -75j, 56 + -76j,
61 + -81j, 62 + -82j, 63 + -83j, 64 + -84j, 65 + -85j, 66 + -86j,
```

6.4.9 Transpose of a matrix

`ctx.`**`mat_transpose`**`(`*matA*`)`

Returns the transposed matrix of matrix matA. Writing `matA.Transpose()` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # returns the transpose matrix of a real matrix
>>> B = A.transpose(); B.show("B")
B:
11, 21, 31, 41, 51, 61,
12, 22, 32, 42, 52, 62,
13, 23, 33, 43, 53, 63,
14, 24, 34, 44, 54, 64,
```

6.4.10 Full Reverse of a matrix

`ctx.mat_reverse_full(matA)`

Returns the full reverse of matrix matA. Writing `matA.ReverseFull()` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # returns the full reverse  of the matrix
>>> B = A.reverse_full(); B.show("B")
B:
64, 63, 62, 61,
54, 53, 52, 51,
44, 43, 42, 41,
34, 33, 32, 31,
24, 23, 22, 21,
14, 13, 12, 11,
```

6.4.11 Row-wise Reverse of a matrix

`ctx.mat_reverse_row_wise(matA)`

Returns the row-wise reverse of matrix matA. Writing `matA.ReverseRowwise()` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # returns the row-wise reverse of the matrix
>>> B = A.reverse_rowwise(); B.show("B")
14, 13, 12, 11,
24, 23, 22, 21,
34, 33, 32, 31,
44, 43, 42, 41,
54, 53, 52, 51,
64, 63, 62, 61,
```

6.4.12 Column-wise Reverse of a matrix

`ctx.mat_reverse_col_wise(matA)`

Returns the column-wise reverse of matrix matA. Writing `matA.ReverseColwise()` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
```

(continues on next page)

(continued from previous page)

```
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # returns the column-wise reverse of the matrix
>>> B = A.reverse_colwise(); B.show("B")
B:
61, 62, 63, 64,
51, 52, 53, 54,
41, 42, 43, 44,
31, 32, 33, 34,
21, 22, 23, 24,
11, 12, 13, 14,
```

6.4.13 Full Replication of a matrix

`ctx.`**`mat_replicate_full`**`(`*matA*, *vertical*, *horizontal*`)`

Returns a full replication of matrix matA. Writing `matA.ReplicateFull(vertical, horizontal)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # returns a full replication (Vertical=2, Horizontal=3) of the matrix
>>> B = A.replicate_full(2, 3); B.show("B")
B:
11, 12, 13, 14, 11, 12, 13, 14, 11, 12, 13, 14,
21, 22, 23, 24, 21, 22, 23, 24, 21, 22, 23, 24,
31, 32, 33, 34, 31, 32, 33, 34, 31, 32, 33, 34,
41, 42, 43, 44, 41, 42, 43, 44, 41, 42, 43, 44,
51, 52, 53, 54, 51, 52, 53, 54, 51, 52, 53, 54,
61, 62, 63, 64, 61, 62, 63, 64, 61, 62, 63, 64,
11, 12, 13, 14, 11, 12, 13, 14, 11, 12, 13, 14,
21, 22, 23, 24, 21, 22, 23, 24, 21, 22, 23, 24,
31, 32, 33, 34, 31, 32, 33, 34, 31, 32, 33, 34,
41, 42, 43, 44, 41, 42, 43, 44, 41, 42, 43, 44,
51, 52, 53, 54, 51, 52, 53, 54, 51, 52, 53, 54,
61, 62, 63, 64, 61, 62, 63, 64, 61, 62, 63, 64,
```

6.4.14 Row-wise Replication of a matrix

`ctx.`**`mat_replicate_row_wise`**`(`*matA*, *horizontal*`)`

Returns a row-wise replication of matrix matA. Writing `matA.ReplicateRowwise(horizontal)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
```

(continues on next page)

(continued from previous page)

```
61, 62, 63, 64,

>>> # returns a row-wise replication (Horizontal=4) of the matrix
>>> B = A.replicate_rowwise(4); B.show("B")
B:
11, 12, 13, 14, 11, 12, 13, 14, 11, 12, 13, 14, 11, 12, 13, 14,
21, 22, 23, 24, 21, 22, 23, 24, 21, 22, 23, 24, 21, 22, 23, 24,
31, 32, 33, 34, 31, 32, 33, 34, 31, 32, 33, 34, 31, 32, 33, 34,
41, 42, 43, 44, 41, 42, 43, 44, 41, 42, 43, 44, 41, 42, 43, 44,
51, 52, 53, 54, 51, 52, 53, 54, 51, 52, 53, 54, 51, 52, 53, 54,
61, 62, 63, 64, 61, 62, 63, 64, 61, 62, 63, 64, 61, 62, 63, 64,
```

6.4.15 Column-wise Replication of a matrix

`ctx.`**`mat_replicate_col_wise`**`(`*matA*, *vertical*`)`

Returns a column-wise replication of matrix matA. Writing `matA.ReplicateColwise(vertical)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x4", ""); A.show("A")
A:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,

>>> # returns a column-wise replication (Vertical=2) of the matrix
>>> B = A.replicate_colwise(2); B.show("B")
B:
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,
11, 12, 13, 14,
21, 22, 23, 24,
31, 32, 33, 34,
41, 42, 43, 44,
51, 52, 53, 54,
61, 62, 63, 64,
```

6.4.16 Horizontal concatenation of two matrices

`ctx.`**`mat_concat_horizontal`**`(`*matA*, *matB*`)`

Returns the horizontal concatenation of matrix matA with matrix matB. Writing `matA.ConcatHorizontal(matB)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,
```

(continues on next page)

(continued from previous page)

```
>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> # returns a column-wise replication (Vertical=2) of the matrix
>>> C = A.concat_horizontal(B); C.show("C")
C:
 11,  12,  13,  14,  15,  16, 911, 912, 913, 914, 915, 916,
 21,  22,  23,  24,  25,  26, 921, 922, 923, 924, 925, 926,
 31,  32,  33,  34,  35,  36, 931, 932, 933, 934, 935, 936,
 41,  42,  43,  44,  45,  46, 941, 942, 943, 944, 945, 946,
 51,  52,  53,  54,  55,  56, 951, 952, 953, 954, 955, 956,
 61,  62,  63,  64,  65,  66, 961, 962, 963, 964, 965, 966,
```

6.4.17 Vertical concatenation of two matrices

`ctx.`**`mat_concat_vertical`**`(`*matA*, *matB*`)`

Returns the vertical concatenation of matrix matA with matrix matB.

Writing `matA.ConcatVertical(matB)` has the same effect

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> # returns a column-wise replication (Vertical=2) of the matrix
>>> C = A.concat_vertical(B); C.show("C")
C:
 11,  12,  13,  14,  15,  16,
 21,  22,  23,  24,  25,  26,
 31,  32,  33,  34,  35,  36,
 41,  42,  43,  44,  45,  46,
 51,  52,  53,  54,  55,  56,
 61,  62,  63,  64,  65,  66,
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,
```

6.5 Basic arithmetic operations

6.5.1 Matrix deep copy (unary plus)

Returns the matrix multiplied with +1. This results in a deep copy of the matrix.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,
>>> B = A
>>> C = +A
>>> A[1,1] = 99
>>> B.show("B")
B:
11, 12, 13, 14, 15, 16,
21, 99, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,
>>> C.show("C")
C:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,
```

6.5.2 Matrix negation (unary minus)

Returns the matrix multiplied with -1.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,
>>> B = -A
>>> B.show("B")
B:
-11, -12, -13, -14, -15, -16,
-21, -99, -23, -24, -25, -26,
-31, -32, -33, -34, -35, -36,
-41, -42, -43, -44, -45, -46,
-51, -52, -53, -54, -55, -56,
-61, -62, -63, -64, -65, -66,
```

6.5.3 General matrix addition

Returns the sum of matrix matA and matrix matB. matA and matB need to be of the same type and need to have the same dimensions. The returned matrix is of the same type as matA. Special rules apply for mixing real and complex matrices.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,
>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,
>>> C = A + B; C.show("C")
C:
 922,  924,  926,  928,  930,  932,
 942,  944,  946,  948,  950,  952,
 962,  964,  966,  968,  970,  972,
 982,  984,  986,  988,  990,  992,
1002, 1004, 1006, 1008, 1010, 1012,
1022, 1024, 1026, 1028, 1030, 1032,
```

6.5.4 Matrix addition of a vector as diagonal matrix

Returns the matrix product of matrix matA and matrix matB. matA and matB need to be of the same type and need to have compatible dimensions. The returned matrix is of the same type as matA. Special rules apply for mixing real and complex matrices.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> # read the first column from the matrix
>>> d = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", "").col(0); d.show("d")
d:
911,
921,
931,
941,
951,
961,

>>> # creates a square matrix with the coefficents of d1 on the diagonal.
>>> D = d.as_diagonal(); D.show("D")
D:
911,   0,   0,   0,   0,   0,
  0, 921,   0,   0,   0,   0,
  0,   0, 931,   0,   0,   0,
  0,   0,   0, 941,   0,   0,
  0,   0,   0,   0, 951,   0,
  0,   0,   0,   0,   0, 961,

>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
```

(continues on next page)

(continued from previous page)

```
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,

>>> C = A + D; C.show("C")    # same as D + A()
C:
 922,   12,   13,   14,   15,   16,
  21,  943,   23,   24,   25,   26,
  31,   32,  964,   34,   35,   36,
  41,   42,   43,  985,   45,   46,
  51,   52,   53,   54, 1006,   56,
  61,   62,   63,   64,   65, 1027,

>>> C = A + d.diagonal_view(); C.show("C")    # same as d.diagonal_view + A()
C:
 922,   12,   13,   14,   15,   16,
  21,  943,   23,   24,   25,   26,
  31,   32,  964,   34,   35,   36,
  41,   42,   43,  985,   45,   46,
  51,   52,   53,   54, 1006,   56,
  61,   62,   63,   64,   65, 1027,
```

6.5.5 Matrix: addition of a scalar

Returns the sum of matrix matA and scalar b, applied to each coefficient of matA. The coefficients of matA and b need to be of the same type. The returned matrix is of the same type as matA. Special rules apply for mixing real and complex matrices and real and complex scalars.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,
>>> B = A + 15;  B.show("B")
B:
26, 27, 28, 29, 30, 31,
36, 37, 38, 39, 40, 41,
46, 47, 48, 49, 50, 51,
56, 57, 58, 59, 60, 61,
66, 67, 68, 69, 70, 71,
76, 77, 78, 79, 80, 81,
```

6.5.6 General matrix subtraction

Returns the difference of matrix matA and matrix matB. matA and matB need to be of the same type and need to have the same dimensions. The returned matrix is of the same type as matA. Special rules apply for mixing real and complex matrices.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,
>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
```

(continues on next page)

(continued from previous page)

```
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,
>>> C = A - B; C.show("C")
C:
-900, -900, -900, -900, -900, -900,
-900, -900, -900, -900, -900, -900,
-900, -900, -900, -900, -900, -900,
-900, -900, -900, -900, -900, -900,
-900, -900, -900, -900, -900, -900,
-900, -900, -900, -900, -900, -900,
```

6.5.7 Matrix: subtraction of a scalar

Returns the difference of matrix matA and scalar b, applied to each coefficient of matA. The coefficients of matA and b need to be of the same type. The returned matrix is of the same type as matA. Special rules apply for mixing real and complex matrices and real and complex scalars.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,
>>> B = A - 15;  B.show("B")
B:
-4, -3, -2, -1,  0,  1,
 6,  7,  8,  9, 10, 11,
16, 17, 18, 19, 20, 21,
26, 27, 28, 29, 30, 31,
36, 37, 38, 39, 40, 41,
46, 47, 48, 49, 50, 51,
```

6.5.8 General matrix multiplication (“gemm”)

See also Eigen [240].

In its general form, matrix multiplication takes the form $\boldsymbol{C} = \boldsymbol{AB}$, where $\boldsymbol{A}$ is a m-by-k matrix, $\boldsymbol{B}$ is a k-by-n matrix, and $\boldsymbol{C}$ is a m-by-n matrix. Also, the transposed or adjoined matrices of $\boldsymbol{A}$ and/or $\boldsymbol{B}$ are often directly used.

In the general form, no information regarding special properties of $\boldsymbol{A}$ and/or $\boldsymbol{B}$ is used. If such information is available, specialized forms of multiplication are often signifantly faster:

- if $\boldsymbol{A}$ or $\boldsymbol{B}$ is a triangular or diagonal matrix, or
- if the multiplication with the inverse of a square, triangular or diagonal matrix is desired, or
- if $\boldsymbol{B}$ is the transpose of $\boldsymbol{A}$.

In the canonical FORTRAN package BLAS, the BLAS Level 3 function ?gemm and the BLAS Level 2 function ?gemv handle this

kind of expression, with the scalars α β and the indicator variables TransA and TransB as additional parameters:

$$?\text{gemm} = \begin{cases} \alpha\boldsymbol{AB} + \beta\boldsymbol{C}, & \text{for TransA = 'N', TransB = 'N',} \\ \alpha\boldsymbol{AB}^T + \beta\boldsymbol{C}, & \text{for TransA = 'N', TransB = 'T',} \\ \alpha\boldsymbol{AB}^H + \beta\boldsymbol{C}, & \text{for TransA = 'N', TransB = 'C',} \\ \alpha\boldsymbol{A}^T\boldsymbol{B} + \beta\boldsymbol{C}, & \text{for TransA = 'T', TransB = 'N',} \\ \alpha\boldsymbol{A}^T\boldsymbol{B}^T + \beta\boldsymbol{C}, & \text{for TransA = 'T', TransB = 'T',} \\ \alpha\boldsymbol{A}^T\boldsymbol{B}^H + \beta\boldsymbol{C}, & \text{for TransA = 'T', TransB = 'C',} \\ \alpha\boldsymbol{A}^H\boldsymbol{B} + \beta\boldsymbol{C}, & \text{for TransA = 'C', TransB = 'N',} \\ \alpha\boldsymbol{A}^H\boldsymbol{B}^T + \beta\boldsymbol{C}, & \text{for TransA = 'C', TransB = 'T',} \\ \alpha\boldsymbol{A}^H\boldsymbol{B}^H + \beta\boldsymbol{C}, & \text{for TransA = 'C', TransB = 'C',} \end{cases}$$

$$?\text{gemv} = \begin{cases} \alpha\boldsymbol{Ax} + \beta\boldsymbol{y}, & \text{for TransA = 'N',} \\ \alpha\boldsymbol{A}^T\boldsymbol{x} + \beta\boldsymbol{y}, & \text{for TransA = 'T',} \\ \alpha\boldsymbol{A}^H\boldsymbol{x} + \beta\boldsymbol{y}, & \text{for TransA = 'C'.} \end{cases}$$

We use the `transpose()` and `adjoint()` properties to compute these expressions efficiently. Here are some examples with real matrices:

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,
>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,
>>> C = A * B; C.show("C")
C:
 75991,  76072,  76153,  76234,  76315,  76396,
132151, 132292, 132433, 132574, 132715, 132856,
188311, 188512, 188713, 188914, 189115, 189316,
244471, 244732, 244993, 245254, 245515, 245776,
300631, 300952, 301273, 301594, 301915, 302236,
356791, 357172, 357553, 357934, 358315, 358696,

>>> C = A.transpose() * B;  C.show("C")
C:
203926, 204142, 204358, 204574, 204790, 205006,
209542, 209764, 209986, 210208, 210430, 210652,
215158, 215386, 215614, 215842, 216070, 216298,
220774, 221008, 221242, 221476, 221710, 221944,
226390, 226630, 226870, 227110, 227350, 227590,
232006, 232252, 232498, 232744, 232990, 233236,

>>> C = A * B.transpose();  C.show("C")
C:
 74011,  74821,  75631,  76441,  77251,  78061,
128821, 130231, 131641, 133051, 134461, 135871,
183631, 185641, 187651, 189661, 191671, 193681,
238441, 241051, 243661, 246271, 248881, 251491,
```

(continues on next page)

(continued from previous page)

```
293251, 296461, 299671, 302881, 306091, 309301,
348061, 351871, 355681, 359491, 363301, 367111,
```

Here are some examples with complex matrices:

```
>>> TName = "DecCplxTableA6x6"; Query = "where row<4 and col<4"
>>> A = ctx.read_from_sqlite(mp14.dbpath(), TName, Query); A.show("A")
A:
11 + 31j, 12 + 32j, 13 + 33j, 14 + 34j,
21 + 41j, 22 + 42j, 23 + 43j, 24 + 44j,
31 + 51j, 32 + 52j, 33 + 53j, 34 + 54j,
41 + 61j, 42 + 62j, 43 + 63j, 44 + 64j,


>>> TName = "DecCplxTableB6x6"; Query = "where row<4 and col<4"
>>> B = ctx.read_from_sqlite(mp14.dbpath(), TName, Query); B.show("B")
B:
45 + 7.5j, 2.9 + 36j,  11 + 13j,  37 + 37j,
 13 + 29j,  38 + 41j,  22 + 44j,  28 + 20j,
32 + 9.0j,  42 + 49j,  47 + 11j,  32 + 49j,
 34 + 42j, 6.0 + 16j,  20 + 38j,  22 + 17j,

>>> C = A.adjoint() * B;  C.show("C")
C:
7596.5 - 2941.5j,  8649.9 - 723.9j,     7946 - 1894j,     8392 - 2226j,
7808.0 - 2978.0j,  8880.8 - 670.8j,     8152 - 1888j,     8634 - 2222j,
8019.5 - 3014.5j,  9111.7 - 617.7j,     8358 - 1882j,     8876 - 2218j,
8231.0 - 3051.0j,  9342.6 - 564.6j,     8564 - 1876j,     9118 - 2214j,

>>> C = B.adjoint() * A;  C.show("C")
C:
7596.5 + 2941.5j, 7808.0 + 2978.0j, 8019.5 + 3014.5j, 8231.0 + 3051.0j,
 8649.9 + 723.9j,  8880.8 + 670.8j,  9111.7 + 617.7j,  9342.6 + 564.6j,
    7946 + 1894j,     8152 + 1888j,     8358 + 1882j,     8564 + 1876j,
    8392 + 2226j,     8634 + 2222j,     8876 + 2218j,     9118 + 2214j,


>>> C = A * B.adjoint();  C.show("C")
C:
 4262.3 + 1907.3j,      5620 + 1634j,  5791.0 + 3459.0j,  4660.0 + 1268.0j,
 6156.3 + 1931.3j,      7970 + 1304j,  8501.0 + 3809.0j,   6610.0 + 958.0j,
 8050.3 + 1955.3j,      10320 + 974j, 11211.0 + 4159.0j,   8560.0 + 648.0j,
 9944.3 + 1979.3j,      12670 + 644j, 13921.0 + 4509.0j,  10510.0 + 338.0j,

>>> C = B * A.adjoint();  C.show("C")
C:
 4262.3 - 1907.3j,  6156.3 - 1931.3j,  8050.3 - 1955.3j,  9944.3 - 1979.3j,
     5620 - 1634j,      7970 - 1304j,      10320 - 974j,      12670 - 644j,
 5791.0 - 3459.0j,  8501.0 - 3809.0j, 11211.0 - 4159.0j, 13921.0 - 4509.0j,
 4660.0 - 1268.0j,   6610.0 - 958.0j,   8560.0 - 648.0j,  10510.0 - 338.0j,
```

6.5.9 Arithmetic comparisons with a scalar or a matrix

`ctx.mat_count`(*matA*, *query*, *comparator*)

Returns the number of coefficients in matrix matA which are greater than the corresponding coefficients in matrix matB.

Writing `matA.count(query, comparator)` has the same effect.

Queries are: ">, <, >=, <=, !=, ==", For complex matrices, the absolute values are compared

```
>>> from xlcalcnet import *
>>> mpm.dps = 40;
>>> A = mp14.xrf().read_from_sqlite(mp14.dbpath(), "MpfrTableA4x4", "")
>>> B = mp14.xrf().read_from_sqlite(mp14.dbpath(), "MpfrTableB4x4", "")
```

6.6 Descriptive Statistics

6.6.1 Partial Sum

`ctx.mat_sum`(*mat*, *partialmode*)

Calculates the PartialSum. Writing `matA.sum(partialmode)` has the same effect.

Here partialmode is set as follows: 1 for full matrix calculation, 2 for rowwise calculation, 3 for columnwise calculation.

See also: Eigen [237].

6.6.2 Partial Product

`ctx.mat_prod`(*matA*, *partialmode*)

Calculates the PartialProd. Writing `matA.prod(partialmode)` has the same effect.

Here partialmode is set as follows: 1 for full matrix calculation, 2 for rowwise calculation, 3 for columnwise calculation.

See also: Eigen [237].

6.6.3 Partial Arithmetic Mean

`ctx.mat_mean`(*matA*, *partialmode*)

Calculates the PartialMean. Writing `matA.mean(partialmode)` has the same effect.

Here partialmode is set as follows: 1 for full matrix calculation, 2 for rowwise calculation, 3 for columnwise calculation.

See also: Eigen [237].

6.6.4 Partial Minimal Coefficient

`ctx.mat_min_coeff`(*matA*, *partialmode*)

Calculates the PartialMinCoeff. Writing `matA.min_coeff(partialmode)` has the same effect.

Here partialmode is set as follows: 1 for full matrix calculation, 2 for rowwise calculation, 3 for columnwise calculation.

See also: Eigen [237].

6.6.5 Partial Maximal Coefficient

`ctx.mat_max_coeff`(*matA*, *partialmode*)

Calculates the PartialMaxCoeff. Writing `matA.max_coeff(partialmode)` has the same effect.

Here partialmode is set as follows: 1 for full matrix calculation, 2 for rowwise calculation, 3 for columnwise calculation.

See also: Eigen [237].

6.6.6 Trace

`ctx.mat_trace`(*matA*)

Returns the trace of the matrix A. Writing `matA.Trace()` has the same effect.

See also: Eigen [237], Wikipedia [1393].

6.6.7 Partial Squared Norm

`ctx.mat_squared_norm(matA, partialmode)`

Calculates the PartialSquaredNorm. Writing `matA.PartialSquaredNorm(partialmode)` has the same effect.

Here partialmode is set as follows: 1 for full matrix calculation, 2 for rowwise calculation, 3 for columnwise calculation.

See also: Eigen [237].

6.6.8 Partial Norm

`ctx.mat_norm(matA, partialmode)`

Calculates the PartialNorm. Writing `matA.PartialNorm(partialmode)` has the same effect.

Here partialmode is set as follows: 1 for full matrix calculation, 2 for rowwise calculation, 3 for columnwise calculation.

See also: Eigen [237].

6.6.9 Partial Stable Norm

`ctx.mat_stable_norm(matA, partialmode)`

Calculates the PartialStableNorm. Writing `matA.PartialStableNorm(partialmode)` has the same effect.

Here partialmode is set as follows: 1 for full matrix calculation, 2 for rowwise calculation, 3 for columnwise calculation.

See also: Eigen [237].

6.6.10 Covariance matrix

`ctx.mat_covariance(matA)`

Calculates the covariance matrix. Writing `matA.Covariance()` has the same effect.

See also: Wikipedia [1389], Wikipedia [1369].

6.6.11 Correlation matrix

`ctx.mat_correlation(matA)`

Calculates the correlation matrix. Writing `matA.Correlation_mat()` has the same effect.

See also: Wikipedia [1367].

6.7 Standard decompositions and linear solving

6.7.1 Cholesky Decomposition with Pivoting

`ctx.mat_ldlt`(*matA*, *Query*, *matB*)

Returns the Cholesky decomposition of the symmetric matrix *matA* $= A = A = P^T LDLP$, with partial pivoting.

Writing `matA.LDLT(Query, matB)` has the same effect

See also Eigen [230], Wikipedia [1425], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

B
Optional. A general n-by-m matrix of the same type as A. You need to specify B only if you want to solve the linear equation $AX = B$

Results:

info
An integer indicating whether the decomposition was successful (=0) or not(=1).

ispos
An integer indicating whether the matrix A is semidefinit positve (=0) or not(=1).

isneg
An integer indicating whether the matrix A is semidefinit negative (=0) or not(=1).

det
A scalar of a return type matching A. The determinant of A.

rcond
A scalar of the same return type as det. The condition number of A.

X
A general matrix of the same type and dimension as B. The solution to $AX = B$.

Inv
A square matrix of the same type and dimension as A. The inverse of A, A^{-1}.

P
A square matrix of the same type and dimension as A. The permutation matrix P in the decomposition $A = P^T LDLP$.

L
A square matrix of the same type and dimension as A, containing the matrix L in the decomposition $A = P^T LDLP$.

D
A square matrix of the same type and dimension as A, containing the matrix D in the decomposition $A = P^T LDLP$.

Perform a robust Cholesky decomposition of a positive semidefinite or negative semidefinite matrix such that $A = P^T LDLP$, where P is a permutation matrix, L is lower triangular with a unit diagonal and D is a diagonal matrix. The decomposition uses pivoting to ensure stability, so that L will have zeros in the bottom right rank(A) - n submatrix. Avoiding the square root on D also stabilizes the computation. Remember that Cholesky decompositions are not rank-revealing. Also, do not use a Cholesky decomposition to determine whether a system of equations has a solution

rankUpdate: Update the LDLT decomposition: given $A = LDLT$, efficiently compute the decomposition of $A+\sigma ww^T$. Parameters: w a vector to be incorporated into the decomposition. σ a scalar, +1 for updates and -1 for "downdates", which correspond to removing previously-added column vectors. Optional; default value is +1.

solve: Returns a solution x of $Ax = b$ using the current decomposition of A. This function also supports in-place solves using the syntax x = decompositionObject.solve(x) .

This method just tries to find as good a solution as possible. If you want to check whether a solution exists or if it is accurate, just call this function to get a result and then compute the error of this result, or use MatrixBase::isApprox() directly, for instance like this:

bool a solution exists = (A*result).isApprox(b, precision);

This method avoids dividing by zero, so that the non-existence of a solution does not by itself mean that you will get inf or nan values.

More precisely, this method solves $Ax = b$ using the decomposition $A = P^T LDL * P$ by solving the systems

$P^T y1 = b, LY2 = y1, Dy3 = y2, L * y4 = y3$ and $Px = y4$ in succession.

If the matrix A is singular, then D will also be singular (all the other matrices are invertible). In that case, the least-square solution of $Dy3 = y2$ is computed. This does not mean that this function computes the least-square solution of $Ax = b$ if A is singular.

6.7.1.1 Example for a real symmetric matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAA6x6", ""); A.show("A")

>>> # This needs to be a positive definite matrix
A:
44.9, 25.5, 50.0, 47.9, 26.4, 62.0,
25.5, 24.3, 49.1, 95.0, 29.0, 46.6,
50.0, 49.1, 55.5, 84.0, 44.4, 26.7,
47.9, 95.0, 84.0, 64.5, 39.5, 87.5,
26.4, 29.0, 44.4, 39.5, 39.8, 12.3,
62.0, 46.6, 26.7, 87.5, 12.3, 85.0,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> Query = "ispos, isneg, info, rcond, L, U, D, P, X, Inverse"
>>> Res = A.eigen_ldlt2(Query, B)
>>> print("Info : ", Res["Info"])
Info :  0
>>> print("Rcond: ", Res["Rcond"])
Rcond:  0.0073622045623195067245064838163673064
>>> print("Ispos: ", Res["Ispos"])
Ispos:  False
>>> print("Isneg: ", Res["Isneg"])
Isneg:  False

>>> Res["X"].show("X")
X:
 -7.46,  -7.47,  -7.48,  -7.48,  -7.49,  -7.50,
 -4.07,  -4.07,  -4.08,  -4.08,  -4.09,  -4.09,
 -25.5,  -25.5,  -25.6,  -25.6,  -25.6,  -25.6,
-0.158, -0.158, -0.158, -0.158, -0.158, -0.158,
  54.5,   54.5,   54.6,   54.6,   54.7,   54.7,
  19.3,   19.3,   19.3,   19.3,   19.4,   19.4,

>>> (A * Res["X"]).show("A * X")
```

(continues on next page)

(continued from previous page)

```
A * X:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> (B - A * Res["x"]).show("B - A * X")
B - A * X:
       0,        0,   -2E-32,   -1E-32,   -1E-32,    1E-32,
 1.6E-32,    4E-33,    9E-33,   -9E-33,  2.0E-32,  2.4E-32,
 1.7E-32,  1.3E-32,    6E-33, -1.0E-32,  2.5E-32,    8E-33,
  -2E-32,   -1E-32,   -3E-32,   -3E-32,   -1E-32,   -2E-32,
  -2E-33,   -3E-33,        0,   -8E-33,    6E-33,    4E-33,
   2E-33,  1.4E-32, -1.4E-32,   -9E-33,  1.1E-32,    5E-33,

>>> Res["P"].T.show("P^T")
P^T:
0, 0, 0, 1, 0, 0,
0, 0, 0, 0, 0, 1,
0, 0, 1, 0, 0, 0,
0, 1, 0, 0, 0, 0,
0, 0, 0, 0, 1, 0,
1, 0, 0, 0, 0, 0,

>>> Res["L"].show("L")
L:
      1,       0,       0,       0,       0,       0,
   1.03,       1,       0,       0,       0,       0,
  0.314,   -2.21,       1,       0,       0,       0,
  0.729,   0.623, -0.0271,       1,       0,       0,
  0.145,   -1.05,   0.580,   0.362,       1,       0,
  0.548,   -1.84,   0.805,   -3.59,   0.514,       1,

>>> Res["D"].show("D")
D:
 85.0,     0,     0,     0,     0,     0,
    0, -25.6,     0,     0,     0,     0,
    0,     0,   172,     0,     0,     0,
    0,     0,     0,  9.46,     0,     0,
    0,     0,     0,     0,  6.99,     0,
    0,     0,     0,     0,     0,  -150,

>>> Res["U"].show("U")
U:
      1,    1.03,   0.314,   0.729,   0.145,   0.548,
      0,       1,   -2.21,   0.623,   -1.05,   -1.84,
      0,       0,       1, -0.0271,   0.580,   0.805,
      0,       0,       0,       1,   0.362,   -3.59,
      0,       0,       0,       0,       1,   0.514,
      0,       0,       0,       0,       0,       1,

>>> Res["P"].show("P")
P:
0, 0, 0, 0, 0, 1,
0, 0, 0, 1, 0, 0,
0, 0, 1, 0, 0, 0,
1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 0,
0, 1, 0, 0, 0, 0,

>>> (A - Res["P"].T * Res["L"]  * Res["D"] * Res["U"] * Res["P"]).show("A - P^T * L * D * U * P")
A - P^T * L * D * U * P:
     0,      0,      0,      0,      0,      0,
     0,      0, -1E-34, -1E-34, -1E-34,      0,
```

(continues on next page)

(continued from previous page)

```
     0,      0,      0, -1E-34,  1E-34,      0,
     0, -1E-34, -1E-34,      0,      0,  3E-34,
     0,      0,  2E-34,      0,      0,      0,
     0,      0,      0,  3E-34,      0,      0,

>>> Res["Inverse"].show("Inverse")
Inverse:
  0.0293,  -0.0252,   0.0436, -0.00894,  -0.0389, -0.00643,
 -0.0252, -0.00666,  0.00269,   0.0130,  0.00342,  0.00733,
  0.0436,  0.00269,   0.0547,  0.00851,  -0.0859,  -0.0468,
-0.00894,   0.0130,  0.00851, -0.00235,  -0.0109, 0.000736,
 -0.0389,  0.00342,  -0.0859,  -0.0109,    0.141,   0.0442,
-0.00643,  0.00733,  -0.0468, 0.000736,   0.0442,   0.0200,

>>> (A * Res["Inverse"]).show("A * Inverse")
A * Inverse:
     1.00,    -5E-36,     1E-35,   1.1E-36,     2E-35,    -1E-35,
  1.1E-35,      1.00,     2E-35,  -4.9E-36,    -6E-35,   1.1E-35,
  3.9E-35,    -1E-36,      1.00,  -2.1E-36,  -1.0E-34,  -2.1E-35,
 -2.8E-35,    -1E-36,    -2E-35,      1.00,     5E-35,     1E-35,
 2.16E-35,   1.6E-36,   2.5E-35, -5.17E-36,      1.00,  -1.2E-35,
   -1E-36,     5E-36,         0,  -1.6E-36,    -1E-35,      1.00,
```

6.7.1.2 Example for a hermitian matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableRandomSAPosDefA6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")

>>> # This needs to be a self-adjoint, positive definite matrix
A:
   91.0 + 0j, 12.0 - 3.60j, 22.0 - 7.40j, 14.0 + 1.10j,
12.0 + 3.60j,    77.0 + 0j, 2.50 - 2.00j, 3.40 - 7.60j,
22.0 + 7.40j, 2.50 + 2.00j,    91.0 + 0j, 17.0 + 3.70j,
14.0 - 1.10j, 3.40 + 7.60j, 17.0 - 3.70j,    74.0 + 0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> Query = "ispos, isneg, info, rcond, L, U, D, P, X, Inverse"
>>> Res = A.eigen_ldlt2(Query, B)
>>> print("Info : ", Res["Info"])
Info :  0 + 0j
>>> print("Rcond: ", Res["Rcond"])
Rcond:  0.41520387595592569380609948826249956 9 + 0j
>>> print("Ispos: ", Res["Ispos"])
Ispos:  False
>>> print("Isneg: ", Res["Isneg"])
Isneg:  False

>>> Res["X"].show("X")
X:
    0.376 - 0.0288j,     -0.222 + 0.281j,   -0.0865 + 0.0417j,      0.229 + 0.303j,
    0.0290 + 0.383j,      0.511 + 0.498j,      0.227 + 0.583j,      0.309 + 0.210j,
    0.219 - 0.0602j,      0.535 + 0.463j,    0.527 - 0.00173j,      0.288 + 0.417j,
     0.379 + 0.583j, 0.000510 + 0.00479j,      0.214 + 0.481j,     0.170 + 0.0528j,

>>> (A * Res["X"]).show("A * X")
```

(continues on next page)

(continued from previous page)

```
A * X:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> (B - A * Res["x"]).show("B - A * X")
B - A * X:
        0 - 5.00E-35j,        -1.80E-34 + 0j,               0 + 0j,               0 + 0j,
              0 + 0j, -1.00E-34 - 1.00E-34j,         0 - 2.00E-34j,         0 - 1.00E-34j,
              0 + 0j,         0 - 1.00E-34j,               0 + 0j,               0 + 0j,
-1.00E-34 + 1.00E-34j,        -1.00E-35 + 0j,         0 - 1.00E-34j, -1.00E-34 - 1.00E-34j,

>>> Res["P"].T.show("P^T")
P^T:
1.00 + 0j,    0 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j, 1.00 + 0j,    0 + 0j,
   0 + 0j, 1.00 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j,    0 + 0j, 1.00 + 0j,

>>> Res["L"].show("L")
L:
         1.00 + 0j,             0 + 0j,             0 + 0j,             0 + 0j,
   0.242 + 0.0813j,          1.00 + 0j,             0 + 0j,             0 + 0j,
   0.132 + 0.0396j, -0.00815 - 0.0223j,          1.00 + 0j,             0 + 0j,
   0.154 - 0.0121j,    0.161 - 0.0270j,    0.0220 + 0.106j,          1.00 + 0j,

>>> Res["D"].show("D")
D:
91.0 + 0j,    0 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j, 85.1 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j, 75.2 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j,    0 + 0j, 68.7 + 0j,

>>> Res["U"].show("U")
U:
         1.00 + 0j,    0.242 - 0.0813j,    0.132 - 0.0396j,    0.154 + 0.0121j,
            0 + 0j,          1.00 + 0j, -0.00815 + 0.0223j,    0.161 + 0.0270j,
            0 + 0j,             0 + 0j,          1.00 + 0j,    0.0220 - 0.106j,
            0 + 0j,             0 + 0j,             0 + 0j,          1.00 + 0j,

>>> Res["P"].show("P")
P:
1.00 + 0j,    0 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j, 1.00 + 0j,    0 + 0j,
   0 + 0j, 1.00 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j,    0 + 0j, 1.00 + 0j,

>>> (A - Res["P"].T * Res["L"]  * Res["D"] * Res["U"] * Res["P"]).show("A - P^T * L * D * U * P")
A - P^T * L * D * U * P:
       0 + 0j,        0 + 0j,        0 + 0j,        0 + 0j,
       0 + 0j,        0 + 0j, 0 + 1.00E-35j, 0 + 2.00E-35j,
       0 + 0j,        0 + 0j,        0 + 0j,        0 + 0j,
       0 + 0j, 0 - 2.00E-35j,        0 + 0j,        0 + 0j,

>>> Res["Inverse"].show("Inverse")
Inverse:
   0.0122 - 1.00E-39j,  -0.00177 + 0.000794j,  -0.00259 + 0.000957j,  -0.00167 - 0.000490j,
 -0.00177 - 0.000794j,    0.0135 - 1.00E-40j, 0.000120 + 0.0000425j,  -0.000321 + 0.00154j,
 -0.00259 - 0.000957j, 0.000120 - 0.0000425j,    0.0121 + 2.60E-40j,  -0.00231 - 0.000373j,
 -0.00167 + 0.000490j,  -0.000321 - 0.00154j,  -0.00231 + 0.000373j,           0.0146 + 0j,

>>> (A * Res["Inverse"]).show("A * Inverse")
A * Inverse:
     1.00 - 1.20E-37j,  1.03E-36 - 2.00E-37j, -8.00E-37 + 1.30E-37j,               0 + 0j,
       -3.90E-37 + 0j,             1.00 + 0j,         6.00E-38 + 0j, -2.00E-37 + 1.00E-36j,
```

(continues on next page)

(continued from previous page)

```
-8.00E-37 - 1.00E-38j,  7.00E-38 - 1.00E-37j,      1.00 + 6.00E-38j,         1.00E-36 + 0j,
 1.00E-36 + 1.00E-37j,         0 + 1.00E-36j,         1.00E-36 + 0j,      1.00 - 5.00E-38j,
```

6.7.2 LU Decomposition with partial Pivoting

`ctx.mat_partial_piv_lu`(*matA*, *Query*, *matB*)

Returns the LU decomposition of the general square matrix *matA* $= A = PLU$, with partial pivoting.

Writing `matA.PartialPivLU(Query, matB)` has the same effect

See also Eigen [232], Wikipedia [1412], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

B
Optional. A general n-by-m matrix of the same type as A. You need to specify B only if you want to solve the linear equation $AX = B$

Results:

det
A scalar of a return type matching A. The determinant of A.

rcond
A scalar of the same return type as det. The condition number of A.

X
A general matrix of the same type and dimension as B. The solution to $AX = B$.

Inv
A square matrix of the same type and dimension as A. The inverse of A, A^{-1}.

P
A square matrix of the same type and dimension as A. The permutation matrix P in the decomposition $A = PLU$.

LU
A square matrix of the same type and dimension as A, containing the matrices L and U in the decomposition $A = PLU$.

This class represents a LU decomposition of a square invertible matrix, with partial pivoting: the matrix A is decomposed as A = PLU where L is unit-lower-triangular, U is upper-triangular, and P is a permutation matrix.

Typically, partial pivoting LU decomposition is only considered numerically stable for square invertible matrices. Thus LAPACK's dgesv and dgesvx require the matrix to be square and invertible. The present class does the same. It will assert that the matrix is square, but it won't (actually it cannot) check that the matrix is invertible: it is your task to check that you only use this decomposition on invertible matrices. The guaranteed safe alternative, working for all matrices, is the full pivoting LU decomposition, provided by class FullPivLU.

This is not a rank-revealing LU decomposition. Many features are intentionally absent from this class, such as rank computation. If you need these features, use class FullPivLU.

This LU decomposition is suitable to invert invertible matrices. It is what MatrixBase::inverse() uses in the general case. On the other hand, it is not suitable to determine whether a given matrix is invertible. The data of the LU decomposition can be directly accessed through the methods matrixLU(), permutationP().

6.7.2.1 Example for a real general square matrix (Python)

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

>>> # This needs to be a square matrix
```

(continues on next page)

(continued from previous page)

```
A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> Query = "rcond, det, LU, P, X, Inverse"
>>> Res = A.eigen_partialPivLu2(Query, B)
>>> print("Rcond: ", Res["Rcond"])
Rcond:  0.0027248962455782943842188862118634889 1
>>> print("Det  : ", Res["Det"])
Det  :  48772174.208000000000000000000000000005

>>> Res["X"].show("X")
X:
 21.6,  21.7,  21.7,  21.7,  21.7,  21.8,
 8.21,  8.22,  8.22,  8.23,  8.23,  8.24,
-20.5, -20.6, -20.6, -20.6, -20.6, -20.6,
-15.3, -15.3, -15.4, -15.4, -15.4, -15.4,
 16.9,  16.9,  17.0,  17.0,  17.0,  17.0,
 8.73,  8.76,  8.78,  8.80,  8.82,  8.85,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> (B - A * Res["x"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
   2E-33,   -4E-33, -1.3E-32,    1E-33,   -1E-33,   -3E-33,
   3E-33,   -1E-33,   -2E-33,    4E-33,   -4E-33, -1.1E-32,
   3E-33,   -4E-33,   -3E-33,   -1E-33,   -2E-33,   -4E-33,
   1E-33,    0E-33,   -1E-33,   -1E-33,    1E-33,   -1E-33,
   2E-33,   -6E-33,   -2E-33,   -2E-33,    0E-33,   -5E-33,
   0E-33,   -2E-33,   -3E-33,   -1E-33,   -1E-33,   -2E-33,

>>> Res["P"].show("P")
P:
1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1,
0, 0, 0, 0, 1, 0,
0, 0, 0, 1, 0, 0,
0, 1, 0, 0, 0, 0,
0, 0, 1, 0, 0, 0,

>>> L = Res["LU"].unit_lower_triangle(); L.show("L")
L:
      1,       0,       0,       0,       0,       0,
   0.25,       1,       0,       0,       0,       0,
  0.479,   0.749,       1,       0,       0,       0,
  0.148,   0.908,   0.510,       1,       0,       0,
```

(continues on next page)

(continued from previous page)

```
  0.958,  -0.795,  -0.465, -0.0452,       1,       0,
  0.563,   0.377, -0.0613,  -0.308,   0.282,       1,

>>> U = Res["LU"].upper_triangle(); U.show("U")
U:
   48,    43,    31,    19,    14,    24,
    0,  39.3, -6.55,  41.3,  32.5,  41.0,
    0,     0,  32.1, -39.6,  11.0, -19.2,
    0,     0,     0, -17.3, -2.17, -7.97,
    0,     0,     0,     0,  31.4,  10.3,
    0,     0,     0,     0,     0,  1.49,

>>> (Res["P"] * L * U).show("P * L * U (should be equal to A)")
P * L * U (should be equal to A):
 48.0,  43.0,  31.0,  19.0,  14.0,  24.0,
 27.0,  39.0,  13.0,  34.0,  29.0,  37.0,
 46.0,  10.0,  20.0,  4.60,  14.0,  10.0,
 7.10,  42.0,  15.0,  2.80,  35.0,  23.0,
 12.0,  50.0,  1.20,  46.0,  36.0,  47.0,
 23.0,  50.0,  42.0, 0.440,  42.0,  23.0,

>>> Res["Inverse"].show("Inverse of A")
Inverse of A:
 0.0113,  0.0426, -0.0633, 0.00768, -0.0128,  0.0375,
 0.0459,  0.0319,  -0.201, -0.0115,  0.0136,   0.127,
-0.0121, -0.0523,   0.129, -0.0340,  0.0322, -0.0835,
0.00719,  0.0757,  -0.283,  -0.141,  0.0901,   0.228,
-0.0289,  0.0940,  -0.220, -0.0636,  0.0627,   0.169,
-0.0363,  -0.190,   0.672,   0.198,  -0.148,  -0.473,

>>> (A * Res["Inverse"]).show("A * Inverse (should be an identity matrix)")
A * Inverse (should be an identity matrix):
    1.00,    0E-35,    0E-34,    0E-35,    0E-35,    0E-34,
  -8E-36,     1.00, -1.2E-34,    1E-35,   -1E-35,   -2E-35,
   0E-35,   -1E-35,     1.00,    0E-35,    0E-35,    1E-34,
   3E-36,    0E-35,    0E-34,     1.00,    2E-35,    1E-34,
   3E-36,   -1E-35,    0E-34,    0E-35,     1.00,    1E-34,
   1E-35,    1E-35,    0E-34,   -1E-35,    0E-35,     1.00,
```

6.7.2.2 Example for a complex matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")

>>> # This needs to be an invertible matrix
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableA6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> Query = "rcond, det, LU, P, X, Inverse"
>>> Res = A.eigen_partialPivLu2(Query, B)
>>> print("Rcond: ", Res["Rcond"])
```

(continues on next page)

(continued from previous page)

```
Rcond:  0.018712448036456211456272670465202252 6 + 0j
>>> print("Det  : ", Res["Det"])
Det  :  1083952.900000000000000000000000000002 - 676519.300000000000000000000000000002j

>>> Res["X"].show("X")
X:
-0.869 - 3.37j, -0.858 - 3.37j, -0.846 - 3.38j, -0.835 - 3.38j,
 -1.82 - 3.22j,  -1.82 - 3.22j,  -1.82 - 3.23j,  -1.81 - 3.23j,
  2.15 + 3.53j,   2.15 + 3.53j,   2.16 + 3.53j,   2.16 + 3.53j,
  1.81 + 3.52j,   1.82 + 3.53j,   1.84 + 3.54j,   1.86 + 3.55j,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> (B - A * Res["x"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
       -1.00E-34 + 0j, -2.00E-34 - 2.00E-33j,         1.00E-33 + 0j,  1.20E-33 - 2.00E-33j,
 3.50E-33 - 1.00E-33j,        -1.20E-33 + 0j, -6.00E-34 - 2.00E-33j,  6.00E-34 - 2.00E-33j,
 1.00E-33 - 1.00E-33j, -1.00E-33 - 1.00E-33j,         0 - 3.00E-33j,         0 - 1.00E-33j,
 1.40E-33 - 1.00E-33j,        -1.00E-34 + 0j,                0 + 0j,         3.00E-34 + 0j,

>>> Res["P"].show("P")
P:
   0 + 0j,    0 + 0j,    0 + 0j, 1.00 + 0j,
   0 + 0j,    0 + 0j, 1.00 + 0j,    0 + 0j,
1.00 + 0j,    0 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j, 1.00 + 0j,    0 + 0j,    0 + 0j,

>>> L = Res["LU"].unit_lower_triangle(); L.show("L")
L:
      1.00 + 0j,          0 + 0j,          0 + 0j,          0 + 0j,
 0.502 - 0.355j,       1.00 + 0j,          0 + 0j,          0 + 0j,
 0.632 - 0.560j,  0.314 + 0.471j,       1.00 + 0j,          0 + 0j,
 0.568 + 0.151j,  0.866 - 0.190j, -0.137 - 0.755j,       1.00 + 0j,

>>> U = Res["LU"].upper_triangle(); U.show("U")
U:
 34.0 + 42.0j,  6.00 + 16.0j,  20.0 + 38.0j,  22.0 + 17.0j,
       0 + 0j,  33.3 + 43.1j, 23.4 - 0.968j,  14.9 + 48.3j,
       0 + 0j,        0 + 0j, -30.7 - 10.6j,  31.7 + 16.4j,
       0 + 0j,        0 + 0j,        0 + 0j, -12.0 - 5.80j,

>>> (Res["P"] * L * U).show("P * L * U (should be equal to A)")
P * L * U (should be equal to A):
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,

>>> Res["Inverse"].show("Inverse of A")
Inverse of A:
   0.0516 + 0.0468j,    0.0667 - 0.0273j, -0.0516 + 0.000462j,   -0.0635 - 0.0282j,
   0.0449 + 0.0423j,    0.0801 - 0.0400j, -0.0447 + 0.000795j,  -0.0814 - 0.00721j,
  -0.0583 - 0.0458j,   -0.0783 + 0.0245j,   0.0599 + 0.00584j,    0.0786 + 0.0125j,
  -0.0338 - 0.0465j,   -0.0674 + 0.0325j,    0.0409 - 0.0104j,    0.0738 + 0.0219j,

>>> (A * Res["Inverse"]).show("A * Inverse (should be an identity matrix)")
A * Inverse (should be an identity matrix):
     1.00 - 1.00E-35j, -2.00E-35 + 2.00E-35j,  1.00E-35 + 1.00E-35j,  1.00E-35 + 1.00E-35j,
-2.60E-35 - 3.00E-35j,      1.00 + 1.00E-36j,         0 - 1.10E-35j,                0 + 0j,
-2.00E-35 - 2.00E-35j, -1.00E-35 + 4.00E-35j,      1.00 - 1.00E-35j,  1.00E-35 + 1.00E-35j,
 3.00E-36 - 1.00E-35j, -1.00E-35 - 1.10E-35j,         0 + 2.00E-36j,      1.00 + 1.00E-35j,
```

6.7.3 LU Decomposition with full Pivoting

`ctx.mat_full_piv_lu`(*matA*, *Query*, *matB*)

Returns the LU decomposition of the general square matrix *matA* $= A = PLUQ$, with full pivoting. See also Eigen [224], Wikipedia [1426], Wikipedia [1430].

Writing `matA.FullPivLU(Query, matB)` has the same effect

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

B
Optional. A general n-by-m matrix of the same type as A. You need to specify B only if you want to solve the linear equation $AX = B$

Results:

isinjective
An boolean value indicating whether A is injective.

isinvertible
An boolean value indicating whether A is invertible.

issurjective
An boolean value indicating whether A is surjective.

rank
An integer. The rank of A.

det
A scalar of a return type matching A. The determinant of A.

rcond
A scalar of the same return type as det. The condition number of A.

X
A general matrix of the same type and dimension as B. The solution to $AX = B$.

Inv
A square matrix of the same type and dimension as A. The inverse of A, A^{-1}.

P
A square matrix of the same type and dimension as A. The permutation matrix P in the decomposition $A = PLUQ$.

Q
A square matrix of the same type and dimension as A. The permutation matrix Q in the decomposition $A = PLUQ$.

LU
A square matrix of the same type and dimension as A, containing the matrices L and U in the decomposition $A = PLUQ$.

This class represents a LU decomposition of any matrix, with complete pivoting: the matrix A is decomposed as $A = PLUQ$ where L is unit-lower-triangular, U is upper-triangular, and P and Q are permutation matrices. This is a rank-revealing LU decomposition. The eigenvalues (diagonal coefficients) of U are sorted in such a way that any zeros are at the end.

This decomposition provides the generic approach to solving systems of linear equations, computing the rank, invertibility, inverse, kernel, and determinant. This LU decomposition is very stable and well tested with large matrices. However there are use cases where the SVD decomposition is inherently more stable and/or flexible. For example, when computing the kernel of a matrix, working with the SVD allows to select the smallest singular values of the matrix, something that the LU decomposition doesn't see.

6.7.3.1 Example for a real matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

>>> # This needs to be a square matrix
A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> Query = "isinjective, isinvertible, issurjective, rcond, det, LU, P, Q, X, Inverse"
>>> Res = A.eigen_fullPivLu2(Query, B)
>>> print("isinjective : ", Res["isinjective"])
isinjective :  True
>>> print("isinvertible: ", Res["isinvertible"])
isinvertible:  True
>>> print("issurjective: ", Res["issurjective"])
issurjective:  True
>>> print("rcond: ", Res["rcond"])
rcond:  0.0027248962455782943842188862118634888 4
>>> print("det  : ", Res["det"])
det  :  48772174.207999999999999999999999991

>>> Res["X"].show("X")
X:
 21.6,  21.7,  21.7,  21.7,  21.7,  21.8,
 8.21,  8.22,  8.22,  8.23,  8.23,  8.24,
-20.5, -20.6, -20.6, -20.6, -20.6, -20.6,
-15.3, -15.3, -15.4, -15.4, -15.4, -15.4,
 16.9,  16.9,  17.0,  17.0,  17.0,  17.0,
 8.73,  8.76,  8.78,  8.80,  8.82,  8.85,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> (B - A * Res["x"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
 8E-33, -6E-33, -5E-33, -2E-33, -8E-33,  9E-33,
 1E-33, -8E-33, -4E-33,  1E-33, -8E-33, -1E-33,
 6E-33,  4E-33,  3E-33,  3E-33, -2E-33,  4E-33,
 1E-33,  0E-33,  2E-33, -2E-33, -1E-33,  3E-33,
 3E-33, -1E-33, -1E-33, -2E-33, -3E-33,  1E-33,
 5E-33,  6E-33,  5E-33,  1E-33, -3E-33,  5E-33,

>>> P1 = Res["P"].eigen_inverse(); P1.show("P^-1")
P^-1:
0, 0, 0, 1, 0, 0,
```

(continues on next page)

(continued from previous page)

```
0, 0, 1, 0, 0, 0,
0, 0, 0, 0, 0, 1,
0, 0, 0, 0, 1, 0,
1, 0, 0, 0, 0, 0,
0, 1, 0, 0, 0, 0,

>>> L = Res["LU"].unit_lower_triangle(); L.show("L")
L:
      1,       0,       0,       0,       0,       0,
      1,       1,       0,       0,       0,       0,
    0.2,  0.0990,       1,       0,       0,       0,
   0.86,   0.409,   0.770,       1,       0,       0,
   0.84,  0.0533,  -0.274, -0.0710,       1,       0,
   0.78,   0.739,   0.404,  0.0750,  -0.295,       1,

>>> U = Res["LU"].upper_triangle(); U.show("U")
U:
    50,   0.44,     23,     42,     42,     23,
     0,   45.6,    -11,     -6,  -40.8,     24,
     0,      0,   42.5,   6.19,   15.6,   3.02,
     0,      0,      0,  -24.4, -0.487,  -7.92,
     0,      0,      0,      0,  -13.9,   2.67,
     0,      0,      0,      0,      0,   1.49,

>>> Q1 = Res["Q"].eigen_inverse(); Q1.show("Q^-1")
Q^-1:
0, 1, 0, 0, 0, 0,
0, 0, 0, 1, 0, 0,
1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 0,
0, 0, 1, 0, 0, 0,
0, 0, 0, 0, 0, 1,

>>> (P1 * L * U * Q1).show("A = P^-1 * L * U * Q^-1 (should be equal to A)")
A = P^-1 * L * U * Q^-1 (should be equal to A):
 48.0,  43.0,  31.0,  19.0,  14.0,  24.0,
 46.0,  10.0,  20.0,  4.60,  14.0,  10.0,
 27.0,  39.0,  13.0,  34.0,  29.0,  37.0,
 7.10,  42.0,  15.0,  2.80,  35.0,  23.0,
 23.0,  50.0,  42.0, 0.440,  42.0,  23.0,
 12.0,  50.0,  1.20,  46.0,  36.0,  47.0,

>>> Res["Inverse"].show("Inverse of A")
Inverse of A:
 0.0113,  0.0426, -0.0633, 0.00768, -0.0128,  0.0375,
 0.0459,  0.0319,  -0.201, -0.0115,  0.0136,   0.127,
-0.0121, -0.0523,   0.129, -0.0340,  0.0322, -0.0835,
0.00719,  0.0757,  -0.283,  -0.141,  0.0901,   0.228,
-0.0289,  0.0940,  -0.220, -0.0636,  0.0627,   0.169,
-0.0363,  -0.190,   0.672,   0.198,  -0.148,  -0.473,

>>> (A * Res["Inverse"]).show("A * Inverse (should be an identity matrix)")
A * Inverse (should be an identity matrix):
  1.00,  2E-35, -2E-34, -1E-35,  0E-35,  1E-34,
-1E-36,   1.00, -2E-35,  0E-35,  0E-35,  3E-35,
 0E-35,  1E-35,   1.00, -1E-35,  0E-35,  1E-34,
-3E-36,  2E-35,  0E-34,   1.00,  0E-35,  0E-34,
-3E-36,  0E-35,  0E-34,  0E-35,   1.00,  0E-34,
 1E-35,  2E-35,  0E-34,  0E-35,  1E-35,   1.00,
```

6.7.3.2 Example for a complex matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")

>>> # This needs to be an invertible matrix
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableA6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> Query = "isinjective, isinvertible, issurjective, rcond, det, LU, P, Q, X, Inverse"
>>> Res = A.eigen_fullPivLu2(Query, B)
>>> print("isinjective : ", Res["isinjective"])
isinjective :  True
>>> print("isinvertible: ", Res["isinvertible"])
isinvertible:  True
>>> print("issurjective: ", Res["issurjective"])
issurjective:  True
>>> print("rcond: ", Res["rcond"])
rcond:  0.018712448036456211456272670465202252 4 + 0j
>>> print("det  : ", Res["det"])
det  :  1083952.899999999999999999999999999 - 676519.300000000000000000000000000006j

>>> Res["X"].show("X")
X:
-0.869 - 3.37j, -0.858 - 3.37j, -0.846 - 3.38j, -0.835 - 3.38j,
 -1.82 - 3.22j,  -1.82 - 3.22j,  -1.82 - 3.23j,  -1.81 - 3.23j,
  2.15 + 3.53j,   2.15 + 3.53j,   2.16 + 3.53j,   2.16 + 3.53j,
  1.81 + 3.52j,   1.82 + 3.53j,   1.84 + 3.54j,   1.86 + 3.55j,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> (B - A * Res["x"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
-2.50E-33 + 1.00E-33j,        -3.00E-34 + 0j, -1.00E-33 + 1.00E-33j,  5.00E-34 + 2.00E-33j,
 6.00E-34 + 2.00E-33j, -1.00E-34 + 1.00E-33j,        -6.00E-34 + 0j,  3.00E-34 + 2.00E-33j,
       -1.00E-33 + 0j,         0 - 1.00E-33j, -1.00E-33 - 1.00E-33j,        -1.00E-33 + 0j,
       -1.10E-33 + 0j,        -1.00E-34 + 0j,  1.00E-34 - 1.00E-33j,  9.00E-34 + 2.00E-33j,

>>> P1 = Res["P"].eigen_inverse(); P1.show("P^-1")
P^-1:
   0 + 0j,    0 + 0j, 1.00 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j,    0 + 0j, 1.00 + 0j,
1.00 + 0j,    0 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j, 1.00 + 0j,    0 + 0j,    0 + 0j,

>>> L = Res["LU"].unit_lower_triangle(); L.show("L")
L:
      1.00 + 0j,          0 + 0j,          0 + 0j,          0 + 0j,
0.249 + 0.0907j,       1.00 + 0j,          0 + 0j,          0 + 0j,
```

(continues on next page)

(continued from previous page)

```
 0.453 + 0.329j,  0.306 - 0.685j,       1.00 + 0j,          0 + 0j,
0.865 - 0.0336j,  0.201 + 0.554j, -0.512 - 0.745j,       1.00 + 0j,

>>> U = Res["LU"].upper_triangle(); U.show("U")
U:
 42.0 + 49.0j,  32.0 + 9.00j,  32.0 + 49.0j,  47.0 + 11.0j,
       0 + 0j,  26.9 + 36.9j,  18.5 + 1.91j,  9.31 + 31.0j,
       0 + 0j,        0 + 0j,  31.7 + 16.4j, -30.7 - 10.6j,
       0 + 0j,        0 + 0j,        0 + 0j, -11.6 - 3.64j,

>>> Q1 = Res["Q"].eigen_inverse(); Q1.show("Q^-1")
Q^-1:
   0 + 0j, 1.00 + 0j,    0 + 0j,    0 + 0j,
1.00 + 0j,    0 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j,    0 + 0j, 1.00 + 0j,
   0 + 0j,    0 + 0j, 1.00 + 0j,    0 + 0j,

>>> (P1 * L * U * Q1).show("A = P^-1 * L * U * Q^-1 (should be equal to A)")
A = P^-1 * L * U * Q^-1 (should be equal to A):
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> Res["Inverse"].show("Inverse of A")
Inverse of A:
   0.0516 + 0.0468j,    0.0667 - 0.0273j, -0.0516 + 0.000462j,   -0.0635 - 0.0282j,
   0.0449 + 0.0423j,    0.0801 - 0.0400j, -0.0447 + 0.000795j,  -0.0814 - 0.00721j,
  -0.0583 - 0.0458j,   -0.0783 + 0.0245j,   0.0599 + 0.00584j,    0.0786 + 0.0125j,
  -0.0338 - 0.0465j,   -0.0674 + 0.0325j,    0.0409 - 0.0104j,    0.0738 + 0.0219j,

>>> (A * Res["Inverse"]).show("A * Inverse (should be an identity matrix)")
A * Inverse (should be an identity matrix):
     1.00 + 1.00E-35j,         0 + 2.00E-35j,         0 + 1.00E-35j,         1.00E-35 + 0j,
        0 + 3.00E-35j,      1.00 + 1.30E-35j,  1.00E-35 - 2.00E-35j,         0 - 2.00E-35j,
       -1.00E-35 + 0j, -2.00E-35 + 1.00E-35j,      1.00 - 2.00E-35j,  2.00E-35 - 1.00E-35j,
        1.30E-35 + 0j,         0 - 8.00E-36j,         0 - 3.00E-36j,      1.00 - 1.00E-35j,
```

6.7.4 Cholesky Decomposition without Pivoting

`ctx.mat_llt`(*matA*, *Query*, *matB*)

Returns the Cholesky decomposition of the symmetric matrix *matA* $= A = LL^T = U^TU$, without pivoting.

Writing `matA.LLT(Query, matB)` has the same effect

See also Eigen [231], Wikipedia [1400], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

B
Optional. A general n-by-m matrix of the same type as A. You need to specify B only if you want to solve the linear equation $AX = B$

Results:

det
A scalar of a return type matching A. The determinant of A.

rcond
A scalar of the same return type as det. The condition number of A.

X
A general matrix of the same type and dimension as B. The solution to $AX = B$.

Inv
A square matrix of the same type and dimension as A. The inverse of A, A^{-1}.

L
A square matrix of the same type and dimension as A, containing the matrix L in the decomposition $A = LL^T = U^TU$.

U
A square matrix of the same type and dimension as A, containing the matrix U in the decomposition $A = LL^T = U^TU$.

These functions perform a LL^T Cholesky decomposition of a symmetric, positive definite matrix A such that $A = LL^T = U^TU$, where L is lower triangular. While the Cholesky decomposition is particularly useful to solve selfadjoint problems like $D^TDx = b$, for that purpose, we recommend the Cholesky decomposition without square root which is more stable and even faster. Nevertheless, this standard Cholesky decomposition remains useful in many other situations like generalised eigen problems with hermitian matrices. Remember that Cholesky decompositions are not rank-revealing. This LL^T decomposition is only stable on positive definite matrices, use LDL^T instead for the semidefinite case. Also, do not use a Cholesky decomposition to determine whether a system of equations has a solution.

6.7.4.1 Example for a real symmetric matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAPosDefA6x6", ""); A.show("A")

>>> # This needs to be a self-adjoint, positive definite matrix
A:
248,  40,  38,  43,  33,  35,
 40, 240,  25,  38,  28,  35,
 38,  25, 245,  40,  33,  17,
 43,  38,  40, 250,  30,  33,
 33,  28,  33,  30, 240,  22,
 35,  35,  17,  33,  22, 243,
```

(continues on next page)

(continued from previous page)

```
>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> Query = "info, rcond, X, L, U, Inverse"
>>> Res = A.eigen_llt2(Query, B)
>>> print("Info : ", Res["Info"])
Info :  0
>>> print("Rcond: ", Res["Rcond"])
Rcond:  0.387103744149799746293987557343114309

>>> Res["X"].show("X")
X:
1.86, 1.86, 1.86, 1.87, 1.87, 1.87,
2.26, 2.26, 2.26, 2.26, 2.27, 2.27,
2.41, 2.41, 2.41, 2.42, 2.42, 2.42,
2.05, 2.05, 2.05, 2.06, 2.06, 2.06,
2.61, 2.61, 2.62, 2.62, 2.62, 2.62,
2.68, 2.68, 2.68, 2.69, 2.69, 2.69,

>>> (A * Res["X"]).show("A * X")
A * X:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> (B - A * Res["x"]).show("B - A * X")
B - A * X:
-3E-33, -3E-33,      0, -1E-33, -4E-33, -2E-33,
-3E-33, -5E-33, -1E-33, -2E-33, -5E-33, -2E-33,
     0,  1E-33,      0,      0,  2E-33,      0,
-2E-33, -3E-33, -2E-33, -1E-33, -3E-33, -3E-33,
 3E-33,  3E-33,  4E-33,  1E-33, -1E-33,  1E-33,
-1E-33, -2E-33, -3E-33, -1E-33, -4E-33, -2E-33,

>>> Res["L"].show("L")
L:
 15.7,     0,     0,     0,     0,     0,
 2.54,  15.3,     0,     0,     0,     0,
 2.41,  1.23,  15.4,     0,     0,     0,
 2.73,  2.03,  2.00,  15.3,     0,     0,
 2.10,  1.48,  1.69,  1.17,  15.1,     0,
 2.22,  1.92, 0.601,  1.43, 0.780,  15.2,

>>> Res["U"].show("U")
U:
 15.7,  2.54,  2.41,  2.73,  2.10,  2.22,
    0,  15.3,  1.23,  2.03,  1.48,  1.92,
    0,     0,  15.4,  2.00,  1.69, 0.601,
    0,     0,     0,  15.3,  1.17,  1.43,
    0,     0,     0,     0,  15.1, 0.780,
    0,     0,     0,     0,     0,  15.2,

>>> (A - Res["L"] * Res["U"]).show("A - L * U")
A - L * U:
 1E-33,      0,  1E-34,      0, -1E-34,  1E-34,
     0,      0,      0,      0, -1E-34,      0,
```

(continues on next page)

(continued from previous page)

```
 1E-34,      0,      0,      0, -1E-34,      0,
     0,      0,      0,      0,      0,  1E-34,
-1E-34, -1E-34, -1E-34,      0,      0,      0,
 1E-34,      0,      0,  1E-34,      0,  1E-33,

>>> Res["Inverse"].show("Inverse")
Inverse:
   0.00438,  -0.000496,  -0.000466,  -0.000502,  -0.000379,  -0.000424,
 -0.000496,    0.00445,  -0.000228,  -0.000455,  -0.000320,  -0.000463,
 -0.000466,  -0.000228,    0.00433,  -0.000513,  -0.000431, -0.0000939,
 -0.000502,  -0.000455,  -0.000513,    0.00433,  -0.000313,  -0.000385,
 -0.000379,  -0.000320,  -0.000431,  -0.000313,    0.00437,  -0.000223,
 -0.000424,  -0.000463, -0.0000939,  -0.000385,  -0.000223,    0.00432,

>>> (A * Res["Inverse"]).show("A * Inverse")
A * Inverse:
     1.00,    -7E-37,   6.3E-37,         0, -1.35E-36,    -1E-36,
   -3E-37,      1.00,  -4.7E-37,    -6E-37,  -3.5E-37,    -1E-36,
  9.7E-37,   1.6E-37,      1.00,   3.6E-37,  -3.7E-37,         0,
    8E-37,  -1.3E-36,  -7.2E-37,      1.00,  -1.3E-37,     1E-36,
 1.25E-36,     3E-37,  -3.5E-37,   8.2E-37,      1.00,     2E-37,
    1E-36,    -2E-36,         0,   1.1E-36,    -3E-37,      1.00,
```

6.7.4.2 Example for a hermitian matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableRandomSAPosDefA6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")

>>> # This needs to be a self-adjoint, positive definite matrix
A:
   91.0 + 0j, 12.0 - 3.60j, 22.0 - 7.40j, 14.0 + 1.10j,
12.0 + 3.60j,    77.0 + 0j, 2.50 - 2.00j, 3.40 - 7.60j,
22.0 + 7.40j, 2.50 + 2.00j,    91.0 + 0j, 17.0 + 3.70j,
14.0 - 1.10j, 3.40 + 7.60j, 17.0 - 3.70j,    74.0 + 0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> Query = "info, rcond, X, L, U, Inverse"
>>> Res = A.eigen_llt2(Query, B)
>>> print("Info : ", Res["Info"])
Info :  0 + 0j
>>> print("Rcond: ", Res["Rcond"])
Rcond:  0.41520387595592569380609948826249957 1 + 0j

>>> Res["X"].show("X")
X:
    0.376 - 0.0288j,     -0.222 + 0.281j,   -0.0865 + 0.0417j,      0.229 + 0.303j,
    0.0290 + 0.383j,      0.511 + 0.498j,      0.227 + 0.583j,      0.309 + 0.210j,
    0.219 - 0.0602j,      0.535 + 0.463j,    0.527 - 0.00173j,      0.288 + 0.417j,
     0.379 + 0.583j, 0.000510 + 0.00479j,      0.214 + 0.481j,     0.170 + 0.0528j,

>>> (A * Res["X"]).show("A * X")
A * X:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
```

(continues on next page)

(continued from previous page)

```
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> (B - A * Res["x"]).show("B - A * X")
B - A * X:
-1.00E-34 + 4.00E-35j, -7.00E-35 - 2.00E-34j,                0 + 0j, -2.00E-34 - 1.00E-34j,
        0 + 1.00E-34j,         1.00E-34 + 0j,  1.00E-34 + 1.00E-34j,                0 + 0j,
-3.00E-34 + 3.00E-35j, -3.00E-34 - 3.00E-34j,        -3.00E-34 + 0j, -2.00E-34 - 2.00E-34j,
 2.00E-34 + 4.00E-34j, -3.00E-35 - 1.00E-34j,  1.00E-34 + 1.00E-34j,  1.00E-34 + 1.00E-34j,

>>> Res["L"].show("L")
L:
       9.54 + 0j,           0 + 0j,           0 + 0j,           0 + 0j,
   1.26 + 0.377j,        8.68 + 0j,           0 + 0j,           0 + 0j,
   2.31 + 0.776j, -0.0800 + 0.218j,        9.22 + 0j,           0 + 0j,
   1.47 - 0.115j,   0.184 + 0.956j,    1.47 - 0.236j,        8.29 + 0j,

>>> Res["U"].show("U")
U:
       9.54 + 0j,    1.26 - 0.377j,    2.31 - 0.776j,    1.47 + 0.115j,
          0 + 0j,        8.68 + 0j, -0.0800 - 0.218j,   0.184 - 0.956j,
          0 + 0j,           0 + 0j,        9.22 + 0j,    1.47 + 0.236j,
          0 + 0j,           0 + 0j,           0 + 0j,        8.29 + 0j,

>>> (A - Res["L"] * Res["U"]).show("A - L * U")
A - L * U:
        0 + 0j,         0 + 0j,         0 + 0j,         0 + 0j,
        0 + 0j, -2.00E-34 + 0j,         0 + 0j,         0 + 0j,
        0 + 0j,         0 + 0j,         0 + 0j,  1.00E-34 + 0j,
        0 + 0j,         0 + 0j,  1.00E-34 + 0j, -1.00E-34 + 0j,

>>> Res["Inverse"].show("Inverse")
Inverse:
   0.0122 + 1.36E-39j,  -0.00177 + 0.000794j,  -0.00259 + 0.000957j,  -0.00167 - 0.000490j,
 -0.00177 - 0.000794j,    0.0135 + 1.96E-40j, 0.000120 + 0.0000425j,  -0.000321 + 0.00154j,
 -0.00259 - 0.000957j, 0.000120 - 0.0000425j,    0.0121 + 2.17E-40j,  -0.00231 - 0.000373j,
 -0.00167 + 0.000490j,  -0.000321 - 0.00154j,  -0.00231 + 0.000373j,           0.0146 + 0j,

>>> (A * Res["Inverse"]).show("A * Inverse")
A * Inverse:
     1.00 + 8.00E-38j, -8.40E-37 + 2.00E-37j, -8.00E-37 + 2.30E-37j, -1.00E-36 - 1.00E-37j,
 1.56E-36 + 4.00E-37j,       1.00 - 4.00E-38j,  1.60E-37 - 1.00E-37j,  1.00E-37 + 1.00E-36j,
 1.20E-36 + 4.00E-38j, -4.00E-38 - 3.00E-37j,      1.00 + 6.00E-38j, -1.00E-36 - 3.00E-37j,
 1.00E-36 - 4.00E-37j,         0 + 1.00E-36j,         1.00E-36 + 0j,      1.00 - 2.00E-38j,
```

6.7.5 QR Decomposition without Pivoting

`ctx.mat_householder_qr`(*matA*, *Query*, *matB*)

Returns the QR decomposition of the symmetric matrix *matA* $= A = QR$, without pivoting.

Writing `mat.HouseholderQR(Query, matB)` has the same effect

See also Eigen [228], Wikipedia [1434], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

B
Optional. A general n-by-m matrix of the same type as A. You need to specify B only if you want to solve the linear equation $AX = B$

Results:

signdet
A scalar of a return type matching A. The sign of the determinant of A.

absdet
A scalar of a return type matching A. The absolute value of the determinant of A.

logabsdet
A scalar of a return type matching A. The logarithm of the absolute value of the determinant of A.

X
A general matrix of the same type and dimension as B. The solution to $AX = B$.

Inv
A square matrix of the same type and dimension as A. The inverse of A, A^{-1}.

QR
A square matrix of the same type and dimension as A, containing the matrices Q and R in the decomposition $A = QR$.

This class performs a QR decomposition of a matrix A into matrices Q and R such that $A = QR$ by using Householder transformations. Here, Q a unitary matrix and R an upper triangular matrix. The result is stored in a compact way compatible with LAPACK. Note that no pivoting is performed.

This is not a rankrevealing decomposition. If you want that feature, use FullPivHouseholderQR or ColPivHouseholderQR instead.

This Householder QR decomposition is faster, but less numerically stable and less feature-rich than FullPivHouseholderQR or ColPivHouseholderQR

6.7.5.1 Example for a real general square matrix (Python)

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

>>> # This needs to be a square matrix
A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
```

(continues on next page)

(continued from previous page)

```
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> Query = "absdet, logabsdet, householderq, QR, X, Inverse"
>>> Res = A.eigen_householderQr2(Query, B)
>>> print("absdet     : ", Res["absdet"])
absdet     :  48772174.207999999999999999999999994
>>> print("logabsdet  : ", Res["logabsdet"])
logabsdet  :  17.702670507541421335076l102045531515

>>> Res["X"].show("X")
X:
 21.6,  21.7,  21.7,  21.7,  21.7,  21.8,
 8.21,  8.22,  8.22,  8.23,  8.23,  8.24,
-20.5, -20.6, -20.6, -20.6, -20.6, -20.6,
-15.3, -15.3, -15.4, -15.4, -15.4, -15.4,
 16.9,  16.9,  17.0,  17.0,  17.0,  17.0,
 8.73,  8.76,  8.78,  8.80,  8.82,  8.85,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> (B - A * Res["x"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
  -5E-33,    8E-33,   -4E-33,    2E-33, -1.3E-32,    2E-33,
-1.0E-32,   -7E-33,   -5E-33,    2E-33, -1.3E-32,    0E-33,
   4E-33,  1.0E-32,    5E-33,  1.0E-32,    5E-33,    5E-33,
   5E-33,    8E-33,    4E-33,    9E-33,    5E-33,    3E-33,
  -2E-33,    4E-33,   -2E-33,    3E-33,   -3E-33,   -2E-33,
   4E-33,    7E-33,    3E-33,    8E-33,    4E-33,    3E-33,

>>> R = Res["QR"].upper_triangle(); R.show("R")
R:
-76.6, -73.4, -50.2, -34.2, -48.9, -50.5,
    0,  69.6,  16.5,  30.4,  52.5,  47.9,
    0,     0,  27.0, -36.2,  3.91, -19.6,
    0,     0,     0,  15.9, -2.42,  6.43,
    0,     0,     0,     0, -18.7, -6.45,
    0,     0,     0,     0,     0, -1.14,

>>> Q = Res["Householderq"]; Q.show("Q")
Q:
 -0.626, -0.0428, 0.00995, -0.0493,   0.776,  0.0413,
 -0.600,  -0.489, -0.0766,  -0.243,  -0.538,   0.215,
 -0.352,   0.189,  -0.288,   0.362,  -0.207,  -0.764,
-0.0927,   0.506,  0.0749,  -0.819, -0.0880,  -0.225,
 -0.300,   0.402,   0.752,   0.328,  -0.218,   0.169,
 -0.157,   0.553,  -0.583,   0.169,  -0.106,   0.538,

>>> (Q.T * Q).show("Q.T * Q (should be an identity matrix)")
Q.T * Q (should be an identity matrix):
    1.00,   -8E-37,    1E-37,   -3E-37,   -8E-37, -1.4E-36,
  -8E-37,     1.00,   -1E-36,   -9E-37, -7.7E-36,    0E-36,
   1E-37,   -1E-36,     1.00, -2.1E-36,    0E-37,    2E-36,
  -3E-37,   -9E-37, -2.1E-36,     1.00, -2.0E-36,   -6E-37,
```

(continues on next page)

(continued from previous page)

```
  -8E-37, -7.7E-36,    0E-37, -2.0E-36,     1.00,    1E-37,
-1.4E-36,    0E-36,    2E-36,   -6E-37,    1E-37,     1.00,

>>> (Q * R).show("Q * R (should be equal to A)")
Q * R (should be equal to A):
 48.0,  43.0,  31.0,  19.0,  14.0,  24.0,
 46.0,  10.0,  20.0,  4.60,  14.0,  10.0,
 27.0,  39.0,  13.0,  34.0,  29.0,  37.0,
 7.10,  42.0,  15.0,  2.80,  35.0,  23.0,
 23.0,  50.0,  42.0, 0.440,  42.0,  23.0,
 12.0,  50.0,  1.20,  46.0,  36.0,  47.0,

>>> Res["Inverse"].show("Inverse of A")
Inverse of A:
 0.0113,  0.0426, -0.0633, 0.00768, -0.0128,  0.0375,
 0.0459,  0.0319,  -0.201, -0.0115,  0.0136,   0.127,
-0.0121, -0.0523,   0.129, -0.0340,  0.0322, -0.0835,
0.00719,  0.0757,  -0.283,  -0.141,  0.0901,   0.228,
-0.0289,  0.0940,  -0.220, -0.0636,  0.0627,   0.169,
-0.0363,  -0.190,   0.672,   0.198,  -0.148,  -0.473,

>>> (A * Res["Inverse"]).show("A * Inverse (should be an identity matrix)")
A * Inverse (should be an identity matrix):
    1.00,   -5E-35,    0E-34,    4E-35,   -1E-35,    0E-34,
 4.1E-35,     1.00, -2.4E-34,   -2E-35,    3E-35,  1.3E-34,
  -1E-35,   -2E-35,     1.00,   -1E-35,    1E-35,    0E-34,
  -7E-36,   -1E-35,    1E-34,     1.00,   -1E-35,    0E-34,
   3E-36,   -2E-35,    0E-34,    4E-35,     1.00,    0E-34,
   0E-35,   -1E-35,    0E-34,    1E-35,    0E-35,     1.00,
```

6.7.5.2 Example for a complex matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")

>>> # This needs to be an invertible matrix
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableA6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> Query = "absdet, logabsdet, householderq, QR, X, Inverse"
>>> Res = A.eigen_householderQr2(Query, B)
>>> print("absdet     : ", Res["absdet"])
absdet     :  1277744.9873472014525224864573725287 9 + 0j
>>> print("logabsdet  : ", Res["logabsdet"])
logabsdet  :  14.060607353591879183379444362428081 2 + 0j

>>> Res["X"].show("X")
X:
-0.869 - 3.37j, -0.858 - 3.37j, -0.846 - 3.38j, -0.835 - 3.38j,
 -1.82 - 3.22j,  -1.82 - 3.22j,  -1.82 - 3.23j,  -1.81 - 3.23j,
  2.15 + 3.53j,   2.15 + 3.53j,   2.16 + 3.53j,   2.16 + 3.53j,
```

(continues on next page)

(continued from previous page)

```
  1.81 + 3.52j,   1.82 + 3.53j,   1.84 + 3.54j,   1.86 + 3.55j,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> (B - A * Res["x"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
-2.00E-34 - 1.00E-33j,  1.00E-34 - 1.00E-33j,        -1.10E-33 + 0j,  2.00E-34 + 1.00E-33j,
       -2.00E-34 + 0j, -2.80E-33 - 1.00E-33j,        -1.80E-33 + 0j,         0 - 2.00E-33j,
        1.00E-33 + 0j,         0 + 1.00E-33j,        -1.00E-33 + 0j,         1.00E-33 + 0j,
-1.00E-34 - 1.00E-33j, -1.50E-33 + 1.00E-33j,         1.30E-33 + 0j, -1.00E-34 + 1.00E-33j,

>>> R = Res["QR"].upper_triangle(); R.show("R")
R:
    -84.3 + 0j,  -56.2 - 29.8j,  -71.5 - 9.69j,  -68.9 - 21.0j,
        0 + 0j,     -69.5 + 0j, -29.8 - 0.903j,  -42.7 + 7.20j,
        0 + 0j,         0 + 0j,     -28.1 + 0j,  31.5 - 0.339j,
        0 + 0j,         0 + 0j,         0 + 0j,     -7.76 + 0j,

>>> Q = Res["Householderq"]; Q.show("Q")
Q:
-0.533 - 0.0889j,   0.352 - 0.217j,   0.557 + 0.167j,   0.262 - 0.361j,
 -0.154 - 0.344j,  -0.569 - 0.246j,  0.0855 - 0.360j,   0.523 + 0.252j,
 -0.379 - 0.107j,  -0.343 - 0.456j,  -0.396 + 0.505j, -0.318 - 0.0806j,
 -0.403 - 0.498j,  0.0266 + 0.345j,   0.125 - 0.313j,  -0.572 + 0.170j,

>>> (Q.T * Q.conjugate()).show("Q.T * Q.conjugate() (should be an identity matrix)")
Q.T * Q.conjugate() (should be an identity matrix):
          1.00 + 0j, 1.00E-36 - 1.00E-36j,        0 + 1.00E-36j, 1.00E-36 + 2.00E-36j,
1.00E-36 + 1.00E-36j,            1.00 + 0j, 3.00E-36 - 1.10E-36j,        1.60E-36 + 0j,
       0 - 1.00E-36j, 3.00E-36 + 1.10E-36j,            1.00 + 0j, 1.00E-36 - 1.00E-36j,
1.00E-36 - 2.00E-36j,        1.60E-36 + 0j, 1.00E-36 + 1.00E-36j,            1.00 + 0j,

>>> (Q * R).show("Q * R (should be equal to A)")
Q * R (should be equal to A):
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> Res["Inverse"].show("Inverse of A")
Inverse of A:
   0.0516 + 0.0468j,    0.0667 - 0.0273j, -0.0516 + 0.000462j,   -0.0635 - 0.0282j,
   0.0449 + 0.0423j,    0.0801 - 0.0400j, -0.0447 + 0.000795j,  -0.0814 - 0.00721j,
  -0.0583 - 0.0458j,   -0.0783 + 0.0245j,   0.0599 + 0.00584j,    0.0786 + 0.0125j,
  -0.0338 - 0.0465j,   -0.0674 + 0.0325j,    0.0409 - 0.0104j,    0.0738 + 0.0219j,

>>> (A * Res["Inverse"]).show("A * Inverse (should be an identity matrix)")
A * Inverse (should be an identity matrix):
     1.00 - 1.00E-35j,  1.00E-35 + 2.00E-35j,  1.00E-35 + 1.00E-35j,         0 - 1.00E-35j,
-1.10E-35 + 1.00E-35j,      1.00 + 3.10E-35j,  2.00E-35 - 2.50E-35j,  3.00E-35 - 1.00E-35j,
        0 + 1.00E-35j,  2.00E-35 + 1.00E-35j,             1.00 + 0j,        -2.00E-35 + 0j,
 1.20E-35 + 2.00E-35j,  3.00E-35 + 9.00E-36j, -1.00E-35 - 4.00E-36j,             1.00 + 0j,
```

6.7.6 QR Decomposition with column Pivoting

`ctx.mat_col_piv_householder_qr`(*matA*, *Query*, *matB*, *threshold*)

Returns the QR decomposition of the symmetric matrix *matA* $= A = QR$, with column pivoting.

Writing `matA.ColPivHouseholderQR(Query, matB, threshold)` has the same effect

See also: Eigen [219], Wikipedia [1436], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

matB
Optional. A general n-by-m matrix of the same type as A. You need to specify B only if you want to solve the linear equation $AX = B$

threshold
Optional. The threshold that will be used by certain methods such as rank().

Results:

info
An integer indicating whether the decompsition was successful(=0) or not(=1).

dimofkernel
An integer returning the dimension of the kernel of A.

rank
An integer returning the rank of A, which need to determine when pivots are to be considered nonzero. This is not used for the QR decomposition itself.

nonzeropivots
An integer returning the number of non-zero ivots in the decomposition.

isinjective
A boolean indicating whether A is injective.

isinvertible
A boolean indicating whether A is invertible.

issurjective
A boolean indicating whether A is surjective.

signdet
A scalar of a return type matching A. The sign of the determinant of A.

absdet
A scalar of a return type matching A. The absolute value of the determinant of A.

logabsdet
A scalar of a return type matching A. The logarithm of the absolute value of the determinant of A.

maxpivot
A scalar of the same return type as det. The absolute value of the biggest pivot, i.e. the biggest diagonal coefficient of R.

X
A general matrix of the same type and dimension as B. The solution to $AX = B$.

Inv
A square matrix of the same type and dimension as A. The inverse of A, A^{-1}.

P
A square matrix of the same type and dimension as A. The matrix P contains the column permutation matrix.

Q
A square matrix of the same type and dimension as A. The matrix Q as a sequence of householder transformations.

R
A square matrix of the same type and dimension as A, containing the matrix R.

QR
A square matrix of the same type and dimension as A, containing the matrix QR.

This class performs a rank-revealing QR decomposition of a matrix A into matrices P, Q and R such that

AP = QR

by using Householder transformations. Here, P is a permutation matrix, Q a unitary matrix and R an upper triangular matrix.

This decomposition performs column pivoting in order to be rank-revealing and improve numerical stability. It is slower than HouseholderQR, and faster than FullPivHouseholderQR.

solve: This method finds a solution x to the equation Ax=b, where A is the matrix of which this is the QR decomposition, if any exists.

Parameters: b the right-hand-side of the equation to solve.

Returns a solution. Note: The case where b is a matrix is not yet implemented. Also, this code is space inefficient.This method just tries to find as good a solution as possible. If you want to check whether a solution exists or if it is accurate, just call this function to get a result and then compute the error of this result, or use

MatrixBase::isApprox() directly, for instance like this:

bool a_solution_exists = (A*result).isApprox(b, precision);

This method avoids dividing by zero, so that the non-existence of a solution doesn't by itself mean that you'll get inf or nan values. If there exists more than one solution, this method will arbitrarily choose one.

6.7.6.1 Example for a real matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

>>> # This needs to be a square matrix
A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> Query = "isinjective, isinvertible, issurjective, info, dimofkernel, rank, nonzeropivots, "
>>> Query += " absdet, logabsdet, maxpivot, QR, R, Householderq, Hqnonzeros, Permcols, X, Inverse"

>>> Res = A.eigen_colPivHouseholderQr2(Query, B)
```

(continues on next page)

(continued from previous page)

```
>>> print("isinjective  : ", Res["isinjective"])
isinjective  :  True
>>> print("isinvertible : ", Res["isinvertible"])
isinvertible :  True
>>> print("issurjective : ", Res["issurjective"])
issurjective :  True

>>> print("info         : ", Res["info"])
info         :  0
>>> print("dimofkernel  : ", Res["dimofkernel"])
dimofkernel  :  0
>>> print("rank         : ", Res["rank"])
rank         :  6
>>> print("nonzeropivots: ", Res["nonzeropivots"])
nonzeropivots:  6

>>> print("absdet     : ", Res["absdet"])
absdet     :  48772174.208000000000000000000000014
>>> print("logabsdet  : ", Res["logabsdet"])
logabsdet  :  17.702670507541421335076l102045531516
>>> print("maxpivot   : ", Res["maxpivot"])
maxpivot   :  101.16323442832381088439063261595990 8

>>> Res["X"].show("X")
X:
 21.6,  21.7,  21.7,  21.7,  21.7,  21.8,
 8.21,  8.22,  8.22,  8.23,  8.23,  8.24,
-20.5, -20.6, -20.6, -20.6, -20.6, -20.6,
-15.3, -15.3, -15.4, -15.4, -15.4, -15.4,
 16.9,  16.9,  17.0,  17.0,  17.0,  17.0,
 8.73,  8.76,  8.78,  8.80,  8.82,  8.85,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> (B - A * Res["X"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
  -3E-33, -1.6E-32,   -5E-33, -1.7E-32,    4E-33,   -9E-33,
   7E-33,    6E-33,   -3E-33,   -5E-33,    7E-33,    9E-33,
   3E-33,   -4E-33,   -2E-33,   -8E-33,    2E-33,   -5E-33,
  -4E-33,   -7E-33,   -4E-33,   -8E-33,    2E-33, -1.3E-32,
  -8E-33, -1.2E-32, -1.1E-32, -1.8E-32,   -2E-33, -1.2E-32,
  -7E-33, -1.0E-32,   -6E-33, -1.0E-32,   -4E-33, -1.3E-32,

>>> P2 = Res["Permcols"].eigen_inverse(); P2.show("P2^-1")
P2^-1:
0, 1, 0, 0, 0, 0,
1, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0, 0,
0, 0, 0, 0, 1, 0,
0, 0, 1, 0, 0, 0,
0, 0, 0, 0, 0, 1,

>>> R = Res["QR"].upper_triangle(); R.show("R")
R:
  -101,  -55.6,  -45.8,  -71.6,  -47.7,  -69.6,
     0,  -52.7,  -1.51,   4.47,  -22.6, 0.0539,
     0,      0,  -39.6,   4.55,   24.8,  -20.5,
     0,      0,      0,  -18.7,  0.371,  -6.51,
     0,      0,      0,      0,  -10.8,   1.75,
```

(continues on next page)

(continued from previous page)

```
     0,      0,      0,      0,      0,  -1.14,

>>> Q = Res["Householderq"]; Q.show("Q")
Q:
 -0.425,  -0.462,  0.0289,   0.774,  0.0677,  0.0413,
-0.0989,  -0.768,  0.0273,  -0.546,   0.234,   0.215,
 -0.386,  -0.106,  -0.409,  -0.199,  -0.223,  -0.764,
 -0.415,   0.303,   0.398,  -0.113,   0.717,  -0.225,
 -0.494,  0.0850,   0.557,  -0.197,  -0.609,   0.169,
 -0.494,   0.294,  -0.602,  -0.109,  0.0756,   0.538,

>>> Res["Hqnonzeros"].show("Hqnonzeros")
Hqnonzeros:
 -0.425,  -0.462,  0.0289,   0.774,  0.0677,  0.0413,
-0.0989,  -0.768,  0.0273,  -0.546,   0.234,   0.215,
 -0.386,  -0.106,  -0.409,  -0.199,  -0.223,  -0.764,
 -0.415,   0.303,   0.398,  -0.113,   0.717,  -0.225,
 -0.494,  0.0850,   0.557,  -0.197,  -0.609,   0.169,
 -0.494,   0.294,  -0.602,  -0.109,  0.0756,   0.538,

>>> (Q.T * Q).show("Q.T * Q (should be an identity matrix)")
Q.T * Q (should be an identity matrix):
     1.00,    -2E-36,    -1E-36,   3.3E-36,   1.6E-36,     1E-36,
   -2E-36,      1.00,    -2E-36,     3E-37,   2.0E-36,     2E-36,
   -1E-36,    -2E-36,      1.00,  -1.1E-36,    -2E-37,     0E-36,
  3.3E-36,     3E-37,  -1.1E-36,      1.00, -1.10E-36,    -6E-37,
  1.6E-36,   2.0E-36,    -2E-37, -1.10E-36,      1.00,    -9E-37,
    1E-36,     2E-36,     0E-36,    -6E-37,    -9E-37,      1.00,

>>> (Q * R * P2).show("Q * R * P2 (should be equal to A)")
Q * R * P2 (should be equal to A):
 48.0,  43.0,  31.0,  19.0,  14.0,  24.0,
 46.0,  10.0,  20.0,  4.60,  14.0,  10.0,
 27.0,  39.0,  13.0,  34.0,  29.0,  37.0,
 7.10,  42.0,  15.0,  2.80,  35.0,  23.0,
 23.0,  50.0,  42.0, 0.440,  42.0,  23.0,
 12.0,  50.0,  1.20,  46.0,  36.0,  47.0,

>>> Res["Inverse"].show("Inverse of A")
Inverse of A:
 0.0113,  0.0426, -0.0633, 0.00768, -0.0128,  0.0375,
 0.0459,  0.0319,  -0.201, -0.0115,  0.0136,   0.127,
-0.0121, -0.0523,   0.129, -0.0340,  0.0322, -0.0835,
0.00719,  0.0757,  -0.283,  -0.141,  0.0901,   0.228,
-0.0289,  0.0940,  -0.220, -0.0636,  0.0627,   0.169,
-0.0363,  -0.190,   0.672,   0.198,  -0.148,  -0.473,

>>> (A * Res["Inverse"]).show("A * Inverse (should be an identity matrix)")
A * Inverse (should be an identity matrix):
   1.00,  -2E-35,   1E-34,  -1E-35,  -3E-35,  -1E-34,
  0E-36,    1.00,   0E-35,  -2E-35,   0E-35,  -1E-35,
  0E-35,   0E-35,    1.00,  -1E-35,  -1E-35,   0E-34,
 -7E-36,   0E-35,   0E-34,    1.00,  -1E-35,   0E-34,
1.3E-35,  -1E-35,   1E-34,  -3E-35,    1.00,   0E-34,
  0E-35,   1E-35,   0E-34,  -2E-35,  -2E-35,    1.00,
```

6.7.6.2 Example for a complex matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")

>>> # This needs to be an invertible matrix, `A^{-1}`
```

(continues on next page)

(continued from previous page)

```
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableA6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> Query = "isinjective, isinvertible, issurjective, info, dimofkernel, rank, nonzeropivots, "
>>> Query += " absdet, logabsdet, maxpivot, QR, R, Householderq, Hqnonzeros, Permcols, X, Inverse"

>>> Res = A.eigen_colPivHouseholderQr2(Query, B)

>>> print("isinjective  : ", Res["isinjective"])
isinjective  :  True
>>> print("isinvertible : ", Res["isinvertible"])
isinvertible :  True
>>> print("issurjective : ", Res["issurjective"])
issurjective :  True

>>> print("info         : ", Res["info"])
info         :  0
>>> print("dimofkernel  : ", Res["dimofkernel"])
dimofkernel  :  0
>>> print("rank         : ", Res["rank"])
rank         :  4
>>> print("nonzeropivots: ", Res["nonzeropivots"])
nonzeropivots:  4

>>> print("absdet     : ", Res["absdet"])
absdet     :  1277744.9873472014525224864573725287 4
>>> print("logabsdet  : ", Res["logabsdet"])
logabsdet  :  14.060607353591879183379444362428081 2
>>> print("maxpivot   : ", Res["maxpivot"])
maxpivot   :  94.267756947961798843331053375853945 0

>>> Res["X"].show("X")
X:
-0.869 - 3.37j, -0.858 - 3.37j, -0.846 - 3.38j, -0.835 - 3.38j,
 -1.82 - 3.22j,  -1.82 - 3.22j,  -1.82 - 3.23j,  -1.81 - 3.23j,
  2.15 + 3.53j,   2.15 + 3.53j,   2.16 + 3.53j,   2.16 + 3.53j,
  1.81 + 3.52j,   1.82 + 3.53j,   1.84 + 3.54j,   1.86 + 3.55j,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> (B - A * Res["X"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
 9.00E-34 + 1.00E-33j,  8.00E-34 - 1.00E-33j,         2.00E-34 + 0j,  1.10E-33 - 1.00E-33j,
 6.00E-34 - 1.00E-33j,  2.00E-34 - 1.00E-33j, -5.00E-34 - 3.00E-33j,  4.00E-34 - 1.00E-33j,
 1.00E-33 - 4.00E-33j,  1.00E-33 - 3.00E-33j,  1.00E-33 - 2.00E-33j,  1.00E-33 - 3.00E-33j,
 3.00E-34 + 1.00E-33j,         5.00E-34 + 0j,        -6.00E-34 + 0j,        -5.00E-34 + 0j,

>>> P2 = Res["Permcols"].eigen_inverse(); P2.show("P2^-1")
P2^-1:
   0 + 0j, 1.00 + 0j,    0 + 0j,    0 + 0j,
```

(continues on next page)

(continued from previous page)

```
1.00 + 0j,    0 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j,    0 + 0j, 1.00 + 0j,
   0 + 0j,    0 + 0j, 1.00 + 0j,    0 + 0j,

>>> R = Res["QR"].upper_triangle(); R.show("R")
R:
    -94.3 + 0j,  -50.3 + 26.6j,  -79.3 + 14.6j,  -67.7 + 16.1j,
        0 + 0j,      62.2 + 0j,   23.1 + 6.28j,   35.3 - 2.79j,
        0 + 0j,         0 + 0j,      32.4 + 0j, -27.3 - 0.294j,
        0 + 0j,         0 + 0j,         0 + 0j,     -6.71 + 0j,

>>> Q = Res["Householderq"]; Q.show("Q")
Q:
 -0.0308 - 0.382j,    0.535 - 0.175j,    0.480 + 0.242j,    0.392 - 0.307j,
  -0.403 - 0.435j,   -0.303 + 0.287j,  -0.0459 - 0.410j,    0.526 + 0.164j,
  -0.445 - 0.520j, -0.0686 - 0.0850j,   -0.303 + 0.514j,  -0.402 + 0.0392j,
 -0.0636 - 0.170j,    0.422 + 0.565j,    0.255 - 0.345j,  -0.527 + 0.0840j,

>>> Res["Hqnonzeros"].show("Hqnonzeros")
Hqnonzeros:
 -0.0308 - 0.382j,    0.535 - 0.175j,    0.480 + 0.242j,    0.392 - 0.307j,
  -0.403 - 0.435j,   -0.303 + 0.287j,  -0.0459 - 0.410j,    0.526 + 0.164j,
  -0.445 - 0.520j, -0.0686 - 0.0850j,   -0.303 + 0.514j,  -0.402 + 0.0392j,
 -0.0636 - 0.170j,    0.422 + 0.565j,    0.255 - 0.345j,  -0.527 + 0.0840j,

>>> (Q.T * Q.conjugate()).show("Q.T * Q (should be an identity matrix)")
Q.T * Q (should be an identity matrix):
            1.00 + 0j,         0 + 1.40E-36j,        -1.60E-36 + 0j,  1.10E-36 + 7.00E-37j,
        0 - 1.40E-36j,             1.00 + 0j,  1.40E-36 - 2.00E-36j, -4.00E-36 + 3.00E-36j,
       -1.60E-36 + 0j,  1.40E-36 + 2.00E-36j,             1.00 + 0j, -3.00E-36 - 1.00E-36j,
 1.10E-36 - 7.00E-37j, -4.00E-36 - 3.00E-36j, -3.00E-36 + 1.00E-36j,             1.00 + 0j,

>>> (Q * R * P2).show("Q * R * P2 (should be equal to A)")
Q * R * P2 (should be equal to A):
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> Res["Inverse"].show("Inverse of A")
Inverse of A:
   0.0516 + 0.0468j,    0.0667 - 0.0273j, -0.0516 + 0.000462j,   -0.0635 - 0.0282j,
   0.0449 + 0.0423j,    0.0801 - 0.0400j, -0.0447 + 0.000795j,  -0.0814 - 0.00721j,
  -0.0583 - 0.0458j,   -0.0783 + 0.0245j,   0.0599 + 0.00584j,    0.0786 + 0.0125j,
  -0.0338 - 0.0465j,   -0.0674 + 0.0325j,    0.0409 - 0.0104j,    0.0738 + 0.0219j,

>>> (A * Res["Inverse"]).show("A * Inverse (should be an identity matrix)")
A * Inverse (should be an identity matrix):
     1.00 - 2.00E-35j, -2.00E-35 - 2.00E-35j,  1.00E-35 + 2.00E-35j,         1.00E-35 + 0j,
 1.00E-36 - 1.00E-35j,      1.00 + 2.00E-36j,  2.00E-35 + 1.30E-35j, -1.00E-35 + 1.00E-35j,
        0 - 5.00E-35j, -4.00E-35 - 1.00E-35j,      1.00 + 2.00E-35j,         0 + 2.00E-35j,
-8.00E-36 - 1.00E-35j, -1.00E-35 + 8.00E-36j,         0 - 1.80E-35j,      1.00 - 1.00E-35j,
```

6.7.7 QR Decomposition with full Pivoting

`ctx.mat_full_piv_householder_qr`(*matA*, *Query*, *matB*, *threshold*)

Returns the QR decomposition (with full pivoting) of the general matrix *matA* such that`PAP'`` = QR` by using Householder transformations. Here, P and P' are permutation matrices, Q is a unitary matrix and R is an upper triangular matrix.

Writing `matA.FullPivHouseholderQR(Query, matB, threshold)` has the same effect

See also Eigen [223], Wikipedia [1436], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

B
Optional. A general n-by-m matrix of the same type as A. You need to specify B only if you want to solve the linear equation $AX = B$

threshold
Optional. The threshold that will be used by certain methods such as rank().

Results:

info
An integer indicating whether the decompsition was successful(=0) or not(=1).

dimofkernel
An integer returning the dimension of the kernel of A.

rank
An integer returning the rank of A, which need to determine when pivots are to be considered nonzero. This is not used for the QR decomposition itself.

nonzeropivots
An integer returning the number of non-zero ivots in the decomposition.

isinjective
A boolean indicating whether A is injective.

isinvertible
A boolean indicating whether A is invertible.

issurjective
A boolean indicating whether A is surjective.

signdet
A scalar of a return type matching A. The sign of the determinant of A.

absdet
A scalar of a return type matching A. The absolute value of the determinant of A.

logabsdet
A scalar of a return type matching A. The logarithm of the absolute value of the determinant of A.

maxpivot
A scalar of the same return type as det. The absolute value of the biggest pivot, i.e. the biggest diagonal coefficient of R.

X
A general matrix of the same type and dimension as B. The solution to $AX = B$.

Inv
A square matrix of the same type and dimension as A. The inverse of A, A^{-1}.

P
A square matrix of the same type and dimension as A. The matrix P contains the column permutation matrix.

P'
A square matrix of the same type and dimension as A. The matrix P' contains the rows transpositions matrix.

Q
A square matrix of the same type and dimension as A. The matrix Q as a sequence of householder transformations.

R
A square matrix of the same type and dimension as A, containing the matrix R.

QR
A square matrix of the same type and dimension as A, containing the matrix QR.

This class performs a rank-revealing QR decomposition of a matrix A into matrices P, P', Q and R such that`PAP' = QR` by using Householder transformations. Here, P and P' are permutation matrices, Q a unitary matrix and R an upper triangular matrix.

This decomposition performs full pivoting in order to be rank-revealing and achieve optimal numerical stability. The trade-off is that it is slower than HouseholderQR and ColPivHouseholderQR.

solve: This method finds a solution x to the equation Ax=b, where A is the matrix of which this is the QR decomposition, if any exists.

Parameters: b the right-hand-side of the equation to solve.

Returns the exact or least-square solution if the rank is greater or equal to the number of columns of A, and an arbitrary solution otherwise. Note: The case where b is a matrix is not yet implemented. Also, this code is space inefficient.This method just tries to find as good a solution as possible. If you want to check whether a solution exists or if it is accurate, just call this function to get a result and then compute the error of this result, or use MatrixBase::isApprox() directly, for instance like this:

bool a_solution_exists = (A*result).isApprox(b, precision);

This method avoids dividing by zero, so that the non-existence of a solution doesn't by itself mean that you'll get inf or nan values. If there exists more than one solution, this method will arbitrarily choose one.

6.7.7.1 Example for a real matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

>>> # This needs to be a square matrix
A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> Query = "isinjective, isinvertible, issurjective, dimofkernel, rank, nonzeropivots, "
>>> Query += "absdet, logabsdet, maxpivot, QR, Q, Permcols, X, Inverse"

>>> Res = A.eigen_fullPivHouseholderQr2(Query, B)
```

(continues on next page)

(continued from previous page)

```
>>> print("isinjective  : ", Res["isinjective"])
isinjective  :  True
>>> print("isinvertible : ", Res["isinvertible"])
isinvertible :  True
>>> print("issurjective : ", Res["issurjective"])
issurjective :  True

>>> print("dimofkernel  : ", Res["dimofkernel"])
dimofkernel  :  0
>>> print("rank         : ", Res["rank"])
rank         :  6
>>> print("nonzeropivots: ", Res["nonzeropivots"])
nonzeropivots:  6

>>> print("absdet     : ", Res["absdet"])
absdet     :  48772174.207999999999999999999999990
>>> print("logabsdet  : ", Res["logabsdet"])
logabsdet  :  17.702670507541421335076110204553151 5
>>> print("maxpivot   : ", Res["maxpivot"])
maxpivot   :  101.16323442832381088439063261595990 8

>>> Res["X"].show("X")
X:
 21.6,  21.7,  21.7,  21.7,  21.7,  21.8,
 8.21,  8.22,  8.22,  8.23,  8.23,  8.24,
-20.5, -20.6, -20.6, -20.6, -20.6, -20.6,
-15.3, -15.3, -15.4, -15.4, -15.4, -15.4,
 16.9,  16.9,  17.0,  17.0,  17.0,  17.0,
 8.73,  8.76,  8.78,  8.80,  8.82,  8.85,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> (B - A * Res["X"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
  -6E-33,    2E-33,    5E-33,   -1E-33, -1.3E-32,   -4E-33,
  -6E-33,   -1E-33,   -1E-33,   -5E-33, -1.7E-32,   -1E-33,
   9E-33,    4E-33,  1.0E-32,    9E-33,    1E-33,  1.0E-32,
   3E-33,   -1E-33,    1E-33,   -4E-33,   -5E-33,    3E-33,
   5E-33,   -2E-33,    6E-33,   -3E-33,   -8E-33,    7E-33,
 1.3E-32,    6E-33,  1.5E-32,  1.0E-32,    5E-33,  1.6E-32,

>>> P2 = Res["Permcols"].eigen_inverse(); P2.show("P2^-1")
P2^-1:
0, 1, 0, 0, 0, 0,
1, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0, 0,
0, 0, 0, 0, 1, 0,
0, 0, 1, 0, 0, 0,
0, 0, 0, 0, 0, 1,

>>> R = Res["QR"].upper_triangle(); R.show("R")
R:
  -101,  -55.6,  -45.8,  -71.6,  -47.7,  -69.6,
     0,  -52.7,  -1.51,   4.47,  -22.6, 0.0539,
     0,      0,  -39.6,   4.55,   24.8,  -20.5,
     0,      0,      0,   18.7, -0.371,   6.51,
     0,      0,      0,      0,   10.8,  -1.75,
     0,      0,      0,      0,      0,   1.14,
```

(continues on next page)

(continued from previous page)

```
>>> Q = Res["Q"]; Q.show("Q")
Q:
 -0.425,  -0.462,  0.0289,   0.774,  0.0677,  0.0413,
-0.0989,  -0.768,  0.0273,  -0.546,   0.234,   0.215,
 -0.386,  -0.106,  -0.409,  -0.199,  -0.223,  -0.764,
 -0.415,   0.303,   0.398,  -0.113,   0.717,  -0.225,
 -0.494,  0.0850,   0.557,  -0.197,  -0.609,   0.169,
 -0.494,   0.294,  -0.602,  -0.109,  0.0756,   0.538,

>>> (Q.T * Q).show("Q.T * Q (should be an identity matrix)")
Q.T * Q (should be an identity matrix):
     1.00,     0E-36,    -2E-36,  -1.8E-36,  -1.5E-36,     0E-36,
    0E-36,      1.00,     2E-36,  -1.9E-36,   2.5E-36,    -1E-36,
   -2E-36,     2E-36,      1.00,     7E-37,   3.1E-36,     0E-36,
 -1.8E-36,  -1.9E-36,     7E-37,      1.00, -2.09E-36,     0E-37,
 -1.5E-36,   2.5E-36,   3.1E-36, -2.09E-36,      1.00,  -2.2E-36,
    0E-36,    -1E-36,     0E-36,     0E-37,  -2.2E-36,      1.00,

>>> (Q * R * P2).show("Q * R * P2 (should be equal to A, except possible permutation of rows)")
Q * R * P2 (should be equal to A):
 48.0,  43.0,  31.0,  19.0,  14.0,  24.0,
 46.0,  10.0,  20.0,  4.60,  14.0,  10.0,
 27.0,  39.0,  13.0,  34.0,  29.0,  37.0,
 7.10,  42.0,  15.0,  2.80,  35.0,  23.0,
 23.0,  50.0,  42.0, 0.440,  42.0,  23.0,
 12.0,  50.0,  1.20,  46.0,  36.0,  47.0,

>>> Res["Inverse"].show("Inverse of A")
Inverse of A:
 0.0113,  0.0426, -0.0633, 0.00768, -0.0128,  0.0375,
 0.0459,  0.0319,  -0.201, -0.0115,  0.0136,   0.127,
-0.0121, -0.0523,   0.129, -0.0340,  0.0322, -0.0835,
0.00719,  0.0757,  -0.283,  -0.141,  0.0901,   0.228,
-0.0289,  0.0940,  -0.220, -0.0636,  0.0627,   0.169,
-0.0363,  -0.190,   0.672,   0.198,  -0.148,  -0.473,

>>> (A * Res["Inverse"]).show("A * Inverse (should be an identity matrix)")
A * Inverse (should be an identity matrix):
   1.00,   2E-35,   1E-34,   0E-35,   1E-35,   0E-34,
  9E-36,    1.00,  -3E-35,   0E-35,  -1E-35,   1E-35,
  1E-35,   3E-35,    1.00,  -3E-35,   3E-35,   0E-34,
  3E-36,   2E-35,   1E-34,    1.00,  -1E-35,   0E-34,
1.3E-35,   4E-35,   0E-34,   1E-35,    1.00,   1E-34,
  2E-35,   4E-35,  -2E-34,  -3E-35,   4E-35,    1.00,
```

6.7.7.2 Example for a complex matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")

>>> # This needs to be an invertible matrix, `A^{-1}`
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableA6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
```

(continues on next page)

(continued from previous page)

```
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> Query = "isinjective, isinvertible, issurjective, dimofkernel, rank, nonzeropivots, "
>>> Query += "absdet, logabsdet, maxpivot, QR, Q, Permcols, X, Inverse"

>>> Res = A.eigen_fullPivHouseholderQr2(Query, B)

>>> print("isinjective  : ", Res["isinjective"])
isinjective  :  True
>>> print("isinvertible : ", Res["isinvertible"])
isinvertible :  True
>>> print("issurjective : ", Res["issurjective"])
issurjective :  True

>>> print("dimofkernel  : ", Res["dimofkernel"])
dimofkernel  :  0
>>> print("rank         : ", Res["rank"])
rank         :  4
>>> print("nonzeropivots: ", Res["nonzeropivots"])
nonzeropivots:  4

>>> print("absdet     : ", Res["absdet"])
absdet     :  1277744.9873472014525224864573725287 2
>>> print("logabsdet  : ", Res["logabsdet"])
logabsdet  :  14.060607353591879183379444362428081 1
>>> print("maxpivot   : ", Res["maxpivot"])
maxpivot   :  101.16323442832381088439063261595990 8

>>> Res["X"].show("X")
X:
-0.869 - 3.37j, -0.858 - 3.37j, -0.846 - 3.38j, -0.835 - 3.38j,
 -1.82 - 3.22j,  -1.82 - 3.22j,  -1.82 - 3.23j,  -1.81 - 3.23j,
  2.15 + 3.53j,   2.15 + 3.53j,   2.16 + 3.53j,   2.16 + 3.53j,
  1.81 + 3.52j,   1.82 + 3.53j,   1.84 + 3.54j,   1.86 + 3.55j,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> (B - A * Res["X"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
 9.00E-34 - 1.00E-33j,  1.50E-33 - 1.00E-33j,  2.00E-34 + 2.00E-33j,  9.00E-34 + 1.00E-33j,
-4.00E-34 - 3.00E-33j,  6.00E-34 - 2.00E-33j,  4.00E-34 - 3.00E-33j,                0 + 0j,
 1.00E-33 - 4.00E-33j,         0 - 5.00E-33j,  1.00E-33 - 4.00E-33j,  2.00E-33 - 4.00E-33j,
        6.00E-34 + 0j,         3.00E-34 + 0j, -1.00E-34 + 1.00E-33j,        -6.00E-34 + 0j,

>>> P2 = Res["Permcols"].eigen_inverse(); P2.show("P2^-1")
P2^-1:
   0 + 0j, 1.00 + 0j,    0 + 0j,    0 + 0j,
1.00 + 0j,    0 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j,    0 + 0j, 1.00 + 0j,
   0 + 0j,    0 + 0j, 1.00 + 0j,    0 + 0j,

>>> R = Res["QR"].upper_triangle(); R.show("R")
R:
   -94.3 + 0j, -50.3 + 26.6j, -79.3 + 14.6j, -67.7 + 16.1j,
       0 + 0j,    -62.2 + 0j, -23.1 - 6.28j, -35.3 + 2.79j,
       0 + 0j,        0 + 0j,    -32.4 + 0j, 27.3 + 0.294j,
       0 + 0j,        0 + 0j,        0 + 0j,     6.71 + 0j,

>>> Q = Res["Q"]; Q.show("Q")
Q:
```

(continues on next page)

(continued from previous page)

```
-0.0308 - 0.382j,  -0.535 + 0.175j,  -0.480 - 0.242j,  -0.392 + 0.307j,
 -0.403 - 0.435j,   0.303 - 0.287j,  0.0459 + 0.410j,  -0.526 - 0.164j,
 -0.445 - 0.520j, 0.0686 + 0.0850j,   0.303 - 0.514j,  0.402 - 0.0392j,
-0.0636 - 0.170j,  -0.422 - 0.565j,  -0.255 + 0.345j,  0.527 - 0.0840j,

>>> (Q.T * Q.conjugate()).show("Q.T * Q (should be an identity matrix)")
Q.T * Q (should be an identity matrix):
           1.00 + 0j,  1.00E-36 - 9.00E-37j,  1.80E-36 + 4.00E-37j,  6.00E-37 - 2.00E-37j,
 1.00E-36 + 9.00E-37j,             1.00 + 0j, -1.50E-36 + 1.00E-36j,                0 + 0j,
 1.80E-36 - 4.00E-37j, -1.50E-36 - 1.00E-36j,             1.00 + 0j, -1.00E-36 - 2.00E-36j,
 6.00E-37 + 2.00E-37j,                0 + 0j, -1.00E-36 + 2.00E-36j,             1.00 + 0j,

>>> (Q * R * P2).show("Q * R * P2 (should be equal to A, except possible permutation of rows)")
Q * R * P2 (should be equal to A, except possible permutation of rows):
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> Res["Inverse"].show("Inverse of A")
Inverse of A:
   0.0516 + 0.0468j,    0.0667 - 0.0273j, -0.0516 + 0.000462j,   -0.0635 - 0.0282j,
   0.0449 + 0.0423j,    0.0801 - 0.0400j, -0.0447 + 0.000795j,  -0.0814 - 0.00721j,
  -0.0583 - 0.0458j,   -0.0783 + 0.0245j,   0.0599 + 0.00584j,    0.0786 + 0.0125j,
  -0.0338 - 0.0465j,   -0.0674 + 0.0325j,    0.0409 - 0.0104j,    0.0738 + 0.0219j,

>>> (A * Res["Inverse"]).show("A * Inverse (should be an identity matrix)")
A * Inverse (should be an identity matrix):
     1.00 + 1.00E-35j,         0 - 3.00E-35j, -1.00E-35 + 2.00E-35j, -2.00E-35 + 4.00E-35j,
 1.10E-35 - 2.00E-35j,      1.00 - 3.20E-35j,  3.00E-35 + 1.00E-35j,  4.00E-35 + 5.00E-35j,
 1.00E-35 - 7.00E-35j, -1.10E-34 - 5.00E-35j,      1.00 + 3.00E-35j,  5.00E-35 + 9.00E-35j,
 7.00E-36 - 1.00E-35j, -1.00E-35 - 1.80E-35j,  1.00E-35 - 9.00E-36j,      1.00 + 1.00E-35j,
```

6.7.8 Complete orthogonal decomposition (COD)

`ctx.mat_cod_householder_qr`(*matA*, *results*, *matB*)

Returns a rank-revealing complete orthogonal decomposition (COD) of the general matrix *matA* $= A$ into matrices P, Q, T, and Z such that

Writing `matA.CODHouseholderQR(results, matB)` has the same effect

$$\mathbf{A\,P} = \mathbf{Q} \begin{bmatrix} \mathbf{T} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{Z}$$

by using Householder transformations. Here, P is a permutation matrix, Q and Z are unitary matrices and T an upper triangular matrix of size rank-by-rank. A may be rank deficient.

See also Eigen [220], Wikipedia [1434], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

B
Optional. A general n-by-m matrix of the same type as A. You need to specify B only if you want to solve the linear equation $AX = B$

Results:

info
An integer indicating whether the decompsition was successful(=0) or not(=1).

dimofkernel
An integer returning the dimension of the kernel of A.

rank
An integer returning the rank of A, which need to determine when pivots are to be considered nonzero. This is not used for the QR decomposition itself.

nonzeropivots
An integer returning the number of non-zero ivots in the decomposition.

isinjective
A boolean indicating whether A is injective.

isinvertible
A boolean indicating whether A is invertible.

issurjective
A boolean indicating whether A is surjective.

signdet
A scalar of a return type matching A. The sign of the determinant of A.

absdet
A scalar of a return type matching A. The absolute value of the determinant of A.

logabsdet
A scalar of a return type matching A. The logarithm of the absolute value of the determinant of A.

maxpivot
A scalar of the same return type as det. The absolute value of the biggest pivot, i.e. the biggest diagonal coefficient of R.

X
A general matrix of the same type and dimension as B. The solution to $AX = B$.

pseudoInverse
A square matrix of the same type and dimension as A. The pseudo-inverse of A, A^{-1}.

P
A square matrix of the same type and dimension as A. The matrix P contains the column permutation matrix.

P'
A square matrix of the same type and dimension as A. The matrix P' contains the rows transpositions matrix.

T
A square matrix of the same type and dimension as A., containing the matrix Z.

Z
A square matrix of the same type and dimension as A, containing the matrix R.

QTZ
A square matrix of the same type and dimension as A, containing the matrix QTZ.

This class performs a a rank-revealing complete orthogonal decomposition (COD) of the general matrix *matA* $= A$ into matrices P, Q, T, and Z such that

$$\mathbf{A\,P = Q} \begin{bmatrix} \mathbf{T} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{Z}$$

by using Householder transformations. Here, P is a permutation matrix, Q and Z are unitary matrices and T an upper triangular matrix of size rank-by-rank. A may be rank deficient.

solve: This method finds a solution x to the equation Ax=b, where A is the matrix of which this is the QR decomposition, if any exists.

Parameters: b the right-hand-side of the equation to solve.

Returns the exact or least-square solution if the rank is greater or equal to the number of columns of A, and an arbitrary solution otherwise. Note: The case where b is a matrix is not yet implemented. Also, this code is space inefficient.This method just tries to find as good a solution as possible. If you want to check whether a solution exists or if it is accurate, just call this function to get a result and then compute the error of this result, or use MatrixBase::isApprox() directly, for instance like this:

bool a_solution_exists = (A*result).isApprox(b, precision);

This method avoids dividing by zero, so that the non-existence of a solution doesn't by itself mean that you'll get inf or nan values. If there exists more than one solution, this method will arbitrarily choose one.

6.7.8.1 Example for a real matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

>>> # This needs to be a square matrix
A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,
```

(continues on next page)

(continued from previous page)

```
>>> Query = "isinjective, isinvertible, issurjective, info, dimofkernel, rank, nonzeropivots, absdet, "
>>> Query += "logabsdet, maxpivot, QTZ, T, Z, Householderq, Hqnonzeros, Permcols, X, PseudoInverse"

>>> Res = A.eigen_COD2(Query, B)

>>> print("isinjective  : ", Res["isinjective"])
isinjective  :  True
>>> print("isinvertible : ", Res["isinvertible"])
isinvertible :  True
>>> print("issurjective : ", Res["issurjective"])
issurjective :  True

>>> print("info         : ", Res["info"])
dimofkernel  :  0
>>> print("dimofkernel  : ", Res["dimofkernel"])
dimofkernel  :  0
>>> print("rank         : ", Res["rank"])
rank         :  6
>>> print("nonzeropivots: ", Res["nonzeropivots"])
nonzeropivots:  6

>>> print("absdet     : ", Res["absdet"])
absdet     :  48772174.208000000000000000000000014
>>> print("logabsdet  : ", Res["logabsdet"])
logabsdet  :  17.702670507541421335076110204553151 6
>>> print("maxpivot   : ", Res["maxpivot"])
maxpivot   :  101.16323442832381088439063261595990 8

>>> Res["X"].show("X")
X:
 21.6,  21.7,  21.7,  21.7,  21.7,  21.8,
 8.21,  8.22,  8.22,  8.23,  8.23,  8.24,
-20.5, -20.6, -20.6, -20.6, -20.6, -20.6,
-15.3, -15.3, -15.4, -15.4, -15.4, -15.4,
 16.9,  16.9,  17.0,  17.0,  17.0,  17.0,
 8.73,  8.76,  8.78,  8.80,  8.82,  8.85,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> (B - A * Res["X"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
  -3E-33, -1.6E-32,   -5E-33, -1.7E-32,    4E-33,   -9E-33,
   7E-33,    6E-33,   -3E-33,   -5E-33,    7E-33,    9E-33,
   3E-33,   -4E-33,   -2E-33,   -8E-33,    2E-33,   -5E-33,
  -4E-33,   -7E-33,   -4E-33,   -8E-33,    2E-33, -1.3E-32,
  -8E-33, -1.2E-32, -1.1E-32, -1.8E-32,   -2E-33, -1.2E-32,
  -7E-33, -1.0E-32,   -6E-33, -1.0E-32,   -4E-33, -1.3E-32,

>>> P2 = Res["Permcols"].eigen_inverse(); P2.show("P2^-1")
P2^-1:
0, 1, 0, 0, 0, 0,
1, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0, 0,
0, 0, 0, 0, 1, 0,
0, 0, 1, 0, 0, 0,
0, 0, 0, 0, 0, 1,

>>> Res["QTZ"].show("QTZ")
```

(continues on next page)

(continued from previous page)

```
QTZ:
   -101,   -55.6,   -45.8,   -71.6,   -47.7,   -69.6,
 0.0694,   -52.7,   -1.51,    4.47,   -22.6,  0.0539,
  0.271, -0.0112,   -39.6,    4.55,    24.8,   -20.5,
  0.291,  -0.252,  -0.279,   -18.7,   0.371,   -6.51,
  0.347,  -0.141,  -0.389,   0.443,   -10.8,    1.75,
  0.347,  -0.261,   0.427,   0.222, -0.0306,   -1.14,

>>> T = Res["T"].upper_triangle(); T.show("T")
T:
  -101,  -55.6,  -45.8,  -71.6,  -47.7,  -69.6,
     0,  -52.7,  -1.51,   4.47,  -22.6, 0.0539,
     0,      0,  -39.6,   4.55,   24.8,  -20.5,
     0,      0,      0,  -18.7,  0.371,  -6.51,
     0,      0,      0,      0,  -10.8,   1.75,
     0,      0,      0,      0,      0,  -1.14,

>>> Z = Res["Z"]; Z.show("Z")
Z:
1, 0, 0, 0, 0, 0,
0, 1, 0, 0, 0, 0,
0, 0, 1, 0, 0, 0,
0, 0, 0, 1, 0, 0,
0, 0, 0, 0, 1, 0,
0, 0, 0, 0, 0, 1,

>>> Q = Res["Householderq"]; Q.show("Q")
Q:
 -0.425,  -0.462,  0.0289,   0.774,  0.0677,  0.0413,
-0.0989,  -0.768,  0.0273,  -0.546,   0.234,   0.215,
 -0.386,  -0.106,  -0.409,  -0.199,  -0.223,  -0.764,
 -0.415,   0.303,   0.398,  -0.113,   0.717,  -0.225,
 -0.494,  0.0850,   0.557,  -0.197,  -0.609,   0.169,
 -0.494,   0.294,  -0.602,  -0.109,  0.0756,   0.538,

>>> Res["Hqnonzeros"].show("Hqnonzeros")
Hqnonzeros:
 -0.425,  -0.462,  0.0289,   0.774,  0.0677,  0.0413,
-0.0989,  -0.768,  0.0273,  -0.546,   0.234,   0.215,
 -0.386,  -0.106,  -0.409,  -0.199,  -0.223,  -0.764,
 -0.415,   0.303,   0.398,  -0.113,   0.717,  -0.225,
 -0.494,  0.0850,   0.557,  -0.197,  -0.609,   0.169,
 -0.494,   0.294,  -0.602,  -0.109,  0.0756,   0.538,

>>> (Q.T * Q).show("Q.T * Q (should be an identity matrix)")
Q.T * Q (should be an identity matrix):
     1.00,    -2E-36,    -1E-36,   3.3E-36,   1.6E-36,     1E-36,
   -2E-36,      1.00,    -2E-36,     3E-37,   2.0E-36,     2E-36,
   -1E-36,    -2E-36,      1.00,  -1.1E-36,    -2E-37,     0E-36,
  3.3E-36,     3E-37,  -1.1E-36,      1.00, -1.10E-36,    -6E-37,
  1.6E-36,   2.0E-36,    -2E-37, -1.10E-36,      1.00,    -9E-37,
    1E-36,     2E-36,     0E-36,    -6E-37,    -9E-37,      1.00,

>>> (Q * T * Z* P2).show("Q * T * Z * P2 (should be equal to A)")
Q * T * Z * P2 (should be equal to A):
 48.0,  43.0,  31.0,  19.0,  14.0,  24.0,
 46.0,  10.0,  20.0,  4.60,  14.0,  10.0,
 27.0,  39.0,  13.0,  34.0,  29.0,  37.0,
 7.10,  42.0,  15.0,  2.80,  35.0,  23.0,
 23.0,  50.0,  42.0, 0.440,  42.0,  23.0,
 12.0,  50.0,  1.20,  46.0,  36.0,  47.0,

>>> Res["PseudoInverse"].show("PseudoInverse")
PseudoInverse:
 0.0113,  0.0426, -0.0633, 0.00768, -0.0128,  0.0375,
 0.0459,  0.0319,  -0.201, -0.0115,  0.0136,   0.127,
```

(continues on next page)

(continued from previous page)

```
-0.0121, -0.0523,   0.129, -0.0340,  0.0322, -0.0835,
0.00719,  0.0757,  -0.283,  -0.141,  0.0901,   0.228,
-0.0289,  0.0940,  -0.220, -0.0636,  0.0627,   0.169,
-0.0363,  -0.190,   0.672,   0.198,  -0.148,  -0.473,

>>> (A * Res["PseudoInverse"]).show("A * PseudoInverse")
A * PseudoInverse:
   1.00,  -2E-35,   1E-34,  -1E-35,  -3E-35,  -1E-34,
  0E-36,    1.00,   0E-35,  -2E-35,   0E-35,  -1E-35,
  0E-35,   0E-35,    1.00,  -1E-35,  -1E-35,   0E-34,
 -7E-36,   0E-35,   0E-34,    1.00,  -1E-35,   0E-34,
1.3E-35,  -1E-35,   1E-34,  -3E-35,    1.00,   0E-34,
  0E-35,   1E-35,   0E-34,  -2E-35,  -2E-35,    1.00,
```

6.7.8.2 Example for a complex matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")

>>> # This needs to be an invertible matrix, `A^{-1}`
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableA6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> Query = "isinjective, isinvertible, issurjective, info, dimofkernel, rank, nonzeropivots, absdet, "
>>> Query += "logabsdet, maxpivot, QTZ, T, Z, Householderq, Hqnonzeros, Permcols, X, PseudoInverse"

>>> Res = A.eigen_COD2(Query, B)

>>> print("isinjective  : ", Res["isinjective"])
isinjective  :  True
>>> print("isinvertible : ", Res["isinvertible"])
isinvertible :  True
>>> print("issurjective : ", Res["issurjective"])
issurjective :  True

>>> print("info         : ", Res["info"])
dimofkernel  :  0
>>> print("dimofkernel  : ", Res["dimofkernel"])
dimofkernel  :  0
>>> print("rank         : ", Res["rank"])
rank         :  4
>>> print("nonzeropivots: ", Res["nonzeropivots"])
nonzeropivots:  4

>>> print("absdet     : ", Res["absdet"])
absdet     :  1277744.9873472014525224864573725287 4
>>> print("logabsdet  : ", Res["logabsdet"])
logabsdet  :  14.060607353591879183379444362428081 2
>>> print("maxpivot   : ", Res["maxpivot"])
maxpivot   :  94.267756947961798843331053375853945 0
```

(continues on next page)

(continued from previous page)

```
>>> Res["X"].show("X")
X:
-0.869 - 3.37j, -0.858 - 3.37j, -0.846 - 3.38j, -0.835 - 3.38j,
 -1.82 - 3.22j,  -1.82 - 3.22j,  -1.82 - 3.23j,  -1.81 - 3.23j,
  2.15 + 3.53j,   2.15 + 3.53j,   2.16 + 3.53j,   2.16 + 3.53j,
  1.81 + 3.52j,   1.82 + 3.53j,   1.84 + 3.54j,   1.86 + 3.55j,

>>> (A * Res["X"]).show("A * X (should be equal to B)")
A * X (should be equal to B):
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> (B - A * Res["X"]).show("B - A * X (should be a zero matrix)")
B - A * X (should be a zero matrix):
 9.00E-34 + 1.00E-33j,  8.00E-34 - 1.00E-33j,         2.00E-34 + 0j,  1.10E-33 - 1.00E-33j,
 6.00E-34 - 1.00E-33j,  2.00E-34 - 1.00E-33j, -5.00E-34 - 3.00E-33j,  4.00E-34 - 1.00E-33j,
 1.00E-33 - 4.00E-33j,  1.00E-33 - 3.00E-33j,  1.00E-33 - 2.00E-33j,  1.00E-33 - 3.00E-33j,
 3.00E-34 + 1.00E-33j,         5.00E-34 + 0j,        -6.00E-34 + 0j,        -5.00E-34 + 0j,

>>> P2 = Res["Permcols"].eigen_inverse(); P2.show("P2^-1")
P2^-1:
   0 + 0j, 1.00 + 0j,    0 + 0j,    0 + 0j,
1.00 + 0j,    0 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j,    0 + 0j, 1.00 + 0j,
   0 + 0j,    0 + 0j, 1.00 + 0j,    0 + 0j,

>>> Res["QTZ"].show("QTZ")
QTZ:
     -94.3 + 0j,   -50.3 + 26.6j,   -79.3 + 14.6j,   -67.7 + 16.1j,
 0.481 + 0.244j,       62.2 + 0j,    23.1 + 6.28j,    35.3 - 2.79j,
 0.544 + 0.303j,  0.238 + 0.130j,       32.4 + 0j,  -27.3 - 0.294j,
 0.108 + 0.125j, -0.162 - 0.347j, -0.360 + 0.302j,      -6.71 + 0j,

>>> T = Res["T"].upper_triangle(); T.show("T")
T:
    -94.3 + 0j,  -50.3 + 26.6j,  -79.3 + 14.6j,  -67.7 + 16.1j,
        0 + 0j,      62.2 + 0j,   23.1 + 6.28j,   35.3 - 2.79j,
        0 + 0j,         0 + 0j,      32.4 + 0j, -27.3 - 0.294j,
        0 + 0j,         0 + 0j,         0 + 0j,     -6.71 + 0j,

>>> Z = Res["Z"]; Z.show("Z")
Z:
1.00 + 0j,    0 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j, 1.00 + 0j,    0 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j, 1.00 + 0j,    0 + 0j,
   0 + 0j,    0 + 0j,    0 + 0j, 1.00 + 0j,

>>> Q = Res["Householderq"]; Q.show("Q")
Q:
 -0.0308 - 0.382j,    0.535 - 0.175j,    0.480 + 0.242j,    0.392 - 0.307j,
  -0.403 - 0.435j,   -0.303 + 0.287j,  -0.0459 - 0.410j,    0.526 + 0.164j,
  -0.445 - 0.520j, -0.0686 - 0.0850j,   -0.303 + 0.514j,  -0.402 + 0.0392j,
 -0.0636 - 0.170j,    0.422 + 0.565j,    0.255 - 0.345j,  -0.527 + 0.0840j,

>>> Res["Hqnonzeros"].show("Hqnonzeros")
Hqnonzeros:
 -0.0308 - 0.382j,    0.535 - 0.175j,    0.480 + 0.242j,    0.392 - 0.307j,
  -0.403 - 0.435j,   -0.303 + 0.287j,  -0.0459 - 0.410j,    0.526 + 0.164j,
  -0.445 - 0.520j, -0.0686 - 0.0850j,   -0.303 + 0.514j,  -0.402 + 0.0392j,
 -0.0636 - 0.170j,    0.422 + 0.565j,    0.255 - 0.345j,  -0.527 + 0.0840j,

>>> (Q.T * Q.conjugate()).show("Q.T * Q.conjugate() (should be an identity matrix)")
Q.T * Q.conjugate() (should be an identity matrix):
            1.00 + 0j,         0 + 1.40E-36j,        -1.60E-36 + 0j,  1.10E-36 + 7.00E-37j,
```

(continues on next page)

(continued from previous page)

```
        0 - 1.40E-36j,             1.00 + 0j,  1.40E-36 - 2.00E-36j, -4.00E-36 + 3.00E-36j,
       -1.60E-36 + 0j,  1.40E-36 + 2.00E-36j,             1.00 + 0j, -3.00E-36 - 1.00E-36j,
 1.10E-36 - 7.00E-37j, -4.00E-36 - 3.00E-36j, -3.00E-36 + 1.00E-36j,             1.00 + 0j,

>>> (Q * T * Z* P2).show("Q * T * Z * P2 (should be equal to A)")
Q * T * Z * P2 (should be equal to A):
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> Res["PseudoInverse"].show("PseudoInverse")
PseudoInverse:
   0.0516 + 0.0468j,    0.0667 - 0.0273j, -0.0516 + 0.000462j,   -0.0635 - 0.0282j,
   0.0449 + 0.0423j,    0.0801 - 0.0400j, -0.0447 + 0.000795j,  -0.0814 - 0.00721j,
  -0.0583 - 0.0458j,   -0.0783 + 0.0245j,   0.0599 + 0.00584j,    0.0786 + 0.0125j,
  -0.0338 - 0.0465j,   -0.0674 + 0.0325j,    0.0409 - 0.0104j,    0.0738 + 0.0219j,

>>> (A * Res["PseudoInverse"]).show("A * PseudoInverse")
A * PseudoInverse:
     1.00 - 2.00E-35j, -2.00E-35 - 2.00E-35j,  1.00E-35 + 2.00E-35j,         1.00E-35 + 0j,
 1.00E-36 - 1.00E-35j,      1.00 + 2.00E-36j,  2.00E-35 + 1.30E-35j, -1.00E-35 + 1.00E-35j,
        0 - 5.00E-35j, -4.00E-35 - 1.00E-35j,      1.00 + 2.00E-35j,         0 + 2.00E-35j,
-8.00E-36 - 1.00E-35j, -1.00E-35 + 8.00E-36j,         0 - 1.80E-35j,      1.00 - 1.00E-35j,
```

6.8 Singular Value and Eigen (selfadjoint) decompositions

6.8.1 Singular Value Decomposition, only singular values

`ctx.mat_jacobi_svd`(*matA*, *Query*)

Returns the two-sided Jacobi SVD decomposition of a rectangular matrix.

Writing `matA.JacobiSVD(Query)` has the same effect

See also: Wikipedia [1418].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

absdet
A scalar of a return type matching A. The absolute value of the determinant of A.

rank
An integer. The rank of A (equal to the number of non-zero singular values)

SV
A vector of the same type`A`. The singular values A (see below).

The singular value decomposition of an $m \times n$ complex matrix $\mathbf{M}$ is a factorization of the form $\mathbf{M} = \mathbf{U\Sigma V}^*$, where $\mathbf{U}$ is an $m \times m$ complex unitary matrix, $\mathbf{\Sigma}$ is an $m \times n$ rectangular diagonal matrix with non-negative real numbers on the diagonal, and $\mathbf{V}$ is an $n \times n$ complex unitary matrix. If $\mathbf{M}$ is real, $\mathbf{U}$ and $\mathbf{V}^{\mathsf{T}} = \mathbf{V}^*$ are real orthogonal matrices.

The diagonal entries $\sigma_i = \Sigma_{ii}$ of $\mathbf{\Sigma}$ are known as the singular values of $\mathbf{M}$. The number of non-zero singular values is equal to the rank of $\mathbf{M}$. The columns of $\mathbf{U}$ and the columns of $\mathbf{V}$ are called the left-singular vectors and right-singular vectors of $\mathbf{M}$, respectively. These vectors are related by

$$\mathbf{Mv} = \sigma\mathbf{u} \text{ and } \mathbf{M}^*\mathbf{u} = \sigma\mathbf{v}.$$

The singular values are the square roots of non-negative eigenvalues of $\mathbf{M}^*\mathbf{M}$.

Matrix transpose and conjugate do not alter singular values.

$$\sigma_i(A) = \sigma_i\left(A^{\mathsf{T}}\right) = \sigma_i\left(A^*\right) = \sigma_i\left(\bar{A}\right).$$

For any unitary $U \in \mathbb{C}^{m\times m}, V \in \mathbb{C}^{n\times n}$.

$$\sigma_i(A) = \sigma_i(UAV).$$

Relation to eigenvalues:

$$\sigma_i^2(A) = \lambda_i\left(AA^*\right) = \lambda_i\left(A^*A\right).$$

The singular value decomposition is very general in the sense that it can be applied to any $m \times n$ matrix, whereas the eigenvalue decomposition can only be applied to diagonalizable matrices. Nevertheless, the two decompositions are related.

Given an SVD of M, as described above, the following two relations hold:

$$\mathbf{M}^*\mathbf{M} = \mathbf{V\Sigma}^*\mathbf{U}^*\,\mathbf{U\Sigma V}^* = \mathbf{V}(\mathbf{\Sigma}^*\mathbf{\Sigma})\mathbf{V}^*$$

$$\mathbf{MM}^* = \mathbf{U\Sigma V}^*\,\mathbf{V\Sigma}^*\mathbf{U}^* = \mathbf{U}(\mathbf{\Sigma\Sigma}^*)\mathbf{U}^*$$

The right-hand sides of these relations describe the eigenvalue decompositions of the left-hand sides. Consequently:

The columns of $\mathbf{V}$ (right-singular vectors) are eigenvectors of $\mathbf{M}^*\mathbf{M}$.

The columns of $\mathbf{U}$ (left-singular vectors) are eigenvectors of $\mathbf{M}\mathbf{M}^*$.

The non-zero elements of $\mathbf{\Sigma}$ (non-zero singular values) are the square roots of the non-zero eigenvalues of $\mathbf{M}^*\mathbf{M}$ or $\mathbf{M}\mathbf{M}^*$.

6.8.1.1 Example for a real matrix: only singular values

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")
A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> Query = "rank, nonzeros, absdet, logabsdet, S, SPlus"
>>> Res = A.eigen_jacobiSvd2(Query)

>>> print("rank         : ", Res["rank"])
rank         :  6
>>> print("nonzeros     : ", Res["nonzeros"])
nonzeropivots:  6

print("absdet   : ", Res["absdet"])
absdet   :  48772174.208000000000000000000000149
print("logabsdet: ", Res["logabsdet"])
logabsdet:  17.702670507541421335076102045531518
print()

print("rcond   : ", Res["rcond"])  # TBD
print("isinvertible: ", Res["isinvertible"])  # TBD
print()

>>> Res["S"].show("Singular values")
Singular values:
  169,
 58.6,
 44.1,
 15.2,
 7.75,
0.950,

>>> Res["SPlus"].show("SPlus")
SPlus:
0.00593,
 0.0171,
 0.0227,
 0.0657,
  0.129,
   1.05,
```

6.8.1.2 Example for a complex matrix: only singular values

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(6,4); A.show("A")
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
```

(continues on next page)

(continued from previous page)

```
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,
23.0 + 43.0j, 15.0 + 11.0j, 28.0 + 19.0j, 35.0 + 32.0j,
33.0 + 49.0j, 42.0 + 48.0j, 47.0 + 32.0j, 34.0 + 25.0j,

>>> Query = "rank, nonzeros, S, SPlus"
>>> Res = A.eigen_jacobiSvd2(Query)

>>> print("rank         : ", Res["rank"])
rank         :  4
>>> print("nonzeros     : ", Res["nonzeros"])
nonzeropivots:  4

>>> Res["S"].show("Singular values")
Singular values:
 208 + 0j,
62.1 + 0j,
45.6 + 0j,
18.6 + 0j,

>>> Res["SPlus"].show("SPlus")
SPlus:
0.00480 + 0j,
 0.0161 + 0j,
 0.0219 + 0j,
 0.0536 + 0j,
```

6.8.2 Singular Value Decomposition, singular values and thin singular vectors

`ctx.mat_jacobi_svd_thin`(*matA*, *Query*, *matB*)

Returns the two-sided Jacobi SVD decomposition of a rectangular matrix.

Writing `matA.JacobiSvdThin(Query, matB)` has the same effect

See also: Wikipedia [1418], Wikipedia [1416].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

matB
Optional. A general n-by-m matrix of the same type as A. You need to specify B only if you want to solve the linear equation $AX = B$

Results:

absdet
A scalar of a return type matching A. The absolute value of the determinant of A.

rank
An integer. The rank of A.

X
A general matrix of the same type and dimension as B. The solution to $AX = B$.

pseudoInverse
A square matrix of the same type and dimension as A. The pseudo-inverse of A, A^{-1}.

SV
A vector of the same type`A`. The singular values A (see below).

thinU
A square matrix of the same type and dimension as A. The thin singular vectors U (see below).

thinV
A square matrix of the same type and dimension as A. The thin singular vectors V (see below).

6.8.2.1 Example for a real matrix: the singular values, pseudoinverse and thin singular vectors

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,
```

(continues on next page)

(continued from previous page)

```
>>> Query = "rank, nonzeros, S, U, V, X, PseudoInverse, SPlus"
>>> Res = A.eigen_jacobiSvdThin(Query, B)

>>> print("rank         : ", Res["rank"])
rank         :  6
>>> print("nonzeros: ", Res["nonzeros"])
nonzeropivots:  6

>>> Res["S"].show("Singular values")
Singular values:
  169,
 58.6,
 44.1,
 15.2,
 7.75,
0.950,

>>> Res["U"].show("U")
U:
  0.435,  -0.390,   0.336,  -0.697,  -0.243, -0.0356,
  0.238,  -0.526,   0.374,   0.680,  -0.132,  -0.220,
  0.439,   0.204,   0.280,   0.153,   0.296,   0.759,
  0.337,  0.0890,  -0.509,   0.161,  -0.737,   0.224,
  0.460,  -0.310,  -0.610, -0.0138,   0.541,  -0.165,
  0.486,   0.653,   0.194,  0.0550,  0.0143,  -0.544,

>>> Res["V"].show("V")
V:
   0.370,   -0.615,    0.580,    0.226,   -0.303,  -0.0706,
   0.591,    0.117,   -0.297,   -0.551,   -0.442,   -0.223,
   0.290,   -0.527,   -0.261,   -0.271,    0.692,    0.135,
   0.283,    0.466,    0.563,   -0.127,    0.473,   -0.382,
   0.419,    0.114,   -0.418,    0.737,   0.0986,   -0.288,
   0.416,    0.317,    0.125,    0.112, -0.00594,    0.836,


>>> # Checking the properties of the decomposition
>>> (Res["U"] * Res["S"].D * Res["V"].H).show("U * S.D * V.H (should be equal to A)")
U * S.D * V.H (should be equal to A):
 48.0,  43.0,  31.0,  19.0,  14.0,  24.0,
 46.0,  10.0,  20.0,  4.60,  14.0,  10.0,
 27.0,  39.0,  13.0,  34.0,  29.0,  37.0,
 7.10,  42.0,  15.0,  2.80,  35.0,  23.0,
 23.0,  50.0,  42.0, 0.440,  42.0,  23.0,
 12.0,  50.0,  1.20,  46.0,  36.0,  47.0,

>>> CheckResult = (A - Res["U"] * Res["S"].D * Res["V"].H).norm()
>>> print("||A - U * S.D * V.H|| (should be zero): ", CheckResult)
||A - U * S.D * V.H|| (should be zero):  8.104128577459762885052053024055112 59E-33


>>> # Checking the properties of the pseudoinverse
>>> Res["SPlus"].show("SPlus")
SPlus:
0.00593,
 0.0171,
 0.0227,
 0.0657,
  0.129,
   1.05,

>>> Pinv = Res["V"] * Res["SPlus"].D * Res["U"].H; Pinv.show("Pinv = V * SPlus.D * U.H")
Pinv = V * SPlus.D * U.H:
 0.0113,  0.0426, -0.0633, 0.00768, -0.0128,  0.0375,
 0.0459,  0.0319,  -0.201, -0.0115,  0.0136,   0.127,
-0.0121, -0.0523,   0.129, -0.0340,  0.0322, -0.0835,
```

(continues on next page)

(continued from previous page)

```
0.00719,  0.0757,  -0.283,  -0.141,  0.0901,   0.228,
-0.0289,  0.0940,  -0.220, -0.0636,  0.0627,   0.169,
-0.0363,  -0.190,   0.672,   0.198,  -0.148,  -0.473,

>>> CheckResult = (Pinv - Res["PseudoInverse"]).norm()
>>> print("||Pinv - PseudoInverse|| (should be zero): ", CheckResult)
||Pinv - PseudoInverse|| (should be zero):  2.28256434739527113292893309427326250E-36

>>> CheckResult = (A - A * Pinv * A).norm()
>>> print("||A - A * Pinv * A|| (should be zero): ", CheckResult)
||A - A * Pinv * A|| (should be zero):  8.984340599064574644416585197367274900E-33



>>> # Checking the properties of the solution x
>>> XPlus = Pinv * B; XPlus.show("XPlus")
XPlus:
 21.6,  21.7,  21.7,  21.7,  21.7,  21.8,
 8.21,  8.22,  8.22,  8.23,  8.23,  8.24,
-20.5, -20.6, -20.6, -20.6, -20.6, -20.6,
-15.3, -15.3, -15.4, -15.4, -15.4, -15.4,
 16.9,  16.9,  17.0,  17.0,  17.0,  17.0,
 8.73,  8.76,  8.78,  8.80,  8.82,  8.85,

>>> CheckResult = (XPlus - Res["X"]).norm()
>>> print("||XPlus - X]|| (should be zero): ", CheckResult)
||XPlus - X]|| (should be zero):  5.9597147582749293858003475468366907 1E-33

>>> CheckResult = (B - A * Res["X"]).norm()
>>> print("||B - A * X|| (should be zero): ", CheckResult)
||B - A * X|| (should be zero):  4.859012245302536794173943421310615 88E-32



>>> # Checking the relationship to the eigenvalues of A * A.H
>>> C = A * A.H if (A.rows > A.cols) else A.H * A
>>> D = C.eigen_SelfAdjointEigenSystem2("eval")["eval"]; D.show("D: Eigenvalues of A * A.H")
D: Eigenvalues of A * A.H:
  0.902,
   60.0,
    232,
1.94E+3,
3.43E+3,
2.85E+4,

>>> E = (Res["S"].cwiseProduct(Res["S"])).reverse_full()
>>> E.show("E: Squared singular values, in ascending order")
E: Squared singular values, in ascending order:
  0.902,
   60.0,
    232,
1.94E+3,
3.43E+3,
2.85E+4,

>>> (D - E).show("D-E (should be a zero vector)")
D-E (should be a zero vector):
-1.48E-32,
-3.34E-32,
 -5.4E-32,
 -1.3E-31,
 -4.1E-31,
    4E-31,
```

6.8.2.2 Example for a complex matrix: the singular values, pseudoinverse and thin singular vectors

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableA6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,

>>> Query = "rank, nonzeros, S, U, V, X, PseudoInverse, SPlus"
>>> Res = A.eigen_jacobiSvdThin(Query, B)

>>> print("rank         : ", Res["rank"])
rank         :  4
>>> print("nonzeros: ", Res["nonzeros"])
nonzeropivots:  4

>>> Res["S"].show("Singular values")
Singular values:
 163 + 0j,
54.2 + 0j,
39.9 + 0j,
3.63 + 0j,

>>> Res["U"].show("U")
U:
 -0.430 - 0.102j,   0.409 - 0.250j,  0.585 - 0.0708j,  -0.324 + 0.348j,
 -0.476 - 0.153j, -0.582 + 0.0348j,  -0.170 - 0.215j,  -0.546 - 0.190j,
-0.617 - 0.0410j,  0.0954 - 0.293j,  -0.386 + 0.495j,  0.359 - 0.0131j,
 -0.375 - 0.176j, -0.0724 + 0.574j,   0.349 - 0.247j,   0.534 - 0.159j,

>>> Res["V"].show("V")
V:
  -0.437 + 0.150j,    0.592 + 0.283j,    0.274 - 0.230j, -0.478 + 0.00322j,
  -0.383 + 0.383j,  -0.556 - 0.0490j,   -0.250 + 0.254j,  -0.513 + 0.0673j,
  -0.411 + 0.231j,    0.214 + 0.327j,  -0.572 + 0.0914j,   0.537 - 0.0308j,
  -0.425 + 0.300j,   -0.237 - 0.221j,   0.634 - 0.0880j,   0.463 - 0.0218j,


>>> # Checking the properties of the decomposition
>>> (Res["U"] * Res["S"].D * Res["V"].H).show("U * S.D * V.H (should be equal to A)")
U * S.D * V.H (should be equal to A):
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> CheckResult = (A - Res["U"] * Res["S"].D * Res["V"].H).norm()
>>> print("||A - U * S.D * V.H|| (should be zero): ", CheckResult)
||A - U * S.D * V.H|| (should be zero):  7.23797623649041832083182110841296889E-33 + 0j


>>> # Checking the properties of the pseudoinverse
>>> Res["SPlus"].show("SPlus")
SPlus:
0.00615 + 0j,
```

(continues on next page)

(continued from previous page)

```
 0.0185 + 0j,
 0.0250 + 0j,
  0.275 + 0j,

>>> Pinv = Res["V"] * Res["SPlus"].D * Res["U"].H; Pinv.show("Pinv = V * SPlus.D * U.H")
Pinv = V * SPlus.D * U.H:
   0.0516 + 0.0468j,    0.0667 - 0.0273j, -0.0516 + 0.000462j,   -0.0635 - 0.0282j,
   0.0449 + 0.0423j,    0.0801 - 0.0400j, -0.0447 + 0.000795j,  -0.0814 - 0.00721j,
  -0.0583 - 0.0458j,   -0.0783 + 0.0245j,   0.0599 + 0.00584j,    0.0786 + 0.0125j,
  -0.0338 - 0.0465j,   -0.0674 + 0.0325j,    0.0409 - 0.0104j,    0.0738 + 0.0219j,

>>> CheckResult = (Pinv - Res["PseudoInverse"]).norm()
>>> print("||Pinv - PseudoInverse|| (should be zero): ", CheckResult)
||Pinv - PseudoInverse|| (should be zero):  6.32060123722419305732811075240482494E-37 + 0j

>>> CheckResult = (A - A * Pinv * A).norm()
>>> print("||A - A * Pinv * A|| (should be zero): ", CheckResult)
||A - A * Pinv * A|| (should be zero):  4.78955112719344504545703109417795418E-33 + 0j



>>> # Checking the properties of the solution x
>>> XPlus = Pinv * B; XPlus.show("XPlus")
XPlus:
-0.869 - 3.37j, -0.858 - 3.37j, -0.846 - 3.38j, -0.835 - 3.38j,
 -1.82 - 3.22j,  -1.82 - 3.22j,  -1.82 - 3.23j,  -1.81 - 3.23j,
  2.15 + 3.53j,   2.15 + 3.53j,   2.16 + 3.53j,   2.16 + 3.53j,
  1.81 + 3.52j,   1.82 + 3.53j,   1.84 + 3.54j,   1.86 + 3.55j,

>>> CheckResult = (XPlus - Res["X"]).norm()
>>> print("||XPlus - X]|| (should be zero): ", CheckResult)
||XPlus - X]|| (should be zero):  8.35763124336076459797867686951053026E-35 + 0j

>>> CheckResult = (B - A * Res["X"]).norm()
>>> print("||B - A * X|| (should be zero): ", CheckResult)
||B - A * X|| (should be zero):  8.25590697622011727085329191499069154E-33 + 0j



>>> # Checking the relationship to the eigenvalues of A * A.H
>>> C = A * A.H if (A.rows > A.cols) else A.H * A
>>> D = C.eigen_SelfAdjointEigenSystem2("eval")["eval"]; D.show("D: Eigenvalues of A * A.H")
D: Eigenvalues of A * A.H:
   13.2 + 0j,
1.60E+3 + 0j,
2.93E+3 + 0j,
2.65E+4 + 0j,

>>> E = (Res["S"].cwiseProduct(Res["S"])).reverse_full();
>>> E.show("E: Squared singular values, in ascending order")
E: Squared singular values, in ascending order:
   13.2 + 0j,
1.60E+3 + 0j,
2.93E+3 + 0j,
2.65E+4 + 0j,

>>> (D - E).show("D-E (should be a zero vector)")
D-E (should be a zero vector):
-5.10E-33 + 0j,
-1.40E-31 + 0j,
 2.40E-31 + 0j,
 7.00E-31 + 0j,
```

6.8.3 Singular Value Decomposition, singular values and full singular vectors

`ctx.mat_jacobi_svd_full`(*matA*, *Query*, *matB*)

Returns the two-sided Jacobi SVD decomposition of a rectangular matrix.

Writing `matA.JacobiSvdFull(Query, matB)` has the same effect

See also Eigen [229], Wikipedia [1418], Wikipedia [1416], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

matB
Optional. A general n-by-m matrix of the same type as A. You need to specify B only if you want to solve the linear equation $AX = B$

Results:

absdet
A scalar of a return type matching A. The absolute value of the determinant of A.

rank
An integer. The rank of A.

X
A general matrix of the same type and dimension as B. The solution to $AX = B$.

pseudoInverse
A square matrix of the same type and dimension as A. The pseudo-inverse of A, A^{-1}.

SV
A vector of the same type`A`. The singular values A (see below).

fullU
A square matrix of the same type and dimension as A. The full singular vectors U (see below).

fullV
A square matrix of the same type and dimension as A. The full singular vectors V (see below).

SVD decomposition consists in decomposing any n-by-p matrix A as a product

$A = USV*$

where U is a n-by-n unitary, V is a p-by-p unitary, and S is a n-by-p real positive matrix which is zero outside of its main diagonal; the diagonal entries of S are known as the singular values of A and the columns of U and V are known as the left and right singular vectors of A respectively. Singular values are always sorted in decreasing order.

This JacobiSVD decomposition computes only singular values by default. If you want U or V , you need to ask for them explicitly.

You can ask for only thin U or V to be computed, meaning the following. In case of a rectangular n-by-p matrix, letting m be the smaller value among n and p, there are only m singular vectors; the remaining columns of U and V do not correspond to actual singular vectors. Asking for thin U or V means asking for only their m first columns to be formed. So U is then a n-by-m matrix, and V is then a p-by-m matrix. Notice that thin U and V are all you need for (least squares) solving.

This JacobiSVD class is a two-sided Jacobi R-SVD decomposition, ensuring optimal reliability and accuracy. The downside is that it's slower than bidiagonalizing SVD algorithms for large square matrices; however its complexity is still where n is the smaller dimension and p is the greater dimension, meaning that it is still of the same order of complexity as the faster bidiagonalizing R-SVD algorithms. In particular, like any R-SVD, it takes advantage of non-squareness in that its complexity is only linear in the greater dimension.

If the input matrix has inf or nan coefficients, the result of the computation is undefined, but the computation is guaranteed to terminate in finite (and reasonable) time.

The possible values for QRPreconditioner are:

- ColPivHouseholderQRPreconditioner is the default. In practice it's very safe. It uses column-pivoting QR.
- FullPivHouseholderQRPreconditioner, is the safest and slowest. It uses full-pivoting QR. Contrary to other QRs, it doesn't allow computing thin unitaries.
- HouseholderQRPreconditioner is the fastest, and less safe and accurate than the pivoting variants. It uses non-pivoting QR. This is very similar in safety and accuracy to the bidiagonalization process used by bidiagonalizing SVD algorithms (since bidiagonalization is inherently non-pivoting). However the resulting SVD is still more reliable than bidiagonalizing SVDs because the Jacobi-based iterarive process is more reliable than the optimized bidiagonal SVD iterations.
- NoQRPreconditioner allows not to use a QR preconditioner at all. This is useful if you know that you will only be computing JacobiSVD decompositions of square matrices. Non-square matrices require a QR preconditioner. Using this option will result in faster compilation and smaller executable code. It won't significantly speed up computation, since JacobiSVD is always checking if QR preconditioning is needed before applying it anyway.

const internal::solve retval¡JacobiSVD, Rhs¿ solve (const MatrixBase¡ Rhs ¿ & b) const Returns a (least squares) solution of using the current SVD decomposition of A. Parameters: b the right-hand-side of the equation to solve.

Note: Solving requires both U and V to be computed. Thin U and V are enough, there is no need for full U or V. SVD solving is implicitly least-squares. Thus, this method serves both purposes of exact solving and least-squaressolving. In other words, the returned solution is guaranteed to minimize the Euclidean norm $||Ax - b||$.

Example for a real matrix: the singular values, pseudoinverse and full singular vectors

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableB6x6", ""); B.show("B")
B:
911, 912, 913, 914, 915, 916,
921, 922, 923, 924, 925, 926,
931, 932, 933, 934, 935, 936,
941, 942, 943, 944, 945, 946,
951, 952, 953, 954, 955, 956,
961, 962, 963, 964, 965, 966,

>>> Query = "rank, nonzeros, S, U, V, X, PseudoInverse, SPlus"
>>> Res = A.eigen_jacobiSvdFull(Query, B)

>>> print("rank         : ", Res["rank"])
rank         :  6
>>> print("nonzeros: ", Res["nonzeros"])
nonzeropivots:  6

>>> Res["S"].show("Singular values")
Singular values:
  169,
 58.6,
 44.1,
 15.2,
 7.75,
0.950,

>>> Res["U"].show("U")
U:
  0.435,  -0.390,   0.336,  -0.697,  -0.243, -0.0356,
```

(continues on next page)

(continued from previous page)

```
  0.238,  -0.526,   0.374,   0.680,  -0.132,  -0.220,
  0.439,   0.204,   0.280,   0.153,   0.296,   0.759,
  0.337,  0.0890,  -0.509,   0.161,  -0.737,   0.224,
  0.460,  -0.310,  -0.610, -0.0138,   0.541,  -0.165,
  0.486,   0.653,   0.194,  0.0550,  0.0143,  -0.544,

>>> Res["V"].show("V")
V:
   0.370,   -0.615,    0.580,    0.226,   -0.303,  -0.0706,
   0.591,    0.117,   -0.297,   -0.551,   -0.442,   -0.223,
   0.290,   -0.527,   -0.261,   -0.271,    0.692,    0.135,
   0.283,    0.466,    0.563,   -0.127,    0.473,   -0.382,
   0.419,    0.114,   -0.418,    0.737,   0.0986,   -0.288,
   0.416,    0.317,    0.125,    0.112, -0.00594,    0.836,


>>> # Checking the properties of the decomposition
>>> (Res["U"] * Res["S"].D * Res["V"].H).show("U * S.D * V.H (should be equal to A)")
U * S.D * V.H (should be equal to A):
 48.0,  43.0,  31.0,  19.0,  14.0,  24.0,
 46.0,  10.0,  20.0,  4.60,  14.0,  10.0,
 27.0,  39.0,  13.0,  34.0,  29.0,  37.0,
 7.10,  42.0,  15.0,  2.80,  35.0,  23.0,
 23.0,  50.0,  42.0, 0.440,  42.0,  23.0,
 12.0,  50.0,  1.20,  46.0,  36.0,  47.0,

>>> CheckResult = (A - Res["U"] * Res["S"].D * Res["V"].H).norm()
>>> print("||A - U * S.D * V.H|| (should be zero): ", CheckResult)
||A - U * S.D * V.H|| (should be zero):  8.104128577459762885052053024055112590E-33


>>> # Checking the properties of the pseudoinverse
>>> Res["SPlus"].show("SPlus")
SPlus:
0.00593,
 0.0171,
 0.0227,
 0.0657,
  0.129,
   1.05,

>>> Pinv = Res["V"] * Res["SPlus"].D * Res["U"].H; Pinv.show("Pinv = V * SPlus.D * U.H")
Pinv = V * SPlus.D * U.H:
 0.0113,  0.0426, -0.0633, 0.00768, -0.0128,  0.0375,
 0.0459,  0.0319,  -0.201, -0.0115,  0.0136,   0.127,
-0.0121, -0.0523,   0.129, -0.0340,  0.0322, -0.0835,
0.00719,  0.0757,  -0.283,  -0.141,  0.0901,   0.228,
-0.0289,  0.0940,  -0.220, -0.0636,  0.0627,   0.169,
-0.0363,  -0.190,   0.672,   0.198,  -0.148,  -0.473,

>>> CheckResult = (Pinv - Res["PseudoInverse"]).norm()
>>> print("||Pinv - PseudoInverse|| (should be zero): ", CheckResult)
||Pinv - PseudoInverse|| (should be zero):  2.282564347395271132928933094273262500E-36

>>> CheckResult = (A - A * Pinv * A).norm()
>>> print("||A - A * Pinv * A|| (should be zero): ", CheckResult)
||A - A * Pinv * A|| (should be zero):  8.984340599064574644416585197367274900E-33



>>> # Checking the properties of the solution x
>>> XPlus = Pinv * B; XPlus.show("XPlus")
XPlus:
 21.6,  21.7,  21.7,  21.7,  21.7,  21.8,
 8.21,  8.22,  8.22,  8.23,  8.23,  8.24,
-20.5, -20.6, -20.6, -20.6, -20.6, -20.6,
```

(continues on next page)

(continued from previous page)

```
-15.3, -15.3, -15.4, -15.4, -15.4, -15.4,
 16.9,  16.9,  17.0,  17.0,  17.0,  17.0,
 8.73,  8.76,  8.78,  8.80,  8.82,  8.85,

>>> CheckResult = (XPlus - Res["X"]).norm()
>>> print("||XPlus - X||| (should be zero): ", CheckResult)
||XPlus - X||| (should be zero):  5.9597147582749293858003475468366907 1E-33

>>> CheckResult = (B - A * Res["X"]).norm()
>>> print("||B - A * X|| (should be zero): ", CheckResult)
||B - A * X|| (should be zero):  4.859012245302536794173943421310615 88E-32



>>> # Checking the relationship to the eigenvalues of A * A.H
>>> C = A * A.H if (A.rows > A.cols) else A.H * A
>>> D = C.eigen_SelfAdjointEigenSystem2("eval")["eval"]; D.show("D: Eigenvalues of A * A.H")
D: Eigenvalues of A * A.H:
  0.902,
   60.0,
    232,
1.94E+3,
3.43E+3,
2.85E+4,

>>> E = (Res["S"].cwiseProduct(Res["S"])).reverse_full()
>>> E.show("E: Squared singular values, in ascending order")
E: Squared singular values, in ascending order:
  0.902,
   60.0,
    232,
1.94E+3,
3.43E+3,
2.85E+4,

>>> (D - E).show("D-E (should be a zero vector)")
D-E (should be a zero vector):
-1.48E-32,
-3.34E-32,
 -5.4E-32,
 -1.3E-31,
 -4.1E-31,
    4E-31,
```

6.8.3.1 Example for a complex matrix: the singular values, pseudoinverse and full singular vectors

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableA6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
11.0 + 31.0j, 12.0 + 32.0j, 13.0 + 33.0j, 14.0 + 34.0j,
21.0 + 41.0j, 22.0 + 42.0j, 23.0 + 43.0j, 24.0 + 44.0j,
31.0 + 51.0j, 32.0 + 52.0j, 33.0 + 53.0j, 34.0 + 54.0j,
41.0 + 61.0j, 42.0 + 62.0j, 43.0 + 63.0j, 44.0 + 64.0j,
```

(continues on next page)

(continued from previous page)

```
>>> Query = "rank, nonzeros, S, U, V, X, PseudoInverse, SPlus"
>>> Res = A.eigen_jacobiSvdThin(Query, B)

>>> print("rank         : ", Res["rank"])
rank         :  4
>>> print("nonzeros: ", Res["nonzeros"])
nonzeropivots:  4

>>> Res["S"].show("Singular values")
Singular values:
 163 + 0j,
54.2 + 0j,
39.9 + 0j,
3.63 + 0j,

>>> Res["U"].show("U")
U:
 -0.430 - 0.102j,   0.409 - 0.250j,  0.585 - 0.0708j,  -0.324 + 0.348j,
 -0.476 - 0.153j, -0.582 + 0.0348j,  -0.170 - 0.215j,  -0.546 - 0.190j,
-0.617 - 0.0410j,  0.0954 - 0.293j,  -0.386 + 0.495j,  0.359 - 0.0131j,
 -0.375 - 0.176j, -0.0724 + 0.574j,   0.349 - 0.247j,   0.534 - 0.159j,

>>> Res["V"].show("V")
V:
  -0.437 + 0.150j,    0.592 + 0.283j,    0.274 - 0.230j, -0.478 + 0.00322j,
  -0.383 + 0.383j,  -0.556 - 0.0490j,   -0.250 + 0.254j,  -0.513 + 0.0673j,
  -0.411 + 0.231j,    0.214 + 0.327j,  -0.572 + 0.0914j,   0.537 - 0.0308j,
  -0.425 + 0.300j,   -0.237 - 0.221j,   0.634 - 0.0880j,   0.463 - 0.0218j,


>>> # Checking the properties of the decomposition
>>> (Res["U"] * Res["S"].D * Res["V"].H).show("U * S.D * V.H (should be equal to A)")
U * S.D * V.H (should be equal to A):
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> CheckResult = (A - Res["U"] * Res["S"].D * Res["V"].H).norm()
>>> print("||A - U * S.D * V.H|| (should be zero): ", CheckResult)
||A - U * S.D * V.H|| (should be zero):  7.23797623649041832083182110841296889E-33 + 0j


>>> # Checking the properties of the pseudoinverse
>>> Res["SPlus"].show("SPlus")
SPlus:
0.00615 + 0j,
 0.0185 + 0j,
 0.0250 + 0j,
  0.275 + 0j,

>>> Pinv = Res["V"] * Res["SPlus"].D * Res["U"].H; Pinv.show("Pinv = V * SPlus.D * U.H")
Pinv = V * SPlus.D * U.H:
   0.0516 + 0.0468j,    0.0667 - 0.0273j, -0.0516 + 0.000462j,   -0.0635 - 0.0282j,
   0.0449 + 0.0423j,    0.0801 - 0.0400j, -0.0447 + 0.000795j,  -0.0814 - 0.00721j,
  -0.0583 - 0.0458j,   -0.0783 + 0.0245j,   0.0599 + 0.00584j,    0.0786 + 0.0125j,
  -0.0338 - 0.0465j,   -0.0674 + 0.0325j,    0.0409 - 0.0104j,    0.0738 + 0.0219j,

>>> CheckResult = (Pinv - Res["PseudoInverse"]).norm()
>>> print("||Pinv - PseudoInverse|| (should be zero): ", CheckResult)
||Pinv - PseudoInverse|| (should be zero):  6.32060123722419305732811075240482494E-37 + 0j

>>> CheckResult = (A - A * Pinv * A).norm()
>>> print("||A - A * Pinv * A|| (should be zero): ", CheckResult)
||A - A * Pinv * A|| (should be zero):  4.78955112719344504545703109417795418E-33 + 0j
```

(continues on next page)

(continued from previous page)

```
>>> # Checking the properties of the solution x
>>> XPlus = Pinv * B; XPlus.show("XPlus")
XPlus:
-0.869 - 3.37j, -0.858 - 3.37j, -0.846 - 3.38j, -0.835 - 3.38j,
 -1.82 - 3.22j,  -1.82 - 3.22j,  -1.82 - 3.23j,  -1.81 - 3.23j,
  2.15 + 3.53j,   2.15 + 3.53j,   2.16 + 3.53j,   2.16 + 3.53j,
  1.81 + 3.52j,   1.82 + 3.53j,   1.84 + 3.54j,   1.86 + 3.55j,

>>> CheckResult = (XPlus - Res["X"]).norm()
>>> print("||XPlus - X]|| (should be zero): ", CheckResult)
||XPlus - X]|| (should be zero):  8.3576312433607645979786768695105302 6E-35 + 0j

>>> CheckResult = (B - A * Res["X"]).norm()
>>> print("||B - A * X|| (should be zero): ", CheckResult)
||B - A * X|| (should be zero):  8.2559069762201172708532919149906915 4E-33 + 0j



>>> # Checking the relationship to the eigenvalues of A * A.H
>>> C = A * A.H if (A.rows > A.cols) else A.H * A
>>> D = C.eigen_SelfAdjointEigenSystem2("eval")["eval"]; D.show("D: Eigenvalues of A * A.H")
D: Eigenvalues of A * A.H:
   13.2 + 0j,
1.60E+3 + 0j,
2.93E+3 + 0j,
2.65E+4 + 0j,

>>> E = (Res["S"].cwiseProduct(Res["S"])).reverse_full();
>>> E.show("E: Squared singular values, in ascending order")
E: Squared singular values, in ascending order:
   13.2 + 0j,
1.60E+3 + 0j,
2.93E+3 + 0j,
2.65E+4 + 0j,

>>> (D - E).show("D-E (should be a zero vector)")
D-E (should be a zero vector):
-5.10E-33 + 0j,
-1.40E-31 + 0j,
 2.40E-31 + 0j,
 7.00E-31 + 0j,
```

6.8.4 Symmetric/Hermitian Eigensystem, only eigen values

`ctx.mat_self_adjoint_eigen_values`(*matA*, *Query*)

Returns the eigendecomposition of the symmetric/hermitian matrix *matA* $= A$.

Writing `matA.SelfAdjointEigenValues(Query)` has the same effect

See also Eigen [238], Wikipedia [1403], Wikipedia [1405], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

info
Reports whether previous computation was successful.

eval
Returns the eigenvalues of given matrix.

evec
Returns the eigenvectors of given matrix.

sqrt
Returns the positive-definite square root of the matrix

inversesqrt
Returns the inverse square root of the matrix

A matrix A is selfadjoint if it equals its adjoint. For real matrices, this means that the matrix is symmetric: it equals its transpose. This class computes the eigenvalues and eigenvectors of a selfadjoint matrix. These are the scalars λ and vectors v such that $Av = \lambda v$. The eigenvalues of a selfadjoint matrix are always real. If D is a diagonal matrix with the eigenvalues on the diagonal, and V is a matrix with the eigenvectors as its columns, then $A = VDV^{-1}$ (for selfadjoint matrices, the matrix V is always invertible). This is called the eigendecomposition. The algorithm exploits the fact that the matrix is selfadjoint, making it faster and more accurate than the general purpose eigenvalue algorithms implemented in EigenSolver and ComplexEigenSolver. Only the lower triangular part of the input matrix is referenced.

Call the function compute() to compute the eigenvalues and eigenvectors of a given matrix. Alternatively, you can use the SelfAdjointEigenSolver(const MatrixType, int) constructor which computes the eigenvalues and eigenvectors at construction time. Once the eigenvalue and eigenvectors are computed, they can be retrieved with the eigenvalues() and eigenvectors() functions. The documentation for SelfAdjointEigenSolver(const MatrixType, int) contains an example of the typical use of this class.

To solve the generalized eigenvalue problem $Av = \lambda Bv$ and the likes, see the class GeneralizedSelfAdjointEigenSolver. This implementation uses a symmetric QR algorithm. The matrix is first reduced to tridiagonal form using the Tridiagonalization class. The tridiagonal matrix is then brought to diagonal form with implicit symmetric QR steps with Wilkinson shift. Details can be found in Section 8.3 of Golub & Van Loan (1996). The cost of the computation is about 9n3 if the eigenvectors are required and 4n3/3 if they are not required.

6.8.4.1 Example for a real symmetric matrix: only eigenvalues

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAA6x6", ""); A.show("A")
A:
44.9, 25.5, 50.0, 47.9, 26.4, 62.0,
25.5, 24.3, 49.1, 95.0, 29.0, 46.6,
50.0, 49.1, 55.5, 84.0, 44.4, 26.7,
47.9, 95.0, 84.0, 64.5, 39.5, 87.5,
26.4, 29.0, 44.4, 39.5, 39.8, 12.3,
62.0, 46.6, 26.7, 87.5, 12.3, 85.0,
```

(continues on next page)

(continued from previous page)

```
>>> Query = "Eval"
>>> Res = A.eigen_SelfAdjointEigenValues2(Query)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i   L[i]
0: -61.7,
1: -27.7,
2:  4.34,
3:  25.8,
4:  62.5,
5:   311,

>>> X = +A; #X.show("X")
>>> AD = X.diagonal(); #AD.show("AD")
>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X.set_diagonal(0, AD - L[i])
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - I * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - I * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - I * L[i]) should be zero):
i   L[i]  Det(A - I * L[i])
0: -61.7,          3.71E-23,
1: -27.7,          7.41E-25,
2:  4.34,         -1.22E-25,
3:  25.8,          2.76E-25,
4:  62.5,         -2.13E-23,
5:   311,          1.24E-20,
```

6.8.4.2 Example for a hermitian matrix: only eigenvalues

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableRandomSAA6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")
A:
   80.0 + 0j, 50.0 - 23.0j, 85.0 + 5.00j, 36.0 - 4.30j,
50.0 + 23.0j,    30.0 + 0j, 43.0 + 9.50j, 27.0 + 11.0j,
85.0 - 5.00j, 43.0 - 9.50j,    85.0 + 0j, 55.0 - 7.00j,
36.0 + 4.30j, 27.0 - 11.0j, 55.0 + 7.00j,    23.0 + 0j,

Query = "Eval"
Res = A.eigen_SelfAdjointEigenValues2(Query)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i        L[i]
0: -21.6 + 0j,
1:  3.50 + 0j,
2:  14.1 + 0j,
3:   222 + 0j,

>>> X = +A; #X.show("X")
>>> AD = X.diagonal(); #AD.show("AD")
>>> Det = L * 0 # creates a zero vector of the same size and type as L
```

(continues on next page)

(continued from previous page)

```
>>> for i in range(A.rows):
>>>     X.set_diagonal(0, AD - L[i])
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - I * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - I * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - I * L[i]) should be zero):
i        L[i]      Det(A - I * L[i])
0: -21.6 + 0j,  5.65E-29 + 4.39E-30j,
1:  3.50 + 0j,  1.82E-29 - 2.71E-30j,
2:  14.1 + 0j, -1.40E-29 - 2.14E-30j,
3:   222 + 0j,  2.52E-26 - 5.79E-28j,
```

6.8.5 Symmetric/Hermitian Eigensystem, eigenvalues and eigenvectors

`ctx.mat_self_adjoint_eigen_system`(*matA*, *Query*)

Returns the eigendecomposition of the symmetric/hermitian matrix *matA* $= A$.

Writing `matA.SelfAdjointEigenSystem(Query)` has the same effect

See also Eigen [238], Wikipedia [1403], Wikipedia [1405], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

info
Reports whether previous computation was successful.

eval
Returns the eigenvalues of given matrix.

evec
Returns the eigenvectors of given matrix.

sqrt
Returns the positive-definite square root of the matrix

inversesqrt
Returns the inverse square root of the matrix

6.8.5.1 Example for a real symmetric matrix: eigenvalues and eigenvectors

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAA6x6", ""); A.show("A")
A:
44.9, 25.5, 50.0, 47.9, 26.4, 62.0,
25.5, 24.3, 49.1, 95.0, 29.0, 46.6,
50.0, 49.1, 55.5, 84.0, 44.4, 26.7,
47.9, 95.0, 84.0, 64.5, 39.5, 87.5,
26.4, 29.0, 44.4, 39.5, 39.8, 12.3,
62.0, 46.6, 26.7, 87.5, 12.3, 85.0,

Query = "Eval, Evec"
Res = A.eigen_SelfAdjointEigenSystem2(Query)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i   L[i]
0: -61.7,
1: -27.7,
2:  4.34,
3:  25.8,
4:  62.5,
5:   311,

>>> V =  Res["Evec"]; mt = "Matrix V of eigenvectors (V0, ... , V" + str(V.cols-1) + ")"
>>> V.show(mt, coltitles = ["V#"] * (V.cols))
Matrix V of eigenvectors (V0, ... , V5):
      V0       V1       V2      V3      V4     V5
  0.0719,  -0.497,  -0.271,  0.730, -0.157, 0.343,
  -0.550,  -0.562,  0.0403, -0.473, 0.0957, 0.382,
  -0.277,   0.514,  -0.515, 0.0931,  0.464, 0.412,
```

(continues on next page)

(continued from previous page)

```
   0.744, -0.0396, -0.0409, -0.370, 0.0252, 0.553,
0.000270,  0.0462,   0.760,  0.310,  0.515, 0.243,
  -0.249,   0.410,   0.284, 0.0486, -0.697, 0.449,

>>> CheckResult = (A - V * L.D * V.eigen_inverse()).norm()
>>> print("||A - V * diag(L) * V^-1|| (should be zero): ", (CheckResult).s())
    ||A - V * diag(L) * V^-1|| (should be zero):  7.70E-33

>>> X = +A; #X.show("X")
>>> AD = X.diagonal(); #AD.show("AD")
>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X.set_diagonal(0, AD - L[i])
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - I * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - I * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - I * L[i]) should be zero):
i   L[i]  Det(A - I * L[i])
0: -61.7,            -0E-34,
1: -27.7,            -0E-34,
2:  4.34,             0E-35,
3:  25.8,             0E-34,
4:  62.5,             0E-34,
5:   311,             0E-33,


>>> for i in range(V.rows):
>>>     AV = AC * V.col(i); VL = V.col(i) * L[i]; X = AV - VL
>>>     Li = "L[" + str(i) + "]"; Vi = "V" + str(i)
>>>     print("Eigenvalue " + Li + ": ", L[i].s())
>>>     Result = V.col(i).concat_horizontal(AV).concat_horizontal(VL).concat_horizontal(X)
>>>     mt = "Checking the properties of eigenvector " + Vi + " (AV - VL should be a zero vector)"
>>>     ct = ["Eigenvector " + Vi, "AV = A * " + Vi, "VL = " + Vi + " * " + Li, "AV - VL"]
>>>     Result.show(mt, coltitles = ct)

Eigenvalue L[0]:  -61.7
Checking the properties of eigenvector V0 (AV - VL should be a zero vector):
Eigenvector V0  AV = A * V0  VL = V0 * L[0]   AV - VL
        0.0719,       -4.44,          -4.44,  3.7E-34,
        -0.550,        33.9,           33.9, -2.2E-33,
        -0.277,        17.1,           17.1,    3E-34,
         0.744,       -45.9,          -45.9,  2.9E-33,
      0.000270,     -0.0166,        -0.0166, 6.52E-34,
        -0.249,        15.4,           15.4, -1.5E-33,

Eigenvalue L[1]:  -27.7
Checking the properties of eigenvector V1 (AV - VL should be a zero vector):
Eigenvector V1  AV = A * V1  VL = V1 * L[1]   AV - VL
        -0.497,        13.8,           13.8,    0E-34,
        -0.562,        15.6,           15.6,    0E-34,
         0.514,       -14.3,          -14.3,   -2E-34,
       -0.0396,        1.10,           1.10, -3.4E-34,
        0.0462,       -1.28,          -1.28,    7E-35,
         0.410,       -11.4,          -11.4,   -6E-34,

Eigenvalue L[2]:  4.34
Checking the properties of eigenvector V2 (AV - VL should be a zero vector):
Eigenvector V2  AV = A * V2  VL = V2 * L[2]    AV - VL
        -0.271,       -1.17,          -1.17,    -4E-35,
        0.0403,       0.175,          0.175,   9.9E-35,
        -0.515,       -2.23,          -2.23,   2.4E-34,
       -0.0409,      -0.177,         -0.177, -4.07E-34,
```

(continues on next page)

(continued from previous page)

```
         0.760,        3.30,          3.30,    -7E-35,
         0.284,        1.23,          1.23,     4E-35,

Eigenvalue L[3]:  25.8
Checking the properties of eigenvector V3 (AV - VL should be a zero vector):
Eigenvector V3  AV = A * V3  VL = V3 * L[3]  AV - VL
         0.730,        18.8,          18.8,   1E-34,
        -0.473,       -12.2,         -12.2,   6E-34,
        0.0931,        2.40,          2.40, 2.8E-34,
        -0.370,       -9.54,         -9.54, 7.1E-34,
         0.310,        7.99,          7.99, 2.6E-34,
        0.0486,        1.25,          1.25,   2E-35,

Eigenvalue L[4]:  62.5
Checking the properties of eigenvector V4 (AV - VL should be a zero vector):
Eigenvector V4  AV = A * V4  VL = V4 * L[4]   AV - VL
        -0.157,       -9.78,         -9.78, 1.05E-33,
        0.0957,        5.98,          5.98, -6.2E-34,
         0.464,        29.0,          29.0, -1.6E-33,
        0.0252,        1.58,          1.58,  1.5E-34,
         0.515,        32.2,          32.2, -1.8E-33,
        -0.697,       -43.5,         -43.5,  2.6E-33,

Eigenvalue L[5]:  311
Checking the properties of eigenvector V5 (AV - VL should be a zero vector):
Eigenvector V5  AV = A * V5  VL = V5 * L[5]   AV - VL
         0.343,         106,           106,   -1E-33,
         0.382,         119,           119,   -3E-33,
         0.412,         128,           128,   -2E-33,
         0.553,         172,           172,   -4E-33,
         0.243,        75.5,          75.5, -1.8E-33,
         0.449,         140,           140,   -1E-33,
```

6.8.5.2 Example for a hermitian matrix: eigenvalues and eigenvectors

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableRandomSAA6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")
A:
   80.0 + 0j, 50.0 - 23.0j, 85.0 + 5.00j, 36.0 - 4.30j,
50.0 + 23.0j,    30.0 + 0j, 43.0 + 9.50j, 27.0 + 11.0j,
85.0 - 5.00j, 43.0 - 9.50j,    85.0 + 0j, 55.0 - 7.00j,
36.0 + 4.30j, 27.0 - 11.0j, 55.0 + 7.00j,    23.0 + 0j,

Query = "Eval, Evec"
Res = A.eigen_SelfAdjointEigenSystem2(Query)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i        L[i]
0: -21.6 + 0j,
1:  3.50 + 0j,
2:  14.1 + 0j,
3:   222 + 0j,

>>> V =  Res["Evec"]; mt = "Matrix V of eigenvectors (V0, ... , V" + str(V.cols-1) + ")"
>>> V.show(mt, coltitles = ["V#"] * (V.cols))
Matrix V of eigenvectors (V0, ... , V3):
             V0                 V1                V2                V3
     0.458 + 0j,       -0.419 + 0j,      -0.502 + 0j,       0.602 + 0j,
-0.305 - 0.364j,    0.473 + 0.350j, -0.262 - 0.476j,  0.343 + 0.124j,
-0.472 + 0.234j, -0.225 + 0.0826j,  0.506 + 0.130j, 0.626 - 0.0122j,
```

(continues on next page)

(continued from previous page)

```
 0.523 - 0.113j,   0.586 - 0.278j,  0.390 + 0.164j, 0.336 + 0.0294j,

>>> AC = +A
>>> CheckResult = (AC - V * L.D * V.eigen_inverse()).norm().real
>>> print("||A - V * diag(L) * V^-1|| (should be zero): ", (CheckResult).s())
||A - V * diag(L) * V^-1|| (should be zero):  3.35E-33

>>> X = +A; #X.show("X")
>>> AD = X.diagonal(); #AD.show("AD")
>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X.set_diagonal(0, AD - L[i])
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - I * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - I * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - I * L[i]) should be zero):
i        L[i]      Det(A - I * L[i])
0: -21.6 + 0j,  5.65E-29 + 4.39E-30j,
1:  3.50 + 0j,  1.82E-29 - 2.71E-30j,
2:  14.1 + 0j, -1.40E-29 - 2.14E-30j,
3:   222 + 0j,  2.52E-26 - 5.79E-28j,


>>> for i in range(V.rows):
>>>     AV = AC * V.col(i); VL = V.col(i) * L[i]; X = AV - VL
>>>     Li = "L[" + str(i) + "]"; Vi = "V" + str(i)
>>>     print("Eigenvalue " + Li + ": ", L[i].s())
>>>     Result = V.col(i).concat_horizontal(AV).concat_horizontal(VL).concat_horizontal(X)
>>>     mt = "Checking the properties of eigenvector " + Vi + " (AV - VL should be a zero vector)"
>>>     ct = ["Eigenvector " + Vi, "AV = A * " + Vi, "VL = " + Vi + " * " + Li, "AV - VL"]
>>>     Result.show(mt, coltitles = ct)

Eigenvalue L[0]:  -21.6 + 0j
Checking the properties of eigenvector V0 (AV - VL should be a zero vector):
 Eigenvector V0        AV = A * V0  VL = V0 * L[0]                AV - VL
     0.458 + 0j, -9.92 + 5.00E-35j,     -9.92 + 0j, -1.90E-34 + 5.00E-35j,
-0.305 - 0.364j,      6.61 + 7.88j,   6.61 + 7.88j, -7.00E-34 - 3.60E-34j,
-0.472 + 0.234j,      10.2 - 5.06j,   10.2 - 5.06j, -2.00E-34 + 1.40E-34j,
 0.523 - 0.113j,     -11.3 + 2.45j,  -11.3 + 2.45j,  5.00E-34 + 1.00E-34j,

Eigenvalue L[1]:  3.50 + 0j
Checking the properties of eigenvector V1 (AV - VL should be a zero vector):
  Eigenvector V1      AV = A * V1   VL = V1 * L[1]                AV - VL
     -0.419 + 0j,       -1.47 + 0j,      -1.47 + 0j,         2.90E-34 + 0j,
  0.473 + 0.350j,    1.66 + 1.23j,    1.66 + 1.23j,  1.00E-35 + 2.50E-34j,
-0.225 + 0.0826j, -0.789 + 0.290j, -0.789 + 0.290j,  6.30E-35 + 1.43E-34j,
  0.586 - 0.278j,   2.06 - 0.974j,   2.06 - 0.974j, -3.70E-34 + 1.53E-34j,

Eigenvalue L[2]:  14.1 + 0j
Checking the properties of eigenvector V2 (AV - VL should be a zero vector):
 Eigenvector V2        AV = A * V2  VL = V2 * L[2]                AV - VL
    -0.502 + 0j, -7.10 - 3.20E-34j,     -7.10 + 0j,  2.30E-34 - 3.20E-34j,
-0.262 - 0.476j,     -3.70 - 6.73j,  -3.70 - 6.73j, -1.70E-34 - 6.20E-34j,
 0.506 + 0.130j,      7.16 + 1.83j,   7.16 + 1.83j, -3.10E-34 - 5.00E-34j,
 0.390 + 0.164j,      5.52 + 2.32j,   5.52 + 2.32j, -1.20E-34 - 1.90E-34j,

Eigenvalue L[3]:  222 + 0j
Checking the properties of eigenvector V3 (AV - VL should be a zero vector):
 Eigenvector V3      AV = A * V3  VL = V3 * L[3]                AV - VL
     0.602 + 0j, 134 + 1.40E-34j,       134 + 0j,         0 + 1.40E-34j,
 0.343 + 0.124j,    76.1 + 27.4j,   76.1 + 27.4j, -1.20E-33 - 8.00E-34j,
0.626 - 0.0122j,     139 - 2.72j,    139 - 2.72j, -1.00E-33 + 7.00E-35j,
```

(continues on next page)

(continued from previous page)

```
0.336 + 0.0294j,    74.5 + 6.54j,   74.5 + 6.54j, -1.20E-33 + 1.10E-34j,
```

6.8.6 Generalized Selfadjoint Eigensystem, only eigenvalues

`ctx.mat_generalized_self_adjoint_eigen_values(`*matA*, *Query*`)`

Returns the eigendecomposition of a generalized selfadjoint eigensystem.

Writing `matA.GeneralizedSelfAdjointEigenValues(Query)` has the same effect

See also Eigen [226], Wikipedia [1435], Wikipedia [1430], Wikipedia [1404].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

eval
Returns the eigenvalues of given matrix.

evec
Returns the eigenvectors of given matrix.

The generalized eigenvalues and eigenvectors of a matrix pair A and B are scalars λ and vectors v such that $Av = \lambda Bv$. If D is a diagonal matrix with the eigenvalues on the diagonal, and V is a matrix with the eigenvectors as its columns, then $AV = BVD$. The matrix V is almost always invertible, in which case we have $A = BVDV^{-1}$. This is called the generalized eigen-decomposition.

This class solves the generalized eigenvalue problem $Av = \lambda Bv$. In this case, the matrix A should be selfadjoint and the matrix B should be positive definite. Only the lower triangular part of the input matrix is referenced.

Parameters: [in] matA Selfadjoint matrix in matrix pencil. Only the lower triangular part of the matrix is referenced. [in] matB Positive-definite matrix in matrix pencil. Only the lower triangular part of the matrix is referenced. [in] options A or-ed set of flags ComputeEigenvectors,EigenvaluesOnly - `Ax_lBx,ABx_lx,BAx_lx`. Default is ComputeEigenvectors `Ax_lBx`.

Computes the eigenvalues and (if requested) the eigenvectors of the generalized eigenproblem $Ax = \lambda Bx$ with matA the selfadjoint matrix A and matB the positive definite matrix B. Each eigenvector x satisfies the property $x^T Bx = 1$. The eigenvectors are computed if options contains ComputeEigenvectors. In addition, the two following variants can be solved via options:

`ABx_lx`: $ABx = \lambda x$

`BAx_lx`: $BAx = \lambda x$

Computes generalized eigendecomposition of given matrix pencil. Parameters [in] matA Selfadjoint matrix in matrix pencil. Only the lower triangular part of the matrix is referenced. [in] matB Positive-definite matrix in matrix pencil. Only the lower triangular part of the matrix is referenced. [in] options A or-ed set of flags

ComputeEigenvectors,EigenvaluesOnly `Ax_lBx,ABx_lx,BAx_lx`. Default is ComputeEigenvectors `Ax_lBx`. According to options, this function computes eigenvalues and (if requested) the eigenvectors of one of the following three generalized eigenproblems:

`Ax_lBx`: $Ax = \lambda Bx$

`ABx_lx`: $ABx = \lambda x$

`BAx_lx`: $BAx = \lambda x$

with matA the selfadjoint matrix A and matB the positive definite matrix B. In addition, each eigenvector satisfies the property $x * Bx = 1$.The eigenvalues() function can be used to retrieve the eigenvalues. If options contains ComputeEigenvectors, then the eigenvectors are also computed and can be retrieved by calling eigenvectors().

The implementation uses LLT to compute the Cholesky decomposition $B = LL^T$ and computes the classical eigendecomposition of the selfadjoint matrix L^{-1} A(L*)^{-1} if options contains `Ax_lBx` and of $L * AL$ otherwise. This solves the generalized eigenproblem, because any solution of the generalized eigenproblem Ax = lambda Bx corresponds to a solution $L^{-1}A(L*)^{-1}(L*x) = \lambda(L * x)$ of the eigenproblem for $L^{-1}A(L*)^{-1}$. Similar statements can be made for the two other variants.

6.8.6.1 Example for a real symmetric matrix: eigenvalues only

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAPosDefB6x6", "")
>>> A.show("A (real symmetric)")
A (real symmetric):
60, 58, 65, 68, 23, 45,
58, 63, 65, 70, 20, 48,
65, 65, 88, 85, 33, 50,
68, 70, 85, 95, 35, 58,
23, 20, 33, 35, 25, 17,
45, 48, 50, 58, 17, 45,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAPosDefA6x6", "")
>>> B.show("B (real symmetric positive definite)")
B (real symmetric positive definite):
248,  40,  38,  43,  33,  35,
 40, 240,  25,  38,  28,  35,
 38,  25, 245,  40,  33,  17,
 43,  38,  40, 250,  30,  33,
 33,  28,  33,  30, 240,  22,
 35,  35,  17,  33,  22, 243,

>>> Query = "Eval"
>>> Res = A.eigen_GeneralizedSelfAdjointEigenValues2(Query, B)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i    L[i]
0: 0.0114,
1: 0.0163,
2: 0.0282,
3: 0.0455,
4: 0.0952,
5:  0.850,
>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X = A - B * L[i]
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - B * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - B * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - B * L[i]) should be zero):
i    L[i]  Det(A - B * L[i])
0: 0.0114,         -3.12E-29,
1: 0.0163,         -1.30E-30,
2: 0.0282,         -6.39E-29,
3: 0.0455,          2.67E-28,
4: 0.0952,         -7.75E-27,
5:  0.850,          1.43E-21,
```

6.8.6.2 Example for a hermitian matrix: eigenvalues only

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableRandomSAPosDefA6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A (hermitian)")
A (hermitian):
   91.0 + 0j, 12.0 - 3.60j, 22.0 - 7.40j, 14.0 + 1.10j,
12.0 + 3.60j,    77.0 + 0j, 2.50 - 2.00j, 3.40 - 7.60j,
22.0 + 7.40j, 2.50 + 2.00j,    91.0 + 0j, 17.0 + 3.70j,
```

(continues on next page)

(continued from previous page)

```
14.0 - 1.10j, 3.40 + 7.60j, 17.0 - 3.70j,    74.0 + 0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableRandomSAPosDefB6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B (hermitian positive definite)")
B (hermitian positive definite):
   85.0 + 0j, 7.30 + 1.40j, 9.20 + 2.30j, 3.10 + 1.80j,
7.30 - 1.40j,    73.0 + 0j, 19.0 - 1.80j, 3.70 + 2.00j,
9.20 - 2.30j, 19.0 + 1.80j,    89.0 + 0j, 13.0 - 8.30j,
3.10 - 1.80j, 3.70 - 2.00j, 13.0 + 8.30j,    84.0 + 0j,

>>> Query = "Eval"
>>> Res = A.eigen_GeneralizedSelfAdjointEigenValues2(Query, B)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
i       L[i]
0: 57.8 + 0j,
1: 67.0 + 0j,
2: 81.6 + 0j,
3:  127 + 0j,

>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X = A - B * L[i]
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - B * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - B * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - B * L[i]) should be zero):
i       L[i]     Det(A - B * L[i])
0: 57.8 + 0j, 4.30E+14 + 2.49E-23j,
1: 67.0 + 0j, 7.87E+14 + 5.51E-23j,
2: 81.6 + 0j, 1.75E+15 + 5.75E-23j,
3:  127 + 0j, 1.03E+16 + 1.19E-21j,
```

6.8.7 Generalized Selfadjoint Eigensystem: eigenvalues and eigenvectors

`ctx.mat_generalized_self_adjoint_eigen_system`(*matA*, *Query*)

Returns the eigendecomposition of a generalized selfadjoint eigensystem.

Writing `matA.GeneralizedSelfAdjointEigenSolver(Query)` has the same effect

See also Eigen [226], Wikipedia [1435], Wikipedia [1430], Wikipedia [1404].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

eval
Returns the eigenvalues of given matrix.

evec
Returns the eigenvectors of given matrix.

6.8.7.1 Example for a real symmetric matrix: eigenvalues and eigenvectors

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAPosDefB6x6", "")
>>> A.show("A (real symmetric)")
A (real symmetric):
60, 58, 65, 68, 23, 45,
58, 63, 65, 70, 20, 48,
65, 65, 88, 85, 33, 50,
68, 70, 85, 95, 35, 58,
23, 20, 33, 35, 25, 17,
45, 48, 50, 58, 17, 45,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAPosDefA6x6", "")
>>> B.show("B (real symmetric positive definite)")
B (real symmetric positive definite):
248,  40,  38,  43,  33,  35,
 40, 240,  25,  38,  28,  35,
 38,  25, 245,  40,  33,  17,
 43,  38,  40, 250,  30,  33,
 33,  28,  33,  30, 240,  22,
 35,  35,  17,  33,  22, 243,

>>> Query = "Eval, Evec"
>>> Res = A.eigen_GeneralizedSelfAdjointEigenSolver2(Query, B)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i    L[i]
0: 0.0114,
1: 0.0163,
2: 0.0282,
3: 0.0455,
4: 0.0952,
5:  0.850,

>>> V =  Res["Evec"]; mt = "Matrix V of eigenvectors (V0, ... , V" + str(V.cols-1) + ")"
>>> V.show(mt, coltitles = ["V#"] * (V.cols))
Matrix V of eigenvectors (V0, ... , V5):
      V0       V1        V2       V3       V4       V5
  0.0347, 0.00906,  -0.0464,  0.0197, -0.0160,  0.0172,
```

(continues on next page)

(continued from previous page)

```
 -0.0472, -0.0274, -0.00784, 0.00953, -0.0293,  0.0215,
-0.00569,  0.0245,   0.0310,  0.0386,  0.0196,  0.0292,
  0.0276, -0.0386,   0.0178, -0.0275,  0.0143,  0.0282,
 -0.0247, 0.00690,  -0.0328, -0.0181,  0.0481, 0.00121,
-0.00467,  0.0434,   0.0104, -0.0400, -0.0216,  0.0157,

>>> # Vinv = V^-1 = V.H
>>> Vinv = V.eigen_inverse(); Vinv.show(" V^-1")
>>> CheckResult = (A - V * L.D * Vinv).norm()
>>> print("||A - B * V * diag(L) * V^-1|| (should be zero): ", (CheckResult).s())
||A - V * diag(L) * V^-1|| (should be zero):  4.59E-33

>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X = A - B * L[i]
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - B * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - B * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - B * L[i]) should be zero):
i    L[i]  Det(A - B * L[i])
0: 0.0114,         -3.12E-29,
1: 0.0163,         -1.30E-30,
2: 0.0282,         -6.39E-29,
3: 0.0455,          2.67E-28,
4: 0.0952,         -7.75E-27,
5:  0.850,          1.43E-21,


>>> for i in range(V.rows):
>>>     AV = A * V.col(i); BVL = (B * L[i]) * V.col(i); X = AV - BVL
>>>     Li = "L[" + str(i) + "]"; Vi = "V" + str(i)
>>>     print("Eigenvalue " + Li + ": ", L[i].s())
>>>     Result = V.col(i).concat_horizontal(AV).concat_horizontal(BVL).concat_horizontal(X)
>>>     mt = "Checking the properties of eigenvector " + Vi + " (AV - BVL should be a zero vector)"
>>>     ct = ["Eigenvector " + Vi, "AV = A * " + Vi, "BVL = B * " + Vi + " * " + Li, "   AV - BVL"]
>>>     Result.show(mt, coltitles = ct)

Eigenvalue L[0]:  0.0114
Checking the properties of eigenvector V0 (AV - BVL should be a zero vector):
Eigenvector V0  AV = A * V0  BVL = B * V0 * L[0]     AV - BVL
        0.0347,      0.0769,              0.0769,    1.89E-35,
       -0.0472,      -0.113,              -0.113,     3.6E-35,
      -0.00569,     -0.0120,             -0.0120,    6.23E-35,
        0.0276,      0.0627,              0.0627,    4.16E-35,
       -0.0247,     -0.0637,             -0.0637,    1.67E-35,
      -0.00467,     -0.0149,             -0.0149,    2.83E-35,

Eigenvalue L[1]:  0.0163
Checking the properties of eigenvector V1 (AV - BVL should be a zero vector):
Eigenvector V1  AV = A * V1  BVL = B * V1 * L[1]     AV - BVL
       0.00906,      0.0354,              0.0354,   -2.30E-35,
       -0.0274,     -0.0876,             -0.0876,   -1.54E-35,
        0.0245,      0.0832,              0.0832,    2.3E-36,
       -0.0386,      -0.125,              -0.125,     -6E-36,
       0.00690,      0.0293,              0.0293,   -1.1E-36,
        0.0434,       0.150,               0.150,     -7E-36,

Eigenvalue L[2]:  0.0282
Checking the properties of eigenvector V2 (AV - BVL should be a zero vector):
Eigenvector V2  AV = A * V2  BVL = B * V2 * L[2]     AV - BVL
       -0.0464,      -0.299,              -0.299,     1.5E-35,
      -0.00784,     -0.0802,             -0.0802,   -1.53E-35,
```

(continues on next page)

(continued from previous page)

```
        0.0310,       0.154,               0.154,    -3.0E-35,
        0.0178,      0.0776,              0.0776,   -2.38E-35,
       -0.0328,       -0.221,              -0.221,     1.1E-35,
        0.0104,      0.0289,              0.0289,     2.2E-36,

Eigenvalue L[3]:  0.0455
Checking the properties of eigenvector V3 (AV - BVL should be a zero vector):
Eigenvector V3  AV = A * V3  BVL = B * V3 * L[3]     AV - BVL
        0.0197,       0.162,               0.162,    -1.0E-35,
       0.00953,      0.0497,              0.0497,     3.7E-36,
        0.0386,       0.368,               0.368,    -2.2E-35,
       -0.0275,      -0.272,              -0.272,     2.7E-35,
       -0.0181,      -0.175,              -0.175,     1.0E-35,
       -0.0400,      -0.425,              -0.425,     1.4E-35,

Eigenvalue L[4]:  0.0952
Checking the properties of eigenvector V4 (AV - BVL should be a zero vector):
Eigenvector V4  AV = A * V4  BVL = B * V4 * L[4]     AV - BVL
       -0.0160,      -0.281,              -0.281,       6E-36,
       -0.0293,      -0.576,              -0.576,       7E-36,
        0.0196,       0.500,               0.500,    -4.9E-35,
        0.0143,       0.313,               0.313,    -1.8E-35,
        0.0481,        1.03,                1.03,      -4E-35,
       -0.0216,      -0.473,              -0.473,      -5E-36,

Eigenvalue L[5]:  0.850
Checking the properties of eigenvector V5 (AV - BVL should be a zero vector):
Eigenvector V5  AV = A * V5  BVL = B * V5 * L[5]     AV - BVL
        0.0172,        6.83,                6.83,    -1.5E-34,
        0.0215,        7.00,                7.00,    -2.1E-34,
        0.0292,        8.30,                8.30,    -2.5E-34,
        0.0282,        8.79,                8.79,    -2.2E-34,
       0.00121,        3.07,                3.07,      -9E-35,
        0.0157,        5.63,                5.63,    -1.9E-34,
```

6.8.7.2 Example for a hermitian matrix: eigenvalues and eigenvectors

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableRandomSAPosDefA6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A (hermitian)")
A (hermitian):
   91.0 + 0j, 12.0 - 3.60j, 22.0 - 7.40j, 14.0 + 1.10j,
12.0 + 3.60j,    77.0 + 0j, 2.50 - 2.00j, 3.40 - 7.60j,
22.0 + 7.40j, 2.50 + 2.00j,    91.0 + 0j, 17.0 + 3.70j,
14.0 - 1.10j, 3.40 + 7.60j, 17.0 - 3.70j,    74.0 + 0j,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableRandomSAPosDefB6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B (hermitian positive definite)")
B (hermitian positive definite):
   85.0 + 0j, 7.30 + 1.40j, 9.20 + 2.30j, 3.10 + 1.80j,
7.30 - 1.40j,    73.0 + 0j, 19.0 - 1.80j, 3.70 + 2.00j,
9.20 - 2.30j, 19.0 + 1.80j,    89.0 + 0j, 13.0 - 8.30j,
3.10 - 1.80j, 3.70 - 2.00j, 13.0 + 8.30j,    84.0 + 0j,

>>> Query = "Eval, Evec"
>>> Res = A.eigen_GeneralizedSelfAdjointEigenSolver2(Query, B)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i        L[i]
0: 0.731 + 0j,
1: 0.779 + 0j,
```

(continues on next page)

(continued from previous page)

```
2:  1.15 + 0j,
3:  1.47 + 0j,

>>> V =  Res["Evec"]; mt = "Matrix V of eigenvectors (V0, ... , V" + str(V.cols-1) + ")"
>>> V.show(mt, coltitles = ["V#"] * (V.cols))
Matrix V of eigenvectors (V0, ... , V3):
              V0                  V1                  V2                  V3
0.0609 + 0.00995j, -0.0160 - 0.000497j,  -0.0833 + 0.00552j, 0.0307 - 0.00215j,
-0.0321 - 0.0508j,   0.00344 - 0.0182j,   -0.0405 - 0.0520j, -0.0529 - 0.0594j,
-0.0290 - 0.0350j,    0.0579 - 0.0201j, -0.00730 + 0.00466j,  0.0488 + 0.0641j,
-0.0469 + 0.0140j,   -0.0206 + 0.0834j,   -0.0120 - 0.0120j,  0.0395 - 0.0268j,

>>> # Vinv = V^-1 = V.H
>>> Vinv = V.eigen_inverse(); Vinv.show(" V^-1")
>>> CheckResult = (A - V * L.D * Vinv).norm()
>>> print("||A - B * V * diag(L) * V^-1|| (should be zero): ", (CheckResult).s())
||A - V * diag(L) * V^-1|| (should be zero):  4.59E-33 + 0j

>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X = A - B * L[i]
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - B * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - B * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - B * L[i]) should be zero):
i        L[i]      Det(A - B * L[i])
0: 0.731 + 0j, -1.97E-29 + 2.21E-32j,
1: 0.779 + 0j,         7.30E-30 + 0j,
2:  1.15 + 0j,  3.23E-30 - 1.27E-31j,
3:  1.47 + 0j,  2.56E-28 + 1.89E-31j,


>>> for i in range(V.rows):
>>>     AV = A * V.col(i); BVL = (B * L[i]) * V.col(i); X = AV - BVL
>>>     Li = "L[" + str(i) + "]"; Vi = "V" + str(i)
>>>     print("Eigenvalue " + Li + ": ", L[i].s())
>>>     Result = V.col(i).concat_horizontal(AV).concat_horizontal(BVL).concat_horizontal(X)
>>>     mt = "Checking the properties of eigenvector " + Vi + " (AV - BVL should be a zero vector)"
>>>     ct = ["Eigenvector " + Vi, "AV = A * " + Vi, "BVL = B * " + Vi + " * " + Li, "   AV - BVL"]
>>>     Result.show(mt, coltitles = ct)

Eigenvalue L[0]:  0.731 + 0j
Checking the properties of eigenvector V0 (AV - BVL should be a zero vector):
   Eigenvector V0       AV = A * V0  BVL = B * V0 * L[0]              AV - BVL
0.0609 + 0.00995j, 3.40 + 1.10E-35j,    3.40 + 2.10E-36j, -1.30E-34 + 8.90E-36j,
-0.0321 - 0.0508j,    -1.98 - 3.20j,       -1.98 - 3.20j,  8.00E-35 + 1.00E-34j,
-0.0290 - 0.0350j,    -2.20 - 2.64j,       -2.20 - 2.64j,  1.10E-34 + 1.50E-34j,
-0.0469 + 0.0140j,   -2.95 + 0.201j,      -2.95 + 0.201j,  1.50E-34 + 1.30E-35j,

Eigenvalue L[1]:  0.779 + 0j
Checking the properties of eigenvector V1 (AV - BVL should be a zero vector):
     Eigenvector V1     AV = A * V1  BVL = B * V1 * L[1]              AV - BVL
-0.0160 - 0.000497j,    -0.738 + 0j,        -0.738 + 0j,         2.40E-35 + 0j,
  0.00344 - 0.0182j,  0.743 - 1.19j,       0.743 - 1.19j, -1.10E-35 + 2.00E-35j,
   0.0579 - 0.0201j,  4.30 - 0.652j,       4.30 - 0.652j, -1.00E-34 + 5.00E-36j,
  -0.0206 + 0.0834j, -0.688 + 5.59j,      -0.688 + 5.59j,  2.00E-36 - 9.00E-35j,

Eigenvalue L[2]:  1.15 + 0j
Checking the properties of eigenvector V2 (AV - BVL should be a zero vector):
     Eigenvector V2        AV = A * V2  BVL = B * V2 * L[2]              AV - BVL
 -0.0833 + 0.00552j, -8.53 - 1.00E-36j,   -8.53 + 1.80E-36j,  1.00E-35 - 2.80E-36j,
  -0.0405 - 0.0520j,     -4.28 - 4.16j,       -4.28 - 4.16j, -4.00E-35 - 5.00E-35j,
```

(continues on next page)

(continued from previous page)

```
-0.00730 + 0.00466j,    -2.70 - 0.530j,      -2.70 - 0.530j,  2.00E-35 + 6.20E-35j,
  -0.0120 - 0.0120j,     -1.90 - 1.09j,       -1.90 - 1.09j,         4.00E-35 + 0j,

Eigenvalue L[3]:  1.47 + 0j
Checking the properties of eigenvector V3 (AV - BVL should be a zero vector):
   Eigenvector V3       AV = A * V3  BVL = B * V3 * L[3]               AV - BVL
0.0307 - 0.00215j, 4.08 - 5.00E-36j,    4.08 + 2.00E-36j, -1.60E-34 - 7.00E-36j,
-0.0529 - 0.0594j,    -3.51 - 4.82j,       -3.51 - 4.82j,  1.00E-35 + 9.00E-35j,
 0.0488 + 0.0641j,     5.89 + 5.45j,        5.89 + 5.45j, -2.40E-34 - 1.30E-34j,
 0.0395 - 0.0268j,     4.69 - 1.74j,        4.69 - 1.74j, -1.90E-34 + 6.00E-35j,
```

6.8.8 Tridiagonalization

`ctx.mat_tridiagonalization`(*matA*, *Query*)

Returns the tridiagonal decomposition of a selfadjoint matrix.

Writing `matA.Tridiag(Query)` has the same effect

See also Eigen [239], Wikipedia [1420], Wikipedia [1410], Wikipedia [1405], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

Q
Returns the unitary matrix Q in the decomposition.

T
Returns an expression of the tridiagonal matrix T in the decomposition..

packed
Returns the internal representation of the decomposition.

hcoeff
Returns the Householder coefficients..

diag
Returns the diagonal of the tridiagonal matrix T in the decomposition.

subdiag
Returns the subdiagonal of the tridiagonal matrix T in the decomposition.

This class performs a tridiagonal decomposition of a selfadjoint matrix A such that: $A = QTQ^*$ where Q is unitary and T a real symmetric tridiagonal matrix. A tridiagonal matrix is a matrix which has nonzero elements only on the main diagonal and the first diagonal below and above it. The Hessenberg decomposition of a selfadjoint matrix is in fact a tridiagonal decomposition. This class is used in SelfAdjointEigenSolver to compute the eigenvalues and eigenvectors of a selfadjoint matrix.

6.8.8.1 Example for a real matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAA6x6", ""); A.show("A")
A:
44.9, 25.5, 50.0, 47.9, 26.4, 62.0,
25.5, 24.3, 49.1, 95.0, 29.0, 46.6,
50.0, 49.1, 55.5, 84.0, 44.4, 26.7,
47.9, 95.0, 84.0, 64.5, 39.5, 87.5,
26.4, 29.0, 44.4, 39.5, 39.8, 12.3,
62.0, 46.6, 26.7, 87.5, 12.3, 85.0,

>>> Query = "Q, T, Packed, Hcoeff, Diag, Subdiag"
>>> Res = A.eigen_tridiag2(Query)

>>> Q = Res["Q"]; Q.show("Q")
Q:
1,       0,        0,      0,      0,      0,
0, -0.255,    0.648,  0.267,  0.628,  0.220,
0, -0.500,   -0.254,  0.734, -0.127, -0.361,
0, -0.479,    0.538, -0.386, -0.470, -0.332,
0, -0.264, -0.00888,  0.149, -0.455,  0.837,
0, -0.620,   -0.474, -0.468,  0.401,  0.101,
```

(continues on next page)

(continued from previous page)

```
>>> T = Res["T"]; T.show("T")
T:
 44.9, -99.9,     0,     0,     0,     0,
-99.9,   255, -73.7,     0,     0,     0,
    0, -73.7,  11.5,  21.7,     0,     0,
    0,     0,  21.7,  20.2, -49.4,     0,
    0,     0,     0, -49.4, -18.5, -24.2,
    0,     0,     0,     0, -24.2, 0.966,

>>> Res["Packed"].show("Packed")
Packed:
 44.9,   25.5,    50.0,   47.9,  26.4,  62.0,
-99.9,    255,    49.1,   95.0,  29.0,  46.6,
0.399,  -73.7,    11.5,   84.0,  44.4,  26.7,
0.382, -0.192,    21.7,   20.2,  39.5,  87.5,
0.210, 0.0961, -0.0240,  -49.4, -18.5,  12.3,
0.494,  0.526,   0.680, -0.524, -24.2, 0.966,

>>> # Check that Q is unitary, i.e. that QQ^H = I
>>> CheckResult = (Q.I - Q * Q.H).norm()
>>> print("||I - Q * Q.H|| (should be zero): ", CheckResult.s())
    ||I - Q * Q.H|| (should be zero):  1.44E-35

>>> # Check the defining property of the decomposition, i.e. A = QTQ^H
>>> CheckResult = (A - Q * T * Q.H).norm()
>>> print("||A - Q * T * Q^H|| (should be zero): ", CheckResult.s())
    ||A - Q * T * Q^H|| (should be zero):  3.96E-33

>>> diag = Res["Diag"]; subdiag = Res["Subdiag"]
>>> Query = "eval, evec"
>>> evaltridiag = diag.eigen_SelfAdjointEigenSystemFromTridiag2(Query, subdiag)

>>> L = evaltridiag["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues of T", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues of T:
i   L[i]
0: -61.7,
1: -27.7,
2:  4.34,
3:  25.8,
4:  62.5,
5:   311,

>>> V =  evaltridiag["Evec"]; mt = "Matrix V of eigenvectors of T (V0, ... , V" + str(V.cols-1) + ")"
>>> V.show(mt, coltitles = ["V#"] * (V.cols))
Matrix V of eigenvectors of T (V0, ... , V5):
    V0      V1        V2      V3      V4        V5
0.0719, -0.497,   -0.271,  0.730, -0.157,    0.343,
0.0767, -0.361,   -0.110,  0.139, 0.0275,   -0.912,
 0.232, -0.711, -0.00659, -0.555,  0.284,    0.226,
-0.521, 0.0579,   -0.370,  0.108,  0.759,   0.0173,
-0.760, -0.257,   -0.122, -0.257, -0.525, -0.00262,
-0.293, -0.216,    0.873,  0.249,  0.206, 0.000204,

>>> CheckResult = (T - V * L.D * V.T).norm()
>>> print("||T - V * diag(L) * V^T|| (should be zero): ", (CheckResult).s())
    ||T - V * diag(L) * V^T|| (should be zero):  6.70E-33

>>> XA = +A; AD = XA.diagonal(); DetA = L * 0
>>> XT = +T; TD = XT.diagonal(); DetT = L * 0

>>> for i in range(A.rows):
>>>     XA.set_diagonal(0, AD - L[i]); DetA[i] = XA.eigen_det()
>>>     XT.set_diagonal(0, TD - L[i]); DetT[i] = XT.eigen_det()

>>> Result = L.concat_horizontal(DetA).concat_horizontal(DetT)
```

(continues on next page)

(continued from previous page)

```
>>> mt = "Checking the Eigenvalues: Det(A - I * L[i]) and Det(T - I * L[i]) should be zero"
>>> ct = ["L[i]", "Det(A - I * L[i])", "Det(T - I * L[i])"]
>>> rt = ["i"] + [x for x in range(L.rows)]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues: Det(A - I * L[i]) and Det(T - I * L[i]) should be zero:
i   L[i]  Det(A - I * L[i])  Det(T - I * L[i])
0: -61.7,          8.41E-24,          8.69E-24,
1: -27.7,         -4.36E-25,         -5.06E-25,
2:  4.34,          3.49E-26,          4.32E-26,
3:  25.8,          4.92E-25,          2.75E-25,
4:  62.5,         -1.13E-23,         -1.08E-23,
5:   311,         -6.62E-21,         -8.92E-21,
```

6.8.8.2 Example for a complex matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableRandomSAA6x6", "")
>>> A = A.top_left_corner(5,5); A.show("A")
A:
   80.0 + 0j, 50.0 - 23.0j, 85.0 + 5.00j, 36.0 - 4.30j, 16.0 - 18.0j,
50.0 + 23.0j,    30.0 + 0j, 43.0 + 9.50j, 27.0 + 11.0j, 50.0 - 24.0j,
85.0 - 5.00j, 43.0 - 9.50j,    85.0 + 0j, 55.0 - 7.00j, 34.0 + 8.50j,
36.0 + 4.30j, 27.0 - 11.0j, 55.0 + 7.00j,    23.0 + 0j, 49.0 + 40.0j,
16.0 + 18.0j, 50.0 + 24.0j, 34.0 - 8.50j, 49.0 - 40.0j,    85.0 + 0j,

>>> Query = "Q, T, Packed, Hcoeff, Diag, Subdiag"
>>> Res = A.eigen_tridiag2(Query)

>>> Q = Res["Q"]; Q.show("Q")
Q:
1.00 + 0j,           0 + 0j,           0 + 0j,            0 + 0j,           0 + 0j,
   0 + 0j,  -0.453 - 0.208j, -0.0465 - 0.186j,    0.480 - 0.389j,   0.144 + 0.558j,
   0 + 0j, -0.770 + 0.0453j,  -0.158 + 0.243j, 0.00105 + 0.0880j,  -0.417 - 0.372j,
   0 + 0j, -0.326 - 0.0390j,   0.258 + 0.207j,   -0.228 + 0.640j,   0.456 + 0.336j,
   0 + 0j,  -0.145 - 0.163j,   0.864 - 0.153j,   -0.224 - 0.314j, -0.0891 - 0.162j,

>>> T = Res["T"]; T.show("T")
T:
80.0 + 0j,  -110 + 0j,     0 + 0j,     0 + 0j,     0 + 0j,
-110 + 0j,   153 + 0j, -59.2 + 0j,     0 + 0j,     0 + 0j,
   0 + 0j, -59.2 + 0j,   104 + 0j,  24.4 + 0j,     0 + 0j,
   0 + 0j,     0 + 0j,  24.4 + 0j, -30.1 + 0j, -19.0 + 0j,
   0 + 0j,     0 + 0j,     0 + 0j, -19.0 + 0j, -4.47 + 0j,

>>> Res["Packed"].show("Packed")
Packed:
       80.0 + 0j,    50.0 - 23.0j,   85.0 + 5.00j, 36.0 - 4.30j, 16.0 - 18.0j,
       -110 + 0j,         153 + 0j,   43.0 + 9.50j, 27.0 + 11.0j, 50.0 - 24.0j,
  0.515 - 0.105j,       -59.2 + 0j,        104 + 0j, 55.0 - 7.00j, 34.0 + 8.50j,
0.224 - 0.00528j, -0.161 - 0.271j,       24.4 + 0j,   -30.1 + 0j, 49.0 + 40.0j,
 0.114 + 0.0960j, -0.732 - 0.105j, 0.251 + 0.187j,   -19.0 + 0j,   -4.47 + 0j,

>>> # Check that Q is unitary, i.e. that QQ^H = I
>>> CheckResult = (Q.I - Q * Q.H).norm()
>>> print("||I - Q * Q.H|| (should be zero): ", CheckResult.s())
    ||I - Q * Q.H|| (should be zero):  1.46E-35 + 0j

>>> # Check the defining property of the decomposition, i.e. A = QTQ^H
>>> CheckResult = (A - Q * T * Q.H).norm()
>>> print("||A - Q * T * Q^H|| (should be zero): ", CheckResult.s())
    ||A - Q * T * Q^H|| (should be zero):  2.51E-33 + 0j

>>> diag = Res["Diag"]; subdiag = Res["Subdiag"]; T = Res["T"]
```

(continues on next page)

(continued from previous page)

```
>>> Query = "eval, evec"
>>> evaltridiag = diag.eigen_SelfAdjointEigenSystemFromTridiag2(Query, subdiag)

>>> L = evaltridiag["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues of T", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues of T:
i        L[i]
0: -44.4 + 0j,
1: -10.3 + 0j,
2:  5.28 + 0j,
3:   103 + 0j,
4:   250 + 0j,

>>> V =  evaltridiag["Evec"]; mt = "Matrix V of eigenvectors of T (V0, ... , V" + str(V.cols-1) + ")"
>>> V.show(mt, coltitles = ["V#"] * (V.cols))
Matrix V of eigenvectors of T (V0, ... , V4):
          V0            V1            V2            V3             V4
-0.0991 + 0j,  -0.708 + 0j,   0.190 + 0j,  -0.432 + 0j,    0.515 + 0j,
 -0.112 + 0j,  -0.580 + 0j,   0.129 + 0j,  0.0896 + 0j,   -0.792 + 0j,
 -0.189 + 0j,  -0.282 + 0j, -0.0325 + 0j,   0.881 + 0j,    0.327 + 0j,
  0.876 + 0j, -0.0839 + 0j,   0.443 + 0j,   0.166 + 0j,   0.0288 + 0j,
  0.418 + 0j,  -0.274 + 0j,  -0.866 + 0j, -0.0295 + 0j, -0.00216 + 0j,

>>> CheckResult = (T - V * L.D * V.T).norm()
>>> print("||T - V * diag(L) * V^T|| (should be zero): ", (CheckResult).s())
    ||T - V * diag(L) * V^T|| (should be zero):  9.71E-33 + 0j

>>> XA = +A; AD = XA.diagonal(); DetA = L * 0
>>> XT = +T; TD = XT.diagonal(); DetT = L * 0

>>> for i in range(A.rows):
>>>     XA.set_diagonal(0, AD - L[i]); DetA[i] = XA.eigen_det()
>>>     XT.set_diagonal(0, TD - L[i]); DetT[i] = XT.eigen_det()

>>> Result = L.concat_horizontal(DetA).concat_horizontal(DetT)
>>> mt = "Checking the Eigenvalues: Det(A - I * L[i]) and Det(T - I * L[i]) should be zero"
>>> ct = ["L[i]", "Det(A - I * L[i])", "Det(T - I * L[i])"]
>>> rt = ["i"] + [x for x in range(L.rows)]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues: Det(A - I * L[i]) and Det(T - I * L[i]) should be zero:
i        L[i]      Det(A - I * L[i])  Det(T - I * L[i])
0: -44.4 + 0j, -5.09E-26 - 2.12E-27j,    -2.15E-26 + 0j,
1: -10.3 + 0j,  5.47E-27 + 2.47E-28j,     8.81E-28 + 0j,
2:  5.28 + 0j, -6.37E-29 + 1.87E-27j,     9.68E-28 + 0j,
3:   103 + 0j, -5.10E-25 - 1.96E-26j,    -4.86E-25 + 0j,
4:   250 + 0j,  1.12E-23 + 3.03E-26j,     1.19E-23 + 0j,
```

6.9 Eigen decompositions of general square matrices

6.9.1 Matrix balancing and eigenvalue/eigenvector computation

Diagonal scaling to improve eigenvalue accuracy

See also: James *et al.* [378].

Example from Matlab: https://de.mathworks.com/help/matlab/ref/balance.html

Sample code for Eigen in C++ : https://stackoverflow.com/questions/43151853/eigen-balancing-matrix-for-eigenvalue

```
void balance_matrix(const Eigen::MatrixXd &A, Eigen::MatrixXd &Aprime, Eigen::MatrixXd &D) {
    // https://arxiv.org/pdf/1401.5766.pdf (Algorithm #3)
    const int p = 2;
    double beta = 2; // Radix base (2?)
    Aprime = A;
    D = Eigen::MatrixXd::Identity(A.rows(), A.cols());
    bool converged = false;
    do {
        converged = true;
        for (Eigen::Index i = 0; i < A.rows(); ++i) {
            double c = Aprime.col(i).lpNorm<p>();
            double r = Aprime.row(i).lpNorm<p>();
            double s = pow(c, p) + pow(r, p);
            double f = 1;
            while (c < r / beta) {
                c *= beta;
                r /= beta;
                f *= beta;
            }
            while (c >= r*beta) {
                c /= beta;
                r *= beta;
                f /= beta;
            }
            if (pow(c, p) + pow(r, p) < 0.95*s) {
                converged = false;
                D(i, i) *= f;
                Aprime.col(i) *= f;
                Aprime.row(i) /= f;
            }
        }
    } while (!converged);
}
```

6.9.2 Hessenberg Decomposition

`ctx.mat_hessenberg`(*matA*, *Query*)

Reduces a square matrix to Hessenberg form by an orthogonal similarity transformation.

Writing `matA.Hessenberg(Query)` has the same effect

See also Eigen [227], Wikipedia [1409], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

H
A square matrix of the same type and dimension as A, containing the matrix H in the decomposition $A = QHQ^T$.

Q
A square matrix of the same type and dimension as A, containing the matrix Q in the decomposition $A = QHQ^T$.

packedMatrix
A square matrix of the same type and dimension as A. Returns the internal representation of the decomposition.

hcoeff
A square matrix of the same type and dimension as A. Returns the Householder coefficients of the decomposition.

Reduces a square matrix to Hessenberg form by an orthogonal similarity transformation.

In the real case, the Hessenberg decomposition consists of an orthogonal matrix Q and a Hessenberg matrix H such that $A = QHQ^T = Q^THQ$. An orthogonal matrix is a matrix whose inverse equals its transpose: $Q^{-1} = Q^T$., and $QQ^T = Q^TQ = I$

The Hessenberg decomposition of a complex matrix is $A = QHQ^*$ with Q unitary, that is, $Q^{-1} = Q^*$, and $QQ^* = Q^*Q = I$.

A Hessenberg matrix has zeros below the subdiagonal, so it is almost upper triangular.

6.9.2.1 Example for a real matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> Query = "H, Q, Hcoeff, Packed"
>>> Res = A.eigen_hessenberg2(Query)

>>> Res["H"].show("H")
H:
   48, -59.6, -10.3,  10.5,  12.3, -5.09,
-59.7,  96.1,  35.2, -1.55, -7.54, -10.7,
    0,  94.1,  26.9,  6.92, -19.6, -27.1,
    0,     0, -24.9, -7.98,  45.0,  9.81,
    0,     0,     0, -3.83,  2.49,  9.07,
    0,     0,     0,     0, -1.38, -2.76,

>>> Res["Q"].show("Q")
Q:
```

(continues on next page)

(continued from previous page)

```
1,      0,      0,       0,      0,      0,
0, -0.770,  0.524, -0.0577,  0.320,  0.162,
0, -0.452, -0.161,   0.398, -0.417, -0.662,
0, -0.119, -0.490,  -0.337,  0.671, -0.426,
0, -0.385, -0.439,  -0.575, -0.473,  0.322,
0, -0.201, -0.516,   0.628,  0.220,  0.501,

>>> Res["Hcoeff"].show("Hcoeff")
Hcoeff:
1.77,
1.29,
1.50,
1.75,
   0,

>>> Res["Packed"].show("Packed")
Packed:
    48, -59.6,  -10.3,   10.5,  12.3, -5.09,
 -59.7,  96.1,   35.2,  -1.55, -7.54, -10.7,
 0.255,  94.1,   26.9,   6.92, -19.6, -27.1,
0.0671, 0.406,  -24.9,  -7.98,  45.0,  9.81,
 0.218, 0.427,  0.492,  -3.83,  2.49,  9.07,
 0.113, 0.444, -0.301, -0.378, -1.38, -2.76,
```

6.9.2.2 Example for a complex matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(5,5); A.show("A")
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j, 5.00 + 14.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j, 7.00 + 3.30j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j, 9.00 + 23.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j, 43.0 + 33.0j,
23.0 + 43.0j, 15.0 + 11.0j, 28.0 + 19.0j, 35.0 + 32.0j, 41.0 + 14.0j,

>>> Query = "H, Q, Hcoeff, Packed"
>>> Res = A.eigen_hessenberg2(Query)

>>> Res["H"].show("H")
H:
45.0 + 7.50j, 18.1 - 51.3j, -17.5 + 4.87j, -25.8 + 10.1j,  18.1 + 13.2j,
  -86.1 + 0j, 92.5 + 84.4j,  14.6 - 49.3j,  4.03 - 10.0j, -2.00 - 20.7j,
      0 + 0j,   -64.4 + 0j,  34.6 + 3.78j,  18.8 - 10.4j,  15.2 - 24.8j,
      0 + 0j,       0 + 0j,    -33.0 + 0j,  19.9 + 19.9j,  3.46 + 20.3j,
      0 + 0j,       0 + 0j,        0 + 0j,     17.5 + 0j, 0.963 - 25.1j,

>>> Res["Q"].show("Q")
Q:
1.00 + 0j,          0 + 0j,           0 + 0j,           0 + 0j,           0 + 0j,
   0 + 0j, -0.151 - 0.337j,   0.179 + 0.305j,   0.728 + 0.398j,   0.129 + 0.183j,
   0 + 0j, -0.372 - 0.105j,  -0.617 + 0.545j, -0.0540 - 0.336j,  -0.105 + 0.213j,
   0 + 0j, -0.395 - 0.488j, -0.0150 - 0.157j,  -0.364 + 0.107j,   0.636 - 0.181j,
   0 + 0j, -0.267 - 0.499j,  0.413 + 0.0291j, -0.213 - 0.0808j, -0.668 - 0.0986j,

>>> Res["Hcoeff"].show("Hcoeff")
Hcoeff:
1.15 - 0.337j,
1.68 + 0.447j,
1.66 - 0.249j,
1.96 - 0.286j,

>>> Res["Packed"].show("Packed")
```

(continues on next page)

(continued from previous page)

```
Packed:
    45.0 + 7.50j,      18.1 - 51.3j,  -17.5 + 4.87j, -25.8 + 10.1j,  18.1 + 13.2j,
      -86.1 + 0j,      92.5 + 84.4j,   14.6 - 49.3j,  4.03 - 10.0j, -2.00 - 20.7j,
0.322 - 0.00339j,        -64.4 + 0j,   34.6 + 3.78j,  18.8 - 10.4j,  15.2 - 24.8j,
  0.430 + 0.298j,  -0.0503 + 0.190j,     -33.0 + 0j,  19.9 + 19.9j,  3.46 + 20.3j,
  0.331 + 0.337j, -0.274 + 0.00564j, 0.288 + 0.308j,     17.5 + 0j, 0.963 - 25.1j,
```

6.9.3 Schur Decomposition

`ctx.mat_schur`(*matA*, *Query*)

Performs a Schur decomposition of a square matrix.

Writing `matA.Schur(Query)` has the same effect

See also Eigen [236], Eigen [222], Wikipedia [1437], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

U
A general matrix of the same type and dimension as A, containing the matrix U in the decomposition.

T
A general matrix of the same type and dimension as A, containing the matrix T in the decomposition.

REAL:

Given a real square matrix A, this class computes the real Schur decomposition: $A = UTU^T$ where U is a real orthogonal matrix and T is a real quasi-triangular matrix. An orthogonal matrix is a matrix whose inverse is equal to its transpose, $U^{-1} = U^T$. A quasi-triangular matrix is a block-triangular matrix whose diagonal consists of 1-by-1 blocks and 2-by-2 blocks with complex eigenvalues. The eigenvalues of the blocks on the diagonal of T are the same as the eigenvalues of the matrix A, and thus the real Schur decomposition is used in EigenSolver to compute the eigendecomposition of a matrix.

COMPLEX:

Given a real or complex square matrix A, this class computes the Schur decomposition: where U is a unitary complex matrix, and T is a complex upper triangular matrix. The diagonal of the matrix T corresponds to the eigenvalues of the matrix A.

6.9.3.1 Example for a real matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> Query = "U, T"
>>> Res = A.eigen_schur2(Query)

>>> Res["U"].show("U")
U:
0.448, -0.245,   0.546,  -0.582, -0.317, 0.0355,
0.283,  0.578,   0.572,   0.481, 0.0718,  0.147,
0.442,  0.344,  -0.230,  -0.241,  0.328, -0.684,
0.312, -0.358, -0.0980,   0.565, -0.534, -0.400,
0.463, -0.512, -0.0393,   0.209,  0.631,  0.283,
0.460,  0.310,  -0.557, -0.0919, -0.323,  0.519,

>>> Res["T"].show("T")
T:
158,  27.1,  34.3,   32.5, -0.516,  19.6,
```

(continues on next page)

(continued from previous page)

```
  0, -20.8, -3.79, -0.982,  -31.1,  2.71,
  0,     0,  21.2,  -47.3,   1.06, -21.6,
  0,     0,     0,  -5.80,   25.8,  6.25,
  0,     0,     0,  -13.8,   12.4, -9.59,
  0,     0,     0,      0,      0, -2.45,
```

6.9.3.2 Example for a complex matrix

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(5,5); A.show("A")
A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j, 5.00 + 14.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j, 7.00 + 3.30j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j, 9.00 + 23.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j, 43.0 + 33.0j,
23.0 + 43.0j, 15.0 + 11.0j, 28.0 + 19.0j, 35.0 + 32.0j, 41.0 + 14.0j,

>>> Query = "U, T"
>>> Res = A.eigen_schur2(Query)

>>> Res["U"].show("U")
U:
 0.338 + 0.106j, -0.0858 - 0.301j,   0.235 - 0.292j, -0.271 - 0.686j, -0.0756 - 0.294j,
 0.409 + 0.152j,  -0.572 + 0.142j,  -0.532 + 0.134j, 0.357 - 0.0924j,  0.0672 - 0.142j,
0.522 + 0.0826j, -0.0964 + 0.376j,   0.507 - 0.348j,  0.129 + 0.234j,  0.0787 + 0.338j,
 0.443 + 0.121j,   0.286 - 0.234j,  -0.153 + 0.362j, -0.381 + 0.116j,   0.559 + 0.165j,
0.438 + 0.0389j,   0.464 - 0.229j, -0.149 + 0.0612j,  0.210 + 0.215j, -0.644 - 0.0920j,

>>> Res["T"].show("T")
T:
125 + 131j, 6.55 - 27.8j, -1.71 - 17.5j,  4.81 - 19.6j,  14.2 + 15.8j,
    0 + 0j, 43.0 + 29.8j, 0.609 + 17.6j,  13.8 - 10.4j, -5.47 - 18.4j,
    0 + 0j,       0 + 0j,  10.5 - 45.0j, -11.8 - 20.4j, -5.32 + 18.9j,
    0 + 0j,       0 + 0j,        0 + 0j,  21.7 - 13.7j,  4.33 + 21.6j,
    0 + 0j,       0 + 0j,        0 + 0j,        0 + 0j, -7.36 - 11.2j,
```

6.9.4 Eigensystem of a general square matrix: only eigenvalues

`ctx.mat_eigen_values`(*matA*, *Query*)

Returns the eigendecomposition of a general square matrix *matA* $= A$.

Writing `matA.EigenValues(Query)` has the same effect

See also Eigen [221], Wikipedia [1403], Wikipedia [1405], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

eval
Returns the eigenvalues of given matrix.

evec
Returns the eigenvectors of given matrix.

Computes eigenvalues and eigenvectors of general matrices. The eigenvalues and eigenvectors of a matrix A are scalars λ and vectors v such that $Av = \lambda v$. If D is a diagonal matrix with the eigenvalues on the diagonal, and V is a matrix with the eigenvectors as its columns, then $AV = VD$. The matrix V is almost always invertible, in which case we have $A = VDV^{-1}$. This is called the eigendecomposition. The eigenvalues and eigenvectors of a matrix may be complex, even when the matrix is real. However, we can choose real matrices V and D satisfying $AV = VD$, just like the eigendecomposition, if the matrix D is not required to be diagonal, but if it is allowed to have blocks of the form

$$\begin{bmatrix} \mathbf{u} & \mathbf{v} \\ -\mathbf{v} & \mathbf{u} \end{bmatrix}$$

(where u and v are real numbers) on the diagonal. These blocks correspond to complex eigenvalue pairs $u \pm iv$. We call this variant of the eigendecomposition the pseudo-eigendecomposition.

Call the function compute() to compute the eigenvalues and eigenvectors of a given matrix. Alternatively, you can use the EigenSolver(const MatrixType, bool) constructor which computes the eigenvalues and eigenvectors at construction time. Once the eigenvalue and eigenvectors are computed, they can be retrieved with the eigenvalues() and eigenvectors() functions. The pseudoEigenvalueMatrix() and pseudoEigenvectors() methods allow the construction of the pseudo-eigendecomposition.

The matrix is first reduced to real Schur form using the RealSchur (or Complex) class. The Schur decomposition is then used to compute the eigenvalues and eigenvectors. The cost of the computation is dominated by the cost of the Schur decomposition, which is very approximately 25n3 (where n is the size of the matrix) if computeEigenvectors is true, and 10n3 if computeEigenvectors is false. This method reuses of the allocated data in the EigenSolver object.

6.9.4.1 Example for a real matrix: only eigenvalues

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> Query = "Eval"
>>> Res = A.eigen_EigenValues2(Query)
```

(continues on next page)

(continued from previous page)

```
>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i          L[i]
0:     158 + 0j,
1:   -20.8 + 0j,
2:    21.2 + 0j,
3: 3.32 + 16.6j,
4: 3.32 - 16.6j,
5:   -2.45 + 0j,

>>> X = A.cplx(); #X.show("X")
>>> AD = X.diagonal(); #AD.show("AD")
>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X.set_diagonal(0, AD - L[i])
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - I * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - I * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - I * L[i]) should be zero):
i          L[i]      Det(A - I * L[i])
0:     158 + 0j,        -5.37E-23 + 0j,
1:   -20.8 + 0j,         7.82E-26 + 0j,
2:    21.2 + 0j,        -5.20E-26 + 0j,
3: 3.32 + 16.6j, -3.07E-26 - 3.37E-26j,
4: 3.32 - 16.6j, -3.07E-26 + 3.37E-26j,
5:   -2.45 + 0j,         2.78E-28 + 0j,
```

6.9.4.2 Example for a complex matrix: only eigenvalues

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", ""); A.show("A")

A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j, 5.00 + 14.0j, 13.0 + 5.00j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j, 7.00 + 3.30j, 35.0 + 25.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j, 9.00 + 23.0j, 5.50 + 35.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j, 43.0 + 33.0j, 24.0 + 19.0j,
23.0 + 43.0j, 15.0 + 11.0j, 28.0 + 19.0j, 35.0 + 32.0j, 41.0 + 14.0j, 5.00 + 48.0j,
33.0 + 49.0j, 42.0 + 48.0j, 47.0 + 32.0j, 34.0 + 25.0j, 46.0 + 31.0j, 4.70 + 23.0j,

>>> Query = "Eval"
>>> Res = A.eigen_EigenSystem2(Query)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i            L[i]
0: -21.5 + 0.565j,
1:   28.1 + 2.54j,
2:  -11.3 - 39.2j,
3:   9.68 - 43.2j,
4:   42.2 + 28.5j,
5:     150 + 164j,

>>> X = +A; #X.show("X")
>>> AD = X.diagonal(); #AD.show("AD")
>>> Det = L * 0 # creates a zero vector of the same size and type as L
```

(continues on next page)

(continued from previous page)

```
>>> for i in range(A.rows):
>>>     X.set_diagonal(0, AD - L[i])
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - I * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - I * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - I * L[i]) should be zero):
i            L[i]      Det(A - I * L[i])
0: -21.5 + 0.565j,  8.89E-26 - 9.48E-26j,
1:   28.1 + 2.54j, -4.38E-27 + 1.28E-25j,
2:  -11.3 - 39.2j,  1.02E-25 - 3.06E-25j,
3:   9.68 - 43.2j, -2.96E-25 - 6.70E-25j,
4:   42.2 + 28.5j,  1.62E-24 - 1.31E-24j,
5:     150 + 164j,  2.79E-21 - 1.17E-21j,
```

6.9.5 Eigensystem of a general square matrix: eigenvalues and eigenvectors

`ctx.`**`mat_eigen_system`**(*matA*, *Query*)

Returns the eigendecomposition of a general square matrix *matA* $= A$.

Writing `matA.EigenSystem(Query)` has the same effect

See also Eigen [221], Wikipedia [1403], Wikipedia [1405], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

eval
Returns the eigenvalues of given matrix.

evec
Returns the eigenvectors of given matrix.

6.9.5.1 Example for a real matrix: eigenvalues and eigenvectors

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", ""); A.show("A")

A:
  48,   43,   31,   19,   14,   24,
  46,   10,   20,  4.6,   14,   10,
  27,   39,   13,   34,   29,   37,
 7.1,   42,   15,  2.8,   35,   23,
  23,   50,   42, 0.44,   42,   23,
  12,   50,  1.2,   46,   36,   47,

>>> Query = "Eval, Evec"
>>> Res = A.eigen_EigenSystem2(Query)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i          L[i]
0:     158 + 0j,
1:   -20.8 + 0j,
2:    21.2 + 0j,
3: 3.32 + 16.6j,
4: 3.32 - 16.6j,
5:   -2.45 + 0j,

>>> V =  Res["Evec"]; mt = "Matrix V of eigenvectors (V0, ... , V" + str(V.cols-1) + ")"
>>> V.show(mt, coltitles = ["V#"] * (V.cols))
        V0           V1            V2                V3                V4            V5
0.448 + 0j, -0.309 + 0j,   0.450 + 0j, -0.0287 + 0.197j, -0.0287 - 0.197j, -0.0395 + 0j,
0.283 + 0j,  0.529 + 0j,   0.440 + 0j,   0.136 + 0.299j,   0.136 - 0.299j,  -0.192 + 0j,
0.442 + 0j,  0.274 + 0j,  -0.353 + 0j,  -0.260 - 0.288j,  -0.260 + 0.288j,  0.0958 + 0j,
0.312 + 0j, -0.401 + 0j,  -0.134 + 0j,  0.245 - 0.0527j,  0.245 + 0.0527j,  -0.451 + 0j,
0.463 + 0j, -0.576 + 0j, -0.0979 + 0j,   0.168 + 0.281j,   0.168 - 0.281j,  -0.274 + 0j,
0.460 + 0j,  0.238 + 0j,  -0.671 + 0j,  -0.282 - 0.672j,  -0.282 + 0.672j,   0.821 + 0j,

>>> AC = A.cplx()
>>> CheckResult = (AC - V * L.D * V.eigen_inverse()).norm().real
>>> print("||A - V * diag(L) * V^-1|| (should be zero): ", (CheckResult).s())
    ||A - V * diag(L) * V^-1|| (should be zero):  4.22E-33
```

(continues on next page)

(continued from previous page)

```
>>> X = A.cplx(); #X.show("X")
>>> AD = X.diagonal(); #AD.show("AD")
>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X.set_diagonal(0, AD - L[i])
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - I * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - I * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - I * L[i]) should be zero):
i          L[i]      Det(A - I * L[i])
0:     158 + 0j,        -5.37E-23 + 0j,
1:   -20.8 + 0j,         7.82E-26 + 0j,
2:    21.2 + 0j,        -5.20E-26 + 0j,
3: 3.32 + 16.6j, -3.07E-26 - 3.37E-26j,
4: 3.32 - 16.6j, -3.07E-26 + 3.37E-26j,
5:   -2.45 + 0j,         2.78E-28 + 0j,


>>> for i in range(V.rows):
>>>     AV = AC * V.col(i); VL = V.col(i) * L[i]; X = AV - VL
>>>     Li = "L[" + str(i) + "]"; Vi = "V" + str(i)
>>>     print("Eigenvalue " + Li + ": ", L[i].s())
>>>     Result = V.col(i).concat_horizontal(AV).concat_horizontal(VL).concat_horizontal(X)
>>>     mt = "Checking the properties of eigenvector " + Vi + " (AV - VL should be a zero vector)"
>>>     ct = ["Eigenvector " + Vi, "AV = A * " + Vi, "VL = " + Vi + " * " + Li, "AV - VL"]
>>>     Result.show(mt, coltitles = ct)

Eigenvalue L[0]:  158 + 0j
Checking the properties of eigenvector V0 (AV - VL should be a zero vector):
Eigenvector V0  AV = A * V0  VL = V0 * L[0]         AV - VL
    0.448 + 0j,   70.8 + 0j,      70.8 + 0j,  1.60E-33 + 0j,
    0.283 + 0j,   44.8 + 0j,      44.8 + 0j,         0 + 0j,
    0.442 + 0j,   70.0 + 0j,      70.0 + 0j, -1.00E-34 + 0j,
    0.312 + 0j,   49.4 + 0j,      49.4 + 0j,  2.00E-34 + 0j,
    0.463 + 0j,   73.2 + 0j,      73.2 + 0j,  1.00E-34 + 0j,
    0.460 + 0j,   72.7 + 0j,      72.7 + 0j, -5.00E-34 + 0j,

Eigenvalue L[1]:  -20.8 + 0j
Checking the properties of eigenvector V1 (AV - VL should be a zero vector):
Eigenvector V1  AV = A * V1  VL = V1 * L[1]         AV - VL
   -0.309 + 0j,   6.42 + 0j,      6.42 + 0j, -1.80E-34 + 0j,
    0.529 + 0j,  -11.0 + 0j,     -11.0 + 0j,  6.00E-34 + 0j,
    0.274 + 0j,  -5.69 + 0j,     -5.69 + 0j, -6.00E-35 + 0j,
   -0.401 + 0j,   8.33 + 0j,      8.33 + 0j, -1.00E-34 + 0j,
   -0.576 + 0j,   12.0 + 0j,      12.0 + 0j, -2.00E-34 + 0j,
    0.238 + 0j,  -4.94 + 0j,     -4.94 + 0j, -5.70E-34 + 0j,

Eigenvalue L[2]:  21.2 + 0j
Checking the properties of eigenvector V2 (AV - VL should be a zero vector):
Eigenvector V2  AV = A * V2  VL = V2 * L[2]         AV - VL
    0.450 + 0j,   9.56 + 0j,      9.56 + 0j,  4.00E-35 + 0j,
    0.440 + 0j,   9.35 + 0j,      9.35 + 0j, -7.00E-35 + 0j,
   -0.353 + 0j,  -7.50 + 0j,     -7.50 + 0j,  4.70E-34 + 0j,
   -0.134 + 0j,  -2.85 + 0j,     -2.85 + 0j,  1.90E-34 + 0j,
  -0.0979 + 0j,  -2.08 + 0j,     -2.08 + 0j,  2.10E-34 + 0j,
   -0.671 + 0j,  -14.3 + 0j,     -14.3 + 0j,  6.00E-34 + 0j,

Eigenvalue L[3]:  3.32 + 16.6j
Checking the properties of eigenvector V3 (AV - VL should be a zero vector):
  Eigenvector V3     AV = A * V3  VL = V3 * L[3]               AV - VL
-0.0287 + 0.197j, -3.35 + 0.178j, -3.35 + 0.178j, 5.10E-34 + 2.10E-34j,
  0.136 + 0.299j,  -4.50 + 3.25j,  -4.50 + 3.25j, 5.40E-34 + 2.20E-34j,
```

(continues on next page)

(continued from previous page)

```
 -0.260 - 0.288j,   3.91 - 5.27j,   3.91 - 5.27j, 5.70E-34 + 3.60E-34j,
 0.245 - 0.0527j,   1.69 + 3.88j,   1.69 + 3.88j, 5.00E-35 + 2.00E-34j,
  0.168 + 0.281j,  -4.11 + 3.72j,  -4.11 + 3.72j, 3.30E-34 + 4.30E-34j,
 -0.282 - 0.672j,   10.2 - 6.91j,   10.2 - 6.91j, 4.00E-34 + 5.10E-34j,

Eigenvalue L[4]:  3.32 - 16.6j
Checking the properties of eigenvector V4 (AV - VL should be a zero vector):
  Eigenvector V4     AV = A * V4  VL = V4 * L[4]               AV - VL
-0.0287 - 0.197j, -3.35 - 0.178j, -3.35 - 0.178j, 5.10E-34 - 2.10E-34j,
  0.136 - 0.299j,  -4.50 - 3.25j,  -4.50 - 3.25j, 5.40E-34 - 2.20E-34j,
 -0.260 + 0.288j,   3.91 + 5.27j,   3.91 + 5.27j, 5.70E-34 - 3.60E-34j,
 0.245 + 0.0527j,   1.69 - 3.88j,   1.69 - 3.88j, 5.00E-35 - 2.00E-34j,
  0.168 - 0.281j,  -4.11 - 3.72j,  -4.11 - 3.72j, 3.30E-34 - 4.30E-34j,
 -0.282 + 0.672j,   10.2 + 6.91j,   10.2 + 6.91j, 4.00E-34 - 5.10E-34j,

Eigenvalue L[5]:  -2.45 + 0j
Checking the properties of eigenvector V5 (AV - VL should be a zero vector):
Eigenvector V5  AV = A * V5  VL = V5 * L[5]         AV - VL
  -0.0395 + 0j, 0.0969 + 0j,    0.0969 + 0j, -4.84E-34 + 0j,
   -0.192 + 0j,  0.471 + 0j,     0.471 + 0j, -6.40E-34 + 0j,
   0.0958 + 0j, -0.235 + 0j,    -0.235 + 0j, -6.15E-34 + 0j,
   -0.451 + 0j,   1.10 + 0j,      1.10 + 0j, -2.00E-34 + 0j,
   -0.274 + 0j,  0.671 + 0j,     0.671 + 0j, -2.59E-34 + 0j,
    0.821 + 0j,  -2.01 + 0j,     -2.01 + 0j, -5.50E-34 + 0j,
```

6.9.5.2 Example for a complex matrix: eigenvalues and eigenvectors

```
>>> from xlcalcnet import *
>>> ctx = mp14.dcf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecCplxTableB6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A")

A:
45.0 + 7.50j, 2.90 + 36.0j, 11.0 + 13.0j, 37.0 + 37.0j,
13.0 + 29.0j, 38.0 + 41.0j, 22.0 + 44.0j, 28.0 + 20.0j,
32.0 + 9.00j, 42.0 + 49.0j, 47.0 + 11.0j, 32.0 + 49.0j,
34.0 + 42.0j, 6.00 + 16.0j, 20.0 + 38.0j, 22.0 + 17.0j,

>>> Query = "Eval, Evec"
>>> Res = A.eigen_EigenSystem2(Query)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i           L[i]
0: 4.21 - 0.418j,
1:  39.6 + 8.06j,
2:  3.46 - 47.6j,
3:    105 + 116j,

>>> V =  Res["Evec"]; mt = "Matrix V of eigenvectors (V0, ... , V" + str(V.cols-1) + ")"
>>> V.show(mt, coltitles = ["V#"] * (V.cols))
              V0                V1                V2               V3
-0.0249 + 0.483j, -0.589 - 0.212j, -0.319 + 0.0888j, 0.165 + 0.359j,
 0.0587 + 0.501j,  0.346 + 0.299j,   0.326 + 0.144j, 0.208 + 0.513j,
0.00311 - 0.567j, 0.484 + 0.0474j, -0.645 - 0.0595j, 0.358 + 0.473j,
-0.0210 - 0.435j, -0.148 - 0.374j,   0.534 - 0.242j, 0.154 + 0.402j,

>>> AC = +A
>>> CheckResult = (AC - V * L.D * V.eigen_inverse()).norm().real
>>> print("||A - V * diag(L) * V^-1|| (should be zero): ", (CheckResult).s())
||A - V * diag(L) * V^-1|| (should be zero):  2.59E-33

>>> X = +A; #X.show("X")
```

(continues on next page)

(continued from previous page)

```
>>> AD = X.diagonal(); #AD.show("AD")
>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X.set_diagonal(0, AD - L[i])
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - I * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - I * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - I * L[i]) should be zero):
i           L[i]     Det(A - I * L[i])
0: 4.21 - 0.418j, 6.96E-29 - 3.88E-29j,
1:  39.6 + 8.06j, 3.40E-28 - 3.72E-28j,
2:  3.46 - 47.6j, 3.35E-30 + 9.69E-29j,
3:    105 + 116j, 2.72E-27 - 2.05E-27j,


>>> for i in range(V.rows):
>>>     AV = AC * V.col(i); VL = V.col(i) * L[i]; X = AV - VL
>>>     Li = "L[" + str(i) + "]"; Vi = "V" + str(i)
>>>     print("Eigenvalue " + Li + ": ", L[i].s())
>>>     Result = V.col(i).concat_horizontal(AV).concat_horizontal(VL).concat_horizontal(X)
>>>     mt = "Checking the properties of eigenvector " + Vi + " (AV - VL should be a zero vector)"
>>>     ct = ["Eigenvector " + Vi, "AV = A * " + Vi, "VL = " + Vi + " * " + Li, "AV - VL"]
>>>     Result.show(mt, coltitles = ct)

Eigenvalue L[0]:  4.21 - 0.418j
Checking the properties of eigenvector V0 (AV - VL should be a zero vector):
  Eigenvector V0     AV = A * V0  VL = V0 * L[0]                AV - VL
-0.0249 + 0.483j, 0.0971 + 2.04j, 0.0971 + 2.04j,  1.98E-34 + 2.00E-34j,
 0.0587 + 0.501j,  0.457 + 2.09j,  0.457 + 2.09j, -2.01E-34 + 3.10E-34j,
0.00311 - 0.567j, -0.224 - 2.39j, -0.224 - 2.39j,  7.70E-35 + 4.10E-34j,
-0.0210 - 0.435j, -0.270 - 1.82j, -0.270 - 1.82j,  4.10E-35 + 8.00E-35j,

Eigenvalue L[1]:  39.6 + 8.06j
Checking the properties of eigenvector V1 (AV - VL should be a zero vector):
 Eigenvector V1    AV = A * V1  VL = V1 * L[1]                AV - VL
-0.589 - 0.212j, -21.6 - 13.1j,  -21.6 - 13.1j,  1.20E-33 + 4.00E-34j,
 0.346 + 0.299j,  11.3 + 14.6j,   11.3 + 14.6j, -6.00E-34 - 6.00E-34j,
0.484 + 0.0474j,  18.8 + 5.78j,   18.8 + 5.78j, -6.00E-34 - 5.90E-34j,
-0.148 - 0.374j, -2.83 - 16.0j,  -2.83 - 16.0j,  1.60E-34 + 3.00E-34j,

Eigenvalue L[2]:  3.46 - 47.6j
Checking the properties of eigenvector V2 (AV - VL should be a zero vector):
  Eigenvector V2    AV = A * V2  VL = V2 * L[2]                AV - VL
-0.319 + 0.0888j,  3.13 + 15.5j,   3.13 + 15.5j,         4.00E-34 + 0j,
  0.326 + 0.144j,  7.99 - 15.0j,   7.99 - 15.0j, -9.00E-35 - 4.00E-34j,
-0.645 - 0.0595j, -5.06 + 30.5j,  -5.06 + 30.5j, -2.70E-34 + 1.00E-34j,
  0.534 - 0.242j, -9.70 - 26.3j,  -9.70 - 26.3j,  8.00E-35 + 3.00E-34j,

Eigenvalue L[3]:  105 + 116j
Checking the properties of eigenvector V3 (AV - VL should be a zero vector):
Eigenvector V3    AV = A * V3  VL = V3 * L[3]               AV - VL
0.165 + 0.359j, -24.5 + 56.8j,  -24.5 + 56.8j, 1.00E-34 + 9.00E-34j,
0.208 + 0.513j, -38.1 + 78.0j,  -38.1 + 78.0j, 6.00E-34 + 2.00E-34j,
0.358 + 0.473j, -17.5 + 91.3j,  -17.5 + 91.3j, 6.00E-34 + 1.00E-34j,
0.154 + 0.402j, -30.7 + 60.1j,  -30.7 + 60.1j,        2.00E-34 + 0j,
```

6.9.6 Real QZ Decomposition

`ctx.mat_real_qz`(*matA*, *Query*, *matB*)

Performs a real QZ decomposition of a pair of square matrices maA $= A$ and matB $= B$.

Writing `matA.RealQZ(Query, matB)` has the same effect

See also Eigen [235], Wikipedia [1424], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

B
Optional. A square matrix of the same type and dimensiion as A.

Results:

S
Returns matrix S in the QZ decomposition.

T
Returns matrix T in the QZ decomposition.

Q
Returns matrix Q in the QZ decomposition.

Z
Returns matrix Z in the QZ decomposition.

Given a real square matrices A and B, this class computes the real QZ decomposition: $A = QSZ, B = QTZ$ where Q and Z are real orthogonal matrixes, T is upper-triangular matrix, and S is upper quasi-triangular matrix. An orthogonal matrix is a matrix whose inverse is equal to its transpose, $U^{-1} = U^T$. A quasi-triangular matrix is a block-triangular matrix whose diagonal consists of 1-by-1 blocks and 2-by-2 blocks where further reduction is impossible due to complex eigenvalues.

The eigenvalues of the pencil $A - zB$ can be obtained from 1x1 and 2x2 blocks on the diagonals of S and T. If computeQZ==false, some time is saved by not computing matrices Q and Z.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", "")
>>> A.show("A")
A:
 48, 43,  31,   19, 14, 24,
 46, 10,  20,  4.6, 14, 10,
 27, 39,  13,   34, 29, 37,
7.1, 42,  15,  2.8, 35, 23,
 23, 50,  42, 0.44, 42, 23,
 12, 50, 1.2,   46, 36, 47,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAA6x6", "")
>>> B.show("B")
B:
44.9, 25.5, 50.0, 47.9, 26.4, 62.0,
25.5, 24.3, 49.1, 95.0, 29.0, 46.6,
50.0, 49.1, 55.5, 84.0, 44.4, 26.7,
47.9, 95.0, 84.0, 64.5, 39.5, 87.5,
26.4, 29.0, 44.4, 39.5, 39.8, 12.3,
62.0, 46.6, 26.7, 87.5, 12.3, 85.0,

>>> Query = "S, T, Q, Z"
>>> Res = A.eigen_realQZ2(Query, B)

>>> Res["S"].show("S"); Res["T"].show("T"); Res["Q"].show("Q"); Res["Z"].show("Z")
```

(continues on next page)

(continued from previous page)

```
S:
  -63.4,    21.2,     2.29,   -7.98,  -109, -78.3,
      0,    54.7,    -9.94,   -22.3, -41.7, -38.8,
      0,       0,    -3.13,    35.8, 0.634,  44.2,
      0,       0,     11.1,    10.1,  15.2,  21.9,
      0,       0,        0,       0,  17.7,  16.6,
      0,       0,        0,       0,     0, -1.85,

T:
  -26.0,    97.8,     19.4,    16.6,  -218,  -121,
      0,    56.9,     41.4,    31.4, -88.6, -49.7,
      0,       0,    -17.9,       0,  28.2,  62.7,
      0,       0,        0,    41.7,  40.0,  15.9,
      0,       0,        0,       0, -72.4, -18.7,
      0,       0,        0,       0,     0,  46.3,

Q:
0.0947,    0.603,  0.134, -0.760,   -0.174, 0.0338,
0.0315,    0.707, -0.256,  0.422,    0.464,  0.201,
 0.444,    0.241,  0.167,  0.302,   -0.256, -0.748,
 0.458,   -0.264, -0.374, -0.371,    0.614, -0.257,
 0.292, -0.00167, -0.745, 0.0408,   -0.558,  0.214,
 0.706,  -0.0920,  0.439,  0.117, -0.00351,  0.536,

Z:
-0.0354,     -0.278,   0.668,  0.0104,  -0.670,  -0.161,
  0.896,     -0.282, -0.0310,   0.325,  0.0639, -0.0829,
 -0.266,     -0.660,   0.323,   0.145,   0.553,   0.249,
 -0.338, -0.0000580,  -0.211,   0.834, -0.0902,  -0.371,
-0.0542,     -0.365,  -0.211,  -0.415,   0.128,  -0.794,
-0.0913,     -0.523,  -0.599, -0.0741,  -0.465,   0.370,

>>> CheckResult = (A - Res["Q"] * Res["S"] *Res["Z"]).norm()
>>> print("||A - Q * S * Z|| (should be zero): ", (CheckResult).s())
||A - Q * S * Z|| (should be zero):  1.11E-32

>>>  CheckResult = (B - Res["Q"] * Res["T"] *Res["Z"]).norm()
>>> print("||B - Q * T * Z|| (should be zero): ", (CheckResult).s())
||B - Q * T * Z|| (should be zero):  1.57E-32
```

6.9.7 PseudoEigenSystem

`ctx.mat_pseudo_eigen_system`(*matA*, *Query*)

Returns the eigenvalues of the general square matrix *matA* $= A$.

Writing `matA.PseudoEigenSystem(Query)` has the same effect

See also Eigen [233], Eigen [234].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

pseudoeval
Returns the pseudo-eigenvalues of given matrix.

pseudoevec
Returns the pseudo-eigenvectors of given matrix.

Computes eigenvalues and eigenvectors of general matrices. The eigenvalues and eigenvectors of a matrix A are scalars λ and vectors v such that $Av = \lambda v$. If D is a diagonal matrix with the eigenvalues on the diagonal, and V is a matrix with the eigenvectors as its columns, then $AV = VD$. The matrix V is almost always invertible, in which case we have $A = VDV^{-1}$. This is called the eigendecomposition. The eigenvalues and eigenvectors of a matrix may be complex, even when the matrix is real. However, we can choose real matrices V and D satisfying $AV = VD$, just like the eigendecomposition, if the matrix D is not required to be diagonal, but if it is allowed to have blocks of the form

$$\begin{bmatrix} \mathbf{u} & \mathbf{v} \\ -\mathbf{v} & \mathbf{u} \end{bmatrix}$$

(where u and v are real numbers) on the diagonal. These blocks correspond to complex eigenvalue pairs $u \pm iv$. We call this variant of the eigendecomposition the pseudo-eigendecomposition.

Call the function compute() to compute the eigenvalues and eigenvectors of a given matrix. Alternatively, you can use the EigenSolver(const MatrixType, bool) constructor which computes the eigenvalues and eigenvectors at construction time. Once the eigenvalue and eigenvectors are computed, they can be retrieved with the eigenvalues() and eigenvectors() functions. The pseudoEigenvalueMatrix() and pseudoEigenvectors() methods allow the construction of the pseudo-eigendecomposition.

The matrix is first reduced to real Schur form using the RealSchur (or Complex) class. The Schur decomposition is then used to compute the eigenvalues and eigenvectors. The cost of the computation is dominated by the cost of the Schur decomposition, which is very approximately 25n3 (where n is the size of the matrix) if computeEigenvectors is true, and 10n3 if computeEigenvectors is false. This method reuses of the allocated data in the EigenSolver object.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", "")
>>> A.show("A")
A:
 48, 43,  31,   19, 14, 24,
 46, 10,  20,  4.6, 14, 10,
 27, 39,  13,   34, 29, 37,
7.1, 42,  15,  2.8, 35, 23,
 23, 50,  42, 0.44, 42, 23,
 12, 50, 1.2,   46, 36, 47,

>>> Query = "PseudoEval, PseudoEvec"
>>> Res = A.eigen_PseudoEigenSystem2(Query)

>>> D = Res["PseudoEval"]; D.show("D")
D:
158,     0,    0,     0,    0,     0,
```

(continues on next page)

(continued from previous page)

```
  0, -20.8,    0,     0,    0,     0,
  0,     0, 21.2,     0,    0,     0,
  0,     0,    0,  3.32, 16.6,     0,
  0,     0,    0, -16.6, 3.32,     0,
  0,     0,    0,     0,    0, -2.45,

>>> V = Res["PseudoEvec"]; V.show("V")
V:
0.448, -0.313,  0.464, -0.0997,  0.684, -0.0912,
0.283,  0.535,  0.454,   0.474,   1.04,  -0.443,
0.442,  0.277, -0.364,  -0.905,  -1.00,   0.221,
0.312, -0.406, -0.138,   0.852, -0.183,   -1.04,
0.463, -0.583, -0.101,   0.584,  0.979,  -0.632,
0.460,  0.240, -0.692,  -0.982,  -2.34,    1.89,

>>> Vinv = V.eigen_inverse(); Vinv.show(" V^-1")
 V^-1:
 0.478,  0.587,  0.337,  0.273,  0.423,   0.412,
-0.731,  0.985,  0.336, -0.141, -0.150,  0.0289,
  1.59, 0.0374, -0.514,  0.680,  -1.54, 0.00611,
-0.287,  0.467, -0.869,  0.481, 0.0293,   0.471,
-0.303,  0.129, 0.0641, -0.688,  0.740,  -0.123,
0.0355,  0.147, -0.684, -0.400,  0.283,   0.519,

>>> CheckResult = (A - V * D * Vinv).norm()
>>> print("||A - V * D * V^-1|| (should be zero): ", (CheckResult).s())
||A - V * D * V^-1|| (should be zero):  3.25E-33
```

6.9.8 Real Generalized Nonsymmetric Eigenvalues

`ctx.mat_gen_eigen_values`(*matA*, *Query*, *matB*)

Returns the Generalized Nonsymmetric Eigensystem; Eigenvalues only of the general matrix *matA* $= A$.

Writing `matA.GenEigenValues(Query, matB)` has the same effect

See also Eigen [225], Wikipedia [1435], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

eval
Returns the eigenvalues of given matrix.

evec
Returns the eigenvectors of given matrix.

Computes the generalized eigenvalues and eigenvectors of a pair of general (nonsymmetric) matrices. Currently, only real matrices are supported.

The generalized eigenvalues and eigenvectors of a matrix pair A and B are scalars λ and vectors v such that $Av = \lambda Bv$. If D is a diagonal matrix with the eigenvalues on the diagonal, and V is a matrix with the eigenvectors as its columns, then $AV = BVD$. The matrix V is almost always invertible, in which case we have $A = BVDV^{-1}$. This is called the generalized eigen-decomposition.

The generalized eigenvalues and eigenvectors of a matrix pair may be complex, even when the matrices are real. Moreover, the generalized eigenvalue might be infinite if the matrix B is singular. To workaround this difficulty, the eigenvalues are provided as a pair of complex α and real β such that: $\lambda_i = \alpha_i/\beta_i$. If β_i is (nearly) zero, then one can consider the well defined left eigenvalue $\mu_i = \beta_i/\alpha_i$ such that: $\mu_i A v_i = B v_i$, or even $\mu_i u_i^T A v_i = u_i^T B$, where u_i is called the left eigenvector.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", "")
>>> A.show("A")
A:
 48, 43,  31,   19, 14, 24,
 46, 10,  20,  4.6, 14, 10,
 27, 39,  13,   34, 29, 37,
7.1, 42,  15,  2.8, 35, 23,
 23, 50,  42, 0.44, 42, 23,
 12, 50, 1.2,   46, 36, 47,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAA6x6", "")
>>> B.show("B")
B:
44.9, 25.5, 50.0, 47.9, 26.4, 62.0,
25.5, 24.3, 49.1, 95.0, 29.0, 46.6,
50.0, 49.1, 55.5, 84.0, 44.4, 26.7,
47.9, 95.0, 84.0, 64.5, 39.5, 87.5,
26.4, 29.0, 44.4, 39.5, 39.8, 12.3,
62.0, 46.6, 26.7, 87.5, 12.3, 85.0,

>>> Query = "Eval"
>>> Res = A.eigen_GenEigenValues2(Query, B)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
i            L[i]
0:      2.44 + 0j,
1:     0.960 + 0j,
2: 0.209 - 0.729j,
```

(continues on next page)

(continued from previous page)

```
3: 0.209 + 0.729j,
4:    -0.245 + 0j,
5:   -0.0398 + 0j,

>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X = A.cplx() - B.cplx() * L[i]
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - B * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - B * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - B * L[i]) should be zero):
i            L[i]      Det(A - B * L[i])
0:      2.44 + 0j,         2.87E-23 + 0j,
1:     0.960 + 0j,         1.17E-25 + 0j,
2: 0.209 - 0.729j, -3.30E-26 + 3.44E-26j,
3: 0.209 + 0.729j, -3.30E-26 - 3.44E-26j,
4:    -0.245 + 0j,        -1.11E-27 + 0j,
5:   -0.0398 + 0j,         3.55E-27 + 0j,
```

6.9.9 Real Generalized Nonsymmetric Eigensystem

`ctx.mat_gen_eigen_system`(*matA*, *Query*, *matB*)

Returns the Generalized Nonsymmetric Eigensystem; Eigenvalues only of the general matrix *matA* $= A$.

Writing `matA.GenEigenSystem(Query, matB)` has the same effect

See also Eigen [225], Wikipedia [1435], Wikipedia [1430].

Parameters:

Query
Required. A string specifying which items of the result section should be computed.

Results:

eval
Returns the eigenvalues of given matrix.

evec
Returns the eigenvectors of given matrix.

```
>>> from xlcalcnet import *
>>> ctx = mp14.drf(); mp14.setdps(35); mp14.setshowdps(3)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomA6x6", "")
>>> A = A.top_left_corner(4,4); A.show("A ")
A :
 48, 43, 31,  19,
 46, 10, 20, 4.6,
 27, 39, 13,  34,
7.1, 42, 15, 2.8,

>>> B = ctx.read_from_sqlite(mp14.dbpath(), "DecTableRandomSAA6x6", "")
>>> B = B.top_left_corner(4,4); B.show("B")
B:
44.9, 25.5, 50.0, 47.9,
25.5, 24.3, 49.1, 95.0,
50.0, 49.1, 55.5, 84.0,
47.9, 95.0, 84.0, 64.5,

>>> Query = "Eval, Evec"
>>> Res = A.eigen_GenEigenSystem2(Query, B)

>>> L = Res["Eval"]; ct = ["L[i]"]; rt = ["i"] + [x for x in range(L.rows)]
>>> L.show("Vector L of eigenvalues", coltitles = ct, rowtitles = rt)
Vector L of eigenvalues:
i            L[i]
0: 0.431 - 0.989j,
1: 0.431 + 0.989j,
2:   -0.0862 + 0j,
3:     0.703 + 0j,

>>> V =  Res["Evec"]; mt = "Matrix V of eigenvectors (V0, ... , V" + str(V.cols-1) + ")"
>>> V.show(mt, coltitles = ["V#"] * (V.cols))
Matrix V of eigenvectors (V0, ... , V3):
              V0                 V1           V2           V3
  0.364 - 0.364j,   0.364 + 0.364j,  0.404 + 0j, -0.581 + 0j,
 -0.281 - 0.488j,  -0.281 + 0.488j,  0.316 + 0j,  0.231 + 0j,
  0.208 + 0.611j,   0.208 - 0.611j, -0.808 + 0j,  0.645 + 0j,
0.0202 + 0.0331j, 0.0202 - 0.0331j, -0.289 + 0j, -0.439 + 0j,

>>> AC = A.cplx(); BC = B.cplx()
>>> # Vinv = V^-1 = V.H
>>> Vinv = V.eigen_inverse(); Vinv.show(" V^-1")
V^-1:
     1.04 + 0.651j,    0.0367 + 1.33j,    0.669 + 0.592j,   -0.376 + 0.707j,
```

(continues on next page)

(continued from previous page)

```
     1.04 - 0.651j,    0.0367 - 1.33j,    0.669 - 0.592j,   -0.376 - 0.707j,
-0.295 + 9.42E-38j, -1.41 - 1.88E-36j, -1.19 - 3.62E-37j, -2.09 - 3.10E-36j,
 0.192 + 1.76E-37j, 0.731 + 1.08E-36j, 0.754 + 8.75E-38j, -1.04 + 1.45E-36j,

>>> CheckResult = (AC - BC * V * L.D * Vinv).norm()
>>> print("||A - B * V * diag(L) * V^-1|| (should be zero): ", (CheckResult).s())
||A - B * V * diag(L) * V^-1|| (should be zero):  2.54E-33 + 0j

>>> Det = L * 0 # creates a zero vector of the same size and type as L

>>> for i in range(A.rows):
>>>     X = AC - BC * L[i]
>>>     Det[i] = X.eigen_det()

>>> Result = L.concat_horizontal(Det)
>>> mt = "Checking the Eigenvalues (Det(A - B * L[i]) should be zero)"
>>> ct = ["L[i]", "Det(A - B * L[i])"]
>>> Result.show(mt, coltitles = ct, rowtitles = rt)
Checking the Eigenvalues (Det(A - B * L[i]) should be zero):
i            L[i]     Det(A - B * L[i])
0: 0.431 - 0.989j, 1.41E-28 - 2.02E-28j,
1: 0.431 + 0.989j, 1.41E-28 + 2.02E-28j,
2:   -0.0862 + 0j,       -1.01E-29 + 0j,
3:     0.703 + 0j,       -8.72E-30 + 0j,


>>> for i in range(V.rows):
>>>     AV = AC * V.col(i); BVL = (BC * L[i]) * V.col(i); X = AV - BVL
>>>     Li = "L[" + str(i) + "]"; Vi = "V" + str(i)
>>>     print("Eigenvalue " + Li + ": ", L[i].s())
>>>     Result = V.col(i).concat_horizontal(AV).concat_horizontal(BVL).concat_horizontal(X)
>>>     mt = "Checking the properties of eigenvector " + Vi + " (AV - BVL should be a zero vector)"
>>>     ct = ["Eigenvector " + Vi, "AV = A * " + Vi, "BVL = B * " + Vi + " * " + Li, "   AV - BVL"]
>>>     Result.show(mt, coltitles = ct)

Eigenvalue L[0]:  0.431 - 0.989j
Checking the properties of eigenvector V0 (AV - BVL should be a zero vector):
  Eigenvector V0    AV = A * V0  BVL = B * V0 * L[0]              AV - BVL
  0.364 - 0.364j,  12.2 - 18.8j,        12.2 - 18.8j,  1.00E-34 + 4.00E-34j,
 -0.281 - 0.488j,  18.2 - 9.23j,        18.2 - 9.23j,         0 + 9.00E-35j,
  0.208 + 0.611j,  2.24 - 19.8j,        2.24 - 19.8j,  1.60E-34 - 2.00E-34j,
0.0202 + 0.0331j, -6.06 - 13.8j,       -6.06 - 13.8j, -4.00E-35 - 4.00E-34j,

Eigenvalue L[1]:  0.431 + 0.989j
Checking the properties of eigenvector V1 (AV - BVL should be a zero vector):
  Eigenvector V1    AV = A * V1  BVL = B * V1 * L[1]              AV - BVL
  0.364 + 0.364j,  12.2 + 18.8j,        12.2 + 18.8j,  1.00E-34 - 4.00E-34j,
 -0.281 + 0.488j,  18.2 + 9.23j,        18.2 + 9.23j,         0 - 9.00E-35j,
  0.208 - 0.611j,  2.24 + 19.8j,        2.24 + 19.8j,  1.60E-34 + 2.00E-34j,
0.0202 - 0.0331j, -6.06 + 13.8j,       -6.06 + 13.8j, -4.00E-35 + 4.00E-34j,

Eigenvalue L[2]:  -0.0862 + 0j
Checking the properties of eigenvector V2 (AV - BVL should be a zero vector):
Eigenvector V2  AV = A * V2  BVL = B * V2 * L[2]        AV - BVL
    0.404 + 0j,   2.42 + 0j,           2.42 + 0j, -1.00E-35 + 0j,
    0.316 + 0j,   4.24 + 0j,           4.24 + 0j, -4.00E-35 + 0j,
   -0.808 + 0j,   2.88 + 0j,           2.88 + 0j,  2.10E-34 + 0j,
   -0.289 + 0j,   3.21 + 0j,           3.21 + 0j,  2.60E-34 + 0j,

Eigenvalue L[3]:  0.703 + 0j
Checking the properties of eigenvector V3 (AV - BVL should be a zero vector):
Eigenvector V3  AV = A * V3  BVL = B * V3 * L[3]        AV - BVL
   -0.581 + 0j,  -6.34 + 0j,          -6.34 + 0j,  1.40E-34 + 0j,
    0.231 + 0j,  -13.6 + 0j,          -13.6 + 0j, -1.00E-34 + 0j,
    0.645 + 0j,  -13.2 + 0j,          -13.2 + 0j,  5.00E-34 + 0j,
   -0.439 + 0j,   14.0 + 0j,           14.0 + 0j,  5.00E-34 + 0j,
```

6.10 Eigen: Polynomials

6.10.1 Building a polynomial from its roots

`ctx.roots_to_monic_poly(results, matB)`

Calculates the monic polynomial from its roots.

See also: Eigen [218].

6.10.2 Polynomial evaluation

`ctx.poly_eval(roots)`

Evaluates a polynomial

See also: Eigen [218].

6.10.3 Polynomial roots

`ctx.poly_roots()`

Computes all roots (real or complex) of a given polynomial.

See also: Eigen [217].

6.11 Eigen: Fast Fourier Transform

The default implementation is based on kissfft.

The following design decisions were made concerning scaling and half-spectrum for real FFT.

The intent is to facilitate generic programming and ease migrating code from Matlab/octave. We think the default behavior of Eigen/FFT should favor correctness and generality over speed. Of course, the caller should be able to "opt-out" from this behavior and get the speed increase if they want it.

1) Scaling: Other libraries (FFTW,IMKL,KISSFFT) do not perform scaling, so there is a constant gain incurred after the forward&inverse transforms , so IFFT(FFT(x)) = Kx; this is done to avoid a vector-by-value multiply. The downside is that algorithms that worked correctly in Matlab/octave don't behave the same way once implemented in C++.

How Eigen/FFT differs: invertible scaling is performed so IFFT(FFT(x)) = x.

2) Real FFT half-spectrum Other libraries use only half the frequency spectrum (plus one extra sample for the Nyquist bin) for a real FFT, the other half is the conjugate-symmetric of the first half. This saves them a copy and some memory. The downside is the caller needs to have special logic for the number of bins in complex vs real.

How Eigen/FFT differs: The full spectrum is returned from the forward transform. This facilitates generic template programming by obviating separate specializations for real vs complex. On the inverse transform, only half the spectrum is actually used if the output type is real.

6.11.1 Forward transform

`ctx.`**`mat_fft_fwd`**`(`*matA*`)`

Calculates the Fourier transform.

Writing `matA.FFTFwd()` has the same effect

See also: Eigen [206].

6.11.2 Inverse transform

`ctx.`**`mat_fft_inv`**`(`*matA*`)`

Calculates the Fourier transform.

Writing `matA.FFTInv()` has the same effect

See also: Eigen [206].

```
Sub DemoCplxFFT()
    Console.WriteLine("Hello DemoCplxFFT!")
    Dim digits = 15
    Dim n As Int32 = 4
    Dim A, B, TA, TB, TC, C, A2, B2, C2 As New cplx_mat_t
    Dim A_real, B_real, C_Real As New dbl_mat_t

    A.setZero(2*n,1)
    A_real.Random(n, 1)
    For i=0 To n-1
        A(i) = New Complex(A_real(i), 0.0)
    Next
    A.Print("A: ")

    B.setZero(2*n,1)
    B_real.Random(n, 1)
    For i=0 To n-1
        B(i) = New Complex(B_real(i), 0.0)
    Next
    B.Print("B: ")
```

(continues on next page)

(continued from previous page)

```
    TA = A.FFT_Fwd()
    TA.Print("TA: ")

    TB = B.FFT_Fwd()
    TB.Print("TB: ")

    TC.setZero(2*n,1)
    For i=0 To 2*n-1
        TC(i) = TA(i) * TB(i)
    Next
    TC.Print("TC: ")
    C2 = TC.FFT_Inv()
    C2.Print("C2: ")

    C_Real.setZero(2*n,1)
    For i As Integer = 0 To n-1
        For j As Integer = 0 To n-1
            C_Real(i+j) = C_Real(i+j) + A_Real(i) * B_Real(j)
        Next j
    Next i
    C_Real.Print("C_Real: ")
End Sub
```

```
Hello DemoCplxFFT!
A:
-0.997497+0.000000j;
 0.127171+0.000000j;
-0.613392+0.000000j;
 0.617481+0.000000j;
 0.000000+0.000000j;
 0.000000+0.000000j;
 0.000000+0.000000j;
 0.000000+0.000000j;

B:
 0.170019+0.000000j;
-0.040254+0.000000j;
-0.299417+0.000000j;
 0.791925+0.000000j;
 0.000000+0.000000j;
 0.000000+0.000000j;
 0.000000+0.000000j;
 0.000000+0.000000j;

TA:
-0.866237+0.000000j;
-1.344199+0.086843j;
-0.384106+0.490310j;
-0.650796-1.139940j;
-2.355541+0.000000j;
-0.650796+1.139940j;
-0.384106-0.490310j;
-1.344199-0.086843j;

TB:
 0.622272+0.000000j;
-0.418421-0.232094j;
 0.469436+0.832179j;
 0.758458-0.830929j;
-0.881069+0.000000j;
 0.758458+0.830929j;
 0.469436-0.832179j;
-0.418421+0.232094j;
```

(continues on next page)

(continued from previous page)

```
TC:
-0.539036+0.000000j;
 0.582597+0.275644j;
-0.588339-0.089476j;
-1.440810-0.323831j;
 2.075395+0.000000j;
-1.440810+0.323831j;
-0.588339+0.089476j;
 0.582597-0.275644j;

C2:
-0.169593+0.000000j;
 0.061775+0.000000j;
 0.189261+0.000000j;
-0.698345+0.000000j;
 0.259513+0.000000j;
-0.670644+0.000000j;
 0.488999+0.000000j;
 0.000000+0.000000j;

C_Real:
-1.695931E-001;
 6.177455E-002;
 1.892607E-001;
-6.983454E-001;
 2.595135E-001;
-6.706443E-001;
 4.889985E-001;
 0.000000E+000;
```

6.12 Eigen and Numpy (Multiprecision): Functions of matrix argument

6.12.1 Matrix Exponential

`ctx.mat_expm(matA)`

Computes the matrix exponential of a square matrix A, which is defined by the power series $\exp(A) = I + A + \frac{A^2}{2!} + \frac{A^3}{3!} + \ldots$

Writing `matA.ExpMat()` has the same effect

See also: Wikipedia [1421], Wikipedia [1431].

See also: Eigen [209], Eigen [208].

Basic examples:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> expm(zeros(3))
[1.0  0.0  0.0]
[0.0  1.0  0.0]
[0.0  0.0  1.0]
>>> expm(eye(3))
[2.71828182845905               0.0               0.0]
[             0.0  2.71828182845905               0.0]
[             0.0               0.0  2.71828182845905]
>>> expm([[1,1,0],[1,0,1],[0,1,0]])
[ 3.86814500615414  2.26812870852145  0.841130841230196]
[ 2.26812870852145  2.44114713886289   1.42699786729125]
[0.841130841230196  1.42699786729125    1.6000162976327]
>>> expm([[1,1,0],[1,0,1],[0,1,0]], method='pade')
[ 3.86814500615414  2.26812870852145  0.841130841230196]
[ 2.26812870852145  2.44114713886289   1.42699786729125]
[0.841130841230196  1.42699786729125    1.6000162976327]
>>> expm([[1+j, 0], [1+j,1]])
[(1.46869393991589 + 2.28735528717884j)                         0.0]
[  (1.03776739863568 + 3.536943175722j)  (2.71828182845905 + 0.0j)]
```

Matrices with large entries are allowed:

```
>>> expm(matrix([[1,2],[2,3]])**25)
[5.65024064048415e+2050488462815550  9.14228140091932e+2050488462815550]
[9.14228140091932e+2050488462815550  1.47925220414035e+2050488462815551]
```

The identity $\exp(A+B) = \exp(A)\exp(B)$ does not hold for noncommuting matrices:

```
>>> A = hilbert(3)
>>> B = A + eye(3)
>>> chop(mnorm(A*B - B*A))
0.0
>>> chop(mnorm(expm(A+B) - expm(A)*expm(B)))
0.0
>>> B = A + ones(3)
>>> mnorm(A*B - B*A)
1.8
>>> mnorm(expm(A+B) - expm(A)*expm(B))
42.0927851137247
```

6.12.2 Matrix Sine

`ctx.mat_sinm(matA)`

Calculates the sine function of the matrix.

Writing `matA.SinMat()` has the same effect

See also: Wikipedia [1421], Wikipedia [1439].

See also: Eigen [209], Eigen [214].

The cosine of a square matrix A is defined in analogy with the matrix exponential.

$$\cos(A) = \frac{e^{iA} + e^{-iA}}{2}$$

$$\sin(A) = \frac{e^{iA} - e^{-iA}}{2i}$$

$$\cos^2(A) + \sin^2(A) = I$$

For real A, we can write $\cos(A) = \Re(e^{iA})$ and $\sin(A) = \Im(e^{iA})$.

$$\cosh(A) = \frac{e^{A} + e^{-A}}{2}$$

$$\sinh(A) = \frac{e^{A} - e^{-A}}{2}$$

Examples:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> X = eye(3)
>>> cosm(X)
[0.54030230586814               0.0               0.0]
[            0.0  0.54030230586814               0.0]
[            0.0               0.0  0.54030230586814]
>>> X = hilbert(3)
>>> cosm(X)
[ 0.424403834569555  -0.316643413047167  -0.221474945949293]
[-0.316643413047167   0.820646708837824  -0.127183694770039]
[-0.221474945949293  -0.127183694770039   0.909236687217541]
>>> X = matrix([[1+j,-2],[0,-j]])
>>> cosm(X)
[(0.833730025131149 - 0.988897705762865j)  (1.07485840848393 - 0.17192140544213j)]
[                                     0.0               (1.54308063481524 + 0.0j)]
```

6.12.3 Matrix Cosine

`ctx.mat_cosm(matA)`

Calculates the cosine function of the matrix.

Writing `matA.CosMat()` has the same effect

See also: Wikipedia [1421], Wikipedia [1439].

See also: Eigen [209], Eigen [207].

The cosine of a square matrix A is defined in analogy with the matrix exponential.

$$\cos(A) = \frac{e^{iA} + e^{-iA}}{2}$$

$$\sin(A) = \frac{e^{iA} - e^{-iA}}{2i}$$

$$\cos^2(A) + \sin^2(A) = I$$

For real A, we can write $\cos(A) = \Re(e^{iA})$ and $\sin(A) = \Im(e^{iA})$.

$$\cosh(A) = \frac{e^{A} + e^{-A}}{2}$$

$$\sinh(A) = \frac{e^{A} - e^{-A}}{2}$$

Examples:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> X = eye(3)
>>> cosm(X)
[0.54030230586814               0.0               0.0]
[            0.0  0.54030230586814               0.0]
[            0.0               0.0  0.54030230586814]
>>> X = hilbert(3)
>>> cosm(X)
[ 0.424403834569555  -0.316643413047167  -0.221474945949293]
[-0.316643413047167   0.820646708837824  -0.127183694770039]
[-0.221474945949293  -0.127183694770039   0.909236687217541]
>>> X = matrix([[1+j,-2],[0,-j]])
>>> cosm(X)
[(0.833730025131149 - 0.988897705762865j)  (1.07485840848393 - 0.17192140544213j)]
[                                     0.0               (1.54308063481524 + 0.0j)]
```

6.12.4 Matrix Hyperbolic Sine

`ctx.mat_sinhm(matA)`

Calculates the hyperbolic sine function of the matrix.

Writing `matA.SinhMat()` has the same effect

See also: Wikipedia [1421], Wikipedia [1439].

See also: Eigen [209], Eigen [211].

The hyperbolic sine of a square matrix A is defined in analogy with the matrix exponential.

$$\sinh(A) = \frac{e^A - e^{-A}}{2}$$

$$\cosh(A) = \frac{e^A + e^{-A}}{2}$$

$$\cosh^2(A) + \sinh^2(A) = I$$

```
Sub DemoCplxMatrixFunctions()
    Console.WriteLine("Hello CplxMatrixFunctions!")
    Dim digits = 15
    Dim n As Int32 = 4
    Dim A, B, C, D, E, F As New cplx_mat_t
    A.RandomSymmetric(n)
    A.Print("A: ")

    B = A.SinhMat()
    B.Print("B = Sinh(A): ")
    C = A.CoshMat()
    C.Print("C = Cosh(A): ")
    D = C * C - B * B
    D.Print("C * C - B * B: ")
End Sub
```

```
Hello CplxMatrixFunctions!
A:
 0.254341-1.994995j,  1.110691+0.032289j, -0.311350-0.647176j,  0.547685-1.281594j;
 1.110691+0.032289j,  1.435774-1.303568j, -0.641133-0.284371j, -0.827662-0.328684j;
-0.311350-0.647176j, -0.641133-0.284371j, -0.217231+1.954100j, -0.776330-0.558306j;
 0.547685-1.281594j, -0.827662-0.328684j, -0.776330-0.558306j,  0.652181-1.335063j;


B = Sinh(A):
-0.344029-0.627502j,  0.956056-0.480516j, -0.134878-0.034162j, -0.590098+0.338244j;
 0.956056-0.480516j,  1.739419-3.149695j, -0.793048-0.141610j, -1.597165+0.360539j;
-0.134878-0.034162j, -0.793048-0.141610j, -0.152880+1.211517j, -0.351965+0.026714j;
-0.590098+0.338244j, -1.597165+0.360539j, -0.351965+0.026714j,  0.171917-0.584919j;

C = Cosh(A):
-0.039723+0.009230j,  0.532073-1.170311j, -0.476465+0.181642j, -1.605643-0.168838j;
 0.532073-1.170311j,  2.022858-2.326341j, -0.294411-0.149198j, -1.095355+1.019715j;
-0.476465+0.181642j, -0.294411-0.149198j, -0.216192+0.186437j,  0.187127+0.027892j;
-1.605643-0.168838j, -1.095355+1.019715j,  0.187127+0.027892j,  0.851060-0.052593j;

C * C - B * B:
 1.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j;
 0.000000+0.000000j,  1.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j;
 0.000000+0.000000j,  0.000000+0.000000j,  1.000000+0.000000j,  0.000000+0.000000j;
 0.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j,  1.000000+0.000000j;
```

6.12.5 Matrix Hyperbolic Cosine

`ctx.mat_coshm(matA)`

Calculates the hyperbolic cosine function of the matrix.

Writing `matA.CoshMat()` has the same effect

See also: Wikipedia [1421], Wikipedia [1439].

See also: Eigen [209], Eigen [210].

The hyperbolic cosine of a square matrix A is defined in analogy with the matrix exponential.

$$\cosh(A) = \frac{e^A + e^{-A}}{2}$$

$$\sinh(A) = \frac{e^A - e^{-A}}{2}$$

$$\cosh^2(A) + \sinh^2(A) = I$$

```
Sub DemoCplxMatrixFunctions()
    Console.WriteLine("Hello CplxMatrixFunctions!")
    Dim digits = 15
    Dim n As Int32 = 4
    Dim A, B, C, D, E, F As New cplx_mat_t
    A.RandomSymmetric(n)
    A.Print("A: ")
    B = A.ExpMat()
    B.Print("B = Exp(A): ")
    C = B.LogMat()
    C.Print("C = Log(B): ")

    D = B.SqrtMat()
    D.Print("D = Sqrt(B): ")
    E = D * D
    E.Print("E = D * D: ")

    B = A.SinMat()
    B.Print("B = Sin(A): ")
    C = A.CosMat()
    C.Print("C = Cos(A): ")
    D = B * B + C * C
    D.Print("B * B + C * C: ")

    B = A.SinhMat()
    B.Print("B = Sinh(A): ")
    C = A.CoshMat()
    C.Print("C = Cosh(A): ")
    D = C * C - B * B
    D.Print("C * C - B * B: ")
End Sub
```

```
Hello CplxMatrixFunctions!
A:
 0.254341-1.994995j,  1.110691+0.032289j, -0.311350-0.647176j,  0.547685-1.281594j;
 1.110691+0.032289j,  1.435774-1.303568j, -0.641133-0.284371j, -0.827662-0.328684j;
-0.311350-0.647176j, -0.641133-0.284371j, -0.217231+1.954100j, -0.776330-0.558306j;
 0.547685-1.281594j, -0.827662-0.328684j, -0.776330-0.558306j,  0.652181-1.335063j;

B = Exp(A):
-0.383752-0.618272j,  1.488129-1.650828j, -0.611343+0.147480j, -2.195741+0.169406j;
 1.488129-1.650828j,  3.762277-5.476036j, -1.087459-0.290808j, -2.692520+1.380254j;
-0.611343+0.147480j, -1.087459-0.290808j, -0.369072+1.397953j, -0.164837+0.054606j;
-2.195741+0.169406j, -2.692520+1.380254j, -0.164837+0.054606j,  1.022977-0.637512j;
```

(continues on next page)

(continued from previous page)

```
C = Log(B):
 0.254341-1.994995j,  1.110691+0.032289j, -0.311350-0.647176j,  0.547685-1.281594j;
 1.110691+0.032289j,  1.435774-1.303568j, -0.641133-0.284371j, -0.827662-0.328684j;
-0.311350-0.647176j, -0.641133-0.284371j, -0.217231+1.954100j, -0.776330-0.558306j;
 0.547685-1.281594j, -0.827662-0.328684j, -0.776330-0.558306j,  0.652181-1.335063j;

D = Sqrt(B):
 0.508620-0.931943j,  0.661969-0.319593j, -0.337003-0.172068j, -0.485698-0.655636j;
 0.661969-0.319593j,  2.077099-1.250104j, -0.401242-0.208424j, -0.746596+0.018077j;
-0.337003-0.172068j, -0.401242-0.208424j,  0.502520+0.963369j, -0.377613-0.193978j;
-0.485698-0.655636j, -0.746596+0.018077j, -0.377613-0.193978j,  1.013700-0.687531j;

E = D * D:
-0.383752-0.618272j,  1.488129-1.650828j, -0.611343+0.147480j, -2.195741+0.169406j;
 1.488129-1.650828j,  3.762277-5.476036j, -1.087459-0.290808j, -2.692520+1.380254j;
-0.611343+0.147480j, -1.087459-0.290808j, -0.369072+1.397953j, -0.164837+0.054606j;
-2.195741+0.169406j, -2.692520+1.380254j, -0.164837+0.054606j,  1.022977-0.637512j;

B = Sin(A):
 3.185996-4.423204j,  1.572018+0.576777j, -0.236161-2.007119j,  3.744383-3.392271j;
 1.572018+0.576777j,  0.829215-0.073722j, -0.452703-0.362702j,  0.213357-0.343298j;
-0.236161-2.007119j, -0.452703-0.362702j, -0.274184+2.749772j, -0.899405-1.828027j;
 3.744383-3.392271j,  0.213357-0.343298j, -0.899405-1.828027j,  3.157659-2.589512j;

C = Cos(A):
 4.524926+3.229556j, -0.735268+1.275592j,  1.749846+0.069384j,  3.245352+3.579443j;
-0.735268+1.275592j, -0.328697+0.628146j,  0.172580+0.683175j,  0.290239+0.309901j;
 1.749846+0.069384j,  0.172580+0.683175j,  3.469410-0.664084j,  0.996030+0.538410j;
 3.245352+3.579443j,  0.290239+0.309901j,  0.996030+0.538410j,  2.603737+2.713445j;

B * B + C * C:
 1.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j;
 0.000000+0.000000j,  1.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j;
 0.000000+0.000000j,  0.000000+0.000000j,  1.000000+0.000000j,  0.000000+0.000000j;
 0.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j,  1.000000+0.000000j;

B = Sinh(A):
-0.344029-0.627502j,  0.956056-0.480516j, -0.134878-0.034162j, -0.590098+0.338244j;
 0.956056-0.480516j,  1.739419-3.149695j, -0.793048-0.141610j, -1.597165+0.360539j;
-0.134878-0.034162j, -0.793048-0.141610j, -0.152880+1.211517j, -0.351965+0.026714j;
-0.590098+0.338244j, -1.597165+0.360539j, -0.351965+0.026714j,  0.171917-0.584919j;

C = Cosh(A):
-0.039723+0.009230j,  0.532073-1.170311j, -0.476465+0.181642j, -1.605643-0.168838j;
 0.532073-1.170311j,  2.022858-2.326341j, -0.294411-0.149198j, -1.095355+1.019715j;
-0.476465+0.181642j, -0.294411-0.149198j, -0.216192+0.186437j,  0.187127+0.027892j;
-1.605643-0.168838j, -1.095355+1.019715j,  0.187127+0.027892j,  0.851060-0.052593j;

C * C - B * B:
 1.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j;
 0.000000+0.000000j,  1.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j;
 0.000000+0.000000j,  0.000000+0.000000j,  1.000000+0.000000j,  0.000000+0.000000j;
 0.000000+0.000000j,  0.000000+0.000000j,  0.000000+0.000000j,  1.000000+0.000000j;
```

6.12.6 Matrix Square Root

`ctx.mat_sqrtm(matA)`

Computes a square root of the square matrix A, i.e. returns a matrix $B = A^{1/2}$ such that $B^2 = A$. The square root of a matrix, if it exists, is not unique.

Writing `matA.SqrtMat()` has the same effect

See also: Wikipedia [1421], Wikipedia [1438].

See also: Eigen [209], Eigen [215].

Examples

Square roots of some simple matrices:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> sqrtm([[1,0], [0,1]])
[1.0  0.0]
[0.0  1.0]
>>> sqrtm([[0,0], [0,0]])
[0.0  0.0]
[0.0  0.0]
>>> sqrtm([[2,0],[0,1]])
[1.4142135623731  0.0]
[            0.0  1.0]
>>> sqrtm([[1,1],[1,0]])
[ (0.920442065259926 - 0.21728689675164j)  (0.568864481005783 + 0.351577584254143j)]
[(0.568864481005783 + 0.351577584254143j)  (0.351577584254143 - 0.568864481005783j)]
>>> sqrtm([[1,0],[0,1]])
[1.0  0.0]
[0.0  1.0]
>>> sqrtm([[-1,0],[0,1]])
[(0.0 - 1.0j)           0.0]
[         0.0  (1.0 + 0.0j)]
>>> sqrtm([[j,0],[0,j]])
[(0.707106781186547 + 0.707106781186547j)                                        0.0]
[                                     0.0  (0.707106781186547 + 0.707106781186547j)]
```

A square root of a rotation matrix, giving the corresponding half-angle rotation matrix:

```
>>> t1 = 0.75
>>> t2 = t1 * 0.5
>>> A1 = matrix([[cos(t1), -sin(t1)], [sin(t1), cos(t1)]])
>>> A2 = matrix([[cos(t2), -sin(t2)], [sin(t2), cos(t2)]])
>>> sqrtm(A1)
[0.930507621912314  -0.366272529086048]
[0.366272529086048   0.930507621912314]
>>> A2
[0.930507621912314  -0.366272529086048]
[0.366272529086048   0.930507621912314]
```

The identity $(A^2)^{1/2} = A$ does not necessarily hold:

```
>>> A = matrix([[4,1,4],[7,8,9],[10,2,11]])
>>> sqrtm(A**2)
[ 4.0  1.0   4.0]
[ 7.0  8.0   9.0]
[10.0  2.0  11.0]
>>> sqrtm(A)**2
[ 4.0  1.0   4.0]
[ 7.0  8.0   9.0]
[10.0  2.0  11.0]
>>> A = matrix([[-4,1,4],[7,-8,9],[10,2,11]])
```

(continues on next page)

(continued from previous page)

```
>>> sqrtm(A**2)
[  7.43715112194995  -0.324127569985474   1.8481718827526]
[-0.251549715716942    9.32699765900402  2.48221180985147]
[  4.11609388833616   0.775751877098258   13.017955697342]
>>> chop(sqrtm(A)**2)
[-4.0   1.0   4.0]
[ 7.0  -8.0   9.0]
[10.0   2.0  11.0]
```

For some matrices, a square root does not exist:

```
>>> sqrtm([[0,1], [0,0]])
Traceback (most recent call last):
    ...
ZeroDivisionError: matrix is numerically singular
```

Two examples from the documentation for Matlab's `sqrtm`:

```
>>> mp.dps = 15; mp.pretty = True
>>> sqrtm([[7,10],[15,22]])
[1.56669890360128  1.74077655955698]
[2.61116483933547  4.17786374293675]
>>>
>>> X = matrix(\
...   [[5,-4,1,0,0],
...   [-4,6,-4,1,0],
...   [1,-4,6,-4,1],
...   [0,1,-4,6,-4],
...   [0,0,1,-4,5]])
>>> Y = matrix(\
...   [[2,-1,-0,-0,-0],
...   [-1,2,-1,0,-0],
...   [0,-1,2,-1,0],
...   [-0,0,-1,2,-1],
...   [-0,-0,-0,-1,2]])
>>> mnorm(sqrtm(X) - Y)
4.53155328326114e-19
```

6.12.7 Matrix Logarithm

`ctx.mat_logm(`*matA*`)`

Calculates the logarithm of the matrix.

Writing `matA.LogMat()` has the same effect

See also: Wikipedia [1421], Wikipedia [1429].

See also: Eigen [209], Eigen [212].

Computes a logarithm of the square matrix A, i.e. returns a matrix $B = \log(A)$ such that $\exp(B) = A$. The logarithm of a matrix, if it exists, is not unique.

Examples

Logarithms of some simple matrices:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> X = eye(3)
>>> logm(X)
[0.0  0.0  0.0]
[0.0  0.0  0.0]
[0.0  0.0  0.0]
>>> logm(2*X)
[0.693147180559945                0.0                0.0]
[              0.0  0.693147180559945                0.0]
[              0.0                0.0  0.693147180559945]
>>> logm(expm(X))
[1.0  0.0  0.0]
[0.0  1.0  0.0]
[0.0  0.0  1.0]
```

A logarithm of a complex matrix:

```
>>> X = matrix([[2+j, 1, 3], [1-j, 1-2*j, 1], [-4, -5, j]])
>>> B = logm(X)
>>> nprint(B)
[ (0.808757 + 0.107759j)    (2.20752 + 0.202762j)   (1.07376 - 0.773874j)]
[ (0.905709 - 0.107795j)  (0.0287395 - 0.824993j)  (0.111619 + 0.514272j)]
[(-0.930151 + 0.399512j)   (-2.06266 - 0.674397j)  (0.791552 + 0.519839j)]
>>> chop(expm(B))
[(2.0 + 1.0j)           1.0           3.0]
[(1.0 - 1.0j)  (1.0 - 2.0j)           1.0]
[        -4.0          -5.0  (0.0 + 1.0j)]
```

A matrix X close to the identity matrix, for which $\log(\exp(X)) = \exp(\log(X)) = X$ holds:

```
>>> X = eye(3) + hilbert(3)/4
>>> X
[              1.25             0.125  0.0833333333333333]
[             0.125  1.08333333333333             0.0625]
[0.0833333333333333            0.0625               1.05]
>>> logm(expm(X))
[              1.25             0.125  0.0833333333333333]
[             0.125  1.08333333333333             0.0625]
[0.0833333333333333            0.0625               1.05]
>>> expm(logm(X))
[              1.25             0.125  0.0833333333333333]
[             0.125  1.08333333333333             0.0625]
[0.0833333333333333            0.0625               1.05]
```

A logarithm of a rotation matrix, giving back the angle of the rotation:

```
>>> t = 3.7
>>> A = matrix([[cos(t),sin(t)],[-sin(t),cos(t)]])
>>> chop(logm(A))
[             0.0  -2.58318530717959]
[2.58318530717959                0.0]
>>> (2*pi-t)
2.58318530717959
```

For some matrices, a logarithm does not exist:

```
>>> logm([[1,0], [0,0]])
Traceback (most recent call last):
    ...
ZeroDivisionError: matrix is numerically singular
```

Logarithm of a matrix with large entries:

```
>>> logm(hilbert(3) * 10**20).apply(re)
[ 45.5597513593433  1.27721006042799  0.317662687717978]
[ 1.27721006042799  42.5222778973542   2.24003708791604]
[0.317662687717978  2.24003708791604    42.395212822267]
```

6.12.8 Matrix power

`ctx.mat_powm(`*matA*, *r*`)`

Computes $A^r = \exp(A \log r)$ for a matrix A and complex number r.

Writing `matA.PowMat(r)` has the same effect

See also: Wikipedia [1421], Wikipedia [1431], Wikipedia [1429].

See also: Eigen [209], Eigen [213].

Examples

Powers and inverse powers of a matrix:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = True
>>> A = matrix([[4,1,4],[7,8,9],[10,2,11]])
>>> powm(A, 2)
[ 63.0  20.0   69.0]
[174.0  89.0  199.0]
[164.0  48.0  179.0]
>>> chop(powm(powm(A, 4), 1/4.))
[ 4.0  1.0   4.0]
[ 7.0  8.0   9.0]
[10.0  2.0  11.0]
>>> powm(extraprec(20)(powm)(A, -4), -1/4.)
[ 4.0  1.0   4.0]
[ 7.0  8.0   9.0]
[10.0  2.0  11.0]
>>> chop(powm(powm(A, 1+0.5j), 1/(1+0.5j)))
[ 4.0  1.0   4.0]
[ 7.0  8.0   9.0]
[10.0  2.0  11.0]
>>> powm(extraprec(5)(powm)(A, -1.5), -1/(1.5))
[ 4.0  1.0   4.0]
[ 7.0  8.0   9.0]
[10.0  2.0  11.0]
```

A Fibonacci-generating matrix:

```
>>> powm([[1,1],[1,0]], 10)
[89.0  55.0]
[55.0  34.0]
>>> fib(10)
55.0
>>> powm([[1,1],[1,0]], 6.5)
[(16.5166626964253 - 0.0121089837381789j)  (10.2078589271083 + 0.0195927472575932j)]
[(10.2078589271083 + 0.0195927472575932j)  (6.30880376931698 - 0.0317017309957721j)]
>>> (phi**6.5 - (1-phi)**6.5)/sqrt(5)
(10.2078589271083 - 0.0195927472575932j)
>>> powm([[1,1],[1,0]], 6.2)
[ (14.3076953002666 - 0.008222855781077j)  (8.81733464837593 + 0.0133048601383712j)]
[(8.81733464837593 + 0.0133048601383712j)  (5.49036065189071 - 0.0215277159194482j)]
>>> (phi**6.2 - (1-phi)**6.2)/sqrt(5)
(8.81733464837593 - 0.0133048601383712j)
```

CHAPTER

SEVEN

NUMPY: USE WITH MULTIPRECISION DATA TYPES

7.1 Numpy array creation from shape or value

Slicing: https://stackoverflow.com/questions/509211/how-slicing-in-python-works

Note: Logic functions are not supported with multiprecision numbers as arguments

Note: input and output should use sqlite, xlread, enumerate

https://numpy.org/doc/stable/reference/generated/numpy.ufunc.html

https://numpy.org/doc/stable/reference/generated/numpy.where.html#numpy.where

https://numpy.org/doc/stable/reference/generated/numpy.choose.html#numpy.choose

https://numpy.org/doc/stable/reference/generated/numpy.select.html

Some text

https://numpy.org/doc/stable/glossary.html

https://numpy.org/doc/stable/reference/routines.array-creation.html

https://numpy.org/doc/stable/reference/maskedarray.baseclass.html#maskedarray-baseclass

https://numpy.org/doc/stable/reference/routines.ma.html

https://numpy.org/doc/stable/reference/arrays.ndarray.html#array-attributes

https://numpy.org/doc/stable/reference/arrays.ndarray.html#array-methods

7.1.1 Conversion from integer, float or complex array

npm.t(*ctx*, *matA*)

The following code prepares the example:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> np.set_printoptions(linewidth=200)
```

The following code creates a 3 x 3 array of type `int`, and then converts it into arrays of the multiprecision type `ctx` in `ctx_all`:

```
>>> matA = npm.array([[1,2,3], [4,5,6], [7,8,9]])
>>> for ctx in ctx_all: matB = npm.t(ctx, matA); print(ctx.name + ':\n', matB)
fpm:
 [[1.0 2.0 3.0]
 [4.0 5.0 6.0]
 [7.0 8.0 9.0]]
mpm:
 [[mpf('1.0') mpf('2.0') mpf('3.0')]
```

(continues on next page)

(continued from previous page)

```
 [mpf('4.0') mpf('5.0') mpf('6.0')]
 [mpf('7.0') mpf('8.0') mpf('9.0')]]
ipm:
 [[mpi('1.0', '1.0') mpi('2.0', '2.0') mpi('3.0', '3.0')]
 [mpi('4.0', '4.0') mpi('5.0', '5.0') mpi('6.0', '6.0')]
 [mpi('7.0', '7.0') mpi('8.0', '8.0') mpi('9.0', '9.0')]]
dpm:
 [[Decimal('1') Decimal('2') Decimal('3')]
 [Decimal('4') Decimal('5') Decimal('6')]
 [Decimal('7') Decimal('8') Decimal('9')]]
qpm:
 [[Fraction(1, 1) Fraction(2, 1) Fraction(3, 1)]
 [Fraction(4, 1) Fraction(5, 1) Fraction(6, 1)]
 [Fraction(7, 1) Fraction(8, 1) Fraction(9, 1)]]
gpm:
 [[mpfr('1.0') mpfr('2.0') mpfr('3.0')]
 [mpfr('4.0') mpfr('5.0') mpfr('6.0')]
 [mpfr('7.0') mpfr('8.0') mpfr('9.0')]]
apm:
 [[1.00000000000000 2.00000000000000 3.00000000000000]
 [4.00000000000000 5.00000000000000 6.00000000000000]
 [7.00000000000000 8.00000000000000 9.00000000000000]]
```

The following code creates a 3 x 3 array of random numbers of type `float`, and then converts it into arrays of the multiprecision type `ctx` in `ctx_all`:

```
>>> matA = matA = np.random.rand(3, 3)
>>> for ctx in ctx_all: matB = npm.t(ctx, matA); print(ctx.name + ':\n', matB)
fpm:
 [[0.6744434515981312 0.666353085652467 0.8970294047839491]
 [0.7559940954369713 0.9865280985239889 0.028695855364054057]
 [0.4402755692990895 0.9833733214486032 0.8904861723663713]]
mpm:
 [[mpf('0.67444345159813124') mpf('0.66635308565246698') mpf('0.89702940478394911')]
 [mpf('0.75599409543697127') mpf('0.98652809852398893') mpf('0.028695855364054057')]
 [mpf('0.44027556929908951') mpf('0.98337332144860323') mpf('0.89048617236637129')]]
ipm:
 [[mpi('0.67444345159813113', '0.67444345159813124') mpi('0.66635308565246698', '0.66635308565246709') mpi('0.
↪89702940478394899', '0.8970294047839491')]
 [mpi('0.75599409543697127', '0.75599409543697138') mpi('0.98652809852398882', '0.98652809852398893') mpi('0.
↪028695855364054054', '0.028695855364054057')]
 [mpi('0.44027556929908945', '0.44027556929908951') mpi('0.98337332144860312', '0.98337332144860323') mpi('0.
↪89048617236637129', '0.8904861723663714')]]
dpm:
 [[Decimal('0.6744434515981312') Decimal('0.666353085652467') Decimal('0.8970294047839491')]
 [Decimal('0.7559940954369713') Decimal('0.9865280985239889') Decimal('0.028695855364054057')]
 [Decimal('0.4402755692990895') Decimal('0.9833733214486032') Decimal('0.8904861723663713')]]
qpm:
 [[Fraction(6586361832013, 9765625000000) Fraction(666353085652467, 1000000000000000) Fraction(8970294047839491,␣
↪10000000000000000)]
 [Fraction(7559940954369713, 10000000000000000) Fraction(9865280985239889, 10000000000000000)␣
↪Fraction(28695855364054057, 100000000000000000)]
 [Fraction(880551138598179, 2000000000000000) Fraction(614608325905377, 625000000000000) Fraction(8904861723663713,␣
↪10000000000000000)]]
gpm:
 [[mpfr('0.67444345159813124') mpfr('0.66635308565246698') mpfr('0.89702940478394911')]
 [mpfr('0.75599409543697127') mpfr('0.98652809852398893') mpfr('0.028695855364054057')]
 [mpfr('0.44027556929908951') mpfr('0.98337332144860323') mpfr('0.89048617236637129')]]
apm:
 [[[0.674443451598131 +/- 2.39e-16] [0.666353085652467 +/- 1.31e-16] [0.897029404783949 +/- 1.20e-16]]
 [[0.755994095436971 +/- 3.81e-16] [0.986528098523989 +/- 2.95e-16] [0.0286958553640541 +/- 5.00e-17]]
 [[0.440275569299089 +/- 5.09e-16] [0.983373321448603 +/- 2.33e-16] [0.890486172366371 +/- 3.98e-16]]]
```

The following code creates a 3 x 3 array of random numbers of type `complex`, and then converts it into arrays of the multiprecision type `ctx` in `ctx_all`:

```
>>> matA = np.random.rand(3, 3) + 1j * np.random.rand(3, 3)
>>> for ctx in ctx_all: matB = npm.t(ctx, matA); print(ctx.name + ':\n', matB)
fpm:
 [[(0.567473450482438+0.7758615587166008j) (0.9943782344345623+0.32179023975050236j) (0.2756393413671573+0.
↪20943264624284674j)]
 [(0.6170089420540226+0.5969465058491088j) (0.694950179213084+0.60703126378221321j) (0.13610785528289382+0.
↪90817196992087246j)]
 [(0.09188433151601583+0.26259046235301586j) (0.2551946534651285+0.92792896573285222j) (0.7147838697585964+0.
↪043110422866829756j)]]
mpm:
 [[mpc(real='0.56747345048243802', imag='0.7758615587166008') mpc(real='0.9943782344345623', imag='0.32179023975050236
↪') mpc(real='0.27563934136715729', imag='0.20943264624284674')]
 [mpc(real='0.61700894205402257', imag='0.5969465058491088') mpc(real='0.694950179213084', imag='0.60703126378221317')␣
↪mpc(real='0.13610785528289382', imag='0.90817196992087246')]
 [mpc(real='0.091884331516015827', imag='0.26259046235301586') mpc(real='0.25519465346512848', imag='0.
↪92792896573285222') mpc(real='0.71478386975859642', imag='0.043110422866829756')]]
ipm:
 [[iv.mpc(mpi('0.56747345048243802', '0.56747345048243802'), mpi('0.7758615587166008', '0.7758615587166008'))
  iv.mpc(mpi('0.9943782344345623', '0.9943782344345623'), mpi('0.32179023975050236', '0.32179023975050236'))
  iv.mpc(mpi('0.27563934136715729', '0.27563934136715729'), mpi('0.20943264624284674', '0.20943264624284674'))]
 [iv.mpc(mpi('0.61700894205402257', '0.61700894205402257'), mpi('0.5969465058491088', '0.5969465058491088'))
  iv.mpc(mpi('0.694950179213084', '0.694950179213084'), mpi('0.60703126378221317', '0.60703126378221317'))
  iv.mpc(mpi('0.13610785528289382', '0.13610785528289382'), mpi('0.90817196992087246', '0.90817196992087246'))]
 [iv.mpc(mpi('0.091884331516015827', '0.091884331516015827'), mpi('0.26259046235301586', '0.26259046235301586'))
  iv.mpc(mpi('0.25519465346512848', '0.25519465346512848'), mpi('0.92792896573285222', '0.92792896573285222'))
  iv.mpc(mpi('0.71478386975859642', '0.71478386975859642'), mpi('0.043110422866829756', '0.043110422866829756'))]]
dpm:
 [[DecCplx('0.567473450482438 + 0.7758615587166008j') DecCplx('0.9943782344345623 + 0.32179023975050236j') DecCplx('0.
↪2756393413671573 + 0.20943264624284674j')]
 [DecCplx('0.6170089420540226 + 0.5969465058491088j') DecCplx('0.694950179213084 + 0.60703126378221321j') DecCplx('0.
↪13610785528289382 + 0.90817196992087246j')]
 [DecCplx('0.09188433151601583 + 0.26259046235301586j') DecCplx('0.2551946534651285 + 0.92792896573285222j') DecCplx('0.
↪7147838697585964 + 0.043110422866829756j')]]
qpm:
 [[QCplx('283736725241219/500000000000000 + 12122836854947/156250000000000j') QCplx('9943782344345623/1000000000000000␣
↪+ 8044755993762559/25000000000000000j')
  QCplx('2756393413671573/10000000000000000 + 10471632312142337/50000000000000000j')]
 [QCplx('3085044710270113/5000000000000000 + 373091566155693/6250000000000000j') QCplx('173737544803271/2500000000000000␣
↪+ 1517578159455533/25000000000000000j')
  QCplx('6805392764144691/50000000000000000 + 363268787968349/4000000000000000j')]
 [QCplx('9188433151601583/10000000000000000 + 13129523117650793/50000000000000000j') QCplx('510389306930257/
↪2000000000000000 + 4639644828664261/50000000000000000j')
  QCplx('1786959674396491/2500000000000000 + 10777605716707439/25000000000000000j')]]
gpm:
 [[mpc('0.56747345048243802+0.7758615587166008j') mpc('0.9943782344345623+0.32179023975050236j') mpc('0.
↪27563934136715729+0.20943264624284674j')]
 [mpc('0.61700894205402257+0.5969465058491088j') mpc('0.694950179213084+0.60703126378221317j') mpc('0.
↪13610785528289382+0.90817196992087246j')]
 [mpc('0.091884331516015827+0.26259046235301586j') mpc('0.25519465346512848+0.92792896573285222j') mpc('0.
↪71478386975859642+0.043110422866829756j')]]
apm:
 [[[0.567473450482438 +/- 2.44e-17] + [0.775861558716608 +/- 1.89e-18]j [0.994378234434562 +/- 3.03e-16] + [0.
↪321790239750502 +/- 3.61e-16]j
  [0.275639341367157 +/- 2.87e-16] + [0.209432646242847 +/- 2.58e-16]j]
 [[0.617008942054023 +/- 4.32e-16] + [0.596946505849109 +/- 2.01e-16]j [0.694950179213084 +/- 3.91e-18] + [0.
↪607031263782213 +/- 1.71e-16]j
  [0.136107855282894 +/- 1.81e-16] + [0.908171969920872 +/- 4.58e-16]j]
 [[0.0918843315160158 +/- 2.70e-17] + [0.262590462353016 +/- 1.37e-16]j [0.255194653465128 +/- 4.85e-16] + [0.
↪927928965732852 +/- 2.18e-16]j
  [0.714783869758596 +/- 4.25e-16] + [0.0431104228668298 +/- 4.38e-17]j]]
```

7.1.2 Ones on the diagonal and zeros elsewhere: numpy.eye

npm.**eye**(*N*, *M=None*, *k=0*, *dtype=<class 'float'>*, *order='C'*, ***, *device=None*, *like=None*)

Return a 2-D array with ones on the diagonal and zeros elsewhere.

See https://numpy.org/doc/stable/reference/generated/numpy.eye.html#numpy.eye for details.

The following code creates a 3 x (M=4) array with ones on the first upper (k=1) diagonal and zeros elsewhere, and then converts it into arrays of the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all: matB = npm.eye(N=3, M=4, k=1, dtype=ctx); print(ctx.name + ':\n', matB)
fpm:
 [[0.0 1.0 0.0 0.0]
 [0.0 0.0 1.0 0.0]
 [0.0 0.0 0.0 1.0]]
mpm:
 [[mpf('0.0') mpf('1.0') mpf('0.0') mpf('0.0')]
 [mpf('0.0') mpf('0.0') mpf('1.0') mpf('0.0')]
 [mpf('0.0') mpf('0.0') mpf('0.0') mpf('1.0')]]
ipm:
 [[mpi('0.0', '0.0') mpi('1.0', '1.0') mpi('0.0', '0.0') mpi('0.0', '0.0')]
 [mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('1.0', '1.0') mpi('0.0', '0.0')]
 [mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('1.0', '1.0')]]
dpm:
 [[Decimal('0.0') Decimal('1.0') Decimal('0.0') Decimal('0.0')]
 [Decimal('0.0') Decimal('0.0') Decimal('1.0') Decimal('0.0')]
 [Decimal('0.0') Decimal('0.0') Decimal('0.0') Decimal('1.0')]]
qpm:
 [[Fraction(0, 1) Fraction(1, 1) Fraction(0, 1) Fraction(0, 1)]
 [Fraction(0, 1) Fraction(0, 1) Fraction(1, 1) Fraction(0, 1)]
 [Fraction(0, 1) Fraction(0, 1) Fraction(0, 1) Fraction(1, 1)]]
gpm:
 [[mpfr('0.0') mpfr('1.0') mpfr('0.0') mpfr('0.0')]
 [mpfr('0.0') mpfr('0.0') mpfr('1.0') mpfr('0.0')]
 [mpfr('0.0') mpfr('0.0') mpfr('0.0') mpfr('1.0')]]
apm:
 [[0 1.00000000000000 0 0]
 [0 0 1.00000000000000 0]
 [0 0 0 1.00000000000000]]
```

7.1.3 Identity array: numpy.identity

npm.**identity**(*n*, *dtype=None*, ** (Keyword-only parameters separator (PEP 3102))*, *like=None*)

Return the identity array. The identity array is a square array with ones on the main diagonal.

See https://numpy.org/doc/stable/reference/generated/numpy.identity.html#numpy.identity for details.

The following code creates a 3 x 3 identity array, and then converts it into arrays of the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all: matB = npm.identity(n=3, dtype=ctx); print(ctx.name + ':\n', matB)
fpm:
 [[1.0 0.0 0.0]
 [0.0 1.0 0.0]
 [0.0 0.0 1.0]]
mpm:
 [[mpf('1.0') mpf('0.0') mpf('0.0')]
 [mpf('0.0') mpf('1.0') mpf('0.0')]
 [mpf('0.0') mpf('0.0') mpf('1.0')]]
ipm:
 [[mpi('1.0', '1.0') mpi('0.0', '0.0') mpi('0.0', '0.0')]
```

(continues on next page)

(continued from previous page)

```
 [mpi('0.0', '0.0') mpi('1.0', '1.0') mpi('0.0', '0.0')]
 [mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('1.0', '1.0')]]
dpm:
 [[Decimal('1.0') Decimal('0.0') Decimal('0.0')]
 [Decimal('0.0') Decimal('1.0') Decimal('0.0')]
 [Decimal('0.0') Decimal('0.0') Decimal('1.0')]]
qpm:
 [[Fraction(1, 1) Fraction(0, 1) Fraction(0, 1)]
 [Fraction(0, 1) Fraction(1, 1) Fraction(0, 1)]
 [Fraction(0, 1) Fraction(0, 1) Fraction(1, 1)]]
gpm:
 [[mpfr('1.0') mpfr('0.0') mpfr('0.0')]
 [mpfr('0.0') mpfr('1.0') mpfr('0.0')]
 [mpfr('0.0') mpfr('0.0') mpfr('1.0')]]
apm:
 [[1.00000000000000 0 0]
 [0 1.00000000000000 0]
 [0 0 1.00000000000000]]
```

7.1.4 Array of ones: numpy.ones

npm.**ones**(*shape*, *dtype=None*, *order='C'*, ***, *device=None*, *like=None*)

Return a new array of given shape and type, filled with ones.

See https://numpy.org/doc/stable/reference/generated/numpy.ones.html#numpy.ones for details.

The following code creates a 1 x 3 array of ones, and then converts it into arrays of the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all: matB = npm.ones(shape=(3,), dtype=ctx); print(ctx.name + ':\n', matB)
fpm:
 [1.0 1.0 1.0]
mpm:
 [mpf('1.0') mpf('1.0') mpf('1.0')]
ipm:
 [mpi('1.0', '1.0') mpi('1.0', '1.0') mpi('1.0', '1.0')]
dpm:
 [Decimal('1') Decimal('1') Decimal('1')]
qpm:
 [Fraction(1, 1) Fraction(1, 1) Fraction(1, 1)]
gpm:
 [mpfr('1.0') mpfr('1.0') mpfr('1.0')]
apm:
 [1.00000000000000 1.00000000000000 1.00000000000000]
```

7.1.5 Array of ones: numpy.ones_like

npm.**ones_like**(*a*, *dtype=None*, *order='K'*, *subok=True*, *shape=None*, ***, *device=None*)

Return an array of ones with the same shape and type as a given array.

See https://numpy.org/doc/stable/reference/generated/numpy.ones_like.html#numpy.ones_like for details.

The following code creates a 2 x 3 array of ones, based on another ndarray, and then converts it into arrays of the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> x = np.arange(6); x = x.reshape((2, 3))
>>> for ctx in ctx_all: matB = npm.ones_like(x, dtype=ctx); print(ctx.name + ':\n', matB)
fpm:
```

(continues on next page)

(continued from previous page)

```
 [[1.0 1.0 1.0]
 [1.0 1.0 1.0]]
mpm:
 [[mpf('1.0') mpf('1.0') mpf('1.0')]
 [mpf('1.0') mpf('1.0') mpf('1.0')]]
ipm:
 [[mpi('1.0', '1.0') mpi('1.0', '1.0') mpi('1.0', '1.0')]
 [mpi('1.0', '1.0') mpi('1.0', '1.0') mpi('1.0', '1.0')]]
dpm:
 [[Decimal('1') Decimal('1') Decimal('1')]
 [Decimal('1') Decimal('1') Decimal('1')]]
qpm:
 [[Fraction(1, 1) Fraction(1, 1) Fraction(1, 1)]
 [Fraction(1, 1) Fraction(1, 1) Fraction(1, 1)]]
gpm:
 [[mpfr('1.0') mpfr('1.0') mpfr('1.0')]
 [mpfr('1.0') mpfr('1.0') mpfr('1.0')]]
apm:
 [[1.00000000000000 1.00000000000000 1.00000000000000]
 [1.00000000000000 1.00000000000000 1.00000000000000]]
```

7.1.6 Array of zeros: numpy.zeros

npm.**zeros**(*shape*, *dtype=None*, *order='C'*, ***, *device=None*, *like=None*)

Return a new array of given shape and type, filled with zeros.

See https://numpy.org/doc/stable/reference/generated/numpy.zeros.html#numpy.zeros for details.

The following code creates a 1 x 3 array of zeros, and then converts it into arrays of the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all: matB = npm.zeros(shape=(3,), dtype=ctx); print(ctx.name + ':\n', matB)
fpm:
 [1.0 1.0 1.0]
mpm:
 [mpf('1.0') mpf('1.0') mpf('1.0')]
ipm:
 [mpi('1.0', '1.0') mpi('1.0', '1.0') mpi('1.0', '1.0')]
dpm:
 [Decimal('1') Decimal('1') Decimal('1')]
qpm:
 [Fraction(1, 1) Fraction(1, 1) Fraction(1, 1)]
gpm:
 [mpfr('1.0') mpfr('1.0') mpfr('1.0')]
apm:
 [1.00000000000000 1.00000000000000 1.00000000000000]
```

7.1.7 Array of zeros: numpy.zeros_like

npm.**zeros_like**(*a*, *dtype=None*, *order='K'*, *subok=True*, *shape=None*, ***, *device=None*)

Return an array of zeros with the same shape and type as a given array.

See https://numpy.org/doc/stable/reference/generated/numpy.zeros_like.html#numpy.zeros_like for details.

The following code creates a 2 x 3 array of zeros, based on another ndarray, and then converts it into arrays of the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> x = np.arange(6); x = x.reshape((2, 3))
```

(continues on next page)

(continued from previous page)

```
>>> for ctx in ctx_all: matB = npm.zeros_like(x, dtype=ctx); print(ctx.name + ':\n', matB)
fpm:
 [[0.0 0.0 0.0]
 [0.0 0.0 0.0]]
mpm:
 [[mpf('0.0') mpf('0.0') mpf('0.0')]
 [mpf('0.0') mpf('0.0') mpf('0.0')]]
ipm:
 [[mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('0.0', '0.0')]
 [mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('0.0', '0.0')]]
dpm:
 [[Decimal('0') Decimal('0') Decimal('0')]
 [Decimal('0') Decimal('0') Decimal('0')]]
qpm:
 [[Fraction(0, 1) Fraction(0, 1) Fraction(0, 1)]
 [Fraction(0, 1) Fraction(0, 1) Fraction(0, 1)]]
gpm:
 [[mpfr('0.0') mpfr('0.0') mpfr('0.0')]
 [mpfr('0.0') mpfr('0.0') mpfr('0.0')]]
apm:
 [[0 0 0]
 [0 0 0]]
```

7.1.8 Array of constant: numpy.full

npm.full(*shape*, *fill_value*, *dtype=None*, *order='C'*, ***, *device=None*, *like=None*)

Return a new array of given shape and type, filled with fill_value.

See https://numpy.org/doc/stable/reference/generated/numpy.full.html#numpy.full for details.

The following code creates a 2 x 2 array of tens, using the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all: matB = npm.full(shape=(2,2), fill_value=ctx.t(10)); print(ctx.name + ':\n', matB)
fpm:
 [[10. 10.]
 [10. 10.]]
mpm:
 [[mpf('10.0') mpf('10.0')]
 [mpf('10.0') mpf('10.0')]]
ipm:
 [[mpi('10.0', '10.0') mpi('10.0', '10.0')]
 [mpi('10.0', '10.0') mpi('10.0', '10.0')]]
dpm:
 [[Decimal('10') Decimal('10')]
 [Decimal('10') Decimal('10')]]
qpm:
 [[Fraction(10, 1) Fraction(10, 1)]
 [Fraction(10, 1) Fraction(10, 1)]]
gpm:
 [[mpfr('10.0') mpfr('10.0')]
 [mpfr('10.0') mpfr('10.0')]]
apm:
 [[10.000000000000000 10.000000000000000]
 [10.000000000000000 10.000000000000000]]
```

The following code creates a 2 x 2 array of +`inf`, using the multiprecision type `ctx` in `ctx_all`. Note that `qpm` is excluded, sine it does not support `inf`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_fp = [fpm, mpm, ipm, dpm, gpm, apm]
>>> for ctx in ctx_fp: matB = npm.full(shape=(2,2), fill_value=ctx.inf); print(ctx.name + ':\n', matB)
```

(continues on next page)

(continued from previous page)

```
fpm:
 [[inf inf]
 [inf inf]]
mpm:
 [[mpf('+inf') mpf('+inf')]
 [mpf('+inf') mpf('+inf')]]
ipm:
 [[mpi('+inf', '+inf') mpi('+inf', '+inf')]
 [mpi('+inf', '+inf') mpi('+inf', '+inf')]]
dpm:
 [[Decimal('Infinity') Decimal('Infinity')]
 [Decimal('Infinity') Decimal('Infinity')]]
gpm:
 [[mpfr('inf') mpfr('inf')]
 [mpfr('inf') mpfr('inf')]]
apm:
 [[[+/- inf] [+/- inf]]
 [[+/- inf] [+/- inf]]]
```

The following code creates a 2 x 2 array of `nan`, using the multiprecision type `ctx` in `ctx_all`. Note that `qpm` is excluded, sine it does not support `nan`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_fp = [fpm, mpm, ipm, dpm, gpm, apm]
>>> for ctx in ctx_fp: matB = npm.full(shape=(2,2), fill_value=ctx.nan); print(ctx.name + ':\n', matB)
fpm:
 [[nan nan]
 [nan nan]]
mpm:
 [[mpf('nan') mpf('nan')]
 [mpf('nan') mpf('nan')]]
ipm:
 [[mpi('nan', 'nan') mpi('nan', 'nan')]
 [mpi('nan', 'nan') mpi('nan', 'nan')]]
dpm:
 [[Decimal('NaN') Decimal('NaN')]
 [Decimal('NaN') Decimal('NaN')]]
gpm:
 [[mpfr('nan') mpfr('nan')]
 [mpfr('nan') mpfr('nan')]]
apm:
 [[nan nan]
 [nan nan]]
```

7.1.9 Array of constant: numpy.full_like

npm.**full_like**(*a*, *fill_value*, *dtype=None*, *order='K'*, *subok=True*, *shape=None*, ***, *device=None*)

Return a full array with the same shape and type as a given array.

See https://numpy.org/doc/stable/reference/generated/numpy.full_like.html#numpy.full_like for details.

The following code creates a 2 x 2 array of tens, using the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> x = np.arange(6); x = x.reshape((2, 3))
>>> for ctx in ctx_all: matB = npm.full_like(x, fill_value=10.0, dtype=ctx); print(ctx.name + ':\n', matB)
fpm:
 [[10.0 10.0 10.0]
 [10.0 10.0 10.0]]
mpm:
 [[mpf('10.0') mpf('10.0') mpf('10.0')]
 [mpf('10.0') mpf('10.0') mpf('10.0')]]
```

(continues on next page)

(continued from previous page)

```
ipm:
 [[mpi('10.0', '10.0') mpi('10.0', '10.0') mpi('10.0', '10.0')]
 [mpi('10.0', '10.0') mpi('10.0', '10.0') mpi('10.0', '10.0')]]
dpm:
 [[Decimal('10') Decimal('10') Decimal('10')]
 [Decimal('10') Decimal('10') Decimal('10')]]
qpm:
 [[Fraction(10, 1) Fraction(10, 1) Fraction(10, 1)]
 [Fraction(10, 1) Fraction(10, 1) Fraction(10, 1)]]
gpm:
 [[mpfr('10.0') mpfr('10.0') mpfr('10.0')]
 [mpfr('10.0') mpfr('10.0') mpfr('10.0')]]
apm:
 [[10.0000000000000 10.0000000000000 10.0000000000000]
 [10.0000000000000 10.0000000000000 10.0000000000000]]
```

7.1.10 Array with ones at and below the given diagonal and zeros elsewhere: numpy.tri

npm.tri(*N*, *M=None*, *k=0*, *dtype=<class 'float'>*, ***, *like=None*)

Create an array with ones at and below the given diagonal and zeros elsewhere.

See https://numpy.org/doc/stable/reference/generated/numpy.tri.html#numpy.tri for details

An array with ones at and below the given diagonal and zeros elsewhere.

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all: x = npm.tri(N=3, M=5, k=-1, dtype=ctx); print(ctx.name + ':\n', x)
fpm:
 [[0.0 0.0 0.0 0.0 0.0]
 [1.0 0.0 0.0 0.0 0.0]
 [1.0 1.0 0.0 0.0 0.0]]
mpm:
 [[mpf('0.0') mpf('0.0') mpf('0.0') mpf('0.0') mpf('0.0')]
 [mpf('1.0') mpf('0.0') mpf('0.0') mpf('0.0') mpf('0.0')]
 [mpf('1.0') mpf('1.0') mpf('0.0') mpf('0.0') mpf('0.0')]]
ipm:
 [[mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('0.0', '0.0')]
 [mpi('1.0', '1.0') mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('0.0', '0.0')]
 [mpi('1.0', '1.0') mpi('1.0', '1.0') mpi('0.0', '0.0') mpi('0.0', '0.0') mpi('0.0', '0.0')]]
dpm:
 [[Decimal('0.0') Decimal('0.0') Decimal('0.0') Decimal('0.0') Decimal('0.0')]
 [Decimal('1.0') Decimal('0.0') Decimal('0.0') Decimal('0.0') Decimal('0.0')]
 [Decimal('1.0') Decimal('1.0') Decimal('0.0') Decimal('0.0') Decimal('0.0')]]
qpm:
 [[Fraction(0, 1) Fraction(0, 1) Fraction(0, 1) Fraction(0, 1) Fraction(0, 1)]
 [Fraction(1, 1) Fraction(0, 1) Fraction(0, 1) Fraction(0, 1) Fraction(0, 1)]
 [Fraction(1, 1) Fraction(1, 1) Fraction(0, 1) Fraction(0, 1) Fraction(0, 1)]]
gpm:
 [[mpfr('0.0') mpfr('0.0') mpfr('0.0') mpfr('0.0') mpfr('0.0')]
 [mpfr('1.0') mpfr('0.0') mpfr('0.0') mpfr('0.0') mpfr('0.0')]
 [mpfr('1.0') mpfr('1.0') mpfr('0.0') mpfr('0.0') mpfr('0.0')]]
apm:
 [[0 0 0 0 0]
 [1.00000000000000 0 0 0 0]
 [1.00000000000000 1.00000000000000 0 0 0]]
```

7.2 Numpy array creation from existing data

7.2.1 Array from object: numpy.array

npm.**array**(*object*, *dtype=None*, ***, *copy=True*, *order='K'*, *subok=False*, *ndmin=0*, *like=None*)

Return an array from any object exposing the array interface, any object whose __array__ method returns an array, or any (nested) sequence. If object is a scalar, a 0-dimensional array containing object is returned.

The following code creates a 1-dimensional array from a 1-dimensionallist, using the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all: matB = npm.array([1, 2, 3], dtype=ctx); print(ctx.name + ':\n', matB)
fpm:
 [1.0 2.0 3.0]
mpm:
 [mpf('1.0') mpf('2.0') mpf('3.0')]
ipm:
 [mpi('1.0', '1.0') mpi('2.0', '2.0') mpi('3.0', '3.0')]
dpm:
 [Decimal('1') Decimal('2') Decimal('3')]
qpm:
 [Fraction(1, 1) Fraction(2, 1) Fraction(3, 1)]
gpm:
 [mpfr('1.0') mpfr('2.0') mpfr('3.0')]
apm:
 [1.00000000000000 2.00000000000000 3.00000000000000]
```

The following code creates a 2-dimensional array from a 2-dimensional list, using the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all: matB = npm.array([[1, 2], [3, 4]], dtype=ctx); print(ctx.name + ':\n', matB)
fpm:
 [[1.0 2.0]
 [3.0 4.0]]
mpm:
 [[mpf('1.0') mpf('2.0')]
 [mpf('3.0') mpf('4.0')]]
ipm:
 [[mpi('1.0', '1.0') mpi('2.0', '2.0')]
 [mpi('3.0', '3.0') mpi('4.0', '4.0')]]
dpm:
 [[Decimal('1') Decimal('2')]
 [Decimal('3') Decimal('4')]]
qpm:
 [[Fraction(1, 1) Fraction(2, 1)]
 [Fraction(3, 1) Fraction(4, 1)]]
gpm:
 [[mpfr('1.0') mpfr('2.0')]
 [mpfr('3.0') mpfr('4.0')]]
apm:
 [[1.00000000000000 2.00000000000000]
 [3.00000000000000 4.00000000000000]]
```

The following code creates a 2-dimensional array from a 1-dimensional list with the keyword `ndmin`=2, using the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all: matB = npm.array([1, 2, 3], dtype=ctx, ndmin=2); print(ctx.name + ':\n', matB)
fpm:
 [[1.0 2.0 3.0]]
mpm:
```

(continues on next page)

(continued from previous page)

```
 [[mpf('1.0') mpf('2.0') mpf('3.0')]]
ipm:
 [[mpi('1.0', '1.0') mpi('2.0', '2.0') mpi('3.0', '3.0')]]
dpm:
 [[Decimal('1') Decimal('2') Decimal('3')]]
qpm:
 [[Fraction(1, 1) Fraction(2, 1) Fraction(3, 1)]]
gpm:
 [[mpfr('1.0') mpfr('2.0') mpfr('3.0')]]
apm:
 [[1.00000000000000 2.00000000000000 3.00000000000000]]
```

The following code creates a 1-dimensional complex array from a 1-dimensional list of integers with the keyword `dtype=complex`, using the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> matA = np.array([1, 2, 3], dtype=complex)
>>> for ctx in ctx_all: matB = npm.t(ctx, matA); print(ctx.name + ':\n', matB)
fpm:
 [(1+0j) (2+0j) (3+0j)]
mpm:
 [mpc(real='1.0', imag='0.0') mpc(real='2.0', imag='0.0') mpc(real='3.0', imag='0.0')]
ipm:
 [iv.mpc(mpi('1.0', '1.0'), mpi('0.0', '0.0')) iv.mpc(mpi('2.0', '2.0'), mpi('0.0', '0.0')) iv.mpc(mpi('3.0', '3.0'),␣
 ↪mpi('0.0', '0.0'))]
dpm:
 [DecCplx('1.0 + 0.0j') DecCplx('2.0 + 0.0j') DecCplx('3.0 + 0.0j')]
qpm:
 [QCplx('1 + 0j') QCplx('2 + 0j') QCplx('3 + 0j')]
gpm:
 [mpc('1.0+0.0j') mpc('2.0+0.0j') mpc('3.0+0.0j')]
apm:
 [1.00000000000000 2.00000000000000 3.00000000000000]
```

7.2.2 Array from object: numpy.asarray

`npm.asarray`(*a*, *dtype=None*, *order=None*, ***, *device=None*, *copy=None*, *like=None*)

Return an array from input data in any form that can be converted to an array. This includes lists, lists of tuples, tuples, tuples of tuples, tuples of lists and ndarrays. Existing arrays are not copied. If dtype is set, array is copied only if dtype does not match. Contrary to asanyarray, ndarray subclasses are not passed through.

See https://numpy.org/doc/stable/reference/generated/numpy.asarray.html#numpy.asarray for details.

The following code creates a 2-dimensional array from a 2-dimensional list, using the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> matA = npm.asarray([[1, 2], [3, 4]])
>>> for ctx in ctx_all: matB = npm.t(ctx, matA); print(ctx.name + ':\n', matB)
fpm:
 [[1.0 2.0]
 [3.0 4.0]]
mpm:
 [[mpf('1.0') mpf('2.0')]
 [mpf('3.0') mpf('4.0')]]
ipm:
 [[mpi('1.0', '1.0') mpi('2.0', '2.0')]
 [mpi('3.0', '3.0') mpi('4.0', '4.0')]]
dpm:
 [[Decimal('1') Decimal('2')]
 [Decimal('3') Decimal('4')]]
```

(continues on next page)

(continued from previous page)

```
qpm:
 [[Fraction(1, 1) Fraction(2, 1)]
 [Fraction(3, 1) Fraction(4, 1)]]
gpm:
 [[mpfr('1.0') mpfr('2.0')]
 [mpfr('3.0') mpfr('4.0')]]
apm:
 [[1.00000000000000 2.00000000000000]
 [3.00000000000000 4.00000000000000]]

>>> # Existing arrays are not copied:
>>> a = npm.array([1, 2])
>>> np.asarray(a) is a
True

>>> # If dtype is set, array is copied only if dtype does not match:
>>> a = np.array([1, 2], dtype=np.float32)
>>> npm.asarray(a, dtype=np.float32) is a
True
>>> np.asarray(a, dtype=np.float64) is a
False

>>> # Contrary to asanyarray, ndarray subclasses are not passed through:
>>> issubclass(np.recarray, np.ndarray)
True
>>> a = np.array([(1.0, 2), (3.0, 4)], dtype='f4,i4').view(np.recarray)
>>> npm.asarray(a) is a
False
>>> np.asanyarray(a) is a
True
```

7.2.3 Array from object: numpy.asanyarray

npm.asanyarray(*a*, *dtype=None*, *order=None*, ***, *device=None*, *copy=None*, *like=None*)

Convert the input to an ndarray, but pass ndarray subclasses through. Return an array from input data in any form that can be converted to an array. This includes lists, lists of tuples, tuples, tuples of tuples, tuples of lists and ndarrays.

See https://numpy.org/doc/stable/reference/generated/numpy.asanyarray.html#numpy.asanyarray for details.

The following code creates a 2-dimensional array from a 2-dimensional list, using the multiprecision type `ctx` in `ctx_all`:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> matA = npm.asanyarray([[1, 2], [3, 4]])
>>> for ctx in ctx_all: matB = npm.t(ctx, matA); print(ctx.name + ':\n', matB)
fpm:
 [[1.0 2.0]
 [3.0 4.0]]
mpm:
 [[mpf('1.0') mpf('2.0')]
 [mpf('3.0') mpf('4.0')]]
ipm:
 [[mpi('1.0', '1.0') mpi('2.0', '2.0')]
 [mpi('3.0', '3.0') mpi('4.0', '4.0')]]
dpm:
 [[Decimal('1') Decimal('2')]
 [Decimal('3') Decimal('4')]]
qpm:
 [[Fraction(1, 1) Fraction(2, 1)]
 [Fraction(3, 1) Fraction(4, 1)]]
gpm:
 [[mpfr('1.0') mpfr('2.0')]
 [mpfr('3.0') mpfr('4.0')]]
```

(continues on next page)

(continued from previous page)

```
apm:
 [[1.00000000000000 2.00000000000000]
 [3.00000000000000 4.00000000000000]]

>>> # Contrary to asyarray, ndarray subclasses are passed through as-is:
>>> a = np.array([(1., 2), (3., 4)], dtype='f4,i4').view(np.recarray)
>>> np.asanyarray(a) is a
True
```

7.2.4 Array from data in a text or binary file: numpy.fromfile

npm.**fromfile**(*file*, *dtype=float*, *count=-1*, *sep=''*, *offset=0*, ***, *like=None*)

Construct an array from data in a text or binary file. A highly efficient way of reading binary data with a known data-type, as well as parsing simply formatted text files. Data written using the tofile method can be read using this function.

See https://numpy.org/doc/stable/reference/generated/numpy.fromfile.html#numpy.fromfile for details.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> # Construct an ndarray:
>>> dt1 = np.dtype([('time', [('min', np.int64), ('sec', np.int64)]), ('temp', float)])
>>> x = np.zeros((1,), dtype=dt1)
>>> x['time']['min'] = 10; x['temp'] = 98.25
>>> x
array([((10, 0), 98.25)], dtype=[('time', [('min', '<i8'), ('sec', '<i8')]), ('temp', '<f8')])

>>> # Save the raw data to disk:
>>> import tempfile
>>> fname = tempfile.mkstemp()[1]
>>> x.tofile(fname)

>>> # Read the raw data from disk:
>>> np.fromfile(fname, dtype=dt1)
array([((10, 0), 98.25)], dtype=[('time', [('min', '<i8'), ('sec', '<i8')]), ('temp', '<f8')])

>>> # The recommended way to store and load data:
>>> np.save(fname, x)
>>> np.load(fname + '.npy')
array([((10, 0), 98.25)], dtype=[('time', [('min', '<i8'), ('sec', '<i8')]), ('temp', '<f8')])
```

7.2.5 Array from a function: numpy.fromfunction

npm.**fromfunction**(*function*, *shape*, ***, *dtype=<class 'float'>*, *like=None*, ***kwargs*)

Construct an array by executing a function over each coordinate. The resulting array therefore has a value `fn(x, y, z)` at coordinate `(x, y, z)`.

See https://numpy.org/doc/stable/reference/generated/numpy.fromfunction.html#numpy.fromfunction for details.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> np.fromfunction(lambda i, j: i, (2, 2), dtype=float)
array([[0., 0.],
   [1., 1.]])

>>> np.fromfunction(lambda i, j: j, (2, 2), dtype=float)
array([[0., 1.],
       [0., 1.]])

>>> np.fromfunction(lambda i, j: i == j, (3, 3), dtype=int)
array([[ True, False, False],
       False,  True, False],
```

(continues on next page)

(continued from previous page)

```
       False, False,  True]])

>>> np.fromfunction(lambda i, j: i + j, (3, 3), dtype=int)
array([[0, 1, 2],
       [1, 2, 3],
       [2, 3, 4]])
```

7.2.6 Array from an iterable object: numpy.fromiter

npm.**fromiter**(*iter*, *dtype*, *count=-1*, ***, *like=None*)

Create a new 1-dimensional array from an iterable object.

See https://numpy.org/doc/stable/reference/generated/numpy.fromiter.html#numpy.fromiter for details.

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all:
...     iterable = (ctx.square(x) for x in range(5))
...     x = npm.fromiter(iterable, object)
...     print(ctx.name + ':\n', x)
fpm:
 [0.0 1.0 4.0 9.0 16.0]
mpm:
 [mpf('0.0') mpf('1.0') mpf('4.0') mpf('9.0') mpf('16.0')]
ipm:
 [mpi('0.0', '0.0') mpi('1.0', '1.0') mpi('4.0', '4.0') mpi('9.0', '9.0') mpi('16.0', '16.0')]
dpm:
 [Decimal('0') Decimal('1') Decimal('4') Decimal('9') Decimal('16')]
qpm:
 [Fraction(0, 1) Fraction(1, 1) Fraction(4, 1) Fraction(9, 1) Fraction(16, 1)]
gpm:
 [mpfr('0.0') mpfr('1.0') mpfr('4.0') mpfr('9.0') mpfr('16.0')]
apm:
 [0 1.00000000000000 4.00000000000000 9.00000000000000 16.0000000000000]
```

A carefully constructed subarray dtype will lead to higher dimensional results:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all:
...     iterable = ((ctx.t(x), ctx.square(x)) for x in range(5))
...     x = npm.fromiter(iterable, dtype=np.dtype((object, 2)))
...     print(ctx.name + ':\n', x)
fpm:
 [[0.0 0.0]
 [1.0 1.0]
 [2.0 4.0]
 [3.0 9.0]
 [4.0 16.0]]
mpm:
 [[mpf('0.0') mpf('0.0')]
 [mpf('1.0') mpf('1.0')]
 [mpf('2.0') mpf('4.0')]
 [mpf('3.0') mpf('9.0')]
 [mpf('4.0') mpf('16.0')]]
ipm:
 [[mpi('0.0', '0.0') mpi('0.0', '0.0')]
 [mpi('1.0', '1.0') mpi('1.0', '1.0')]
 [mpi('2.0', '2.0') mpi('4.0', '4.0')]
 [mpi('3.0', '3.0') mpi('9.0', '9.0')]
 [mpi('4.0', '4.0') mpi('16.0', '16.0')]]
dpm:
 [[Decimal('0') Decimal('0')]
```

(continues on next page)

(continued from previous page)

```
 [Decimal('1') Decimal('1')]
 [Decimal('2') Decimal('4')]
 [Decimal('3') Decimal('9')]
 [Decimal('4') Decimal('16')]]
qpm:
 [[Fraction(0, 1) Fraction(0, 1)]
 [Fraction(1, 1) Fraction(1, 1)]
 [Fraction(2, 1) Fraction(4, 1)]
 [Fraction(3, 1) Fraction(9, 1)]
 [Fraction(4, 1) Fraction(16, 1)]]
gpm:
 [[mpfr('0.0') mpfr('0.0')]
 [mpfr('1.0') mpfr('1.0')]
 [mpfr('2.0') mpfr('4.0')]
 [mpfr('3.0') mpfr('9.0')]
 [mpfr('4.0') mpfr('16.0')]]
apm:
 [[0 0]
 [1.00000000000000 1.00000000000000]
 [2.00000000000000 4.00000000000000]
 [3.00000000000000 9.00000000000000]
 [4.00000000000000 16.0000000000000]]
```

7.2.7 Array from text data in a string: numpy.fromstring

npm.**fromstring**(*string*, *dtype=float*, *count=-1*, ***, *sep*, *like=None*)

Create a new 1-dimensional array initialized from text data in a string.

See https://numpy.org/doc/stable/reference/generated/numpy.fromstring.html#numpy.fromstring for details.

Real numbers:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all:
...     x = npm.fromstring('1.1, 2.1', dtype=float, sep=',')
...     matB = npm.t(ctx, x)
...     print(ctx.name + ':\n', matB)
fpm:
 [1.1 2.1]
mpm:
 [mpf('1.1000000000000001') mpf('2.1000000000000001')]
ipm:
 [mpi('1.0999999999999999', '1.1000000000000001') mpi('2.0999999999999996', '2.1000000000000001')]
dpm:
 [Decimal('1.1') Decimal('2.1')]
qpm:
 [Fraction(11, 10) Fraction(21, 10)]
gpm:
 [mpfr('1.1000000000000001') mpfr('2.1000000000000001')]
apm:
 [[1.10000000000000 +/- 3.56e-16] [2.10000000000000 +/- 8.00e-16]]
```

Complex numbers:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]
>>> for ctx in ctx_all:
...     x = npm.fromstring('1+3j, 2+4j', dtype=complex, sep=',')
...     matB = npm.t(ctx, x)
...     print(ctx.name + ':\n', matB)
fpm:
 [(1+3j) (2+4j)]
```

(continues on next page)

(continued from previous page)

```
mpm:
 [mpc(real='1.0', imag='3.0') mpc(real='2.0', imag='4.0')]
ipm:
 [iv.mpc(mpi('1.0', '1.0'), mpi('3.0', '3.0')) iv.mpc(mpi('2.0', '2.0'), mpi('4.0', '4.0'))]
dpm:
 [DecCplx('1.0 + 3.0j') DecCplx('2.0 + 4.0j')]
qpm:
 [QCplx('1 + 3j') QCplx('2 + 4j')]
gpm:
 [mpc('1.0+3.0j') mpc('2.0+4.0j')]
apm:
 [1.00000000000000 + 3.00000000000000j 2.00000000000000 + 4.00000000000000j]
```

7.2.8 Array from text data in a formatted file: numpy.loadtxt

npm.**loadtxt**(*fname*, *dtype=<class 'float'>*, *comments='#'*, *delimiter=None*, *converters=None*, *skiprows=0*, *usecols=None*, *unpack=False*, *ndmin=0*, *encoding=None*, *max_rows=None*, ***, *quotechar=None*, *like=None*)

Load data from a text file.

See https://numpy.org/doc/stable/reference/generated/numpy.loadtxt.html#numpy.loadtxt for details.

This function aims to be a fast reader for simply formatted files. The genfromtxt function provides more sophisticated handling of, e.g., lines with missing values.

Each row in the input text file must have the same number of values to be able to read all values. If all rows do not have same number of values, a subset of up to n columns (where n is the least number of values present in all rows) can be read by specifying the columns via usecols.

```
import os
import xlcalcnet.userpaths as userpaths
for ctx in ctx_all:
    to_ctx = npm.vectorize(ctx.t, otypes=[object])

    csvpath = os.sep.join([userpaths.get_my_documents(), \
        'DataXlCalcNet', 'DataExamples', 'MainExamples', 'CSV'])
    csvfile = 'Hald.csv'
    csvname = os.sep.join([csvpath, csvfile])

    with open(csvname) as f: header = f.readline().strip('\n')
    print('Header: ', header)

    nd_data = npm.loadtxt(csvname, dtype=np.float64, delimiter=',', \
    skiprows=1, usecols=(0,1,2,3,4))
    A = to_ctx(nd_data)
    N = len(A)
    print('N:', N)
    print('A: \n', A)
    print()

Header:  x1,x2,x3,x4,y
N: 13
A:
 [[7.0 26.0 6.0 60.0 78.5]
 [1.0 29.0 15.0 52.0 74.3]
 [11.0 56.0 8.0 20.0 104.3]
 [11.0 31.0 8.0 47.0 87.6]
 [7.0 52.0 6.0 33.0 95.9]
 [11.0 55.0 9.0 22.0 109.2]
 [3.0 71.0 17.0 6.0 102.7]
 [1.0 31.0 22.0 44.0 72.5]
 [2.0 54.0 18.0 22.0 93.1]
 [21.0 47.0 4.0 26.0 115.9]
 [1.0 40.0 23.0 34.0 83.8]
```

(continues on next page)

(continued from previous page)

```
 [11.0 66.0 9.0 12.0 113.3]
 [10.0 68.0 8.0 12.0 109.4]]

Header:  x1,x2,x3,x4,y
N: 13
A:
 [[mpf('7.0') mpf('26.0') mpf('6.0') mpf('60.0') mpf('78.5')]
 [mpf('1.0') mpf('29.0') mpf('15.0') mpf('52.0') mpf('74.299999999999997')]
 [mpf('11.0') mpf('56.0') mpf('8.0') mpf('20.0') mpf('104.3')]
 [mpf('11.0') mpf('31.0') mpf('8.0') mpf('47.0') mpf('87.599999999999994')]
 [mpf('7.0') mpf('52.0') mpf('6.0') mpf('33.0') mpf('95.900000000000006')]
 [mpf('11.0') mpf('55.0') mpf('9.0') mpf('22.0') mpf('109.2')]
 [mpf('3.0') mpf('71.0') mpf('17.0') mpf('6.0') mpf('102.7')]
 [mpf('1.0') mpf('31.0') mpf('22.0') mpf('44.0') mpf('72.5')]
 [mpf('2.0') mpf('54.0') mpf('18.0') mpf('22.0') mpf('93.099999999999994')]
 [mpf('21.0') mpf('47.0') mpf('4.0') mpf('26.0') mpf('115.90000000000001')]
 [mpf('1.0') mpf('40.0') mpf('23.0') mpf('34.0') mpf('83.799999999999997')]
 [mpf('11.0') mpf('66.0') mpf('9.0') mpf('12.0') mpf('113.3')]
 [mpf('10.0') mpf('68.0') mpf('8.0') mpf('12.0') mpf('109.40000000000001')]]

Header:  x1,x2,x3,x4,y
N: 13
A:
 [[mpi('7.0', '7.0') mpi('26.0', '26.0') mpi('6.0', '6.0') mpi('60.0', '60.0') mpi('78.5', '78.5')]
 [mpi('1.0', '1.0') mpi('29.0', '29.0') mpi('15.0', '15.0') mpi('52.0', '52.0') mpi('74.299999999999997', '74.
↪3000000000000011')]
 [mpi('11.0', '11.0') mpi('56.0', '56.0') mpi('8.0', '8.0') mpi('20.0', '20.0') mpi('104.3', '104.30000000000001')]
 [mpi('11.0', '11.0') mpi('31.0', '31.0') mpi('8.0', '8.0') mpi('47.0', '47.0') mpi('87.599999999999994', '87.
↪6000000000000009')]
 [mpi('7.0', '7.0') mpi('52.0', '52.0') mpi('6.0', '6.0') mpi('33.0', '33.0') mpi('95.899999999999991', '95.
↪9000000000000006')]
 [mpi('11.0', '11.0') mpi('55.0', '55.0') mpi('9.0', '9.0') mpi('22.0', '22.0') mpi('109.19999999999999', '109.2')]
 [mpi('3.0', '3.0') mpi('71.0', '71.0') mpi('17.0', '17.0') mpi('6.0', '6.0') mpi('102.69999999999999', '102.7')]
 [mpi('1.0', '1.0') mpi('31.0', '31.0') mpi('22.0', '22.0') mpi('44.0', '44.0') mpi('72.5', '72.5')]
 [mpi('2.0', '2.0') mpi('54.0', '54.0') mpi('18.0', '18.0') mpi('22.0', '22.0') mpi('93.099999999999994', '93.
↪1000000000000009')]
 [mpi('21.0', '21.0') mpi('47.0', '47.0') mpi('4.0', '4.0') mpi('26.0', '26.0') mpi('115.89999999999999', '115.
↪90000000000001')]
 [mpi('1.0', '1.0') mpi('40.0', '40.0') mpi('23.0', '23.0') mpi('34.0', '34.0') mpi('83.799999999999997', '83.
↪8000000000000011')]
 [mpi('11.0', '11.0') mpi('66.0', '66.0') mpi('9.0', '9.0') mpi('12.0', '12.0') mpi('113.3', '113.30000000000001')]
 [mpi('10.0', '10.0') mpi('68.0', '68.0') mpi('8.0', '8.0') mpi('12.0', '12.0') mpi('109.39999999999999', '109.
↪40000000000001')]]

Header:  x1,x2,x3,x4,y
N: 13
A:
 [[Decimal('7.0') Decimal('26.0') Decimal('6.0') Decimal('60.0') Decimal('78.5')]
 [Decimal('1.0') Decimal('29.0') Decimal('15.0') Decimal('52.0') Decimal('74.3')]
 [Decimal('11.0') Decimal('56.0') Decimal('8.0') Decimal('20.0') Decimal('104.3')]
 [Decimal('11.0') Decimal('31.0') Decimal('8.0') Decimal('47.0') Decimal('87.6')]
 [Decimal('7.0') Decimal('52.0') Decimal('6.0') Decimal('33.0') Decimal('95.9')]
 [Decimal('11.0') Decimal('55.0') Decimal('9.0') Decimal('22.0') Decimal('109.2')]
 [Decimal('3.0') Decimal('71.0') Decimal('17.0') Decimal('6.0') Decimal('102.7')]
 [Decimal('1.0') Decimal('31.0') Decimal('22.0') Decimal('44.0') Decimal('72.5')]
 [Decimal('2.0') Decimal('54.0') Decimal('18.0') Decimal('22.0') Decimal('93.1')]
 [Decimal('21.0') Decimal('47.0') Decimal('4.0') Decimal('26.0') Decimal('115.9')]
 [Decimal('1.0') Decimal('40.0') Decimal('23.0') Decimal('34.0') Decimal('83.8')]
 [Decimal('11.0') Decimal('66.0') Decimal('9.0') Decimal('12.0') Decimal('113.3')]
 [Decimal('10.0') Decimal('68.0') Decimal('8.0') Decimal('12.0') Decimal('109.4')]]

Header:  x1,x2,x3,x4,y
N: 13
A:
 [[Fraction(7, 1) Fraction(26, 1) Fraction(6, 1) Fraction(60, 1) Fraction(157, 2)]
 [Fraction(1, 1) Fraction(29, 1) Fraction(15, 1) Fraction(52, 1) Fraction(743, 10)]
```

(continues on next page)

(continued from previous page)

```
 [Fraction(11, 1) Fraction(56, 1) Fraction(8, 1) Fraction(20, 1) Fraction(1043, 10)]
 [Fraction(11, 1) Fraction(31, 1) Fraction(8, 1) Fraction(47, 1) Fraction(438, 5)]
 [Fraction(7, 1) Fraction(52, 1) Fraction(6, 1) Fraction(33, 1) Fraction(959, 10)]
 [Fraction(11, 1) Fraction(55, 1) Fraction(9, 1) Fraction(22, 1) Fraction(546, 5)]
 [Fraction(3, 1) Fraction(71, 1) Fraction(17, 1) Fraction(6, 1) Fraction(1027, 10)]
 [Fraction(1, 1) Fraction(31, 1) Fraction(22, 1) Fraction(44, 1) Fraction(145, 2)]
 [Fraction(2, 1) Fraction(54, 1) Fraction(18, 1) Fraction(22, 1) Fraction(931, 10)]
 [Fraction(21, 1) Fraction(47, 1) Fraction(4, 1) Fraction(26, 1) Fraction(1159, 10)]
 [Fraction(1, 1) Fraction(40, 1) Fraction(23, 1) Fraction(34, 1) Fraction(419, 5)]
 [Fraction(11, 1) Fraction(66, 1) Fraction(9, 1) Fraction(12, 1) Fraction(1133, 10)]
 [Fraction(10, 1) Fraction(68, 1) Fraction(8, 1) Fraction(12, 1) Fraction(547, 5)]]

Header:  x1,x2,x3,x4,y
N: 13
A:
 [[mpfr('7.0') mpfr('26.0') mpfr('6.0') mpfr('60.0') mpfr('78.5')]
 [mpfr('1.0') mpfr('29.0') mpfr('15.0') mpfr('52.0') mpfr('74.299999999999997')]
 [mpfr('11.0') mpfr('56.0') mpfr('8.0') mpfr('20.0') mpfr('104.3')]
 [mpfr('11.0') mpfr('31.0') mpfr('8.0') mpfr('47.0') mpfr('87.599999999999994')]
 [mpfr('7.0') mpfr('52.0') mpfr('6.0') mpfr('33.0') mpfr('95.900000000000006')]
 [mpfr('11.0') mpfr('55.0') mpfr('9.0') mpfr('22.0') mpfr('109.2')]
 [mpfr('3.0') mpfr('71.0') mpfr('17.0') mpfr('6.0') mpfr('102.7')]
 [mpfr('1.0') mpfr('31.0') mpfr('22.0') mpfr('44.0') mpfr('72.5')]
 [mpfr('2.0') mpfr('54.0') mpfr('18.0') mpfr('22.0') mpfr('93.099999999999994')]
 [mpfr('21.0') mpfr('47.0') mpfr('4.0') mpfr('26.0') mpfr('115.90000000000001')]
 [mpfr('1.0') mpfr('40.0') mpfr('23.0') mpfr('34.0') mpfr('83.799999999999997')]
 [mpfr('11.0') mpfr('66.0') mpfr('9.0') mpfr('12.0') mpfr('113.3')]
 [mpfr('10.0') mpfr('68.0') mpfr('8.0') mpfr('12.0') mpfr('109.40000000000001')]]

Header:  x1,x2,x3,x4,y
N: 13
A:
 [[7.00000000000000 26.0000000000000 6.00000000000000 60.0000000000000 78.5000000000000]
 [1.00000000000000 29.0000000000000 15.0000000000000 52.0000000000000 [74.3000000000000 +/- 1.71e-14]]
 [11.0000000000000 56.0000000000000 8.00000000000000 20.0000000000000 [104.300000000000 +/- 1.71e-14]]
 [11.0000000000000 31.0000000000000 8.00000000000000 47.0000000000000 [87.6000000000000 +/- 2.00e-14]]
 [7.00000000000000 52.0000000000000 6.00000000000000 33.0000000000000 [95.9000000000000 +/- 2.28e-14]]
 [11.0000000000000 55.0000000000000 9.00000000000000 22.0000000000000 [109.200000000000 +/- 2.56e-14]]
 [3.00000000000000 71.0000000000000 17.0000000000000 6.00000000000000 [102.700000000000 +/- 2.56e-14]]
 [1.00000000000000 31.0000000000000 22.0000000000000 44.0000000000000 72.5000000000000]
 [2.00000000000000 54.0000000000000 18.0000000000000 22.0000000000000 [93.1000000000000 +/- 2.00e-14]]
 [21.0000000000000 47.0000000000000 4.00000000000000 26.0000000000000 [115.900000000000 +/- 2.28e-14]]
 [1.00000000000000 40.0000000000000 23.0000000000000 34.0000000000000 [83.8000000000000 +/- 1.71e-14]]
 [11.0000000000000 66.0000000000000 9.00000000000000 12.0000000000000 [113.300000000000 +/- 1.71e-14]]
 [10.0000000000000 68.0000000000000 8.00000000000000 12.0000000000000 [109.400000000000 +/- 2.28e-14]]]
```

7.2.9 Array from text data in a text file (can handle missing values): numpy.genfromtxt

npm.**genfromtxt**(*fname*, *dtype=<class 'float'>*, *comments='#'*, *delimiter=None*, *skip_header=0*, *skip_footer=0*, *converters=None*, *missing_values=None*, *filling_values=None*, *usecols=None*, *names=None*, *excludelist=None*, *deletechars=" !#$%&'()*+, -./:;<=>?@[\\]^{|}~"*, *replace_space='_'*, *autostrip=False*, *case_sensitive=True*, *defaultfmt='f%i'*, *unpack=None*, *usemask=False*, *loose=True*, *invalid_raise=True*, *max_rows=None*, *encoding=None*, *, *ndmin=0*, *like=None*)

Load data from a text file, with missing values handled as specified.

Each line past the first skip_header lines is split at the delimiter character, and characters following the comments character are discarded.

See https://numpy.org/doc/stable/reference/generated/numpy.genfromtxt.html#numpy.genfromtxt for details.

When spaces are used as delimiters, or when no delimiter has been given as input, there should not be any missing data between two fields.

When variables are named (either by a flexible dtype or with a names sequence), there must not be any header in the file (else a ValueError exception is raised).

Individual values are not stripped of spaces by default. When using a custom converter, make sure the function does remove spaces.

```
import os
import xlcalcnet.userpaths as userpaths
for ctx in ctx_all:
    to_ctx = npm.vectorize(ctx.t, otypes=[object])

    csvpath = os.sep.join([userpaths.get_my_documents(), \
        'DataXlCalcNet', 'DataExamples', 'MainExamples', 'CSV'])
    csvfile = 'Hald.csv'
    csvname = os.sep.join([csvpath, csvfile])

    with open(csvname) as f: header = f.readline().strip('\n')
    print('Header: ', header)

    nd_data = npm.genfromtxt(csvname, dtype=np.float64, delimiter=',', \
    skip_header=1, usecols=(0,1,2,3,4))
    A = to_ctx(nd_data)
    N = len(A)
    print('N:', N)
    print('A: \n', A)
    print()
```

The output is the same as for the `loadtxt` example shown above and is not repeated here.

7.3 Building special arrays for numerical work

7.3.1 Evenly spaced values within a given interval: numpy.arange

npm.**arange**([*start*,]*stop*, [*step*,]*dtype=None*, *, *device=None*, *like=None*)

Return evenly spaced values within a given interval.

See https://numpy.org/doc/stable/reference/generated/numpy.arange.html#numpy.arange for details.

See also: https://numpy.org/doc/stable/user/how-to-partition.html#how-to-partition

For integer arguments the function is roughly equivalent to the Python built-in range, but returns an ndarray rather than a range instance.

When using a non-integer step, such as 0.1, it is often better to use numpy.linspace. The length of the output might not be numerically stable.

Another stability issue is due to the internal implementation of numpy.arange. The actual step value used to populate the array is dtype(start + step) - dtype(start) and not step. Precision loss can occur here, due to casting or due to using floating points when start is much larger than step. This can lead to unexpected behaviour. For example:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]

>>> # arange with 1 argument:
>>> for ctx in ctx_all: print(npm.arange(4, dtype=ctx))
[0. 1. 2. 3.]
[mpf('0.0') mpf('1.0') mpf('2.0') mpf('3.0')]
[mpi('0.0', '0.0') mpi('1.0', '1.0') mpi('2.0', '2.0') mpi('3.0', '3.0')]
[Decimal('0') Decimal('1') Decimal('2') Decimal('3')]
[Fraction(0, 1) Fraction(1, 1) Fraction(2, 1) Fraction(3, 1)]
[mpfr('0.0') mpfr('1.0') mpfr('2.0') mpfr('3.0')]
[0 1.00000000000000 2.00000000000000 3.00000000000000]

>>> # arange with 2 arguments:
>>> for ctx in ctx_all: print(npm.arange(0.3, 4, dtype=ctx))
[0.3 1.3 2.3 3.3]
[mpf('0.29999999999999999') mpf('1.3') mpf('2.2999999999999998') mpf('3.2999999999999998')]
[mpi('0.29999999999999999', '0.30000000000000004') mpi('1.2999999999999998', '1.3') mpi('2.2999999999999994', '2.
↪3000000000000007') mpi('3.2999999999999989', '3.3000000000000012')]
[Decimal('0.3') Decimal('1.3') Decimal('2.3') Decimal('3.3')]
[Fraction(3, 10) Fraction(13, 10) Fraction(23, 10) Fraction(33, 10)]
[mpfr('0.29999999999999999') mpfr('1.3') mpfr('2.2999999999999998') mpfr('3.2999999999999998')]
[[0.300000000000000 +/- 6.67e-17] [1.30000000000000 +/- 4.56e-16] [2.30000000000000 +/- 1.79e-15] [3.30000000000000 +/-
↪ 3.12e-15]]

>>> # arange with 3 arguments, increasing:
>>> for ctx in ctx_all: print(npm.arange(1, 2, 0.25, dtype=ctx))
[1.   1.25 1.5  1.75]
[mpf('1.0') mpf('1.25') mpf('1.5') mpf('1.75')]
[mpi('1.0', '1.0') mpi('1.25', '1.25') mpi('1.5', '1.5') mpi('1.75', '1.75')]
[Decimal('1') Decimal('1.25') Decimal('1.50') Decimal('1.75')]
[Fraction(1, 1) Fraction(5, 4) Fraction(3, 2) Fraction(7, 4)]
[mpfr('1.0') mpfr('1.25') mpfr('1.5') mpfr('1.75')]
[1.00000000000000 1.25000000000000 1.50000000000000 1.75000000000000]

>>> # arange with 3 arguments, decreasing:
>>> for ctx in ctx_all: print(npm.arange(1, -1, -0.75, dtype=ctx))
[1.   1.25 1.5  1.75]
[mpf('1.0') mpf('1.25') mpf('1.5') mpf('1.75')]
[mpi('1.0', '1.0') mpi('1.25', '1.25') mpi('1.5', '1.5') mpi('1.75', '1.75')]
[Decimal('1') Decimal('1.25') Decimal('1.50') Decimal('1.75')]
[Fraction(1, 1) Fraction(5, 4) Fraction(3, 2) Fraction(7, 4)]
[mpfr('1.0') mpfr('1.25') mpfr('1.5') mpfr('1.75')]
[1.00000000000000 1.25000000000000 1.50000000000000 1.75000000000000]
```

From mpmath. See also Mpmath [686].

This is a generalized version of Python's `range()` function that accepts fractional endpoints and step sizes and returns a list of `mpf` instances.

Like `range()`, *arange()* can be called with 1, 2 or 3 arguments:

arange(b)
$[0, 1, 2, \ldots, x]$

arange(a, b)
$[a, a+1, a+2, \ldots, x]$

arange(a, b, h)
$[a, a+h, a+h, \ldots, x]$

where $b-1 \le x < b$ (in the third case, $b-h \le x < b$).

Like Python's `range()`, the endpoint is not included. To produce ranges where the endpoint is included, *linspace()* is more convenient.

Examples:

```
>>> from xlcalcnet import fp, mp, iv, dp, gp, ap

>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(ctx.arange(4))
[0.0, 1.0, 2.0, 3.0]
[mpf('0.0'), mpf('1.0'), mpf('2.0'), mpf('3.0')]
[mpi('0.0', '0.0'), mpi('1.0', '1.0'), mpi('2.0', '2.0'), mpi('3.0', '3.0')]
[Decimal('0'), Decimal('1'), Decimal('2'), Decimal('3')]
[mpfr('0.0'), mpfr('1.0'), mpfr('2.0'), mpfr('3.0')]
[0, 1.00000000000000, 2.00000000000000, 3.00000000000000]

>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(ctx.arange(1, 2, 0.25))
[1.0, 1.25, 1.5, 1.75]
[mpf('1.0'), mpf('1.25'), mpf('1.5'), mpf('1.75')]
[mpi('1.0', '1.0'), mpi('1.25', '1.25'), mpi('1.5', '1.5'), mpi('1.75', '1.75')]
[Decimal('1.00'), Decimal('1.25'), Decimal('1.50'), Decimal('1.75')]
[mpfr('1.0'), mpfr('1.25'), mpfr('1.5'), mpfr('1.75')]
[1.00000000000000, 1.25000000000000, 1.50000000000000, 1.75000000000000]

>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(ctx.arange(1, -1, -0.75))
[1.0, 0.25, -0.5]
[mpf('1.0'), mpf('0.25'), mpf('-0.5')]
[mpi('1.0', '1.0'), mpi('0.25', '0.25'), mpi('-0.5', '-0.5')]
[Decimal('1.00'), Decimal('0.25'), Decimal('-0.50')]
[mpfr('1.0'), mpfr('0.25'), mpfr('-0.5')]
[1.00000000000000, 0.250000000000000, -0.500000000000000]
```

7.3.2 Evenly spaced values within a given interval: numpy.linspace

npm.linspace(*start*, *stop*, *num=50*, *endpoint=True*, *retstep=False*, *dtype=None*, *axis=0*, ***, *device=None*)

Return evenly spaced numbers over a specified interval.

See https://numpy.org/doc/stable/reference/generated/numpy.linspace.html#numpy.linspace for details.

Returns num evenly spaced samples, calculated over the interval [start, stop]. The endpoint of the interval can optionally be excluded.

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]

>>> # linspace with endpoint=True:
>>> for ctx in ctx_all: print(npm.linspace(start=1, stop=4, num=4, dtype=ctx))
```

(continues on next page)

(continued from previous page)

```
[1. 2. 3. 4.]
[mpf('1.0') mpf('2.0') mpf('3.0') mpf('4.0')]
[mpi('1.0', '1.0') mpi('1.9999999999999998', '2.0000000000000004') mpi('2.9999999999999996', '3.0000000000000004') mpi(
↪'4.0', '4.0')]
[Decimal('1') Decimal('2.00000000000000') Decimal('3.00000000000000') Decimal('4')]
[Fraction(1, 1) Fraction(2, 1) Fraction(3, 1) Fraction(4, 1)]
[mpfr('1.0') mpfr('2.0') mpfr('3.0') mpfr('4.0')]
[1.00000000000000 [2.00000000000000 +/- 7.22e-16] [3.00000000000000 +/- 1.45e-15] 4.00000000000000]
```

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]

>>> # linspace with endpoint=False:
>>> for ctx in ctx_all: print(npm.linspace(start=1, stop=4, num=4, dtype=ctx, endpoint=False))
[1.   1.75 2.5  3.25]
[mpf('1.0') mpf('1.75') mpf('2.5') mpf('3.25')]
[mpi('1.0', '1.0') mpi('1.75', '1.75') mpi('2.5', '2.5') mpi('3.25', '3.25')]
[Decimal('1') Decimal('1.75') Decimal('2.5') Decimal('3.25')]
[Fraction(1, 1) Fraction(7, 4) Fraction(5, 2) Fraction(13, 4)]
[mpfr('1.0') mpfr('1.75') mpfr('2.5') mpfr('3.25')]
[1.00000000000000 1.75000000000000 2.50000000000000 3.25000000000000]
```

From mpmath. . See also Mpmath [685].

`linspace(a, b, n)` returns a list of n evenly spaced samples from a to b.

The syntax `linspace(mpi(a,b), n)` is also valid.

This function is often more convenient than *arange()* for partitioning an interval into subintervals, since the endpoint is included:

```
>>> from xlcalcnet import fp, mp, iv, dp, gp, ap
>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(ctx.linspace(1, 4, 4))
[1.0, 2.0, 3.0, 4.0]
[mpf('1.0'), mpf('2.0'), mpf('3.0'), mpf('4.0')]
[mpi('1.0', '1.0'), mpi('2.0', '2.0'), mpi('3.0', '3.0'), mpi('4.0', '4.0')]
[Decimal('1'), Decimal('2'), Decimal('3'), Decimal('4')]
[mpfr('1.0'), mpfr('2.0'), mpfr('3.0'), mpfr('4.0')]
[1.00000000000000, 2.00000000000000, 3.00000000000000, 4.00000000000000]
```

You may also provide the keyword argument `endpoint=False`:

```
>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(ctx.linspace(1, 4, 4, endpoint=False))
[1.0, 1.75, 2.5, 3.25]
[mpf('1.0'), mpf('1.75'), mpf('2.5'), mpf('3.25')]
[mpi('1.0', '1.0'), mpi('1.75', '1.75'), mpi('2.5', '2.5'), mpi('3.25', '3.25')]
[Decimal('1.00'), Decimal('1.75'), Decimal('2.50'), Decimal('3.25')]
[mpfr('1.0'), mpfr('1.75'), mpfr('2.5'), mpfr('3.25')]
[1.00000000000000, 1.75000000000000, 2.50000000000000, 3.25000000000000]
```

7.3.3 Evenly spaced values on a log scale: numpy.logspace

`npm.logspace`(*start*, *stop*, *num=50*, *endpoint=True*, *base=10.0*, *dtype=None*, *axis=0*)

Return numbers spaced evenly on a log scale. In linear space, the sequence starts at base ** start (base to the power of start) and ends with base ** stop.

See https://numpy.org/doc/stable/reference/generated/numpy.logspace.html#numpy.logspace for details.

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]

>>> # linspace with endpoint=True:
```

(continues on next page)

(continued from previous page)

```
>>> for ctx in ctx_all: print(npm.logspace(start=2, stop=3, num=4, dtype=ctx))
[100.0 215.44346900318845 464.15888336127773 1000.0]
[mpf('100.0') mpf('215.44346900318845') mpf('464.15888336127773') mpf('1000.0')]
[mpi('100.0', '100.0') mpi('215.4434690031882', '215.44346900318845') mpi('464.15888336127773', '464.15888336127824')␣
↪mpi('1000.0', '1000.0')]
[Decimal('100.00') Decimal('215.443469003187') Decimal('464.158883361281') Decimal('1000.000')]
[mpfr('100.0') mpfr('215.44346900318845') mpfr('464.15888336127773') mpfr('1000.0')]
[100.000000000000 [215.443469003188 +/- 7.77e-13] [464.15888336128 +/- 4.29e-12] 1000.00000000000]
```

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, qpm, gpm, apm]

>>> # linspace with endpoint=False:
>>> for ctx in ctx_all: print(npm.logspace(start=2, stop=3, num=4, dtype=ctx, endpoint=False))
[100.0 177.82794100389228 316.22776601683796 562.341325190349]
[mpf('100.0') mpf('177.82794100389228') mpf('316.22776601683796') mpf('562.34132519034904')]
[mpi('100.0', '100.0') mpi('177.82794100389228', '177.82794100389231') mpi('316.2277660168379', '316.22776601683796')␣
↪mpi('562.34132519034904', '562.34132519034915')]
[Decimal('100.00') Decimal('177.827941003892') Decimal('316.227766016838') Decimal('562.341325190349')]
[mpfr('100.0') mpfr('177.82794100389228') mpfr('316.22776601683796') mpfr('562.34132519034904')]
[100.000000000000 [177.827941003892 +/- 4.44e-13] [316.227766016838 +/- 1.44e-13] [562.34132519035 +/- 3.06e-12]]
```

7.3.4 Evenly spaced values on a log scale (a geometric progression): numpy.geomspace

npm.**geomspace**(*start*, *stop*, *num=50*, *endpoint=True*, *dtype=None*, *axis=0*)

Return numbers spaced evenly on a log scale (a geometric progression).

See https://numpy.org/doc/stable/reference/generated/numpy.geomspace.html#numpy.geomspace for details.

This is similar to logspace, but with endpoints specified directly. Each output sample is a constant multiple of the previous. If the inputs or dtype are complex, the output will follow a logarithmic spiral in the complex plane. (There are an infinite number of spirals passing through two points; the output will follow the shortest such path.)

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, gpm, apm]

>>> # geomspace with endpoint=True:
>>> for ctx in ctx_all: print(npm.geomspace(start=1, stop=1000, num=4, dtype=ctx))
[1.0 10.0 100.0 1000.0]
[mpf('1.0') mpf('10.0') mpf('100.0') mpf('1000.0')]
[mpi('1.0', '1.0') mpi('9.9999999999999947', '10.000000000000011') mpi('99.999999999999886', '100.00000000000021') mpi(
↪'999.99999999999898', '1000.0000000000001')]
[Decimal('1') Decimal('9.99999999999998') Decimal('100.00') Decimal('1000.000')]
[mpfr('1.0') mpfr('10.0') mpfr('100.0') mpfr('1000.0')]
[1.00000000000000 [10.000000000000 +/- 3.50e-14] [100.00000000000 +/- 6.79e-13] [1000.0000000000 +/- 6.72e-12]]
```

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, gpm, apm]

>>> # geomspace with endpoint=False:
>>> for ctx in ctx_all: print(npm.geomspace(start=1, stop=1000, num=4, dtype=ctx, endpoint=False))
[1.0 5.623413251903491 31.622776601683793 177.82794100389228]
[mpf('1.0') mpf('5.6234132519034912') mpf('31.622776601683793') mpf('177.82794100389228')]
[mpi('1.0', '1.0') mpi('5.6234132519034885', '5.6234132519034929') mpi('31.622776601683775', '31.62277660168381') mpi(
↪'177.82794100389208', '177.82794100389248')]
[Decimal('1') Decimal('5.62341325190349') Decimal('31.6227766016838') Decimal('177.827941003892')]
[mpfr('1.0') mpfr('5.6234132519034912') mpfr('31.622776601683793') mpfr('177.82794100389228')]
[1.00000000000000 [5.6234132519035 +/- 2.01e-14] [31.622776601684 +/- 3.16e-13] [177.827941003892 +/- 8.76e-13]]
```

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, gpm, apm]
```

(continues on next page)

(continued from previous page)

```
>>> # geomspace with powers of 2:
>>> for ctx in ctx_all: print(npm.geomspace(start=1, stop=1000, num=4, dtype=ctx, endpoint=False))
[1.0 2.0 4.0 7.999999999999999 16.0 32.00000000000001 63.999999999999986 127.99999999999999 256.0]
[mpf('1.0') mpf('2.0') mpf('4.0') mpf('7.9999999999999991') mpf('16.0') mpf('32.000000000000007') mpf('63.
↪9999999999999986') mpf('127.99999999999999') mpf('256.0')]
[mpi('1.0', '1.0') mpi('1.9999999999999996', '2.0000000000000004') mpi('3.9999999999999987', '4.0000000000000009') mpi(
↪'7.9999999999999964', '8.0000000000000018')
 mpi('15.999999999999991', '16.000000000000004') mpi('31.999999999999972', '32.000000000000007') mpi('63.99999999999995
↪', '64.000000000000028') mpi('127.99999999999986', '128.00000000000014')
 mpi('255.99999999999974', '256.00000000000006')]
[Decimal('1') Decimal('2.00000000000000') Decimal('4.00000000000000') Decimal('8.00000000000001') Decimal('15.
↪9999999999998') Decimal('32.0000000000003') Decimal('64.0000000000004')
 Decimal('128.000000000000') Decimal('256.000000000000')]
[mpfr('1.0') mpfr('2.0') mpfr('4.0') mpfr('7.9999999999999991') mpfr('16.0') mpfr('32.000000000000007') mpfr('63.
↪9999999999999986') mpfr('127.99999999999999') mpfr('256.0')]
[1.00000000000000 [2.00000000000000 +/- 1.94e-15] [4.00000000000000 +/- 6.84e-15] [8.0000000000000 +/- 2.44e-14] [16.
↪0000000000000 +/- 5.11e-14] [32.000000000000 +/- 1.50e-13]
 [64.000000000000 +/- 3.82e-13] [128.000000000000 +/- 8.82e-13] [256.00000000000 +/- 1.61e-12]]
```

7.3.5 Extract a diagonal or construct a diagonal array: numpy.diag

npm.**diag**(*v*, *k=0*)

Extract a diagonal or construct a diagonal array.

See https://numpy.org/doc/stable/reference/generated/numpy.diag.html#numpy.diag for details.

See the more detailed documentation for numpy.diagonal if you use this function to extract a diagonal and wish to write to the resulting array; whether it returns a copy or a view depends on what version of numpy you are using.

Array x:

```
>>> from xlcalcnet import fpm, mpm, ipm, dpm, qpm, gpm, apm, npm, np
>>> ctx_all = [fpm, mpm, ipm, dpm, gpm, apm]

>>> for ctx in ctx_all: x = npm.arange(9, dtype=ctx).reshape((3,3)); print(ctx.name +', x: \n', x)
fpm, x:
 [[0. 1. 2.]
 [3. 4. 5.]
 [6. 7. 8.]]
mpm, x:
 [[mpf('0.0') mpf('1.0') mpf('2.0')]
 [mpf('3.0') mpf('4.0') mpf('5.0')]
 [mpf('6.0') mpf('7.0') mpf('8.0')]]
ipm, x:
 [[mpi('0.0', '0.0') mpi('1.0', '1.0') mpi('2.0', '2.0')]
 [mpi('3.0', '3.0') mpi('4.0', '4.0') mpi('5.0', '5.0')]
 [mpi('6.0', '6.0') mpi('7.0', '7.0') mpi('8.0', '8.0')]]
dpm, x:
 [[Decimal('0') Decimal('1') Decimal('2')]
 [Decimal('3') Decimal('4') Decimal('5')]
 [Decimal('6') Decimal('7') Decimal('8')]]
qpm, x:
 [[Fraction(0, 1) Fraction(1, 1) Fraction(2, 1)]
 [Fraction(3, 1) Fraction(4, 1) Fraction(5, 1)]
 [Fraction(6, 1) Fraction(7, 1) Fraction(8, 1)]]
gpm, x:
 [[mpfr('0.0') mpfr('1.0') mpfr('2.0')]
 [mpfr('3.0') mpfr('4.0') mpfr('5.0')]
 [mpfr('6.0') mpfr('7.0') mpfr('8.0')]]
apm, x:
 [[0 1.00000000000000 2.00000000000000]
 [3.00000000000000 4.00000000000000 5.00000000000000]
 [6.00000000000000 7.00000000000000 8.00000000000000]]
```

Diagonal of array x:

```
>>> for ctx in ctx_all: x = npm.arange(9, dtype=ctx).reshape((3,3)); print(ctx.name +', diag(x): \n', npm.diag(x))
fpm, diag(x):
 [0. 4. 8.]
mpm, diag(x):
 [mpf('0.0') mpf('4.0') mpf('8.0')]
ipm, diag(x):
 [mpi('0.0', '0.0') mpi('4.0', '4.0') mpi('8.0', '8.0')]
dpm, diag(x):
 [Decimal('0') Decimal('4') Decimal('8')]
qpm, diag(x):
 [Fraction(0, 1) Fraction(4, 1) Fraction(8, 1)]
gpm, diag(x):
 [mpfr('0.0') mpfr('4.0') mpfr('8.0')]
apm, diag(x):
 [0 4.00000000000000 8.00000000000000]
```

Superdiagonal of array x:

```
>>> for ctx in ctx_all: x = npm.arange(9, dtype=ctx).reshape((3,3)); print(ctx.name +', diag(x, k=1): \n', npm.diag(x,␣
↪k=1))
fpm, diag(x, k=1):
 [1. 5.]
mpm, diag(x, k=1):
 [mpf('1.0') mpf('5.0')]
ipm, diag(x, k=1):
 [mpi('1.0', '1.0') mpi('5.0', '5.0')]
dpm, diag(x, k=1):
 [Decimal('1') Decimal('5')]
qpm, diag(x, k=1):
 [Fraction(1, 1) Fraction(5, 1)]
gpm, diag(x, k=1):
 [mpfr('1.0') mpfr('5.0')]
apm, diag(x, k=1):
 [1.00000000000000 5.00000000000000]
```

Subdiagonal of array x:

```
>>> for ctx in ctx_all: x = npm.arange(9, dtype=ctx).reshape((3,3)); print(ctx.name +', diag(x, k=-1): \n', npm.diag(x,
↪ k=-1))
fpm, diag(x, k=-1):
 [3. 7.]
mpm, diag(x, k=-1):
 [mpf('3.0') mpf('7.0')]
ipm, diag(x, k=-1):
 [mpi('3.0', '3.0') mpi('7.0', '7.0')]
dpm, diag(x, k=-1):
 [Decimal('3') Decimal('7')]
qpm, diag(x, k=-1):
 [Fraction(3, 1) Fraction(7, 1)]
gpm, diag(x, k=-1):
 [mpfr('3.0') mpfr('7.0')]
apm, diag(x, k=-1):
 [3.00000000000000 7.00000000000000]
```

Setting the diagonal. Note that only the diagonal values are set, the others are just 0.

```
>>> for ctx in ctx_all: x = npm.arange(9, dtype=ctx).reshape((3,3)); print(ctx.name +', npm.diag(npm.diag(x)): \n',␣
↪npm.diag(npm.diag(x)))
fpm, npm.diag(npm.diag(x)):
 [[0. 0. 0.]
 [0. 4. 0.]
 [0. 0. 8.]]
mpm, npm.diag(npm.diag(x)):
```

(continues on next page)

(continued from previous page)

```
 [[mpf('0.0') 0 0]
 [0 mpf('4.0') 0]
 [0 0 mpf('8.0')]]
ipm, npm.diag(npm.diag(x)):
 [[mpi('0.0', '0.0') 0 0]
 [0 mpi('4.0', '4.0') 0]
 [0 0 mpi('8.0', '8.0')]]
dpm, npm.diag(npm.diag(x)):
 [[Decimal('0') 0 0]
 [0 Decimal('4') 0]
 [0 0 Decimal('8')]]
qpm, npm.diag(npm.diag(x)):
 [[Fraction(0, 1) 0 0]
 [0 Fraction(4, 1) 0]
 [0 0 Fraction(8, 1)]]
gpm, npm.diag(npm.diag(x)):
 [[mpfr('0.0') 0 0]
 [0 mpfr('4.0') 0]
 [0 0 mpfr('8.0')]]
apm, npm.diag(npm.diag(x)):
 [[0 0 0]
 [0 4.00000000000000 0]
 [0 0 8.00000000000000]]
```

7.3.6 Extract specified diagonals: numpy.diagonal

npm.**diagonal**(*a*, *offset=0*, *axis1=0*, *axis2=1*)

Return specified diagonals.

See https://numpy.org/doc/stable/reference/generated/numpy.diagonal.html for details.

If a is 2-D, returns the diagonal of a with the given offset, i.e., the collection of elements of the form a[i, i+offset]. If a has more than two dimensions, then the axes specified by axis1 and axis2 are used to determine the 2-D sub-array whose diagonal is returned. The shape of the resulting array can be determined by removing axis1 and axis2 and appending an index to the right equal to the size of the resulting diagonals.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> a = np.arange(4).reshape(2,2)
>>> a
array([[0, 1],
       [2, 3]])
>>> a.diagonal()
array([0, 3])
>>> a.diagonal()
array([0, 3])
a.diagonal(1)
array([1])

>>> # A 3-D example:
>>> a = np.arange(8).reshape(2,2,2); a
array([[[0, 1],
    [2, 3]],
   [[4, 5],
    [6, 7]]])
>>> a.diagonal(0,0,1)
array([[0, 6],
   [1, 7]])

>>> # The sub-arrays whose main diagonals we just obtained; note that each corresponds
>>> # to fixing the right-most
>>> # (column) axis, and that the diagonals are "packed" in rows.
>>> a[:,:,0]  # main diagonal is [0 6]
```

(continues on next page)

(continued from previous page)

```
array([[0, 2],
   [4, 6]])
>>> a[:,:,1]  # main diagonal is [1 7]
array([[1, 3],
   [5, 7]])


>>> # The anti-diagonal can be obtained by reversing the order of elements using either
>>> # numpy.flipud or numpy.fliplr.
>>> a = np.arange(9).reshape(3, 3); a
array([[0, 1, 2],
   [3, 4, 5],
   [6, 7, 8]])
>>> np.fliplr(a).diagonal()  # Horizontal flip
array([2, 4, 6])
>>> np.flipud(a).diagonal()  # Vertical flip
array([2, 4, 6])
array([6, 4, 2])
```

7.3.7 Two-dimensional array with the flattened input as a diagonal: numpy.diagflat

npm.**diagflat**(*v*, *k=0*)

Create a two-dimensional array with the flattened input as a diagonal.

See https://numpy.org/doc/stable/reference/generated/numpy.diagflat.html#numpy.diagflat for details.

Create a two-dimensional array with the flattened input as a diagonal.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> np.diagflat([[1,2], [3,4]])
array([[1, 0, 0, 0],
   [0, 2, 0, 0],
   [0, 0, 3, 0],
   [0, 0, 0, 4]])

>>> np.diagflat([1,2], 1)
array([[0, 1, 0],
   [0, 0, 2],
   [0, 0, 0]])
```

7.3.8 Lower triangle of an array: numpy.tril

npm.**tril**(*m*, *k=0*)

Returns the lower triangle of an array.

See https://numpy.org/doc/stable/reference/generated/numpy.tril.html#numpy.tril for details

Return a copy of an array with elements above the k-th diagonal zeroed. For arrays with ndim exceeding 2, tril will apply to the final two axes.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> np.tril([[1,2,3],[4,5,6],[7,8,9],[10,11,12]], -1)
array([[ 0,  0,  0],
   [ 4,  0,  0],
   [ 7,  8,  0],
   [10, 11, 12]])

>>> np.tril(np.arange(3*4*5).reshape(3, 4, 5))
array([[[ 0,  0,  0,  0,  0],
```

(continues on next page)

(continued from previous page)

```
    [ 5,  6,  0,  0,  0],
    [10, 11, 12,  0,  0],
    [15, 16, 17, 18,  0]],
   [[20,  0,  0,  0,  0],
    [25, 26,  0,  0,  0],
    [30, 31, 32,  0,  0],
    [35, 36, 37, 38,  0]],
   [[40,  0,  0,  0,  0],
    [45, 46,  0,  0,  0],
    [50, 51, 52,  0,  0],
    [55, 56, 57, 58,  0]]])
```

7.3.9 Upper triangle of an array: numpy.triu

npm.**triu**(*m*, *k=0*)

Returns the upper triangle of an array.

See https://numpy.org/doc/stable/reference/generated/numpy.triu.html#numpy.triu for details

Return a copy of an array with the elements below the k-th diagonal zeroed. For arrays with ndim exceeding 2, triu will apply to the final two axes.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> np.triu([[1,2,3],[4,5,6],[7,8,9],[10,11,12]], -1)
array([[ 1,  2,  3],
   [ 4,  5,  6],
   [ 0,  8,  9],
   [ 0,  0, 12]])

>>> np.triu(np.arange(3*4*5).reshape(3, 4, 5))
array([[[ 0,  1,  2,  3,  4],
    [ 0,  6,  7,  8,  9],
    [ 0,  0, 12, 13, 14],
    [ 0,  0,  0, 18, 19]],
   [[20, 21, 22, 23, 24],
    [ 0, 26, 27, 28, 29],
    [ 0,  0, 32, 33, 34],
    [ 0,  0,  0, 38, 39]],
   [[40, 41, 42, 43, 44],
    [ 0, 46, 47, 48, 49],
    [ 0,  0, 52, 53, 54],
    [ 0,  0,  0, 58, 59]]])
```

7.3.10 Vandermonde matrix.: numpy.vander

npm.**vander**(*x*, *N=None*, *increasing=False*)

Generate a Vandermonde matrix.

See https://numpy.org/doc/stable/reference/generated/numpy.vander.html#numpy.vander for details

The columns of the output matrix are powers of the input vector. The order of the powers is determined by the increasing boolean argument. Specifically, when increasing is False, the i-th output column is the input vector raised element-wise to the power of N - i - 1.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.array([1, 2, 3, 5])
>>> N = 3
>>> np.vander(x, N)
array([[ 1,  1,  1],
```

(continues on next page)

(continued from previous page)

```
       [ 4,  2,  1],
       [ 9,  3,  1],
       [25,  5,  1]])

>>> np.column_stack([x**(N-1-i) for i in range(N)])
array([[ 1,  1,  1],
   [ 4,  2,  1],
   [ 9,  3,  1],
   [25,  5,  1]])


>>> x = np.array([1, 2, 3, 5])
>>> np.vander(x)
array([[  1,   1,   1,   1],
   [  8,   4,   2,   1],
   [ 27,   9,   3,   1],
   [125,  25,   5,   1]])
>>> np.vander(x, increasing=True)
array([[  1,   1,   1,   1],
   [  1,   2,   4,   8],
   [  1,   3,   9,  27],
   [  1,   5,  25, 125]])


>>> # The determinant of a square Vandermonde matrix is the product of the differences
>>> # between the values of the input vector:
>>> np.linalg.det(np.vander(x))
8.000000000000043 # may vary
>>> (5-3)*(5-2)*(5-1)*(3-2)*(3-1)*(2-1)
48
```

Example: Creating a Hilbert matrix as a dictionary (mpmath matrix)

Create (pseudo) hilbert matrix m x n. One given dimension will create hilbert matrix n x n.

The matrix is very ill-conditioned and symmetric, positive definite if square.

Example: Creating a random matrix as a dictionary (mpmath matrix)

Returns a random matrix as a dictionary (mpmath matrix).

Create a random m x n matrix. All values are >= min and <max. n defaults to m.

Example: Swap of rows in a mpmath matrix

Swaps rows in a mpmath matrix. Swap row i with row j.

Example: Extending a mpmath matrix by another column

Extends a mpmath matrix. Extend matrix A with column b and return result.

Example: Unit vectors

Returns the i-th n-dimensional unit vector as a mpmath matrix

7.4 Numpy indexing

https://numpy.org/doc/stable/user/basics.indexing.html

ndarrays can be indexed using the standard Python x[obj] syntax, where x is the array and obj the selection. There are different kinds of indexing available depending on obj: basic indexing, advanced indexing and field access.

Most of the following examples show the use of indexing when referencing data in an array. The examples work just as well when assigning to an array. See Assigning values to indexed arrays for specific examples and explanations on how assignments work.

Note that in Python, x[(exp1, exp2, ..., expN)] is equivalent to x[exp1, exp2, ..., expN]; the latter is just syntactic sugar for the former.

7.4.1 Single element indexing

See https://numpy.org/doc/stable/user/basics.indexing.html#basic-indexing

Single element indexing works exactly like that for other standard Python sequences. It is 0-based, and accepts negative indices for indexing from the end of the array.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> x = np.arange(10)
>>> x[2]
2
>>> x[-2]
8
```

It is not necessary to separate each dimension's index into its own set of square brackets.

```
>>> x.shape = (2, 5)  # now x is 2-dimensional
>>> x[1, 3]
8
>>> x[1, -1]
9
```

Note that if one indexes a multidimensional array with fewer indices than dimensions, one gets a subdimensional array. For example:

```
>>> x[0]
array([0, 1, 2, 3, 4])
```

That is, each index specified selects the array corresponding to the rest of the dimensions selected. In the above example, choosing 0 means that the remaining dimension of length 5 is being left unspecified, and that what is returned is an array of that dimensionality and size. It must be noted that the returned array is a view, i.e., it is not a copy of the original, but points to the same values in memory as does the original array. In this case, the 1-D array at the first position (0) is returned. So using a single index on the returned array, results in a single element being returned. That is:

```
>>> x[0][2]
2
```

7.4.2 Slicing and striding

Basic slicing extends Python's basic concept of slicing to N dimensions. Basic slicing occurs when obj is a slice object (constructed by start:stop:step notation inside of brackets), an integer, or a tuple of slice objects and integers. Ellipsis and newaxis objects can be interspersed with these as well.

The simplest case of indexing with N integers returns an array scalar representing the corresponding item. As in Python, all indices are zero-based: for the i-th index n_i, the valid range is $0 \leq n_i < d_i$, where d_i is the i-th element of the shape of the array. Negative indices are interpreted as counting from the end of the array (i.e., if $n_i < 0$, it means $n_i + d_i$).

All arrays generated by basic slicing are always views of the original array. NumPy slicing creates a view instead of a copy as in the case of built-in Python sequences such as string, tuple and list. Care must be taken when extracting a small portion from a large array

which becomes useless after the extraction, because the small portion extracted contains a reference to the large original array whose memory will not be released until all arrays derived from it are garbage-collected. In such cases an explicit copy() is recommended.

The standard rules of sequence slicing apply to basic slicing on a per-dimension basis (including using a step index). Some useful concepts to remember include:

The basic slice syntax is i:j:k where i is the starting index, j is the stopping index, and k is the step ($k \neq 0$). This selects the m elements (in the corresponding dimension) with index values $i, i+k, \ldots, i+(m-1)k$ where $m = q + (r \neq 0)$ and q and r are the quotient and remainder obtained by dividing $j - i$ by $k : j - i = qk + r$, so that $i + (m-1)k < j$. For example:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> x[1:7:2]
array([1, 3, 5])
```

Negative i and j are interpreted as $n + i$ and $n + j$ where n is the number of elements in the corresponding dimension. Negative k makes stepping go towards smaller indices. From the above example:

```
>>> x[-2:10]
array([8, 9])
>>> x[-3:3:-1]
array([7, 6, 5, 4])
```

Assume n is the number of elements in the dimension being sliced. Then, if i is not given it defaults to 0 for $k > 0$ and $n - 1$ for $k < 0$. If j is not given it defaults to n for $k > 0$ and $-n - 1$ for $k < 0$. If k is not given it defaults to 1. Note that `::` is the same as `:` and means select all indices along this axis. From the above example:

```
>>> x[5:]
array([5, 6, 7, 8, 9])
```

If the number of objects in the selection tuple is less than N, then `:` is assumed for any subsequent dimensions. For example:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> x = np.array([[[1],[2],[3]], [[4],[5],[6]]])
>>> x.shape
(2, 3, 1)
>>> x[1:2]
array([[[4],
    [5],
    [6]]])
```

An integer, i, returns the same values as `i:i+1` except the dimensionality of the returned object is reduced by 1. In particular, a selection tuple with the p-th element an integer (and all other entries :) returns the corresponding sub-array with dimension $N - 1$. If $N = 1$ then the returned object is an array scalar.

If the selection tuple has all entries : except the p-th entry which is a slice object `i:j:k`, then the returned array has dimension N formed by concatenating the sub-arrays returned by integer indexing of elements $i, i+k, \ldots, i+(m-1)k < j$.

Basic slicing with more than one non-`:` entry in the slicing tuple, acts like repeated application of slicing using a single non-`:` entry, where the non-`:` entries are successively taken (with all other non-`:` entries replaced by :). Thus, `x[ind1, ..., ind2,:]` acts like `x[ind1][..., ind2, :]` under basic slicing.

You may use slicing to set values in the array, but (unlike lists) you can never grow the array. The size of the value to be set in `x[obj] = value` must be (broadcastable to) the same shape as `x[obj]`.

A slicing tuple can always be constructed as *obj* and used in the `x[obj]` notation. Slice objects can be used in the construction in place of the `[start:stop:step]` notation. For example, `x[1:10:5, ::-1]` can also be implemented as `obj = (slice(1, 10, 5), slice(None, None, -1)); x[obj]`. This can be useful for constructing generic code that works on arrays of arbitrary dimensions.

7.4.3 Dimensional indexing tools

There are some tools to facilitate the easy matching of array shapes with expressions and in assignments.

Ellipsis expands to the number of : objects needed for the selection tuple to index all dimensions. In most cases, this means that the length of the expanded selection tuple is x.ndim. There may only be a single ellipsis present. From the above example:

```
>>> x[..., 0]
array([[1, 2, 3],
  [4, 5, 6]])
```

This is equivalent to:

```
>>> x[:, :, 0]
array([[1, 2, 3],
  [4, 5, 6]])
```

Each newaxis object in the selection tuple serves to expand the dimensions of the resulting selection by one unit-length dimension. The added dimension is the position of the newaxis object in the selection tuple. newaxis is an alias for None, and None can be used in place of this with the same result. From the above example:

```
>>> x[:, np.newaxis, :, :].shape
(2, 1, 3, 1)
>>> x[:, None, :, :].shape
(2, 1, 3, 1)
```

This can be handy to combine two arrays in a way that otherwise would require explicit reshaping operations. For example:

```
>>> x = np.arange(5)
>>> x[:, np.newaxis] + x[np.newaxis, :]
array([[0, 1, 2, 3, 4],
  [1, 2, 3, 4, 5],
  [2, 3, 4, 5, 6],
  [3, 4, 5, 6, 7],
  [4, 5, 6, 7, 8]])
```

7.4.4 Broadcasting Overview

https://numpy.org/doc/stable/user/basics.broadcasting.html#basics-broadcasting

The term broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations. Subject to certain constraints, the smaller array is "broadcast" across the larger array so that they have compatible shapes. Broadcasting provides a means of vectorizing array operations so that looping occurs in C instead of Python. It does this without making needless copies of data and usually leads to efficient algorithm implementations. There are, however, cases where broadcasting is a bad idea because it leads to inefficient use of memory that slows computation.

NumPy operations are usually done on pairs of arrays on an element-by-element basis. In the simplest case, the two arrays must have exactly the same shape, as in the following example:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> a = np.array([1.0, 2.0, 3.0])
>>> b = np.array([2.0, 2.0, 2.0])
>>> a * b
array([2.,  4.,  6.])
```

NumPy's broadcasting rule relaxes this constraint when the arrays' shapes meet certain constraints. The simplest broadcasting example occurs when an array and a scalar value are combined in an operation:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> a = np.array([1.0, 2.0, 3.0])
>>> b = 2.0
```

(continues on next page)

(continued from previous page)

```
>>> a * b
array([2.,  4.,  6.])
```

The result is equivalent to the previous example where b was an array. We can think of the scalar b being stretched during the arithmetic operation into an array with the same shape as a. The new elements in b, as shown in Figure 1, are simply copies of the original scalar. The stretching analogy is only conceptual. NumPy is smart enough to use the original scalar value without actually making copies so that broadcasting operations are as memory and computationally efficient as possible.

FIGURE 1

The code in the second example is more efficient than that in the first because broadcasting moves less memory around during the multiplication (b is a scalar rather than an array).

7.4.5 General Broadcasting Rules

When operating on two arrays, NumPy compares their shapes element-wise. It starts with the trailing (i.e. rightmost) dimension and works its way left. Two dimensions are compatible when they are equal, or one of them is 1.

If these conditions are not met, a `ValueError: operands could not be broadcast together` exception is thrown, indicating that the arrays have incompatible shapes.

Input arrays do not need to have the same *number* of dimensions. The resulting array will have the same number of dimensions as the input array with the greatest number of dimensions, where the size of each dimension is the largest size of the corresponding dimension among the input arrays. Note that missing dimensions are assumed to have size one.

For example, if you have a 256x256x3 array of RGB values, and you want to scale each color in the image by a different value, you can multiply the image by a one-dimensional array with 3 values. Lining up the sizes of the trailing axes of these arrays according to the broadcast rules, shows that they are compatible:

```
Image  (3d array): 256 x 256 x 3
Scale  (1d array):             3
Result (3d array): 256 x 256 x 3
```

When either of the dimensions compared is one, the other is used. In other words, dimensions with size 1 are stretched or "copied" to match the other.

In the following example, both the A and B arrays have axes with length one that are expanded to a larger size during the broadcast operation:

```
A      (4d array):  8 x 1 x 6 x 1
B      (3d array):      7 x 1 x 5
Result (4d array):  8 x 7 x 6 x 5
```

7.4.6 Broadcastable arrays

A set of arrays is called "broadcastable" to the same shape if the above rules produce a valid result.

For example, if a.shape is (5,1), b.shape is (1,6), c.shape is (6,) and d.shape is () so that d is a scalar, then a, b, c, and d are all broadcastable to dimension (5,6); and

a acts like a (5,6) array where a[:,0] is broadcast to the other columns,

b acts like a (5,6) array where b[0,:] is broadcast to the other rows,

c acts like a (1,6) array and therefore like a (5,6) array where c[:] is broadcast to every row, and finally,

d acts like a (5,6) array where the single value is repeated.

Here are some more examples:

```
A      (2d array):  5 x 4
B      (1d array):      1
Result (2d array):  5 x 4

A      (2d array):  5 x 4
B      (1d array):      4
Result (2d array):  5 x 4

A      (3d array):  15 x 3 x 5
B      (3d array):  15 x 1 x 5
Result (3d array):  15 x 3 x 5

A      (3d array):  15 x 3 x 5
B      (2d array):       3 x 5
Result (3d array):  15 x 3 x 5

A      (3d array):  15 x 3 x 5
B      (2d array):       3 x 1
Result (3d array):  15 x 3 x 5
```

Here are examples of shapes that do not broadcast:

```
A      (1d array):  3
B      (1d array):  4 # trailing dimensions do not match

A      (2d array):      2 x 1
B      (3d array):  8 x 4 x 3 # second from last dimensions mismatched
```

An example of broadcasting when a 1-d array is added to a 2-d array:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> a = np.array([[ 0.0,  0.0,  0.0], [10.0, 10.0, 10.0], [20.0, 20.0, 20.0], [30.0, 30.0, 30.0]])
>>> b = np.array([1.0, 2.0, 3.0])
>>> a + b
array([[  1.,   2.,   3.],
       [11.,  12.,  13.],
       [21.,  22.,  23.],
       [31.,  32.,  33.]])
>>> b = np.array([1.0, 2.0, 3.0, 4.0])
>>> a + b
Traceback (most recent call last):
ValueError: operands could not be broadcast together with shapes (4,3) (4,)
```

When the trailing dimensions of the arrays are unequal, broadcasting fails because it is impossible to align the values in the rows of the 1st array with the elements of the 2nd arrays for element-by-element addition.

Broadcasting provides a convenient way of taking the outer product (or any other outer operation) of two arrays. The following example shows an outer addition operation of two 1-d arrays:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> a = np.array([0.0, 10.0, 20.0, 30.0])
>>> b = np.array([1.0, 2.0, 3.0])
>>> a[:, np.newaxis] + b
array([[  1.,   2.,   3.],
       [11.,  12.,  13.],
       [21.,  22.,  23.],
       [31.,  32.,  33.]])
```

Here the newaxis index operator inserts a new axis into a, making it a two-dimensional 4x1 array. Combining the 4x1 array with b, which has shape (3,), yields a 4x3 array.

7.4.7 Assigning values to indexed arrays

Some text

https://numpy.org/doc/stable/user/basics.indexing.html#assigning-values-to-indexed-arrays

As mentioned, one can select a subset of an array to assign to using a single index, slices, and index and mask arrays. The value being assigned to the indexed array must be shape consistent (the same shape or broadcastable to the shape the index produces). For example, it is permitted to assign a constant to a slice:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.arange(10)
>>> x[2:7] = 1
```

or an array of the right size:

```
>>> x[2:7] = np.arange(5)
```

Note that assignments may result in changes if assigning higher types to lower types (like floats to ints) or even exceptions (assigning complex to floats or ints):

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x[1] = 1.2
>>> x[1]
1
>>> x[1] = 1.2j
Traceback (most recent call last):
  ...
TypeError: can't convert complex to int
```

Unlike some of the references (such as array and mask indices) assignments are always made to the original data in the array (indeed, nothing else would make sense!). Note though, that some actions may not work as one may naively expect. This particular example is often surprising to people:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.arange(0, 50, 10)
>>> x
array([ 0, 10, 20, 30, 40])
>>> x[np.array([1, 1, 3, 1])] += 1
>>> x
array([ 0, 11, 20, 31, 40])
```

Where people expect that the 1st location will be incremented by 3. In fact, it will only be incremented by 1. The reason is that a new array is extracted from the original (as a temporary) containing the values at 1, 1, 3, 1, then the value 1 is added to the temporary, and then the temporary is assigned back to the original array. Thus the value of the array at x[1] + 1 is assigned to x[1] three times, rather than being incremented 3 times.

7.5 Numpy basic array manipulation routines

Some text

https://numpy.org/doc/stable/reference/routines.array-manipulation.html

7.5.1 Copy values from one array to another: numpy.copyto

npm.**copyto**(*dst*, *src*, *casting='same_kind'*, *where=True*)

Copies values from one array to another, broadcasting as necessary. Raises a TypeError if the casting rule is violated, and if where is provided, it selects which elements to copy.

See https://numpy.org/doc/stable/reference/generated/numpy.copyto.html#numpy.copyto for details

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> A = np.array([4, 5, 6])
>>> B = [1, 2, 3]
>>> np.copyto(A, B)
>>> A
array([1, 2, 3])

>>> A = np.array([[1, 2, 3], [4, 5, 6]])
>>> B = [[4, 5, 6], [7, 8, 9]]
>>> np.copyto(A, B)
>>> A
array([[4, 5, 6],
       [7, 8, 9]])
```

7.5.2 Get the shape of an array: numpy.shape

npm.**shape**(*a*)

Return the shape of an array.

See https://numpy.org/doc/stable/reference/generated/numpy.shape.html#numpy.shape for details

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> np.shape(np.eye(3))
(3, 3)
>>> np.shape([[1, 3]])
(1, 2)
>>> np.shape([0])
(1,)
>>> np.shape(0)
()

>>> a = np.array([(1, 2), (3, 4), (5, 6)], dtype=[('x', 'i4'), ('y', 'i4')])
>>> np.shape(a)
(3,)
>>> a.shape
(3,)
```

7.5.3 Reshape an array without changing its data: numpy.reshape

npm.**reshape**(*a*, */ (Positional-only parameter separator (PEP 570))*, *shape=None*, *order='C'*, ***, *newshape=None*, *copy=None*)

Gives a new shape to an array without changing its data.

See https://numpy.org/doc/stable/reference/generated/numpy.reshape.html#numpy.reshape for details

It is not always possible to change the shape of an array without copying the data.

The order keyword gives the index ordering both for fetching the values from a, and then placing the values into the output array. For example, let's say you have an array:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> a = np.arange(6).reshape((3, 2))
>>> a
array([[0, 1],
       [2, 3],
       [4, 5]])
```

You can think of reshaping as first raveling the array (using the given index order), then inserting the elements from the raveled array into the new array using the same kind of index ordering as was used for the raveling.

```
>>> np.reshape(a, (2, 3)) # C-like index ordering
array([[0, 1, 2],
       [3, 4, 5]])
>>> np.reshape(np.ravel(a), (2, 3)) # equivalent to C ravel then C reshape
array([[0, 1, 2],
       [3, 4, 5]])
>>> np.reshape(a, (2, 3), order='F') # Fortran-like index ordering
array([[0, 4, 3],
       [2, 1, 5]])
>>> np.reshape(np.ravel(a, order='F'), (2, 3), order='F')
array([[0, 4, 3],
       [2, 1, 5]])
```

Additional examples:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> a = np.array([[1,2,3], [4,5,6]])
>>> np.reshape(a, 6)
array([1, 2, 3, 4, 5, 6])
>>> np.reshape(a, 6, order='F')
array([1, 4, 2, 5, 3, 6])

>>> np.reshape(a, (3,-1))       # the unspecified value is inferred to be 2
array([[1, 2],
       [3, 4],
       [5, 6]])
```

7.5.4 Get a contiguous flattened array: numpy.ravel

npm.**ravel**(*a*, *order='C'*)

Return a contiguous flattened array.

See https://numpy.org/doc/stable/reference/generated/numpy.ravel.html#numpy.ravel for details

A 1-D array, containing the elements of the input, is returned. A copy is made only if needed.

As of NumPy 1.10, the returned array will have the same type as the input array. (for example, a masked array will be returned for a masked array input)

It is equivalent to reshape(-1, order=order).

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.array([[1, 2, 3], [4, 5, 6]])
>>> np.ravel(x)
array([1, 2, 3, 4, 5, 6])

>>> x.reshape(-1)
```

(continues on next page)

(continued from previous page)

```
array([1, 2, 3, 4, 5, 6])

>>> np.ravel(x, order='F')
array([1, 4, 2, 5, 3, 6])
```

When order is 'A', it will preserve the array's 'C' or 'F' ordering:

```
>>> np.ravel(x.T)
array([1, 4, 2, 5, 3, 6])
>>> np.ravel(x.T, order='A')
array([1, 2, 3, 4, 5, 6])
```

When order is 'K', it will preserve orderings that are neither 'C' nor 'F', but won't reverse axes:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> a = np.arange(3)[::-1]; a
array([2, 1, 0])
>>> a.ravel(order='C')
array([2, 1, 0])
>>> a.ravel(order='K')
array([2, 1, 0])

>>> a = np.arange(12).reshape(2,3,2).swapaxes(1,2); a
array([[[ 0,  2,  4], [ 1,  3,  5]], [[ 6,  8, 10], [ 7,  9, 11]]])
>>> a.ravel(order='C')
array([ 0,  2,  4,  1,  3,  5,  6,  8, 10,  7,  9, 11])
>>> a.ravel(order='K')
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

7.5.5 1-D iterator over the array: ndarry.flat

`ndarry.flat()`

A 1-D iterator over the array.

See https://numpy.org/doc/stable/reference/generated/numpy.ndarray.flat.html#numpy.ndarray.flat for details.

This is a numpy.flatiter instance, which acts similarly to, but is not a subclass of, Python's built-in iterator object.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.arange(1, 7).reshape(2, 3)
>>> x
array([[1, 2, 3],
       [4, 5, 6]])
>>> x.flat[3]
4
>>> x.T
array([[1, 4],
       [2, 5],
       [3, 6]])
>>> x.T.flat[3]
5
>>> type(x.flat)
<class 'numpy.flatiter'>
```

An assignment example:

```
>>> x.flat = 3; x
array([[3, 3, 3],
       [3, 3, 3]])
>>> x.flat[[1,4]] = 1; x
```

(continues on next page)

(continued from previous page)

```
array([[3, 1, 3],
       [3, 1, 3]])
```

7.5.6 Get a copy of the array collapsed into one dimension: ndarry.flatten

ndarry.**flatten**(*order='C'*)

Return a copy of the array collapsed into one dimension.

See https://numpy.org/doc/stable/reference/generated/numpy.ndarray.flatten.html#numpy.ndarray.flatten for details.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> a = np.array([[1,2], [3,4]])
>>> a.flatten()
array([1, 2, 3, 4])
>>> a.flatten('F')
array([1, 3, 2, 4])
```

7.6 Numpy array manipulation: Transpose-like operations

7.6.1 Move axes of an array to new positions: numpy.flatten

npm.**moveaxis**(*a*, *source*, *destination*)

Move axes of an array to new positions. Other axes remain in their original order.

See https://numpy.org/doc/stable/reference/generated/numpy.moveaxis.html#numpy.moveaxis for details.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.zeros((3, 4, 5))
>>> np.moveaxis(x, 0, -1).shape
(4, 5, 3)
>>> np.moveaxis(x, -1, 0).shape
(5, 3, 4)
```

These all achieve the same result:

```
>>> np.transpose(x).shape
(5, 4, 3)
>>> np.swapaxes(x, 0, -1).shape
(5, 4, 3)
>>> np.moveaxis(x, [0, 1], [-1, -2]).shape
(5, 4, 3)
>>> np.moveaxis(x, [0, 1, 2], [-1, -2, -3]).shape
(5, 4, 3)
```

7.6.2 Interchange two axes of an array: numpy.swapaxes

npm.**swapaxes**(*a*, *axis1*, *axis2*)

Interchange two axes of an array.

See https://numpy.org/doc/stable/reference/generated/numpy.swapaxes.html#numpy.swapaxes for details.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.array([[1,2,3]])
>>> np.swapaxes(x,0,1)
array([[1],
       [2],
       [3]])

>>> x = np.array([[[0,1],[2,3]],[[4,5],[6,7]]])
>>> x
array([[[0, 1],
        [2, 3]],
       [[4, 5],
        [6, 7]]])

>>> np.swapaxes(x,0,2)
array([[[0, 4],
        [2, 6]],
       [[1, 5],
        [3, 7]]])
```

7.6.3 Get an array with axes transposed: numpy.transpose

npm.**transpose**(*a*, *axes=None*)

Returns an array with axes transposed.

See https://numpy.org/doc/stable/reference/generated/numpy.transpose.html#numpy.transpose for details.

For a 1-D array, this returns an unchanged view of the original array, as a transposed vector is simply the same vector. To convert a 1-D array into a 2-D column vector, an additional dimension must be added, e.g., np.atleast2d(a).T achieves this, as does a[:, np.newaxis]. For a 2-D array, this is the standard matrix transpose. For an n-D array, if axes are given, their order indicates how the axes are permuted (see Examples). If axes are not provided, then transpose(a).shape == a.shape[::-1].

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> a = np.array([[1, 2], [3, 4]])
a
array([[1, 2],
       [3, 4]])
>>> np.transpose(a)
array([[1, 3],
       [2, 4]])

>>> a = np.array([1, 2, 3, 4])
>>> a
array([1, 2, 3, 4])
>>> np.transpose(a)
array([1, 2, 3, 4])

>>> a = np.ones((1, 2, 3))
>>> np.transpose(a, (1, 0, 2)).shape
(2, 1, 3)

>>> a = np.ones((2, 3, 4, 5))
>>> np.transpose(a).shape
(5, 4, 3, 2)
```

ndarray.T

https://numpy.org/doc/stable/reference/generated/numpy.ndarray.T.html#numpy.ndarray.T

View of the transposed array. Same as self.transpose().

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> a = np.array([[1, 2], [3, 4]])
a
array([[1, 2],
       [3, 4]])
>>> a.T
array([[1, 3],
       [2, 4]])

>>> a = np.array([1, 2, 3, 4])
>>> a
array([1, 2, 3, 4])
>>> a.T
array([1, 2, 3, 4])
```

7.7 Numpy array manipulation: Changing number of dimensions

7.7.1 Convert inputs to arrays with at least one dimension: numpy.atleast_1d

npm.**atleast_1d**(**arys*)

Convert inputs to arrays with at least one dimension.

See https://numpy.org/doc/stable/reference/generated/numpy.atleast_1d.html#numpy.atleast_1d for details

Scalar inputs are converted to 1-dimensional arrays, whilst higher-dimensional inputs are preserved.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> np.atleast_1d(1.0)
array([1.])

>>> x = np.arange(9.0).reshape(3,3)
>>> np.atleast_1d(x)
array([[0., 1., 2.],
       [3., 4., 5.],
       [6., 7., 8.]])
>>> np.atleast_1d(x) is x
True

>>> np.atleast_1d(1, [3, 4])
[array([1]), array([3, 4])]
```

7.7.2 Convert inputs to arrays with at least two dimensions: numpy.atleast_2d

npm.**atleast_2d**(**arys*)

Convert inputs as arrays with at least two dimensions.

See https://numpy.org/doc/stable/reference/generated/numpy.atleast_2d.html#numpy.atleast_2d for details

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> np.atleast_2d(3.0)
array([[3.]])

>>> x = np.arange(3.0)
>>> np.atleast_2d(x)
array([[0., 1., 2.]])
>>> np.atleast_2d(x).base is x
True

>>> np.atleast_2d(1, [1, 2], [[1, 2]])
[array([[1]]), array([[1, 2]]), array([[1, 2]])]
```

7.7.3 Convert inputs to arrays with at least two dimensions: numpy.atleast_3d

npm.**atleast_3d**(**arys*)

Convert inputs as arrays with at least three dimensions.

See https://numpy.org/doc/stable/reference/generated/numpy.atleast_3d.html#numpy.atleast_3d for details.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> np.atleast_3d(3.0)
array([[[3.]]])

>>> x = np.arange(3.0)
```

(continues on next page)

(continued from previous page)

```
>>> np.atleast_3d(x).shape
(1, 3, 1)

>>> x = np.arange(12.0).reshape(4,3)
>>> np.atleast_3d(x).shape
(4, 3, 1)
>>> np.atleast_3d(x).base is x.base  # x is a reshape, so not base itself
True

for arr in np.atleast_3d([1, 2], [[1, 2]], [[[1, 2]]]): print(arr, arr.shape)
[[[1]
  [2]]] (1, 2, 1)
[[[1]
  [2]]] (1, 2, 1)
[[[1 2]]] (1, 1, 2)
```

7.7.4 Broadcast an array to a new shape: numpy.broadcast_to

npm.**broadcast_to**(*array*, *shape*, *subok=False*)

Broadcast an array to a new shape.

See https://numpy.org/doc/stable/reference/generated/numpy.broadcast_to.html#numpy.broadcast_to for details.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.array([1, 2, 3])
>>> np.broadcast_to(x, (3, 3))
array([[1, 2, 3],
       [1, 2, 3],
       [1, 2, 3]])
```

7.7.5 Broadcast any number of arrays against each other: numpy.broadcast_arrays

npm.**broadcast_arrays**(**args*, *subok=False*)

Broadcast any number of arrays against each other.

See https://numpy.org/doc/stable/reference/generated/numpy.broadcast_arrays.html#numpy.broadcast_arrays for details.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.array([[1,2,3]])
>>> y = np.array([[4],[5]])
>>> np.broadcast_arrays(x, y)
[array([[1, 2, 3],
       [1, 2, 3]]), array([[4, 4, 4],
       [5, 5, 5]])]
```

Here is a useful idiom for getting contiguous copies instead of non-contiguous views.

```
>>> [np.array(a) for a in np.broadcast_arrays(x, y)]
[array([[1, 2, 3],
       [1, 2, 3]]), array([[4, 4, 4],
       [5, 5, 5]])]
```

7.7.6 Expand the shape of an array: numpy.expand_dims

npm.**expand_dims**(*a*, *axis*)

Expand the shape of an array. Insert a new axis that will appear at the axis position in the expanded array shape.

See https://numpy.org/doc/stable/reference/generated/numpy.expand_dims.html#numpy.expand_dims for details.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.array([1, 2])
>>> x.shape
(2,)
```

The following is equivalent to `x[np.newaxis, :]` or `x[np.newaxis]`:

```
>>> y = np.expand_dims(x, axis=0)
>>> y
array([[1, 2]])
>>> y.shape
(1, 2)
```

The following is equivalent to `x[:, np.newaxis]`:

```
>>> y = np.expand_dims(x, axis=1)
>>> y
array([[1],
       [2]])
>>> y.shape
(2, 1)
```

`axis` may also be a tuple:

```
>>> y = np.expand_dims(x, axis=(0, 1))
>>> y
array([[[1, 2]]])
>>> y = np.expand_dims(x, axis=(2, 0))
>>> y
array([[[1],
        [2]]])
```

Note that some examples may use `None` instead of `np.newaxis`. These are the same objects:

```
>>> np.newaxis is None
True
```

7.7.7 Remove axes of length one from an array: numpy.squeeze

npm.**squeeze**(*a*, *axis=None*)

Remove axes of length one from a.

https://numpy.org/doc/stable/reference/generated/numpy.squeeze.html#numpy.squeeze

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> x = np.array([[[0], [1], [2]]])
>>> x.shape
(1, 3, 1)
>>> np.squeeze(x).shape
(3,)
>>> np.squeeze(x, axis=0).shape
(3, 1)
>>> np.squeeze(x, axis=1).shape
Traceback (most recent call last):
...
ValueError: cannot select an axis to squeeze out which has size not equal to one

>>> np.squeeze(x, axis=2).shape
(1, 3)
>>> x = np.array([[1234]])
```

(continues on next page)

(continued from previous page)

```
>>> x.shape
(1, 1)
>>> np.squeeze(x)
array(1234)  # 0d array
>>> np.squeeze(x).shape
()
>>> np.squeeze(x)[()]
1234
```

7.8 Numpy array manipulation: Joining arrays

7.8.1 Join a sequence of arrays along an existing axis: numpy.concatenate

npm.**concatenate**(*(a1, a2, ...)*, *axis=0*, *out=None*, *dtype=None*, *casting="same_kind"*)

Join a sequence of arrays along an existing axis.

See https://numpy.org/doc/stable/reference/generated/numpy.concatenate.html#numpy.concatenate for details.

When one or more of the arrays to be concatenated is a MaskedArray, this function will return a MaskedArray object instead of an ndarray, but the input masks are not preserved. In cases where a MaskedArray is expected as input, use the ma.concatenate function from the masked array module instead.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> a = np.array([[1, 2], [3, 4]])
>>> b = np.array([[5, 6]])
>>> np.concatenate((a, b), axis=0)
array([[1, 2],
       [3, 4],
       [5, 6]])
>>> np.concatenate((a, b.T), axis=1)
array([[1, 2, 5],
       [3, 4, 6]])
>>> np.concatenate((a, b), axis=None)
array([1, 2, 3, 4, 5, 6])
```

This function will not preserve masking of MaskedArray inputs.

```
>>> a = np.ma.arange(3)
>>> a[1] = np.ma.masked
>>> b = np.arange(2, 5)
>>> a
masked_array(data=[0, --, 2],
             mask=[False,  True, False],
       fill_value=999999)
>>> b
array([2, 3, 4])
>>> np.concatenate([a, b])
masked_array(data=[0, 1, 2, 2, 3, 4],
             mask=False,
       fill_value=999999)
>>> np.ma.concatenate([a, b])
masked_array(data=[0, --, 2, 2, 3, 4],
             mask=[False,  True, False, False, False, False],
       fill_value=999999)
```

7.8.2 Join a sequence of arrays along an existing axis: numpy.stack

npm.**stack**(*arrays*, *axis=0*, *out=None*, *, *dtype=None*, *casting='same_kind'*)

Join a sequence of arrays along a new axis.

See https://numpy.org/doc/stable/reference/generated/numpy.stack.html#numpy.stack for details.

The axis parameter specifies the index of the new axis in the dimensions of the result. For example, if axis=0 it will be the first dimension and if axis=-1 it will be the last dimension.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> arrays = [np.random.randn(3, 4) for _ in range(10)]
>>> np.stack(arrays, axis=0).shape
(10, 3, 4)
```

(continues on next page)

(continued from previous page)

```
>>> np.stack(arrays, axis=1).shape
(3, 10, 4)

>>> np.stack(arrays, axis=2).shape
(3, 4, 10)

>>> a = np.array([1, 2, 3])
>>> b = np.array([4, 5, 6])
>>> np.stack((a, b))
array([[1, 2, 3],
       [4, 5, 6]])

>>> np.stack((a, b), axis=-1)
array([[1, 4],
       [2, 5],
       [3, 6]])
```

7.8.3 Assemble an nd-array from nested lists of blocks: numpy.block

npm.block(*arrays*)

Assemble an nd-array from nested lists of blocks.

See https://numpy.org/doc/stable/reference/generated/numpy.block.html#numpy.block for details.

Blocks in the innermost lists are concatenated (see concatenate) along the last dimension (-1), then these are concatenated along the second-last dimension (-2), and so on until the outermost list is reached.

Blocks can be of any dimension, but will not be broadcasted using the normal rules. Instead, leading axes of size 1 are inserted, to make block.ndim the same for all blocks. This is primarily useful for working with scalars, and means that code like np.block([v, 1]) is valid, where v.ndim == 1.

When the nested list is two levels deep, this allows block matrices to be constructed from their components.

When called with only scalars, `np.block` is equivalent to an ndarray call. So `np.block([[1, 2], [3, 4]])` is equivalent to `np.array([[1, 2], [3, 4]])`.

This function does not enforce that the blocks lie on a fixed grid. `np.block([[a, b], [c, d]])` is not restricted to arrays of the form:

```
AAAbb
AAAbb
cccDD
```

But is also allowed to produce, for some a, b, c, d:

```
AAAbb
AAAbb
cDDDD
```

Since concatenation happens along the last axis first, block is not capable of producing the following directly:

```
AAAbb
cccbb
cccDD
```

```
>>> import numpy as np; from xlcalcnet import mpm, dpm

>>> matA = np.eye(2) * 2
>>> B = np.eye(3) * 3
>>> np.block([
```

(continues on next page)

(continued from previous page)

```
    [matA,               np.zeros((2, 3))],
    [np.ones((3, 2)), B               ]
])
array([[2., 0., 0., 0., 0.],
       [0., 2., 0., 0., 0.],
       [1., 1., 3., 0., 0.],
       [1., 1., 0., 3., 0.],
       [1., 1., 0., 0., 3.]])
```

With a list of depth 1, block can be used as hstack

```
>>> np.block([1, 2, 3])              # hstack([1, 2, 3])
array([1, 2, 3])

>>> a = np.array([1, 2, 3])
>>> b = np.array([4, 5, 6])
>>> np.block([a, b, 10])             # hstack([a, b, 10])
array([ 1,  2,  3,  4,  5,  6, 10])

>>> A = np.ones((2, 2), int)
>>> B = 2 * A
>>> np.block([A, B])                 # hstack([A, B])
array([[1, 1, 2, 2],
       [1, 1, 2, 2]])
```

With a list of depth 2, block can be used in place of vstack:

```
>>> a = np.array([1, 2, 3])
>>> b = np.array([4, 5, 6])
>>> np.block([[a], [b]])             # vstack([a, b])
array([[1, 2, 3],
       [4, 5, 6]])

>>> A = np.ones((2, 2), int)
>>> B = 2 * A
>>> np.block([[A], [B]])             # vstack([A, B])
array([[1, 1],
       [1, 1],
       [2, 2],
       [2, 2]])
```

It can also be used in places of atleast_1d and atleast_2d

```
>>> a = np.array(0)
>>> b = np.array([1])
>>> np.block([a])                    # atleast_1d(a)
array([0])
>>> np.block([b])                    # atleast_1d(b)
array([1])

>>> np.block([[a]])                  # atleast_2d(a)
array([[0]])
np.block([[b]])                   # atleast_2d(b)
array([[1]])
```

7.8.4 Stack arrays in sequence vertically (row wise): numpy.vstack

npm.**vstack**(*tup*, *, *dtype=None*, *casting='same_kind'*)

Stack arrays in sequence vertically (row wise).

See https://numpy.org/doc/stable/reference/generated/numpy.vstack.html#numpy.vstack for details.

This is equivalent to concatenation along the first axis after 1-D arrays of shape (N,) have been reshaped to (1,N). Rebuilds arrays divided by vsplit.

This function makes most sense for arrays with up to 3 dimensions. For instance, for pixel-data with a height (first axis), width (second axis), and r/g/b channels (third axis). The functions concatenate, stack and block provide more general stacking and concatenation operations.

```
>>> a = np.array([1, 2, 3])
>>> b = np.array([4, 5, 6])
>>> np.vstack((a,b))
array([[1, 2, 3],
       [4, 5, 6]])

>>> a = np.array([[1], [2], [3]])
>>> b = np.array([[4], [5], [6]])
>>> np.vstack((a,b))
array([[1],
       [2],
       [3],
       [4],
       [5],
       [6]])
```

7.8.5 Stack arrays in sequence horizontally (column wise): numpy.hstack

npm.**hstack**(*tup*, ***, *dtype=None*, *casting='same_kind'*)

Stack arrays in sequence horizontally (column wise).

See https://numpy.org/doc/stable/reference/generated/numpy.hstack.html#numpy.hstack for details.

This is equivalent to concatenation along the second axis, except for 1-D arrays where it concatenates along the first axis. Rebuilds arrays divided by hsplit.

This function makes most sense for arrays with up to 3 dimensions. For instance, for pixel-data with a height (first axis), width (second axis), and r/g/b channels (third axis). The functions concatenate, stack and block provide more general stacking and concatenation operations.

```
>>> a = np.array((1,2,3))
>>> b = np.array((4,5,6))
>>> np.hstack((a,b))
array([1, 2, 3, 4, 5, 6])

>>> a = np.array([[1],[2],[3]])
>>> b = np.array([[4],[5],[6]])
>>> np.hstack((a,b))
array([[1, 4],
       [2, 5],
       [3, 6]])
```

7.8.6 Stack arrays in sequence depth wise (along third axis): numpy.dstack

npm.**dstack**(*tup*)

Stack arrays in sequence depth wise (along third axis).

https://numpy.org/doc/stable/reference/generated/numpy.dstack.html#numpy.dstack

This is equivalent to concatenation along the third axis after 2-D arrays of shape (M,N) have been reshaped to (M,N,1) and 1-D arrays of shape (N,) have been reshaped to (1,N,1). Rebuilds arrays divided by dsplit.

This function makes most sense for arrays with up to 3 dimensions. For instance, for pixel-data with a height (first axis), width (second axis), and r/g/b channels (third axis). The functions concatenate, stack and block provide more general stacking and concatenation operations.

```
>>> a = np.array((1,2,3))
>>> b = np.array((2,3,4))
>>> np.dstack((a,b))
array([[[1, 2],
        [2, 3],
        [3, 4]]])

>>> a = np.array([[1],[2],[3]])
>>> b = np.array([[2],[3],[4]])
>>> np.dstack((a,b))
array([[[1, 2]],
       [[2, 3]],
       [[3, 4]]])
```

7.8.7 Stack 1-D arrays as columns into a 2-D array: numpy.column_stack

npm.**column_stack**(*tup*)

Stack 1-D arrays as columns into a 2-D array.

See https://numpy.org/doc/stable/reference/generated/numpy.column_stack.html#numpy.column_stack for details.

Take a sequence of 1-D arrays and stack them as columns to make a single 2-D array. 2-D arrays are stacked as-is, just like with hstack. 1-D arrays are turned into 2-D columns first.

```
>>> a = np.array((1,2,3))
>>> b = np.array((2,3,4))
>>> np.column_stack((a,b))
array([[1, 2],
        [2, 3],
        [3, 4]])
```

7.9 Numpy array manipulation: Splitting and tiling arrays

7.9.1 Split an array into multiple sub-arrays as views: numpy.split

npm.**split**(*ary*, *indices_or_sections*, *axis=0*)

Split an array into multiple sub-arrays as views.

See https://numpy.org/doc/stable/reference/generated/numpy.split.html#numpy.split for details.

```
>>> x = np.arange(9.0)
>>> np.split(x, 3)
[array([0.,  1.,  2.]), array([3.,  4.,  5.]), array([6.,  7.,  8.])]

>>> x = np.arange(8.0)
>>> np.split(x, [3, 5, 6, 10])
[array([0.,  1.,  2.]),
 array([3.,  4.]),
 array([5.]),
 array([6.,  7.]),
 array([], dtype=float64)]
```

7.9.2 Split an array into multiple sub-arrays: numpy.array_split

npm.**array_split**(*ary*, *indices_or_sections*, *axis=0*)

Split an array into multiple sub-arrays.

See https://numpy.org/doc/stable/reference/generated/numpy.array_split.html#numpy.array_split for details.

Please refer to the split documentation. The only difference between these functions is that array_split allows indices_or_sections to be an integer that does not equally divide the axis. For an array of length l that should be split into n sections, it returns l % n sub-arrays of size l//n + 1 and the rest of size l//n.

```
>>> x = np.arange(8.0)
>>> np.array_split(x, 3)
[array([0.,  1.,  2.]), array([3.,  4.,  5.]), array([6.,  7.])]

>>> x = np.arange(9)
>>> np.array_split(x, 4)
[array([0, 1, 2]), array([3, 4]), array([5, 6]), array([7, 8])]
```

7.9.3 Split array into multiple sub-arrays along the 3rd axis (depth): numpy.dsplit

npm.**dsplit**(*ary*, *indices_or_sections*)

Split array into multiple sub-arrays along the 3rd axis (depth).

See https://numpy.org/doc/stable/reference/generated/numpy.dsplit.html#numpy.dsplit for details.

Please refer to the split documentation. dsplit is equivalent to split with axis=2, the array is always split along the third axis provided the array dimension is greater than or equal to 3.

```
>>> x = np.arange(16.0).reshape(2, 2, 4)
>>> x
array([[[ 0.,   1.,   2.,   3.],
        [ 4.,   5.,   6.,   7.]],
       [[ 8.,   9.,  10.,  11.],
        [12.,  13.,  14.,  15.]]])

>>> np.dsplit(x, 2)
[array([[[ 0.,  1.],
        [ 4.,  5.]],
```

(continues on next page)

(continued from previous page)

```
       [[ 8.,  9.],
        [12., 13.]]]), array([[[ 2.,  3.],
        [ 6.,  7.]],
       [[10., 11.],
        [14., 15.]]])]

>>> np.dsplit(x, np.array([3, 6]))
[array([[[ 0.,  1.,  2.],
        [ 4.,  5.,  6.]],
       [[ 8.,  9., 10.],
        [12., 13., 14.]]]),
 array([[[ 3.],
        [ 7.]],
       [[11.],
        [15.]]]),
array([], shape=(2, 2, 0), dtype=float64)]
```

7.9.4 Split an array into multiple sub-arrays horizontally (column-wise): numpy.hsplit

npm.**hsplit**(*ary*, *indices_or_sections*)

Split an array into multiple sub-arrays horizontally (column-wise).

See https://numpy.org/doc/stable/reference/generated/numpy.hsplit.html#numpy.hsplit for details.

Please refer to the split documentation. hsplit is equivalent to split with axis=1, the array is always split along the second axis except for 1-D arrays, where it is split at axis=0.

```
>>> x = np.arange(16.0).reshape(4, 4)
>>> x
array([[ 0.,  1.,  2.,  3.],
       [ 4.,  5.,  6.,  7.],
       [ 8.,  9., 10., 11.],
       [12., 13., 14., 15.]])

>>> np.hsplit(x, 2)
[array([[ 0.,  1.],
       [ 4.,  5.],
       [ 8.,  9.],
       [12., 13.]]),
 array([[ 2.,  3.],
       [ 6.,  7.],
       [10., 11.],
       [14., 15.]])]

>>> np.hsplit(x, np.array([3, 6]))
[array([[ 0.,  1.,  2.],
       [ 4.,  5.,  6.],
       [ 8.,  9., 10.],
       [12., 13., 14.]]),
 array([[ 3.],
       [ 7.],
       [11.],
       [15.]]),
 array([], shape=(4, 0), dtype=float64)]
```

With a higher dimensional array the split is still along the second axis.

```
>>> x = np.arange(8.0).reshape(2, 2, 2)
>>> x
array([[[0.,  1.],
        [2.,  3.]],
       [[4.,  5.],
        [6.,  7.]]])
```

(continues on next page)

(continued from previous page)

```
>>> np.hsplit(x, 2)
[array([[[0.,  1.]],
       [[4.,  5.]]]),
 array([[[2.,  3.]],
       [[6.,  7.]]])]
```

With a 1-D array, the split is along axis 0.

```
>>> x = np.array([0, 1, 2, 3, 4, 5])
>>> np.hsplit(x, 2)
[array([0, 1, 2]), array([3, 4, 5])]
```

7.9.5 Split an array into multiple sub-arrays vertically (row-wise): numpy.vsplit

npm.**vsplit**(*ary*, *indices_or_sections*)

Split an array into multiple sub-arrays vertically (row-wise).

See https://numpy.org/doc/stable/reference/generated/numpy.vsplit.html#numpy.vsplit for details.

Please refer to the split documentation. vsplit is equivalent to split with axis=0 (default), the array is always split along the first axis regardless of the array dimension.

```
>>> x = np.arange(16.0).reshape(4, 4)
>>> x
array([[ 0.,   1.,   2.,   3.],
       [ 4.,   5.,   6.,   7.],
       [ 8.,   9.,  10.,  11.],
       [12.,  13.,  14.,  15.]])

>>> np.vsplit(x, 2)
[array([[0., 1., 2., 3.],
       [4., 5., 6., 7.]]), array([[ 8.,  9., 10., 11.],
       [12., 13., 14., 15.]])]

>>> np.vsplit(x, np.array([3, 6]))
[array([[ 0.,  1.,  2.,  3.],
       [ 4.,  5.,  6.,  7.],
       [ 8.,  9., 10., 11.]]), array([[12., 13., 14., 15.]]), array([], shape=(0, 4), dtype=float64)]
```

With a higher dimensional array the split is still along the first axis.

```
>>> x = np.arange(8.0).reshape(2, 2, 2)
>>> x
array([[[0.,  1.],
        [2.,  3.]],
       [[4.,  5.],
        [6.,  7.]]])

>>> np.vsplit(x, 2)
[array([[[0., 1.],
        [2., 3.]]]), array([[[4., 5.],
        [6., 7.]]])]
```

7.9.6 Construct an array by repeating a given array: numpy.tile

npm.**tile**(*A*, *reps*)

Construct an array by repeating A the number of times given by reps.

See https://numpy.org/doc/stable/reference/generated/numpy.tile.html#numpy.tile for details.

If reps has length d, the result will have dimension of max(d, A.ndim).

If A.ndim < d, A is promoted to be d-dimensional by prepending new axes. So a shape (3,) array is promoted to (1, 3) for 2-D replication, or shape (1, 1, 3) for 3-D replication. If this is not the desired behavior, promote A to d-dimensions manually before calling this function.

If A.ndim > d, reps is promoted to A.ndim by pre-pending 1's to it. Thus for an A of shape (2, 3, 4, 5), a reps of (2, 2) is treated as (1, 1, 2, 2).

Note : Although tile may be used for broadcasting, it is strongly recommended to use numpy's broadcasting operations and functions.

```
>>> a = np.array([0, 1, 2])
>>> np.tile(a, 2)
array([0, 1, 2, 0, 1, 2])
>>> np.tile(a, (2, 2))
array([[0, 1, 2, 0, 1, 2],
       [0, 1, 2, 0, 1, 2]])
>>> np.tile(a, (2, 1, 2))
array([[[0, 1, 2, 0, 1, 2]],
       [[0, 1, 2, 0, 1, 2]]])

>>> b = np.array([[1, 2], [3, 4]])
>>> np.tile(b, 2)
array([[1, 2, 1, 2],
       [3, 4, 3, 4]])
>>> np.tile(b, (2, 1))
array([[1, 2],
       [3, 4],
       [1, 2],
       [3, 4]])

>>> c = np.array([1,2,3,4])
>>> np.tile(c,(4,1))
array([[1, 2, 3, 4],
       [1, 2, 3, 4],
       [1, 2, 3, 4],
       [1, 2, 3, 4]])
```

7.9.7 Repeat each element of an array after themselves: numpy.repeat

npm.**repeat**(*a*, *repeats*, *axis=None*)

Repeat each element of an array after themselves.

See https://numpy.org/doc/stable/reference/generated/numpy.repeat.html#numpy.repeat for details.

```
>>> np.repeat(3, 4)
array([3, 3, 3, 3])

>>> x = np.array([[1,2],[3,4]])
>>> np.repeat(x, 2)
array([1, 1, 2, 2, 3, 3, 4, 4])

>>> np.repeat(x, 3, axis=1)
array([[1, 1, 1, 2, 2, 2],
       [3, 3, 3, 4, 4, 4]])

>>> np.repeat(x, [1, 2], axis=0)
array([[1, 2],
       [3, 4],
       [3, 4]])
```

7.10 Numpy array manipulation: Adding and removing elements

7.10.1 Pad an array: numpy.pad

npm.**pad**(*array*, *pad_width*, *mode='constant'*, ***kwargs*)

Pad an array.

See https://numpy.org/doc/stable/reference/generated/numpy.pad.html for details.

For an array with rank greater than 1, some of the padding of later axes is calculated from padding of previous axes. This is easiest to think about with a rank 2 array where the corners of the padded array are calculated by using padded values from the first axis.

```
>>> a = [1, 2, 3, 4, 5]
np.pad(a, (2, 3), 'constant', constant_values=(4, 6))
array([4, 4, 1, ..., 6, 6, 6])

>>> np.pad(a, (2, 3), 'edge')
array([1, 1, 1, ..., 5, 5, 5])

>>> np.pad(a, (2, 3), 'linear_ramp', end_values=(5, -4))
array([ 5,  3,  1,  2,  3,  4,  5,  2, -1, -4])

>>> np.pad(a, (2,), 'maximum')
array([5, 5, 1, 2, 3, 4, 5, 5, 5])

>>> np.pad(a, (2,), 'mean')
array([3, 3, 1, 2, 3, 4, 5, 3, 3])

>>> np.pad(a, (2,), 'median')
array([3, 3, 1, 2, 3, 4, 5, 3, 3])

>>> a = [[1, 2], [3, 4]]
>>> np.pad(a, ((3, 2), (2, 3)), 'minimum')
array([[1, 1, 1, 2, 1, 1, 1],
       [1, 1, 1, 2, 1, 1, 1],
       [1, 1, 1, 2, 1, 1, 1],
       [1, 1, 1, 2, 1, 1, 1],
       [3, 3, 3, 4, 3, 3, 3],
       [1, 1, 1, 2, 1, 1, 1],
       [1, 1, 1, 2, 1, 1, 1]])

>>> a = [1, 2, 3, 4, 5]
>>> np.pad(a, (2, 3), 'reflect')
array([3, 2, 1, 2, 3, 4, 5, 4, 3, 2])

>>> np.pad(a, (2, 3), 'reflect', reflect_type='odd')
array([-1,  0,  1,  2,  3,  4,  5,  6,  7,  8])

>>> np.pad(a, (2, 3), 'symmetric')
array([2, 1, 1, 2, 3, 4, 5, 5, 4, 3])

>>> np.pad(a, (2, 3), 'symmetric', reflect_type='odd')
array([0, 1, 1, 2, 3, 4, 5, 5, 6, 7])

>>> np.pad(a, (2, 3), 'wrap')
array([4, 5, 1, 2, 3, 4, 5, 1, 2, 3])

>>> def pad_with(vector, pad_width, iaxis, kwargs):
        pad_value = kwargs.get('padder', 10)
        vector[:pad_width[0]] = pad_value
        vector[-pad_width[1]:] = pad_value
>>> a = np.arange(6)
>>> a = a.reshape((2, 3))
>>> np.pad(a, 2, pad_with)
```

(continues on next page)

(continued from previous page)

```
array([[10, 10, 10, 10, 10, 10, 10],
       [10, 10, 10, 10, 10, 10, 10],
       [10, 10,  0,  1,  2, 10, 10],
       [10, 10,  3,  4,  5, 10, 10],
       [10, 10, 10, 10, 10, 10, 10],
       [10, 10, 10, 10, 10, 10, 10]])
>>> np.pad(a, 2, pad_with, padder=100)
array([[100, 100, 100, 100, 100, 100, 100],
       [100, 100, 100, 100, 100, 100, 100],
       [100, 100,   0,   1,   2, 100, 100],
       [100, 100,   3,   4,   5, 100, 100],
       [100, 100, 100, 100, 100, 100, 100],
       [100, 100, 100, 100, 100, 100, 100]])
```

7.10.2 Return a new array with sub-arrays along an axis deleted: numpy.delete

npm.**delete**(*arr*, *obj*, *axis=None*)

Return a new array with sub-arrays along an axis deleted. For a one dimensional array, this returns those entries not returned by arr[obj].

See https://numpy.org/doc/stable/reference/generated/numpy.delete.html#numpy.delete for details.

Often it is preferable to use a boolean mask. For example:

```
>>> arr = np.arange(12) + 1
>>> mask = np.ones(len(arr), dtype=bool)
>>> mask[[0,2,4]] = False
>>> result = arr[mask,...]
```

Is equivalent to `np.delete(arr, [0,2,4], axis=0)`, but allows further use of *mask*.

```
>>> arr = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
>>> arr
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
>>> np.delete(arr, 1, 0)
array([[ 1,  2,  3,  4],
       [ 9, 10, 11, 12]])

>>> np.delete(arr, np.s_[::2], 1)
array([[ 2,  4],
       [ 6,  8],
       [10, 12]])
>>> np.delete(arr, [1,3,5], None)
array([ 1,  3,  5,  7,  8,  9, 10, 11, 12])
```

7.10.3 Insert values along the given axis before the given indices: numpy.insert

npm.**insert**(*arr*, *obj*, *values*, *axis=None*)

Insert values along the given axis before the given indices.

See https://numpy.org/doc/stable/reference/generated/numpy.insert.html#numpy.insert for details.

Note that for higher dimensional inserts `obj=0` behaves very different from `obj=[0]` just like `arr[:,0,:] = values` is different from `arr[:,[0],:] = values`.

```
>>> a = np.array([[1, 1], [2, 2], [3, 3]])
>>> a
array([[1, 1],
       [2, 2],
```

(continues on next page)

(continued from previous page)

```
       [3, 3]])
>>> np.insert(a, 1, 5)
array([1, 5, 1, ..., 2, 3, 3])
>>> np.insert(a, 1, 5, axis=1)
array([[1, 5, 1],
       [2, 5, 2],
       [3, 5, 3]])
```

Difference between sequence and scalars:

```
>>> np.insert(a, [1], [[1],[2],[3]], axis=1)
array([[1, 1, 1],
       [2, 2, 2],
       [3, 3, 3]])
>>> np.array_equal(np.insert(a, 1, [1, 2, 3], axis=1),
               np.insert(a, [1], [[1],[2],[3]], axis=1))
True

>>> b = a.flatten()
>>> b
array([1, 1, 2, 2, 3, 3])
>>> np.insert(b, [2, 2], [5, 6])
array([1, 1, 5, ..., 2, 3, 3])

>>> np.insert(b, slice(2, 4), [5, 6])
array([1, 1, 5, ..., 2, 3, 3])

>>> np.insert(b, [2, 2], [7.13, False]) # type casting
array([1, 1, 7, ..., 2, 3, 3])

>>> x = np.arange(8).reshape(2, 4)
>>> idx = (1, 3)
np.insert(x, idx, 999, axis=1)
array([[  0, 999,   1,   2, 999,   3],
       [  4, 999,   5,   6, 999,   7]])
```

7.10.4 Append values to the end of an array: numpy.append

npm.**append**(*arr*, *values*, *axis=None*)

Append values to the end of an array.

See https://numpy.org/doc/stable/reference/generated/numpy.append.html#numpy.append for details.

```
>>> np.np.append([1, 2, 3], [[4, 5, 6], [7, 8, 9]])
array([1, 2, 3, ..., 7, 8, 9])
```

When axis is specified, values must have the correct shape.

```
>>> np.append([[1, 2, 3], [4, 5, 6]], [[7, 8, 9]], axis=0)
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
>>> np.append([[1, 2, 3], [4, 5, 6]], [7, 8, 9], axis=0)
Traceback (most recent call last):
    ...
ValueError: all the input arrays must have same number of dimensions, but
the array at index 0 has 2 dimension(s) and the array at index 1 has 1
dimension(s)
```

See https://numpy.org/doc/stable/reference/generated/numpy.resize.html#numpy.resize for details.

If the new array is larger than the original array, then the new array is filled with repeated copies of a. Note that this behavior is different from a.resize(new_shape) which fills with zeros instead of repeated copies of a.

When the total size of the array does not change reshape should be used. In most other cases either indexing (to reduce the size) or padding (to increase the size) may be a more appropriate solution.

Warning: This functionality does not consider axes separately, i.e. it does not apply interpolation/extrapolation. It fills the return array with the required number of elements, iterating over a in C-order, disregarding axes (and cycling back from the start if the new shape is larger). This functionality is therefore not suitable to resize images, or data where each axis represents a separate and distinct entity.

```
>>> a=np.array([[0,1],[2,3]])
>>> np.resize(a,(2,3))
array([[0, 1, 2],
       [3, 0, 1]])

>>> np.resize(a,(1,4))
array([[0, 1, 2, 3]])

>>> np.resize(a,(2,4))
array([[0, 1, 2, 3],
       [0, 1, 2, 3]])
```

7.10.5 Trim zeros from a 1-D array: numpy.trim_zeros

npm.**trim_zeros**(*filt*, *trim='fb'*, *axis=None*)

Trim the leading and/or trailing zeros from a 1-D array or sequence.

See https://numpy.org/doc/stable/reference/generated/numpy.trim_zeros.html#numpy.trim_zeros for details.

```
>>> a = np.array((0, 0, 0, 1, 2, 3, 0, 2, 1, 0))
>>> np.trim_zeros(a)
array([1, 2, 3, 0, 2, 1])
>>> np.trim_zeros(a, 'b')
array([0, 0, 0, ..., 0, 2, 1])
```

The input data type is preserved, list/tuple in means list/tuple out.

```
>>> np.trim_zeros([0, 1, 2, 0])
[1, 2]
```

7.10.6 Find the unique elements of an array: numpy.unique

npm.**unique**(*ar*, *return_index=False*, *return_inverse=False*, *return_counts=False*, *axis=None*, *, *equal_nan=True*)

Find the unique elements of an array.

See https://numpy.org/doc/stable/reference/generated/numpy.unique.html#numpy.unique for details.

Returns the sorted unique elements of an array. There are three optional outputs in addition to the unique elements:

- the indices of the input array that give the unique values
- the indices of the unique array that reconstruct the input array
- the number of times each unique value comes up in the input array

When an axis is specified the subarrays indexed by the axis are sorted. This is done by making the specified axis the first dimension of the array (move the axis to the first dimension to keep the order of the other axes) and then flattening the subarrays in C order. The flattened subarrays are then viewed as a structured type with each element given a label, with the effect that we end up with a 1-D array of structured types that can be treated in the same way as any other 1-D array. The result is that the flattened subarrays are sorted in lexicographic order starting with the first element.

```
>>> np.unique([1, 1, 2, 2, 3, 3])
array([1, 2, 3])
>>> a = np.array([[1, 1], [2, 3]])
>>> np.unique(a)
array([1, 2, 3])
```

Return the unique rows of a 2D array

```
>>> a = np.array([[1, 0, 0], [1, 0, 0], [2, 3, 4]])
>>> np.unique(a, axis=0)
array([[1, 0, 0], [2, 3, 4]])
```

Return the indices of the original array that give the unique values:

```
>>> a = np.array(['a', 'b', 'b', 'c', 'a'])
>>> u, indices = np.unique(a, return_index=True)
>>> u
array(['a', 'b', 'c'], dtype='<U1')
>>> indices
array([0, 1, 3])
>>> a[indices]
array(['a', 'b', 'c'], dtype='<U1')
```

Reconstruct the input array from the unique values and inverse:

```
>>> a = np.array([1, 2, 6, 4, 2, 3, 2])
>>> u, indices = np.unique(a, return_inverse=True)
>>> u
array([1, 2, 3, 4, 6])
>>> indices
array([0, 1, 4, 3, 1, 2, 1])
>>> u[indices]
array([1, 2, 6, 4, 2, 3, 2])
```

Reconstruct the input values from the unique values and counts:

```
>>> a = np.array([1, 2, 6, 4, 2, 3, 2])
>>> values, counts = np.unique(a, return_counts=True)
>>> values
array([1, 2, 3, 4, 6])
>>> counts
array([1, 3, 1, 1, 1])
>>> np.repeat(values, counts)
array([1, 2, 2, 2, 3, 4, 6])    # original order not preserved
```

7.11 Numpy array manipulation: Rearranging elements

7.11.1 Reverse the order of elements in an array along the given axis: numpy.flip

npm.**flip**(*m*, *axis=None*)

Reverse the order of elements in an array along the given axis.

See https://numpy.org/doc/stable/reference/generated/numpy.flip.html#numpy.flip for details.

The shape of the array is preserved, but the elements are reordered.

flip(m, 0) is equivalent to flipud(m).

flip(m, 1) is equivalent to fliplr(m).

flip(m, n) corresponds to `m[...,::-1,...]` with `::-1` at position n.

flip(m) corresponds to `m[::-1,::-1,...,::-1]` with `::-1` at all positions.

flip(m, (0, 1)) corresponds to `m[::-1,::-1,...]` with `::-1` at position 0 and position 1.

```
>>> A = np.arange(8).reshape((2,2,2))
>>> A
array([[[0, 1],
        [2, 3]],
       [[4, 5],
        [6, 7]]])

>>> np.flip(A, 0)
array([[[4, 5],
        [6, 7]],
       [[0, 1],
        [2, 3]]])

>>> np.flip(A, 1)
array([[[2, 3],
        [0, 1]],
       [[6, 7],
        [4, 5]]])

>>> np.flip(A)
array([[[7, 6],
        [5, 4]],
       [[3, 2],
        [1, 0]]])

>>> np.flip(A, (0, 2))
array([[[5, 4],
        [7, 6]],
       [[1, 0],
        [3, 2]]])

>>> A = np.random.randn(3,4,5)
>>> np.all(np.flip(A,2) == A[:,:,::-1,...])
True
```

7.11.2 Reverse the order of elements along axis 1 (left/right): numpy.fliplr

npm.**fliplr**(*m*)

Reverse the order of elements along axis 1 (left/right).

See https://numpy.org/doc/stable/reference/generated/numpy.fliplr.html#numpy.fliplr for details.

For a 2-D array, this flips the entries in each row in the left/right direction. Columns are preserved, but appear in a different order than before.

```
>>> A = np.diag([1.,2.,3.])
>>> A
array([[1.,  0.,  0.],
       [0.,  2.,  0.],
       [0.,  0.,  3.]])
>>> np.fliplr(A)
array([[0.,  0.,  1.],
       [0.,  2.,  0.],
       [3.,  0.,  0.]])

>>> A = np.random.randn(2,3,5)
>>> np.all(np.fliplr(A) == A[:,::-1,...])
True
```

7.11.3 Reverse the order of elements along axis 0 (up/down): numpy.flipud

npm.**flipud**(*m*)

Reverse the order of elements along axis 0 (up/down).

See https://numpy.org/doc/stable/reference/generated/numpy.flipud.html#numpy.flipud for details.

For a 2-D array, this flips the entries in each column in the up/down direction. Rows are preserved, but appear in a different order than before.

Equivalent to `m[::-1, ...]` or `np.flip(m, axis=0)`. Requires the array to be at least 1-D.

```
>>> A = np.diag([1.0, 2, 3])
>>> A
array([[1.,  0.,  0.],
       [0.,  2.,  0.],
       [0.,  0.,  3.]])
>>> np.flipud(A)
array([[0.,  0.,  3.],
       [0.,  2.,  0.],
       [1.,  0.,  0.]])

>>> A = np.random.randn(2,3,5)
>>> np.all(np.flipud(A) == A[::-1,...])
True

>>> np.flipud([1,2])
array([2, 1])
```

7.11.4 Roll array elements along a given axis: numpy.roll

npm.**roll**(*a*, *shift*, *axis=None*)

Roll array elements along a given axis.

See https://numpy.org/doc/stable/reference/generated/numpy.roll.html#numpy.roll for details.

Elements that roll beyond the last position are re-introduced at the first.

```
>>> x = np.arange(10)
>>> np.roll(x, 2)
array([8, 9, 0, 1, 2, 3, 4, 5, 6, 7])
>>> np.roll(x, -2)
array([2, 3, 4, 5, 6, 7, 8, 9, 0, 1])

>>> x2 = np.reshape(x, (2, 5))
>>> x2
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])
```

(continues on next page)

(continued from previous page)

```
>>> np.roll(x2, 1)
array([[9, 0, 1, 2, 3],
       [4, 5, 6, 7, 8]])
>>> np.roll(x2, -1)
array([[1, 2, 3, 4, 5],
       [6, 7, 8, 9, 0]])
>>> np.roll(x2, 1, axis=0)
array([[5, 6, 7, 8, 9],
       [0, 1, 2, 3, 4]])
>>> np.roll(x2, -1, axis=0)
array([[5, 6, 7, 8, 9],
       [0, 1, 2, 3, 4]])
>>> np.roll(x2, 1, axis=1)
array([[4, 0, 1, 2, 3],
       [9, 5, 6, 7, 8]])
>>> np.roll(x2, -1, axis=1)
array([[1, 2, 3, 4, 0],
       [6, 7, 8, 9, 5]])
>>> np.roll(x2, (1, 1), axis=(1, 0))
array([[9, 5, 6, 7, 8],
       [4, 0, 1, 2, 3]])
>>> np.roll(x2, (2, 1), axis=(1, 0))
array([[8, 9, 5, 6, 7],
       [3, 4, 0, 1, 2]])
```

7.11.5 Rotate an array by 90 degrees: numpy.rot90

npm.**rot90**(*m*, *k=1*, *axes=(0, 1)*)

Rotate an array by 90 degrees in the plane specified by axes.

See https://numpy.org/doc/stable/reference/generated/numpy.rot90.html#numpy.rot90 for details.

Rotation direction is from the first towards the second axis. This means for a 2D array with the default k and axes, the rotation will be counterclockwise

`rot90(m, k=1, axes=(1,0))` is the reverse of `rot90(m, k=1, axes=(0,1))`

`rot90(m, k=1, axes=(1,0))` is equivalent to `rot90(m, k=-1, axes=(0,1))`

```
>>> m = np.array([[1,2],[3,4]], int)
>>> m
array([[1, 2],
       [3, 4]])
>>> np.rot90(m)
array([[2, 4],
       [1, 3]])
>>> np.rot90(m, 2)
array([[4, 3],
       [2, 1]])
>>> m = np.arange(8).reshape((2,2,2))
>>> np.rot90(m, 1, (1,2))
array([[[1, 3],
        [0, 2]],
       [[5, 7],
        [4, 6]]])
```

7.12 Numpy array manipulation: Sorting

7.12.1 Sorted copy of an array: numpy.sort

npm.**sort**(*a*, *axis=-1*, *kind=None*, *order=None*, ***, *stable=None*)

Return a sorted copy of an array.

See https://numpy.org/doc/stable/reference/generated/numpy.sort.html#numpy.sort for details.

The various sorting algorithms are characterized by their average speed, worst case performance, work space size, and whether they are stable. A stable sort keeps items with the same key in the same relative order. The four algorithms implemented in NumPy have the following properties:

All the sort algorithms make temporary copies of the data when sorting along any but the last axis. Consequently, sorting along the last axis is faster and uses less space than sorting along any other axis.

The sort order for complex numbers is lexicographic. If both the real and imaginary parts are non-nan then the order is determined by the real parts except when they are equal, in which case the order is determined by the imaginary parts.

Previous to numpy 1.4.0 sorting real and complex arrays containing nan values led to undefined behaviour. In numpy versions >= 1.4.0 nan values are sorted to the end. The extended sort order is:

Real: [R, nan]

Complex: [R + Rj, R + nanj, nan + Rj, nan + nanj]

where R is a non-nan real value. Complex values with the same nan placements are sorted according to the non-nan part if it exists. Non-nan values are sorted as before.

quicksort has been changed to introsort. When sorting does not make enough progress it switches to heapsort. This implementation makes quicksort O(n*log(n)) in the worst case.

'stable' automatically chooses the best stable sorting algorithm for the data type being sorted. It, along with 'mergesort' is currently mapped to timsort or radix sort depending on the data type. API forward compatibility currently limits the ability to select the implementation and it is hardwired for the different data types.

Timsort is added for better performance on already or nearly sorted data. On random data timsort is almost identical to mergesort. It is now used for stable sort while quicksort is still the default sort if none is chosen. For timsort details, refer to CPython listsort.txt. 'mergesort' and 'stable' are mapped to radix sort for integer data types. Radix sort is an O(n) sort instead of O(n log n).

```
>>> a = np.array([[1,4],[3,1]])
>>> np.sort(a)                # sort along the last axis
array([[1, 4],
       [1, 3]])
>>> np.sort(a, axis=None)     # sort the flattened array
array([1, 1, 3, 4])
>>> np.sort(a, axis=0)        # sort along the first axis
array([[1, 1],
       [3, 4]])
```

Use the order keyword to specify a field to use when sorting a structured array:

```
>>> dtype = [('name', 'S10'), ('height', float), ('age', int)]
>>> values = [('Arthur', 1.8, 41), ('Lancelot', 1.9, 38),
          ('Galahad', 1.7, 38)]
>>> a = np.array(values, dtype=dtype)       # create a structured array
>>> np.sort(a, order='height')
array([('Galahad', 1.7, 38), ('Arthur', 1.8, 41),
       ('Lancelot', 1.8999999999999999, 38)],
      dtype=[('name', '|S10'), ('height', '<f8'), ('age', '<i4')])
Sort by age, then height if ages are equal:
```

(continues on next page)

(continued from previous page)

```
np.sort(a, order=['age', 'height'])
array([('Galahad', 1.7, 38), ('Lancelot', 1.8999999999999999, 38),
       ('Arthur', 1.8, 41)],
      dtype=[('name', '|S10'), ('height', '<f8'), ('age', '<i4')])
```

ndarray.sort

https://numpy.org/doc/stable/reference/generated/numpy.ndarray.sort.html#numpy.ndarray.sort

Sort an array in-place. Refer to numpy.sort for full documentation.

```
>>> a = np.array([[1,4], [3,1]])
>>> a.sort(axis=1)
>>> a
array([[1, 4],
       [1, 3]])
>>> a.sort(axis=0)
>>> a
array([[1, 3],
       [1, 4]])
```

Use the order keyword to specify a field to use when sorting a structured array:

```
>>> a = np.array([('a', 2), ('c', 1)], dtype=[('x', 'S1'), ('y', int)])
>>> a.sort(order='y')
>>> a
array([(b'c', 1), (b'a', 2)],
      dtype=[('x', 'S1'), ('y', '<i8')])
```

7.12.2 Indirect stable sort using a sequence of keys: numpy.lexsort

npm.**lexsort**(*keys*, *axis=-1*)

Perform an indirect stable sort using a sequence of keys.

See https://numpy.org/doc/stable/reference/generated/numpy.lexsort.html#numpy.lexsort for details.

Given multiple sorting keys, which can be interpreted as columns in a spreadsheet, lexsort returns an array of integer indices that describes the sort order by multiple columns. The last key in the sequence is used for the primary sort order, the second-to-last key for the secondary sort order, and so on. The keys argument must be a sequence of objects that can be converted to arrays of the same shape. If a 2D array is provided for the keys argument, its rows are interpreted as the sorting keys and sorting is according to the last row, second last row etc.

Sort names: first by surname, then by name.

```
>>> surnames =    ('Hertz',    'Galilei', 'Hertz')
>>> first_names = ('Heinrich', 'Galileo', 'Gustav')
>>> ind = np.lexsort((first_names, surnames))
>>> ind
array([1, 2, 0])

>>> [surnames[i] + ", " + first_names[i] for i in ind]
['Galilei, Galileo', 'Hertz, Gustav', 'Hertz, Heinrich']
```

Sort two columns of numbers:

```
>>> a = [1,5,1,4,3,4,4] # First column
>>> b = [9,4,0,4,0,2,1] # Second column
>>> ind = np.lexsort((b,a)) # Sort by a, then by b
>>> ind
array([2, 0, 4, 6, 5, 3, 1])
```

(continues on next page)

(continued from previous page)

```
>>> [(a[i],b[i]) for i in ind]
[(1, 0), (1, 9), (3, 0), (4, 1), (4, 2), (4, 4), (5, 4)]
```

Note that sorting is first according to the elements of a. Secondary sorting is according to the elements of b.

A normal argsort would have yielded:

```
>>> [(a[i],b[i]) for i in np.argsort(a)]
[(1, 9), (1, 0), (3, 0), (4, 4), (4, 2), (4, 1), (5, 4)]
```

Structured arrays are sorted lexically by argsort:

```
>>> x = np.array([(1,9), (5,4), (1,0), (4,4), (3,0), (4,2), (4,1)], dtype=np.dtype([('x', int), ('y', int)]))
>>> np.argsort(x) # or np.argsort(x, order=('x', 'y'))
array([2, 0, 4, 6, 5, 3, 1])
```

7.12.3 Indices that would sort an array: numpy.argsort

npm.**argsort**(*a*, *axis=-1*, *kind=None*, *order=None*, *, *stable=None*)

Returns the indices that would sort an array.

See https://numpy.org/doc/stable/reference/generated/numpy.argsort.html#numpy.argsort for details.

Perform an indirect sort along the given axis using the algorithm specified by the kind keyword. It returns an array of indices of the same shape as a that index data along the given axis in sorted order.

One dimensional array:

```
>>> x = np.array([3, 1, 2])
>>> np.argsort(x)
array([1, 2, 0])
```

Two-dimensional array:

```
>>> x = np.array([[0, 3], [2, 2]])
>>> x
array([[0, 3],
       [2, 2]])

>>> ind = np.argsort(x, axis=0)  # sorts along first axis (down)
>>> ind
array([[0, 1],
       [1, 0]])
>>> np.take_along_axis(x, ind, axis=0)  # same as np.sort(x, axis=0)
array([[0, 2],
       [2, 3]])

>>> ind = np.argsort(x, axis=1)  # sorts along last axis (across)
>>> ind
array([[0, 1],
       [0, 1]])
>>> np.take_along_axis(x, ind, axis=1)  # same as np.sort(x, axis=1)
array([[0, 3],
       [2, 2]])
```

Indices of the sorted elements of a N-dimensional array:

```
>>> ind = np.unravel_index(np.argsort(x, axis=None), x.shape)
>>> ind
(array([0, 1, 1, 0]), array([0, 0, 1, 1]))
x[ind]  # same as np.sort(x, axis=None)
array([0, 2, 2, 3])
```

Sorting with keys:

```
>>> x = np.array([(1, 0), (0, 1)], dtype=[('x', '<i4'), ('y', '<i4')])
>>> x
array([(1, 0), (0, 1)],
      dtype=[('x', '<i4'), ('y', '<i4')])

>>> np.argsort(x, order=('x','y'))
array([1, 0])

>>> np.argsort(x, order=('y','x'))
array([0, 1])
```

7.12.4 Matching 1d index and data slices: numpy.take_along_axis

npm.**take_along_axis**(*arr*, *indices*, *axis=-1*)

Take values from the input array by matching 1d index and data slices.

This iterates over matching 1d slices oriented along the specified axis in the index and data arrays, and uses the former to look up values in the latter. These slices can be different lengths.

See https://numpy.org/doc/stable/reference/generated/numpy.argsort.html#numpy.argsort for details.

One dimensional array:

```
>>> a = np.array([[10, 30, 20], [60, 40, 50]])
>>> np.sort(a, axis=1)
array([[10, 20, 30],
       [40, 50, 60]])

>>> ai = np.argsort(a, axis=1)
>>> ai
array([[0, 2, 1],
       [1, 2, 0]])

>>> np.take_along_axis(a, ai, axis=1)
array([[10, 20, 30],
       [40, 50, 60]])
```

7.12.5 Sorting a complex array: numpy.sort_complex

npm.**sort_complex**(*a*, *axis=-1*, *kind=None*, *order=None*, ***, *stable=None*)

Sort a complex array using the real part first, then the imaginary part.

See https://numpy.org/doc/stable/reference/generated/numpy.sort_complex.html#numpy.sort_complex for details.

```
>>> np.sort_complex([5, 3, 6, 2, 1])
array([1.+0.j, 2.+0.j, 3.+0.j, 5.+0.j, 6.+0.j])

>>> np.sort_complex([1 + 2j, 2 - 1j, 3 - 2j, 3 - 3j, 3 + 5j])
array([1.+2.j,  2.-1.j,  3.-3.j,  3.-2.j,  3.+5.j])
```

7.13 Numpy array manipulation: Searching

7.13.1 Indices of the maximum values along an axis: numpy.argmax

npm.**argmax**(*a*, *axis=None*, *out=None*, ***, *keepdims=<no value>*)

Returns the indices of the maximum values along an axis.

See https://numpy.org/doc/stable/reference/generated/numpy.argmax.html#numpy.argmax for details.

In case of multiple occurrences of the maximum values, the indices corresponding to the first occurrence are returned.

```
>>> a = np.arange(6).reshape(2,3) + 10
>>> a
array([[10, 11, 12],
       [13, 14, 15]])
>>> np.argmax(a)
5
>>> np.argmax(a, axis=0)
array([1, 1, 1])
>>> np.argmax(a, axis=1)
array([2, 2])
```

Indexes of the maximal elements of a N-dimensional array:

```
>>> ind = np.unravel_index(np.argmax(a, axis=None), a.shape)
>>> ind
(1, 2)
>>> a[ind]
15

>>> b = np.arange(6)
>>> b[1] = 5
>>> b
array([0, 5, 2, 3, 4, 5])
>>> np.argmax(b)  # Only the first occurrence is returned.
1

>>> x = np.array([[4,2,3], [1,0,3]])
>>> index_array = np.argmax(x, axis=-1)
>>> # Same as np.amax(x, axis=-1, keepdims=True)
>>> np.take_along_axis(x, np.expand_dims(index_array, axis=-1), axis=-1)
array([[4],
       [3]])
>>> # Same as np.amax(x, axis=-1)
>>> np.take_along_axis(x, np.expand_dims(index_array, axis=-1), axis=-1).squeeze(axis=-1)
array([4, 3])
```

Setting keepdims to True,

```
>>> x = np.arange(24).reshape((2, 3, 4))
>>> res = np.argmax(x, axis=1, keepdims=True)
>>> res.shape
(2, 1, 4)
```

7.13.2 Get indices of the minimum values along an axis: numpy.argmin

npm.**argmin**(*a*, *axis=None*, *out=None*, ***, *keepdims=<no value>*)

Returns the indices of the minimum values along an axis.

See https://numpy.org/doc/stable/reference/generated/numpy.argmin.html#numpy.argmin for details.

```
>>> a = np.arange(6).reshape(2,3) + 10
>>> a
array([[10, 11, 12],
       [13, 14, 15]])
>>> np.argmin(a)
0
>>> np.argmin(a, axis=0)
array([0, 0, 0])
>>> np.argmin(a, axis=1)
array([0, 0])
```

Indices of the minimum elements of a N-dimensional array:

```
>>> ind = np.unravel_index(np.argmin(a, axis=None), a.shape)
>>> ind
(0, 0)
>>> a[ind]
10

>>> b = np.arange(6) + 10
>>> b[4] = 10
b
>>> array([10, 11, 12, 13, 10, 15])
np.argmin(b)  # Only the first occurrence is returned.
0

>>> x = np.array([[4,2,3], [1,0,3]])
>>> index_array = np.argmin(x, axis=-1)
>>> # Same as np.amin(x, axis=-1, keepdims=True)
>>> np.take_along_axis(x, np.expand_dims(index_array, axis=-1), axis=-1)
array([[2],
       [0]])
>>> # Same as np.amax(x, axis=-1)
>>> np.take_along_axis(x, np.expand_dims(index_array, axis=-1), axis=-1).squeeze(axis=-1)
array([2, 0])
```

Setting keepdims to True,

```
>>> x = np.arange(24).reshape((2, 3, 4))
>>> res = np.argmin(x, axis=1, keepdims=True)
>>> res.shape
(2, 1, 4)
```

7.13.3 Find the indices of array elements that are non-zero, grouped by element: numpy.argwhere

npm.**argwhere**(*a*)

Find the indices of array elements that are non-zero, grouped by element.

See https://numpy.org/doc/stable/reference/generated/numpy.argwhere.html#numpy.argwhere for details.

np.argwhere(a) is almost the same as np.transpose(np.nonzero(a)), but produces a result of the correct shape for a 0D array.

The output of argwhere is not suitable for indexing arrays. For this purpose use nonzero(a) instead.

```
>>> x = np.arange(6).reshape(2,3)
>>> x
array([[0, 1, 2],
       [3, 4, 5]])
>>> np.argwhere(x>1)
array([[0, 2],
       [1, 0],
       [1, 1],
       [1, 2]])
```

7.13.4 Get the indices of array elements that are non-zero: numpy.nonzero

npm.**nonzero**(*a*)

Return the indices of the elements that are non-zero.

See https://numpy.org/doc/stable/reference/generated/numpy.nonzero.html#numpy.nonzero for details.

Returns a tuple of arrays, one for each dimension of a, containing the indices of the non-zero elements in that dimension. The values in a are always tested and returned in row-major, C-style order.

To group the indices by element, rather than dimension, use argwhere, which returns a row for each non-zero element.

While the nonzero values can be obtained with a[nonzero(a)], it is recommended to use x[x.astype(bool)] or x[x != 0] instead, which will correctly handle 0-d arrays.

```
>>> x = np.array([[3, 0, 0], [0, 4, 0], [5, 6, 0]])
>>> x
array([[3, 0, 0],
       [0, 4, 0],
       [5, 6, 0]])
>>> np.nonzero(x)
(array([0, 1, 2, 2]), array([0, 1, 0, 1]))

>>> x[np.nonzero(x)]
array([3, 4, 5, 6])
>>> np.transpose(np.nonzero(x))
array([[0, 0],
       [1, 1],
       [2, 0],
       [2, 1]])
```

A common use for nonzero is to find the indices of an array, where a condition is True. Given an array a, the condition a > 3 is a boolean array and since False is interpreted as 0, np.nonzero(a > 3) yields the indices of the a where the condition is true.

```
>>> a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
a > 3
array([[False, False, False],
       [ True,  True,  True],
       [ True,  True,  True]])
>>> np.nonzero(a > 3)
(array([1, 1, 1, 2, 2, 2]), array([0, 1, 2, 0, 1, 2]))
```

Using this result to index a is equivalent to using the mask directly:

```
>>> a[np.nonzero(a > 3)]
array([4, 5, 6, 7, 8, 9])
>>> a[a > 3]  # prefer this spelling
array([4, 5, 6, 7, 8, 9])
```

nonzero can also be called as a method of the array.

```
>>> (a > 3).nonzero()
(array([1, 1, 1, 2, 2, 2]), array([0, 1, 2, 0, 1, 2]))
```

7.13.5 Return elements depending on a condition: numpy.where

npm.**where**(*condition*, [*x*, *y*,]/)

Return elements chosen from x or y depending on condition.

See https://numpy.org/doc/stable/reference/generated/numpy.where.html#numpy.where for details

```
>>> a = np.arange(10)
>>> a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.where(a < 5, a, 10*a)
array([ 0,  1,  2,  3,  4, 50, 60, 70, 80, 90])
```

This can be used on multidimensional arrays too:

```
>>> np.where([[True, False], [True, True]],
         [[1, 2], [3, 4]],
         [[9, 8], [7, 6]])
array([[1, 8],
       [3, 4]])
```

The shapes of x, y, and the condition are broadcast together:

```
>>> x, y = np.ogrid[:3, :4]
>>> np.where(x < y, x, 10 + y)  # both x and 10+y are broadcast
array([[10,  0,  0,  0],
       [10, 11,  1,  1],
       [10, 11, 12,  2]])

>>> a = np.array([[0, 1, 2],
               [0, 2, 4],
               [0, 3, 6]])
>>> np.where(a < 4, a, -1)  # -1 is broadcast
array([[ 0,  1,  2],
       [ 0,  2, -1],
       [ 0,  3, -1]])
```

7.13.6 Return an array drawn from elements in choicelist, depending on a condition: numpy.select

npm.**select**(*condlist*, *choicelist*, *default=0*)

Return an array drawn from elements in choicelist, depending on conditions.

See https://numpy.org/doc/stable/reference/generated/numpy.select.html for details

Beginning with an array of integers from 0 to 5 (inclusive), elements less than 3 are negated, elements greater than 3 are squared, and elements not meeting either of these conditions (exactly 3) are replaced with a default value of 42.

```
>>> a = np.arange(6)
>>> condlist = [x<3, x>3]
>>> choicelist = [-x, x**2]
>>> np.select(condlist, choicelist, 42)
array([ 0,  -1,  -2, 42, 16, 25])
```

```
>>> condlist = [x<=4, x>3]
>>> choicelist = [x, x**2]
>>> np.select(condlist, choicelist, 55)
array([ 0,  1,  2,  3,  4, 25])
```

7.14 Numpy mathematical functions: Sums, products, differences

See also: https://numpy.org/doc/stable/reference/routines.math.html

7.14.1 Product of array elements: numpy.prod

npm.**prod**(*a*, *axis=None*, *dtype=None*, *out=None*, *keepdims=<no value>*, *initial=<no value>*, *where=<no value>*)

Return the product of array elements over a given axis. The product of an empty array is the neutral element 1.

See https://numpy.org/doc/stable/reference/generated/numpy.prod.html#numpy.prod for details.

```
>>> np.prod([])
1.0
```

By default, calculate the product of all elements:

```
>>> np.prod(a, axis=1)
array([  2.,  12.])
>>> np.prod(a, axis=0)
array([3., 8.])
```

Or select specific elements to include:

```
>>> np.prod([1., np.nan, 3.], where=[True, False, True])
3.0
```

If the type of x is unsigned, then the output type is the unsigned platform integer:

```
>>> x = np.array([1, 2, 3], dtype=np.uint8)
np.prod(x).dtype == np.uint
True
```

If x is of a signed integer type, then the output type is the default platform integer:

```
>>> x = np.array([1, 2, 3], dtype=np.int8)
np.prod(x).dtype == int
True
```

You can also start the product with a value other than one:

```
>>> np.prod([1, 2], initial=5)
10
```

7.14.2 Sum of array elements: numpy.sum

npm.**sum**(*a*, *axis=None*, *dtype=None*, *out=None*, *keepdims=<no value>*, *initial=<no value>*, *where=<no value>*)

Sum of array elements over a given axis. The sum of an empty array is the neutral element 0.

See https://numpy.org/doc/stable/reference/generated/numpy.sum.html#numpy.sum for details.

```
>>> np.prod([])
1.0
```

For floating point numbers the numerical precision of sum (and np.add.reduce) is in general limited by directly adding each number individually to the result causing rounding errors in every step. However, often numpy will use a numerically better approach (partial pairwise summation) leading to improved precision in many use-cases. This improved precision is always provided when no axis is given. When axis is given, it will depend on which axis is summed. Technically, to provide the best speed possible, the improved precision is only used when the summation is along the fast axis in memory. Note that the exact precision may vary depending on other parameters. In contrast to NumPy, Python's math.fsum function uses a slower but more

precise approach to summation. Especially when summing a large number of lower precision floating point numbers, such as float32, numerical errors can become significant. In such cases it can be advisable to use dtype="float64" to use a higher precision for the output.

```
>>> np.sum([0.5, 1.5])
2.0
>>> np.sum([0.5, 0.7, 0.2, 1.5], dtype=np.int32)
1
>>> np.sum([[0, 1], [0, 5]])
6
>>> np.sum([[0, 1], [0, 5]], axis=0)
array([0, 6])
>>> np.sum([[0, 1], [0, 5]], axis=1)
array([1, 5])
>>> np.sum([[0, 1], [np.nan, 5]], where=[False, True], axis=1)
array([1., 5.])
```

If the accumulator is too small, overflow occurs:

```
>>> np.ones(128, dtype=np.int8).sum(dtype=np.int8)
-128
```

You can also start the sum with a value other than zero:

```
>>> np.sum([10], initial=5)
15
```

7.14.3 Cumulative product of array elements: numpy.cumprod

npm.**cumprod**(*a*, *axis=None*, *dtype=None*, *out=None*)

Return the cumulative product of elements along a given axis.

See https://numpy.org/doc/stable/reference/generated/numpy.cumprod.html#numpy.cumprod for details.

```
>>> a = np.array([1,2,3])
>>> np.cumprod(a) # intermediate results 1, 1*2
              # total product 1*2*3 = 6
array([1, 2, 6])
>>> a = np.array([[1, 2, 3], [4, 5, 6]])
>>> np.cumprod(a, dtype=float) # specify type of output
array([   1.,    2.,    6.,   24.,  120.,  720.])
```

The cumulative product for each column (i.e., over the rows) of a:

```
>>> np.cumprod(a, axis=0)
array([[ 1,  2,  3],
       [ 4, 10, 18]])
```

The cumulative product for each row (i.e. over the columns) of a:

```
>>> np.cumprod(a,axis=1)
array([[  1,   2,   6],
       [  4,  20, 120]])
```

7.14.4 Cumulative sum of array elements: numpy.cumsum

npm.**cumsum**(*a*, *axis=None*, *dtype=None*, *out=None*)

Return the cumulative sum of the elements along a given axis.

See https://numpy.org/doc/stable/reference/generated/numpy.cumsum.html#numpy.cumsum for details.

Arithmetic is modular when using integer types, and no error is raised on overflow.

cumsum(a)[-1] may not be equal to sum(a) for floating-point values since sum may use a pairwise summation routine, reducing the roundoff-error. See sum for more information.

The cumulative product for each row (i.e. over the columns) of a:

```
>>> a = np.array([[1,2,3], [4,5,6]])
>>> a
array([[1, 2, 3],
       [4, 5, 6]])
>>> np.cumsum(a)
array([ 1,  3,  6, 10, 15, 21])
>>> np.cumsum(a, dtype=float)     # specifies type of output value(s)
array([  1.,   3.,   6.,  10.,  15.,  21.])

>>> np.cumsum(a,axis=0)      # sum over rows for each of the 3 columns
array([[1, 2, 3],
       [5, 7, 9]])
>>> np.cumsum(a,axis=1)      # sum over columns for each of the 2 rows
array([[ 1,  3,  6],
       [ 4,  9, 15]])
```

cumsum(b)[-1] may not be equal to sum(b)

```
>>> b = np.array([1, 2e-9, 3e-9] * 1000000)
>>> b.cumsum()[-1]
1000000.0050045159
>>> b.sum()
1000000.0050000029
```

7.14.5 N-th discrete difference: numpy.diff

npm.**diff**(*a*, *n=1*, *axis=-1*, *prepend=<no value>*, *append=<no value>*)

Calculate the n-th discrete difference along the given axis.

See https://numpy.org/doc/stable/reference/generated/numpy.diff.html#numpy.diff for details.

The first difference is given by out[i] = a[i+1] - a[i] along the given axis, higher differences are calculated by using diff recursively.

```
>>> x = np.array([1, 2, 4, 7, 0])
>>> np.diff(x)
array([ 1,  2,  3, -7])
>>> np.diff(x, n=2)
array([  1,   1, -10])

>>> x = np.array([[1, 3, 6, 10], [0, 5, 6, 8]])
>>> np.diff(x)
array([[2, 3, 4],
       [5, 1, 2]])
>>> np.diff(x, axis=0)
array([[-1,  2,  0, -2]])

>>> x = np.arange('1066-10-13', '1066-10-16', dtype=np.datetime64)
>>> np.diff(x)
array([1, 1], dtype='timedelta64[D]')
```

7.14.6 Differences between consecutive elements: numpy.ediff1d

npm.**ediff1d**(*ary*, *to_end=None*, *to_begin=None*)

Calculate the differences between consecutive elements of an array.

See https://numpy.org/doc/stable/reference/generated/numpy.ediff1d.html#numpy.ediff1d for details.

The differences between consecutive elements of an array.

```
>>> x = np.array([1, 2, 4, 7, 0])
>>> np.ediff1d(x)
array([ 1,  2,  3, -7])

>>> np.ediff1d(x, to_begin=-99, to_end=np.array([88, 99]))
array([-99,   1,   2, ...,  -7,  88,  99])
```

The returned array is always 1D.

```
>>> y = [[1, 2, 4], [1, 6, 24]]
>>> np.ediff1d(y)
array([ 1,  2, -3,  5, 18])
```

7.14.7 Gradient of an N-dimensional array: numpy.gradient

npm.**gradient**(*f*, **varargs*, *axis=None*, *edge_order=1*)

Return the gradient of an N-dimensional array.

See https://numpy.org/doc/stable/reference/generated/numpy.gradient.html#numpy.gradient for details.

The gradient is computed using second order accurate central differences in the interior points and either first or second order accurate one-sides (forward or backwards) differences at the boundaries. The returned gradient hence has the same shape as the input array.

```
>>> f = np.array([1, 2, 4, 7, 11, 16], dtype=float)
>>> np.gradient(f)
array([1. , 1.5, 2.5, 3.5, 4.5, 5. ])
>>> np.gradient(f, 2)
array([0.5 ,  0.75,  1.25,  1.75,  2.25,  2.5 ])
```

Spacing can be also specified with an array that represents the coordinates of the values F along the dimensions. For instance a uniform spacing:

```
>>> x = np.arange(f.size)
>>> np.gradient(f, x)
array([1. ,  1.5,  2.5,  3.5,  4.5,  5. ])
```

Or a non uniform one:

```
>>> x = np.array([0., 1., 1.5, 3.5, 4., 6.], dtype=float)
>>> np.gradient(f, x)
array([1. ,  3. ,  3.5,  6.7,  6.9,  2.5])
```

For two dimensional arrays, the return will be two arrays ordered by axis. In this example the first array stands for the gradient in rows and the second one in columns direction:

```
>>> np.gradient(np.array([[1, 2, 6], [3, 4, 5]], dtype=float))
[array([[ 2.,  2., -1.],
       [ 2.,  2., -1.]]), array([[1. , 2.5, 4. ],
       [1. , 1. , 1. ]])]
```

In this example the spacing is also specified: uniform for axis=0 and non uniform for axis=1

```
>>> dx = 2.
>>> y = [1., 1.5, 3.5]
>>> np.gradient(np.array([[1, 2, 6], [3, 4, 5]], dtype=float), dx, y)
[array([[ 1. ,  1. , -0.5],
       [ 1. ,  1. , -0.5]]), array([[2. , 2. , 2. ],
       [2. , 1.7, 0.5]])]
```

It is possible to specify how boundaries are treated using edge_order

```
>>> x = np.array([0, 1, 2, 3, 4])
>>> f = x**2
>>> np.gradient(f, edge_order=1)
array([1.,  2.,  4.,  6.,  7.])
>>> np.gradient(f, edge_order=2)
array([0., 2., 4., 6., 8.])
```

The axis keyword can be used to specify a subset of axes of which the gradient is calculated

```
>>> np.gradient(np.array([[1, 2, 6], [3, 4, 5]], dtype=float), axis=0)
array([[ 2.,  2., -1.],
       [ 2.,  2., -1.]])
```

7.14.8 Trapezoidal rule: numpy.trapezoid

npm.**trapezoid**(*y*, *x=None*, *dx=1.0*, *axis=-1*)

Integrate along the given axis using the composite trapezoidal rule.

Note was called trapez before NumPy 2.0.

See https://numpy.org/doc/stable/reference/generated/numpy.trapezoid.html#numpy-trapezoid for details.

If x is provided, the integration happens in sequence along its elements - they are not sorted.

Use the trapezoidal rule on evenly spaced points:

```
>>> np.trapz([1, 2, 3])
4.0
```

The spacing between sample points can be selected by either the x or dx arguments:

```
>>> np.trapz([1, 2, 3], x=[4, 6, 8])
8.0
>>> np.trapz([1, 2, 3], dx=2)
8.0
```

Using a decreasing x corresponds to integrating in reverse:

```
>>> np.trapz([1, 2, 3], x=[8, 6, 4])
-8.0
```

More generally x is used to integrate along a parametric curve. We can estimate the integral using:

```
>>> x = np.linspace(0, 1, num=50)
>>> y = x**2
>>> np.trapz(y, x)
0.33340274885464394
```

Or estimate the area of a circle, noting we repeat the sample which closes the curve:

```
>>> theta = np.linspace(0, 2 * np.pi, num=1000, endpoint=True)
>>> np.trapz(np.cos(theta), x=np.sin(theta))
3.141571941375841
```

np.trapz can be applied along a specified axis to do multiple computations in one call:

```
>>> a = np.arange(6).reshape(2, 3)
>>> a
array([[0, 1, 2],
       [3, 4, 5]])
>>> np.trapz(a, axis=0)
array([1.5, 2.5, 3.5])
>>> np.trapz(a, axis=1)
array([2.,  8.])
```

7.15 Numpy mathematical functions: Extrema Finding

7.15.1 Element-wise maximum of array elements: numpy.maximum

npm.**maximum**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Element-wise maximum of array elements.

See https://numpy.org/doc/stable/reference/generated/numpy.maximum.html#numpy.maximum for details.

Compare two arrays and return a new array containing the element-wise maxima. If one of the elements being compared is a NaN, then that element is returned. If both elements are NaNs then the first is returned. The latter distinction is important for complex NaNs, which are defined as at least one of the real or imaginary parts being a NaN. The net effect is that NaNs are propagated.

```
>>> np.maximum([2, 3, 4], [1, 5, 2])
array([2, 5, 4])

>>> np.maximum(np.eye(2), [0.5, 2]) # broadcasting
array([[ 1. ,  2. ],
       [ 0.5,  2. ]])

>>> np.maximum([np.nan, 0, np.nan], [0, np.nan, np.nan])
array([nan, nan, nan])
>>> np.maximum(np.Inf, 1)
inf
```

7.15.2 Element-wise minimum of array elements: numpy.minimum

npm.**minimum**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

See https://numpy.org/doc/stable/reference/generated/numpy.minimum.html#numpy.minimum for details.

Compare two arrays and return a new array containing the element-wise minima. If one of the elements being compared is a NaN, then that element is returned. If both elements are NaNs then the first is returned. The latter distinction is important for complex NaNs, which are defined as at least one of the real or imaginary parts being a NaN. The net effect is that NaNs are propagated.

```
>>> np.minimum([2, 3, 4], [1, 5, 2])
array([1, 3, 2])

>>> np.minimum(np.eye(2), [0.5, 2]) # broadcasting
array([[ 0.5,  0. ],
       [ 0. ,  1. ]])

>>> np.minimum([np.nan, 0, np.nan],[0, np.nan, np.nan])
array([nan, nan, nan])
>>> np.minimum(-np.Inf, 1)
-inf
```

7.15.3 Maximum of an array or maximum along an axis: numpy.max

npm.**max**(*a*, *axis=None*, *out=None*, *keepdims=<no value>*, *initial=<no value>*, *where=<no value>*)

Return the maximum of an array or maximum along an axis.

See https://numpy.org/doc/stable/reference/generated/numpy.max.html#numpy.max for details

NaN values are propagated, that is if at least one item is NaN, the corresponding max value will be NaN as well. To ignore NaN values (MATLAB behavior), please use nanmax.

Don't use max for element-wise comparison of 2 arrays; when a.shape[0] is 2, maximum(a[0], a[1]) is faster than max(a, axis=0).

```
>>> a = np.arange(4).reshape((2,2))
>>> a
array([[0, 1],
       [2, 3]])
>>> np.max(a)           # Maximum of the flattened array
3
>>> np.max(a, axis=0)   # Maxima along the first axis
array([2, 3])
>>> np.max(a, axis=1)   # Maxima along the second axis
array([1, 3])
>>> np.max(a, where=[False, True], initial=-1, axis=0)
array([-1,  3])
>>> b = np.arange(5, dtype=float)
>>> b[2] = np.NaN
>>> np.max(b)
nan
>>> np.max(b, where=~np.isnan(b), initial=-1)
4.0
>>> np.nanmax(b)
4.0
```

You can use an initial value to compute the maximum of an empty slice, or to initialize it to a different value:

```
>>> np.max([[-50], [10]], axis=-1, initial=0)
array([ 0, 10])
```

Notice that the initial value is used as one of the elements for which the maximum is determined, unlike for the default argument Python's max function, which is only used for empty iterables.

```
>>> np.max([5], initial=6)
6
>>> max([5], default=6)
5
```

7.15.4 Minimum of an array or maximum along an axis: numpy.min

npm.**min**(*a*, *axis=None*, *out=None*, *keepdims=<no value>*, *initial=<no value>*, *where=<no value>*)

Return the minimum of an array or minimum along an axis.

See https://numpy.org/doc/stable/reference/generated/numpy.min.html#numpy.min for details.

NaN values are propagated, that is if at least one item is NaN, the corresponding min value will be NaN as well. To ignore NaN values (MATLAB behavior), please use nanmin.

Don't use min for element-wise comparison of 2 arrays; when a.shape[0] is 2, minimum(a[0], a[1]) is faster than min(a, axis=0).

```
>>> a = np.arange(4).reshape((2,2))
>>> a
array([[0, 1],
       [2, 3]])
>>> np.min(a)           # Minimum of the flattened array
0
>>> np.min(a, axis=0)   # Minima along the first axis
array([0, 1])
>>> np.min(a, axis=1)   # Minima along the second axis
array([0, 2])
>>> np.min(a, where=[False, True], initial=10, axis=0)
array([10,  1])

>>> b = np.arange(5, dtype=float)
>>> b[2] = np.NaN
>>> np.min(b)
nan
```

(continues on next page)

(continued from previous page)

```
>>> np.min(b, where=~np.isnan(b), initial=10)
0.0
>>> np.nanmin(b)
0.0

>>> np.min([[-50], [10]], axis=-1, initial=0)
array([-50,   0])
```

Notice that the initial value is used as one of the elements for which the minimum is determined, unlike for the default argument Python's max function, which is only used for empty iterables. Notice that this isn't the same as Python's default argument.

```
>>> np.min([6], initial=5)
5
>>> min([6], default=5)
6
```

7.16 Numpy mathematical functions: Arithmetic operations, elementwise

7.16.1 Numerical positive, element-wise: numpy.positive

npm.**positive**(*x*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Numerical positive, element-wise. Equivalent to x.copy(), but only defined for types that support arithmetic.

See https://numpy.org/doc/stable/reference/generated/numpy.positive.html for details.

```
>>> x1 = np.array(([1., -1.]))
>>> np.positive(x1)
array([ 1., -1.])
```

The unary + operator can be used as a shorthand for np.positive on ndarrays.

```
>>> x1 = np.array(([1., -1.]))
>>> +x1
array([ 1., -1.])
```

7.16.2 Numerical negative, element-wise: numpy.negative

npm.**negative**(*x*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Numerical negative, element-wise.

See https://numpy.org/doc/stable/reference/generated/numpy.negative.html for details.

```
>>> np.negative([1.,-1.])
array([-1.,  1.])
```

The unary - operator can be used as a shorthand for np.negative on ndarrays.

```
x1 = np.array(([1., -1.]))
-x1
array([-1.,  1.])
```

7.16.3 Add arguments element-wise: numpy.add

npm.**add**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Add arguments element-wise. Equivalent to x1 + x2 in terms of array broadcasting.

See https://numpy.org/doc/stable/reference/generated/numpy.add.html#numpy.add for details

```
>>> np.add(1.0, 4.0)
5.0
>>> x1 = np.arange(9.0).reshape((3, 3))
>>> x2 = np.arange(3.0)
>>> np.add(x1, x2)
array([[  0.,   2.,   4.],
       [  3.,   5.,   7.],
       [  6.,   8.,  10.]])
```

The + operator can be used as a shorthand for np.add on ndarrays.

```
>>> x1 = np.arange(9.0).reshape((3, 3))
>>> x2 = np.arange(3.0)
>>> x1 + x2
array([[ 0.,  2.,  4.],
       [ 3.,  5.,  7.],
       [ 6.,  8., 10.]])
```

7.16.4 Subtract arguments element-wise: numpy.subtract

npm.**subtract**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Subtract arguments, element-wise. Equivalent to x1 - x2 in terms of array broadcasting.

See https://numpy.org/doc/stable/reference/generated/numpy.subtract.html#numpy.subtract for details.

```
>>> np.subtract(1.0, 4.0)
-3.0
>>> x1 = np.arange(9.0).reshape((3, 3))
>>> x2 = np.arange(3.0)
>>> np.subtract(x1, x2)
array([[ 0.,  0.,  0.],
       [ 3.,  3.,  3.],
       [ 6.,  6.,  6.]])
```

The - operator can be used as a shorthand for np.subtract on ndarrays.

```
>>> x1 = np.arange(9.0).reshape((3, 3))
>>> x2 = np.arange(3.0)
>>> x1 - x2
array([[0., 0., 0.],
       [3., 3., 3.],
       [6., 6., 6.]])
```

7.16.5 Multiply arguments element-wise: numpy.multiply

npm.**multiply**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Multiply arguments element-wise. Equivalent to x1 * x2 in terms of array broadcasting.

See https://numpy.org/doc/stable/reference/generated/numpy.multiply.html for details.

```
>>> np.multiply(2.0, 4.0)
8.0
>>> x1 = np.arange(9.0).reshape((3, 3))
>>> x2 = np.arange(3.0)
>>> np.multiply(x1, x2)
array([[  0.,   1.,   4.],
       [  0.,   4.,  10.],
       [  0.,   7.,  16.]])
```

The * operator can be used as a shorthand for np.multiply on ndarrays.

```
>>> x1 = np.arange(9.0).reshape((3, 3))
>>> x2 = np.arange(3.0)
>>> x1 * x2
array([[  0.,   1.,   4.],
       [  0.,   4.,  10.],
       [  0.,   7.,  16.]])
```

7.16.6 Divide arguments element-wise: numpy.divide

npm.**divide**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Divide arguments element-wise. Equivalent to x1 / x2 in terms of array-broadcasting.

See https://numpy.org/doc/stable/reference/generated/numpy.divide.html for details.

`true_divide` is an alias.

```
>>> np.divide(2.0, 4.0)
0.5
>>> x1 = np.arange(9.0).reshape((3, 3))
>>> x2 = np.arange(3.0)
>>> np.divide(x1, x2)
array([[nan, 1. , 1. ],
       [inf, 4. , 2.5],
       [inf, 7. , 4. ]])
```

The / operator can be used as a shorthand for np.divide on ndarrays.

```
>>> x1 = np.arange(9.0).reshape((3, 3))
>>> x2 = 2 * np.ones(3)
x>>> 1 / x2
array([[0. , 0.5, 1. ],
       [1.5, 2. , 2.5],
       [3. , 3.5, 4. ]])
```

7.16.7 Floor-divide arguments element-wise: numpy.floor_divide

npm.**floor_divide**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the largest integer smaller or equal to the division of the inputs. It is equivalent to the Python // operator and pairs with the Python % (remainder), function so that a = a % b + b * (a // b) up to roundoff.

See https://numpy.org/doc/stable/reference/generated/numpy.floor_divide.html for details.

```
>>> np.floor_divide(7,3)
2
>>> np.floor_divide([1., 2., 3., 4.], 2.5)
array([ 0.,  0.,  1.,  1.])
```

The // operator can be used as a shorthand for `np.floor_divide` on ndarrays.

```
>>> x1 = np.array([1., 2., 3., 4.])
>>> x1 // 2.5
array([0., 0., 1., 1.])
```

7.16.8 Element-wise remainder of division: numpy.remainder

npm.**remainder**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Returns the element-wise remainder of division.

https://numpy.org/doc/stable/reference/generated/numpy.remainder.html

Computes the remainder complementary to the `floor_divide` function. It is equivalent to the Python modulus operator `x1 % x2` and has the same sign as the divisor x2. Returns 0 when x2 is 0 and both x1 and x2 are (arrays of) integers. The function `mod` is an alias of remainder.

```
>>> np.np.remainder([4, 7], [2, 3])
array([0, 1])
>>> np.remainder(np.arange(7), 5)
array([0, 1, 2, 3, 4, 0, 1])
```

The % operator can be used as a shorthand for `np.remainder` on ndarrays.

```
>>> x1 = np.arange(7)
>>> x1 % 5
array([0, 1, 2, 3, 4, 0, 1])
```

7.16.9 Element-wise square: numpy.square

npm.**square**(*x*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the element-wise square of the input.

See https://numpy.org/doc/stable/reference/generated/numpy.square.html for details.

```
>>> np.square([-1j, 1])
array([-1.-0.j,  1.+0.j])
```

7.16.10 Element-wise reciprocal: numpy.reciprocal

npm.**reciprocal**(*x*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the reciprocal of the argument, element-wise.

See https://numpy.org/doc/stable/reference/generated/numpy.reciprocal.html for details.

```
>>> np.reciprocal(2.)
0.5
>>> np.reciprocal([1, 2., 3.33])
array([ 1.       ,  0.5      ,  0.3003003])
```

7.17 Numpy mathematical functions: Averages and variances

7.17.1 Median of the array elements.: numpy.median

npm.**median**(*a*, *axis=None*, *out=None*, *overwrite_input=False*, *keepdims=False*)

Returns the median of the array elements.

See https://numpy.org/doc/stable/reference/generated/numpy.median.html for details.

Computes the median along the specified axis.

Given a vector `V` of length `N`, the median of `V` is the middle value of a sorted copy of `V`, `V_sorted` - i e., `V_sorted[(N-1)/2]`, when `N` is odd, and the average of the two middle values of `V_sorted` when `N` is even.

```
>>> a = np.array([[10, 7, 4], [3, 2, 1]])
>>> a
array([[10,  7,  4],
       [ 3,  2,  1]])
>>> np.median(a)
3.5

>>> np.median(a, axis=0)
array([6.5, 4.5, 2.5])

>>> np.median(a, axis=1)
array([7.,  2.])

>>> m = np.median(a, axis=0)
out = np.zeros_like(m)
>>> np.median(a, axis=0, out=m)
array([6.5,  4.5,  2.5])
>>> m
array([6.5,  4.5,  2.5])

>>> b = a.copy()
>>> np.median(b, axis=1, overwrite_input=True)
array([7.,  2.])

>>> assert not np.all(a==b)
>>> b = a.copy()
>>> np.median(b, axis=None, overwrite_input=True)
3.5
assert not np.all(a==b)
```

7.17.2 Average of the array elements.: numpy.average

npm.**average**(*a*, *axis=None*, *weights=None*, *returned=False*, ***, *keepdims=<no value>*)

Compute the weighted average along the specified axis.

See https://numpy.org/doc/stable/reference/generated/numpy.average.html for details.

```
>>> data = np.arange(1, 5)
>>> data
array([1, 2, 3, 4])
>>> np.average(data)
2.5
>>> np.average(np.arange(1, 11), weights=np.arange(10, 0, -1))
4.0

>>> data = np.arange(6).reshape((3, 2))
>>> data
array([[0, 1],
       [2, 3],
```

(continues on next page)

(continued from previous page)

```
       [4, 5]])
>>> np.average(data, axis=1, weights=[1./4, 3./4])
array([0.75, 2.75, 4.75])
>>> np.average(data, weights=[1./4, 3./4])
Traceback (most recent call last):
    ...
TypeError: Axis must be specified when shapes of a and weights differ.

>>> a = np.ones(5, dtype=np.float128)
>>> w = np.ones(5, dtype=np.complex64)
>>> avg = np.average(a, weights=w)
>>> print(avg.dtype)
complex256
```

With keepdims=True, the following result has shape (3, 1).

```
>>> np.average(data, axis=1, keepdims=True)
array([[0.5],
       [2.5],
       [4.5]])
```

7.17.3 Arithmetic mean: numpy.mean

npm.**mean**(*a*, *axis=None*, *dtype=None*, *out=None*, *keepdims=<no value>*, ***, *where=<no value>*)

Compute the arithmetic mean along the specified axis.

See https://numpy.org/doc/stable/reference/generated/numpy.mean.html for details.

Returns the average of the array elements. The average is taken over the flattened array by default, otherwise over the specified axis. The arithmetic mean is the sum of the elements along the axis divided by the number of elements.

```
>>> a = np.array([[1, 2], [3, 4]])
>>> np.mean(a)
2.5
>>> np.mean(a, axis=0)
array([2., 3.])
>>> np.mean(a, axis=1)
array([1.5, 3.5])
```

In single precision, mean can be inaccurate:

```
>>> a = np.zeros((2, 512*512), dtype=np.float32)
>>> a[0, :] = 1.0
>>> a[1, :] = 0.1
>>> np.mean(a)
0.54999924
```

Computing the mean in float64 is more accurate:

```
>>> np.mean(a, dtype=np.float64)
0.55000000074505806 # may vary
```

Specifying a where argument:

```
>>> a = np.array([[5, 9, 13], [14, 10, 12], [11, 15, 19]])
>>> np.mean(a)
12.0
>>> np.mean(a, where=[[True], [False], [False]])
9.0
```

7.17.4 Variance: numpy.var

npm.**var**(*a*, *axis=None*, *dtype=None*, *out=None*, *ddof=0*, *keepdims=<no value>*, ***, *where=<no value>*, *mean=<no value>*, *correction=<no value>*)

Compute the variance along the specified axis.

See https://numpy.org/doc/stable/reference/generated/numpy.var.html for details.

Returns the variance of the array elements, a measure of the spread of a distribution. The variance is computed for the flattened array by default, otherwise over the specified axis.

The variance is the average of the squared deviations from the mean, i.e., var = mean(x), where x = abs(a - a.mean())**2.

The mean is typically calculated as x.sum() / N, where N = len(x). If, however, ddof is specified, the divisor N - ddof is used instead. In standard statistical practice, ddof=1 provides an unbiased estimator of the variance of a hypothetical infinite population. ddof=0 provides a maximum likelihood estimate of the variance for normally distributed variables.

Note that for complex numbers, the absolute value is taken before squaring, so that the result is always real and nonnegative.

For floating-point input, the variance is computed using the same precision the input has. Depending on the input data, this can cause the results to be inaccurate, especially for float32 (see example below). Specifying a higher-accuracy accumulator using the dtype keyword can alleviate this issue.

```
>>> a = np.array([[1, 2], [3, 4]])
>>> np.var(a)
1.25
>>> np.var(a, axis=0)
array([1.,  1.])
>>> np.var(a, axis=1)
array([0.25,  0.25])
```

In single precision, var() can be inaccurate:

```
>>> a = np.zeros((2, 512*512), dtype=np.float32)
>>> a[0, :] = 1.0
>>> a[1, :] = 0.1
>>> np.var(a)
0.20250003
```

Computing the variance in float64 is more accurate:

```
>>> np.var(a, dtype=np.float64)
0.20249999932944759 # may vary
>>> ((1-0.55)**2 + (0.1-0.55)**2)/2
0.2025
```

Specifying a where argument:

```
>>> a = np.array([[14, 8, 11, 10], [7, 9, 10, 11], [10, 15, 5, 10]])
>>> np.var(a)
6.833333333333333 # may vary
>>> np.var(a, where=[[True], [True], [False]])
4.0
```

7.18 Numpy mathematical functions: Matrix and vector products

7.18.1 Dot product: numpy.dot

npm.**dot**(*a*, *b*, *out=None*)

Compute the Dot product of two arrays. .

See https://numpy.org/doc/stable/reference/generated/numpy.dot.html#numpy.dot for details.

Dot product of two arrays. Specifically,

If both a and b are 1-D arrays, it is inner product of vectors (without complex conjugation).

If both a and b are 2-D arrays, it is matrix multiplication, but using matmul or a @ b is preferred.

If either a or b is 0-D (scalar), it is equivalent to multiply and using numpy.multiply(a, b) or a * b is preferred.

If a is an N-D array and b is a 1-D array, it is a sum product over the last axis of a and b.

If a is an N-D array and b is an M-D array (where M>=2), it is a sum product over the last axis of a and the second-to-last axis of b:

```
>>> np.dot(3, 4)
12
```

Neither argument is complex-conjugated:

```
np.dot([2j, 3j], [2j, 3j])
(-13+0j)
```

For 2-D arrays it is the matrix product:

```
a = [[1, 0], [0, 1]]
b = [[4, 1], [2, 2]]
np.dot(a, b)
array([[4, 1],
       [2, 2]])

a = np.arange(3*4*5*6).reshape((3,4,5,6))
b = np.arange(3*4*5*6)[::-1].reshape((5,4,6,3))
np.dot(a, b)[2,3,2,1,2,2]
499128
sum(a[2,3,2,:] * b[1,2,:,2])
499128
```

7.18.2 Dot product of two vectors: numpy.vdot

npm.**vdot**(*a*, *b*, */*)

Return the dot product of two vectors.

See https://numpy.org/doc/stable/reference/generated/numpy.vdot.html#numpy.vdot for details.

The vdot(a, b) function handles complex numbers differently than dot(a, b). If the first argument is complex the complex conjugate of the first argument is used for the calculation of the dot product.

Note that vdot handles multidimensional arrays differently than dot: it does not perform a matrix product, but flattens input arguments to 1-D vectors first. Consequently, it should only be used for vectors.

```
>>> a = np.array([1+2j,3+4j])
>>> b = np.array([5+6j,7+8j])
>>> np.vdot(a, b)
(70-8j)
```

(continues on next page)

(continued from previous page)

```
>>> np.vdot(b, a)
(70+8j)
```

Note that higher-dimensional arrays are flattened!

```
>>> a = np.array([[1, 4], [5, 6]])
>>> b = np.array([[4, 1], [2, 2]])
>>> np.vdot(a, b)
30
>>> np.vdot(b, a)
30
1*4 + 4*1 + 5*2 + 6*2
30
```

7.18.3 Inner product of two arrays: numpy.inner

npm.**inner**(*a*, *b*, */*)

Return the inner product of two arrays.

See https://numpy.org/doc/stable/reference/generated/numpy.inner.html#numpy.inner for details.

Ordinary inner product of vectors for 1-D arrays (without complex conjugation), in higher dimensions a sum product over the last axes.

For vectors (1-D arrays) it computes the ordinary inner-product:

```
np.inner(a, b) = sum(a[:]*b[:])
```

More generally, if ndim(a) = r > 0 and ndim(b) = s > 0:

```
np.inner(a, b) = np.tensordot(a, b, axes=(-1,-1))
```

or explicitly:

```
np.inner(a, b)[i0,...,ir-2,j0,...,js-2]
     = sum(a[i0,...,ir-2,:]*b[j0,...,js-2,:])
```

In addition a or b may be scalars, in which case:

```
np.inner(a,b) = a*b
```

Ordinary inner product for vectors:

```
>>> a = np.array([1,2,3])
>>> b = np.array([0,1,0])
>>> np.inner(a, b)
2
```

Some multidimensional examples:

```
>>> a = np.arange(24).reshape((2,3,4))
>>> b = np.arange(4)
>>> c = np.inner(a, b)
>>> c.shape
(2, 3)
>>> c
array([[ 14,  38,  62],
       [ 86, 110, 134]])

>>> a = np.arange(2).reshape((1,1,2))
```

(continues on next page)

(continued from previous page)

```
>>> b = np.arange(6).reshape((3,2))
>>> c = np.inner(a, b)
>>> c.shape
(1, 1, 3)
>>> c
array([[[1, 3, 5]]])
```

An example where b is a scalar:

```
>>> np.inner(np.eye(2), 7)
array([[7., 0.],
       [0., 7.]])
```

7.18.4 Outer product of two vectors: numpy.outer

npm.**outer**(*a*, *b*, *out=None*)

Compute the outer product of two vectors.

See https://numpy.org/doc/stable/reference/generated/numpy.outer.html#numpy.outer for details.

Given two vectors a and b of length M and N, repsectively, the outer product [1] is:

```
[[a_0*b_0  a_0*b_1 ... a_0*b_{N-1} ]
 [a_1*b_0    .
 [ ...          .
 [a_{M-1}*b_0            a_{M-1}*b_{N-1} ]]
```

Make a (very coarse) grid for computing a Mandelbrot set:

```
>>> rl = np.outer(np.ones((5,)), np.linspace(-2, 2, 5))
>>> rl
array([[-2., -1.,  0.,  1.,  2.],
       [-2., -1.,  0.,  1.,  2.],
       [-2., -1.,  0.,  1.,  2.],
       [-2., -1.,  0.,  1.,  2.],
       [-2., -1.,  0.,  1.,  2.]])
>>> im = np.outer(1j*np.linspace(2, -2, 5), np.ones((5,)))
>>> im
array([[0.+2.j, 0.+2.j, 0.+2.j, 0.+2.j, 0.+2.j],
       [0.+1.j, 0.+1.j, 0.+1.j, 0.+1.j, 0.+1.j],
       [0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
       [0.-1.j, 0.-1.j, 0.-1.j, 0.-1.j, 0.-1.j],
       [0.-2.j, 0.-2.j, 0.-2.j, 0.-2.j, 0.-2.j]])
>>> grid = rl + im
>>> grid
array([[-2.+2.j, -1.+2.j,  0.+2.j,  1.+2.j,  2.+2.j],
       [-2.+1.j, -1.+1.j,  0.+1.j,  1.+1.j,  2.+1.j],
       [-2.+0.j, -1.+0.j,  0.+0.j,  1.+0.j,  2.+0.j],
       [-2.-1.j, -1.-1.j,  0.-1.j,  1.-1.j,  2.-1.j],
       [-2.-2.j, -1.-2.j,  0.-2.j,  1.-2.j,  2.-2.j]])
```

An example using a “vector” of letters:

```
>>> x = np.array(['a', 'b', 'c'], dtype=object)
>>> np.outer(x, [1, 2, 3])
array([['a', 'aa', 'aaa'],
       ['b', 'bb', 'bbb'],
       ['c', 'cc', 'ccc']], dtype=object)
```

7.18.5 Matrix product of two arrays: numpy.matmul

npm.**matmul**(*x1*, *x2*, */*, *out=None*, ***, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Matrix product of two arrays.

See https://numpy.org/doc/stable/reference/generated/numpy.matmul.html#numpy.matmul for details.

The behavior depends on the arguments in the following way.

- If both arguments are 2-D they are multiplied like conventional matrices.
- If either argument is N-D, N > 2, it is treated as a stack of matrices residing in the last two indexes and broadcast accordingly.
- If the first argument is 1-D, it is promoted to a matrix by prepending a 1 to its dimensions. After matrix multiplication the prepended 1 is removed.
- If the second argument is 1-D, it is promoted to a matrix by appending a 1 to its dimensions. After matrix multiplication the appended 1 is removed.

`matmul` differs from `dot` in two important ways:

- Multiplication by scalars is not allowed, use `*` instead.
- Stacks of matrices are broadcast together as if the matrices were elements, respecting the signature `(n,k),(k,m)->(n,m)`:

```
>>> a = np.ones([9, 5, 7, 4])
>>> c = np.ones([9, 5, 4, 3])
>>> np.dot(a, c).shape
(9, 5, 7, 9, 5, 3)
>>> np.matmul(a, c).shape
(9, 5, 7, 3)
>>> # n is 7, k is 4, m is 3
```

The matmul function implements the semantics of the `@` operator introduced in Python 3.5 following PEP 465.

For 2-D arrays it is the matrix product:

```
>>> a = np.array([[1, 0], [0, 1]])
>>> b = np.array([[4, 1], [2, 2]])
>>> np.matmul(a, b)
array([[4, 1],
       [2, 2]])
```

For 2-D mixed with 1-D, the result is the usual.

```
>>> a = np.array([[1, 0], [0, 1]])
>>> b = np.array([1, 2])
>>> np.matmul(a, b)
array([1, 2])
>>> np.matmul(b, a)
array([1, 2])
```

Broadcasting is conventional for stacks of arrays

```
>>> a = np.arange(2 * 2 * 4).reshape((2, 2, 4))
>>> b = np.arange(2 * 2 * 4).reshape((2, 4, 2))
>>> np.matmul(a,b).shape
(2, 2, 2)
>>> np.matmul(a, b)[0, 1, 1]
98
>>> sum(a[0, 1, :] * b[0 , :, 1])
98
```

Vector, vector returns the scalar inner product, but neither argument is complex-conjugated:

```
>>> np.matmul([2j, 3j], [2j, 3j])
(-13+0j)
```

Scalar multiplication raises an error.

```
>>> np.matmul([1,2], 3)
Traceback (most recent call last):
...
ValueError: matmul: Input operand 1 does not have enough dimensions ...
```

The `@` operator can be used as a shorthand for `np.matmul` on ndarrays.

```
>>> x1 = np.array([2j, 3j])
x2 = np.array([2j, 3j])
>>> x1 @ x2
(-13+0j)
```

7.18.6 Tensor dot product: numpy.matmul

npm.**tensordot**(*a*, *b*, *axes=2*)

Compute tensor dot product along specified axes.

See https://numpy.org/doc/stable/reference/generated/numpy.tensordot.html#numpy.tensordot for details.

Given two tensors, a and b, and an array_like object containing two array_like objects, (a_axes, b_axes), sum the products of a's and b's elements (components) over the axes specified by a_axes and b_axes. The third argument can be a single non-negative integer_like scalar, N; if it is such, then the last N dimensions of a and the first N dimensions of b are summed over.

Three common use cases are:

- `axes = 0` : tensor product $a \otimes b$
- `axes = 1` : tensor dot product $a \cdot b$
- `axes = 2` : (default) tensor double contraction $a : b$

When axes is integer_like, the sequence for evaluation will be: first the -Nth axis in a and 0th axis in b, and the -1th axis in a and Nth axis in b last.

When there is more than one axis to sum over - and they are not the last (first) axes of a (b) - the argument axes should consist of two sequences of the same length, with the first axis to sum over given first in both sequences, the second axis second, and so forth.

The shape of the result consists of the non-contracted axes of the first tensor, followed by the non-contracted axes of the second.

A "traditional" example:

```
>>> a = np.arange(60.).reshape(3,4,5)
>>> b = np.arange(24.).reshape(4,3,2)
>>> c = np.tensordot(a,b, axes=([1,0],[0,1]))
>>> c.shape
(5, 2)
>>> c
array([[4400., 4730.],
       [4532., 4874.],
       [4664., 5018.],
       [4796., 5162.],
       [4928., 5306.]])
>>> # A slower but equivalent way of computing the same...
>>> d = np.zeros((5,2))
>>> for i in range(5):
```

(continues on next page)

(continued from previous page)

```
  for j in range(2):
    for k in range(3):
      for n in range(4):
        d[i,j] += a[k,n,i] * b[n,k,j]
>>> c == d
array([[ True,  True],
       [ True,  True],
       [ True,  True],
       [ True,  True],
       [ True,  True]])
```

An extended example taking advantage of the overloading of + and * :

```
>>> a = np.array(range(1, 9))
>>> a.shape = (2, 2, 2)
>>> A = np.array(('a', 'b', 'c', 'd'), dtype=object)
A.shape = (2, 2)
>>> a; A
array([[[1, 2],
        [3, 4]],
       [[5, 6],
        [7, 8]]])
array([['a', 'b'],
       ['c', 'd']], dtype=object)

>>> np.tensordot(a, A) # third argument default is 2 for double-contraction
array(['abbcccdddd', 'aaaaabbbbbbccccccdddddddd'], dtype=object)

>>> np.tensordot(a, A, 1)
array([[['acc', 'bdd'],
        ['aaacccc', 'bbbdddd']],
       [['aaaaacccccc', 'bbbbbddddddd'],
        ['aaaaaaaccccccc', 'bbbbbbbddddddd']]], dtype=object)

>>> np.tensordot(a, A, 0) # tensor product (result too long to incl.)
array([[[[['a', 'b'],
          ['c', 'd']],
          ...
>>> np.tensordot(a, A, (0, 1))
array([[['abbbbb', 'cddddd'],
        ['aabbbbbb', 'ccdddddd']],
       [['aaabbbbbbb', 'cccddddddd'],
        ['aaaabbbbbbbb', 'ccccddddddd']]], dtype=object)

>>> np.tensordot(a, A, (2, 1))
array([[['abb', 'cdd'],
        ['aaabbbb', 'cccdddd']],
       [['aaaaabbbbbb', 'ccccddddddd'],
        ['aaaaaaabbbbbbbb', 'cccccccddddddd']]], dtype=object)

>>> np.tensordot(a, A, ((0, 1), (0, 1)))
array(['abbbcccccddddddd', 'aabbbbcccccccddddddddd'], dtype=object)

>>> np.tensordot(a, A, ((2, 1), (1, 0)))
array(['acccbbdddd', 'aaaaacccccccbbbbbbddddddddd'], dtype=object)
```

7.18.7 Einstein summation: numpy.einsum

npm.**einsum**(*subscripts*, **operands*, *out=None*, *dtype=None*, *order='K'*, *casting='safe'*, *optimize=False*)

Evaluates the Einstein summation convention on the operands.

See https://numpy.org/doc/stable/reference/generated/numpy.einsum.html#numpy.einsum for details.

Using the Einstein summation convention, many common multi-dimensional, linear algebraic array operations can be repre-

sented in a simple fashion. In implicit mode einsum computes these values.

In explicit mode, einsum provides further flexibility to compute other array operations that might not be considered classical Einstein summation operations, by disabling, or forcing summation over specified subscript labels.

Typically a 'greedy' algorithm is applied which empirical tests have shown returns the optimal path in the majority of cases. In some cases 'optimal' will return the superlative path through a more expensive, exhaustive search. For iterative calculations it may be advisable to calculate the optimal path once and reuse that path by supplying it as an argument. An example is given below.

See numpy.einsum_path for more details.

```
>>> a = np.arange(25).reshape(5,5)
>>> b = np.arange(5)
>>> c = np.arange(6).reshape(2,3)
```

Trace of a matrix:

```
>>> np.einsum('ii', a)
60
>>> np.einsum(a, [0,0])
60
>>> np.trace(a)
60
```

Extract the diagonal (requires explicit form):

```
>>> np.einsum('ii->i', a)
array([ 0,  6, 12, 18, 24])
>>> np.einsum(a, [0,0], [0])
array([ 0,  6, 12, 18, 24])
>>> np.diag(a)
array([ 0,  6, 12, 18, 24])
```

Sum over an axis (requires explicit form):

```
>>> np.einsum('ij->i', a)
array([ 10,  35,  60,  85, 110])
>>> np.einsum(a, [0,1], [0])
array([ 10,  35,  60,  85, 110])
>>> np.sum(a, axis=1)
array([ 10,  35,  60,  85, 110])
```

For higher dimensional arrays summing a single axis can be done with ellipsis:

```
>>> np.einsum('...j->...', a)
array([ 10,  35,  60,  85, 110])
>>> np.einsum(a, [Ellipsis,1], [Ellipsis])
array([ 10,  35,  60,  85, 110])
```

7.18.8 Kronecker product: numpy.kron

npm.**kron**(*a*, *b*)

Kronecker product of two arrays.

numpy.kron

https://numpy.org/doc/stable/reference/generated/numpy.kron.html#numpy.kron

Computes the Kronecker product, a composite array made of blocks of the second array scaled by the first.

The function assumes that the number of dimensions of a and b are the same, if necessary prepending the smallest with ones. If a.shape = (r0,r1,..,rN) and b.shape = (s0,s1,...,sN), the Kronecker product has shape (r0*s0, r1*s1, ..., rN*SN). The elements are products of elements from a and b, organized explicitly by:

```
kron(a,b)[k0,k1,...,kN] = a[i0,i1,...,iN] * b[j0,j1,...,jN]
```

where:

```
kt = it * st + jt,  t = 0,...,N
```

In the common 2-D case (N=1), the block structure can be visualized:

```
[[ a[0,0]*b,   a[0,1]*b,  ... , a[0,-1]*b  ],
 [  ...                          ...   ],
 [ a[-1,0]*b,  a[-1,1]*b, ... , a[-1,-1]*b ]]
```

Further examples:

```
>>> np.kron([1,10,100], [5,6,7])
array([  5,   6,   7, ..., 500, 600, 700])
>>> np.kron([5,6,7], [1,10,100])
array([  5,  50, 500, ...,   7,  70, 700])

>>> np.kron(np.eye(2), np.ones((2,2)))
array([[1.,  1.,  0.,  0.],
       [1.,  1.,  0.,  0.],
       [0.,  0.,  1.,  1.],
       [0.,  0.,  1.,  1.]])

>>> a = np.arange(100).reshape((2,5,2,5))
>>> b = np.arange(24).reshape((2,3,4))
>>> c = np.kron(a,b)
>>> c.shape
(2, 10, 6, 20)
>>> I = (1,3,0,2)
>>> J = (0,2,1)
>>> J1 = (0,) + J              # extend to ndim=4
>>> S1 = (1,) + b.shape
>>> K = tuple(np.array(I) * np.array(S1) + np.array(J1))
c[K] == a[I]*b[J]
True
```

7.18.9 Discrete, linear convolution: numpy.convolve

npm.**convolve**(*a*, *v*, *mode='full'*)

Returns the discrete, linear convolution of two one-dimensional sequences.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.convolve.html

See also: Wikipedia, "Convolution", https://en.wikipedia.org/wiki/Convolution

Returns the discrete, linear convolution of two one-dimensional sequences.

The convolution operator is often seen in signal processing, where it models the effect of a linear time-invariant system on a signal [1]. In probability theory, the sum of two independent random variables is distributed according to the convolution of their individual distributions.

If v is longer than a, the arrays are swapped before computation.

The discrete convolution operation is defined as

$$(a * v)_n = \sum_{m=-\infty}^{\infty} a_m v_{n-m}$$

It can be shown that a convolution $x(t) * y(t)$ in time/space is equivalent to the multiplication $X(f)Y(f)$ in the Fourier domain, after appropriate padding (padding is necessary to prevent circular convolution). Since multiplication is more efficient (faster) than convolution, the function `scipy.signal.fftconvolve` exploits the FFT to calculate the convolution of large data-sets.

Note how the convolution operator flips the second array before "sliding" the two across one another:

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.convolve([1, 2, 3], [0, 1, 0.5])
array([0. , 1. , 2.5, 4. , 1.5])
```

Only return the middle values of the convolution. Contains boundary effects, where zeros are taken into account:

```
>>> np.convolve([1,2,3],[0,1,0.5], 'same')
array([1. ,  2.5,  4. ])
```

The two arrays are of the same length, so there is only one position where they completely overlap:

```
>>> np.convolve([1,2,3],[0,1,0.5], 'valid')
array([2.5])
```

7.19 Numpy logical functions: Truth value testing

7.19.1 Test whether all array elements evaluate to True.: numpy.all, ctx.all

npm.**all**(*a*, *axis=None*, *out=None*, *keepdims=<no value>*, ***, *where=<no value>*)

Test whether all array elements along a given axis evaluate to True.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.all.html#numpy.all

Not a Number (NaN), positive infinity and negative infinity evaluate to True because these are not equal to zero

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.all([[True,False],[True,True]])
False

>>> np.all([[True,False],[True,True]], axis=0)
array([ True, False])

np.all([-1, 4, 5])
True

>>> np.all([1.0, np.nan])
True

>>> np.all([[True, True], [False, True]], where=[[True], [False]])
True

>>> o=np.array(False)
>>> z=np.all([-1, 4, 5], out=o)
>>> id(z), id(o), z
(28293632, 28293632, array(True)) # may vary
```

7.19.2 Test whether any array element evaluates to True: numpy.any, ctx.any

npm.**any**(*a*, *axis=None*, *out=None*, *keepdims=<no value>*, ***, *where=<no value>*)

Test whether any array element along a given axis evaluates to True.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.any.html#numpy.any

Returns single boolean if axis is None. Not a Number (NaN), positive infinity and negative infinity evaluate to True because these are not equal to zero.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.any([[True, False], [True, True]])
True

>>> np.any([[True, False], [False, False]], axis=0)
array([ True, False])

>>> np.any([-1, 0, 5])
True

>>> np.any(np.nan)
True

>>> np.any([[True, False], [False, False]], where=[[False], [True]])
False

>>> o=np.array(False)
```

(continues on next page)

(continued from previous page)

```
>>> z=np.any([-1, 4, 5], out=o)
>>> z, o
(array(True), array(True))
>>> # Check now that z is a reference to o
>>> z is o
True
>>> id(z), id(o) # identity of z and o
(191614240, 191614240)
```

7.19.3 Logical AND: numpy.logical_and, ctx.logical_and

npm.**logical_and**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Compute the truth value of x1 AND x2 element-wise.

numpy.logical_and, **ctx.logical_and**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.logical_and.html#numpy.logical_and

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.logical_and(True, False)
False
>>> np.logical_and([True, False], [False, False])
array([False, False])

>>> x = np.arange(5)
>>> np.logical_and(x>1, x<4)
array([False, False,  True,  True, False])
```

The & operator can be used as a shorthand for np.logical_and on boolean ndarrays.

```
>>> a = np.array([True, False])
>>> b = np.array([False, False])
>>> a & b
array([False, False])
```

7.19.4 Logical OR: numpy.logical_or, ctx.logical_or

npm.**logical_or**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Compute the truth value of x1 OR x2 element-wise.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.logical_or.html#numpy.logical_or

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.logical_or(True, False)
True
>>> np.logical_or([True, False], [False, False])
array([ True, False])

>>> x = np.arange(5)
>>> np.logical_or(x < 1, x > 3)
array([ True, False, False, False,  True])
```

The | operator can be used as a shorthand for np.logical_or on boolean ndarrays.

```
>>> a = np.array([True, False])
>>> b = np.array([False, False])
```

(continues on next page)

(continued from previous page)

```
>>> a | b
array([ True, False])
```

7.19.5 Logical XOR: numpy.logical_xor, ctx.logical_xor

npm.**logical_xor**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Compute the truth value of x1 XOR x2, element-wise.

numpy.logical_xor, **ctx.logical_xor**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.logical_xor.html#numpy.logical_xor

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.logical_xor(True, False)
True
>>> np.logical_xor([True, True, False, False], [True, False, True, False])
array([False,  True,  True, False])

>>> x = np.arange(5)
>>> np.logical_xor(x < 1, x > 3)
array([ True, False, False, False,  True])
```

Simple example showing support of broadcasting

```
>>> np.logical_xor(0, np.eye(2))
array([[ True, False],
       :cite:t:`False,  True]])
```

7.19.6 Testing array equality: numpy.array_equal, ctx.array_equal

npm.**array_equal**(*a1*, *a2*, *equal_nan=False*)

True if two arrays have the same shape and elements, False otherwise.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.array_equal.html#numpy.array_equal

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.array_equal([1, 2], [1, 2])
True
>>> np.array_equal(np.array([1, 2]), np.array([1, 2]))
True
>>> np.array_equal([1, 2], [1, 2, 3])
False
>>> np.array_equal([1, 2], [1, 4])
False
>>> a = np.array([1, np.nan])
>>> np.array_equal(a, a)
False
>>> np.array_equal(a, a, equal_nan=True)
True
```

When equal_nan is True, complex values with nan components are considered equal if either the real or the imaginary components are nan.

```
>>> a = np.array([1 + 1j])
>>> b = a.copy()
>>> a.real = np.nan
```

(continues on next page)

(continued from previous page)

```
>>> b.imag = np.nan
>>> np.array_equal(a, b, equal_nan=True)
True
```

7.19.7 Testing array array_equivalence: numpy.array_equiv, ctx.array_equiv

npm.**array_equiv**(*a1*, *a2*)

True if two arrays have the same shape and elements, False otherwise.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.array_equiv.html#numpy.array_equiv

Returns True if input arrays are shape consistent and all elements equal. Shape consistent means they are either the same shape, or one input array can be broadcasted to create the same shape as the other one.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.array_equiv([1, 2], [1, 2])
True
>>> np.array_equiv([1, 2], [1, 3])
False
```

Showing the shape equivalence:

```
>>> np.array_equiv([1, 2], [[1, 2], [1, 2]])
True
>>> np.array_equiv([1, 2], [[1, 2, 1, 2], [1, 2, 1, 2]])
False
>>> np.array_equiv([1, 2], [[1, 2], [1, 3]])
False
```

7.19.8 Truth value of (x1 > x2) element-wise: numpy.greater, ctx.greater

npm.**greater**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the truth value of (x1 > x2) element-wise.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.greater_equal.html#numpy.greater

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.greater_equal([4, 2, 1], [2, 2, 2])
array([ True, True, False])
```

The > operator can be used as a shorthand for `np.greater_equal` on ndarrays.

```
>>> a = np.array([4, 2, 1])
>>> b = np.array([2, 2, 2])
>>> a > b
array([ True,  True, False])
```

7.19.9 Truth value of (x1 >= x2) element-wise: numpy.greater_equal, ctx.greater_equal

npm.**greater_equal**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the truth value of (x1 >= x2) element-wise.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.greater_equal.html#numpy.greater_equal

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.greater_equal([4, 2, 1], [2, 2, 2])
array([ True, True, False])
```

The >= operator can be used as a shorthand for `np.greater_equal` on ndarrays.

```
>>> a = np.array([4, 2, 1])
>>> b = np.array([2, 2, 2])
>>> a >= b
array([ True,  True, False])
```

7.19.10 Truth value of (x1 < x2) element-wise: numpy.less, ctx.less

npm.**less**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the truth value of (x1 < x2) element-wise.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.less.html#numpy.less

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.less([1, 2], [2, 2])
array([ True, False])
```

The < operator can be used as a shorthand for `np.less` on ndarrays.

```
>>> a = np.array([1, 2])
>>> b = np.array([2, 2])
>>> a < b
array([ True, False])
```

7.19.11 Truth value of (x1 <= x2) element-wise: numpy.less_equal, ctx.less_equal

npm.**less_equal**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the truth value of (x1 <= x2) element-wise.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.less_equal.html#numpy.less_equal

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.less_equal([4, 2, 1], [2, 2, 2])
array([False,  True,  True])
```

The <= operator can be used as a shorthand for `np.less_equal` on ndarrays.

```
>>> a = np.array([4, 2, 1])
>>> b = np.array([2, 2, 2])
>>> a <= b
array([False,  True,  True])
```

7.19.12 Truth value of (x1 == x2) element-wise: numpy.equal, ctx.equal

npm.**equal**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the truth value of (x1 == x2) element-wise.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.equal.html#numpy.equal

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.equal([0, 1, 3], np.arange(3))
array([ True,  True, False])
```

What is compared are values, not types. So an `int(1)` and an array of length one can evaluate as True:

```
>>> np.equal(1, np.ones(1))
array([ True])
```

The == operator can be used as a shorthand for `np.equal` on ndarrays.

```
>>> a = np.array([2, 4, 6])
>>> b = np.array([2, 4, 2])
>>> a == b
array([ True,  True, False])
```

7.19.13 Truth value of (x1 != x2) element-wise: numpy.equal, ctx.equal

npm.**not_equal**(*x1*, *x2*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the truth value of (x1 != x2) element-wise.

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.not_equal.html#numpy.not_equal

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.not_equal([1.,2.], [1., 3.])
array([False,  True])
>>> np.not_equal([1, 2], [[1, 3],[1, 4]])
array([[False,  True],
       :cite:t:`False,  True]])
```

The != operator can be used as a shorthand for `np.not_equal` on ndarrays.

```
>>> a = np.array([1., 2.])
>>> b = np.array([1., 3.])
>>> a != b
array([False,  True])
```

7.20 Numpy mathematical functions: Integer and fractional

7.20.1 Floor of the input, element-wise.: numpy.floor, ctx.floor

npm.**floor**(*x*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the floor of the input, element-wise.

https://numpy.org/doc/stable/reference/generated/numpy.floor.html

Some spreadsheet programs calculate the "floor-towards-zero", where floor(-2.5) == -2. NumPy instead uses the definition of floor where floor(-2.5) == -3. The "floor-towards-zero" function is called fix in NumPy.

```
>>> a = np.array([-1.7, -1.5, -0.2, 0.2, 1.5, 1.7, 2.0])
>>> np.floor(a)
array([-2., -2., -1.,  0.,  1.,  1.,  2.])
```

From mpmath. See also Mpmath [672].

!!! 'MPIntervalContext' object has no attribute 'floor' !!!

Computes the floor of x, $\lfloor x \rfloor$, defined as the largest integer less than or equal to x:

Example:

```
>>> from xlcalcnet import mp, iv, fp, dp, gp, ap
>>> mp.dps = 15; mp.pretty = False
>>> x = 3.5
>>> fp.floor(x), mp.floor(x), dp.floor(x), gp.floor(x), ap.floor(x)
(3, mpf('3.0'), Decimal('3'), mpfr('3.0'), arb3_t('3.00000000000000'))
```

Note

floor(), *ceil()* and `nint()` return a floating-point number, not a Python `int`. If $\lfloor x \rfloor$ is too large to be represented exactly at the present working precision, the result will be rounded, not necessarily in the direction implied by the mathematical definition of the function.

To avoid rounding, use *prec=0*:

```
>>> mp.dps = 15
>>> print(int(floor(10**30+1)))
1000000000000000019884624838656
>>> print(int(floor(10**30+1, prec=0)))
1000000000000000000000000000001
```

The floor function is defined for complex numbers and acts on the real and imaginary parts separately:

```
>>> x = 3.25+4.75j
>>> fp.floor(x), mp.floor(x), dp.floor(x), gp.floor(x), ap.floor(x)
((3+4j),
 mpc(real='3.0', imag='4.0'),
 DecCplx('3 + 4j'),
 mpc('3.0+4.0j'),
 acb3_t('3.00000000000000 + 4.00000000000000j'))
```

7.20.2 Ceiling of the input, element-wise.: numpy.ceil, ctx.ceil

npm.**ceil**(*x*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the ceiling of the input, element-wise.

https://numpy.org/doc/stable/reference/generated/numpy.ceil.html

```
>>> a = np.array([-1.7, -1.5, -0.2, 0.2, 1.5, 1.7, 2.0])
>>> np.ceil(a)
array([-1., -1., -0.,  1.,  2.,  2.,  2.])
```

From mpmath. See also Mpmath [666].

Computes the ceiling of x, $\lceil x \rceil$, defined as the smallest integer greater than or equal to x:

Example:

```
>>> from xlcalcnet import mp, iv, fp, dp, gp, ap
>>> mp.dps = 15; mp.pretty = False
>>> x = 3.5
>>> fp.ceil(x), mp.ceil(x), dp.ceil(x), gp.ceil(x), ap.ceil(x)
(4, mpf('4.0'), Decimal('4'), mpfr('4.0'), arb3_t('4.00000000000000'))
```

The ceiling function is defined for complex numbers and acts on the real and imaginary parts separately:

```
>>> x = 3.25+4.75j
>>> fp.ceil(x), mp.ceil(x), dp.ceil(x), gp.ceil(x), ap.ceil(x)
((4+5j),
 mpc(real='4.0', imag='5.0'),
 DecCplx('4 + 5j'),
 mpc('4.0+5.0j'),
 acb3_t('4.00000000000000 + 5.00000000000000j'))
```

7.20.3 Truncated value of the input, element-wise.: numpy.trunc, ctx.trunc

npm.**trunc**(*x*, */*, *out=None*, ***, *where=True*, *casting='same_kind'*, *order='K'*, *dtype=None*, *subok=True*)

Return the truncated value of the input, element-wise.

https://numpy.org/doc/stable/reference/generated/numpy.trunc.html

The truncated value of the scalar x is the nearest integer i which is closer to zero than x is. In short, the fractional part of the signed number x is discarded.

not working with mpm and ipm: trunc

```
>>> a = np.array([-1.7, -1.5, -0.2, 0.2, 1.5, 1.7, 2.0])
>>> np.trunc(a)
array([-1., -1., -0.,  0.,  1.,  1.,  2.])
```

7.20.4 Fix value of the input, element-wise.: numpy.fix, ctx.fix

npm.**fix**(*x*, *out=None*)

Round to nearest integer towards zero.

https://numpy.org/doc/stable/reference/generated/numpy.fix.html

Round an array of floats element-wise to nearest integer towards zero.

```
>>> np.fix(3.14)
3.0
>>> np.fix(3)
3.0
>>> np.fix([2.1, 2.9, -2.1, -2.9])
array([ 2.,  2., -2., -2.])
```

7.21 Numpy mathematical functions: Miscellaneous

7.21.1 Random Generator

Some text

https://numpy.org/doc/stable/reference/random/generator.html#accessing-the-bitgenerator-and-spawning

ctx.rand

From mpmath. See also Mpmath [689].

!!! AttributeError: 'MPIntervalContext' object has no attribute 'rand' !!

Returns an `mpf` with value chosen randomly from $[0, 1)$. The number of randomly generated bits in the mantissa is equal to the working precision.

Examples:

```
>>> from xlcalcnet import fp, mp, iv, dp, gp, ap
>>> mp.dps = iv.dps = dp.dps = gp.dps = ap.dps = 45;
>>> for ctx in [fp, mp, dp, gp, ap]: print(repr(ctx.rand()))
0.6379004385592022
mpf('0.22530711244873659663745003088710468813066280856')
Decimal('0.2572014323118752')
mpfr('0.47800233849533735330794570472789928317070007324 2',153)
arb3_t('[0.900607107324606026566016225842759013175964355 +/- 4.69e-46]')
```

7.21.2 Vectorization of a function: numpy.vectorize

npm.**vectorize**(*pyfunc=np._NoValue*, *otypes=None*, *doc=None*, *excluded=None*, *cache=False*, *signature=None*)

Returns an object that acts like pyfunc, but takes arrays as input.

https://numpy.org/doc/stable/reference/generated/numpy.vectorize.html#numpy-vectorize

Define a vectorized function which takes a nested sequence of objects or numpy arrays as inputs and returns a single numpy array or a tuple of numpy arrays. The vectorized function evaluates pyfunc over successive tuples of the input arrays like the python map function, except it uses the broadcasting rules of numpy.

The data type of the output of vectorized is determined by calling the function with the first element of the input. This can be avoided by specifying the otypes argument.

```
>>> import numpy as np
>>> def myfunc(a, b):
... "Return a-b if a>b, otherwise return a+b"
... if a > b:
...     return a - b
... else:
...     return a + b

>>> vfunc = np.vectorize(myfunc)
>>> vfunc([1, 2, 3, 4], 2)
array([3, 4, 1, 2])
```

The docstring is taken from the input function to vectorize unless it is specified:

```
>>> vfunc.__doc__
'Return a-b if a>b, otherwise return a+b'
>>> vfunc = np.vectorize(myfunc, doc='Vectorized `myfunc`')
>>> vfunc.__doc__
'Vectorized `myfunc`'
```

The output type is determined by evaluating the first element of the input, unless it is specified:

```
>>> out = vfunc([1, 2, 3, 4], 2)
>>> type(out[0])
<class 'numpy.int64'>
>>> vfunc = np.vectorize(myfunc, otypes=[float])
>>> out = vfunc([1, 2, 3, 4], 2)
>>> type(out[0])
<class 'numpy.float64'>
```

The excluded argument can be used to prevent vectorizing over certain arguments. This can be useful for array-like arguments of a fixed length such as the coefficients for a polynomial as in polyval:

```
>>> def mypolyval(p, x):
...     _p = list(p)
...     res = _p.pop(0)
...     while _p:
...         res = res*x + _p.pop(0)
...     return res
```

Here, we exclude the zeroth argument from vectorization whether it is passed by position or keyword.

```
>>> vpolyval = np.vectorize(mypolyval, excluded={0, 'p'})
>>> vpolyval([1, 2, 3], x=[0, 1])
array([3, 6])
>>> vpolyval(p=[1, 2, 3], x=[0, 1])
array([3, 6])
```

7.22 Summary and examples: Numpy utility functions

7.22.1 Exact mathematical constants

ctx.zero

Returns zero.

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(repr(ctx.zero))
0.0
mpf('0.0')
mpi('0.0', '0.0')
Decimal('0')
mpfr('0.0')
arb('0')
```

ctx.one

Returns one.

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(repr(ctx.one))
1.0
mpf('1.0')
mpi('1.0', '1.0')
Decimal('1')
mpfr('1.0')
arb('1.00')
```

ctx.j

Returns the imaginary unit.

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(repr(ctx.j))
1j
mpc(real='0.0', imag='1.0')
iv.mpc(mpi('0.0', '0.0'), mpi('1.0', '1.0'))
DecCplx('0 + 1j')
mpc('0.0+1.0j')
acb('0 + 1.00j')
```

ctx.inf

Returns positive infinity.

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(repr(ctx.inf))
inf
mpf('+inf')
mpi('+inf', '+inf')
Decimal('Infinity')
mpfr('inf')
arb('+inf')
```

ctx.ninf

Returns negative infinity.

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(repr(ctx.ninf))
-inf
mpf('-inf')
```

(continues on next page)

(continued from previous page)

```
mpi('-inf', '-inf')
Decimal('-Infinity')
mpfr('-inf')
arb('-inf')
```

ctx.nan

Returns Not-a-Number (NaN).

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(repr(ctx.nan))
nan
mpf('nan')
mpi('nan', 'nan')
Decimal('NaN')
mpfr('nan')
arb('nan')
```

7.22.2 Complex components

numpy.real, **ctx.real**, **ctx.re**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.real.html#numpy.real

Return the real part of the complex argument.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.real(1 + 1j)
1.0
>>> a = np.array([1+2j, 3+4j, 5+6j])
>>> np.real(a)
array([1.,  3.,  5.])
```

From mpmath See also Mpmath [679].

Returns the real part of x, $\Re(x)$. `re()` converts a non-mpmath number to an mpmath number:

```
>>> from xlcalcnet import mp, iv, fp, dp, gp, ap
>>> mp.dps = 15; mp.pretty = False
>>> x = 3
>>> fp.re(x), mp.re(x), iv.re(x), dp.re(x), gp.re(x), ap.re(x)
(3.0,
 mpf('3.0'),
 mpi('3.0', '3.0'),
 Decimal('3'),
 mpfr('3.0'),
 arb3_t('3.00000000000000'))

>>> x = -1+4j
>>> fp.re(x), mp.re(x), iv.re(x), dp.re(x), gp.re(x), ap.re(x)
(-1.0,
 mpf('-1.0'),
 mpi('-1.0', '-1.0'),
 Decimal('-1'),
 mpfr('-1.0'),
 arb3_t('-1.00000000000000'))
```

numpy.imag, **ctx.imag**, **ctx.im**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.imag

Return the imaginary part of the complex argument.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.imag(1 + 1j)
1.0
>>> a = np.array([1+2j, 3+4j, 5+6j])
>>> np.imag(a)
array([2.,  4.,  6.])
```

From mpmath. See also Mpmath [673].

Returns the imaginary part of x, $\Im(x)$. `im()` converts a non-mpmath number to an mpmath number:

```
>>> from xlcalcnet import mp, iv, fp, dp, gp, ap
>>> mp.dps = 15; mp.pretty = False
>>> x = 3
>>> fp.im(x), mp.im(x), iv.im(x), dp.im(x), gp.im(x), ap.im(x)
(0.0, mpf('0.0'), mpi('0.0', '0.0'), Decimal('0'), mpfr('0.0'), arb3_t('0'))

>>> x = -1+4j
>>> fp.im(x), mp.im(x), iv.im(x), dp.im(x), gp.im(x), ap.im(x)
(4.0,
 mpf('4.0'),
 mpi('4.0', '4.0'),
 Decimal('4.0'),
 mpfr('4.0'),
 arb3_t('4.00000000000000'))
```

numpy.absolute, **numpy.abs**, **ctx.fabs**

https://numpy.org/doc/stable/reference/generated/numpy.absolute.html

See also: https://numpy.org/doc/stable/reference/generated/numpy.fabs.html

Calculate the absolute value of x, $|x|$, element-wise.

```
>>> x = np.array([-1.2, 1.2])
>>> np.absolute(x)
array([ 1.2,  1.2])
>>> np.absolute(1.2 + 1j)
1.5620499351813308
```

The abs function can be used as a shorthand for np.absolute on ndarrays.

```
>>> x = np.array([-1.2, 1.2])
>>> abs(x)
array([1.2, 1.2])
```

From mpmath. See also Mpmath [664].

```
>>> from xlcalcnet import mp, iv, fp, dp, gp, ap
>>> fp.fabs(3), mp.fabs(3), iv.fabs(3), dp.fabs(3), gp.fabs(3), ap.fabs(3)
(3.0, mpf('3.0'), mpi('3.0', '3.0'), Decimal('3'), 3, 3)
>>> fp.fabs(-3), mp.fabs(-3), iv.fabs(-3), dp.fabs(-3), gp.fabs(-3), ap.fabs(-3)
(3.0, mpf('3.0'), mpi('3.0', '3.0'), Decimal('3'), 3, 3)
>>> fp.fabs(3+4j), mp.fabs(3+4j), iv.fabs(3+4j), dp.fabs(3+4j), gp.fabs(3+4j), ap.fabs(3+4j)
(5.0, mpf('5.0'), mpi('5.0', '5.0'), Decimal('5.0'), 5.0, 5.0)
```

numpy.angle, **ctx.angle**, **ctx.arg**, **ctx.phase**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.angle.html

Return the angle of the complex argument. Although the angle of the complex number 0 is undefined, numpy.angle(0) returns the value 0.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.angle([1.0, 1.0j, 1+1j])               # in radians
array([ 0.        ,  1.57079633,  0.78539816]) # may vary
>>> np.angle(1+1j, deg=True)                  # in degrees
45.0
```

From mpmath. See also Mpmath [667].

Example:

```
>>> from xlcalcnet import mp, iv, fp, dp, gp, ap
>>> mp.dps = 15; mp.pretty = True
>>> x = 3
>>> fp.arg(x), mp.arg(x), iv.arg(x), dp.arg(x), gp.arg(x), ap.arg(x)
(0.0, mpf('0.0'), mpi('0.0', '0.0'), Decimal('0.0'), mpfr('0.0'), arb3_t('0'))

>>> x = 3+3j
>>> fp.arg(x), mp.arg(x), iv.arg(x), dp.arg(x), gp.arg(x), ap.arg(x)
(0.7853981633974483,
 mpf('0.78539816339744828'),
 mpi('0.78539816339744828', '0.78539816339744839'),
 Decimal('0.785398163397448'),
 mpfr('0.78539816339744828'),
 arb3_t('[0.785398163397448 +/- 3.35e-16]'))

>>> x = 3j
>>> fp.arg(x), mp.arg(x), iv.arg(x), dp.arg(x), gp.arg(x), ap.arg(x)
(1.5707963267948966,
 mpf('1.5707963267948966'),
 mpi('1.5707963267948966', '1.5707963267948968'),
 Decimal('1.5707963267949'),
 mpfr('1.5707963267948966'),
 arb3_t('[1.57079632679490 +/- 3.56e-15]'))

>>> x = -3
>>> fp.arg(x), mp.arg(x), iv.arg(x), dp.arg(x), gp.arg(x), ap.arg(x)
(3.141592653589793,
 mpf('3.1415926535897931'),
 mpi('3.1415926535897931', '3.1415926535897936'),
 Decimal('3.14159265358979'),
 mpfr('3.1415926535897931'),
 arb3_t('[3.14159265358979 +/- 3.34e-15]'))
```

The angle is defined to satisfy $-\pi < \arg(x) \leq \pi$ and with the sign convention that a nonnegative imaginary part results in a nonnegative argument.

The value returned by `arg()` is an `mpf` instance.

numpy.sign, **ctx.sign**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.sign.html

Returns an element-wise indication of the sign of a number.

The sign function returns -1 if x < 0, 0 if x==0, 1 if x > 0. nan is returned for nan inputs.

For complex inputs, the sign function returns sign(x.real) + 0j if x.real != 0 else sign(x.imag) + 0j.

complex(nan, 0) is returned for complex nan inputs.

```
>>> np.sign([-5., 4.5])
array([-1.,  1.])
>>> np.sign(0)
0
```

(continues on next page)

(continued from previous page)

```
>>> np.sign(5-2j)
(1+0j)
```

From mpmath. See also Mpmath [680].

Returns the sign of x, defined as $\text{sign}(x) = x/|x|$ (with the special case $\text{sign}(0) = 0$):

```
>>> from xlcalcnet import mp, iv, fp, dp, gp, ap
>>> mp.dps = 15; mp.pretty = False
>>> x = 10
>>> fp.sign(x), mp.sign(x), iv.sign(x), dp.sign(x), gp.sign(x), ap.sign(x)
(1.0,
 mpf('1.0'),
 mpi('1.0', '1.0'),
 Decimal('1'),
 1,
 arb3_t('1.00000000000000'))

>>> x = -10
>>> fp.sign(x), mp.sign(x), iv.sign(x), dp.sign(x), gp.sign(x), ap.sign(x)
(-1.0,
 mpf('-1.0'),
 mpi('-1.0', '-1.0'),
 Decimal('-1'),
 -1,
 arb3_t('-1.00000000000000'))

>>> x = 0
>>> fp.sign(x), mp.sign(x), iv.sign(x), dp.sign(x), gp.sign(x), ap.sign(x)
(0.0, mpf('0.0'), mpi('-1.0', '-1.0'), Decimal('0'), 0, arb3_t('0'))
```

Note that the sign function is also defined for complex numbers, for which it gives the projection onto the unit circle:

```
>>> mp.dps = 15; mp.pretty = True
>>> x = 1+1j
>>> fp.sign(x), mp.sign(x), iv.sign(x), dp.sign(x), gp.sign(x), ap.sign(x)
((0.707106781186475+0.707106781186475j),
 mpc(real='0.70710678118654746', imag='0.70710678118654746'),
 iv.mpc(mpi('0.7071067811703', '0.7071067811994'), mpi('0.7071067811703', '0.7071067811994')),
 DecCplx('0.707106781186547 + 0.707106781186547j'),
 mpc('0.70710678118654746+0.70710678118654746j'),
 acb3_t('[0.707106781186548 +/- 6.50e-16] + [0.707106781186548 +/- 6.50e-16]j'))
```

numpy.conjugate, **numpy.conj**

https://numpy.org/doc/stable/reference/generated/numpy.conj.html

Return the complex conjugate, element-wise.

The complex conjugate of a complex number is obtained by changing the sign of its imaginary part.

```
>>> np.conjugate(1+2j)
(1-2j)

>>> x = np.eye(2) + 1j * np.eye(2)
>>> np.conjugate(x)
array([[ 1.-1.j,  0.-0.j],
       [ 0.-0.j,  1.-1.j]])
```

From mpmath. See also Mpmath [668].

!!! Error in iv with complex input !!!

Returns the complex conjugate of x, $\overline{x}$. Unlike `x.conjugate()`, `im()` converts x to a mpmath number:

Example:

```
>>> from xlcalcnet import mp, iv, fp, dp, gp, ap
>>> mp.dps = 15; mp.pretty = True
>>> x = 3
>>> fp.conj(x), mp.conj(x), iv.conj(x), dp.conj(x), gp.conj(x), ap.conj(x)
(3.0,
 mpf('3.0'),
 mpi('3.0', '3.0'),
 Decimal('3'),
 mpfr('3.0'),
 arb3_t('3.00000000000000'))

>>> x = -1+4j
>>> fp.conj(x), mp.conj(x), dp.conj(x), gp.conj(x), ap.conj(x)
((-1-4j),
 mpc(real='-1.0', imag='-4.0'),
 DecCplx('-1 - 4.0j'),
 mpc('-1.0-4.0j'),
 acb3_t('-1.00000000000000 - 4.00000000000000j'))
```

7.22.3 Array contents

numpy.isfinite, **ctx.isfinite**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.isfinite.html

Test element-wise for finiteness (not infinity and not Not a Number). The result is returned as a boolean array. Not a Number, positive infinity and negative infinity are considered to be non-finite.

NumPy uses the IEEE Standard for Binary Floating-Point for Arithmetic (IEEE 754). This means that Not a Number is not equivalent to infinity. Also that positive infinity is not equivalent to negative infinity. But infinity is equivalent to positive infinity. Errors result if the second argument is also supplied when x is a scalar input, or if first and second arguments have different shapes.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.isfinite(1)
True
>>> np.isfinite(0)
True
>>> np.isfinite(np.nan)
False
>>> np.isfinite(np.inf)
False
>>> np.isfinite(np.NINF)
False
>>> np.isfinite([np.log(-1.),1.,np.log(0)])
array([False,  True, False])

>>> x = np.array([-np.inf, 0., np.inf])
>>> y = np.array([2, 2, 2])
>>> np.isfinite(x, y)
array([0, 1, 0])
>>> y
array([0, 1, 0])
```

From mpmath. See also Mpmath [691].

Return *True* if *x* is a finite number, i.e. neither an infinity or a NaN.

Example:

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(ctx.ctx.isfinite(ctx.inf), end=', ')
```

(continues on next page)

(continued from previous page)

```
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ctx.isfinite(ctx.ninf), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ctx.isfinite(ctx.nan), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ctx.isfinite(3), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.ctx.isfinite(3+4j), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.ctx.isfinite(ctx.mpc(3,ctx.nan)), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ctx.isfinite(ctx.mpc(ctx.nan,3)), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ctx.isfinite(ctx.mpc(3,ctx.inf)), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ctx.isfinite(ctx.mpc(ctx.inf,3)), end=', ')
False, False, False, False, False, False,
```

numpy.isinf, ctx.isinf

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.isinf

Test element-wise for positive or negative infinity.

Returns a boolean array of the same shape as x, True where `x == +/-inf`, otherwise False. NumPy uses the IEEE Standard for Binary Floating-Point for Arithmetic (IEEE 754). Errors result if the second argument is supplied when the first argument is a scalar, or if the first and second arguments have different shapes.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.isinf(np.inf)
True
>>> np.isinf(np.nan)
False
>>> np.isinf(np.NINF)
True
>>> np.isinf([np.inf, -np.inf, 1.0, np.nan])
array([ True,  True, False, False])

>>> x = np.array([-np.inf, 0., np.inf])
>>> y = np.array([2, 2, 2])
>>> np.isinf(x, y)
array([1, 0, 1])
>>> y
array([1, 0, 1])
```

From mpmath. See also Mpmath [692].

Return *True* if the absolute value of *x* is infinite; otherwise return *False*.

Example:

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(ctx.isinf(ctx.inf), end=', ')
True, True, True, True, True, True,
```

(continues on next page)

(continued from previous page)

```
>>> for ctx in ctxall: print(ctx.isinf(ctx.ninf), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.isinf(ctx.nan), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isinf(3), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isinf(3+4j), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isinf(ctx.mpc(3,ctx.nan)), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isinf(ctx.mpc(ctx.nan,3)), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isinf(ctx.mpc(3,ctx.inf)), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.isinf(ctx.mpc(ctx.inf,3)), end=', ')
True, True, True, True, True, True,
```

numpy.isnan, ctx.isnan

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.isnan.html#numpy.isnan

Test element-wise for NaN and return result as a boolean array. NumPy uses the IEEE Standard for Binary Floating-Point for Arithmetic (IEEE 754). This means that Not a Number is not equivalent to infinity.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.isnan(np.nan)
True
>>> np.isnan(np.inf)
False
>>> np.isnan([np.log(-1.),1.,np.log(0)])
array([ True, False, False])
```

From mpmath. See also Mpmath [690].

Return *True* if *x* is a NaN (not-a-number), or for a complex number, whether either the real or complex part is NaN; otherwise return *False*:

Example:

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(ctx.isnan(ctx.nan), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.isnan(ctx.inf), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnan(ctx.ninf), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnan(3), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnan(3+4j), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnan(ctx.mpc(3,ctx.inf)), end=', ')
```

(continues on next page)

(continued from previous page)

```
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnan(ctx.mpc(ctx.inf,3)), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnan(ctx.mpc(3,ctx.nan)), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.isnan(ctx.mpc(ctx.nan,3)), end=', ')
True, True, True, True, True, True,
```

numpy.isnat, **ctx.isnat**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.isnat.html#numpy.isnat

Test element-wise for NaT (not a time) and return result as a boolean array.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.isnat(np.datetime64("NaT"))
True
>>> np.isnat(np.datetime64("2016-01-01"))
False
>>> np.isnat(np.array(["NaT", "2016-01-01"], dtype="datetime64[ns]"))
array([ True, False])
```

numpy.isneginf, **ctx.isneginf**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.isneginf.html#numpy.isneginf

Test element-wise for negative infinity, return result as bool array. NumPy uses the IEEE Standard for Binary Floating-Point for Arithmetic (IEEE 754). Errors result if the second argument is also supplied when x is a scalar input, if first and second arguments have different shapes, or if the first argument has complex values.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.isneginf(np.NINF)
True
>>> np.isneginf(np.inf)
False
>>> np.isneginf(np.PINF)
False
>>> np.isneginf([-np.inf, 0., np.inf])
array([ True, False, False])

>>> x = np.array([-np.inf, 0., np.inf])
>>> y = np.array([2, 2, 2])
>>> np.isneginf(x, y)
array([1, 0, 0])
>>> y
array([1, 0, 0])
```

numpy.isposinf, **ctx.isposinf**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.isposinf.html#numpy.isposinf

Test element-wise for positive infinity, return result as bool array. NumPy uses the IEEE Standard for Binary Floating-Point for Arithmetic (IEEE 754). Errors result if the second argument is also supplied when x is a scalar input, if first and second arguments have different shapes, or if the first argument has complex values.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.isposinf(np.PINF)
True
>>> np.isposinf(np.inf)
True
>>> np.isposinf(np.NINF)
False
>>> np.isposinf([-np.inf, 0., np.inf])
array([False, False,  True])

>>> x = np.array([-np.inf, 0., np.inf])
>>> y = np.array([2, 2, 2])
>>> np.isposinf(x, y)
array([0, 0, 1])
>>> y
array([0, 0, 1])
```

ctx.isint(x, gaussian=False)

From mpmath. See also Mpmath [693].

Return *True* if *x* is integer-valued; otherwise return *False*:

Example:

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(ctx.isint(ctx.inf), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isint(ctx.ninf), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isint(ctx.nan), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isint(3), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.isint(3+0j), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.isint(3+4j), end=', ')
False, False, False, False, False, False,
```

Optionally, Gaussian integers can be checked for:

```
>>> isint(3+2j, gaussian=True)
True
```

ctx.isnormal(x)

From mpmath. . See also Mpmath [694].

Determine whether *x* is "normal" in the sense of floating-point representation; that is, return *False* if *x* is zero, an infinity or NaN; otherwise return *True*.

By extension, a complex number *x* is considered "normal" if its magnitude is normal:

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(ctx.isnormal(ctx.nan), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnormal(ctx.inf), end=', ')
False, False, False, False, False, False,
```

(continues on next page)

(continued from previous page)

```
>>> for ctx in ctxall: print(ctx.isnormal(ctx.ninf), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnormal(0), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnormal(3), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.isnormal(3+4j), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.isnormal(ctx.mpc(3,ctx.inf)), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnormal(ctx.mpc(ctx.inf,3)), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnormal(ctx.mpc(3,ctx.nan)), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.isnormal(ctx.mpc(ctx.nan,3)), end=', ')
False, False, False, False, False, False,
```

7.22.4 Array type testing

numpy.iscomplex, **ctx.iscomplex**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.iscomplex.html#numpy.iscomplex

Returns a bool array, where True if input element is complex. What is tested is whether the input has a non-zero imaginary part, not if the input type is complex.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.iscomplex([1+1j, 1+0j, 4.5, 3, 2, 2j])
array([ True, False, False, False, False,  True])
```

numpy.iscomplexobj, **ctx.iscomplexobj**, **ctx.ismpc**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.iscomplexobj.html#numpy.iscomplexobj

Check for a complex type or an array of complex numbers. The type of the input is checked, not the value. Even if the input has an imaginary part equal to zero, iscomplexobj evaluates to True.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.iscomplexobj(1)
False
np.iscomplexobj(1+0j)
True
np.iscomplexobj([3, 1+0j, True])
True
```

From mpmath:

Returns *True* if *x* is an instance of `ctx.mpc`; otherwise returns *False*.

Example:

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(ctx.ismpc(ctx.inf), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ismpc(ctx.ninf), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ismpc(ctx.nan), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ismpc(ctx.one), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ismpc(1.0), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ismpc(ctx.j), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.ismpc(1j), end=', ')
True, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ismpc(ctx.mpc(3,ctx.inf)), end=', ')
True, True, True, True, True, True,
```

numpy.isreal, **ctx.isreal**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.isreal.html#numpy.isreal

Returns a bool array, where True if input element is real. If element has complex type with zero complex part, the return value for that element is True. This function may behave unexpectedly for string or object arrays (see examples)

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> a = np.array([1+1j, 1+0j, 4.5, 3, 2, 2j], dtype=complex)
>>> np.isreal(a)
array([False,  True,  True,  True,  True, False])
```

The function does not work on string arrays.

```
>>> a = np.array([2j, "a"], dtype="U")
>>> np.isreal(a)  # Warns about non-elementwise comparison
False
```

Returns True for all elements in input array of dtype=object even if any of the elements is complex.

```
>>> a = np.array([1, "2", 3+4j], dtype=object)
>>> np.isreal(a)
array([ True,  True,  True])
```

`isreal` should not be used with object arrays

```
>>> a = np.array([1+2j, 2+1j], dtype=object)
>>> np.isreal(a)
array([ True,  True])
```

numpy.isrealobj, **ctx.isrealobj**, **ctx.ismpf**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.isrealobj.html#numpy.isrealobj

Return True if x is a not complex type or an array of complex numbers. The type of the input is checked, not the value. So even if the input has an imaginary part equal to zero, `isrealobj` evaluates to False if the data type is complex. The function is only meant

for arrays with numerical values but it accepts all other objects. Since it assumes array input, the return value of other objects may be True.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.isrealobj('A string')
True
>>> np.isrealobj(False)
True
>>> np.isrealobj(None)
True

>>> np.isrealobj(1)
True
>>> np.isrealobj(1+0j)
False
>>> np.isrealobj([3, 1+0j, True])
False
```

From mpmath:

Returns *True* if *x* is an instance of `ctx.mpf`; otherwise returns *False*.

Example:

```
>>> from xlcalcnet import fpm, mpm, ipm, dec, gmp, apm; ctxall = [fpm, mpm, ipm, dec, gmp, apm]
>>> for ctx in ctxall: print(ctx.ismpf(ctx.inf), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.ismpf(ctx.ninf), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.ismpf(ctx.nan), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.ismpf(ctx.one), end=', ')
True, True, True, True, True, True,

>>> for ctx in ctxall: print(ctx.ismpf(1.0), end=', ')
True, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ismpf(ctx.j), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ismpf(1j), end=', ')
False, False, False, False, False, False,

>>> for ctx in ctxall: print(ctx.ismpf(ctx.mpc(3,ctx.inf)), end=', ')
False, False, False, False, False, False,
```

numpy.isscalar, **ctx.isscalar**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.isscalar.html#numpy.isscalar

Returns True if the type of element is a scalar type. In most cases `np.ndim(x) == 0` should be used instead of this function, as that will also return true for 0d arrays.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.isscalar(3.1)
True
>>> np.isscalar(np.array(3.1))
False
>>> np.isscalar([3.1])
False
>>> np.isscalar(False)
```

(continues on next page)

(continued from previous page)

```
True
>>> np.isscalar('numpy')
True
```

NumPy supports PEP 3141 numbers:

```
>>> from fractions import Fraction
>>> np.isscalar(Fraction(5, 17))
True
>>> from numbers import Number
>>> np.isscalar(Number())
True
```

7.22.5 Using tolerances

numpy.allclose, **ctx.allclose**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.allclose.html#numpy.allclose

Returns True if two arrays are element-wise equal within a tolerance.

The tolerance values are positive, typically very small numbers. The relative difference (rtol * abs(b)) and the absolute difference atol are added together to compare against the absolute difference between a and b.

NaNs are treated as equal if they are in the same place and if `equal_nan=True`. Infs are treated as equal if they are in the same place and of the same sign in both arrays.

If the following equation is element-wise True, then allclose returns True.

absolute(a - b) <= (atol + rtol * absolute(b))

The above equation is not symmetric in a and b, so that `allclose(a, b)` might be different from `allclose(b, a)` in some rare cases.

The comparison of a and b uses standard broadcasting, which means that a and b need not have the same shape in order for `allclose(a, b)` to evaluate to True. The same is true for equal but not array_equal.

allclose is not defined for non-numeric data types. bool is considered a numeric data-type for this purpose.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.allclose([1e10,1e-7], [1.00001e10,1e-8])
False
>>> np.allclose([1e10,1e-8], [1.00001e10,1e-9])
True
>>> np.allclose([1e10,1e-8], [1.0001e10,1e-9])
False
>>> np.allclose([1.0, np.nan], [1.0, np.nan])
False
>>> np.allclose([1.0, np.nan], [1.0, np.nan], equal_nan=True)
True
```

numpy.isclose, **ctx.isclose**, **ctx.almosteq(s, t, rel_eps=None, abs_eps=None)**

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.isclose.html#numpy.isclose

Returns a boolean array where two arrays are element-wise equal within a tolerance.

The tolerance values are positive, typically very small numbers. The relative difference (rtol * abs(b)) and the absolute difference atol are added together to compare against the absolute difference between a and b.

For finite values, isclose uses the following equation to test whether two floating point values are equivalent.

absolute(a - b) <= (atol + rtol * absolute(b))

Unlike the built-in math.isclose, the above equation is not symmetric in a and b – it assumes b is the reference value – so that isclose(a, b) might be different from isclose(b, a). Furthermore, the default value of atol is not zero, and is used to determine what small values should be considered close to zero. The default value is appropriate for expected values of order unity: if the expected values are significantly smaller than one, it can result in false positives. atol should be carefully selected for the use case at hand. A zero value for atol will result in False if either a or b is zero.

isclose is not defined for non-numeric data types. bool is considered a numeric data-type for this purpose.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.isclose([1e10,1e-7], [1.00001e10,1e-8])
array([ True, False])
>>> np.isclose([1e10,1e-8], [1.00001e10,1e-9])
array([ True, True])
>>> np.isclose([1e10,1e-8], [1.0001e10,1e-9])
array([False,  True])

>>> np.isclose([1.0, np.nan], [1.0, np.nan])
array([ True, False])
>>> np.isclose([1.0, np.nan], [1.0, np.nan], equal_nan=True)
array([ True, True])

>>> np.isclose([1e-8, 1e-7], [0.0, 0.0])
array([ True, False])
>>> np.isclose([1e-100, 1e-7], [0.0, 0.0], atol=0.0)
array([False, False])
>>> np.isclose([1e-10, 1e-10], [1e-20, 0.0])
array([ True,  True])
>>> np.isclose([1e-10, 1e-10], [1e-20, 0.999999e-10], atol=0.0)
array([False,  True])
```

From mpmath. See also Mpmath [695].

Determine whether the difference between s and t is smaller than a given epsilon, either relatively or absolutely.

Both a maximum relative difference and a maximum difference ('epsilons') may be specified. The absolute difference is defined as $|s-t|$ and the relative difference is defined as $|s-t|/\max(|s|,|t|)$.

If only one epsilon is given, both are set to the same value. If none is given, both epsilons are set to 2^{-p+m} where p is the current working precision and m is a small integer. The default setting typically allows `almosteq()` to be used to check for mathematical equality in the presence of small rounding errors.

Examples

```
>>> from xlcalcnet import mp, iv, fp, dp, gp, ap
>>> mp.dps = 15; mp.pretty = False
>>> s = 3.141592653589793; t = 3.141592653589790
>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(ctx.almosteq(s, t), end=', ')
True, True, True, True, True, True,

>>> s = 3.141592653589793; t = 3.141592653589700
>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(ctx.almosteq(s, t), end=', ')
False, False, False, False, False, False,

>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(ctx.almosteq(s, t, 1e-10), end=', ')
True, True, True, True, True, True,

>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(ctx.almosteq(1e-20, 2e-20), end=', ')
True, True, True, True, True, True,

>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(ctx.almosteq(1e-20, 2e-20, rel_eps=0, abs_eps=0), end=', ')
False, False, False, False, False, False,
```

7.22.6 Clipping (limiting) the values in an array.

numpy.clip

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.clip.html

Clip (limit) the values in an array.

Given an interval, values outside the interval are clipped to the interval edges. For example, if an interval of `[0, 1]` is specified, values smaller than 0 become 0, and values larger than 1 become 1.

Equivalent to but faster than `np.minimum(a_max, np.maximum(a, a_min))`.

No check is performed to ensure `a_min < a_max`.

When a_min is greater than a_max, `clip` returns an array in which all values are equal to a_max, as shown in the second example.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> a = np.arange(10)
>>> a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.clip(a, 1, 8)
array([1, 1, 2, 3, 4, 5, 6, 7, 8, 8])
>>> np.clip(a, 8, 1)
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
>>> np.clip(a, 3, 6, out=a)
array([3, 3, 3, 3, 4, 5, 6, 6, 6, 6])
>>> a
array([3, 3, 3, 3, 4, 5, 6, 6, 6, 6])
>>> a = np.arange(10)
>>> a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.clip(a, [3, 4, 1, 1, 1, 4, 4, 4, 4, 4], 8)
array([3, 4, 2, 3, 4, 5, 6, 7, 8, 8])
```

numpy.real_if_close

For a detailed description of parameters and return values see:

https://numpy.org/doc/stable/reference/generated/numpy.real_if_close.html

If input is complex with all imaginary parts close to zero, return real parts.

“Close to zero” is defined as tol * (machine epsilon of the type for a).

Machine epsilon varies from machine to machine and between data types but Python floats on most platforms have a machine epsilon equal to 2.2204460492503131e-16. You can use ‘np.finfo(float).eps’ to print out the machine epsilon for floats.

```
>>> import numpy as np; from xlcalcnet import mpm, dpm
>>> np.finfo(float).eps
2.22044604925031e-16 # may vary

>>> np.real_if_close([2.1 + 4e-14j, 5.2 + 3e-15j], tol=1000)
array([2.1, 5.2])
>>> np.real_if_close([2.1 + 4e-13j, 5.2 + 3e-15j], tol=1000)
array([2.1+4.e-13j, 5.2 + 3e-15j])
```

ctx.chop(x, tol=None)

See also Mpmath [683].

!!! iv context does not accept lists: NotImplementedError !!!

!!! dp, gp and ap contexts do not accept lists: return None !!!

Chops off small real or imaginary parts, or converts numbers close to zero to exact zeros. The input can be a single number or an iterable:

Example:

```
>>> from xlcalcnet import mp, iv, fp, dp, gp, ap
>>> mp.dps = 15; mp.pretty = False

>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(repr(ctx.chop(1e-10, tol = 1e-11)))
1e-10
mpf('1.0e-10')
mpi('1.0e-10', '1.0e-10')
Decimal('1E-10')
mpfr('1e-10')
arb3_t('[1.00000000000000e-10 +/- 3.66e-27]')

>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(repr(ctx.chop(1e-10, tol = 1e-9)))
0.0
mpf('0.0')
mpi('0.0', '0.0')
Decimal('0')
mpfr('0.0')
arb3_t('0')

>>> for ctx in [fp, mp, iv, dp, gp, ap]: print(repr(ctx.chop(5+1e-10j, tol = 1e-9)))
5.0
mpf('5.0')
mpi('5.0', '5.0')
Decimal('5')
mpfr('5.0')
arb3_t('5.00000000000000')

>>> for ctx in [mp, fp, dp, gp, ap]: print(ctx.chop([1.0, 1e-20, 3+1e-18j, -4, 2]))
[mpf('1.0'), mpf('0.0'), mpf('3.0'), mpf('-4.0'), mpf('2.0')]
[1.0, 0.0, 3.0, -4.0, 2.0]

None
None
None
```

The tolerance defaults to `100*eps`.

7.23 Arithmetic operations with scalars and iterables

7.23.1 fadd: Addition using a custom precision and rounding mode

`ctx.fadd`(*x*, *y*, ***kwargs*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`. See also Mpmath [665].

!!! There is no special code for fadd except mp !!!

!!! Add special code for dp, gp, ap !!!

!!! fp and iv still missing !!!

Adds the numbers *x* and *y*, giving a floating-point result, optionally using a custom precision and rounding mode.

The default precision is the working precision of the context. You can specify a custom precision in bits by passing the *prec* keyword argument, or by providing an equivalent decimal precision with the *dps* keyword argument. If the precision is set to `+inf`, or if the flag *exact=True* is passed, an exact addition with no rounding is performed. Changing the precision has no effect for the fp context.

For the mp, dp, gp, and ap (in point mode only) contexts: when the precision is finite, the optional *rounding* keyword argument specifies the direction of rounding. Valid options are `'n'` for nearest (default), `'f'` for floor, `'c'` for ceiling, `'d'` for down, `'u'` for up.

Examples

Using *fadd()* with precision and rounding control:

```
>>> from xlcalcnet import fp, mp, iv, dp, gp, ap
>>> mp.fadd(2, 1e-20)
mpf('2.0')
>>> mp.nprint(mp.fadd(2, 1e-20, prec=100), 25)
2.000000000000000000001
>>> mp.nprint(mp.fadd(2, 1e-20, dps=15), 25)
2.0
>>> mp.nprint(mp.fadd(2, 1e-20, dps=25), 25)
2.000000000000000000001
>>> mp.nprint(mp.fadd(2, 1e-20, exact=True), 25)
2.000000000000000000001




>>> gp.dps = 15; gp.pretty = False
>>> gp.fadd(2, 1e-20)
mpfr('2.0')
>>> gp.nprint(gp.fadd(2, 1e-20, prec=100), 25)
2.0000000000000000000000000E+00
>>> gp.nprint(gp.fadd(2, 1e-20, dps=15), 25)
2.0000000000000000000000000E+00
>>> gp.nprint(gp.fadd(2, 1e-20, dps=25), 25)
2.0000000000000000000000000E+00
>>> gp.nprint(gp.fadd(2, 1e-20, exact=True), 25)
2.0000000000000000000000000E+00


>>> dp.dps = 15; dp.pretty = False
>>> dp.fadd(2, 1e-20)
Decimal('2.000000000000000')
>>> dp.nprint(dp.fadd(2, 1e-20, prec=100), 25)
2.0000000000000000000000000E+0
>>> dp.dps=100
>>> dp.mpf(2) + dp.mpf('1e-20')
Decimal('2.000000000000000000001')
>>> dp.nprint(dp.fadd(2, 1e-20, dps=15), 25)
```

(continues on next page)

(continued from previous page)

```
2.0000000000000000000010000E+0
>>> dp.nprint(dp.fadd(2, 1e-20, dps=25), 25)
2.0000000000000000000010000E+0
>>> dp.nprint(dp.fadd(2, 1e-20, exact=True), 25)
2.0000000000000000000010000E+0


>>> ap.dps = 15; ap.pretty = False
>>> ap.fadd(2, 1e-20)
arb3_t('[2.00000000000000 +/- 4.45e-16]')
>>> ap.fadd(2, 1e-20, rounding='u')
 arb3_t('[2.00000000000000 +/- 4.45e-16]')
>>> ap.nprint(ap.fadd(2, 1e-20, prec=100), 25)
# nprint not properly implemented
>>> ap.dps=25
>>> ap.mpf(2) + ap.mpf('1e-20')
arb3_t('[2.000000000000000000010000 +/- 5.85e-26]')
>>> ap.nprint(ap.fadd(2, 1e-20, dps=15), 25)
# nprint not properly implemented
>>> ap.nprint(ap.fadd(2, 1e-20, dps=25), 25)
# nprint not properly implemented
>>> ap.nprint(ap.fadd(2, 1e-20, exact=True), 25)
# nprint not properly implemented
```

This is the current state of affairs in iv:

```
>>> iv.dps = 15; iv.pretty = False
>>> iv.fadd(2, 1e-20)
mpi('2.0', '2.0000000000000004')
>>> iv.nprint(iv.fadd(2, 1e-20, prec=100), 25)
[2.0, 2.0000000000000000044408921]#    nprint not properly implemented
>>> iv.nprint(iv.fadd(2, 1e-20, dps=15), 25)
[2.0, 2.0000000000000000044408921]#    nprint not properly implemented
>>> iv.nprint(iv.fadd(2, 1e-20, dps=25), 25)
[2.0, 2.0000000000000000044408921]#    nprint not properly implemented
>>> iv.nprint(iv.fadd(2, 1e-20, exact=True), 25)
[2.0, 2.0000000000000000044408921]#    nprint not properly implemented
```

TODO: Another ap example in floating point mode

See also: https://docs.python.org/3/tutorial/floatingpoint.html#tut-fp-issues.

Exact addition avoids cancellation errors, enforcing familiar laws of numbers such as $x + y - x = y$, which don't hold in floating-point arithmetic with finite precision:

```
>>> x, y = mp.mpf(2), mp.mpf('1e-1000')
>>> print(x + y - x)
0.0
>>> print(mp.fadd(x, y, prec=inf) - x)
1.0e-1000
>>> print(mp.fadd(x, y, exact=True) - x)
1.0e-1000
```

Exact addition can be inefficient and may be impossible to perform with large magnitude differences:

```
>>> fadd(1, '1e-1000000000000000000000', prec=inf)
Traceback (most recent call last):
    ...
OverflowError: the exact result does not fit in memory
```

7.23.2 fsub: Subtraction using a custom precision and rounding mode

`ctx.`**`fsub`**(*x*, *y*, ***kwargs*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`. See also Mpmath [681].

Subtracts the numbers *x* and *y*, giving a floating-point result, optionally using a custom precision and rounding mode.

See the documentation of *`fadd()`* for a detailed description of how to specify precision and rounding.

Examples

Using *`fsub()`* with precision and rounding control:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = False
>>> fsub(2, 1e-20)
mpf('2.0')
>>> fsub(2, 1e-20, rounding='d')
mpf('1.9999999999999998')
>>> nprint(fsub(2, 1e-20, prec=100), 25)
1.99999999999999999999
>>> nprint(fsub(2, 1e-20, dps=15), 25)
2.0
>>> nprint(fsub(2, 1e-20, dps=25), 25)
1.99999999999999999999
>>> nprint(fsub(2, 1e-20, exact=True), 25)
1.99999999999999999999
```

Exact subtraction avoids cancellation errors, enforcing familiar laws of numbers such as $x - y + y = x$, which don't hold in floating-point arithmetic with finite precision:

```
>>> x, y = mpf(2), mpf('1e1000')
>>> print(x - y + y)
0.0
>>> print(fsub(x, y, prec=inf) + y)
2.0
>>> print(fsub(x, y, exact=True) + y)
2.0
```

Exact subtraction can be inefficient and may be impossible to perform with large magnitude differences:

```
>>> fsub(1, '1e-1000000000000000000000', prec=inf)
Traceback (most recent call last):
    ...
OverflowError: the exact result does not fit in memory
```

7.23.3 fneg: Negation of a number using a custom precision and rounding mode

`ctx.`**`fneg`**(*x*, ***kwargs*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`. See also Mpmath [676].

!!! NEEDS TO DESCRIBE PARAMETERS !!!!

Negates the number *x*, giving a floating-point result, optionally using a custom precision and rounding mode.

See the documentation of *`fadd()`* for a detailed description of how to specify precision and rounding.

Examples

An mpmath number is returned:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = False
>>> fneg(2.5)
```

(continues on next page)

(continued from previous page)

```
mpf('-2.5')
>>> fneg(-5+2j)
mpc(real='5.0', imag='-2.0')
```

Precise control over rounding is possible:

```
>>> x = fadd(2, 1e-100, exact=True)
>>> fneg(x)
mpf('-2.0')
>>> fneg(x, rounding='f')
mpf('-2.0000000000000004')
```

Negating with and without roundoff:

```
>>> n = 200000000000000000000001
>>> print(int(-mpf(n)))
-200000000000000016777216
>>> print(int(fneg(n)))
-200000000000000016777216
>>> print(int(fneg(n, prec=log(n,2)+1)))
-200000000000000000000001
>>> print(int(fneg(n, dps=log(n,10)+1)))
-200000000000000000000001
>>> print(int(fneg(n, prec=inf)))
-200000000000000000000001
>>> print(int(fneg(n, dps=inf)))
-200000000000000000000001
>>> print(int(fneg(n, exact=True)))
-200000000000000000000001
```

7.23.4 fmul: Multiplication using a custom precision and rounding mode

`ctx.`**`fmul`**(*x*, *y*, ***kwargs*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`. See also Mpmath [675].

Multiplies the numbers *x* and *y*, giving a floating-point result, optionally using a custom precision and rounding mode.

See the documentation of *`fadd()`* for a detailed description of how to specify precision and rounding.

Examples

The result is an mpmath number:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = False
>>> fmul(2, 5.0)
mpf('10.0')
>>> fmul(0.5j, 0.5)
mpc(real='0.0', imag='0.25')
```

Avoiding roundoff:

```
>>> x, y = 10**10+1, 10**15+1
>>> print(x*y)
10000000001000010000000001
>>> print(mpf(x) * mpf(y))
1.0000000001e+25
>>> print(int(mpf(x) * mpf(y)))
10000000001000011026399232
>>> print(int(fmul(x, y)))
10000000001000011026399232
>>> print(int(fmul(x, y, dps=25)))
```

(continues on next page)

(continued from previous page)

```
100000000010000010000000001
>>> print(int(fmul(x, y, exact=True)))
100000000010000010000000001
```

Exact multiplication with complex numbers can be inefficient and may be impossible to perform with large magnitude differences between real and imaginary parts:

```
>>> x = 1+2j
>>> y = mpc(2, '1e-1000000000000000000000')
>>> fmul(x, y)
mpc(real='2.0', imag='4.0')
>>> fmul(x, y, rounding='u')
mpc(real='2.0', imag='4.0000000000000009')
>>> fmul(x, y, exact=True)
Traceback (most recent call last):
    ...
OverflowError: the exact result does not fit in memory
```

7.23.5 fdiv: Division using a custom precision and rounding mode

`ctx.fdiv`(*x*, *y*, ***kwargs*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`. See also Mpmath [670].

Divides the numbers *x* and *y*, giving a floating-point result, optionally using a custom precision and rounding mode.

See the documentation of *fadd()* for a detailed description of how to specify precision and rounding.

Examples

The result is an mpmath number:

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = False
>>> fdiv(3, 2)
mpf('1.5')
>>> fdiv(2, 3)
mpf('0.66666666666666663')
>>> fdiv(2+4j, 0.5)
mpc(real='4.0', imag='8.0')
```

The rounding direction and precision can be controlled:

```
>>> fdiv(2, 3, dps=3)    # Should be accurate to at least 3 digits
mpf('0.6666259765625')
>>> fdiv(2, 3, rounding='d')
mpf('0.66666666666666663')
>>> fdiv(2, 3, prec=60)
mpf('0.66666666666666667')
>>> fdiv(2, 3, rounding='u')
mpf('0.66666666666666674')
```

Checking the error of a division by performing it at higher precision:

```
>>> fdiv(2, 3) - fdiv(2, 3, prec=100)
mpf('-3.7007434154172148e-17')
```

Unlike *fadd()*, *fmul()*, etc., exact division is not allowed since the quotient of two floating-point numbers generally does not have an exact floating-point representation. (In the future this might be changed to allow the case where the division is actually exact.)

```
>>> fdiv(2, 3, exact=True)
Traceback (most recent call last):
    ...
ValueError: division is not an exact operation
```

7.23.6 fsum: Sum of a finite number of terms

`ctx.`**`fsum`**(*terms*, *absolute=False*, *squared=False*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`. See also Mpmath [682].

Calculates a sum containing a finite number of terms . With squared=True each term is squared, and with absolute=True the absolute value of each term is used.

```
>>> from xlcalcnet import fp, mp, iv, dp, gp, ap
>>> A = [1, 2, 0.5, 7]
>>> for ctx in [fp, mp, iv, dp, gp, ap]: ctx.dps = 15;  print(repr(ctx.fsum(A)), end=', ')
10.5, mpf('10.5'), mpi('10.5', '10.5'), Decimal('10.5'), mpfr('10.5'), arb3_t('10.5'),
```

7.23.7 fprod: Product of a finite number of factors

`ctx.`**`fprod`**(*factors*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`. See also Mpmath [678].

Calculates a product containing a finite number of factors (for infinite products, see *nprod()*).

The factors will be converted to mpmath numbers.

```
>>> from xlcalcnet import fp, mp, iv, dp, gp, ap
>>> A = [1, 2, 0.5, 7]
>>> for ctx in [fp, mp, iv, dp, gp, ap]: ctx.dps = 15;  print(repr(ctx.fprod(A)), end=', ')
7.0, mpf('7.0'), mpi('7.0', '7.0'), Decimal('7.0'), mpfr('7.0'), arb3_t('7.0'),

>>> A = [1001, 2003, 0.5, 70005]
>>> for ctx in [fp, mp, iv, dp, gp, ap]: ctx.dps = 15;  print(repr(ctx.fprod(A)))
70180117507.5
mpf('70180117507.5')
mpi('70180117507.5', '70180117507.5')
Decimal('70180117507.5')
mpfr('70180117507.5')
arb3_t('70180117507.5')
```

7.23.8 fdot: Dot product

`ctx.`**`fdot`**(*A*, *B=None*, *conjugate=False*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`. See also Mpmath [671].

!!! conjugate not working with iv !!!

Computes the dot product of the iterables A and B,

$$\sum_{k=0} A_k B_k.$$

Alternatively, *fdot()* accepts a single iterable of pairs. In other words, `fdot(A,B)` and `fdot(zip(A,B))` are equivalent. The elements are automatically converted to mpmath numbers.

With `conjugate=True`, the elements in the second vector will be conjugated:

$$\sum_{k=0} A_k \overline{B_k}$$

Examples

```
>>> from xlcalcnet import fp, mp, iv, dp, gp, ap
>>> A = [2, 1.5, 3]; B = [1, -1, 2]
>>> for ctx in [fp, mp, iv, dp, gp, ap]: ctx.dps = 15;  print(repr(ctx.fdot(A, B)), end=', ')
6.5, mpf('6.5'), mpi('6.5', '6.5'), Decimal('6.5'), mpfr('6.5'), arb3_t('6.5'),

>>> C = list(zip(A, B)); print(C)
[(2, 1), (1.5, -1), (3, 2)]
>>> for ctx in [fp, mp, iv, dp, gp, ap]: ctx.dps = 15;  print(repr(ctx.fdot(C)), end=', ')
6.5, mpf('6.5'), mpi('6.5', '6.5'), Decimal('6.5'), mpfr('6.5'), arb3_t('6.5'),

>>> A = [2, 1.5, 3j]; B = [1+1j, 3, -1-1j]
>>> for ctx in [fp, mp, iv, dp, gp, ap]: ctx.dps = 15;  print(repr(ctx.fdot(A, B)))
(9.5-1j)
mpc(real='9.5', imag='-1.0')
iv.mpc(mpi('9.5', '9.5'), mpi('-1.0', '-1.0'))
DecCplx('9.50 - 1.0j')
mpc('9.5-1.0j')
acb3_t('9.5 - 1.0j')

>>> A = [2, 1.5, 3j]; B = [1+1j, 3, -1-1j]
>>> for ctx in [fp, mp, dp, gp, ap]:
...     ctx.dps = 15;  print(repr(ctx.fdot(A, B, conjugate=True)))
(3.5-5j)
mpc(real='3.5', imag='-5.0')
DecCplx('3.50 - 5.0j')
mpc('3.5-5.0j')
acb3_t('3.5 - 5.0j')


>>> x = ap.mpf(2); x
>>> c = 3+4j; c
>>> x + c
```

7.24 Numerical transformations and descriptive statistics

7.24.1 Centering of a matrix

mat.**centered**(*res*, *data*, *population*, *opt*)

Returns the centered version of the matrix

```
>>> from arbeigenlab import mp14
>>> ctx = mp14.drf(); mp14.setdps(15)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,

>>> B = A.centered(); B.show("B")
B:
-25, -25, -25, -25, -25, -25,
-15, -15, -15, -15, -15, -15,
 -5,  -5,  -5,  -5,  -5,  -5,
  5,   5,   5,   5,   5,   5,
 15,  15,  15,  15,  15,  15,
 25,  25,  25,  25,  25,  25,

>>> B = A.variance(); B.show("B")
B:
350, 350, 350, 350, 350, 350,


>>> B = A.stdev(); B.show("B")
```

7.24.2 Standardization of a matrix

mat.**standardized**(*res*, *data*, *population*, *opt*)

Returns the standardized version of the matrix

```
>>> from arbeigenlab import mp14
>>> ctx = mp14.drf(); mp14.setdps(5)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,

>>> B = A.standardized(); B.show("B")
B:
 -1.336,  -1.336,  -1.336,  -1.336,  -1.336,  -1.336,
-0.8017, -0.8017, -0.8017, -0.8017, -0.8017, -0.8017,
-0.2672, -0.2672, -0.2672, -0.2672, -0.2672, -0.2672,
 0.2672,  0.2672,  0.2672,  0.2672,  0.2672,  0.2672,
 0.8017,  0.8017,  0.8017,  0.8017,  0.8017,  0.8017,
  1.336,   1.336,   1.336,   1.336,   1.336,   1.336,


>>> C = B.variance(); C.show("C")
C:
0.9996, 0.9996, 0.9996, 0.9996, 0.9996, 0.9996,
```

See also: https://en.m.wikipedia.org/wiki/Quantile

Hyndman, 1996.

The quantile functions are not working:

https://numpy.org/doc/stable/reference/generated/numpy.ptp.html#numpy.ptp

https://numpy.org/doc/stable/reference/generated/numpy.percentile.html#numpy.percentile

https://numpy.org/doc/stable/reference/generated/numpy.nanpercentile.html#numpy.nanpercentile

https://numpy.org/doc/stable/reference/generated/numpy.quantile.html#numpy.quantile

https://numpy.org/doc/stable/reference/generated/numpy.nanquantile.html#numpy.nanquantile

The correlation functions are not working:

https://numpy.org/doc/stable/reference/generated/numpy.corrcoef.html#numpy.corrcoef

https://numpy.org/doc/stable/reference/generated/numpy.correlate.html#numpy.correlate

https://numpy.org/doc/stable/reference/generated/numpy.cov.html#numpy.cov

See also Eigen [237].

7.24.3 Trace

`mat.trace()`

Returns the trace of the matrix. See also: Wikipedia [1393].

```
>>> from arbeigenlab import mp14
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,

>>> Res = A.trace(); Res.show("Res")

!!! MISSING !!!
```

7.24.4 Squared Norm

`mat.squaredNorm(`*partialmode=full*`)`

Returns the squared norm of the matrix.

```
>>> from arbeigenlab import mp14
>>> ctx = mp14.drf()
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,

>>> Res = A.squaredNorm(); Res.show("Res")
Res:
 9526,  9964, 10414, 10876, 11350, 11836,
```

7.24.5 Euclidian Norm

mat.**Norm**(*partialmode=full*)

Returns the norm of the matrix.

```
>>> from arbeigenlab import mp14
>>> ctx = mp14.drf(); mp14.setdps(6)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,

>>> Res = A.Norm(); Res.show("Res")
Res:
97.601, 99.820, 102.05, 104.29, 106.54, 108.79,
```

7.24.6 Covariance matrix

mat.**covariance_matrix**(*use_crossproduct=False*)

Returns the covariance matrix of the matrix. See also Wikipedia [1389], Wikipedia [1369].

```
>>> from arbeigenlab import mp14
>>> ctx = mp14.drf(); mp14.setdps(15)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,

>>> B = A.centered(); B.show("B")
B:
-25, -25, -25, -25, -25, -25,
-15, -15, -15, -15, -15, -15,
 -5,  -5,  -5,  -5,  -5,  -5,
  5,   5,   5,   5,   5,   5,
 15,  15,  15,  15,  15,  15,
 25,  25,  25,  25,  25,  25,

>>> C = (B.T * B)/(B.cols-1); C.show("C")
C:
350, 350, 350, 350, 350, 350,
350, 350, 350, 350, 350, 350,
350, 350, 350, 350, 350, 350,
350, 350, 350, 350, 350, 350,
350, 350, 350, 350, 350, 350,
350, 350, 350, 350, 350, 350,

>>> D = A.covariance(); D.show("D")
D:
350, 350, 350, 350, 350, 350,
350, 350, 350, 350, 350, 350,
350, 350, 350, 350, 350, 350,
350, 350, 350, 350, 350, 350,
350, 350, 350, 350, 350, 350,
350, 350, 350, 350, 350, 350,
```

7.24.7 Correlation matrix

mat.**correlation**(*use_crossproduct=False*)

Returns the correlation matrix of matrix matA. See also Wikipedia [1367].

```
>>> from arbeigenlab import mp14
>>> ctx = mp14.drf(); mp14.setdps(6)
>>> A = ctx.read_from_sqlite(mp14.dbpath(), "DecTableA6x6", ""); A.show("A")
A:
11, 12, 13, 14, 15, 16,
21, 22, 23, 24, 25, 26,
31, 32, 33, 34, 35, 36,
41, 42, 43, 44, 45, 46,
51, 52, 53, 54, 55, 56,
61, 62, 63, 64, 65, 66,

>>> B = A.standardized(); B.show("B")
B:
 -1.3363,  -1.3363,  -1.3363,  -1.3363,  -1.3363,  -1.3363,
-0.80180, -0.80180, -0.80180, -0.80180, -0.80180, -0.80180,
-0.26727, -0.26727, -0.26727, -0.26727, -0.26727, -0.26727,
 0.26727,  0.26727,  0.26727,  0.26727,  0.26727,  0.26727,
 0.80180,  0.80180,  0.80180,  0.80180,  0.80180,  0.80180,
  1.3363,   1.3363,   1.3363,   1.3363,   1.3363,   1.3363,

>>> C = (B.T * B)/(B.cols-1); C.show("C")
C:
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,


>>> D = A.correlation(); D.show("D")
D:
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
```

7.24.8 Summary statistic

mat.**summary**(*summary*, *data*, *population*, *opt*)

Sets *summary* to a matrix containing summary statistics of *data*.

Constant	Meaning
ARB_STAT_COUNT	1
ARB_STAT_SUM	4 = $\sum_0^1 07^e + 11^f$
ARB_STAT_MEAN	4
ARB_STAT_MIN	13
ARB_STAT_MEDIAN	1
ARB_STAT_MAX	11
ARB_STAT_AVERAGE_DEVIATION	2
ARB_STAT_SUM_OF_SQUARES_OF_DEV	14
ARB_STAT_SUM_OF_SQUARES	8
ARB_STAT_VARIANCE	11
ARB_STAT_STANDARD_DEVIATION	6
ARB_STAT_SKEWNESS	6
ARB_STAT_KURTOSIS	8
ARB_STAT_TRIMMED_MEAN	8
ARB_STAT_HARMONIC_MEAN	9
ARB_STAT_GEOMETRIC_MEAN	9

```
>>> from arbeigenlab import mp14
>>> mpm.dps = 40;
>>> A = mp14.xrf().read_from_sqlite(mp14.dbpath(), "MpfrTableA4x4", "")
>>> B = mp14.xrf().read_from_sqlite(mp14.dbpath(), "MpfrTableB4x4", "")
```

7.24.9 Vector norm of a matrix

`ctx.norm`(*x*, *p=2*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Gives the entrywise p-norm of an iterable x, i.e. the vector norm $(\sum_k |x_k|^p)^{1/p}$, for any given $1 \le p \le \infty$.

Special cases:

If x is not iterable, this just returns `absmax(x)`.

`p=1` gives the sum of absolute values.

`p=2` is the standard Euclidean vector norm.

`p=inf` gives the magnitude of the largest element.

For x a matrix, `p=2` is the Frobenius norm. For operator matrix norms, use `mnorm()` instead.

You can use the string 'inf' as well as float('inf') or mpf('inf') to specify the infinity norm.

Examples

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = False
>>> x = matrix([-10, 2, 100])
>>> norm(x, 1)
mpf('112.0')
>>> norm(x, 2)
mpf('100.5186549850325')
>>> norm(x, inf)
mpf('100.0')
```

7.24.10 Matrix norm

`ctx.mnorm(`*A*, *p=1*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Gives the matrix (operator) p-norm of A. Currently `p=1` and `p=inf` are supported:

`p=1` gives the 1-norm (maximal column sum)

`p=inf` gives the ∞-norm (maximal row sum). You can use the string 'inf' as well as float('inf') or mpf('inf')

`p=2` (not implemented) for a square matrix is the usual spectral matrix norm, i.e. the largest singular value.

`p='f'` (or 'F', 'fro', 'Frobenius, 'frobenius') gives the Frobenius norm, which is the elementwise 2-norm. The Frobenius norm is an approximation of the spectral norm and satisfies

$$\frac{1}{\sqrt{\operatorname{rank}(A)}}\|A\|_F \leq \|A\|_2 \leq \|A\|_F$$

The Frobenius norm lacks some mathematical properties that might be expected of a norm.

For general elementwise p-norms, use `norm()` instead.

Examples

```
>>> from xlcalcnet import *
>>> mp.dps = 15; mp.pretty = False
>>> A = matrix([[1, -1000], [100, 50]])
>>> mnorm(A, 1)
mpf('1050.0')
>>> mnorm(A, inf)
mpf('1001.0')
>>> mnorm(A, 'F')
mpf('1006.2310867787777')
```

7.25 Standard decompositions and linear solving

Returns the Cholesky decomposition of the symmetric matrix *matA* $= A = PLU$, without pivoting. Here *matA* is an instance of one of the above classes.

See also Eigen [231], Wikipedia [1400], Wikipedia [1430].

7.25.1 Cholesky decomposition

`ctx.cholesky`(*A*, *tol=None*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Cholesky decomposition of a symmetric positive-definite matrix A. Returns a lower triangular matrix L such that $A = L \times L^T$. More generally, for a complex Hermitian positive-definite matrix, a Cholesky decomposition satisfying $A = L \times L^H$ is returned.

The Cholesky decomposition can be used to solve linear equation systems twice as efficiently as LU decomposition, or to test whether A is positive-definite.

The optional parameter `tol` determines the tolerance for verifying positive-definiteness.

Examples

Cholesky decomposition of a positive-definite symmetric matrix:

```
>>> from xlcalcnet import *
>>> mp.dps = 25; mp.pretty = True
>>> A = eye(3) + hilbert(3)
>>> nprint(A)
[     2.0      0.5  0.333333]
[     0.5  1.33333      0.25]
[0.333333     0.25       1.2]
>>> L = cholesky(A)
>>> nprint(L)
[ 1.41421      0.0      0.0]
[0.353553  1.09924      0.0]
[0.235702  0.15162  1.05899]
>>> chop(A - L*L.T)
[0.0  0.0  0.0]
[0.0  0.0  0.0]
[0.0  0.0  0.0]
```

Cholesky decomposition of a Hermitian matrix:

```
>>> A = eye(3) + matrix([[0,0.25j,-0.5j],[-0.25j,0,0],[0.5j,0,0]])
>>> L = cholesky(A)
>>> nprint(L)
[          1.0                0.0                0.0]
[(0.0 - 0.25j)  (0.968246 + 0.0j)                0.0]
[ (0.0 + 0.5j)  (0.129099 + 0.0j)  (0.856349 + 0.0j)]
>>> chop(A - L*L.H)
[0.0  0.0  0.0]
[0.0  0.0  0.0]
[0.0  0.0  0.0]
```

Attempted Cholesky decomposition of a matrix that is not positive definite:

```
>>> A = -eye(3) + hilbert(3)
>>> L = cholesky(A)
Traceback (most recent call last):
  ...
ValueError: matrix is not positive-definite
```

References

1. Wikipedia: http://en.wikipedia.org/wiki/Cholesky_decomposition

7.25.2 Cholesky decomposition, solve

`ctx.cholesky_solve(A, b, **kwargs)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Solves a symmetric positive-definite linear equation system. This is twice as efficient as lu_solve.

Returns the LU decomposition of the general square matrix *matA* $= A = PLU$, with partial pivoting.

See also Eigen [232], Wikipedia [1412], Wikipedia [1430].

7.25.3 Matrix LU factorization

`ctx.lu(A)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

A -> P, L, U

LU factorisation of a square matrix A. L is the lower, U the upper part. P is the permutation matrix indicating the row swaps.

P*A = L*U

If you need efficiency, use the low-level method LU_decomp instead, it's much more memory efficient.

The function `lu` computes an explicit LU factorization of a matrix:

```
>>> P, L, U = lu(matrix([[0,2,3],[4,5,6],[7,8,9]]))
>>> print(P)
[0.0  0.0  1.0]
[1.0  0.0  0.0]
[0.0  1.0  0.0]
>>> print(L)
[              1.0                0.0  0.0]
[              0.0                1.0  0.0]
[0.571428571428571  0.214285714285714  1.0]
>>> print(U)
[7.0  8.0                9.0]
[0.0  2.0                3.0]
[0.0  0.0  0.214285714285714]
>>> print(P.T*L*U)
[0.0  2.0  3.0]
[4.0  5.0  6.0]
[7.0  8.0  9.0]
```

7.25.4 Determinant of a matrix, using LU decomposition

`ctx.det(A)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the determinant of a matrix, using the LU factorization.

7.25.5 Inverse of a matrix, using the LU factorization

`ctx.inverse(A, **kwargs)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the inverse of a matrix, using the LU factorization.

If you want to solve an equation system Ax = b, it's recommended to use solve(A, b) instead, it's about 3 times more efficient.

7.25.6 Linear equations: LU solve

`ctx.lu_solve`(*A*, *b*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Basic linear algebra is implemented; you can for example solve the linear equation system:

```
  x + 2*y = -10
3*x + 4*y =  10
```

using `lu_solve`:

```
>>> from xlcalcnet import *
>>> mp.pretty = False
>>> A = matrix([[1, 2], [3, 4]])
>>> b = matrix([-10, 10])
>>> x = lu_solve(A, b)
>>> x
matrix(
[['30.0'],
 ['-20.0']])
```

7.25.7 Linear equations: residual of LU solve

`ctx.residual`(*A*, *x*, *b*, ***kwargs*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculate the residual of a solution to a linear equation system.

r = A*x - b for A*x = b

If you don't trust the result, use `residual` to calculate the residual ||A*x-b||:

```
>>> residual(A, x, b)
matrix(
[['3.46944695195361e-18'],
 ['3.46944695195361e-18']])
>>> str(eps)
'2.22044604925031e-16'
```

As you can see, the solution is quite accurate. The error is caused by the inaccuracy of the internal floating point arithmetic. Though, it's even smaller than the current machine epsilon, which basically means you can trust the result.

`lu_solve` accepts overdetermined systems. It is usually not possible to solve such systems, so the residual is minimized instead. Internally this is done using Cholesky decomposition to compute a least squares approximation. This means that `lu_solve` will square the errors. If you can't afford this, use `qr_solve` instead. It is twice as slow but more accurate, and it calculates the residual automatically.

7.25.8 ??? LU improve solution

`improve_solution`(*ctx*, *A*, *x*, *b*, *maxsteps=1*)

Improve a solution to a linear equation system iteratively.

This re-uses the LU decomposition and is thus cheap. Usually 3 up to 4 iterations are giving the maximal improvement.

7.25.9 mpmath: LU condition number

`ctx.cond`(*A*, *norm=None*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Calculates the condition number of a matrix using a specified matrix norm.

The condition number estimates the sensitivity of a matrix to errors. Example: small input errors for ill-conditioned coefficient matrices alter the solution of the system dramatically.

For ill-conditioned matrices it's recommended to use qr_solve() instead of lu_solve(). This does not help with input errors however, it just avoids to add additional errors.

Definition: cond(A) = ||A|| * ||A**-1||

Returns the QR decomposition of the symmetric matrix *matA* $= A = QR$, without pivoting.

See also Eigen [228], Wikipedia [1434], Wikipedia [1430].

7.25.10 QR factorization

`ctx.qr`(*A*, *mode='full'*, *edps=10*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Compute a QR factorization $A = QR$ where A is an m x n matrix of real or complex numbers where m >= n

mode has following meanings: (1) mode = 'raw' returns two matrixes (A, tau) in the internal format used by LAPACK (2) mode = 'skinny' returns the leading n columns of Q and n rows of R (3) Any other value returns the leading m columns of Q and m rows of R

edps is the increase in mp precision used for calculations

Examples

```
>>> from xlcalcnet import *
>>> mp.dps = 15
>>> mp.pretty = True
>>> A = matrix([[1, 2], [3, 4], [1, 1]])
>>> Q, R = qr(A)
>>> Q
[-0.301511344577764   0.861640436855329   0.408248290463863]
[-0.904534033733291  -0.123091490979333  -0.408248290463863]
[-0.301511344577764  -0.492365963917331   0.816496580927726]
>>> R
[-3.3166247903554  -4.52267016866645]
[             0.0  0.738548945875996]
[             0.0                0.0]
>>> Q * R
[1.0  2.0]
[3.0  4.0]
[1.0  1.0]
>>> chop(Q.T * Q)
[1.0  0.0  0.0]
[0.0  1.0  0.0]
[0.0  0.0  1.0]
>>> B = matrix([[1+0j, 2-3j], [3+j, 4+5j]])
>>> Q, R = qr(B)
>>> nprint(Q)
[     (-0.301511 + 0.0j)   (0.0695795 - 0.95092j)]
[(-0.904534 - 0.301511j)  (-0.115966 + 0.278318j)]
>>> nprint(R)
[(-3.31662 + 0.0j)  (-5.72872 - 2.41209j)]
[              0.0       (3.91965 + 0.0j)]
>>> Q * R
[(1.0 + 0.0j)  (2.0 - 3.0j)]
[(3.0 + 1.0j)  (4.0 + 5.0j)]
>>> chop(Q.T * Q.conjugate())
[1.0  0.0]
[0.0  1.0]
```

7.25.11 QR solve

`ctx.qr_solve`(*A*, *b*, *norm=None*, ***kwargs*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Ax = b => x, ||Ax - b||

Solve a determined or overdetermined linear equations system and calculate the norm of the residual (error). QR decomposition using Householder factorization is applied, which gives very accurate results even for ill-conditioned matrices. lu_solve is twice as efficient.

7.26 Singular Value and Eigen decompositions

See: https://math.stackexchange.com/questions/1816364/the-svd-solution-to-linear-least-squares-linear-system-of-equations

Returns the SVD decomposition of a rectangular matrix.

See also Eigen [229], Wikipedia [1418], Wikipedia [1430].

7.26.1 Real singular value decomposition of a matrix A

`ctx.svd_r`(*A*, *full_matrices=False*, *compute_uv=True*, *overwrite_a=False*)

where `ctx` is `dec`, `mpm`, `fpm``or ``gmp`.

This routine computes the singular value decomposition of a matrix A. Given A, two orthogonal matrices U and V are calculated such that

A = U S V and U' U = 1 and V V' = 1

where S is a suitable shaped matrix whose off-diagonal elements are zero. Here ' denotes the transpose. The diagonal elements of S are the singular values of A, i.e. the positive square roots of the eigenvalues of A' A or A A'.

input:

A : a real matrix of shape (m, n)

full_matrices
: [if true, U and V are of shape (m, m) and (n, n).] if false, U and V are of shape (m, min(m, n)) and (min(m, n), n).

compute_uv : if true, U and V are calculated. if false, only S is calculated.

overwrite_a
: [if true, allows modification of A which may improve] performance. if false, A is not modified.

output:

U
: [an orthogonal matrix: U' U = 1. if full_matrices is true, U is of] shape (m, m). ortherwise it is of shape (m, min(m, n)).

S
: [an array of length min(m, n) containing the singular values of A sorted by] decreasing magnitude.

V
: [an orthogonal matrix: V V' = 1. if full_matrices is true, V is of] shape (n, n). ortherwise it is of shape (min(m, n), n).

return value:

S if compute_uv is false

(U, S, V) if compute_uv is true

overview of the matrices:

full_matrices true:

A : m*n

U : m*m U' U = 1

S as matrix : m*n

V : n*n V V' = 1

full_matrices false:

A : m*n

U : m*min(n,m) U' U = 1

S as matrix : min(m,n)*min(m,n)

V : min(m,n)*n V V' = 1

examples:

```
>>> from mpmath import mp
>>> A = mp.matrix([[2, -2, -1], [3, 4, -2], [-2, -2, 0]])
>>> S = mp.svd_r(A, compute_uv = False)
>>> print(S)
[6.0]
[3.0]
[1.0]
```

```
>>> U, S, V = mp.svd_r(A)
>>> print(mp.chop(A - U * mp.diag(S) * V))
[0.0  0.0  0.0]
[0.0  0.0  0.0]
[0.0  0.0  0.0]
```

see also: svd, svd_c

7.26.2 Complex singular value decomposition of a matrix A

`ctx.svd_c`(*A*, *full_matrices=False*, *compute_uv=True*, *overwrite_a=False*)

where `ctx` is `dec`, `mpm`, `fpm``or ``gmp`.

This routine computes the singular value decomposition of a matrix A. Given A, two unitary matrices U and V are calculated such that

A = U S V and U' U = 1 and V V' = 1

where S is a suitable shaped matrix whose off-diagonal elements are zero. Here ' denotes the hermitian transpose (i.e. transposition and complex conjugation). The diagonal elements of S are the singular values of A, i.e. the positive square roots of the eigenvalues of A' A or A A'.

input:

A : a complex matrix of shape (m, n)

full_matrices : if true, U and V are of shape (m, m) and (n, n).

if false, U and V are of shape (m, min(m, n)) and (min(m, n), n).

compute_uv : if true, U and V are calculated. if false, only S is calculated.

overwrite_a
[if true, allows modification of A which may improve] performance. if false, A is not modified.

output:

U
[an unitary matrix: U' U = 1. if full_matrices is true, U is of] shape (m, m). ortherwise it is of shape (m, min(m, n)).

S
[an array of length min(m, n) containing the singular values of A sorted by] decreasing magnitude.

V
[an unitary matrix: V V' = 1. if full_matrices is true, V is of] shape (n, n). ortherwise it is of shape (min(m, n), n).

return value:

S if compute_uv is false

(U, S, V) if compute_uv is true

overview of the matrices:

full_matrices true:

A : m*n

U : m*m U' U = 1

S as matrix : m*n

V : n*n V V' = 1

full_matrices false:

A : m*n

U : m*min(n,m) U' U = 1

S as matrix : min(m,n)*min(m,n)

V : min(m,n)*n V V' = 1

example:

```
>>> from mpmath import mp
>>> A = mp.matrix([[-2j, -1-3j, -2+2j], [2-2j, -1-3j, 1], [-3+1j,-2j,0]])
>>> S = mp.svd_c(A, compute_uv = False)
>>> print(mp.chop(S - mp.matrix([mp.sqrt(34), mp.sqrt(15), mp.sqrt(6)])))
[0.0]
[0.0]
[0.0]
```

```
>>> U, S, V = mp.svd_c(A)
>>> print(mp.chop(A - U * mp.diag(S) * V))
[0.0  0.0  0.0]
[0.0  0.0  0.0]
[0.0  0.0  0.0]
```

see also: svd, svd_r

7.26.3 mpmath: Singular value decomposition of a matrix A (real or complex)

`ctx.svd`(*A*, *full_matrices=False*, *compute_uv=True*, *overwrite_a=False*)

where `ctx` is `dec`, `mpm`, `fpm``or ``gmp`.

"svd" is a unified interface for "svd_r" and "svd_c". Depending on whether A is real or complex the appropriate function is called.

This routine computes the singular value decomposition of a matrix A. Given A, two orthogonal (A real) or unitary (A complex) matrices U and V are calculated such that

A = U S V and U' U = 1 and V V' = 1

where S is a suitable shaped matrix whose off-diagonal elements are zero. Here ' denotes the hermitian transpose (i.e. transposition and complex conjugation). The diagonal elements of S are the singular values of A, i.e. the squareroots of the eigenvalues of A' A or A A'.

input:

A : a real or complex matrix of shape (m, n)

full_matrices : if true, U and V are of shape (m, m) and (n, n). if false, U and V are of shape (m, min(m, n)) and (min(m, n), n).

compute_uv : if true, U and V are calculated. if false, only S is calculated.

overwrite_a : if true, allows modification of A which may improve performance. if false, A is not modified.

output:

U

[an orthogonal or unitary matrix: U' U = 1. if full_matrices is true, U is of] shape (m, m). ortherwise it is of shape (m, min(m, n)).

S

[an array of length min(m, n) containing the singular values of A sorted by] decreasing magnitude.

V

[an orthogonal or unitary matrix: V V' = 1. if full_matrices is true, V is of] shape (n, n). ortherwise it is of shape (min(m, n), n).

return value:

S if compute_uv is false

(U, S, V) if compute_uv is true

overview of the matrices:

full_matrices true: A : m*n U : m*m U' U = 1 S as matrix : m*n V : n*n V V' = 1

full_matrices false: A : m*n U : m*min(n,m) U' U = 1 S as matrix : min(m,n)*min(m,n) V : min(m,n)*n V V' = 1

examples:

```
>>> from mpmath import mp
>>> A = mp.matrix([[2, -2, -1], [3, 4, -2], [-2, -2, 0]])
>>> S = mp.svd(A, compute_uv = False)
>>> print(S)
[6.0]
[3.0]
[1.0]
```

```
>>> U, S, V = mp.svd(A)
>>> print(mp.chop(A - U * mp.diag(S) * V))
[0.0  0.0  0.0]
[0.0  0.0  0.0]
[0.0  0.0  0.0]
```

see also: svd_r, svd_c

7.26.4 Eigenvalue problem for a real symmetric square matrix A

`ctx.eigsy`(*d*, *e*, *z=False*)

where `ctx` is `dec`, `mpm`, `fpm``or ``gmp`.

Returns the eigendecomposition of the symmetric/hermitian matrix *matA* $= A$.

See also Eigen [238], Wikipedia [1403], Wikipedia [1405], Wikipedia [1430].

This routine solves the (ordinary) eigenvalue problem for a real symmetric square matrix A. Given A, an orthogonal matrix Q is calculated which diagonalizes A : $Q' A Q = \mathrm{diag}(E)$ and $Q Q^T = Q^T Q = 1$

Here diag(E) is a diagonal matrix whose diagonal is E.

The columns of Q are the eigenvectors of A and E contains the eigenvalues:

```
A * Q[:,i] = E[i] * Q[:,i]
```

input:

A: real matrix of format (n,n) which is symmetric (i.e. A=A' or A[i,j]=A[j,i])

eigvals_only: if true, calculates only the eigenvalues E. if false, calculates both eigenvectors and eigenvalues.

overwrite_a: if true, allows modification of A which may improve performance. if false, A is not modified.

output:

E: vector of format (n). contains the eigenvalues of A in ascending order.

Q: orthogonal matrix of format (n,n). contains the eigenvectors of A as columns.

return value:

E if eigvals_only is true

(E, Q) if eigvals_only is false

example:

```
>>> from mpmath import mp
>>> A = mp.matrix([[3, 2], [2, 0]])
>>> E = mp.eigsy(A, eigvals_only = True)
>>> print(E)
[-1.0]
[ 4.0]
```

```
>>> A = mp.matrix([[1, 2], [2, 3]])
>>> E, Q = mp.eigsy(A)
>>> print(mp.chop(A * Q[:,0] - E[0] * Q[:,0]))
[0.0]
[0.0]
```

see also: eighe, eigh, eig

7.26.5 Eigenvalue problem for a complex hermitian square matrix A

`ctx.eighe`(*A*, *eigvals_only=False*, *overwrite_a=False*)

where `ctx` is `dec`, `mpm`, `fpm``or ``gmp`.

This routine solves the (ordinary) eigenvalue problem for a complex hermitian square matrix A. Given A, an unitary matrix Q is calculated which diagonalizes A:

Q' A Q = diag(E) and Q Q' = Q' Q = 1

Here diag(E) a is diagonal matrix whose diagonal is E. ' denotes the hermitian transpose (i.e. ordinary transposition and complex conjugation).

The columns of Q are the eigenvectors of A and E contains the eigenvalues:

A Q[:,i] = E[i] Q[:,i]

input:

A: complex matrix of format (n,n) which is hermitian
(i.e. A=A' or A[i,j]=conj(A[j,i]))

eigvals_only: if true, calculates only the eigenvalues E.
if false, calculates both eigenvectors and eigenvalues.

overwrite_a: if true, allows modification of A which may improve
performance. if false, A is not modified.

output:

E: vector of format (n). contains the eigenvalues of A in ascending order.

Q: unitary matrix of format (n,n). contains the eigenvectors
of A as columns.

return value:

E if eigvals_only is true

(E, Q) if eigvals_only is false

example:

```
>>> from mpmath import mp
>>> A = mp.matrix([[1, -3 - 1j], [-3 + 1j, -2]])
>>> E = mp.eighe(A, eigvals_only = True)
>>> print(E)
[-4.0]
[ 3.0]
```

```
>>> A = mp.matrix([[1, 2 + 5j], [2 - 5j, 3]])
>>> E, Q = mp.eighe(A)
>>> print(mp.chop(A * Q[:,0] - E[0] * Q[:,0]))
[0.0]
[0.0]
```

see also: eigsy, eigh, eig

7.26.6 mpmath: Eigenvalue problem for a selfadjoint square matrix A

`ctx.eigh`(*A*, *eigvals_only=False*, *overwrite_a=False*)

where `ctx` is `dec`, `mpm`, `fpm``or ``gmp`.

“eigh” is a unified interface for “eigsy” and “eighe”. Depending on whether A is real or complex the appropriate function is called.

This routine solves the (ordinary) eigenvalue problem for a real symmetric or complex hermitian square matrix A. Given A, an orthogonal (A real) or unitary (A complex) matrix Q is calculated which diagonalizes A:

Q’ A Q = diag(E) and Q Q’ = Q’ Q = 1

Here diag(E) a is diagonal matrix whose diagonal is E. ‘ denotes the hermitian transpose (i.e. ordinary transposition and complex conjugation).

The columns of Q are the eigenvectors of A and E contains the eigenvalues:

A Q[:,i] = E[i] Q[:,i]

input:

A: a real or complex square matrix of format (n,n) which is symmetric
(i.e. A[i,j]=A[j,i]) or hermitian (i.e. A[i,j]=conj(A[j,i])).

eigvals_only: if true, calculates only the eigenvalues E.
if false, calculates both eigenvectors and eigenvalues.

overwrite_a: if true, allows modification of A which may improve
performance. if false, A is not modified.

output:

E: vector of format (n). contains the eigenvalues of A in ascending order.

Q: an orthogonal or unitary matrix of format (n,n). contains the
eigenvectors of A as columns.

return value:

E if eigvals_only is true

(E, Q) if eigvals_only is false

example:

```
>>> from mpmath import mp
>>> A = mp.matrix([[3, 2], [2, 0]])
>>> E = mp.eigh(A, eigvals_only = True)
>>> print(E)
[-1.0]
[ 4.0]
```

```
>>> A = mp.matrix([[1, 2], [2, 3]])
>>> E, Q = mp.eigh(A)
>>> print(mp.chop(A * Q[:,0] - E[0] * Q[:,0]))
[0.0]
[0.0]
```

```
>>> A = mp.matrix([[1, 2 + 5j], [2 - 5j, 3]])
>>> E, Q = mp.eigh(A)
>>> print(mp.chop(A * Q[:,0] - E[0] * Q[:,0]))
[0.0]
[0.0]
```

see also: eigsy, eighe, eig

7.26.7 mpmath: tridiag_sym

r_sy_tridiag(*ctx*, *A*, *D*, *E*, *calc_ev=True*)

This routine transforms a real symmetric matrix A to a real symmetric tridiagonal matrix S using an orthogonal similarity transformation:

$$Q^T A Q = S$$

Returns the tridiagonal decomposition of a selfadjoint matrix. See also Eigen [239], Wikipedia [1420], Wikipedia [1410], Wikipedia [1405], Wikipedia [1430].

The orthogonal matrix Q is build up from Householder reflectors.

parameters:

A: On input, A contains the real symmetric matrix of dimension (n, n). On output, if calc_ev is true, A contains the orthogonal matrix Q, otherwise A is destroyed.

D: Returns a real array of length n, containing the diagonal elements of the tridiagonal matrix.

E: Returns a real array of length n, contains the offdiagonal elements of the tridiagonal matrix in ''E[0:(n-1)]" where ''n" is the dimension of the matrix A. ''E[n-1]" is undefined.

calc_ev: If calc_ev is true, this routine explicitly calculates the orthogonal matrix Q which is then returned in A. If calc_ev is false, Q is not explicitly calculated resulting in a shorter run time.

This routine is a python translation of the fortran routine tred2.f in the software library EISPACK (see netlib.org) which itself is based on the algol procedure tred2 described in:

Num. Math. 11, p.181-195 (1968) by Martin, Reinsch and Wilkonson

Handbook for auto. comp., Vol II, Linear Algebra, p.212-226 (1971)

For a good introduction to Householder reflections, see also Stoer, Bulirsch - Introduction to Numerical Analysis.

7.26.8 mpmath tridiag_her

c_he_tridiag_0(*ctx*, *A*, *D*, *E*, *S*)

This routine transforms a complex hermitian matrix A to a real symmetric tridiagonal matrix S using an unitary similarity transformation:

$$Q^H A Q = S$$

where Q^H denotes the hermitian matrix transpose, i.e. transposition und conjugation.The unitary matrix Q is build up from Householder reflectors and an unitary diagonal matrix.

parameters:

A: (input/output) On input, A contains the complex hermitian matrix of dimension (n,n). On output, A contains the unitary matrix Q in compressed form.

D: Returns a real array of length n, contains the diagonal elements of the tridiagonal matrix.

E: Returns a real array of length n, contains the offdiagonal elements of the tridiagonal matrix in E[0:(n-1)] where is the dimension of the matrix A. E[n-1] is undefined.

S: Returns a complex array of length n, contains a unitary diagonal matrix.

This routine is a python translation (in slightly modified form) of the fortran routine htridi.f in the software library EISPACK (see netlib.org) which itself is a complex version of the algol procedure tred1 described in:

Num. Math. 11, p.181-195 (1968) by Martin, Reinsch and Wilkonson Handbook for auto. comp., Vol II, Linear Algebra, p.212-226 (1971)

For a good introduction to Householder reflections, see also Stoer, Bulirsch - Introduction to Numerical Analysis.

7.26.9 mpmath: tridiag_eigen_sym

tridiag_eigen(*ctx*, *d*, *e*, *z=False*)

This subroutine find the eigenvalues and the first components of the eigenvectors of a real symmetric tridiagonal matrix using the implicit QL method.

parameters:

d: (input/output) real array of length n. on input, d contains the diagonal elements of the input matrix. on output, d contains the eigenvalues in ascending order.

e: (input) real array of length n. on input, e contains the offdiagonal elements of the input matrix in e[0:(n-1)]. On output, e has been destroyed.

z: (input/output) If z is equal to False, no eigenvectors will be computed. Otherwise on input z should have the format z[0:m,0:n] (i.e. a real or complex matrix of dimension (m,n)). On output this matrix will be multiplied by the matrix of the eigenvectors (i.e. the columns of this matrix are the eigenvectors): z –> z*EV That means if z[i,j]={1 if j==j; 0 otherwise} on input, then on output z will contain the first m components of the eigenvectors. That means if m is equal to n, the i-th eigenvector will be z[:,i].

This routine is a python translation (in slightly modified form) of the fortran routine imtql2.f in the software library EISPACK (see netlib.org) which itself is based on the algol procudure imtql2 desribed in:

num. math. 12, p. 377-383(1968) by matrin and wilkinson modified in num. math. 15, p. 450(1970) by dubrulle handbook for auto. comp., vol. II-linear algebra, p. 241-248 (1971)

See also the routine gaussq.f in netlog.org or acm algorithm 726.

7.26.10 Eigensystem decomposition of a matrix A (real or complex)

Returns the eigendecomposition of a general square matrix *matA* $= A$. See also Eigen [221], Wikipedia [1403], Wikipedia [1405], Wikipedia [1430].

`ctx.eig`(*A*, *left=False*, *right=True*, *overwrite_a=False*)

where `ctx` is `dec`, `mpm`, `fpm``or ``gmp`.

This routine computes the eigenvalues and optionally the left and right eigenvectors of a square matrix A. Given A, a vector E and matrices ER and EL are calculated such that

A ER[:,i] = E[i] ER[:,i]

EL[i,:] A = EL[i,:] E[i]

E contains the eigenvalues of A. The columns of ER contain the right eigenvectors of A whereas the rows of EL contain the left eigenvectors.

input:

A : a real or complex square matrix of shape (n, n) left : if true, the left eigenvectors are calulated. right : if true, the right eigenvectors are calculated. overwrite_a : if true, allows modification of A which may improve performance. if false, A is not modified.

output:

E : a list of length n containing the eigenvalues of A. ER : a matrix whose columns contain the right eigenvectors of A. EL : a matrix whose rows contain the left eigenvectors of A.

return values:

E if left and right are both false. (E, ER) if right is true and left is false. (E, EL) if left is true and right is false. (E, EL, ER) if left and right are true.

examples:

```
>>> from mpmath import mp
>>> A = mp.matrix([[3, -1, 2], [2, 5, -5], [-2, -3, 7]])
>>> E, ER = mp.eig(A)
>>> print(mp.chop(A * ER[:,0] - E[0] * ER[:,0]))
[0.0]
[0.0]
[0.0]
```

```
>>> E, EL, ER = mp.eig(A,left = True, right = True)
>>> E, EL, ER = mp.eig_sort(E, EL, ER)
>>> mp.nprint(E)
[2.0, 4.0, 9.0]
>>> print(mp.chop(A * ER[:,0] - E[0] * ER[:,0]))
[0.0]
[0.0]
[0.0]
>>> print(mp.chop( EL[0,:] * A - EL[0,:] * E[0]))
[0.0  0.0  0.0]
```

warning:

- If there are multiple eigenvalues, the eigenvectors do not necessarily span the whole vectorspace, i.e. ER and EL may have not full rank. Furthermore in that case the eigenvectors are numerical ill-conditioned.
- In the general case the eigenvalues have no natural order.

see also:

- eigh (or eigsy, eighe) for the symmetric eigenvalue problem.
- eig_sort for sorting of eigenvalues and eigenvectors

7.26.11 Sorting Eigenvalues

`ctx.`**`eig_sort`**(*E*, *EL=False*, *ER=False*, *f='real'*)

where `ctx` is `dec`, `mpm`, `fpm``or ``gmp`.

This routine sorts the eigenvalues and eigenvectors delivered by `eig`.

parameters:

E : the eigenvalues as delivered by eig

EL : the left eigenvectors as delivered by eig, or false

ER : the right eigenvectors as delivered by eig, or false

f : either a string (`real` sort by increasing real part, `imag` sort by increasing imag part, `abs` sort by absolute value) or a function mapping complexs to the reals, i.e. `f = lambda x: -mp.re(x)` would sort the eigenvalues by decreasing real part.

return values:

E if EL and ER are both false.

(E, ER) if ER is not false and left is false.

(E, EL) if EL is not false and right is false.

(E, EL, ER) if EL and ER are not false.

example:

```
>>> from mpmath import mp
>>> A = mp.matrix([[3, -1, 2], [2, 5, -5], [-2, -3, 7]])
>>> E, EL, ER = mp.eig(A,left = True, right = True)
>>> E, EL, ER = mp.eig_sort(E, EL, ER)
>>> mp.nprint(E)
[2.0, 4.0, 9.0]
>>> E, EL, ER = mp.eig_sort(E, EL, ER,f = lambda x: -mp.re(x))
>>> mp.nprint(E)
[9.0, 4.0, 2.0]
>>> print(mp.chop(A * ER[:,0] - E[0] * ER[:,0]))
[0.0]
[0.0]
[0.0]
>>> print(mp.chop( EL[0,:] * A - EL[0,:] * E[0]))
[0.0  0.0  0.0]
```

7.26.12 mpmath: Hessenberg decomposition of a matrix A (real or complex)

`ctx.`**`hessenberg`**(*A*, *overwrite_a=False*)

where `ctx` is `dec`, `mpm`, `fpm``or ``gmp`.

Reduces a square matrix to Hessenberg form by an orthogonal similarity transformation. See also Eigen [227], Wikipedia [1409], Wikipedia [1430].

This routine computes the Hessenberg decomposition of a square matrix A. Given A, an unitary matrix Q is determined such that

Q' A Q = H and Q' Q = Q Q' = 1

where H is an upper right Hessenberg matrix. Here ' denotes the hermitian transpose (i.e. transposition and conjugation).

input:

A : a real or complex square matrix overwrite_a : if true, allows modification of A which may improve performance. if false, A is not modified.

output:

Q : an unitary matrix H : an upper right Hessenberg matrix

example:

```
>>> from mpmath import mp
>>> A = mp.matrix([[3, -1, 2], [2, 5, -5], [-2, -3, 7]])
>>> Q, H = mp.hessenberg(A)
>>> mp.nprint(H, 3)
[  3.15  2.23  4.44]
[-0.769  4.85  3.05]
[   0.0  3.61   7.0]
>>> print(mp.chop(A - Q * H * Q.transpose_conj()))
[0.0  0.0  0.0]
[0.0  0.0  0.0]
[0.0  0.0  0.0]
```

return value: (Q, H)

7.26.13 Schur decomposition of a matrix A (real or complex)

`ctx.schur`(*A*, *overwrite_a=False*)

where `ctx` is `dec`, `mpm`, `fpm``or ``gmp`.

Performs a Schur decomposition of a square matrix.

See also Eigen [236], Eigen [222], Wikipedia [1437], Wikipedia [1430].

This routine computes the Schur decomposition of a square matrix A. Given A, an unitary matrix Q is determined such that

Q' A Q = R and Q' Q = Q Q' = 1

where R is an upper right triangular matrix. Here ' denotes the hermitian transpose (i.e. transposition and conjugation).

input:

A : a real or complex square matrix overwrite_a : if true, allows modification of A which may improve performance. if false, A is not modified.

output:

Q : an unitary matrix R : an upper right triangular matrix

return value: (Q, R)

example:

```
>>> from mpmath import mp
>>> A = mp.matrix([[3, -1, 2], [2, 5, -5], [-2, -3, 7]])
>>> Q, R = mp.schur(A)
>>> mp.nprint(R, 3)
[2.0  0.417  -2.53]
[0.0    4.0  -4.74]
[0.0    0.0    9.0]
>>> print(mp.chop(A - Q * R * Q.transpose_conj()))
```

(continues on next page)

(continued from previous page)

```
[0.0  0.0  0.0]
[0.0  0.0  0.0]
[0.0  0.0  0.0]
```

warning: The Schur decomposition is not unique.

Part II

Special Functions

CHAPTER

EIGHT

ELLIPTIC FUNCTIONS AND RELATED

Elliptic functions historically comprise the elliptic integrals and their inverses, and originate from the problem of computing the arc length of an ellipse. From a more modern point of view, an elliptic function is defined as a doubly periodic function, i.e. a function which satisfies

$$f(z + 2\omega_1) = f(z + 2\omega_2) = f(z)$$

for some half-periods ω_1, ω_2 with $\mathrm{Im}[\omega_1/\omega_2] > 0$. The canonical elliptic functions are the Jacobi elliptic functions. More broadly, this section includes quasi-doubly periodic functions (such as the Jacobi theta functions) and other functions useful in the study of elliptic functions.

Many different conventions for the arguments of elliptic functions are in use. It is even standard to use different parameterizations for different functions in the same text or software. The usual parameters are the elliptic nome q or $\bar{q} = q^2$ (see Wikipedia [1360], MathWorld [1000]), which usually must satisfy $|q| < 1$; the elliptic parameter m (an arbitrary complex number, see MathWorld [1001]); the elliptic modulus k (an arbitrary complex number, see MathWorld [999].); and the half-period ratio τ (see Wikipedia [1358], MathWorld [1004].), which usually must satisfy $\mathrm{Im}[\tau] > 0$.

In the context of Weierstrass elliptic functions, we also have the lattice roots e_1, e_2, e_3, which are the roots of the polynomial $4z^3 - g_2 z - g_3$, where g_2 and g_3 are the lattice invariants.

These quantities can be expressed in terms of each other using the following relations:

$$m = k^2 = \frac{e_3 - e_2}{e_1 - e_2}$$

$$\tau(m) = i\frac{K(1-m)}{K(m)}$$

$$q(\tau) = e^{i\pi\tau}$$

$$\bar{q}(\tau) = q^2(\tau) = e^{2i\pi\tau}$$

$$q(k) = \exp\left(-\pi\frac{K'(k)}{K(k)}\right)$$

$$k(q) = \frac{\theta_2^2(q)}{\theta_3^2(q)},$$

where $K(\cdot)$ denotes the complete elliptic integral of the first kind (Legendre), and $\theta_i(q)$ denote the Jacobi theta functions.

See also Wikipedia [1311], Wikipedia [1304], MathWorld [964], MathWorld [956], NIST [476].

The source code for the C# tests in Visual Studio is divided by real and complex modules:

The source code for the C# tests in Visual Studio using real modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B08a_RealEllipticFunctions.cs,

The source code for the C# tests in Visual Studio using complex modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B08b_CplxEllipticFunctions.cs,

8.1 Carlson symmetric elliptic integrals

8.1.1 Carlson symmetric elliptic integral of the first kind, $R_F(x, y, z)$

`ctx.elliptic_rf`(x, y, z)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Carlson symmetric elliptic integral of the first kind, $R_F(x, y, z)$, which is defined for $x, y, z \notin (-\infty, 0)$, and with at most one of x, y, z being zero. See also Wikipedia [1299], NIST [172], BoostMath [136], Ehrhardt [297] (3.2.2.2), Flint [748], Mpmath [644].

$$R_F(x, y, z) = \frac{1}{2} \int_0^\infty \frac{\mathrm{d}t}{\sqrt{(t+x)(t+y)(t+z)}}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticRF(0.12, 0.5, 3.5)
ereal('5.235987755982988730 7E-1')
>>> math53.EllipticRF(0.12, 0.5, 3.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '11.0'; y = '12.0'; z = '32.0'
>>> \mathrm{d}x = dec.elliprf(x, y, z); mx = mpm.elliprf(x, y, z); gx = gmp.elliprf(x, y, z)
>>> fx = fpm.elliprf(x, y, z); ax = apm.elliprf(x, y, z)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.429201214189074468052339993024079155901E-1
mpm:  2.429201214189074468052339993024079155901e-1
gmp:  2.429201214189074468052339993024079155901E-01
fpm:  2.42920121418907E-01
apm:  2.429201214189074468052339993024079155901e-1 (5.907e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; x = '11.0 + 2.0j'; y = '12.0 + 3.0j'; z = '42.0 + 3.0j'
>>> \mathrm{d}z = dec.elliprf(x, y, z); mz = mpm.elliprf(x, y, z); gz = gmp.elliprf(x, y, z)
>>> fz = fpm.elliprf(x, y, z); az = apm.elliprf(x, y, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 2.2678182726287324689E-1              - 1.7011644356805646798E-2j
mpm: 2.2678182726287324689e-1              - 1.7011644356805646798e-2j
gmp: 2.2678182726287324689E-01             - 1.7011644356805646798E-02j
fpm: 2.26781827262873E-01                  - 1.70116443568056E-02j
apm: 2.2678182726287324689e-1 (4.669e-20%) - 1.7011644356805646798e-2 (-1.556e-19%)j
```

8.1.2 Carlson completely symmetric elliptic integral of the second kind, $R_G(x, y, z)$

`ctx.elliptic_rg`(x, y, z)

where `ctx` is `math53` or `ctxflint`.

Returns the Carlson completely symmetric elliptic integral of the second kind, $R_G(x, y, z)$. See also Wikipedia [1299], NIST [172], BoostMath [136], Ehrhardt [297] (3.2.2.4), Flint [748], Mpmath [645].

$$R_G(x, y, z) = \frac{1}{4} \int_0^\infty \frac{t}{\sqrt{(t+x)(t+y)(t+z)}} \left(\frac{x}{t+x} + \frac{y}{t+y} + \frac{z}{t+z} \right) \mathrm{d}t.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticRG(0.12, 0.5, 3.5)
ereal('5.2359877559829887307E-1')
>>> math53.EllipticRG(0.12, 0.5, 3.5)
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '11.0'; y = '12.0'; z = '32.0'
>>> \mathrm{d}x = dec.elliprg(x, y, z); mx = mpm.elliprg(x, y, z); gx = gmp.elliprg(x, y, z)
>>> fx = fpm.elliprg(x, y, z); ax = apm.elliprg(x, y, z)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.22483801080739171837702427892069884713 6E+0
mpm:  4.22483801080739171837702427892069884713 6e+0
gmp:  4.22483801080739171837702427892069884713 6E+00
fpm:  4.22483801080739E+00
apm:  4.22483801080739171837702427892069884713 6e+0 (1.087e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; x = '11.0 + 2.0j'; y = '12.0 + 3.0j'; z = '42.0 + 3.0j'
>>> \mathrm{d}z = dec.elliprg(x, y, z); mz = mpm.elliprg(x, y, z); gz = gmp.elliprg(x, y, z)
>>> fz = fpm.elliprg(x, y, z); az = apm.elliprg(x, y, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 4.5675181312350784769E+0              + 3.0066935677293686497E-1j
mpm: 4.5675181312350784769e+0              + 3.0066935677293686497e-1j
gmp: 4.5675181312350784769E+00             + 3.0066935677293686497E-01j
fpm: 4.56751813123508E+00                  + 3.00669356772937E-01j
apm: 4.5675181312350784769e+0 (1.484e-19%) + 3.0066935677293686497e-1 (7.043e-20%)j
```

8.1.3 Carlson symmetric elliptic integral of the third kind, $R_J(x, y, z, p)$

`ctx.elliptic_rj`(x, y, z, p)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Carlson symmetric elliptic integral of the third kind, $R_J(x, y, z, p)$, with $x, y, z \geq 0$, at most one may be zero and $p \neq 0$. See also Wikipedia [1299], NIST [172], BoostMath [136], Ehrhardt [297] (3.2.2.5), Flint [748], Mpmath [646].

$$R_J(x, y, z, p) = \frac{3}{2} \int_0^\infty \frac{\mathrm{d}t}{(t+p)\sqrt{(t+x)(t+y)(t+z)}}.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticRJ(0.12, 0.5, 3.5, 0.4)
ereal('5.2359877559829887307E-1')
>>> math53.EllipticRJ(0.12, 0.5, 3.5, 0.4)
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '11.0'; y = '12.0'; z = '32.0'; p = '32.0'
>>> \mathrm{d}x = dec.elliprj(x, y, z, p); mx = mpm.elliprj(x, y, z, p); gx = gmp.elliprj(x, y, z, p)
>>> fx = fpm.elliprj(x, y, z, p); ax = apm.elliprj(x, y, z, p)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.67698189249460679377435937072975494096 8E-3
mpm:  9.67698189249460679377435937072975494096 8e-3
gmp:  9.67698189249460679377435937072975494096 8E-03
fpm:  9.67698189249461E-03
apm:  9.67698189249460679377435937072975494096 8e-3 (9.268e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; x = '11.0 + 2.0j'; y = '12.0 + 3.0j'; z = '42.0 + 3.0j'; p = '42.0 + 3.0j'
>>> \mathrm{d}z = dec.elliprj(x, y, z, p); mz = mpm.elliprj(x, y, z, p); gz = gmp.elliprj(x, y, z, p)
>>> fz = fpm.elliprj(x, y, z, p); az = apm.elliprj(x, y, z, p)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 7.13660119554007821 43E-3              - 1.252169581555734331 8E-3j
mpm: 7.13660119554007821 43e-3              - 1.252169581555734331 8e-3j
gmp: 7.13660119554007821 43E-03             - 1.252169581555734331 8E-03j
fpm: 7.13660119554008E-03                   - 1.25216958155573E-03j
apm: 7.13660119554007821 43e-3 (4.636e-20%) - 1.252169581555734331 8e-3 (-1.321e-19%)j
```

8.1.4 Carlson symmetric elliptic integral of the second kind, $R_D(x, y, z)$

`ctx.elliptic_rd`(x, y, z)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Carlson symmetric elliptic integral of the second kind, $R_D(x, y, z) = R_J(x, y, z, z)$, with $z > 0, x, y \geq 0$, at most one of x, y may be zero. See also Wikipedia [1299], NIST [172], BoostMath [136], Ehrhardt [297] (3.2.2.3), Flint [748], Mpmath [643].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticRD(0.12, 0.5, 3.5)
ereal('5.235987755982988730 7E-1')
>>> math53.EllipticRD(0.12, 0.5, 3.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '11.0'; y = '12.0'; z = '32.0'
>>> \mathrm{d}x = dec.elliprd(x, y, z); mx = mpm.elliprd(x, y, z); gx = gmp.elliprd(x, y, z)
>>> fx = fpm.elliprd(x, y, z); ax = apm.elliprd(x, y, z)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.67698189249460679377435937072975494096 8E-3
mpm:  9.67698189249460679377435937072975494096 8e-3
gmp:  9.67698189249460679377435937072975494096 8E-03
fpm:  9.67698189249461E-03
apm:  9.67698189249460679377435937072975494096 8e-3 (9.268e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; x = '11.0 + 2.0j'; y = '12.0 + 3.0j'; z = '42.0 + 3.0j'
>>> \mathrm{d}z = dec.elliprd(x, y, z); mz = mpm.elliprd(x, y, z); gz = gmp.elliprd(x, y, z)
>>> fz = fpm.elliprd(x, y, z); az = apm.elliprd(x, y, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 7.13660119554007821 43E-3              - 1.252169581555734331 8E-3j
mpm: 7.13660119554007821 43e-3              - 1.252169581555734331 8e-3j
gmp: 7.13660119554007821 43E-03             - 1.252169581555734331 8E-03j
fpm: 7.13660119554008E-03                   - 1.25216958155573E-03j
apm: 7.13660119554007821 43e-3 (4.636e-20%) - 1.252169581555734331 8e-3 (-1.321e-19%)j
```

8.1.5 Carlson degenerate symmetric elliptic integral of the first kind, $R_C(x, y)$

`ctx.elliptic_rc`(x, y)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Carlson degenerate symmetric elliptic integral of the first kind, $R_C(x, y) = R_F(x, y, y)$, for $x \geq 0, y \neq 0$. See also Wikipedia [1299], NIST [172], BoostMath [136], Ehrhardt [297] (3.2.2.1), Flint [748], Mpmath [642].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticRC(0.12, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.EllipticRC(0.12, 0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '11.0'; y = '12.0'
>>> \mathrm{d}x = dec.elliprc(x, y); mx = mpm.elliprc(x, y); gx = gmp.elliprc(x, y)
>>> fx = fpm.elliprc(x, y); ax = apm.elliprc(x, y)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.9284277172857547980887876923875888310 02E-1
mpm:  2.9284277172857547980887876923875888310 02e-1
gmp:  2.9284277172857547980887876923875888310 02E-01
fpm:  2.92842771728575E-01
apm:  2.9284277172857547980887876923875888310 02e-1 (1.96e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; x = '11.0 + 2.0j'; y = '12.0 + 3.0j'
>>> \mathrm{d}z = dec.elliprc(x, y); mz = mpm.elliprc(x, y); gz = gmp.elliprc(x, y)
>>> fz = fpm.elliprc(x, y); az = apm.elliprc(x, y)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 2.8731514984420510133E-1             - 3.22932944258792065 59E-2j
mpm: 2.8731514984420510133e-1             - 3.22932944258792065 59e-2j
gmp: 2.8731514984420510133E-01            - 3.22932944258792065 59E-02j
fpm: 2.87315149844205E-01                 - 3.22932944258792E-02j
apm: 2.8731514984420510133e-1 (7.37e-20%) - 3.22932944258792065 59e-2 (-8.197e-20%)j
```

8.2 Legendre elliptic integrals (elliptic parameter m)

While the Legendre/Bulirsch/Maple style functions have the modulus k (or k', k_c) as an argument, the Mathematica style functions (prefixed with `M`) use the parameter ($m = k^2$) and therefore cover the imaginary modulus functions with $m < 0$.

8.2.1 Legendre complete elliptic integral of the first kind, $K(m)$

`ctx.`**`m_elliptic_k`**`(`m`)`

where `ctx` is `math53` or `ctxflint`.

Returns the complete elliptic integral of the first kind, $\mathrm{M_EllipticK}(m) = \int_0^{\pi/2} \frac{\mathrm{d}t}{\sqrt{1 - m\sin^2(t)}}$, with parameter $m \neq 1$.

See also Wikipedia [1301], MathWorld [952], NIST [171], BoostMath [137], Ehrhardt [297] (3.2.5.1), Flint [749].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.MEllipticK(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.MEllipticK('0.51')
ereal('5.351847902755998475 4E-1')
```

Note

This function is called ellipk in mpmath.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_k(m); mx = mpm.elliptic_k(m); gx = gmp.elliptic_k(m)
>>> fx = fpm.elliptic_k(m); ax = apm.elliptic_k(m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.07536313529246914385344055588241580573 8E+0
mpm:  2.07536313529246914385344055588241580573 8e+0
gmp:  2.07536313529246914385344055588241580573 8E+00
fpm:  2.07536313529247E+00
apm:  2.07536313529246914385344055588241580573 8e+0 (5.531e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_k(m); mz = mpm.elliptic_k(m); gz = gmp.elliptic_k(m)
>>> fz = fpm.elliptic_k(m); az = apm.elliptic_k(m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 5.3766585026572650659E-1              + 7.1363115405781075467E-1j
mpm: 5.3766585026572650659e-1              + 7.1363115405781075467e-1j
gmp: 5.3766585026572650659E-01             + 7.1363115405781075467E-01j
fpm: 5.37665850265727E-01                  + 7.13631154057811E-01j
apm: 5.3766585026572650659e-1 (1.575e-19%) + 7.1363115405781075467e-1 (1.187e-19%)j
```

8.2.2 Legendre complete elliptic integral of the second kind, $E(m)$

`ctx.`**`m_elliptic_e`**`(`m`)`

where `ctx` is `math53` or `ctxflint`.

Returns the complete elliptic integral of the second kind, $\mathrm{M_EllipticEC}(m) = \int_0^{\pi/2} \sqrt{1 - m\sin^2(t)}\,\mathrm{d}t$, with parameter $m \neq 1$.

See also Wikipedia [1302], MathWorld [953], NIST [171], BoostMath [138], Ehrhardt [297] (3.2.5.2), Flint [749].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.MEllipticEC(0.5)
ereal('5.235987755982988730E-1')
>>> math53.MEllipticEC('0.51')
ereal('5.351847902755998475E-1')
```

Note

This function is called ellipe in mpmath, with ellipe taking one argument.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_e(m); mx = mpm.elliptic_e(m); gx = gmp.elliptic_e(m)
>>> fx = fpm.elliptic_e(m); ax = apm.elliptic_e(m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.2416705679458227508715113251723844272200E+0
mpm:  1.2416705679458227508715113251723844272200e+0
gmp:  1.2416705679458227508715113251723844272200E+00
fpm:  1.24167056794582E+00
apm:  1.2416705679458227508715113251723844272200e+0 (3.051e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_e(m); mz = mpm.elliptic_e(m); gz = gmp.elliptic_e(m)
>>> fz = fpm.elliptic_e(m); az = apm.elliptic_e(m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 7.2362394000819105306E-1              - 2.9177047805082638786E+0j
mpm: 7.2362394000819105306e-1              - 2.9177047805082638786e+0j
gmp: 7.2362394000819105306E-01             - 2.9177047805082638786E+00j
fpm: 7.23623940008191E-01                  - 2.91770478050826E+00j
apm: 7.2362394000819105306e-1 (2.517e-18%) - 2.9177047805082638786e+0 (-6.967e-19%)j
```

8.2.3 Legendre complete elliptic integral of the third kind, $\Pi(n, m)$

`ctx.m_elliptic_pi(`n, m`)`

where `ctx` is `math53` or `ctxflint`.

Returns the complete elliptic integral of the third kind , $\mathrm{M_EllipticPiC}(n, m) = \int_0^{\pi/2} \frac{\mathrm{d}t}{(1 - n\sin^2(t))\sqrt{1 - m\sin^2(t)}}$, with characteristic $n \neq 1$ and parameter $m \neq 1$.

See also Wikipedia [1303], MathWorld [954], NIST [171], BoostMath [139], Ehrhardt [297] (3.2.5.3), Flint [753].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.MEllipticPiC(3, 0.5)
```

(continues on next page)

(continued from previous page)

```
ereal('5.235987755982988730 7E-1')
>>> math53.MEllipticPiC(3, 0.5)
ereal('5.351847902755998475 4E-1')
```

Note

This function is called ellippi in mpmath, with ellippi taking two arguments.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '0.3'; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_pi(n, m); mx = mpm.elliptic_pi(n, m); gx = gmp.elliptic_pi(n, m)
>>> fx = fpm.elliptic_pi(n, m); ax = apm.elliptic_pi(n, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.54702065718735685622879937571980942764 1E+0
mpm:  2.54702065718735685622879937571980942764 1e+0
gmp:  2.54702065718735685622879937571980942764 1E+00
fpm:  2.54702065718736E+00
apm:  (2.54702065718735685622879937571980942764 1e+0 (1.803e-39%) + 0.0e+0 (0.0%)j)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '7.0 + 3.0j'; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_pi(n, m); mz = mpm.elliptic_pi(n, m); gz = gmp.elliptic_pi(n, m)
>>> fz = fpm.elliptic_pi(n, m); az = apm.elliptic_pi(n, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 2.1895146240011771984E-2              + 3.3171027919457516393E-1j
mpm: 2.1895146240011771984e-2              + 3.3171027919457516393e-1j
gmp: 2.1895146240011771984E-02             + 3.3171027919457516393E-01j
fpm: 2.18951462400118E-02                  + 3.31710279194575E-01j
apm: 2.1895146240011771984e-2 (1.142e-17%) + 3.3171027919457516393e-1 (8.299e-19%)j
```

8.2.4 Legendre incomplete elliptic integral of the first kind, $F(\phi, m)$

`ctx.m_elliptic_f`(*phi*, *m*)

where `ctx` is `math53` or `ctxflint`.

Returns the complete elliptic integral of the first kind, $\mathrm{M_EllipticF}(\phi, m) = \int_0^{\phi} \frac{\mathrm{d}t}{\sqrt{1 - m\sin^2(t)}}$, with parameter $m\sin^2(\phi) \leq 1$.

See also Wikipedia [1308], MathWorld [960], NIST [171], BoostMath [137], Ehrhardt [297] (3.2.5.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.MEllipticF(0.12, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.MEllipticF(0.12, 0.5)
ereal('5.351847902755998475 4E-1')
```

Note

This function is called ellipf in mpmath.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; phi = '0.3'; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_f(phi, m); mx = mpm.elliptic_f(phi, m); gx = gmp.elliptic_f(phi, m)
>>> fx = fpm.elliptic_f(phi, m); ax = apm.elliptic_f(phi, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  3.0318259675289643160378610668160462004 12E-1
mpm:  3.0318259675289643160378610668160462004 12e-1
gmp:  3.0318259675289643160378610668160462004 12E-01
fpm:  3.03182596752896E-01
apm:  3.0318259675289643160378610668160462004 12e-1 (1.893e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; phi = '7.0 + 3.0j'; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_f(phi, m); mz = mpm.elliptic_f(phi, m); gz = gmp.elliptic_f(phi, m)
>>> fz = fpm.elliptic_f(phi, m); az = apm.elliptic_f(phi, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 2.2337086166522330643E+0              + 3.6096091763407986549E+0j
mpm: 2.2337086166522330643e+0              + 3.6096091763407986549e+0j
gmp: 2.2337086166522330643E+00             + 3.6096091763407986549E+00j
fpm: 2.23370861665223E+00                  + 3.60960917634080E+00j
apm: 2.2337086166522330643e+0 (1.441e-18%) + 3.6096091763407986549e+0 (8.917e-19%)j
```

8.2.5 Legendre incomplete elliptic integral of the second kind, $E(\phi, m)$

`ctx.m_elliptic_e_inc`(*phi*, *m*)

where `ctx` is `math53` or `ctxflint`.

Returns the complete elliptic integral of the second kind, $\mathrm{M_EllipticE}(\phi, m) = \int_0^{\phi} \sqrt{1 - m\sin^2(t)}\,\mathrm{d}t$ with parameter $m\sin^2(\phi) \leq 1$.

See also Wikipedia [1309], MathWorld [961], NIST [171], BoostMath [138], Ehrhardt [297] (3.2.5.5), Flint [753].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.MEllipticE(0.12, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.MEllipticE(0.12, 0.5)
ereal('5.351847902755998475 4E-1')
```

Note

This function is called ellipe in mpmath, with ellipe taking two arguments.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; phi = '0.3'; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_e_inc(phi, m); mx = mpm.elliptic_e_inc(phi, m); gx = gmp.elliptic_e_inc(phi, m)
>>> fx = fpm.elliptic_e_inc(phi, m); ax = apm.elliptic_e_inc(phi, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.9687705450179863394833522188403876929 91E-1
mpm:  2.9687705450179863394833522188403876929 91e-1
gmp:  2.9687705450179863394833522188403876929 91E-01
fpm:  2.96877054501799E-01
apm:  2.9687705450179863394833522188403876929 91e-1 (1.933e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; phi = '7.0 + 3.0j'; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_e_inc(phi, m); mz = mpm.elliptic_e_inc(phi, m); gz = gmp.elliptic_e_inc(phi, m)
>>> fz = fpm.elliptic_e_inc(phi, m); az = apm.elliptic_e_inc(phi, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 2.2018638848426980990E+1              + 1.3783034022089380754E+1j
mpm: 2.2018638848426980990e+1              + 1.3783034022089380754e+1j
gmp: 2.2018638848426980990E+01             + 1.3783034022089380754E+01j
fpm: 2.20186388484270E+01                  + 1.37830340220894E+01j
apm: 2.2018638848426980990e+1 (8.617e-19%) + 1.3783034022089380754e+1 (1.377e-18%)j
```

8.2.6 Legendre incomplete elliptic integral of the third kind, $\Pi(n, \phi, m)$

`ctx.m_elliptic_pi_inc`(n, *phi*, m)

where `ctx` is `math53` or `ctxflint`.

Returns the complete elliptic integral of the third kind, $\mathrm{M_EllipticPi}(n, \phi, m) = \int_0^{\phi} \frac{\mathrm{d}t}{(1 - n\sin^2(t))\sqrt{1 - m\sin^2(t)}}$ with characteristic $n \neq 1$ and parameter $m \neq 1$.

See also Wikipedia [1310], MathWorld [962], NIST [171], BoostMath [139], Ehrhardt [297] (3.2.5.6), Flint [753].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.MEllipticPi(0.3, 0.12, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.MEllipticPi(0.3, 0.12, 0.5)
ereal('5.3518479027559984754E-1')
```

Note

This function is called ellippi in mpmath, with ellippi taking three arguments.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '0.6'; phi = '0.3'; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_pi_inc(n, phi, m); mx = mpm.elliptic_pi_inc(n, phi, m);
>>> gx = gmp.elliptic_pi_inc(n, phi, m)
>>> fx = fpm.elliptic_pi_inc(n, phi, m); ax = apm.elliptic_pi_inc(n, phi, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  3.0876549467783772880302306859508111382 12E-1
mpm:  3.0876549467783772880302306859508111382 12e-1
gmp:  3.0876549467783772880302306859508111382 12E-01
fpm:  6.49290594268217E-01
apm:  (3.0876549467783772880302306859508111382 12e-1 (1.859e-39%) + 0.0e+0 (0.0%)j)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '5.0 + 2.0j'; phi = '7.0 + 3.0j'; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_pi_inc(n, phi, m); mz = mpm.elliptic_pi_inc(n, phi, m);
>>> gz = gmp.elliptic_pi_inc(n, phi, m)
>>> fz = fpm.elliptic_pi_inc(n, phi, m); az = apm.elliptic_pi_inc(n, phi, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 8.92356276513367442 87E-2             + 1.8343289835828317105E+0j
mpm: 8.92356276513367442 87e-2             + 1.8343289835828317105e+0j
```

(continues on next page)

(continued from previous page)

```
gmp: 8.92356276513367287E-02              + 1.83432898358283105E+00j
fpm: 8.92356276513367E-02                 + 1.83432898358283E+00j
apm: 8.92356276513367288e-2 (4.509e-17%) + 1.83432898358283105e+0 (2.309e-18%)j
```

8.3 Legendre elliptic integrals (elliptic modulus k), and related functions

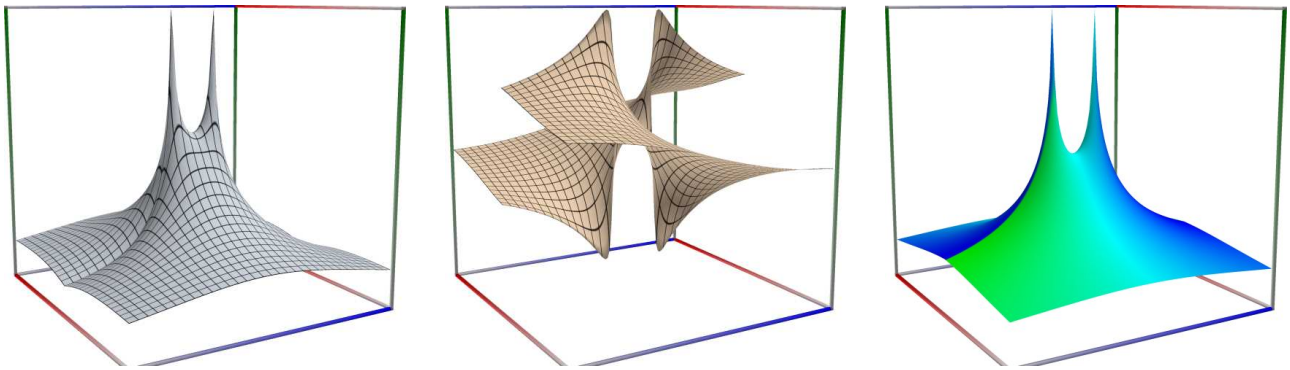
For an overview see NIST [171], BoostMath [135], Mpmath [648].

8.3.1 Legendre complete elliptic integral of the first kind, $K(k)$

`ctx.elliptic_k(k)`

where `ctx` is `math53`, `mathc53`, `ctxboost` or `ctxflint`.

Returns the Legendre complete elliptic integral of the first kind, $K(k) = \int_0^{\pi/2} \frac{\mathrm{d}t}{\sqrt{1-k^2\sin^2(t)}}$. See also Wikipedia [1301], MathWorld [952], NIST [171], BoostMath [137], Ehrhardt [297] (3.2.1.2), Ehrhardt [297] (4.2.30), Flint [749].



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

Note: the original names are: ctx.compEllint1(k), ctxboost.Ellint_1_K(k), mathc53.Ellk(k), ctx.ellipticK(k).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CompEllint1(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.CompEllint1('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_k(m); mx = mpm.elliptic_k(m); gx = gmp.elliptic_k(m)
>>> fx = fpm.elliptic_k(m); ax = apm.elliptic_k(m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.0753631352924691438534405558824158057 38E+0
mpm:  2.0753631352924691438534405558824158057 38e+0
gmp:  2.0753631352924691438534405558824158057 38E+00
fpm:  2.07536313529247E+00
apm:  2.0753631352924691438534405558824158057 38e+0 (5.531e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_k(m); mz = mpm.elliptic_k(m); gz = gmp.elliptic_k(m)
>>> fz = fpm.elliptic_k(m); az = apm.elliptic_k(m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
```

(continues on next page)

(continued from previous page)

```
dec: 5.3766585026572650659E-1              + 7.1363115405781075467E-1j
mpm: 5.3766585026572650659e-1              + 7.1363115405781075467e-1j
gmp: 5.3766585026572650659E-01             + 7.1363115405781075467E-01j
fpm: 5.37665850265727E-01                  + 7.13631154057811E-01j
apm: 5.3766585026572650659e-1 (1.575e-19%) + 7.1363115405781075467e-1 (1.187e-19%)j
```

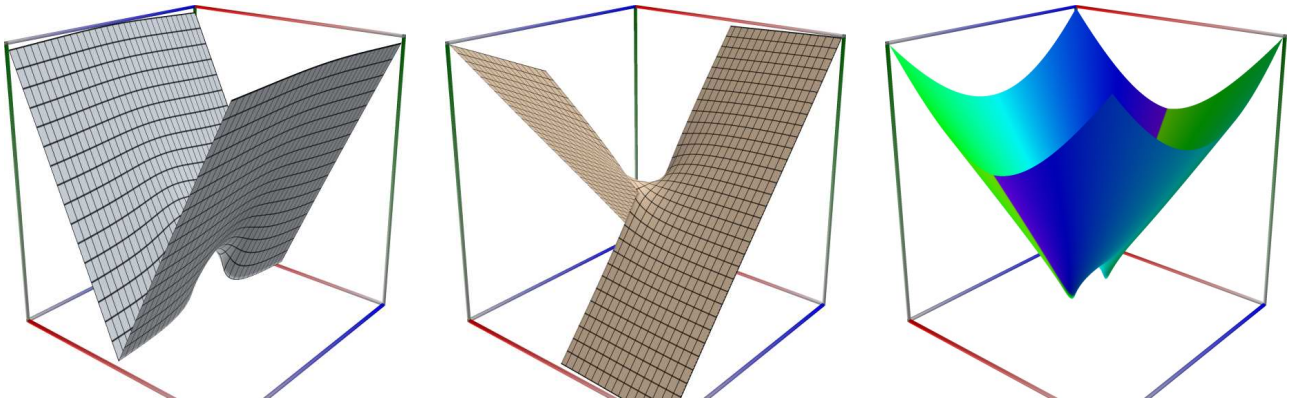
8.3.2 Legendre complete elliptic integral of the second kind, $E(k)$

`ctx.elliptic_e(`k`)`

where `ctx` is `math53`, `mathc53`, `ctxboost` or `ctxflint`.

Note: the original names are: ctx.compEllint2(k), ctxboost.Ellint_2_K(k), mathc53.Elle(k), ctx.ellipticE(k).

Returns the Legendre complete elliptic integral of the second kind, $E(k) = \int_0^{\pi/2} \sqrt{1 - k^2 \sin^2(t)}\, \mathrm{d}t$. See also Wikipedia [1302], MathWorld [953], NIST [171], BoostMath [138], Ehrhardt [297] (3.2.1.2), Ehrhardt [297] (4.2.29), Flint [749].



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CompEllint2(0.5)
ereal('5.235987755982988730E-1')
>>> math53.CompEllint2('0.51')
ereal('5.351847902755998475E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_e(m); mx = mpm.elliptic_e(m); gx = gmp.elliptic_e(m)
>>> fx = fpm.elliptic_e(m); ax = apm.elliptic_e(m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.241670567945822750871511325172384427220E+0
mpm:  1.241670567945822750871511325172384427220e+0
gmp:  1.241670567945822750871511325172384427220E+00
fpm:  1.24167056794582E+00
apm:  1.241670567945822750871511325172384427220e+0 (3.051e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_e(m); mz = mpm.elliptic_e(m); gz = gmp.elliptic_e(m)
>>> fz = fpm.elliptic_e(m); az = apm.elliptic_e(m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 7.2362394000819105306E-1              - 2.9177047805082638786E+0j
mpm: 7.2362394000819105306e-1              - 2.9177047805082638786e+0j
gmp: 7.2362394000819105306E-01             - 2.9177047805082638786E+00j
fpm: 7.23623940008191E-01                  - 2.91770478050826E+00j
apm: 7.2362394000819105306e-1 (2.517e-18%) - 2.9177047805082638786e+0 (-6.967e-19%)j
```

8.3.3 Legendre complete elliptic integral of the third kind, $\Pi(n;k)$

`ctx.elliptic_pi`(n, k)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

The original names are: ctx.compEllint3(n, k), ctxboost.Ellint3K(n, k), ctx.ellipticPi(n, k).

Returns the Legendre complete elliptic integral of the third kind, $\Pi(n;k) = \int_0^{\pi/2} \frac{\mathrm{d}t}{(1-n\sin^2(t))\sqrt{1-k^2\sin^2(t)}}$. See also Wikipedia [1303], MathWorld [954], NIST [171], BoostMath [139], Ehrhardt [297] (3.2.1.3), Flint [749].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CompEllint3(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.CompEllint3(3, 0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '0.3'; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_pi(n, m); mx = mpm.elliptic_pi(n, m); gx = gmp.elliptic_pi(n, m)
>>> fx = fpm.elliptic_pi(n, m); ax = apm.elliptic_pi(n, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.5470206571873568562287993757198094276 41E+0
mpm:  2.5470206571873568562287993757198094276 41e+0
gmp:  2.5470206571873568562287993757198094276 41E+00
fpm:  2.54702065718736E+00
apm:  (2.5470206571873568562287993757198094276 41e+0 (1.803e-39%) + 0.0e+0 (0.0%)j)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '7.0 + 3.0j'; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_pi(n, m); mz = mpm.elliptic_pi(n, m); gz = gmp.elliptic_pi(n, m)
>>> fz = fpm.elliptic_pi(n, m); az = apm.elliptic_pi(n, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 2.1895146240011771984E-2              + 3.3171027919457516393E-1j
mpm: 2.1895146240011771984e-2              + 3.3171027919457516393e-1j
gmp: 2.1895146240011771984E-02             + 3.3171027919457516393E-01j
fpm: 2.18951462400118E-02                  + 3.31710279194575E-01j
apm: 2.1895146240011771984e-2 (1.142e-17%) + 3.3171027919457516393e-1 (8.299e-19%)j
```

8.3.4 Legendre incomplete elliptic integral of the first kind, $F(\phi, k)$

`ctx.elliptic_f`(*phi*, *k*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

The original names are: ctx.ellint1(phi, k), ctxboost.Ellint1F(phi, k), ctx.ellipticF(phi, k),

Returns the Legendre incomplete elliptic integral of the first kind, $F(\phi, k) = \int_0^{\phi} \frac{\mathrm{d}t}{\sqrt{1 - k^2 \sin^2(t)}}$. See also Wikipedia [1308], MathWorld [960], NIST [171], BoostMath [137], Ehrhardt [297] (3.2.1.6), Flint [753], Mpmath [649].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Ellint1(0.12, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Ellint1(0.12, 0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; phi = '0.3'; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_f(phi, m); mx = mpm.elliptic_f(phi, m); gx = gmp.elliptic_f(phi, m)
>>> fx = fpm.elliptic_f(phi, m); ax = apm.elliptic_f(phi, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  3.0318259675289643160378610668160462004 12E-1
mpm:  3.0318259675289643160378610668160462004 12e-1
gmp:  3.0318259675289643160378610668160462004 12E-01
fpm:  3.03182596752896E-01
apm:  3.0318259675289643160378610668160462004 12e-1 (1.893e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; phi = '7.0 + 3.0j'; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_f(phi, m); mz = mpm.elliptic_f(phi, m); gz = gmp.elliptic_f(phi, m)
>>> fz = fpm.elliptic_f(phi, m); az = apm.elliptic_f(phi, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 2.2337086166522330643E+0              + 3.6096091763407986549E+0j
mpm: 2.2337086166522330643e+0              + 3.6096091763407986549e+0j
gmp: 2.2337086166522330643E+00             + 3.6096091763407986549E+00j
fpm: 2.23370861665223E+00                  + 3.60960917634080E+00j
apm: 2.2337086166522330643e+0 (1.441e-18%) + 3.6096091763407986549e+0 (8.917e-19%)j
```

8.3.5 Legendre incomplete elliptic integral of the second kind, $E(\phi, k)$

`ctx.elliptic_e_inc`(*phi*, *k*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

The original names are: ctx.ellint2(phi, k), ctxboost.Ellint2E(phi, k), ctx.ellipticEInc(phi, k).

Returns the Legendre incomplete elliptic integral of the second kind, $E(\phi, k) = \int_0^{\phi} \sqrt{1 - k^2 \sin^2(t)}\, \mathrm{d}t$. See also Wikipedia [1309], MathWorld [961], NIST [171], BoostMath [138], Ehrhardt [297] (3.2.1.7), Flint [753], Mpmath [657].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Ellint2(0.12, 0.5)
ereal('5.235987755982988730 7E-1')
```

(continues on next page)

(continued from previous page)

```
>>> math53.Ellint2(0.12, 0.5)
ereal('5.351847902755984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; phi = '0.3'; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_e_inc(phi, m); mx = mpm.elliptic_e_inc(phi, m); gx = gmp.elliptic_e_inc(phi, m)
>>> fx = fpm.elliptic_e_inc(phi, m); ax = apm.elliptic_e_inc(phi, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.968770545017986339483352218840387692991E-1
mpm:  2.968770545017986339483352218840387692991e-1
gmp:  2.968770545017986339483352218840387692991E-01
fpm:  2.96877054501799E-01
apm:  2.968770545017986339483352218840387692991e-1 (1.933e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; phi = '7.0 + 3.0j'; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_e_inc(phi, m); mz = mpm.elliptic_e_inc(phi, m); gz = gmp.elliptic_e_inc(phi, m)
>>> fz = fpm.elliptic_e_inc(phi, m); az = apm.elliptic_e_inc(phi, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 2.2018638848426980990E+1              + 1.3783034022089380754E+1j
mpm: 2.2018638848426980990e+1              + 1.3783034022089380754e+1j
gmp: 2.2018638848426980990E+01             + 1.3783034022089380754E+01j
fpm: 2.20186388484270E+01                  + 1.37830340220894E+01j
apm: 2.2018638848426980990e+1 (8.617e-19%) + 1.3783034022089380754e+1 (1.377e-18%)j
```

8.3.6 Legendre incomplete elliptic integral of the third kind, $\Pi(n, \phi, k)$

`ctx.`**`elliptic_pi_inc`**(*n*, *phi*, *k*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

The original names are: ctx.ellint3(phi, n, k), ctxboost.Ellint3F(phi, n, k), ctx.ellipticPiInc(phi, n, k).

Returns the Legendre incomplete elliptic integral of the third kind, $\Pi(n, \phi, k) = \int_0^{\phi} \frac{\mathrm{d}t}{(1 - n\sin^2(t))\sqrt{1 - k^2\sin^2(t)}}$. See also Wikipedia [1310], MathWorld [962], NIST [171], BoostMath [139], Ehrhardt [297] (3.2.1.8), Flint [753], Mpmath [658].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Ellint3(0.3, 0.12, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Ellint3(0.3, 0.12, 0.5)
ereal('5.351847902755984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '0.6'; phi = '0.3'; m = '0.7'
>>> \mathrm{d}x = dec.elliptic_pi_inc(n, phi, m); mx = mpm.elliptic_pi_inc(n, phi, m);
>>> gx = gmp.elliptic_pi_inc(n, phi, m)
>>> fx = fpm.elliptic_pi_inc(n, phi, m); ax = apm.elliptic_pi_inc(n, phi, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  3.087654946778377288030230685950811138212E-1
mpm:  3.087654946778377288030230685950811138212e-1
gmp:  3.087654946778377288030230685950811138212E-01
fpm:  6.49290594268217E-01
apm:  (3.087654946778377288030230685950811138212e-1 (1.859e-39%) + 0.0e+0 (0.0%)j)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '5.0 + 2.0j'; phi = '7.0 + 3.0j'; m = '11.0 + 3.0j'
>>> \mathrm{d}z = dec.elliptic_pi_inc(n, phi, m); mz = mpm.elliptic_pi_inc(n, phi, m);
>>> gz = gmp.elliptic_pi_inc(n, phi, m)
>>> fz = fpm.elliptic_pi_inc(n, phi, m); az = apm.elliptic_pi_inc(n, phi, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 8.923562765133674428 7E-2              + 1.8343289835828317105E+0j
mpm: 8.923562765133674428 7e-2              + 1.8343289835828317105e+0j
gmp: 8.923562765133674428 7E-02             + 1.8343289835828317105E+00j
fpm: 8.92356276513367E-02                   + 1.83432898358283E+00j
apm: 8.923562765133674428 8e-2 (4.509e-17%) + 1.8343289835828317105e+0 (2.309e-18%)j
```

8.4 Jacobi elliptic functions

For an introduction, see Wikipedia [1311], MathWorld [964], BoostMath [153], NIST [476], Mpmath [656].

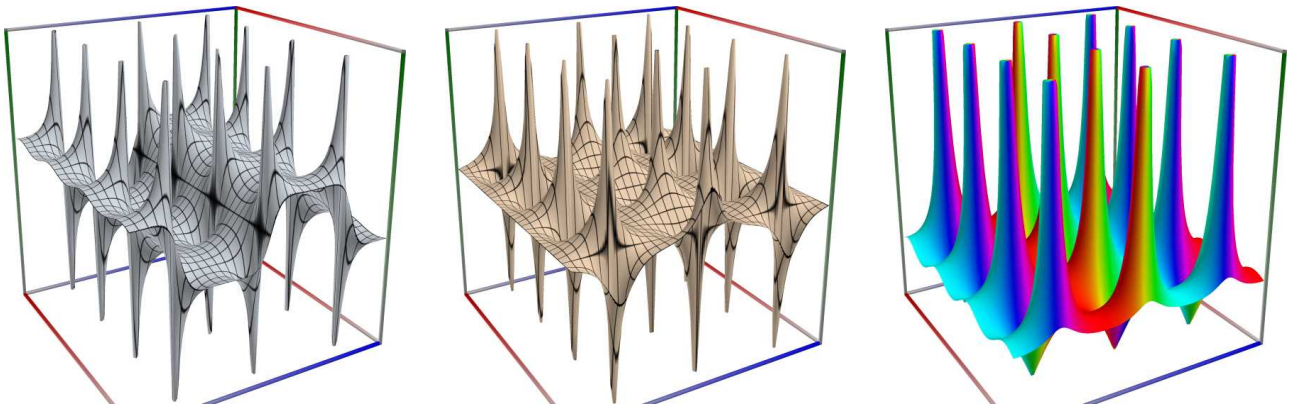
8.4.1 Jacobi elliptic function $\mathrm{sn}(x, k)$

`ctx.jacobi_sn`(x, k)

where `ctx` is `math53`, `mathc53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{sn}(x, k) = \sin(\mathrm{am}(x, k))$, where $\mathrm{am}(x, k)$ denotes the Jacobi amplitude function, and $\mathrm{sn}(x, 0) = \sin(x)$. See also BoostMath [157], Wikipedia [1311], MathWorld [277], NIST [476], Ehrhardt [297] (3.2.11.1), Ehrhardt [297] (4.2.58).

The version for `ecplx` has the restriction that k must be real.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiSN(0.8, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.JacobiSN(0.8, '0.51')
ereal('5.351847902755998475 4E-1')
```

The function is defined as

$$\mathrm{sn}(u, k) = \frac{\theta_3(0, q)}{\theta_2(0, q)} \frac{\theta_1(t, q)}{\theta_4(t, q)}.$$

Here $t = u/\theta_3^2(0, q)$, $q = q(k) = \exp\left[-\pi K'(k)/K(k)\right]$ denotes the nome, $K(k)$ denotes the complete elliptic integral of the first kind and $K'(k) = K(\sqrt{1 - k^2})$ denotes the complementary complete elliptic integral of the first kind.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_sn(u, m); mx = mpm.jacobi_sn(u, m); gx = gmp.jacobi_sn(u, m)
>>> fx = fpm.jacobi_sn(u, m); ax = apm.jacobi_sn(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  6.1856562602158125727536054884810479000 65E-1
mpm:  6.1856562602158125727536054884810479000 65e-1
gmp:  6.1856562602158125727536054884810479000 65E-01
fpm:  6.18565626021581E-01
apm:  6.1856562602158125727536054884810479000 65e-1 (9.279e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_sn(u, m); mz = mpm.jacobi_sn(u, m); gz = gmp.jacobi_sn(u, m)
>>> fz = fpm.jacobi_sn(u, m); az = apm.jacobi_sn(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 1.9171467233805943128E+0              + 4.6311952375545897275E-1j
mpm: 1.9171467233805943128e+0              + 4.6311952375545897275e-1j
gmp: 1.9171467233805943128E+00             + 4.6311952375545897275E-01j
fpm: 1.91714672338059E+00                  + 4.63119523755459E-01j
apm: 1.9171467233805943128e+0 (4.418e-20%) + 4.6311952375545897275e-1 (4.572e-20%)j
```

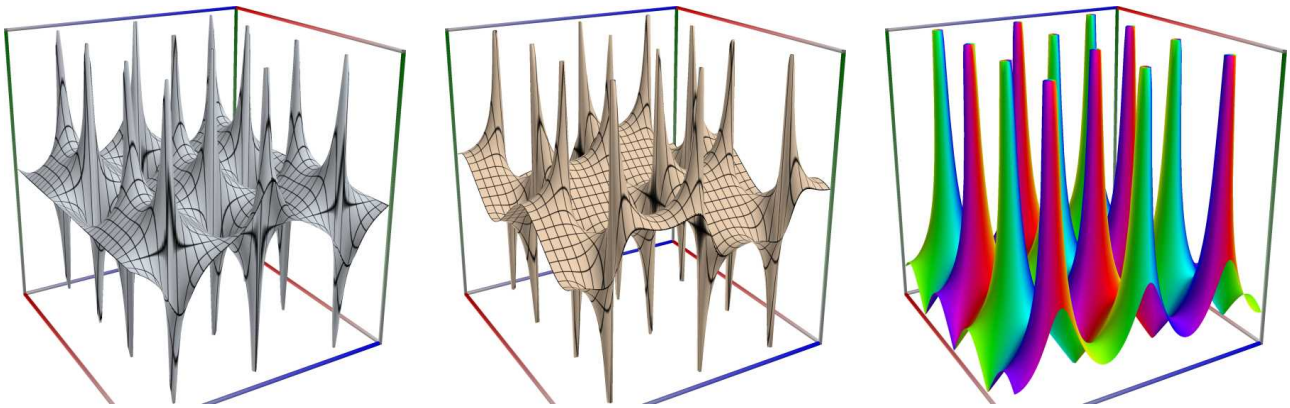
8.4.2 Jacobi elliptic function $\mathrm{cn}(x,k)$

`ctx.jacobi_cn`(x, k)

where `ctx` is `math53`, `mathc53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{cn}(x,k) = \cos(\mathrm{am}(x,k))$, where $\mathrm{am}(x,k)$ denotes the Jacobi amplitude function, and $\mathrm{cn}(x,0) = \cos(x)$. See also BoostMath [143], Wikipedia [1311], MathWorld [267], NIST [476], Ehrhardt [297] (3.2.11.2), Ehrhardt [297] (4.2.18).

The version for `ecplx` has the restriction that k must be real.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x+iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiCN(0.8, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.JacobiCN(0.8, '0.51')
ereal('5.351847902755998475 4E-1')
```

The function is defined as

$$\mathrm{cn}(u,k) = \frac{\theta_4(0,q)}{\theta_2(0,q)} \frac{\theta_2(t,q)}{\theta_4(t,q)}.$$

Here $t = u/\theta_3^2(0,q)$, and $q = q(k)$ denotes the nome.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_cn(u, m); mx = mpm.jacobi_cn(u, m); gx = gmp.jacobi_cn(u, m)
>>> fx = fpm.jacobi_cn(u, m); ax = apm.jacobi_cn(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -7.8573313937018672508204794793539942153 72E-1
mpm:  -7.8573313937018672508204794793539942153 72e-1
gmp:  -7.8573313937018672508204794793539942153 72E-01
fpm:  -7.85733139370187E-01
apm:  -7.8573313937018672508204794793539942153 73e-1 (-7.305e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_cn(u, m); mz = mpm.jacobi_cn(u, m); gz = gmp.jacobi_cn(u, m)
```

(continues on next page)

(continued from previous page)

```
>>> fz = fpm.jacobi_cn(u, m); az = apm.jacobi_cn(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 5.3561363916431377722E-1              - 1.6576651761270445419E+0j
mpm: 5.3561363916431377722e-1              - 1.6576651761270445419e+0j
gmp: 5.3561363916431377722E-01             - 1.6576651761270445419E+00j
fpm: 5.356136391643 14E-01                 - 1.65766517612704E+00j
apm: 5.3561363916431377721e-1 (7.907e-20%) - 1.6576651761270445419e+0 (-5.11e-20%)j
```

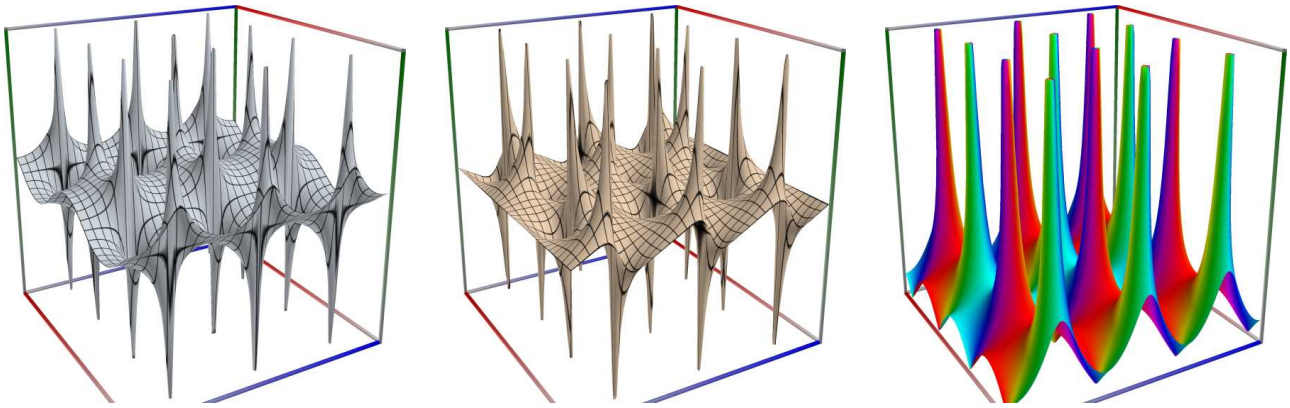
8.4.3 Jacobi elliptic function $\mathrm{dn}(x,k)$

`ctx.jacobi_dn`(x,k)

where `ctx` is `math53`, `mathc53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{dn}(x,k) = \sqrt{1-k^2\mathrm{sn}^2(x,k)}$ with $\mathrm{dn}(x,0) = 1$. See also BoostMath [146], Wikipedia [1311], MathWorld [270], NIST [476], Ehrhardt [297] (3.2.11.3), Ehrhardt [297] (4.2.26).

The version for `ecplx` has the restriction that k must be real.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x+iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiDN(0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiDN(0.8, '0.51')
ereal('5.351847902755998475 4E-1')
```

The function is defined as

$$\mathrm{dn}(u,k) = \frac{\theta_4(0,q)}{\theta_3(0,q)}\frac{\theta_3(t,q)}{\theta_4(t,q)}.$$

Here $t = u/\theta_3^2(0,q)$, and $q = q(k)$ denotes the nome.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_dn(u, m); mx = mpm.jacobi_dn(u, m); gx = gmp.jacobi_dn(u, m)
>>> fx = fpm.jacobi_dn(u, m); ax = apm.jacobi_dn(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  8.7773910690063113178012121710137689060 43E-1
mpm:  8.7773910690063113178012121710137689060 43e-1
gmp:  8.7773910690063113178012121710137689060 43E-01
fpm:  8.77739106900631E-01
apm:  8.7773910690063113178012121710137689060 43e-1 (6.539e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_dn(u, m); mz = mpm.jacobi_dn(u, m); gz = gmp.jacobi_dn(u, m)
>>> fz = fpm.jacobi_dn(u, m); az = apm.jacobi_dn(u, m)
```

(continues on next page)

(continued from previous page)

```
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -4.6801583272820746144E-1               + 1.1382538992226457533E+0j
mpm: -4.6801583272820746144e-1               + 1.1382538992226457533e+0j
gmp: -4.6801583272820746144E-01              + 1.1382538992226457533E+00j
fpm: -4.68015832728208E-01                   + 1.13825389922265E+00j
apm: -4.6801583272820746143e-1 (-4.525e-20%) + 1.1382538992226457533e+0 (7.442e-20%)j
```

8.4.4 Jacobi elliptic function $\mathrm{nc}(x, k)$

`ctx.jacobi_nc`(x, k)

where `ctx` is `math53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{nc}(x, k) = 1/\mathrm{cn}(x, k)$, with $\mathrm{nc}(x, 0) = 1/\cos(x)$. See also BoostMath [148], Wikipedia [1311], MathWorld [272], NIST [476], Ehrhardt [297] (3.2.11.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiNC(0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiNC(0.8, '0.51')
ereal('5.351847902755998475 4E-1')
```

The function is defined as

$$\mathrm{nc}(u, k) = \frac{1}{\mathrm{cn}(u, k)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_nc(u, m); mx = mpm.jacobi_nc(u, m); gx = gmp.jacobi_nc(u, m)
>>> fx = fpm.jacobi_nc(u, m); ax = apm.jacobi_nc(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -1.272696733653822082392877366275776826356E+0
mpm:  -1.272696733653822082392877366275776826356e+0
gmp:  -1.272696733653822082392877366275776826356E+00
fpm:  -1.27269673365382E+00
apm:  -1.272696733653822082392877366275776826356e+0 (-9.02e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_nc(u, m); mz = mpm.jacobi_nc(u, m); gz = gmp.jacobi_nc(u, m)
>>> fz = fpm.jacobi_nc(u, m); az = apm.jacobi_nc(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 1.7649432217423873247E-1              + 5.4623047334802764261E-1j
mpm: 1.7649432217423873247e-1              + 5.4623047334802764261e-1j
gmp: 1.7649432217423873247E-01             + 5.4623047334802764261E-01j
fpm: 1.76494322174239E-01                  + 5.46230473348028E-01j
apm: 1.7649432217423873247e-1 (5.999e-20%) + 5.4623047334802764261e-1 (7.753e-20%)j
```

8.4.5 Jacobi elliptic function $\mathrm{sc}(x, k)$

`ctx.jacobi_sc`(x, k)

where `ctx` is `math53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{sc}(x, k) = \mathrm{sn}(x, k)/\mathrm{cn}(x, k)$. See also BoostMath [151], Wikipedia [1311], MathWorld [275], NIST [476], Ehrhardt [297] (3.2.11.5).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiSC(0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiSC(0.8, '0.51')
ereal('5.3518479027559984754E-1')
```

The function is defined as

$$\mathrm{sc}(u, k) = \frac{\mathrm{sn}(u, k)}{\mathrm{cn}(u, k)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_sc(u, m); mx = mpm.jacobi_sc(u, m); gx = gmp.jacobi_sc(u, m)
>>> fx = fpm.jacobi_sc(u, m); ax = apm.jacobi_sc(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -7.872464517881981193304679205001097295818E-1
mpm:  -7.872464517881981193304679205001097295818e-1
gmp:  -7.872464517881981193304679205001097295818E-01
fpm:  -7.87246451788198E-01
apm:  -7.872464517881981193304679205001097295818e-1 (-7.291e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_sc(u, m); mz = mpm.jacobi_sc(u, m); gz = gmp.jacobi_sc(u, m)
>>> fz = fpm.jacobi_sc(u, m); az = apm.jacobi_sc(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 8.53955147739632691 21E-2              + 1.128941928620678229 2E+0j
mpm: 8.53955147739632691 21e-2              + 1.128941928620678229 2e+0j
gmp: 8.53955147739632691 21E-02             + 1.128941928620678229 2E+00j
fpm: 8.53955147739632E-02                   + 1.12894192862068E+00j
apm: 8.53955147739632691 21e-2 (6.199e-20%) + 1.128941928620678229 2e+0 (7.503e-20%)j
```

8.4.6 Jacobi elliptic function $\mathrm{dc}(x, k)$

`ctx.jacobi_dc`(x, k)

where `ctx` is `math53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{dc}(x, k) = \mathrm{dn}(x, k)/\mathrm{cn}(x, k)$. See also BoostMath [145], Wikipedia [1311], MathWorld [269], NIST [476], Ehrhardt [297] (3.2.11.6).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiDC(0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiDC(0.8, '0.51')
ereal('5.3518479027559984754E-1')
```

The function is defined as

$$\mathrm{dc}(u, k) = \frac{\mathrm{dn}(u, k)}{\mathrm{cn}(u, k)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_dc(u, m); mx = mpm.jacobi_dc(u, m); gx = gmp.jacobi_dc(u, m)
>>> fx = fpm.jacobi_dc(u, m); ax = apm.jacobi_dc(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -1.1170956943526562077260877896397738069 12E+0
mpm:  -1.1170956943526562077260877896397738069 12e+0
gmp:  -1.1170956943526562077260877896397738069 12E+00
fpm:  -1.11709569435266E+00
apm:  -1.1170956943526562077260877896397738069 12e+0 (-1.028e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_dc(u, m); mz = mpm.jacobi_dc(u, m); gz = gmp.jacobi_dc(u, m)
>>> fz = fpm.jacobi_dc(u, m); az = apm.jacobi_dc(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -7.0435110332680081506E-1               - 5.4749159440014983159E-2j
mpm: -7.0435110332680081506e-1               - 5.4749159440014983159e-2j
gmp: -7.0435110332680081506E-01              - 5.4749159440014983159E-02j
fpm: -7.04351103326801E-01                   - 5.4749159400150E-02j
apm: -7.0435110332680081505e-1 (-6.013e-20%) - 5.4749159440014983159e-2 (-4.835e-20%)j
```

8.4.7 Jacobi elliptic function $\mathrm{nd}(x, k)$

`ctx.jacobi_nd`(x, k)

where `ctx` is `math53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{nd}(x, k) = 1/\mathrm{dn}(x, k)$. See also BoostMath [149], Wikipedia [1311], MathWorld [273], NIST [476], Ehrhardt [297] (3.2.11.7).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiNC(0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiNC(0.8, '0.51')
ereal('5.3518479027559984754E-1')
```

The function is defined as

$$\mathrm{nd}(u, k) = \frac{1}{\mathrm{dn}(u, k)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_nd(u, m); mx = mpm.jacobi_nd(u, m); gx = gmp.jacobi_nd(u, m)
>>> fx = fpm.jacobi_nd(u, m); ax = apm.jacobi_nd(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.139290698270334703522055295154469970359E+0
mpm:  1.139290698270334703522055295154469970359e+0
gmp:  1.1392906982703347035220552951544699703 59E+00
fpm:  1.13929069827033E+00
apm:  1.139290698270334703522055295154469970359e+0 (1.008e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_nd(u, m); mz = mpm.jacobi_nd(u, m); gz = gmp.jacobi_nd(u, m)
>>> fz = fpm.jacobi_nd(u, m); az = apm.jacobi_nd(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -3.089905313881816412 0E-1                - 7.514909808184303598 9E-1j
mpm: -3.089905313881816412 0e-1                - 7.514909808184303598 9e-1j
gmp: -3.089905313881816412 0E-01               - 7.514909808184303598 9E-01j
fpm: -3.08990531388182E-01                     - 7.514909808184 30E-01j
apm: -3.089905313881816412 0e-1 (-6.853e-20%) - 7.514909808184303598 9e-1 (-5.636e-20%)j
```

8.4.8 Jacobi elliptic function $\mathrm{sd}(x, k)$

`ctx.jacobi_sd`(x, k)

where `ctx` is `math53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{sd}(x, k) = \mathrm{sn}(x, k)/\mathrm{dn}(x, k)$. See also BoostMath [156], Wikipedia [1311], MathWorld [276], NIST [476], Ehrhardt [297] (3.2.11.8).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiSD(0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiSD(0.8, '0.51')
ereal('5.3518479027559984754E-1')
```

The function is defined as

$$\mathrm{sd}(u, k) = \frac{\mathrm{sn}(u, k)}{\mathrm{dn}(u, k)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_sd(u, m); mx = mpm.jacobi_sd(u, m); gx = gmp.jacobi_sd(u, m)
>>> fx = fpm.jacobi_sd(u, m); ax = apm.jacobi_sd(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  7.047260639961540287844417991398276789242E-1
mpm:  7.047260639961540287844417991398276789242e-1
gmp:  7.047260639961540287844417991398276789242E-01
fpm:  7.04726063996154E-01
apm:  7.047260639961540287844417991398276789242e-1 (8.145e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_sd(u, m); mz = mpm.jacobi_sd(u, m); gz = gmp.jacobi_sd(u, m)
>>> fz = fpm.jacobi_sd(u, m); az = apm.jacobi_sd(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -2.4435003966332689103E-1               - 1.5838180192675636948E+0j
mpm: -2.4435003966332689103e-1               - 1.5838180192675636948e+0j
gmp: -2.4435003966332689103E-01              - 1.5838180192675636948E+00j
fpm: -2.44350039663327E-01                   - 1.58381801926756E+00j
apm: -2.4435003966332689103e-1 (-4.333e-20%) - 1.5838180192675636948e+0 (-5.348e-20%)j
```

8.4.9 Jacobi elliptic function $\mathrm{cd}(x, k)$

`ctx.jacobi_cd`(x, k)

where `ctx` is `math53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{cd}(x, k) = \mathrm{cn}(x, k)/\mathrm{dn}(x, k)$. See also BoostMath [142], Wikipedia [1311], MathWorld [266], NIST [476], Ehrhardt [297] (3.2.11.9).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiCD(0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiCD(0.8, '0.51')
ereal('5.3518479027559984754E-1')
```

The function is defined as

$$\mathrm{cd}(u, k) = \frac{\mathrm{cn}(u, k)}{\mathrm{dn}(u, k)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_cd(u, m); mx = mpm.jacobi_cd(u, m); gx = gmp.jacobi_cd(u, m)
>>> fx = fpm.jacobi_cd(u, m); ax = apm.jacobi_cd(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -8.9517845700720224968815369523893806822 50E-1
mpm:  -8.9517845700720224968815369523893806822 50e-1
gmp:  -8.9517845700720224968815369523893806822 50E-01
fpm:  -8.95178457007202E-01
apm:  -8.9517845700720224968815369523893806822 50e-1 (-6.412e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_cd(u, m); mz = mpm.jacobi_cd(u, m); gz = gmp.jacobi_cd(u, m)
>>> fz = fpm.jacobi_cd(u, m); az = apm.jacobi_cd(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -1.4112199720604079057E+0               + 1.0969402459986020016E-1j
mpm: -1.4112199720604079057e+0               + 1.0969402459986020016e-1j
gmp: -1.4112199720604079057E+00              + 1.0969402459986020016E-01j
fpm: -1.41121997206041E+00                   + 1.09694024599860E-01j
apm: -1.4112199720604079057e+0 (-6.002e-20%) + 1.0969402459986020016e-1 (4.826e-20%)j
```

8.4.10 Jacobi elliptic function $\mathrm{ns}(x, k)$

`ctx.jacobi_ns`(x, k)

where `ctx` is `math53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{ns}(x, k) = 1/\mathrm{dn}(x, k)$. See also BoostMath [150], Wikipedia [1311], MathWorld [274], NIST [476], Ehrhardt [297] (3.2.11.10).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiNS(0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiNS(0.8, '0.51')
ereal('5.3518479027559984754E-1')
```

The function is defined as

$$\mathrm{ns}(u, k) = \frac{1}{\mathrm{sn}(u, k)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_ns(u, m); mx = mpm.jacobi_ns(u, m); gx = gmp.jacobi_ns(u, m)
>>> fx = fpm.jacobi_ns(u, m); ax = apm.jacobi_ns(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.6166433405484946946358843054865440088759E+0
mpm:  1.6166433405484946946358843054865440088759e+0
gmp:  1.6166433405484946946358843054865440088759E+00
fpm:  1.61664334054850E+00
apm:  1.6166433405484946946358843054865440088759e+0 (7.101e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_ns(u, m); mz = mpm.jacobi_ns(u, m); gz = gmp.jacobi_ns(u, m)
>>> fz = fpm.jacobi_ns(u, m); az = apm.jacobi_ns(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 4.9284848473105701998E-1              - 1.1905596621721617534E-1j
mpm: 4.9284848473105701998e-1              - 1.1905596621721617534e-1j
gmp: 4.9284848473105701998E-01             - 1.1905596621721617534E-01j
fpm: 4.92848484731057E-01                  - 1.19055966217216E-01j
apm: 4.9284848473105701998e-1 (4.297e-20%) - 1.1905596621721617534e-1 (-4.447e-20%)j
```

8.4.11 Jacobi elliptic function $\mathrm{cs}(x, k)$

`ctx.jacobi_cs`(x, k)

where `ctx` is `math53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{cs}(x, k) = \mathrm{cn}(x, k)/\mathrm{sn}(x, k)$. See also BoostMath [144], Wikipedia [1311], MathWorld [268], NIST [476], Ehrhardt [297] (3.2.11.11).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiCS(0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiCS(0.8, '0.51')
ereal('5.3518479027559984754E-1')
```

The function is defined as

$$\mathrm{cs}(u, k) = \frac{\mathrm{cn}(u, k)}{\mathrm{sn}(u, k)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_cs(u, m); mx = mpm.jacobi_cs(u, m); gx = gmp.jacobi_cs(u, m)
>>> fx = fpm.jacobi_cs(u, m); ax = apm.jacobi_cs(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -1.270250247211074622004449060713343376866E+0
mpm:  -1.270250247211074622004449060713343376866e+0
gmp:  -1.270250247211074622004449060713343376866E+00
fpm:  -1.27025024721108E+00
apm:  -1.270250247211074622004449060713343376866e+0 (-9.037e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_cs(u, m); mz = mpm.jacobi_cs(u, m); gz = gmp.jacobi_cs(u, m)
>>> fz = fpm.jacobi_cs(u, m); az = apm.jacobi_cs(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 6.6621441254982066702E-2              - 8.8074576957548141330E-1j
mpm: 6.6621441254982066702e-2              - 8.8074576957548141330e-1j
gmp: 6.66214412549820667O2E-02             - 8.8074576957548141330E-01j
fpm: 6.66214412549821E-02                  - 8.80745769575481E-01j
apm: 6.6621441254982066702e-2 (7.946e-20%) - 8.8074576957548141330e-1 (-4.809e-20%)j
```

8.4.12 Jacobi elliptic function $\mathrm{ds}(x, k)$

`ctx.jacobi_ds`(x, k)

where `ctx` is `math53` or `ctxboost`.

Returns the Jacobi elliptic function $\mathrm{ds}(x, k) = \mathrm{dn}(x, k)/\mathrm{sn}(x, k)$. See also BoostMath [147], Wikipedia [1311], MathWorld [271], NIST [476], Ehrhardt [297] (3.2.11.12).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiDS(0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiDS(0.8, '0.51')
ereal('5.3518479027559984754E-1')
```

The function is defined as

$$\mathrm{ds}(u, k) = \frac{\mathrm{dn}(u, k)}{\mathrm{sn}(u, k)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; u = '11.0'; m = '0.6'
>>> \mathrm{d}x = dec.jacobi_ds(u, m); mx = mpm.jacobi_ds(u, m); gx = gmp.jacobi_ds(u, m)
>>> fx = fpm.jacobi_ds(u, m); ax = apm.jacobi_ds(u, m)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.4189910819098886043984307934421336225 11E+0
mpm:  1.4189910819098886043984307934421336225 11e+0
gmp:  1.4189910819098886043984307934421336225 11E+00
fpm:  1.41899108190989E+00
apm:  1.4189910819098886043984307934421336225 11e+0 (8.09e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; u = '11.0 + 2.0j'; m = '0.6'
>>> \mathrm{d}z = dec.jacobi_ds(u, m); mz = mpm.jacobi_ds(u, m); gz = gmp.jacobi_ds(u, m)
>>> fz = fpm.jacobi_ds(u, m); az = apm.jacobi_ds(u, m)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -9.5144976217774993594E-2               + 6.1670678664151000551E-1j
mpm: -9.5144976217774993594e-2               + 6.1670678664151000551e-1j
gmp: -9.5144976217774993594E-02              + 6.1670678664151000551E-01j
fpm: -9.5144976217775 0E-02                  + 6.16706786641510E-01j
apm: -9.5144976217774993595e-2 (-5.564e-20%) + 6.1670678664151000551e-1 (6.867e-20%)j
```

8.5 Jacobi theta functions and related functions

The theta functions are functions of two variables:

- z is the *argument*, an arbitrary real or complex number
- q is the *nome*, which must be a real or complex number in the unit disk (i.e. $|q| < 1$). For $|q| \ll 1$, the series converge very quickly, so the Jacobi theta functions can efficiently be evaluated to high precision.

The compact notations $\theta_n(q) = \theta_n(0,q)$ and $\theta_n = \theta_n(0,q)$ are also frequently encountered. Finally, Jacobi theta functions are frequently considered as functions of the half-period ratio τ and then usually denoted by $\theta_n(z|\tau)$.

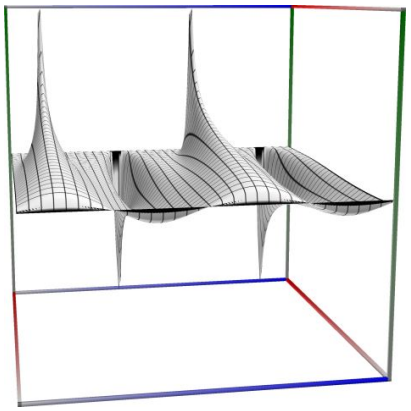
8.5.1 Jacobi theta function $\theta_1(z,q)$

`ctx.jacobi_theta1`(x, q)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Jacobi theta function $\theta_1(z,q) = 2q^{1/4}\sum_{n=0}^{\infty}(-1)^n q^{n^2+n}\sin((2n+1)z)$

Note: the Amath version needs to be adapted.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x+iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

See also Wikipedia [1318], MathWorld [965], MathWorld [278], NIST [478], Ehrhardt [297] (3.2.13), Flint [752].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiTheta(1, 0.8, 0.5)
ereal('5.235987755982988730E-1')
>>> math53.JacobiTheta(2, 0.8, '0.51')
ereal('5.351847902755998475E-1')
```

An example with real input for $\theta_1(z,q)$:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '3.8'; q = '0.7'
>>> \mathrm{d}x = dec.jtheta(1, x, q); mx = mpm.jtheta(1, x, q); gx = gmp.jtheta(1, x, q)
>>> fx = fpm.jtheta(1, x, q); ax = apm.jtheta(1, x, q)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -2.8762231759388901391542580060536909694 77E-1
mpm:  -2.8762231759388901391542580060536909694 77e-1
gmp:  -2.8762231759388901391542580060536909694 77E-01
```

(continues on next page)

(continued from previous page)

```
fpm:  -2.87622317593889E-01
apm:  -2.87622317593889013915425800605369096948 0e-1 (-2.326e-35%)
```

An example with complex input for $\theta_1(z, q)$:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '11.0 + 3.0j'; q = '0.7 + 0.1j'
>>> \mathrm{d}z = dec.jtheta(1, z, q); mz = mpm.jtheta(1, z, q); gz = gmp.jtheta(1, z, q)
>>> fz = fpm.jtheta(1, z, q); az = apm.jtheta(1, z, q)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 1.3257637053506044723E+10              - 2.5508683032905869130E+9j
mpm: 1.3257637053506044723e+10              - 2.5508683032905869130e+9j
gmp: 1.3257637053506044723E+10              - 2.5508683032905869130E+09j
fpm: 1.32576370535060E+10                   - 2.55086830329060E+09j
apm: 1.3257637053506044630e+10 (4.315e-14%) - 2.5508683032905869160e+9 (-2.258e-13%)j
```

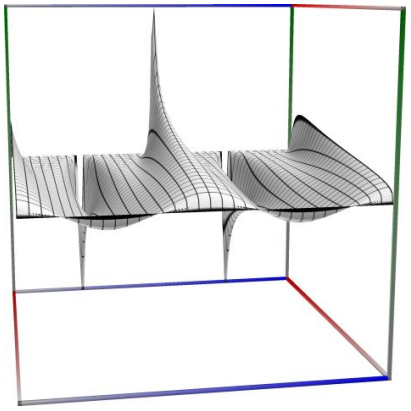
8.5.2 Jacobi theta function $\theta_2(z,q)$

`ctx.jacobi_theta2`(x, q)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Jacobi theta function $\theta_2(z,q) = 2q^{1/4}\sum_{n=0}^{\infty} q^{n^2+n}\cos((2n+1)z)$

Note: the Amath version needs to be adapted.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x+iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

See also Wikipedia [1318], MathWorld [965], MathWorld [278], NIST [478], Ehrhardt [297] (3.2.13), Flint [752].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiTheta(1, 0.8, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.JacobiTheta(2, 0.8, '0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input for $\theta_2(z,q)$:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '3.8'; q = '0.7'
>>> \mathrm{d}x = dec.jtheta(2, x, q); mx = mpm.jtheta(2, x, q); gx = gmp.jtheta(2, x, q)
>>> fx = fpm.jtheta(2, x, q); ax = apm.jtheta(2, x, q)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -8.8023799832717479554342994457525942401 70E-1
mpm:  -8.8023799832717479554342994457525942401 70e-1
gmp:  -8.8023799832717479554342994457525942401 70E-01
fpm:  -8.80237998327175E-01
apm:  -8.8023799832717479554342994457525942401 80e-1 (-1.375e-35%)
```

An example with complex input for $\theta_2(z,q)$:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '11.0 + 3.0j'; q = '0.7 + 0.1j'
>>> \mathrm{d}z = dec.jtheta(2, z, q); mz = mpm.jtheta(2, z, q); gz = gmp.jtheta(2, z, q)
>>> fz = fpm.jtheta(2, z, q); az = apm.jtheta(2, z, q)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 2.6679401530814916188E+11              - 3.0867614993820689042E+11j
mpm: 2.6679401530814916188e+11              - 3.0867614993820689042e+11j
gmp: 2.6679401530814916188E+11              - 3.0867614993820689042E+11j
```

(continues on next page)

(continued from previous page)

```
fpm: 2.66794015308149E+11                    - 3.08676149938205E+11j
apm: 2.667940153081491613e+11 (4.123e-14%) - 3.086761499382068889e+11 (-3.758e-14%)j
```

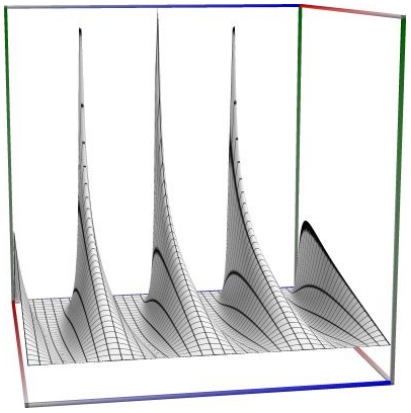
8.5.3 Jacobi theta function $\theta_3(z, q)$

`ctx.jacobi_theta3`(x, q)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Jacobi theta function $\theta_3(z,q) = 1 + 2\sum_{n=1}^{\infty} q^{n^2} \cos(2nz)$

Note: the Amath version needs to be adapted.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

See also Wikipedia [1318], MathWorld [965], MathWorld [278], NIST [478], Ehrhardt [297] (3.2.13), Flint [752].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiTheta(1, 0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiTheta(2, 0.8, '0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input for $\theta_3(z, q)$:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '3.8'; q = '0.7'
>>> \mathrm{d}x = dec.jtheta(3, x, q); mx = mpm.jtheta(3, x, q); gx = gmp.jtheta(3, x, q)
>>> fx = fpm.jtheta(3, x, q); ax = apm.jtheta(3, x, q)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  8.80238182561265443549153736619152129192 3E-1
mpm:  8.80238182561265443549153736619152129192 3e-1
gmp:  8.80238182561265443549153736619152129192 3E-01
fpm:  8.80238182561266E-01
apm:  8.80238182561265443549153736619152129194 0e-1 (1.375e-35%)
```

An example with complex input for $\theta_3(z, q)$:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '11.0 + 3.0j'; q = '0.7 + 0.1j'
>>> \mathrm{d}z = dec.jtheta(3, z, q); mz = mpm.jtheta(3, z, q); gz = gmp.jtheta(3, z, q)
>>> fz = fpm.jtheta(3, z, q); az = apm.jtheta(3, z, q)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -2.6679401441538351941E+11                + 3.0867614546580330581E+11j
mpm: -2.6679401441538351941e+11                + 3.0867614546580330581e+11j
gmp: -2.6679401441538351941E+11                + 3.0867614546580330581E+11j
```

(continues on next page)

(continued from previous page)

```
fpm: -2.66794014415383E+11                    + 3.08676145465801E+11j
apm: -2.667940144153835188 0e+11 (-4.086e-14%) + 3.086761454658033043 0e+11 (3.758e-14%)j
```

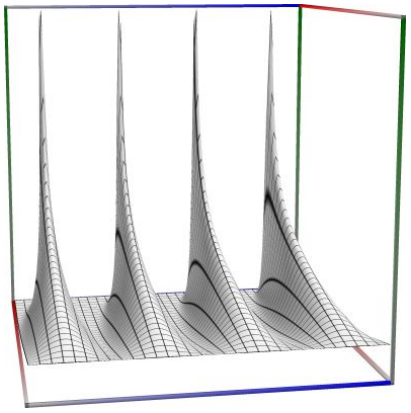
8.5.4 Jacobi theta function $\theta_4(z, q)$

`ctx.jacobi_theta4`(x, q)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Jacobi theta function $\theta_4(z,q) = 1 + 2\sum_{n=1}^{\infty}(-q)^{n^2}\ \cos(2nz)$

Note: the Amath version needs to be adapted.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

See also Wikipedia [1318], MathWorld [965], MathWorld [278], NIST [478], Ehrhardt [297] (3.2.13), Flint [752].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiTheta(1, 0.8, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiTheta(2, 0.8, '0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input for $\theta_4(z, q)$:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '3.8'; q = '0.7'
>>> \mathrm{d}x = dec.jtheta(4, x, q); mx = mpm.jtheta(4, x, q); gx = gmp.jtheta(4, x, q)
>>> fx = fpm.jtheta(4, x, q); ax = apm.jtheta(4, x, q)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.8762759828499249331900812813152778933900E-1
mpm:  2.8762759828499249331900812813152778933900e-1
gmp:  2.8762759828499249331900812813152778933900E-01
fpm:  2.87627598284992E-01
apm:  2.8762759828499249331900812813152778934000e-1 (2.587e-35%)
```

An example with complex input for $\theta_4(z, q)$:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '11.0 + 3.0j'; q = '0.7 + 0.1j'
>>> \mathrm{d}z = dec.jtheta(4, z, q); mz = mpm.jtheta(4, z, q); gz = gmp.jtheta(4, z, q)
>>> fz = fpm.jtheta(4, z, q); az = apm.jtheta(4, z, q)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -1.3378357586306498825E+10               + 2.5798286002171674621E+9j
mpm: -1.3378357586306498825e+10               + 2.5798286002171674621e+9j
gmp: -1.3378357586306498825E+10               + 2.5798286002171674621E+09j
```

(continues on next page)

(continued from previous page)

```
fpm: -1.33783575863064E+10                    + 2.57982860021718E+09j
apm: -1.337835758630649873e+10 (-4.313e-14%) + 2.579828600217167465e+9 (2.252e-13%)j
```

8.6 Conversions of parameters of Weierstrass $\wp$

Wikipedia [1355], Wikipedia [1356], MathWorld [992], MathWorld [990], MathWorld [284], MathWorld [283].

The Weierstrass elliptic functions may be defined on a general lattice $\Lambda = \{m2\omega_1 + n2\omega_2 : m, n \in \mathbb{Z}\}$ with half-periods $\omega_1, \omega_2 \in \mathbb{C}$.

In XlCalcNet, two interfaces are used:

The first interface (described in this section) uses $\omega = \omega_1$ and $\tau = \omega_2/\omega_1$ as parameters; these have a natural geometrical interpretation, specifying respectively the size and shape of the "fundamental parallelogram". In Flint there is a default choice of $\omega = 1$; therefore ω is omitted from the function parameters in Flint, but ω is a required parameter in XlCalcNet.

In some contexts the lattice invariants g_2 and g_3 are the natural parameters because they correspond directly with physical quantities; usually $g_2, g_3 \in \mathbb{R}$ in such cases (and also in XlCalcNet). This interface is described in the next section.

To keep these two interfaces apart with regard to mathematical notation, we write $\wp(z|\omega,\tau)$ for the first interface and $\wp_g(z; g_2, g_3)$ for the second, and likewise for the other Weierstrass functions.

To evaluate the functions on a general lattice, we can use the the following homogeneity relations (for $t \neq 0$). Note that the period ratio τ is preserved:

$$\wp'(tz|t\omega,\tau) = t^{-3}\wp'(z|\omega,\tau)$$

$$\wp(tz|t\omega,\tau) = t^{-2}\wp(z|\omega,\tau)$$

$$\zeta(tz|t\omega,\tau) = t^{-1}\zeta(z|\omega,\tau)$$

$$\sigma(tz|t\omega,\tau) = t\sigma(z|\omega,\tau)$$

$$g_2(t\omega,\tau) = t^{-4}g_2(\omega,\tau)$$

$$g_3(t\omega,\tau) = t^{-6}g_3(\omega,\tau)$$

$$e_i(t\omega,\tau) = t^{-2}e_i(\omega,\tau), \quad i = 1,2,3$$

$$\Delta(t\omega,\tau) = t^{-12}\Delta(\omega,\tau)$$

8.6.1 Elliptic lattice invariants g_2, g_3 from lattice roots e_1, e_2, e_3

`ctx.elliptic_invariants_from_roots(`*e1*, *e2*`)`

Computes the lattice invariants g_2, g_3. The Weierstrass elliptic function satisfies the differential equation $[\wp'(z,\tau)]^2 = 4[\wp(z,\tau)]^3 - g_2\wp(z,\tau) - g_3$. Up to constant factors, the lattice invariants are the first two Eisenstein series.

See also: MathWorld [284], MathWorld [283], Flint [770].

$$g_2 = 2({e_1}^2 + {e_2}^2 + {e_3}^2) = -4(e_2e_3 + e_3e_1 + e_1e_2),$$

$$g_3 = 4e_1e_2e_3 = \tfrac{4}{3}({e_1}^3 + {e_2}^3 + {e_3}^3).$$

An example with purely imaginary input, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; tau = '0.0 + 0.7j'
>>> dx2, dx3 = dec.elliptic_invariants(tau); mx2, mx3 = mpm.elliptic_invariants(tau);
>>> gx2, gx3  = gmp.elliptic_invariants(tau)
>>> fx2, fx3 = fpm.elliptic_invariants(tau); ax2, ax3 = apm.elliptic_invariants(tau)
>>> print("lattice invariant g2")
>>> mpm.show([dx2, mx2, gx2, fx2, ax2], aligned=True)
>>> print("lattice invariant g3")
>>> mpm.show([dx3, mx3, gx3, fx3, ax3], aligned=True)
```

(continues on next page)

(continued from previous page)

```
lattice invariant g2
dec: 5.5736606022929792322897789424188426717 44E+2
mpm: 5.5736606022929792322897789424188426717 37e+2
gmp: 5.5736606022929792322897789424188426717 49E+02
fpm: 5.57366060229298E+02
apm: 5.5736606022929792322897789424188426717 35e+2 (7.909e-38%)

lattice invariant g3
dec: -2.2663337633041457434772839748348735435 70E+3
mpm: -2.2663337633041457434772839748348735435 63e+3
gmp: -2.2663337633041457434772839748348735435 71E+03
fpm: -2.26633376330415E+03
apm: -2.2663337633041457434772839748348735435 60e+3 (-1.184e-36%)
```

8.6.2 Elliptic lattice invariants g_2, g_3 from (ω, τ)

`ctx.g2g3_from_tau(`*omega*, *tau*`)`

Computes the lattice invariants g_2, g_3 in terms of half-period ω_1 and elliptic period ratio τ.

We have $g_2(t\omega, \tau) = t^{-4} g_2(\omega, \tau)$ and $g_3(t\omega, \tau) = t^{-6} g_3(\omega, \tau)$.

See also: MathWorld [284], MathWorld [283], Flint [770].

See also: https://en.wikipedia.org/wiki/J-invariant#Expressions_in_terms_of_theta_functions

See also: https://dlmf.nist.gov/23.3#i

We obtain the elliptic lattice invariants from the Eisenstein series $G_4(\tau)$ and $G_6(\tau)$:

$$g_2(\tau) = 60 G_4(\tau) = 60 \sum_{(m,n) \neq (0,0)} (m + n\tau)^{-4}$$

$$g_3(\tau) = 140 G_6(\tau) = 140 \sum_{(m,n) \neq (0,0)} (m + n\tau)^{-6}$$

Define $q = e^{\pi i \tau}$

Let $a = \theta_2(q)$, $b = \theta_3(q)$, $c = \theta_4(q)$, where $a^4 - b^4 + c^4 = 0$. Then

$g_2(\tau) = \frac{2}{3}\pi^4 \left(a^8 + b^8 + c^8\right)$, and

$g_3(\tau) = \frac{4}{27}\pi^6 \sqrt{\frac{(a^8+b^8+c^8)^3 - 54(abc)^8}{2}}$.

8.6.3 Elliptic lattice roots e_1, e_2, e_3 from (ω, τ)

`ctx.elliptic_roots_from_tau(`*omega*, *tau*`)`

Computes the lattice roots e_1, e_2, e_3, in terms of half-period ω_1 and elliptic period ratio τ. They are the roots of the polynomial $4z^3 - g_2 z - g_3$.

We have $e_i(t\omega, \tau) = t^{-2} e_i(\omega, \tau), \quad i = 1, 2, 3.$

See also Wikipedia [1356], Flint [770].

See also: https://dlmf.nist.gov/23.3#i

With τ as input, the roots e_1, e_2, and e_3 can be expressed in terms of the Jacobi theta functions $\theta_i(0; q)$. Let $q = e^{2i\pi\tau}$ denote the nome. Then

$$a = \theta_2(0; q); \quad b = \theta_3(0; q); \quad c = \theta_4(0; q), \quad \text{with } a^4 - b^4 + c^4 = 0, \quad \text{and}$$

$$e_1(\tau) = \tfrac{\pi^2}{3}(b^4 + c^4); \quad e_2(\tau) = \tfrac{\pi^2}{3}(-a^4 - b^4); \quad e_3(\tau) = \tfrac{\pi^2}{3}(a^4 - c^4).$$

An example with purely imaginary input, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; tau = '0.0 + 0.7j'
>>> dx1, dx2, dx3 = dec.elliptic_roots(tau); mx1, mx2, mx3 = mpm.elliptic_roots(tau);
>>> gx1, gx2, gx3  = gmp.elliptic_roots(tau)
>>> fx1, fx2, fx3 = fpm.elliptic_roots(tau); ax1, ax2, ax3 = apm.elliptic_roots(tau)
>>> print("lattice root e1")
>>> mpm.show([dx1, mx1, gx1, fx1, ax1], aligned=True)
lattice root e1
dec: 8.546997557335192078343940404355992077317E+0
mpm: 8.546997557335192078343940404355992077314e+0
gmp: 8.546997557335192078343940404355992077321E+00
fpm: 8.54699755733519E+00
apm: 8.546997557335192078343940404355992077315e+0 (4.298e-39%)

>>> print("lattice root e2")
>>> mpm.show([dx2, mx2, gx2, fx2, ax2], aligned=True)
lattice root e2
dec: -1.346877689428238019477524701740978037594E+1
mpm: -1.346877689428238019477524701740978037593e+1
gmp: -1.346877689428238019477524701740978037595E+01
fpm: -1.34687768942824E+01
apm: 4.921779336947188116431306613053788298616e+0 (1.493e-38%)

>>> print("lattice root e3")
>>> mpm.show([dx3, mx3, gx3, fx3, ax3], aligned=True)
lattice root e3
dec: 4.921779336947188116431306613053788298624E+0
mpm: 4.921779336947188116431306613053788298616e+0
gmp: 4.921779336947188116431306613053788298625E+00
fpm: 4.92177933694719E+00
apm: -1.346877689428238019477524701740978037593e+1 (-7.023e-38%)
```

!!! NEED TO FIX: Decimal complex negation and multiplication from left

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; tau = '0.4 + 0.7j'
>>> dx1, dx2, dx3 = dec.elliptic_roots(tau); mx1, mx2, mx3 = mpm.elliptic_roots(tau);
>>> gx1, gx2, gx3  = gmp.elliptic_roots(tau)
>>> fx1, fx2, fx3 = fpm.elliptic_roots(tau); ax1, ax2, ax3 = apm.elliptic_roots(tau)
>>> print("lattice root e1")
>>> mpm.show([dx1, mx1, gx1, fx1, ax1], aligned=True)
lattice root e1
```

(continues on next page)

(continued from previous page)

```
dec: 5.0162095528741509613E+0              + 1.1199909932847896589E+0j
mpm: 5.0162095528741509605e+0              + 1.1199909932847896584e+0j
gmp: 5.0162095528741509611E+00             + 1.11999099328478965888E+00j
fpm: 5.01620955287415E+00                  + 1.11999099328479E+00j
apm: 5.0162095528741509605e+0 (4.525e-18%) + 1.1199909932847896584e+0 (1.989e-17%)j

>>> print("lattice root e2")
>>> mpm.show([dx2, mx2, gx2, fx2, ax2], aligned=True)
lattice root e2
dec: -3.2965087919479326553E+0               - 3.2515656088055451348E+0j
mpm: -4.873316822969647033e+0                - 8.6347058331318457444e+0j
gmp: -4.8733168229696470039E+00              - 8.63470583313184574450E+00j
fpm: -4.87331682296965E+00                   - 8.63470583313184E+00j
apm: -1.4289272990450395721e-1 (-1.792e-16%) + 7.5147148398470560860e+0 (3.562e-18%)j

>>> print("lattice root e3")
>>> mpm.show([dx3, mx3, gx3, fx3, ax3], aligned=True)
lattice root e3
dec: -1.4289272990450395785E-1              + 7.5147148398470560861E+0j
mpm: -1.4289272990450395722e-1              + 7.5147148398470560860e+0j
gmp: -1.42892729904503957200E-01            + 7.51471483984706862E+00j
fpm: -1.42892729904505E-01                  + 7.51471483984706E+00j
apm: -4.873316822969647032e+0 (-6.82e-17%) - 8.6347058331318457441e+0 (-3.869e-17%)j
```

8.7 Weierstrass elliptic functions, in terms of elliptic period ratio τ

8.7.1 Weierstrass function $\wp(z|\tau)$

`ctx.weierstrass_p_t(`z, *tau*`)`

Computes Weierstrass's elliptic function in terms of half-period ω_1 and elliptic period ratio τ.

We have $\wp(tz|t\omega,\tau) = t^{-2}\wp(z|\omega,\tau)$.

See also MathWorld [992], Flint [770].

See also: https://dlmf.nist.gov/23.2

$$\wp(z,\tau,\omega=1) = \frac{1}{z^2} + \sum_{n^2+m^2\neq 0}\left[\frac{1}{(z+m+n\tau)^2} - \frac{1}{(m+n\tau)^2}\right]$$

which satisfies $\wp(z,\tau,\omega=1) = \wp(z+1,\tau,\omega=1) = \wp(z+\tau,\tau,\omega=1)$. To evaluate the function efficiently, we use the formula (with with $q = e^{\pi i\tau}$)

$$\wp(z,\tau) = \pi^3\theta_2^2(0,q)\theta_3^2(0,q)\frac{\theta_4^2(\pi z,q)}{\theta_1^2(\pi z,q)} - \frac{\pi^3}{3}\left[\theta_2^4(0,q)+\theta_3^4(0,q)\right].$$

$$\wp(tz|t\omega_1,t\omega_2) = t^{-2}\wp(z|\omega_1,\omega_2)$$

An example with real input for z and with purely imaginary input for τ, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; z = '0.1'; tau = '0.0 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p(z, tau); mx = mpm.weierstrass_p(z, tau)
>>> gx = gmp.weierstrass_p(z, tau)
>>> fx = fpm.weierstrass_p(z, tau); ax = apm.weierstrass_p(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.0012029137285933516203337364613247608 25E+2
mpm:  1.0012029137285933516203337364613247608 25e+2
gmp:  1.0012029137285933516203337364613247608 25E+02
fpm:  1.00120291372859E+02
apm:  1.0012029137285933516203337364613247608 25e+2 (1.321e-38%)
```

An example with complex input for z and τ:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '0.1 + 0.5j'; tau = '0.7 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p(z, tau); mx = mpm.weierstrass_p(z, tau)
>>> gx = gmp.weierstrass_p(z, tau)
>>> fx = fpm.weierstrass_p(z, tau); ax = apm.weierstrass_p(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec:  -2.3859027427781674104E+0
mpm:  -2.3859027427781674104e+0
gmp:  -2.3859027427781674104E+00
fpm:  -2.38590274277817E+00
apm:  (-2.3859027427781674104e+0 (-2.038e-17%) + 2.4323666240267878020e-2 (1.982e-15%)j)
```

8.7.2 Weierstrass function, first derivative: $\wp' z|\tau)$

`ctx.weierstrass_p_prime_t(`z, *tau*`)`

Computes the first derivative of Weierstrass's elliptic function in terms of half-period ω_1 and elliptic period ratio τ.

We have $\wp'(tz|t\omega,\tau) = t^{-3}\wp'(z|\omega,\tau)$.

The Weierstrass elliptic function satisfies the differential equation $[\wp'(z,\tau)]^2 = 4[\wp(z,\tau)]^3 - g_2\wp(z,\tau) - g_3$.

$$\wp'(z,\tau) = \sqrt{4[\wp(z,\tau)]^3 - g_2\wp(z,\tau) - g_3}.$$

See also MathWorld [993], Flint [770].

See also: https://dlmf.nist.gov/23.2

An example with real input for z and with purely imaginary input for τ, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; z = '0.1'; tau = '0.0 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p_prime(z, tau); mx = mpm.weierstrass_p_prime(z, tau)
>>> gx = gmp.weierstrass_p_prime(z, tau)
>>> fx = fpm.weierstrass_p_prime(z, tau); ax = apm.weierstrass_p_prime(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -1.997612036772329460780279347633189585828E+3
mpm:  -1.997612036772329460780279347633189585828e+3
gmp:  -1.997612036772329460780279347633189585828E+03
fpm:  -1.99761203677233E+03
apm:  -1.997612036772329460780279347633189585828e+3 (-1.883e-38%)
```

An example with complex input for z and τ:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '0.1 + 0.5j'; tau = '0.7 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p_prime(z, tau); mx = mpm.weierstrass_p_prime(z, tau)
>>> gx = gmp.weierstrass_p_prime(z, tau)
>>> fx = fpm.weierstrass_p_prime(z, tau); ax = apm.weierstrass_p_prime(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 1.4787544521497118400E+1
mpm: 1.4787544521497118400e+1
gmp: 1.4787544521497118400E+01
fpm: 1.47875445214971E+01
apm: 1.4787544521497118399e+1 (5.664e-17%) - 2.3205214399360215619e+1 (-3.609e-17%)j
```

8.7.3 Inverse Weierstrass function $\wp^{-1}(z|\tau)$

`ctx.`**`weierstrass_p_inv_t`**`(`z, *tau*`)`

Computes the inverse of the Weierstrass elliptic function in terms of half-period ω_1 and elliptic period ratio τ. It which satisfies $\wp(\wp^{-1}(z,\tau),\tau) = z$. This function is given by the elliptic integral

$$\wp^{-1}(z,\tau) = \frac{1}{2}\int_z^\infty \frac{\mathrm{d}t}{\sqrt{(t-e_1)(t-e_2)(t-e_3)}} = R_F(z-e_1, z-e_2, z-e_3).$$

See also MathWorld [988], Flint [770].

See also: https://dlmf.nist.gov/23.2

An example with real input for z and with purely imaginary input for τ, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; z = '100'; tau = '0.0 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p_inv(z, tau); mx = mpm.weierstrass_p_inv(z, tau)
>>> gx = gmp.weierstrass_p_inv(z, tau)
>>> fx = fpm.weierstrass_p_inv(z, tau); ax = apm.weierstrass_p_inv(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.000602721184701332310197413726890982085E-1
mpm:  1.000602721184701332310197413726890982085e-1
gmp:  1.000602721184701332310197413726890982085E-01
fpm:  1.00060272118470E-01
apm:  1.000602721184701332310197413726890982085e-1 (7.17e-40%)
```

An example with complex input for z and τ:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '0.1 + 0.5j'; tau = '0.7 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p_inv(z, tau); mx = mpm.weierstrass_p_inv(z, tau)
>>> gx = gmp.weierstrass_p_inv(z, tau)
>>> fx = fpm.weierstrass_p_inv(z, tau); ax = apm.weierstrass_p_inv(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 4.2630465458606458544E-1              - 3.0967048559182771279E-1j
mpm: 4.2630465458606458544e-1              - 3.0967048559182771279e-1j
gmp: 4.2630465458606458544E-01             - 3.0967048559182771279E-01j
fpm: 4.26304654586065E-01                  - 3.09670485591828E-01j
apm: 4.2630465458606458544e-1 (1.265e-16%) - 3.0967048559182771279e-1 (-1.063e-16%)j
```

8.7.4 Weierstrass Zeta function $\zeta(z,\tau)$

`ctx.weierstrass_zeta_t(`*z*, *tau*`)`

Computes the Weierstrass zeta function in terms of half-period ω_1 and elliptic period ratio τ.

We have $\zeta(tz|t\omega,\tau) = t^{-1}\zeta(z|\omega,\tau)$.

The function can be defined as

$$\zeta(z|\omega,\tau) = \frac{1}{z} + \sum_{n^2+m^2\neq 0}\left[\frac{1}{z-m-n\tau} + \frac{1}{m+n\tau} + \frac{z}{(m+n\tau)^2}\right]$$

and is quasiperiodic with $\zeta(z+1|\omega,\tau) = \zeta(z|\omega,\tau) + \zeta(1/2|\omega,\tau)$ and $\zeta(z+\tau|\omega,\tau) = \zeta(z|\omega,\tau) + \zeta(\tau/2|\omega,\tau)$.

The function is related to $\wp(z|\omega,\tau)$ by $\dfrac{d\zeta(z|\omega,\tau)}{\mathrm{d}z} = -\wp(z|\omega,\tau)$ and $\zeta(z|\omega,\tau) - z^{-1} = \displaystyle\int_0^z \left(\wp(z|\omega,\tau) - z^{-2}\right)$.

See also MathWorld [995], Flint [770].

See also: https://dlmf.nist.gov/23.2

To evaluate the function efficiently, we use the formula

$$\zeta(z|\omega,\tau) = \frac{\pi z}{3}\frac{\theta_1'''(0,q)}{\theta_1'(0,q)} + \frac{\theta_1'(\pi z,q)}{\theta_1(\pi z,q)}$$

An example with real input for z and with purely imaginary input for τ, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; z = '0.1'; tau = '0.0 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_zeta(z, tau); mx = mpm.weierstrass_zeta(z, tau)
>>> gx = gmp.weierstrass_zeta(z, tau)
>>> fx = fpm.weierstrass_zeta(z, tau); ax = apm.weierstrass_zeta(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.995978010353406976694591385190992698486E+0
mpm:  9.995978010353406976694591385190992698486e+0
gmp:  9.995978010353406976694591385190992698486E+00
fpm:  9.99597801035341E+00
apm:  9.995978010353406976694591385190992698486e+0 (1.654e-38%)
```

An example with complex input for z and τ:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '0.1 + 0.5j'; tau = '0.7 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_zeta(z, tau); mx = mpm.weierstrass_zeta(z, tau)
>>> gx = gmp.weierstrass_zeta(z, tau)
>>> fx = fpm.weierstrass_zeta(z, tau); ax = apm.weierstrass_zeta(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 6.3880720497021003302E-1              - 2.0488020595059115066E+0j
mpm: 6.3880720497021003302e-1              - 2.0488020595059115066e+0j
gmp: 6.3880720497021003302E-01             - 2.0488020595059115066E+00j
fpm: 6.38807204970210E-01                  - 2.04880205950591E+00j
apm: 6.3880720497021003302e-1 (4.323e-17%) - 2.0488020595059115066e+0 (-1.356e-17%)j
```

8.7.5 Weierstrass Sigma function $\sigma(z|\tau)$

`ctx.weierstrass_sigma_t(`z, *omega*, *tau*`)`

Computes the Weierstrass sigma function in terms of half-period ω_1 and elliptic period ratio τ. We have $\sigma(tz|t\omega,\tau) = t\sigma(z|\omega,\tau)$.

See also MathWorld [994], Flint [770].

See also: https://dlmf.nist.gov/23.2

The function can be defined as

$$\sigma(z|\omega,\tau) = z \prod_{n^2+m^2\neq 0} \left[\left(1-\frac{z}{m+n\tau}\right)\exp\left(\frac{z}{m+n\tau}+\frac{z^2}{2(m+n\tau)^2}\right)\right]$$

and is quasiperiodic with $\sigma(z+1|\omega,\tau) = -e^{2\zeta(1/2|\omega,\tau)(z+1/2)}\sigma(z|\omega,\tau)$ and $\sigma(z+\tau|\omega,\tau) = -e^{2\zeta(\tau/2|\omega,\tau)(z+\tau/2)}\sigma(z|\omega,\tau)$.

The function is related to $\zeta(z|\omega,\tau)$ by $\frac{d}{dz}\log\sigma(z|\omega,\tau) = \zeta(z|\omega,\tau)$.

See also MathWorld [994], Flint [770].

To evaluate the function efficiently, we use the formula

$$\sigma(z,\tau) = \exp\left(-\frac{(\pi z)^2}{6}\frac{\theta_1''(0,q)}{\theta_1'(0,q)}\right)\times\frac{\theta_1(\pi z,q)}{\theta_1'(\pi z,q)}$$

An example with real input for z and with purely imaginary input for τ, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; z = '0.1'; tau = '0.0 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_sigma(z, tau); mx = mpm.weierstrass_sigma(z, tau)
>>> gx = gmp.weierstrass_sigma(z, tau)
>>> fx = fpm.weierstrass_sigma(z, tau); ax = apm.weierstrass_sigma(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.998992989830383649264433694319757846077E-2
mpm:  9.998992989830383649264433694319757846077e-2
gmp:  9.998992989830383649264433694319757846077E-02
fpm:  9.99899298983038E-02
apm:  9.998992989830383649264433694319757846077e-2 (5.023e-39%)
```

An example with complex input for z and τ:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '0.1 + 0.5j'; tau = '0.7 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_sigma(z, tau); mx = mpm.weierstrass_sigma(z, tau)
>>> gx = gmp.weierstrass_sigma(z, tau)
>>> fx = fpm.weierstrass_sigma(z, tau); ax = apm.weierstrass_sigma(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 8.505200635260751509 6E-2             + 5.106589225335005929 0E-1j
mpm: 8.505200635260751509 6e-2             + 5.106589225335005929 0e-1j
gmp: 8.505200635260751509 6E-02            + 5.106589225335005929 0E-01j
fpm: 8.50520063526075E-02                  + 5.10658922533501E-01j
apm: 8.505200635260751509 6e-2 (3.71e-17%) + 5.106589225335005929 0e-1 (6.22e-18%)j
```

8.8 Modular forms, in terms of half-period ω_1 and elliptic period ratio τ

8.8.1 Dedekind eta function $\eta(\tau)$

`ctx.dedekind_eta(tau)`

Returns Dedekind η in terms of elliptic period ratio τ. . See also Flint [765], MathWorld [974], Mathworld, equation 20:

$$\eta(\tau) = \frac{\theta_2(\pi/6, \bar{q}^{1/6})}{\sqrt{3}}, \quad \text{where } \bar{q} = q^2 = e^{2i\pi\tau}.$$

See also: https://dlmf.nist.gov/23.15#ii

Returns the Dedekind eta function $\eta(ix)$ for $x \geq 0$, with $\eta(x) = q^{1/24}(q)_\infty$, and $(q)_\infty$ is is the q-Pochhammer Euler function.

See also: MathWorld [974], Ehrhardt [297] (3.2.17.10).

An example with purely imaginary input, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; tau = '0.0 + 0.7j'
>>> \mathrm{d}x = dec.dedekind_eta(tau); mx = mpm.dedekind_eta(tau); gx = gmp.dedekind_eta(tau)
>>> fx = fpm.dedekind_eta(tau); ax = apm.dedekind_eta(tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 8.22186447762493341411764649497245858116 0E-1
mpm: 8.22186447762493341411764649497245858116 0e-1
gmp: 8.22186447762493341411764649497245858116 0E-01
fpm: 8.22186447762493E-01
apm: 8.22186447762493335609213823568505737654 2e-1 (2.094e-39%) + 0.0e+0 (0.0%)j
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; tau = '0.4 + 0.7j'
>>> \mathrm{d}z = dec.dedekind_eta(tau); mz = mpm.dedekind_eta(tau); gz = gmp.dedekind_eta(tau)
>>> fz = fpm.dedekind_eta(tau); az = apm.dedekind_eta(tau)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 8.3680777459719110894E-1              + 8.2020616070546877819E-2j
mpm: 8.3680777459719110894e-1              + 8.2020616070546877819e-2j
gmp: 8.3680777459719110894E-01             + 8.2020616070546877819E-02j
fpm: 8.3680777459719 1E-01                 + 8.2020616070546 9E-02j
apm: 8.3680777459719109849e-1 (3.543e-19%) + 8.2020616070546884265e-2 (2.84e-18%)j
```

8.8.2 Elliptic modular lambda function $\lambda(\tau)$ (also DAMath)

`ctx.ctx.elliptic_modular_lambda(tau)`

Computes the lambda function $\lambda(\tau) = \theta_2^4(0,\tau)/\theta_3^4(0,\tau)$ in terms of elliptic period ratio τ. It is invariant under modular transformations $(a, b; c, d)$ where a, d are odd and b, c are even.

See also MathWorld [976], Flint [769].

See also: https://dlmf.nist.gov/23.15#ii

Returns the elliptic modular function $\lambda(\tau), \tau = iy, y \geq 0$, $\lambda(\tau) = \dfrac{\theta_2^4(0,q)}{\theta_e^4(0,q)}, q = e^{i\pi\tau} = e^{-\pi y}$.

See also: MathWorld [976], Ehrhardt [297] (3.2.17.11).

An example with purely imaginary input, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; tau = '0.0 + 0.7j'
>>> \mathrm{d}x = dec.modular_lambda(tau); mx = mpm.modular_lambda(tau); gx = gmp.modular_lambda(tau)
```

(continues on next page)

(continued from previous page)

```
>>> fx = fpm.modular_lambda(tau); ax = apm.modular_lambda(tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 8.353354215017565686693789102199828213560E-1
mpm: 8.353354215017565686693789102199828213560e-1
gmp: 8.353354215017565686693789102199828213560E-01
fpm: 8.35335421501756E-01
apm: 8.353354215017565686693789102199828213560e-1 (1.374e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; tau = '0.4 + 0.7j'
>>> \mathrm{d}z = dec.modular_lambda(tau); mz = mpm.modular_lambda(tau); gz = gmp.modular_lambda(tau)
>>> fz = fpm.modular_lambda(tau); az = apm.modular_lambda(tau)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 1.058860386531187060 7E+0              + 5.885578742256214064 7E-1j
mpm: 1.058860386531187060 7e+0              + 5.885578742256214064 7e-1j
gmp: 1.058860386531187060 7E+00             + 5.885578742256214064 7E-01j
fpm: 1.05886038653119E+00                   + 5.88557874225621E-01j
apm: 1.058860386531187060 7e+0 (1.04e-18%) + 5.885578742256214064 7e-1 (1.583e-18%)j
```

8.8.3 Elliptic modular delta function $\Delta(\omega, \tau)$

`ctx.ctx.elliptic_modular_delta`(*omega*, *tau*)

Computes the modular discriminant $\Delta(\tau) = \eta(\tau)^{24}$ in terms of half-period ω_1 and elliptic period ratio τ. It transforms as

$$\Delta\left(\frac{a\tau+b}{c\tau+d}\right) = (c\tau+d)^{12}\Delta(\tau).$$

The modular discriminant is sometimes defined with an extra factor $(2\pi)^{12}$, which we omit in this implementation.

We have $\Delta(t\omega, \tau) = t^{-12}\Delta(\omega, \tau)$.

$$\Delta = g_2^3 - 27g_3^2 = 4096\pi^{12}\eta(\tau)^{24}$$

for Weierstrass invariants g_2, g_3, and Dedekind eta function $\eta(\tau)$.

See also Wikipedia [1355], MathWorld [990], Flint [769].

See also: https://dlmf.nist.gov/23.3#i

Computes the modular discriminant $\Delta(\tau) = \eta(\tau)^{24}$ in terms of elliptic period ratio τ. It transforms as

$$\Delta(g_2, g_3) = g_2^3 - 27g_3^2 = \Delta(e_1, e_2, e_3) = 16(e_2-e_3)^2(e_3-e_1)^2(e_1-e_2)^2.$$

See also Wikipedia [1355], MathWorld [990], Flint [769].

An example with purely imaginary input, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; tau = '0.0 + 0.7j'
>>> \mathrm{d}x = dec.modular_delta(tau); mx = mpm.modular_delta(tau); gx = gmp.modular_delta(tau)
>>> fx = fpm.modular_delta(tau); ax = apm.modular_delta(tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 9.105159016440059772399614671325862712295E-3
mpm: 9.105159016440059772399614671325862712295e-3
gmp: 9.105159016440059772399614671325862712295E-03
fpm: 9.10515901644004E-03
apm: 9.105159016440058230175761728846878617935e-3 (2.167e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; tau = '0.4 + 0.7j'
>>> \mathrm{d}z = dec.modular_delta(tau); mz = mpm.modular_delta(tau); gz = gmp.modular_delta(tau)
>>> fz = fpm.modular_delta(tau); az = apm.modular_delta(tau)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -1.0898535260853354339E-2               + 1.1147639721385956418E-2j
mpm: -1.0898535260853354339e-2               + 1.1147639721385956418e-2j
gmp: -1.0898535260853354339E-02              + 1.1147639721385956418E-02j
fpm: -1.08985352608534E-02                   + 1.11476397213860E-02j
apm: -1.0898535260853353666e-2 (-9.411e-18%) + 1.1147639721385950997e-2 (9.082e-18%)j
```

8.8.4 Klein j-invariant $j(\tau)$ (also DAMath)

`ctx.klein_j(`*tau*`)`

Returns the Klein j-invariant in terms of elliptic period ratio τ. See also Wikipedia [1364], MathWorld [1006], NIST [476], Flint [769], Mpmath [687].

See also: https://dlmf.nist.gov/23.15#ii

Computes Klein's j-invariant $j(\tau)$ given τ in the upper half-plane. The function is normalized so that $j(i) = 1728$. We first move τ to the fundamental domain, which does not change the value of the function. Then we use the formula

$$j(\tau) = 1728\frac{g_2^3}{g_2^3 - 27g_3^2} = 32(\theta_2^8 + \theta_3^8 + \theta_4^8)^3/(\theta_2\theta_3\theta_4)^8$$

where $\theta_i = \theta_i(0, \tau)$.

Returns the Klein j-invariant $J(\tau), \tau = iy, y \geq 0$, $J(\tau) = \dfrac{\left(\theta_2^8(q) + \theta_3^8(q) + \theta_4^8(q)\right)^3}{54(\theta_1')^8(q)}, q = e^{i\pi\tau} = e^{-\pi y}$.

See also: Wikipedia [1364], MathWorld [1006], Ehrhardt [297] (3.2.17.12).

Returns the Klein j-invariant in terms of elliptic period ratio τ. See also Wikipedia [1364], MathWorld [1006], NIST [476], Flint [769].

$$j(g_1, g_2) = 1728\frac{g_2^3}{g_2^3 - 27g_3^2}$$

An example with purely imaginary input, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; tau = '0.0 + 0.7j'
>>> \mathrm{d}x = dec.kleinj(tau); mx = mpm.kleinj(tau); gx = gmp.kleinj(tau)
>>> fx = fpm.kleinj(tau); ax = apm.kleinj(tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 5.0231437141844694999020127368511412070 74E+0
mpm: 5.0231437141844694999020127368511412070 74e+0
gmp: 5.0231437141844694999020127368511412070 74E+00
fpm: 5.02314371418447E+00
apm: 5.0231437141844694999020127368511412070 74e+0 (1.554e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; tau = '0.4 + 0.7j'
>>> \mathrm{d}z = dec.kleinj(tau); mz = mpm.kleinj(tau); gz = gmp.kleinj(tau)
>>> fz = fpm.kleinj(tau); az = apm.kleinj(tau)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -4.1369846595151262778E-2               - 2.6193939695223693294E-1j
mpm: -4.1369846595151262778e-2               - 2.6193939695223693294e-1j
gmp: -4.1369846595151262778E-02              - 2.6193939695223693294E-01j
fpm: -4.13698465951511E-02                   - 2.61939396952237E-01j
apm: -4.1369846595151262777e-2 (-1.358e-16%) - 2.6193939695223693294e-1 (-2.158e-17%)j
```

CHAPTER

NINE

LERCH'S PHI AND RELATED

The Lerch transcendent generalizes various other functions:

The polygamma function (see *polygamma()*) is given by

$$\psi^{(n)}(z) = (-1)^{n+1} n!\, \Phi(1, n+1, z).$$

The polylogarithm (see *polylog()*) is given by

$$\mathrm{Li}_s(z) = z\Phi(z, s, 1)).$$

The Legendre chi function (see *legendre_chi()*) is given by

$$\chi_n(z) = 2^{-n} z \Phi(z^2, n, 1/2).$$

The Hurwitz zeta function (see *hurwitz()*) is given by

$$\zeta(s, a) = \Phi(1, s, a).$$

The Riemann zeta function (see *zeta()*) is given by

$$\zeta(s) = \Phi(1, s, 1).$$

The Dirichlet eta function (see *dirichlet_eta()*) is given by

$$\eta(s) = \Phi(-1, s, 1).$$

Various identities include:

$$\Phi(z, s, a) = z^n \Phi(z, s, a+n) + \sum_{k=0}^{n-1} \frac{z^k}{(k+a)^s}$$

and

$$\Phi(z, s-1, a) = \left(a + z\frac{\partial}{\partial z}\right) \Phi(z, s, a)$$

and

$$\Phi(z, s+1, a) = -\frac{1}{s}\frac{\partial}{\partial a}\Phi(z, s, a).$$

The source code for the C# tests in Visual Studio is divided by real and complex modules:

The source code for the C# tests in Visual Studio using real modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B09a_RealLerchPhi.cs,

The source code for the C# tests in Visual Studio using complex modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B09b_CplxLerchPhi.cs,

9.1 Lerch's transcendent and Lerch's zeta

9.1.1 Lerch's transcendent, $\Phi(x, s, a)$

`ctx.lerch_phi`(s, z, a)

where `ctx` is `math53` or `ctxflint`.

Returns the Lerch transcendent $\Phi(z, s, a)$. See also Wikipedia [1359], MathWorld [1007], NIST [12], Ehrhardt [297] (3.6.10), Flint [768], Mpmath [700].

The function is defined for $|z| < 1$ and $\Re a > 0$ by

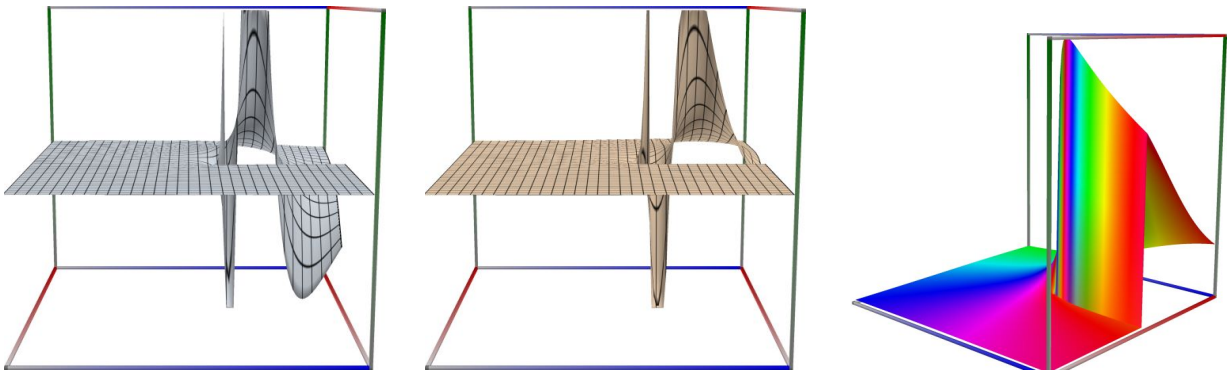
$$\Phi(z, s, a) = \sum_{k=0}^{\infty} \frac{z^k}{(a+k)^s}$$

and generally by the recurrence $\Phi(z, s, a) = z\Phi(z, s, a+1) + a^{-s}$ along with the integral representation valid for $\Re a > 0$

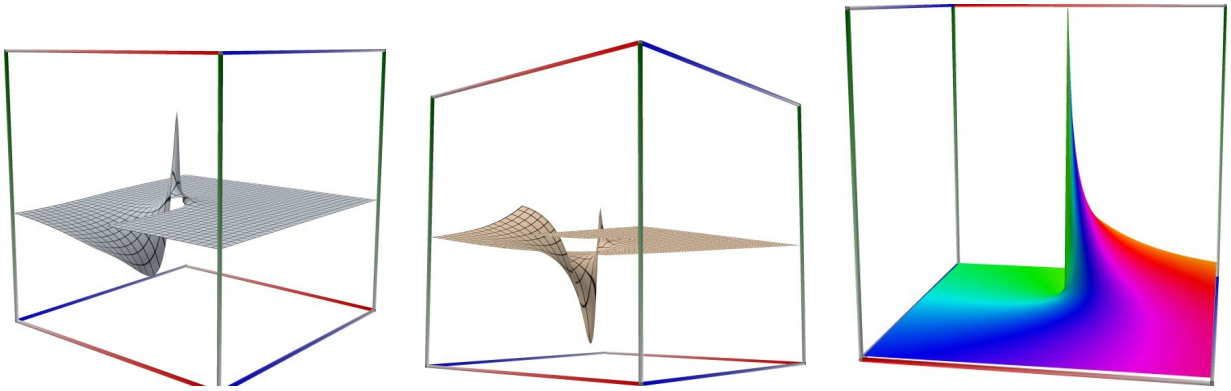
$$\Phi(z, s, a) = \frac{1}{2a^s} + \int_0^{\infty} \frac{z^t}{(a+t)^s} \mathrm{d}t - 2\int_0^{\infty} \frac{\sin(t\log z - s\arctan(t/a)}{(a^2+t^2)^{s/2}(e^{2\pi t}-1)} \mathrm{d}t.$$

The Amath implementation requires $x \le 1, s \ge -1, a \ge 0$. We have also $\Phi(x, 0, a) = 1/(1-x)$ and $\Phi(x, s, 0) = \mathrm{Li}_s(x)$.

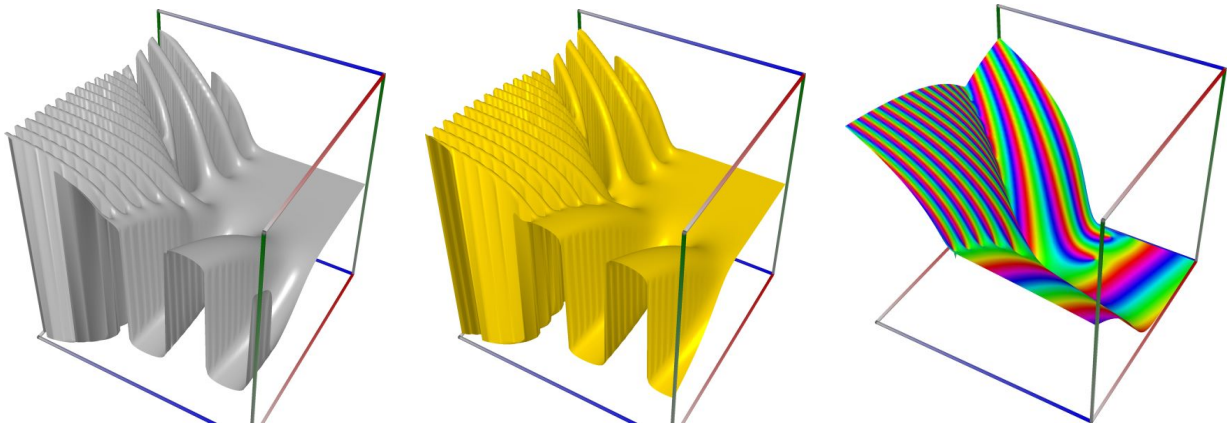
See also: https://fredrikj.net/blog/2022/02/computing-the-lerch-transcendent/



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LerchPhi(2,5, 0.5, 3)
ereal('5.2359877559829887307E-1')
>>> math53.LerchPhi('5.1', 0.5, 3)
ereal('5.3518479027559984754E-1')
```

An example with real input, $x = 1$ (same result as Hurwitz zeta):

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '1.0'; s = '2.5'; a = '3.0'
>>> \mathrm{d}x = dec.lerchphi(x, s, a); mx = mpm.lerchphi(x, s, a); gx = gmp.lerchphi(x, s, a)
>>> fx = fpm.lerchphi(x, s, a); ax = apm.lerchphi(x, s, a)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 1.647105619542802986565586028223998768018E-1
mpm: 1.647105619542802986565586028223998768018e-1
gmp: 1.647105619542802986565586028223998768018E-01
fpm: 1.64710561954280E-01
apm: 1.647105619542802986565586028223998768018e-1 (1.742e-39%)
```

An example with real input, $x < 1, s \geq 0, a > 0$ (real result for real x, s, a):

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '-11.5'; s = '2.5'; a = '7'
>>> \mathrm{d}x = dec.lerchphi(x, s, a); mx = mpm.lerchphi(x, s, a); gx = gmp.lerchphi(x, s, a)
>>> fx = fpm.lerchphi(x, s, a); ax = apm.lerchphi(x, s, a)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 8.683791984836949848109532607614130875396E-4
mpm: 8.683791984836949848109532607614130875396e-4
gmp: 8.683791984836949848109532607614130875396E-04
fpm: 8.68379198483695E-04
apm: 8.683791984836949848109532607614130875396e-4 (4.518e-39%)
```

An example with real input, complex result because $x > 1$:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '11.5'; s = '2.5'; a = '7'
>>> \mathrm{d}z = dec.lerchphi(z, s, a); mz = mpm.lerchphi(z, s, a); gz = gmp.lerchphi(z, s, a)
>>> fz = fpm.lerchphi(z, s, a); az = apm.lerchphi(z, s, a)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -1.1467900536878642113E-3               - 3.3910941716939727021E-7j
mpm: -1.1467900536878642113e-3               - 3.3910941716939727021e-7j
gmp: -1.1467900536878642113E-03              - 3.3910941716939727021E-07j
```

(continues on next page)

(continued from previous page)

```
fpm: -1.14679005368786E-03                   - 3.39109417169397E-07j
apm: -1.1467900536878642113e-3 (-3.607e-19%) - 3.391094171693972702 1e-7 (-2.263e-18%)j
```

An example with complex input for z and s, but integer a:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '11.0 + 2.0j'; s = '12.0 + 3.0j'; a = '7'
>>> \mathrm{d}z = dec.lerchphi(z, s, a); mz = mpm.lerchphi(z, s, a); gz = gmp.lerchphi(z, s, a)
>>> fz = fpm.lerchphi(z, s, a); az = apm.lerchphi(z, s, a)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -1.37197530011373 08013E-10              - 4.4570178228236195866E-12j
mpm: -1.37197530011373 08013e-10              - 4.4570178228236195866e-12j
gmp: -1.37197530011373 08013E-10              - 4.4570178228236195866E-12j
fpm: -1.37197530011373E-10                    - 4.45701782282362E-12j
apm: -1.37197529647403 20417e-10 (-0.002002%) - 4.4570185825924147934e-12 (-0.06162%)j
```

Evaluation works for complex arguments for z, s, a, and $|z| \geq 1$, currently only for `mpm`:

```
>>> mpm.lerchphi(1+2j, 3-j, 4+2j)
(0.00202500995700990860053946 9 + 0.00332789753681355880743808 9j)
>>> mpm.lerchphi(-2,2,-2.5)
-12.28676272353094275265944
>>> mpm.lerchphi(10,10,10)
(-4.46213072710218570181734 9e-11 - 1.57517219898109621882348 1e-12j)
>>> mpm.lerchphi(10,10,-10.5)
(112658784011940.5605789002 - 498113185.5756221777743631j)
```

9.1.2 Lerch's zeta

`ctx.lerch_zeta`(*lambda1*, *alpha*, *s*)

Returns the lerch zeta function. See also Wikipedia [1359], Apostol [11], Ferreira and López [301].

The Lerch zeta function is given by

$$L(\lambda_1, \alpha, s) = \sum_{n=0}^{\infty} \frac{e^{2\pi i \lambda n}}{(n+\alpha)^s}.$$

The Lerch zeta function is related to the Lerch transcendent by

$$L(\lambda_1, \alpha, s) = \Phi(e^{2\pi i \lambda_1}, s, \alpha).$$

For λ_1 rational, $L(\lambda_1, \alpha, s)$ may be expressed as a finite sum over the Hurwitz zeta-function. Suppose $\lambda_1 = \frac{p}{q}$ with $p, q \in \mathbb{Z}$ and $q > 0$. Then $z = \omega = e^{2\pi i \frac{p}{q}}$ and $\omega^q = 1$.

$$L(\lambda_1, \alpha, s) = \Phi(\omega, s, \alpha) = \sum_{n=0}^{\infty} \frac{\omega^n}{(n+\alpha)^s} = \sum_{m=0}^{q-1} \omega^m q^{-s} \zeta\left(s, \frac{m+\alpha}{q}\right)$$

An example with complex input for λ_1 and s, but integer α:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; lambda1 = '1.0 + 0.5j'; s = '1.0 + 0.5j'; alpha = '1'
>>> \mathrm{d}z = dec.lerch_zeta(lambda1, alpha, s); mz = mpm.lerch_zeta(lambda1, alpha, s)
>>> gz = gmp.lerch_zeta(lambda1, alpha, s)
>>> fz = fpm.lerch_zeta(lambda1, alpha, s); az = apm.lerch_zeta(lambda1, alpha, s)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 1.0208691797881172335E+0              - 7.6777982868597598415E-3j
mpm: 1.0208691797881172335e+0              - 7.6777982868597598415e-3j
gmp: 1.0208691797881172335E+00             - 7.6777982868597598415E-03j
fpm: 1.02086917978812E+00                  - 7.67779828685976E-03j
apm: 1.0208691797881172333e+0 (5.128e-17%) - 7.6777982868597598352e-3 (-4.862e-16%)j
```

9.2 Polygamma and related functions

9.2.1 Polygamma function, $\psi^{(n)}(x)$

`ctx.polygamma(`n, x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the polygamma function $\psi^{(n)}(x), \quad x \neq 0, -1, -2, \ldots$ The function returns the Hurwitz zeta value $(-1)^{n+1} n! \zeta(n+1, x)$ if x is positive; for $x < 0$ it is calculated from $\psi^{(n)}(1-x) + (-1)^{n+1}\psi^{(n)}(x) = (-1)^n \pi \frac{\mathrm{d}^n}{\mathrm{d}x^n} \cot(\pi x)$.

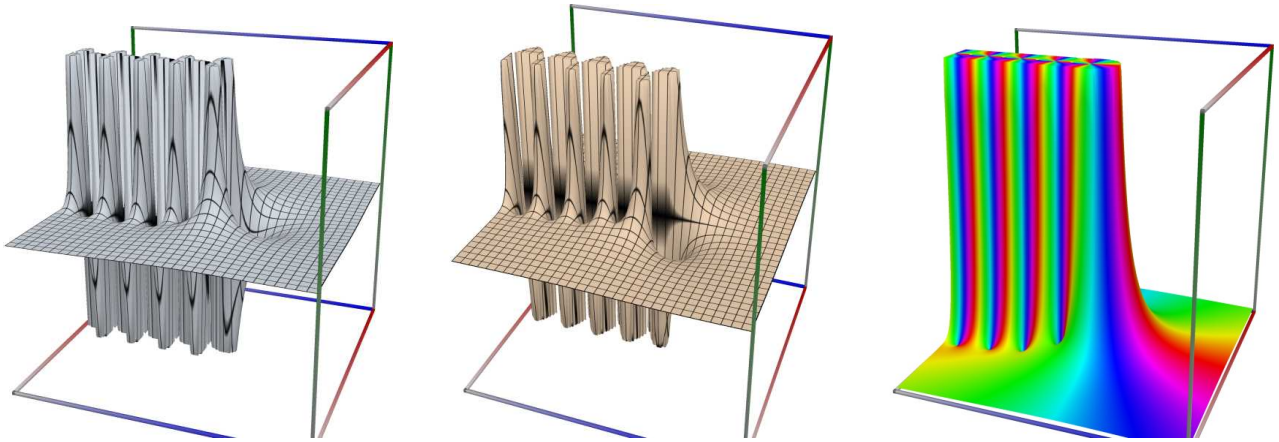
See also Wikipedia [1267], MathWorld [893], NIST [23], BoostMath [109], Ehrhardt [297] (3.5.6.6), Flint [729], Mpmath [598].

This function computes the polygamma function $\psi^{(n)}(x)$, i.e. the n^{th} derivative of the ψ function, with $n \geq 0$ and $x \neq 0, -1, -2, \ldots$. For $x > 0$ the result is calculated as

$$\psi^{(n)}(x) = (-1)^{n+1} n!\, \Phi(1, n+1, x) = (-1)^{n+1} n! \zeta(n+1, x).$$

The generalization to other values of s is due to Espinosa and Moll (see Espinosa and Moll [298]) :

$$\psi(s, z) = \frac{\zeta'(s+1, z) + (\gamma + \psi(-s))\zeta(s+1, z)}{\Gamma(-s)}$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

> **Note**
>
> This function is called `psi` in mpmath.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.PolyGamma(6, 7.1)
ereal('5.2359877559829887307E-1')
>>> math53.PolyGamma(12, '4.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; m = '10'; x = '5.0'
>>> \mathrm{d}x = dec.polygamma(m, x); mx = mpm.polygamma(m, x); gx = gmp.polygamma(m, x)
>>> fx = fpm.polygamma(m, x); ax = apm.polygamma(m, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -8.675107579196581317296465584299728817903E-2
mpm:  -8.675107579196581317296465584299728817903e-2
gmp:  -8.675107579196581317296465584299728817903E-02
fpm:  -8.67510757919658E-02
apm:  -8.675107579196581317296465584299728817903e-2 (-8.27e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; m = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.polygamma(m, z); mz = mpm.polygamma(m, z); gz = gmp.polygamma(m, z)
>>> fz = fpm.polygamma(m, z); az = apm.polygamma(m, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -1.3604463879878727646E-2               - 8.1339079821692487285E-3j
mpm: -1.3604463879878727646e-2               - 8.1339079821692487285e-3j
gmp: -1.3604463879878727646E-02              - 8.1339079821692487285E-03j
fpm: -1.36044638798787E-02                   - 8.13390798216925E-03j
apm: -1.3604463879878727646e-2 (-4.864e-20%) - 8.1339079821692487285e-3 (-8.136e-20%)j
```

9.2.2 TriGamma function, $\psi'(x)$

`ctx.trigamma(x)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the trigamma function $\psi'(x), \quad x \neq 0, -1, -2, \ldots$

The function returns the Hurwitz zeta value $\zeta(2, x)$ if x is positive; for $x < 0$ the polygamma reflection formula for $n = 1$ is used to compute the result

$$\psi'(x) = \left(\frac{\pi}{\sin(\pi x)}\right)^2 - \zeta(2, 1 - x).$$

See also Wikipedia [1294], MathWorld [937], NIST [23], BoostMath [131], Ehrhardt [297] (3.5.6.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.TriGamma(7)
ereal('5.235987755982988730 7E-1')
>>> math53.TriGamma('4.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '5.0'
>>> \mathrm{d}x = dec.trigamma(x); mx = mpm.trigamma(x); gx = gmp.trigamma(x)
>>> fx = fpm.trigamma(x); ax = apm.trigamma(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.213229557371153253613040555349140781078E-1
mpm:  2.213229557371153253613040555349140781078e-1
gmp:  2.213229557371153253613040555349140781078E-01
fpm:  2.21322955737115E-01
apm:  2.213229557371153253613040555349140781078e-1 (6.483e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.trigamma(z); mz = mpm.trigamma(z); gz = gmp.trigamma(z)
>>> fz = fpm.trigamma(z); az = apm.trigamma(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.5394122582225896863E-1              - 1.0204851974302677197E-1j
mpm: 1.5394122582225896863e-1              - 1.0204851974302677197e-1j
gmp: 1.5394122582225896863E-01             - 1.0204851974302677197E-01j
fpm: 1.53941225822259E-01                  - 1.02048519743027E-01j
apm: 1.5394122582225896863e-1 (1.376e-19%) - 1.0204851974302677197e-1 (-1.038e-19%)j
```

9.2.3 DiGamma function $\psi(x)$

`ctx.digamma(x)`

where `ctx` is `math53`, `mathc53`, `ctxboost`, `ctxflint`.

Returns the digamma function $\psi(x) = \dfrac{\mathrm{d}(\log\Gamma(x)}{\mathrm{d}x} = \dfrac{\Gamma'(x)}{\Gamma(x)}, \quad x \neq 0, -1, -2, \ldots$.

See also Wikipedia [1275], MathWorld [917], NIST [23], BoostMath [117], Ehrhardt [297] (3.5.6.1), Ehrhardt [297] (4.2.50), Flint [739], Flint [729], Mpmath [606].

This function returns the digamma or ψ function, which is defines as

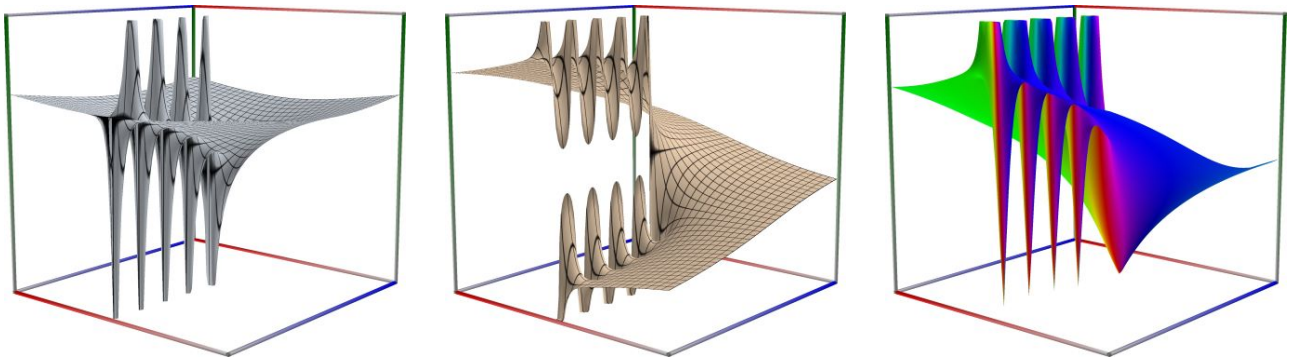
$$\psi(x) = \frac{d(\log\Gamma(x))}{\mathrm{d}x} = \frac{\Gamma'(x)}{\Gamma(x)}, \quad x \neq 0, -1, -2, \ldots$$

If $x < 0$ it is transformed to positive values with the reflection formula

$$\psi(1-x) = \psi(x) + \pi\cot(\pi x)$$

and for $0 < x < 12$ the recurrence formula

$$\psi(x+1) = \psi(x) + \frac{1}{x}$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Psi(7)
ereal('5.235987755982988730 7E-1')
>>> math53.Psi('4.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '5.0'
>>> \mathrm{d}x = dec.digamma(x); mx = mpm.digamma(x); gx = gmp.digamma(x)
>>> fx = fpm.digamma(x); ax = apm.digamma(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.5061176684318004727268212432509309022 91E+0
mpm:  1.5061176684318004727268212432509309022 91e+0
gmp:  1.5061176684318004727268212432509309022 91E+00
fpm:  1.50611766843180E+00
apm:  1.5061176684318004727268212432509309022 91e+0 (7.622e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.digamma(z); mz = mpm.digamma(z); gz = gmp.digamma(z)
>>> fz = fpm.digamma(z); az = apm.digamma(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.6884935312229713936E+0              + 5.8669378315167877376E-1j
mpm: 1.6884935312229713936e+0              + 5.8669378315167877376e-1j
gmp: 1.6884935312229713936E+00             + 5.8669378315167877376E-01j
fpm: 1.68849353122297E+00                  + 5.86693783151679E-01j
apm: 1.6884935312229713936e+0 (1.003e-19%) + 5.8669378315167877376e-1 (7.219e-20%)j
```

`ctx.`**`psi`**`(`*x*`)`

is an alias of `ctx.digamma(x)`.

9.2.4 Harmonic number function, H_x

`ctx.harmonic(x)`

Returns the Harmonic number $H_x = \psi(x+1) + \gamma, \quad x \neq -1, -2, \ldots$.

See also: Wikipedia [1284], MathWorld [923], Ehrhardt [297] (3.6.20), Mpmath [613].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Harmonic(10.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Harmonic('10.1')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '1.5'
>>> \mathrm{d}x = dec.harmonic(x); mx = mpm.harmonic(x); gx = gmp.harmonic(x)
>>> fx = fpm.harmonic(x); ax = apm.harmonic(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.2803723055467760478322024237503135305 16E+0
mpm:  1.2803723055467760478322024237503135305 16e+0
gmp:  1.2803723055467760478322024237503135305 16E+00
fpm:  1.28037230554678E+00
apm:  1.2803723055467760478322024237503135305 16e+0 (1.793e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.harmonic(z); mz = mpm.harmonic(z); gz = gmp.harmonic(z)
>>> fz = fpm.harmonic(z); az = apm.harmonic(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 2.4127680196539160189E+0              + 4.9845848903403171494E-1j
mpm: 2.4127680196539160189e+0              + 4.9845848903403171494e-1j
gmp: 2.4127680196539160189E+00             + 4.9845848903403171494E-01j
fpm: 2.41276801965392E+00                  + 4.98458489034032E-01j
apm: 2.4127680196539160189e+0 (1.404e-19%) + 4.9845848903403171494e-1 (4.248e-20%)j
```

9.3 Polylogarithm and related functions

9.3.1 Polylogarithm, $\mathrm{Li}_s(z)$

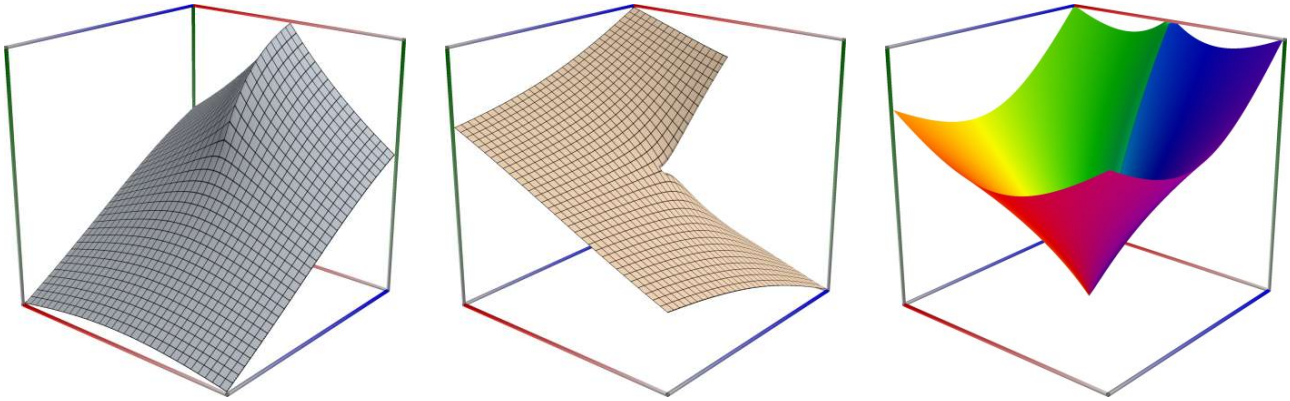
`ctx.polylog(`x, s`)`

where `ctx` is `math53`, `ctxflint`.

Returns the polylogarithm of real order s, $\mathrm{Li}_s(x) = \sum_{k=1}^{\infty} \frac{x^k}{k^s}$, $s \geq -1, |x| \leq 1$, or its analytic continuation. For $s \leq 1$ there is the additional arguments restriction $z \neq 1$.

See also Wikipedia [1332], MathWorld [969], NIST [18], Ehrhardt [297] (3.6.12), Flint [774], Mpmath [660].

Special cases are $\mathrm{Li}_s(0) = 0$, $\mathrm{Li}_s(1) = \zeta(s)$, $\mathrm{Li}_s(-1) = -\eta(s)$. For $x > 1$ the real part of $\mathrm{Li}_s(x)$ is returned.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.PolylogR(2, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.PolylogR(2, '0.1')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; s = '10'; x = '0.5'
>>> \mathrm{d}x = dec.polylog(s, x); mx = mpm.polylog(s, x); gx = gmp.polylog(s, x)
>>> fx = fpm.polylog(s, x); ax = apm.polylog(s, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  5.0024632060600677500967524049602755533 44E-1
mpm:  5.0024632060600677500967524049602755533 44e-1
gmp:  5.0024632060600677500967524049602755533 44E-01
fpm:  5.00246320606007E-01
apm:  5.0024632060600677500967524047567154631 82e-1 (4.658e-24%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; s = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.polylog(s, z); mz = mpm.polylog(s, z); gz = gmp.polylog(m, z)
>>> fz = fpm.polylog(m, z); az = apm.polylog(m, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
```

(continues on next page)

(continued from previous page)

```
dec: 5.0140954368998825423E+0             + 3.0330881889981661901E+0j
mpm: 5.0140954368998825423e+0             + 3.0330881889981661901e+0j
gmp: 5.0140954368998825423E+00            + 3.0330881889981661901E+00j
fpm: 5.01409543689988E+00                 + 3.03308818899817E+00j
apm: 5.0140954366116116588e+0 (1.145e-5%) + 3.0330881892210954918e+0 (2.687e-5%)j
```

9.3.2 Trilogarithm Function, $\mathrm{Li}_3(z)$

`ctx.trilog(`x`)`

Returns the trilogarithm $\mathrm{trilog}(x) = \Re\mathrm{Li}_3(x)$. See also Wikipedia [1336], MathWorld [955], NIST [16], Ehrhardt [297] (3.6.14), Mpmath [660].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Trilog(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Trilog('0.1')
ereal('5.351847902755998475 4E-1')
```

9.3.3 Dilogarithm Function, $\mathrm{Li}_2(z)$

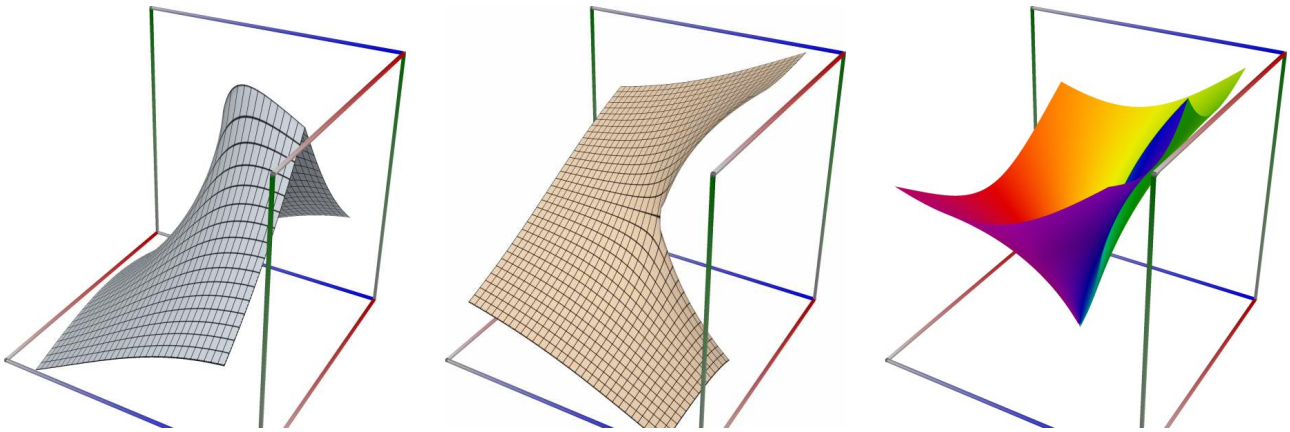
`ctx.dilog`(x)

where `ctx` is `math53`, `mathc53` or `ctxflint`.

Returns the dilogarithm $\mathrm{dilog}(x) = \Re\mathrm{Li}_2(x) = -\Re\int_0^x \frac{\log(1-t)}{t}\,\mathrm{d}t$.

See also Wikipedia [1336], MathWorld [955], NIST [16], Ehrhardt [297] (3.6.13), Ehrhardt [297] (4.2.25), Flint [773], Mpmath [660].

Note that there is some confusion about the naming: some authors and/or computer algebra systems use $\mathrm{dilog}(x) = \mathrm{Li}_2(1-x)$ and then call $\mathrm{Li}_2(x)$ Spence function/integral or similar.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Dilog(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Dilog('0.1')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '0.5'
>>> \mathrm{d}x = dec.dilog(x); mx = mpm.dilog(x); gx = gmp.dilog(x)
>>> fx = fpm.dilog(x); ax = apm.dilog(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  5.8224052646501250590265632015968010874 42E-1
mpm:  5.8224052646501250590265632015968010874 42e-1
gmp:  5.8224052646501250590265632015968010874 42E-01
fpm:  5.82240526465013E-01
apm:  5.8224052646501250590265632015968010874 42e-1 (9.858e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.dilog(z); mz = mpm.dilog(z); gz = gmp.dilog(z)
>>> fz = fpm.dilog(z); az = apm.dilog(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 3.3260008208192192025E-2              + 4.6816673723804519905E+0j
```

(continues on next page)

(continued from previous page)

```
mpm: 3.3260008208192192025e-2              + 4.6816673723804519905e+0j
gmp: 3.3260008208192192025E-02             + 4.6816673723804519905E+00j
fpm: 3.32600082081922E-02                  + 4.68166737238045E+00j
apm: 3.3260008208192192025e-2 (7.958e-20%) + 4.6816673723804519905e+0 (7.237e-20%)j
```

9.3.4 Generalized Clausen sine function

`ctx.clausen_sin`(s, z)

Returns the Clausen sine function. See also Wikipedia [1357], MathWorld [1020], Mpmath [699].

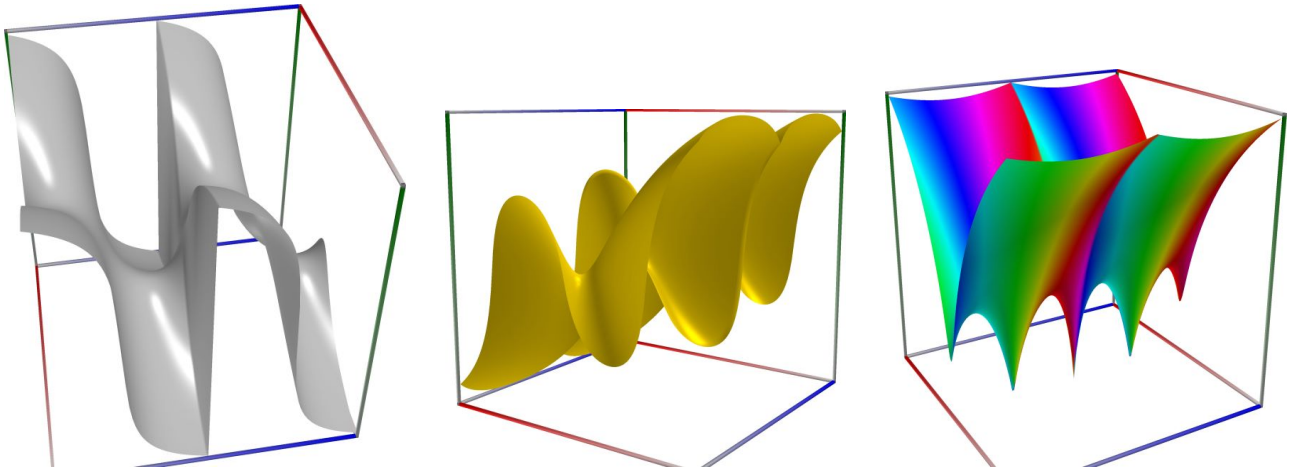
Computes the Clausen sine function, defined formally by the series

$$\mathrm{Cl}_s(z) = \sum_{k=1}^{\infty} \frac{\sin(kz)}{k^s}.$$

The special case $\mathrm{Cl}_2(z)$ (i.e. `clsin(2,z)`) is the classical "Clausen function". More generally, the Clausen function is defined for complex s and z, even when the series does not converge. The Clausen function is related to the polylogarithm (*polylog()*) as

$$\begin{aligned}\mathrm{Cl}_s(z) &= \frac{1}{2i}\left(\mathrm{Li}_s\left(e^{iz}\right) - \mathrm{Li}_s\left(e^{-iz}\right)\right) \\ &= \mathrm{Im}\left[\mathrm{Li}_s(e^{iz})\right] \quad (s, z \in \mathbb{R}),\end{aligned}$$

and this representation can be taken to provide the analytic continuation of the series. The complementary function *clcos()* gives the corresponding cosine sum.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; s = '3'; x = '4'
>>> \mathrm{d}x = dec.clsin(s, x); mx = mpm.clsin(s, x); gx = gmp.clsin(s, x)
>>> fx = fpm.clsin(s, x); ax = apm.clsin(s, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -6.5330101363293387462757953320057744657 95E-1
mpm:  -6.5330101363293387462757953320057744657 95e-1
gmp:  -6.5330101363293387462757953320057744657 95E-01
fpm:  -6.53301013632934E-01
apm:  -6.5330101363293387462757953320057748860 00e-1 (-2.242e-29%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; s = '3'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.clsin(s, z); mz = mpm.clsin(s, z); gz = gmp.clsin(s, z)
>>> fz = fpm.clsin(s, z); az = apm.clsin(s, z)
```

(continues on next page)

(continued from previous page)

```
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -5.17503361345137410 48E+0                - 2.12714270137876997 91E+0j
mpm: -5.17503361345137410 48e+0                - 2.12714270137876997 91e+0j
gmp: -5.17503361345137410 48E+00               - 2.12714270137876997 91E+00j
fpm: -5.17503361345137E+00                     - 2.12714270137877E+00j
apm: -5.17503361345137410 59e+0 (-3.604e-14%) - 2.12714270137876997 77e+0 (-7.287e-14%)j
```

9.3.5 Generalized Clausen cosine function

`ctx.clausen_cos`(s, z)

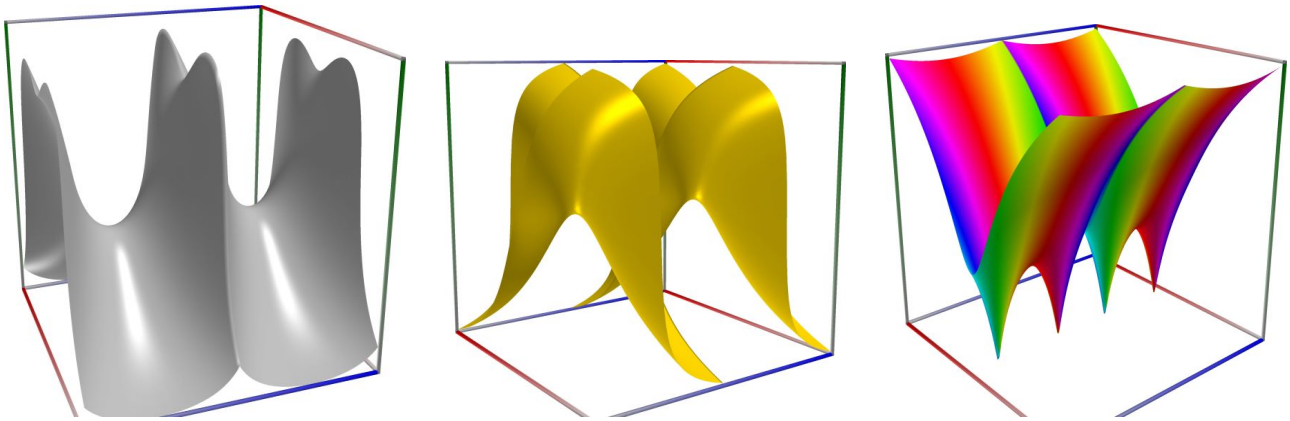
Returns the Clausen cosine function. See also Wikipedia [1357], MathWorld [1020], Mpmath [698].

Computes the Clausen cosine function, defined formally by the series

$$\widetilde{\mathrm{Cl}}_s(z) = \sum_{k=1}^{\infty} \frac{\cos(kz)}{k^s}.$$

This function is complementary to the Clausen sine function *clsin()*. In terms of the polylogarithm,

$$\begin{aligned}\widetilde{\mathrm{Cl}}_s(z) &= \frac{1}{2}\left(\mathrm{Li}_s\left(e^{iz}\right) + \mathrm{Li}_s\left(e^{-iz}\right)\right) \\ &= \mathrm{Re}\left[\mathrm{Li}_s(e^{iz})\right] \quad (s, z \in \mathbb{R}).\end{aligned}$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; s = '3'; x = '4'
>>> \mathrm{d}x = dec.clcos(s, x); mx = mpm.clcos(s, x); gx = gmp.clcos(s, x)
>>> fx = fpm.clcos(s, x); ax = apm.clcos(s, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -6.518926267198991308332758909517770104595E-1
mpm:  -6.518926267198991308332758909517770104595e-1
gmp:  -6.518926267198991308332758909517770104595E-01
fpm:  -6.51892626719899E-01
apm:  -6.518926267198991308332758909517763900500e-1 (-5.188e-29%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; s = '3'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.clcos(s, z); mz = mpm.clcos(s, z); gz = gmp.clcos(s, z)
>>> fz = fpm.clcos(s, z); az = apm.clcos(s, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -2.1132834047743479436E+0               + 5.1271260828837504331E+0j
mpm: -2.1132834047743479436e+0               + 5.1271260828837504331e+0j
gmp: -2.1132834047743479436E+00              + 5.1271260828837504331E+00j
fpm: -2.11328340477435E+00                   + 5.12712608288375E+00j
apm: -2.1132834047743479423e+0 (-7.335e-14%) + 5.1271260828837504341e+0 (3.638e-14%)j
```

9.3.6 Classical Clausen function, $\mathrm{Cl}_2(x)$

`ctx.clausen2(`x`)`

Returns the Clausen function $\mathrm{Cl}_2(x) = \Im\mathrm{Li}_2(e^{ix}) = \int_0^x \log|2\sin(t/2)|\,\mathrm{d}t$.

See also: Wikipedia [1357], MathWorld [1020], Ehrhardt [297] (3.6.15), Mpmath [698].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Clausen2(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Clausen2('0.1')
ereal('5.351847902755998475 4E-1')
```

9.3.7 Bose-Einstein integrals, $G_s(x)$

`ctx.bose_einstein(s, x)`

Returns the Bose-Einstein integral of real order s, $G_s(x) = \dfrac{1}{\Gamma(s+1)} \displaystyle\int_0^\infty \frac{t^s}{e^{t-x}-1} = \mathrm{Li}_{s+1}(e^x)$. If $x > 0$ the real part of $G_s(x)$ is returned.

See also: MathWorld [951], Ehrhardt [297] (3.6.7), NIST [15].

$$G_s(x) = \frac{1}{\Gamma(s+1)} \int_0^\infty \frac{t^s}{e^{t-x}-1} \mathrm{d}t,$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BoseEinstein(2,5)
ereal('5.235987755982988730 7E-1')
>>> math53.BoseEinstein(2,'51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; s = '10'; x = '0.5'
>>> \mathrm{d}x = dec.bose_einstein(s, x); mx = mpm.bose_einstein(s, x); gx = gmp.bose_einstein(s, x)
>>> fx = fpm.bose_einstein(s, x); ax = apm.bose_einstein(s, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.650075952950667128565820621131669931198E+0
mpm:  1.650075952950667128565820621131669931198e+0
gmp:  1.650075952950667128565820621131669931198E+00
fpm:  1.65007595295067E+00
apm:  1.650075952950667128565820642266282738366e+0 (2.139e-21%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; s = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.bose_einstein(s, z); mz = mpm.bose_einstein(s, z); gz = gmp.bose_einstein(s, z)
>>> fz = fpm.bose_einstein(s, z); az = apm.bose_einstein(s, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -1.4096051587148306166E+2             + 1.947665621569247173 8E+1j
mpm: -1.4096051587148306166e+2             + 1.947665621569247173 8e+1j
gmp: -1.4096051587148306166E+02            + 1.947665621569247173 8E+01j
fpm: -1.40960515871483E+02                 + 1.94766562156925E+01j
apm: -1.4096051581016388546e+2 (-7.08e-5%) + 1.947665621550704436 3e+1 (0.0007137%)j
```

9.3.8 Fermi-Dirac integrals, $F_s(x)$

`ctx.fermi_dirac(`s, x`)`

where `ctx` is `math53` or `ctxflint`.

Returns the Fermi-Dirac integral of order s, $F_s(x) = \dfrac{1}{\Gamma(s+1)} \displaystyle\int_0^\infty \frac{t^s}{e^{t-x}+1} = -\mathrm{Li}_{s+1}(-e^x)$.

See also: Wikipedia [1319], MathWorld [957], Ehrhardt [297] (3.6.8.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.FermiDiracR(2,5)
ereal('5.235987755982988730 7E-1')
>>> math53.FermiDiracR(2,'51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; s = '10'; x = '1.5'
>>> \mathrm{d}x = dec.fermi_dirac(s, x); mx = mpm.fermi_dirac(s, x); gx = gmp.fermi_dirac(s, x)
>>> fx = fpm.fermi_dirac(s, x); ax = apm.fermi_dirac(s, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.4723178120683686623507745267636538610 48E+0
mpm:  4.4723178120683686623507745267636538610 48e+0
gmp:  4.4723178120683686623507745267636538610 48E+00
fpm:  4.47231781206837E+00
apm:  4.4723178120683686623507745209460643837 82e+0 (1.711e-22%)
```

An example with complex input:

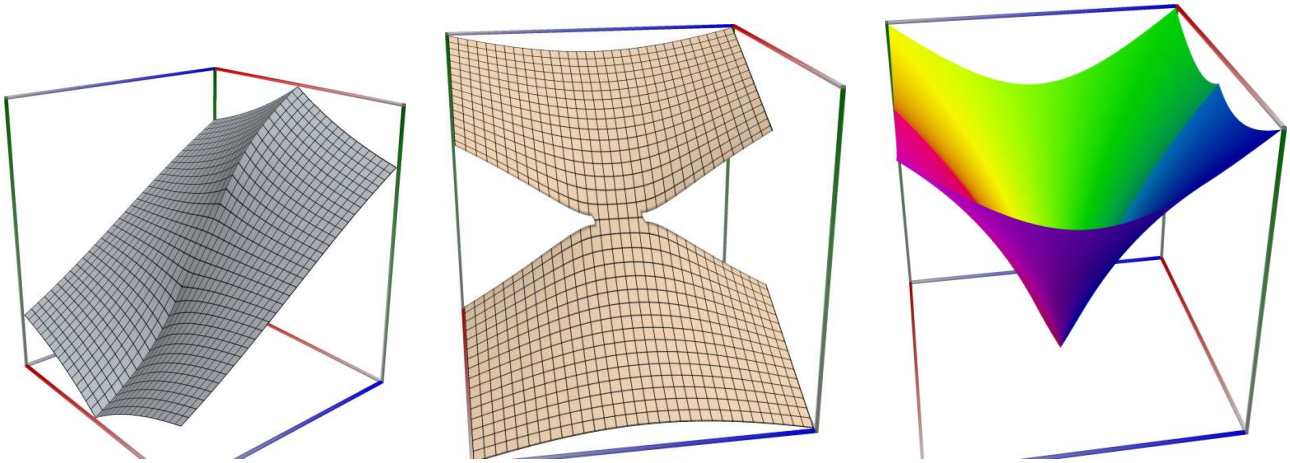
```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; s = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.fermi_dirac(s, z); mz = mpm.fermi_dirac(s, z); gz = gmp.fermi_dirac(s, z)
>>> fz = fpm.fermi_dirac(s, z); az = apm.fermi_dirac(s, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -1.5096075557263597965E+2               + 3.0204317063642472857E+1j
mpm: -1.5096075557263597965e+2               + 3.0204317063642472857e+1j
gmp: -1.50960755572635979 65E+02             + 3.0204317063642472857E+01j
fpm: -1.50960755572636E+02                   + 3.02043170636425E+01j
apm: -1.5096075564796857312e+2 (-7.353e-5%) + 3.0204317086095309026e+1 (0.0004834%)j
```

9.3.9 Legendre's Chi function, $\chi_s(x)$

`ctx.legendre_chi`(s, x)

Returns Legendre's Chi function, defined as $\chi_s(x) = \sum_{n=0}^{\infty} \frac{x^{2n+1}}{(2n+1)^s} = \frac{1}{2}\left(\mathrm{Li}_s(x) - \mathrm{Li}_s(-x)\right)$, for $s \geq 0, |x| \leq 1$.

See also: Wikipedia [1328], MathWorld [967], Ehrhardt [297] (3.6.9).



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LegendreChi(2,5, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.LegendreChi('5.1', 0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; s = '10'; x = '1.5'
>>> \mathrm{d}x = dec.legendre_chi(s, x); mx = mpm.legendre_chi(s, x); gx = gmp.legendre_chi(s, x)
>>> fx = fpm.legendre_chi(s, x); ax = apm.legendre_chi(s, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.5000580118286136445746632455954396762 43E+0
mpm:  1.5000580118286136445746632455954396762 43e+0
gmp:  1.5000580118286136445746632455954396762 43E+00
fpm:  1.50005801182861E+00
apm:  1.5000580118286136445746632465864593271 24e+0 (2.88e-22%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; s = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.legendre_chi(s, z); mz = mpm.legendre_chi(s, z); gz = gmp.legendre_chi(s, z)
>>> fz = fpm.legendre_chi(s, z); az = apm.legendre_chi(s, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 4.9993028299912224901E+0             + 3.0032350570493713371E+0j
mpm: 4.9993028299912224901e+0             + 3.0032350570493713371e+0j
gmp: 4.9993028299912224901E+00            + 3.0032350570493713371E+00j
fpm: 4.99930282999122E+00                 + 3.00323505704937E+00j
apm: 4.9993028304470985305e+0 (2.364e-5%) + 3.0032350577905899996e+0 (4.62e-5%)j
```

9.3.10 Generalized inverse tangent integral

`ctx.inverse_tan_integral`(s, z)

Returns the generalized inverse tangent integral $Ti_s(z)$, which can be expressed in terms of polylogarithms:

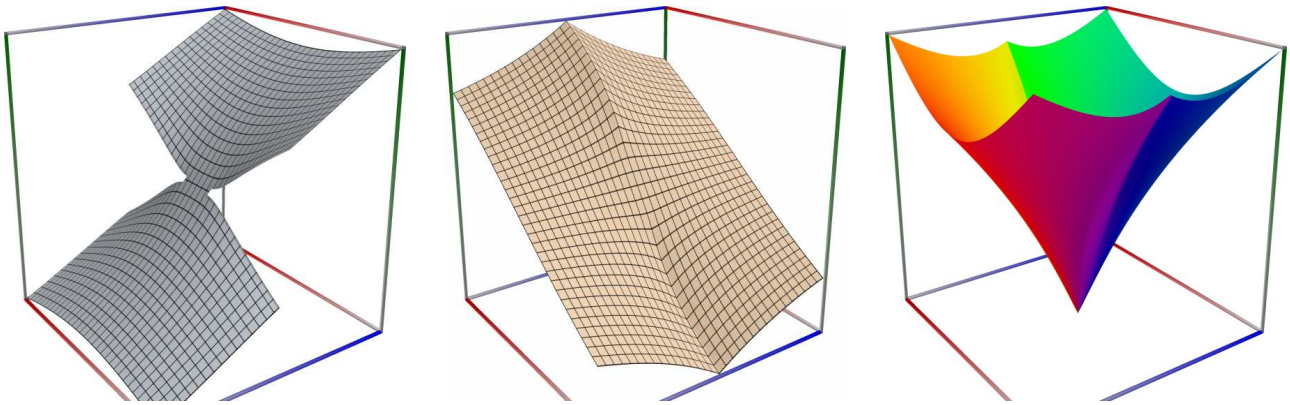
$$\mathrm{Ti}_s(z) = \frac{1}{2i}\left[\mathrm{Li}_s(iz) - \mathrm{Li}_s(-iz)\right].$$

The relation in particular implies:

$$\mathrm{Ti}_0(z) = \frac{z}{1+z^2}, \quad \mathrm{Ti}_1(z) = \arctan z, \quad \mathrm{Ti}_2(z) = \int_0^z \frac{\arctan t}{t}\mathrm{d}t, \quad \ldots \quad \mathrm{Ti}_{n+1}(z) = \int_0^z \frac{\mathrm{Ti}_n(t)}{t}\mathrm{d}t,$$

which explains the function name.

See also: Wikipedia [1333], MathWorld [963], Ehrhardt [297] (3.6.17).



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; s = '10'; x = '1.5'
>>> \mathrm{d}x = dec.ti(s, x); mx = mpm.ti(s, x); gx = gmp.ti(s, x)
>>> fx = fpm.ti(s, x); ax = apm.ti(s, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.4999435698220040123050796501160435725 38E+0
mpm:  1.4999435698220040123050796501160435725 38e+0
gmp:  1.4999435698220040123050796501160435725 38E+00
fpm:  1.49994356982200E+00
apm:  1.4999435698220040123050796499051891764 82e+0 (9.317e-24%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; s = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.ti(s, z); mz = mpm.ti(s, z); gz = gmp.ti(s, z)
>>> fz = fpm.ti(s, z); az = apm.ti(s, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 4.99986416234152 15E+0              + 2.99695525566636 45E+0j
mpm: 4.99986416234152 15e+0              + 2.99695525566636 45e+0j
gmp: 4.99986416234152 15E+00             + 2.99695525566636 45E+00j
fpm: 4.99986416234152E+00                + 2.99695525566636E+00j
apm: 4.99986416361496 85e+0 (3.517e-5%) + 2.99695525492615 24e+0 (4.218e-5%)j
```

9.4 Hurwitz zeta and related functions

9.4.1 Hurwitz zeta function, $\zeta(s, a)$

`ctx.hurwitz_zeta(`s, a`)`

where `ctx` is `math53`, `ctxflint`.

Returns the Hurwitz zeta function, defined as $\zeta(s,a) = \sum_{k=0}^{\infty} \frac{1}{(a+k)^s}$, $s > 1, a \neq 0, -1, -2, \ldots$, and by analytic continuation for $s \neq 0$. The amath implementation requires $a > 0$. If $a = 1$ then $\zeta(s)$ is returned, and if $s = 0$ the result is $\frac{1}{2} - a$.

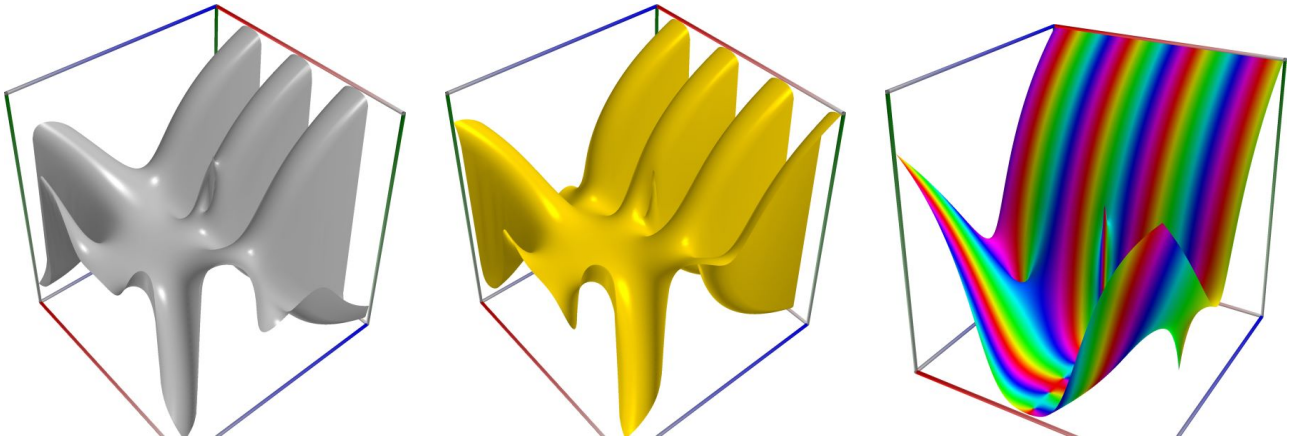
See also Wikipedia [1325], MathWorld [959], NIST [17], Ehrhardt [297] (3.6.6), Flint [761].

$$\zeta(-n, a) = -\frac{B_{n+1}(a)}{n+1}.$$

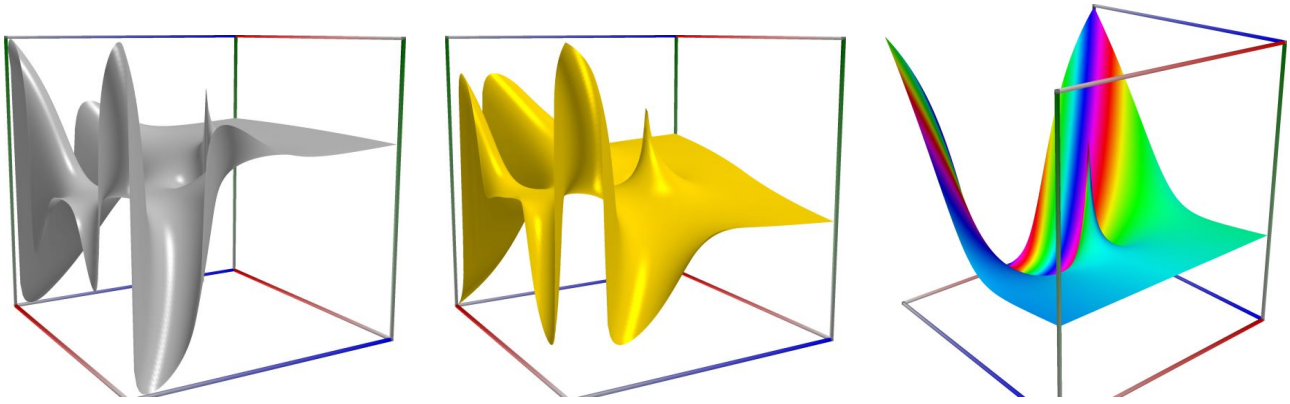
$$\zeta(s, 1) = \zeta(s).$$

$$\zeta\left(s, \tfrac{1}{2}\right) = (2^s - 1)\zeta(s).$$

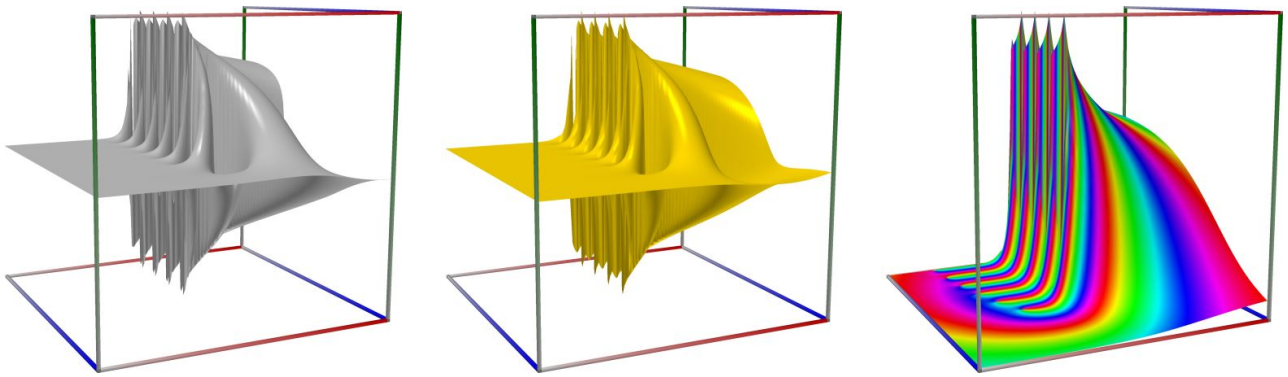
$$\zeta'(0, a) = \log\Gamma(a) - \tfrac{1}{2}\log(2\pi).$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.HurwitzZeta(2,5)
ereal('5.235987755982988730 7E-1')
>>> math53.HurwitzZeta(2,'51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; s = '10'; a = '1.5'
>>> \mathrm{d}x = dec.hurwitz(s, a); mx = mpm.hurwitz(s, a); gx = gmp.hurwitz(s, a)
>>> fx = fpm.hurwitz(s, a); ax = apm.hurwitz(s, a)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.7450355757901299900315955026354397157 98E-2
mpm:  1.7450355757901299900315955026354397157 98e-2
gmp:  1.7450355757901299900315955026354397157 98E-02
fpm:  1.74503557579013E-02
apm:  1.7450355757901299900315955026354397157 98e-2 (1.028e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; s = '10 + 5j'; a = '5.0 + 3j'
>>> \mathrm{d}z = dec.hurwitz(s, a); mz = mpm.hurwitz(s, a); gz = gmp.hurwitz(s, a)
>>> fz = fpm.hurwitz(s, a); az = apm.hurwitz(s, a)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -2.9834537360028672178E-8                - 3.9761878565611985695E-7j
mpm: -2.9834537360028672178e-8                - 3.9761878565611985695e-7j
gmp: -2.98345373600286721 78E-08              - 3.9761878565611985695E-07j
fpm: -2.98345373600287E-08                    - 3.97618785656120E-07j
apm: -2.9834537360028672177e-8 (-8.461e-20%) - 3.9761878565611985695e-7 (-5.079e-20%)j
```

9.4.2 Generalized harmonic number function, $H_x^{(r)}$

`ctx.`**`harmonic2`**(x, s)

Returns the generalized harmonic function $H_x^{(r)} = \zeta(r) - \zeta(r, x+1)$ for $r \neq 1$ and $H_x^{(r)} = H_x$ for $r = 1$.

See also: Wikipedia [1284], MathWorld [922], Ehrhardt [297] (3.6.21).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Harmonic2(5, 10.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Harmonic2(5, '10.1')
ereal('5.351847902755998 4754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '10'; r = '1.5'
>>> \mathrm{d}x = dec.harmonic2(x, r); mx = mpm.harmonic2(x, r); gx = gmp.harmonic2(x, r)
>>> fx = fpm.harmonic2(x, r); ax = apm.harmonic2(x, r)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.995336493345601714521693592714339476261E+0
mpm:  1.995336493345601714521693592714339476261e+0
gmp:  1.995336493345601714521693592714339476261E+00
fpm:  1.99533649334560E+00
apm:  1.995336493345601714521693592714339476261e+0 (1.668e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '10 + 5j'; r = '5.0 + 3j'
>>> \mathrm{d}z = dec.harmonic2(z, r); mz = mpm.harmonic2(z, r); gz = gmp.harmonic2(z, r)
>>> fz = fpm.harmonic2(z, r); az = apm.harmonic2(z, r)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 9.8046716104400925422E-1              - 2.5426375920749034566E-2j
mpm: 9.8046716104400925422e-1              - 2.5426375920749034566e-2j
gmp: 9.8046716104400925422E-01             - 2.5426375920749034566E-02j
fpm: 9.804671610440 09E-01                 - 2.54263759207490E-02j
apm: 9.8046716104400925422e-1 (8.639e-20%) - 2.5426375920749034566e-2 (-1.041e-19%)j
```

9.4.3 Bernoulli numbers, B_n

`ctx.bernoulli(n)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Note ctxboost.BernoulliB2n(n)

Returns the Bernoulli numbers B_n, which are defined by their generating function $\frac{1}{e^t - 1} \sum_{n=0}^{\infty} B_n \frac{t^n}{n!}$, $|t < 2\pi|$. If $n < 0$ or if $n > 2$ is odd, the result is 0, and $B_1 = -1/2$.

See also Wikipedia [1248], MathWorld [876], NIST [242], BoostMath [93], Ehrhardt [297] (3.10.2), Mpmath [581].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Bernoulli(8)
ereal('5.2359877559829887307E-1')
>>> math53.Bernoulli(14)
ereal('5.3518479027559984754E-1')
```

An example:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '16'
>>> \mathrm{d}x = dec.bernoulli(n); mx = mpm.bernoulli(n); gx = gmp.bernoulli(n)
>>> fx = fpm.bernoulli(n); ax = apm.bernoulli(n)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -7.0921568627450980392156862745098039215690E+0
mpm:  -7.0921568627450980392156862745098039215690e+0
gmp:  -7.0921568627450980392156862745098039215690E+00
fpm:  -7.09215686274510E+00
apm:  -7.0921568627450980392156862745098039215690e+0 (-1.295e-39%)
```

9.4.4 Bernoulli polynomials, $B_n(x)$

`ctx.bernpoly(n, x)`

Returns $B_n(x) = \sum_{n=0}^{\infty} \binom{n}{k} B_k x^{n-k}$, the Bernoulli polynomial of degree $n \geq 0$.

See also Wikipedia [1249], MathWorld [877], NIST [242], Ehrhardt [297] (3.10.3), Flint [726], Mpmath [582].

See the Amath documentation (p. 146) for connection formula to Hurwitz Zeta.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Bernpoly(3,4)
ereal('5.2359877559829887307E-1')
>>> math53.Bernpoly(13,14)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '16'; x = '1.5'
>>> \mathrm{d}x = dec.bernpoly(n, x); mx = mpm.bernpoly(n, x); gx = gmp.bernpoly(n, x)
>>> fx = fpm.bernpoly(n, x); ax = apm.bernpoly(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  7.092428708543964460784313725490196078431E+0
mpm:  7.092428708543964460784313725490196078431e+0
gmp:  7.092428708543964460784313725490196078431E+00
fpm:  7.09242870854397E+00
apm:  7.092428708543964460784313725490196078432e+0 (1.457e-37%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '16'; z = '10 + 5j'
>>> \mathrm{d}z = dec.bernpoly(n, z); mz = mpm.bernpoly(n, z); gz = gmp.bernpoly(n, z)
>>> fz = fpm.bernpoly(n, z); az = apm.bernpoly(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 9.1935042603632624118E+14              + 2.9617209867479000000E+16j
mpm: 9.1935042603632624118e+14              + 2.9617209867479000000e+16j
gmp: 9.1935042603632624118E+14              + 2.9617209867479000000E+16j
fpm: 9.19350426036326E+14                   + 2.9617209867479 0E+16j
apm: 9.1935042603632624118e+14 (1.089e-18%) + 2.9617209867479000000e+16 (1.03e-19%)j
```

9.4.5 Euler numbers

`ctx.eulernum(`n`)`

where `ctx` is `math53`, `ctxflint`.

Returns the Euler numbers E_n. See also Wikipedia [1277], MathWorld [919], Flint [760], Ehrhardt [297] (3.10.8), Mpmath [608].

The Euler numbers E_n can be defined as

$$E_n = \frac{4^n \beta(n+1)}{\zeta(n)} \frac{2B_n}{\pi}$$

An example:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '16'
>>> \mathrm{d}x = dec.eulernum(n); mx = mpm.eulernum(n); gx = gmp.eulernum(n)
>>> fx = fpm.eulernum(n); ax = apm.eulernum(n)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.939151214500000000000000000000000000000E+10
mpm:  1.939151214500000000000000000000000000000e+10
gmp:  1.939151214500000000000000000000000000000E+10
fpm:  1.93915121450000E+10
apm:  1.939151214500000000000000000000000000000e+10 (0.0%)
```

9.4.6 Euler polynomials, $E_n(x)$

`ctx.eulerpoly(n, x)`

Returns $E_n(x) = \dfrac{2}{n+1}\left(B_{n+1}(x) - 2^{n+1}B_{n+1}\left(\dfrac{x}{2}\right)\right)$, the Euler polynomial of degree $n \geq 0$. Special values include the Euler numbers $E_n = 2^n E_n(1/2)$.

See also Wikipedia [1278], MathWorld [920], NIST [242], Ehrhardt [297] (3.10.9), Mpmath [609].

$$E_{n-1}(x) = \frac{2}{n}\left(B_n(x) - 2^n B_n\left(\tfrac{1}{2}x\right)\right),$$

$$E_{n-1}(x) = \frac{2^n}{n}\left(B_n\left(\tfrac{1}{2}x + \tfrac{1}{2}\right) - B_n\left(\tfrac{1}{2}x\right)\right).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Eulerpoly(3, 4)
ereal('5.235987755982988730 7E-1')
>>> math53.Eulerpoly(3, 12)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '16'; x = '1.5'
>>> \mathrm{d}x = dec.eulerpoly(n, x); mx = mpm.eulerpoly(n, x); gx = gmp.eulerpoly(n, x)
>>> fx = fpm.eulerpoly(n, x); ax = apm.eulerpoly(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -2.9589099339294433593750000000000000000000E+5
mpm:  -2.958909933929443359375000000000000000000e+5
gmp:  -2.9589099339294433593750000000000000000000E+05
fpm:  -2.95890993392944E+05
apm:  -2.958909933929443359375000000000000000000e+5 (-3.254e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '16'; z = '10 + 5j'
>>> \mathrm{d}z = dec.eulerpoly(n, z); mz = mpm.eulerpoly(n, z); gz = gmp.eulerpoly(n, z)
>>> fz = fpm.eulerpoly(n, z); az = apm.eulerpoly(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -2.7730057139461850000E+15               + 2.6625051419981595000E+16j
mpm: -2.773005713946185000 0e+15              + 2.662505141998159500 0e+16j
gmp: -2.7730057139461850000E+15               + 2.6625051419981595000E+16j
fpm: -2.77300571394618E+15                    + 2.66250514199816E+16j
apm: -2.773005713946185000 0e+15 (-6.053e-18%) + 2.662505141998159500 0e+16 (7.45e-19%)j
```

9.4.7 Barnes G-function

`ctx.barnes_g(z)`

Returns the Barnes G-function of z. See also Wikipedia [1297], MathWorld [914], NIST [20], Whittaker and Watson [1079], Mpmath [625].

Evaluates the Barnes G-function, which generalizes the superfactorial (*superfac()*) and by extension also the hyperfactorial (*hyperfac()*) to the complex numbers in an analogous way to how the gamma function generalizes the ordinary factorial.

The Barnes G-function may be defined in terms of a Weierstrass product:

$$G(z+1) = (2\pi)^{z/2} e^{-[z(z+1)+\gamma z^2]/2} \prod_{n=1}^{\infty} \left[\left(1+\frac{z}{n}\right)^n e^{-z+z^2/(2n)} \right]$$

For positive integers n, we have have relation to superfactorials $G(n) = \mathrm{sf}(n-2) = 0! \cdot 1! \cdots (n-2)!$.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '1.5'
>>> \mathrm{d}x = dec.barnesg(x); mx = mpm.barnesg(x); gx = gmp.barnesg(x)
>>> fx = fpm.barnesg(x); ax = apm.barnesg(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.0692226492664129495430088786978916046 53E+0
mpm:  1.0692226492664129495430088786978916046 53e+0
gmp:  1.0692226492664129495430088786978916046 53E+00
fpm:  1.06922264926641E+00
apm:  1.0692226492664129495430088786978916046 53e+0 (2.147e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '10 + 5j'
>>> \mathrm{d}z = dec.barnesg(z); mz = mpm.barnesg(z); gz = gmp.barnesg(z)
>>> fz = fpm.barnesg(z); az = apm.barnesg(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 4.0615260490805827996E+3              - 1.5756073051910381056E+3j
mpm: 4.0615260490805827996e+3              - 1.5756073051910381056e+3j
gmp: 4.0615260490805827996E+03             - 1.5756073051910381056E+03j
fpm: 4.06152604908058E+03                  - 1.57560730519104E+03j
apm: 4.0615260490805827996e+3 (8.969e-19%) - 1.5756073051910381056e+3 (-2.808e-18%)j
```

9.4.8 Logarithm of Barnes G function

`ctx.logbarnes_g(`x`)`

where `ctx` is `math53` or `ctxflint`.

Returns $\log G(z)$, the logarithm of Barnes G function, with log G(z) = (z-1) log Gamma(z) - zeta'(-1,z) + zeta'(-1).

See also: Wikipedia [1297], MathWorld [914], MathWorld [930], NIST [20], Ehrhardt [297] (3.5.6.9).

Computes Barnes G-function or the logarithmic Barnes G-function, respectively. The logarithmic version has branch cuts on the negative real axis and is continuous elsewhere in the complex plane, in analogy with the logarithmic gamma function. The functional equation

$$\log G(z+1) = \log \Gamma(z) + \log G(z).$$

holds for all z.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LogBarnesG(7.1)
ereal('5.235987755982988730 7E-1')
>>> math53.LogBarnesG('4.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '1.5'
>>> \mathrm{d}x = dec.Logbarnesg(x); mx = mpm.Logbarnesg(x); gx = gmp.Logbarnesg(x)
>>> fx = fpm.Logbarnesg(x); ax = apm.Logbarnesg(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  6.6931888435004704274028685868184404102 25E-2
mpm:  6.6931888435004704274028685868184404102 25e-2
gmp:  6.6931888435004704274028685868184404102 25E-02
fpm:  6.69318884350047E-02
apm:  6.6931888435004704274028685868184404102 25e-2 (2.144e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '10 + 5j'
>>> \mathrm{d}z = dec.Logbarnesg(z); mz = mpm.Logbarnesg(z); gz = gmp.Logbarnesg(z)
>>> fz = fpm.Logbarnesg(z); az = apm.Logbarnesg(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 8.3794095077852315352E+0              - 3.7006227106476637064E-1j
mpm: 8.3794095077852315352e+0              - 3.7006227106476637064e-1j
gmp: 8.3794095077852315352E+00             - 3.7006227106476637064E-01j
fpm: 8.37940950778523E+00                  - 3.70062271064766E-01j
apm: 8.3794095077852315352e+0 (1.617e-19%) + 5.6178605493551511922e+1 (4.825e-20%)j
```

9.4.9 Hyperfactorial

`ctx.hyperfactorial(z)`

Returns the hyperfactorial of z. See also Wikipedia [1285], MathWorld [925], of Integer Sequences [427], Mpmath [632].

Computes the hyperfactorial, defined for integers as the product

$$H(n) = \prod_{k=1}^{n} k^k.$$

The hyperfactorial satisfies the recurrence formula $H(z) = z^z H(z-1)$. It can be defined more generally in terms of the Barnes G-function (see *barnesg()*) and the gamma function by the formula

$$H(z) = \frac{\Gamma(z+1)^z}{G(z+1)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '1.5'
>>> \mathrm{d}x = dec.hyperfac(x); mx = mpm.hyperfac(x); gx = gmp.hyperfac(x)
>>> fx = fpm.hyperfac(x); ax = apm.hyperfac(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.617488527948946062817035460231792759012E+0
mpm:  1.617488527948946062817035460231792759012e+0
gmp:  1.617488527948946062817035460231792759012E+00
fpm:  1.61748852794895E+00
apm:  1.617488527948946062817035460231792771778e+0 (2.059e-33%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '10.5 + 1j'
>>> \mathrm{d}z = dec.hyperfac(z); mz = mpm.hyperfac(z); gz = gmp.hyperfac(z)
>>> fz = fpm.hyperfac(z); az = apm.hyperfac(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 4.6669290555030590228E+48              + 1.2961711093304818907E+49j
mpm: 4.6669290555030590228e+48              + 1.2961711093304818907e+49j
gmp: 4.6669290555030590228E+48              + 1.2961711093304818907E+49j
fpm: 4.66692905550306E+48                   + 1.29617110933048E+49j
apm: 4.6669290555030590228e+48 (5.257e-17%) + 1.2961711093304818907e+49 (3.064e-17%)j
```

9.4.10 Superfactorial

`ctx.superfactorial(z)`

Returns the Superfactorial of z. See also Wikipedia [1293], MathWorld [936], of Integer Sequences [428], Mpmath [637].

Computes the superfactorial, defined as the product of consecutive factorials

$$\mathrm{sf}(n) = \prod_{k=1}^{n} k!$$

For general complex z, $\mathrm{sf}(z)$ is defined in terms of the Barnes G-function (see *barnesg()*).

$$\mathrm{sf}(z) = G(z+2)$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '1.5'
>>> \mathrm{d}x = dec.superfac(x); mx = mpm.superfac(x); gx = gmp.superfac(x)
>>> fx = fpm.superfac(x); ax = apm.superfac(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.259648257495192144086307951096069825563E+0
mpm:  1.259648257495192144086307951096069825563e+0
gmp:  1.259648257495192144086307951096069825563E+00
fpm:  1.25964825749519E+00
apm:  1.259648257495192144086307951096069825563e+0 (9.113e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '10.5 + 1j'
>>> \mathrm{d}z = dec.superfac(z); mz = mpm.superfac(z); gz = gmp.superfac(z)
>>> fz = fpm.superfac(z); az = apm.superfac(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.8638249390462723001E+30              - 8.943232330275503035 3E+30j
mpm: 1.8638249390462723001e+30              - 8.943232330275503035 3e+30j
gmp: 1.8638249390462723001E+30              - 8.943232330275503035 3E+30j
fpm: 1.86382493904627E+30                   - 8.94323233027550E+30j
apm: 1.8638249390462723001e+30 (2.42e-18%) - 8.943232330275503035 3e+30 (-1.345e-18%)j
```

9.5 Riemann zeta function, and related functions

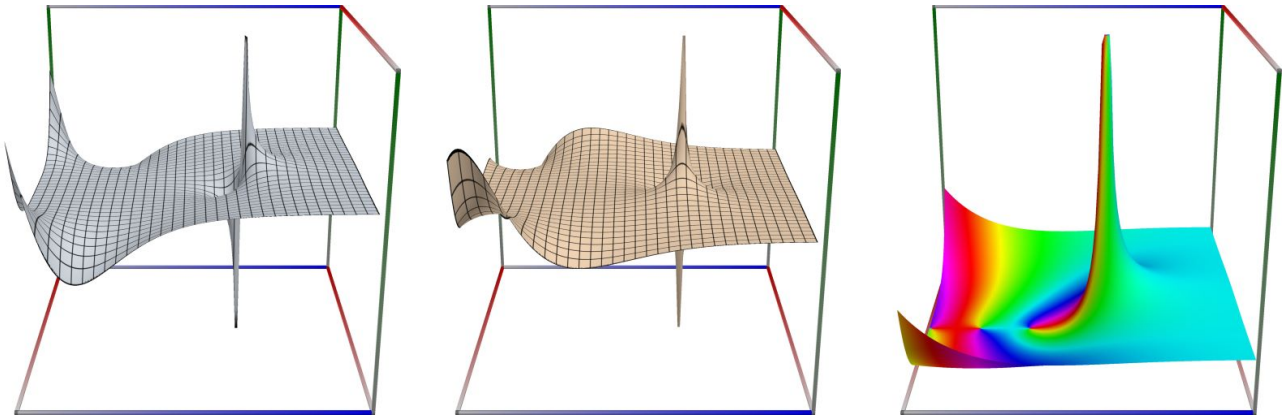
9.5.1 Riemann zeta function, $\zeta(s)$

`ctx.zeta(`s`)`

where `ctx` is `math53`, `mathc53`, `ctxboost` `ctxflint`.

Returns the Riemann zeta function, defined as $\zeta(s) = \sum_{k=1}^{\infty} \frac{1}{k^s}$ for $s > 1$, and by analytic continuation for $s \neq 1$.

See also Wikipedia [1334], MathWorld [970], NIST [19], BoostMath [158], Ehrhardt [297] (3.6.1.1), Ehrhardt [297] (4.2.63), Flint [763], Mpmath [661].



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Zeta(1.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Zeta('1.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '1.5'
>>> \mathrm{d}x = dec.zeta(x); mx = mpm.zeta(x); gx = gmp.zeta(x)
>>> fx = fpm.zeta(x); ax = apm.zeta(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.6123753486854883433485675679240716305 71E+0
mpm:  2.6123753486854883433485675679240716305 71e+0
gmp:  2.6123753486854883433485675679240716305 71E+00
fpm:  2.61237534868549E+00
apm:  2.6123753486854883433485675679240716305 71e+0 (8.789e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.zeta(z); mz = mpm.zeta(z); gz = gmp.zeta(z)
>>> fz = fpm.zeta(z); az = apm.zeta(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 9.8042867050578744177E-1             - 2.5411901380637479903E-2j
```

(continues on next page)

(continued from previous page)

```
mpm: 9.8042867050578744177e-1             - 2.5411901380637479903e-2j
gmp: 9.8042867050578744177E-01            - 2.5411901380637479903E-02j
fpm: 9.80428670505787E-01                 - 2.54119013806375E-02j
apm: 9.8042867050578744177e-1 (4.32e-20%) - 2.5411901380637479903e-2 (-5.208e-20%)j
```

9.5.2 Riemann $\zeta(s) - 1$

`ctx.zetam1(`s`)`

Returns the Riemann function $\zeta(s) - 1 = \zeta(s, 2)$ for $s \neq 1$.

See also Wikipedia [1334], MathWorld [970], NIST [19], BoostMath [158], Ehrhardt [297] (3.6.1.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Zetam1(12)
ereal('5.235987755982988730 7E-1')
>>> math53.Zetam1('10.0001')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '26'
>>> \mathrm{d}x = dec.zetam1(x); mx = mpm.zetam1(x); gx = gmp.zetam1(x)
>>> fx = fpm.zetam1(x); ax = apm.zetam1(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.4901554828365041234658506630698628864 79E-8
mpm:  1.4901554828365041234658506630698628864 79e-8
gmp:  1.4901554828365041234658506630698628864 79E-08
fpm:  1.49015548283650E-08
apm:  1.4901554828365041234658506630698628864 79e-8 (1.148e-39%)
```

An example with complex input:

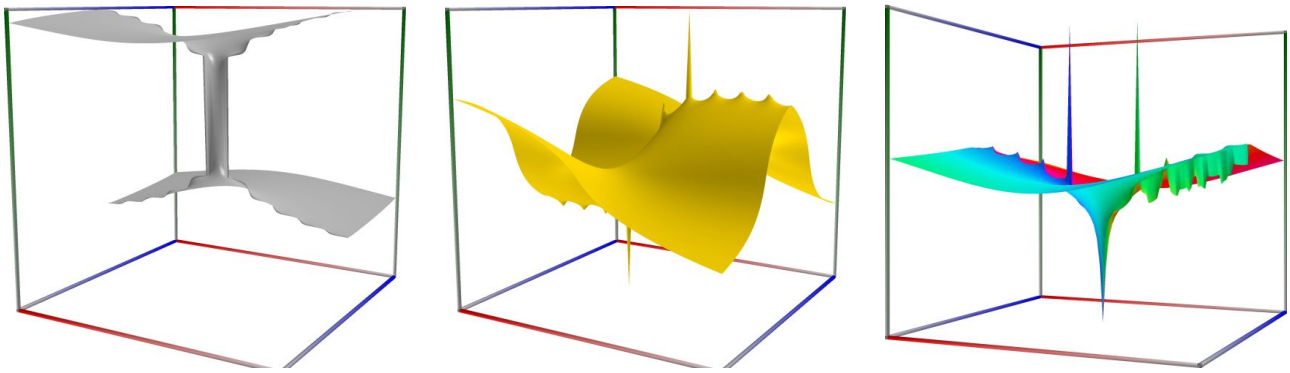
```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '26.0 + 3j'
>>> \mathrm{d}z = dec.zetam1(z); mz = mpm.zetam1(z); gz = gmp.zetam1(z)
>>> fz = fpm.zetam1(z); az = apm.zetam1(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -7.2571711804358415550E-9                - 1.3014689280327770930E-8j
mpm: -7.2571711804358415550e-9                - 1.3014689280327770930e-8j
gmp: -7.2571711804358415550E-09               - 1.3014689280327770930E-08j
fpm: -7.25717118043584E-09                    - 1.30146892803278E-08j
apm: -7.2571711804358415431e-9 (-4.348e-20%) - 1.3014689280327770920e-8 (-9.698e-20%)j
```

9.5.3 Hardy (or Riemann-Siegel) theta function

mathc53.**hardy_theta**(*z*)

Returns the Hardy (or Riemann-Siegel) theta function. See also Wikipedia [1363], MathWorld [1016], Ehrhardt [297] (4.2.52), Mpmath [696] Mpmath [702].

$$\theta(t) = \frac{\log\Gamma\left(\frac{1+2it}{4}\right) - \log\Gamma\left(\frac{1-2it}{4}\right)}{2i} - \frac{\log\pi}{2}t.$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Rstheta(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Rstheta('0.1')
ecplx('5.351847902755998475 4E-1')
```

9.5.4 Hardy (or Riemann-Siegel) Z function

`mathc53.hardy_z(z)`

Returns the Hardy (or Riemann-Siegel) Z function. See also Wikipedia [1362], MathWorld [1017], NIST [13], Flint [767], Ehrhardt [297] (4.2.52), Mpmath [701].

$$Z(t) = e^{i\theta(t)}\zeta(1/2 + it)$$

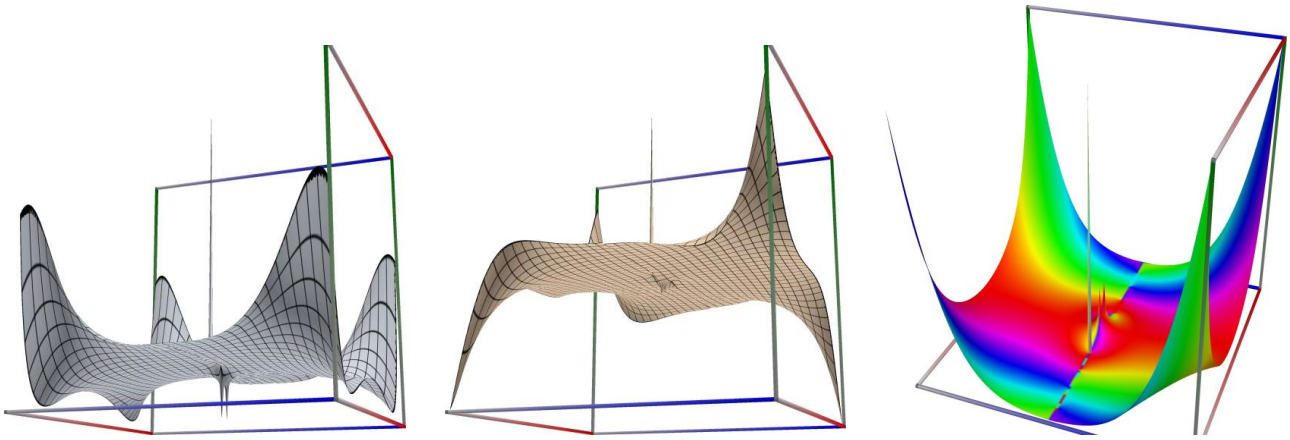
where $\zeta(s)$ is the Riemann zeta function and $\theta(t)$ denotes the Riemann-Siegel theta function.

This calls `acb_dirichlet_hardy_z`.

Computes the Z-function, also known as the Riemann-Siegel Z function,

$$Z(t) = e^{i\theta(t)}\zeta(1/2 + it)$$

where $\zeta(s)$ is the Riemann zeta function (*zeta()*) and where $\theta(t)$ denotes the Riemann-Siegel theta function (see *siegeltheta()*).



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.SiegelZ(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.SiegelZ('0.1')
ecplx('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '2.6'
>>> \mathrm{d}x = dec.siegelz(x); mx = mpm.siegelz(x); gx = gmp.siegelz(x)
>>> fx = fpm.siegelz(x); ax = apm.siegelz(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -5.270127547934236224731698070416108808706E-1
mpm:  -5.270127547934236224731698070416108808706e-1
gmp:  -5.270127547934236224731698070416108808706E-01
fpm:  -5.27012754793424E-01
apm:  -5.270127547934236224731698070416108808706e-1 (-1.002e-37%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '2.6 + 3j'
>>> \mathrm{d}z = dec.siegelz(z); mz = mpm.siegelz(z); gz = gmp.siegelz(z)
>>> fz = fpm.siegelz(z); az = apm.siegelz(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -2.8270067277806269605E-1                - 1.6028967412508714310E-1j
mpm: -2.8270067277806269605e-1                - 1.6028967412508714310e-1j
gmp: -2.8270067277806269605E-01               - 1.6028967412508714310E-01j
fpm: -2.82700672778063E-01                    - 1.60289674125087E-01j
apm: -2.8270067277806269605e-1 (-2.097e-18%) - 1.6028967412508714310e-1 (-4.228e-18%)j
```

9.5.5 Riemann (Landau) function $\xi(s)$

`ctx.riemann_xi(`s`)`

Returns the Riemann (Landau) function $\xi(s)$. See also MathWorld [1014], Wikipedia [1354], Flint [762].

Landau's lower-case ξ ("xi") is defined as

$$\xi(s) = \frac{1}{2} s(s-1) \pi^{-s/2} \Gamma\left(\frac{s}{2}\right) \zeta(s)$$

for $s \in \mathbb{C}$. Here $\zeta(s)$ denotes the Riemann zeta function and $\Gamma(s)$ is the Gamma function. The functional equation (or reflection formula) for Landau's ξ is

$$\xi(1-s) = \xi(s)$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '2.6'
>>> \mathrm{d}x = dec.riemann_xi(x); mx = mpm.riemann_xi(x); gx = gmp.riemann_xi(x)
>>> fx = fpm.riemann_xi(x); ax = apm.riemann_xi(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  5.502477451681679934572096474654698087218E-1
mpm:  5.502477451681679934572096474654698087218e-1
gmp:  5.502477451681679934572096474654698087218E-01
fpm:  5.50247745168168E-01
apm:  5.502477451681679934572096474654698087218e-1 (1.46e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '2.6 + 3j'
>>> \mathrm{d}z = dec.riemann_xi(z); mz = mpm.riemann_xi(z); gz = gmp.riemann_xi(z)
>>> fz = fpm.riemann_xi(z); az = apm.riemann_xi(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 4.29157131070547904 38E-1              + 1.29567490254131445 93E-1j
mpm: 4.29157131070547904 38e-1              + 1.29567490254131445 93e-1j
gmp: 4.29157131070547904 38E-01             + 1.29567490254131445 93E-01j
fpm: 4.29157131070548E-01                   + 1.29567490254131E-01j
apm: 4.29157131070547904 37e-1 (1.974e-18%) + 1.29567490254131445 93e-1 (6.537e-18%)j
```

9.5.6 Dirichlet eta function, $\eta(s)$

`ctx.dirichlet_eta(x)`

Returns the Dirichlet eta function, defined as $\eta(s) = (1 - 2^{1-s})\zeta(s) = \sum_{k=0}^{\infty} \frac{(-1)^k}{k^s}$ for $s > 0$ and by analytic continuation for $s \leq 0$.

See also: MathWorld [997], Ehrhardt [297] (3.6.3.1), Flint [762], Mpmath [697].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.DirichletEta(12)
ereal('5.235987755982988730E-1')
>>> math53.DirichletEta('10.0001')
ereal('5.351847902755998475E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '2.6'
>>> \mathrm{d}x = dec.dirichlet_eta(x); mx = mpm.dirichlet_eta(x); gx = gmp.dirichlet_eta(x)
>>> fx = fpm.dirichlet_eta(x); ax = apm.dirichlet_eta(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  8.748307349702805779574670518491745461882E-1
mpm:  8.748307349702805779574670518491745461882e-1
gmp:  8.748307349702805779574670518491745461882E-01
fpm:  8.74830734970281E-01
apm:  8.748307349702805779574670518491745461882e-1 (1.968e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '2.6 + 3j'
>>> \mathrm{d}z = dec.dirichlet_eta(z); mz = mpm.dirichlet_eta(z); gz = gmp.dirichlet_eta(z)
>>> fz = fpm.dirichlet_eta(z); az = apm.dirichlet_eta(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.0368930023009351181E+0              + 1.3961421241509646940E-1j
mpm: 1.0368930023009351181e+0              + 1.3961421241509646940e-1j
gmp: 1.0368930023009351181E+00             + 1.3961421241509646940E-01j
fpm: 1.03689300230094E+00                  + 1.39614212415096E-01j
apm: 1.0368930023009351181e+0 (1.634e-19%) + 1.3961421241509646940e-1 (6.825e-19%)j
```

9.5.7 Dirichlet $\eta(s) - 1$

`ctx.dirichlet_eta_m1(s)`

Returns the Dirichlet function $\eta(s) - 1 = (\zeta(s) - 1) - (2^{1-s}\zeta(s))$.

See also: MathWorld [997], Ehrhardt [297] (3.6.3.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.DirichletEtam1(5)
ereal('5.235987755982988730 7E-1')
>>> math53.DirichletEtam1('51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '2.6'
>>> \mathrm{d}x = dec.etam1(x); mx = mpm.etam1(x); gx = gmp.etam1(x)
>>> fx = fpm.etam1(x); ax = apm.etam1(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -1.2516926502971942204253294815082545381 18E-1
mpm:  -1.2516926502971942204253294815082545381 18e-1
gmp:  -1.2516926502971942204253294815082545381 18E-01
fpm:  -1.25169265029719E-01
apm:  -1.2516926502971942204253294815082545381 18e-1 (-1.376e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '2.6 + 3j'
>>> \mathrm{d}z = dec.etam1(z); mz = mpm.etam1(z); gz = gmp.etam1(z)
>>> fz = fpm.etam1(z); az = apm.etam1(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 3.6893002300935118091E-2              + 1.3961421241509646940E-1j
mpm: 3.6893002300935118091e-2              + 1.3961421241509646940e-1j
gmp: 3.6893002300935118091E-02             + 1.3961421241509646940E-01j
fpm: 3.689300230093 51E-02                 + 1.39614212415096E-01j
apm: 3.6893002300935118089e-2 (4.592e-18%) + 1.3961421241509646940e-1 (6.825e-19%)j
```

9.5.8 Dirichlet beta function, $\beta(s)$

`ctx.dirichlet_beta(s)`

Returns the Dirichlet beta function, defined as $\beta(s) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)^s}$, for $s > 0$, and by analytic continuation for $s \leq 0$.

See also: Wikipedia [1341], MathWorld [975], Ehrhardt [297] (3.6.4).

Alternatively, the following definition, in terms of the Hurwitz zeta function, is valid in the whole complex s-plane:

$$\beta(s) = 4^{-s}\left(\zeta\left(s, \frac{1}{4}\right) - \zeta\left(s, \frac{3}{4}\right)\right).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.DirichletBeta(5)
ereal('5.2359877559829887307E-1')
>>> math53.DirichletBeta('51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '2.6'
>>> \mathrm{d}x = dec.dirichlet_beta(x); mx = mpm.dirichlet_beta(x); gx = gmp.dirichlet_beta(x)
>>> fx = fpm.dirichlet_beta(x); ax = apm.dirichlet_beta(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.535048662378325267539346079940401993601E-1
mpm:  9.535048662378325267539346079940401993601e-1
gmp:  9.535048662378325267539346079940401993601E-01
fpm:  9.53504866237832E-01
apm:  9.535048662378325267539346079940401993601e-1 (1.084e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '2.6 + 3j'
>>> \mathrm{d}z = dec.dirichlet_beta(z); mz = mpm.dirichlet_beta(z); gz = gmp.dirichlet_beta(z)
>>> fz = fpm.dirichlet_beta(z); az = apm.dirichlet_beta(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.0550279685803642739E+0              + 3.3718823063670455627E-3j
mpm: 1.0550279685803642739e+0              + 3.3718823063670455627e-3j
gmp: 1.0550279685803642739E+00             + 3.3718823063670455627E-03j
fpm: 1.0550279685803 6E+00                 + 3.37188230636704E-03j
apm: 1.0550279685803642739e+0 (7.226e-19%) + 3.3718823063670455625e-3 (1.828e-16%)j
```

9.5.9 Dirichlet lambda function, $\lambda(s)$

`ctx.dirichlet_lambda(`s`)`

Returns the Dirichlet lambda function, defined as $\lambda(s) = \sum_{n=0}^{\infty} (2n+1)^{-s}$, $for`s > 1$, and by analytic continuation for $s < 1$.

See also: MathWorld [998], Ehrhardt [297] (3.6.5), Hu and Kim [376].

The function is calculated as

$$\lambda(s) = (1 - 2^{-s})\zeta(s) = -\text{exp2m1}(-s)\zeta(s)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.DirichletLambda(5)
ereal('5.23598775598298873O7E-1')
>>> math53.DirichletLambda('51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = '2.6'
>>> \mathrm{d}x = dec.dirichlet_lambda(x); mx = mpm.dirichlet_lambda(x); gx = gmp.dirichlet_lambda(x)
>>> fx = fpm.dirichlet_lambda(x); ax = apm.dirichlet_lambda(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.0901542720215305859190184188308321160 26E+0
mpm:  1.0901542720215305859190184188308321160 26e+0
gmp:  1.0901542720215305859190184188308321160 26E+00
fpm:  1.09015427202153E+00
apm:  1.0901542720215305859190184188308321160 56e+0 (2.815e-36%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '2.6 + 3j'
>>> \mathrm{d}z = dec.dirichlet_lambda(z); mz = mpm.dirichlet_lambda(z); gz = gmp.dirichlet_lambda(z)
>>> fz = fpm.dirichlet_lambda(z); az = apm.dirichlet_lambda(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 9.5326918762855854694E-1              + 2.2012809770791952023E-2j
mpm: 9.5326918762855854694e-1              + 2.2012809770791952023e-2j
gmp: 9.5326918762855854694E-01             + 2.2012809770791952023E-02j
fpm: 9.53269187628559E-01                  + 2.20128097707919E-02j
apm: 9.5326918762855854695e-1 (7.108e-19%) + 2.2012809770791952017e-2 (1.972e-17%)j
```

9.5.10 Zeros of the Riemann zeta function

`ctx.zeta_zero(`n`)`

Returns the zeros of the Riemann zeta function. See also Wikipedia [1361], MathWorld [1015], NIST [14], Mpmath [703].

This calls `acb_dirichlet_zeta_zero`.

Computes the n-th nontrivial zero of $\zeta(s)$ on the critical line, i.e. returns an approximation of the n-th largest complex number $s = \frac{1}{2} + ti$ for which $\zeta(s) = 0$. Equivalently, the imaginary part t is a zero of the Z-function (*siegelz()*).

An example :

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '20'
>>> \mathrm{d}z = dec.zetazero(n); mz = mpm.zetazero(n); gz = gmp.zetazero(n)
>>> fz = fpm.zetazero(n); az = apm.zetazero(n)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 5.00000000000000000000E-1        + 7.7144840068874805373E+1j
mpm: 5.00000000000000000000e-1        + 7.7144840068874805373e+1j
gmp: 5.00000000000000000000E-01       + 7.7144840068874805373E+01j
fpm: 5.00000000000000E-01             + 7.71448400688748E+01j
apm: 5.00000000000000000000e-1 (0.0%) + 7.7144840068874805373e+1 (7.027e-20%)j
```

CHAPTER

TEN

HYPERGEOMETRIC FUNCTION ${}_0F_1$ AND RELATED

Text explaining this chapter

The source code for the C# tests in Visual Studio is divided by real and complex modules:

The source code for the C# tests in Visual Studio using real modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B10a_RealHypergeometric_0F1.cs,

The source code for the C# tests in Visual Studio using complex modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B10b_CplxHypergeometric_0F1.cs,

10.1 Hypergeometric Limit Function ${}_0F_1$

10.1.1 Confluent Hypergeometric Limit Function ${}_0F_1(b, x)$

`ctx.hyperg_0f1`(b, x)

Returns ${}_0F_1(b, x) = \sum_{k=0}^{\infty} \frac{1}{(b)_k} \frac{x^k}{k!}$, the confluent hypergeometric limit function, where $b \neq 0, -1, -2, \ldots$

See also Wikipedia [1254], MathWorld [901] BoostMath [104], Ehrhardt [297] (3.8.6), Flint [757], Flint [750], Mpmath [590].

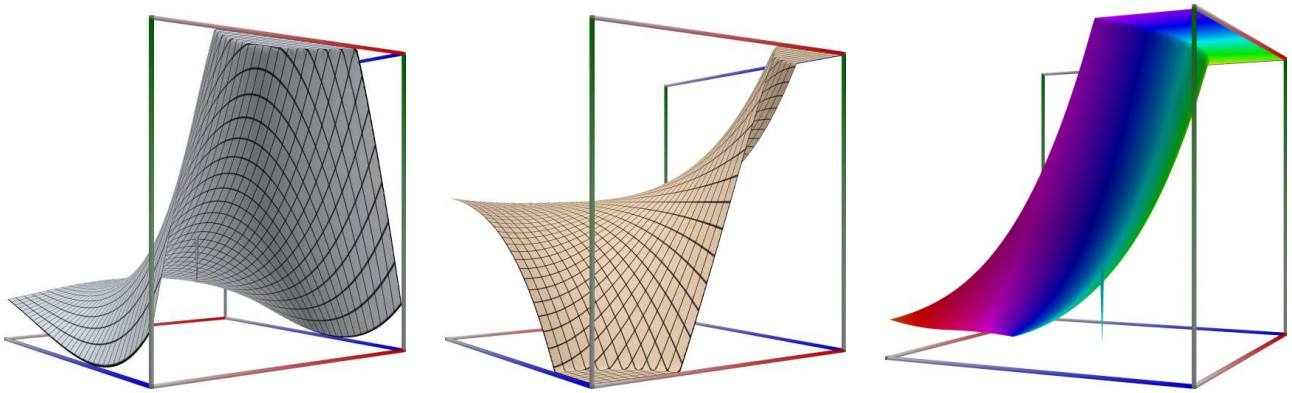
If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $b, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $b \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $b, x \in \mathbb{C}$ is accepted.

Calls `arb_hypgeom_0f1` or `acb_hypgeom_0f1`.

Returns the hypergeometric function ${}_0F_1$.

Gives the hypergeometric function ${}_0F_1$, sometimes known as the confluent limit function, defined as

$$ {}_0F_1(a, z) = \sum_{k=0}^{\infty} \frac{1}{(a)_k} \frac{z^k}{k!}. $$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Hyperg0F1(5.1,0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Hyperg0F1(15.2,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a= 10; x = 30
>>> \mathrm{d}x = dec.hyp0f1(a, x); mx = mpm.hyp0f1(a, x); gx = gmp.hyp0f1(a, x)
>>> fx = fpm.hyp0f1(a, x); ax = apm.hyp0f1(a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.4686155009681100647083147826942249952 60E+1
mpm:  1.4686155009681100647083147826942249952 60e+1
gmp:  1.4686155009681100647083147826942249952 60E+01
fpm:  1.46861550096811E+01
apm:  1.4686155009681100647083147826942249952 60e+1 (1.251e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.hyp0f1(a, z); mz = mpm.hyp0f1(a, z); gz = gmp.hyp0f1(a, z)
>>> fz = fpm.hyp0f1(a, z); az = apm.hyp0f1(a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.2521260436915235828E+0              + 5.13718623935804996 00E-1j
mpm: 1.2521260436915235828e+0              + 5.13718623935804996 00e-1j
gmp: 1.2521260436915235828E+00             + 5.13718623935804996 00E-01j
fpm: 1.25212604369152E+00                  + 5.13718623935805E-01j
apm: 1.2521260436915235828e+0 (1.353e-19%) + 5.13718623935804996 00e-1 (8.244e-20%)j
```

10.1.2 Regularized Confluent Hypergeometric Limit Function ${}_0\widetilde{F}_1(b;x)$

`ctx.hyperg_0f1r`(b, x)

Returns ${}_0\widetilde{F}_1(b;x) = \dfrac{1}{\Gamma(b)}{}_0F_1(b;x)$, the regularized confluent hypergeometric limit function, for $b \neq 0, -1, -2, \cdots$, or, if $b = 0, -1, -2, \cdots = -n$, the corresponding limit ${}_0\widetilde{F}_1(b;x) = x^{n+1}{}_0\widetilde{F}_1(n+2;x) = \dfrac{x^{n+1}}{\Gamma(n+2)}{}_0F_1(n+2;x)$.

See also Wikipedia [1254], MathWorld [261] BoostMath [104], Ehrhardt [297] (3.8.7), Flint [757], Flint [750], Mpmath [590].

If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $b, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $b \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $b, x \in \mathbb{C}$ is accepted.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Hyperg0F1r(5.1,0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Hyperg0F1r(15.2,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a= 10; x = 30
>>> \mathrm{d}x = dec.hyp0f1r(a, x); mx = mpm.hyp0f1r(a, x); gx = gmp.hyp0f1r(a, x)
>>> fx = fpm.hyp0f1r(a, x); ax = apm.hyp0f1r(a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.0471106177472168890771461163310873987 54E-5
mpm:  4.0471106177472168890771461163310873987 54e-5
gmp:  4.0471106177472168890771461163310873987 54E-05
fpm:  4.0471106177472 2E-05
apm:  4.0471106177472168890771461163310873987 54e-5 (1.731e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.hyp0f1r(a, z); mz = mpm.hyp0f1r(a, z); gz = gmp.hyp0f1r(a, z)
>>> fz = fpm.hyp0f1r(a, z); az = apm.hyp0f1r(a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 3.4505237094673820071E-6              + 1.4156708111105737324E-6j
mpm: 3.4505237094673820071e-6              + 1.4156708111105737324e-6j
gmp: 3.4505237094673820071E-06             + 1.41567081111057324E-06j
fpm: 3.45052370946738E-06                  + 1.41567081111057E-06j
apm: 3.4505237094673820071e-6 (9.364e-20%) + 1.4156708111105737324e-6 (1.141e-19%)j
```

10.2 Bessel functions

10.2.1 Bessel function of the 1st kind, $J_\nu(x)$

`ctx.bessel_jv`(*nu*, *x*, *scaled=False*)

Returns $J_\nu(x)$, the Bessel function of the 1st kind.

If *scaled* is *True*, then $J_\nu(x) \cdot \exp(-|\Im(x)|)$ is returned, which means that for purely real x just $J_\nu(x)$ is returned.

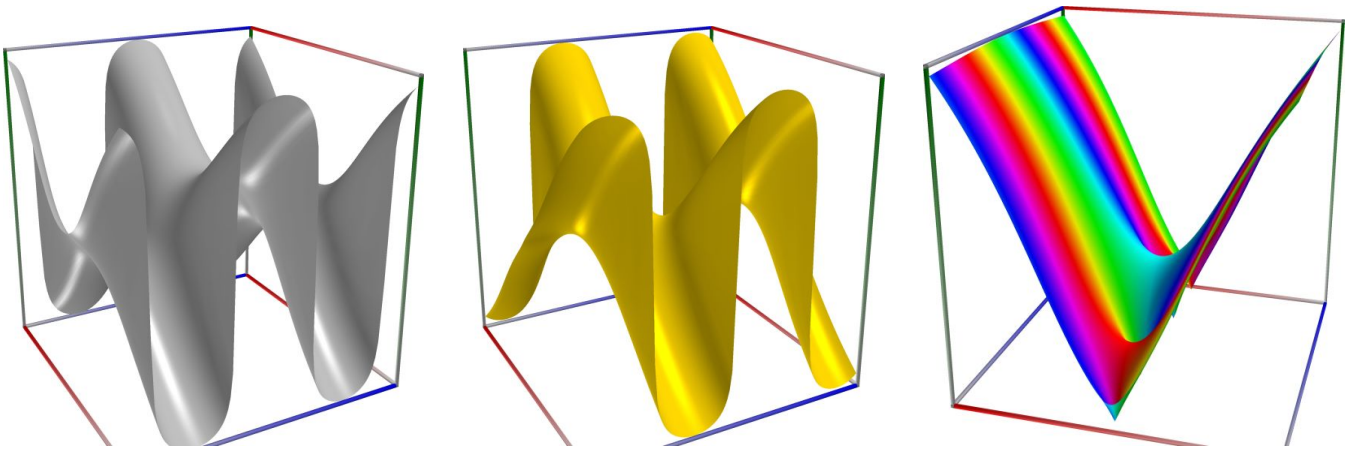
See also Wikipedia [1250], MathWorld [878], NIST [440], BoostMath [94], Ehrhardt [297] (3.1.3.1), Flint [756], Flint [747], Mpmath [583].

If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $\nu, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $\nu, x \in \mathbb{C}$ is accepted.

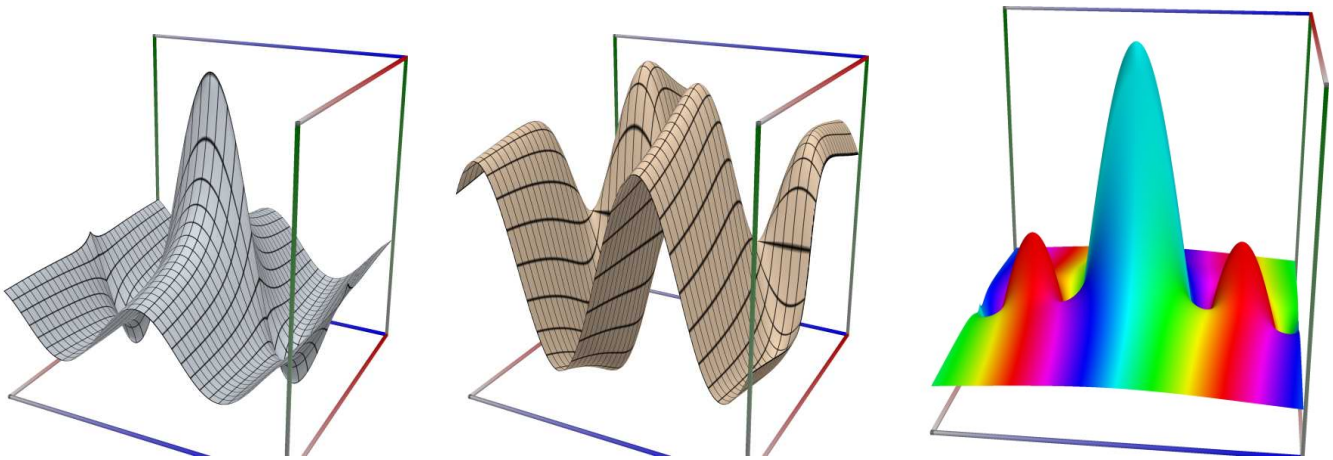
The function is defined as

$$J_\nu(x) = \left(\tfrac{1}{2}x\right)^\nu \sum_{k=0}^{\infty} (-1)^k \frac{(x^2/4)^k}{k!\Gamma(\nu+k+1)} = \frac{1}{\Gamma(\nu+1)} \left(\frac{z}{2}\right)^\nu {}_0F_1\left(\nu+1, -\frac{z^2}{4}\right).$$

The *wpf figures* below are showing the real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex function $z = J_\nu(x+iy)$ with $\nu = 0$ and $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.



The corresponding scaled function looks like this:



An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselJ(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.BesselJ(3, '0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n= 10; x = 30
>>> \mathrm{d}x = dec.besselj(n, x); mx = mpm.besselj(n, x); gx = gmp.besselj(n, x)
>>> fx = fpm.besselj(n, x); ax = apm.besselj(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -1.298768939985887681859474347649534305196E-1
mpm:  -1.298768939985887681859474347649534305196e-1
gmp:  -1.298768939985887681859474347649534305196E-01
fpm:  -1.29876893998589E-01
apm:  -1.298768939985887681859474347649551192015e-1 (-9.855e-30%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.besselj(n, z); mz = mpm.besselj(n, z); gz = gmp.besselj(n, z)
>>> fz = fpm.besselj(n, z); az = apm.besselj(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -2.40287346111284405858E-3               + 1.9815132418922270634E-3j
mpm: -2.40287346111284405858e-3               + 1.9815132418922270634e-3j
gmp: -2.40287346111284405858E-03              + 1.9815132418922270634E-03j
fpm: -2.40287346112844E-03                    + 1.98151324189223E-03j
apm: -2.40287346111284405858e-3 (-6.885e-20%) + 1.9815132418922270634e-3 (8.349e-20%)j
```

10.2.2 Bessel function of the 2nd kind, $Y_\nu(x)$

`ctx.bessel_yv`(*nu*, *x*, *scaled=False*)

Returns $Y_\nu(z)$ the Bessel function of the second kind.

If *scaled* is *True*, then $Y_\nu(x) \cdot \exp(-|\Im(x)|)$ is returned, which means that for purely real x just $Y_\nu(x)$ is returned.

See also Wikipedia [1251], MathWorld [879], NIST [441], BoostMath [94], Ehrhardt [297] (3.1.3.2), Flint [756], Flint [747], Mpmath [584].

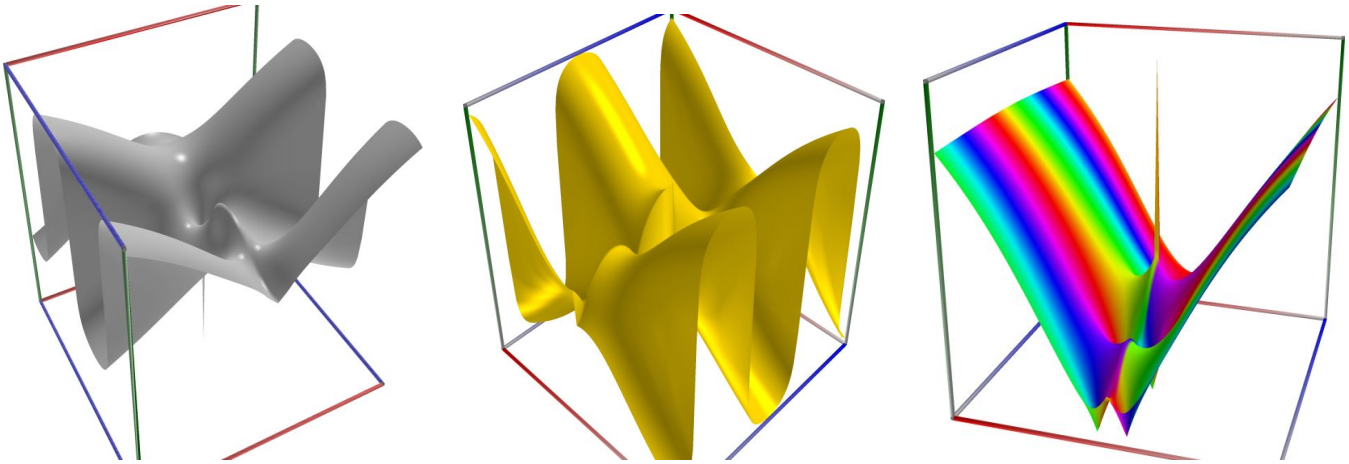
If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $\nu, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $\nu, x \in \mathbb{C}$ is accepted.

The function is computed from the formula

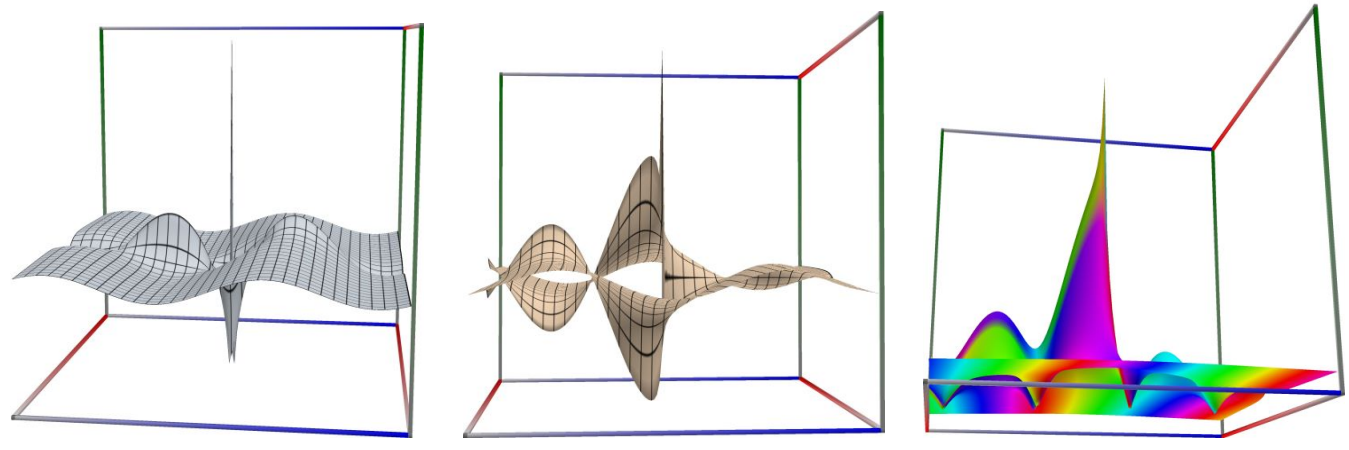
$$Y_\nu(x) = \frac{\cos(\nu\pi)J_\nu(x) - J_{-\nu}(x)}{\sin(\nu\pi)}$$

unless $\nu = n$ is an integer in which case the limit value is computed.

The *wpf figures* below are showing the real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex function $z = Y_\nu(x + iy)$ with $\nu = 0$ and $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.



The corresponding scaled function looks like this:



An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselY(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.BesselY(3, '0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n= 10; x = 30
>>> \mathrm{d}x = dec.bessely(n, x); mx = mpm.bessely(n, x); gx = gmp.bessely(n, x)
>>> fx = fpm.bessely(n, x); ax = apm.bessely(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  7.505670212239711328867640811591620556875E-2
mpm:  7.505670212239711328867640811591620556875e-2
gmp:  7.505670212239711328867640811591620556875E-02
fpm:  7.50567021223971E-02
apm:  7.505670212239711328867640811591512447084e-2 (2.412e-27%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.bessely(n, z); mz = mpm.bessely(n, z); gz = gmp.bessely(n, z)
>>> fz = fpm.bessely(n, z); az = apm.bessely(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 6.7895703730606195794E+0              + 6.9959972339074326549E+0j
mpm: 6.7895703730606195794e+0              + 6.9959972339074326549e+0j
gmp: 6.7895703730606195794E+00             + 6.9959972339074326549E+00j
fpm: 6.78957037306062E+00                  + 6.99599723390743E+00j
apm: 6.7895703730606195794e+0 (3.493e-19%) + 6.9959972339074326549e+0 (3.39e-19%)j
```

10.2.3 First derivative of the Bessel function of the first kind, $J'_\nu(x)$

`ctx.bessel_jv_prime(`*nu*, *x*, *scaled=False*`)`

Returns $J'_\nu(x)$, the first derivative (with respect to x) of $J_\nu(z)$, the Bessel function of the second kind.

If *scaled* is *True*, then $J'_\nu(x) \cdot \exp(-|\Im(x)|)$ is returned, which means that for purely real x just $J'_\nu(x)$ is returned.

See also Wikipedia [1250], MathWorld [878], NIST [440], BoostMath [94], BoostMath [116].

If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $\nu, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $\nu, x \in \mathbb{C}$ is accepted.

The function is calculated as $J'_\nu(x) = \frac{1}{2}(J_{\nu-1}(x) - J_{\nu+1}(x))$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselJPrime(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.BesselJPrime(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.2.4 First derivative of the Bessel function of the second kind, $Y'_\nu(x)$

`ctx.bessel_yv_prime`(*nu*, *x*, *scaled=False*)

Returns $Y'_\nu(x)$, the first derivative (with respect to x) of $Y_\nu(z)$, the Bessel function of the second kind.

If *scaled* is *True*, then $Y'_\nu(x) \cdot \exp(-|\Im(x)|)$ is returned, which means that for purely real x just $Y'_\nu(x)$ is returned.

See also Wikipedia [1251], MathWorld [879], NIST [441], BoostMath [94], BoostMath [116].

If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $\nu, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $\nu, x \in \mathbb{C}$ is accepted.

The function is calculated as $Y'_\nu(x) = \frac{1}{2}(Y_{\nu-1}(x) - Y_{\nu+1}(x))$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselYPrime(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.BesselYPrime(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.2.5 Boost: Zeros $j_{\nu,m}$ of the Bessel function of the first kind: $J_\nu(j_{\nu,m}) = 0$

`ctx.bessel_jv_zero(nu, m)`

where `ctx` is `math53` or `ctxboost`.

For a real order $\nu \geq 0$ and a positive integer m, returns $j_{\nu,m}$, the m-th positive zero of the Bessel function of the first kind $J_\nu(x)$.

See also Wikipedia [1250], MathWorld [949], NIST [442], BoostMath [119], Mpmath [640].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselJZero(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselJZero(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.2.6 Boost: Zeros $y_{\nu,m}$ of the Bessel function of the first kind: $Y_\nu(y_{\nu,m}) = 0$

`ctx.bessel_yv_zero(nu, m)`

where `ctx` is `math53` or `ctxboost`.

For a real order $\nu \geq 0$ and a positive integer m, returns $y_{\nu,m}$, the m-th positive zero of the Bessel function of the second kind $Y_\nu(x)$.

See also Wikipedia [1251], MathWorld [950], NIST [442], BoostMath [119], Mpmath [641].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselJZero(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselJZero(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.3 Modified Bessel functions

10.3.1 Modified Bessel function of the 1st kind, $I_\nu(x)$

`ctx.bessel_iv`(*x*, *nu*, *scaled=False*)

Returns $I_\nu(x)$, the modified Bessel function of the first kind.

If *scaled* is *True*, then $I_\nu(x) \cdot \exp(-|\Re(x)|)$ is returned.

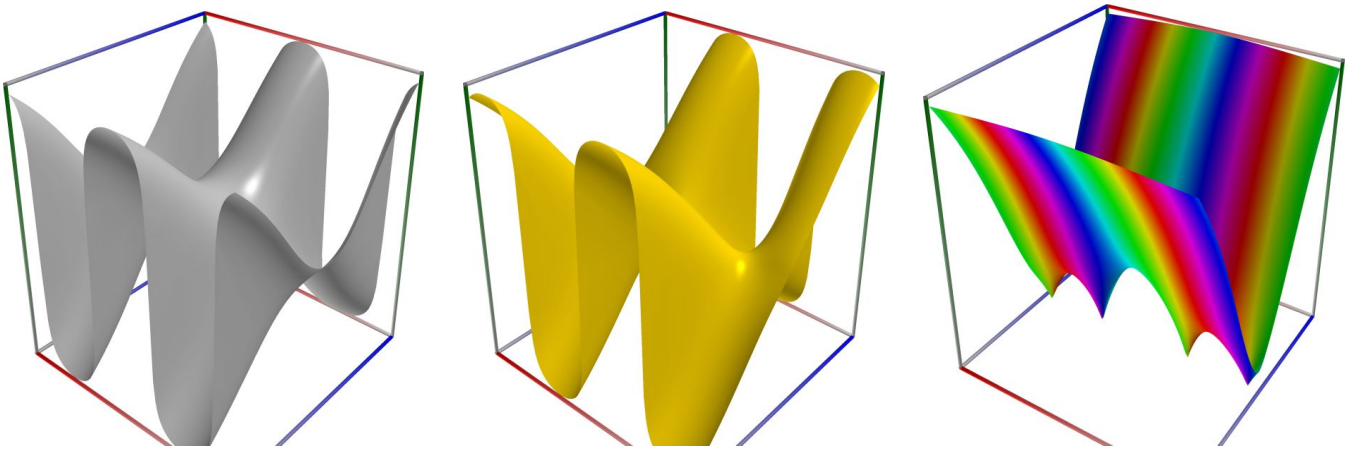
See also Wikipedia [1264], MathWorld [907], NIST [445], BoostMath [108], Ehrhardt [297] (3.1.4.1), Flint [756], Flint [754], Mpmath [596].

If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $\nu, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $\nu, x \in \mathbb{C}$ is accepted.

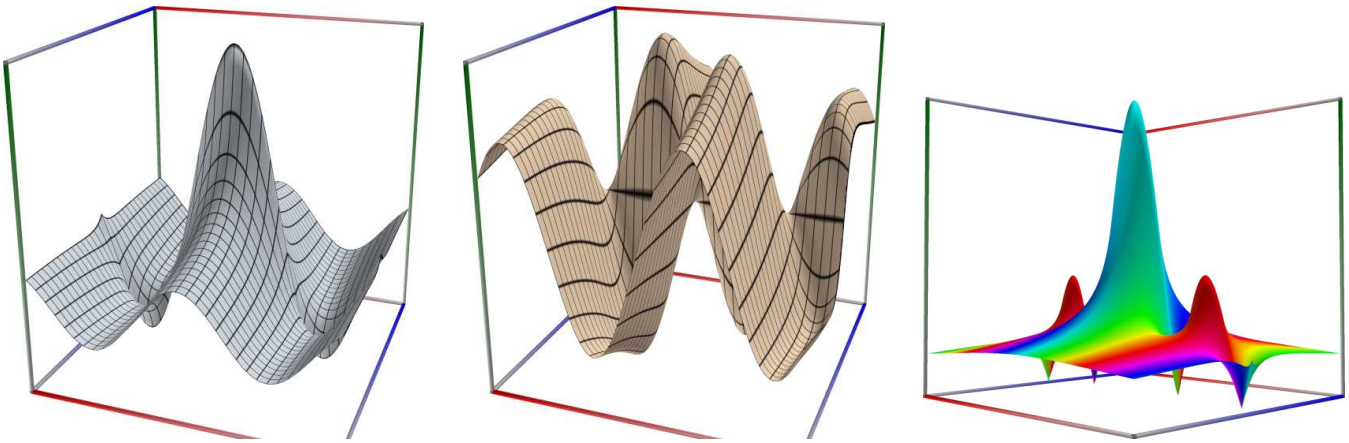
The function is defined as

$$I_\nu(z) = z^\nu (iz)^{-\nu} J_\nu(iz) = \frac{1}{\Gamma(\nu+1)} \left(\frac{z}{2}\right)^\nu {}_0F_1\left(\nu+1, \frac{z^2}{4}\right).$$

The *wpf figures* below are showing the real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex function $z = I_\nu(x+iy)$ with $\nu = 0$ and $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.



The corresponding scaled function looks like this:



An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselI(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.BesselI(3, '0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n= 10; x = 30
>>> \mathrm{d}x = dec.besseli(n, x); mx = mpm.besseli(n, x); gx = gmp.besseli(n, x)
>>> fx = fpm.besseli(n, x); ax = apm.besseli(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.458318099759671237651634761704819246948E+11
mpm:  1.458318099759671237651634761704819246948e+11
gmp:  1.458318099759671237651634761704819246948E+11
fpm:  1.45831809975967E+11
apm:  1.458318099759671237651634761704819246948e+11 (2.164e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.besseli(n, z); mz = mpm.besseli(n, z); gz = gmp.besseli(n, z)
>>> fz = fpm.besseli(n, z); az = apm.besseli(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -2.089047601208094054 5E-3               - 8.8394935038889455024E-4j
mpm: -2.089047601208094054 5e-3               - 8.8394935038889455024e-4j
gmp: -2.089047601208094054 5E-03              - 8.8394935038889455024E-04j
fpm: -2.08904760120809E-03                    - 8.83949350388895E-04j
apm: -2.089047601208094054 5e-3 (-7.919e-20%) - 8.8394935038889455024e-4 (-9.358e-20%)j
```

10.3.2 Modified Bessel function of the 2nd kind, $K_\nu(x)$

`ctx.bessel_kv`(*x*, *nu*, *scaled=False*)

Returns $K_\nu(x)$, the modified Bessel function of the second kind.

If *scaled* is *True*, then $K_\nu(x) \cdot \exp(x)$ is returned.

See also Wikipedia [1264], MathWorld [908], NIST [446], BoostMath [108], Ehrhardt [297] (3.1.4.3), Flint [756], Flint [754], Mpmath [597].

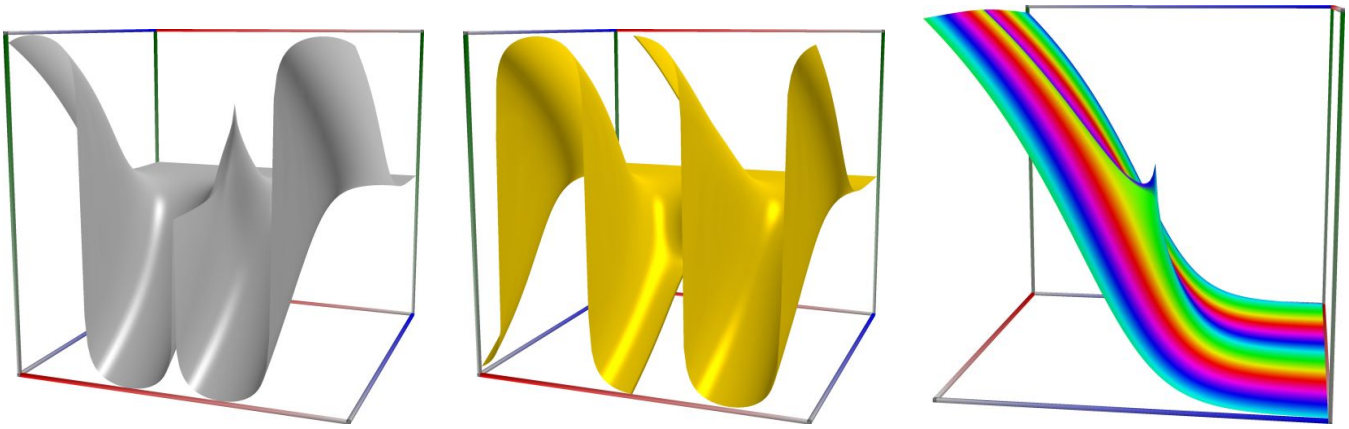
If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $\nu, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $\nu, x \in \mathbb{C}$ is accepted.

The function is defined as

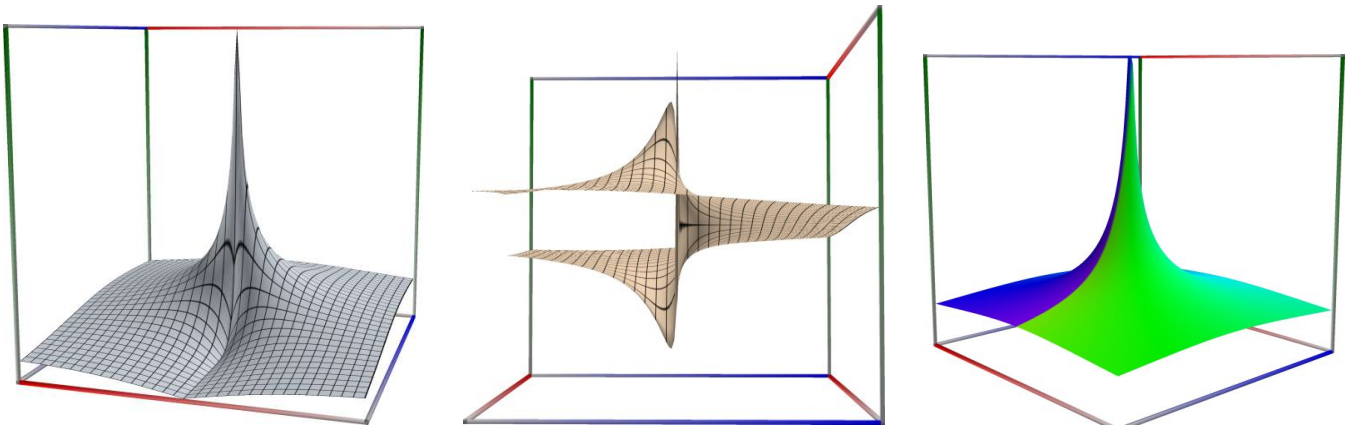
$$K_\nu(z) = \tfrac{1}{2}\pi \frac{I_{-\nu}(z) - I_\nu(z)}{\sin(\nu\pi)}.$$

if $\nu \notin \mathbb{Z}$. If $\nu \in \mathbb{Z}$, it computes the limit value.

The *wpf figures* below are showing the real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex function $z = K_\nu(x + iy)$ with $\nu = 0$ and $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.



The corresponding scaled function looks like this:



An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselK(3, 0.5)
```

(continues on next page)

(continued from previous page)

```
ereal('5.2359877559829887307E-1')
>>> math53.BesselK(3, '0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n= 10; x = 30
>>> \mathrm{d}x = dec.besselk(n, x); mx = mpm.besselk(n, x); gx = gmp.besselk(n, x)
>>> fx = fpm.besselk(n, x); ax = apm.besselk(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.0842816942222973911037536135816849208 80E-13
mpm:  1.0842816942222973911037536135816849208 80e-13
gmp:  1.0842816942222973911037536135816849208 80E-13
fpm:  1.08428169422230E-13
apm:  1.0842816942222973908438382241781852751 64e-13 (1.245e-14%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.besselk(n, z); mz = mpm.besselk(n, z); gz = gmp.besselk(n, z)
>>> fz = fpm.besselk(n, z); az = apm.besselk(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -1.9416209202983682785E+1               + 1.1318334583389307477E+1j
mpm: -1.9416209202983682785e+1               + 1.1318334583389307477e+1j
gmp: -1.9416209202983682785E+01              + 1.1318334583389307477E+01j
fpm: -1.94162092029837E+01                   + 1.13183345833893E+01j
apm: -1.9416209202983682785e+1 (-2.792e-19%) + 1.1318334583389307477e+1 (3.592e-19%)j
```

10.3.3 First derivative of the modified Bessel function of the first kind $I'_\nu(x)$

`ctx.bessel_iv_prime`(*x*, *nu*, *scaled=False*)

Returns $I'_\nu(x)$, the first derivative (with respect to x) of $I_\nu(z)$, the modified Bessel function of the first kind

If *scaled* is *True*, then $I'_\nu(x) \cdot \exp(-|\Re(x)|)$ is returned.

See also Wikipedia [1264], MathWorld [907], NIST [445], BoostMath [108], BoostMath [116].

If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $\nu, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $\nu, x \in \mathbb{C}$ is accepted.

The function is calculated as $I'_\nu(x) = \frac{1}{2}(I_{\nu-1}(x) + I_{\nu+1}(x))$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselIPrime(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.BesselIPrime(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.3.4 First derivative of the modified Bessel function of the second kind $K'_\nu(x)$

`ctx.bessel_kv_prime`(*x*, *nu*, *scaled=False*)

Returns $K'_\nu(x)$, the first derivative (with respect to x) of $K_\nu(x)$, the modified Bessel function of the second kind.

If *scaled* is *True*, then $K'_\nu(x) \cdot \exp(x)$ is returned.

See also Wikipedia [1250], MathWorld [878], NIST [440], BoostMath [94], BoostMath [116].

If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $\nu, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $\nu, x \in \mathbb{C}$ is accepted.

The function is calculated as $K'_\nu(x) = -\frac{1}{2}(K_{\nu-1}(x) + K_{\nu+1}(x))$.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; x = 30
>>> \mathrm{d}x = dec.hankel1(n, x); mx = mpm.hankel1(n, x); gx = gmp.hankel1(n, x)
>>> fx = fpm.hankel1(n, x); ax = apm.hankel1(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -1.2987689399858876819E-1                + 7.5056702122397113289E-2j
mpm: -1.2987689399858876819e-1                + 7.5056702122397113289e-2j
gmp: -1.2987689399858876819E-01               + 7.5056702122397113289E-02j
fpm: -1.29876893998589E-01                    + 7.50567021223971E-02j
apm: -1.2987689399889094748e-1 (-9.932e-10%) + 7.50567021139008222249e-2 (2.838e-6%)j
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.hankel1(n, z); mz = mpm.hankel1(n, z); gz = gmp.hankel1(n, z)
>>> fz = fpm.hankel1(n, z); az = apm.hankel1(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -6.9984001073685610955E+0               + 6.7915518863025118064E+0j
mpm: -6.9984001073685610955e+0               + 6.7915518863025118064e+0j
gmp: -6.9984001073685610955E+00              + 6.7915518863025118064E+00j
fpm: -6.99840010736856E+00                   + 6.79155188630251E+00j
apm: -6.9984001073685610955e+0 (-2.13e-18%) + 6.7915518863025118064e+0 (2.145e-18%)j
```

10.3.5 Marcum Q function, $Q_m(a, b)$ (up to octuple precision)

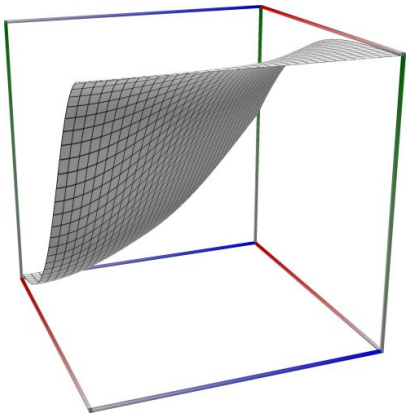
`ctx.marcum_q(`*m*, *a*, *b*`)`

Returns the Marcum Q-function $Q_m(a,b) = \frac{1}{a^{m-1}} \int_b^\infty x^m \exp\left(-\frac{x^2+a^2}{2}\right) I_{m-1}(ax)\,\mathrm{d}x$, where $m > 0$, $b \geq 0$, $a > 0$ and I_{m-1} is the modified Bessel function of first kind of order $m-1$.

See also: Short [499], Wikipedia [1352], MathWorld [1012], Ehrhardt [297] (3.3.17).

It is calculated using its relationship to the noncentral chi-squared cumulative distribution function:

If $X \sim \chi^2_k(\lambda)$ is a non-central chi-squared distribution with non-centrality parameter λ and k degrees of freedom, then its cdf is given by $F_X(x) = 1 - Q_{k/2}(\sqrt{\lambda}, \sqrt{x})$.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.MarcumQ(2, 3 ,0.5)
ereal('5.2359877559829887307E-1')
>>> math53.MarcumQ(2.4, 3.6, ' 0.51')
ereal('5.3518479027559984754E-1')
```

10.4 Spherical Bessel functions

10.4.1 Spherical Bessel function of the first kind, $j_n(x)$

`ctx.sph_bessel_jn(`*n*, *x*, *scaled=False*`)`

Returns $j_n(x)$, the spherical Bessel function of the first kind.

If *scaled* is *True*, then $j_\nu(x) \cdot \exp(-|\Im(x)|)$ is returned, which means that for purely real x just $j_\nu(x)$ is returned.

See also Wikipedia [1315], MathWorld [944], NIST [447], BoostMath [128], Ehrhardt [297] (3.1.6.1).

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $n, x \in \mathbb{C}$ is accepted.

This function is traditionally defined as $j_n(x) = \sqrt{\frac{1}{2}\pi/x} J_{n+\frac{1}{2}}(x)$, where $J_n(x)$ is a Bessel function of the first kind.

In XlCalcNet, the definition $j_n(x) = \sqrt{\frac{1}{2}\pi}\frac{1}{\sqrt{x}} J_{n+\frac{1}{2}}(x)$ is used instead, which gives identical results for $\Re(x) \geq 0$, but differs from the "traditional version" along the branch cut of the square root function. This ensures that the general definition for complex n and complex x agrees with the results from Rayleigh's formula $j_n(x) = (-x)^n \left(\frac{1}{x}\frac{d}{dx}\right)^n \frac{\sin x}{x}$ for integer $n \geq 0$ and real or complex $x \neq 0$. Given $j_0(x)$ and $j_1(x)$, Rayleigh's formula can also be computed recursively as $j_n(x) = \frac{2n-1}{x} j_{n-1}(x) - j_{n-2}(x)$ for $n \geq 2$.

We extend this to negative integer indices using $j_{-n}(x) = (-1)^n y_{n+1}(x)$ for $n > 0$, where $y_n(x)$ is the spherical Bessel function of the second kind. We also have $j_n(-x) = (-1)^n j_{n+1}(x)$ for $x > 0$.

For $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ the function is real-valued. With $j_0(0) = 1$, $j_n(0) = 0$ for $n > 0$ and $n \neq 0$, $j_n(0) = -\infty$ for even $n < 0$, $j_n(0) =$ NaN for odd $n < 0$, $j_n(-\infty) = j_n(\infty) = 0$, we have the following explicit versions of Rayleigh's formula for $x \neq 0$ and $n = 0, 1, 2, 3$:

$$j_0(x) = \frac{\sin x}{x}.$$

$$j_1(x) = \frac{\sin x}{x^2} - \frac{\cos x}{x}.$$

$$j_2(x) = y_{-1}(x) = \left(\frac{3}{x^2} - 1\right)\frac{\sin x}{x} - \frac{3\cos x}{x^2}.$$

$$j_3(x) = y_{-2}(x) = \left(\frac{15}{x^3} - \frac{6}{x}\right)\frac{\sin x}{x} - \left(\frac{15}{x^2} - 1\right)\frac{\cos x}{x}.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SphBesseljn(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.SphBesseljn(3, '0.51')
ereal('5.3518479027559984754E-1')
```

10.4.2 Spherical Bessel function of the second kind, $y_n(x)$

`ctx.sph_bessel_yn`(*n*, *x*, *scaled=False*)

Returns $y_n(x)$, the Spherical Bessel function of the second kind.

If *scaled* is *True*, then $y_\nu(x) \cdot \exp(-|\Im(x)|)$ is returned, which means that for purely real x just $y_\nu(x)$ is returned.

See also Wikipedia [1315], MathWorld [945], NIST [447], BoostMath [128], Ehrhardt [297] (3.1.6.2).

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $n, x \in \mathbb{C}$ is accepted.

This function is traditionally defined as $y_n(x) = \sqrt{\frac{1}{2}\pi/x} Y_{n+\frac{1}{2}}(x)$, where $Y_n(x)$ is a Bessel function of the second kind and, in general, n and x are complex numbers.

In XlCalcNet, the definition $y_n(x) = \sqrt{\frac{1}{2}\pi}\frac{1}{\sqrt{x}} Y_{n+\frac{1}{2}}(x)$ is used instead, which gives identical results for $\Re(x) \geq 0$, but differs from the "traditional version" along the branch cut of the square root function. This ensures that results for real input and complex input with zero imaginary part are the same. It also ensures that the general definition for complex n and complex x agrees with the results from Rayleigh's formula $y_n(x) = -(-x)^n \left(\frac{1}{x}\frac{d}{dx}\right)^n \frac{\cos x}{x}$ for integer $n \geq 0$ and real or complex $x \neq 0$. Given $y_0(x)$ and $y_1(x)$, Rayleigh's formula can also be computed recursively as $y_n(x) = \frac{2n+1}{x} y_{n-1}(x) - y_{n-2}(x)$ for $n \geq 2$.

With $y_n(0) =$ NaN for even n, $y_n(0) = -\infty$ for odd n, $y_n(-\infty) = y_n(\infty) = 0$, we have the following explicit versions of Rayleigh's formula for $x \neq 0$ and $n = 0, 1, 2, 3$:

$$y_0(x) = -j_{-1}(x) = -\frac{\cos x}{x},$$

$$y_1(x) = j_{-2}(x) = -\frac{\cos x}{x^2} - \frac{\sin x}{x},$$

$$y_2(x) = -j_{-3}(x) = \left(-\frac{3}{x^2} + 1\right)\frac{\cos x}{x} - \frac{3\sin x}{x^2},$$

$$y_3(x) = j_{-4}(x) = \left(-\frac{15}{x^3} + \frac{6}{x}\right)\frac{\cos x}{x} - \left(\frac{15}{x^2} - 1\right)\frac{\sin x}{x}.$$

We extend this to negative integer indices using $y_{-n}(x) = (-1)^{n+1} j_{n+1}(x)$ for $n > 0$, where $j_n(x)$ is the spherical Bessel function of the first kind. We also have $y_n(-x) = (-1)^{n+1} y_n(x)$ for $x > 0$. Therefore we can calculate $y_n(x)$ for any $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ without resorting to complex functions, using the general definition with $n \geq 0, x \geq 0$, always obtaining a real result.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SphBesselyn(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.SphBesselyn(3, '0.51')
ereal('5.3518479027559984754E-1')
```

10.4.3 First derivative of the spherical Bessel function of the first kind, $j_n'(x)$

`ctx.sph_bessel_jn_prime`(*n*, *x*, *scaled=False*)

Returns $j_n'(x)$, the first derivative (with respect to x) of $j_n(x)$, the spherical Bessel function of the first kind.

If *scaled* is *True*, then $j_\nu'(x) \cdot \exp(-|\Im(x)|)$ is returned, which means that for purely real x just $j_\nu'(x)$ is returned.

See also Wikipedia [1315], MathWorld [944], NIST [447], BoostMath [128], Ehrhardt [297] (3.1.6.1).

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $n, x \in \mathbb{C}$ is accepted.

In all cases, the function is calculated as

$j_n'(x) = \dfrac{n j_{n-1}(x) - (n+1) j_{n+1}(x)}{2n+1}$ for $|2n+1| > 0.1$ and $j_n'(x) = j_{n-1}(x) - \dfrac{n+1}{x} j_n(x)$ otherwise.

For $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ the function is real-valued with $j_1'(0) = 1/3$, $j_n'(0) = 0$ for $n \geq 0$ and $n \neq 1$, $j_{-n}'(0) = -\infty$ for odd $n > 0$ and $j_{-n}'(0) =$ NaN for even $n > 0$. We also have $j_n(-\infty) = 0$ and $y_n(\infty) = 0$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SphBesseljn(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.SphBesseljn(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.4.4 First derivative of the spherical Bessel function of the second kind, $y_n'(x)$

`ctx.sph_bessel_yn_prime`(*n*, *x*, *scaled=False*)

Returns $y_n'(x)$, the first derivative (with respect to x) of $y_n(x)$, the spherical Bessel function of the second kind.

If *scaled* is *True*, then $y_\nu'(x) \cdot \exp(-|\Im(x)|)$ is returned, which means that for purely real x just $y_\nu'(x)$ is returned.

See also Wikipedia [1315], MathWorld [945], NIST [447], BoostMath [128], Ehrhardt [297] (3.1.6.2).

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $n, x \in \mathbb{C}$ is accepted.

In all cases, the function is calculated as

$y_n'(x) = \dfrac{n y_{n-1}(x) - (n+1) y_{n+1}(x)}{2n+1}$ for $|2n+1| > 0.1$ and $y_n'(x) = y_{n-1}(x) - \dfrac{n+1}{x} y_n(x)$ otherwise.

For $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ the function is real-valued with $y_{-2}'(0) = -1/3$, $y_n'(0) = 0$ for $n < 0$ and $n \neq -2$, $y_n'(0) = \infty$ for even $n \geq 0$ and $y_n'(0) =$ NaN for odd $n > 0$. We also have $y_n(-\infty) = 0$ and $y_n(\infty) = 0$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SphBesselyn(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.SphBesselyn(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.4.5 Boost: Zeros $a_{n,m}$ of the spherical Bessel function of the first kind: $j_n(a_{n,m}) = 0$

`ctx.sph_bessel_jn_zero(`*n*, *m*`)`

where `ctx` is `math53` or `ctxboost`.

For positive integers n and m, returns $a_{n,m}$, the m-th positive zero of the spherical Bessel function of the first kind $j_n(x)$.

See also Wikipedia [1250], MathWorld [949], NIST [442], BoostMath [119], Mpmath [640].

The function is calculated as $a_{n,m} = j_{n+\frac{1}{2},m}$, where $j_{\nu,m}$ is the m-th positive zero of the Bessel function of the first kind $J_\nu(x)$.

See: https://dlmf.nist.gov/10.58

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselJZero(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselJZero(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.4.6 Boost: Zeros $b_{n,m}$ of the spherical Bessel function of the second kind: $y_n(b_{n,m}) = 0$

`ctx.sph_bessel_yn_zero(`*n*, *m*`)`

where `ctx` is `math53` or `ctxboost`.

For positive integers n and m, returns $b_{n,m}$, the m-th positive zero of the spherical Bessel function of the second kind $y_n(x)$.

Returns the zero of the spherical Bessel function of the second kind. See also Wikipedia [1251], MathWorld [950], NIST [442], BoostMath [119], Mpmath [641].

The function is calculated as $b_{n,m} = y_{n+\frac{1}{2},m}$, where $y_{\nu,m}$ is the m-th positive zero of the Bessel function of the first kind $Y_\nu(x)$.

See https://dlmf.nist.gov/10.58

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselJZero(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.BesselJZero(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.5 Modified spherical Bessel functions

10.5.1 Modified spherical Bessel function of the first kind, $i_n(x)$

`ctx.sph_bessel_in(n, x, scaled=False)`

Returns $i_n(x)$, the modified spherical Bessel function of the first kind.

If *scaled* is *True*, then $i_n(x) \cdot \exp(-|\Re(x)|)$ is returned.

See also Wikipedia [1315], MathWorld [942], NIST [447], BoostMath [128], Ehrhardt [297] (3.1.6.3).

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $n, x \in \mathbb{C}$ is accepted.

This function is traditionally defined as $i_n(x) = \sqrt{\frac{1}{2}\pi/x} I_{n+\frac{1}{2}}(x)$, where $I_n(x)$ is a modified Bessel function of the first kind.

In XlCalcNet, the definition $i_n(x) = \sqrt{\frac{1}{2}\pi}\frac{1}{\sqrt{x}} I_{n+\frac{1}{2}}(x)$ is used instead, which gives identical results for $\Re(x) \geq 0$, but differs from the "traditional version" along the branch cut of the square root function. This ensures that results for real input and complex input with zero imaginary part are the same.

It also ensures that the general definition for complex n and complex x agrees with the results from the following two Rayleigh-type formulas:

$i_n(x) = x^n \left(\frac{1}{x}\frac{d}{dx}\right)^n \frac{\sinh(x)}{x}$ for integer $n \geq 0$ and real or complex $x \neq 0$. Given $i_0(x)$ and $i_1(x)$, this Rayleigh-type formula can also be computed recursively as $i_n(x) = \frac{2n-1}{x} i_{n-1}(x) - i_{n-2}(x)$ for $n \geq 2$.

$i_{-n-1}(x) = x^n \left(\frac{1}{x}\frac{d}{dx}\right)^n \frac{\cosh(x)}{x}$ for integer $n \geq 0$ and real or complex $x \neq 0$. Given $i_0(x)$ and $i_1(x)$, this Rayleigh-type formula can also be computed recursively as $i_n(x) = \frac{2n+3}{x} i_{n+1}(x) - i_{n+2}(x)$ for $n \leq -1$.

With $i_0(0) = 1$, $i_n(0) = \text{NaN}$ for even $n \neq 0$, $i_n(0) = -\infty$ for odd n, $i_n(-\infty) = i_n(\infty) = 0$, we have the following explicit versions of the Rayleigh-type formulas for $x \neq 0$ and $n = -3, -2, -1, 0, 1, 2$:

$$i_{-3}(x) = \frac{(3+x^2)\cosh x}{x^3} - \frac{3\sinh x}{x^2},$$

$$i_{-2}(x) = -\frac{\cosh x}{x^2} + \frac{\sinh x}{x},$$

$$i_{-1}(x) = \frac{\cosh x}{x},$$

$$i_0(x) = \frac{\sinh x}{x},$$

$$i_1(x) = -\frac{\sinh x}{x^2} + \frac{\cosh x}{x},$$

$$i_2(x) = \frac{(3+x^2)\sinh x}{x^3} - \frac{3\cosh x}{x^2}.$$

We also have $i_n(-x) = (-1)^n i_n(x)$ for $x > 0$. Therefore we can calculate $i_n(x)$ for any $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ without resorting to complex functions, using the general definition with $n \geq 0$, always obtaining a real result.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SphBesselin(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.SphBesselin(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.5.2 Modified Spherical Bessel function of the second kind, $k_n(x)$

`ctx.sph_bessel_kn`(*n*, *x*, *scaled=False*)

Returns $k_n(x)$, the modified spherical Bessel function of the second kind.

If *scaled* is *True*, then $k_n(x) \cdot \exp(x)$ is returned.

See also Wikipedia [1315], MathWorld [943], NIST [447], BoostMath [128], Ehrhardt [297] (3.1.6.5).

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $n, x \in \mathbb{C}$ is accepted.

This function is traditionally defined as $k_n(x) = \sqrt{\frac{1}{2}\pi/x} K_{n+\frac{1}{2}}(x)$, where $K_n(x)$ is a modified Bessel function of the first kind.

In XlCalcNet, the definition $k_n(x) = \sqrt{\frac{1}{2}\pi} \frac{1}{\sqrt{x}} K_{n+\frac{1}{2}}(x)$ is used instead, which gives identical results for $\Re(x) \geq 0$, but differs from the "traditional version" along the branch cut of the square root function. This ensures that results for real input and complex input with zero imaginary part are the same.

It also ensures that the general definition for complex n and complex x agrees with the results from the following Rayleigh-type formula:

$k_n(x) = \frac{\pi}{2}(-x)^n \left(\frac{1}{x}\frac{d}{dx}\right)^n \frac{e^{-x}}{x}$ for integer $n \geq 0$ and real or complex $x \neq 0$. Given $k_0(x)$ and $k_1(x)$, this Rayleigh-type formula can also be computed recursively as $k_n(x) = \frac{2n-1}{x} k_{n-1}(x) - k_{n-2}(x)$ for $n \geq 2$.

With $k_n(0) = \text{NaN}$ for even n, $k_n(0) = +\infty$ for odd n, $k_n(-\infty) = -\infty$, $k_n(+\infty) = 0$, we have the following explicit versions of the Rayleigh-type formula for $x \neq 0$ and $n = 0, 1, 2$:

$$k_0(x) = \frac{\pi}{2}\frac{1}{x}e^{-x},$$

$$k_1(x) = \frac{\pi}{2}\left(\frac{1}{x^2} + \frac{1}{x}\right)e^{-x},$$

$$k_2(x) = \frac{\pi}{2}\left(\frac{3}{x^3} + \frac{3}{x^2} + \frac{1}{x}\right)e^{-x}.$$

We also have $k_{-n}(x) = k_n(x)$ for $n > 0$ and $k_n(-x) = \frac{\pi}{2}(i_n(x) - i_{-n-1}(x))$ for $x > 0$. Therefore we can calculate $k_n(x)$ for any $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ without resorting to complex functions, using the general definition with $n \geq 0$, always obtaining a real result.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SphBesselkn(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.SphBesselkn(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.5.3 First derivative of the modified spherical Bessel function of the first kind, $i'_n(x)$

`ctx.sph_bessel_in_prime`(*n*, *x*, *scaled=False*)

Returns $i'_n(x)$, the first derivative (with respect to x) of $i_n(x)$, the modified spherical Bessel function of the first kind.

If *scaled* is *True*, then $i'_n(x) \cdot \exp(-|\Re(x)|)$ is returned.

See also Wikipedia [1315], MathWorld [942], NIST [447], BoostMath [128], Ehrhardt [297] (3.1.6.3).

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $n, x \in \mathbb{C}$ is accepted.

In all cases, the function is calculated as

$i'_n(x) = \dfrac{ni_{n-1}(x) - (n+1)i_{n+1}(x)}{2n+1}$ for $|2n+1| > 0.1$ and $i'_n(x) = i_{n-1}(x) - \dfrac{n+1}{x} i_n(x)$ otherwise.

For $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ the function is real-valued with $i'_1(0) = 1/3$, $i'_n(0) = 0$ for $n \geq 0$ and $n \neq 1$, $i'_{-n}(0) = -\infty$ for odd $n > 0$ and $i'_{-n}(0) =$ NaN for even $n > 0$. We also have $i_n(-\infty) = 0$ and $y_n(\infty) = 0$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SphBesselin(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.SphBesselin(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.5.4 First derivative of the modified spherical Bessel function of the second kind, $k_n'(x)$

`ctx.sph_bessel_kn_prime`(*n*, *x*, *scaled=False*)

Returns $k_n'(x)$, the first derivative (with respect to x) of $k_n(x)$, the modified spherical Bessel function of the second kind.

If *scaled* is *True*, then $K_\nu(x) \cdot \exp(x)$ is returned.

See also Wikipedia [1315], MathWorld [943], NIST [447], BoostMath [128], Ehrhardt [297] (3.1.6.5).

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $n, x \in \mathbb{C}$ is accepted.

In all cases, the function is calculated as

$k_n'(x) = \dfrac{nk_{n-1}(x) - (n+1)k_{n+1}(x)}{2n+1}$ for $|2n+1| > 0.1$ and $k_n'(x) = k_{n-1}(x) - \dfrac{n+1}{x}k_n(x)$ otherwise.

For $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ the function is real-valued with $k_n'(0) = -\infty$ for even $n > 0$ and odd $n < 0$ and $k_n'(0) =$ NaN for odd $n > 0$ and even $n < 0$. We also have $k_n(-\infty) = 0$ and $y_n(\infty) = 0$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SphBesselkn(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.SphBesselkn(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.5.5 Bessel polynomials $y_n(x)$

`ctx.besselpoly(n, x)`

Returns $y_n(x)$, the Bessel polynomial of order n.

For $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ the function can be defined in terms of the modified spherical Bessel function of the second kind, $k_n(x)$:

$$y_n(x) = 2\pi^{-1}x^{-1}e^{1/x}k_n\left(x^{-1}\right), \quad n \geq 0,$$

and $y_{-n}(x) = y_{n-1}(x)$ for negative order.

The Bessel polynomial may also be defined by a recursion formula:

$$\begin{aligned} y_0(x) &= 1 \\ y_1(x) &= x + 1 \\ y_n(x) &= (2n-1)xy_{n-1}(x) + y_{n-2}(x). \end{aligned} \tag{10.1}$$

The function can be generalized to arbitrary complex n and x by expressing it in terms of Tricomi's confluent hypergeometric function, $U(a, b; x)$:

$$y_n(x) = 2^{n+1}\left(\frac{1}{x}\right)^{n+1} U\left(n+1, 2n+2; \frac{2}{x}\right), x \neq 0$$

and the limit (if it exists) for $x = 0$.

See also Ehrhardt [297] (3.7.19).

See also: https://en.wikipedia.org/wiki/Bessel_polynomials

See also: https://dlmf.nist.gov/18.34

See also: https://mathworld.wolfram.com/BesselPolynomial.html

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselPoly(4, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselPoly(4, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.5.6 Reverse Bessel polynomials $\theta_n(x)$

`ctx.besseltheta(n, x)`

Returns $\theta_n(x)$, the reverse Bessel polynomial of order n.

For $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ the function can be defined in terms of the modified spherical Bessel function of the second kind, $k_n(x)$:

$$\theta_n(x) = 2\pi^{-1}x^{n+1}e^x k_n(x), \quad n \geq 0,$$

and $\theta_{-n}(x) = x^{-n}y_{n-1}(1/x)$ for negative order, where $y_n(x)$ is the Bessel polynomial.

The reverse Bessel polynomial may also be defined by a recursion formula:

$$\begin{aligned} \theta_0(x) &= 1 \\ \theta_1(x) &= x + 1 \\ \theta_n(x) &= (2n-1)\theta_{n-1}(x) + x^2\theta_{n-2}(x). \end{aligned} \tag{10.2}$$

The function can be generalized to arbitrary complex n and x by expressing it in terms of Tricomi's confluent hypergeometric function, $U(a, b; x)$:

$$\theta_n(x) = 2^{n+1}x^{2n+1}U\left(n+1, 2n+2, 2x\right), x \neq 0$$

and the limit (if it exists) for $x = 0$.

See also: https://en.wikipedia.org/wiki/Bessel_polynomials

See also: https://dlmf.nist.gov/18.34

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselPoly(4, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.BesselPoly(4, '0.51')
ereal('5.351847902755998475 4E-1')
```

10.6 Hankel functions

10.6.1 Hankel function of the first kind $H_\nu^{(1)}(x)$

ctx.**hankel_h1**(*nu*, *x*, *scaled=False*)

where ctx is math53 or ctxboost.

Returns the Hankel function of the first kind, defined as $H_\nu^{(1)}(x) = J_\nu(x) + iY_\nu(x)$.

If *scaled* is *True*, then $H_\nu^{(1)e}(x) = H_\nu^{(1)}(x) \cdot \exp(-ix)$ is returned, except for a real ctx where just $H_\nu^{(1)}(x)$ is returned.

If ctx is math53, ctxboost or ctxflintreal, then $\nu, x \in \mathbb{R}$ is exspected. If ctx is cmath53, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If ctx is ctxflintcplx then $\nu, x \in \mathbb{C}$ is accepted.

See also Wikipedia [1306], MathWorld [940], NIST [443], BoostMath [115], Mpmath [629].

```
>>> from xlcalcnet import ecplx
>>> ecplx.HankelH1(10.5, 6.3)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.HankelH1(10.5, 6.3)
ecplx('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; x = 30
>>> \mathrm{d}x = dec.hankel1(n, x); mx = mpm.hankel1(n, x); gx = gmp.hankel1(n, x)
>>> fx = fpm.hankel1(n, x); ax = apm.hankel1(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -1.2987689399858876819E-1                + 7.5056702122397113289E-2j
mpm: -1.2987689399858876819e-1                + 7.5056702122397113289e-2j
gmp: -1.2987689399858876819E-01               + 7.5056702122397113289E-02j
fpm: -1.29876893998589E-01                    + 7.50567021223971E-02j
apm: -1.2987689399889094748e-1 (-9.932e-10%) + 7.5056702113900822249e-2 (2.838e-6%)j
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.hankel1(n, z); mz = mpm.hankel1(n, z); gz = gmp.hankel1(n, z)
>>> fz = fpm.hankel1(n, z); az = apm.hankel1(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -6.9984001073685610955E+0               + 6.7915518863025118064E+0j
mpm: -6.9984001073685610955e+0               + 6.7915518863025118064e+0j
gmp: -6.9984001073685610955E+00              + 6.7915518863025118064E+00j
fpm: -6.99840010736856E+00                   + 6.79155188630251E+00j
apm: -6.9984001073685610955e+0 (-2.13e-18%) + 6.7915518863025118064e+0 (2.145e-18%)j
```

10.6.2 Hankel function of the second kind $H_\nu^{(2)}(x)$

`ctx.hankel_h2(nu, x, scaled=False)`

where `ctx` is `math53` or `ctxboost`.

Returns the Hankel function of the second kind, defined as $H_\nu^{(2)}(x) = J_\nu(x) - iY_\nu(x)$.

If *scaled* is *True*, then $H_\nu^{(2)e}(x) = H_\nu^{(2)}(x) \cdot \exp(ix)$ is returned, except for a real `ctx` where just $H_\nu^{(2)}(x)$ is returned.

If `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $\nu, x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $\nu, x \in \mathbb{C}$ is accepted.

See also Wikipedia [1306], MathWorld [941], NIST [443], BoostMath [115], Mpmath [630].

```
>>> from xlcalcnet import ecplx
>>> ecplx.HankelH2(10.5, 6.3)
ecplx('5.2359877559829887307E-1')
>>> ecplx.HankelH2(10.5, 6.3)
ecplx('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; x = 30
>>> \mathrm{d}x = dec.hankel2(n, x); mx = mpm.hankel2(n, x); gx = gmp.hankel2(n, x)
>>> fx = fpm.hankel2(n, x); ax = apm.hankel2(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -1.2987689399858876819E-1                - 7.5056702122397113289E-2j
mpm: -1.2987689399858876819e-1                - 7.5056702122397113289e-2j
gmp: -1.2987689399858876819E-01               - 7.5056702122397113289E-02j
fpm: -1.29876893998589E-01                    - 7.50567021223971E-02j
apm: -1.2987689399889094748e-1 (-9.932e-10%) - 7.505670211390082249e-2 (-2.838e-6%)j
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.hankel2(n, z); mz = mpm.hankel2(n, z); gz = gmp.hankel2(n, z)
>>> fz = fpm.hankel2(n, z); az = apm.hankel2(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 6.9935943604463042143E+0               - 6.7875888598187273523E+0j
mpm: 6.9935943604463042143e+0               - 6.7875888598187273523e+0j
gmp: 6.9935943604463042143E+00              - 6.7875888598187273523E+00j
fpm: 6.99359436044630E+00                   - 6.78758885981873E+00j
apm: 6.9935943604463042143e+0 (2.132e-18%) - 6.7875888598187273523e+0 (-2.146e-18%)j
```

10.6.3 Spherical Hankel function of the first kind, $h_\nu^{(1)}(x)$

`ctx.`**sph_hankel_h1**(*n*, *x*, *scaled=False*)

where `ctx` is `math53` or `ctxboost`.

Returns the spherical Hankel function of the first kind, defined as $h_n^{(1)}(x) = \sqrt{\frac{1}{2}\pi}\frac{1}{\sqrt{x}}H_\nu^{(1)}(x) = j_\nu(x) + iy_\nu(x)$.

If *scaled* is *True*, then $h_n^{(1)e}(x) = \sqrt{\frac{1}{2}\pi}\frac{1}{\sqrt{x}}H_\nu^{(1)e}(x)$ is returned.

See also Wikipedia [1316], MathWorld [946], NIST [448], BoostMath [129].

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $n, x \in \mathbb{C}$ is accepted.

```
>>> from xlcalcnet import ecplx
>>> ecplx.SphHankelH1(10.5, 6.3)
ecplx('5.2359877559829887307E-1')
>>> ecplx.SphHankelH1(10.5, 6.3)
ecplx('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n= 10; x = 30
>>> \mathrm{d}x = dec.sph_bessel_yn(n, x); mx = mpm.sph_bessel_yn(n, x); gx = gmp.sph_bessel_yn(n, x)
>>> fx = fpm.sph_bessel_yn(n, x); ax = apm.sph_bessel_yn(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: 3.12195910647549354077503891181253979113 8E-2
mpm: 3.12195910647549354077503891181253979113 8e-2
gmp: 3.12195910647549354077503891181253979113 8E-02
fpm: 3.12195910647549E-02
apm: 3.12195910647549354077503891181973673722 6e-2 (1.048e-27%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.sph_bessel_yn(n, z); mz = mpm.sph_bessel_yn(n, z); gz = gmp.sph_bessel_yn(n, z)
>>> fz = fpm.sph_bessel_yn(n, z); az = apm.sph_bessel_yn(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.0803151721461599990E+1              - 1.7336496520486643470E+0j
mpm: 1.0803151721461599990e+1              - 1.7336496520486643470e+0j
gmp: 1.0803151721461599990E+01             - 1.7336496520486643470E+00j
fpm: 1.08031517214616E+01                  - 1.733649652048 66E+00j
apm: 1.0803151721461599990e+1 (6.962e-18%) - 1.7336496520486643470e+0 (-3.835e-17%)j
```

10.6.4 Spherical Hankel function of the second kind, $h_\nu^{(2)}(x)$

`ctx.sph_hankel_h2`(*n*, *x*, *scaled=False*)

where `ctx` is `math53` or `ctxboost`.

Returns the spherical Hankel function of the first kind, defined as $h_n^{(2)}(x) = \sqrt{\frac{1}{2}\pi}\frac{1}{\sqrt{x}}H_\nu^{(2)}(x) = j_\nu(x) - iy_\nu(x)$.

If *scaled* is *True*, then $h_n^{(2)e}(x) = \sqrt{\frac{1}{2}\pi}\frac{1}{\sqrt{x}}H_\nu^{(2)e}(x)$ is returned.

See also Wikipedia [1316], MathWorld [946], NIST [448], BoostMath [129].

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, `ctxboost` or `ctxflintreal`, then $n \in \mathbb{Z}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflintcplx` then $n, x \in \mathbb{C}$ is accepted.

```
>>> from xlcalcnet import ecplx
>>> ecplx.SphHankelH1(10.5, 6.3)
ecplx('5.2359877559829887307E-1')
>>> ecplx.SphHankelH1(10.5, 6.3)
ecplx('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; x = 30
>>> \mathrm{d}x = dec.hankel2(n, x); mx = mpm.hankel2(n, x); gx = gmp.hankel2(n, x)
>>> fx = fpm.hankel2(n, x); ax = apm.hankel2(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -1.2987689399858876819E-1                - 7.5056702122397113289E-2j
mpm: -1.2987689399858876819e-1                - 7.5056702122397113289e-2j
gmp: -1.2987689399858876819E-01               - 7.5056702122397113289E-02j
fpm: -1.29876893998589E-01                    - 7.50567021223971E-02j
apm: -1.2987689399889094748e-1 (-9.932e-10%) - 7.505670211390082249e-2 (-2.838e-6%)j
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n= 10; z = '3 + 4j'
>>> \mathrm{d}z = dec.hankel2(n, z); mz = mpm.hankel2(n, z); gz = gmp.hankel2(n, z)
>>> fz = fpm.hankel2(n, z); az = apm.hankel2(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 6.9935943604463042143E+0               - 6.7875888598187273523E+0j
mpm: 6.9935943604463042143e+0               - 6.7875888598187273523e+0j
gmp: 6.9935943604463042143E+00              - 6.7875888598187273523E+00j
fpm: 6.99359436044630E+00                   - 6.78758885981873E+00j
apm: 6.9935943604463042143e+0 (2.132e-18%) - 6.7875888598187273523e+0 (-2.146e-18%)j
```

10.7 Airy functions

10.7.1 Airy function $\mathrm{Ai}(x)$

`ctx.airy_ai`(*x*, *scaled=False*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Airy function $\mathrm{Ai}(x)$, with $x \in \mathbb{C}$.

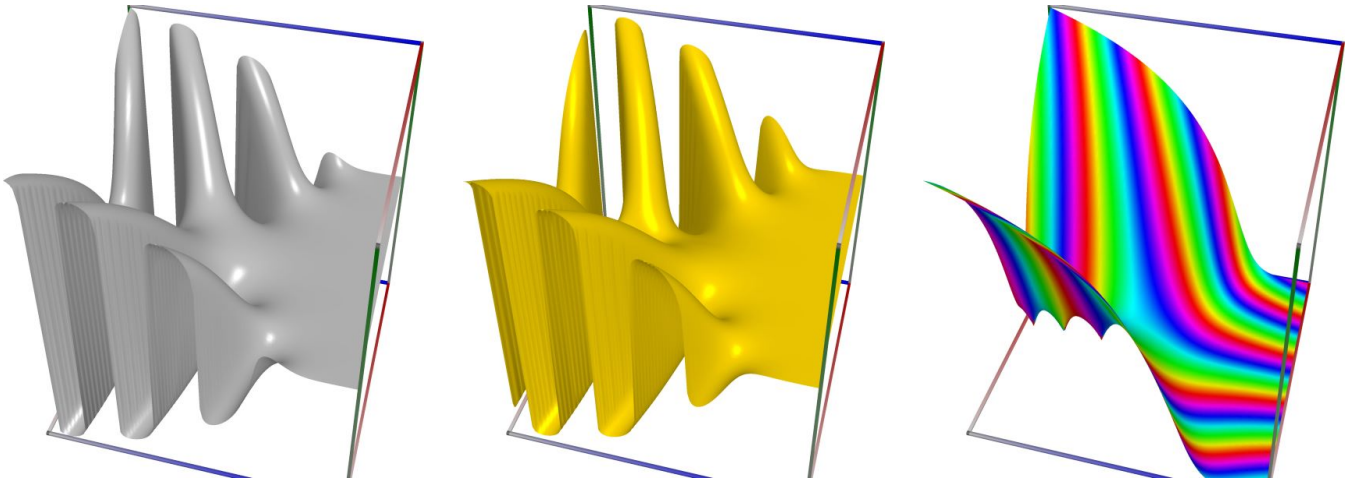
If *scaled* is *True*, then $\mathrm{Ai}(x)\exp(\sqrt{x}\cdot 2x/3)$ is returned, except for a real `ctx` with $x \le 0$ where just $\mathrm{Ai}(x)$ is returned.

See also Wikipedia [1295], MathWorld [939], MathWorld [262], NIST [433], BoostMath [112], Ehrhardt [297] (3.1.7.1), Mpmath [621].

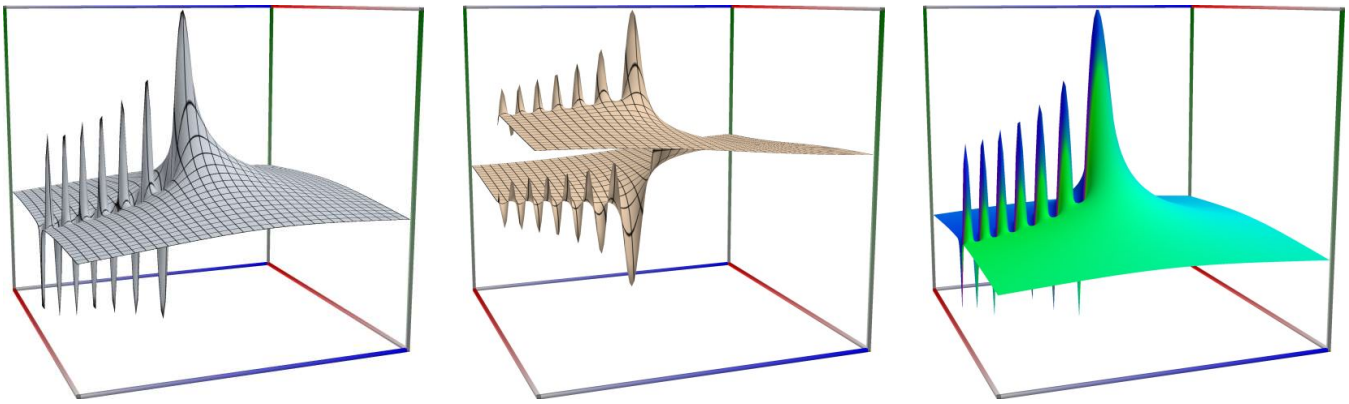
The function $\mathrm{Ai}(x)$ can be defined as

$$\mathrm{Ai}(x) = \frac{1}{3^{2/3}\Gamma(2/3)}{}_0F_1\left(\frac{2}{3}, \frac{x^3}{9}\right) - \frac{x}{3^{1/3}\Gamma(1/3)}{}_0F_1\left(\frac{4}{3}, \frac{x^3}{9}\right).$$

The *wpf figures* below are showing the real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex function $z = \mathrm{Ai}(x+iy)$ $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.



The corresponding scaled function looks like this:



An example in Python

```
>>> from xlcalcnet import math53
>>> math53.AiryAi(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.AiryAi('0.51')
ereal('5.3518479027559984754E-1')
```

10.7.2 Airy function $\mathrm{Bi}(x)$

`ctx.airy_bi`(*x*, *scaled=False*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the Airy function $\mathrm{Bi}(x)$, with $x \in \mathbb{C}$.

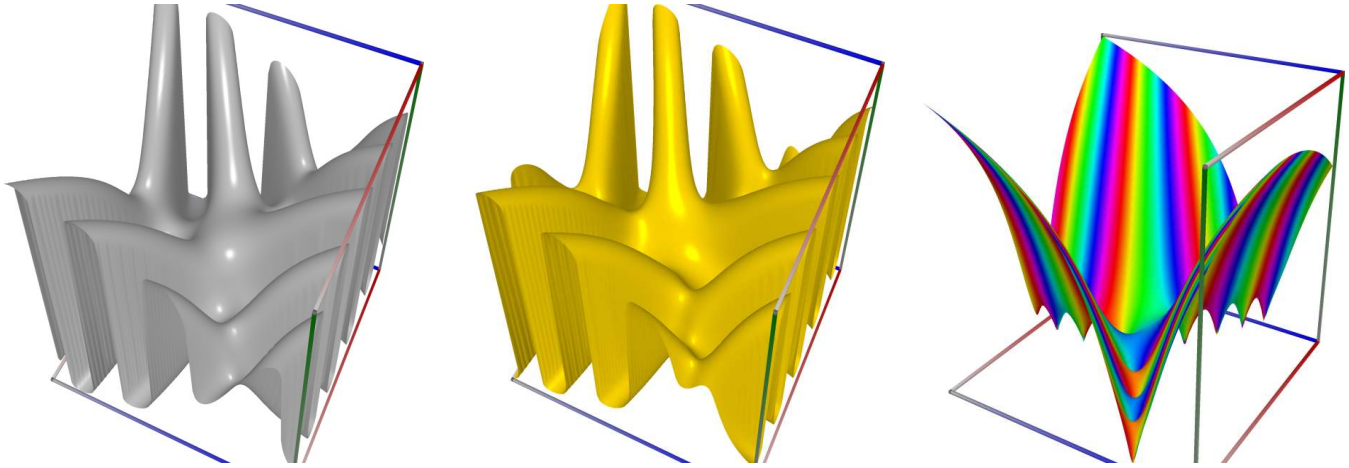
If *scaled* is *True*, then $\mathrm{Bi}(x)\exp(-|\Re(\sqrt{x}\cdot 2x/3)|)$ is returned, except for a real `ctx` with $x \le 0$ where just $\mathrm{Bi}(x)$ is returned.

See also Wikipedia [1295], MathWorld [939], MathWorld [264], NIST [433], BoostMath [113], Ehrhardt [297] (3.1.7.2), Mpmath [622].

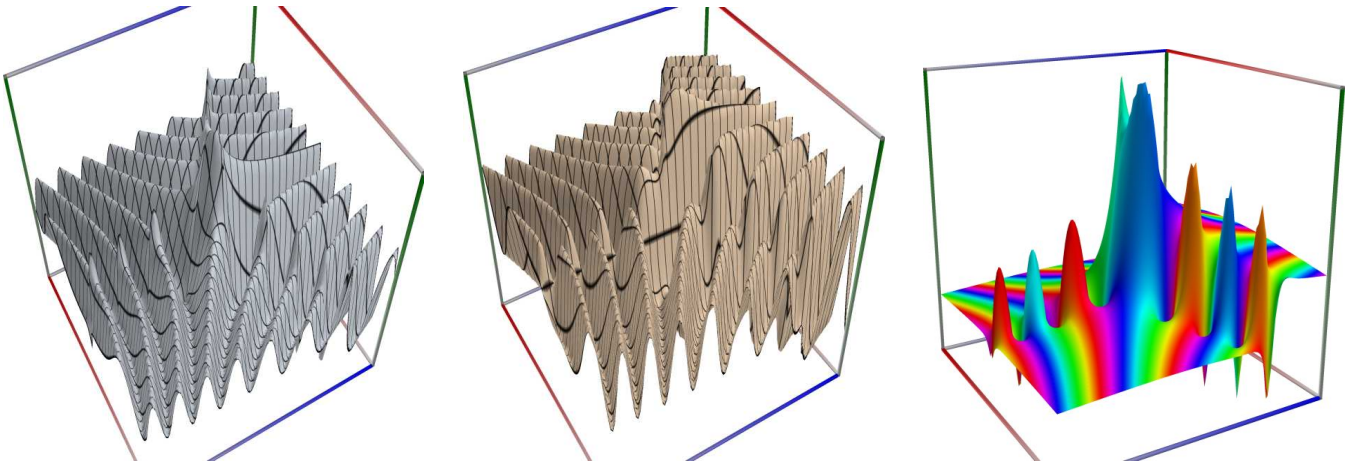
The function $\mathrm{Bi}(x)$ can be defined as

$$\mathrm{Bi}(x) = \frac{1}{3^{1/6}\Gamma(2/3)}{}_0F_1\left(\frac{2}{3}, \frac{x^3}{9}\right) + \frac{3^{1/6}x}{\Gamma(1/3)}{}_0F_1\left(\frac{4}{3}, \frac{x^3}{9}\right).$$

The *wpf figures* below are showing the real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex function $z = \mathrm{Bi}(x + iy)$ with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.



The corresponding scaled function looks like this:



An example in Python

```
>>> from xlcalcnet import math53
>>> math53.AiryBi(0.5)
```

(continues on next page)

(continued from previous page)

```
ereal('5.235987755982988730 7E-1')
>>> math53.AiryBi('0.51')
ereal('5.3518479027559984754E-1')
```

10.7.3 First derivative of the Airy function Ai, $\mathrm{Ai}'(x)$

`ctx.airy_ai_prime`(*x*, *scaled=False*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $\mathrm{Ai}'(x)$, the first derivative (with respect to x) of the Airy function $\mathrm{Ai}(x)$, with $x \in \mathbb{C}$.

If *scaled* is *True*, then $\mathrm{Ai}'(x)\exp(\sqrt{x}\cdot 2x/3)$ is returned, except for a real `ctx` with $x \le 0$ where just $\mathrm{Ai}'(x)$ is returned.

See also Wikipedia [1295], MathWorld [939], MathWorld [263], NIST [433], BoostMath [132], Ehrhardt [297] (3.1.7.3).

The function $\mathrm{Ai}'(x)$ can be defined as

$$\mathrm{Ai}'(x) = \frac{x^2}{2\cdot 3^{2/3}\Gamma(2/3)}{}_0F_1\left(\frac{5}{3}, \frac{x^3}{9}\right) - \frac{1}{3^{1/3}\Gamma(1/3)}{}_0F_1\left(\frac{1}{3}, \frac{x^3}{9}\right).$$

The *wpf figures* below are showing the real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex function $z = \mathrm{Ai}'(x+iy)$ with $\nu = 0$ and $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.AiryAiPrime(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.AiryAiPrime('0.51')
ereal('5.3518479027559984754E-1')
```

10.7.4 First derivative of the Airy function Bi, $\mathrm{Bi}'(x)$

`ctx.airy_bi_prime`(*x*, *scaled=False*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $\mathrm{Bi}'(x)$, the first derivative (with respect to x) of the Airy function $\mathrm{Bi}(x)$, with $x \in \mathbb{C}$.

If *scaled* is *True*, then $\mathrm{Bi}'(x)\exp(-|\Re(\sqrt{x}\cdot 2x/3)|)$ is returned, except for a real `ctx` with $x \le 0$ where just $\mathrm{Bi}'(x)$ is returned.

See also Wikipedia [1295], MathWorld [939], MathWorld [265], NIST [433], BoostMath [133], Ehrhardt [297] (3.1.7.4) and (3.1.7.6).

The function can be defined as

$$\mathrm{Bi}'(x) = \frac{x^2}{2\cdot 3^{1/6}\Gamma(2/3)}{}_0F_1\left(\frac{5}{3}, \frac{x^3}{9}\right) + \frac{3^{1/6}}{\Gamma(1/3)}{}_0F_1\left(\frac{1}{3}, \frac{x^3}{9}\right).$$

The *wpf figures* below are showing the real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex function $z = \mathrm{Bi}'(x+iy)$ with $\nu = 0$ and $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.AiryAiPrime(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.AiryAiPrime('0.51')
ereal('5.3518479027559984754E-1')
```

10.7.5 Real zeros a_k of the Airy function Ai, $\mathrm{Ai}(a_k) = 0$

`ctx.airy_ai_zero(`k`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the k-th zero of the Airy Ai-function. See also MathWorld [947], NIST [433], BoostMath [113], BoostMath [133], Mpmath [638].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.AiryAiZero(5)
ereal('5.235987755982988730 7E-1')
>>> math53.AiryAiZero(17)
ereal('5.351847902755998475 4E-1')
```

10.7.6 Real zeros b_k of the Airy function Bi, $\mathrm{Bi}(b_k) = 0$

`ctx.airy_bi_zero(`k`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the k-th zero of the Airy Bi-function. See also Wikipedia [1295], NIST [433], MathWorld [948], BoostMath [140], Mpmath [639].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.AiryBiZero(5)
ereal('5.2359877559829887307E-1')
>>> math53.AiryBiZero(17)
ereal('5.3518479027559984754E-1')
```

10.8 Kelvin functions

10.8.1 Kelvin function $\mathrm{ber}(\nu, x)$

`ctx.kelvin_ber`(*nu*, *z*, *scaled=False*)

Returns the Kelvin function $\mathrm{ber}(\nu, x)$.

If *scaled* is *True*, then $\mathrm{ber}(\nu, x) \cdot \exp(-|x|/\sqrt{2})$ is returned.

See also Wikipedia [1381], MathWorld [1022], NIST [444], Mpmath [708].

Here ν and x are, in general, complex numbers. However, if `ctx` is `math53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $n, x \in \mathbb{C}$ is accepted.

This function is traditionally defined for $\nu \in \mathbb{R}$ and real $x \geq 0$ as the real part of the Bessel J function of a rotated argument:

$$\mathrm{ber}(\nu, x) = \Re\Big(J_\nu\big(xe^{3\pi i/4}\big)\Big).$$

In XlCalcNet this is generalized to complex ν and x following the definition given in Maple [369] as

$$\mathrm{ber}(\nu, x) = \frac{J_\nu(x(-a+ia)) + J_\nu(x(-a-ia))}{2}, \quad \text{where } a = \tfrac{1}{2}\sqrt{2},$$

which is equivalent to the traditional definition for $\nu \in \mathbb{R}$ and real $x \geq 0$.

Note that this differs from Mpmath, which uses the conventions of Mathematica.

The Kelvin functions are all real valued for real x and positive ν.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinber(n, x); mx = mpm.kelvinber(n, x); gx = gmp.kelvinber(n, x)
>>> fx = fpm.kelvinber(n, x); ax = apm.kelvinber(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -2.213802495986938888682464345899509922321E-1
mpm: -2.213802495986938888682464345899509922321e-1
gmp: -2.213802495986938888682464345899509922321E-01
fpm: -2.21380249598694E-01
apm: -2.213802495986938888682464345899509922332e-1 (-1.283e-37%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.kelvinber(n, z); mz = mpm.kelvinber(n, z); gz = gmp.kelvinber(n, z)
>>> fz = fpm.kelvinber(n, z); az = apm.kelvinber(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.0336162101792974294E+1              + 7.7667689132221259171E+0j
mpm: 1.0336162101792974294e+1              + 7.7667689132221259171e+0j
gmp: 1.0336162101792974294E+01             + 7.7667689132221259171E+00j
fpm: 1.03361621017930E+01                  + 7.76676891322213E+00j
apm: 1.0336162101792974294e+1 (2.032e-18%) + 7.7667689132221259171e+0 (2.225e-18%)j
```

10.8.2 Kelvin function $\mathrm{bei}(\nu, x)$

`ctx.kelvin_bei`(*nu*, *z*, *scaled=False*)

Returns the Kelvin function $\mathrm{bei}(\nu, x)$.

If *scaled* is *True*, then $\mathrm{bei}(\nu, x) \cdot \exp(-|x|/\sqrt{2})$ is returned.

See also Wikipedia [1380], MathWorld [1021], NIST [444], Mpmath [707].

Here ν and x are, in general, complex numbers. However, if `ctx` is `math53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $n, x \in \mathbb{C}$ is accepted.

This function is traditionally defined for $\nu \in \mathbb{R}$ and real $x \geq 0$ as the imaginary part of the Bessel J function of a rotated argument:

$$\mathrm{bei}(\nu, x) = \Im\Big(J_\nu\big(xe^{3\pi i/4}\big)\Big).$$

In XlCalcNet this is generalized to complex ν and x following the definition given in Maple [369] as

$$\mathrm{bei}(\nu, x) = \frac{J_\nu(x(-a+ia)) - J_\nu(x(-a-ia))}{2}, \quad \text{where } a = \tfrac{1}{2i}\sqrt{2},$$

which is equivalent to the traditional definition for $\nu \in \mathbb{R}$ and real $x \geq 0$.

Note that this differs from Mpmath, which uses the conventions of Mathematica.

The Kelvin functions are all real valued for real x and positive ν.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinbei(n, x); mx = mpm.kelvinbei(n, x); gx = gmp.kelvinbei(n, x)
>>> fx = fpm.kelvinbei(n, x); ax = apm.kelvinbei(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: 1.937586785266042766896808122272260201255E+0
mpm: 1.937586785266042766896808122272260201255e+0
gmp: 1.937586785266042766896808122272260201255E+00
fpm: 1.93758678526604E+00
apm: 1.937586785266042766896808122272260201256e+0 (1.54e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.kelvinbei(n, z); mz = mpm.kelvinbei(n, z); gz = gmp.kelvinbei(n, z)
>>> fz = fpm.kelvinbei(n, z); az = apm.kelvinbei(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -7.5192613426702233947E+0               + 1.0562898806171521533E+1j
mpm: -7.5192613426702233947e+0               + 1.0562898806171521533e+1j
gmp: -7.5192613426702233947E+00              + 1.0562898806171521533E+01j
fpm: -7.51926134267022E+00                   + 1.05628988061715E+01j
apm: -7.5192613426702233946e+0 (-2.298e-18%) + 1.0562898806171521533e+1 (1.989e-18%)j
```

10.8.3 Kelvin function $\ker(\nu, x)$

`ctx.kelvin_ker`(*nu*, *z*, *scaled=False*)

Returns the Kelvin function $\ker(\nu, x)$.

If *scaled* is *True*, then $\mathrm{bei}(\nu, x) \cdot \exp(|x|/\sqrt{2})$ is returned.

See also Wikipedia [1383], MathWorld [1024], NIST [444], Mpmath [710].

Here ν and x are, in general, complex numbers. However, if `ctx` is `math53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $n, x \in \mathbb{C}$ is accepted.

This function is traditionally defined for $\nu \in \mathbb{R}$ and real $x \geq 0$ as the real part of the Bessel K function of a rotated argument:

$$\ker(\nu, x) = \Re\left(e^{-\pi i/2} K_n\left(x e^{3\pi i/4}\right)\right).$$

In XlCalcNet this is generalized to complex ν and x following the definition given in Maple [369] as

$$\ker(\nu, x) = \frac{e^{-i\nu\pi/2} K_\nu(x(a+ia)) + e^{i\nu\pi/2} K_\nu(x(a-ia))}{2}, \quad \text{where } a = \tfrac{1}{2}\sqrt{2},$$

which is equivalent to the traditional definition for $\nu \in \mathbb{R}$ and real $x \geq 0$.

Note that this differs from Mpmath, which uses the conventions of Mathematica.

The Kelvin functions are all real valued for real x and positive ν.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinker(n, x); mx = mpm.kelvinker(n, x); gx = gmp.kelvinker(n, x)
>>> fx = fpm.kelvinker(n, x); ax = apm.kelvinker(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -6.702923330379869775199782194748322134382E-2
mpm: -6.702923330379869775199782194748322134382e-2
gmp: -6.702923330379869775199782194748322134382E-02
fpm: -6.70292333037987E-02
apm: -6.702923330379869775199782194748322134370e-2 (-8.669e-36%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.kelvinker(n, z); mz = mpm.kelvinker(n, z); gz = gmp.kelvinker(n, z)
>>> fz = fpm.kelvinker(n, z); az = apm.kelvinker(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 5.0016351579287135729E-1              + 2.7244626208949455251E-1j
mpm: 5.0016351579287135729e-1              + 2.7244626208949455251e-1j
gmp: 5.0016351579287135729E-01             + 2.7244626208949455251E-01j
fpm: 5.00163515792871E-01                  + 2.72446262089495E-01j
apm: 5.0016351579287135710e-1 (2.917e-15%) + 2.7244626208949455183e-1 (4.106e-15%)j
```

10.8.4 Kelvin function $\mathrm{kei}(\nu, x)$

`ctx.kelvin_kei`(*nu*, *z*, *scaled=False*)

Returns the Kelvin function $\mathrm{kei}(\nu, x)$.

If *scaled* is *True*, then $\mathrm{kei}(\nu, x) \cdot \exp(|x|/\sqrt{2})$ is returned.

See also Wikipedia [1382], MathWorld [1023], NIST [444], Mpmath [709].

Here ν and x are, in general, complex numbers. However, if `ctx` is `math53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $n, x \in \mathbb{C}$ is accepted.

This function is traditionally defined for $\nu \in \mathbb{R}$ and real $x \geq 0$ as the imaginary part of the Bessel K function of a rotated argument:

$$\mathrm{kei}(\nu, x) = \Im\left(e^{-\pi i/2} K_n\left(x e^{3\pi i/4}\right)\right).$$

In XlCalcNet this is generalized to complex ν and x following the definition given in Maple [369] as

$$\mathrm{kei}(\nu, x) = \frac{e^{-i\nu\pi/2} K_\nu(x(a+ia)) - e^{i\nu\pi/2} K_\nu(x(a-ia))}{2i}, \quad \text{where } a = \tfrac{1}{2}\sqrt{2}.$$

which is equivalent to the traditional definition for $\nu \in \mathbb{R}$ and real $x \geq 0$.

Note that this differs from Mpmath, which uses the conventions of Mathematica.

The Kelvin functions are all real valued for real x and positive ν.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinkei(n, x); mx = mpm.kelvinkei(n, x); gx = gmp.kelvinkei(n, x)
>>> fx = fpm.kelvinkei(n, x); ax = apm.kelvinkei(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -5.1121884045986781402466877539305017057 62E-2
mpm: -5.1121884045986781402466877539305017057 62e-2
gmp: -5.1121884045986781402466877539305017057 62E-02
fpm: -5.11218840459868E-02
apm: -5.1121884045986781402466877539305017057 53e-2 (-9.353e-36%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.kelvinkei(n, z); mz = mpm.kelvinkei(n, z); gz = gmp.kelvinkei(n, z)
>>> fz = fpm.kelvinkei(n, z); az = apm.kelvinkei(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 2.7516275915865256214E-1              - 4.9739028524739862760E-1j
mpm: 2.7516275915865256214e-1              - 4.9739028524739862760e-1j
gmp: 2.7516275915865256214E-01             - 4.9739028524739862760E-01j
fpm: 2.75162759158653E-01                  - 4.97390285247399E-01j
apm: 2.7516275915865859000e-1 (2.965e-11%) - 4.9739028524740313000e-1 (-2.154e-11%)j
```

10.8.5 First derivative of the Kelvin function $\mathrm{ber}(\nu, x)$, $\mathrm{ber}'(\nu, x)$

`ctx.kelvin_ber_prime`(*nu*, *z*, *scaled=False*)

Returns $\mathrm{ber}'(\nu, x)$, the first derivative (with respect to x) of the Kelvin function $\mathrm{ber}(\nu, x)$.

If *scaled* is *True*, then $\mathrm{ber}(\nu, x) \cdot \exp(-|x|/\sqrt{2})$ is returned.

See also Wikipedia [1381], MathWorld [1022], NIST [444], Mpmath [708].

Here ν and x are, in general, complex numbers. However, if `ctx` is `math53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $n, x \in \mathbb{C}$ is accepted.

The function is calculated as

$$\mathrm{ber}'(\nu, x) = \frac{a_1 J'_\nu(x \cdot a_1) + a_2 J'_\nu(x \cdot a_2)}{2},$$

where $a = \frac{1}{2}\sqrt{2}$, $a_1 = -a + ia$, $a_2 = -a - ia$, and $J'_\nu(x)$ is the first derivative (with respect to x) of the Bessel function $J_\nu(x)$.

The Kelvin functions are all real valued for real x and positive ν.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinber(n, x); mx = mpm.kelvinber(n, x); gx = gmp.kelvinber(n, x)
>>> fx = fpm.kelvinber(n, x); ax = apm.kelvinber(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -2.213802495986938888682464345899509922321E-1
mpm: -2.213802495986938888682464345899509922321e-1
gmp: -2.213802495986938888682464345899509922321E-01
fpm: -2.21380249598694E-01
apm: -2.213802495986938888682464345899509922332e-1 (-1.283e-37%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.kelvinber(n, z); mz = mpm.kelvinber(n, z); gz = gmp.kelvinber(n, z)
>>> fz = fpm.kelvinber(n, z); az = apm.kelvinber(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.033616210179297429 4E+1              + 7.766768913222125917 1E+0j
mpm: 1.033616210179297429 4e+1              + 7.766768913222125917 1e+0j
gmp: 1.033616210179297429 4E+01             + 7.766768913222125917 1E+00j
fpm: 1.03361621017930E+01                   + 7.76676891322213E+00j
apm: 1.033616210179297429 4e+1 (2.032e-18%) + 7.766768913222125917 1e+0 (2.225e-18%)j
```

10.8.6 First derivative of the Kelvin function $\mathrm{bei}(\nu, x)$, $\mathrm{bei}'(\nu, x)$

`ctx.kelvin_bei_prime`(*nu*, *z*, *scaled=False*)

Returns $\mathrm{bei}'(\nu, x)$, the first derivative (with respect to x) of the Kelvin function $\mathrm{bei}(\nu, x)$.

If *scaled* is *True*, then $\mathrm{bei}(\nu, x) \cdot \exp(-|x|/\sqrt{2})$ is returned.

See also Wikipedia [1381], MathWorld [1022], NIST [444], Mpmath [708].

Here ν and x are, in general, complex numbers. However, if `ctx` is `math53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $n, x \in \mathbb{C}$ is accepted.

The function is calculated as

$$\mathrm{bei}'(\nu, x) = \frac{a_1 J'_\nu(x \cdot a_1) - a_2 J'_\nu(x \cdot a_2)}{2i},$$

where $a = \frac{1}{2}\sqrt{2}$, $a_1 = -a + ia$, $a_2 = -a - ia$, and $J'_\nu(x)$ is the first derivative (with respect to x) of the Bessel function $J_\nu(x)$.

The Kelvin functions are all real valued for real x and positive ν.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinbei(n, x); mx = mpm.kelvinbei(n, x); gx = gmp.kelvinbei(n, x)
>>> fx = fpm.kelvinbei(n, x); ax = apm.kelvinbei(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: 1.937586785266042766896808122272260201255E+0
mpm: 1.937586785266042766896808122272260201255e+0
gmp: 1.937586785266042766896808122272260201255E+00
fpm: 1.93758678526604E+00
apm: 1.937586785266042766896808122272260201256e+0 (1.54e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.kelvinbei(n, z); mz = mpm.kelvinbei(n, z); gz = gmp.kelvinbei(n, z)
>>> fz = fpm.kelvinbei(n, z); az = apm.kelvinbei(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -7.5192613426702233947E+0               + 1.0562898806171521533E+1j
mpm: -7.5192613426702233947e+0               + 1.0562898806171521533e+1j
gmp: -7.5192613426702233947E+00              + 1.0562898806171521533E+01j
fpm: -7.51926134267022E+00                   + 1.05628988061715E+01j
apm: -7.5192613426702233946e+0 (-2.298e-18%) + 1.0562898806171521533e+1 (1.989e-18%)j
```

10.8.7 First derivative of the Kelvin function $\ker(\nu, x)$, $\ker'(\nu, x)$

`ctx.kelvin_ker_prime`(*nu*, *z*, *scaled=False*)

Returns $\ker'(\nu, x)$, the first derivative (with respect to x) of the Kelvin function $\ker(\nu, x)$.

If *scaled* is *True*, then $\mathrm{ber}(\nu, x) \cdot \exp(|x|/\sqrt{2})$ is returned.

See also Wikipedia [1383], MathWorld [1024], NIST [444], Mpmath [710].

Here ν and x are, in general, complex numbers. However, if `ctx` is `math53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $n, x \in \mathbb{C}$ is accepted.

The function is calculated as

$$\ker'(\nu, x) = \frac{e^{-\nu\pi i/2} a_1 K'_\nu(x \cdot a_1) + e^{\nu\pi i/2} a_2 K'_\nu(x \cdot a_2)}{2},$$

where $a = \frac{1}{2}\sqrt{2}$, $a_1 = -a + ia$, $a_2 = -a - ia$, and $K'_\nu(x)$ is the first derivative (with respect to x) of the Bessel function $K_\nu(x)$.

The Kelvin functions are all real valued for real x and positive ν.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinker(n, x); mx = mpm.kelvinker(n, x); gx = gmp.kelvinker(n, x)
>>> fx = fpm.kelvinker(n, x); ax = apm.kelvinker(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -6.702923330379869775199782194748322134382E-2
mpm: -6.702923330379869775199782194748322134382e-2
gmp: -6.702923330379869775199782194748322134382E-02
fpm: -6.70292333037987E-02
apm: -6.702923330379869775199782194748322134370e-2 (-8.669e-36%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.kelvinker(n, z); mz = mpm.kelvinker(n, z); gz = gmp.kelvinker(n, z)
>>> fz = fpm.kelvinker(n, z); az = apm.kelvinker(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 5.0016351579287135729E-1              + 2.7244626208949455251E-1j
mpm: 5.0016351579287135729e-1              + 2.7244626208949455251e-1j
gmp: 5.0016351579287135729E-01             + 2.7244626208949455251E-01j
fpm: 5.00163515792871E-01                  + 2.72446262089495E-01j
apm: 5.0016351579287135710e-1 (2.917e-15%) + 2.7244626208949455183e-1 (4.106e-15%)j
```

10.8.8 First derivative of the Kelvin function $\mathrm{kei}(\nu, x)$, $\mathrm{kei}'(\nu, x)$

`ctx.kelvin_kei_prime`(*nu*, *z*, *scaled=False*)

Returns $\mathrm{kei}'(\nu, x)$, the first derivative (with respect to x) of the Kelvin function $\mathrm{kei}(\nu, x)$.

If *scaled* is *True*, then $\mathrm{kei}(\nu, x) \cdot \exp(|x|/\sqrt{2})$ is returned.

See also Wikipedia [1382], MathWorld [1023], NIST [444], Mpmath [709].

Here ν and x are, in general, complex numbers. However, if `ctx` is `math53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $\nu \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $n, x \in \mathbb{C}$ is accepted.

The function is calculated as

$$\ker'(\nu, x) = \frac{e^{-\nu\pi i/2} a_1 K'_\nu(x \cdot a_1) - e^{\nu\pi i/2} a_2 K'_\nu(x \cdot a_2)}{2i},$$

where $a = \frac{1}{2}\sqrt{2}$, $a_1 = -a + ia$, $a_2 = -a - ia$, and $K'_\nu(x)$ is the first derivative (with respect to x) of the Bessel function $K_\nu(x)$.

The Kelvin functions are all real valued for real x and positive ν.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinkei(n, x); mx = mpm.kelvinkei(n, x); gx = gmp.kelvinkei(n, x)
>>> fx = fpm.kelvinkei(n, x); ax = apm.kelvinkei(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -5.112188404598678140246687753930501705762E-2
mpm: -5.112188404598678140246687753930501705762e-2
gmp: -5.112188404598678140246687753930501705762E-02
fpm: -5.11218840459868E-02
apm: -5.112188404598678140246687753930501705753e-2 (-9.353e-36%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.kelvinkei(n, z); mz = mpm.kelvinkei(n, z); gz = gmp.kelvinkei(n, z)
>>> fz = fpm.kelvinkei(n, z); az = apm.kelvinkei(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 2.7516275915865256214E-1              - 4.9739028524739862760E-1j
mpm: 2.7516275915865256214e-1              - 4.9739028524739862760e-1j
gmp: 2.7516275915865256214E-01             - 4.9739028524739862760E-01j
fpm: 2.75162759158653E-01                  - 4.97390285247399E-01j
apm: 2.7516275915865859000e-1 (2.965e-11%) - 4.9739028524740313000e-1 (-2.154e-11%)j
```

CHAPTER

ELEVEN

HYPERGEOMETRIC FUNCTION ${}_1F_1$ AND RELATED

Text explaining this chapter

The source code for the C# tests in Visual Studio is divided by real and complex modules:

The source code for the C# tests in Visual Studio using real modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B11a_RealHypergeometric_1F1.cs,

The source code for the C# tests in Visual Studio using complex modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B11b_CplxHypergeometric_1F1.cs,

11.1 Hypergeometric Functions ${}_1F_1$ (Kummer) and U (Tricomi)

11.1.1 Kummer's Confluent Hypergeometric Function ${}_1F_1(a, b; x)$

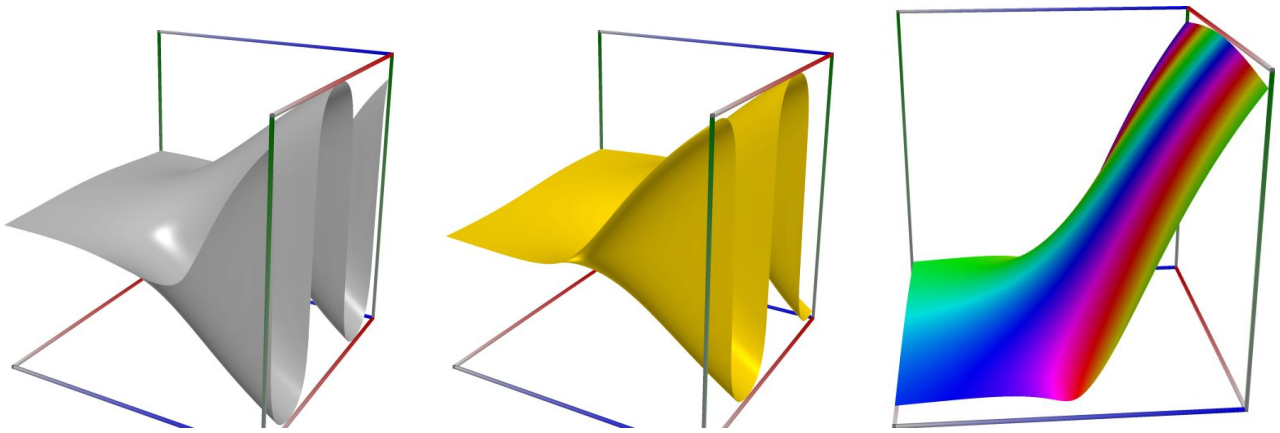
`ctx.hyperg_1f1`(a, b, x)

Returns ${}_1F_1(a, b; x)$, the confluent hypergeometric function of the first kind, where $b \neq 0, -1, -2, \ldots$.

See also Wikipedia [1255], MathWorld [899], NIST [193], BoostMath [103], Ehrhardt [297] (3.8.3), Abramowitz and Stegun. [2], Flint [757], Flint [750], Mpmath [591].

Here a, b and x are, in general, complex numbers. However, if `ctx` is `math53`, then $a, b \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $a, b \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $a, b, x \in \mathbb{C}$ is accepted.

The function is defined as ${}_1F_1(a, b; x) = \sum_{k=0}^{\infty} \frac{(a)_k}{(b)_k} \frac{z^k}{k!}$.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Hyperg1F1(4,5,0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Hyperg1F1(14,15,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = 11.0; b = 12.0; x = 3.0
>>> \mathrm{d}x = dec.hyp1f1(a, b, x); mx = mpm.hyp1f1(a, b, x); gx = gmp.hyp1f1(a, b, x)
>>> fx = fpm.hyp1f1(a, b, x); ax = apm.hyp1f1(a, b, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.6016380797644591021426991674784163061 25E+1
mpm:  1.6016380797644591021426991674784163061 25e+1
gmp:  1.6016380797644591021426991674784163061 25E+01
fpm:  1.60163807976446E+01
apm:  1.6016380797644591021426991674784163061 25e+1 (1.147e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '11.0 + 2.0j'; b = '12.0 + 3.0j'; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.hyp1f1(a, b, z); mz = mpm.hyp1f1(a, b, z); gz = gmp.hyp1f1(a, b, z)
>>> fz = fpm.hyp1f1(a, b, z); az = apm.hyp1f1(a, b, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -1.7071475892855456616E+1               - 6.5250702392280765696E+0j
mpm: -1.7071475892855456616e+1               - 6.5250702392280765696e+0j
gmp: -1.7071475892855456616E+01              - 6.5250702392280765696E+00j
fpm: -1.70714758928555E+01                   - 6.52507023922808E+00j
apm: -1.7071475892855456616e+1 (-1.588e-19%) - 6.5250702392280765696e+0 (-5.192e-20%)j
```

11.1.2 Regularized Kummer Confluent Hypergeometric Function, ${}_1\widetilde{F}_1(a,b;x)$

`ctx.hyperg_1f1r(`a, b, x`)`

Returns ${}_1\widetilde{F}_1(a,b;z)$, the regularized Kummer confluent hypergeometric function.

See also Wikipedia [1255], MathWorld [260], NIST [193], BoostMath [103], Ehrhardt [297] (3.8.4), Abramowitz and Stegun. [2], Flint [757], Flint [750], Mpmath [591].

Here a, b and x are, in general, complex numbers. However, if `ctx` is `math53`, then $a, b \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $a, b \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $a, b, x \in \mathbb{C}$ is accepted.

We have ${}_1\widetilde{F}_1(a,b;z) = \dfrac{1}{\Gamma(b)}{}_1F_1(a;b;z) = \mathbf{M}(a,b;x) = \dfrac{1}{\Gamma(b)}M(a;b;z)$, for $b \neq 0, -1, -2, \cdots$.

If $b = 0, -1, -2, \cdots = -n$, the corresponding limit ${}_1\widetilde{F}_1(a,b;x) = \dfrac{(a)_{n+1}}{(n+1)!}x^{n+1}{}_1F_1(a+n+1, n+2; x)$ is calculated.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Hyperg1F1r(4,5,0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Hyperg1F1r(14,15,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = 11.0; b = 12.0; x = 3.0
>>> \mathrm{d}x = dec.hyp1f1r(a, b, x); mx = mpm.hyp1f1r(a, b, x); gx = gmp.hyp1f1r(a, b, x)
>>> fx = fpm.hyp1f1r(a, b, x); ax = apm.hyp1f1r(a, b, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.012441076850997830844905321765312615552E-7
mpm:  4.012441076850997830844905321765312615552e-7
gmp:  4.012441076850997830844905321765312615552E-07
fpm:  4.01244107685100E-07
apm:  4.012441076850997830844905321765312615552e-7 (2.728e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '11.0 + 2.0j'; b = '12.0 + 3.0j'; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.hyp1f1r(a, b, z); mz = mpm.hyp1f1r(a, b, z); gz = gmp.hyp1f1r(a, b, z)
>>> fz = fpm.hyp1f1r(a, b, z); az = apm.hyp1f1r(a, b, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -5.0984734340262287981E-7               + 4.4090124699188539207E-7j
mpm: -5.0984734340262287981e-7               + 4.4090124699188539207e-7j
gmp: -5.0984734340262287981E-07              + 4.4090124699188539207E-07j
fpm: -5.09847343402623E-07                   + 4.40901246991885E-07j
apm: -5.0984734340262287982e-7 (-7.922e-20%) + 4.4090124699188539207e-7 (9.161e-20%)j
```

11.1.3 Tricomi's Confluent Hypergeometric Function, $U(a,b;x)$

`ctx.hyperg_u`(a, b, x)

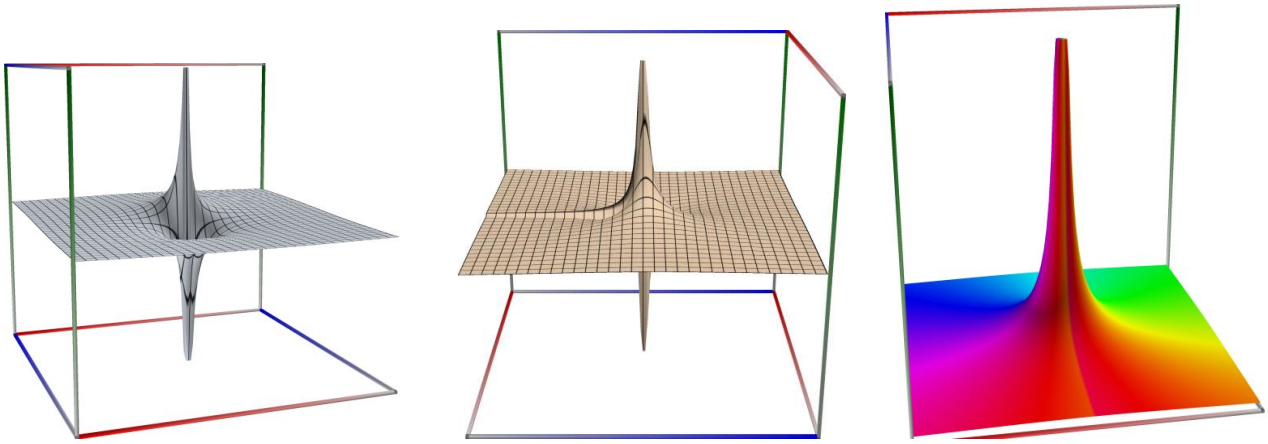
Returns Tricomi's confluent hypergeometric function of the second kind, $U(a,b;x)$.

See also Wikipedia [1255], MathWorld [900], NIST [193], Ehrhardt [297] (3.8.5), Flint [757], Flint [750], Mpmath [593].

Here a, b and x are, in general, complex numbers. However, if `ctx` is `math53`, then $a, b \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $a, b \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $a, b, x \in \mathbb{C}$ is accepted.

For all $z \neq 0$ and $b \notin \mathbb{Z}$ (but valid for all b as a limit) we have

$$U(a,b;x) = \frac{\Gamma(1-b)}{\Gamma(1+a-b)} M(a,b;c;z) + \frac{\Gamma(1-b)}{\Gamma(a)} x^{1-b} M(1+a-b, 2-b; x)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x+iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.HypergU(4,5.1,0.5)
ereal('5.2359877559829887307E-1')
>>> math53.HypergU(14,15.2,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = 11.0; b = 12.0; x = 3.0
>>> \mathrm{d}x = dec.hyperu(a, b, x); mx = mpm.hyperu(a, b, x); gx = gmp.hyperu(a, b, x)
>>> fx = fpm.hyperu(a, b, x); ax = apm.hyperu(a, b, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  5.64502926947676223701219890825133928319 4E-6
mpm:  5.64502926947676223701219890825133928319 4e-6
gmp:  5.64502926947676223701219890825133928319 4E-06
fpm:  5.64502926947676E-06
apm:  5.64502926947676223701219890825133928319 4e-6 (7.757e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '11.0 + 2.0j'; b = '12.0 + 3.0j'; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.hyperu(a, b, z); mz = mpm.hyperu(a, b, z); gz = gmp.hyperu(a, b, z)
>>> fz = fpm.hyperu(a, b, z); az = apm.hyperu(a, b, z)
```

(continues on next page)

(continued from previous page)

```
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 2.0656756869642262785E-7              + 5.4116585242285070055E-8j
mpm: 2.0656756869642262785e-7              + 5.4116585242285070055e-8j
gmp: 2.0656756869642262785E-07             + 5.4116585242285070055E-08j
fpm: 2.06567568696423E-07                  + 5.41165852422851E-08j
apm: 2.0656756869642262785e-7 (9.776e-20%) + 5.4116585242285070055e-8 (4.665e-20%)j
```

11.1.4 Generalized Laguerre polynomials, $L_n^{(a)}(x)$

`ctx.laguerre_l`(n, a, x)

where `ctx` is `math53` or `ctxflint`.

Note: ctx.laguerre(z, n, alpha)

Returns $L_n^{(a)}(x) = \binom{n+a}{n} M(-n, a+1, x) = \dfrac{\Gamma(n+a+1)}{\Gamma(n+1)\Gamma(a+1)} {}_1F_1(-n, a+1, x)$, the generalized Laguerre polynomials of degree $n \geq 0$ with parameter $a; x \geq 0$ and $a > -1$ are the standard ranges.

For integer degree $n \geq 0$ and integer order $m \geq 0$, we have, as a special case, the (associated) Laguerre polynomial $L_n^m(x) = L_n^{(m)}(x)$. If $m = 0$, it is just called Laguerre polynomial: $L_n(x) = L_n^0(x) = L_n^{(0)}(x)$

These polynomials are orthogonal on the interval $(0, \infty)$, with respect to the weight function $w(x) = e^{-x}x^a$. The following standard recurrence formulas are used:

$$\begin{aligned} L_0^{(a)}(x) &= 1 \\ L_1^{(a)}(x) &= -x + 1 + a \\ nL_n^{(a)}(x) &= (2n + a - 1 - x)L_{n-1}^{(a)}(x) - (n + a - 1)L_{n-2}^{(a)}(x). \end{aligned} \tag{11.1}$$

See also Wikipedia [1313], MathWorld [927], NIST [397], BoostMath [124], Ehrhardt [297] (3.7.10) and (3.7.12), Flint [759], Flint [755], Mpmath [634].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Laguerre(2, 3, 2, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Laguerre('6, 2, 0.51')
ereal('5.3518479027559984754E-1')
```

11.1.5 Hermite polynomial (physicist), $H_n(x)$

`ctx.hermite_h(n, z)`

Returns $H_n(z)$, the Hermite polynomial (physicist) of degree n.

See also Wikipedia [1307], MathWorld [924], NIST [397], BoostMath [121], Ehrhardt [297] (3.7.7) and (3.8.11.4), Ehrhardt [297] Flint [759], Flint [755], Mpmath [631].

Here n and x are, in general, complex numbers. However, if `ctx` is `math53`, then $n \in \mathbb{R}$ and $x \in \mathbb{R}$ is exspected. If `ctx` is `cmath53`, then $n \in \mathbb{R}$ and $x \in \mathbb{C}$ is exspected. If `ctx` is `ctxflint` then $n, x \in \mathbb{C}$ is accepted.

We have $H_n(z) = 2^n\sqrt{\pi}\left(\frac{1}{\Gamma\left(\frac{1-n}{2}\right)}\,{}_1F_1\left(-\frac{n}{2}, \frac{1}{2}, z^2\right) - \frac{2z}{\Gamma\left(-\frac{n}{2}\right)}\,{}_1F_1\left(\frac{1-n}{2}, \frac{3}{2}, z^2\right)\right)$, where $\frac{1}{\Gamma(n)}$ is evaluated calling the reciprocal gamma function.

For integer $n \geq 0$, the H_n are orthogonal on the interval $(-\infty, \infty)$, with respect to the weight function $w(x) = e^{-x^2}$. They can be computed with the standard recurrence formulas:

$$\begin{aligned} H_0(x) &= 1 \\ H_1(x) &= 2x \\ H_n(x) &= 2xH_{n-1}(x) - 2(n-1)H_{n-2}(x). \end{aligned} \tag{11.2}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.HermiteH(2, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.HermiteH(6, 0.51)
ereal('5.351847902755998475 4E-1')
```

11.1.6 Hermite polynomials (probabilist) $\mathrm{He}_n(x)$

`ctx.hermite_he(n, x)`

Returns $\mathrm{He}_n(x) = 2^{-n/2} H_n(x/\sqrt{2})$, the probabilist's Hermite polynomial of degree $n \geq 0$.

For integer n, the He_n are orthogonal on the interval $(-\infty, \infty)$, with respect to the weight function $w(x) = \exp(-x^2/2)$. They are computed with the standard recurrence formulas:

$$\begin{aligned} \mathrm{He}_0(x) &= 1 \\ \mathrm{He}_1(x) &= x \\ \mathrm{He}_n(x) &= x\,\mathrm{He}_{n-1}(x) - (n-1)\,\mathrm{He}_{n-2}(x). \end{aligned} \tag{11.3}$$

See also Wikipedia [1307], MathWorld [924], NIST [397], BoostMath [121], Ehrhardt [297] (3.7.8), Flint [759], Flint [755], Mpmath [631].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.HermiteHe(2, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.HermiteHe(6, 0.51)
ereal('5.351847902755998475 4E-1')
```

11.2 Incomplete gamma functions

11.2.1 Lower normalized incomplete gamma function, $P(a, x)$

`ctx.gamma_p`(a, x)

where `ctx` is `math53`, `ctxboost`, `ctxflint`.

Note: ctx.incGammaP(a, x)

Returns the lower normalized incomplete gamma function $P(a,x) = \frac{1}{\Gamma(a)} \int_0^x t^{a-1} e^{-t}\,\mathrm{d}t,$ for $a \geq 0$ and $x \geq 0$.

See also Wikipedia [1223], MathWorld [855], NIST [459], BoostMath [81], Ehrhardt [297] (3.5.2.1), Flint [741], Flint [731], Mpmath [550].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IncGammaP(3.1, 1.5)
ereal('5.2359877559829887307E-1')
>>> math53.IncGammaP(3.4, '1.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a= 10; x = 30
>>> \mathrm{d}x = dec.gamma_p(a, x); mx = mpm.gamma_p(a, x); gx = gmp.gamma_p(a, x)
>>> fx = fpm.gamma_p(a, x); ax = apm.gamma_p(a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.999928782491371844229083533391656597094E-1
mpm:  9.999928782491371844229083533391656597094e-1
gmp:  9.999928782491371844229083533391656597094E-01
fpm:  9.99992878249137E-01
apm:  9.999928782491371844229083533391656597094e-1 (5.74e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a= '10 + 3j'; z = '3 + 2j'
>>> \mathrm{d}z = dec.gamma_p(a, z); mz = mpm.gamma_p(a, z); gz = gmp.gamma_p(a, z)
>>> fz = fpm.gamma_p(a, z); az = apm.gamma_p(a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.34252104629843434451E-3              + 1.2911646547819672560E-3j
mpm: 1.34252104629843434451e-3              + 1.2911646547819672560e-3j
gmp: 1.342521046298434434451E-03            + 1.2911646547819672560E-03j
fpm: 1.34252104629843E-03                   + 1.29116465478197E-03j
apm: 1.34252104629843434451e-3 (3.697e-19%) + 1.2911646547819672560e-3 (3.203e-19%)j
```

11.2.2 Upper normalized incomplete gamma functions , $Q(a, x)$

`ctx.gamma_q`(a, x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Note: ctx.incGammaQ(a, x)

Returns the upper normalized incomplete gamma function $Q(a,x) = \dfrac{1}{\Gamma(a)} \displaystyle\int_x^\infty t^{a-1} e^{-t}\,\mathrm{d}t,$ for $a \geq 0$ and $x \geq 0$.

See also Wikipedia [1223], MathWorld [855], NIST [459], BoostMath [81], Ehrhardt [297] (3.5.2.1), Flint [741], Flint [731], Mpmath [550].

If *regularized* is 0, computes the upper incomplete gamma function $\Gamma(s,z)$.

If *regularized* is 1, computes the regularized upper incomplete gamma function $Q(s,z) = \Gamma(s,z)/\Gamma(s)$.

If *regularized* is 2, computes the generalized exponential integral $z^{-s}\Gamma(s,z) = E_{1-s}(z)$.

The different methods respectively implement the formulas

$$\Gamma(s,z) = e^{-z} U(1-s, 1-s, z)$$

$$\Gamma(s,z) = \Gamma(s) - \frac{z^s}{s} {}_1F_1(s, s+1, -z)$$

$$\Gamma(s,z) = \Gamma(s) - \frac{z^s e^{-z}}{s} {}_1F_1(1, s+1, z)$$

$$\Gamma(s,z) = \frac{(-1)^n}{n!}(\psi(n+1) - \log(z)) + \frac{(-1)^n}{(n+1)!} z\, {}_2F_2(1,1,2,2+n,-z) - z^{-n} \sum_{k=0}^{n-1} \frac{(-z)^k}{(k-n)k!}, \quad n = -s \in \mathbb{Z}_{\geq 0}$$

and an automatic algorithm choice. The automatic version also handles other special input such as $z = 0$ and $s = 1, 2, 3$. The *singular* version evaluates the finite sum directly and therefore assumes that s is not too large.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IncGammaQ(3.1, 1.5)
ereal('5.235987755982988730 7E-1')
>>> math53.IncGammaQ(3.4, '1.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a= 10; x = 30
>>> \mathrm{d}x = dec.gamma_q(a, x); mx = mpm.gamma_q(a, x); gx = gmp.gamma_q(a, x)
>>> fx = fpm.gamma_q(a, x); ax = apm.gamma_q(a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  7.12175086281557709164666083434029059 3535E-6
mpm:  7.12175086281557709164666083434029059 3535e-6
gmp:  7.12175086281557709164666083434029059 3535E-06
fpm:  7.12175086281558E-06
apm:  7.12175086281557709164666083434029059 5390e-6 (4.704e-35%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a= '10 + 3j'; z = '3 + 2j'
>>> \mathrm{d}z = dec.gamma_q(a, z); mz = mpm.gamma_q(a, z); gz = gmp.gamma_q(a, z)
>>> fz = fpm.gamma_q(a, z); az = apm.gamma_q(a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 9.9865747895370156565E-1              - 1.2911646547819672560E-3j
```

(continues on next page)

(continued from previous page)

```
mpm: 9.9865747895370156565e-1              - 1.2911646547819672560e-3j
gmp: 9.9865747895370156565E-01             - 1.2911646547819672560E-03j
fpm: 9.98657478953702E-01                  - 1.29116465478197E-03j
apm: 9.9865747895370156566e-1 (4.241e-20%) - 1.2911646547819672560e-3 (-3.844e-19%)j
```

11.2.3 Lower non-normalized incomplete gamma function, $\gamma(a, x)$

`ctx.gamma_lower`(a, x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Note: ctx.incgammaL(a, x), ctxboost.TgammaLower(a, x)

Returns the real lower non-normalized incomplete gamma function $\gamma(a, x) = \int_0^x t^{a-1} e^{-t}\,\mathrm{d}t,$ for $a \geq 0$ and $x \geq 0$.

See also Wikipedia [1223], MathWorld [848], NIST [459], BoostMath [81], Ehrhardt [297] (3.5.2.2), Flint [741], Flint [731], Mpmath [550].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IncgammaL(3.1, 1.5)
ereal('5.2359877559829887307E-1')
>>> math53.IncgammaL(3.4, '1.51')
ereal('5.3518479027559984754E-1')
```

11.2.4 Upper non-normalized incomplete gamma function, $\Gamma(a, x)$

`ctx.gamma_upper`(a, x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Note: ctx.incGammaU(a, x)

Returns the real upper non-normalized incomplete gamma function $\Gamma(a,x) = \int_x^\infty t^{a-1}e^{-t}\,\mathrm{d}t,$ for $a \geq 0$ and $x \geq 0$.

See also Wikipedia [1223], MathWorld [848], NIST [459], BoostMath [81], Ehrhardt [297] (3.5.2.2), Flint [741], Flint [731], Mpmath [550].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IncGammaU(3.1, 1.5)
ereal('5.235987755982988730 7E-1')
>>> math53.IncGammaU(3.4, '1.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a= 10; x = 30
>>> \mathrm{d}x = dec.gamma_upper(a, x); mx = mpm.gamma_upper(a, x); gx = gmp.gamma_upper(a, x)
>>> fx = fpm.gamma_upper(a, x); ax = apm.gamma_upper(a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.5843409530985166150167402835654046505 82E+0
mpm:  2.5843409530985166150167402835654046505 82e+0
gmp:  2.5843409530985166150167402835654046505 82E+00
fpm:  2.58434095309852E+00
apm:  2.5843409530985166150167402835654046511 98e+0 (4.411e-35%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a= '10 + 3j'; z = '3 + 2j'
>>> \mathrm{d}z = dec.gamma_upper(a, z); mz = mpm.gamma_upper(a, z); gz = gmp.gamma_upper(a, z)
>>> fz = fpm.gamma_upper(a, z); az = apm.gamma_upper(a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.97505052549980 15902E+5              + 1.12845709229070 13455E+5j
mpm: 1.97505052549980 15902e+5              + 1.12845709229070 13455e+5j
gmp: 1.97505052549980 15902E+05             + 1.12845709229070 13455E+05j
fpm: 1.97505052549980E+05                   + 1.12845709229070E+05j
apm: 1.97505052549980 15902e+5 (5.621e-20%) + 1.12845709229070 13455e+5 (9.838e-20%)j
```

11.2.5 Derivative of the incomplete gamma function

`ctx.gamma_p_prime`(a, z)

Returns the partial derivative with respect to x of the incomplete gamma function $P(a, x)$:

The partial derivative with respect to x of the incomplete gamma function $P(a, x)$ is defined as:

$$\frac{\partial}{\partial x} P(a, x) = \frac{e^{-x} x^{a-1}}{\Gamma(a)}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a= 10; x = 30
>>> \mathrm{d}x = dec.gamma_derivative(a, x); mx = mpm.gamma_derivative(a, x); gx = gmp.gamma_derivative(a, x)
>>> fx = fpm.gamma_derivative(a, x); ax = apm.gamma_derivative(a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  5.07567495854500542186282863902O146738895E-6
mpm:  5.07567495854500542186282863902O146738895e-6
gmp:  5.07567495854500542186282863902O146738895E-06
fpm:  5.07567495854500E-06
apm:  5.07567495854500542186282863902O146738895e-6 (8.628e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a= '10 + 3j'; z = '3 + 2j'
>>> \mathrm{d}z = dec.gamma_derivative(a, z); mz = mpm.gamma_derivative(a, z); gz = gmp.gamma_derivative(a, z)
>>> fz = fpm.gamma_derivative(a, z); az = apm.gamma_derivative(a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 3.6415328970281122640E-3              + 1.2712795456433263145E-3j
mpm: 3.6415328970281122640e-3              + 1.2712795456433263145e-3j
gmp: 3.6415328970281122640E-03             + 1.2712795456433263145E-03j
fpm: 3.64153289702811E-03                  + 1.27127954564333E-03j
apm: 3.6415328970281122640e-3 (2.272e-19%) + 1.2712795456433263145e-3 (5.856e-19%)j
```

11.2.6 Inverse of the real lower normalised incomplete gamma function, $P^{-1}(a, q)$

`ctx.gamma_p_inv`(a, q)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Note: ctx.incGammaPInv(a, p)

Returns $P^{-1}(a, p)$, the functional inverse of the real lower normalized incomplete gamma function, i.e. the function calculates x with $P(a, x) = p$ where $a > 0$ and $0 < p < 1$.

See also BoostMath [80], Wikipedia [1223], MathWorld [258], NIST [459], Ehrhardt [297] (3.5.2.4).

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; a = '10.4'; prob = '0.7'
>>> \mathrm{d}x = dec.gamma_p_inv(a, prob); mx = mpm.gamma_p_inv(a, prob)
>>> ix = ipm.gamma_p_inv(a, prob); fx = fpm.gamma_p_inv(a, prob)
>>> gx = gmp.gamma_p_inv(a, prob); ax = apm.gamma_p_inv(a, prob)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  1.18206531273240015823054804964431258208 3E+1
mpm:  1.18206531273240015823054804964431258208 3e+1
ipm:  1.18206531273240015823054804964431258208 3e+1 (7.769e-40%)
fpm:  1.18206531273240E+01
gmp:  1.18206531273240015823054804964431258208 3E+01
apm:  1.18206531273240015823054804964431258208 3e+1 (7.769e-40%)
```

11.2.7 Inverse of the real upper normalised incomplete gamma function, $Q^{-1}(a, q)$

`ctx.gamma_q_inv`(a, q)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Note: ctx.incGammaQInv(a, q)

Returns $Q^{-1}(a, q)$, the functional inverse of the real upper normalized incomplete gamma function, i.e. the function calculates x with $Q(a, x) = q$ where $a > 0$ and $0 < q < 1$.

See also BoostMath [80], Wikipedia [1223], MathWorld [258], NIST [459], Ehrhardt [297] (3.5.2.4).

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; a = '10.4'; prob = '0.7'
>>> \mathrm{d}x = dec.gamma_q_inv(a, prob); mx = mpm.gamma_q_inv(a, prob)
>>> ix = ipm.gamma_q_inv(a, prob); fx = fpm.gamma_q_inv(a, prob)
>>> gx = gmp.gamma_q_inv(a, prob); ax = apm.gamma_q_inv(a, prob)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  8.4994072827546379443001464562670863769 57E+0
mpm:  8.4994072827546379443001464562670863769 57e+0
ipm:  8.4994072827546379443001464562670863769 57e+0 (1.08e-39%)
fpm:  8.49940728275464E+00
gmp:  8.4994072827546379443001464562670863769 57E+00
apm:  8.4994072827546379443001464562670863769 57e+0 (1.08e-39%)
```

11.2.8 Inverse (on parameter a) of the real lower normalised incomplete gamma function

`ctx.gamma_p_inva`(x, q)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Note: ctx.incGammaPInva(x, p)

Returns the functional inverse (on parameter a) of the lower normalized incomplete gamma function $P(a, x)$, i.e. the function calculates a with $P(a, x) = p$ where $x > 0$ and $0 < p < 1$.

See also BoostMath [80], Wikipedia [1223], NIST [459]

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10.4'; prob = '0.7'
>>> \mathrm{d}x = dec.gamma_p_inva(x, prob); mx = mpm.gamma_p_inva(x, prob)
>>> ix = ipm.gamma_p_inva(x, prob); fx = fpm.gamma_p_inva(x, prob)
>>> gx = gmp.gamma_p_inva(x, prob); ax = apm.gamma_p_inva(x, prob)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  9.0912237806574900243957402146331986854 11E+0
mpm:  9.0912237806574900243957402146331986854 11e+0
ipm:  9.0912237806574900243957402146331986854 11e+0 (1.01e-39%)
fpm:  9.09122378065749E+00
gmp:  9.0912237806574900243957402146331986854 11E+00
apm:  9.0912237806574900243957402146331986854 11e+0 (1.01e-39%)
```

11.2.9 Inverse (on parameter a) of the real upper normalised incomplete gamma function

`ctx.gamma_q_inva`(x, q)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Note: ctx.incGammaQInva(x, p)

Returns the functional inverse (on parameter a) of the upper normalized incomplete gamma function $Q(a, x)$, i.e. the function calculates a with $Q(a, x) = q$ where $x > 0$ and $0 < q < 1$.

See also BoostMath [80], Wikipedia [1223], NIST [459]

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10.4'; prob = '0.7'
>>> \mathrm{d}x = dec.gamma_q_inva(x, prob); mx = mpm.gamma_q_inva(x, prob)
>>> ix = ipm.gamma_q_inva(x, prob); fx = fpm.gamma_q_inva(x, prob)
>>> gx = gmp.gamma_q_inva(x, prob); ax = apm.gamma_q_inva(x, prob)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  1.2463747582846862236028997886519722948 25E+1
mpm:  1.2463747582846862236028997886519722948 25e+1
ipm:  1.2463747582846862236028997886519722948 25e+1 (7.368e-40%)
fpm:  1.24637475828469E+01
gmp:  1.2463747582846862016481281898450106382 37E+01
apm:  1.2463747582846862236028997886519722948 25e+1 (7.368e-40%)
```

11.3 Coulomb, Whittaker and parabolic cylinder function

11.3.1 Regular Coulomb wave function $F_\ell(\eta, x)$

`ctx.coulomb_f`(*l*, *eta*, *x*)

where `ctx` is `math53`` or `ctxflint`.

Returns the Coulomb wave function F. See also Wikipedia [1368], MathWorld [1029], NIST [782], Ehrhardt [297] (3.1.10.4), Mpmath [714].

Calculates the regular Coulomb wave function

$$F_\ell(\eta, x) = \frac{1}{2i}\left(H_\ell^{(+)}(\eta, x) - H_\ell^{(-)}(\eta, x)\right),$$

where $H_\ell^{(+)}(\eta, x)$ and $H_\ell^{(-)}(\eta, x)$ are irregular Coulomb wave functions.

Coulomb wave functions are solutions of the Coulomb wave equation

$$y'' + \left(1 - \frac{2\eta}{z} - \frac{\ell(\ell+1)}{z^2}\right) y = 0$$

which is the radial Schrödinger equation for a charged particle in a Coulomb potential $1/z$, where ℓ is the orbital angular momentum and η is the Sommerfeld parameter. The standard solutions are named $F_\ell(\eta, z)$ (regular at the origin $z = 0$) and $G_\ell(\eta, z)$ (irregular at the origin). The irregular solutions $H_\ell^{\pm}(\eta, z) = G_\ell(\eta, z) \pm iF_\ell(\eta, z)$ are also used. The redundant functions $H^{\pm}$ are provided explicitly since taking the linear combination of F and G suffers from cancellation in parts of the complex plane.

Coulomb wave functions are special cases of confluent hypergeometric functions. The normalization constants and connection formulas are discussed in Dzieciol *et al.* [296], Gaspard [342], Michel [414] and chapter 33 in NIST [782]. In this implementation, we define the analytic continuations of all the functions so that the branch cut with respect to z is placed on the negative real axis.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CoulombF(3, 0.5, 2.25)
ereal('5.2359877559829887307E-1')
>>> math53.CoulombF(3, 0.5, 8.25)
ereal('5.351847902755998475 4E-1')
```

11.3.2 Irregular Coulomb wave function $G_\ell(\eta, z)$

`ctx.coulomb_g`(*L*, *eta*, *x*)

where `ctx` is `math53`` or `ctxflint`.

Returns the irregular Coulomb wave function. See also Wikipedia [1368], MathWorld [1029], NIST [782], Ehrhardt [297] (3.1.10.5), Mpmath [715].

Calculates the irregular Coulomb wave function

$$G_\ell(\eta, x) = \frac{1}{2}\left(H_\ell^{(+)}(\eta, x) - H_\ell^{(-)}(\eta, x)\right),$$

where $H_\ell^{(+)}(\eta, x)$ and $H_\ell^{(-)}(\eta, x)$ are irregular Coulomb wave functions.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; l = '10'; eta = '3.0'; x = '5.0'
>>> \mathrm{d}x = dec.coulombg(l, eta, x); mx = mpm.coulombg(l, eta, x); gx = gmp.coulombg(l, eta, x)
>>> fx = fpm.coulombg(l, eta, x); ax = apm.coulombg(l, eta, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.961542114057365010741557537693681681810E+3
mpm:  4.961542114057365010741557537693681681810e+3
gmp:  4.961542114057365010741557537693681681810E+03
fpm:  4.96154211405737E+03
apm:  4.961542114057365010741557537693681681810e+3 (3.791e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; l = '10'; eta = '3 + 4j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.coulombg(l, eta, z); mz = mpm.coulombg(l, eta, z); gz = gmp.coulombg(l, eta, z)
>>> fz = fpm.coulombg(l, eta, z); az = apm.coulombg(l, eta, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.2323201237533547519E+3              - 1.026967516056281371 5E+2j
mpm: 1.2323201237533547519e+3              - 1.026967516056281371 5e+2j
gmp: 1.2323201237533547519E+03             - 1.026967516056281371 5E+02j
fpm: 1.23232012375335E+03                  - 1.02696751605628E+02j
apm: 1.2323201237533547519e+3 (4.223e-19%) - 1.026967516056281371 5e+2 (-3.537e-18%)j
```

11.3.3 Irregular Coulomb wave function $H_\ell^{(+)}(\eta, x)$

`ctx.`**`coulomb_hplus`**(*L*, *eta*, *x*)

where `ctx` is `math53`` or `ctxflint`.

Returns the irregular Coulomb wave function. See also Wikipedia [1368], MathWorld [1029], NIST [782], Ehrhardt [297] (3.1.10.5), Mpmath [715].

Calculates the irregular Coulomb wave function

$$H_\ell^{(+)}(\eta, x) = -2i(-2)^\ell e^{\pi\eta/2} e^{+i\sigma_\ell} x^{\ell+1} e^{+ix} U(\ell + 1 + i\eta, 2l + 2, -2ix),$$

where $\sigma_\ell = \arg\Gamma(\ell + 1 + i\eta)$ is called the Coulomb phase shift.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; l = '10'; eta = '3.0'; x = '5.0'
>>> \mathrm{d}x = dec.coulombg(l, eta, x); mx = mpm.coulombg(l, eta, x); gx = gmp.coulombg(l, eta, x)
>>> fx = fpm.coulombg(l, eta, x); ax = apm.coulombg(l, eta, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.961542114057365010741557537693681681810E+3
mpm:  4.961542114057365010741557537693681681810e+3
gmp:  4.961542114057365010741557537693681681810E+03
fpm:  4.96154211405737E+03
apm:  4.961542114057365010741557537693681681810e+3 (3.791e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; l = '10'; eta = '3 + 4j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.coulombg(l, eta, z); mz = mpm.coulombg(l, eta, z); gz = gmp.coulombg(l, eta, z)
>>> fz = fpm.coulombg(l, eta, z); az = apm.coulombg(l, eta, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.2323201237533547519E+3              - 1.026967516056281371 5E+2j
mpm: 1.2323201237533547519e+3              - 1.026967516056281371 5e+2j
gmp: 1.2323201237533547519E+03             - 1.026967516056281371 5E+02j
fpm: 1.23232012375335E+03                  - 1.02696751605628E+02j
apm: 1.2323201237533547519e+3 (4.223e-19%) - 1.026967516056281371 5e+2 (-3.537e-18%)j
```

11.3.4 Irregular Coulomb wave function $H_\ell^{(-)}(\eta, x)$

`ctx.coulomb_hminus(`*L*, *eta*, *x*`)`

where `ctx` is `math53`` or `ctxflint`.

Returns the irregular Coulomb wave function. See also Wikipedia [1368], MathWorld [1029], NIST [782], Ehrhardt [297] (3.1.10.5), Mpmath [715].

Calculates the irregular Coulomb wave function

$$H_\ell^{(-)}(\eta, x) = +2i(-2)^\ell e^{\pi\eta/2} e^{+i\sigma_\ell} x^{\ell+1} e^{+ix} U(\ell + 1 + i\eta, 2l + 2, -2ix),$$

where $\sigma_\ell = \arg\Gamma(\ell + 1 + i\eta)$ is called the Coulomb phase shift.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; l = '10'; eta = '3.0'; x = '5.0'
>>> \mathrm{d}x = dec.coulombg(l, eta, x); mx = mpm.coulombg(l, eta, x); gx = gmp.coulombg(l, eta, x)
>>> fx = fpm.coulombg(l, eta, x); ax = apm.coulombg(l, eta, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.961542114057365010741557537693681681810E+3
mpm:  4.961542114057365010741557537693681681810e+3
gmp:  4.961542114057365010741557537693681681810E+03
fpm:  4.96154211405737E+03
apm:  4.961542114057365010741557537693681681810e+3 (3.791e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; l = '10'; eta = '3 + 4j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.coulombg(l, eta, z); mz = mpm.coulombg(l, eta, z); gz = gmp.coulombg(l, eta, z)
>>> fz = fpm.coulombg(l, eta, z); az = apm.coulombg(l, eta, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.2323201237533547519E+3              - 1.026967516056281371 5E+2j
mpm: 1.2323201237533547519e+3              - 1.026967516056281371 5e+2j
gmp: 1.2323201237533547519E+03             - 1.026967516056281371 5E+02j
fpm: 1.23232012375335E+03                  - 1.02696751605628E+02j
apm: 1.2323201237533547519e+3 (4.223e-19%) - 1.026967516056281371 5e+2 (-3.537e-18%)j
```

11.3.5 Whittaker function $M_{\kappa,\mu}(x)$

`ctx.whittaker_m(k, m, x)`

Returns $M_{\kappa,\mu}(x) = e^{-\frac{1}{2}x} x^{\frac{1}{2}+\mu} M(\frac{1}{2}+\mu-\kappa, 1+2\mu, x)$, the Whittaker function M.

See also MathWorld [1038], Wikipedia [1395], Ehrhardt [297] (3.8.9), NIST [191], Mpmath [724].

$$M(k,m,z) = e^{-\frac{1}{2}z} z^{\frac{1}{2}+m} \, {}_1F_1(\tfrac{1}{2}+m-k, 1+2m, z)$$

Evaluates the Whittaker function $M(k,m,z)$, which gives a solution to the Whittaker differential equation

$$\frac{d^2f}{\mathrm{d}z^2} + \left(-\frac{1}{4} + \frac{k}{z} + \frac{(\frac{1}{4}-m^2)}{z^2}\right) f = 0.$$

A second solution is given by *whitw()*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.WhittakerM(5.1,0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.WhittakerM(15.2,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; k = '10'; m = '3.0'; x = '5.0'
>>> \mathrm{d}x = dec.whitm(k, m, x); mx = mpm.whitm(k, m, x); gx = gmp.whitm(k, m, x)
>>> fx = fpm.whitm(k, m, x); ax = apm.whitm(k, m, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  5.037724223377349504794983027900485241922E-2
mpm:  5.037724223377349504794983027900485241922e-2
mpm:  5.037724223377349504794983027900485241922e-2
fpm:  5.03772422337738E-02
apm:  5.037724223377349504794983027900485241947e-2 (1.976e-36%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; k = '10'; m = '3 + 4j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.whitm(k, m, z); mz = mpm.whitm(k, m, z); gz = gmp.whitm(k, m, z)
>>> fz = fpm.whitm(k, m, z); az = apm.whitm(k, m, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 4.9746800935982291618E-1              - 5.5703088408623297578E-1j
mpm: 4.9746800935982291618e-1              - 5.5703088408623297578e-1j
mpm: 4.9746800935982291618e-1              - 5.5703088408623297578e-1j
fpm: 4.97468009359823E-01                  - 5.57030884086233E-01j
apm: 4.9746800935982291618e-1 (8.939e-19%) - 5.5703088408623297578e-1 (-6.082e-19%)j
```

11.3.6 Whittaker function $W_{\kappa,\mu}(x)$

`ctx.whittaker_w(`k, m, x`)`

Returns $W_{\kappa,\mu}(x) = e^{-\frac{1}{2}x} x^{\frac{1}{2}+\mu} U(\frac{1}{2}+\mu-\kappa, 1+2\mu, x)$, the Whittaker function W.

See also MathWorld [1039], Wikipedia [1395], Ehrhardt [297] (3.8.10), NIST [191], Mpmath [725].

$$W(k,m,z) = e^{-\frac{1}{2}z} z^{\frac{1}{2}+m} U(\tfrac{1}{2}+m-k, 1+2m, z).$$

Evaluates the Whittaker function $W(k,m,z)$, which gives a second solution to the Whittaker differential equation.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.WhittakerW(5.1,0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.WhittakerW(15.2,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; k = '10'; m = '3.0'; x = '5.0'
>>> \mathrm{d}x = dec.whitw(k, m, x); mx = mpm.whitw(k, m, x); gx = gmp.whitw(k, m, x)
>>> fx = fpm.whitw(k, m, x); ax = apm.whitw(k, m, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -9.20686924410858987028753002919421737427 0E+5
mpm:  -9.20686924410858987028753002919421737427 0e+5
mpm:  -9.20686924410858987028753002919421737427 0e+5
fpm:  -9.20686924410859E+05
apm:  -9.20686924410858987028753002919421737427 0e+5 (-7.191e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; k = '10'; m = '3 + 4j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.whitw(k, m, z); mz = mpm.whitw(k, m, z); gz = gmp.whitw(k, m, z)
>>> fz = fpm.whitw(k, m, z); az = apm.whitw(k, m, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 7.57236386665901821 21E+7              - 1.66734719054390616 12E+8j
mpm: 7.57236386665901821 21e+7              - 1.66734719054390616 12e+8j
mpm: 7.57236386665901821 21e+7              - 1.66734719054390616 12e+8j
fpm: 7.57236386665902E+07                   - 1.66734719054391E+08j
apm: 7.57236386665901821 20e+7 (6.756e-19%) - 1.66734719054390616 12e+8 (-4.091e-19%)j
```

11.3.7 Parabolic cylinder function $D_\nu(x)$

`ctx.cylinder_d(nu, x)`

Returns Whittaker's parabolic cylinder function $D_n(z) = U(-n - 1/2, z)$ (see *pcfu()*).

See also: MathWorld [285], Wikipedia [1388], NIST [780], Mpmath [718], Ehrhardt [297] (3.8.11.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CylinderD(5.1,0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.CylinderD(15.2,0.5)
ereal('5.351847902755998475 4E-1')
>>> math53.CylinderD(15.2,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '10.1'; x = '5.0'
>>> \mathrm{d}x = dec.pcfd(n, x); mx = mpm.pcfd(n, x); gx = gmp.pcfd(n, x)
>>> fx = fpm.pcfd(n, x); ax = apm.pcfd(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.9896375768626179568207551164298692593 51E+2
mpm:  2.9896375768626179568207551164298692593 51e+2
mpm:  2.9896375768626179568207551164298692593 52e+2
fpm:  2.98963757686262E+02
apm:  2.9896375768626179568207551164298692593 52e+2 (1.914e-36%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10.1'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.pcfd(n, z); mz = mpm.pcfd(n, z); gz = gmp.pcfd(n, z)
>>> fz = fpm.pcfd(n, z); az = apm.pcfd(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 5.7176371555791286919E+5              - 4.367100634652110 4926E+5j
mpm: 5.7176371555791286919e+5              - 4.367100634652110 4926e+5j
mpm: 5.7176371555791286919e+5              - 4.367100634652110 4926e+5j
fpm: 5.71763715557912E+05                  - 4.36710063465211E+05j
apm: 5.7176371555791286919e+5 (1.577e-17%) - 4.367100634652110 4926e+5 (-1.825e-17%)j
```

11.3.8 Parabolic cylinder function $U(a,x)$

ctx.cylinder_u(a,x)

Returns the parabolic cylinder function $U(a,x)$.

See also: MathWorld [1035], Wikipedia [1388], Ehrhardt [297] (3.8.11.2), NIST [780], Mpmath [719].

The function is defined, for arbitrary z, as

$$U\left(a,z\right)=U\left(a,0\right)u_1(a,z)+U'\left(a,0\right)u_2(a,z),$$

$$u_1(a,z)=e^{-\frac{1}{4}z^2}\,{}_1F_1\left(\tfrac{1}{2}a+\tfrac{1}{4},\tfrac{1}{2},\tfrac{1}{2}z^2\right)=e^{\frac{1}{4}z^2}\,{}_1F_1\left(-\tfrac{1}{2}a+\tfrac{1}{4},\tfrac{1}{2},-\tfrac{1}{2}z^2\right),$$

$$u_2(a,z)=ze^{-\frac{1}{4}z^2}\,{}_1F_1\left(\tfrac{1}{2}a+\tfrac{3}{4},\tfrac{3}{2},\tfrac{1}{2}z^2\right)=ze^{\frac{1}{4}z^2}\,{}_1F_1\left(-\tfrac{1}{2}a+\tfrac{3}{4},\tfrac{3}{2},-\tfrac{1}{2}z^2\right).$$

$$U\left(a,0\right)=\frac{\sqrt{\pi}}{2^{\frac{1}{2}a+\frac{1}{4}}\Gamma\left(\frac{3}{4}+\frac{1}{2}a\right)},\quad\text{and } U'\left(a,0\right)=-\frac{\sqrt{\pi}}{2^{\frac{1}{2}a-\frac{1}{4}}\Gamma\left(\frac{1}{4}+\frac{1}{2}a\right)},$$

For $\Re(z)>0$, the function may be in terms of the confluent U-function (see *hyperu()*) by

$$U(a,z)=2^{-\frac{1}{4}-\frac{a}{2}}e^{-\frac{1}{4}z^2}U\left(\frac{a}{2}+\frac{1}{4},\frac{1}{2},\frac{1}{2}z^2\right)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CylinderU(5.1,0.5)
ereal('5.2359877559829887307E-1')
>>> math53.CylinderU(15.2,0.5)
ereal('5.351847902755998475 4E-1')
>>> math53.CylinderU(15.2,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = '10'; x = '5.0'
>>> \mathrm{d}x = dec.pcfu(a, x); mx = mpm.pcfu(a, x); gx = gmp.pcfu(a, x)
>>> fx = fpm.pcfu(a, x); ax = apm.pcfu(a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.5496568962879376487011318502323980324 54E-11
mpm:  1.5496568962879376487011318502323980324 54e-11
mpm:  1.5496568962879376487011318502323980324 54e-11
fpm:  1.54965689628794E-11
apm:  1.5496568962879376487011318502323980324 54e-11 (1.078e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.pcfu(a, z); mz = mpm.pcfu(a, z); gz = gmp.pcfu(a, z)
>>> fz = fpm.pcfu(a, z); az = apm.pcfu(a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 5.62444689095875 87524688E-11                + 3.21133245467725 87524688E-11j
mpm: 5.62444689095875 87524688e-11                + 3.21133245467725 87524688e-11j
mpm: 5.62444689095875 87524688e-11                + 3.21133245467725 87524688e-11j
fpm: 5.62444689095875E-11                         + 3.21133245467725E-11j
apm: 5.62444689095875 87524688e-11 (4.383e-20%) + 3.21133245467725 87524688e-11 (7.677e-20%)j
```

11.3.9 Parabolic cylinder function $V(a,x)$

`ctx.cylinder_v`(a,x)

Returns the parabolic cylinder function $V(a,x)$, which can be represented in terms of *pcfu()* as

$$V(a,z) = \frac{\Gamma(a+\frac{1}{2})(U(a,-z) - \sin(\pi a)U(a,z)}{\pi}.$$

See also: MathWorld [1035], Wikipedia [1388], Ehrhardt [297] (3.8.11.3), NIST [780], Mpmath [720].

In DAMath, this is restricted to $2a \in \mathbb{Z}$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CylinderV(5.1,0.5)
ereal('5.2359877559829887307E-1')
>>> math53.CylinderV(15.2,0.5)
ereal('5.3518479027559984754E-1')
>>> math53.CylinderV(15.2,0.5)
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = '10'; x = '5.0'
>>> \mathrm{d}x = dec.pcfv(a, x); mx = mpm.pcfv(a, x); gx = gmp.pcfv(a, x)
>>> fx = fpm.pcfv(a, x); ax = apm.pcfv(a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  6.3862155065142163776920010554351110 83031E+9
mpm:  6.3862155065142163776920010554351110 83031e+9
mpm:  6.3862155065142163776920010554351110 83031e+9
fpm:  6.38621550651422E+09
apm:  6.3862155065142163776920010554351110 83031e+9 (5.404e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.pcfv(a, z); mz = mpm.pcfv(a, z); gz = gmp.pcfv(a, z)
>>> fz = fpm.pcfv(a, z); az = apm.pcfv(a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.1155789211452870283E+9              - 1.0701503668604253775E+9j
mpm: 1.1155789211452870283e+9              - 1.0701503668604253775e+9j
mpm: 1.1155789211452870283e+9              - 1.0701503668604253775e+9j
fpm: 1.11557892114529E+09                  - 1.07015036686043E+09j
apm: 1.1155789211452870283e+9 (8.968e-19%) - 1.0701503668604253775e+9 (-8.499e-19%)j
```

11.3.10 Parabolic cylinder function $W(a,x)$

`ctx.cylinder_w(a, z)`

Returns the parabolic cylinder function $W(a,z)$. See also Wikipedia [1388], MathWorld [1035], NIST [780], Mpmath [721].

The function is defined as

$$W\left(a,x\right)=W\left(a,0\right)w_1(a,x)+W'\left(a,0\right)w_2(a,x).$$

$$W\left(a,0\right)=2^{-\frac{3}{4}}\left|\frac{\Gamma\left(\frac{1}{4}+\frac{1}{2}ia\right)}{\Gamma\left(\frac{3}{4}+\frac{1}{2}ia\right)}\right|^{\frac{1}{2}},\quad W'\left(a,0\right)=-2^{-\frac{1}{4}}\left|\frac{\Gamma\left(\frac{3}{4}+\frac{1}{2}ia\right)}{\Gamma\left(\frac{1}{4}+\frac{1}{2}ia\right)}\right|^{\frac{1}{2}}.$$

$$w_1(a,x)=e^{-\frac{1}{4}ix^2}\,{}_1F_1\left(\tfrac{1}{4}-\tfrac{1}{2}ia,\tfrac{1}{2},\tfrac{1}{2}ix^2\right),\quad w_2(a,x)=xe^{-\frac{1}{4}ix^2}\,{}_1F_1\left(\tfrac{3}{4}-\tfrac{1}{2}ia,\tfrac{3}{2},\tfrac{1}{2}ix^2\right).$$

The function can also be represented as

$$W\left(a,x\right)=\sqrt{k/2}\,e^{\frac{1}{4}\pi a}\left[e^{i\rho}U\left(ia,xe^{-\pi i/4}\right)+e^{-i\rho}U\left(-ia,xe^{\pi i/4}\right)\right].$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = '10'; x = '5.0'
>>> \mathrm{d}x = dec.pcfw(a, x); mx = mpm.pcfw(a, x); gx = gmp.pcfw(a, x)
>>> fx = fpm.pcfw(a, x); ax = apm.pcfw(a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.662395750711469900575353811000346598048E-7
mpm:  4.662395750711469900575353811000346598048e-7
mpm:  4.662395750711469900575353811000346598048e-7
fpm:  4.66239575071147E-07
apm:  4.662395750711469899862962927504081215235e-7 (5.87e-40%)
```

An example with complex input (this is an example where all digits returned by mpmath are wrong because of insufficient precision):

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.pcfw(a, z); mz = mpm.pcfw(a, z); gz = gmp.pcfw(a, z)
>>> fz = fpm.pcfw(a, z); az = apm.pcfw(a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 8.80667539337001241O7E-7              - 6.312850075134565494 9E-7j
mpm: 8.80667539337001241O7e-7              - 6.312850075134565494 9e-7j
mpm: 8.80667539337001241O7e-7              - 6.312850075134565494 9e-7j
fpm: 8.80667539337001E-07                  - 6.31285007513457E-07j
apm: 6.9582517470815592536e-8 (7.256e-20%) - 4.9878527483184454344e-8 (-5.061e-20%)j
```

11.4 Error function and related functions

11.4.1 Error function, $\mathrm{erf}(x)$

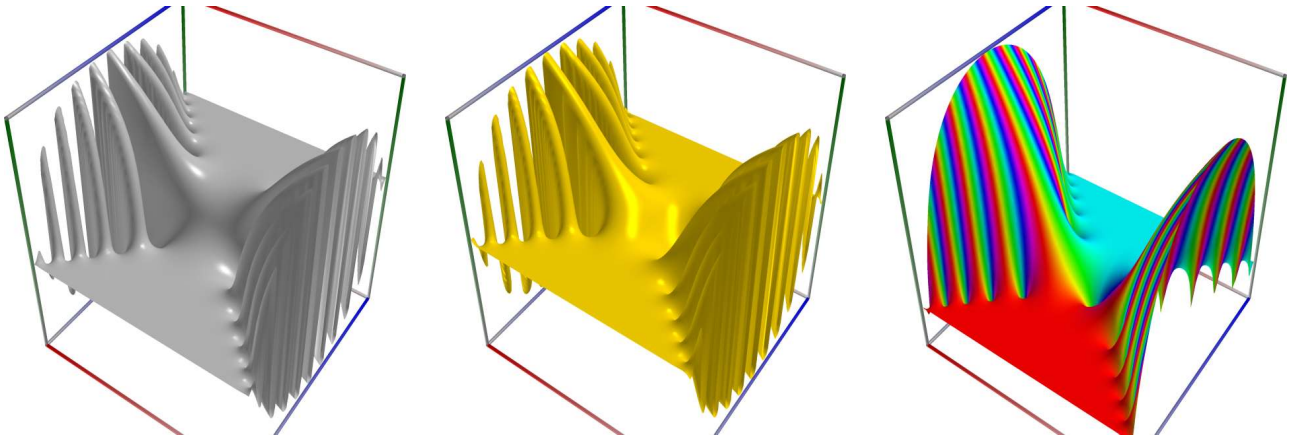
`ctx.erf(x)`

where `ctx` is `math53`, `mathc53`, `ctxboost`, `ctxflint`.

Returns the error function $\mathrm{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x \exp(-t^2) \mathrm{d}t$. See also BoostMath [94], BoostMath [74], Wikipedia [1234], MathWorld [843], NIST [775], Ehrhardt [297] (4.2.32), Flint [737], Flint [727], Mpmath [545].

This function returns the value of the error function defined by

$$\mathrm{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-z^2} \mathrm{d}t = \frac{2z}{\sqrt{\pi}} {}_1F_1\left(\tfrac{1}{2}, \tfrac{3}{2}, -z^2\right).$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Dawson(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Dawson('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.erf(x); mx = mpm.erf(x); gx = gmp.erf(x)
>>> fx = fpm.erf(x); ax = apm.erf(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.9997790950300141455862722387041767962 01E-1
mpm:  9.9997790950300141455862722387041767962 01e-1
gmp:  9.9997790950300141455862722387041767962 01E-01
fpm:  9.99977909503001E-01
apm:  9.9997790950300141455862722387041767962 02e-1 (5.74e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
```

(continues on next page)

(continued from previous page)

```
>>> \mathrm{d}z = dec.erf(z); mz = mpm.erf(z); gz = gmp.erf(z)
>>> fz = fpm.erf(z); az = apm.erf(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -1.2018699139507944410E+2               - 2.77503372936239023902498E+1j
mpm: -1.2018699139507944410e+2               - 2.77503372936239023902498e+1j
gmp: -1.2018699139507944410E+02              - 2.77503372936239023902498E+01j
fpm: -1.20186991395079E+02                   - 2.77503372936239E+01j
apm: -1.2018699139507944410e+2 (-9.021e-20%) - 2.77503372936239023902498e+1 (-4.884e-20%)j
```

11.4.2 Complementary error function, $\operatorname{erfc}(x)$

`ctx.erfc(x)`

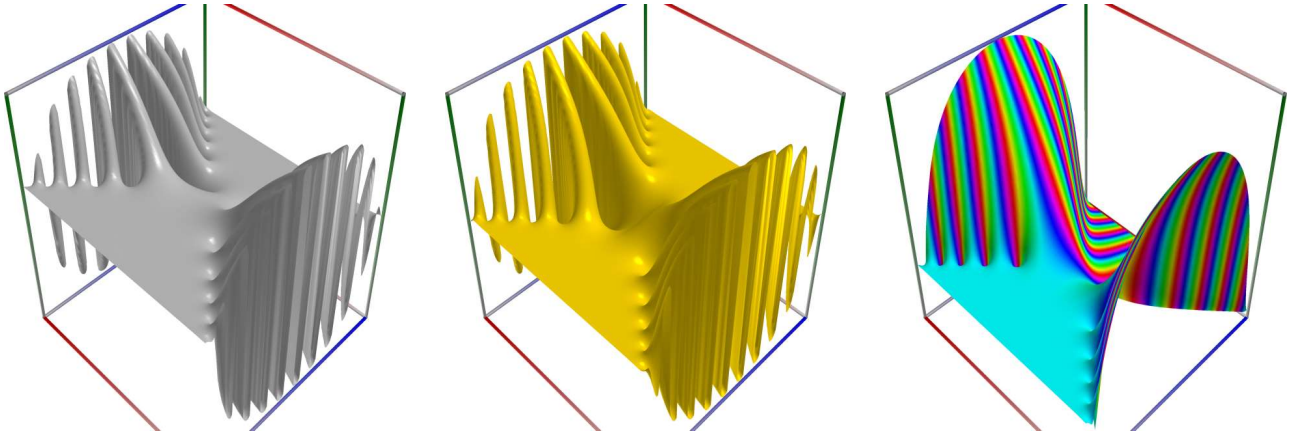
where `ctx` is `math53`, `mathc53`, `ctxboost`, `ctxflint`.

Returns the complementary error function $\operatorname{erfc}(x) = 1 - \operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_x^\infty \exp(-t^2)\, dt$.

See also BoostMath [74], Wikipedia [1231], MathWorld [844], NIST [775], MathWorld [977], Ehrhardt [297] (3.3.5), Ehrhardt [297] (4.2.33), Mpmath [544].

Returns the value of the complementary error function defined by

$$\operatorname{erfc}(x) = 1 - \operatorname{erfc}(x) = \frac{2}{\sqrt{\pi}} \int_x^\infty e^{-x^2}\, dt,$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Erfc(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Erfc('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.erfc(x); mx = mpm.erfc(x); gx = gmp.erfc(x)
>>> fx = fpm.erfc(x); ax = apm.erfc(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.2090496998585441372776129582320379847 71E-5
mpm:  2.2090496998585441372776129582320379847 71e-5
gmp:  2.2090496998585441372776129582320379847 71E-05
fpm:  2.20904969985854E-05
apm:  2.2090496998585441372776129582320379847 71e-5 (1.586e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.erfc(z); mz = mpm.erfc(z); gz = gmp.erfc(z)
```

(continues on next page)

(continued from previous page)

```
>>> fz = fpm.erfc(z); az = apm.erfc(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 1.2118699139507944410E+2              + 2.7750337293623902498E+1j
mpm: 1.2118699139507944410e+2              + 2.7750337293623902498e+1j
gmp: 1.2118699139507944410E+02             + 2.7750337293623902498E+01j
fpm: 1.211869991395079E+02                 + 2.77503372936239E+01j
apm: 1.2118699139507944410e+2 (8.947e-20%) + 2.7750337293623902498e+1 (4.884e-20%)j
```

11.4.3 Inverse of the real error function, $\mathrm{erf}^{-1}(x)$

`ctx.erf_inv(q)`

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the inverse of the real error function, satisfying $\mathrm{erf}(\mathrm{erfinv}(x)) = \mathrm{erfinv}(\mathrm{erf}(x)) = x$. See also BoostMath [73], Wikipedia [1234], MathWorld [849], NIST [775], Flint [737], Mpmath [551].

This function is defined only for $-1 \le x \le 1$.

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; q = '0.007'
>>> \mathrm{d}x = dec.erfinv(q); mx = mpm.erfinv(q); ix = ipm.erfinv(q)
>>> fx = fpm.erfinv(q); gx = gmp.erfinv(q); ax = apm.erfinv(q)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  6.203668061000835402417689089205287381720E-3
mpm:  6.203668061000835531560785950441696784170e-3
ipm:  6.203668061000835531560785950441696784170e-3 (7.228e-40%)
fpm:  6.20366806100084E-03
gmp:  6.203668061000835315388357571464439388365E-03
apm:  6.203668061000835531560785950441696784170e-3 (7.228e-40%)
```

11.4.4 Inverse of the real complementory error function, $\mathrm{erfc}^{-1}(x)$

`ctx.`**`erfc_inv`**`(`q`)`

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the inverse of the real complementory error function, satisfying $\mathrm{erfc}(\mathrm{erfcinv}(x)) = \mathrm{erfcinv}(\mathrm{erfc}(x)) = x$. See also BoostMath [73], Wikipedia [1231], MathWorld [850], NIST [775], Flint [737].

This function is defined only for $-1 \le x \le 1$.

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; q = '0.007'
>>> \mathrm{d}x = dec.erfcinv(q); mx = mpm.erfcinv(q); ix = ipm.erfcinv(q)
>>> fx = fpm.erfcinv(q); gx = gmp.erfcinv(q); ax = apm.erfcinv(q)
>>> mpm.show([gx, fx, ax])
dec:  1.906956864670945611335498085438891420125E+0
mpm:  1.906956864670945606433652487714446631717e+0
ipm:  1.906956864670945606433652487714446631717e+0 (6.02e-40%)
fpm:  1.90695686467095E+00
gmp:  1.906956864670945611335498085438891420125E+00
apm:  1.906956864670945606433652487714446631717e+0 (6.02e-40%)
```

11.4.5 Standard normal density function $\phi(x)$

`ctx.ndens`(x)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Note: also ctx.erfZ(x), ctx.ndens(x), mathc53.Ndens(x)

Returns the Gaussian density function $\phi(z) = \frac{1}{\sqrt{2\pi}} \exp(-z^2)$. See also: Ehrhardt [297] (3.3.12.3) and (3.9.28).

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.2'; mu = '0'; sd = '1';
>>> \mathrm{d}x = dec.normal_pdf(x, mu, sd); mx = mpm.normal_pdf(x, mu, sd)
>>> ix = ipm.normal_pdf(x, mu, sd); fx = fpm.normal_pdf(x, mu, sd)
>>> gx = gmp.normal_pdf(x, mu, sd); ax = apm.normal_pdf(x, mu, sd)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  1.941860549832129404120911390335162607571E-1
mpm:  1.941860549832129404120911390335162607571e-1
ipm:  1.941860549832129404120911390335162607571e-1 (5.173e-39%)
fpm:  1.94186054983213E-01
gmp:  1.941860549832129404120911390335162607571E-01
ipm:  1.941860549832129404120911390335162607571e-1 (5.173e-39%)
```

11.4.6 Standard normal cumulative distribution function $\Phi(x)$

`ctx.ndis(x)`

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Note: also ctx.erfP(x), ctx.ndis(x), mathc53.Ndis(x)

Returns the integral $\Phi(z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{z} \exp(-t^2)\,\mathrm{d}t = \frac{1}{2}\mathrm{erfc}\left(-\frac{z}{\sqrt{2}}\right)$.

See also BoostMath [74], Wikipedia [1231], MathWorld [844], NIST [775], MathWorld [977], Ehrhardt [297] (3.3.12.1) and (3.9.28).

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '1.2'; mu = '0'; sd = '1';
>>> \mathrm{d}x = dec.normal_cdf(x, mu, sd); mx = mpm.normal_cdf(x, mu, sd)
>>> ix = ipm.normal_cdf(x, mu, sd); fx = fpm.normal_cdf(x, mu, sd)
>>> gx = gmp.normal_cdf(x, mu, sd); ax = apm.normal_cdf(x, mu, sd)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  8.84930329778291731977779793043364851324 5E-1
mpm:  8.84930329778291731977779793043364851324 6e-1
ipm:  8.84930329778291731977779793043364851324 5e-1 (6.486e-40%)
fpm:  8.84930329778292E-01
gmp:  8.84930329778291731977779793043364851324 6E-01
ipm:  8.84930329778291731977779793043364851324 6e-1 (1.297e-39%)
```

11.4.7 Standard normal quantile function $\Phi^{-1}(q)$

`math53.ndisx(q)`

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the standard normal quantile function $\Phi^{-1}(q)$, defined as $\Phi^{-1}(q) = -\sqrt{2}\,\mathrm{erfc_inv}(2q)$.

See also BoostMath [74], Wikipedia [1231], MathWorld [844], NIST [775], MathWorld [977], Ehrhardt [297] (3.3.12.1) and (3.9.28).

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; q = '0.2'; mu = '0'; sd = '1';
>>> \mathrm{d}x = dec.normal_qtf(q, mu, sd); mx = mpm.normal_qtf(q, mu, sd)
>>> ix = ipm.normal_qtf(q, mu, sd); fx = fpm.normal_qtf(q, mu, sd)
>>> gx = gmp.normal_qtf(q, mu, sd); ax = apm.normal_qtf(q, mu, sd)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  -8.41621233572914205178706121363248100626 5E-1
mpm:  -8.41621233572914205178706121363248100626 3e-1
ipm:  -8.41621233572914205178706121363248100626 3e-1 (-2.728e-39%)
fpm:  -8.41621233572914E-01
gmp:  -8.41621233572914205178706121363248100626 3E-01
ipm:  -8.41621233572914205178706121363248100626 3e-1 (-2.728e-39%)
```

11.4.8 Imaginary error function, $\operatorname{erfi}(x)$

`ctx.erfi`(x)

where `ctx` is `math53`, `mathc53`, `ctxflint`.

Returns the imaginary error function $\operatorname{erfi}(x) = \frac{2}{\sqrt{\pi}} \int_0^x \exp(t^2)\,\mathrm{d}t = \frac{1}{i}\operatorname{erf}(ix) = \frac{2}{\sqrt{\pi}} e^{x^2} \operatorname{dawson}(x)$.

See also Wikipedia [1239], MathWorld [845], Ehrhardt [297] (3.3.8), Flint [737], Flint [727], Mpmath [546].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Erfi(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Erfi('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.erfi(x); mx = mpm.erfi(x); gx = gmp.erfi(x)
>>> fx = fpm.erfi(x); ax = apm.erfi(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.6299946226015656510616479520762741627 79E+3
mpm:  1.6299946226015656510616479520762741627 79e+3
gmp:  1.6299946226015656510616479520762741627 79E+03
fpm:  1.62999462260157E+03
apm:  1.6299946226015656510616479520762741627 79e+3 (7.212e-40%)
```

An example with complex input:

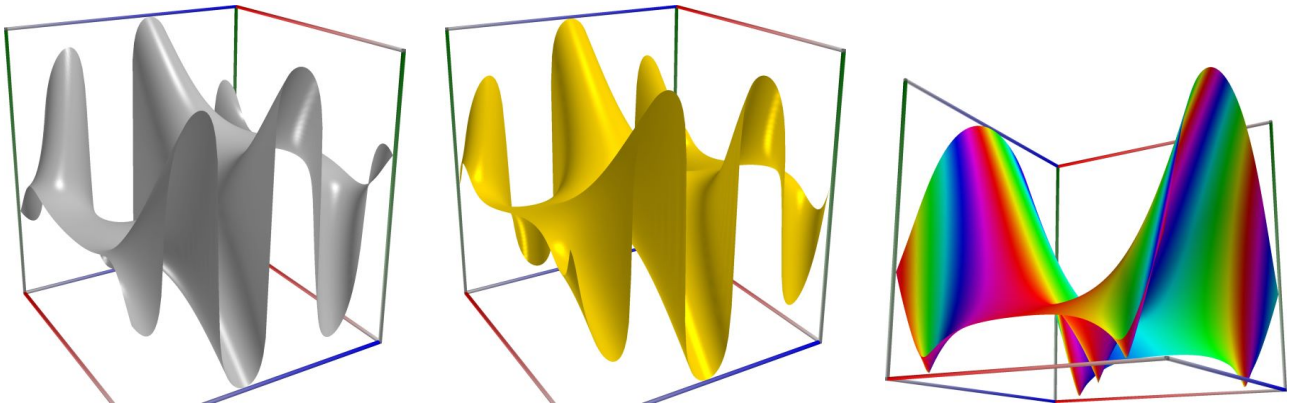
```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.erfi(z); mz = mpm.erfi(z); gz = gmp.erfi(z)
>>> fz = fpm.erfi(z); az = apm.erfi(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -4.9720260544966036460E-5              + 9.9991066178539168236E-1j
mpm: -4.9720260544966036460e-5              + 9.9991066178539168236e-1j
gmp: -4.9720260544966036460E-05             + 9.9991066178539168236E-01j
fpm: -4.97202605449660E-05                  + 9.99910661785392E-01j
apm: -4.9720260544966036460e-5 (-1.56e-18%) + 9.9991066178539168236e-1 (4.236e-20%)j
```

11.4.9 Dawson integral, $F(x)$

`ctx.dawson(`x`)`

Returns the Dawson integral $F(z) = e^{-z^2} \int_0^z e^{t^2}\,\mathrm{d}t = \frac{\sqrt{\pi}}{2} e^{-z^2}\,\mathrm{erfi}(z) = \frac{i\sqrt{\pi}}{2}\left[e^{-z^2} - w(z)\right]$.

See also Wikipedia [1339], MathWorld [973], Ehrhardt [297] (3.3.1), NIST [434].



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Dawson(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Dawson('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.dawson(x); mx = mpm.dawson(x); gx = gmp.dawson(x)
>>> fx = fpm.dawson(x); ax = apm.dawson(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.782710306105582873425994922405126302292E-1
mpm:  1.782710306105582873425994922405126302292e-1
gmp:  1.782710306105582873425994922405126302292E-01
fpm:  1.78271030610558E-01
apm:  1.782710306105582873425994922405126302293e-1 (1.409e-37%)
```

An example with complex input:

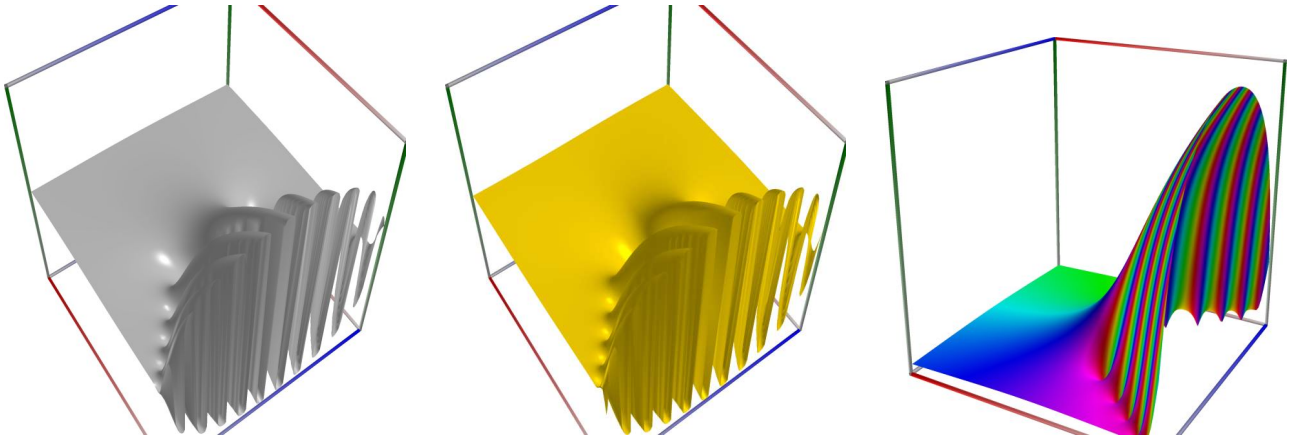
```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.dawson(z); mz = mpm.dawson(z); gz = gmp.dawson(z)
>>> fz = fpm.dawson(z); az = apm.dawson(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -8.8004253885450449691E+2               + 4.1216449595391869731E+2j
mpm: -8.8004253885450449691e+2               + 4.1216449595391869731e+2j
gmp: -8.8004253885450449691E+02              + 4.1216449595391869731E+02j
fpm: -8.80042538854505E+02                   + 4.12164495953919E+02j
apm: -8.8004253885450449493e+2 (-4.535e-16%) + 4.1216449595391869602e+2 (4.615e-16%)j
```

11.4.10 Faddeeva function, $w(z)$

`ctx.faddeeva(z)`

Returns the Faddeeva function function $w(z) := e^{-z^2}\operatorname{erfc}(-iz) = \operatorname{erfcx}(-iz) = e^{-z^2}\left(1 + \frac{2i}{\sqrt{\pi}}\int_0^z e^{t^2}\,\mathrm{d}t\right)$.

See also Wikipedia [1322], NIST [435].



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Faddeeva(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Faddeeva('0.1')
ecplx('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; x = 3.0
>>> \mathrm{d}x = dec.faddeeva(x); mx = mpm.faddeeva(x); gx = gmp.faddeeva(x)
>>> fx = fpm.faddeeva(x); ax = apm.faddeeva(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 1.2340980408667954950E-4              + 2.0115731703760038666E-1j
mpm: 1.2340980408667954950e-4              + 2.0115731703760038666e-1j
gmp: 1.2340980408667954950E-04             + 2.0115731703760038666E-01j
fpm: 1.23409804086680E-04                  + 2.01157317037600E-01j
apm: 1.2340980408667954950e-4 (8.378e-20%) + 2.0115731703760038666e-1 (3.158e-19%)j
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.faddeeva(z); mz = mpm.faddeeva(z); gz = gmp.faddeeva(z)
>>> fz = fpm.faddeeva(z); az = apm.faddeeva(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 9.0933904194765342461E-2              + 6.5592330527914277737E-2j
mpm: 9.0933904194765342461e-2              + 6.5592330527914277737e-2j
gmp: 9.0933904194765342461E-02             + 6.5592330527914277737E-02j
fpm: 9.09339041947653E-02                  + 6.55923305279143E-02j
apm: 9.0933904194765342460e-2 (2.911e-19%) + 6.5592330527914277737e-2 (4.035e-19%)j
```

11.4.11 Fresnel sine integral, $S(x)$

`ctx.fresnel_s(z)`

where `ctx` is `math53`, `mathc53`, `ctxflint`.

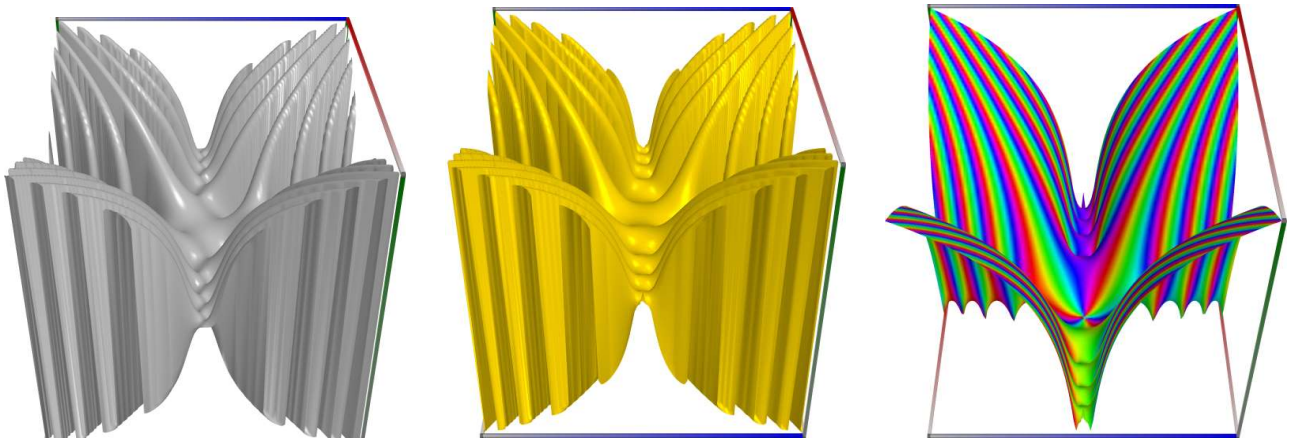
Returns the Fresnel sine integral $S(x) = \int_0^x \sin\left(\frac{\pi t^2}{2}\right) \mathrm{d}t$.

See also Wikipedia [1323], MathWorld [981], NIST [777],, Ehrhardt [297] (3.3.14), Flint [737], Flint [727], Mpmath [655].

The complex Fresnel sine integral can be expressed using the error function as follows:

$$S(z) = \sqrt{\frac{\pi}{2}} \frac{1+i}{4} \left[\operatorname{erf}\left(\frac{1+i}{\sqrt{2}} z\right) - i \operatorname{erf}\left(\frac{1-i}{\sqrt{2}} z\right) \right].$$

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.fresnel.html#scipy.special.fresnel



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.FresnelS(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.FresnelS('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.fresnels(x); mx = mpm.fresnels(x); gx = gmp.fresnels(x)
>>> fx = fpm.fresnels(x); ax = apm.fresnels(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.96312998967375036097612265299112103856 5E-1
mpm:  4.96312998967375036097612265299112103856 5e-1
gmp:  4.96312998967375036097612265299112103856 5E-01
fpm:  4.96312998967375E-01
apm:  4.96312998967375036097612265299112103856 5e-1 (9.252e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
```

(continues on next page)

(continued from previous page)

```
>>> \mathrm{d}z = dec.fresnels(z); mz = mpm.fresnels(z); gz = gmp.fresnels(z)
>>> fz = fpm.fresnels(z); az = apm.fresnels(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 6.0975480744693149379E+14             + 4.5370167667746719242E+14j
mpm: 6.0975480744693149379e+14             + 4.5370167667746719242e+14j
gmp: 6.0975480744693149379E+14             + 4.5370167667746719242E+14j
fpm: 6.09754807446932E+14                  + 4.53701676677467E+14j
apm: 6.0975480744693149379e+14 (7.82e-20%) + 4.5370167667746719242e+14 (5.255e-20%)j
```

11.4.12 Fresnel cosine integral, $C(x)$

`ctx.fresnel_c(z)`

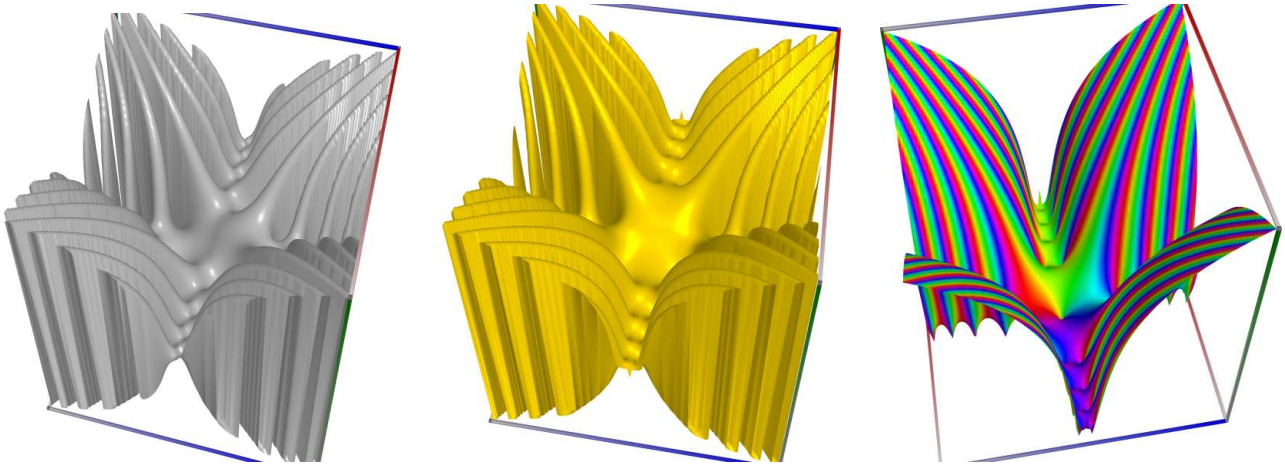
where `ctx` is `math53`, `mathc53`, `ctxflint`.

Returns the Fresnel cosine integral $C(x) = \int_0^x \cos\left(\frac{\pi t^2}{2}\right) \mathrm{d}t$.

See also Wikipedia [1323], MathWorld [980], NIST [777], Ehrhardt [297] (3.3.14), Flint [737], Flint [727], Mpmath [654].

The complex Fresnel cosine integral can be expressed using the error function as follows:

$$C(z) = \sqrt{\frac{\pi}{2}} \frac{1-i}{4} \left[\operatorname{erf}\left(\frac{1+i}{\sqrt{2}} z\right) + i \operatorname{erf}\left(\frac{1-i}{\sqrt{2}} z\right) \right].$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.FresnelC(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.FresnelC('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.fresnelc(x); mx = mpm.fresnelc(x); gx = gmp.fresnelc(x)
>>> fx = fpm.fresnelc(x); ax = apm.fresnelc(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  6.0572078929768562955616107428715469714 52E-1
mpm:  6.0572078929768562955616107428715469714 52e-1
gmp:  6.0572078929768562955616107428715469714 52E-01
fpm:  6.05720789297686E-01
apm:  6.0572078929768562955616107428715469714 52e-1 (9.476e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.fresnelc(z); mz = mpm.fresnelc(z); gz = gmp.fresnelc(z)
```

(continues on next page)

(continued from previous page)

```
>>> fz = fpm.fresnelc(z); az = apm.fresnelc(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 4.5370167667746769242E+14              - 6.0975480744693099379E+14j
mpm: 4.5370167667746769242e+14              - 6.0975480744693099379e+14j
gmp: 4.5370167667746769242E+14              - 6.0975480744693099379E+14j
fpm: 4.53701676677468E+14                   - 6.09754807446931E+14j
apm: 4.5370167667746769242e+14 (5.255e-20%) - 6.0975480744693099379e+14 (-7.82e-20%)j
```

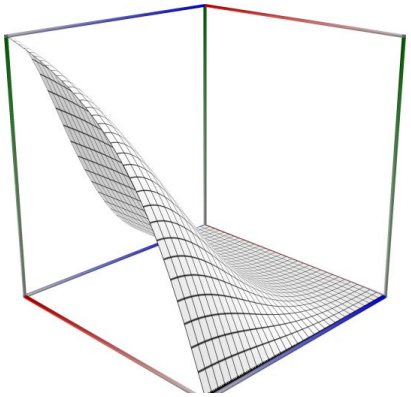
11.4.13 Owen's T function, $T(h, a)$

`ctx.owen_t`(h, a)

where `ctx` is `math53` or `ctxflint`.

Returns Owen's T function $T(h,a) = \dfrac{1}{2\pi}\displaystyle\int_0^a f(x)\mathrm{d}x = \dfrac{a}{4\pi}\int_{-1}^1 f(ax)\mathrm{d}x, \quad f(x) = \dfrac{e^{-\frac{1}{2}h^2(1+x^2)}}{1+x^2}, \quad (-\infty < h, a < +\infty).$

See also Owen [453], and Patefield and Tand [463], Wikipedia [1353], MathWorld [1013], Ehrhardt [297] (3.3.17).



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.OwenT(2.5, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.OwenT(2.5, '0.51')
ereal('5.3518479027559984754E-1')
```

11.5 Exponential integrals, and related functions

11.5.1 Exponential integral $E_1(x)$

`ctx.exp_integral_e1(`x`)`

where `ctx` is `math53` or `ctxflint`.

Returns the exponential integral $E_1(x) = \int_1^\infty \frac{e^{-xt}}{t}\,\mathrm{d}t, = e^{-z}U(1,1,z), \quad x \neq 0.$

For real $x < 0$ the integral can be calculated as $E_1(x) = -\mathrm{Ei}(-x)$.

See also Wikipedia [1321], MathWorld [978], NIST [776], BoostMath [154], Ehrhardt [297] (3.4.5), Ehrhardt [297] (4.2.27), Flint [772], Flint [766], Mpmath [651].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.E1(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.E1('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.e1(x); mx = mpm.e1(x); gx = gmp.e1(x)
>>> fx = fpm.e1(x); ax = apm.e1(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.3048381094197037412500745828645022948 48E-2
mpm:  1.3048381094197037412500745828645022948 48e-2
gmp:  1.3048381094197037412500745828645022948 48E-02
fpm:  1.30483810941970E-02
apm:  1.3048381094197037412500745828645022948 48e-2 (8.248e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.e1(z); mz = mpm.e1(z); gz = gmp.e1(z)
>>> fz = fpm.e1(z); az = apm.e1(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 8.63953958979585118 58E-4              + 8.78620837719744204 18E-3j
mpm: 8.6395395897958511858e-4              + 8.7862083771974420418e-3j
gmp: 8.63953958979585118 58E-04            + 8.78620837719744204 18E-03j
fpm: 8.63953958979585E-04                  + 8.78620837719744E-03j
apm: 8.6395395897958511848e-4 (8.234e-17%) + 8.7862083771974420418e-3 (7.381e-18%)j
```

11.5.2 Exponential integral $\mathrm{Ei}(x)$

`ctx.exp_integral_ei(x)`

where `ctx` is `math53`, `mathc53`, `ctxboost` or `ctxflint`.

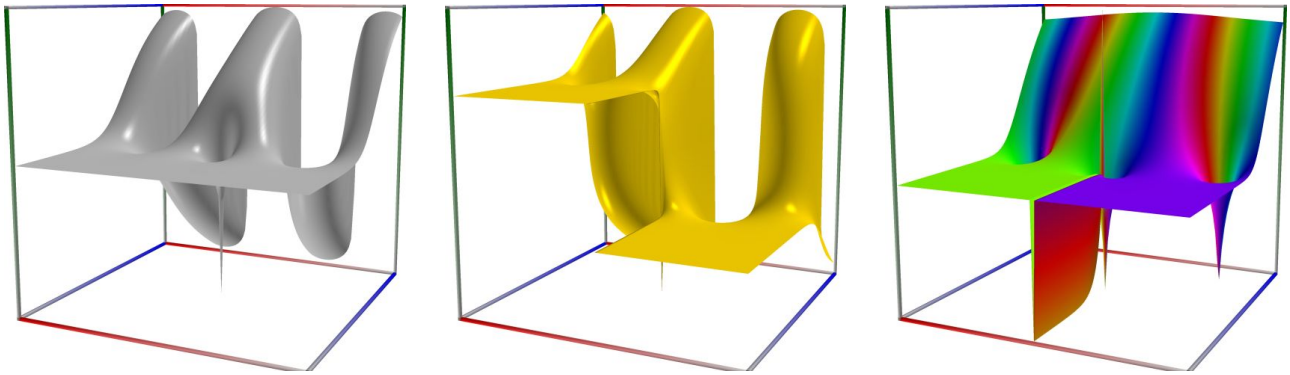
Returns the exponential integral $\mathrm{Ei}(x) = -PV\int_{-x}^{\infty} \frac{e^{-t}}{t}\,\mathrm{d}t = PV\int_{-\infty}^{x} \frac{e^{t}}{t}\,\mathrm{d}t$.

For real $x < 0$ we have $\mathrm{Ei}(x) = -E_1(-x)$.

See also Wikipedia [1321], MathWorld [978], NIST [776], BoostMath [154], Ehrhardt [297] (3.4.7), Ehrhardt [297] (4.2.27), Flint [772], Flint [766], Mpmath [652].

The exponential integral $\mathrm{Ei}(z)$ can also be calculated as using

$$\mathrm{Ei}(z) = -e^z U(1,1,-z) - \log(-z) + \frac{1}{2}\left(\log(z) - \log\left(\frac{1}{z}\right)\right)$$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Ei(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Ei('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.ei(x); mx = mpm.ei(x); gx = gmp.ei(x)
>>> fx = fpm.ei(x); ax = apm.ei(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.9338325706254165580083360192167652629 91E+0
mpm:  9.9338325706254165580083360192167652629 91e+0
gmp:  9.9338325706254165580083360192167652629 91E+00
fpm:  9.93383257062542E+00
apm:  9.9338325706254165580083360192167652629 91e+0 (9.245e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.ei(z); mz = mpm.ei(z); gz = gmp.ei(z)
```

(continues on next page)

(continued from previous page)

```
>>> fz = fpm.ei(z); az = apm.ei(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -4.1540916516426898225E+0               + 4.2944186200243574770E+0j
mpm: -4.1540916516426898225e+0               + 4.2944186200243574770e+0j
gmp: -4.1540916516426898225E+00              + 4.2944186200243574770E+00j
fpm: -4.15409165164269E+00                   + 4.29441862002436E+00j
apm: -4.1540916516426898225e+0 (-8.156e-20%) + 4.2944186200243574770e+0 (7.89e-20%)j
```

11.5.3 Logarithmic integral $\mathrm{li}(x)$

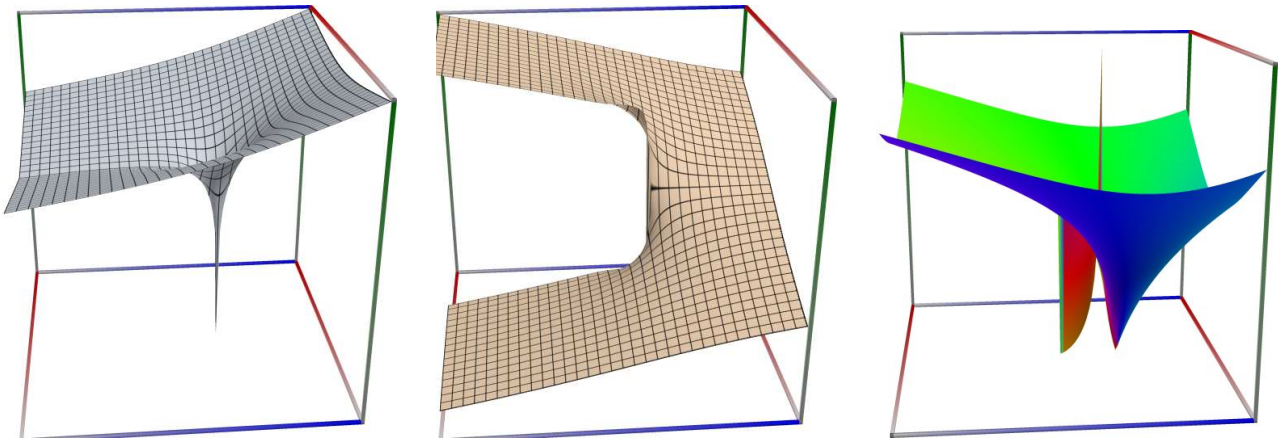
`ctx.log_integral(z)`

where `ctx` is `math53`, `mathc53` or `ctxflint`.

Returns the logarithmic integral $\mathrm{li}(x) = PV \int_0^x \frac{1}{\log(t)}\,\mathrm{d}t, \quad (x \geq 0, x \neq 1)$.

For $x \neq 0$ the integral is calculated as $\mathrm{li}(x) = \mathrm{Ei}(\log(x))$.

See also Wikipedia [1330], MathWorld [989], NIST [779], Ehrhardt [297] (3.4.15), Ehrhardt [297] (4.2.40), Flint [772], Flint [766], Mpmath [659].



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Li(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Li('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.li(x); mx = mpm.li(x); gx = gmp.li(x)
>>> fx = fpm.li(x); ax = apm.li(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.1635885946671919728769223673477217 3665 42E+0
mpm:  2.1635885946671919728769223673477217 3665 42e+0
gmp:  2.1635885946671919728769223673477217 3665 42E+00
fpm:  2.16358859466719E+00
apm:  2.1635885946671919728769223673477217 3665 42e+0 (1.061e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.li(z); mz = mpm.li(z); gz = gmp.li(z)
>>> fz = fpm.li(z); az = apm.li(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 3.1343755504645775265E+0              + 2.6769247817778742392E+0j
mpm: 3.1343755504645775265e+0              + 2.6769247817778742392e+0j
```

(continues on next page)

(continued from previous page)

```
gmp: 3.134375550464577526 5E+00              + 2.676924781777874239 2E+00j
fpm: 3.13437555046458E+00                    + 2.67692478177787E+00j
apm: 3.134375550464577526 5e+0 (1.081e-19%) + 2.676924781777874239 2e+0 (6.328e-20%)j
```

11.5.4 Hyperbolic sine integral $\mathrm{Shi}(x)$

`ctx.sinh_integral(`x`)`

where `ctx` is `math53`, `ctxflint`.

Returns the hyperbolic sine integral $\mathrm{Shi}(x) = \int_0^x \frac{\sinh(t)}{t}\,\mathrm{d}t$.

See also Wikipedia [1327], MathWorld [985], NIST [778], Ehrhardt [297] (3.4.17), Flint [772], Flint [766], Mpmath [662].

For real x the integral is calculated using the relations $\mathrm{Shi}(x) = \frac{1}{2}\left(\mathrm{Ei}(x) + \mathrm{E}_1(x)\right)$ for $x > 0$ and $\mathrm{Shi}(x) = -\mathrm{Shi}(-x)$ for $x < 0$.

For complex x the integral is calculated using the relation $\mathrm{Shi}(x) = -i\mathrm{Si}(ix)$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Shi(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Shi('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.shi(x); mx = mpm.shi(x); gx = gmp.shi(x)
>>> fx = fpm.shi(x); ax = apm.shi(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.9734404758598067977104183825227051429 70E+0
mpm:  4.9734404758598067977104183825227051429 70e+0
gmp:  4.9734404758598067977104183825227051429 70E+00
fpm:  4.97344047585981E+00
apm:  4.9734404758598067977104183825227051429 70e+0 (9.233e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.shi(z); mz = mpm.shi(z); gz = gmp.shi(z)
>>> fz = fpm.shi(z); az = apm.shi(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -2.0766138488418551187E+0               + 2.1516024142007774595E+0j
mpm: -2.0766138488418551187e+0               + 2.1516024142007774595e+0j
gmp: -2.0766138488418551187E+00              + 2.1516024142007774595E+00j
fpm: -2.07661384884186E+00                   + 2.15160241420078E+00j
apm: -2.0766138488418551187e+0 (-1.632e-19%) + 2.1516024142007774595e+0 (7.874e-20%)j
```

11.5.5 Hyperbolic cosine integral $\mathrm{Chi}(x)$

`ctx.cosh_integral(`x`)`

where `ctx` is `math53`, `ctxflint`.

Note: Also ctx.chi(z), ctx.coshIntegral(z)

Returns the hyperbolic cosine integral $\mathrm{Chi}(x) = -\int_x^\infty \frac{\cosh(t)}{t}\,\mathrm{d}t = \gamma + \log(x) + \int_0^x \frac{\cosh(t)-1}{t}\,\mathrm{d}t$.

See also Wikipedia [1326], MathWorld [984], NIST [778], Ehrhardt [297] (3.4.1), Flint [772], Flint [766], Mpmath [647].

For real x the integral is calculated using the relations $\mathrm{Chi}(x) = \frac{1}{2}\left(\mathrm{Ei}(x) - \mathrm{E}_1(x)\right)$ for $x > 0$ and $\mathrm{Chi}(x) = \mathrm{Chi}(-x)$ for $x < 0$.

For complex x the integral is calculated using the relation $\mathrm{Chi}(x) = \mathrm{Ci}(ix) - \log(ix) + \log(x)$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Chi(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Chi('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.chi(x); mx = mpm.chi(x); gx = gmp.chi(x)
>>> fx = fpm.chi(x); ax = apm.chi(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.960392094765609760297917636694060120021E+0
mpm:  4.960392094765609760297917636694060120021e+0
gmp:  4.960392094765609760297917636694060120021E+00
fpm:  4.96039209476561E+00
apm:  4.960392094765609760297917636694060120020e+0 (6.387e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.chi(z); mz = mpm.chi(z); gz = gmp.chi(z)
>>> fz = fpm.chi(z); az = apm.chi(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -2.0774778028008347038E+0               + 2.1428162058235800175E+0j
mpm: -2.0774778028008347038e+0               + 2.1428162058235800175e+0j
gmp: -2.0774778028008347038E+00              + 2.1428162058235800175E+00j
fpm: -2.07747780280083E+00                   + 2.14281620582358E+00j
apm: -2.0774778028008347038e+0 (-1.631e-19%) + 2.1428162058235800175e+0 (1.581e-19%)j
```

11.5.6 Generalized exponential integral $E_n(x)$

`ctx.exp_integral_en(`n, z`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the generalized exponential integral of integer order $E_n(x) = \int_1^\infty \frac{e^{-xt}}{t^n}\,\mathrm{d}t, \quad (n \geq 0)$.

See also Wikipedia [1321], MathWorld [979], NIST [462], BoostMath [155], Ehrhardt [297] (3.4.12), Mpmath [653].

The function can be generalized for complex s and complex z using

$$\mathrm{E}_{1-s}(z) = z^{-s}\Gamma(s, z).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.En(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.En(3, '0.51')
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '4'; x = '5.0'
>>> \mathrm{d}x = dec.expint(n, x); mx = mpm.expint(n, x); gx = gmp.expint(n, x)
>>> fx = fpm.expint(n, x); ax = apm.expint(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  7.829808450774252432788031803000123235715E-4
mpm:  7.829808450774252432788031803000123235715e-4
gmp:  7.829808450774252432788031803000123235715E-04
fpm:  7.82980845077425E-04
apm:  7.829808450774252432788031803000123235723e-4 (2.172e-36%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '4'; z = '5.0 + 3.0j'
>>> \mathrm{d}z = dec.expint(n, z); mz = mpm.expint(n, z); gz = gmp.expint(n, z)
>>> fz = fpm.expint(n, z); az = apm.expint(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -7.18201622646516495 11E-4               + 1.4860820449934938829E-4j
mpm: -7.18201622646516495 11e-4               + 1.4860820449934938829e-4j
gmp: -7.18201622646516495 11E-04              + 1.4860820449934938829E-04j
fpm: -7.18201622646517E-04                    + 1.48608204499349E-04j
apm: -7.18201622646516495 71e-4 (-1.103e-15%) + 1.4860820449934939066e-4 (8.075e-15%)j
```

11.5.7 Sine integral $\mathrm{Si}(x)$

`ctx.sin_integral(z)`

where `ctx` is `math53`, `ctxflint`.

Returns the sine integral $\mathrm{Si}(x) = \int_0^x \frac{\sin(t)}{t}\,\mathrm{d}t$, and $\mathrm{Si}(x) = -\mathrm{Si}(-x)$ for $x < 0$.

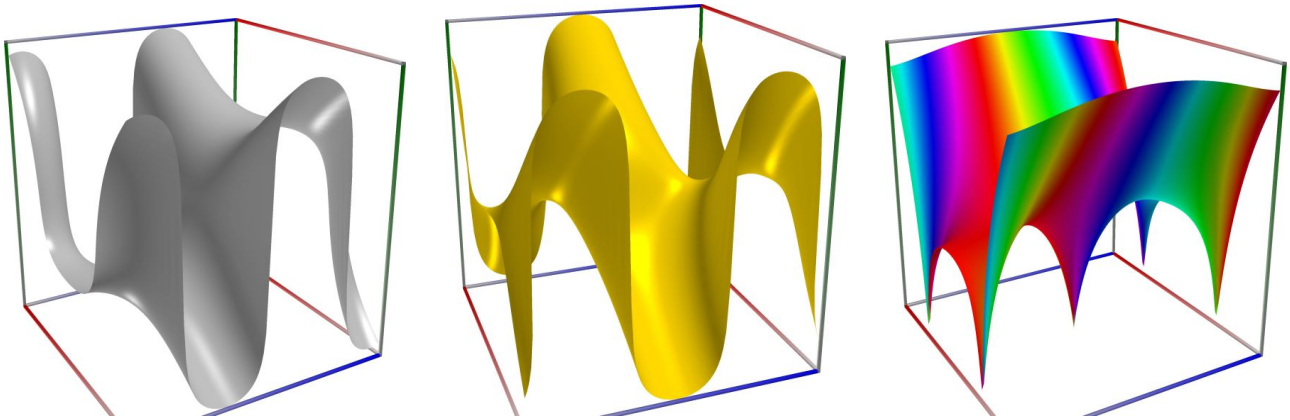
See also Wikipedia [1335], MathWorld [991], NIST [781], Ehrhardt [297] (3.4.18), Flint [772], Flint [766], Mpmath [663].

For complex z, the sine integral $\mathrm{Si}(z)$ can be computed using

$$\mathrm{Si}(z) = \frac{i}{2}\left[e^{iz}U(1,1,-iz) - e^{-iz}U(1,1,iz) + \log(-iz) - \log(iz)\right]$$

$$\mathrm{Si}(z) = z\,{}_1F_2(\tfrac{1}{2}; \tfrac{3}{2}, \tfrac{3}{2}; -\tfrac{z^2}{4})$$

and an automatic algorithm choice.



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Si(0.5)
ereal('5.23598775598298873O7E-1')
>>> math53.Si('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.si(x); mx = mpm.si(x); gx = gmp.si(x)
>>> fx = fpm.si(x); ax = apm.si(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.84865252799946825639773025111197324 5165E+0
mpm:  1.84865252799946825639773025111197324 5165e+0
gmp:  1.84865252799946825639773025111197324 5165E+00
fpm:  1.84865252799947E+00
apm:  1.84865252799946825639773025111197324 5165e+0 (6.21e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.si(z); mz = mpm.si(z); gz = gmp.si(z)
>>> fz = fpm.si(z); az = apm.si(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 6.74799508140403209 27E+0              - 3.498663721131909 4706E+0j
mpm: 6.74799508140403209 27e+0              - 3.498663721131909 4706e+0j
gmp: 6.74799508140403209 27E+00             - 3.498663721131909 4706E+00j
fpm: 6.74799508140403E+00                   - 3.49866372113191E+00j
apm: 6.74799508140403209 27e+0 (1.004e-19%) - 3.498663721131909 4706e+0 (-4.842e-20%)j
```

11.5.8 Cosine integral $\mathrm{Ci}(x)$

`ctx.cos_integral(z)`

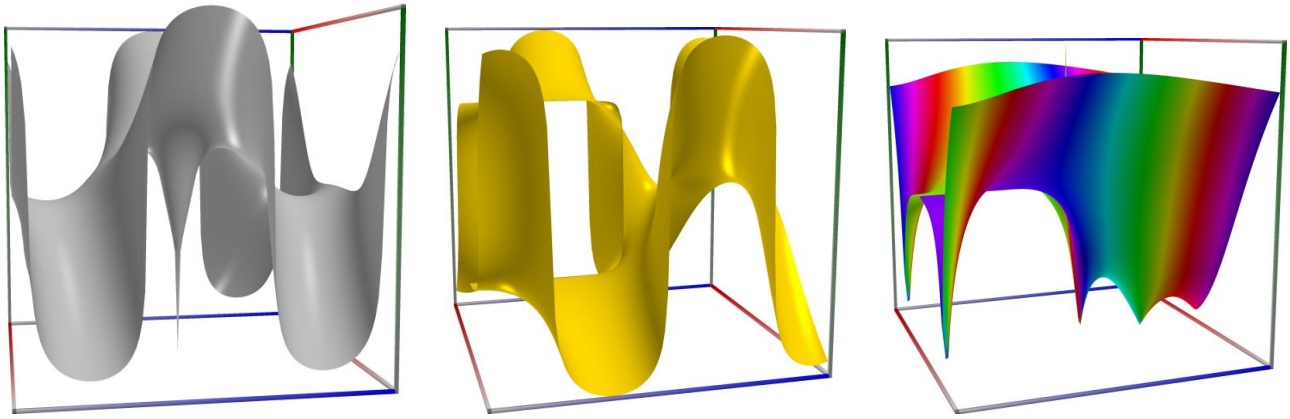
where `ctx` is `math53` or `ctxflint`.

Returns the cosine integral $\mathrm{Ci}(x) = -\int_x^\infty \frac{\cos(t)}{t}\,\mathrm{d}t = \gamma + \log(x) + \int_0^x \frac{\cos(t)-1}{t}\,\mathrm{d}t$, and $\mathrm{Ci}(x) = \mathrm{Ci}(-x)$ for $x < 0$.

For complex z, the cosine integral $\mathrm{Ci}(z)$, can be computed using

$$\mathrm{Ci}(z) = \log(z) - \frac{1}{2}\left[e^{iz}U(1,1,-iz) + e^{-iz}U(1,1,iz) + \log(-iz) + \log(iz)\right]$$

See also Wikipedia [1320], MathWorld [972], NIST [781], Ehrhardt [297] (3.4.2), Flint [772], Flint [766], Mpmath [650].



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \le x \le 6$ (blue axis), $-6 \le y \le 6$ (red axis), $-10 \le z \le 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Ci(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Ci('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = 3.0
>>> \mathrm{d}x = dec.ci(x); mx = mpm.ci(x); gx = gmp.ci(x)
>>> fx = fpm.ci(x); ax = apm.ci(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.1962978600800032762647228117667785054 68E-1
mpm:  1.1962978600800032762647228117667785054 68e-1
gmp:  1.1962978600800032762647228117667785054 68E-01
fpm:  1.19629786008000E-01
apm:  1.1962978600800032762647228117667785054 68e-1 (1.799e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.ci(z); mz = mpm.ci(z); gz = gmp.ci(z)
>>> fz = fpm.ci(z); az = apm.ci(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -3.4957570339825683441E+0               - 5.1759052151768084089E+0j
```

(continues on next page)

(continued from previous page)

```
mpm: -3.4957570339825683441e+0               - 5.1759052151768084089e+0j
gmp: -3.4957570339825683441E+00              - 5.1759052151768084089E+00j
fpm: -3.49575703398257E+00                   - 5.17590521517681E+00j
apm: -3.4957570339825683441e+0 (-9.692e-20%) - 5.1759052151768084089e+0 (-6.546e-20%)j
```

CHAPTER

TWELVE

HYPERGEOMETRIC FUNCTIONS ${}_2F_1$ AND ${}_1F_2$ (AND RELATED FUNCTIONS)

The source code for the C# tests in Visual Studio is divided by real and complex modules:

The source code for the C# tests in Visual Studio using real modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B12a_RealHypergeometric_pFq.cs,

The source code for the C# tests in Visual Studio using complex modules can be found online in the `XlCalcNet` repository or in the corresponding local `XlCalcNet` folder in the file B12b_CplxHypergeometric_pFq.cs,

12.1 Gauss Hypergeometric Function ${}_2F_1$

12.1.1 Gauss Hypergeometric Function, ${}_2F_1(a, b; c; x)$

`ctx.hyperg_2f1`(a, b, c, x)

where `ctx` is `math53` or `ctxflint`.

Note: ctx.hyperg2F1(a, b, c, x)

Returns ${}_2F_1(a, b, c, x) = \sum_{k=0}^{\infty} \frac{(a)_k (b)_k}{(c)_k} \frac{x^k}{k!}$, the Gauss hypergeometric function, defined for $|x| < 1$.

See also Wikipedia [1259], MathWorld [896], NIST [192], Ehrhardt [297] (3.8.1), BoostMath [105], Flint [758], Flint [751], Mpmath [592].

Except for special cases it is required that $-c \neq \mathbb{N}$. For $x > 1$ the function is generally complex and not implemented in AMath; but if a or b is a non-positive integer, then ${}_2F_1(a, b, c, x)$ becomes a polynomial in x and there is no restriction on x.

Special values are ${}_2F_1(0, b, c, x) = {}_2F_1(a, 0, c, x) = {}_2F_1(a, b, c, 0) = 1$ and, if $c - a - b > 0$, ${}_2F_1(a, b, c, 1) = \frac{\Gamma(c)\Gamma(c-a-b)}{\Gamma(c-a)\Gamma(c-b)}$.

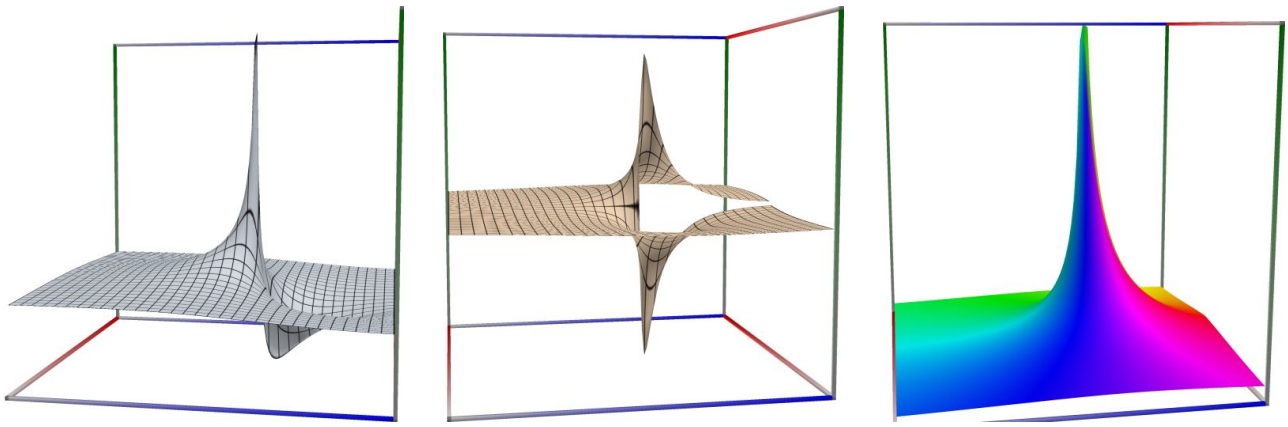
In AMath the analytic continuation is done using one or two linear transformations: For all $x < 1$ Abramowitz and Stegun. [4] equations (15.3.3-5) are used, and for $0 < x < 1$ if c and $c - a - b$ are no integers Abramowitz and Stegun. [4] equation (15.3.6). For $c = a + b \pm m, (m = 0, 1, \ldots)$ the (complicated) formulas from Abramowitz and Stegun. [4] (15.3.10-12) are implemented. If $a = -m$ is a negative integer (or b with a and b swapped, or both and $a \geq b$) the limiting cases for the (polynomial) transformations are implemented as in Abramowitz and Stegun. [4] equations (15.8.6/7).

This calls `arb_hypgeom_2f1` or `acb_hypgeom_2f1`

Returns the Gauss hypergeometric function ${}_2F_1(a, b; c; z)$.

The Gauss hypergeometric function ${}_2F_1$ is defined for $|z| < 1$ by the series

$$ {}_2F_1(a, b; c; z) = \sum_{k=0}^{\infty} \frac{(a)_k (b)_k}{(c)_k} \cdot \frac{z^k}{k!} $$



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Hyperg2F1(3,4,5,0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Hyperg2F1(13,14,15,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = '11.0'; b = '12.0'; c = '32.0'; x = '0.3'
>>> \mathrm{d}x = dec.hyp2f1(a, b, c, x); mx = mpm.hyp2f1(a, b, c, x); gx = gmp.hyp2f1(a, b, c, x)
>>> fx = fpm.hyp2f1(a, b, c, x); ax = apm.hyp2f1(a, b, c, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  3.9275802252632065666964888447254974488 41E+0
mpm:  3.9275802252632065666964888447254974488 41e+0
gmp:  3.9275802252632065666964888447254974488 41E+00
fpm:  3.92758022526321E+00
apm:  3.9275802252632065666964888447254974488 41e+0 (2.338e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '11.0 + 2.0j'; b = '12.0 + 3.0j'; c = '42.0 + 3.0j';z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.hyp2f1(a, b, c, z); mz = mpm.hyp2f1(a, b, c, z); gz = gmp.hyp2f1(a, b, c, z)
>>> fz = fpm.hyp2f1(a, b, c, z); az = apm.hyp2f1(a, b, c, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -1.0355451917135984282E-3              - 1.4654778601071805787E-3j
mpm: -1.0355451917135984282e-3              - 1.4654778601071805787e-3j
gmp: -1.0355451917135984282E-03             - 1.4654778601071805787E-03j
fpm: -1.03554519171360E-03                  - 1.46547786010718E-03j
apm: -1.0355451917135951286e-3 (-1.197e-9%) - 1.4654778601071774453e-3 (-8.461e-10%)j
```

12.1.2 Regularized Hypergeometric Function, ${}_2\widetilde{F}_1(a,b;c;x)$

`ctx.hyperg_2f1r`(a, b, c, x)

where `ctx` is `math53` or `ctxflint`.

Note: ctx.hyperg2F1r(a, b, c, x)

Returns ${}_2\widetilde{F}_1(a,b;c;x) = \dfrac{1}{\Gamma(c)}{}_2F_1(a,b;c;x)$, the regularized Gauss hypergeometric function, for $c \neq 0, -1, -2, \cdots$, or, if $c = 0, -1, -2, \cdots = -m$, the corresponding limit ${}_2\widetilde{F}_1(a,b;-m;x) = \dfrac{(a)_{m+1}(b)_{m+1}}{(m+1)!}x^{m+1}{}_2F_1(a+m+1, b+m+1; m+2; x)$.

See also Wikipedia [1259], MathWorld [259], NIST [192], Ehrhardt [297] (3.8.2), BoostMath [105], Flint [758], Flint [751], Mpmath [592].

This calls `arb_hypgeom_2f1` or `acb_hypgeom_2f1` with regularized set.

Returns the regularized Gauss hypergeometric function ${}_2\widetilde{F}_1(a,b;c;z)$.

The regularized Gauss hypergeometric function ${}_2\widetilde{F}_1(a,b;c;z)$ for unrestricted c, is defined by

$$ {}_2\widetilde{F}_1(a,b;c;z) = \frac{1}{\Gamma(c)}{}_2F_1(a,b;c;z) = \boldsymbol{F}(a,b;c;z), \qquad (c \neq 0, -1, -2, \cdots) $$

and by the corresponding limit if $c = 0, -1, -2, \cdots, = -n$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Hyperg2F1r(3,4,5,0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Hyperg2F1r(13,14,15,0.5)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = '11.0'; b = '12.0'; c = '32.0'; x = '0.3'
>>> \mathrm{d}x = dec.hyp2f1r(a, b, c, x); mx = mpm.hyp2f1r(a, b, c, x); gx = gmp.hyp2f1r(a, b, c, x)
>>> fx = fpm.hyp2f1r(a, b, c, x); ax = apm.hyp2f1r(a, b, c, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.7764286646529924750101609327504609858 59E-34
mpm:  4.7764286646529924750101609327504609858 59e-34
gmp:  4.7764286646529924750101609327504609858 59E-34
fpm:  4.77642866465299E-34
apm:  4.7764286646529924750101609327504609858 59e-34 (1.851e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '11.0 + 2.0j'; b = '12.0 + 3.0j'; c = '42.0 + 3.0j';z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.hyp2f1r(a, b, c, z); mz = mpm.hyp2f1r(a, b, c, z); gz = gmp.hyp2f1r(a, b, c, z)
>>> fz = fpm.hyp2f1r(a, b, c, z); az = apm.hyp2f1r(a, b, c, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 4.167625725629229627 8E-53             - 4.285539071374009067 2E-53j
mpm: 4.167625725629229627 8e-53             - 4.285539071374009067 2e-53j
gmp: 4.167625725629229627 8E-53             - 4.285539071374009067 2E-53j
fpm: 4.16762572562923E-53                   - 4.28553907137401E-53j
apm: 4.167625725629221379 1e-53 (8.47e-10%) - 4.285539071373996349 4e-53 (-8.237e-10%)j
```

12.2 Chebyshev, Gegenbauer and Jacobi polynomials

12.2.1 Chebyshev polynomial (or function) of the first kind, $T_n(x)$

`ctx.chebyshev_t(`n, x`)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $T_n(z) = {}_2F_1\left(-n, n, \frac{1}{2}, \frac{1-z}{2}\right)$, the Chebyshev polynomial of the first kind. The $T_n(x)$ are orthogonal on the interval $(-1, 1)$, with respect to the weight function $w(x) = (1-x^2)^{-1/2}$.

For integer n, the following recursion can be used, with $T_n(x) = T_{-n}(x)$:

$$\begin{aligned} T_0(x) &= 1 \\ T_1(x) &= x \\ T_{n+1}(x) &= 2xT_n(x) - T_{n-1}(x). \end{aligned} \tag{12.1}$$

See also Wikipedia [1300], MathWorld [915], NIST [397], BoostMath [114], Ehrhardt [297] (3.7.1), Flint [759], Flint [755], Mpmath [626], Ehrhardt [297] (3.7.5).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ChebyshevT(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.ChebyshevT('6, 0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '10'; x = '5.0'
>>> \mathrm{d}x = dec.chebyt(n, x); mx = mpm.chebyt(n, x); gx = gmp.chebyt(n, x)
>>> fx = fpm.chebyt(n, x); ax = apm.chebyt(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.5172512490000000000000000000000000000E+9
mpm:  4.5172512490000000000000000000000000000e+9
gmp:  4.5172512490000000000000000000000000000E+09
fpm:  4.51725124900000E+09
apm:  4.5172512490000000000000000000000000000e+9 (0.0%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.chebyt(n, z); mz = mpm.chebyt(n, z); gz = gmp.chebyt(n, z)
>>> fz = fpm.chebyt(n, z); az = apm.chebyt(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.5445350751000000000E+10        - 1.6336798500000000000E+10j
mpm: 1.5445350751000000000e+10        - 1.6336798500000000000e+10j
gmp: 1.5445350751000000000E+10        - 1.6336798500000000000E+10j
fpm: 1.54453507510000E+10             - 1.63367985000000E+10j
apm: 1.5445350751000000000e+10 (0.0%) - 1.6336798500000000000e+10 (0.0%)j
```

12.2.2 Chebyshev polynomial of the second kind, $U_n(x)$

`ctx.chebyshev_u(n, x)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $U_n(z) = (n+1)_2F_1\left(-n, n+2, \frac{3}{2}, \frac{1-z}{2}\right)$, the Chebyshev polynomial of the second kind. The $U_n(x)$ are orthogonal on the interval $(-1, 1)$, with respect to the weight function $w(x) = (1-x^2)^{1/2}$.

For integer n, the following recursion can be used, with $U_{-1}(x) = 0$ and $U_n(x) = -U_{-n-2}(x)$:

$$\begin{aligned} U_0(x) &= 1 \\ U_1(x) &= 2x \\ U_{n+1}(x) &= 2xU_n(x) - U_{n-1}(x). \end{aligned} \tag{12.2}$$

See also Wikipedia [1300], MathWorld [916], NIST [397], Ehrhardt [297] (3.7.2), BoostMath [114], Flint [759], Flint [755], Mpmath [627].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ChebyshevU(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.ChebyshevU('6, 0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '10'; x = '5.0'
>>> \mathrm{d}x = dec.chebyu(n, x); mx = mpm.chebyu(n, x); gx = gmp.chebyu(n, x)
>>> fx = fpm.chebyu(n, x); ax = apm.chebyu(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.12765149900000000000000000000000000000E+9
mpm:  9.12765149900000000000000000000000000000e+9
gmp:  9.12765149900000000000000000000000000000E+09
fpm:  9.12765149900000E+09
apm:  9.12765149900000000000000000000000000000e+9 (0.0%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.chebyu(n, z); mz = mpm.chebyu(n, z); gz = gmp.chebyu(n, z)
>>> fz = fpm.chebyu(n, z); az = apm.chebyu(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 3.0778307711000000000E+10        - 3.2988132600000000000E+10j
mpm: 3.0778307711000000000e+10        - 3.2988132600000000000e+10j
gmp: 3.0778307711000000000E+10        - 3.2988132600000000000E+10j
fpm: 3.07783077110000E+10             - 3.29881326000000E+10j
apm: 3.0778307711000000000e+10 (0.0%) - 3.2988132600000000000e+10 (0.0%)j
```

12.2.3 Chebyshev polynomials of the third kind, $V_n(x)$

`ctx.chebyshev_v(n, x)`

Returns $V_n(x)$, the Chebyshev polynomial of the first kind. The $V_n(x)$ are orthogonal on the interval $(-1, 1)$, with respect to the weight function $w(x) = (1+x^2)^{1/2}(1-x)^{-1/2}$.

The function can also be calculated as $V_n(x) = (-1)^n(2n+1)_2F_1\left(-n, n+1, \frac{3}{2}, \frac{1+x}{2}\right)$, which allows for complex n and x.

We also have $V_n(x) = \sqrt{\frac{2}{1+x}}T_{2n+1}\left(\sqrt{\frac{x+1}{2}}\right)$ for $x \geq 0$, and $V_n(x) = (-1)^n W_n(-x)$ for $x < 0$.

For integer n and real x, the following recursion can be used, with $V_{-n-1}(x) = V_n(x)$:

$$\begin{aligned} V_0(x) &= 1 \\ V_1(x) &= 2x - 1 \\ V_{n+1}(x) &= 2xV_n(x) - V_{n-1}(x). \end{aligned} \tag{12.3}$$

See also Wikipedia [1300], MathWorld [915], NIST [397], Ehrhardt [297] (3.7.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ChebyshevV(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.ChebyshevV('6, 0.51')
ereal('5.351847902755998475 4E-1')
```

12.2.4 Chebyshev polynomials of the fourth kind, $W_n(x)$

`ctx.chebyshev_w(n, x)`

Returns $W_k(z)$, the Chebyshev polynomial of the fourth kind of degree $n \geq 0$. The $W_n(x)$ are orthogonal on the interval $(-1, 1)$, with respect to the weight function $w(x) = (1-x)^{1/2}(1+x^2)^{-1/2}$.

The function can also be calculated as $W_n(z) = (-1)^n {}_2F_1\left(-n, n+1, \frac{1}{2}, \frac{1+x}{2}\right)$, which allows for complex n and x.

We also have $W_n(x) = U_{2n}\left(\sqrt{\frac{x+1}{2}}\right)$ for $x \geq 0$, and $W_n(x) = (-1)^n V_n(-x)$ for $x < 0$.

For integer n, the following recursion can be used, with $W_{-n-1}(x) = -W_n(x)$:

$$\begin{aligned} W_0(x) &= 1 \\ W_1(x) &= 2x + 1 \\ W_{n+1}(x) &= 2xW_n(x) - W_{n-1}(x). \end{aligned} \tag{12.4}$$

See also Wikipedia [1300], MathWorld [915], NIST [397], Ehrhardt [297] (3.7.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ChebyshevW(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.ChebyshevW('6, 0.51')
ereal('5.351847902755998475 4E-1')
```

12.2.5 Gegenbauer (ultraspherical) polynomial, $C_n^{\alpha}(x)$

`ctx.gegenbauer_c`(*n*, *alpha*, *x*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $C_n^{(a)}(z)$, the Gegenbauer polynomial of degree n with parameter a. The Gegenbauer polynomials are orthogonal on the interval $(-1, 1)$, with respect to the weight function $w(x) = (1 - x^2)^{a-1/2}$.

If $a \neq 0$ the function uses the recurrence formulas

$$\begin{aligned} C_0^{(a)}(x) &= 1 \\ C_1^{(a)}(x) &= 2ax \\ nC_n^{(a)}(x) &= 2(n + a - 1)xC_{n-1}^{(a)}(x) - (n + 2a - 2)C_{n-2}^{(a)}(x). \end{aligned} \tag{12.5}$$

For $a = 0$ the result can be expressed in Chebyshev polynomials: $C_0^{(0)}(x) = 1, \quad C_n^{(0)}(x) = 2/nT_n(x)$.

See also Wikipedia [1305], MathWorld [921], NIST [397], BoostMath [120], Ehrhardt [297] (3.7.6), Flint [759], Flint [755], Mpmath [628].

The function can also be expressed as

$$C_n^a(z) = \frac{(2a)_n}{\Gamma(n+1)} {}_2F_1\left(-n, 2a + n, a + \frac{1}{2}, \frac{1-z}{2}\right).$$

For nonnegative integer *n*, this is a polynomial in *a* and *z*, even when the parameters are such that the hypergeometric series is undefined. In such cases, the polynomial is evaluated using direct methods.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.GegenbauerC(3, 2, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.GegenbauerC('6, 2, 0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '10' ; a = '7.0'; x = '5.0'
>>> \mathrm{d}x = dec.gegenbauer(n, a, x); mx = mpm.gegenbauer(n, a, x); gx = gmp.gegenbauer(n, a, x)
>>> fx = fpm.gegenbauer(n, a, x); ax = apm.gegenbauer(n, a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  7.56589847855380000000000000000000000000E+13
mpm:  7.56589847855380000000000000000000000000e+13
gmp:  7.56589847855380000000000000000000000000E+13
fpm:  7.56589847855380E+13
apm:  7.56589847855380000000000000000000000000e+13 (2.135e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; a = '7.0 + 1j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.gegenbauer(n, a, z); mz = mpm.gegenbauer(n, a, z); gz = gmp.gegenbauer(n, a, z)
>>> fz = fpm.gegenbauer(n, a, z); az = apm.gegenbauer(n, a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 3.8222147440285373854E+14              + 3.3043282427731541887E+13j
mpm: 3.8222147440285373854e+14              + 3.3043282427731541887e+13j
gmp: 3.8222147440285373854E+14              + 3.3043282427731541887E+13j
fpm: 3.82221474402854E+14                   + 3.30432824277315E+13j
apm: 3.8222147440285373854e+14 (6.238e-20%) + 3.3043282427731541887e+13 (2.255e-19%)j
```

12.2.6 Jacobi polynomials, $P_n^{(a,b)}(x)$

`ctx.jacobi_p`(n, a, b, x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $P_n^{(a,b)}(x)$, the Jacobi polynomial of degree $n \geq 0$ with parameters (a, b), where a, b should be greater than -1, and $a+b$ must not be an integer less than -1. Jacobi polynomials are orthogonal on the interval $(-1, 1)$, with respect to the weight function $w(x) = (1-x)^a(1+x)^b$, if a, b are greater than -1.

The cases $n \leq 1$ are computed with the explicit formulas

$$P_0^{(a,b)} = 1, \quad 2P_1^{(a,b)} = (a-b) + (a+b+2)x,$$

and for $n > 1$ there are the somewhat complicated recurrence relations

$$\begin{aligned} P_{n+1}^{(a,b)} &= (A_n x + B_n)P_n^{(a,b)} - C_n P_{n-1}^{(a,b)} \\ A_n &= \frac{(2n+a+b+1)(2n+a+b+2)}{2(n+1)(n+a+b+1)} \\ B_n &= \frac{(a^2-b^2)(2n+a+b+1)}{2(n+1)(n+a+b+1)(2n+a+b)} \\ C_n &= \frac{(n+a)(n+b)(2n+a+b+2)}{(n+1)(n+a+b+1)(2n+a+b)}. \end{aligned} \tag{12.6}$$

See also Wikipedia [1312], MathWorld [926], NIST [397], BoostMath [123], Ehrhardt [297] (3.7.9), Mpmath [633].

The function can also be expressed as

$$P_n^{(a,b)}(x) = \frac{(a+1)_n}{\Gamma(n+1)} {}_2F_1\left(-n, n+a+b+1, a+1, \frac{1-x}{2}\right).$$

For nonnegative integer *n*, this is a polynomial in *a*, *b* and *z*, even when the parameters are such that the hypergeometric series is undefined. In such cases, the polynomial is evaluated using direct methods.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.JacobiP(2, 3, 2, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.JacobiP('6, 2, 0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '10' ; a = '7.0'; b = '8.0'; x = '5.0'
>>> \mathrm{d}x = dec.jacobi(n, a, b, x); mx = mpm.jacobi(n, a, b, x); gx = gmp.jacobi(n, a, b, x)
>>> fx = fpm.jacobi(n, a, b, x); ax = apm.jacobi(n, a, b, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.5664928912880000000000000000000000000000E+12
mpm:  1.5664928912880000000000000000000000000000e+12
gmp:  1.5664928912880000000000000000000000000000E+12
fpm:  1.56649289128800E+12
apm:  1.5664928912880000000000000000000000000000e+12 (1.611e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; a = '7.0 + 1j'; b = '8.0 + 2j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.jacobi(n, a, b, z); mz = mpm.jacobi(n, a, b, z); gz = gmp.jacobi(n, a, b, z)
>>> fz = fpm.jacobi(n, a, b, z); az = apm.jacobi(n, a, b, z)
```

(continues on next page)

(continued from previous page)

```
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 7.8001821589253876064E+12               + 1.0625055373536412146E+12j
mpm: 7.8001821589253876064e+12               + 1.0625055373536412146e+12j
gmp: 7.8001821589253876064E+12               + 1.0625055373536412146E+12j
fpm: 7.80018215892539E+12                    + 1.06250553735364E+12j
apm: 7.8001821589253876064e+12 (4.776e-20%) + 1.0625055373536412146e+12 (1.315e-19%)j
```

12.2.7 Zernike radial polynomials $R_n^m(r)$

`ctx.zernike_r(n, m, r)`

Returns the Zernike radial polynomial $R_n^m(r)$, with $r \geq 0$, and $n \geq m \geq 0$, $n - m$ even, zero otherwise. The orthogonality relation is

$$\int_0^1 R_n^m(r) R_{n'}^m(r) r \, \mathrm{d}r = \frac{1}{2(n+1)} \delta_{nn'}.$$

$$R_n^m(r) = (-1)^{(n-m)/2} r^m P_{(n-m)/2}^{(m,0)}(1 - 2r^2) = r^m P_{(n-m)/2}^{(0,m)}(2r^2 - 1).$$

See also: http://mathworld.wolfram.com/ZernikePolynomial.html

See also https://en.wikipedia.org/wiki/Zernike_polynomials

See also Ehrhardt [297] (3.7.20).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ZernikeR(4, 5, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.ZernikeR(4, 5,'0.51')
ereal('5.351847902755998475 4E-1')
```

12.3 Legendre polynomials and related

12.3.1 Legendre polynomial (or function) of the first kind, $P_n(x)$

`ctx.legendre_p(n, x)`

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $P_n(x) = {}_2F_1\left(-n, n+1, 1, \frac{1-x}{2}\right)$, the Legendre polynomial of degree n. The Legendre polynomials are orthogonal on the interval $(-1, 1)$ with $w(x) = 1$. If $n \geq 0$ the function uses the following recurrence relation, with $P_n(x) = P_{-n-1}(x)$:

$$\begin{aligned} P_0(x) &= 1 \\ P_1(x) &= x \\ (n+1)P_{n+1}(x) &= (2n+1)P_n(x) - nP_{n-1}(x). \end{aligned} \tag{12.7}$$

See also Wikipedia [1314], MathWorld [929], NIST [293], BoostMath [125], Ehrhardt [297] (3.7.13), Flint [759], Flint [755], Mpmath [635].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LegendreP(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.LegendreP(6, '0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '10'; x = '5.0'
>>> \mathrm{d}x = dec.legendre(n, x); mx = mpm.legendre(n, x); gx = gmp.legendre(n, x)
>>> fx = fpm.legendre(n, x); ax = apm.legendre(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.6004726770000000000000000000000000000E+9
mpm:  1.6004726770000000000000000000000000000e+9
gmp:  1.6004726770000000000000000000000000000E+09
fpm:  1.60047267700000E+09
apm:  1.6004726770000000000000000000000000000e+9 (1.54e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.legendre(n, z); mz = mpm.legendre(n, z); gz = gmp.legendre(n, z)
>>> fz = fpm.legendre(n, z); az = apm.legendre(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 5.4324745448476562500E+9        - 5.7861538517578125000E+9j
mpm: 5.4324745448476562500e+9        - 5.7861538517578125000e+9j
gmp: 5.4324745448476562500E+09       - 5.7861538517578125000E+09j
fpm: 5.43247454484766E+09            - 5.78615385175781E+09j
apm: 5.4324745448476562500e+9 (0.0%) - 5.7861538517578125000e+9 (0.0%)j
```

12.3.2 Associated Legendre function of the first kind, $P_l^m(x)$

`ctx.legendre_plm`(l, m, x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns $\frac{1}{\Gamma(1-m)} \frac{(1+z)^{m/2}}{(1-z)^{m/2}} {}_2F_1\left(-n, n+1, 1-m, \frac{1-z}{2}\right)$., the (associated) Legendre function of the first kind of degree n and order m. Taking $m = 0$ gives the ordinary Legendre function of the first kind, $P_n(z)$.

See also Wikipedia [1296], MathWorld [928], NIST [293], BoostMath [125], Ehrhardt [297] (3.7.14), Flint [759], Flint [755], Mpmath [623].

Many different branch cut conventions appear in the literature. If *type* is 0, the version

$$P_n^m(z) = \frac{(1+z)^{m/2}}{(1-z)^{m/2}} \mathbf{F}\left(-n, n+1, 1-m, \frac{1-z}{2}\right)$$

is computed, and if *type* is 1, the alternative version

$$\mathcal{P}_n^m(z) = \frac{(z+1)^{m/2}}{(z-1)^{m/2}} \mathbf{F}\left(-n, n+1, 1-m, \frac{1-z}{2}\right).$$

is computed. Type 0 and type 1 respectively correspond to type 2 and type 3 in *Mathematica* and *mpmath*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LegendrePlm(2, 3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.LegendrePlm(2, 6, '0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '10'; m = '7'; x = '0.5'
>>> \mathrm{d}x = dec.legenp(n, m, x); mx = mpm.legenp(n, m, x); gx = gmp.legenp(n, m, x)
>>> fx = fpm.legenp(n, m, x); ax = apm.legenp(n, m, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -1.836027792321961447325321009149754756328E+6
mpm:  -1.836027792321961447325321009149754756328e+6
gmp:  -1.836027792321961447325321009149754756328E+06
fpm:  -1.83602779232196E+06
apm:  -1.836027792321961447325321009149754756328e+6 (-6.556e-40%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10 + 0j'; m = '7 + 1j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.legenp(n, m, z); mz = mpm.legenp(n, m, z); gz = gmp.legenp(n, m, z)
>>> fz = fpm.legenp(n, m, z); az = apm.legenp(n, m, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 5.1676243722757488933E+14              - 9.9095186680298524778E+14j
mpm: 5.1676243722757488933e+14              - 9.9095186680298524778e+14j
gmp: 5.1676243722757488933E+14              - 9.9095186680298524778E+14j
fpm: 5.16762437227575E+14                   - 9.90951866802985E+14j
apm: 5.1676243722757488933e+14 (2.768e-19%) - 9.9095186680298524778e+14 (-1.925e-19%)j
```

12.3.3 Legendre function of the second kind, $Q_l(x)$

`ctx.legendre_q`(l, x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

See also: https://en.wikipedia.org/wiki/Legendre_polynomials

Returns $Q_n(x) = {}_2F_1\left(\frac{l+1}{2}, \frac{l+2}{2}; l+\frac{3}{2}; \frac{1}{x^2}\right)$, the Legendre function of the second kind of degree l. For integer $l \geq 0$ and $x \neq 1$, the following recurrence relations hold:

$$\begin{aligned} Q_0(x) &= \frac{1}{2}\log\left(\frac{1+x}{1-x}\right) \\ Q_1(x) &= \frac{x}{2}\log\left(\frac{1+x}{1-x}\right) - 1 \\ (k+1)Q_{k+1}(x) &= (2k+1)Q_k(x) - kQ_{k-1}(x). \end{aligned} \tag{12.8}$$

See also Wikipedia [1314], MathWorld [929], NIST [293], BoostMath [125], Ehrhardt [297] (3.7.15), Flint [759], Flint [755], Mpmath [624].

This function returns $Q_l^m(x)$, the associated Legendre functions of the second kind with $l \geq 0$, $l+m \geq 0$ and $x \neq 1$, defined as

$$Q_l^m(x) = (-1)^m(1-x^2)^{m/2}\frac{d^m}{dx^m}Q_l(x), \quad |x| < 1,$$

$$Q_l^m(x) = (x^2-1)^{m/2}\frac{d^m}{dx^m}Q_l(x), \quad |x| > 1.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LegendreQ(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.LegendreQ(6, '0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '10'; m = '7'; x = '0.5'
>>> \mathrm{d}x = dec.legenq(n, m, x); mx = mpm.legenq(n, m, x); gx = gmp.legenq(n, m, x)
>>> fx = fpm.legenq(n, m, x); ax = apm.legenq(n, m, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  2.71665140966003723409498678333672825 4923E+6
mpm:  2.71665140966003723409498678333672825 4923e+6
gmp:  2.71665140966003723409498678333672825 4923E+06
fpm:  2.71665140966004E+06
apm:  2.71665140966003723409498678333672825 4923e+6 (3.811e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10 + 0j'; m = '7 + 1j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.legenq(n, m, z); mz = mpm.legenq(n, m, z); gz = gmp.legenq(n, m, z)
>>> fz = fpm.legenq(n, m, z); az = apm.legenq(n, m, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.55658355240467488 18E+15               + 8.11728538222652975 87E+14j
mpm: 1.55658355240467488 18e+15               + 8.11728538222652975 87e+14j
gmp: 1.55658355240467488 18E+15               + 8.11728538222652975 87E+14j
fpm: 1.55658355240468E+15                     + 8.11728538222653E+14j
apm: 1.55658355240467488 18e+15 (9.619e-18%) + 8.11728538222652975 86e+14 (1.78e-17%)j
```

12.3.4 Associated Legendre function of the second kind, $Q_l^m(x)$

`ctx.legendre_qlm`(l, m, x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

For generalization to hypergeometric functions, see https://en.wikipedia.org/wiki/Legendre_function#Solutions_of_the_differential_equation

and https://en.wikipedia.org/wiki/Associated_Legendre_polynomials#Generalization_via_hypergeometric_functions

Returns $Q_l^m(z) = \frac{\pi}{2\sin(\pi m)}\left(\cos(\pi m)P_l^m(z) - \frac{\Gamma(1+m+l)}{\Gamma(1-m+l)}P_l^{-m}(z)\right)$, the (associated) Legendre function of the second kind of degree l and order m.

Here $P_l^m(z)$ is the Legendre function of the first kind of degree l and order m. The formula above should be understood as limit when m is an integer. Taking $m = 0$ gives the ordinary Legendre function of the second kind, $Q_n(z)$.

A different formula from Wikipedia:

$$Q_\lambda^\mu(z) = \frac{\sqrt{\pi}\,\Gamma(\lambda+\mu+1)}{2^{\lambda+1}\Gamma(\lambda+3/2)}\frac{1}{z^{\lambda+\mu+1}}(1-z^2)^{\mu/2}\,{}_2F_1\left(\frac{\lambda+\mu+1}{2}, \frac{\lambda+\mu+2}{2}; \lambda+\frac{3}{2}; \frac{1}{z^2}\right)$$

For integer m, $l \geq 0$, $l + m \geq 0$ and $x \neq 1$, the function can be defined by

$$Q_l^m(x) = (-1)^m(1-x^2)^{m/2}\frac{d^m}{\mathrm{d}x^m}Q_l(x),$$

$$Q_l^{-m}(x) = (x^2-1)^{m/2}\frac{(l-m)!}{(l+m)!}Q_l^m(x).$$

The factor $(-1)^m$ is omitted if $|x| > 1$, see NIST [293] (14.9.14).

See also Wikipedia [1314], MathWorld [929], NIST [293], BoostMath [125], Ehrhardt [297] (3.7.16), Mpmath [624].

Sets *res* to the associated Legendre function of the second kind evaluated for degree *n*, order *m*, and argument *z*. When *m* is zero, this reduces to the Legendre function $Q_n(z)$.

Many different branch cut conventions appear in the literature. If *type* is 0, the version

$$Q_n^m(z) = \frac{\pi}{2\sin(\pi m)}\left(\cos(\pi m)P_n^m(z) - \frac{\Gamma(1+m+n)}{\Gamma(1-m+n)}P_n^{-m}(z)\right)$$

is computed, and if *type* is 1, the alternative version

$$\mathcal{Q}_n^m(z) = \frac{\pi}{2\sin(\pi m)}e^{\pi i m}\left(\mathcal{P}_n^m(z) - \frac{\Gamma(1+m+n)}{\Gamma(1-m+n)}\mathcal{P}_n^{-m}(z)\right)$$

is computed. Type 0 and type 1 respectively correspond to type 2 and type 3 in *Mathematica* and *mpmath*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LegendrePlm(2, 3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.LegendrePlm(2, 6, '0.51')
ereal('5.351847902755998475 4E-1')
```

12.3.5 Spherical harmonics, $Y_n^m(\theta, \phi)$

`ctx.spherical_harmonic`(*theta*, *phi*, *n*, *m*)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Note: ctx.spherHarm(theta, phi, n, m), ctxboost.SphericalHarmonicR(theta, phi, n, m), ctxboost.SphericalHarmonicI(theta, phi, n, m)

!!! n and m need to be integer !!!

Returns $Y_l^m(\theta, \phi) = \sqrt{\frac{2l+1}{4\pi}\frac{(l-m)!}{(l+m)!}} P_l^m(\cos\theta) e^{im\phi}$ the spherical harmonic, where $P_l^m(z)$ is the Legendre function of the first kind of degree l and order m, $\theta \in [0, \pi]$ denotes the polar coordinate (ranging from the north pole to the south pole) and $\phi \in [0, 2\pi]$ denotes the azimuthal coordinate on a sphere. Care should be used since many different conventions for spherical coordinate variables are used.

See also Wikipedia [1317], MathWorld [935], NIST [294], BoostMath [130], Ehrhardt [297] (3.7.17), Flint [755], Mpmath [636].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SpherHarm(2, 3, 5, 4)
ereal('5.2359877559829887307E-1')
>>> math53.SpherHarm(2.1, 3.1, 5.1, 4.1)
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; l = '10'; m = '7'; theta = '0.1'; phi = '0.2'
>>> \mathrm{d}x = dec.spherharm(l, m, theta, phi); mx = mpm.spherharm(l, m, theta, phi);
>>> gx = gmp.spherharm(l, m, theta, phi)
>>> fx = fpm.spherharm(l, m, theta, phi); ax = apm.spherharm(l, m, theta, phi)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: -2.5484347403024134998E-7                - 1.4775528280770230150E-6j
mpm: -2.5484347403024134998e-7                - 1.4775528280770230150e-6j
gmp: -2.5484347403024134998E-07               - 1.4775528280770230150E-06j
fpm: -2.54843474030241E-07                    - 1.47755282807702E-06j
apm: -2.5484347403024134998e-7 (-9.509e-19%) - 1.4775528280770230150e-6 (-4.92e-19%)j
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; m = '7'; theta = '0.1 + 03j'; phi = '0.2 + 04j'
>>> \mathrm{d}z = dec.spherharm(l, m, theta, phi); mz = mpm.spherharm(l, m, theta, phi);
>>> gz = gmp.spherharm(l, m, theta, phi)
>>> fz = fpm.spherharm(l, m, theta, phi); az = apm.spherharm(l, m, theta, phi)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -5.0629719914753183582E-2                + 1.2051935898383241506E-1j
mpm: -5.0629719914753183582e-2                + 1.2051935898383241506e-1j
gmp: -5.0629719914753183582E-02               + 1.2051935898383241506E-01j
fpm: -5.06297199147532E-02                    + 1.20519358983832E-01j
apm: -5.0629719914753183582e-2 (-6.797e-19%) + 1.2051935898383241506e-1 (3.075e-19%)j
```

12.3.6 Toroidal harmonics $P^m_{l-1/2}(x)$

`ctx.toroidal_plm`(*l*, *m*, *x*)

Returns the toroidal harmonic $P^m_{l-1/2}(x)$, which is an associated Legendre function $P_l^m(x)$ of the first kind with half-integer degree. The current implementation is based on Amath and is restricted to $l, m = 0, 1; x > 1$, using Legendre elliptic integrals or Bulirsch elliptic integrals for numerical evaluation.

See also: https://mathworld.wolfram.com/ToroidalFunction.html

See also: https://dlmf.nist.gov/14.19

See also: Ehrhardt [297] (3.7.18).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ToroidalPlm(0, 0, 1.5)
ereal('5.2359877559829887307E-1')
>>> math53.ToroidalPlm(0, 1, 1.5)
ereal('5.3518479027559984754E-1')
>>> math53.ToroidalPlm(1, 0, 1.5)
ereal('5.2359877559829887307E-1')
>>> math53.ToroidalPlm(1, 1, 1.5)
ereal('5.3518479027559984754E-1')
```

12.3.7 Toroidal harmonics $Q^m_{l-1/2}(x)$

`ctx.toroidal_qlm`(l, m, x)

Returns the toroidal harmonic $Q^m_{l-1/2}(x)$, which is an associated Legendre function $Q^m_l(x)$ of the second kind with half-integer degree. The current implementation is based on Amath and is restricted to $l, m = 0, 1; x > 1$, using Legendre elliptic integrals or Bulirsch elliptic integrals for numerical evaluation.

See also: Majic (2019), https://mathworld.wolfram.com/ToroidalFunction.html

See also: Ehrhardt [297] (3.7.18).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ToroidalQlm(0, 0, 1.5)
ereal('5.2359877559829887307E-1')
>>> math53.ToroidalQlm(0, 1, 1.5)
ereal('5.3518479027559984754E-1')
>>> math53.ToroidalQlm(1, 0, 1.5)
ereal('5.2359877559829887307E-1')
>>> math53.ToroidalQlm(1, 1, 1.5)
ereal('5.3518479027559984754E-1')
```

12.4 Incomplete beta functions

12.4.1 Non-normalized lower incomplete beta function, $B_{\text{lower}}(a, b; x)$

`ctx.beta_lower`(a, b, x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Note: ctx.beta3(a, b, x), ctxboost.IBetaNonNormalized(a, b, x)

Returns the non-normalized incomplete beta function $B_x(a, b) = B_{\text{lower}}(a, b; x) = \int_0^x t^{a-1}(1-t)^{b-1}\,\mathrm{d}t$, for $a > 0$, $b > 0$, and $0 \le x \le 1$.

We also have $B_{\text{lower}}(a, b; x) = \frac{x^a}{a}{}_2F_1(a, 1-b, a+1, x)$

The function is undefined for nonpositive integer a.

See also Wikipedia [1222], MathWorld [847], NIST [458], BoostMath [79], BoostMath [87], Ehrhardt [297] (3.5.3.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Beta3(3.1, 0.5, 0.3)
ereal('5.235987755982988730 7E-1')
>>> math53.Beta3(3.4, '0.51', 0.3)
ereal('5.351847902755998475 4E-1')
```

12.4.2 Normalized incomplete beta function, $I_x(a, b)$

`ctx.ibeta`(a, b, x)

where `ctx` is `math53`, `ctxboost`, `ctxflint`.

Returns the normalized incomplete beta function $I_x(a,b) = \dfrac{1}{B(a,b)} \displaystyle\int_0^x t^{a-1}(1-t)^{b-1}\,\mathrm{d}t,$ for $a > 0$, $b > 0$, and $0 \le x \le 1$.

See also Wikipedia [1222], MathWorld [854], NIST [458], BoostMath [79], BoostMath [87], Ehrhardt [297] (3.5.3.3), Flint [741], Flint [731].

Returns the normalised incomplete beta function $I_x(a,b)$ for $a > 0$, $b > 0$, and $0 \le x \le 1$.

$$I_x(a,b) = \frac{B_x(a,b)}{B(a,b)}, \quad B_x(a,b) = \int_0^x t^{a-1}(1-t)^{b-1}\mathrm{d}t.$$

There are some special cases

$$I_0(a,b) = 0, \quad I_1(a,b) = 1, \quad I_x(a,1) = x^a,$$

and the symmetry relation $I_x(a,b) = 1 - I_{1-x}(b,a)$, which is used for $x > a/(a+b)$.

Computes the (lower) incomplete beta function, defined by $B(a,b;z) = \int_0^z t^{a-1}(1-t)^{b-1}$, optionally the regularized incomplete beta function $I(a,b;z) = B(a,b;z)/B(a,b;1)$.

In general, the integral must be interpreted using analytic continuation. The precise definitions for all parameter values are

$$B(a,b;z) = \frac{z^a}{a}{}_2F_1(a, 1-b, a+1, z)$$

$$I(a,b;z) = \frac{\Gamma(a+b)}{\Gamma(b)} z^a {}_2\widetilde{F}_1(a, 1-b, a+1, z).$$

Note that both functions with this definition are undefined for nonpositive integer a, and I is undefined for nonpositive integer $a+b$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.IBeta(3.1, 0.5, 0.3)
ereal('5.2359877559829887307E-1')
>>> math53.IBeta(3.4, '0.51', 0.3)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = '10'; b = '7'; x = '0.1'
>>> \mathrm{d}x = dec.ibeta(a, b, x); mx = mpm.ibeta(a, b, x); gx = gmp.ibeta(a, b, x)
>>> fx = fpm.ibeta(a, b, x); ax = apm.ibeta(a, b, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 4.526064685000000000000000000000000000000E-7
mpm: 4.526064685000000000000000000000000000000e-7
gmp: 4.526064685000000000000000000000000000000E-07
fpm: 4.52606468500001E-07
apm: 4.526064685000000000000000000000000000000e-7 (9.07e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '10 + 0j'; b = '7 + 1j'; z = '0.1 + 03j'
>>> \mathrm{d}z = dec.ibeta(a, b, z); mz = mpm.ibeta(a, b, z); gz = gmp.ibeta(a, b, z)
>>> fz = fpm.ibeta(a, b, z); az = apm.ibeta(a, b, z)
```

(continues on next page)

(continued from previous page)

```
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -9.8493766176109694014E+11                - 4.27248355641058190 32E+11j
mpm: -9.8493766176109694014e+11                - 4.27248355641058190 32e+11j
gmp: -9.8493766176109694014E+11                - 4.27248355641058190 32E+11j
fpm: -9.84937661761097E+11                     - 4.27248355641058E+11j
apm: -9.8493766176109694013e+11 (-4.728e-19%) - 4.27248355641058190 32e+11 (-1.199e-18%)j
```

12.4.3 Derivative of the incomplete beta function

`ctx.`**`ibeta_prime`**(a, b, x)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the partial derivative with respect to x of the incomplete beta function $I_x(a, b)$. See also BoostMath [72]. The function is defined as:

$$\frac{\partial}{\partial x} I_x(a, b) = \frac{x^{a-1}(1-x)^{b-1}}{B(a, b)}.$$

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; a = '8.3'; b = '10.4'; x = '0.7'
>>> \mathrm{d}x = dec.beta_derivative(a, b, x); mx = mpm.beta_derivative(a, b, x)
>>> ix = ipm.beta_derivative(a, b, x); fx = fpm.beta_derivative(a, b, x)
>>> gx = gmp.beta_derivative(a, b, x); ax = apm.beta_derivative(a, b, x)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  2.878999879544799566557091075303033672075E-1
mpm:  2.878999879544799566557091075303033672075e-1
ipm:  2.878999879544799566557091075303033672074e-1 (1.396e-37%)
fpm:  2.87899987954481E-01
gmp:  2.878999879544799566557091075303033672076E-01
apm:  2.878999879544799566557091075303033672074e-1 (3.01e-37%)
```

12.4.4 Real non-normalized upper incomplete beta function, $B_{\text{upper}}(a, b; x)$

`ctx.beta_upper`(a, b, x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Note: ctx.betac(a, b, x), ctxboost.IBetacNonNormalized(a, b, x)

Returns the non-normalized incomplete beta function $B_{1-x}(a, b) = B_{\text{upper}}(a, b; x) = \int_x^1 t^{a-1}(1-t)^{b-1}\,\mathrm{d}t$, for $a > 0$, $b > 0$, and $0 \leq x \leq 1$.

This can be generalized to complex a, b and x as $B_{\text{upper}}(a, b; x) = B(a, b) - B_x(a, b)$.

See also Wikipedia [1222], MathWorld [847], NIST [458], BoostMath [79], BoostMath [87].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Betac(3.1, 0.5, 0.3)
ereal('5.2359877559829887307E-1')
>>> math53.Betac(3.4, '0.51', 0.3)
ereal('5.351847902755998475 4E-1')
```

12.4.5 Real normalized complementory incomplete beta function, $I_{1-x}(a, b)$

`ctx.ibetac`(a, b, x)

where `ctx` is `math53`, `ctxboost` or `ctxflint`.

Returns the complement of the normalized incomplete beta function $1 - I_x(a, b) = \dfrac{1}{B(a, b)} \displaystyle\int_x^1 t^{a-1}(1-t)^{b-1}\,\mathrm{d}t$, for $a > 0$, $b > 0$, and $0 \le x \le 1$.

See also Wikipedia [1222], MathWorld [854], NIST [458], BoostMath [79], BoostMath [87], Flint [741], Flint [731].

Returns the non-normalised incomplete beta function $I_x(a, b)$ for $a > 0$, $b > 0$, and $0 \le x \le 1$:

$$I_x(a, b) = \frac{B_x(a, b)}{B(a, b)}, \quad B_x(a, b) = \int_0^x t^{a-1}(1-t)^{b-1}\mathrm{d}t.$$

$$B_x(a, b) = \int_0^x t^{a-1}(1-t)^{b-1}\mathrm{d}t.$$

There are some special cases

$$B_0(a, b) = 0, \quad B_1(a, b) = B(a, b), \quad B_x(a, 1) = \frac{x^a}{a}, \quad B_x(1, b) = \frac{1-(1-x)^b}{b},$$

and the relation $B_{1-x}(a, b) = B(a, b) - B_x(b, a)$, which is used if $x > a/(a + b)$. When $a \le 0$ or $b \le 0$, the Gauss hypergeometric function ${}_2F_2(\cdot)$ is applied: If $a \ne 0$ is not a negative integer, the result is

$$B_x(a, b) = \frac{x^a}{a}{}_2F_2(a, 1-b, a+1, x), \quad -a \notin \mathbb{N}$$

else if $b \ne 0$ is not a negative integer, the result is

$$B_x(a, b) = B(a, b) - \frac{(1-x)^b x^a}{b}{}_2F_2(1, a+b, b+1, 1-x), \quad -b \notin \mathbb{N}.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Ibetac(3.1, 0.5, 0.3)
ereal('5.2359877559829887307E-1')
>>> math53.Ibetac(3.4, '0.51', 0.3)
ereal('5.3518479027559984754E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a = '10'; b = '7'; x = '0.1'
>>> \mathrm{d}x = dec.beta3(a, b, x); mx = mpm.beta3(a, b, x); gx = gmp.beta3(a, b, x)
>>> fx = fpm.beta3(a, b, x); ax = apm.beta3(a, b, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 5.651928927322677322677322677322677322677E-12
mpm: 5.651928927322677322677322677322677322677e-12
gmp: 5.651928927322677322677322677322677322677E-12
fpm: 5.65192892732268E-12
apm: 5.651928927322677322677322677322677322677e-12 (8.867e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a = '10 + 0j'; b = '7 + 1j'; z = '0.1 + 03j'
>>> \mathrm{d}z = dec.beta3(a, b, z); mz = mpm.beta3(a, b, z); gz = gmp.beta3(a, b, z)
>>> fz = fpm.beta3(a, b, z); az = apm.beta3(a, b, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
```

(continues on next page)

(continued from previous page)

```
dec: -1.1121024734248714310E+7               + 6.3403283967816369421E+6j
mpm: -1.1121024734248714310e+7               + 6.3403283967816369421e+6j
gmp: -1.1121024734248714310E+07              + 6.3403283967816369421E+06j
fpm: -1.11210247342487E+07                   + 6.34032839678164E+06j
apm: -1.1121024734248714310e+7 (-4.472e-19%) + 6.3403283967816369421e+6 (5.603e-19%)j
```

12.4.6 Inverse of the real normalised incomplete beta function

`ctx.`**`ibeta_inv`**(a, b, q)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the inverse of the normalised incomplete beta function. Calculates x with $Q(a, x) = p$. The input parameters are $a > 0, b > 0, p \geq 0$.

See also Wikipedia [1222], MathWorld [854], NIST [458], BoostMath [79], BoostMath [87], Ehrhardt [297] (3.5.3.5).

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; a = '8.3'; b = '10.4'; prob = '0.7'
>>> \mathrm{d}x = dec.ibeta_inv(a, b, prob); mx = mpm.ibeta_inv(a, b, prob)
>>> ix = ipm.ibeta_inv(a, b, prob); fx = fpm.ibeta_inv(a, b, prob)
>>> gx = gmp.ibeta_inv(a, b, prob); ax = apm.ibeta_inv(a, b, prob)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  5.031911971011064339721270139988401063680E-1
mpm:  5.031911971011064339721270139988401063680e-1
ipm:  5.031911971011064339721270139988401063680e-1 (1.141e-39%)
fpm:  5.03191197101107E-01
gmp:  5.031911971011064339721270139988401063680E-01
apm:  5.031911971011064339721270139988401063680e-1 (1.141e-39%)

>>> fx = fpm.ibeta_inv(a, b, prob); mpm.show([fx]) # boost
fpm:  5.03191197101107E-01
>>> fx = fpm.ibeta(a, b, fx); mpm.show([fx]) # boost
fpm:  7.00000000000000E-01
```

12.4.7 Inverse of the real normalised complementary incomplete beta function

`ctx.`**`ibetac_inv`**(a, b, q)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the functional inverse of the complement of the upper normalized incomplete beta function. Calculates x with $I_{1-x}(a, b) = p$. The input parameters are $a > 0, b > 0, p \geq 0, \leq 1$.

See also BoostMath [87], Wikipedia [1222], MathWorld [257], NIST [458].

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; a = '8.3'; b = '10.4'; prob = '0.7'
>>> \mathrm{d}x = dec.ibetac_inv(a, b, prob); mx = mpm.ibetac_inv(a, b, prob)
>>> ix = ipm.ibetac_inv(a, b, prob); fx = fpm.ibetac_inv(a, b, prob)
>>> gx = gmp.ibetac_inv(a, b, prob); ax = apm.ibetac_inv(a, b, prob)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  3.81597461556170969245992999159160038768 8E-1
mpm:  3.81597461556170969245992999159160038768 8e-1
ipm:  3.81597461556170969245992999159160038768 8e-1 (7.521e-40%)
fpm:  3.81597461556171E-01
gmp:  3.81597461556170969245992999159160038768 8E-01
apm:  3.81597461556170969245992999159160038768 8e-1 (7.521e-40%)

>>> fx = fpm.ibetac_inv(a, b, prob); mpm.show([fx]) # boost
fpm:  3.81597461556171E-01
>>> fx = fpm.ibetac(a, b, fx); mpm.show([fx]) # boost
fpm:  7.00000000000000E-01
```

12.4.8 Inverse (on parameter a) of the real normalised incomplete beta function

`ctx.`**`ibeta_inva`**(b, x, q)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the inverse of the normalized incomplete beta function with regard to parameter a, i.e. calculates a with $I_x(a, b) = p$. The input parameters are $x > 0$, $b > 0$, $p \geq 0, \leq 1$.

See also BoostMath [87], Wikipedia [1222], MathWorld [257], NIST [458].

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.3'; b = '10.4'; prob = '0.7'
>>> \mathrm{d}x = dec.ibeta_inva(b, x, prob); mx = mpm.ibeta_inva(b, x, prob)
>>> ix = ipm.ibeta_inva(b, x, prob); fx = fpm.ibeta_inva(b, x, prob)
>>> gx = gmp.ibeta_inva(b, x, prob); ax = apm.ibeta_inva(b, x, prob)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  3.434764629588725033018175504012263060447E+0
mpm:  3.434764629588725033018175504012263060447e+0
ipm:  3.434764629588725033018175504012263060447e+0 (6.684e-40%)
fpm:  3.43476462958872E+00
gmp:  3.434764629588725033018175504012263060447E+00
apm:  3.434764629588725033018175504012263060447e+0 (6.684e-40%)

>>> fxa = fpm.ibeta_inva(b, x, prob); mpm.show([fxa]) # boost
fpm:  3.43476462958872E+00
>>> fx = fpm.ibeta(fxa, b, x); mpm.show([fx]) # boost
fpm:  7.00000000000000E-01
```

12.4.9 Inverse (on parameter a) of the real normalised complementary incomplete beta function

`ctx.ibetac_inva`(b, x, q)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the inverse of the complement of the normalized incomplete beta function with regard to parameter a, i.e. calculates a with $I_{1-x}(a, b) = p$. The input parameters are $x > 0$, $b > 0$, $p \geq 0, \leq 1$.

See also BoostMath [87], Wikipedia [1222], MathWorld [257], NIST [458].

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.3'; b = '10.4'; prob = '0.7'
>>> \mathrm{d}x = dec.ibetac_inva(b, x, prob); mx = mpm.ibetac_inva(b, x, prob)
>>> ix = ipm.ibetac_inva(b, x, prob); fx = fpm.ibetac_inva(b, x, prob)
>>> gx = gmp.ibetac_inva(b, x, prob); ax = apm.ibetac_inva(b, x, prob)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  6.022956553898772220485519886468373977550E+0
mpm:  6.022956553898772220485519886468373977550e+0
ipm:  6.022956553898772220485519886468373977550e+0 (7.624e-40%)
fpm:  6.02295655389877E+00
gmp:  6.022956553898772220485519886468373977550E+00
apm:  6.022956553898772220485519886468373977550e+0 (7.624e-40%)

>>> fxa = fpm.ibetac_inva(b, x, prob); mpm.show([fxa]) # boost
fpm:  6.02295655389877E+00
>>> fx = fpm.ibetac(fxa, b, x); mpm.show([fx]) # boost
fpm:  7.00000000000000E-01
```

12.4.10 Inverse (on parameter b) of the real normalised incomplete beta function

`ctx.`**`ibeta_invb`**(a, x, q)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the inverse of the normalised incomplete beta function with regard to parameter b, i.e. calculates a with $Q(a, x) = p$. The input parameters are $a > 0$, $b > 0$, $p \geq 0$.

See also BoostMath [87], Wikipedia [1222], MathWorld [257], NIST [458].

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.3'; a = '7.8'; prob = '0.7'
>>> \mathrm{d}x = dec.ibeta_invb(a, x, prob); mx = mpm.ibeta_invb(a, x, prob)
>>> ix = ipm.ibeta_invb(a, x, prob); fx = fpm.ibeta_invb(a, x, prob)
>>> gx = gmp.ibeta_invb(a, x, prob); ax = apm.ibeta_invb(a, x, prob)
>>> mpm.show([\mathrm{d}x, mx, ix, fx, gx, ax])
dec:  2.2004295916555214543537953946491293468 38E+1
mpm:  2.2004295916555214543537953946491293468 38e+1
ipm:  2.2004295916555214543537953946491293468 38e+1 (8.347e-40%)
fpm:  2.20042959165552E+01
gmp:  2.2004295916555214543537953946491293468 38E+01
apm:  2.2004295916555214543537953946491293468 38e+1 (8.347e-40%)

>>> fxb = fpm.ibeta_invb(a, x, prob); mpm.show([fxb]) # boost
fpm:  2.20042959165552E+01
>>> fx = fpm.ibeta(a, fxb, x); mpm.show([fx]) # boost
fpm:  7.00000000000000E-01
```

12.4.11 Inverse (on parameter b) of the real normalised complementary incomplete beta function

`ctx.ibetac_invb`(a, x, q)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the inverse of the complement of the normalized incomplete beta function with regard to parameter b, i.e. calculates b with $I_{1-x}(a, b) = p$. The input parameters are $a > 0$, $x > 0$, $p \geq 0, \leq 1$.

See also BoostMath [87], Wikipedia [1222], MathWorld [257], NIST [458].

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.3'; a = '7.8'; prob = '0.7'
>>> \mathrm{d}x = dec.ibetac_invb(a, x, prob); mx = mpm.ibetac_invb(a, x, prob)
>>> ix = ipm.ibetac_invb(a, x, prob); fx = fpm.ibetac_invb(a, x, prob)
>>> gx = gmp.ibetac_invb(a, x, prob); ax = apm.ibetac_invb(a, x, prob)
>>> mpm.show([gx, fx, ax])
dec:  1.4035769060760995572408050341049104057 88E+1
mpm:  1.4035769060760995572408050341049104057 88e+1
ipm:  1.4035769060760995572408050341049104057 88e+1 (6.543e-40%)
fpm:  1.40357690607610E+01
gmp:  1.4035769060760995572408050341049104057 88E+01
apm:  1.4035769060760995572408050341049104057 88e+1 (6.543e-40%)

>>> fxb = fpm.ibetac_invb(a, x, prob); mpm.show([fxb]) # boost
fpm:  1.40357690607610E+01
>>> fx = fpm.ibetac(a, fxb, x); mpm.show([fx]) # boost
fpm:  6.99999999999999E-01
```

12.5 Hypergeometric function ${}_1F_2$

12.5.1 Non-regularized hypergeometric function ${}_1F_2$

`ctx.hyperg_1f2(`*a*, *b1*, *b2*, *z*`)`

Returns the generalized hypergeometric function ${}_1F_2$.

Returns ${}_1F_2(a_1;b_1,b_2;x) = \sum_{k=0}^{\infty} \frac{(a_1)_k}{(b_1)_k(b_2)_k} \frac{x^k}{k!}$

We also have ${}_1F_2(a_1;b_1,b_2;x) = \frac{\Gamma(b_2)}{\Gamma(a_1)\Gamma(b_2-a_1)} \int_0^1 (1-t)^{b_2-a_1-1} t^{a_1-1} {}_0F_1(b_1,tx)\,\mathrm{d}t$, where $\Re(b_2) > \Re(a_1) > 0$.

See also MathWorld [1031], MathWorld [286], Wikipedia [1376], NIST [24], Nijimbere [423], Tarasov [514], Mpmath [716].

See also: https://functions.wolfram.com/HypergeometricFunctions/Hypergeometric1F2/07/01/01/

See also: https://functions.wolfram.com/HypergeometricFunctions/Hypergeometric1F2/17/01/01/

See also: https://functions.wolfram.com/HypergeometricFunctions/Hypergeometric1F2/06/02/03/

See also: https://functions.wolfram.com/HypergeometricFunctions/Hypergeometric1F2/25/01/

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a1 = '11.0'; b1 = '12.0'; b2 = '32.0'; x = '0.3'
>>> \mathrm{d}x = dec.hyp1f2(a1, b1, b2, x); mx = mpm.hyp1f2(a1, b1, b2, x); gx = gmp.hyp1f2(a1, b1, b2, x)
>>> fx = fpm.hyp1f2(a1, b1, b2, x); ax = apm.hyp1f2(a1, b1, b2, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.0086299063664603622263058671178027837 39E+0
mpm:  1.0086299063664603622263058671178027837 39e+0
gmp:  1.0086299063664603622263058671178027837 39E+00
fpm:  1.00862990636646E+00
apm:  1.0086299063664603622263058671178027837 39e+0 (1.138e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a1 = '11.0 + 2.0j'; b1 = '12.0 + 3.0j'; b2 = '42.0 + 3.0j';z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.hyp1f2(a1, b1, b2, z); mz = mpm.hyp1f2(a1, b1, b2, z); gz = gmp.hyp1f2(a1, b1, b2, z)
>>> fz = fpm.hyp1f2(a1, b1, b2, z); az = apm.hyp1f2(a1, b1, b2, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 1.0752386838476564793E+0              + 8.2118112439611053461E-2j
mpm: 1.0752386838476564793e+0              + 8.2118112439611053461e-2j
gmp: 1.0752386838476564793E+00             + 8.2118112439611053461E-02j
fpm: 1.07523868384766E+00                  + 8.21181124396110E-02j
apm: 1.0752386838476564793e+0 (7.878e-20%) + 8.2118112439611053461e-2 (6.447e-20%)j
```

12.5.2 Regularized hypergeometric function ${}_1\widetilde{F}_2$

`ctx.`**`hyperg_1f2r`**$(a, b1, b2, z)$

Returns the regularized generalized hypergeometric function ${}_1\widetilde{F}_2(a, b1, b2; z)$.

See also MathWorld [1031], MathWorld [286], Wikipedia [1376], NIST [24], Nijimbere [423], Tarasov [514], Mpmath [716].

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a1 = '11.0'; b1 = '12.0'; b2 = '32.0'; x = '0.3'
>>> \mathrm{d}x = dec.hyp1f2r(a1, b1, b2, x); mx = mpm.hyp1f2r(a1, b1, b2, x); gx = gmp.hyp1f2r(a1, b1, b2, x)
>>> fx = fpm.hyp1f2r(a1, b1, b2, x); ax = apm.hyp1f2r(a1, b1, b2, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  3.072941936207392693664766625229320203604E-42
mpm:  3.072941936207392693664766625229320203604e-42
gmp:  3.072941936207392693664766625229320203604E-42
fpm:  3.07294193620739E-42
apm:  3.072941936207392693664766625229320203604e-42 (1.072e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a1 = '11.0 + 2.0j'; b1 = '12.0 + 3.0j'; b2 = '42.0 + 3.0j';z = '3.0 + 4.0j'
>>> \mathrm{d}z = dec.hyp1f2r(a1, b1, b2, z); mz = mpm.hyp1f2r(a1, b1, b2, z); gz = gmp.hyp1f2r(a1, b1, b2, z)
>>> fz = fpm.hyp1f2r(a1, b1, b2, z); az = apm.hyp1f2r(a1, b1, b2, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: 1.2280824173267661851E-57              + 4.9727461410239566810E-58j
mpm: 1.2280824173267661851e-57              + 4.9727461410239566810e-58j
gmp: 1.2280824173267661851E-57              + 4.9727461410239566810E-58j
fpm: 1.22808241732677E-57                   + 4.97274614102396E-58j
apm: 1.2280824173267661851e-57 (1.319e-19%) + 4.9727461410239566810e-58 (1.085e-19%)j
```

12.6 Scorer functions

12.6.1 Scorer function $\mathrm{Gi}(x)$

ctx.scorer_gi(x)

Returns the Scorer function Gi, which gives a particular solution to the inhomogeneous Airy differential equation $f''(x) - xf(x) = 1/\pi$.

See also: MathWorld [1036], Wikipedia [1390], Ehrhardt [297] (3.1.7.7), NIST [438], Mpmath [722].

The Scorer function Gi is defined as

$$\mathrm{Gi}(x) = \frac{1}{\pi}\int_0^\infty \sin\left(xt - \frac{1}{3}t^3\right)\mathrm{d}t.$$

Another particular solution is given by the Scorer Hi-function. The two functions are related as $\mathrm{Gi}(z) + \mathrm{Hi}(z) = \mathrm{Bi}(z)$.

We also have

$$\mathrm{Gi}(z) = \tfrac{1}{3}\mathrm{Bi}(z) - \frac{z^2}{2\pi}\,{}_1F_2\left(1; \tfrac{4}{3}, \tfrac{5}{3}; \tfrac{1}{9}z^3\right)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.AiryGi(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.AiryGi('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = -3
>>> \mathrm{d}x = dec.scorergi(x); mx = mpm.scorergi(x); gx = gmp.scorergi(x)
>>> fx = fpm.scorergi(x); ax = apm.scorergi(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -2.9905471837139642380158189762383534741 93E-1
mpm: -2.9905471837139642380158189762383534741 93e-1
gmp: -2.9905471837139642380158189762383534741 93E-01
fpm: -2.99054718371396E-01
apm: -2.9905471837139642380158189762383534741 92e-1 (-4.827e-37%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.scorergi(z); mz = mpm.scorergi(z); gz = gmp.scorergi(z)
>>> fz = fpm.scorergi(z); az = apm.scorergi(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 8.4889448371208839089E-2             - 3.5862043476779870404E-2j
mpm: 8.4889448371208839089e-2             - 3.5862043476779870404e-2j
gmp: 8.4889448371208839089E-02            - 3.5862043476779870404E-02j
fpm: 8.48894483712088E-02                 - 3.58620434767799E-02j
apm: 8.4889448371208839123e-2 (6.78e-15%) - 3.5862043476779870473e-2 (-1.391e-14%)j
```

12.6.2 Scorer function $\mathrm{Hi}(x)$

`ctx.scorer_hi(x)`

Returns the Scorer function Hi, which gives a particular solution to the inhomogeneous Airy differential equation $f''(x) - xf(x) = 1/\pi$.

See also: MathWorld [1037], Wikipedia [1390], Ehrhardt [297] (3.1.7.8), Mpmath [723].

Returns the Scorer function Hi, defined as

$$\mathrm{Hi}(x) = \frac{1}{\pi}\int_0^\infty \exp\left(xt - \frac{1}{3}t^3\right) \mathrm{d}t$$

We also have

$$\mathrm{Hi}(z) = \tfrac{2}{3}\mathrm{Bi}(z) + \frac{z^2}{2\pi}\,{}_1F_2\left(1; \tfrac{4}{3}, \tfrac{5}{3}; \tfrac{1}{9}z^3\right)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.AiryHi(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.AiryHi('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = -3
>>> \mathrm{d}x = dec.scorerhi(x); mx = mpm.scorerhi(x); gx = gmp.scorerhi(x)
>>> fx = fpm.scorerhi(x); ax = apm.scorerhi(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: 1.0076509199646988058093704305134699072 05E-1
mpm: 1.0076509199646988058093704305134699072 05e-1
gmp: 1.0076509199646988058093704305134699072 05E-01
fpm: 1.00765091996470E-01
apm: 1.0076509199646988058093704305134699064 31e-1 (1.334e-33%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.scorerhi(z); mz = mpm.scorerhi(z); gz = gmp.scorerhi(z)
>>> fz = fpm.scorerhi(z); az = apm.scorerhi(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 9.5150834628338203598E-1              + 1.0872383260084919901E+0j
mpm: 9.5150834628338203598e-1              + 1.0872383260084919901e+0j
gmp: 9.5150834628338203598E-01             + 1.0872383260084919901E+00j
fpm: 9.51508346283382E-01                  + 1.08723832600849E+00j
apm: 9.5150834628338203594e-1 (6.049e-16%) + 1.0872383260084919901e+0 (4.588e-16%)j
```

12.6.3 First derivative of the Scorer function $\mathrm{Gi}(x)$, $\mathrm{Gi}'(x)$

`ctx.scorer_gi_prime(x)`

Returns $\mathrm{Gi}'(x)$, the first derivative of the Airy (Scorer) function $\mathrm{Gi}(x)$.

See also: MathWorld [1036], Wikipedia [1390], Ehrhardt [297] (3.1.7.7), NIST [438], Mpmath [722].

The function is calculated as

$$\mathrm{Gi}'(z) = \tfrac{1}{3}\mathrm{Bi}'(z) - \frac{1}{40\pi}\left[40z \cdot {}_1F_2\left(1; \tfrac{4}{3}, \tfrac{5}{3}; \tfrac{1}{9}z^3\right) + 3z^4 \cdot {}_1F_2\left(2; \tfrac{7}{3}, \tfrac{8}{3}; \tfrac{1}{9}z^3\right)\right]$$

We also have $\mathrm{Gi}'(x) = \mathrm{Bi}'(x) - \mathrm{Hi}'(x)$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.AiryGi(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.AiryGi('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = -3
>>> \mathrm{d}x = dec.scorergi(x); mx = mpm.scorergi(x); gx = gmp.scorergi(x)
>>> fx = fpm.scorergi(x); ax = apm.scorergi(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -2.9905471837139642380158189762383534741 93E-1
mpm: -2.9905471837139642380158189762383534741 93e-1
gmp: -2.9905471837139642380158189762383534741 93E-01
fpm: -2.99054718371396E-01
apm: -2.9905471837139642380158189762383534741 92e-1 (-4.827e-37%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.scorergi(z); mz = mpm.scorergi(z); gz = gmp.scorergi(z)
>>> fz = fpm.scorergi(z); az = apm.scorergi(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 8.4889448371208839089E-2              - 3.5862043476779870404E-2j
mpm: 8.4889448371208839089e-2              - 3.5862043476779870404e-2j
gmp: 8.4889448371208839089E-02             - 3.5862043476779870404E-02j
fpm: 8.48894483712088E-02                  - 3.58620434767799E-02j
apm: 8.4889448371208839123e-2 (6.78e-15%) - 3.5862043476779870473e-2 (-1.391e-14%)j
```

12.6.4 First derivative of the Scorer function $\mathrm{Hi}(x)$, $\mathrm{Hi}'(x)$

`ctx.scorer_hi_prime(x)`

Returns $\mathrm{Hi}'(x)$, the first derivative of the Airy (Scorer) function $\mathrm{Hi}(x)$.

See also: MathWorld [1037], Wikipedia [1390], Ehrhardt [297] (3.1.7.8), Mpmath [723].

The function is calculated as

$$\mathrm{Hi}'(z) = \tfrac{2}{3}\mathrm{Bi}'(z) + \frac{1}{40\pi}\left[40z \cdot {}_1F_2\left(1; \tfrac{4}{3}, \tfrac{5}{3}; \tfrac{1}{9}z^3\right) + 3z^4 \cdot {}_1F_2\left(2; \tfrac{7}{3}, \tfrac{8}{3}; \tfrac{1}{9}z^3\right)\right]$$

It can also be represented as an integral:

$$\mathrm{Hi}'(x) = \frac{1}{\pi}\int_0^\infty t \exp\left(xt - \tfrac{1}{3}t^3\right) \mathrm{d}t$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.AiryHi(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.AiryHi('0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; x = -3
>>> \mathrm{d}x = dec.scorerhi(x); mx = mpm.scorerhi(x); gx = gmp.scorerhi(x)
>>> fx = fpm.scorerhi(x); ax = apm.scorerhi(x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: 1.0076509199646988058093704305134699072 05E-1
mpm: 1.0076509199646988058093704305134699072 05e-1
gmp: 1.0076509199646988058093704305134699072 05E-01
fpm: 1.00765091996470E-01
apm: 1.0076509199646988058093704305134699069 31e-1 (1.334e-33%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '3 + 4j'
>>> \mathrm{d}z = dec.scorerhi(z); mz = mpm.scorerhi(z); gz = gmp.scorerhi(z)
>>> fz = fpm.scorerhi(z); az = apm.scorerhi(z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 9.5150834628338203598E-1              + 1.0872383260084919901E+0j
mpm: 9.5150834628338203598e-1              + 1.0872383260084919901e+0j
gmp: 9.5150834628338203598E-01             + 1.0872383260084919901E+00j
fpm: 9.51508346283382E-01                  + 1.08723832600849E+00j
apm: 9.5150834628338203594e-1 (6.049e-16%) + 1.0872383260084919901e+0 (4.588e-16%)j
```

12.7 Struve functions

12.7.1 Struve function $\mathrm{H}_n(x)$

ctx.struve_h(*nu*, *x*)

Returns the Struve function $H_\nu(x)$. See also Wikipedia [1391], MathWorld [1026], NIST [461], Ehrhardt [297] (3.1.9.3), Mpmath [712].

$$\mathbf{H}_n(x) = \sum_{k=0}^{\infty} \frac{(-1)^k}{\Gamma(k+\frac{3}{2})\Gamma(k+n+\frac{3}{2})} \left(\frac{z}{2}\right)^{2k+n+1}$$

which is a solution to the Struve differential equation

$$z^2 f''(z) + z f'(z) + (z^2 - n^2) f(z) = \frac{2z^{n+1}}{\pi(2n-1)!!}.$$

We also have

$$\mathbf{H}_\nu(x) = \left(\frac{z}{2}\right)^{\nu+1} {}_1\widetilde{F}_2\left(1; \frac{3}{2}, \nu+\frac{3}{2}; -\frac{z^2}{4}\right)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.StruveH(3, 0.5)
ereal('5.235987755982988730E-1')
>>> math53.StruveH(3, '0.51')
ereal('5.351847902755998475E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 10; x = -3
>>> \mathrm{d}x = dec.struveh(n, x); mx = mpm.struveh(n, x); gx = gmp.struveh(n, x)
>>> fx = fpm.struveh(n, x); ax = apm.struveh(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -7.2058762694527534388921007769900586494 03E-6
mpm: -7.2058762694527534388921007769900586494 03e-6
gmp: -7.2058762694527534388921007769900586494 03E-06
fpm: -7.20587626945275E-06
apm: -7.2058762694527534388921007769900586494 03e-6 (-1.215e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10 + 1j'; z = '3 + 4j'
>>> \mathrm{d}z = dec.struveh(n, z); mz = mpm.struveh(n, z); gz = gmp.struveh(n, z)
>>> fz = fpm.struveh(n, z); az = apm.struveh(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -4.8345130788092493588E-4                + 8.7404070391062002932E-4j
mpm: -4.8345130788092493588e-4                + 8.7404070391062002932e-4j
gmp: -4.8345130788092493588E-04               + 8.7404070391062002932E-04j
fpm: -4.83451307880925E-04                    + 8.74040703910620E-04j
apm: -4.8345130788092493588e-4 (-2.994e-19%) + 8.7404070391062002932e-4 (2.366e-19%)j
```

12.7.2 Struve function $\mathrm{L}_\nu(x)$

`ctx.struve_l(nu, x)`

Returns the Struve function L. See also Wikipedia [1391], MathWorld [1025], NIST [461], Ehrhardt [297] (3.1.9.4), Mpmath [713].

Returns the Struve function $\mathbf{L}_\nu(x)$, defined as

$$\mathbf{L}_\nu(x) = \left(\tfrac{1}{2}x\right)^{\nu+1} \sum_{k=0}^{\infty} \frac{\left(\frac{1}{2}x\right)^{2k}}{\Gamma\left(k+\frac{3}{2}\right)\Gamma\left(k+\nu+\frac{3}{2}\right)}.$$

which solves to the modified Struve differential equation

$$z^2 f''(z) + z f'(z) - (z^2+n^2) f(z) = \frac{2z^{n+1}}{\pi(2n-1)!!}.$$

We also have

$$\mathbf{L}_n(z) = -ie^{-n\pi i/2}\mathbf{H}_n(iz)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.StruveL(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.StruveL(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 10; x = -3
>>> \mathrm{d}x = dec.struvel(n, x); mx = mpm.struvel(n, x); gx = gmp.struvel(n, x)
>>> fx = fpm.struvel(n, x); ax = apm.struvel(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -9.3529365166294384085698625084370213559 81E-6
mpm: -9.3529365166294384085698625084370213559 81e-6
gmp: -9.3529365166294384085698625084370213559 81E-06
fpm: -9.35293651662944E-06
apm: -9.3529365166294384085698625084370213524 70e-6 (-5.114e-35%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; z = '3 + 4j'
>>> \mathrm{d}z = dec.struvel(n, z); mz = mpm.struvel(n, z); gz = gmp.struvel(n, z)
>>> fz = fpm.struvel(n, z); az = apm.struvel(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -8.8399682322365238730E-4                - 1.8271632883005884894E-3j
mpm: -8.8399682322365238730e-4                - 1.8271632883005884894e-3j
gmp: -8.8399682322365238730E-04               - 1.8271632883005884894E-03j
fpm: -8.83996823223652E-04                    - 1.82716328830059E-03j
apm: -8.8399682322365238730e-4 (-1.871e-19%) - 1.8271632883005884894e-3 (-9.054e-20%)j
```

12.7.3 Struve function $\mathrm{K}_\nu(x)$

`ctx.struve_k(nu, z)`

Returns the Struve function K(nu, z) = H(nu,z) - Y(n, z). See also Wikipedia [1391], MathWorld [1025], NIST [461].

$$\mathbf{K}_\nu\left(z\right) = \mathbf{H}_\nu\left(z\right) - Y_\nu\left(z\right)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.StruveK(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.StruveK(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 10; x = 3
>>> \mathrm{d}x = dec.struvek(n, x); mx = mpm.struvek(n, x); gx = gmp.struvek(n, x)
>>> fx = fpm.struvek(n, x); ax = apm.struvek(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: 2.582607136690175938562877297993741088665E+3
mpm: 2.582607136690175938562877297993741088665e+3
gmp: 2.582607136690175938562877297993741088665E+03
fpm: 2.58260713669018E+03
apm: 2.582607136690175938562877297993741088665e+3 (1.274e-38%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; z = '3 + 4j'
>>> \mathrm{d}z = dec.struvek(n, z); mz = mpm.struvek(n, z); gz = gmp.struvek(n, z)
>>> fz = fpm.struvek(n, z); az = apm.struvek(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -6.7918399227722424068E+0               - 6.9970201805301178844E+0j
mpm: -6.7918399227722424068e+0               - 6.9970201805301178844e+0j
gmp: -6.7918399227722424068E+00              - 6.9970201805301178844E+00j
fpm: -6.79183992277224E+00                   - 6.99702018053012E+00j
apm: -6.7918399227722424068e+0 (-2.145e-18%) - 6.9970201805301178844e+0 (-2.131e-18%)j
```

12.7.4 Struve function $\mathrm{M}_\nu(x)$

`ctx.struve_m`(*nu*, *z*)

Returns the Struve function M(nu, z) = L(nu,z) - I(n, z).. See also Wikipedia [1391], MathWorld [1025], NIST [461].

$$\mathbf{M}_\nu\left(z\right) = \mathbf{L}_\nu\left(z\right) - I_\nu\left(z\right).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.StruveM(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.StruveM(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 10; x = 3
>>> \mathrm{d}x = dec.struvem(n, x); mx = mpm.struvem(n, x); gx = gmp.struvem(n, x)
>>> fx = fpm.struvem(n, x); ax = apm.struvem(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -1.011145695398353025999970444305917861401E-5
mpm: -1.011145695398353025999970444305917861401e-5
gmp: -1.011145695398353025999970444305917861401E-05
fpm: -1.01114569539835E-05
apm: -1.011145695398353025999970444305917861401e-5 (-7.795e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; z = '3 + 4j'
>>> \mathrm{d}z = dec.struvem(n, z); mz = mpm.struvem(n, z); gz = gmp.struvem(n, z)
>>> fz = fpm.struvem(n, z); az = apm.struvem(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.20505077798444167 2E-3              - 9.432139379116939391 6E-4j
mpm: 1.20505077798444167 2e-3              - 9.432139379116939391 6e-4j
gmp: 1.20505077798444167 2E-03             - 9.432139379116939391 6E-04j
fpm: 1.20505077798444E-03                  - 9.43213937911694E-04j
apm: 1.20505077798444167 2e-3 (8.237e-19%) - 9.432139379116939391 5e-4 (-4.385e-19%)j
```

12.8 Anger, Weber and Lommel functions

12.8.1 Anger function $\mathrm{J}_\nu(x)$

`ctx.anger_j(`*n*, *z*`)`

Returns the Anger function J. See also Wikipedia [1365], MathWorld [1018], NIST [457].

Gives the Anger function

$$\mathbf{J}_\nu(z) = \frac{1}{\pi}\int_0^\pi \cos(\nu t - z\sin t)dt$$

which is an entire function of both the parameter ν and the argument z. It solves the inhomogeneous Bessel differential equation

$$f''(z) + \frac{1}{z}f'(z) + \left(1 - \frac{\nu^2}{z^2}\right) f(z) = \frac{(z-\nu)}{\pi z^2}\sin(\pi\nu).$$

We also have

$$\begin{aligned}\mathbf{J}_\nu(z) &= \frac{z}{2}\sin\left(\tfrac{1}{2}\pi\nu\right) {}_1\widetilde{F}_2\left(1; \tfrac{1}{2}(3-\nu), \tfrac{1}{2}(3+\nu); -\frac{z^2}{4}\right) \\ &+ \cos\left(\tfrac{1}{2}\pi\nu\right) {}_1\widetilde{F}_2\left(1; 1-\tfrac{1}{2}\nu, 1+\tfrac{1}{2}\nu; -\frac{z^2}{4}\right)\end{aligned} \tag{12.9}$$

An example with real input:

```
>>> from xlcalcnet import dpm, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 10; x = 3
>>> dx = dpm.angerj(n, x); mx = mpm.angerj(n, x); gx = gmp.angerj(n, x)
>>> fx = fpm.angerj(n, x); ax = apm.angerj(n, x)
>>> mpm.show([dx, mx, gx, fx, ax],  aligned=True)
dpm: 1.2928351645715883777534530802580170743 42E-5
mpm: 1.2928351645715883777534530802580170743 42e-5
gmp: 1.2928351645715883777534530802580170743 42E-05
fpm: 1.29283516457159E-05
apm: 1.2928351645715883777534530802580170654 27e-5 (4.139e-35%)
```

An example with complex input:

```
>>> from xlcalcnet import dpm, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; z = '3 + 4j'
>>> dz = dpm.angerj(n, z); mz = mpm.angerj(n, z); gz = gmp.angerj(n, z)
>>> fz = fpm.angerj(n, z); az = apm.angerj(n, z)
>>> mpm.show([dz, mz, gz, fz, az],  aligned=True)
dpm: -2.4028734611284405858E-3               + 1.9815132418922270634E-3j
mpm: -2.4028734611284405858e-3               + 1.9815132418922270634e-3j
gmp: -2.4028734611284405858E-03              + 1.9815132418922270634E-03j
fpm: -2.40287346112844E-03                   + 1.98151324189223E-03j
apm: -2.4028734611284405757e-3 (-4.846e-16%) + 1.9815132418922270691e-3 (4.828e-16%)j
```

12.8.2 Weber function $\mathbf{E}_\nu(x)$

`ctx.weber_e(`n, z`)`

Returns the Weber function E. See also Wikipedia [1365], MathWorld [1027], NIST [457].

Gives the Weber function

$$\mathbf{E}_\nu(z) = \frac{1}{\pi}\int_0^\pi \sin(\nu t - z\sin t)dt$$

which is an entire function of both the parameter ν and the argument z. It solves the inhomogeneous Bessel differential equation

$$f''(z) + \frac{1}{z}f'(z) + \left(1 - \frac{\nu^2}{z^2}\right)f(z) = -\frac{1}{\pi z^2}(z + \nu + (z-\nu)\cos(\pi\nu)).$$

We also have

$$\begin{aligned}\mathbf{E}_\nu(z) &= \sin\left(\tfrac{1}{2}\pi\nu\right){}_1\widetilde{F}_2\left(1; \tfrac{1}{2}(2-\nu), \tfrac{1}{2}(2+\nu); -\frac{z^2}{4}\right) \\ &- \frac{z}{2}\cos\left(\tfrac{1}{2}\pi\nu\right){}_1\widetilde{F}_2\left(1; \tfrac{1}{2}(3-\nu), \tfrac{1}{2}(3+\nu); -\frac{z^2}{4}\right)\end{aligned} \tag{12.10}$$

An example with real input:

```
>>> from xlcalcnet import dpm, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 10; x = 3
>>> dx = dpm.webere(n, x); mx = mpm.webere(n, x); gx = gmp.webere(n, x)
>>> fx = fpm.webere(n, x); ax = apm.webere(n, x)
>>> mpm.show([dx, mx, gx, fx, ax],  aligned=True)
dpm: 2.14807501662584748777555733056854280462 1E-2
mpm: 2.14807501662584748777555733056854280462 1e-2
gmp: 2.14807501662584748777555733056854280462 1E-02
fpm: 2.14807501662585E-02
apm: 2.14807501662584748777555733056854280465 0e-2 (2.929e-36%)
```

An example with complex input:

```
>>> from xlcalcnet import dpm, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '10'; z = '3 + 4j'
>>> dz = dpm.webere(n, z); mz = mpm.webere(n, z); gz = gmp.webere(n, z)
>>> fz = fpm.webere(n, z); az = apm.webere(n, z)
>>> mpm.show([dz, mz, gz, fz, az],  aligned=True)
dpm: 1.3585638942510994279E-2              + 2.8890345857931466183E-2j
mpm: 1.3585638942510994279e-2              + 2.8890345857931466183e-2j
gmp: 1.3585638942510994279E-02             + 2.8890345857931466183E-02j
fpm: 1.35856389425110E-02                  + 2.88903458579315E-02j
apm: 1.3585638942510994281e-2 (3.872e-16%) + 2.8890345857931466164e-2 (2.077e-16%)j
```

12.8.3 Lommel function $s_{\mu,\nu}(x) = s^{(1)}_{\mu,\nu}(x)$

`ctx.lommel_s1(mu, nu, z)`

Returns the Lommel function S1. See also Wikipedia [1387], MathWorld [1033], NIST [460].

Gives the Lommel function $s_{\mu,\nu}$ or $s^{(1)}_{\mu,\nu}$

$$s_{\mu,\nu}(z) = \frac{z^{\mu+1}}{(\mu-\nu+1)(\mu+\nu+1)}\,{}_1F_2\left(1; \frac{\mu-\nu+3}{2}, \frac{\mu+\nu+3}{2}; -\frac{z^2}{4}\right)$$

which solves the inhomogeneous Bessel equation

$$z^2 f''(z) + z f'(z) + (z^2 - \nu^2) f(z) = z^{\mu+1}.$$

An integral representation is given by

$$s_{\mu,\nu}(z) = \frac{\pi^2}{2}\left[Y_\nu(z)\int_0^z t^\mu J_\nu(t)\,dt - J_\nu(z)\int_0^z t^\mu Y_\nu(t)\,dt\right].$$

A second solution is given by *lommels2()*.

An example with real input:

```
>>> from xlcalcnet import dpm, mpm, gmp, fpm, apm
>>> mpm.dps = 40; mu = '11.3'; nu = '2.7'; x = '0.3'
>>> dx = dpm.lommels1(nu, mu, x); mx = mpm.lommels1(nu, mu, x); gx = gmp.lommels1(nu, mu, x)
>>> fx = fpm.lommels1(nu, mu, x); ax = apm.lommels1(nu, mu, x)
>>> mpm.show([dx, mx, gx, fx, ax])
dpm:  -1.020597995063898424938319205823615501143E-4
mpm:  -1.020597995063898424938319205823615501143e-4
gmp:  -1.020597995063898424938319205823615501143E-04
fpm:  -1.02059799506390E-04
apm:  -1.020597995063898424938319205823615501143e-4 (-2.293e-37%)
```

An example with complex input:

```
>>> from xlcalcnet import dpm, mpm, gmp, fpm, apm
>>> mpm.dps = 20; nu = '11.0 + 2.0j'; mu = '12.0 + 3.0j'; z = '3.0 + 4.0j'
>>> dz = dpm.lommels1(nu, mu, z); mz = mpm.lommels1(nu, mu, z); gz = gmp.lommels1(nu, mu, z)
>>> fz = fpm.lommels1(nu, mu, z); az = apm.lommels1(nu, mu, z)
>>> mpm.show([dz, mz, gz, fz, az], aligned=True)
dpm: -1.8060086283535056970E+6               + 6.53535064305699991508E+5j
mpm: -1.8060086283535056970e+6               + 6.53535064305699991508e+5j
gmp: -1.8060086283535056970E+06              + 6.53535064305699991508E+05j
fpm: -1.80600862835351E+06                   + 6.53535064305700E+05j
apm: -1.8060086283535056970e+6 (-3.443e-19%) + 6.53535064305699991508e+5 (4.077e-19%)j
```

12.8.4 Lommel function $S_{\mu,\nu}(x) = s^{(2)}_{\mu,\nu}(x)$

`ctx.lommel_s2(mu, nu, z)`

Returns the Lommel function S2. See also Wikipedia [1387], MathWorld [1033], NIST [460].

Gives the second Lommel function $S_{\mu,\nu}$ or $s^{(2)}_{\mu,\nu}$

$$S_{\mu,\nu}(z) = s_{\mu,\nu}(z) + 2^{\mu-1}\Gamma\left(\tfrac{1}{2}(\mu-\nu+1)\right)\Gamma\left(\tfrac{1}{2}(\mu+\nu+1)\right)\times \left[\sin(\tfrac{1}{2}(\mu-\nu)\pi)J_\nu(z) - \cos(\tfrac{1}{2}(\mu-\nu)\pi)Y_\nu(z)\right]$$

which solves the same differential equation as *lommels1()*.

An example with real input:

```
>>> from xlcalcnet import dpm, mpm, gmp, fpm, apm
>>> mpm.dps = 40; mu = '11.3'; nu = '2.7'; x = '0.3'
>>> dx = dpm.lommels2(nu, mu, x); mx = mpm.lommels2(nu, mu, x); gx = gmp.lommels2(nu, mu, x)
>>> fx = fpm.lommels2(nu, mu, x); ax = apm.lommels2(nu, mu, x)
>>> mpm.show([dx, mx, gx, fx, ax])
dpm:  5.14837292139577959642391778711232466148 0E+18
mpm:  5.14837292139577959642391778711232466148 0e+18
gmp:  5.14837292139577959642391778711232466148 0E+18
fpm:  5.14837292139580E+18
apm:  5.14837292139577959642391778711232093063 4e+18 (7.037e-31%)
```

An example with complex input:

```
>>> from xlcalcnet import dpm, mpm, gmp, fpm, apm
>>> mpm.dps = 20; nu = '11.0 + 2.0j'; mu = '12.0 + 3.0j'; z = '3.0 + 4.0j'
>>> dz = dpm.lommels2(nu, mu, z); mz = mpm.lommels2(nu, mu, z); gz = gmp.lommels2(nu, mu, z)
>>> fz = fpm.lommels2(nu, mu, z); az = apm.lommels2(nu, mu, z)
>>> mpm.show([dz, mz, gz, fz, az], aligned=True)
dpm: -5.9447048505419644513E+13              + 2.6440996022817513605E+14j
mpm: -5.9447048505419644513e+13              + 2.6440996022817513605e+14j
gmp: -5.9447048505419644513E+13              + 2.6440996022817513605E+14j
fpm: -5.94470485054196E+13                   + 2.64409960228175E+14j
apm: -5.9447048505428070102e+13 (-1.1e-10%) + 2.6440996022816726372e+14 (3.001e-11%)j
```

Part III

Supporting Functions

CHAPTER

THIRTEEN

ALGEBRA WITH RANDOM VARIABLES

13.1 Probability density function (pdf)

13.1.1 Calculating the pdf from the cdf

`ctx.pdf_from_cdf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns $\text{pdf}_X(x)$ from the cumulative distribution function (cdf) of a random variable X:

$$\text{pdf}_X(x) = \frac{\mathrm{d}}{\mathrm{d}x}\text{cdf}_X(x).$$

Using this method can be a viable option when the cdf, but not the pdf, is available in closed form.

13.1.2 Calculating the pdf from the characteristic function

`ctx.pdf_from_cf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Calculates the pdf as the inverse Fourier transform of its characteristic function.

The PDF of Y is the inverse Fourier transform of its characteristic function,

$$\text{pdf}_X(x) = \frac{1}{2\pi}\int_{-\infty}^{\infty} e^{ity}C_X(t)\mathrm{d}y = \frac{1}{\pi}\int_0^{\infty} \Re\left(e^{-itx}C_X(t)\right)\mathrm{d}t.$$

where $\Re(z)$ denotes the real part of z. We also have

$$\int_0^{\infty} \Re\left(e^{-itx}C_X(t)\right)\mathrm{d}t = \int_0^{\infty} \Re\left(C_X(t)\right)\cos(tx)\mathrm{d}t + \int_0^{\infty} \Im\left(C_X(t)\right)\sin(tx)\mathrm{d}t.$$

Using the right-hand side of this equation allows for efficient use of the quadrature formula of Fillon, with the half-period $\omega = \pi/x$.

13.2 Probability mass function (pmf)

13.2.1 Calculating the pmf from the cdf

`ctx.pmf_from_cdf`(*x*, *cdf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns $\text{pmf}_X(x)$, the probability mass function, calculated from the cumulative distribution function (cdf) of a random variable X:

$$\text{pmf}_X(x) = (\text{cdf}_X(x+d) - \text{cdf}_X(x))/d.$$

Using this method can be a viable option when the cdf, but not the pmf, is available in closed form.

13.2.2 Calculating the pmf from the characteristic function

`ctx.pmf_from_cf_lattice`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns $\text{pmf}_X(x)$, the probability mass function, calculated from the characteristic function $C_X(t)$ of a random variable X:

$$\text{pmf}(x) = \frac{1}{2\pi}\int_{-\pi}^{\pi} e^{-itx} C_X(t)\mathrm{d}t.$$

Using this method can be a viable option when the characteristic function, but not the pmf, is available in closed form.

13.2.3 Calculating the pmf from the factorial moments

`ctx.pmf_from_factorialmoments`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns $\text{pmf}_X(x)$, the probability mass function, of a random variable X, calculated from the factorial moments $\mu_{[r]}$:

$$\Pr[X = x] = \sum_{j \ge x} (-1)^{x+j} \binom{j}{x} \frac{\mu'_{[j]}}{j!} = \sum_{r \ge 0} (-1)^r \frac{\mu'_{[x+r]}}{x!r!}$$

and

$$\sum_{i \ge x} \Pr[X = i] = \sum_{j \ge x} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!}$$

Using this method can be a viable option when the factorial moments, but not the pmf, are available in closed form.

13.2.4 Approximating the pmf with asymptotic expansions

This call Edgeworth, Cornish-Fisher, or Lugganinni-Rice.

13.3 Cumulative distribution function (cdf)

13.3.1 Calculating the cdf from the pdf

`ctx.cdf_from_pdf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function (cdf) of a random variable X:

$$\mathrm{cdf}_X(x) = \int_0^x \mathrm{pdf}_X(x)\mathrm{d}t.$$

Using this method can be a viable option when the pdf, but not the cdf, is available in closed form.

13.3.2 Calculating the cdf from the pmf

`ctx.cdf_from_pmf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

If X is a purely discrete random variable, then it attains values $x_1, x_2, \ldots$ with probability $p_i = p(x_i)$, and the CDF of X will be discontinuous at the points x_i:

$$F_X(x) = \mathrm{P}(X \leq x) = \sum_{x_i \leq x} \mathrm{P}(X = x_i) = \sum_{x_i \leq x} p(x_i).$$

13.3.3 Calculating the cdf from the characteristic function

`ctx.cdf_from_cf_continuous`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Calculates the cdf from the characteristic function using the procedure of Gil-Pelaez.

Assuming that the characteristic function is absolutely integrable over $(-\infty, \infty)$, Gil-Pelaez derived the following inversion formula which requires integration of a real-valued function only. In particular,

$$\mathrm{cdf}_X(x) = \frac{1}{2} - \frac{1}{2\pi}\int_{-\infty}^{\infty} \frac{e^{-itx}C_X(t) - e^{itx}C_X(t)}{it}\mathrm{d}t = \frac{1}{2} - \frac{1}{\pi}\int_0^{\infty} \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t.$$

where $\Im(z)$ denotes the imaginary part of z. We also have

$$\int_0^{\infty} \Im\left(\frac{e^{-itx}C_X(t)}{t}\right)\mathrm{d}t = \int_0^{\infty} \Im\left(\frac{C_X(t)}{t}\right)\cos(tx)\mathrm{d}t - \int_0^{\infty} \Re\left(\frac{C_X(t)}{t}\right)\sin(tx)\mathrm{d}t.$$

Using the right-hand side of this equation allows for efficient use of the quadrature formula of Fillon, with the half-period $\omega = \pi/x$.

The python code is currently in Charfun.py

The following code provides a test-suite for the numerical inversion of the characteristic function:

```
class tests_charfunc(rv_cont):

    def __init__(self, rv2, x = 5, a = 0, b = 2):

        cdf_value = rv2.cdf(x)
        print ("rv2.cdf(x): ", cdf_value)

        rv2.set_x(x)
        plot(rv2.gil_pelaez_imag, [a, b], points=200)
```

(continues on next page)

(continued from previous page)

```
        print

        rv2.set_x(x)
        plot(rv2.gil_pelaez_cos, [a, b], points=200)
        print

        rv2.set_x(x)
        plot(rv2.gil_pelaez_sin, [a, b], points=200)
        print

        rv2.set_x(x)
        I0 = quad(rv2.gil_pelaez_imag, [0, +inf])
        print("Integral: ", I0)
        result0 = 0.5 - I0/pi
        print("result0:", result0 )
        print("diff0:", result0 - cdf_value)


        rv2.set_x(x)
        I1 =quadosc(rv2.gil_pelaez_cos, [0, inf], period=1*pi/x) # half period
        print("I1:", I1 )

        rv2.set_x(x)
        I2 =quadosc(rv2.gil_pelaez_sin, [0, inf], period=1*pi/x) # half period
        print("I2:", I2 )

        I3 = I1 + I2
        print("I3:", I3 )
        print("Int diff:", I3 - I0)
        result3 = 0.5 - I3/pi
        print("result3:", result3 )
        print("diff3:", result3 - cdf_value)
```

Example: non-central chi-squared distribution

The following code shows the difference between using the generic integration (error: diff0: -6.1218939724378892896e-6) and quadrature for oscillatory functions (error: diff3: 8.4703294725430033907e-22).

```
mp.dps = 20
print()
print ("Hello mpDistributions local ! ")
print()


a = 0.0
b = 2

n = mpf("5")
x = mpf("10")
rv2 = mpr().chisquare(n)

tests_charfunc(rv2, x, a, b)
```

This produces the following output (plots are to be added):

```
Hello mpDistributions local !

rv2.cdf(x):  0.92476475385348782128

Integral:  -1.334418597712864291
result0: 0.92475863195951538339
diff0: -6.1218939724378892896e-6
I1: 0.11817924829245123086
I2: -1.4526170785024453884
```

(continues on next page)

(continued from previous page)

```
I3: -1.3344378302099941575
Int diff: -0.00001923249712986651 1475
result3: 0.92476475385348782128
diff3: 8.4703294725430033907e-22
```

13.3.4 Calculating the cdf and sf from the characteristic function (lattice distribution)

`ctx.cdf_from_cf_lattice`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

The corresponding inversion formula for discrete distributions on the nonnegative integers is

$$\mathrm{cdf}(x) = \frac{1}{\pi}\int_0^\pi \Re\left(C_X(t)\sum_{z=0}^{x} e^{-itz}\right)\mathrm{d}t.$$

$$\mathrm{sf}(x) = 1 - \mathrm{cdf}(x) = \frac{1}{\pi}\int_0^\pi \Re\left(C_X(t)\sum_{z=x+1}^{N(N+1)/2} e^{-itz}\right)\mathrm{d}t.$$

13.3.5 Calculating the cdf from the factorial moments (lattice distributions)

The following relationships hold between the probabilities and the factorial moments in the case of a discrete distribution:

$$\Pr[X = x] = \sum_{j\geq x}(-1)^{x+j}\binom{j}{x}\frac{\mu'_{[j]}}{j!} = \sum_{r\geq 0}(-1)^r\frac{\mu'_{[x+r]}}{x!r!}$$

and

$$\sum_{i\geq x}\Pr[X = i] = \sum_{j\geq x}(-1)^{x+j}\binom{j-1}{x-1}\frac{\mu'_{[j]}}{j!}$$

See also Johnson(2005), page 59.

13.4 Quantile function

See also Wikipedia [1137].

13.4.1 Calculating the quantile function from the cdf

ctx.qtf_from cdf(x, cf)

where ctx is dec, mpm, or gmp.

This requires a starting point for a Newton iteration. If a specific starting point is not available, then a Cornish-Fisher expansion can be tried first.

13.4.2 Approximating the quantile function from the cumulant generating function

See inverse Jension saddlepoint approximation for examples

13.4.3 Approximating the quantile function from the cumulants

See Cornish-Fisher expansion for example.

13.5 Characteristic function

13.5.1 Calculating the characteristic function from the pdf (continuous distribution)

ctx.cf_from pdf(x, cf)

where ctx is dec, mpm, or gmp.

Let X denote a continuous univariate random variable with probability density function (PDF) $\text{pdf}_X(x)$, cumulative distribution function (CDF) $\text{cdf}_X(x)$, and moment generating function $M_X(t)$. The characteristic function of the distribution of X, given by the Fourier transform of its PDF, is defined as

$$C_X(t) = \mathrm{E}\left[e^{itX}\right] = M_X(it) = \int_{-\infty}^{\infty} e^{itx}\text{pdf}_X(x)\mathrm{d}x.$$

Note however that the characteristic function of a distribution always exists, even when the probability density function or moment-generating function do not.

13.5.2 Calculating the characteristic function from the pmf (lattice distribution)

ctx.cf_from pmf(x, cf)

where ctx is dec, mpm, or gmp.

For a discrete distribution on the nonnegative integers, it is defined as

$$C_X(t) = \mathrm{E}[e^{itX}] = \sum_{j=0}^{\infty} e^{ijt}\text{pmf}(j)$$

See Johnson(2005), p. 50

13.5.3 Calculating the characteristic function from the quantile function

ctx.cf_from qtf(x, cf)

where ctx is dec, mpm, or gmp.

$$C_X(t) = \mathrm{E}\left[e^{itX}\right] = \int_0^1 e^{it\,\text{qtf}_X(p)}dp.$$

13.5.4 Calculating the characteristic function from the raw moments

ctx.**cf_from_rawmoments**(*x*, *cf*)

where ctx is dec, mpm, or gmp.

Formally, we can write

$$C_X(t) = 1 + \sum_{k=1}^{\infty} \mu'_X(k)\frac{i^k t^k}{k!}$$

$$C_X(t) = 1 + \sum_{k=1}^{\infty} \mathrm{E}\left[X^k\right]\frac{i^k t^k}{k!}$$

See also http://mathworld.wolfram.com/RawMoment.html for an expression involving cumulants.

13.6 Moment generating function

See also Wikipedia [1134].

13.6.1 Calculating the moment-generating function from the pdf

`ctx.mgf_from_pdf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

The moment-generating function's definition expands to

$$M_X(t) = \mathrm{E}\left[e^{tX}\right] = \int_{-\infty}^{\infty} e^{tx} f_X(x)\,\mathrm{d}x$$

13.6.2 Calculating the moment-generating function from the characteristic function

`ctx.mgf_from_cf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

The characteristic function $\varphi_X(t)$ is related to the moment-generating function via $\varphi_X(t) = M_{iX}(t) = M_X(it)$: the characteristic function is the moment-generating function of iX or the moment generating function of X evaluated on the imaginary axis. This function can also be viewed as the Fourier transform of the probability density function, which can therefore be deduced from it by inverse Fourier transform.

13.6.3 Calculating the moment-generating function from the cumulant-generating function

`ctx.mgf_from_cgf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

The cumulant-generating function is defined as the logarithm of the moment-generating function; some instead define the cumulant-generating function as the logarithm of the characteristic function, while others call this latter the second cumulant-generating function.

13.6.4 Calculating the moment-generating function from the probability-generating function

`ctx.mgf_from_pgf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

The probability-generating function is defined as $G(z) = E\left[z^X\right]$. This immediately implies that $G(e^t) = E\left[e^{tX}\right] = M_X(t)$.

```
>>> from xlcalcnet import *
>>> mp.dps = 30
>>> mu = 0; sigma = 1; t = 0.3;
>>> print ("c_x: ", chi_squared(mu, sigma).c_x(t))
6.356352346256452561561561561456 1356E+00
```

13.6.5 Calculating the moment-generating function from the raw moments

`ctx.mgf_from_rawmoments`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

$$M_X(t) = 1 + tm_1 + \frac{t^2 m_2}{2!} + \frac{t^3 m_3}{3!} + \cdots + \frac{t^n m_n}{n!} + \cdots$$

where m_n is the nth moment. Differentiating $M_X(t) i$ times with respect to t and setting $t = 0$, we obtain the ith moment about the origin.

13.7 Cumulant generating function

See also Wikipedia [1122].

13.7.1 Calculating the cumulant-generating function from the characteristic function

`ctx.cgf_from_cf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

The characteristic function $\varphi_X(t)$ is related to the moment-generating function via $\varphi_X(t) = M_{iX}(t) = M_X(it)$: the characteristic function is the moment-generating function of iX or the moment generating function of X evaluated on the imaginary axis. This function can also be viewed as the Fourier transform of the probability density function, which can therefore be deduced from it by inverse Fourier transform.

13.7.2 Calculating the cumulant-generating function from the moment-generating function

`ctx.cgf_from_mgf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

The cumulant-generating function is defined as the logarithm of the moment-generating function; some instead define the cumulant-generating function as the logarithm of the characteristic function, while others call this latter the second cumulant-generating function.

13.7.3 Calculating the cumulant-generating function from the probability-generating function

`ctx.cgf_from_pgf`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

The cumulant-generating function is defined as the logarithm of the moment-generating function; some instead define the cumulant-generating function as the logarithm of the characteristic function, while others call this latter the second cumulant-generating function.

13.7.4 Calculating the cumulant-generating function from the cumulants

`ctx.cgf_from_cumulants`(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

The cumulants κ_n are obtained from a power series expansion of the cumulant generating function:

$$K(t) = \sum_{n=1}^{\infty} \kappa_n \frac{t^n}{n!} = \mu t + \sigma^2 \frac{t^2}{2} + \cdots .$$

This expansion is a Maclaurin series, so the n-th cumulant can be obtained by differentiating the above expansion n times and evaluating the result at zero:

$$\kappa_n = K^{(n)}(0).$$

13.8 Probability generating function

13.8.1 Calculating the probability-generating function from the pmf vector

`ctx.`**`pgf_from_pmf_vector`**(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Some text

13.8.2 Calculating the probability-generating function from the moment-generating function

`ctx.`**`pgf_from_mgf`**(*x*, *cf*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Some text

13.9 Factorial Moments

13.9.1 Calculating the factorial moments from the raw moments

`ctx.factorial_moments_from_rawmoments`(x, cf)

where `ctx` is `dec`, `mpm`, or `gmp`.

Calculates the factorial moments $\mu'_{[r]}$ from the raw moments μ'_r

$$\mu'_{[r]} = \sum_{j=0}^{r} s(r,j)\mu'_j,$$

where $s(r,j)$ is the Stirling number of the first kind.

13.9.2 Calculating the factorial moments from the cumulants

`ctx.factorial_moments_from_cumulants`(x, cf)

where `ctx` is `dec`, `mpm`, or `gmp`.

Calculates the factorial moments $\mu'_{[r]}$ from the raw moments μ'_r

$$\mu'_{[r]} = \sum_{j=0}^{r} s(r,j)\mu'_j,$$

where $s(r,j)$ is the Stirling number of the first kind.

13.10 Raw Moments

See also Wikipedia [1133]

13.10.1 Calculating the raw moments from the pdf

`ctx.rawmoments_from_pdf(x, cf)`

where `ctx` is `dec`, `mpm`, or `gmp`.

The n-th moment of a real-valued continuous function f(x) of a real variable about a value c is

$$\mu_n = \int_{-\infty}^{\infty} (x-c)^n \, f(x) \, \mathrm{d}x.$$

It is possible to define moments for random variables in a more general fashion than moments for real values—see moments in metric spaces. The moment of a function, without further explanation, usually refers to the above expression with c = 0.

13.10.2 Calculating the raw moments from the pmf

`ctx.rawmoments_from_pmf_vector(x, cf)`

where `ctx` is `dec`, `mpm`, or `gmp`.

If X is a purely discrete random variable, then it attains values $x_1, x_2, \ldots$ with probability $p_i = p(x_i)$, and the CDF of X will be discontinuous at the points x_i:

$$\mu_n = \sum_{x_i \leq x} x_i^n p(x_i).$$

13.10.3 Calculating the raw moments from the factorial moments

`ctx.rawmoments_from_factorial_moments(x, cf)`

where `ctx` is `dec`, `mpm`, or `gmp`.

Calculates the raw moments μ'_r from the factorial moments $\mu'_{[r]}$

$$\mu'_r = \sum_{j=0}^{r} S(r,j) \mu'_{[j]},$$

where $S(r,j)$ is the Stirling number of the second kind.

13.10.4 Calculating the raw moments from the central moments

`ctx.rawmoments_from_centralmoments(central, raw)`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the raw moments μ'_r from the central moments μ_r (see Lee and Lin [404], Rinne [480], p. 36):

$$\mu'_r = \sum_{j=0}^{r} \binom{r}{j} \mu_{r-j} (\mu'_1)^r$$

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> ivr.exp([-inf,0])
[0.0, 1.0]
>>> ivr.exp([0,1])
[1.0, 2.7182818284590452353602874955 8]
```

13.10.5 Calculating the raw moments from the cumulants

`ctx.rawmoments_from_cumulants()`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the raw moments μ'_r from the cumulants κ_r (see Lee and Lin [404], Rinne [480], p. 36):

$$\mu'_r = \kappa_r + \sum_{j=1}^{r-1} \binom{r-1}{j-1} \mu'_{r-j} \kappa_j$$

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> ivr.exp([-inf,0])
[0.0, 1.0]
>>> ivr.exp([0,1])
[1.0, 2.7182818284590452353602874955 8]
```

13.10.6 Calculating the raw moments from the moment-generating function

`ctx.rawmoments_from_mgf()`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

The moment-generating function is so called because if it exists on an open interval around t = 0, then it is the exponential generating function of the moments of the probability distribution:

$$m_n = E\left(X^n\right) = M_X^{(n)}(0) = \left.\frac{d^n M_X}{\mathrm{d}t^n}\right|_{t=0}.$$

That is, with n being a nonnegative integer, the nth moment about 0 is the nth derivative of the moment generating function, evaluated at t = 0.

13.10.7 Calculating the raw moments from the characteristic function

`ctx.rawmoments_from_cf()`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Characteristic functions can also be used to find moments of a random variable. Provided that the nth moment exists, the characteristic function can be differentiated n times and

$$\mathrm{E}\left[X^n\right] = i^{-n}\,\varphi_X^{(n)}(0) = i^{-n}\left[\frac{d^n}{\mathrm{d}t^n}\varphi_X(t)\right] t = 0$$

13.10.8 Calculating the raw moments from the probability-generating function

`ctx.rawmoments_from_pgf()`

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

The kth raw moment of X is given by

$$\mathrm{E}(X^k) = \left(z\frac{\partial}{\partial z}\right)^k G(z)\Big|_{z=1^-}$$

More generally, the kth factorial moment,

$\mathrm{E}(X(X-1)\cdots(X-k+1))$ of X is given by

$$\mathrm{E}\left(\frac{X!}{(X-k)!}\right) = G^{(k)}(1^-), \quad k \geq 0.$$

13.11 Central Moments

See also Wikipedia [1122]

13.11.1 Calculating the central moments from the factorial moments

`ctx.centralmoments_from_factorialmoments`(*self*, *mfac*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the central moments μ_r from the factorial moments.

13.11.2 Calculating the central moments from the raw moments

`ctx.centralmoments_from_rawmoments`(*central*, *raw*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the central moments μ_r from raw the moments μ'_r (see Lee and Lin [404], Rinne [480], p. 36):

$$\mu_r = \sum_{j=0}^{r} \binom{r}{j} \mu'_{r-j} (-\mu'_1)^j$$

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> ivr.exp([-inf,0])
[0.0, 1.0]
>>> ivr.exp([0,1])
[1.0, 2.7182818284590452353602874955 8]
```

13.11.3 Calculating the central moments from the cumulants

`ctx.centralmoments_from_cumulants`()

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Calculates the central moments μ_r from the cumulants κ_r (see Lee and Lin [404], Rinne [480], p. 36):

$$\mu_r = \kappa_r + \sum_{j=1}^{r-1} \binom{r-1}{j-1} \mu_{r-j} \kappa_j$$

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> ivr.exp([-inf,0])
[0.0, 1.0]
>>> ivr.exp([0,1])
[1.0, 2.7182818284590452353602874955 8]
```

13.12 Cumulants

See also Wikipedia [1122]

13.12.1 Calculating the cumulants from the pmf vector

cumulants_from_pmfvector(*self*, *x*, *nl*, *order*, *show=False*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the cumulants κ_r from the pmf vector.

13.12.2 Calculating the cumulants from the factorial moments

ctx.**cumulants_from_factorialmoments**(*self*, *mfac*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the cumulants κ_r from the factorial moments.

13.12.3 Calculating the cumulants from the raw moments

ctx.**cumulants_from_rawmoments**(*raw_moments*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the cumulants κ_r from the raw moments μ'_r (see Lee and Lin [404], Rinne [480], p. 36):

$$\kappa_r = \mu'_r - \sum_{j=1}^{r-1} \binom{r-1}{j-1} \mu'_{r-j} \kappa_j$$

13.12.4 Calculating the cumulants from the central moments

ctx.**cumulants_from_centralmoments**(*cumulants*, *central*)

where ctx is ipm, dec, mpm, or gmp.

Calculates the cumulants κ_r from the central moments μ_r :

$$\kappa_r = \mu_r - \sum_{j=2}^{r-1} \binom{r-1}{j-1} \mu_{r-j} \kappa_j, \quad r > 1, \quad \kappa_1 = \mu_1.$$

```
>>> from xlcalcnet import dec, mpr, ivr, ivc
>>> ivr.dps = 25; ivr.pretty = True
>>> ivr.exp([-inf,0])
[0.0, 1.0]
>>> ivr.exp([0,1])
[1.0, 2.71828182845904523536028749558]
```

13.12.5 Calculating the cumulants from the cumulant-generating function

ctx.**cumulants_from_cdf**(*k*, *cgf*)

where ctx is ipm, dec, mpm, or gmp.

The cumulants κ_n are obtained from a power series expansion of the cumulant generating function:

$$K(t) = \sum_{n=1}^{\infty} \kappa_n \frac{t^n}{n!} = \mu t + \sigma^2 \frac{t^2}{2} + \cdots .$$

This expansion is a Maclaurin series, so the n-th cumulant can be obtained by differentiating the above expansion n times and evaluating the result at zero:

$$\kappa_n = K^{(n)}(0).$$

CHAPTER

FOURTEEN

SERIES AND INTEGRALS

14.1 Finite series algorithms for selected distributions

This section provides distribution functions in interval arithmetic for some special cases. These function can be used to assess the accuracy for other algorithms without verification.

14.1.1 Central χ^2 distribution, cdf (integer degrees of freedom)

`ctx.chi2_cohen_cdf`(*x*, *nu*, *cdf=True*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the cdf of the central chi-square distribution for integer degrees of freedom. See also Wikipedia [1148], MathWorld [796], BoostMath [53], Witkovský [1464].

The cdf can be expressed as a finite sum if n is an integer:

$$F_{\chi^2}(n,x) = 1 + 2\Phi(-\sqrt{x}) + 2\phi\left(\sqrt{x}\right) \sum_{r=1}^{(n-1)/2} \frac{\sqrt{x}^{2r-1}}{1\cdot 3\cdot 5\ldots(2r-1)}, \qquad \text{for } n \text{ odd},$$

$$F_{\chi^2}(n,x) = e^{-x/2}\left(1 + \sum_{r=1}^{(n-2)/2} \frac{x^r}{2\cdot 4\cdot 6\ldots(2r)}\right), \qquad \text{for } n \text{ even},$$

where $\phi(\cdot)$ denotes the pdf of the normal distribution (see) and $\Phi(\cdot)$ denotes the cdf of the normal distribution (see).

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = 12; n = 10
>>> dx = dec.chi2_cohen_cdf(x, n); mx = mpm.chi2_cohen_cdf(x, n)
>>> ix = ipm.chi2_cohen_cdf(x, n); fx = fpm.chi2_cohen_cdf(x, n)
>>> gx = gmp.chi2_cohen_cdf(x, n); ax = apm.chi2_cohen_cdf(x, n)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.1.2 Central Student t distribution, cdf (integer degrees of freedom)

ctx.**student_t_owen_cdf**(*x*, *nu*, *cdf=True*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Returns the cdf of the central Student t distribution for integer degrees of freedom. See also Wikipedia [1171], MathWorld [813], BoostMath [69], Broda and Paolella [160], Witkovský [1460], Witkovský [1479].

The cdf can be expressed as a finite sum if n is an integer:

$$F_t\left(n,x\right)=\tfrac{1}{2}+z_n+(c_1+c_3+\cdots+c_{n-2}),\qquad\text{for } n \text{ odd},$$

$$\text{where } z_n=\frac{1}{\pi}\arctan(\frac{x}{\sqrt{n}});a_n=\frac{1}{\sqrt{n}\pi};b_n=\frac{n}{n+x^2};c_1=xa_nb_n;c_k=c_{k-2}b_n(1-1/k)$$

$$F_t\left(n,x\right)=\tfrac{1}{2}+(c_0+c_2+\cdots+c_{n-2}),\qquad\text{for } n \text{ even},$$

$$\text{where } d_n=\frac{1}{2\sqrt{n+x^2}};b_n=\frac{n}{n+x^2};c_0=xd_n;c_k=c_{k-2}b_n(1-1/k)$$

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = 2; n = 10;
>>> dx = dec.student_t_owen_cdf(x, n); mx = mpm.student_t_owen_cdf(x, n)
>>> ix = ipm.student_t_owen_cdf(x, n); fx = fpm.student_t_owen_cdf(x, n)
>>> gx = gmp.student_t_owen_cdf(x, n); ax = apm.student_t_owen_cdf(x, n)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.1138774234168360557146160900744444149943E-1
mpm:  3.1138774234168360557146160900744444149943e-1
ipm:  3.1138774234168360557146160900744444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.1138774234168360557146160900744444149943E-01
ipm:  3.1138774234168360557146160900744444149943e-1 (4.055e-38%)
```

14.1.3 Central Fisher F distribution, cdf (integer degrees of freedom)

ctx.**fisher_f_seber_cdf**(*x*, *m*, *n*, *cdf=True*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Returns the cdf of the central Fisher F distribution for integer degrees of freedom. See also Wikipedia [1151], MathWorld [799], BoostMath [56], Witkovský [1466], Abramowitz and Stegun. [3], Butler and Paolella [166], Chattamvelli and Jones [175], Witkovský [1460].

The cdf can be expressed as a finite sum if m is an integer, and n is a positive real number:

$$1-F_F\left(m,n,x\right)=a_m+b_m(c_1+c_3+\cdots+c_{m-2}),\qquad\text{for } m \text{ odd},$$

$$\text{where } a_m=2T(n,-z_m);b_m=2t(n,z_m)\cdot z_m;z_m=\sqrt{mx};$$

$$1-F_F\left(m,n,x\right)=d_m(c_0+c_2+\cdots+c_{m-2}),\qquad\text{for } m \text{ even},$$

$$\text{where } d_m=(1-u_m)^{n/2}$$

$$\text{and } u_m=mx/(mx+n),\quad c_0=c_1=1,\quad c_k=c_{k-2}u_m\cdot(n+k-2)/k$$

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = 3; m = 10; n = 20
>>> dx = dec.fisher_f_seber_cdf(x, m, n); mx = mpm.fisher_f_seber_cdf(x, m, n)
>>> ix = ipm.fisher_f_seber_cdf(x, m, n); fx = fpm.fisher_f_seber_cdf(x, m, n)
>>> gx = gmp.fisher_f_seber_cdf(x, m, n); ax = apm.fisher_f_seber_cdf(x, m, n)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
```

(continues on next page)

(continued from previous page)

```
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.1.4 Central Beta distribution, cdf ($2a$ an integer, $2b$ an integer)

`ctx.beta_seber_cdf`(*x*, *a*, *b*, *cdf=True*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the cdf of the central Fisher F distribution for integer degrees of freedom. See also Wikipedia [1146], MathWorld [795], BoostMath [51], Witkovský [1463].

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.3'; a = '10'; b = '20'
>>> dx = dec.beta_seber_cdf(x, a, b); mx = mpm.beta_seber_cdf(x, a, b)
>>> ix = ipm.beta_seber_cdf(x, a, b); fx = fpm.beta_seber_cdf(x, a, b)
>>> gx = gmp.beta_seber_cdf(x, a, b); ax = apm.beta_seber_cdf(x, a, b)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.1.5 Noncentral χ^2 distribution, cdf (integer degrees of freedom)

`ctx.chi2_nc_cohen_cdf`(*x*, *nu*, *lambda1*, *cdf=True*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the cdf of the central chi-square distribution for integer degrees of freedom. See also Wikipedia [1164], MathWorld [808], Patnaik [464], Penev and Raykov [466], Wang and Gray [793], Winterbottom [1454], BoostMath [64], Witkovský [1474], Johansson [386], András and Baricz [10].

For odd degrees of freedom, the pdf and cdf can be expressed as a finite sum, using the recurrence relations given in section ref{NoncentralChiSquareDistributionRecur}, and defining $h(n, x, \lambda) = e^{(1/2)(x+\lambda)} f_{\chi^2}(n, x; \lambda)$:

$$F_{\chi^2}\left(1, x; \delta^2\right) = \Phi(x - \delta) - \Phi(-x - \delta)$$

$$h(1, x; \lambda) = \frac{\cosh(\sqrt{x\lambda})}{\sqrt{2\pi x}}, \quad h(3, x; \lambda) = \frac{\sinh(\sqrt{x\lambda})}{\sqrt{2\pi\lambda}}$$

where $\Phi(\cdot)$ denotes the cdf of the normal distribution.

NOTE: This needs to be extended using Marcum Q.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = 12; nu = 7; l = 30
>>> dx = dec.chi2_nc_cohen_cdf(x, n, l); mx = mpm.chi2_nc_cohen_cdf(x, n, l)
>>> ix = ipm.chi2_nc_cohen_cdf(x, n, l); fx = fpm.chi2_nc_cohen_cdf(x, n, l)
>>> gx = gmp.chi2_nc_cohen_cdf(x, n, l); ax = apm.chi2_nc_cohen_cdf(x, n, l)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
```

(continues on next page)

(continued from previous page)

```
mpm:  3.11387742341683605571461609007444414994 3e-1
ipm:  3.11387742341683605571461609007444414994 4e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.11387742341683605571461609007444414994 3E-01
ipm:  3.11387742341683605571461609007444414994 3e-1 (4.055e-38%)
```

14.1.6 Noncentral Student t distribution, cdf (integer degrees of freedom)

`ctx.student_t_nc_owen_cdf`(*x*, *n*, *delta*, *cdf=True*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the cdf of the noncentral Student t distribution for integer degrees of freedom $n \geq 1$, noncentrality parameter $\delta \in \mathbb{R}$ and the support interval $(-\infty, +\infty)$. See also Wikipedia [1165], MathWorld [810], BoostMath [66], Benton and Krishnamoorthy [30], Broda and Paolella [160], Owen [454], Wang and Gray [793], Witkovský [1461], Johansson [386].

The function uses the following algorithm given by Owen [454]:

$$F_{t'}\left(n, x, \delta\right) = \Phi(-\delta\sqrt{B}) + 2(M_1 + M_3 + \cdots + M_n) \quad \text{for odd degrees of freedom}$$

$$F_{t'}\left(n, x, \delta\right) = \Phi(-\delta) + \sqrt{2\pi}(M_2 + M_4 + \cdots + M_n) \quad \text{for even degrees of freedom}$$

$$A = \frac{t}{\sqrt{n}}, \quad B = \frac{n}{n + t^2}$$

$$M_1 = T_{\text{Owen}}(\delta\sqrt{B}, A), \quad M_2 = A\sqrt{B}\phi(\delta\sqrt{B}) - \Phi(\delta A\sqrt{B})$$

$$M_3 = B(\delta A M_2 + A\phi(\delta)/\sqrt{2\pi}), \quad M_4 = \tfrac{1}{2}B(\delta A M_3 + M_2)$$

$$M_k = \frac{k-3}{k-2}B(a_k \delta A M_{k-1} + M_{k-2}), \quad a_k = \frac{1}{(k-4)a_{k-1}} \quad \text{for } k \geq 5,$$

where $T_{\text{Owen}}(\cdot, \cdot)$ denotes Owen's T function , and $\Phi(\cdot)$ and $\phi(\cdot)$ denote the cdf and pdf of the normal distribution, respectively.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = 20; n = 10; d = 30
>>> dx = dec.student_t_nc_owen_cdf(x, n, d); mx = mpm.student_t_nc_owen_cdf(x, n, d)
>>> ix = ipm.student_t_nc_owen_cdf(x, n, d); fx = fpm.student_t_nc_owen_cdf(x, n, d)
>>> gx = gmp.student_t_nc_owen_cdf(x, n, d); ax = apm.student_t_nc_owen_cdf(x, n, d)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.11387742341683605571461609007444414994 3E-1
mpm:  3.11387742341683605571461609007444414994 3e-1
ipm:  3.11387742341683605571461609007444414994 4e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.11387742341683605571461609007444414994 3E-01
ipm:  3.11387742341683605571461609007444414994 3e-1 (4.055e-38%)
```

14.1.7 Noncentral Fisher F distribution, cdf (m an even integer)

`ctx.fisher_f_nc_seber_cdf`(*x*, *m*, *n*, *lambda1*, *cdf=True*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the cdf of the noncentral Fisher F distribution for integer degrees of freedom. See also Wikipedia [1162], MathWorld [809], BoostMath [65], Benton and Krishnamoorthy [30], Butler and Paolella [166], Chou [179], Chattamvelli and Jones [175], Wang and Gray [793], Witkovský [1475], Johansson [386].

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = 13; m = 10; n = 20; l1 = 30
>>> dx = dec.fisher_f_nc_seber_cdf(x, m, n, l1); mx = mpm.fisher_f_nc_seber_cdf(x, m, n, l1)
>>> ix = ipm.fisher_f_nc_seber_cdf(x, m, n, l1); fx = fpm.fisher_f_nc_seber_cdf(x, m, n, l1)
>>> gx = gmp.fisher_f_nc_seber_cdf(x, m, n, l1); ax = apm.fisher_f_nc_seber_cdf(x, m, n, l1)
>>> mpm.show([dx, mx, ix]); mpm.show([fx, gx, ax])
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.1.8 Noncentral Beta distribution, cdf (b an integer)

`ctx.beta_nc_seber_cdf`(x, a, b, *lambda1*, *cdf=True*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the cdf of the noncentral Fisher F distribution for integer degrees of freedom. See also Wikipedia [1163], BoostMath [63], Wang and Gray [793], Witkovský [1473], Seber [496], Johansson [386].

CDF: Finite Series for even error degrees of freedom:

This is code for noncentral beta and needs to be adapted for noncentral F.

The cdf can be calculated using the following finite series, if b is an integer:

$$I(x; a, b, \lambda) = e^{-\lambda(1-x)} \sum_{n=0}^{b-1} L_n, \quad \text{where}$$

$$L_0 = 1, \quad L_1 = (1-x)(a+\lambda x),$$

$$L_n = \frac{1-x}{n}\left((2n-2+a+\lambda x)L_{n-1} - (n+a-2)(1-x)L_{n-2}\right)$$

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = 0.25; a = 10; b = 20; l = 30
>>> dx = dec.beta_nc_seber_cdf(x, a, b, l); mx = mpm.beta_nc_seber_cdf(x, a, b, l)
>>> ix = ipm.beta_nc_seber_cdf(x, a, b, l); fx = fpm.beta_nc_seber_cdf(x, a, b, l)
>>> gx = gmp.beta_nc_seber_cdf(x, a, b, l); ax = apm.beta_nc_seber_cdf(x, a, b, l)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.1.9 Pearson's ρ distribution, pdf (integer N)

`ctx.pearson_rho_nc_owen_pdf`(r, N, *rho*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the cdf of the Pearson rho distribution for integer degrees of freedom. See also Wikipedia [1168], MathWorld [797], Hotelling [375], Odeh [426].

See Hotelling [375].

For $N = 3, 4$ the probability density function can be expressed in closed form Odeh [426]:

$$f_R(3, r; \rho) = \frac{A^2(1 + xU)}{\pi B^2 C}$$

$$f_R(4, r; \rho) = \frac{AC^3(B^2U + 3x(1 + xU))}{\pi B^4}$$

where x, A, B, C and U are defined as follows:

$$x = \rho r, \qquad A = \sqrt{1 - \rho^2}, \quad B = \sqrt{1 - x^2}, \quad C = \sqrt{1 - r^2}, \quad U = \frac{\arccos(-x)}{B}$$

The probability density function $f_R(N, r; \rho)$ satisfies the following recurrence formula for $N \geq 5$ (see Hotelling [375]):

$$f_R(N, r; \rho) = \frac{2N - 5}{B^2(N - 3)} xAC f_R(N - 1, r; \rho) + \frac{N - 3}{B^2(N - 4)} A^2C^2 f_R(N - 2, r; \rho)$$

For $0 \leq x \leq 1$,this recurrence formula can be safely used to find a sequence of values for $f_R(5, r; \rho)$, $f_R(6, r; \rho), ..., f_R(N, r; \rho)$. However, for $-1 < x < 0$ the recurrence formula is numerically unstable, since the two terms on the right-hand of the equation are of opposite sign.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; r = '0.25'; N = '20'; rho = '0.1'
>>> dx = dec.pearson_rho_nc_owen_pdf(x, N, rho); mx = mpm.pearson_rho_nc_owen_pdf(x, N, rho)
>>> ix = ipm.pearson_rho_nc_owen_pdf(x, N, rho); fx = fpm.pearson_rho_nc_owen_pdf(x, N, rho)
>>> gx = gmp.pearson_rho_nc_owen_pdf(x, N, rho); ax = apm.pearson_rho_nc_owen_pdf(x, N, rho)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.1.10 Pearson's ρ distribution, cdf (integer N)

`ctx.pearson_rho_nc_owen_cdf`(*x*, *N*, *rho*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the cdf of the Pearson rho distribution for integer degrees of freedom. See also Wikipedia [1168], MathWorld [797], Hotelling [375], Odeh [426].

For $N = 3, 4, 5, 6$ the cumulative distribution function $F_R(N, r; \rho)$ can be expressed in closed form

$$F_R(3, r; \rho) = \frac{\arccos(-r)}{\pi} - \frac{\rho CU}{\pi}$$

$$F_R(4, r; \rho) = \frac{\arccos(\rho)}{\pi} - \frac{\rho AC^2}{\pi B^2} - \frac{rA^3U}{\pi B^2}$$

$$F_R(5, r; \rho) = \frac{\arccos(-r)}{\pi} - \frac{(x^2 - 3\rho^3 + 2)rA^2C}{2\pi B^4} + \frac{(\rho^2 - 3 + 2\rho^2x^2)\rho C^3U}{2\pi B^4}$$

$$F_R(6, r; \rho) = \frac{\arccos(\rho)}{\pi} - \frac{[\rho r^2(2x^2 + 13) - 2\rho(4x^4 + 6x^2 + 5) + \rho^3(11x^2 + 4)`AC^2}{6\pi B^6} + \frac{2x^2(-2r^2 + 1)rA^5U}{6\pi B^6} \tag{14.1}$$

where x, A, B, C and U are defined as follows:

$$x = \rho r, \qquad A = \sqrt{1-\rho^2}, \quad B = \sqrt{1-x^2}, \quad C = \sqrt{1-r^2}, \quad U = \frac{\arccos(-x)}{B}$$

The cumulative distribution function $F_R(N, r; \rho)$ satisfies the following recurrence formula for $N \geq 7$ (see Hotelling [375]):

$$\begin{aligned} F_R(N,r;\rho) \quad = \quad & \frac{2(N-4)\rho^2 - N + 5}{(N-3)\rho^2} F_R(N-2,r;\rho) \\ & + \frac{(N-5)A^2}{(N-3)\rho^2} F_R(N-4,r;\rho) \\ & + \frac{(N-4)A^2C^2 - (2N-9)B^2}{(N-4)(N-3)\rho^2 AC} \rho f_R(N-1,r;\rho) \\ & + \frac{(N-4)^2 + (3N(N-8)+47)\rho^2}{(N-4)^2(N-3)\rho^2} r f_R(N-2,r;\rho) \end{aligned} \tag{14.2}$$

For N odd, the above formula can be used repeatedly to find $F_7, F_9, ..., F_N$ starting with values of F_3 and F_5.

For N even, the formula can be used repeatedly to find $F_8, F_{10}, ..., F_N$ starting with values of F_4 and F_6.

However, in many situations the formula is numerically unstable, e.g. if $\rho^2 < \frac{N-5}{2(N-4)}$, the first two terms on the right hand side of equation (ref{eq:PearsonRho_Closed_Recur_1}) are of opposite sign.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; r = '0.25'; N = '20'; rho = '0.1'
>>> dx = dec.pearson_rho_nc_owen_cdf(x, N, rho); mx = mpm.pearson_rho_nc_owen_cdf(x, N, rho)
>>> ix = ipm.pearson_rho_nc_owen_cdf(x, N, rho); fx = fpm.pearson_rho_nc_owen_cdf(x, N, rho)
>>> gx = gmp.pearson_rho_nc_owen_cdf(x, N, rho); ax = apm.pearson_rho_nc_owen_cdf(x, N, rho)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.1.11 Fisher's R^2 distribution, cdf (finite sum for $N - p$ even)

`ctx.fisher_r2_gd1_cdf`(*x*, *a*, *b*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the cdf of the multiple correlation coefficient distribution for $N - p$ an even integer. See also Lee [407], Lee [408], Gurland [357], Gurland and Milton [359], Gurland and Asiribo [358], Muirhead [418], Benton and Krishnamoorthy [30], Fisher [336], Gatsonis and Sampson [343].

CDF: Finite Series of Gurland:

$$F_{R^2}(x; p, N, \rho^2) = \sum_{j=0}^{k} b_j I_y\left(\tfrac{1}{2}(p-1+2j), k\right) \quad \text{where}$$

$$k = \tfrac{1}{2}(N-p), \quad y = \frac{x(1-\rho^2)}{1-x\rho^2}, \quad b_j = \binom{k}{j}(\rho^2)^j(1-\rho^2)^{k-j}$$

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; r2 = '0.25'; p = '3'; N = '20'; rho2 = '0.1'
>>> dx = dec.fisher_r2_gd1_cdf(r2, p, N, rho2); mx = mpm.fisher_r2_gd1_cdf(r2, p, N, rho2)
>>> ix = ipm.fisher_r2_gd1_cdf(r2, p, N, rho2); fx = fpm.fisher_r2_gd1_cdf(r2, p, N, rho2)
>>> gx = gmp.fisher_r2_gd1_cdf(r2, p, N, rho2); ax = apm.fisher_r2_gd1_cdf(r2, p, N, rho2)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.1.12 Roy's largest root distribution, pdf, cdf and sf

`ctx.`**`roy_pdf_cdf_sf`**(*x*, *p*, *n1*, *n2*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns simultaneouly $\text{pdf}_X(r)$, the probability density function , $\text{cdf}_X(x)$, the cumulative distribution function, and $\text{sf}_X(x)$, the survival function, of a random variable X, following the distribution of Roy's largest root, with parameters *p*, *n1* and *n2* and the support interval $(0, 1)$.

Let X, Y denote two independent real Gaussian $p \times n_1$ and $p \times n_2$ matrices with $n_1, n_2 \geq p$, each constituted by zero mean independent, identically distributed columns with common covariance. Then Roy's largest root criterion, used in multivariate analysis of variance (MANOVA), is based on the statistic of the largest eigenvalue, Θ_1, of $(X + Y)^{-1}Y$, where X and Y are independent central Wishart matrices.

See also Anderson [9], Muirhead [418], Butler [165], Chiani [177], Chiani [178], and *dist_roy()*.

See also Chen 2002 (Tables), Turgeon (2018), R Package rootWishart.

The pdf and cdf of Θ_1 are given by:

$$\text{pdf}_X(\theta_1) = C\sqrt{|A(\theta_1)|} \times \tfrac{1}{2}\text{tr}\left(A(\theta_1)^{-1} \cdot \frac{\text{d}A(\theta_1)}{\text{d}\theta_1}\right)$$

$$\text{cdf}_X(\theta_1) = C\sqrt{|A(\theta_1)|}, \quad \text{where}$$

$$C = \pi^{s/2} \prod_{i=1}^{s} \frac{\Gamma\left(\frac{1}{2}(i + 2m + 2n + s + 2)\right)}{\Gamma\left(\frac{i}{2}\right)\Gamma\left(\frac{1}{2}(i + 2m + 1)\right)\Gamma\left(\frac{1}{2}(i + 2n + 1)\right)},$$

$$s = p, \quad m = (n_1 - p - 1)/2, \quad n = (n_2 - p - 1)/2.$$

When s is even, we have $n_{mat} = s$ and the elements of the $s \times s$ skew-symmetric matrix $A(\theta_1)$ are:

$$a_{i,j}(\theta_1) = E(\theta_1; m + j, m + i) - E(\theta_1; m + i, m + j) \quad i, j = 1, \ldots, s, \quad \text{where}$$

$$E(x; a, b) = \int_0^x t^{a-1}(1 - t)^n B(t; b, n + 1)\text{d}t.$$

When s is odd, we have $n_{mat} = s + 1$ and the elements of the $(s + 1) \times (s + 1)$ skew-symmetric matrix $A(\theta_1)$ are as above, with the additional elements

$$a_{i,s+1}(\theta_1) = I(\theta_1; m + i, n + 1) \quad i = 1, \ldots, s; \quad a_{s+1,j}(\theta_1) = -a_{j,s+1}(\theta_1) \quad j = 1, \ldots, s..$$

Note that $a_{i,j}(\theta_1) = -a_{j,i}(\theta_1)$ and $a_{i,i}(\theta_1) = a_{s+1,s+1}(\theta_1) = 0$.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; p = 3; n1 = 10; n2 = 12
>>> dx = dec.roy_pdf_cdf_sf(x, p, n1, n2); mx = mpm.roy_pdf_cdf_sf(x, p, n1, n2);
>>> ix = ipm.roy_pdf_cdf_sf(x, p, n1, n2); fx = fpm.roy_pdf_cdf_sf(x, p, n1, n2);
>>> gx = gmp.roy_pdf_cdf_sf(x, p, n1, n2); ax = apm.roy_pdf_cdf_sf(x, p, n1, n2)
>>> mpm.show([dx, mx, ix])
dec:  9.72730704058195372049161324674614667467 6E-1
mpm:  9.72730704058195372049161324674614667467 6e-1
ipm:  9.72730704058195372049161324674614667467 6e-1 (5.901e-40%)
fpm:  9.72730704058195E-01
gmp:  9.72730704058195372049161324674614667467 6E-01
ipm:  9.72730704058195372049161324674614667467 6e-1 (5.901e-40%)
```

14.2 Infinite series algorithms for selected functions and distributions

This section provides additional finite sums algorithms.

14.2.1 Incomplete gamma function, continued fractions (Peizer)

`ctx.gamma_peizer_cdf_sf_pdf(`a, x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

See also Peizer and Pratt [465].

The integral can be calculated using continued fractions. This method provides rigorous error bounds for for `ipm.` and `apm.`.

If $a - \frac{1}{2} > x$ let P be a left tail. Then

$$P = \frac{M^b}{\Gamma(b+1)} e^{-M} \frac{1}{(1 + u_1/(v_1 + u_2/(v_2 + u_3/(v_3 + \ldots))))}$$

where $M = x, b = a, u_1 = -M, u_{2j} = jM, u_{2j+1} = -(b+j)M, v_j = b + j, j = 1, 2, \ldots$

The approximants obtained by terminating at u_k/v_k for $k = 1, 2, 5, 6, 9, 10, \ldots$ decrease monotonically toward P and those for $k = 0, 3, 4, 7, 8, 11, 12, \ldots$ increase monotonically toward P as long as $k \leq 2B - 1$, where B is the integer part of b; for $k \geq 2B - 2$, the even approximants approach P monotonically from above and the odd ones from below if B is even, vice versa if B is odd. The monotonicity is strict throughout except that, if b is an integer, the fraction terminates at $k = 2b - 2$.

If $a - \frac{1}{2} \leq x$ let Q be a right tail. Then

$$Q = \frac{M^{b-1}}{\Gamma(b)} e^{-M} \frac{1}{(1 + u_1/(v_1 + u_2/(v_2 + u_3/(v_3 + \ldots))))}$$

where $M = x, b = a, u_{2j-1} = j - b, v_{2j-1} = M, u_{2j} = j, v_{2j} = 1, j = 1, 2, \ldots$

The approximants obtained by terminating at u_k/v_k approach Q strictly monotonically from above for

$k = 1, 2, 5, 6, 9, 10, \ldots$ and from below for $k = 0, 3, 4, 7, 8, 11, 12, \ldots$

> **Note**
>
> While the algorithm given above seems to offer an easy way to calculate the function in interval arithmetic, one needs to remember that forward recurrence of continuous fractions is entirely unstable in interval arithmetic (see Cuyt *et al.* [189], pages 154 - 159). Therefore one first needs to evaluate the algorithm using forward recurrence in floating point arithmetic to determine the index of the (likely) highest required approximant, and then again using backward recurrence to actually evaluate it in interval arithmetic.

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; a = 10.4; x = 8.7
>>> dx = dec.gamma_peizer_cdf_sf_pdf(a, x); mx = mpm.gamma_peizer_cdf_sf_pdf(a, x);
>>> ix = ipm.gamma_peizer_cdf_sf_pdf(a, x); fx = fpm.gamma_peizer_cdf_sf_pdf(a, x);
>>> gx = gmp.gamma_peizer_cdf_sf_pdf(a, x); ax = apm.gamma_peizer_cdf_sf_pdf(a, x)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.2.2 Incomplete gamma function, asymptotic expansion (Paris)

`ctx.gamma_paris_cdf_sf(a, x)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

This method does not provide rigorous error bounds for for `ipm.` and `apm.`.

[Paris2016` gives the following expansion (for source code, see Sub demoParis() in DistXArb):

Let $m = 0, 1, 2...$ and $\chi = (z-a)/\sqrt{z}$. Then, when $\Re(z-a) \geq 0$, we have the expansion

$$\Gamma(a,z) = z^{a-\frac{1}{2}} e^{-z} \left(d_0(\chi) \sum_{k=0}^{m} \frac{A_k(\chi)}{z^{k/2}} - \sum_{k=1}^{m} \frac{B_k(\chi)}{z^{k/2}} + O\left(z^{-(m+1)/2}\right) \right)$$

as $z \to \infty$ in the sector $|\arg z| < \frac{1}{2}\pi$.

Let $m = 0, 1, 2...$ and $\chi = (z-a)/\sqrt{z}$. Then, when $\Re(z-a) \leq 0$, we have the expansion

$$\gamma(a,z) = z^{a-\frac{1}{2}} e^{-z} \left(d_0(-\chi) \sum_{k=0}^{m} \frac{A_k(\chi)}{z^{k/2}} + \sum_{k=1}^{m} \frac{B_k(\chi)}{z^{k/2}} + O\left(z^{-(m+1)/2}\right) \right)$$

as $z \to \infty$ in the sector $|\arg z| < \frac{1}{2}\pi$.

$$A_k(\chi) = \sum_{j=0}^{k} (-1)^j S_3(k+2j, j) p_{k+2j}(\chi), \quad B_k(\chi) = \sum_{j=0}^{k} (-1)^j S_3(k+2j, j) q_{k+2j}(\chi).$$

$$d_0(\chi) = \sqrt{\pi/2}\, e^{\chi^2/2}\, \operatorname{erfc}(\chi/\sqrt{2})$$

$$p_0(\chi) = 1, \quad p_1(\chi) = -\chi, \quad p_{k+1}(\chi) = \frac{1}{k+1}\left(p_{k-1}(\chi) - \chi p_k(\chi)\right), \quad k \geq 1$$

$$q_0(\chi) = 0, \quad q_1(\chi) = -1, \quad q_{k+1}(\chi) = \frac{1}{k+1}\left(q_{k-1}(\chi) - \chi q_k(\chi)\right), \quad k \geq 1$$

The coefficients $S_3(k, j)$ are the 3-associated Stirling numbers of the second kind. They are determined recursively:

$$S_3(n+1, k) = k\, S_3(n, k) + \binom{n}{2} S_3(n-2, k-1)$$

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; a = 1.0E5; x = 1.001E5
>>> dx = dec.gamma_paris_cdf_sf(a, x); mx = mpm.gamma_paris_cdf_sf(a, x)
>>> ix = ipm.gamma_paris_cdf_sf(a, x); fx = fpm.gamma_paris_cdf_sf(a, x)
>>> gx = gmp.gamma_paris_cdf_sf(a, x); ax = apm.gamma_paris_cdf_sf(a, x)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.2.3 Incomplete beta function, continued fractions (Peizer)

`ctx.beta_peizer_cdf_sf_pdf(a, b, q, p)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The integral can be calculated using continued fractions (see Peizer and Pratt [465]). This method provides rigorous error bounds for for `ipm.` and `apm.`. By transforming the problem if necessary (using the equation $I_x(a,b) = 1 - I_{1-x}(b,a)$), let P be a lefttail probability with $b - \frac{1}{2} \le (a+b-1)(1-x)$. Then

$$P = I_x(a,b) = \binom{n}{a} p^{b-1} q^a \frac{1}{(1 + u_1/(v_1 + u_2/(v2 + u3/(v_3 + \cdots))))}, \quad \text{where}$$

$$p = (1-x), \quad q = x, \quad n = a+b-1, \quad u_1 = \frac{-(b-1)q}{p}, \quad u_{2j} = \frac{j(n+j)q}{p},$$

$$u_{2j+1} = \frac{-(a+j)(b-j-1)q}{p}, \quad v_j = a+j, \quad j = 1, 2, \ldots$$

The approximants obtained by terminating at u_k/v_k for $k = 1, 2, 5, 6, 9, 10, \ldots$ decrease monotonically toward P and those for $k = 0, 3, 4, 7, 8, 11, 12, \ldots$ increase monotonically toward P as long as $k \le 2B - 1$, where B is the integer part of b; for $k \ge 2B - 2$, the even approximants approach P monotonically from above and the odd ones from below if B is even, vice versa if B is odd. The monotonicity is strict throughout except that, if b is an integer, the fraction terminates at $k = 2b - 2$.

For a combined fraction see Tretter and Walster [787].

Note

While the algorithm given above seems to offer an easy way to calculate the function in interval arithmetic, one needs to remember that forward recurrence of continuous fractions is entirely unstable in interval arithmetic (see Cuyt *et al.* [189], pages 154 - 159). Therefore one first needs to evaluate the algorithm using forward recurrence in floating point arithmetic to determine the index of the (likely) highest required approximant, and then again using backward recurrence to actually evaluate it in interval arithmetic.

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; a = 8.3; b = 10.4; x = 0.7
>>> dx = dec.beta_peizer_cdf_sf_pdf(a, b, x); mx = mpm.beta_peizer_cdf_sf_pdf(a, b, x)
>>> ix = ipm.beta_peizer_cdf_sf_pdf(a, b, x); fx = fpm.beta_peizer_cdf_sf_pdf(a, b, x)
>>> gx = gmp.beta_peizer_cdf_sf_pdf(a, b, x); ax = apm.beta_peizer_cdf_sf_pdf(a, b, x)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.2.4 Noncentral χ^2 distribution, pdf, cdf and sf (Boost)

`ctx.chi2_nc_benton_cdf_sf`(*x*, *n*, *lambda1*, *cdf=True*, ***options*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns $\text{cdf}_X(x)$, the cumulative distribution function, for `cdf=True`, or $\text{sf}_X(x)$, the survival function, for `cdf=False`, of a random variable X, noncentral chi-squared distribution with degrees of freedom $n > 0$, noncentrality parameter λ_1, and support interval $(0, +\infty)$. See also Wikipedia [1164], MathWorld [808], Patnaik [464], Penev and Raykov [466], Wang and Gray [793], Winterbottom [1454], BoostMath [64], Witkovský [1474], Johansson [386].

Noncentral CDF: Infinite series in terms of the central cdf:

The cdf of a noncentral chi-square variable with n degrees of freedom and λ is given by

$$F_{\chi^2}(n, x; \lambda) = e^{-\lambda/2} \sum_{j=0}^{\infty} \frac{(\lambda/2)^j}{j!} F_{\chi^2}(n + 2 + j, x)$$

where $F_{\chi^2}(n,\cdot)$ is the cdf of the (central) χ^2 distribution.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = 12; nu = 10; l1 = 30
>>> dx = dec.chi2_nc_benton_cdf_sf(x, nu, l1); mx = mpm.chi2_nc_benton_cdf_sf(x, nu, l1)
>>> ix = ipm.chi2_nc_benton_cdf_sf(x, nu, l1); fx = fpm.chi2_nc_benton_cdf_sf(x, nu, l1)
>>> gx = gmp.chi2_nc_benton_cdf_sf(x, nu, l1); ax = apm.chi2_nc_benton_cdf_sf(x, nu, l1)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.2.5 Noncentral Student t distribution, pdf, cdf and sf (Boost)

`ctx.student_t_nc_benton_cdf_sf`(*x*, *nu*, *delta*, *cdf=True*, ***options*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns $\mathrm{cdf}_X(x)$, the cumulative distribution function, for `cdf=True`, or $\mathrm{sf}_X(x)$, the survival function, for `cdf=False`, of a random variable X, following a noncentral t-distribution with $n>0$ degrees of freedom, noncentrality parameter δ, and support interval $(-\infty,+\infty)$. See also Wikipedia [1165], MathWorld [810], BoostMath [66], Benton and Krishnamoorthy [30], Broda and Paolella [160], Owen [454], Wang and Gray [793], Witkovský [1461], Johansson [386].

The pdf of a variable following a noncentral t-distribution with n degrees of freedom and noncentrality parameter δ is given by (boost_math)

$$f_{t'}(n,x,\delta)=\frac{nt}{n^2+2nt^2+t^4}+\frac{1}{2}\sum_{i=0}^{\infty}P_iI'_x\left(i+\frac{1}{2},\frac{n}{2}\right)+\frac{\delta}{\sqrt{2}}Q_iI'_x\left(i+1,\frac{n}{2}\right),\quad\text{and}$$

$I'_x(\cdot,\cdot)$ denotes the density of the beta distribution (see section ref{BetaDistributionDensity}), and P_i and Q_i are defined in equation (eq:NonCentralTSeriesCoeff).

The cdf of a variable following a noncentral t-distribution with n degrees of freedom and noncentrality parameter δ is given by (Benton, 2003)

$$F_{t'}(n,x,\delta)=\Phi(-\delta)+\frac{1}{2}\sum_{i=0}^{\infty}P_iI_x\left(i+\frac{1}{2},\frac{n}{2}\right)+\frac{\delta}{\sqrt{2}}Q_iI_x\left(i+1,\frac{n}{2}\right),\quad\text{and}$$

$$1-F_{t'}(n,x,\delta)=\frac{1}{2}\sum_{i=0}^{\infty}P_iI_y\left(\frac{n}{2},i+\frac{1}{2}\right)+\frac{\delta}{\sqrt{2}}Q_iI_y\left(\frac{n}{2},i+1\right),\quad\text{where}$$

$$\lambda=\tfrac{1}{2}\delta^2;\quad P_i=\frac{e^{-\lambda}\lambda^i}{i!};\quad Q_i=\frac{e^{-\lambda}\lambda^i}{\Gamma(i+3/2)};\quad x=\frac{t^2}{n+t^2};\quad y=1-x,$$

$I_x(\cdot,\cdot)$ denotes the (normalized) incomplete beta function, and $\Phi(\cdot)$ denotes the cdf of the normal distribution.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = 20.2; nu = 10; d = 30
>>> dx = dec.student_t_nc_benton_cdf_sf(x, n, d); mx = mpm.student_t_nc_benton_cdf_sf(x, n, d)
>>> ix = ipm.student_t_nc_benton_cdf_sf(x, n, d); fx = fpm.student_t_nc_benton_cdf_sf(x, n, d)
>>> gx = gmp.student_t_nc_benton_cdf_sf(x, n, d); ax = apm.student_t_nc_benton_cdf_sf(x, n, d)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
```

(continues on next page)

(continued from previous page)

```
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.2.6 Noncentral Beta distribution, pdf, cdf and sf (Boost)

`ctx.beta_nc_benton_cdf_sf(`*x*, *a*, *b*, *lambda1*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns simulataneously $\text{pdf}_X(x)$, the probability density function, $\text{cdf}_X(x)$, the cumulative distribution function, and $\text{sf}_X(x)$, the survival function, of a random variable X, following a noncentral Beta distribution with shape parameters a and b, noncentrality parameter λ_1 and the support interval $(0, 1)$. See also Wikipedia [1163], BoostMath [63], Wang and Gray [793], Witkovský [1473], Johansson [386].

CDF: Infinite Series:

The cdf of a variable following a (singly) noncentral F-distribution with n and m degrees of freedom and noncentrality parameter λ_1 and is given by

$$\Pr(F \le x) = F_{F'}(x; m, n, \lambda) = e^{-\lambda} \sum_{j=0}^{\infty} \frac{(\lambda/2)^j}{j!} F(m + 2j, n, x)$$

where $F_F(\cdot)$ denotes the cdf of the central F-distribution.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.3'; a = '10'; b = '20'; l1 = '30'
>>> dx = dec.beta_nc_benton_cdf_sf(x, a, b, l1); mx = mpm.beta_nc_benton_cdf_sf(x, a, b, l1)
>>> ix = ipm.beta_nc_benton_cdf_sf(x, a, b, l1); fx = fpm.beta_nc_benton_cdf_sf(x, a, b, l1)
>>> gx = gmp.beta_nc_benton_cdf_sf(x, a, b, l1); ax = apm.beta_nc_benton_cdf_sf(x, a, b, l1)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.2.7 Noncentral F distribution, pdf, cdf and sf (Benton)

`ctx.fisher_f_nc_benton_cdf_sf(`*x*, *m*, *n*, *lambda1*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns simulataneously $\text{pdf}_X(x)$, the probability density function, $\text{cdf}_X(x)$, the cumulative distribution function, and $\text{sf}_X(x)$, the survival function, of a random variable X, following a noncentral F distribution with degrees of freedom m and n, noncentrality parameter λ_1 and the support interval $(0, \infty)$. See also Wikipedia [1163], BoostMath [63], Wang and Gray [793], Witkovský [1473], Johansson [386].

CDF: Infinite Series:

The cdf of a variable following a (singly) noncentral F-distribution with n and m degrees of freedom and noncentrality parameter λ_1 and is given by

$$\Pr(F \le x) = F_{F'}(x; m, n, \lambda) = e^{-\lambda} \sum_{j=0}^{\infty} \frac{(\lambda/2)^j}{j!} F(m + 2j, n, x)$$

where $F_F(\cdot)$ denotes the cdf of the central F-distribution.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.3'; m = '10'; n = '20'; l1 = '30'
>>> dx = dec.fisher_f_nc_benton_cdf_sf(x,m,n,l1); mx = mpm.fisher_f_nc_benton_cdf_sf(x,m,n,l1)
>>> ix = ipm.fisher_f_nc_benton_cdf_sf(x,m,n,l1); fx = fpm.fisher_f_nc_benton_cdf_sf(x,m,n,l1)
>>> gx = gmp.fisher_f_nc_benton_cdf_sf(x,m,n,l1); ax = apm.fisher_f_nc_benton_cdf_sf(x,m,n,l1)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.2.8 Pearson's ρ distribution, cdf and sf (Hotelling's series)

`ctx.`**`pearson_rho_nc_ht_cdf`**`(`r, N, *rho*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns simulaneously $\mathrm{cdf}_X(r)$, the cumulative distribution function, and $\mathrm{sf}_X(r)$, the survival function, of a random variable X, following the distribution of Pearson's rho (the distribution of the sample correlation coefficient), with sample size $N \geq 3$, noncentrality parameter $\rho \in (-1, +1)$ and support interval $(-1, +1)$. See also Wikipedia [1168], MathWorld [797], Hotelling [375], Guenther [355], Winterbottom [1454], Winterbottom [1455], Odeh [426], Ruben [493], Subrahmaniam and Subrahmaniam [508].

Hotelling (1953) defines $Q_N(r;\rho) = \Pr(\rho < R < r)$ and develops $Q_N(r;\rho)$ in the following uniformly convergent series for $-1 < \rho < r < 1$.

$$Q_N(r,\rho) = K_1 \sum_{j=0}^{\infty} \frac{(1 \cdot 3 \cdots (2j-1))^2 S_j}{j! 2^{2j} \cdot (2N+1) \cdots (2N+2j-1)},$$

where K_1 and S_j are defined in equations (ref{eq:PearsonRho_K1}) and (ref{eq:PearsonRho_Hotelling_Sj}), respectively. From the relationships

$$P_N(r,\rho) = 1 - F_R(r,N;\rho) = Q_N(1,\rho) - Q_N(r,\rho)$$

$$P_N(-1,\rho) = 1$$

$$F_R(r,N;\rho) = 1 - P_N(r,\rho)$$

$$F_R(-r,N;-\rho) = 1 - F_R(r,N;\rho)$$

we can compute $F_N(r;\rho)$ for any value of r and ρ with $-1 \leq r \leq 1$, $-1 < \rho < 1$. Hotelling shows that the error committed by truncating the series at any point is less than $\frac{2}{1-|\rho|}$ times the last term used. However, it is important to note that the series converges very slowly for small values of N, and in this case a large number of terms must be computed.

$$S_j = \sum_{k=0}^{j} \binom{j}{k} (-1)^k \, \tfrac{1}{2}(1-\rho^2)^k \, 2^{j-k} N_k$$

$$N_k = \sum_{s=0}^{\infty} \frac{\Gamma\left(\frac{3}{2}-k\right)}{\Gamma\left(\frac{3}{2}-k-s\right) s!} \cdot I\left(\tfrac{1}{2}(s+1), \tfrac{1}{2}(n-1), \frac{(r-\rho)^2}{(1-\rho r)^2}\right),$$

where $I(a,b;x)$ denotes the Incomplete Beta Function (see section ref{sec:IncompleteBetaFunction}). Hotelling shows that in the evaluation of N_k for large s the absolute value of the ratio of the term of order $(s+1)$ to the term of order s is bounded by $|\rho(r-\rho)/(1-\rho r)|$, so that the series converges rapidly.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; r = '0.3'; N = '10'; rho = '0.1'
>>> dx = dec.pearson_rho_nc_ht_cdf(r, N, rho); mx = mpm.pearson_rho_nc_ht_cdf(r, N, rho)
>>> ix = ipm.pearson_rho_nc_ht_cdf(r, N, rho); fx = fpm.pearson_rho_nc_ht_cdf(r, N, rho)
>>> gx = gmp.pearson_rho_nc_ht_cdf(r, N, rho); ax = apm.pearson_rho_nc_ht_cdf(r, N, rho)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.2.9 Pearson's ρ distribution, cdf and sf (Guenther's series)

`ctx.pearson_rho_nc_gt_cdf`(r, N, *rho*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns simulaneously $\text{cdf}_X(r)$, the cumulative distribution function, and $\text{sf}_X(r)$, the survival function, of a random variable X, following the distribution of Pearson's rho (the distribution of the sample correlation coefficient), with sample size $N \geq 3$, noncentrality parameter $\rho \in (-1, +1)$ and support interval $(-1, +1)$. See also Wikipedia [1168], MathWorld [797], Hotelling [375], Guenther [355], Winterbottom [1454], Winterbottom [1455], Odeh [426], Ruben [493], Subrahmaniam and Subrahmaniam [508].

The functions uses an algorithm by Guenther [355], :

Guenther writes $P_N(r;\rho) = \Pr(R > 0) - \Pr(0 < R < r)$ and develops $\Pr(0 < R < r)$ in an infinite series involving the Incomplete Beta Function denoted by $I(a, b; x)$. The result is

$$\Pr(R > 0) = \tfrac{1}{2}\left(1 + \text{sgn}(\rho) \cdot I\left(\tfrac{1}{2}(N-1), \tfrac{1}{2}; \rho^2\right)\right)$$

$$\begin{aligned} \Pr(0 < R < r) \quad &= \textstyle\sum_{j=0}^{\infty} K_1(j) \cdot I\left(\tfrac{1}{2}(N-2), \tfrac{1}{2}(2j+1); r^2\right) \\ &+ \textstyle\sum_{j=0}^{\infty} K_2(j) \cdot I\left(\tfrac{1}{2}(N-2), j+1; r^2\right) \end{aligned} \tag{14.3}$$

where $K_1(j), K_2(j)$ are defined recursively by

$$K_1(0) = \tfrac{1}{2}\left(1-\rho^2\right)^{\frac{1}{2}(N-1)}, \quad K_1(j) = \frac{2j+N-3}{2j}\rho^2 K_1(j-1)$$

$$K_2(0) = \frac{\Gamma\left(\frac{1}{2}N\right)}{\sqrt{\pi}\,\Gamma\left(\frac{1}{2}(N-1)\right)}\rho\left(1-\rho^2\right)^{\frac{1}{2}(N-1)}, \quad K_2(j) = \frac{2j+N-2}{2j+1}\rho^2 K_2(j-1)$$

These equations are used together to give $P_N(r;\rho)$. Guenther also obtains error bounds for truncating the infinite series.

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; r = '0.3'; N = '10'; rho = '0.1'
>>> dx = dec.pearson_rho_nc_gt_cdf(r, N, rho); mx = mpm.pearson_rho_nc_gt_cdf(r, N, rho)
>>> ix = ipm.pearson_rho_nc_gt_cdf(r, N, rho); fx = fpm.pearson_rho_nc_gt_cdf(r, N, rho)
>>> gx = gmp.pearson_rho_nc_gt_cdf(r, N, rho); ax = apm.pearson_rho_nc_gt_cdf(r, N, rho)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

14.2.10 Fisher's R^2 distribution, cdf and sf (Boost, Benton)

`ctx.fisher_r2_gd2_cdf`(*x*, *p*, *N*, *rho2*, ***options*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns $\text{pdf}_X(x)$, the probability density function (pdf) of a random variable X, following the distribution of Fisher's R^2 (the distribution of the square of the sample multiple correlation coefficient), with $p \geq 1$ predictor variables, sample size $N \geq p + 2$, noncentrality parameter $\rho^2 \in (0, 1)$ and the support interval $(0, 1)$. See also Lee [407], Lee [408], Gurland [357], Gurland and Milton [359], Gurland and Asiribo [358], Muirhead [418], Benton and Krishnamoorthy [30], Fisher [336], Gatsonis and Sampson [343].

Density: Infinite series:

The density function of the multiple sample correlation coefficient is given by citep{Ding_1996,Benton_2003}

$$f_{R^2}(x; p, N, \rho^2) = \sum_{i=0}^{\infty} f_{\text{NegBin}}\left((N-1)/2, i; 1-\rho^2\right) \times f_{\text{Beta}}\left(x; \tfrac{1}{2}(p-1)+i, \tfrac{1}{2}(N-p)\right)$$

where $f_{\text{NegBin}}(\cdot)$ denotes the pmf of the negative binomial distribution and $f_{\text{Beta}}(\cdot)$ denotes the pdf of the Beta distribution

CDF: Infinite series:

The CDF of the multiple sample correlation coefficient is given by citep{Ding_1996,Benton_2003}

$$F_{R^2}(x; p, N, \rho^2) = \sum_{i=0}^{\infty} f_{\text{NegBin}}\left((N-1)/2, i; 1-\rho^2\right) \times F_{\text{Beta}}\left(x; \tfrac{1}{2}(p-1)+i, \tfrac{1}{2}(N-p)\right)$$

where $f_{\text{NegBin}}(\cdot)$ denotes the pmf of the negative binomial distribution and $F_{\text{Beta}}(\cdot)$ denotes the CDF of the Beta distribution

CDF: Series of Gurland:

$$\Pr[F \leq x` = F_{R^2}(x; a, b, \rho^2) = \frac{b^m}{a^{n/2}} \sum_{j=0}^{\infty} c_j I(m+j, k, y) \quad \text{where}$$

$$z = F(p-1), \quad y = \frac{z}{z+(N-p)}, \quad a = \frac{1}{1-\rho^2}, \quad n = (N-p)/2, \quad m = (p-1)/2$$

$$c_0 = 1, \quad c_j = \frac{c_{j-1}(-n/2-j+1)\rho^2}{j}$$

An example:

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '0.3'; p = '3'; N = '20'; rho2 = '0.6'
>>> dx = dec.fisher_r2_gd2_cdf(x, p, N, rho2); mx = mpm.fisher_r2_gd2_cdf(x, p, N, rho2)
>>> ix = ipm.fisher_r2_gd2_cdf(x, p, N, rho2); fx = fpm.fisher_r2_gd2_cdf(x, p, N, rho2)
>>> gx = gmp.fisher_r2_gd2_cdf(x, p, N, rho2); ax = apm.fisher_r2_gd2_cdf(x, p, N, rho2)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.1138774234168360557146160900744444149943E-1
mpm:  3.1138774234168360557146160900744444149943e-1
ipm:  3.1138774234168360557146160900744444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.1138774234168360557146160900744444149943E-01
ipm:  3.1138774234168360557146160900744444149943e-1 (4.055e-38%)
```

14.3 Finite series for lattice distributions, based on factorial moments

14.3.1 Fréchet's formula for calculating the pmf from the factorial moments

`ctx.frechet_pmf`(x, N)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

For lattice distributions, the pmf can be calculated from the factorial moments as follows (see Fréchet 1940, 1943):

$$\mathrm{pmf}_X(x) = \sum_{j=x}^{M} (-1)^{x+j} \binom{j}{x} \frac{\mu'_{[j]}}{j!}, \tag{14.4}$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

An example (pmf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; k = '34'; n = '100'; p = '0.5';
>>> dx = dec.binomial_pmf(k, n, p); mx = mpm.binomial_pmf(k, n, p)
>>> ix = ipm.binomial_pmf(k, n, p); fx = fpm.binomial_pmf(k, n, p)
>>> gx = gmp.binomial_pmf(k, n, p); ax = apm.binomial_pmf(k, n, p)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  4.581052772872401245494881411350031838140E-4
mpm:  4.581052772872401245494881411350031838141e-4
ipm:  4.581052772872401245494881411350031838141e-4 (4.894e-39%)
fpm:  4.58105277287240E-04
gmp:  4.581052772872401245494881411350031838141E-04
ipm:  4.581052772872401245494881411350031838141e-4 (4.894e-39%)
```

14.3.2 Laurent's formula for calculating the cdf from the factorial moments

`ctx.laurent_cdf`(x, N)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

For lattice distributions, the cdf can be calculated from the factorial moments as follows (see Laurent 1965):

$$\mathrm{cdf}_X(x) = \sum_{j=x}^{M} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!}, \tag{14.5}$$

$$\mathrm{sf}_X(x) = 1 - \sum_{j=x}^{M} (-1)^{x+j} \binom{j-1}{x-1} \frac{\mu'_{[j]}}{j!}, \tag{14.6}$$

where $\mu'_{[j]}$ is the j^{th} factorial moment.

An example (cdf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; k = '34'; n = '100'; p = '0.5';
>>> dx = dec.binomial_cdf(k, n, p); mx = mpm.binomial_cdf(k, n, p)
>>> ix = ipm.binomial_cdf(k, n, p); fx = fpm.binomial_cdf(k, n, p)
>>> gx = gmp.binomial_cdf(k, n, p); ax = apm.binomial_cdf(k, n, p)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  8.949651957434262965349913219213965127993E-4
mpm:  8.949651957434262965349913219213965127993e-4
ipm:  8.949651957434262965349913219213965127993e-4 (6.263e-40%)
fpm:  8.94965195743426E-04
gmp:  8.949651957434262965349913219213965127993E-04
ipm:  8.949651957434262965349913219213965127993e-4 (6.263e-40%)
```

14.3.3 Binomial distribution, pmf, cdf

`ctx.binomial_fm_pmf`(x, N)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The pmf and cdf are calculated using equations (14.4) and (14.5), respectively.

If a random variable X has a binomial distribution with success probability $p \in [0, 1]$ and number of trials n, then the factorial moments of X are

$$\mathrm{E}\big[(X)_r\big] = \binom{n}{r} p^r r! = (n)_r p^r = \frac{n!}{(n-r)!} p^r, \tag{14.7}$$

where by convention, $\binom{n}{r}$ is understood to be zero if $r > n$.

An example (pmf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; k = '34'; n = '100'; p = '0.5';
>>> dx = dec.binomial_pmf(k, n, p); mx = mpm.binomial_pmf(k, n, p)
>>> ix = ipm.binomial_pmf(k, n, p); fx = fpm.binomial_pmf(k, n, p)
>>> gx = gmp.binomial_pmf(k, n, p); ax = apm.binomial_pmf(k, n, p)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  4.581052772872401245494881411350031838140E-4
mpm:  4.581052772872401245494881411350031838141e-4
ipm:  4.581052772872401245494881411350031838141e-4 (4.894e-39%)
fpm:  4.58105277287240E-04
gmp:  4.581052772872401245494881411350031838141E-04
ipm:  4.581052772872401245494881411350031838141e-4 (4.894e-39%)
```

An example (cdf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; k = '34'; n = '100'; p = '0.5';
>>> dx = dec.binomial_cdf(k, n, p); mx = mpm.binomial_cdf(k, n, p)
>>> ix = ipm.binomial_cdf(k, n, p); fx = fpm.binomial_cdf(k, n, p)
>>> gx = gmp.binomial_cdf(k, n, p); ax = apm.binomial_cdf(k, n, p)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  8.949651957434262965349913219213965127993E-4
mpm:  8.949651957434262965349913219213965127993e-4
ipm:  8.949651957434262965349913219213965127993e-4 (6.263e-40%)
fpm:  8.94965195743426E-04
gmp:  8.949651957434262965349913219213965127993E-04
ipm:  8.949651957434262965349913219213965127993e-4 (6.263e-40%)
```

14.3.4 Classical hypergeometric distribution, pmf, cdf

`ctx.hypergeo_fm_pmf`(x, N)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The pmf and cdf are calculated using equations (14.4) and (14.5), respectively.

If a random variable X has a hypergeometric distribution with population size N, number of success states $K \in \{0, ..., N\}$ in the population, and draws $n \in \{0, ..., N\}$, then the factorial moments of X are

$$\mathrm{E}\big[(X)_r\big] = \frac{\binom{K}{r}\binom{n}{r} r!}{\binom{N}{r}} = \frac{(K)_r (n)_r}{(N)_r}. \tag{14.8}$$

An example (pmf):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; k = 2; n = 3; K = 10; N = 30
>>> dx = dec.hypergeo_pmf(k, n, K, N); mx = mpm.hypergeo_pmf(k, n, K, N)
>>> ix = ipm.hypergeo_pmf(k, n, K, N); fx = fpm.hypergeo_pmf(k, n, K, N)
>>> gx = gmp.hypergeo_pmf(k, n, K, N); ax = apm.hypergeo_pmf(k, n, K, N)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  1.889036358989624579176488249803987683086E-2
mpm:  1.889036358989624579176488249803987683086e-2
ipm:  1.889036358989624579176488249803987683087e-2 (1.785e-37%)
fpm:  1.88903635898961E-02
gmp:  1.889036358989624579176488249803987683086E-02
ipm:  1.889036358989624579176488249803987683087e-2 (1.785e-37%)
```

An example (cdf):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; k = 2; n = 3; K = 10; N = 30
>>> dx = dec.hypergeo_cdf(k, n, K, N); mx = mpm.hypergeo_cdf(k, n, K, N)
>>> ix = ipm.hypergeo_cdf(k, n, K, N); fx = fpm.hypergeo_cdf(k, n, K, N)
>>> gx = gmp.hypergeo_cdf(k, n, K, N); ax = apm.hypergeo_cdf(k, n, K, N)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  9.484909869856324992497161107476089538277E-1
mpm:  9.484909869856324992497161107476089538277e-1
ipm:  9.484909869856324992497161107476089538276e-1 (6.051e-40%)
fpm:  9.48490986985633E-01
gmp:  9.484909869856324992497161107476089538277E-01
ipm:  9.484909869856324992497161107476089538276e-1 (6.051e-40%)
```

14.3.5 Wilcoxon distribution, pmf, cdf

`ctx.`**`wilcoxon_fm_pmf`**(x, N)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The pmf and cdf are calculated using equations (14.4) and (14.5), respectively.

The factorial moments are calculated from the cumulants (see *factorial_moments_from_cumulants()*), and the cumulants are given by

$$\kappa_{2j} = \frac{2^{2j}(2^{2j}-1)B_{2j}}{2j}\sum_{i=1}^{N} r_i^{2j} = \frac{2^{2j}(2^{2j}-1)B_{2j}}{2j}\,\frac{B_{2j+1}(N+1)-B_{2j+1}}{2j+1}, \tag{14.9}$$

where $\kappa_1 = N(N+1)/4$, $\kappa_{2j+1} = 0$ for $j \geq 1$, and B_{2j} and $B_{2j}(x)$ are the Bernoulli numbers and polynomials, respectively, of degree $2j$.

An example (pmf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10'; N = '30';
>>> dx = dec.wilcoxon_pmf(x, N); mx = mpm.wilcoxon_pmf(x, N)
>>> ix = ipm.wilcoxon_pmf(x, N); fx = fpm.wilcoxon_pmf(x, N)
>>> gx = gmp.wilcoxon_pmf(x, N); ax = apm.wilcoxon_pmf(x, N)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  2.503894462302860479794674544578743383256E-2
mpm:  2.503894462302860479794674544578743383257e-2
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
fpm:  2.50389446230288E-02
gmp:  2.503894462302860479794674544578743383257E-02
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
```

An example (cdf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10'; N = '30';
>>> dx = dec.wilcoxon_cdf(x, N); mx = mpm.wilcoxon_cdf(x, N)
>>> ix = ipm.wilcoxon_cdf(x, N); fx = fpm.wilcoxon_cdf(x, N)
>>> gx = gmp.wilcoxon_cdf(x, N); ax = apm.wilcoxon_cdf(x, N)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  2.503894462302860479794674544578743383256E-2
mpm:  2.503894462302860479794674544578743383257e-2
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
fpm:  2.50389446230288E-02
gmp:  2.503894462302860479794674544578743383257E-02
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
```

14.3.6 Mann-Whitney distribution, pmf, cdf

`ctx.`**`mannwhitney_fm_pmf`**(x, N)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The pmf and cdf are calculated using equations (14.4) and (14.5), respectively.

The factorial moments are calculated from the cumulants (see *factorial_moments_from_cumulants()*), and the cumulants are given by

$$\kappa_{2j} = \frac{B_{2j}}{2j(2j+1)}\left[B_{2j+1}(N_2+N_1+1) + B_{2j+1} - B_{2j+1}(N_1+1) - B_{2j+1}(N_2+1)\right] \tag{14.10}$$

and $\kappa_{2j+1} = 0$, $j \geq 1$, and B_{2j} and $B_{2j}(x)$ are the Bernoulli numbers and polynomials, respectively, of degree $2j$.

An example (pmf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10'; m = '20'; n = '20';
>>> dx = dec.mannwhitney_pmf(x, m, n); mx = mpm.mannwhitney_pmf(x, m, n)
>>> ix = ipm.mannwhitney_pmf(x, m, n); fx = fpm.mannwhitney_pmf(x, m, n)
>>> gx = gmp.mannwhitney_pmf(x, m, n); ax = apm.mannwhitney_pmf(x, m, n)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  2.503894462302860479794674544578743383256E-2
mpm:  2.503894462302860479794674544578743383257e-2
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
fpm:  2.50389446230288E-02
gmp:  2.503894462302860479794674544578743383257E-02
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
```

An example (CDF):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10'; m = '20'; n = '20';
>>> dx = dec.mannwhitney_cdf(x, m, n); mx = mpm.mannwhitney_cdf(x, m, n)
>>> ix = ipm.mannwhitney_cdf(x, m, n); fx = fpm.mannwhitney_cdf(x, m, n)
>>> gx = gmp.mannwhitney_cdf(x, m, n); ax = apm.mannwhitney_cdf(x, m, n)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  2.503894462302860479794674544578743383256E-2
mpm:  2.503894462302860479794674544578743383257e-2
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
fpm:  2.50389446230288E-02
gmp:  2.503894462302860479794674544578743383257E-02
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
```

14.3.7 Kendall distribution, pmf, cdf

`ctx.kendall_fm_pmf`(x, N)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The pmf is calculated using equation (14.4).

The factorial moments are calculated from the cumulants (see *factorial_moments_from_cumulants()*), and the cumulants are given by

$$\kappa_{2j}(T_N) = \frac{B_{2j}}{2j}\sum_{s=1}^{N} s^{2j} = \frac{B_{2j}}{2j}\left[\frac{B_{2j+1}(N+1) - B_{2j+1}}{2j+1} - N\right], \quad \text{and} \tag{14.11}$$

$$\kappa_{2j+1}(W_N) = 0, \quad \text{for } j \geq 1.$$

B_{2j} and $B_{2j}(x)$ are the Bernoulli numbers and polynomials, respectively, of degree $2j$.

An example (pmf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10'; N = '30';
>>> dx = dec.kendall_tau_pmf(x, N); mx = mpm.kendall_tau_pmf(x, N)
>>> ix = ipm.kendall_tau_pmf(x, N); fx = fpm.kendall_tau_pmf(x, N)
>>> gx = gmp.kendall_tau_pmf(x, N); ax = apm.kendall_tau_pmf(x, N)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  2.503894462302860479794674544578743383256E-2
mpm:  2.503894462302860479794674544578743383257e-2
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
fpm:  2.50389446230288E-02
gmp:  2.503894462302860479794674544578743383257E-02
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
```

An example (cdf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10'; N = '30';
>>> dx = dec.kendall_tau_cdf(x, N); mx = mpm.kendall_tau_cdf(x, N)
>>> ix = ipm.kendall_tau_cdf(x, N); fx = fpm.kendall_tau_cdf(x, N)
>>> gx = gmp.kendall_tau_cdf(x, N); ax = apm.kendall_tau_cdf(x, N)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  2.503894462302860479794674544578743383256E-2
mpm:  2.503894462302860479794674544578743383257e-2
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
fpm:  2.50389446230288E-02
gmp:  2.503894462302860479794674544578743383257E-02
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
```

14.3.8 Jonckheere-Terpsta T distribution, pmf, cdf

`ctx.jterpsta_fm_pmf`(x, N)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The pmf and cdf are calculated using equations (14.4) and (14.5), respectively.

The factorial moments are calculated from the cumulants (see *factorial_moments_from_cumulants()*), and the cumulants are given by

$$\kappa_{2j} = \frac{B_{2j}}{2j(2j+1)}\left[B_{2j+1}(N+1) + (k-1)B_{2j+1} - \sum_{i=1}^{k} B_{2j+1}(n_i+1)\right] \tag{14.12}$$

where $\kappa_1 = M/2$, $\kappa_{2j+1} = 0$ for $j \geq 1$, and B_{2j} and $B_{2j}(x)$ are the Bernoulli numbers and polynomials, respectively, of degree $2j$.

An example (pmf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10'; k = '4'; n = '10';
>>> dx = dec.jterpsta_s_pmf(x, k, n); mx = mpm.jterpsta_s_pmf(x, k, n)
>>> ix = ipm.jterpsta_s_pmf(x, k, n); fx = fpm.jterpsta_s_pmf(x, k, n)
>>> gx = gmp.jterpsta_s_pmf(x, k, n); ax = apm.jterpsta_s_pmf(x, k, n)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  2.503894462302860479794674544578743383256E-2
mpm:  2.503894462302860479794674544578743383257e-2
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
fpm:  2.50389446230288E-02
gmp:  2.503894462302860479794674544578743383257E-02
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
```

An example (cdf):

```
>>> from xlcalcnet import dec, mpm, ipm, fpm, gmp, apm
>>> mpm.dps = 40; x = '10'; k = '4'; n = '10';
>>> dx = dec.jterpsta_s_cdf(x, k, n); mx = mpm.jterpsta_s_cdf(x, k, n)
>>> ix = ipm.jterpsta_s_cdf(x, k, n); fx = fpm.jterpsta_s_cdf(x, k, n)
>>> gx = gmp.jterpsta_s_cdf(x, k, n); ax = apm.jterpsta_s_cdf(x, k, n)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  2.503894462302860479794674544578743383256E-2
mpm:  2.503894462302860479794674544578743383257e-2
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
fpm:  2.50389446230288E-02
gmp:  2.503894462302860479794674544578743383257E-02
ipm:  2.503894462302860479794674544578743383257e-2 (3.582e-39%)
```

14.4 Efficient integration of bell-shaped functions

quad_bellshaped(*f*, **points*, ***kwargs*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Computes a single, double or triple integral over a given, 1D interval, 2D rectangle, or 3D cuboid, providing optionalle a guaranteed error bound.

14.4.1 Introduction

Some text explaining search for optimum and using 4 standard subintervals

14.4.2 Error function

`ctx.`**quad_erf**(a, x)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Returns the error function of erf(z). See also BoostMath [74], Wikipedia [1234] , MathWorld [843], NIST [775], Mpmath [545]. The error function defined by

$$\mathrm{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-z^2} \mathrm{d}t,$$

References for integration with erf(z) as argument:

https://math.stackexchange.com/questions/712434/erfaib-error-function-separate-into-real-and-imaginary-part

https://www.wolframalpha.com/input/?i=integrate+exp%28x%5E2%29

http://fredrikj.net/blog/2016/03/taking-the-error-out-of-the-error-function/

Bounds on the complex error function:

$$\mathrm{erf}(x + iy) = \mathrm{erf}x \pm ie^{-x^2}\mathrm{erfi}y \pm e^{-x^2}\mathrm{erfi}y.$$

14.4.3 Lower non-normalised incomplete gamma function, $\gamma(a, x)$ (Continued fractions)

`ctx.`**quad_gamma_lower**(a, x)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Returns the real lower non-normalised incomplete gamma function $\gamma(a, x)$. See also Wikipedia [1223], MathWorld [848], NIST [459], BoostMath [81], Mpmath [550]. The function is defined as

$$\gamma(a, x) = \int_0^x t^{a-1} e^{-t} \mathrm{d}t$$

for $a \geq 0$ and $x \geq 0$.

14.4.4 Real upper non-normalised incomplete gamma function, $\Gamma(a, x)$ (Quadrature)

`ctx.`**quad_gamma_upper**(a, x)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Returns the real lower non-normalised incomplete gamma function $\Gamma(a, x)$. See also Wikipedia [1223], MathWorld [848], NIST [459], BoostMath [81], Mpmath [550]. The function is defined as:

$$\Gamma(a, x) = \int_x^\infty t^{a-1} e^{-t} \mathrm{d}t$$

for $a \geq 0$ and $x \geq 0$.

14.4.5 Normalised incomplete beta function, $I_x(a, b)$

`ctx.quad_ibeta`(a, b, x)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Returns the normalised incomplete beta function $I_x(a, b)$ for $a > 0$, $b > 0$, and $0 \le x \le 1$:

$$I_x(a,b) = \frac{B_x(a,b)}{B(a,b)}, \quad B_x(a,b) = \int_0^x t^{a-1}(1-t)^{b-1}\mathrm{d}t.$$

See also Wikipedia [1222], MathWorld [854], NIST [458], BoostMath [79], BoostMath [87], Mpmath [549].

```
>>> from mpformula import mp4
>>> mp4.dps = 30;
>>> matrix(2)
matrix(
[['0.0', '0.0'],
 ['0.0', '0.0']])
>>> matrix(2, 3)
```

14.4.6 Non-central chi-square cdf and sf (Chow)

`ctx.chi_squared_nc_quad_cdf`(n, x, *lambda*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

[Chou1985` gives the following representation for $n \ge 2$ and $\lambda \ge 0$:

$$F_{\chi^2}(n, x; \lambda) = \frac{2^{(1-n)/2}\sqrt{2\pi}}{\Gamma((n-1)/2))} \int_0^x y^{(n-3)/2}\phi\left(\sqrt{y}\right)\left[\Phi\left(\sqrt{x-y}-\sqrt{\lambda}\right) - \Phi\left(-\sqrt{x-y}-\sqrt{\lambda}\right)\right]\mathrm{d}y$$

where $\phi(\cdot)$ denotes the pdf of the normal distribution (see section ref{sec:NormalDistribution_pdf}) and $\Phi(\cdot)$ denotes the cdf of the normal distribution (see section ref{sec:NormalDistribution_CDF}).

14.4.7 Marcum Q function

`ctx.marcumq1`(a, b)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The Marcum Q-function is defined as

$$Q(a,b) = \int_b^\infty x \exp\left(-\frac{x^2+a^2}{2}\right) I_0(ax)\,\mathrm{d}x$$

where $b \ge 0$, $a > 0$ and I_0 is the modified Bessel function of first kind of order 0. It is a special case of the generalized Marcum $Q_m(a, b)$ function with $m = 1$. It is made available as a separate function because it can be calculated in a particularly simple way.

Special cases are

$$Q(a,0) = 1, \quad Q(0,b) = e^{-b^2/2}, \quad Q(a,a) = \tfrac{1}{2}\left(1 + \frac{I_0(a^2)}{\exp(a^2)}\right)$$

The function can also be evaluated as

$$Q(a,b) = \begin{cases} H(a,b), & a < b, \\ \frac{1}{2} + H(a,a), & a = b, \\ 1 + H(a,b) & a > b, \end{cases}$$

where

$$H(a,b) = \frac{1}{\pi}\exp\left(-\frac{a^2+b^2}{2}\right)\int_0^\pi G(t)\exp(ab\cos(t))\mathrm{d}t, \quad \text{and}$$

$$G(t) = \begin{cases} \frac{1}{2}, & t = 0, a = b \\ \frac{1-z\cos(t)}{1-2z\cos(t)+z^2} & \text{otherwise, with } z = \frac{a}{b} \end{cases}$$

Available in Amath.

See also: https://en.wikipedia.org/wiki/Marcum_Q-function

See also: Short, 2012

See also Morales-Jimenez, 2013

See Marcum, 1960

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; a = '3.7'; b = '10.3'
>>> dx = dec.marcumq(a, b); mx = mpm.marcumq(a, b); ix = ipm.marcumq(a, b)
>>> mpm.show([dx, mx, ix])
dec:  3.4653770308369288610422290755224371774 83E-11
mpm:  3.4653770308369288610422290755224371774 83e-11
ipm:  3.4653770308369288610422290755224371774 86e-11 (9.641e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; a = '3.7'; b = '10.3'
>>> fx = fpm.marcumq(a, b); gx = gmp.marcumq(a, b); ax = apm.marcumq(a, b)
>>> mpm.show([gx, fx, ax])
gmp:  3.4653770308369288610422290755224371774 83E-11
fpm:  3.46537703083691E-11
apm:  3.4653770308369288610422290755224371774 86e-11 (9.641e-40%)
```

14.4.8 Owen's T function

`ctx.owent`(*h*, *a*, *boost=True*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns Owen's T function $T(h,a)$. See also Wikipedia [1353], MathWorld [1013], Owen [453], and Patefield and Tand [463]. The function is defined by

$$T(h,a) = \frac{1}{2\pi}\int_0^a f(x)\mathrm{d}x = \frac{a}{4\pi}\int_{-1}^1 f(ax)\mathrm{d}x, \quad f(x) = \frac{e^{-\frac{1}{2}h^2(1+x^2)}}{1+x^2}, \quad (-\infty < h, a < +\infty)\,.$$

It has the following properties:

$$T(h,0) = 0$$

$$T(0,a) = \frac{1}{2\pi}\arctan(a)$$

$$T(-h,a) = T(h,a)$$

$$T(h,-a) = -T(h,a)$$

$$T(h,a) + T(ah,\frac{1}{a}) = \frac{1}{2}\left(\Phi(h)+\Phi(ah)\right) - \Phi(h)\Phi(ah) \quad \text{if} \quad a \geq 0$$

$$T(h,a) + T(ah,\frac{1}{a}) = \frac{1}{2}\left(\Phi(h)+\Phi(ah)\right) - \Phi(h)\Phi(ah) - \frac{1}{2} \quad \text{if} \quad a < 0$$

$$T(h,1) = \frac{1}{2}\Phi(h)\left(1-\Phi(h)\right).$$

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; h = 3.7; a = 10.3
>>> dx = dec.owent(h, a); mx = mpm.owent(h, a); ix = ipm.owent(h, a)
>>> mpm.show([dx, mx, ix])
dec:  5.3899866738694168468734716435226525868 97E-5
mpm:  5.3899866738694130740667774679411113471 02e-5
ipm:  5.3899866738694130740667774679411113471 02e-5 (6.5e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; h = 3.7; a = 10.3
>>> fx = fpm.owent(h, a); gx = gmp.owent(h, a); ax = apm.owent(h, a)
>>> mpm.show([gx, fx, ax])
gmp:  5.3899866738694168468734716435226525868 97E-05
fpm:  5.38998667386941E-05
apm:  5.3899866738694130740667774679411113471 02e-5 (6.5e-40%)

>>> fx = fpm.owent(h, a); mpm.show([fx]) # boost
fpm:  5.38998667386941E-05
>>> fx = fpm.owent(h, a, False); mpm.show([fx]) # amath
fpm:  5.38998667386941E-05
```

14.5 Verified numerical integration

14.5.1 Error function

`ctx.quad_erf_verified`(a, x)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Returns the error function of $\mathrm{erf}(z)$. See also BoostMath [74], Wikipedia [1234] , MathWorld [843], NIST [775], Mpmath [545]. The error function defined by

$$\mathrm{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-z^2} \mathrm{d}t,$$

Bounds on the complex error function:

$$\mathrm{erf}(x + iy) = \mathrm{erf}x \pm ie^{-x^2}\mathrm{erfi}y \pm e^{-x^2}\mathrm{erfi}y.$$

14.5.2 Lower non-normalised incomplete gamma function, $\gamma(a, x)$ (Continued fractions)

`ctx.quad_gamma_lower_verified`(a, x)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Returns the real lower non-normalised incomplete gamma function $\gamma(a, x)$. See also Wikipedia [1223], MathWorld [848], NIST [459], BoostMath [81], Mpmath [550]. The function is defined as

$$\gamma(a, x) = \int_0^x t^{a-1} e^{-t} \mathrm{d}t$$

for $a \geq 0$ and $x \geq 0$.

14.5.3 Real upper non-normalised incomplete gamma function, $\Gamma(a, x)$ (Quadrature)

`ctx.quad_gamma_upper_verified`(a, x)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Returns the real lower non-normalised incomplete gamma function $\Gamma(a, x)$. See also Wikipedia [1223], MathWorld [848], NIST [459], BoostMath [81], Mpmath [550]. The function is defined as:

$$\Gamma(a, x) = \int_x^\infty t^{a-1} e^{-t} \mathrm{d}t$$

for $a \geq 0$ and $x \geq 0$.

14.5.4 Normalised incomplete beta function, $I_x(a, b)$

`ctx.quad_ibeta_verified`(a, b, x)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

Returns the normalised incomplete beta function $I_x(a, b)$ for $a > 0$, $b > 0$, and $0 \leq x \leq 1$:

$$I_x(a, b) = \frac{B_x(a, b)}{B(a, b)}, \quad B_x(a, b) = \int_0^x t^{a-1}(1 - t)^{b-1} \mathrm{d}t.$$

See also Wikipedia [1222], MathWorld [854], NIST [458], BoostMath [79], BoostMath [87], Mpmath [549].

```
>>> from mpformula import mp4
>>> mp4.dps = 30;
>>> matrix(2)
matrix(
[['0.0', '0.0'],
 ['0.0', '0.0']])
>>> matrix(2, 3)
```

14.5.5 Non-central chi-square cdf and sf (Chow)

`ctx.chi_squared_nc_quad_cdf_verified`(*n*, *x*, *lambda*)

where `ctx` is `ipm`, `dec`, `mpm`, or `gmp`.

[Chou1985` gives the following representation for $n \geq 2$ and $\lambda \geq 0$:

$$F_{\chi^2}(n, x; \lambda) = \frac{2^{(1-n)/2}\sqrt{2\pi}}{\Gamma((n-1)/2))} \int_0^x y^{(n-3)/2} \phi(\sqrt{y}) \left[\Phi\left(\sqrt{x-y} - \sqrt{\lambda}\right) - \Phi\left(-\sqrt{x-y} - \sqrt{\lambda}\right)\right] \mathrm{d}y$$

where $\phi(\cdot)$ denotes the pdf of the normal distribution (see section ref{sec:NormalDistribution_pdf}) and $\Phi(\cdot)$ denotes the cdf of the normal distribution (see section ref{sec:NormalDistribution_CDF}).

14.5.6 Marcum Q function

`ctx.marcumq1_verified`(*a*, *b*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

The Marcum Q-function is defined as

$$Q(a, b) = \int_b^\infty x \exp\left(-\frac{x^2 + a^2}{2}\right) I_0(ax)\, \mathrm{d}x$$

where $b \geq 0$, $a > 0$ and I_0 is the modified Bessel function of first kind of order 0. It is a special case of the generalized Marcum $Q_m(a, b)$ function with $m = 1$. It is made available as a separate function because it can be calculated in a particularly simple way.

Special cases are

$$Q(a, 0) = 1, \quad Q(0, b) = e^{-b^2/2}, \quad Q(a, a) = \tfrac{1}{2}\left(1 + \frac{I_0(a^2)}{\exp(a^2)}\right)$$

The function can also be evaluated as

$$Q(a, b) = \begin{cases} H(a, b), & a < b, \\ \frac{1}{2} + H(a, a), & a = b, \\ 1 + H(a, b) & a > b, \end{cases}$$

where

$$H(a, b) = \frac{1}{\pi} \exp\left(-\frac{a^2 + b^2}{2}\right) \int_0^\pi G(t) \exp(ab\cos(t)) \mathrm{d}t, \quad \text{and}$$

$$G(t) = \begin{cases} \frac{1}{2}, & t = 0, a = b \\ \frac{1 - z\cos(t)}{1 - 2z\cos(t) + z^2} & \text{otherwise, with } z = \frac{a}{b} \end{cases}$$

Available in Amath.

See also: https://en.wikipedia.org/wiki/Marcum_Q-function

See also: Short, 2012

See also Morales-Jimenez, 2013

See Marcum, 1960

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; a = '3.7'; b = '10.3'
>>> dx = dec.marcumq(a, b); mx = mpm.marcumq(a, b); ix = ipm.marcumq(a, b)
>>> mpm.show([dx, mx, ix])
dec:  3.4653770308369288610422290755224371774 83E-11
mpm:  3.4653770308369288610422290755224371774 83e-11
ipm:  3.4653770308369288610422290755224371774 86e-11 (9.641e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; a = '3.7'; b = '10.3'
>>> fx = fpm.marcumq(a, b); gx = gmp.marcumq(a, b); ax = apm.marcumq(a, b)
>>> mpm.show([gx, fx, ax])
gmp:  3.4653770308369288610422290755224371774 83E-11
fpm:  3.46537703083691E-11
apm:  3.4653770308369288610422290755224371774 86e-11 (9.641e-40%)
```

14.5.7 Owen's T function

`ctx.owent_verified`(*h*, *a*, *boost=True*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns Owen's T function $T(h, a)$. See also Wikipedia [1353], MathWorld [1013], Owen [453], and Patefield and Tand [463].

See also: https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.owens_t.html#scipy.special.owens_t

The function is defined by

$$T(h,a) = \frac{1}{2\pi}\int_0^a f(x)\mathrm{d}x = \frac{a}{4\pi}\int_{-1}^1 f(ax)\mathrm{d}x, \quad f(x) = \frac{e^{-\frac{1}{2}h^2(1+x^2)}}{1+x^2}, \quad (-\infty < h, a < +\infty)\,.$$

It has the following properties:

$$T(h,0) = 0$$

$$T(0,a) = \frac{1}{2\pi}\arctan(a)$$

$$T(-h,a) = T(h,a)$$

$$T(h,-a) = -T(h,a)$$

$$T(h,a) + T(ah,\frac{1}{a}) = \frac{1}{2}\left(\Phi(h)+\Phi(ah)\right) - \Phi(h)\Phi(ah) \quad \text{if} \quad a \geq 0$$

$$T(h,a) + T(ah,\frac{1}{a}) = \frac{1}{2}\left(\Phi(h)+\Phi(ah)\right) - \Phi(h)\Phi(ah) - \frac{1}{2} \quad \text{if} \quad a < 0$$

$$T(h,1) = \frac{1}{2}\Phi(h)\left(1-\Phi(h)\right).$$

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; h = 3.7; a = 10.3
>>> dx = dec.owent(h, a); mx = mpm.owent(h, a); ix = ipm.owent(h, a)
>>> mpm.show([dx, mx, ix])
dec:  5.3899866738694168468734716435226525868 97E-5
mpm:  5.3899866738694130740667774679411113471 02e-5
ipm:  5.3899866738694130740667774679411113471 02e-5 (6.5e-40%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; h = 3.7; a = 10.3
>>> fx = fpm.owent(h, a); gx = gmp.owent(h, a); ax = apm.owent(h, a)
>>> mpm.show([gx, fx, ax])
gmp:  5.3899866738694168468734716435226525868 97E-05
```

(continues on next page)

(continued from previous page)

```
fpm:  5.389986673869 41E-05
apm:  5.3899866738694130740667774679 41111347102e-5 (6.5e-40%)

>>> fx = fpm.owent(h, a); mpm.show([fx]) # boost
fpm:  5.389986673869 41E-05
>>> fx = fpm.owent(h, a, False); mpm.show([fx]) # amath
fpm:  5.389986673869 41E-05
```

CHAPTER

FIFTEEN

PMF VECTORS

15.1 Basic discrete (lattice) distribution functions

15.1.1 Poisson distribution, pmf vector

`ctx.poisson_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Poisson distribution. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix. See also Wikipedia [1189], MathWorld [835], BoostMath [84] , and Witkovský [1492].

$\mathrm{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Poisson distribution is given by:

$$\mathrm{pmf}_X(x) = \frac{\mu^k}{k!}e^{-\mu}.$$

The following recursions are used for the pmf:

$$\Pr(X = k+1|n) = \frac{\lambda}{k+1}\Pr(X = k|n)$$

$$\Pr(X = k-1|n) = \frac{k}{\lambda}\Pr(X = k|n)$$

15.1.2 Binomial distribution, pmf vector

`ctx.binomial_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the binomial distribution. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix. See also Wikipedia [1173], MathWorld [816], BoostMath [71] , and Witkovský [1482].

$\mathrm{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following an binomial distribution is given by:

$$\mathrm{pmf}_X(x) = \binom{n}{k}p^k(1-p)^{n-k} = f_{\mathrm{Beta}}(k+1, n-k+1, p)/(n+1).$$

and $f_{\mathrm{Beta}}(\cdot)$ denote the pdf of the central beta distribution. The following recursions are used for the pmf:

$$\Pr(X = k+1|n) = \frac{(n-k)P}{(k+1)Q}\Pr(X = k|n),$$

$$\Pr(X = k-1|n) = \frac{kQ}{(n-k+1)P}\Pr(X = k|n).$$

15.1.3 Negative binomial distribution, pmf vector

`ctx.negative_binomial_pmf_vector(x, n, lambda)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the negative binomial distribution. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix. See also Wikipedia [1188], MathWorld [832], BoostMath [83] , and Witkovský [1489].

$\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following an negative binomial distribution is given by:

$$\text{pmf}_X(x) = \frac{\Gamma(n+k)}{K!\Gamma(n)} P^n (1-P)^k.$$

and $f_{\text{Beta}}(\cdot)$ denotes the pdf of the central beta distribution. The following recursions are used for the pmf:

$$\Pr(X = k+1|n) = \frac{(n+k)(1-P)}{(k+1)} \Pr(X = k|n)$$

$$\Pr(X = k-1|n) = \frac{k}{(n-k+1)(1-P)} \Pr(X = k|n)$$

15.1.4 Hypergeometric distribution, pmf vector

`ctx.hypergeometric_pmf_vector(x, n, lambda)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the hypergeometric distribution. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1179], MathWorld [824], BoostMath [78], Berkopec [31], and *dist_hypergeometric()*.

$\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following an hypergeometric distribution is given by:

$$\text{pmf}_X(x) = \frac{\binom{n_1}{k}\binom{n_2}{n-k}}{\binom{n_1+n_2}{n}}, \quad (n, n_1, n_2 \geq 0; n \leq n_1 + n_2).$$

The following recursions are used for the pmf:

$$f(k+1) = \frac{(n_1-k)(n-k)}{(k+1)(n_2-n+k+1)} f(k)$$

$$f(k-1) = \frac{k(n_2-n+k)}{(n_1-k+1)(n-k+1)} f(k)$$

15.1.5 Noncentral hypergeometric distribution (Fisher), pmf vector

`ctx.hypergeo_nc_pmf_vector(x, n, lambda)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the noncentral hypergeometric distribution (Fisher). The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix. See also: Wikipedia [1221], Johnson *et al.* [389] page 293.

$$\text{pmf}_X(x) = \text{h}(x; n_1, m_1, N, \theta) = \frac{\binom{n_1}{x}\binom{n_2}{m_1-x}\theta^x}{\binom{n_2}{m_1}{}_2F_1(-n_1, -m_1; n_2+1-m_1; \theta)},$$

where $\theta = p_1 q_2/(q_1 p_2)$ and $\max(0, m_1 - n_2) \leq x \leq \min(n_1, m_1)$.

The following recursions are used for the PMF (see Wikipedia):

$$\mathrm{h}(x; n_1, m_1, N, \theta) = \frac{(m_1 - x + 1)(n_1 - x + 1)\theta}{x(m_2 - n_1 + x)} \mathrm{h}(x - 1; n_1, m_1, N, \theta)$$

15.2 Discrete (lattice) distribution functions related to (stratified) rank tests

15.2.1 Sign test distribution (under H_0), pmf vector

`ctx.signtest_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Sign test distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1195], vandeWiel [789], Bennett [29] and Zimmermann [1501], and *dist_wilcoxon()*.

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Wilcoxon signed rank T distribution. The null distribution can be calculated as follows: Let $p_N(w)$ denote the probability $\Pr[W_N = w]$ in a sample of size N. Then the following recurrence relation holds (see Zimmermann [1501]) :

$$p_N(w) = \tfrac{1}{2}\left(p_{N-1}(w) + p_{N-1}(w-N)\right).$$

15.2.2 Wilcoxon T distribution (under H_0), pmf vector

`ctx.wilcoxon_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Wilcoxon distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1195], Fellingham and Stoker [300], vandeWiel [789], Bennett [29] and Zimmermann [1501], and *dist_wilcoxon()*.

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Wilcoxon signed rank T distribution. The null distribution can be calculated as follows: Let $p_N(w)$ denote the probability $\Pr[W_N = w]$ in a sample of size N. Then the following recurrence relation holds (see Zimmermann [1501]) :

$$p_N(w) = \tfrac{1}{2}\left(p_{N-1}(w) + p_{N-1}(w-N)\right).$$

15.2.3 Wilcoxon T distribution (under Bennett alternatives), pmf vector

`ctx.wilcoxon_bennett_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Wilcoxon T distribution under Bennett alternatives. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1195], Fellingham and Stoker [300], vandeWiel [789], Bennett [29] and Zimmermann [1501], and *dist_wilcoxon()*.

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Wilcoxon signed rank T distribution. The null distribution can be calculated as follows: Let $p_N(w)$ denote the probability $\Pr[W_N = w]$ in a sample of size N. Then the following recurrence relation holds (see Zimmermann [1501]) :

$$p_N(w) = \tfrac{1}{2}\left(p_{N-1}(w) + p_{N-1}(w-N)\right).$$

15.2.4 Kendall S (or tau) distribution (under H_0), pmf vector

ctx.**kendall_tau_pmf_vector**(*x*, *n*, *lambda*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Returns the vector of all pmf values of the Kendall S (or tau) distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix. See also Wikipedia [1182], Noether [424], vandeWiel [789], and *dist_kendall()*.

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Kendall tau distribution.

The null distribution can be calculated as follows: Let $p_N(t) = \Pr[T_N = t]$. Then the following recurrence relation holds:

$$p_N(t) = p_N(t-1) + [p_{N-1}(t) - p_{N-1}(t-N)]/N,$$

where $p_N(t) = 0$ for $t < 0$ or $t > N(N-1)/2$, and $p_N(0) = 1/N!$.

15.2.5 Mann-Whitney U distribution (under H_0), pmf vector

ctx.**mannwhitney_u_pmf_vector**(*x*, *n*, *lambda*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Returns the vector of all pmf values of the Mann-Whitney U distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1185], Murakami and Kamakura [420], Robillard [482], vandeWiel [789] and Zimmermann [1502], and *dist_mann_whitney()*.

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Mann-Whitney U distribution. The null distribution of the MW test can be calculated as follows: Let $p_{n,m}(u)$ denote the probability that $U = u$ in samples of size n and m. Then (see Zimmermann [1502])

$$(m+n)p_{n,m}(u) = np_{n-1,m}(u-m) + mp_{n,m-1}(u) = np_{n-1,m}(u) + mp_{n,m-1}(u-n)$$

15.2.6 Jonckheere-Terpsta S distribution (under H_0), pmf vector

ctx.**jterpsta_s_pmf_vector**(*x*, *n*, *lambda*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Returns the vector of all pmf values of the Jonckheere-Terpsta S distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], Murakami and Kamakura [420], Robillard [482], vandeWiel [789], and Skillings [501] and *dist_jterpsta()*.

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t-x)$$

where the sum is over all x with positive $p(\cdot)$.

15.2.7 Spearman ρ distribution (under H_0), pmf vector

ctx.**spearman_rho_pmf_vector**(*x*, *n*, *lambda*)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Returns the vector of all pmf values of the Page L distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], vandeWiel [789] and Skillings [501] .

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

15.2.8 Page L distribution (under H_0), pmf vector

`ctx.page_l_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Page L distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], vandeWiel [789], and Skillings [501].

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

15.2.9 Quade L distribution (under H_0), pmf vector

`ctx.quade_l_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Quade L distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], vandeWiel [789], and Skillings [501].

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

15.2.10 Mann-Whitney U distribution (under Lehmann alternatives), pmf vector

`ctx.mannwhitney_u_lehmann_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Lehmann alternatives are of the form $F_1(x) =: cite : t : `F_0(x)]^k$ or $F_1(x) = 1 - [1 - F_0(x)]^k$.

See also: Shorack [497], Shorack [498].

Under Lehmann alternatives, rank order probabilities can be expressed in closed form for $F_1 = (F_0)^k$: let $S_1, \ldots, S_n$ denote the ranks of Y in the combined sample, e.g. $\Pr[S_1 = 3, S_2 = 5)] = P_{3,2}(0, 0, 1, 0, 1)$. Then

$$\Pr[S_1 = s_1, \ldots, , S_n = s_n) = k^n \frac{n!m!}{\Gamma(n + m + 1 + n(k-1))} \prod_{j=1}^{n} \frac{\Gamma(s_j + j(k-1))}{\Gamma(s_j + (j-1)(k-1))}.$$

The exact distribution under this alternative can also be calculated using recurrence relations, similar to the null-distribution (Shorack_1966):

Let $p_{n,m}(u)$ denote the probability that $U = u$ in samples of size n and m. Then

$$(km + n)p_{n,m}(u) = np_{n-1,m}(u - m) + kmp_{n,m-1}(u) \quad \text{for } F_1 = (F_0)^k$$

$$(km + n)p_{n,m}(u) = np_{n-1,m}(u) + kmp_{n,m-1}(u - n) \quad \text{for } F_1 = 1 - (1 - F_0)^k$$

This recursive procedure allows the calculation of the exact noncentral distribution also for larger samples.

15.2.11 Mann-Whitney U distribution (under Milton alternatives), pmf vector

`ctx.`**`mannwhitney_u_milton_pmf_vector`**(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Mann-Whitney U distribution under Milton alternatives. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also: Milton [415]

We consider k continuous random variables X_i with density functions f_i and sample sizes n_i, $i = 1 \ldots k$, and $N = n_1 + \ldots + n_k$.

Let $\mathbf{U} = (U_1, \ldots, U_N), U_1 < \cdots < U_N$, denote the order statistics of the random variables $(X_{1,1}, \ldots, X_{1,n_1}, \ldots, X_{k,1}, \ldots, X_{k,n_k})$, and let $\mathbf{Z} = (Z_1, \ldots, Z_N)$ denote a random vector of integers $1, 2, \ldots, k$, where the i^{th} component $\mathbf{Z}_i$ is i if U_i is an X_i.

If $\mathbf{z} = (z_1, \ldots, z_N)$ is a fixed vector of integers $1, 2, \ldots, k$ (with each i occurring n_i times), the probability of the rank order z, Pr $[\mathbf{Z} = \mathbf{z}]$, is given by

$$P_{n_1,\ldots,n_k}(\mathbf{z}|d) = n_1! \ldots n_k! \int_R \cdots \int \prod_{i=1}^{N} f_i(t_i) dt_i,$$

where the region of integration R is $-\infty < t_1 \leq t_2 \leq \cdots \leq t_N < \infty$

The original algorithm by Milton [415] has been developed for the normal distribution, but it has been found to also work well for the logistic distribution and for Lehmann alternatives, if the reference distribution is normal.

15.3 Discrete (non-lattice) distribution functions related to rank tests

15.3.1 Cochran S distribution (under H_0), pmf vector

`ctx.cochran_s_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Cochran S distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], vandeWiel [789], and Skillings [501].

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

15.3.2 Friedman S distribution (under H_0), pmf vector

`ctx.friedman_s_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Friedman S distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also: https://www.statsdirect.com/help/nonparametric_methods/friedman.htm

See also Wikipedia [1181], vandeWiel [789] and Skillings [501].

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

15.3.3 Quade S distribution (under H_0), pmf vector

`ctx.quade_s_pmf_vector`(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Quade S distribution distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], vandeWiel [789], and Skillings [501] .

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

15.3.4 Kruskal-Wallis H distribution (under H_0), pmf vector

`ctx.`**`kruskal_wallis_h_pmf_vector`**(*x*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the vector of all pmf values of the Kruskal-Wallis H distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], Murakami and Kamakura [420], Robillard [482], vandeWiel [789], and Skillings [501] .

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

CHAPTER

SIXTEEN

FAST APPROXIMATIONS

16.1 Approximations based on the normal distribution

16.1.1 Non-central chi-squared distribution: cdf and sf (Penev)

`ctx.chi2_nc_penev_cdf(`*x*, *nu*, *nc*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the (non-)central chi-squared distribution.

[Penev2000` give the following first and second order Wiener germ approximation:

$$F_{\chi^2}(n,x;\lambda) \approx \Phi\left(\operatorname{sgn}(s)\sqrt{n(s-1)^2(1/(2s)+m-h(1-s)/s)-\log(A(s))+2B(s)/n}\right)$$

$$\text{where } m=\lambda/n; \quad h(y)=\frac{(1-y)\log(1-y)+y-\frac{1}{2}y^2}{y^2}; \quad s=\frac{\sqrt{1+4xm/n}-1}{2m}$$

$$A(s)=\frac{1}{s}-\frac{2}{s}\cdot\frac{h(1-s)}{1+2ms}; \quad B(s)=\frac{(1+3m)^2}{9(1+2m)^3}$$

where $\Phi(\cdot)$ denotes the cdf of the normal distribution (see).

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.cdisn_penev(x, nu, nc); mx = mpm.cdisn_penev(x, nu, nc)
>>> ix = ipm.cdisn_penev(x, nu, nc); fx = fpm.cdisn_penev(x, nu, nc)
>>> gx = gmp.cdisn_penev(x, nu, nc); ax = apm.cdisn_penev(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.2 (Non-central) chi-squared distribution: qtf and isf (Canal)

`ctx.chi2_nc_canal_qtf(`*LeftTail*, *RightTail*, *n*`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the quantile of the (non-)central chi-squared distribution.

Let z_α and $\chi^2_{n,\alpha}$ be the α-quantiles of the standard normal distribution and central chi-square distribution with n the degrees of freedom. For $n=1$ and $n=2$, the following closed form expressions can be used:

$$\chi^2_{1,\alpha}=z^2_\alpha, \quad \chi^2_{2,\alpha}=2\log(1-\alpha)$$

At the extreme left tail of the distribution, for small x, the CDF of a χ^2 variable with n degrees of freedom can be approximated by the density of a χ^2 variable with $n+2$ degrees of freedom:

$$F_{\chi^2}(n,x) \approx 2f_{\chi^2}(n+2,x).$$

The density of a χ^2 variable with $n+2$ degrees of freedom can be inverted in closed form using the Lambert W function, which leads to the following approximation:

$$\chi^2_{n,\alpha} \approx f^{-1}_{\chi^2}(n+2,\alpha) = -2W(t)/a, \quad \text{where}$$

$$a = \frac{1}{(n+2)/2-1}, \quad k = \log(\Gamma((n+2)/2), \quad d = a - \log(1-\alpha) + k, \quad t = -ae^{p+d}$$

This approximation is used for $|t| < 0.1$, and the Lambert W function is approximated as

$$W(x) \approx x - x^2 + \tfrac{3}{2}x^3 - \tfrac{8}{3}x^4 - \tfrac{125}{24}x^5.$$

Otherwise, the quantile is approximated by inverting a formula proposed by Canal [170]:

$$\chi^2_{n,\alpha} \approx n\left(\frac{1}{2} + \frac{t}{2} - \frac{3}{2t}\right)^6, \quad \text{where}$$

$$t = \left(-5 + 2L + 2\sqrt{13 - 5L + L^2}\right)^{1/3}, \quad L = 6\left(m + s\left(z_\alpha + a(z_\alpha^2 - 1) - a^2(2z_\alpha^3 - 5z_\alpha)\right)\right)$$

$$m = \frac{5}{6} - \frac{1}{9n} - \frac{7}{648n^2} - \frac{25}{2187n^3}, \quad s^2 = \frac{1}{18n} + \frac{1}{162n^2} - \frac{37}{11664n^3}, \quad a = \frac{1}{162\sqrt{2n^3}} = \gamma_1/6.$$

The noncentral quantile is approximated as

$$\chi^2_{n,\lambda,\alpha} \approx (1+b)\chi^2_{n_1,\alpha}, \quad \text{where } n_1 = \frac{(n+\lambda)^2}{n+2\lambda}, \quad b = \frac{\lambda}{n+\lambda}$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.chi2_nc_canal_qtf(x, nu, nc); mx = mpm.chi2_nc_canal_qtf(x, nu, nc)
>>> ix = ipm.chi2_nc_canal_qtf(x, nu, nc); fx = fpm.chi2_nc_canal_qtf(x, nu, nc)
>>> gx = gmp.chi2_nc_canal_qtf(x, nu, nc); ax = apm.chi2_nc_canal_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.1138774234168360557146160900 74444149943E-1
mpm:  3.1138774234168360557146160900 74444149943e-1
ipm:  3.1138774234168360557146160900 74444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.1138774234168360557146160900 74444149943E-01
ipm:  3.1138774234168360557146160900 74444149943e-1 (4.055e-38%)
```

16.1.3 Gamma distribution: qtf and isf (Canal)

`ctx.gamma_canal_qtf`(*LeftTail*, *RightTail*, *a*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the quantile of the gamma distribution.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.gamma_canal_qtf(x, nu, nc); mx = mpm.gamma_canal_qtf(x, nu, nc)
>>> ix = ipm.gamma_canal_qtf(x, nu, nc); fx = fpm.gamma_canal_qtf(x, nu, nc)
>>> gx = gmp.gamma_canal_qtf(x, nu, nc); ax = apm.gamma_canal_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.4 F distribution: qtf and isf (Davis)

`ctx.`**`fisher_f_davis_qtf`**(*LeftTail*, *RightTail*, *m*, *n*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the quantile of the F distribution.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.fisher_f_davis_qtf(x, nu, nc); mx = mpm.fisher_f_davis_qtf(x, nu, nc)
>>> ix = ipm.fisher_f_davis_qtf(x, nu, nc); fx = fpm.fisher_f_davis_qtf(x, nu, nc)
>>> gx = gmp.fisher_f_davis_qtf(x, nu, nc); ax = apm.fisher_f_davis_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.5 Beta distribution: qtf and isf (Davis)

`ctx.`**`beta_davis_qtf`**(*LeftTail*, *RightTail*, *a*, *b*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the quantile of the beta distribution.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.beta_davis_qtf(x, nu, nc); mx = mpm.beta_davis_qtf(x, nu, nc)
>>> ix = ipm.beta_davis_qtf(x, nu, nc); fx = fpm.beta_davis_qtf(x, nu, nc)
>>> gx = gmp.beta_davis_qtf(x, nu, nc); ax = apm.beta_davis_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.6 Pearson's rho distribution: pdf (Winterbottom)

`ctx.pearson_rho_wb_pdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the pdf of Pearson's rho distribution. See also Winterbottom [1454], Winterbottom [1455].

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.pearson_rho_wb_pdf(x, nu, nc); mx = mpm.pearson_rho_wb_pdf(x, nu, nc)
>>> ix = ipm.pearson_rho_wb_pdf(x, nu, nc); fx = fpm.pearson_rho_wb_pdf(x, nu, nc)
>>> gx = gmp.pearson_rho_wb_pdf(x, nu, nc); ax = apm.pearson_rho_wb_pdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.7 Pearson's rho distribution: cdf and sf (Winterbottom)

`ctx.pearson_rho_wb_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of Pearson's rho distribution. See also Winterbottom [1454], Winterbottom [1455].

An improved approximation has been found by inverting Winterbottom's approximation for the confidence limits of ρ:

$$F_R(N, r, \rho) \approx \Phi\left(\frac{k}{6} - \frac{2c}{k} - \frac{b}{3}\right), \quad \text{where} \quad m = N - 1,$$

$$a = \frac{1}{12\sqrt{m^3}} + \frac{6r^4 - 3r^2 + 2}{48\sqrt{m^5}}, \quad b = \frac{-r3}{6am^2}, \quad s = \frac{1}{a\sqrt{m}} + \frac{1 + r^2}{4a\sqrt{m^3}} + \frac{11r^4 - 2r^2 + 1}{32a\sqrt{m^5}}$$

$$t = \frac{Z(r) - Z(\rho)}{a} + \frac{r}{2am} + \frac{5r^3 + 9r}{24am^2}, \quad d = t + \frac{bs}{3} - \frac{2b^3}{27}$$

$$c = s - b^2/3, \quad p = \sqrt{|12c^3 + 81d^2|}, \quad k = (108d + 12p)^{1/3}$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.pearson_rho_wb_cdf(x, nu, nc); mx = mpm.pearson_rho_wb_cdf(x, nu, nc)
>>> ix = ipm.pearson_rho_wb_cdf(x, nu, nc); fx = fpm.pearson_rho_wb_cdf(x, nu, nc)
>>> gx = gmp.pearson_rho_wb_cdf(x, nu, nc); ax = apm.pearson_rho_wb_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.8 Pearson's rho distribution: qtf and isf (Winterbottom)

`ctx.pearson_rho_wb_qtf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the qtf of Pearson's rho distribution. See also Winterbottom [1454], Winterbottom [1455].

Asymptotic expansions typically rely on the Fisher z-transform $Z(a) = \text{atanh}(a)$ and its inverse $Z^{-1}(a) = \tanh(a)$.

Let $m = N - 1$ and let $u_\alpha = \Phi^{-1}(\alpha)$ be the lower 100α percentage point of the standard normal distribution. An approximation to the 100α percentage point r_α and the lower $100(1-\alpha)$ confidence limit on ρ, ρ_L, is then obtained by $r_\alpha \approx Z^{-1}(y_1)$, where

$$\begin{aligned} y_1 &= Z(\rho) + \frac{u_\alpha}{\sqrt{m}} + \frac{\rho}{2m} + \frac{u_\alpha^3 + 3(3-\rho^2)u_\alpha}{12\sqrt{m^3}} + \frac{4\rho^3 u_\alpha^2 + 15\rho - \rho^3}{24m^2} \\ &\quad + \frac{u_\alpha^5 + (-60\rho^4 + 30\rho^2 + 80)u_\alpha^3 + (45\rho^4 - 21\rho^2 + 375)u_\alpha}{480\sqrt{m^5}}, \quad \text{and} \end{aligned} \tag{16.1}$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.pearson_rho_wb_qtf(x, nu, nc); mx = mpm.pearson_rho_wb_qtf(x, nu, nc)
>>> ix = ipm.pearson_rho_wb_qtf(x, nu, nc); fx = fpm.pearson_rho_wb_qtf(x, nu, nc)
>>> gx = gmp.pearson_rho_wb_qtf(x, nu, nc); ax = apm.pearson_rho_wb_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.9 Pearson's rho distribution: confidence limit for ρ (Winterbottom)

`ctx.pearson_rho_wb_cl(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the confidence interval (CI) for the noncentrality parameter ρ of Pearson's rho distribution, as decribed in Winterbottom [1455].

We use the Fisher z-transform $Z(a) = \text{atanh}(a)$ and its inverse $Z^{-1}(a) = \tanh(a)$.

Let $m = N - 1$ and let $u_\alpha = \Phi^{-1}(\alpha)$ be the lower 100α percentage point of the standard normal distribution. An approximation to the 100α percentage point r_α and the lower $100(1-\alpha)$ confidence limit on ρ, ρ_L, is then obtained by $\rho_L \approx Z^{-1}(y_2)$, where

$$\begin{aligned} y_2 &= Z(r) + \frac{u_\alpha}{\sqrt{m}} - \frac{r}{2m} + \frac{u_\alpha^3 + 3(1+r^2)u_\alpha}{12\sqrt{m^3}} - \frac{4r^3 u_\alpha^2 + 5r^3 + 9r}{24m^2} \\ &\quad + \frac{u_\alpha^5 + (60r^4 - 30r^2 + 20)u_\alpha^3 + (165r^4 + 30r^2 + 15)u_\alpha}{480\sqrt{m^5}}. \end{aligned} \tag{16.2}$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.pearson_rho_wb_cl(x, nu, nc); mx = mpm.pearson_rho_wb_cl(x, nu, nc)
>>> ix = ipm.pearson_rho_wb_cl(x, nu, nc); fx = fpm.pearson_rho_wb_cl(x, nu, nc)
>>> gx = gmp.pearson_rho_wb_cl(x, nu, nc); ax = apm.pearson_rho_wb_cl(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.10 Singly noncentral t: pdf (Broda)

`ctx.student_t_nc_broda_pdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the pdf of the singly noncentral t distribution, as decribed in Witkovský [1461], using the approximation to the cdf of the singly noncentral t distribution as decribed Broda and Paolella [160].

$$f_{\text{StudentT}}(x,n,\delta) = \frac{n}{x}\left(F_{\text{StudentT}}\left(x\sqrt{(n+2)/n}, n+2, \delta\right) - F_{\text{StudentT}}(x,n,\delta)\right), \quad x \neq 0,$$

$$f_{\text{StudentT}}(x,n,\delta) = \frac{\Gamma((n+1)/2)}{\sqrt{n\pi}\Gamma(n/2)} e^{-\frac{1}{2}\delta^2}$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.student_t_nc_broda_pdf(x, nu, nc); mx = mpm.student_t_nc_broda_pdf(x, nu, nc)
>>> ix = ipm.student_t_nc_broda_pdf(x, nu, nc); fx = fpm.student_t_nc_broda_pdf(x, nu, nc)
>>> gx = gmp.student_t_nc_broda_pdf(x, nu, nc); ax = apm.student_t_nc_broda_pdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.11 Singly noncentral t: cdf, sf (Broda)

`ctx.student_t_nc_broda_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the singly noncentral t distribution, as decribed in Broda and Paolella [160].

Singly noncentral t: A saddlepoint approximation based on the joint cumulant generating function of (x_1, x_2), with $x_1 \sim N(\mu, 1)$ and $x_2 \sim \chi^2(n)$, is possible (see Broda (2007)), and takes the following form:

$$F_{t'}(x;n,\mu) = \Phi(w) + \phi(w)\left(\frac{1}{w} - \frac{d}{u}\right) + O(n^{-3/2}), \quad \text{where}$$

$$s = \left(\mu x + \sqrt{4n(x^2+n) + \mu^2 x^2}\right)/(2x^2+2n),$$

$$t_1 = -\mu + xs, \quad t_2 = -xt_1/(2ns), \quad d = 1/(t_1 s),$$

$$u = \sqrt{(\mu x s + 2n)/(2n)}/s, \quad w = \text{sgn}(x-\mu)\sqrt{-\mu t_1 - 2n\log(s)},$$

and $\Phi(\cdot)$ and $\phi(\cdot)$ denote the cdf and pdf of the normal distribution, respectively.

For $\mu = 0$, these equations simplify to

$$s = \sqrt{n/(x^2+n)}, \quad d = 1/(xs), \quad u = 1/s, \quad w = \text{sgn}(x)\sqrt{-2n\log(s)},$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.student_t_nc_broda_cdf(x, nu, nc); mx = mpm.student_t_nc_broda_cdf(x, nu, nc)
>>> ix = ipm.student_t_nc_broda_cdf(x, nu, nc); fx = fpm.student_t_nc_broda_cdf(x, nu, nc)
>>> gx = gmp.student_t_nc_broda_cdf(x, nu, nc); ax = apm.student_t_nc_broda_cdf(x, nu, nc)
```

(continues on next page)

(continued from previous page)

```
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.11387742341683605571461609007444414994 3E-1
mpm:  3.11387742341683605571461609007444414994 3e-1
ipm:  3.11387742341683605571461609007444414994 4e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.11387742341683605571461609007444414994 3E-01
ipm:  3.11387742341683605571461609007444414994 3e-1 (4.055e-38%)
```

16.1.12 Singly noncentral t: qtf, isf (Harley)

`ctx.student_t_nc_harley_qtf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the qtf of the singly noncentral t distribution, as decribed in Harley [363].

The qtf of the noncentral t-distribution can be approximated by the qtf of Pearson's rho as follows: Let $N = f + 2$ and

$$\rho = \Re\left(\delta\sqrt{\frac{2}{2N - 3 + \delta^2}}\right).$$

Determine $r_{\alpha;\rho;N} = r_\alpha$, using Winterbottom's approximation. Calculate $t_{\alpha;\delta;f} = t_\alpha$ as

$$t_\alpha = \Re\left(\frac{r_\alpha}{1 - r_\alpha^2}\sqrt{\frac{2f(1-\rho^2)}{1-\rho^2}}\right)$$

Note that in these equations both ρ and r_α can assume values > 1. Luckily, Winterbottom's approximation can handle this; we always take only the real part of the functions results to avoid issues with the branch cuts of the complex square root.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.student_t_nc_broda_qtf(x, nu, nc); mx = mpm.student_t_nc_broda_qtf(x, nu, nc)
>>> ix = ipm.student_t_nc_broda_qtf(x, nu, nc); fx = fpm.student_t_nc_broda_qtf(x, nu, nc)
>>> gx = gmp.student_t_nc_broda_qtf(x, nu, nc); ax = apm.student_t_nc_broda_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.11387742341683605571461609007444414994 3E-1
mpm:  3.11387742341683605571461609007444414994 3e-1
ipm:  3.11387742341683605571461609007444414994 4e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.11387742341683605571461609007444414994 3E-01
ipm:  3.11387742341683605571461609007444414994 3e-1 (4.055e-38%)
```

16.1.13 Singly noncentral t: confidence limit for δ (Akahira)

`ctx.student_t_nc_akahira_cl(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the confidence interval (CI) for the noncentrality parameter of the singly noncentral t distribution, as decribed in Akahira *et al.* [6].

Let T be a statistic according to the non-central t-distribution with n degrees of freedom and a non-centrality parameter δ. Then the lower confidence limit $\widehat{\delta}$ of level $1 - \alpha$ and the two-sided confidence interval $(\underline{\delta}, \overline{\delta})$ of the non-centrality parameter δ of level $1 - \alpha$ are given by Akahira *et al.* [6]:

$$\widehat{\delta} = bT - z_\alpha\sqrt{k} + hT^3(z_\alpha^2 - 1)/k,$$

$$\underline{\delta} = bT - z_{\alpha/2}\sqrt{k} + hT^3(z_{\alpha/2}^2 - 1)/k,$$

$$\overline{\delta} = bT + z_{\alpha/2}\sqrt{k} - hT^3(z_{\alpha/2}^2 - 1)/k,$$

$$h = \frac{1}{24}\left(\frac{1}{n^2} + \frac{1}{4n^3}\right) \quad b = \sqrt{\frac{2}{n}}\frac{\Gamma(\frac{1}{2}n + \frac{1}{2})}{\Gamma(\frac{1}{2}n)}$$

where $k = 1 + (1 - b^2)T^2$, and z_α denotes the α-quantile of the normal distribution.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.student_t_nc_akahira_cl(x, nu, nc); mx = mpm.student_t_nc_akahira_cl(x, nu, nc)
>>> ix = ipm.student_t_nc_akahira_cl(x, nu, nc); fx = fpm.student_t_nc_akahira_cl(x, nu, nc)
>>> gx = gmp.student_t_nc_akahira_cl(x, nu, nc); ax = apm.student_t_nc_akahira_cl(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.14 Doubly noncentral t: cdf, sf (Broda)

`ctx.student_t_nc2_broda_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the doubly noncentral t distribution, as decribed in Broda and Paolella [160].

A saddlepoint approximation based on the joint cumulant generating function of (x_1, x_2), with $x_1 \sim N(\mu, 1)$ and $x_2 \sim \chi^2(n, \theta)$, is possible (see Broda (2007)), and takes the following form:

$$F_{t''}(x; n; \mu, \theta) = \Phi(w) + \phi(w)\left(\frac{1}{w} - \frac{d}{u}\right) + O(n^{-3/2}), \quad \text{where}$$

$$a = x^4 + 2nx^2 + n^2, \quad c_2 = (-2x^3\mu - 2xn\mu)/a, \quad c_1 = (x^2\mu^2 - nx^2 - n^2 - \theta n)/a, \quad c_0 = (xn\mu)/a,$$

$$q = \tfrac{1}{3}c_1 - \tfrac{1}{9}c_2^2, \quad r = \tfrac{1}{6}(c_1c_2 - 3c_0) - \tfrac{1}{27}c_2^3,$$

$$s = \sqrt{-4q}\cos\left(\tfrac{1}{3}\arccos\left(r/\sqrt{-q^3}\right)\right) - \tfrac{1}{3}c_2,$$

$$t_1 = -\mu + xs, t_2 = -xt_1/(2ns), \quad d = 1/(t_1 s), \quad \nu = 1/(1 - 2t_2), \quad \alpha = \mu/\sqrt{1 + \theta/n},$$

$$u = \sqrt{(x^2 + 2nt_2)(2n\nu^2 + 4\theta\nu^3) + 4n^2s^2}/(2ns^2), \quad w = \operatorname{sgn}(x - \alpha)\sqrt{-\mu t_1 - n\log(\nu) - 2\theta\nu t_2},$$

and $\Phi(\cdot)$ and $\phi(\cdot)$ denote the cdf and pdf of the normal distribution, respectively.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.student_t_nc2_broda_cdf(x, nu, nc); mx = mpm.student_t_nc2_broda_cdf(x, nu, nc)
>>> ix = ipm.student_t_nc2_broda_cdf(x, nu, nc); fx = fpm.student_t_nc2_broda_cdf(x, nu, nc)
>>> gx = gmp.student_t_nc2_broda_cdf(x, nu, nc); ax = apm.student_t_nc2_broda_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.15 Doubly noncentral t: qtf, isf (Broda)

`ctx.student_t_nc2_broda_qtf(x)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the qtf of the doubly noncentral t distribution, as decribed in Broda and Paolella [160].

Broda_2007_WorkingPaper proposes the following approximation:

$$F_{t''}(t;n;\mu,\theta) \approx F_{t'}(t/c;f;\mu), \quad \text{where}$$

$$f = \frac{7}{2}\left(-1+\sqrt{15-7g^2/h}\right)^{-1}, \quad c = \sqrt{h(1-2/f)}, \quad g = m_1/\sqrt{\mu^2/2}, h = m_2/(1+\mu^2),$$

$$m_1 = \mu\binom{n}{2}^{1/2}\frac{\Gamma((n-1)/2)}{\Gamma(n/2)}{}_1F_1\left(\frac{1}{2},\frac{n}{2},-\frac{\theta}{2}\right)$$

$$m_2 = (1+\mu^2)\frac{n}{n-2}{}_1F_1\left(\frac{2}{2},\frac{n}{2},-\frac{\theta}{2}\right)$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.student_t_nc2_broda_qtf(x, nu, nc); mx = mpm.student_t_nc2_broda_qtf(x, nu, nc)
>>> ix = ipm.student_t_nc2_broda_qtf(x, nu, nc); fx = fpm.student_t_nc2_broda_qtf(x, nu, nc)
>>> gx = gmp.student_t_nc2_broda_qtf(x, nu, nc); ax = apm.student_t_nc2_broda_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.1.16 Spearman's rho, first 8 cumulants (David)

`ctx.spearman_mu8(N)`

where `ctx` is `dec`, `mpm`, `fpm`, `gmp` or `apm`.

Returns an approximation to the central distribution of Spearman's ρ, based on the first 8 moments.

Spearman's ρ is calculated like Pearsons's correlation coefficient, using the rank-transform on the x_i and y_i. The exact test for H_0 is the permutation test. Techniques for obtaining the exact distribution and approximations are given below

The first 8 cumulants are given by David *et al.* [199]

$$\kappa_2 = \frac{1}{n-1}$$

$$\kappa_4 = \frac{-6(19n^2+5n-36)}{25n(n+1)(n-1)^3}$$

$$\kappa_6 = \frac{48(583n^6+723n^5-2603n^4-2637n^3+4054n^2+2760n-1800)}{(245n^3(n-1)^5n1^3)}$$

$$\begin{aligned}\kappa_8 &= \frac{144(41939n^{10}-83709n^9+304254n^8+578442n^7-1012323n^6-1690125n^5)}{875n^5(n+1)^5(n-1)^7} \\ &+ \frac{144(1800776n^4+2358048n^3-1616688n^2-1080567n+846720)}{875n^5(n+1)^5(n-1)^7}\end{aligned} \tag{16.3}$$

Note: See PermCumulants.SpearmanCum for further details.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.spearman_mu8(x, nu, nc); mx = mpm.spearman_mu8(x, nu, nc)
>>> ix = ipm.spearman_mu8(x, nu, nc); fx = fpm.spearman_mu8(x, nu, nc)
>>> gx = gmp.spearman_mu8(x, nu, nc); ax = apm.spearman_mu8(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.1138774234168360557146160900744444149943E-1
mpm:  3.1138774234168360557146160900744444149943e-1
ipm:  3.1138774234168360557146160900744444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.1138774234168360557146160900744444149943E-01
ipm:  3.1138774234168360557146160900744444149943e-1 (4.055e-38%)
```

16.1.17 Mann-Whitney U distribution: general alternatives specified by rank order probabilities (Sundrum)

`ctx.mannwhitney_nc_mu4(`N`)`

where `ctx` is `dec`, `mpm`, `fpm`, `gmp` or `apm`.

Returns an approximation to the noncentral distribution of Mann-Whitney's U, based on the first 4 moments.

The first 4 moments under the alternative are given by Sundrum [510] (note that $\kappa_4 = \mu_4 - 3\mu 2^3$):

$$\mu_1 = p$$

$$\mu_2 = (p^2 + p - q - r)/mn + (q - p^2)/n + (r - p^2)/m + p^2$$

$$\begin{aligned}\mu_3 &= 6(p^3 + u - pq - pr)/mn + (2p^3 + s - 3pq)/n^2 + (2p^3 + t - 3pr)/m^2 \\ &+ 3(3pq + 2(pr - u - p^3) + q - s - p^2)/mn^2 + 3(3pr + 2(pq - u - p^3) \\ &+ r - t - p^2)/m^2n + (4p^3 + 3p^2 + p + 6u + 2(s + t) - 3(q + r)(1 + 2p))/m^2n^2\end{aligned} \tag{16.4}$$

$$\begin{aligned}\mu_4 &= 3(q - p^2)2/n^2 + 6(q - p^2)(r - p^2)/mn + 3(r - p^2)2/m^2 \\ &+ (12qp^2 + a - 4sp - 3q^2 - 6p^4)/n^3 + (12rp^2 + b - 4tp - 3r2 - 6p^4)/m^3 \\ &+ 6(7(rp^2 - p^4) + 12qp^2 + qp + 2(w - sp) - 3(q^2 + qr) - 8up - p^3)/mn^2 \\ &+ (42qp^2 + 72rp^2 + 6rp + 12x - 42p^4 - 18r2 - 18qr - 12tp - 48up - 6p^3)/m^2n \\ &+ 6(6(p^4 - rp^2) + 3(q^2 - qp) - 12qp^2 + 4sp + 8up + 2(p^3 + qr - w) + s - a)/mn^3 \\ &+ (36p^4 + 18r2 + 12qr - 72rp^2 - 36qp^2 + 24tp - 6b + 48up \\ &- 12x + 12p^3 - 18rp + 6t)/m^3n \\ &+ (105p^4 + 42p^3 + 3p^2 + 33q^2 + 33r2 + 54qr - 174qp^2 - 174rp^2 - 42pq \\ &- 42pr + 36sp + 36tp + 192up - 36w - 36x + 6v + 36u)/m^2n^2 \\ &+ (132qp^2 + 108rp^2 - 66p^4 - 33q^2 - 36qr - 18r2 - 44sp - 24tp + 11a \\ &- 144up + 36w + 24x - 6v - 36p^3 - 36u - 7p^2 + 54pq + 36pr - 18s + 7q)/m^2n^3 \\ &+ (132rp^2 + 108qp^2 - 66p^4 - 33r2 - 36qr - 18q^2 - 44tp - 24sp + 11b \\ &- 144up + 24w + 36x - 6v - 36p^3 - 36u - 7p^2 + 54pr + 36pq - 18t + 7r)/m^3n^2 \\ &+ (6(3(q^2 + r2) - 12((q + r)p^2) + 4(p^3 + qr + sp + tp - w - x) - (a + b - v) + 16up \\ &- 6(p^4 + pq + pr + u) + 2(s + t)) - 7(q + r - p^2) + p)/m^3n^3\end{aligned} \tag{16.5}$$

The parameters $p, q, r, s, t, v, u, a, b, w$ and x can be calculated from the following rank order probabilities:

$$\begin{aligned} p &= P_{1,1}(0,0) \\ q &= P_{2,1}(0,0,1) \\ r &= P_{1,2}(0,1,1) \\ s &= P_{3,1}(0,0,0,1) \\ t &= P_{1,3}(0,1,1,1) \\ v &= P_{2,2}(0,0,1,1) \\ u &= v + (1/4)P_{2,2}(0,1,0,1) \\ a &= P_{4,1}(0,0,0,0,1) \\ b &= P_{1,4}(0,1,1,1,1) \\ w &= 2a + (2/3)P_{3,2}(0,0,1,0,1) + (1/6)P_{3,2}(0,1,0,0,1) \\ x &= 2b + (2/3)P_{2,3}(0,1,0,1,1) + (1/6)P_{2,3}(0,1,1,0,1) \end{aligned} \tag{16.6}$$

The rank order probabilities which are required for the calculation of the first 4 moments can be estimated from the sample as follows: Let U_i be the number of $Y `sinthesamplegreaterthan` X_{(i)}$, where $X_{(i)}$ is the ith ordered values of the X amongst themselves. Then the probabilities are calculated according to the following scheme :

$$P_{1,1}(0,1) = \frac{1}{mn}\sum_{i=1}^{m} U_i$$

$$P_{2,1}(0,0,1) = \frac{2}{mn(m-1)}\sum_{i=1}^{m}(i-1)U_i$$

$$P_{3,1}(0,0,0,1) = \frac{3}{mn(m-1)(m-2)}\sum_{i=1}^{m}(i-1)(i-2)U_i$$

$$P_{4,1}(0,0,0,0,1) = \frac{4}{mn(m-1)(m-2)(m-3)}\sum_{i=1}^{m}(i-1)(i-2)(i-3)U_i$$

$$P_{1,2}(0,1,1) = \frac{1}{mn(n-1)}\sum_{i=1}^{m} U_i(U_i-1)$$

$$P_{1,3}(0,1,1,1) = \frac{1}{mn(n-1)(n-2)}\sum_{i=1}^{m} U_i(U_i-1)(U_i-2)$$

$$P_{1,4}(0,1,1,1,1) = \frac{1}{mn(n-1)(n-2)(n-3)}\sum_{i=1}^{m} U_i(U_i-1)(U_i-2)(U_i-3)$$

$$P_{2,2}(0,0,1,1) = \frac{2}{mn(m-1)(n-1)}\sum_{i=1}^{m}(i-1)U_i(U_i-1)$$

$$P_{3,2}(0,0,0,1,1) = \frac{3}{mn(m-1)(m-2)(n-1)}\sum_{i=1}^{m}(i-1)(i-2)U_i(U_i-1)$$

$$P_{2,3}(0,0,1,1,1) = \frac{2}{mn(m-1)(n-1)(n-2)}\sum_{i=1}^{m}(i-1)U_i(U_i-1)(U_i-2)$$

$$P_{2,2}(0,1,0,1) = \frac{4}{mn(m-1)(n-1)}\sum_{i=1}^{m}\sum_{j=i+1}^{m}(U_i-U_j)U_j$$

$$P_{3,2}(0,0,1,0,1) = \frac{12}{mn(m-1)(m-2)(n-1)} \sum_{i=1}^{m} \sum_{j=i+1}^{m} (i-1)(U_i - U_j)U_j$$

$$P_{3,2}(0,1,0,0,1) = \frac{12}{mn(m-1)(m-2)(n-1)} \sum_{i=1}^{m} \sum_{j=i+1}^{m} (i-j-1)(U_i - U_j)U_j$$

$$P_{2,3}(0,1,0,1,1) = \frac{6}{mn(m-1)(n-1)(n-2)} \sum_{i=1}^{m} \sum_{j=i+1}^{m} (U_i - U_j)U_j(U_j - 1)$$

$$P_{2,3}(0,1,0,1,1) = \frac{6}{mn(m-1)(n-1)(n-2)} \sum_{i=1}^{m} \sum_{j=i+1}^{m} (U_i - U_j)U_j(U_j - 1)$$

$$P_{2,3}(0,1,1,0,1) = \frac{6}{mn(m-1)(n-1)(n-2)} \sum_{i=1}^{m} \sum_{j=i+1}^{m} (U_i - U_j)(U_i - U_j - 1)U_j$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.mannwhitney_nc_mu4(x, nu, nc); mx = mpm.mannwhitney_nc_mu4(x, nu, nc)
>>> ix = ipm.mannwhitney_nc_mu4(x, nu, nc); fx = fpm.mannwhitney_nc_mu4(x, nu, nc)
>>> gx = gmp.mannwhitney_nc_mu4(x, nu, nc); ax = apm.mannwhitney_nc_mu4(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.1138774234168360557146160900744444149943E-1
mpm:  3.1138774234168360557146160900744444149943e-1
ipm:  3.1138774234168360557146160900744444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.1138774234168360557146160900744444149943E-01
ipm:  3.1138774234168360557146160900744444149943e-1 (4.055e-38%)
```

16.1.18 First 4 moments of Kendalls τ in the general case (Sundrum)

`ctx.kendall_tau_nc_mu4(N)`

where `ctx` is `dec`, `mpm`, `fpm`, `gmp` or `apm`.

Returns an approximation to the noncentral distribution of Kendall's τ, based on the first 4 moments.

Consider k pairs of random variables $(x_1, y_1), \ldots, (x_k, y_k)$. Let $r_1, \ldots, r_k$ be a permutaton of $1, \ldots, k$ and let $s_1, \ldots, s_k$ be its reciprocal. We define as in Snow [503].

$P(r_1, \ldots, r_k)$ = Pr[if x's are ranked in order, ranks of corresponding y's are $r_1, \ldots, r_k$].

$P(r_1, \ldots, r_k)$ = Pr[if y's are ranked in order, ranks of corresponding x's are $s_1, \ldots, s_k$].

Let $H(t) = 1$ if $t > 0$ and $H(t) = 0$ otherwise. Then

$$P(r_1, \ldots, r_k) = k!P\left[\prod_{i=2}^{k} H(x_i - x_{i-1})H(y_{s_i} - y_{s_{i-1}})\right] = k!P\left[\prod_{i=2}^{k} H(y_i - y_{i-1})H(x_{r_i} - x_{r_{i-1}})\right]$$

The rank order probabilities can then be estimated from the sample as

$$\hat{P}_{(12)} = \frac{1}{n(n-1)} \sum_{i \neq j} I\left[(x_i - x_j)(y_i - y_j) > 0\right]$$

$$\hat{P}_{(123)} = \frac{1}{n(n-1)(n-2)} \sum_{i \neq j_1 \neq j_2} I\left[(x_i - x_{j_1})(y_i - y_{j_1})(x_i - x_{j_2})(y_i - y_{j_2}) > 0\right]$$

The first 4 moments of T_N are given as Sundrum [509]:

$$\mu_1 = p$$

$$\mu_2 = 2[p(1-p) + 2(k-p^2)(n-2)]/[n(n-1)]$$

$$\begin{aligned} \mu_3 &= 4[(t - 18kp + 10p^3)n^2 + (6k + 2u - 6p^2 - 5t + 72kp - 34p^3)n \\ &+ (p + 9p^2 + 30p^3 - 12k - 4u + 6t - 72kp)]/[n(n-1)]^2 \end{aligned} \tag{16.7}$$

$$\begin{aligned} \mu_4 &= 8[6(k-p^2)2n^4 + (6kp + 2y - 6p^3 - 16tp - 84k2 + 270kp^2 - 108p4)n^3 \\ &+ (1.5p^2 + 6t + 2b - 126kp - 24up - 18y + 75p^3 + 120tp + 426k2 - 1446kp^2 + 505.5p4)n^2 \\ &+ (14k + 12u - 15.5p^2 - 30t - 10b + 444kp + 96up + 52y - 213p^3 - 296tp - 924k2 \\ &+ 2988kp^2 - 943.5p4)n \\ &+ (p - 28k - 24u + 21p^2 + 36t + 12b - 432kp - 96up - 48y + 180p^3 + 240tp + 720k2 \\ &- 2160kp^2 + 630p4)]/[n(n-1)]^3 \end{aligned} \tag{16.8}$$

The parameters p, k, u, t, b, y can be calculated from the following rank order probabilities, writing $P_{(1423)}$ for the probility that the ranks of the Y occur in this order when the X are sorted in ascending order:

$$\begin{aligned} p &= P_{(12)} \\ u &= P_{(123)} \\ k &= P_{(123)} + \tfrac{1}{4}P_{(132)} \\ t &= 8(P_{(1234)} + P_{(1243))} + 6P_{(1342)} + 4P_{(1324)} + 2P_{(2143)} + P_{(2413)} + P_{(1432)} \\ b &= 15P_{(1234)} + 10P_{(1243)} + 5P_{(1324)} + 4P_{(1342)} + P_{(2143)} \\ y &= (160P_{(12453)} + 150(P_{(12435)} + P_{(12354))} + 125P_{(12345)} + 96P_{(21453)} + 90P_{(13254)} + 84P_{(13524)} \\ &+ 80P_{(13425)} + 64P_{(13452)} + 45P_{(21354)} + 40P_{(12543)} + 32(P_{(23514)} + P_{(14352)} + P_{(13542))} \\ &+ 24P_{(21543)} + 22P_{(24153)} + 20P_{(14325)} + 18P_{(14523)} + 16P_{(25143)} + 12(P_{(24513)} + P_{(14532))} \\ &+ 6(P_{(25314)} + P_{(15342))} + 4P_{(25413)} + 2(P_{(15432)} + P_{(35142)}))/5 \end{aligned} \tag{16.9}$$

When $(x_i, y_i), i = 1, 2, \ldots, k$, are normal variables with correlation ρ, then closed form expressions exist for p, k and u (writing $a = \arcsin(\rho)/\pi$ and $b = 2\arcsin\left(\frac{1}{2}\rho\right)/\pi$:

$$\begin{aligned} p &= \tfrac{1}{2} + a \\ k &= \tfrac{1}{4}(\tfrac{10}{9} + 4a + 4a^2 - b^2) \\ u &= \tfrac{3}{8}(\tfrac{4}{9} + 4a - 2b + 4a^2 - b^2) \end{aligned} \tag{16.10}$$

The parameters t, b and y have to be evaluated by numerical intergration of the rank order probabilities

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.kendall_tau_nc_mu4(x, nu, nc); mx = mpm.kendall_tau_nc_mu4(x, nu, nc)
>>> ix = ipm.kendall_tau_nc_mu4(x, nu, nc); fx = fpm.kendall_tau_nc_mu4(x, nu, nc)
>>> gx = gmp.kendall_tau_nc_mu4(x, nu, nc); ax = apm.kendall_tau_nc_mu4(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.1138774234168360557146160900744444149943E-1
mpm:  3.1138774234168360557146160900744444149943e-1
ipm:  3.1138774234168360557146160900744444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.1138774234168360557146160900744444149943E-01
ipm:  3.1138774234168360557146160900744444149943e-1 (4.055e-38%)
```

16.2 Approximations based on the chi-squared distribution

16.2.1 Non-Central chi-squared : cdf, sf (Patnaik)

`ctx.chi2_nc_mu2_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the noncentral chi-squared distribution by matching the first 2 moments.

Patnaik [464] gives the following 2-moment approximation based on the central χ^2-distribution:

$$F_{\chi^2}(n, x; \lambda) \approx F_{\chi^2}(n_1, x_1;), \quad \text{where } n_1 = \frac{(n+\lambda)^2}{n+2\lambda}, \quad x_1 = \frac{x(n+\lambda)}{n+2\lambda}$$

where $F_{\chi^2}(n, \cdot)$ is the cdf of the (central) χ^2 distribution (see section ref{sec:ChiSquareDistribution_cdf}).

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.chi2_nc_mu2_cdf(x, nu, nc); mx = mpm.chi2_nc_mu2_cdf(x, nu, nc)
>>> ix = ipm.chi2_nc_mu2_cdf(x, nu, nc); fx = fpm.chi2_nc_mu2_cdf(x, nu, nc)
>>> gx = gmp.chi2_nc_mu2_cdf(x, nu, nc); ax = apm.chi2_nc_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.2.2 Non-Central chi-squared: qtf, isf (Patnaik)

`ctx.chi2_nc_mu2_qtf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the qtf of the noncentral chi-squared distribution.

The noncentral quantile is approximated as in Patnaik [464]:

$$\chi^2_{n,\lambda,\alpha} \approx (1+b)\chi^2_{n_1,\alpha}, \quad \text{where } n_1 = \frac{(n+\lambda)^2}{n+2\lambda}, \quad b = \frac{\lambda}{n+\lambda}$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.chi2_nc_mu2_qtf(x, nu, nc); mx = mpm.chi2_nc_mu2_qtf(x, nu, nc)
>>> ix = ipm.chi2_nc_mu2_qtf(x, nu, nc); fx = fpm.chi2_nc_mu2_qtf(x, nu, nc)
>>> gx = gmp.chi2_nc_mu2_qtf(x, nu, nc); ax = apm.chi2_nc_mu2_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.2.3 Non-Central chi-squared: confidence limit for λ (Winterbottom)

`ctx.chi2_nc_wb_cl(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the confidence interval (CI) for the noncentrality parameter λ of noncentral chi-squared distribution, as decribed in Winterbottom [1455].

[Winterbottom1979` gives the following formula to determine the paramter λ of a noncentral χ^2 distribution with n degrees of freedom, so that $F_{\chi^2}(n, x; 0) = 1 - \alpha$ and $F_{\chi^2}(n, x; \lambda) = 1 - \beta$. Let c be the $1 - \alpha$ percentage point of a χ^2-distribution with n degrees of freedom, let x be the $1 - \beta$ percentage point of a $N(0, 1)$ distribution, and $T = (c - n)/n$, $Y = 2T + 1$. Then

$$\begin{aligned}\lambda \approx\ & nT + \sqrt{2nY}x + \frac{2((3T+2)x^2 + 3T + 1))}{3Y} - \frac{(6T+5)x^3 - (36T^2 + 42T + 17)x}{18\sqrt{nY^5/2}} \\ & + \frac{(324T^2 + 594T + 276)x^4}{405nY^4} - \frac{(1080T^3 + 2484T^2 + 976)x^2}{405nY^4} \\ & + \frac{1080T^3 + 1512T^2 + 612T + 148}{405nY^4} - \frac{(10368T^3 + 30780T^2 + 30564T + 10143)x^5}{9720\sqrt{n^3Y^{11}/2}} \\ & + \frac{(25920T^4 + 98928T^3 + 163080T^2 + 137544T + 47188)x^3}{9720\sqrt{n^3Y^{11}/2}} \\ & + \frac{(45360T^4 + 106704T^3 + 80460T^2 + 31092T + 13489)x}{9720\sqrt{n^3Y^{11}/2}}\end{aligned} \tag{16.11}$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.chi2_nc_wb_cl(x, nu, nc); mx = mpm.chi2_nc_wb_cl(x, nu, nc)
>>> ix = ipm.chi2_nc_wb_cl(x, nu, nc); fx = fpm.chi2_nc_wb_cl(x, nu, nc)
>>> gx = gmp.chi2_nc_wb_cl(x, nu, nc); ax = apm.chi2_nc_wb_cl(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.2.4 Roy's largest root θ: pdf (Chiani)

`ctx.roy_chiani_pdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the pdf of the distribution of Roy's largest root θ.

See Chiani [177], Chiani [178] proposes the following algorithm:

$$F_{\Theta_1}(\theta_1) \approx P\left(k, \frac{\log(\theta_1/(1-\theta_1)) - \mu + \sigma\alpha}{\delta}\right)$$

and for its inverse, useful for evaluating the percentiles,

$$F^{-1}_{\Theta_1}(\theta_1) \approx \frac{\exp(\sigma(\delta P^{-1}(k, y) - \alpha)) + \mu}{1 + \exp(\sigma(\delta P^{-1}(k, y) - \alpha)) + \mu},$$

where

$$\mu = 2\log\tan\left(\frac{\gamma + \phi}{2}\right), \quad \sigma^3 = \frac{16}{(m+n+1)^2}\frac{1}{\sin^2(\gamma+\phi)\sin\gamma\sin\phi}$$

$$\gamma = \arccos\left(\frac{m+n-2p}{m+n-1}\right), \quad \phi = \arccos\left(\frac{m-n}{m+n-1}\right),$$

and the constants $k = 46.446$, $\delta = 0.186054$, $\alpha = 9.84801$ have been chosen to match the moments of the approximation to that of the Tracy-Widom.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.roy_chiani_pdf(x, nu, nc); mx = mpm.roy_chiani_pdf(x, nu, nc)
>>> ix = ipm.roy_chiani_pdf(x, nu, nc); fx = fpm.roy_chiani_pdf(x, nu, nc)
>>> gx = gmp.roy_chiani_pdf(x, nu, nc); ax = apm.roy_chiani_pdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.2.5 Roy's largest root: cdf and sf (Chiani)

`ctx.roy_chiani_cdf`(x)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the distribution of Roy's largest root θ.

See Chiani [177], Chiani [178] proposes the following algorithm:

$$F_{\Theta_1}(\theta_1) \approx P\left(k, \frac{\log(\theta_1/(1-\theta_1)) - \mu + \sigma\alpha}{\delta}\right)$$

and for its inverse, useful for evaluating the percentiles,

$$F^{-1}_{\Theta_1}(\theta_1) \approx \frac{\exp(\sigma(\delta P^{-1}(k,y) - \alpha)) + \mu}{1 + \exp(\sigma(\delta P^{-1}(k,y) - \alpha)) + \mu},$$

where

$$\mu = 2\log\tan\left(\frac{\gamma+\phi}{2}\right), \quad \sigma^3 = \frac{16}{(m+n+1)^2}\frac{1}{\sin^2(\gamma+\phi)\sin\gamma\sin\phi}$$

$$\gamma = \arccos\left(\frac{m+n-2p}{m+n-1}\right), \quad \phi = \arccos\left(\frac{m-n}{m+n-1}\right),$$

and the constants $k = 46.446$, $\delta = 0.186054$, $\alpha = 9.84801$ have been chosen to match the moments of the approximation to that of the Tracy-Widom.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.roy_chiani_cdf(x, nu, nc); mx = mpm.roy_chiani_cdf(x, nu, nc)
>>> ix = ipm.roy_chiani_cdf(x, nu, nc); fx = fpm.roy_chiani_cdf(x, nu, nc)
>>> gx = gmp.roy_chiani_cdf(x, nu, nc); ax = apm.roy_chiani_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.2.6 Roy's largest root: qtf and isf (Chiani)

`ctx.roy_chiani_qtf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the qtf of the distribution of Roy's largest root θ.

See Chiani [177], Chiani [178] proposes the following algorithm:

$$F_{\Theta_1}(\theta_1) \approx P\left(k, \frac{\log(\theta_1/(1-\theta_1)) - \mu + \sigma\alpha}{\delta}\right)$$

and for its inverse, useful for evaluating the percentiles,

$$F^{-1}_{\Theta_1}(\theta_1) \approx \frac{\exp(\sigma(\delta P^{-1}(k,y) - \alpha)) + \mu}{1 + \exp(\sigma(\delta P^{-1}(k,y) - \alpha)) + \mu},$$

where

$$\mu = 2\log\tan\left(\frac{\gamma+\phi}{2}\right), \quad \sigma^3 = \frac{16}{(m+n+1)^2}\frac{1}{\sin^2(\gamma+\phi)\sin\gamma\sin\phi}$$

$$\gamma = \arccos\left(\frac{m+n-2p}{m+n-1}\right), \quad \phi = \arccos\left(\frac{m-n}{m+n-1}\right),$$

and the constants $k = 46.446$, $\delta = 0.186054$, $\alpha = 9.84801$ have been chosen to match the moments of the approximation to that of the Tracy-Widom.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.roy_chiani_qtf(x, nu, nc); mx = mpm.roy_chiani_qtf(x, nu, nc)
>>> ix = ipm.roy_chiani_qtf(x, nu, nc); fx = fpm.roy_chiani_qtf(x, nu, nc)
>>> gx = gmp.roy_chiani_qtf(x, nu, nc); ax = apm.roy_chiani_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.1138774234168360557146160900744444149943E-1
mpm:  3.1138774234168360557146160900744444149943e-1
ipm:  3.1138774234168360557146160900744444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.1138774234168360557146160900744444149943E-01
ipm:  3.1138774234168360557146160900744444149943e-1 (4.055e-38%)
```

16.3 Approximations based on the central t, F or beta distribution

16.3.1 Dunn-Šidák percentage points

`ctx.dunn_sidak_qtf`(q, *nu*, C)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns the percentage points of Dunn-Šidák.

See also: Kirk, Games.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.dunn_sidak_qtf(x, nu, nc); mx = mpm.dunn_sidak_qtf(x, nu, nc)
>>> ix = ipm.dunn_sidak_qtf(x, nu, nc); fx = fpm.dunn_sidak_qtf(x, nu, nc)
>>> gx = gmp.dunn_sidak_qtf(x, nu, nc); ax = apm.dunn_sidak_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.2 Singly non-central Fisher F distribution: cdf, sf (Patnaik)

`ctx.fisher_f_nc_mu2_cdf`(x)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the noncentral Fisher F distribution.

[Patnaik1949` suggests the following approximation, see also Tiku [784]:

$$F_{F'}(x; n_1, n_2, \lambda) \approx F_F\left(y; m_1, n_2\right), \quad \text{where}$$

$A_1 = (n_1 + \lambda)$,

$B_1 = (n_1 + 2\lambda)$,

$m_1 = A_1^2 / B_1$,

$y = n_1 / A_1$,

and $F_F\left(\cdot; m_1, n_2\right)$ denotes the CDF of a central F distribution with m_1 and n_2 degrees of freedom.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.fisher_f_nc_mu2_cdf(x, nu, nc); mx = mpm.fisher_f_nc_mu2_cdf(x, nu, nc)
>>> ix = ipm.fisher_f_nc_mu2_cdf(x, nu, nc); fx = fpm.fisher_f_nc_mu2_cdf(x, nu, nc)
>>> gx = gmp.fisher_f_nc_mu2_cdf(x, nu, nc); ax = apm.fisher_f_nc_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.3 Singly non-central F distribution: qtf, isf (Patnaik)

`ctx.fisher_f_nc_mu2_qtf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to $F_{\alpha,n_1,n_2,\lambda}$, the α-quantile of a non-central F-distribution with n_1 and n_2 degress of freedom and noncentrality parameter λ.

$$F_{\alpha,n_1,n_2,\lambda} \approx c \cdot F_{\alpha,m_1,n_2,}$$

$A_1 = (n_1 + \lambda)$,

$B_1 = (n_1 + 2\lambda)$,

$m_1 = A_1^2/B_1$,

$c = A_1/n_1$,

and $F_{\alpha,m_1,n_2,}$ denotes the α-quantile of a central F-distribution with m_1 and n_2 degress of freedom.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.fisher_f_nc_mu2_qtf(x, nu, nc); mx = mpm.fisher_f_nc_mu2_qtf(x, nu, nc)
>>> ix = ipm.fisher_f_nc_mu2_qtf(x, nu, nc); fx = fpm.fisher_f_nc_mu2_qtf(x, nu, nc)
>>> gx = gmp.fisher_f_nc_mu2_qtf(x, nu, nc); ax = apm.fisher_f_nc_mu2_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.4 Singly non-central F: confidence interval for the noncentrality parameter λ

`ctx.fisher_f_nc_mu2_cl(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the confidence interval (CI) for the noncentrality parameter λ of a singly non-central F-distribution.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.fisher_f_nc_mu2_cl(x, nu, nc); mx = mpm.fisher_f_nc_mu2_cl(x, nu, nc)
>>> ix = ipm.fisher_f_nc_mu2_cl(x, nu, nc); fx = fpm.fisher_f_nc_mu2_cl(x, nu, nc)
>>> gx = gmp.fisher_f_nc_mu2_cl(x, nu, nc); ax = apm.fisher_f_nc_mu2_cl(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.5 Doubly non-central F distribution: cdf, sf (Patnaik)

`ctx.fisher_f_nc2_mu2_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the doubly noncentral Fisher F distribution.

Mudholkar (1976) suggests an approximation by noncentral F, which can be converted into an approximation by central F as follows:

$$F_{F''}(x; n_1, n_2, \lambda_1, \lambda_2) \approx F_F(y; m_1, m_2), \quad \text{where}$$

$A_1 = (n_1 + \lambda_1), A_2 = (n_2 + \lambda_2),$

$B_1 = (n_1 + 2\lambda_1), B_2 = (n_2 + 2\lambda_2),$

$m_1 = A_1^2/B_1, m_2 = A_2^2/B_2,$

$y = x(n_1 A_2)/(n_2 A_1),$

and $F_F(\cdot; m_1, m_2)$ denotes the CDF of a central F distribution with m_1 and m_2 degrees of freedom.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.fisher_f_nc2_mu2_cdf(x, nu, nc); mx = mpm.fisher_f_nc2_mu2_cdf(x, nu, nc)
>>> ix = ipm.fisher_f_nc2_mu2_cdf(x, nu, nc); fx = fpm.fisher_f_nc2_mu2_cdf(x, nu, nc)
>>> gx = gmp.fisher_f_nc2_mu2_cdf(x, nu, nc); ax = apm.fisher_f_nc2_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.6 Doubly non-central F distribution: qtf, isf (Patnaik)

`ctx.fisher_f_nc2_mu2_qtf(x)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to $F_{\alpha,n_1,n_2,\lambda_1,\lambda_2}$, the α-quantile of a doubly non-central F-distribution with n_1 and n_2 degress of freedom and noncentrality parameters λ_1 and λ_2, is obtained as

$$F_{\alpha,n_1,n_2,\lambda_1,\lambda_2} \approx c \cdot F_{\alpha,m_1,m_2,}$$

$A_1 = (n_1 + \lambda_1), A_2 = (n_1 + \lambda_2),$

$B_1 = (n_1 + 2\lambda_1), B_2 = (n_1 + 2\lambda_2),$

$m_1 = A_1^2/B_1, m_2 = A_2^2/B_2,$

$c = (n_2 A_1)/(n_1 A_2),$

and denotes the α-quantile of a central F-distribution with m_1 and m_2 degress of freedom.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.fisher_f_nc2_mu2_qtf(x, nu, nc); mx = mpm.fisher_f_nc2_mu2_qtf(x, nu, nc)
>>> ix = ipm.fisher_f_nc2_mu2_qtf(x, nu, nc); fx = fpm.fisher_f_nc2_mu2_qtf(x, nu, nc)
>>> gx = gmp.fisher_f_nc2_mu2_qtf(x, nu, nc); ax = apm.fisher_f_nc2_mu2_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.7 Multiple correlation coefficient: cdf, sf (Lee and Gurland)

`ctx.fisher_r2_lee_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the Fisher R^2 distribution.

See also Lee [407], Gurland [357], Gurland and Milton [359].

$$F_{F'}(x; n_1, n_2, \rho^2) \approx F_F\left(x/c; m_1, n_2\right),$$

where $c = A_1/n_1$, $m_1 = A_1^2/A_2$, $A_1 = (n_1 + n_2)(\gamma - 1) + n_1$, $A_2 = (n_1 + n_2)(\gamma^2 - 1) + n_1$, $\gamma = 1/(1 - \rho^2)$, and $F_F\left(\cdot; m_1, n_2\right)$ denotes the CDF of a central F distribution with m_1 and n_2 degrees of freedom.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.fisher_r2_lee_cdf(x, nu, nc); mx = mpm.fisher_r2_lee_cdf(x, nu, nc)
>>> ix = ipm.fisher_r2_lee_cdf(x, nu, nc); fx = fpm.fisher_r2_lee_cdf(x, nu, nc)
>>> gx = gmp.fisher_r2_lee_cdf(x, nu, nc); ax = apm.fisher_r2_lee_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.8 Multiple correlation coefficient: qtf, isf (Lee and Gurland)

`ctx.fisher_r2_lee_qtf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the qtf of the Fisher R^2 distribution.

See also Lee [407], Gurland [357], Gurland and Milton [359].

$$F_{F'}(x; n_1, n_2, \rho^2) \approx F_F\left(x/c; m_1, n_2\right),$$

where $c = A_1/n_1$, $m_1 = A_1^2/A_2$, $A_1 = (n_1 + n_2)(\gamma - 1) + n_1$, $A_2 = (n_1 + n_2)(\gamma^2 - 1) + n_1$, $\gamma = 1/(1 - \rho^2)$, and $F_F\left(\cdot; m_1, n_2\right)$ denotes the CDF of a central F distribution with m_1 and n_2 degrees of freedom.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.fisher_r2_lee_qtf(x, nu, nc); mx = mpm.fisher_r2_lee_qtf(x, nu, nc)
>>> ix = ipm.fisher_r2_lee_qtf(x, nu, nc); fx = fpm.fisher_r2_lee_qtf(x, nu, nc)
>>> gx = gmp.fisher_r2_lee_qtf(x, nu, nc); ax = apm.fisher_r2_lee_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.9 Fisher R^2,: confidence limit for ρ^2

ctx.**fisher_r2_lee_cl**(x)

where ctx is fpm, mpm, ipm, dec, gmp or apm.

Returns an approximation to the confidence interval (CI) for the noncentrality parameter ρ^2 of Fisher R^2 distribution.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.fisher_r2_lee_cl(x, nu, nc); mx = mpm.fisher_r2_lee_cl(x, nu, nc)
>>> ix = ipm.fisher_r2_lee_cl(x, nu, nc); fx = fpm.fisher_r2_lee_cl(x, nu, nc)
>>> gx = gmp.fisher_r2_lee_cl(x, nu, nc); ax = apm.fisher_r2_lee_cl(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.1138774234168360557146160900744444149943E-1
mpm:  3.1138774234168360557146160900744444149943e-1
ipm:  3.1138774234168360557146160900744444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.1138774234168360557146160900744444149943E-01
ipm:  3.1138774234168360557146160900744444149943e-1 (4.055e-38%)
```

16.3.10 Central Wilks' Lambda: cdf, sf (Rao)

`ctx.wilks_lambda_rao_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns Rao's F approximation to the cdf of Wilk's Λ

Define

$$z = \frac{1-\Lambda^{1/r}}{\Lambda^{1/r}}\frac{rt-2u}{pq}, \quad \text{where } r = v - \frac{p-q+1}{2}, u = \frac{pq-2}{4}, \quad \text{and}$$

$$t = \sqrt{\frac{p^2q^2-4}{p^2+q^2-5}} \text{ if } p^2+q^2-5 > 0 \text{ or 1 otherwise.}$$

Then z is approximately distributed as $F(z; pq, rt-2u)$.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.wilks_lambda_rao_cdf(x, nu, nc); mx = mpm.wilks_lambda_rao_cdf(x, nu, nc)
>>> ix = ipm.wilks_lambda_rao_cdf(x, nu, nc); fx = fpm.wilks_lambda_rao_cdf(x, nu, nc)
>>> gx = gmp.wilks_lambda_rao_cdf(x, nu, nc); ax = apm.wilks_lambda_rao_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.11 Central Wilks' Lambda: qtf, isf (Rao)

`ctx.wilks_lambda_rao_qtf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns Rao's F approximation to the cdf of Wilk's Λ

Define

$$z = \frac{1-\Lambda^{1/r}}{\Lambda^{1/r}}\frac{rt-2u}{pq}, \quad \text{where } r = v - \frac{p-q+1}{2}, u = \frac{pq-2}{4}, \quad \text{and}$$

$$t = \sqrt{\frac{p^2q^2-4}{p^2+q^2-5}} \text{ if } p^2+q^2-5 > 0 \text{ or 1 otherwise.}$$

Then z is approximately distributed as $F(z; pq, rt-2u)$.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.wilks_lambda_rao_qtf(x, nu, nc); mx = mpm.wilks_lambda_rao_qtf(x, nu, nc)
>>> ix = ipm.wilks_lambda_rao_qtf(x, nu, nc); fx = fpm.wilks_lambda_rao_qtf(x, nu, nc)
>>> gx = gmp.wilks_lambda_rao_qtf(x, nu, nc); ax = apm.wilks_lambda_rao_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.12 Central Hotelling's T^2: cdf, sf (Pillai and Young)

`ctx.hotelling_t2_mu3_cdf(x)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the central Hotelling's T^2 distribution. See also Pillai and Young [471].

Let $m = (n_1 - p - 1)/2$, $n = (n_2 - p - 1)/2$, and $r = 2m + p + 1$. Define

$$\mu_1 = \frac{pr}{2n}, \quad \mu_2 = \mu_1 \frac{(2n+r)(2n+p)}{2n(n-1)(2n+1)}, \quad \mu_3 = \mu_2 \frac{2(n+r)(n+p)}{n(n-2)(n+1)},$$

$$a = \frac{(2\mu_1 3\mu_2 + 3\mu_1^2\mu_3 - 6\mu_1\mu_2^2 - \mu_2\mu_3)}{(\mu_2\mu_3 + 4\mu_1\mu_2^2 - \mu_1^2\mu_3)} + 1, \quad b = \frac{a(a+2) - \mu_1^2/\mu_2}{a - \mu_1^2/\mu_2} - a,$$

$$K = \frac{\mu_1(b-1)}{a}, \quad w = \frac{x}{x+K}.$$

Then $x = T^2/n_2$ is approximately distributed as $I_w(a, b)$.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.hotelling_t2_mu3_cdf(x, nu, nc); mx = mpm.hotelling_t2_mu3_cdf(x, nu, nc)
>>> ix = ipm.hotelling_t2_mu3_cdf(x, nu, nc); fx = fpm.hotelling_t2_mu3_cdf(x, nu, nc)
>>> gx = gmp.hotelling_t2_mu3_cdf(x, nu, nc); ax = apm.hotelling_t2_mu3_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.13 Central Hotelling's T^2: qtf, isf (Pillai and Young)

`ctx.hotelling_t2_mu3_qtf(x)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the qtf of the central Hotelling's T^2 distribution, based on matching the first 3 moments.. See also Pillai and Young [471].

Let $m = (n_1 - p - 1)/2$, $n = (n_2 - p - 1)/2$, and $r = 2m + p + 1$. Define

$$\mu_1 = \frac{pr}{2n}, \quad \mu_2 = \mu_1 \frac{(2n+r)(2n+p)}{2n(n-1)(2n+1)}, \quad \mu_3 = \mu_2 \frac{2(n+r)(n+p)}{n(n-2)(n+1)},$$

$$a = \frac{(2\mu_1 3\mu_2 + 3\mu_1^2\mu_3 - 6\mu_1\mu_2^2 - \mu_2\mu_3)}{(\mu_2\mu_3 + 4\mu_1\mu_2^2 - \mu_1^2\mu_3)} + 1, \quad b = \frac{a(a+2) - \mu_1^2/\mu_2}{a - \mu_1^2/\mu_2} - a,$$

$$K = \frac{\mu_1(b-1)}{a}, \quad w = \frac{x}{x+K}.$$

Then $x = T^2/n_2$ is approximately distributed as $I_w(a, b)$.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.hotelling_t2_mu3_qtf(x, nu, nc); mx = mpm.hotelling_t2_mu3_qtf(x, nu, nc)
>>> ix = ipm.hotelling_t2_mu3_qtf(x, nu, nc); fx = fpm.hotelling_t2_mu3_qtf(x, nu, nc)
```

(continues on next page)

(continued from previous page)

```
>>> gx = gmp.hotelling_t2_mu3_qtf(x, nu, nc); ax = apm.hotelling_t2_mu3_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.11387742341683605571461609007444414994 3E-1
mpm:  3.11387742341683605571461609007444414994 3e-1
ipm:  3.11387742341683605571461609007444414994 4e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.11387742341683605571461609007444414994 3E-01
ipm:  3.11387742341683605571461609007444414994 3e-1 (4.055e-38%)
```

16.3.14 Central Pillai's V: cdf, sf (Ginzberg)

`ctx.pillay_v_mu3_cdf`(x)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the central Pillai V distribution, based on matching the first 3 moments. See also Ginzberg [349].

Let $m = (n_1 - p - 1)/2$, $n = (n_2 - p - 1)/2$, and $r = m + n + p$. Define

$$\mu_1 = \frac{p(2m+p+1)}{2(r+1)}, \quad \mu_2 = \mu_1 \frac{(2n+p+1)(2r-p+2)}{2(r+1)(r+2)(2r+1)}, \quad \mu_3 = \mu_2 \frac{4(n-m)(m+n+1)}{(r+1)(r+3)(2r)},$$

$$c = 4\mu_1\mu_2^2 - \mu_1^2\mu_3 + \mu_2\mu_3, \quad d = 2\mu_2^2 - \mu_1\mu_3,$$

$$a = \frac{2\mu_1(\mu_1^2\mu_2 - \mu_2^2 + \mu_1\mu_3)}{c}, \quad b = \frac{a}{d}\,\frac{\mu_2(2\mu_1\mu_2 + \mu_3)}{\mu_1}, \quad w = \frac{d}{c}x.$$

Then $x = V/n_2$ is approximately distributed as $I_w(a, b)$.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.pillay_v_mu3_cdf(x, nu, nc); mx = mpm.pillay_v_mu3_cdf(x, nu, nc)
>>> ix = ipm.pillay_v_mu3_cdf(x, nu, nc); fx = fpm.pillay_v_mu3_cdf(x, nu, nc)
>>> gx = gmp.pillay_v_mu3_cdf(x, nu, nc); ax = apm.pillay_v_mu3_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.11387742341683605571461609007444414994 3E-1
mpm:  3.11387742341683605571461609007444414994 3e-1
ipm:  3.11387742341683605571461609007444414994 4e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.11387742341683605571461609007444414994 3E-01
ipm:  3.11387742341683605571461609007444414994 3e-1 (4.055e-38%)
```

16.3.15 Central Pillai's V: qtf, isf (Ginzberg)

`ctx.pillay_v_mu3_qtf`(x)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the central Pillai V distribution. See also Ginzberg [349].

Let $m = (n_1 - p - 1)/2$, $n = (n_2 - p - 1)/2$, and $r = m + n + p$. Define

$$\mu_1 = \frac{p(2m+p+1)}{2(r+1)}, \quad \mu_2 = \mu_1 \frac{(2n+p+1)(2r-p+2)}{2(r+1)(r+2)(2r+1)}, \quad \mu_3 = \mu_2 \frac{4(n-m)(m+n+1)}{(r+1)(r+3)(2r)},$$

$$c = 4\mu_1\mu_2^2 - \mu_1^2\mu_3 + \mu_2\mu_3, \quad d = 2\mu_2^2 - \mu_1\mu_3,$$

$$a = \frac{2\mu_1(\mu_1^2\mu_2 - \mu_2^2 + \mu_1\mu_3)}{c}, \quad b = \frac{a}{d}\,\frac{\mu_2(2\mu_1\mu_2 + \mu_3)}{\mu_1}, \quad w = \frac{d}{c}x.$$

Then $x = V/n_2$ is approximately distributed as $I_w(a, b)$.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.pillay_v_mu3_qtf(x, nu, nc); mx = mpm.pillay_v_mu3_qtf(x, nu, nc)
>>> ix = ipm.pillay_v_mu3_qtf(x, nu, nc); fx = fpm.pillay_v_mu3_qtf(x, nu, nc)
>>> gx = gmp.pillay_v_mu3_qtf(x, nu, nc); ax = apm.pillay_v_mu3_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.16 Product of independent beta variables: cdf, sf (Nagarsenker)

`ctx.beta_product_mu3_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the product of independent beta variables distribution.

Nagarsenker and Suniaga [421] proposes the following approximation:

$$\Pr(W_p \le \lambda) = I(x; sm + d, \nu_1 + r) + O(sm^{-3}), \quad \text{where}$$

$$x = \lambda^{1/s}; \quad a = \frac{\nu_1 - \nu_2}{2\nu_1}; \quad d = \frac{1 - \nu_1}{2}; \quad \nu_r = \sum_{i=1}^{p} c_i^r - b_i^r$$

$$s^2 = \frac{-2B_2((1 + \nu_1)/2)}{\sum_{i=1}^{p} B_3(a + b_i) - B_3(a + c_i)}$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.beta_product_mu3_cdf(x, nu, nc); mx = mpm.beta_product_mu3_cdf(x, nu, nc)
>>> ix = ipm.beta_product_mu3_cdf(x, nu, nc); fx = fpm.beta_product_mu3_cdf(x, nu, nc)
>>> gx = gmp.beta_product_mu3_cdf(x, nu, nc); ax = apm.beta_product_mu3_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.3.17 Product of independent beta variables: qtf, isf (Nagarsenker)

`ctx.beta_product_mu3_qtf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the qtf of the product of independent beta variables distribution.

Nagarsenker and Suniaga [421] proposes the following approximation:

$$\Pr(W_p \le \lambda) = I(x; sm + d, \nu_1 + r) + O(sm^{-3}), \quad \text{where}$$

$$x = \lambda^{1/s}; \quad a = \frac{\nu_1 - \nu_2}{2\nu_1}; \quad d = \frac{1 - \nu_1}{2}; \quad \nu_r = \sum_{i=1}^{p} c_i^r - b_i^r$$

$$s^2 = \frac{-2B_2((1+\nu_1)/2)}{\sum_{i=1}^{p} B_3(a+b_i) - B_3(a+c_i)}$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.beta_product_mu3_qtf(x, nu, nc); mx = mpm.beta_product_mu3_qtf(x, nu, nc)
>>> ix = ipm.beta_product_mu3_qtf(x, nu, nc); fx = fpm.beta_product_mu3_qtf(x, nu, nc)
>>> gx = gmp.beta_product_mu3_qtf(x, nu, nc); ax = apm.beta_product_mu3_qtf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.4 Approximations based on the noncentral chi-squared distribution

16.4.1 Non-central Wilks' Lambda (GLM): cdf and sf (Fujikoshi)

`ctx.`**`wilks_lambda_glm_chi2_cdf`**(*x*, *p*, *q*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the noncentral Wilks' Lambda (GLM) distribution.

Fujikoshi [339] proposes the following noncentral chi-square approximation to the cdf and sf:

$$\begin{aligned} F_{\Lambda,GLM}(p,q,n;z;\Theta) &= F_{\chi^2}\left(pq,x;\theta_1\right) + \frac{1}{4n}\sum_{k=1}^{3} a_k F_{\chi^2}\left(pq+2k,x;\theta_1\right) \\ &+ \frac{1}{96n^2}\sum_{k=0}^{6} b_k F_{\chi^2}\left(pq+2k,x;\theta_1\right) + \frac{1}{96n^3}\sum_{k=1}^{9} c_k F_{\chi^2}\left(pq+2k,x;\theta_1\right) + O(n^{-4}) \end{aligned} \tag{16.12}$$

$$\Theta = \Lambda^{-1}MM', \quad \theta_j = \text{tr}\,\Theta^j,$$

$$m = n + (pq-1)/2, \quad x = m\log(z), \quad s = (p+q+1)/4, \quad r = pq(p^2+q^2-5)/48,$$

$$\begin{aligned} a_1 &= 2s\theta_1 \\ a_2 &= 2s\theta_1 - \theta_2 \\ a_3 &= \theta_2 \end{aligned} \tag{16.13}$$

$$\begin{aligned} b_0 &= r \\ b_1 &= 0 \\ b_2 &= r4s^2\theta_1 + 2s^2\theta_1^2 + 2s\theta_2 \\ b_3 &= 4s^2\theta_1 - (1+4s^2)\theta_1^2 - (1+8s)\theta_2 + 2s\theta_1\theta_2 + (4/3)\theta_3 \\ b_4 &= (1+2s^2)\theta_1^2 + (1+6s)\theta_2 - 4s\theta_1\theta_2 - 4\theta_3 + \theta_2^2/2 \\ b_5 &= 2s\theta_1\theta_2 + (8/3)\theta_3 - \theta_2^2 \\ b_6 &= \theta_2^2/2 \end{aligned} \tag{16.14}$$

$$\begin{aligned} c_1 &= 2rs\theta_1 \\ c_2 &= r(2s\theta_1\theta_2) \\ c_3 &= 2s(r+4s^2)\theta_1 + 2s(1+4s^2)\theta_1^2 + (-r+2s+12s^2)\theta_2 - \tfrac{4}{3}s^3\theta_1^3 - 4s^2\theta_1\theta_2 - \tfrac{8}{3}s\theta_3 \\ c_4 &= 2s(r+4s^2)\theta_1 - (1+10s+16s^3)\theta_1^2 - (3+r+10s+36s^2)\theta_2 + 2s(1+2s^2)\theta_1^3 \\ &+ 2(2+s+12s^2)\theta_1\theta_2 + 4(1+6s)\theta_3 - 2s^2\theta_1^2\theta_2 - 2s\theta_2^2 - \tfrac{8}{3}s\theta_1\theta_3 - 2\theta_4 \\ c_5 &= (1+8s+8s^3)\theta_1^2 + (3+r+8s+24s^2)\theta_2 - 4s(1+s^2)\theta_1^3 - 4(3+s+9s^2)\theta_1\theta_2 - 12(1+4s)\theta_3 \\ &+ (1+6s^2)\theta_1^2\theta_2 + (1+10s)\theta_2^2 + \tfrac{32}{3}s\theta_1\theta_3 + 12\theta_4 - \tfrac{4}{3}\theta_2\theta_3 - s\theta_1\theta_2^2 \\ c_6 &= s(2+\tfrac{4}{3}s^2)\theta_1^3 + 2(4+s+8s^2)\theta_1\theta_2 + 8(1+\tfrac{10}{3}s)\theta_3 - 2(1+3s^2)\theta_1^2\theta_2 - 2(1+7s)\theta_2^2 \\ &- \tfrac{40}{3}s\theta_1\theta_3 - 20\theta_4 + \tfrac{16}{3}\theta_2\theta_3 + 3s\theta_1\theta_2^2 - \tfrac{1}{6}\theta_2^3 \\ c_7 &= (1+2s^2)\theta_1^2\theta_2 + (1+6s)\theta_2^2 + \tfrac{16}{3}s\theta_1\theta_3 + 10\theta_4 - \tfrac{20}{3}\theta_2\theta_3 - 3s\theta_1\theta_2^2 + \tfrac{1}{2}\theta_2^3 \\ c_8 &= \tfrac{8}{3}\theta_2\theta_3 + s\theta_1\theta_2^2 - \tfrac{1}{2}\theta_2^3 \\ c_9 &= \tfrac{1}{6}\theta_2^3 \end{aligned} \tag{16.15}$$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.wilks_lambda_glm_chi2_cdf(x, nu, nc); mx = mpm.wilks_lambda_glm_chi2_cdf(x, nu, nc)
>>> ix = ipm.wilks_lambda_glm_chi2_cdf(x, nu, nc); fx = fpm.wilks_lambda_glm_chi2_cdf(x, nu, nc)
>>> gx = gmp.wilks_lambda_glm_chi2_cdf(x, nu, nc); ax = apm.wilks_lambda_glm_chi2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

Fujikoshi [339]

Pillai: coeff p. 93 (5.10), GLM p. 96 (5.2), CORR p. 113/114 (8.1)

Hotelling: coeff p. 105 (6.18), GLM p. 105 (6.19), CORR p. 119 (9.9)

16.4.2 Non-central Wilks' Lambda (independence): cdf and sf (Lee)

`ctx.`**`wilks_lambda_ind_chi2_cdf`**(*x*, *p*, *q*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the noncentral Wilks' Lambda (independence) distribution.

Lee [406] proposes the following noncentral chi-square approximation:

$$\text{cdf}_X(x) == F_{\chi^2}\left(pq, x; s1\right) + \frac{1}{4n}\sum_{k=0}^{3} a_k F_{\chi^2}\left(pq+2k, x; s1\right) + \frac{1}{96n^2}\sum_{k=0}^{6} b_k F_{\chi^2}\left(pq+2k, x; s1\right) + O(n^{-3}),$$

$\Omega = \Lambda^{-1}MM'$, $\theta_j = \text{tr}\,\Omega^j$, $m = n + (pq-1)/2$, $x = m\log(z)$, $f = pq$, $s = (p+q+1)/4$, $r = f(p^2+q^2-5)/48$,

$a_0 = -q\theta_1 + \theta_2$, $a_1 = (2s+q)\theta_1 - 2\theta_2$, $a_2 = -2s\theta_1 + 2\theta_2$, $a_3 = -\theta_2$,

$b_0 = -r - ql\theta_1 + (q+l)\theta_2 + \frac{1}{2}q^2\theta_1^2 - \frac{4}{3}\theta_3 - q\theta_1\theta_2 + \frac{1}{2}\theta_2^2$,

$b_1 = q^2\theta_1 - 4q\theta_2 - q(q+2s)\theta_1^2 + 4\theta_3 + (3q+2s)\theta_1\theta_2 - 2\theta_2^2$,

$b_2 = r - 2s(q+2s)\theta_1 + (2p+6q+3)\theta_2 + (\frac{1}{2}l^2 + 6qs + 1)\theta_1^2 - 8\theta_3 - (4q+6s)\theta_1\theta_2 + 4\theta_2^2$,

$b_3 = 4s^2\theta_1 - (3p+5q+5)\theta_2 - (4s^2+2qs+2)\theta_1^2 + \frac{32}{3}\theta_3 + (3q+8s)\theta_1\theta_2 - 5\theta_2^2$,

$b_4 = (6s+1)\theta_2 + (2s^2+1)\theta_1^2 - 8\theta_3 - (q+6s)\theta_1\theta_2 + 4\theta_2^2$,

$b_5 = \frac{8}{3}\theta_3 + 2s\theta_1\theta_2 - 2\theta_2^2$,

$b_6 = \frac{1}{2}\theta_2^2$.

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.wilks_lambda_ind_chi2_cdf(x, nu, nc); mx = mpm.wilks_lambda_ind_chi2_cdf(x, nu, nc)
>>> ix = ipm.wilks_lambda_ind_chi2_cdf(x, nu, nc); fx = fpm.wilks_lambda_ind_chi2_cdf(x, nu, nc)
>>> gx = gmp.wilks_lambda_ind_chi2_cdf(x, nu, nc); ax = apm.wilks_lambda_ind_chi2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.4.3 Non-central Pillai's V (GLM): cdf and sf Fujikoshi

`ctx.pillai_v_glm_chi2_cdf`(*x*, *p*, *q*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the noncentral Pillai's V (GLM) distribution.

Fujikoshi [340] proposes the following noncentral chi-square approximation to the cdf and sf:

$$\text{cdf}_X(x) == F_{\chi^2}\left(pq, x; s1\right) + \frac{1}{4n}\sum_{k=0}^{3} a_k F_{\chi^2}\left(pq+2k, x; s1\right) + \frac{1}{96n^2}\sum_{k=0}^{6} b_k F_{\chi^2}\left(pq+2k, x; s1\right) + O(n^{-3}),$$

$l_0 = (3f-8)g^2 + 4g + 4(f+2)$

$l_1 = -12fg^2$

$l_2 = 6(3f+8)g^2$

$l_3 = -4((3f+16)g^2 + 4g + 4(f+2))$

$l_4 = 3((f+8)g^2 + 4g + 4(f+2))$

$a_0 = -fg$

$a_1 = 2fg$

$a_2 = -fg + 4g\lambda_1 + 4\lambda_2$

$a_3 = -4g\lambda_1$

$a_4 = -4\lambda_2$

$b_0 = fl_0$

$b_1 = fl_1$

$b_2 = fl_2 + 2l_1\lambda_1 - 24fg\lambda_2$

$b_3 = fl_3 + 4l_2\lambda_1 + 48(f+4)g\lambda_2 + 128\lambda_3$

$b_4 = fl_4 + 6l_3\lambda_1 + 48(g^2-2)\lambda_1^2 - 96(g+1)\lambda_2 + 96g\lambda_1\lambda_2 + 48\lambda_2^2$

$b_5 = 8(l_4\lambda_1 - 12(g^2+2)\lambda_1^2 - 6((f+12)g+4)\lambda_2 - 12g\lambda_1\lambda_2 - 48\lambda_3)$

$b_6 = 8(6(g^2+6)\lambda_1^2 + 3((f+20)g+12)\lambda_2 - 12g\lambda_1\lambda_2 - 16\lambda_3 - 12\lambda_2^2)$

$b_7 = 96(g\lambda_1\lambda_2 + 4\lambda_3)$

$b_8 = 48\lambda_2^2$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.pillai_v_glm_chi2_cdf(x, nu, nc); mx = mpm.pillai_v_glm_chi2_cdf(x, nu, nc)
>>> ix = ipm.pillai_v_glm_chi2_cdf(x, nu, nc); fx = fpm.pillai_v_glm_chi2_cdf(x, nu, nc)
>>> gx = gmp.pillai_v_glm_chi2_cdf(x, nu, nc); ax = apm.pillai_v_glm_chi2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.4.4 Non-central Pillai's V (independence): cdf and sf (Lee)

`ctx.pillai_v_ind_chi2_cdf`(*x*, *p*, *q*, *n*, *lambda*)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the noncentral Pillai's V (independence) distribution.

Lee [406] proposes the following noncentral chi-square approximation. See also

$$\text{cdf}_X(x) == F_{\chi^2}\left(pq, x; s1\right) + \frac{1}{4n}\sum_{k=0}^{3} a_k F_{\chi^2}\left(pq+2k, x; s1\right) + \frac{1}{96n^2}\sum_{k=0}^{6} b_k F_{\chi^2}\left(pq+2k, x; s1\right) + O(n^{-3}),$$

$a_0 = -fg - 4\lambda_2$

$a_1 = 2fg$

$a_2 = -fg + 4g\lambda_1 + 8\lambda_2$

$a_3 = -4g\lambda_1$

$a_4 = -4\lambda_2$

$b_0 = fl_0 + 24fg\lambda_2 - 128\lambda_3 + 48\lambda_2^2$

$b_1 = fl_1 - 48fg\lambda_2$

$b_2 = fl_2 + 2l_1\lambda_1 + 96\lambda_1^2 - 24(qp^2 + q(q+1)p - 4)\lambda_2 - 96g\lambda_1\lambda_2 - 192\lambda_2^2$

$b_3 = fl_3 + 4l_2\lambda_1 + 96(qp^2 + (q^2+q+4)p + 4(q+1))\lambda_2 + 96g\lambda_1\lambda_2 + 640\lambda_3$

$b_4 = fl_4 + 6l_3\lambda_1 + 48(p^2 + 2(q+1)p + q^2 + 2q - 3)\lambda_1^2 - 24(qp^2 + (q^2+q+12)p + 4(3q+5))\lambda_2 + 192g\lambda_1\lambda_2 + 288\lambda_2^2$

$b_5 = 8l_4\lambda_1 - 96(p^2 + 2(q+1)p + q^2 + 2q + 3)\lambda_1^2 - 48(qp^2 + (q^2+q+12)p + 4(3q+4))\lambda_2 - 192g\lambda_1\lambda_2 - 768\lambda_3$

$b_6 = 48(p^2 + 2(q+1)p + q^2 + 2q + 7)\lambda_1^2 + 24(qp^2 + (q^2+q+20)p + 4(5q+8))\lambda_2 - 96g\lambda_1\lambda_2 - 128\lambda_3 - 192\lambda_2^2$

$b_7 = 96(g\lambda_1\lambda_2 + 4\lambda_3)$

$b_8 = 48\lambda_2^2$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.pillai_v_ind_chi2_cdf(x, nu, nc); mx = mpm.pillai_v_ind_chi2_cdf(x, nu, nc)
>>> ix = ipm.pillai_v_ind_chi2_cdf(x, nu, nc); fx = fpm.pillai_v_ind_chi2_cdf(x, nu, nc)
>>> gx = gmp.pillai_v_ind_chi2_cdf(x, nu, nc); ax = apm.pillai_v_ind_chi2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.4.5 Non-central Hotelling T^2 (GLM): cdf and sf (Fujikoshi)

`ctx.hotelling_t2_glm_chi2_cdf(x)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the noncentral Hotelling T^2 (GLM) distribution.

Fujikoshi [340] proposes the following noncentral chi-square approximation to the pdf and cdf:

$$\mathrm{cdf}_X(x) == F_{\chi^2}\left(pq, x; s1\right) + \frac{1}{4n}\sum_{k=0}^{3} a_k F_{\chi^2}\left(pq+2k, x; s1\right) + \frac{1}{96n^2}\sum_{k=0}^{6} b_k F_{\chi^2}\left(pq+2k, x; s1\right) + O(n^{-3}),$$

$a_0 = fg$

$a_1 = -2g(f - 2\lambda_1)$

$a_2 = fg - 8g\lambda_1 + 4\lambda_2$

$a_3 = 4(g\lambda_1 - 2\lambda_2)$

$a_4 = 4\lambda_2$

$b_0 = fl_0$

$b_1 = l_1(f - 2\lambda_1)$

$b_2 = fl_2 + 2(l_1 - 2l_2)\lambda_1 + 48g^2\lambda_1^2 + 24(f+4)g\lambda_2$

$b_3 = fl_3 + 2(2l_2 - 3l_3)\lambda_1 - 192(g^2+1)\lambda_1^2 - 96((f+8)g + 2)\lambda_2 + 96g\lambda_1\lambda_2 + 128\lambda_3$

$b_4 = fl_4 + 2(3l_3 - 4l_4)\lambda_1 + 96(3g^2+7)\lambda_1^2 + 48(3(f+12)g + 14)\lambda_2 - 384g\lambda_1\lambda_2 - 768\lambda_3 + 48\lambda_2^2$

$b_5 = 8l_4\lambda_1 - 192(g^2+4)\lambda_1^2 - 96((f+16)g + 8)\lambda_2 + 576g\lambda_1\lambda_2 + 1536\lambda_3 - 192\lambda_2^2$

$b_6 = 48(g^2+6)\lambda_1^2 + 24((f+20)g + 12)\lambda_2 - 384g\lambda_1\lambda_2 - 1280\lambda_3 + 288\lambda_2^2$

$b_7 = 96g\lambda_1\lambda_2 + 384\lambda_3 - 192\lambda_2^2$

$b_8 = 48\lambda_2^2$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.hotelling_t2_glm_chi2_cdf(x, nu, nc); mx = mpm.hotelling_t2_glm_chi2_cdf(x, nu, nc)
>>> ix = ipm.hotelling_t2_glm_chi2_cdf(x, nu, nc); fx = fpm.hotelling_t2_glm_chi2_cdf(x, nu, nc)
>>> gx = gmp.hotelling_t2_glm_chi2_cdf(x, nu, nc); ax = apm.hotelling_t2_glm_chi2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.4.6 Non-central Hotelling T^2 (independence): cdf and sf (Lee)

`ctx.hotelling_t2_ind_chi2_cdf(x)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Returns an approximation to the cdf of the noncentral Hotelling T^2 (independence) distribution.

Lee [406] proposes the following noncentral chi-square approximation:

$$\text{cdf}_X(x) == F_{\chi^2}\left(pq, x; s1\right) + \frac{1}{4n}\sum_{k=0}^{3} a_k F_{\chi^2}\left(pq+2k, x; s1\right) + \frac{1}{96n^2}\sum_{k=0}^{6} b_k F_{\chi^2}\left(pq+2k, x; s1\right) + O(n^{-3}),$$

$s_1 = 2\lambda_1$

$s_2 = 4\lambda_2$

$s_3 = 8\lambda_3$

$a_0 = qp(q-p-1) - 2qs_1 + s_2$

$a_1 = -2q^2p + 4qs_1 - 2s_2$

$a_2 = qp(q+p+1) - 2(2q+p+1)s_1 + 2s_2$

$a_3 = 2(q+p+1)s_1 - 2s_2$

$a_4 = s_2$

$b_0 = qp(3qp^3 - 2(3q^2 - 3q + 4)p^2 + 3(q^3 - 2q^2 + 5q - 4)p - 8q^2 + 12q + 4) - 12q^2p(q-p-1)s_1 - 6q(p^2 - qp + p - 4)s_2 + 12q^2s_1^2 - 16s_3 - 12qs_1s_2 + 3s_22$

$b_1 = -12q^3p^2(q-p-1) - 24q^2(p^2 - 2qp + p - 2)s_1 + 12q(p^2 - 2qp + p - 8)s_2 - 48q^2s_1^2 + 48s_3 + 48qs_1s_2 - 12s_22$

$b_2 = -6q^2p^4 - 12q^2p^3 + 18q^2(q^2+1)p^2 + 24q^2(2q+1)p + 12q(p^3 + 2p^2 - 7(q^2+1)p - 16q - 8)s_1 - 6(qp^2 - (7q^2 - q + 8)p - 40q - 12)s_2 + 24(qp + 4q^2 + q + 1)s_1^2 - 12(p + 8q + 1)s_1s_2 - 96s_3 + 24s_22$

$b_3 = -(12q^3+16q)p^3 - (12q^4+12q^3+96q^2+48q)p^2 - (64q^3+96q^2+64q)p + 12(-qp^3 + (4q^2-2q+4)p^2 + (7q^3+4q^2+31q+12)p + 4(7q^2+8q+4))s_1 - 48((q^2+3)p+9q+5)s_2 - 24(3qp+5q^2+3q+4)s_1^2 + 176s_3 + 12(3p+11q+3)s_1s_2 - 36s_22$

$b_4 = 3q^2p^4 + (6q^3+6q^2+24q)p^3 + (3q^4+6q^3+63q^2+60q)p^2 + (24q^3+60q^2+60q)p - 12(qp^3 + (5q^2+2q+12)p^2 + (4q^3+5q^2+45q+32)p + 4(6q^2+11q+9))s_1 + 6(qp^2 + (7q^2+q+44)p + 88q + 76)s_2 + 12(p^2 + 2(4q+1)p + 8q^2 + 8q + 17)s_1^2 - 12(4p + 11q + 4)s_1s_2 - 240s_3 + 42s_22$

$b_5 = (12qp^3 + 24(q^2+q+4)p^2 + 12(q^3+2q^2+21q+20)p + 48(2q^2+5q+5))s_1 - 12(qp^2 + (2q^2+q+24)p + 32q + 40)s_2 - 24(p^2 + (3q+2)p + 2q^2 + 3q + 9)s_1^2 + 240s_3 + 48(p+2q+1)s_1s_2 - 36s_22$

$b_6 = (6qp^2+6(q^2+q+20)p+120q+192)s_2 + (12p^2+24(q+1)p+12(q^2+2q+7))s_1^2 - 12(3p+4q+3)s_1s_2 - 160s_3 + 24s_22$

$b_7 = 48s_3 + 12(q+p+1)s_1s_2 - 12s_22$

$b_8 = 3s_22$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.hotelling_t2_ind_chi2_cdf(x, nu, nc); mx = mpm.hotelling_t2_ind_chi2_cdf(x, nu, nc)
>>> ix = ipm.hotelling_t2_ind_chi2_cdf(x, nu, nc); fx = fpm.hotelling_t2_ind_chi2_cdf(x, nu, nc)
>>> gx = gmp.hotelling_t2_ind_chi2_cdf(x, nu, nc); ax = apm.hotelling_t2_ind_chi2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.5 Approximations based on the noncentral F or beta distribution

16.5.1 Multiple correlation coefficient (Lee and Gurland)

`ctx.fisher_r2_lee_mu3_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

Lee [407] suggests the following approximation, based on the noncentral F distribution:

$$F_{R^2}(x; n_1, n_2, \rho^2) \approx F_{F'}(y; \nu, n_2, \lambda),$$

where $\gamma = 1/(1-\rho^2), \quad A_j = (n_1+n_2)(\gamma^j - 1) + n_1, \quad j = 1, 2, 3,$

$G = (A_2 - \sqrt{A_2^2 - A_1 A_3})/A_1, \quad \lambda = \rho^2 \gamma \sqrt{\gamma(n_1+n_2)n_2}/G^2, \quad \nu = (A_2/G^2) - 2\lambda, \quad y = x/(1-x) \times n_2/(\nu \cdot G),$

and $F_{F'}(\cdot; \nu, n_2, \lambda)$ denotes the CDF of the noncentral F distribution with ν and n_2 degrees of freedom and noncentrality parameter λ

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.hotelling_t2_ind_chi2_cdf(x, nu, nc); mx = mpm.hotelling_t2_ind_chi2_cdf(x, nu, nc)
>>> ix = ipm.hotelling_t2_ind_chi2_cdf(x, nu, nc); fx = fpm.hotelling_t2_ind_chi2_cdf(x, nu, nc)
>>> gx = gmp.hotelling_t2_ind_chi2_cdf(x, nu, nc); ax = apm.hotelling_t2_ind_chi2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.5.2 Noncentral Wilks' Lambda under the GLM alternative

`ctx.wilks_lambda_glm_mu2_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

See also: O'Brien and Shieh [425], Kulp and Nagarsenker [401]

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.wilks_lambda_glm_mu2_cdf(x, nu, nc); mx = mpm.wilks_lambda_glm_mu2_cdf(x, nu, nc)
>>> ix = ipm.wilks_lambda_glm_mu2_cdf(x, nu, nc); fx = fpm.wilks_lambda_glm_mu2_cdf(x, nu, nc)
>>> gx = gmp.wilks_lambda_glm_mu2_cdf(x, nu, nc); ax = apm.wilks_lambda_glm_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.5.3 Noncentral Wilks' Lambda under the independence alternative

`ctx.wilks_lambda_ind_mu2_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

See also: O'Brien and Shieh [425], Kulp and Nagarsenker [401]

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.wilks_lambda_ind_mu2_cdf(x, nu, nc); mx = mpm.wilks_lambda_ind_mu2_cdf(x, nu, nc)
>>> ix = ipm.wilks_lambda_ind_mu2_cdf(x, nu, nc); fx = fpm.wilks_lambda_ind_mu2_cdf(x, nu, nc)
>>> gx = gmp.wilks_lambda_ind_mu2_cdf(x, nu, nc); ax = apm.wilks_lambda_ind_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.5.4 Noncentral Hotelling's T under the GLM alternative

`ctx.`**`hotelling_t2_glm_mu2_cdf`**(x)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

See also: O'Brien and Shieh [425]

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.hotelling_t2_glm_mu2_cdf(x, nu, nc); mx = mpm.hotelling_t2_glm_mu2_cdf(x, nu, nc)
>>> ix = ipm.hotelling_t2_glm_mu2_cdf(x, nu, nc); fx = fpm.hotelling_t2_glm_mu2_cdf(x, nu, nc)
>>> gx = gmp.hotelling_t2_glm_mu2_cdf(x, nu, nc); ax = apm.hotelling_t2_glm_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.5.5 Noncentral Hotelling's T under the independence alternative

`ctx.`**`hotelling_t2_ind_mu2_cdf`**(x)

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

See also: O'Brien and Shieh [425]

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.hotelling_t2_ind_mu2_cdf(x, nu, nc); mx = mpm.hotelling_t2_ind_mu2_cdf(x, nu, nc)
>>> ix = ipm.hotelling_t2_ind_mu2_cdf(x, nu, nc); fx = fpm.hotelling_t2_ind_mu2_cdf(x, nu, nc)
>>> gx = gmp.hotelling_t2_ind_mu2_cdf(x, nu, nc); ax = apm.hotelling_t2_ind_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.5.6 Noncentral Pillai's V under the GLM alternative

`ctx.pillai_v_glm_mu2_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

See also: O'Brien and Shieh [425]

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.pillai_v_glm_mu2_cdf(x, nu, nc); mx = mpm.pillai_v_glm_mu2_cdf(x, nu, nc)
>>> ix = ipm.pillai_v_glm_mu2_cdf(x, nu, nc); fx = fpm.pillai_v_glm_mu2_cdf(x, nu, nc)
>>> gx = gmp.pillai_v_glm_mu2_cdf(x, nu, nc); ax = apm.pillai_v_glm_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.5.7 Noncentral Pillai's V under the independence alternative

`ctx.pillai_v_ind_mu2_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

See also: O'Brien and Shieh [425]

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.pillai_v_ind_mu2_cdf(x, nu, nc); mx = mpm.pillai_v_ind_mu2_cdf(x, nu, nc)
>>> ix = ipm.pillai_v_ind_mu2_cdf(x, nu, nc); fx = fpm.pillai_v_ind_mu2_cdf(x, nu, nc)
>>> gx = gmp.pillai_v_ind_mu2_cdf(x, nu, nc); ax = apm.pillai_v_ind_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.5.8 Noncentral Roy's largest root under the GLM alternative

`ctx.roy_glm_mu2_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

See Muirhead and Chikuse [419].

Let $l_1 \geq l_2 \geq \cdots \geq l_m > 0$ be the latent roots of $S_1 S_2^{-1}$.$Let`\Omega = \Sigma^{-1} M' M$, and ω_i are the latent roots of Ω. An upper bound for the distribution function of l_1 is given by

$$P(l_1 \leq x) \leq \prod_{i=1}^{m} (F_{n1,n2}(\omega_i) \leq x)$$

and a lower bound for the distribution function of l_m is given by

$$P(l_m \leq x) \geq \prod_{i=1}^{m} (F_{n1,n2}(\omega_i) \geq x)$$

where $F_{n1,n2}(\omega_i)$ denotes a random variable having the noncentral F distribution on n1 and n2 degrees of freedom and noncentrality parameter ω_i.

Muirhead,R.J.; Chikuse,Y. (1975): Approximations for the distributions of the extreme latent roots of three matrices. Ann. Inst. Statist. Math., 27, 473-478

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.roy_glm_mu2_cdf(x, nu, nc); mx = mpm.roy_glm_mu2_cdf(x, nu, nc)
>>> ix = ipm.roy_glm_mu2_cdf(x, nu, nc); fx = fpm.roy_glm_mu2_cdf(x, nu, nc)
>>> gx = gmp.roy_glm_mu2_cdf(x, nu, nc); ax = apm.roy_glm_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.5.9 Noncentral Roy's largest root under the independence alternative

`ctx.roy_ind_mu2_cdf(`x`)`

where `ctx` is `fpm`, `mpm`, `ipm`, `dec`, `gmp` or `apm`.

See Muirhead and Chikuse [419].

Let $l_1 \geq l_2 \geq \cdots \geq l_m > 0$ be the latent roots of $S_1 S_2^{-1}$.$Let`\Omega = \Sigma^{-1} M' M$, and ω_i are the latent roots of Ω. An upper bound for the distribution function of l_1 is given by

$$P(l_1 \leq x) \leq \prod_{i=1}^{m} (F_{n1,n2}(\omega_i) \leq x)$$

and a lower bound for the distribution function of l_m is given by

$$P(l_m \leq x) \geq \prod_{i=1}^{m} (F_{n1,n2}(\omega_i) \geq x)$$

where $F_{n1,n2}(\omega_i)$ denotes a random variable having the noncentral F distribution on n1 and n2 degrees of freedom and noncentrality parameter ω_i.

Muirhead,R.J.; Chikuse,Y. (1975): Approximations for the distributions of the extreme latent roots of three matrices. Ann. Inst. Statist. Math., 27, 473-478

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.roy_ind_mu2_cdf(x, nu, nc); mx = mpm.roy_ind_mu2_cdf(x, nu, nc)
>>> ix = ipm.roy_ind_mu2_cdf(x, nu, nc); fx = fpm.roy_ind_mu2_cdf(x, nu, nc)
>>> gx = gmp.roy_ind_mu2_cdf(x, nu, nc); ax = apm.roy_ind_mu2_cdf(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.6 Approximations based on hypergeometric functions of scalar argument

See also Koev and Edelman [395], Butler and Wood [167], Butler [165].

16.6.1 Hypergeometric function ${}_1F_1$ for matrix argument (Butler's approximation)

`ctx.hypermat_1f1_butler`(*res*, *a*, *b*, *X*)

where `ctx` is `dec`, `mpm`, `fpm`, `gmp` or `apm`.

Returns Butler's approximation to Kummer's confluent hypergeometric function for matrix argument, ${}_1F_1(a;b;\mathbf{T})$, which is defined as (see {NIST})

$$ {}_1F_1(a;b;\mathbf{T}) = \sum_{k=0}^{\infty} \frac{1}{k!} \sum_{|\kappa|=k} \frac{[a]_\kappa}{[b]_\kappa} Z_\kappa(\mathbf{T}) $$

with $-b + \frac{1}{2}(j+1) \notin \mathbb{N}, 1 \le j \le m; ||\mathbf{T}|| < 1$. Here $Z_\kappa(\mathbf{T})$ is a zonal polynomial.

Butler's approximation takes the form

$$ {}_1F_1(a;b;X) \approx \frac{b^{pb-p(p+1)/4}}{\sqrt{R_{1,1}}} \times \prod_{i=1}^{p} \left[\left(\frac{y_i}{a}\right)^a \left(\frac{1-y_i}{b-a}\right)^{b-a} e^{x_i y_i} \right], \quad \text{where} $$

$$ X = \text{diag}(x_1, \ldots, x_p, y_i = \frac{2a}{b - x_i + \sqrt{(x_i-b)^2 + 4a x_i}}, \text{ and} $$

$$ R_{1,1} = \prod_{i=1}^{p} \prod_{j=i}^{p} \left[\frac{y_i y_j}{a} + \frac{(1-y_i)(1-y_j)}{b-a} \right]. $$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.hypermat_1f1_butler(x, nu, nc); mx = mpm.hypermat_1f1_butler(x, nu, nc)
>>> ix = ipm.hypermat_1f1_butler(x, nu, nc); fx = fpm.hypermat_1f1_butler(x, nu, nc)
>>> gx = gmp.hypermat_1f1_butler(x, nu, nc); ax = apm.hypermat_1f1_butler(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.6.2 Comparisons with MATLAB: some results

Text for examples

Matlab insert

EXAMPLE 1: CF of log MeansRatio RV with and n = 5 and alpha = 7/2

```
n     = 5;
alpha = 7/2;
t     = linspace(-100,100,201);
cf    = cf_LogRV_MeansRatioW(t,n,alpha);
figure; plot(t,real(cf),t,imag(cf)); grid on;
title('CF of log MeansRatio RV with and n = 5 and alpha = 7/2')
```

16.6.3 Hypergeometric function ${}_2F_1$ for matrix argument (Butler's approximation)

`ctx.hypermat_2f1_butler(`*res*, *a*, *b*, *c*, *X*`)`

where `ctx` is `dec`, `mpm`, `fpm`, `gmp` or `apm`.

Returns Butler's approximation to the Gauss hypergeometric function for matrix argument, ${}_2F_1(a, b; c; \mathbf{T})$, which is defined as (see {NIST})

$$ {}_2F_1(a,b;c;\mathbf{T}) = \sum_{k=0}^{\infty} \frac{1}{k!} \sum_{|\kappa|=k} \frac{[a]_\kappa [b]_\kappa}{[c]_\kappa} Z_\kappa(\mathbf{T}) $$

with $-c + \frac{1}{2}(j+1) \notin \mathbb{N}, 1 \le j \le m; ||\mathbf{T}|| < 1$. Here $Z_\kappa(\mathbf{T})$ is a zonal polynomial.

Butler's approximation takes the form

$$ {}_2F_1(a,b;c;X) \approx \frac{c^{pc-p(p+1)/4}}{\sqrt{R_{2,1}}} \times \prod_{i=1}^{p} \left[\left(\frac{y_i}{a}\right)^a \left(\frac{1-y_i}{c-a}\right)^{c-a} (1-x_i y_i)^{-b} \right], $$

where $X = \mathrm{diag}(x_1, \ldots, x_p)$, $S_i = x_i y_i (1-y_i)/(1-x_i y_i)$, $\tau_i = x_i(b-a) - c$,

$$ y_i = \frac{2a}{\sqrt{t_i^2 - 4a x_i (c-b)} - \tau_i}, \text{ and} $$

$$ R_{2,1} = \prod_{i=1}^{p} \prod_{j=i}^{p} \left[\frac{y_i y_j}{a} + \frac{(1-y_i)(1-y_j)}{c-a} - \frac{b}{a(c-a)} S_i S_j \right]. $$

An example (CDF):

```
>>> from xlcalcnet import fpm, mpm
>>> mpm.dps = 40; x = 12; nu = 10; nc = 30
>>> dx = dec.hypermat_2f1_butler(x, nu, nc); mx = mpm.hypermat_2f1_butler(x, nu, nc)
>>> ix = ipm.hypermat_2f1_butler(x, nu, nc); fx = fpm.hypermat_2f1_butler(x, nu, nc)
>>> gx = gmp.hypermat_2f1_butler(x, nu, nc); ax = apm.hypermat_2f1_butler(x, nu, nc)
>>> mpm.show([dx, mx, ix, fx, gx, ax])
dec:  3.113877423416836055714616090074444149943E-1
mpm:  3.113877423416836055714616090074444149943e-1
ipm:  3.113877423416836055714616090074444149944e-1 (4.147e-38%)
fpm:  3.11387742341684E-01
gmp:  3.113877423416836055714616090074444149943E-01
ipm:  3.113877423416836055714616090074444149943e-1 (4.055e-38%)
```

16.6.4 Comparison with MATLAB: some results

Text for examples

Matlab insert

EXAMPLE 1: CF of log MeansRatio RV with and n = 5 and alpha = 7/2

```
n     = 5;
alpha = 7/2;
t     = linspace(-100,100,201);
cf    = cf_LogRV_MeansRatioW(t,n,alpha);
figure; plot(t,real(cf),t,imag(cf)); grid on;
title('CF of log MeansRatio RV with and n = 5 and alpha = 7/2')
```

Part IV

Gallery of Plots

CHAPTER

SEVENTEEN

VISUALISATION OF DATASETS

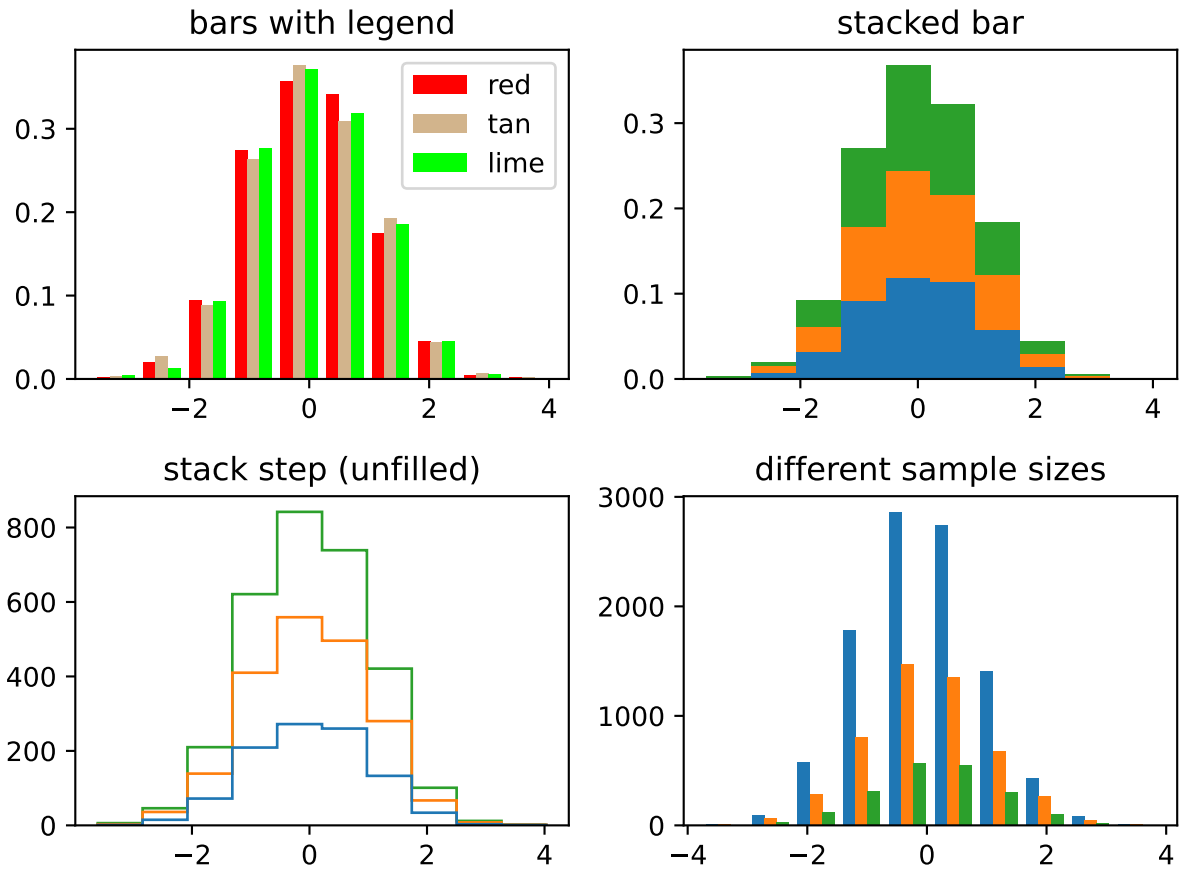
Text explaining the use of the Gallery

Text explaining the use of the Visualisation of datasets

17.1 Bar Charts

17.1.1 Bar chart, part 1

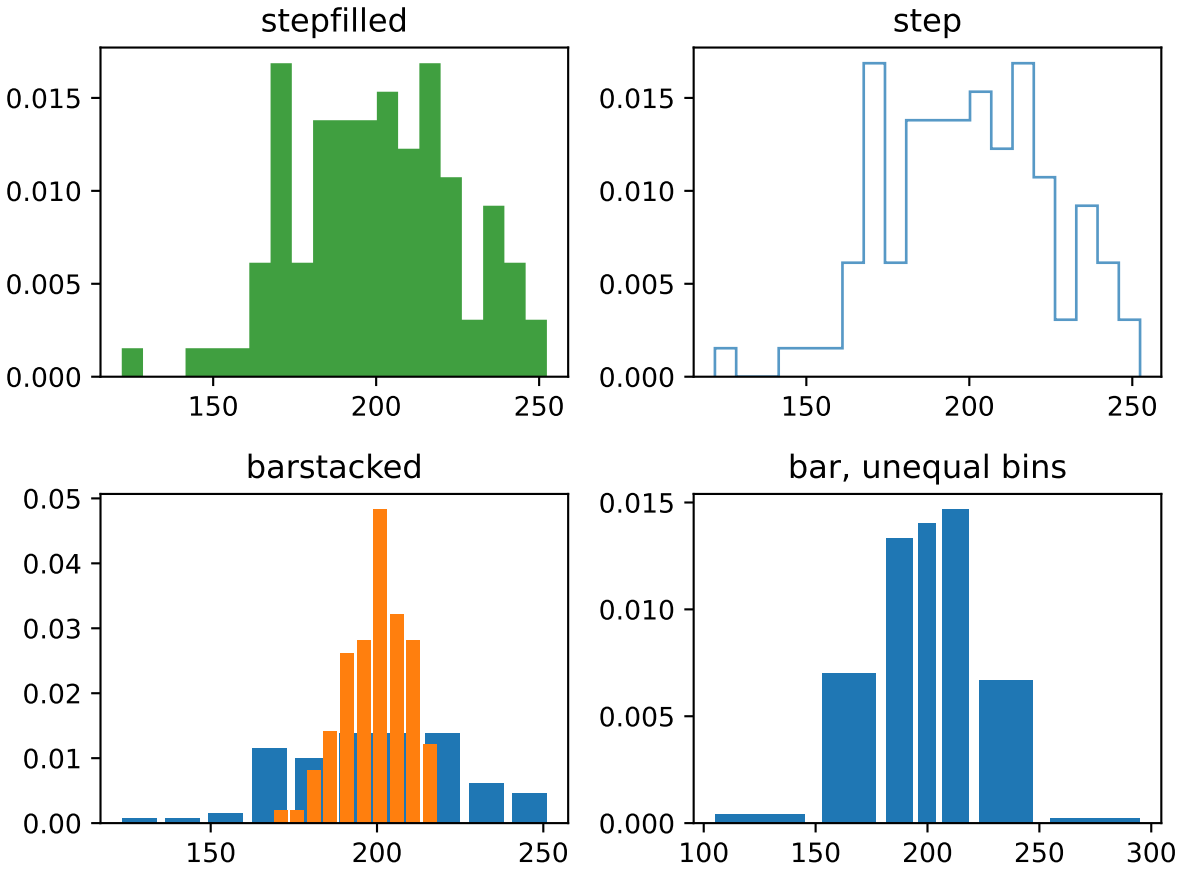
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_BarChartPart1.py, and the XML code in the file D01_BarChartPart1.2D.xml.



Bar chart, part 1

17.1.2 Bar chart, part 2

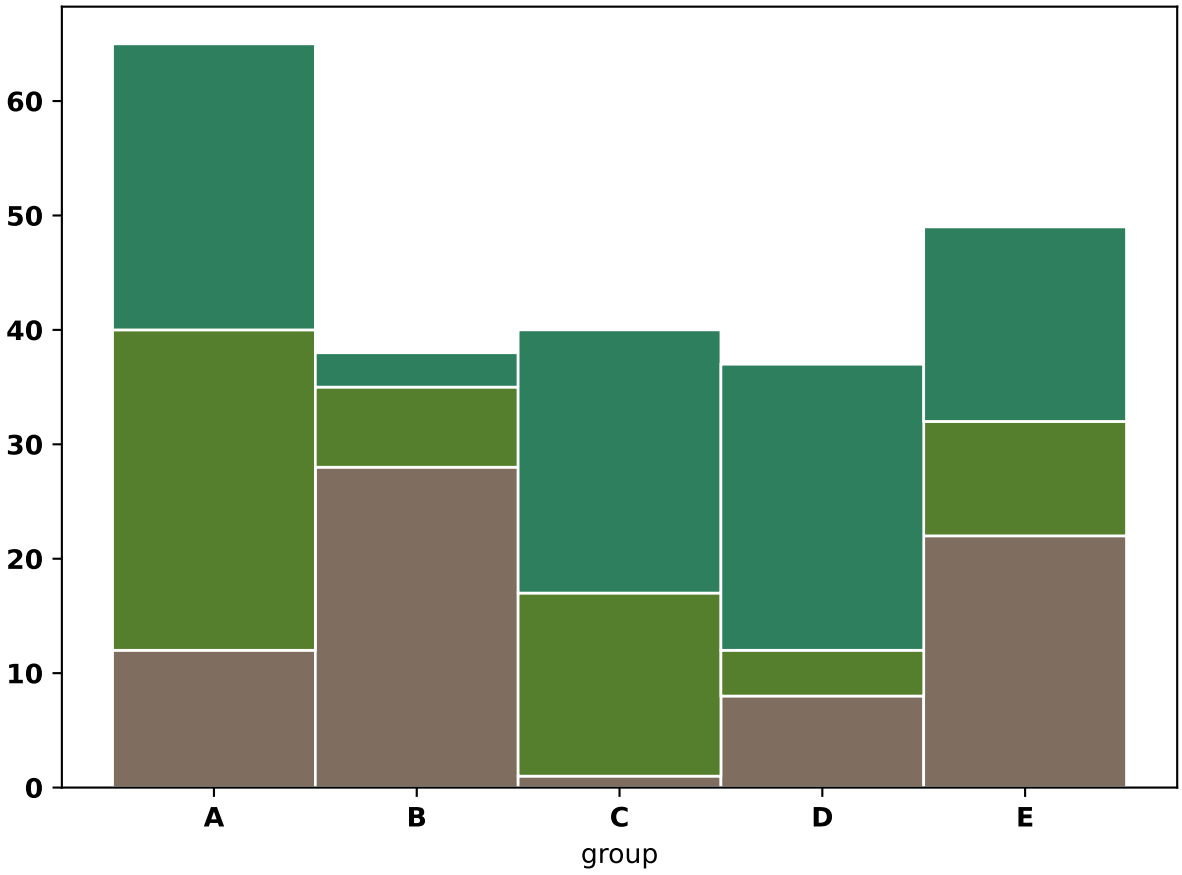
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_BarChartPart2.py, and the XML code in the file D02_BarChartPart2.2D.xml.



Bar chart, part 2

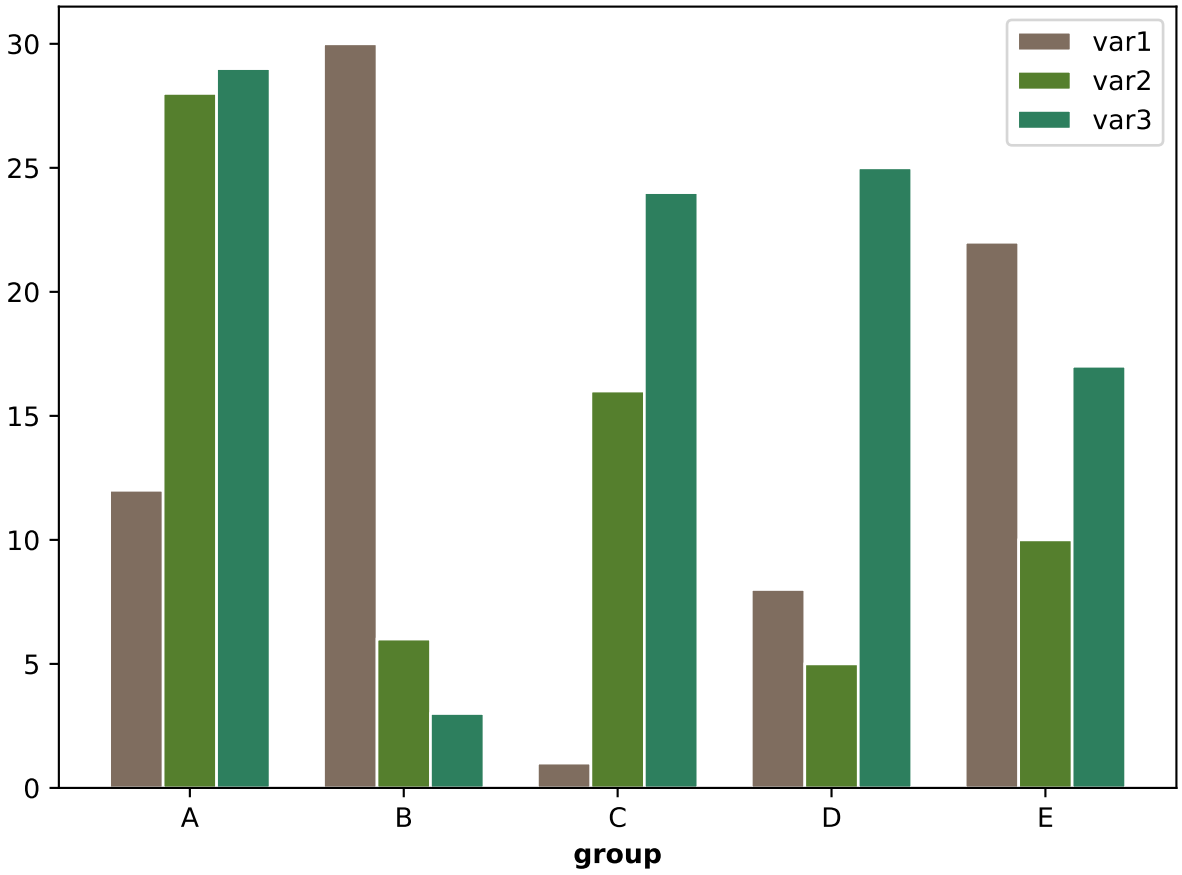
17.1.3 Stacked Bar Plot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_StackedBarPlot.py, and the XML code in the file D03_StackedBarPlot.2D.xml.



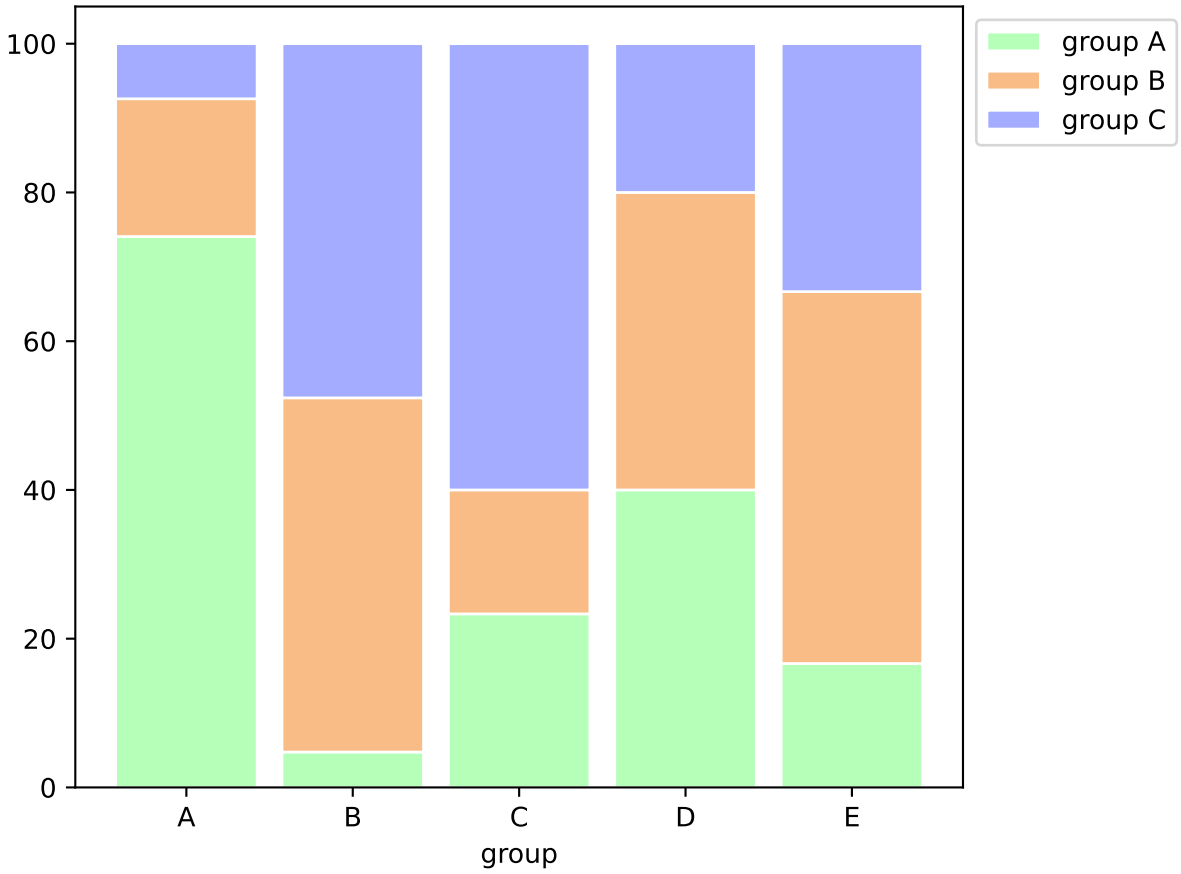
17.1.4 Grouped Bar Plot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_GroupedBarPlot.py, and the XML code in the file D04_GroupedBarPlot.2D.xml.



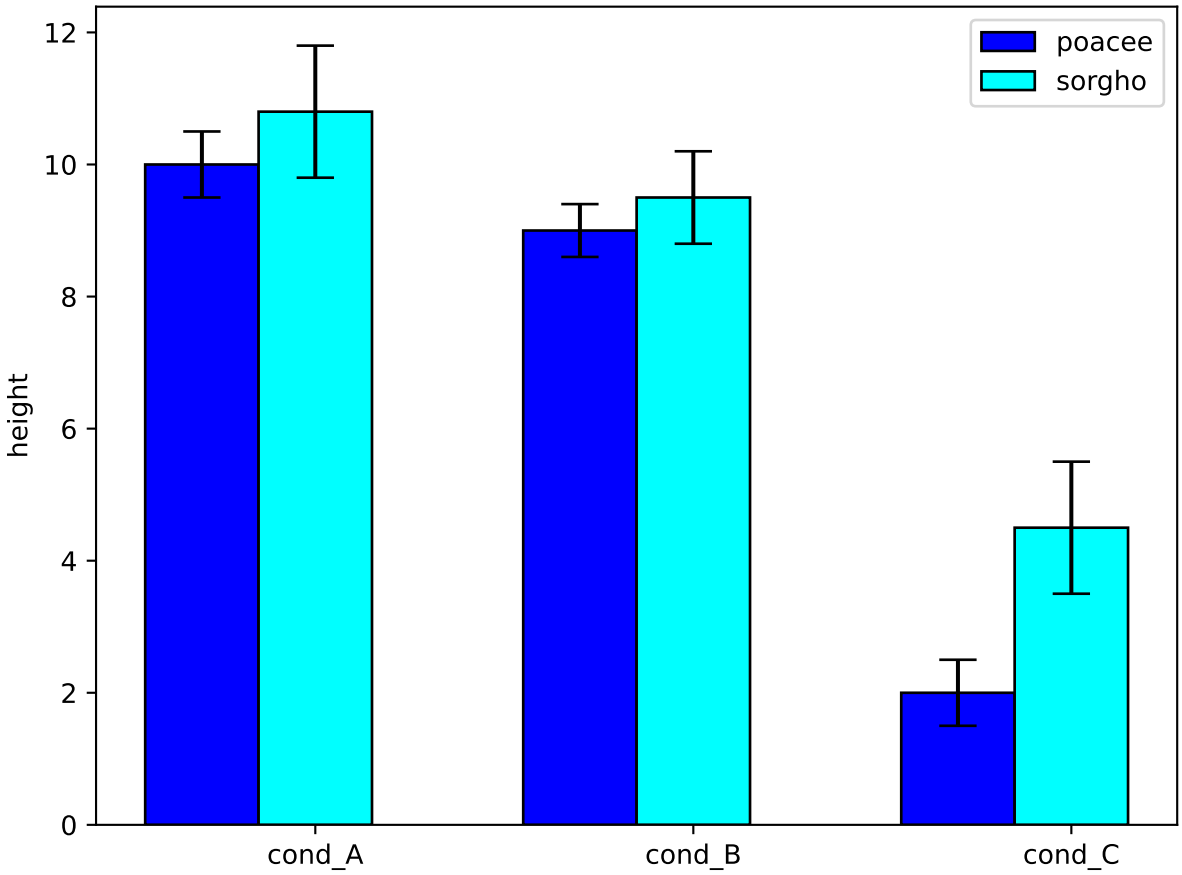
17.1.5 Grouped StackedBar Plot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_GroupedStackedBarPlot.py, and the XML code in the file D05_GroupedStackedBarPlot.2D.xml.



17.1.6 Barplot with error bar

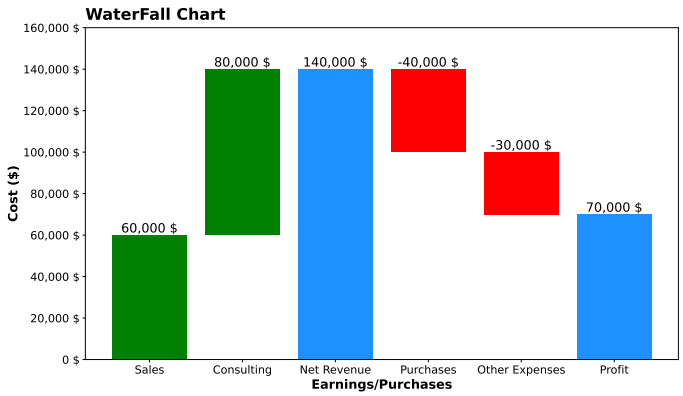
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_BarPlotWithErrorBar.py, and the XML code in the file D06_BarPlotWithErrorBar.2D.xml.

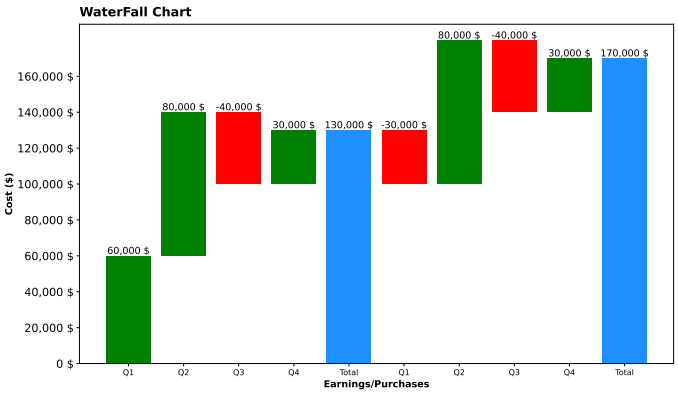


17.1.7 Waterfall Plot

The Python code for the first example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_WaterfallChart1.py, and the XML code in the file D07a_WaterfallChart1.2D.xml.

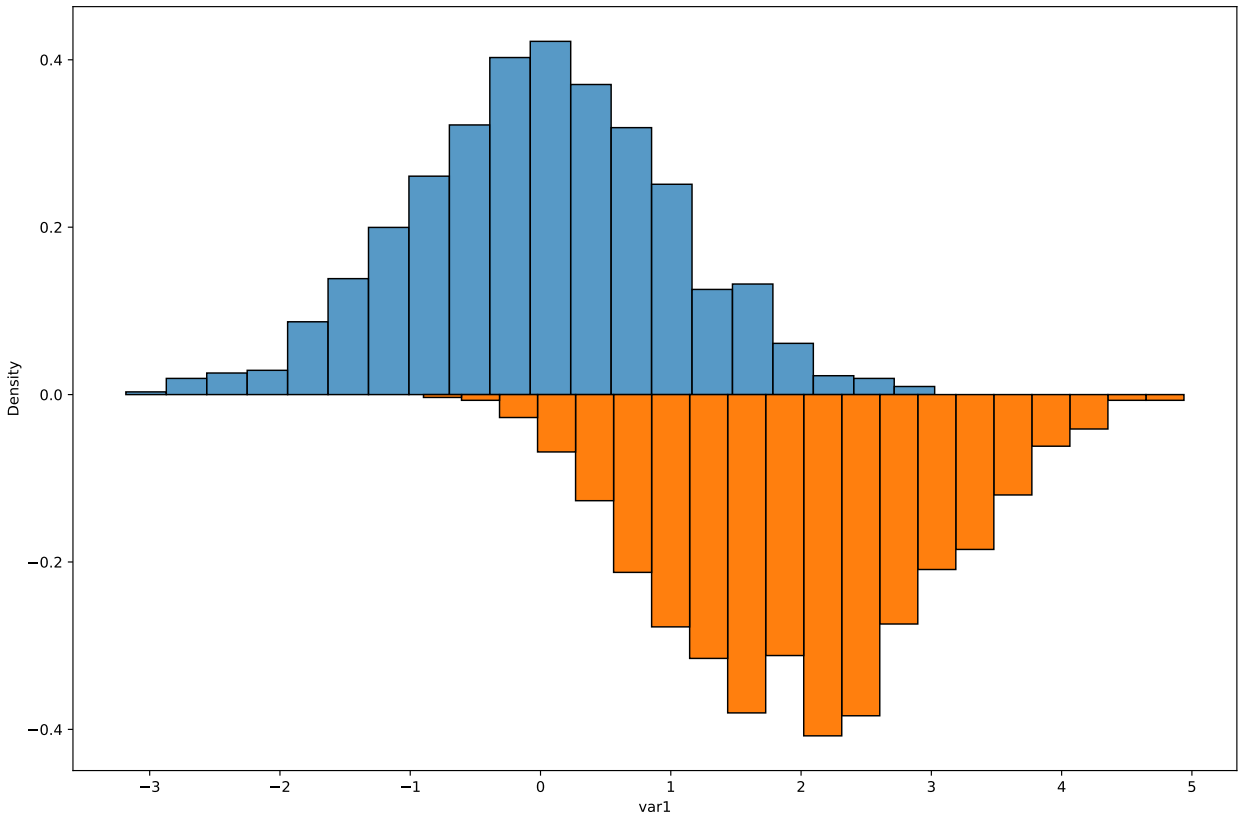
The Python code for the second example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_WaterfallChart2.py, and the XML code in the file D07b_WaterfallChart2.2D.xml.





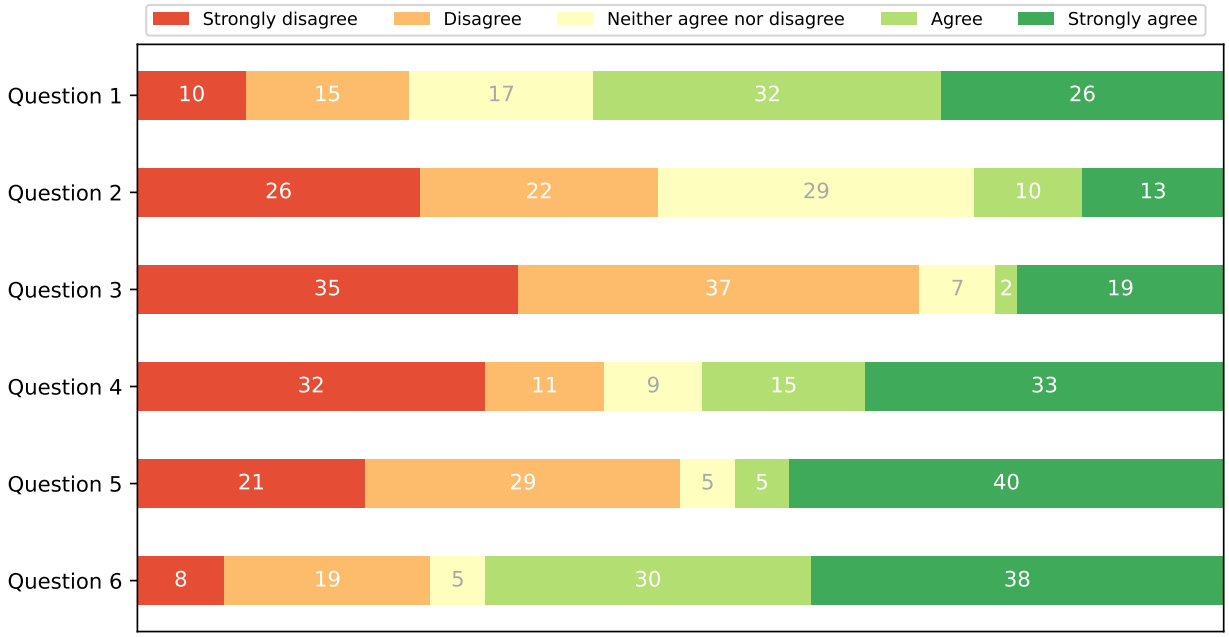
17.1.8 Mirror Histogram

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_MirrorHistogram.py, and the XML code in the file D08_MirrorHistogram.2D.xml.



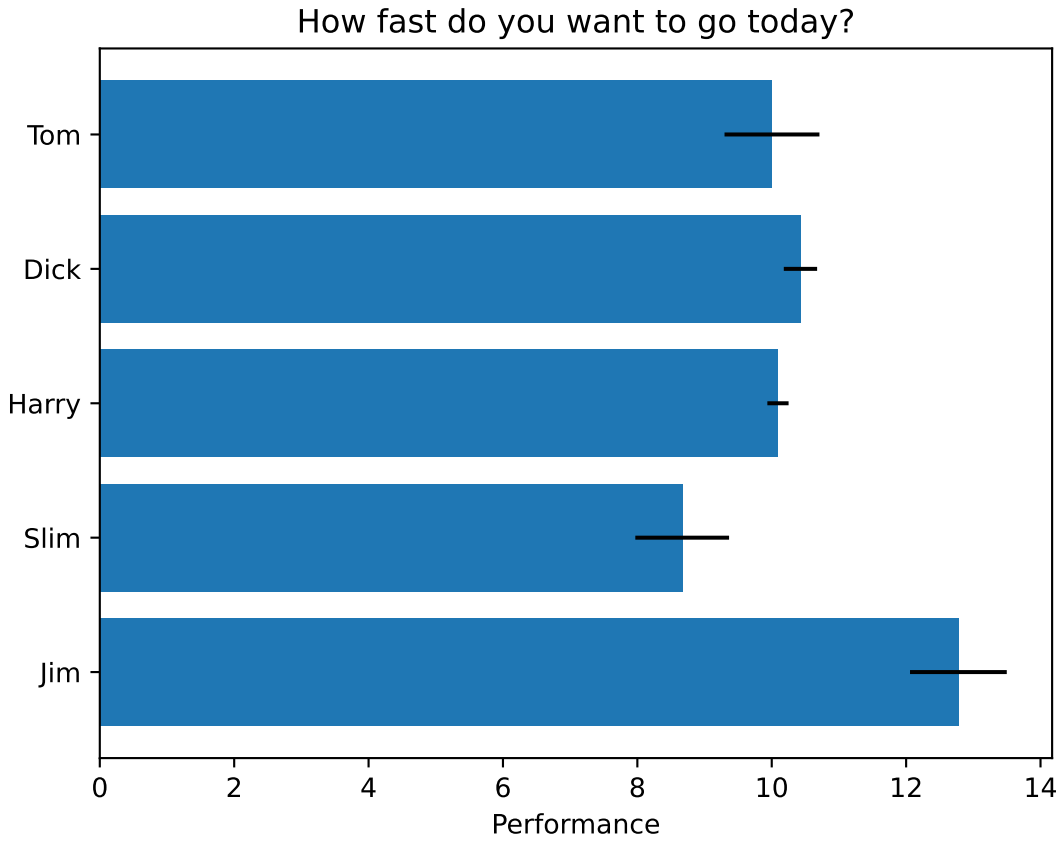
17.1.9 Horizontal Bar

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_HorizontalBarDist.py, and the XML code in the file D09_HorizontalBarDist.2D.xml.



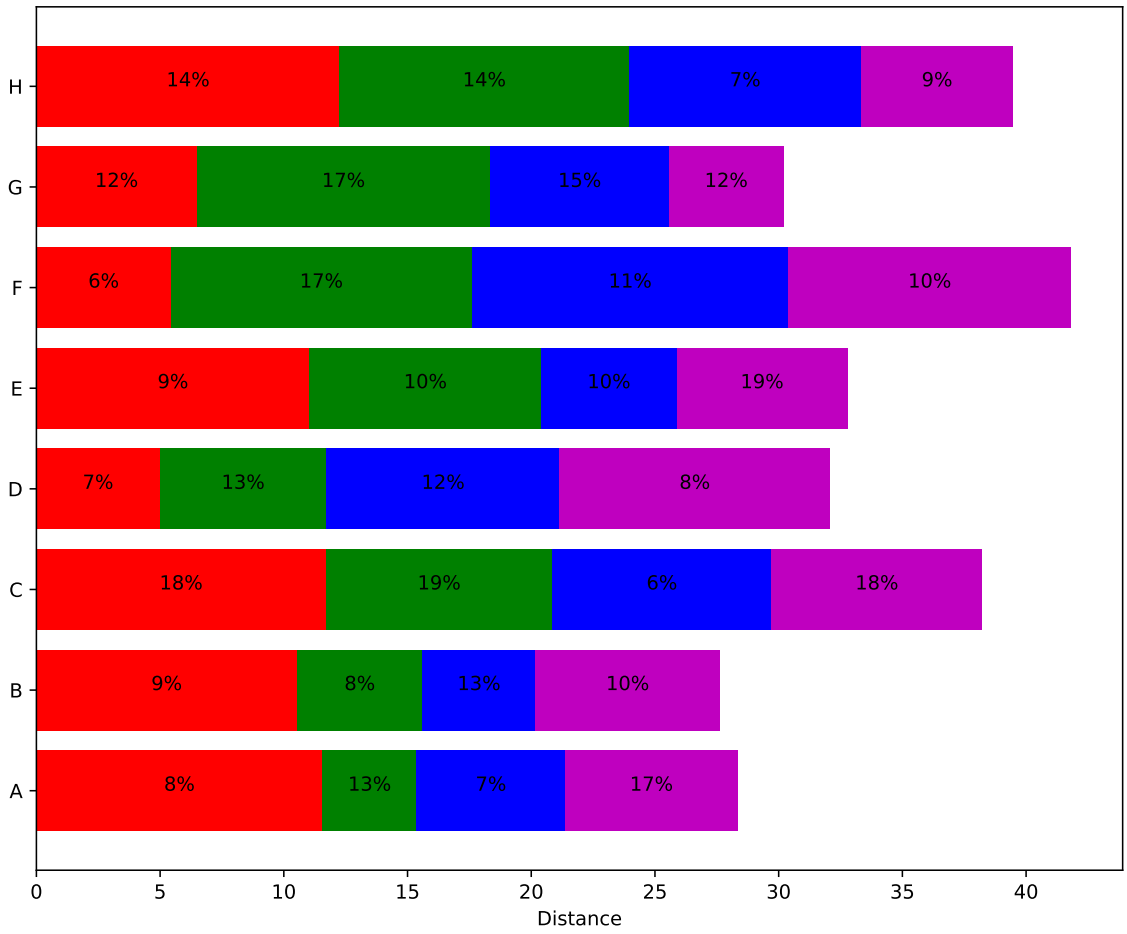
17.1.10 Horizontal Error Bar

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_HorizontalErrorBar.py, and the XML code in the file D10_HorizontalErrorBar.2D.xml.



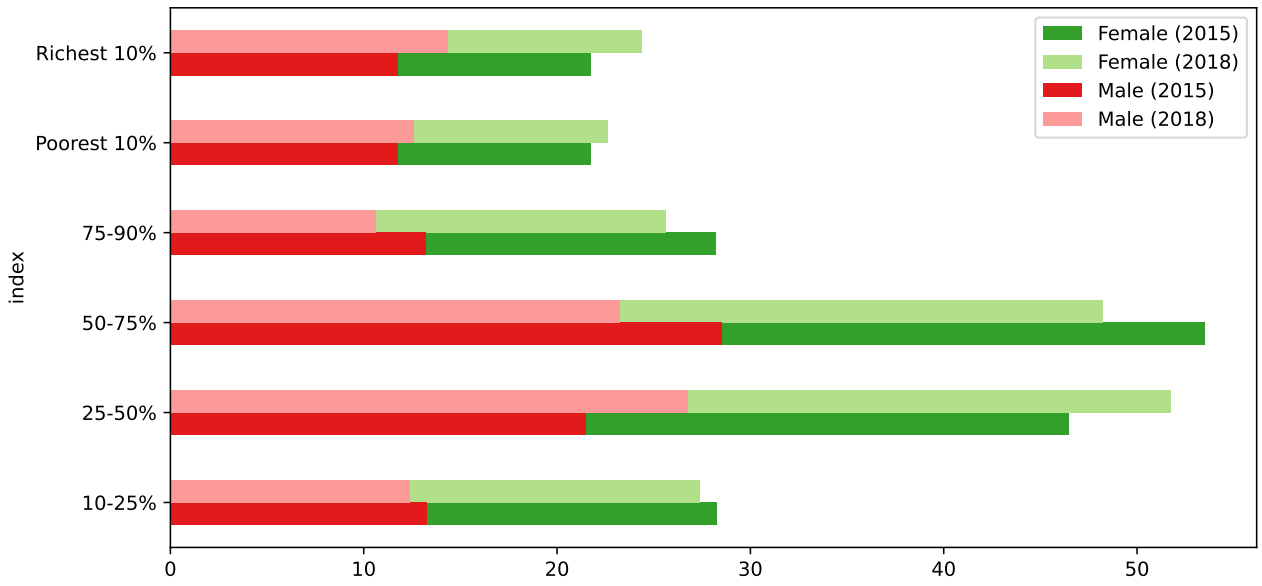
17.1.11 Stacked horizontal bar with labels

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11_StackedHBarWithLabels.py, and the XML code in the file D11_StackedHBarWithLabels.2D.xml.



17.1.12 Stacked and grouped horizontal bar

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D12_StackedAndGroupedHBar.py, and the XML code in the file D12_StackedAndGroupedHBar.2D.xml.

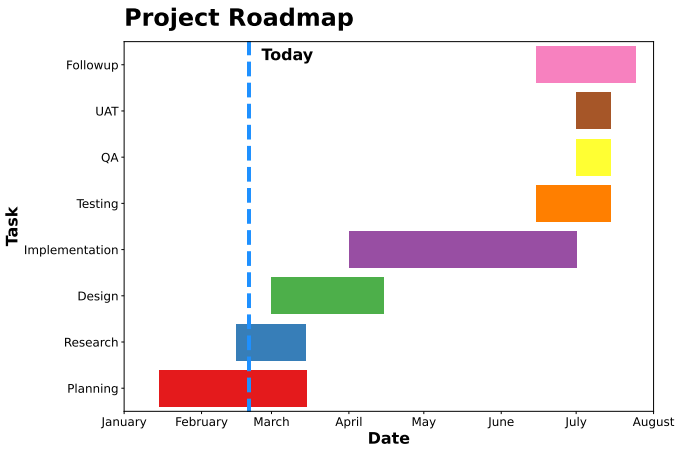


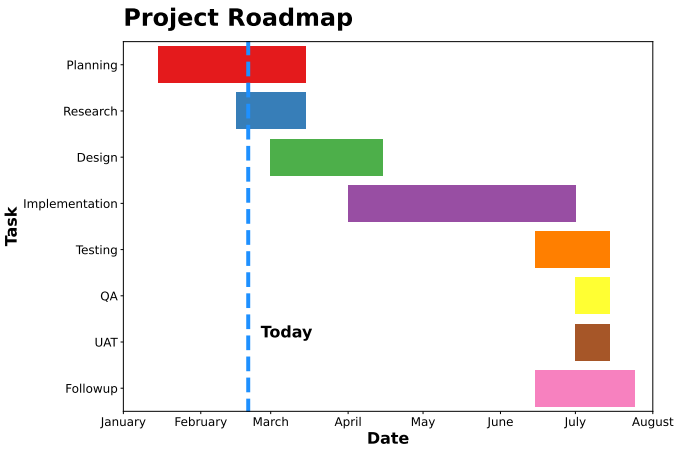
17.1.13 Gantt Chart

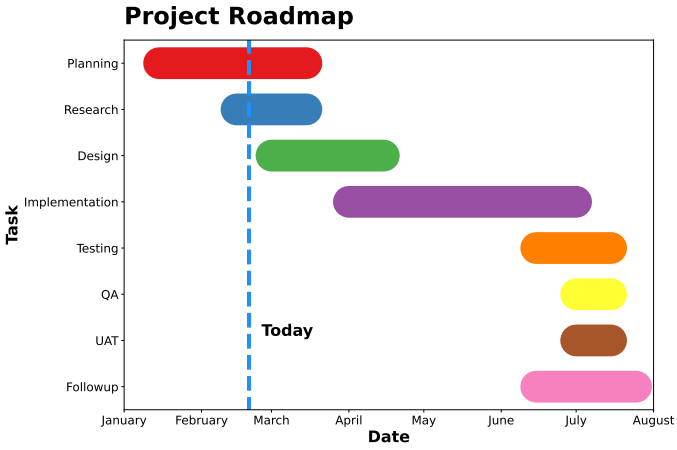
The Python code for the first example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D13a_GanttChart1.py, and the XML code in the file D13a_GanttChart1.2D.xml.

The Python code for the second example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D13b_GanttChart2.py, and the XML code in the file D13b_GanttChart2.2D.xml.

The Python code for the third example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D13c_GanttChart3.py, and the XML code in the file D13c_GanttChart3.2D.xml.

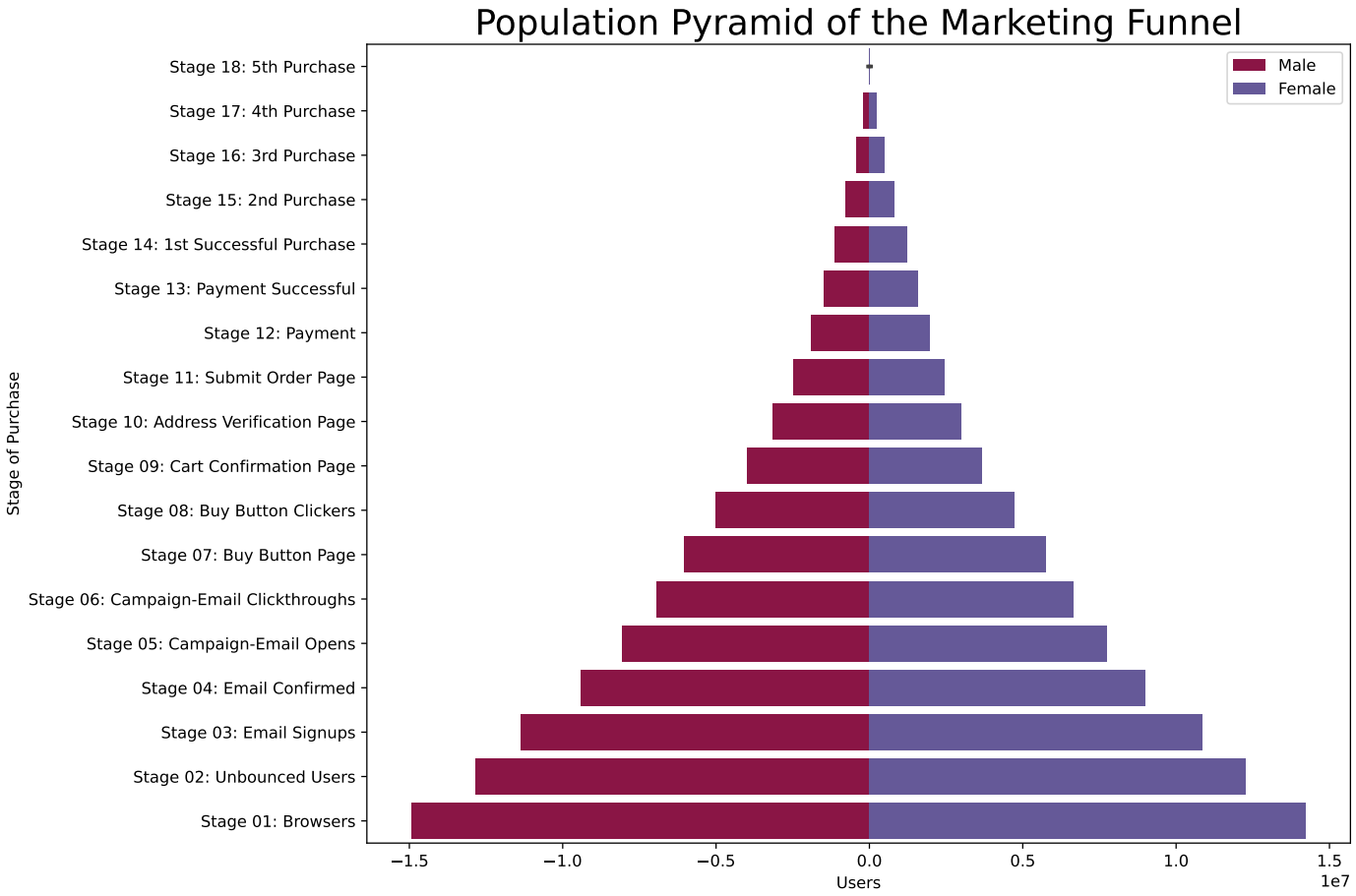






17.1.14 Marketing Funnel

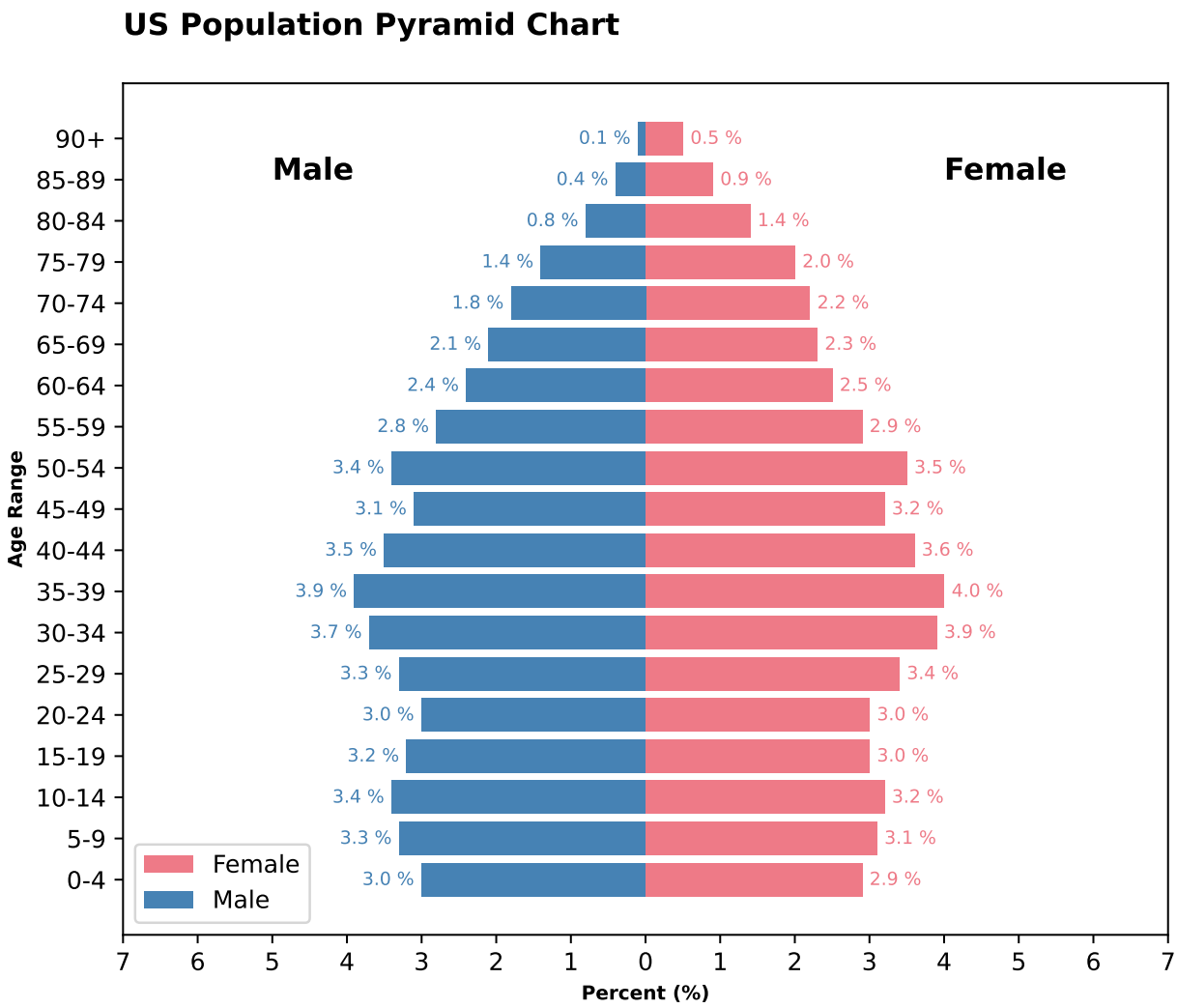
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D14_MarketingFunnel.py, and the XML code in the file D14_MarketingFunnel.2D.xml.



Some text

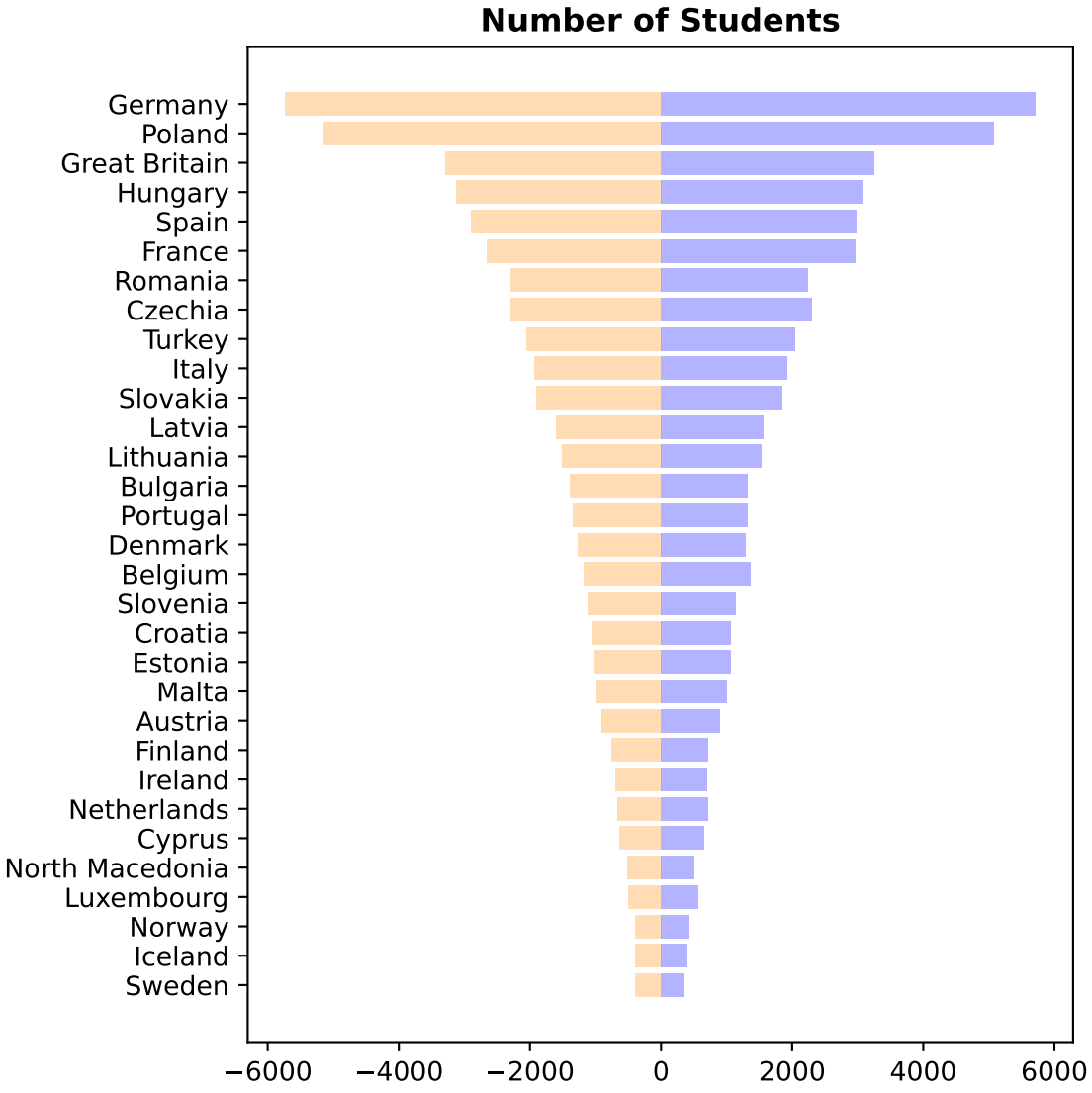
17.1.15 Population Pyramid

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D15_PopulationPyramid.py, and the XML code in the file D15_PopulationPyramid.2D.xml.



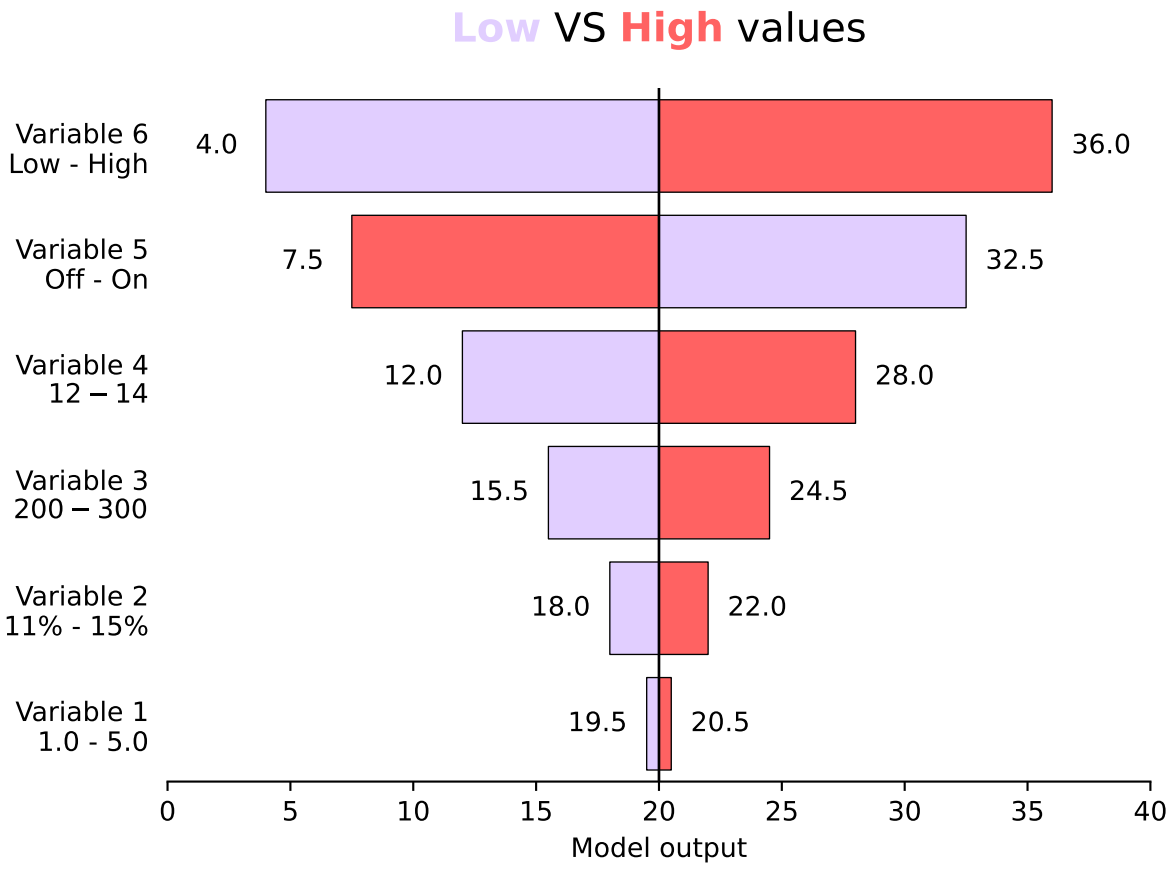
17.1.16 Mirror Bar Plot (horizontal)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D16_MirrorBarPlot.py, and the XML code in the file D16_MirrorBarPlot.2D.xml.



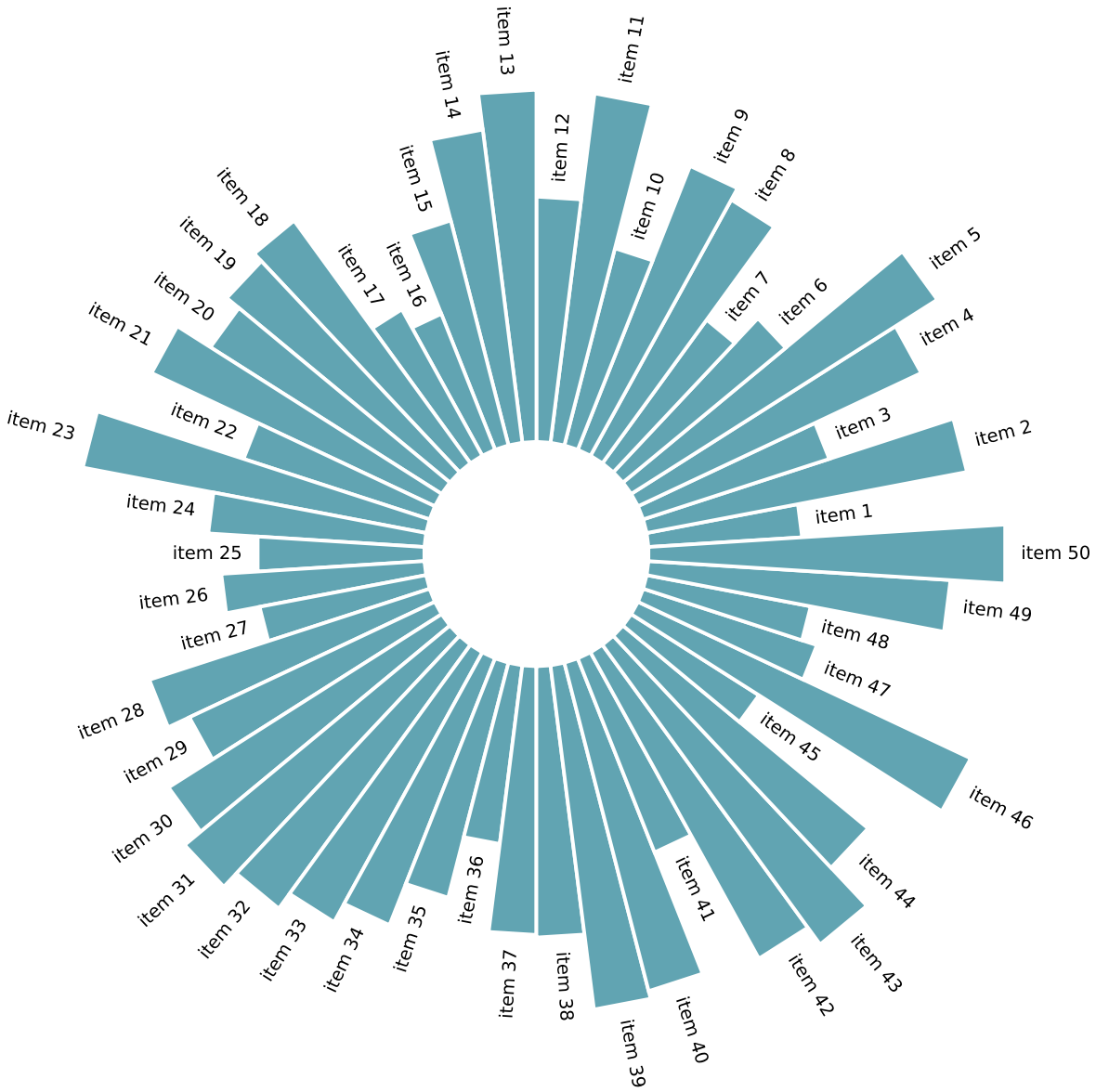
17.1.17 Tornado Chart

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D17_TornadoChart.py, and the XML code in the file D17_TornadoChart.2D.xml.



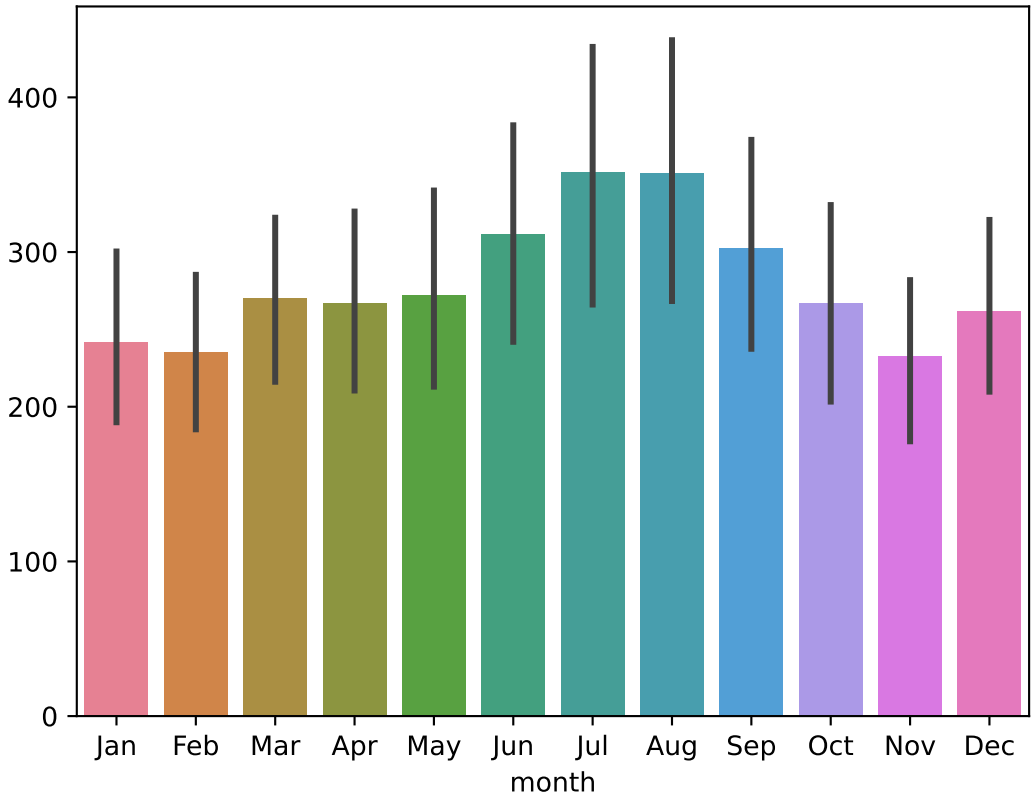
17.1.18 Circular Barplot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D18_CircularBarplot.py, and the XML code in the file D18_CircularBarplot.2D.xml.



17.1.19 Barplot with color

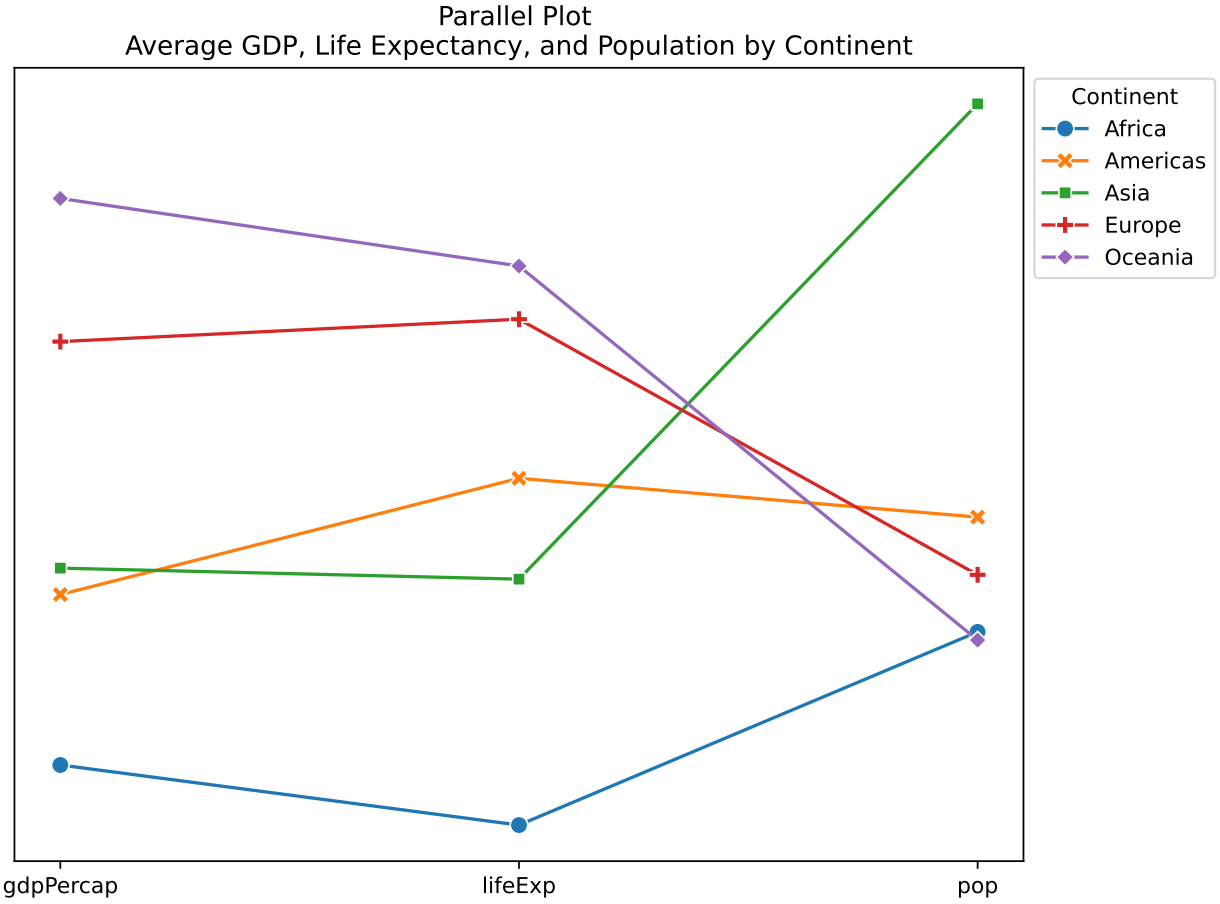
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D19_BarplotColor.py, and the XML code in the file D19_BarplotColor.2D.xml.



17.2 Line and lollipop charts

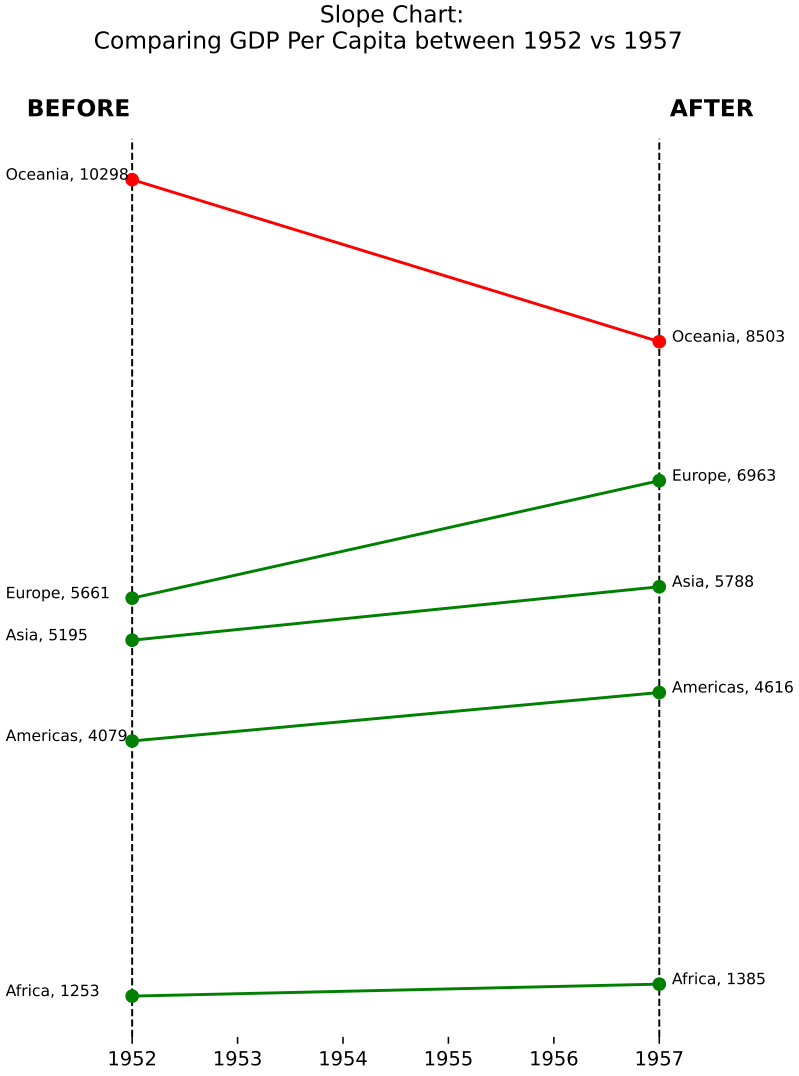
17.2.1 Line chart (Seaborn)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_LineChartSeaborn.py, and the XML code in the file D01_LineChartSeaborn.2D.xml.



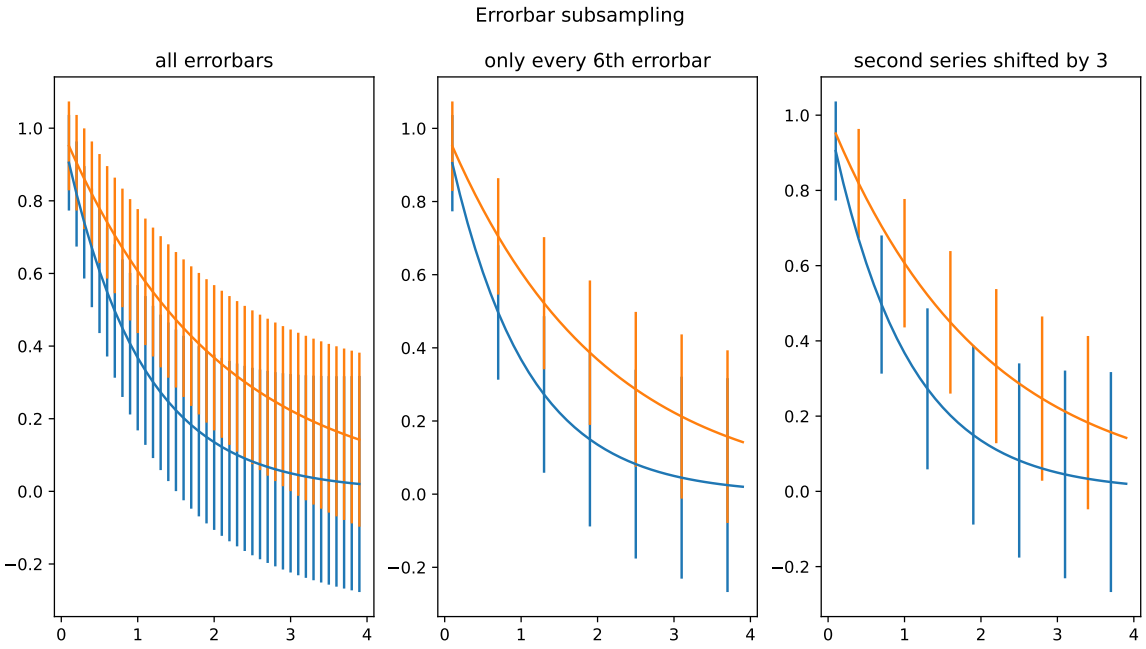
17.2.2 Slope Chart

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_SlopeCharts.py, and the XML code in the file D02_SlopeCharts.2D.xml.



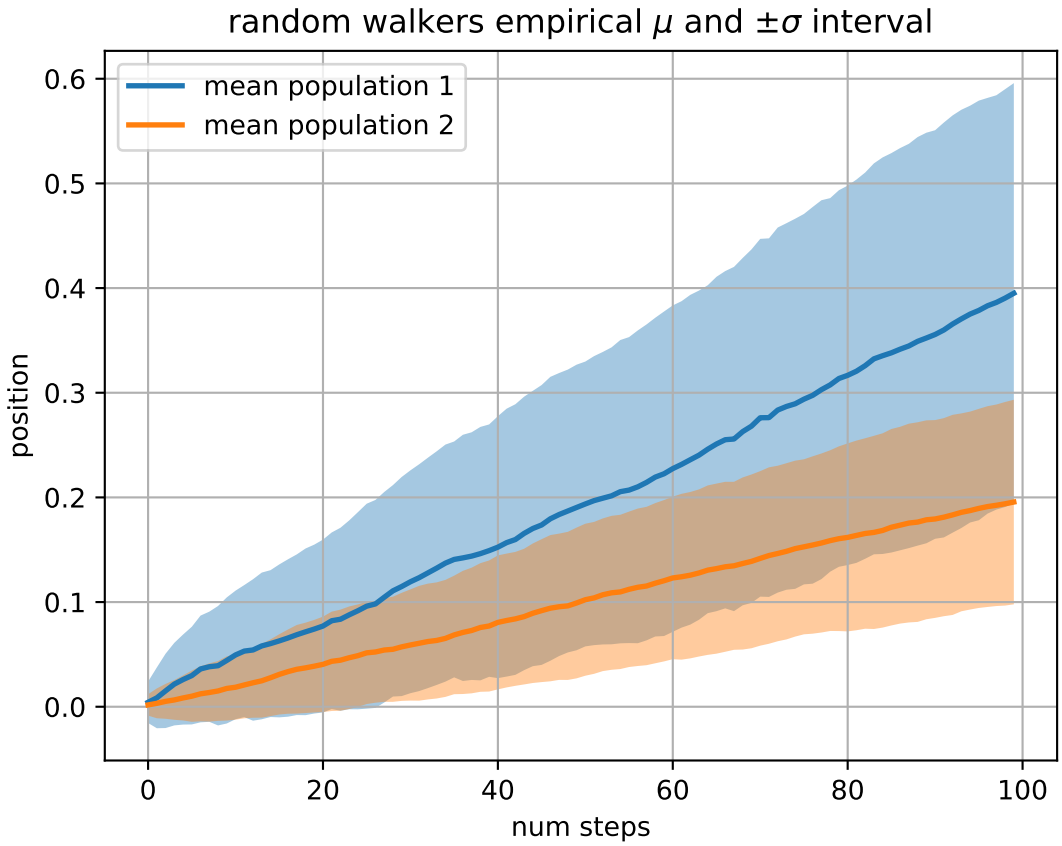
17.2.3 Line chart with error Bar

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_LineErrorBars.py, and the XML code in the file D03_LineErrorBars.2D.xml.



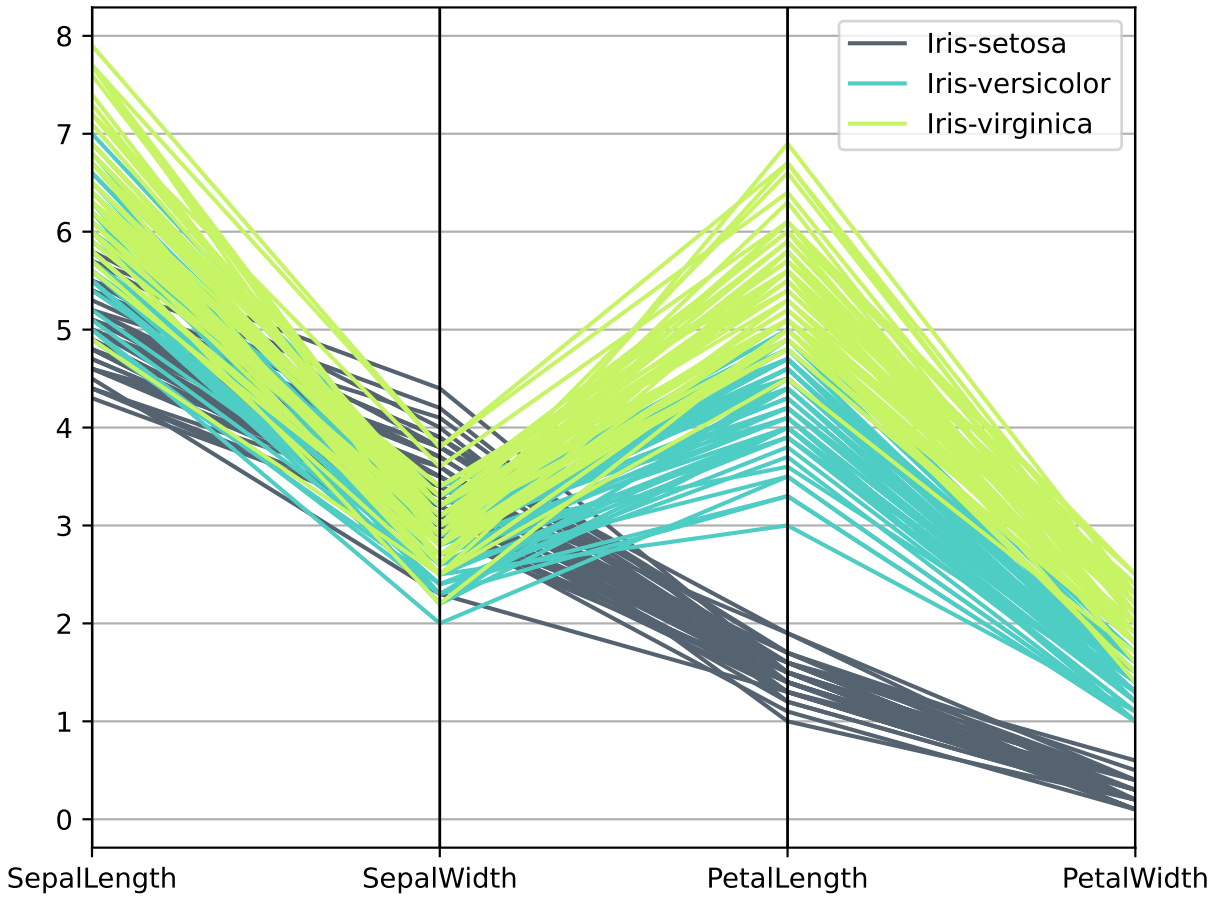
17.2.4 Confidence band (RandomWalk)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_ConfidenceBand_RandomWalk.py, and the XML code in the file D04_ConfidenceBand_RandomWalk.2D.xml.



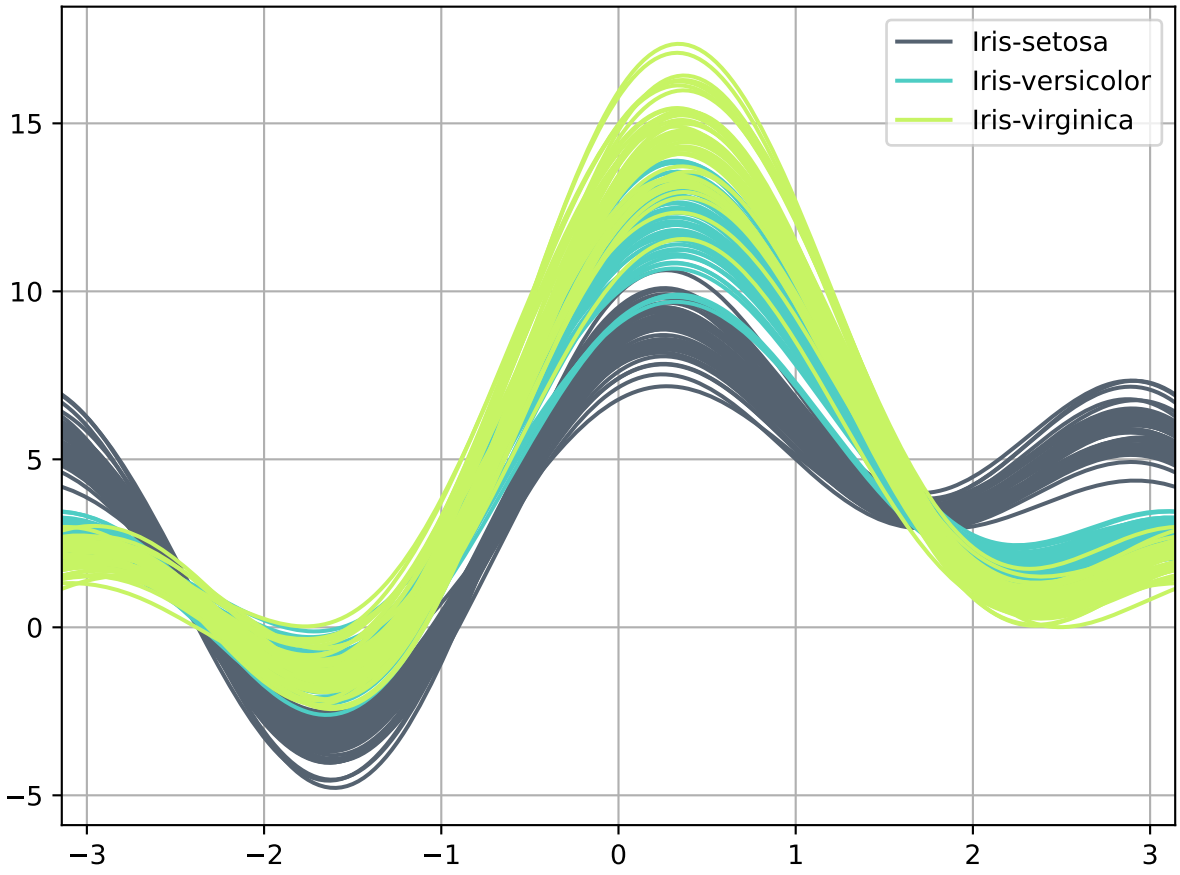
17.2.5 Plotting Parallel (Pandas)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_PandasPlottingParallel.py, and the XML code in the file D05_PandasPlottingParallel.2D.xml.



17.2.6 Andrew Curves (Pandas)

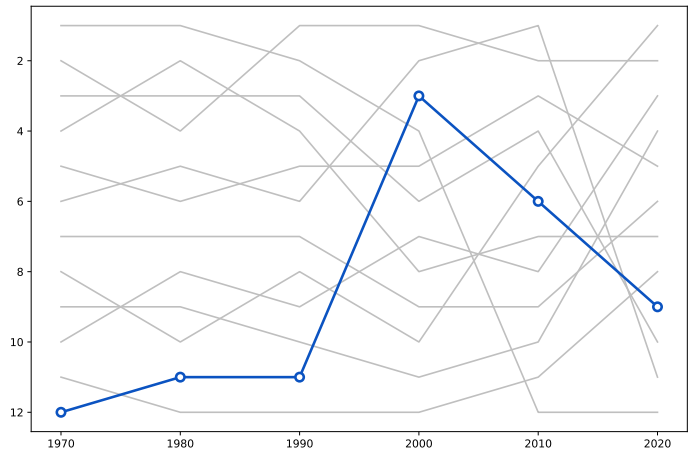
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_PandasAndrewCurves.py, and the XML code in the file D06_PandasAndrewCurves.2D.xml.



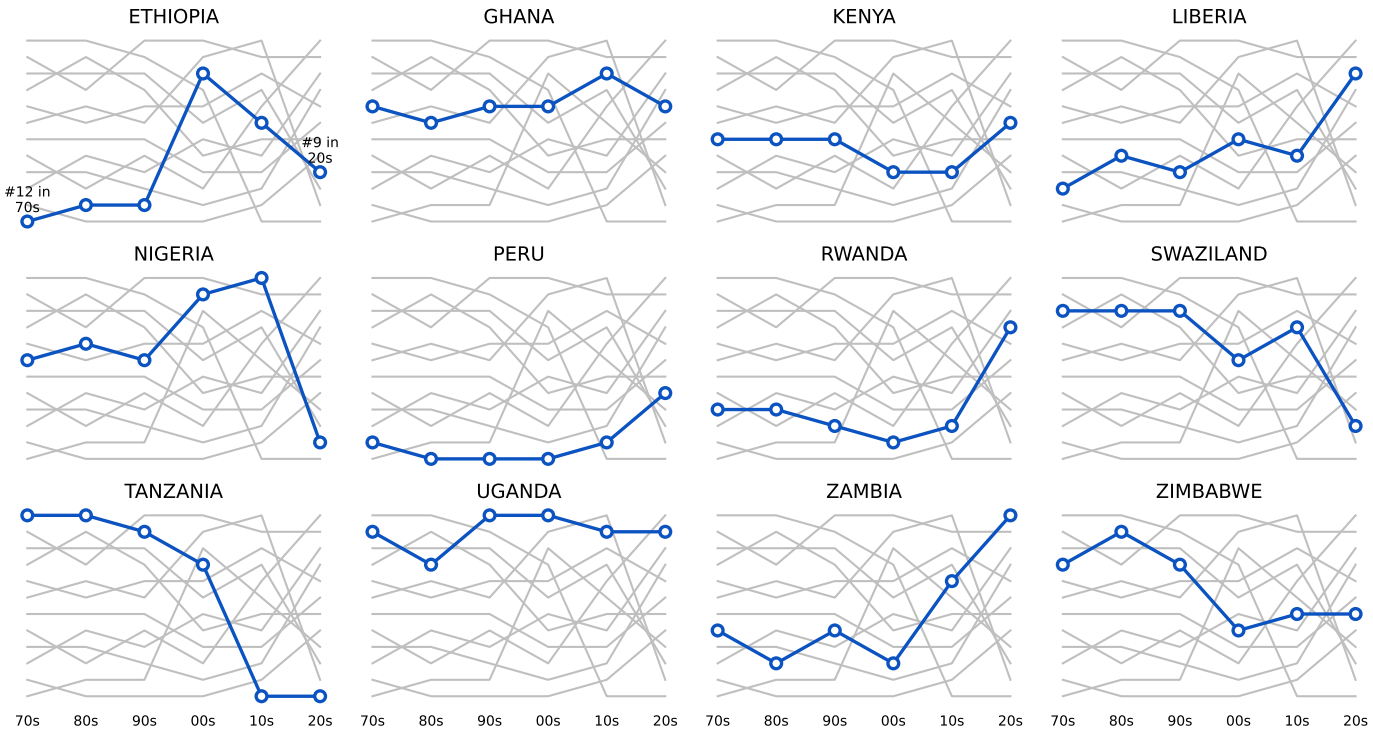
17.2.7 Highlighted LinePlot

The Python code for the first example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_HighlightedLinePlot1.py, and the XML code in the file D07a_HighlightedLinePlot1.2D.xml.

The Python code for the second example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_HighlightedLinePlot2.py, and the XML code in the file D07b_HighlightedLinePlot2.2D.xml.



Some text

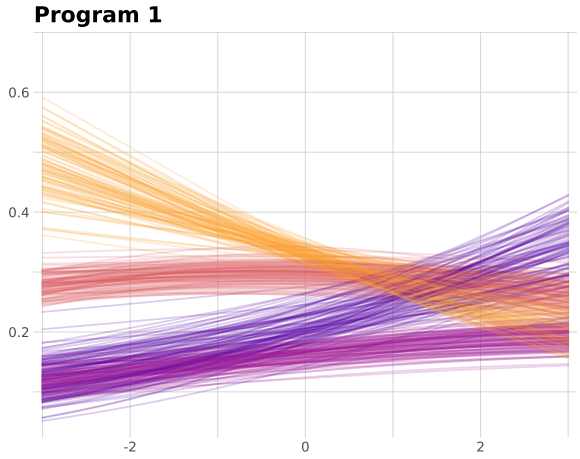


Some text

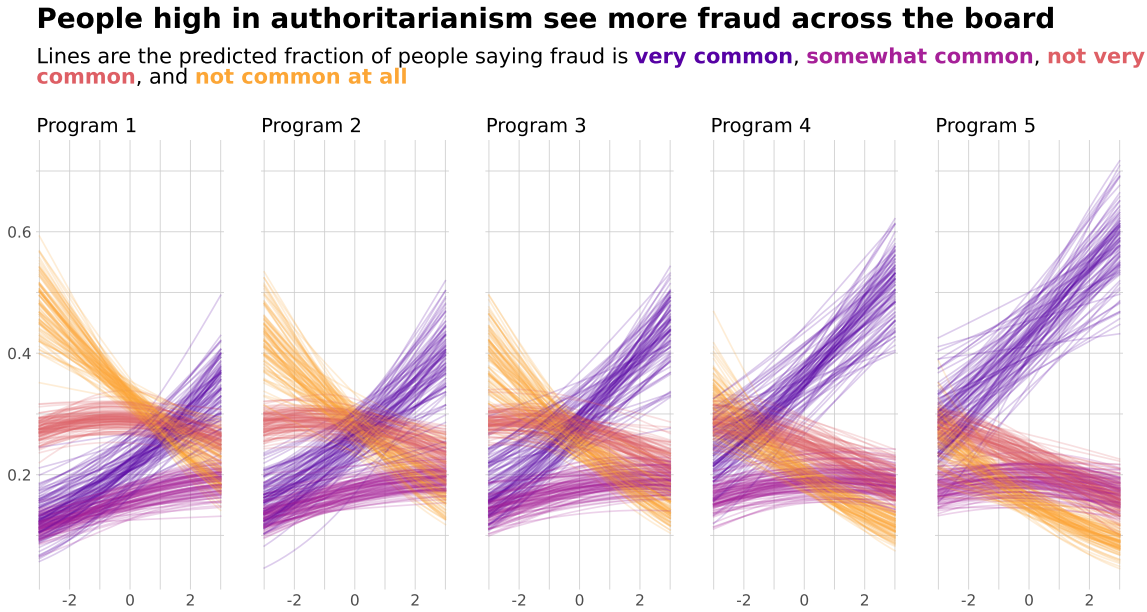
17.2.8 Multiple Lines And Panels

The Python code for the first example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08a_MultipleLinesAndPanels1.py, and the XML code in the file D08a_MultipleLinesAndPanels1.2D.xml.

The Python code for the second example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08b_MultipleLinesAndPanels2.py, and the XML code in the file D08b_MultipleLinesAndPanels2.2D.xml.



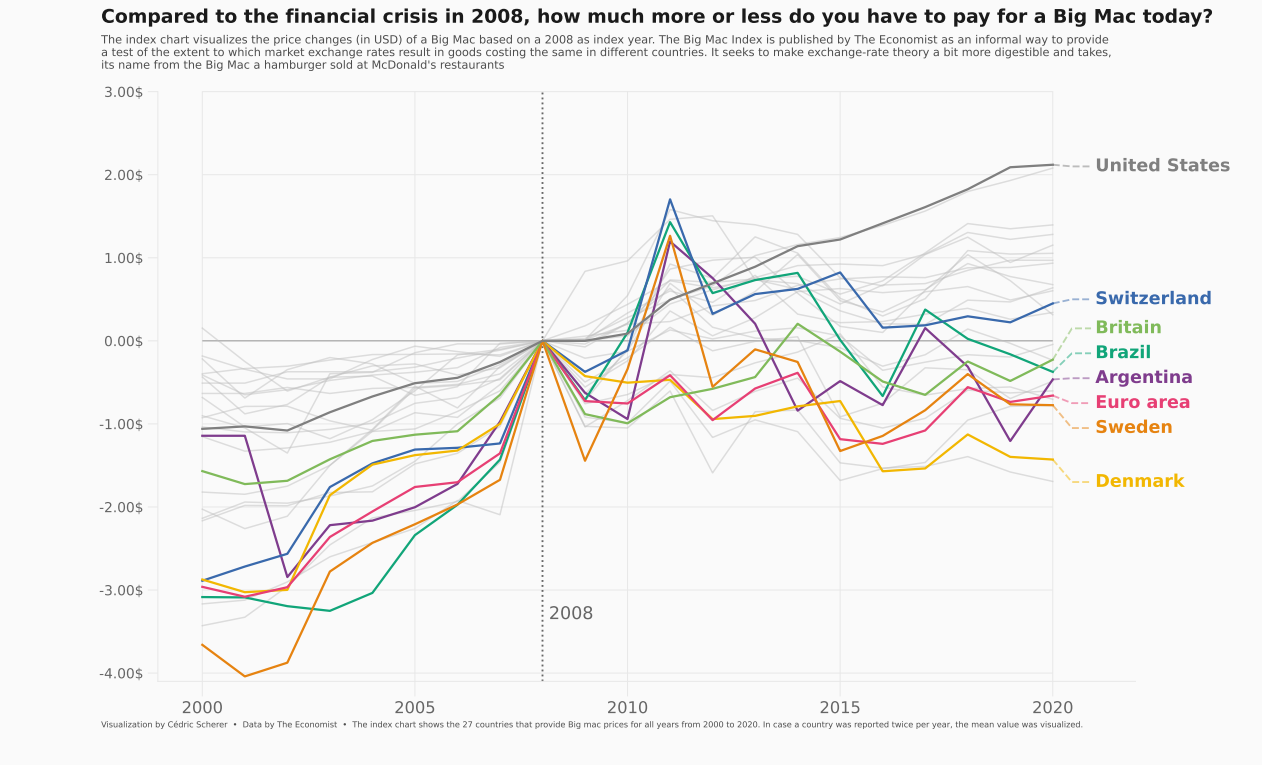
Some text



Some text

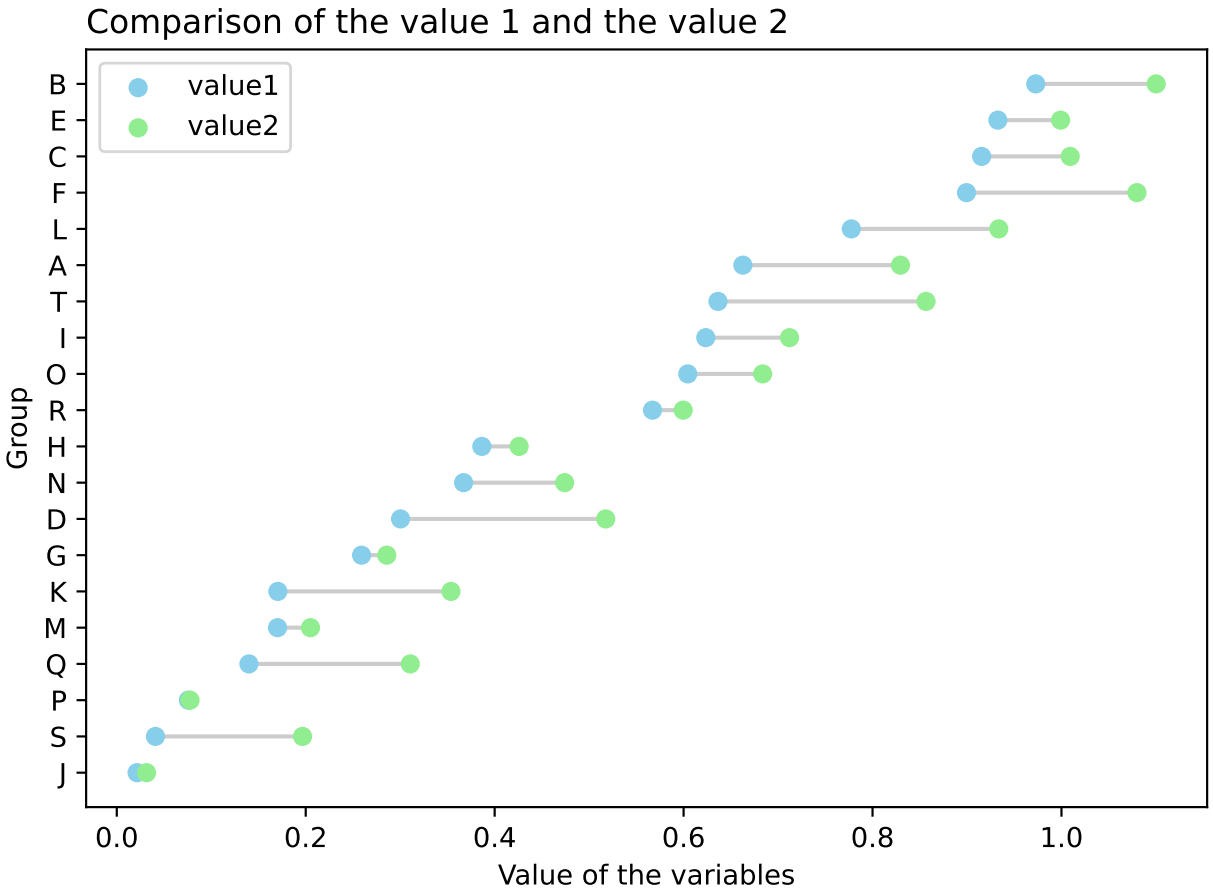
17.2.9 Line chart with fancy formatting

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_LineChartWithFancyFormatting.py, and the XML code in the file D09_LineChartWithFancyFormatting.2D.xml.



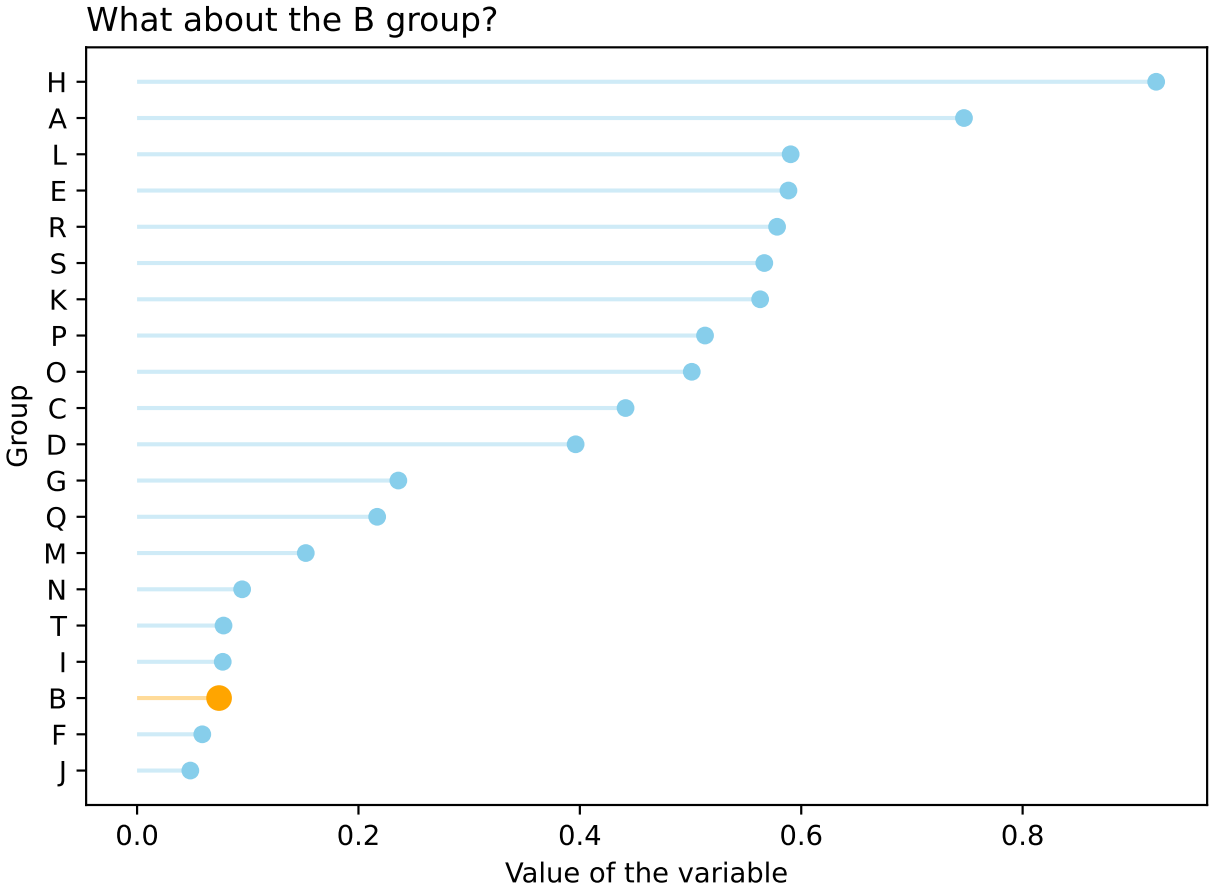
17.2.10 Lolllipop 2 Groups

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_Lolllipop2Groups.py, and the XML code in the file D10_Lolllipop2Groups.2D.xml.



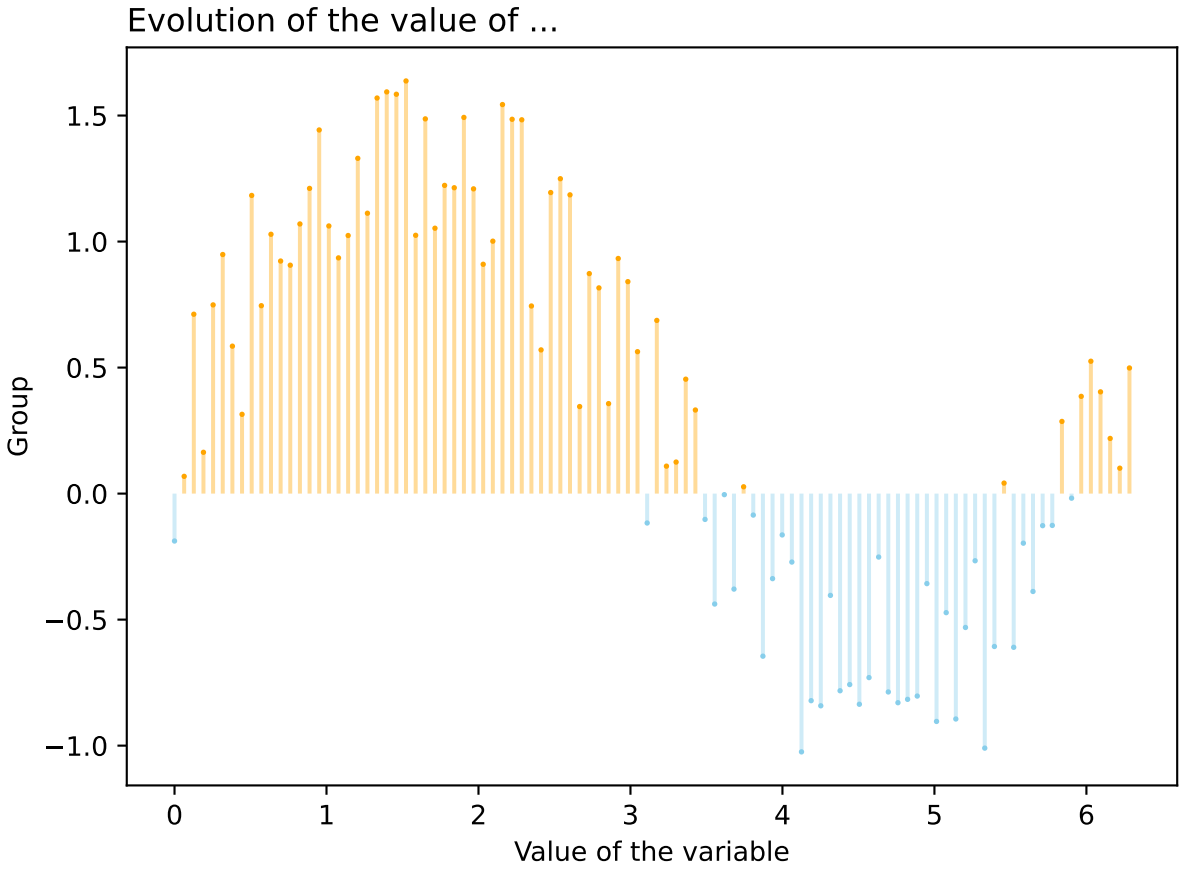
17.2.11 Highlight Group In Lollipop

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11_HighlightGroupInLollipop.py, and the XML code in the file D11_HighlightGroupInLollipop.2D.xml.



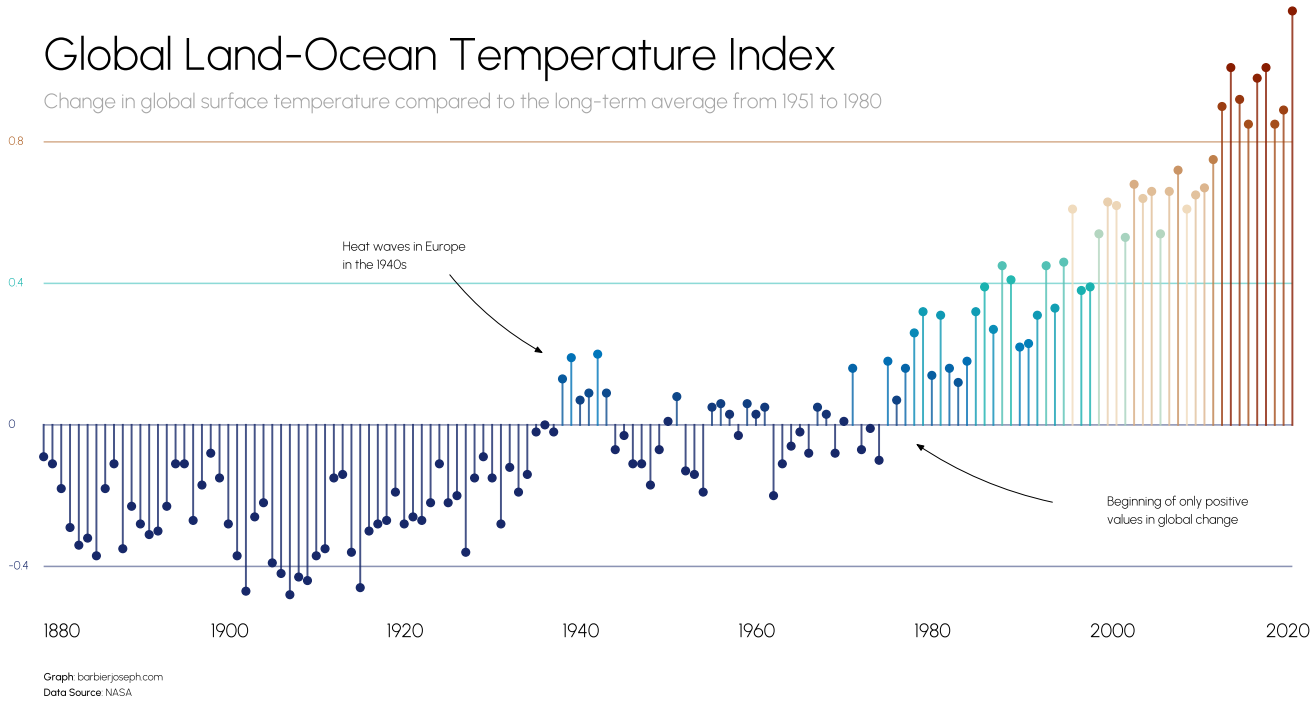
17.2.12 Lolllipop Conditional Color

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D12_LolllipopConditionalColor.py, and the XML code in the file D12_LolllipopConditionalColor.2D.xml.



17.2.13 Lollipop chart with fancy formatting

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D13_LollipopFancyFormatting.py, and the XML code in the file D13_LollipopFancyFormatting.2D.xml.

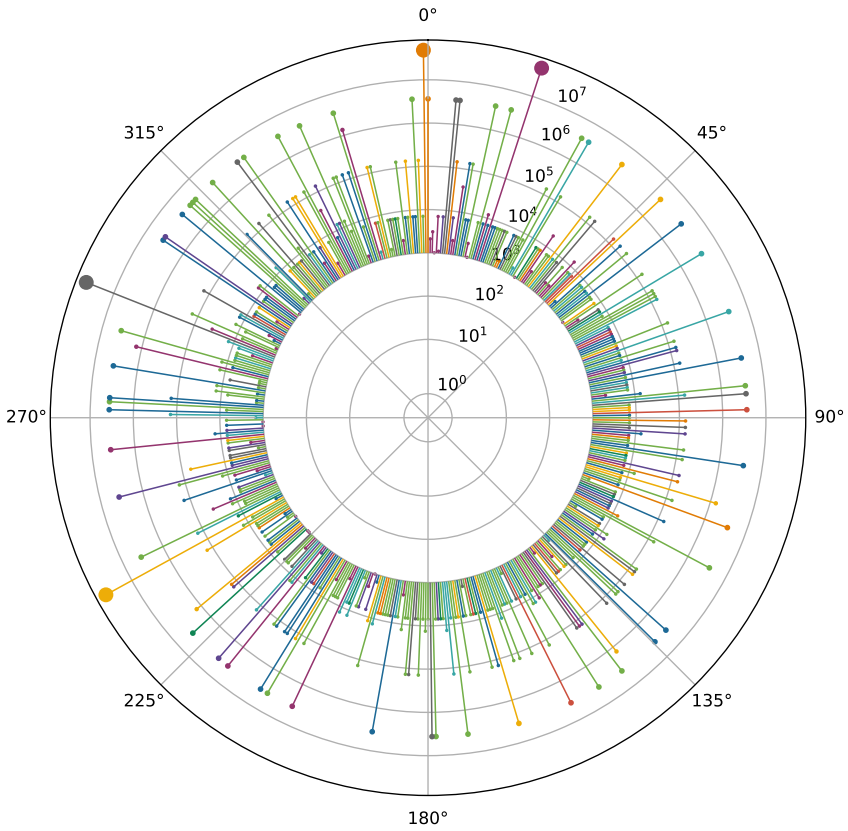


17.2.14 Circular Lolllipop

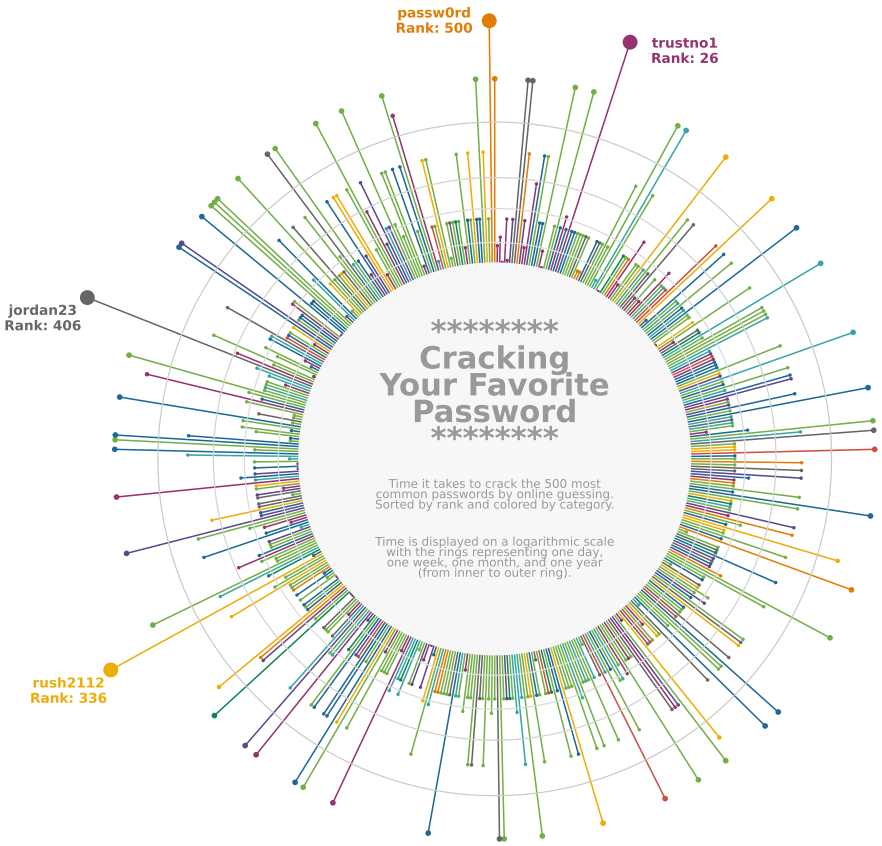
The Python code for the first example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D14a_CircularLolllipop1.py, and the XML code in the file D14a_CircularLolllipop1.2D.xml.

The Python code for the second example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D14b_CircularLolllipop2.py, and the XML code in the file D14b_CircularLolllipop2.2D.xml.

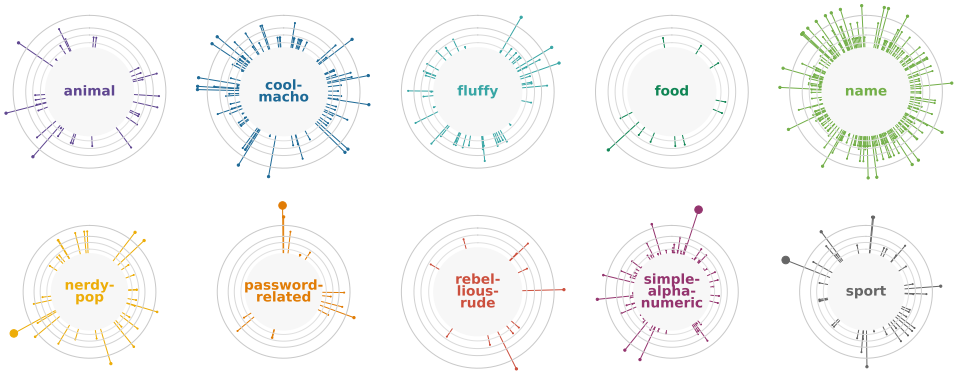
The Python code for the third example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D14b_CircularLolllipop3.py, and the XML code in the file D14c_CircularLolllipop3.2D.xml.



Some text



Some text

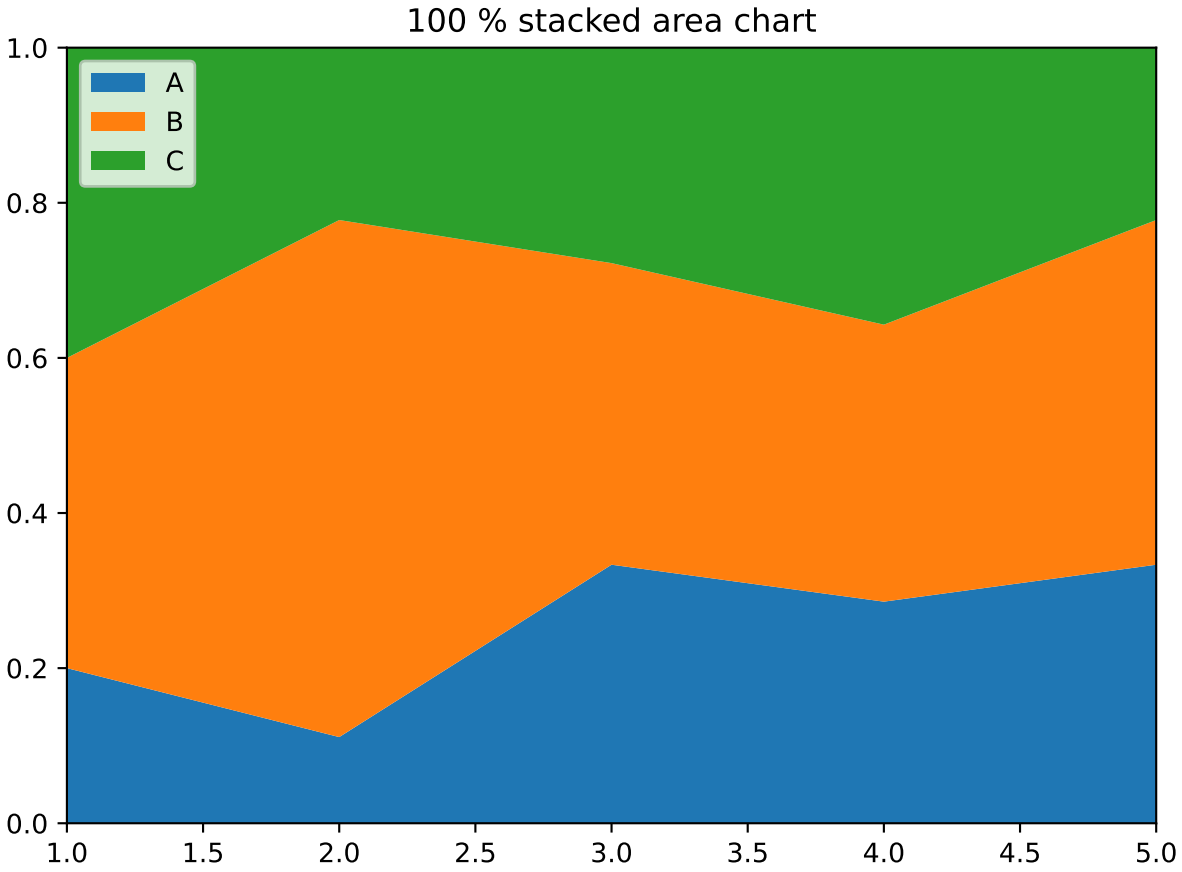


Some text

17.3 Area Plots

17.3.1 Percentage Stacked Area Chart

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_PercentageStackedAreaChart.py, and the XML code in the file D01_PercentageStackedAreaChart.2D.xml.



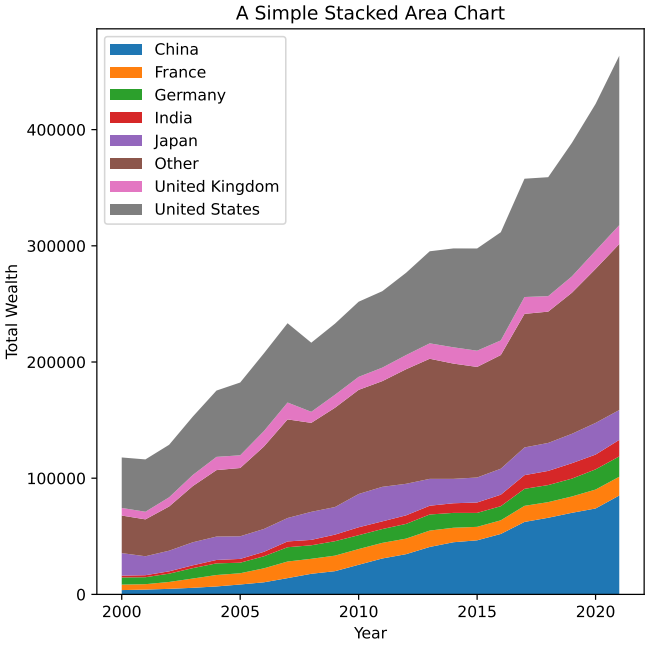
17.3.2 StackedAreaChart1

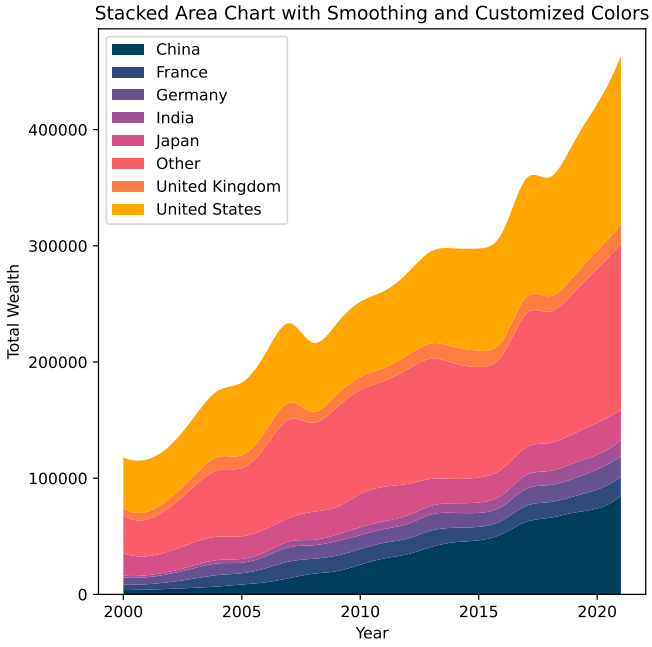
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02a_StackedAreaChart1.py, and the XML code in the file D02a_StackedAreaChart1.2D.xml.

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02b_StackedAreaChart2.py, and the XML code in the file D02b_StackedAreaChart2.2D.xml.

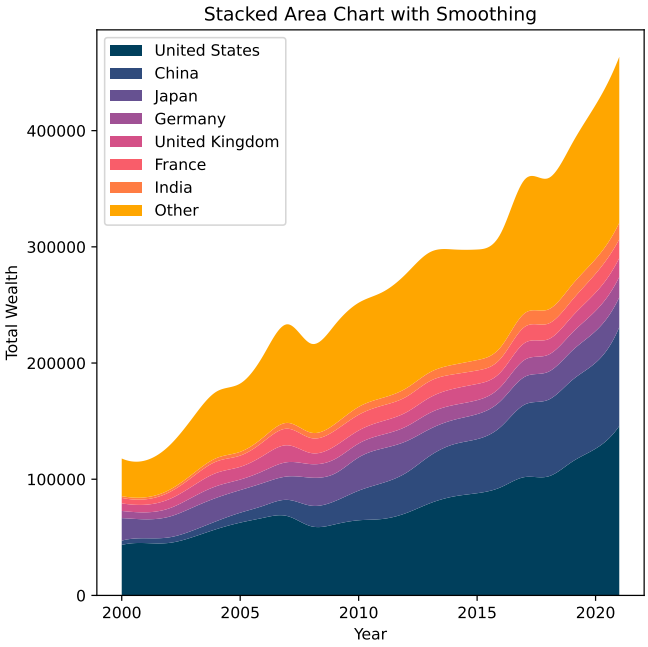
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02c_StackedAreaChart3.py, and the XML code in the file D02c_StackedAreaChart3.2D.xml.

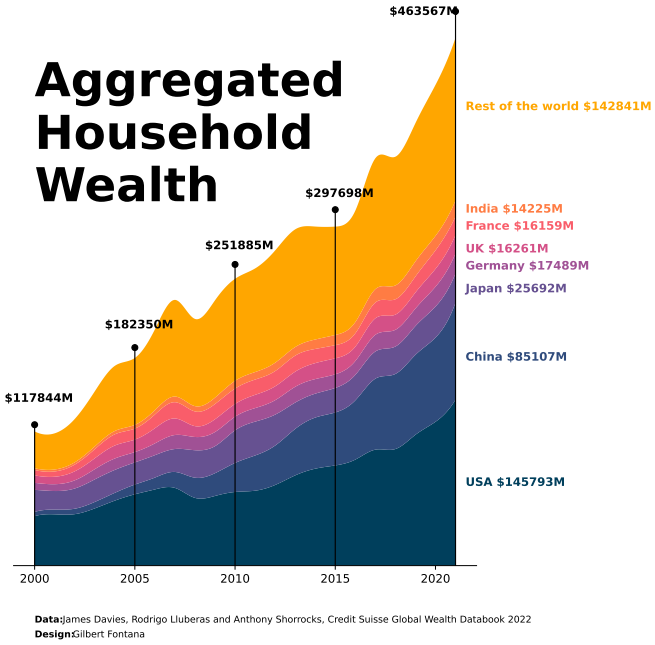
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02d_StackedAreaChart4.py, and the XML code in the file D02d_StackedAreaChart4.2D.xml.





Some text


Stacked Area Chart with Smoothing
United States
China
Japan
Germany
United Kingdom
France
India
Other
Total Wealth
0
100000
200000
300000
400000
2000
2005
2010
2015
2020
Year


Aggregated Household Wealth
$463567M
$297698M
$251885M
$182350M
$117844M
Rest of the world $142841M
India $14225M
France $16159M
UK $16261M
Germany $17489M
Japan $25692M
China $85107M
USA $145793M
2000
2005
2010
2015
2020
Data: James Davies, Rodrigo Lluberas and Anthony Shorrocks, Credit Suisse Global Wealth Databook 2022
Design: Gilbert Fontana

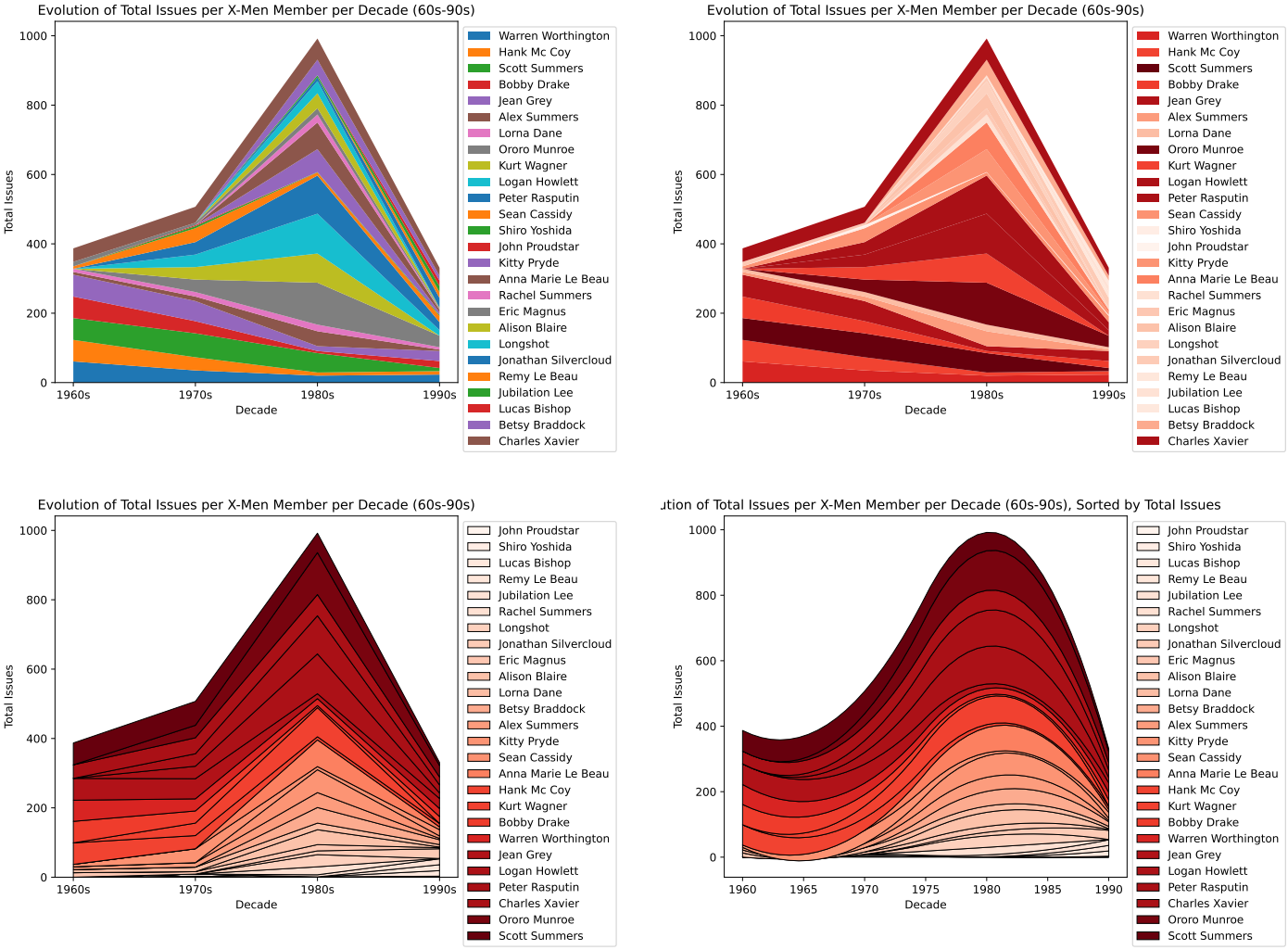
17.3.3 Steam graph

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03a_Steamgraph1.py, and the XML code in the file D03a_Steamgraph1.2D.xml.

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03b_Steamgraph2.py, and the XML code in the file D03b_Steamgraph2.2D.xml.

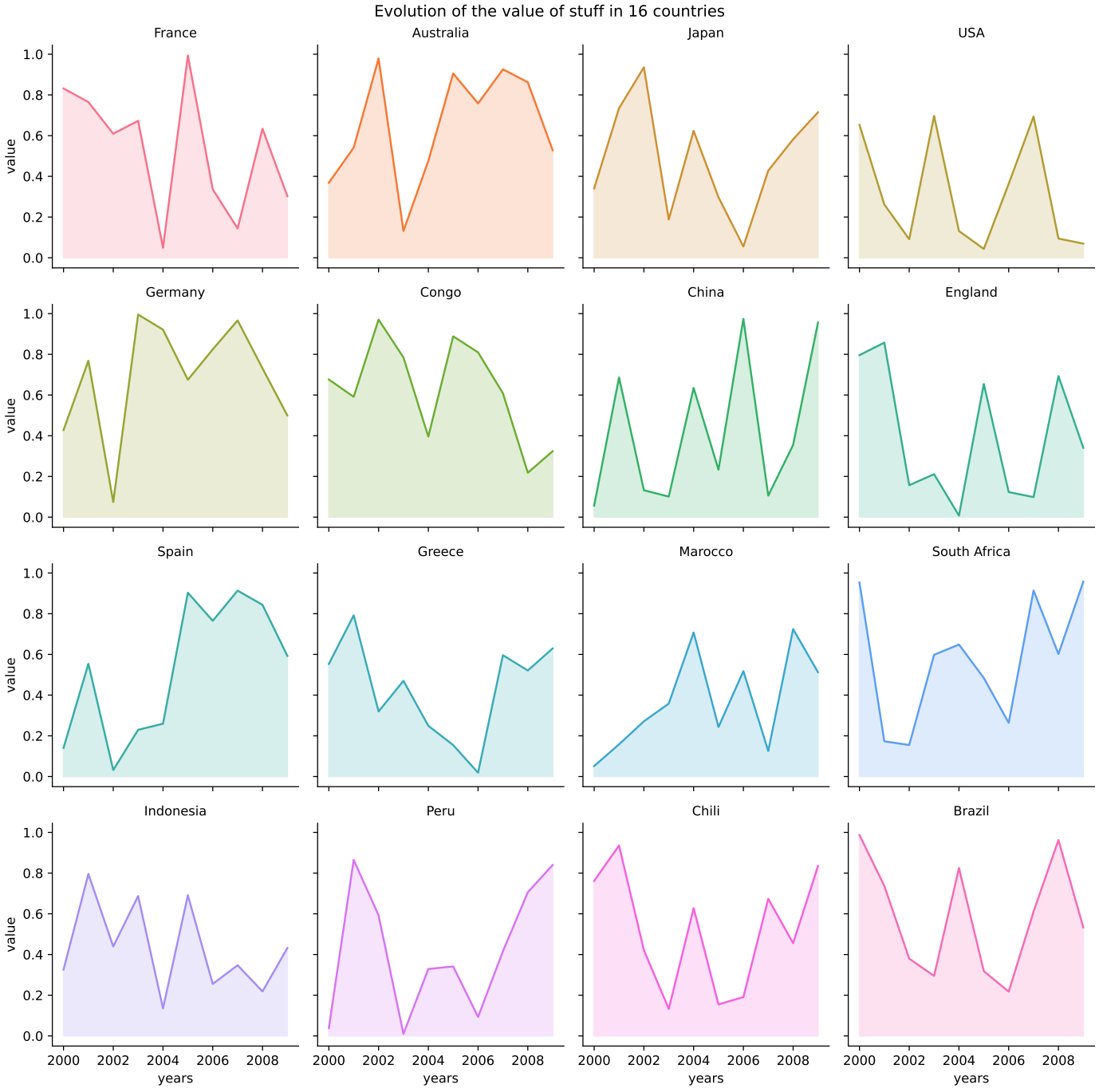
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03c_Steamgraph3.py, and the XML code in the file D03c_Steamgraph3.2D.xml.

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03d_Steamgraph4.py, and the XML code in the file D03d_Steamgraph4.2D.xml.



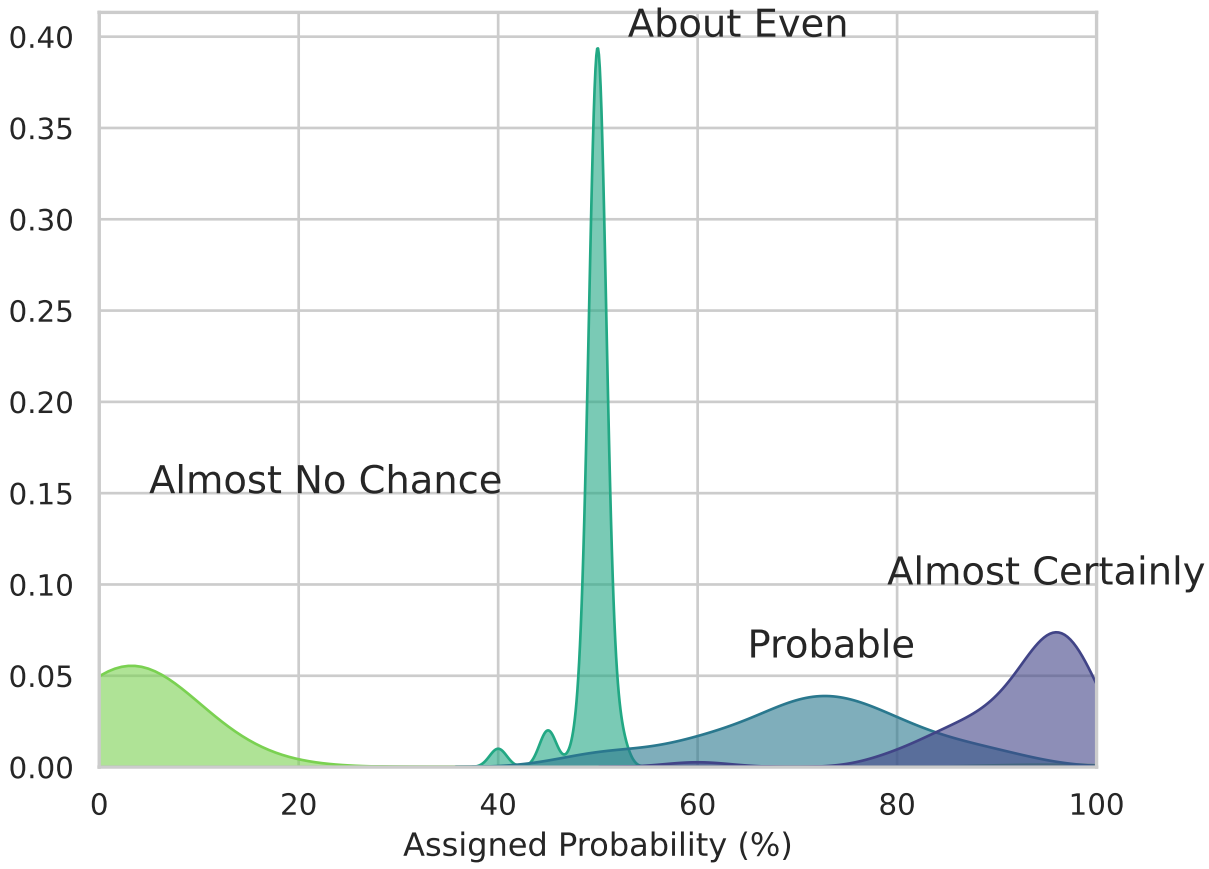
17.3.4 Area Chart

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_MultipleAreaCharts.py, and the XML code in the file D04_MultipleAreaCharts.2D.xml.



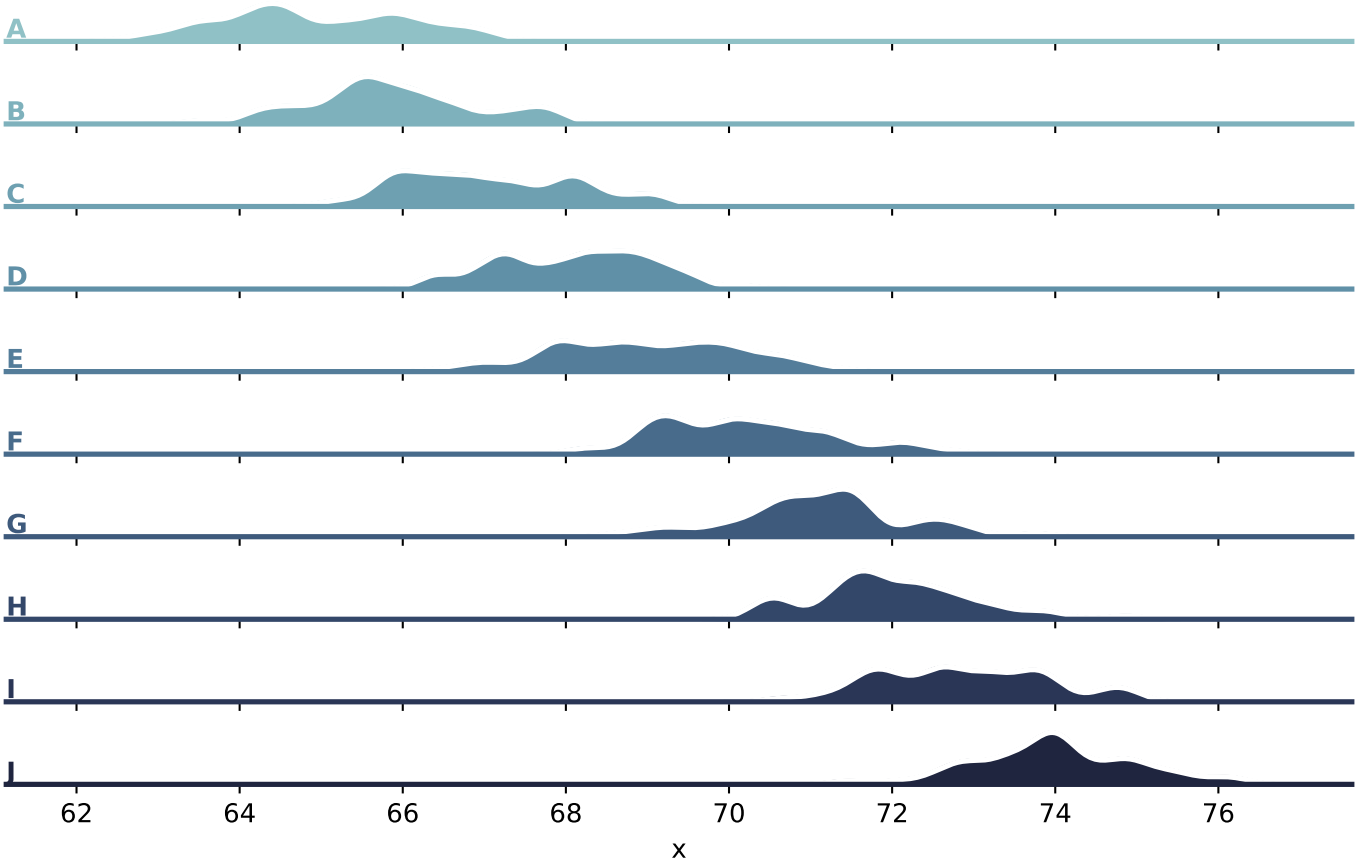
17.3.5 Density Chart

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_DensityChart.py, and the XML code in the file D05_DensityChart.2D.xml.



17.3.6 Ridgetline Plot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_RidgetlinePlot.py, and the XML code in the file D06_RidgetlinePlot.2D.xml.

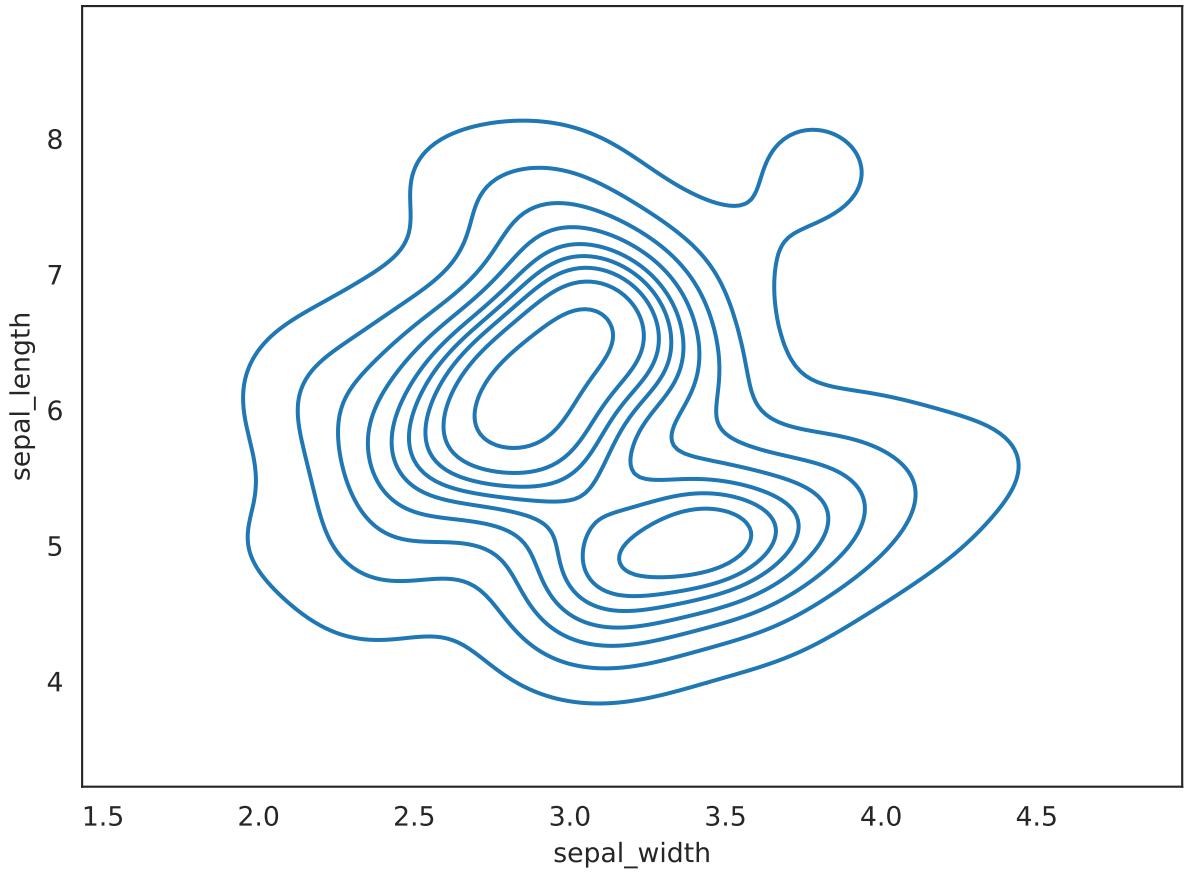


17.3.7 Kde Plot, 3 versions

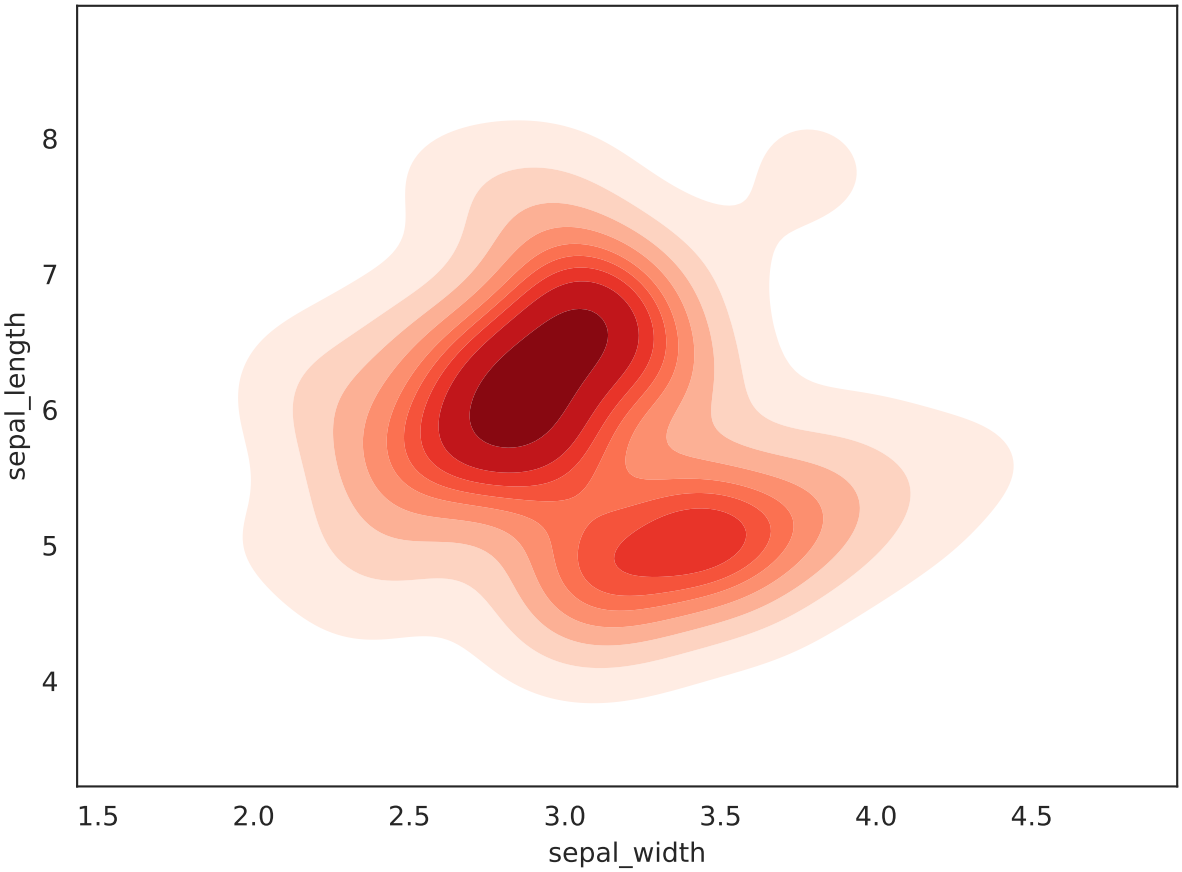
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_KdePlot1.py, and the XML code in the file D07a_KdePlot1.2D.xml.

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_KdePlot2.py, and the XML code in the file D07b_KdePlot2.2D.xml.

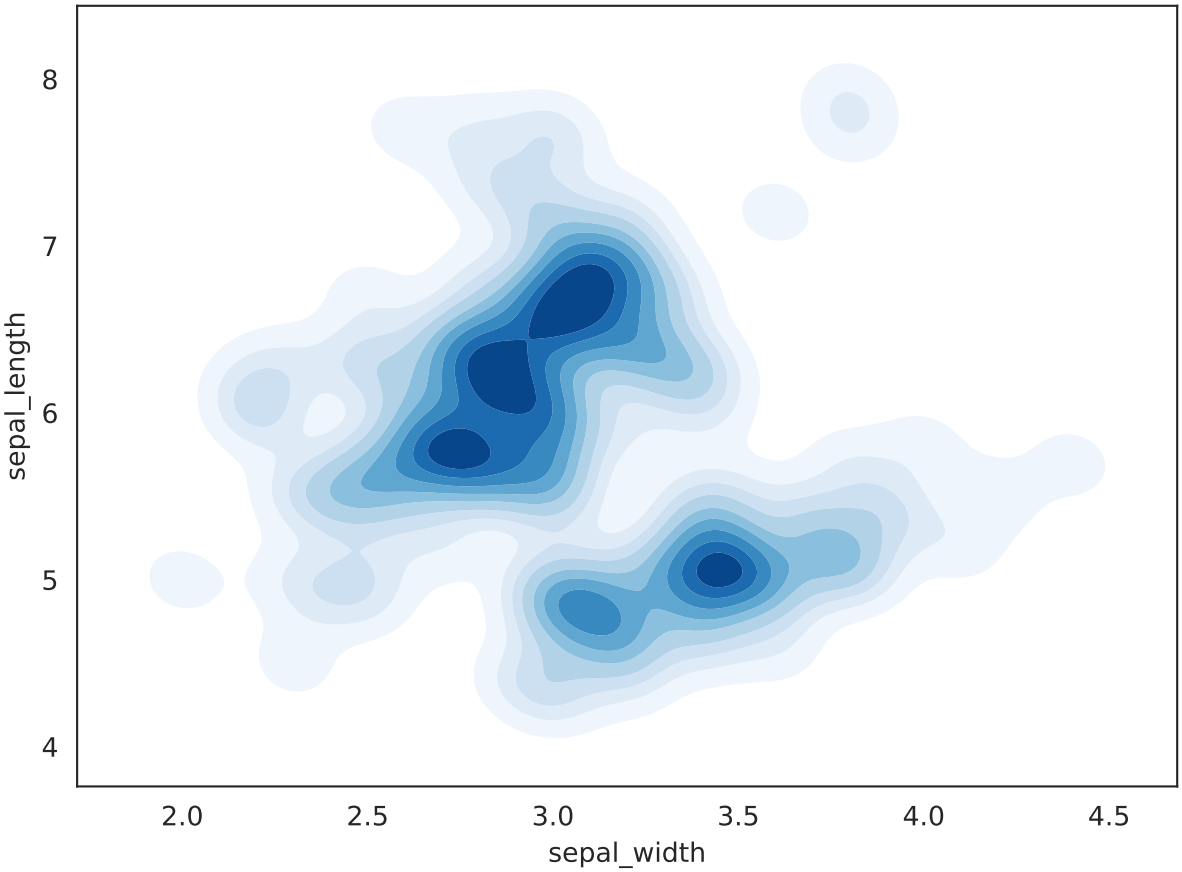
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07c_KdePlot3.py, and the XML code in the file D07c_KdePlot3.2D.xml.



Some text



Some text

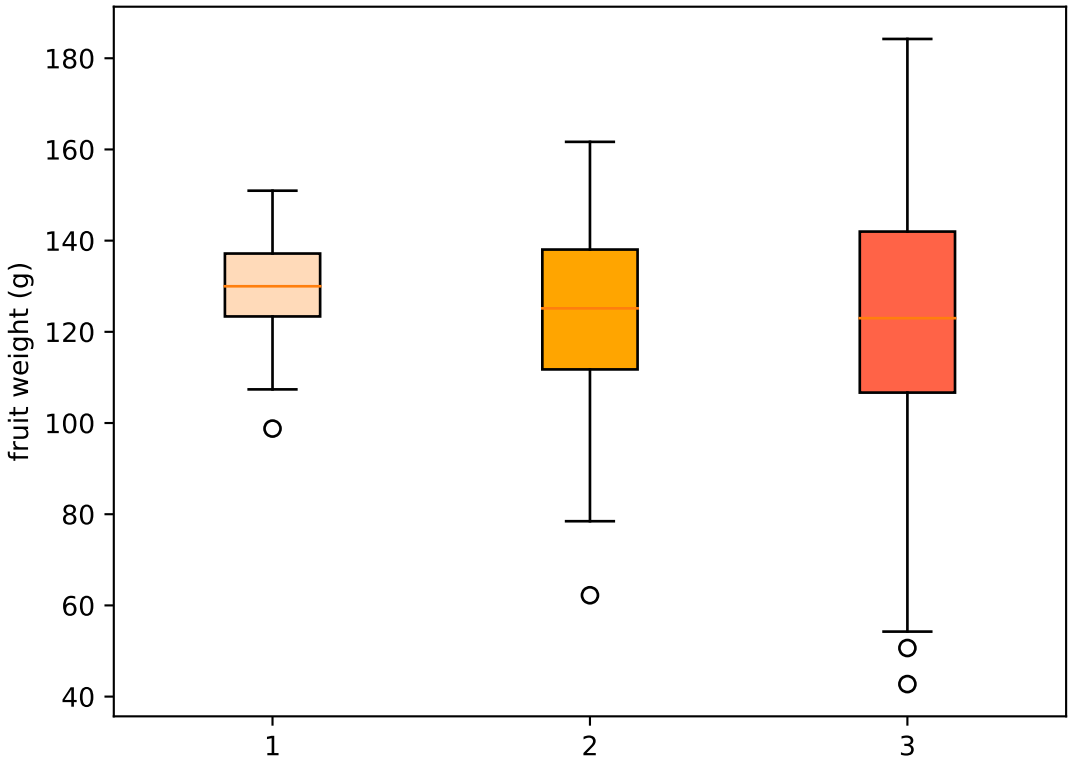


Some text

17.4 Boxplot, Violinplot and Raincloud plot

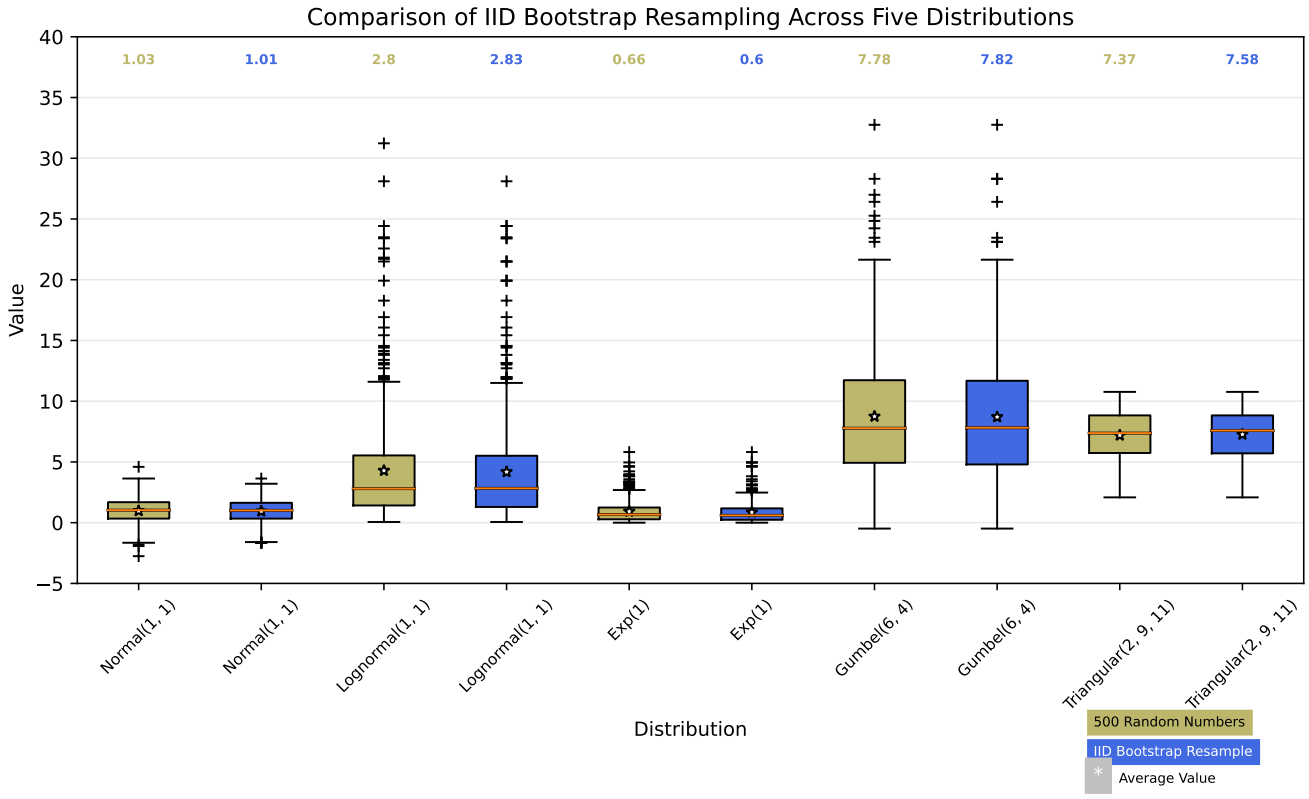
17.4.1 Boxplot, simple

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_BoxplotSimple.py, and the XML code in the file D01_BoxplotSimple.2D.xml.



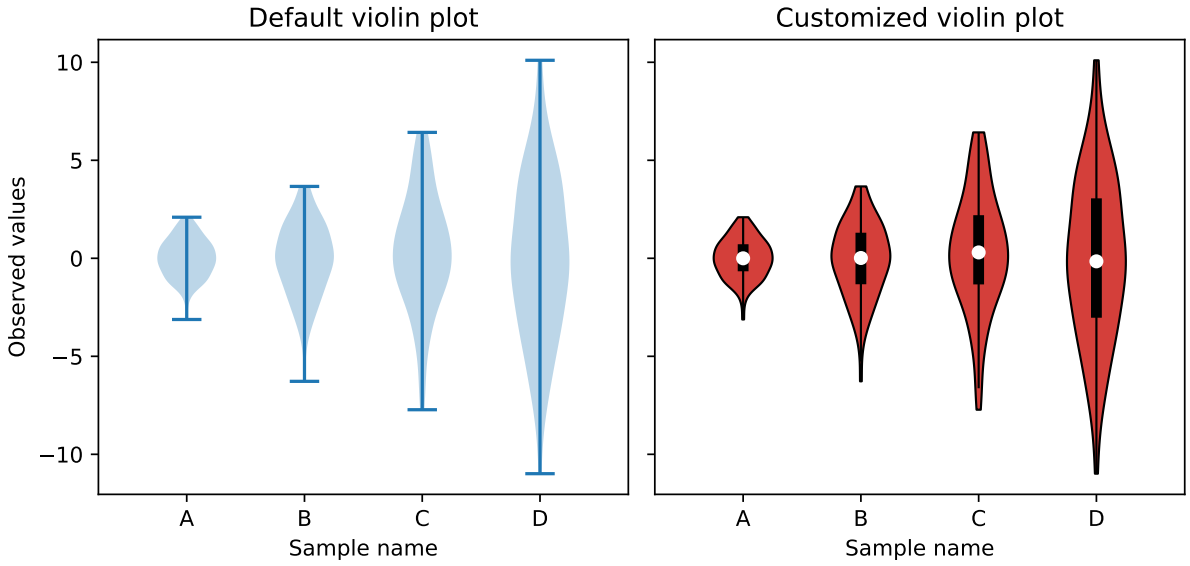
17.4.2 Boxplot, multiple groups

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_BoxplotMultipleGroups.py, and the XML code in the file D02_BoxplotMultipleGroups.2D.xml.



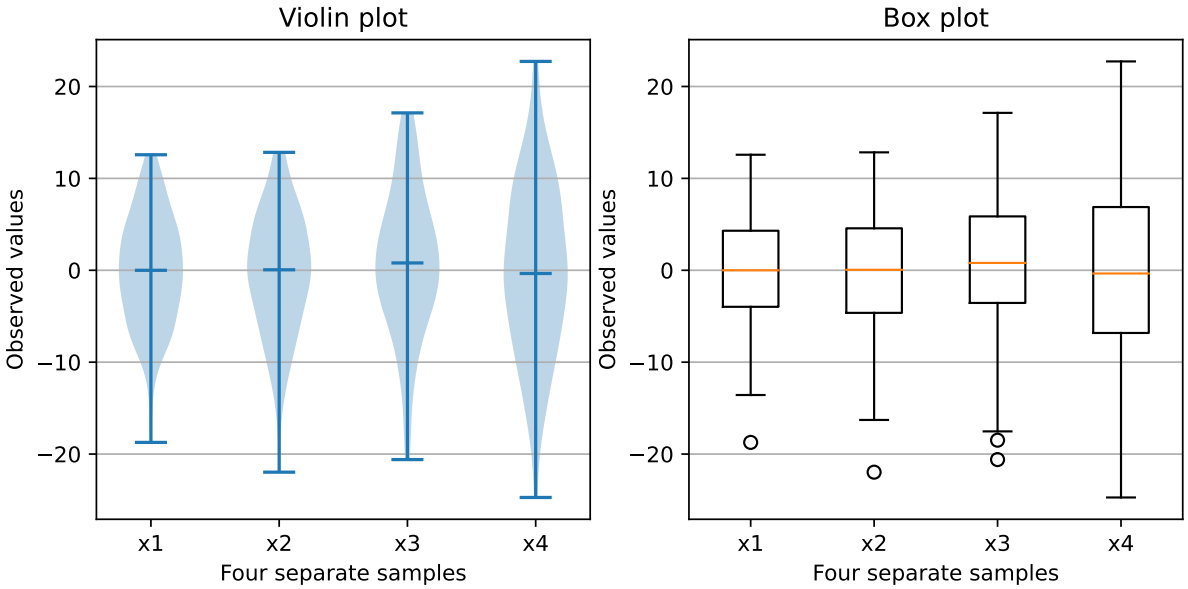
17.4.3 ViolinPlot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_ViolinPlot.py, and the XML code in the file D03_ViolinPlot.2D.xml.



17.4.4 Boxplot vs violinplot

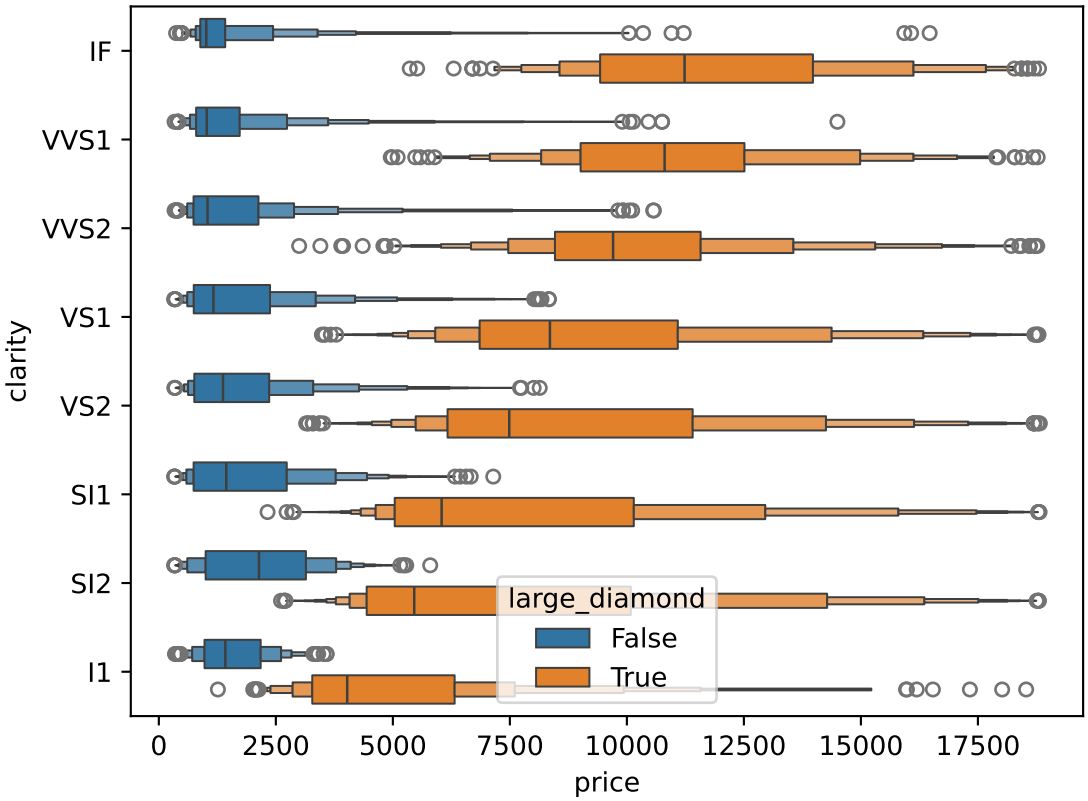
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_BoxplotVsViolinplot.py, and the XML code in the file D04_BoxplotVsViolinplot.2D.xml.



Some text

17.4.5 Boxenplot

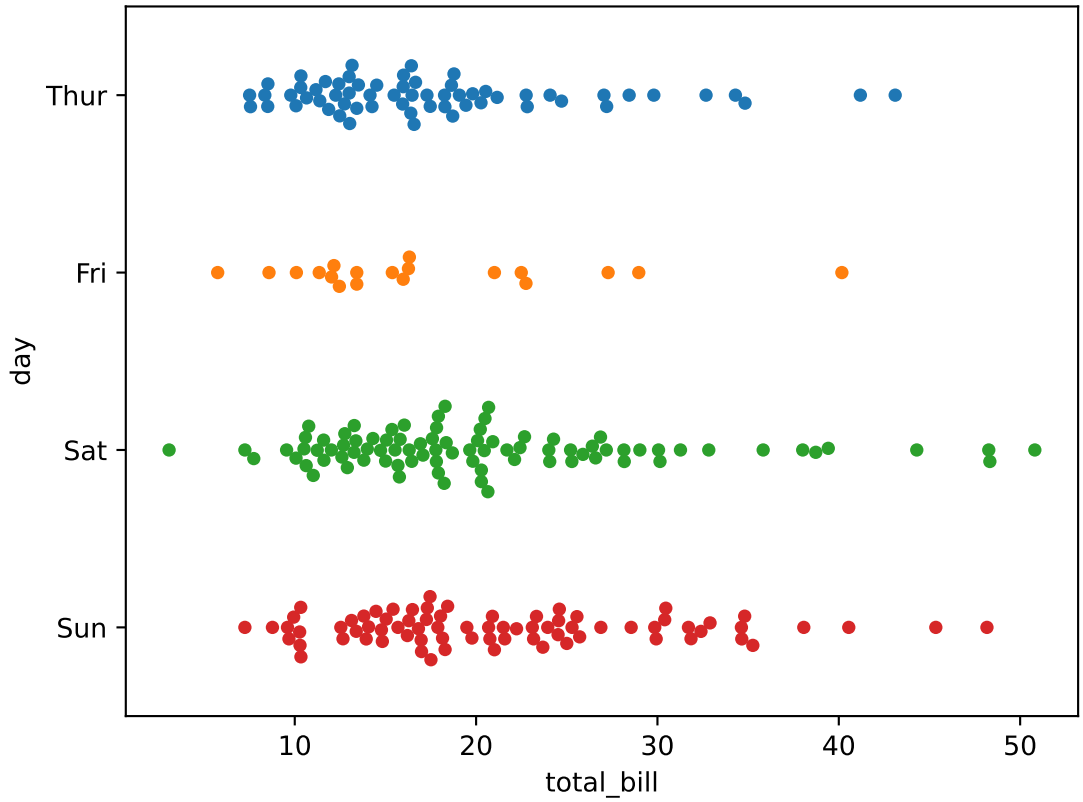
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Boxenplot.py, and the XML code in the file D05_Boxenplot.2D.xml.



Some text

17.4.6 Swarmplot

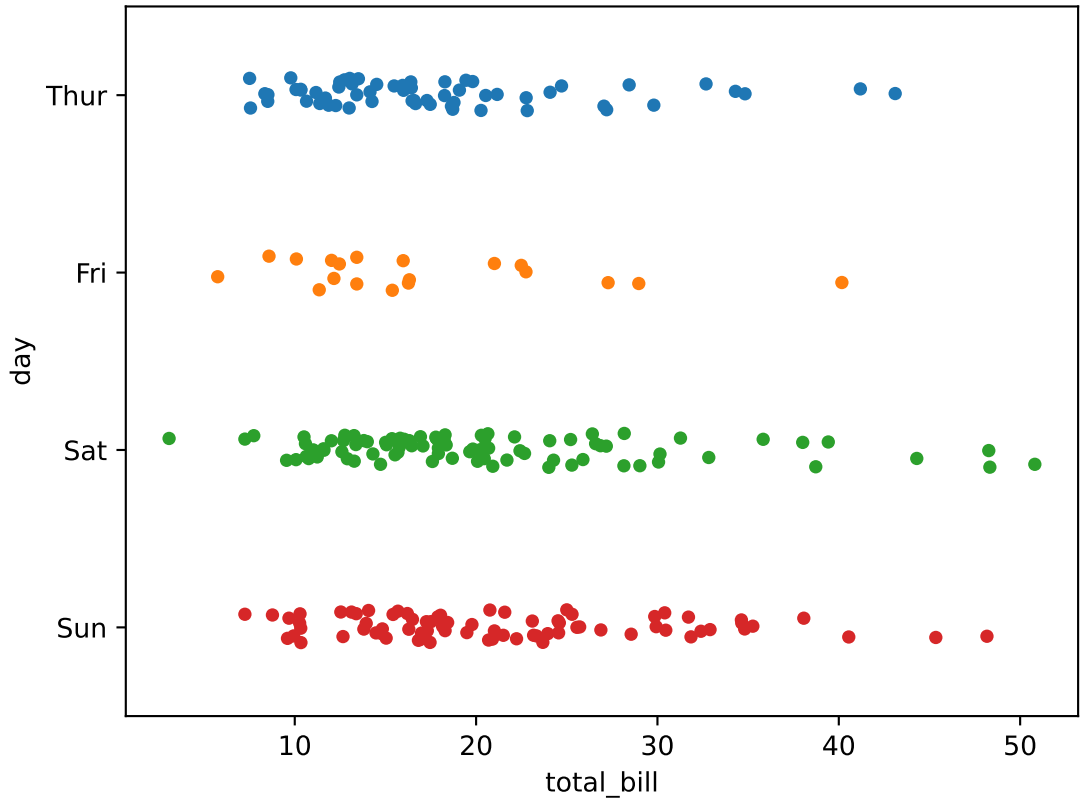
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Swarmplot.py, and the XML code in the file D06_Swarmplot.2D.xml.



Some text

17.4.7 Stripplot

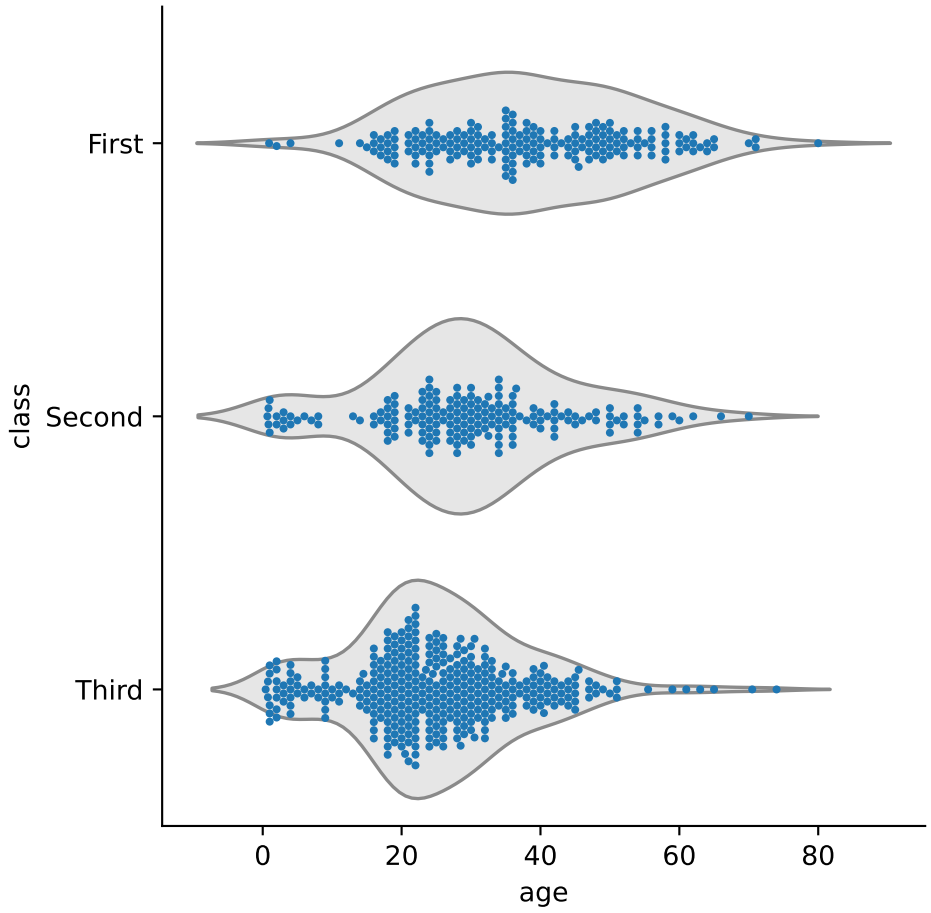
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_Stripplot.py, and the XML code in the file D07_Stripplot.2D.xml.



Some text

17.4.8 Catplot

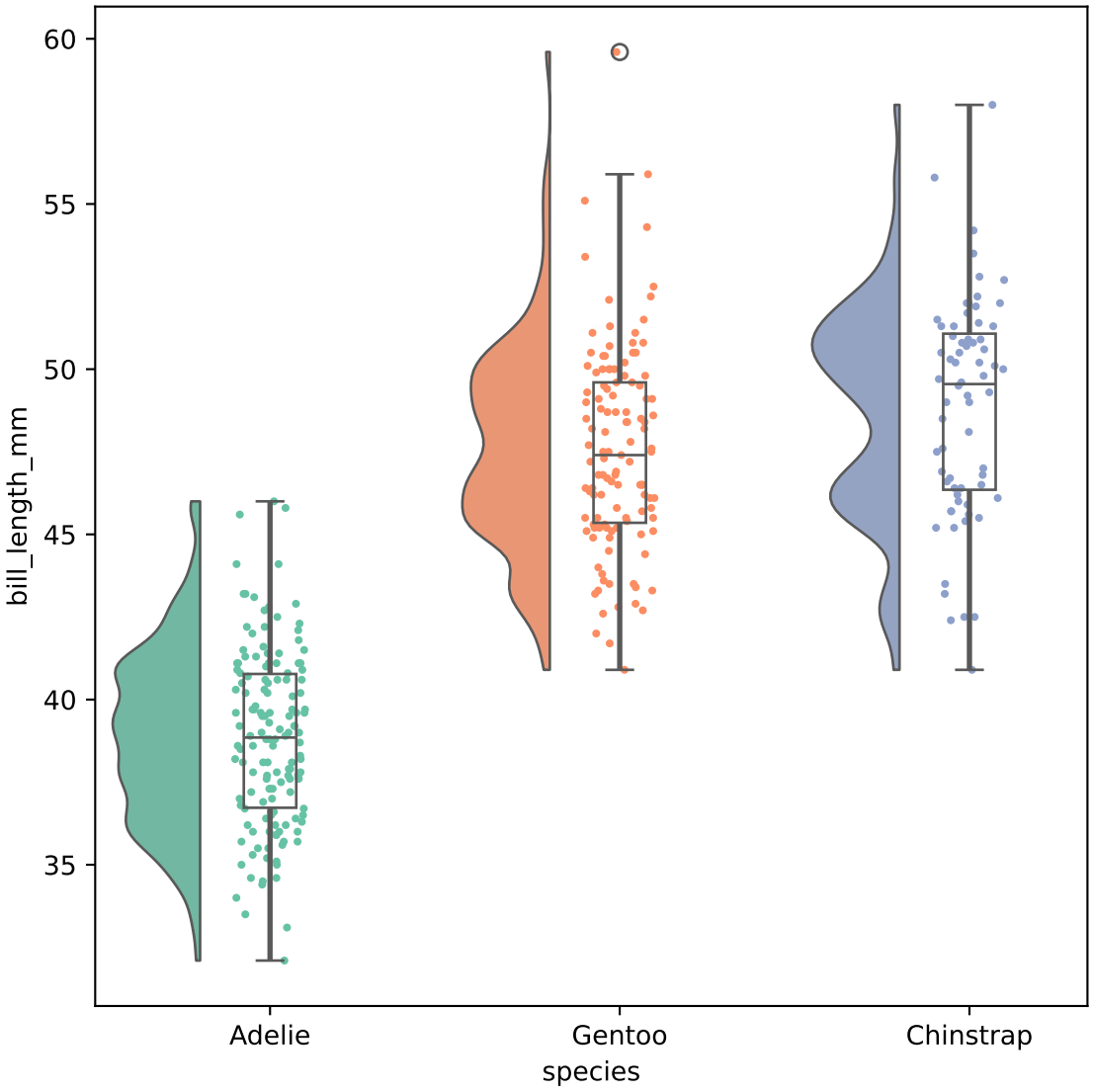
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_Catplot.py, and the XML code in the file D08_Catplot.2D.xml.



Some text

17.4.9 Raincloud Plot

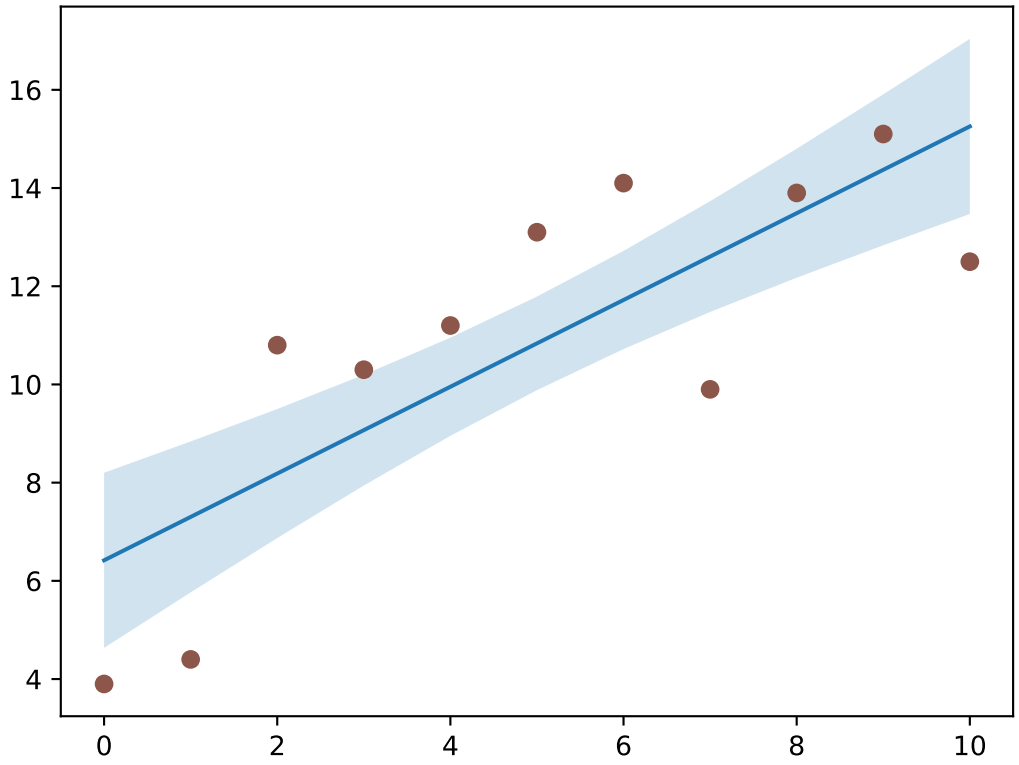
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_RaincloudPlot.py, and the XML code in the file D09_RaincloudPlot.2D.xml.



17.5 Correlation and regression

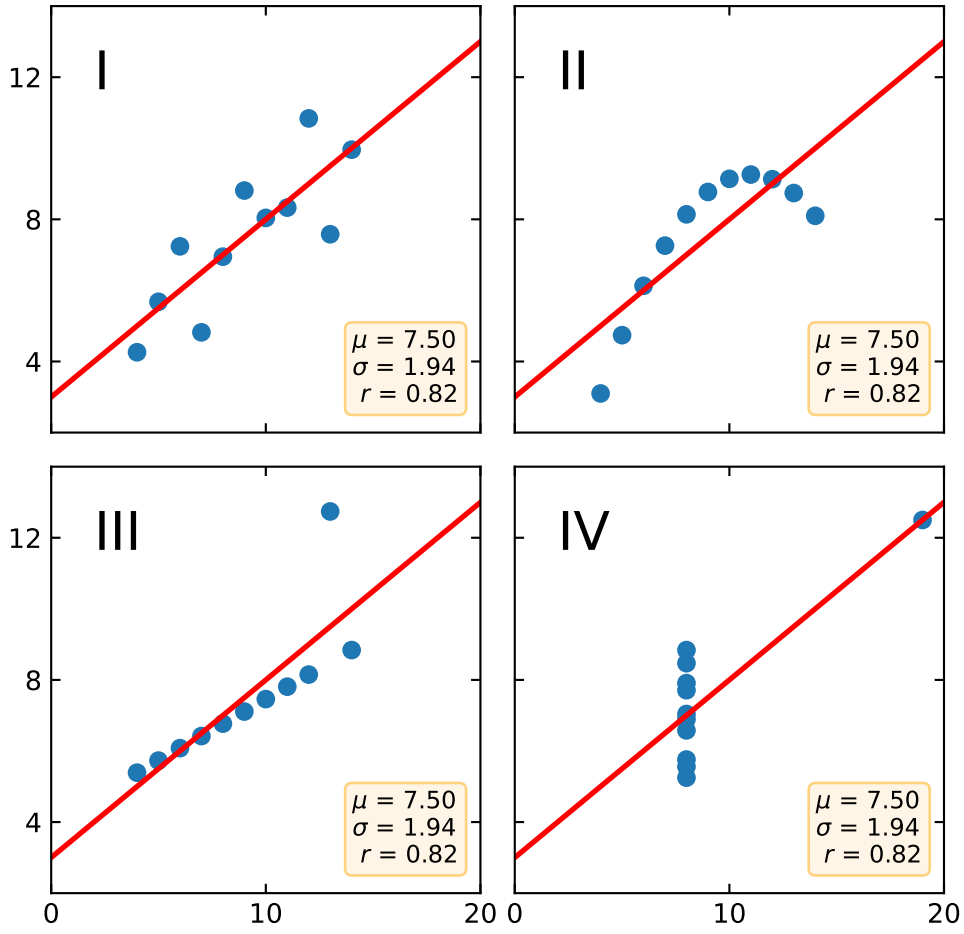
17.5.1 Linear regression with confidence Band

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_LinRegWithConfidenceBand.py, and the XML code in the file D01_LinRegWithConfidenceBand.2D.xml.



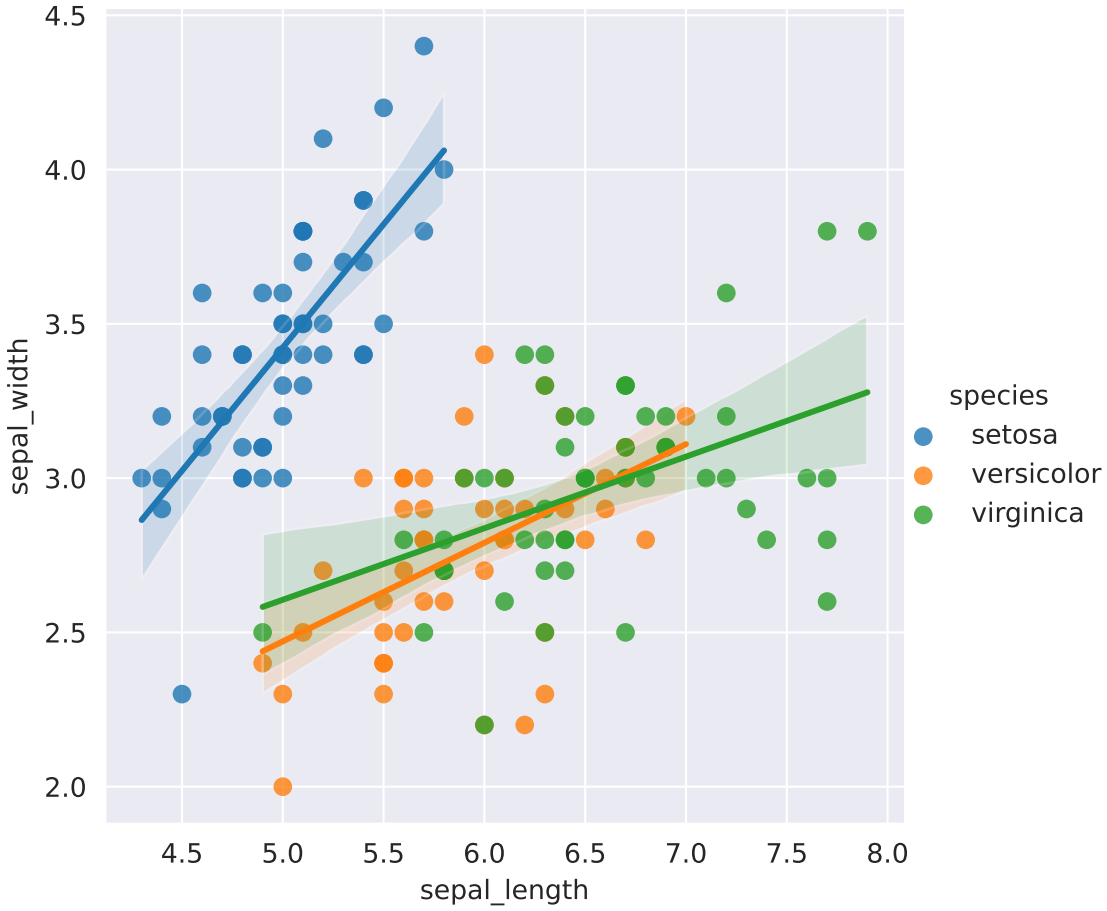
17.5.2 Anscombe example

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_AnscombeExample.py, and the XML code in the file D02_AnscombeExample.2D.xml.



17.5.3 Faceted Scatter Plots: all in one plot

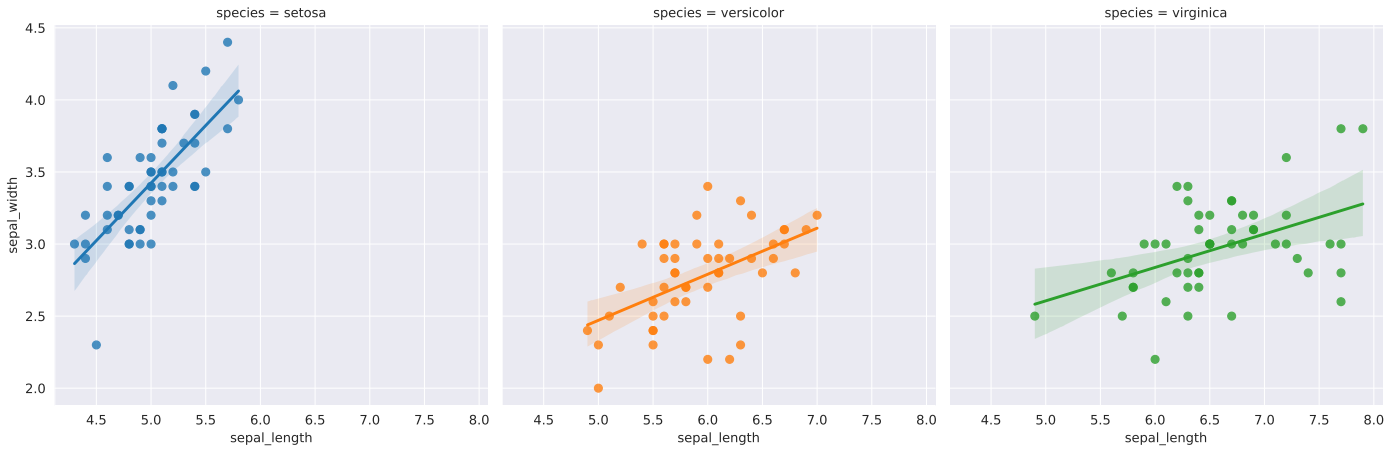
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_FacetedScatterPlot1.py, and the XML code in the file D03_FacetedScatterPlot1.2D.xml.



Some text

17.5.4 Faceted Scatter Plots: separate plots

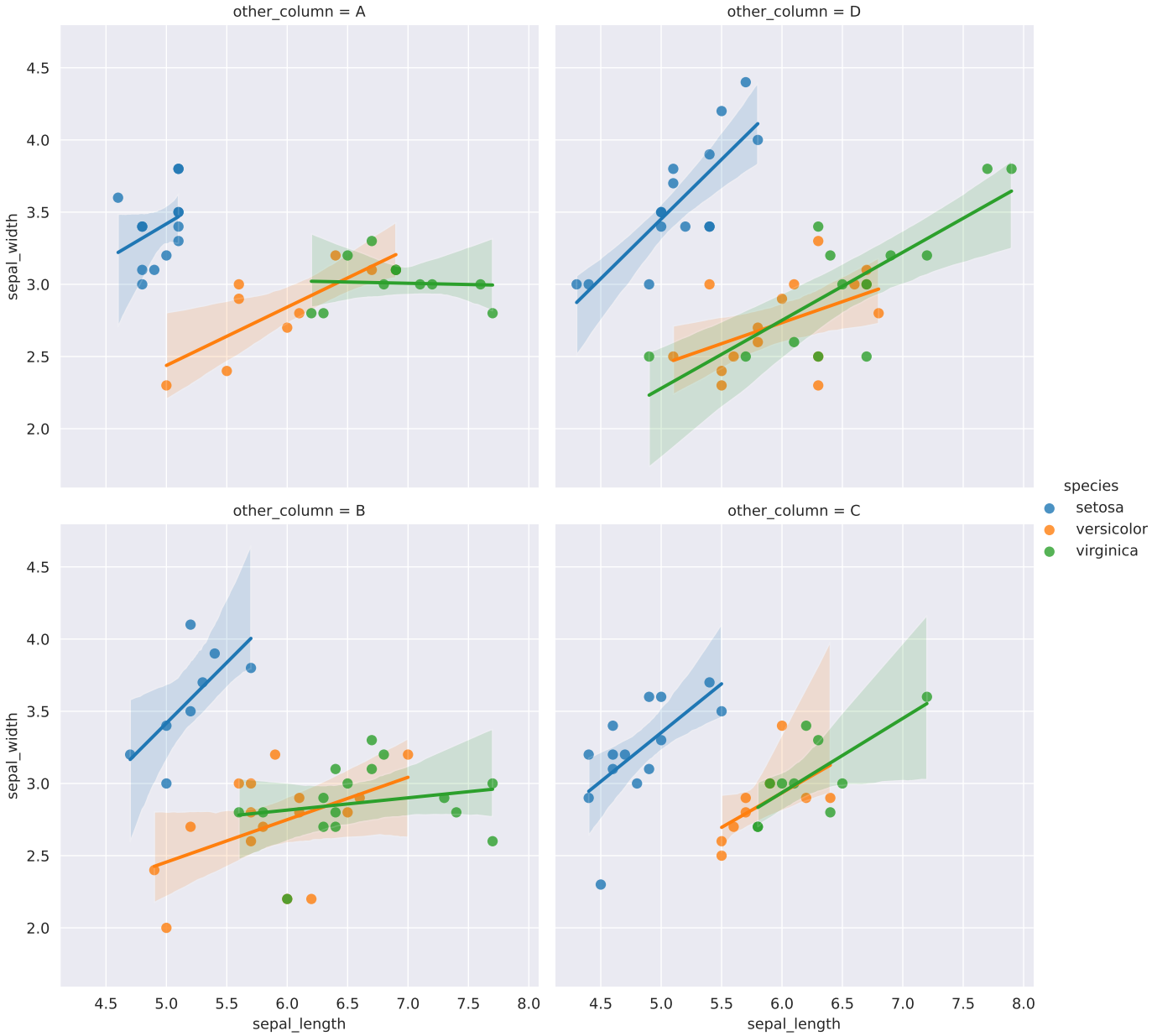
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_FacetedScatterPlot2.py, and the XML code in the file D04_FacetedScatterPlot2.2D.xml.



Some text

17.5.5 Faceted Scatter Plots: further subgrouping

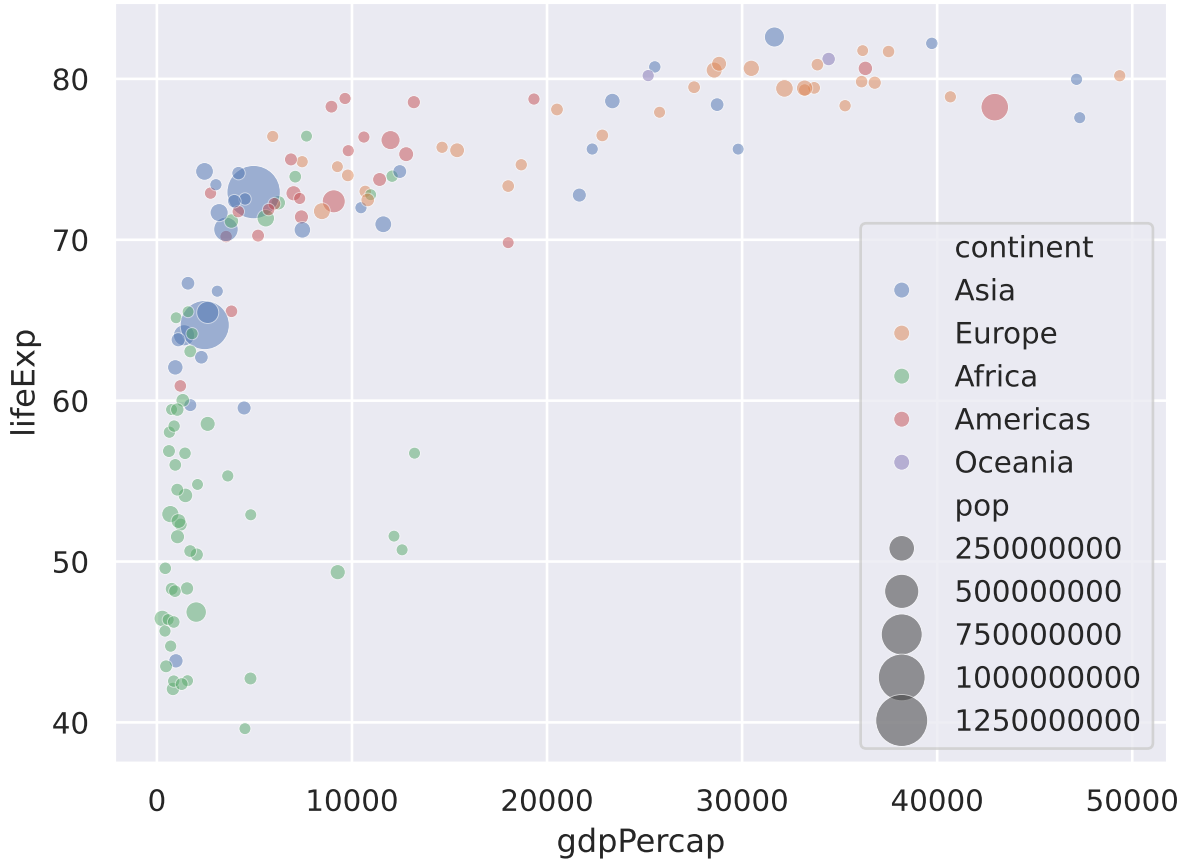
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_FacetedScatterPlot3.py, and the XML code in the file D05_FacetedScatterPlot3.2D.xml.



Some text

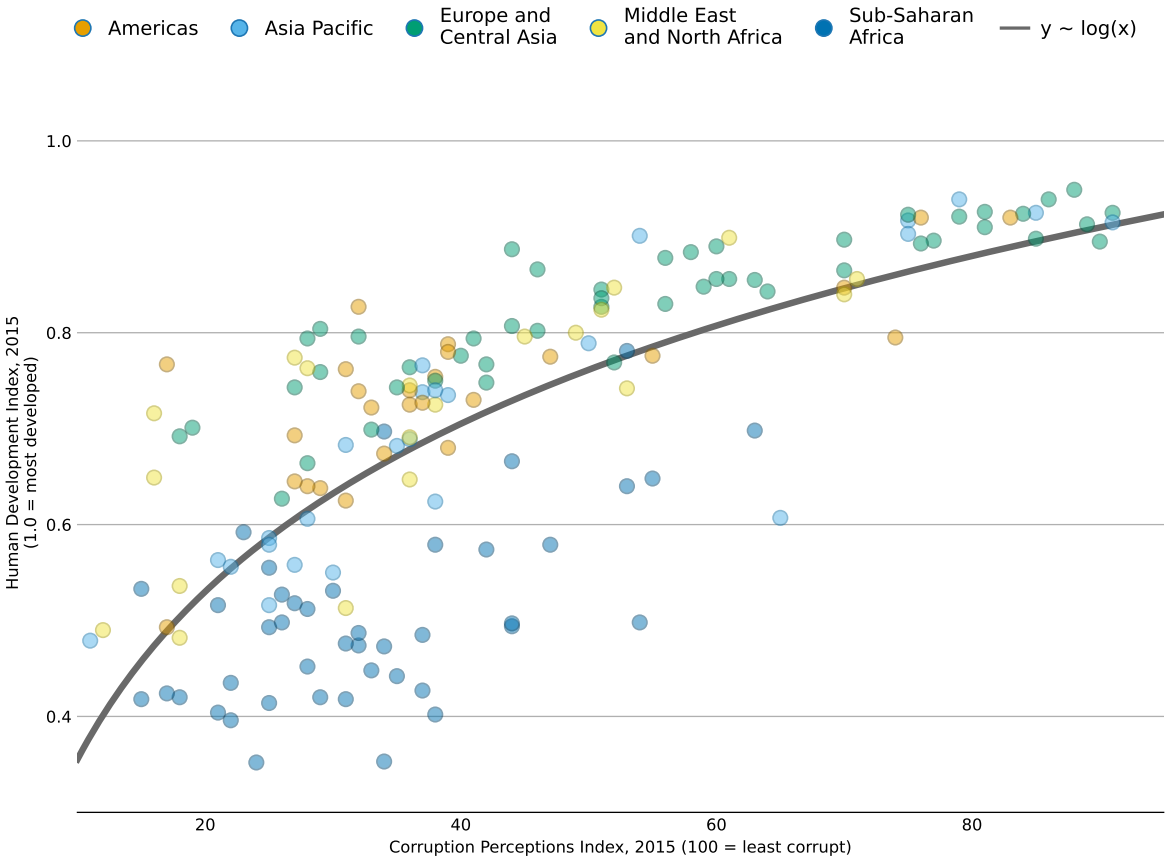
17.5.6 Bubbleplot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Bubbleplot.py, and the XML code in the file D06_Bubbleplot.2D.xml.



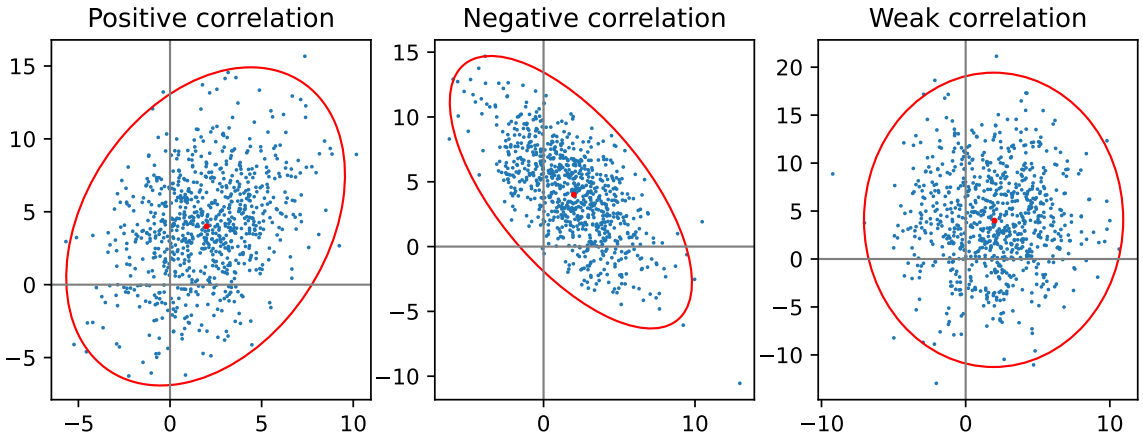
17.5.7 Scatterplot with fancy formatting

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_ScatterplotWithFancyFomatting.py, and the XML code in the file D07_ScatterplotWithFancyFomatting.2D.xml.



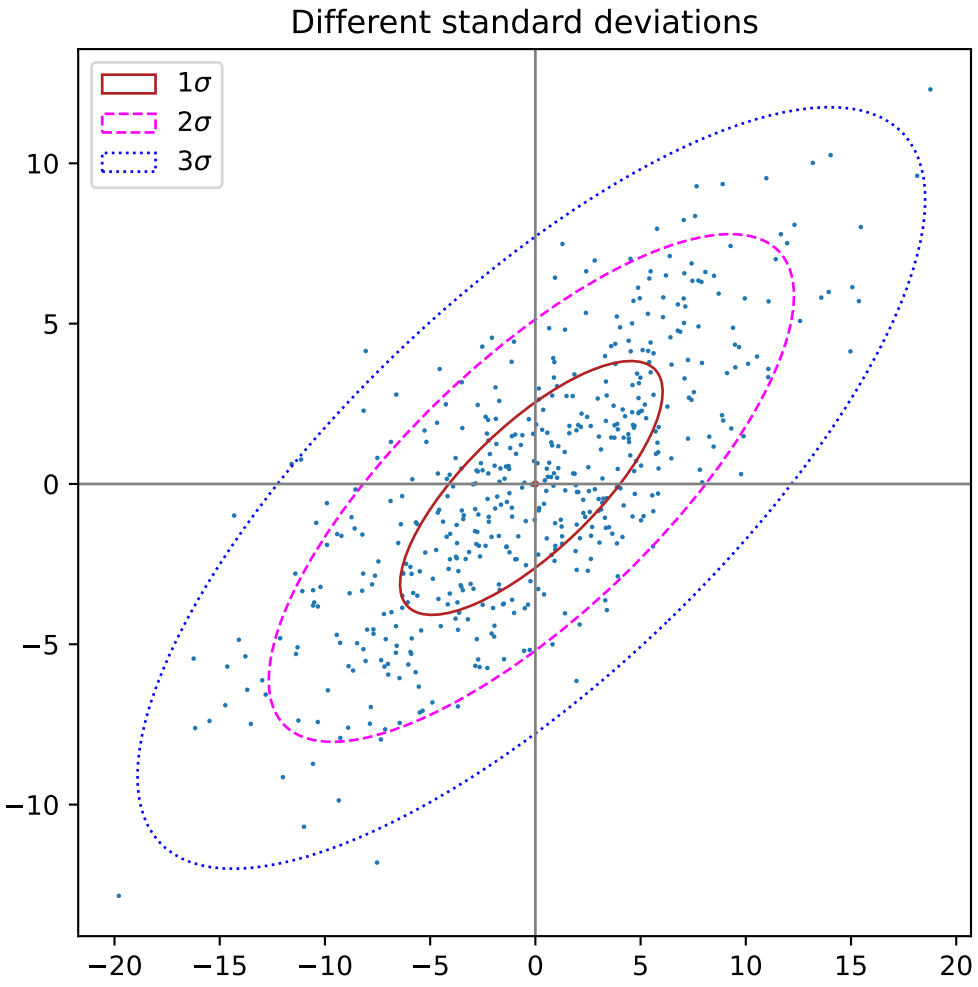
17.5.8 Scatterplots with positive, negative and weak correlation: confidence ellipses

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_ConfidenceEllipse.py, and the XML code in the file D08_ConfidenceEllipse.2D.xml.



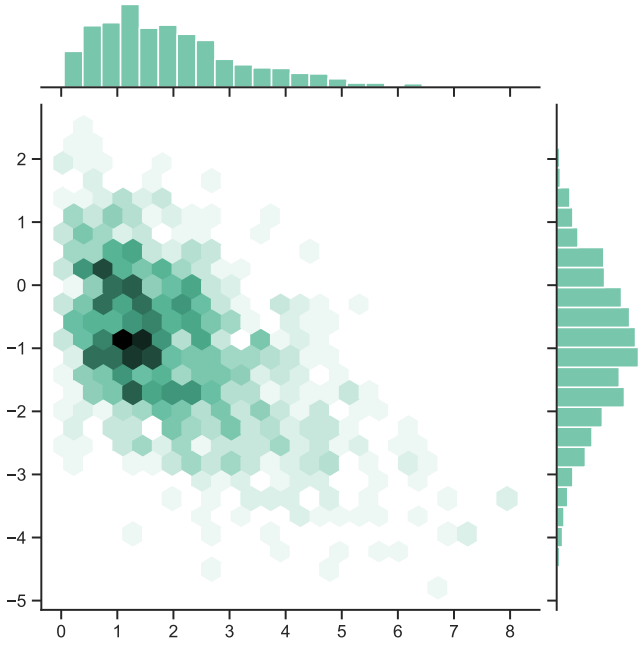
17.5.9 Scatterplot with confidence confidence ellipses at different standard deviations

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_ConfidenceEllipse2.py, and the XML code in the file D09_ConfidenceEllipse2.2D.xml.



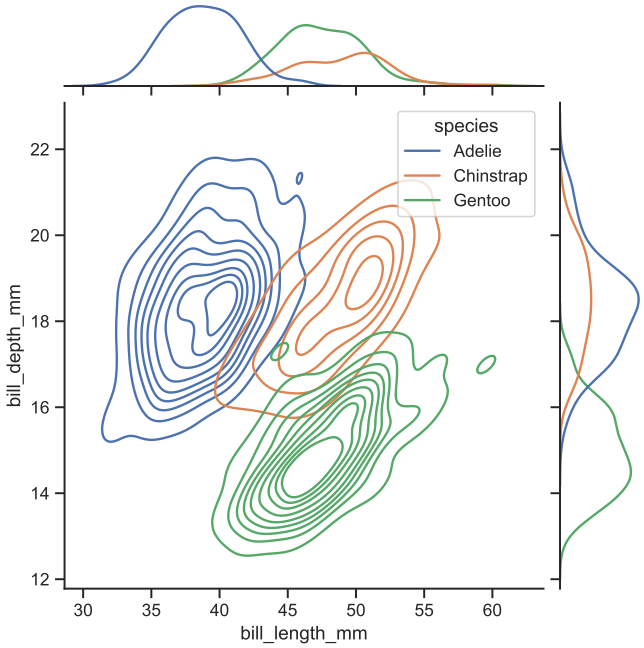
17.5.10 Hexagonal binned plot and marginal histograms (Seaborn)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_Hexagonalbinnedplot.py, and the XML code in the file D10_Hexagonalbinnedplot.2D.xml.



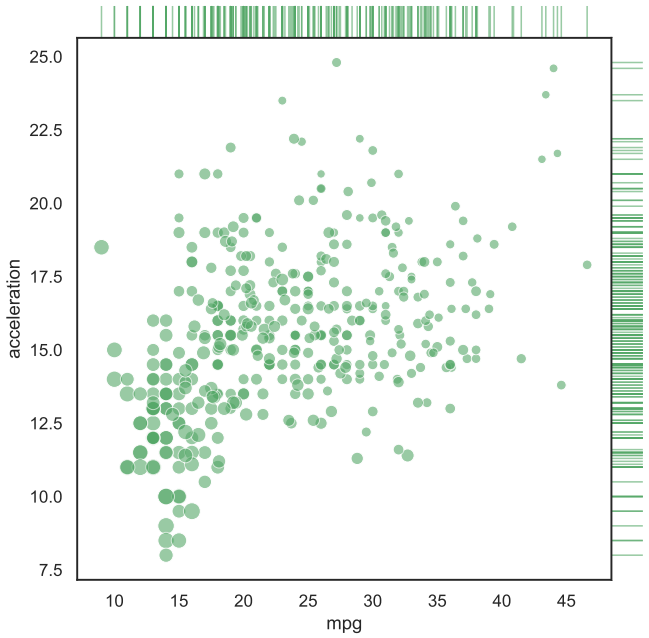
17.5.11 Joint and marginal Kde (Seaborn)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11_JointAndMarginalKde.py, and the XML code in the file D11_JointAndMarginalKde.2D.xml.



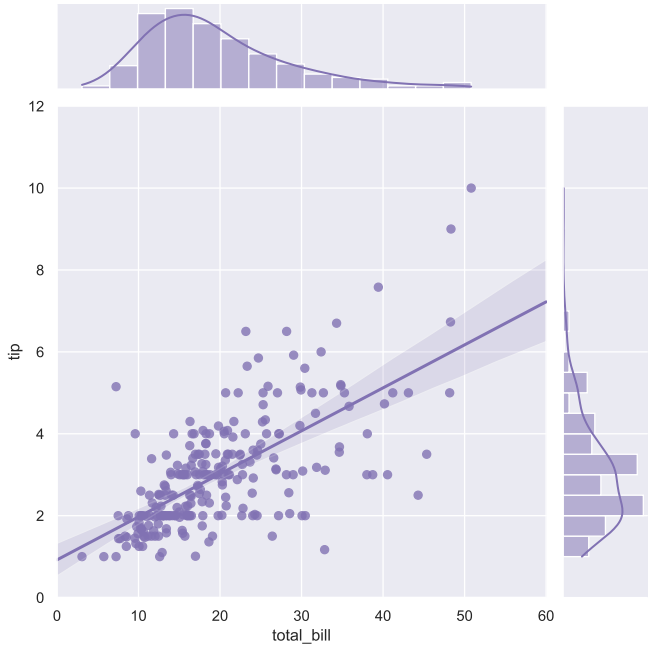
17.5.12 Marginal ticks

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D12_MarginalTicks.py, and the XML code in the file D12_MarginalTicks.2D.xml.



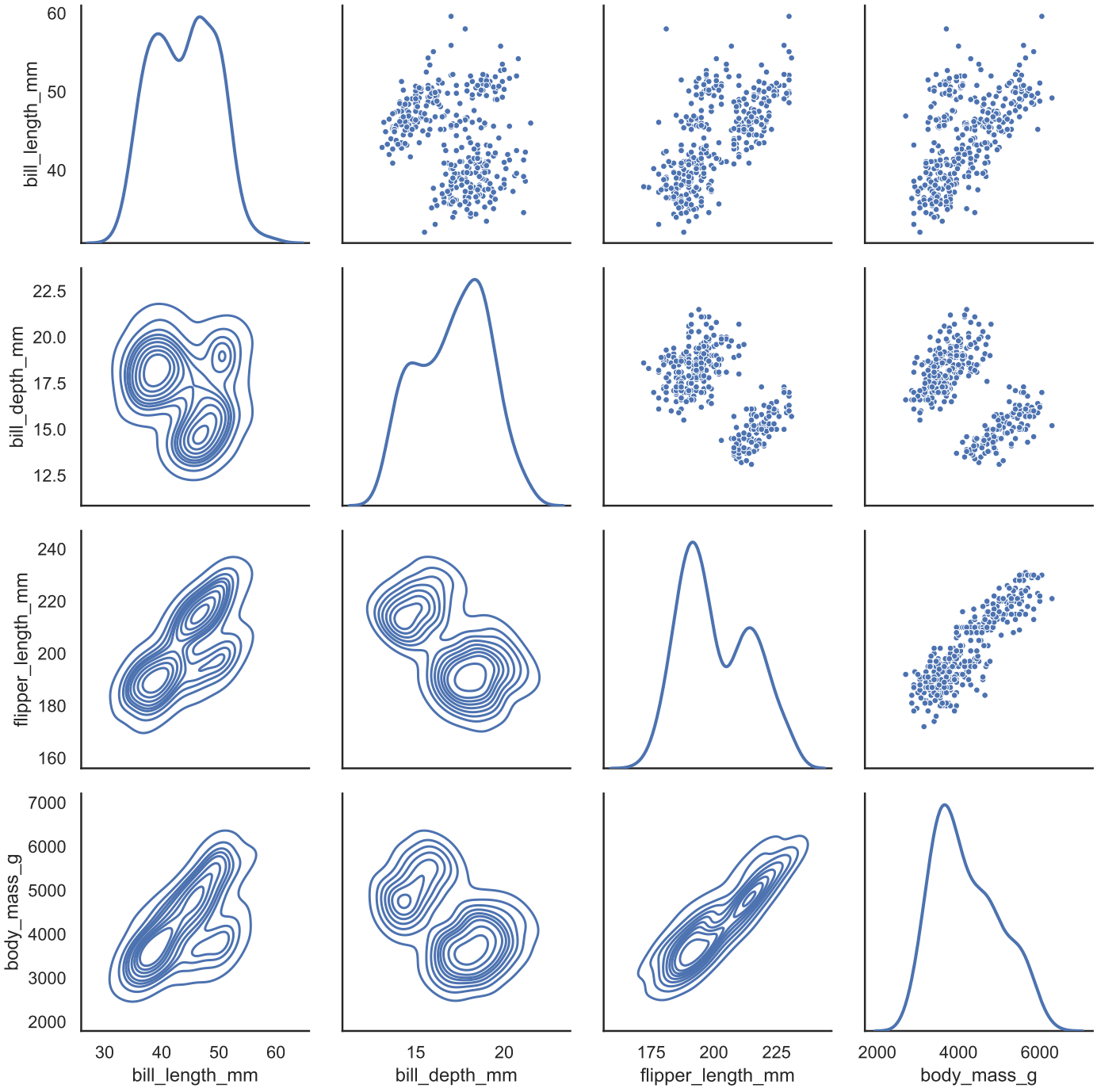
17.5.13 Regression_marginals (Seaborn)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D13_RegressionMarginals.py, and the XML code in the file D13_RegressionMarginals.2D.xml.



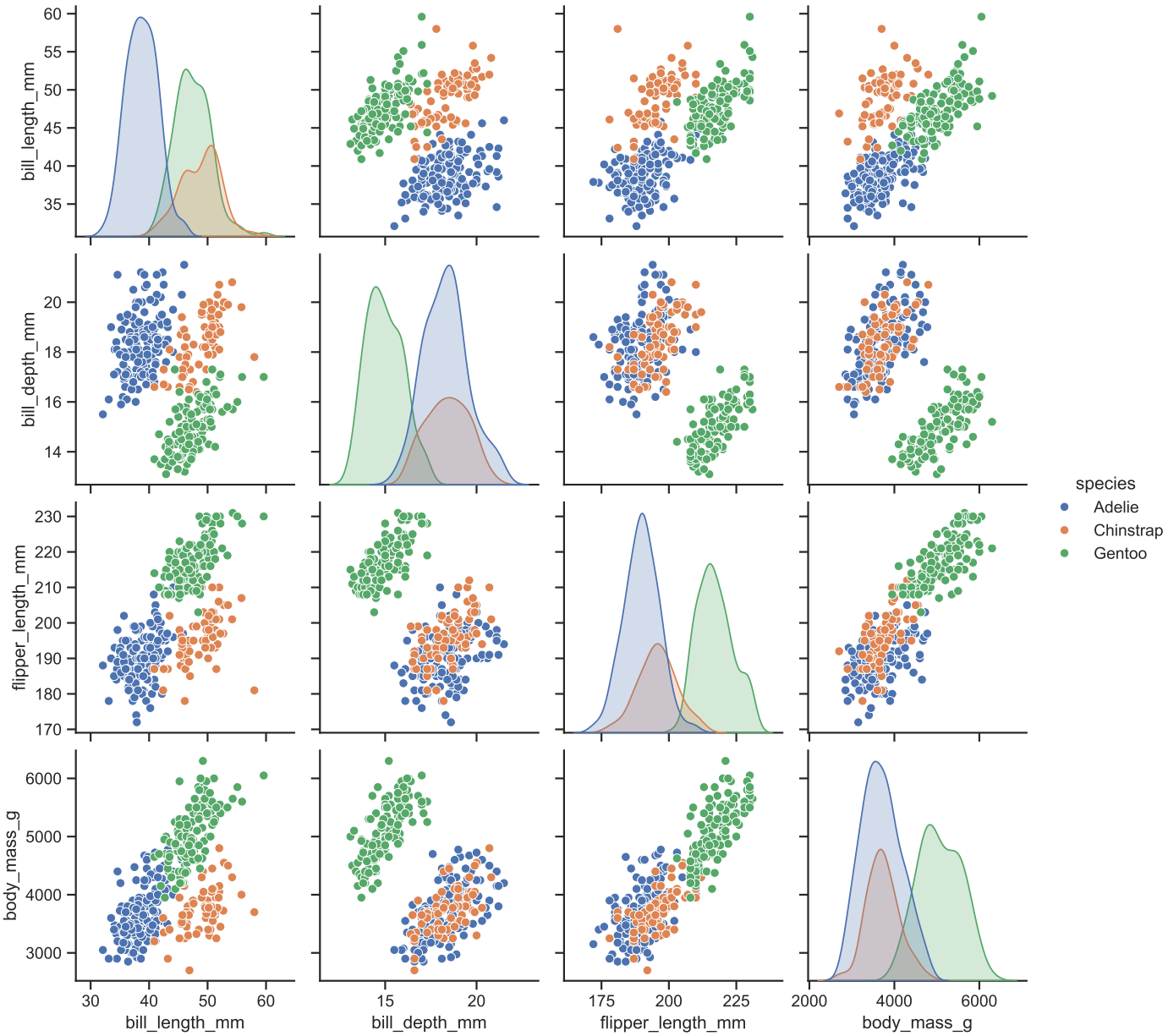
17.5.14 Pair grid with kde (Seaborn)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D14_PairGridWithKde.py, and the XML code in the file D14_PairGridWithKde.2D.xml.



17.5.15 Scatterplot matrix (Seaborn)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D15_ScatterplotMatrix.py, and the XML code in the file D15_ScatterplotMatrix.2D.xml.



17.6 Financial plots (requires mplfinance)

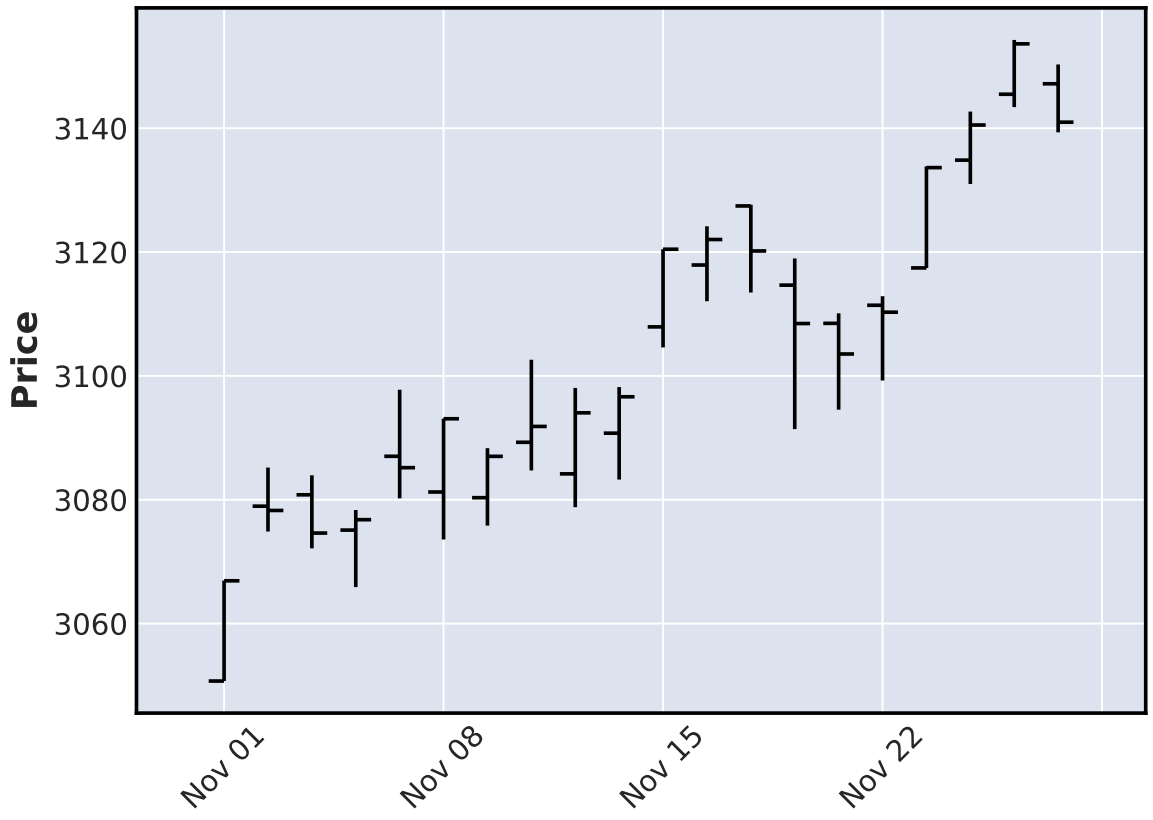
See also: https://pypi.org/project/mplfinance/

See also: https://github.com/matplotlib/mplfinance

See also for styling : https://coderzcolumn.com/tutorials/data-science/candlestick-chart-in-python-mplfinance-plotly-bokeh

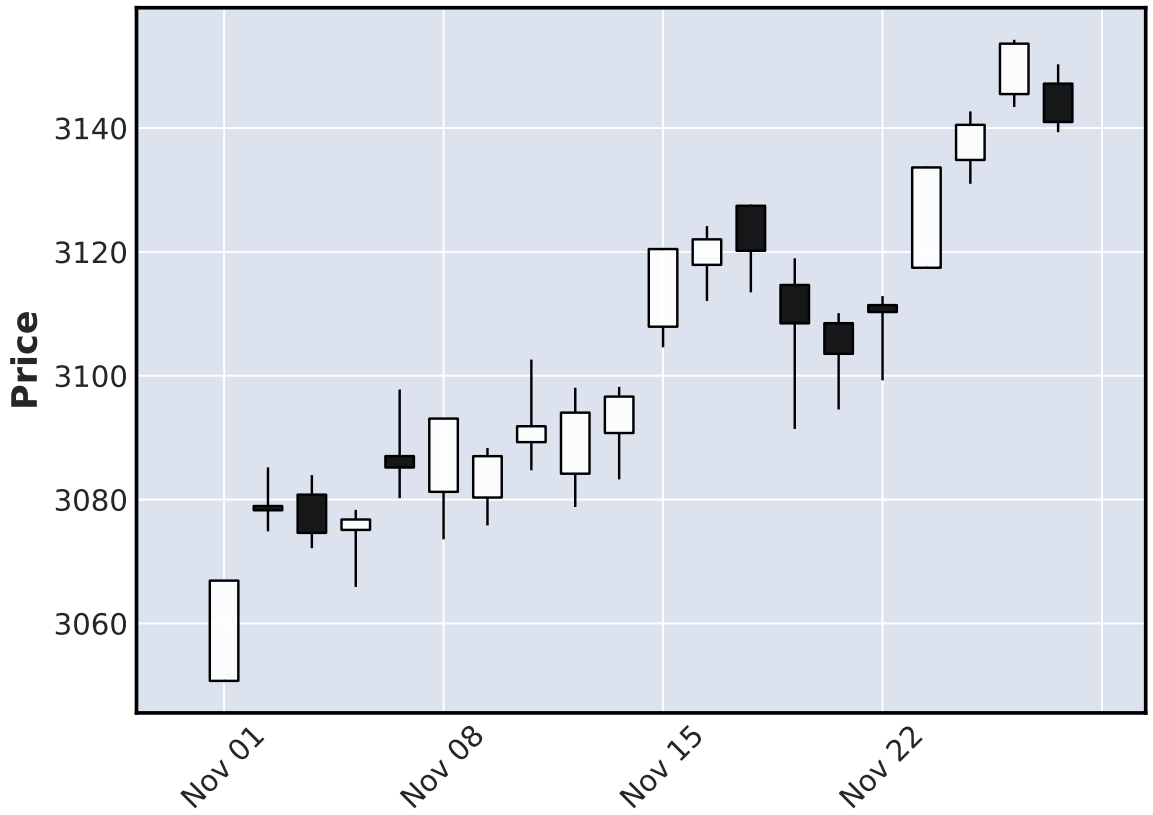
17.6.1 HiLoPlot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Mpl_HiLo.py, and the XML code in the file D01_Mpl_HiLo.2D.xml.



17.6.2 CandlePlot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Mpl_Candle.py, and the XML code in the file D02_Mpl_Candle.2D.xml.

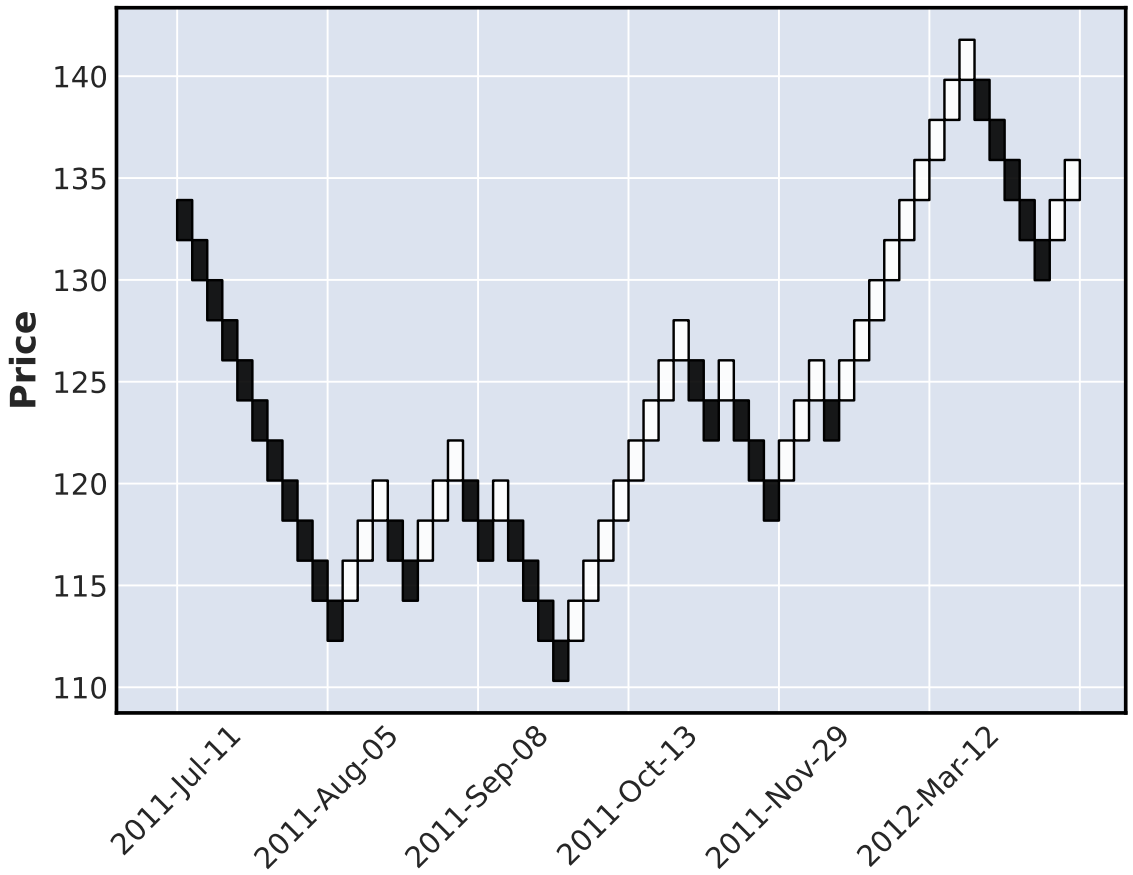


17.6.3 LinePlot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Mpl_Line.py, and the XML code in the file D03_Mpl_Line.2D.xml.

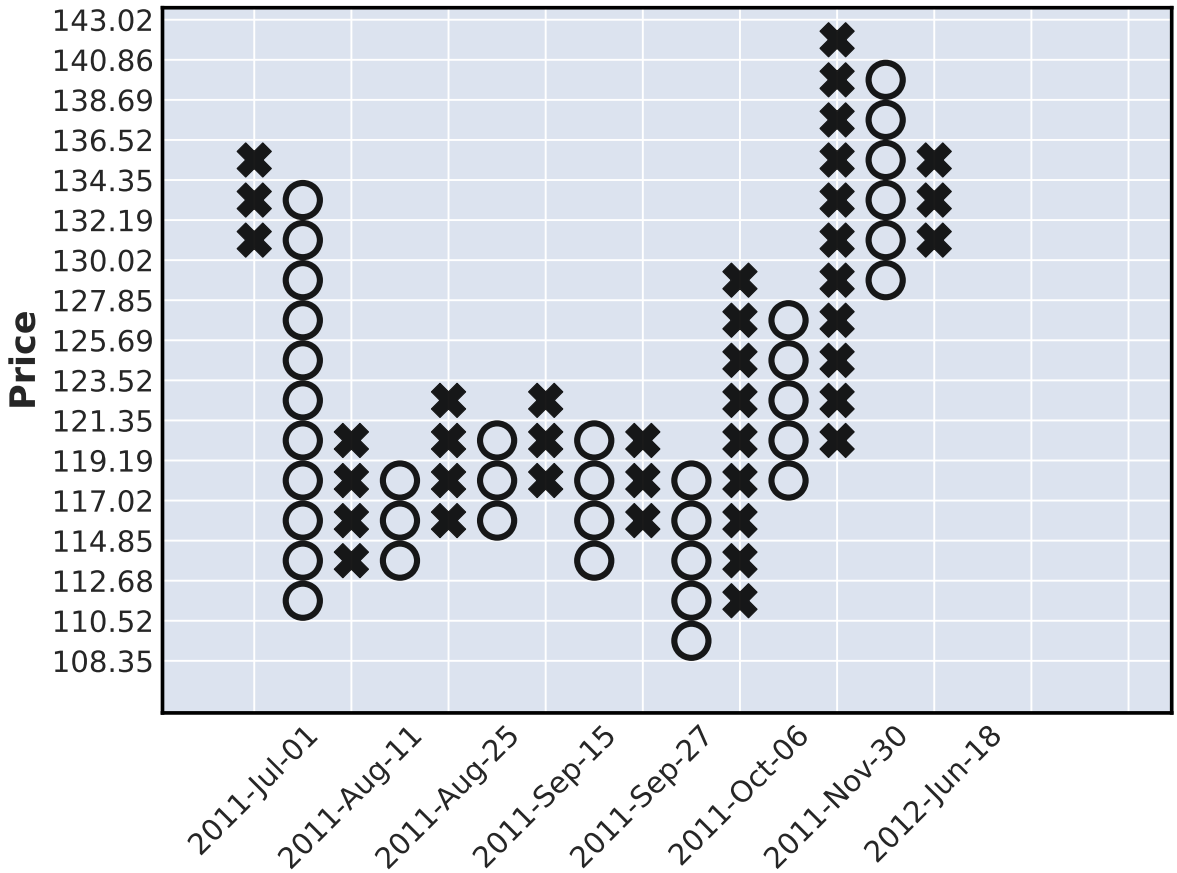
17.6.4 RenkoPlot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Mpl_Renko.py, and the XML code in the file D04_Mpl_Renko.2D.xml.



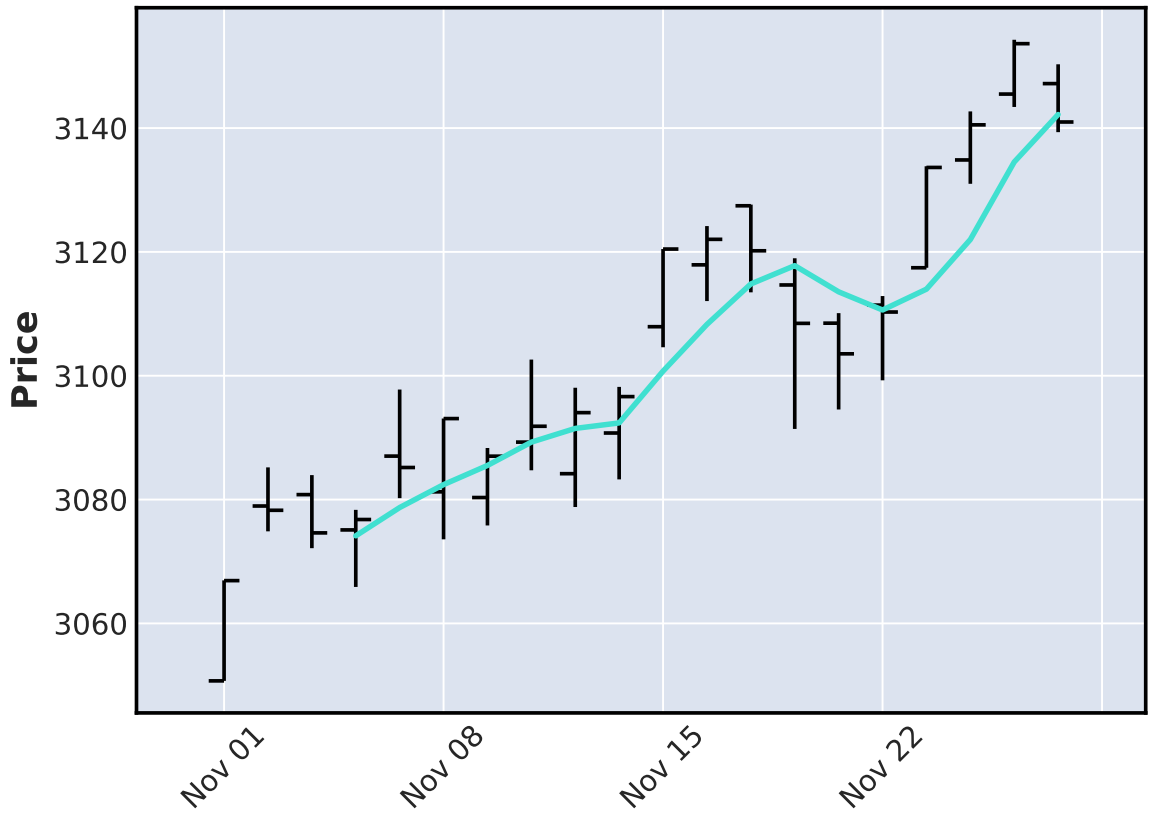
17.6.5 PnfPlot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Mpl_Pnf.py, and the XML code in the file D05_Mpl_Pnf.2D.xml.



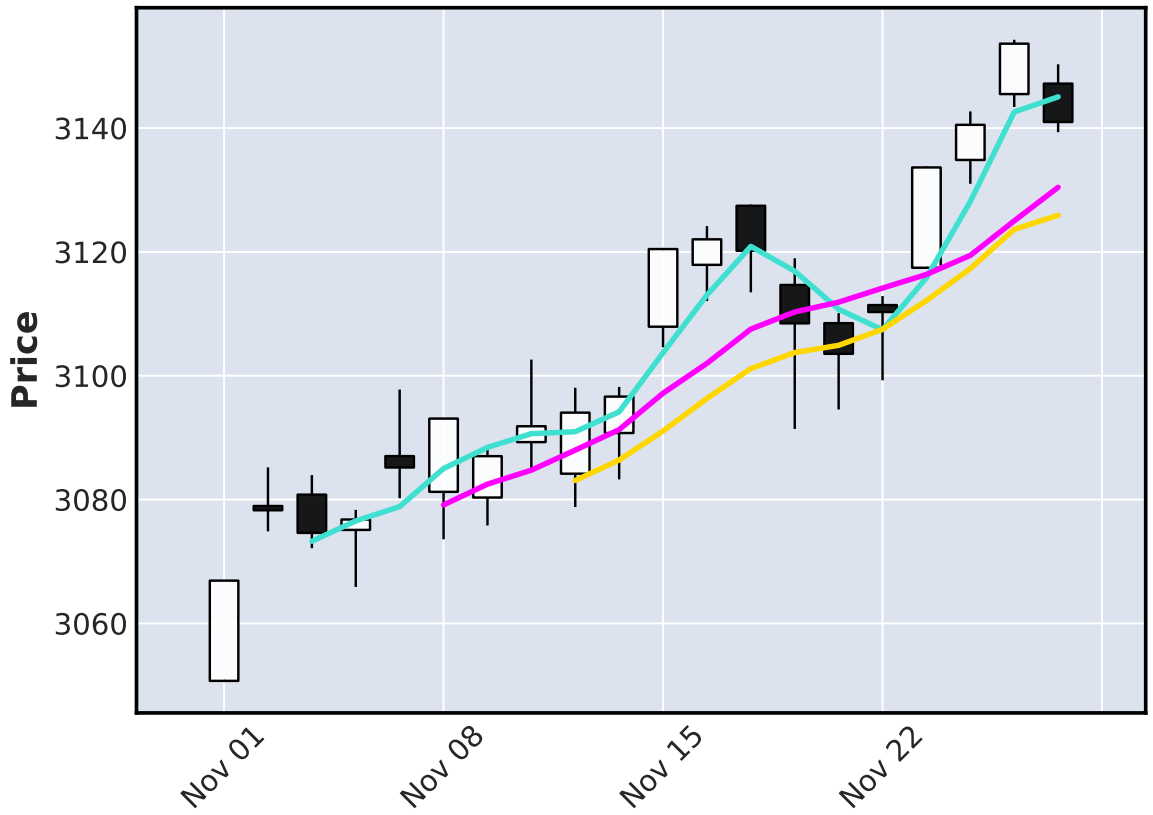
17.6.6 OhlcMav4Plot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Mpl_ohlc_mav.py, and the XML code in the file D06_Mpl_ohlc_mav.2D.xml.



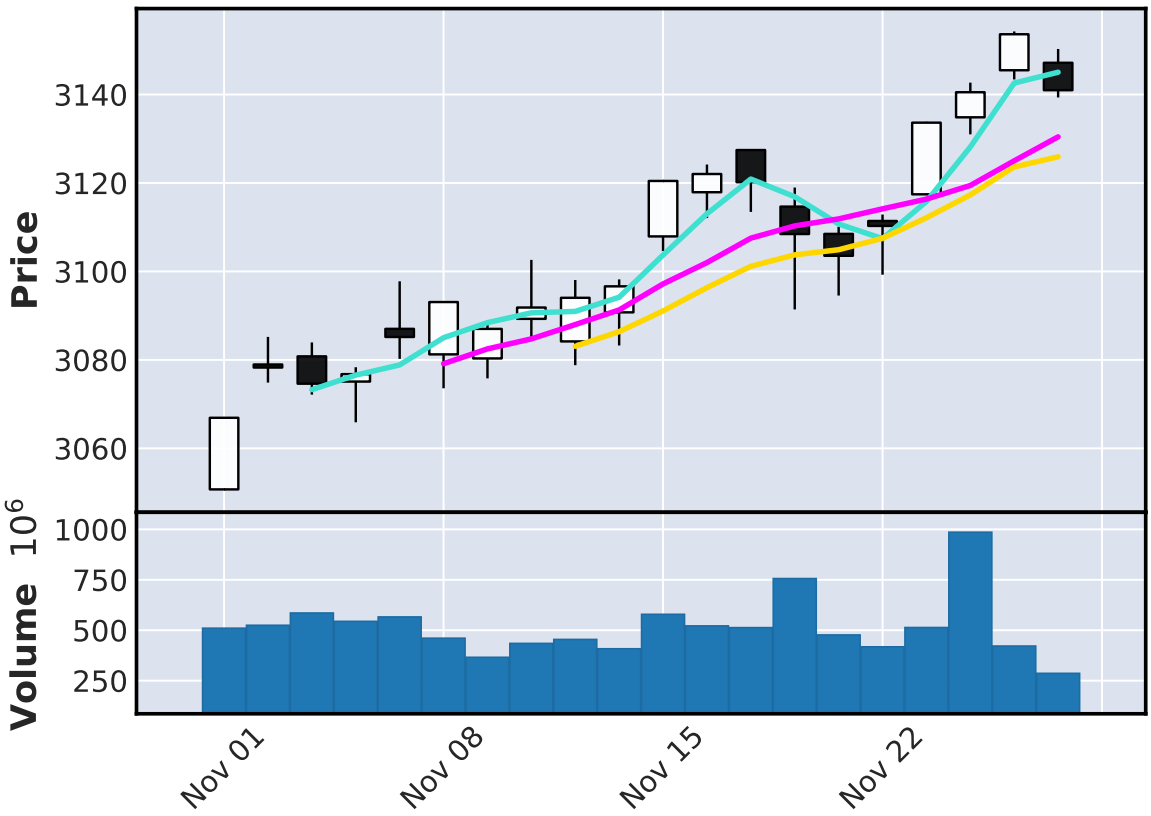
17.6.7 CandleMav369Plot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_Mpl_candle_mav369.py, and the XML code in the file D07_Mpl_candle_mav369.2D.xml.



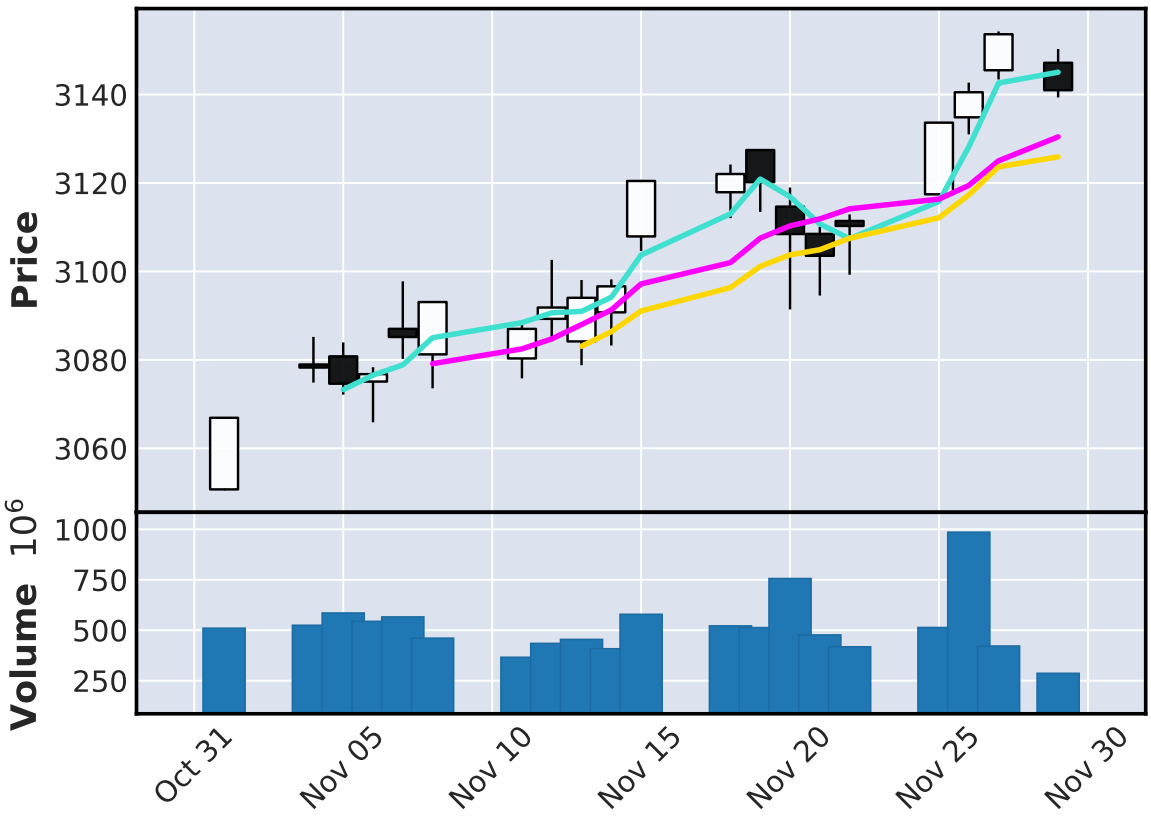
17.6.8 CandleMav369VolPlot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_Mpl_candle_mav369_vol.py, and the XML code in the file D08_Mpl_candle_mav369_vol.2D.xml.



17.6.9 CandleMav369VolNonTradPlot

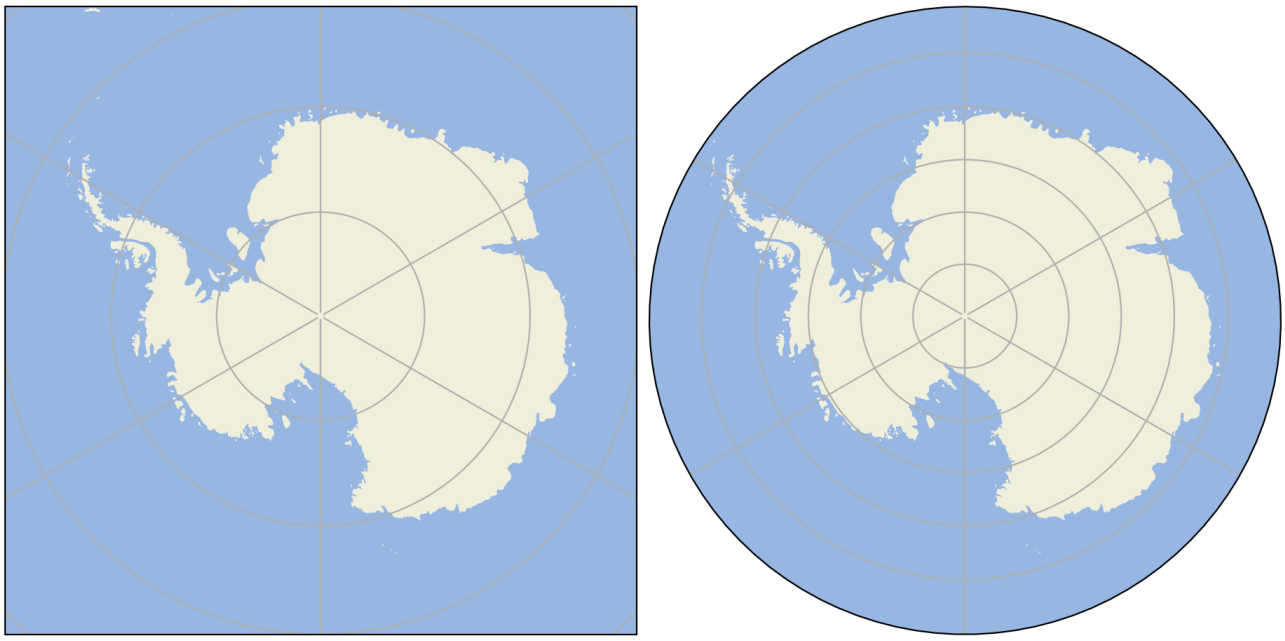
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_Mpl_candle_mav369_vol_nontrad.py, and the XML code in the file D09_Mpl_candle_mav369_vol_nontrad.2D.xml.



17.7 Geographic data (requires cartopy)

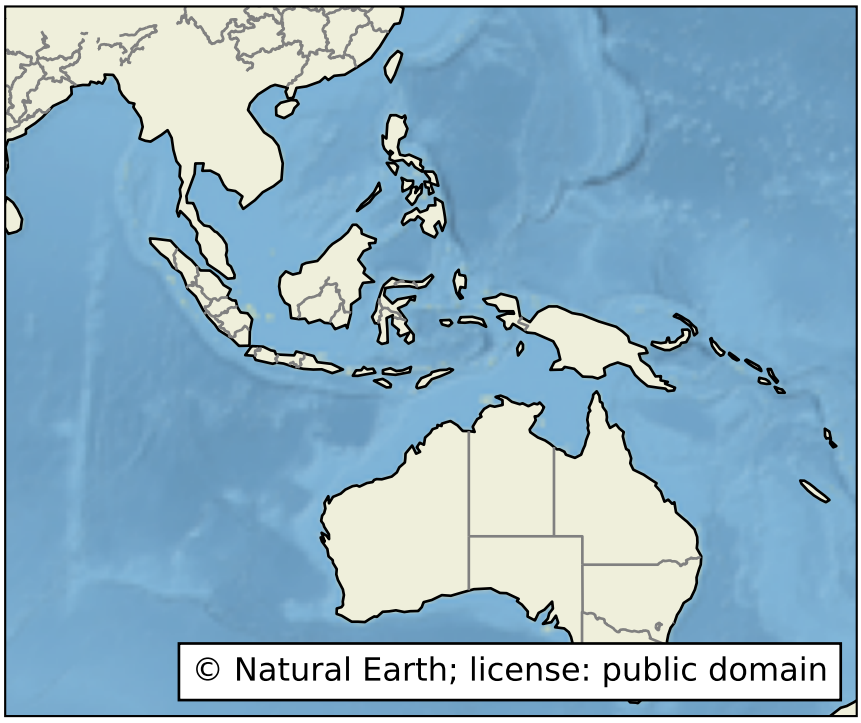
17.7.1 Antarctis

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Antarctis.py, and the XML code in the file D01_Antarctis.2D.xml.



17.7.2 Australia

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Australia.py, and the XML code in the file D02_Australia.2D.xml.

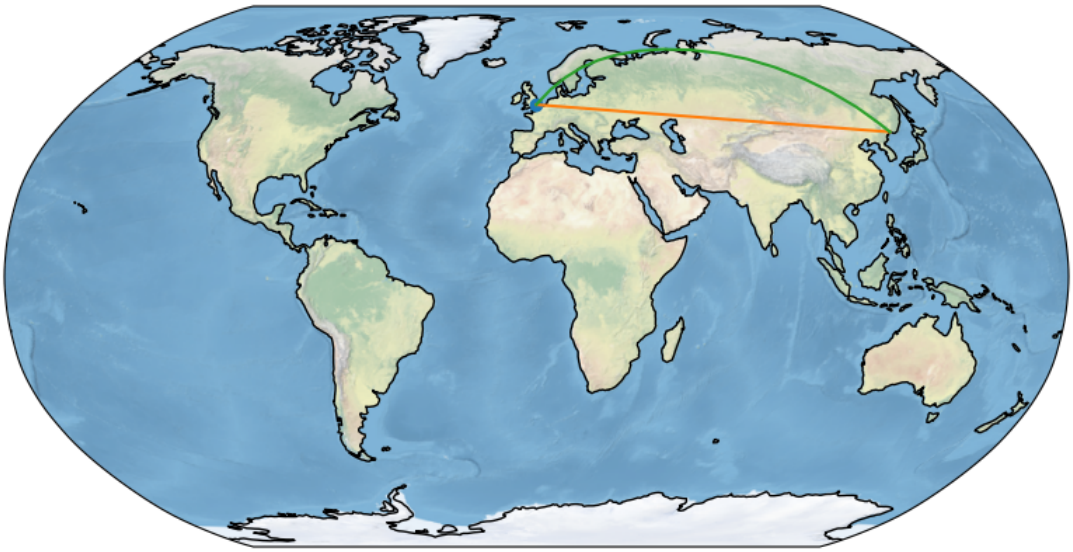


17.7.3 Africa

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Africa.py, and the XML code in the file D03_Africa.2D.xml.

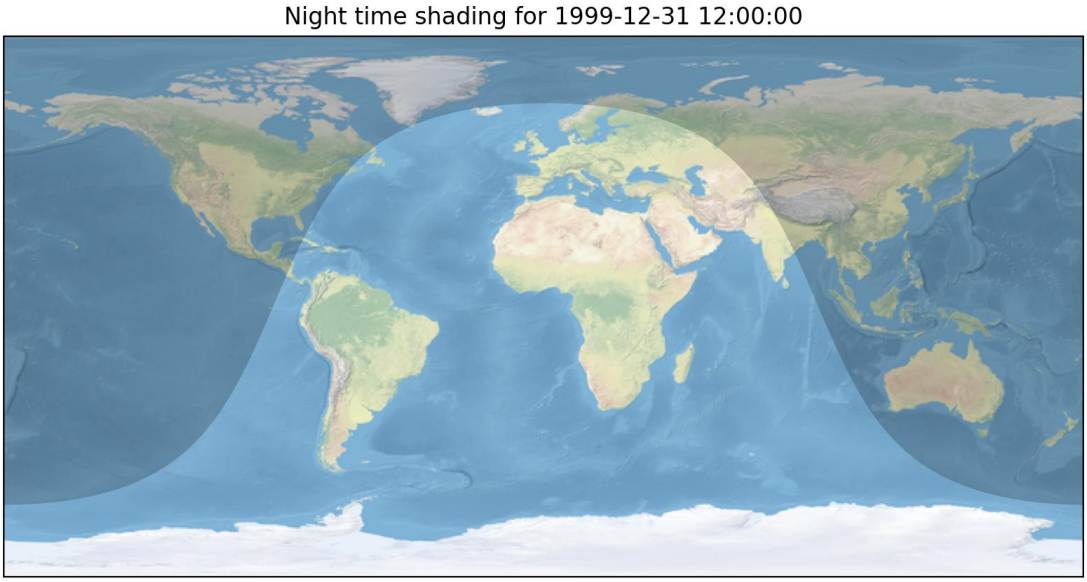
17.7.4 World Flight

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_WorldFlight.py, and the XML code in the file D04_WorldFlight.2D.xml.



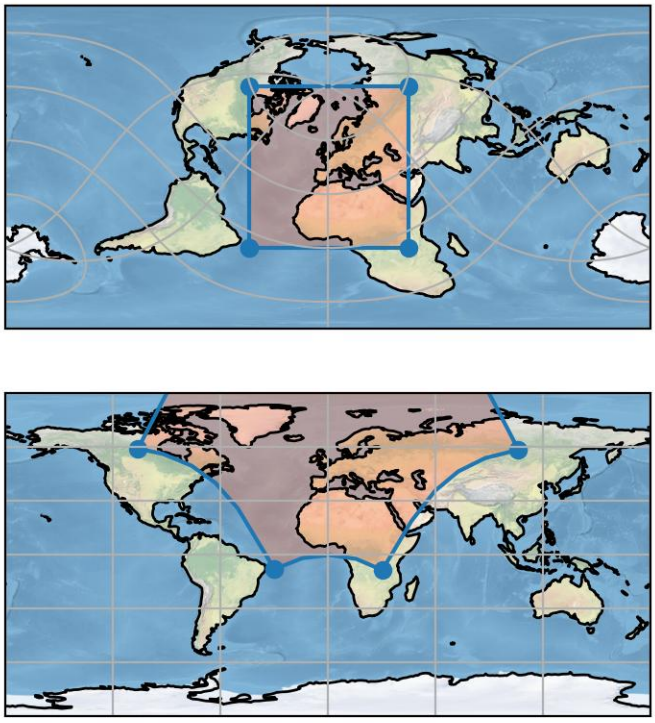
17.7.5 World Night Shading

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_WorldNightShading.py, and the XML code in the file D05_WorldNightShading.2D.xml.



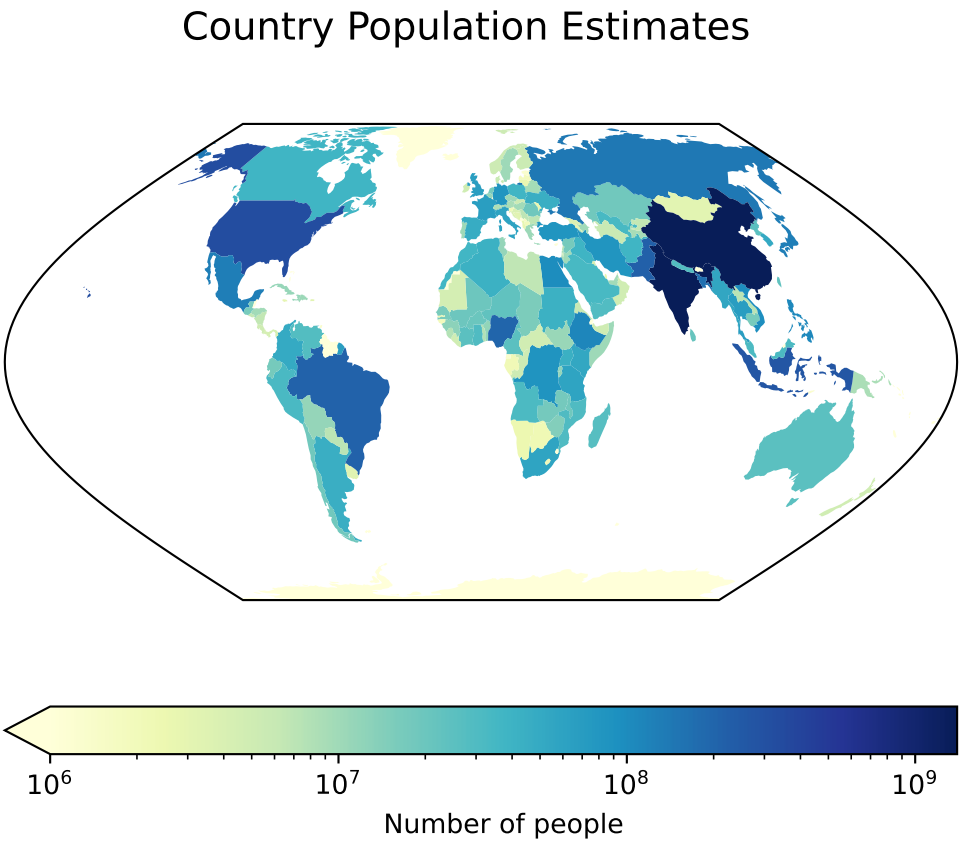
17.7.6 World2projections

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_World2Projections.py, and the XML code in the file D06_World2Projections.2D.xml.



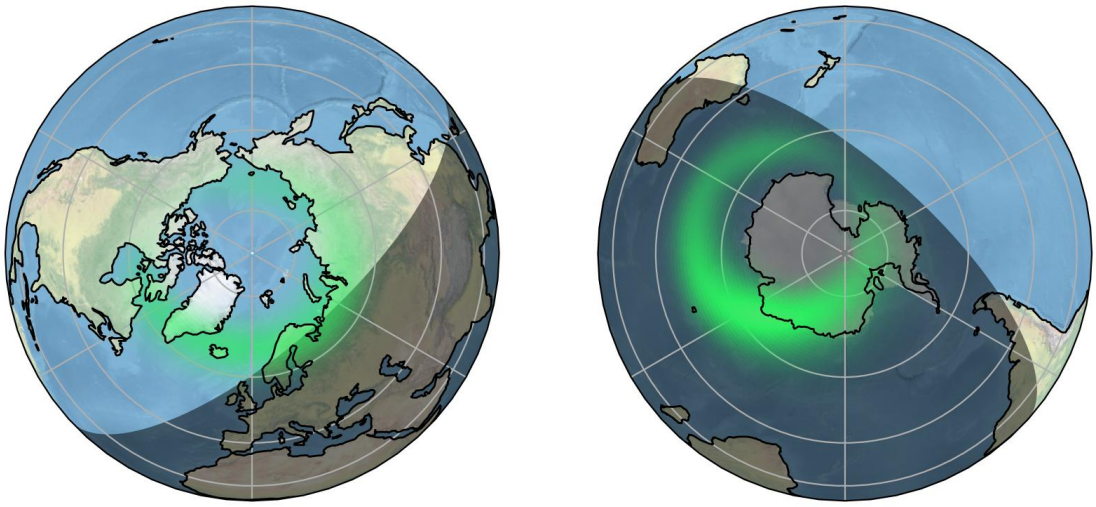
17.7.7 Country population estimates

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_CountryPopulationEstimates.py, and the XML code in the file D07_CountryPopulationEstimates.2D.xml.



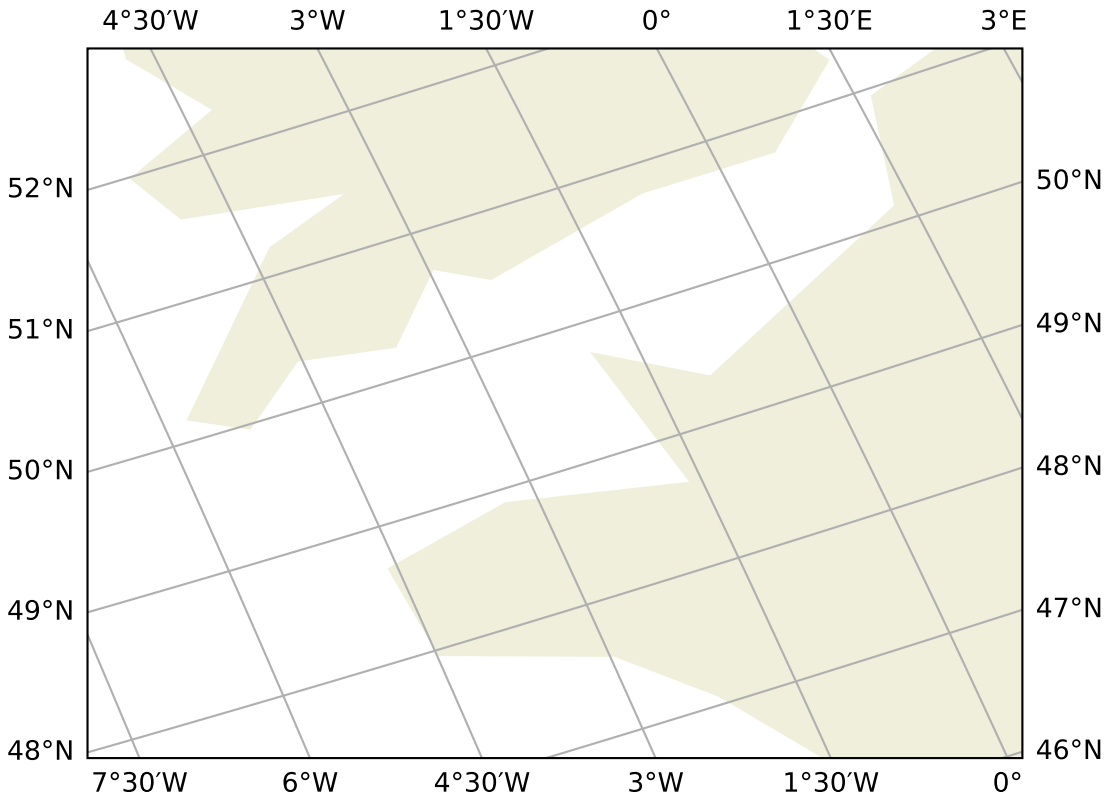
17.7.8 Arctis and Antarctis

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_ArctisAndAntarctis.py, and the XML code in the file D08_ArctisAndAntarctis.2D.xml.



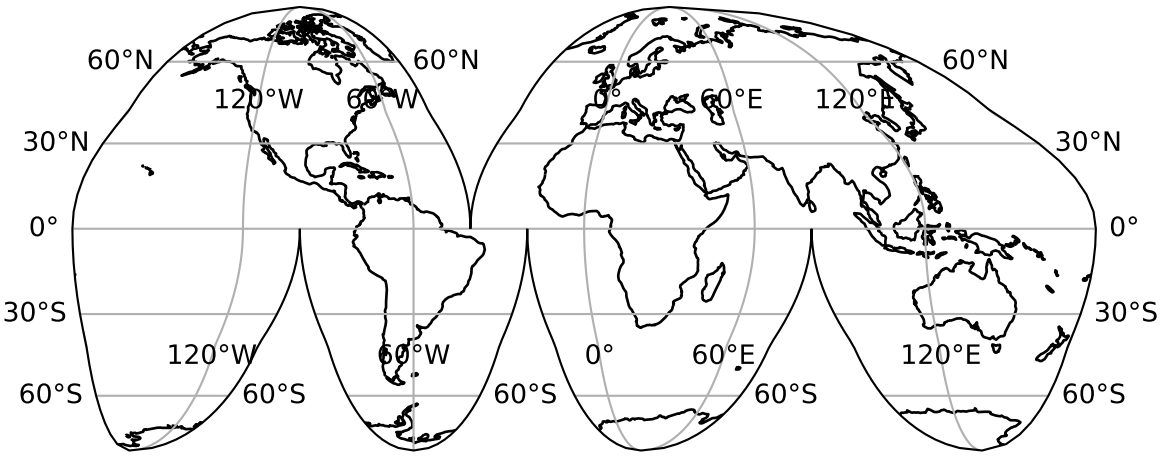
17.7.9 Geographic Mapping, with longitude and altitude on 4 axes crs=ccrs.PlateCarree

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_GeographicMapping1.py, and the XML code in the file D09_GeographicMapping1.2D.xml.



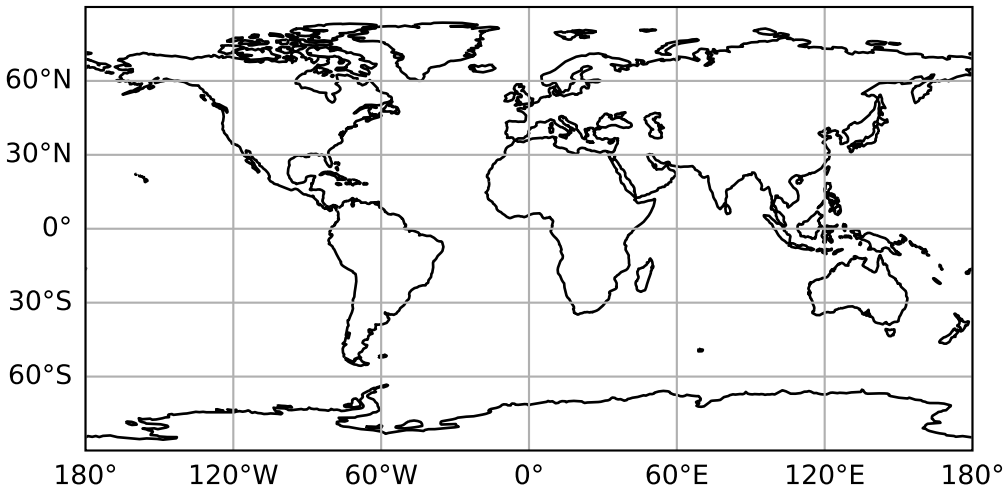
17.7.10 Geographic Mapping, World, projection=ccrs.InterruptedGoodeHomolosine

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_GeographicMapping2.py, and the XML code in the file D10_GeographicMapping2.2D.xml.



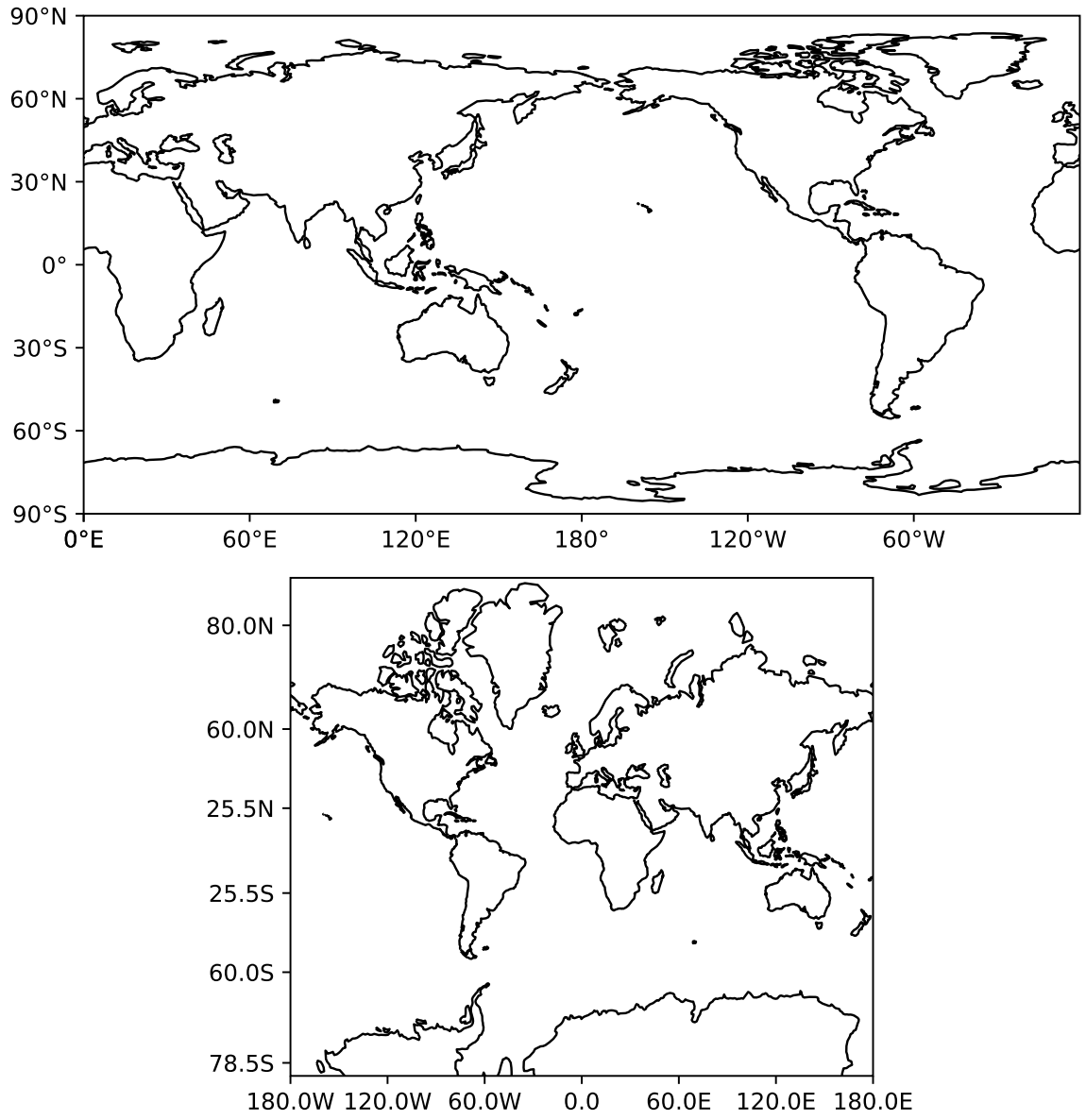
17.7.11 Geographic Mapping, World, projection=ccrs.PlateCarree

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11_GeographicMapping3.py, and the XML code in the file D11_GeographicMapping3.2D.xml.



17.7.12 Labelling projection axes: PlateCarree and Mercator

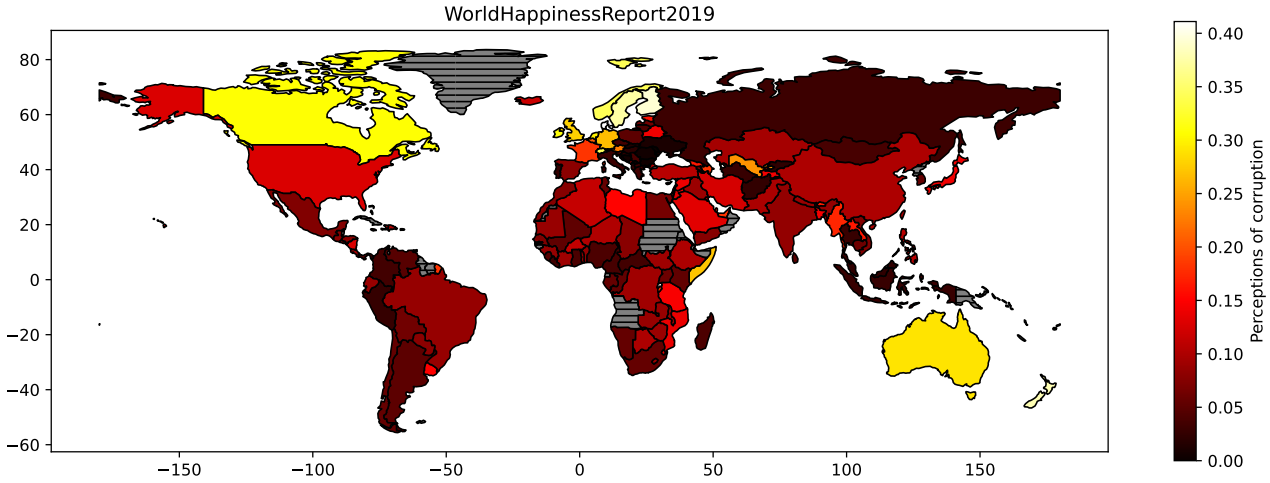
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D12_LabellingProjectionAxis.py, and the XML code in the file D12_LabellingProjectionAxis.2D.xml.



17.8 Geographic data (requires geopandas)

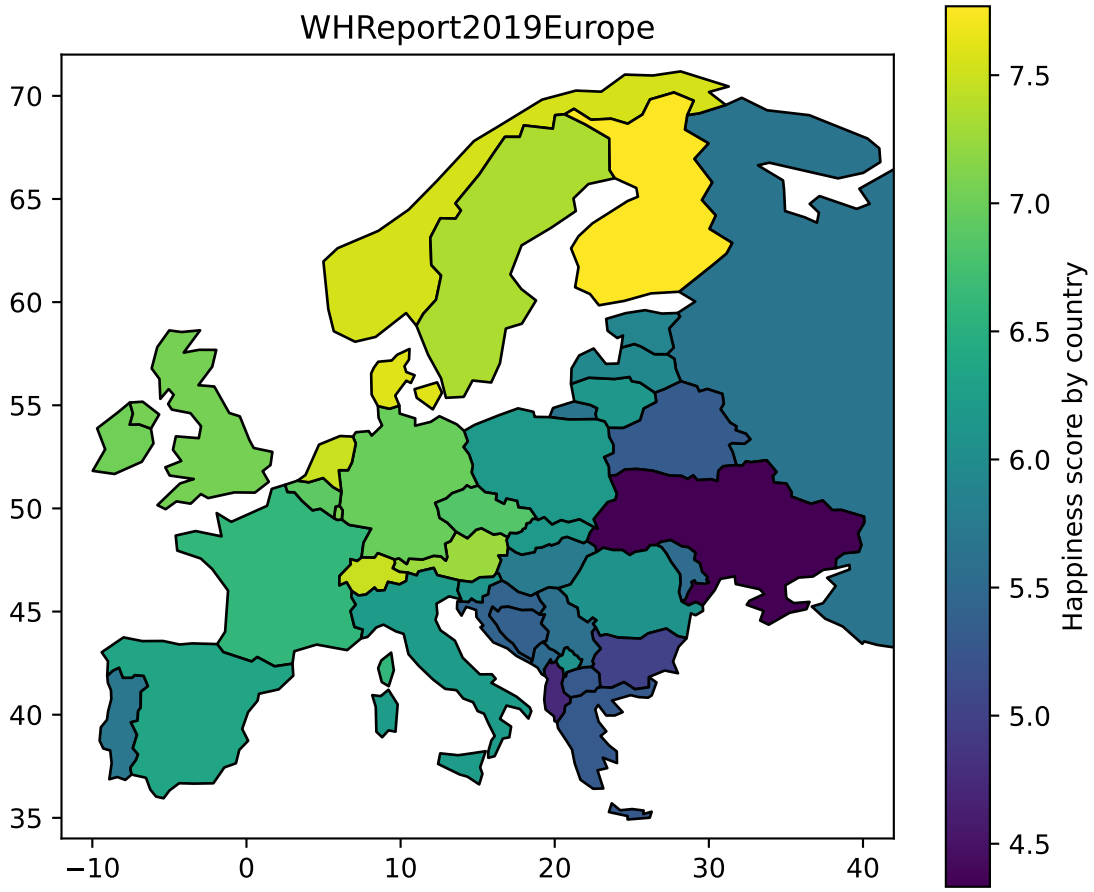
17.8.1 World Happiness Report 2019

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_World.py, and the XML code in the file D01_World.2D.xml.



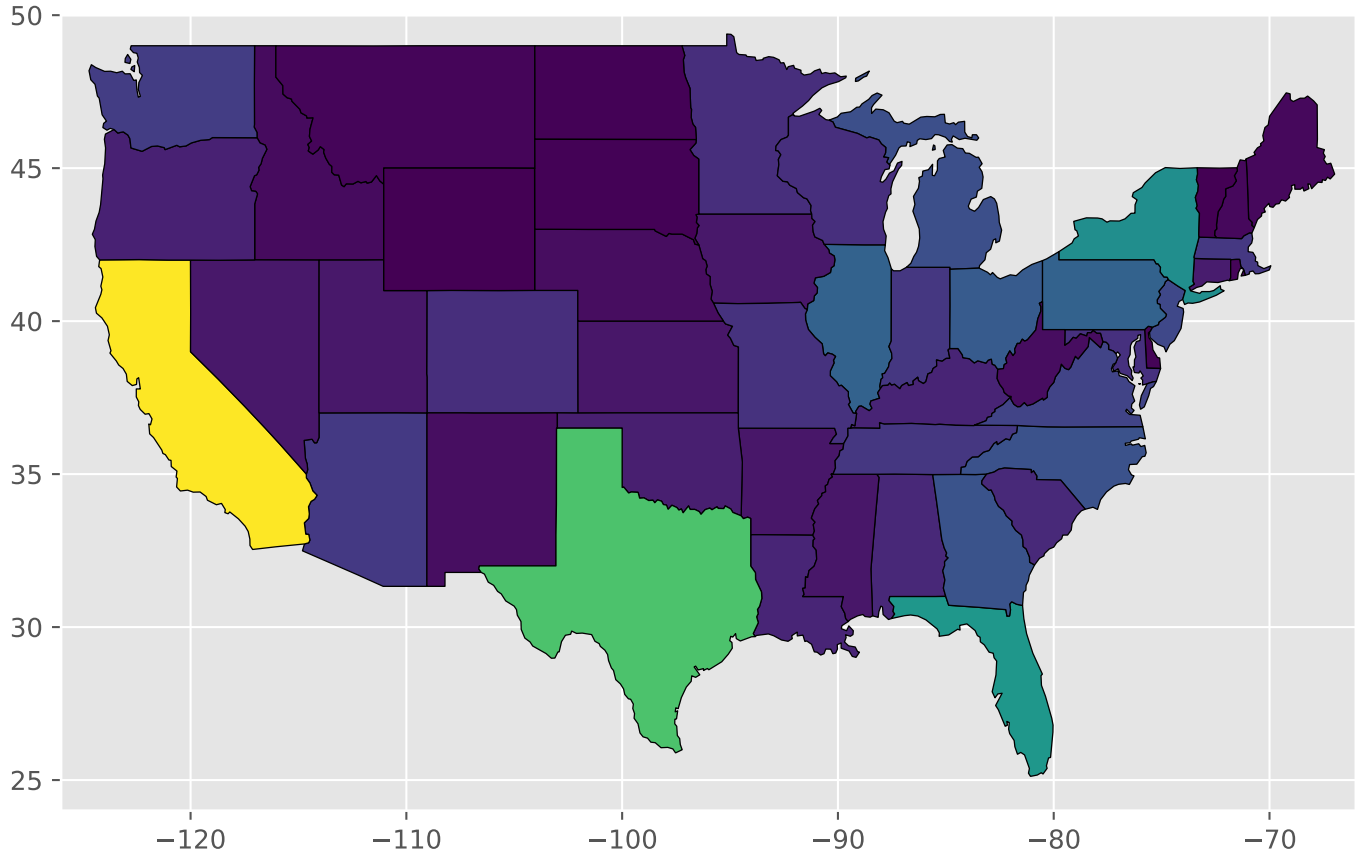
17.8.2 World Happiness Report 2019: Europe

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Europe.py, and the XML code in the file D02_Europe.2D.xml.



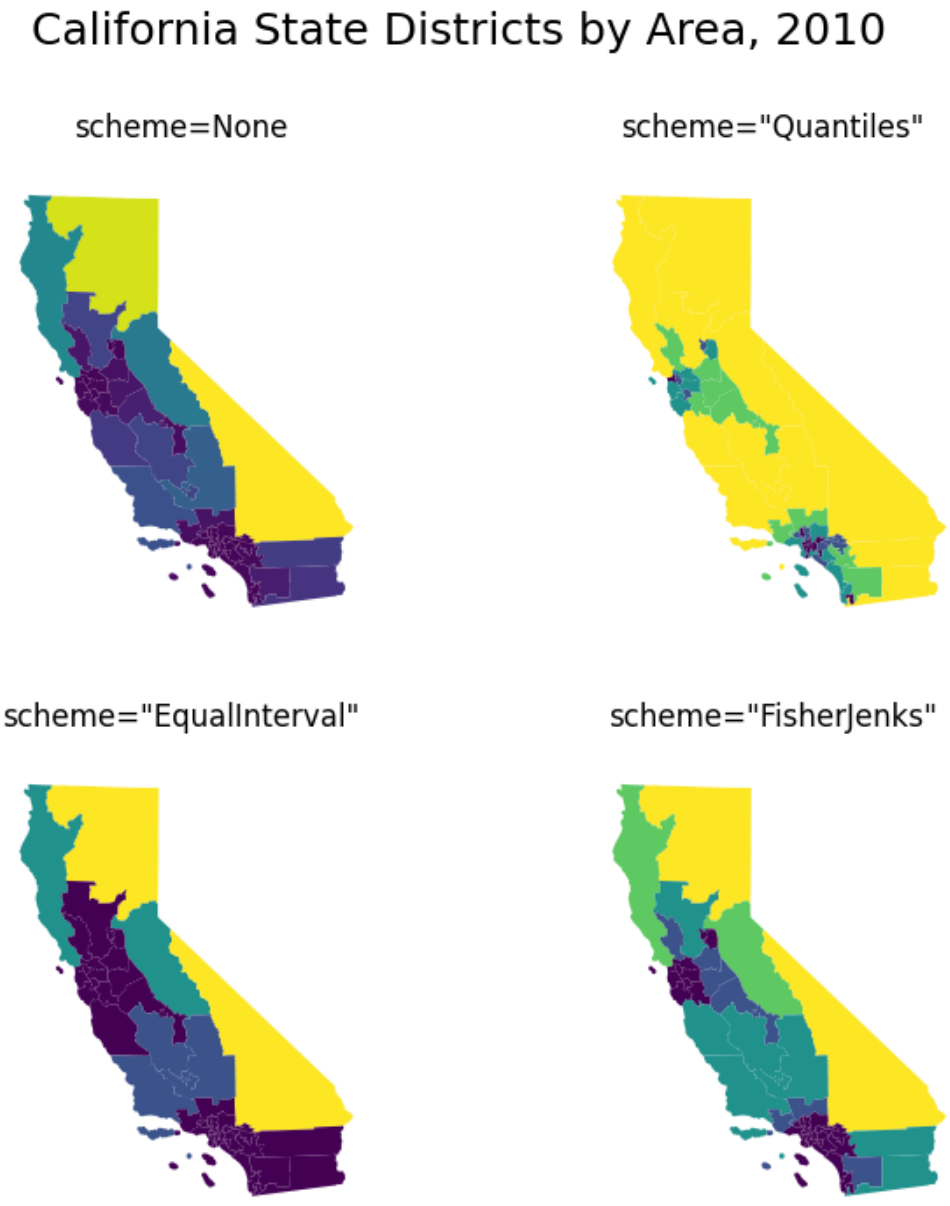
17.8.3 US state populations

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_USPopulation.py, and the XML code in the file D03_USPopulation.2D.xml.



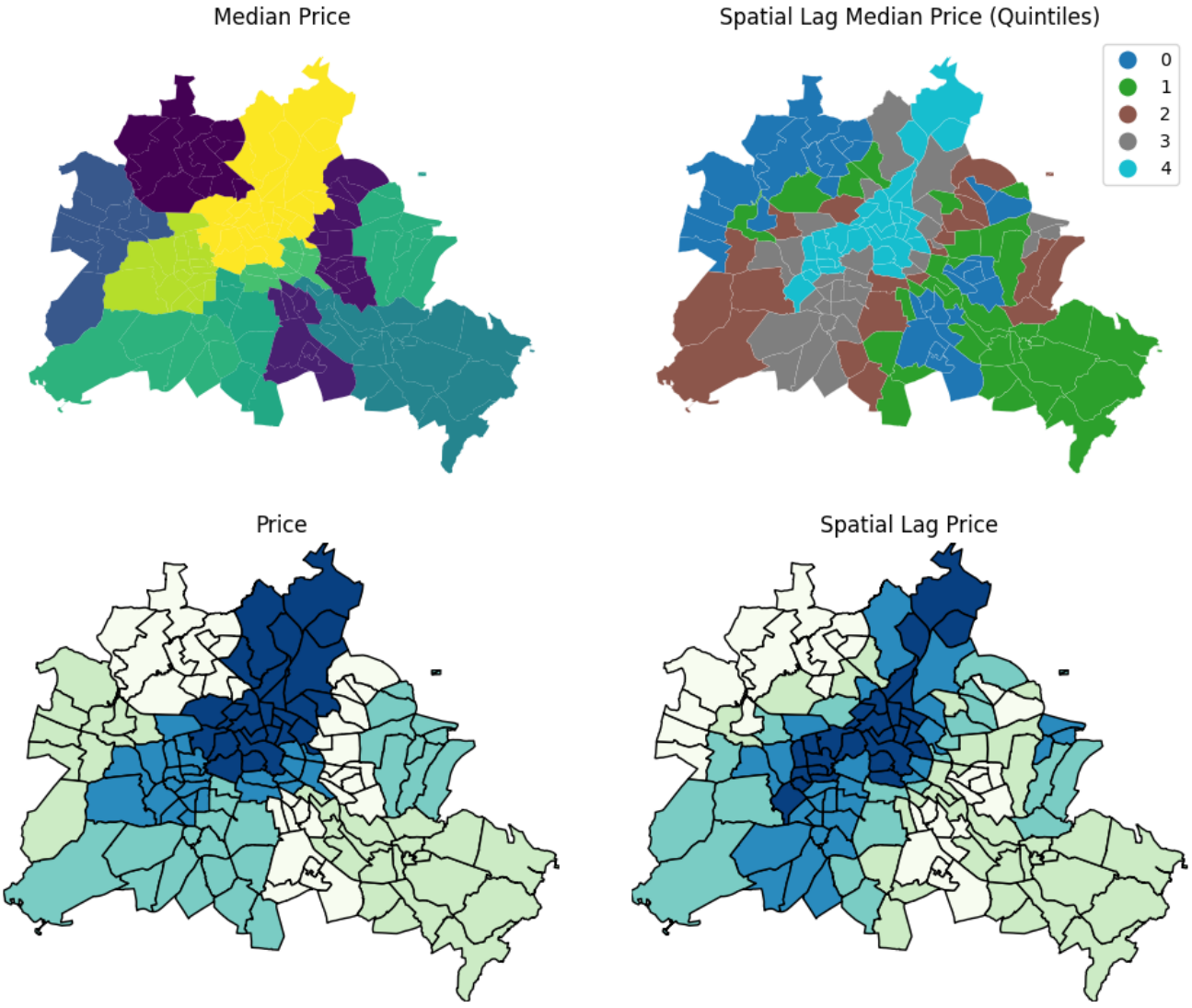
17.8.4 California

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Californa.py, and the XML code in the file D04_Californa.2D.xml.



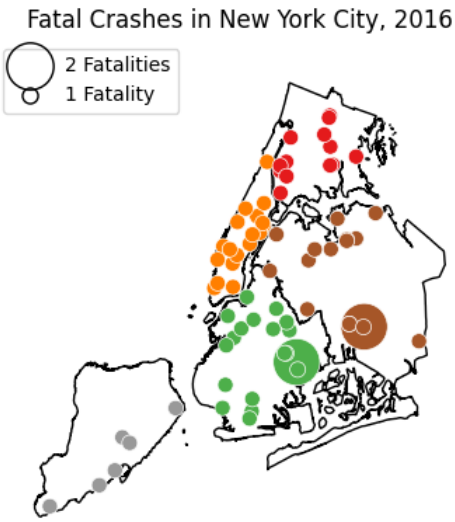
17.8.5 Berlin, Esda

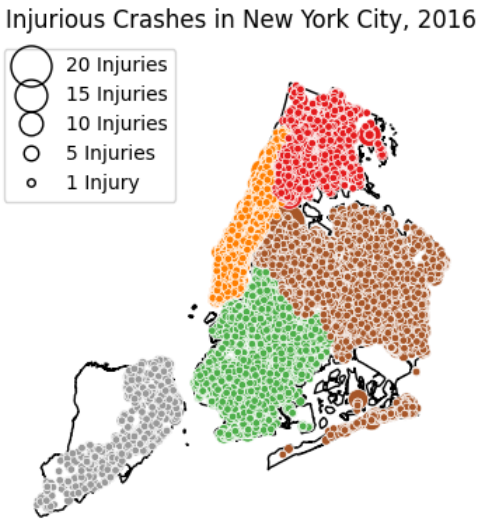
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Berlin.py, and the XML code in the file D05_Berlin.2D.xml.



17.8.6 New York, Collisions

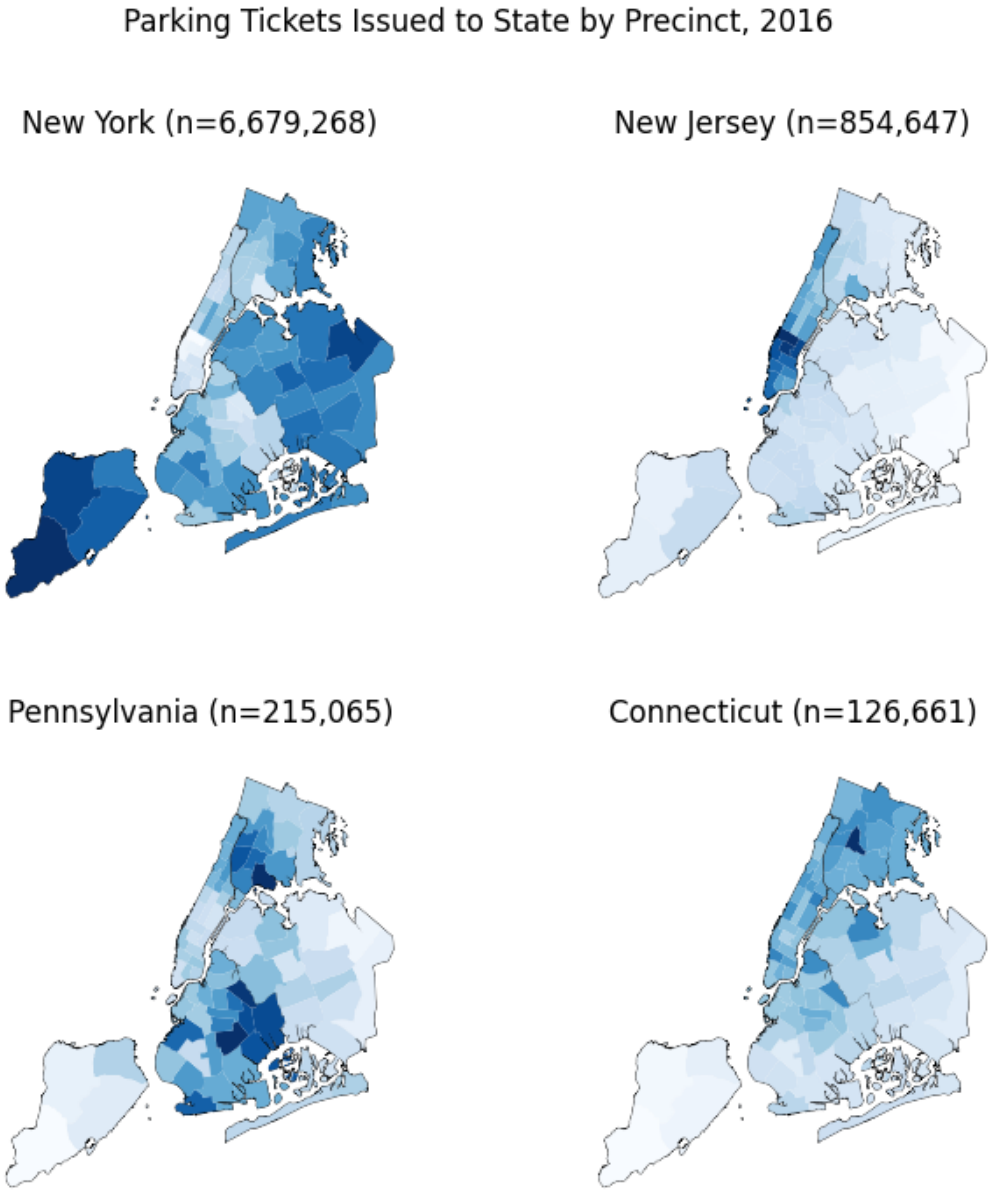
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_NY_Collisions.py, and the XML code in the file D06_NY_Collisions.2D.xml.





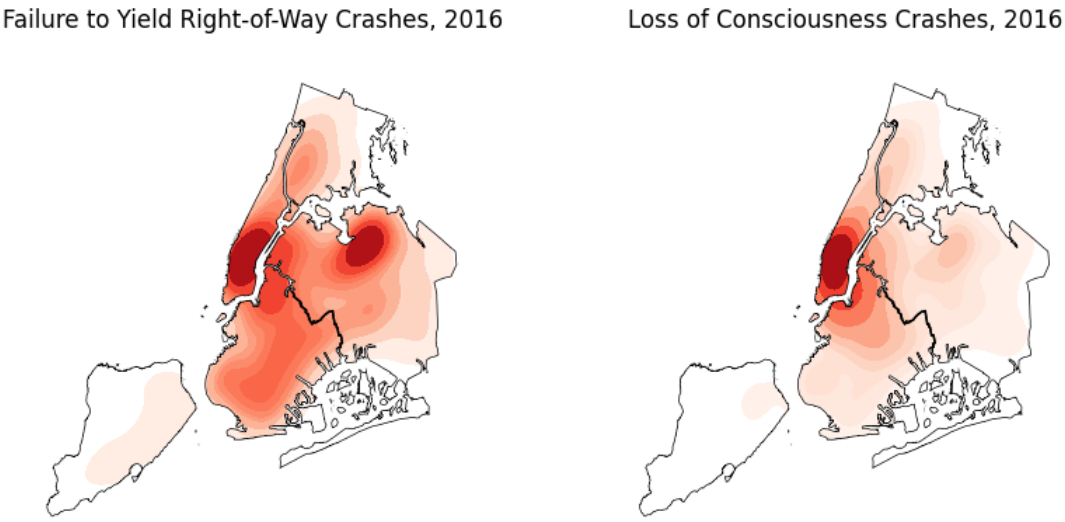
17.8.7 New York, Tickets

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_NY_Tickets.py, and the XML code in the file D07_NY_Tickets.2D.xml.



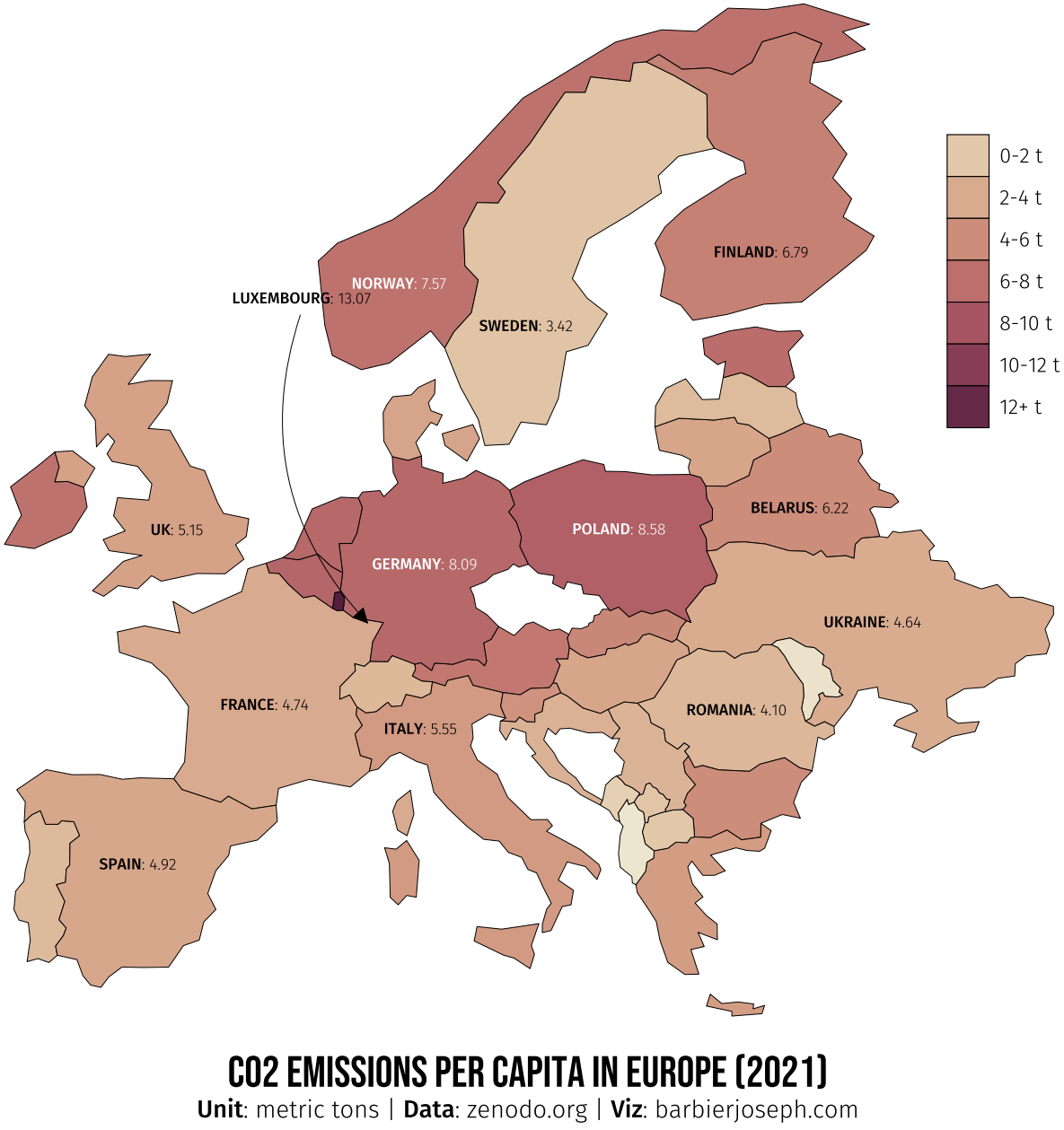
17.8.8 New York, Factors

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_NY_Factors.py, and the XML code in the file D08_NY_Factors.2D.xml.



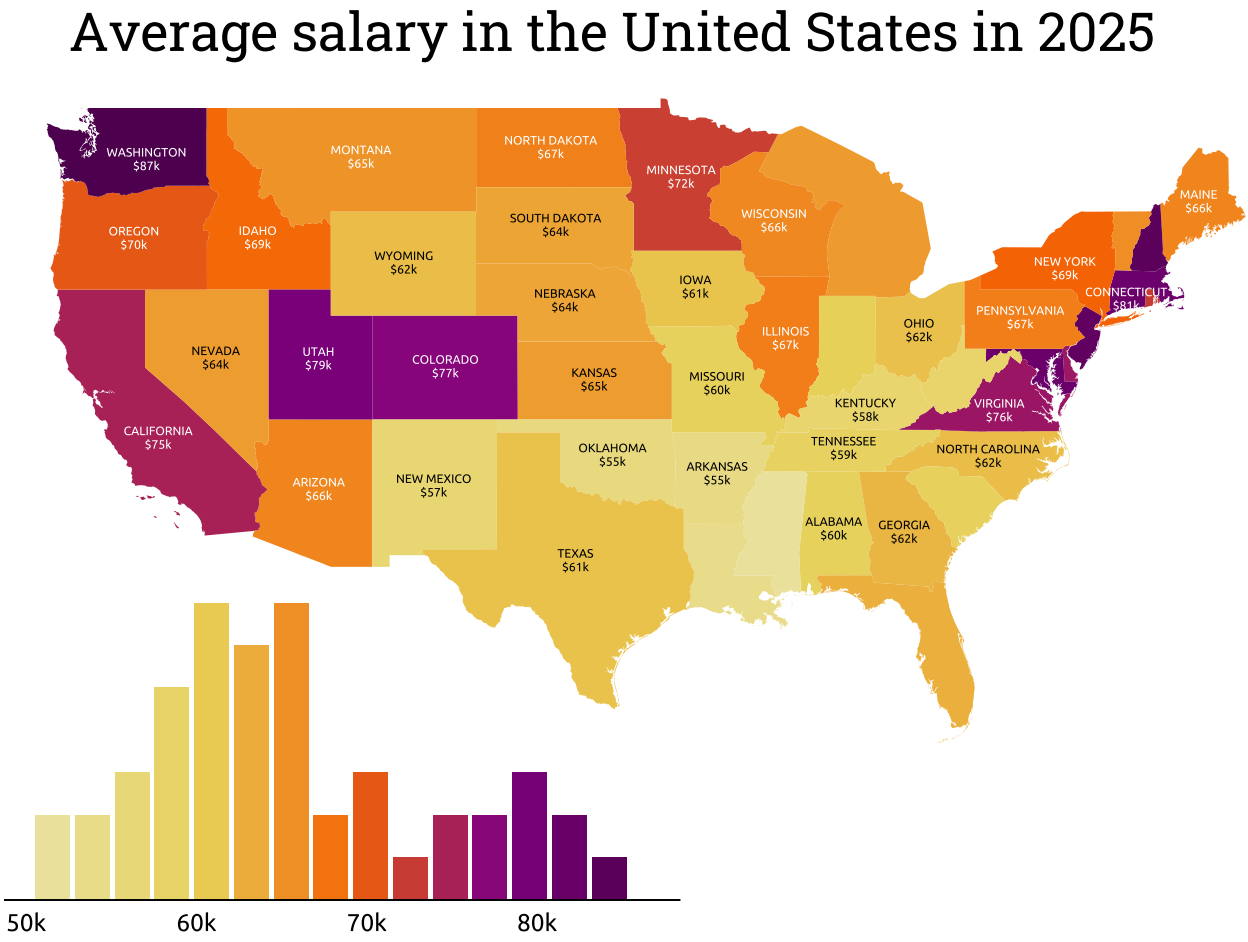
17.8.9 Europe CO2 Emissions

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_EuropeCO2.py, and the XML code in the file D09_EuropeCO2.2D.xml.



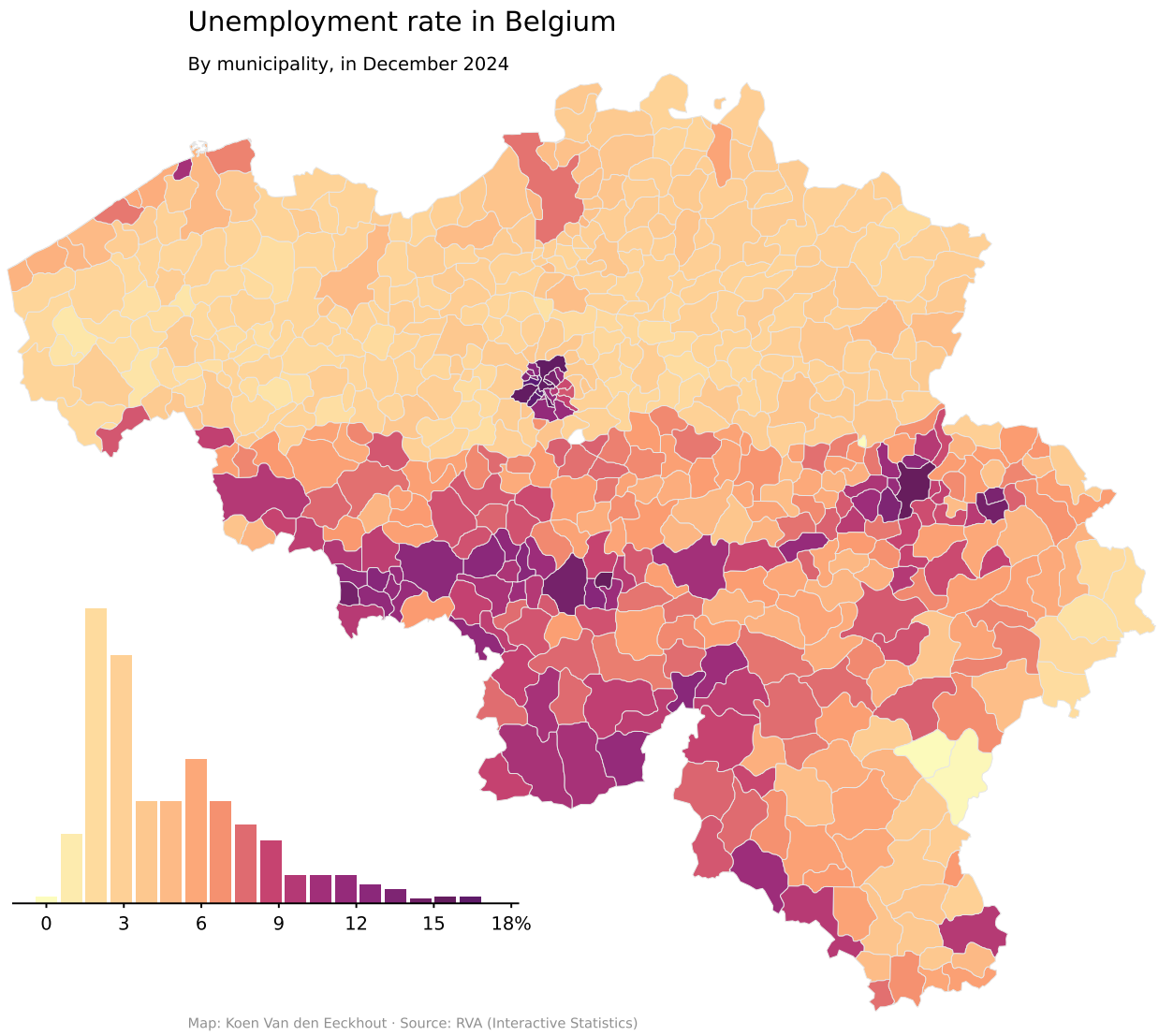
17.8.10 US, Salary

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_USSalary.py, and the XML code in the file D10_USSalary.2D.xml.



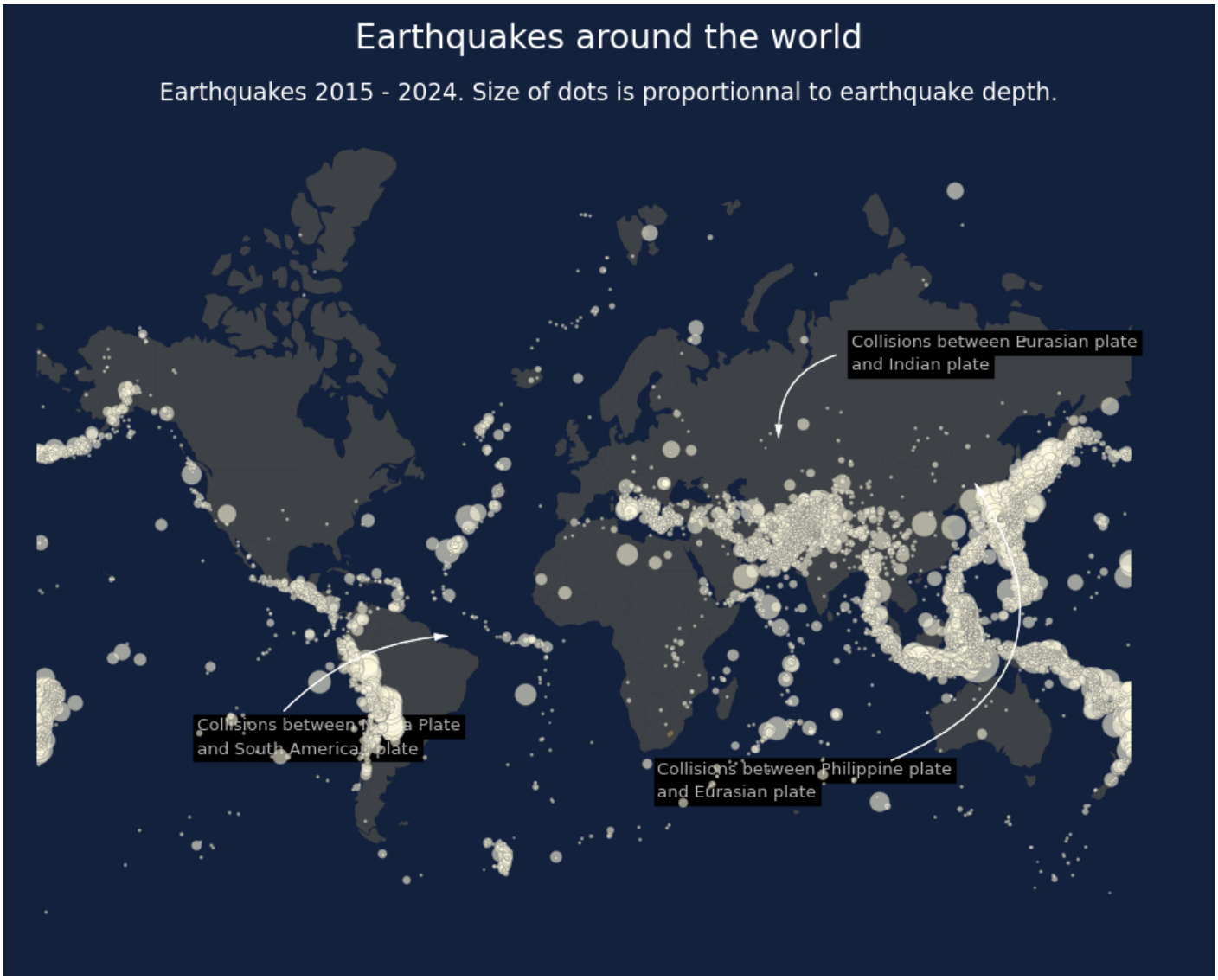
17.8.11 Belgium, unemployment

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11_Belgium.py, and the XML code in the file D11_Belgium.2D.xml.



17.8.12 Earthquakes

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D12_Earthquakes.py, and the XML code in the file D12_Earthquakes.2D.xml.

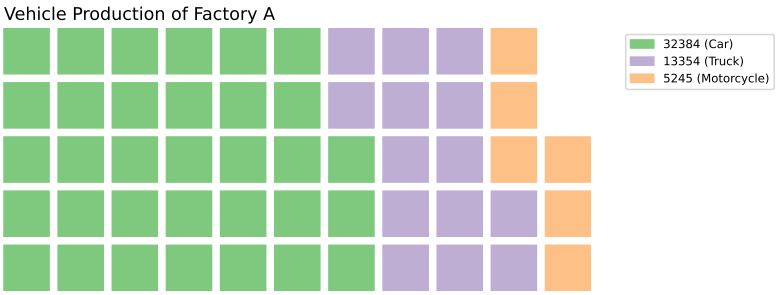


17.9 Parts of a whole

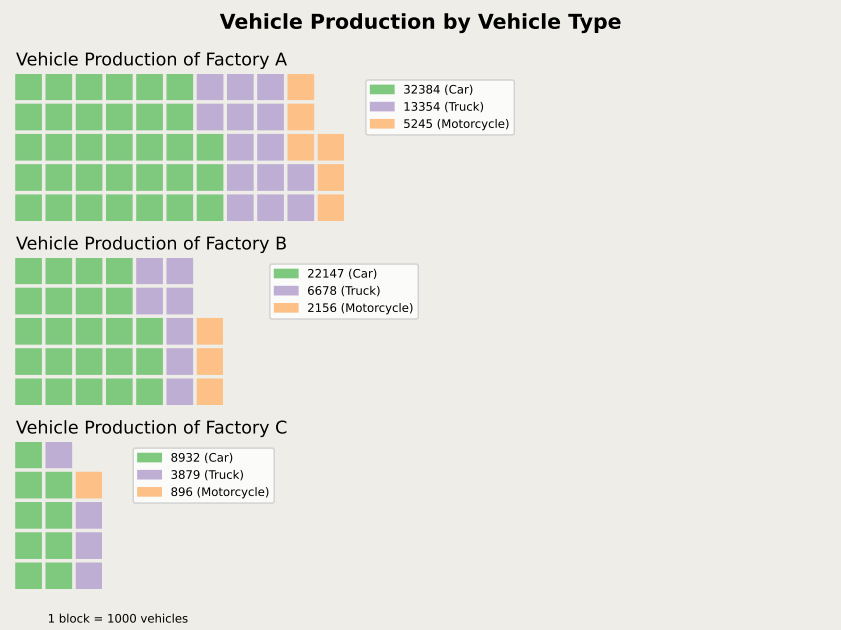
17.9.1 Waffle Chart

The Python code for the first example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01a_WaffleChart1.py, and the XML code in the file D01a_WaffleChart1.2D.xml.

The Python code for the second example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01b_WaffleChart2.py, and the XML code in the file D01b_WaffleChart2.2D.xml.



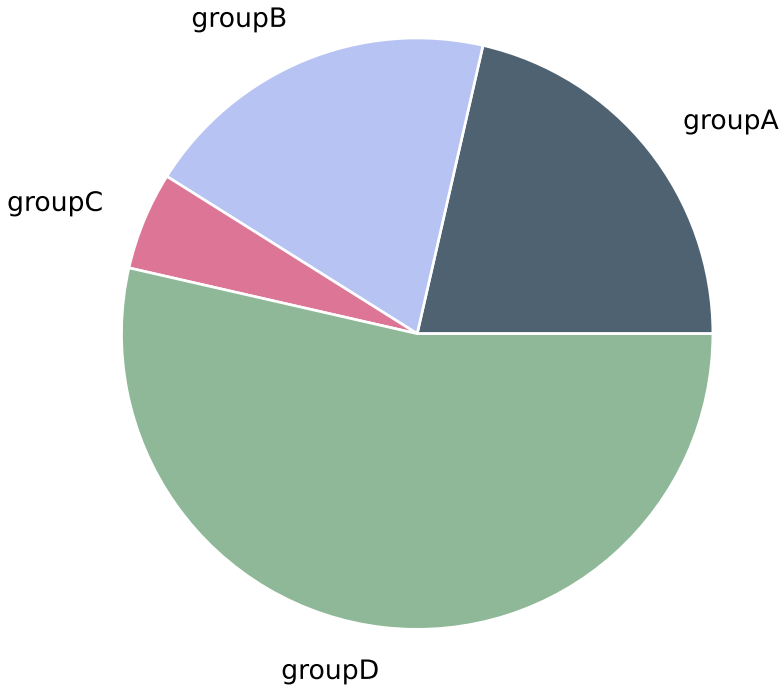
Some text



Some text

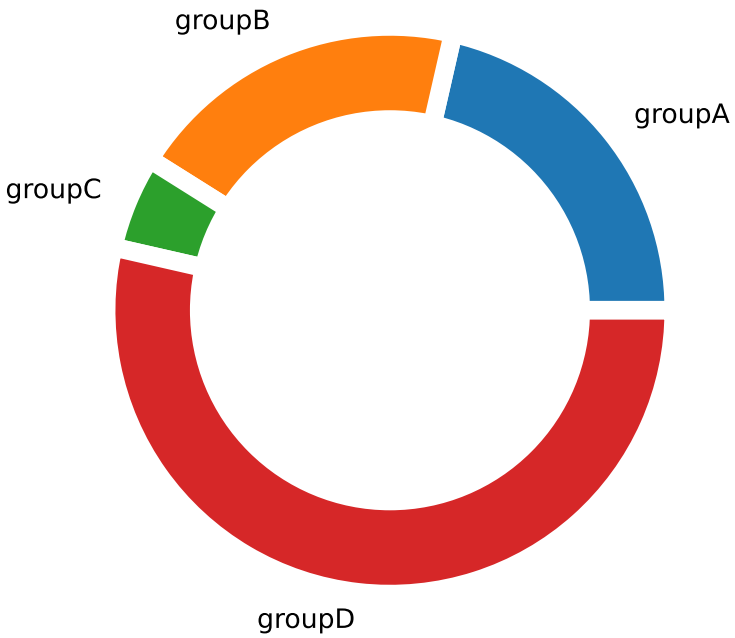
17.9.2 Pie Chart

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_PieChart.py, and the XML code in the file D02_PieChart.2D.xml.



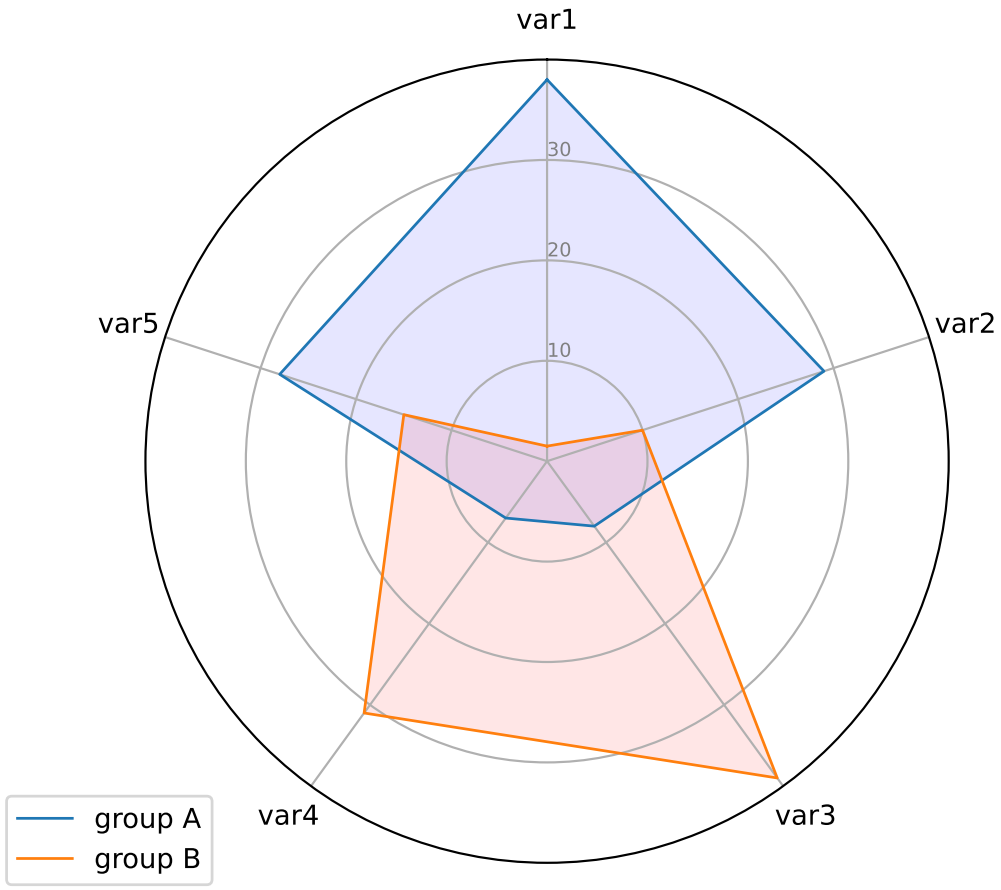
17.9.3 Donut Chart

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_DonutChart.py, and the XML code in the file D03_DonutChart.2D.xml.



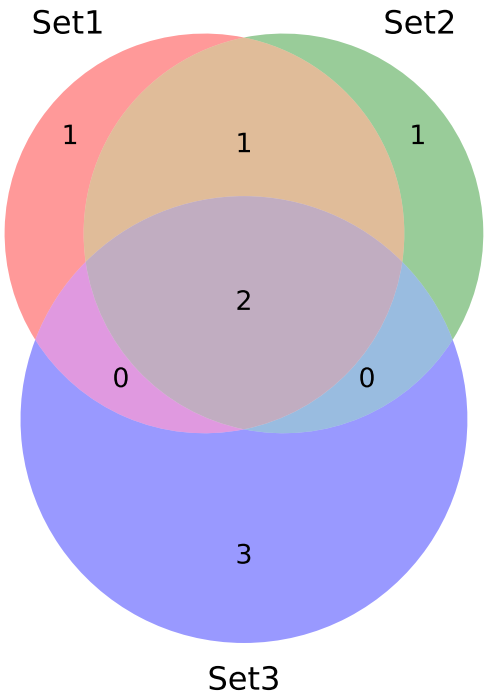
17.9.4 Radar Chart

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_RadarChart.py, and the XML code in the file D04_RadarChart.2D.xml.



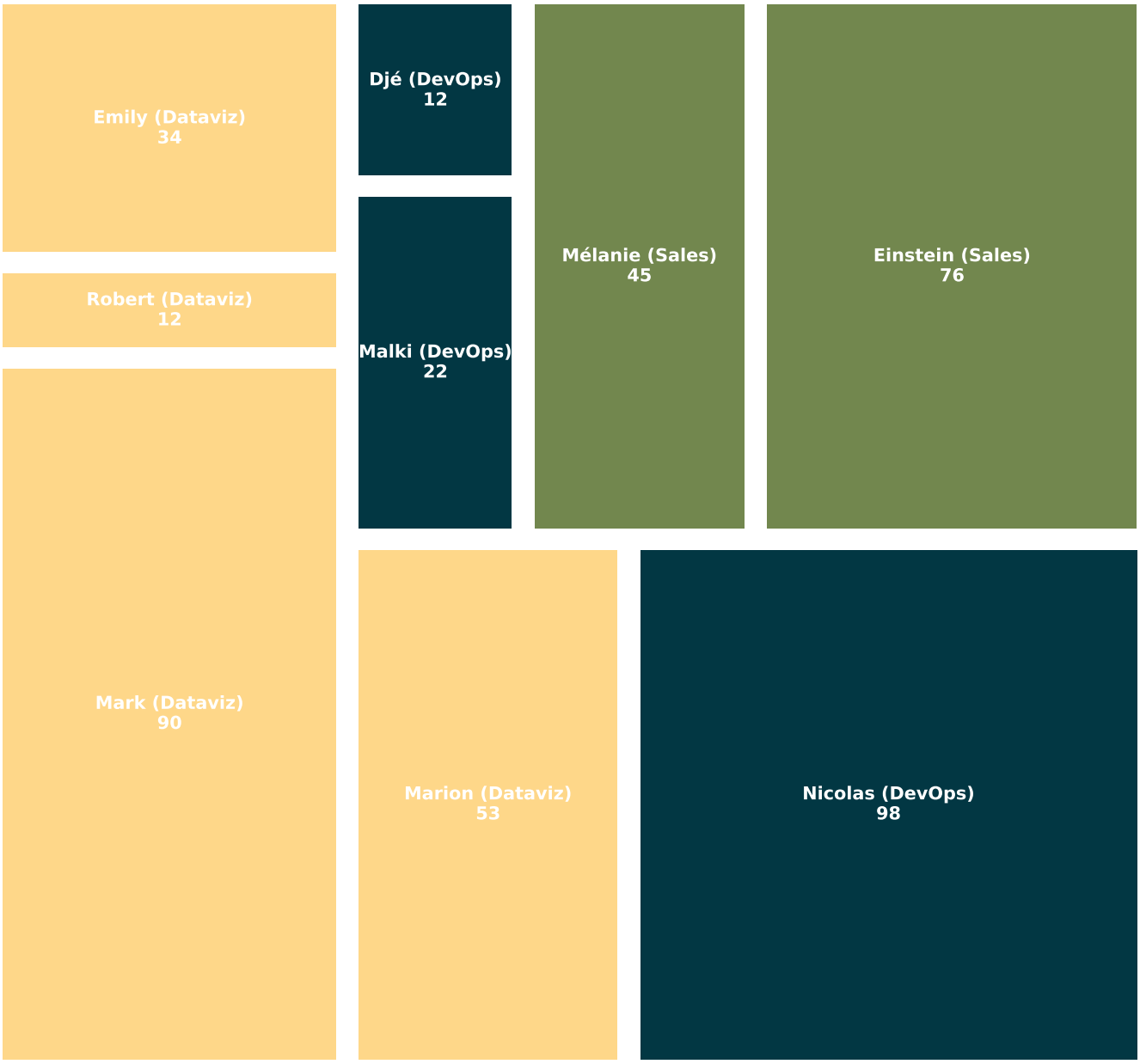
17.9.5 Venn Diagram

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_VennDiagram.py, and the XML code in the file D05_VennDiagram.2D.xml.



17.9.6 Treemap

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Treemap.py, and the XML code in the file D06_Treemap.2D.xml.



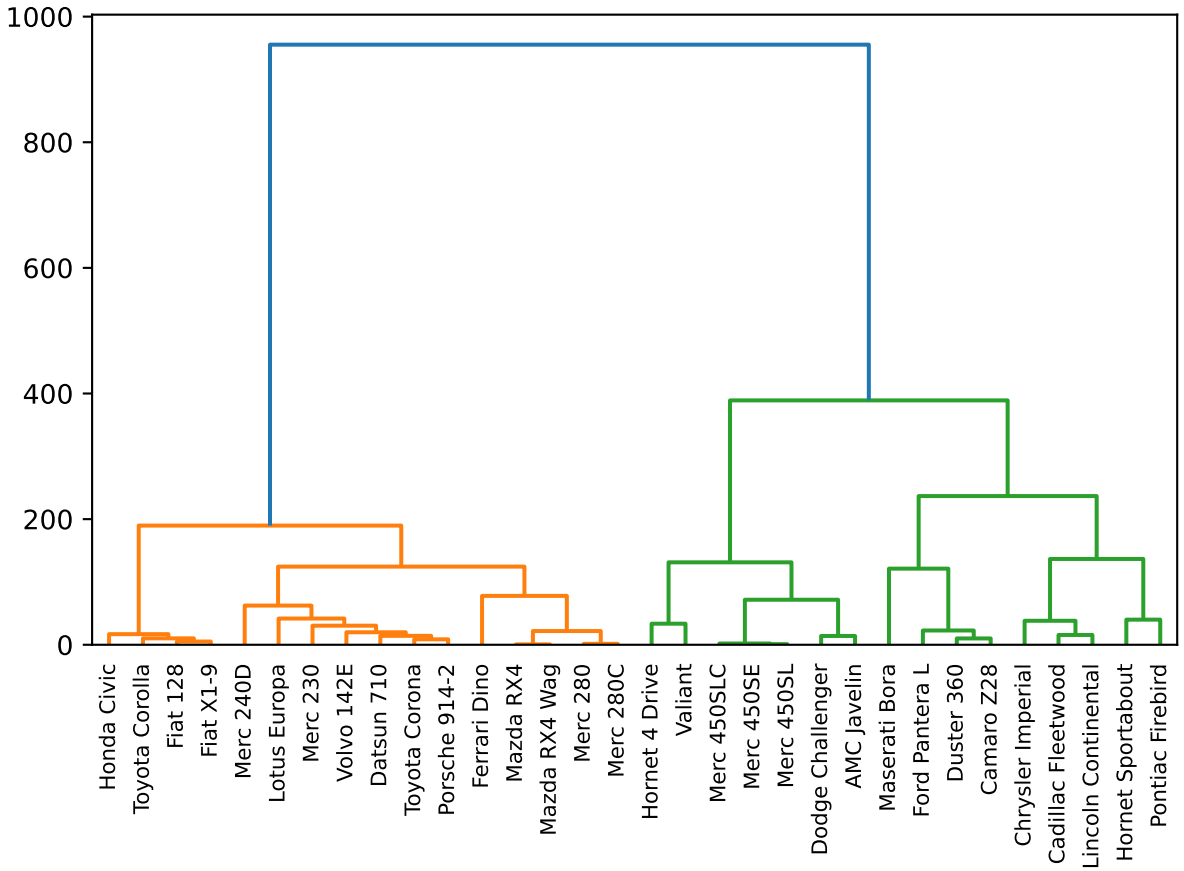
17.10 Dendrograms, heatmaps and clustermaps

17.10.1 Dendrogram

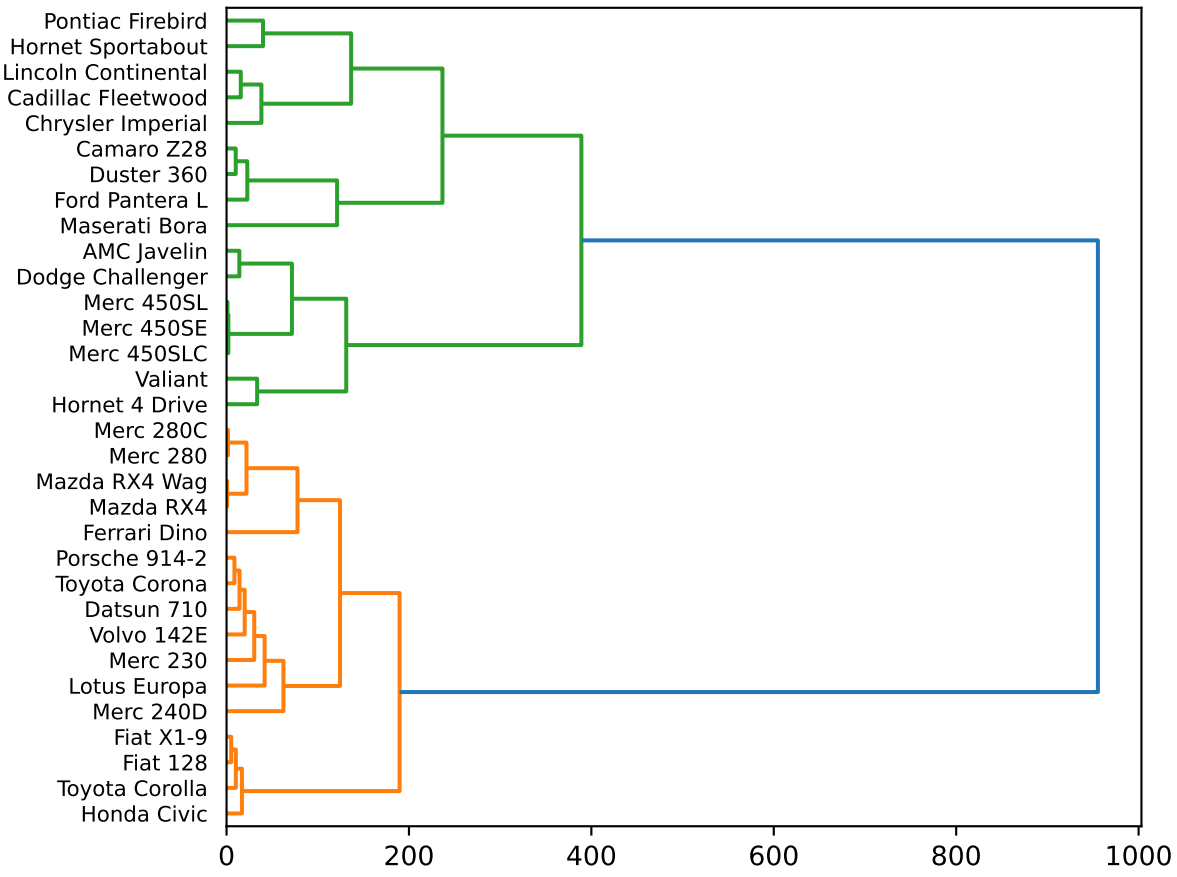
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01a_Dendrogram1.py, and the XML code in the file D01a_Dendrogram1.2D.xml.

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01b_Dendrogram2.py, and the XML code in the file D01b_Dendrogram2.2D.xml.

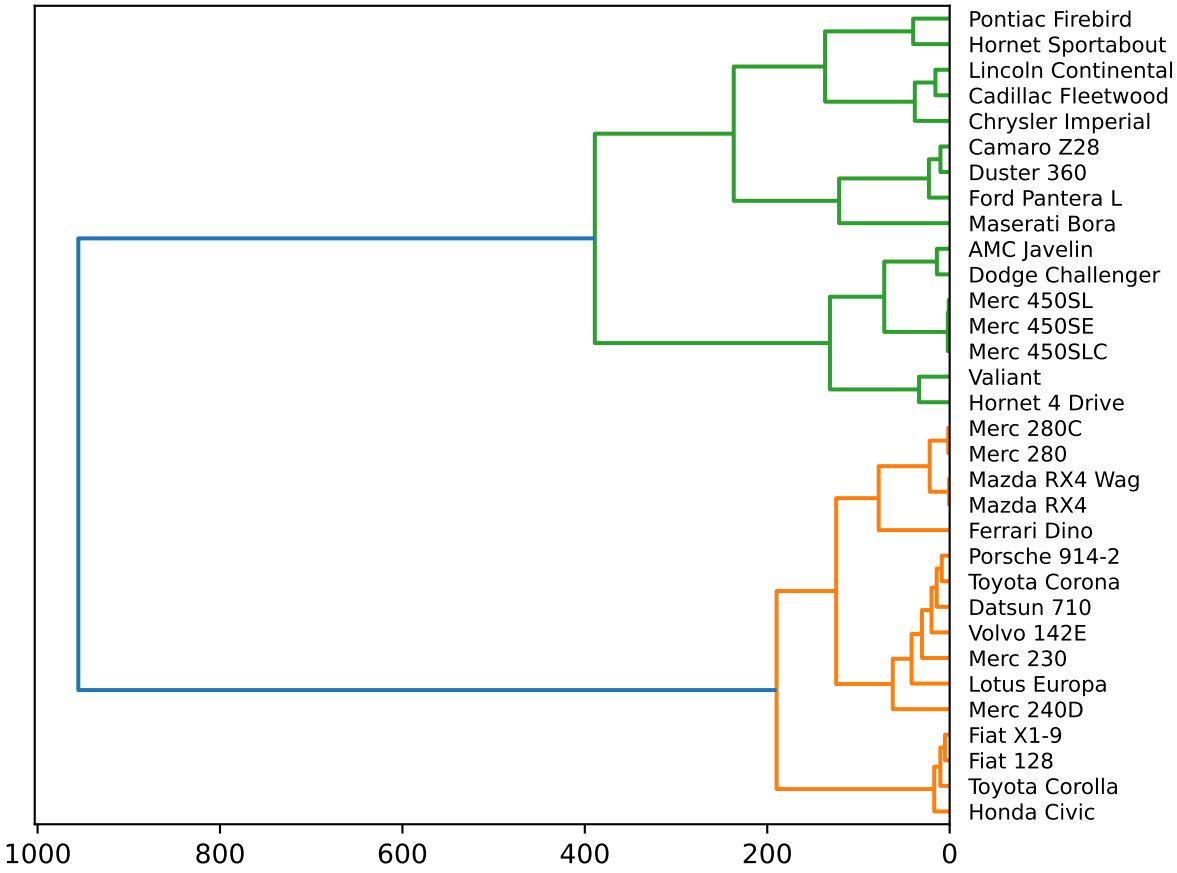
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01c_Dendrogram3.py, and the XML code in the file D01c_Dendrogram3.2D.xml.



Some text

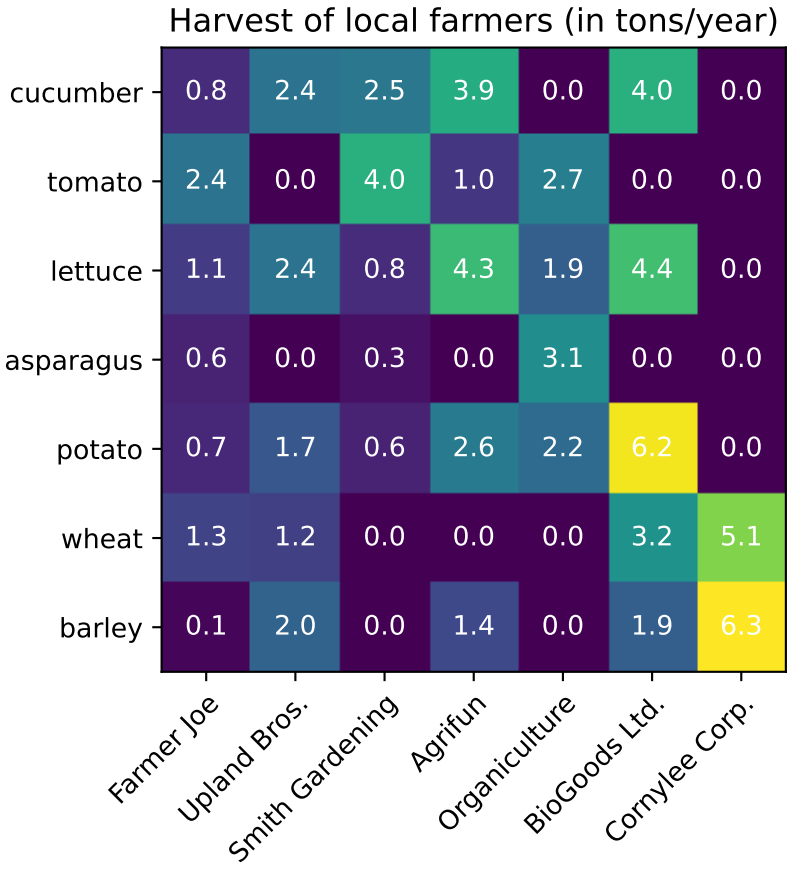

Pontiac Firebird
Hornet Sportabout
Lincoln Continental
Cadillac Fleetwood
Chrysler Imperial
Camaro Z28
Duster 360
Ford Pantera L
Maserati Bora
AMC Javelin
Dodge Challenger
Merc 450SL
Merc 450SE
Merc 450SLC
Valiant
Hornet 4 Drive
Merc 280C
Merc 280
Mazda RX4 Wag
Mazda RX4
Ferrari Dino
Porsche 914-2
Toyota Corona
Datsun 710
Volvo 142E
Merc 230
Lotus Europa
Merc 240D
Fiat X1-9
Fiat 128
Toyota Corolla
Honda Civic
0
200
400
600
800
1000

Some text


Pontiac Firebird
Hornet Sportabout
Lincoln Continental
Cadillac Fleetwood
Chrysler Imperial
Camaro Z28
Duster 360
Ford Pantera L
Maserati Bora
AMC Javelin
Dodge Challenger
Merc 450SL
Merc 450SE
Merc 450SLC
Valiant
Hornet 4 Drive
Merc 280C
Merc 280
Mazda RX4 Wag
Mazda RX4
Ferrari Dino
Porsche 914-2
Toyota Corona
Datsun 710
Volvo 142E
Merc 230
Lotus Europa
Merc 240D
Fiat X1-9
Fiat 128
Toyota Corolla
Honda Civic
1000
800
600
400
200
0

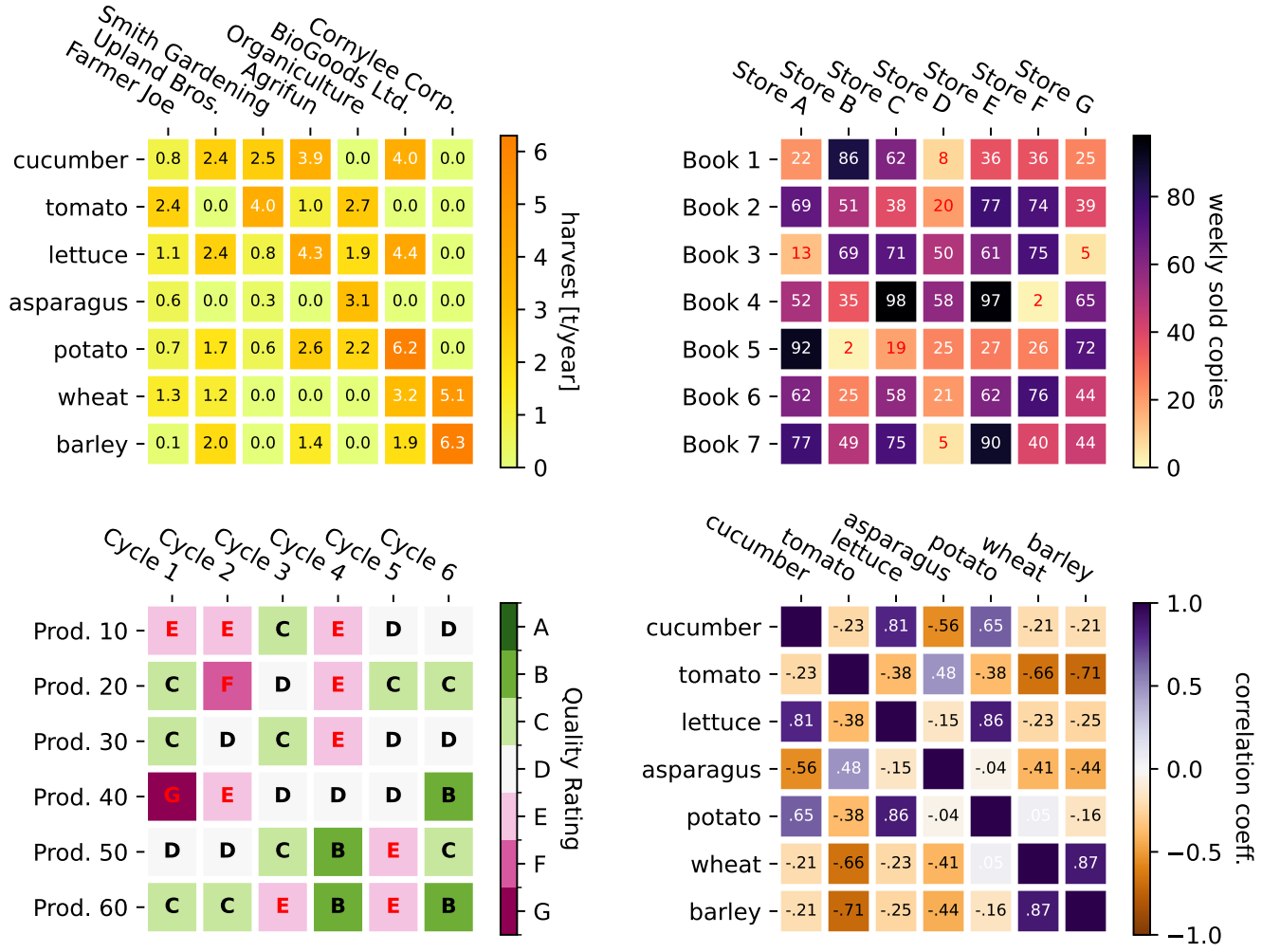
17.10.2 Heatmap, simple

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Heatmap1.py, and the XML code in the file D02_Heatmap1.2D.xml.



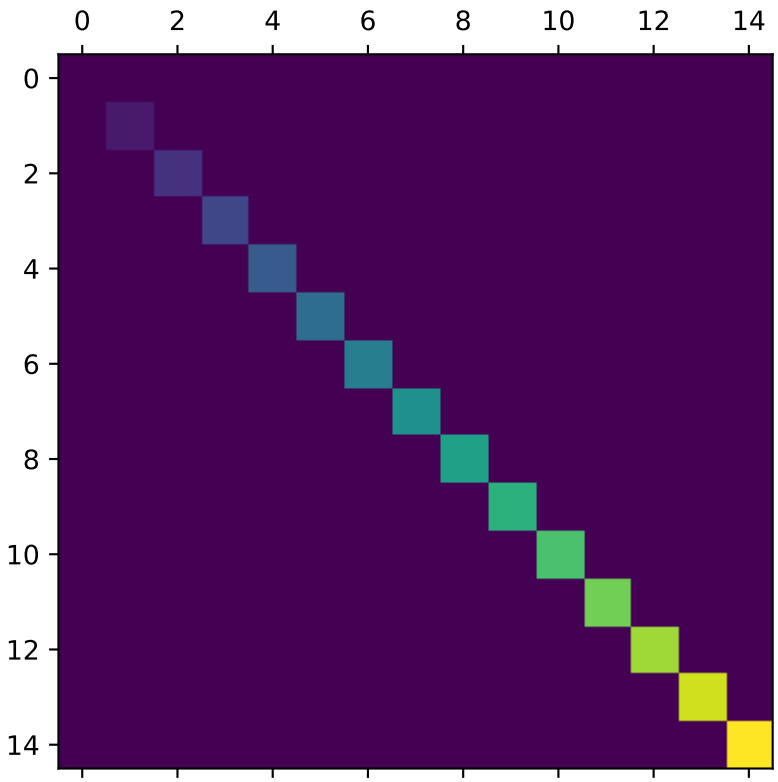
17.10.3 Heatmap, complex

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Heatmap2.py, and the XML code in the file D03_Heatmap2.2D.xml.



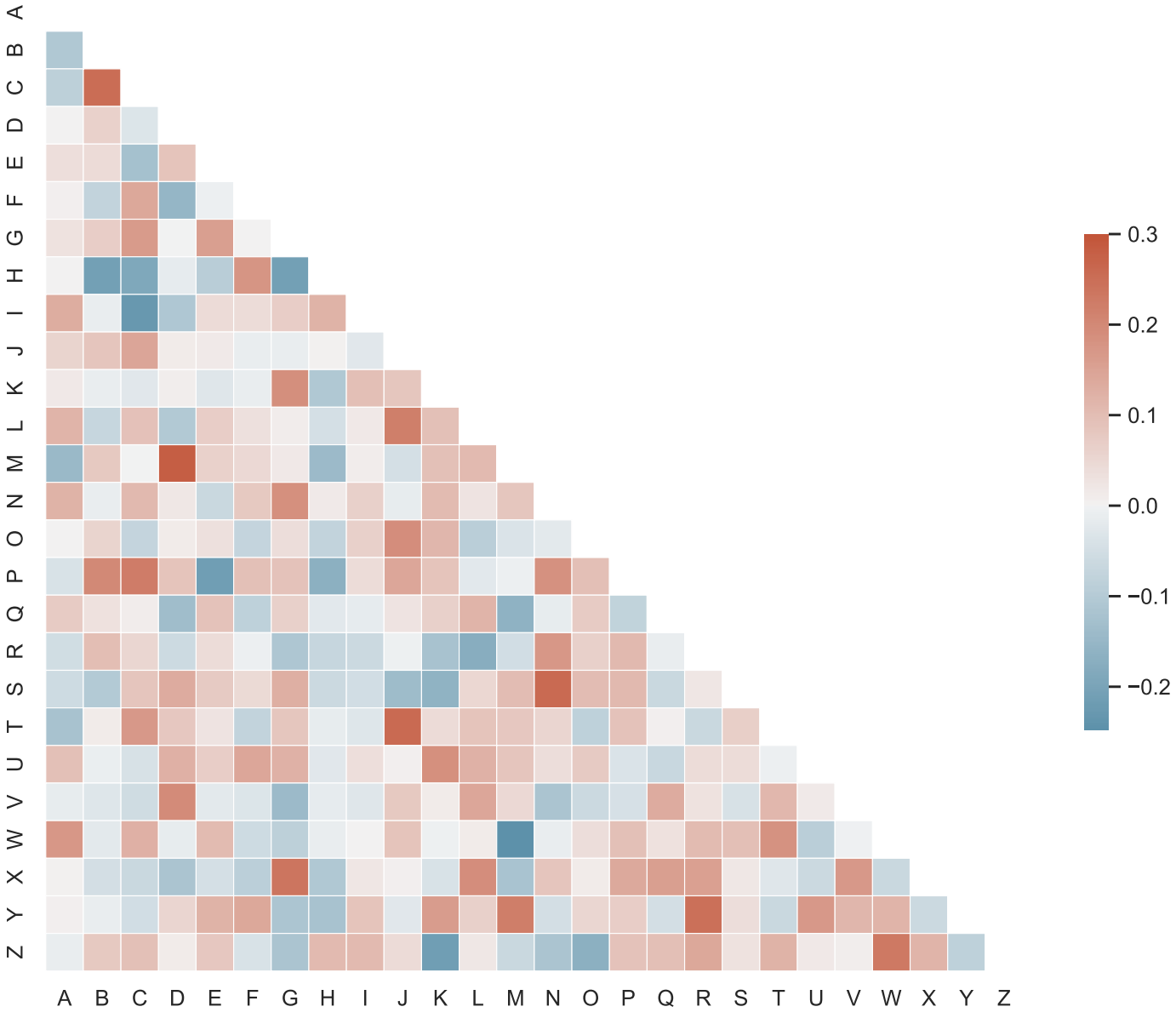
17.10.4 Matshow

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_MatShow.py, and the XML code in the file D04_MatShow.2D.xml.



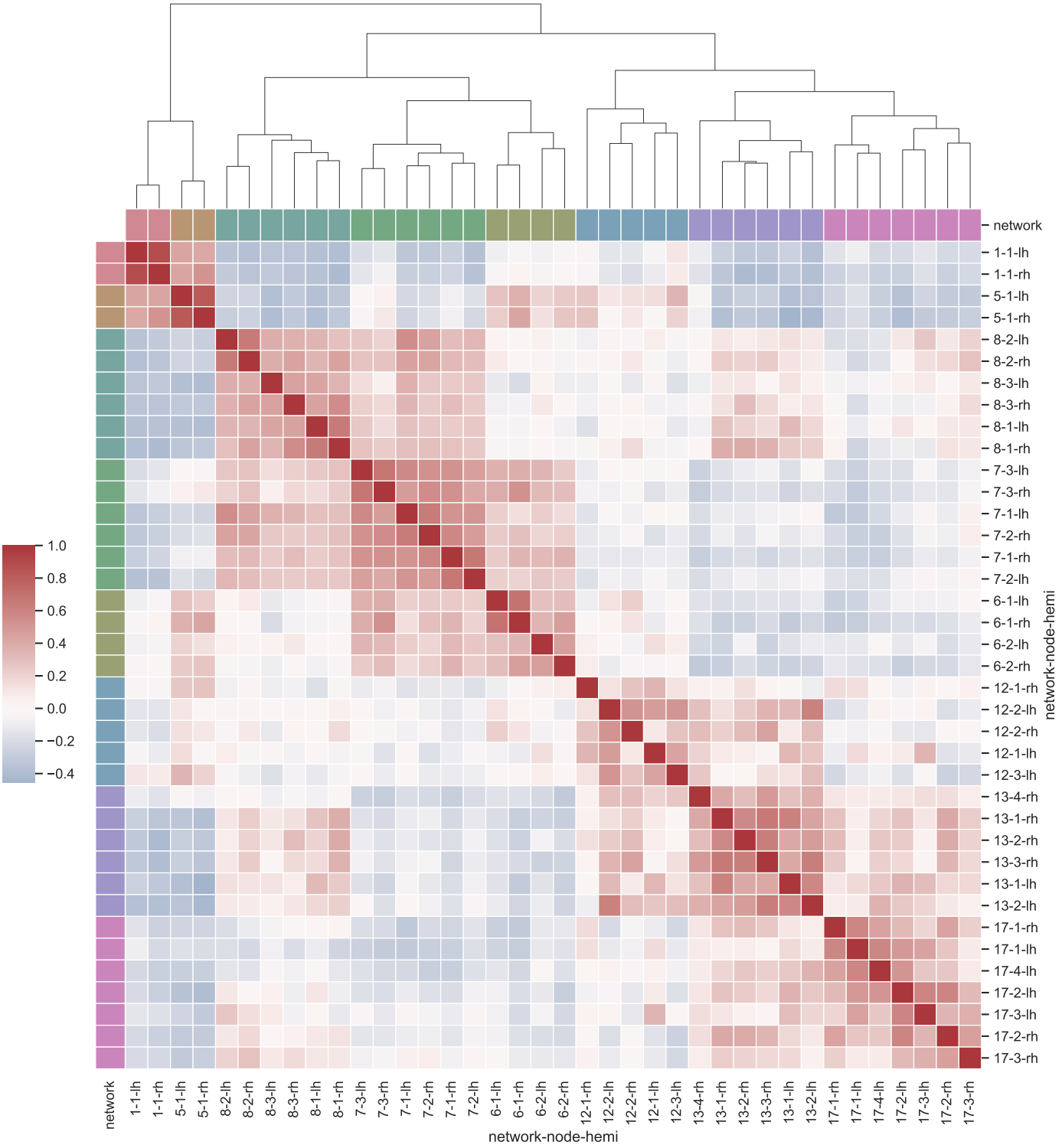
17.10.5 Many pairwise correlations (Seaborn)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_ManyPairwiseCorrelations.py, and the XML code in the file D05_ManyPairwiseCorrelations.2D.xml.



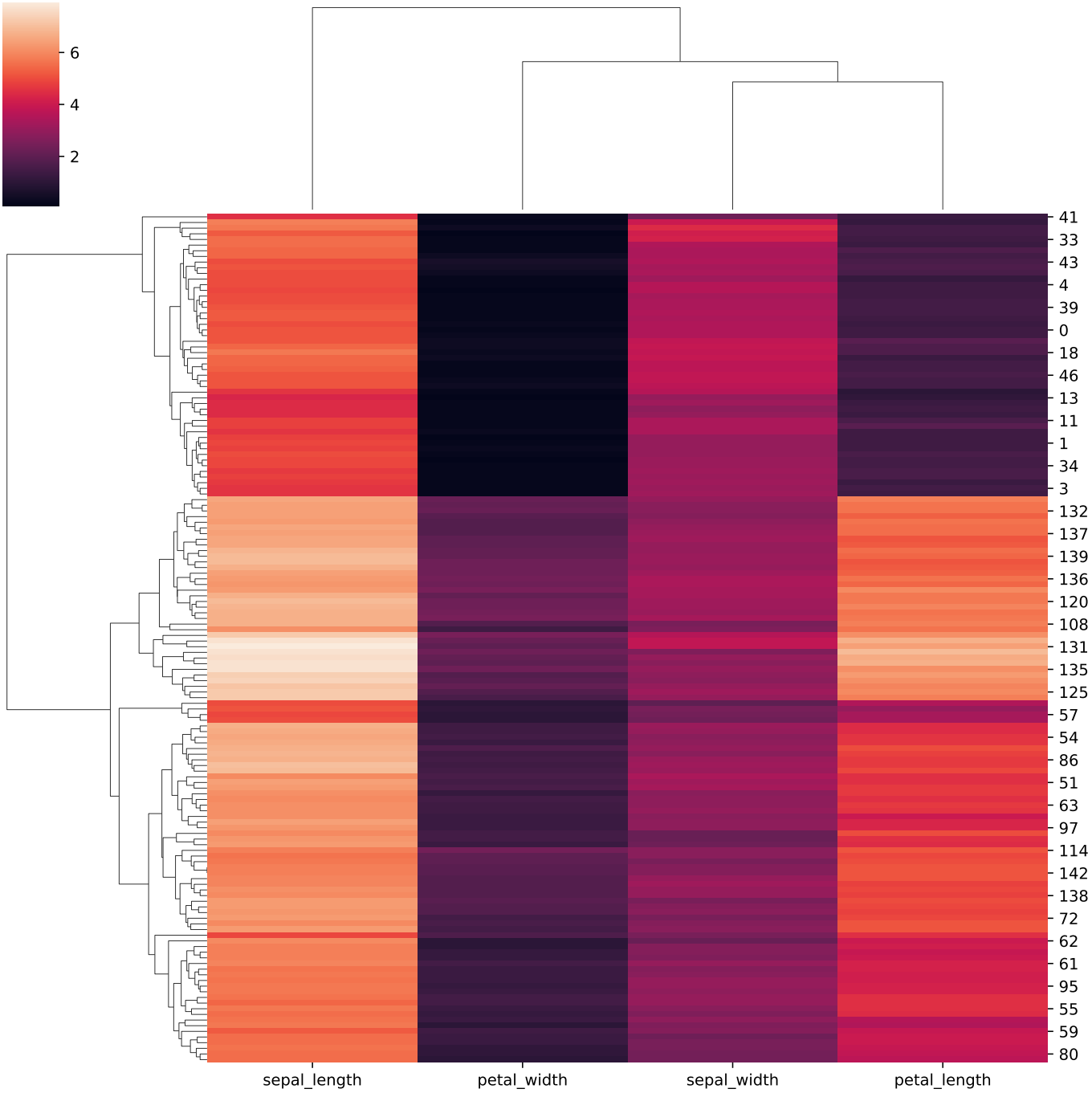
17.10.6 Structured heatmap (Seaborn)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_StructuredHeatmap.py, and the XML code in the file D06_StructuredHeatmap.2D.xml.



17.10.7 Clustermap (Seaborn)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_Clustermap.py, and the XML code in the file D07_Clustermap.2D.xml.



see: https://seaborn.pydata.org/generated/seaborn.clustermap.html

17.11 PlotTable

17.11.1 PlotTable1

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_PlotTable1.py, and the XML code in the file D01_PlotTable1.2D.xml.

index	A	B	C	D	E
0	0.03	0.23	0.61	0.9	0.08
1	0.16	0.53	0.33	0.62	0.28
2	0.31	0.37	0.01	0.0	0.3
3	0.88	0.43	0.31	0.49	0.58
4	0.57	0.73	0.32	0.89	0.31

17.11.2 PlotTable2

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_PlotTable2.py, and the XML code in the file D02_PlotTable2.2D.xml.

		Team Rating			Group Stage Chances			Knockout Stage Chances				
Team	Group	SPI	OFF	DEF	1st Place	2nd Place	3rd Place	Make Rd Of 16	Make Quarters	Make Semis	Make Finals	Win World Cup
USA	F	98.3	5.5	0.6	83%	17%	-	✓	78%	47%	35%	24%
France	A	96.3	4.3	0.5	>99%	<1%	<1%	✓	78%	42%	30%	19%
Germany	B	93.8	4.0	0.7	98%	2%	-	✓	89%	48%	28%	12%
Canada	E	93.5	3.7	0.6	39%	61%	-	✓	59%	36%	20%	9%
England	D	91.9	3.5	0.6	71%	29%	-	✓	69%	43%	16%	8%
Netherlands	E	92.7	3.9	0.7	61%	39%	-	✓	59%	37%	19%	8%
Australia	C	92.8	4.2	0.9	13%	54%	34%	>99%	54%	26%	10%	5%
Sweden	F	88.4	3.0	0.6	17%	83%	-	✓	47%	20%	10%	4%
Japan	D	90.3	3.8	0.9	29%	63%	8%	✓	46%	27%	9%	4%
Brazil	C	89.5	3.6	0.9	26%	22%	52%	>99%	43%	17%	7%	3%
Spain	B	86.5	3.1	0.8	1%	70%	29%	>99%	31%	11%	5%	2%
Norway	A	83.7	3.0	0.9	<1%	94%	5%	>99%	44%	18%	4%	2%
China	B	82.7	2.7	0.8	<1%	29%	70%	93%	34%	11%	3%	<1%
Italy	C	76.1	3.0	1.3	61%	24%	15%	✓	37%	9%	2%	<1%
New Zealand	E	77.6	2.8	1.2	-	-	48%	40%	10%	4%	<1%	<1%
Nigeria	A	71.7	2.4	1.1	<1%	5%	91%	50%	13%	3%	<1%	<1%
Cameroon	E	65.8	2.1	1.2	-	-	52%	23%	4%	1%	<1%	<1%
South Korea	A	76.4	2.7	1.1	-	<1%	4%	2%	<1%	<1%	<1%	<1%
Scotland	D	54.0	2.0	1.7	-	-	53%	52%	3%	<1%	<1%	<1%
Argentina	D	39.2	1.5	1.9	-	8%	40%	25%	<1%	<1%	<1%	<1%
Chile	F	46.5	1.8	1.8	-	-	66%	15%	<1%	<1%	<1%	-
South Africa	B	56.7	1.7	1.3	-	-	<1%	<1%	<1%	<1%	<1%	-
Thailand	F	40.7	2.0	2.4	-	-	34%	2%	<1%	<1%	-	-
Jamaica	C	53.5	2.5	2.1	-	-	<1%	<1%	<1%	-	-	-

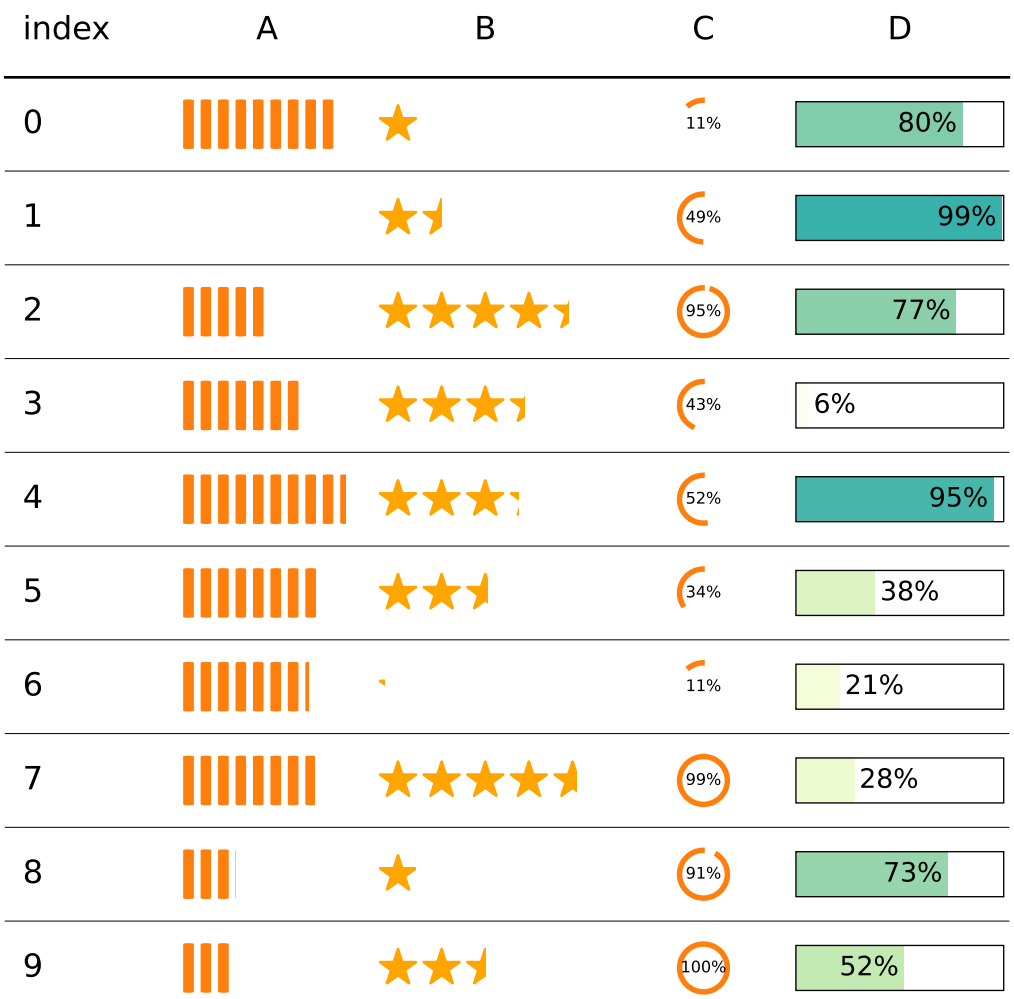
17.11.3 PlotTable3

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_PlotTable3.py, and the XML code in the file D03_PlotTable3.2D.xml.

		Games	W	D	L	Goals		Points
1	Bayern Munich	15	10	4	1	49:13	+36	34
2	RB Leipzig	15	8	4	3	30:21	+9	28
3	1. FC Union Berlin	14	8	3	3	23:16	+7	27
4	SC Freiburg	14	8	3	3	21:16	+5	27
5	Eintracht Frankfurt	15	8	2	4	31:23	+8	26
6	Borussia Dortmund	15	8	1	6	25:21	+4	25
7	VfL Wolfsburg	15	6	5	4	24:20	+4	23
8	Borussia Monchengladbach	14	6	4	5	28:24	+4	22
9	Werder Bremen	15	6	3	6	25:27	-2	21
10	TSG Hoffenheim	15	5	3	7	22:22	+0	18
11	Bayer Leverkusen	15	5	3	7	25:26	-1	18
12	Mainz	14	5	3	6	18:23	-5	18
13	FC Cologne	15	4	5	6	21:29	-8	17
14	FC Augsburg	15	4	3	8	18:26	-8	15
15	Hertha Berlin	15	3	5	7	19:22	-3	14
16	VfB Stuttgart	15	3	5	7	18:27	-9	14
17	VfL Bochum	15	4	1	10	14:36	-22	13
18	Schalke 04	15	2	3	10	13:32	-19	9

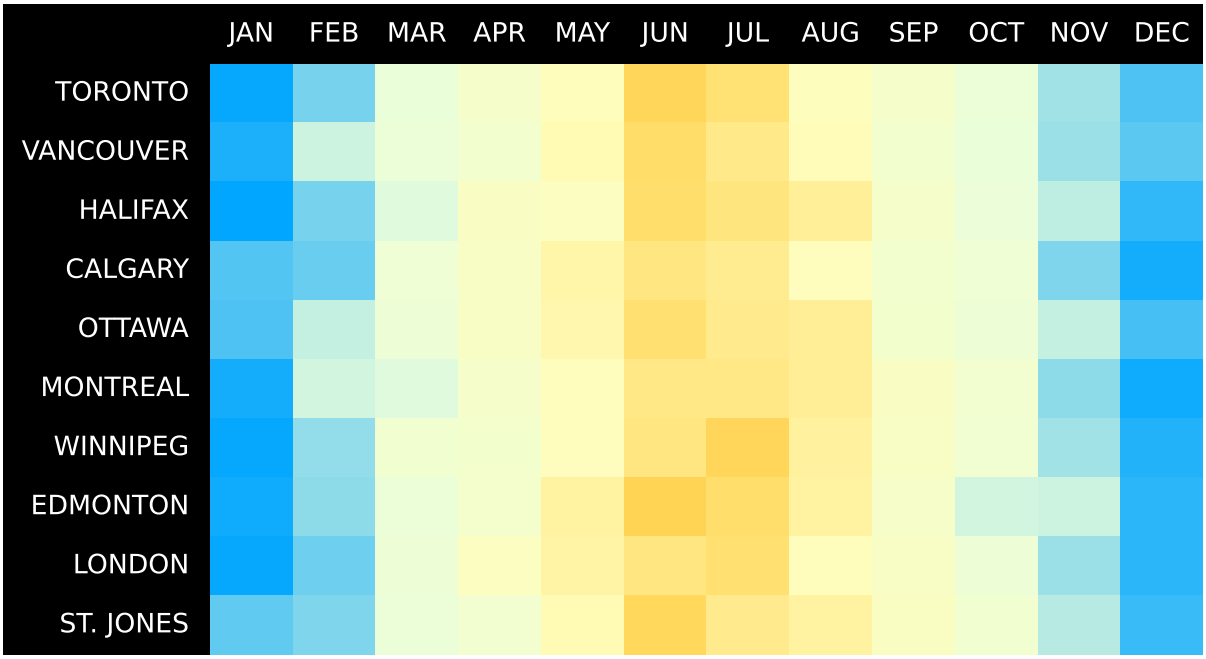
17.11.4 PlotTable4

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_PlotTable4.py, and the XML code in the file D04_PlotTable4.2D.xml.



17.11.5 PlotTable5

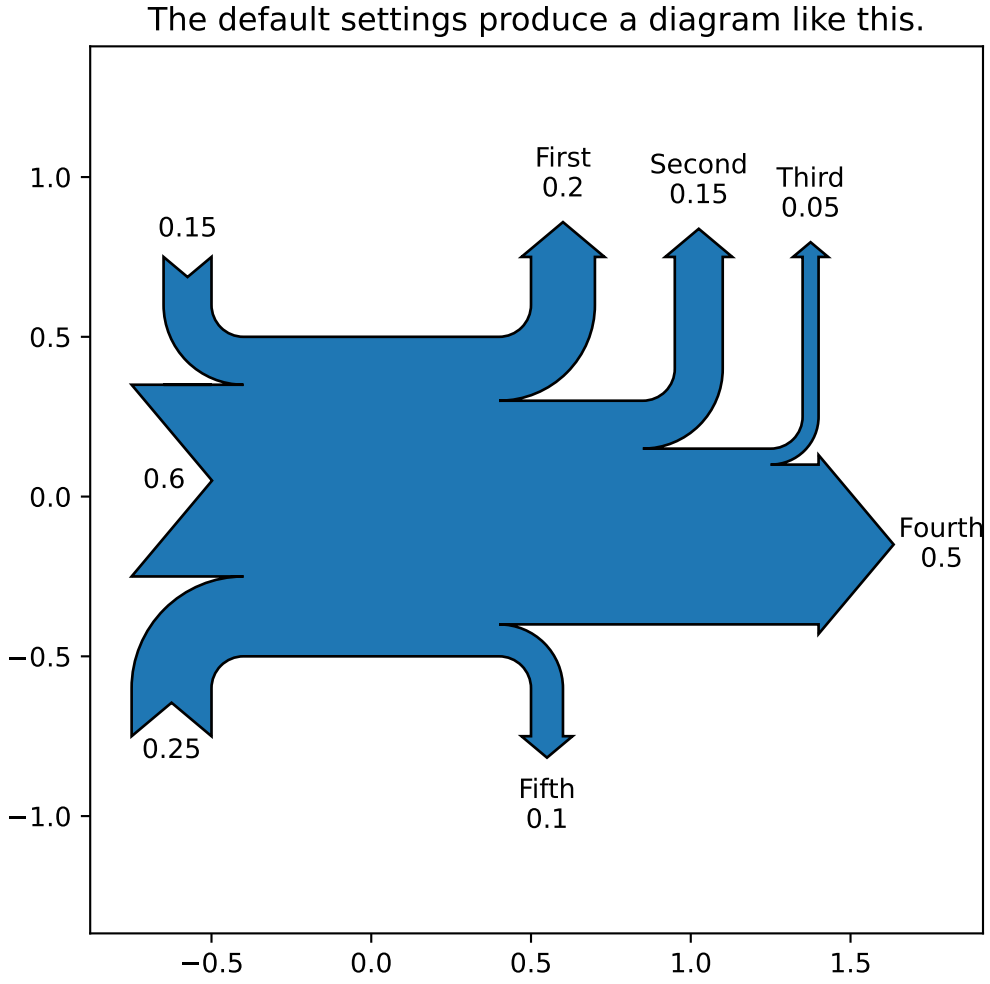
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_PlotTable5.py, and the XML code in the file D05_PlotTable5.2D.xml.



17.12 Flow and connections

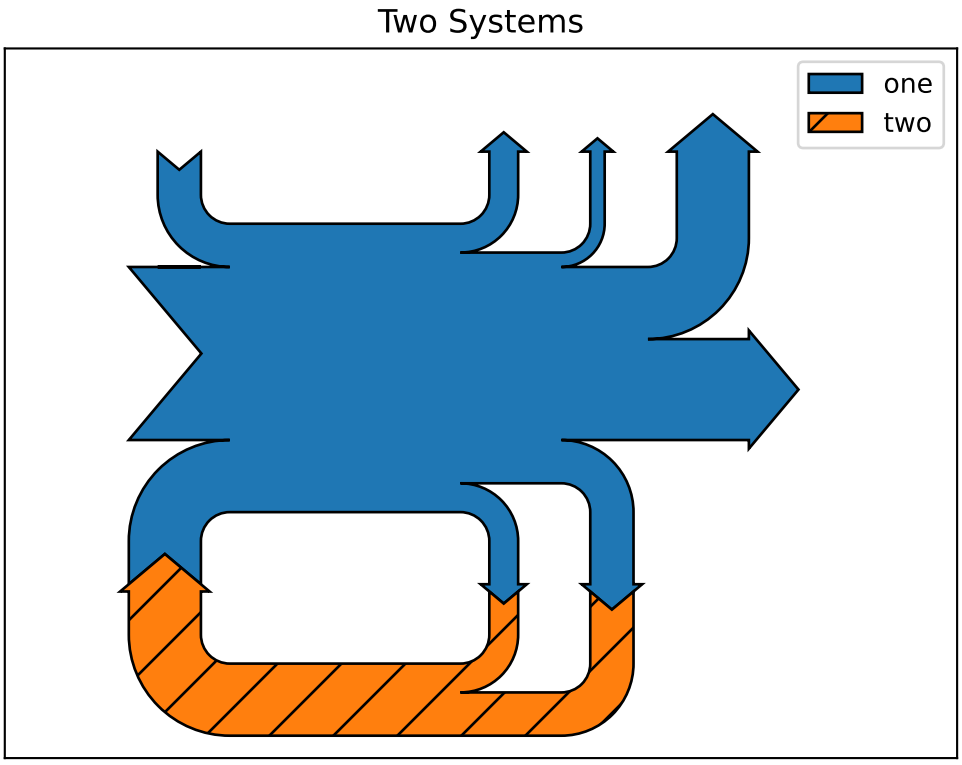
17.12.1 Sankey Default

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Sankey_Default.py, and the XML code in the file D01_Sankey_Default.2D.xml.



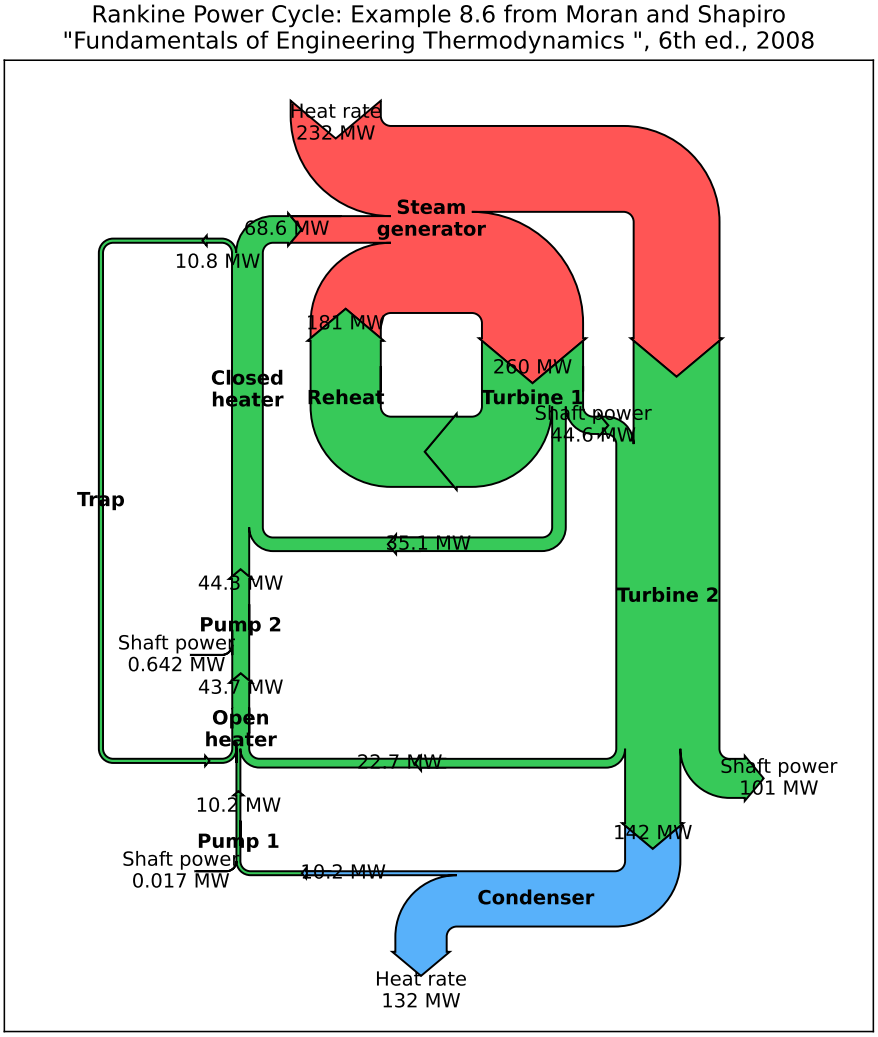
17.12.2 Sankey 2 Systems

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Sankey_TwoSystems.py, and the XML code in the file D02_Sankey_TwoSystems.2D.xml.



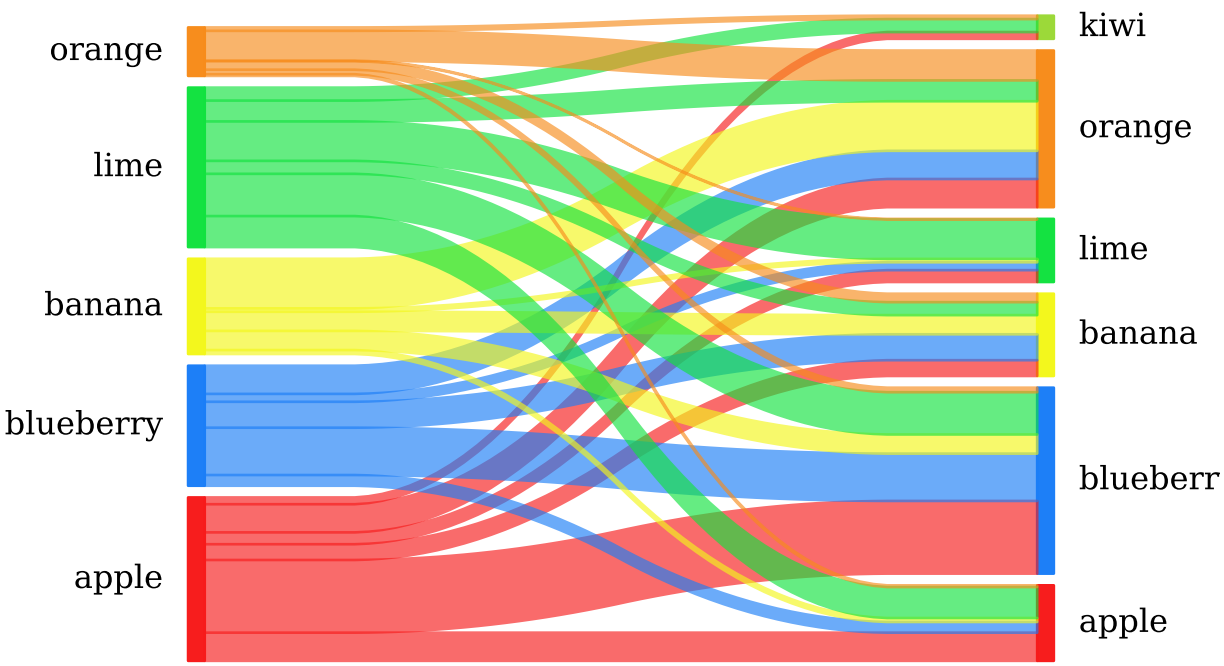
17.12.3 Sankey Rankine

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Sankey_Rankine.py, and the XML code in the file D03_Sankey_Rankine.2D.xml.



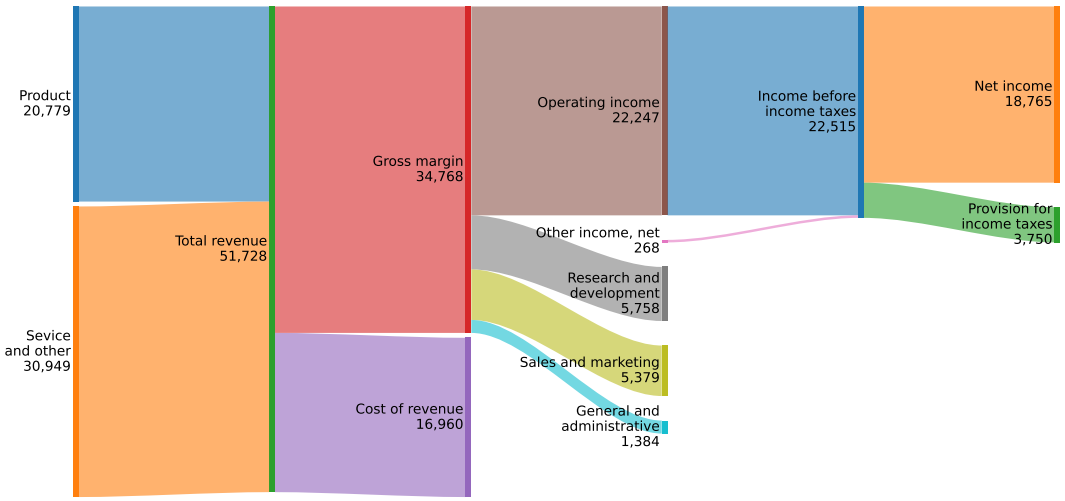
17.12.4 Sankey One 2 one

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Sankey_One2one.py, and the XML code in the file D04_Sankey_One2one.2D.xml.



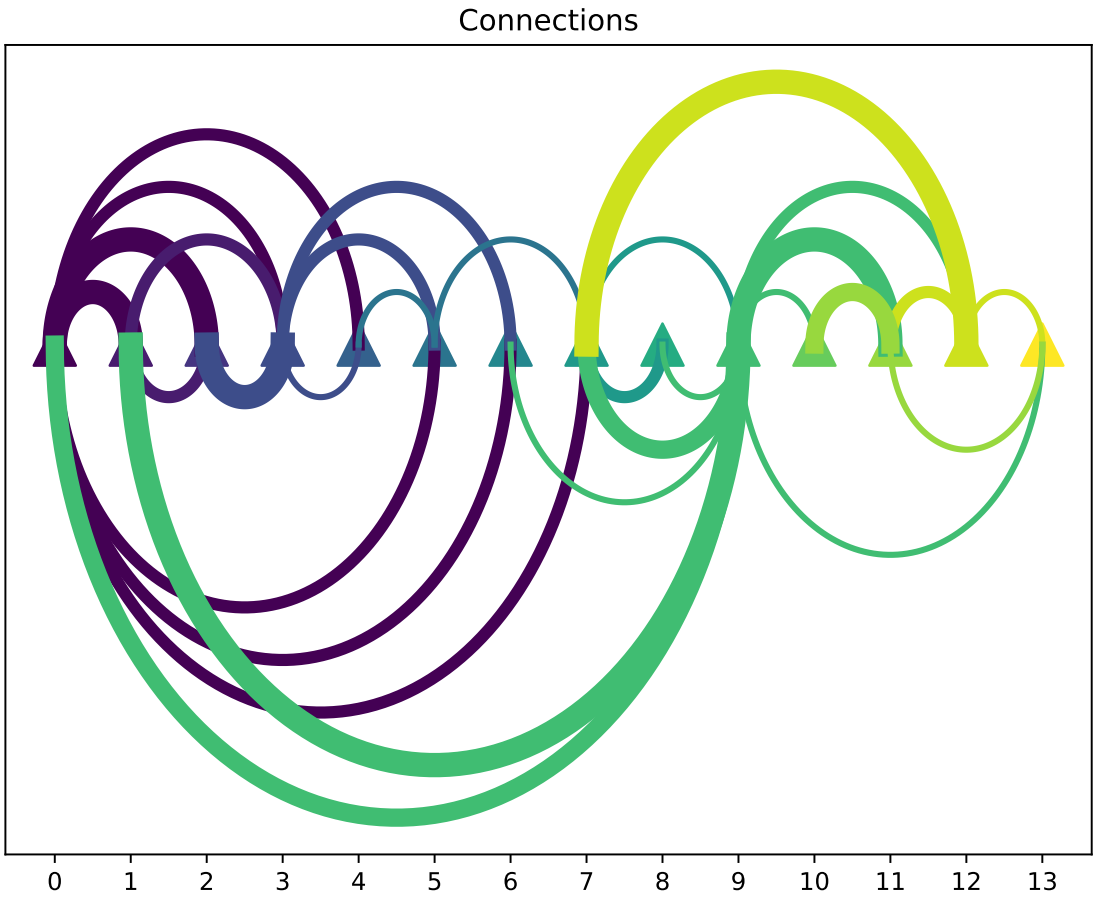
17.12.5 Sankey Flow

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_SankeyFlow.py, and the XML code in the file D05_SankeyFlow.2D.xml.



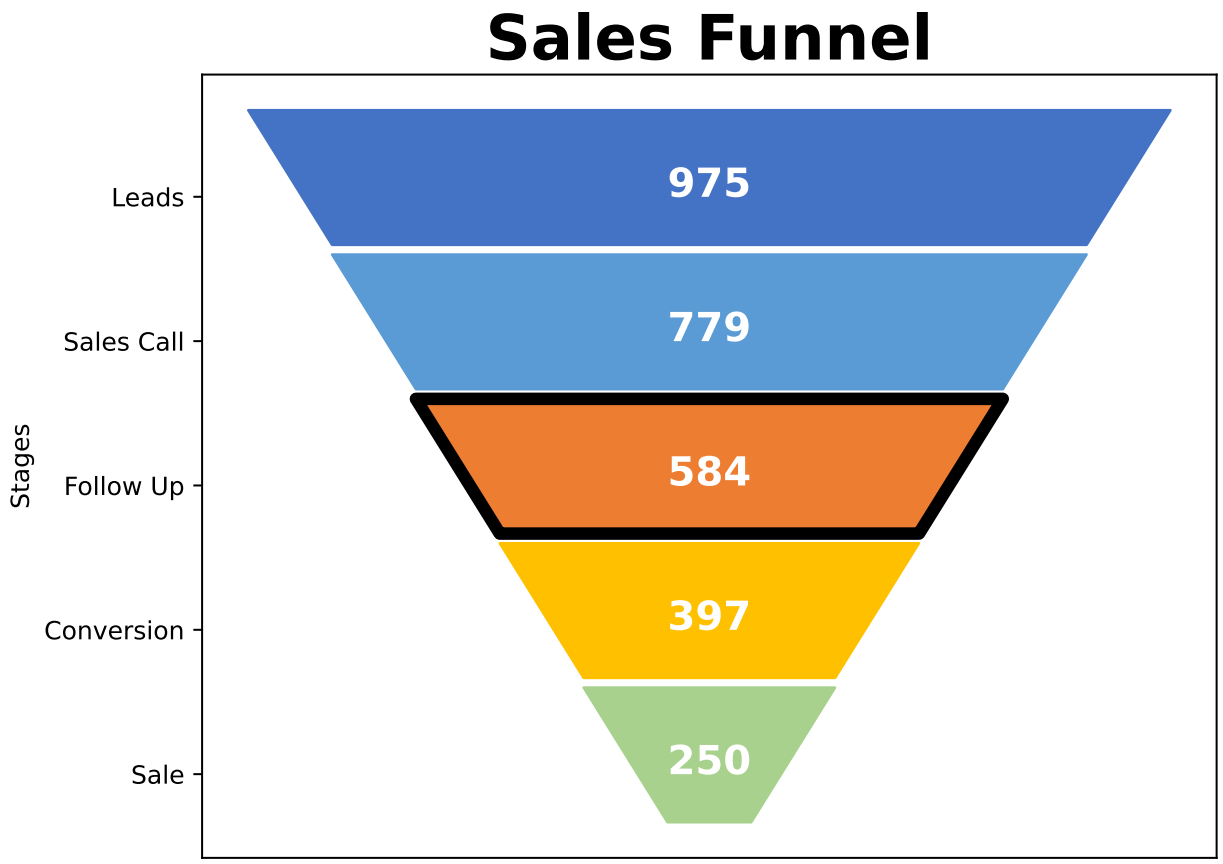
17.12.6 Arcplot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_ArcPlot.py, and the XML code in the file D06_ArcPlot.2D.xml.



17.12.7 Funnel Chart

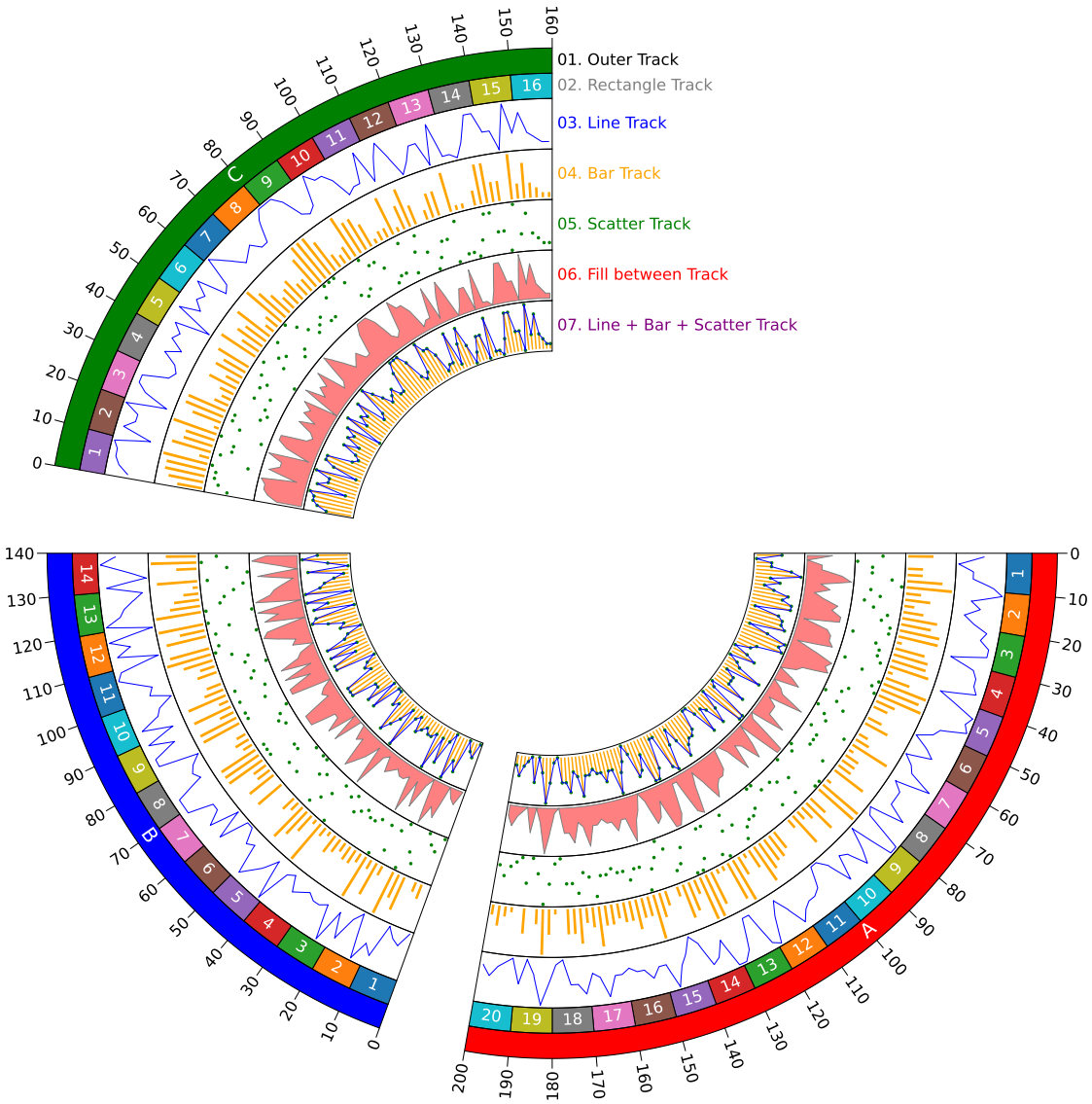
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_FunnelChart.py, and the XML code in the file D07_FunnelChart.2D.xml.



17.13 Circular plots and flows

17.13.1 Circular track plotting

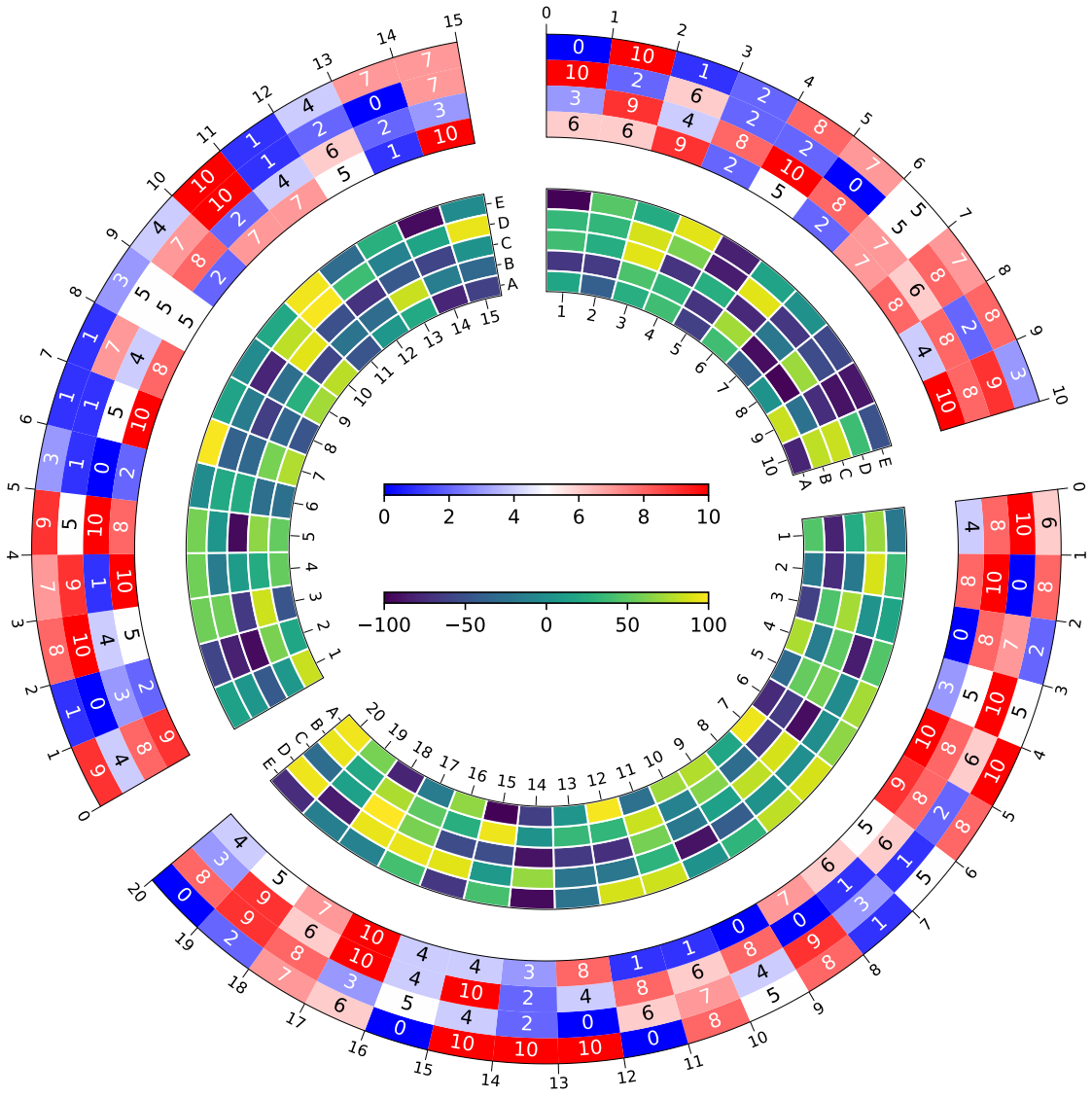
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_CircularTrackplotting.py, and the XML code in the file D01_CircularTrackplotting.2D.xml.



Some text

17.13.2 Circular heatmap

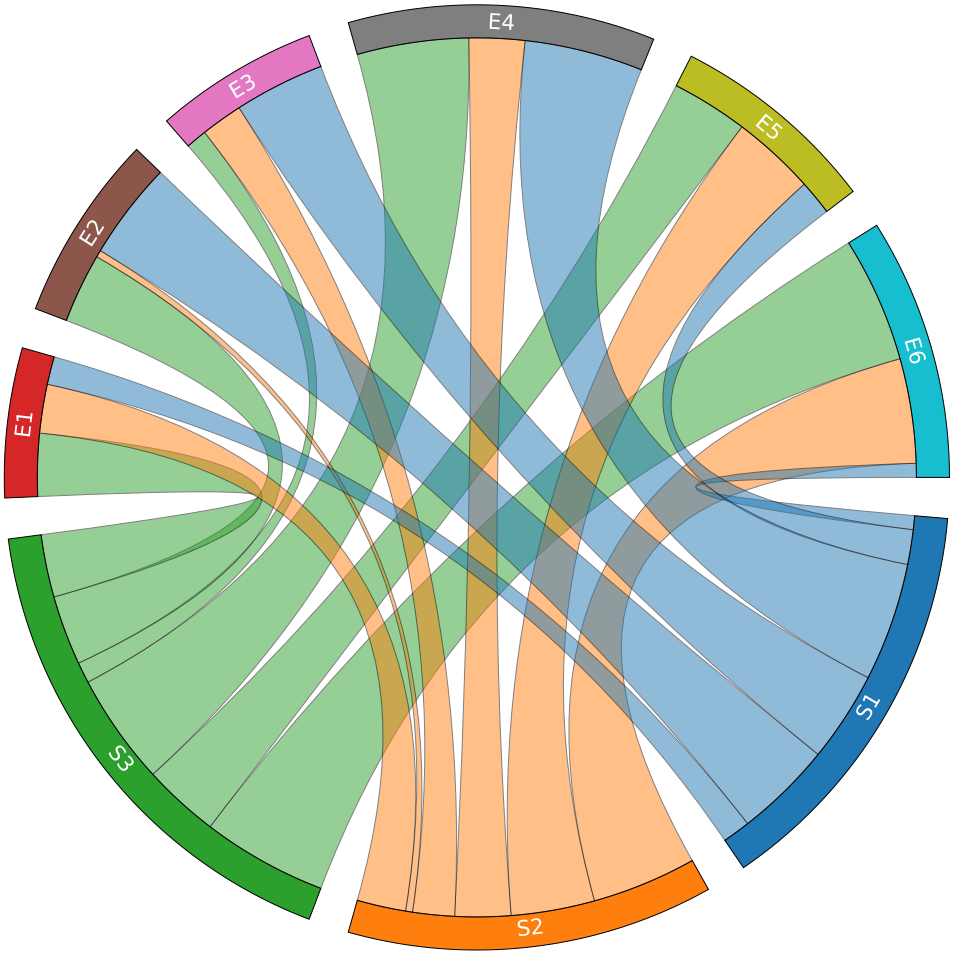
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_CircularHeatmap.py, and the XML code in the file D02_CircularHeatmap.2D.xml.



Some text

17.13.3 Chord diagram (matrix 3x6)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_ChordDiagramMatrix3x6.py, and the XML code in the file D03_ChordDiagramMatrix3x6.2D.xml.



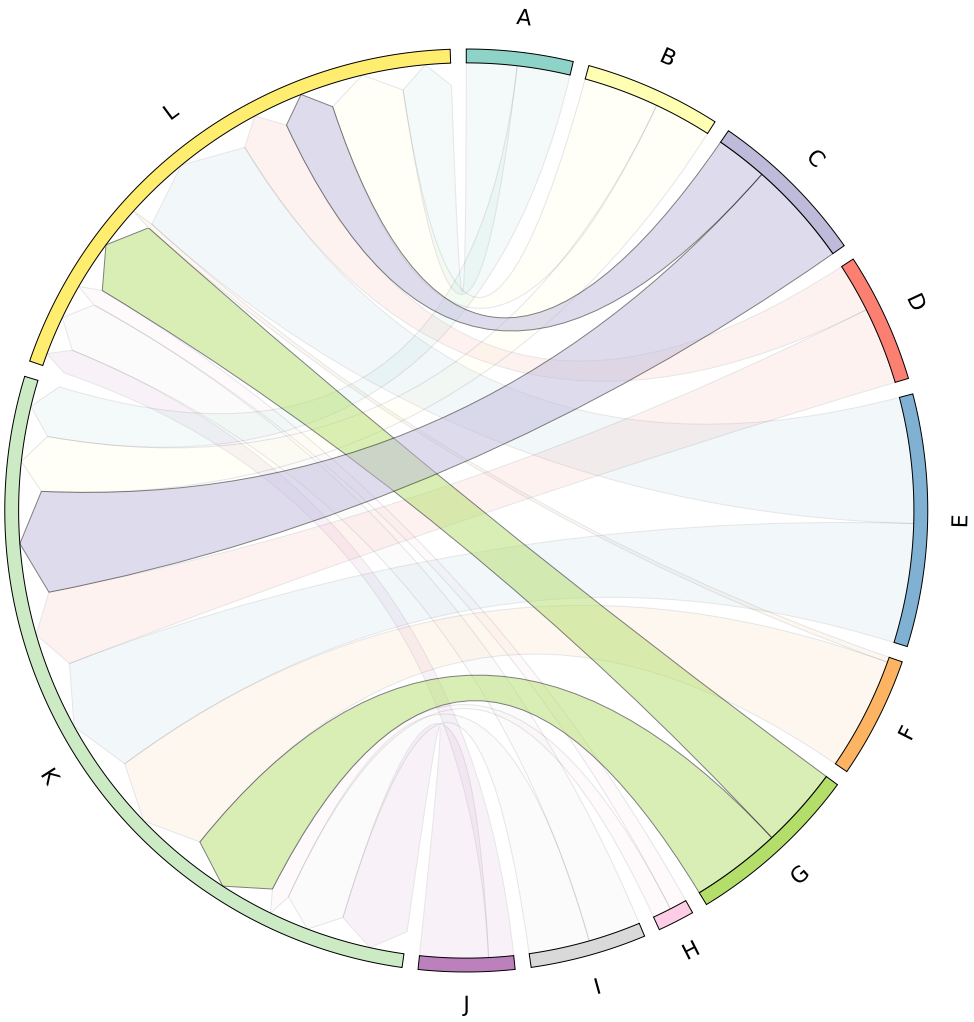
Some text

17.13.4 Chord diagram (matrix10x10)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_ChordDiagramMatrix10x10.py, and the XML code in the file D04_ChordDiagramMatrix10x10.2D.xml.

17.13.5 Chord diagram (from-to table)

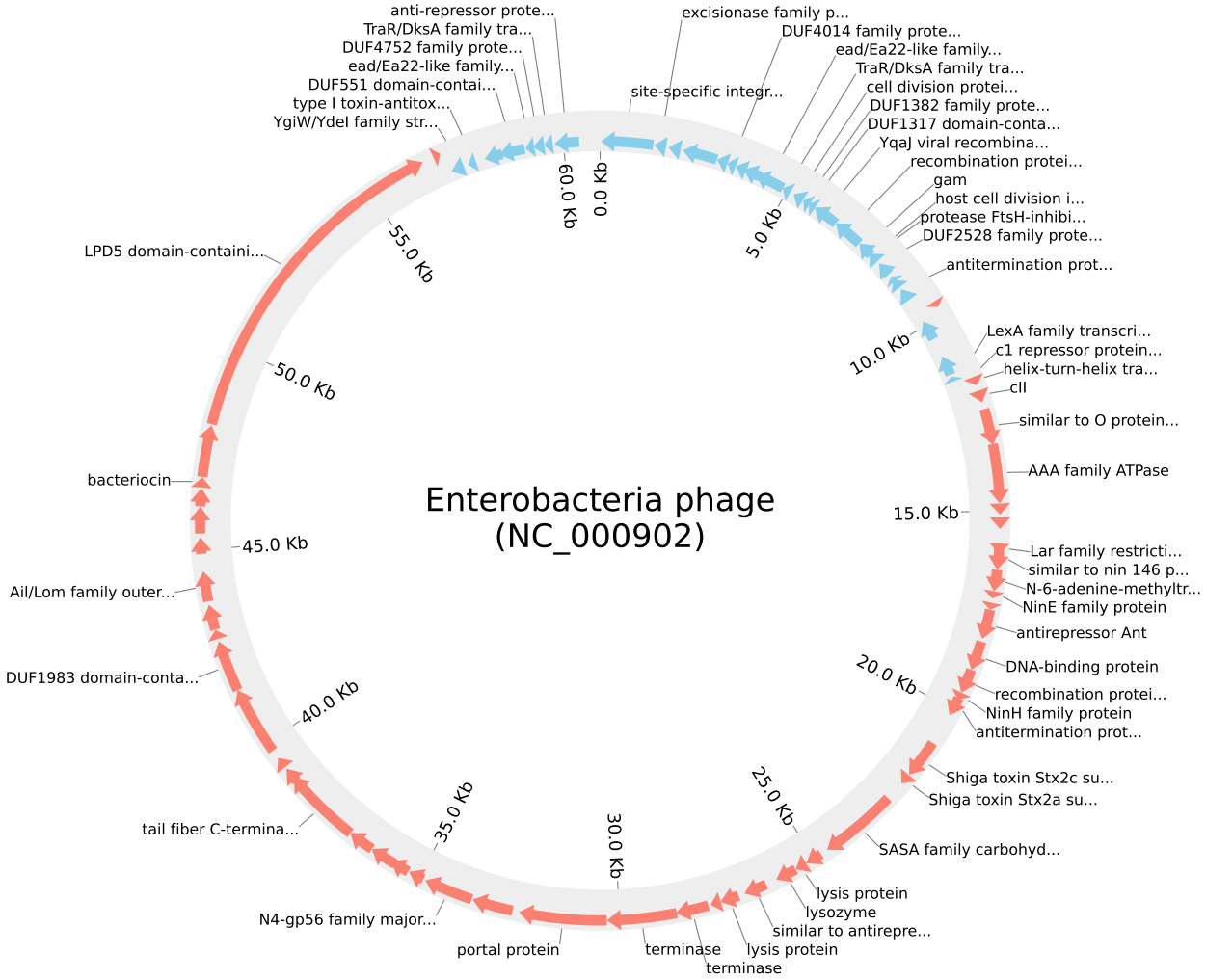
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_ChordDiagramFromTo.py, and the XML code in the file D05_ChordDiagramFromTo.2D.xml.



Some text

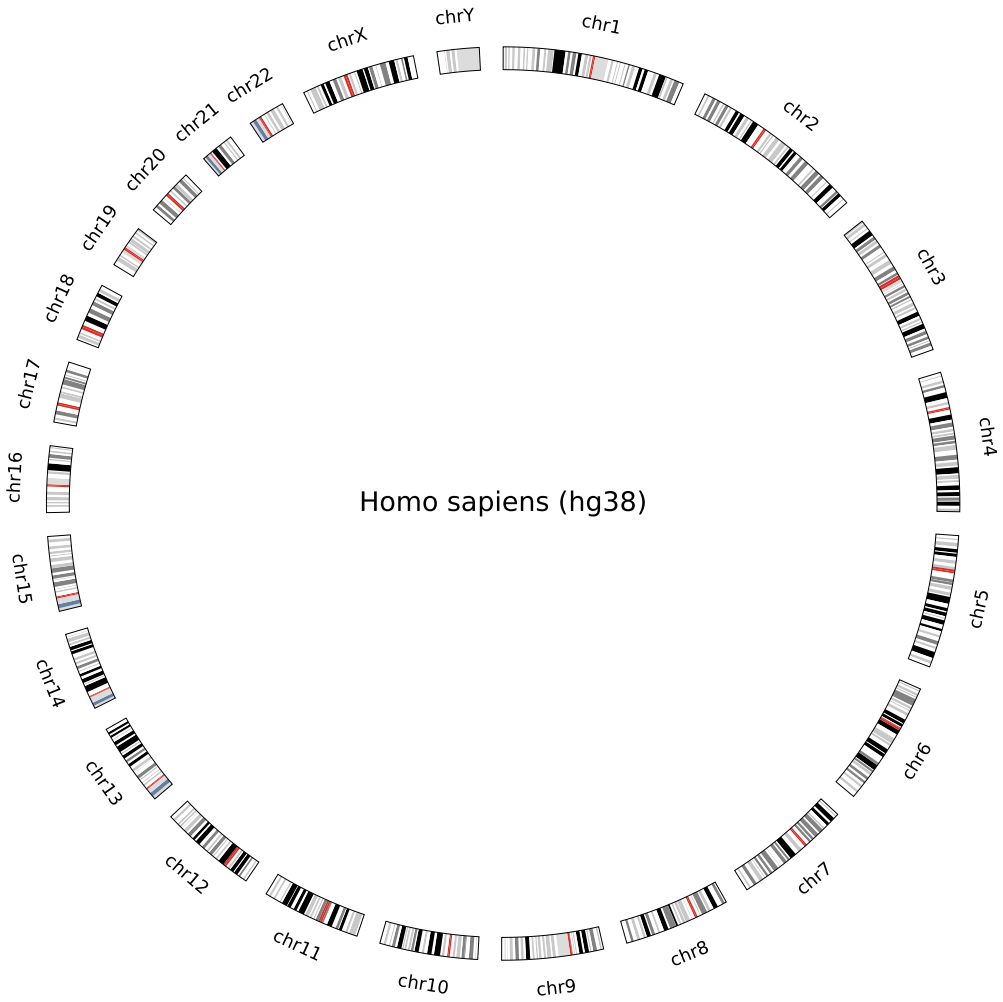
17.13.6 Circos plot: genom of a phage

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_CircosPlotGenomPhage.py, and the XML code in the file D06_CircosPlotGenomPhage.2D.xml.



17.13.7 Circos plot: genom of homo sapiens

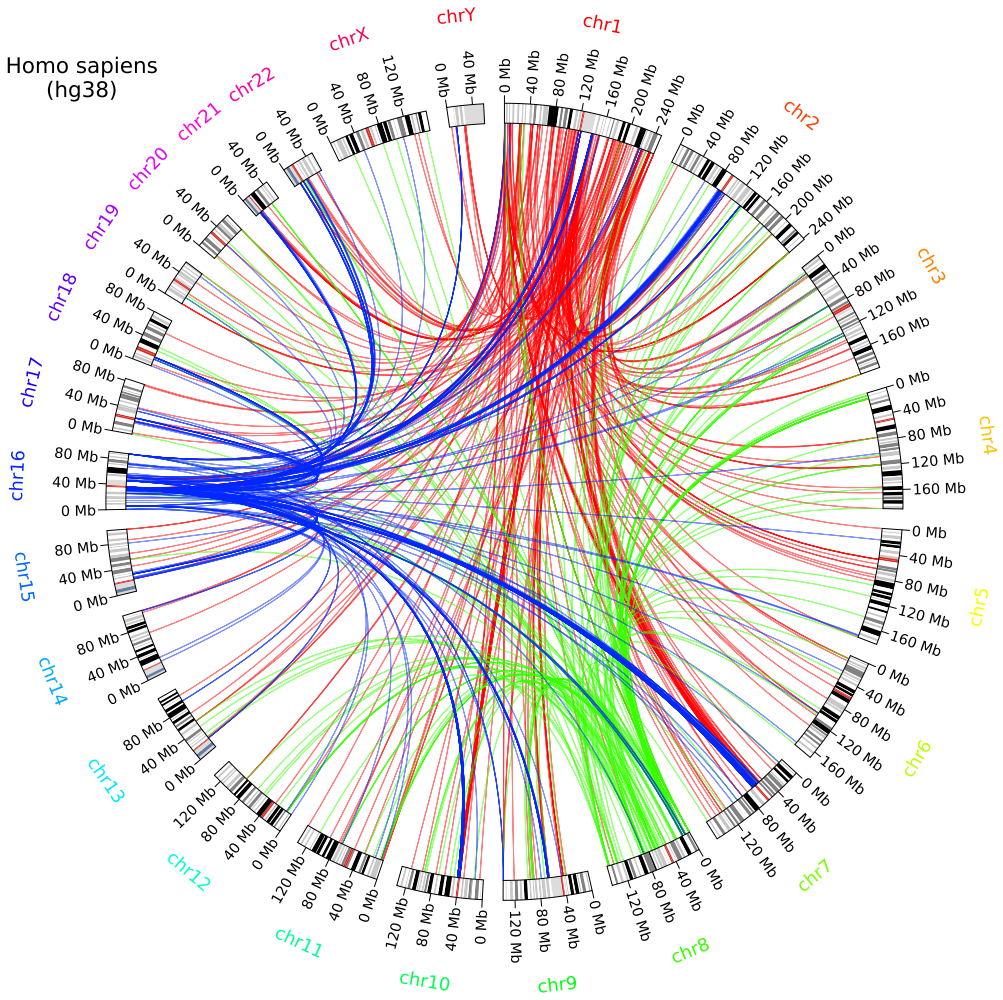
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_CircosPlotGenomHomoSapiens.py, and the XML code in the file D07_CircosPlotGenomHomoSapiens.2D.xml.



Some text

17.13.8 Circos plot: chromosome links

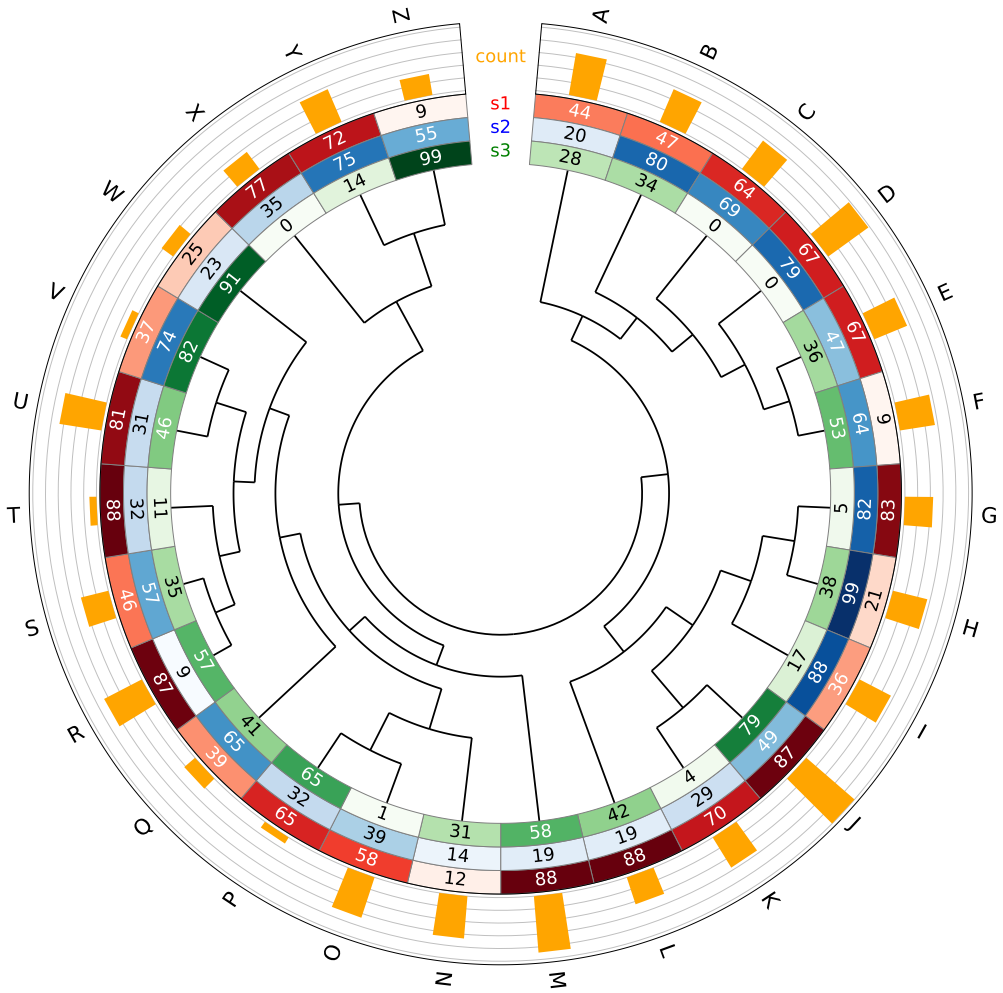
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_CircosPlotChromosomeLinks.py, and the XML code in the file D08_CircosPlotChromosomeLinks.2D.xml.



Some text

17.13.9 Philogenetic tree: bars

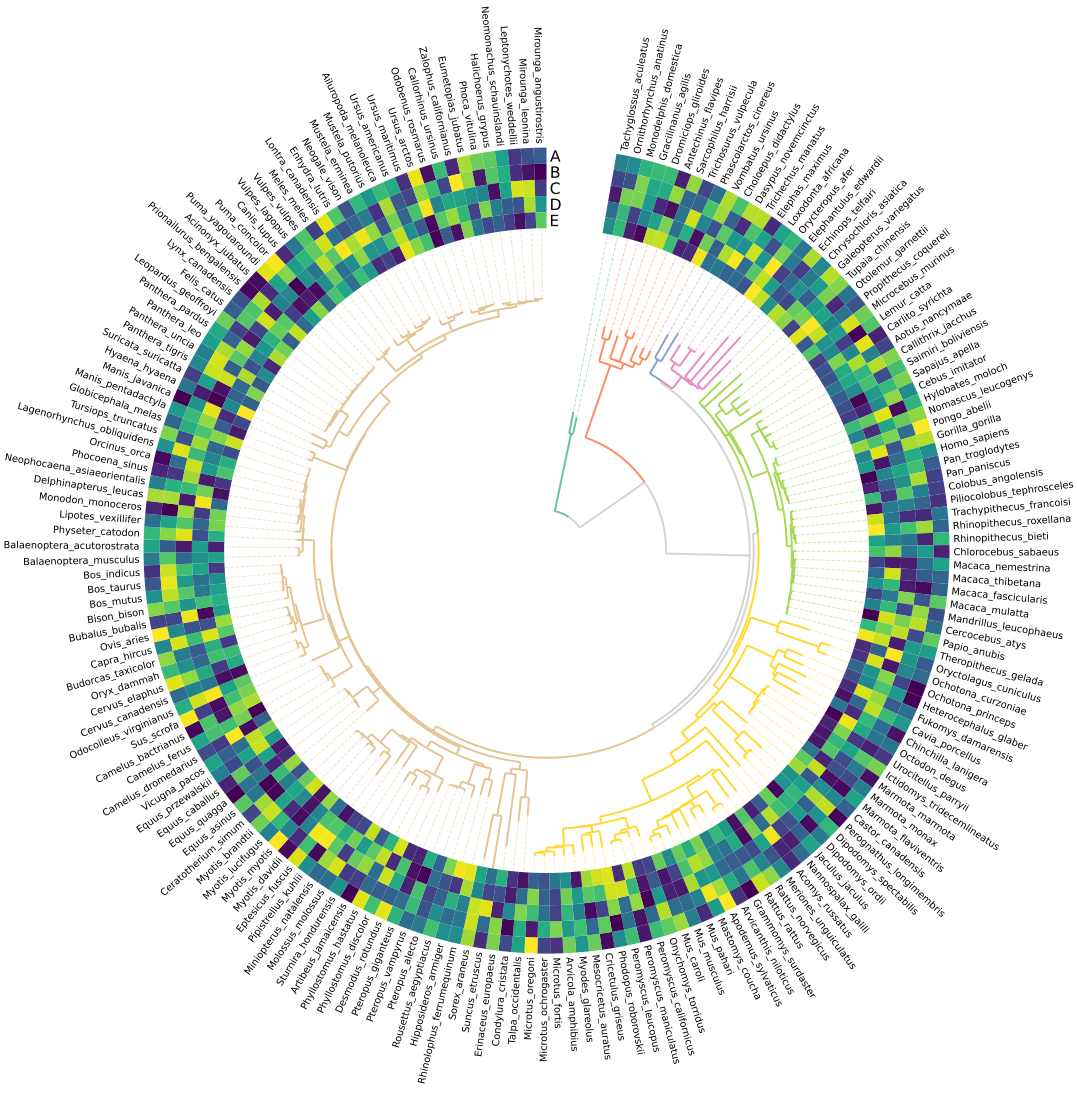
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_PhilogeneticTreeBars.py, and the XML code in the file D09_PhilogeneticTreeBars.2D.xml.



Some text

17.13.10 Philogenetic tree: heatmap

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_PhilogeneticTreeHeatmap.py, and the XML code in the file D10_PhilogeneticTreeHeatmap.2D.xml.



Some text

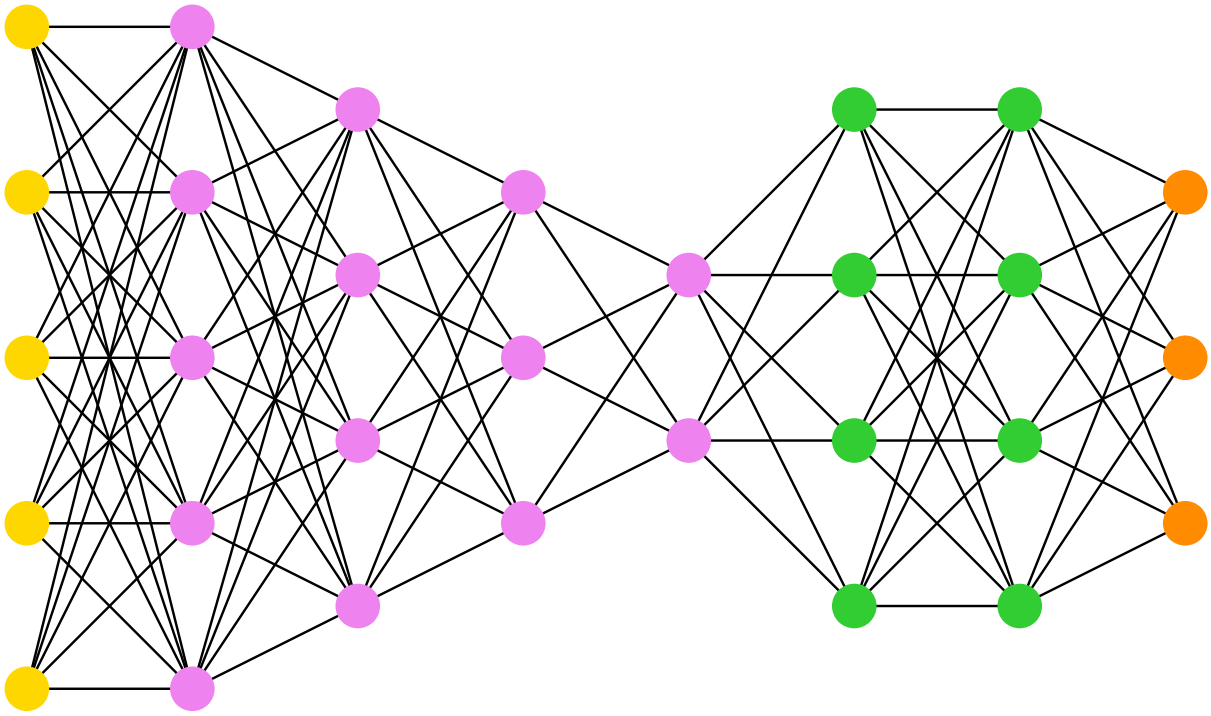
17.14 Network data (requires networkx)

See also: https://networkx.org/documentation/stable/index.html

See also: https://networkx.org/documentation/stable/auto_examples/drawing/index.html

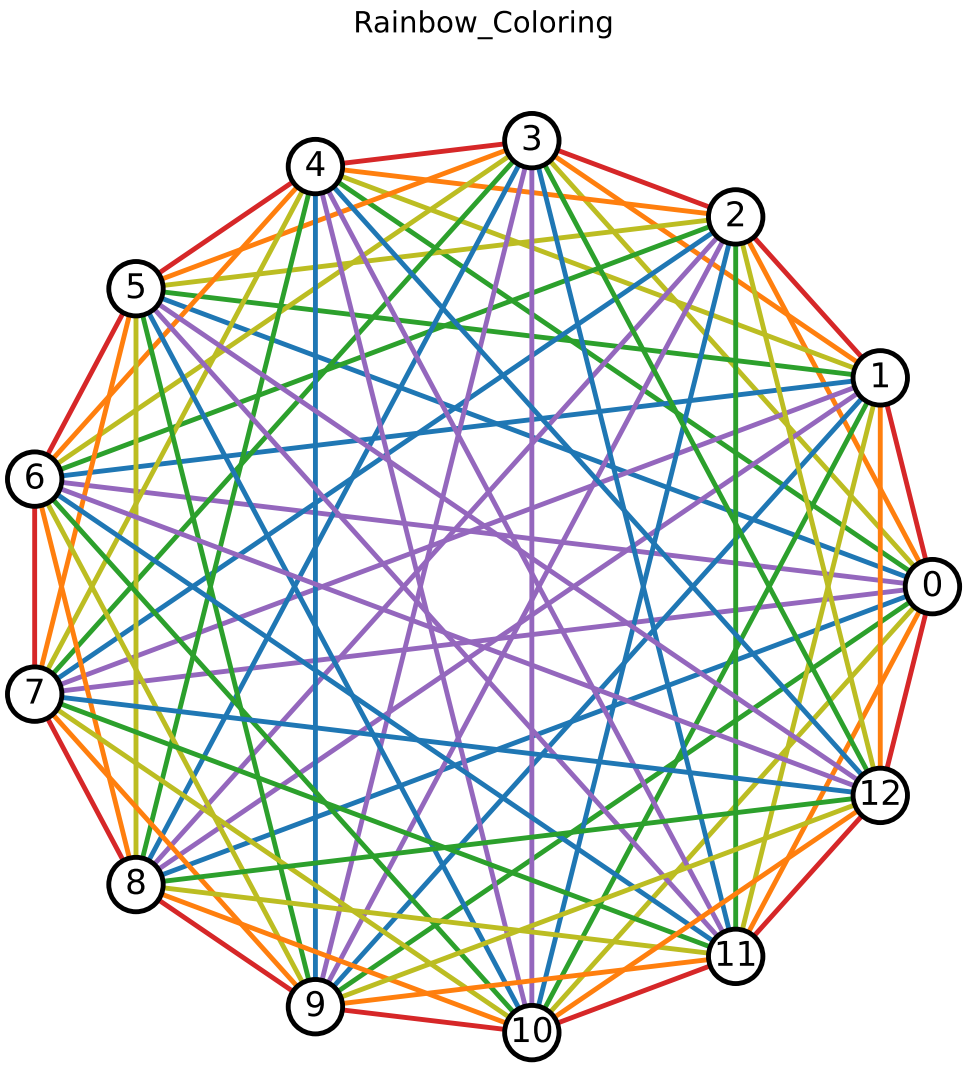
17.14.1 Multipartite_Layout

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_MultipartiteLayout.py, and the XML code in the file D01_Multipartite.2D.xml.



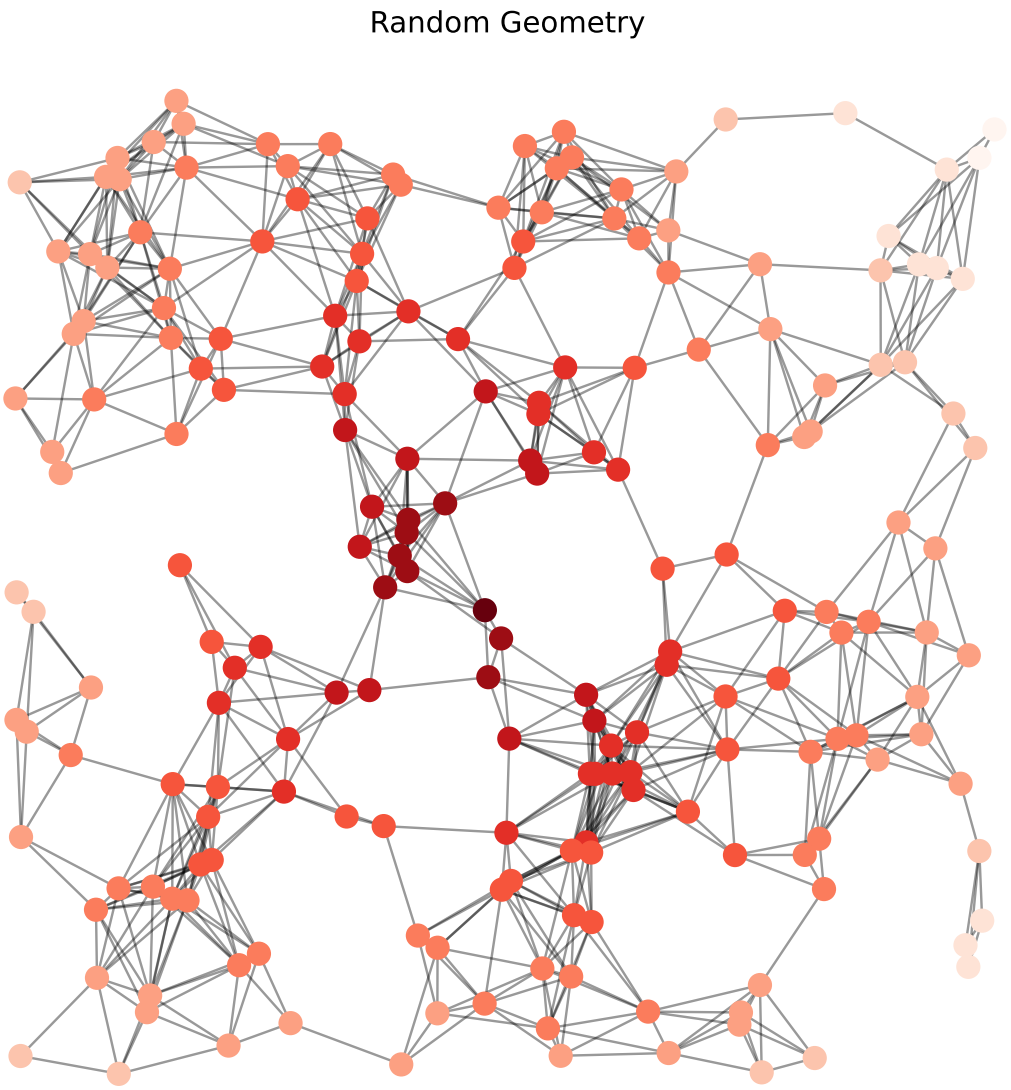
17.14.2 Rainbow_Coloring

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_RainbowColoring.py, and the XML code in the file D02_RainbowColoring.2D.xml.



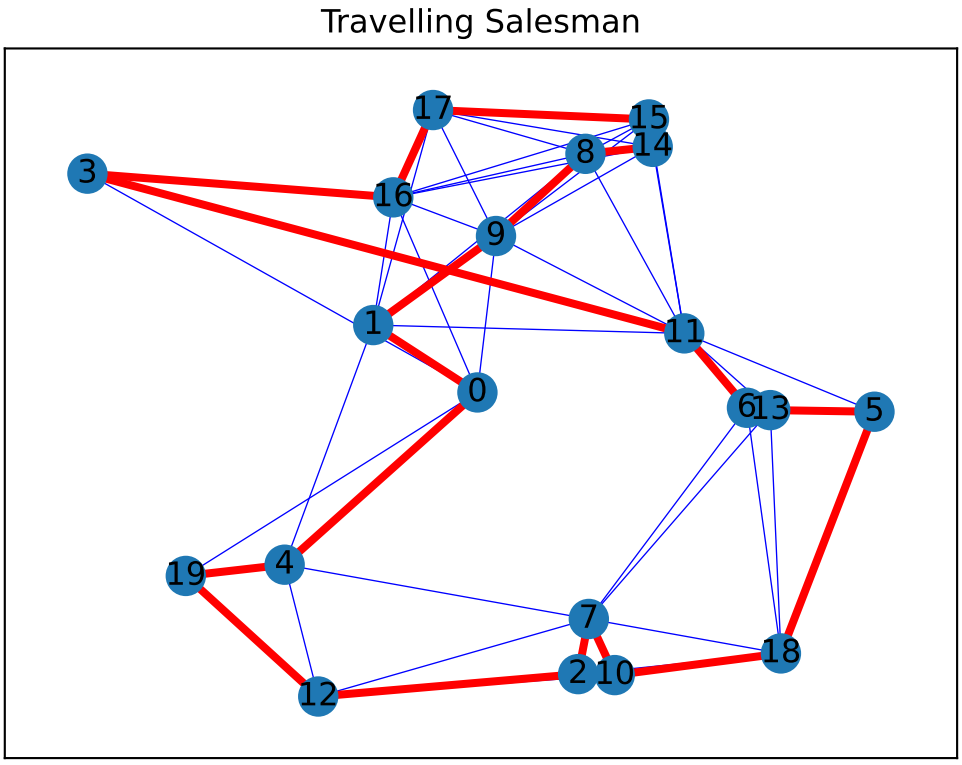
17.14.3 Random_Geometry

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_RandomGeometric.py, and the XML code in the file D03_RandomGeometric.2D.xml.



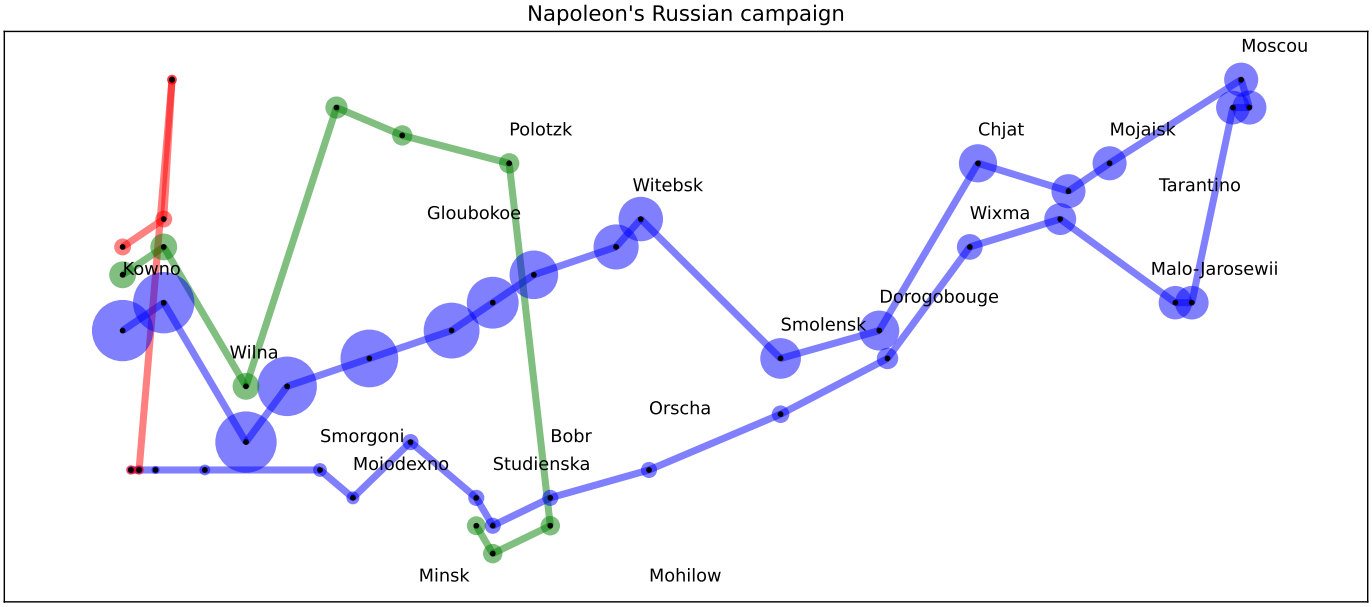
17.14.4 Travelling_Salesman

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_TravellingSalesman.py, and the XML code in the file D04_TravellingSalesman.2D.xml.



17.14.5 NapoleonRussianCampaign

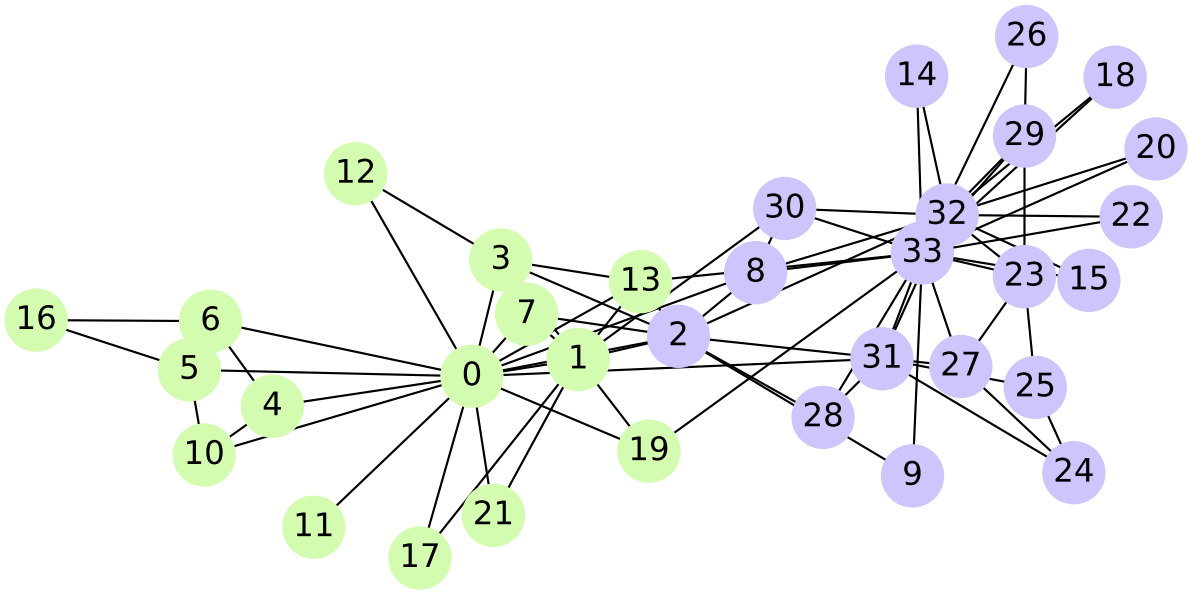
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_NapoleonRussianCampaign.py, and the XML code in the file D05_Napoleon.2D.xml.



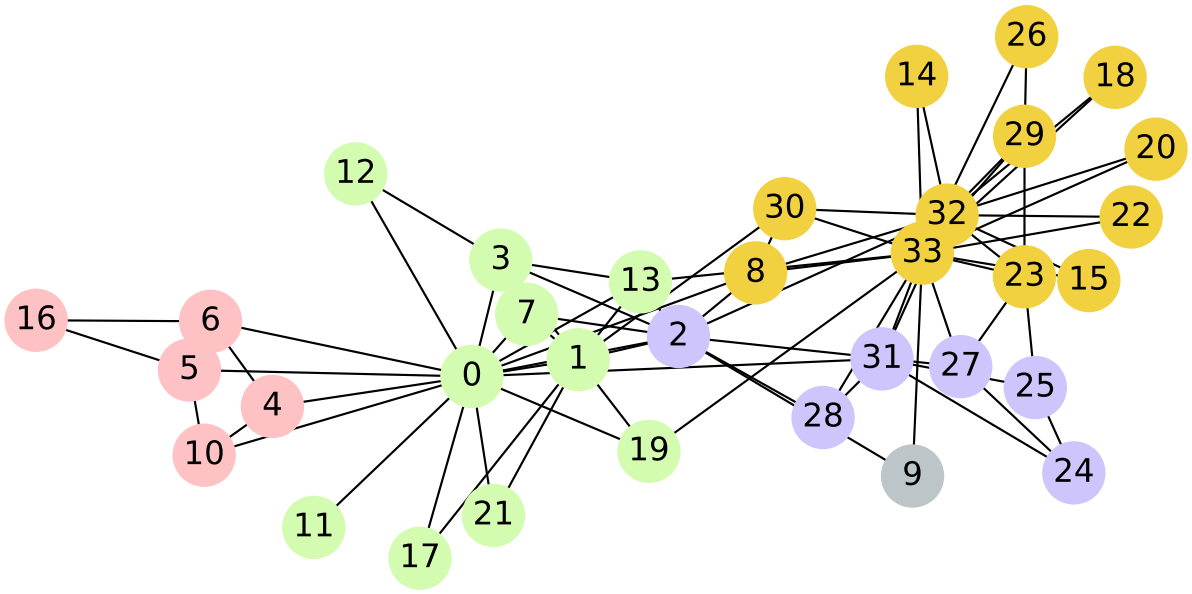
17.14.6 Community Visualisation

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_CommunityVisualisation.py, and the XML code in the file D06_CommunityVisualisation.2D.xml.

Community Visualization of 2 communities with modularity of 0.34766

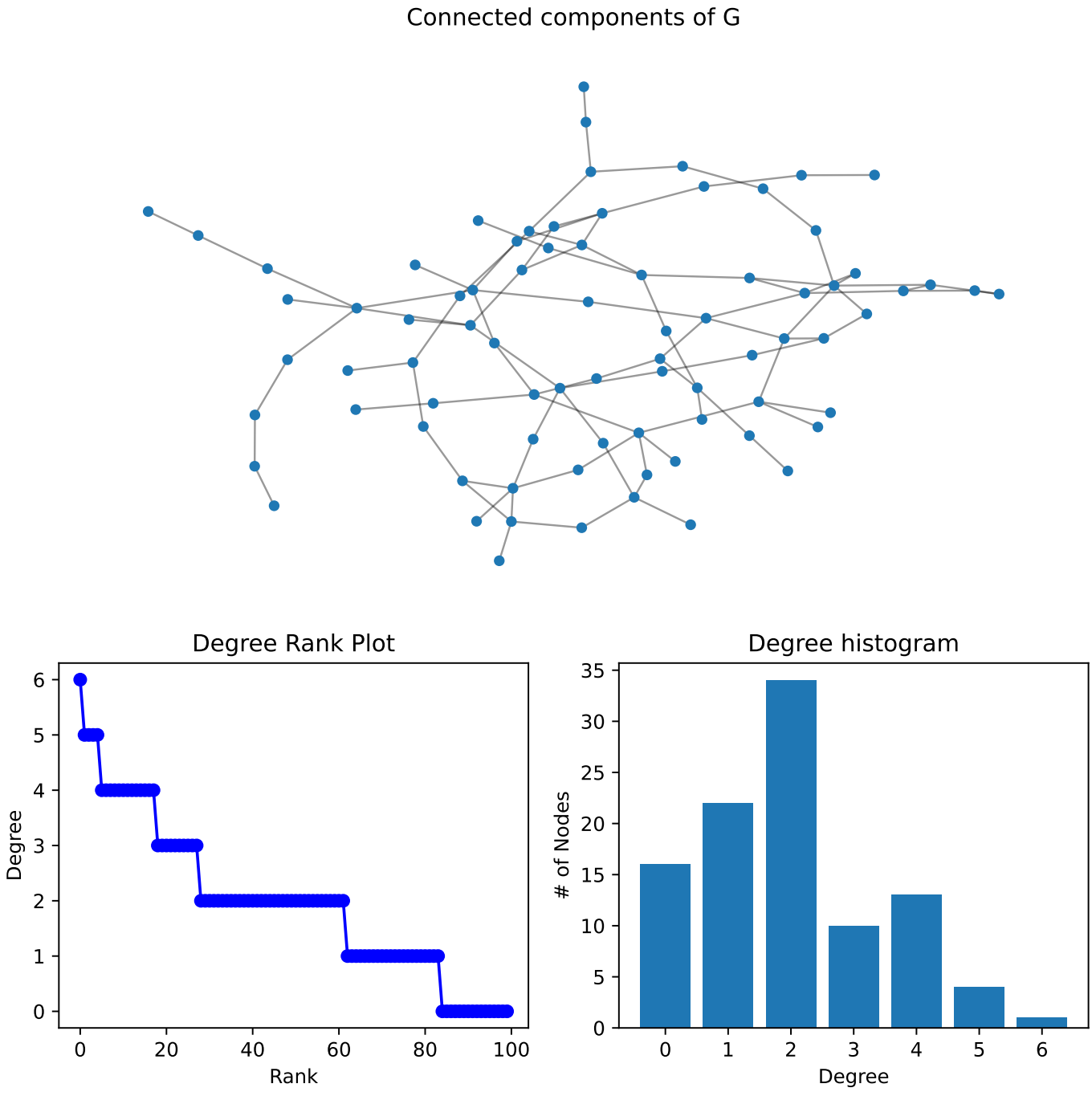


Community Visualization of 5 communities with modularity of 0.384972



17.14.7 Connected Components

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_ConnectedComponents.py, and the XML code in the file D07_ConnectedComponents.2D.xml.



CHAPTER

EIGHTEEN

VISUALISATION OF FUNCTIONS AND CURVES (2D)

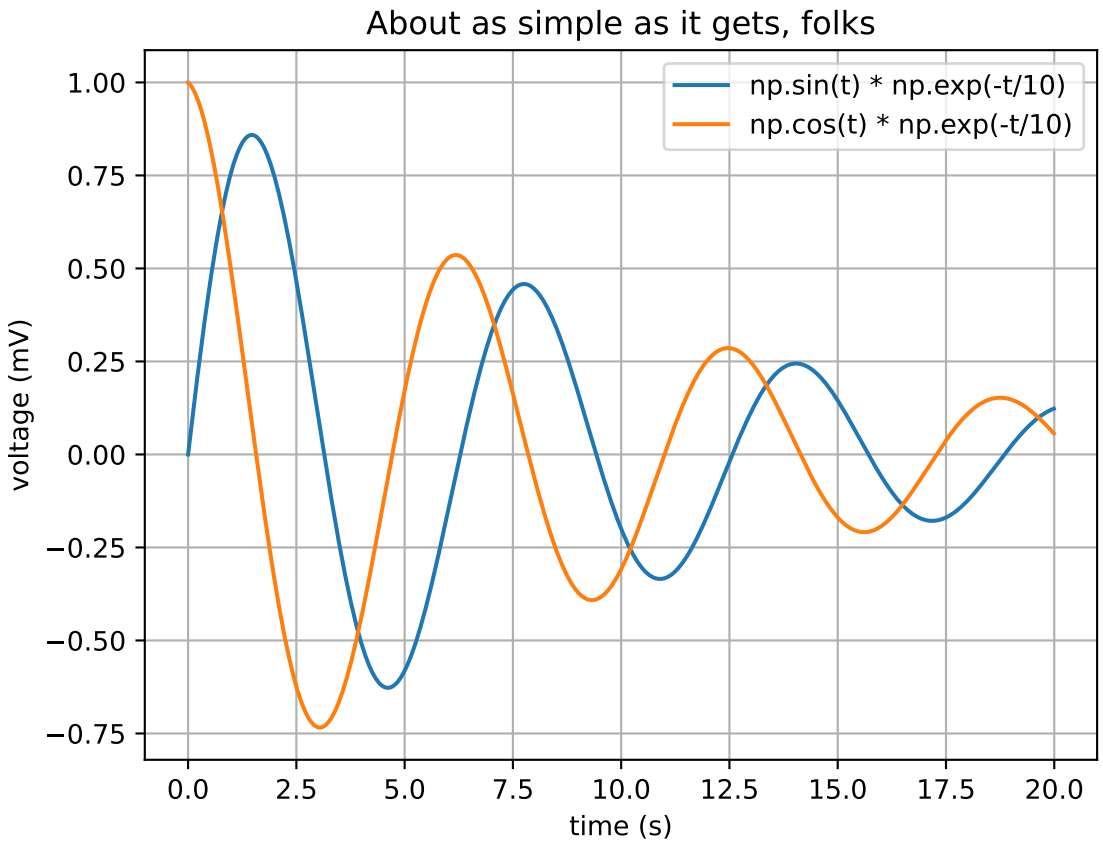
18.1 Introduction to 2D functions plots

18.1.1 Simple plot of (well-behaved) functions in 2D

This is a (slightly modified) example from the Matplotlib tutorials.

See https://matplotlib.org/stable/gallery/lines_bars_and_markers/simple_plot.html

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Simpleplot.py, and the XML code in the file D01_Simpleplot.2D.xml.

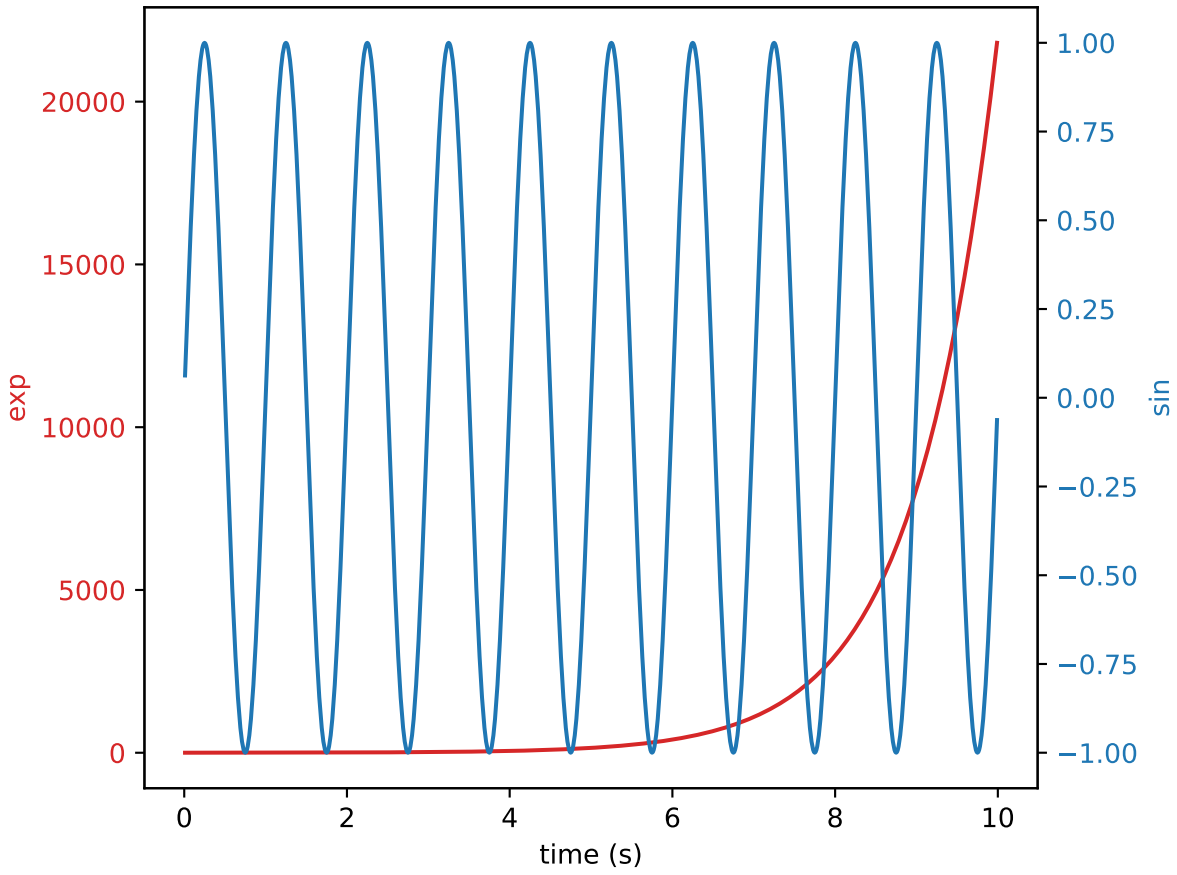


18.1.2 Function plots on different scales

This is an example from the Matplotlib tutorials.

See https://matplotlib.org/stable/gallery/subplots_axes_and_figures/two_scales.html

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_PlotDiffScales.py, and the XML code in the file D02_PlotDiffScales.2D.xml.

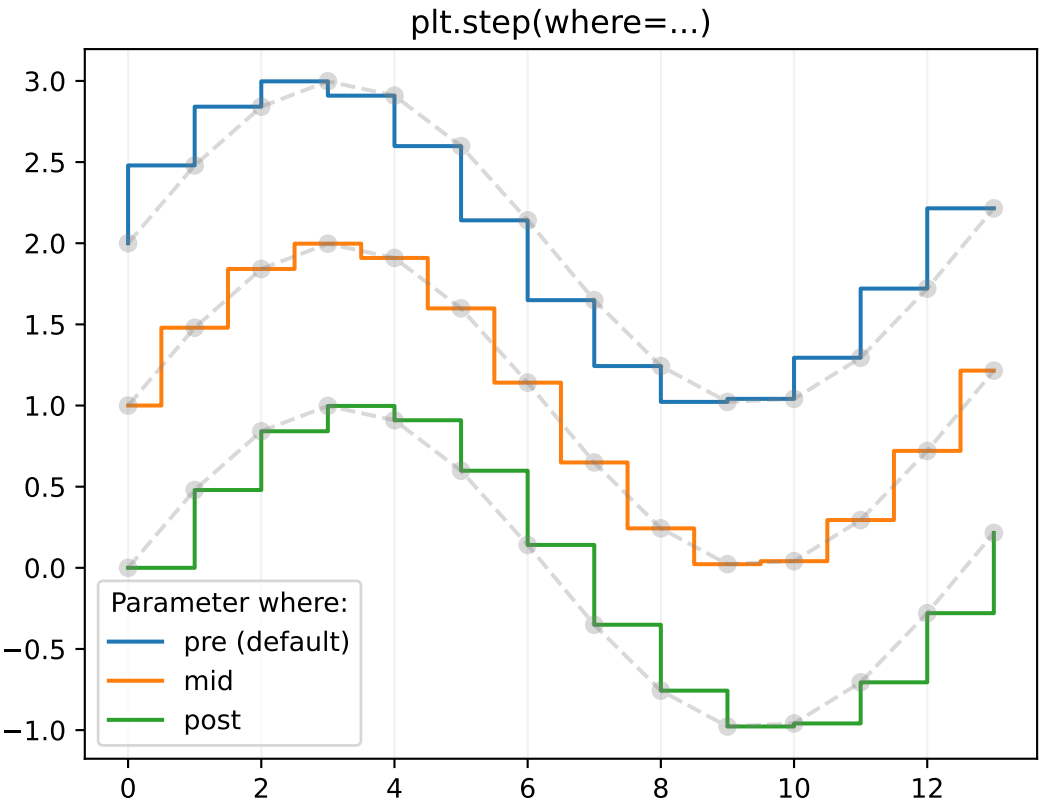


18.1.3 Stepplot

This is an example from the Matplotlib tutorials.

See https://matplotlib.org/stable/gallery/lines_bars_and_markers/step_demo.html

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Stepplot.py, and the XML code in the file D03_Stepplot.2D.xml.

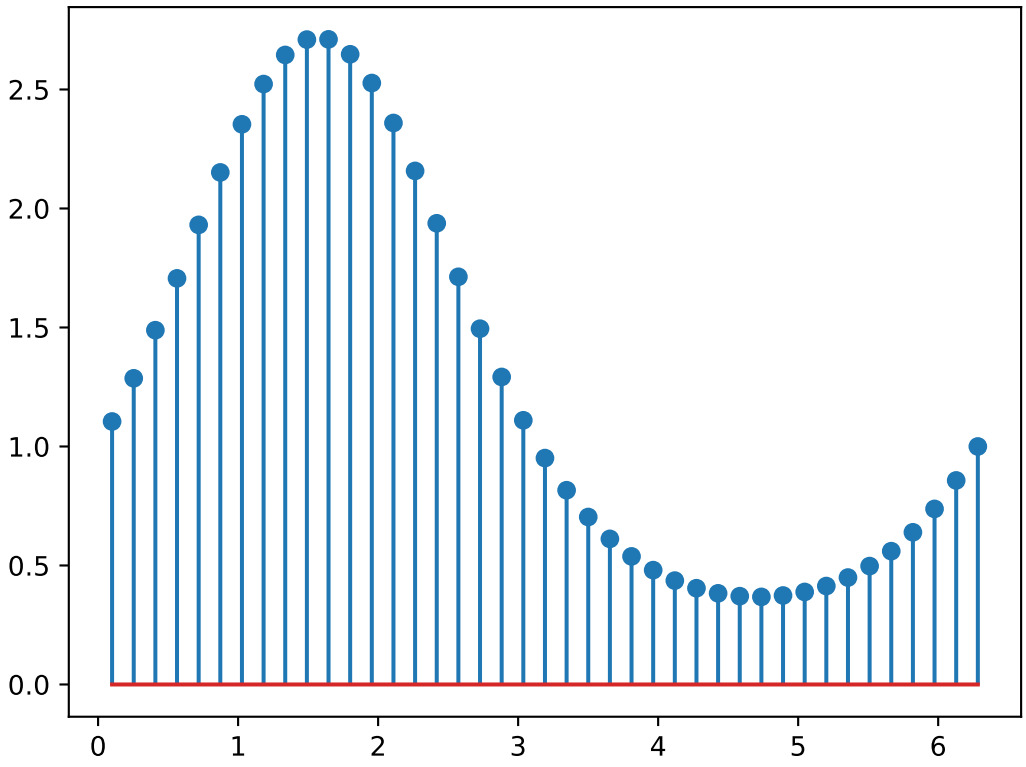


18.1.4 Stemplot

This is an example from the Matplotlib tutorials.

See https://matplotlib.org/stable/gallery/lines_bars_and_markers/stem_plot.html#sphx-glr-gallery-lines-bars-and-markers-stem-plot-py

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Stemplot.py, and the XML code in the file D04_Stemplot.2D.xml.

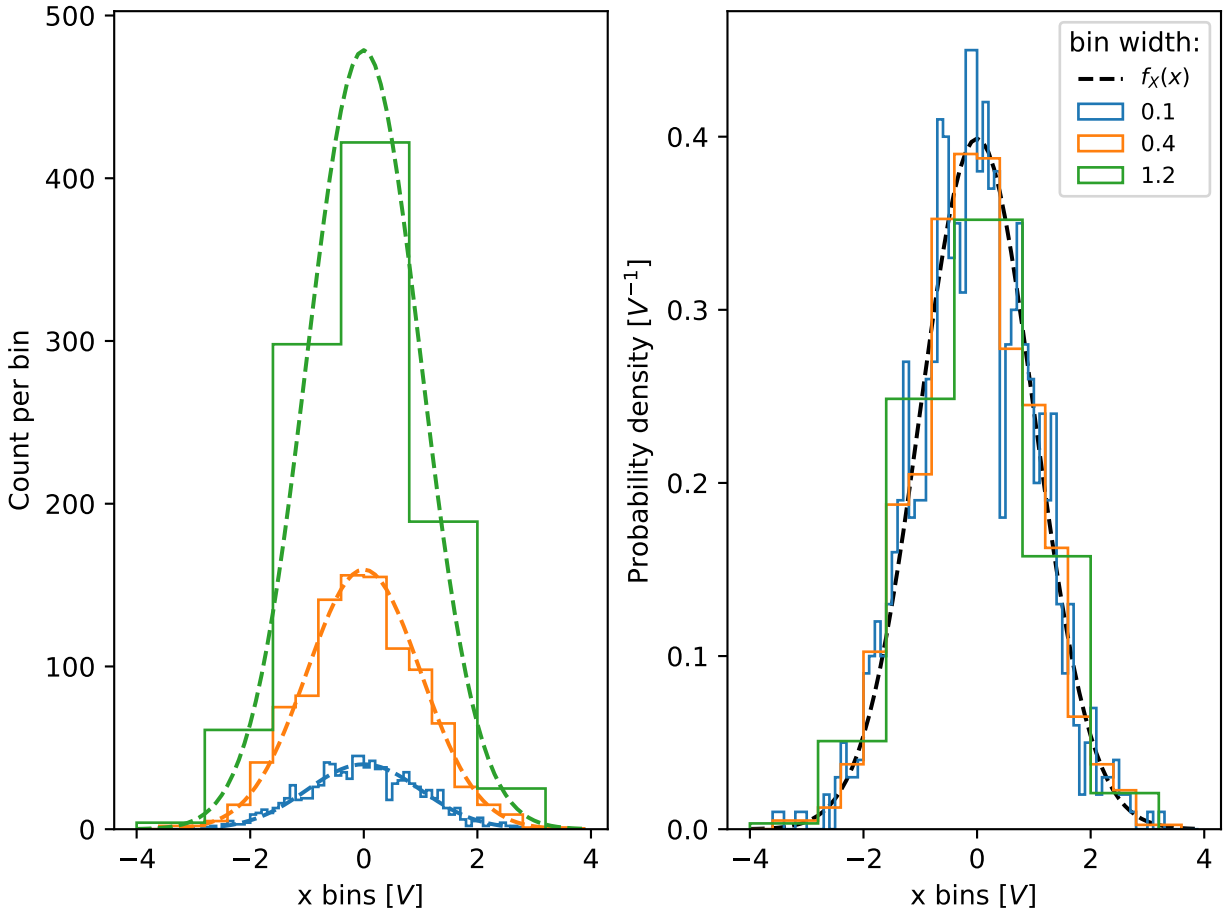


18.1.5 Histograms and densities

This is an example from the Matplotlib tutorials.

See https://matplotlib.org/stable/gallery/statistics/histogram_normalization.html#

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_HistogramsAndDensities.py, and the XML code in the file D05_HistogramsAndDensities.2D.xml.

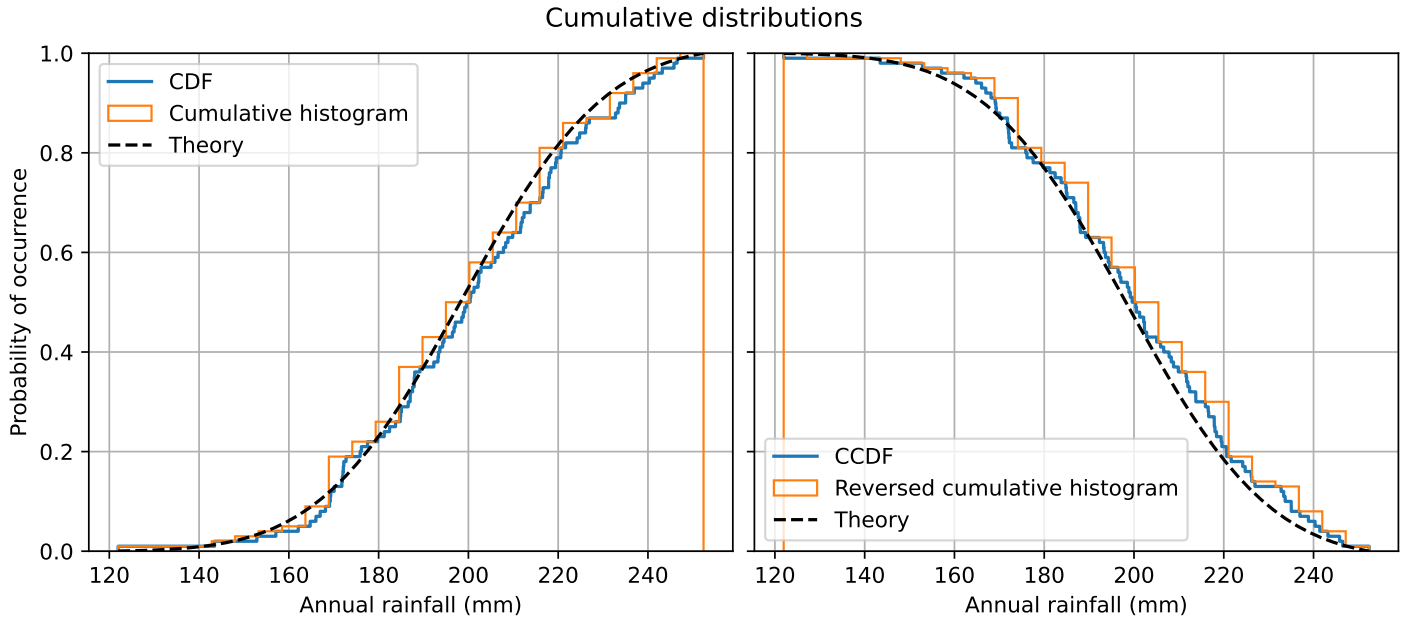


18.1.6 Cumulative histogram and CDF

This is an example from the Matplotlib tutorials.

see https://matplotlib.org/stable/gallery/statistics/histogram_cumulative.html#

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_CumulativeHistogramAndCDF.py, and the XML code in the file D06_CumulativeHistogramAndCDF.2D.xml.



18.1.7 Plots of continuous distribution functions

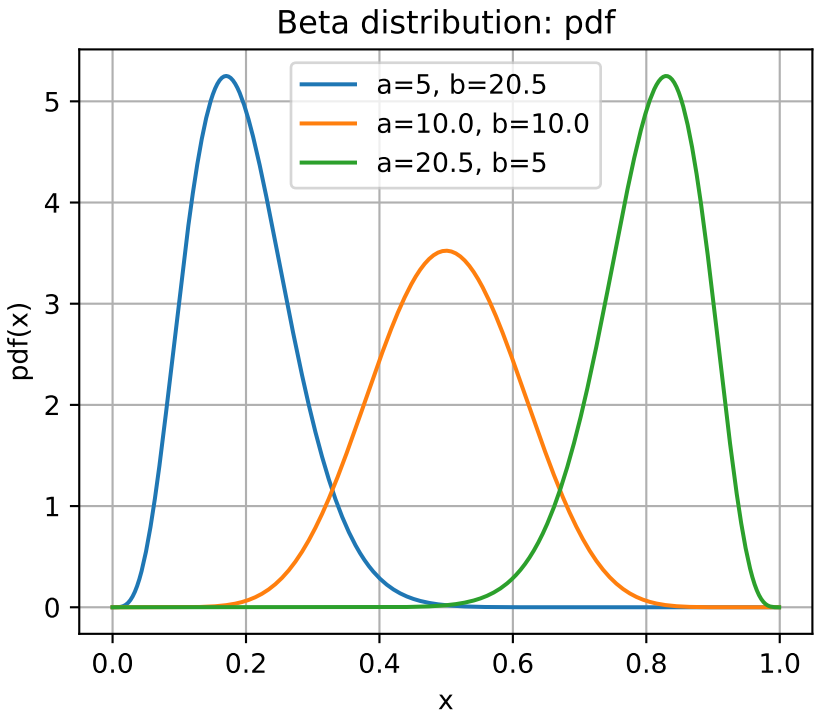
Some text

18.1.7.1 Probability density function (pdf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_DistPlotContinuous.py, and the XML code in the file D07_DistPlotContinuous.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None
        target = 'pdf' # pdf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'

        Title = 'Beta distribution'
        a = [5, 10.0, 20.5]
        b = [20.5, 10.0, 5]
        xlim = [0, 0.999]
        if target=='hf': ylim=[0, 100]

        dlist = []
        ltext = []
        for j in range(len(a)):
            dlist.append(dreal.dist_beta(a[j], b[j]))
            ltext.append('a=' + str(a[j]) + ', b=' + str(b[j]))

        DistPlotContinuous(Title = Title, dlist=dlist, xlim = xlim,
            ylim = ylim, target = target, ltext = ltext, marker='o',
            markersize=0)
```

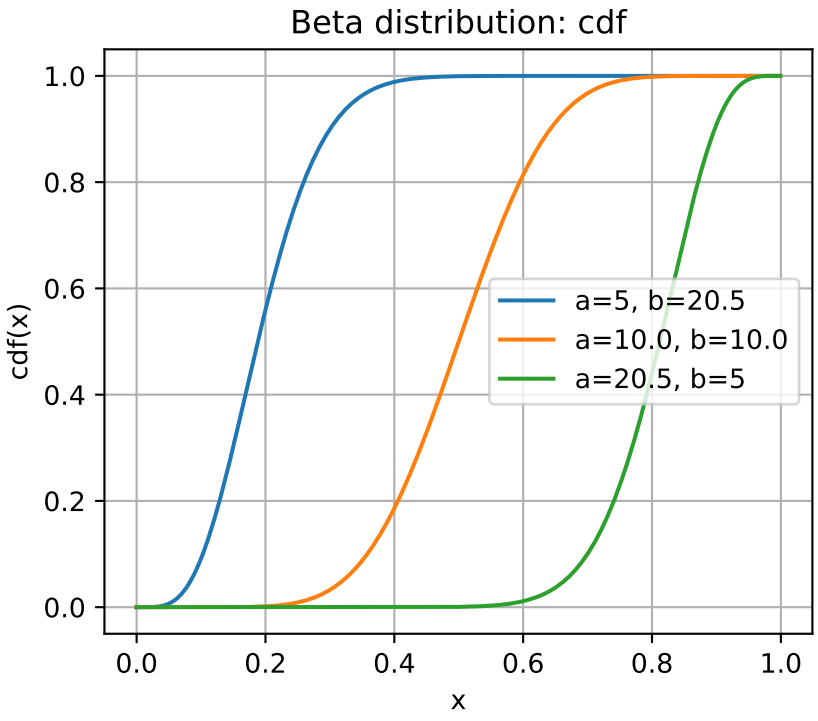


18.1.7.2 Cumulative distribution function (cdf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_DistPlotContinuous.py, and the XML code in the file D07_DistPlotContinuous.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None
        target = 'cdf' # pdf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'

        Title = 'Beta distribution'
        a = [5, 10.0, 20.5]
        b = [20.5, 10.0, 5]
        xlim = [0, 0.999]
        if target=='hf': ylim=[0, 100]

        dlist = []
        ltext = []
        for j in range(len(a)):
            dlist.append(dreal.dist_beta(a[j], b[j]))
            ltext.append('a=' + str(a[j]) + ', b=' + str(b[j]))

        DistPlotContinuous(Title = Title, dlist=dlist, xlim = xlim,
            ylim = ylim, target = target, ltext = ltext, marker='o',
            markersize=0)
```

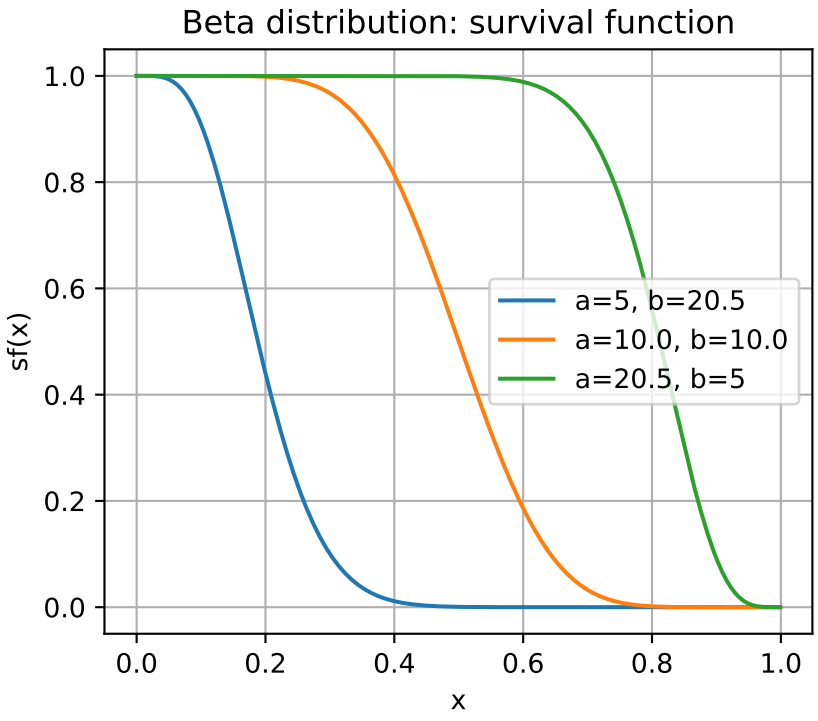


18.1.7.3 Survival function (sf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_DistPlotContinuous.py, and the XML code in the file D07_DistPlotContinuous.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None
        target = 'sf' # pdf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'

        Title = 'Beta distribution'
        a = [5, 10.0, 20.5]
        b = [20.5, 10.0, 5]
        xlim = [0, 0.999]
        if target=='hf': ylim=[0, 100]

        dlist = []
        ltext = []
        for j in range(len(a)):
            dlist.append(dreal.dist_beta(a[j], b[j]))
            ltext.append('a=' + str(a[j]) + ', b=' + str(b[j]))

        DistPlotContinuous(Title = Title, dlist=dlist, xlim = xlim,
            ylim = ylim, target = target, ltext = ltext, marker='o',
            markersize=0)
```

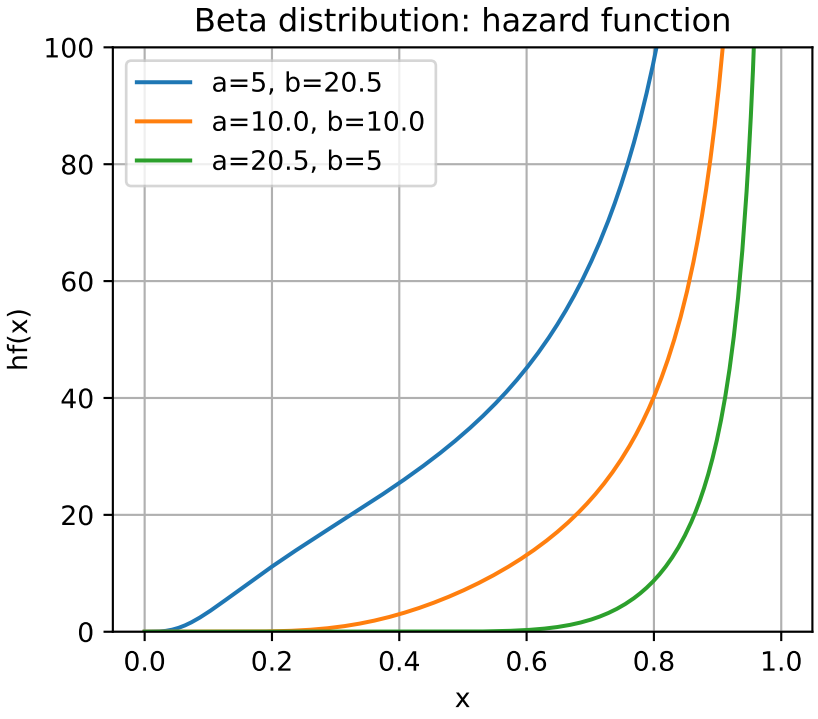


18.1.7.4 Hazard function (hf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_DistPlotContinuous.py, and the XML code in the file D07_DistPlotContinuous.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None
        target = 'hf' # pdf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'

        Title = 'Beta distribution'
        a = [5, 10.0, 20.5]
        b = [20.5, 10.0, 5]
        xlim = [0, 0.999]
        if target=='hf': ylim=[0, 100]

        dlist = []
        ltext = []
        for j in range(len(a)):
            dlist.append(dreal.dist_beta(a[j], b[j]))
            ltext.append('a=' + str(a[j]) + ', b=' + str(b[j]))

        DistPlotContinuous(Title = Title, dlist=dlist, xlim = xlim,
            ylim = ylim, target = target, ltext = ltext, marker='o',
            markersize=0)
```

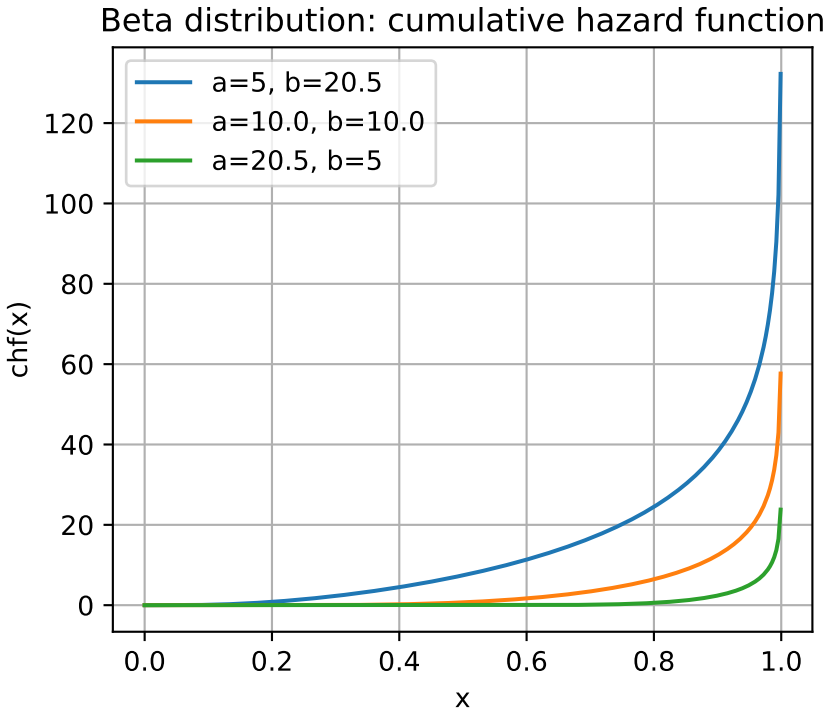


18.1.7.5 Cumulative hazard function (chf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_DistPlotContinuous.py, and the XML code in the file D07_DistPlotContinuous.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None
        target = 'chf' # pdf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'

        Title = 'Beta distribution'
        a = [5, 10.0, 20.5]
        b = [20.5, 10.0, 5]
        xlim = [0, 0.999]
        if target=='hf': ylim=[0, 100]

        dlist = []
        ltext = []
        for j in range(len(a)):
            dlist.append(dreal.dist_beta(a[j], b[j]))
            ltext.append('a=' + str(a[j]) + ', b=' + str(b[j]))

        DistPlotContinuous(Title = Title, dlist=dlist, xlim = xlim,
            ylim = ylim, target = target, ltext = ltext, marker='o',
            markersize=0)
```

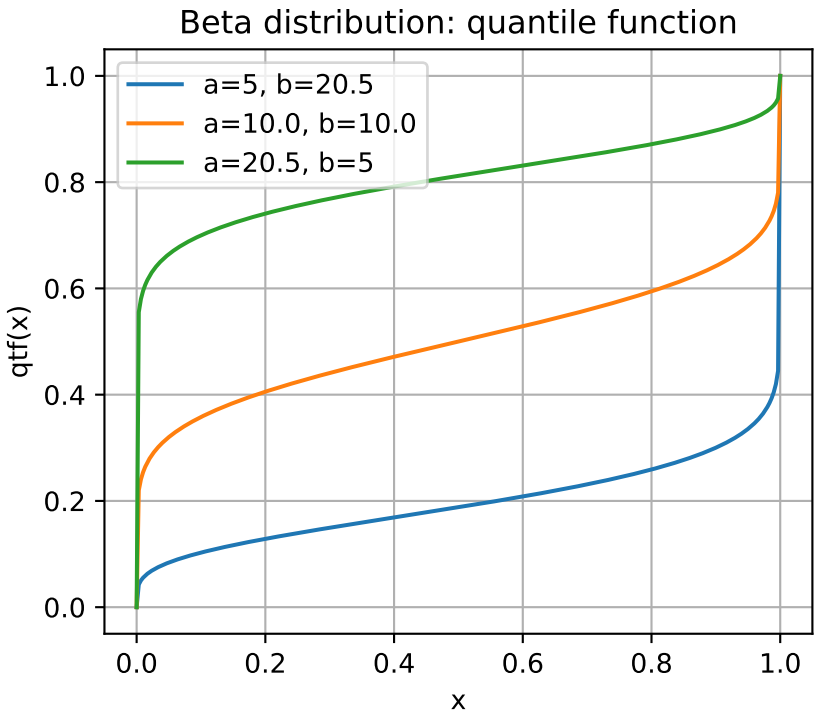


18.1.7.6 Quantile function (qtf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_DistPlotContinuous.py, and the XML code in the file D07_DistPlotContinuous.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None
        target = 'qtf' # pdf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'

        Title = 'Beta distribution'
        a = [5, 10.0, 20.5]
        b = [20.5, 10.0, 5]
        xlim = [0, 0.999]
        if target=='hf': ylim=[0, 100]

        dlist = []
        ltext = []
        for j in range(len(a)):
            dlist.append(dreal.dist_beta(a[j], b[j]))
            ltext.append('a=' + str(a[j]) + ', b=' + str(b[j]))

        DistPlotContinuous(Title = Title, dlist=dlist, xlim = xlim,
            ylim = ylim, target = target, ltext = ltext, marker='o',
            markersize=0)
```

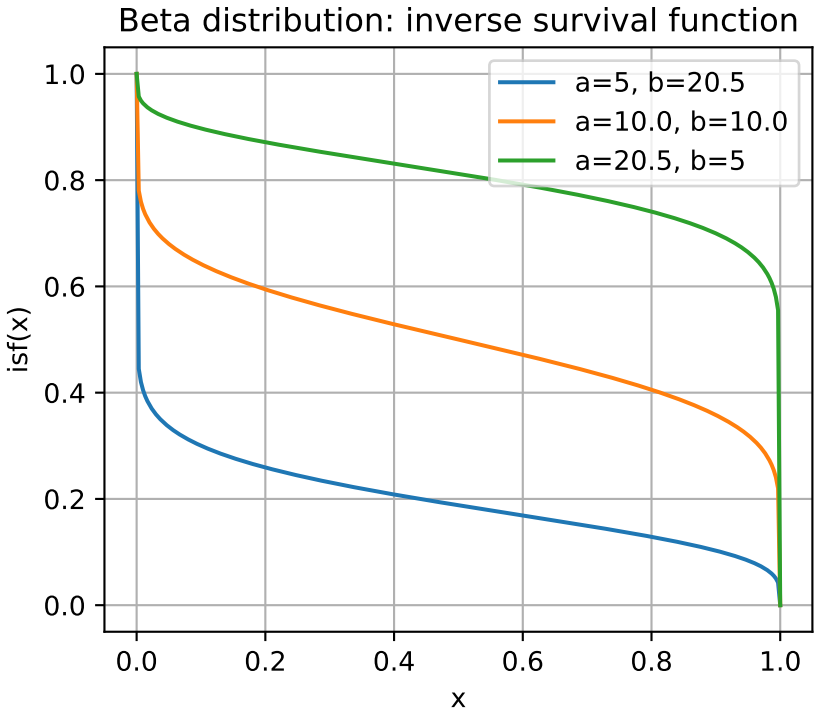


18.1.7.7 Inverse survival function (isf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_DistPlotContinuous.py, and the XML code in the file D07_DistPlotContinuous.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None
        target = 'qtf' # pdf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'

        Title = 'Beta distribution'
        a = [5, 10.0, 20.5]
        b = [20.5, 10.0, 5]
        xlim = [0, 0.999]
        if target=='hf': ylim=[0, 100]

        dlist = []
        ltext = []
        for j in range(len(a)):
            dlist.append(dreal.dist_beta(a[j], b[j]))
            ltext.append('a=' + str(a[j]) + ', b=' + str(b[j]))

        DistPlotContinuous(Title = Title, dlist=dlist, xlim = xlim,
            ylim = ylim, target = target, ltext = ltext, marker='o',
            markersize=0)
```



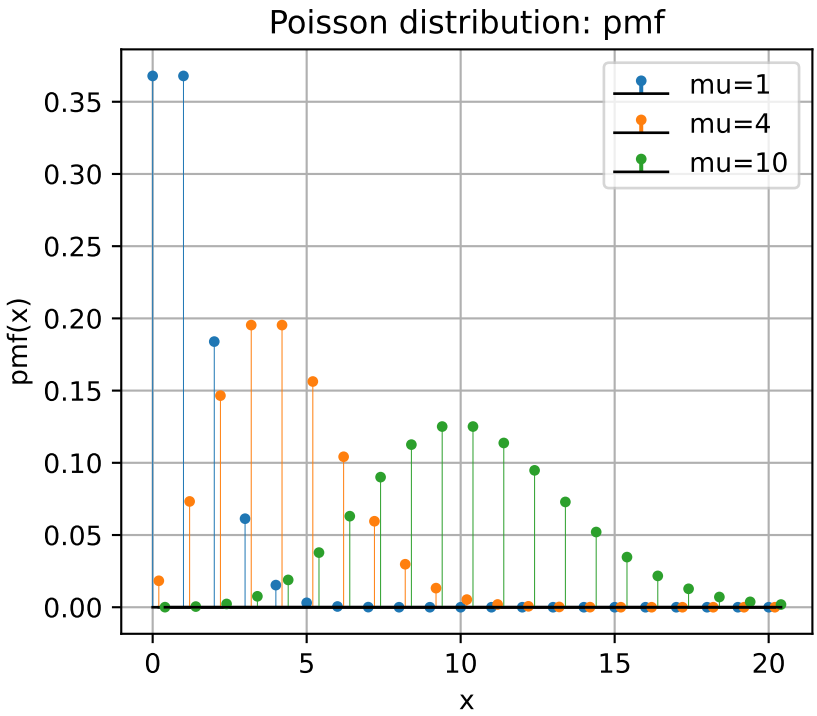
18.1.8 Plots of discrete distribution functions

18.1.8.1 Probability mass function (pmf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_DistPlotDiscrete.py, and the XML code in the file D08_DistPlotDiscrete.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None

        Title = 'Poisson distribution'
        target = 'pmf' # pmf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'
        mu = [1, 4, 10]
        xlim = [0.0, 20.0]
        ylim = None
        if target=='qtf': ylim=[0, 20]

        dlist = []
        ltext = []
        for j in range(len(mu)):
            dlist.append(dreal.dist_poisson(mu[j]))
            ltext.append('mu=' + str(mu[j]))
        DistPlotDiscrete(Title = Title, dlist=dlist, xlim = xlim,  ylim = ylim,
            target = target, ltext = ltext, lattice=True, marker='o',
            markersize=3, vertical_lines=True)
```

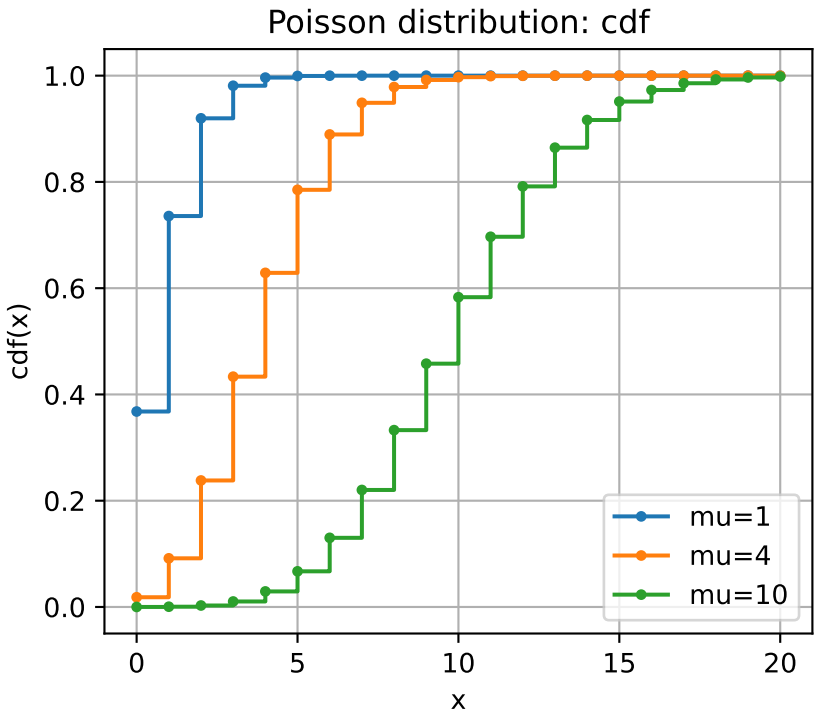


18.1.8.2 Cumulative distribution function (cdf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_DistPlotDiscrete.py, and the XML code in the file D08_DistPlotDiscrete.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None

        Title = 'Poisson distribution'
        target = 'cdf' # pmf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'
        mu = [1, 4, 10]
        xlim = [0.0, 20.0]
        ylim = None
        if target=='qtf': ylim=[0, 20]

        dlist = []
        ltext = []
        for j in range(len(mu)):
            dlist.append(dreal.dist_poisson(mu[j]))
            ltext.append('mu=' + str(mu[j]))
        DistPlotDiscrete(Title = Title, dlist=dlist, xlim = xlim,  ylim = ylim,
            target = target, ltext = ltext, lattice=True, marker='o',
            markersize=3, vertical_lines=True)
```

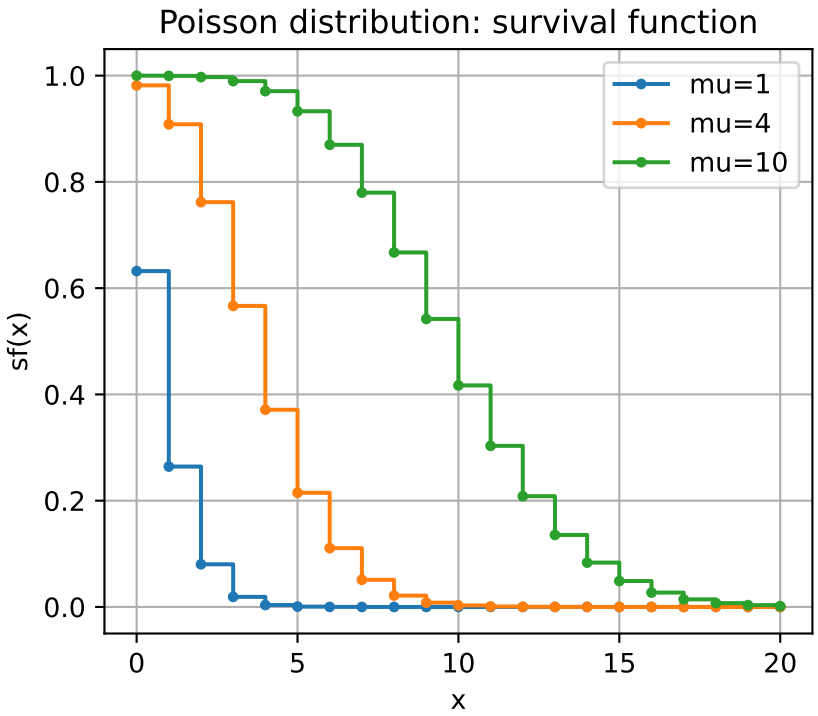


18.1.8.3 Survival function (sf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_DistPlotDiscrete.py, and the XML code in the file D08_DistPlotDiscrete.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None

        Title = 'Poisson distribution'
        target = 'sf' # pmf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'
        mu = [1, 4, 10]
        xlim = [0.0, 20.0]
        ylim = None
        if target=='qtf': ylim=[0, 20]

        dlist = []
        ltext = []
        for j in range(len(mu)):
            dlist.append(dreal.dist_poisson(mu[j]))
            ltext.append('mu=' + str(mu[j]))
        DistPlotDiscrete(Title = Title, dlist=dlist, xlim = xlim,  ylim = ylim,
            target = target, ltext = ltext, lattice=True, marker='o',
            markersize=3, vertical_lines=True)
```

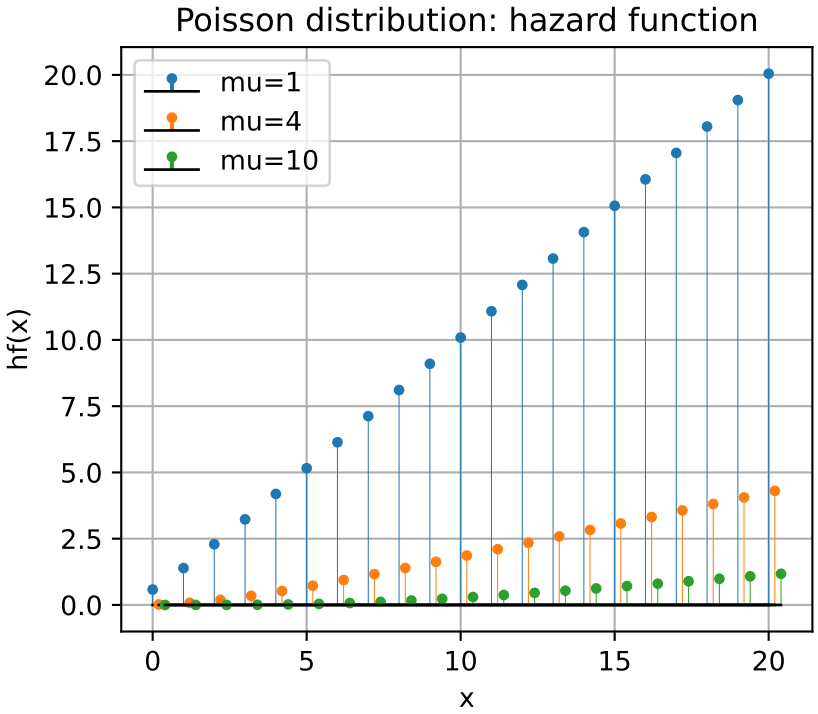


18.1.8.4 Hazard function (hf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_DistPlotDiscrete.py, and the XML code in the file D08_DistPlotDiscrete.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None

        Title = 'Poisson distribution'
        target = 'hf' # pmf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'
        mu = [1, 4, 10]
        xlim = [0.0, 20.0]
        ylim = None
        if target=='qtf': ylim=[0, 20]

        dlist = []
        ltext = []
        for j in range(len(mu)):
            dlist.append(dreal.dist_poisson(mu[j]))
            ltext.append('mu=' + str(mu[j]))
        DistPlotDiscrete(Title = Title, dlist=dlist, xlim = xlim,  ylim = ylim,
            target = target, ltext = ltext, lattice=True, marker='o',
            markersize=3, vertical_lines=True)
```

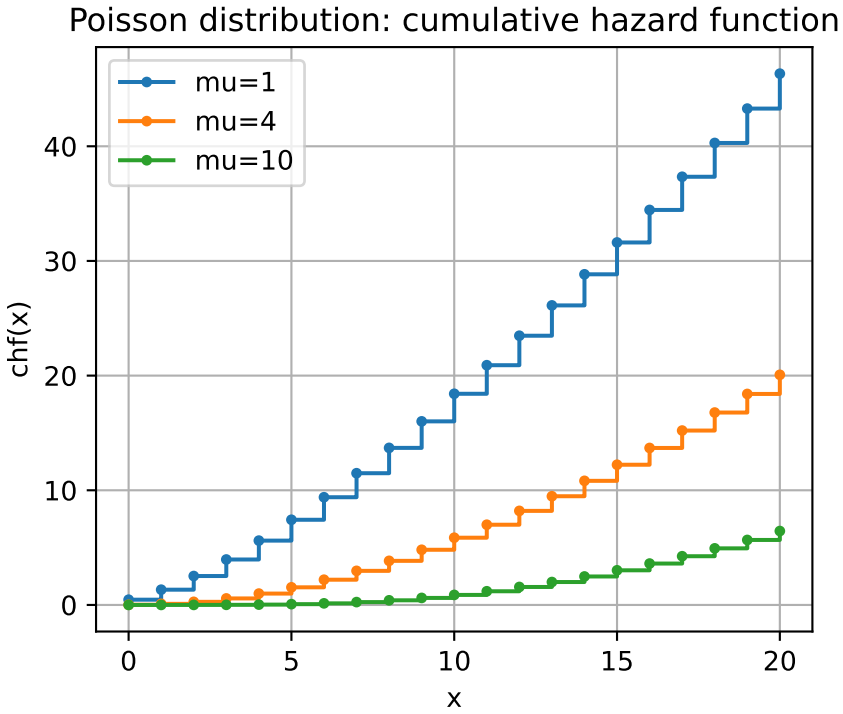


18.1.8.5 Cumulative hazard function (chf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_DistPlotDiscrete.py, and the XML code in the file D08_DistPlotDiscrete.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None

        Title = 'Poisson distribution'
        target = 'chf' # pmf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'
        mu = [1, 4, 10]
        xlim = [0.0, 20.0]
        ylim = None
        if target=='qtf': ylim=[0, 20]

        dlist = []
        ltext = []
        for j in range(len(mu)):
            dlist.append(dreal.dist_poisson(mu[j]))
            ltext.append('mu=' + str(mu[j]))
        DistPlotDiscrete(Title = Title, dlist=dlist, xlim = xlim,  ylim = ylim,
            target = target, ltext = ltext, lattice=True, marker='o',
            markersize=3, vertical_lines=True)
```

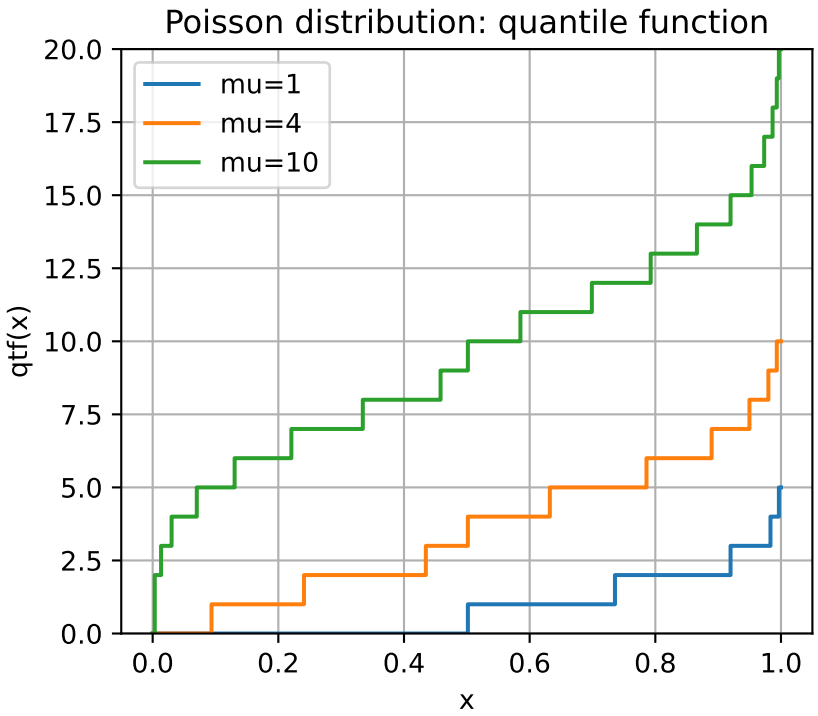


18.1.8.6 Quantile function (qtf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_DistPlotDiscrete.py, and the XML code in the file D08_DistPlotDiscrete.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None

        Title = 'Poisson distribution'
        target = 'qtf' # pmf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'
        mu = [1, 4, 10]
        xlim = [0.0, 20.0]
        ylim = None
        if target=='qtf': ylim=[0, 20]

        dlist = []
        ltext = []
        for j in range(len(mu)):
            dlist.append(dreal.dist_poisson(mu[j]))
            ltext.append('mu=' + str(mu[j]))
        DistPlotDiscrete(Title = Title, dlist=dlist, xlim = xlim,  ylim = ylim,
            target = target, ltext = ltext, lattice=True, marker='o',
            markersize=0, vertical_lines=True)
```

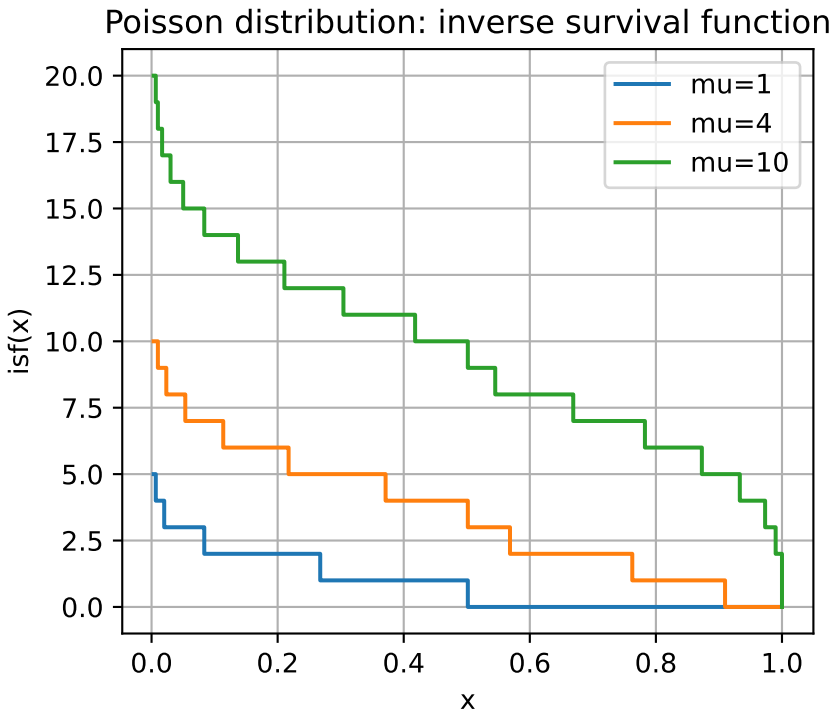


18.1.8.7 Inverse survival function (isf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_DistPlotDiscrete.py, and the XML code in the file D08_DistPlotDiscrete.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None

        Title = 'Poisson distribution'
        target = 'isf' # pmf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'
        mu = [1, 4, 10]
        xlim = [0.0, 20.0]
        ylim = None
        if target=='qtf': ylim=[0, 20]

        dlist = []
        ltext = []
        for j in range(len(mu)):
            dlist.append(dreal.dist_poisson(mu[j]))
            ltext.append('mu=' + str(mu[j]))
        DistPlotDiscrete(Title = Title, dlist=dlist, xlim = xlim,  ylim = ylim,
            target = target, ltext = ltext, lattice=True, marker='o',
            markersize=0, vertical_lines=True)
```



18.1.9 Mpmath-inspired functions plots in 2D

The plots discussed here show some possibilities to deal with not-so-well-behaved plots, i.e plots with real input, but (domain-wise) complex output, plots with singularities, and plots which grow very rapidly. The functions which are used here have been modified from Mpmath (see https://mpmath.readthedocs.io/en/stable/plotting.html).

These plots can use both Python-only contexts like `mpm` as well as `FixedPrec` contexts like `math53`, however, results may differ since the `FixedPrec` contexts enforce strict typing, whereas the Python-only contexts do not.

18.1.9.1 Simultaneous plot of sin and cos

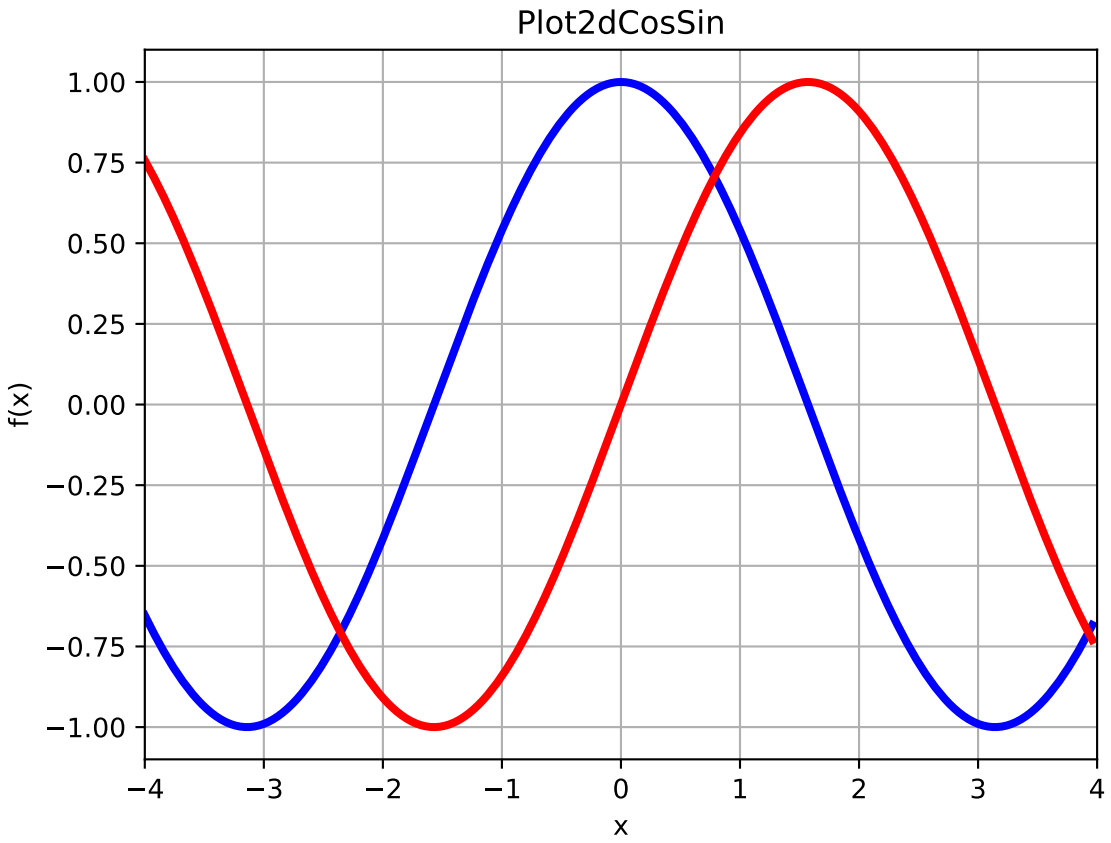
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_FuncPlot2d.py, and the XML code in the file D09_FuncPlot2d.2D.xml.

To produce the figure as shown below, using the `mpm` context, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        FuncPlot2d(ctx=mpm, f=[mpm.cos, mpm.sin], xlim=[-4, 4],
            Title = 'Plot2dCosSin')
```

To produce the figure as shown below, using the `math53` context, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        FuncPlot2d(ctx=math53, f=[ctx.cos, ctx.sin], xlim=[-4, 4],
            Title = 'Plot2dCosSin')
```

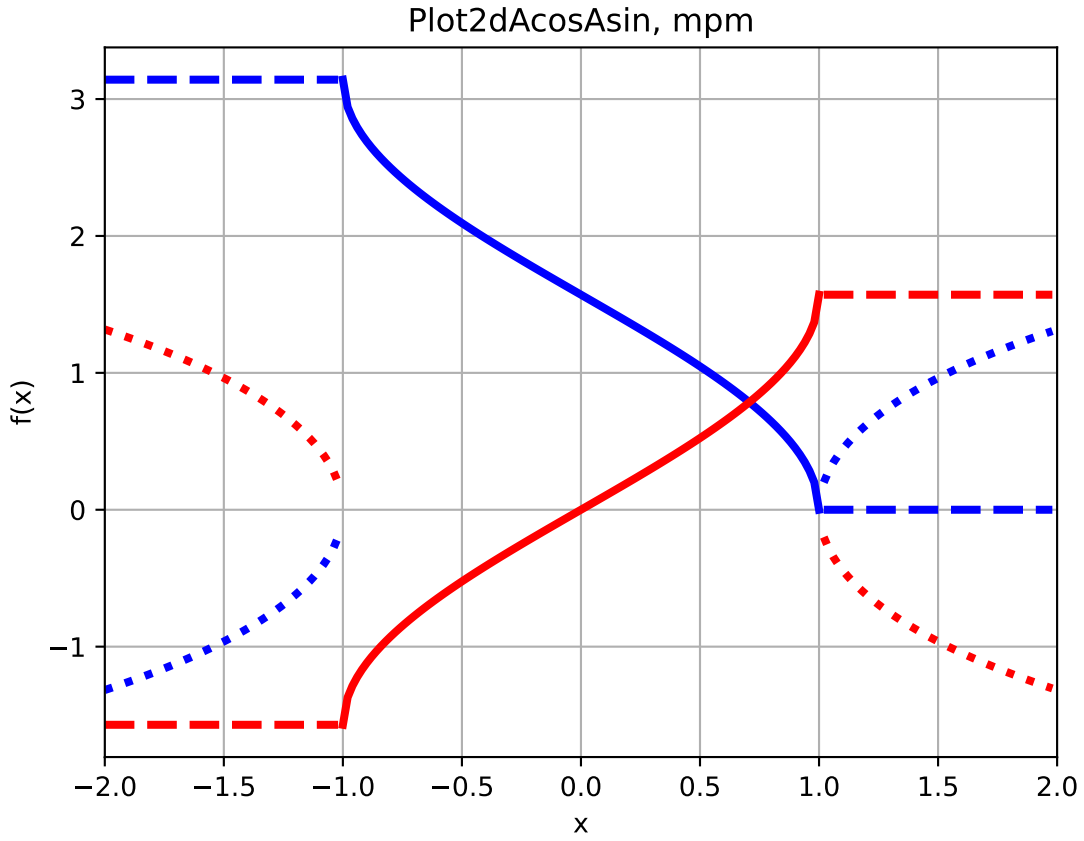


18.1.9.2 Simultaneous plots of asin and acos, for mpm and math53

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_FuncPlot2d.py.

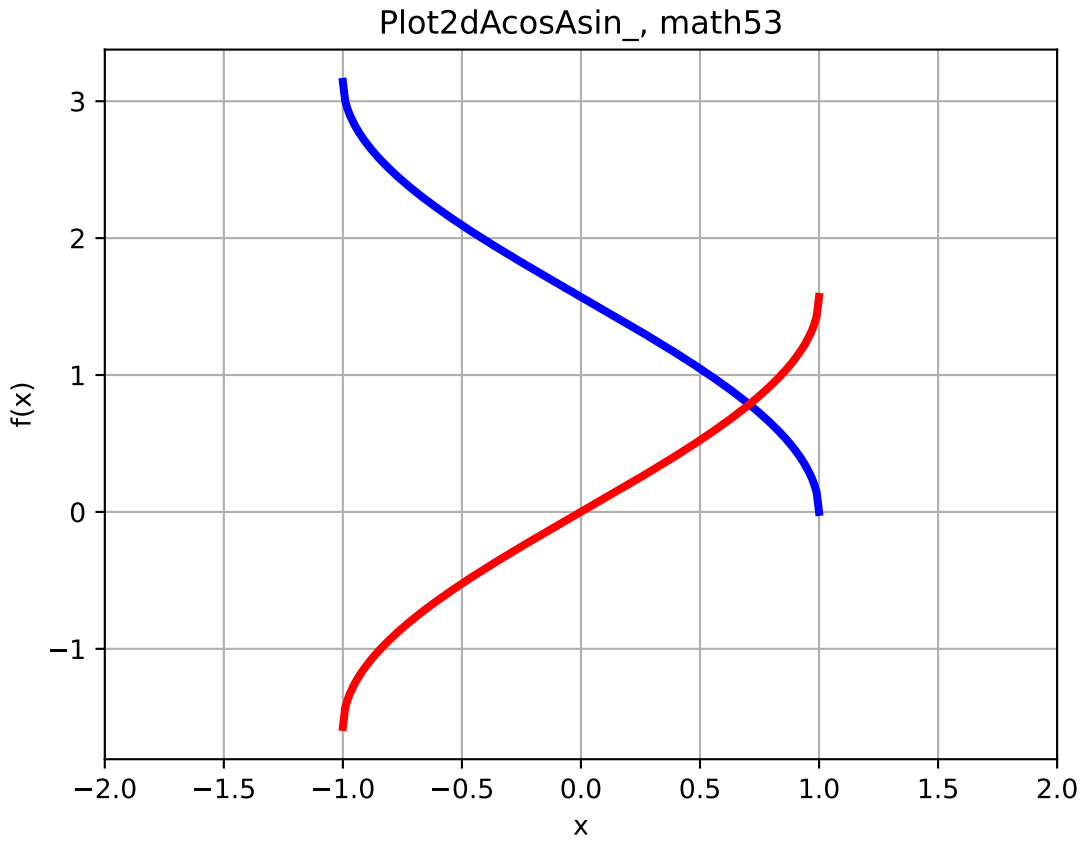
To produce the figure as shown below, using the `mpm` context, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        FuncPlot2d(ctx=mpm, f=[mpm.acos, mpm.asin], xlim=[-2, 2],
            Title = 'Simultaneous plot of asin an acos, using mpm')
```



To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        FuncPlot2d(ctx=math53, f=[ctx.acos, ctx.asin], xlim=[-2, 2],
            Title = 'Simultaneous plot of asin and acos, using math53')
```


Plot2dAcosAsin_, math53
f(x)
x
3
2
1
0
−1
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

18.1.9.3 Plot of cotangent without singularities specified

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_FuncPlot2d.py.

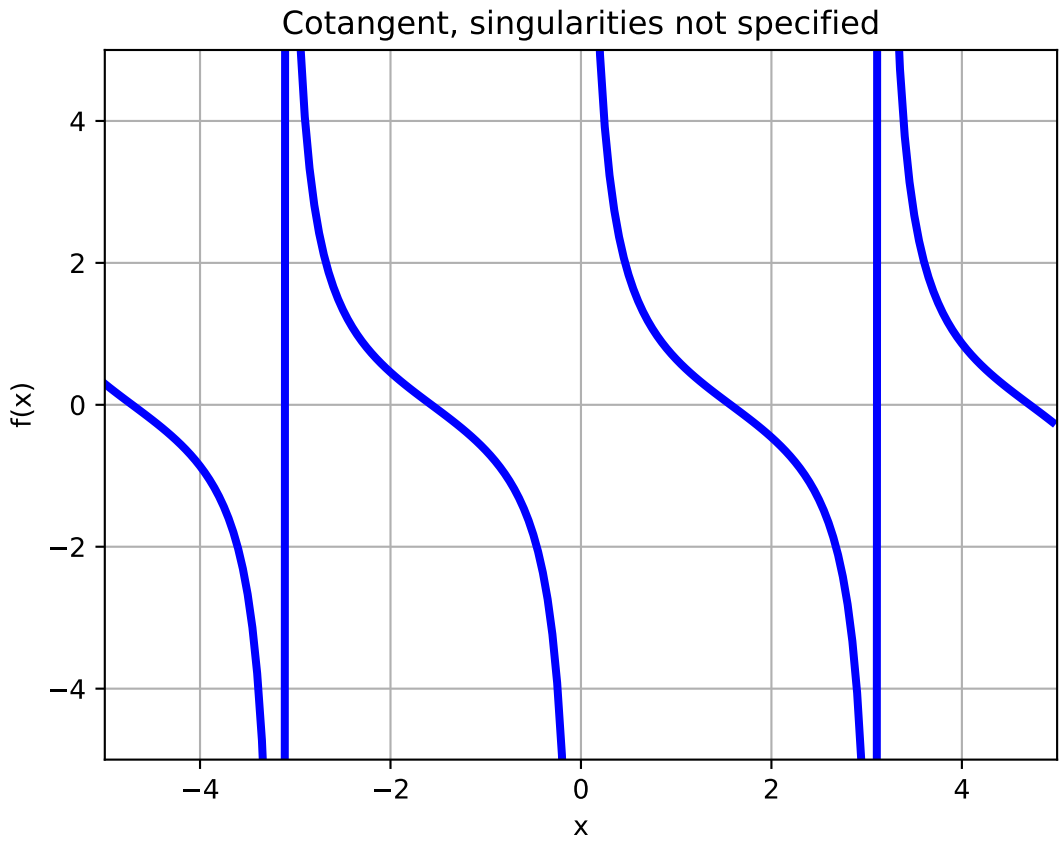
If we plot the `cotangent` without specifying the singularities, we get the results shown below:

To produce the left figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        FuncPlot2d(ctx=mpm, f=mpm.cot, xlim=[-5, 5], ylim=[-5, 5],
            Title = 'Cotangent, singularities not specified, Mpm')
```

To produce the right figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        FuncPlot2d(ctx=math53, f=ctx.cot, xlim=[-5, 5], ylim=[-5, 5],
            Title = 'Cotangent, singularities not specified, Math53')
```



18.1.9.4 Plot of cotangent with singularities specified

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_FuncPlot2d.py.

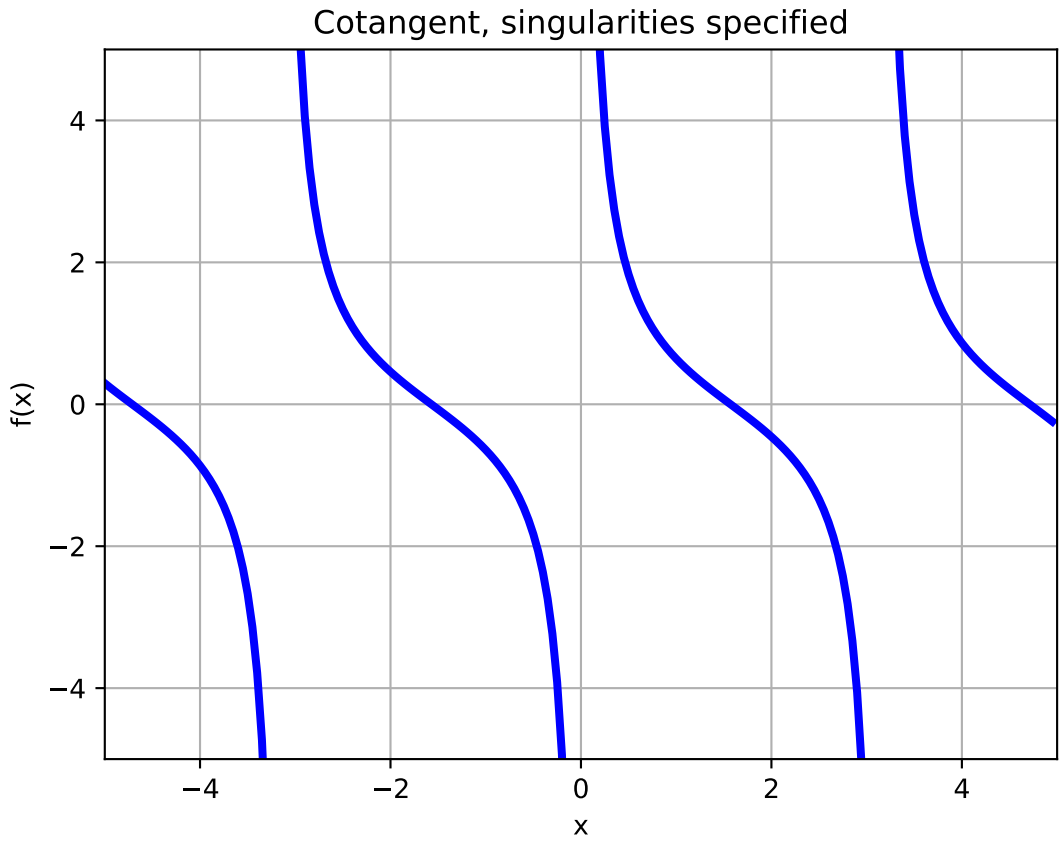
If we plot the `cotangent` and specify the singularities, we get the results shown below:

To produce the left figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        FuncPlot2d(ctx=mpm, f=mpm.cot, xlim=[-5, 5], ylim=[-5, 5],
            singularities=[-mpm.pi, 0, mpm.pi],
            Title = 'Cotangent, singularities specified, mpm')
```

To produce the right figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        FuncPlot2d(ctx=math53, f=ctx.cot, xlim=[-5, 5], ylim=[-5, 5],
            singularities=[-ctx.pi(), 0, ctx.pi()],
            Title = 'Cotangent, singularities specified, math53')
```



18.2 Basic curves

18.2.1 Regular convex polygon

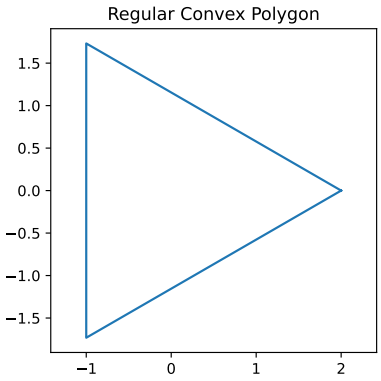
A regular polygon is a polygon that is direct equiangular (all angles are equal in measure) and equilateral (all sides have the same length). Regular polygons may be either convex, star or skew. In the limit, a sequence of regular polygons with an increasing number of sides approximates a circle.

$$r = \frac{a}{\cos\left(t - \alpha/2 - \alpha\,\lfloor\theta/\alpha\rfloor\right)}, \quad \text{where } \alpha = \frac{2\pi}{n}.$$

$$x(t) = r\cos(t)),$$

$$y(t) = r\sin(t).$$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_RegularConvexPolygon.py, and the XML code in the file D01_RegularConvexPolygon.2D.xml.



See also: https://en.wikipedia.org/wiki/Star_polygon

See also: https://mathcurve.com/polyedres/regulier/polygoneregulier.shtml

See also: https://mathcurve.com/courbes2d/goursat/goursat.shtml

See also: https://mathcurve.com/courbes3d.gb/polygramme/polygramme.shtml

See also: https://mathcurve.com/courbes3d.gb/billardcylindrique/billardcylindrique.shtml

18.2.2 Circle

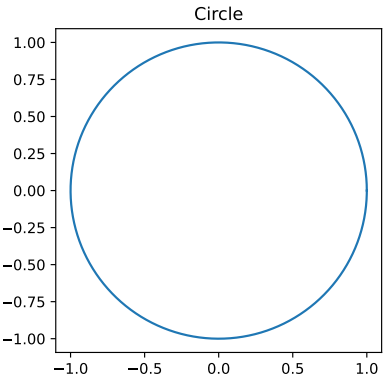
A circle is the set of points in a plane that are equidistant from a given point O. The parametric equations for a circle of radius a can be given by

$$x(t) = a\cos(t)),$$

$$y(t) = a\sin(t).$$

See also Wikipedia [1366], MathWorld [1040], MathCurve [305].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Circle.py, and the XML code in the file D02_Circle.2D.xml.



18.2.3 Ellipse

An ellipse is a plane curve surrounding two focal points, such that for all points on the curve, the sum of the two distances to the focal points is a constant. The parametric equations of a standard ellipse centered at the origin with width $2a$ and height $2b$ can be given by (with $0 \leq t < 2\pi$):

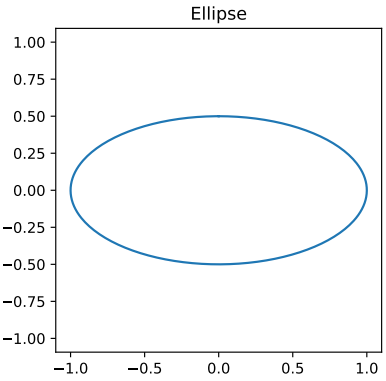
$$x(t) = a\cos(t)),$$

$$y(t) = b\sin(t).$$

Assuming $a \geq b$, the focal points are $(\pm\sqrt{a^2 - b^2}, 0)$.

See also Wikipedia [1371], MathWorld [1042], MathCurve [307].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Ellipse.py, and the XML code in the file D03_Ellipse.2D.xml.



18.2.4 Parabola

A parabola is a plane curve which is mirror-symmetrical and is approximately U-shaped. It fits several superficially different mathematical descriptions, which can all be proved to define exactly the same curves.

The graph of a quadratic function $y = ax^2 + bx + c$ (with $a \neq 0$) is a parabola with its axis parallel to the y-axis. Conversely, every such parabola is the graph of a quadratic function.

The surface of revolution obtained by rotating a parabola about its axis of symmetry is called a paraboloid.

See also: https://mathworld.wolfram.com/Parabola.html

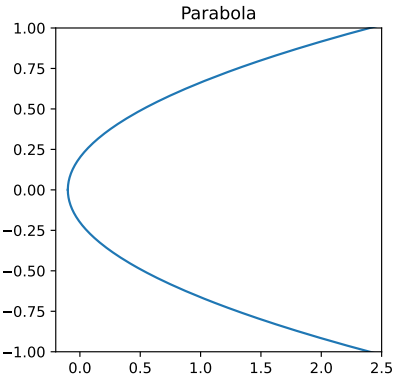
See also: https://en.wikipedia.org/wiki/Parabola

In polar coordinates, the equation of a parabola with parameter a and center (0, 0) is given by $r(t) = -\dfrac{2a}{1 + \cos(t)}$. It can also be written parametrically as

$$x(t) = at^2,$$

$$y(t) = 2at.$$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Parabola.py, and the XML code in the file D04_Parabola.2D.xml.



18.2.5 Hyperbola

In polar coordinates, the equation of a hyperbola centered at the origin (i.e., with $x_0 = y_0 = 0$) is $r2(t) = \dfrac{a^2 b^2}{b^2 \cos^2(t) - a^2 \sin^2(t)}$.

Parametric equations for the right branch of a hyperbola are given by

$$x(t) = a \cosh(t),$$

$$y(t) = b \sinh(t),$$

which ranges over the right branch of the hyperbola.

A parametric representation which ranges over both branches of the hyperbola is

$$x(t) = a \sec(t),$$

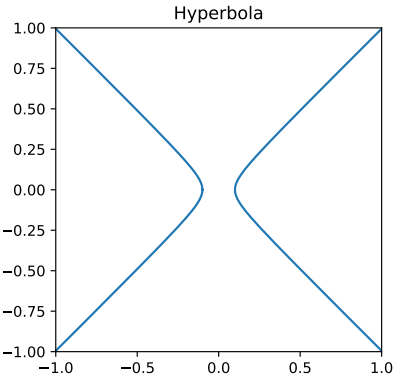
$$y(t) = b \tan(t),$$

with $t \in (-\pi, \pi)$ and discontinuities at $\pm\pi/2$.

See also: https://mathworld.wolfram.com/Hyperbola.html

See also: https://mathworld.wolfram.com/ConicSection.html

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Hyperbola.py, and the XML code in the file D05_Hyperbola.2D.xml.



18.2.6 Cycloid

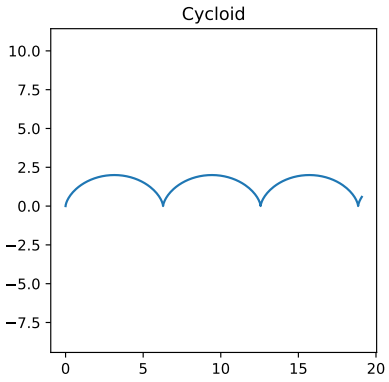
The cycloid is the locus of a point on the rim of a circle of radius a rolling along a straight line. If the cycloid has a cusp at the origin and its humps are oriented upward, its parametric equation is

$$x(t) = a(t - \sin(t)),$$

$$y(t) = a(t - \cos(t)).$$

See also Wikipedia [1370], MathWorld [1041], MathCurve [306].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Cycloid.py, and the XML code in the file D06_Cycloid.2D.xml.



18.2.7 Trochoid

A trochoid is the locus of a point at a distance b from the center of a circle of radius a rolling on a fixed line. A trochoid has parametric equations

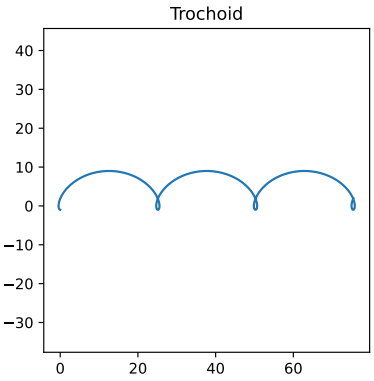
$$x(t) = at - b\sin(t),$$

$$y(t) = at - b\cos(t)).$$

If $b < a$, the trochoid is known as a curtate cycloid; if $b = a$, it is a cycloid; and if $b > a$, the curve is a prolate cycloid.

See also Wikipedia [1394], MathWorld [1053], MathCurve [326].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_Trochoid.py, and the XML code in the file D07_Trochoid.2D.xml.



18.2.8 Cardioid

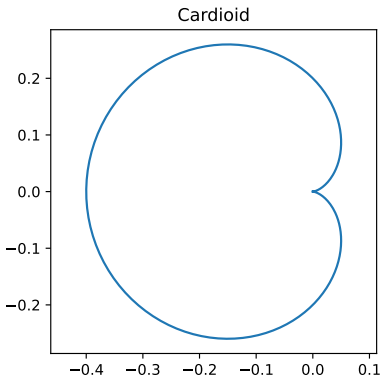
In geometry, a cardioid is a plane curve traced by a point on the perimeter of a circle that is rolling around a fixed circle of the same radius.

The polar equation is given by

$$r = a(1 - \cos(\theta)).$$

See also: Wikipedia [1399], MathWorld [1056]

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_Cardioid.py, and the XML code in the file D08_Cardioid.2D.xml.



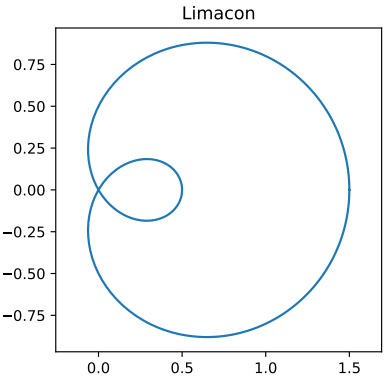
18.2.9 Limaçon curve

Returns the curve given by the polar equation

$$r = b + a\cos(\theta).$$

See also: see also Wikipedia [1413], MathWorld [1063]

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_LimaconCurve.py, and the XML code in the file D09_LimaconCurve.2D.xml.



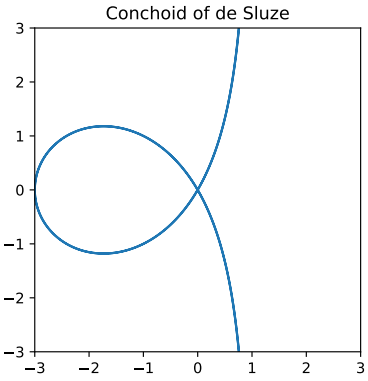
18.2.10 Conchoid of de Sluze

Returns the curve given by the polar equation

$$r = \sec(\theta) + a\cos(\theta).$$

See also: see also Wikipedia [1401], MathWorld [1057]

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_ConchoidOfDeSluze.py, and the XML code in the file D10_ConchoidOfDeSluze.2D.xml.



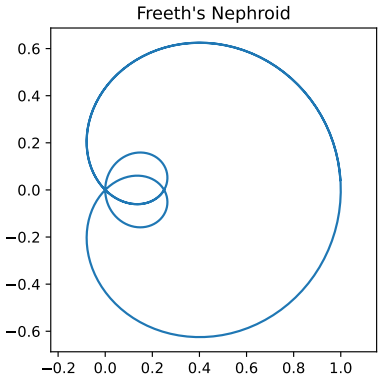
18.2.11 Freeth's Nephroid

The curve has the polar equation

$$r = a\left[1 + 2\sin\left(\tfrac{1}{2}\theta\right)\right]$$

See also: Wikipedia [1408], MathWorld [1061]

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11_FreethNephroid.py, and the XML code in the file D11_FreethNephroid.2D.xml.



18.2.12 Strophoid

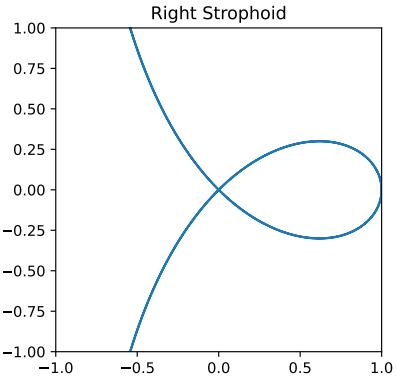
The name strophoid means "belt with a twist". The polar form for a general strophoid is

$$r = \frac{b \sin(a - 2\theta)}{\sin(a - \theta)}$$

See also: Wikipedia [1419], MathWorld [1067]

A special cases is the right strophoid with $a = \pi/2$.

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D12_Strophoid.py, and the XML code in the file D12_Strophoid.2D.xml.



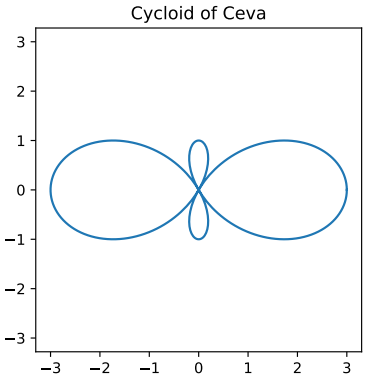
18.2.13 Cycloid of Ceva

A curve with the polar form

$$r = 1 + 2\cos(2\theta).$$

See also: Wikipedia [1402], MathWorld [1058]

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D13_CycloidOfCeva.py, and the XML code in the file D13_CycloidOfCeva.2D.xml.



18.2.14 Lemniscate of Gerono

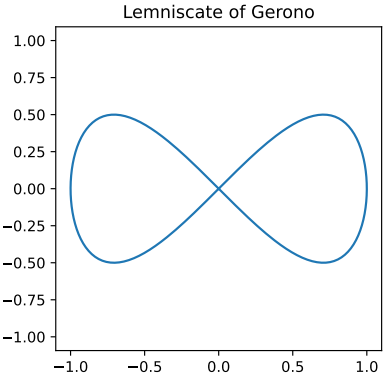
This curve is also known as Eight curve. It has parametric equations

$$x(t) = a\sin(t),$$

$$y(t) = x(t)\cos(t)).$$

See also Wikipedia [1385], MathWorld [1050], MathCurve [318].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D14_LemniscateOfGerono.py, and the XML code in the file D14_LemniscateOfGerono.2D.xml.



18.2.15 Lemniscate of Bernoulli

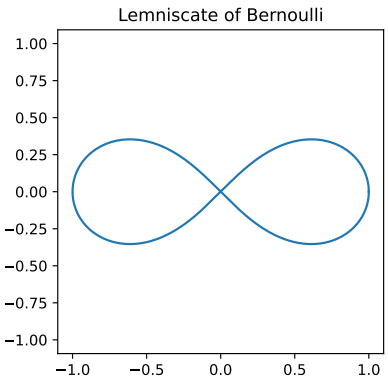
The lemniscate of Bernoulli is a polar curve defined as the locus of points such that the the product of distances from two fixed points $(-a, 0)$ and $(a, 0)$ is a constant a^2. It has parametric equations

$$x(t) = \frac{a\cos(t)}{1 + \sin^2(t)},$$

$$y(t) = x(t)\sin(t)).$$

See also Wikipedia [1384], MathWorld [1049], MathCurve [317].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D15_LemniscateOfBernoulli.py, and the XML code in the file D15_LemniscateOfBernoulli.2D.xml.



18.3 Spirals

18.3.1 Archimedes' spiral

The parametric equations of Archimedes' Spiral are

$$r(t) = at,$$

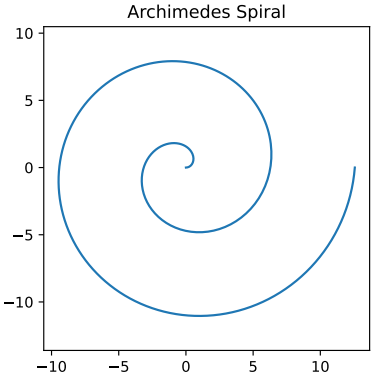
$$x(t) = r\cos(t),$$

$$y(t) = r\sin(t).$$

Changing the parameter a controls the distance between loops.

See also Wikipedia [1397], MathWorld [1054], MathCurve [302].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_ArchimedesSpiral.py, and the XML code in the file D01_ArchimedesSpiral.2D.xml.



18.3.2 Fermat's spiral

The parametric equations of Fermat' spiral are

$$r(t) = \pm a\sqrt{t},$$

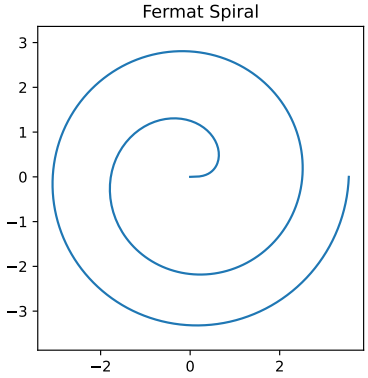
$$x(t) = r\cos(t),$$

$$y(t) = r\sin(t).$$

Changing the parameter a controls the distance between loops.

See also Wikipedia [1407], MathWorld [1060], MathCurve [1407].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_FermatSpiral.py, and the XML code in the file D02_FermatSpiral.2D.xml.



18.3.3 Hyperbolic Spiral

The parametric equations of the hyperbolic spiral are

$$r(t) = \frac{a}{t},$$

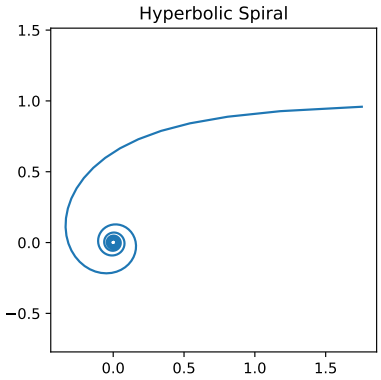
$$x(t) = r\cos(t),$$

$$y(t) = r\sin(t).$$

Changing the parameter a controls the distance between loops.

See also Wikipedia [1411], MathWorld [1062], MathCurve [313].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_HyperbolicSpiral.py, and the XML code in the file D03_HyperbolicSpiral.2D.xml.



18.3.4 Lituus

The parametric equations of the Lituus are

$$r(t) = \frac{a}{\sqrt{t}},$$

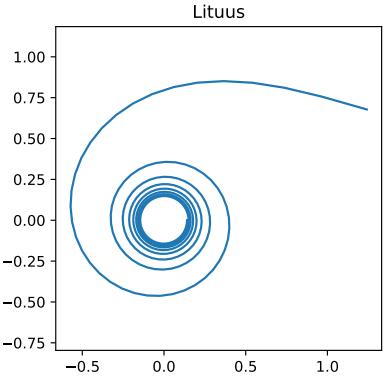
$$x(t) = r\cos(t),$$

$$y(t) = r\sin(t).$$

Changing the parameter a controls the distance between loops.

See also Wikipedia [1414], MathWorld [1064], MathCurve [321].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Lituus.py, and the XML code in the file D04_Lituus.2D.xml.



18.3.5 Logarithmic Spiral

The parametric equations of the logarithmic spiral are

$$r(t) = a\exp(kt),$$

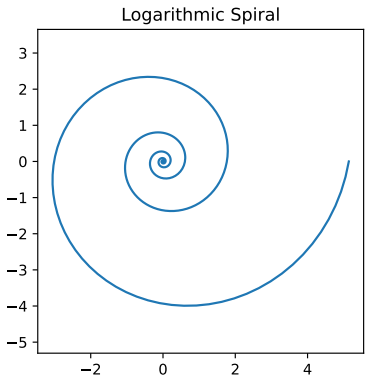
$$x(t) = r\cos(t),$$

$$y(t) = r\sin(t).$$

where $a > 0$ and $k \neq 0$ are real constants.

See also Wikipedia [1415], MathWorld [1065], MathCurve [322].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_LogarithmicSpiral.py, and the XML code in the file D05_LogarithmicSpiral.2D.xml.



18.3.6 Poinsot's Spiral

The parametric equations of the Poinsot spiral are

$$r(t) = \frac{a}{\alpha \cosh(kt) + \beta \sinh(kt)}, \quad \text{with } \alpha^2 + \beta^2 \neq 0.$$

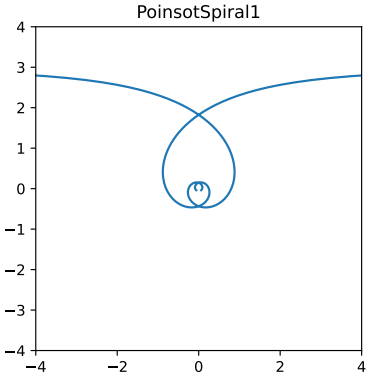
$$x(t) = r \cos(t),$$

$$y(t) = r \sin(t).$$

where $a > 0$ and $k \neq 0$ are real constants.

See also Wikipedia [1448], MathWorld [1077], MathCurve [332].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_PoinsotSpiral.py, and the XML code in the file D06_PoinsotSpiral.2D.xml.



18.3.7 Cotes's spiral

The shape of spirals in the family depends on the parameters. The curves in polar coordinates, $(r, \theta), r > 0$ are defined by one of the following five equations:

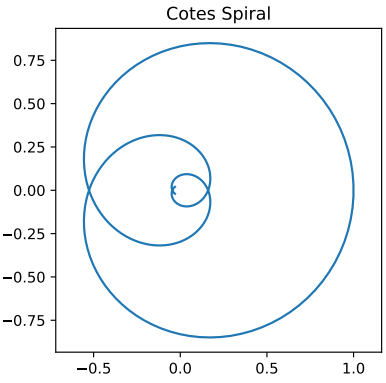
$$\frac{1}{r} = \begin{cases} A\cosh(k\theta + \varepsilon) \\ A\exp(k\theta + \varepsilon) \\ A\sinh(k\theta + \varepsilon) \\ A(k\theta + \varepsilon) \\ A\cos(k\theta + \varepsilon) \end{cases}$$

$A > 0$, $k > 0$ and ε are arbitrary real constants. A determines the size, k determines the shape, and ε determines the angular position of the spiral.

The first and third forms are Poinsot's spirals; the second is the logarithmic spiral; the fourth is the hyperbolic spiral; the fifth is the epispiral.

See also Wikipedia [1443], MathWorld [1072].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_CotesSpiral.py, and the XML code in the file D07_CotesSpiral.2D.xml.



18.3.8 Tanh Spiral

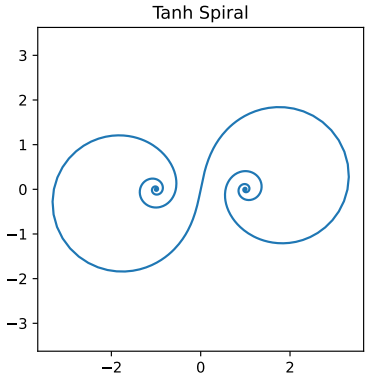
This is a spiral proposed by Bourke [34], and also discussed by Meier [413]. It has the following parametric equations:

$$x(t) = \frac{\sinh(2t)}{\cos(2at) + \cosh(2t)},$$

$$y(t) = \frac{\sin(2at)}{\cos(2at) + \cosh(2t)},$$

with $-\pi/2 \leq t \leq \pi/2$ and $a > 0$ (eg:2).

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_TanhSpiral.py, and the XML code in the file D08_TanhSpiral.2D.xml.



18.3.9 Nielsens Spiral

Nielsen's spiral, also called the sici spiral is the spiral with parametric equations

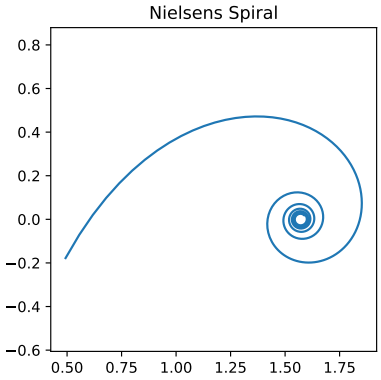
$$x(t) = a \cdot \mathrm{ci}(t),$$

$$y(t) = a \cdot \mathrm{si}(t),$$

where $\mathrm{ci}(t)$ is the cosine integral and $\mathrm{si}(t)$ is the sine integral.

See also Wikipedia [1447], MathWorld [1076], MathCurve [335].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_NielsenSpiral.py, and the XML code in the file D09_NielsenSpiral.2D.xml.



18.3.10 Cornu Spiral

The Cornu spiral, also known as a Euler spiral or clothoid, is the curve generated by a parametric plot of S(t) against C(t). A Cornu spiral has the property that its curvature at any point is proportional to the distance along the spiral, measured from the origin.

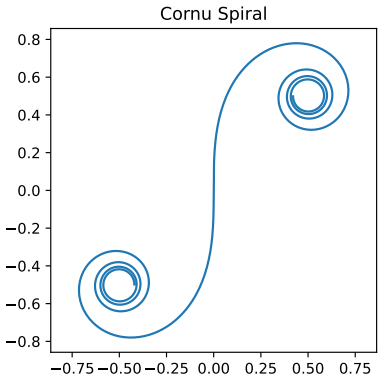
$$x(t) = a \cdot C(t),$$

$$y(t) = a \cdot S(t),$$

where $C(t)$ and $S(t)$ are the Fresnel integrals.

See also Wikipedia [1446], MathWorld [1071], MathCurve [328].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_CornuSpiral.py, and the XML code in the file D10_CornuSpiral.2D.xml.



18.4 Decorative curves

18.4.1 Fish curve

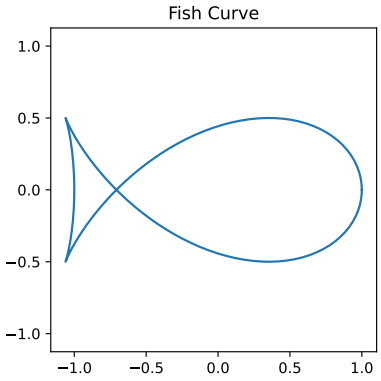
The fish curve has the parametric equation

$$x(t) = a\cos(t) - \frac{a\sin^2(t)}{\sqrt{2}},$$

$$y(t) = a\cos(t)\sin(t).$$

See also Wikipedia [1375], MathWorld [1045], MathCurve [311].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_FishCurve.py, and the XML code in the file D01_FishCurve.2D.xml.



18.4.2 Heart curve

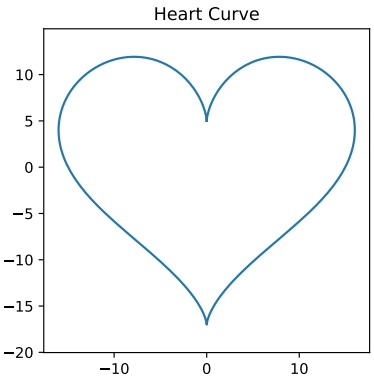
The heart curve has the parametric equation

$$x(t) = 16\sin^3(t),$$

$$y(t) = 13\cos(t) - 5\cos(2t) - 2\cos(3t) - \cos(4t)$$

See also Wikipedia [1377], MathWorld [1046].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_HeartCurve.py, and the XML code in the file D02_HeartCurve.2D.xml.



18.4.3 Chrysanthemum curve

The chrysanthemum curve (see Bourke [33]) is given in polar coordinates by the following

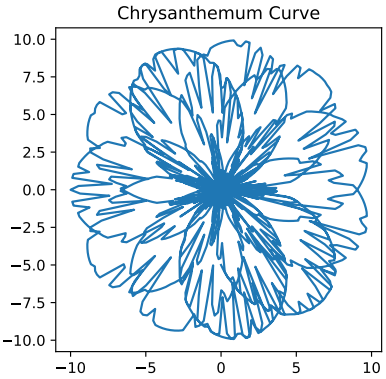
$$r = 5(1 + \sin(11t/5)) - 4\sin^4(17t/3)\sin^8(2\cos(3t) - 28t) \quad \text{where } 0 \leq t \leq 21\pi$$

and in Cartesian coordinates

$$x = r\cos(t)$$

$$y = r\sin(t)$$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_ChrysanthemumCurve.py, and the XML code in the file D03_ChrysanthemumCurve.2D.xml.



18.4.4 Butterfly curve

The butterfly curve is given in polar coordinates by the following

$$r = \exp(\cos(t)) - 2\cos(4t) - \sin^5(t/12)$$

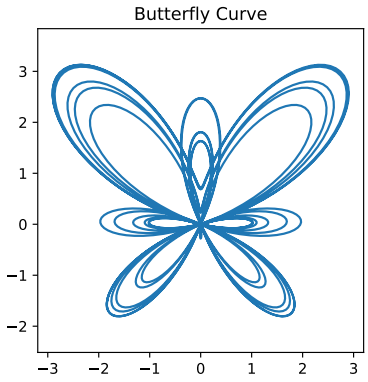
and in Cartesian coordinates

$$x = r\cos(t)$$

$$y = r\sin(t)$$

See also Wikipedia [1398], MathWorld [1055].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_ButterflyCurve.py, and the XML code in the file D04_ButterflyCurve.2D.xml.



18.5 General curve families

18.5.1 Regular star polygon

There exist infinitely many regular star polytopes in two dimensions, whose Schläfli symbols consist of rational numbers $\{n/m\}$. They are called star polygons and share the same vertex arrangements of the convex regular polygons.

$$r = \frac{a}{\cos\left(t - \alpha/2 - \alpha \left\lfloor \theta/\alpha \right\rfloor\right)}, \quad \text{where } \alpha = \frac{2m\pi}{n}.$$

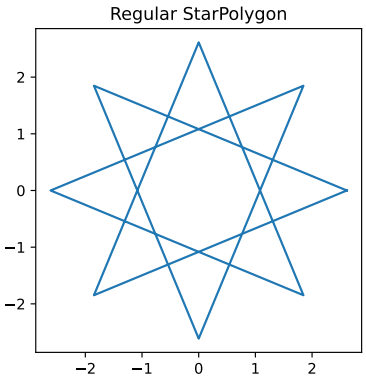
$$x(t) = r\cos(t)),$$

$$y(t) = r\sin(t).$$

See also: https://en.wikipedia.org/wiki/Star_polygon

See also: https://mathworld.wolfram.com/StarPolygon.html

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_RegularStarPolygon.py, and the XML code in the file D01_RegularStarPolygon.2D.xml.



18.5.2 Regular compound polygon (polygram)

The n vertices of the regular polygon being determined, all the associated regular polygons are obtained by joining the vertices from m to m, where m is coprime with n and between 1 and $n/2$. The case $m = 1$ gives the only uncrossed polygon, which is convex.

$$r = \frac{a}{\cos\left(t - \alpha/2 - \alpha\,\lfloor\theta/\alpha\rfloor\right)}, \quad \text{where } \alpha = \frac{2m\pi}{n}.$$

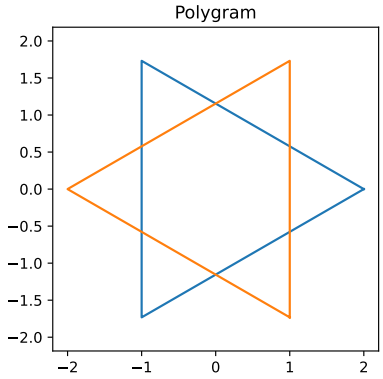
$$x(t) = r\cos(t)),$$

$$y(t) = r\sin(t).$$

See also: https://en.wikipedia.org/wiki/Polygram_(geometry)#Regular_compound_polygons

See also: https://mathworld.wolfram.com/Polygram.html

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Polygram.py, and the XML code in the file D02_Polygram.2D.xml.



18.5.3 Epicycloid

The epicycloids are the curves described by a point on a circle (C) rolling without slipping on a base circle (C_0), the open disks with boundaries (C) and (C_0) being disjoint. Therefore, they are special cases of epitrochoids. An epicycloid has parametric equations

$$qx(t) = a\left((q+1)\cos(t) - \cos\left((q+1)t\right)\right),$$

$$qy(t) = a\left((q+1)\sin(t) - \sin\left((q+1)t\right)\right).$$

See also Wikipedia [1372], MathWorld [1043], MathCurve [308].

The epicycloids are curves composed of isometric arcs (the arches) joining at cuspidal points (obtained for $t = \dfrac{2\pi}{q}$) in a finite number equal to the numerator of q if q is rational, or in an infinite number otherwise.

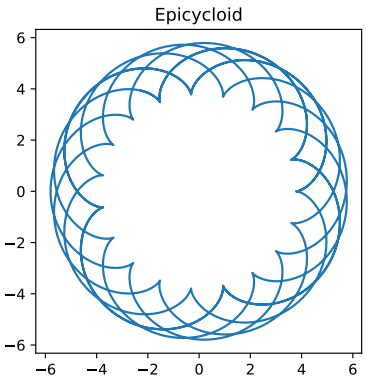
When q is rational, $q = \dfrac{n}{m}$, the curve is algebraic and rational (take $u = \tan\left(\dfrac{t}{2m}\right)$ as a parameter).

Its looks like a regular polygon, crossed if $m \geq 2$, with n vertices joined by m points by the curves located outside the circle (C_0).

The notation of simple epicycloid with n cusps (E_n) refers to the case $q = n$, i.e. when there are no crossovers.

Special cases are the cardioid $(q = 1)$, the nephroid $(q = 2)$, and the double cardioid $(q = 1/2)$,

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Epicycloid.py, and the XML code in the file D03_Epicycloid.2D.xml.



18.5.4 Hypocycloid

The hypocycloids are the curves described by a point on a circle (C) rolling without slipping on, and inside, a base circle (C_0), when the rolling circle is smaller than the fixed one. Therefore, they are special cases of hypotrochoids. A hypocycloid has parametric equations

$$qx(t) = a\left((q-1)\cos(t) - \cos\left((q-1)t\right)\right),$$

$$qy(t) = a\left((q-1)\sin(t) - \sin\left((q-1)t\right)\right).$$

See also Wikipedia [1378], MathWorld [1047], MathCurve [314].

The hypocycloids are curves composed of isometric arcs (the arches) connecting at cuspidal points (obtained for $t = \dfrac{2\pi}{q}$) in a finite number equal to the numerator of q if q is rational, or in an infinite number otherwise.

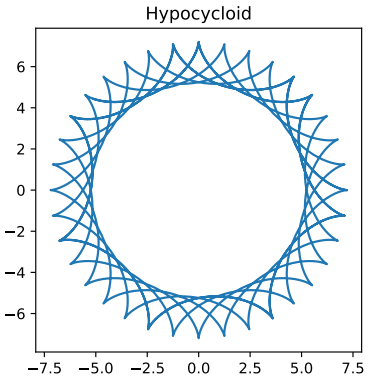
When q is rational, $q = \dfrac{n}{m}$, the curve is algebraic and rational (take $u = \tan\left(\dfrac{t}{2m}\right)$ as a parameter).

It has the same structure as a regular polygon, crossed if $m \geq 2$, with n vertices linked from m to m by curves located inside the circle (C_0).

The notation of simple hypocycloid with n cusps (E_n) refers to the case $q = n$, i.e. when there are no crossings.

Special cases are the point $(q = 1)$, the La Hire line $(q = 2)$, the deltoid $(q = 3)$, and the astroid $(q = 4)$.

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Hypocycloid.py, and the XML code in the file D04_Hypocycloid.2D.xml.



18.5.5 Epitrochoid

The epitrochoids are the curves described by a point linked to a circle (C) rolling without slipping on a base circle (C_0), the open disks with boundaries (C) and (C_0) being disjoint. An epitrochoid has parametric equations

$$qx(t) = a\left((q+1)\cos(t) - k\cos\left((q+1)t\right)\right),$$

$$qy(t) = a\left((q+1)\sin(t) - k\sin\left((q+1)t\right)\right).$$

See also Wikipedia [1373], MathWorld [1044], MathCurve [310].

For $d = a + b$, we get the roses.

For $k < 1$, the curve is also called shortened epicycloid.

For $k > 1$, the curve is also called elongated epicycloid.

The limit case is $k = \dfrac{1}{q+1}$; there is then a flat point.

For $\dfrac{1}{q+1} < k < 1$, the curve undulates, with points of inflection.

For $k = 1$, we obtain the epicycloid, with cusps.

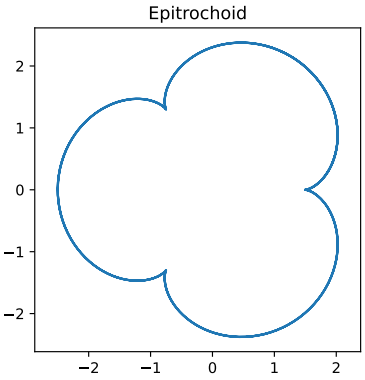
For $1 < k < q + 1$, the curve makes loops, with concave and convex portions, (positive angular speed, then negative alternately).

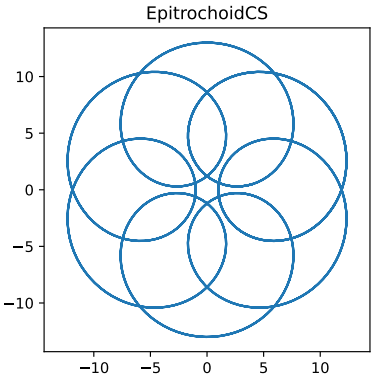
For $k = q + 1$, we get a rose with index $n < 1$, of polar equation $\rho = 2d\cos\dfrac{q}{q+2}\theta$.

The case $k > q - 1$ leads to a more complicated pattern.

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Epitrochoids.py, and the XML code in the file D05_Epitrochoids.2D.xml.

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_EpitrochoidCS.py, and the XML code in the file D05_EpitrochoidCS.2D.xml.





18.5.6 Hypotrochoid

The hypotrochoids are the curves described by a point linked to a circle (C) rolling without slipping internally on a base circle $(C0)$. A hypocycloid has parametric equations

$$qx(t) = a\left((q-1)\cos(t) - k\cos\left((q-1)t\right)\right),$$

$$qy(t) = a\left((q-1)\sin(t) - k\sin\left((q-1)t\right)\right).$$

See also Wikipedia [1379], MathWorld [1048], MathCurve [315].

For $k = 1$, we get the hypocycloids.

If a is fixed but q is changed into $\dfrac{q}{q-1}$ and k into $\dfrac{1}{k}$, then the hypotrochoid obtained is the homothetic image of the initial one with ratio k. Therefore, we get all the hypotrochoids by considering only the case $q \geq 2$.

For $q = 2$, we get the ellipses.

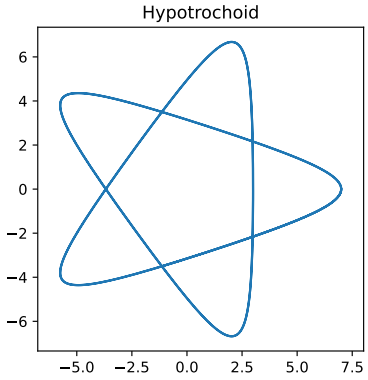
For $q > 2$, the curve is also called curtate hypocycloid if $k < 1$, and prolate hypocycloid if $k > 1$.

However, according to the preceding paragraph, in the case $1 < q < 2$, the curtate hypocycloids are paradoxically obtained for $k > 1$ and the prolate ones for $k < 1$.

For $k = q - 1$, we get a rose of index $n > 1$.

The hypocycloid with parameter $q = n/m$ constitutes a "rounded" approximation of the regular polygon of type (n, m); for the portions between two vertices to be as linear as possible, we can consider the limit case where this portion does not have an inflexion point, which corresponds to the case $k = 1/(q-1)$.

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Hypotrochoid.py, and the XML code in the file D06_Hypotrochoid.2D.xml.



18.5.7 Rose curve

A rose curve, also called Grandi's rose or the multifolium, is a curve which has the shape of a petalled flower. The polar equation of the rose is generally given as $r = a\cos(nt)$ or by the version rotated by 90 degrees, $r = a\sin(nt)$. The sine version has the advantage that roses with odd n have a petal oriented vertically (up or down depending on n), whereas the cosine orientation gives a petal oriented to the right.

See also Wikipedia [1417], MathWorld [1066], MathCurve [323].

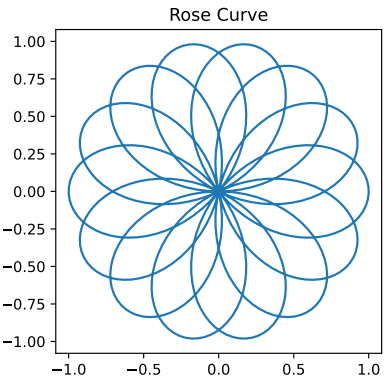
If n is odd, the rose is n-petalled. If n is even, the rose is $2n$-petalled.

The curve is algrebraic iff $n = p/q$ is rational, with degree $p + q$ when pq is odd and $2(p + q)$ when pq is even.

If $n = p/q$ is a rational number, then the curve closes at a polar angle of $\theta = \pi qm$, where $m = 1$ if pq is odd and $m = 2$ if pq is even. If n is irrational, then there are an infinite number of petals. For $n > 1$ the curve is a hypotrochoid, and for $0 < n < 1$ it is an epitrochoid. The roses are views from above of the clelias.

Special cases are the limaçon trisectrix ($n = 1/3$), the Dürer folium ($n = 1/2$), the circle ($n = 1$), the quadrifolium ($n = 2$) and the trifolium ($n = 3$).

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_RoseCurve.py, and the XML code in the file D07_RoseCurve.2D.xml.



18.5.8 Lissajous curves

Lissajous curves are the family of curves described by the parametric equations

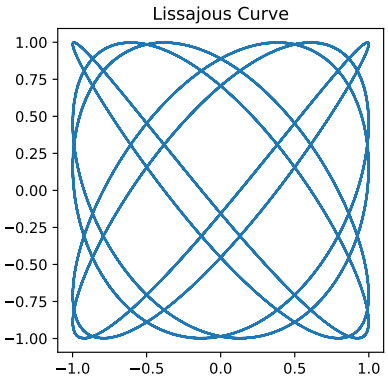
$$x(t) = a\sin(\omega t + \delta),$$

$$y(t) = b\sin(t).$$

See also Wikipedia [1386], MathWorld [1051], MathCurve [320].

Special cases are the line ($\omega = 1, \delta = 0$), the circle ($a = b, \omega = 1, \delta = \pi/2$), the ellipse ($a = b, \omega = 1, \delta = \pi/2$), the section of a parabola ($\omega = 2, \delta = \pi/2$).

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_LissajousCurve.py, and the XML code in the file D08_LissajousCurve.2D.xml.



18.5.9 Superellipses (Lamé curves)

Superellipses are the family of curves described by the parametric equations

$$x(t) = |\cos(t)|^{2/r} \cdot a \cdot \mathrm{sgn}(\cos(t)),$$

$$y(t) = |\sin(t)|^{2/r} \cdot b \cdot \mathrm{sgn}(\sin(t)),$$

with $0 \leq t \leq \pi/2$.

See also Wikipedia [1392], MathWorld [1052], MathCurve [316].

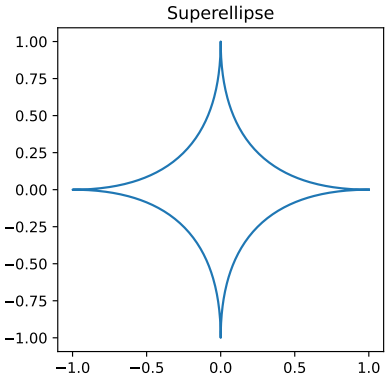
> The restriction to r>2 is sometimes made.

The generalization to a three-dimensional surface is known as a superellipsoid.

Superellipses with a=b are also known as Lamé curves or Lamé ovals, and the case $a = b$ with $r = 4$ is sometimes known as the squircle.

Special cases are the astroid ($r = 2/3$), the diamond ($r = 1$), the ellipse ($r = 2$).

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_Superellipses.py, and the XML code in the file D09_Superellipses.2D.xml.



18.5.10 Epispirals

The epispirals are solutions of the problem that consists in determining the trajectories in space of a massive point subject to a force centred on O proportional to $\frac{1}{\rho^3}$ (this force is, thanks to the Binet formula, proportional to $u^2(u+u'')$ which is here $(1+n^2)u^2$, with $u=\frac{1}{\rho}$); the other solutions are the Poinsot spirals, with intermediary case the hyperbolic spiral.

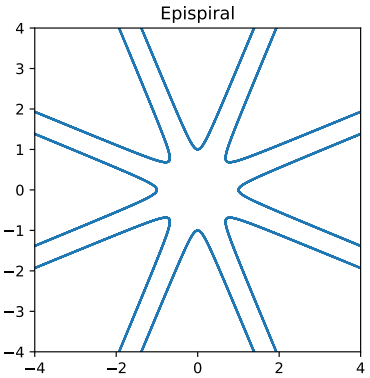
The curve is composed of an infinite branch obtained for $-\frac{\pi}{2n} < 0 < \frac{\pi}{2n}$, and of all its images by rotations by angle $k\left(\pi+\frac{\pi}{n}\right)$ for integer values of k. When n is a rational number with numerator p, and denominator q, the curve is symmetrical about O iff p or q is even.

In this case, the curve is composed of $2p$ branches derived from the initial branch by rotations by angles $\frac{2k\pi}{n}$ and $\frac{2k\pi}{n}+\pi$.

Special cases are the line ($n=1$), the crosscurve ($n=2$), the rectangukar trefoil ($n=3$), the Delanges trisectrix ($n=1/2$), the MacLaurin trisectrix ($n=1/3$).

See also Wikipedia [1406], MathWorld [1059], MathCurve [309].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_Epispirals.py, and the XML code in the file D10_Epispirals.2D.xml.



18.5.11 Nodal curves (stereographic projections of the clelia)

Polar equation: $\rho = a\tan(n\theta)$.

The nodal curves are the Brocard transforms of the Kappa, when the pole is at the centre of the Kappa. Each curve is composed of an infinite branch, the base, obtained for $-\frac{\pi}{2n} < 0 < \frac{\pi}{2n}$, and all its images by the rotations of angle $k\frac{\pi}{n}$ when k is an integer.

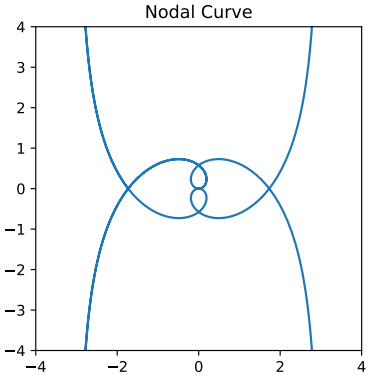
If n is rational and its numerator is p and its denominator q, then the curve is composed of 2p branches, images of the base branch by rotation when q is odd, and p branches when it is even.

Special cases are the Kappa ($n = 1$), the windmill ($n = 2$),the right strophoid ($n = 1/2$).

All nodal curves are stereographic projections of the clelia. The inverse (with centre O and radius a^2) of a nodal curve is the same curve turned by $\frac{\pi}{2n}$.

See also: MathCurve [331].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11_NodalCurves.py, and the XML code in the file D11_NodalCurves.2D.xml.



18.5.12 Cyclic-harmonic curves

Polar equation: $\rho = a(1 + e\cos(n\theta))$.

The cyclic-harmonic curves are the conchoids of roses with respect to their centre. The curve is composed of a base pattern, symmetrical about Ox, obtained for $-\frac{\pi}{n} \leq 0 \leq \frac{\pi}{n}$, transformed by all the rotations of $2k\frac{\pi}{n}$ for integer values of k. When n is a rational number and p is its numerator, p rotations give the whole curve.

Case $e < 1$: For $n = p/q$, the cyclic-harmonic curve of parameter n is one of the possible projections of the Turk's head knot of type (p, q); its p external summits and its p internal vertices form a regular polygon, and it has $p(q - 1)$ double points.

Special cases are the trefoil knot ($n = 3/2$), the projection of the 5.1 knot ($n = 5/2$) the projection of the figure-eight knot ($n = 2/3$).

Case $e = 1$:

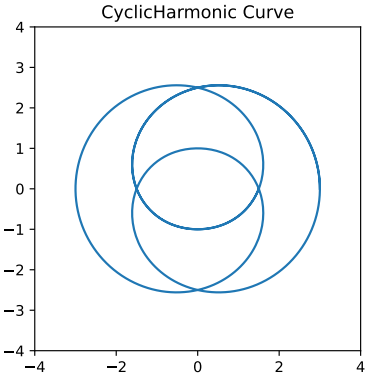
Special cases are the cardioid for $n = 1$ (see also Wikipedia [1399], MathWorld [1056], MathCurve [303]), the double egg ($n = 2$)

Case $e > 1$:

Special cases are the limaçon of Pascal for $n = 1$ (see also Wikipedia [1413], MathWorld [1063], MathCurve [319]), the cycloid (or trisectrix) of Ceva for $n = 2$ (see also Wikipedia [1402], MathWorld [1058], MathCurve [304]), and Freeth's nephroid for $n = 1/2$ (see also Wikipedia [1408], MathWorld [1061], MathCurve [312]).

See also: MathCurve [329].

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D12_CyclicHarmonicCurves.py, and the XML code in the file D12_CyclicHarmonicCurves.2D.xml.



18.5.13 Rational circular cubics

The rational circular cubics have the following parametrization (see MathCurve [333]):

$$x(t) = \frac{dt^2 + 2bt + 2a + d}{1 + t^2}$$

$$y(t) = \frac{dt^3 + 2bt^2 + 2at + dt}{1 + t^2} = tx(t).$$

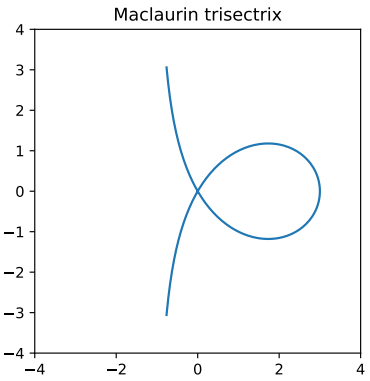
They have a real singularity (see Wikipedia [1449]), which is necessarily unique.

- For $\sqrt{a^2 + b^2} > |a+d|$, the cubic is called crunodal, i.e. this singularity is a double point with different tangents (see Wikipedia [1444], MathWorld [1073]).
- For $\sqrt{a^2 + b^2} = |a+d|$, the cubic is called cuspidal, i.e. this singularity is a cuspidal point (see Wikipedia [1445], MathWorld [1074]).
- For $\sqrt{a^2 + b^2} < |a+d|$, the cubic is called acnodal, i.e. this singularity is an isolated point (see Wikipedia [1441], MathWorld [1069]).

Many other parametric curves can be obtained as special cases:

- For $b = 0$, we get the right rational circular cubics (see also: MathCurve [334]).
- For $\sqrt{a^2 + b^2} = |a + d|$, we get the cissoids (see Wikipedia [1442], MathWorld [1070], MathCurve [327]).
- For $d = -a$, we get the strophoids (see Wikipedia [1419], MathWorld [1067], MathCurve [325]).
- For $a = -2d, b = 0$, we get the Maclaurin trisectrix (see Wikipedia [1450], MathWorld [1075], MathCurve [330]).
- In the right and acnodal case (i.e. b = 0, d > 0), we get the cubic (or conchoid) of de Sluze (see Wikipedia [1401], MathWorld [1057], MathCurve [324]).

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D13_RationalCircularCubics.py, and the XML code in the file D13_RationalCircularCubics.2D.xml.



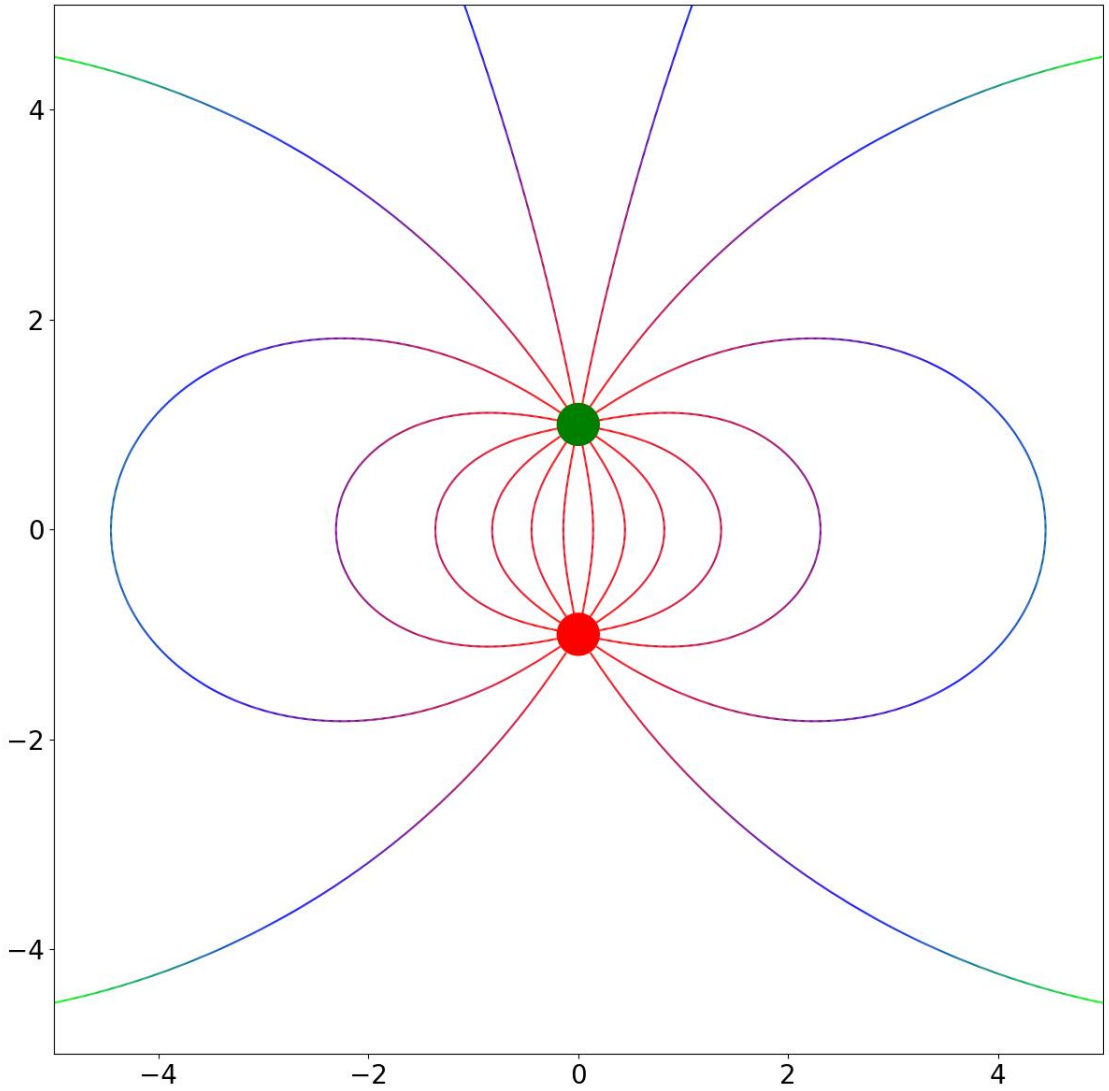
18.6 Field lines

18.6.1 Colored electric field lines: 1 positive vs 1 negative charge

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_4_ColoredFieldLines.py, and the XML code in the file D01_4_ColoredFieldLines.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        UseColors_ = True
        Model_ = 1  # 1, 2, 3, 4
        ColoredFieldLines(UseColors=UseColors_, Model=Model_)
```

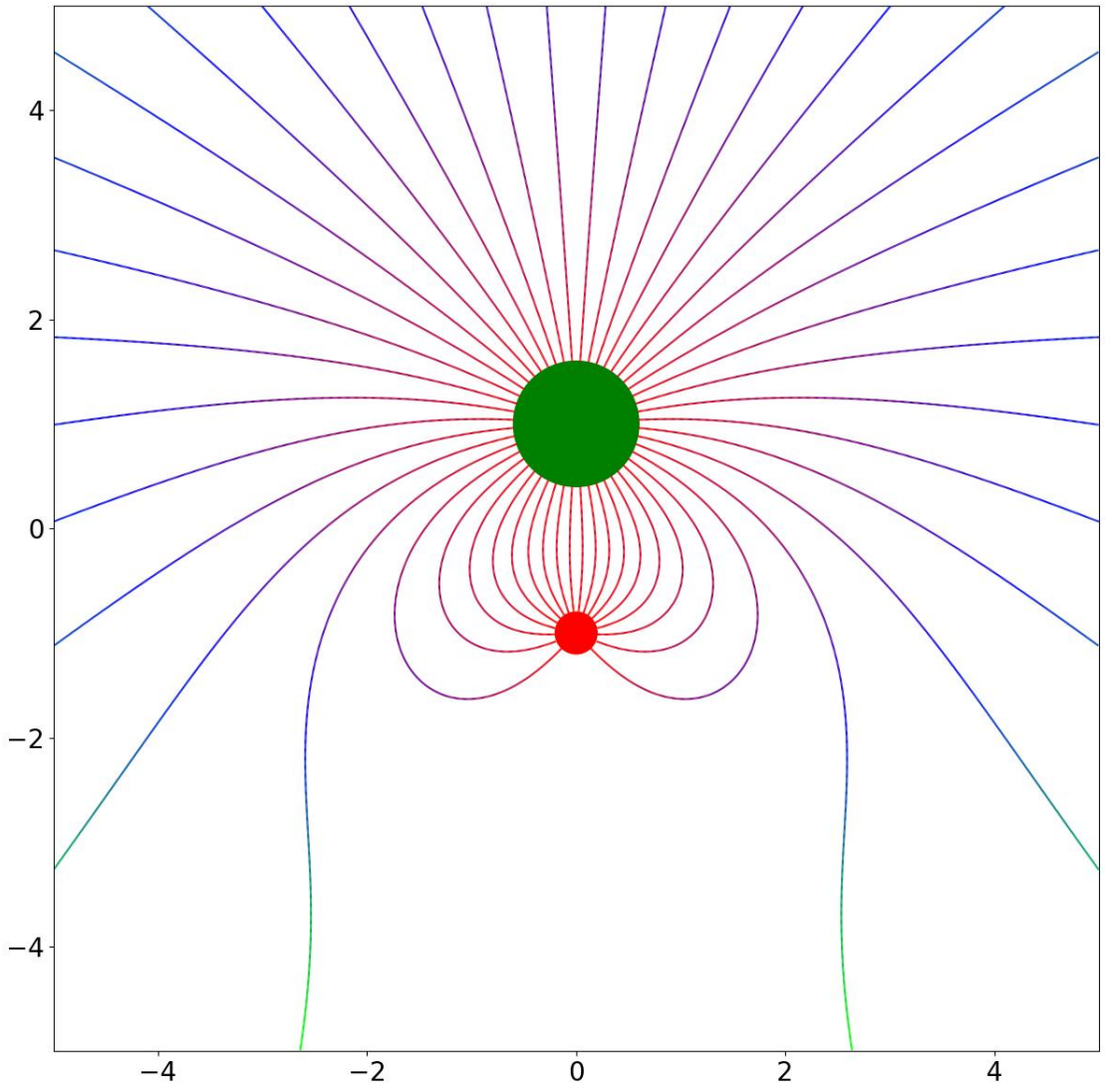


18.6.2 Colored electric field lines: 3 positive vs 1 negative charge(s)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_4_ColoredFieldLines.py, and the XML code in the file D01_4_ColoredFieldLines.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        UseColors_ = True
        Model_ = 2  # 1, 2, 3, 4
        ColoredFieldLines(UseColors=UseColors_, Model=Model_)
```

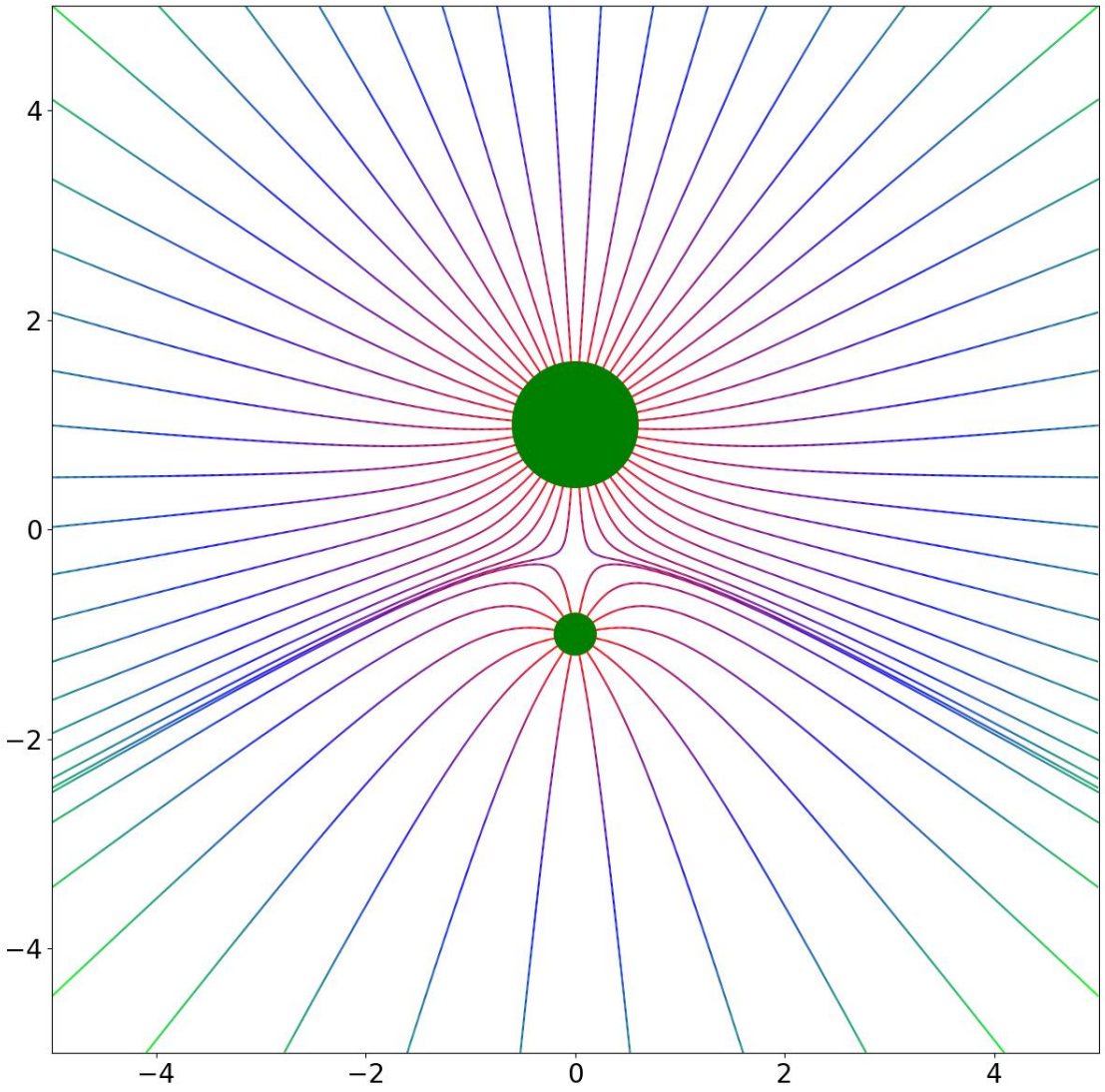


18.6.3 Colored electric field lines: 3 positive vs 1 positive charge(s)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_4_ColoredFieldLines.py, and the XML code in the file D01_4_ColoredFieldLines.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        UseColors_ = True
        Model_ = 3  # 1, 2, 3, 4
        ColoredFieldLines(UseColors=UseColors_, Model=Model_)
```

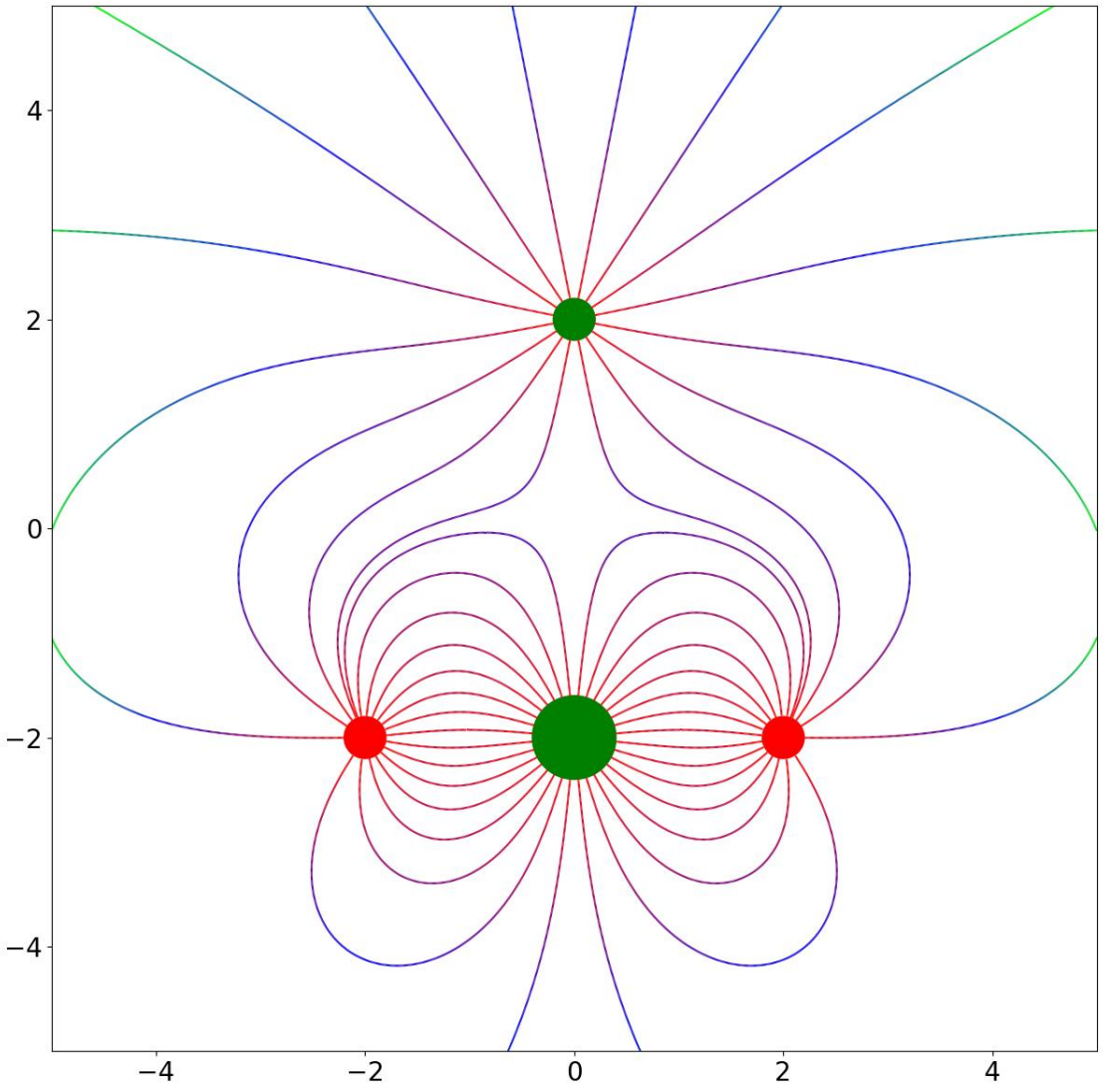


18.6.4 Colored electric field lines: 3 positive and 2 negative vs 1 positive charge(s)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_4_ColoredFieldLines.py, and the XML code in the file D01_4_ColoredFieldLines.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

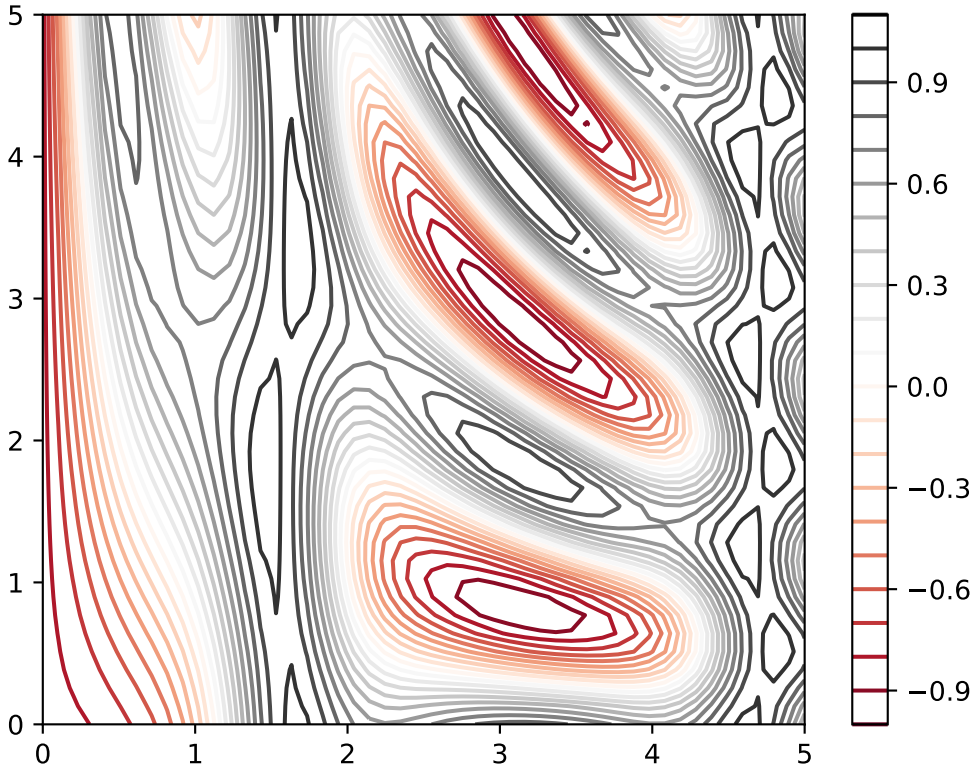
```
try:
    if __name__ == '__main__':
        UseColors_ = True
        Model_ = 4  # 1, 2, 3, 4
        ColoredFieldLines(UseColors=UseColors_, Model=Model_)
```



18.7 Contours

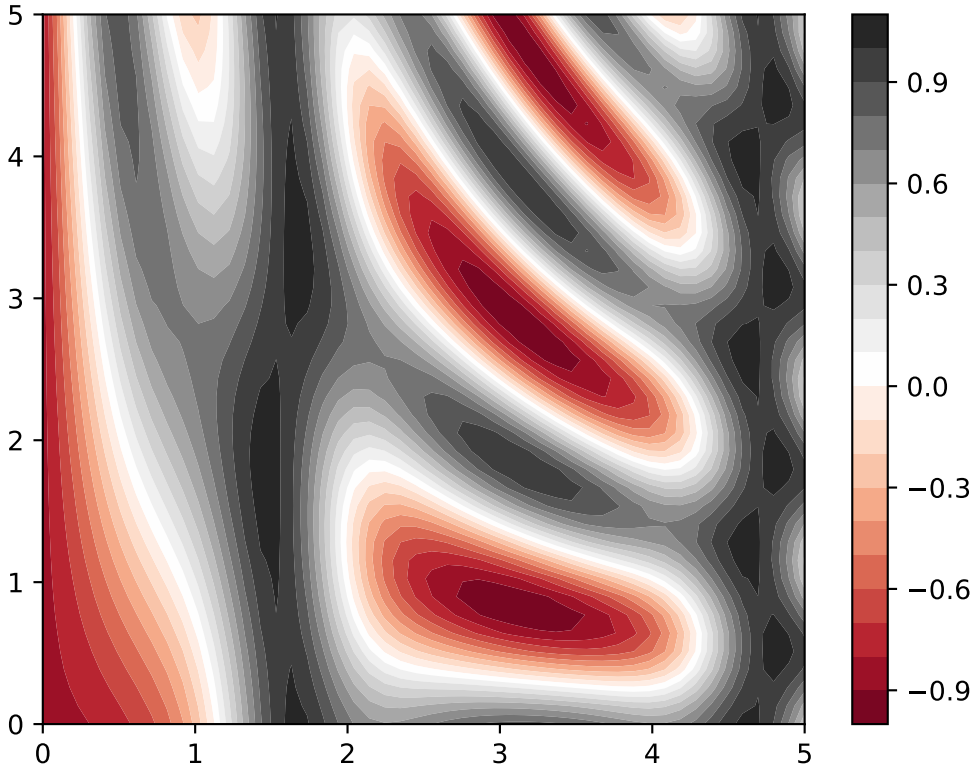
18.7.1 Open Contourplot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_ContourOpen.py, and the XML code in the file D01_ContourOpen.2D.xml.



18.7.2 Filled Contourplot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_ContourFilled.py, and the XML code in the file D02_ContourFilled.2D.xml.

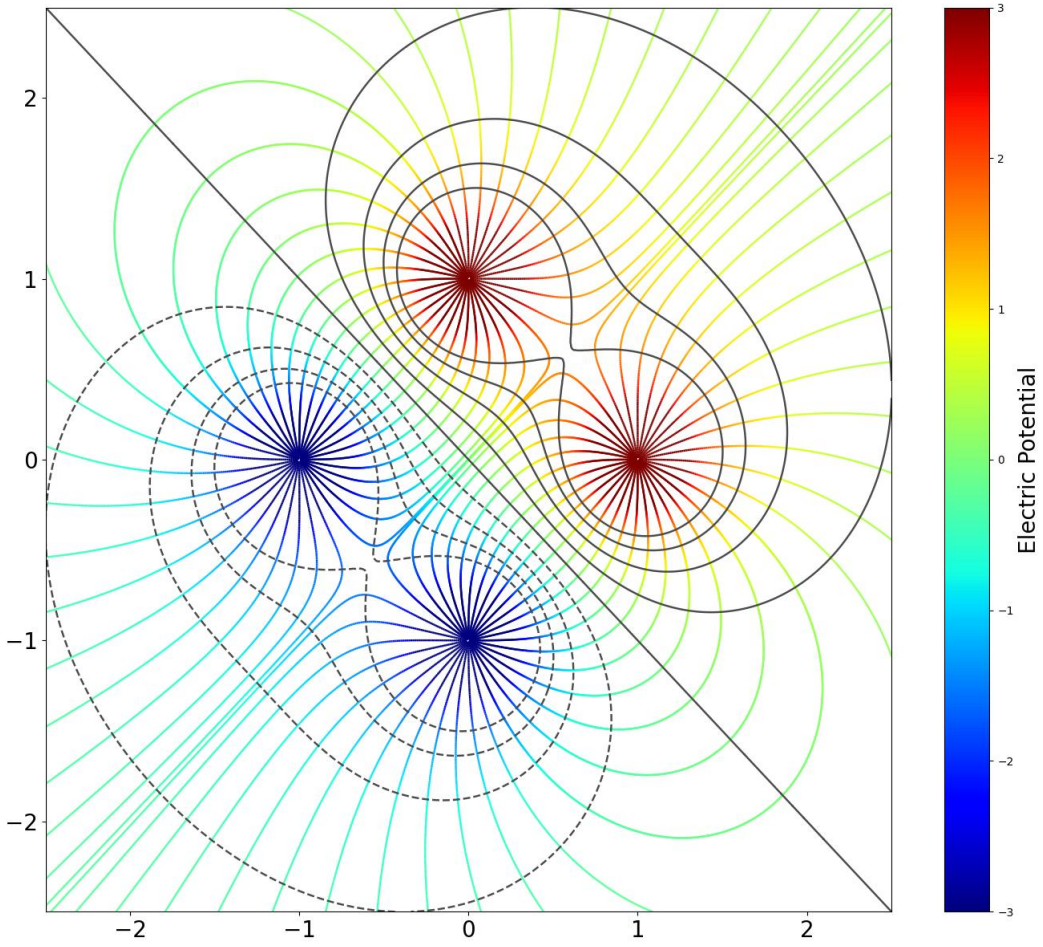


18.7.3 Electric dipole: field lines and open contours

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_08_FieldLinesContours.py, and the XML code in the file D03_08_FieldLinesContours.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ContourFilled_ = False
        Model_ = 1  # 1, 2, 3, 4
        FieldLinesContours(ContourFilled=ContourFilled_, Model=Model_)
```

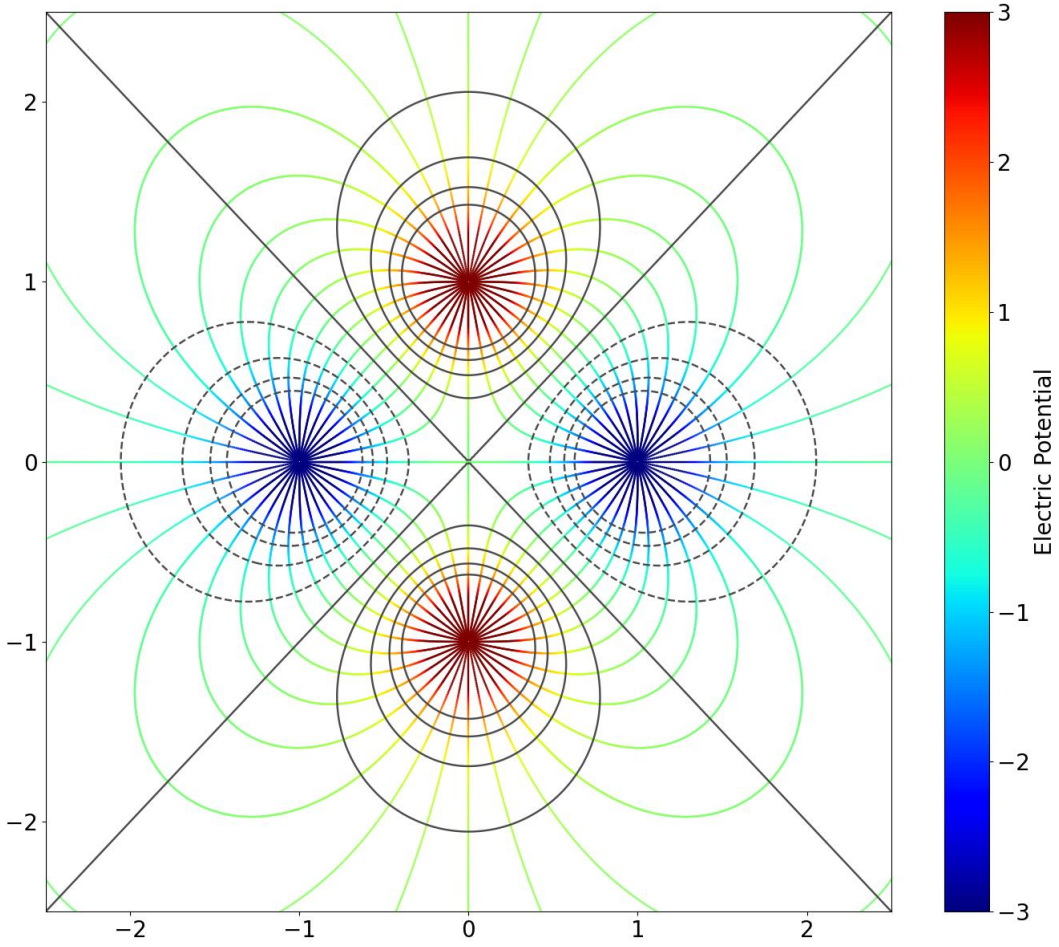


18.7.4 Electric quadrupole: field lines and open contours

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_08_FieldLinesContours.py, and the XML code in the file D03_08_FieldLinesContours.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ContourFilled_ = False
        Model_ = 2  # 1, 2, 3, 4
        FieldLinesContours(ContourFilled=ContourFilled_, Model=Model_)
```

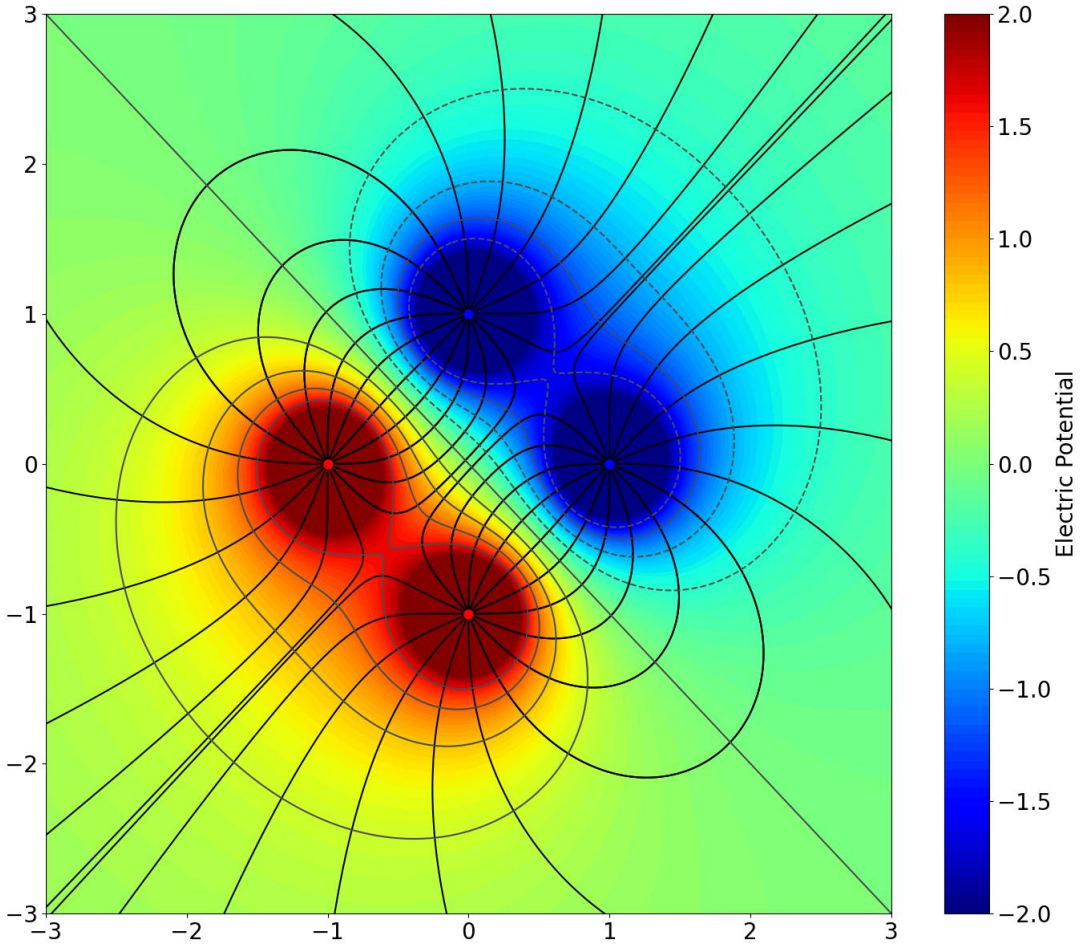


18.7.5 Electric dipoles: field lines and filled contours

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_08_FieldLinesContours.py, and the XML code in the file D03_08_FieldLinesContours.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ContourFilled_ = True
        Model_ = 1  # 1, 2, 3, 4
        FieldLinesContours(ContourFilled=ContourFilled_, Model=Model_)
```

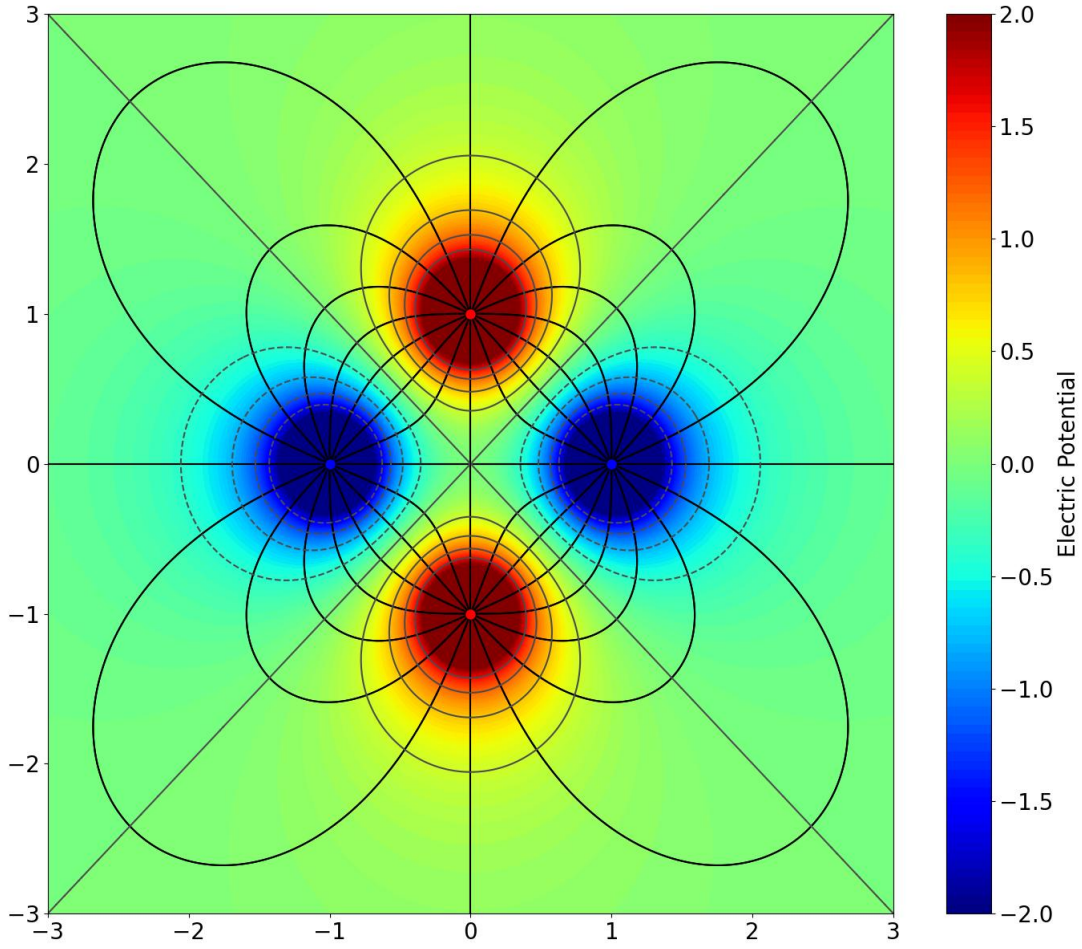


18.7.6 Electric quadrupole: field lines and filled contours

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_08_FieldLinesContours.py, and the XML code in the file D03_08_FieldLinesContours.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ContourFilled_ = True
        Model_ = 2  # 1, 2, 3, 4
        FieldLinesContours(ContourFilled=ContourFilled_, Model=Model_)
```

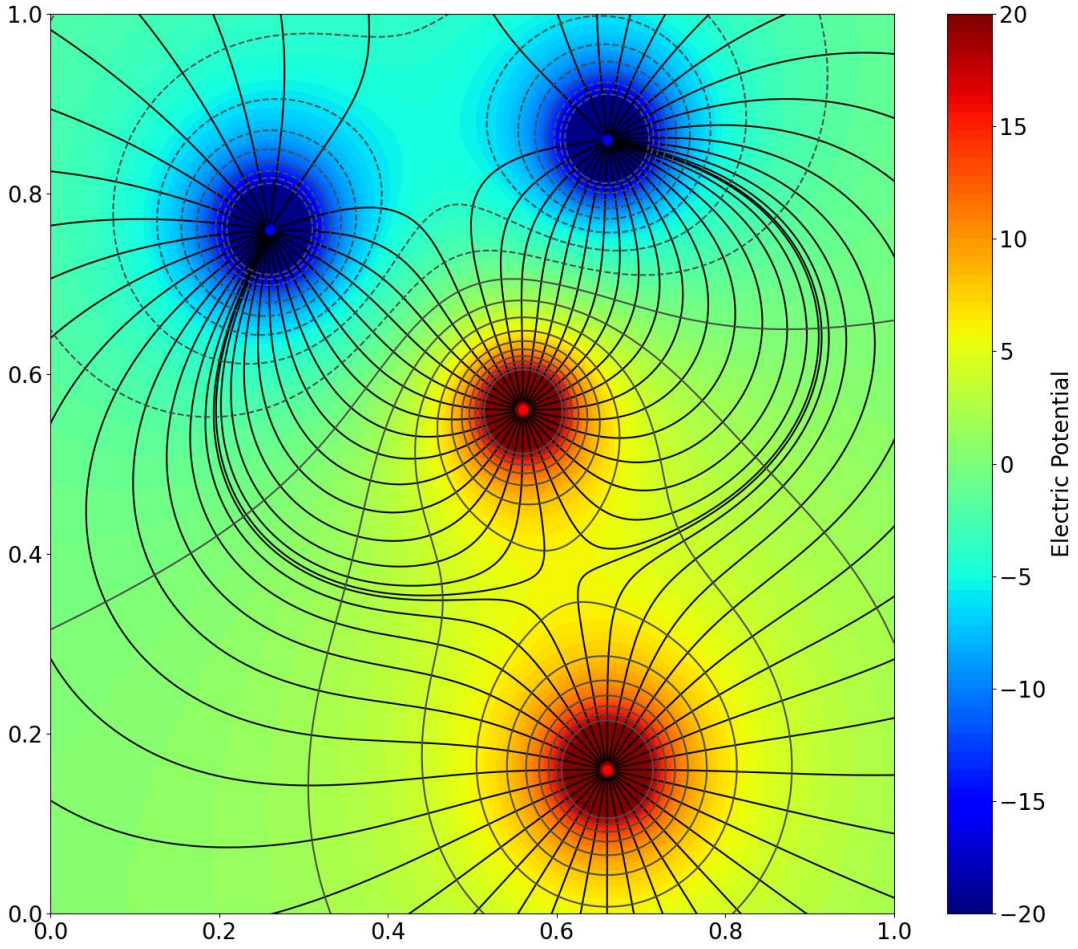


18.7.7 Orthogonal electric dipoles: field lines and filled contours

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_08_FieldLinesContours.py, and the XML code in the file D03_08_FieldLinesContours.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ContourFilled_ = True
        Model_ = 3  # 1, 2, 3, 4
        FieldLinesContours(ContourFilled=ContourFilled_, Model=Model_)
```

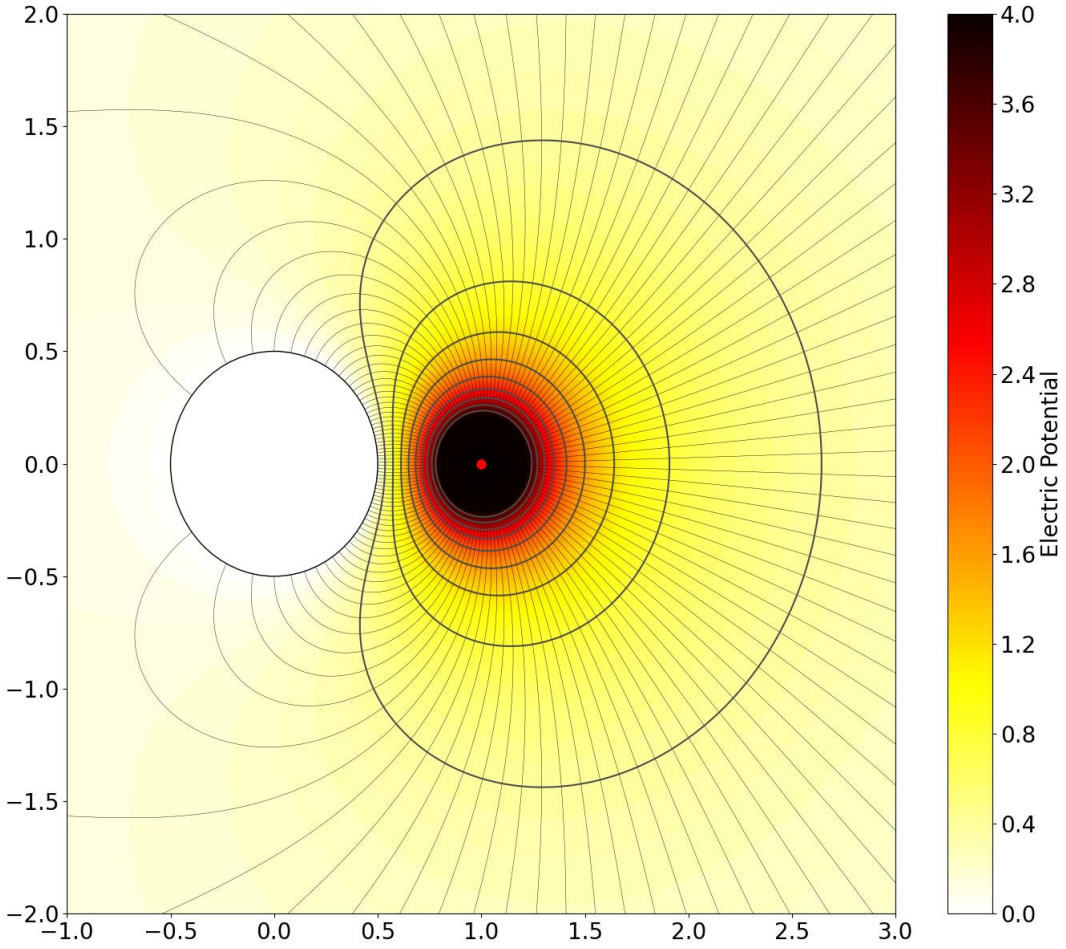


18.7.8 Electric field lines around a point charge with grounded sphere

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_08_FieldLinesContours.py, and the XML code in the file D03_08_FieldLinesContours.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ContourFilled_ = True
        Model_ = 4  # 1, 2, 3, 4
        FieldLinesContours(ContourFilled=ContourFilled_, Model=Model_)
```



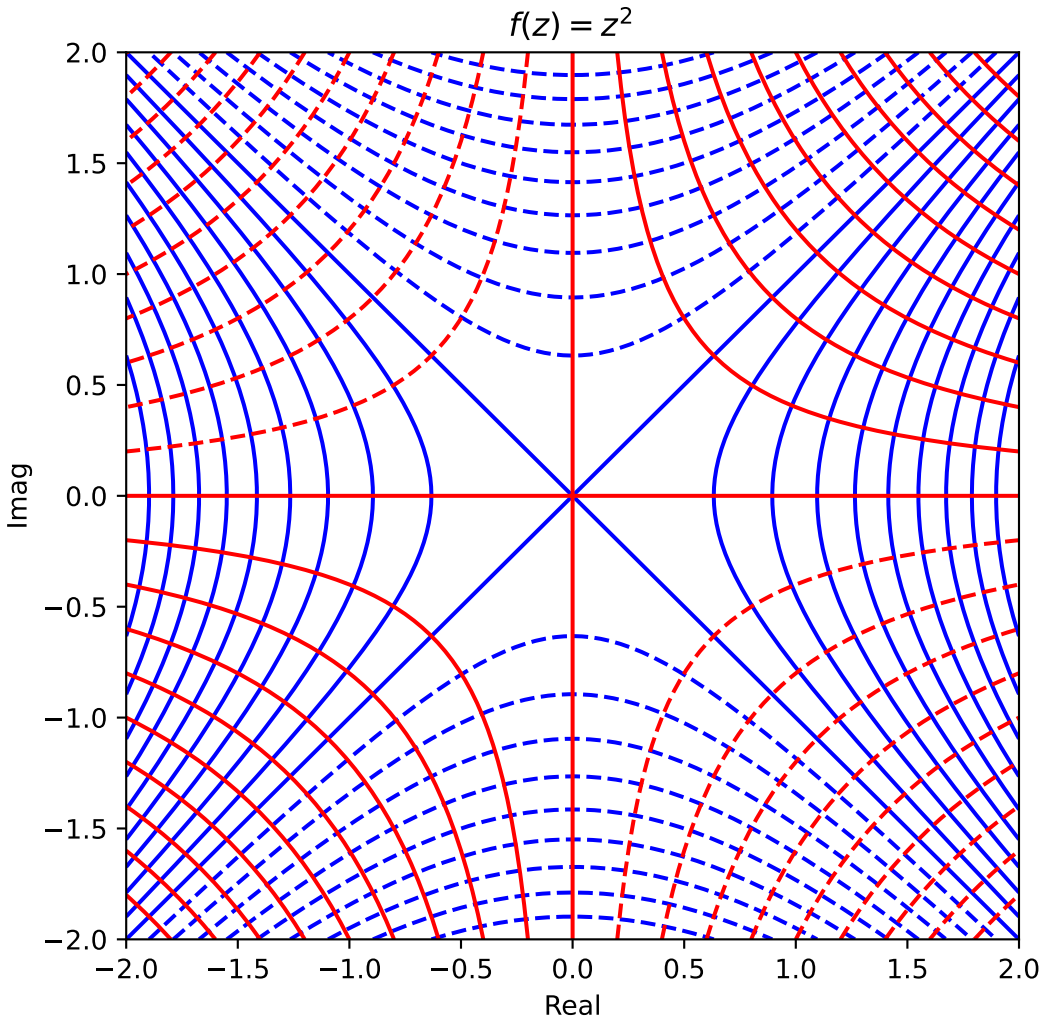
18.8 Complex functions rendered as contours

18.8.1 Complex square function

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_8_ContourComplex.py, and the XML code in the file D01_8_ContourComplex.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        Contour2dComplex(func=lambda z:z*z, wclip=20.0,
            showlevels=True, Title=r'$f(z)=z^2$')
```

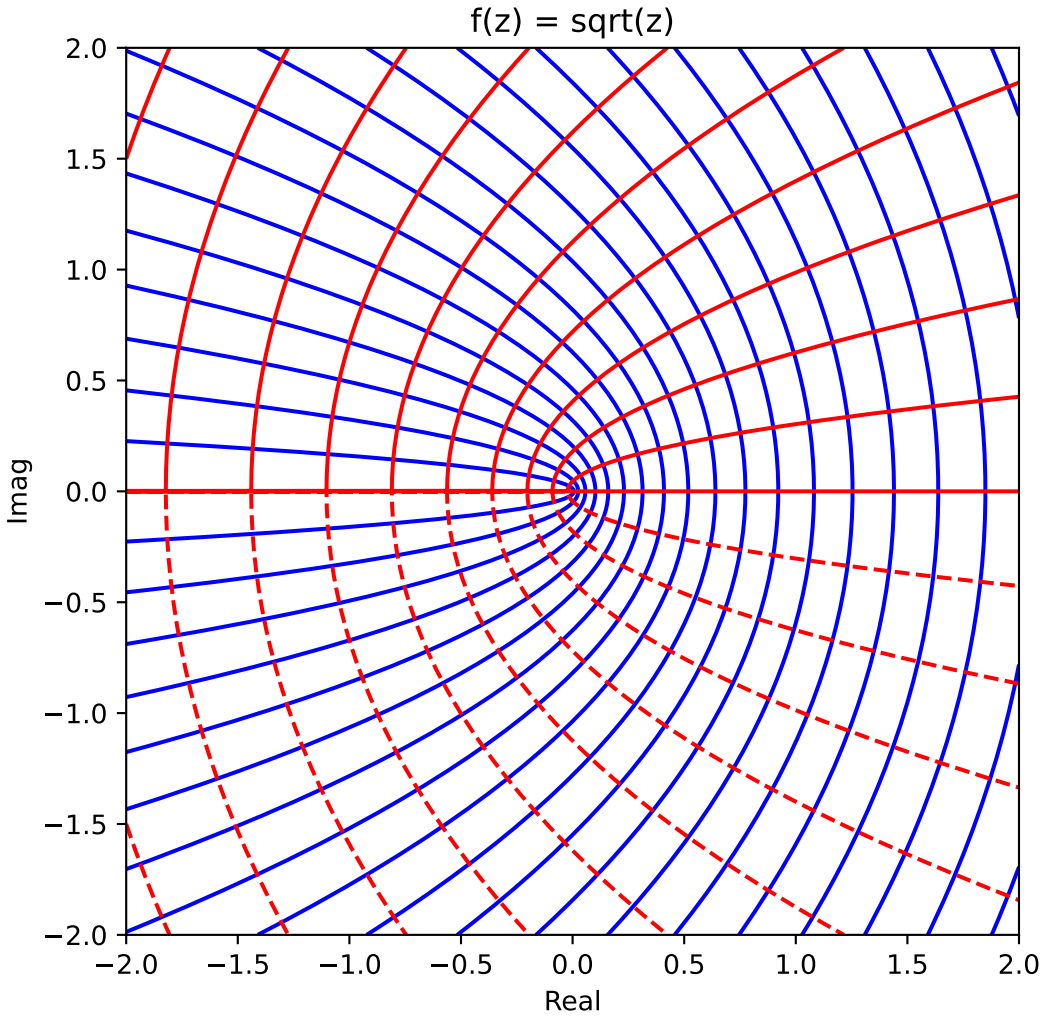


18.8.2 Complex sqrt function

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_8_ContourComplex.py, and the XML code in the file D01_8_ContourComplex.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        Contour2dComplex(func=np.sqrt, wclip=20.0, Title='f(z)=sqrt(z)')
```

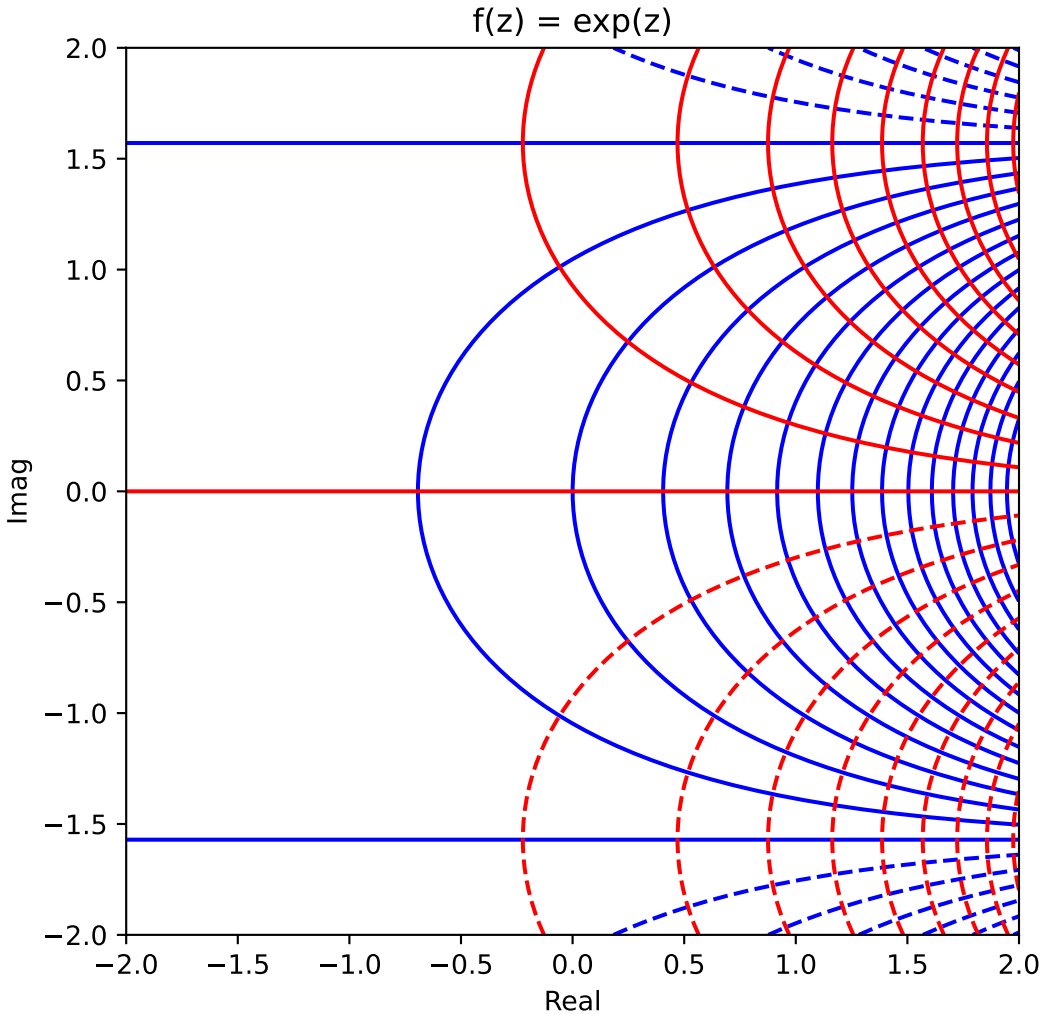


18.8.3 Complex exp function

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_8_ContourComplex.py, and the XML code in the file D01_8_ContourComplex.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        Contour2dComplex(func=np.exp, wclip=20.0, Title='f(z)=exp(z)')
```

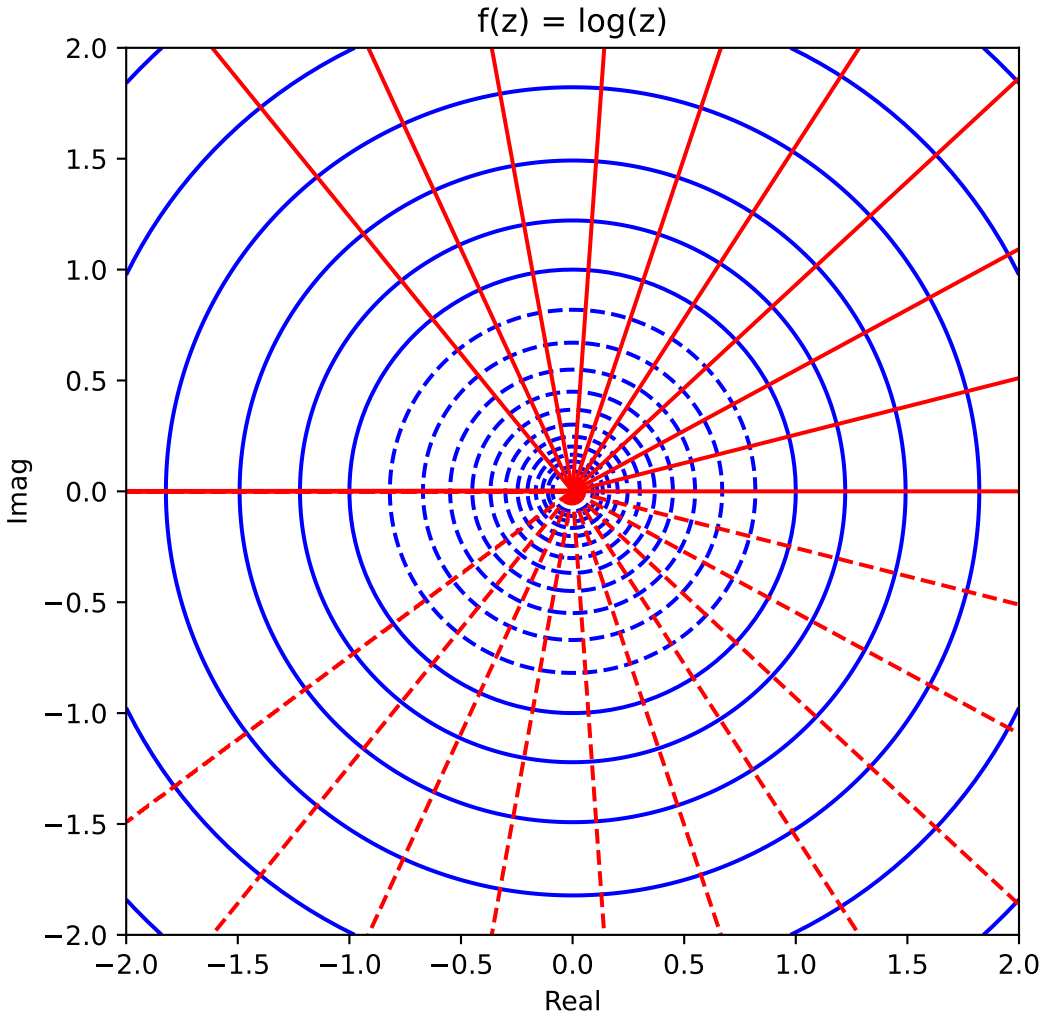


18.8.4 Complex log function

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_8_ContourComplex.py, and the XML code in the file D01_8_ContourComplex.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        Contour2dComplex(func=np.log, wclip=2.5, Title='f(z)=log(z)')
```

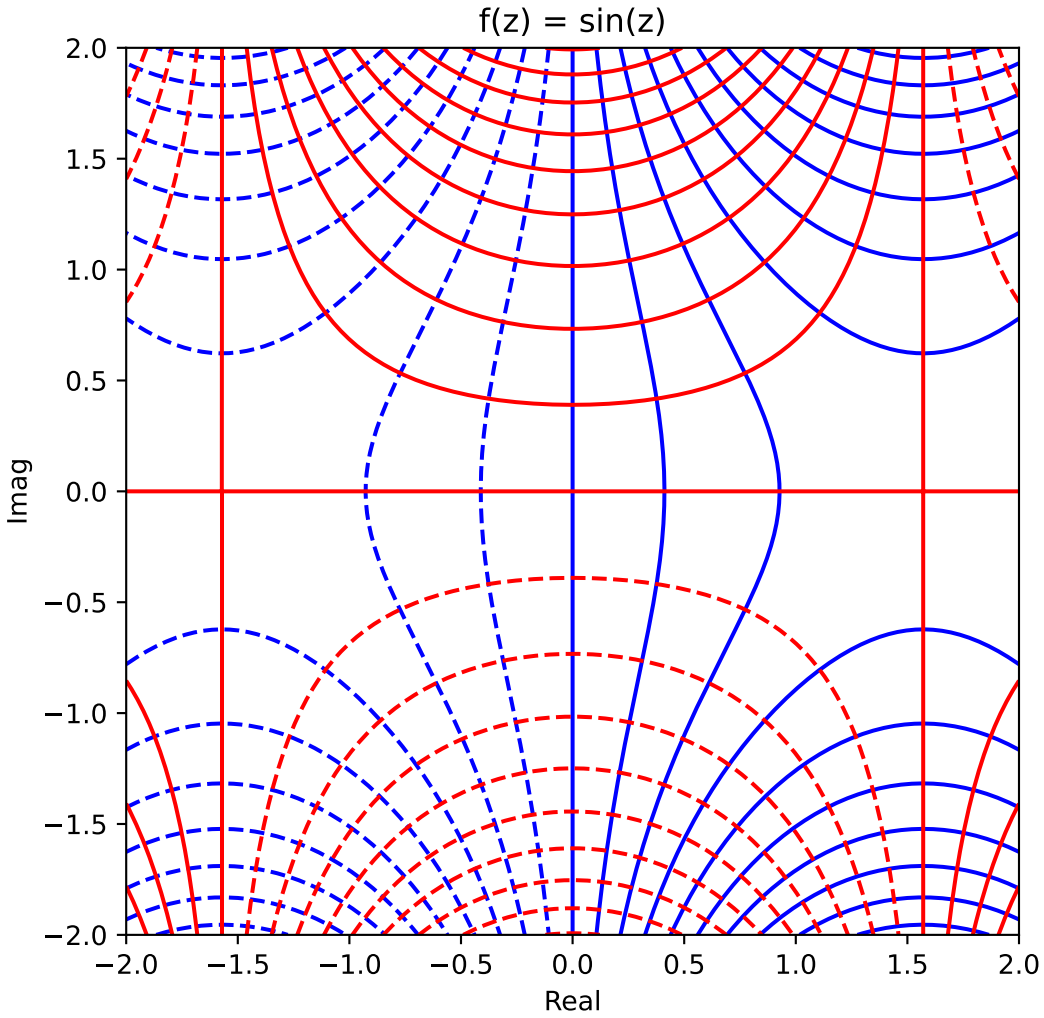


18.8.5 Complex sine function

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_8_ContourComplex.py, and the XML code in the file D01_8_ContourComplex.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        Contour2dComplex(func=np.sin, wclip=20.0, Title='f(z)=sin(z)')
```

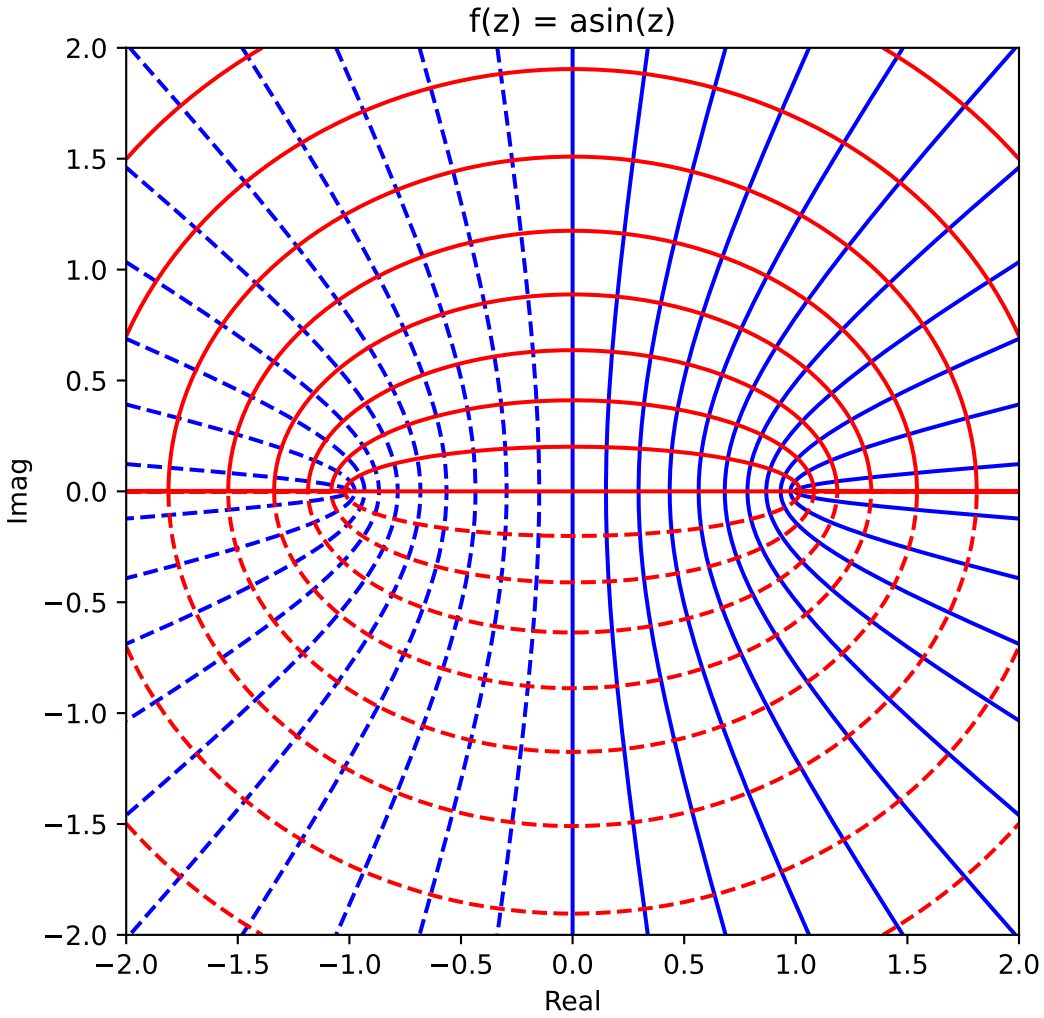


18.8.6 Complex asin function

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_8_ContourComplex.py, and the XML code in the file D01_8_ContourComplex.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        Contour2dComplex(func=np.asin, wclip=20.0, Title='f(z)=asin(z)')
```

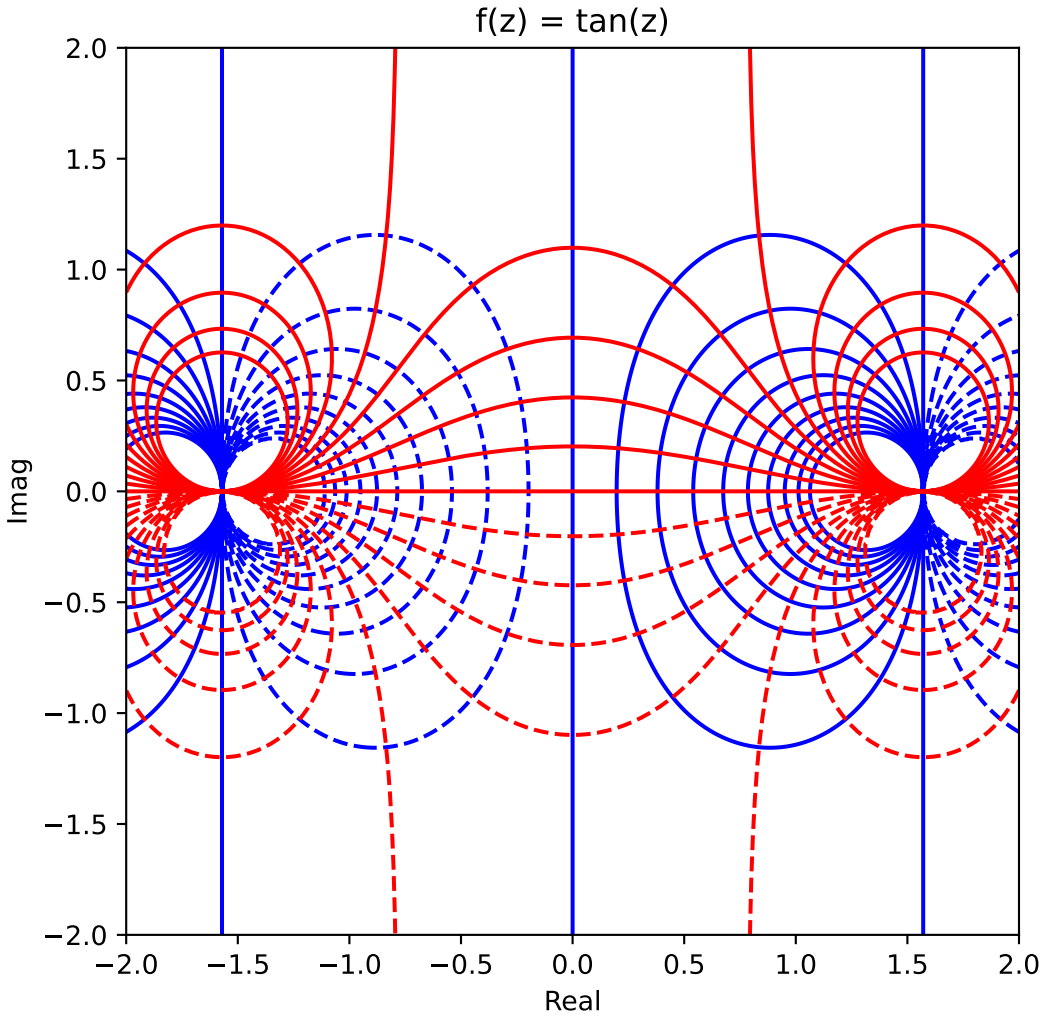


18.8.7 Complex tan function

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_8_ContourComplex.py, and the XML code in the file D01_8_ContourComplex.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        Contour2dComplex(func=np.tan, wclip=2.0, Title='f(z)=tan(z)')
```

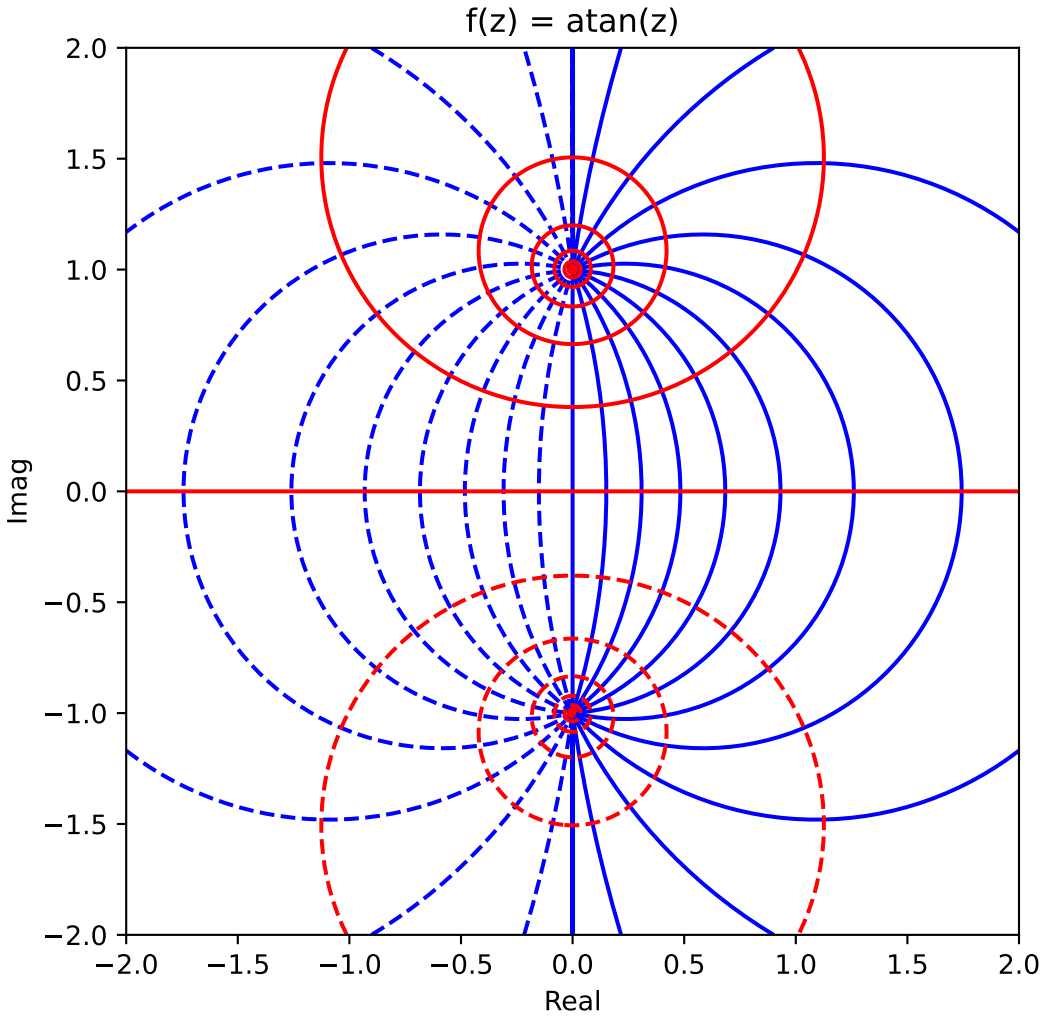


18.8.8 Complex atan function

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_8_ContourComplex.py, and the XML code in the file D01_8_ContourComplex.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        Contour2dComplex(func=np.atan, wclip=20.0, Title='f(z)=atan(z)')
```



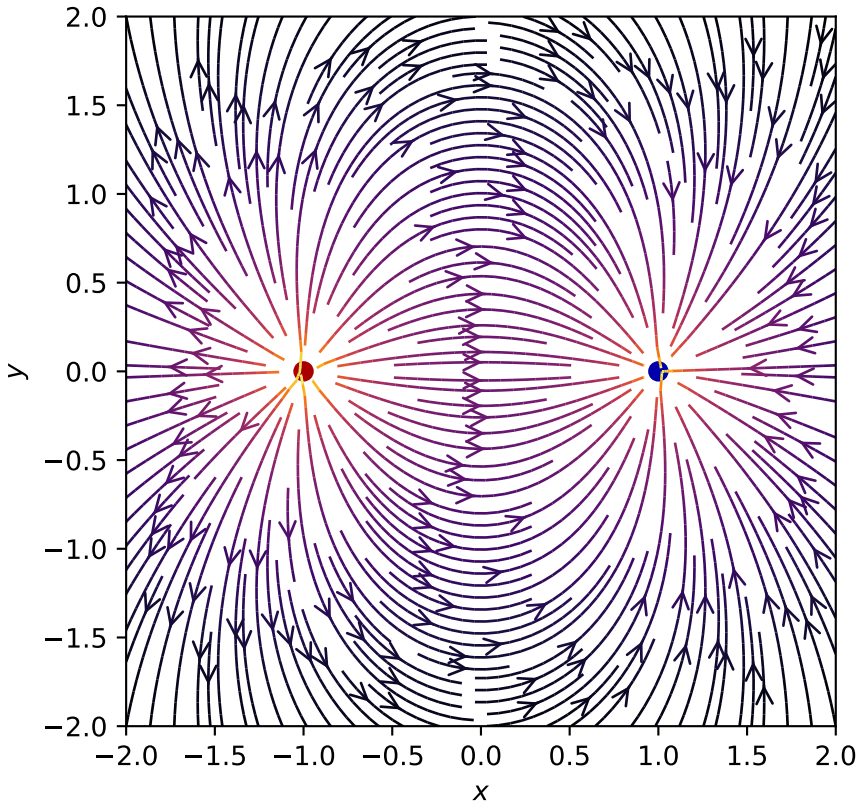
18.9 Streams, barbs and quivers (some require cartopy)

18.9.1 Streams, Electric Dipole: vector field

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_04_VectorFieldMultipole.py, and the XML code in the file D01_04_VectorFieldMultipole.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        nq_ = 2**1
        VectorFieldMultipole(nq=nq_)
```

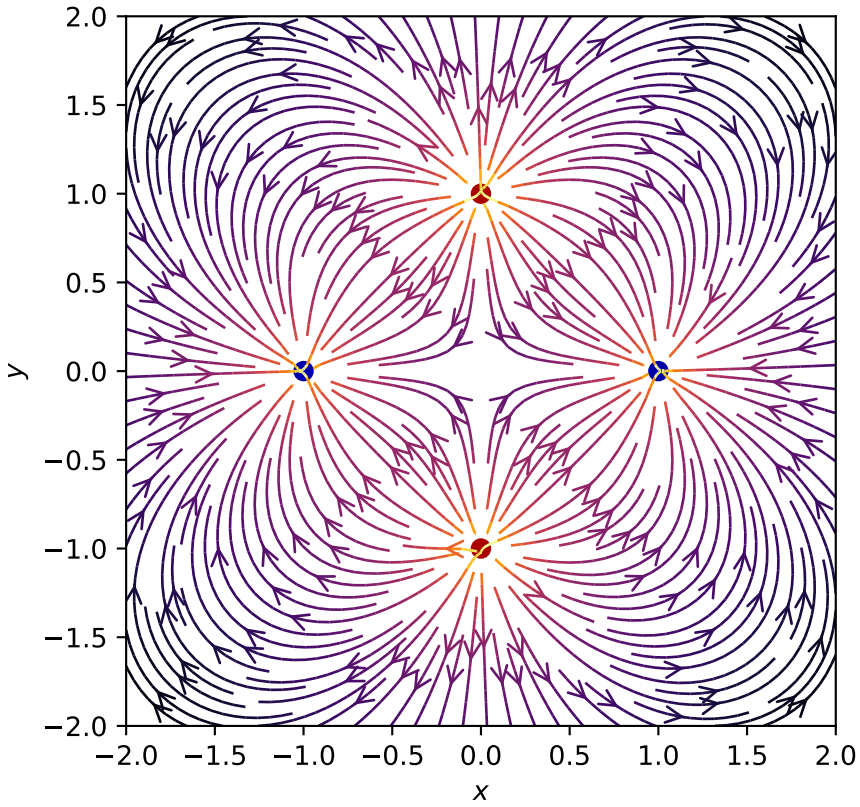


18.9.2 Streams, Electric Quadrupole: vector field

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_04_VectorFieldMultipole.py, and the XML code in the file D01_04_VectorFieldMultipole.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        nq_ = 2**2
        VectorFieldMultipole(nq=nq_)
```

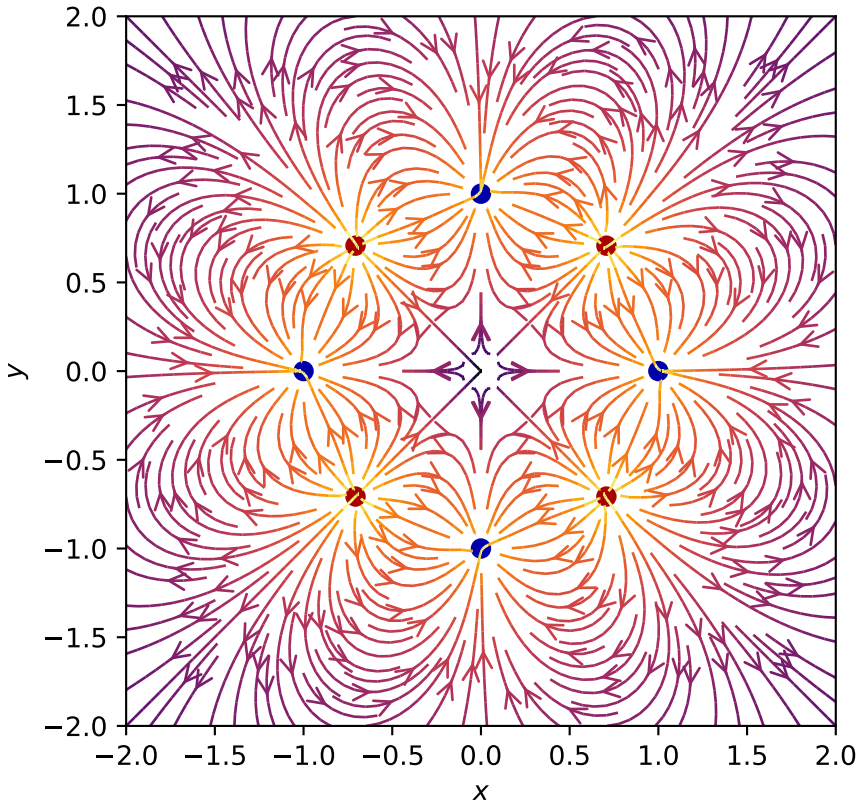


18.9.3 Streams, Electric Octopole: vector field

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_04_VectorFieldMultipole.py, and the XML code in the file D01_04_VectorFieldMultipole.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        nq_ = 2**3
        VectorFieldMultipole(nq=nq_)
```

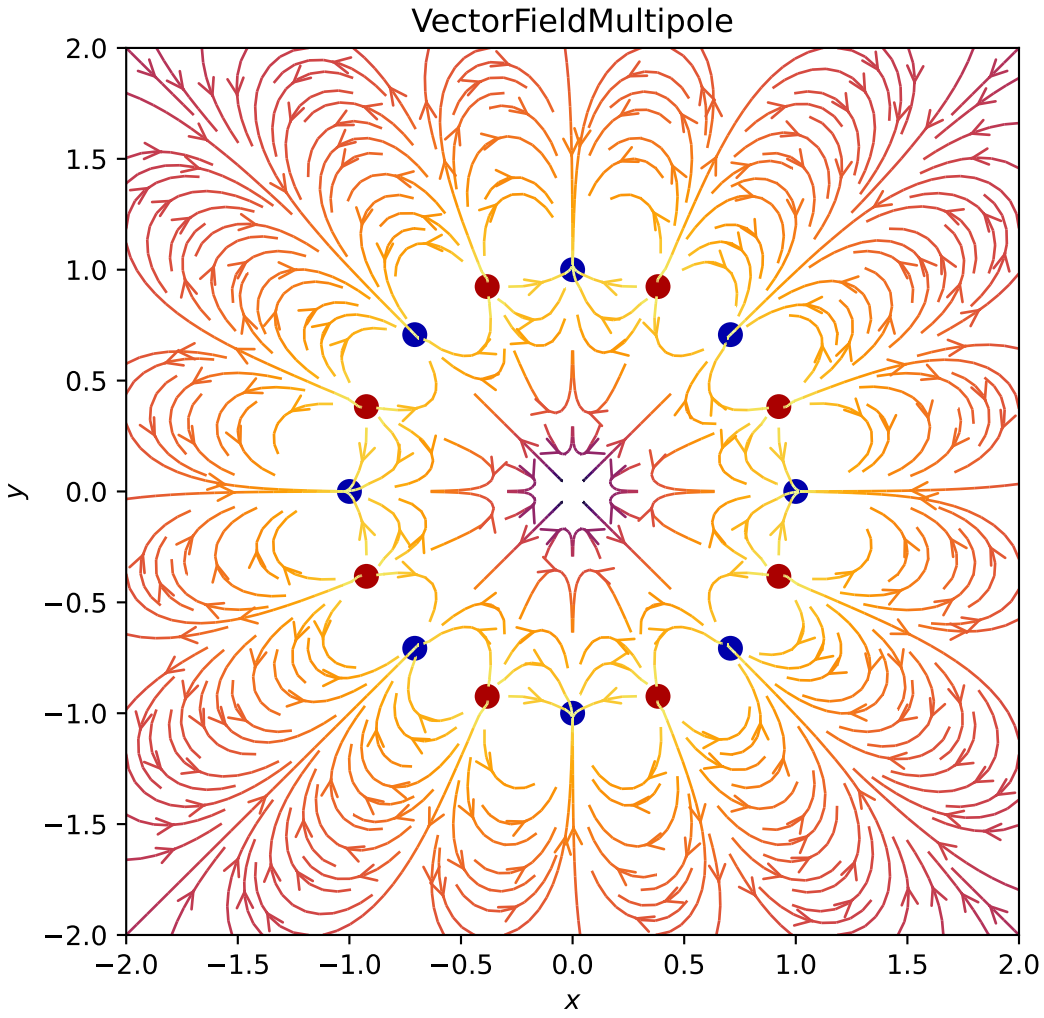


18.9.4 Streams, Electric Hexapole: vector field

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_04_VectorFieldMultipole.py, and the XML code in the file D01_04_VectorFieldMultipole.2D.xml.

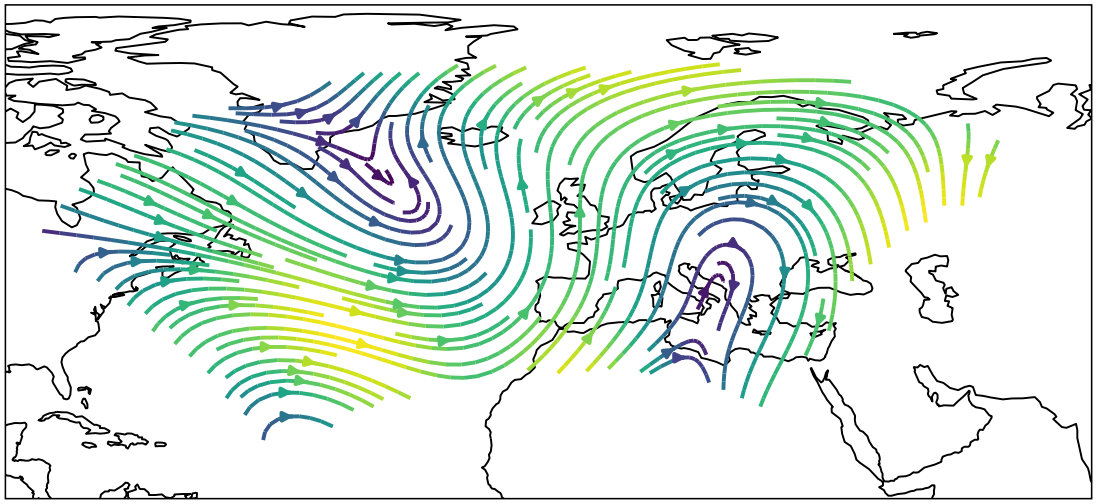
To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        nq_ = 2**4
        VectorFieldMultipole(nq=nq_)
```



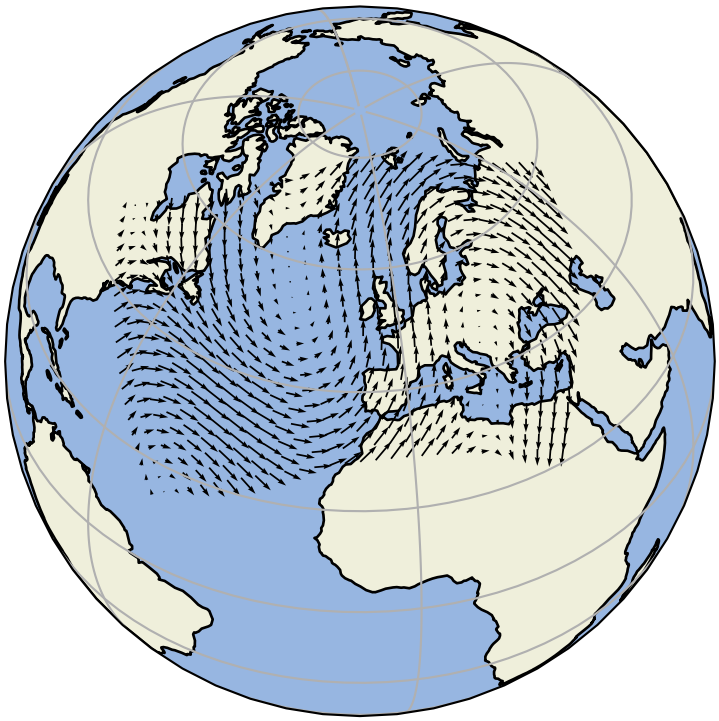
18.9.5 Streams, North Atlantic Ocean and Europe (requires cartopy)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_StreamsNorthAtlantic.py, and the XML code in the file D05_StreamsNorthAtlantic.2D.xml.



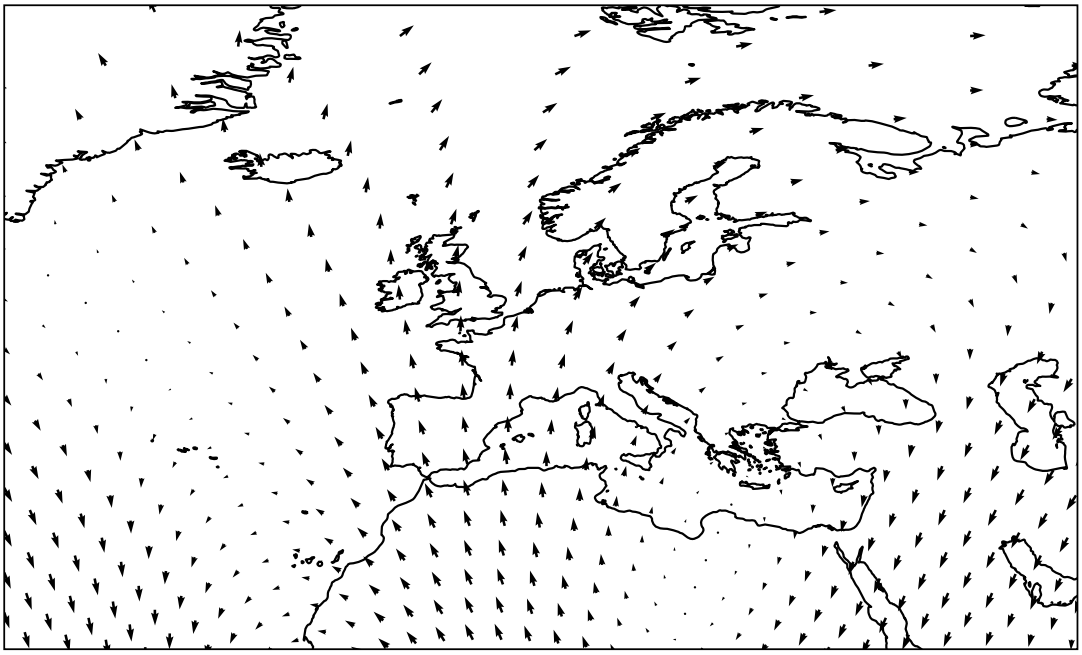
18.9.6 Quivers, North Atlantic Ocean and Europe (requires cartopy)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_QuiversNorthAtlantic.py, and the XML code in the file D06_QuiversNorthAtlantic.2D.xml.

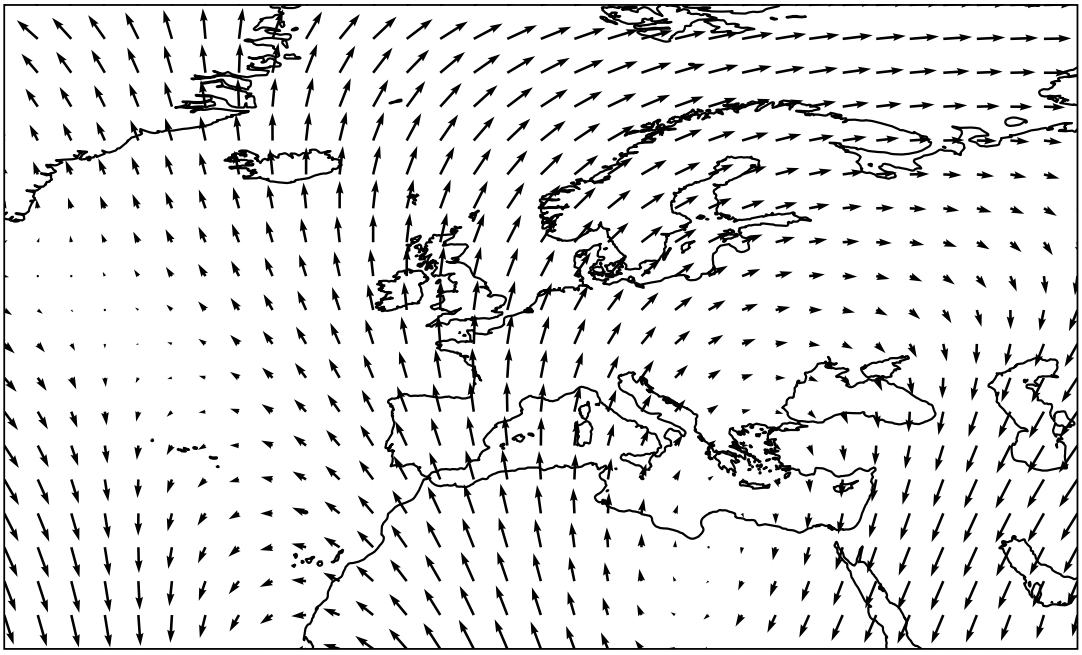


18.9.7 Quivers, plain and regridded (requires cartopy)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_QuiversPlainAndRegridded.py, and the XML code in the file D07_QuiversPlainAndRegridded.2D.xml.

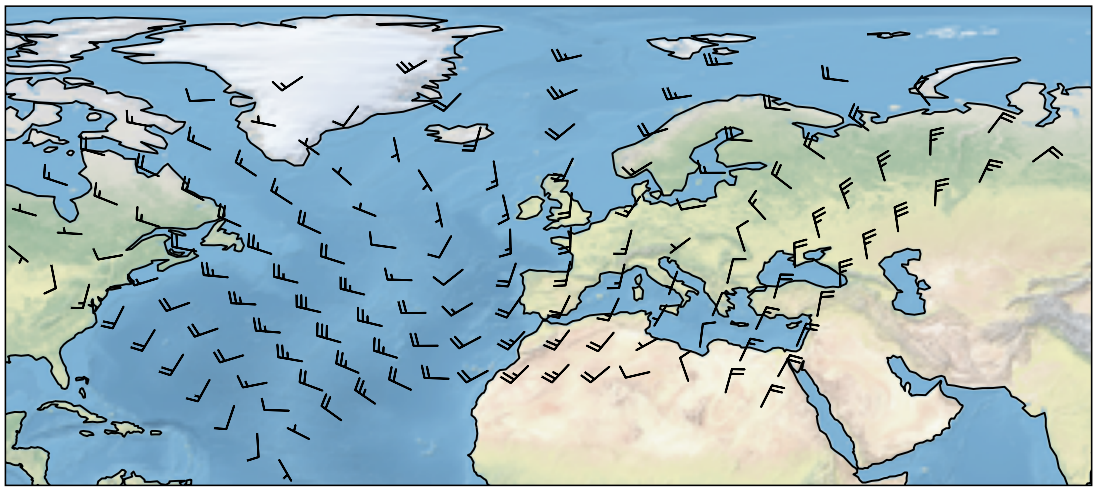


The same vector field regridded



18.9.8 Barbs, North Atlantic Ocean and Europe (requires cartopy)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_BarbsNorthAtlantic.py, and the XML code in the file D08_BarbsNorthAtlantic.2D.xml.



CHAPTER

NINETEEN

BITMAPS

19.1 Fractals: introduction

See also: http://www.juliasets.dk/Pictures_of_Julia_and_Mandelbrot_sets.htm

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen

See also: http://www.dhushara.com/DarkHeart/quasif/quasi.htm (includes Kleinian fractals)

Difference Method: https://en.wikibooks.org/wiki/Fractals/Iterations_in_the_complex_plane/demm

MandelbrotSetExterior: https://en.wikibooks.org/wiki/Fractals/Iterations_in_the_complex_plane/MandelbrotSetExterior#Real_Escape_Time

Fractals/fractalzoomer: https://en.wikibooks.org/wiki/Fractals/fractalzoomer

https://en.wikipedia.org/wiki/Fractal

19.1.1 Barnsley's Fern

https://mathworld.wolfram.com/BarnsleysFern.html

https://en.wikipedia.org/wiki/Barnsley_fern

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_BarnsleyFern.py, and the XML code in the file D01_BarnsleyFern.2D.xml.

Left figure: Barnsley's Fern, 20000 points

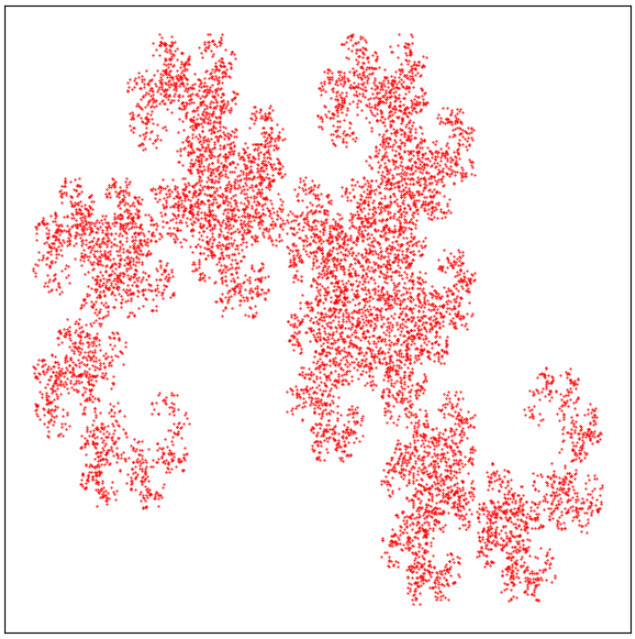
Right figure: Barnsley's Fern, 100000 points

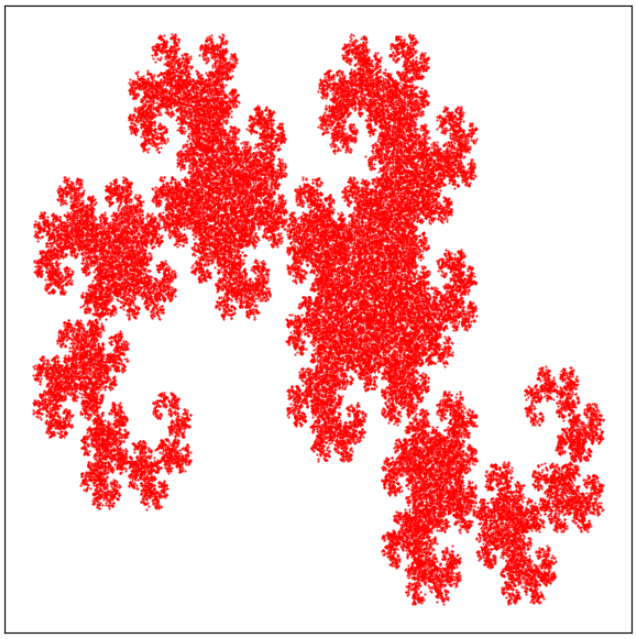
19.1.2 Dragon curve

https://en.wikipedia.org/wiki/Dragon_curve

https://mathworld.wolfram.com/DragonCurve.html

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_DragonCurve.py, and the XML code in the file D02_DragonCurve.2D.xml.





Left figure: Dragon Curve, 10000 points

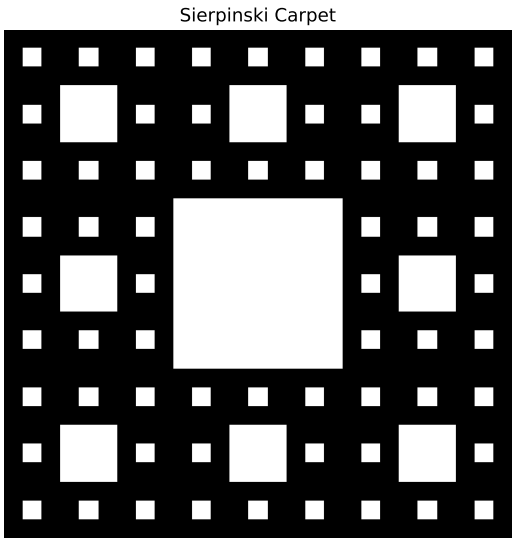
Right figure: Dragon Curve, 50000 points

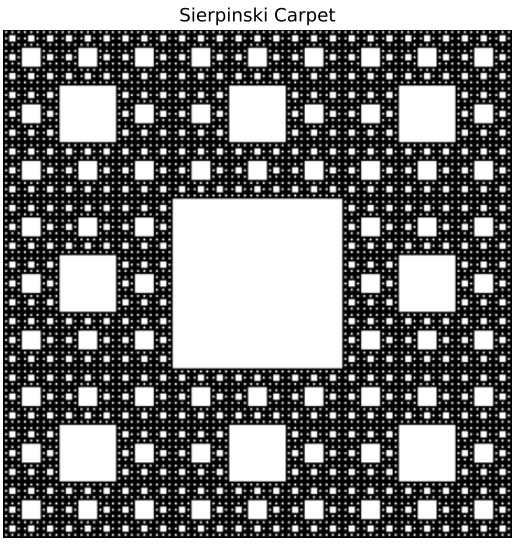
19.1.3 Sierpiński carpet

https://en.wikipedia.org/wiki/Sierpi%C5%84ski_carpet

https://mathworld.wolfram.com/SierpinskiCarpet.html

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_SierpinskiCarpet.py, and the XML code in the file D03_SierpinskiCarpet.2D.xml.





Left figure: Sierpiński carpet, $n = 3$.

Right figure: Sierpiński carpet, $n = 5$.

19.2 Fractals related to the Mandelbrot set

See also: http://www.juliasets.dk/Pictures_of_Julia_and_Mandelbrot_sets.htm

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen

See also: http://www.dhushara.com/DarkHeart/quasif/quasi.htm (includes Kleinian fractals)

See also: https://en.wikipedia.org/wiki/Mandelbrot_set

Difference Method: https://en.wikibooks.org/wiki/Fractals/Iterations_in_the_complex_plane/demm

MandelbrotSetExterior: https://en.wikibooks.org/wiki/Fractals/Iterations_in_the_complex_plane/MandelbrotSetExterior#Real_Escape_Time

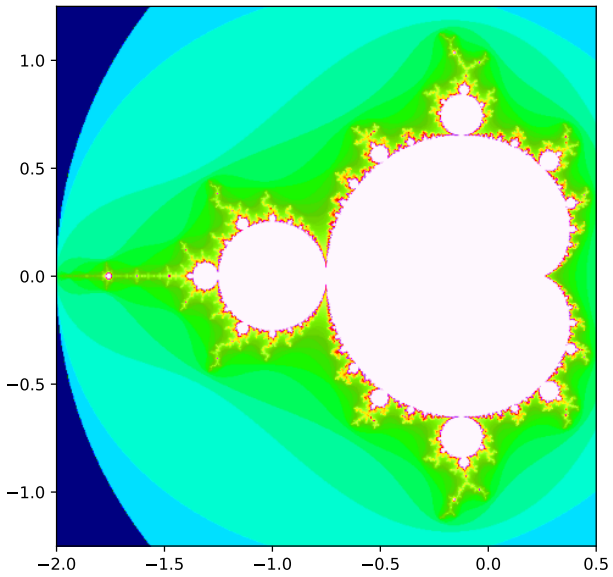
Fractals/fractalzoomer: https://en.wikibooks.org/wiki/Fractals/fractalzoomer

19.2.1 Mandelbrot set, example 1, classical set: $z_{n+1} = z_n^2 + c$

Classical formula: $z_{n+1} = z_n^2 + c$.

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01a_Mandelbrot1.py, and the XML code in the file D01a_Mandelbrot1.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01b_Mandelbrot1.py, and the XML code in the file D01b_Mandelbrot1.2D.xml.



19.2.2 Mandelbrot set, example 2: higher powers

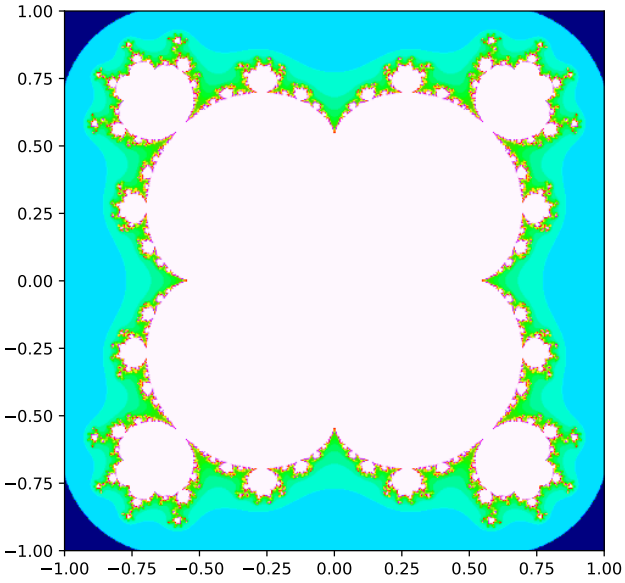
Recursion formula: $z_{n+1} = z_n^5 + c$.

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02a_Mandelbrot2.py, and the XML code in the file D02a_Mandelbrot2.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02b_Mandelbrot2.py, and the XML code in the file D02b_Mandelbrot2.2D.xml.

See also: https://paulbourke.net/fractals/mandelpower/

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen



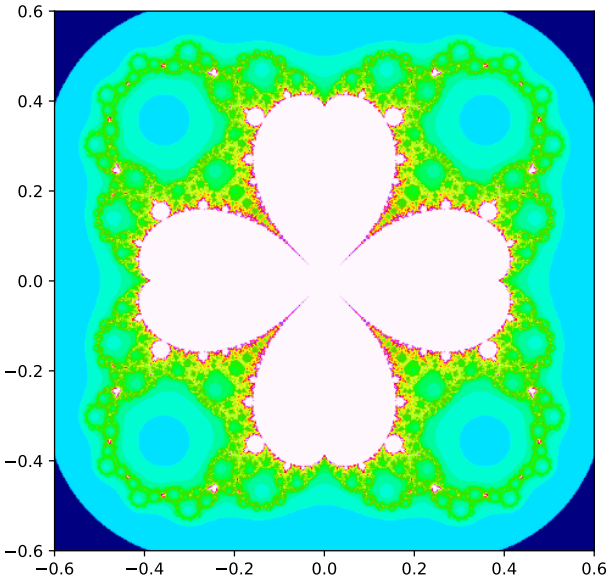
19.2.3 Mandelbrot set, example 3: $z_{n+1} = z_n^5 + \frac{c}{z_n}$

Recursion formula: $z_{n+1} = z_n^5 + \frac{c}{z_n}$.

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03a_Mandelbrot3.py, and the XML code in the file D03a_Mandelbrot3.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03b_Mandelbrot3.py, and the XML code in the file D03b_Mandelbrot3.2D.xml.



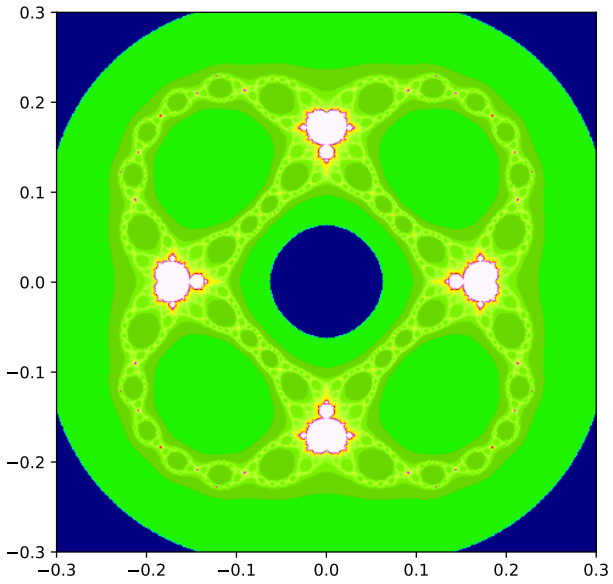
19.2.4 Mandelbrot set, example 4: $z_{n+1} = z_n^5 + \frac{c}{z_n^5}$

Recursion formula: $z_{n+1} = z_n^5 + \frac{c}{z_n^5}$.

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04a_Mandelbrot4.py, and the XML code in the file D04a_Mandelbrot4.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04b_Mandelbrot4.py, and the XML code in the file D04b_Mandelbrot4.2D.xml.



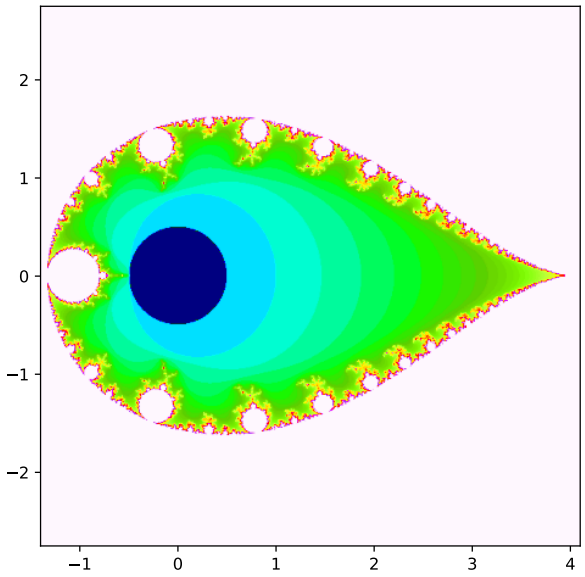
19.2.5 Mandelbrot set, example 5, "Inverse Mandelbrot"

Recursion formula: $z_{n+1} = z_n^2 + \frac{1}{c}$.

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05a_Mandelbrot5.py, and the XML code in the file D05a_Mandelbrot5.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05b_Mandelbrot5.py, and the XML code in the file D05b_Mandelbrot5.2D.xml.



19.2.6 Mandelbrot set, example 6, "Marek Dragon Fractal":

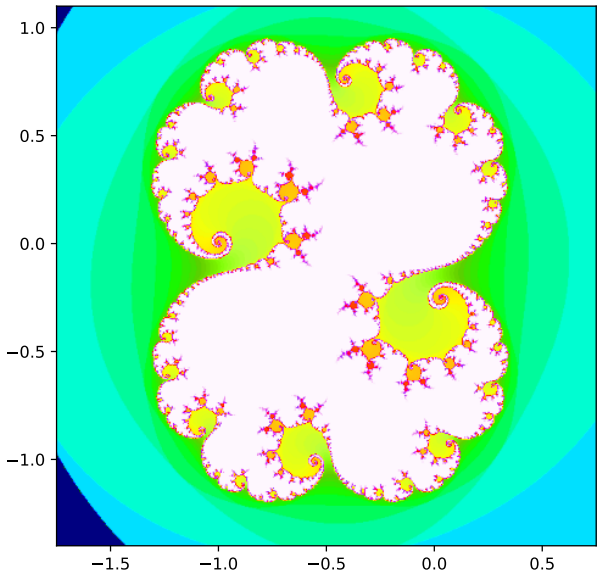
Compare this with "Phoenix Julia fractals"

Recursion formula: $z_{n+1} = \exp(2\pi ir)z_n + z_n^2$ where r is an irrational number, in this example it is $1 - (\sqrt{2} - \sqrt{3} + \sqrt{5})/2$.

See also: https://paulbourke.net/fractals/marek/

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06a_Mandelbrot6.py, and the XML code in the file D06a_Mandelbrot6.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06b_Mandelbrot6.py, and the XML code in the file D06b_Mandelbrot6.2D.xml.



19.2.7 Mandelbrot set, example 7, "Burning Ship Fractal"

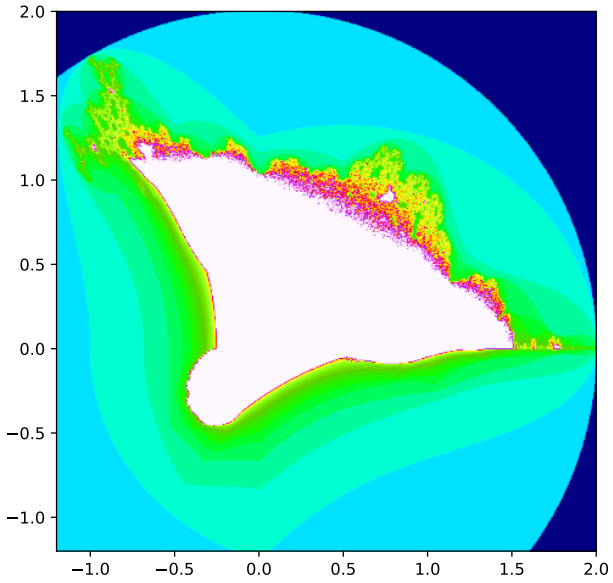
Recursion formula: $z_{n+1} = (|\Re(z_n)| + i|\Im(z_n)|)^2 + c$.

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen

See also: https://paulbourke.net/fractals/burnship/

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_Mandelbrot7.py, and the XML code in the file D07a_Mandelbrot7.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_Mandelbrot7.py, and the XML code in the file D07b_Mandelbrot7.2D.xml.



19.3 Fractals related to the Julia set

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen

See also: https://en.wikipedia.org/wiki/Julia_set

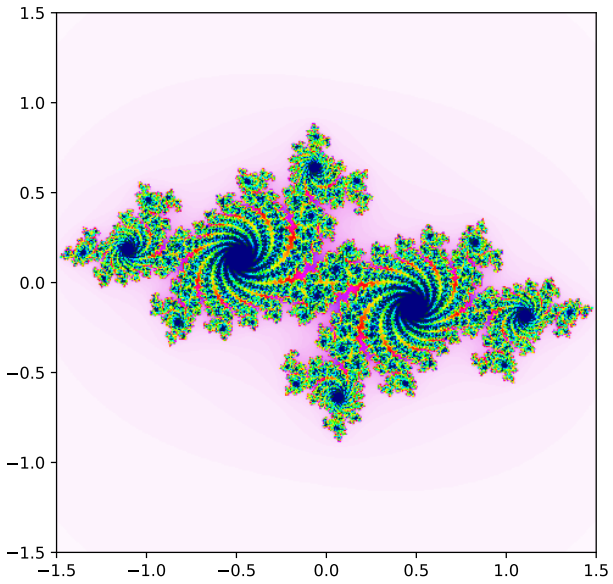
19.3.1 Julia sets, example 1: Classical set

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01a_Julia01.py, and the XML code in the file D01a_Julia01.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01b_Julia01.py, and the XML code in the file D01b_Julia01.2D.xml.

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen

See also: https://paulbourke.net/fractals/sinjulia/julia_set.py (contains suggestions for c.) in the section "QUADRATIC JULIA ITERATION AND BAILOUT".

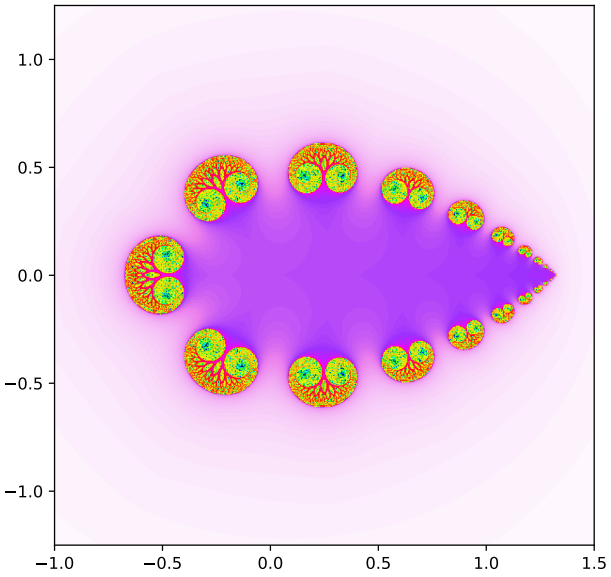


19.3.2 Julia sets, example 2: Glynn fractals

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02a_Julia02.py, and the XML code in the file D02a_Julia02.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02b_Julia02.py, and the XML code in the file D02b_Julia02.2D.xml.

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen

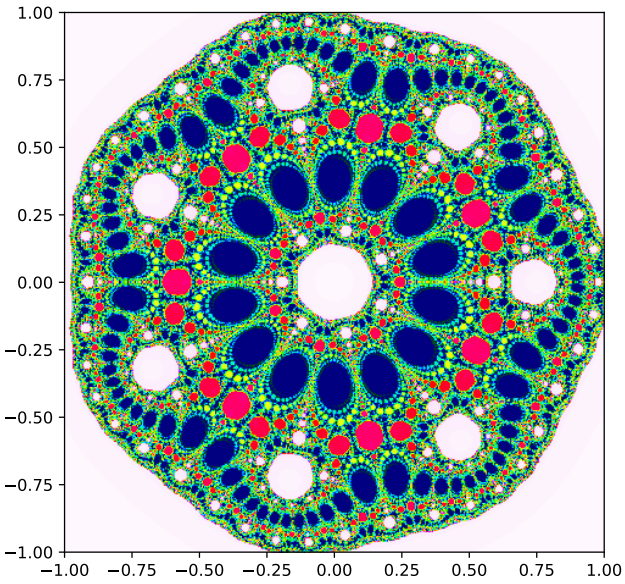


19.3.3 Julia sets, example 3: Spiral Septagon fractal

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03a_Julia03.py, and the XML code in the file D03a_Julia03.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03b_Julia03.py, and the XML code in the file D03b_Julia03.2D.xml.

See also: https://paulbourke.net/fractals/septagon/

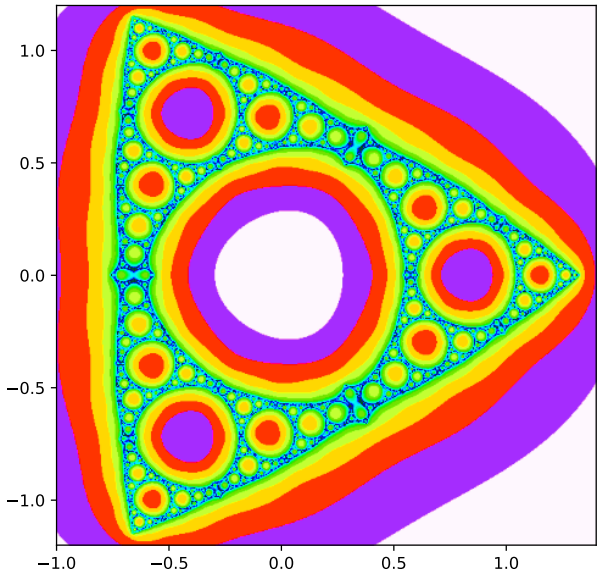


19.3.4 Julia sets, example 4: Reuleaux Triangle fractal

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04a_Julia04.py, and the XML code in the file D04a_Julia04.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04b_Julia04.py, and the XML code in the file D04b_Julia04.2D.xml.

See also: https://paulbourke.net/fractals/triangle/



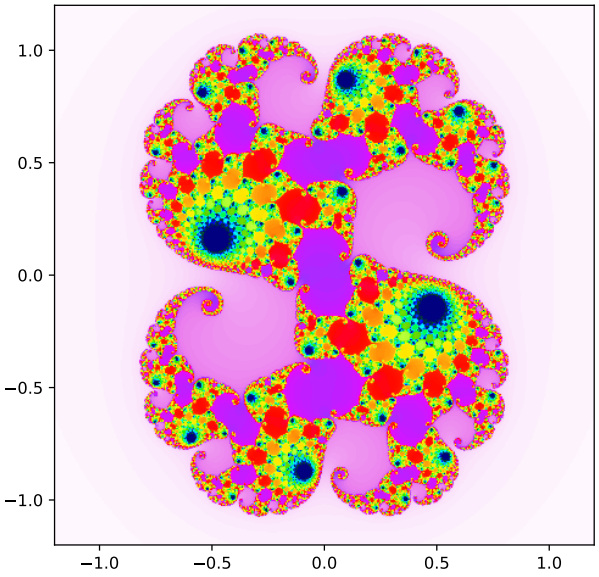
19.3.5 Julia sets, example 5: Phoenix Julia fractals

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05a_Julia05.py, and the XML code in the file D05a_Julia05.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05b_Julia05.py, and the XML code in the file D05b_Julia05.2D.xml.

Compare this with "Marek Dragon"

See also: https://paulbourke.net/fractals/phoenix_julia/

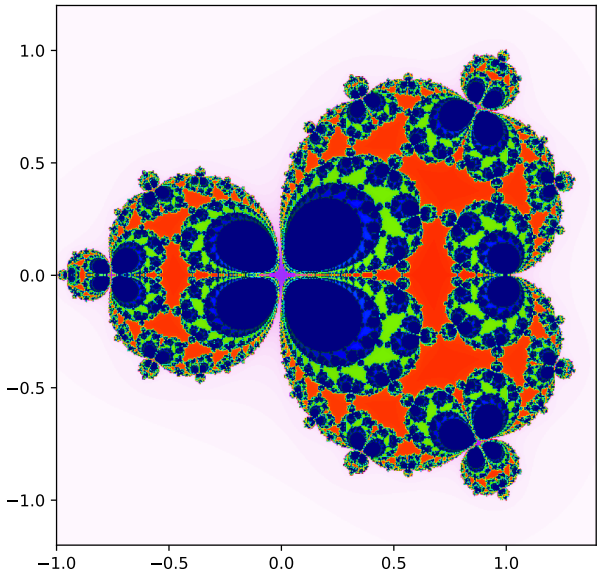


19.3.6 Julia sets, example 6: Cubic Julia fractals

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06a_Julia06.py, and the XML code in the file D06a_Julia06.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06b_Julia06.py, and the XML code in the file D06b_Julia06.2D.xml.

See also: https://paulbourke.net/fractals/cubicjulia/

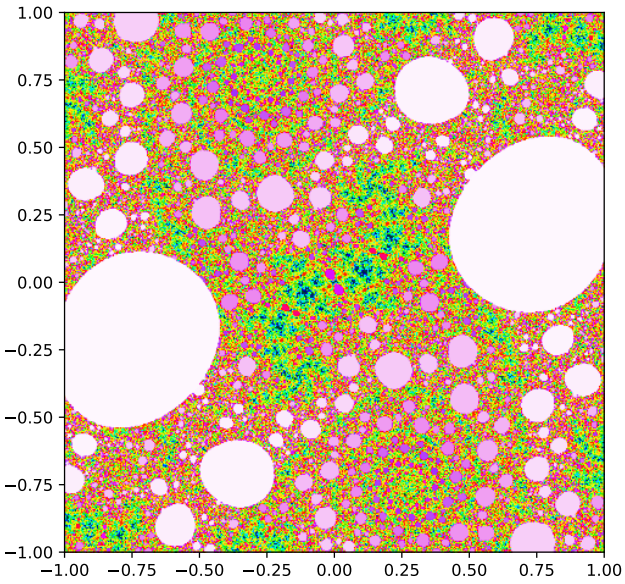


19.3.7 Julia sets, example 7: Spiral Julia fractal (uses $z_{n+1} = \tan(z_n^2 + c)$)

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_Julia07.py, and the XML code in the file D07a_Julia07.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_Julia07.py, and the XML code in the file D07b_Julia07.2D.xml.

See also: https://paulbourke.net/fractals/spiraljulia/



19.3.8 Julia sets, example 8: Julia set of $\sin(z)$

To do this we form the series $zk + 1 = csin(zk)$, starting with some initial $z0$.

See also: https://paulbourke.net/fractals/sinjulia/julia_set.py

See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen, 4.7. zn+1 = c sin(zn)

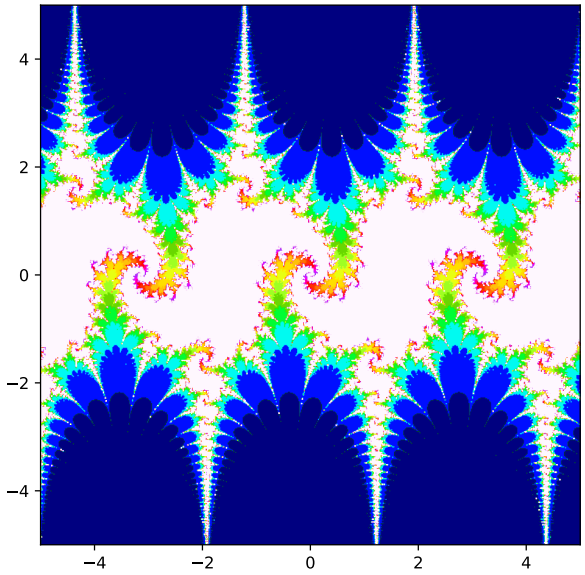
The behaviour of this series determines whether or not the initial z0 is part of the julia set or not. More precisely, if the series tends to infinity then z0 is part of the Julia set, otherwise it isn't. In the examples shown on the right the white regions are in the Julia set, black points are outside the Julia set.

To create images of the Julia set we map pixels in the image onto values of z0 and colour the pixel dependent on the behaviour of the series. In the examples on the right the image is mapped onto the range +-2pi in both the real and imaginary axes. The series is tested after 50 terms, it is decided that it tends to infinity if the absolute value of the imaginary part of zk is greater than 50.

See also: https://paulbourke.net/fractals/sinjulia/

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08a_Julia08.py, and the XML code in the file D08a_Julia08.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08b_Julia08.py, and the XML code in the file D08b_Julia08.2D.xml.

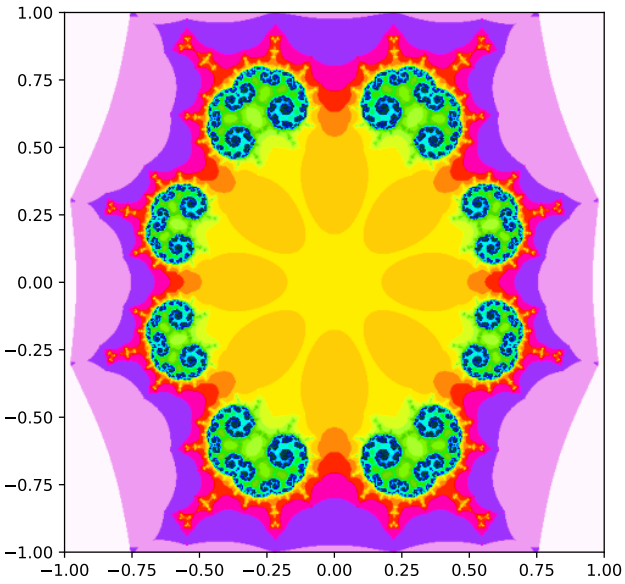


19.3.9 Julia sets, example 9: Julia09 fractals

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09a_Julia09.py, and the XML code in the file D09a_Julia09.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09b_Julia09.py, and the XML code in the file D09b_Julia09.2D.xml.

See also: https://paulbourke.net/fractals/cubicjulia/

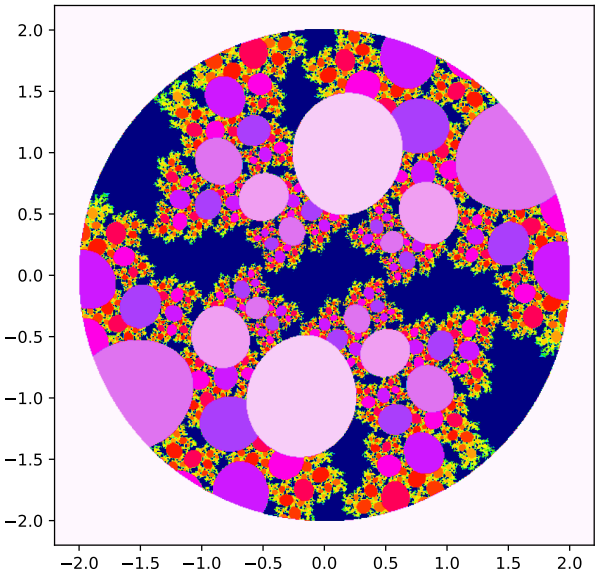


19.3.10 Julia sets, example 10: Julia10 fractals

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10a_Julia10.py, and the XML code in the file D10a_Julia10.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10b_Julia10.py, and the XML code in the file D10b_Julia10.2D.xml.

See also: https://paulbourke.net/fractals/cubicjulia/

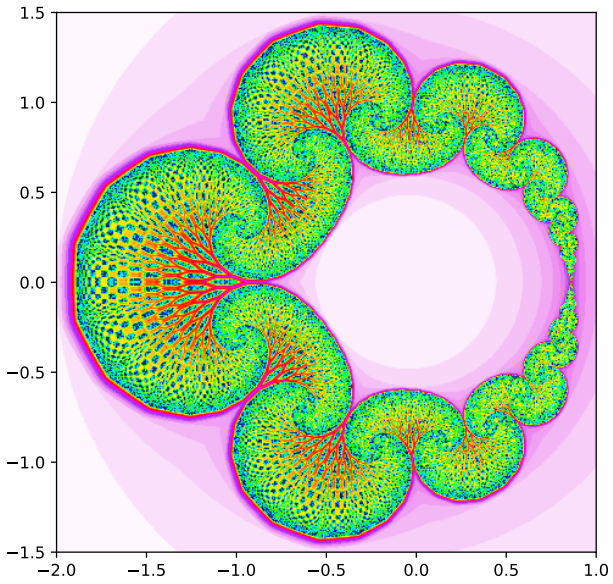


19.3.11 Julia sets, example 11: Julia11 fractals

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11a_Julia11.py, and the XML code in the file D11a_Julia11.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11b_Julia11.py, and the XML code in the file D11b_Julia11.2D.xml.

See also: https://paulbourke.net/fractals/cubicjulia/



19.4 Newton Fractals

See also: http://yozh.org/2012/10/02/newton_fractals/

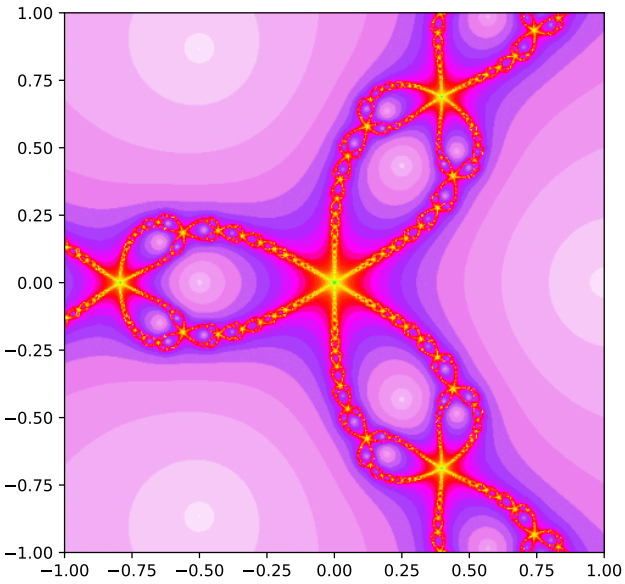
See also: http://www.3d-meier.de/, Mandelbrot und Julia Mengen

See also: https://en.wikipedia.org/wiki/Newton_fractal

19.4.1 Newton, zeros of $z^3 - 1$

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01a_Newton1.py, and the XML code in the file D01a_Newton1.2D.xml.

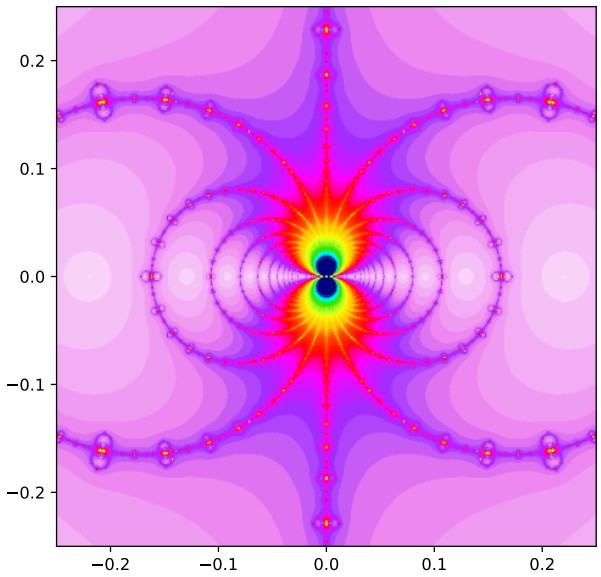
The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01b_Newton1.py, and the XML code in the file D01b_Newton1.2D.xml.



19.4.2 Newton, zeros of $\cos(z)$

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02a_Newton2.py, and the XML code in the file D02a_Newton2.2D.xml.

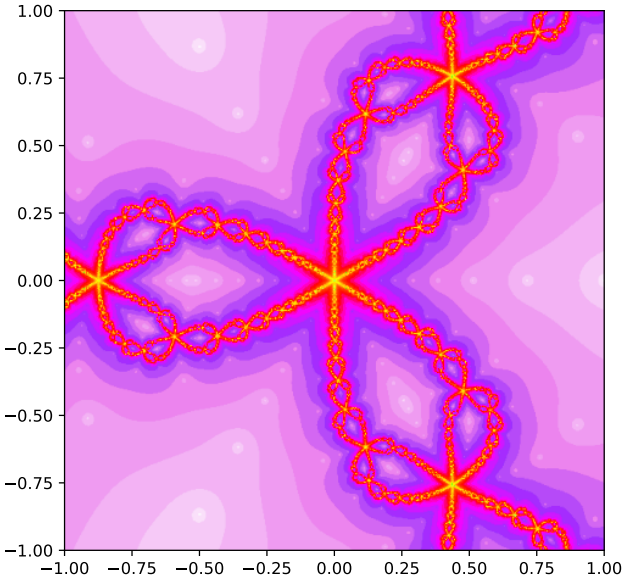
The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02b_Newton2.py, and the XML code in the file D02b_Newton2.2D.xml.



19.4.3 Modified Newton (containing an error), zeros of $z^3 - 1$

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03a_Newton3.py, and the XML code in the file D03a_Newton3.2D.xml.

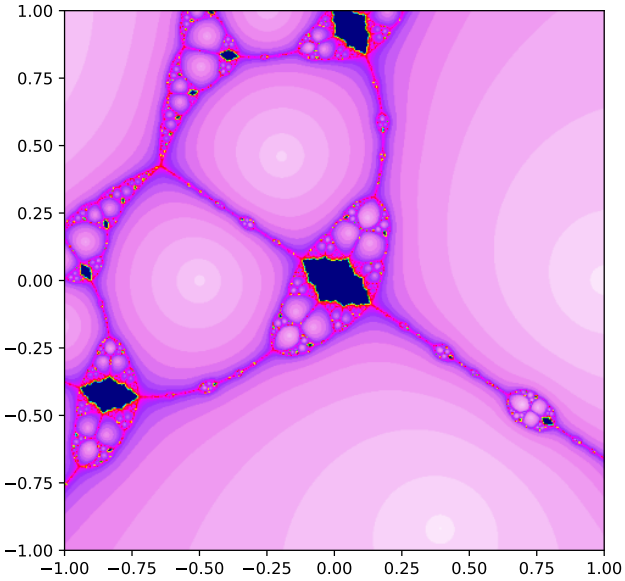
The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03b_Newton3.py, and the XML code in the file D03b_Newton3.2D.xml.



19.4.4 Modified Newton (containing an error), not converging

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04a_Newton4.py, and the XML code in the file D04a_Newton4.2D.xml.

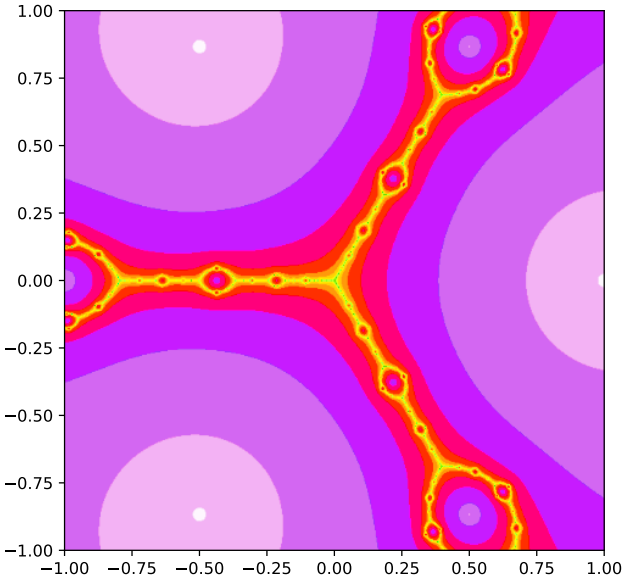
The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04b_Newton4.py, and the XML code in the file D04b_Newton4.2D.xml.



19.4.5 Halley, zeros of $z^3 - 1$

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05a_Newton5.py, and the XML code in the file D05a_Newton5.2D.xml.

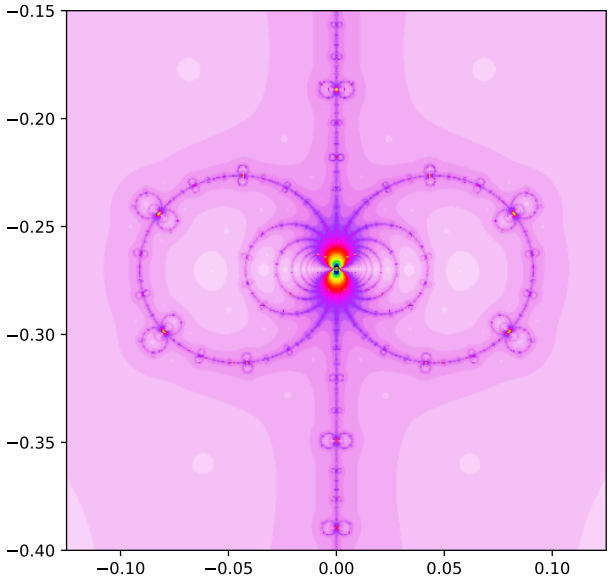
The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05b_Newton5.py, and the XML code in the file D05b_Newton5.2D.xml.



19.4.6 Halley, zeros of $\cos(z)$

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06a_Newton6.py, and the XML code in the file D06a_Newton6.2D.xml.

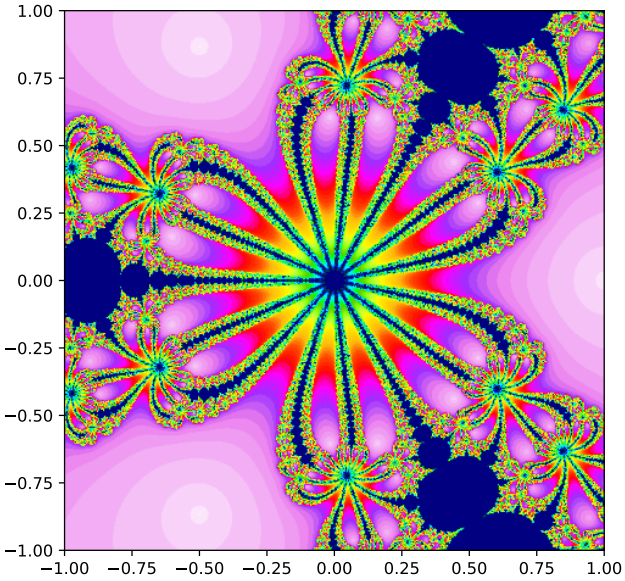
The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06b_Newton6.py, and the XML code in the file D06b_Newton6.2D.xml.



19.4.7 Modified Halley (containing an error), zeros of $z^3 - 1$

The Python code for the example below, *without* using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_Newton7.py, and the XML code in the file D07a_Newton7.2D.xml.

The Python code for the example below, using `numba`, can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_Newton7.py, and the XML code in the file D07b_Newton7.2D.xml.



19.5 Domain coloring

See also: https://www.dynamicmath.xyz/domain-coloring/

See also: https://iquilezles.org/articles/palettes/

See also: https://github.com/jcponce/complex/tree/gh-pages/dctools

See also: Wegert [794].

See also: Matplotlib and MayaVi implementation of domain coloring by E. Petrisor, https://nbviewer.org/github/empet/Math/blob/master/DomainColoring.ipynb

See also: https://en.wikipedia.org/wiki/Domain_coloring

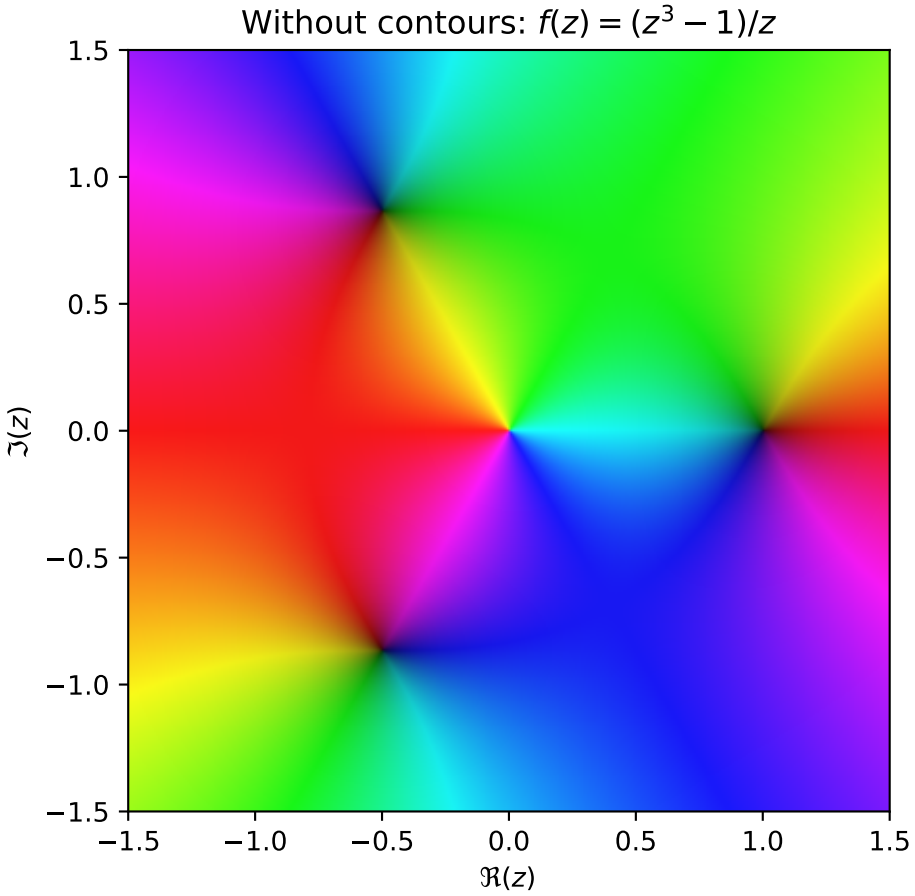
See also: https://notebook.community/empet/Math/DomainColoring

19.5.1 Domain coloring without contours: $f(z) = (z^3 - 1)/z$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:(z**3-1)/z, style='c', re=[-1.5, 1.5],
            im=[-1.5, 1.5], Title=r'$f(z)=\dfrac{z^3-1}{z}$', daxis=True)
```

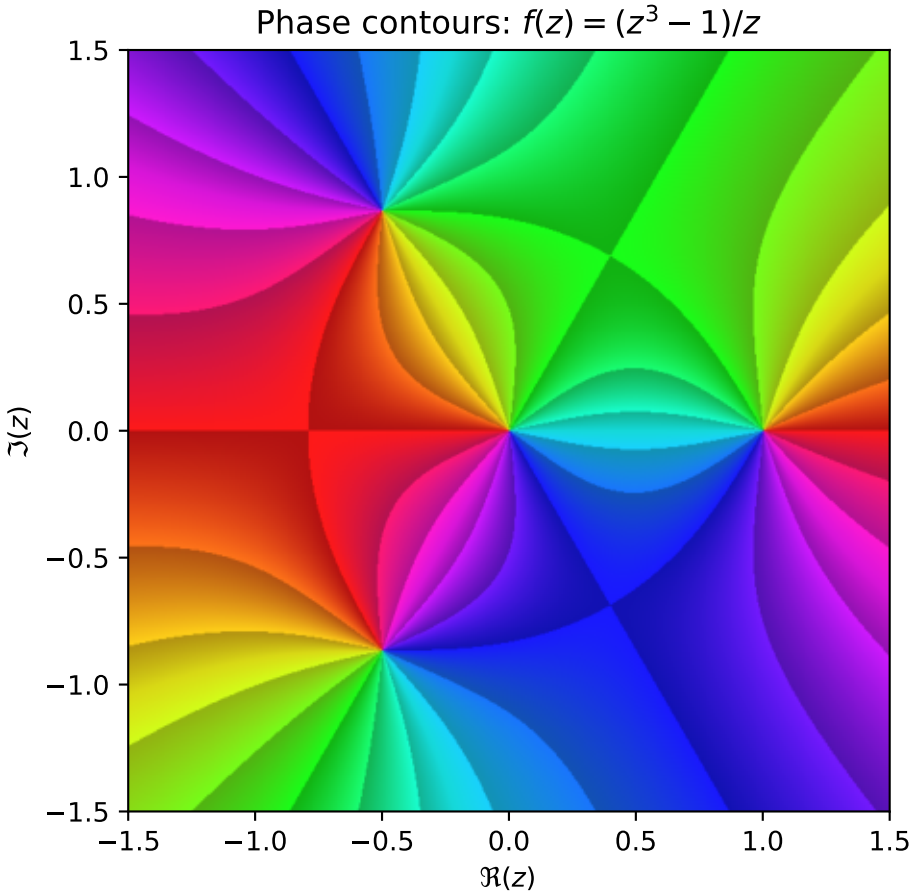


19.5.2 Domain coloring with contours of the phase: $f(z) = (z^3 - 1)/z$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:(z**3-1)/z, style='p', re=[-1.5, 1.5],
            im=[-1.5, 1.5], Title=r'$f(z)=\dfrac{z^3-1}{z}$', daxis=True)
```

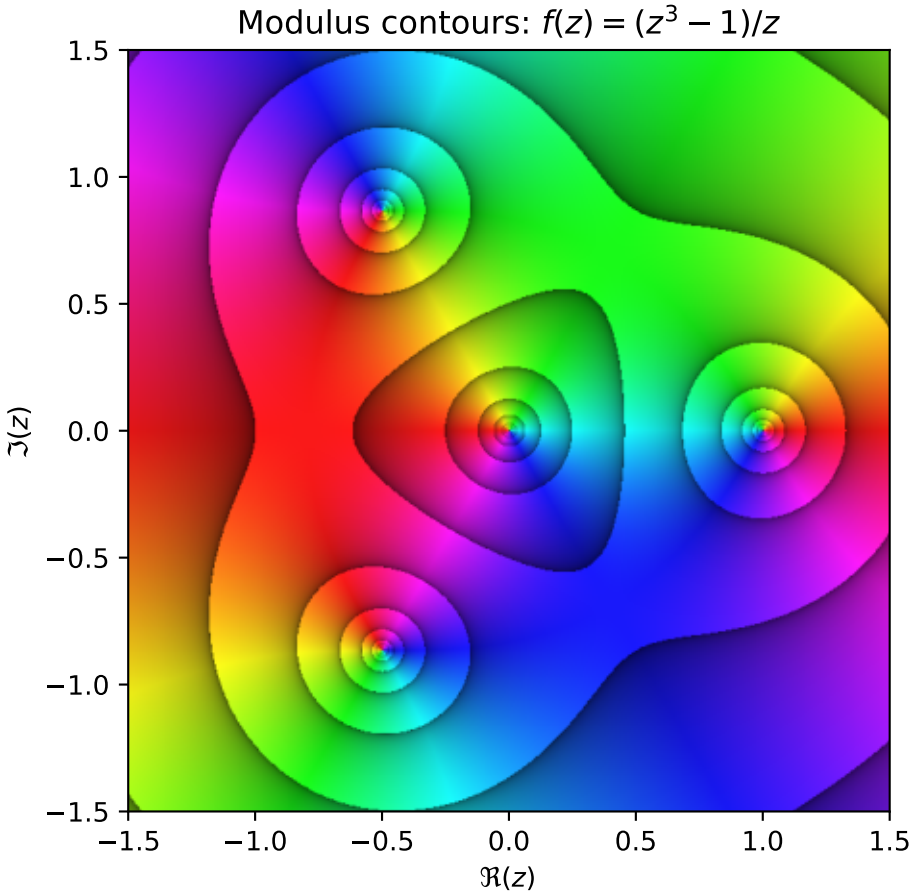


19.5.3 Domain coloring with contours of the modulus: $f(z) = (z^3 - 1)/z$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:(z**3-1)/z, style='m', re=[-1.5, 1.5],
            im=[-1.5, 1.5], Title=r'$f(z)=\dfrac{z^3-1}{z}$', daxis=True)
```

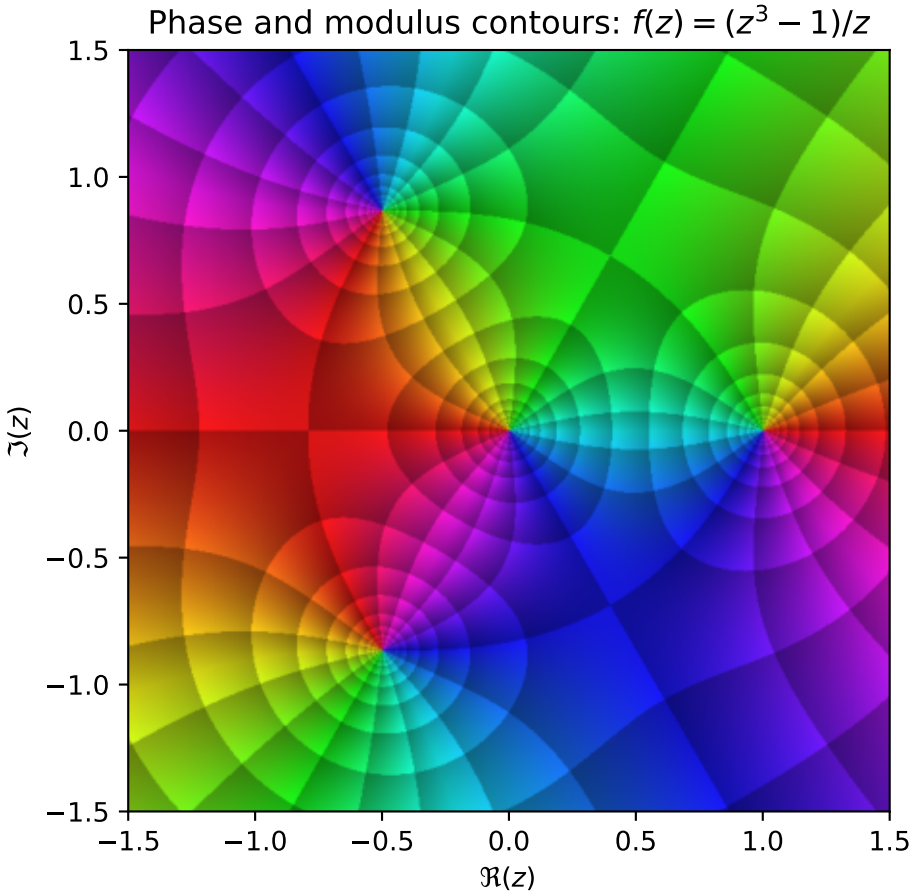


19.5.4 Domain coloring with contours of phase and modulus: $f(z) = (z^3 - 1)/z$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:(z**3-1)/z, style='pm', re=[-1.5, 1.5],
            im=[-1.5, 1.5], Title=r'$f(z)=\dfrac{z^3-1}{z}$', daxis=True)
```

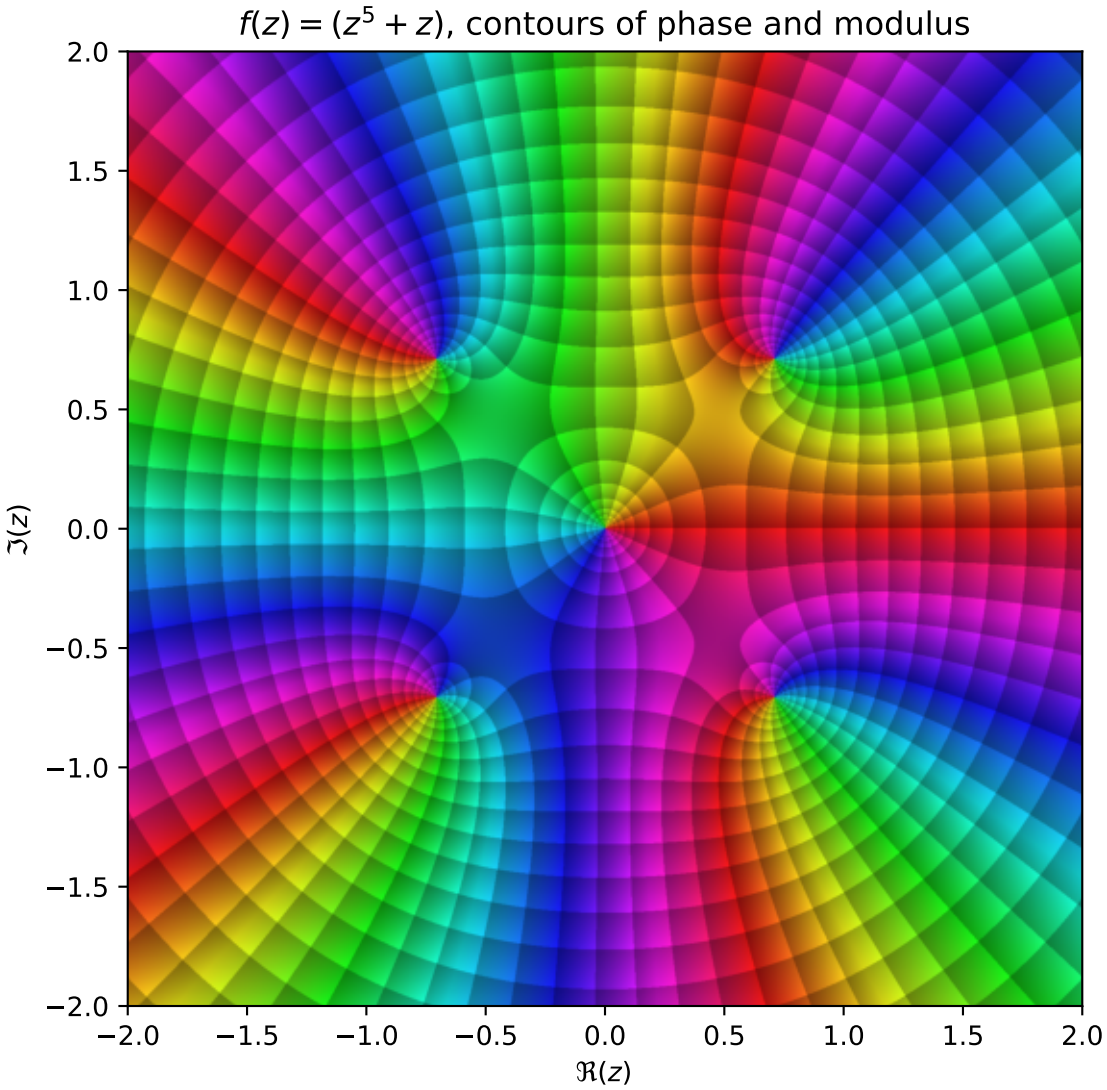


19.5.5 Domain coloring with contours of phase and modulus: $f(z) = z^5 + z$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:z**5 + z, style='pm', re=[-2, 2],
            im=[-2, 2], Title=r'$f(z)=(z^5+z)$', daxis=True)
```

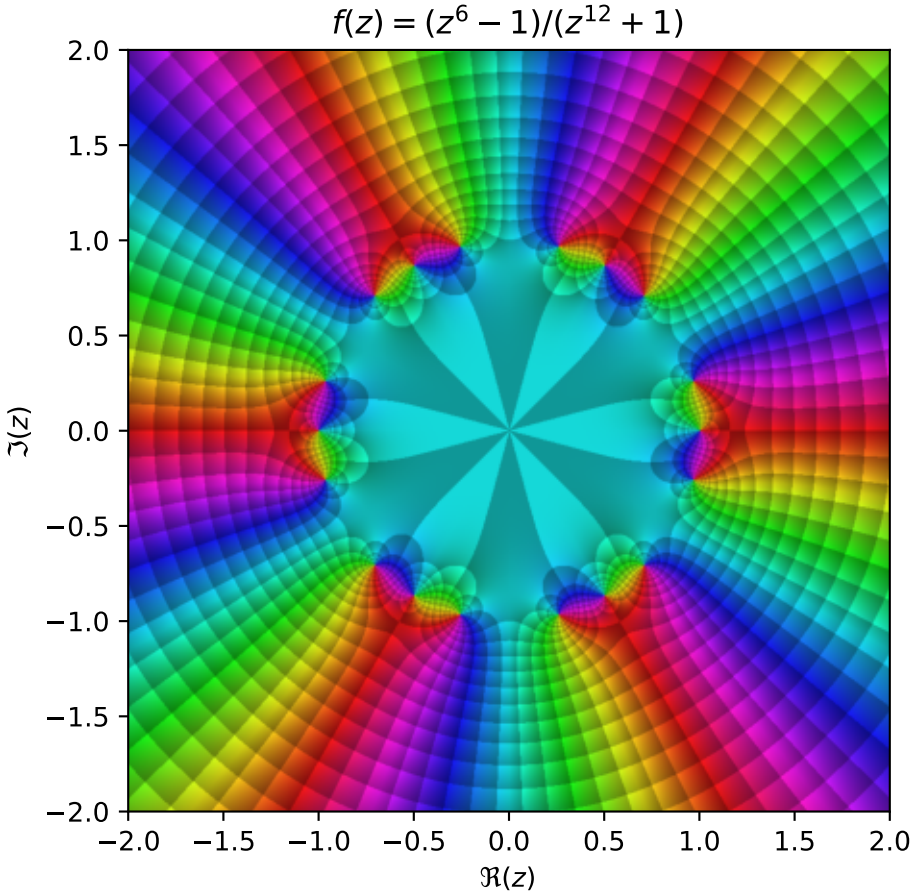


19.5.6 Domain coloring with contours of phase and modulus: $f(z) = (z^6 - 1)/(z^{12} + 1)$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:(z**6-1) / (z**12+1), style='pm', re=[-2.0, 2.0],
            im=[-2.0, 2.0], Title=r'$f(z)=\dfrac{z^6-1}{z^{12}+1}$', daxis=True)
```

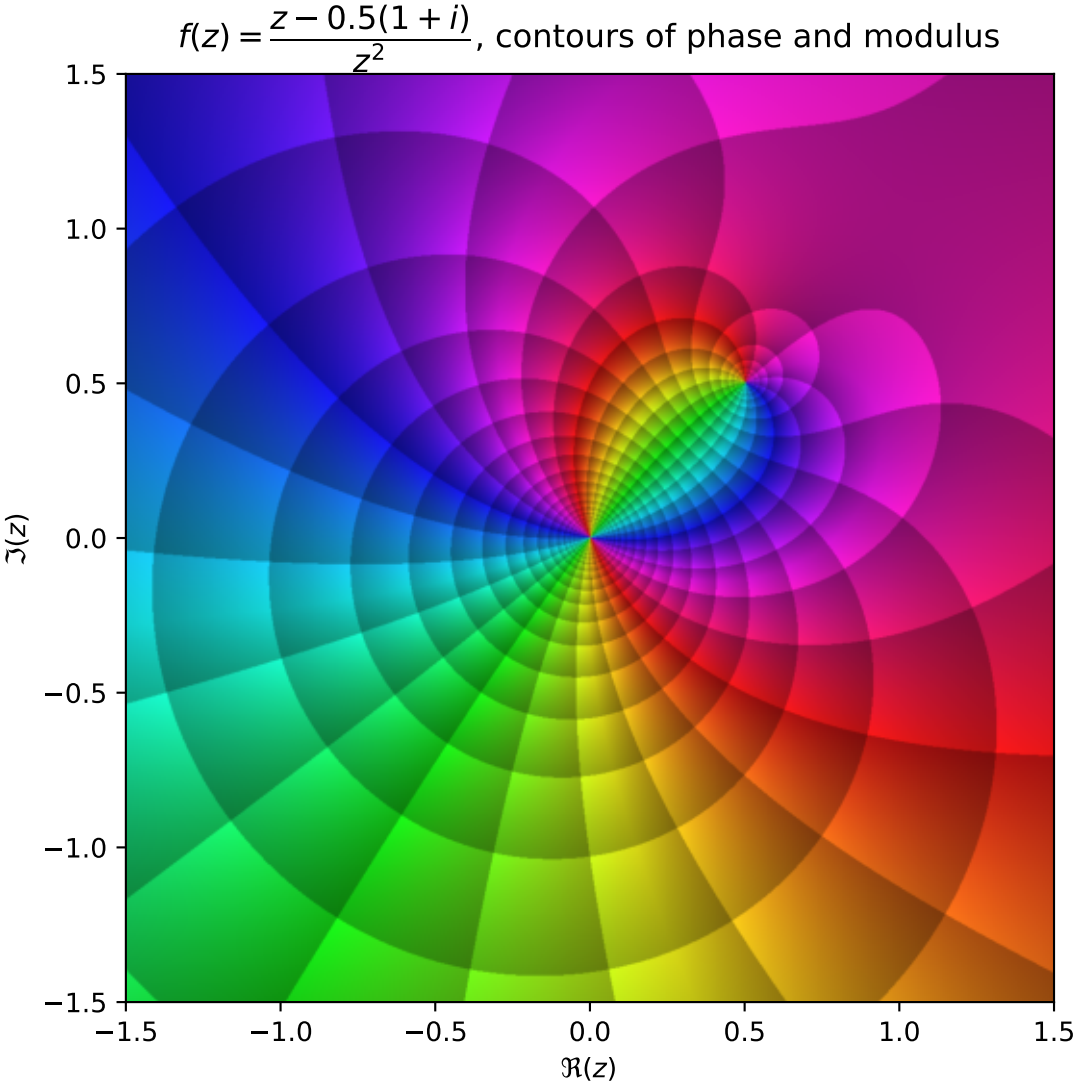


19.5.7 Domain coloring with contours of phase and modulus: $f(z) = (z - 0.5(1 + i))/z^2$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:(z - 0.5 - 0.5*1j) / z**2, style='pm', re=[-1.5, 1.5],
            im=[-1.5, 1.5], Title=r'$f(z)=\dfrac{z-0.5(1+i)}{z^2}$', daxis=True)
```

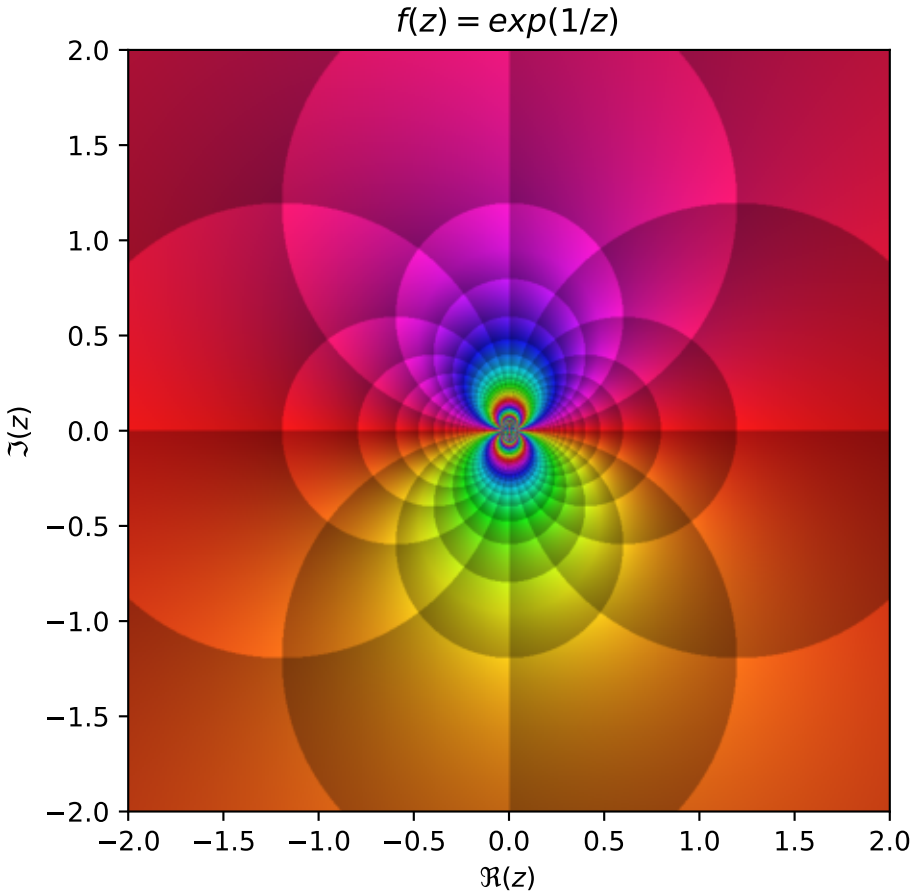


19.5.8 Domain coloring with contours of phase and modulus: $f(z) = \exp(1/z)$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:np.exp(1/z), style='pm', re=[-2.0, 2.0],
            im=[-2.0, 2.0], Title=r'$f(z)=\exp(1/z)$', daxis=True)
```

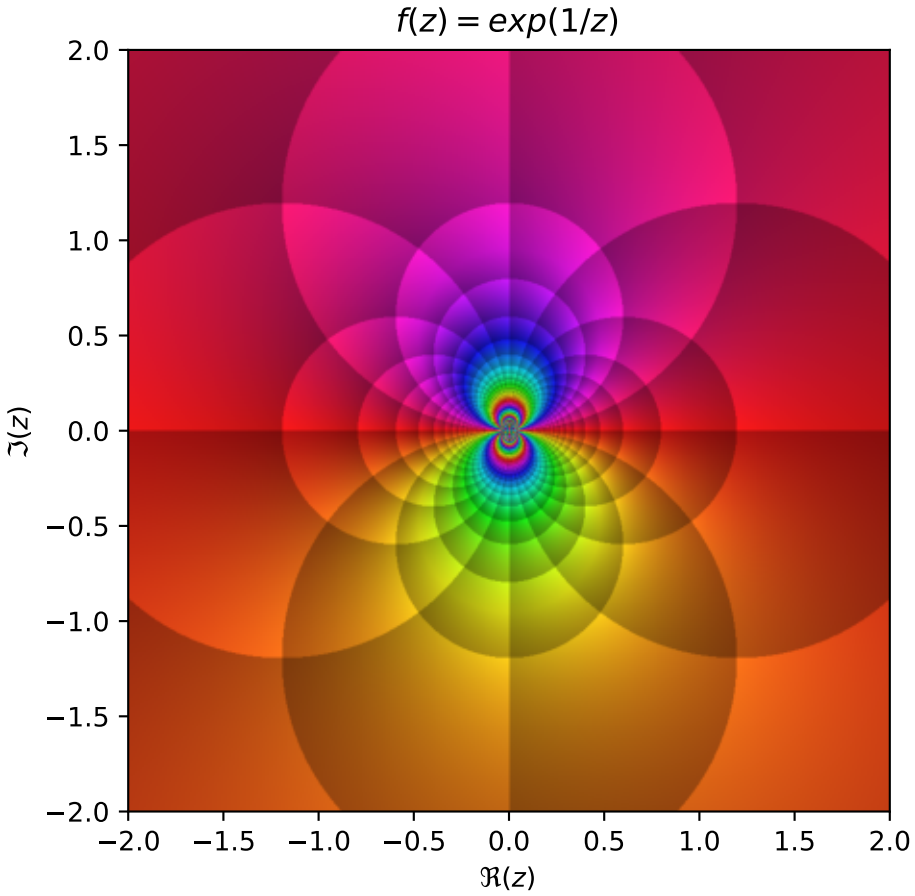


19.5.9 Domain coloring with contours of phase and modulus: $f(z) = z\exp(1/z)$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:np.exp(1/z), style='pm', re=[-2.0, 2.0],
            im=[-2.0, 2.0], Title=r'$f(z)=\exp(1/z)$', daxis=True)
```

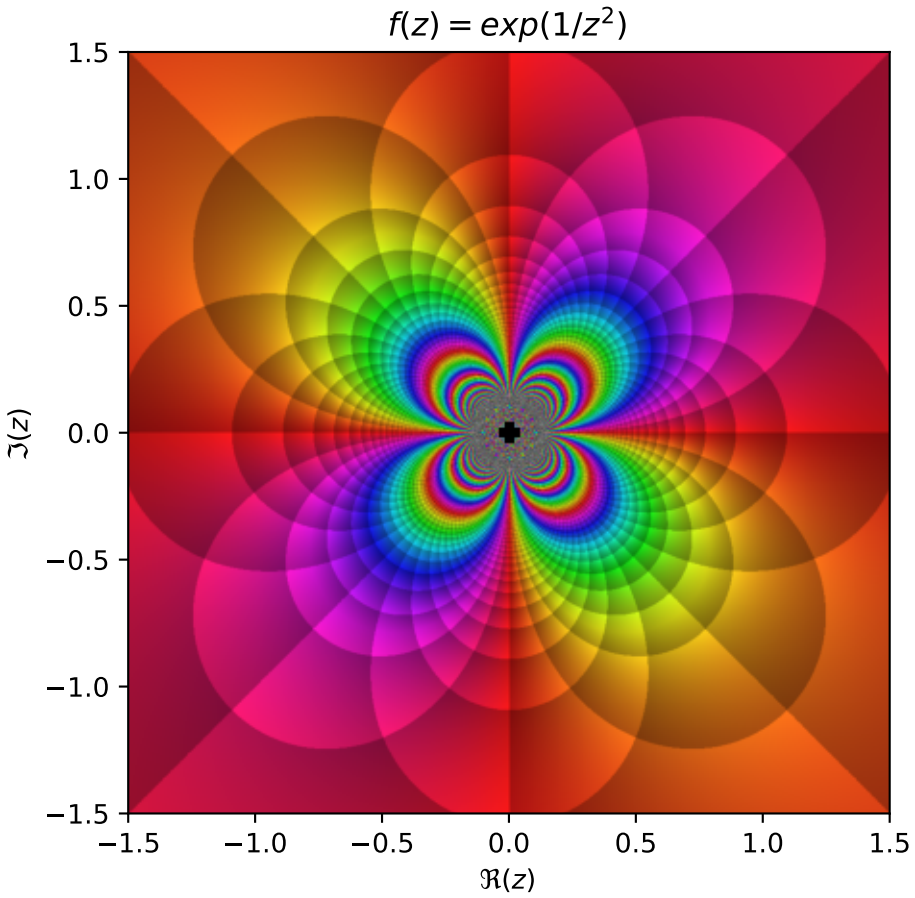


19.5.10 Domain coloring with contours of phase and modulus: $f(z) = \exp(1/z^2)$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:np.exp(1.0 / z**2), style='pm', re=[-1.5, 1.5],
            im=[-1.5, 1.5], Title=r'$f(z)=\exp(1/z^2)$', daxis=True)
```

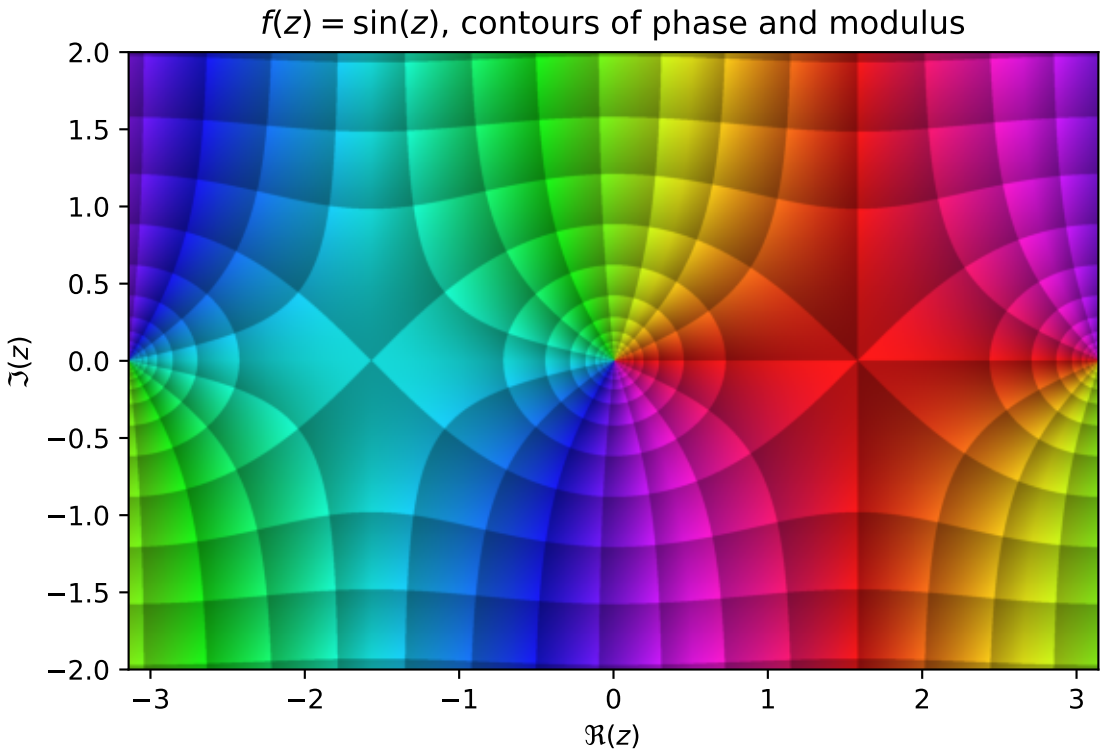


19.5.11 Domain coloring with contours of phase and modulus: $f(z) = \sin(z)$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:np.sin(z), style='pm', re=[-np.pi, np.pi],
            im=[-2, 2], Title=r'$f(z)=\sin(z)$', daxis=True)
```

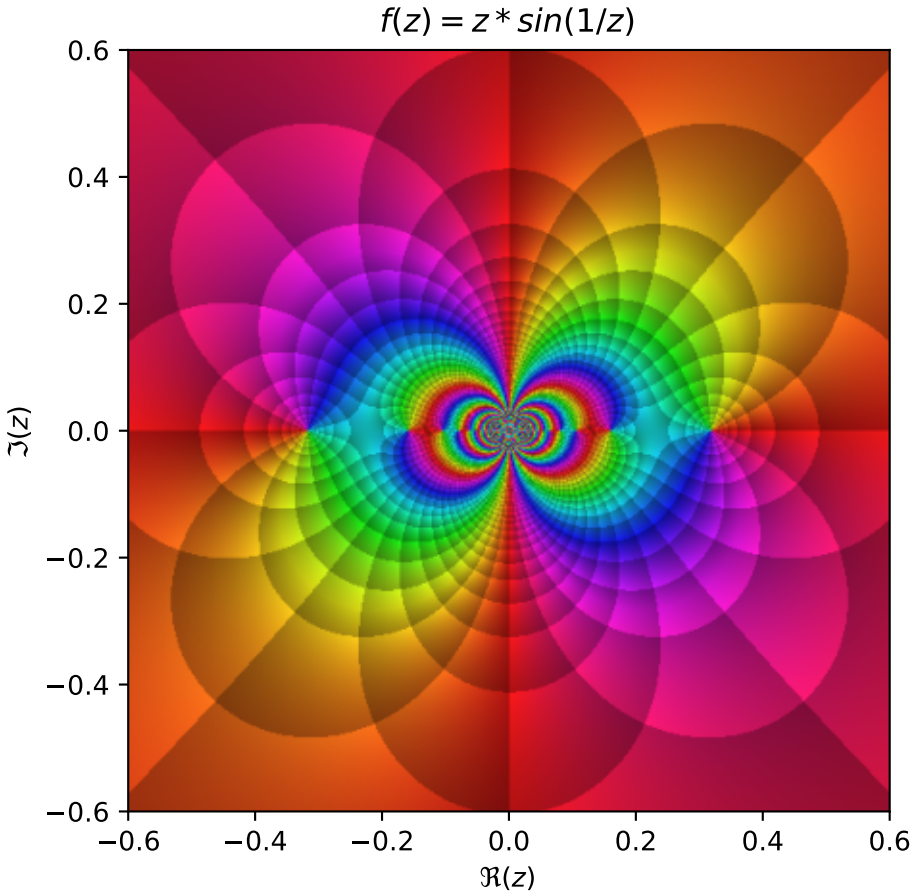


19.5.12 Domain coloring with contours of phase and modulus: $f(z) = z\sin(1/z)$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:z*np.sin(1/z), style='pm', re=[-0.6, 0.6],
            im=[-0.6, 0.6], Title=r'$f(z)=z \cdot \sin(1/z)$', daxis=True)
```

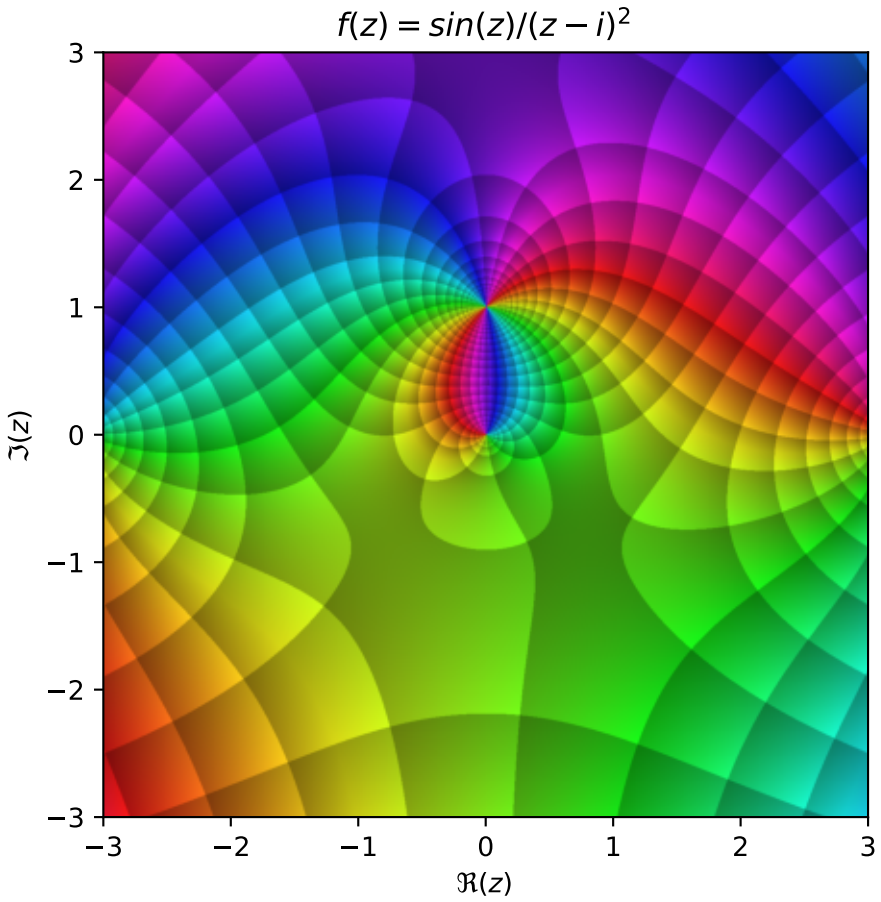


19.5.13 Domain coloring with contours of phase and modulus: $f(z) = \sin(z)/(z-i)^2$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:np.sin(z) / (z-1j)**2, style='pm', re=[-3, 3],
            im=[-3, 3], Title=r'$f(z)=\dfrac{\sin(z)}{(z-i)^2}$', daxis=True)
```

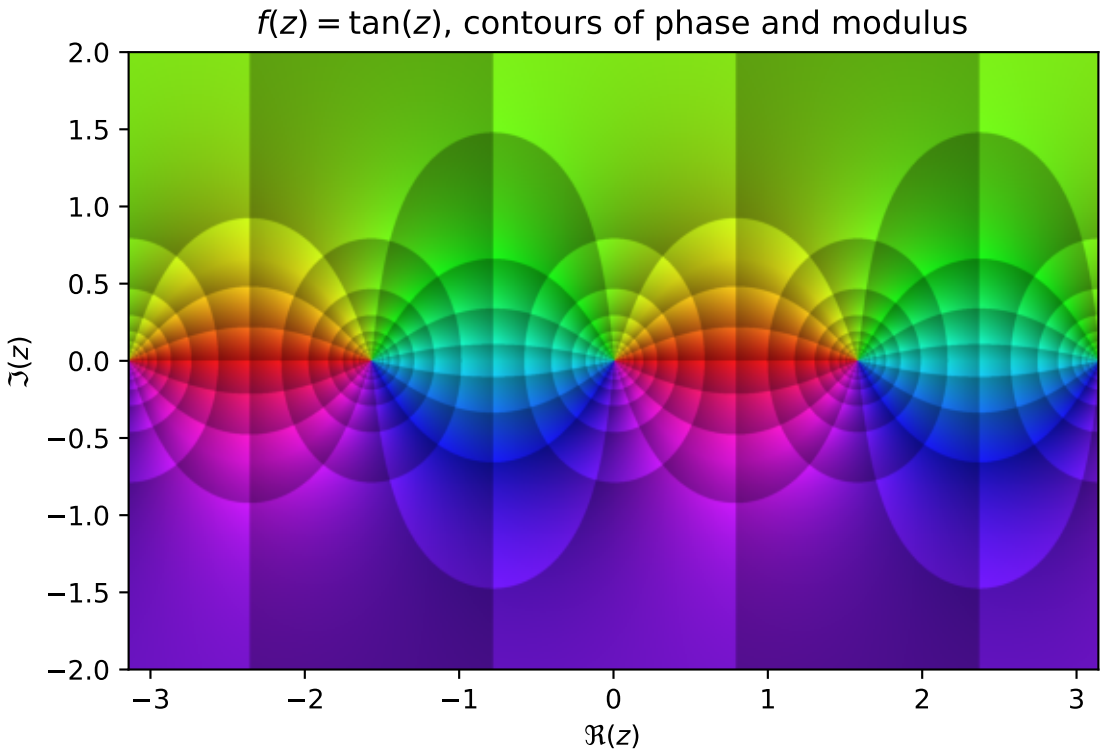


19.5.14 Domain coloring with contours of phase and modulus: $f(z) = \tan(z)$

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_DomainColoring.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        DomainColoring(func=lambda z:np.tan(z), style='pm', re=[-np.pi, np.pi],
            im=[-2, 2], Title=r'$f(z)=\tan(z)$', daxis=True)
```



CHAPTER

TWENTY

MATPLOTLIB 3D GRAPHICS

20.1 Introduction

20.1.1 Overview

Matplotlib was initially designed with only two-dimensional plotting in mind. Around the time of the 1.0 release, some three-dimensional plotting utilities were built on top of Matplotlib's two-dimensional display, and the result is a convenient (if somewhat limited) set of tools for three-dimensional data visualization. Three-dimensional plots are enabled by importing the mplot3d toolkit, included with the main Matplotlib installation.

Once this submodule is imported, a three-dimensional axes can be created by passing the keyword projection='3d' to any of the normal axes creation routines.

With this three-dimensional axes enabled, we can now plot a variety of three-dimensional plot types. Three-dimensional plotting is one of the functionalities that benefits immensely from viewing figures interactively.

See also: https://matplotlib.org/stable/api/toolkits/mplot3d/faq.html

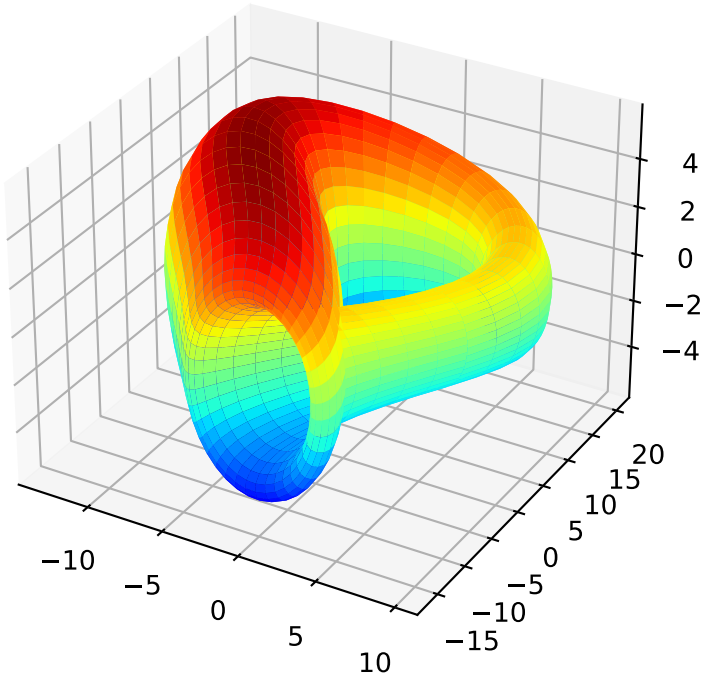
mplot3d was intended to allow users to create simple 3D graphs with the same "look-and-feel" as matplotlib's 2D plots. Furthermore, users can use the same toolkit that they are already familiar with to generate both their 2D and 3D plots.

20.2 Matplotlib: 3D Graphics (from Matplotlib tutorials)

See Test2DFunctions

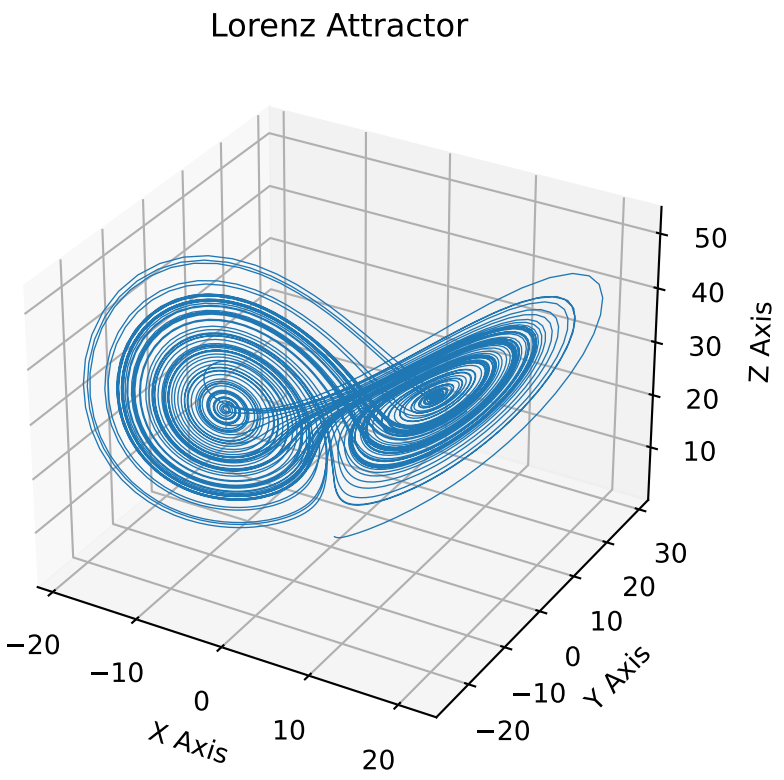
20.2.1 Klein bottle

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_KleinBottle.py, and the XML code in the file D01_DomainColoringRF1.2D.xml.



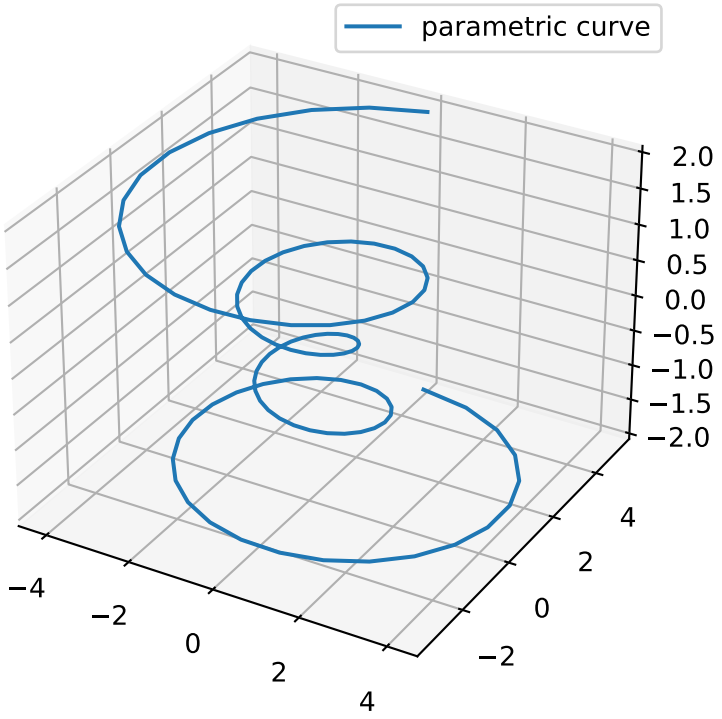
20.2.2 Lorentz attractor

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_LorenzAttractor.py, and the XML code in the file D02_LorenzAttractor.2D.xml.



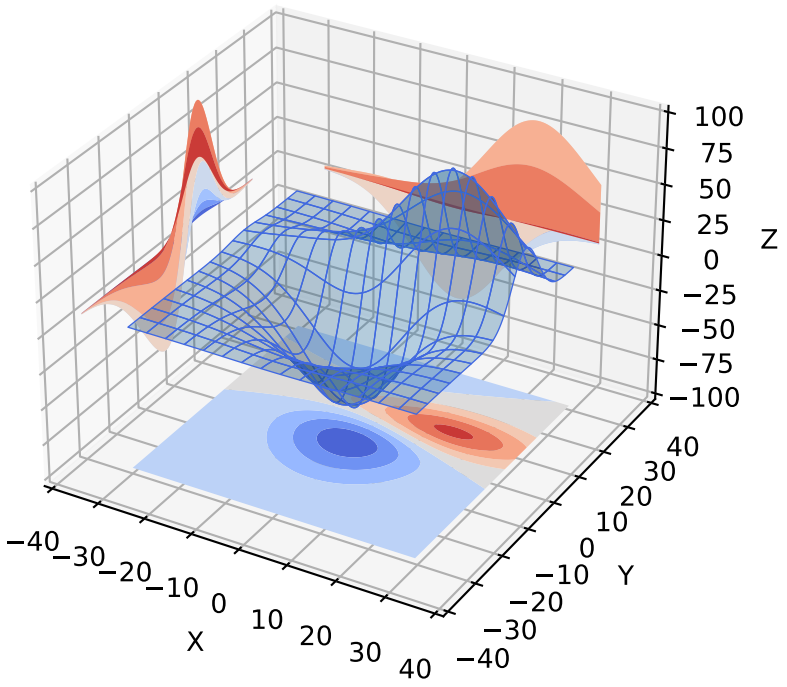
20.2.3 Parametric curve

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_ParametricCurve.py, and the XML code in the file D03_ParametricCurve.2D.xml.



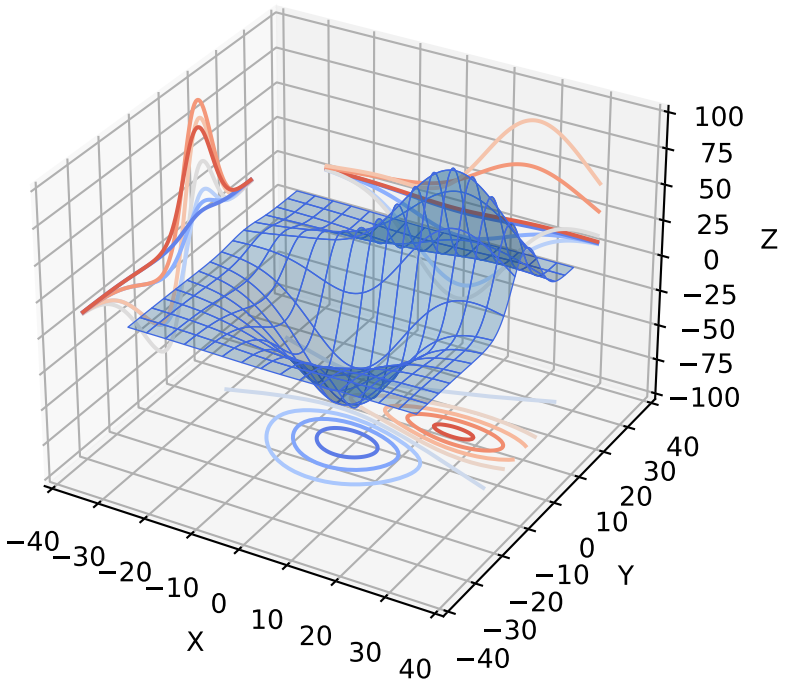
20.2.4 Projected filled contour

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_ProjectFilledContour.py, and the XML code in the file D04_ProjectFilledContour.2D.xml.



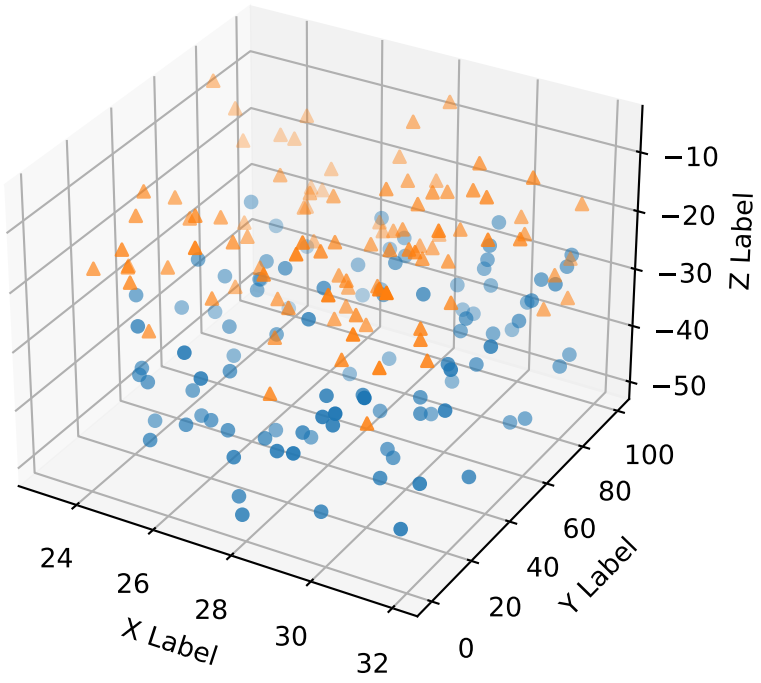
20.2.5 Projected open contour

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_ProjectOpenContour.py, and the XML code in the file D05_ProjectOpenContour.2D.xml.



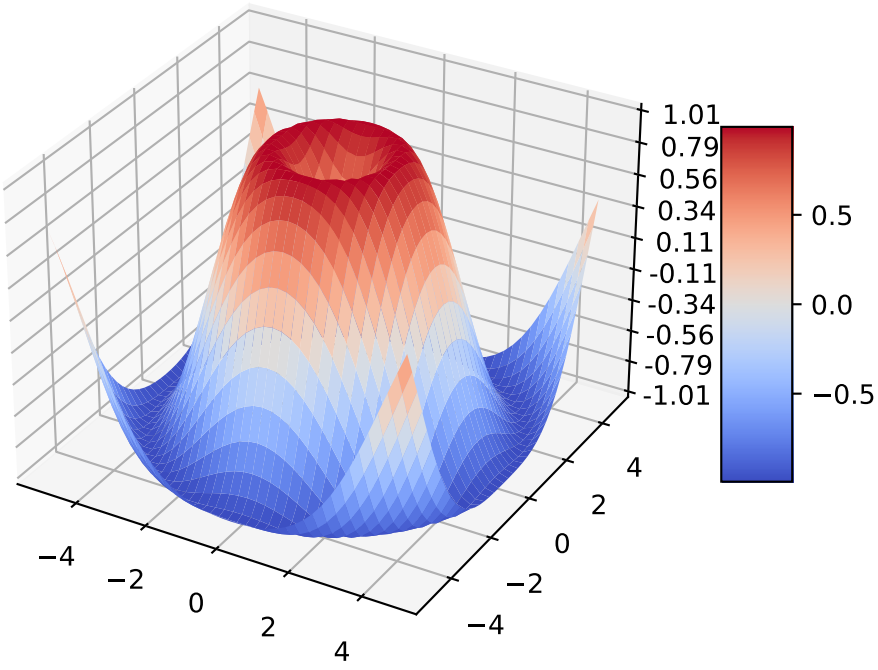
20.2.6 Scatterplot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Scatterplot3D.py, and the XML code in the file D06_Scatterplot3D.2D.xml.



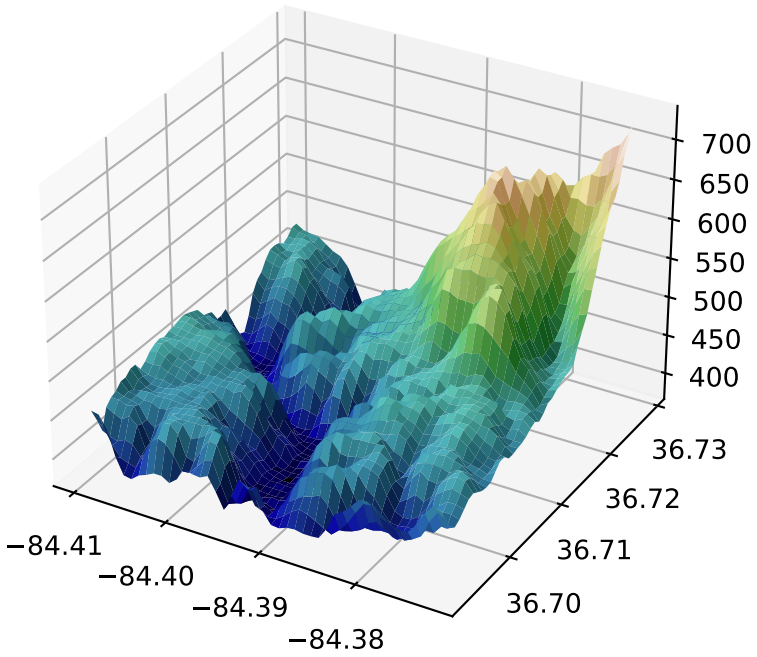
20.2.7 Surface colormap

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_SurfaceColormap.py, and the XML code in the file D07_SurfaceColormap.2D.xml.



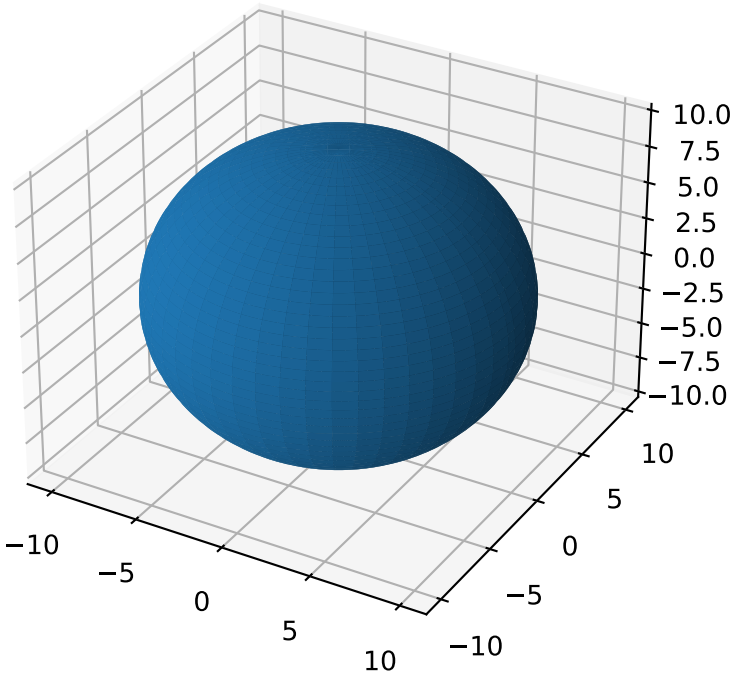
20.2.8 Surface hillshading

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_SurfaceHillshading.py, and the XML code in the file D08_SurfaceHillshading.2D.xml.



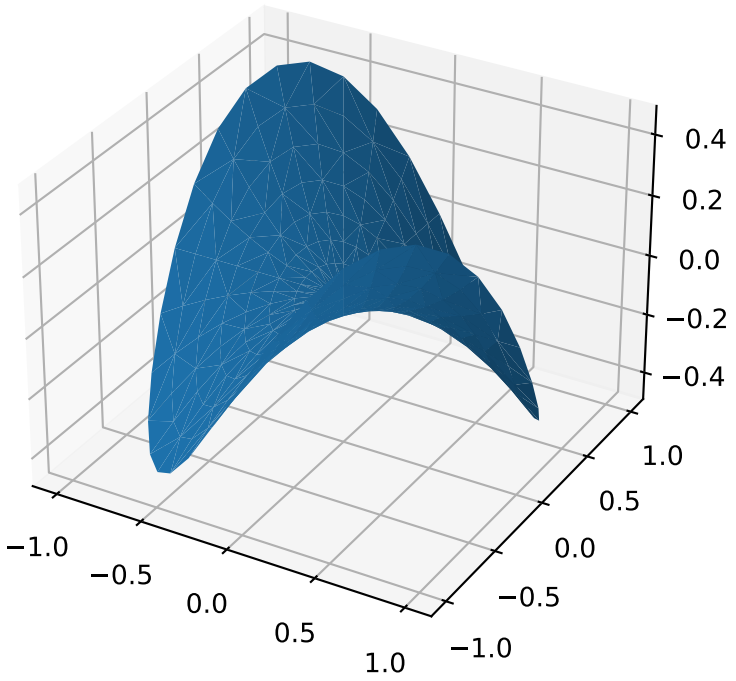
20.2.9 Surface solid

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_SurfaceSolid.py, and the XML code in the file D09_SurfaceSolid.2D.xml.



20.2.10 Surface triangular

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_SurfaceTriangular.py, and the XML code in the file D10_SurfaceTriangular.2D.xml.

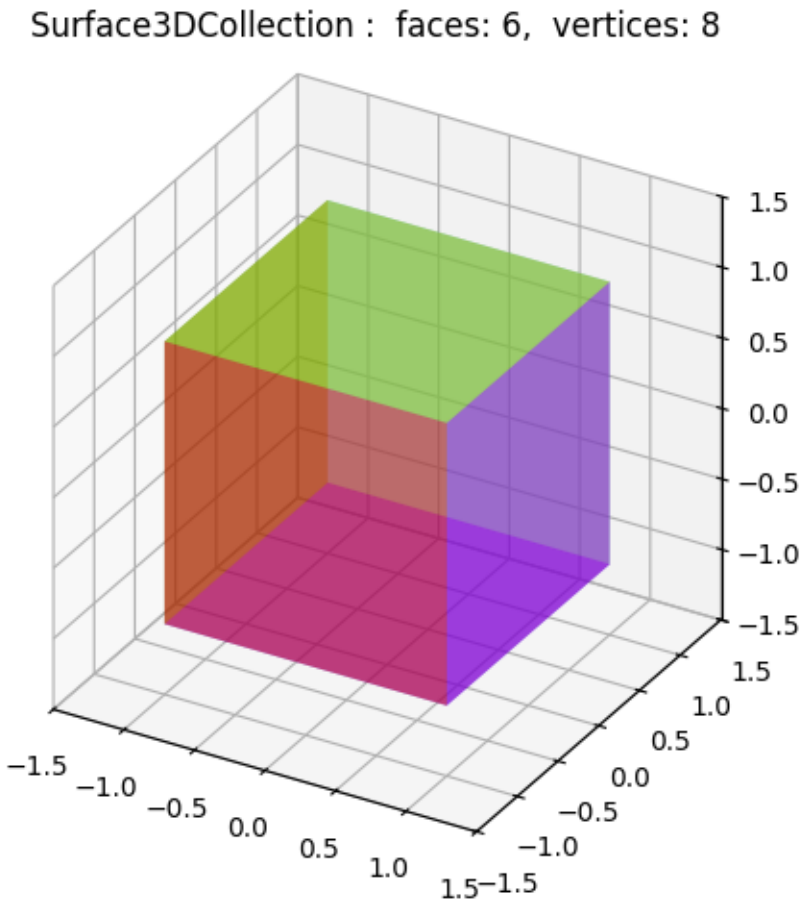


20.3 S3dlib: Basic geometric figures

20.3.1 Base 3D class surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Base3DClassSurface.py, and the XML code in the file D01_Base3DClassSurface.2D.xml.

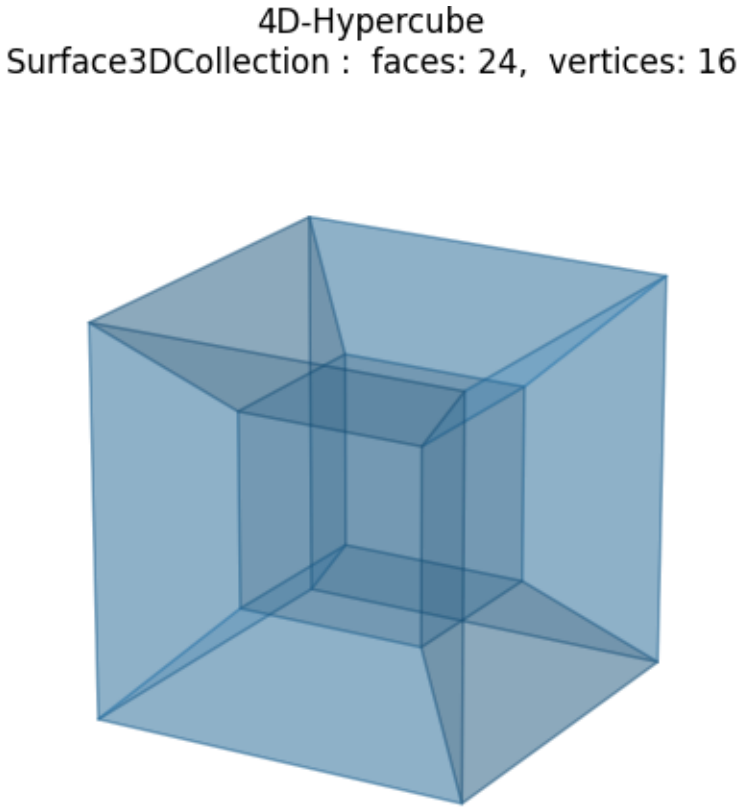
See: https://s3dlib.org/examples/functional/cube.html



20.3.2 Hypercube

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Hypercube.py, and the XML code in the file D02_Hypercube.2D.xml.

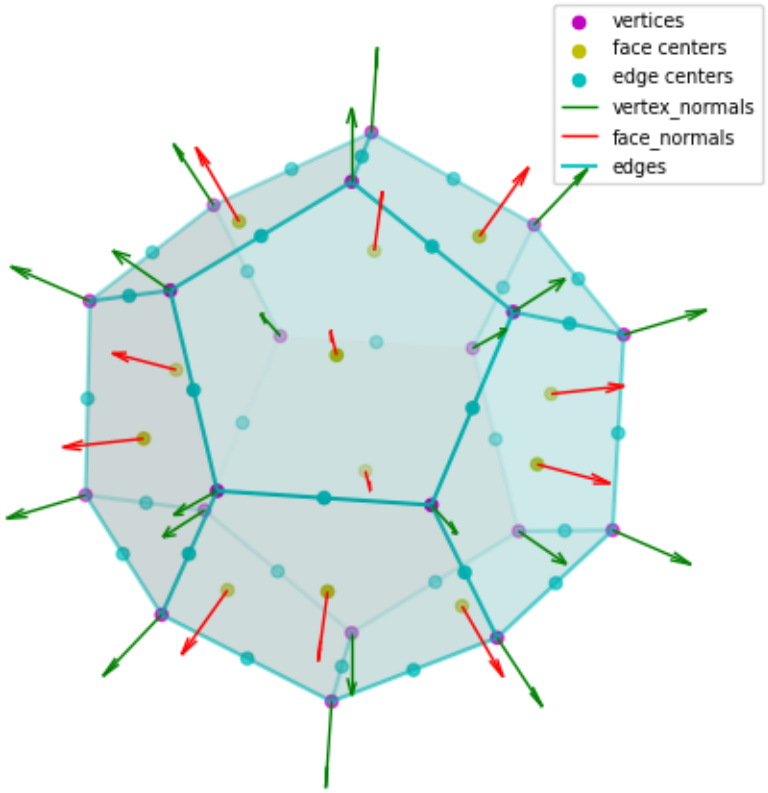
See: https://s3dlib.org/examples/functional/hypercube.html



20.3.3 Surface elements

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_SurfaceElements.py, and the XML code in the file D03_SurfaceElements.2D.xml.

See: https://s3dlib.org/examples/functional/fev_fn_vn.html



SphericalSurface (0 , dodeca*): faces: 12, vertices: 20

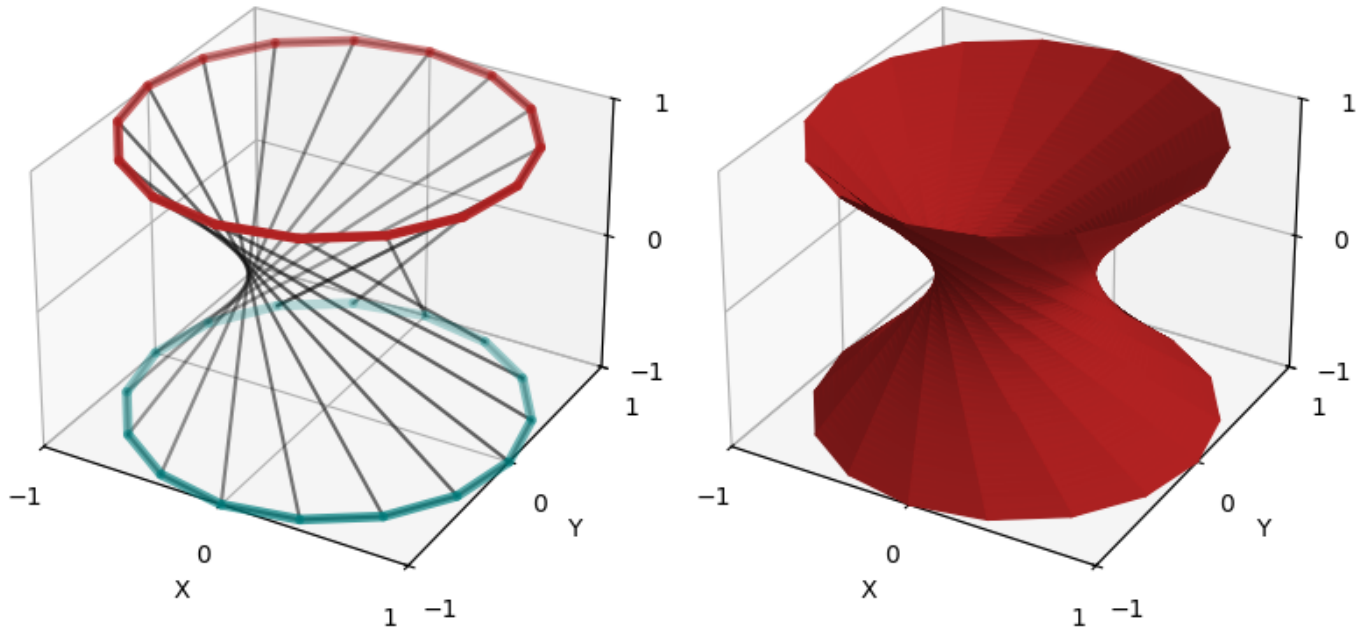
20.3.4 Hyperboloid 3D

See also: https://mathworld.wolfram.com/One-SheetedHyperboloid.html

See also: https://en.wikipedia.org/wiki/Hyperboloid

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Hyperboloid3D.py, and the XML code in the file D04_Hyperboloid3D.2D.xml .

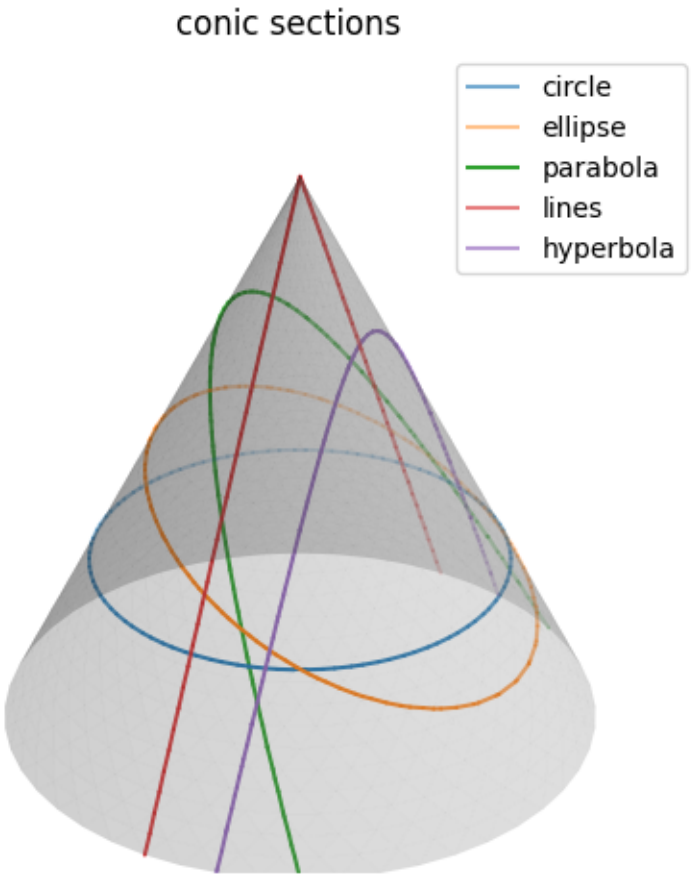
See: https://github.com/fzaverl/s3dlib



20.3.5 Conic sections

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Conic_Sections.py, and the XML code in the file D05_Conic_Sections.2D.xml.

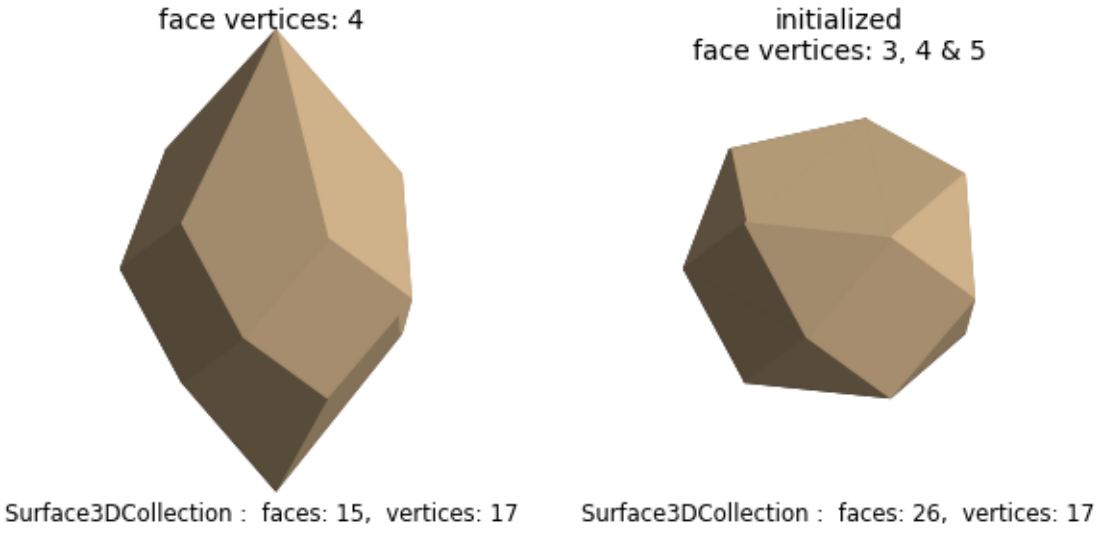
See: https://s3dlib.org/examples/lines/conic.html



20.3.6 Base 3D class surface, part 2

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Base3DClassSurface2.py, and the XML code in the file D06_Base3DClassSurface2.2D.xml.

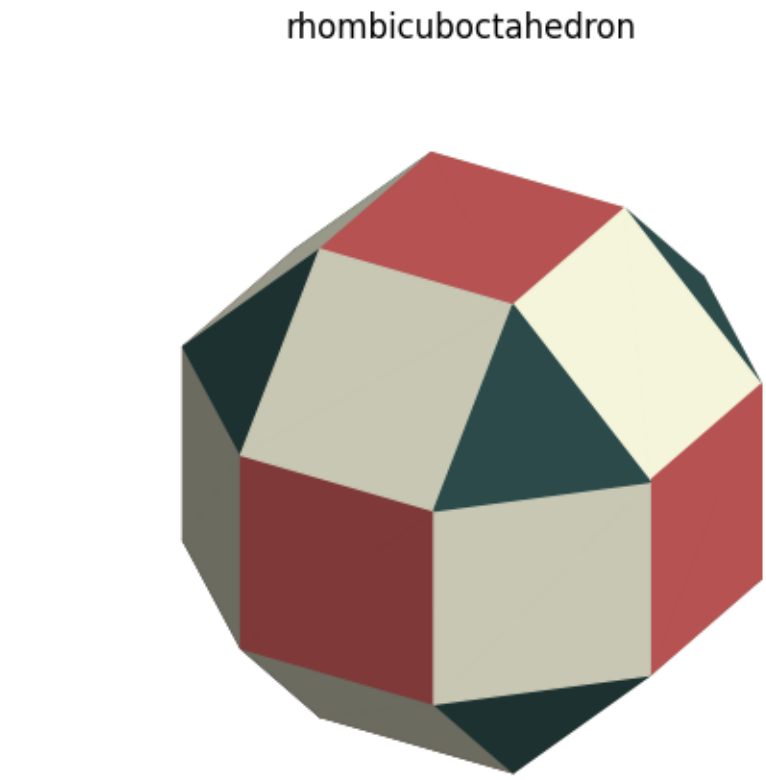
See: https://s3dlib.org/examples/functional/base_surface.html



20.3.7 Rhombicuboctahedron

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_RhombicuboctahedronPlain.py, and the XML code in the file D07a_RhombicuboctahedronPlain.2D.xml.

See: https://s3dlib.org/examples/functional/arch_solid.html

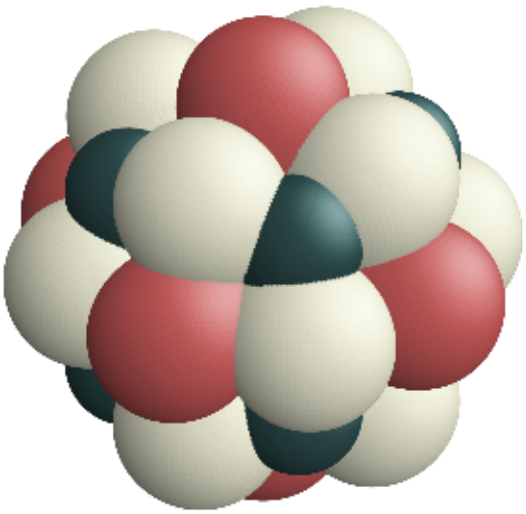


Surface3DCollection : faces: 44, vertices: 24

20.3.8 Rhombicuboctahedron, bubbles

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07b_RhombicuboctahedronBubbles.py, and the XML code in the file D07b_RhombicuboctahedronBubbles.2D.xml.

See: https://s3dlib.org/examples/functional/arch_solid.html



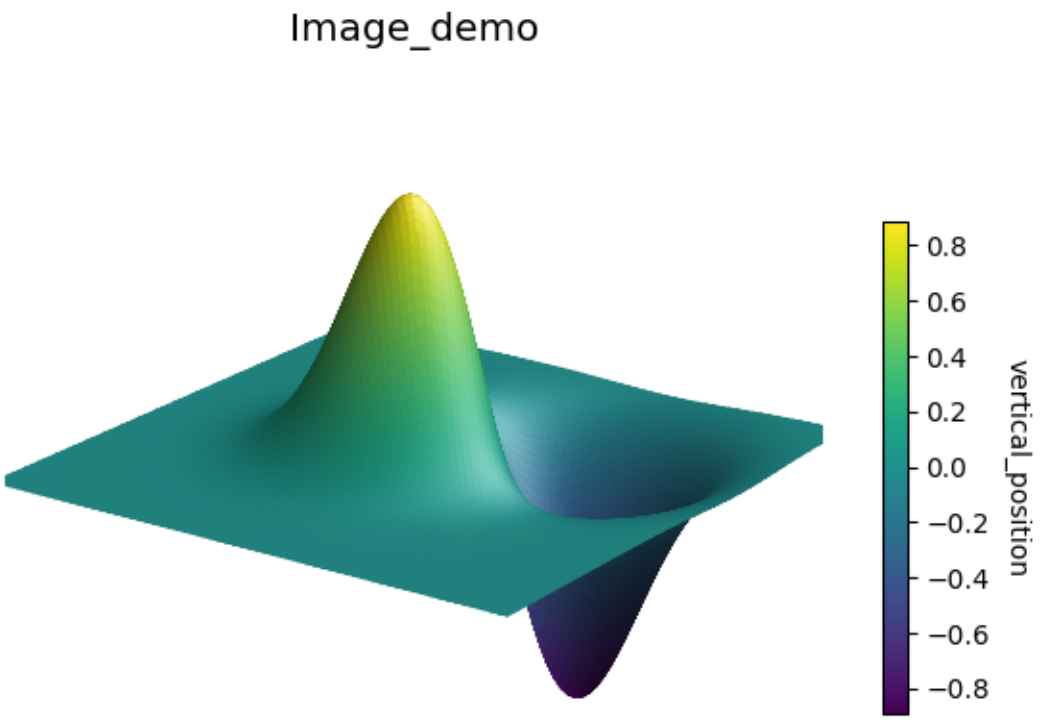
```
Surface3DCollection :  faces: 45056,  vertices: 22530
```

20.4 S3dlib: Real functions

20.4.1 Surface plot of a real function, basic example

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_HelloWorldExample.py, and the XML code in the file D01_HelloWorldExample.2D.xml.

See: https://s3dlib.org/examples/functional/hw_example.html

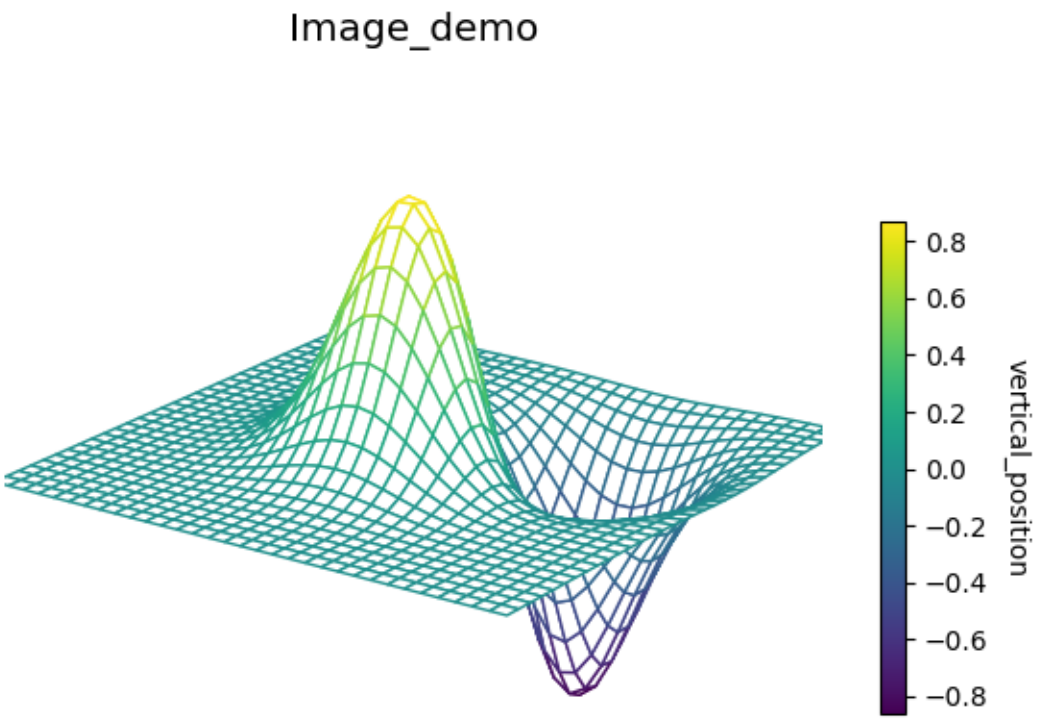



```
PlanarSurface ( 6 , quad ):  faces: 16384,  vertices: 8321
```

20.4.2 Surface plot of a real function, grid

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_HelloWorldExampleGrid.py, and the XML code in the file D02_HelloWorldExampleGrid.2D.xml.

See: https://s3dlib.org/examples/functional/hw_example_grid.html

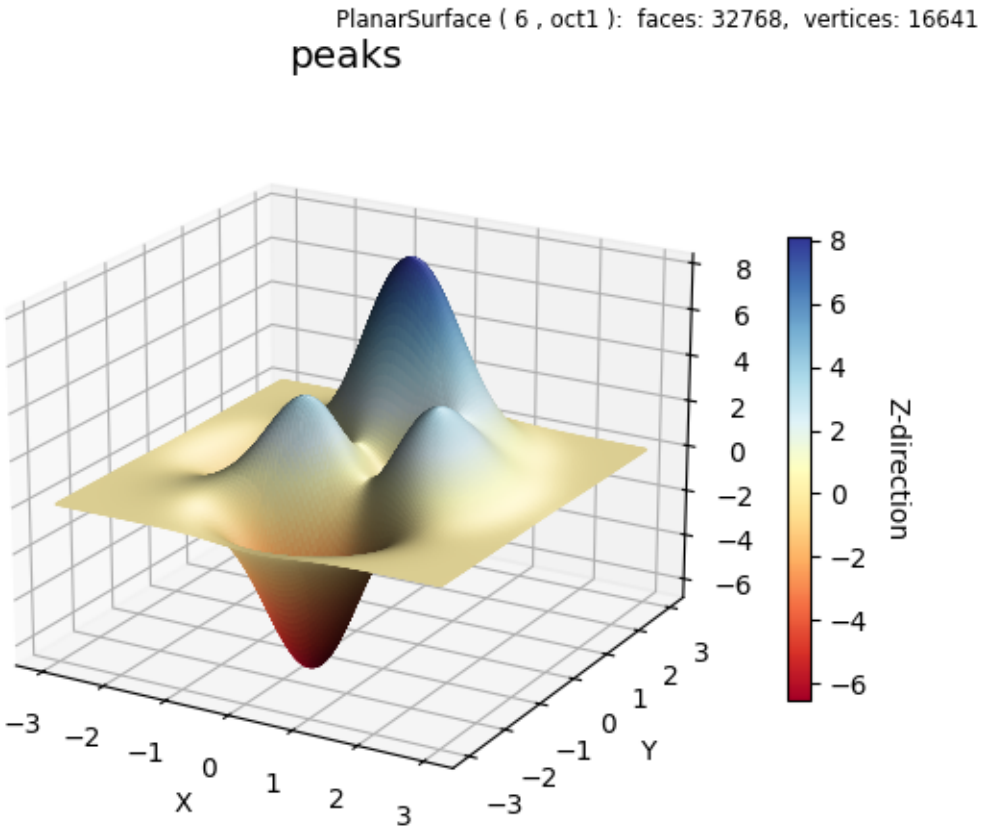



```
PlanarSurface ( 4 , squ ):  faces: 1024,  vertices: 1089
```

20.4.3 Surface plot of a real function, peaks example

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_HelloWorldExample2b.py, and the XML code in the file D03_HelloWorldExample2b.2D.xml.

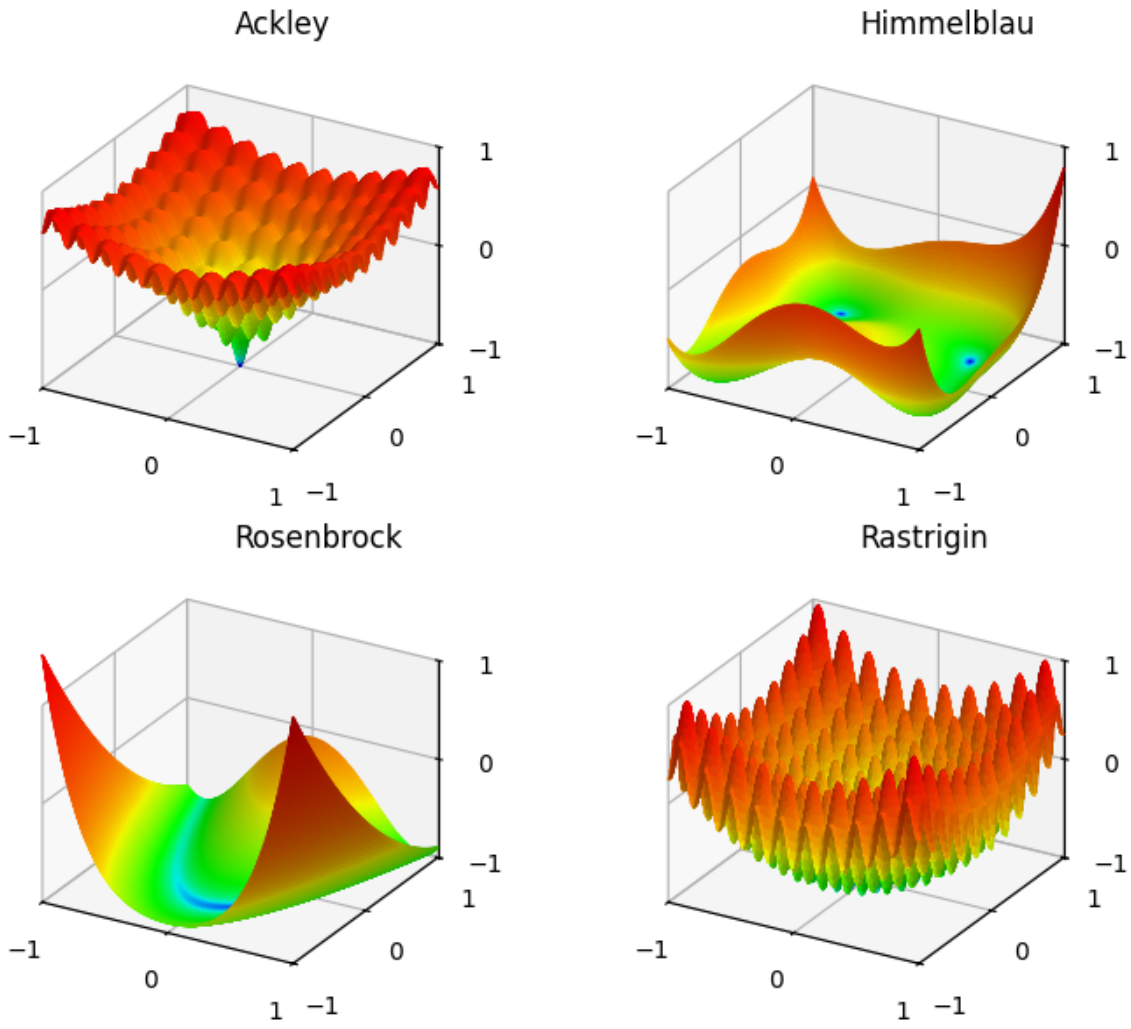
See: https://s3dlib.org/examples/functional/swirl.html



20.4.4 Function plots: Ackley, Himmelblau, Rosenbrock, Rastrigin

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_FunctionPlots.py, and the XML code in the file D04_FunctionPlots.2D.xml.

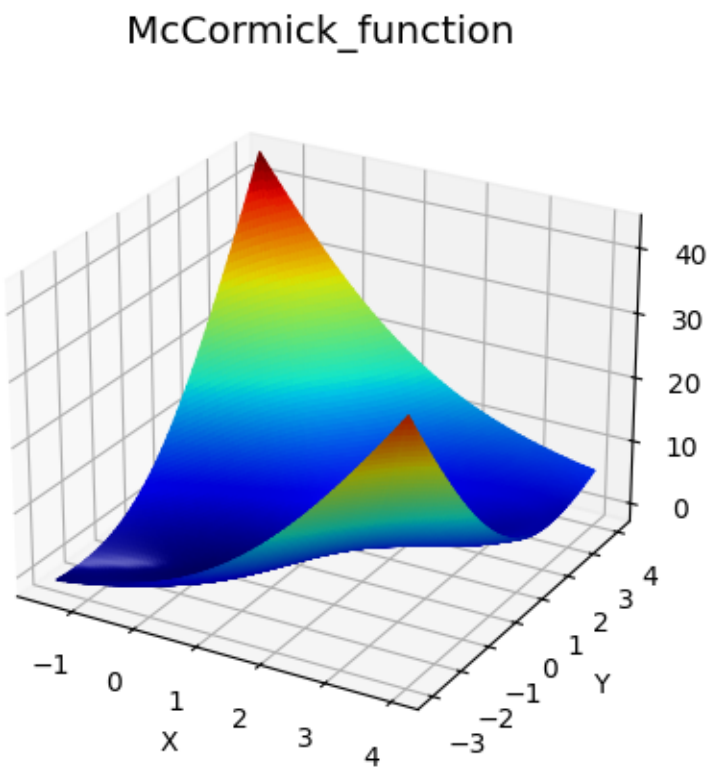
See: https://s3dlib.org/examples/functional/funcplots.html



20.4.5 McCormick function

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_FunctionPlots2.py, and the XML code in the file D05_FunctionPlots2.2D.xml.

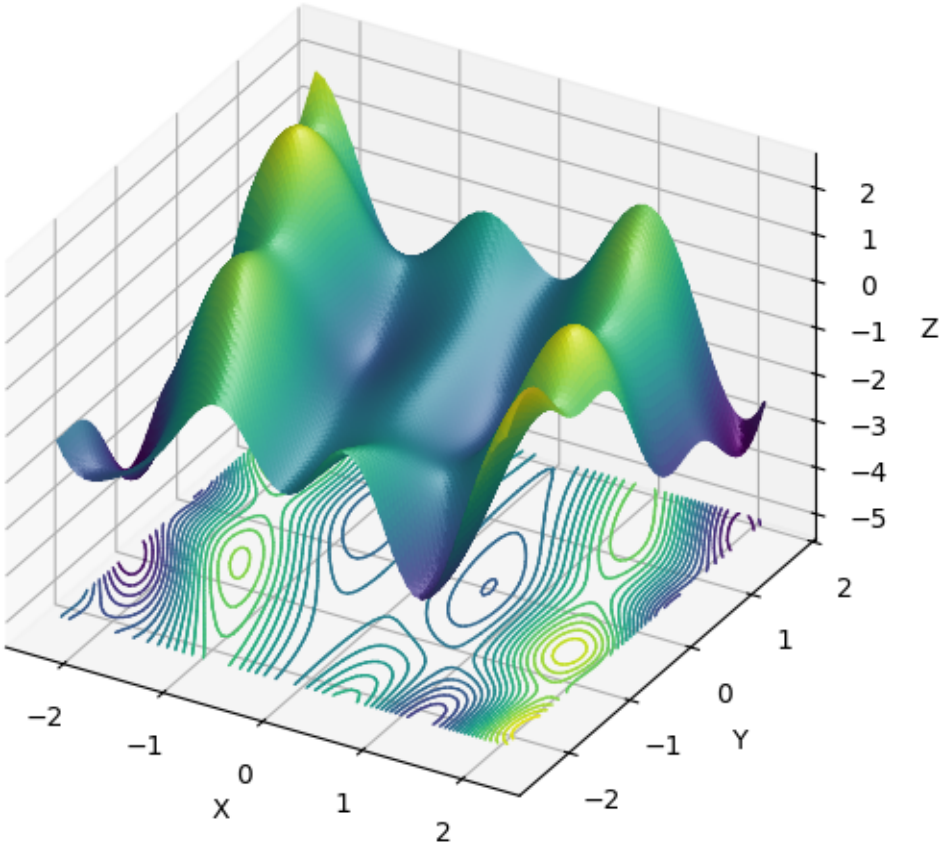
See: https://s3dlib.org/examples/functional/domain.html



20.4.6 Surface plot combined with contour

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_SurfaceContours.py, and the XML code in the file D06_SurfaceContours.2D.xml.

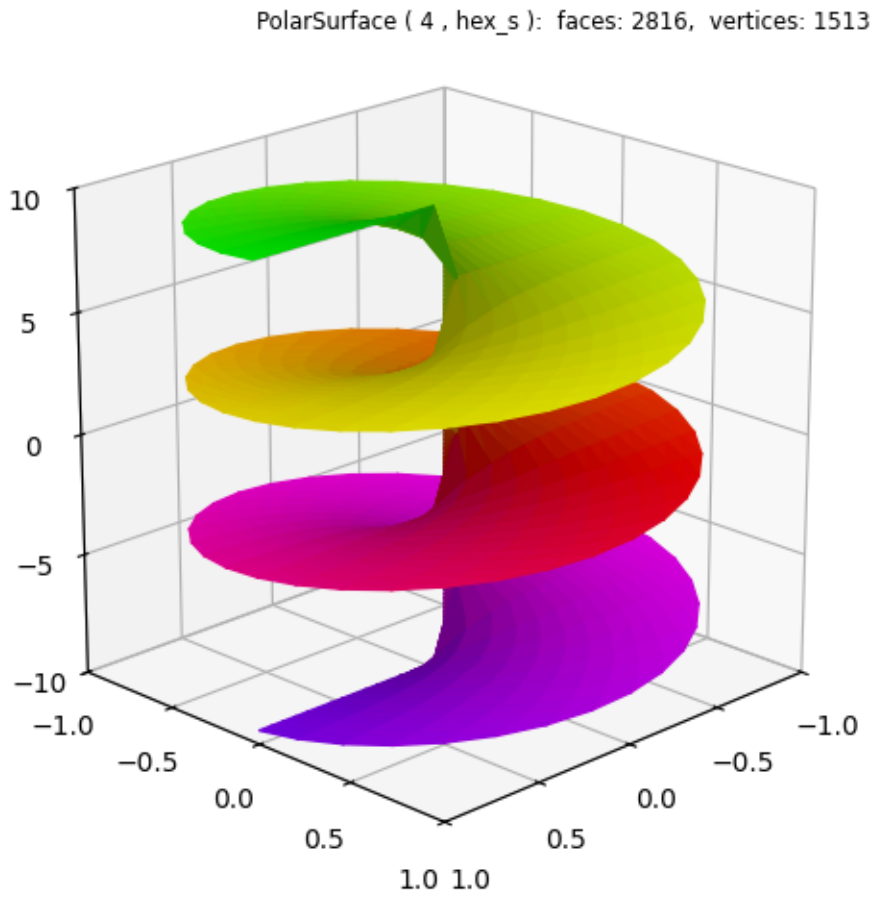
See: https://github.com/fzaverl/s3dlib



20.4.7 Sliced polar surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_SlicedPolarSurface.py, and the XML code in the file D07_SlicedPolarSurface.2D.xml.

See: https://s3dlib.org/examples/functional/screw.html



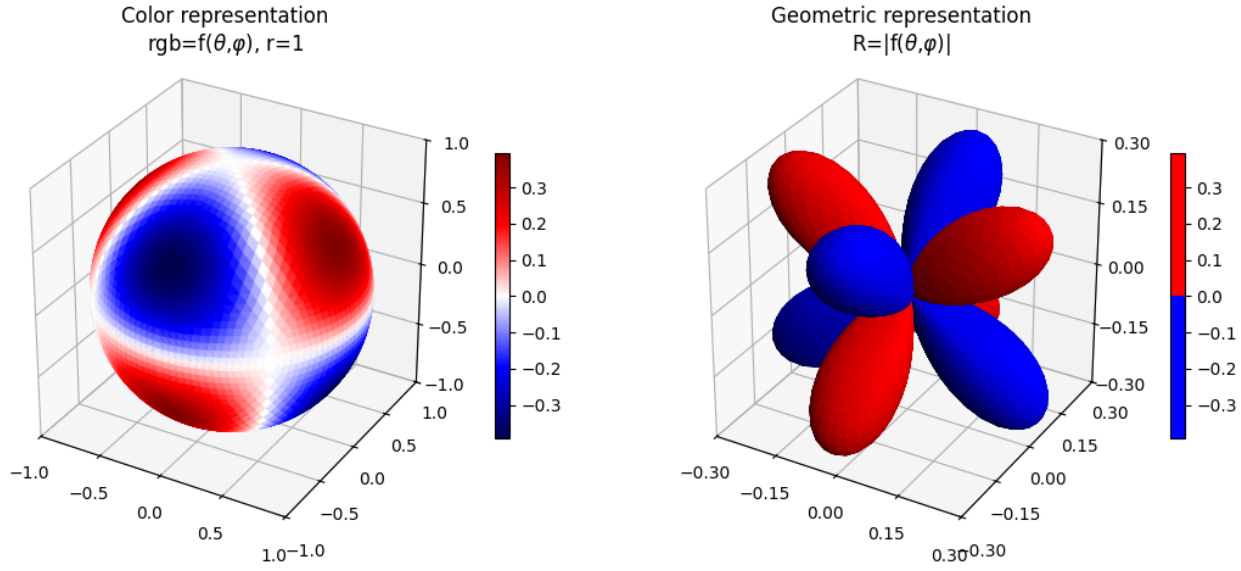
20.5 S3dlib: Parametric Surfaces

See Test2DFunctions

20.5.1 Color representation vs geometric representation, part 1

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_TwoMethodsA.py, and the XML code in the file D01_TwoMethodsA.2D.xml.

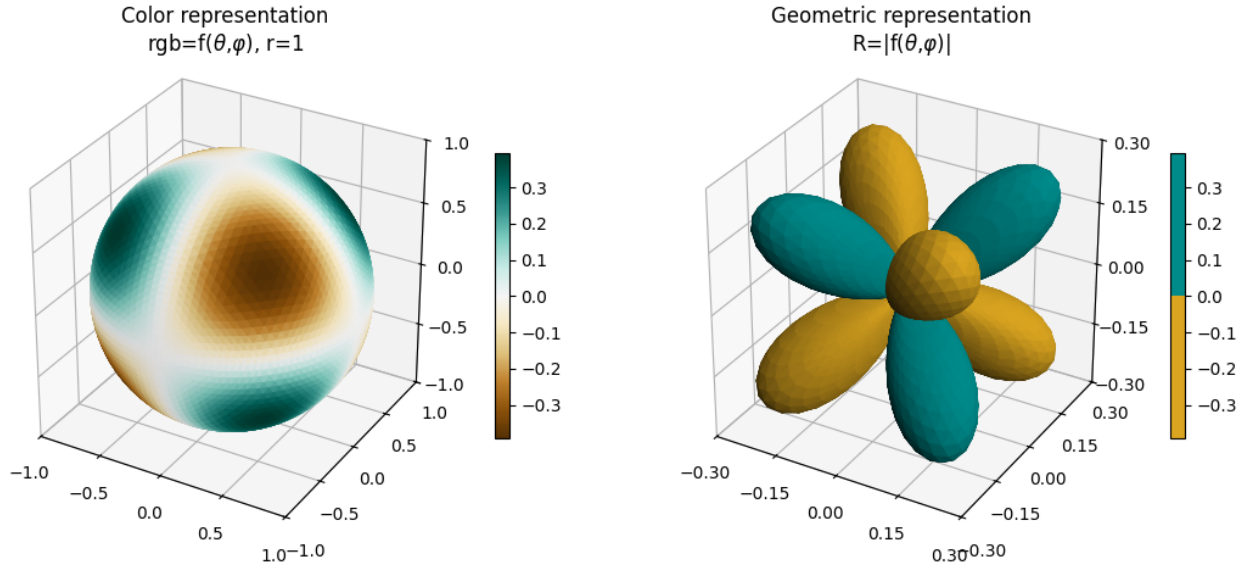
See: https://s3dlib.org/examples/functional/sph_harm.html



20.5.2 Color representation vs geometric representation, part 2

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_TwoMethodsB.py, and the XML code in the file D02_TwoMethodsB.2D.xml.

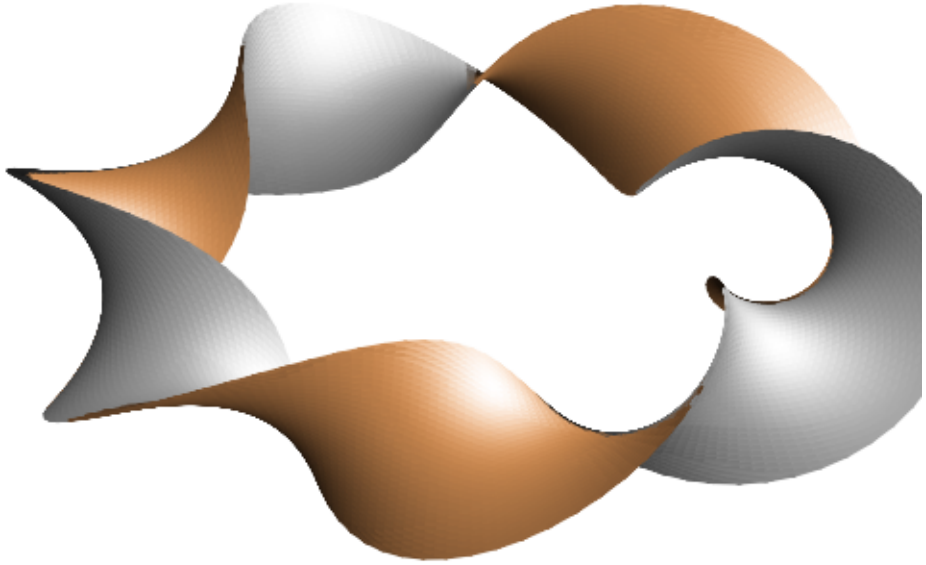
See: https://s3dlib.org/examples/functional/sph_harm.html



20.5.3 Color maps: Möbius function example

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Colormaps.py, and the XML code in the file D03_Colormaps.2D.xml.

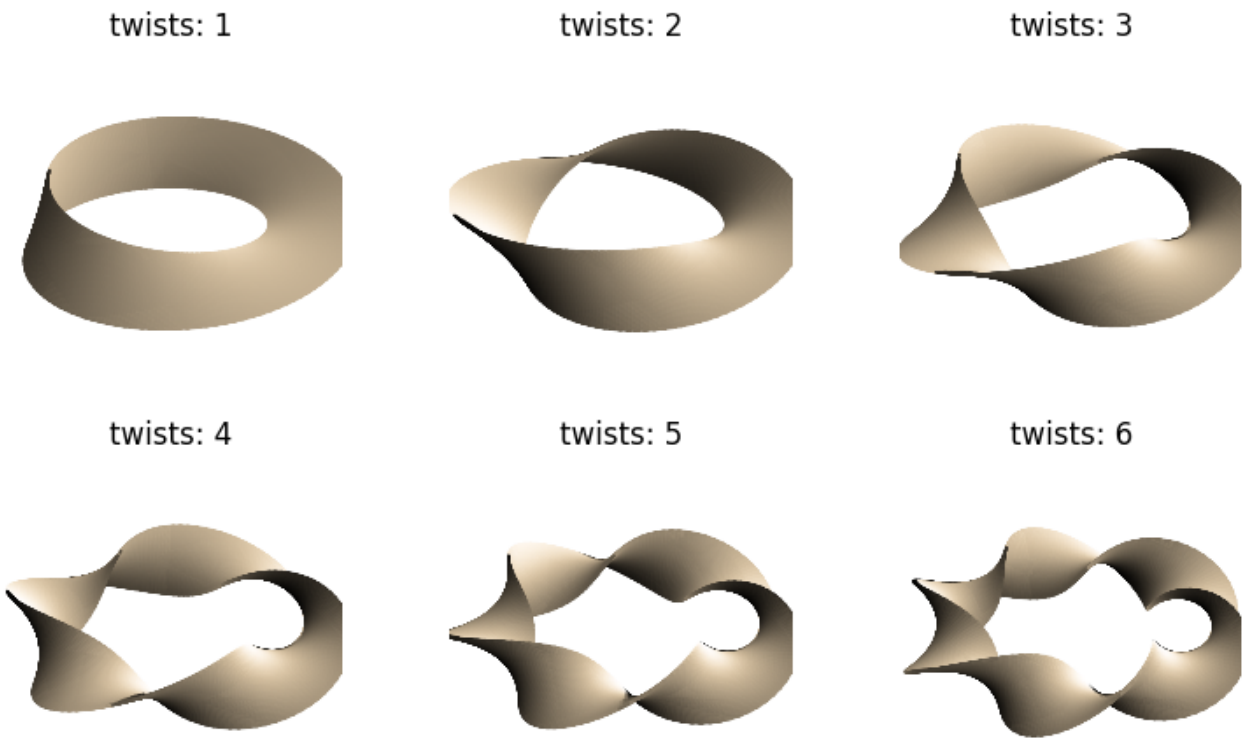
See: https://github.com/fzaverl/s3dlib



20.5.4 Shading: Möbius function example

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_TwistSet.py, and the XML code in the file D04_TwistSet.2D.xml.

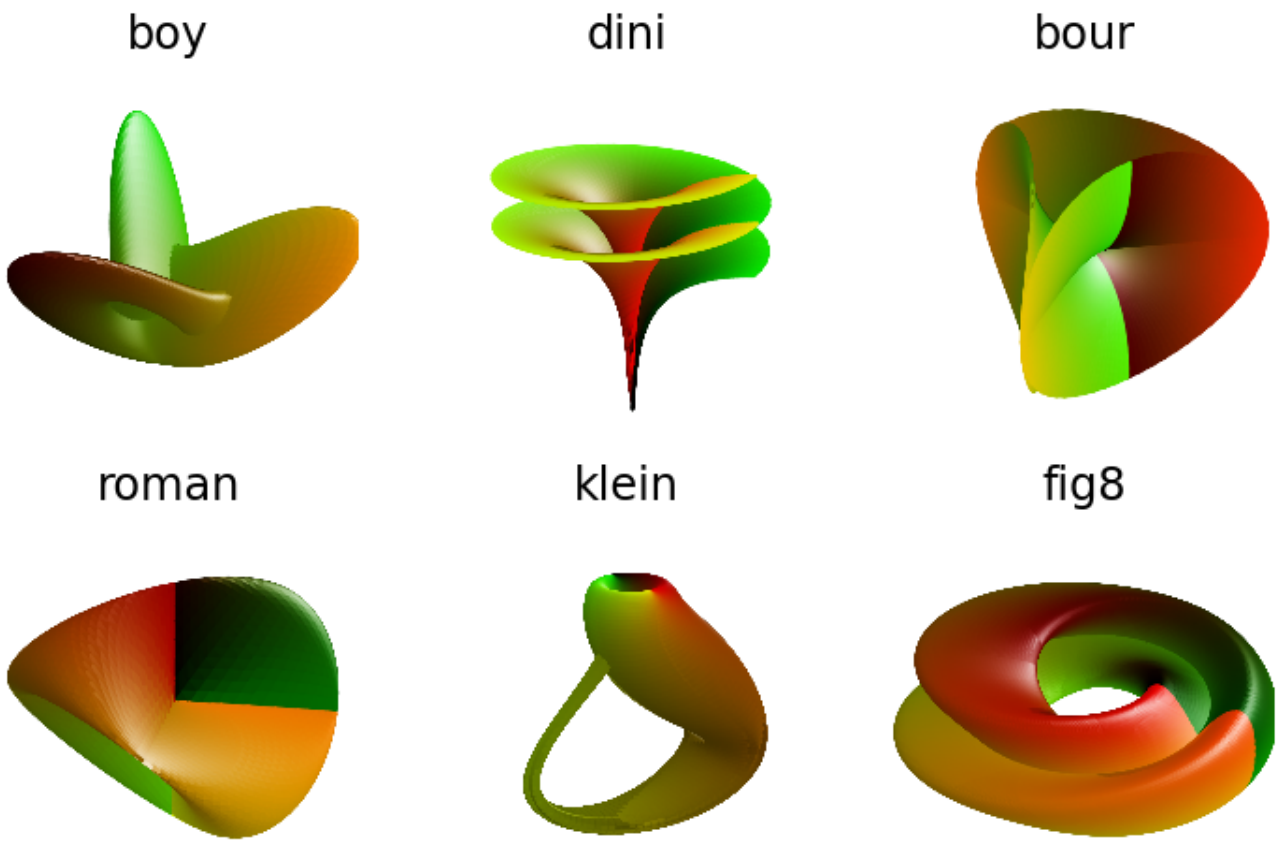
See: https://github.com/fzaverl/s3dlib



20.5.5 Color maps: Boy, Dini, Bour, Roman, Klein, and Figure8 examples

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Colormaps2.py, and the XML code in the file D05_Colormaps2.2D.xml.

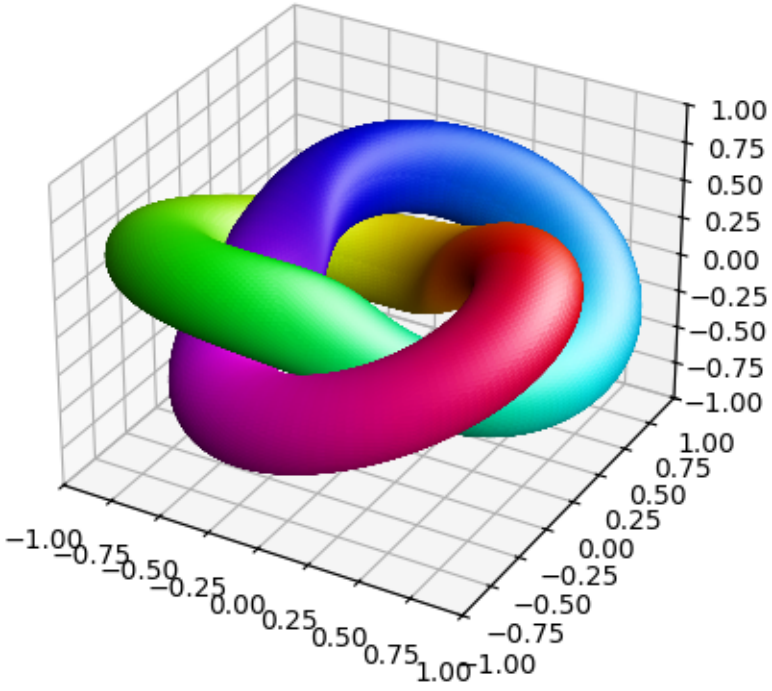
See: https://github.com/fzaverl/s3dlib



20.5.6 Colored knot in 3D

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Knot.py, and the XML code in the file D06_Knot.2D.xml.

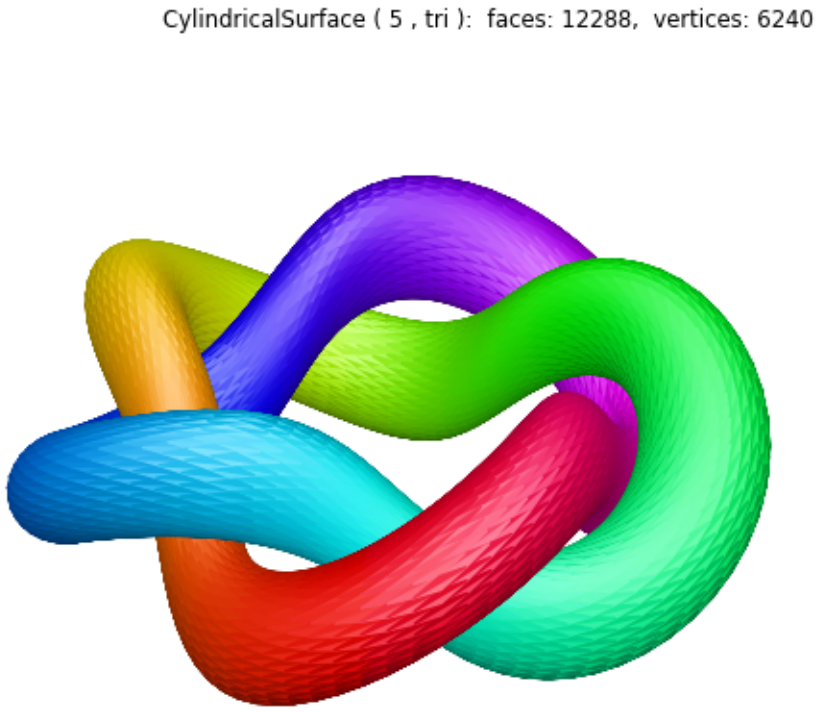
See: https://github.com/fzaverl/s3dlib



20.5.7 Multiple geometric maps

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_MultipleGeometricMaps.py, and the XML code in the file D07_MultipleGeometricMaps.2D.xml.

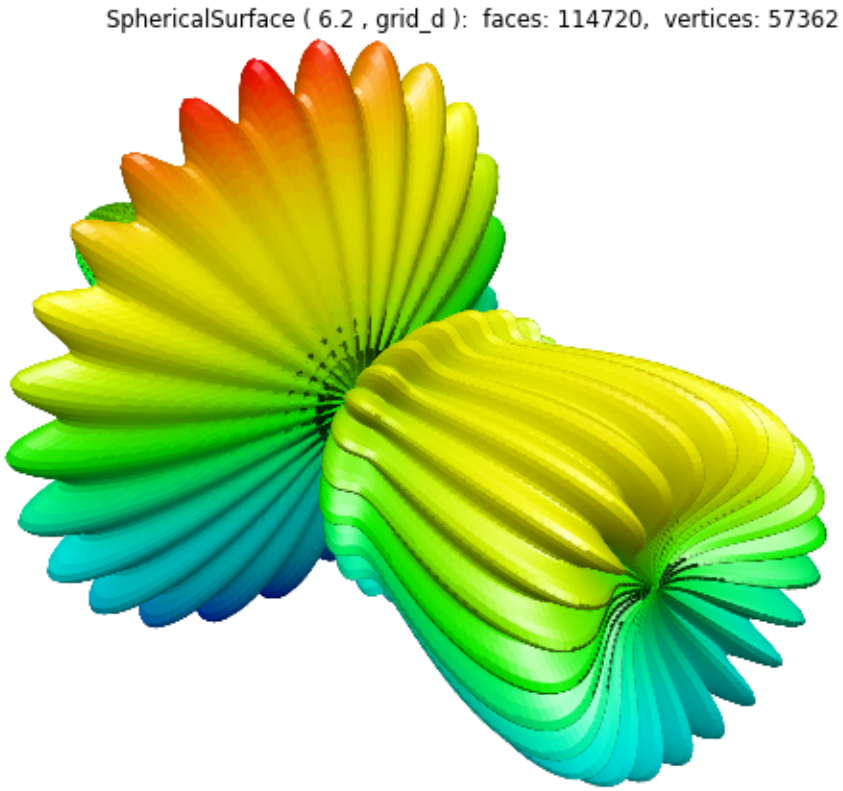
See: https://s3dlib.org/examples/functional/knot.html



20.5.8 Example from Mayavi

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_Mayavi.py, and the XML code in the file D08_Mayavi.2D.xml.

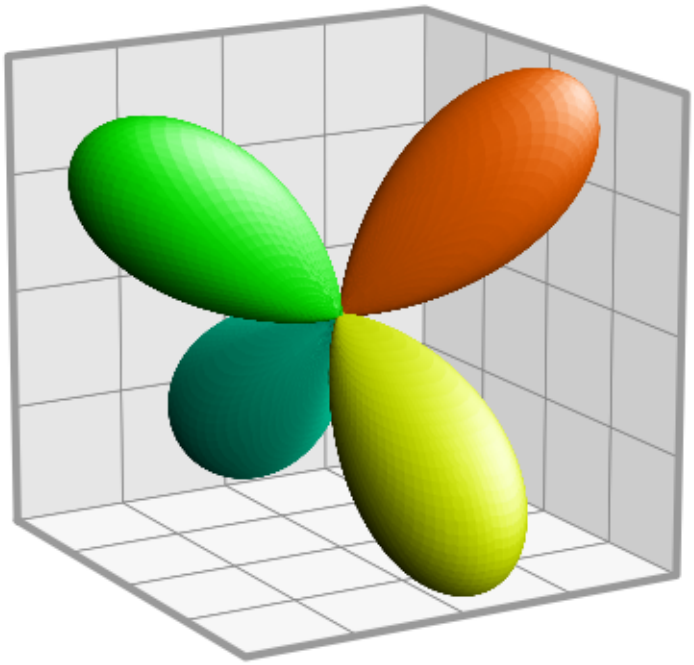
See: https://s3dlib.org/examples/functional/a_demo.html



20.5.9 S3Dlib Logo

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_S3dlibLogo.py, and the XML code in the file D09_S3dlibLogo.2D.xml.

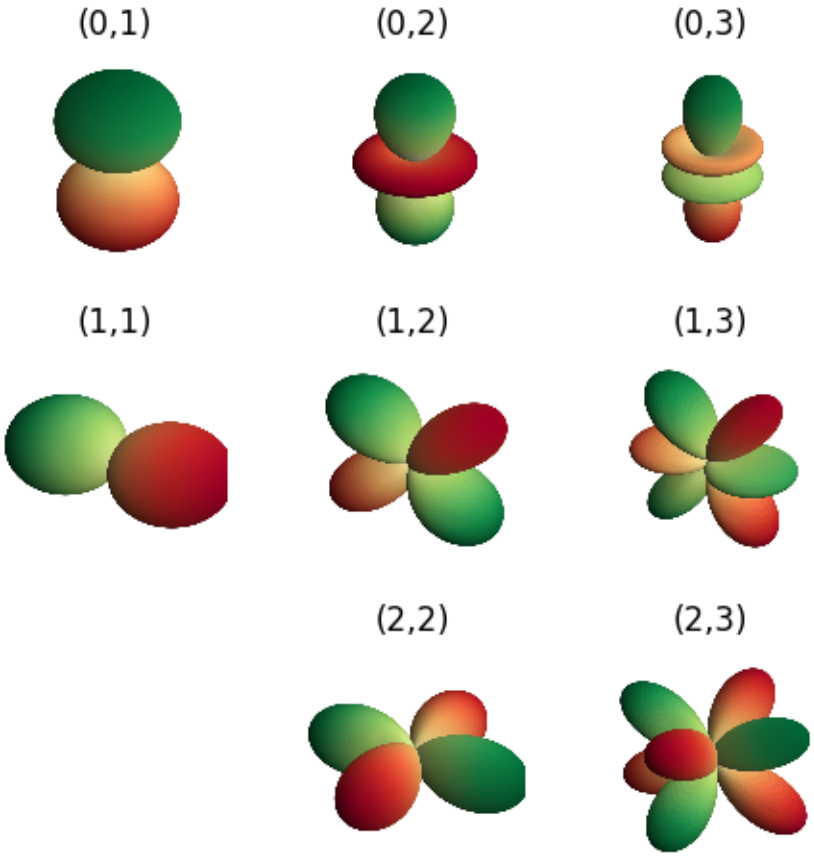
See: https://s3dlib.org/examples/functional/s3d_sph.html



20.5.10 Spherical harmonics, 3D representation

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10_SphericalHarmonics.py, and the XML code in the file D10_SphericalHarmonics.2D.xml.

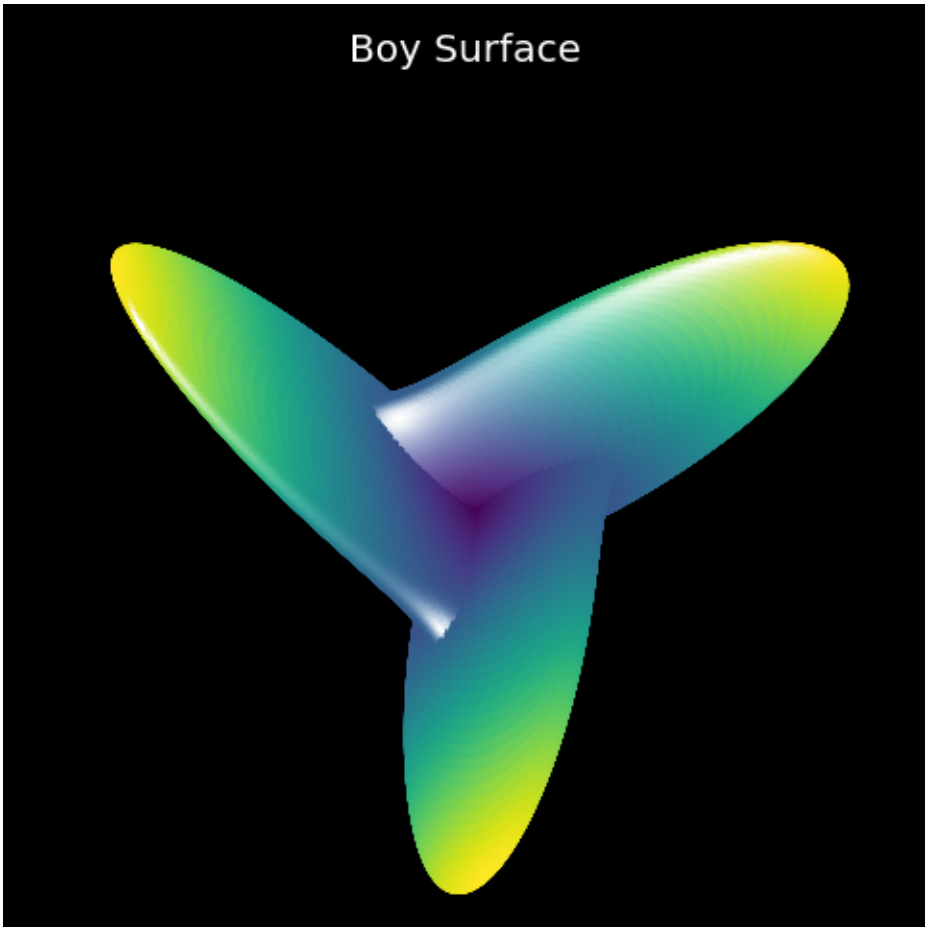
See: https://github.com/fzaverl/s3dlib



20.5.11 Boy surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11_BoySurface.py

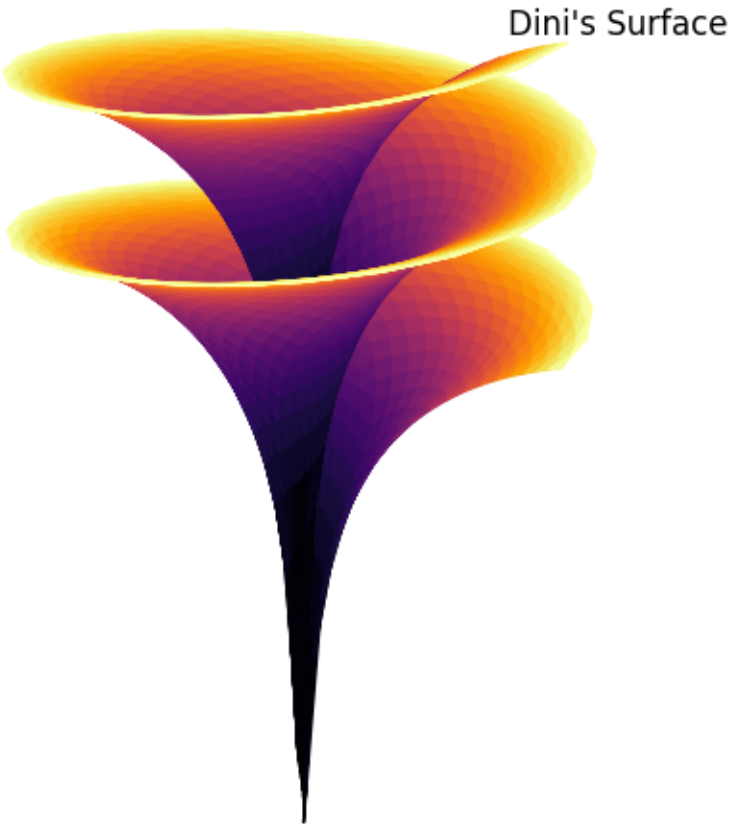
See: https://s3dlib.org/examples/functional/boy.html



20.5.12 Dini surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D12_DiniSurface.py, and the XML code in the file D12_DiniSurface.2D.xml.

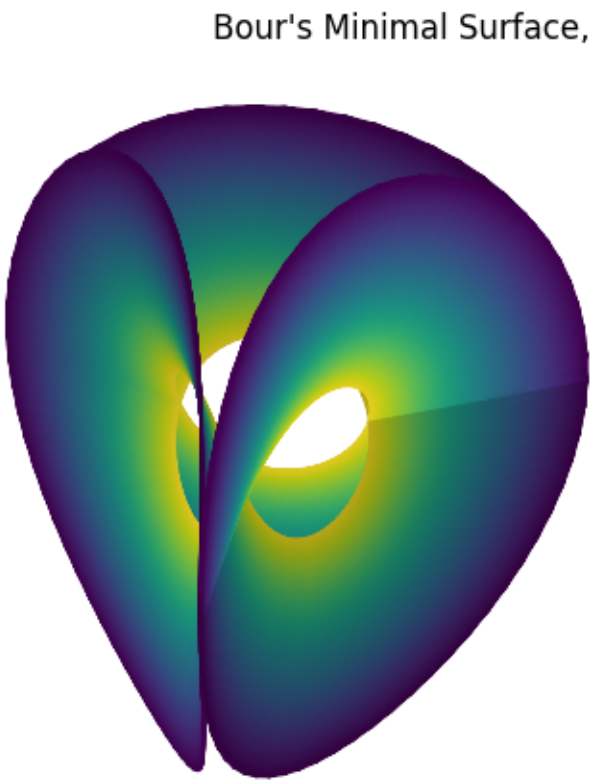
See: https://s3dlib.org/examples/functional/dini.html



20.5.13 Bour surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D13_BourSurface.py, and the XML code in the file D13_BourSurface.2D.xml.

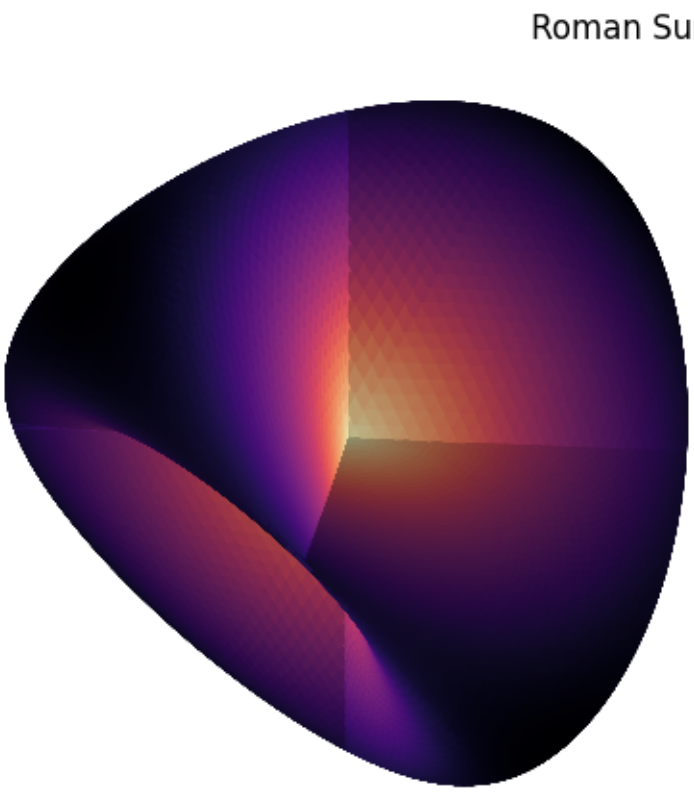
See: https://s3dlib.org/examples/functional/dini.html



20.5.14 Roman surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D14_RomanSurface.py, and the XML code in the file D14_RomanSurface.2D.xml.

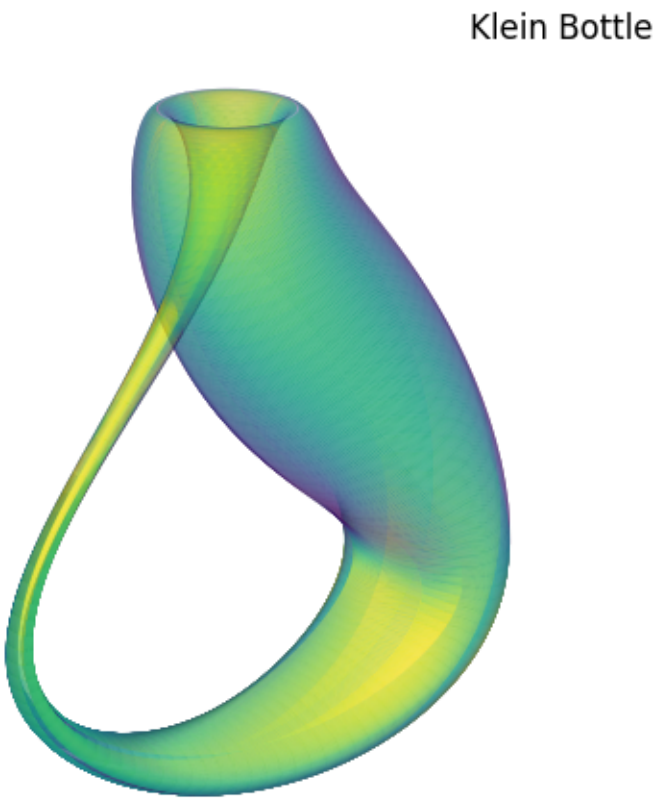
See: https://s3dlib.org/examples/functional/roman.html



20.5.15 Klein bottle

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D15_KleinBottle.py, and the XML code in the file D15_KleinBottle.2D.xml.

See: https://s3dlib.org/examples/functional/klein_bottle.html

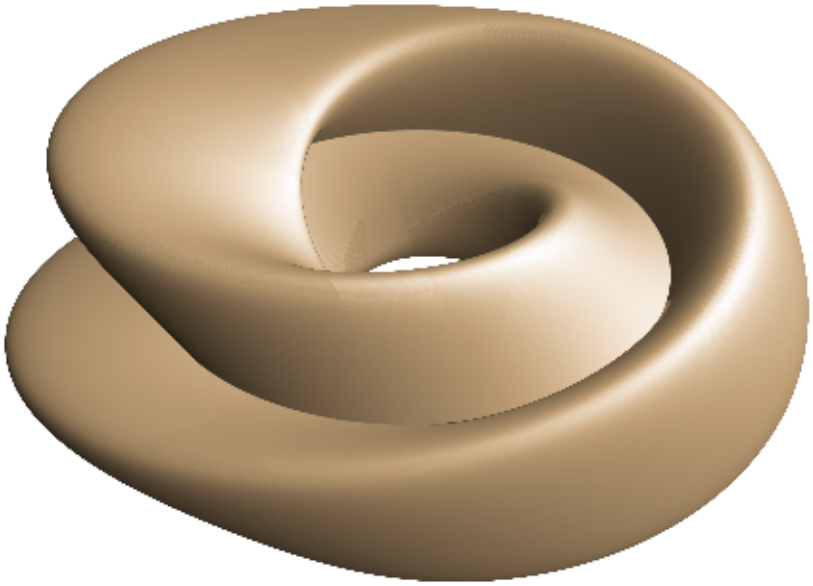


20.5.16 Klein bottle8

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D16_KleinBottle8.py, and the XML code in the file D16_KleinBottle8.2D.xml.

See: https://s3dlib.org/examples/functional/klein_figure8.html

Figure 8 Immersion of the Klein Bottle

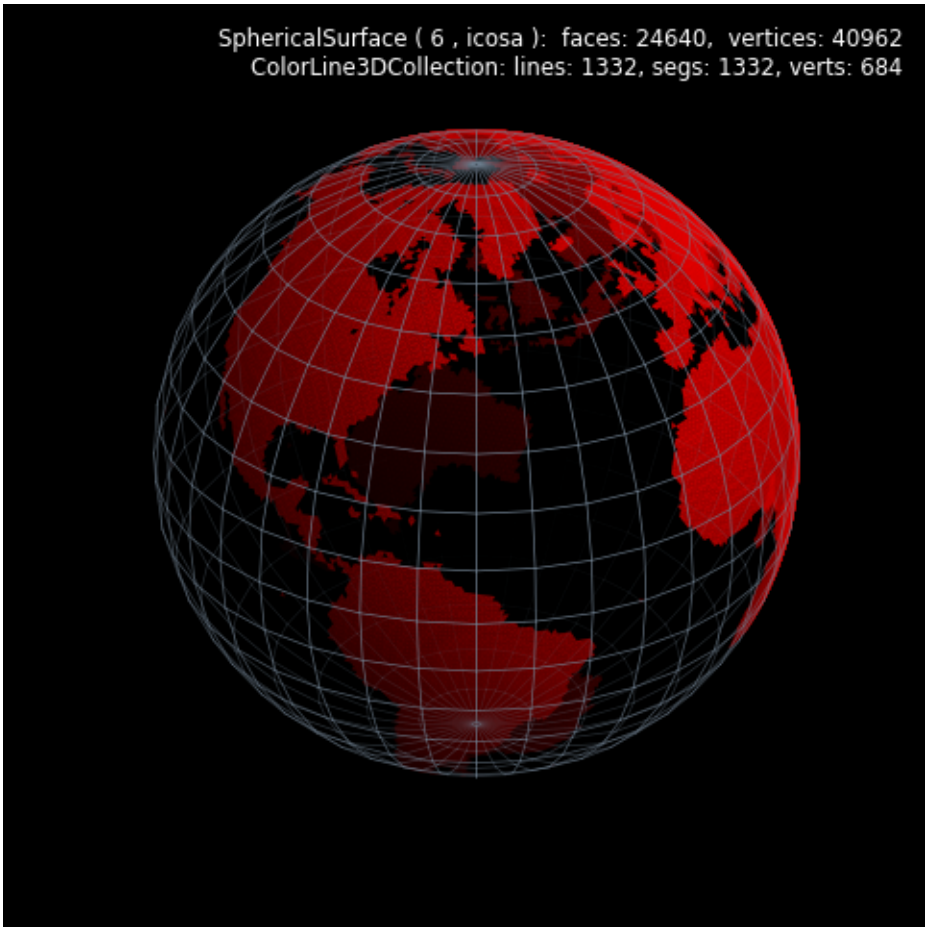


20.6 S3dlib: Image mapping and clipping

20.6.1 Image HSV value clipping

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Image_HSV_Value_Clipping.py, and the XML code in the file D01_Image_HSV_Value_Clipping.2D.xml.

See: https://s3dlib.org/examples/imagemap/earth_land_clipped.html



20.6.2 Spherical image mapping

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Spherical_ImageMapping.py, and the XML code in the file D02_Spherical_ImageMapping.2D.xml.

See: https://s3dlib.org/examples/imagemap/earth_shaded.html

20.6.3 Earth in 3D

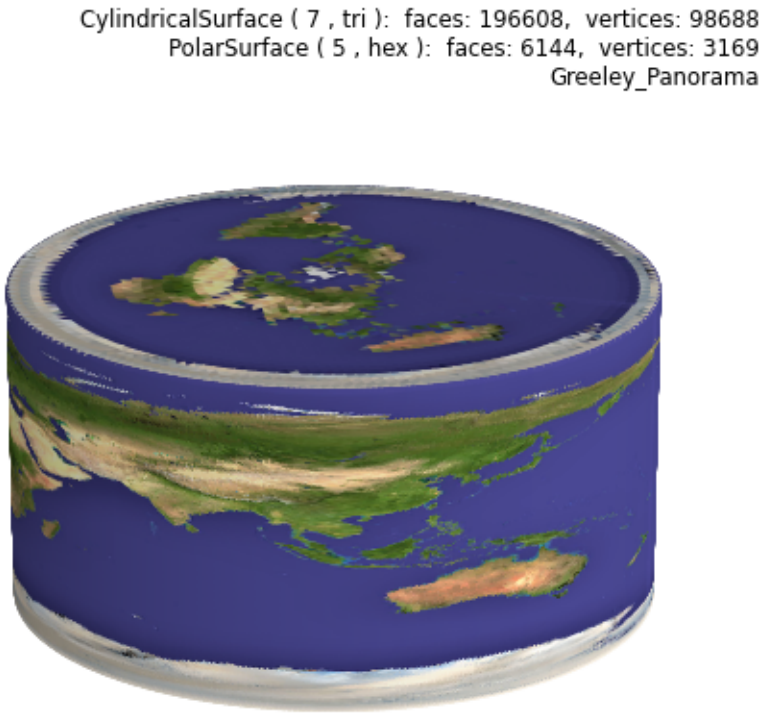
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Earth3D.py, and the XML code in the file D03_Earth3D.2D.xml.

See: https://github.com/fzaverl/s3dlib

20.6.4 Cylindrical image mapping

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Cylindrical_ImageMapping.py, and the XML code in the file D04_Cylindrical_ImageMapping.2D.xml.

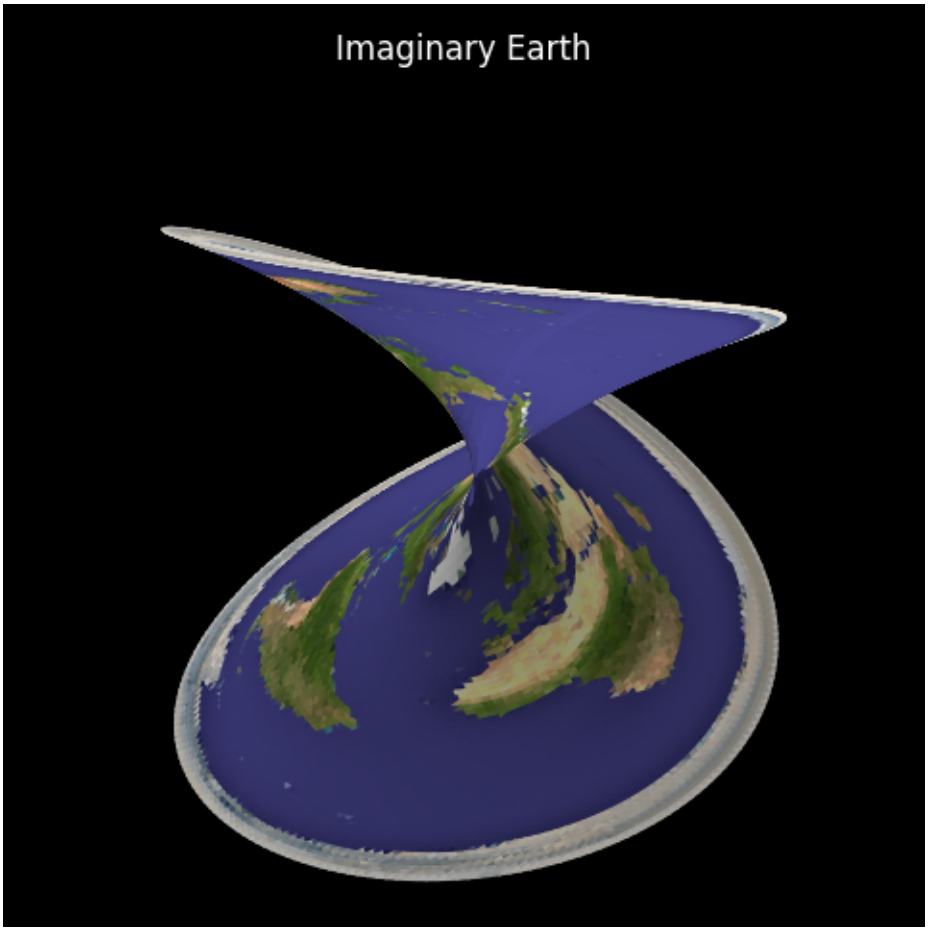
See: https://s3dlib.org/examples/imagemap/mars_cyl.html



20.6.5 Imaginary Earth, part A

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Imaginary_Earth_a.py, and the XML code in the file D05_Imaginary_Earth_a.2D.xml.

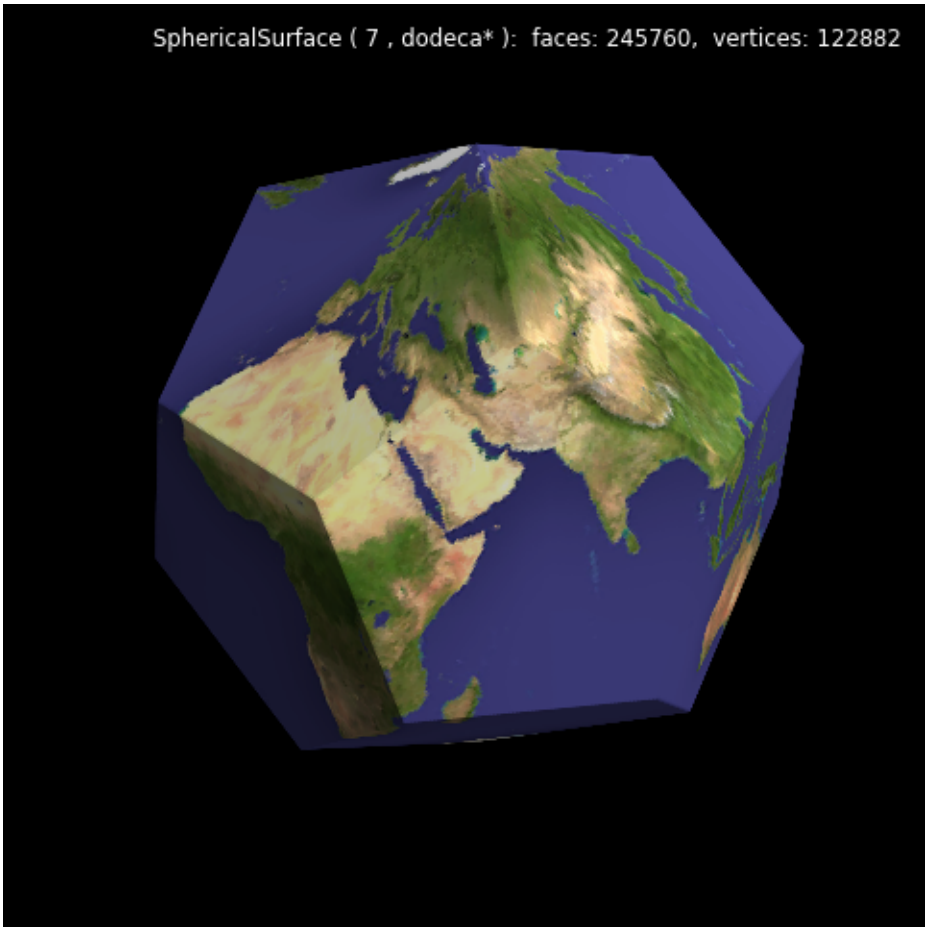
See: https://s3dlib.org/examples/imagemap/imaginary_earth.html



20.6.6 Imaginary Earth, part B

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Imaginary_Earth_b.py, and the XML code in the file D06_Imaginary_Earth_b.2D.xml.

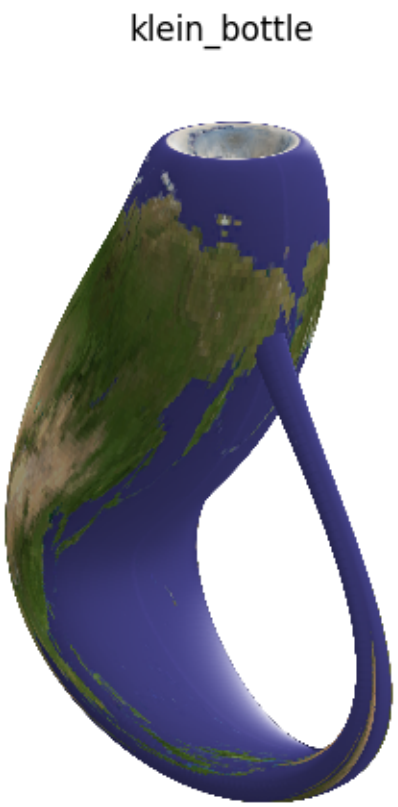
See: https://s3dlib.org/examples/imagemap/imaginary_earth.html



20.6.7 Earth, projected on Klein bottle

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_Klein_Earth_a.py, and the XML code in the file D07_Klein_Earth_a.2D.xml.

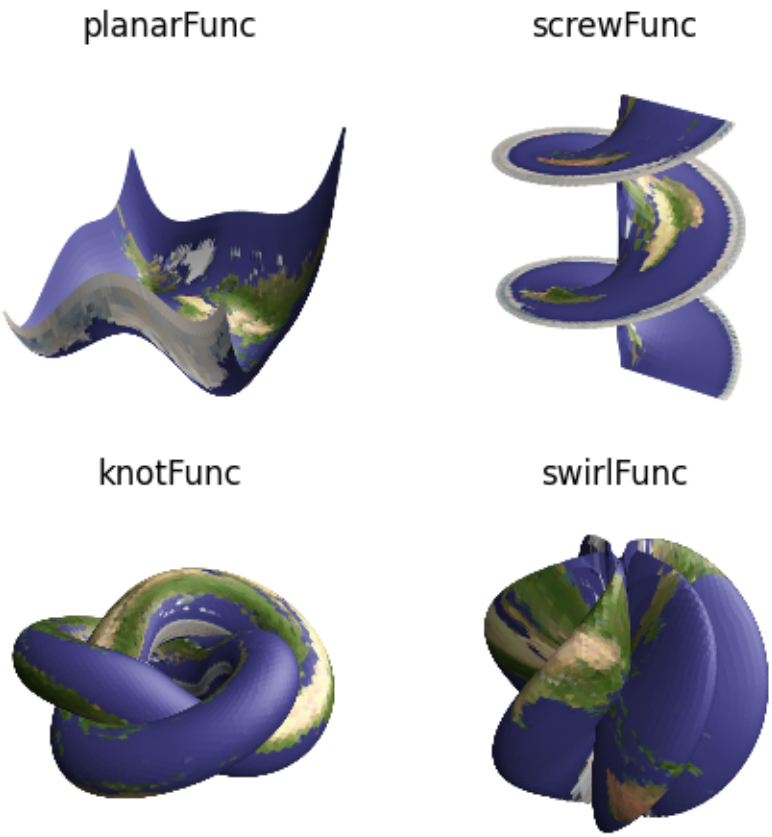
See: https://s3dlib.org/examples/imagemap/earth_klein.html



20.6.8 Earth, projected on planar, screw, knot and swirl surfaces

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_Klein_Earth_b.py, and the XML code in the file D08_Klein_Earth_b.2D.xml.

See: https://s3dlib.org/examples/imagemap/earth_klein.html

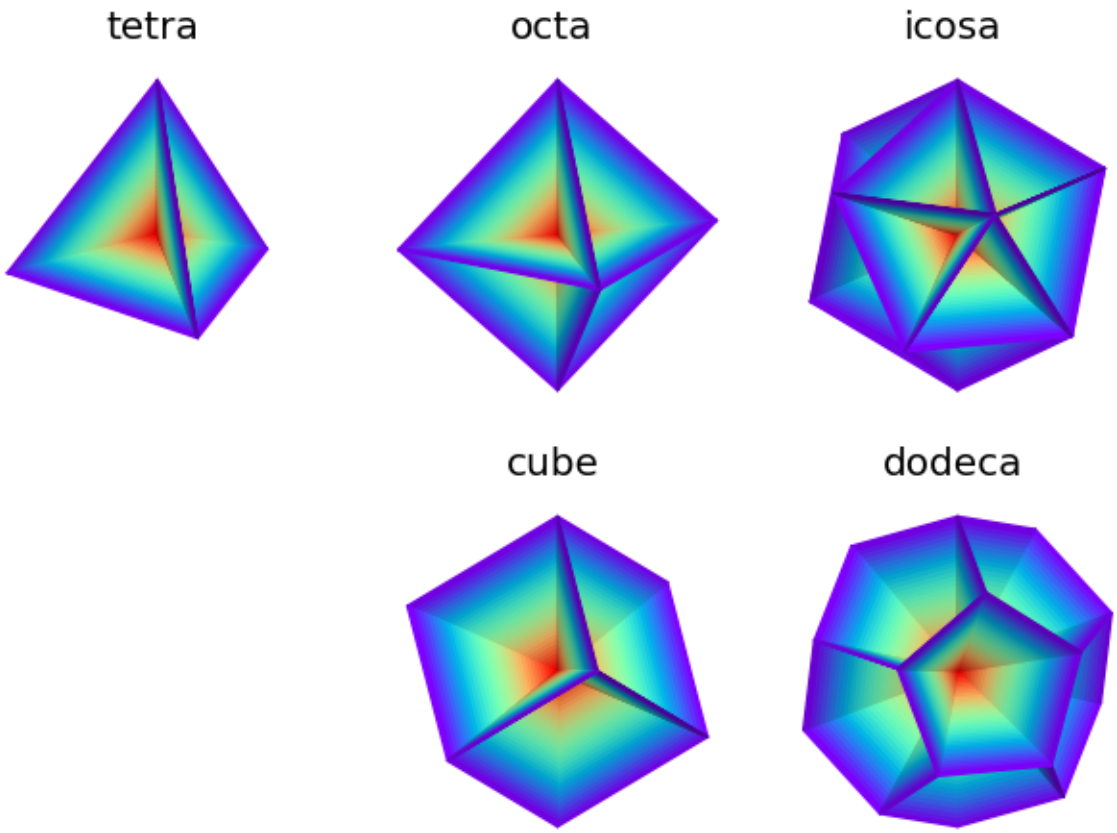


20.7 S3dlib: Edge-to-edge surface coloring

20.7.1 Platonic solid, with surface edges filled, part 1

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Platonic_Solid_Surface_Edges_Filled.py, and the XML code in the file D01_Platonic_Solid_Surface_Edges_Filled.2D.xml.

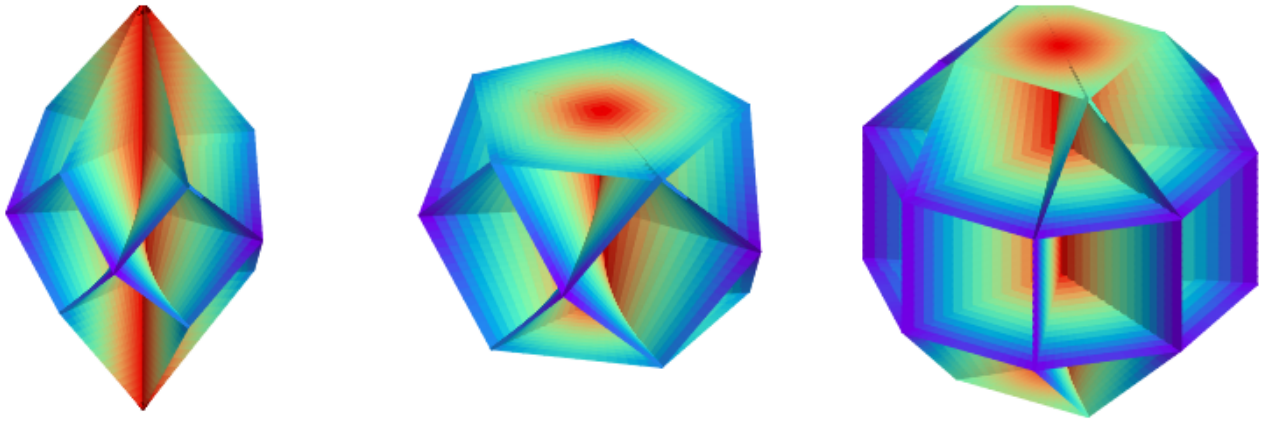
See: https://s3dlib.org/examples/lines/platonic_proj.html



20.7.2 Platonic solid, with surface edges filled, part 2

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Platonic_Solid_Surface_Edges_Filled.py, and the XML code in the file D02_Platonic_Solid_Surface_Edges_Filled.2D.xml.

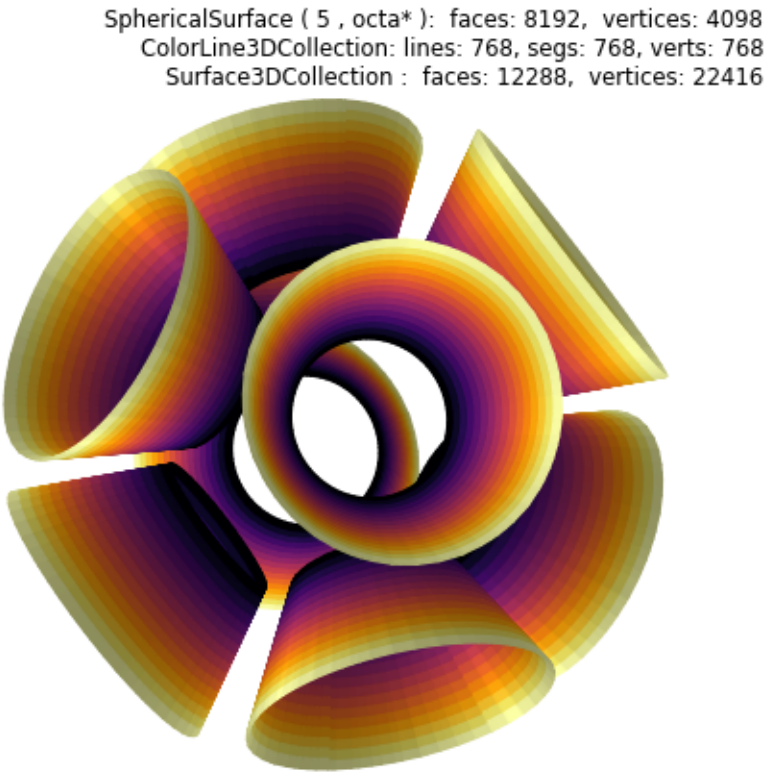
See: https://s3dlib.org/examples/lines/platonic_proj.html



20.7.3 Spherical contour surface, filled

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Spherical_Contour_Filled_Surface.py, and the XML code in the file D03_Spherical_Contour_Filled_Surface.2D.xml.

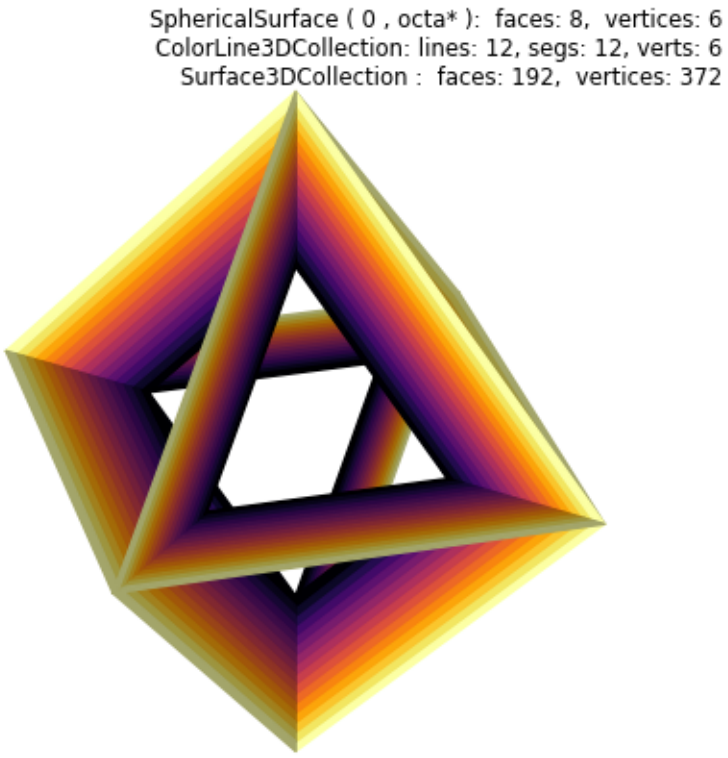
See: https://s3dlib.org/examples/lines/filled_octa_cont.html



20.7.4 Edge-to-edge surface, part 1

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Edge_to_Edge_Surface.py, and the XML code in the file D04_Edge_to_Edge_Surface.2D.xml.

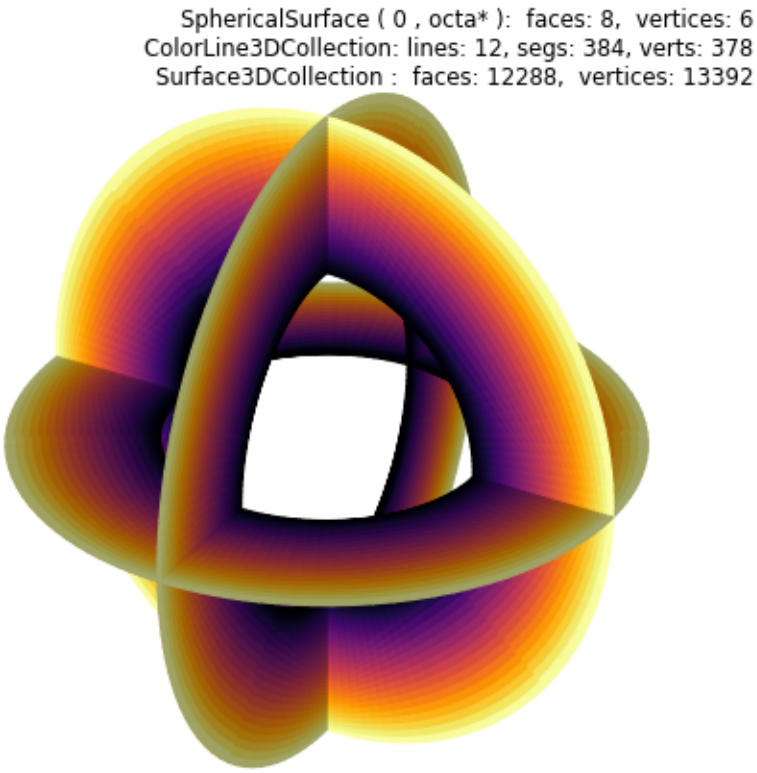
See: https://s3dlib.org/examples/lines/surf_edge2edge.html



20.7.5 Spherical edge surface, filled

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Spherical_Edge_Filled_Surface_2.py, and the XML code in the file D06_Spherical_Edge_Filled_Surface_2.2D.xml.

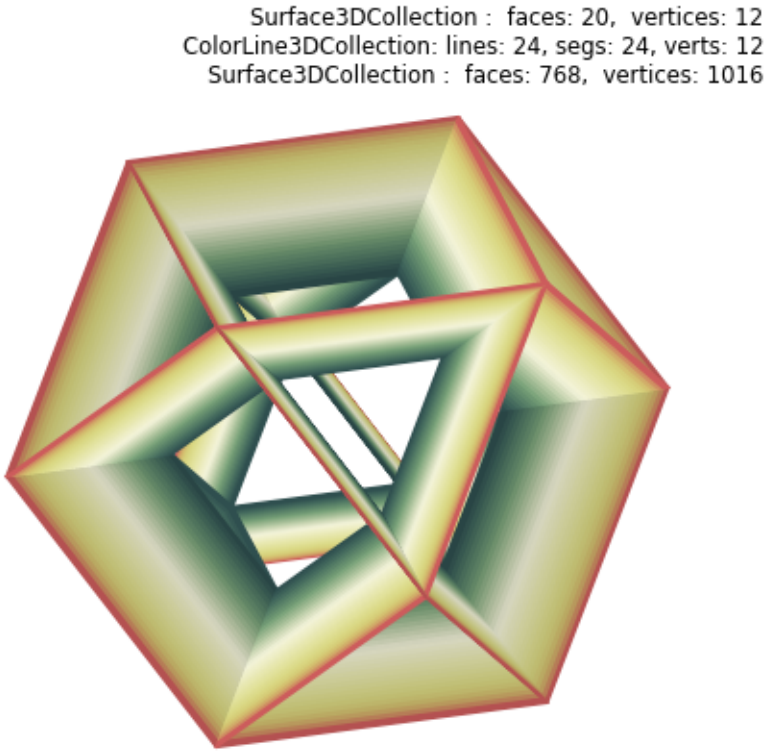
See: https://s3dlib.org/examples/lines/filled_octa_edges.html



20.7.6 Edge-to-edge surface, part 2

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Spherical_Edge_Filled_Surface_2.py, and the XML code in the file D06_Spherical_Edge_Filled_Surface_2.2D.xml.

See: https://s3dlib.org/examples/lines/surf_edge2edge_2.html



20.7.7 Line colormap sequence

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_Line_Colormap_Sequence.py, and the XML code in the file D07_Line_Colormap_Sequence.2D.xml.

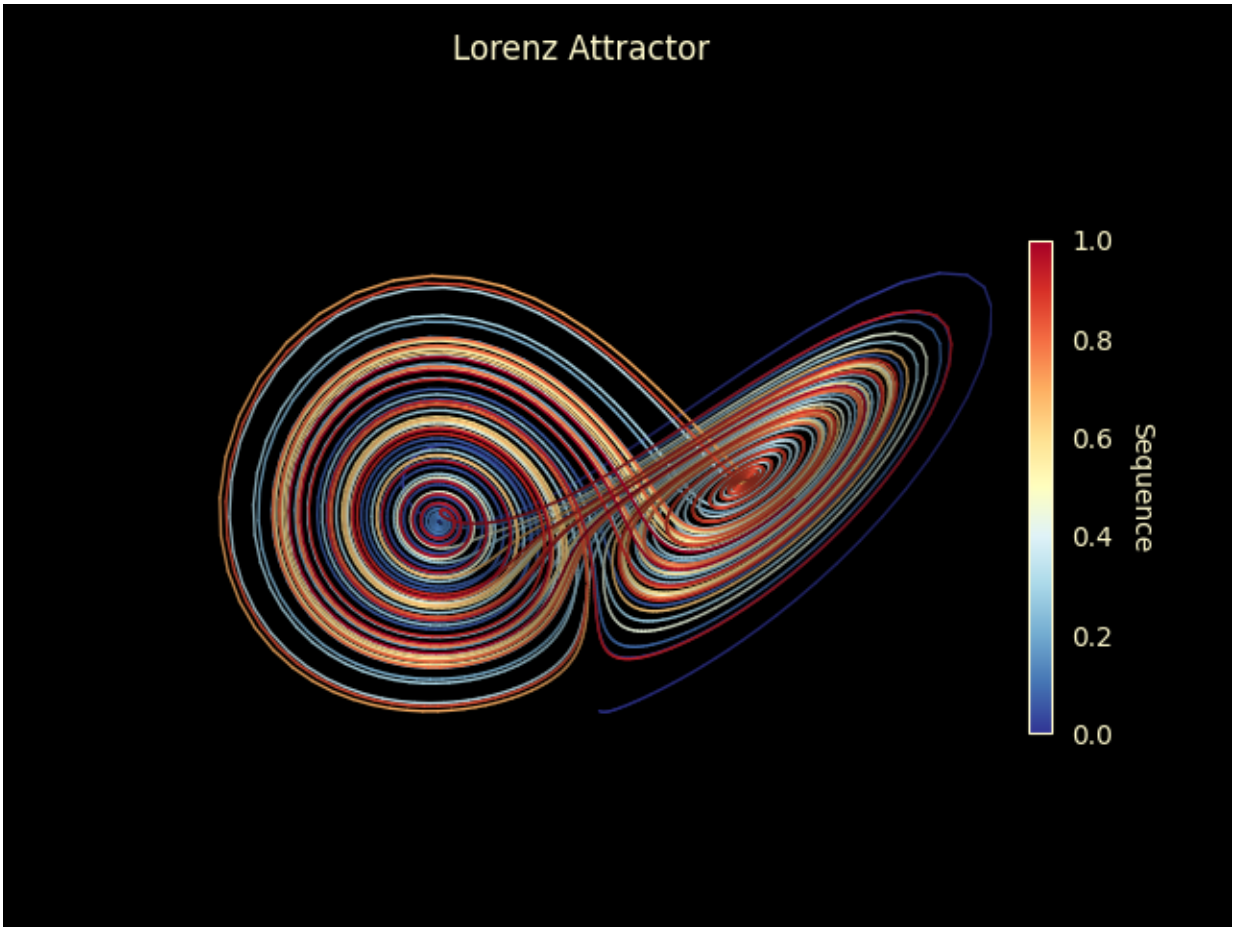
See: https://s3dlib.org/examples/lines/loop3d.html

ParametricLine (9):: lines: 1, segs: 1024, verts: 1025
sop() sequential operation

20.7.8 Sequential line color, part 1

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_Sequential_Line_Color.py, and the XML code in the file D08_Sequential_Line_Color.2D.xml.

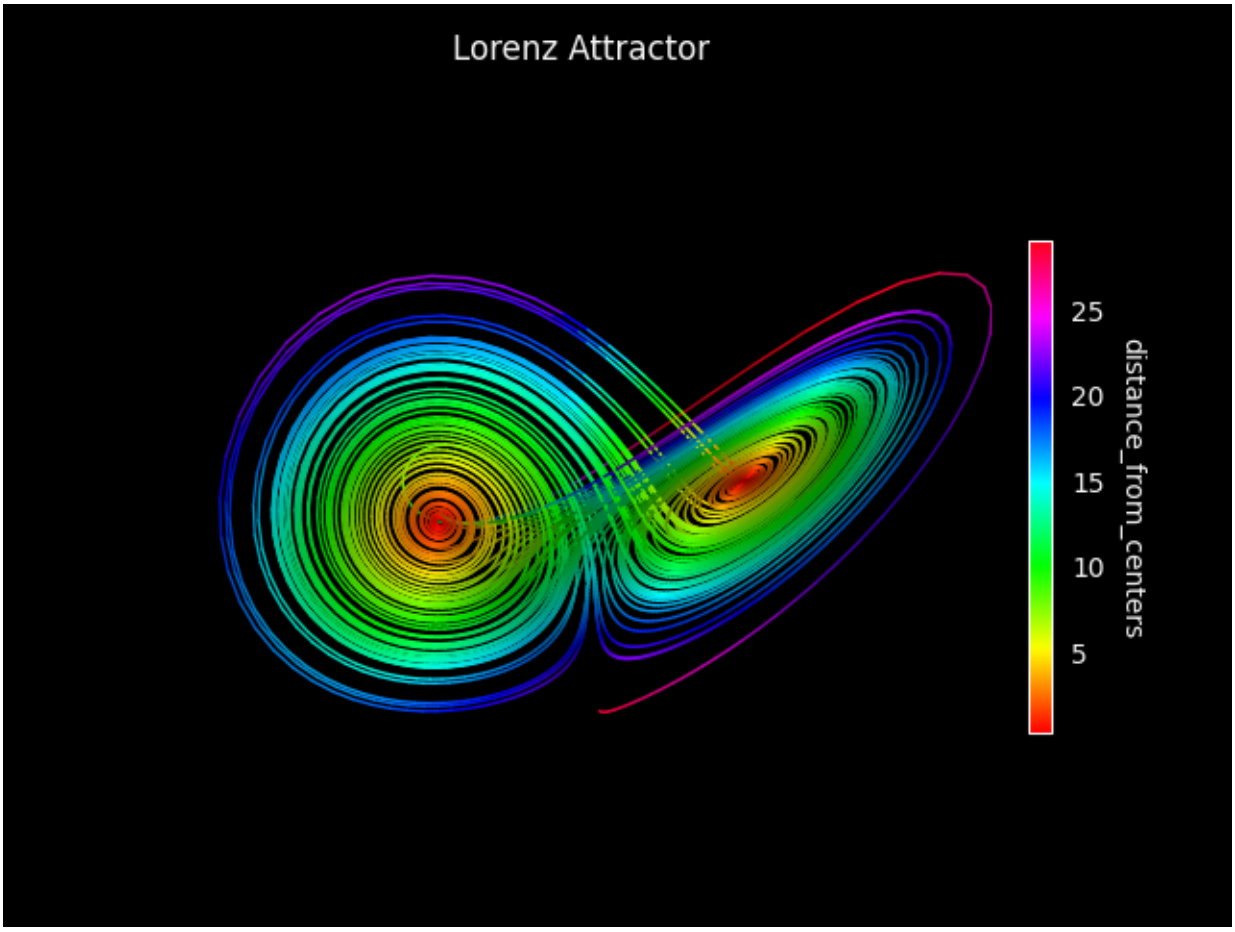
See: https://s3dlib.org/examples/lines/lorenz_seq.html



20.7.9 Sequential line color, part 2

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09_Sequential_Line_Color2.py, and the XML code in the file D09_Sequential_Line_Color2.2D.xml.

See: https://s3dlib.org/examples/lines/lorenz_op.html

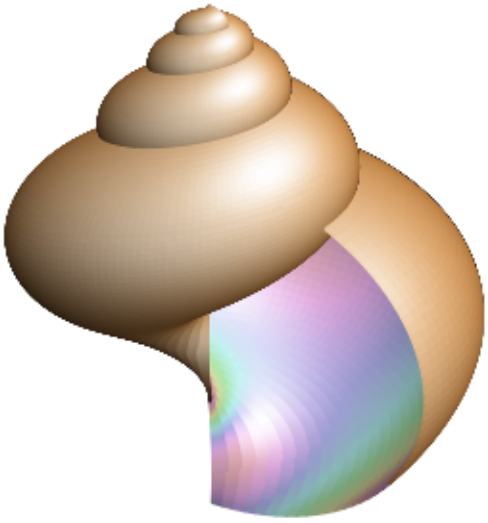


20.8 S3dlib: Decorative parametric surfaces

20.8.1 Shell surface with colormap

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Surface_Colormap_2.py, and the XML code in the file D01_Surface_Colormap_2.2D.xml.

See: https://s3dlib.org/examples/colormaps/shell.html



```
SphericalSurface ( 5.3 , grid_x ):  faces: 32400,  vertices: 33001
```

20.8.2 Escher Knot

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Escher_Knot_2.py, and the XML code in the file D02_Escher_Knot_2.2D.xml.

See: https://s3dlib.org/examples/showcase/mceknot_2.html

20.8.3 Escher sphere spirals

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Escher_Sphere_Spirals.py, and the XML code in the file D03_Escher_Sphere_Spirals.2D.xml.

See: https://s3dlib.org/examples/showcase/sphericalspirals.html

20.8.4 Escher spirals

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Escher_Spirals.py, and the XML code in the file D04_Escher_Spirals.2D.xml.

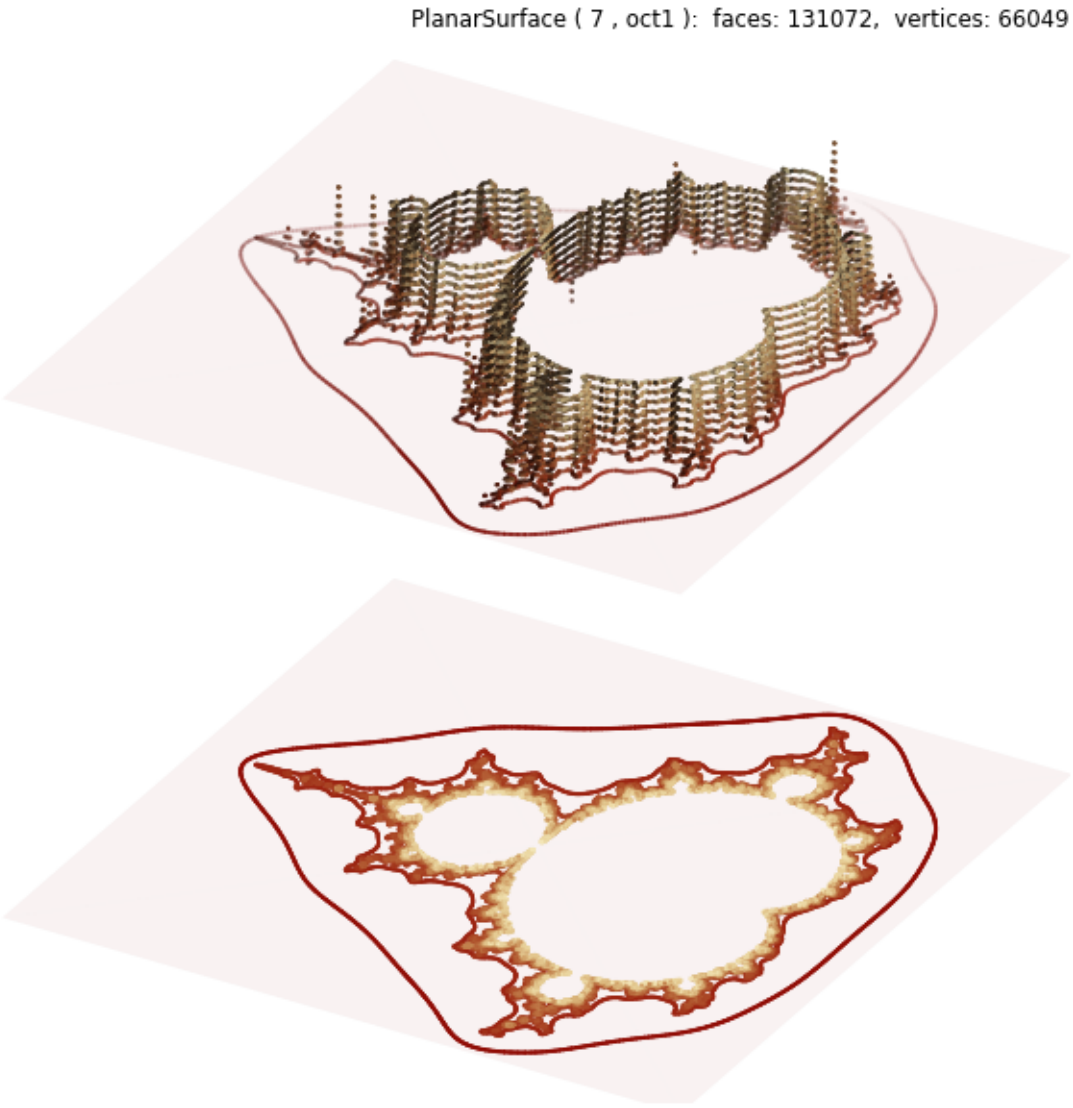
See: https://s3dlib.org/examples/showcase/spirals.html

20.9 S3dlib: Geometric and color datagrid mapping (requires scipy)

20.9.1 Mandelbrot contours

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Mandelbrot_Contours.py, and the XML code in the file D01_Mandelbrot_Contours.2D.xml.

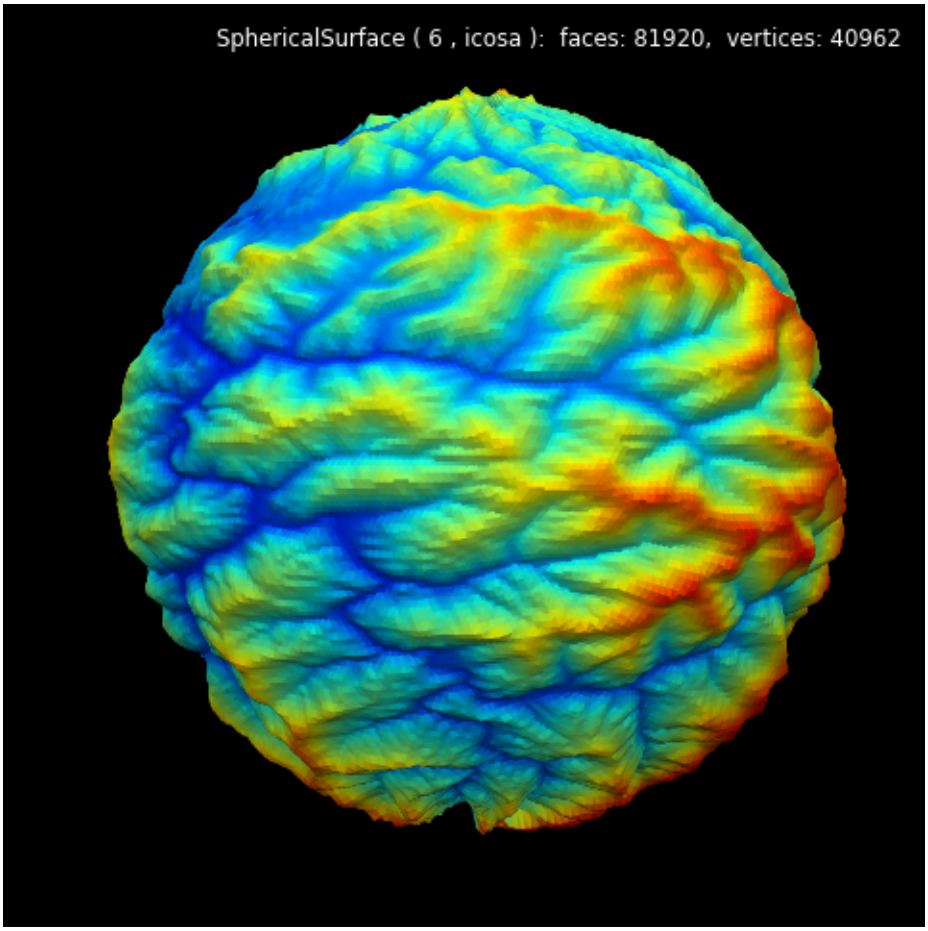
See: https://s3dlib.org/examples/lines/mand_contours.html



20.9.2 Geometric and color datagrid mapping

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_GeometricMapping.py, and the XML code in the file D02_GeometricMapping.2D.xml.

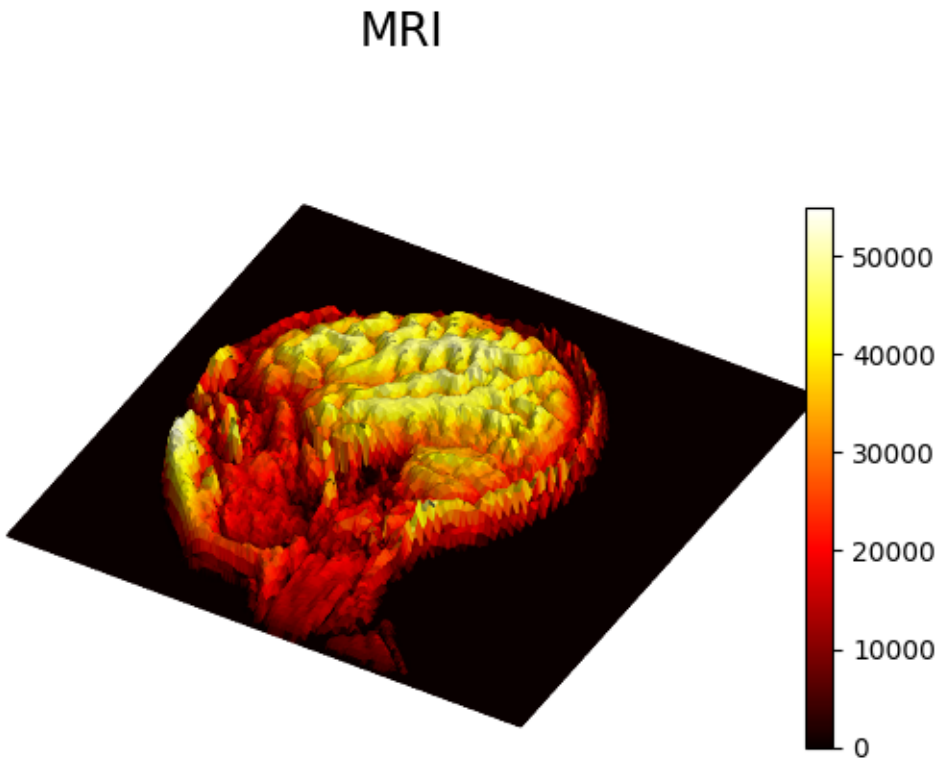
See: https://s3dlib.org/examples/datagridmap/jacks_shaded.html



20.9.3 MRI 3D

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_MRI_3D.py, and the XML code in the file D03_MRI_3D.2D.xml.

See: https://s3dlib.org/examples/datagridmap/mri_3d.html

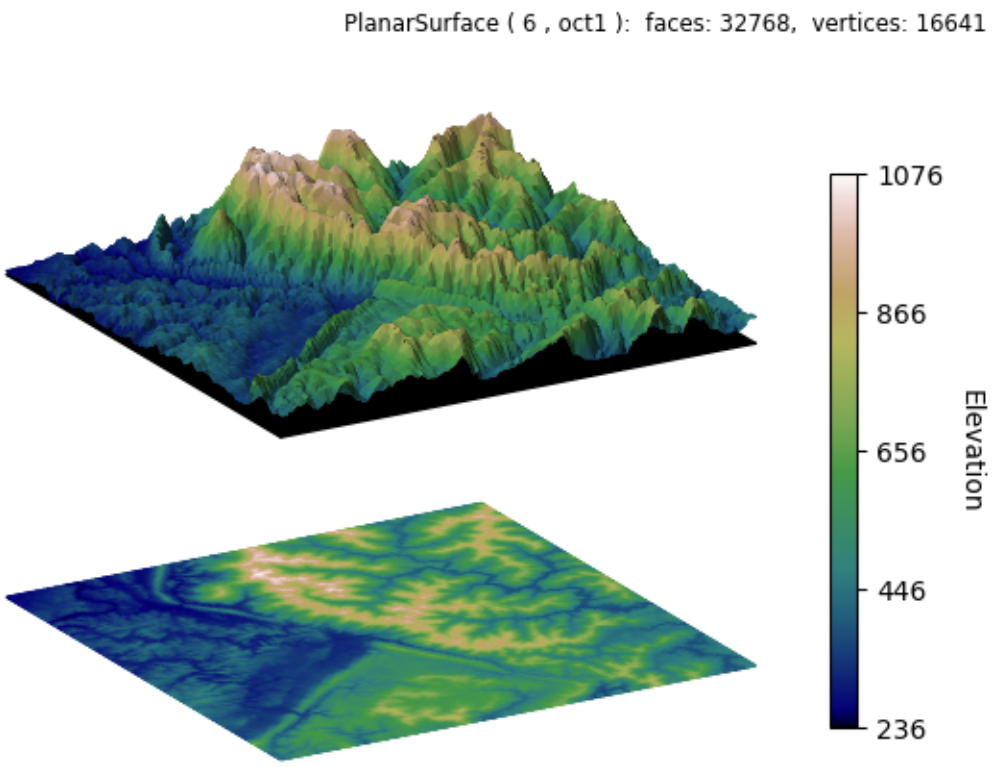


PlanarSurface (6 , oct1): faces: 32768, vertices: 16641

20.9.4 Jacksboro fault

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Jacksboro_Fault.py, and the XML code in the file D04_Jacksboro_Fault.2D.xml.

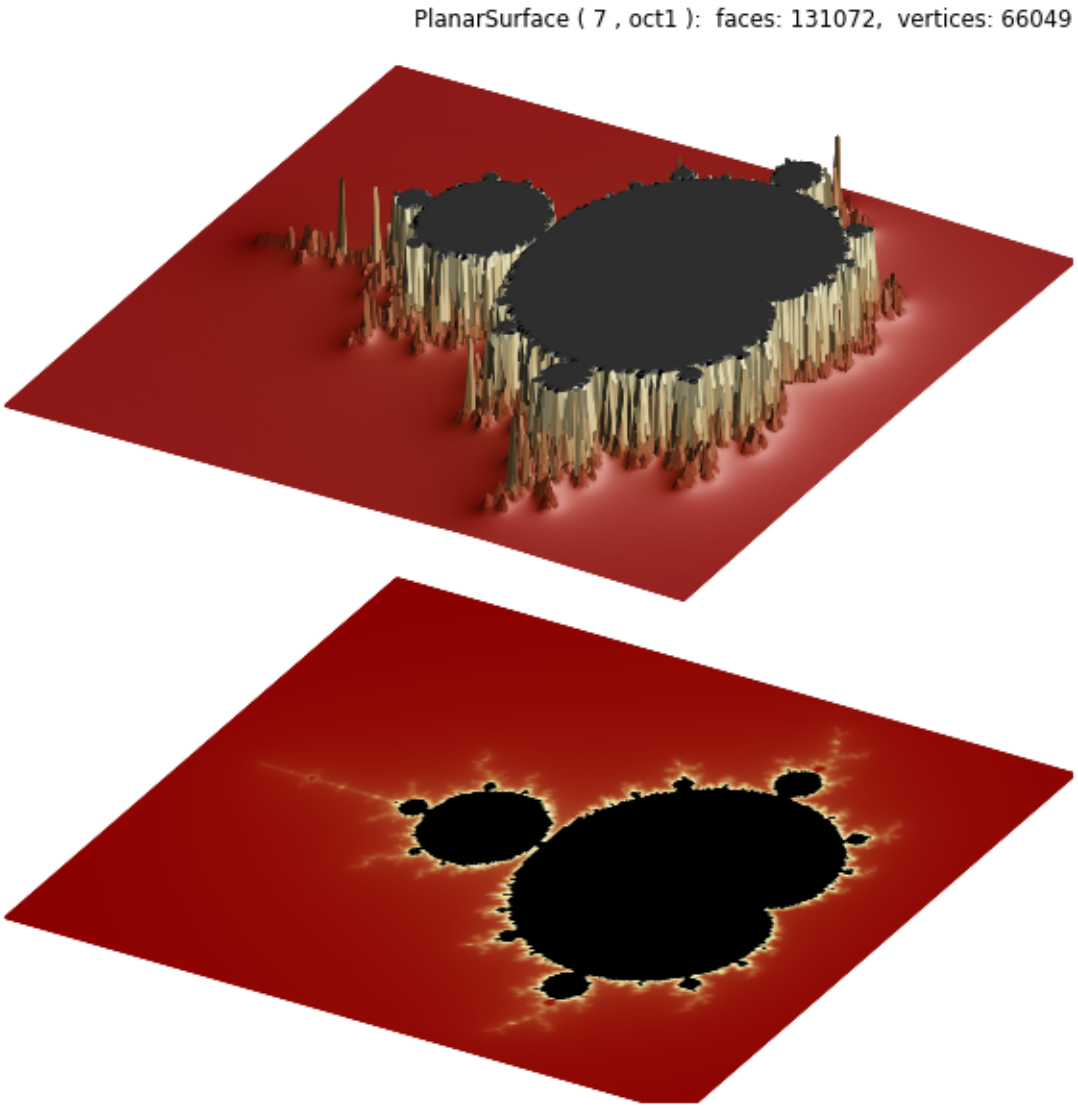
See: https://s3dlib.org/examples/datagridmap/jacks_shaded2.html



20.9.5 Mandelbrot 3D

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Mandelbrot_3D.py, and the XML code in the file D05_Mandelbrot_3D.2D.xml.

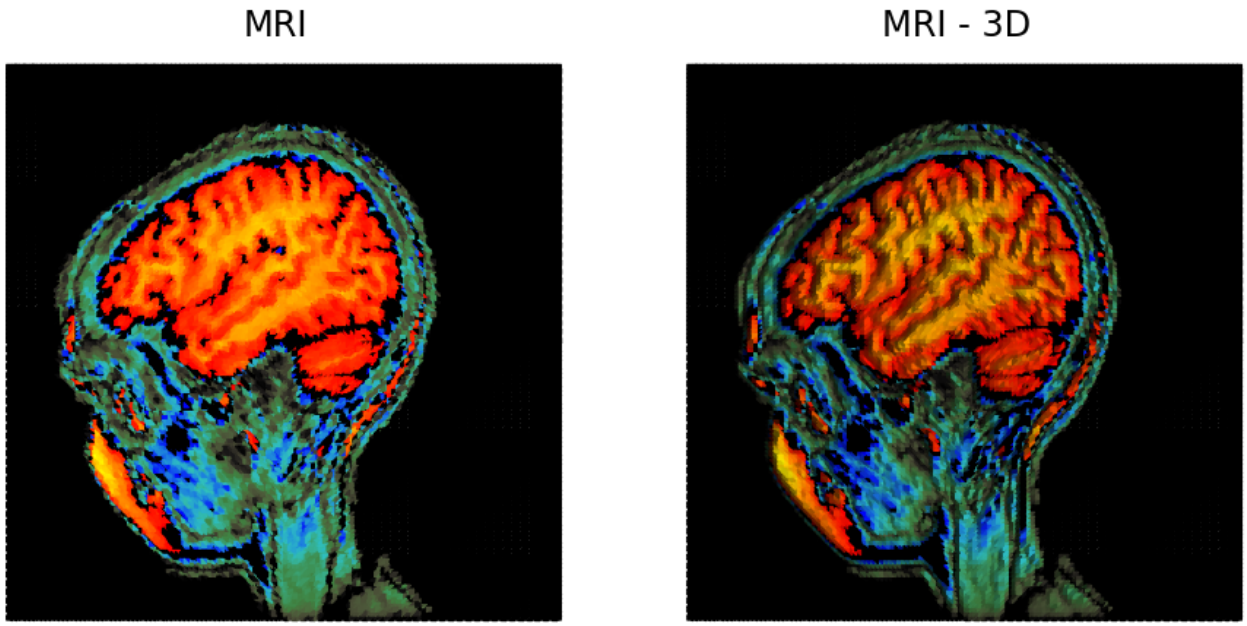
See: https://s3dlib.org/examples/datagridmap/mandelbrot.html



20.9.6 MRI regions

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_MRI_Regions.py, and the XML code in the file D06_MRI_Regions.2D.xml.

See: https://s3dlib.org/examples/colormaps/mri.html



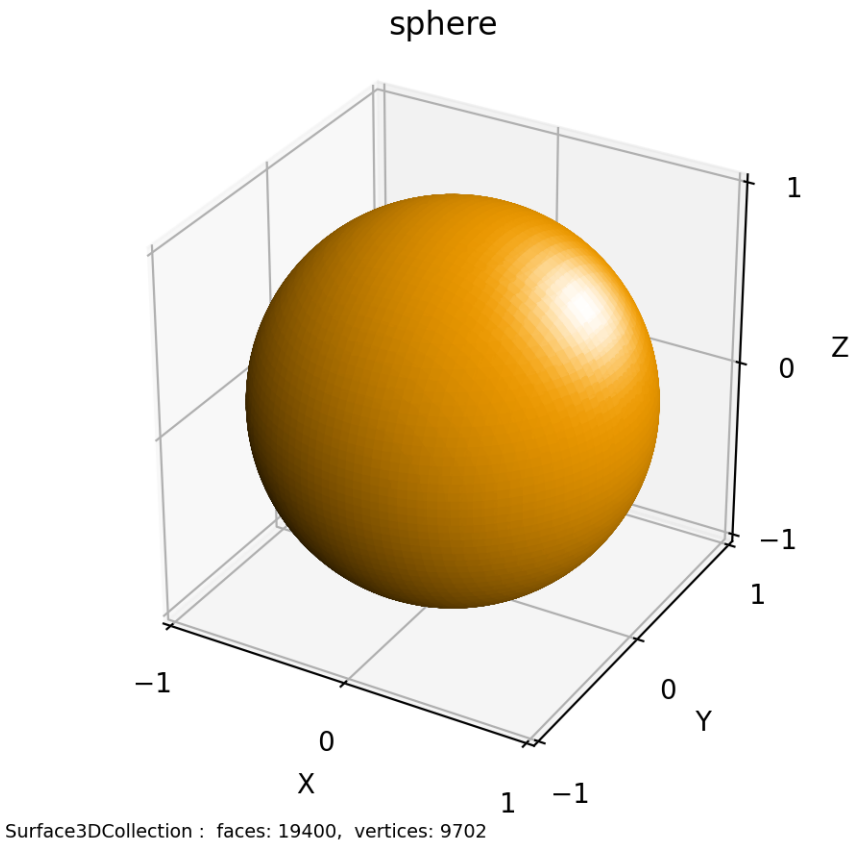
PlanarSurface (6 , oct1): faces: 32768, vertices: 16641

20.10 S3dlib: Implicit surfaces (requires scikit-image)

20.10.1 Sphere (implicit definition), part 1

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Sphere_Implicit_a.py, and the XML code in the file D01_Sphere_Implicit_a.2D.xml.

See: https://s3dlib.org/examples/functional/world_hello.html

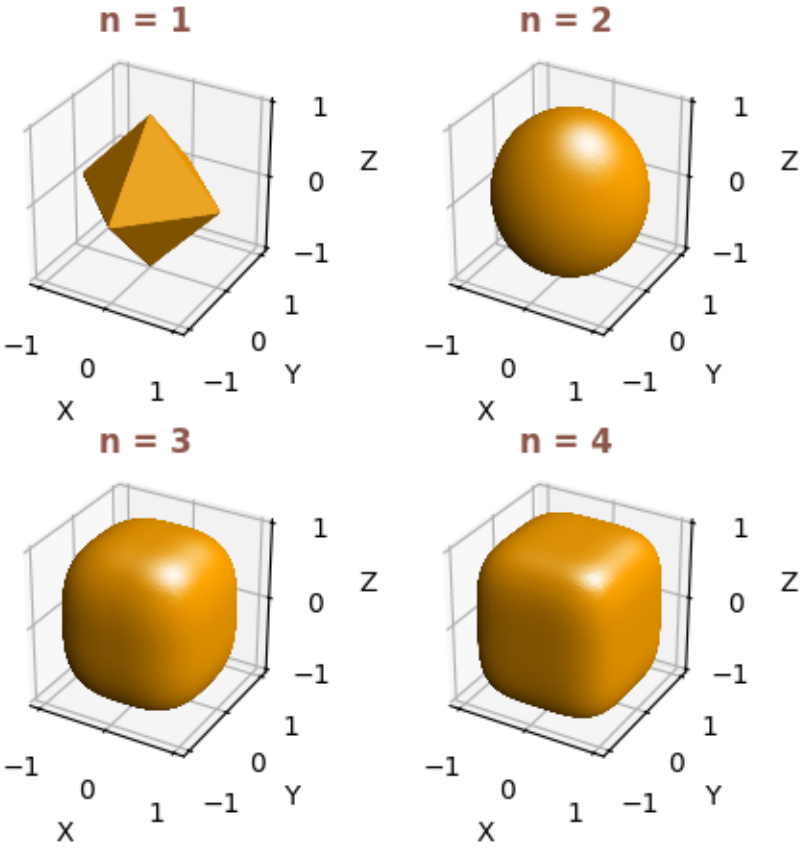


Surface3DCollection : faces: 19400, vertices: 9702

20.10.2 Sphere (implicit definition), part 2

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Sphere_Implicit_b.py, and the XML code in the file D02_Sphere_Implicit_b.2D.xml.

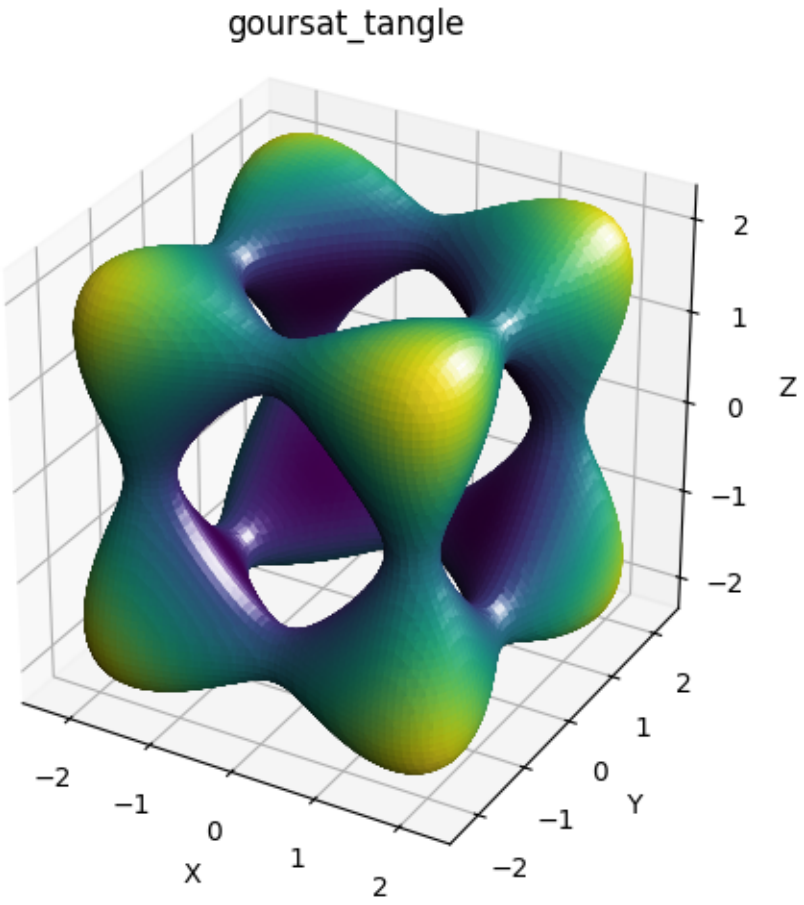
See: https://s3dlib.org/examples/functional/world_hello.html



20.10.3 Goursat surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Goursat_Surface.py, and the XML code in the file D03_Goursat_Surface.2D.xml.

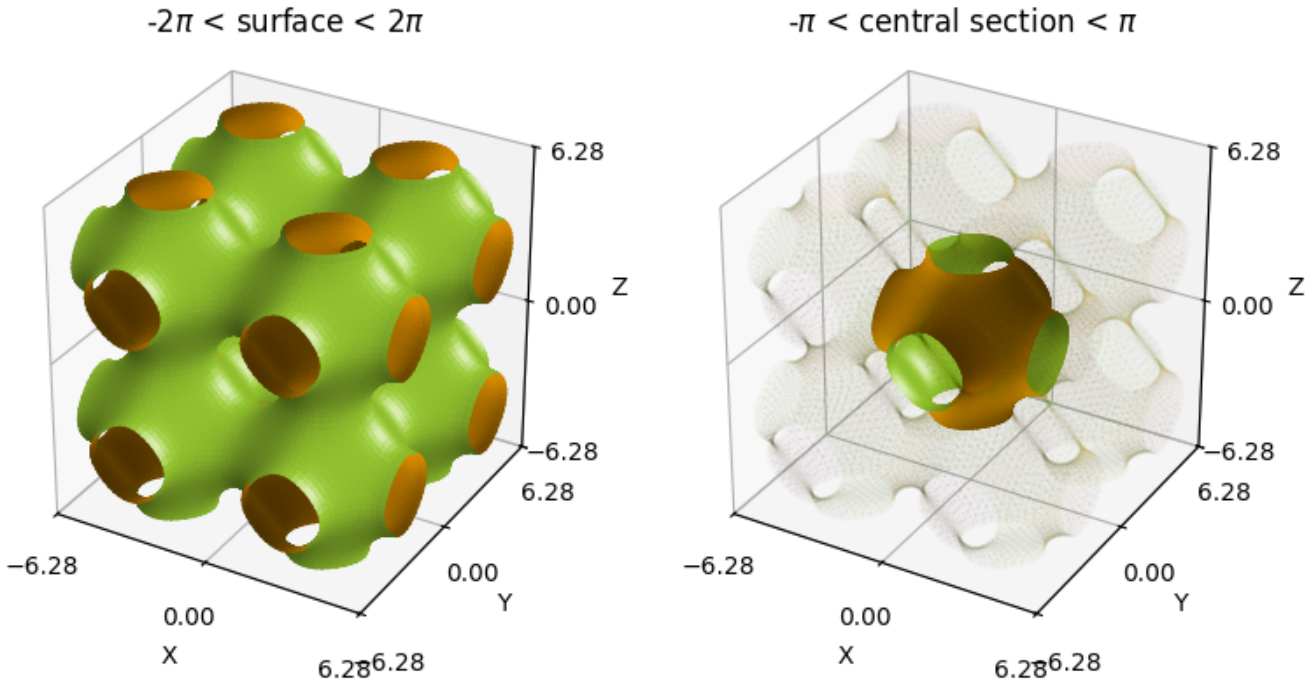
See: https://s3dlib.org/examples/functional/goursat.html



20.10.4 Schwarz P surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Schwarz_P_Surface.py, and the XML code in the file D04_Schwarz_P_Surface.2D.xml.

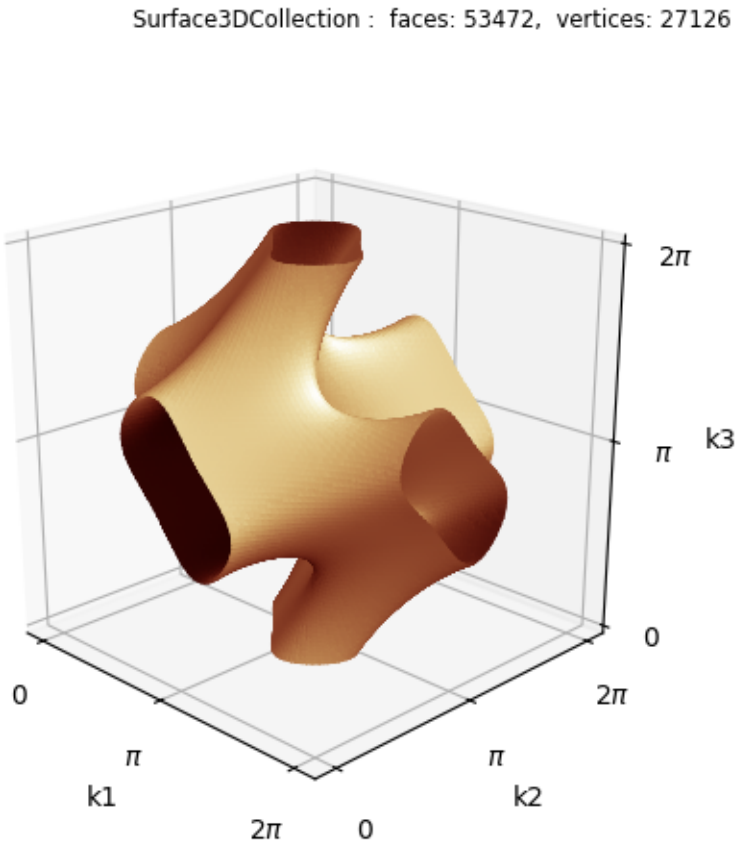
See: https://s3dlib.org/examples/functional/schwarz.html



20.10.5 Nodal surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Nodal_Surface.py, and the XML code in the file D05_Nodal_Surface.2D.xml.

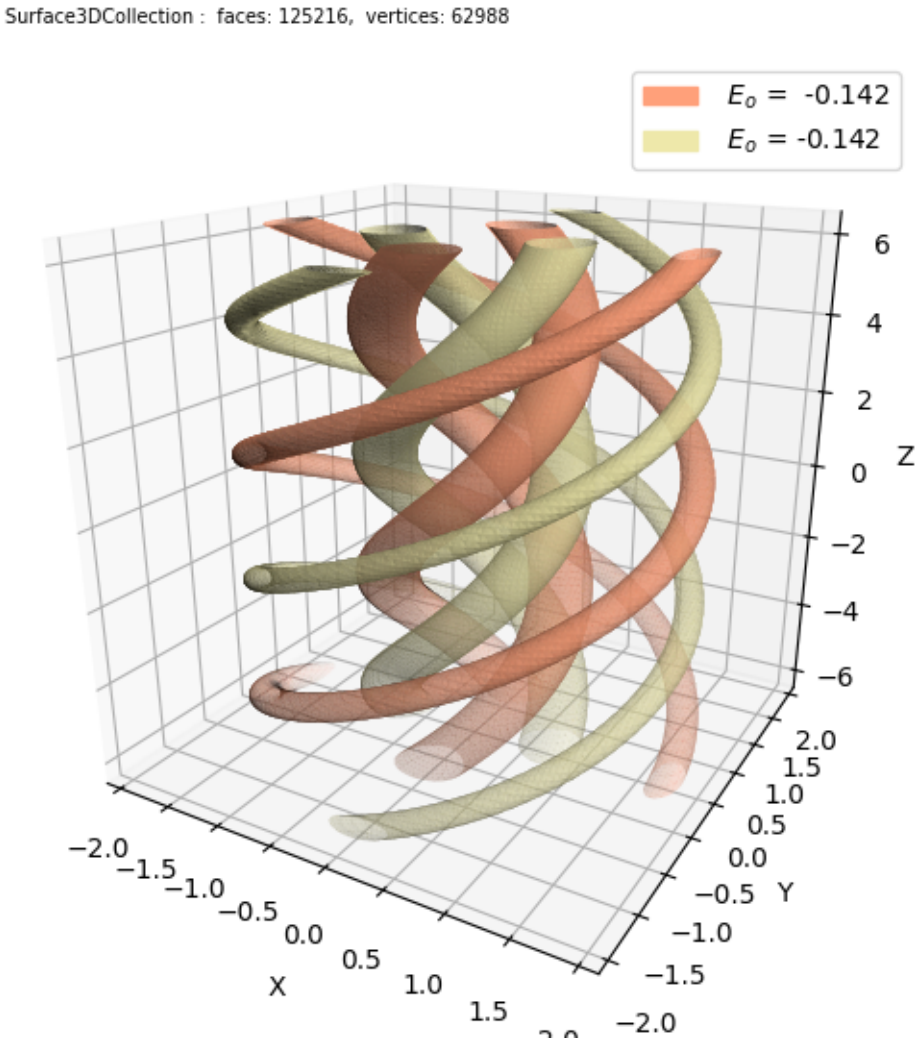
See: https://s3dlib.org/examples/functional/nodesurf.html



20.10.6 Spirals surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Spirals_Surface.py, and the XML code in the file D06_Spirals_Surface.2D.xml.

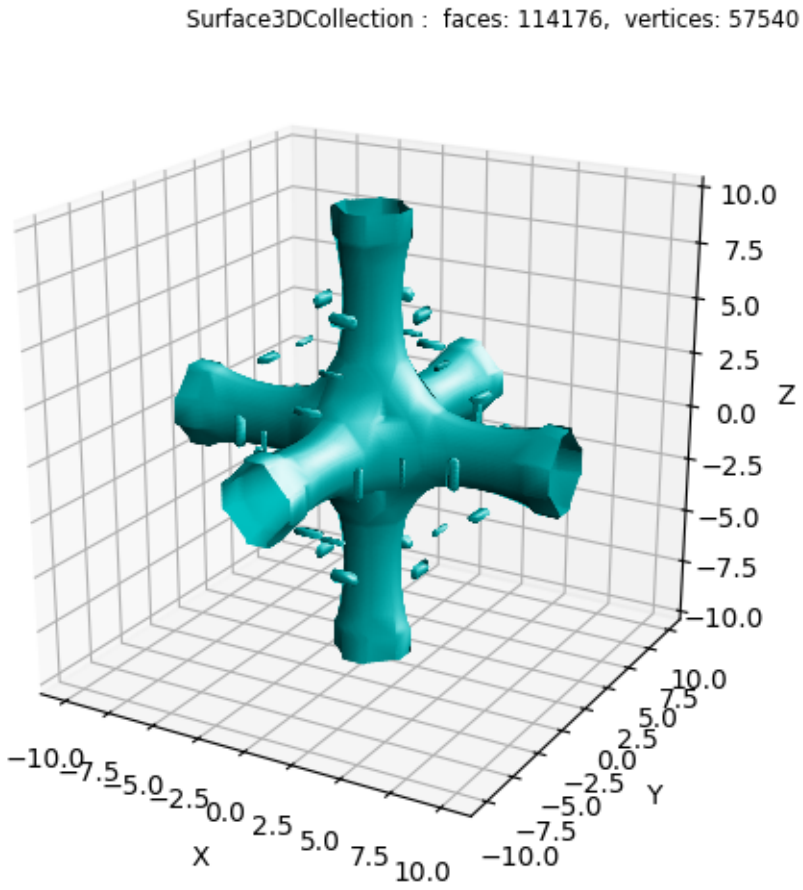
See: https://s3dlib.org/examples/functional/spiralsurf.html



20.10.7 Iso surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_Iso_Surface.py, and the XML code in the file D07_Iso_Surface.2D.xml.

See: https://s3dlib.org/examples/functional/mayavi_surfcont.html



20.10.8 Implicit surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_Implicit_Surface.py, and the XML code in the file D08_Implicit_Surface.2D.xml.

See: https://s3dlib.org/examples/functional/bc_surf.html

CHAPTER

TWENTYONE

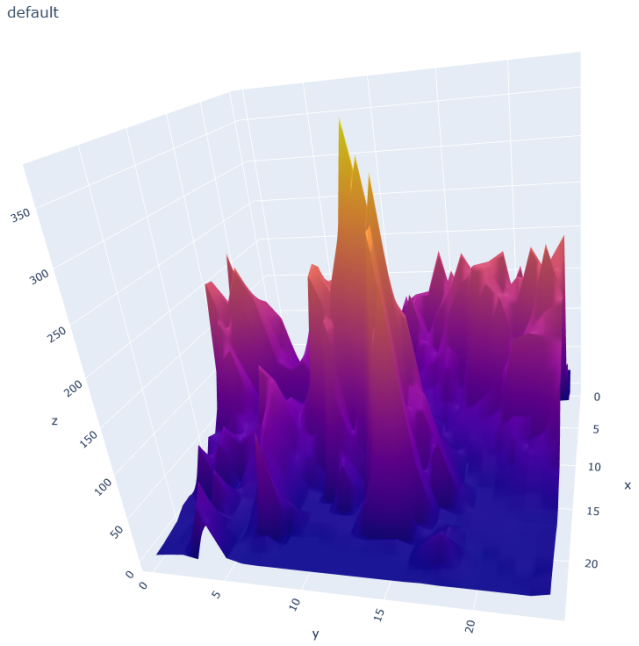
PLOTLY

21.1 General surface plots in 3D

See Test2DFunctions

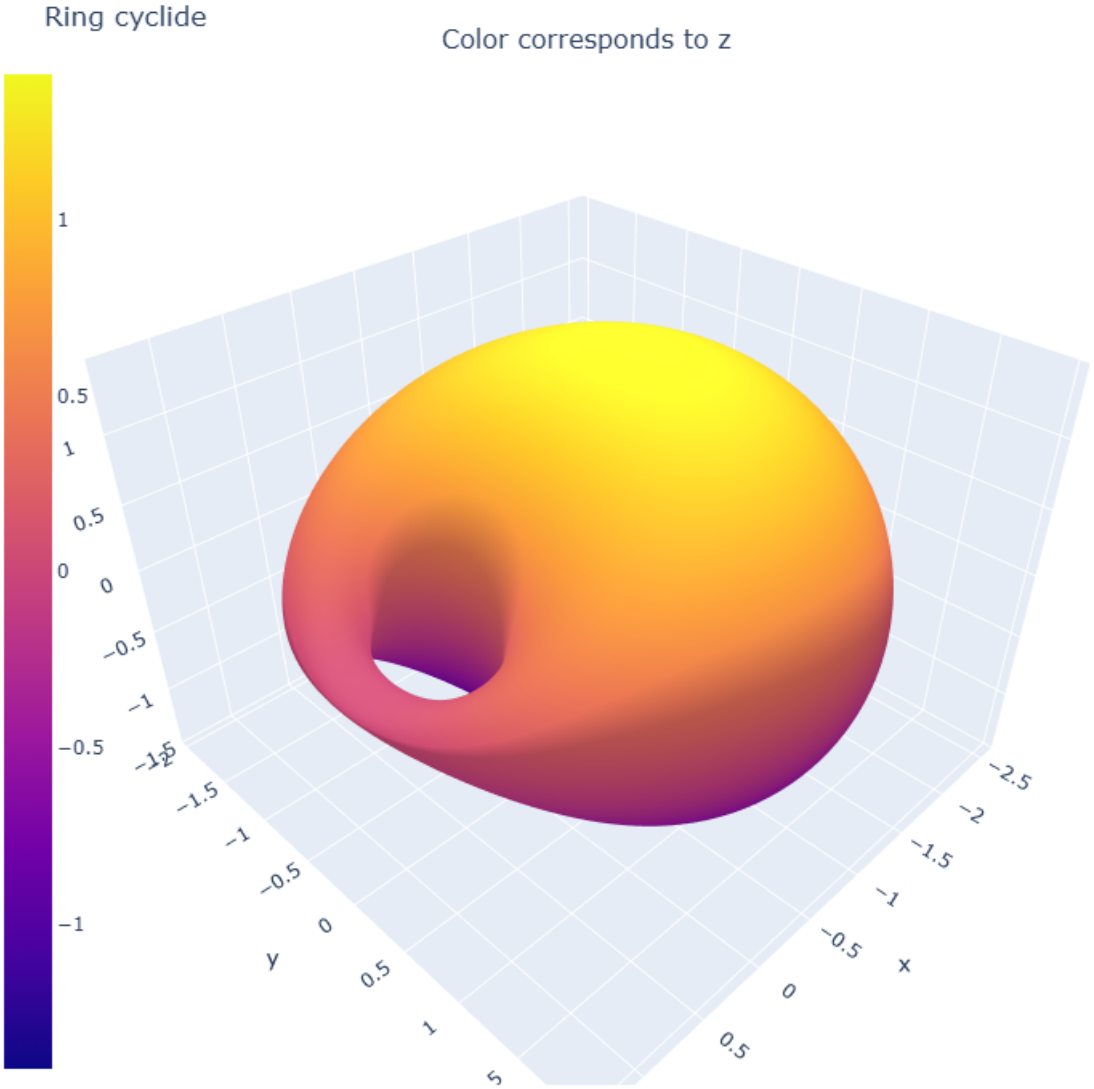
21.1.1 Mount Bruno

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_Plotly3D_MtBruno.py, and the XML code in the file D01_Plotly3D_MtBruno.2D.xml.



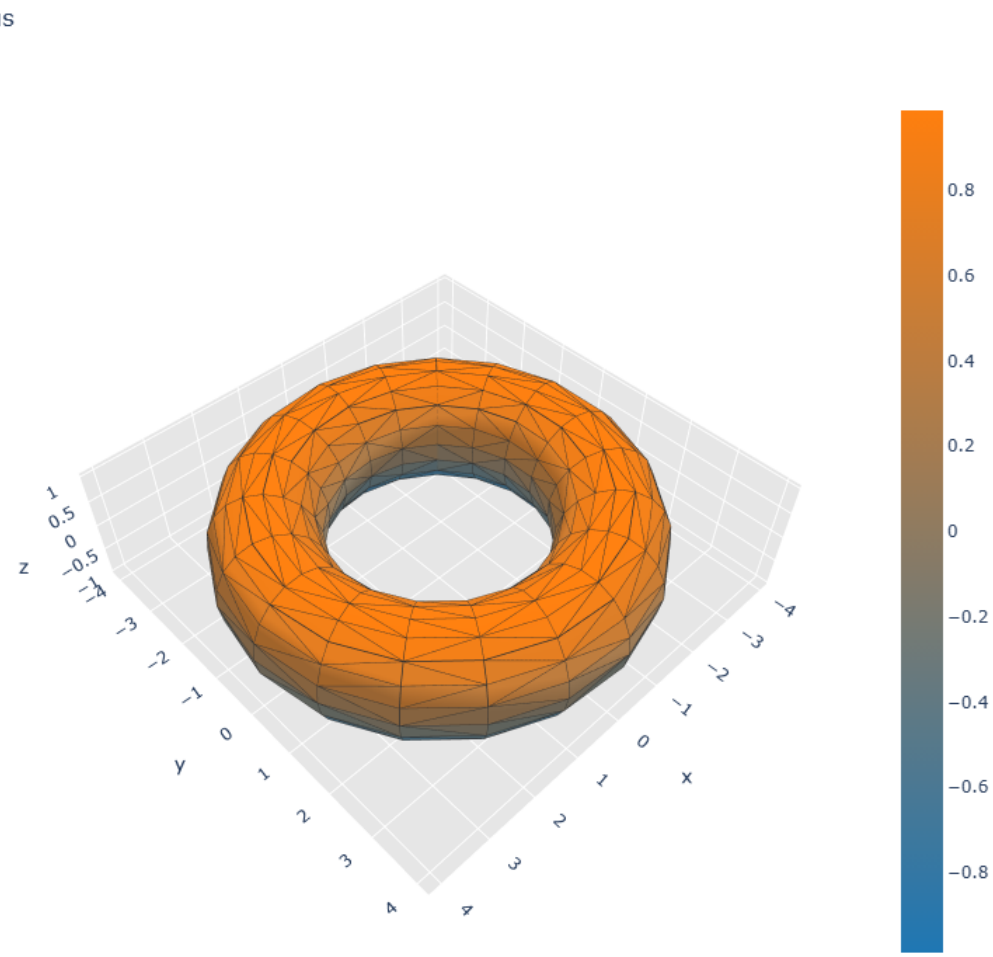
21.1.2 Ring Cyclide

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_Plotly3D_RingCyclide.py, and the XML code in the file D02_Plotly3D_RingCyclide.2D.xml.



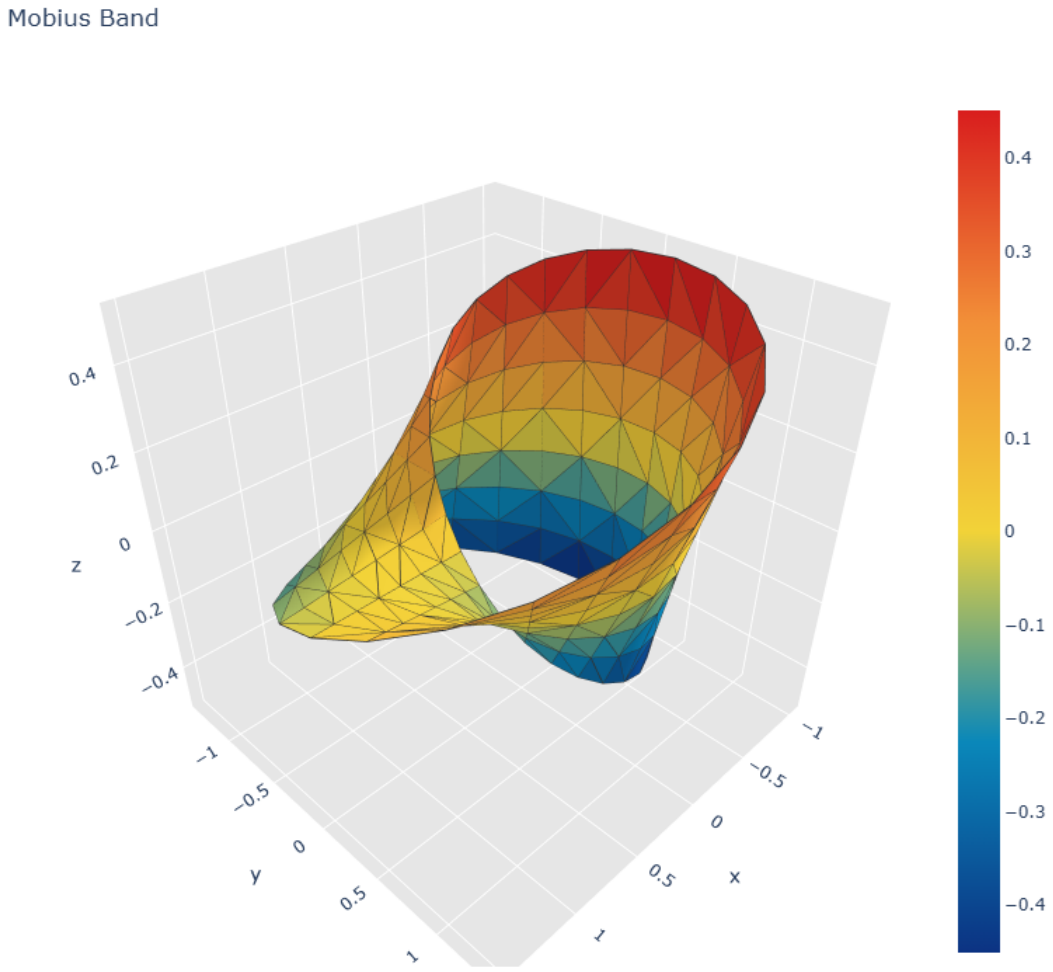
21.1.3 Torus

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_Plotly3D_Torus.py, and the XML code in the file D03_Plotly3D_Torus.2D.xml.



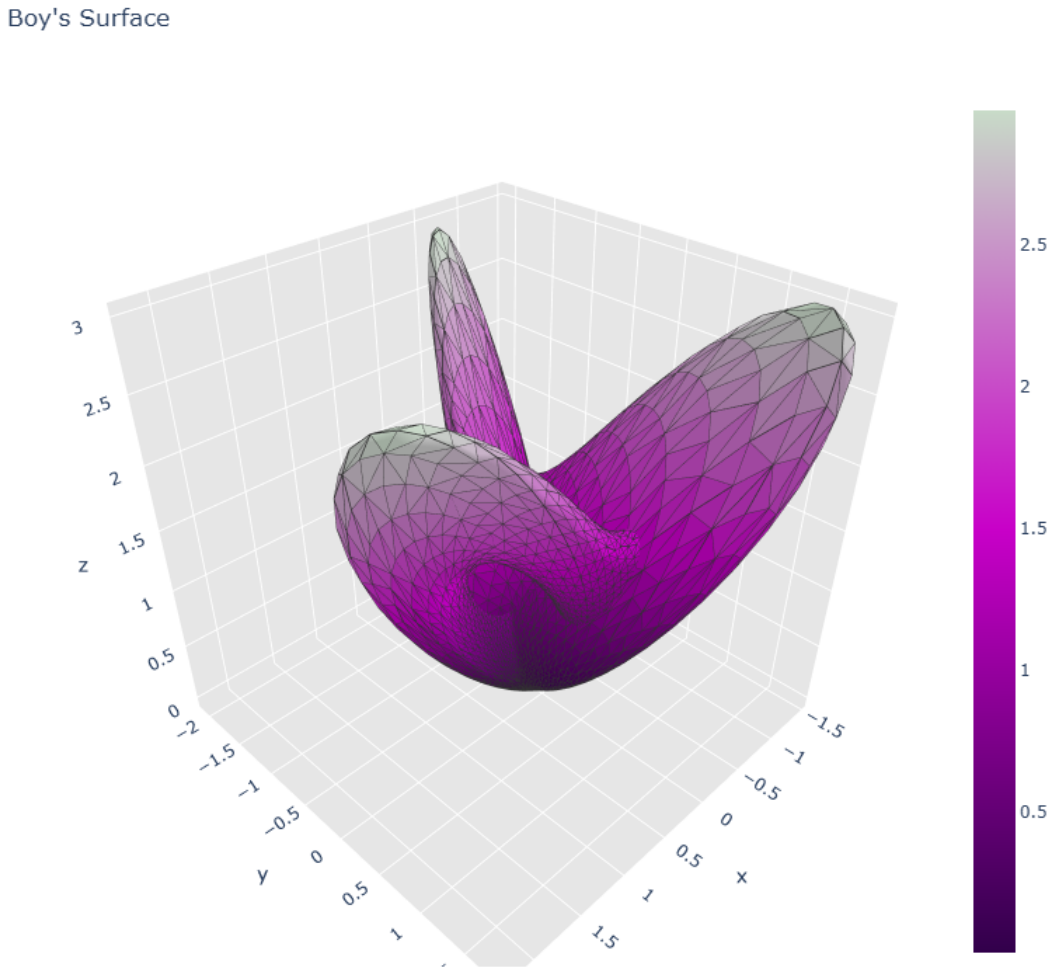
21.1.4 Möbius surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_Plotly3D_M%C3%B6bius.py, and the XML code in the file D04_Plotly3D_M%C3%B6bius.2D.xml.



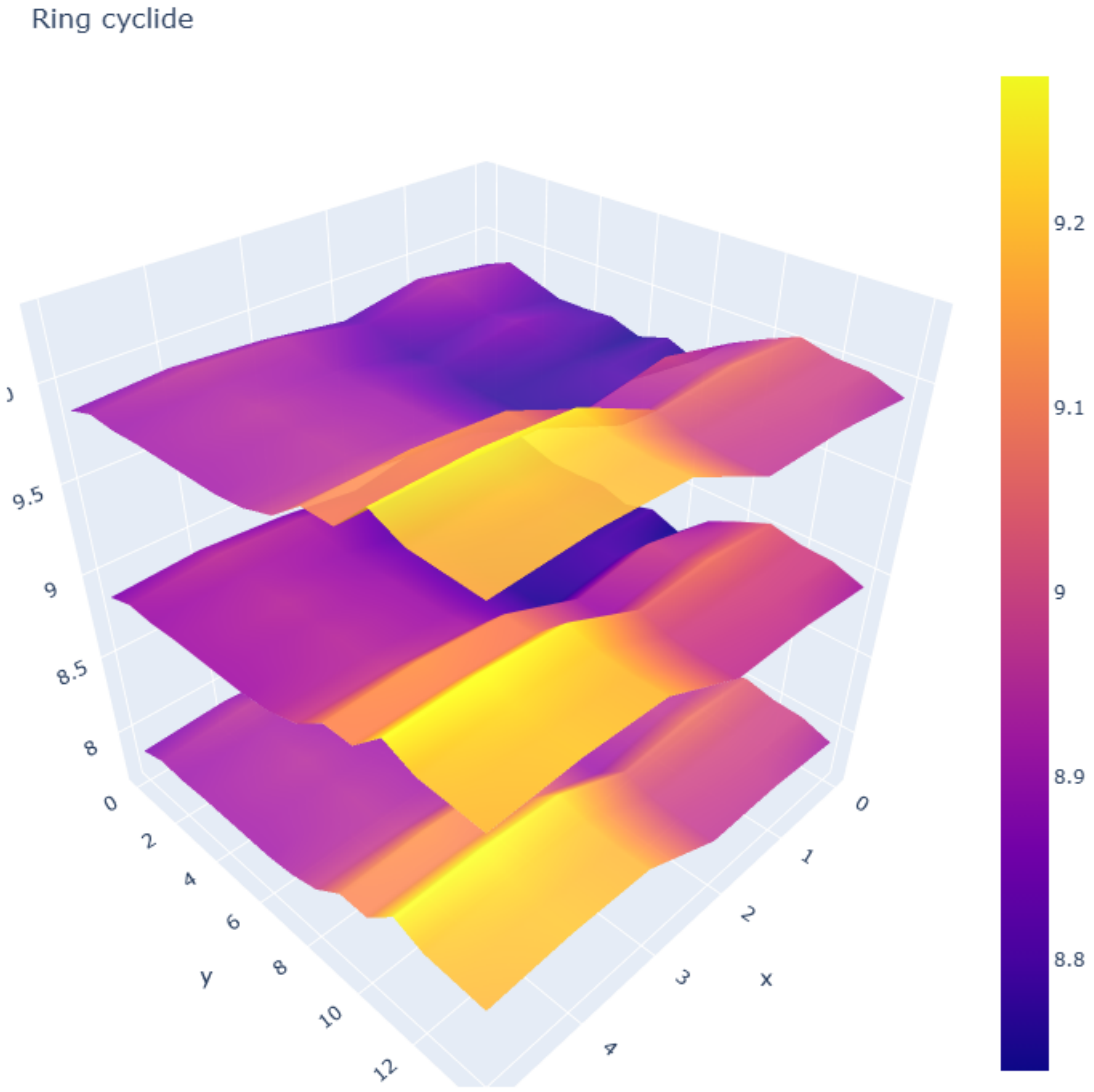
21.1.5 Boy surface

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05_Plotly3D_Boy.py, and the XML code in the file D05_Plotly3D_Boy.2D.xml.



21.1.6 Overlaid function

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06_Plotly3D_OverlaidFunctions.py, and the XML code in the file D06_Plotly3D_OverlaidFunctions.2D.xml.



21.1.7 Iris data

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_Plotly3D_IrisData.py, and the XML code in the file D07_Plotly3D_IrisData.2D.xml.

21.2 Geographical plots

See Test2DFunctions

21.2.1 Movable projection of the world

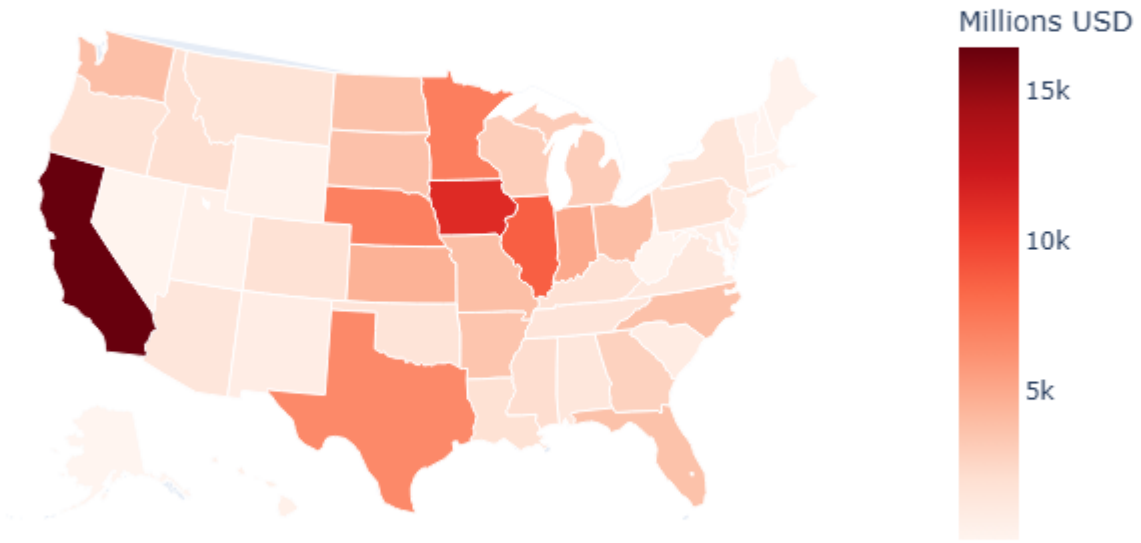
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_PlotlyGeo_MovableProjection.py, and the XML code in the file D01_PlotlyGeo_MovableProjection.2D.xml.

21.2.2 Rotatable globe

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_PlotlyGeo_MovableGlobe.py, and the XML code in the file D02_PlotlyGeo_MovableGlobe.2D.xml.

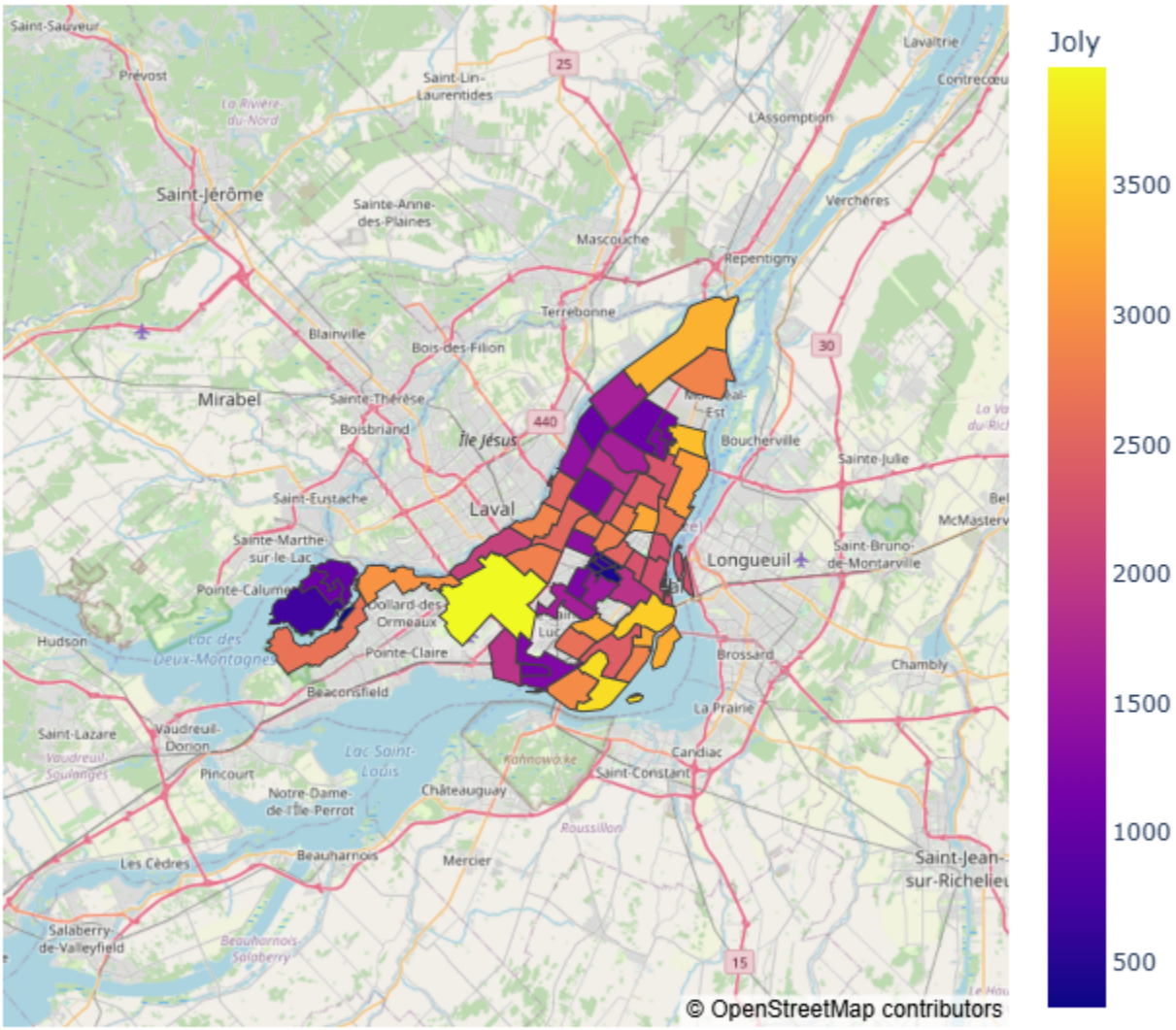
21.2.3 Choropleth of the US with Alaska and Hawai

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_PlotlyGeo_USA_Agriculture.py, and the XML code in the file D03_PlotlyGeo_USA_Agriculture.2D.xml.



21.2.4 Voting results embedded in map

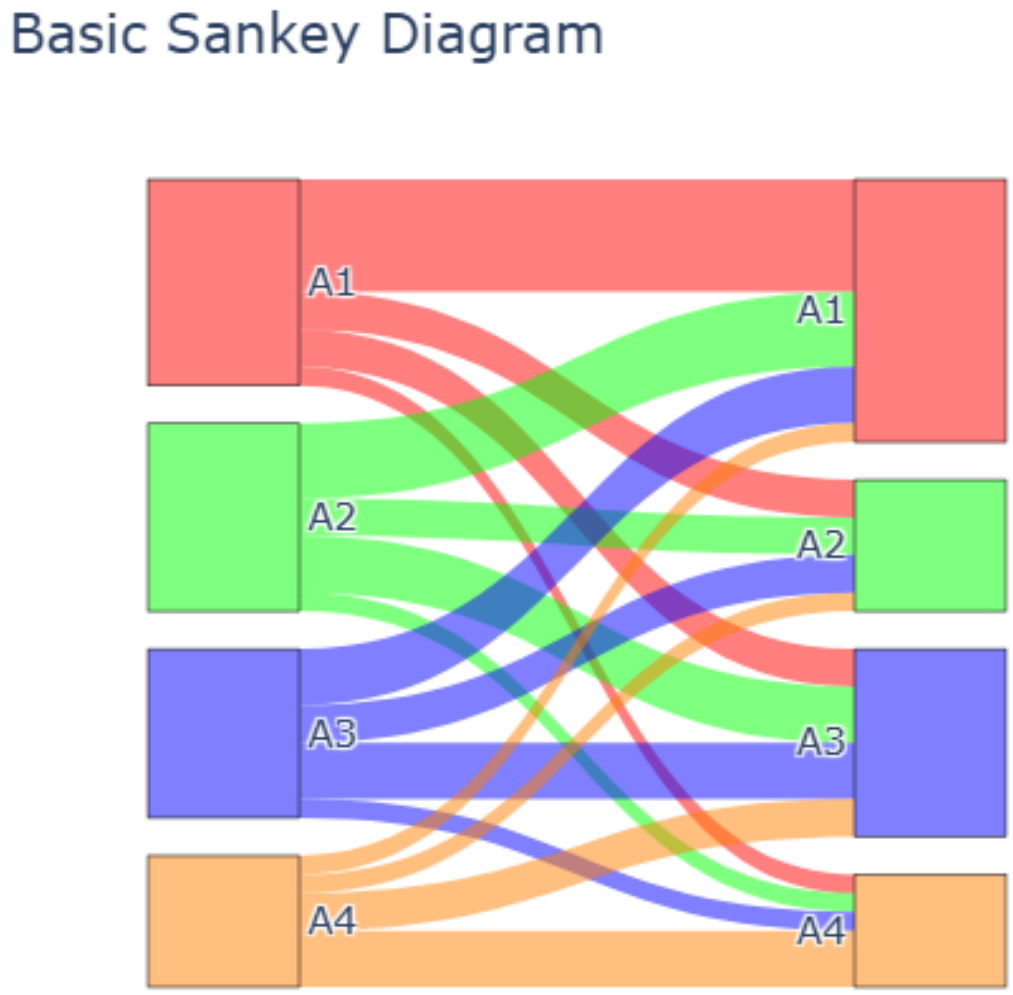
The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04_PlotlyGeo_MontrealVotingWithMap.py, and the XML code in the file D04_PlotlyGeo_MontrealVotingWithMap.2D.xml.



21.3 Sankey plots

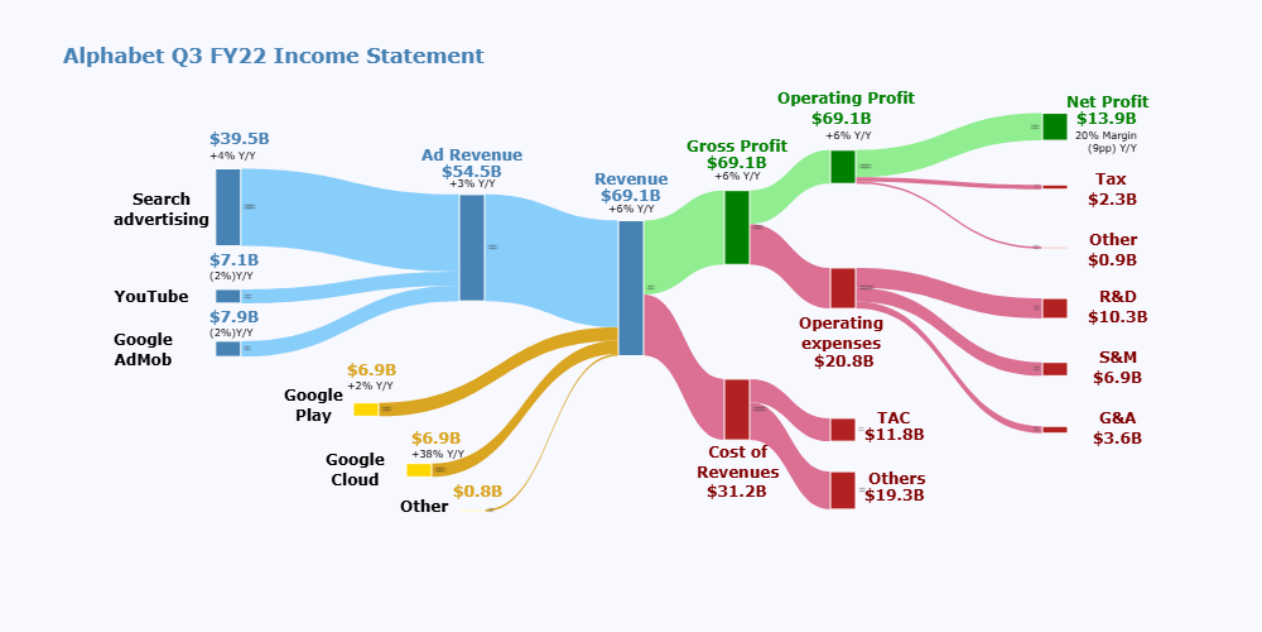
21.3.1 Sankey plot 1

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01_PlotlySankey.py, and the XML code in the file D01_PlotlySankey.2D.xml.



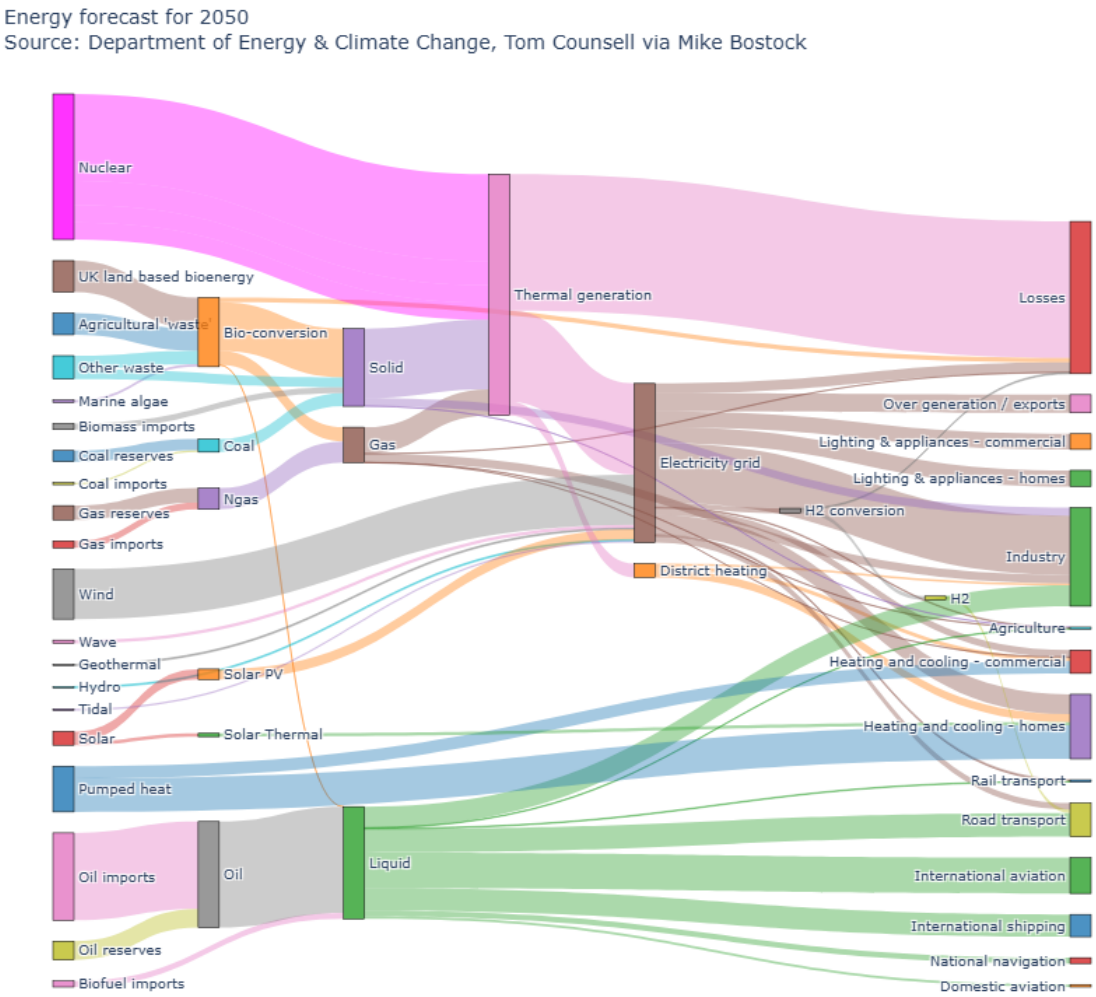
21.3.2 Sankey plot 2

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02_PlotlySankey.py, and the XML code in the file D02_PlotlySankey.2D.xml.



21.3.3 Sankey plot 3

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03_PlotlySankey.py, and the XML code in the file D03_PlotlySankey.2D.xml.



Part V

Interactive 3D Wpf Plots

CHAPTER

TWENTYTWO

WPF: ALTITUDE SURFACES IN 3D, REAL AND COMPLEX FUNCTIONS

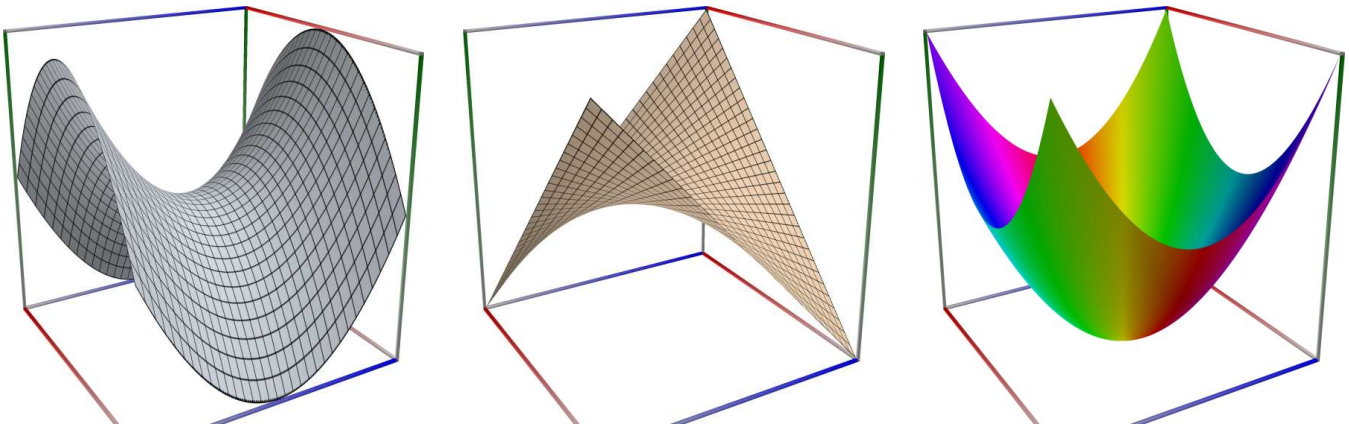
Explicit Surfaces: $z = f(x, y)$

22.1 Special techniques for height surfaces, real and complex functions

22.1.1 Wpf figures as standard display of complex functions

This is a description of the standard display of complex functions, using the square function as an example:

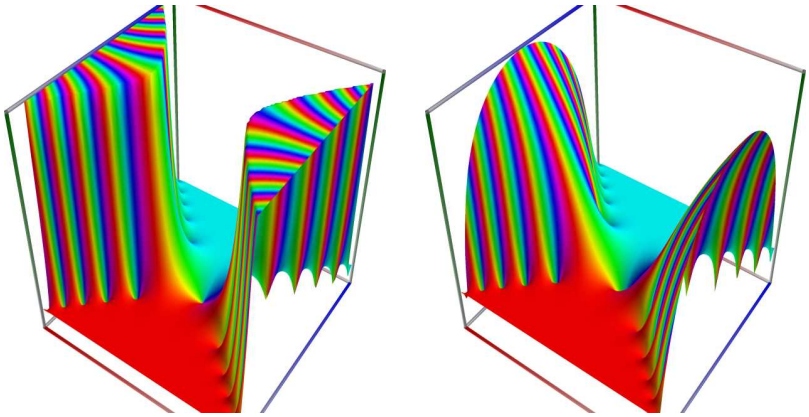
The figures below are showing the real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex function $z = \mathrm{sqr}(x + iy)$ with $-2 \leq x \leq 2$ (blue axis), $-2 \leq y \leq 2$ (red axis), $-10 \leq z \leq 10$ (green axis).



Note

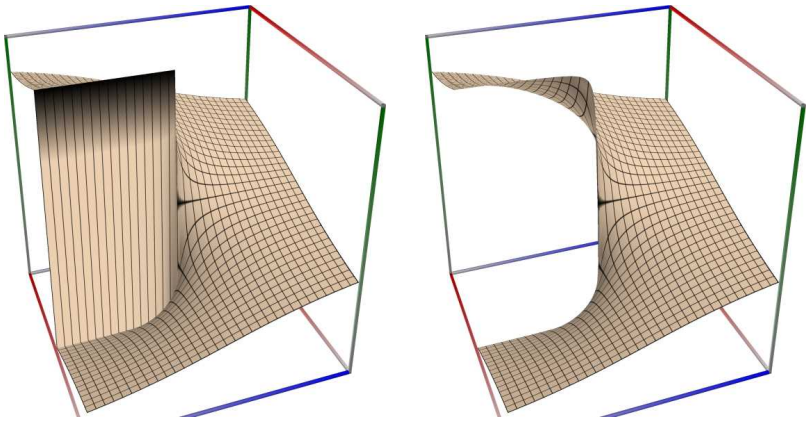
Although the range of the green axis is stated in the form $z_{\min} \leq z \leq z_{\max}$, with $z_{\min} \neq 0$ in general, this applies only for the figures showing the real and imaginary part. For the figure showing the absolute value we have always $z_{\min} = 0$. This note is omitted from the standard text describing complex functions in this manual for better readability.

22.1.2 Truncation vs loglog transformation



3D wpf plot: Surface plot without cutting of the branch cut (left figure), Surface plot with cutting of the branch cut (right figure)

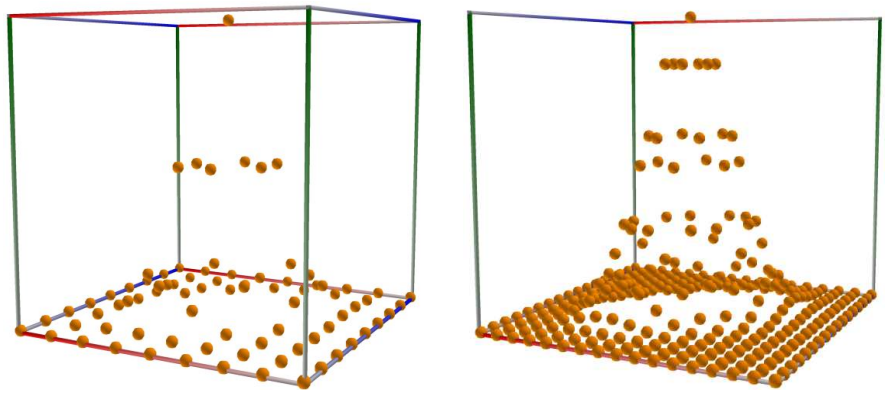
22.1.3 Displaying branch cuts



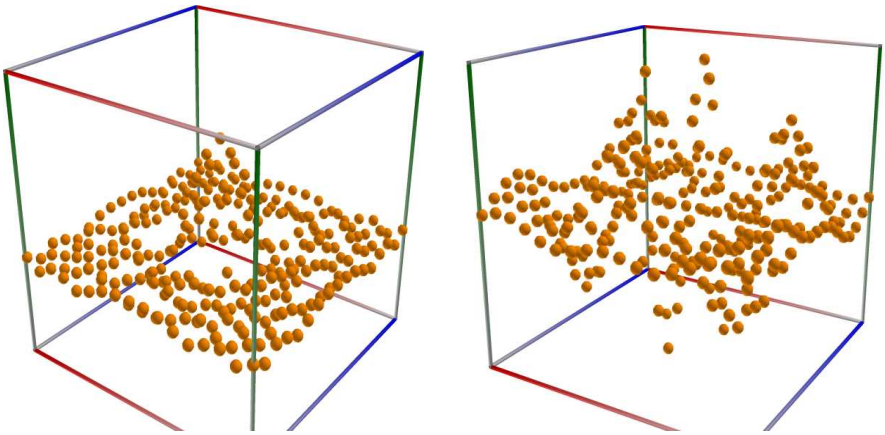
3D wpf plot: Surface plot without cutting of the branch cut (left figure). Surface plot with cutting of the branch cut (right figure).

22.2 Scatterplots and building 3D scenes

22.2.1 Scatter plot of bivariate normal



22.2.2 Scatter plot of surfaces



22.3 Height plots of general bivariate real functions

22.3.1 Hyperbolic paraboloid

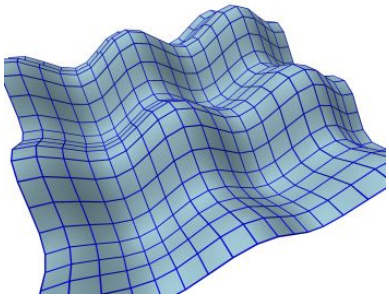
`User.`**`HyperbolicParaboloid`**`(`*a*, *Resolution*`)`

See also: https://mathworld.wolfram.com/HyperbolicParaboloid.html

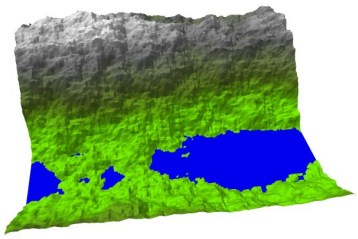
References

Gray, A. "The Hyperbolic Paraboloid." Modern Differential Geometry of Curves and Surfaces with Mathematica, 2nd ed. Boca Raton, FL: CRC Press, pp. 297-298 and 449, 1997.

22.3.2 Array Surface, general



22.3.3 Fractal Landscape surface

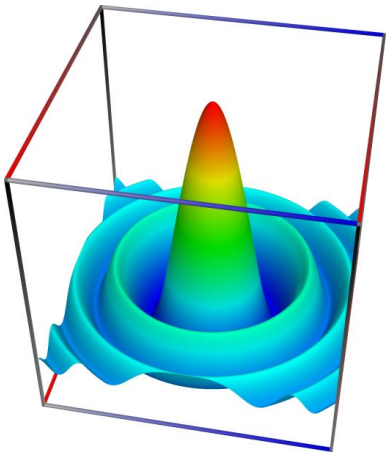


22.3.4 Sombrero function

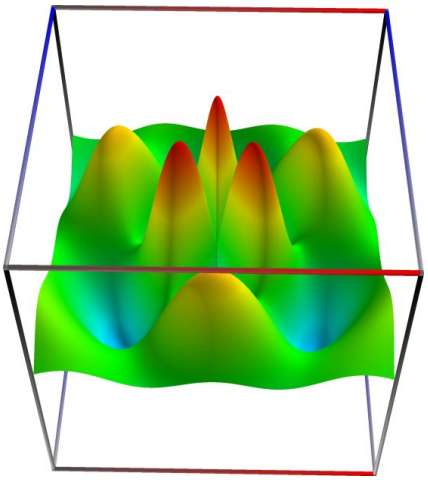
See also: https://en.m.wikipedia.org/wiki/File:Sombrero_function_3d.png

See also: https://mathworld.wolfram.com/JincFunction.html

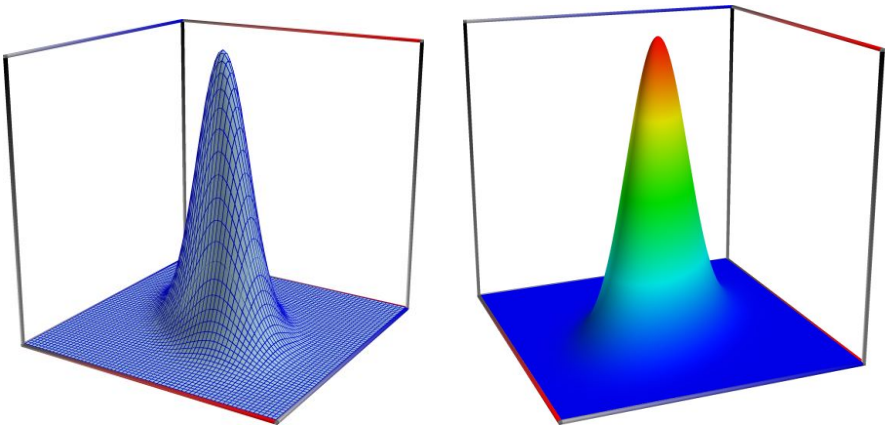
See also: https://en.wikipedia.org/wiki/Sombrero_function



22.3.5 General 3D Wave function



22.3.6 Bivariate normal function



CHAPTER

TWENTYTHREE

WPF: PARAMETRIC SURFACES

23.1 Surfaces of translation

See also: https://en.wikipedia.org/wiki/Translation_surface_(differential_geometry)

See also:http://www.3d-meier.de/tut3/Seite0.html

See also: https://en.wikipedia.org/wiki/Implicit_surface

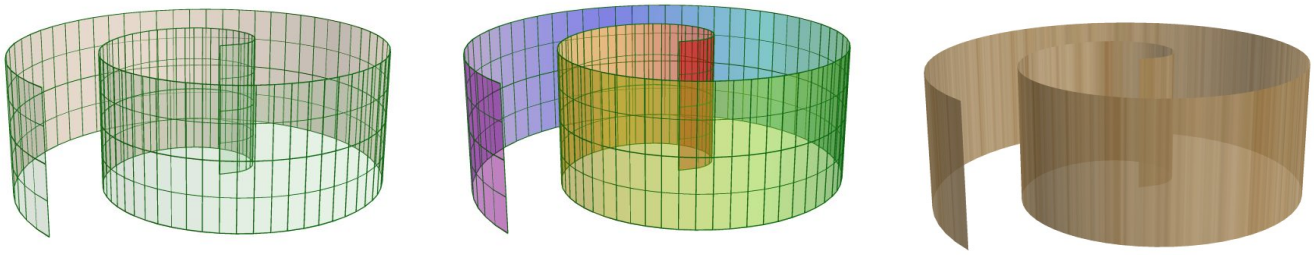
23.1.1 Fermat spiral as generator

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01a_FermatSpiral.3D.xml (D01a-c).

The example below uses the following code in C#

```
var h = 2;
var x = 4*Math.Sqrt(u) * Math.Cos(u);
var y = 4*Math.Sqrt(u) * Math.Sin(u);
var z = h * v;
```

See also Wikipedia [1366], MathWorld [1068].



23.1.2 Lemniscate as generator

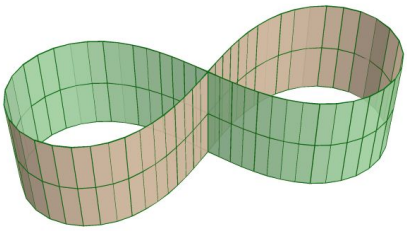
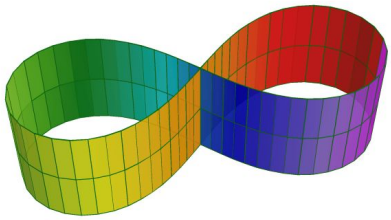
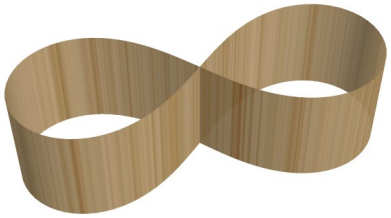
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Lemniscate.3D.xml (D02a-c).

The example below uses the following code in C#

```
double a = 0.3;
double c = 1.0;
double r = c * 0.5;
var x = r * (Math.Cos(u) / (1 + Math.Sin(u) * Math.Sin(u)));
var y = r * (Math.Sin(u) * Math.Cos(u) / (1 + Math.Sin(u) * Math.Sin(u)));
var z = a * v;
```

// see also: http://www.3d-meier.de/tut3/Seite152.html

See also Wikipedia [1366], MathWorld [1068].

23.1.3 Tanh spiral as generator

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_TanhSpiral.3D.xml (D03a-c).

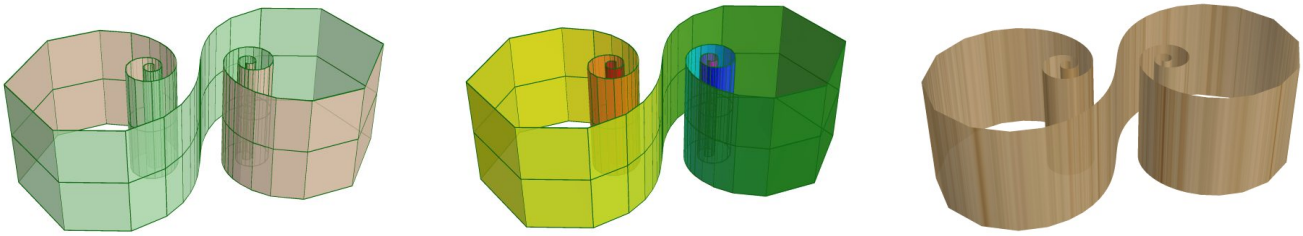
The example below uses the following code in C#

```
var h = 0.5;
double c = 0.2;
double a = 6.0;
var x = c * (Math.Sinh(2 * u) / (Math.Cos(2 * a * u) + Math.Cosh(2 * u)));
var y = c * (Math.Sin(2 * a * u) / (Math.Cos(2 * a * u) + Math.Cosh(2 * u)));
var z = h * v;
```

// see http://www.3d-meier.de/tut3/Seite187.html // see Krivoshapko, p. 498

See also Wikipedia [1366], MathWorld [1068].

See also Wikipedia [1366], MathWorld [1068].



23.1.4 Rose curve as generator

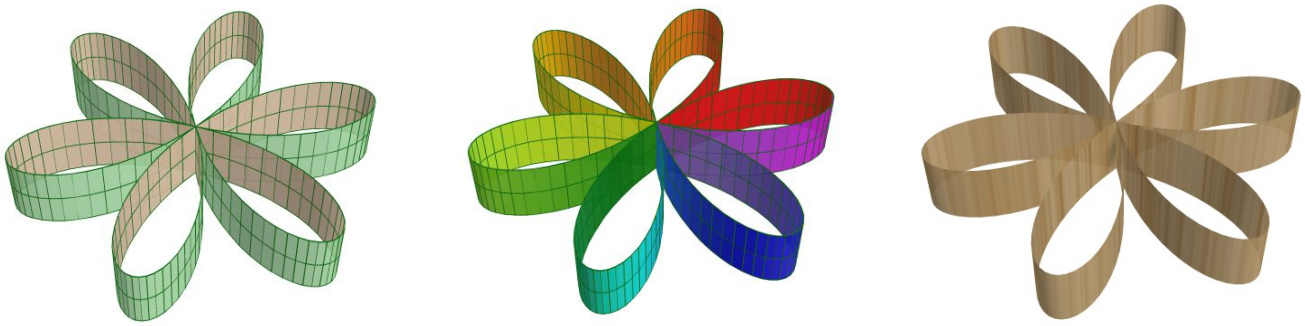
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_RoseCurve.3D.xml (D04a-c).

The example below uses the following code in C#

```
double a = 0.6;
double h = 2.0;
double n = 3.0;
double ca = Math.Cos(a);

var x = (h * Math.Abs(Math.Cos(n * u)) * ca) * Math.Cos(u);
var y = (h * Math.Abs(Math.Cos(n * u)) * ca) * Math.Sin(u);
var z = a * v;
```

// see http://www.3d-meier.de/tut3/Seite189.html

See also Wikipedia [1366], MathWorld [1068].



23.1.5 Hypocycloid as generator

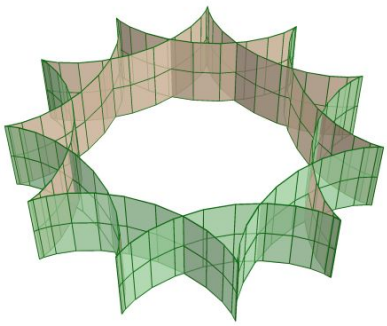
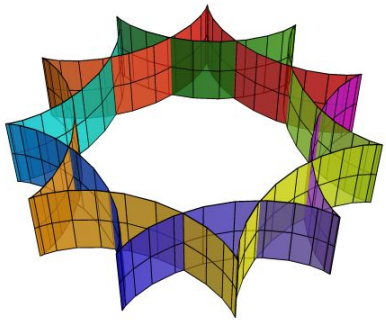
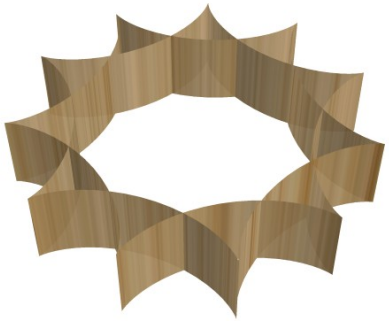
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Hypocycloid_55.3D.xml (D05a-c).

The example below uses the following code in C#

```
double a = 3.7;
double r = 1.5;
double k = 5.5;
var x = r * (k - 1) * Math.Cos(u) + r * Math.Cos((k - 1) * u);
var y = r * (k - 1) * Math.Sin(u) - r * Math.Sin((k - 1) * u);
var z = a * v;
```

// see http://www.3d-meier.de/tut3/Seite189.html

See also Wikipedia [1366], MathWorld [1068].

23.1.6 Epicycloid as generator

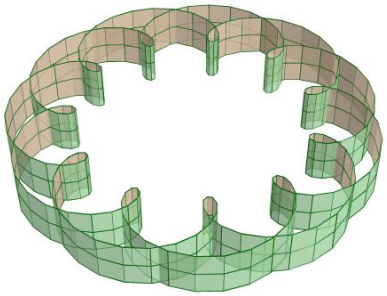
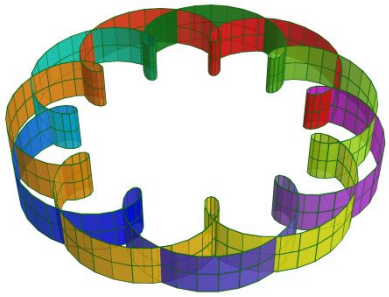
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_Epicycloid_55.3D.xml (D06a-c).

The example below uses the following code in C#

```
double a = 2.0;
double r = 1.0;
double k = 5.5;
var h = 1.5;
var x = r * (k + 1) * Math.Cos(u) - h * r * Math.Cos((k + 1) * u);
var y = r * (k + 1) * Math.Sin(u) - h * r * Math.Sin((k + 1) * u);
var z = a * v;
```

// see http://www.3d-meier.de/tut3/Seite189.html

See also Wikipedia [1366], MathWorld [1068].

23.2 Surfaces of revolution

See also: https://mathworld.wolfram.com/SurfaceofRevolution.html

A surface of revolution is a surface generated by rotating a two-dimensional curve about an axis. The resulting surface therefore always has azimuthal symmetry. Examples of surfaces of revolution include the apple surface, cone (excluding the base), conical frustum (excluding the ends), cylinder (excluding the ends), Darwin-de Sitter spheroid, Gabriel's horn, hyperboloid, lemon surface, oblate spheroid, paraboloid, prolate spheroid, pseudosphere, sphere, spheroid, and torus (and its generalization, the toroid).

The standard parameterization of a surface of revolution is given by

$$x(u, v) = \phi(v)\cos(u)$$

$$y(u, v) = \phi(v)\sin(u)$$

$$z(u, v) = \psi(v)$$

23.2.1 One-Sheeted Hyperboloid

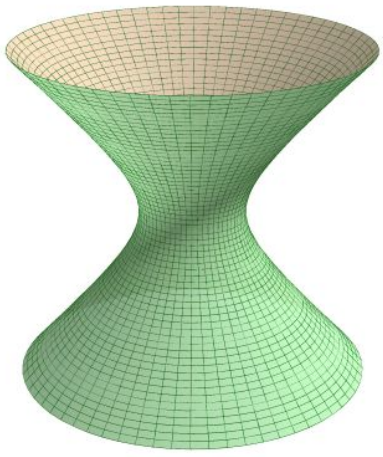
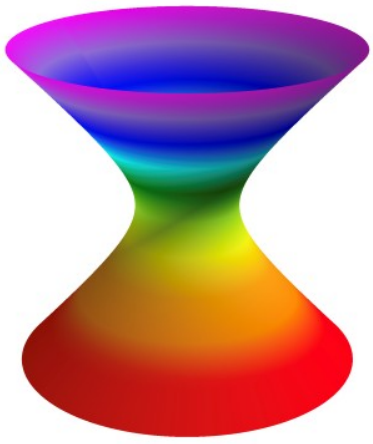
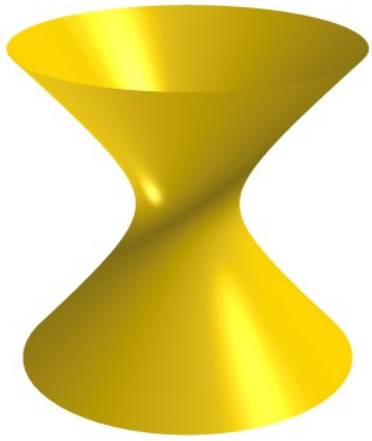
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Hyperboloid1.3D.xml (D01a-c).

The example below uses the following code in C#

```
var a = 1;
var c = 1;
var b = Math.Sqrt(1 + u * u);
var x = a * b * Math.Sin(v);
var y = a * b * Math.Cos(v);
var z = c * u;
```

See also: https://mathworld.wolfram.com/One-SheetedHyperboloid.html

See also: https://en.wikipedia.org/wiki/Hyperboloid

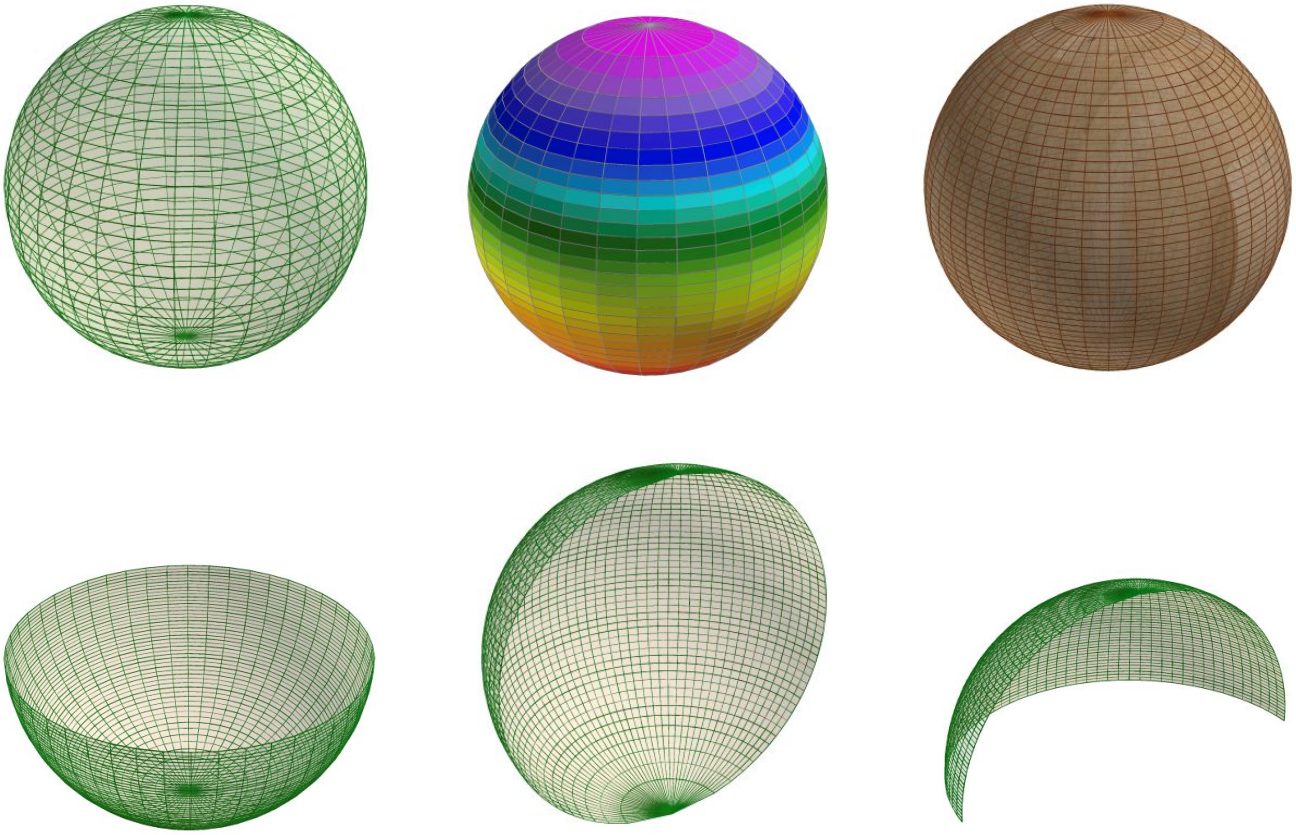
23.2.2 Sphere

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_SphereSqrt.3D.xml (D02a-f).

The example below uses the following code in C#

```
var y = Math.Sqrt(1-u*u) * Math.Cos(v);
var x = Math.Sqrt(1-u*u) * Math.Sin(v);
var z =  u;
```

See also: http://www.3d-meier.de/tut3/Seite86.html, Sphere



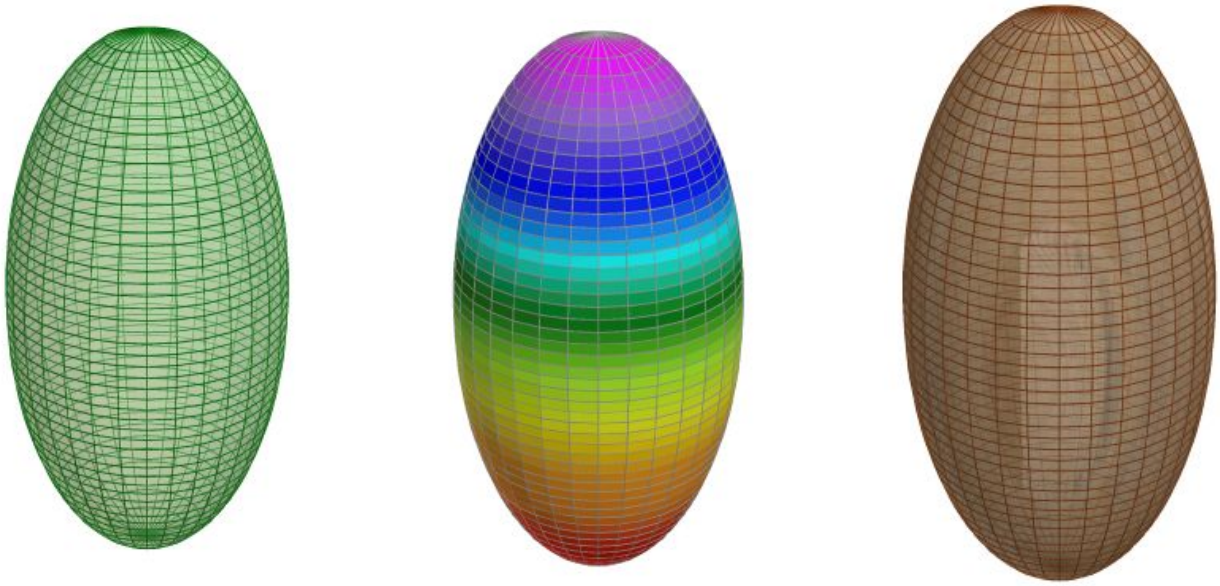
23.2.3 Prolate Spheroid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_ProlateSpheroidoid.3D.xml (D03a-c).

The example below uses the following code in C#

```
var a = 1;
var b = 1;
var c = 2;
var x = b * Math.Sqrt(1-u*u) * Math.Sin(v);
var y = a * Math.Sqrt(1-u*u) * Math.Cos(v);
var z =  c * u;
```

See also: http://paulbourke.net/geometry/spherical/ prolate spheroid, or ellipsoid of revolution See also: https://en.wikipedia.org/wiki/Ellipsoid#Parameterization ellipsoid of revolution See also: https://mathworld.wolfram.com/OblateSpheroid.html Oblate Spheroid



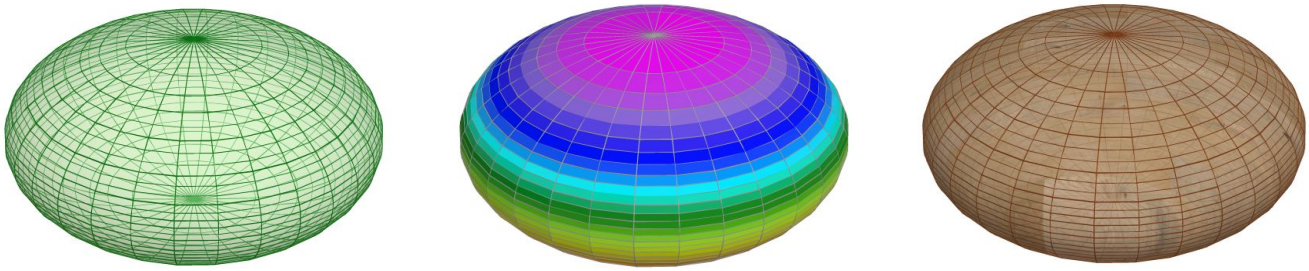
23.2.4 Oblate Spheroid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_OblateSpheroidoid.3D.xml (D04a-c).

The example below uses the following code in C#

```
var a = 2;
var b = 2;
var c = 1;
var x = b * Math.Sqrt(1-u*u) * Math.Sin(v);
var y = a * Math.Sqrt(1-u*u) * Math.Cos(v);
var z =  c * u;
```

See also: http://paulbourke.net/geometry/spherical/ prolate spheroid, or ellipsoid of revolution See also: https://en.wikipedia.org/wiki/Ellipsoid#Parameterization ellipsoid of revolution See also: https://mathworld.wolfram.com/OblateSpheroid.html Oblate Spheroid



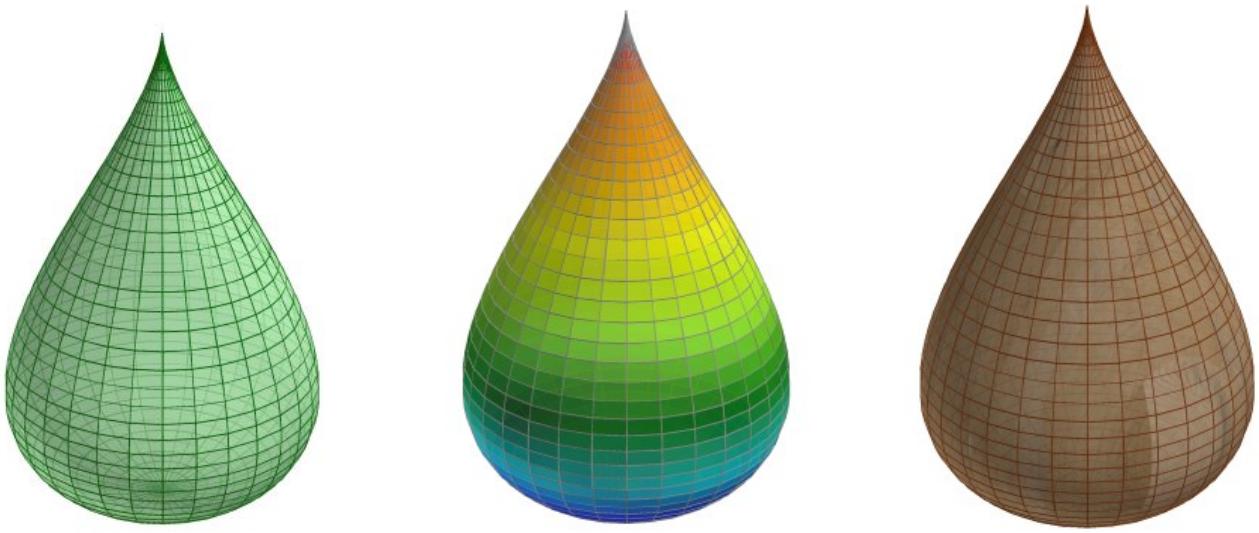
23.2.5 Drop surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_DropSurface.3D.xml (D05a-c).

The example below uses the following code in C#

```
double a = 0.5;  //(0 to 1)
double b = 1.00;  //(0 to 1)
var x = a * (b - Math.Cos(u)) * Math.Sin(u) * Math.Cos(v);
var y = a * (b - Math.Cos(u)) * Math.Sin(u) * Math.Sin(v);
var z = Math.Cos(u);
```

// see http://www.3d-meier.de/tut3/Seite44.html



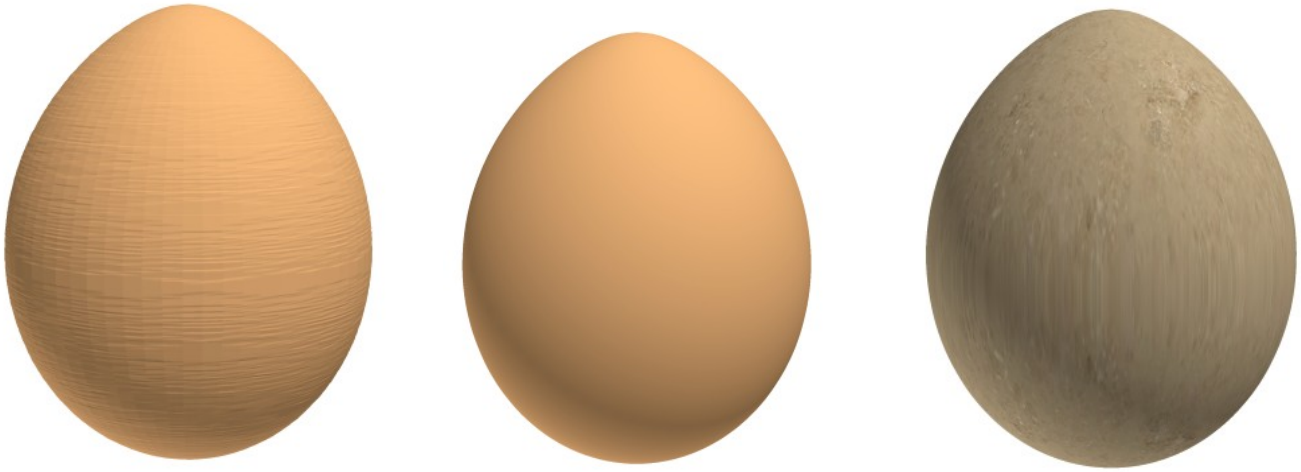
23.2.6 Egg surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_EggSurface.3D.xml (D06a-c).

The example below uses the following code in C#

```
double a = 2.0;  //(0 to 1)
double b = 0.75;  //(0 to 1)
var x = Math.Sqrt(0.75 * u * (2 * a - u) * (1 - (a * a * b * b) / ((u + a) * (u + a)))) * Math.Sin(v);
var y = Math.Sqrt(0.75 * u * (2 * a - u) * (1 - (a * a * b * b) / ((u + a) * (u + a)))) * Math.Cos(v);
var z = u;
```

For the egg surface, also see Meier



23.2.7 Apple surface

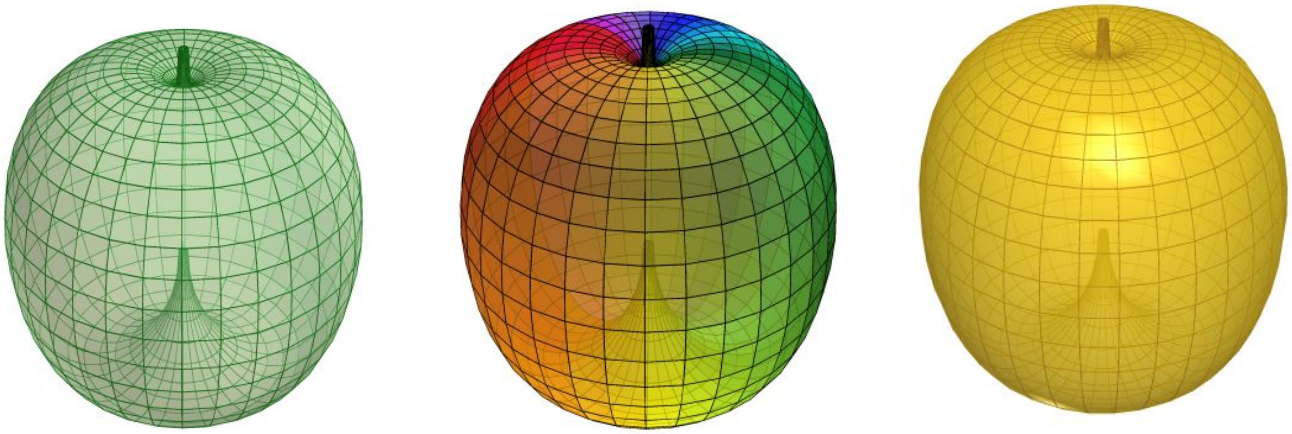
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D07a_Apple.3D.xml (D07a-c).

The example below uses the following code in C#

```
var x =  Math.Cos(u) * (4 + 3.8 * Math.Cos(v));
var y =  Math.Sin(u) * (4 + 3.8 * Math.Cos(v));
var z = (Math.Cos(v) + Math.Sin(v) - 1) * (1 + Math.Sin(v)) * Math.Log(1 - Math.PI * v / 10) + 7.5 * Math.Sin(v);
```

// see http://www.3d-meier.de/tut3/Seite54.html

For the apple surface, also see Meier



23.2.8 Spinning top

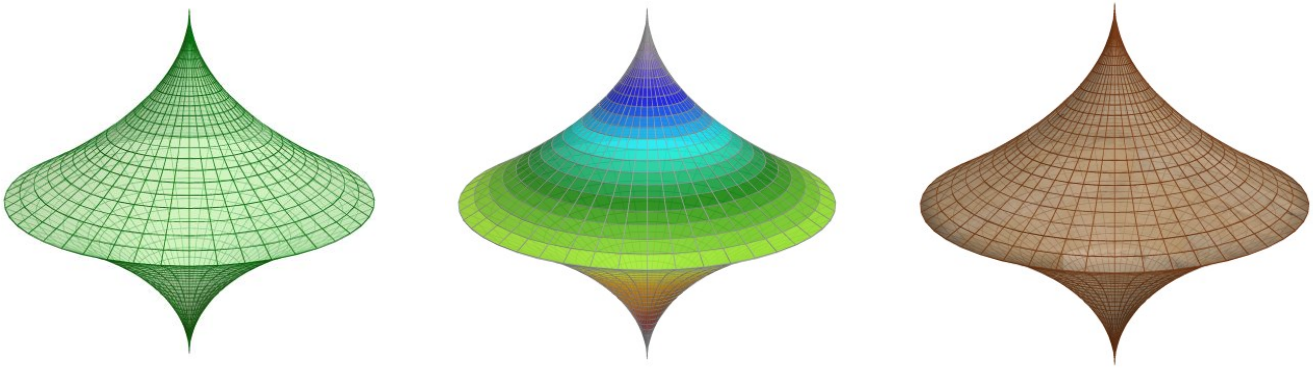
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D08a_Spinningtop.3D.xml (D08a-c).

The example below uses the following code in C#

```
var x = (Math.Abs(u) - 1) * (Math.Abs(u) - 1) * Math.Sin(v);
var y = (Math.Abs(u) - 1) * (Math.Abs(u) - 1) * Math.Cos(v);
var z = u;
```

See also http://www.3d-meier.de/tut3/Seite150.html Kreisel

See also Wikipedia [1366], MathWorld [1068].



23.2.9 Pseudosphere

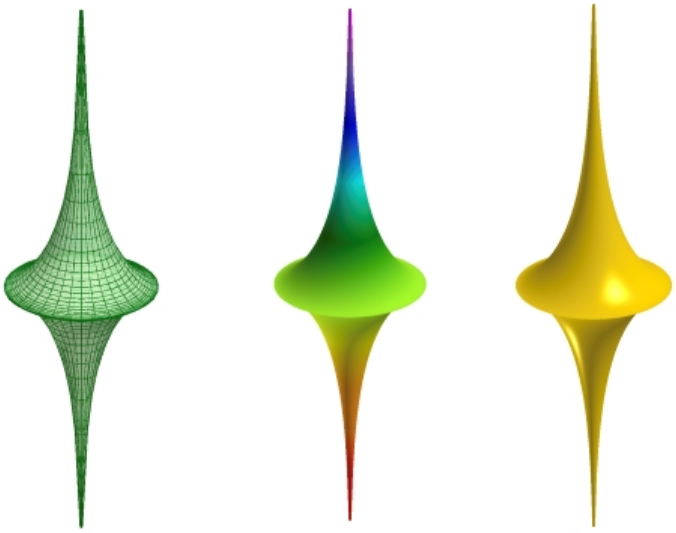
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D09a_PseudoSphere.3D.xml (D09a-c).

The example below uses the following code in C#

```
var x = Math.Cos(u) * Math.Sin(v);
var y = Math.Sin(u) * Math.Sin(v);
var z = (Math.Cos(v) + Math.Log(Math.Tan(0.5 * v)));
```

See also: https://mathworld.wolfram.com/Pseudosphere.html

// see http://www.3d-meier.de/tut3/Seite28.html



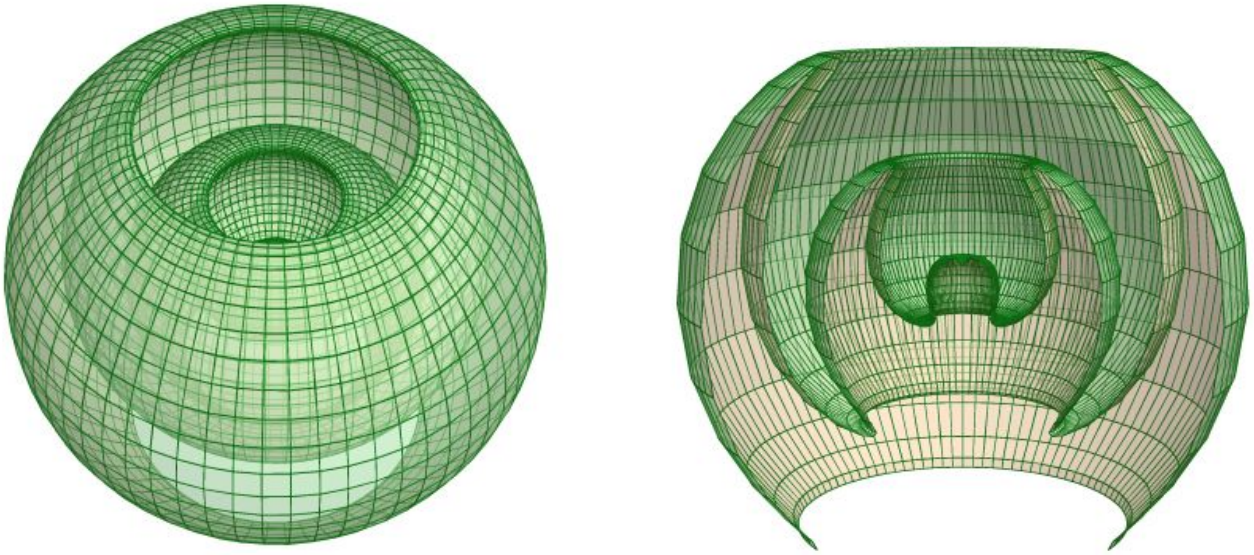
23.2.10 Wave sphere Surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D10a_WaveSphere.3D.xml (D10a-b).

The example below uses the following code in C#

```
var x = u * Math.Cos(Math.Cos(u)) * Math.Cos(v);
var y = u * Math.Cos(Math.Cos(u)) * Math.Sin(v);
var z = u * Math.Sin(Math.Cos(u));
```

// see http://www.3d-meier.de/tut3/Seite63.html



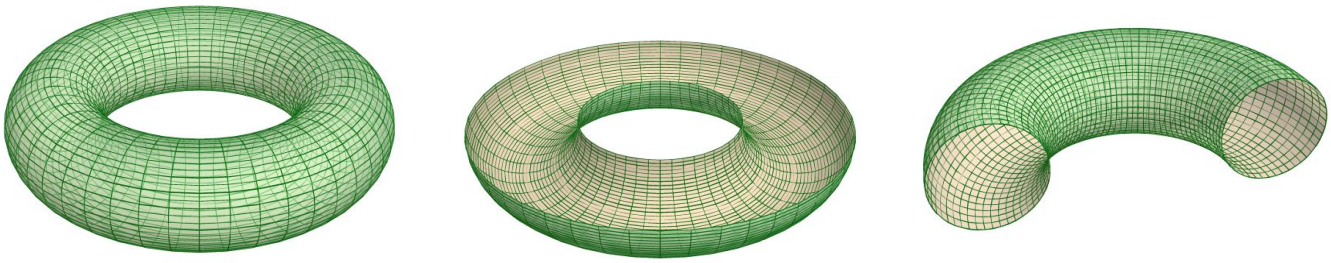
23.2.11 Torus

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D11a_Torus.3D.xml (D11a-c).

The example below uses the following code in C#

```
var R = 1.5;
var r = 0.5;
var x = (R + r * Math.Cos(v)) * Math.Cos(u);
var y = (R + r * Math.Cos(v)) * Math.Sin(u);
var z = r * Math.Sin(v);
```

See also Wikipedia [1366], MathWorld [1068].



23.2.12 Horn Torus

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D12a_HornTorus.3D.xml (D12a-c).

The example below uses the following code in C#

```
var R = 1.5;
var r = 1.5;
var x = (R + r * Math.Cos(v)) * Math.Cos(u);
var y = (R + r * Math.Cos(v)) * Math.Sin(u);
var z = r * Math.Sin(v);
```

See also Wikipedia [1366], MathWorld [1068].

23.2.13 Spindle Torus

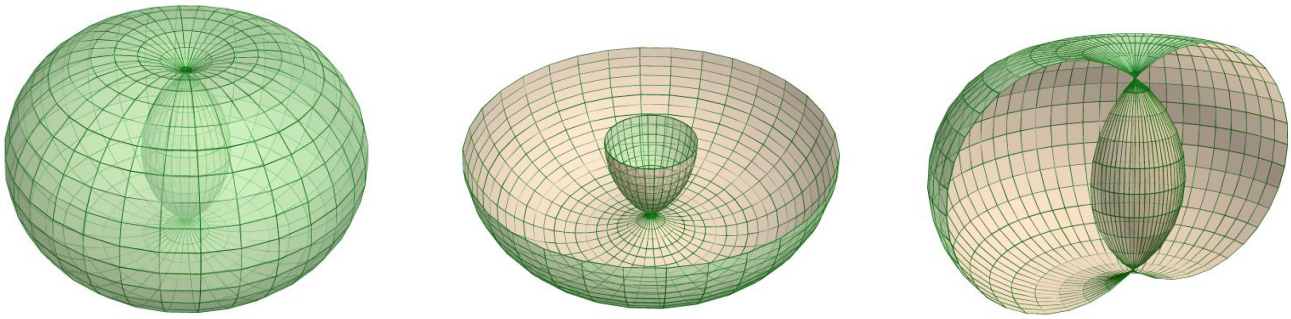
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D13a_SpindelTorus.3D.xml (D13a-c).

The example below uses the following code in C#

```
var R = 1.5;
var r = 2.5;
var x = (R + r * Math.Cos(v)) * Math.Cos(u);
var y = (R + r * Math.Cos(v)) * Math.Sin(u);
var z = r * Math.Sin(v);
```

See also: https://mathworld.wolfram.com/EllipticTorus.html

See also Wikipedia [1366], MathWorld [1068].



23.2.14 Elliptic Torus

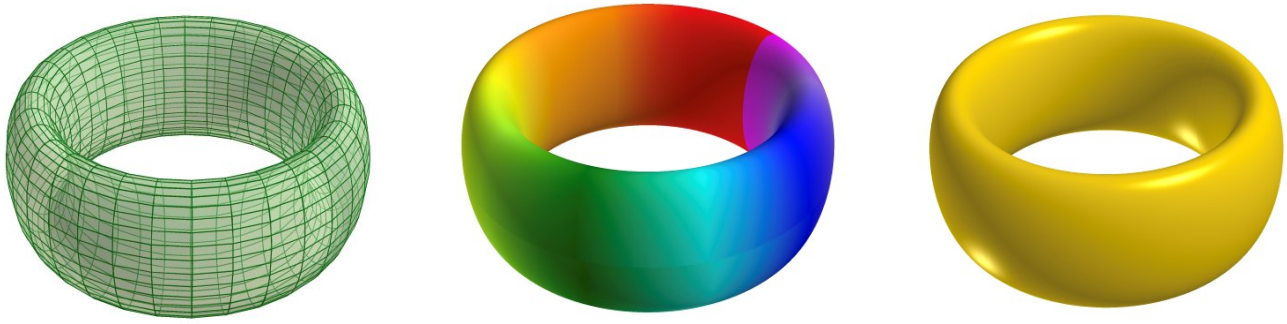
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D14a_EllipticTorus.3D.xml (D14a-c).

The example below uses the following code in C#

```
var a = 0.2;
var b = 0.5;
var c = 1;
var su = Math.Sin(u);
var sv = Math.Sin(v);
var cu = Math.Cos(u);
var cv = Math.Cos(v);

var x = (c + a * cv) * cu;
var y = (c + a * cv) * su;
var z = b * sv;
```

See also: https://mathworld.wolfram.com/EllipticTorus.html

See also Wikipedia [1366], MathWorld [1068].



23.3 Generalisations of common surfaces

Parametric Surfaces: $x = f(u, v)$, $y = g(u, v)$, $z = h(u, v)$

23.3.1 Ellipsoid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Ellipsoid.3D.xml (D01a-c).

The parametric equations of an ellipsoid can be written as

$$x(u, v) = a\cos(u)\sin(v),$$

$$y(u, v) = b\sin(u)\sin(v),$$

$$z(u, v) = r\sin(t),$$

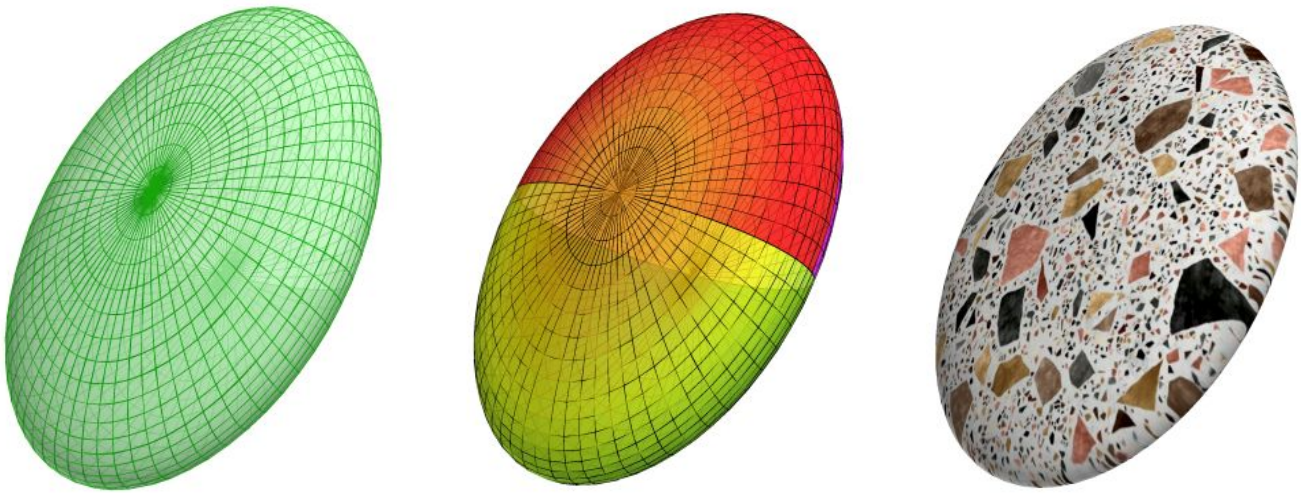
for $u \in [0, 2\pi)$ and $v \in [0, \pi]$.

See also: http://paulbourke.net/geometry/spherical/ See also: https://mathcurve.com/surfaces.gb/lame/lame.shtml See also: https://mathworld.wolfram.com/Ellipsoid.html

See also Gray *et al.* [353], Krivoshapko and Ivanov [399].

```
var a = 1.0;
var b = 2.0;
var c = 3.0;
var x = a * Math.Cos(u) * Math.Cos(v);
var y = b * Math.Cos(u) * Math.Sin(v);
var z = c * Math.Sin(u);
```

If the lengths of two axes of an ellipsoid are the same, the figure is called an ellipsoid of revolution or spheroid. Denote the equal semi-axes lengths of a spheroid $a = b$, call a the equatorial radius, and call the other semi-axis length the polar radius c. Then if $a > c$, the spheroid is called an oblate spheroid, and if $a < c$, the spheroid is called an prolate spheroid. If all three semi-axes lengths are the same so $a = b = c$, the ellipsoid is a sphere.



23.3.2 Superellipsoid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_SuperEllipse.3D.xml (D02a-c).

Superellipsoid is the name given to a family of shapes formed from the spherical product of two superquadratric curves. These shapes can be used to model a wide range of shapes including spheres, cylinders, and parallelepipeds as well as shapes in between. The parametric equations of an superellipsoid can be written as

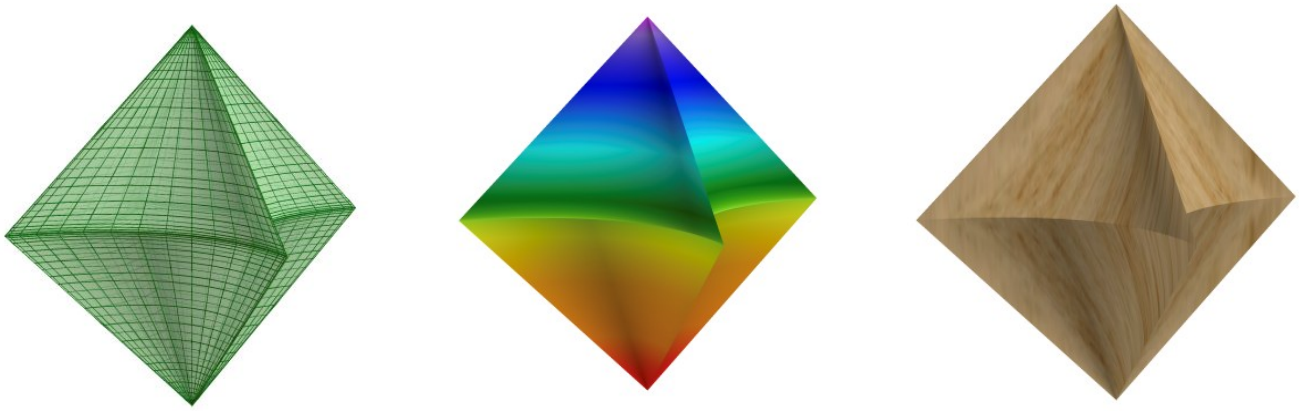
$$x(u,v) = \text{sgn}(\cos(u)) \cdot |\cos(u)|^{p1} \cdot \text{sgn}(\cos(v)) \cdot |\cos(v)|^{p2},$$

$$y(u,v) = \text{sgn}(\cos(u)) \cdot |\cos(u)|^{p1} \cdot \text{sgn}(\sin(v)) \cdot |\sin(v)|^{p2},$$

$$z(u,v) = \text{sgn}(\sin(u)) \cdot |\sin(u)|^{p1},$$

where $\frac{-\pi}{2} \leq u \leq \frac{\pi}{2}$, $-\pi \leq v \leq \pi$, and $0 < p1, p2 < \infty$.

```
double p1 = 2.0;
double p2 = 3.8;

double u = u;
double v = v;

\cos(u) = Math.Cos(u);
\cos(v) = Math.Cos(v);
\sin(u) = Math.Sin(u);
\sin(v) = Math.Sin(v);

tmp  = Math.Sign(\cos(u)) * Math.Pow(Math.Abs(\cos(u)),p1);
x = tmp * Math.Sign(\cos(v)) * Math.Pow(Math.Abs(\cos(v)),p2);
y = -Math.Sign(\sin(u)) * Math.Pow(Math.Abs(\sin(u)),p1);
z = tmp * Math.Sign(\sin(v)) * Math.Pow(Math.Abs(\sin(v)),p2);
```

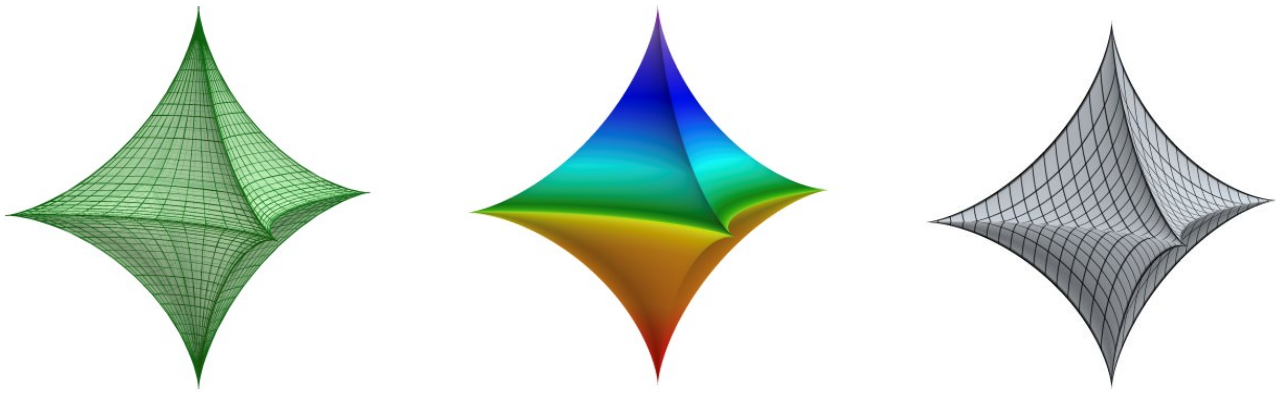


23.3.3 Hexaedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_Hexaedron.3D.xml (D03a-c).

See also Wikipedia [1366], MathWorld [1068].

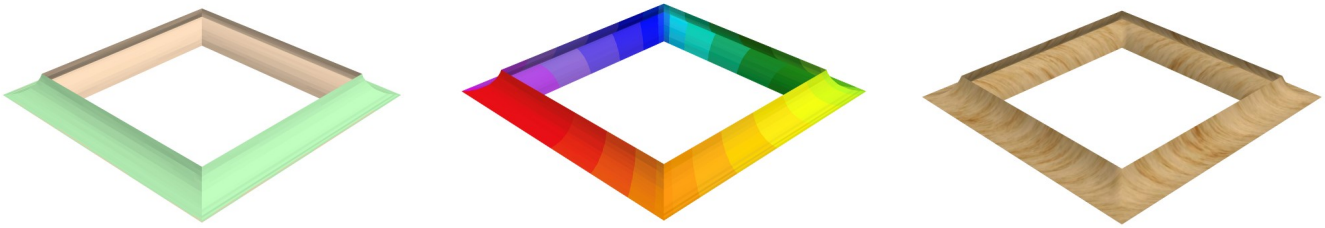
```
double cosu = Math.Cos(u);
double sinu = Math.Sin(u);
double cosv = Math.Cos(v);
double sinv = Math.Sin(v);
x = cosv * cosv * cosv * cosu * cosu * cosu;
y = -sinu * sinu * sinu;
z = sinv * sinv * sinv * cosu * cosu * cosu;
```



23.3.4 Super Toroid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_SuperToroid.3D.xml (D04a-c).

See also: http://paulbourke.net/geometry/toroidal/



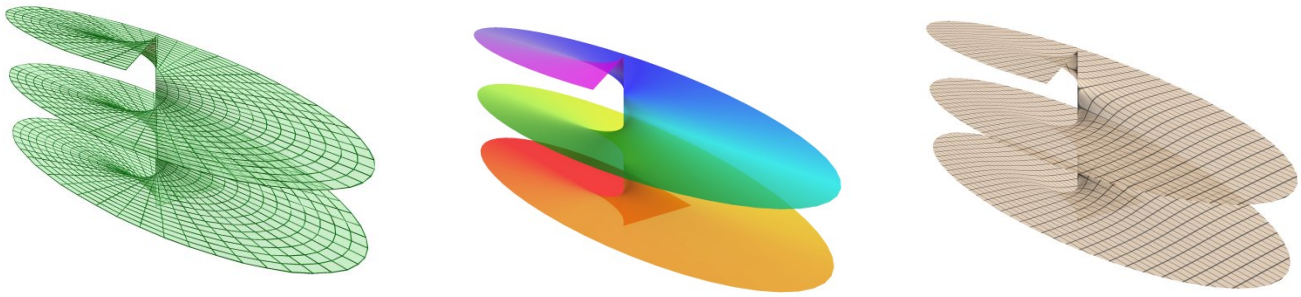
```
double r0 = 1;
double r1 = 0.3;

double tmp;
double ct1,ct2,st1,st2;

double p1 = 2.0;
double p2 = 3.8;
double t1 = u;
double t2 = v;

ct1 = Math.Cos(t1);
ct2 = Math.Cos(t2);
st1 = Math.Sin(t1);
st2 = Math.Sin(t2);

tmp  = r0 + r1 * Math.Sign(ct2) * Math.Pow(Math.Abs(ct2),p2);

x = tmp * Math.Sign(ct1) * Math.Pow(Math.Abs(ct1),p1);
y = -tmp * Math.Sign(st1) * Math.Pow(Math.Abs(st1),p1);
z = r1 * Math.Sign(st2) * Math.Pow(Math.Abs(st2),p2);
```

23.3.5 Elliptic Helicoid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_EllipticHelicoid.3D.xml (D05a-c).

See also: https://mathworld.wolfram.com/EllipticHelicoid.html

See also Wikipedia [1366], MathWorld [1068].



```
double a = 0.5;
double b = 1.5;
double c = 1;
double su = Math.Sin(u);
double cu = Math.Cos(u);

x = a * v * cu;
z = b * v * su;
y = c * u;
```

23.3.6 Hyperbolic Helicoid

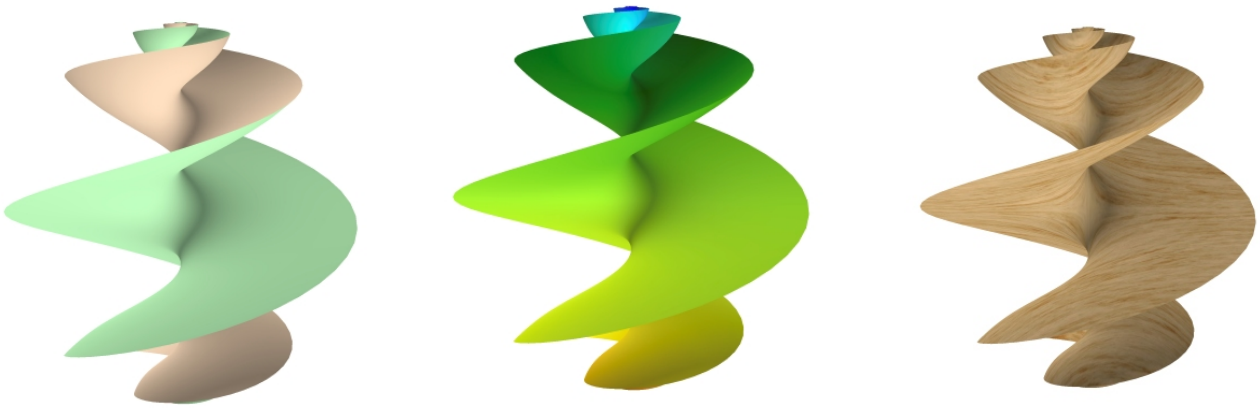
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_Hyperhelicoid.3D.xml (D06a-c).

The example below uses the following code in C#

```
var x = (Math.Sinh(v) * Math.Cos(3 * u)) / (1 + Math.Cosh(u) * Math.Cosh(v));
var y = (Math.Sinh(v) * Math.Sin(3 * u)) / (1 + Math.Cosh(u) * Math.Cosh(v));
var z = (Math.Cosh(v) * Math.Sinh(u)) / (1 + Math.Cosh(u) * Math.Cosh(v));
```

See also: https://mathworld.wolfram.com/HyperbolicHelicoid.html

See also Wikipedia [1366], MathWorld [1068].

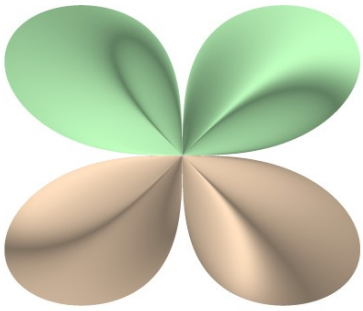
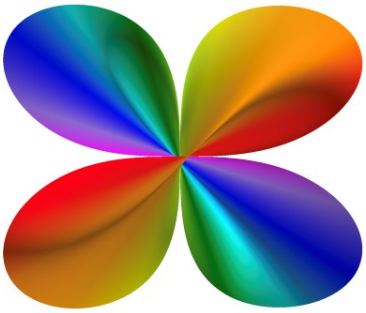
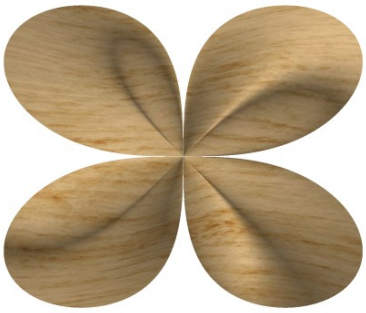


23.3.7 Lemniscate

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D07a_Lemniscate.3D.xml (D07a-c).

See also: http://paulbourke.net/geometry/lemniscape/

```
var x = Math.Cos(v) * Math.Sqrt(Math.Abs(Math.Sin(2 * u))) * Math.Cos(u);
var y = Math.Cos(v) * Math.Sqrt(Math.Abs(Math.Sin(2 * u))) * Math.Sin(u);
var z = x * x - y * y + 2 * x * y * Math.Tan(v) * Math.Tan(v);
```

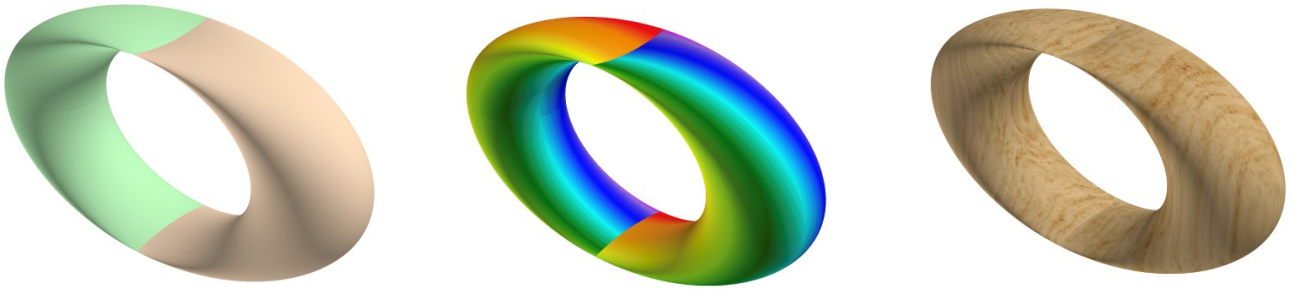
23.3.8 Bohemian Dome

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D08a_BohemianDome.3D.xml (D08a-c).

See also: https://mathworld.wolfram.com/BohemianDome.html

See also Wikipedia [1366], MathWorld [1068].

```
var a = 0.5;
var b = 1.5;
var c = 1;
var su = Math.Sin(u);
var sv = Math.Sin(v);
var cu = Math.Cos(u);
var cv = Math.Cos(v);

var x = a * cu;
var z = b * cv + a * su;
var y = c * sv;
```



23.3.9 Dupin1 surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D09a_Dupin1.3D.xml (D09a-c).

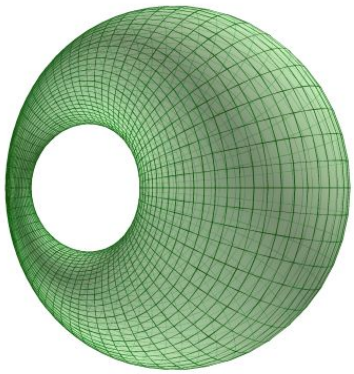
The example below uses the following code in C#

```
var a = 1.5;
var b = 1.4;
var c = Math.Sqrt(a * a - b * b);
var d = b / 2;
var su = Math.Sin(u);
var sv = Math.Sin(v);
var cu = Math.Cos(u);
var cv = Math.Cos(v);
var den = a - c * cu * cv;

var x = (b * sv * (c * cu - d)) / den;
var y = (d * (c - a * cu * cv) + b * b * cu) / den;
var z = (b * su * (a - d * cv)) / den;
```

See also: https://mathcurve.com/surfaces.gb/cyclidedupin/cyclidededupin.shtml

See also: https://en.wikipedia.org/wiki/Dupin_cyclide#Elliptic_cyclides

See also Wikipedia [1366], MathWorld [1068].



23.3.10 Dupin2 surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D10a_Dupin2.3D.xml (D10a-c).

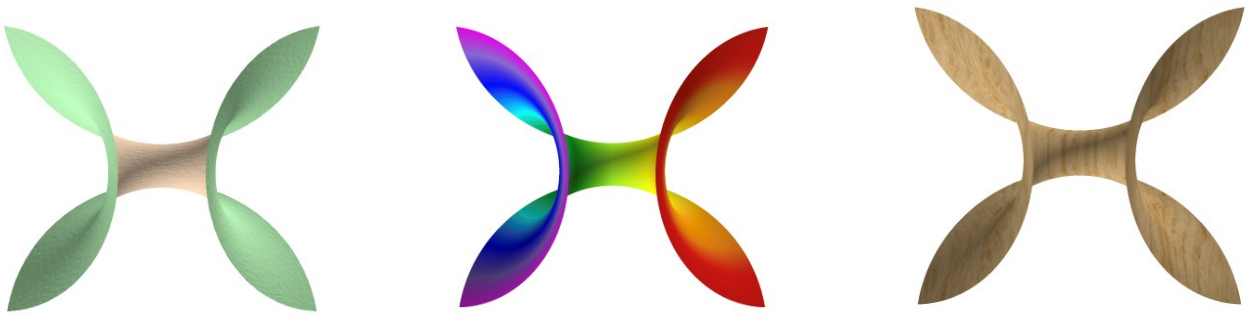
The example below uses the following code in C#

```
var p = 2;
var k = 0.7;
var den = 1 + u * u + v * v;
var x = 0.5 * p * (2 * v * v + k * (1 - u * u - v * v)) / den;
var y = p * u * (v * v + k) / den;
var z = p * v * (1 + u * u - k) / den;
```

See also: https://mathcurve.com/surfaces.gb/cycliddedupin/cyclidededupin.shtml

See also: https://en.wikipedia.org/wiki/Dupin_cyclide#Parabolic_cyclides

See also Wikipedia [1366], MathWorld [1068].



23.3.11 Dinis Surface (twisted pseudosphere)

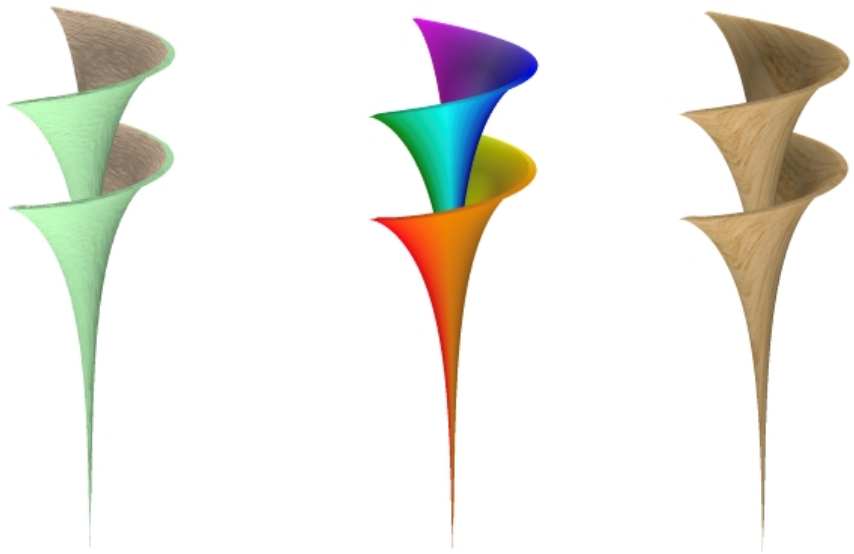
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D11a_Dini.3D.xml (D11a-c).

The example below uses the following code in C#

```
var a = 1;
var b = 0.2;
var x = a * Math.Cos(u) * Math.Sin(v);
var y = a * Math.Sin(u) * Math.Sin(v);
var z = a * (Math.Cos(v) + Math.Log(Math.Tan(0.5 * v))) + b * u;
```

See also: https://mathworld.wolfram.com/DinisSurface.html

See also Wikipedia [1366], MathWorld [1068].

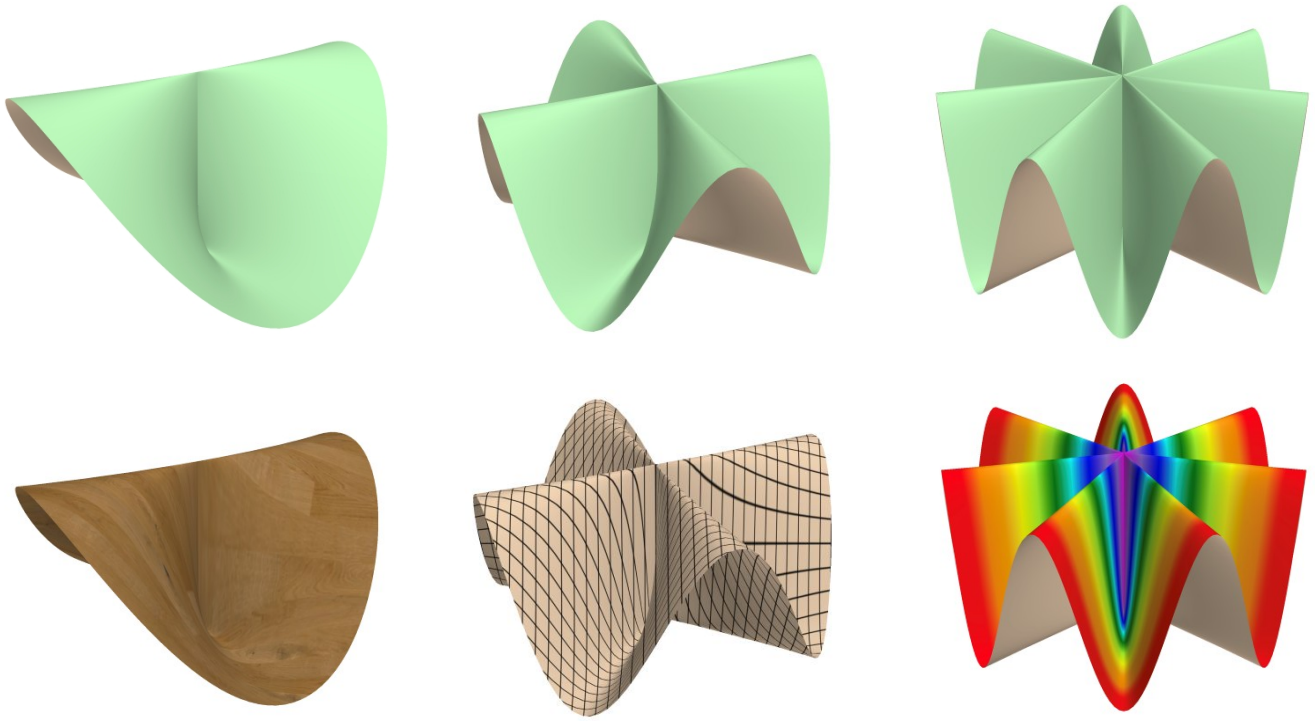


23.3.12 Plueckers conoid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D12a_Pluecker.3D.xml (D12a-f).

// See : https://mathworld.wolfram.com/PlueckersConoid.html // See : Gray, p. 436 // See : https://en.wikipedia.org/wiki/Pl%C3%BCcker%27s_conoid

```
var n = 2;
var r = u;
var theta = v;

var x = r * Math.Cos(theta);
var y = r * Math.Sin(theta);
var z = Math.Sin(n * theta);
```



References

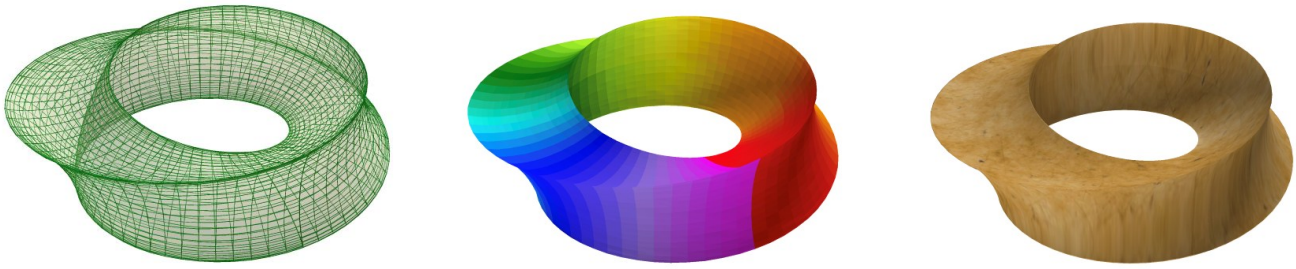
Gray, A. “Plücker’s Conoid.” Modern Differential Geometry of Curves and Surfaces with Mathematica, 2nd ed. Boca Raton, FL: CRC Press, pp. 435-437, 1997.

23.3.13 Umbilic Torus

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D13a_UmbilicTorus.3D.xml (D13a-c).

```
var x = Math.Cos(u) * (7 + Math.Cos(u / 3 - 2 * v) + 2 * Math.Cos(u / 3 + v));
var y = Math.Sin(u) * (7 + Math.Cos(u / 3 - 2 * v) + 2 * Math.Cos(u / 3 + v));
var z = Math.Sin(u / 3 - 2 * v) + 2 * Math.Sin(u / 3 + v);
```

// See also: http://www.3d-meier.de/tut3/Seite61.html // Umbilic Torus

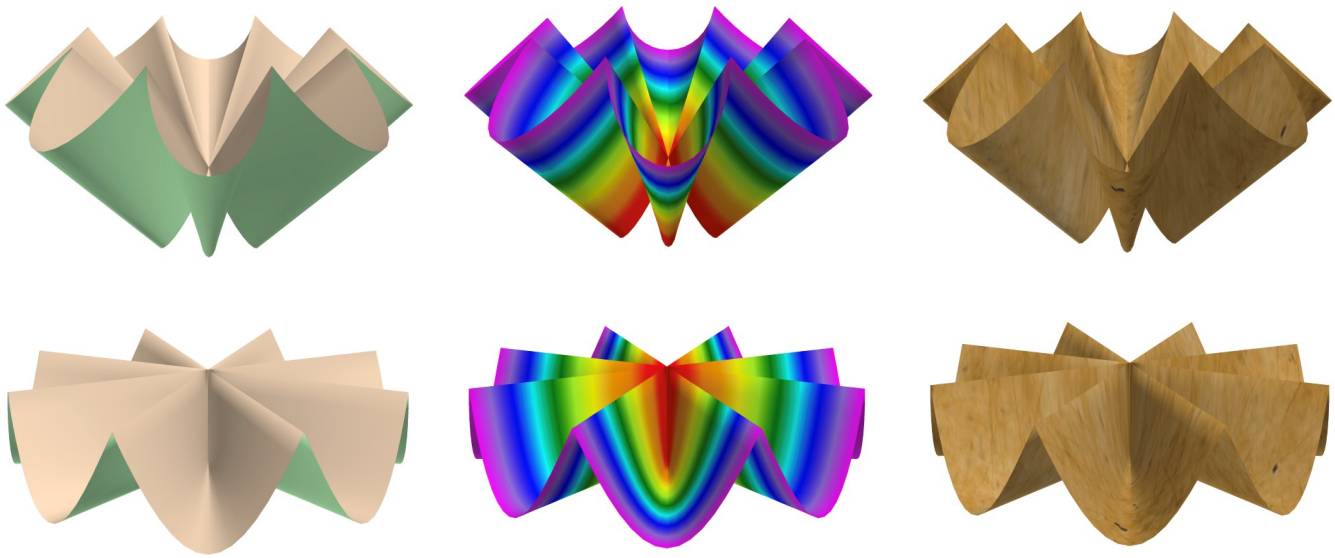


23.3.14 Skidian's ruled surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D14a_Skidan.3D.xml (D14a-f).

```
var a = Math.PI / 4.0;
var h = 2.0;
var n = 4.0;

var b = h * Math.Abs(Math.Cos(n * v));
var ca = Math.Cos(a);
var sa = Math.Sin(a);

var x = (u * sa + b * ca) * Math.Sin(v);
var y = (u * sa + b * ca) * Math.Cos(v);
var z = (u * ca - b * sa);
```

// See also: http://www.3d-meier.de/tut3/Seite227.html // Skidan Ruled Surface // See Krivoshapko, p. 499

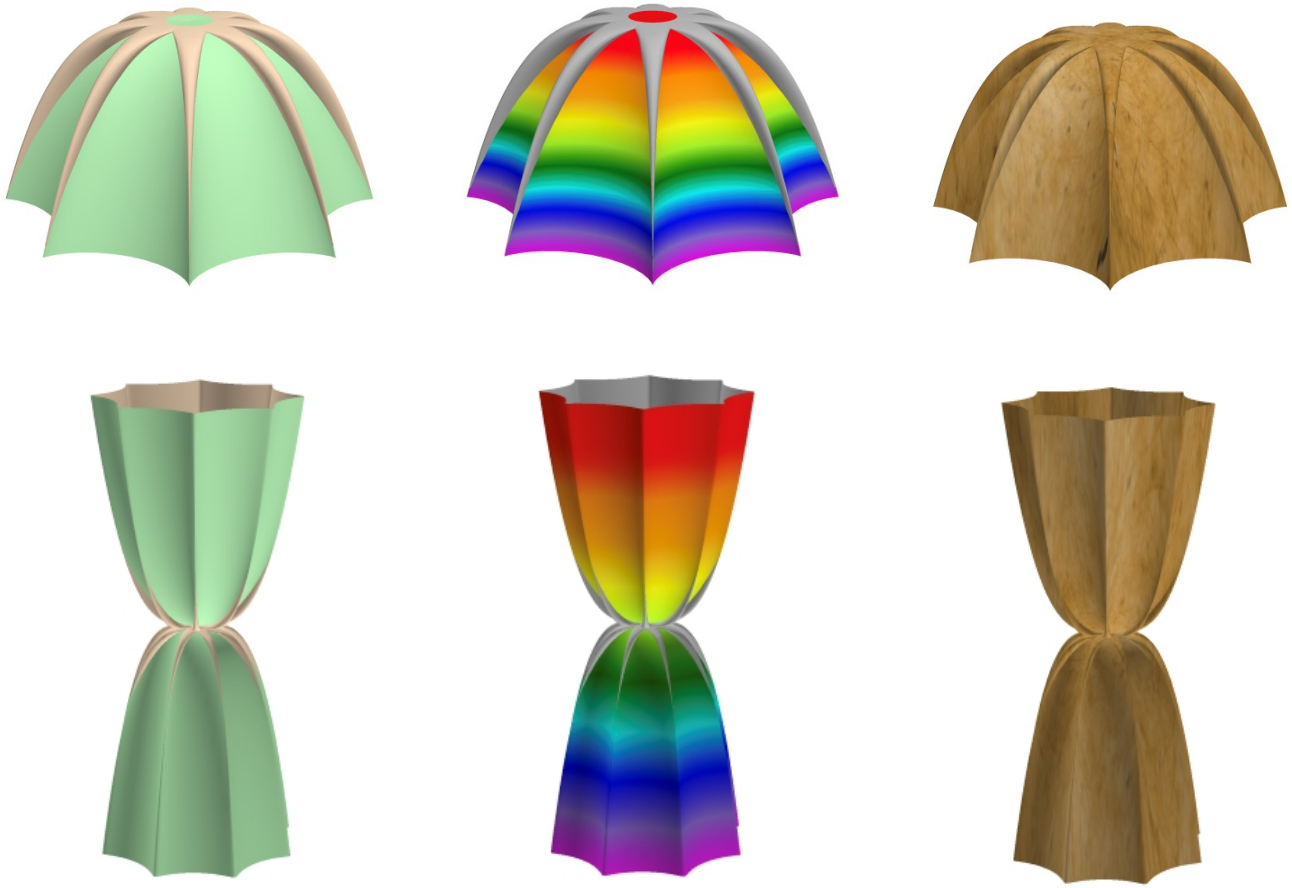


23.3.15 Umbrella surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D15a_Umbrella.3D.xml (D15a-f).

```
var R = 0.6;
var h = 0.6;
var n = 8.0;

var r = R / n;
var x = ctx.cbrt(u) * ((R - r)) * Math.Cos(v) + r * Math.Cos((n - 1) * v);
var y = ctx.cbrt(u) * ((R - r)) * Math.Sin(v) - r * Math.Sin((n - 1) * v);
var z = h * (1 - u);
```

// See also: http://www.3d-meier.de/tut3/Seite215.html // Umbrella Surface // See Krivoshapko, p. 507 - 509 // See Krivoshapko, p. 513 - 515 // See Krivoshapko, p. 521, 526, 530, 531, 533

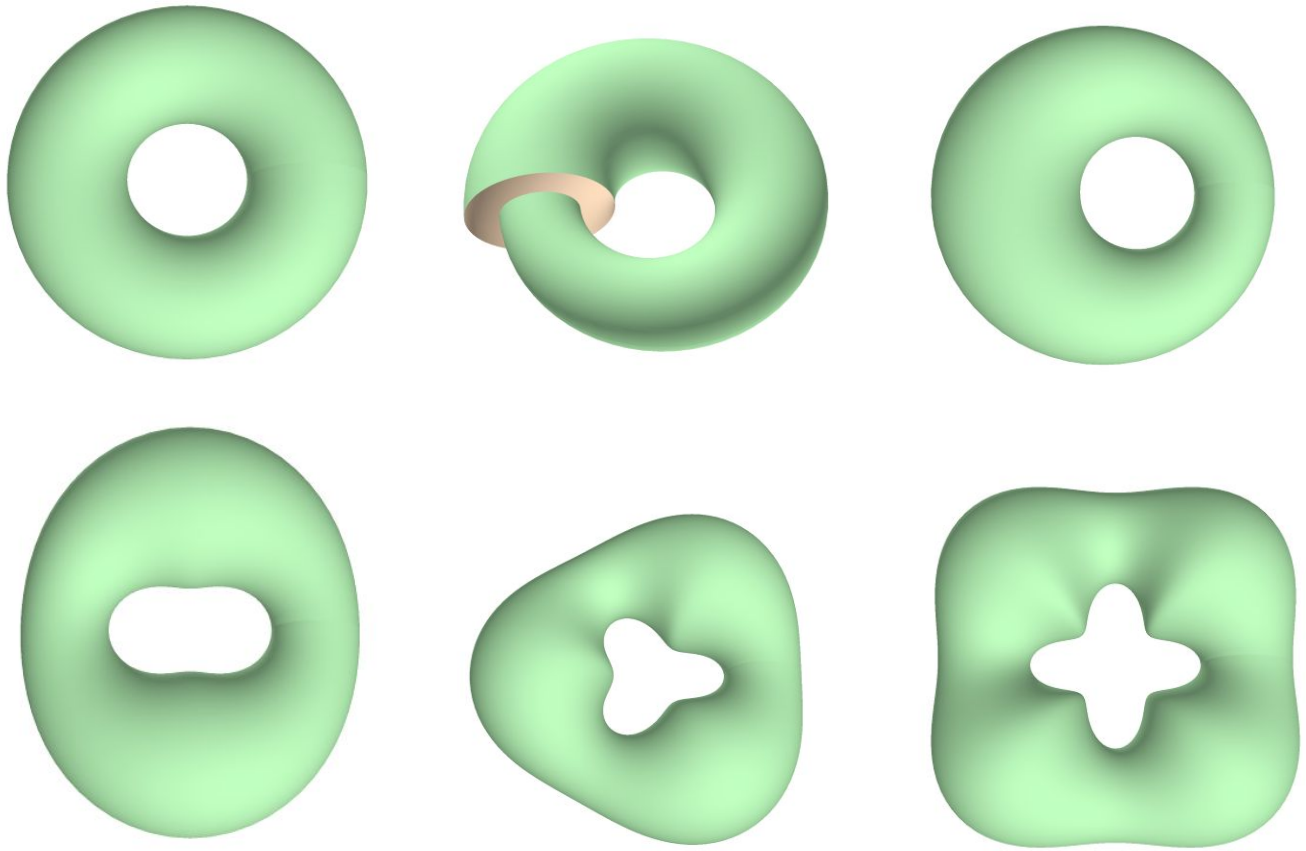


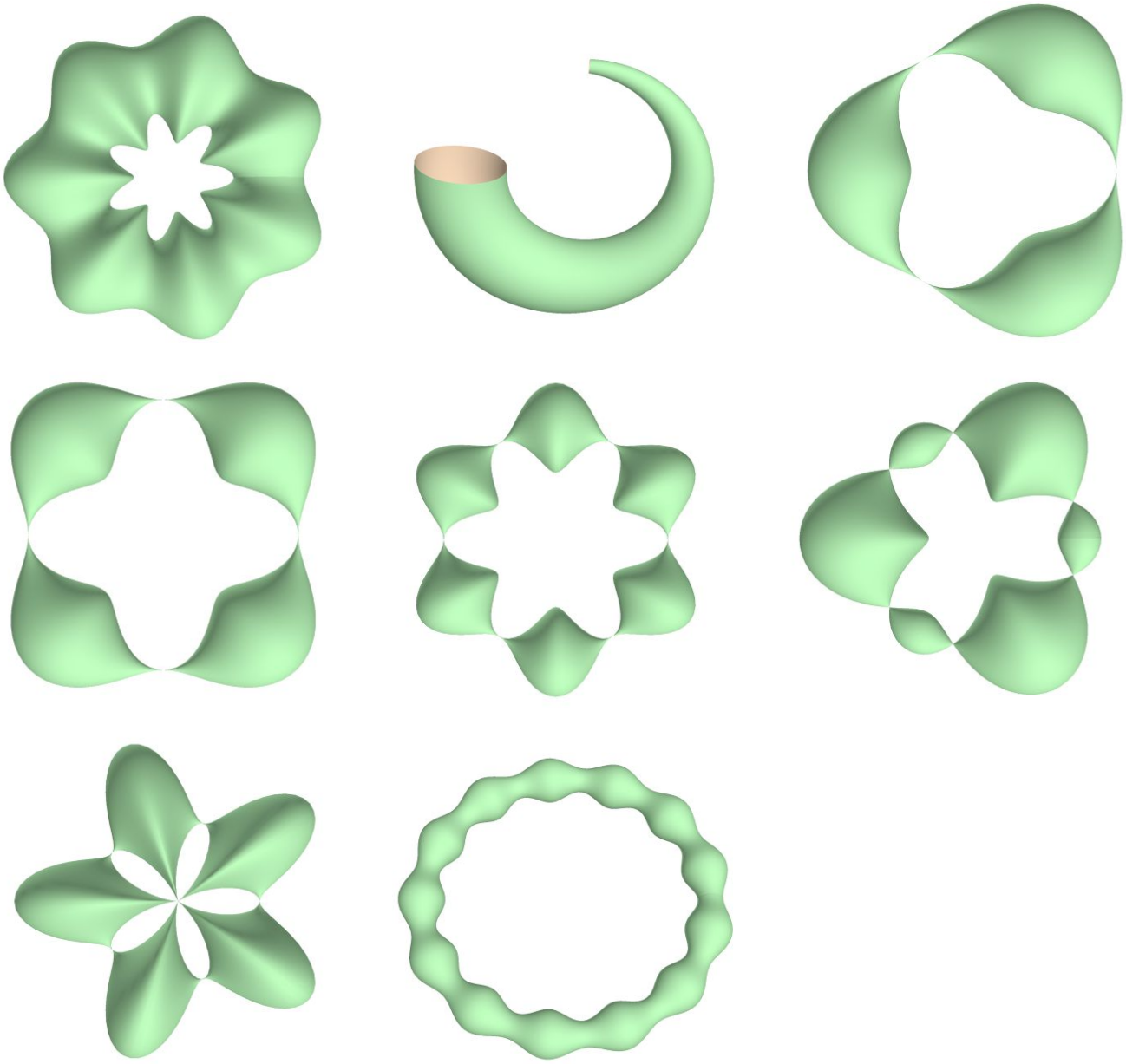
23.3.16 Cyclic surfaces (generalized torus)

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D16a_CyclicSurface1.3D.xml (D16a-n).

```
var a = 1.0;
var b = 2.0;
var d = 0.0;
var p = 1.0;

var R = a * (1 - d * Math.Cos(p * u));
var x = (b + R * Math.Cos(v)) * Math.Cos(u);
var y = (b + R * Math.Cos(v)) * Math.Sin(u);
var z = R * Math.Sin(v);
```

// See Krivoshapko, p. 376





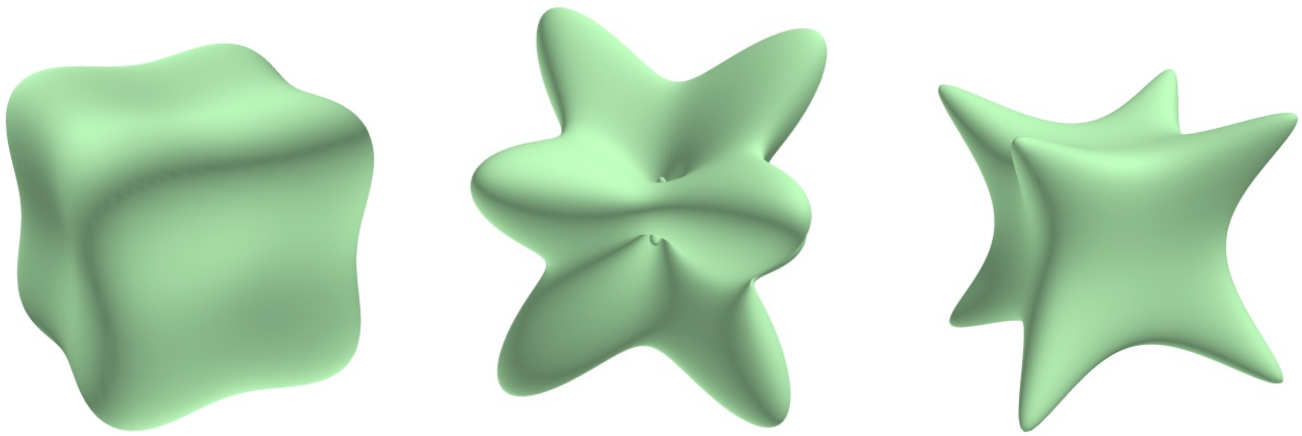
23.3.17 Goursat surfaces

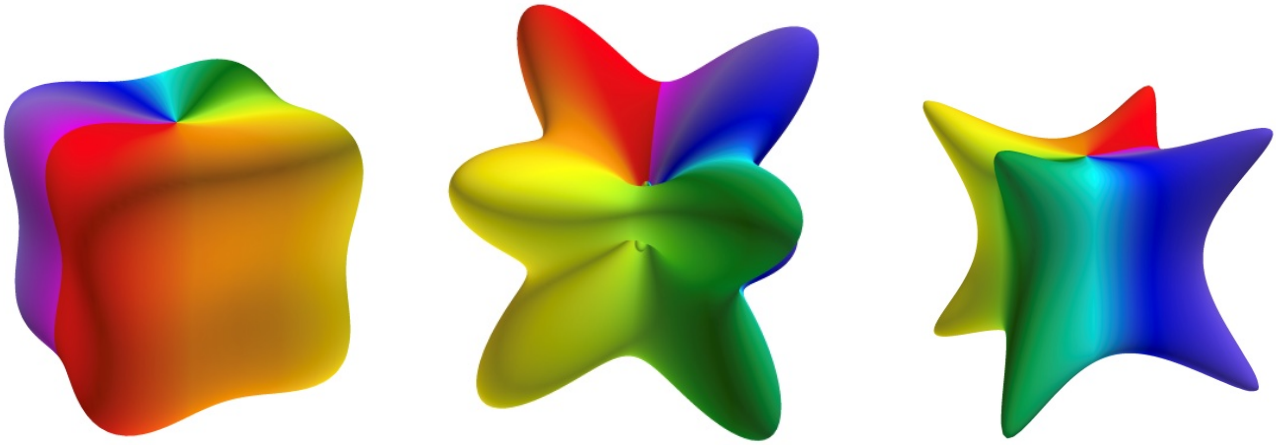
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D17a_Goursat1.3D.xml (D17a-f).

The example below uses the following code in C#

```
var a = -0.33;
var b = 0.2;
var c = -3;
var p = 1;

var su = Math.Sin(u);
var cu = Math.Cos(u);
var sv = Math.Sin(v);
var cv = Math.Cos(v);

var su4 = su * su * su * su;
var cu4 = cu * cu * cu * cu;

var sv4 = sv * sv * sv * sv;
var cv4 = cv * cv * cv * cv;

var D = (su4 + cu4) * cv4 + sv4;
var R = Math.Sqrt((-b + p * Math.Sqrt(b * b - 4 * c * (a + D))) / (2 * (a + D)));
var x = R * cv * cu;
var y = R * cv * su;
var z = R * sv;
```

// See : Krivoshapko (2015), p. 643 // See: https://mathworld.wolfram.com/GoursatsSurface.html // See Gray 1997, p. 314) // See: https://mathcurve.com/surfaces.gb/goursat/goursat.shtml





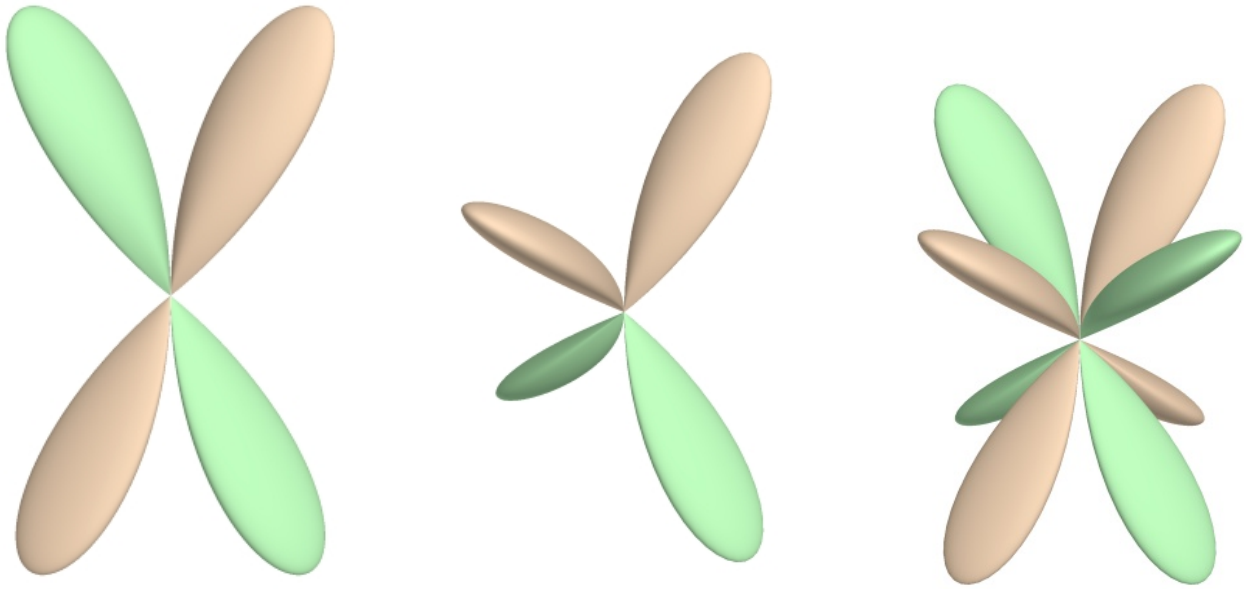
23.3.18 Cyclides triples

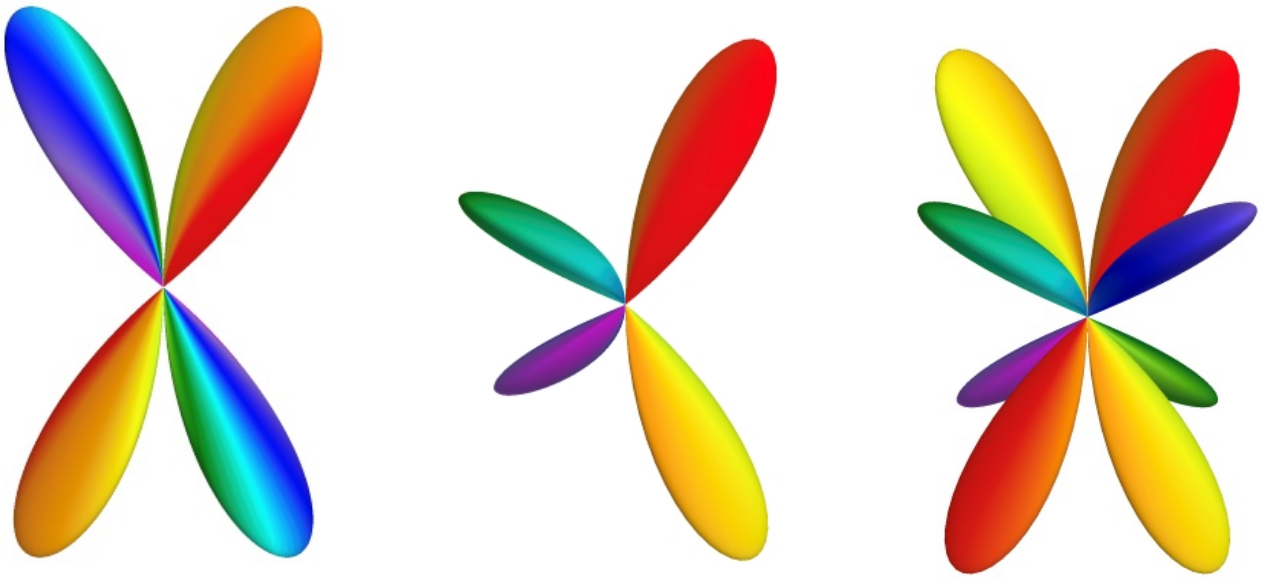
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D18a_CyclidesTriple.3D.xml (D18a-f).

The example below uses the following code in C#

```
var su = Math.Sin(u);
var s2u = Math.Sin(2 * u);
var cu = Math.Cos(u);
var sv = Math.Sin(v);
var cv = Math.Cos(v);

var x = cv * sv * sv * sv * s2u * cu;
var y = cv * sv * sv * sv * s2u * su;
var z = cv * sv * sv * s2u;
```

// See : Krivoshapko (2015), p. 651





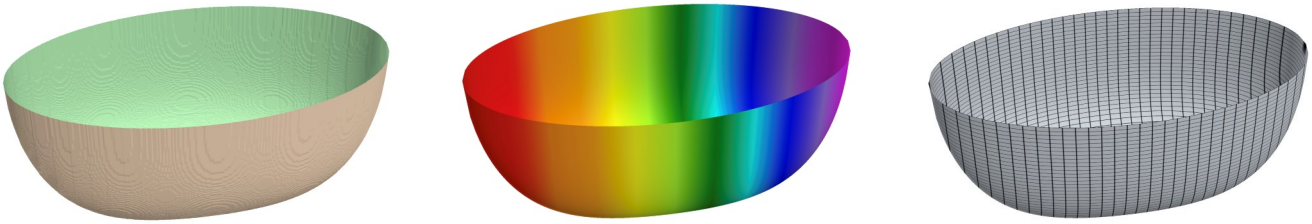
23.3.19 Ship Lamé

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D19a_ShipLame.3D.xml (D19a-c).

The example below uses the following code in C#

```
var B = 6.5;
var L = 4.0;
var T = 2.0;

var x = u * L;
var y = Math.Sign(Math.Cos(v)) * B * Math.Sqrt(1 - u * u * u * u) * Math.Sqrt(Math.Abs(Math.Cos(v))) / 2.0;
var z = Math.Sign(Math.Sin(v)) * T * Math.Sqrt(1 - u * u) * Math.Sqrt(Math.Abs(Math.Sin(v)));
```

// See : Krivoshapko (2015), p. 671



23.4 Minimal surfaces

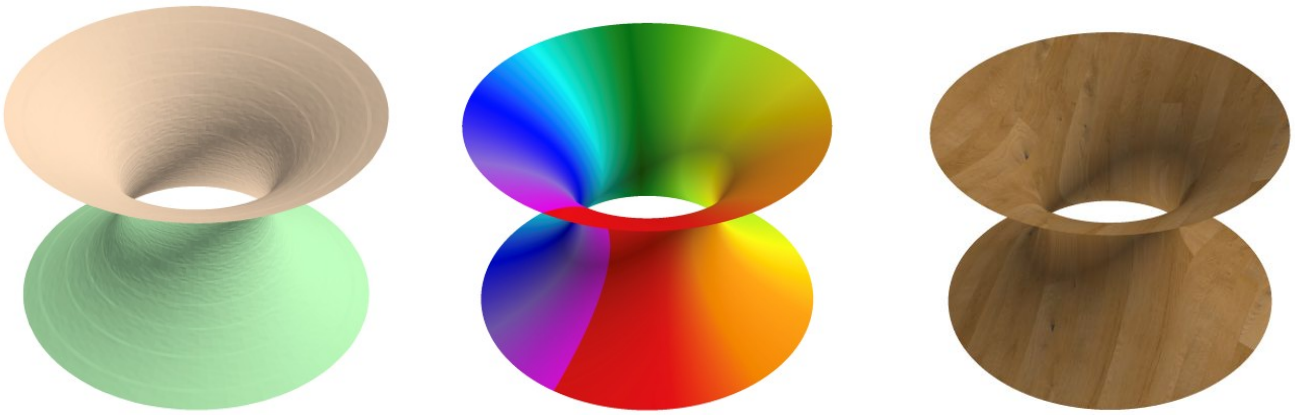
23.4.1 Catenoid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Catenoid.3D.xml (D01a-c).

The example below uses the following code in C#

```
var c = 2;
var x = c * Math.Cosh(v / c) * Math.Cos(u);
var y = c * Math.Cosh(v / c) * Math.Sin(u);
var z = v;
```

See also: https://mathworld.wolfram.com/Catenoid.html



23.4.2 Helicoid

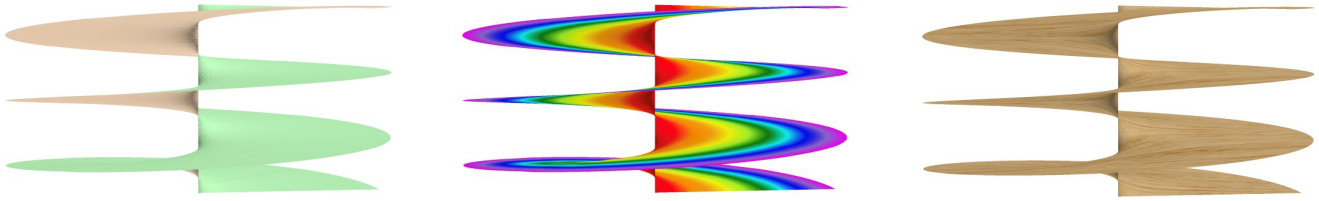
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Helicoid.3D.xml (D02a-c).

The example below uses the following code in C#

```
var x = u * Math.Cos(v);
var y = u * Math.Sin(v);
var z = v;
```

See also: https://mathworld.wolfram.com/Helicoid.html

See also Wikipedia [1366], MathWorld [1068].



References

Gray, A. Modern Differential Geometry of Curves and Surfaces with Mathematica, 2nd ed. Boca Raton, FL: CRC Press, pp. 449 and 644, 1997.

23.4.3 Bours minimal surface

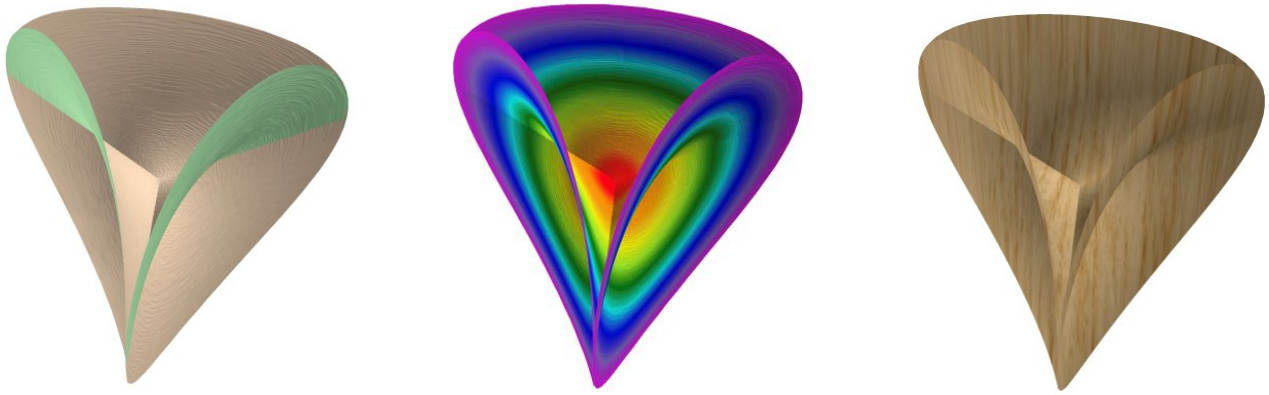
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_Bour.3D.xml (D03a-c).

The example below uses the following code in C#

```
var sv = Math.Sin(v);
var s2v = Math.Sin(2 * v);
var c32u = Math.Cos(1.5 * v);
var cv = Math.Cos(v);
var c2v = Math.Cos(2 * v);
var u2 = 0.5 * u * u;
var u32 = (4.0 / 3.0) * Math.Sqrt(u * u * u);

var x = u * cv - u2 * c2v;
var y = -u * sv - u2 * s2v;
var z = u32 * c32u;
```

See also: https://mathworld.wolfram.com/BoursMinimalSurface.html

See also Wikipedia [1366], MathWorld [1068].



23.4.4 Catalan minimal surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Catalan.3D.xml (D04a-c).

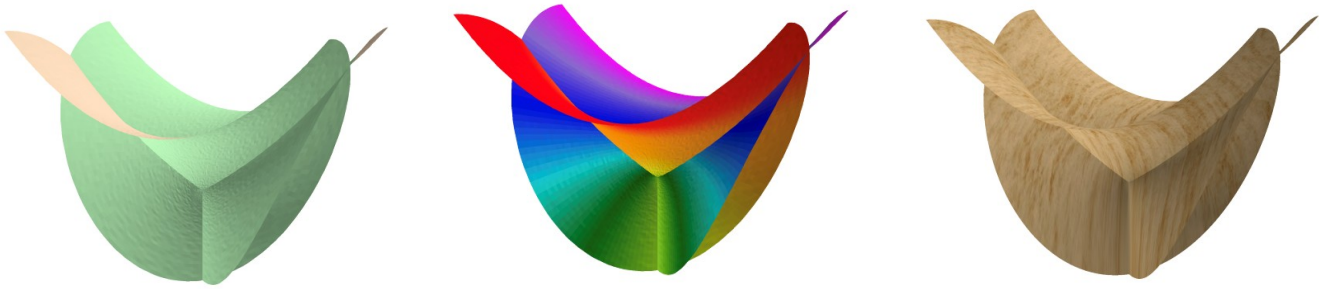
The example below uses the following code in C#

```
var x = u - Math.Sin(u) * Math.Cosh(v);
var y = 4 * Math.Sin(u / 2) * Math.Sinh(v / 2);
var z = 1 - Math.Cos(u) * Math.Cosh(v);
```

See also: https://mathworld.wolfram.com/CatalanMinimalSurface.html

See also: https://en.wikipedia.org/wiki/Catalan%27s_minimal_surface

See also Wikipedia [1366], MathWorld [1068].



23.4.5 Ennepers first minimal surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Enneper.3D.xml (D05a-c).

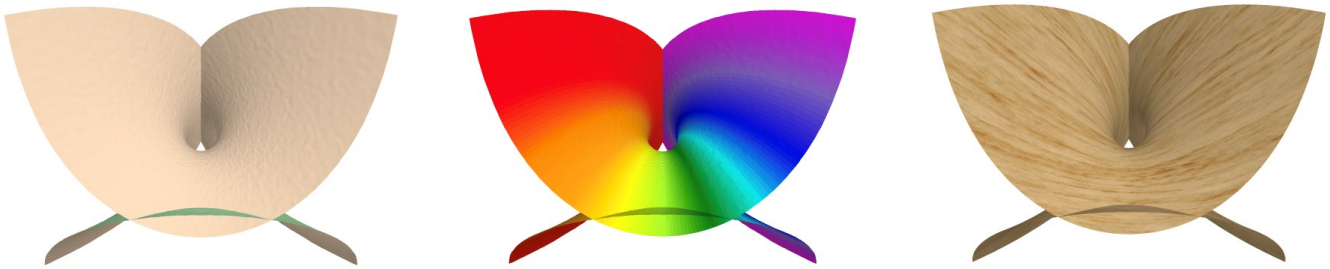
The example below uses the following code in C#

```
var x = u - u * u * u / 3 + u * v * v;
var y = u * u - v * v;
var z = v - v * v * v / 3 + v * u * u;
```

See also: https://mathworld.wolfram.com/EnnepersMinimalSurface.html

See also: https://en.wikipedia.org/wiki/Enneper_surface

See also Wikipedia [1366], MathWorld [1068].



23.4.6 Enneper second minimal surface

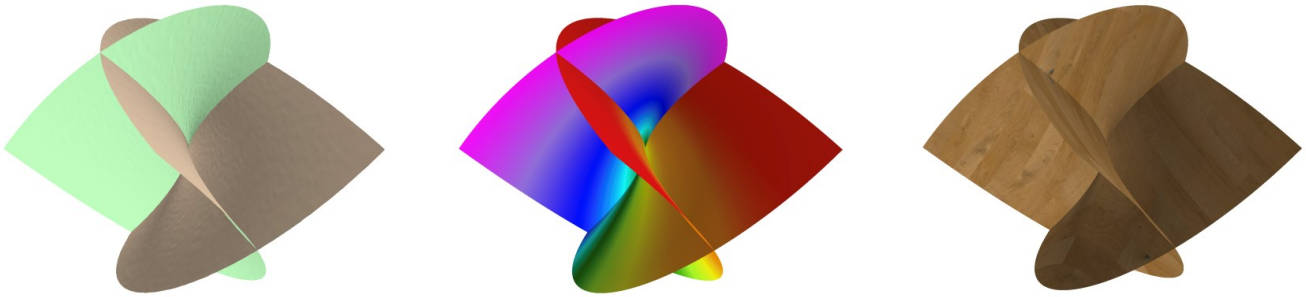
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_Enneper2.3D.xml (D06a-c).

The example below uses the following code in C#

```
var n = 2;
var a = 1;

Complex i = Complex.ImaginaryOne;
Complex w = new Complex(u, v);
Complex w2nm1 = Complex.Pow(w, 2 * n - 1) / (2 * n - 1);
Complex wn = Complex.Pow(w, n);

var x = a * (w - w2nm1).Real;
var y = a * (-i * (w + w2nm1)).Real;
var z = 2 * a * (wn / n).Real;
```

See also: https://mathcurve.com/surfaces.gb/enneper/enneper.shtml

See also Wikipedia [1366], MathWorld [1068].



23.4.7 Hennebergs minimal surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D07a_Henneberg.3D.xml (D07a-c).

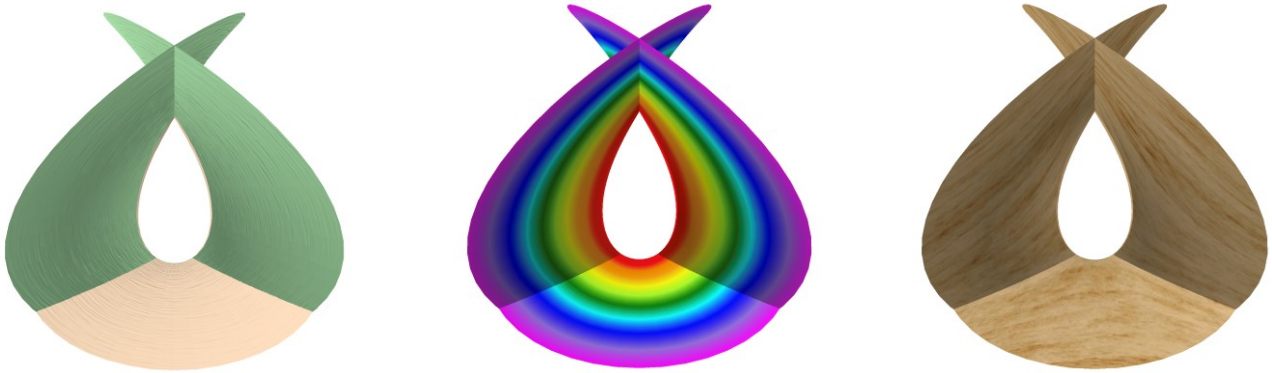
The example below uses the following code in C#

```
var sv = Math.Sin(v);
var s3v = Math.Sin(3 * v);
var cv = Math.Cos(v);
var c2v = Math.Cos(2 * v);
var c3v = Math.Cos(3 * v);
var shu = Math.Sinh(u);
var ch2u = Math.Cosh(2 * u);
var sh3u = Math.Sinh(3 * u);

var x = 2 * shu * cv - (2.0 / 3.0) * sh3u * c3v;
var y = 2 * shu * sv + (2.0 / 3.0) * sh3u * s3v;
var z = 2 * ch2u * c2v;
```

See also: https://mathworld.wolfram.com/HennebergsMinimalSurface.html

See also: https://en.wikipedia.org/wiki/Henneberg_surface

See also Wikipedia [1366], MathWorld [1068].



23.4.8 Scherk's first minimal surface

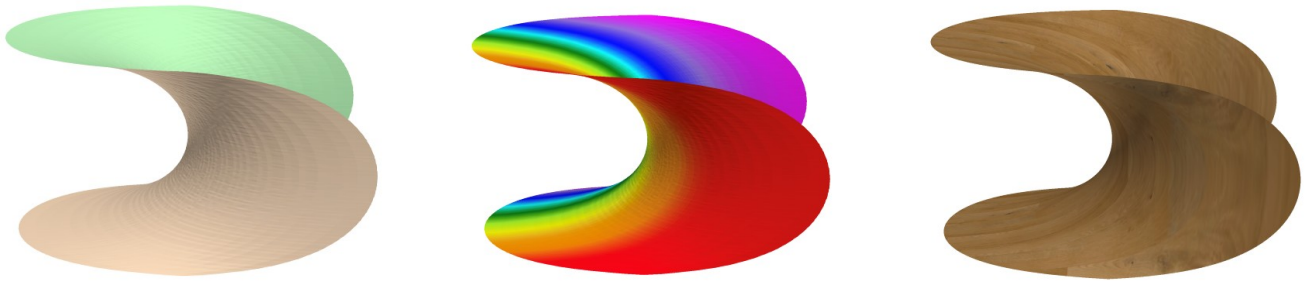
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D08a_Scherk1.3D.xml (D08a-c).

The example below uses the following code in C#

```
var a = 1 / Math.PI;
var y = u;
var x = a * Math.Log(Math.Cos(v / a) / Math.Cos(u / a));
var z = v;
```

See also Wikipedia [1366], MathWorld [1068].

See also https://mathcurve.com/surfaces.gb/scherk/scherk.shtml See also Krivoshapko and Ivanov [399], p. 431



23.4.9 Scherk's second minimal surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D09a_Scherk2.3D.xml (D09a-c).

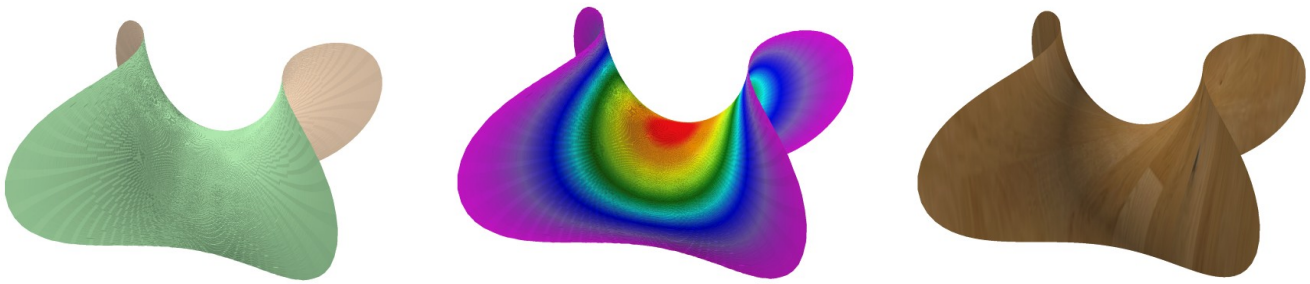
The example below uses the following code in C#

```
var t = v;
var r = u;
var r2 = r * r;
var ct = Math.Cos(t);
var st = Math.Sin(t);
var x = Math.Log((1 + r2 + 2 * r * ct) / (1 + r2 - 2 * r * ct));
var y = Math.Log((1 + r2 - 2 * r * st) / (1 + r2 + 2 * r * st));
var z = 2 * Math.Atan((2 * r2 * Math.Sin(2 * t)) / (r2 * r2 - 1));
y = -y;
```

See also Wikipedia [1366], MathWorld [1068].

See also https://mathworld.wolfram.com/ScherksMinimalSurfaces.html

See also Krivoshapko and Ivanov [399], p. 442



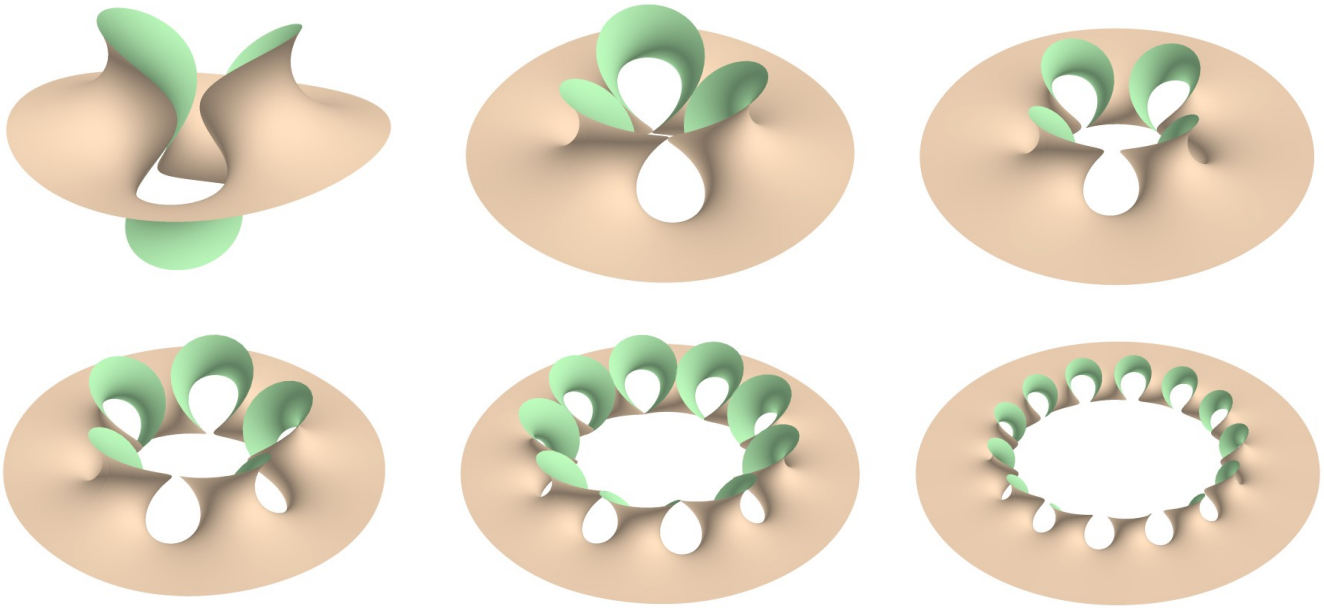
23.4.10 Richmond minimal surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D10a_Richmond.3D.xml (D10a-f).

The example below uses the following code in C#

```
var n = 2.0;
var u2s = Math.Pow(u, 2 * n + 1) / (4 * n + 2);
var x = -Math.Cos(v) / (2 * u) - u2s * Math.Cos(-(2 * n + 1) * v);
var y = -Math.Sin(v) / (2 * u) + u2s * Math.Sin(-(2 * n + 1) * v);
var z = Math.Pow(u, n) * Math.Cos(n * v) / n;
```

// See also: http://www.3d-meier.de/tut3/Seite250.html // Richmond Surface III // See also: https://en.wikipedia.org/wiki/Richmond_surface



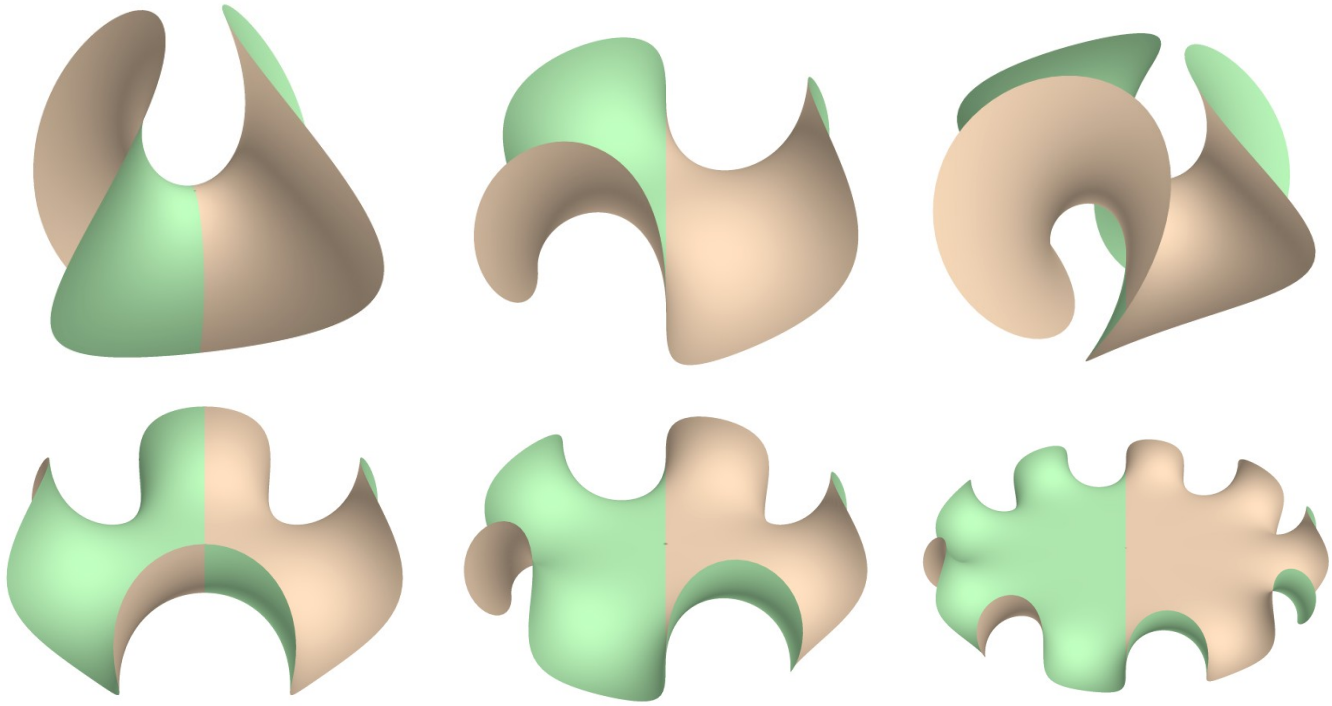
23.4.11 Generalized Ennneper surfaces

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D11a_GenEnneper.3D.xml (D11a-f).

The example below uses the following code in C#

```
var s = 2.0;
var u2s = Math.Pow(u, 2 * s - 1) / (2 * s - 1);
var x = u * Math.Cos(v) - u2s * Math.Cos((2 * s - 1) * v);
var y = -u * Math.Sin(v) - u2s * Math.Sin((2 * s - 1) * v);
var z = 2 * Math.Pow(u, s) * Math.Cos(s * v) / s;
```

// See also: http://www.3d-meier.de/tut3/Seite247.html // Wavy Enneper Surface



23.5 Nonorientable (one-sided) Surfaces

23.5.1 Moebius Strip

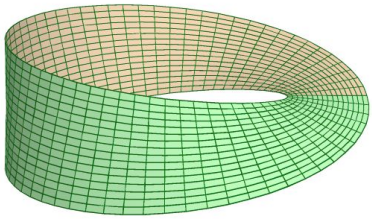
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Moebius.3D.xml (D01a-c).

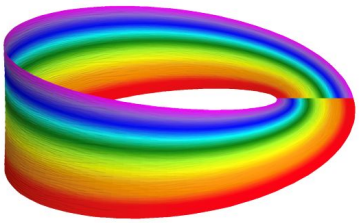
The example below uses the following code in C#

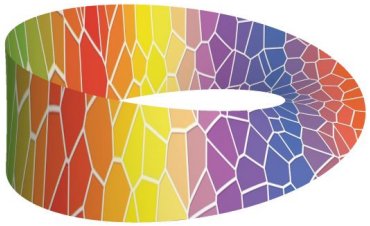
```
var a = 1.0;
var b = 1.0;
int m = 1;

var x = -(a + u * Math.Sin(m * v / 2)) * Math.Cos(v);
var y = (b + u * Math.Sin(m * v / 2)) * Math.Sin(v);
var z = 3 * (u / 2) * Math.Cos(v / 2);
```

See also: http://en.wikipedia.org/wiki/M%C3%B6bius_strip

See also: https://mathworld.wolfram.com/MoebiusStrip.html







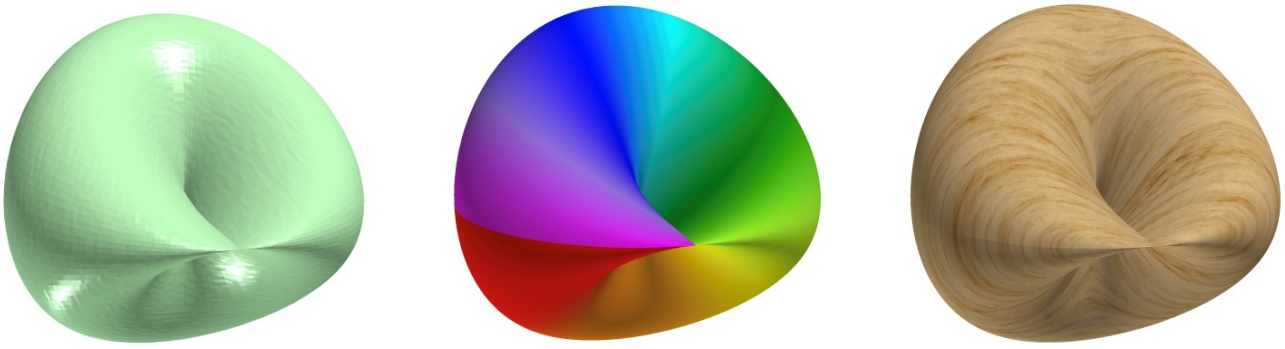
23.5.2 Cross-Cap Surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_CrossCap.3D.xml (D02a-c).

See also: https://mathworld.wolfram.com/Cross-Cap.html

See also Wikipedia [1366], MathWorld [1068].

```
double su = Math.Sin(u);
double sv = Math.Sin(v);
double s2v = Math.Sin(2*v);
double cu = Math.Cos(u);
double cv = Math.Cos(v);
x = 0.5 * cu * s2v;
z = 0.5 * su * s2v;
y = 0.5 * (cv*cv - cu*cu * sv*sv);
```



23.5.3 Pseudo Cross-Cap Surface

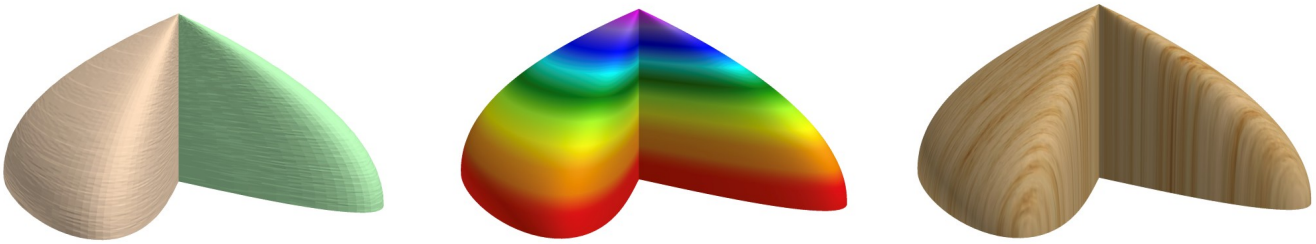
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_PseudoCrossCap.3D.xml (D03a-c).

See also: https://mathworld.wolfram.com/Pseudocrosscap.html

See also: http://www.3d-meier.de/tut3/Seite51.html

See also Wikipedia [1366], MathWorld [1068].

```
double sv = Math.Sin(v);
double s2v = Math.Sin(2*v);
x = (1 - u*u) * sv;
y = (1 - u*u) * s2v;
z = u;
```



23.5.4 Roman Surface (or Steiner Surface)

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Roman.3D.xml (D04a-c).

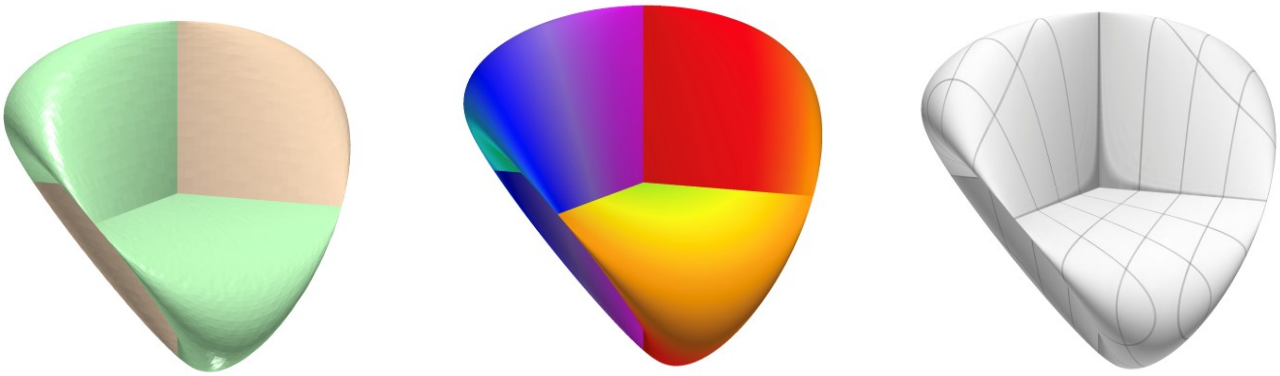
See also: https://mathworld.wolfram.com/RomanSurface.html

See also: https://en.wikipedia.org/wiki/Roman_surface

See also: http://paulbourke.net/geometry/steiner/

```
double r2 = 1;
double su = Math.Sin(u);
double sv = Math.Sin(v);
double cu = Math.Cos(u);
double cv = Math.Cos(v);
x = r2 * cu * su * sv;
y = r2 * cu * su * cv;
z = r2 * cu*cu * sv * cv;
```

See also Wikipedia [1366], MathWorld [1068].



23.5.5 Klein bagel

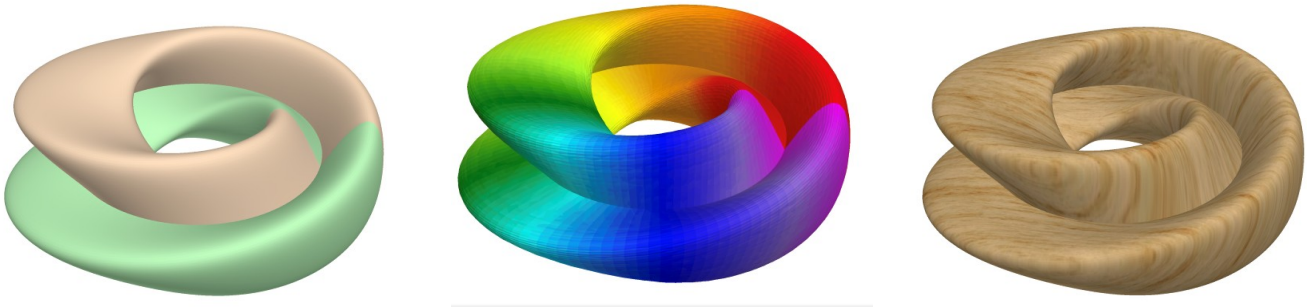
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_KleinBagel.3D.xml (D05a-c).

See also: https://en.wikipedia.org/wiki/Klein_bottle#The_figure_8_immersion

See also: https://mathworld.wolfram.com/KleinBottle.html

```
double r = 2.1;
x = (r + Math.Cos(u / 2) * Math.Sin(v) - Math.Sin(u / 2) * Math.Sin(2 * v)) * Math.Cos(u);
z = (r + Math.Cos(u / 2) * Math.Sin(v) - Math.Sin(u / 2) * Math.Sin(2 * v)) * Math.Sin(u);
y = Math.Sin(u / 2) * Math.Sin(v) + Math.Cos(u / 2) * Math.Sin(2 * v);
```

This is the 'bagel' form of a Klein bottle, a 4 dimensional object with a single surface (lacking 'inside' or 'outside'), projected into 3-space as a self-intersecting solid.



23.5.6 Klein bottle

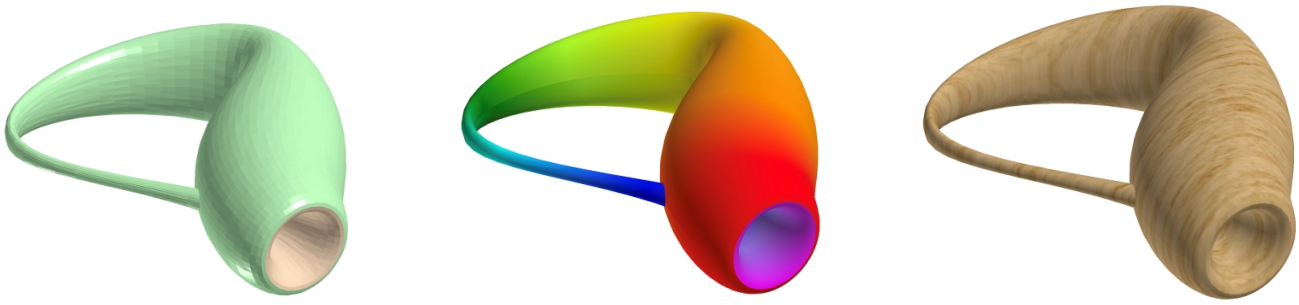
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_KleinBottle3.3D.xml (D06a-c).

See also: http://www.mapleprimes.com/maplesoftblog/95570-Klein-Bottle-Plot

See also: http://www.chebfun.org/examples/geom/ParametricSurfaces.html

See also: https://mathworld.wolfram.com/KleinBottle.html

```
double a = Math.Cos(u);
double b = Math.Sin(u);
double c = Math.Cos(v);
double a2 = a * a;
double a4 = a2 * a2;
x = -(2.0 / 15.0) * a * (3 * c + b * (-30 + a4 * (90 - 60 * a2) + 5 * a * c));
z = -(1.0 / 15.0) * b * b * (c * b * (3 - 48 * a4 + 5 * a * b * (1 - 16 * a4)) - 60);
y = -(2.0 / 15.0) * (3 + 5 * a * b) * Math.Sin(v);
```



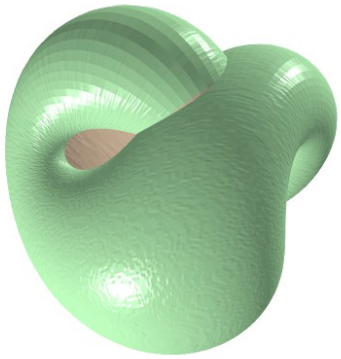
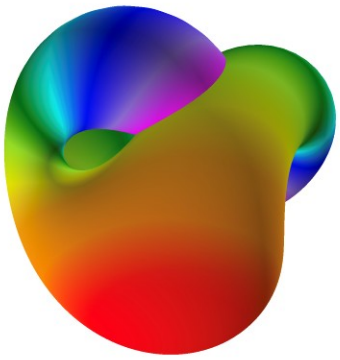
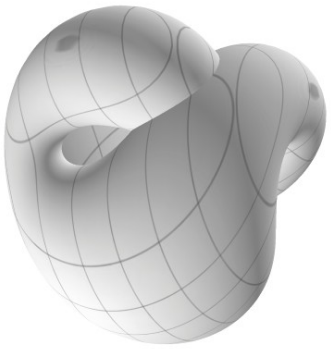
23.5.7 3D Boy Surface, version 1

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D07a_BoySurface.3D.xml (D07a-c).

See also: https://en.wikipedia.org/wiki/Boy%27s_surface

See also: http://mathworld.wolfram.com/BoySurface.html

```
double sqrt5 = Math.Sqrt(5);
// w = u*e^(iv)
double wr = Math.Cos(v);
double wi = Math.Sin(v);
Complex w = u * new Complex(wr, wi);
Complex w3 = w * w * w;
Complex w4 = w3 * w;
Complex w6 = w3 * w3;
Complex d = w6 + sqrt5 * w3 - 1;
Complex wa = w * (1 - w4) / d;
Complex wb = w * (1 + w4) / d;
Complex wc = (1 + w6) / d;
double g1 = -1.5 * wa.Imaginary;
double g2 = -1.5 * wb.Real;
double g3 = wc.Imaginary - 0.5;
double l2 = g1 * g1 + g2 * g2 + g3 * g3;
x = g1 / l2;
y = -g2 / l2;
z = g3 / l2;
```

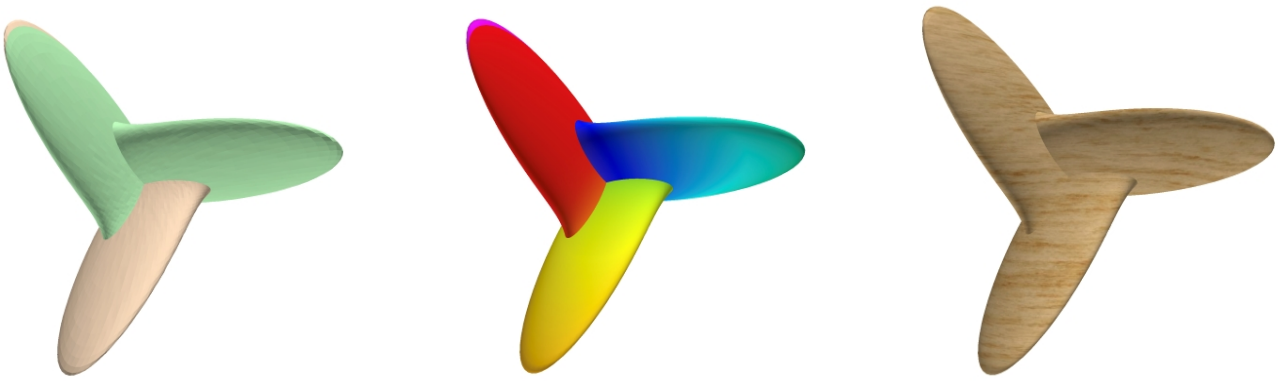
23.5.8 3D Boy Surface, version 2

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D08a_BoySurface2.3D.xml (D08a-c).

See also: https://en.wikipedia.org/wiki/Boy%27s_surface

See also: http://mathworld.wolfram.com/BoySurface.html

```
double sqrt2 = Math.Sqrt(2);
double s2v = Math.Sin(2 * v);
double cu = Math.Cos(u);
double cv = Math.Cos(v);
double cv2 = cv * cv;
double n1 = sqrt2 * cv2 * Math.Cos(2*u);
double xn = sqrt2 * cv2 * Math.Cos(2 * u) + cu * s2v;
double yn = sqrt2 * cv2 * Math.Sin(2 * u) - Math.Sin(u) * s2v;
double zn = 3 * cv2;
double d = 2 - sqrt2 * Math.Sin(3 * u) * s2v;
x = xn / d;
y = yn / d;
z = zn / d;
```

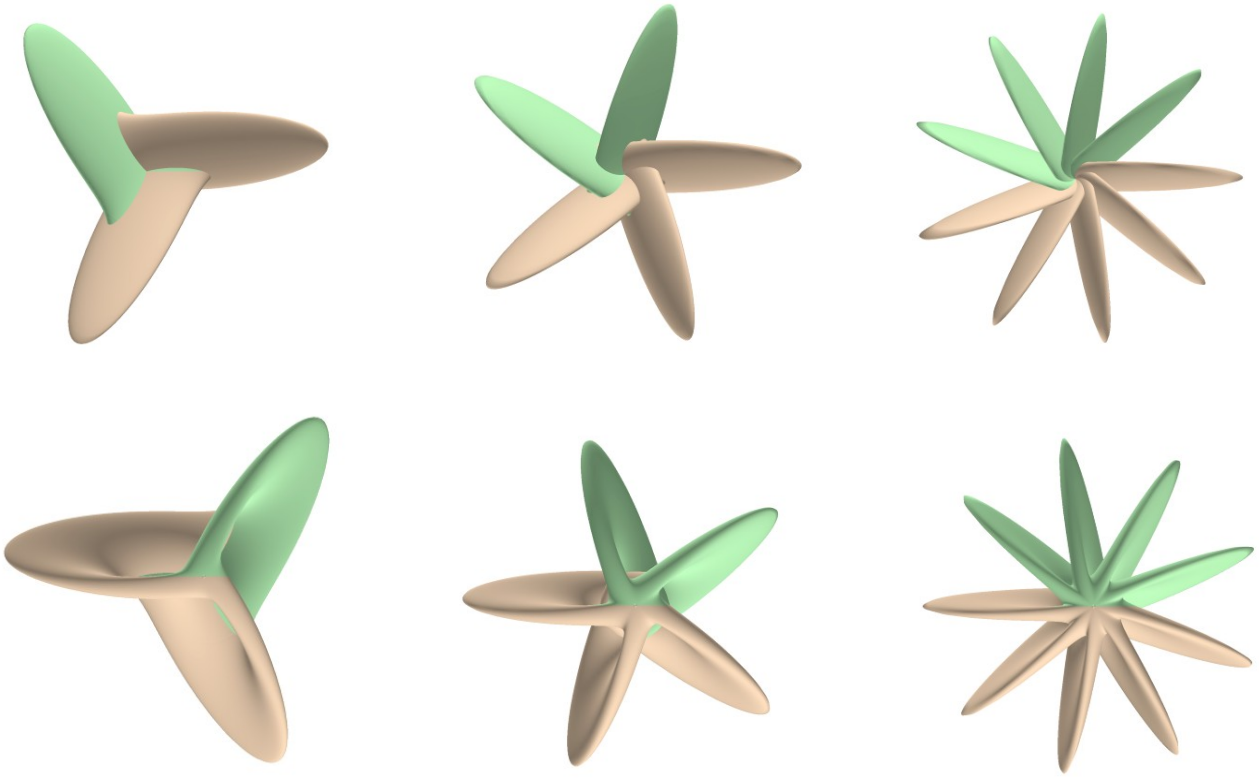


23.5.9 Morin Surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D08a_BoySurface2.3D.xml (D09a-f).

// See also: http://www.3d-meier.de/tut3/Seite221.html // Morin Surface // See also: https://mathcurve.com/surfaces.gb/morin/morin.shtml // See also: https://en.wikipedia.org/wiki/Morin_surface // See also: Bednorz 2019

```
var k = 1.0;
var n = 3.0;

var Sqrt2 = Math.Sqrt(2);
var cu = Math.Cos(u);
var su = Math.Sin(u);
var K = cu / (Sqrt2 - k * Math.Sin(2 * u) * Math.Sin(n * v));

var x = K * (2 / (n - 1) * cu * Math.Cos((n - 1) * v) + Sqrt2 * su * Math.Cos(v));
var y = K * (2 / (n - 1) * cu * Math.Sin((n - 1) * v) - Sqrt2 * su * Math.Sin(v));
var z = K * cu;
```



23.6 Decorative parametric surfaces

https://de.mathworks.com/help/matlab/ref/plot3.html

https://doc.sagemath.org/html/en/reference/plot3d/index.html

http://www.3d-meier.de/tut3/Seite0.html

http://www.3d-meier.de/tut8/Seite3.html

https://docs.ambientcg.com/books/website-licensing/page/license-information

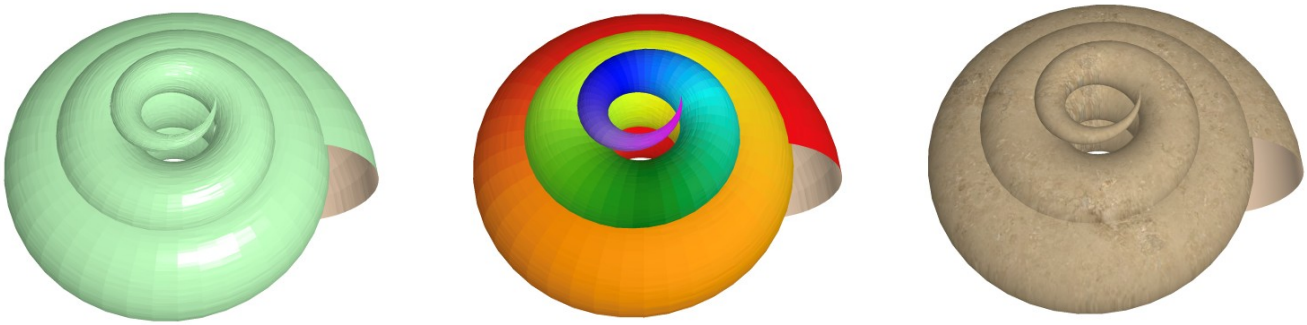
https://ambientcg.com/list?category=&date=&createdUsing=&basedOn=&q=&method=&type=&sort=Downloads

23.6.1 Bourke Seashell

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_BourkeSeaShell.3D.xml (D01a-c).

See also: http://paulbourke.net/geometry/spiral/

```
double n = 3;  // number of spirals
double a = 1;  // final shell radius
double b = 2;  // height
double c = 0.4;  // inner radius

double s = v;
double t = u;
double pt = t/(2*Math.PI);  // inner radius

x = a * (1 - pt) * Math.Cos(n*t) * (1 + Math.Cos(s)) + c * Math.Cos(n*t);
z = a * (1 - pt) * Math.Sin(n*t) * (1 + Math.Cos(s)) + c * Math.Sin(n*t);
y = b * pt + a * (1 - pt) * Math.Sin(s);
```

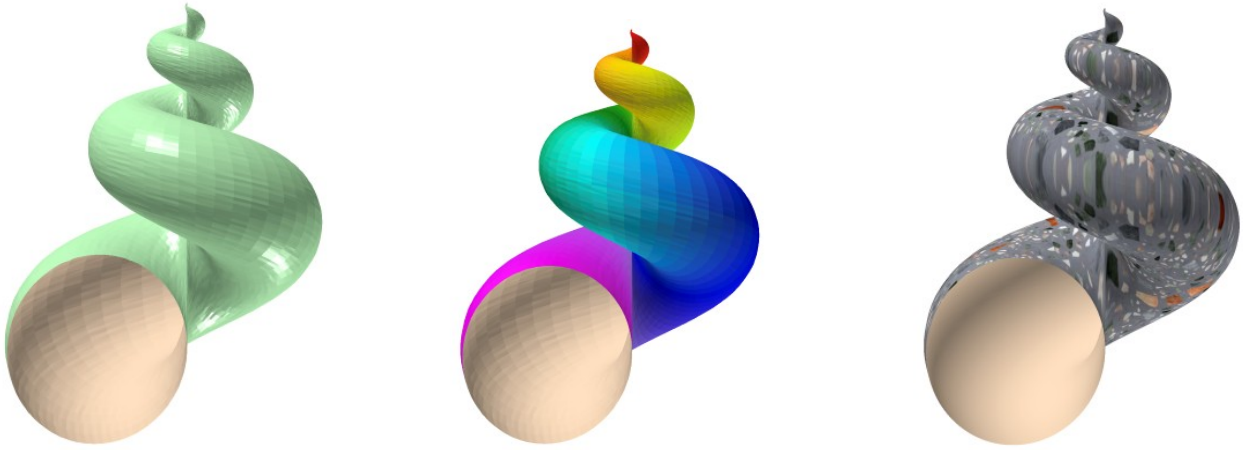


23.6.2 Seashell (mathworld.wolfram)

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Seashell.3D.xml (D02a-c).

See also: https://mathworld.wolfram.com/Seashell.html

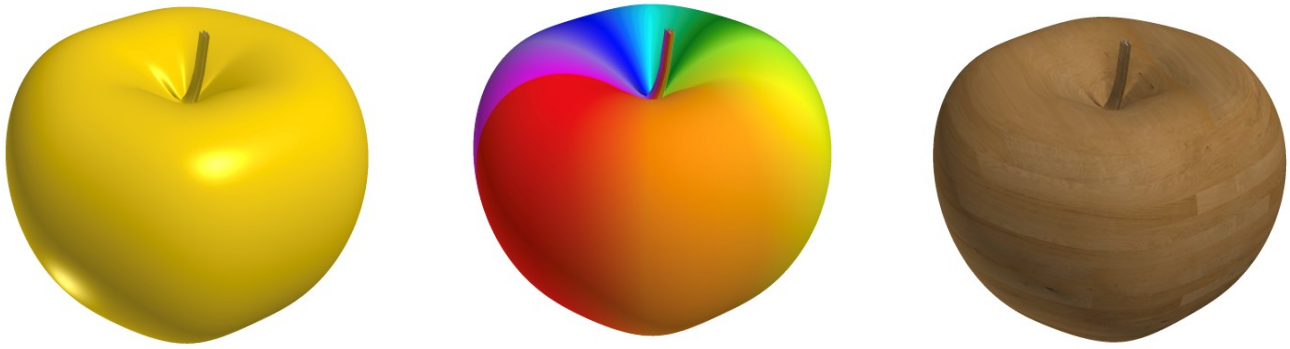
```
double a = Math.Exp(u / (6.0 * Math.PI));
double b = Math.Cos(v / 2.0);

x = 2.0 * (1.0 - a) * Math.Cos(u) * b * b;
y = 2.0 * (-1.0 + a) * Math.Sin(u) * b * b;
z = (1.0 - a * a - Math.Sin(v) * (1.0 - a));
```



23.6.3 3D Apple

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_Apple.3D.xml (D03a-c).

```
double R1 = 5.0;
double R2 = 4.8;
double su = Math.Sin(u);
double sv = Math.Sin(v);
double cu = Math.Cos(u);
double c5u = Math.Cos(5*u);
double cv = Math.Cos(v);
x = cu * (R1 + R2 * cv) + Math.Pow(v/Math.PI, 20);
z = su * (R1 + R2 * cv) + 0.25 * c5u;
y = -2.3 * Math.Log(1 - v * 0.3157) + 6 * sv + 2 * cv;
```

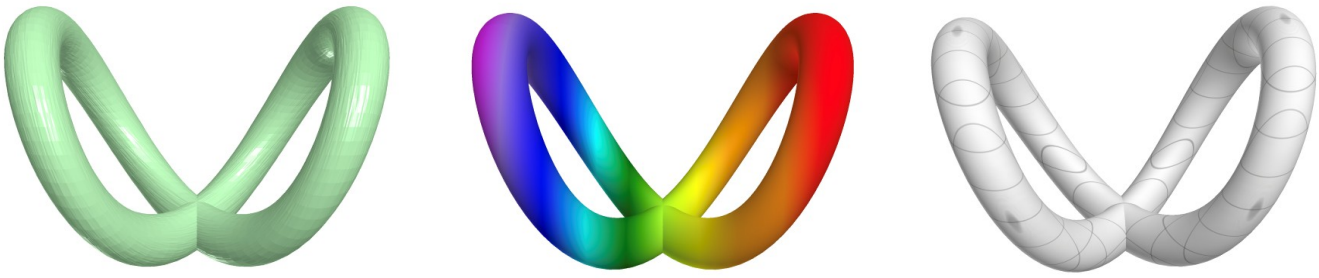


23.6.4 Bow curve

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_BowCurve.3D.xml (D04a-c).

See also: http://paulbourke.net/geometry/toroidal/

```
double p2 = 2*Math.PI;
double p4 = 4*Math.PI;
double T = 0.5; //Thickness

x = (2 + T * Math.Sin(p2 * u)) * Math.Sin(p4 * v);
y = (2 + T * Math.Sin(p2 * u)) * Math.Cos(p4 * v);
z = T * Math.Cos(p2 * u) + 3 * Math.Cos(p2 * v);
```



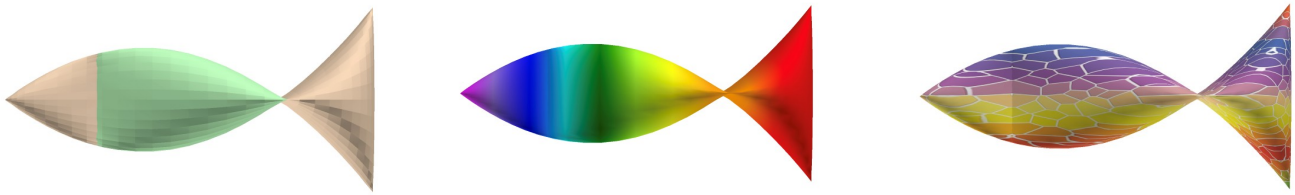
23.6.5 Fish Surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Fish.3D.xml (D05a-c).

See also: http://www.3d-meier.de/tut3/Seite47.html

See also Wikipedia [1366], MathWorld [1068].

```
double su = Math.Sin(u);
double s2u = Math.Sin(2*u);
double sv = Math.Sin(v);
double cu = Math.Cos(u);
double c2u = Math.Cos(2*u);
double cv = Math.Cos(v);
x = (cu - c2u) * cv / 4.0;
y = (su - s2u) * sv / 4.0;
z = cu;
```

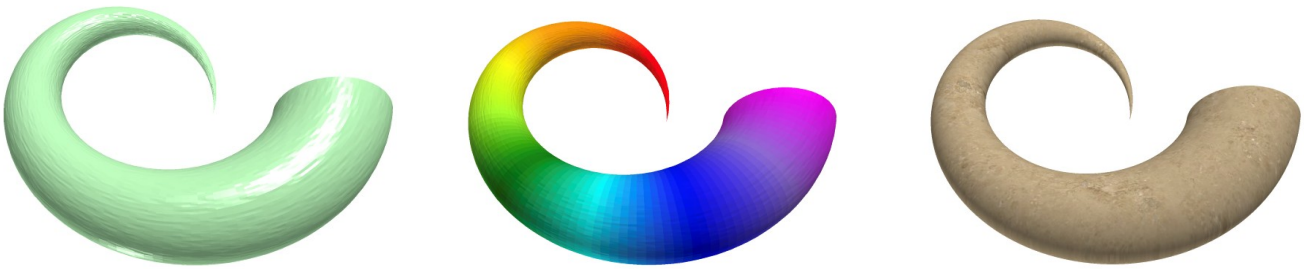


23.6.6 Bourke Horn

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_BourkeHorn.3D.xml (D06a-c).

See also: http://paulbourke.net/geometry/spiral/

```
double p2 = 2*Math.PI;
x = (2 + u * Math.Cos(v)) * Math.Sin(p2 * u);
y = (2 + u * Math.Cos(v)) * Math.Cos(p2 * u) + 2 * u;
z = u * Math.Sin(v);
```



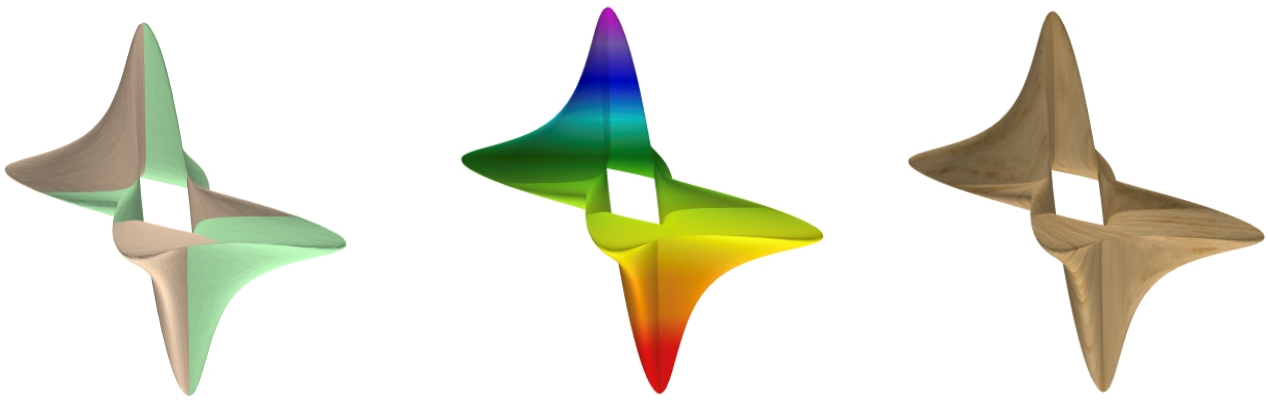
23.6.7 Hexa torus

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D07a_HexaTorus.3D.xml (D07a-c).

See also: http://paulbourke.net/geometry/toroidal/

The C# code for the Klein bottle:

```
double a = Math.Cos(u);
double b = Math.Sin(u);
double c = Math.Cos(v);
double a2 = a * a;
double a4 = a2 * a2;

x = -(2.0 / 15.0) * a * (3 * c + b * (-30 + a4 * (90 - 60 * a2) + 5 * a * c));
z = -(1.0 / 15.0) * b * b * (c * b * (3 - 48 * a4 + 5 * a * b * (1 - 16 * a4)) - 60);
y = (2.0 / 15.0) * (3 + 5 * a * b) * Math.Sin(v);
```



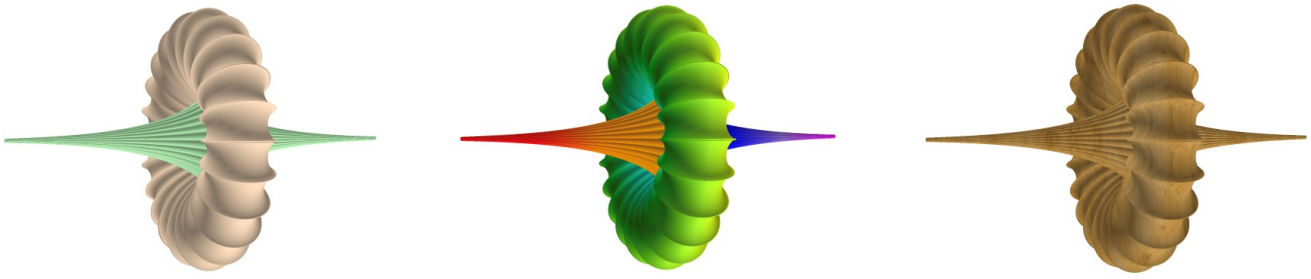
23.6.8 3D Breather Surface

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D08a_Breather.3D.xml (D08a-c).

See also: https://en.wikipedia.org/wiki/Breather_surface

See also: https://mathworld.wolfram.com/BreatherSurface.html

```
const double b = 0.4;
double r = 1 - b * b;
double w = Math.Sqrt(r);
double denom = b * (
    (w * Math.Cosh(b * u)) * (w * Math.Cosh(b * u)) +
    (b * Math.Sin(w * v)) * (b * Math.Sin(w * v)));

x = -u + (2 * r * Math.Cosh(b * u) * Math.Sinh(b * u)) / denom;
y = (2 * w * Math.Cosh(b * u) * (-(w * Math.Cos(v) * Math.Cos(w * v)) - Math.Sin(v) * Math.Sin(w * v))) / denom;
z = (2 * w * Math.Cosh(b * u) * (-(w * Math.Sin(v) * Math.Cos(w * v)) + Math.Cos(v) * Math.Sin(w * v))) / denom;
```



23.6.9 Kuen surface

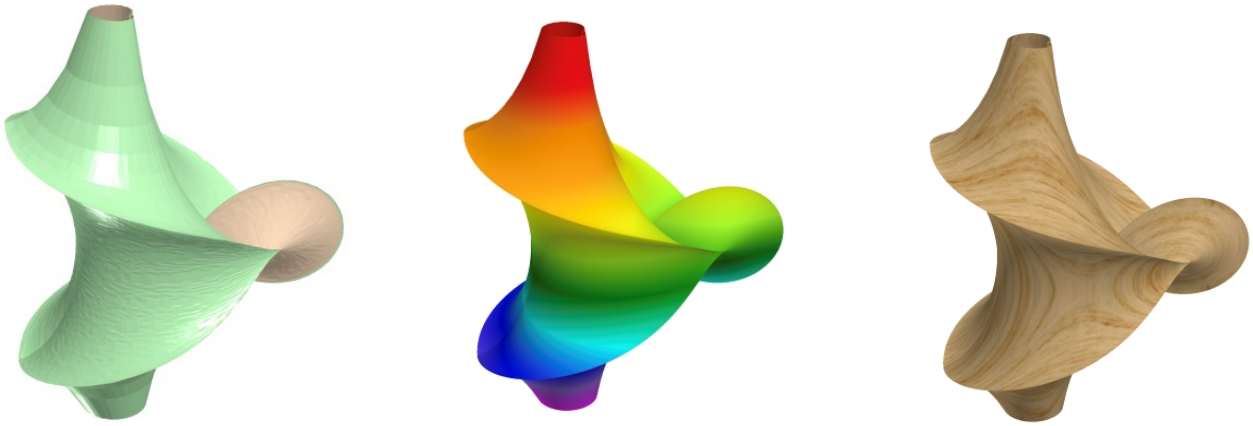
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D09a_Kuen.3D.xml (D09a-c).

See also: https://mathworld.wolfram.com/KuenSurface.html

See also: https://virtualmathmuseum.org/Surface/kuen/kuen.html

See also: https://mathcurve.com/surfaces.gb/kuen/kuen.shtml

```
double a = 1.0 * Math.Sin(v);
double b = 1.0 + u * u * a * a;

x = 2.0 * a * (Math.Cos(u) + u * Math.Sin(u)) / b;
z = 2.0 * a * (Math.Sin(u) - u * Math.Cos(u)) / b;
y = Math.Log(Math.Tan(v / 2.0)) + 2.0 * Math.Cos(v) / b;
```



23.6.10 Tranguloid trefoil

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D10a_TranguloidTrefoil.3D.xml (D10a-c).

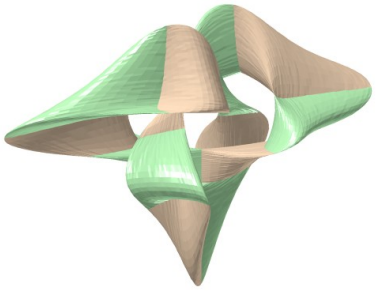
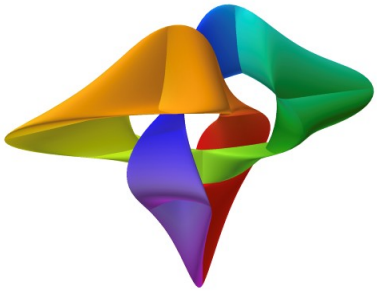
See also: http://paulbourke.net/geometry/tranguloid/

There are other parametrizations:

See also: http://www.3d-meier.de/tut3/Seite56.html See also: https://mathart.org/sw/GreenSnake/Toroid2.html

See also: http://www.3d-meier.de/tut3/Seite159.html

```
double p2 = 2*Math.PI/3;
x = 2 * Math.Sin(3 * u) / (2 + Math.Cos(v));
y = 2 * (Math.Sin(u) + 2 * Math.Sin(2 * u)) / (2 + Math.Cos(v + p2));
z = (Math.Cos(u) - 2 * Math.Cos(2 * u)) * (2 + Math.Cos(v)) * (2 + Math.Cos(v + p2)) / 4;
```

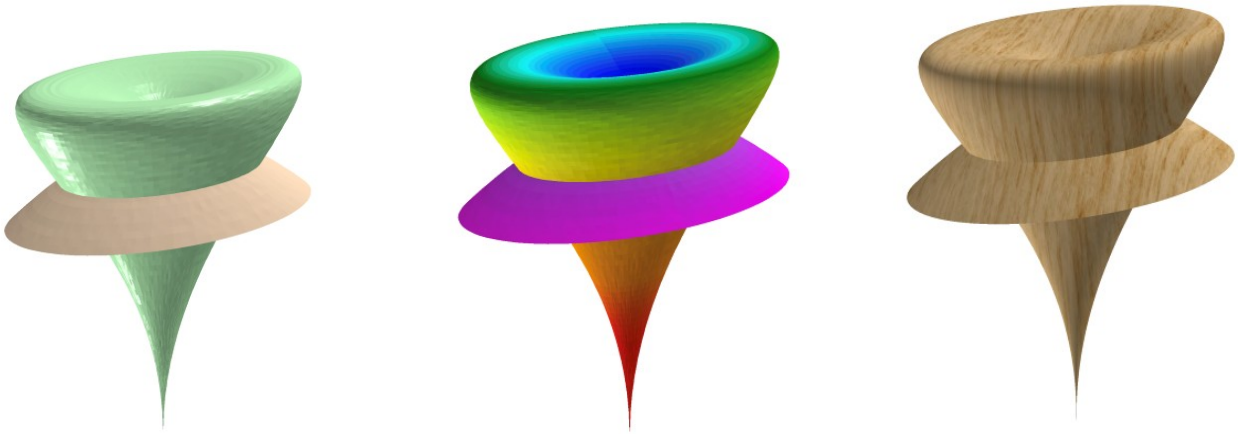
23.6.11 Triaxial teardrop

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D11a_TearDrop.3D.xml (D11a-c).

See also: http://paulbourke.net/geometry/triaxtear/

See also Wikipedia [1366], MathWorld [1068].

```
double p2 = 2*Math.PI/3;
x = ( 1 - Math.Cos(u) ) * Math.Cos(u + p2) * Math.Cos(v + p2) / 2;
y = -( 1 - Math.Cos(u) ) * Math.Cos(u + p2) * Math.Cos(v - p2) / 2;
z = Math.Cos(u - p2);
```

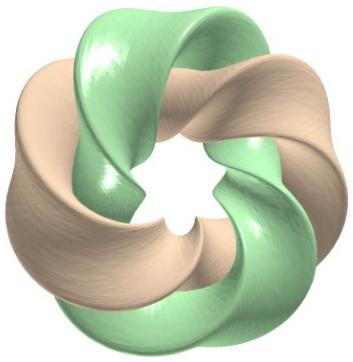
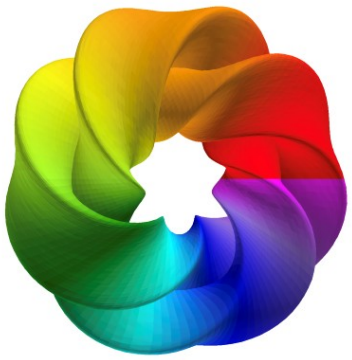


23.6.12 Gray Bottle

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D12a_GrayBottel.3D.xml (D12a-c).

See also: http://paulbourke.net/geometry/toroidal/

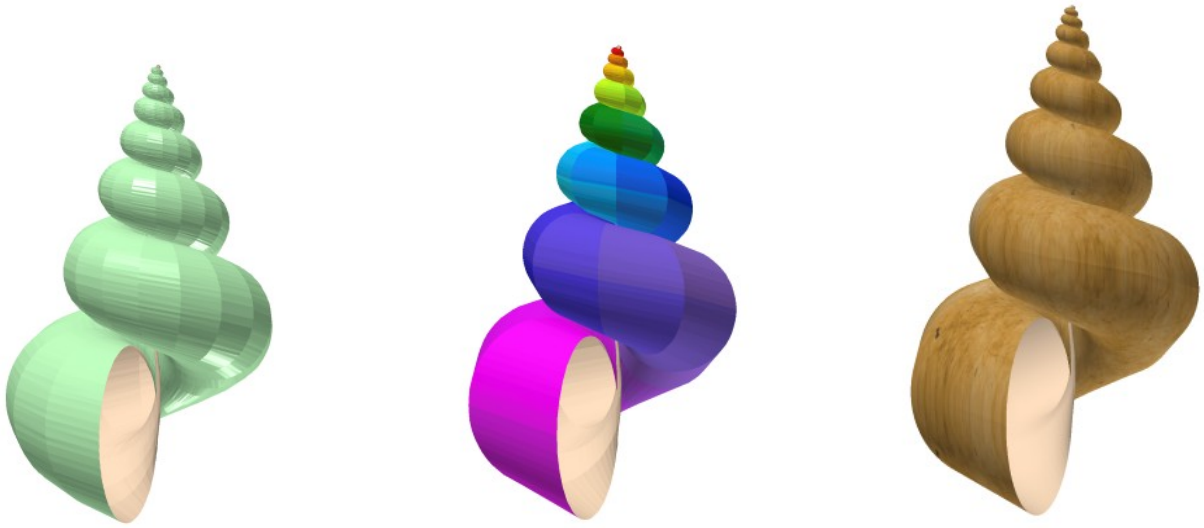
```
x = Math.Cos(v) * Math.Sqrt(Math.Abs(Math.Sin(2 * u))) * Math.Cos(u);
y = Math.Cos(v) * Math.Sqrt(Math.Abs(Math.Sin(2 * u))) * Math.Sin(u);
z = x*x - y*y + 2 * x * y * Math.Tan(v) * Math.Tan(v);
```

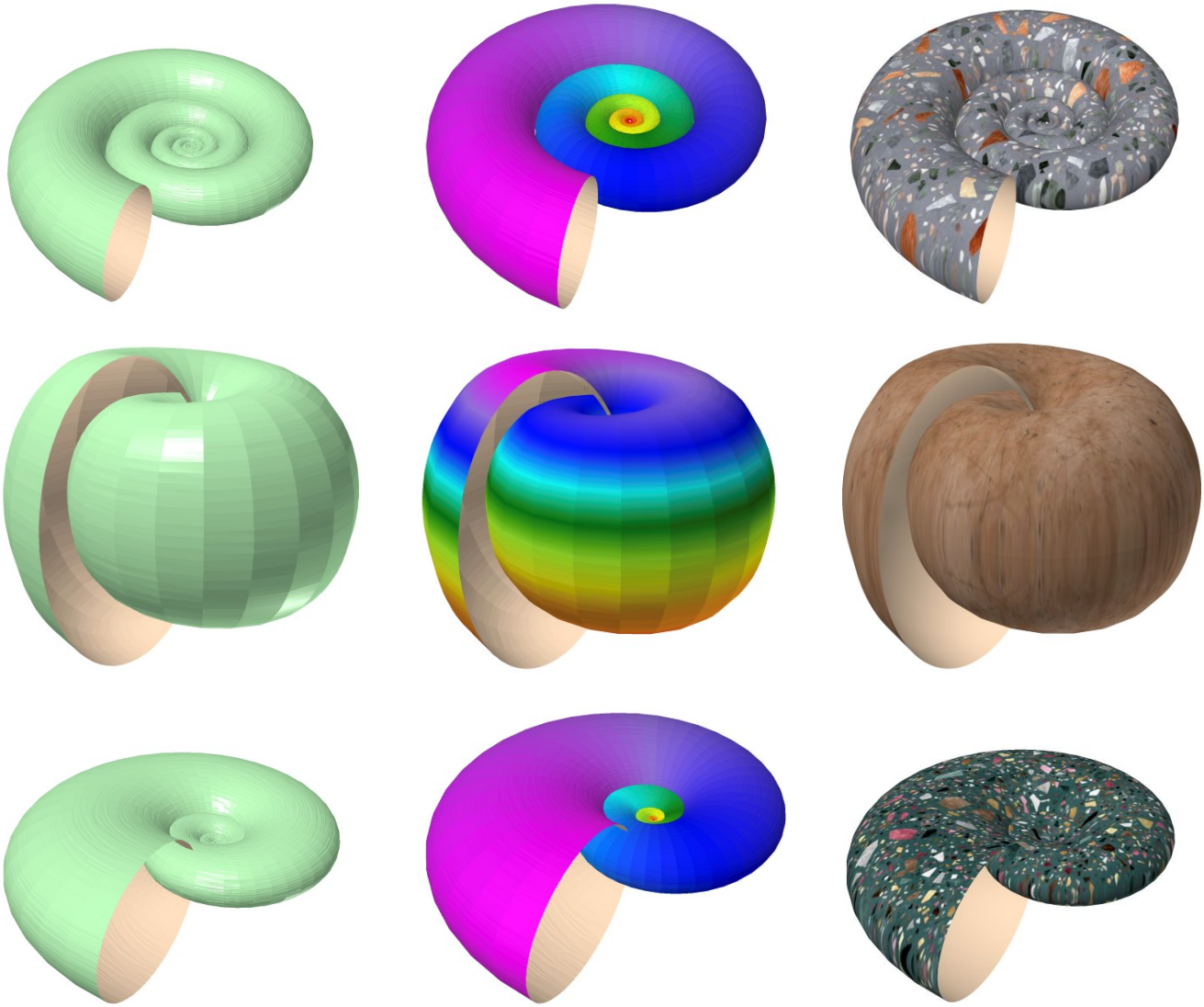
23.6.13 Surfaces mimicking snail shells

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D13a_Snail1.3D.xml (D13a-u).

See also: http://www.3d-meier.de/tut3/Seite89.html



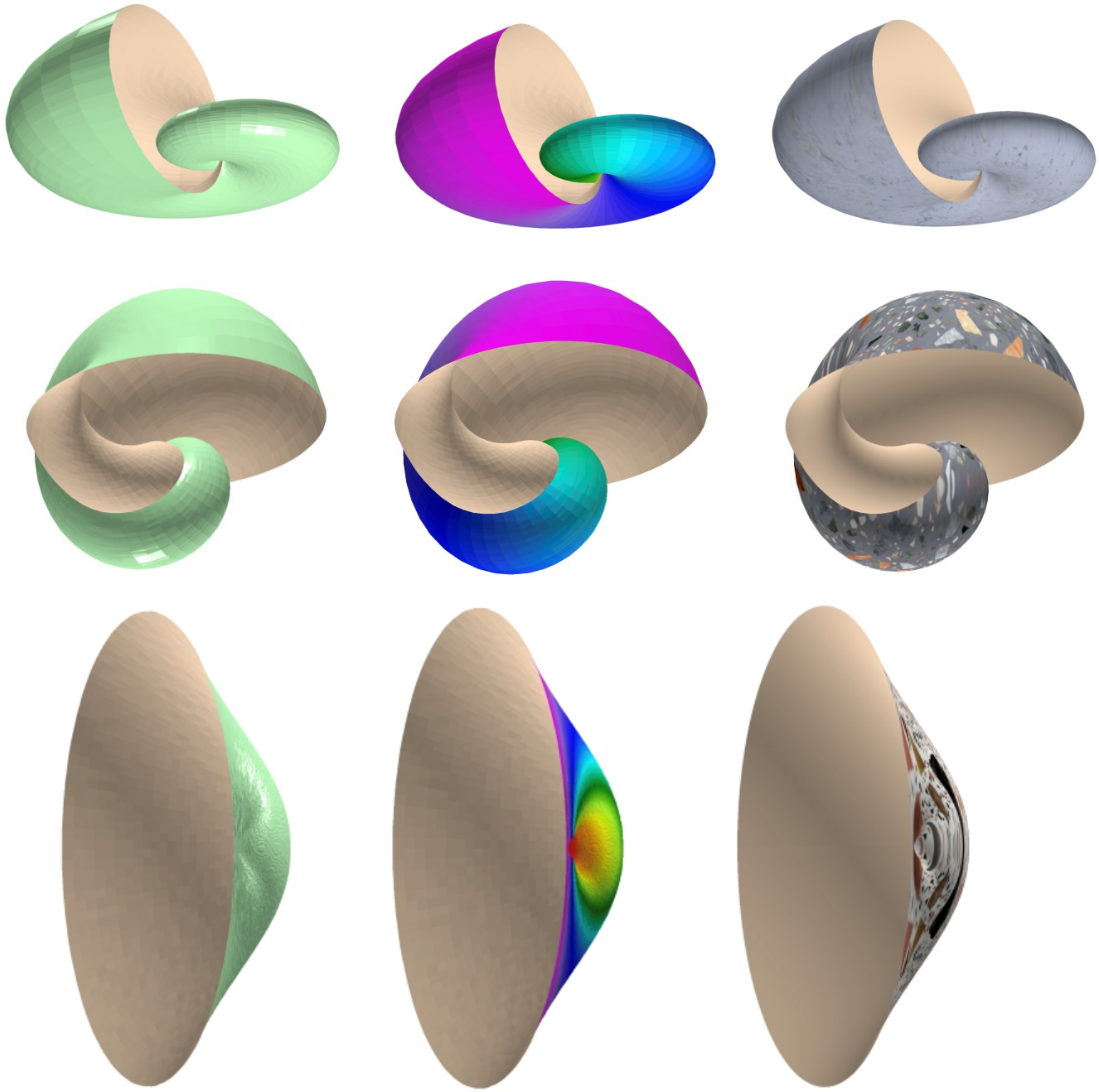
```
double R = 1;
double a = 1.6;
double b = 1.6;
double c = 1.0;
double h = 1.5;
double k = -7.0;
double w = 0.075;
//  umin = -50, umax = -1,    name:  Pseudoheliceras subcatenatum

double su = Math.Sin(u);
double scu = Math.Sin(c*u);
double sv = Math.Sin(v);
double cv = Math.Cos(v);
double ccu = Math.Cos(c*u);

double ewu = Math.Exp(w*u);

x = ewu * (h+a*cv) * ccu;
z = R * ewu * (h+a*cv) * scu;
y = ewu * (k + b * sv);
```



Left figure: Astroceras surface.

Middle figure: Bellerophina surface.

Right figure: Euhoplites Surface.



Left figure: Nautilus surface.

Middle figure: Natica stellata surface.

Right figure: Mya arenaria surface.

CHAPTER

TWENTYFOUR

WPF: PATH SURFACES IN 3D

Surfaces based on parametric curves: $x = f(t), y = g(t)\ [, z = h(t)]$

24.1 Introduction to path surfaces

24.1.1 Borromean rings, A, B, C

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Path_BorromeanA.3D.xml (D01a-c).

Some text explaining the importance of building the mesh in x-direction.

The example below uses the following code in C#, for A

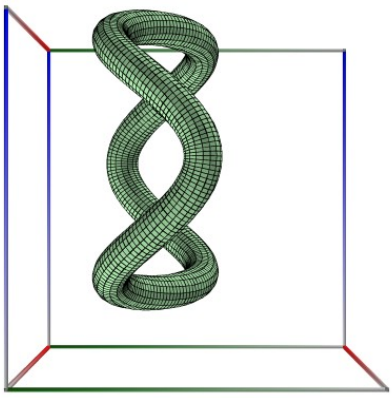
```
var r = Math.Sqrt(3) / 3;
var x = Math.Cos(t);
var y = Math.Sin(t) + r;
var z = Math.Cos(3 * t) / 3;
```

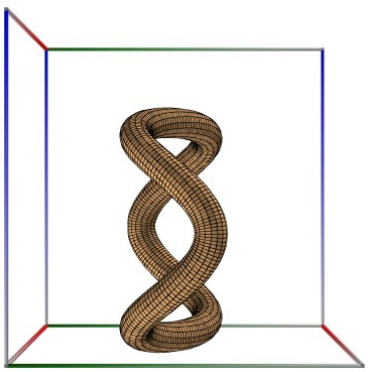
The example below uses the following code in C#, for B

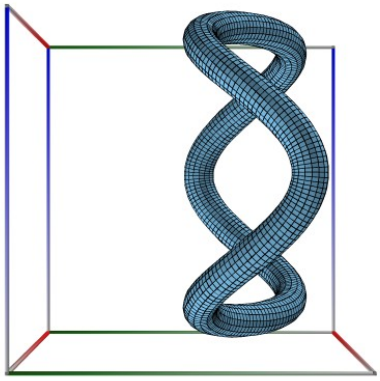
```
var r = Math.Sqrt(3) / 3;
var x = Math.Cos(t) + 0.5;
var y = Math.Sin(t) - r / 2;
var z = Math.Cos(3 * t) / 3;
```

The example below uses the following code in C#, for C

```
var r = Math.Sqrt(3) / 3;
var x = Math.Cos(t) - 0.5;
var y = Math.Sin(t) - r / 2;
var z = Math.Cos(3 * t) / 3;
```







24.1.2 Ellipses, A, B, C

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Path_Ellipses.3D.xml (D01a-i).

Some text explaining the importance of building the mesh in x-direction.

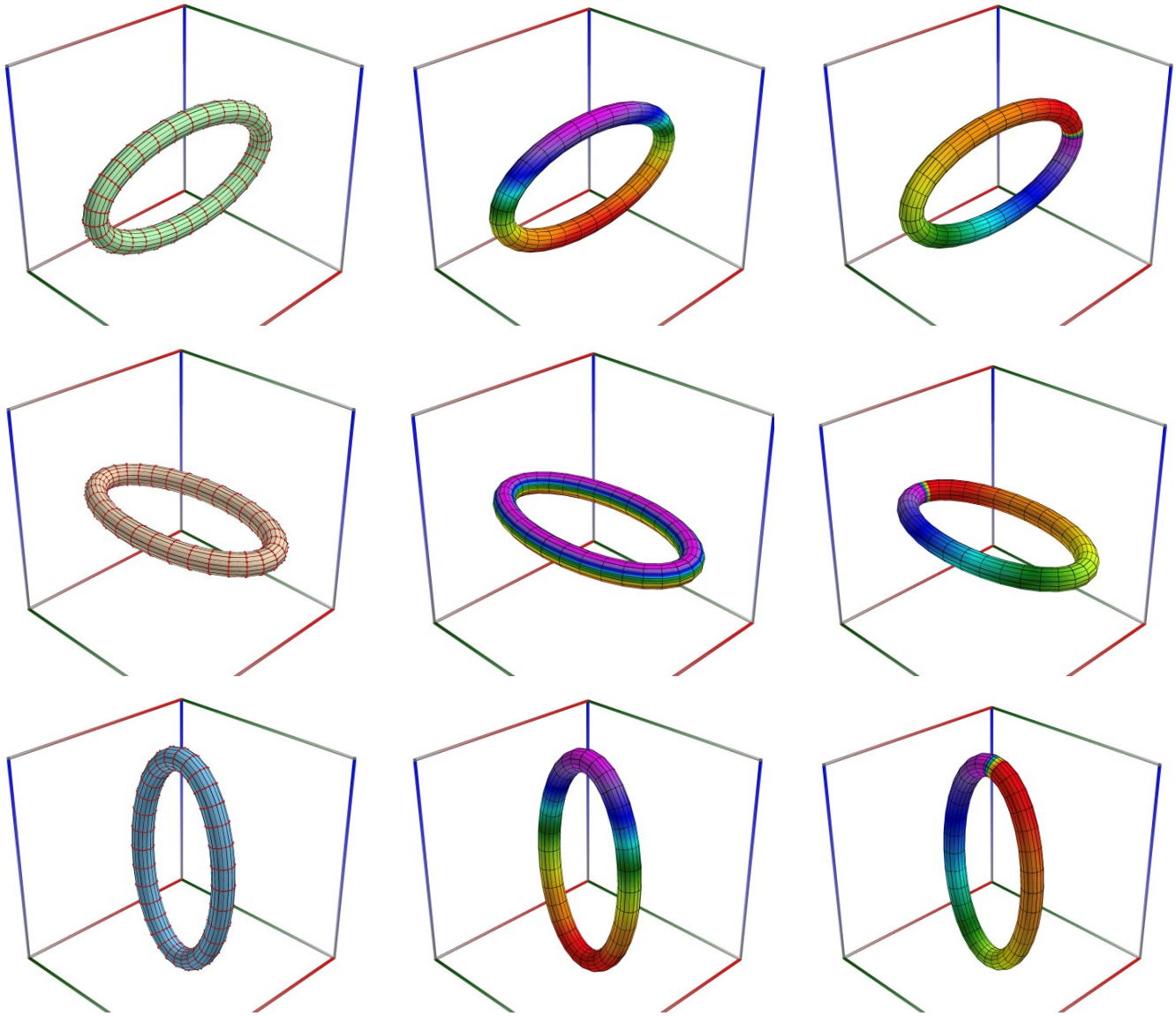
The example below uses the following code in C#, for A, B, C

```
var x = 2 * Math.Cos(t);
var y = Math.Sin(t);
var z = 0.0;
```

For A, the final rotations are: X=0, Y=0, Z=0.

For B, the final rotations are: X=90, Y=0, Z=90.

For C, the final rotations are: X=0, Y=90, Z=90.



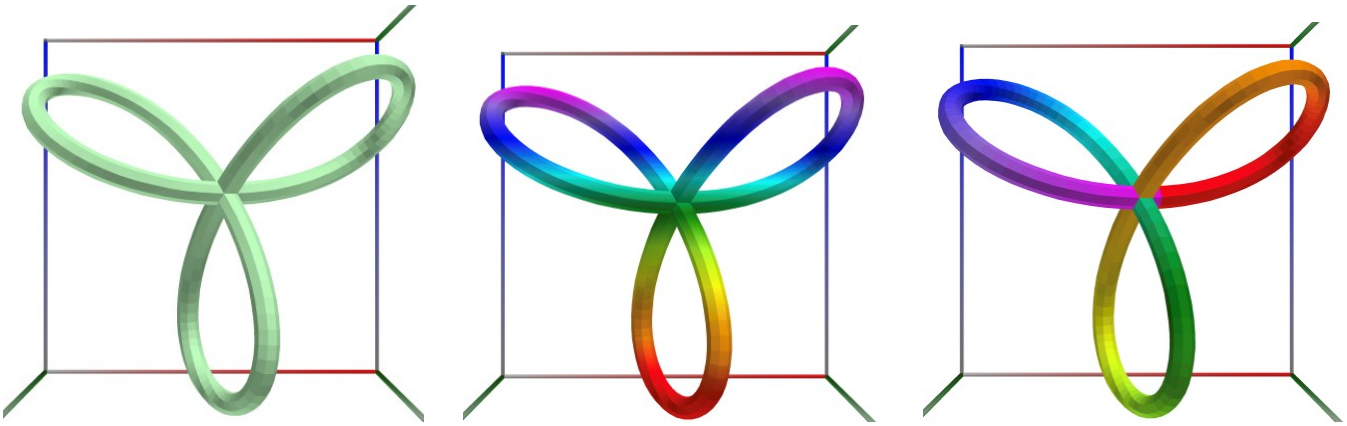
24.1.3 Trefoil 2

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_Path_Trefoil02.3D.xml (D01a-c).

Some text explaining the importance of building the mesh in x-direction.

The example below uses the following code in C#, for A, B, C

```
var D = 2.0; //  D = 1.0;  D = 2.0;
var x = D * Math.Sin(t) + 2 * Math.Sin(2 * t);
var y = D * Math.Cos(t) - 2 * Math.Cos(2 * t);
var z = -D * Math.Sin(3 * t);
```



24.1.4 Formatting options for path surfaces, Trefoil 5

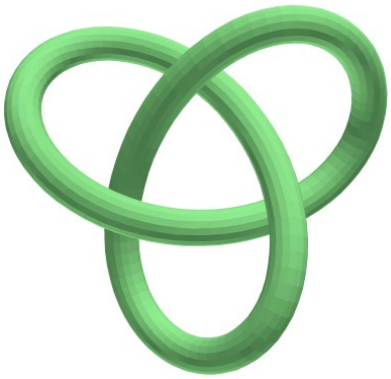
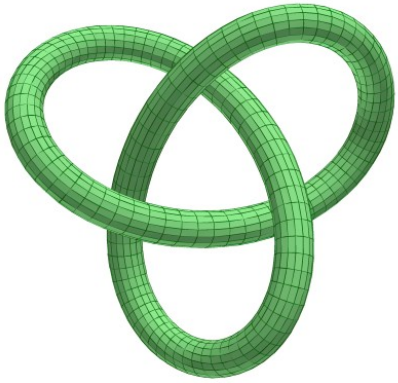
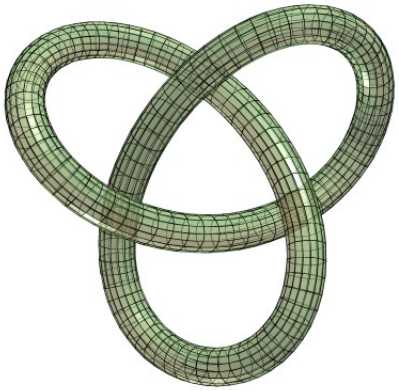
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Path_Trefoil05.3D.xml (D01a-f).

These are the rough versions

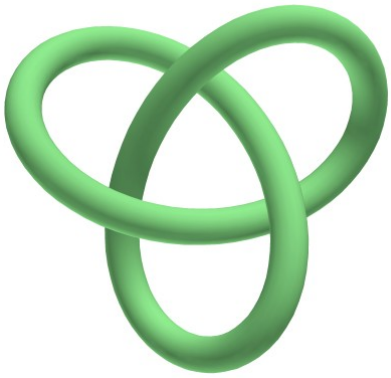
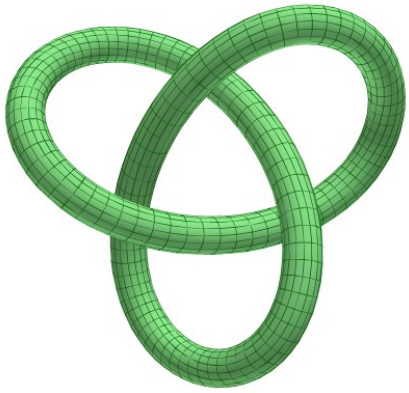
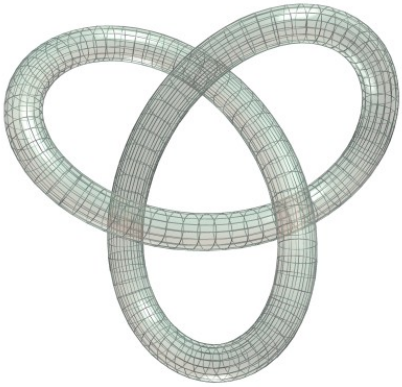
The example below uses the following code in C#, for A, B, C

```
var D = 1.0; //  D = 1.0;  D = 2.0;
var x = D * Math.Sin(t) + 2 * Math.Sin(2 * t);
var y = D * Math.Cos(t) - 2 * Math.Cos(2 * t);
var z = -D * Math.Sin(3 * t);
```

These are the rough versions

These are the smooth versions

24.2 Functions with real input and complex results

24.2.1 Expj (Cis) function

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Path_Func_Expj_2Dx.3D.xml (D01a-e).

Returns $e^{iz} = \cos(z) + i\sin(z)$. See also Wikipedia [1374], MathWorld [1019].

The example below uses the following code in C#, for real part only

```
var y = Math.Sin(t);
var x = t;
var z = 0;
```

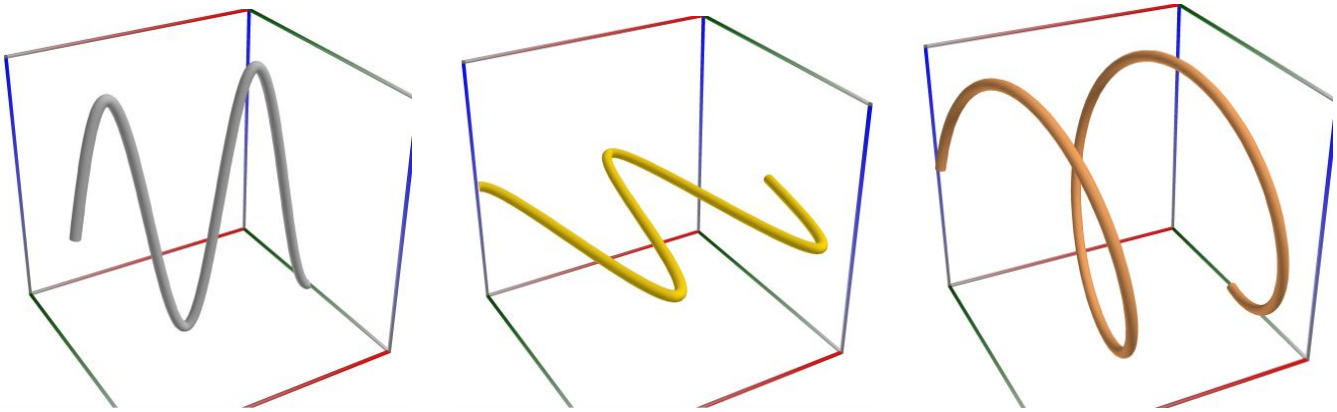
The example below uses the following code in C#, for imaginary part only

```
var z = -Math.Cos(t);
var x = t;
var y = 0;
```

The example below uses the following code in C#, for real and imaginary part combined

```
var x = Math.Sin(t);
var y = -Math.Cos(t);
var z = t;
```

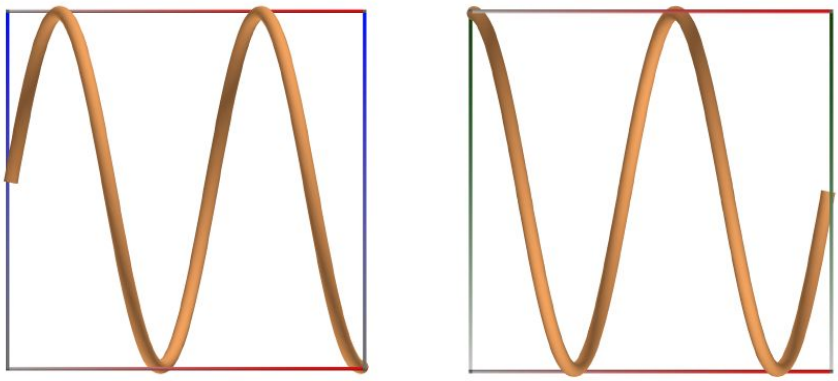
Some text



Left figure: Expj (Cis) function, real part only. Orthographic camera.

Middle figure: Expj (Cis) function, imaginary part only. Orthographic camera.

Right figure: Expj (Cis) function as full 3D curve. Orthographic camera.



24.2.2 Hankel function of the first kind $H_{1,\nu}(x)$

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Path_Func_Hankel1_2Dx.3D.xml (D02a-c).

Returns the Hankel function of the first kind, defined as $H_\nu^{(1)}(x) = J_\nu(x) + iY_\nu(x)$.

See also Wikipedia [1306], MathWorld [940], NIST [443], BoostMath [115].

The example below uses the following code in C#, for real part only

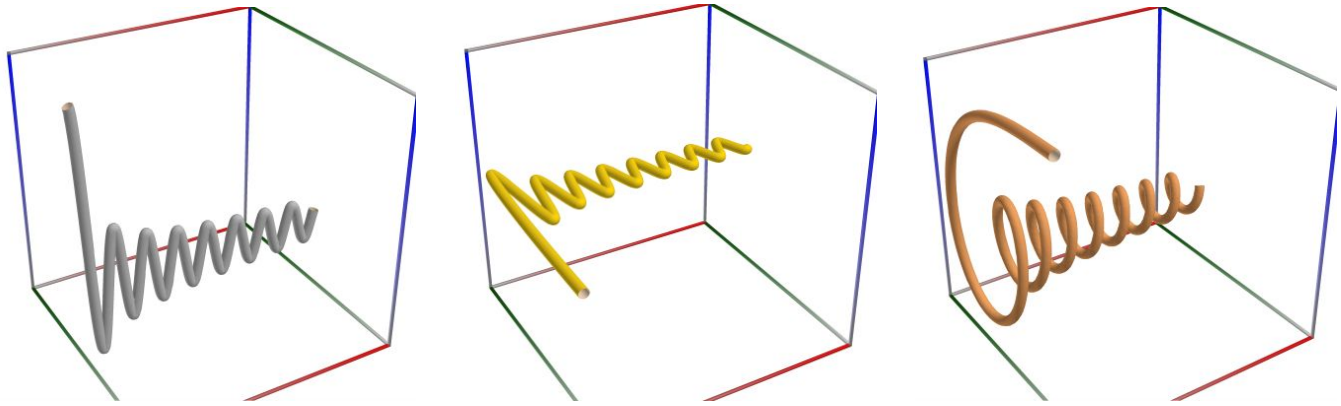
```
var y = ctx.bessel_j0(t);
var x = t;
var z = 0;
```

The example below uses the following code in C#, for imaginary part only

```
var z = -ctx.bessel_y0(t);
var x = t;
var y = 0;
```

The example below uses the following code in C#, for real and imaginary part combined

```
var z = ctx.bessel_j0(t);
var y = -ctx.bessel_y0(t);
var x = t;
```



Left figure: Hankel function of the first kind $H_{1,\nu}(x)$, real part only. Orthographic camera.

Middle figure: Hankel function of the first kind $H_{1,\nu}(x)$, imaginary part only. Orthographic camera.

Right figure: Hankel function of the first kind $H_{1,\nu}(x)$, as full 3D curve. Orthographic camera.

24.2.3 Hankel function of the second kind $H_{2,\nu}(x)$

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_Path_Func_Hankel2_2Dx.3D.xml (D03a-c).

Returns the Hankel function of the second kind, defined as $H_\nu^{(2)}(x) = J_\nu(x) - iY_\nu(x)$.

See also Wikipedia [1306], MathWorld [941], NIST [443], BoostMath [115].

The example below uses the following code in C#, for real part only

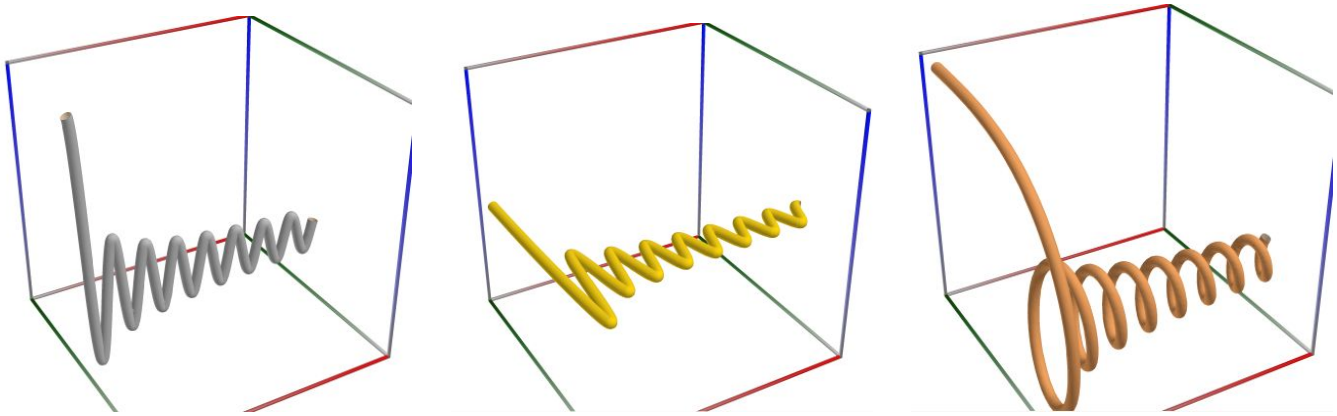
```
var y = ctx.bessel_j0(t);
var x = t;
var z = 0;
```

The example below uses the following code in C#, for imaginary part only

```
var z = ctx.bessel_y0(t);
var x = t;
var y = 0;
```

The example below uses the following code in C#, for real and imaginary part combined

```
var z = ctx.bessel_j0(t);
var y = ctx.bessel_y0(t);
var x = t;
```



Left figure: Hankel function of the second kind $H_{2,\nu}(x)$, real part only. Orthographic camera.

Middle figure: Hankel function of the second kind $H_{2,\nu}(x)$, imaginary part only. Orthographic camera.

Right figure: Hankel function of the second kind $H_{2,\nu}(x)$, as full 3D curve. Orthographic camera.

24.2.4 Kelvin functions ber and bei

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Path_Func_KelvinBerBei_2Dx.3D.xml (D04a-c).

Returns the Kelvin functions ber bei. See also Wikipedia [1381], MathWorld [1022], NIST [444].

The following conventions are used as in Maple (with $a = \sqrt{2}/2$):

$$\text{ber}(\nu, x) + i\text{bei}(\nu, x) = J_\nu(x(-a + ia))$$

$$\text{ber}(\nu, x) - i\text{bei}(\nu, x) = J_\nu(x(-a - ia))$$

The example below uses the following code in C#, for the real part only

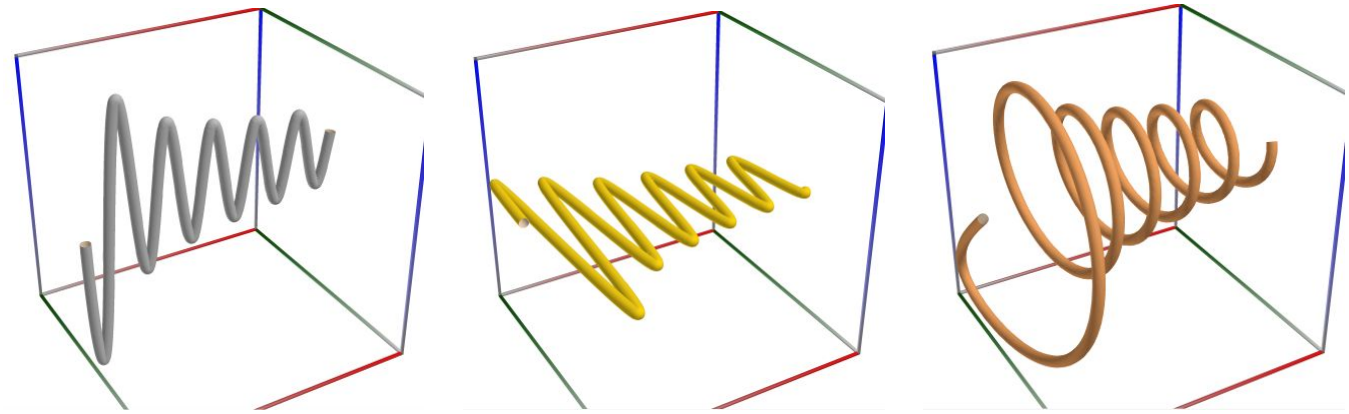
```
var a = ctx.exp(t / ctx.sqrt(2));
var x = t;
var y = 0;
var z = ctx.kelvin_ber(1,t) / a;
```

The example below uses the following code in C#, for the imaginary part only

```
var a = ctx.exp(t / ctx.sqrt(2));
var x = t;
var y = ctx.kelvin_bei(1,t) / a;
var z = 0;
```

The example below uses the following code in C#, for the real and imaginary parts combined

```
var a = ctx.exp(t / ctx.sqrt(2));
var x = t;
var y = ctx.kelvin_bei(1,t) / a;
var z = ctx.kelvin_ber(1,t) / a;
```



Left figure: Kelvin functions ber. Orthographic camera.

Middle figure: Kelvin function bei. Orthographic camera.

Right figure: Kelvin functions ber and bei, as full 3D curve. Orthographic camera.

24.2.5 Kelvin functions ker and kei

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Path_Func_KelvinKerKei_2Dx.3D.xml (D05a-c).

Returns the Kelvin function ker. See also Wikipedia [1383], MathWorld [1024], NIST [444].

The following conventions are used as in Maple (with $a = \sqrt{2}/2$):

$$\text{ker}(\nu, x) + i\text{kei}(\nu, x) = e^{-i\nu\pi/2} K_\nu(x(a + ia))$$

$$\text{ker}(\nu, x) - i\text{kei}(\nu, x) = e^{i\nu\pi/2} K_\nu(x(a - ia))$$

The example below uses the following code in C#, for real part only

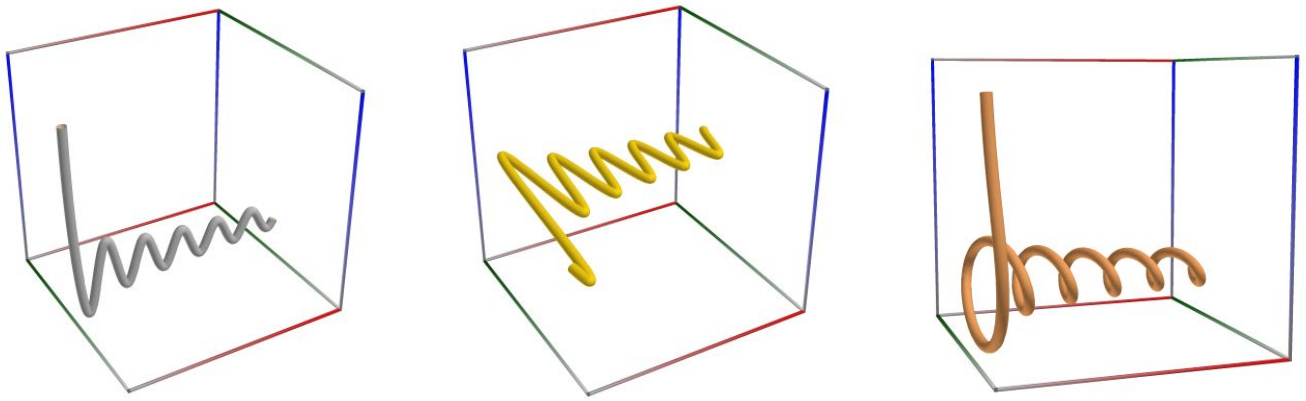
```
var a = ctx.exp(t / ctx.sqrt(2));
var x = t;
var y = 0;
var z = ctx.kelvin_ker(0,t) * a;
```

The example below uses the following code in C#, for imaginary part only

```
var a = ctx.exp(t / ctx.sqrt(2));
var x = t;
var y = ctx.kelvin_kei(0,t) * a;
var z = 0;
```

The example below uses the following code in C#, for real and imaginary part combined

```
var a = ctx.exp(t / ctx.sqrt(2));
var x = t;
var y = ctx.kelvin_kei(0,t) * a;
var z = ctx.kelvin_ker(0,t) * a;
```



Left figure: Kelvin functions ker. Orthographic camera.

Middle figure: Kelvin function kei. Orthographic camera.

Right figure: Kelvin functions ker and kei, as full 3D curve. Orthographic camera.

24.3 Characteristic functions of statistical distributions

24.3.1 Uniform distribution

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_CfUniform.3D.xml (D01a-c).

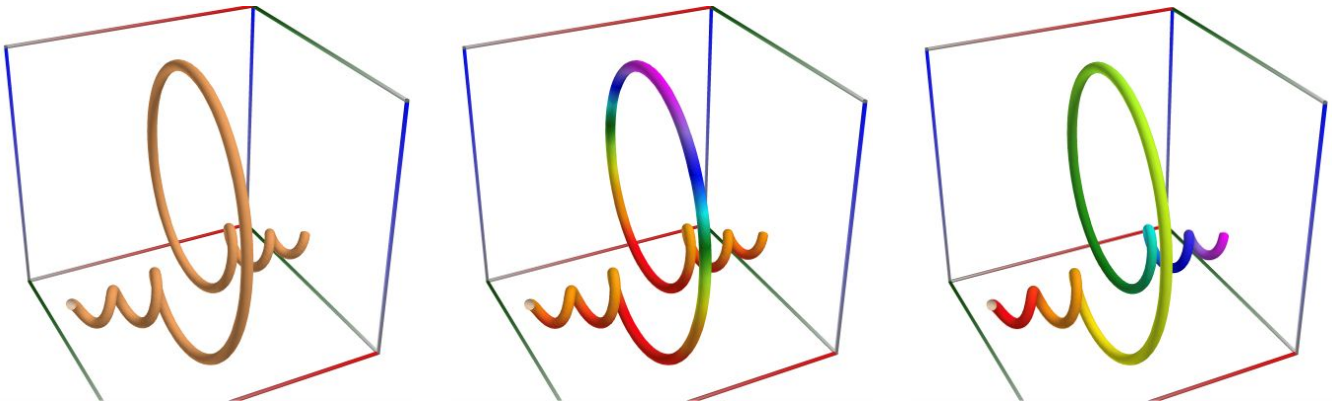
The uniform distribution is a continuous probability distribution on the support interval $[a, b]$ with finite $a < b$. See also Wikipedia [1193], MathWorld [838], BoostMath [89], Witkovský [1478], R (Statistical System) [536].

Returns $C_X(t)$, the characteristic function of a random variable X, following a uniform distribution:

$$C_X(t) = \frac{e^{itb} - e^{ita}}{it(b-a)}, \quad \text{for } t \neq 0, 0 \text{ otherwise.}$$

The example below uses the following code in C#

```
double a = 0.0;
double b = 1.0;
Complex i1 = Complex.ImaginaryOne;
Complex fz = 1.0;
if (t != 0.0)
{
    fz = Complex.Exp(i1 * t * b) - Complex.Exp(i1 * t * a);
    fz /= (i1 * t * (b - a));
}
var y = fz.Real;
var z = -fz.Imaginary;
var x = t;
```



Left figure: Characteristic function of the uniform distribution, with $a = 0.0$, $b = 1.0$, $t_0 = -20.0$ and $t_1 = 20.0$. Perspective camera.

24.3.2 Normal distribution

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Path_Func_CfNormal.3D.xml (D02a-c).

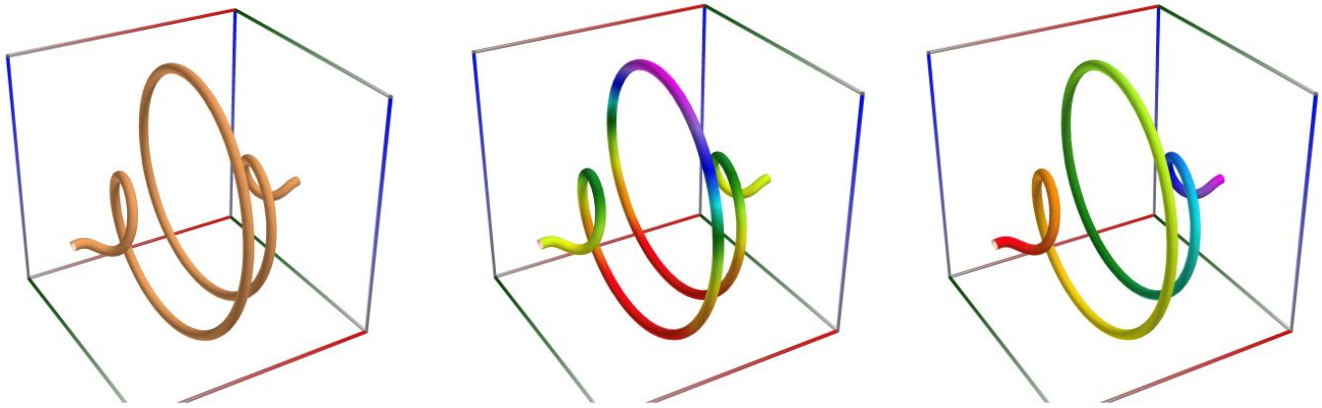
The normal distribution is a continuous probability distribution with mean $\mu \in \mathbb{R}$, standard deviation $\sigma > 0$, and the support interval $(-\infty, +\infty)$. See also Wikipedia [1166], MathWorld [833], BoostMath [67], R (Statistical System) [526], Mpmath [543], Mpmath [539].

Returns $C_X(t)$, the characteristic function of a random variable X, following a normal distribution:

$$C_X(t) = \exp\left(i\mu t - \tfrac{1}{2}\sigma^2 t^2\right).$$

The example below uses the following code in C#

```
double mu = 10.0;
double sigma = 1.0;
Complex i1 = Complex.ImaginaryOne;
Complex fz = Complex.Zero;
fz = Complex.Exp(i1 * t * mu - 0.5 * sigma * sigma * t * t);
var y = fz.Real;
var z = -fz.Imaginary;
var x = t;
```



Left figure: Characteristic function of the normal distribution, with $\mu = 10$, $\sigma = 1$, $t_0 = -2.0$ and $t_1 = 2.0$. Perspective camera.

24.3.3 Chi-squared distribution

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_Path_Func_CfChiSquared.3D.xml (D03a-c).

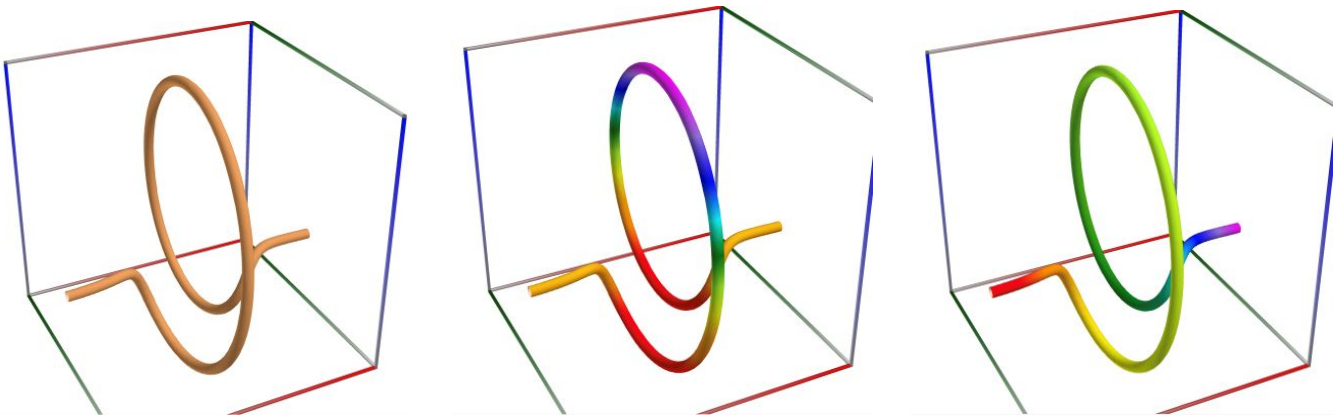
The chi-squared distribution is a continuous probability distribution with $n > 0$ degrees of freedom and the support interval $(0, +\infty)$. See also Wikipedia [1148], MathWorld [796], BoostMath [53], Witkovský [1464], R (Statistical System) [521].

Returns $C_X(t)$, the characteristic function of a random variable X, following a chi-squared distribution:

$$C_X(t) = (1 - 2it)^{-n/2}.$$

The example below uses the following code in C#

```
double nu = 100.0;
Complex i1 = Complex.ImaginaryOne;
Complex fz = Complex.Zero;
fz = Complex.Pow(1 - 2 * i1 * t, -nu / 2);
var x = fz.Real;
var y = -fz.Imaginary;
var z = t;
```



Left figure: Characteristic function of the chi-squared distribution, with $n = 100$, $t_0 = -0.2$ and $t_1 = 0.2$. Perspective camera.

24.3.4 Beta distribution

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Path_Func_CfBeta.3D.xml (D04a-c).

The beta distribution is a continuous probability distribution with parameters $a > 0$, $b > 0$, and the support interval $(0, 1)$. See also Wikipedia [1146], MathWorld [795], BoostMath [51], Witkovský [1463], R (Statistical System) [519].

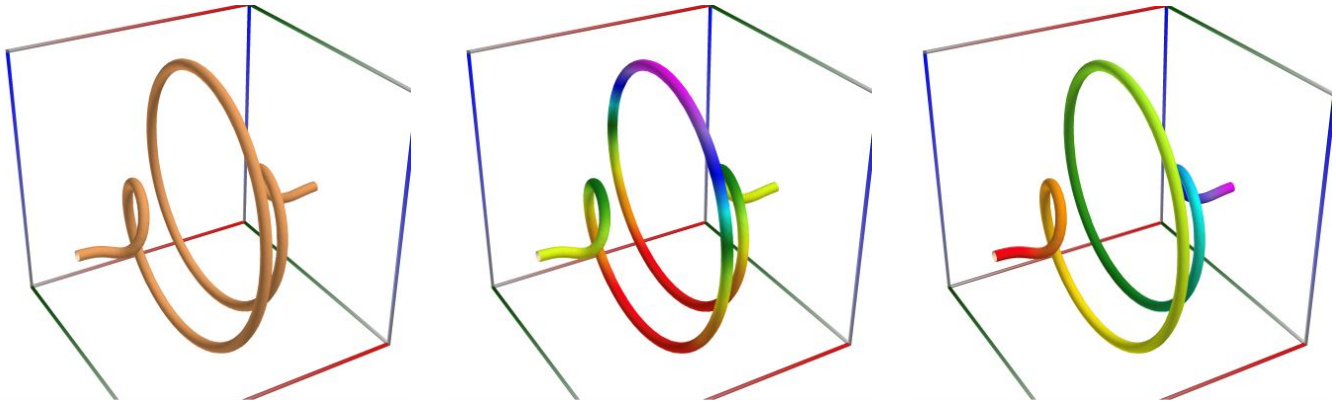
Returns $C_X(t)$, the characteristic function of a random variable X, following a beta distribution:

$$C_X(t) = {}_1F_1(a, a + b; it).$$

where ${}_1F_1()$ is Kummer's confluent hypergeometric function (of the first kind).

The example below uses the following code in C#

```
double a = 10.0;
double b = 20.0;
Complex i1 = Complex.ImaginaryOne;
var fz = dcplx.zero();
fz = cmath53.hyperg_1f1(a, a + b, i1 * t);
var y = fz.Real;
var z = -fz.Imaginary;
var x = t;
```



Left figure: Characteristic function of the beta distribution, with $a = 10$, $b = 20$, $t_0 = -40.0$ and $t_1 = 40.0$. Perspective camera.

24.3.5 F distribution

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Path_Func_CfFisherF.3D.xml (D05a-c).

The Fisher F-distribution is a continuous probability distribution with $m > 0$ and $n > 0$ degrees of freedom, and the support interval $(0, +\infty)$. See also Wikipedia [1151], MathWorld [799], BoostMath [56], Witkovský [1466], R (Statistical System) [523], Abramowitz and Stegun. [3], Butler and Paolella [166], Chattamvelli and Jones [175], Witkovský [1460].

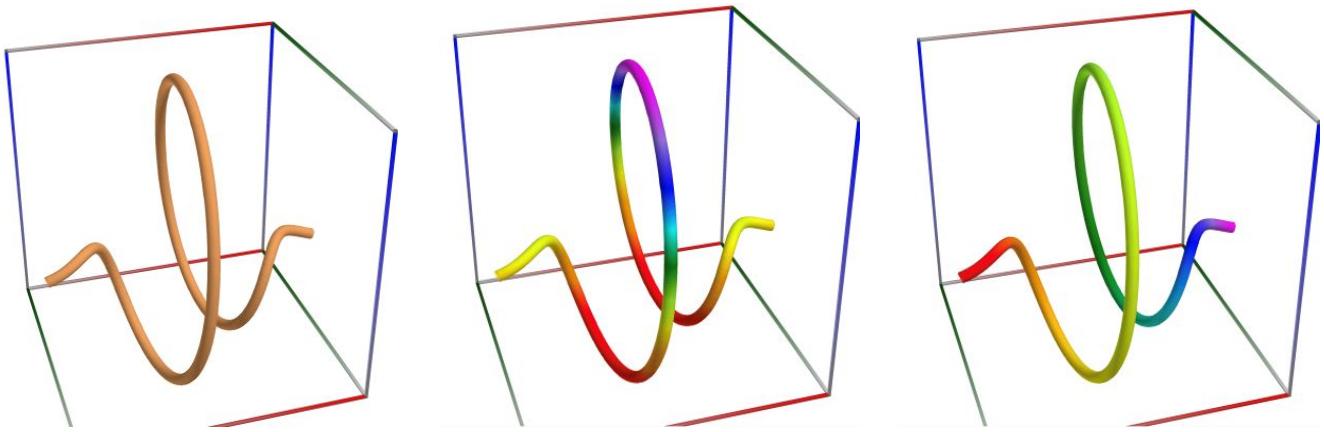
Returns $C_X(t)$, the characteristic function of a random variable X, following a central Fisher F distribution:

$$C_X(t) = \int_0^\infty e^{itx} \text{pdf}_X(x) \text{d}x = \frac{\Gamma(m/2 + n/2)}{\Gamma(n/2)} U\left(\frac{m}{2}, 1 - \frac{n}{2}, -\frac{n}{m} it\right)$$

where $U(\cdot)$ denotes the confluent hypergeometric function of the second kind.

The example below uses the following code in C#

```
double nu = 21.0;
double mu = 40.0;
double G = ctx.gamma(mu / 2 + nu / 2) / ctx.gamma(nu / 2);
Complex i1 = Complex.ImaginaryOne;
var fz = dcplx.zero();
fz = G * cmath53.hyperg_u(mu / 2, 1 - nu / 2, -(nu / mu) * i1 * t);
var y = fz.Real;
var z = -fz.Imaginary;
var x = t;
```



Left figure: Characteristic function of the Fisher F-distribution, with $m = 40$, $n = 20$, $t_0 = -20.0$ and $t_1 = 20.0$. Perspective camera.

24.3.6 Non-central Chi-squared distribution

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_Path_Func_CfChiSquaredNc.3D.xml (D06a-c).

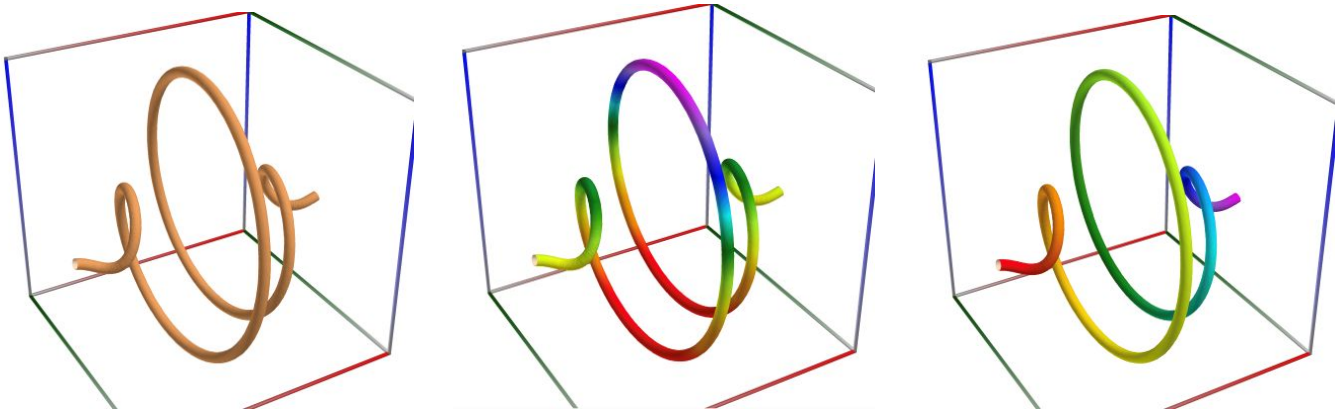
The noncentral chi-square distribution is a continuous probability distribution with degrees of freedom $n > 0$, noncentrality parameter λ_1, and support interval $(0, \infty)$. See also Wikipedia [1164], MathWorld [808], Patnaik [464], Penev and Raykov [466], Wang and Gray [793], Winterbottom [1454], BoostMath [64], Witkovský [1474], Johansson [386], R (Statistical System) [516], Yu [1498].

Returns $C_X(t)$, the characteristic function of a random variable X, following a non-central chi-squared distribution:

$$C_X(t) = \exp\left(\frac{i\lambda t}{1-2it}\right)(1-2it)^{-n/2}.$$

The example below uses the following code in C#

```
double nu = 10.0;
double lambda1 = 50.0;
Complex i1 = Complex.ImaginaryOne;
Complex fz = Complex.Zero;
fz = Complex.Pow(1 - 2 * i1 * t, -nu / 2);
fz *= Complex.Exp((i1 * t * lambda1) / (1 - 2 * i1 * t));
var y = fz.Real;
var z = -fz.Imaginary;
var x = t;
```



Left figure: Characteristic function of the noncentral chi-square distribution, with $\lambda = 50$, $n = 10$, $t_0 = -0.2$ and $t_1 = 0.2$. Perspective camera.

24.3.7 Binomial distribution

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D07a_Path_Func_CfBinomial.3D.xml (D07a-c).

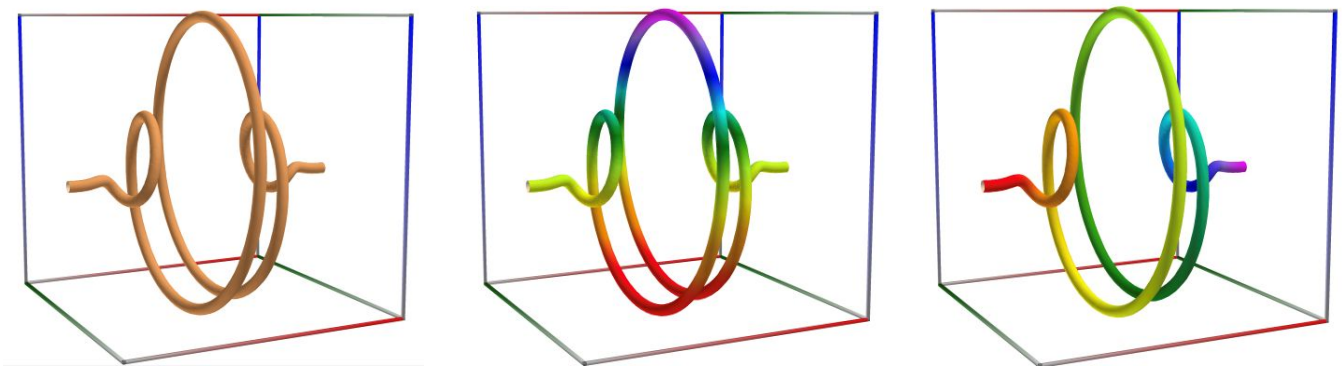
The binomial distribution is a discrete (lattice) probability distribution with number of trials $n \geq 0$ and success probability $0 \leq p \leq 1$. See also Wikipedia [1173], MathWorld [816], BoostMath [71], Witkovský [1482], R (Statistical System) [530].

Returns $C_X(t)$, the characteristic function of a random variable X, following an binomial distribution:

$$C_X(t) = \left(Pe^{it} + Q\right)^n .$$

The example below uses the following code in C#

```
double n1 = 20.0;
double p = 0.5;
Complex i1 = Complex.ImaginaryOne;
Complex fz = Complex.Zero;
fz = Complex.Pow(1 - p + p * Complex.Exp(i1 * t), n1);
var x = fz.Real;
var y = -fz.Imaginary;
var z = t;
```



Left figure: Characteristic function of the binomial distribution, with $p = 0.5$, $n = 20$, $t_0 = -2.0$ and $t_1 = 2.0$. Perspective camera.

24.3.8 Hypergeometric distribution

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D08a_Path_Func_CfHypergeo.3D.xml (D08a-c).

The hypergeometric distribution is a discrete (lattice) probability distribution with k successes (random draws for which the object drawn has a specified feature) in $n \in \{0, 1, 2, \ldots, N\}$ draws, without replacement, from a finite population of size $N \in \{0, 1, 2, \ldots\}$‘ that contains exactly $K \in \{0, 1, 2, \ldots, N\}$ objects with that feature, wherein each draw is either a success or a failure, and the support interval $(\max(0, n + K - N), \min(K, n))$. See also Wikipedia [1179], MathWorld [824], BoostMath [78], R (Statistical System) [532], Berkopec [31], Johnson *et al.* [389] page 251.

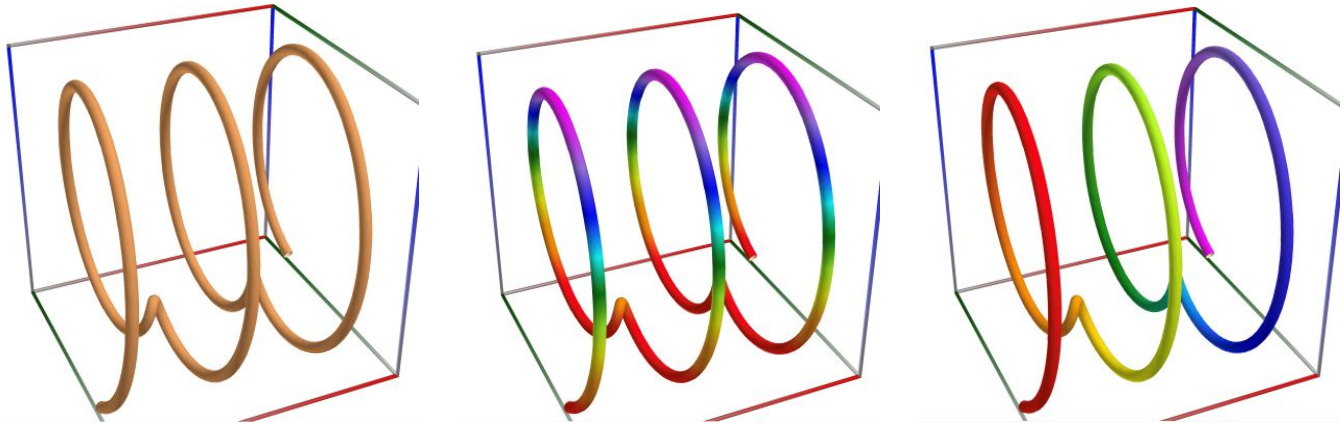
Returns $C_X(t)$, the characteristic function of a random variable X, following an hypergeometric distribution:

$$C_X(t) = {}_2F_1(-n, -K; N - K - n + 1; e^{it}) \binom{N-K}{n} \Big/ \binom{N}{n}.$$

where ${}_2F_1(\cdot)$ is the Gauss hypergeometric function.

The example below uses the following code in C#

```
int N = 50;
int K = 16;
int n1 = 12;
Complex i1 = Complex.ImaginaryOne;
var fz = dcplx.zero();
fz = cmath53.hyperg_2f1(-n1, -K, N - K - n1 + 1, cmath53.exp(i1 * t));
fz = fz * ctx.binomial(N - K, n1) / ctx.binomial(N, n1);
var x = fz.Real;
var y = -fz.Imaginary;
var z = t;
```



Left figure: Characteristic function of the hypergeometric distribution, with $N = 50$, $K = 16$, $n = 12$, $t_0 = -10.0$ and $t_1 = 10.0$. Perspective camera.

24.4 Helices and related curves traced on cylinders, cones and spheres

See also: https://en.wikipedia.org/wiki/Helix

See also: https://mathworld.wolfram.com/Helix.html

See also: https://mathcurve.com/courbes3d.gb/helice/helice.shtml

The helices are the curves the tangents of which form a constant angle a with respect to a fixed plane (P0), or a fixed direction d (orthogonal to (P0)). Therefore, the notion of helix in mathematics is more connected to a mountain road with constant slope than to the helix of a boat!

Examples (note that the helices are described by their base or the surface on which they are traced):

- the straight line (lines on a surface are therefore helices of this surface)
- the cylindrical helix (base = circle)
- the conical helix (traced on a vertical cone of revolution, base = logarithmic spiral)
- the elliptic helix (base = ellipse)
- the spherical helix (traced on a sphere, base = epicycloid)
- the helix of the paraboloid (base = involute of a circle).
- the helix of the one-sheeted hyperboloid (traced on the one-sheeted hyperboloid)
- the striction line of the milk carton

NOTE: the item “H. Stop of y, v [STOP2]” determines the diameter of the helix.

24.4.1 Cylindrical helix

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Path_SpiralCylindrical.3D.xml (D01a-f).

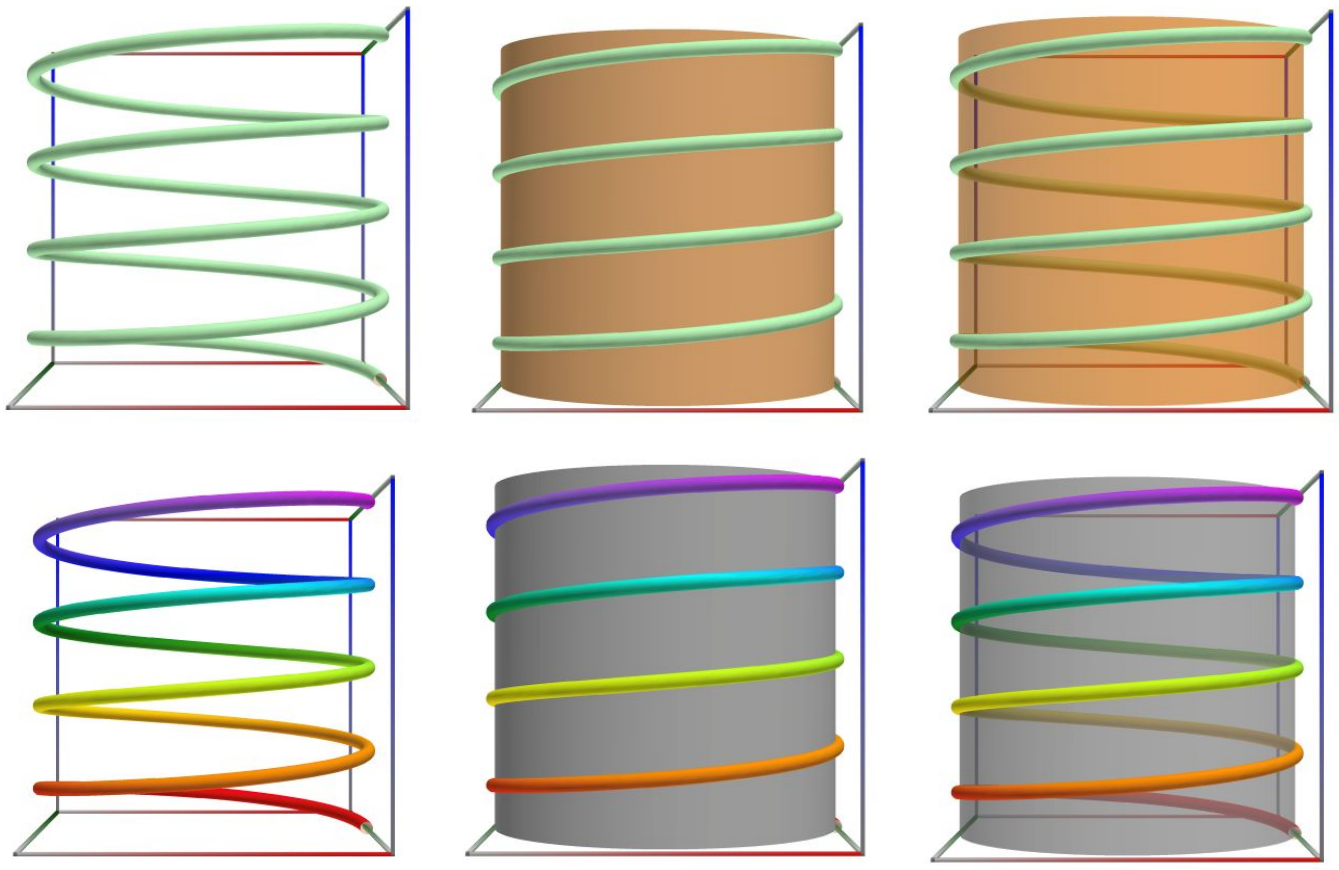
See also: https://mathcurve.com/courbes3d.gb/helicecirculaire/helicecirculaire.shtml

The cylindrical helix can be defined as a helix traced on a vertical cylinder of revolution, or a rhumb line of this cylinder (i.e., in both cases, a curve forming a constant angle with respect to the axis of the cylinder), or a geodesic of this cylinder (in other words, a curve that becomes a line when the cylinder is developed) or a solenoid with linear bore. Intrinsic characterization: constant curvature and torsion.

The radius of the helix is a, and its shift is (it is the distance between two consecutive coils) and b is sometimes called reduced shift of the helix. The angle of the helix is the constant angle (equal to) formed by its tangent with respect to any plane orthogonal to Oz. The helix is right-handed when e = 1 (it "goes up" counterclockwise and an observer located outside of it sees it going up from left to right) and left-handed when e = - 1 (it "goes up" clockwise).

The example below uses the following code in C#

```
double a = 0.2;
// double b = 1.0;
double b = 0.2;
var x = a * Math.Cos(t);
var y = a * Math.Sin(t);
var z = b * t;
```



24.4.2 Conical helix based on Archimedes spiral (spiral of Pappus)

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Path_SpiralAConical.3D.xml (D02a-f).

See also: https://en.wikipedia.org/wiki/Conical_spiral

See also: https://mathworld.wolfram.com/ConicalSpiral.html

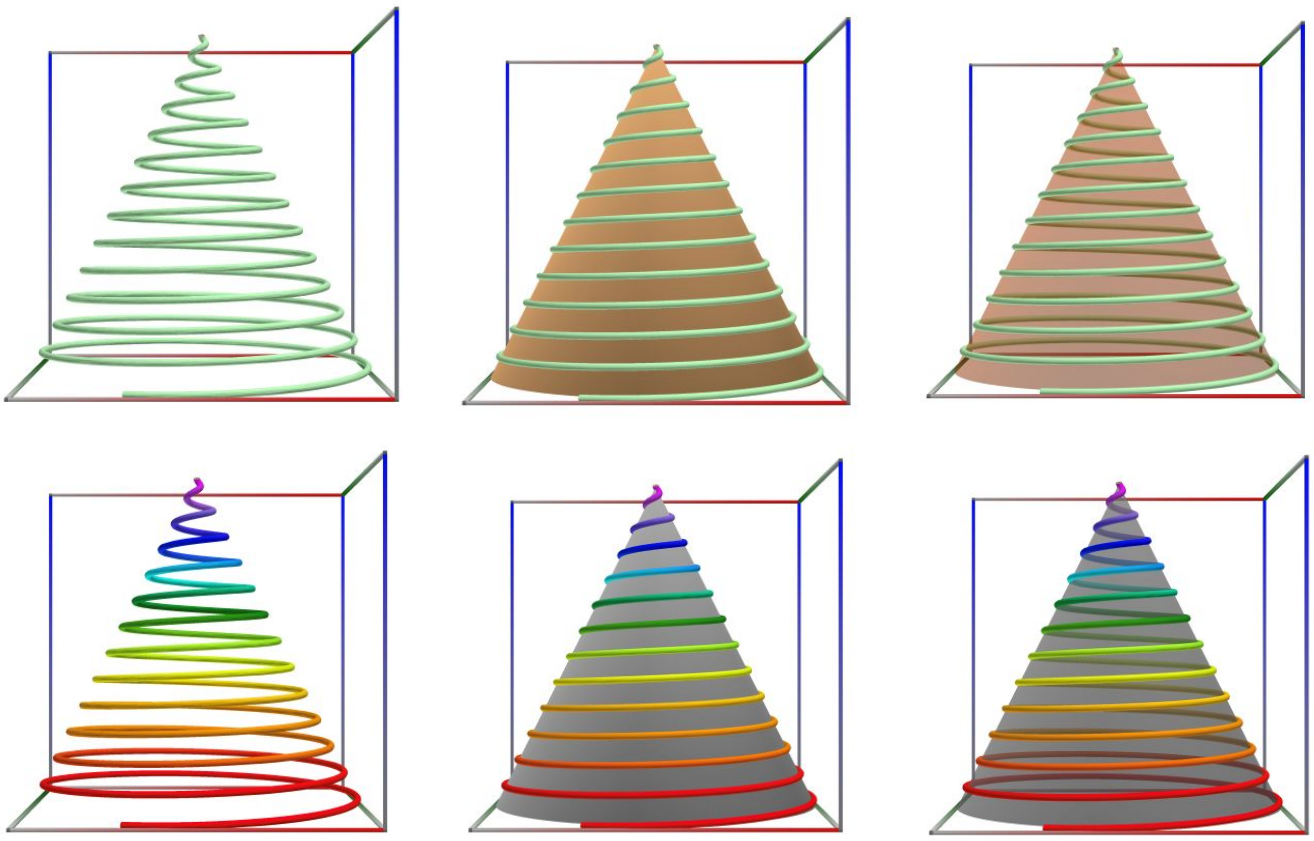
See also: https://mathcurve.com/courbes3d.gb/helicecirculaire/helicecirculaire.shtml

The conical helix can be defined as a helix traced on a cone of revolution (i.e. a curve forming a constant angle with respect to the axis of the cone), or a rhumb line of this cone (i.e. a curve forming a constant angle with the meridians); it is not a geodesic of the cone. In concrete terms, we get a conical helix when we trace a path with constant slope on a cone placed vertically.

The projection on xOy is a logarithmic spiral (), which is also the locus of the intersection between the tangents and xOy; the curve obtained by developing the cone is also a logarithmic spiral. As for all helices, it is a geodesic of the vertical cylinder based on the aforementioned spiral, projection of the curve on xOy. The principal normal is always perpendicular to Oy. The radii of curvature and torsion are proportional to z. The helix is right-handed when (it "goes up" clockwise) and left-handed when (it "goes up" counterclockwise).

The example below uses the following code in C#

```
double a = 1.0;
double phi = t * Math.PI;
double rphi = a * phi;
var x = (rphi * Math.Cos(phi));
var y = (rphi * Math.Sin(phi));
//var z = (0.2 * t);
var z = (1.2 * t);
```



24.4.3 Conical helix based on Fermat's spiral

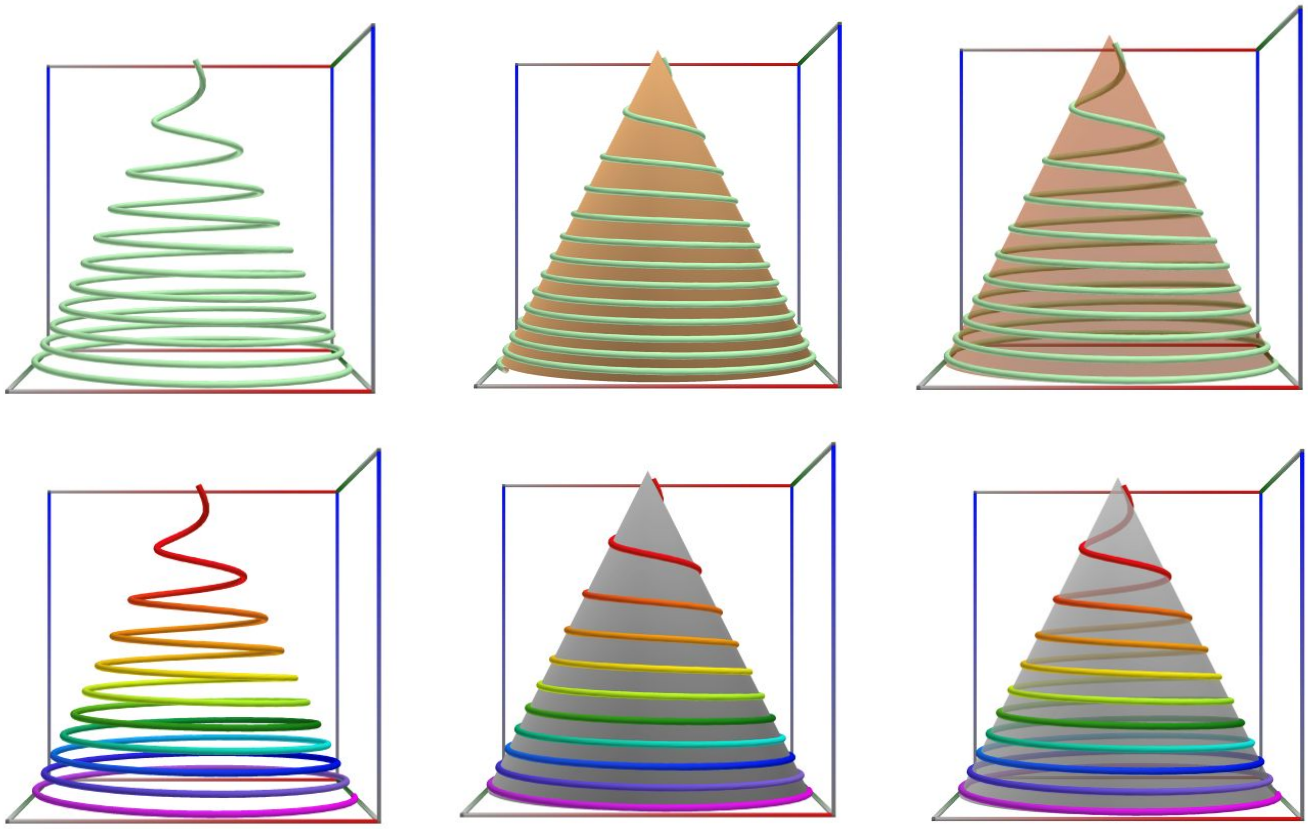
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_Path_SpiralFConical.3D.xml (D03a-f).

See also: https://en.wikipedia.org/wiki/Conical_spiral

Conical helix based on Fermat's spiral

The example below uses the following code in C#

```
double a = 1.0;
double m = 0.2 / Math.PI;
double phi = t * Math.PI;
double rphi = a * Math.Sqrt(phi);
var x = (rphi * Math.Cos(phi));
var y = (rphi * Math.Sin(phi));
var z = (m * rphi);
```



24.4.4 Conical helix based on the logarithmic spiral (Concho-Spiral)

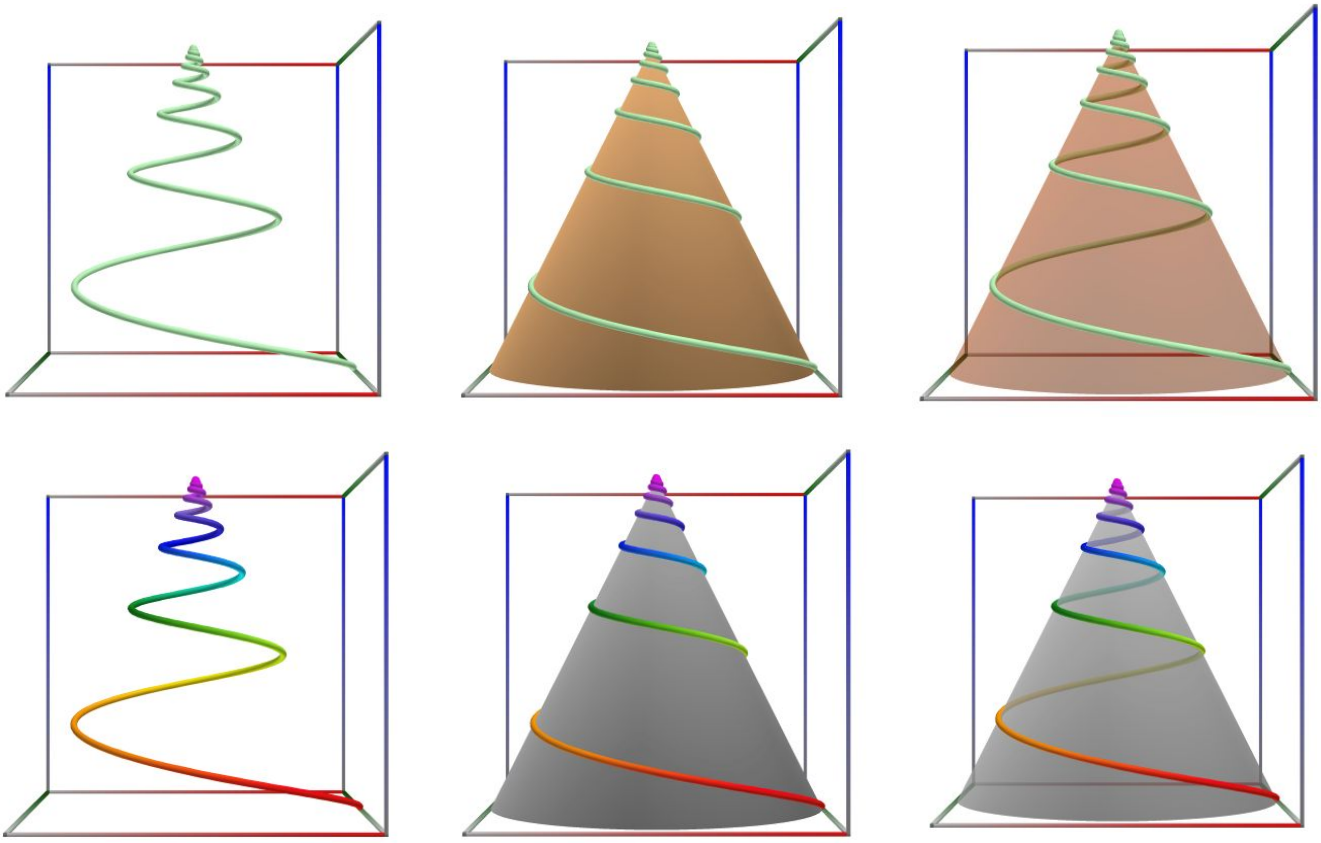
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Path_SpiralLConical.3D.xml (D04a-f).

See also: https://mathworld.wolfram.com/Concho-Spiral.html

See also: https://en.wikipedia.org/wiki/Conchospiral

The example below uses the following code in C#

```
double k = 0.1;
double a = 1.0;
double m = 0.2 / Math.PI;
double phi = t * Math.PI;
double rphi = a * Math.Exp(k * phi);
var x = (rphi * Math.Cos(phi));
var y = (rphi * Math.Sin(phi));
var z = (m * rphi);
```



24.4.5 Rhumb line of the sphere

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Path_SphericalRhumbline.3D.xml (D05a-f).

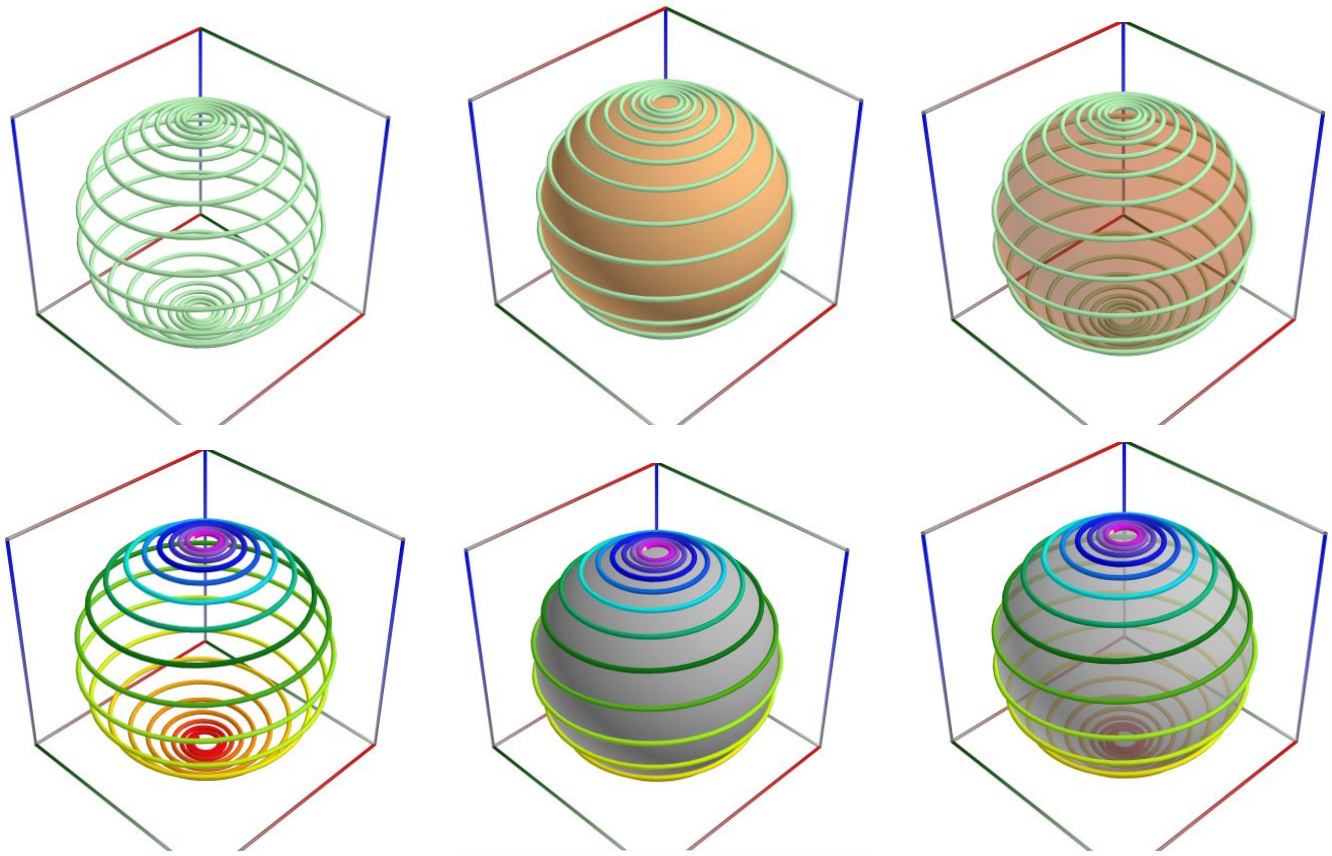
See also: https://mathcurve.com/courbes3d.gb/loxodromie/sphereloxodromie.shtml

The rhumb lines of the sphere, associated to a given axis, are the curves that form a constant angle with the parallel (or the meridians). Do not mistake the rhumb lines for the spherical helices, that form a constant angle with the equatorial plane, nor for the clelias.

The rhumb lines correspond to the straight lines in Mercator coordinates ; in other words, on the maps of the Earth that use the Mercator projection, the rhumb lines are represented by straight lines. The angle a that the images of the rhumb lines form on the map with respect to the horizontal is the same as the angle they form on the sphere with respect to the parallels. If we know the geographic coordinates and of two points, the angle a associated to the shortest rhumb line joining these two points is obtained by the formula: and the length is given by: . The notion of rhumb line is opposed to that of geodesic, shortest path joining two points on the sphere, which is an arc of a great circle.

The example below uses the following code in C#

```
double R = 1.0;
double alpha = 0.05;
double k = Math.Tan(alpha);
var x = R * Math.Cos(t) / Math.Cosh(k * t);
var y = R * Math.Sin(t) / Math.Cosh(k * t);
var z = R * Math.Tanh(k * t);
```



24.4.6 Clelia

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_Path_SphericalClelia.3D.xml (D06a-f).

See also: https://mathcurve.com/courbes3d.gb/clelie/clelie.shtml

See also: https://en.wikipedia.org/wiki/Cl%C3%A9lie

The clelias are the loci of a point M on a meridian of a sphere rotating at constant speed w around the polar axis, the point M also moving at constant speed nw along this meridian. Therefore, physically, we obtain a clelia when peeling an orange or when rewinding regularly a spherical wool ball.

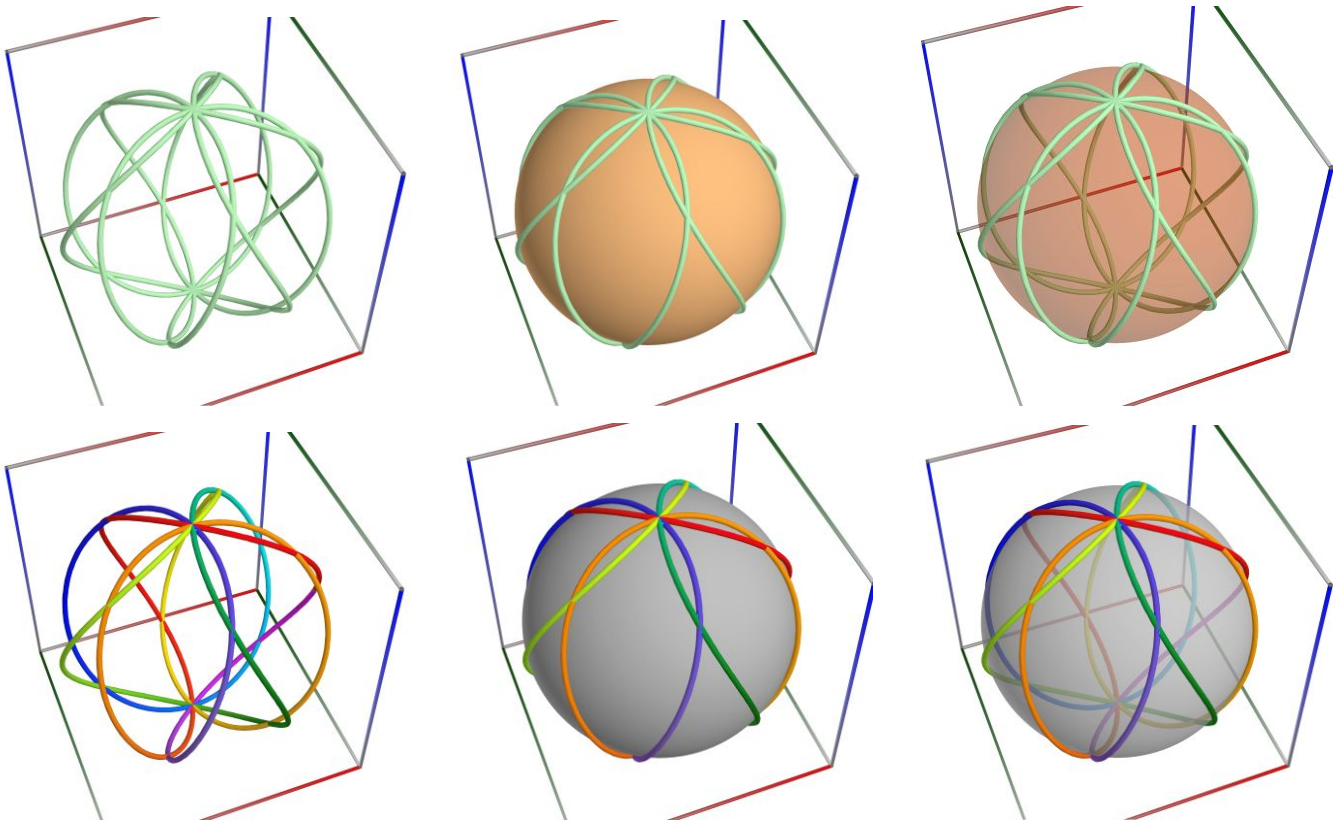
When n is rational with numerator p and denominator q:

In this case, the curve is composed of 2p patterns, obtained from the base pattern by rotations of axis Oz and angles $2k\pi/n$ and $p + 2k\pi/n$.

When p and q are odd, the curve is composed of p patterns, obtained from the base pattern by rotations of axis Oz and angles $2k\pi/n$.

The example below uses the following code in C#

```
double R = 1.0;
double n1 = 5.0 / 2.0;    // Example 1
var x = R * Math.Cos(n1 * t) * Math.Cos(t);
var y = R * Math.Cos(n1 * t) * Math.Sin(t);
var z = R * Math.Sin(n1 * t);
```



24.4.7 Spherical helix

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D07a_Path_SphericalHelix.3D.xml (D07a-f).

See also: https://mathworld.wolfram.com/SphericalSpiral.html

See also: https://mathworld.wolfram.com/SphericalHelix.html

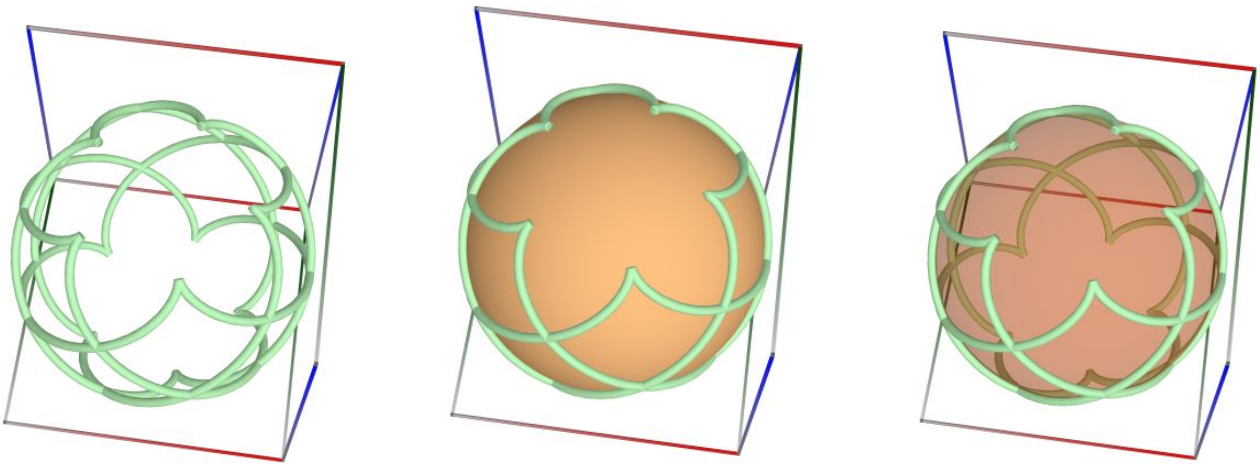
See also: https://mathcurve.com/courbes3d.gb/helicespheric/helicespheric.shtml

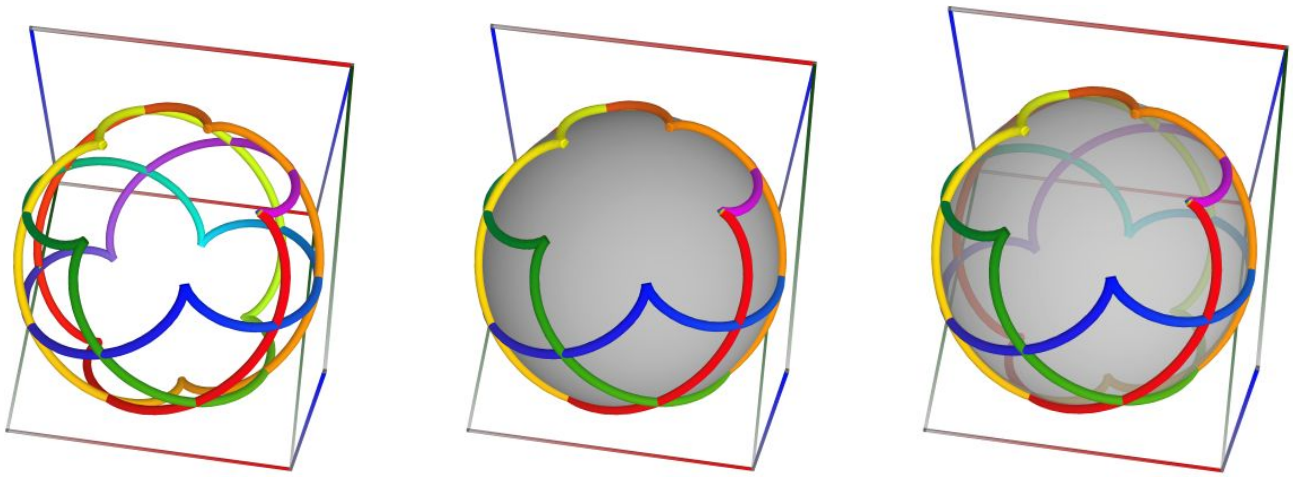
A spherical cycloid is the locus of a point on a circle rolling without slipping on a fixed circle, the angle between the two circles remaining constant equal to ; here, a is the radius of the fixed circle, that of the moving circle, and xOy the plane where lies the fixed circle.

When = 0, we get the hypocycloid, and when = , the epicycloid; apart from these two cases, the cycloid is traced on the sphere corresponding to both the base and the rolling circles, hence its name of spherical cycloid. The center W of this sphere is the point on Oz at height and its radius is . Therefore, the spherical cycloid is a roulette of the motion of a sphere over a sphere.

The example below uses the following code in C#

```
double q = 5.0 / 2.0;    // Example 1
//double q = 2.0 / 5.0;    // Example 2
//double q = 1.0 / 5.0;    // Example 3
double R = 1.0;
double k = q / (q + 2);
var x = R * (k * Math.Cos(t) * Math.Cos(k * t) + Math.Sin(t) * Math.Sin(k * t));
var y = R * (k * Math.Sin(t) * Math.Cos(k * t) - Math.Cos(t) * Math.Sin(k * t));
var z = R * Math.Sqrt(1 - k * k) * Math.Cos(k * t);
```





24.4.8 Satellite curve

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D08a_Path_SphericalSatellite.3D.xml (D08a-f).

See also: https://mathcurve.com/courbes3d.gb/satellite/satellite.shtml

See also: https://mathcurve.com/courbes3d.gb/capareda/capareda.shtml

The satellite curves are the various trajectories of a point M on a given great circle of a sphere rotating around one of its axes, while M has a uniform motion along the circle. These curves can also be seen as the trajectories of points on a circle in uniform rotation around an axis, this axis being itself in uniform rotation around an axis passing by the center of the circle.

The name of satellite curve comes from the fact that the trajectory, in the frame associated to the Earth, of a satellite in uniform circular rotation around the center of the Earth is such a curve: see for example this book (pages 177 to 181).

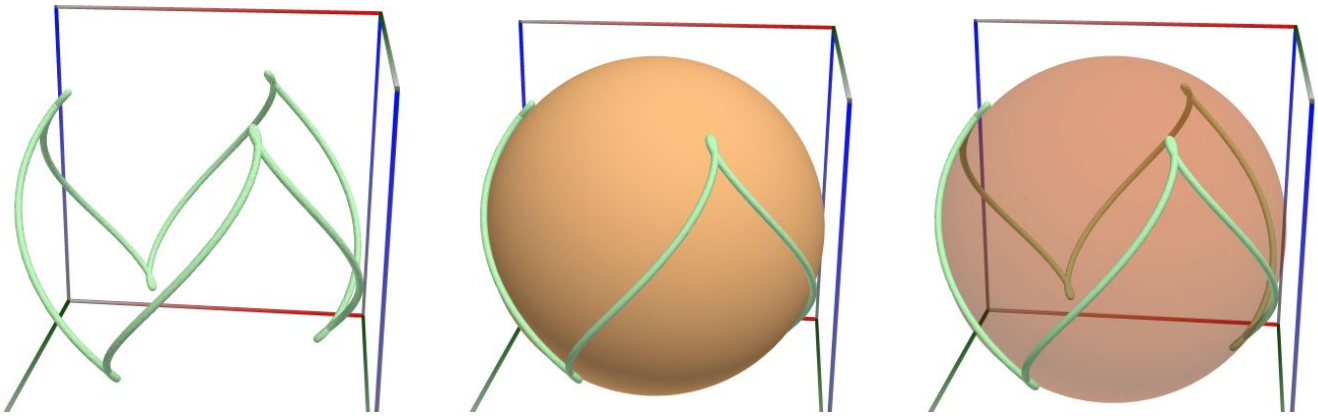
In the above parametrization, the sphere centered on O turns around Oz and the plane of the circle forms an angle with respect to xOy; k is the ratio of the speed of rotation of M on the circle over the speed of rotation of the sphere around its axis.

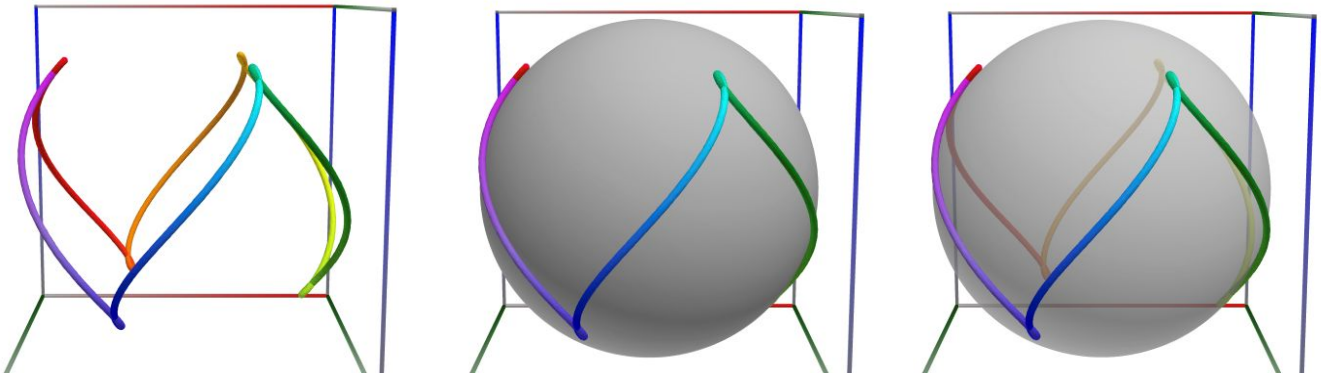
Special cases of satellite curves include:

- the clelias when the great circle meets the axis of rotation of the sphere ().
- the spherical helices when (in the second definition above, the circle rolls without slipping on a fixed circle); it is the case where the curve has cuspidal points.

The example below uses the following code in C#

```
double R = 1.0;
//double alpha = 1 * Math.PI / 2;  // This corresponds to clelias
double alpha = 3 * Math.PI / 4;
double k = 3.0 / 4.0;
//double k = -Math.Cos(alpha);  // This corresponds to spherical helices
var x = R * (Math.Cos(alpha) * Math.Cos(t) * Math.Cos(k * t) - Math.Sin(t) * Math.Sin(k * t));
var y = R * (Math.Cos(alpha) * Math.Sin(t) * Math.Cos(k * t) + Math.Cos(t) * Math.Sin(k * t));
var z = R * Math.Sin(alpha) * Math.Cos(k * t);
```





24.5 (Coil) springs

See also: https://en.wikipedia.org/wiki/Coil_spring

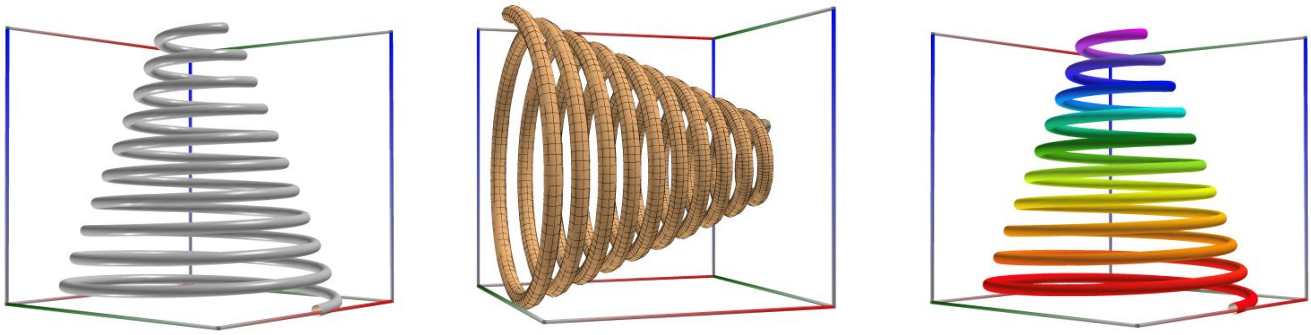
See also: https://en.wikipedia.org/wiki/Spring_(device)

24.5.1 Spring 1

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Path_Spring1.3D.xml (D01a-c).

The example below uses the following code in C#

```
var x = ((-3.75 * t + 100) * Math.Sin(t * Math.PI)) / 100;
var z = (20 * t) / 100;
var y = ((-3.75 * t + 100) * Math.Cos(t * Math.PI)) / 100;
```

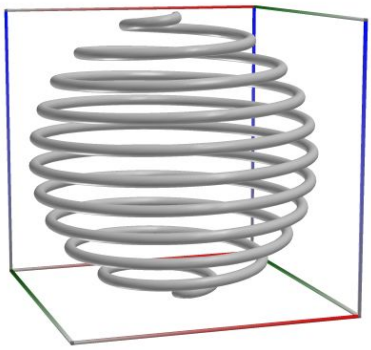


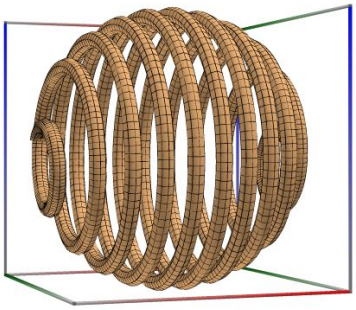
24.5.2 Spring 2

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Path_Spring2.3D.xml (D02a-b).

The example below uses the following code in C#

```
var x = (Math.Sqrt(10000 - 100 * t * t) * Math.Sin(t * Math.PI)) / 100;
var z = (20 * t) / 100;
var y = (Math.Sqrt(10000 - 100 * t * t) * Math.Cos(t * Math.PI)) / 100;
```



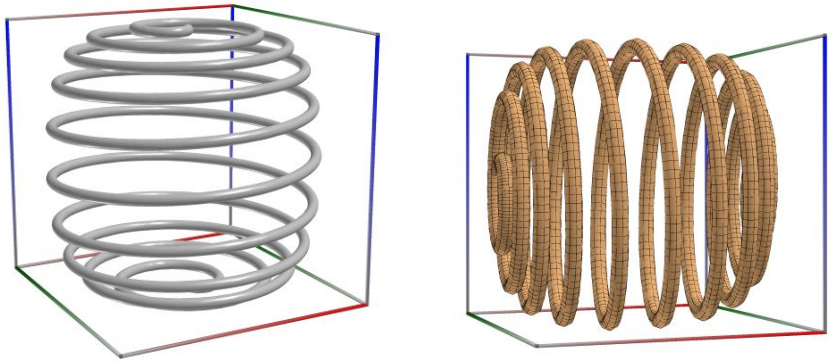


24.5.3 Spring 3

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Path_Spring4.3D.xml (D04a-b).

The example below uses the following code in C#

```
var x = (Math.Sqrt(10000 - 100 * t * t) * Math.Sin(t * Math.PI)) / 100;
var z = 209.22 * (Math.Tanh(0.1896 * t)) / 100;
var y = (Math.Sqrt(10000 - 100 * t * t) * Math.Cos(t * Math.PI)) / 100;
```

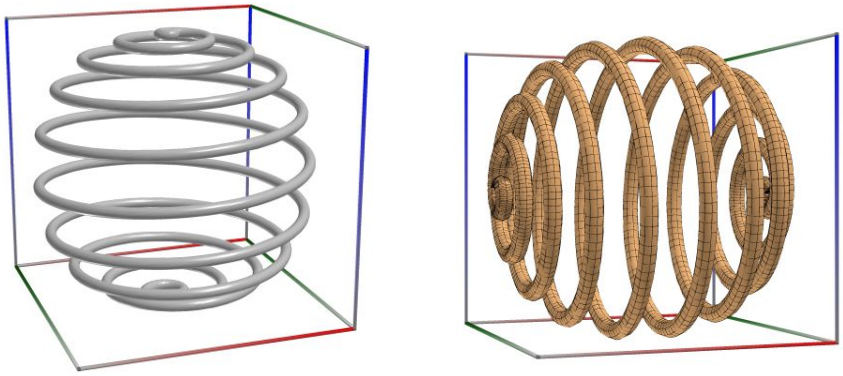


24.5.4 Spring 4

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Path_Spring4.3D.xml (D04a-b).

The example below uses the following code in C#

```
var x = (200 * Math.Cos(0.05 * t * Math.PI) * Math.Sin(t * Math.PI)) / 100;
var z = 209.22 * (Math.Tanh(0.1896 * t)) / 100;
var y = (200 * Math.Cos(0.05 * t * Math.PI) * Math.Cos(t * Math.PI)) / 100;
```

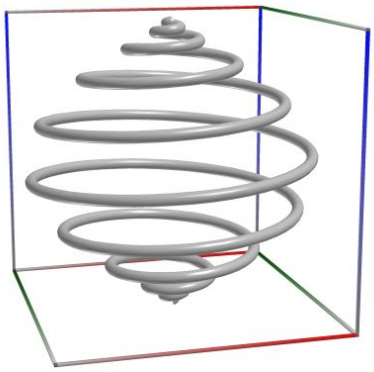


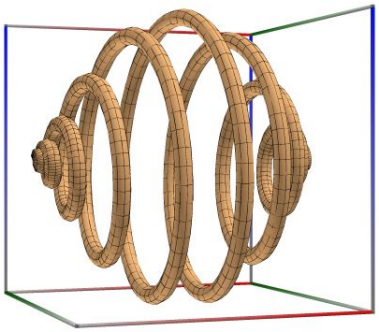
24.5.5 Spring 5

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Path_Spring5.3D.xml (D05a-b).

The example below uses the following code in C#

```
double e = Math.Exp(-(0.2 * t) * (0.2 * t));
var x = (200 * e * Math.Sin(t * Math.PI)) / 100;
var z = 209.22 * (Math.Tanh(0.1896 * t)) / 100;
var y = (200 * e * Math.Cos(t * Math.PI)) / 100;
```



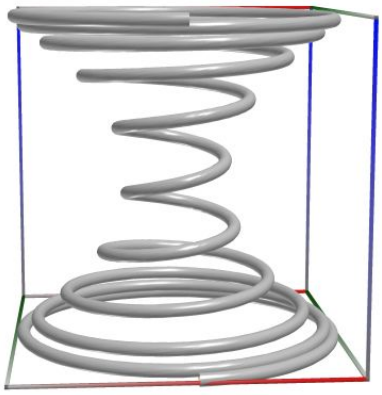


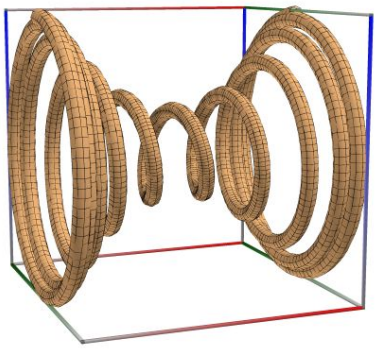
24.5.6 Spring 6

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_Path_Spring6.3D.xml (D06a-b).

The example below uses the following code in C#

```
double e = Math.Exp(-(0.2 * t) * (0.2 * t));
var x = ((200 - 140 * e) * Math.Sin(t * Math.PI)) / 100;
var z = 207.46 * (Math.Tanh(0.2 * t)) / 100;
var y = ((200 - 140 * e) * Math.Cos(t * Math.PI)) / 100;
```



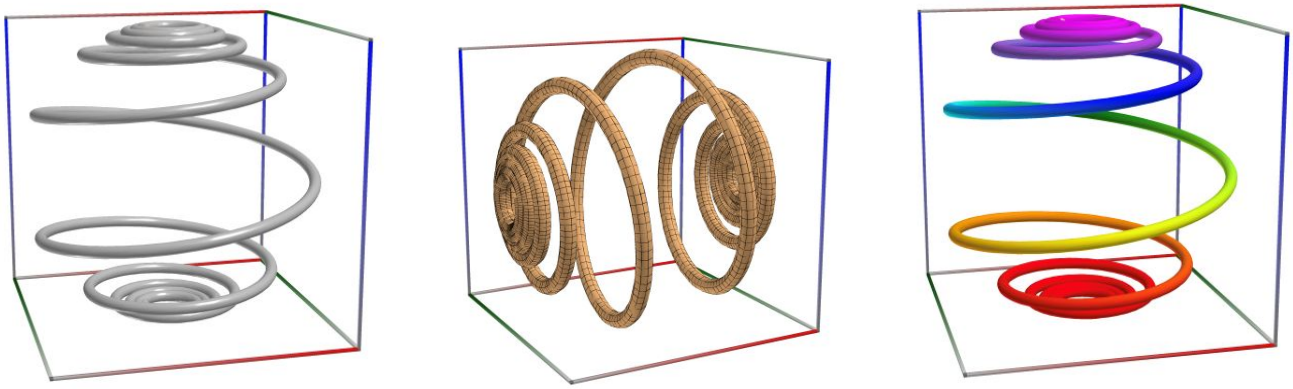


24.5.7 Spring 7

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D07a_Path_Spring7.3D.xml (D07a-b).

The example below uses the following code in C#

```
var x = (200 * Math.Cos(t) * Math.Cos(Math.Atan(0.15 * t))) / 100;
var z = -200 * Math.Sin(Math.Atan(0.15 * t)) / 100;
var y = (200 * Math.Sin(t) * Math.Cos(Math.Atan(0.15 * t))) / 100;
```

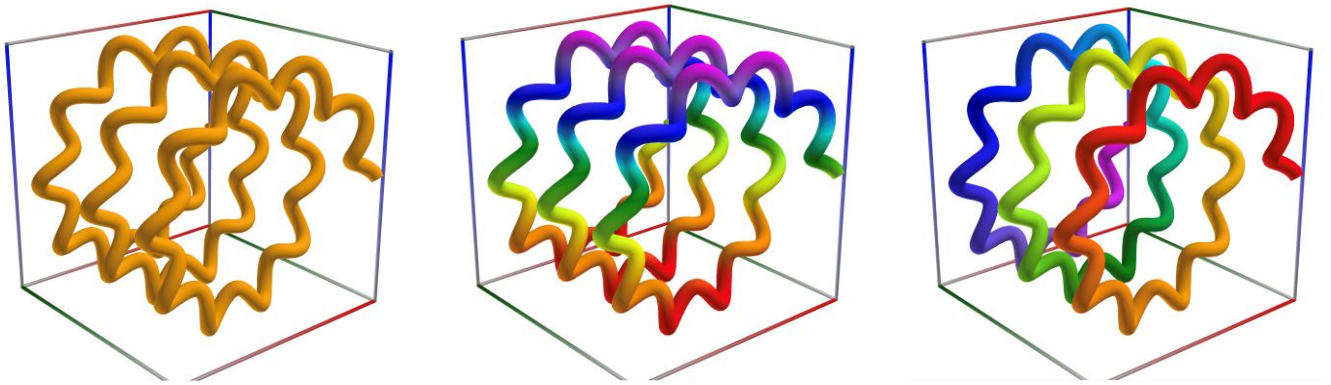


24.5.8 Spring 8

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D08a_Path_Spring8.3D.xml (D08a-b).

The example below uses the following code in C#

```
double R = 100.0;
double G = 160.0;
double D = 15.0;
double N = 10.0;
var x = (Math.Cos(t * Math.PI) * R + Math.Cos(t * Math.PI) * D * Math.Cos(N * t * Math.PI)) / 100;
var y = (D * Math.Sin(N * t * Math.PI) + 0.5 * G * t) / 100;
var z = (Math.Sin(t * Math.PI) * R + Math.Sin(t * Math.PI) * D * Math.Cos(N * t * Math.PI)) / 100;
```



24.6 General knots

24.6.1 Trefoil A

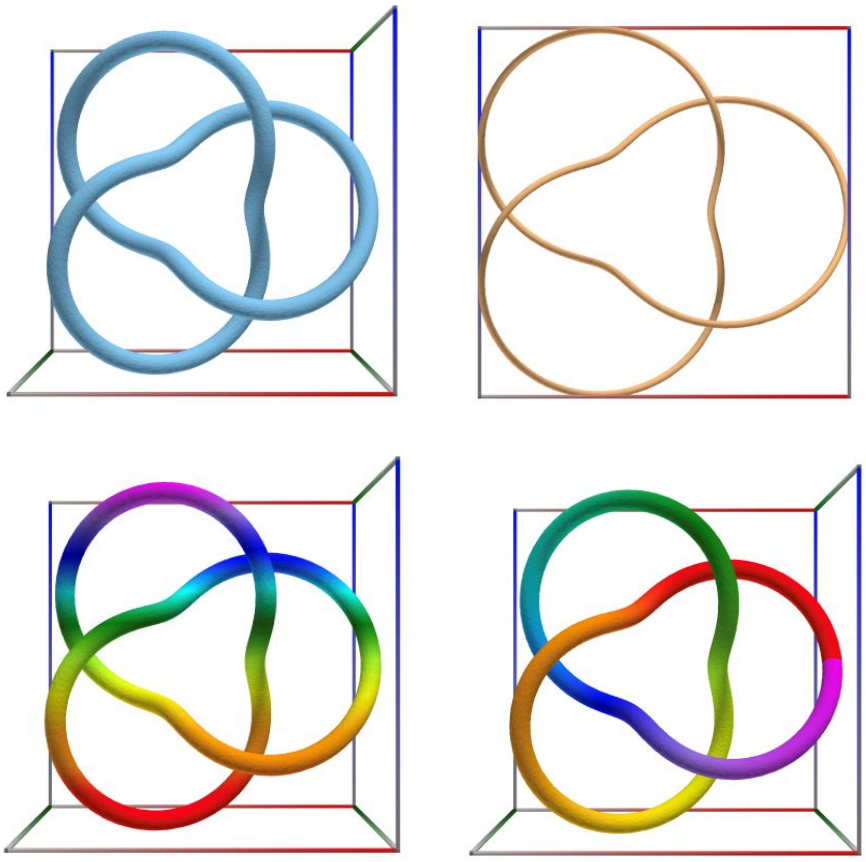
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Path_Trefoil_Knoten_A.3D.xml (D01a-d).

See also: https://en.wikipedia.org/wiki/Trefoil_knot

See also: https://katlas.org/wiki/T(3,2)

The example below uses the following code in C#

```
var x = (2 + Math.Cos(3 * t)) * Math.Cos(2 * t);
var y = (2 + Math.Cos(3 * t)) * Math.Sin(2 * t);
var z = Math.Sin(3 * t);
```



24.6.2 Trefoil B

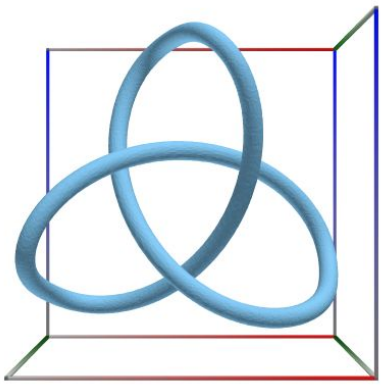
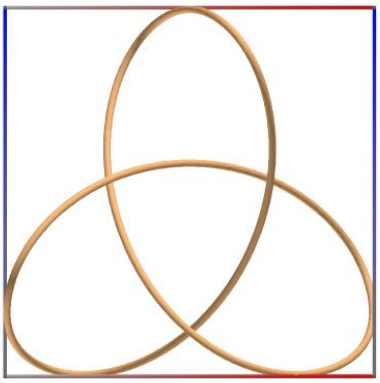
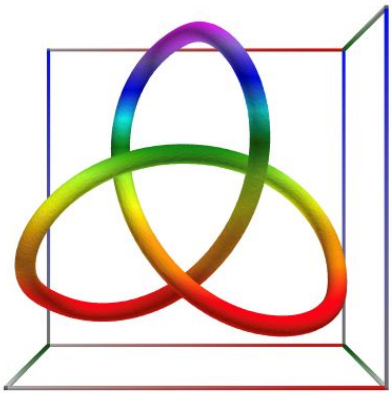
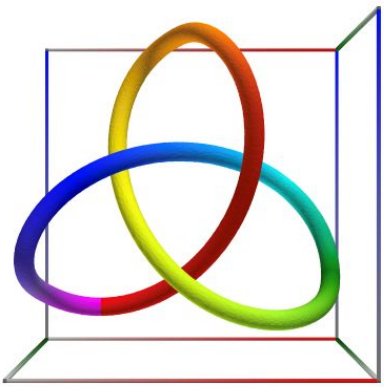
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Path_Trefoil_Knoten_B.3D.xml (D02a-d).

See also: https://en.wikipedia.org/wiki/Trefoil_knot

See also: https://katlas.org/wiki/T(3,2)

The example below uses the following code in C#

```
var x = (-10 * Math.Cos(t) - 2 * Math.Cos(5 * t) + 15 * Math.Sin(2 * t)) / 10;
var y = (-15 * Math.Cos(2 * t) + 10 * Math.Sin(t) - 2 * Math.Sin(5 * t)) / 10;
var z = (10 * Math.Cos(3 * t)) / 10;
```

24.6.3 Square knot

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_Path_SquareKnotP_P90T90.3D.xml (D03a-f).

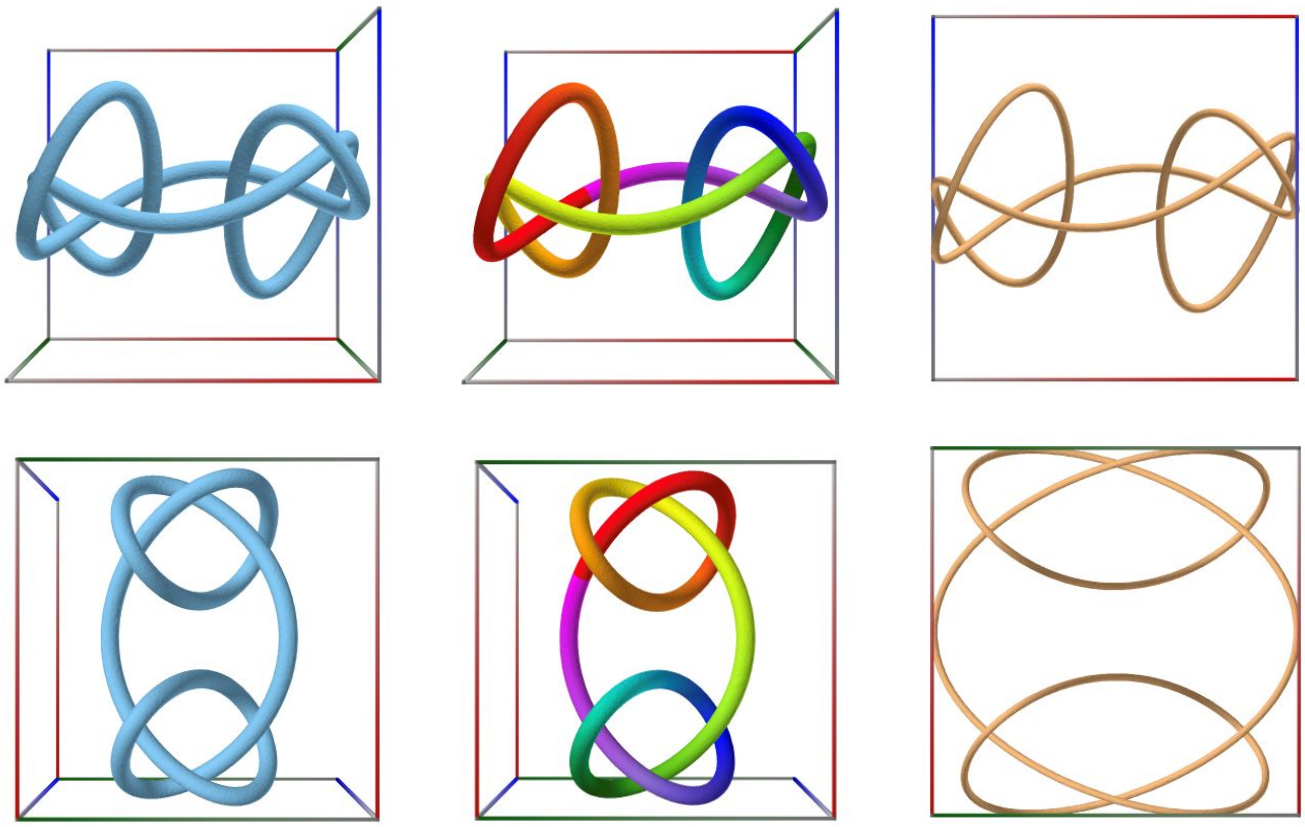
See also: https://en.wikipedia.org/wiki/Square_knot_(mathematics)

See also: https://mathworld.wolfram.com/SquareKnot.html

See also: https://mathcurve.com/courbes3d.gb/plat.vache/plat_vache.shtml

The example below uses the following code in C#

```
var x = (-22 * Math.Cos(t) - 128 * Math.Sin(t) - 44 * Math.Cos(3 * t) - 78 * Math.Sin(3 * t)) / 10;
var y = (11 * Math.Cos(t) - 43 * Math.Cos(3 * t) + 34 * Math.Cos(5 * t) - 39 * Math.Sin(5 * t)) / 10;
var z = (70 * Math.Cos(3 * t) - 40 * Math.Sin(3 * t) + 8 * Math.Cos(5 * t) - 9 * Math.Sin(5 * t)) / 10;
```



24.6.4 Granny knot

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Path_GrannyKnot.3D.xml (D04a-f).

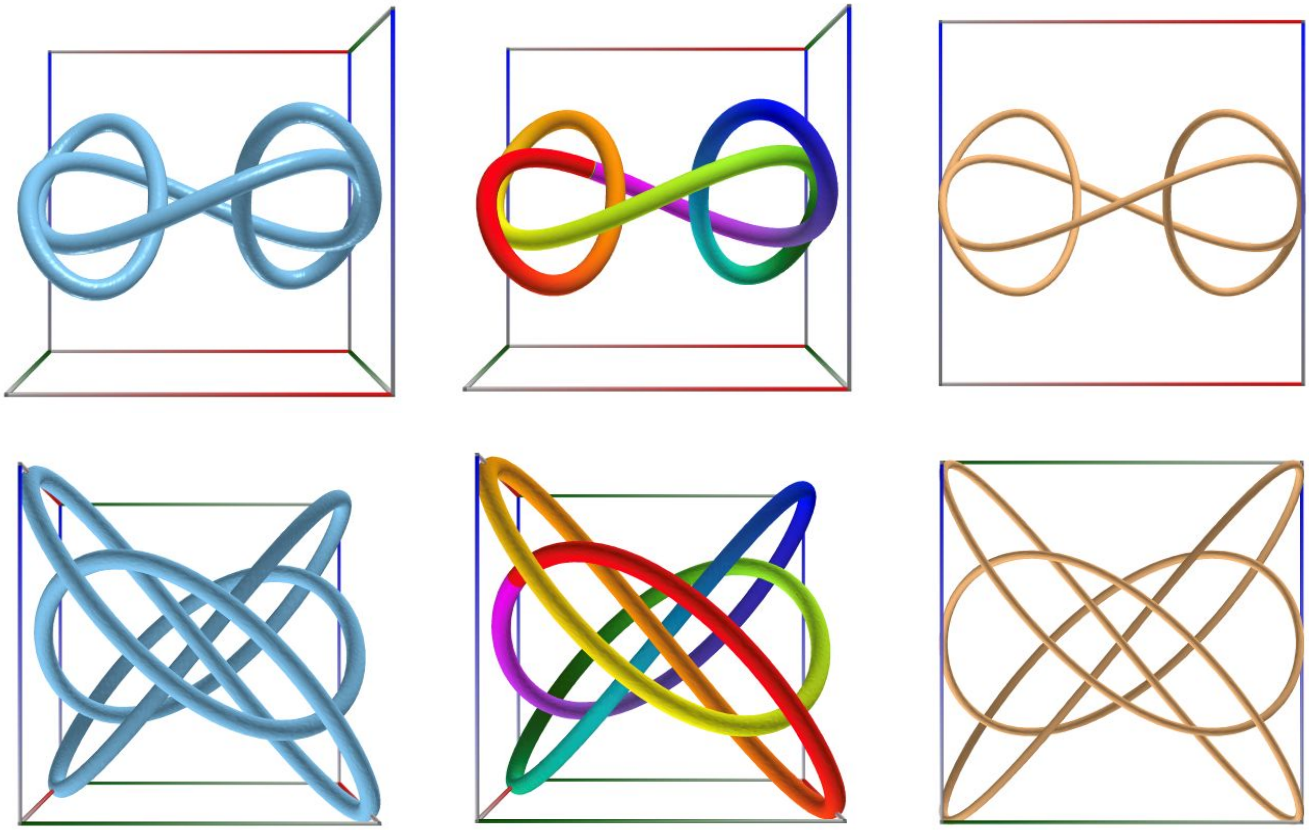
See also: https://en.wikipedia.org/wiki/Granny_knot_(mathematics)

See also: https://mathworld.wolfram.com/GrannyKnot.html

See also: https://mathcurve.com/courbes3d.gb/plat.vache/plat_vache.shtml

The example below uses the following code in C#

```
var x = (-22 * Math.Cos(t) - 128 * Math.Sin(t) - 44 * Math.Cos(3 * t) - 78 * Math.Sin(3 * t)) / 10;
var y = (-10 * Math.Cos(2 * t) - 27 * Math.Sin(2 * t) + 38 * Math.Cos(4 * t) + 46 * Math.Sin(4 * t)) / 10;
var z = (70 * Math.Cos(3 * t) - 40 * Math.Sin(3 * t)) / 10;
```



24.6.5 Cinquefoil Knot

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Path_CinquefoilKnot.3D.xml (D05a-d).

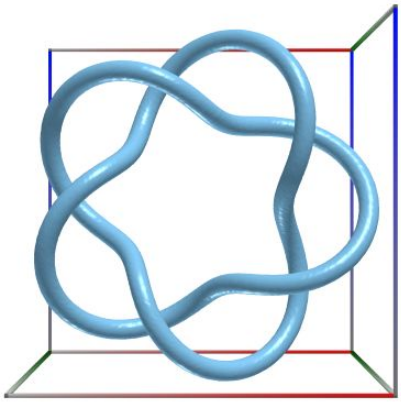
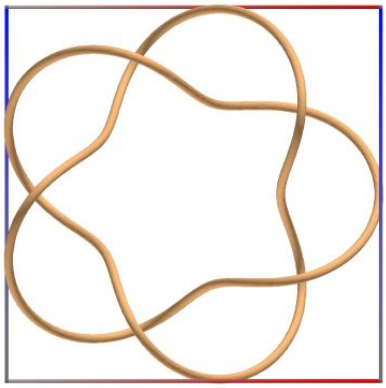
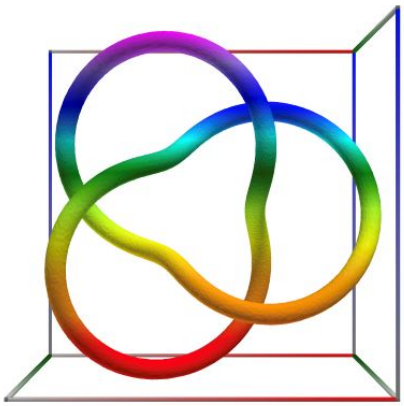
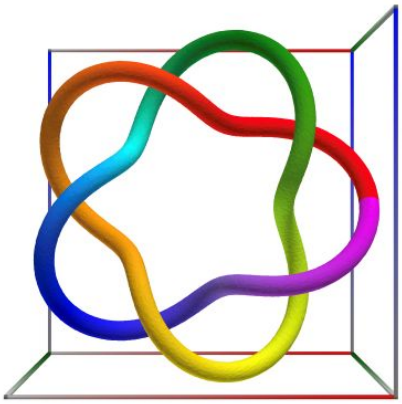
See also: https://en.wikipedia.org/wiki/Cinquefoil_knot

See also: https://katlas.org/wiki/T(5,2)

Cinquefoil Knot: https://home.adelphi.edu/~stemkoski/knotgallery/

The example below uses the following code in C#

```
var x = Math.Cos(2 * t) * (3 + Math.Cos(5 * t)) / 10;
var y = Math.Sin(2 * t) * (3 + Math.Cos(5 * t)) / 10;
var z = Math.Sin(5 * t) / 10;
```

24.6.6 74 Knot

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_Path_74Knot.3D.xml (D06a-f).

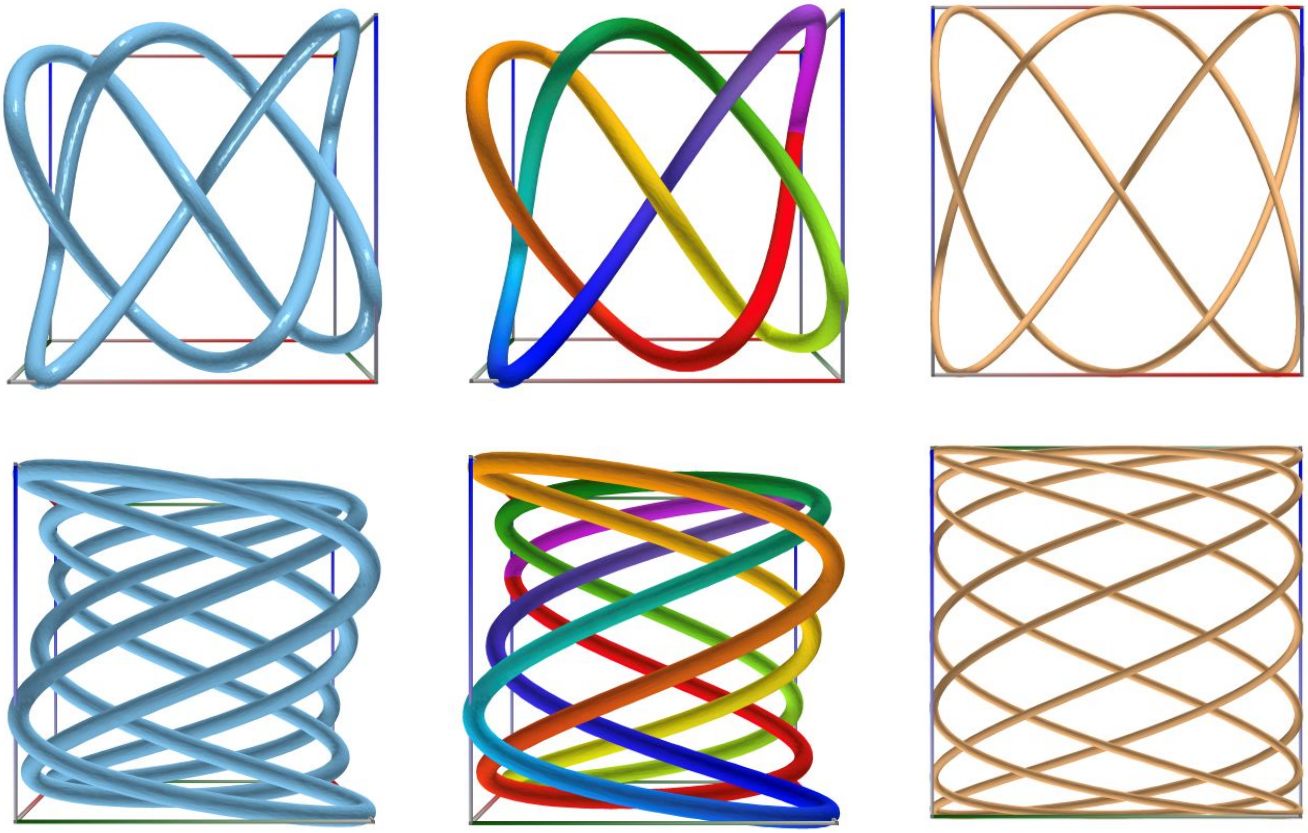
See also: https://en.wikipedia.org/wiki/74_knot

This is a Lissajous knot.

See also: https://katlas.org/wiki/7_4

The example below uses the following code in C#

```
var x = Math.Cos(2 * t + 0.22) / 10;
var y = Math.Cos(3 * t + 1.10) / 10;
var z = Math.Cos(7 * t) / 10;
```



24.6.7 Figure-eight knot

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D07a_Path_FigureEightKnot.3D.xml (D07a-f).

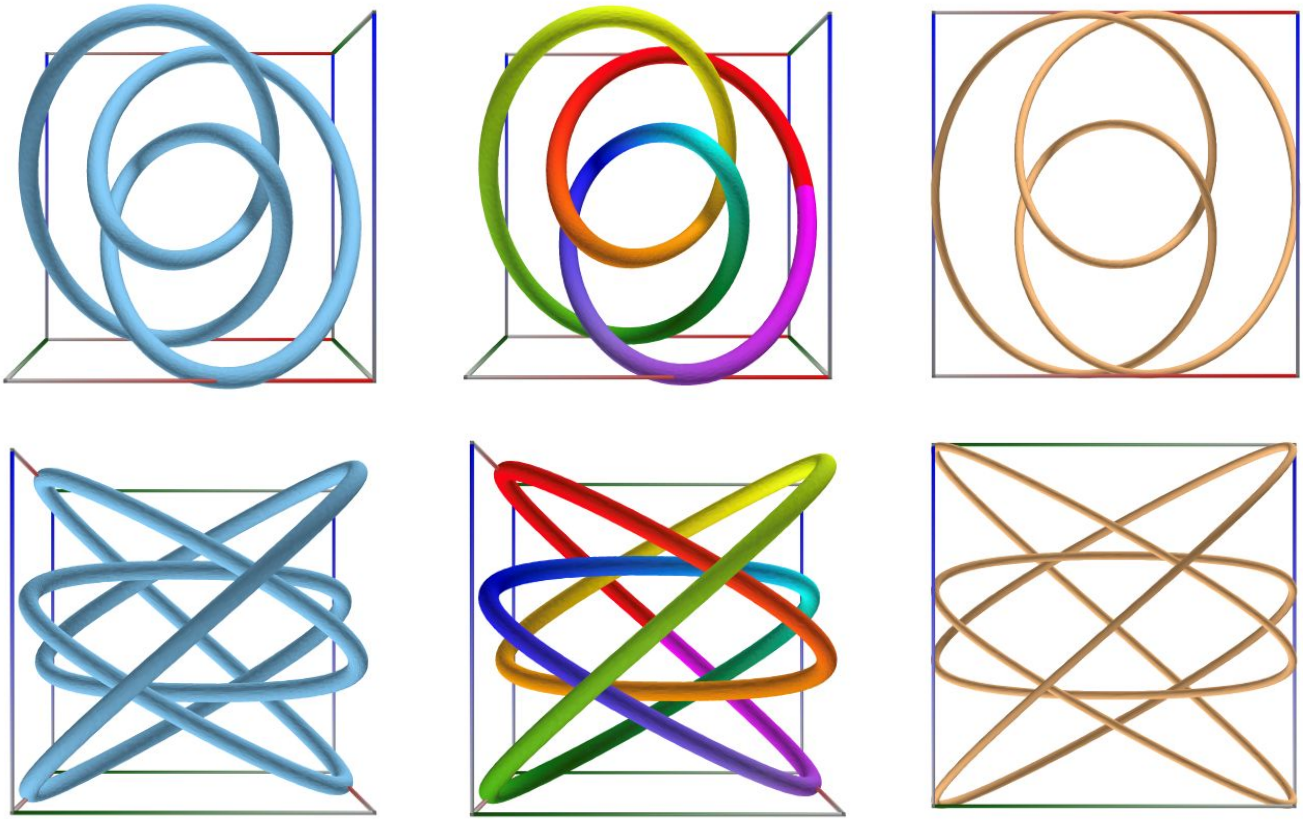
See also: https://mathcurve.com/courbes3d.gb/noeuds/noeudenhuit.shtml

See also: https://en.wikipedia.org/wiki/Figure-eight_knot_(mathematics)

See also: https://katlas.org/wiki/4_1

The example below uses the following code in C#

```
var x = ((2 + Math.Cos(2 * t)) * Math.Cos(3 * t)) / 10;
var y = ((2 + Math.Cos(2 * t)) * Math.Sin(3 * t)) / 10;
var z = (Math.Sin(4 * t)) / 10;
```



24.7 Torus knots

See also: https://katlas.org/wiki/36_Torus_Knots

24.7.1 Torus knot 3:2

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_TorusKnotp3q2.3D.xml (D01a-f).

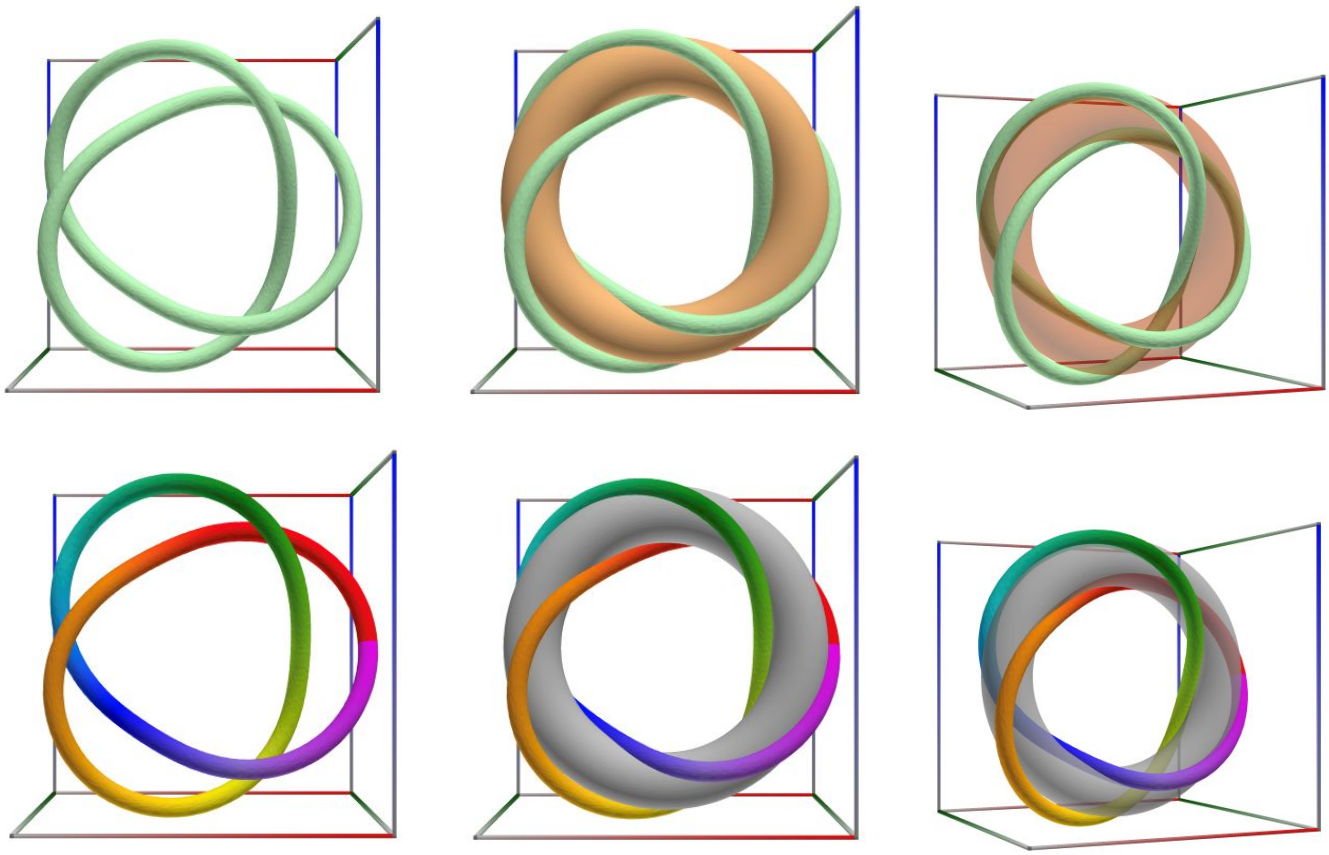
Some text

See also: https://mathworld.wolfram.com/TrefoilKnot.html

See also: https://katlas.org/wiki/T(3,2)

The example below uses the following code in C#

```
double R = 100;
double r = 25;
double p = 3;
double q = 2;
var x = ((R + r * Math.Cos(p * t)) * Math.Cos(q * t)) / 50;
var y = ((R + r * Math.Cos(p * t)) * Math.Sin(q * t)) / 50;
var z = (r * Math.Sin(p * t)) / 50;
```



24.7.2 Torus knot 5:2

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_TorusKnotp5q2.3D.xml (D02a-f).

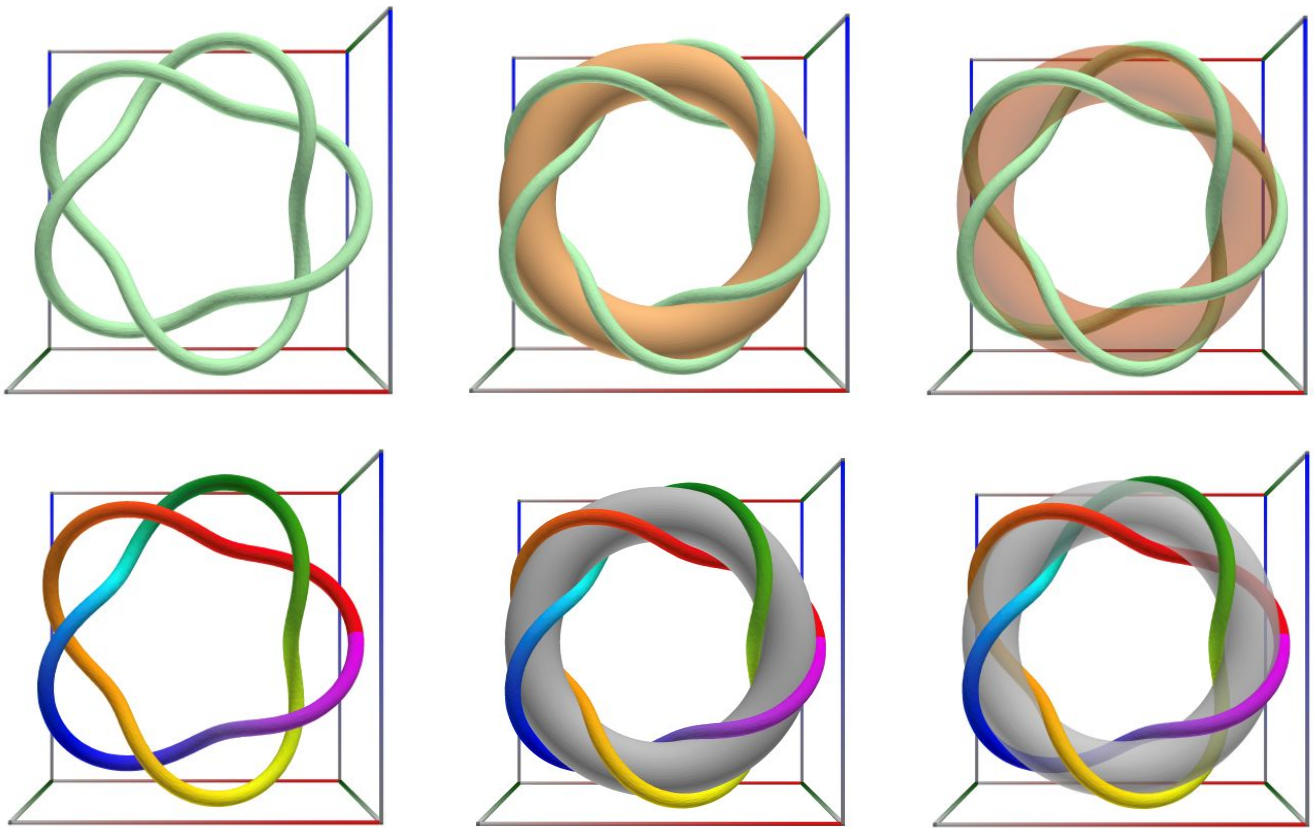
Some text

See also: https://en.wikipedia.org/wiki/Cinquefoil_knot

See also: https://katlas.org/wiki/T(5,2)

The example below uses the following code in C#

```
double R = 100;
double r = 25;
double p = 5;
double q = 2;
var x = ((R + r * Math.Cos(p * t)) * Math.Cos(q * t)) / 50;
var y = ((R + r * Math.Cos(p * t)) * Math.Sin(q * t)) / 50;
var z = (r * Math.Sin(p * t)) / 50;
```



24.7.3 Torus knot 7:2

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_TorusKnotp7q2.3D.xml (D03a-f).

Some text

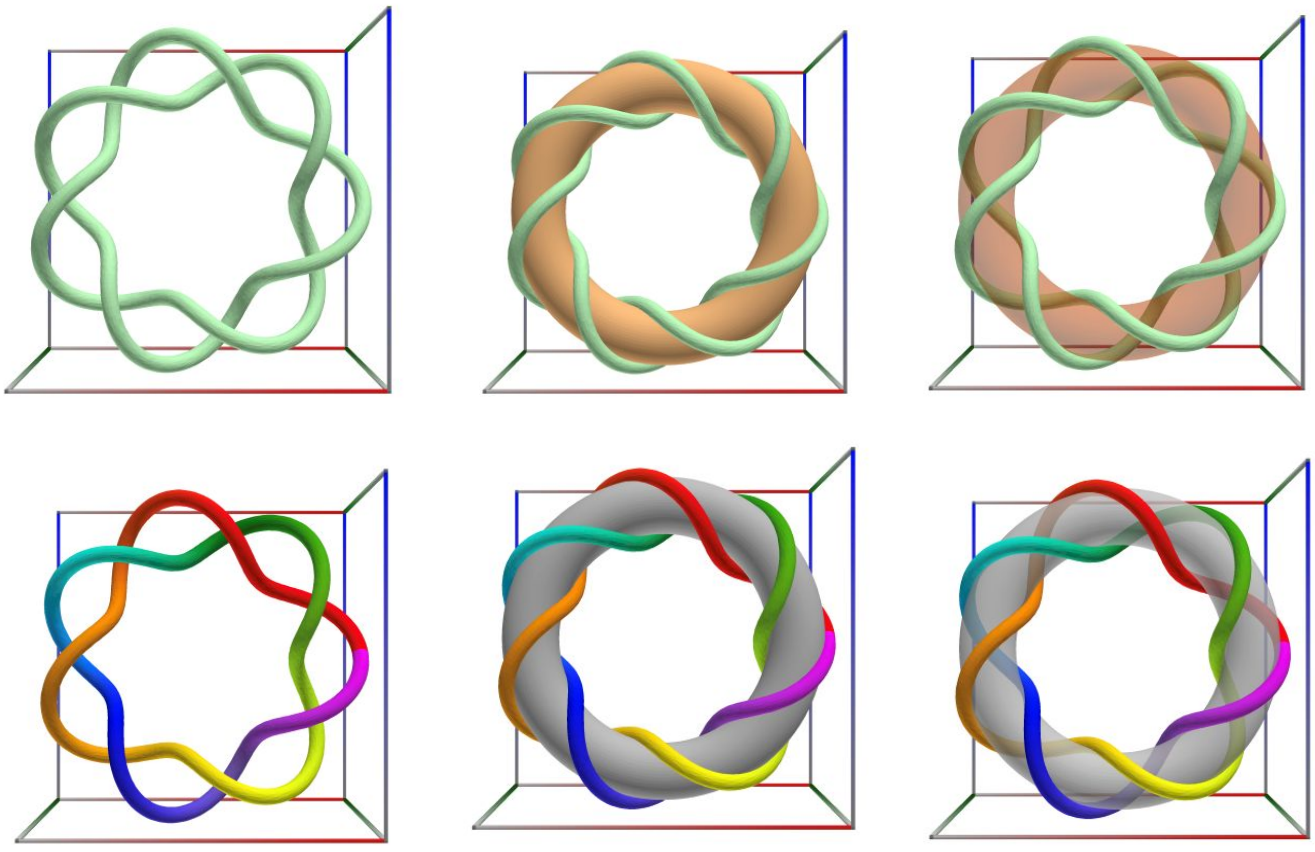
See also: https://en.wikipedia.org/wiki/71_knot

See also: https://katlas.org/wiki/T(7,2)

See also: https://katlas.org/wiki/7_1

The example below uses the following code in C#

```
double R = 100;
double r = 25;
double p = 7;
double q = 2;
var x = ((R + r * Math.Cos(p * t)) * Math.Cos(q * t)) / 50;
var y = ((R + r * Math.Cos(p * t)) * Math.Sin(q * t)) / 50;
var z = (r * Math.Sin(p * t)) / 50;
```



24.7.4 Torus knot 7:3

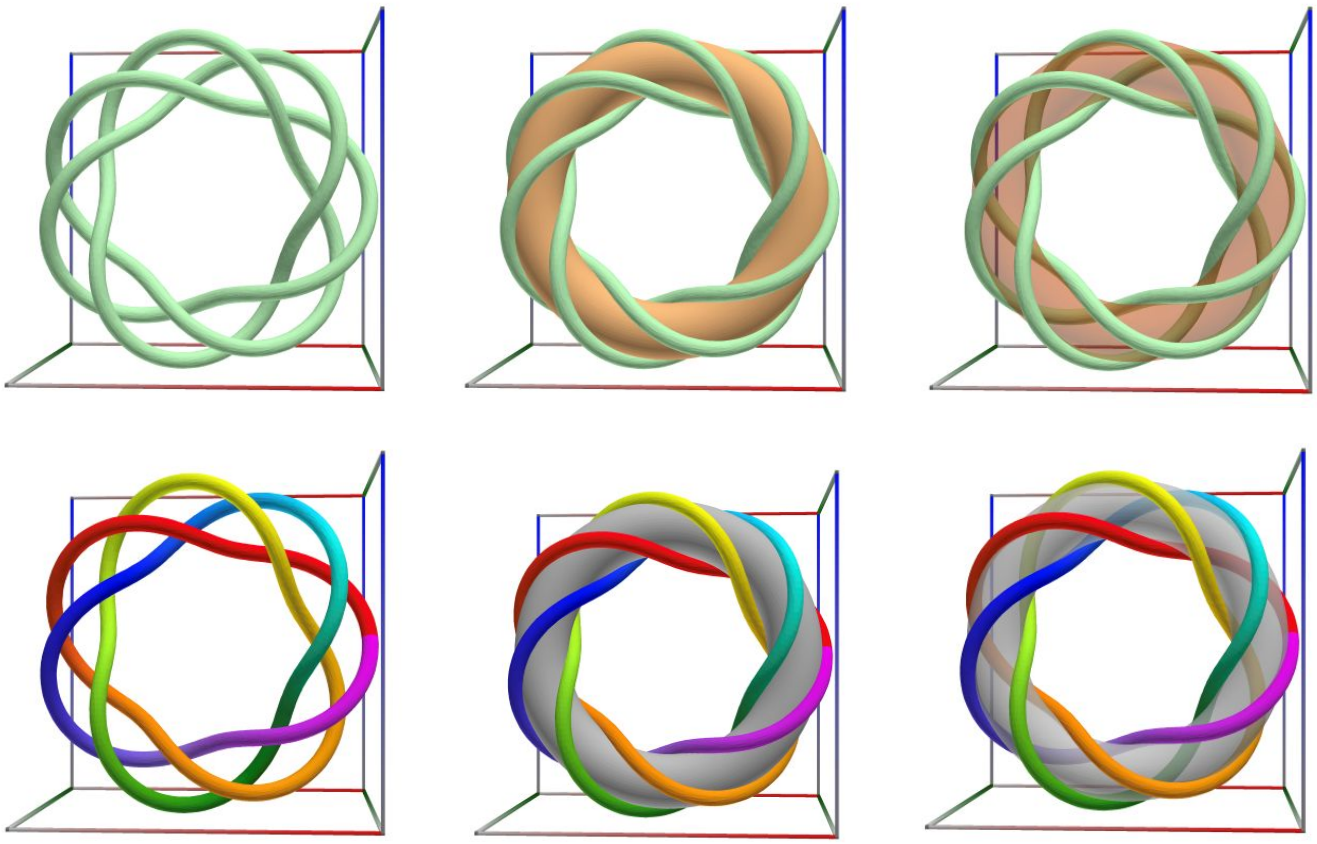
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_TorusKnotp7q3.3D.xml (D04a-f).

Some text

See also: https://katlas.org/wiki/T(7,3)

The example below uses the following code in C#

```
double R = 100;
double r = 25;
double p = 7;
double q = 3;
var x = ((R + r * Math.Cos(p * t)) * Math.Cos(q * t)) / 50;
var y = ((R + r * Math.Cos(p * t)) * Math.Sin(q * t)) / 50;
var z = (r * Math.Sin(p * t)) / 50;
```



24.7.5 Torus knot 15:2

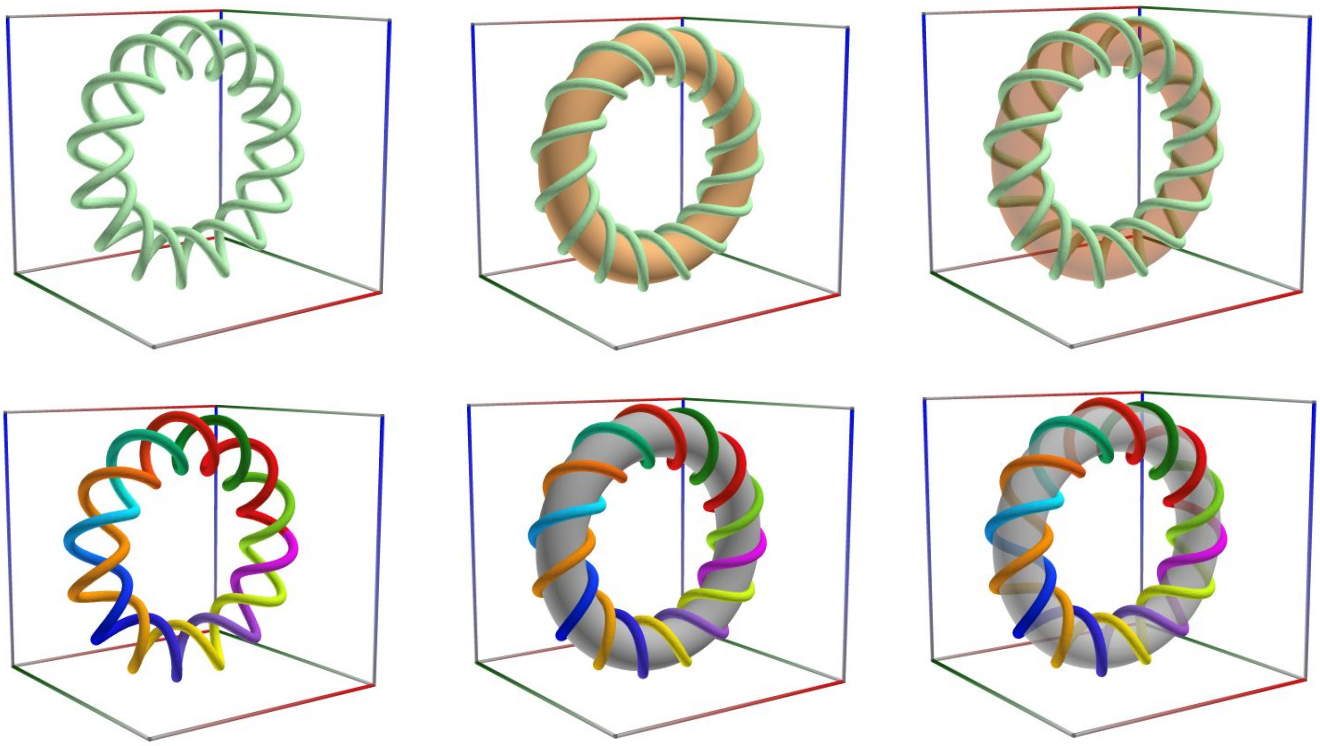
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_TorusKnotp15q2.3D.xml (D05a-f).

Some text

See also: https://katlas.org/wiki/T(15,2)

The example below uses the following code in C#

```
double R = 100;
double r = 25;
double p = 15;
double q = 2;
var x = ((R + r * Math.Cos(p * t)) * Math.Cos(q * t)) / 50;
var y = ((R + r * Math.Cos(p * t)) * Math.Sin(q * t)) / 50;
var z = (r * Math.Sin(p * t)) / 50;
```



24.8 Lissajous knots

See also: https://en.wikipedia.org/wiki/Lissajous_knot

See also: https://mathcurve.com/courbes3d.gb/lissajous3d/noeudlissajous.shtml

See also: https://knotplot.com/manual/LissajousParams.html

The Lissajous knots are the knots associated to the 3D Lissajous curves when they are closed and without double point.

As is proved in the above article by Jones and Przytycki, they also are the knots associated to the trajectories of a ball (not subject to gravity) in a parallelepipedic billiard, or even a cubic one (imagine a glass box).

It can be proved that for all values of p, q, r, there exist values of j and y such that the knot is trivial, and that certain knots such as the trefoil knot are not Lissajous knots.

See also: https://mathcurve.com/courbes3d.gb/lissajous3d/lissajous3d.shtml

The 3D Lissajous curves are the trajectories of a point in space the rectangular components of which have a sinusoidal motion. The projections on the 3 coordinate planes are the classic 2D Lissajous curves.

For n = 1 or n = m, we get a cylindrical sine wave. We get a closed curve if and only if n and m are rational.

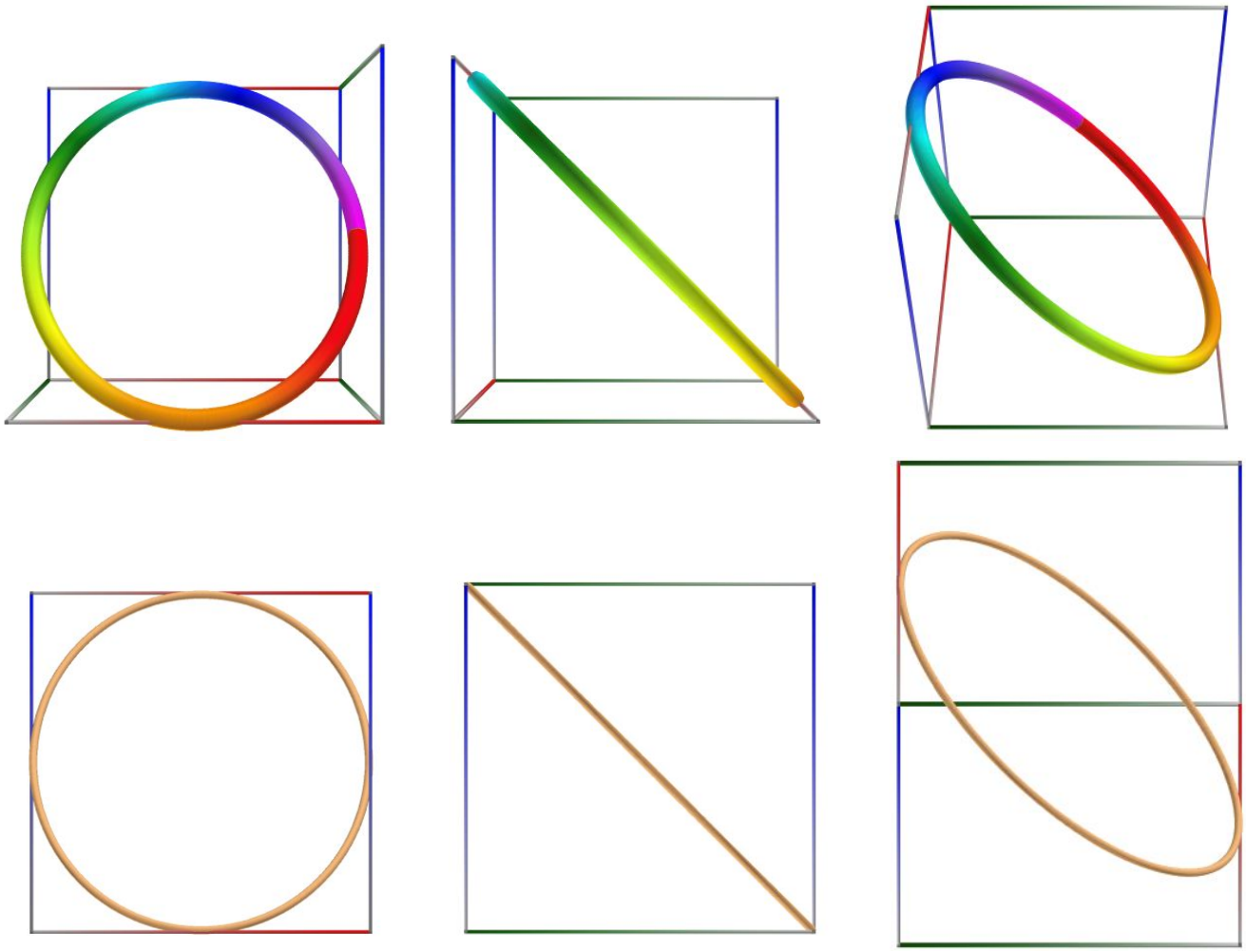
When the curve does not have double points, nor a cusp, it forms a knot in space, called Lissajous knot, equivalent to a cubic billiard knot.

24.8.1 Lissajous knot 1-1-1

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Path_111_LKnot_P090T090.3D.xml (D01a-f).

The example below uses the following code in C#

```
double pi = Math.PI;
double a1, k1, l1;
double a2, k2, l2;
double a3, k3, l3;
a1 = 100; k1 = 1; l1 = 0;
a2 = 100; k2 = 1; l2 = pi / 2;
a3 = 100; k3 = 1; l3 = pi / 2;
var x = (a1 * Math.Cos(k1 * t + l1)) / 50;
var y = (a3 * Math.Cos(k3 * t + l3)) / 50;
var z = (a2 * Math.Cos(k2 * t + l2)) / 50;
```

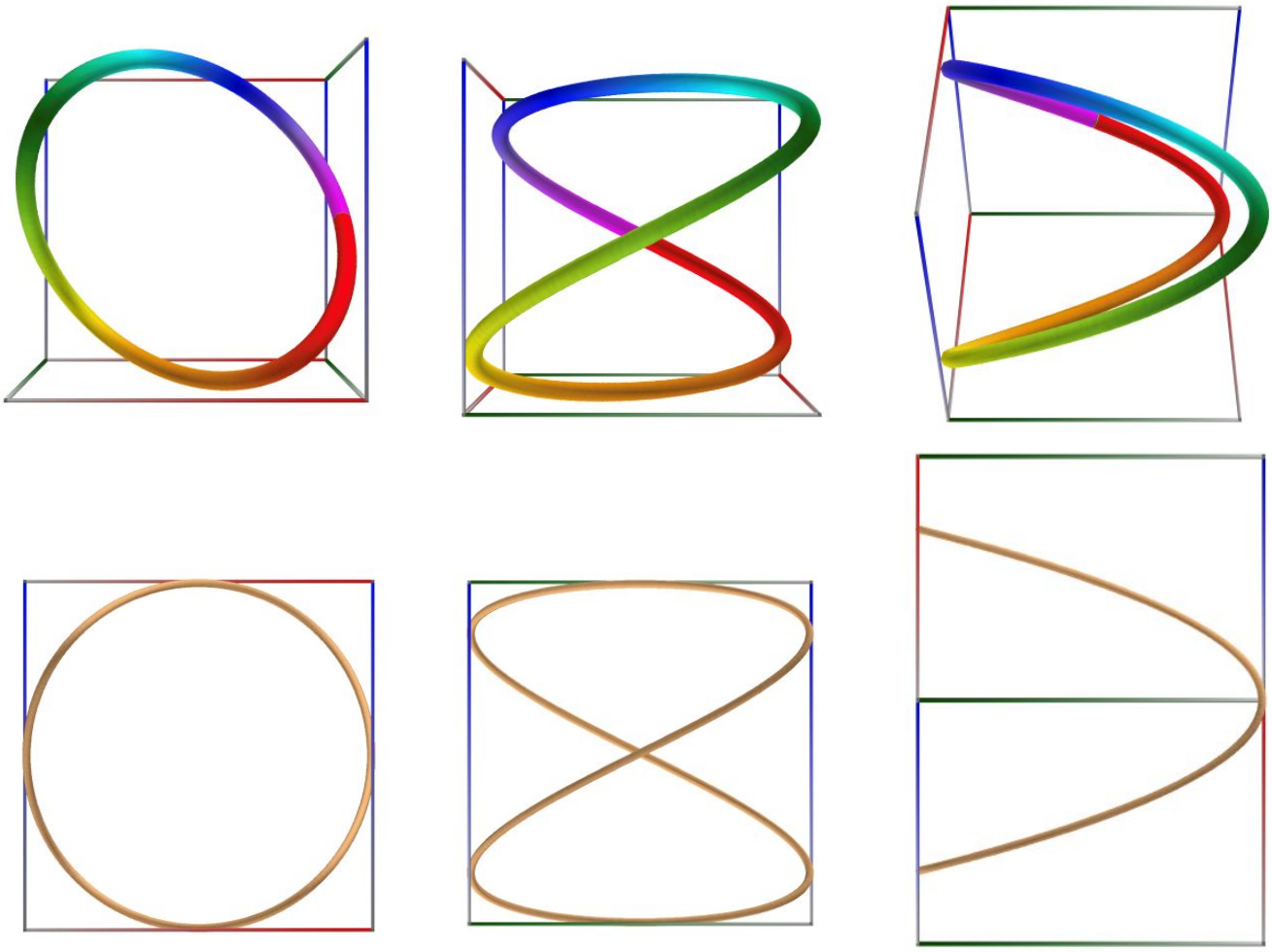


24.8.2 Lissajous knot 1-2-1

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Path_121_LKnot_P090T090.3D.xml (D02a-f).

The example below uses the following code in C#

```
double pi = Math.PI;
double a1, k1, l1;
double a2, k2, l2;
double a3, k3, l3;
a1 = 100; k1 = 1; l1 = 0;
a2 = 100; k2 = 2; l2 = pi / 2;
a3 = 100; k3 = 1; l3 = pi / 2;
var x = (a1 * Math.Cos(k1 * t + l1)) / 50;
var y = (a3 * Math.Cos(k3 * t + l3)) / 50;
var z = (a2 * Math.Cos(k2 * t + l2)) / 50;
```

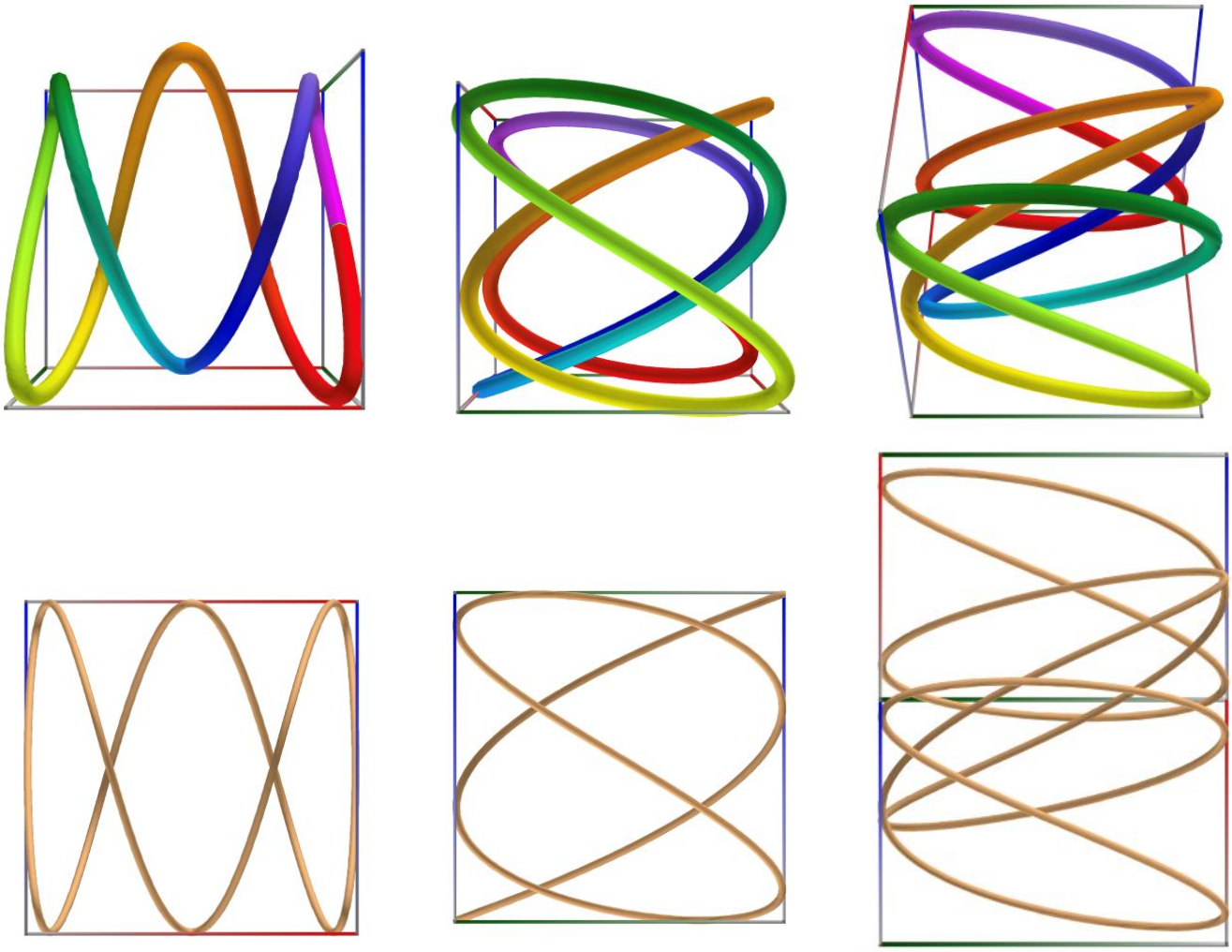


24.8.3 Lissajous knot 1-5-3

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_Path_153_LKnot_P090T090.3D.xml (D03a-f).

The example below uses the following code in C#

```
double pi = Math.PI;
double a1, k1, l1;
double a2, k2, l2;
double a3, k3, l3;
a1 = 100; k1 = 1; l1 = 0;
a2 = 100; k2 = 5; l2 = pi / 2;
a3 = 100; k3 = 3; l3 = pi / 2;
var x = (a1 * Math.Cos(k1 * t + l1)) / 50;
var y = (a3 * Math.Cos(k3 * t + l3)) / 50;
var z = (a2 * Math.Cos(k2 * t + l2)) / 50;
```

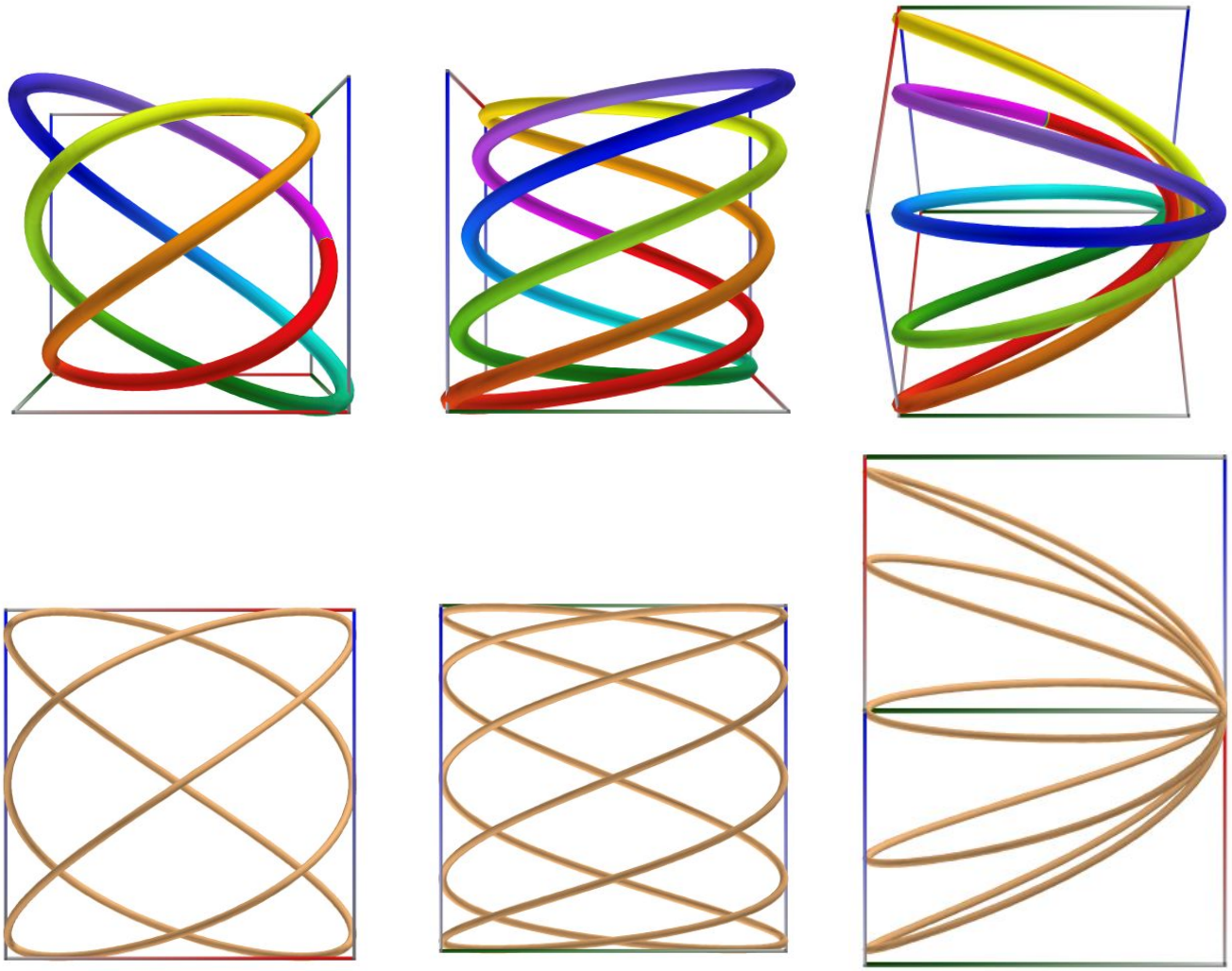


24.8.4 Lissajous knot 3-5-2

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Path_352_LKnot_P090T090.3D.xml (D04a-f).

The example below uses the following code in C#

```
double pi = Math.PI;
double a1, k1, l1;
double a2, k2, l2;
double a3, k3, l3;
a1 = 100; k1 = 3; l1 = 0;
a2 = 100; k2 = 5; l2 = pi / 2;
a3 = 100; k3 = 2; l3 = pi / 2;
var x = (a1 * Math.Cos(k1 * t + l1)) / 50;
var y = (a3 * Math.Cos(k3 * t + l3)) / 50;
var z = (a2 * Math.Cos(k2 * t + l2)) / 50;
```

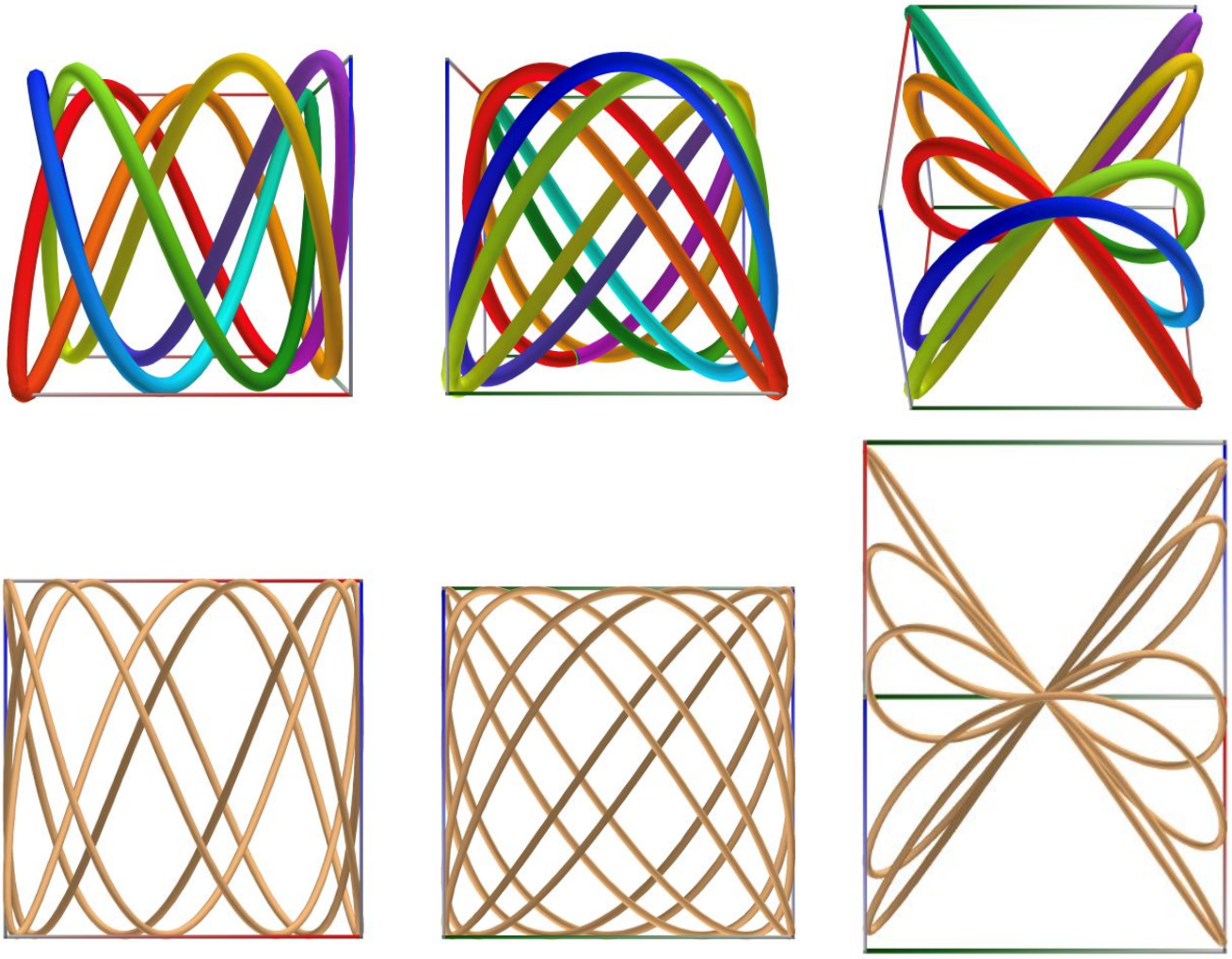


24.8.5 Lissajous knot 3-5-7

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Path_357_LKnot_P090T090.3D.xml (D05a-f).

The example below uses the following code in C#

```
double pi = Math.PI;
double a1, k1, l1;
double a2, k2, l2;
double a3, k3, l3;
a1 = 100; k1 = 3; l1 = 7;
a2 = 100; k2 = 5; l2 = 5;
a3 = 100; k3 = 7; l3 = 3;
var x = (a1 * Math.Cos(k1 * t + l1)) / 50;
var y = (a3 * Math.Cos(k3 * t + l3)) / 50;
var z = (a2 * Math.Cos(k2 * t + l2)) / 50;
```



24.9 Polynomial knots

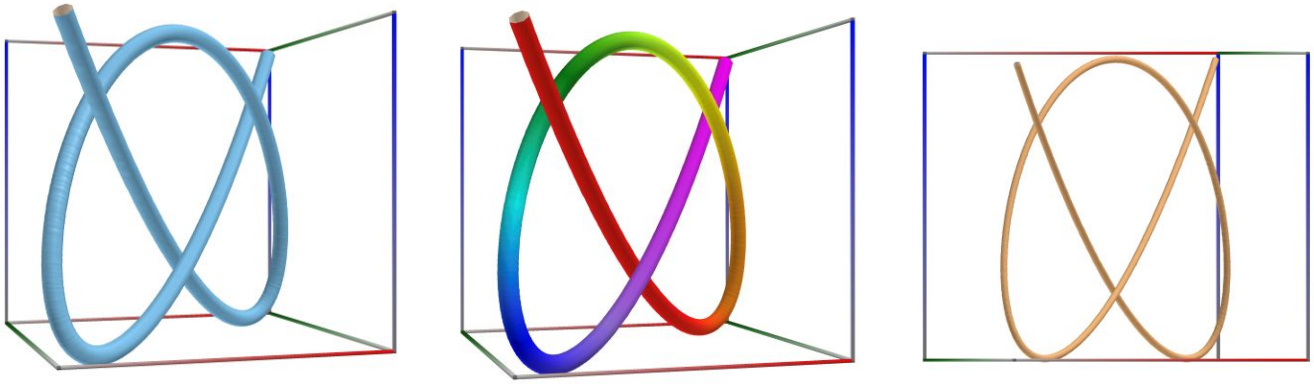
See also: https://home.adelphi.edu/~stemkoski/knotgallery/

24.9.1 Polynomial knot 3-1

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Path_Poly31Knot.3D.xml (D01a-c).

Polynomial Knot. https://home.adelphi.edu/~stemkoski/knotgallery/poly3.html

```
double t2 = t * t;
double t3 = t2 * t;
double t4 = t3 * t;
double t5 = t4 * t;
var x = (t3 - 3 * t) / 10;
var y = (t4 - 4 * t2) / 10;
var z = (t5 / 5 - 2 * t) / 10;
```

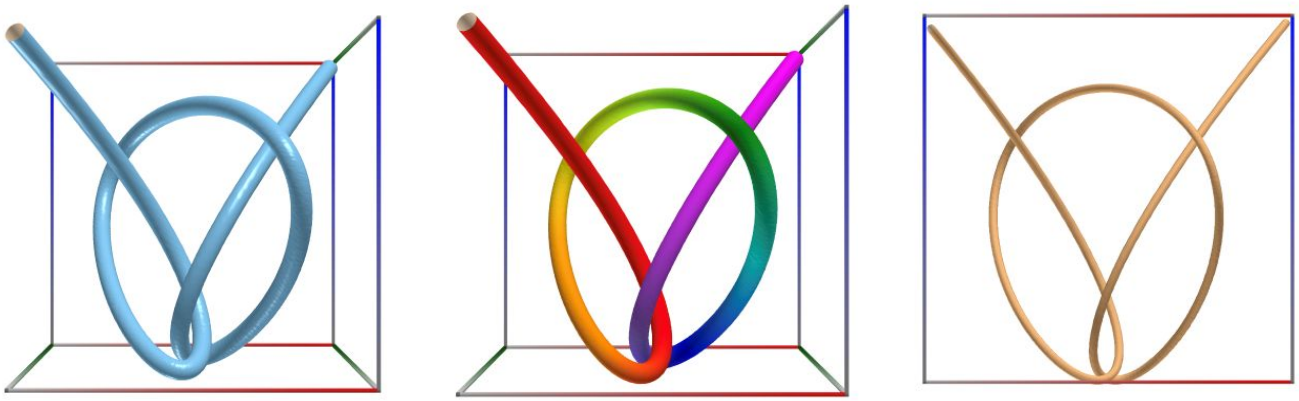


24.9.2 Polynomial knot 4-1

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_Path_Poly41Knot.3D.xml (D02a-c).

Figure-Eight (41) Polynomial Knot. https://home.adelphi.edu/~stemkoski/knotgallery/poly3.html

```
double t2 = t * t;
double t4 = t2 * t2;
var x = (2.0 / 5.0) * t * (t2 - 7) * (t2 - 10) / 10;
var y = (t4 - 13 * t2) / 10;
var z = (1.0 / 10.0) * t * (t2 - 4) * (t2 - 9) * (t2 - 12) / 10;
```

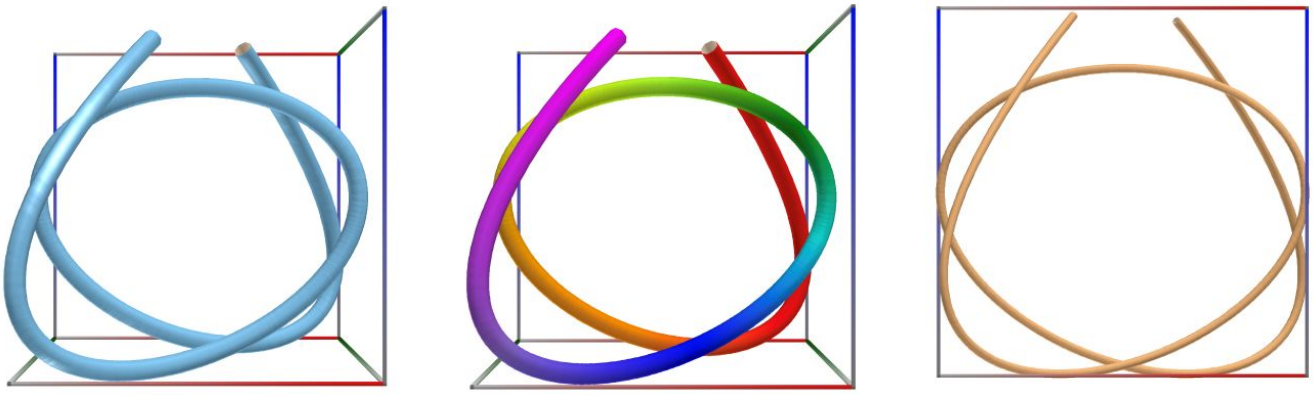


24.9.3 Polynomial knot 5-1

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_Path_Poly51Knot.3D.xml (D03a-c).

Cinquefoil (51) Polynomial Knot. https://home.adelphi.edu/~stemkoski/knotgallery/poly3.html

```
double t2 = t * t;
double t3 = t2 * t;
double t4 = t3 * t;
double t5 = t4 * t;
double t6 = t5 * t;
double t7 = t6 * t;
var x = (1.0 / 5.0) * (t5 - 36 * t3 + 260 * t) / 10;
var y = (1.0 / 2.0) * (t4 - 24 * t2) / 10;
var z = (1.0 / 100.0) * (t7 - 31 * t5 + 164 * t3 - 560 * t) / 10;
```

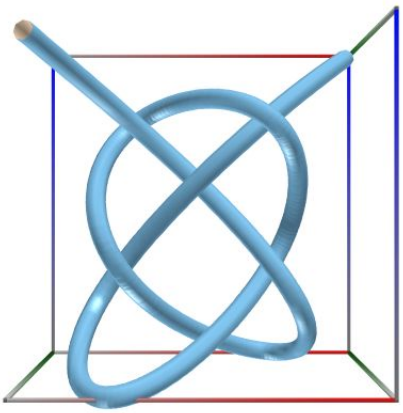
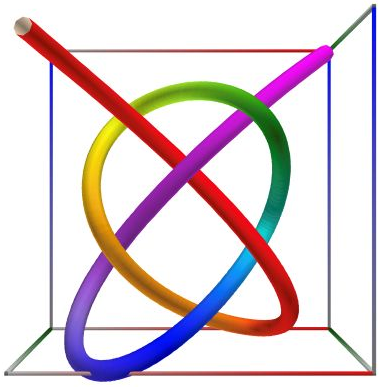
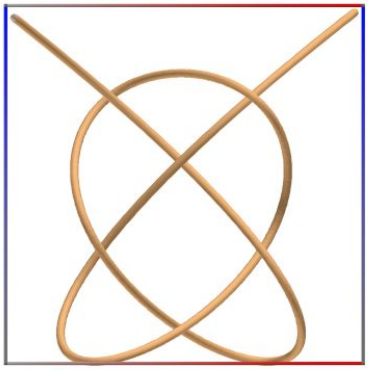


24.9.4 Polynomial knot 6-2

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Path_Poly62Knot.3D.xml (D04a-c).

Six-Crossing (62) Polynomial Knot. https://home.adelphi.edu/~stemkoski/knotgallery/poly3.html

```
double t2 = t * t;
double t3 = t2 * t;
double t4 = t3 * t;
var x = (3.0 / 4.0) * t * (t2 - 4) * (t2 - 11) / 10;
var y = (t4 - 12 * t2) / 10;
var z = (1.0 / 200.0) * t * (t2 - 1) * (t2 - 3) * (t2 - 9) * (t2 - 10) * (t2 - 12) / 10;
```

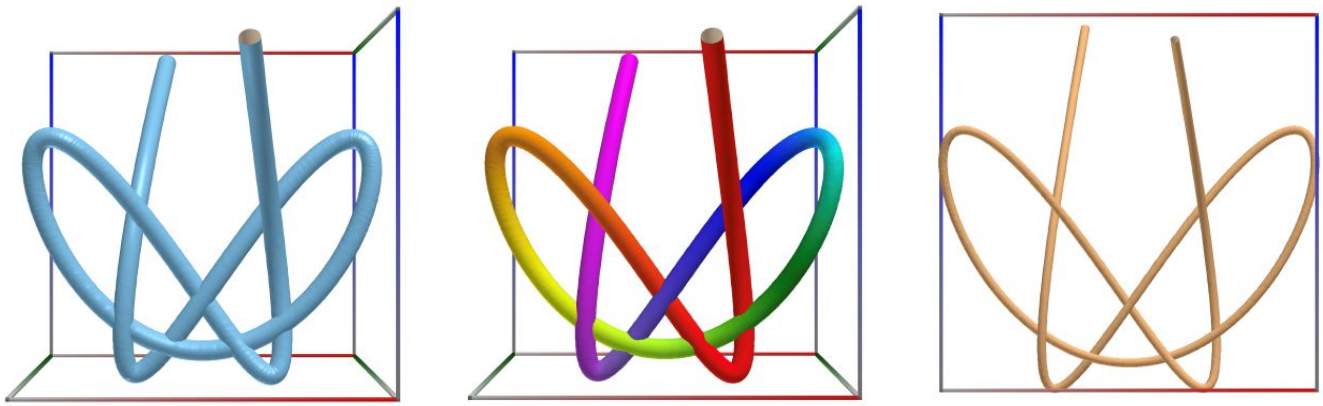




24.9.5 Polynomial knot 7-4

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Path_Poly74Knot.3D.xml (D05a-c).

Seven-Crossing (74) Polynomial Knot. https://home.adelphi.edu/~stemkoski/knotgallery/poly3.html

```
double t2 = t * t;
double t3 = t2 * t;
double t4 = t3 * t;
double t5 = t4 * t;
var x = (4.0 / 5.0) * t * (t2 - 6) * (t2 - 12) / 10;
var y = t2 * (t2 - 7) * (t2 - 9) / 10;
var z = (1.0 / 200.0) * t * (t2 - 0.2) * (t2 - 1) * (t2 - 5) * (t2 - 6.5) * (t2 - 9) * (t2 - 10) / 10;
```



CHAPTER

TWENTYFIVE

WPF: BUILT-IN 3D OBJECTS

Some general text regarding Built-in 3D objects.

25.1 Builtin solids with support for textures

25.1.1 Sphere

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_Sphere.3D.xml (D01a-f).

A sphere is a geometrical object that is a three-dimensional analogue to a two-dimensional circle.

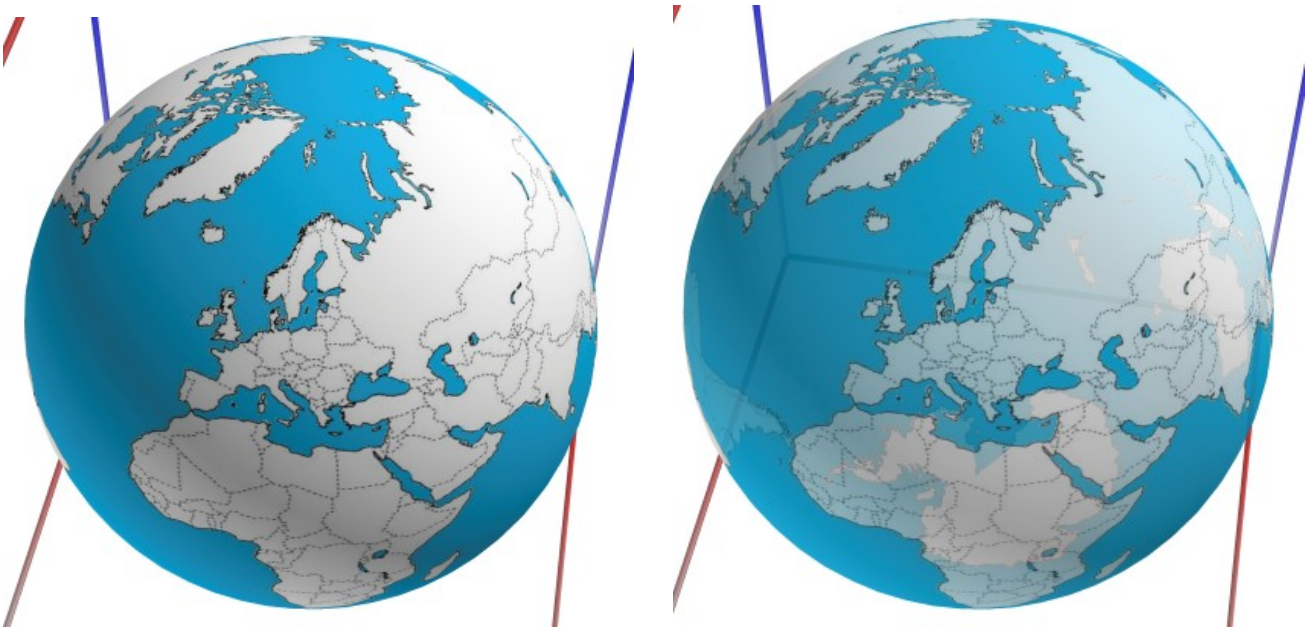
See also: https://en.wikipedia.org/wiki/Sphere

See also: https://mathworld.wolfram.com/Sphere.html

See also: https://mathcurve.com/surfaces.gb/sphere/sphere.shtml

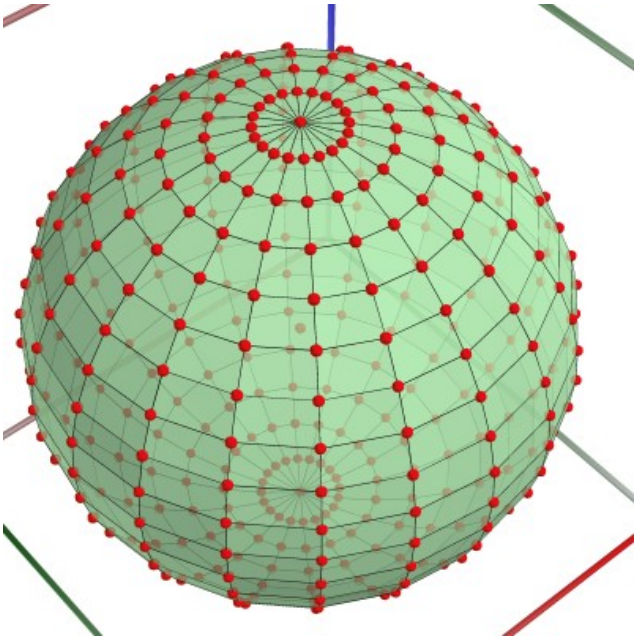
The example below uses the following code in C#

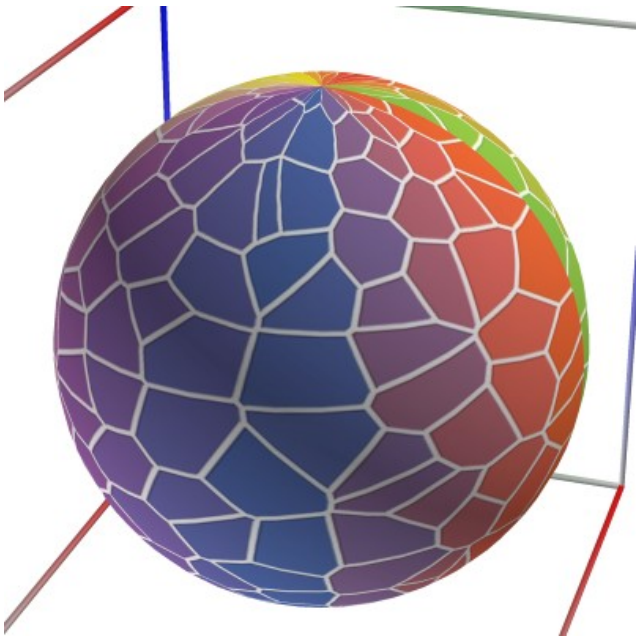
```
var radius = 1.0;
var numTheta = 20;
var numPhi = 20;
var proc = BuiltIn.SetSphere(radius, numTheta, numPhi);
```



The example below uses the following code in C#

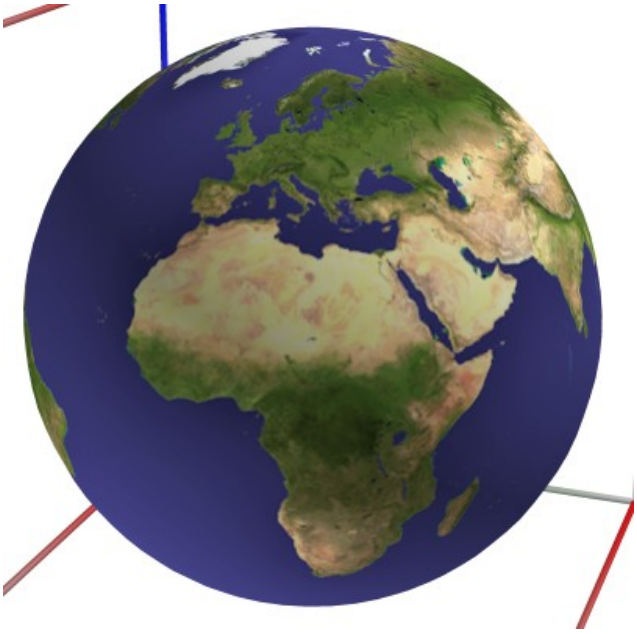
```
var radius = 1.0;
var numTheta = 20;
var numPhi = 20;
var proc = BuiltIn.SetSphere(radius, numTheta, numPhi);
```

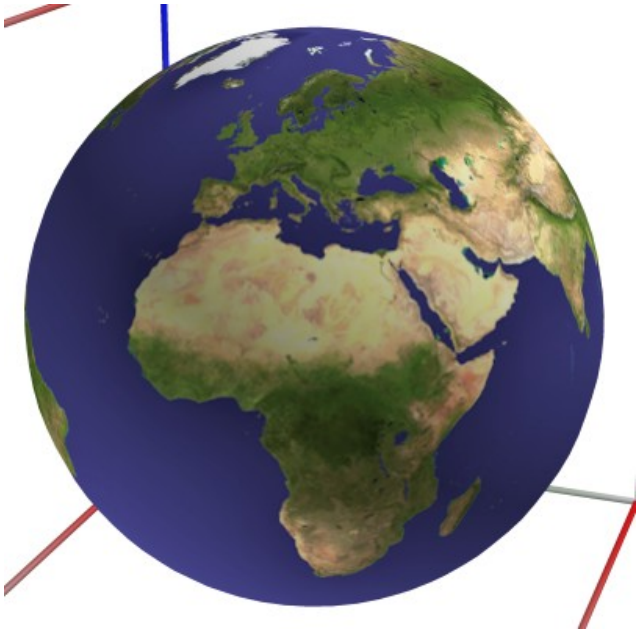




The example below uses the following code in C#

```
var radius = 1.0;
var numTheta = 20;
var numPhi = 20;
var proc = BuiltIn.SetSphere(radius, numTheta, numPhi);
```





25.1.2 Prolate Spheroid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_PSpheroid.3D.xml (D02a-b).

See also: https://mathworld.wolfram.com/ProlateSpheroid.html

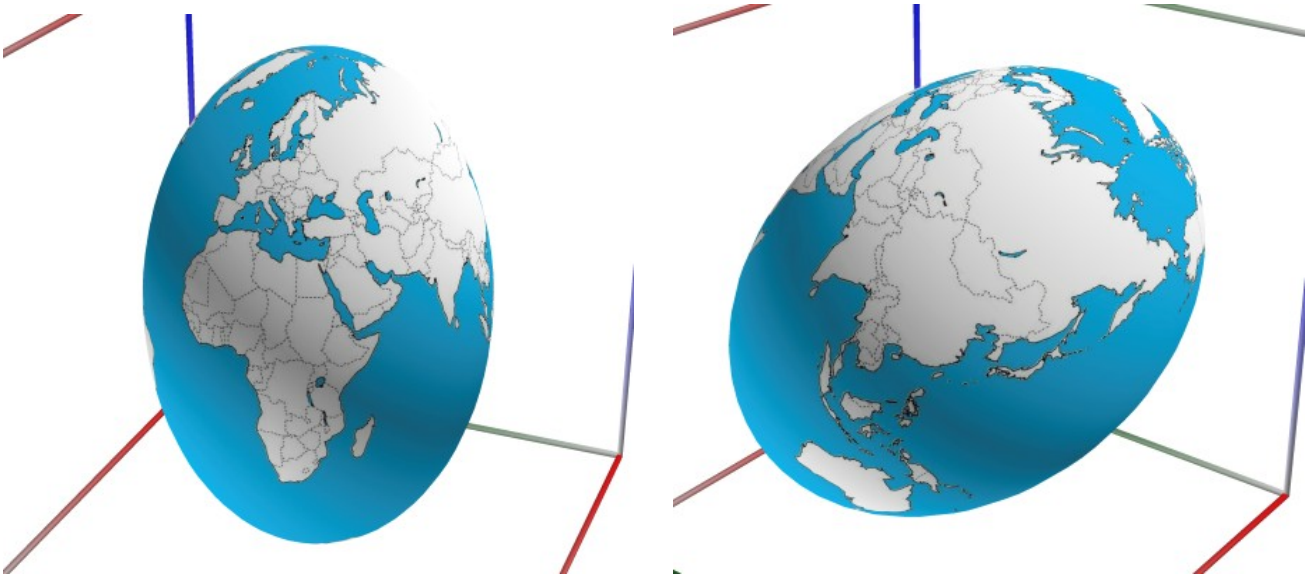
See also: https://mathworld.wolfram.com/Spheroid.html

See also: https://en.wikipedia.org/wiki/Spheroid#Prolate_spheroids

See also: https://en.wikipedia.org/wiki/Spheroid#

The example below uses the following code in C#

```
var radius = 1.0;
var factor1 = 1.5;
var numTheta = 20;
var numPhi = 20;
var proc = BuiltIn.SetProlateSpheroid(radius, factor1, numTheta, numPhi);
```



25.1.3 Oblate Spheroid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_OSpheroid.3D.xml (D03a-d).

See also: https://mathworld.wolfram.com/OblateSpheroid.html

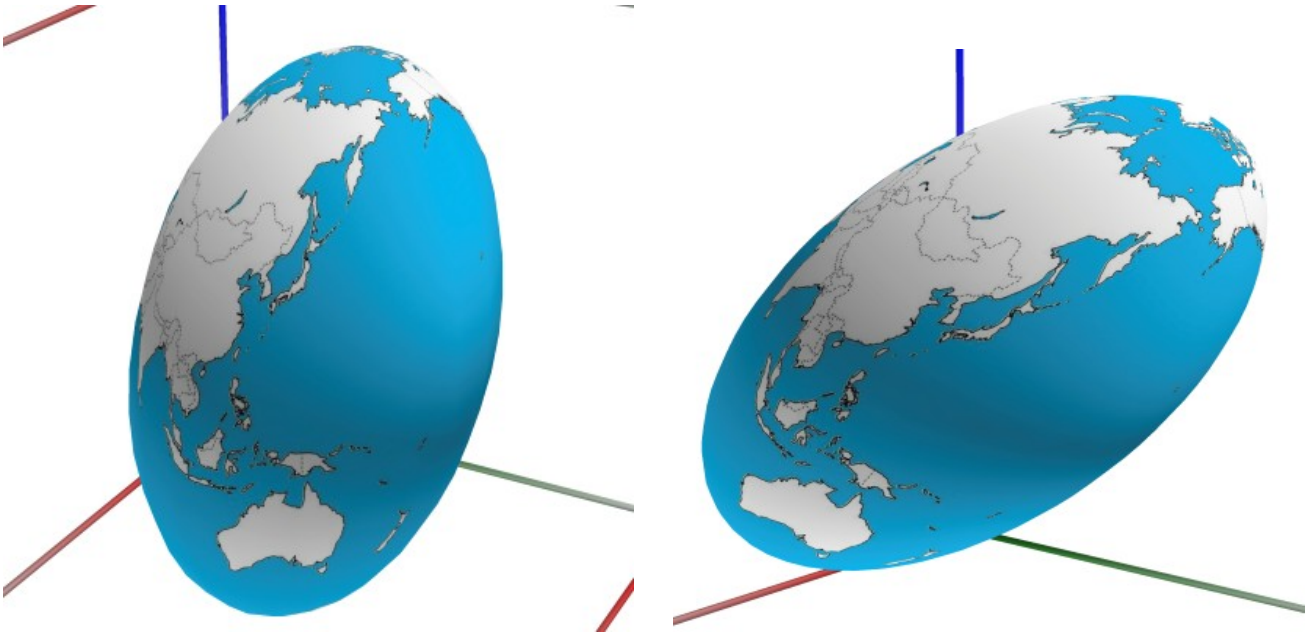
See also: https://mathworld.wolfram.com/Spheroid.html

See also: https://en.wikipedia.org/wiki/Spheroid#Oblate_spheroids

See also: https://en.wikipedia.org/wiki/Spheroid#

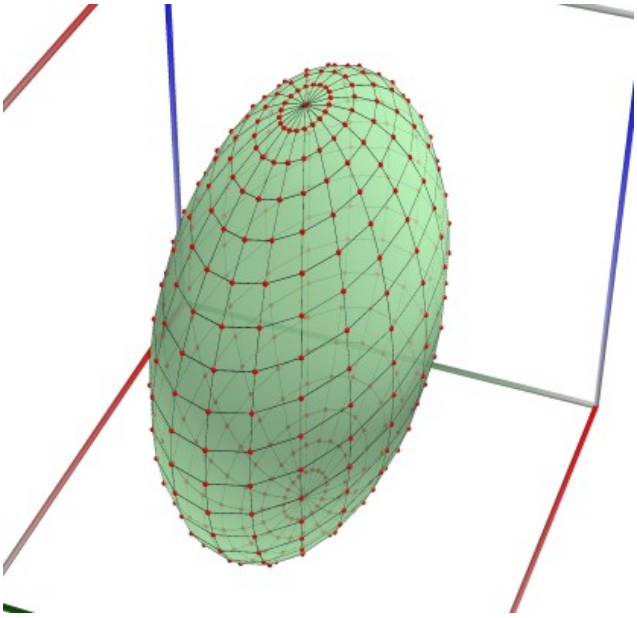
The example below uses the following code in C#

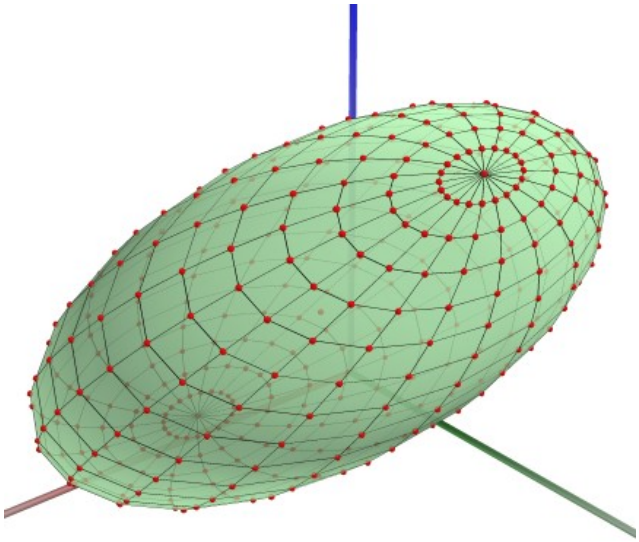
```
var radius = 1.0;
var factor2 = 0.5;
var numTheta = 20;
var numPhi = 20;
var proc = BuiltIn.SetOblateSpheroid(radius, factor2, numTheta, numPhi);
```



The example below uses the following code in C#

```
var radius = 1.0;
var factor2 = 0.5;
var numTheta = 20;
var numPhi = 20;
var proc = BuiltIn.SetOblateSpheroid(radius, factor2, numTheta, numPhi);
```





25.1.4 Ellipsoid

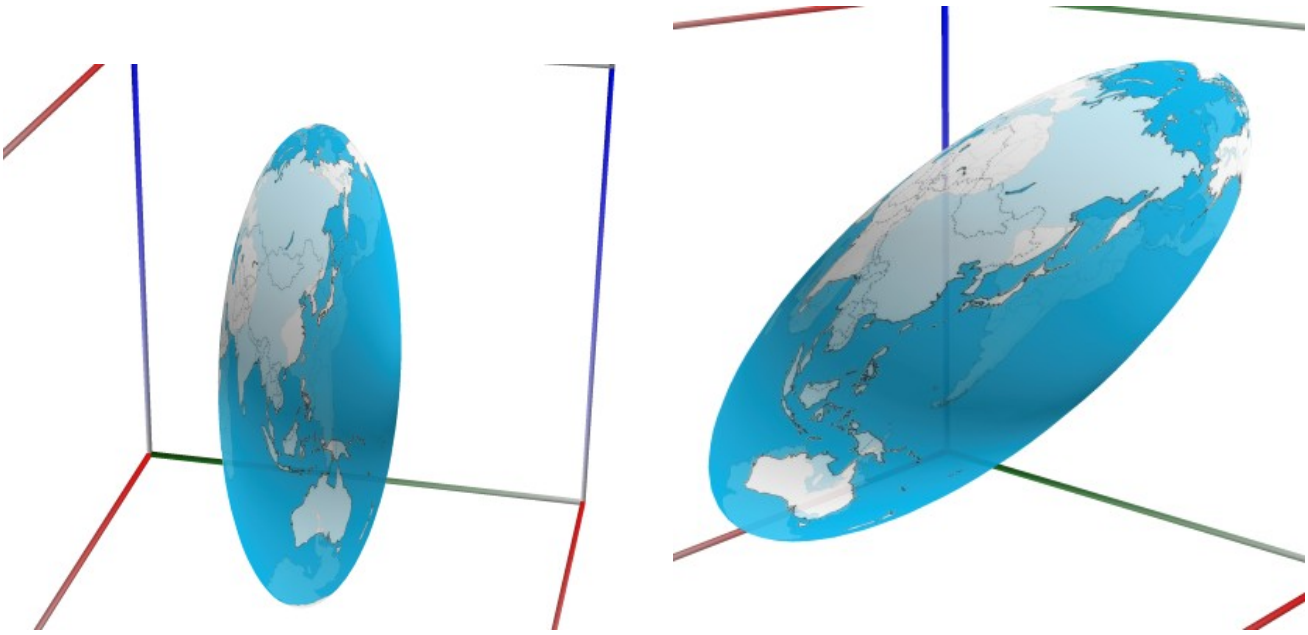
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_Ellipsoid.3D.xml (D04a-b).

See also: https://mathworld.wolfram.com/Ellipsoid.html

See also: https://en.wikipedia.org/wiki/Ellipsoid

The example below uses the following code in C#

```
var radius = 1.0;
var factor1 = 1.5;
var factor2 = 0.5;
numTheta = 20;
var numPhi = 20;
var proc = BuiltIn.SetEllipsoid(radius, factor1, factor2, numTheta, numPhi);
```



25.1.5 Torus

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D05a_Torus.3D.xml (D05a-b).

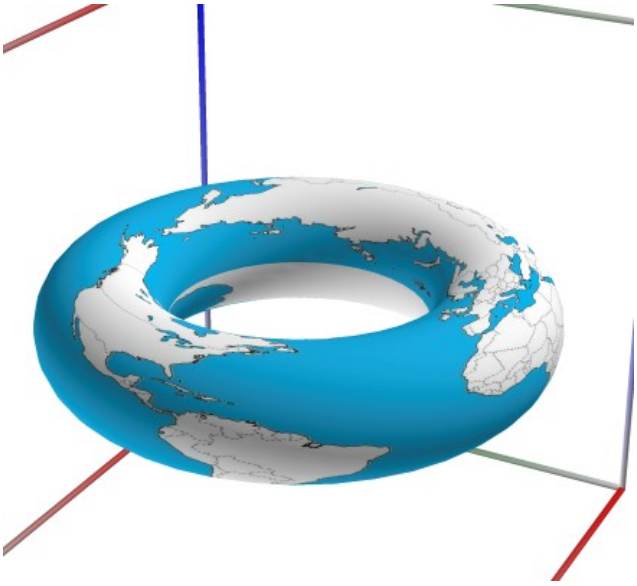
A torus is a surface of revolution generated by revolving a circle in three-dimensional space one full revolution about an axis that is coplanar with the circle. The main types of toruses include ring toruses, horn toruses, and spindle toruses. A ring torus is sometimes colloquially referred to as a donut or doughnut.

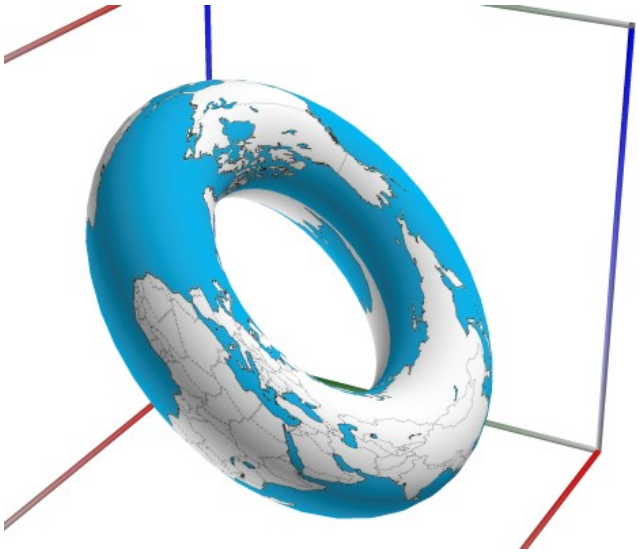
See also: https://en.wikipedia.org/wiki/Torus

See also: https://mathworld.wolfram.com/Torus.html

See also: https://mathcurve.com/surfaces.gb/tore/tore.shtml

```
var Radius = 0.6;
var radius = 0.2;
var numTheta = 40;
var numPhi = 40;
var proc = BuiltIn.SetTorus(Radius, radius, numTheta, numPhi);
```





25.1.6 Rectangular cuboid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_Cuboid.3D.xml (D06a-b).

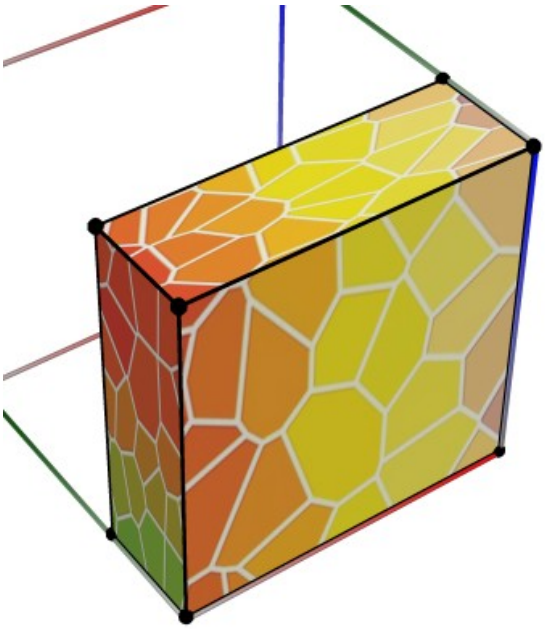
A rectangular cuboid is a special case of a cuboid with rectangular faces in which all of its dihedral angles are right angles. This shape is also called rectangular parallelepiped or orthogonal parallelepiped.

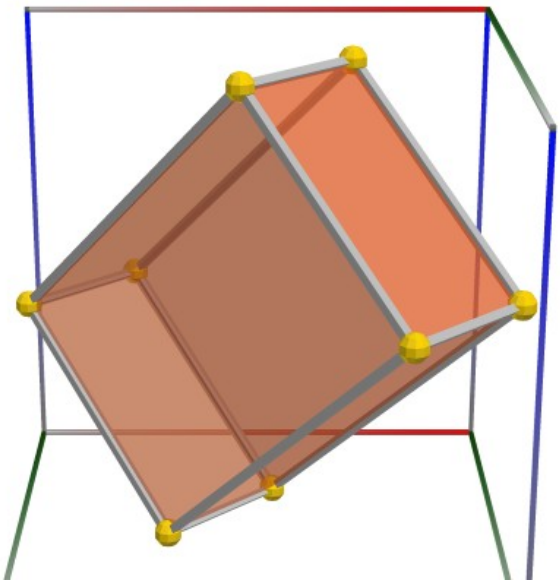
See also: https://en.wikipedia.org/wiki/Rectangular_cuboid

See also: https://mathworld.wolfram.com/Cuboid.html

The example below uses the following code in C#

```
var a = 1.0;
var b = 3.0;
var c = 2.0;
var proc = BuiltIn.SetCuboid(a, b, c);
```





25.1.7 Rhombohedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D07a_Rhombohedron_60.3D.xml (D07a-c).

A rhombohedron is a parallelepiped bounded by six rhombi such that opposite faces are congruent. Special cases include the cube, acute golden rhombohedron, and obtuse golden rhombohedron.

See also: https://en.wikipedia.org/wiki/Rhombohedron

See also: https://mathworld.wolfram.com/Rhombohedron.html

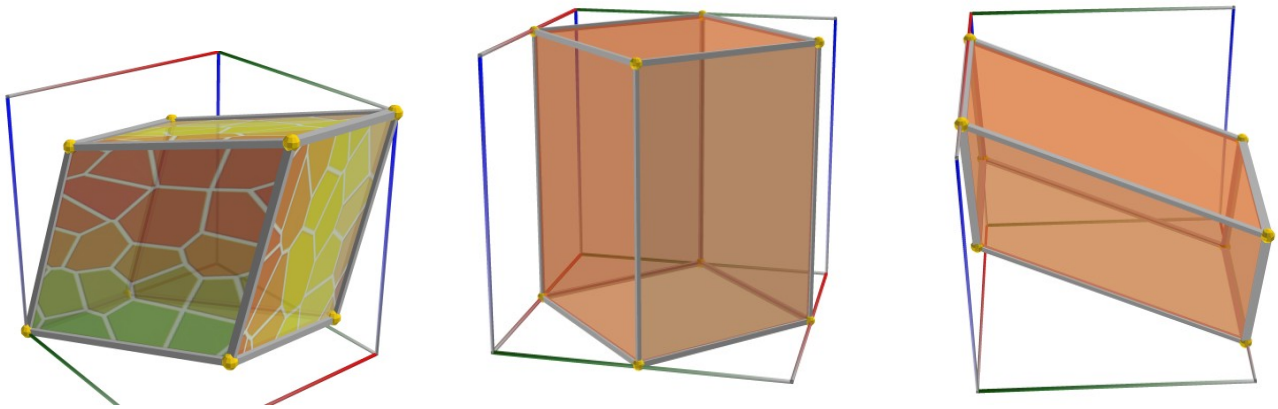
See also: https://mathworld.wolfram.com/GoldenRhombohedron.html

See also: https://mathworld.wolfram.com/AcuteGoldenRhombohedron.html

See also: https://mathworld.wolfram.com/ObtuseGoldenRhombohedron.html

The example below uses the following code in C#

```
var theta = 60.0;
var proc = BuiltIn.SetRhombohedron(theta);
```



25.1.8 Parallelepiped

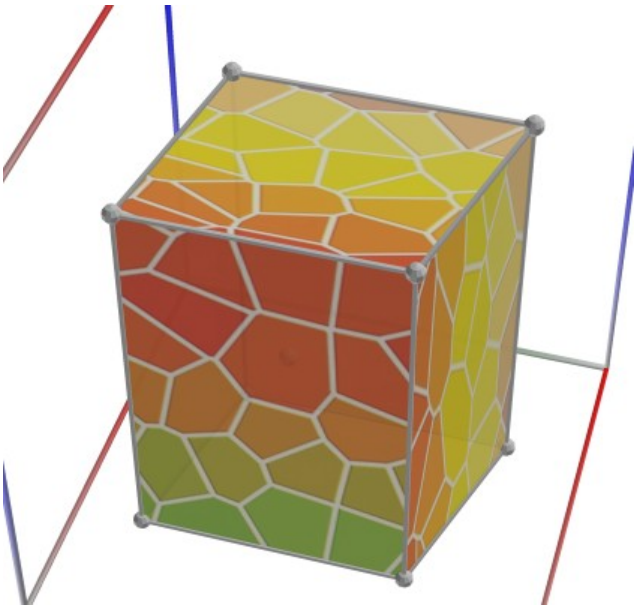
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D08a_Parallelepiped.3D.xml (D08a-b).

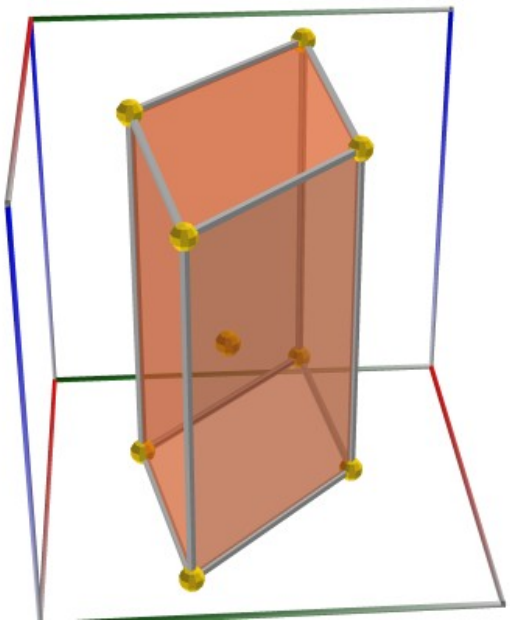
See also: https://en.wikipedia.org/wiki/Parallelepiped

See also: https://mathworld.wolfram.com/Parallelepiped.html

The example below uses the following code in C#

```
var a = 1.0;
var b = 3.0;
var c = 2.0;
var alpha = 90.0;
var beta = 90.0;

var proc = BuiltIn.SetParallelepiped(a, b, c, alpha, beta);
```





25.2 Builtin solids without support for textures

See also: https://mathworld.wolfram.com/topics/Prisms.html

See also: https://en.wikipedia.org/wiki/Prism_(geometry)

A prism is a polyhedron comprising an n-sided polygon base, a second base which is a translated copy (rigidly moved without rotation) of the first, and n other faces, necessarily all parallelograms, joining corresponding sides of the two bases. All cross-sections parallel to the bases are translations of the bases. Prisms are named after their bases, e.g. a prism with a pentagonal base is called a pentagonal prism.

25.2.1 Triangular Prism

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D01a_TriangularPrism.3D.xml (D01a-b).

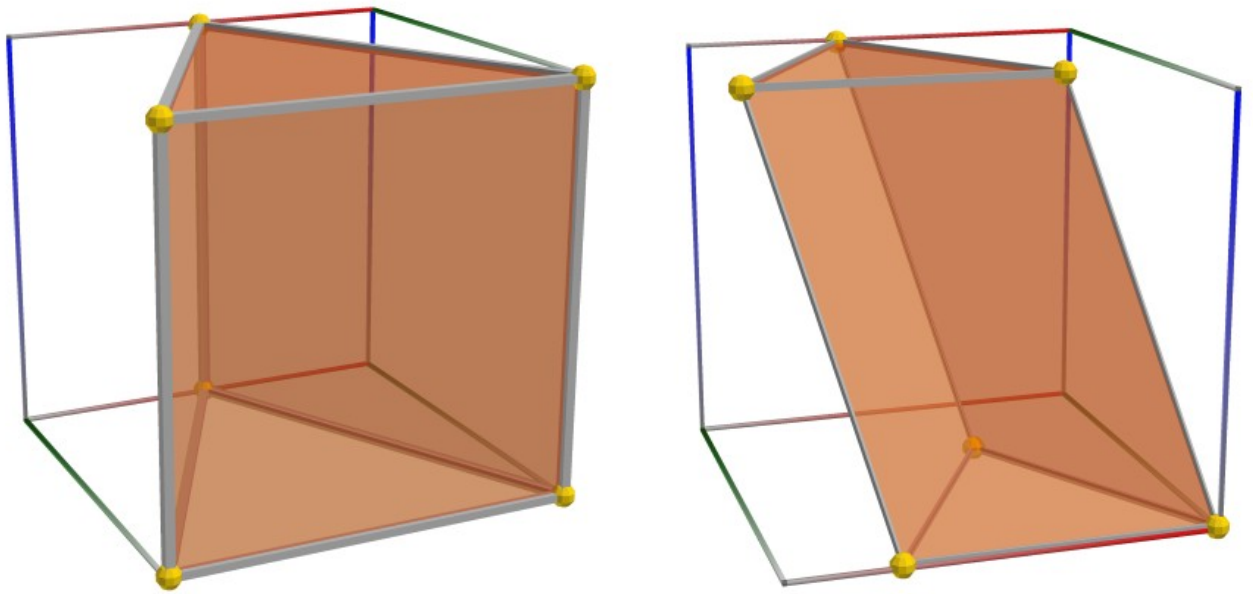
A triangular prism or trigonal prism[1] is a prism with 2 triangular bases. If the edges pair with each triangle's vertex and if they are perpendicular to the base, it is a right triangular prism.

See also: https://mathworld.wolfram.com/TriangularPrism.html

See also: https://en.wikipedia.org/wiki/Triangular_prism

The example below uses the following code in C#

```
var a = 0.50;
var b = 0.50;
var height = 1.0;
var proc = BuiltIn.SetTriangularPrism(a, b, height, 0,0);
```



25.2.2 Square prism

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D02a_SquarePrism.3D.xml (D02a-b).

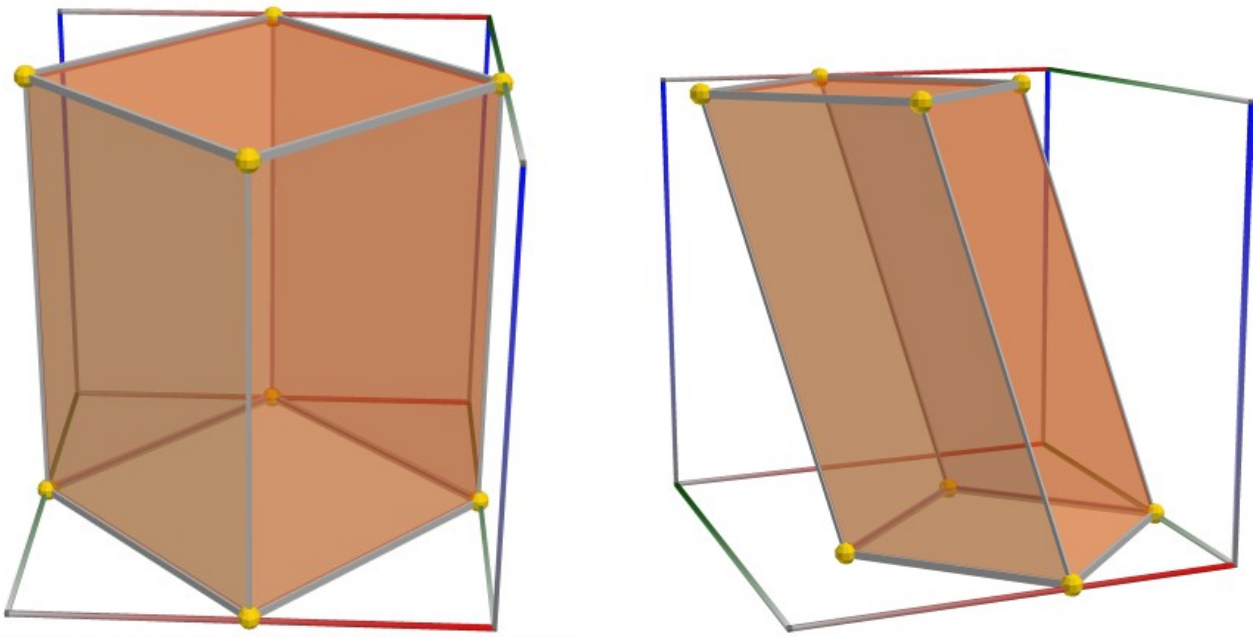
See also: https://mathworld.wolfram.com/Cube.html

See also: https://en.wikipedia.org/wiki/Cuboid

Cuboids have different types. A special case of a cuboid is a rectangular cuboid, with six rectangle faces and adjacent faces meeting at right angles. When all of the rectangular cuboid's edges are equal in length, it results in a cube, with six square faces and adjacent faces meeting at right angles.[1][3] Along with the rectangular cuboids, parallelepiped is a cuboid with six parallelogram. Rhombohedron is a cuboid with six rhombus faces. A square frustum is a frustum with a square base, but the rest of its faces are quadrilaterals.

The example below uses the following code in C#

```
var a = 0.50;
var b = 0.50;
var height = 0.55;
var proc = BuiltIn.SetSquarePrism(a, b, height, 0,0);
```



25.2.3 Hexagonal prism

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D03a_HexagonalPrism.3D.xml (D03a-b).

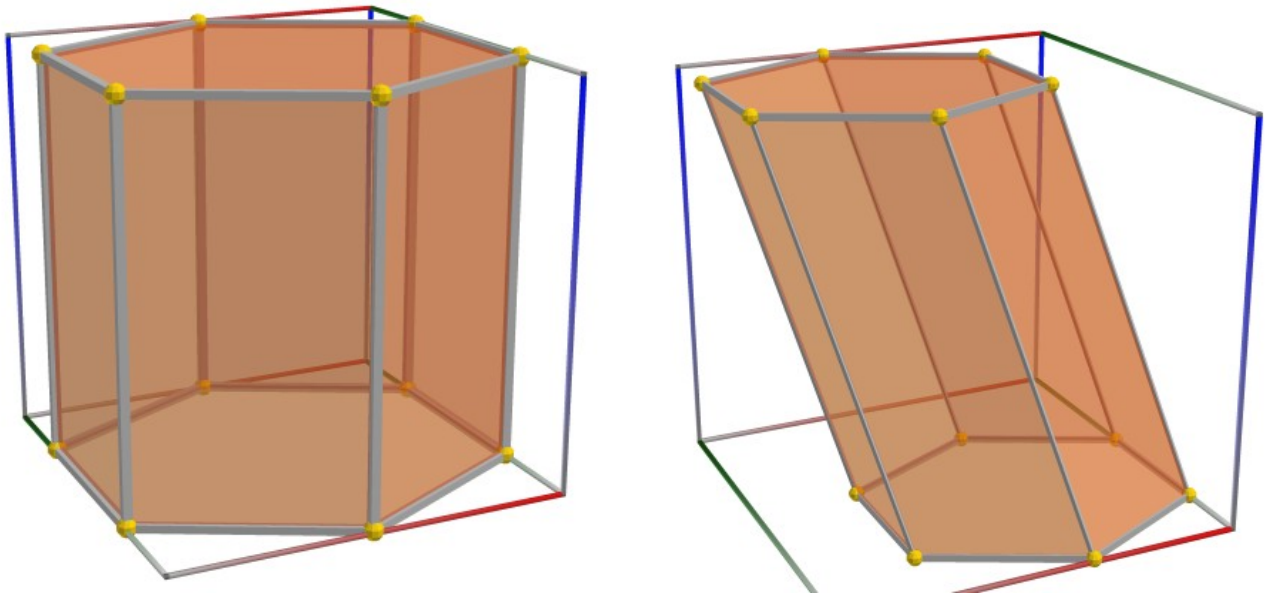
The hexagonal prism is a prism with hexagonal base. Prisms are polyhedrons; this polyhedron has 8 faces, 18 edges, and 12 vertices.

See also: https://mathworld.wolfram.com/HexagonalPrism.html

See also: https://en.wikipedia.org/wiki/Hexagonal_prism

The example below uses the following code in C#

```
var a = 0.50;
var b = 0.50;
var height = 1.0;
var proc = BuiltIn.SetHexagonalPrism(a, b, height, 0,0);
```



25.2.4 Octagonal prism

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D04a_OctagonalPrism.3D.xml (D04a-b).

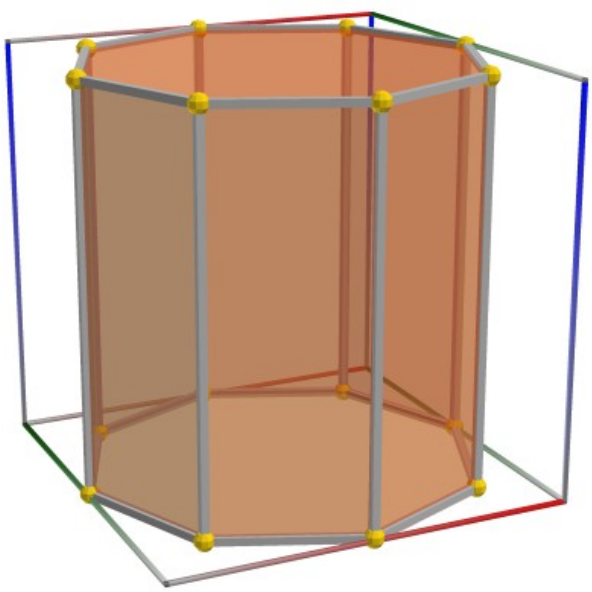
The octagonal prism is a prism comprising eight rectangular sides joining two regular octagon caps.

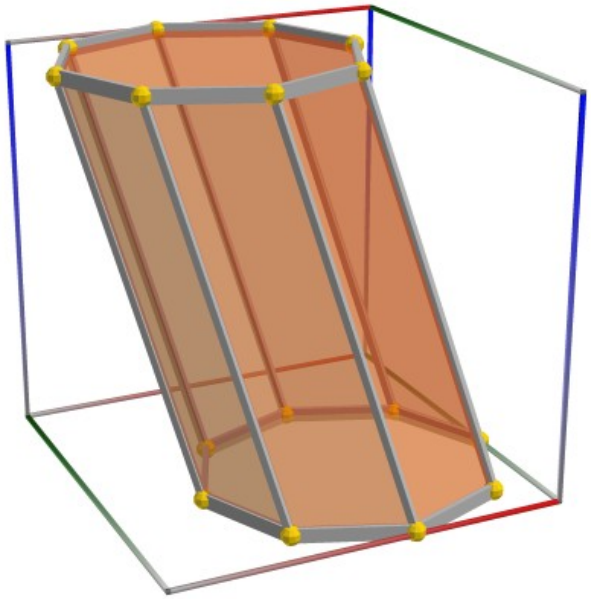
See also: https://mathworld.wolfram.com/OctagonalPrism.html

See also: https://en.wikipedia.org/wiki/Octagonal_prism

The example below uses the following code in C#

```
var a = 0.50;
var b = 0.50;
var height = 1.0;
var proc = BuiltIn.SetOctagonalPrism(a, b, height, 0,0);
```





25.2.5 Cylinder

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05a_Cylinder.3D.xml.

A cylinder is considered a prism with a circle as its base. The cylinder obtained by rotating a line segment about a fixed line that it is parallel to is a cylinder of revolution. A cylinder of revolution is a right circular cylinder.

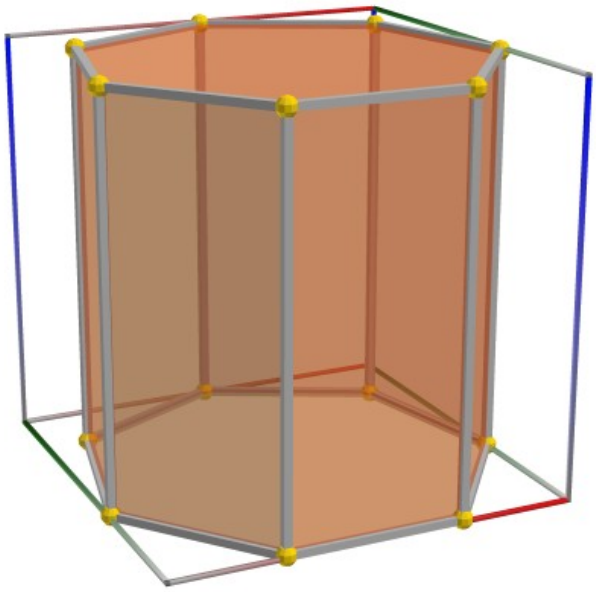
See also: https://en.wikipedia.org/wiki/Cylinder

See also: https://en.wikipedia.org/wiki/Right_circular_cylinder

See also: https://mathworld.wolfram.com/Cylinder.html

The example below uses the following code in C#

```
var numSides = 7;
var a = 0.50;
var b = 0.50;
var height = 1.0;
var proc = BuiltIn.SetCylinder(numSides, a, b, height, 0,0);
```



25.2.6 Cylinder, truncated by an inclined plane

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the files D06a_Cylinder_Tilted.3D.xml (D06a-b).

A cylinder is considered a prism with a circle as its base. The cylinder obtained by rotating a line segment about a fixed line that it is parallel to is a cylinder of revolution. A cylinder of revolution is a right circular cylinder.

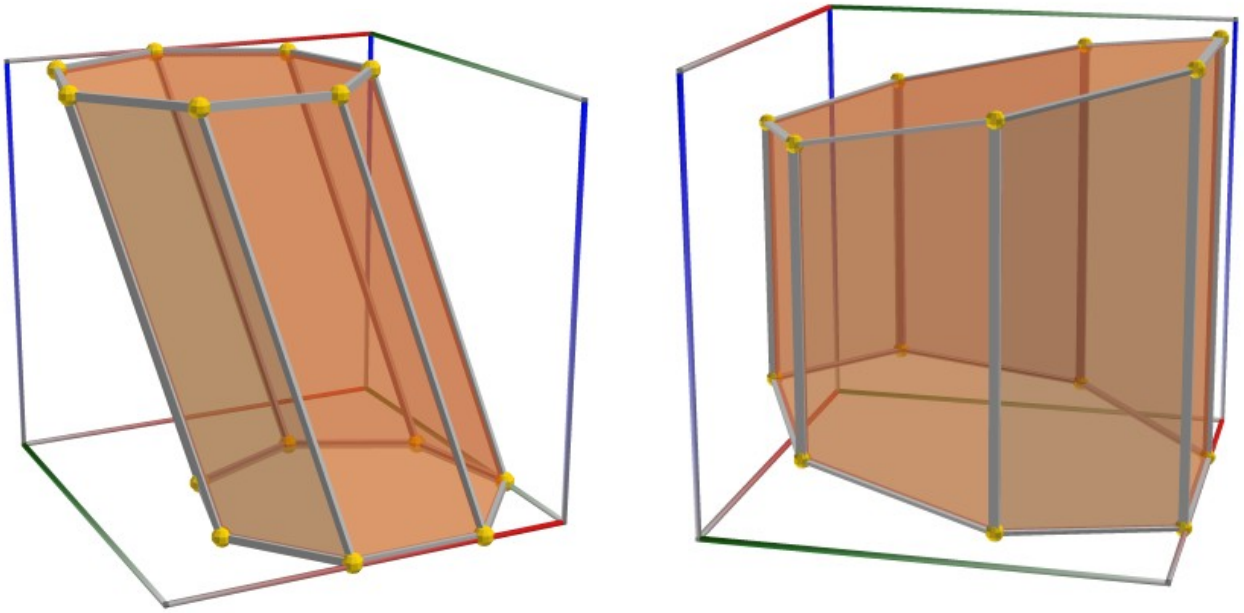
See also: https://en.wikipedia.org/wiki/Cylinder

See also: https://en.wikipedia.org/wiki/Right_circular_cylinder

See also: https://mathworld.wolfram.com/Cylinder.html

The example below uses the following code in C#

```
var numSides = 7;
var a = 0.50;
var b = 0.50;
var height = 1.0;
var cutslope1 = -0.5;
var cutslope2 = 0.5;
var proc = BuiltIn.SetCylinder2CP(numSides, a, b, height, cutslope1, cutslope2);
```



25.2.7 Pyramid

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_Pyramid.3D.xml.

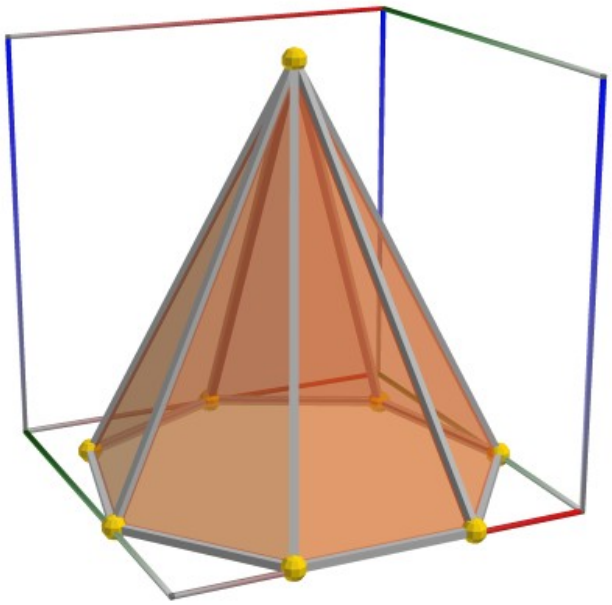
A pyramid is a polyhedron formed by connecting a polygonal base and a point, called the apex. Each base edge and apex form a triangle, called a lateral face. It is a conic solid with a polygonal base. Many types of pyramids can be found by determining the shape of bases, or cutting off the apex.

See also: https://en.wikipedia.org/wiki/Pyramid_(geometry)

See also: https://mathworld.wolfram.com/Pyramid.html

The example below uses the following code in C#

```
var numSides = 7;
var a = 0.50;
var b = 0.50;
var height = 1.0;
var proc = BuiltIn.SetPyramid(numSides, a, b, height);
```



25.2.8 Pyramid frustum

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08a_Frustum.3D.xml.

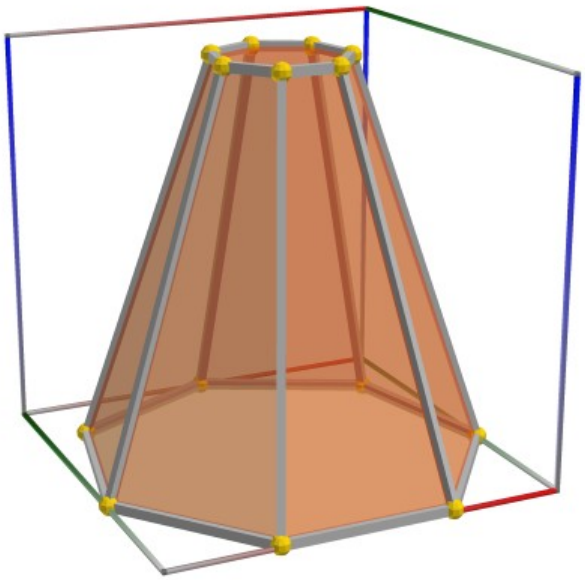
A frustum of a pyramid is the portion of the pyramid that lies between two parallel planes cutting the pyramid. In a truncated pyramid, the truncation plane is not necessarily parallel to the pyramid's base (as in a frustum), i.e. it is inclined.

See also: https://en.wikipedia.org/wiki/Frustum

See also: https://mathworld.wolfram.com/PyramidalFrustum.html

The example below uses the following code in C#

```
var numSides = 7;
var a = 0.50;
var b = 0.50;
var height = 1.2;
var cutheight = 0.8;
var cutslope = 0.0;
var proc = BuiltIn.SetFrustum(numSides, a, b, height, cutheight, cutslope);
```



25.2.9 Pyramid, truncated by an inclined plane

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09a_Frustum_Inclined.3D.xml.

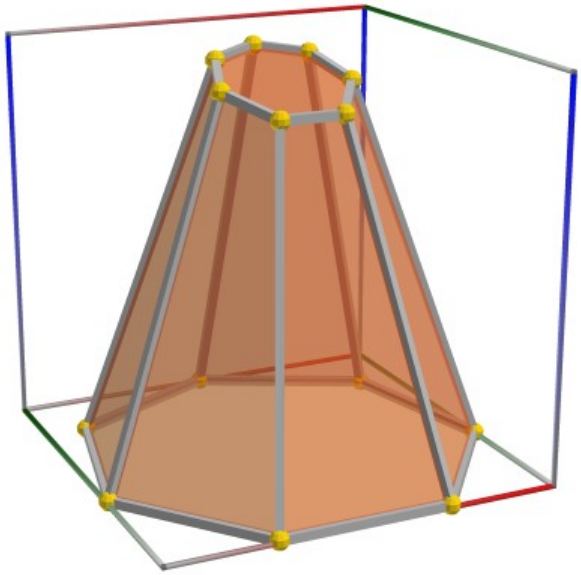
A frustum of a pyramid is the portion of the pyramid that lies between two parallel planes cutting the pyramid. In a truncated pyramid, the truncation plane is not necessarily parallel to the pyramid's base (as in a frustum), i.e. it is inclined.

See also: https://en.wikipedia.org/wiki/Frustum

See also: https://mathworld.wolfram.com/PyramidalFrustum.html

The example below uses the following code in C#

```
var numSides = 7;
var a = 0.50;
var b = 0.50;
var height = 1.2;
var cutheight = 0.8;
var cutslope = 0.3;
var proc = BuiltIn.SetFrustum(numSides, a, b, height, cutheight, cutslope);
```



25.2.10 Cone

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10a_Cone.3D.xml.

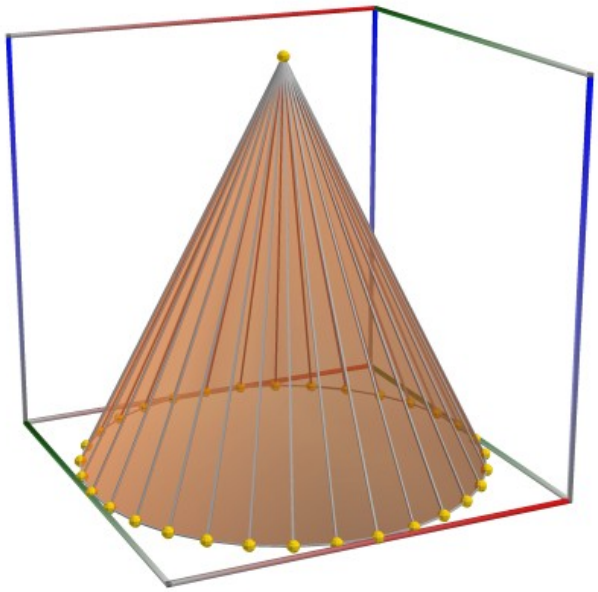
A cone is a three-dimensional geometric shape that tapers smoothly from a flat base (frequently, though not necessarily, circular) to a point called the apex or vertex. A cone with a polygonal base is called a pyramid.

See also: https://en.wikipedia.org/wiki/Cone

See also: https://mathworld.wolfram.com/Cone.html

The example below uses the following code in C#

```
var numSides = 32;
var a = 0.50;
var b = 0.50;
var height = 1.0;
var proc = BuiltIn.SetCone(numSides, a, b, height);
```



25.2.11 Cone Frustum

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11a_ConeFrustum.3D.xml.

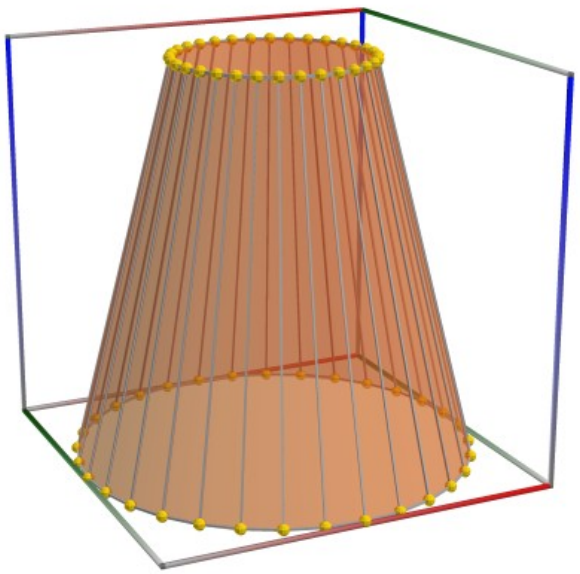
A cone is a three-dimensional geometric shape that tapers smoothly from a flat base (frequently, though not necessarily, circular) to a point called the apex or vertex. A cone with a polygonal base is called a pyramid.

See also: https://en.wikipedia.org/wiki/Frustum

See also: https://mathworld.wolfram.com/ConicalFrustum.html

The example below uses the following code in C#

```
var numSides = 32;
var a = 0.50;
var b = 0.50;
var height = 1.2;
var cutheight = 0.6;
var cutslope = 0.0;
var proc = BuiltIn.SetConeFrustum(numSides, a, b, height, cutheight, cutslope);
```



25.2.12 Cone, truncated by an inclined plane

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D12a_ConeFrustum_Inclined.3D.xml.

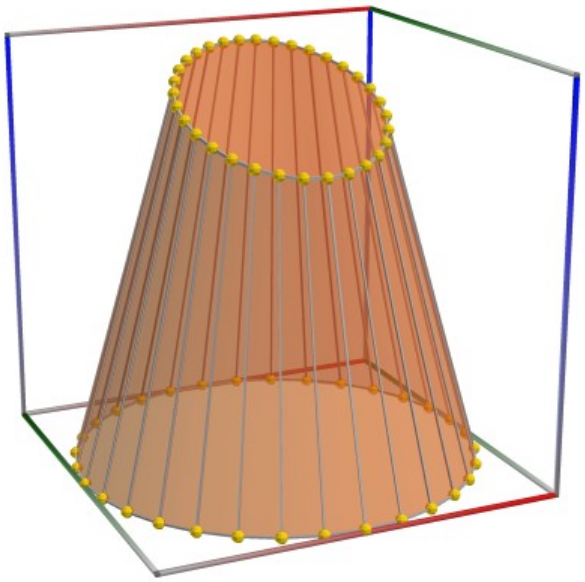
A cone is a three-dimensional geometric shape that tapers smoothly from a flat base (frequently, though not necessarily, circular) to a point called the apex or vertex. A cone with a polygonal base is called a pyramid.

See also: https://en.wikipedia.org/wiki/Frustum

See also: https://mathworld.wolfram.com/ConicalFrustum.html

The example below uses the following code in C#

```
var numSides = 32;
var a = 0.50;
var b = 0.50;
var height = 1.2;
var cutheight = 0.6;
var cutslope = 0.3;
var proc = BuiltIn.SetConeFrustum(numSides, a, b, height, cutheight, cutslope);
```



25.3 Platonic solids, and related solids

A Platonic solid is a convex, regular polyhedron in three-dimensional Euclidean space. Being a regular polyhedron means that the faces are congruent (identical in shape and size) regular polygons (all angles congruent and all edges congruent), and the same number of faces meet at each vertex.

See also: https://en.wikipedia.org/wiki/Platonic_solid

See also: https://mathworld.wolfram.com/PlatonicSolid.html

Augmentation is the operation of replacing the faces of a polyhedron with pyramids of height h (where h may be positive, zero, or negative) having the face as the base. Augmentation with $h = 0$ gives a triangulated version of the original solid.

See also: https://mathworld.wolfram.com/Augmentation.html

25.3.1 Tetrahedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D01a_Tetrahedron.3D.xml.

A regular tetrahedron is a tetrahedron in which all four faces are equilateral triangles. In other words, all of its faces are the same size and shape (congruent) and all edges are the same length.

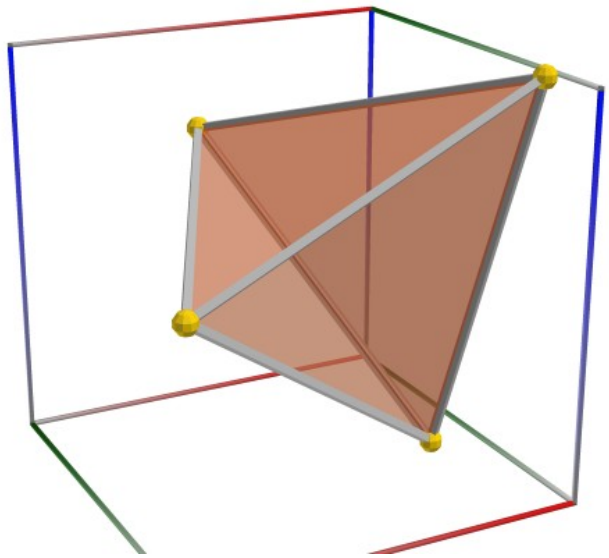
See also: https://en.wikipedia.org/wiki/Tetrahedron#Regular_tetrahedron

See also: https://mathworld.wolfram.com/RegularTetrahedron.html

See also: https://mathcurve.com/polyedres/tetraedre/tetraedre.shtml

The example below uses the following code in C#

```
var proc = BuiltIn.SetTetrahedron();
```



25.3.2 Cube

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D02a_Cube.3D.xml.

A cube is a three-dimensional solid object bounded by six square faces. It has twelve edges and eight vertices. It can be represented as a rectangular cuboid with six square faces, or a parallelepiped with equal edges.

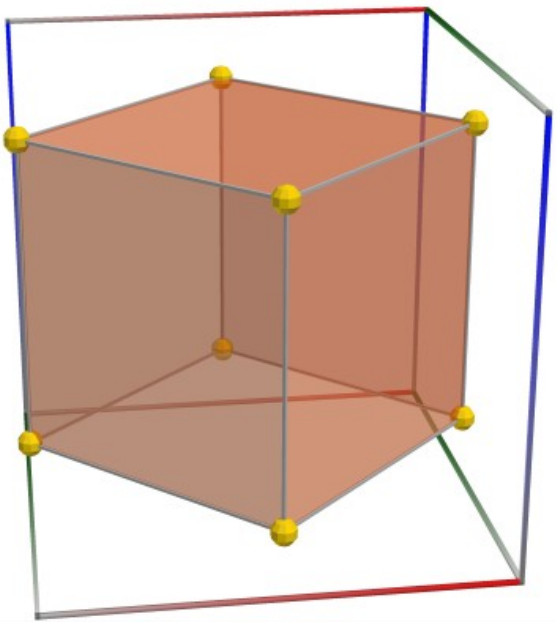
See also: https://en.wikipedia.org/wiki/Cube

See also: https://mathworld.wolfram.com/Cube.html

See also: https://mathcurve.com/polyedres/cube/cube.shtml

The example below uses the following code in C#

```
var proc = BuiltIn.SetCube();
```



25.3.3 Octahedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D03a_Octahedron.3D.xml.

A regular octahedron is an octahedron that is a regular polyhedron. All the faces of a regular octahedron are equilateral triangles of the same size, and exactly four triangles meet at each vertex. A regular octahedron is convex, meaning that for any two points within it, the line segment connecting them lies entirely within it.

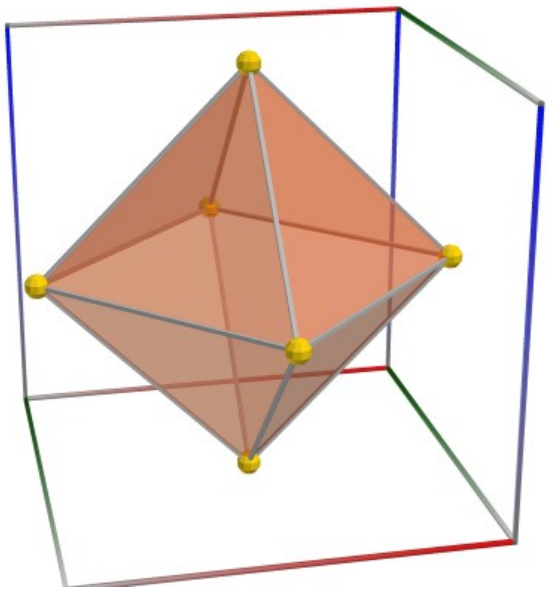
See also: https://en.wikipedia.org/wiki/Octahedron#Regular_octahedron

See also: https://mathworld.wolfram.com/RegularOctahedron.html

See also: https://mathcurve.com/polyedres/octaedre/octaedre.shtml

The example below uses the following code in C#

```
var proc = BuiltIn.SetOctahedron();
```



25.3.4 Dodecahedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D04a_Dodecahedron.3D.xml.

A regular dodecahedron or pentagonal dodecahedron is a dodecahedron composed of regular pentagonal faces, three meeting at each vertex.

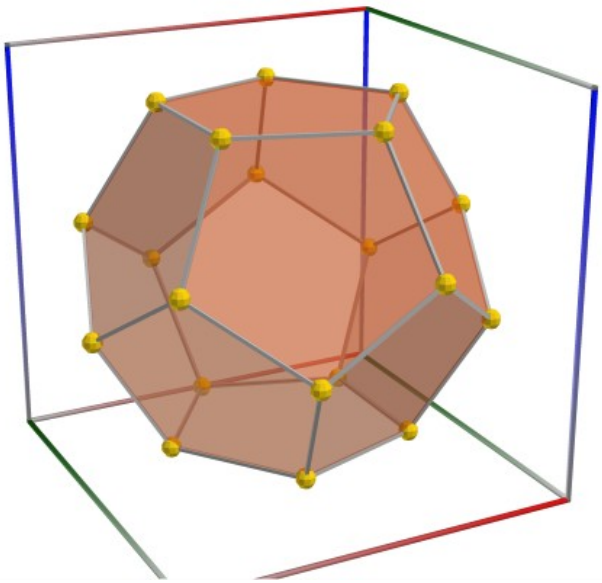
See also: https://en.wikipedia.org/wiki/Regular_dodecahedron

See also: https://mathworld.wolfram.com/RegularDodecahedron.html

See also: https://mathcurve.com/polyedres/dodecaedre/dodecaedre.shtml

The example below uses the following code in C#

```
var proc = BuiltIn.SetDodecahedron();
```



25.3.5 Icosahedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D05a_Icosahedron.3D.xml.

A regular icosahedron (or simply icosahedron) is a convex polyhedron that can be constructed from pentagonal antiprism by attaching two pentagonal pyramids with regular faces to each of its pentagonal faces, or by putting points onto the cube.

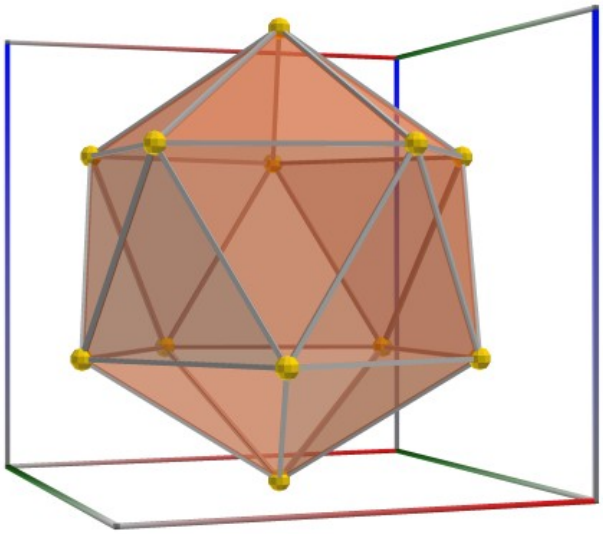
See also: https://en.wikipedia.org/wiki/Regular_icosahedron

See also: https://mathworld.wolfram.com/RegularIcosahedron.html

See also: https://mathcurve.com/polyedres/icosaedre/icosaedre.shtml

The example below uses the following code in C#

```
var proc = BuiltIn.SetIcosahedron();
```



25.3.6 Geodesic sphere

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D06a_Geodesic_Sphere1.3D.xml (D06a-f).

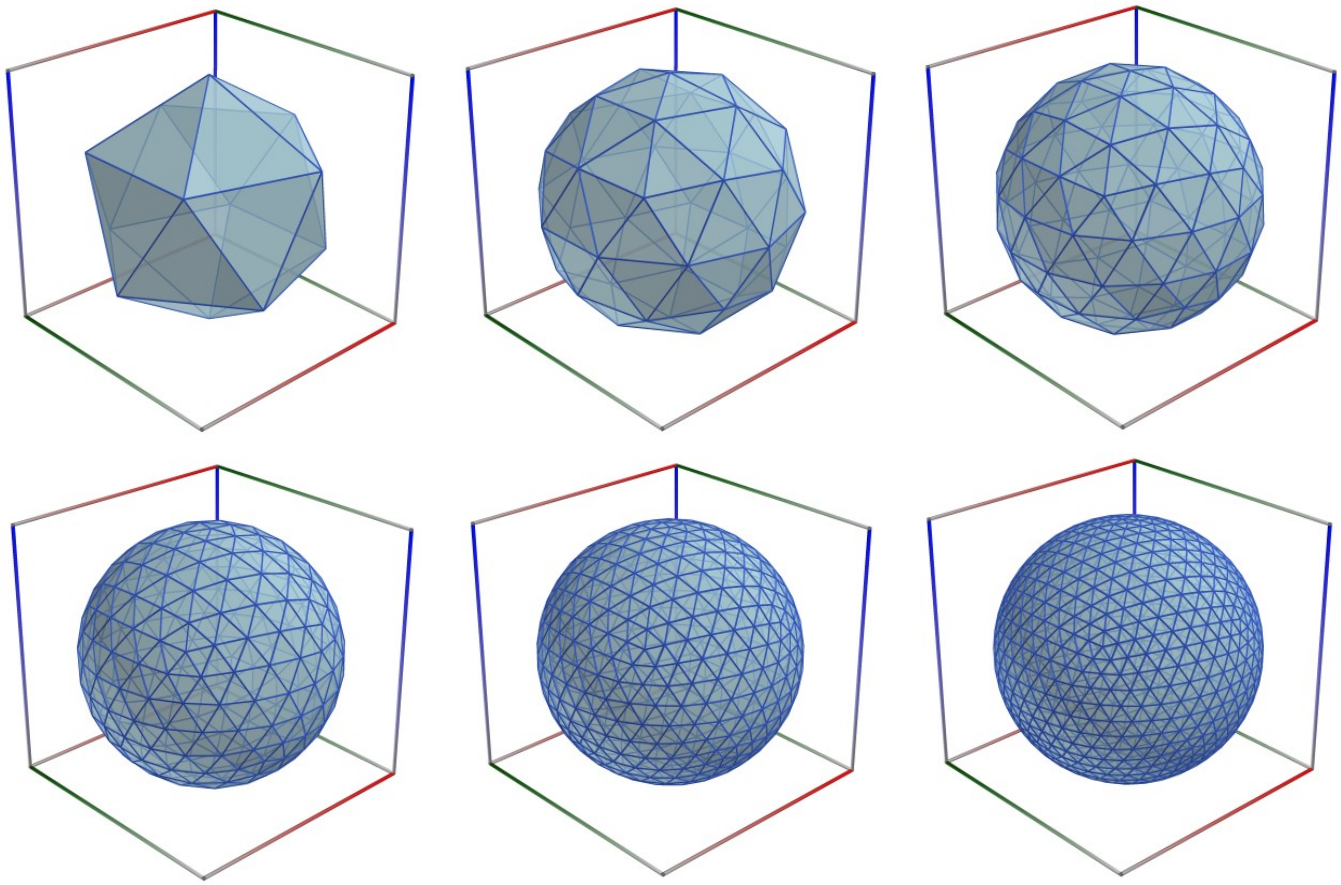
A spherical polyhedron or spherical tiling is a tiling of the sphere in which the surface is divided or partitioned by great arcs into bounded regions called spherical polygons

See also: https://en.wikipedia.org/wiki/Geodesic_polyhedron

See also: https://en.wikipedia.org/wiki/Spherical_polyhedron

The example below uses the following code in C#

```
var radius = 1.0;
var numDiv = 1;
var proc = BuiltIn.SetGeodesicSphere(radius, numDiv);
```



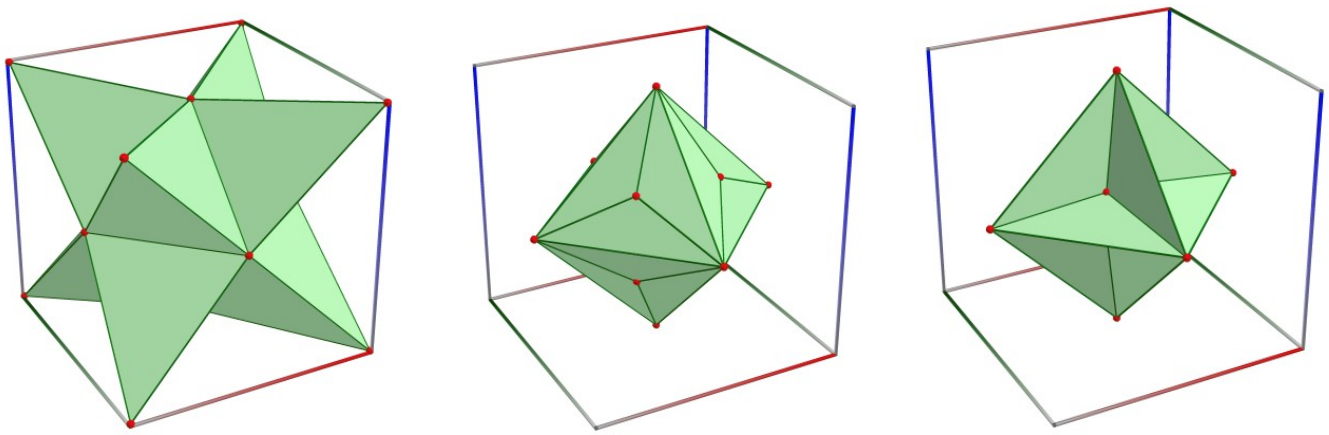
25.3.7 Augmented Octahedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07a_Augm_Octahedron_a.3D.xml (D07a-c).

Returns the augmented Octahedron.

The example below uses the following code in C#

```
var starRadius = 3.0;
var proc = BuiltIn.SetAugmentedOctahedron(starRadius );
```



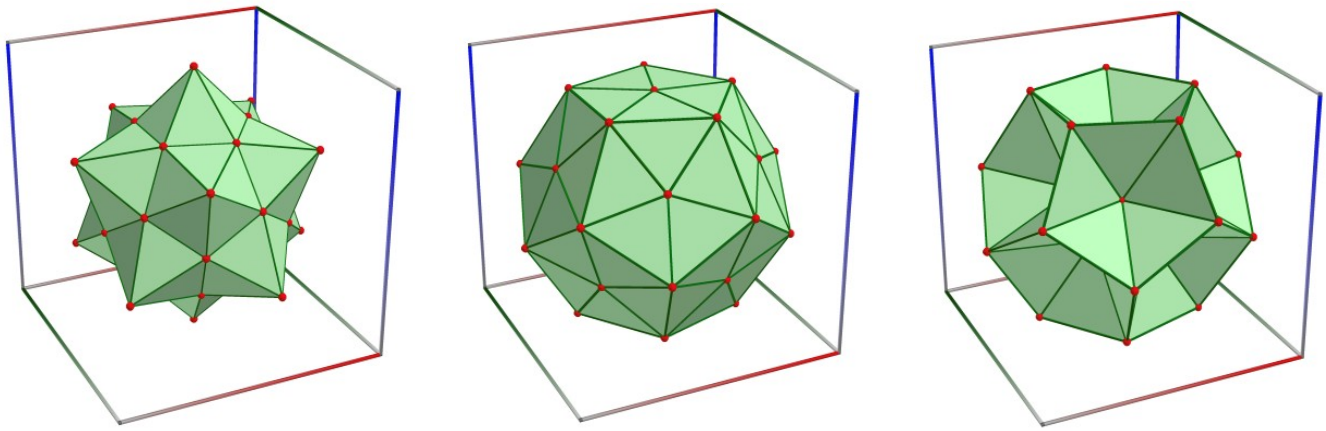
25.3.8 Augmented Dodecahedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08a_Augm_Dodecahedron_a.3D.xml (D08a-c).

Returns the augmented Dodecahedron.

The example below uses the following code in C#

```
var starRadius = 6.0;
var proc = BuiltIn.SetAugmentedDodecahedron(starRadius );
```



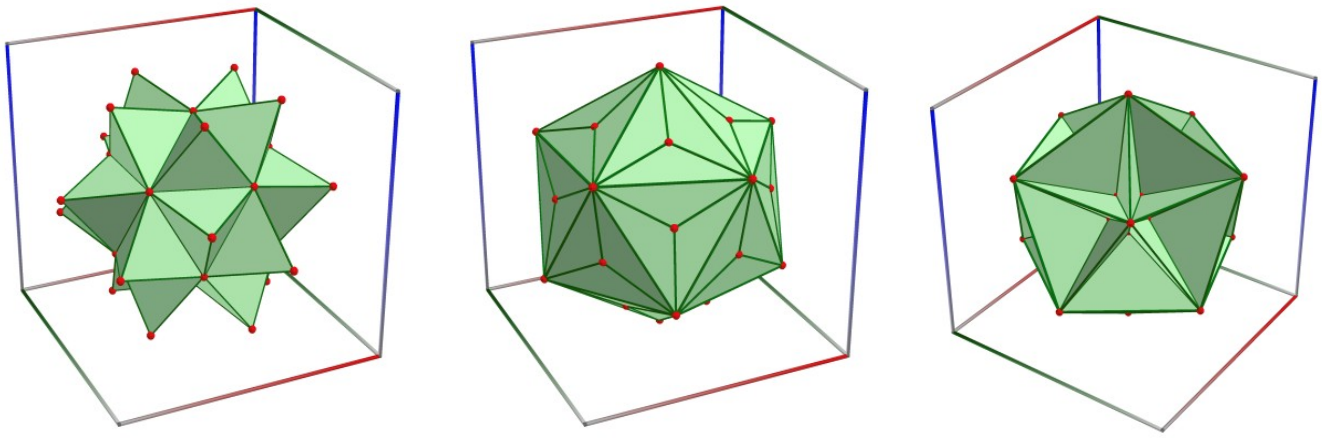
25.3.9 Augmented Icosahedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D09a_Augm_Icosahedron_a.3D.xml (D09a-c).

Returns the augmented Dodecahedron.

The example below uses the following code in C#

```
var starRadius = 2.0;
var proc = BuiltIn.SetAugmentedIcosahedron(starRadius );
```



25.3.10 Augmented Geodesic Sphere

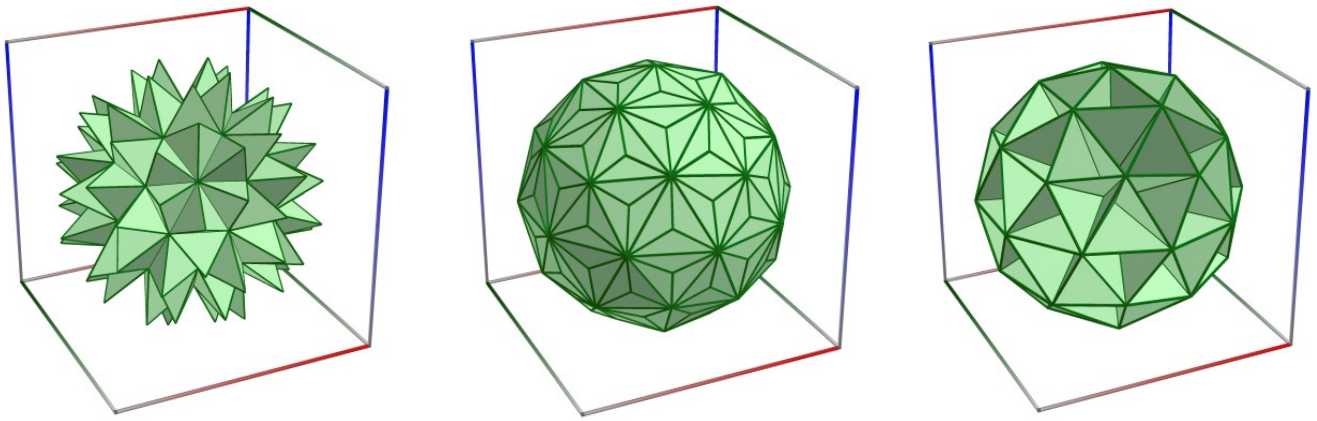
The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D10a_Augm_Geodesic_a.3D.xml (D10a-c).

Returns augmented versions of the Geodesic.

See also: https://mathworld.wolfram.com/IcosahedronStellations.html

The example below uses the following code in C#

```
var starRadius = 2.0;
var numDiv = 2;
var proc = BuiltIn.SetAugmentedGeodesic(starRadius, numDiv);
```



25.3.11 Stella Octangula

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D11a_StellaOctangula.3D.xml (D10a-c).

The stella octangula is a polyhedron compound composed of a tetrahedron and its dual (a second tetrahedron rotated 180 degrees with respect to the first). The stella octangula is also (incorrectly) called the augmented tetrahedron, and is the only stellation of the octahedron.

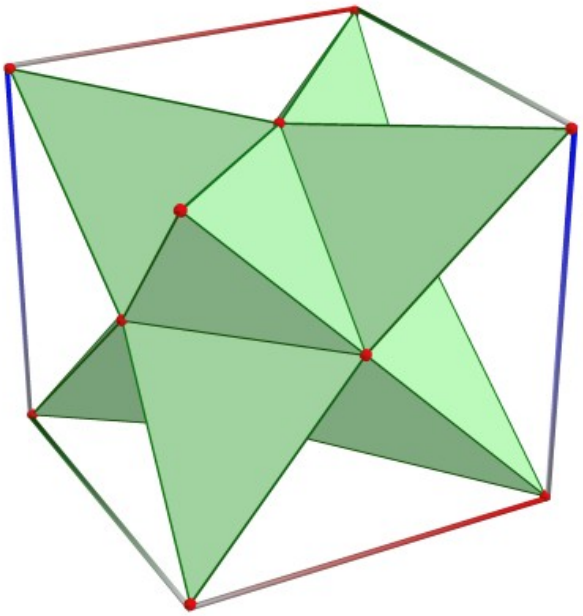
It can be constructed from a regular octahedron by augmentation with $h = \sqrt{6}/3$.

See also: https://mathworld.wolfram.com/StellaOctangula.html

See also: https://en.wikipedia.org/wiki/Stellated_octahedron

The example below uses the following code in C#

```
var starRadius = 2.0;
var proc = BuiltIn.SetAugmentedOctahedron(starRadius );
```



25.3.12 Small stellated Dodecahedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D12a_SmallStellatedDodecahedron.3D.xml (D10a-c).

The small stellated dodecahedron is the Kepler-Poinsot polyhedra whose dual polyhedron is the great dodecahedron.

It can be constructed from a regular dodecahedron by augmentation with $h = \sqrt{(5 + 2\sqrt{5})/5}$.

See also: https://mathworld.wolfram.com/SmallStellatedDodecahedron.html

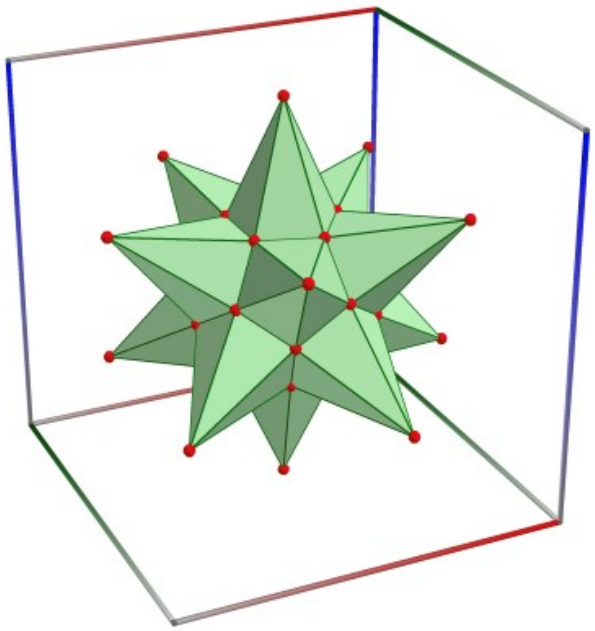
See also: https://mathworld.wolfram.com/Augmentation.html

See also: https://en.wikipedia.org/wiki/Small_stellated_dodecahedron

See also: https://mathworld.wolfram.com/Kepler-PoinsotPolyhedron.html

The example below uses the following code in C#

```
var starRadius = 6.0;
var proc = BuiltIn.SetAugmentedDodecahedron(starRadius );
```



25.3.13 Great Dodecahedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D13a_GreatDodecahedron.3D.xml (D10a-c).

The great dodecahedron is the Kepler-Poinsot polyhedron whose dual is the small augmented dodecahedron.

It can be constructed from a regular icosahedron by augmentation with $h = (\sqrt{3}(\sqrt{5} - 3))/6$.

See also: https://mathworld.wolfram.com/GreatDodecahedron.html

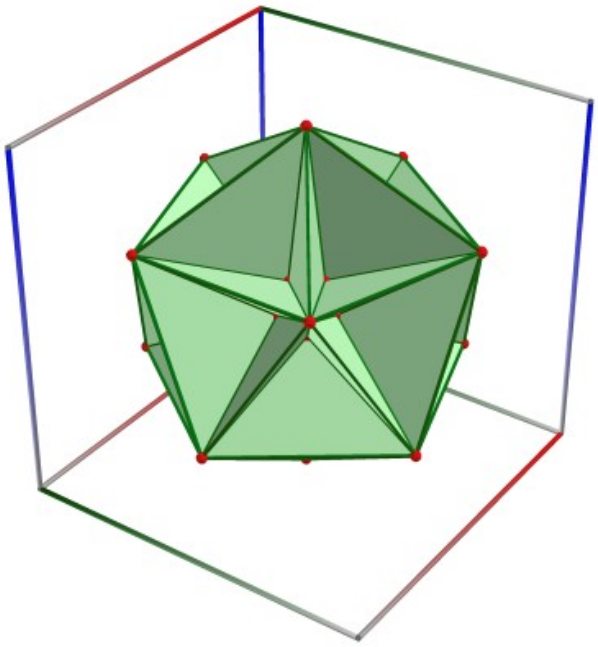
See also: https://mathworld.wolfram.com/Augmentation.html

See also: https://en.wikipedia.org/wiki/Great_dodecahedron

See also: https://mathworld.wolfram.com/Kepler-PoinsotPolyhedron.html

The example below uses the following code in C#

```
var starRadius = 0.25;
var proc = BuiltIn.SetAugmentedIcosahedron(starRadius );
```



25.3.14 Great stellated Dodecahedron

The XML code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D14a_GreatStellatedDodecahedron.3D.xml (D10a-c).

The great augmented dodecahedron is one of the Kepler-Poinsot polyhedra.

It can be constructed from a regular icosahedron by augmentation with $h = (\sqrt{3}(3 + \sqrt{5}))/6$.

See also: https://mathworld.wolfram.com/GreatStellatedDodecahedron.html

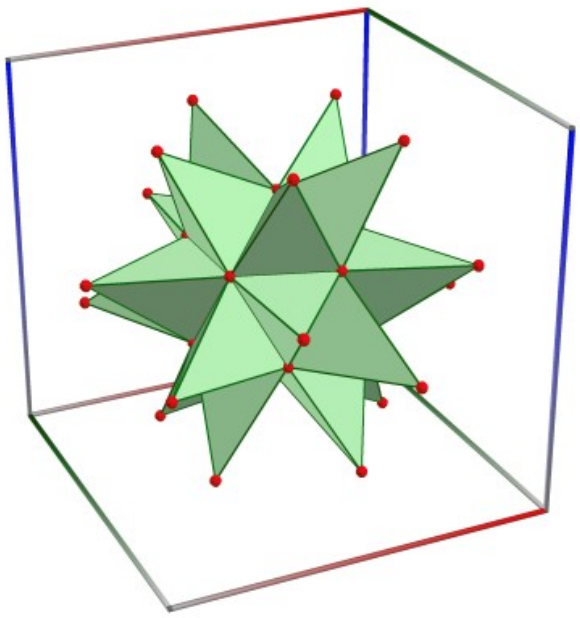
See also: https://mathworld.wolfram.com/Augmentation.html

See also: https://en.wikipedia.org/wiki/Great_stellated_dodecahedron

See also: https://mathworld.wolfram.com/Kepler-PoinsotPolyhedron.html

The example below uses the following code in C#

```
var starRadius = 2.0;
var proc = BuiltIn.SetAugmentedIcosahedron(starRadius );
```



Part VI

Additional functions in double precision

CHAPTER

TWENTYSIX

REAL ELEMENTARY FUNCTIONS

26.1 Root, exponential, logarithmic and power functions

26.1.1 Auxiliary function $\mathrm{sqrt1pmx}(x) = \sqrt{1+x^2} - x$

`math53.sqrt1pmx`(x)

Returns $\sqrt{1+x^2} - x$, accurate also for x near 0.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.sqrt1pmx(-4.333)
8.779896558275222
>>> math53.sqrt1pmx(0.0)
1.0
>>> math53.sqrt1pmx(4.333)
0.1138965582752203
```

26.1.2 Bring radical

`math53.bring`(x)

Returns the Bring radical $b = \mathrm{BR}(x)$ of x, i.e. the unique real b with $b^5 + b + x = 0$. The function can be used (together with standard radicals) to solve a class of quintic equations in closed form.

See also: https://en.wikipedia.org/wiki/Bring_radical, Ehrhardt [297] (3.10.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.bring(-4.333)
1.252357338752218
>>> math53.bring(0.0)
0.0
>>> math53.bring(4.333)
-1.252357338752218
```

26.1.3 Auxiliary function $\mathrm{expmx2h}(x) = \exp(-x^2/2)$

`math53.expmx2h`(x)

Returns $\exp(-x^2/2)$ with damped error amplification.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.expmx2h(-4.333)
8.376925527113537e-05
>>> math53.expmx2h(0.0)
1.0
>>> math53.expmx2h(4.333)
8.376925527113537e-05
```

26.1.4 Relative error exponential $\mathrm{exprel}(x) = (\exp(x) - 1)/x$

`math53.exprel`(x)

where `ctx` is `math53`, `ctxcpp` or `ctxflint`.

Returns exprel(x) = $(\exp(x) - 1)/x$, 1 for $x = 0$.

$$\mathrm{exprel}(x) = \begin{cases} (\exp(x) - 1)/x = \mathrm{expm1}(x)/x, & \text{if } x \neq 0 \\ 1, & \text{if } x = 0. \end{cases}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.exprel(-4.333)
0.22775718808161466
>>> math53.exprel(0.0)
1.0
>>> math53.exprel(4.333)
17.348825645093516
```

26.1.5 Auxiliary function $\mathrm{expx2}(x) = \exp(x \cdot |x|)$

math53.**expx2**(x)

Returns $\exp(x \cdot |x|)$ with damped error amplification.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.expx2(-4.333)
7.01728812868064e-09
>>> math53.expx2(0.0)
1.0
>>> math53.expx2(4.333)
142505193.1262249
```

26.1.6 Auxiliary function $\mathrm{logistic}(x) = 1/(1 + \exp(-x))$

math53.**logistic**(x)

Returns $\mathrm{logistic}(x) = 1/(1 + \exp(-x))$. See also Wikipedia [1348], MathWorld [1008].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logistic(-4.333)
0.012957990198853193
>>> math53.logistic(0.0)
0.5
>>> math53.logistic(4.333)
0.9870420098011469
```

26.1.7 Einstein functions

math53.**einstein**(n, x)

Returns the Einstein function E_n for $n = 1, 2, 3, 4$, with $E_1(x) = \dfrac{x^2 e^x}{(e^x - 1)^2}$, $E_2(x) = \dfrac{x}{e^x - 1}$, $E_3(x) = \log(1 - e^{-x}), x > 0$, $E_4(x) = \dfrac{x}{e^x - 1} - \log(1 - e^{-x}), x > 0$.

See also https://mathworld.wolfram.com/EinsteinFunctions.html

See also: Abramowitz and Stegun. [4] table 27.3, Ehrhardt [297] (3.10.7).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.einstein(1,4.333)
0.253079999702088
>>> math53.einstein(2,4.333)
0.05764078909184343
>>> math53.einstein(3,4.333)
```

(continues on next page)

(continued from previous page)

```
-0.013215039301928083
>>> math53.einstein(4,4.333)
0.07085582839377151
```

26.1.8 Auxiliary function $\mathrm{log1mexp}(x) = \log(1 - \exp(x))$

`math53.log1mexp`(x)

Returns $\log(1 - \exp(x)), x < 0$, calculated in an accurate and efficient way.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.log1mexp(-4.333)
-0.013215039301928083
>>> math53.log1mexp(0.0)
-inf
>>> math53.log1mexp(4.333)
nan
```

26.1.9 Auxiliary function $\mathrm{log1pexp}(x) = \log(1 + \exp(x))$

`math53.log1pexp`(x)

Returns $\mathrm{ln1pexp}(x)) = \log(1 + \exp(x)) = \mathrm{log1p}(\exp(x))$. See also *log1p()*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.log1pexp(-4.333)
0.013042677332710145
>>> math53.log1pexp(0.0)
0.6931471805599453
>>> math53.log1pexp(4.333)
4.3460426773327105
```

26.1.10 Auxiliary function $\mathrm{log1pmx}(x) = \log(1 + x) - x$

`math53.log1pmx`(x)

Returns $\mathrm{ln1pmx}(x)) = \log(1 + x) - x = \mathrm{log1p}(x) - x$, accurate also for $-0.5 \leq x \leq 0.5$. See also *log1p()*.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.log1pmx(-4.333)
nan
>>> math53.log1pmx(0.0)
0.0
>>> math53.log1pmx(4.333)
-2.659086068381535
```

26.1.11 Auxiliary function $\mathrm{logaddexp}(x, y) = \log[\exp(x) + \exp(y)]$

`math53.logaddexp`(x, y)

Accurately compute ln[exp(x) + exp(y)].

See also: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/powers/logaddexp.html

See also: https://nhigham.com/2021/01/05/what-is-the-log-sum-exp-function/

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logaddexp(1.2, -4.333)
1.203946312008691
>>> math53.logaddexp(3.7, 0)
3.724422845933779
>>> math53.logaddexp(4.8, 4.333)
5.2866641257129325
```

26.1.12 Auxiliary function $\mathrm{logsubexp}(x, y) = \log[\exp(x) - \exp(y)]$

`math53.logsubexp`(x, y)

Accurately compute $\log[\exp(x) - \exp(y)], x > y$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logsubexp(1.2, -4.333)
1.1960380528914856
>>> math53.logsubexp(3.7, 0)
3.6749657144139602
>>> math53.logsubexp(4.8, 4.333)
3.8141445624254304
```

26.1.13 Auxiliary function $\mathrm{logit}(x) = \log(x/(1.0 - x))$

`math53.logit`(x)

Returns $\mathrm{logit}(x) = \log(x/(1.0 - x))$, accurate also near $x = 0.5$. See also Wikipedia [1349], MathWorld [1009].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logit(-0.333)
nan
>>> math53.logit(0.0)
-inf
>>> math53.logit(0.333)
-0.6946475559351799
```

26.1.14 Wright ω function

`math53.wright_omega`(x)

Returns the Wright $\omega(x)$ function, which is defined as the unique solution of $\omega(x) + \log(\omega(x)) = x$. For real x it can be written in terms of the Lambert W function as $\omega(x) = W(e^x)$

See also: Ehrhardt [297] (3.10.26).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.wright_omega(-0.333)
0.4548304865283553
>>> math53.wright_omega(0.0)
0.5671432904097838
>>> math53.wright_omega(0.333)
0.695756104553223
```

26.1.15 Auxiliary function $\mathrm{compound}(x, n) = (1 + x)^n$

`ctx.compound_si`(*x*, *n*)

Returns $(1 + x)^n$, computed accurately also when x is very close to 0.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.compound_si(-4.333, 3)
-37.02592703700001
>>> math53.compound_si(0.0, 2)
1.0
>>> math53.compound_si(4.333, 3)
151.675261037
```

26.1.16 Auxiliary function $\mathrm{comprel}(x, n) = (1 + x)^n - 1$

`ctx.comprel`(*x*, *n*)

Returns $(1 + x)^n - 1$, computed accurately also when x is very close to 0.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.comprel(-4.333, 3)
8.775889000000001
>>> math53.comprel(0.0, 2)
2.0
>>> math53.comprel(4.333, 3)
34.773889000000004
```

26.1.17 Auxiliary function $\mathrm{hypot3}(x, y, z) = \sqrt{x^2 + y^2 + z^2}$

`math53.hypot3`(*x*, *y*, *z*)

Returns $\mathrm{hypot3}(x, y, z) = \sqrt{x^2 + y^2 + z^2}$ in a way which avoids undue underflow and overflow. See also BoostMath [122], Wikipedia [1286].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.hypot3(1.2, -4.333, -1.333)
4.689539209773174
>>> math53.hypot3(2.4, 0, 0)
2.4
>>> math53.hypot3(4.8, -4.333, -1.333)
6.602406985334969
```

26.1.18 Fibonacci polynomials, $\mathrm{fibpoly}(n, x)$

`math53.fibpoly`(*n*, *x*)

where `ctx` is `math53`, `ctxcpp` or `ctxflint`.

Returns $F_n(x)$, the Fibonacci polynomial of index $n \in \mathbb{Z}$. See also Wikipedia [1342], Wikipedia [1343], Jin [383], MathWorld [1002], MathWorld [1003], Ehrhardt [297] (3.10.11).

https://en.wikipedia.org/wiki/Fibonacci_polynomials#Properties

https://functions.wolfram.com/HypergeometricFunctions/Fibonacci2General/26/03/01/0002/

For any non-negative integer n, the Fibonacci polynomials $\{F_n(x)\}$ are defined by the second order linear recursive formula $F_{n+2}(x) = xF_{n+1}(x) + F_n(x)$ with $F_0(x) = 0$, $F_1(x) = 1$, $L_0(x) = 2$, and $L_1(x) = x$. The general terms of $F_n(x)$ are

given by

$$F_n(x) = \frac{1}{\sqrt{x^2+4}}\left[\left(\frac{x+\sqrt{x^2+4}}{2}\right)^n - \left(\frac{x-\sqrt{x^2+4}}{2}\right)^n\right]$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.fibpoly(2, -4.333)
-4.333
>>> math53.fibpoly(3, -4.333)
19.774889
>>> math53.fibpoly(4, 4.333)
90.01759403700001
```

26.1.19 Lucas polynomials, $\mathrm{lucpoly}(n, x)$

`math53.lucpoly(`n, x`)`

where `ctx` is `math53`, `ctxcpp` or `ctxflint`.

Returns $L_n(x)$, the Lucas polynomial of index $n \in \mathbb{Z}$. See also Ehrhardt [297] (3.10.18), Wikipedia [1350], Wikipedia [1351], Jin [383], MathWorld [1010], MathWorld [1011].

For any non-negative integer n, Lucas polynomials $\{L_n(x)\}$ are defined by the second order linear recursive formulas $L_{n+2}(x) = xL_{n+1}(x) + L_n(x)$, $L_0(x) = 2$, and $L_1(x) = x$. For negative indices we have $L_{-n}(x) = (-1)^n L_n(x)$. The general terms of $L_n(x)$ are given by

$$L_n(x) = \left(\frac{x+\sqrt{x^2+4}}{2}\right)^n + \left(\frac{x-\sqrt{x^2+4}}{2}\right)^n.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.lucpoly(2, -4.333)
20.774889
>>> math53.lucpoly(3, -4.333)
-94.35059403700001
>>> math53.lucpoly(4, 4.333)
429.5960129623211
```

26.1.20 Fibonacci function, $F_\nu(x)$, of real index ν

`math53.fibfun(`ν, x`)`

Returns $F_\nu(x) = \dfrac{2^{-\nu}(x+\sqrt{x^2+4})^\nu - \cos(\pi\nu)\cdot 2^\nu (x+\sqrt{x^2+4})^{-\nu}}{\sqrt{x^2+4}}$, the general Fibonacci function of real index ν.

See also Ehrhardt [297] (3.10.12) , https://en.wikipedia.org/wiki/Fibonacci_polynomials.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.fibfun(1.2, -4.333)
1.0790638603231733
>>> math53.fibfun(3.7, -4.333)
-33.57833497912526
>>> math53.fibfun(4.8, 4.333)
302.6505730178296
```

26.2 Trigonometric functions

26.2.1 Sine, x in degrees, $\mathrm{sind}(x)$

`math53.sind(x)`

Returns the sine of x, with x in degrees, $\mathrm{sind}(x)$. See also Wikipedia [1244], MathWorld [874], NIST [492].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.sind(-4.333)
-0.07555305176474306
>>> math53.sind(0.0)
0.0
>>> math53.sind(4.333)
0.07555305176474306
```

26.2.2 Inverse sine, input in degrees, $\mathrm{asind}(x)$

`math53.asind(x)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the inverse sine of x, $\mathrm{asin}(x)$. See also Wikipedia [1260], MathWorld [871], NIST [487], Ehrhardt [297] (4.2.13), Mpmath [577].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.asind(-0.333)
-19.45096478286684
>>> math53.asind(0.0)
0.0
>>> math53.asind(0.333)
19.45096478286684
```

26.2.3 Cosine, x in degrees, $\mathrm{cosd}(x)$

`math53.cosd(x)`

Returns the cosine of x, with x in degrees, $\mathrm{cosd}(x)$. See also Wikipedia [1247], MathWorld [859], NIST [492].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.cosd(-4.333)
0.99714178348369
>>> math53.cosd(0.0)
1.0
>>> math53.cosd(4.333)
0.99714178348369
```

26.2.4 Inverse cosine, input in degrees, $\mathrm{acosd}(x)$

`math53.acosd(x)`

where `ctx` is `math53`, `mathc53`, `ctxcpp`, `ctxflint`.

Returns the inverse cosine of x, $\mathrm{acos}(x)$. See also Wikipedia [1260], MathWorld [868], NIST [487], Ehrhardt [297] (4.2.3), Flint [743], Flint [733], Mpmath [568].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.acosd(-0.333)
109.45096478286685
>>> math53.acosd(0.0)
90.0
>>> math53.acosd(0.333)
70.54903521713317
```

26.2.5 Tangent, with x in degrees, $\mathrm{tand}(x)$

math53.**tand**(x)

Returns the tangent of x, with x in degrees, $\mathrm{tand}tan(x)$. See also Wikipedia [1247], MathWorld [875], NIST [492].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.tand(-4.333)
-0.07576961773759515
>>> math53.tand(0.0)
0.0
>>> math53.tand(4.333)
0.07576961773759515
```

26.2.6 Inverse tangent, input in degrees, $\mathrm{atand}(x)$

math53.**atand**(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the inverse tangent of x, $\mathrm{atan}(x)$. See also Wikipedia [1260], MathWorld [872], NIST [487], Ehrhardt [297] (4.2.15), Flint [743], Flint [733], Mpmath [578].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.atand(-4.333)
-77.00441747834749
>>> math53.atand(0.0)
0.0
>>> math53.atand(4.333)
77.00441747834749
```

26.2.7 Cotangent, with x in degrees, $\mathrm{cotd}(x)$

math53.**cotd**(x)

Returns the cotangent of x, with x in degrees, $\mathrm{cotd}(x)$. See also Wikipedia [1247], MathWorld [860], NIST [492].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.cotd(-4.333)
-13.197902138865127
>>> math53.cotd(0.0)
inf
>>> math53.cotd(4.333)
13.197902138865127
```

26.2.8 Inverse cotangent, input in degrees, $\mathrm{acotd}(x)$

math53.**acotd**(x)

Returns the inverse cotangent of x, $\mathrm{acot}(x)$. See also Wikipedia [1260], MathWorld [869], NIST [487], Ehrhardt [297] (4.2.5).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.acotd(-4.333)
-12.995582521652512
>>> math53.acotd(0.0)
90.0
>>> math53.acotd(4.333)
12.995582521652512
```

26.2.9 Auxiliary function, $\mathrm{acos}(1-x)$

math53.**acos1m**(x)

Returns $\mathrm{acos}(1-x)$, $0 \le x \le 2$, accurate also for x near 0.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.acos1m(-0.333)
nan
>>> math53.acos1m(0.0)
0.0
>>> math53.acos1m(0.333)
0.8406213674789956
```

26.2.10 Haversine function $\mathrm{hav}(x) = (1-\cos(x))/2$

math53.**hav**(x)

Returns the haversine function $\mathrm{hav}(x) = (1-\cos(x))/2 = \sin^2(x/2)$. See also MathWorld [983], Wikipedia [1345].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.hav(-4.333)
0.6851764839496324
>>> math53.hav(0.0)
0.0
>>> math53.hav(4.333)
0.6851764839496324
```

26.2.11 Inverse haversine function $\mathrm{archav}(x) = \mathrm{acos}(1-2x)$

math53.**archav**(z)

Returns the inverse haversine function $\mathrm{archav}(x) = \mathrm{acos}(1-2x) = 2\mathrm{asin}(\sqrt{x})$, $0 \le x \le 1$. See also MathWorld [987], Wikipedia [1347].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.archav(-0.333)
nan
>>> math53.archav(0.0)
0.0
>>> math53.archav(0.333)
1.2302522220901777
```

26.2.12 Versine function $\mathrm{vers}(x) = 1 - \cos(x)$

`math53.vers`(x)

Returns the versine function $\mathrm{vers}(x) = 1 - \cos(x)$.

See also: https://en.wikipedia.org/wiki/Versine#Haversine

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.vers(-4.333)
1.3703529678992648
>>> math53.vers(0.0)
0.0
>>> math53.vers(4.333)
1.3703529678992648
```

26.2.13 Coversine, $\mathrm{covers}(x) = 1 - \sin(x)$

`math53.covers`(x)

Returns the $\mathrm{covers}(x) = 1 - \sin(x)$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.covers(-4.333)
0.07110889811119075
>>> math53.covers(0.0)
1.0
>>> math53.covers(4.333)
1.928891101888809
```

26.2.14 Versint function $\mathrm{versint}(x) = x - \sin(x)$

`math53.versint`(x)

Returns $\mathrm{versint}(x) = \int_0^x \mathrm{vers}(t)\mathrm{d}t = x - \sin(x)$, accurate also near 0.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.versint(-4.333)
-5.26189110188881
>>> math53.versint(0.0)
0.0
>>> math53.versint(4.333)
5.26189110188881
```

26.2.15 Integral of cos powers, $\mathrm{cosint}(n, x)$

`math53.cosint`(n, x)

Returns $\mathrm{IC}_n(x) = \int_0^x \cos^n(t)\,\mathrm{d}t$, the integral of the nth cos power, for $n \geq 0$.

See also Ehrhardt [297] (3.10.13).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.cosint(2, 4.333)
2.33850878821987
>>> math53.cosint(3, 0.0)
0.0
>>> math53.cosint(4, -4.333)
-1.7656780674583596
```

26.2.16 Integral of sin powers, $\mathrm{sinint}(n, x)$

`math53.sinint`(*n*, *x*)

Returns sinint(n,x) = integral(sin(t)^n, t=0..x), n >= 0

Returns $\mathrm{IS}_n(x) = \int_0^x \sin^n(t)\,\mathrm{d}t$, the integral of the nth sin power, for $n \geq 0$.

See also Ehrhardt [297] (3.10.14).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.sinint(2, 4.333)
1.9944912117801308
>>> math53.sinint(3, 0.0)
0.0
>>> math53.sinint(4, -4.333)
-1.421660491018621
```

26.3 Hyperbolic functions

26.3.1 Cardinal hyperbolic sine, $\mathrm{sinhc}(x) = \sinh(x)/x$

math53.**sinhc**(x)

Returns $\mathrm{sinhc}(x) = \sinh(x)/x$, accurate also for x near 0.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.sinhc(-4.333)
8.788291416587565
>>> math53.sinhc(0.0)
1.0
>>> math53.sinhc(4.333)
8.788291416587565
```

26.3.2 Auxiliary function, $\mathrm{sinhmx}(x) = \sinh(x) - x$

math53.**sinhmx**(x)

Returns sinh(x)-x, accurate also for x near 0.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.sinhmx(-4.333)
-33.746666708073924
>>> math53.sinhmx(0.0)
0.0
>>> math53.sinhmx(4.333)
33.746666708073924
```

26.3.3 Auxiliary function, $\mathrm{coshm1}(x) = \cosh(x) - 1$

math53.**coshm1**(x)

Returns $\cosh(x) - 1$, accurate also for x near 0.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.coshm1(-4.333)
37.09279481211629
>>> math53.coshm1(0.0)
0.0
>>> math53.coshm1(4.333)
37.09279481211629
```

26.3.4 Auxiliary function $\mathrm{acosh}(1 + x)$

math53.**acosh1p**(z)

Returns $\mathrm{acosh}(1 + x), x \geq 0$, accurate also for x near 0.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.acosh1p(-4.333)
nan
>>> math53.acosh1p(0.0)
0.0
```

(continues on next page)

(continued from previous page)

```
>>> math53.acosh1p(4.333)
2.3581527329799776
```

26.3.5 Auxiliary function $\log(\cosh(x))$

`math53.logcosh`(x)

Returns ln(cosh(x)), accurate for x ~ 0 and without overflow for large x

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logcosh(-4.333)
3.640025151705744
>>> math53.logcosh(0.0)
0.0
>>> math53.logcosh(4.333)
3.640025151705744
```

26.3.6 Auxiliary function $\log(\sinh(x))$

`math53.logsinh`(x)

Returns ln(sinh(x)), x > 0, accurate for x ~ 0 and without overflow for large x

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logsinh(-4.333)
nan
>>> math53.logsinh(0.0)
-inf
>>> math53.logsinh(4.333)
3.639680457470837
```

26.3.7 Gudermannian function $\mathrm{gd}(x) = \mathrm{asin}(\tanh(x))$

`math53.gd`(x)

Returns the Gudermannian function $\mathrm{gd}(x) = \mathrm{asin}(\tanh(x))$. See also Wikipedia [1344], MathWorld [982].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.gd(-4.333)
-1.5445416269481194
>>> math53.gd(0.0)
0.0
>>> math53.gd(4.333)
1.5445416269481194
```

26.3.8 Inverse Gudermannian function $\mathrm{arcgd}(x) = \mathrm{atanh}(\sin(x))$

`math53.arcgd`(z)

Returns the inverse Gudermannian function $\mathrm{arcgd}(x) = \mathrm{atanh}(\sin(x)), |x| < \pi/2$.

See also Wikipedia [1346], MathWorld [986].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.arcgd(-0.733)
-0.8091157114590968
>>> math53.arcgd(0.0)
0.0
>>> math53.arcgd(0.733)
0.8091157114590968
```

26.3.9 Langevin function, $L(x)$

`math53.langevinl(x)`

Returns the Langevin function $L(x)$, defined as $L(x) = \coth(x) - 1/x$ for $x \neq 0$, and $L(0) = 0$ for $x = 0$.

See also Ehrhardt [297] (3.10.16).

https://en.wikipedia.org/wiki/Brillouin_and_Langevin_functions

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.langevinl(-0.733)
-0.23600650435321588
>>> math53.langevinl(0.0)
0.0
>>> math53.langevinl(0.733)
0.23600650435321588
```

26.3.10 Inverse Langevin function, $L^{-1}(x)$

`math53.langevinlinv(x)`

Returns the functional inverse L^{-1} of the Langevin function, i.e. $L(L^{-1}(x)) = x$, $|x| < 1$.

See also Ehrhardt [297] (3.10.17).

https://en.wikipedia.org/wiki/Brillouin_and_Langevin_functions

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.langevinlinv(-0.733)
-3.7292029668920454
>>> math53.langevinlinv(0.0)
0.0
>>> math53.langevinlinv(0.733)
3.7292029668920454
```

26.3.11 Solutions of Kepler's equation, $\mathrm{kepler}(M, e)$

`math53.kepler(M, e)`

Returns the solutions (eccentric anomaly x) of Kepler's equation from the mean anomaly M and the eccentricity e, more precisely the solutions x of

$$M = \begin{cases} x - e\sin(x), & e < 1, \\ x + x^3/3, & e = 1 \text{ (Barker's equation)}, \\ e\sinh(x) - x, & e > 1. \end{cases}$$

See also Ehrhardt [297] (3.10.23).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.kepler(1.2, 0.333)
1.5327591321519451
>>> math53.kepler(2.4, 0.333)
2.577919997073644
>>> math53.kepler(4.8, 4.333)
1.1179306170995853
```

26.4 Gamma functions

26.4.1 Sign of the gamma function

math53.**signgamma**(x)

Returns the sign of $\Gamma(x)$, which is $+1$ if $x > 0$ or if $\lfloor x \rfloor$ is even, -1 otherwise, and meaningless for 0 or negative integers.

See also Wikipedia [1258], MathWorld [884], NIST [22], BoostMath [101], Ehrhardt [297] (3.5.1.9).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.signgamma(-4.333)
-1.0
>>> math53.signgamma(0.0)
1.0
>>> math53.signgamma(4.333)
1.0
```

26.4.2 Logarithm and sign of the gamma function

math53.**lgamma_s**(x, s)

Returns (as a tuple) the logarithm and sign of the gamma function.

See also Wikipedia [1258], MathWorld [884], NIST [22], BoostMath [101], Ehrhardt [297] (3.5.1.10).

An example in Python

```
>>> from xlcalcnet import math53
>>> print(math53.lgamma_s(-4.333))
(-2.40239615347222, -1)
>>> print(math53.lgamma_s(0.0))
(NaN, 1)
>>> print(math53.lgamma_s(4.333))
(2.22531226395457, 1)
```

26.4.3 Logarithm of $\Gamma(1 + x)$

math53.**lgamma1p**(x)

Returns $\log|\Gamma(1 + x)|$ with increased accuracy for x near 0.

See also Wikipedia [1269], MathWorld [892], BoostMath [102], Ehrhardt [297] (3.5.1.7).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.lgamma1p(-4.333)
-0.9361360107144484
>>> math53.lgamma1p(0.0)
0.0
>>> math53.lgamma1p(4.333)
3.691572406712339
```

26.4.4 Logarithm of factorials: $\log(x!)$

math53.**logfactorial**(n)

Returns $\log(x!) = \log(\Gamma(x + 1))$. See also Wikipedia [1256], MathWorld [882], BoostMath [99], Ehrhardt [297] (3.5.4.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logfactorial(2)
0.6931471805599453
>>> math53.logfactorial(5)
4.787491742782046
>>> math53.logfactorial(15)
27.89927138384089
```

26.4.5 Logarithm of the binomial coefficient

`math53.logbinomial`(n, k)

Returns the logarithm of the binomial coefficient, $\log \binom{n}{k} = \log\left(\frac{n!}{k!(n-k)!}\right)$, for $n \geq k \geq 0$.

See also Wikipedia [1253], MathWorld [881], NIST [484], BoostMath [96], Ehrhardt [297] (3.5.4.5).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logbinomial(13, 3)
5.655991810819852
>>> math53.logbinomial(24, 8)
13.508266389286396
>>> math53.logbinomial(37, 12)
21.33979273614502
```

26.4.6 Log-Beta function

`math53.logbeta`(a, b)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the logarithm of $B(a, b)$

See also Wikipedia [1252], MathWorld [880], NIST [21], BoostMath [95], Ehrhardt [297] (3.5.3.2), Mpmath [585].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logbeta(1.2, -4.333)
-2.573142285269107
>>> math53.logbeta(3.7, 0.1)
2.134462769230688
>>> math53.logbeta(4.8, 4.333)
-5.783706647804463
```

26.4.7 Inverse of the gamma function, $\Gamma^{-1}(y)$

`math53.gamma_inv`(y)

Returns $\Gamma^{-1}(y)$, the functional inverse of the gamma function, i.e. it returns x with $\Gamma(x) = y$, $y \geq 0.8857421875$.

See also Wikipedia [1258], MathWorld [884], NIST [22], BoostMath [101], Ehrhardt [297] (3.5.1.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.gamma_inv(0.9)
1.6492265028621345
>>> math53.gamma_inv(1.5)
2.662766345320147
>>> math53.gamma_inv(4.333)
3.7325782628811806
```

26.4.8 Inverse of the logarithm of the gamma function, $\log \Gamma^{-1}(y)$

`math53.lgamma_inv`(y)

Returns the functional inverse of $\log \Gamma(x)$, i.e. it returns $x = \log \Gamma^{-1}(y)$ with $\log \Gamma(x) = y$ for $y \geq -0.12142 > y_m$ (the minimum of $\log \Gamma(x)$ for positive arguments). The result is greater than $x_m = 1.46163\ldots$ (the positive zero of the ψ function).

See also Wikipedia [1269], MathWorld [892], BoostMath [102], Ehrhardt [297] (3.5.1.6).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.lgamma_inv(0.9)
3.214573262668465
>>> math53.lgamma_inv(1.5)
3.7611332794709584
>>> math53.lgamma_inv(4.333)
5.729662140740046
```

26.4.9 Temme's regulated gamma function, $\Gamma^*(x)$

`math53.gammastar`(x)

Returns Temme's $\Gamma^*(x)$, defined by $\Gamma(x) = \sqrt{2\pi} e^{-x} x^{x-1/2} \Gamma^*(x)$.

See also Wikipedia [1258], MathWorld [884], NIST [22], BoostMath [101], Ehrhardt [297] (3.5.1.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.gammastar(0.9)
1.093716919051002
>>> math53.gammastar(1.5)
1.05634424426856
>>> math53.gammastar(4.333)
1.0193840818217619
```

26.4.10 Relative Pochhammer symbol, $((a)_x - 1)/x$

`math53.poch1`(a, x)

Returns $\dfrac{(a)_x - 1}{x}$, accurate also for small $|x|$. For $x = 0$ the value $\psi(a)$ is returned.

See also Ehrhardt [297] (3.5.4.7).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.poch1(1.2, -4.333)
-0.04297013038959621
>>> math53.poch1(2.4, 0)
0.6529011697005977
>>> math53.poch1(4.8, 4.333)
693.9682866581799
```

26.4.11 Catalan function $C(x)$

`math53.catalan_c`(x)

where `ctx` is `math53`, `ctxcpp`, `ctxboost` or `ctxflint`.

Returns the Catalan function $C(x) = \dfrac{1}{x+1}\binom{2x}{x} = \dfrac{\Gamma(2x+1)}{(x+1)\Gamma(x+1)^2}$.

See also: MathWorld [971], Wikipedia [1338], Ehrhardt [297] (3.10.5).

```
>>> from xlcalcnet import math53
>>> math53.catalan_c(0.9)
0.9539132521406414
>>> math53.catalan_c(1.5)
1.3581221810508415
>>> math53.catalan_c(4.333)
20.057821670219337
```

26.4.12 Bessel Lambda function, $\Lambda(\nu, x)$

math53.**bessel_lambda**(ν, x)

Returns the Bessel $\Lambda_\nu(x)$ function, defined for $x, \nu \geq 0$ as

$$\Lambda(\nu, x) = \Gamma(\nu + 1)\frac{J_\nu(x)}{(x/2)^\nu}$$

See also: Ehrhardt [297] (3.1.3.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.bessel_lambda(1.2, 1.333)
0.8116122406406188
>>> math53.bessel_lambda(2.4, 0)
1.0
>>> math53.bessel_lambda(4.8, 4.333)
0.4208160109325211
```

CHAPTER

TWENTYSEVEN

REAL ELLIPTIC FUNCTIONS

27.1 Conversions of parameters of elliptic functions

27.1.1 Elliptic nome $q(k)$ (DAMath)

`math53.elliptic_nome(`z`)`

Returns the elliptic nome $q(k)$ as a function of the modulus $|k| < 1$:. See also Wikipedia [1360], MathWorld [1000], Ehrhardt [297] (3.2.8).

$$q(k) = \exp\left(-\pi \frac{K'(k)}{K(k)}\right)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.elliptic_nome(-0.9333)
0.12535593275727377
>>> math53.elliptic_nome(0.0)
0.0
>>> math53.elliptic_nome(0.9333)
0.12535593275727377
```

27.1.2 Elliptic modulus $k(q)$ (DAMath)

`math53.elliptic_modulus(`q`)`

Returns the elliptic modulus k. as a function of the nome $|q|$ le 1. See also MathWorld [999], Ehrhardt [297] (3.2.9).The modulus k is often used as argument of elliptic integrals and Jacobi elliptic functions, the nome q is used with Jacobi theta functions. $k(q)$ is explicitly given by

$$k(q) = \frac{\theta_2^2(q)}{\theta_3^2(q)}.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.elliptic_modulus(0.0)
0.0
>>> math53.elliptic_modulus(0.01)
0.3845443947629648
>>> math53.elliptic_modulus(0.6333)
0.9999999966889508
```

27.1.3 Jacobi amplitude, $\mathrm{am}(x, k)$ (DAMath)

math53.**jacobi_amplitude**(x, k)

Returns the Jacobi amplitude function $\mathrm{am}(x, k)$ for a given modulus k. This is the inverse function of Legendre's elliptic function of the first kind: $\mathrm{am}(F(x, k), k) = x$. When $|k| < 1, \mathrm{am}(x, k)$ is a monotone quasi-periodic function NIST [477], 22.16.2: $\mathrm{am}(x + 2K(k), k) = \mathrm{am}(x, k) + \pi$, with the special case $\mathrm{am}(x, 0) = x$. When $|k| > 1, \mathrm{am}(x, k)$ is periodic with period $4K(1/k)/k$, and if $|k| = 1$, then it is equal to the Gudermannian function $\mathrm{am}(x, \pm 1) = \mathrm{gd}(x)$.

See also MathWorld [1005], NIST [477], Ehrhardt [297] (3.2.10).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_amplitude(1.2, -4.333)
-0.21310912002774368
>>> math53.jacobi_amplitude(2.4, 0.0)
2.4
>>> math53.jacobi_amplitude(3.7, 4.333)
-0.024983643980939094
```

27.2 Elliptic integrals

27.2.1 Legendre complete elliptic integral $B(k)$

`math53.elliptic_b(`*k*`)`

Returns the Legendre complete elliptic integral $B(k) = \int_0^{\pi/2} \frac{\cos^2(t)\mathrm{d}t}{\sqrt{1-k^2\sin^2(t)}} = \frac{E(k)-\sqrt{1-k^2}K(k)}{k^2}$. See also Wikipedia [1301], MathWorld [952], NIST [171], BoostMath [137], Ehrhardt [297] (3.2.1.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.elliptic_b(-0.9333)
0.9305395801783577
>>> math53.elliptic_b(0.0)
0.7853981633974483
>>> math53.elliptic_b(0.9333)
0.9305395801783577
```

27.2.2 Legendre complete elliptic integral $D(k)$

`math53.elliptic_d(`*k*`)`

Returns the Legendre complete elliptic integral $D(k) = \int_0^{\pi/2} \frac{\sin^2(t)\mathrm{d}t}{\sqrt{1-k^2\sin^2(t)}} = \frac{K(k)-E(k)}{k^2}$. See also Wikipedia [1301], MathWorld [952], NIST [171], BoostMath [137], Ehrhardt [297] (3.2.1.5).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.elliptic_d(-0.9333)
1.5285669226805516
>>> math53.elliptic_d(0.0)
0.7853981633974483
>>> math53.elliptic_d(0.9333)
1.5285669226805516
```

27.2.3 Legendre incomplete elliptic integral $B(\phi, k)$

`math53.elliptic_b_inc(`*phi*, *k*`)`

Returns the Legendre complete elliptic integral , $B(k) = \int_0^{\phi} \frac{\cos^2(t)\mathrm{d}t}{\sqrt{1-k^2\sin^2(t)}} = \frac{E(\phi,k)-\sqrt{1-k^2}F(\phi,k)}{k^2}$. See also Wikipedia [1301], MathWorld [952], NIST [171], BoostMath [137], Ehrhardt [297] (3.2.1.9).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.elliptic_b_inc(0.4, -0.9333)
0.3880334013629784
>>> math53.elliptic_b_inc(0.7, 0.0)
0.5963624324971151
>>> math53.elliptic_b_inc(1.2, 0.9333)
0.8930544029455502
```

27.2.4 Legendre incomplete elliptic integral $D(\phi, k)$

math53.**elliptic_d_inc**(*phi*, *k*)

Returns the Legendre complete elliptic integral , $D(k) = \int_0^{\phi} \frac{\sin^2(t)\mathrm{d}t}{\sqrt{1-k^2\sin^2(t)}} = \frac{F(\phi,k)-E(\phi,k)}{k^2}$. See also Wikipedia [1301], MathWorld [952], NIST [171], BoostMath [134], Ehrhardt [297] (3.2.1.10).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.elliptic_d_inc(0.4, -0.9333)
0.021551828843784506
>>> math53.elliptic_d_inc(0.7, 0.0)
0.10363756750288494
>>> math53.elliptic_d_inc(1.2, 0.9333)
0.6473333304003238
```

27.2.5 Heuman's Lambda function, $\Lambda_0(\phi, k)$

math53.**heuman_lambda**(*phi*, *k*)

Returns Heuman's Lambda function $\Lambda_0(\phi, k)$. See also MathWorld [958], BoostMath [141], Ehrhardt [297] (3.2.6).

$$\Lambda_0(\phi,k) = \frac{F(\phi,\sqrt{1-k^2})}{K(\sqrt{1-k^2})} + \frac{2}{\pi}K(k)Z(\phi,\sqrt{1-k^2}), \quad |k| \le 1.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.heuman_lambda(0.4, -0.9333)
0.28392272091599396
>>> math53.heuman_lambda(0.7, 0.0)
0.644217687237691
>>> math53.heuman_lambda(1.2, 0.9333)
0.7921071008822748
```

27.2.6 Jacobi Zeta function, $Z(\phi, k)$

math53.**jacobi_zeta**(*phi*, *k*)

Returns the Jacobi Zeta function $Z(\phi, k)$. See also MathWorld [966], BoostMath [152], Ehrhardt [297] (3.2.7).

$$Z(\phi,k) = E(\phi,k) - \frac{E(k)}{K(k)}F(\phi,k), \quad |k| \le 1.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_zeta(0.4, -0.9333)
0.2029927800760033
>>> math53.jacobi_zeta(0.7, 0.0)
0.0
>>> math53.jacobi_zeta(1.2, 0.9333)
0.2701672377293978
```

27.3 Bulirsch elliptic integrals

See Carlson (1994)

Bulirsch elliptic integrals can be calculated by

$$\mathrm{cel1}(k_c) = \mathrm{cel}(k_c, 0, a, 0) = aR_F\left(0, k_c^2, 1\right)$$

$$\mathrm{cel2}(k_c) = \mathrm{cel}(k_c, 0, a, b) = aR_F\left(0, k_c^2, 1\right) + \frac{b}{3}R_J\left(0, k_c^2, 1, 0\right)$$

$$\mathrm{cel}(k_c, p, a, b) = aR_F\left(0, k_c^2, 1\right) + \frac{1}{3}(b - pa)R_J\left(0, k_c^2, 1, p\right)$$

$$\mathrm{el1}(x, k_c) = xR_F\left(1, 1 + k_c^2x^2, 1 + x^2\right)$$

$$\mathrm{el2}(x, k_c, a, b) = axR_F\left(1, 1 + k_c^2x^2, 1 + x^2\right) + (b - a)\frac{x^3}{3}R_D\left(1, 1 + k_c^2x^2, 1 + x^2\right)$$

$$\mathrm{el3}(x, k_c, p) = xR_F\left(1, 1 + k_c^2x^2, 1 + x^2\right) + (1 - p)\frac{x^3}{3}R_J\left(1, 1 + k_c^2x^2, 1 + x^2, 1 + px^2\right)$$

27.3.1 Complete elliptic integral of the 1st kind $\mathrm{cel1}(k_c)$

`math53.cel1`(*kc*)

Returns Bulirsch's complete elliptic integral of the first kind. See also Bulirsch [162], See also Bulirsch [163], NIST [173], Wikipedia [1298], Ehrhardt [297] (3.2.3.1).

$$\mathrm{cel1}(k_c) = \int_0^\infty \frac{\mathrm{d}t}{\sqrt{(1 + t^2)(1 + k_c^2t^2)}}$$

with the complementary modulus $K_c \neq 0$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.cel1(-0.9333)
1.6254736111454573
>>> math53.cel1(0.01)
5.991589340506996
>>> math53.cel1(0.9333)
1.6254736111454573
```

27.3.2 Complete elliptic integral of the 2nd kind $\mathrm{cel2}(k_c, a, b)$

`math53.cel2`(*kc*, *a*, *b*)

Returns Bulirsch's complete elliptic integral of the second kind. See also Bulirsch [162], See also Bulirsch [163], NIST [173], Wikipedia [1298], Ehrhardt [297] (3.2.3.2).

$$\mathrm{cel2}(k_c, a, b) = \int_0^\infty \frac{a + bt^2}{(1 + t^2)\sqrt{(1 + t^2)(1 + k_c^2t^2)}}\mathrm{d}t$$

with the complementary modulus $k_c \neq 0$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.cel2(1.2, -4.333, -1.333)
-4.151578844884948
>>> math53.cel2(3.7, -4.333, 0.0)
-2.078160086136537
>>> math53.cel2(4.8, 4.333, 1.333)
2.082811909491336
```

27.3.3 General complete elliptic integral $\mathrm{cel}(k_c, p, a, b)$

`math53.cel`(*kc*, *p*, *a*, *b*)

Returns Bulirsch's general complete elliptic integral. See also Bulirsch [162], See also Bulirsch [163], NIST [173], Wikipedia [1298], Ehrhardt [297] (3.2.3.3).

$$\mathrm{cel}(k_c, p, a, b) = \int_0^\infty \frac{a + bt^2}{(1 + pt^2)\sqrt{(1 + t^2)(1 + k_c^2 t^2)}} \mathrm{d}t$$

with the complementary modulus $k_c \neq 0$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.cel(1.2, -4.333, -1.333, -1.333)
-0.0708205057509719
>>> math53.cel(3.7, -4.333, 0.0,  -1.333)
0.12075231537983412
>>> math53.cel(4.8, 4.333, 1.333, 1.333)
0.5162685345242685
```

27.3.4 Incomplete elliptic integral of the 1st kind $\mathrm{el1}(x, k_c)$

`math53.el1`(*x*, *kc*)

Returns Bulirsch's incomplete elliptic integral of the first kind. See also Bulirsch [162], See also Bulirsch [163], NIST [173], Wikipedia [1298], Ehrhardt [297] (3.2.3.4).

$$\mathrm{el1}(x, k_c) = \int_0^x \frac{\mathrm{d}t}{\sqrt{(1 + t^2)(1 + k_c^2 t^2)}}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.el1(1.2, -4.333)
0.49012453573756454
>>> math53.el1(3.7, -4.333)
0.6025865718715321
>>> math53.el1(4.8, 4.333)
0.6164652942503261
```

27.3.5 Incomplete elliptic integral of the 2nd kind $\mathrm{el2}(x, k_c, a, b)$

`math53.el2`(*x*, *kc*, *a*, *b*)

Returns Bulirsch's incomplete elliptic integral of the second kind. See also Bulirsch [162], See also Bulirsch [163], NIST [173], Wikipedia [1298], Ehrhardt [297] (3.2.3.5).

$$\mathrm{el2}(x, k_c, a, b) = \int_0^x \frac{a + bt^2}{(1 + t^2)\sqrt{(1 + t^2)(1 + k_c^2 t^2)}} \mathrm{d}t$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.el2(1.2, -4.333, -1.333, -1.333)
-0.6533360061381738
>>> math53.el2(3.7, -4.333, 0.0,  -1.333)
-0.21559574153402697
>>> math53.el2(4.8, 4.333, 1.333, 1.333)
0.8217482372356851
```

27.3.6 Incomplete elliptic integral of the 3rd kind $\mathrm{el3}(x, k_c, p)$

`math53.el3`(x, kc, p)

Returns Bulirsch's incomplete elliptic integral of the third kind. See also Bulirsch [162], See also Bulirsch [163], NIST [173], Wikipedia [1298], Ehrhardt [297] (3.2.3.6).

$$\mathrm{el3}(x, k_c, p) = \int_0^x \frac{1 + t^2}{(1 + pt^2)\sqrt{(1 + t^2)(1 + k_c^2 t^2)}} \mathrm{d}t$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.el3(1.2, -4.333, -1.333)
0.5440817141850122
>>> math53.el3(3.7, -4.333, 0.0)
1.2433057985726927
>>> math53.el3(4.8, 4.333, 1.333)
0.5677080218351377
```

27.4 Maple style elliptic integrals

27.4.1 Complete integral of the 1st kind, $\mathrm{EllipticK}(k)$

math53.**EllipticK**(k)

Returns the complete elliptic integral of the first kind K(k) and the real part if $|k| > 1$. See also Maplesoft [367], Ehrhardt [297] (3.2.4.1).

$$\mathrm{EllipticK}(k) = \int_0^1 \frac{\mathrm{d}t}{\sqrt{(1-t^2)(1-k^2t^2)}}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticK(-0.9333)
2.459106502858909
>>> math53.EllipticK(0.01)
1.570835598912152
>>> math53.EllipticK(0.9333)
2.459106502858909
```

27.4.2 Complete integral of the 1st kind for imaginary modulus, $\mathrm{EllipticKim}(k)$

math53.**EllipticKim**(k)

Returns the complete elliptic integral of the first kind for the imaginary modulus ik with $k \in \mathbb{R}$. See also Maplesoft [367], Ehrhardt [297] (3.2.4.2).

$$\mathrm{EllipticKim}(k) = \int_0^1 \frac{\mathrm{d}t}{\sqrt{(1-t^2)(1+k^2t^2)}}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticKim(-0.9333)
1.334879017781761
>>> math53.EllipticKim(0.01)
1.5707570590955058
>>> math53.EllipticKim(0.9333)
1.334879017781761
```

27.4.3 Complementary complete integral of the 1st kind, $\mathrm{EllipticCK}(k) = K'(k)$

math53.**EllipticCK**(k)

Returns the complementary complete elliptic integral of the first kind with $k \neq 0$. See also Maplesoft [367], Ehrhardt [297] (3.2.4.3), Ehrhardt [297] (4.2.28).

$$\mathrm{EllipticCK}(k) = \mathrm{EllipticK}(k_c) = \int_0^1 \frac{\mathrm{d}t}{\sqrt{(1-t^2)(1-k_c^2t^2)}}$$

Note: the complex version is called Ellck(z) in Amath.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticCK(-0.9333)
1.6254736111454573
>>> math53.EllipticCK(0.01)
5.991589340506996
>>> math53.EllipticCK(0.9333)
1.6254736111454573
```

27.4.4 Complete integral of the 2nd kind, $\mathrm{EllipticEC}(k)$

`math53.EllipticEC(k)`

Returns the complete elliptic integral of the second kind $E(k)$ and the real part if $|k| > 1$. See also Maplesoft [366], Ehrhardt [297] (3.2.4.4).

$$\mathrm{EllipticEC}(k) = \int_0^1 \frac{\sqrt{(1-k^2t^2)}}{\sqrt{(1-t^2)}}\mathrm{d}t$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticEC(-0.9333)
1.127649981567299
>>> math53.EllipticEC(0.01)
1.5707570561503852
>>> math53.EllipticEC(0.9333)
1.127649981567299
```

27.4.5 Complete integral of the 2nd kind for imaginary modulus, $\mathrm{EllipticECim}(k)$

`math53.EllipticECim(k)`

Returns the complete elliptic integral of the second kind for the imaginary modulus ik with $k \in \mathbb{R}$. See also Maplesoft [366], Ehrhardt [297] (3.2.4.5).

$$\mathrm{EllipticECim}(k) = E(ik) = \int_0^1 \frac{\sqrt{(1+k^2t^2)}}{\sqrt{(1-t^2)}}\mathrm{d}t$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticECim(-0.9333)
1.870953551376072
>>> math53.EllipticECim(0.01)
1.5708355959667861
>>> math53.EllipticECim(0.9333)
1.870953551376072
```

27.4.6 Complementary complete integral of the 2nd kind, $\mathrm{EllipticCE}(k)$

`math53.EllipticCE(k)`

Returns the complementary complete elliptic integral of the second kind. See also Maplesoft [366], Ehrhardt [297] (3.2.4.6).

$$\mathrm{EllipticEC}(k) = \mathrm{EllipticEC}(k_c) = \int_0^1 \frac{\sqrt{(1-k_c^2t^2)}}{\sqrt{(1-t^2)}}\mathrm{d}t$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticCE(-0.9333)
1.5188621404561577
>>> math53.EllipticCE(0.01)
1.0002745824306631
>>> math53.EllipticCE(0.9333)
1.5188621404561577
```

27.4.7 Complete integral of the 3rd kind, $\mathrm{EllipticPiC}(\nu, k)$

`math53.EllipticPiC(nu, k)`

Returns the complete elliptic integral of the third kind with $|k| \neq 1, \nu \neq 1$ (or its real part if $|k| > 1$). See also Maplesoft [368], Ehrhardt [297] (3.2.4.7).

$$\mathrm{EllipticPiC}(\nu, k) = \int_0^1 \frac{\mathrm{d}t}{(1-\nu t^2)\sqrt{(1-t^2)(1-k^2t^2)}}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticPiC(-0.9333, 0.01)
1.129742159472719
>>> math53.EllipticPiC(0.01, -0.9333)
2.4745180633508044
>>> math53.EllipticPiC(0.9333, 0.01)
6.0823893714986585
```

27.4.8 Complete integral of the 3rd kind for imaginary modulus, $\mathrm{EllipticPiCim}(\nu, k)$

`math53.EllipticPiCim(nu, k)`

Returns the complementary complete elliptic integral of the third kind with $\nu \neq 1$. See also Maplesoft [366], Ehrhardt [297] (3.2.4.8).

$$\mathrm{EllipticPiCim}(\nu, k) = \mathrm{EllipticPiC}(\nu, k) = \int_0^1 \frac{\mathrm{d}t}{(1-\nu t^2)\sqrt{(1-t^2)(1+k^2t^2)}}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticPiCim(-0.9333, 0.01)
1.1296948994491276
>>> math53.EllipticPiCim(0.01, -0.9333)
1.3410787572966438
>>> math53.EllipticPiCim(0.9333, 0.01)
6.08190599516054
```

27.4.9 Complementary complete integral of the 3rd kind, $\mathrm{EllipticCPi}(\nu, k)$

`math53.EllipticCPi(nu, x)`

Returns the complementary complete elliptic integral of the third kind with $|k| \neq 0, \nu \neq 1$. See also Maplesoft [366], Ehrhardt [297] (3.2.4.9).

$$\mathrm{EllipticCPi}(\nu, k) = \mathrm{EllipticPiC}(\nu, k_c) = \int_0^1 \frac{\mathrm{d}t}{(1-\nu t^2)\sqrt{(1-t^2)(1-k_c^2t^2)}}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticCPi(-0.9333, 0.01)
3.4829363034929215
>>> math53.EllipticCPi(0.01, -0.9333)
1.6338040919783785
>>> math53.EllipticCPi(0.9333, 0.01)
60.46495296527363
```

27.4.10 Incomplete integral of the 1st kind, $\mathrm{EllipticF}(z, k)$

`math53.EllipticF`(z, k)

Returns the incomplete elliptic integral of the first kind with $|z| \le 1, |kz| \le 1$. See also Maplesoft [367], Ehrhardt [297] (3.2.4.10).

$$\mathrm{EllipticF}(z, k) = \int_0^z \frac{\mathrm{d}t}{\sqrt{(1-t^2)(1-k^2t^2)}}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticF(-0.9333, 0.01)
-1.2035171794595962
>>> math53.EllipticF(0.01, -0.9333)
0.010000311859028539
>>> math53.EllipticF(0.9333, 0.01)
1.2035171794595962
```

27.4.11 Incomplete integral of the 2nd kind, $\mathrm{EllipticE}(z, k)$

`math53.EllipticE`(z, k)

Returns the incomplete elliptic integral of the second kind with $|z| \le 1, |kz| \le 1$. See also Maplesoft [366], Ehrhardt [297] (3.2.4.11).

$$\mathrm{EllipticE}(z, k) = \int_0^z \frac{\sqrt{(1-k^2t^2)}}{\sqrt{(1-t^2)}}\mathrm{d}t$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticE(-0.9333, 0.01)
-1.2034737607138424
>>> math53.EllipticE(0.01, -0.9333)
0.01000002149309969
>>> math53.EllipticE(0.9333, 0.01)
1.2034737607138424
```

27.4.12 Incomplete integral of the 3rd kind, $\mathrm{EllipticPi}(z, \nu, k)$

`math53.EllipticPi`(z, *nu*, k)

Returns the incomplete elliptic integral of the third kind with $|z| \le 1, |kz| \le 1$. See also Maplesoft [368], Ehrhardt [297] (3.2.4.12).

$$\mathrm{EllipticPi}(z, \nu, k) = \int_0^z \frac{\mathrm{d}t}{(1-\nu t^2)\sqrt{(1-t^2)(1-k_c^2t^2)}}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EllipticPi(-0.9333, 0.01, 0.9333)
-1.5540650009536827
>>> math53.EllipticPi(0.01, -0.9333, 0.9333)
0.010000000758985845
>>> math53.EllipticPi(0.9333, 0.01, -0.9333)
1.5540650009536827
```

27.5 Jacobi theta functions at $x = 0$ for $0 \leq q < 1$

27.5.1 Jacobi theta function $\theta_1'(q)$

math53.**jacobi_theta1p**(q)

Returns the Jacobi theta function $\theta_1'(q) = \partial\theta_1(x, q)/\partial x$ at $x = 0$ for $0 \leq q < 1$. See also Wikipedia [1318], MathWorld [965], MathWorld [278], NIST [478], Ehrhardt [297] (3.2.14.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_theta1p(0.01)
0.632265795377228
>>> math53.jacobi_theta1p(0.5)
0.5489785325603408
>>> math53.jacobi_theta1p(0.9333)
1.8387255175389385e-13
```

27.5.2 Jacobi theta function $\theta_2(q)$

math53.**jacobi_theta02**(q)

Returns the Jacobi theta function $\theta_2(q) = \theta_2(0, q)$ for $0 \leq q < 1$. See also Wikipedia [1318], MathWorld [965], NIST [478], Ehrhardt [297] (3.2.14.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_theta02(0.01)
0.6325187775875117
>>> math53.jacobi_theta02(0.5)
2.128931250513028
>>> math53.jacobi_theta02(0.9333)
6.74621917080838
```

27.5.3 Jacobi theta function $\theta_3(q)$

math53.**jacobi_theta03**(q)

Returns the Jacobi theta function $\theta_3(q) = \theta_3(0, q)$ for $0 \leq q < 1$. See also Wikipedia [1318], MathWorld [965], NIST [478], Ehrhardt [297] (3.2.14.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_theta03(0.01)
1.0200000200000001
>>> math53.jacobi_theta03(0.5)
2.128936827211877
>>> math53.jacobi_theta03(0.9333)
6.74621917080838
```

27.5.4 Jacobi theta function $\theta_4(q)$

math53.**jacobi_theta04**(q)

Returns the Jacobi theta function $\theta_4(q) = \theta_4(0, q)$ for $-1 < q < 1$. See also Wikipedia [1318], MathWorld [965], NIST [478], Ehrhardt [297] (3.2.14.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_theta04(0.01)
0.98000002
>>> math53.jacobi_theta04(0.5)
0.12112420800258053
>>> math53.jacobi_theta04(0.9333)
4.040136238779326e-15
```

27.6 Inverse Jacobi elliptic functions

The inverse Jacobi elliptic functions can be defined like the inverse trigonometric functions: e.g. if $\mathrm{sn}(y,k)=x$ then $y=\mathrm{arcsn}(x,k)$; they are multivalued and their principal values are returned. The functions can be represented as elliptic integrals [30, §22.15(ii)] and they are computed with the incomplete elliptic integral $F(.,k)$ using the table from Abramowitz and Stegun [1, p.596].

27.6.1 Inverse Jacobi elliptic function $\mathrm{arcsn}(x,k)$

`math53.jacobi_arcsn`(x, k)

Returns the inverse Jacobi elliptic function $\mathrm{arcsn}(x,k)$ for $|x|\leq 1$ and $|kx|\leq 1$. See also NIST [476], Ehrhardt [297] (3.2.12.1).

$$\mathrm{arcsn}(x,k)=F(\arcsin(x),k).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arcsn(0.01, 0.5)
0.010000208342552622
>>> math53.jacobi_arcsn(0.5, 0.9333)
0.5455929205316519
>>> math53.jacobi_arcsn(0.9333, 0.5)
1.2646631607926928
```

27.6.2 Inverse Jacobi elliptic function $\mathrm{arccn}(x,k)$

`math53.jacobi_arccn`(x, k)

Returns the inverse Jacobi elliptic function $\mathrm{arccn}(x,k)$ for $|x|<1$ if $k\leq 1$, and $x^2>1-1/k^2$ if $k>1$. See also NIST [476], Ehrhardt [297] (3.2.12.2).

$$\mathrm{arccn}(x,k)=F(\arccos(x),k).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arccn(0.01, 0.5)
1.6742032211210454
>>> math53.jacobi_arccn(0.5, 0.9333)
1.2569383151950466
>>> math53.jacobi_arccn(0.9333, 0.5)
0.3693405114569477
```

27.6.3 Inverse Jacobi elliptic function $\mathrm{arcdn}(x,k)$

`math53.jacobi_arcdn`(x, k)

Returns the inverse Jacobi elliptic function $\mathrm{arcdn}(x,k)$ for $0\leq x\leq 1$ and $k^2+x^2>1$ if $|k|<1$; and $|x|\leq 1$ if $|k|>1$. See also NIST [476], Ehrhardt [297] (3.2.12.3).

$$\mathrm{arcdn}(x,k)=F\left(\arcsin\left(\sqrt{\frac{1-x^2}{k^2}}\right),k\right).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arcdn(0.81, 0.85)
0.818383723341855
```

(continues on next page)

(continued from previous page)

```
>>> math53.jacobi_arcdn(0.85, 0.9333)
0.6333740262678361
>>> math53.jacobi_arcdn(0.9333, 0.85)
0.44639702745946724
```

27.6.4 Inverse Jacobi elliptic function $\operatorname{arccd}(x, k)$

`math53.jacobi_arccd`(x, k)

Returns the inverse Jacobi elliptic function $\operatorname{arccd}(x, k)$ for $|x| \leq 1$ if $|k| < 1$; and $|x| \geq 1$ if $|k| > 1$. See also NIST [476], Ehrhardt [297] (3.2.12.4).

$$\operatorname{arccd}(x, k) = F\left(\arcsin\left(\sqrt{\frac{1 - x^2}{1 - k^2 x^2}}\right), k\right).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arccd(0.01, 0.5)
1.6757501464700435
>>> math53.jacobi_arccd(0.5, 0.9333)
1.9135135823272573
>>> math53.jacobi_arccd(0.9333, 0.5)
0.4210871940199031
```

27.6.5 Inverse Jacobi elliptic function $\operatorname{arcsd}(x, k)$

`math53.jacobi_arcsd`(x, k)

Returns the inverse Jacobi elliptic function $\operatorname{arcsd}(x, k)$ for $x \in \mathbb{R}$ if $|k| \geq 1$ and $|x| < 1/\sqrt{1 - k^2}$ otherwise. See also NIST [476], Ehrhardt [297] (3.2.12.5).

$$\operatorname{arcsd}(x, k) = F\left(\arcsin\left(\sqrt{\frac{x^2}{1 + k^2 x^2}}\right), k\right).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arcsd(1.01, 1.5)
0.01000008333708349
>>> math53.jacobi_arcsd(1.5, 1.9333)
0.48599631639488927
>>> math53.jacobi_arcsd(1.9333, 1.5)
1.0461374961953611
```

27.6.6 Inverse Jacobi elliptic function $\operatorname{arcnd}(x, k)$

`math53.jacobi_arcnd`(x, k)

Returns the inverse Jacobi elliptic function $\operatorname{arcnd}(x, k)$ for $x \geq 1$ and $x^2 \leq k^2/(1 - k^2)$ if $|k| < 1$. See also NIST [476], Ehrhardt [297] (3.2.12.6).

$$\operatorname{arcnd}(x, k) = F\left(\arcsin\left(\sqrt{\frac{x^2 - 1}{k^2 x^2}}\right), k\right).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arcnd(1.01, 1.5)
0.09402852893196235
>>> math53.jacobi_arcnd(1.5, 1.9333)
0.44786848710680505
>>> math53.jacobi_arcnd(1.9333, 1.5)
0.7366908030186489
```

27.6.7 Inverse Jacobi elliptic function $\operatorname{arcdc}(x, k)$

`math53.jacobi_arcdc`(x, k)

Returns the inverse Jacobi elliptic function $\operatorname{arcdc}(x, k)$ for $|x| \geq 1$ if $|k| < 1$; and $|x| \leq 1$ if $|k| > 1$. See also NIST [476], Ehrhardt [297] (3.2.12.7).

$$\operatorname{arcdc}(x, k) = F\left(\arcsin\left(\sqrt{\frac{1 - x^2 - 1}{k^2 - x^2}}\right), k\right).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arcdc(0.01, 1.5)
1.1997781698230954
>>> math53.jacobi_arcdc(0.5, 1.9333)
0.602871393168387
>>> math53.jacobi_arcdc(0.9333, 1.5)
0.3230254324880706
```

27.6.8 Inverse Jacobi elliptic function $\operatorname{arcnc}(x, k)$

`math53.jacobi_arcnc`(x, k)

Returns the inverse Jacobi elliptic function $\operatorname{arcnc}(x, k)$ for $x \geq 1$, and $x^2 \leq k^2/(k^2 - 1)$ for $|k| > 1$. See also NIST [476], Ehrhardt [297] (3.2.12.8).

$$\operatorname{arcnc}(x, k) = F(\operatorname{arcsec}(x), k).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arcnc(1.01, 0.5)
0.1409520642226382
>>> math53.jacobi_arcnc(1.5, 0.9333)
0.9408474060083183
>>> math53.jacobi_arcnc(1.9333, 0.5)
1.0673523684807162
```

27.6.9 Inverse Jacobi elliptic function $\operatorname{arcsc}(x, k)$

`math53.jacobi_arcsc`(x, k)

Returns the inverse Jacobi elliptic function $\operatorname{arcsc}(x, k)$ for $x \in \mathbb{R}$ if $|k| \leq 1$ and $|x| \leq 1/\sqrt{k^2 - 1}$ if $|k| > 1$. See also NIST [476], Ehrhardt [297] (3.2.12.9).

$$\operatorname{arcsc}(x, k) = F(\arctan(x), k).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arcsc(0.01, 0.5)
0.009999708348801099
>>> math53.jacobi_arcsc(0.5, 0.9333)
0.4787388167314662
>>> math53.jacobi_arcsc(0.9333, 0.5)
0.7675883759164223
```

27.6.10 Inverse Jacobi elliptic function $\operatorname{arcns}(x, k)$

`math53.jacobi_arcns`(x, k)

Returns the inverse Jacobi elliptic function $\operatorname{arcns}(x, k)$ for $|x| \geq 1$ if $|k| < 1$ and $|x| \geq |k|$ if $|k| > 1$. See also NIST [476], Ehrhardt [297] (3.2.12.10).

$$\operatorname{arcns}(x, k) = F(\operatorname{arccsc}(x), k).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arcns(1.01, 0.5)
1.523305059639121
>>> math53.jacobi_arcns(1.5, 0.9333)
0.7925159210200553
>>> math53.jacobi_arcns(1.9333, 0.5)
0.5501460517169048
```

27.6.11 Inverse Jacobi elliptic function $\operatorname{arcds}(x, k)$

`math53.jacobi_arcds`(x, k)

Returns the inverse Jacobi elliptic function $\operatorname{arcds}(x, k)$ for $x \in \mathbb{R}$ if $|k| > 1$ and $|x| \geq \sqrt{1 - k^2}$ if $|k| < 1$. See also NIST [476], Ehrhardt [297] (3.2.12.11).

$$\operatorname{arcds}(x, k) = F\left(\arcsin\left(\sqrt{\frac{1}{k^2 + x^2}}\right), k\right).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arcds(0.01, 1.5)
1.2004822727201372
>>> math53.jacobi_arcds(0.5, 1.9333)
0.7245630486544666
>>> math53.jacobi_arcds(0.9333, 1.5)
0.7261996704910932
```

27.6.12 Inverse Jacobi elliptic function $\operatorname{arccs}(x, k)$

`math53.jacobi_arccs`(x, k)

Returns the inverse Jacobi elliptic function $\operatorname{arccs}(x, k)$ for $x \in \mathbb{R}$ if $|k| < 1$ and $|x| \geq \sqrt{k^2 - 1}$ if $|k| > 1$. See also NIST [476], Ehrhardt [297] (3.2.12.12).

$$\operatorname{arccs}(x, k) = F(\operatorname{arccot}(x), k).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.jacobi_arccs(0.01, 0.5)
1.6742037984472615
>>> math53.jacobi_arccs(0.5, 0.9333)
1.3622156081449894
>>> math53.jacobi_arccs(0.9333, 0.5)
0.8413253980268582
```

27.7 Lemniscate functions

27.7.1 Lemniscate sine function, $\mathrm{sinlemn}(x)$

`math53.sinlemn(`x`)`

Returns the lemniscate sine function $\mathrm{sinlemn}(x) = \frac{\sqrt{2}}{2}\mathrm{sd}\left(x\sqrt{2}, \frac{\sqrt{2}}{2}\right)$, where $\mathrm{sd}(\cdot)$ is a Jacobi elliptic function.

See also: Wikipedia [1329], MathWorld [968], Ehrhardt [297] (3.2.16.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.sinlemn(0.01)
0.00999999999
>>> math53.sinlemn(0.5)
0.496891190419312
>>> math53.sinlemn(0.9333)
0.8666952283443882
```

27.7.2 Lemniscate cosine function, $\mathrm{coslemn}(x)$

`math53.coslemn(`x`)`

Returns the lemniscate cosine function $\mathrm{coslemn}(x) = \mathrm{cn}\left(x\sqrt{2}, \frac{\sqrt{2}}{2}\right)$, where $\mathrm{cn}(\cdot)$ is a Jacobi elliptic function.

See also: Wikipedia [1329], MathWorld [968], Ehrhardt [297] (3.2.16.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.coslemn(0.01)
0.9999000049996999
>>> math53.coslemn(0.5)
0.7771594206519663
>>> math53.coslemn(0.9333)
0.3769611253836269
```

27.7.3 Inverse lemniscate sine function, $\mathrm{arcsl}(x)$

`math53.arcsl(`x`)`

Returns the inverse lemniscate sine function $\mathrm{arcsl}(x) = \frac{1}{\sqrt{2}}\mathrm{arcsd}\left(x\sqrt{2}, \frac{\sqrt{2}}{2}\right)$, $|x| \leq 1$, where $\mathrm{arcsd}(\cdot)$ is an inverse Jacobi elliptic function.

See also: Wikipedia [1329], MathWorld [968], Ehrhardt [297] (3.2.16.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.arcsl(0.01)
1.3010287771360596
>>> math53.arcsl(0.5)
0.8078193339687287
>>> math53.arcsl(0.9333)
0.2626495240692557
```

27.7.4 Inverse lemniscate cosine function, $\operatorname{arccl}(x)$

`math53.arccl(x)`

Returns the inverse lemniscate cosine function $\operatorname{arccl}(x) = \frac{1}{\sqrt{2}}\operatorname{arccn}\left(x\sqrt{2}, \frac{\sqrt{2}}{2}\right), |x| \leq 1$, where $\operatorname{arccn}(\cdot)$ is an inverse Jacobi elliptic function.

See also: Wikipedia [1329], MathWorld [968], Ehrhardt [297] (3.2.16.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.arccl(0.01)
0.010000000010000001
>>> math53.arccl(0.5)
0.5032094431773307
>>> math53.arccl(0.9333)
1.0483792530768037
```

27.8 Neville theta functions

27.8.1 Neville $\theta_c(x, k)$

math53.**neville_theta_c**(x, k)

Returns the Neville theta function $\theta_c(x,k) = \dfrac{\theta_2(v, q(k))}{\theta_2(0, q(k))}$, $v = \dfrac{\pi x}{2K(k)}$, $|k| \leq 1$, and $q(k)$ is the elliptic nome.

See also: Wikipedia [1331], MathWorld [279], Ehrhardt [297] (3.2.15.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.neville_theta_c(0.01, 0.5)
0.999956474832081
>>> math53.neville_theta_c(0.5, 0.9333)
0.9436331810661398
>>> math53.neville_theta_c(0.9333, 0.5)
0.6446023742055126
```

27.8.2 Neville $\theta_d(x, k)$

math53.**neville_theta_d**(x, k)

Returns the Neville theta function $\theta_d(x,k) = \dfrac{\theta_3(v, q(k))}{\theta_3(0, q(k))}$, $v = \dfrac{\pi x}{2K(k)}$, $|k| \leq 1$, and $q(k)$ is the elliptic nome.

See also: Wikipedia [1331], MathWorld [280], Ehrhardt [297] (3.2.15.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.neville_theta_d(0.01, 0.5)
0.9999939746842494
>>> math53.neville_theta_d(0.5, 0.9333)
0.9602114827440184
>>> math53.neville_theta_d(0.9333, 0.5)
0.9594825353083286
```

27.8.3 Neville $\theta_n(x, k)$

math53.**neville_theta_n**(x, k)

Returns the Neville theta function $\theta_n(x,k) = \dfrac{\theta_4(v, q(k))}{\theta_4(0, q(k))}$, $v = \dfrac{\pi x}{2K(k)}$, $|k| \leq 1$, and $q(k)$ is the elliptic nome.

See also: Wikipedia [1331], MathWorld [281], Ehrhardt [297] (3.2.15.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.neville_theta_n(0.01, 0.5)
1.0000064743224766
>>> math53.neville_theta_n(0.5, 0.9333)
1.065460920528952
>>> math53.neville_theta_n(0.9333, 0.5)
1.0435380078392624
```

27.8.4 Neville $\theta_s(x, k)$

math53.**neville_theta_s**(x, k)

Returns the Neville theta function $\theta_s(x,k) = \dfrac{2K(k)}{\pi}\dfrac{\theta_1(v,q(k))}{\theta_1'(0,q(k))}, \quad v = \dfrac{\pi x}{2K(k)}, |k| \le 1$, and $q(k)$ is the elliptic nome.

See also: Wikipedia [1331], MathWorld [282], Ehrhardt [297] (3.2.15.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.neville_theta_s(0.01, 0.5)
0.009999856412344637
>>> math53.neville_theta_s(0.5, 0.9333)
0.4947356796971494
>>> math53.neville_theta_s(0.9333, 0.5)
0.8206456927162612
```

CHAPTER

TWENTYEIGHT

REAL FUNCTIONS RELATED TO LERCHS PHI

28.1 Polygamma, and related functions

28.1.1 Pentagamma function $\psi'''(x)$

math53.**pentagamma**(x)

Returns the pentagamma function $\psi'''(x), \quad x \neq 0, -1, -2, \ldots$ The function returns the Hurwitz zeta value $6\zeta(4, x)$ if x is positive; for $x < 0$ it returns $\psi'''(x) = -2\pi^4 \left(1 + 4\cot^2(\pi x) + 3\cot^4(\pi x)\right) - 6\zeta(4, 1 - x)$.

See also Wikipedia [1267], MathWorld [893], NIST [23], BoostMath [109], Ehrhardt [297] (3.5.6.5).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.pentagamma(0.01)
600000006.2510617
>>> math53.pentagamma(10.5)
0.001990086237144355
>>> math53.pentagamma(20.9333)
0.00023415053202383002
```

28.1.2 Tetragamma function, $\psi''(x)$

math53.**tetragamma**(x)

Returns the tetragamma function $\psi''(x), \quad x \neq 0, -1, -2, \ldots$ The function returns the Hurwitz zeta value $-2\zeta(3, x)$ if x is positive; for $x < 0$ it returns $\psi''(x) = -2\pi^3 \cot(\pi x)\left(1 + \cot^2(\pi x)\right) - 2\zeta(3, 1 - x)$.

See also Wikipedia [1267], MathWorld [893], NIST [23], BoostMath [109], Ehrhardt [297] (3.5.6.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.tetragamma(0.01)
-2000002.3403986772
>>> math53.tetragamma(10.5)
-0.009975144247715168
>>> math53.tetragamma(20.9333)
-0.0023936641630670093
```

28.1.3 Auxiliary function $\psi^*(x)$

math53.**psistar**(x)

Returns $\psi^*(x) = \psi(x) - \log(x), x > 0$. The function is useful when computing differences of ψ functions, because it can avoid cancellation for larger x values. See also Ehrhardt [297] (3.5.6.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.psistar(0.01)
-95.95571527188056
>>> math53.psistar(10.5)
-0.04837422286579131
>>> math53.psistar(20.9333)
-0.024075515594002023
```

28.1.4 Inverse digamma function $\psi^{-1}(y)$

math53.**psi_inv**(y)

Returns $\psi^{-1}(y)$, the functional inverse of of the digamma function, i.e. it returns x with $\psi(x) = y$.

See also Wikipedia [1275], MathWorld [917], NIST [23], BoostMath [117], Ehrhardt [297] (3.5.6.7).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.psi_inv(0.01)
1.4720152539294011
>>> math53.psi_inv(10.5)
36316.00267309933
>>> math53.psi_inv(20.9333)
1233720207.3863456
```

28.1.5 Bateman function $G(x)$

math53.**bateman_g**(x)

Returns the Bateman function $G(x) = \psi((x+1)/2) - \psi(x/2); x \neq 0, -1, -2, \ldots$ See also Ehrhardt [297] (3.5.6.8).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.bateman_g(0.01)
198.6299765458315
>>> math53.bateman_g(10.5)
0.09975303466039102
>>> math53.bateman_g(20.9333)
0.048910504209268506
```

28.2 Polylogarithm, and related functions

28.2.1 Polylogarithm of integer order, $\mathrm{Li}_n(x)$

math53.**polylog_i**(*n*, *x*)

Returns the polylogarithm of integer order n, $\mathrm{Li}_n(x) = \sum_{k=1}^{\infty} \frac{x^k}{k^n}, \quad n \in \mathbb{Z}, |x| < 1$, or its analytic continuation; for $n > 0$, $x > 1$ the real part of $\mathrm{Li}_n(x)$ is returned.

See also Wikipedia [1332], MathWorld [969], NIST [18], Ehrhardt [297] (3.6.11).

Returns the polylogarithm function of integer order n.

This function returns the polylogarithm function of integer order n

$$\mathrm{Li}_n(z) = \sum_{k=1}^{\infty} \frac{z^k}{k^n}, \quad n \in \mathbb{Z}, |z| < 1.$$

or its analytic continuation; for $n > 0$ there is the arguments restriction $x \leq 1$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.polylog_i(2, 0.01)
0.010025111740139094
>>> math53.polylog_i(3, 10.5)
5.66534093854826
>>> math53.polylog_i(4, 20.9333)
13.767054582329026
```

28.2.2 Fermi-Dirac integrals of integer order, $F_n(x)$

math53.**fermi_dirac_i**(*n*, *x*)

Returns the Fermi-Dirac integral of integer order n, $F_s(x) = \frac{1}{\Gamma(n+1)} \int_0^{\infty} \frac{t^n}{e^{t-x}+1} = -\mathrm{Li}_{n+1}(-e^x)\,.$

See also: Wikipedia [1319], MathWorld [957], Ehrhardt [297] (3.6.8.2).

This function returns the complete Fermi-Dirac integrals $F_n(x)$ of integer order. They are defined for real orders $s > -1$ by

$$F_s(x) = \frac{1}{\Gamma(s+1)} \int_0^{\infty} \frac{t^s}{e^{t-x}+1} \mathrm{d}t$$

and by analytic continuation for $s \leq -1$ using polylogarithms

$$F_s(x) = -\mathrm{Li}_{s+1}(-e^x).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.fermi_dirac_i(2, 0.01)
0.9098020885004644
>>> math53.fermi_dirac_i(3, 10.5)
599.031966057601
>>> math53.fermi_dirac_i(4, 20.9333)
36051.591345246416
```

28.2.3 Fermi-Dirac integral $F_{-1/2}(x)$

math53.**fermi_dirac_m05**(x)

Returns the Fermi-Dirac integral $F_{-1/2}(x)$. See also: Wikipedia [1319], MathWorld [957], Ehrhardt [297] (3.6.8.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.fermi_dirac_m05(0.01)
0.608705610911171
>>> math53.fermi_dirac_m05(10.5)
3.6420723652672895
>>> math53.fermi_dirac_m05(20.9333)
5.157769932482041
```

28.2.4 Fermi-Dirac integral $F_{1/2}(x)$

math53.**fermi_dirac_p05**(x)

Returns the Fermi-Dirac integral $F_{1/2}(x)$. See also: Wikipedia [1319], MathWorld [957], Ehrhardt [297] (3.6.8.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.fermi_dirac_p05(0.01)
0.7712150360437194
>>> math53.fermi_dirac_p05(10.5)
25.88346611317671
>>> math53.fermi_dirac_p05(20.9333)
72.25098647441794
```

28.2.5 Fermi-Dirac integral $F_{3/2}(x)$

math53.**fermi_dirac_p15**(x)

Returns the Fermi-Dirac integral $F_{3/2}(x)$. See also: Wikipedia [1319], MathWorld [957], Ehrhardt [297] (3.6.8.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.fermi_dirac_p15(0.01)
0.874881667590789
>>> math53.fermi_dirac_p15(10.5)
113.49527140479812
>>> math53.fermi_dirac_p15(20.9333)
611.7653726268513
```

28.2.6 Fermi-Dirac integral $F_{5/2}(x)$

math53.**fermi_dirac_p25**(x, s)

Returns the Fermi-Dirac integral $F_{5/2}(x)$. See also: Wikipedia [1319], MathWorld [957], Ehrhardt [297] (3.6.8.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.fermi_dirac_p25(0.01)
0.9362639349902176
>>> math53.fermi_dirac_p25(10.5)
364.9252593211593
>>> math53.fermi_dirac_p25(20.9333)
3726.9235556746216
```

28.2.7 Inverse tangent integral, $\mathrm{Ti}_2(x)$

`math53.tangent_int_2(x)`

Returns the inverse tangent integral $\mathrm{Ti}_2(x) = \int_0^x \frac{\arctan(t)}{t}\,\mathrm{d}t$.

See also: Wikipedia [1333], MathWorld [963], Ehrhardt [297] (3.6.16).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.tangent_int_2(0.01)
0.009999888892888685
>>> math53.tangent_int_2(10.5)
3.788674042119505
>>> math53.tangent_int_2(20.9333)
4.8250862474762
```

28.2.8 Lobachevsky's log-cos integral, $L(x)$

`math53.lobachevsky_c(x)`

Returns Lobachevsky's log-cos integral $L(x) = \int_0^x \log|\cos(t)|\,\mathrm{d}t$. See also: Ehrhardt [297] (3.6.18).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.lobachevsky_c(0.01)
1.6666833336508016e-07
>>> math53.lobachevsky_c(10.5)
6.7712697858797855
>>> math53.lobachevsky_c(20.9333)
15.017229619310305
```

28.2.9 Lobachevsky's log-sin integral, $\Lambda(x)$

`math53.lobachevsky_s(x, s)`

Returns Lobachevsky's log-sin integral $\Lambda(x) = \int_0^x \log|2\sin(t)|\,\mathrm{d}t$. See also: Ehrhardt [297] (3.6.19).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.lobachevsky_s(0.01)
0.049120285609948126
>>> math53.lobachevsky_s(10.5)
0.3226979802070858
>>> math53.lobachevsky_s(20.9333)
-0.33243071505340765
```

28.2.10 Debye functions, $\mathrm{D}_n(x)$

`math53.debye(n, x)`

Returns the Debye function $\mathrm{D}_n(x) = \frac{n}{x^n}\int_0^x \frac{t^n}{e^t-1}\,\mathrm{d}t, \quad n > 0, x \geq 0$.

See also: Wikipedia [1340], MathWorld [996], Abramowitz and Stegun. [4] section 27.1, Ehrhardt [297] (3.10.6), Dubinov and Dubinova [289].

This routine returns the Debye functions

$$\mathrm{D}_n(x) = \frac{n}{x^n}\int_0^x \frac{t^n}{e^t-1}\mathrm{d}t \quad (n>0, x\geq 0).$$

$$\mathrm{D}_n^{(1)}(x) = \int_0^x \frac{t^n}{e^t-1}\mathrm{d}t = n!\zeta(n+1) - n!Z_{n+1}(z).$$

$$\mathrm{D}_n^{(2)}(x) = \int_x^\infty \frac{t^n}{e^t-1}\mathrm{d}t = n!Z_{n+1}(z).$$

$$\mathrm{D}_n^{(2)}(x) = \Gamma(n+1)\zeta(n+1) - \mathrm{D}_n^{(1)}(x).$$

$$\mathrm{D}_n^{(1)}(x) = n!\zeta(n+1) - \mathrm{D}_n^{(2)}(x).$$

$$Z_{n+1}(z) = \sum_{k=0}^{n} \mathrm{Li}_{n-k+1}(e^{-z})\frac{z^k}{k!} \quad (n=1,2,3,\ldots).$$

In terms of the incomplete zeta functions or "Debye functions" (Abramowitz & Stegun 1972, § 27.1):

$$Z_n(z) = \frac{1}{(n-1)!}\int_z^\infty \frac{t^{n-1}}{e^t-1}\mathrm{d}t \qquad (n=1,2,3,\ldots),$$

the polylogarithm Lin(z) for positive integer n may be expressed as the finite sum (Wood 1992, § 16):

$$\mathrm{Li}_n(e^\mu) = \sum_{k=0}^{n-1} Z_{n-k}(-\mu)\frac{\mu^k}{k!} \qquad (n=1,2,3,\ldots).$$

A remarkably similar expression relates the "Debye functions" Zn(z) to the polylogarithm:

$$Z_n(z) = \sum_{k=0}^{n-1} \mathrm{Li}_{n-k}(e^{-z})\frac{z^k}{k!} \qquad (n=1,2,3,\ldots).$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.debye(2, 0.01)
0.9966708333287038
>>> math53.debye(3, 10.5)
0.016717989806569248
>>> math53.debye(4, 20.9333)
0.0005184007550452067
```

28.2.11 Transport integral, $\mathrm{J}_n(x)$

`math53.transport_jn(`n, x`)`

Returns the transport function $\mathrm{J}_n(x) = \int_0^x \frac{t^n e^t}{(e^t-1)^2}\,\mathrm{d}t, \quad n\geq 2, x\geq 0.$

See also: Ehrhardt [297] (3.10.24).

A.J. MacLeod, The numerical computation of transport integrals, 1992, Computer Physics Communications, 69, pp. 229-234, https://doi.org/10.1016/0010-4655(92)90162-R,

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.transport_jn(2, 0.01)
0.009999972222305555
>>> math53.transport_jn(3, 10.5)
7.169455825440913
>>> math53.transport_jn(4, 20.9333)
25.97556753445377
```

28.3 Riemann zeta, and related functions

28.3.1 Riemann $\zeta(n)$ for integer arguments

math53.**zeta_i**(n)

Returns the Riemann zeta function $\zeta(n)$ for integer arguments $n \neq 1$. For $n > 63$ the result is 1, for $0 \leq n \leq 63$ the value is taken from a table, otherwise the Bernoulli numbers are used: $\zeta(n) = B_{1-n}/(n-1)$ for $n < 0$.

See also Wikipedia [1334], MathWorld [970], NIST [19], BoostMath [158], Ehrhardt [297] (3.6.1.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.zeta_i(2)
1.6449340668482264
>>> math53.zeta_i(3)
1.2020569031595942
>>> math53.zeta_i(4)
1.0823232337111381
```

28.3.2 Riemann $\zeta(1+x)$

math53.**zeta1p**(x)

Returns the Riemann zeta function $\zeta(1+x)$ for $x \neq 0$. Normally used with $|x| \ll 1$ for increased accuracy near the pole of $\zeta(s)$ at $s = 1$.

See also Wikipedia [1334], MathWorld [970], NIST [19], BoostMath [158], Ehrhardt [297] (3.6.1.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.zeta1p(0.01)
100.57794333849687
>>> math53.zeta1p(10.5)
1.0003486558834918
>>> math53.zeta1p(20.9333)
1.0000002497344977
```

28.3.3 Dirichlet eta function for integer argument, $\eta(n)$

math53.**dirichlet_eta_i**(n)

Returns the Dirichlet eta function $\eta(n)$ for integer arguments. For $n > 64$ the result is 1, for $0 \leq n \leq 64$ the value is taken from a table, otherwise for $n < 0$ the Bernoulli numbers are used: $\eta(n) = (2^{1-n} - 1)B_{1-n}/(1-n)$.

See also: MathWorld [997], Ehrhardt [297] (3.6.3.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.dirichlet_eta_i(2)
0.8224670334241132
>>> math53.dirichlet_eta_i(3)
0.9015426773696957
>>> math53.dirichlet_eta_i(4)
0.9470328294972459
```

28.3.4 Prime zeta function

math53.**primezeta**(s)

Returns for $x > 0.2$ the prime zeta function $P(x) = \sum_{p \text{ prime}} p^{-x}$, or its real part for $x < 1$.

See also Ehrhardt [297] (3.6.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.primezeta(2.01)
0.4473533160988981
>>> math53.primezeta(10.5)
0.0006898729399747471
>>> math53.primezeta(20.9333)
4.995031614994917e-07
```

28.3.5 Riemann prime counting function, $R(x)$

math53.**riemann_r**(x)

Returns $R(x) = \sum_{n=1}^{\infty} \frac{\mu(n)}{n} \text{li}(x^{1/n}) 1 + \sum_{k=1}^{\infty} \frac{\log^k x}{k k! \zeta(k+1)}$, the Riemann prime counting function, for $x > 0$.

See also Wikipedia [1289], Wikipedia [1290], MathWorld [931], Wikipedia [1288], MathWorld [932], Ehrhardt [297] (3.10.20).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.riemann_r(2.01)
1.5459341840849095
>>> math53.riemann_r(10.5)
4.725293716386582
>>> math53.riemann_r(220.9333)
48.32330658563835
```

28.3.6 Inverse of the Riemann prime counting function, $R^{-1}(x)$

math53.**riemann_r_inv**(x)

Returns the functional inverse of the Riemann prime counting function, i.e. $R(R^{-1}(x)) = x$, for $x \geq 1.125$.

See also Ehrhardt [297] (3.10.21).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.riemann_r_inv(2.01)
3.0121183149089292
>>> math53.riemann_r_inv(10.5)
31.308893414099234
>>> math53.riemann_r_inv(220.9333)
1378.8514211496115
```

28.3.7 Rogers-Ramanujan continued fraction

math53.**rogers_ramanujan_cf**(q)

Returns $R(q) = \frac{q^{1/5}}{1+} \frac{q}{1+} \frac{q^2}{1+} \frac{q^3}{1+} \frac{q^5}{1+}$, the Rogers-Ramanujan continued fraction, for $|q| < 1$.

See also Ehrhardt [297] (3.10.22).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.rogers_ramanujan_cf(0.01)
0.3941659056233625
>>> math53.rogers_ramanujan_cf(0.5)
0.6180183781955219
>>> math53.rogers_ramanujan_cf(0.9333)
0.6180339887498949
```

28.3.8 Moebius function, $\mu(n)$

math53.**moebius**(*n*)

Returns $\mu(n)$, the Möbius function.

For integer n, the Moebius μ function $\mu(n)$ equals 0 if n has repeated integer factors. Otherwise, if n is the product of k distinct primes, the Moebius μ function $\mu(n)$ equals $(-1)^k$.

https://en.wikipedia.org/wiki/M%C3%B6bius_function

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.moebius(21)
1
>>> math53.moebius(30)
-1
>>> math53.moebius(40)
0
```

28.3.9 q-Pochhammer Euler function, $(q)_\infty$

math53.**euler_q**(*n*, *x*)

Returns $\phi(q) = (q)_\infty = \prod_{k=1}^{\infty} \left(1 - q^k\right)$, the q-Pochhammer Euler function, for $-1 \leq q \leq 1$.

See also: MathWorld [1028], Wikipedia [1396], Ehrhardt [297] (3.10.19).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.euler_q(0.01)
0.989900000010001
>>> math53.euler_q(0.5)
0.2887880950866024
>>> math53.euler_q(0.9333)
4.2824759927091205e-10
```

28.4 Miscellaneous functions

28.4.1 Log-series distribution, pmf

`math53.logseries_pmf(p, k)`

Returns pmf(x), the value of the probability mass function (*Pmf*) of the log-series distribution with shape parameters $p, 0 \leq p \leq 1$, $k > 1$. and the support interval $(0, +\infty)$.

See also Wikipedia [1225], MathWorld [828], Johnson *et al.* [389], Ehrhardt [297] (3.9.17).

$$\text{pmf}(x) = \frac{-1}{\log(1-p)} \frac{p^k}{k}.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logseries_pmf(0.01, 1)
0.9949916247342216
>>> math53.logseries_pmf(0.5, 2)
0.18033688011112042
>>> math53.logseries_pmf(0.9333, 3)
0.10008431129451109
```

28.4.2 Log-series distribution, cdf

`math53.logseries_cdf(p, k)`

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the log-series distribution with shape parameters $p, 0 \leq p \leq 1$, $k > 1$. and the support interval $(0, +\infty)$.

See also Wikipedia [1225], MathWorld [828], Johnson *et al.* [389], Ehrhardt [297] (3.9.17).

$$\text{cdf}(x) = 1 + \frac{B(p; k+1, 0)}{\log(1-p)} = 1 + \frac{a^k(\Phi(p, 1, k) - 1/k)}{\log(1-p)},$$

where $B(\cdot)$ is the incomplete beta function, and $\Phi(\cdot)$ is Lerch's transcendent.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.logseries_cdf(0.01, 1)
0.9949916247342216
>>> math53.logseries_cdf(0.5, 2)
0.9016844005556022
>>> math53.logseries_cdf(0.9333, 3)
0.6056425762046286
```

28.4.3 Zeta distribution, pmf

`math53.zeta_pmf(r, k)`

Returns pmf(x), the value of the probability mass function (*Pmf*) of the zeta distribution with parameter r, and $0 \leq q \leq 1$.

See also Wikipedia [1229], Rinne [480], Johnson *et al.* [389] page 527, Ehrhardt [297] (3.9.34).

$$\text{pmf}_X(k) = \frac{k^{-(r+1)}}{\zeta(r+1)}.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.zeta_pmf(0.01, 1)
0.009942537765307869
>>> math53.zeta_pmf(0.5, 2)
0.1353378988096703
>>> math53.zeta_pmf(0.9333, 3)
0.0698271086340029
```

28.4.4 Zeta distribution, cdf

`math53.zeta_cdf`(r, k)

Returns cdf(x), the value of the cumulative distribution function (*Cdf*) of the zeta distribution:

$$\text{cdf}(x) = \frac{H_k^{r+1}}{\zeta(r+1)} = 1 - \frac{\zeta(r+1, k+1)}{\zeta(r+1)}.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.zeta_cdf(0.01, 1)
0.009942537765308734
>>> math53.zeta_cdf(0.5, 2)
0.5181312828090969
>>> math53.zeta_cdf(0.9333, 3)
0.806786037677741
```

CHAPTER

TWENTYNINE

REAL FUNCTIONS RELATED TO ${}_0F_1$

29.1 Bessel functions of integer order

29.1.1 Bessel function of the 1st kind, order 0, $J_0(x)$

`math53.bessel_j0`(x)

where `ctx` is `math53` or `ctxcpp`.

Returns $J_0(x)$, the Bessel function of the 1st kind, order zero. See also Wikipedia [1250], MathWorld [878], NIST [440], BoostMath [94], Ehrhardt [297] (3.1.1.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselJ0(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselJ0('0.51')
ereal('5.351847902755998475 4E-1')
```

29.1.2 Bessel function of the 1st kind, order 1, $J_1(x)$

`math53.bessel_j1`(x)

where `ctx` is `math53` or `ctxcpp`.

Returns $J_1(x)$, the Bessel function of the 1st kind, order one. See also Wikipedia [1250], MathWorld [878], NIST [440], BoostMath [94], Ehrhardt [297] (3.1.1.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselJ1(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselJ1('0.51')
ereal('5.351847902755998475 4E-1')
```

29.1.3 Bessel function of the 1st kind, integer order n, $J_n(x)$

`math53.bessel_jn`(n, x)

where `ctx` is `math53` or `ctxcpp`.

Returns $J_n(x)$, the Bessel function of the 1st kind, order n. See also Wikipedia [1250], MathWorld [878], NIST [440], BoostMath [94], Ehrhardt [297] (3.1.1.1).

$$J_n(x) = \left(\tfrac{1}{2}x\right)^n \sum_{k=0}^{\infty} (-1)^k \frac{(x^2/4)^k}{k!\Gamma(n+k+1)}.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselJn(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselJn(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

29.1.4 Bessel function of the 2nd kind, order $0, Y_0(x)$

`math53.bessel_y0`(x)

where `ctx` is `math53` or `ctxcpp`.

Returns $Y_0(x)$, the Bessel function of the 2nd kind, order zero, $x > 0$. See also Wikipedia [1251], MathWorld [879], NIST [441], BoostMath [94], Ehrhardt [297] (3.1.1.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselY0(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselY0('0.51')
ereal('5.351847902755998475 4E-1')
```

29.1.5 Bessel function of the 2nd kind, order $1, Y_1(x)$

`math53.bessel_y1`(x)

where `ctx` is `math53` or `ctxcpp`.

Returns $Y_1(x)$, the Bessel function of the 2nd kind, order one, $x > 0$. See also Wikipedia [1251], MathWorld [879], NIST [441], BoostMath [94], Ehrhardt [297] (3.1.1.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselY1(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselY1('0.51')
ereal('5.351847902755998475 4E-1')
```

29.1.6 Bessel function of the 2nd kind, integer order $n, Y_n(x)$

`math53.bessel_yn`(x)

where `ctx` is `math53` or `ctxcpp`.

Returns $Y_n(x)$, the Bessel function of the 2nd kind, order n, $x > 0$. See also Wikipedia [1251], MathWorld [879], NIST [441], BoostMath [94], Ehrhardt [297] (3.1.1.1).

$$Y_n(z) = -\frac{\left(\frac{z}{2}\right)^{-n}}{\pi}\sum_{k=0}^{n-1}\frac{(n-k-1)!}{k!}\left(\frac{z^2}{4}\right)^k + \frac{2}{\pi}J_n(z)\log\frac{z}{2} - \frac{\left(\frac{z}{2}\right)^n}{\pi}\sum_{k=0}^{\infty}(\psi(k+1)+\psi(n+k+1))\frac{\left(-\frac{z^2}{4}\right)^k}{k!(n+k)!}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselYn(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselYn(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

An example in double precision (32 bit version of xlcalcnet)

```
>>> from xlcalcnet import math53
>>> math53.BesselYn(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.BesselYn(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

29.2 Modified Bessel functions of integer order

29.2.1 Modified Bessel function of the 1st kind, order 0, $I_0(x)$

math53.**bessel_i0**(x)

Returns I0(x), the modified Bessel function of the 1st kind, order zero. See also Wikipedia [1264], MathWorld [907], NIST [445], BoostMath [108], Ehrhardt [297] (3.1.2.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselI0(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselI0('0.51')
ereal('5.351847902755998475 4E-1')
```

29.2.2 Modified Bessel function of the 1st kind, order 1, $I_1(x)$

math53.**bessel_i1**(x)

Returns I1(x), the modified Bessel function of the 1st kind, order one. See also Wikipedia [1264], MathWorld [907], NIST [445], BoostMath [108], Ehrhardt [297] (3.1.2.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselI1(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselI1('0.51')
ereal('5.351847902755998475 4E-1')
```

29.2.3 Modified Bessel function of the 1st kind, integer order n, $I_n(x)$

math53.**damath_bessel_in**(x)

Returns I_n(x), the modified Bessel function of the 1st kind, order n. See also Wikipedia [1264], MathWorld [907], NIST [445], BoostMath [108], Ehrhardt [297] (3.1.2.5).

$$I_\nu\left(z\right) = \left(\tfrac{1}{2}z\right)^\nu \sum_{k=0}^{\infty} \frac{\left(\frac{1}{4}z^2\right)^k}{k!\Gamma\left(\nu+k+1\right)}.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselIn(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselIn(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

29.2.4 Modified Bessel function of the 2nd kind, order 0, $K_0(x)$

math53.**bessel_k0**(x)

Returns K0(x), the modified Bessel function of the 2nd kind, order zero, x>0. See also Wikipedia [1264], MathWorld [908], NIST [446], BoostMath [108], Ehrhardt [297] (3.1.2.6).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselK0(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselK0('0.51')
ereal('5.351847902755998475 4E-1')
```

29.2.5 Modified Bessel function of the 2nd kind, order $1, K_1(x)$

`math53.bessel_k1`(x)

Returns K1(x), the modified Bessel function of the 2nd kind, order one, x>0. See also Wikipedia [1264], MathWorld [908], NIST [446], BoostMath [108], Ehrhardt [297] (3.1.2.8).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselK1(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselK1('0.51')
ereal('5.351847902755998475 4E-1')
```

29.2.6 Modified Bessel function of the 2nd kind, integer order $n, K_n(x)$

`math53.bessel_kn`(x)

Returns K_n(x), the modified Bessel function of the 2nd kind, order n, x > 0. See also Wikipedia [1264], MathWorld [908], NIST [446], BoostMath [108], Ehrhardt [297] (3.1.2.10).

$$\begin{aligned} K_n\left(z\right) &= \tfrac{1}{2}(\tfrac{1}{2}z)^{-n}\sum_{k=0}^{n-1}\frac{(n-k-1)!}{k!}(-\tfrac{1}{4}z^2)^k + (-1)^{n+1}\log\left(\tfrac{1}{2}z\right)I_n\left(z\right) \\ &+ (-1)^n\tfrac{1}{2}(\tfrac{1}{2}z)^n\sum_{k=0}^{\infty}\left(\psi\left(k+1\right)+\psi\left(n+k+1\right)\right)\frac{(\frac{1}{4}z^2)^k}{k!(n+k)!} \end{aligned} \tag{29.1}$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselKn(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselKn(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

29.2.7 Exponentially scaled modified Bessel function of the 1st kind, order $0, I_{0,e}(x)$

`math53.bessel_i0e`(x)

Returns I0(x)*exp(-|x|), the exponentially scaled modified Bessel function of the 1st kind, order zero. See also Wikipedia [1264], MathWorld [907], NIST [445], BoostMath [108], Ehrhardt [297] (3.1.2.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselI0e(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselI0e('0.51')
ereal('5.351847902755998475 4E-1')
```

29.2.8 Exponentially scaled modified Bessel function of the 1st kind, order 1, $I_{1,e}(x)$

math53.**bessel_i1e**(x)

Returns I1(x)*exp(-|x|), the exponentially scaled modified Bessel function of the 1st kind, order one. See also Wikipedia [1264], MathWorld [907], NIST [445], BoostMath [108], Ehrhardt [297] (3.1.2.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselI1e(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselI1e('0.51')
ereal('5.351847902755998475 4E-1')
```

29.2.9 Exponentially scaled modified Bessel function of the 2nd kind, order 0, $K_{0,e}(x)$

math53.**bessel_k0e**(x)

Returns K0(x)*exp(x), the exponentially scaled modified Bessel function of the 2nd kind, order zero, x>0. See also Wikipedia [1264], MathWorld [908], NIST [446], BoostMath [108], Ehrhardt [297] (3.1.2.7).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselK0e(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselK0e('0.51')
ereal('5.351847902755998475 4E-1')
```

29.2.10 Exponentially scaled modified Bessel function of the 2nd kind, order 1, $K_{1,e}(x)$

math53.**bessel_k1e**(x)

Returns K1(x)*exp(x), the exponentially scaled modified Bessel function of the 2nd kind, order one, x>0. See also Wikipedia [1264], MathWorld [908], NIST [446], BoostMath [108], Ehrhardt [297] (3.1.2.9).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselK1e(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselK1e('0.51')
ereal('5.351847902755998475 4E-1')
```

29.3 Integrals of zero-order Bessel functions

29.3.1 Integral of J_0

math53.**bessel_j0_int**(x)

Returns the integral $\int_0^x J_0(t)\mathrm{d}t$. See also Wikipedia [1250], MathWorld [878], NIST [440], BoostMath [94], Ehrhardt [297] (3.1.5.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselJ0Int(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselJ0Int('0.51')
ereal('5.351847902755998475 4E-1')
```

29.3.2 Integral of Y_0

math53.**bessel_y0_int**(x)

Returns the integral $\int_0^x Y_0(t)\mathrm{d}t, \quad x \geq 0$. See also Wikipedia [1251], MathWorld [879], NIST [441], BoostMath [94], Ehrhardt [297] (3.1.5.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselY0Int(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselY0Int('0.51')
ereal('5.351847902755998475 4E-1')
```

29.3.3 Integral of I_0

math53.**bessel_i0_int**(x)

Returns the integral $\int_0^x I_0(t)\mathrm{d}t$. See also Wikipedia [1264], MathWorld [907], NIST [445], BoostMath [108], Ehrhardt [297] (3.1.5.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselI0Int(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.BesselI0Int('0.51')
ereal('5.351847902755998475 4E-1')
```

29.3.4 Integral of K_0

math53.**bessel_k0_int**(x)

Returns the integral $\int_0^x K_0(t)\mathrm{d}t, \quad x \geq 0$. See also Wikipedia [1264], MathWorld [908], NIST [446], BoostMath [108], Ehrhardt [297] (3.1.5.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.BesselK0Int(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.BesselK0Int('0.51')
ereal('5.351847902755998475 4E-1')
```

29.4 Kelvin functions of order 0

29.4.1 Kelvin function $\mathrm{ber}0(x)$

math53.**kelvin_ber0**(x)

Returns the Kelvin function ber. See also Wikipedia [1381], MathWorld [1022], NIST [444], Mpmath [708].

Returns the Kelvin function $\mathrm{ber}_0(x)$, defined as

$$\mathrm{ber}_0(x) = \sum_{k=0}^{\infty} (-1)^k \frac{\left(\frac{1}{4}x^2\right)^{2k}}{((2k)!)^2}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinber(n, x); mx = mpm.kelvinber(n, x); gx = gmp.kelvinber(n, x)
>>> fx = fpm.kelvinber(n, x); ax = apm.kelvinber(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -2.213802495986938888682464345899509922321E-1
mpm: -2.213802495986938888682464345899509922321e-1
gmp: -2.213802495986938888682464345899509922321E-01
fpm: -2.21380249598694E-01
apm: -2.213802495986938888682464345899509922332e-1 (-1.283e-37%)
```

29.4.2 Kelvin function $\mathrm{bei}0(x)$

math53.**kelvin_bei0**(x)

Returns the Kelvin function bei. See also Wikipedia [1380], MathWorld [1021], NIST [444], Mpmath [707].

Returns the Kelvin function $\mathrm{bei}_0(x)$ is defined as

$$\mathrm{bei}_0(x) = \sum_{k=0}^{\infty} (-1)^k \frac{\left(\frac{1}{4}x^2\right)^{2k+1}}{((2k+1)!)^2}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinbei(n, x); mx = mpm.kelvinbei(n, x); gx = gmp.kelvinbei(n, x)
>>> fx = fpm.kelvinbei(n, x); ax = apm.kelvinbei(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: 1.937586785266042766896808122272260201255E+0
mpm: 1.937586785266042766896808122272260201255e+0
gmp: 1.937586785266042766896808122272260201255E+00
fpm: 1.93758678526604E+00
apm: 1.937586785266042766896808122272260201256e+0 (1.54e-38%)
```

29.4.3 Kelvin function $\mathrm{ker}0(x)$

math53.**kelvin_ker0**(x)

Returns the Kelvin function ker. See also Wikipedia [1383], MathWorld [1024], NIST [444], Mpmath [710].

The Kelvin function $\mathrm{ker}_0(x)$ is defined as

$$\mathrm{ker}_0(x) = -\log\left(\tfrac{1}{2}x\right)\mathrm{ber}_0(x) + \tfrac{1}{4}\pi\,\mathrm{bei}_0(x) + \sum_{k=0}^{\infty} (-1)^k \frac{\psi(2k+1)\left(\frac{1}{4}x^2\right)^{2k}}{((2k)!)^2}.$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinker(n, x); mx = mpm.kelvinker(n, x); gx = gmp.kelvinker(n, x)
>>> fx = fpm.kelvinker(n, x); ax = apm.kelvinker(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -6.702923330379869775199782194748322134382E-2
mpm: -6.702923330379869775199782194748322134382e-2
gmp: -6.702923330379869775199782194748322134382E-02
fpm: -6.70292333037987E-02
apm: -6.702923330379869775199782194748322134370e-2 (-8.669e-36%)
```

29.4.4 Kelvin function $\mathrm{kei0}(x)$

math53.**kelvin_kei0**(x)

Returns the Kelvin function kei. See also Wikipedia [1382], MathWorld [1023], NIST [444], Mpmath [709].

The Kelvin function $\mathrm{kei}_0(x)$ is defined as

$$\mathrm{kei}_0(x) = -\log\left(\tfrac{1}{2}x\right)\mathrm{ber}_0(x) - \tfrac{1}{4}\pi\,\mathrm{bei}_0(x) + \sum_{k=0}^{\infty}(-1)^k\frac{\psi(2k+2)\left(\frac{1}{4}x^2\right)^{2k+1}}{((2k+1)!)^2}$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = 0; x = 3
>>> \mathrm{d}x = dec.kelvinkei(n, x); mx = mpm.kelvinkei(n, x); gx = gmp.kelvinkei(n, x)
>>> fx = fpm.kelvinkei(n, x); ax = apm.kelvinkei(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax],  aligned=True)
dec: -5.112188404598678140246687753930501705762E-2
mpm: -5.112188404598678140246687753930501705762e-2
gmp: -5.112188404598678140246687753930501705762E-02
fpm: -5.11218840459868E-02
apm: -5.112188404598678140246687753930501705753e-2 (-9.353e-36%)
```

29.4.5 First derivative of the Kelvin function $\mathrm{ber0}, \mathrm{ber0}'(x)$

math53.**kelvin_ber0_prime**(x)

Returns the Kelvin function ber'(x), x >= 0

See also: Ehrhardt [297] (3.1.8.8).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.KelvinBerPrime(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.KelvinBerPrime('0.51')
ereal('5.3518479027559984754E-1')
```

29.4.6 First derivative of the Kelvin function $\mathrm{bei0}, \mathrm{bei0}'(x)$

math53.**kelvin_bei0_prime**(x)

Returns the Kelvin function bei'(x), x >= 0

See also: Ehrhardt [297] (3.1.8.9).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.KelvinBeiPrime(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.KelvinBeiPrime('0.51')
ereal('5.351847902755998475 4E-1')
```

29.4.7 First derivative of the Kelvin function $\mathrm{ker}0, \mathrm{ker}0'(\nu, x)$

math53.**kelvin_ker0_prime**(x)

Returns the Kelvin function ker'(x), x > 0

See also: Ehrhardt [297] (3.1.8.10).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.KelvinKerPrime(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.KelvinKerPrime('0.51')
ereal('5.351847902755998475 4E-1')
```

29.4.8 First derivative of the Kelvin function $\mathrm{kei}0, \mathrm{kei}0'(x)$

math53.**kelvin_kei0_prime**(x)

Returns the Kelvin function kei'(x), x >= 0

See also: Ehrhardt [297] (3.1.8.11).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.KelvinKeiPrime(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.KelvinKeiPrime('0.51')
ereal('5.351847902755998475 4E-1')
```

29.5 Synchrotron functions

29.5.1 First synchrotron function $F(x)$

math53.**synchrotron_f**(x)

Returns the first synchrotron function $F(x)$ for $x \geq 0$, $F(x) = x \int_x^\infty K_{5/3}(t)\mathrm{d}t$.

See also: Ehrhardt [297] (3.1.11.1).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SynchF(2.25)
ereal('5.235987755982988730 7E-1')
>>> math53.SynchF(12.25)
ereal('5.351847902755998475 4E-1')
```

29.5.2 Second synchrotron function $G(x)$

math53.**synchrotron_g**(x)

Returns the second synchrotron function $G(x) = xK_{2/3}(x)$ for $x \geq 0$.

See also: Ehrhardt [297] (3.1.11.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SynchG(2.25)
ereal('5.235987755982988730 7E-1')
>>> math53.SynchG(12.25)
ereal('5.351847902755998475 4E-1')
```

CHAPTER

THIRTY

REAL FUNCTIONS RELATED TO ${}_1F_1$

30.1 Incomplete gamma functions

30.1.1 Tricomi's entire incomplete gamma function: $\gamma^*(a,x)$

`ctx.gamma_tricomi(`*a*, *z*, *len*`)`

Returns Tricomi's entire incomplete gamma function $\gamma^*(a,x)$.

See also: Flint [741], Flint [731].

This routine returns Tricomi's incomplete gamma function γ^*, defined as

$$\gamma^*(a,x) = e^{-x}\frac{M(1,a+1,x)}{\Gamma(a+1)}$$

Special cases are $\gamma^*(0,x) = 1, \gamma^*(a,0) = 1/\Gamma(a+1)$, and $\gamma^*(-n,x) = x^n$, if $-n$ is a negative integer. Otherwise there are the following relations to the other incomplete functions:

$$\gamma^*(a,x) = \frac{x^{-a}}{\Gamma(a)}\gamma(a,x) = x^{-a}P(a,x).$$

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; a= 10; x = 30
>>> \mathrm{d}x = dec.gamma_tricomi(a, x); mx = mpm.gamma_tricomi(a, x); gx = gmp.gamma_tricomi(a, x)
>>> fx = fpm.gamma_tricomi(a, x); ax = apm.gamma_tricomi(a, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.693496720095407516508168391232985587748E-15
mpm:  1.693496720095407516508168391232985587748e-15
gmp:  1.693496720095407516508168391232985587748E-15
fpm:  1.69349672009541E-15
apm:  1.693496720095407516508168391232985587748e-15 (4.214e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; a= '10 + 3j'; z = '3 + 2j'
>>> \mathrm{d}z = dec.gamma_tricomi(a, z); mz = mpm.gamma_tricomi(a, z); gz = gmp.gamma_tricomi(a, z)
>>> fz = fpm.gamma_tricomi(a, z); az = apm.gamma_tricomi(a, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: -2.6190844813399921279E-8               - 1.3081710985777839884E-8j
mpm: -2.6190844813399921279e-8               - 1.3081710985777839884e-8j
gmp: -2.6190844813399921279E-08              - 1.3081710985777839884E-08j
fpm: -2.61908448133999E-08                   - 1.3081710985778E-08j
apm: -2.6190844813399921279e-8 (-1.446e-18%) - 1.3081710985777839884e-8 (-2.75e-18%)j
```

30.1.2 Truncated exponential function, $e_n(x)$

`math53.expn(n, x)`

Returns $e_n(x) = \sum_{k=0}^{n} \frac{x^k}{k!} = \frac{\Gamma(n+1,x)}{\Gamma(n+1)} e^x$, the truncated exponential sum function, for $n > 0$.

See also: Ehrhardt [297] (3.10.25).

https://mathworld.wolfram.com/ExponentialSumFunction.html

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Expn(2,3)
ereal('5.235987755982988730 7E-1')
>>> math53.Expn(4,13)
ereal('5.351847902755998475 4E-1')
```

30.1.3 Relative exponential, $\mathrm{exprel}_n(x)$

`math53.expreln(n, x)`

Returns $\mathrm{exprel}_n(x) = \frac{n!}{x^n}\left(e^x - \sum_{k=0}^{n-1} \frac{x^k}{k!}\right) = e^x x^{-n}\left(\Gamma(1+n) - n\Gamma(n,x)\right) = {}_1F_1(1, 1+n, x)$.

See also Ehrhardt [297] (3.10.10).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Expreln(3, 4)
ereal('5.235987755982988730 7E-1')
>>> math53.Expreln(3, 12)
ereal('5.351847902755998475 4E-1')
```

30.2 Coulomb wave functions

Coulomb wave functions are solutions of the Coulomb wave equation

$$y'' + \left(1 - \frac{2\eta}{z} - \frac{\ell(\ell+1)}{z^2}\right) y = 0$$

which is the radial Schrödinger equation for a charged particle in a Coulomb potential $1/z$, where ℓ is the orbital angular momentum and η is the Sommerfeld parameter. The standard solutions are named $F_\ell(\eta, z)$ (regular at the origin $z = 0$) and $G_\ell(\eta, z)$ (irregular at the origin). The irregular solutions $H_\ell^{\pm}(\eta, z) = G_\ell(\eta, z) \pm iF_\ell(\eta, z)$ are also used. The redundant functions $H^{\pm}$ are provided explicitly since taking the linear combination of F and G suffers from cancellation in parts of the complex plane.

Coulomb wave functions are special cases of confluent hypergeometric functions. The normalization constants and connection formulas are discussed in Dzieciol *et al.* [296], Gaspard [342], Michel [414] and chapter 33 in NIST [782]. In this implementation, we define the analytic continuations of all the functions so that the branch cut with respect to z is placed on the negative real axis.

30.2.1 Normalizing Gamow constant for Coulomb wave functions

`math53.coulomb_cl`(*l*, *eta*)

Returns the normalizing Gamow constant for Coulomb wave functions. See also Wikipedia [1368], MathWorld [1029], NIST [782], Thompson and Barnett [783], Michel [414], Ehrhardt [297] (3.1.10.1), Mpmath [717].

Gives the normalizing Gamow constant for Coulomb wave functions,

$$C_l(\eta) = 2^l \exp\left(-\pi\eta/2 + [\log\Gamma(1 + l + i\eta) + \log\Gamma(1 + l - i\eta)]/2 - \log\Gamma(2l + 2)\right),$$

where the log gamma function with continuous imaginary part away from the negative half axis is implied.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CoulombCL(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.CoulombCL(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; l = '10'; eta = '3.0'
>>> \mathrm{d}x = dec.coulombc(l, eta); mx = mpm.coulombc(l, eta); gx = gmp.coulombc(l, eta)
>>> fx = fpm.coulombc(l, eta); ax = apm.coulombc(l, eta)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.2816496261265795621632016101181570 25275E-13
mpm:  4.2816496261265795621632016101181570 25275e-13
gmp:  4.2816496261265795621632016101181570 25275E-13
fpm:  4.28164962612658E-13
apm:  4.2816496261265795621632016101181570 25382e-13 (1.141e-35%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; l = '10'; eta = '3 + 4j'
>>> \mathrm{d}z = dec.coulombc(l, eta); mz = mpm.coulombc(l, eta); gz = gmp.coulombc(l, eta)
>>> fz = fpm.coulombc(l, eta); az = apm.coulombc(l, eta)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 3.4717690236181999723E-13               - 8.0361659919956253403E-13j
mpm: 3.4717690236181999723e-13               - 8.0361659919956253403e-13j
gmp: 3.4717690236181999723E-13               - 8.0361659919956253403E-13j
fpm: 3.47176902361820E-13                    - 8.03616599199563E-13j
apm: 3.4717690236182000026e-13 (1.386e-15%) - 8.0361659919956253696e-13 (-9.468e-16%)j
```

30.2.2 Coulomb phase shift $\sigma_L(\eta)$

math53.**coulomb_sl**(*L*, *eta*)

Returns the Coulomb phase shift for Coulomb wave functions.

See also: Ehrhardt [297] (3.1.10.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CoulombSL(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.CoulombSL(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

30.2.3 Regular Coulomb wave functions $F_l(\eta, x), F_l'(\eta, x)$

math53.**coulomb_f_fprime**(*L*, *eta*, *x*)

Returns the Coulomb wave function F. See also Wikipedia [1368], MathWorld [1029], NIST [782], Ehrhardt [297] (3.1.10.3), Flint [771], Flint [764], Mpmath [714].

Calculates the regular Coulomb wave function

$$F_l(\eta, z) = C_l(\eta) z^{l+1} e^{-iz} \, {}_1F_1(l + 1 - i\eta, 2l + 2, 2iz)$$

Calculates the regular Coulomb wave function

$$F_l(\eta, z) = C_l(\eta) z^{l+1} e^{-iz} \, {}_1F_1(l + 1 - i\eta, 2l + 2, 2iz)$$

where the normalization constant $C_l(\eta)$ is as calculated by *coulombc()*. This function solves the differential equation

$$f''(z) + \left(1 - \frac{2\eta}{z} - \frac{l(l+1)}{z^2}\right) f(z) = 0.$$

A second linearly independent solution is given by the irregular Coulomb wave function $G_l(\eta, z)$ (see *coulombg()*) and thus the general solution is $f(z) = C_1 F_l(\eta, z) + C_2 G_l(\eta, z)$ for arbitrary constants C_1, C_2.

Physically, the Coulomb wave functions give the radial solution to the Schrodinger equation for a point particle in a $1/z$ potential; z is then the radius and l, η are quantum numbers.

The Coulomb wave functions with real parameters are defined in Abramowitz & Stegun, section 14. However, all parameters are permitted to be complex in this implementation (see references).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CoulombFFp(3, 0.5, 2.25)
ereal('5.2359877559829887307E-1')
>>> math53.CoulombFFp(3, 0.5, 8.25)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; l = '10'; eta = '3.0'; x = '5.0'
>>> \mathrm{d}x = dec.coulombf(l, eta, x); mx = mpm.coulombf(l, eta, x); gx = gmp.coulombf(l, eta, x)
>>> fx = fpm.coulombf(l, eta, x); ax = apm.coulombf(l, eta, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.6969378111954143710067591411013518594 62E-5
mpm:  4.6969378111954143710067591411013518594 62e-5
gmp:  4.6969378111954143710067591411013518594 62E-05
fpm:  4.69693781119541E-05
apm:  4.6969378111954143710067591411013518594 62e-5 (2.238e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; l = '10'; eta = '3 + 4j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.coulombf(l, eta, z); mz = mpm.coulombf(l, eta, z); gz = gmp.coulombf(l, eta, z)
>>> fz = fpm.coulombf(l, eta, z); az = apm.coulombf(l, eta, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 2.0437700551051474840E-4              + 1.0838222727919637420E-4j
mpm: 2.0437700551051474840e-4              + 1.0838222727919637420e-4j
gmp: 2.0437700551051474840E-04             + 1.0838222727919637420E-04j
fpm: 2.04377005510515E-04                  + 1.08382227279196E-04j
apm: 2.0437700551051474840e-4 (5.059e-19%) + 1.0838222727919637420e-4 (9.063e-19%)j
```

30.2.4 Irregular Coulomb wave functions $G_l(\eta, x), G_l'(\eta, x)$

`math53.coulomb_g`(*L*, *eta*, *x*)

where `ctx` is `math53`` or `ctxflint`.

Returns the irregular Coulomb wave function. See also Wikipedia [1368], MathWorld [1029], NIST [782], Ehrhardt [297] (3.1.10.5), Mpmath [715].

Calculates the irregular Coulomb wave function

$$G_l(\eta, z) = \frac{F_l(\eta, z)\cos(\chi) - F_{-l-1}(\eta, z)}{\sin(\chi)}$$

where $\chi = \sigma_l - \sigma_{-l-1} - (l + 1/2)\pi$ and $\sigma_l(\eta) = (\log\Gamma(1 + l + i\eta) - \log\Gamma(1 + l - i\eta))/(2i)$.

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.CoulombGGp(3, 0.5, 2.25)
ereal('5.235987755982988730 7E-1')
>>> math53.CoulombGGp(3, 0.5, 8.25)
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; l = '10'; eta = '3.0'; x = '5.0'
>>> \mathrm{d}x = dec.coulombg(l, eta, x); mx = mpm.coulombg(l, eta, x); gx = gmp.coulombg(l, eta, x)
>>> fx = fpm.coulombg(l, eta, x); ax = apm.coulombg(l, eta, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  4.96154211405736501074155753769368168181 0E+3
mpm:  4.96154211405736501074155753769368168181 0e+3
gmp:  4.96154211405736501074155753769368168181 0E+03
fpm:  4.96154211405737E+03
apm:  4.96154211405736501074155753769368168181 0e+3 (3.791e-39%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; l = '10'; eta = '3 + 4j'; z = '5.0 + 3j'
>>> \mathrm{d}z = dec.coulombg(l, eta, z); mz = mpm.coulombg(l, eta, z); gz = gmp.coulombg(l, eta, z)
>>> fz = fpm.coulombg(l, eta, z); az = apm.coulombg(l, eta, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az],  aligned=True)
dec: 1.232320123753354 7519E+3              - 1.026967516056281 3715E+2j
mpm: 1.232320123753354 7519e+3              - 1.026967516056281 3715e+2j
gmp: 1.232320123753354 7519E+03             - 1.026967516056281 3715E+02j
fpm: 1.23232012375335E+03                   - 1.02696751605628E+02j
apm: 1.232320123753354 7519e+3 (4.223e-19%) - 1.026967516056281 3715e+2 (-3.537e-18%)j
```

30.3 Error function, and related functions

30.3.1 Exponentially scaled complementary error function, $\mathrm{erfcx}(x)$

`math53.`**`erfcx`**(*x*)

Returns the exponentially scaled complementary error function $\mathrm{erfcx}(z) = \exp(z^2) \cdot \mathrm{erfc}(z) = w(iz)$. See also Ehrhardt [297] (3.3.6).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Erfce(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Erfce('0.51')
ereal('5.351847902755998475 4E-1')
```

30.3.2 Difference of error functions, $\mathrm{erfh}(x, h) = \mathrm{erf}(x + h) - \mathrm{erf}(x - h)$

`math53.`**`erfh`**(*x*, *h*)

Returns the difference of error functions $\mathrm{erfh}(x, h) = \mathrm{erf}(x + h) - \mathrm{erf}(x - h) = \mathrm{erfc}(x - h) - \mathrm{erfc}(x + h)$.

See also Ehrhardt [297] (3.3.9).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Erfh(0.5, 0.6)
ereal('5.235987755982988730 7E-1')
>>> math53.Erfh('0.51', 0.61)
ereal('5.351847902755998475 4E-1')
```

30.3.3 Difference of error functions, $\mathrm{erf2}(x_1, x_2) = \mathrm{erf}(x_2) - \mathrm{erf}(x_1)$

`math53.`**`erf2`**(*x1*, *x2*)

Returns the difference of error functions $\mathrm{erf2}(x_1, x_2) = \mathrm{erf}(x_2) - \mathrm{erf}(x_1) = \mathrm{erfh}\left(\frac{1}{2}(x_2 + x_1), \frac{1}{2}(x_2 - x_1)\right)$.

See also Ehrhardt [297] (3.3.10).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Erf2(0.5, 0.6)
ereal('5.235987755982988730 7E-1')
>>> math53.Erf2('0.51', 0.61)
ereal('5.351847902755998475 4E-1')
```

30.3.4 Probability function $Q(x) = \Phi(-x)$

`math53.`**`erfq`**(*x*)

Returns the integral $Q(x) = \Phi(-x) = \dfrac{1}{\sqrt{2\pi}} \displaystyle\int_x^\infty \exp(-t^2)\,\mathrm{d}t = P(-x)$.

See also: Ehrhardt [297] (3.3.12.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ErfQ(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.ErfQ('0.51')
ereal('5.351847902755998475 4E-1')
```

30.3.5 Inverse of the exponentially scaled complementary error function, $\mathrm{erfcx}^{-1}(x)$

`math53.`**`erfcx_inv`**(x)

Returns $\mathrm{erfcx}^{-1}(x)$, the functional inverse of erfcx, satisfying $\mathrm{erfcx}(\mathrm{erfcx}^{-1}(x)) = x$, for $0 \leq x \leq \infty$.

See also Ehrhardt [297] (3.3.11.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ErfceInv(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.ErfceInv('0.51')
ereal('5.351847902755998475 4E-1')
```

30.3.6 Inverse of the imaginary error function, $\mathrm{erfi}^{-1}(x)$

`math53.`**`erfi_inv`**(x)

Returns $\mathrm{erfi}^{-1}(x)$, the functional inverse of erfi, satisfying $\mathrm{erfi}(\mathrm{erfi}^{-1}(x)) = x$.

See also Ehrhardt [297] (3.3.11.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ErfiInv(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.ErfiInv('0.51')
ereal('5.351847902755998475 4E-1')
```

30.3.7 Generalized Dawson integral, $F(p, x)$

`math53.`**`dawson2`**(p, x)

Returns the generalized Dawson integral $F(p, x) = e^{-x^p} \int_0^x e^{t^p} \mathrm{d}t, p \geq 0, x \geq 0$. See also Ehrhardt [297] (3.3.2).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Dawson2(1.5, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Dawson2(1.5, '0.51')
ereal('5.351847902755998475 4E-1')
```

30.3.8 Generalized error function, $\mathrm{erfg}(p, x)$

`math53.`**`erfg`**(p, x)

Returns the generalized error function $\mathrm{erfg}(p, x) = \int_0^x e^{-t^p} \mathrm{d}t = \frac{1}{p} \gamma \left(\frac{1}{p}, x^p \right)$, where $\gamma(\cdot)$ denotes the non-normalised lower incomplete gamma function. See also Ehrhardt [297] (3.3.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Erfg(1.5, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Erfg(1.5, '0.51')
ereal('5.351847902755998475 4E-1')
```

30.3.9 Expint3, $\mathrm{erfg}(3, x)$

`math53.`**`expint3`**(x)

Returns $\mathrm{expint3}(p, x) = \mathrm{erfg}(3, x) = \int_0^x e^{-t^3}\,\mathrm{d}t = \frac{1}{3}\gamma\left(\frac{1}{3}, x^3\right)$, where $\gamma(\cdot)$ denotes the non-normalised lower incomplete gamma function. See also Ehrhardt [297] (3.3.13).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Expint3(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.Expint3('0.51')
ereal('5.351847902755998475 4E-1')
```

30.3.10 Scaled repeated integrals of the complementary error function, $i^n\mathrm{erfc}(x)$

`math53.`**`inerfc`**(n, x)

Returns the scaled repeated integrals of the complementary error function, defined for $n \geq -1$ using the awkward but standard notation

$$i^n\mathrm{erfc}(x) = \int_x^\infty \mathrm{erfc}(t)\,\mathrm{d}t = \frac{2}{\sqrt{\pi}}\int_x^\infty \frac{(t-x)^n}{n!}e^{-t^2}\,\mathrm{d}t, \quad (n = 0, 1, 2, ...).$$

See also Ehrhardt [297] (3.3.7), NIST [437] (eq. 7.18.10), MathWorld [977].

These functions compute the scaled repeated integrals of complementary error function, defined for $n \geq -1$ using the awkward but standard notation

$$i^n\mathrm{erfc}(x) = \int_x^\infty i^{n-1}\mathrm{erfc}(t)\mathrm{d}t = \frac{2}{\sqrt{\pi}}\int_x^\infty \frac{(t-x)^n}{n!}e^{-t^2}\,\mathrm{d}t, \quad (n = 0, 1, 2, \ldots)$$

$$i^{-1}\mathrm{erfc}(x) = \frac{2}{\sqrt{\pi}}e^{-x^2}, \quad i^0\mathrm{erfc}(x) = \mathrm{erfc}(x).$$

$$i^n\mathrm{erfc}(x) = -\frac{z}{n}i^{n-1}\mathrm{erfc}(x) + \frac{1}{2n}i^{n-2}\mathrm{erfc}(x.$$

We also have

$$i^n\mathrm{erfc}(x) = \frac{e^{-z^2}}{2^n\sqrt{\pi}}U\left(\tfrac{1}{2}n + \tfrac{1}{2}, \tfrac{1}{2}, z^2\right)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.InErfc(3, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.InErfc(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

An example with real input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; n = '4'; x = '5.0'
>>> \mathrm{d}x = dec.inerfc(n, x); mx = mpm.inerfc(n, x); gx = gmp.inerfc(n, x)
>>> fx = fpm.inerfc(n, x); ax = apm.inerfc(n, x)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  8.631401306269731883125812071651943647995E-6
mpm:  8.631401306269731883125812071651943647995e-6
gmp:  8.631401306269731883125812071651943647995E-06
fpm:  8.63140130626973E-06
apm:  8.631401306269731883125812071651943657349e-6 (1.945e-32%)
```

An example with complex input:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; n = '4'; z = '5.0 + 3.0j'
>>> \mathrm{d}z = dec.inerfc(n, z); mz = mpm.inerfc(n, z); gz = gmp.inerfc(n, z)
>>> fz = fpm.inerfc(n, z); az = apm.inerfc(n, z)
>>> mpm.show([\mathrm{d}z, mz, gz, fz, az], aligned=True)
dec: -3.8136067248438755019E-6                - 2.7044284620493310438E-6j
mpm: -3.8136067248438755019e-6                - 2.7044284620493310438e-6j
gmp: -3.8136067248438755019E-06               - 2.7044284620493310438E-06j
fpm: -3.81360672484388E-06                    - 2.70442846204933E-06j
apm: -3.8136067248438754005e-6 (-2.848e-13%) - 2.7044284620493310996e-6 (-3.025e-13%)j
```

30.3.11 Fresnel auxiliary function $\mathrm{f}(x)$

`math53.`**`fresnel_f`**`(`x`)`

Returns the Fresnel auxiliary function $\mathrm{f}\left(x\right)=\left(\frac{1}{2}-S\left(x\right)\right)\cos\left(\frac{1}{2}\pi x^{2}\right)-\left(\frac{1}{2}-C\left(x\right)\right)\sin\left(\frac{1}{2}\pi x^{2}\right)$.

See also Wikipedia [1323], MathWorld [980], NIST [777], Ehrhardt [297] (3.3.15).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.FresnelF(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.FresnelF('0.51')
ereal('5.351847902755998475 4E-1')
```

30.3.12 Fresnel auxiliary function $\mathrm{g}(x)$

`math53.`**`fresnel_g`**`(`x`)`

Returns the Fresnel auxiliary function $\mathrm{g}\left(x\right)=\left(\frac{1}{2}-C\left(x\right)\right)\cos\left(\frac{1}{2}\pi x^{2}\right)+\left(\frac{1}{2}-S\left(x\right)\right)\sin\left(\frac{1}{2}\pi x^{2}\right)$.

See also Wikipedia [1323], MathWorld [980], NIST [777], Ehrhardt [297] (3.3.15).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.FresnelG(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.FresnelG('0.51')
ereal('5.351847902755998475 4E-1')
```

30.3.13 Goodwin-Staton integral, $G(x)$

`math53.`**`goodwin_staton`**`(`x`)`

Returns the Goodwin-Staton integral $G(x)=\int_{0}^{\infty}\frac{e^{-t^{2}}}{t+x}\,\mathrm{d}t=\sqrt{\pi}F(x)-\frac{1}{2}e^{-x^{2}}\mathrm{Ei}(x^{2}),\quad x\neq 0.$

See also Wikipedia [1324], NIST [436], Ehrhardt [297] (3.3.16).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.GoodwinStaton(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.GoodwinStaton('0.51')
ereal('5.3518479027559984754E-1')
```

30.3.14 Voigt function U

math53.**voigt_u**(z)

Returns the Voigt function U. See also Wikipedia [1337], NIST [439].

The Voigt functions[1] U, V, and H (sometimes called the line broadening function) are defined by

$$U(x,t) + iV(x,t) = \sqrt{\frac{\pi}{4t}} e^{z^2} \operatorname{erfc}(z) = \sqrt{\frac{\pi}{4t}} w(iz),$$

$$H(a,u) = \frac{U(u/a, 1/4a^2)}{a\sqrt{\pi}},$$

where

$$z = (1 - ix)/2\sqrt{t},$$

erfc is the complementary error function, and w(z) is the Faddeeva function.

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.VoigtU(0.5)
ecplx('5.2359877559829887307E-1')
>>> ecplx.VoigtU('0.1')
ecplx('5.3518479027559984754E-1')
```

30.3.15 Voigt function V

math53.**voigt_v**(x, t)

Returns the Voigt function V. See also Wikipedia [1337], NIST [439].

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.VoigtV(0.5)
ecplx('5.2359877559829887307E-1')
>>> ecplx.VoigtV('0.1')
ecplx('5.3518479027559984754E-1')
```

30.3.16 Voigt function H

math53.**voigt_h**(x, t)

Returns the Voigt function H. See also Wikipedia [1337], NIST [439].

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.VoigtH(0.5)
ecplx('5.2359877559829887307E-1')
>>> ecplx.VoigtH('0.1')
ecplx('5.3518479027559984754E-1')
```

30.3.17 Voigt Profile distribution, pdf

math53.**voigt_profile_pdf**(*x*, *sigma*, *gamma*)

Returns pdf(x), the probability density function of a random variable X, following a Voigt Profile distribution with shape $sigma > 0$, scale $gamma > 0$, and the support interval $(0, +\infty)$. See also Wikipedia [1220].

$$\text{pdf}(x) = V(x; \sigma, \gamma) = \frac{\Re[w(z)]}{\sigma\sqrt{2\pi}}, \quad \text{where } z = \frac{x + i\gamma}{\sigma\sqrt{\pi}}.$$

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.VoigtProfilePdf(0.5)
ecplx('5.2359877559829887307E-1')
>>> ecplx.VoigtProfilePdf('0.1')
ecplx('5.351847902755998475 4E-1')
```

30.4 Exponential integrals, and related functions

30.4.1 Entire cosine integral $\mathrm{Cin}(x)$

`math53.cin(z)`

Returns the entire cosine integral of $\mathrm{Cin}(x) = \int_0^x \frac{1-\cos(t)}{t}\,\mathrm{d}t$.

See also Wikipedia [1320], MathWorld [972], NIST [781], Ehrhardt [297] (3.4.3).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Cin(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Cin('0.51')
ereal('5.3518479027559984754E-1')
```

30.4.2 Entire hyperbolic cosine integral $\mathrm{Cinh}(x)$

`math53.cinh(z)`

Returns the entire hyperbolic cosine integral $\mathrm{Cinh}(x) = \int_0^x \frac{\cosh(t)-1}{t}\,\mathrm{d}t$.

See also Wikipedia [1326], MathWorld [984], NIST [778], Ehrhardt [297] (3.4.4).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Cinh(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Cinh('0.51')
ereal('5.3518479027559984754E-1')
```

30.4.3 Entire exponential integral $\mathrm{Ein}(x)$

`math53.ein(x)`

Returns the entire exponential integral $\mathrm{Ein}(x) = -\int_0^x \frac{1-e^{-t}}{t}\,\mathrm{d}t$.

See also Wikipedia [1326], MathWorld [984], NIST [778], Ehrhardt [297] (3.4.11).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Ein(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Ein('0.51')
ereal('5.3518479027559984754E-1')
```

30.4.4 Scaled exponential integral $\mathrm{e1s}(x)$

`math53.exp_integral_e1_scaled(z)`

Returns the scaled exponential integral $\mathrm{e1s}(x) = e^{-x}E_1(x)$ for $x \neq 0$.

See also Wikipedia [1321], MathWorld [978], NIST [776], BoostMath [154], Ehrhardt [297] (3.4.6).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.E1s(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.E1s('0.51')
ereal('5.3518479027559984754E-1')
```

30.4.5 Scaled exponential integral $\mathrm{eis}(x)$

`math53.exp_integral_ei_scaled(`x`)`

Returns the scaled exponential integral $\mathrm{eis}(x) = e^{-x}\mathrm{Ei}(x)$ for $x \neq 0$.

See also Wikipedia [1321], MathWorld [978], NIST [776], BoostMath [154], Ehrhardt [297] (3.4.8).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Eis(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Eis('0.51')
ereal('5.3518479027559984754E-1')
```

30.4.6 Scaled exponential integral $\mathrm{eisx2}(x)$

`math53.eisx2(`x`)`

Returns the scaled exponential integral $\mathrm{eisx2}(x) = e^{-x^2}\mathrm{Ei}(x^2)$ for $x \neq 0$.

See also Wikipedia [1321], MathWorld [978], NIST [776], BoostMath [154], Ehrhardt [297] (3.4.9).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Eisx2(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Eisx2('0.51')
ereal('5.3518479027559984754E-1')
```

30.4.7 Inverse of the exponential integral, $\mathrm{Ei}^{-1}(x)$

`math53.ei_inv(`x`)`

Returns $\mathrm{Ei}^{-1}(x) = \log(\mathrm{li}^{-1}(x))$, the functional inverse of the exponential integral, i.e. $\mathrm{Ei}(\mathrm{Ei}^{-1}(x)) = x$.

See also Wikipedia [1321], MathWorld [978], NIST [776], BoostMath [154], Ehrhardt [297] (3.4.10).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.EiInv(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.EiInv('0.51')
ereal('5.3518479027559984754E-1')
```

30.4.8 Generalized exponential integral $E_p(p, x)$

`math53.gei(`p, x`)`

Returns the generalized exponential integral of real order $E_p(x) = \int_1^\infty \frac{e^{-xt}}{t^p}\,\mathrm{d}t = x^{p-1}\Gamma(1-p,x) = x^{p-1}Q(1-p,x)\Gamma(1-p)$.

See also Wikipedia [1321], MathWorld [979], NIST [462], BoostMath [155], Ehrhardt [297] (3.4.13).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Gei(3.1, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Gei(3.1, '0.51')
ereal('5.351847902755998475 4E-1')
```

30.4.9 Exponential integral $\beta_n(x)$

`math53.eibeta`(n, x)

Returns the exponential integral $\beta_n(x) = \int_1^1 t^n e^{-xt}\,\mathrm{d}t = x^{-n-1}\left(\Gamma(n+1,-x) - \Gamma(n+1,x)\right), n \geq 0$.

See also Abramowitz and Stegun. [4] (5.1.6) and (5.1.47), and Ehrhardt [297] (3.4.14).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Eibeta(3, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Eibeta(3, '0.51')
ereal('5.351847902755998475 4E-1')
```

30.4.10 Inverse of the logarithmic integral, $\mathrm{li}^{-1}(x)$

`math53.log_integral_inv`(x)

Returns $\mathrm{li}^{-1}(x)$, the functional inverse of the logarithmic integral, i.e. $\mathrm{li}(\mathrm{li}^{-1}(x)) = x$.

See also Wikipedia [1330], MathWorld [989], NIST [779], Ehrhardt [297] (3.4.16).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.LiInv(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.LiInv('0.51')
ereal('5.351847902755998475 4E-1')
```

30.4.11 Shifted sine integral $\mathrm{si}(x) = \mathrm{Si}(x) - \pi/2$

`math53.shifted_sin_integral`(x)

Returns the shifted sine integral, $\mathrm{si}(x) = \mathrm{Si}(x) - \pi/2$.

See also Wikipedia [1335], MathWorld [991], NIST [781], Ehrhardt [297] (3.4.19).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.ShiftedSi(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.ShiftedSi('0.51')
ereal('5.351847902755998475 4E-1')
```

30.5 Struve functions

30.5.1 Struve function $\mathrm{H}_0(x)$

math53.**struve_h0**(x)

Returns the Struve function $H_0(x)$

See also: Ehrhardt [297] (3.1.9.1), Mpmath [712].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.StruveH0(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.StruveH0('0.51')
ereal('5.351847902755998475 4E-1')
```

30.5.2 Struve function $\mathrm{H}_1(x)$

math53.**struve_h1**(x)

Returns the Struve function $H_1(x)$

See also: Ehrhardt [297] (3.1.9.2), Mpmath [712].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.StruveH1(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.StruveH1('0.51')
ereal('5.351847902755998475 4E-1')
```

30.5.3 Struve function $\mathrm{L}_0(x)$

math53.**struve_l0**(x)

Returns the Struve function $L_0(x)$

See also: Ehrhardt [297] (3.1.9.5).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.StruveL0(0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.StruveL0('0.51')
ereal('5.351847902755998475 4E-1')
```

30.5.4 Struve function $\mathrm{L}_1(x)$

math53.**struve_l1**(x)

Returns the Struve function $\mathbf{L}_1(x)$

See also: Ehrhardt [297] (3.1.9.6).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.StruveL1(0.5)
ereal('5.235987755982988730 7E-1')
```

(continues on next page)

(continued from previous page)

```
>>> math53.StruveL1('0.51')
ereal('5.351847902755998475 4E-1')
```

CHAPTER

THIRTYONE

REAL AND COMPLEX FUNCTIONS

31.1 Additional elementary complex functions

31.1.1 Auxiliary function $\sqrt{1-z^2}$

`math53.`**`sqrt1mz2`**`(`z`)`

Returns the square root of $1-z^2$. For very large z (when z^2 would overflow) we have $\sqrt{1-z^2} = \pm iz$, where the sign is chosen to make $\Re\sqrt{1-z^2} \geq 0$; otherwise the result is computed from the definition.

See also Wikipedia [1233], MathWorld [842], BoostMath [97].

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Cuberoot(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Cuberoot('0.1')
ecplx('5.351847902755998475 4E-1')
```

31.1.2 Cube root, $\mathrm{cuberoot}(x) = \sqrt[3]{x} = y$, with $\arg(y)$ closest to $\arg(x)$

`math53.`**`cuberoot`**`(`z`)`

Returns the cube root of x, $x^{1/3}$ in a way which gives a negative number for negative input. See also Wikipedia [1233], MathWorld [842], BoostMath [97].

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Cuberoot(0.5)
ecplx('5.235987755982988730 7E-1')
>>> ecplx.Cuberoot('0.1')
ecplx('5.351847902755998475 4E-1')
```

31.1.3 Nth root, $\mathrm{surd}(x,n) = \sqrt[n]{x} = y$, with $\arg(y)$ closest to $\arg(x)$

`mathc53.`**`surd`**`(`x`, `n`)`

Returns the complex nth root $w = z^{1/n}$ with arg(w) closest to arg(z), e.g. surd(-8, 3) = -2 or surd(i, 5) = i, compared to the cnroot results $\sqrt[3]{-8} = 1 + i\sqrt{3}$ and $\sqrt[5]{i} = \cos(\pi/10) + i\sin(\pi/10)$. See Ehrhardt [297] (4.2.60).

An example in Python

```
>>> from xlcalcnet import ecplx
>>> ecplx.Surd(0.5)
ecplx('5.235987755982988730 7E-1')
```

(continues on next page)

(continued from previous page)

```
>>> ecplx.Surd('0.1')
ecplx('5.351847902755998475 4E-1')
```

31.1.4 Continuous inverse cotangent, $\mathrm{acotc}(x)$

`ctx.acotc`(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the continuous inverse cotangent of x, $\mathrm{acotc}(x) = \pi/2 - \mathrm{atan}(x)$. See also Wikipedia [1260], MathWorld [869], NIST [487], Mpmath [569].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Acotc(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Acotc('0.51')
ereal('5.351847902755998475 4E-1')
```

31.1.5 Continuous inverse hyperbolic cotangent, $\mathrm{acothc}(x)$

`ctx.acothc`(x)

where `ctx` is `math53`, `mathc53`, `ctxcpp` or `ctxflint`.

Returns the continuous inverse hyperbolic cotangent of x, $\mathrm{acothc}(x) = \pi/2 - \mathrm{atan}(x)$. See also Wikipedia [1260], MathWorld [869], NIST [487], Mpmath [569].

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Acotc(0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Acotc('0.51')
ereal('5.351847902755998475 4E-1')
```

31.2 Weierstrass elliptic functions, in terms of (real) lattice invariants g_2, g_3

The Weierstrass functions take real values on the real axis iff the lattice is fixed under complex conjugation, or, equivalently, when $g_2, g_3 \in \mathbb{R}$.

31.2.1 Weierstrass function $\wp_g(z, g_2, g_3)$

`ctx.weierstrass_p_g(z, g2, g3)`

Computes Weierstrass's elliptic function $\wp_g(z; g_2, g_3)$.

We have $\wp_g(tz; t^{-4}g_2, t^{-6}g_3) = t^{-2}\wp_g(z; g_2, g_3)$ and $\wp_g(iz; g_2, g_3) = -\wp_g(z; g_2, -g_3)$.

See also MathWorld [992], Flint [770].

Returns the Weierstrass function $\wp_g(x, g_2, g_3)$ based on the lattice invariants g_2, g_3.

In AMath the computation of the Weierstass elliptic function (for real and imaginary arguments) is based on the lattice invariants g_2, g_3 or the lattice roots $e_1, e_2, e_3 = -e_1 - e_2$, if the discriminant of the cubic equation $4x^3 - g_2x - g_3 = 0$ is positive

$$\Delta = g_2^3 - 27g_3^2 = 16(e_2 - e_3)^2(e_3 - e_1)^2(e_1 - e_2)^2;$$

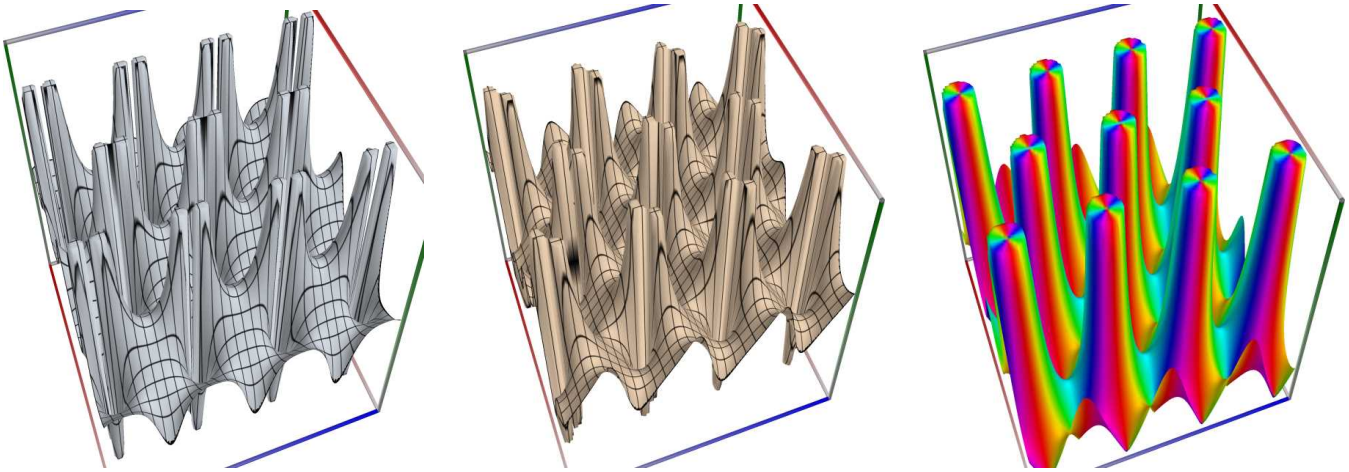
symbolically written as $\wp_g(x, g_2, g_3)$ or $\wp_e(x, e_1, e_2)$. See also

When $g_2 = 1, g_3 = 0$ the result is $\wp_l(x)$ and for $g_2 = g_3 = 0$ it is $x - 2$. If $g_2^2 - 27g_3^2 > 0$ all lattice roots are real, and $\wp'_e(x, e_1, e_2)$ is returned, otherwise the following Jacobi relation is used:

$$\wp_g(x, g_2, g_3) = e_2 + H\frac{1 + \mathrm{cn}(u, k)}{1 - \mathrm{cn}(u, k)}, \quad k = \sqrt{\tfrac{1}{2}\frac{3e_2}{4H}}, \quad u = 2x\sqrt{H}, \quad H = \sqrt{(e_2 - e_1)(e_2 - e_3)}.$$

See also Wikipedia [1355], MathWorld [992], Ehrhardt [297] (3.2.17.5).

Returns the Weierstrass function $\wp_g(iy, g_2, g_3) = -\wp_g(y, g_2, -g_3)$. See also: MathWorld [993], Ehrhardt [297] (3.2.17.7).



Left figure: real part of the Weierstrass function $\wp_g(z, g_2, g_3)$.

Middle figure: imaginary part of the Weierstrass function $\wp_g(z, g_2, g_3)$.

Right figure: absolute value of the Weierstrass function $\wp_g(z, g_2, g_3)$, with color-coded phase.

For $\Delta = g_2^3 - 27g_3^2 \neq 0$, we first compute the elliptic half periods ω_1, ω_2 from the lattice invariants g_2, g_3 , and set $\omega = \omega_1, \tau = \omega_2/\omega_1$. Then $\wp_g(z; g_2, g_3) = \wp(z|\omega, \tau)$.

For $\Delta = 0, g_2 > 0, g_3 \neq 0$, the function $\wp_g(\cdot)$ degenerates to a simply periodic function, which can be expressed in closed form in terms of elementary functions:

For $\Delta = 0, g_2 > 0, g_3 < 0$: $\wp_g(z; g_2, g_3) = c + \dfrac{3c}{\sinh^2\left(\sqrt{3c}z\right)}$, where $c = \sqrt{g_2/12}$.

For $\Delta = 0, g_2 > 0, g_3 > 0$: $\wp_g(z; g_2, g_3) = c + \dfrac{3c}{\sin^2\left(\sqrt{3c}z\right)}$, where $c = \sqrt{g_2/12}$.

If both g_2 and g_3 are zero, the function $\wp_g(\cdot)$ degenerates to a function that is not periodic at all, namely $\wp_g(z; 0, 0) = z^{-2}$.

An example with real input for z and with purely imaginary input for τ, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; z = '0.1'; tau = '0.0 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p(z, tau); mx = mpm.weierstrass_p(z, tau)
>>> gx = gmp.weierstrass_p(z, tau)
>>> fx = fpm.weierstrass_p(z, tau); ax = apm.weierstrass_p(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.0012029137285933516203337364613247608 25E+2
mpm:  1.0012029137285933516203337364613247608 25e+2
gmp:  1.0012029137285933516203337364613247608 25E+02
fpm:  1.00120291372859E+02
apm:  1.0012029137285933516203337364613247608 25e+2 (1.321e-38%)
```

An example with complex input for z and τ:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '0.1 + 0.5j'; tau = '0.7 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p(z, tau); mx = mpm.weierstrass_p(z, tau)
>>> gx = gmp.weierstrass_p(z, tau)
>>> fx = fpm.weierstrass_p(z, tau); ax = apm.weierstrass_p(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec:  -2.3859027427781674104E+0
mpm:  -2.3859027427781674104e+0
gmp:  -2.3859027427781674104E+00
fpm:  -2.38590274277817E+00
apm:  (-2.3859027427781674104e+0 (-2.038e-17%) + 2.4323666240267878020e-2 (1.982e-15%)j)
```

31.2.2 Weierstrass function, first derivative: $\wp'_g(z, g_2, g_3)$

`ctx.weierstrass_p_prime_(z, g2, g3)`

Computes the first derivative of the Weierstrass function, $\wp'_g(z, g_2, g_3)$.

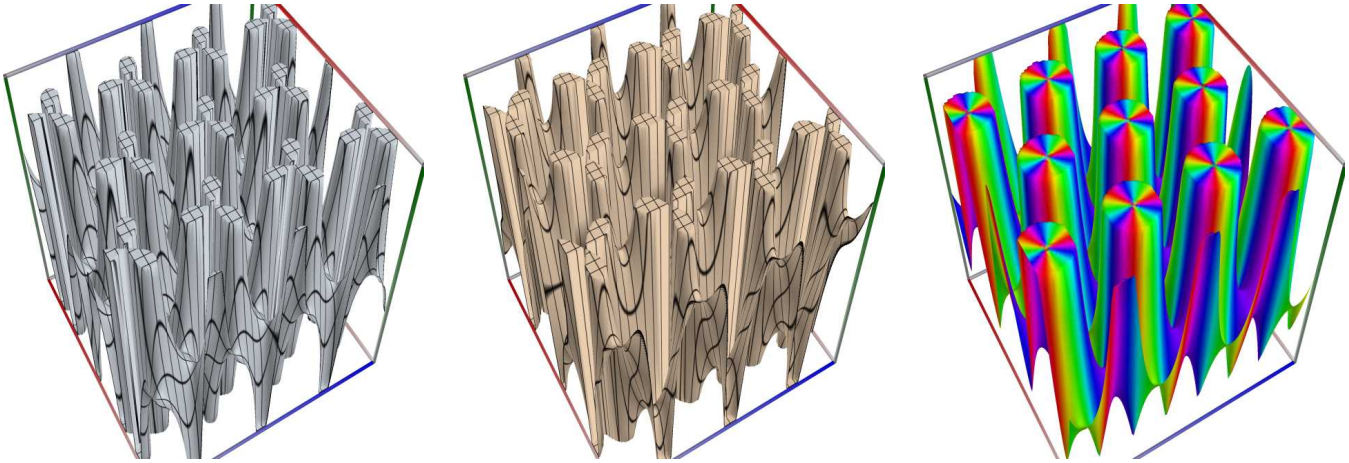
We have $\wp'_g(tz; t^{-4}g_2, t^{-6}g_3) = t^{-3}\wp'_g(z; g_2, g_3)$ and $\wp'_g(iz; g_2, g_3) = i\wp'_g(z; g_2, -g_3)$.

See also MathWorld [993], Flint [770].

Returns the Weierstrass function $\wp_g(x, g_2, g_3)$ based on the lattice invariants g_2, g_3.

$$\wp'_g(x, g_2, g_3) = -H^{3/2}\frac{\mathrm{cn}(u,k)\mathrm{dn}(u,k)}{1-\mathrm{cn}(u,k)}, \quad k = \sqrt{\frac{1}{2}\frac{3e_2}{4H}}, \quad u = 2x\sqrt{H}, \quad H = \sqrt{(e_2-e_1)(e_2-e_3)}.$$

See also: MathWorld [993], Ehrhardt [297] (3.2.17.6).



Left figure: real part of the Weierstrass function, first derivative: $\wp'_g(z, g_2, g_3)$.

Middle figure: imaginary part of the Weierstrass function, first derivative: $\wp'_g(z, g_2, g_3)$.

Right figure: absolute value of the Weierstrass function, first derivative: $\wp'_g(z, g_2, g_3)$, with color-coded phase.

For $\Delta = g_2^3 - 27g_3^2 \neq 0$, we first compute the elliptic half periods ω_1, ω_2 from the lattice invariants g_2, g_3, and set $\omega = \omega_1, \tau = \omega_2/\omega_1$. Then $\wp'_g(z; g_2, g_3) = \wp'(z|\omega, \tau)$.

For $\Delta = 0, g_2 > 0, g_3 \neq 0$, the function $\wp'_g(\cdot)$ degenerates to a simply periodic function, which can be expressed in closed form in terms of elementary functions:

For $\Delta = 0, g_2 > 0, g_3 < 0$: $\wp'_g(z; g_2, g_3) = -6\sqrt{3}c^{3/2}\coth(\sqrt{3c}z)\mathrm{csch}^2(\sqrt{3c}z)$, where $c = \sqrt{g_2/12}$.

For $\Delta = 0, g_2 > 0, g_3 > 0$: $\wp'_g(z; g_2, g_3) = -6\sqrt{3}c^{3/2}\cot(\sqrt{3c}z)\csc^2(\sqrt{3c}z)$, where $c = \sqrt{g_2/12}$.

If both g_2 and g_3 are zero, the function $\wp'_g(\cdot)$ degenerates to a function that is not periodic at all, namely $\wp'_g(z; 0, 0) = -2z^{-3}$.

An example with real input for z and with purely imaginary input for τ, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; z = '0.1'; tau = '0.0 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p_prime(z, tau); mx = mpm.weierstrass_p_prime(z, tau)
>>> gx = gmp.weierstrass_p_prime(z, tau)
>>> fx = fpm.weierstrass_p_prime(z, tau); ax = apm.weierstrass_p_prime(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  -1.997612036772329460780279347633189585828E+3
mpm:  -1.997612036772329460780279347633189585828e+3
gmp:  -1.997612036772329460780279347633189585828E+03
fpm:  -1.99761203677233E+03
apm:  -1.997612036772329460780279347633189585828e+3 (-1.883e-38%)
```

An example with complex input for z and τ:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '0.1 + 0.5j'; tau = '0.7 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p_prime(z, tau); mx = mpm.weierstrass_p_prime(z, tau)
>>> gx = gmp.weierstrass_p_prime(z, tau)
>>> fx = fpm.weierstrass_p_prime(z, tau); ax = apm.weierstrass_p_prime(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 1.4787544521497118400E+1
mpm: 1.4787544521497118400e+1
gmp: 1.4787544521497118400E+01
fpm: 1.47875445214971E+01
apm: 1.4787544521497118399e+1 (5.664e-17%) - 2.3205214399360215619e+1 (-3.609e-17%)j
```

31.2.3 Weierstrass Zeta function $\zeta_g(z, g_2, g_3)$

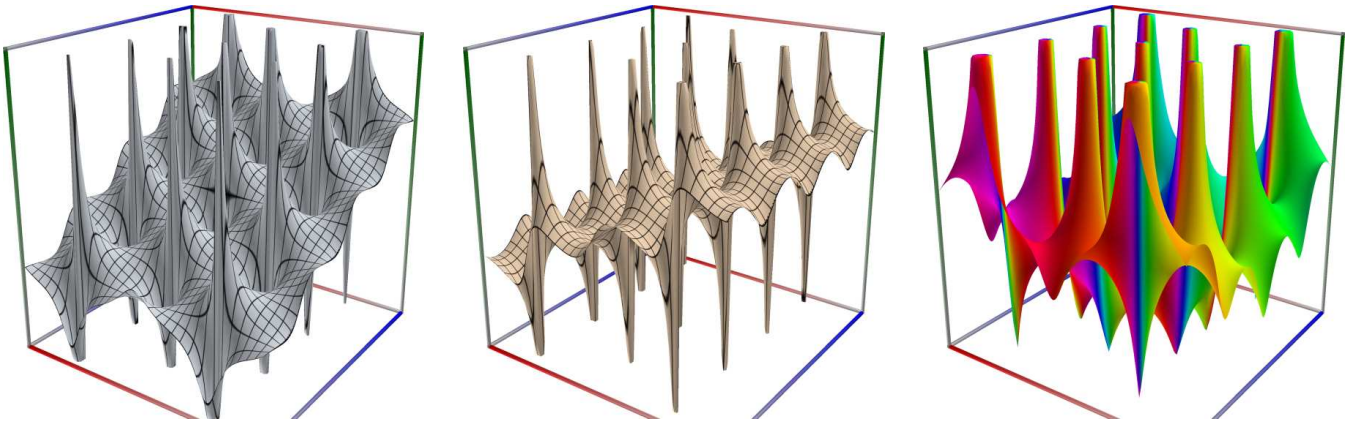
`ctx.weierstrass_zeta_g(z, g2, g3)`

Computes the Weierstrass zeta function $\zeta_g(z; g_2, g_3)$.

We have $\zeta_g(tz; t^{-4}g_2, t^{-6}g_3) = t^{-1}\zeta_g(z; g_2, g_3)$ and $\zeta_g(iz; g_2, g_3) = -i\zeta_g(z; g_2, -g_3)$.

The function is related to $\wp(z; g_2, g_3)$ by $\dfrac{d\zeta(z; g_2, g_3)}{\mathrm{d}z} = -\wp(z; g_2, g_3)$ and $\zeta(z; g_2, g_3) - z^{-1} = \displaystyle\int_0^z \left(\wp(z; g_2, g_3) - z^{-2}\right)$.

See also MathWorld [995], Flint [770].



Left figure: real part of the Weierstrass Zeta function $\zeta_g(z, g_2, g_3)$.

Middle figure: imaginary part of the Weierstrass Zeta function $\zeta_g(z, g_2, g_3)$.

Right figure: absolute value of the Weierstrass Zeta function $\zeta_g(z, g_2, g_3)$, with color-coded phase.

For $\Delta = g_2^3 - 27g_3^2 \neq 0$, we first compute the elliptic half periods ω_1, ω_2 from the lattice invariants g_2, g_3, and set $\omega = \omega_1, \tau = \omega_2/\omega_1$. Then $\zeta_g(z; g_2, g_3) = \zeta(z|\omega, \tau)$.

For $\Delta = 0, g_2 > 0, g_3 \neq 0$, the function $\zeta_g(\cdot)$ can be expressed in closed form in terms of elementary functions:

For $\Delta = 0, g_2 > 0, g_3 < 0$: $\quad \zeta_g(z; g_2, g_3) = -cz + \sqrt{3c}\coth(\sqrt{3c}z), \quad$ where $c = \sqrt{g_2/12}$.

For $\Delta = 0, g_2 > 0, g_3 > 0$: $\quad \zeta_g(z; g_2, g_3) = cz + \sqrt{3c}\cot(\sqrt{3c}z), \quad$ where $c = \sqrt{g_2/12}$.

If both g_2 and g_3 are zero, the function $\zeta_g(\cdot)$ becomes $\zeta_g(z; 0, 0) = z^{-1}$.

An example with real input for z and with purely imaginary input for τ, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; z = '0.1'; tau = '0.0 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_zeta(z, tau); mx = mpm.weierstrass_zeta(z, tau)
>>> gx = gmp.weierstrass_zeta(z, tau)
>>> fx = fpm.weierstrass_zeta(z, tau); ax = apm.weierstrass_zeta(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.995978010353406976694591385190992698486E+0
mpm:  9.995978010353406976694591385190992698486e+0
gmp:  9.995978010353406976694591385190992698486E+00
fpm:  9.99597801035341E+00
apm:  9.995978010353406976694591385190992698486e+0 (1.654e-38%)
```

31.2.4 Weierstrass Sigma function $\sigma_g(z, g_2, g_3)$

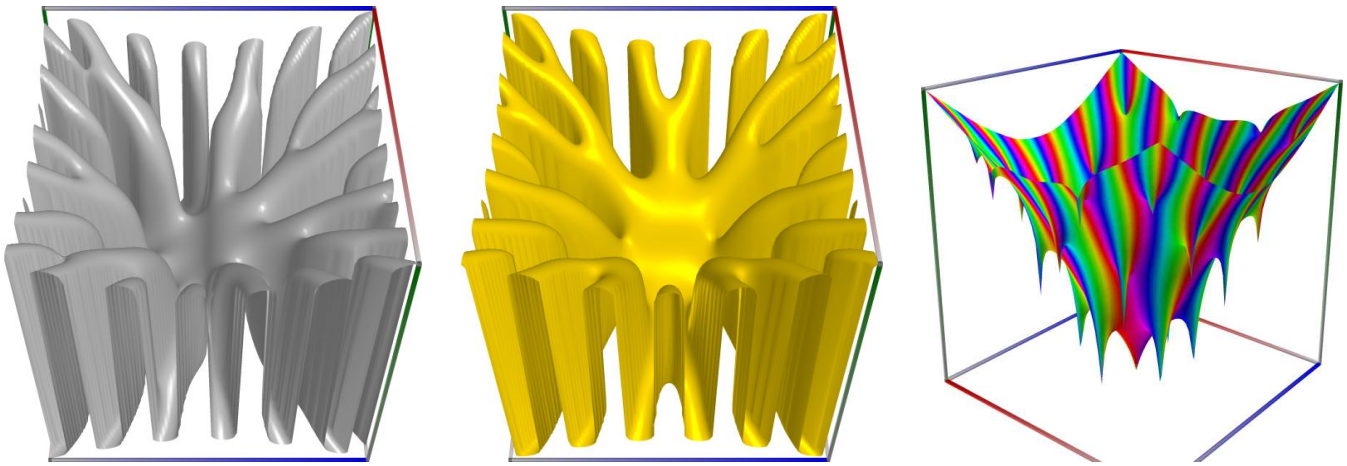
`ctx.weierstrass_sigma_g(z, g2, g3)`

Computes the Weierstrass sigma function, $\sigma_g(z; g_2, g_3)$. We have $\sigma_g(tz; t^{-4}g_2, t^{-6}g_3) = t\sigma_g(z; g_2, g_3)$.

The function is related to $\zeta(z; g_2, g_3)$ by $\dfrac{d}{dz}\log\sigma(z; g_2, g_3) = \zeta(z; g_2, g_3)$.

See also MathWorld [994], Flint [770].

See also: https://dlmf.nist.gov/23.2



Left figure: real part of the Weierstrass Sigma function $\sigma_g(z, g_2, g_3)$.

Middle figure: imaginary part of the Weierstrass Sigma function $\sigma_g(z, g_2, g_3)$.

Right figure: absolute value of the Weierstrass Sigma function $\sigma_g(z, g_2, g_3)$, with color-coded phase.

For $\Delta = g_2^3 - 27g_3^2 \neq 0$, we first compute the elliptic half periods ω_1, ω_2 from the lattice invariants g_2, g_3, and set $\omega = \omega_1, \tau = \omega_2/\omega_1$. Then $\sigma_g(z; g_2, g_3) = \sigma(z|\omega, \tau)$.

For $\Delta = 0, g_2 > 0, g_3 \neq 0$, the function $\sigma_g(\cdot)$ can be expressed in closed form in terms of elementary functions:

For $\Delta = 0, g_2 > 0, g_3 < 0$: $\quad \sigma_g(z; g_2, g_3) = \dfrac{\sinh(\sqrt{3c}z)}{\sqrt{3c}\cdot e^{cx^2/2}}, \quad$ where $c = \sqrt{g_2/12}$.

For $\Delta = 0, g_2 > 0, g_3 > 0$: $\quad \sigma_g(z; g_2, g_3) = \dfrac{\sin(\sqrt{3c}z)}{\sqrt{3c}\cdot e^{cx^2/2}}, \quad$ where $c = \sqrt{g_2/12}$.

If both g_2 and g_3 are zero, the function $\sigma_g(\cdot)$ becomes $\sigma_g(z; 0, 0) = z$.

An example with real input for z and with purely imaginary input for τ, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; z = '0.1'; tau = '0.0 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_sigma(z, tau); mx = mpm.weierstrass_sigma(z, tau)
>>> gx = gmp.weierstrass_sigma(z, tau)
>>> fx = fpm.weierstrass_sigma(z, tau); ax = apm.weierstrass_sigma(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  9.9989929898303836492644336943197578460 77E-2
mpm:  9.9989929898303836492644336943197578460 77e-2
gmp:  9.9989929898303836492644336943197578460 77E-02
fpm:  9.99899298983038E-02
apm:  9.9989929898303836492644336943197578460 77e-2 (5.023e-39%)
```

31.2.5 Inverse Weierstrass function $\wp_g^{-1}(z, g_2, g_3)$

`ctx.weierstrass_p_invG`(z, $g2$, $g3$)

Computes the Inverse Weierstrass function $\wp_g^{-1}(z, g_2, g_3)$

See also MathWorld [988], Flint [770].

$$\wp_g^{-1}(z; g_2, g_3) = \wp^{-1}(z|\tau = \omega_2/\omega_1, \omega = \omega_1)$$

Returns the functional inverse $\wp_g^{-1}$ of the Weierstrass function for $y \geq e_1$, i.e. the smallest positive x with $\wp_g^{-1}(y, g_2, g_3) = y$, if it exists. See also: MathWorld [988], Ehrhardt [297] (3.2.17.9).

An example with real input for z and with purely imaginary input for τ, producing real output:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 40; z = '100'; tau = '0.0 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p_inv(z, tau); mx = mpm.weierstrass_p_inv(z, tau)
>>> gx = gmp.weierstrass_p_inv(z, tau)
>>> fx = fpm.weierstrass_p_inv(z, tau); ax = apm.weierstrass_p_inv(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax])
dec:  1.000602721184701332310197413726890982085E-1
mpm:  1.000602721184701332310197413726890982085e-1
gmp:  1.000602721184701332310197413726890982085E-01
fpm:  1.00060272118470E-01
apm:  1.000602721184701332310197413726890982085e-1 (7.17e-40%)
```

An example with complex input for z and τ:

```
>>> from xlcalcnet import dec, mpm, gmp, fpm, apm
>>> mpm.dps = 20; z = '0.1 + 0.5j'; tau = '0.7 + 0.9j'
>>> \mathrm{d}x = dec.weierstrass_p_inv(z, tau); mx = mpm.weierstrass_p_inv(z, tau)
>>> gx = gmp.weierstrass_p_inv(z, tau)
>>> fx = fpm.weierstrass_p_inv(z, tau); ax = apm.weierstrass_p_inv(z, tau)
>>> mpm.show([\mathrm{d}x, mx, gx, fx, ax], aligned=True)
dec: 4.2630465458606458544E-1              - 3.0967048559182771279E-1j
mpm: 4.2630465458606458544e-1              - 3.0967048559182771279e-1j
gmp: 4.2630465458606458544E-01             - 3.0967048559182771279E-01j
fpm: 4.26304654586065E-01                  - 3.09670485591828E-01j
apm: 4.2630465458606458544e-1 (1.265e-16%) - 3.0967048559182771279e-1 (-1.063e-16%)j
```

31.3 Weierstrass elliptic functions, in terms of (real) lattice roots e_1, e_2

The Weierstrass functions take real values on the real axis iff the lattice is fixed under complex conjugation, or, equivalently, when $g_2, g_3 \in \mathbb{R}$.

31.3.1 Weierstrass function $\wp_e(x, e_1, e_2)$

`ctx.weierstrass_p_e`(*x*, *e1*, *e2*)

Returns the Weierstrass function $\wp_e(x, e_1, e_2)$ using the lattice roots and the Jacobi functions (where the equation with the smallest $e_k \geq 0$ is used). See also Wikipedia [1355], MathWorld [992], Ehrhardt [297] (3.2.17.2).

$$\wp_e(x, e_1, e_2) = e_3 + \frac{e_1 - e_3}{\mathrm{sn}^2(u,k)} = e_2 + (e_1 - e_3)\frac{\mathrm{dn}^2(u,k)}{\mathrm{sn}^2(u,k)} = e_1 + (e_1 - e_3)\frac{\mathrm{cn}^2(u,k)}{\mathrm{sn}^2(u,k)}, \quad k = \sqrt{\frac{e_2 - e_3}{e_1 - e_3}}, \quad u = x\sqrt{e_1 - e_3}.$$

Returns the the basic lemniscatic case $\wp_l(x) = \wp_g(x, 1, 0) = \wp_e\left(x, \frac{1}{2}, 0\right)$. See also Wikipedia [1355], MathWorld [992], Ehrhardt [297] (3.2.17.1).

Returns the Weierstrass function $\wp_e(iy, e_1, e_2) = -\wp_e(y, -e_1, -e_2)$. See also Wikipedia [1355], MathWorld [992], Ehrhardt [297] (3.2.17.4).

$$g_2 = 2({e_1}^2 + {e_2}^2 + {e_3}^2),$$

$$g_3 = 4e_1e_2e_3.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Wpe(2.5, 1.5, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.Wpe(2.5, 1.5, '0.51')
ereal('5.351847902755998475 4E-1')
```

31.3.2 Weierstrass function $\wp'_e(x, e_1, e_2)$

`ctx.weierstrass_pprime_e`(*x*, *e1*, *e2*)

Returns the derivative of the Weierstrass function $\wp'_e(x, e_1, e_2)$. See also Wikipedia [1355], MathWorld [992], Ehrhardt [297] (3.2.17.3).

$$\wp'_e(x, e_1, e_2) = -2(e_1 - e_3)^{3/2}\frac{\mathrm{cn}(u,k)\mathrm{dn}(u,k)}{\mathrm{sn}^3(u,k)}, \quad k = \sqrt{\frac{e_2 - e_3}{e_1 - e_3}}, \quad u = x\sqrt{e_1 - e_3}.$$

$$g_2 = 2({e_1}^2 + {e_2}^2 + {e_3}^2),$$

$$g_3 = 4e_1e_2e_3.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.WpeDer(2.5, 1.5, 0.5)
ereal('5.2359877559829887307E-1')
>>> math53.WpeDer(2.5, 1.5, '0.51')
ereal('5.351847902755998475 4E-1')
```

31.3.3 Weierstrass Zeta function $\zeta_e(z, e_1, e_2)$

ctx.**weierstrass_zeta_e**(*z*, *e_1*, *e_2*)

Computes the Weierstrass zeta function $\zeta_g(z; g_2, g_3)$.

We have $\zeta_g(tz; t^{-4}g_2, t^{-6}g_3) = t^{-1}\zeta_g(z; g_2, g_3)$ and $\zeta_g(iz; g_2, g_3) = -i\zeta_g(z; g_2, -g_3)$.

The function is related to $\wp(z; g_2, g_3)$ by $\dfrac{d\zeta(z; g_2, g_3)}{\mathrm{d}z} = -\wp(z; g_2, g_3)$ and $\zeta(z; g_2, g_3) - z^{-1} = \displaystyle\int_0^z \left(\wp(z; g_2, g_3) - z^{-2}\right)$.

See also MathWorld [995], Flint [770].

$$g_2 = 2({e_1}^2 + {e_2}^2 + {e_3}^2),$$

$$g_3 = 4e_1e_2e_3.$$

31.3.4 Weierstrass Sigma function $\sigma_e(z, e_1, e_2)$

ctx.**weierstrass_sigma_e**(*z*, *e_1*, *e_2*)

Computes the Weierstrass sigma function, $\sigma_g(z; g_2, g_3)$. We have $\sigma_g(tz; t^{-4}g_2, t^{-6}g_3) = t\sigma_g(z; g_2, g_3)$.

The function is related to $\zeta(z; g_2, g_3)$ by $\dfrac{d}{\mathrm{d}z}\log\sigma(z; g_2, g_3) = \zeta(z; g_2, g_3)$.

See also MathWorld [994], Flint [770].

See also: https://dlmf.nist.gov/23.2

$$g_2 = 2({e_1}^2 + {e_2}^2 + {e_3}^2),$$

$$g_3 = 4e_1e_2e_3 = .$$

31.3.5 Inverse Weierstrass function $\wp_e^{-1}(y, e_1, e_2)$

ctx.**weierstrass_p_inv_e**(*y*, *e1*, *e2*)

Returns the functional inverse $\wp_e^{-1}$ of the Weierstrass function for $y \geq e_1$, i.e. the smallest positive x with $\wp_e(x, e_1, e_2) = y$. The result is computed with the symmetric Carlson integral,

$$\wp_e^{-1}(y, e_1, e_2) = \frac{1}{2}\int_y^\infty \frac{\mathrm{d}t}{\sqrt{(t-e_1)(t-e_2)(t-e_3)}} = R_F(y-e_1, y-e_2, y-e_3)$$

See also: MathWorld [988], Ehrhardt [297] (3.2.17.8).

$$g_2 = 2({e_1}^2 + {e_2}^2 + {e_3}^2),$$

$$g_3 = 4e_1e_2e_3.$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.WpeInv(2.5, 1.5, 0.5)
ereal('5.235987755982988730 7E-1')
>>> math53.WpeInv(2.5, 1.5, '0.51')
ereal('5.351847902755998475 4E-1')
```

Part VII

User library: numerical

CHAPTER

THIRTYTWO

ADDITIONAL CLASSES

General concepts

See also: https://en.wikipedia.org/wiki/Power_of_a_test

See also: https://en.wikipedia.org/wiki/Sample_size_determination

32.1 User defined functions based on multiple precision arithmetic (Python)

32.1.1 Overview

High-level code in xlcalcnet is, as in mpmath, implemented as methods on a "context object". The context implements arithmetic, type conversions and other fundamental operations. The context also holds settings such as precision, and stores cache data. A total of 6 different contexts (with a mostly compatible interface) are provided so that the high-level algorithms can be used with different implementations of the underlying arithmetic, allowing different features and speed-accuracy tradeoffs.

32.1.2 Double-precision arithmetic (`fpmlib`)

Although mpmath is generally designed for arbitrary-precision arithmetic, many of the high-level algorithms work perfectly well with ordinary Python `float` and `complex` numbers, which use hardware double precision (on most systems, this corresponds to 53 bits of precision). Whereas the global functions (which are methods of the `mp` object) always convert inputs to mpmath numbers, the `fp` object instead converts them to `float` or `complex`, and in some cases employs basic functions optimized for double precision. When large amounts of function evaluations (numerical integration, plotting, etc) are required, and when `fp` arithmetic provides sufficient accuracy, this can give a significant speedup over `mp` arithmetic.

32.1.3 Binary floating-point in arbitrary-precision and with arbitrary exponent (`mpmlib`)

The `mp` context is what most users probably want to use most of the time, as it supports the most functions, is most well-tested, and is implemented with a high level of optimization. Nearly all examples in this documentation use `mp` functions.

32.1.4 Interval arithmetic in arbitrary-precision and with arbitrary exponent (`ipmlib`)

The `iv.mpf` type represents a closed interval $[a, b]$; that is, the set $\{x : a \le x \le b\}$, where a and b are arbitrary-precision floating-point values, possibly $\pm\infty$. The `iv.mpc` type represents a rectangular complex interval $[a, b] + [c, d]i$; that is, the set $\{z = x + iy : a \le x \le b \wedge c \le y \le d\}$.

Interval arithmetic provides rigorous error tracking. If f is a mathematical function and $\hat{f}$ is its interval arithmetic version, then the basic guarantee of interval arithmetic is that $f(v) \subseteq \hat{f}(v)$ for any input interval v. Put differently, if an interval represents the known uncertainty for a fixed number, any sequence of interval operations will produce an interval that contains what would be the result of applying the same sequence of operations to the exact number. The principal drawbacks of interval arithmetic are speed (`iv` arithmetic is typically at least two times slower than `mp` arithmetic) and that it sometimes provides far too pessimistic bounds.

32.1.5 Decimal floating-point in arbitrary-precision with limited exponent (`dpmlib`)

Additional contexts are used in xlcalcnet to implement its functions for the mpmath data types, and the Decimal data type, which is part of Python.

Both real numbers (mpf) and complex numbers (mpc) are implemented.

In CPython, the `decimal` module provides support for fast correctly-rounded decimal floating point arithmetic. See https://docs.python.org/3.3/library/decimal.html for a decription of the `Decimal` data type.

32.1.6 Rational numbers (quotients) in arbitrary-precision (`qpmlib`)

The `qpm` data type is mostly useful in the context of linear algebra, where it can provide exact results.

Both real numbers (mpf) and complex numbers (mpc) are implemented.

The internal representation dependes on what else is installed on the system:

If `apm` is available, the `fmpq` data type is used; otherwise, if `gpm` is available, the `mpq` data type is used; otherwise, Python's built in `Fraction` data type is used.

32.1.7 Binary floating-point in arbitrary-precision with limited exponent (gpmlib)

gmpy2 is a C-coded Python extension module that supports multiple-precision arithmetic.

https://gmpy2.readthedocs.io/en/latest/

gmpy2 is the successor to the original gmpy module. The gmpy module only supported the GMP multiple-precision library. gmpy2 adds support for the MPFR (correctly rounded real floating-point arithmetic) and MPC (correctly rounded complex floating-point arithmetic) libraries. gmpy2 also updates the API and naming conventions to be more consistent and support the additional functionality. The following libraries are supported:

32.1.8 Binary balls in arbitrary-precision and with arbitrary exponent (apmlib)

The apm context is what most users probably want to use most of the time, as it supports the most functions, is most well-tested, and is implemented with a high level of optimization.

pythonflint is a C-coded Python extension module that supports multiple-precision arithmetic.

32.2 User defined functions based on fixed precision arithmetic (C#)

Various additional numerical context support C# and Python. It is convenient to divide them into groups to make referencing them per function description more efficient.

The following groups of contexts are available:

Note: this has to mention the userlib functions

32.2.1 Scalar functions with `Double` and `Complex`: `m53lib` and `m53libc`

DAMath library.

Double and Complex fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Double-precision_floating-point_format

32.2.2 Binary floating point, single precision: `slib` and `slibc`

sreal and scplx fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Single-precision_floating-point_format

32.2.3 Binary floating point, double precision: `dlib` and `dlibc`

dreal and dcplx fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Double-precision_floating-point_format

32.2.4 Binary floating point, extended precision: `elib` and `elibc`

ereal and ecplx fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Extended_precision

32.2.5 Binary floating point, quadruple precision: `qlib` and `qlib`

qreal and qcplx fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Quadruple-precision_floating-point_format

32.2.6 Binary floating point, octuple precision: `olib` and `olibc`

oreal and ocplx fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Octuple-precision_floating-point_format

32.3 User defined functions based on arbitrary precision arithmetic (C#)

32.3.1 Binary floating point, multiple precision: `mlib`, `mlibc`

Some text

arbitrary precision significand, fixed size exponent

32.3.2 Binary floating point, single precision: `sflib` and `sflibc`

sreal and scplx fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Single-precision_floating-point_format

32.3.3 Binary floating point, double precision: `dflib` and `dflibc`

dreal and dcplx fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Double-precision_floating-point_format

32.3.4 Binary floating point, extended precision: `eflib` and `eflibc`

ereal and ecplx fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Extended_precision

32.3.5 Binary floating point, quadruple precision: `qflib` and `qflibc`

qreal and qcplx fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Quadruple-precision_floating-point_format

32.3.6 Binary floating point, octuple precision: `oflib` and `oflibc`

oreal and ocplx fully support Boost.Math and Eigen.

https://en.wikipedia.org/wiki/Octuple-precision_floating-point_format

32.3.7 Binary floating point, multiple precision: `mflib`, `mflibc`

Some text

arbitrary precision significand, fixed size exponent

32.3.8 Binary arbitrary precision ball: `aflib` and `aflibc`

Some text

arbitrary precision significand and exponent

https://flib.org/

Note: Exponent soft maximum and minimum

32.4 Scalar functions

32.4.1 Real and complex sine

`ctxlib.`**`sin`**(x)

where `ctx` is `ctx_pm` (see *Python contexts* for details), `ctx53`, `ctxcpp`, `ctxflint` (see *.NET contexts* for details). The corresponding `ctx` python lists are `ctxlistreal` and `ctxlistcplx`.

Returns the sine of x, $\sin(x)$. See also Wikipedia [1244], MathWorld [874], NIST [492], Ehrhardt [297] (4.2.55), Flint [746], Flint [736], Mpmath [557].

32.4.2 Quadratic equation

`ctx.eval_quadratic`(x, A, B, C)

Returns the value of a quadratic polynomial, $Ax^2 + Bx + C$.

`ctx.quadratic_equation`(A, B, C)

Returns the roots x_1, x_2 of the quadratic equation $Ax^2 + Bx + C = 0$. See also Wikipedia [1097], Press *et al.* [474].

See also: https://dlmf.nist.gov/1.11#iii

$$x_1 = \frac{Q}{A}, \quad x_2 = \frac{C}{Q}, \quad \text{where } Q = -\frac{1}{2}\left(B + \sqrt{B^2 - 4AC)}\right)$$

The sign of the square root is chosen so as to make $\Re(B^* \sqrt{B^2 - 4AC}) \geq 0$, where the asterisk denotes complex conjugation.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '125'; n = '3'
>>> \mathrm{d}x = dec.nthroot(x, n); mx = mpm.nthroot(x, n); ix = ipm.nthroot(x, n)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  5.0000000000000000000000000000000000000000E+0
mpm:  5.0000000000000000000000000000000000000000e+0
ipm:  5.0000000000000000000000000000000000000000e+0 (2.755e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '125'; n = '3'
>>> fx = fpm.nthroot(x, n); gx = gmp.nthroot(x, n); ax = apm.nthroot(x, n)
>>> mpm.show([fx, gx, ax])
fpm:  5.00000000000000E+00
gmp:  5.0000000000000000000000000000000000000000E+00
apm:  5.0000000000000000000000000000000000000002e+0 (3.673e-39%)
```

32.4.3 Monic cubic equation

`ctx.eval_monic_cubic`(x, a, b, c)

Returns the value of a monic cubic polynomial, $x^3 + ax^2 + bx + c$.

`ctx.cubic_equation_monic`(a, b, c)

Returns the roots x_1, x_2, x_3 of the monic cubic equation $x^3 + ax^2 + bx + c = 0$. See also Wikipedia [1088], Press *et al.* [474].

See also: https://dlmf.nist.gov/1.11#iii

$$Q = \frac{a^2 - 3b}{9}, \quad R = \frac{2a^3 - 9ab + 27c}{54}.$$

If Q and R are real *and* $R^2 < Q^3$, then the cubic equation has three real roots, with $\theta = \arccos\left(RQ^{-3/2}\right)$:

$$x_1 = -2\sqrt{Q}\cos\left(\frac{\theta}{3}\right) - \frac{a}{3}, \quad x_2 = -2\sqrt{Q}\cos\left(\frac{\theta + 2\pi}{3}\right) - \frac{a}{3}, \quad x_3 = -2\sqrt{Q}\cos\left(\frac{\theta - 2\pi}{3}\right) - \frac{a}{3}.$$

Otherwise, $A = -\left(R + \sqrt{R^2 - Q^3})\right)^{1/3}$, where the sign of the square root is chosen so as to make $\Re(R^*\sqrt{R^2 - Q^3}) \geq 0$, and the asterisk denotes complex conjugation. Define $B = 0$ if $A = 0$ and $B = Q/A$ if $A \neq 0$. Then the three roots are given by

$$x_1 = (A + B) - \frac{a}{3}, \quad x_2 = -\frac{1}{2}(A + B) - \frac{a}{3} + i\frac{\sqrt{3}}{2}(A - B), \quad x_3 = -\frac{1}{2}(A + B) - \frac{a}{3} - i\frac{\sqrt{3}}{2}(A - B).$$

Note that x_1 is real if a, b, c are real.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '125'; n = '3'
>>> \mathrm{d}x = dec.nthroot(x, n); mx = mpm.nthroot(x, n); ix = ipm.nthroot(x, n)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  5.000000000000000000000000000000000000000E+0
mpm:  5.000000000000000000000000000000000000000e+0
ipm:  5.000000000000000000000000000000000000000e+0 (2.755e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '125'; n = '3'
>>> fx = fpm.nthroot(x, n); gx = gmp.nthroot(x, n); ax = apm.nthroot(x, n)
>>> mpm.show([fx, gx, ax])
fpm:  5.00000000000000E+00
gmp:  5.000000000000000000000000000000000000000E+00
apm:  5.000000000000000000000000000000000000002e+0 (3.673e-39%)
```

32.4.4 Cubic equation

`ctx.eval_cubic`(x, A, B, C, D)

Returns the value of a cubic polynomial, $Ax^3 + Bx^2 + Cx + D$.

`ctx.cubic_equation`(A, B, C, D)

Returns the roots x_1, x_2, x_3 of the cubic equation $Ax^3 + Bx^2 + Cx + D = 0$. See also Wikipedia [1088], Press *et al.* [474].

See also: https://dlmf.nist.gov/1.11#iii

This just calls *CubicEquationMonicRoots(z, a, b, c)* with $a = B/A, b = C/A, c = D/A$.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '125'; n = '3'
>>> \mathrm{d}x = dec.nthroot(x, n); mx = mpm.nthroot(x, n); ix = ipm.nthroot(x, n)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  5.0000000000000000000000000000000000000000E+0
mpm:  5.0000000000000000000000000000000000000000e+0
ipm:  5.0000000000000000000000000000000000000000e+0 (2.755e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '125'; n = '3'
>>> fx = fpm.nthroot(x, n); gx = gmp.nthroot(x, n); ax = apm.nthroot(x, n)
>>> mpm.show([fx, gx, ax])
fpm:  5.00000000000000E+00
gmp:  5.0000000000000000000000000000000000000000E+00
apm:  5.0000000000000000000000000000000000000002e+0 (3.673e-39%)
```

32.4.5 Quartic equation

`ctx.eval_quartic`(x, A, B, C, D, E)

Returns the value of a quartic polynomial, $Ax^4 + Bx^3 + Cx^2 + Dx + E$.

`ctx.quartic_equation`(A, B, C, D, E)

Returns the roots x_1, x_2, x_3, x_4 of the quartic equation $Ax^4 + Bx^3 + Cx^2 + Dx + E = 0$. See also Wikipedia [1098].

See also: https://dlmf.nist.gov/1.11#iii

Define $a = \frac{-3B^2}{8A^2} + \frac{C}{A}$, $b = \frac{B^3}{8A^3} - \frac{BC}{2A^2} + \frac{D}{A}$, $c = \frac{-3B^4}{256A^4} + \frac{CB^2}{16A^3} - \frac{BD}{4A^2} + \frac{E}{A}$, $V = \frac{B}{4A}$.

If $b = 0$ then

$x_1 = V + Z_1$, $x_2 = V - Z_1$, $x_3 = V + Z_2$, $x_4 = V - Z_2$,

where $W = \sqrt{a^2 - 4c}$, $Z_1 = \sqrt{\frac{1}{2}(-a + W)}$, $Z_2 = \sqrt{\frac{1}{2}(-a - W)}$.

If $b \neq 0$ then

$x_1 = V + \frac{1}{2}(W + Z_1)$, $x_2 = V + \frac{1}{2}(W - Z_1)$, $x_3 = V - \frac{1}{2}(W + Z_2)$, $x_4 = V - \frac{1}{2}(W - Z_2)$,

where $W = \sqrt{a + 2y}$, $Z_1 = \sqrt{-3a - 2y - \frac{2b}{W}}$, $Z_2 = \sqrt{-3a - 2y + \frac{2b}{W}}$,

and y is any root of the monic cubic equation $y^3 + ey^2 + fy + g = 0$, with $e = \frac{5a}{2}$, $f = 2a^2 - c$, $g = \frac{a^3}{2} - \frac{ac}{2} - \frac{b^2}{8}$;

y is calculated as the first root returned by *CubicEquationMonicRoots(y, e, f, g)*.

An example with real input:

```
>>> from xlcalcnet import dec, mpm, ipm
>>> mpm.dps = 40; x = '125'; n = '3'
>>> \mathrm{d}x = dec.nthroot(x, n); mx = mpm.nthroot(x, n); ix = ipm.nthroot(x, n)
>>> mpm.show([\mathrm{d}x, mx, ix])
dec:  5.000000000000000000000000000000000000000E+0
mpm:  5.000000000000000000000000000000000000000e+0
ipm:  5.000000000000000000000000000000000000000e+0 (2.755e-39%)

>>> from xlcalcnet import mpm, fpm, gmp, apm
>>> mpm.dps = 40; x = '125'; n = '3'
>>> fx = fpm.nthroot(x, n); gx = gmp.nthroot(x, n); ax = apm.nthroot(x, n)
>>> mpm.show([fx, gx, ax])
fpm:  5.00000000000000E+00
gmp:  5.000000000000000000000000000000000000000E+00
apm:  5.000000000000000000000000000000000000002e+0 (3.673e-39%)
```

32.4.6 Transscribed from Julia: complex elliptic functions in double precision

Text describing functions

32.4.7 Transscribed from Julia: complex hurwitz function, and related, in double precision

Text describing functions

32.4.8 Speeedups for iterative algorithms which require an initial guess

Text describing functions

Inverses of cdfs: inverses of noncentral functions

inverses livk gamma_inva

32.4.9 Examples for matrix functions

Text describing functions

Descriptive statistics via numpy in multiple precision

Multiple linear regression via numpy in multiple precision

Canonical correlation via numpy in multiple precision

Descriptive statistics via Eigen in multiple precision

Multiple linear regression via Eigen in multiple precision

Canonical correlation via Eigen in multiple precision

Levenberg-Marquardt algorithm via Eigen in multiple precision

L-BFGS algorithm via Eigen in multiple precision

CHAPTER

THIRTYTHREE

DISTRIBUTION FUNCTIONS

Text explaining this chapter

33.1 Distributions related to multiple comparisons of means

33.1.1 Normal maximum distribution, $\rho_{ij,i\neq j} = \rho$: pdf, cdf, qtf, boost class

`math53lib.`**`NmaxRhoPdf`**(*x*, *rho*, *k*)

`math53lib.`**`NmaxRhoCdf`**(*x*, *rho*, *k*)

`math53lib.`**`NmaxRhoQtf`**(*q*, *rho*, *k*)

`math53lib.`**`NmaxRhoDist`**(*rho*, *k*)

Returns the pdf, cdf, qtf or boost class of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variates, with common correlation $-1/(k-1) \leq \rho \leq 1$ and the support interval $(-\infty, +\infty)$, and $0 \leq q \leq 1$.

See also Dunnett [292], Bechhofer and Dunnett [26], Grubbs [354], Bechhofer and Dunnett [26], Stoline and Ury [507], and Hahn and Hendrickson [361].

$$\text{pdf}(x) = f_{\text{nmaxrho}}(x, \rho, k) = \frac{k}{b} \int_{-\infty}^{\infty} (\Phi(z_1))^{k-1} \, \phi(z_1) \, \phi(y) \, dy,$$

$$\text{cdf}(x) = F_{\text{nmaxrho}}(x, \rho, k) = \int_{-\infty}^{\infty} (\Phi(z_1))^{k} \, \phi(y) dy,$$

$$\text{sf}(x) = 1 - \int_{-\infty}^{\infty} (\Phi(z_1))^{k} \, \phi(y) dy,$$

where $z_1 = \dfrac{x+a}{b}$, $a = y\sqrt{\rho}$, and $b = \sqrt{1-\rho}$. Note that while z_1 is a complex number for $\rho < 0$, the integral always evaluates to a real number. In the context of the Dunnett t-test with reference sample size n_0 and common comparator sample size n_i, $\rho = \dfrac{1}{1+n_0/n_i}$. In the context of maximum deviation from the common mean (equal sample sizes), $\rho = -\dfrac{1}{k-1}$.

There is no known explicit expression for $\text{qtf}(q)$ or $\text{isf}(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.NmaxRhoPdf(x, rho, k)
ereal('5.2359877559829887307E-1')
```

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.NmaxRhoPdf(x, rho, k)
ereal('5.2359877559829887307E-1')

>>> from xlcalcnet import math53
>>> math53.NmaxRhoCdf(x, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmaxRhoCdf(x, rho, k)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.NmaxRhoQtf(q, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmaxRhoQtf(q, rho, k)
ereal('5.3518479027559984754E-1')
```

33.1.2 Normal maximum modulus distribution, $\rho_{ij,i\neq j} = \rho$: pdf, cdf, qtf, boost class

`math53lib.NmmRhoPdf`(*x*, *rho*, *k*)

`math53lib.NmmRhoCdf`(*x*, *rho*, *k*)

`math53lib.NmmRhoQtf`(*q*, *rho*, *k*)

`math53lib.NmmRhoDist`(*rho*, *k*)

Returns the pdf, cdf, qtf or boost class of a random variable X, following the distribution of the maximum of the absolute value of $k \geq 2$ correlated standard normal variates, with common correlation $-1/(k-1) \leq \rho \leq 1$ and the support interval $(0, +\infty)$, and $0 \leq q \leq 1$.

See also Dunnett [292], Bechhofer and Dunnett [26], Grubbs [354], Bechhofer and Dunnett [26], Stoline and Ury [507], and Hahn and Hendrickson [361].

$$\text{pdf}(x) = \frac{k}{b} \int_{-\infty}^{\infty} \left(\Phi(z_1) - \Phi(z_2)\right)^{k-1} \left(\phi(z_1) + \phi(z_1)\right) \phi(y)\, dy,$$

$$\text{cdf}(x) = F_{\text{nmaxrho}}(x, \rho, k) = \int_{-\infty}^{\infty} \left(\Phi(z_1) - \Phi(z_2)\right)^{k} \phi(y) dy,$$

$$\text{sf}(x) = 1 - \int_{-\infty}^{\infty} \left(\Phi(z_1) - \Phi(z_2)\right)^{k} \phi(y) dy,$$

where $z_1 = \dfrac{x+a}{b}$, $z_2 = \dfrac{-x+a}{b}$, $a = y\sqrt{\rho}$, and $b = \sqrt{1-\rho}$. Note that while z_1 and z_2 are complex numbers for $\rho < 0$, the integral always evaluates to a real number. In the context of the Dunnett t-test with reference sample size n_0 and common comparator sample size n_i, $\rho = \dfrac{1}{1 + n_0/n_i}$. In the context of maximum deviation from the common mean (equal sample sizes), $\rho = -\dfrac{1}{k-1}$.

There is no known explicit expression for $\text{qtf}(q)$ or $\text{isf}(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.NmmRhoPdf(x, rho, k)
ereal('5.2359877559829887307E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.NmmRhoPdf(x, rho, k)
ereal('5.2359877559829887307E-1')

>>> from xlcalcnet import math53
>>> math53.NmmRhoCdf(x, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmmRhoCdf(x, rho, k)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.NmmRhoQtf(q, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmmRhoQtf(q, rho, k)
ereal('5.3518479027559984754E-1')
```

33.1.3 Normal maximum distribution, $\rho_{ij,i\neq j} = \lambda_i\lambda_j$: pdf, cdf, qtf, boost class

math53lib.**NmaxRhoijPdf**(*x*, *lambda_i*, *k*)

math53lib.**NmaxRhoijCdf**(*x*, *lambda_i*, *k*)

math53lib.**NmaxRhoijQtf**(*q*, *lambda_i*, *k*)

math53lib.**NmaxRhoijDist**(*lambda_i*, *k*)

Returns the pdf, cdf, qtf or boost class of a random variable X, following the distribution of the maximum deviate of $k \geq 2$ correlated standard normal variates, with common correlation $0 \leq \rho \leq 1$ and the support interval $(-\infty, +\infty)$, and $0 \leq q \leq 1$. See also Dunnett [292], Bechhofer and Dunnett [26], Grubbs [354], Bechhofer and Dunnett [26], Stoline and Ury [507], and Hahn and Hendrickson [361].

$$\text{pdf}(x) = f_{\text{nmaxrhoij}}(x, \rho, k) = \frac{k}{b}\int_{-\infty}^{\infty} (\Phi(z_1))^{k-1}\,\phi(z_1)\,\phi(y)\,dy, \quad \text{where}$$

$$z_1 = \frac{x+a}{b}, \quad a = y\sqrt{|\rho|}, \quad b = \sqrt{1-\rho}, \rho = -\frac{1}{k-1}.$$

$$\text{cdf}(x) = F_{\text{nmaxrhoij}}(x, \rho, k) = \int_{-\infty}^{\infty}\left[\Phi\left(\frac{x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)\right]^n \phi(y)dy, \rho = -\frac{1}{k-1}.$$

$$\text{sf}(x) = 1 - \int_{-\infty}^{\infty}\left[\Phi\left(\frac{x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)\right]^n \phi(y)dy, \rho = -\frac{1}{k-1}.$$

For $\lambda_i = \sqrt{\rho} \geq 0$ for all i, this reduces to the equicorrelated case.

With $\rho_{ij} = \lambda_i\lambda_j$, we have for a one-sided test:

$$F_n(h; \rho_{ij}) = \int_{-\infty}^{\infty}\prod_{i=1}^{n}\left[\Phi\left(\frac{(a_i-\mu_i)/\sigma_i + \lambda_i z}{\sqrt{1-\lambda_i^2}}\right)\right]\phi(y)dz$$

There is no known explicit expression for $\text{qtf}(q)$ or $\text{isf}(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.NmaxRhoijPdf(q, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmaxRhoijPdf(q, rho, k)
ereal('5.3518479027559984754E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.NmaxRhoijPdf(q, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmaxRhoijPdf(q, rho, k)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.NmaxRhoijCdf(q, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmaxRhoijCdf(q, rho, k)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.NmaxRhoijQtf(q, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmaxRhoijQtf(q, rho, k)
ereal('5.3518479027559984754E-1')
```

33.1.4 Normal maximum modulus distribution, $\rho_{ij,i\neq j} = \lambda_i\lambda_j$: pdf, cdf, qtf, boost class

`math53lib.`**`NmmRhoijPdf`**(*x*, *rho*, *k*)

`math53lib.`**`NmmRhoijCdf`**(*x*, *rho*, *k*, *cdf=True*)

`math53lib.`**`NmmRhoijQtf`**(*q*, *rho*, *k*, *qtf=True*)

`math53lib.`**`NmmRhoijDist`**(*q*, *rho*, *k*, *qtf=True*)

Returns the pdf, cdf, qtf or boost class of a random variable X, following the distribution of the maximum of $k \geq 2$ correlated standard normal variates, with correlation $\rho_{ij,i\neq j} = \lambda_i\lambda_j$ and the support interval $(-\infty, +\infty)$, and $0 \leq q \leq 1$.

See also Dunnett [292], Bechhofer and Dunnett [26], Grubbs [354], Bechhofer and Dunnett [26], Stoline and Ury [507], and Hahn and Hendrickson [361].

$$\text{pdf}(x) = \frac{k}{b}\int_{-\infty}^{\infty} \left(\Phi(z_1) - \Phi(z_2)\right)^{k-1} \left(\phi(z_1) + \phi(z_1)\right) \phi(y)\, dy, \quad \text{where}$$

$$z_1 = \frac{x+a}{b}, z_2 = \frac{-x+a}{b}, \quad a = y\sqrt{|\rho|}, \quad b = \sqrt{1-\rho}, \rho = -\frac{1}{k-1}.$$

$$\text{cdf}(x) = \int_{-\infty}^{\infty} \left[\Phi\left(\frac{x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right) - \Phi\left(\frac{-x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)\right]^k \phi(y)dy, \rho = -\frac{1}{k-1}.$$

$$\text{sf}(x) = 1 - \int_{-\infty}^{\infty} \left[\Phi\left(\frac{x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right) - \Phi\left(\frac{-x+\sqrt{|\rho|}y}{\sqrt{1-\rho}}\right)\right]^k \phi(y)dy, \rho = -\frac{1}{k-1}.$$

For $\lambda_i = \sqrt{\rho} \geq 0$ for all i, this reduces to the equicorrelated case.

With $\rho_{ij} = \lambda_i\lambda_j$, we have for a two-sided test:

$$F_n(h;\rho_{ij}) = \int_{-\infty}^{\infty} \prod_{i=1}^{n} \left[\Phi\left(\frac{(a_i-\mu_i)/\sigma_i + \lambda_i z}{\sqrt{1-\lambda_i^2}}\right) - \Phi\left(\frac{(b_i-\mu_i)/\sigma_i + \lambda_i z}{\sqrt{1-\lambda_i^2}}\right)\right] \phi(y)dz$$

There is no known explicit expression for $\text{qtf}(q)$ or $\text{isf}(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.NmmRhoijPdf(x, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmmRhoijPdf(x, rho, k)
ereal('5.3518479027559984754E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.NmmRhoijPdf(x, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmmRhoijPdf(x, rho, k)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.NmmRhoijCdf(x, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmmRhoijCdf(x, rho, k)
ereal('5.3518479027559984754E-1')
```

(continues on next page)

(continued from previous page)

```
>>> from xlcalcnet import math53
>>> math53.NmmRhoijQtf(q, rho, k)
ereal('5.2359877559829887307E-1')
>>> math53.NmmRhoijQtf(q, rho, k)
ereal('5.3518479027559984754E-1')
```

33.1.5 Normal range distribution: pdf, cdf, qtf, boost class

math53lib.**NrangePdf**(x, k)

math53lib.**NrangeCdf**(n, x)

math53lib.**NrangeQtf**(q, k)

math53lib.**NrangeDist**(k)

Returns the pdf, cdf, qtf or boost class of a random variable X, following a normal range distribution, with $k \geq 2$ groups, and the support interval $(0, +\infty)$, and $0 \leq q \leq 1$. See also Wikipedia [1218], Harter [364], Team [535].

The density in the central case is given by

$$\text{pdf}(x) = k(k-1) \int_{-\infty}^{\infty} (\Phi(y) - \Phi(y-x))^{k-2} \phi(y)\phi(y-x)dy$$

We have

$$\text{cdf}(x) = k \int_{-\infty}^{\infty} \phi(y) \left(\Phi(y) - \Phi(y-x)\right)^{k-1} dy$$

$$\text{sf}(x) = 1 - k \int_{-\infty}^{\infty} \phi(y) \left(\Phi(y) - \Phi(y-x)\right)^{k-1} dy$$

In the noncentral case, this becomes

$$\text{cdf}(x) = F_s r\left(m, n, x\right) = \sum_{i=1}^{k} \int_{-\infty}^{\infty} \phi(y_i - \mu_i) \left(\prod_{i=1, j \neq i}^{k} (\Phi(y_i - \mu_i) - \Phi(y_i - \mu_i - x) \right) dy_i.$$

There is no known explicit expression for qtf(q) or isf(q): These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.NrangePdf(x, k)
ereal('5.235987755982988730 7E-1')
>>> math53.NrangePdfxq, k)
ereal('5.351847902755998475 4E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.NrangePdf(x, k)
ereal('5.235987755982988730 7E-1')
>>> math53.NrangePdfxq, k)
ereal('5.351847902755998475 4E-1')

>>> from xlcalcnet import math53
>>> math53.NrangeCdf(x, k)
ereal('5.235987755982988730 7E-1')
>>> math53.NrangePdf(x, k)
ereal('5.351847902755998475 4E-1')

>>> from xlcalcnet import math53
>>> math53.NrangeQtf(q, k)
ereal('5.235987755982988730 7E-1')
>>> math53.NrangeQtf(q, k)
ereal('5.351847902755998475 4E-1')
```

33.1.6 Studentized maximum distribution: pdf, cdf, qtf, boost class

math53lib.**SmaxPdf**(*x*, *k*, *nu*)

math53lib.**SmaxCdf**(*x*, *k*, *nu*)

math53lib.**SmaxQtf**(*q*, *k*, *nu*)

math53lib.**SmaxDist**(*k*, *nu*)

Returns the pdf, cdf, qtf or boost class of a random variable X, following a studentized maximum distribution with $k \geq 1$ groups, ν error degrees of freedom, and the support interval $(-\infty, +\infty)$, and $0 \leq q \leq 1$. See also Stoline and Ury [507], Hochberg and Tamhane [373], Narula [422].

$$\text{pdf}(x) = \int_0^\infty f_{\text{nnax}}(sx, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $f_{\text{nmax}}(\cdot, k)$ is the pdf of the normal maximum with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{cdf}(x) = \int_0^\infty F_{\text{nmax}}(sx, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $F_{\text{nnax}}(\cdot, k)$ is the cdf of the normal maximum with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{sf}(x) = 1 - \text{cdf}(x)$$

There is no known explicit expression for $\text{qtf}(q)$ or $\text{isf}(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SmaxPdf(x, k)
ereal('5.2359877559829887307E-1')
>>> math53.SmaxPdf(x, k)
ereal('5.3518479027559984754E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.SmaxPdf(x, k)
ereal('5.2359877559829887307E-1')
>>> math53.SmaxPdf(x, k)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.SmaxCdf(x, k)
ereal('5.2359877559829887307E-1')
>>> math53.SmaxCdf(x, k)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.SmaxQtf(q, k)
ereal('5.2359877559829887307E-1')
>>> math53.SmaxQtf(q, k)
ereal('5.3518479027559984754E-1')
```

33.1.7 Studentized maximum modulus distribution: pdf, cdf, qtf, boost class

math53lib.**SmmPdf**(*x*, *k*, *nu*)

math53lib.**SmmCdf**(*x*, *k*, *nu*)

math53lib.**SmmQtf**(*q*, *k*, *nu*)

math53lib.**SmmDist**(*k*, *nu*)

Returns the pdf, cdf, qtf or boost class of a random variable X, of a random variable X, following a studentized maximum modulus distribution with $k \geq 1$ groups, ν error degrees of freedom, and the support interval $(0, +\infty)$, and $0 \leq q \leq 1$. See also Stoline and Ury [507], Hochberg and Tamhane [373], Narula [422].

$$\text{pdf}(x) = \int_0^\infty f_{\text{nmm}}(sx, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $f_{\text{nmm}}(\cdot, k)$ is the pdf of the normal maximum modulus with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{cdf}(x) = \int_0^\infty F_{\text{nmm}}(sx, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $F_{\text{nmm}}(\cdot, k)$ is the cdf of the normal maximum modulus with k groups, and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{sf}(x) = 1 - \text{cdf}(x)$$

There is no known explicit expression for $\text{qtf}(q)$ or $\text{isf}(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.SmmPdf(x, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.SmmPdf(x, k, nu)
ereal('5.3518479027559984754E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.SmmPdf(x, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.SmmPdf(x, k, nu)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.SmmCdf(x, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.SmmCdf(x, k, nu)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.SmmQtf(q, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.SmmQtf(q, k, nu)
ereal('5.3518479027559984754E-1')
```

33.1.8 Dunnett t-distribution, 1-sided: pdf, cdf, qtf, boost class

math53lib.**Dunnett1Pdf**(*x*, *rho*, *k*, *nu*)

math53lib.**Dunnett1Cdf**(*x*, *rho*, *k*, *nu*)

math53lib.**Dunnett1Qtf**(*q*, *rho*, *k*, *nu*)

math53lib.**Dunnett1Dist**(*rho*, *k*, *nu*)

Returns the pdf, cdf, qtf or boost class of a random variable X, of a random variable X, following a 1-sided Dunnett t-distribution with common correlation ρ, $k \geq 2$ groups (including control group), error degrees of freedom ν, and the support interval $(-\infty, +\infty)$, and $0 \leq q \leq 1$. See also Dunnett [292], Bechhofer and Dunnett [26].

$$\text{pdf}(x) = \int_0^\infty f_{\text{nmaxrho}}(sx, \rho, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $f_{\text{nmaxrho}}(\cdot, \rho, k)$ is the pdf of the normal maximum (equicorrelated case) with common correlation ρ and k groups (see *nmax_rho_pdf()*), and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{cdf}(x) = \int_0^\infty F_{\text{nmaxrho}}(sx, \rho, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $F_{\text{nmaxrho}}(\cdot, \rho, k)$ is the cdf of the normal maximum (equicorrelated case) with common correlation ρ and k groups (see *nmax_rho_cdf()*), and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{sf}(x) = 1 - \text{cdf}(x)$$

There is no known explicit expression for $\text{qtf}(q)$ or $\text{isf}(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Dunnett1Pdf(x, rho, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.Dunnett1Pdf(x, rho, k, nu)
ereal('5.3518479027559984754E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.Dunnett1Pdf(x, rho, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.Dunnett1Pdf(x, rho, k, nu)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.Dunnett1Cdf(x, rho, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.Dunnett1Cdf(x, rho, k, nu)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.Dunnett1Cdf(q, rho, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.Dunnett1Cdf(q, rho, k, nu)
ereal('5.3518479027559984754E-1')
```

33.1.9 Dunnett t-distribution, 2-sided: pdf, cdf, qtf, boost class

`math53lib.`**`Dunnett2Pdf`**(*x*, *rho*, *k*, *nu*)

`math53lib.`**`Dunnett2Cdf`**(*x*, *rho*, *k*, *nu*)

`math53lib.`**`Dunnett2Qtf`**(*q*, *rho*, *k*, *nu*)

`math53lib.`**`Dunnett2Dist`**(*rho*, *k*, *nu*)

Returns the pdf, cdf, qtf or boost class of a random variable X, following a 2-sided Dunnett t-distribution with common correlation ρ, $k \geq 2$ groups (including control group), error degrees of freedom ν, and the support interval $(0, +\infty)$, and $0 \leq q \leq 1$. See also Dunnett [292], Bechhofer and Dunnett [26].

$$\text{pdf}(x) = \int_0^\infty f_{\text{nmmrho}}(sx, \rho, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $f_{\text{nmmrho}}(\cdot, \rho, k)$ is the pdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups (see *nmm_rho_pdf()*), and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{cdf}(x) = \int_0^\infty F_{\text{nmmrho}}(sx, \rho, k)\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $F_{\text{nmmrho}}(\cdot, \rho, k)$ is the cdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups (see *nmm_rho_cdf()*), and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{sf}(x) = 1 - \text{cdf}(x)$$

There is no known explicit expression for $\text{qtf}(q)$ or $\text{isf}(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.Dunnett2Pdf(x, rho, k, nu)
ereal('5.235987755982988730 7E-1')
>>> math53.Dunnett2Pdf(x, rho, k, nu)
ereal('5.351847902755998475 4E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.Dunnett2Pdf(x, rho, k, nu)
ereal('5.235987755982988730 7E-1')
>>> math53.Dunnett2Pdf(x, rho, k, nu)
ereal('5.351847902755998475 4E-1')

>>> from xlcalcnet import math53
>>> math53.Dunnett2Cdf(x, rho, k, nu)
ereal('5.235987755982988730 7E-1')
>>> math53.Dunnett2Cdf(x, rho, k, nu)
ereal('5.351847902755998475 4E-1')

>>> from xlcalcnet import math53
>>> math53.Dunnett2Qtf(q, rho, k, nu)
ereal('5.235987755982988730 7E-1')
>>> math53.Dunnett2Qtf(q, rho, k, nu)
ereal('5.351847902755998475 4E-1')
```

33.1.10 Nair t-distribution: pdf, cdf, qtf, boost class

math53lib.**NairPdf**(*x*, *rho*, *k*, *nu*)

math53lib.**NairCdf**(*x*, *rho*, *k*, *nu*)

math53lib.**NairQtf**(*q*, *rho*, *k*, *nu*)

math53lib.**NairDist**(*rho*, *k*, *nu*)

Returns the pdf, cdf, qtf or boost class of a random variable X, of a random variable X, following the Nair t-distribution with common correlation ρ, $k \geq 2$ groups (including control group), error degrees of freedom ν, and the support interval $(-\infty, +\infty)$, and $0 \leq q \leq 1$. See also Dunnett [292], Bechhofer and Dunnett [26].

$$\text{pdf}(x) = \int_0^\infty f_{\text{nmaxrho}}(sx, \rho, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $f_{\text{nmaxrho}}(\cdot, \rho, k)$ is the pdf of the normal maximum (equicorrelated case) with common correlation ρ and k groups (see *nmax_rho_pdf()*), and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{cdf}(x) = \int_0^\infty F_{\text{nmaxrho}}(sx, \rho, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $F_{\text{nmaxrho}}(\cdot, \rho, k)$ is the cdf of the normal maximum (equicorrelated case) with common correlation ρ and k groups (see *nmax_rho_cdf()*), and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{sf}(x) = 1 - \text{cdf}(x)$$

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.NairPdf(x, rho, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.NairPdf(x, rho, k, nu)
ereal('5.3518479027559984754E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.NairPdf(x, rho, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.NairPdf(x, rho, k, nu)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.NairCdf(x, rho, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.NairCdf(x, rho, k, nu)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.NairQtf(q, rho, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.NairQtf(q, rho, k, nu)
ereal('5.3518479027559984754E-1')
```

33.1.11 Halperin t-distribution: pdf, cdf, qtf, boost class

`math53lib.`**`HalperinPdf`**(*x*, *rho*, *k*, *nu*)

`math53lib.`**`HalperinCdf`**(*x*, *rho*, *k*, *nu*)

`math53lib.`**`HalperinQtf`**(*q*, *rho*, *k*, *nu*)

`math53lib.`**`HalperinDist`**(*rho*, *k*, *nu*)

Returns the pdf, cdf, qtf or boost class of a random variable X, following a Halperin t-distribution with common correlation ρ, $k \geq 2$ groups (including control group), error degrees of freedom ν, and the support interval $(0, +\infty)$, and $0 \leq q \leq 1$. See also Dunnett [292], Bechhofer and Dunnett [26].

$$\text{pdf}(x) = \int_0^\infty f_{\text{nmmrho}}(sx, \rho, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $f_{\text{nmmrho}}(\cdot, \rho, k)$ is the pdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups (see *nmm_rho_pdf()*), and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{cdf}(x) = \int_0^\infty F_{\text{nmmrho}}(sx, \rho, k)\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $F_{\text{nmmrho}}(\cdot, \rho, k)$ is the cdf of the normal maximum modulus (equicorrelated case) with common correlation ρ and k groups (see *nmm_rho_cdf()*), and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{sf}(x) = 1 - \text{cdf}(x)$$

There is no known explicit expression for $\text{qtf}(q)$ or $\text{isf}(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.HalperinPdf(x, rho, k, nu)
ereal('5.235987755982988730 7E-1')
>>> math53.HalperinPdf(x, rho, k, nu)
ereal('5.351847902755998475 4E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.HalperinPdf(x, rho, k, nu)
ereal('5.235987755982988730 7E-1')
>>> math53.HalperinPdf(x, rho, k, nu)
ereal('5.351847902755998475 4E-1')

>>> from xlcalcnet import math53
>>> math53.HalperinCdf(x, rho, k, nu)
ereal('5.235987755982988730 7E-1')
>>> math53.HalperinCdf(x, rho, k, nu)
ereal('5.351847902755998475 4E-1')

>>> from xlcalcnet import math53
>>> math53.HalperinQtf(q, rho, k, nu)
ereal('5.235987755982988730 7E-1')
>>> math53.HalperinQtf(q, rho, k, nu)
ereal('5.351847902755998475 4E-1')
```

33.1.12 Studentized range distribution: pdf, cdf, qtf, boost class

`math53lib.StdrangePdf`(*x*, *k*, *nu*)

`math53lib.StdrangeCdf`(*x*, *k*, *nu*)

`math53lib.StdrangeQtf`(*q*, *k*, *nu*)

`math53lib.StdrangeDist`(*k*, *nu*)

Returns the pdf, cdf, qtf or boost class of a random variable X, following a studentized range distribution, with $k \geq 2$ groups, ν error degrees of freedom, and the support interval $(0, +\infty)$, and $0 \leq q \leq 1$. See also Wikipedia [1218], Harter [364], Team [535].

$$\text{pdf}(x) = \int_0^\infty f_{\text{nrange}}(sx, k) \cdot s\sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $f_{\text{nrange}}(\cdot, k)$ is the pdf of the normal range with k groups (see *nrange_pdf()*), and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{cdf}(x) = \int_0^\infty F_{\text{nrange}}(sx, k) \cdot \sqrt{\nu} \cdot f_\chi\left(s\sqrt{\nu}, \nu\right)\ ds$$

where $F_{\text{nrange}}(\cdot, k)$ is the cdf of the normal range with k groups (see *nrange_cdf()*), and $f_\chi(\cdot, \nu)$ is the pdf of the χ-distribution with ν degrees of freedom.

$$\text{sf}(x) = 1 - \text{cdf}(x)$$

There is no known explicit expression for $\text{qtf}(q)$ or $\text{isf}(q)$: These functions are computed with Newton iterations where the starting values are from the corresponding Boost functions (in double precision).

An example in Python

```
>>> from xlcalcnet import math53
>>> math53.StdrangePdf(x, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.StdrangePdf(x, k, nu)
ereal('5.3518479027559984754E-1')
```

```
>>> from xlcalcnet import math53
>>> math53.StdrangePdf(x, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.StdrangePdf(x, k, nu)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.StdrangeCdf(x, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.StdrangeCdf(x, k, nu)
ereal('5.3518479027559984754E-1')

>>> from xlcalcnet import math53
>>> math53.StdrangeQtf(q, k, nu)
ereal('5.2359877559829887307E-1')
>>> math53.StdrangeQtf(q, k, nu)
ereal('5.3518479027559984754E-1')
```

33.2 Discrete (lattice) distribution functions related to (stratified) rank tests

33.2.1 Kendall S (or tau) distribution (under H_0), pmf vector

`math53lib.KendallTauPmfVector(x, n, lambda)`

Returns the vector of all pmf values of the Kendall S (or tau) distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix. See also Wikipedia [1182], Noether [424], vandeWiel [789].

Returns pmf(x), the probability mass function (pmf) of a random variable X, following a Kendall tau distribution.

The null distribution can be calculated as follows: Let $p_N(t) = \Pr[T_N = t]$. Then the following recurrence relation holds:

$$p_N(t) = p_N(t-1) + [p_{N-1}(t) - p_{N-1}(t-N)]/N,$$

where $p_N(t) = 0$ for $t < 0$ or $t > N(N-1)/2$, and $p_N(0) = 1/N!$.

33.2.2 Mann-Whitney U distribution (under H_0), pmf vector

`math53lib.MannWhitneyUPmfVector(x, n, lambda)`

Returns the vector of all pmf values of the Mann-Whitney U distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1185], Murakami and Kamakura [420], Robillard [482], [vandeWiel2000` and Zimmermann [1502].

Returns pmf(x), the probability mass function (pmf) of a random variable X, following a Mann-Whitney U distribution. The null distribution of the MW test can be calculated as follows: Let $p_{n,m}(u)$ denote the probability that $U = u$ in samples of size n and m. Then (see Zimmermann [1502])

$$(m+n)p_{n,m}(u) = np_{n-1,m}(u-m) + mp_{n,m-1}(u) = np_{n-1,m}(u) + mp_{n,m-1}(u-n)$$

33.2.3 Jonckheere-Terpsta S distribution (under H_0), pmf vector

`math53lib.JTerpstaSPmfVector(x, n, lambda)`

Returns the vector of all pmf values of the Jonckheere-Terpsta S distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], Murakami and Kamakura [420], Robillard [482], [vandeWiel2000` and Skillings [501].

Returns pmf(x), the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t-x)$$

where the sum is over all x with positive $p(\cdot)$.

33.2.4 Spearman ρ distribution (under H_0), pmf vector

`math53lib.SpearmanRhoPmfVector(x, n, lambda)`

Returns the vector of all pmf values of the Page L distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], [vandeWiel2000` and Skillings [501] .

Returns pmf(x), the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

33.2.5 Sign test distribution (under H_0), pmf vector

`math53lib.SigntestPmfVector(x, n, lambda)`

Returns the vector of all pmf values of the Sign test distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1195], [vandeWiel2000`, Bennett [29] and Zimmermann [1501].

Returns pmf(x), the probability mass function (pmf) of a random variable X, following a Wilcoxon signed rank T distribution. The null distribution can be calculated as follows: Let $p_N(w)$ denote the probability $\Pr[W_N = w]$ in a sample of size N. Then the following recurrence relation holds (see Zimmermann [1501]) :

$$p_N(w) = \tfrac{1}{2} \left(p_{N-1}(w) + p_{N-1}(w - N) \right).$$

33.2.6 Wilcoxon T distribution (under H_0), pmf vector

`math53lib.WilcoxonPmfVector(x, n, lambda)`

Returns the vector of all pmf values of the Wilcoxon distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1195], Fellingham and Stoker [300], [vandeWiel2000`, Bennett [29] and Zimmermann [1501].

Returns pmf(x), the probability mass function (pmf) of a random variable X, following a Wilcoxon signed rank T distribution. The null distribution can be calculated as follows: Let $p_N(w)$ denote the probability $\Pr[W_N = w]$ in a sample of size N. Then the following recurrence relation holds (see Zimmermann [1501]) :

$$p_N(w) = \tfrac{1}{2} \left(p_{N-1}(w) + p_{N-1}(w - N) \right).$$

33.2.7 Page L distribution (under H_0), pmf vector

`math53lib.PageLPmfVector(x, n, lambda)`

Returns the vector of all pmf values of the Page L distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], [vandeWiel2000` and Skillings [501].

Returns pmf(x), the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$:

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

33.2.8 Quade L distribution (under H_0), pmf vector

`math53lib.QuadeLPmfVector`(*x*, *n*, *lambda*)

Returns the vector of all pmf values of the Quade L distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], [vandeWiel2000` and Skillings [501].

Returns pmf(x), the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$:

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

33.3 Discrete (non-lattice) distribution functions related to rank tests

33.3.1 Cochran S distribution (under H_0), pmf vector

math53lib.**cochran_s_pmf_vector**(*x*, *n*, *lambda*)

Returns the vector of all pmf values of the Cochran S distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], vandeWiel [789], and Skillings [501].

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

33.3.2 Friedman S distribution (under H_0), pmf vector

math53lib.**friedman_s_pmf_vector**(*x*, *n*, *lambda*)

Returns the vector of all pmf values of the Friedman S distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also: https://www.statsdirect.com/help/nonparametric_methods/friedman.htm

See also Wikipedia [1181], vandeWiel [789] and Skillings [501].

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

33.3.3 Quade S distribution (under H_0), pmf vector

math53lib.**quade_s_pmf_vector**(*x*, *n*, *lambda*)

Returns the vector of all pmf values of the Quade S distribution distribution under H_0. The vector is returned as a nested list of Decimals or as a mp.matrix or as a iv.matrix.

See also Wikipedia [1181], vandeWiel [789], and Skillings [501] .

Returns $\text{pmf}_X(x)$, the probability mass function (pmf) of a random variable X, following a Jonckheere-Terpsta S distribution. Let $p(n_1, \ldots, n_k; t) = \Pr[J_N = t]$. If J_N is based on k independent samples of sizes $n_1, \ldots, n_k$, then (Skillings 1980):

$$p(n_1, \ldots, n_k; t) = \sum_x p(n_1, \ldots, n_k; x) \times p(n_1, \ldots, n_k; t - x)$$

where the sum is over all x with positive $p(\cdot)$.

CHAPTER

THIRTYFOUR

INFERENTIAL STATISTICS

General concepts

See also: https://en.wikipedia.org/wiki/Power_of_a_test

See also: https://en.wikipedia.org/wiki/Sample_size_determination

34.1 Basic classical statistical tests (stratified)

34.1.1 Student t-test for 1 sample: tests (p-values) and confidence intervals

`ctx.student_t_1sample_test`(*n*, *mu0*, *mean*, *stdev*, *alpha*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns tests and/or confidence intervals for Student's t-test for 1 sample with sample size *N* (N), reference mean *mu0* (μ_0), sample mean *mean* ($\overline{x}_1$), sample standard deviation *stdev* (s) and type I error *alpha* (α). See also: Wikipedia [1440].

The following boolean keyword arguments determine the output:

I: if `True`, the input parameters are shown.

D: if `True`, the descriptive statistics is shown.

T: if `True`, the tests are shown.

C: if `True`, the confidence intervals are shown.

Onesided: if `True`, onesided tests and/or confidence intervals are shown.

Twosided: if `True`, onesided tests and/or confidence intervals are shown.

Examples

- Skellam: poisson with same rate back to back
- Laplace: exponential with same rate back to back
- Asymmetric Laplace: exponential with different rates back to back
- Hyperexponential distribution

Examples

1. Skellam: poisson with same rate back to back
2. Laplace: exponential with same rate back to back
3. Asymmetric Laplace: exponential with different rates back to back
4. Hyperexponential distribution

Let $(X_1, X_2, \ldots, X_N)$ denote a random sample of size N from a normal distribution with mean μ and variance σ^2, and let

$$\overline{x}_1 = \frac{1}{N}\sum_{i=1}^{N} X_i \quad \text{and } s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(X_i - \overline{x}_1)^2$$

be the usual sample estimates of the unkown population mean μ and unkown population variance σ^2. Then Student's t-test can be used to test hypotheses concerning μ with regard to a reference value μ_0.

Let $F_t(\cdot, \nu)$ denote the CDF and let $t_{\nu,\alpha}$ denote the α-quantile of the t-distribution with ν degrees of freedom. Define

$$t = \frac{\overline{x}_1 - \mu_0}{s}, \quad s = \sqrt{s_1^2/N}, \quad \nu = N - 1.$$

The test can also be expressed in terms of a correlation coefficient r between the combined X and an indicator variable, where t and r are related by

$$r = \frac{t}{\sqrt{t^2 + \nu}}, \quad t = \nu\frac{r}{1 - r^2}.$$

Then p-values and rejection criteria for H_0 can be calculated as summarized below:

Test problem	p-value	Reject H_0
$H_{01} : \mu \leq \mu_0$ vs $H_{A1} : \mu > \mu_0$	$F_t(-t, \nu)$	$t > t_{\nu;1-\alpha}$
$H_{02} : \mu \geq \mu_0$ vs $H_{A2} : \mu < \mu_0$	$F_t(t, \nu)$	$t > t_{\nu;\alpha}$
$H_{03} : \mu = \mu_0$ vs $H_{A3} : \mu \neq \mu_0$	$F_t(t, \nu) - F_t(-t, \nu)$	$t > t_{\nu;1-\alpha/2}$ or $t > t_{\nu;\alpha/2}$

Let $A_1 = t_{\nu,\alpha} \cdot s$ and $A_2 = t_{\nu,\alpha/2} \cdot s$. Then the confidence intervals at confidence level $1 - \alpha$ can be calculated as follows:

Type	Confidence Interval (Difference of Means)
Left-sided	$-\infty \leq \mu_1 - \mu_0 \leq (\overline{x}_1 - \mu_0) + A_1$
Right-sided	$(\overline{x}_1 - \mu_0) - A_1 \leq \mu_1 - \mu_0 \leq +\infty$
Two-sided	$(\overline{x}_1 - \mu_0) - A_2 \leq \mu_1 - \mu_0 \leq (\overline{x}_1 - \mu_0) + A_2$

Examples

An actual call to the function, requesting Student's t-test for 1 independent sample of size 10 and standard deviation 1 each, with means 2.3 and reference mean = 1.0, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> n = 16; mu0 = 4.05; mean = 5.24; stdev = 1.5; alpha=0.05
>>> mpm.student_t_1sample_test(n, mu0, mean, stdev, alpha, \
  I=True, D=True, T=True, C=True, Onesided=True, Twosided = True)
Student t-test for 1 sample: tests and confidence intervals
                          Parameter  Variable1
                                  n: [16.0]
                               mean: [5.24]
                                mu0: [4.05]
                              stdev: [1.5]
                              alpha: [0.05]
                 degrees of freedom: [15.0]
               difference of means: [1.19]
```

(continues on next page)

(continued from previous page)

```
       rho-tilde = (mean-mu0)/stdev: [0.7933333]
                   t-value (=delta): [3.173333]
                t(1-alpha, 1-sided): [1.75305]
                t(1-alpha, 2-sided): [2.13145]
    test, p-value (H01: mu1 >= mu0): [0.9968508]
    test, p-value (H02: mu1 <= mu0): [0.003149208]
     test, p-value (H03: mu1 = mu2): [0.006298416]
mu1 - mu0, CI upper limit (1-sided): [1.847394]
mu1 - mu0, CI lower limit (1-sided): [0.5326061]
mu1 - mu0, CI upper limit (2-sided): [1.989294]
mu1 - mu0, CI lower limit (2-sided): [0.3907064]
     mu1 - mu0, CI-length (2-sided): [1.598587]
```

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> n = [10, 20, 30]; mu0 = 1.0; mean = [4.5,4.6]; stdev = [1,2,3,4]; alpha=0.015
>>> mpm.student_t_1sample_test(n, mu0, mean, stdev, alpha, \
  I=True, D=True, T=True, C=True, Onesided=True, Twosided = True)
Student t-test for 1 sample: tests and confidence intervals
                          Parameter  Variable1
                                  n: [10.0 20.0 30.0 30.0]
                               mean: [4.5 4.6 4.6 4.6]
                                mu0: [1.0 1.0 1.0 1.0]
                              stdev: [1.0 2.0 3.0 4.0]
                              alpha: [0.015 0.015 0.015 0.015]
                 degrees of freedom: [9.0, 19.0, 29.0, 29.0]
                difference of means: [3.5, 3.6, 3.6, 3.6]
       rho-tilde = (mean-mu0)/stdev: [3.5, 1.8, 1.2, 0.9]
                   t-value (=delta): [11.06797, 8.049845, 6.572671, 4.929503]
                t(1-alpha, 1-sided): [2.573804, 2.345648, 2.282175, 2.282175]
                t(1-alpha, 2-sided): [2.998203, 2.674209, 2.585992, 2.585992]
    test, p-value (H01: mu1 >= mu0): [0.9999992, 0.9999999, 0.9999998, 0.9999846]
    test, p-value (H02: mu1 <= mu0): [7.642012e-7, 7.64034e-8, 1.67689e-7, 1.542853e-5]
     test, p-value (H03: mu1 = mu2): [1.528402e-6, 1.528068e-7, 3.35378e-7, 3.085705e-5]
mu1 - mu0, CI upper limit (1-sided): [4.313908, 4.649005, 4.849998, 5.266665]
mu1 - mu0, CI lower limit (1-sided): [2.686092, 2.550995, 2.350002, 1.933335]
mu1 - mu0, CI upper limit (2-sided): [4.448115, 4.795942, 5.016406, 5.488542]
mu1 - mu0, CI lower limit (2-sided): [2.551885, 2.404058, 2.183594, 1.711458]
     mu1 - mu0, CI-length (2-sided): [1.89623, 2.391885, 2.832812, 3.777083]
```

34.1.2 Student t-test for 1 sample: power calculations

`ctx.student_t_1sample_power`(*n*, *mu0*, *mu1*, *sigma*, *alpha*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns power calculations for Student's t-test for 1 sample with sample size *N* (N), reference mean *mu0* (μ_0), population mean *mu1* (μ_1), population standard deviation *sigma* (σ) and type I error *alpha* (α). See also: Wikipedia [1440].

The following boolean keyword arguments determine the output:

I: if `True`, the input parameters are shown.

D: if `True`, the descriptive statistics is shown.

P: if `True`, the power calculations are shown.

E: if `True`, some extra calculations are shown.

Onesided: if `True`, onesided tests are shown.

Twosided: if `True`, twosided tests are shown.

Let $\sigma_1^2 = \sigma^2$ and $\nu = N - 1$ and define $\widetilde{\rho} = \dfrac{\mu_1 - \mu_0}{\sigma}$ and $\delta = \sqrt{N}\widetilde{\rho}$. Let $F_{t'}(\cdot, \nu, \delta)$ denote the CDF of the noncentral t-distribution with ν degrees of freedom and noncentrality parameter δ and let $t_{\nu,\alpha}$ denote the α-quantile of the central t-distribution with ν degrees of freedom. Then the power for accepting H_A at the confidence level α can be calculated as summarized below:

Test	Null Hypothesis	Alternative	Power
onesided	$H_{01} : \mu \leq \mu_0$	$H_{A1} : \mu > \mu_0$	$F_{t'}\left(-t_{\nu;1-\alpha}, \nu, \delta\right)$
onesided	$H_{02} : \mu \geq \mu_0$	$H_{A2} : \mu < \mu_0$	$F_{t'}\left(t_{\nu;1-\alpha}, \nu, \delta\right)$
twosided	$H_{03} : \mu = \mu_0$	$H_{A1} : \mu > \mu_0$	$F_{t'}\left(-t_{\nu;1-\alpha/2}, \nu, \delta\right)$
twosided	$H_{03} : \mu = \mu_0$	$H_{A2} : \mu < \mu_0$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right)$
twosided	$H_{03} : \mu = \mu_0$	$H_{A3} : \mu \neq \mu_0$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right) - F_t\left(-t_{\nu;1-\alpha/2}, \nu\,\delta\right)$

Examples

An actual call to the function, requesting Student's t-test with description, the critical calue for a two-sided test, the power for H_{A3} for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> n = 56; mu0 = 4.05; mu1 = 5.24; sigma = 1.5; alpha=0.05
>>> mpm.student_t_1sample_power(n, mu0, mu1, sigma, alpha, \
  I=True, D=True, T=True, C=True, Onesided=True, Twosided = True)
Student t-test for 1 sample: power
                           Parameter  variable1
                                   n: [56.0]
                                 mu0: [4.05]
                                 mu1: [5.24]
                               sigma: [1.5]
                               alpha: [0.05]
                  degrees of freedom: [55.0]
                           mu1 - mu0: [1.19]
                                 rho: [0.6249392]
                               delta: [5.936763]
                t(1-alpha, 1-sided): [1.673034]
```

(continues on next page)

(continued from previous page)

```
                 t(1-alpha, 2-sided): [2.004045]
 1-sided test, power (HA1: mu1 < mu2): 7.46e-08
 1-sided test, power (HA2: mu1 > mu2): 0.974564
 2-sided test, power (HA1: mu1 < mu2): 1.65e-08
 2-sided test, power (HA2: mu1 > mu2): 0.943648
2-sided test, power (HA1: mu1 <> mu2): 0.943648
                   Pr(Mean1 < Mean2): 9.92e-05
                   Pr(Mean1 > Mean2): 0.999901
```

Another example

```
>>> from xlcalcnet import mpm
>>> n = [10, 20, 30]; mu0 = 1.0; mu1 = [4.5,4.6]; sigma = [1,2,3,4]; alpha=0.015
>>> mpm.student_t_1sample_power(n, mu0, mu1, sigma, alpha, \
  I=True, D=True, T=True, C=True, Onesided=True, Twosided = True)
Student t-test for 1 sample: power
                           Parameter  variable1
                                   n: [10.0 20.0 30.0 30.0]
                                 mu0: [1.0 1.0 1.0 1.0]
                                 mu1: [4.5 4.6 4.6 4.6]
                               sigma: [1.0 2.0 3.0 4.0]
                               alpha: [0.015 0.015 0.015 0.015]
                 degrees of freedom: [9.0, 19.0, 29.0, 29.0]
                           mu1 - mu0: [3.5, 3.6, 3.6, 3.6]
                                 rho: [0.965173, 0.8793576, 0.7735231, 0.675211]
                               delta: [11.06797, 8.049845, 6.572671, 4.929503]
                t(1-alpha, 1-sided): [2.573804, 2.345648, 2.282175, 2.282175]
                t(1-alpha, 2-sided): [2.998203, 2.674209, 2.585992, 2.585992]
 1-sided test, power (HA1: mu1 < mu2): 7.46e-08
 1-sided test, power (HA2: mu1 > mu2): 0.974564
 2-sided test, power (HA1: mu1 < mu2): 1.65e-08
 2-sided test, power (HA2: mu1 > mu2): 0.943648
2-sided test, power (HA1: mu1 <> mu2): 0.943648
                   Pr(Mean1 < Mean2): 9.92e-05
                   Pr(Mean1 > Mean2): 0.999901
```

34.1.3 Student t-test for 1 sample: sample size calculation

`ctx.student_t_1sample_samplesize`(*mu0*, *mu1*, *sigma*, *alpha*, *beta*, ***kwargs*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns sample size calculations for Student's t-test for 1 sample with reference mean *mu0* (μ_0), population mean *mu1* (μ_1), population standard deviation *sigma* (σ), type I error *alpha* (α), and type II error *beta* (β). See also: Wikipedia [1440].

The following boolean keyword arguments determine the output:

I: if `True`, the input parameters are shown.

D: if `True`, the descriptive statistics is shown.

N: if `True`, the sample size calculations are shown.

P: if `True`, the actual power (for the calculated sample size) is shown.

Onesided: if `True`, onesided tests are shown.

Twosided: if `True`, twosided tests are shown.

Let $N_{t'}(\alpha, \beta, \widetilde{\rho})$ denote the sample size function of the (singly) noncentral t-distribution for a given type I error α, type II error β and noncentrality parameter $\widetilde{\rho} = \dfrac{\mu_1 - \mu_0}{\sigma}$. The required total sample size N can be calculated as summarized below. Note that the returned value of N will in general not be an integer, and rounding up may be required.

Test	Null Hypothesis	Alternative	Minimal sample size
onesided	$H_{01} : \mu \leq \mu_0$	$H_{A1} : \mu > \mu_0$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
onesided	$H_{02} : \mu \geq \mu_0$	$H_{A2} : \mu < \mu_0$	$N_{t'}(\alpha, \beta, -\widetilde{\rho})$
twosided	$H_{03} : \mu = \mu_0$	$H_{A1} : \mu > \mu_0$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
twosided	$H_{03} : \mu = \mu_0$	$H_{A2} : \mu < \mu_0$	$N_{t'}(\alpha, \beta, -\widetilde{\rho})$
twosided	$H_{03} : \mu = \mu_0$	$H_{A3} : \mu \neq \mu_0$	$N^{(2)}_{t'}(\alpha, \beta, \widetilde{\rho})$

Examples

An actual call to the function, requesting an upper sample size estimate (and actual power) for $\alpha = 0.95$, $\beta = 0.1$, and standard deviations $\sigma_1 = \sigma_2 = 1$, means $\mu_1 = 2.3$ and $\mu_2 = 4.5$, would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, alpha:=0.05, beta:=0.1)
                                    df: 21
                        difference of means: 1.19
                           t-value (=delta): 3.721063
                        t(1-alpha, 1-sided): 1.720743
                        t(1-alpha, 2-sided): 2.079614
  1-sided test, required N (HA1: mu1 < mu2): 18
1-sided test, actual power (HA1: mu1 < mu2): 0.974564
  1-sided test, required N (HA2: mu1 > mu2): 148
1-sided test, actual power (HA2: mu1 > mu2): 0.964564
  2-sided test, required N (HA1: mu1 < mu2): 22
2-sided test, actual power (HA1: mu1 < mu2): 0.954564
  2-sided test, required N (HA2: mu1 > mu2): 212
2-sided test, actual power (HA2: mu1 > mu2): 0.977456
  2-sided test, required N (HA2: mu1 <>mu2): 24
2-sided test, actual power (HA2: mu1 <>mu2): 0.955544
```

34.1.4 Chi-squared-test for the variance of 1 sample: tests (p-values)

`ctx.chi2_variance_1sample_test`(*x*, *k*, *n*, *method='default'*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Text

Then p-values and rejection criteria for H_0 can be calculated as summarized below

Test problem	p-value	Reject H_0
$H_{01} : \sigma \leq \sigma_0$ vs $H_{A1} : \sigma > \sigma_0$	$F_t\left(-t, \nu\right)$	$t > t_{\nu;1-\alpha}$
$H_{02} : \sigma \geq \sigma_0$ vs $H_{A2} : \sigma < \sigma_0$	$F_t\left(t, \nu\right)$	$t > t_{\nu;\alpha}$
$H_{03} : \sigma = \sigma_0$ vs $H_{A3} : \sigma \neq \sigma_0$	$F_t\left(t, \nu\right) - F_t\left(-t, \nu\right)$	$t > t_{\nu;1-\alpha/2}$ or $t > t_{\nu;\alpha/2}$

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1Test(mean:=5.24, mean0:=4.05, sd:=1.5, n:=22, resultstring:='All')

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5

Chi2-test for the variance of 1 sample
Parameter Result
df 21
Variance-Ratio 0.183673469
Chi2-value 3.857142857
Chi2. 1 - alpha(1 - sided) 10.28289778
Chi2 . 1 - alpha(2 - sided) 35.47887591
Chi2-test. p-value (H01: s1 >= s0) 1.44636E-05
Chi2-test. p-value (H02: s1 <= s0) 0.999985536
Chi2-test. p-value (H03: s1 = s0) 2.89273E-05
```

34.1.5 Chi-squared-test for the variance of 1 sample: confidence intervals

`ctx.chi2_variance_1sample_ci`(*x*, *k*, *n*, *method='default'*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Text

Then confidence intervals can be calculated as summarized below

Type	Confidence Interval (Difference of Means)
Left-sided	$-\infty \leq \sigma_1 - \sigma_0 \leq (\overline{x}_1 - \sigma_0) + A_1$
Right-sided	$(\overline{x}_1 - \sigma_0) - A_1 \leq \sigma_1 - \sigma_0 \leq +\infty$
Two-sided	$(\overline{x}_1 - \sigma_0) - A_2 \leq \sigma_1 - \sigma_0 \leq (\overline{x}_1 - \sigma_0) + A_2$

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1Test(mean:=5.24, mean0:=4.05, sd:=1.5, n:=22, resultstring:='All')

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
Type 1 Error 0.05

Chi2-test for the variance of 1 sample. CI
Parameter Result
df 21
Variance-Ratio 0.183673469
Chi2-value 3.857142857
Chi2. 1 - alpha(1 - sided) 10.28289778
Chi2 . 1 - alpha(2 - sided) 35.47887591
s1. CI - Length(2 - sided) 3.263229852
s1. CI Upper Limit (2-sided) 4.595008236
s1. CI Lower Limit (2-sided) 1.331778383
```

34.1.6 Chi-squared-test for the variance of 1 sample: power

`ctx.chi2_variance_1sample_power`(*x*, *k*, *n*, *method='default'*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns the results of the Chi-squared-test for the variance of 1 sample, under H_0.

Then the power for accepting H_A at the confidence level α can be calculated as summarized below:

Test	Null Hypothesis	Alternative	Power
1 sided	$H_{01} : \sigma \leq \sigma_0$	$H_{A1} : \sigma > \sigma_0$	$F_{t'}\left(-t_{\nu;1-\alpha}, \nu, \delta\right)$
1 sided	$H_{02} : \sigma \geq \sigma_0$	$H_{A2} : \sigma < \sigma_0$	$F_{t'}\left(t_{\nu;1-\alpha}, \nu, \delta\right)$
2 sided	$H_{03} : \sigma = \sigma_0$	$H_{A1} : \sigma > \sigma_0$	$F_{t'}\left(-t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \sigma = \sigma_0$	$H_{A1} : \sigma > \sigma_0$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \sigma = \sigma_0$	$H_{A3} : \sigma \neq \sigma_0$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right) - F_t\left(-t_{\nu;1-\alpha/2}, \nu\,\delta\right)$

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1Test(mean:=5.24, mean0:=4.05, sd:=1.5, n:=22, resultstring:='All')

Input
Variable Variable 1
Common N 22
Mean, Group 1 5,24
Mean, Group 2 4,05
StDev, Group 1 1,5
StDev, Group 2 3,5
Type 1 Error 0,05

Chi2-test for the variance of 1 sample , power
Parameter Result
df 21
Variance-Ratio 0,183673469
Chi2-value 3,857142857
Chi2, 1 - alpha(1 - sided) 10,28289778
Chi2 , 1 - alpha(2 - sided) 35,47887591
1-sided test, power (HA1: s1 < s0) 1,44636E-05
1-sided test, power (HA2: s1 > s0) 1,44636E-05
2-sided test, power (HA1: s1 < s0) 1,44636E-05
2-sided test, power (HA2: s1 > s0) 1,44636E-05
2-sided test, power (HA3: s1 <> s0) 1,44636E-05
```

34.1.7 Chi-squared-test for the variance of 1 sample: sample size

`ctx.chi2_variance_1sample_samplesize`(*x*, *k*, *n*, *method='default'*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns the results of the Chi-squared-test for the variance of 1 sampl, under H_0.

The required total sample size N can be calculated as summarized below

Test	Null Hypothesis	Alternative	Minimal sample size
1 sided	$H_{01} : \sigma \leq \sigma_0$	$H_{A1} : \sigma > \sigma_0$	$N_{t'}\left(\alpha, \beta, \widetilde{\rho}\right)$
1 sided	$H_{02} : \sigma \geq \sigma_0$	$H_{A2} : \sigma < \sigma_0$	$N_{t'}\left(\alpha, \beta, \widetilde{\rho}\right)$
2 sided	$H_{03} : \sigma = \sigma_0$	$H_{A1} : \sigma > \sigma_0$	$N_{t'}\left(\alpha, \beta, \widetilde{\rho}\right)$
2 sided	$H_{03} : \sigma = \sigma_0$	$H_{A1} : \sigma > \sigma_0$	$N_{t'}\left(\alpha, \beta, \widetilde{\rho}\right)$
2 sided	$H_{03} : \sigma = \sigma_0$	$H_{A3} : \sigma \neq \sigma_0$	$N2_{t'}\left(\alpha, \beta, \widetilde{\rho}\right)$

Note that the returned value of N will in general not be an integer, and rounding up may be required.

34.2 Basic classical statistical tests for 2 independent samples (stratified)

34.2.1 Student t-test for 2 independent samples: tests (p-values)

`ctx.studentt_2isamples_test`(*mean*, *sd*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results for Student's t-test for for 2 independent samples.

See also: https://en.wikipedia.org/wiki/Student%27s_t-test#Independent_(unpaired)_samples

Parameters:

Mean
The mean of the sample.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $(X_1, X_2, \ldots, X_N)$ denote a random sample of size N from a normal distribution with mean μ and variance σ^2, and let

$$\overline{x}_1 = \frac{1}{N}\sum_{i=1}^{N} X_i \quad \text{and } s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(X_i - \overline{x}_1)$$

be the usual sample estimates of the unkown population mean μ and unkown population variance σ^2. Then Student's t-test can be used to test hypotheses concerning μ with regard to a reference value μ_2.

Let $F_t(\cdot, \nu)$ denote the CDF (see section ref{tDistributionCDF}) and let $t_{\nu,\alpha}$ denote the α-quantile (see section ref{tDistributionQuantile}) of the t-distribution with ν degrees of freedom. Define

$$t = \frac{\overline{x}_1 - \mu_2}{s}, \quad s = \sqrt{s_1^2/N}, \quad \nu = N - 1.$$

Then p-values and rejection criteria for H_0 can be calculated as summarized below

Test problem	p-value	Reject H_0
$H_{01} : \mu_1 \leq \mu_2$ vs $H_{A1} : \mu_1 > \mu_2$	$F_t(-t, \nu)$	$t > t_{\nu;1-\alpha}$
$H_{02} : \mu_1 \geq \mu_2$ vs $H_{A2} : \mu_1 < \mu_2$	$F_t(t, \nu)$	$t > t_{\nu;\alpha}$
$H_{03} : \mu_1 = \mu_2$ vs $H_{A3} : \mu_1 \neq \mu_2$	$F_t(t, \nu) - F_t(-t, \nu)$	$t > t_{\nu;1-\alpha/2}$ or $t > t_{\nu;\alpha/2}$

The test can also be expressed in terms of a correlation coefficient r between the combined X and an indicator variable, where t and r are related by

$$r = \frac{t}{\sqrt{t^2 + \nu}}, \quad t = \nu\frac{r}{1 - r^2}.$$

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} (in the case of TTest this is $\mu_1 \neq \mu_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1Test(means:=[5.24, 4.05], sd:=1.5, n:=22)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5

Student's t-test for 2 samples
Parameter Result
df 42
Difference of means 1.19
t-value (=delta) 1.465798594
t1 - alpha(1 - sided) 1.681952357
t1 - alpha(2 - sided) 2.018081703
test. p-value (H01: μ1 >= μ2) 0.924924828
test. p-value (H02: μ1 <= μ2) 0.075075172
test. p-value (H03: μ1 = μ2) 0.150150344
```

34.2.2 Student t-test for 2 independent samples: confidence intervals

`ctx.studentt_2isamples_ci`(*mean*, *sd*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results for Student's t-test for for 2 independent samples.

Parameters:

Mean
The mean of the sample.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $A_1 = t_{\nu,\alpha} \cdot s$ and $A_2 = t_{\nu,\alpha/2} \cdot s$, where s and ν are defined in (ref{eq:TTest1}), and $t_{\nu,\alpha}$ denotes the α-quantile of the (central) t-distribution with ν degrees of freedom (see section ref{tDistributionQuantile}).

Type	Confidence Interval (Difference of Means)
Left-sided	$-\infty \leq \mu_1 - \mu_2 \leq (\overline{x}_1 - \mu_2) + A_1$
Right-sided	$(\overline{x}_1 - \mu_2) - A_1 \leq \mu_1 - \mu_2 \leq +\infty$
Two-sided	$(\overline{x}_1 - \mu_2) - A_2 \leq \mu_1 - \mu_2 \leq (\overline{x}_1 - \mu_2) + A_2$

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} (in the case of textsf{TTest} this is $\mu_1 \neq \mu_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mean:=5.24, mean0:=4.05, sd:=1.5, n:=22, alpha=0.05, resultstring)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
Type 1 Error 0.05

Student's t-test for 2 samples
Parameter Result
df 42
Difference of means 1.19
t-value (=delta) 1.465798594
t1 - alpha(1 - sided) 1.681952357
t1 - alpha(2 - sided) 2.018081703
μ1 - μ2. CI - Length (2 - sided) 3.276735613
```

(continues on next page)

(continued from previous page)

```
μ1 - μ2. CI Upper Limit (2-sided) 2.828367806
μ1 - μ2. CI Lower Limit (2-sided) -0.448367806
```

34.2.3 Student t-test for 2 independent samples: power

`ctx.studentt_2isamples_power`(*mean*, *sd*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns the results of Student's t-test for 2 independent samples: power and sample size.

Parameters:

Mean
The mean of the sample.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $\sigma_1^2 = \sigma^2$ and $\nu = N - 1$. Define

$$\widetilde{\rho} = \frac{\mu_1 - \mu_2}{\sigma} \text{ and } \delta = \sqrt{N}\widetilde{\rho}.$$

Let $F_{t'}(\cdot, \nu, \delta)$ denote the CDF of the (singly) noncentral t-distribution with ν degrees of freedom and noncentrality parameter δ and let $t_{\nu,\alpha}$ denote the α-quantile of the central t-distribution with ν degrees of freedom. Then the power for accepting H_A at the confidence level α can be calculated as summarized below:

Test	Null Hypothesis	Alternative	Power
1 sided	$H_{01} : \mu_1 \leq \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$F_{t'}\left(-t_{\nu;1-\alpha}, \nu, \delta\right)$
1 sided	$H_{02} : \mu_1 \geq \mu_2$	$H_{A2} : \mu_1 < \mu_2$	$F_{t'}\left(t_{\nu;1-\alpha}, \nu, \delta\right)$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$F_{t'}\left(-t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A3} : \mu_1 \neq \mu_2$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right) - F_t\left(-t_{\nu;1-\alpha/2}, \nu\, \delta\right)$

An actual call to the function, requesting Student's t-test with description, the critical calue for a two-sided test, the power for H_{A3} (in the case of textsf{TTest} this is $\mu_1 \neq \mu_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, n:=22, alpha:=0.05)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
```

(continues on next page)

(continued from previous page)

```
Type 1 Error 0.05

Student's t-test for 2 samples
Parameter Result
df 42
Difference of means 1.19
t-value (=delta) 1.465798594
t1 - alpha(1 - sided) 1.681952357
t1 - alpha(2 - sided) 2.018081703
1-sided test. power (HA1: μ1 < μ2) 0.001009403
1-sided test. power (HA2: μ1 > μ2) 0.419683559
2-sided test. power (HA1: μ1 < μ2) 0.000345576
2-sided test. power (HA2: μ1 > μ2) 0.298885242
2-sided test. power (HA3: μ1 <> μ2) 0.299230817
test. Pr[Mean 1 < Mean 2] 0.071351582
test. Pr[Mean 1 > Mean 2] 0.928648418
```

34.2.4 Student t-test for 2 independent samples: sample size calculation

`ctx.studentt_2isamples_samplesize`(*mu*, *sd*, *alpha=0.05*, *beta=0.1*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns the results of sample size calculations for Student's t-test for 2 independent samples

Parameters:

Mean
The mean of the sample.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Beta
The beta-level used for power

Let $N_{t'}(\alpha, \beta, \widetilde{\rho})$ denote the sample size function of the (singly) noncentral t-distribution (see section ref{NoncentralTDistributionSampleSize}) for a given confidence level α, power β and noncentrality parameter $\widetilde{\rho}$ (as defined in equation ref{eq:TTestPower1}. The required total sample size N can be calculated as summarized below

Test	Null Hypothesis	Alternative	Minimal sample size
1 sided	$H_{01} : \mu_1 \leq \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
1 sided	$H_{02} : \mu_1 \geq \mu_2$	$H_{A2} : \mu_1 < \mu_2$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A3} : \mu_1 \neq \mu_2$	$N2_{t'}(\alpha, \beta, \widetilde{\rho})$

Note that the returned value of N will in general not be an integer, and rounding up may be required.

An actual call to the function, requesting an upper sample size estimate (and actual power) for $\alpha = 0.95$, $\beta = 0.1$, and standard deviations $\sigma_1 = \sigma_2 = 1$, means $\mu_1 = 2.3$ and $\mu_2 = 4.5$, would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, alpha:=0.05, beta:=0.1)
                                      df: 21
                        difference of means: 1.19
                           t-value (=delta): 3.721063
                        t(1-alpha, 1-sided): 1.720743
                        t(1-alpha, 2-sided): 2.079614
  1-sided test, required N (HA1: mu1 < mu2): 18
1-sided test, actual power (HA1: mu1 < mu2): 0.974564
  1-sided test, required N (HA2: mu1 > mu2): 148
1-sided test, actual power (HA2: mu1 > mu2): 0.964564
```

(continues on next page)

(continued from previous page)

```
  2-sided test, required N (HA1: mu1 < mu2): 22
2-sided test, actual power (HA1: mu1 < mu2): 0.954564
  2-sided test, required N (HA2: mu1 > mu2): 212
2-sided test, actual power (HA2: mu1 > mu2): 0.977456
  2-sided test, required N (HA2: mu1 <>mu2): 24
2-sided test, actual power (HA2: mu1 <>mu2): 0.955544
```

34.2.5 Student t-test for 2 independent samples: power and sample size: failure to stratify

ctx.**studentt_2isamples_power2**(*mean*, *sd*, *n*, *alpha=0.05*)

Returns the results of the F-Test with $\delta > 0$ and $\eta > 0$

See "Butler Paolella 1999 Doubly noncentral F Draft.pdf" in "References B" for an example.

34.2.6 Student t-test for 2 independent samples: equivalence and non-inferiority

`ctx.studentt_2isamples_equivalence`(*x*, *k*, *n*, *method='default'*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns the results of the Sign test, under H_0.

Refer to Schuirman procedure.

See Shieh 2019

See also: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5012670/

34.2.7 F-test for the variances of 2 independent samples: tests (p-values)

`ctx.fratio_variance_2isamples_test`(*s2*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results for the F-test for the variances of 2 independent samples.

See also: https://en.wikipedia.org/wiki/F-test_of_equality_of_variances

Parameters:

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $(X_1, X_2, \ldots, X_N)$ denote a random sample of size N from a normal distribution with mean σ and variance σ^2, and let

$$\overline{x}_1 = \frac{1}{N}\sum_{i=1}^{N} X_i \quad \text{and } s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(X_i - \overline{x}_1)$$

be the usual sample estimates of the unkown population mean σ and unkown population variance σ^2. Then Student's t-test can be used to test hypotheses concerning σ with regard to a reference value σ_2.

Let $F_t(\cdot, \nu)$ denote the CDF (see section ref{tDistributionCDF}) and let $t_{\nu,\alpha}$ denote the α-quantile (see section ref{tDistributionQuantile}) of the t-distribution with ν degrees of freedom. Define

$$t = \frac{\overline{x}_1 - \sigma_2}{s}, \quad s = \sqrt{s_1^2/N}, \quad \nu = N - 1.$$

Then p-values and rejection criteria for H_0 can be calculated as summarized below

Test problem	p-value	Reject H_0
$H_{01} : \sigma_1 \leq \sigma_2$ vs $H_{A1} : \sigma_1 > \sigma_2$	$F_t(-t, \nu)$	$t > t_{\nu;1-\alpha}$
$H_{02} : \sigma_1 \geq \sigma_2$ vs $H_{A2} : \sigma_1 < \sigma_2$	$F_t(t, \nu)$	$t > t_{\nu;\alpha}$
$H_{03} : \sigma_1 = \sigma_2$ vs $H_{A3} : \sigma_1 \neq \sigma_2$	$F_t(t, \nu) - F_t(-t, \nu)$	$t > t_{\nu;1-\alpha/2}$ or $t > t_{\nu;\alpha/2}$

The test can also be expressed in terms of a correlation coefficient r between the combined X and an indicator variable, where t and r are related by

$$r = \frac{t}{\sqrt{t^2 + \nu}}, \quad t = \nu\frac{r}{1 - r^2}.$$

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} (in the case of TTest this is $\sigma_1 \neq \sigma_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1Test(means:=[5.24, 4.05], sd:=1.5, n:=22)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5

F-test for 2 variances
Parameter Result
df1 21
df2 21
Variance-Ratio 0.183673469
F. 1 - alpha(1 - sided) 2.084188623
F . 1 - alpha(2 - sided) 2.408589482
F-test. p-value (H01: s1 >= s2) 0.999864065
F-test. p-value (H02: s1 <= s2) 0.000135935
F-test. p-value (H03: s1 = s2) 1.999728131
```

34.2.8 F-test for the variances of 2 independent samples: confidence intervals

`ctx.fratio_variance_2isamples_ci`(*s2*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results of the confidence intervals for the F-test for the variances of 2 independent samples.

Parameters:

Mean
The mean of the sample.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $A_1 = t_{\nu,\alpha} \cdot s$ and $A_2 = t_{\nu,\alpha/2} \cdot s$, where s and ν are defined in (ref{eq:TTest1}), and $t_{\nu,\alpha}$ denotes the α-quantile of the (central) t-distribution with ν degrees of freedom (see section ref{tDistributionQuantile}).

Type	Confidence Interval (Difference of Means)
Left-sided	$-\infty \leq \sigma_1 - \sigma_2 \leq (\overline{x}_1 - \sigma_2) + A_1$
Right-sided	$(\overline{x}_1 - \sigma_2) - A_1 \leq \sigma_1 - \sigma_2 \leq +\infty$
Two-sided	$(\overline{x}_1 - \sigma_2) - A_2 \leq \sigma_1 - \sigma_2 \leq (\overline{x}_1 - \sigma_2) + A_2$

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} (in the case of textsf{TTest} this is $\sigma_1 \neq \sigma_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mean:=5.24, mean0:=4.05, sd:=1.5, n:=22, alpha=0.05, resultstring)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
Type 1 Error 0.05

F-test for 2 variances
Parameter Result
df1 21
df2 21
Variance-Ratio 0.183673469
F. 1 - alpha(1 - sided) 2.084188623
F . 1 - alpha(2 - sided) 2.408589482
s1/s2. CI - Length(2 - sided) 0.366136297
```

(continues on next page)

(continued from previous page)

```
s1/s2. CI Upper Limit (2-sided) 0.442393986
s1/s2. CI Lower Limit (2-sided) 0.07625769
```

34.2.9 F-test for the variances of 2 independent samples: power

`ctx.`**`fratio_variance_2isamples_power`**(*s2*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results of the power calculation for the F-test for the variances of 2 independent samples.

Parameters:

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $\sigma_1^2 = \sigma^2$ and $\nu = N - 1$. Define

$$\widetilde{\rho} = \frac{\sigma_1 - \sigma_2}{\sigma} \text{ and } \delta = \sqrt{N}\widetilde{\rho}.$$

Let $F_{t'}(\cdot, \nu, \delta)$ denote the CDF of the (singly) noncentral t-distribution with ν degrees of freedom and noncentrality parameter δ and let $t_{\nu,\alpha}$ denote the α-quantile of the central t-distribution with ν degrees of freedom. Then the power for accepting H_A at the confidence level α can be calculated as summarized below:

Test	Null Hypothesis	Alternative	Power
1 sided	$H_{01} : \sigma_1 \leq \sigma_2$	$H_{A1} : \sigma_1 > \sigma_2$	$F_{t'}\left(-t_{\nu;1-\alpha}, \nu, \delta\right)$
1 sided	$H_{02} : \sigma_1 \geq \sigma_2$	$H_{A2} : \sigma_1 < \sigma_2$	$F_{t'}\left(t_{\nu;1-\alpha}, \nu, \delta\right)$
2 sided	$H_{03} : \sigma_1 = \sigma_2$	$H_{A1} : \sigma_1 > \sigma_2$	$F_{t'}\left(-t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \sigma_1 = \sigma_2$	$H_{A1} : \sigma_1 > \sigma_2$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \sigma_1 = \sigma_2$	$H_{A3} : \sigma_1 \neq \sigma_2$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right) - F_t\left(-t_{\nu;1-\alpha/2}, \nu\, \delta\right)$

An actual call to the function, requesting Student's t-test with description, the critical calue for a two-sided test, the power for H_{A3} (in the case of textsf{TTest} this is $\sigma_1 \neq \sigma_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, n:=22, alpha:=0.05)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
Type 1 Error 0.05

F-test for 2 variances
```

(continues on next page)

(continued from previous page)

```
Parameter Result
df1 21
df2 21
Variance-Ratio 0.183673469
F. 1 - alpha(1 - sided) 2.084188623
F . 1 - alpha(2 - sided) 2.408589482
1-sided test. power (HA1: s1 < s2) 0.999864065
1-sided test. power (HA2: s1 > s2) 0.999864065
2-sided test. power (HA1: s1 < s2) 0.999864065
2-sided test. power (HA2: s1 > s2) 0.999864065
2-sided test. power (HA3: s1 <> s2) 0.999864065
```

34.2.10 F-test for the variances of 2 independent samples: sample size

`ctx.fratio_variance_2isamples_samplesize`(*s2*, *alpha=0.05*, *beta=0.1*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results of sample size calculations for the variances of 2 independent samples.

Parameters:

Sd
The standard deviation of the sample

Alpha
The alpha-level used for confidence intervals

Beta
The beta-level used for power

Let $N_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$ denote the sample size function of the (singly) noncentral t-distribution (see section ref{NoncentralTDistributionSampleSize}) for a given confidence level α, power β and noncentrality parameter $\widetilde{\rho}$ (as defined in equation ref{eq:TTestPower1}. The required total sample size N can be calculated as summarized below

Test	Null Hypothesis	Alternative	Minimal sample size
1 sided	$H_{01}:\sigma_1 \leq \sigma_2$	$H_{A1}:\sigma_1 > \sigma_2$	$N_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$
1 sided	$H_{02}:\sigma_1 \geq \sigma_2$	$H_{A2}:\sigma_1 < \sigma_2$	$N_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$
2 sided	$H_{03}:\sigma_1 = \sigma_2$	$H_{A1}:\sigma_1 > \sigma_2$	$N_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$
2 sided	$H_{03}:\sigma_1 = \sigma_2$	$H_{A1}:\sigma_1 > \sigma_2$	$N_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$
2 sided	$H_{03}:\sigma_1 = \sigma_2$	$H_{A3}:\sigma_1 \neq \sigma_2$	$N2_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$

Note that the returned value of N will in general not be an integer, and rounding up may be required.

An actual call to the function, requesting an upper sample size estimate (and actual power) for $\alpha = 0.95$, $\beta = 0.1$, and standard deviations $\sigma_1 = \sigma_2 = 1$, means $\sigma_1 = 2.3$ and $\sigma_2 = 4.5$, would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, alpha:=0.05, beta:=0.1)
                                    df: 21
                        difference of means: 1.19
                           t-value (=delta): 3.721063
                        t(1-alpha, 1-sided): 1.720743
                        t(1-alpha, 2-sided): 2.079614
  1-sided test, required N (HA1: mu1 < mu2): 18
1-sided test, actual power (HA1: mu1 < mu2): 0.974564
  1-sided test, required N (HA2: mu1 > mu2): 148
1-sided test, actual power (HA2: mu1 > mu2): 0.964564
  2-sided test, required N (HA1: mu1 < mu2): 22
2-sided test, actual power (HA1: mu1 < mu2): 0.954564
  2-sided test, required N (HA2: mu1 > mu2): 212
2-sided test, actual power (HA2: mu1 > mu2): 0.977456
  2-sided test, required N (HA2: mu1 <>mu2): 24
2-sided test, actual power (HA2: mu1 <>mu2): 0.955544
```

34.3 Basic classical statistical tests for 2 correlated samples (stratified)

34.3.1 Student t-test for 2 correlated samples: tests (p-values)

`ctx.studentt_2csamples_test`(*mean*, *sd*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results for Student's t-test for for 2 correlated samples.

See also: https://en.wikipedia.org/wiki/Student%27s_t-test#Paired_samples

Parameters:

Mean
The mean of the sample.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $(X_1, X_2, \ldots, X_N)$ denote a random sample of size N from a normal distribution with mean μ and variance σ^2, and let

$$\overline{x}_1 = \frac{1}{N}\sum_{i=1}^{N} X_i \quad \text{and } s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(X_i - \overline{x}_1)$$

be the usual sample estimates of the unkown population mean μ and unkown population variance σ^2. Then Student's t-test can be used to test hypotheses concerning μ with regard to a reference value μ_2.

Let $F_t(\cdot, \nu)$ denote the CDF (see section ref{tDistributionCDF}) and let $t_{\nu,\alpha}$ denote the α-quantile (see section ref{tDistributionQuantile}) of the t-distribution with ν degrees of freedom. Define

$$t = \frac{\overline{x}_1 - \mu_2}{s}, \quad s = \sqrt{s_1^2/N}, \quad \nu = N - 1.$$

Then p-values and rejection criteria for H_0 can be calculated as summarized below

Test problem	p-value	Reject H_0
$H_{01} : \mu_1 \leq \mu_2$ vs $H_{A1} : \mu_1 > \mu_2$	$F_t(-t, \nu)$	$t > t_{\nu;1-\alpha}$
$H_{02} : \mu_1 \geq \mu_2$ vs $H_{A2} : \mu_1 < \mu_2$	$F_t(t, \nu)$	$t > t_{\nu;\alpha}$
$H_{03} : \mu_1 = \mu_2$ vs $H_{A3} : \mu_1 \neq \mu_2$	$F_t(t, \nu) - F_t(-t, \nu)$	$t > t_{\nu;1-\alpha/2}$ or $t > t_{\nu;\alpha/2}$

The test can also be expressed in terms of a correlation coefficient r between the combined X and an indicator variable, where t and r are related by

$$r = \frac{t}{\sqrt{t^2 + \nu}}, \quad t = \nu\frac{r}{1 - r^2}.$$

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} (in the case of TTest this is $\mu_1 \neq \mu_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1Test(means:=[5.24, 4.05], sd:=1.5, n:=22)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
Pearson's rho 0.7
Pearson's rho0. reference 0.2

Student's t-test for 2 corr. samples
Parameter Result
df 21
Difference of means 1.19
t-value (=delta) 2.087398086
t1 - alpha(1 - sided) 1.720742903
t1 - alpha(2 - sided) 2.079613845
test. p-value (H01: μ1 >= μ2) 0.97538843
test. p-value (H02: μ1 <= μ2) 0.02461157
test. p-value (H03: μ1 = μ2) 0.04922314
```

34.3.2 Student t-test for 2 correlated samples: confidence intervals

`ctx.studentt_2csamples_ci`(*mean*, *sd*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results for Student's t-test for for 2 correlated samples.

Parameters:

Mean
The mean of the sample.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $A_1 = t_{\nu,\alpha} \cdot s$ and $A_2 = t_{\nu,\alpha/2} \cdot s$, where s and ν are defined in (ref{eq:TTest1}), and $t_{\nu,\alpha}$ denotes the α-quantile of the (central) t-distribution with ν degrees of freedom (see section ref{tDistributionQuantile}).

Type	Confidence Interval (Difference of Means)
Left-sided	$-\infty \leq \mu_1 - \mu_2 \leq (\overline{x}_1 - \mu_2) + A_1$
Right-sided	$(\overline{x}_1 - \mu_2) - A_1 \leq \mu_1 - \mu_2 \leq +\infty$
Two-sided	$(\overline{x}_1 - \mu_2) - A_2 \leq \mu_1 - \mu_2 \leq (\overline{x}_1 - \mu_2) + A_2$

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} (in the case of textsf{TTest} this is $\mu_1 \neq \mu_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mean:=5.24, mean0:=4.05, sd:=1.5, n:=22, alpha=0.05, resultstring)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
Type 1 Error 0.05
Pearson's rho 0.7
Pearson's rho0. reference 0.2

Student's t-test for 2 corr. samples
Parameter Result
df 21
Difference of means 1.19
t-value (=delta) 2.087398086
t1 - alpha(1 - sided) 1.720742903
```

(continues on next page)

(continued from previous page)

```
t1 - alpha(2 - sided) 2.079613845
μ1 - μ2. CI - Length (2 - sided) 2.371124599
μ1 - μ2. CI Upper Limit (2-sided) 2.3755623
μ1 - μ2. CI Lower Limit (2-sided) 0.0044377
```

34.3.3 Student t-test for 2 correlated samples: power

`ctx.studentt_2csamples_power`(*mean*, *sd*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns the results of Student's t-test for 2 correlated samples: power and sample size.

Parameters:

Mean
The mean of the sample.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $\sigma_1^2 = \sigma^2$ and $\nu = N - 1$. Define

$$\widetilde{\rho} = \frac{\mu_1 - \mu_2}{\sigma} \text{ and } \delta = \sqrt{N}\widetilde{\rho}.$$

Let $F_{t'}(\cdot, \nu, \delta)$ denote the CDF of the (singly) noncentral t-distribution with ν degrees of freedom and noncentrality parameter δ and let $t_{\nu,\alpha}$ denote the α-quantile of the central t-distribution with ν degrees of freedom. Then the power for accepting H_A at the confidence level α can be calculated as summarized below:

Test	Null Hypothesis	Alternative	Power
1 sided	$H_{01} : \mu_1 \leq \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$F_{t'}\left(-t_{\nu;1-\alpha}, \nu, \delta\right)$
1 sided	$H_{02} : \mu_1 \geq \mu_2$	$H_{A2} : \mu_1 < \mu_2$	$F_{t'}\left(t_{\nu;1-\alpha}, \nu, \delta\right)$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$F_{t'}\left(-t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A3} : \mu_1 \neq \mu_2$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right) - F_t\left(-t_{\nu;1-\alpha/2}, \nu\,\delta\right)$

An actual call to the function, requesting Student's t-test with description, the critical calue for a two-sided test, the power for H_{A3} (in the case of textsf{TTest} this is $\mu_1 \neq \mu_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, n:=22, alpha:=0.05)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
```

(continues on next page)

(continued from previous page)

```
Type 1 Error 0.05
Pearson's rho 0.7
Pearson's rho0. reference 0.2

Student's t-test for 2 corr. samples
Parameter Result
df 21
Difference of means 1.19
t-value (=delta) 2.087398086
t1 - alpha(1 - sided) 1.720742903
t1 - alpha(2 - sided) 2.079613845
1-sided test. power (HA1: μ1 < μ2) 0.000122287
1-sided test. power (HA2: μ1 > μ2) 0.646013238
2-sided test. power (HA1: μ1 < μ2) 3.78799E-05
2-sided test. power (HA2: μ1 > μ2) 0.512603358
2-sided test. power (HA3: μ1 <> μ2) 0.512641238
test. Pr[Mean 1 < Mean 2] 0.018426082
test. Pr[Mean 1 > Mean 2] 0.981573918
```

34.3.4 Student t-test for 2 correlated samples: sample size calculation

`ctx.studentt_2csamples_samplesize`(*mu*, *sd*, *alpha=0.05*, *beta=0.1*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns the results of sample size calculations for Student's t-test for 2 correlated samples

Parameters:

Mean
The mean of the sample.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Beta
The beta-level used for power

Let $N_{t'}(\alpha, \beta, \widetilde{\rho})$ denote the sample size function of the (singly) noncentral t-distribution (see section ref{NoncentralTDistributionSampleSize}) for a given confidence level α, power β and noncentrality parameter $\widetilde{\rho}$ (as defined in equation ref{eq:TTestPower1}. The required total sample size N can be calculated as summarized below

Test	Null Hypothesis	Alternative	Minimal sample size
1 sided	$H_{01} : \mu_1 \leq \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
1 sided	$H_{02} : \mu_1 \geq \mu_2$	$H_{A2} : \mu_1 < \mu_2$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A1} : \mu_1 > \mu_2$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
2 sided	$H_{03} : \mu_1 = \mu_2$	$H_{A3} : \mu_1 \neq \mu_2$	$N2_{t'}(\alpha, \beta, \widetilde{\rho})$

Note that the returned value of N will in general not be an integer, and rounding up may be required.

An actual call to the function, requesting an upper sample size estimate (and actual power) for $\alpha = 0.95$, $\beta = 0.1$, and standard deviations $\sigma_1 = \sigma_2 = 1$, means $\mu_1 = 2.3$ and $\mu_2 = 4.5$, would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, alpha:=0.05, beta:=0.1)
                                      df: 21
                        difference of means: 1.19
                           t-value (=delta): 3.721063
                        t(1-alpha, 1-sided): 1.720743
                        t(1-alpha, 2-sided): 2.079614
  1-sided test, required N (HA1: mu1 < mu2): 18
1-sided test, actual power (HA1: mu1 < mu2): 0.974564
  1-sided test, required N (HA2: mu1 > mu2): 148
1-sided test, actual power (HA2: mu1 > mu2): 0.964564
```

(continues on next page)

(continued from previous page)

```
  2-sided test, required N (HA1: mu1 < mu2): 22
2-sided test, actual power (HA1: mu1 < mu2): 0.954564
  2-sided test, required N (HA2: mu1 > mu2): 212
2-sided test, actual power (HA2: mu1 > mu2): 0.977456
  2-sided test, required N (HA2: mu1 <>mu2): 24
2-sided test, actual power (HA2: mu1 <>mu2): 0.955544
```

34.3.5 Morgan-Pitman test for the variances of 2 correlated samples: tests (p-values)

`ctx.fratio_variance_2csamples_test(s2, n, alpha=0.05)`

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results for the Morgan-Pitman test for the variances of 2 correlated samples.

Parameters:

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $(X_1, X_2, \ldots, X_N)$ denote a random sample of size N from a normal distribution with mean σ and variance σ^2, and let

$$\overline{x}_1 = \frac{1}{N}\sum_{i=1}^{N} X_i \quad \text{and } s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(X_i - \overline{x}_1)$$

be the usual sample estimates of the unkown population mean σ and unkown population variance σ^2. Then Student's t-test can be used to test hypotheses concerning σ with regard to a reference value σ_2.

Let $F_t(\cdot, \nu)$ denote the CDF (see section ref{tDistributionCDF}) and let $t_{\nu,\alpha}$ denote the α-quantile (see section ref{tDistributionQuantile}) of the t-distribution with ν degrees of freedom. Define

$$t = \frac{\overline{x}_1 - \sigma_2}{s}, \quad s = \sqrt{s_1^2/N}, \quad \nu = N - 1.$$

Then p-values and rejection criteria for H_0 can be calculated as summarized below

Test problem	p-value	Reject H_0
$H_{01} : \sigma_1 \leq \sigma_2$ vs $H_{A1} : \sigma_1 > \sigma_2$	$F_t(-t, \nu)$	$t > t_{\nu;1-\alpha}$
$H_{02} : \sigma_1 \geq \sigma_2$ vs $H_{A2} : \sigma_1 < \sigma_2$	$F_t(t, \nu)$	$t > t_{\nu;\alpha}$
$H_{03} : \sigma_1 = \sigma_2$ vs $H_{A3} : \sigma_1 \neq \sigma_2$	$F_t(t, \nu) - F_t(-t, \nu)$	$t > t_{\nu;1-\alpha/2}$ or $t > t_{\nu;\alpha/2}$

The test can also be expressed in terms of a correlation coefficient r between the combined X and an indicator variable, where t and r are related by

$$r = \frac{t}{\sqrt{t^2 + \nu}}, \quad t = \nu\frac{r}{1 - r^2}.$$

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} (in the case of TTest this is $\sigma_1 \neq \sigma_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1Test(means:=[5.24, 4.05], sd:=1.5, n:=22)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
Pearson's rho 0.7
Pearson's rho0. reference 0.2

Pitman-Morgan-test for 2 corr. variances
Parameter Result
df1 21
df2 21
Variance-Ratio 0.183673469
F. 1 - alpha(1 - sided) 2.084188623
F . 1 - alpha(2 - sided) 2.408589482
Pitman-Morgan F-test. p-value (H01: s1 >= s2) 0.999864065
Pitman-Morgan F-test. p-value (H02: s1 <= s2) 0.000135935
Pitman-Morgan F-test. p-value (H03: s1 = s2) 0.999728131
```

34.3.6 Morgan-Pitman test for the variances of 2 correlated samples: confidence intervals

`ctx.fratio_variance_2csamples_ci`(*s2*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results of the confidence intervals for the Morgan-Pitman test for the variances of 2 correlated samples.

Parameters:

Mean
The mean of the sample.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $A_1 = t_{\nu,\alpha} \cdot s$ and $A_2 = t_{\nu,\alpha/2} \cdot s$, where s and ν are defined in (ref{eq:TTest1}), and $t_{\nu,\alpha}$ denotes the α-quantile of the (central) t-distribution with ν degrees of freedom (see section ref{tDistributionQuantile}).

Type	Confidence Interval (Difference of Means)
Left-sided	$-\infty \leq \sigma_1 - \sigma_2 \leq (\overline{x}_1 - \sigma_2) + A_1$
Right-sided	$(\overline{x}_1 - \sigma_2) - A_1 \leq \sigma_1 - \sigma_2 \leq +\infty$
Two-sided	$(\overline{x}_1 - \sigma_2) - A_2 \leq \sigma_1 - \sigma_2 \leq (\overline{x}_1 - \sigma_2) + A_2$

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} (in the case of textsf{TTest} this is $\sigma_1 \neq \sigma_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mean:=5.24, mean0:=4.05, sd:=1.5, n:=22, alpha=0.05, resultstring)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
Type 1 Error 0.05
Pearson's rho 0.7
Pearson's rho0. reference 0.2

Pitman-Morgan-test for 2 corr. variances
Parameter Result
df1 21
df2 21
Variance-Ratio 0.183673469
F. 1 - alpha(1 - sided) 2.084188623
```

(continues on next page)

(continued from previous page)

```
F . 1 - alpha(2 - sided) 2.408589482
s1/s2. CI - Length(2 - sided) 0.366136297
s1/s2. CI Upper Limit (2-sided) 0.442393986
s1/s2. CI Lower Limit (2-sided) 0.07625769
```

34.3.7 Morgan-Pitman test for the variances of 2 correlated samples: power

`ctx.`**`fratio_variance_2csamples_power`**(*s2*, *n*, *alpha=0.05*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results of the power calculation for the Morgan-Pitman test for the variances of 2 correlated samples.

Parameters:

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $\sigma_1^2 = \sigma^2$ and $\nu = N - 1$. Define

$$\widetilde{\rho} = \frac{\sigma_1 - \sigma_2}{\sigma} \text{ and } \delta = \sqrt{N}\widetilde{\rho}.$$

Let $F_{t'}(\cdot, \nu, \delta)$ denote the CDF of the (singly) noncentral t-distribution with ν degrees of freedom and noncentrality parameter δ and let $t_{\nu,\alpha}$ denote the α-quantile of the central t-distribution with ν degrees of freedom. Then the power for accepting H_A at the confidence level α can be calculated as summarized below:

Test	Null Hypothesis	Alternative	Power
1 sided	$H_{01} : \sigma_1 \leq \sigma_2$	$H_{A1} : \sigma_1 > \sigma_2$	$F_{t'}\left(-t_{\nu;1-\alpha}, \nu, \delta\right)$
1 sided	$H_{02} : \sigma_1 \geq \sigma_2$	$H_{A2} : \sigma_1 < \sigma_2$	$F_{t'}\left(t_{\nu;1-\alpha}, \nu, \delta\right)$
2 sided	$H_{03} : \sigma_1 = \sigma_2$	$H_{A1} : \sigma_1 > \sigma_2$	$F_{t'}\left(-t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \sigma_1 = \sigma_2$	$H_{A1} : \sigma_1 > \sigma_2$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \sigma_1 = \sigma_2$	$H_{A3} : \sigma_1 \neq \sigma_2$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right) - F_t\left(-t_{\nu;1-\alpha/2}, \nu\, \delta\right)$

An actual call to the function, requesting Student's t-test with description, the critical calue for a two-sided test, the power for H_{A3} (in the case of textsf{TTest} this is $\sigma_1 \neq \sigma_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, n:=22, alpha:=0.05)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
Type 1 Error 0.05
Pearson's rho 0.7
Pearson's rho0. reference 0.2
```

(continues on next page)

(continued from previous page)

```
Pitman-Morgan-test for 2 corr. variances
Parameter Result
df1 21
df2 21
Variance-Ratio 0.183673469
F. 1 - alpha(1 - sided) 2.084188623
F . 1 - alpha(2 - sided) 2.408589482
Pitman-Morgan 1-sided test. power (HA1: s1 < s2) 0.999864065
Pitman-Morgan 1-sided test. power (HA2: s1 > s2) 0.999864065
Pitman-Morgan 2-sided test. power (HA1: s1 < s2) 0.999864065
Pitman-Morgan 2-sided test. power (HA2: s1 > s2) 0.999864065
Pitman-Morgan 2-sided test. power (HA3: s1 <> s2) 0.999864065
```

34.3.8 Morgan-Pitman test for the variances of 2 correlated samples: sample size

`ctx.fratio_variance_2csamples_samplesize`(*s2*, *alpha=0.05*, *beta=0.1*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results of sample size calculations for the Morgan-Pitman test for the variances of 2 correlated samples.

Parameters:

Sd
The standard deviation of the sample

Alpha
The alpha-level used for confidence intervals

Beta
The beta-level used for power

Let $N_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$ denote the sample size function of the (singly) noncentral t-distribution (see section ref{NoncentralTDistributionSampleSize}) for a given confidence level α, power β and noncentrality parameter $\widetilde{\rho}$ (as defined in equation ref{eq:TTestPower1}. The required total sample size N can be calculated as summarized below

Test	Null Hypothesis	Alternative	Minimal sample size
1 sided	$H_{01}:\sigma_1 \leq \sigma_2$	$H_{A1}:\sigma_1 > \sigma_2$	$N_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$
1 sided	$H_{02}:\sigma_1 \geq \sigma_2$	$H_{A2}:\sigma_1 < \sigma_2$	$N_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$
2 sided	$H_{03}:\sigma_1 = \sigma_2$	$H_{A1}:\sigma_1 > \sigma_2$	$N_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$
2 sided	$H_{03}:\sigma_1 = \sigma_2$	$H_{A1}:\sigma_1 > \sigma_2$	$N_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$
2 sided	$H_{03}:\sigma_1 = \sigma_2$	$H_{A3}:\sigma_1 \neq \sigma_2$	$N2_{t'}\left(\alpha,\beta,\widetilde{\rho}\right)$

Note that the returned value of N will in general not be an integer, and rounding up may be required.

An actual call to the function, requesting an upper sample size estimate (and actual power) for $\alpha = 0.95$, $\beta = 0.1$, and standard deviations $\sigma_1 = \sigma_2 = 1$, means $\sigma_1 = 2.3$ and $\sigma_2 = 4.5$, would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, alpha:=0.05, beta:=0.1)
                                    df: 21
                        difference of means: 1.19
                           t-value (=delta): 3.721063
                        t(1-alpha, 1-sided): 1.720743
                        t(1-alpha, 2-sided): 2.079614
  1-sided test, required N (HA1: mu1 < mu2): 18
1-sided test, actual power (HA1: mu1 < mu2): 0.974564
  1-sided test, required N (HA2: mu1 > mu2): 148
1-sided test, actual power (HA2: mu1 > mu2): 0.964564
  2-sided test, required N (HA1: mu1 < mu2): 22
2-sided test, actual power (HA1: mu1 < mu2): 0.954564
  2-sided test, required N (HA2: mu1 > mu2): 212
2-sided test, actual power (HA2: mu1 > mu2): 0.977456
  2-sided test, required N (HA2: mu1 <>mu2): 24
2-sided test, actual power (HA2: mu1 <>mu2): 0.955544
```

34.3.9 Pearson's rho, 2 correlated samples: tests (p-values)

`ctx.pearson_rho_test`(*rho*, *rh0*, *n*, *alpha=0.05*, *rtype=1*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results of tests for Pearson's rho, 2 correlated samples.

Parameters:

Mean
The mean of the sample.

Mean0
The reference mean.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $(X_1, X_2, \ldots, X_N)$ denote a random sample of size N from a normal distribution with mean ρ and variance σ^2, and let

$$\overline{x}_1 = \frac{1}{N}\sum_{i=1}^{N} X_i \quad \text{and } s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(X_i - \overline{x}_1)$$

be the usual sample estimates of the unkown population mean ρ and unkown population variance σ^2. Then Student's t-test can be used to test hypotheses concerning ρ with regard to a reference value ρ_0.

Let $F_t(\cdot, \nu)$ denote the CDF (see section ref{tDistributionCDF}) and let $t_{\nu,\alpha}$ denote the α-quantile (see section ref{tDistributionQuantile}) of the t-distribution with ν degrees of freedom. Define

$$t = \frac{\overline{x}_1 - \rho_0}{s}, \quad s = \sqrt{s_1^2/N}, \quad \nu = N - 1.$$

Then p-values and rejection criteria for H_0 can be calculated as summarized below

Test problem	p-value	Reject H_0
$H_{01} : \rho \leq \rho_0$ vs $H_{A1} : \rho > \rho_0$	$F_t(-t, \nu)$	$t > t_{\nu;1-\alpha}$
$H_{02} : \rho \geq \rho_0$ vs $H_{A2} : \rho < \rho_0$	$F_t(t, \nu)$	$t > t_{\nu;\alpha}$
$H_{03} : \rho = \rho_0$ vs $H_{A3} : \rho \neq \rho_0$	$F_t(t, \nu) - F_t(-t, \nu)$	$t > t_{\nu;1-\alpha/2}$ or $t > t_{\nu;\alpha/2}$

The test can also be expressed in terms of a correlation coefficient r between the combined X and an indicator variable, where t and r are related by

$$r = \frac{t}{\sqrt{t^2 + \nu}}, \quad t = \nu\frac{r}{1 - r^2}.$$

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} (in the case of TTest this is $\rho_1 \neq \rho_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1Test(mean:=5.24, mean0:=4.05, sd:=1.5, n:=22, resultstring:='All')

Input
VariableVariable1
CommonN22
Mean.Group15.24
Mean.Group24.05
StDev.Group11.5
StDev.Group23.5
Pearson'srho0.7
Pearson'srho0.reference0.2

Pearson'srho
ParameterResult
df
rho0.1-alpha(1-sided)
rho0.1-alpha(2-sided)
p-value(H01:rho>=0)
p-value(H02:rho<=0)
p-value(H03:rho=0)
p-value(H01:rho>=rho0)
p-value(H02:rho<=rho0)
p-value(H03:rho=rho0)
p-value(H04:rho^2=rho0^2)
```

34.3.10 Pearson's rho, 2 correlated samples: confidence intervals

`ctx.pearson_rho_ci`(*rho*, *rh0*, *n*, *alpha=0.05*, *rtype=1*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns results of confidence intervals for Pearson's rho, 2 correlated samples.

Parameters:

Mean
The mean of the sample.

Mean0
The reference mean.

Sd
The standard deviation of the sample

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $A_1 = t_{\nu,\alpha} \cdot s$ and $A_2 = t_{\nu,\alpha/2} \cdot s$, where s and ν are defined in (ref{eq:TTest1}), and $t_{\nu,\alpha}$ denotes the α-quantile of the (central) t-distribution with ν degrees of freedom (see section ref{tDistributionQuantile}).

Type	Confidence Interval (Difference of Means)
Left-sided	$-\infty \leq \rho_1 - \rho_0 \leq (\overline{x}_1 - \rho_0) + A_1$
Right-sided	$(\overline{x}_1 - \rho_0) - A_1 \leq \rho_1 - \rho_0 \leq +\infty$
Two-sided	$(\overline{x}_1 - \rho_0) - A_2 \leq \rho_1 - \rho_0 \leq (\overline{x}_1 - \rho_0) + A_2$

An actual call to the function, requesting Student's t-test with description, the critical value for a two-sided test, the p-value for H_{03} (in the case of textsf{TTest} this is $\rho_1 \neq \rho_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mean:=5.24, mean0:=4.05, sd:=1.5, n:=22, alpha=0.05, resultstring)

Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
StDev. Group 2 3.5
Type 1 Error 0.05
Pearson's rho 0.7
Pearson's rho0. reference 0.2

Pearson's rho
Parameter Result
df
```

(continues on next page)

(continued from previous page)

```
rho0. 1 - alpha(1 - sided)
rho0 . 1 - alpha(2 - sided)
rho. CI Upper Limit (2-sided)
rho. estimate
rho. CI Lower Limit (2-sided)
rho. CI - Length(2 - sided)
```

34.3.11 Pearson's rho, 2 correlated samples:: power

`ctx.pearson_rho_power`(*rho*, *rho0*, *n*, *alpha=0.05*, *rtype=1*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns the results of power calculations for Pearson's rho, 2 correlated samples.

Parameters:

Rho
The mean of the sample.

Rho0
The reference mean.

N
The sample size

Alpha
The alpha-level used for confidence intervals

Let $\sigma_1^2 = \sigma^2$ and $\nu = N - 1$. Define

$$\widetilde{\rho} = \frac{\rho_1 - \rho_0}{\sigma} \text{ and } \delta = \sqrt{N}\widetilde{\rho}.$$

Let $F_{t'}(\cdot, \nu, \delta)$ denote the CDF of the (singly) noncentral t-distribution with ν degrees of freedom and noncentrality parameter δ and let $t_{\nu,\alpha}$ denote the α-quantile of the central t-distribution with ν degrees of freedom. Then the power for accepting H_A at the confidence level α can be calculated as summarized below:

Test	Null Hypothesis	Alternative	Power
1 sided	$H_{01} : \rho \leq \rho_0$	$H_{A1} : \rho > \rho_0$	$F_{t'}\left(-t_{\nu;1-\alpha}, \nu, \delta\right)$
1 sided	$H_{02} : \rho \geq \rho_0$	$H_{A2} : \rho < \rho_0$	$F_{t'}\left(t_{\nu;1-\alpha}, \nu, \delta\right)$
2 sided	$H_{03} : \rho = \rho_0$	$H_{A1} : \rho > \rho_0$	$F_{t'}\left(-t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \rho = \rho_0$	$H_{A1} : \rho > \rho_0$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right)$
2 sided	$H_{03} : \rho = \rho_0$	$H_{A3} : \rho \neq \rho_0$	$F_{t'}\left(t_{\nu;1-\alpha/2}, \nu, \delta\right) - F_t\left(-t_{\nu;1-\alpha/2}, \nu\,\delta\right)$

An actual call to the function, requesting Student's t-test with description, the critical calue for a two-sided test, the power for H_{A3} (in the case of textsf{TTest} this is $\rho_1 \neq \rho_2$), for 2 independent samples of size 10 and standard deviation 1 each, with means 2.3 and 4.5, and a type I error $\alpha = 0.05$ would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, n:=22, alpha:=0.05)


Input
Variable Variable 1
Common N 22
Mean. Group 1 5.24
Mean. Group 2 4.05
StDev. Group 1 1.5
```

(continues on next page)

(continued from previous page)

```
StDev. Group 2 3.5
Type 1 Error 0.05
Pearson's rho 0.7
Pearson's rho0. reference 0.2

Pearson's rho
Parameter Result
df
rho0. 1 - alpha(1 - sided)
rho0 . 1 - alpha(2 - sided)
Power (HA1: rho > rho0)
Power (HA2: rho < rho0)
Power (HA3: rho <> rho0)
Power (HA4: rho^2 <> rho0^2)
```

34.3.12 Pearson's rho, 2 correlated samples:: sample size calculation

`ctx.pearson_rho_samplesize`(*rho*, *rho0*, *n*, *alpha=0.05*, *beta=0.1*, *rtype=1*)

where `ctx` is `dec`, `mpm`, or `gmp`.

Returns the results of power calculations for Pearson's rho, 2 correlated samples.

Parameters:

Rho
The mean of the sample.

Rho0
The reference mean.

N
The sample size

Alpha
The alpha-level used for confidence intervals

Beta
The beta-level used for power

Let $N_{t'}(\alpha, \beta, \widetilde{\rho})$ denote the sample size function of the (singly) noncentral t-distribution (see section ref{NoncentralTDistributionSampleSize}) for a given confidence level α, power β and noncentrality parameter $\widetilde{\rho}$ (as defined in equation ref{eq:TTestPower1}. The required total sample size N can be calculated as summarized below

Test	Null Hypothesis	Alternative	Minimal sample size
1 sided	$H_{01} : \rho \leq \rho_0$	$H_{A1} : \rho > \rho_0$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
1 sided	$H_{02} : \rho \geq \rho_0$	$H_{A2} : \rho < \rho_0$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
2 sided	$H_{03} : \rho = \rho_0$	$H_{A1} : \rho > \rho_0$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
2 sided	$H_{03} : \rho = \rho_0$	$H_{A1} : \rho > \rho_0$	$N_{t'}(\alpha, \beta, \widetilde{\rho})$
2 sided	$H_{03} : \rho = \rho_0$	$H_{A3} : \rho \neq \rho_0$	$N2_{t'}(\alpha, \beta, \widetilde{\rho})$

Note that the returned value of N will in general not be an integer, and rounding up may be required.

An actual call to the function, requesting an upper sample size estimate (and actual power) for $\alpha = 0.95$, $\beta = 0.1$, and standard deviations $\sigma_1 = \sigma_2 = 1$, means $\rho_1 = 2.3$ and $\rho_2 = 4.5$, would be

```
>>> from xlcalcnet import mpm
>>> math53.StudentT1CI(mu1:=5.24, mu0:=4.05, sd:=1.5, alpha:=0.05, beta:=0.1)
                                      df: 21
                        difference of means: 1.19
                           t-value (=delta): 3.721063
                        t(1-alpha, 1-sided): 1.720743
                        t(1-alpha, 2-sided): 2.079614
  1-sided test, required N (HA1: mu1 < mu2): 18
1-sided test, actual power (HA1: mu1 < mu2): 0.974564
  1-sided test, required N (HA2: mu1 > mu2): 148
1-sided test, actual power (HA2: mu1 > mu2): 0.964564
```

(continues on next page)

(continued from previous page)

```
  2-sided test, required N (HA1: mu1 < mu2): 22
2-sided test, actual power (HA1: mu1 < mu2): 0.954564
  2-sided test, required N (HA2: mu1 > mu2): 212
2-sided test, actual power (HA2: mu1 > mu2): 0.977456
  2-sided test, required N (HA2: mu1 <>mu2): 24
2-sided test, actual power (HA2: mu1 <>mu2): 0.955544
```

Part VIII

User library: graphics

CHAPTER

THIRTYFIVE

DISTRIBUTION FUNCTIONS

Text explaining this chapter

35.1 Introduction to 2D functions plots

35.1.1 Plots of continuous distribution functions

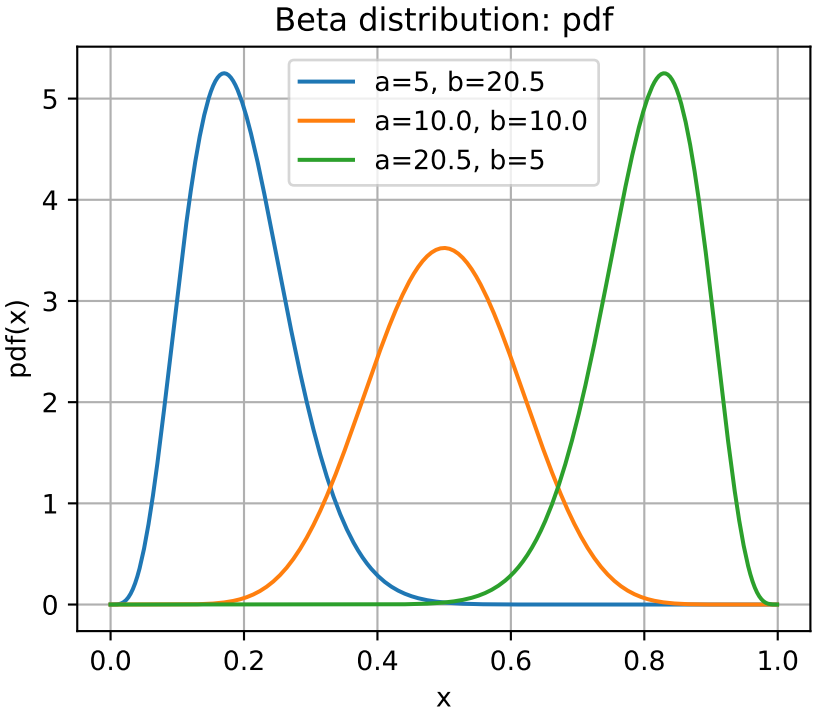
Some text

35.1.1.1 Probability density function (pdf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_DistPlotContinuous.py, and the XML code in the file D07_DistPlotContinuous.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None
        target = 'pdf' # pdf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'

        Title = 'Beta distribution'
        a = [5, 10.0, 20.5]
        b = [20.5, 10.0, 5]
        xlim = [0, 0.999]
        if target=='hf': ylim=[0, 100]

        dlist = []
        ltext = []
        for j in range(len(a)):
            dlist.append(dreal.dist_beta(a[j], b[j]))
            ltext.append('a=' + str(a[j]) + ', b=' + str(b[j]))

        DistPlotContinuous(Title = Title, dlist=dlist, xlim = xlim,
            ylim = ylim, target = target, ltext = ltext, marker='o',
            markersize=0)
```

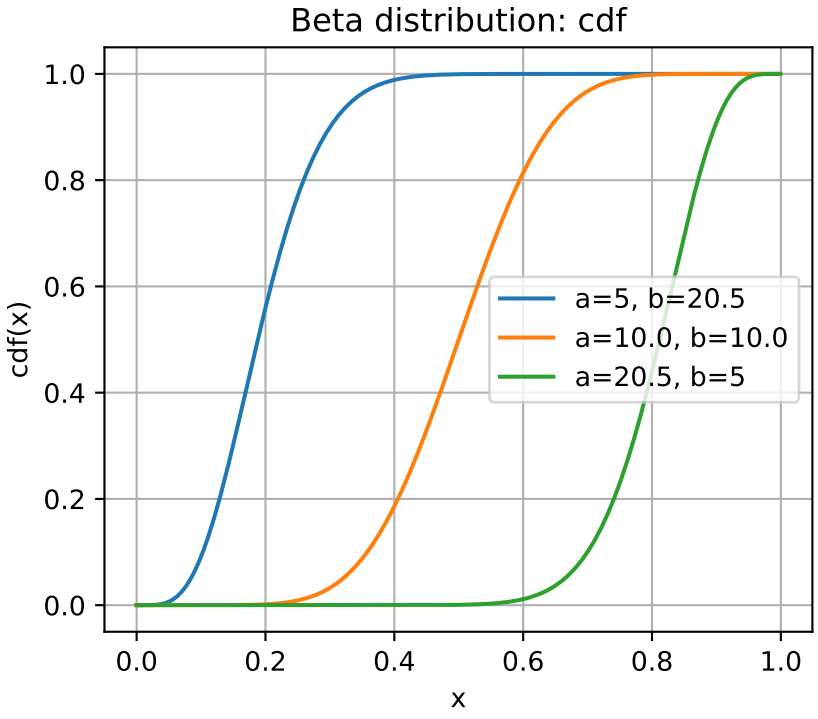

Beta distribution: pdf
a=5, b=20.5
a=10.0, b=10.0
a=20.5, b=5
pdf(x)
x
0
1
2
3
4
5
0.0
0.2
0.4
0.6
0.8
1.0

35.1.1.2 Cumulative distribution function (cdf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D07_DistPlotContinuous.py, and the XML code in the file D07_DistPlotContinuous.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None
        target = 'cdf' # pdf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'

        Title = 'Beta distribution'
        a = [5, 10.0, 20.5]
        b = [20.5, 10.0, 5]
        xlim = [0, 0.999]
        if target=='hf': ylim=[0, 100]

        dlist = []
        ltext = []
        for j in range(len(a)):
            dlist.append(dreal.dist_beta(a[j], b[j]))
            ltext.append('a=' + str(a[j]) + ', b=' + str(b[j]))

        DistPlotContinuous(Title = Title, dlist=dlist, xlim = xlim,
            ylim = ylim, target = target, ltext = ltext, marker='o',
            markersize=0)
```



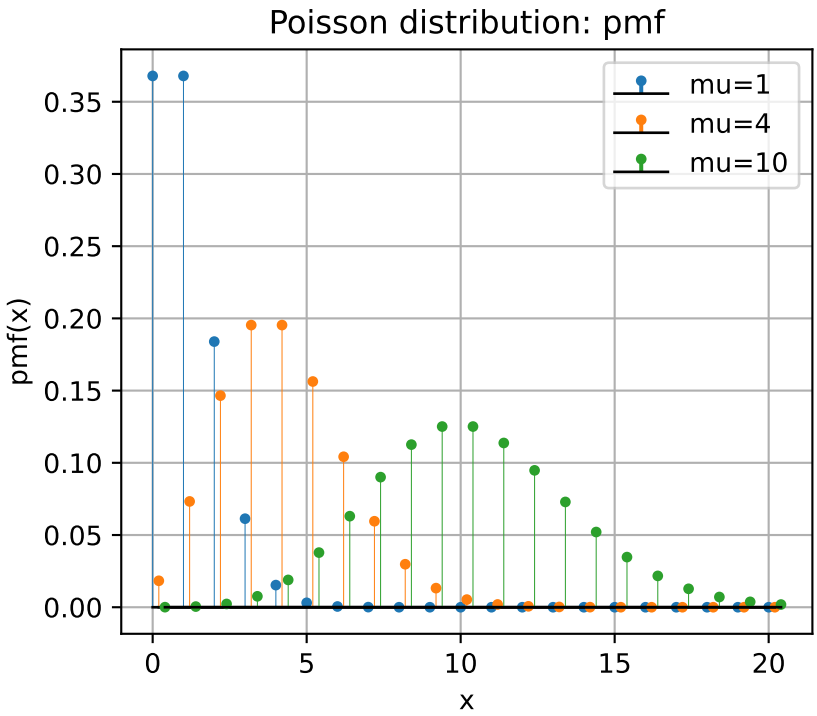
35.1.2 Plots of discrete distribution functions

35.1.2.1 Probability mass function (pmf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_DistPlotDiscrete.py, and the XML code in the file D08_DistPlotDiscrete.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None

        Title = 'Poisson distribution'
        target = 'pmf' # pmf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'
        mu = [1, 4, 10]
        xlim = [0.0, 20.0]
        ylim = None
        if target=='qtf': ylim=[0, 20]

        dlist = []
        ltext = []
        for j in range(len(mu)):
            dlist.append(dreal.dist_poisson(mu[j]))
            ltext.append('mu=' + str(mu[j]))
        DistPlotDiscrete(Title = Title, dlist=dlist, xlim = xlim,  ylim = ylim,
            target = target, ltext = ltext, lattice=True, marker='o',
            markersize=3, vertical_lines=True)
```

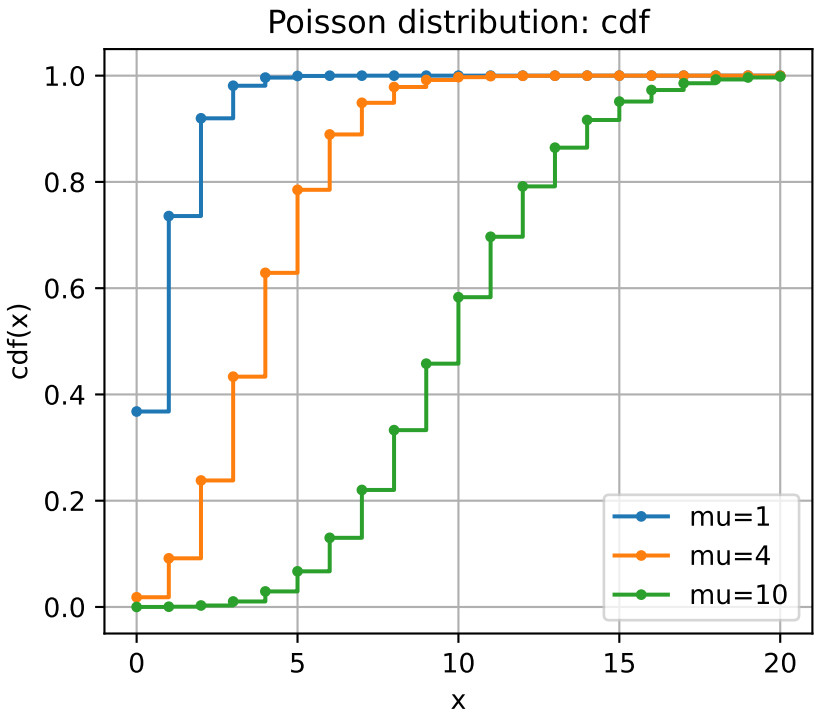


35.1.2.2 Cumulative distribution function (cdf)

The Python code for the example below can be found online in the `DataXlCalcNet` repository or in the corresponding local `DataXlCalcNet` folder in the file D08_DistPlotDiscrete.py, and the XML code in the file D08_DistPlotDiscrete.2D.xml.

To produce the figure as shown below, the try-block at the end of the file should look like this:

```
try:
    if __name__ == '__main__':
        ylim = None

        Title = 'Poisson distribution'
        target = 'cdf' # pmf, cdf, 'sf', 'hf', 'chf', 'qtf', 'isf'
        mu = [1, 4, 10]
        xlim = [0.0, 20.0]
        ylim = None
        if target=='qtf': ylim=[0, 20]

        dlist = []
        ltext = []
        for j in range(len(mu)):
            dlist.append(dreal.dist_poisson(mu[j]))
            ltext.append('mu=' + str(mu[j]))
        DistPlotDiscrete(Title = Title, dlist=dlist, xlim = xlim,  ylim = ylim,
            target = target, ltext = ltext, lattice=True, marker='o',
            markersize=3, vertical_lines=True)
```



CHAPTER

THIRTYSIX

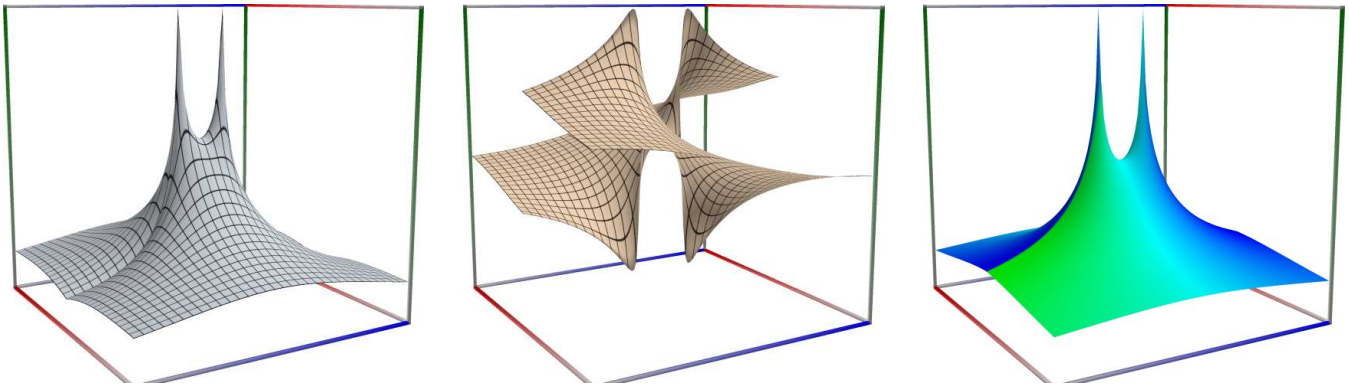
COMPLEX FUNCTIONS

Text explaining this chapter

36.1 Additional complex functions

36.1.1 Elliptic K

Some Text



3D wpf plot: real part (left figure), imaginary part (middle figure) and absolute value with color-coded phase (right figure) of the complex sine function $z = \sin(x + iy)$, with $-6 \leq x \leq 6$ (blue axis), $-6 \leq y \leq 6$ (red axis), $-10 \leq z \leq 10$ (green axis). Function values are *loglog-transformed*.

Part IX

Back matter

CHAPTER

THIRTYSEVEN

LICENSE AND HISTORY

37.1 Mozilla Public License Version 2.0

```
1. Definitions
--------------

1.1. "Contributor"
    means each individual or legal entity that creates, contributes to
    the creation of, or owns Covered Software.

1.2. "Contributor Version"
    means the combination of the Contributions of others (if any) used
    by a Contributor and that particular Contributor's Contribution.

1.3. "Contribution"
    means Covered Software of a particular Contributor.

1.4. "Covered Software"
    means Source Code Form to which the initial Contributor has attached
    the notice in Exhibit A, the Executable Form of such Source Code
    Form, and Modifications of such Source Code Form, in each case
    including portions thereof.

1.5. "Incompatible With Secondary Licenses"
    means

    (a) that the initial Contributor has attached the notice described
        in Exhibit B to the Covered Software; or

    (b) that the Covered Software was made available under the terms of
        version 1.1 or earlier of the License, but not also under the
        terms of a Secondary License.

1.6. "Executable Form"
    means any form of the work other than Source Code Form.

1.7. "Larger Work"
    means a work that combines Covered Software with other material, in
    a separate file or files, that is not Covered Software.

1.8. "License"
    means this document.

1.9. "Licensable"
    means having the right to grant, to the maximum extent possible,
    whether at the time of the initial grant or subsequently, any and
    all of the rights conveyed by this License.

1.10. "Modifications"
    means any of the following:
```

(continues on next page)

(continued from previous page)

```
    (a) any file in Source Code Form that results from an addition to,
        deletion from, or modification of the contents of Covered
        Software; or

    (b) any new file in Source Code Form that contains any Covered
        Software.

1.11. "Patent Claims" of a Contributor
    means any patent claim(s), including without limitation, method,
    process, and apparatus claims, in any patent Licensable by such
    Contributor that would be infringed, but for the grant of the
    License, by the making, using, selling, offering for sale, having
    made, import, or transfer of either its Contributions or its
    Contributor Version.

1.12. "Secondary License"
    means either the GNU General Public License, Version 2.0, the GNU
    Lesser General Public License, Version 2.1, the GNU Affero General
    Public License, Version 3.0, or any later versions of those
    licenses.

1.13. "Source Code Form"
    means the form of the work preferred for making modifications.

1.14. "You" (or "Your")
    means an individual or a legal entity exercising rights under this
    License. For legal entities, "You" includes any entity that
    controls, is controlled by, or is under common control with You. For
    purposes of this definition, "control" means (a) the power, direct
    or indirect, to cause the direction or management of such entity,
    whether by contract or otherwise, or (b) ownership of more than
    fifty percent (50%) of the outstanding shares or beneficial
    ownership of such entity.

2. License Grants and Conditions
--------------------------------

2.1. Grants

Each Contributor hereby grants You a world-wide, royalty-free,
non-exclusive license:

(a) under intellectual property rights (other than patent or trademark)
    Licensable by such Contributor to use, reproduce, make available,
    modify, display, perform, distribute, and otherwise exploit its
    Contributions, either on an unmodified basis, with Modifications, or
    as part of a Larger Work; and

(b) under Patent Claims of such Contributor to make, use, sell, offer
    for sale, have made, import, and otherwise transfer either its
    Contributions or its Contributor Version.

2.2. Effective Date

The licenses granted in Section 2.1 with respect to any Contribution
become effective for each Contribution on the date the Contributor first
distributes such Contribution.

2.3. Limitations on Grant Scope

The licenses granted in this Section 2 are the only rights granted under
this License. No additional rights or licenses will be implied from the
distribution or licensing of Covered Software under this License.
Notwithstanding Section 2.1(b) above, no patent license is granted by a
Contributor:
```

(continues on next page)

(continued from previous page)

```
(a) for any code that a Contributor has removed from Covered Software;
    or

(b) for infringements caused by: (i) Your and any other third party's
    modifications of Covered Software, or (ii) the combination of its
    Contributions with other software (except as part of its Contributor
    Version); or

(c) under Patent Claims infringed by Covered Software in the absence of
    its Contributions.

This License does not grant any rights in the trademarks, service marks,
or logos of any Contributor (except as may be necessary to comply with
the notice requirements in Section 3.4).

2.4. Subsequent Licenses

No Contributor makes additional grants as a result of Your choice to
distribute the Covered Software under a subsequent version of this
License (see Section 10.2) or under the terms of a Secondary License (if
permitted under the terms of Section 3.3).

2.5. Representation

Each Contributor represents that the Contributor believes its
Contributions are its original creation(s) or it has sufficient rights
to grant the rights to its Contributions conveyed by this License.

2.6. Fair Use

This License is not intended to limit any rights You have under
applicable copyright doctrines of fair use, fair dealing, or other
equivalents.

2.7. Conditions

Sections 3.1, 3.2, 3.3, and 3.4 are conditions of the licenses granted
in Section 2.1.

3. Responsibilities
-------------------

3.1. Distribution of Source Form

All distribution of Covered Software in Source Code Form, including any
Modifications that You create or to which You contribute, must be under
the terms of this License. You must inform recipients that the Source
Code Form of the Covered Software is governed by the terms of this
License, and how they can obtain a copy of this License. You may not
attempt to alter or restrict the recipients' rights in the Source Code
Form.

3.2. Distribution of Executable Form

If You distribute Covered Software in Executable Form then:

(a) such Covered Software must also be made available in Source Code
    Form, as described in Section 3.1, and You must inform recipients of
    the Executable Form how they can obtain a copy of such Source Code
    Form by reasonable means in a timely manner, at a charge no more
    than the cost of distribution to the recipient; and

(b) You may distribute such Executable Form under the terms of this
    License, or sublicense it under different terms, provided that the
    license for the Executable Form does not attempt to limit or alter
    the recipients' rights in the Source Code Form under this License.
```

(continues on next page)

(continued from previous page)

```
3.3. Distribution of a Larger Work

You may create and distribute a Larger Work under terms of Your choice,
provided that You also comply with the requirements of this License for
the Covered Software. If the Larger Work is a combination of Covered
Software with a work governed by one or more Secondary Licenses, and the
Covered Software is not Incompatible With Secondary Licenses, this
License permits You to additionally distribute such Covered Software
under the terms of such Secondary License(s), so that the recipient of
the Larger Work may, at their option, further distribute the Covered
Software under the terms of either this License or such Secondary
License(s).

3.4. Notices

You may not remove or alter the substance of any license notices
(including copyright notices, patent notices, disclaimers of warranty,
or limitations of liability) contained within the Source Code Form of
the Covered Software, except that You may alter any license notices to
the extent required to remedy known factual inaccuracies.

3.5. Application of Additional Terms

You may choose to offer, and to charge a fee for, warranty, support,
indemnity or liability obligations to one or more recipients of Covered
Software. However, You may do so only on Your own behalf, and not on
behalf of any Contributor. You must make it absolutely clear that any
such warranty, support, indemnity, or liability obligation is offered by
You alone, and You hereby agree to indemnify every Contributor for any
liability incurred by such Contributor as a result of warranty, support,
indemnity or liability terms You offer. You may include additional
disclaimers of warranty and limitations of liability specific to any
jurisdiction.

4. Inability to Comply Due to Statute or Regulation
---------------------------------------------------

If it is impossible for You to comply with any of the terms of this
License with respect to some or all of the Covered Software due to
statute, judicial order, or regulation then You must: (a) comply with
the terms of this License to the maximum extent possible; and (b)
describe the limitations and the code they affect. Such description must
be placed in a text file included with all distributions of the Covered
Software under this License. Except to the extent prohibited by statute
or regulation, such description must be sufficiently detailed for a
recipient of ordinary skill to be able to understand it.

5. Termination
--------------

5.1. The rights granted under this License will terminate automatically
if You fail to comply with any of its terms. However, if You become
compliant, then the rights granted under this License from a particular
Contributor are reinstated (a) provisionally, unless and until such
Contributor explicitly and finally terminates Your grants, and (b) on an
ongoing basis, if such Contributor fails to notify You of the
non-compliance by some reasonable means prior to 60 days after You have
come back into compliance. Moreover, Your grants from a particular
Contributor are reinstated on an ongoing basis if such Contributor
notifies You of the non-compliance by some reasonable means, this is the
first time You have received notice of non-compliance with this License
from such Contributor, and You become compliant prior to 30 days after
Your receipt of the notice.

5.2. If You initiate litigation against any entity by asserting a patent
```

(continues on next page)

(continued from previous page)

```
infringement claim (excluding declaratory judgment actions,
counter-claims, and cross-claims) alleging that a Contributor Version
directly or indirectly infringes any patent, then the rights granted to
You by any and all Contributors for the Covered Software under Section
2.1 of this License shall terminate.

5.3. In the event of termination under Sections 5.1 or 5.2 above, all
end user license agreements (excluding distributors and resellers) which
have been validly granted by You or Your distributors under this License
prior to termination shall survive termination.

************************************************************************
*                                                                      *
*  6. Disclaimer of Warranty                                           *
*  -------------------------                                           *
*                                                                      *
*  Covered Software is provided under this License on an "as is"       *
*  basis, without warranty of any kind, either expressed, implied, or  *
*  statutory, including, without limitation, warranties that the       *
*  Covered Software is free of defects, merchantable, fit for a        *
*  particular purpose or non-infringing. The entire risk as to the     *
*  quality and performance of the Covered Software is with You.        *
*  Should any Covered Software prove defective in any respect, You     *
*  (not any Contributor) assume the cost of any necessary servicing,   *
*  repair, or correction. This disclaimer of warranty constitutes an   *
*  essential part of this License. No use of any Covered Software is   *
*  authorized under this License except under this disclaimer.         *
*                                                                      *
************************************************************************

************************************************************************
*                                                                      *
*  7. Limitation of Liability                                          *
*  --------------------------                                          *
*                                                                      *
*  Under no circumstances and under no legal theory, whether tort      *
*  (including negligence), contract, or otherwise, shall any           *
*  Contributor, or anyone who distributes Covered Software as          *
*  permitted above, be liable to You for any direct, indirect,         *
*  special, incidental, or consequential damages of any character      *
*  including, without limitation, damages for lost profits, loss of    *
*  goodwill, work stoppage, computer failure or malfunction, or any    *
*  and all other commercial damages or losses, even if such party      *
*  shall have been informed of the possibility of such damages. This   *
*  limitation of liability shall not apply to liability for death or   *
*  personal injury resulting from such party's negligence to the       *
*  extent applicable law prohibits such limitation. Some               *
*  jurisdictions do not allow the exclusion or limitation of           *
*  incidental or consequential damages, so this exclusion and          *
*  limitation may not apply to You.                                    *
*                                                                      *
************************************************************************

8. Litigation
-------------

Any litigation relating to this License may be brought only in the
courts of a jurisdiction where the defendant maintains its principal
place of business and such litigation shall be governed by laws of that
jurisdiction, without reference to its conflict-of-law provisions.
Nothing in this Section shall prevent a party's ability to bring
cross-claims or counter-claims.

9. Miscellaneous
----------------
```

(continues on next page)

(continued from previous page)

```
This License represents the complete agreement concerning the subject
matter hereof. If any provision of this License is held to be
unenforceable, such provision shall be reformed only to the extent
necessary to make it enforceable. Any law or regulation which provides
that the language of a contract shall be construed against the drafter
shall not be used to construe this License against a Contributor.

10. Versions of the License
---------------------------

10.1. New Versions

Mozilla Foundation is the license steward. Except as provided in Section
10.3, no one other than the license steward has the right to modify or
publish new versions of this License. Each version will be given a
distinguishing version number.

10.2. Effect of New Versions

You may distribute the Covered Software under the terms of the version
of the License under which You originally received the Covered Software,
or under the terms of any subsequent version published by the license
steward.

10.3. Modified Versions

If you create software not governed by this License, and you want to
create a new license for such software, you may create and use a
modified version of this License if you rename the license and remove
any references to the name of the license steward (except to note that
such modified license differs from this License).

10.4. Distributing Source Code Form that is Incompatible With Secondary
Licenses

If You choose to distribute Source Code Form that is Incompatible With
Secondary Licenses under the terms of this version of the License, the
notice described in Exhibit B of this License must be attached.

Exhibit A - Source Code Form License Notice
-------------------------------------------

  This Source Code Form is subject to the terms of the Mozilla Public
  License, v. 2.0. If a copy of the MPL was not distributed with this
  file, You can obtain one at https://mozilla.org/MPL/2.0/.

If it is not possible or desirable to put the notice in a particular
file, then You may include the notice in a location (such as a LICENSE
file in a relevant directory) where a recipient would be likely to look
for such a notice.

You may add additional accurate notices of copyright ownership.

Exhibit B - "Incompatible With Secondary Licenses" Notice
---------------------------------------------------------

  This Source Code Form is "Incompatible With Secondary Licenses", as
  defined by the Mozilla Public License, v. 2.0.
```

37.2 History

Description of version 0.1

Description of version 0.2

Description of version 0.3

BIBLIOGRAPHY

[1] J. Abate and P. Valko. Multi-precision laplace transform inversion. *International Journal for Numerical Methods in Engineering*, 60:979–993, 2004.

[2] M. Abramowitz and I.A. Stegun. *Confluent Hypergeometric Functions*. in Handbook of Mathematical Functions, New York: Dekker, 1970. Available as https://personal.math.ubc.ca/ÂăcbmÂăands/page_503.htm.

[3] M. Abramowitz and I.A. Stegun. *F-(Variance-Ratio) Distribution Function*. in Handbook of Mathematical Functions, New York: Dekker, 1970. Available as https://personal.math.ubc.ca/ÂăcbmÂăands/page_946.htm.

[4] M. Abramowitz and I.A. Stegun. *Handbook of Mathematical Functions*. New York: Dekker, 1970. Available as https://archive.org/details/AandS-mono600/mode/2up.

[5] Karsten Ahnert and Mario Mulansky. Boost odeint library. https://www.boost.org/doc/libs/latest/libs/numeric/odeint/doc/html/index.html June 2013. Documentation available as https://www.boost.org/doc/libs/latest/libs/numeric/odeint/doc/html/index.html.

[6] M. Akahira, M. Sato, and N. Torigoe. On the new approximation to non-central t-distributions. *J. Japan Statist. Soc*, 25(1):1–18, 1995. Available at http://www.tulips.tsukuba.ac.jp/dspace/bitstream/2241/118436/1/JJSS_25-1.pdf.

[7] A. Al-Babtain, A. M. Eid, and F. Merovci. The five parameter lindley distribution. *Pakistan Journal of Statistics*, 31(4):363–384, 2014.

[8] G. E. Alefeld, F. A. Potra, and Y. Shi. Algorithm 748: enclosing zeros of continuous functions. *ACM Transactions on Mathematical Software*, 21(3):327–344, 1995.

[9] T. W. Anderson. *An Introduction to Multivariate Statistical Analysis*. John Wiley & Sons, Inc., 3 edition, 2003.

[10] S. András and Á. Baricz. Properties of the probability density function of the non-central chi-squared distribution. *Journal of Mathematical Analysis and Applications*, 346(2):395 – 402, 2008. Available at http://www.sciencedirect.com/science/article/pii/S0022247X08005696. doi:http://dx.doi.org/10.1016/j.jmaa.2008.05.074.

[11] T. M. Apostol. On the lerch zeta function. *Pacific Journal of Mathematics*, 1(2):161–167, 1951. Available as https://pdfs.semanticscholar.org/3e70/b0ac7757934c92ea529e3c5fecb967281bab.pdf.

[12] T. M. Apostol. *Lerch's Transcendent*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/25.14.

[13] T. M. Apostol. *Riemann-Siegel Formula*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/25.10#ii.

[14] T. M. Apostol. *Zeta function Zeros*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/25.10.

[15] T.M. Apostol. *Bose-Einstein distribution*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/25.12#iii.

[16] T.M. Apostol. *Dilogarithms*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/25.12#i.

[17] T.M. Apostol. *Hurwitz Zeta Function*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/25.11.

[18] T.M. Apostol. *Polylogarithms*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/25.12#ii.

[19] T.M. Apostol. *Riemann Zeta Function*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/25.2.

[20] R. Askey, R.A.; Roy. *Barnes' G-Function (Double Gamma Function)*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/5.17.

[21] R. Askey, R.A.; Roy. *Beta Function*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/5.2.

[22] R. Askey, R.A.; Roy. *Gamma and Psi Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/5.2.

[23] R. Askey, R.A.; Roy. *Polygamma Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/5.15.

[24] R. A. Askey and A.B.O. Daalhuis. *Generalized Hypergeometric Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/16.2.

[25] N. Balakrishnan and D. Kundi. Birnbaum-saunders distribution: a review of models, analysis, and applications. *Applied Stochastic Models in Business in Industry*, 35(1):4–49, 2018. Available as https://onlinelibrary.wiley.com/doi/full/10.1002/asmb.2348.

[26] R. E. Bechhofer and C. W. Dunnett. *Tables of percentage points of multivariate student T distributions*. Selected Tables in Mathematical Statistics, 11, 1–371, 1988.

[27] Becker. Pearson distribution system, version 1.2.2. r package "pearsonds". 2022. Documentation available as https://cran.r-project.org/web/packages/PearsonDS/PearsonDS.pdf.

[28] R. Bellman, R.E. Kalaba, and J.A. Lockett. *Numerical inversion of the Laplace transform: Applications to Biology, Economics, Engineering, and Physics*. Elsevier, 1966.

[29] B.M. Bennett. On the non-null distribution of WILCOXON's signed rank test. *Metrika*, 19:36–38, 1972.

[30] Denise Benton and K. Krishnamoorthy. Computing discrete mixtures of continuous distributions: noncentral chisquare, noncentral t and the distribution of the square of the sample multiple correlation coefficient. *Computational Statistics & Data Analysis*, 14:249–267, 2003. Available as http://www.ucs.louisiana.edu/Âăkxk4695/CSDA-03.pdf.

[31] A. Berkopec. Hyperquick algorithm for discrete hypergeometric distribution. *Journal of Discrete Algorithms*, 5:341–347, 2007.

[32] S. Blinnikov and R. Moessner. Expansions for nearly gaussian distributions. *Astronomy and Astrophysics Supplement*, 130:193–205, 1998. Available as http://arxiv.org/abs/astro-ph/9711239.

[33] P. Bourke. *Chrysanthemum curve*. Geometry, Surfaces, Curves, Polyhedra, 2001. [Online; last edited on January 2001; accessed 22-Dec-2024]. URL: http://paulbourke.net/geometry/chrysanthemum/.

[34] P. Bourke. *Tanh Spiral*. Geometry, Surfaces, Curves, Polyhedra, 2001. [Online; last edited on January 2001; accessed 22-Dec-2024]. URL: https://paulbourke.net/geometry/spiral/.

[35] G.E.P. Box. A general distribution theory for a class of likelihood criteria. *Biometrika*, 36:317–346, 1949.

[36] R. Brent. *Algorithms for Minimization without Derivatives*. Prentice-Hall (Reprinted by Dover, 2002.), 1st edition, 1973.

[37] R.P. Brent. An algorithm with guaranteed convergence for finding a zero of a function. *The Computer Journal*, 1971.

[38] P. A. Bristow and H. Holin. *Bracket and Solve Root.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/roots_noderiv/bracket_solve.html.

[39] P. A. Bristow and H. Holin. *Cumulative Hazard Function.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/1_74_0/libs/math/doc/html/math_toolkit/dist_ref/nmp.html#math_toolkit.dist_ref.nmp.chf.

[40] P. A. Bristow and H. Holin. *Double-exponential quadrature: exp_sinh.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/double_exponential/de_exp_sinh.html.

[41] P. A. Bristow and H. Holin. *Double-exponential quadrature: sinh_sinh.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/double_exponential/de_sinh_sinh.html.

[42] P. A. Bristow and H. Holin. *Double-exponential quadrature: tanh_sinh.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/double_exponential/de_tanh_sinh.html.

[43] P. A. Bristow and H. Holin. *Fourier Integrals.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/fourier_integrals.html.

[44] P. A. Bristow and H. Holin. *Gauss-Kronrod Quadrature.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/gauss_kronrod.html.

[45] P. A. Bristow and H. Holin. *Gauss-Legendre quadrature.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/roots_deriv.html.

[46] P. A. Bristow and H. Holin. *Hazard Function.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/1_74_0/libs/math/doc/html/math_toolkit/dist_ref/nmp.html#math_toolkit.dist_ref.nmp.hazard.

[47] P. A. Bristow and H. Holin. *Locating Function Minima using Brent's algorithm.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/roots_deriv.html.

[48] P. A. Bristow and H. Holin. *Root Finding With Derivatives: Newton-Raphson, Halley and Schröder.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/roots_deriv.html.

[49] P. A. Bristow and H. Holin. *Trapezoidal Quadrature.* Boost Math Toolkit Library, 2013. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/trapezoidal.html.

[50] P. A. Bristow and H. Holin. *Arcsine Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/arcine_dist.html.

[51] P. A. Bristow and H. Holin. *Beta Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/beta_dist.html.

[52] P. A. Bristow and H. Holin. *Cauchy-Lorentz Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/cauchy_dist.html.

[53] P. A. Bristow and H. Holin. *Chi Squared Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/chi_squared_dist.html.

[54] P. A. Bristow and H. Holin. *Exponential Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/exp_dist.html.

[55] P. A. Bristow and H. Holin. *Extreme Value Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/extreme_dist.html.

[56] P. A. Bristow and H. Holin. *F Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/f_dist.html.

[57] P. A. Bristow and H. Holin. *Gamma (and Erlang) Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/gamma_dist.html.

[58] P. A. Bristow and H. Holin. *Inverse Gamma Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/inverse_gamma_dist.html.

[59] P. A. Bristow and H. Holin. *Inverse Gaussian (or Inverse Normal) Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/inverse_gamma_dist.html.

[60] P. A. Bristow and H. Holin. *Laplace Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/laplace_dist.html.

[61] P. A. Bristow and H. Holin. *Log Normal Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/lognormal_dist.html.

[62] P. A. Bristow and H. Holin. *Logistic Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/logistic_dist.html.

[63] P. A. Bristow and H. Holin. *Noncentral Beta Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/nc_beta_dist.html.

[64] P. A. Bristow and H. Holin. *Noncentral Chi-Squared Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/nc_chi_squared_dist.html.

[65] P. A. Bristow and H. Holin. *Noncentral F Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/nc_f_dist.html.

[66] P. A. Bristow and H. Holin. *Noncentral T Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/nc_t_dist.html.

[67] P. A. Bristow and H. Holin. *Normal (Gaussian) Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/normal_dist.html.

[68] P. A. Bristow and H. Holin. *Pareto Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/pareto.html.

[69] P. A. Bristow and H. Holin. *Students t Distribution.* Boost Math Toolkit Library, 2014. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/students_t_dist.html.

[70] P. A. Bristow and H. Holin. *Bernoulli Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/bernoulli_dist.html.

[71] P. A. Bristow and H. Holin. *Binomial Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/binomial_dist.html.

[72] P. A. Bristow and H. Holin. *Derivative of the Incomplete Beta Function.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_gamma/igamma_inv.html.

[73] P. A. Bristow and H. Holin. *Error Function Inverses.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_beta/ibeta_function.html.

[74] P. A. Bristow and H. Holin. *Error Function erf and complement erfc.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_gamma/gamma_derivatives.html.

[75] P. A. Bristow and H. Holin. *Expm1.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/powers/expm1.html.

[76] P. A. Bristow and H. Holin. *Geometric Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/geometric_dist.html.

[77] P. A. Bristow and H. Holin. *Hyperexponential Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/1_74_0/libs/math/doc/html/math_toolkit/dist_ref/dists/hyperexponential_dist.html.

[78] P. A. Bristow and H. Holin. *Hypergeometric Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/hypergeometric_dist.html.

[79] P. A. Bristow and H. Holin. *Incomplete Beta Functions.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_beta/ibeta_function.html.

[80] P. A. Bristow and H. Holin. *Incomplete Gamma Function Inverses.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_gamma/igamma_inv.html.

[81] P. A. Bristow and H. Holin. *Incomplete Gamma Functions.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_gamma/igamma.html.

[82] P. A. Bristow and H. Holin. *Inverse Chi Squared Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/1_74_0/libs/math/doc/html/math_toolkit/dist_ref/dists/inverse_chi_squared_dist.html.

[83] P. A. Bristow and H. Holin. *Negative Binomial Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/negative_binomial_dist.html.

[84] P. A. Bristow and H. Holin. *Poisson Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/poisson_dist.html.

[85] P. A. Bristow and H. Holin. *Rayleigh Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/rayleigh.html.

[86] P. A. Bristow and H. Holin. *Skew Normal Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/1_74_0/libs/math/doc/html/math_toolkit/dist_ref/dists/skew_normal_dist.html.

[87] P. A. Bristow and H. Holin. *The Incomplete Beta Function Inverses.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_gamma/igamma.html.

[88] P. A. Bristow and H. Holin. *Triangular Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/triangular_dist.html.

[89] P. A. Bristow and H. Holin. *Uniform Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/uniform_dist.html.

[90] P. A. Bristow and H. Holin. *Weibull Distribution.* Boost Math Toolkit Library, 2015. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/dist_ref/dists/weibull_dist.html.

[91] P. A. Bristow and H. Holin. *Auxiliary function Powm1.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/powers/powm1.html.

[92] P. A. Bristow and H. Holin. *Auxiliary function Sqrt1pm1.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/powers/sqrt1pm1.html.

[93] P. A. Bristow and H. Holin. *Bernoulli Numbers.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/number_series/bernoulli_numbers.html.

[94] P. A. Bristow and H. Holin. *Bessel Functions of the First and Second Kinds.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/bessel/bessel_first.html.

[95] P. A. Bristow and H. Holin. *Beta.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_beta/beta_function.html.

[96] P. A. Bristow and H. Holin. *Binomial Coefficients.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/factorials/sf_binomial.html.

[97] P. A. Bristow and H. Holin. *Cbrt.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/powers/log1p.html.

[98] P. A. Bristow and H. Holin. *CosPi.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/powers/cos_pi.html.

[99] P. A. Bristow and H. Holin. *Factorial.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/factorials/sf_factorial.html.

[100] P. A. Bristow and H. Holin. *Falling Factorial.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/factorials/sf_falling_factorial.html.

[101] P. A. Bristow and H. Holin. *Gamma.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_gamma/tgamma.html.

[102] P. A. Bristow and H. Holin. *Gamma.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_gamma/lgamma.html.

[103] P. A. Bristow and H. Holin. *Hypergeometric 1F1.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/hypergeometric/hypergeometric_1f1.html.

[104] P. A. Bristow and H. Holin. *Hypergeometric Function 0F1.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/hypergeometric/hypergeometric_0f1.html.

[105] P. A. Bristow and H. Holin. *Hypergeometric pFq.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/hypergeometric/hypergeometric_pfq.html.

[106] P. A. Bristow and H. Holin. *Lambert W function.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/lambert_w.html.

[107] P. A. Bristow and H. Holin. *Log1p.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/powers/log1p.html.

[108] P. A. Bristow and H. Holin. *Modified Bessel Functions of the First and Second Kinds.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/bessel/mbessel.html.

[109] P. A. Bristow and H. Holin. *Polygamma.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/lambert_w.html.

[110] P. A. Bristow and H. Holin. *Rising Factorial.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/factorials/sf_rising_factorial.html.

[111] P. A. Bristow and H. Holin. *SinPi.* Boost Math Toolkit Library, 2016. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/powers/sin_pi.html.

[112] P. A. Bristow and H. Holin. *Airy Ai Function.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/airy/ai.html.

[113] P. A. Bristow and H. Holin. *Airy Bi Function.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/airy/bi.html.

[114] P. A. Bristow and H. Holin. *Chebyshev Polynomials.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_poly/chebyshev.html.

[115] P. A. Bristow and H. Holin. *Cyclic Hankel Functions.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/hankel/cyl_hankel.html.

[116] P. A. Bristow and H. Holin. *Derivatives of the Bessel Functions.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/bessel/bessel_derivatives.html.

[117] P. A. Bristow and H. Holin. *Digamma.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_gamma/digamma.html.

[118] P. A. Bristow and H. Holin. *Double Factorial.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/factorials/sf_double_factorial.html.

[119] P. A. Bristow and H. Holin. *Finding Zeros of Bessel Functions of the First and Second Kinds.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/bessel/bessel_root.html.

[120] P. A. Bristow and H. Holin. *Gegenbauer Polynomials.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_poly/gegenbauer.html.

[121] P. A. Bristow and H. Holin. *Hermite Polynomials.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_poly/hermite.html.

[122] P. A. Bristow and H. Holin. *Hypotenuse.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/powers/hypot.html.

[123] P. A. Bristow and H. Holin. *Jacobi Polynomials.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_poly/jacobi.html.

[124] P. A. Bristow and H. Holin. *Laguerre (and Associated) Polynomials.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_poly/laguerre.html.

[125] P. A. Bristow and H. Holin. *Legendre (and Associated) Polynomials.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_poly/legendre.html.

[126] P. A. Bristow and H. Holin. *Ratios of Gamma Functions.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_gamma/gamma_ratios.html.

[127] P. A. Bristow and H. Holin. *SincPi.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sinc/sinc_pi.html.

[128] P. A. Bristow and H. Holin. *Spherical Bessel Functions of the First and Second Kinds.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/bessel/sph_bessel.html.

[129] P. A. Bristow and H. Holin. *Spherical Hankel Functions.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/hankel/sph_hankel.html.

[130] P. A. Bristow and H. Holin. *Spherical Harmonics.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_poly/sph_harm.html.

[131] P. A. Bristow and H. Holin. *Trigamma.* Boost Math Toolkit Library, 2017. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/sf_gamma/trigamma.html.

[132] P. A. Bristow and H. Holin. *Airy Ai' Function.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/airy/aip.html.

[133] P. A. Bristow and H. Holin. *Airy Bi' Function.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/airy/aip.html.

[134] P. A. Bristow and H. Holin. *Elliptic Integral D - Legendre Form *Boost Math Toolkit Library.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/ellint/ellint_d.html.

[135] P. A. Bristow and H. Holin. *Elliptic Integral Overview.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/ellint/ellint_intro.html.

[136] P. A. Bristow and H. Holin. *Elliptic Integrals - Carlson Form.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/1_85_0/libs/math/doc/html/math_toolkit/ellint/ellint_carlson.html.

[137] P. A. Bristow and H. Holin. *Elliptic Integrals of the First Kind - Legendre Form.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/ellint/ellint_1.html.

[138] P. A. Bristow and H. Holin. *Elliptic Integrals of the Second Kind - Legendre Form.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/ellint/ellint_2.html.

[139] P. A. Bristow and H. Holin. *Elliptic Integrals of the Third Kind - Legendre Form.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/ellint/ellint_3.html.

[140] P. A. Bristow and H. Holin. *Finding Zeros of Airy Functions.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/airy/airy_root.html.

[141] P. A. Bristow and H. Holin. *Heuman Lambda Function.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/ellint/heuman_lambda.html.

[142] P. A. Bristow and H. Holin. *Jacobi Elliptic Function cd.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_cd.html.

[143] P. A. Bristow and H. Holin. *Jacobi Elliptic Function cn.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_cn.html.

[144] P. A. Bristow and H. Holin. *Jacobi Elliptic Function cs.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_cs.html.

[145] P. A. Bristow and H. Holin. *Jacobi Elliptic Function dc.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_dc.html.

[146] P. A. Bristow and H. Holin. *Jacobi Elliptic Function dn.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_dn.html.

[147] P. A. Bristow and H. Holin. *Jacobi Elliptic Function ds.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_ds.html.

[148] P. A. Bristow and H. Holin. *Jacobi Elliptic Function nc.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_nc.html.

[149] P. A. Bristow and H. Holin. *Jacobi Elliptic Function nd.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_nd.html.

[150] P. A. Bristow and H. Holin. *Jacobi Elliptic Function ns.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_ns.html.

[151] P. A. Bristow and H. Holin. *Jacobi Elliptic Function sc.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_sc.html.

[152] P. A. Bristow and H. Holin. *Jacobi Zeta Function.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/ellint/jacobi_zeta.html.

[153] P. A. Bristow and H. Holin. *Overview of the Jacobi Elliptic Functions.* Boost Math Toolkit Library, 2018. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jac_over.html.

[154] P. A. Bristow and H. Holin. *Exponential Integral Ei.* Boost Math Toolkit Library, 2019. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/expint/expint_i.html.

[155] P. A. Bristow and H. Holin. *Exponential Integral En.* Boost Math Toolkit Library, 2019. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/expint/expint_n.html.

[156] P. A. Bristow and H. Holin. *Jacobi Elliptic Function sd.* Boost Math Toolkit Library, 2019. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_sd.html.

[157] P. A. Bristow and H. Holin. *Jacobi Elliptic Function sn.* Boost Math Toolkit Library, 2019. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/jacobi/jacobi_sn.html.

[158] P. A. Bristow and H. Holin. *Riemann Zeta Function.* Boost Math Toolkit Library, 2019. [Online; accessed 22-July-2024]. URL: https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/zetas/zeta.html.

[159] Paul A. Bristow, Hubert Holin, Christopher Kormanyos, Bruno Lalande, John Maddock, Johan Råde, Benjamin Sobotta, Gautam Sewani, Thijs van den Berg, Daryle Walker, and and Xiaogang Zhang. Boost math toolkit library. https://www.boost.org/doc/libs/1_54_0/libs/math/doc/html/index.html, June 2013. Documentation available as https://www.boost.org/doc/libs/latest/libs/math/doc/html/index.html.

[160] S.A. Broda and M. Paolella. Saddlepoint approximations for the doubly noncentral t distribution. *Computational Statistics & Data Analysis*, 51:2907–2918, 2007. Postprint available as http://www.zora.uzh.ch/16956/2/broda_dnct_07V.pdf.

[161] B. Malcolm Brown, Matthias Langer, Marco Marletta, Christiane Tretter, and Markus Wagenhofer. Eigenvalue enclosures and exclosures for non-self-adjoint problems in hydrodynamics. *LMS J. Comput. Math.*, 13:65 – 81, 2010.

[162] R. Bulirsch. An extension of the bartky-transformation to incomplete elliptic integrals of the third kind. *Numer. Math.*, 13(3):266–284, 1969.

[163] R. Bulirsch. Numerical calculation of elliptic integrals and elliptic functions. iii. *Numer. Math.*, 13(4):305–315, 1969.

[164] J. Burkardt. *The Truncated Normal Distribution*. Department of Scientific Computing, Florida State University, 2014. Available as https://people.sc.fsu.edu/Âăjburkardt/presentations/truncated_normal.pdf.

[165] R.W. Butler. *Saddlepoint Approximations with Applications*. Cambridge University Press, 1st edition, 2007.

[166] R.W. Butler and M.S. Paolella. Calculating the density and distribution function for the singly and doubly noncentral f. *Statistics and Computing*, 12:9–16, 2002. Working paper (1999) available at http://129.82.101.115/statresearch/stattechreports/Technical%20Reports/1999/99-6%20Butler%20Paolella.pdf.

[167] R.W. Butler and A.T.A. Wood. Laplace approximations for hypergeometric functions with matrix arguments. *Ann. Statist.*, 30:1155–1177, 2002. Available at (permanent url): http://projecteuclid.org/DPubS?service=UI&version=1.0&verb=Display&handle=euclid.aos/1031689021.

[168] R.W. Butler and A.T.A. Wood. Approximation of power in multivariate analysis. *Statistics and Computing*, 15(4):281–287, October 2005. URL: http://dx.doi.org/10.1007/s11222-005-4071-x, doi:10.1007/s11222-005-4071-x.

[169] Bender C.M. and S.A. Orszag. *Advanced Mathematical Methods for Scientists and Engineers*. Springer New York, 1999.

[170] L. Canal. A normal approximation for the chi-square distribution. *Computational Statistics & Data Analysis*, 48:803—808, 2005. Available at http://www.sciencedirect.com/science/article/pii/S0167947304001069.

[171] B. C. Carlson. *Elliptic Integrals*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/19.2.

[172] B. C. Carlson. *Elliptic Integrals (Symmetric Integrals)*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/19.16.

[173] B. C. Carlson. *Elliptic Integrals, Bulirsch-Integrals*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/19.2#iii.

[174] J. R. Cash and A. H. Karp. A variable order runge-kutta method for initial value problems with rapidly varying right-hand sides. *ACM Transactions on Mathematical Software*, 16:201–222, 1990.

[175] R. Chattamvelli and M. C. Jones. Recurrence relations for noncentral density, distribution functions and inverse moments. *Journal of Statistical Computation and Simulation*, 52(3):289–299, 1995. URL: http://www.tandfonline.com/doi/abs/10.1080/00949659508811679, arXiv:http://www.tandfonline.com/doi/pdf/10.1080/00949659508811679, doi:10.1080/00949659508811679.

[176] C. Chesneau. A new family of distributions based on the hypoexponential distribution with fitting reliability data. *Statistica*, 78(2):127—147, 2018. URL: https://doi.org/10.6092/issn.1973-2201/8260.

[177] M. Chiani. Distribution of the largest eigenvalue for real Wishart and Gaussian random matrices and a simple approximation for the Tracy-Widom distribution. *ArXiv e-prints*, September 2012. Available at http://adsabs.harvard.edu/abs/2012arXiv1209.3394C. arXiv:1209.3394.

[178] M. Chiani. Distribution of the largest root of a matrix for Roy's test in multivariate analysis of variance. *ArXiv e-prints*, January 2014. Available at http://adsabs.harvard.edu/abs/2014arXiv1401.3987C. arXiv:1401.3987.

[179] Youn-Min Chou. New representations for the doubly noncentral `f`-distribution and derived distribution. *Communications in Statistics - Theory and Methods*, 14(3):527–534, 1985. URL: http://www.tandfonline.com/doi/abs/10.1080/03610928508828931, arXiv:http://www.tandfonline.com/doi/pdf/10.1080/03610928508828931, doi:10.1080/03610928508828931.

[180] C.A. Coelho, F.J. Marques, and S.R. Oliveira. Near-exact distributions for likelihood ratio statistics used in the simultaneous test of conditions on mean vectors and patterns of covariance matrices. *Mathematical Problems in Engineering*, 2016:1 – 25, 2016.

[181] A.M. Cohen. *Numerical Methods for Laplace Transform Inversion*. Springer, Berlin, 2007.

[182] H. Cohen, F. Rodriguez Villegas, and D. Zagier. Convergence acceleration of alternating series. *Experiment. Math*, 9(1):3–12, 2000.

[183] J. D. Cohen. Noncentral chi-square: some observations on recurrence. *The American Statistician*, 42:120–122, 1988.

[184] S. Coles. *An Introduction to Statistical Modeling of Extreme Values*. Springer, London, 2007.

[185] R. M. Corless, G. H. Gonnet, D. E. G. Hare, D. J. Jeffrey, and D. E. Knuth. On the lambert w function. In *Advances in Computational Mathematics*, 329–359. 1996.

[186] E. A. Cornish and Ronald A. Fisher. Moments and cumulants in the specification of distributions. *Revue de l'Institut International de Statistique*, 5(4):307–320, 1938.

[187] G. E. Crooks. *Field Guide to Continuous Probability Distributions*. Berkeley Institute for Theoretical Sciences (BITS). ISBN: 978-1-7339381-0-5., 1996. Aavailable at http://www.arpapress.com/Volumes/Vol10Issue2/IJRRAS_10_2_02.pdf.

[188] G. E. Crooks. *The amoroso distribution*. arXiv preprint arXiv:1005.3274., 2010.

[189] A.A.M. Cuyt, V. Petersen, B. Verdonk, H. Waadeland, and W.B. Jones. *Handbook of Continued Fractions for Special Functions*. Springer, 2008.

[190] A.A.M. Cuyt, B. Verdonk, S. Becuve, and P. Kuterna. A remarkable example of catastrophic cancellation unraveled. *Computing*, 66(3):309–320, 2001.

[191] A. B. O. Daalhuis. *Whittaker Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/13.14.

[192] A.O. Daalhuis. *Hypergeometric Function: Gauss series*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/15.2#i.

[193] A.O. Daalhuis. *Kummer functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/13.2.

[194] C. Dagum. A new model of personal income distribution: specification and estimation. *Économie Appliquée*, 30:413–437, 1977.

[195] H. E. Daniels. Saddlepoint approximations in statistics. *Annals of Mathematical Statistics*, 25:631–650, 1954.

[196] H. E. Daniels. Tail probability approximations. *International Statistical Review*, 55:37–48, 1987.

[197] H. A. David. The power function of some tests based on range. *Biometrika*, 40:347–353, 1953.

[198] H. A. David, P. A. Lachenbruch, and H. P. Brandis. The power function of range and studentized range tests in normal samples. *Biometrika*, 59(1):161–168, 1972. Article Stable url: http://www.jstor.org/stable/2334627.

[199] S. T. David, M. G. Kendall, and A. Stuart. Some questions of distribution in the theory of rank correlation. *Biometrika*, 38(1/2):131–140, 1951. Article Stable url: http://www.jstor.org/stable/2332322.

[200] B. Davies. *Integral Transforms and their Applications*. Springer, 2008.

[201] B. Davies and B. Martin. Numerical inversion of the laplace transform: a survey and comparison of methods. *Journal of Computational Physics*, 33:1–32, 1979.

[202] A. W. Davis. A system of linear differential equations for the distribution of hotelling's generalized `t_0^2`. *The Annals of Mathematical Statistics*, 39(3):815–832, 06 1968. Available at http://dx.doi.org/10.1214/aoms/1177698313. URL: http://dx.doi.org/10.1214/aoms/1177698313, doi:10.1214/aoms/1177698313.

[203] A.W. Davis. Further applications of a differential equation for Hotelling's generalized `T_0^2`. *Annals of the Institute of Statistical Mathematics*, 22(1):77–87, 1970. Available at http://dx.doi.org/10.1007/BF02506325. URL: http://dx.doi.org/10.1007/BF02506325, doi:10.1007/BF02506325.

[204] A.W. Davis. Percentile approximations for a class of likelihood ratio criteria. *Biometrika*, 58:349–356, 1971.

[205] F. R. S. De Gusmao, E. M. M. Ortega, and G. M. Cordeiro. The generalized inverse weibull distribution. *Statistical Papers*, 52(3):591–619, 2011.

[206] Eigen Core Developers. *Fast Fourier Transform module*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/unsupported/group__FFT__Module.html.

[207] Eigen Core Developers. *Matrix cosine*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/unsupported/group__MatrixFunctions__Module.html#matrixbase_cos.

[208] Eigen Core Developers. *Matrix exponential*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/unsupported/group__MatrixFunctions__Module.html#matrixbase_exp.

[209] Eigen Core Developers. *Matrix functions module*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: http://eigen.tuxfamily.org/dox/unsupported/group__MatrixFunctions__Module.html.

[210] Eigen Core Developers. *Matrix hyberbolic cosine*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/unsupported/group__MatrixFunctions__Module.html#matrixbase_cosh.

[211] Eigen Core Developers. *Matrix hyperbolic sine*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/unsupported/group__MatrixFunctions__Module.html#matrixbase_sinh.

[212] Eigen Core Developers. *Matrix logarithm*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/unsupported/group__MatrixFunctions__Module.html#matrixbase_log.

[213] Eigen Core Developers. *Matrix raised to arbitrary real power*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/unsupported/group__MatrixFunctions__Module.html#matrixbase_pow.

[214] Eigen Core Developers. *Matrix sine*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/unsupported/group__MatrixFunctions__Module.html#matrixbase_sin.

[215] Eigen Core Developers. *Matrix square root*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/unsupported/group__MatrixFunctions__Module.html#matrixbase_sqrt.

[216] Eigen Core Developers. *Non-linear optimization*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: http://eigen.tuxfamily.org/dox/unsupported/group__NonLinearOptimization__Module.html.

[217] Eigen Core Developers. *Polynomial Solver Base*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/unsupported/classEigen_1_1PolynomialSolverBase.html.

[218] Eigen Core Developers. *Polynomials module*. Eigen, a C++ linear algebra library, 2023. [Online; accessed 22-July-2024]. URL: http://eigen.tuxfamily.org/dox/unsupported/group__Polynomials__Module.html.

[219] Eigen Core Developers. *ColPivHouseholderQR*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1ColPivHouseholderQR.html.

[220] Eigen Core Developers. *CompleteOrthogonalDecomposition*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1CompleteOrthogonalDecomposition.html.

[221] Eigen Core Developers. *ComplexEigenSolver*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1ComplexEigenSolver.html.

[222] Eigen Core Developers. *ComplexSchur*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1ComplexSchur.html.

[223] Eigen Core Developers. *FullPivHouseholderQR*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1FullPivHouseholderQR.html.

[224] Eigen Core Developers. *FullPivLU*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1FullPivLU.html.

[225] Eigen Core Developers. *GeneralizedEigenSolver*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1GeneralizedEigenSolver.html.

[226] Eigen Core Developers. *GeneralizedSelfAdjointEigenSolver*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1GeneralizedSelfAdjointEigenSolver.html.

[227] Eigen Core Developers. *HessenbergDecomposition*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1ComplexEigenSolver.html.

[228] Eigen Core Developers. *HouseholderQR*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1HouseholderQR.html.

[229] Eigen Core Developers. *JacobiSVD*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1JacobiSVD.html.

[230] Eigen Core Developers. *LDLT*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1LDLT.html.

[231] Eigen Core Developers. *LLT*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1LLT.html.

[232] Eigen Core Developers. *PartialPivLU*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1PartialPivLU.html.

[233] Eigen Core Developers. *PseudoEigenvalueMatrix*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1EigenSolver.html#a4979eafe0aeef06b19ada7fa5e19db17.

[234] Eigen Core Developers. *PseudoEigenvectors*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1EigenSolver.html#a4979eafe0aeef06b19ada7fa5e19db17.

[235] Eigen Core Developers. *RealQZ*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1RealQZ.html.

[236] Eigen Core Developers. *RealSchur*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1RealSchur.html.

[237] Eigen Core Developers. *Reductions and Partial Reductions*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/group__TutorialReductionsVisitorsBroadcasting.html.

[238] Eigen Core Developers. *SelfAdjointEigenSolver*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1SelfAdjointEigenSolver.html.

[239] Eigen Core Developers. *Tridiagonalization*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1Tridiagonalization.html.

[240] Eigen Core Developers. *Using BLAS/LAPACK from Eigen*. Eigen, a C++ linear algebra library, 2024. [Online; accessed 22-July-2024]. URL: https://libeigen.gitlab.io/eigen/docs-nightly/TopicUsingBlasLapack.html.

[241] Dharmawansa P, Rajatheva N. and K. Ahmed. On the distribution of the sum of nakagami-m random variables. *IEEE Transactions on Communications*, 55(7):1407–1416, 2007.

[242] K. Dilcher. *Bernoulli and Euler Polynomials*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/24.2.

[243] Wolfram Language Documentation. *Arcsine Distribution*. Wolfram Research, 2004. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/ArcSinDistribution.html.

[244] Wolfram Language Documentation. *Inverse Gamma Distribution*. Wolfram Research, 2004. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/InverseGammaDistribution.html.

[245] Wolfram Language Documentation. *Noncentral Student's t-Distribution*. Wolfram Research, 2004. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/NoncentralBetaDistribution.html.

[246] Wolfram Language Documentation. *Birnbaum-Saunders Distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/BirnbaumSaundersDistribution.html.

[247] Wolfram Language Documentation. *Exponential power distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/ExponentialPowerDistribution.html.

[248] Wolfram Language Documentation. *Gompertz-Makeham distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/GompertzMakehamDistribution.html.

[249] Wolfram Language Documentation. *Hyperexponential Distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/HyperexponentialDistribution.html.

[250] Wolfram Language Documentation. *Hypoexponential distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/HypoexponentialDistribution.html.

[251] Wolfram Language Documentation. *Inverse ChiSquare Distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/InverseChiSquareDistribution.html.

[252] Wolfram Language Documentation. *Kumaraswamy Distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/KumaraswamyDistribution.html.

[253] Wolfram Language Documentation. *Moyal Distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/MoyalDistribution.html.

[254] Wolfram Language Documentation. *Nakagami Distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/NakagamiDistribution.html.

[255] Wolfram Language Documentation. *Shifted Gompertz distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/ShiftedGompertzDistribution.html.

[256] Wolfram Language Documentation. *SkewNormal Distribution*. Wolfram Research, 2005. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/SkewNormalDistribution.html.

[257] Wolfram Language Documentation. *Inverse of the Regularized Incomplete Beta Function*. Wolfram Research, 2006. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/InverseBetaRegularized.html.

[258] Wolfram Language Documentation. *Inverse of the Regularized Incomplete Gamma Function*. Wolfram Research, 2006. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/InverseGammaRegularized.html.

[259] Wolfram Language Documentation. *Regularized (Gauss) Hypergeometric Function*. Wolfram Research, 2009. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/Hypergeometric2F1Regularized.html.

[260] Wolfram Language Documentation. *Regularized Confluent Hypergeometric Function of the First Kind*. Wolfram Research, 2009. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/Hypergeometric1F1Regularized.html.

[261] Wolfram Language Documentation. *Regularized Confluent Hypergeometric Limit Function*. Wolfram Research, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ConfluentHypergeometricLimitFunction.html.

[262] Wolfram Language Documentation. *Airy Ai Function*. Wolfram Research, 2011. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/AiryAi.html.

[263] Wolfram Language Documentation. *Airy Ai' Function*. Wolfram Research, 2011. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/AiryAiPrime.html.

[264] Wolfram Language Documentation. *Airy Bi Function*. Wolfram Research, 2011. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/AiryBi.html.

[265] Wolfram Language Documentation. *Airy Bi' Function*. Wolfram Research, 2011. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/AiryBiPrime.html.

[266] Wolfram Language Documentation. *Jacobi Elliptic Function cd*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiCD.html.

[267] Wolfram Language Documentation. *Jacobi Elliptic Function cn*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiCN.html.

[268] Wolfram Language Documentation. *Jacobi Elliptic Function cs*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiCS.html.

[269] Wolfram Language Documentation. *Jacobi Elliptic Function dc*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiDC.html.

[270] Wolfram Language Documentation. *Jacobi Elliptic Function dn*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiDN.html.

[271] Wolfram Language Documentation. *Jacobi Elliptic Function ds*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiDS.html.

[272] Wolfram Language Documentation. *Jacobi Elliptic Function nc*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiNC.html.

[273] Wolfram Language Documentation. *Jacobi Elliptic Function nd*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiND.html.

[274] Wolfram Language Documentation. *Jacobi Elliptic Function ns*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiNS.html.

[275] Wolfram Language Documentation. *Jacobi Elliptic Function sc*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiSC.html.

[276] Wolfram Language Documentation. *Jacobi Elliptic Function sd*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiSD.html.

[277] Wolfram Language Documentation. *Jacobi Elliptic Function sn*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/JacobiSN.html.

[278] Wolfram Language Documentation. *Jacobi Theta Functions, First Derivative*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/EllipticThetaPrime.html.

[279] Wolfram Language Documentation. *Neville ThetaC*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/NevilleThetaC.html.

[280] Wolfram Language Documentation. *Neville ThetaD*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/NevilleThetaD.html.

[281] Wolfram Language Documentation. *Neville ThetaN*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/NevilleThetaN.html.

[282] Wolfram Language Documentation. *Neville ThetaS*. Wolfram Research, 2012. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/NevilleThetaS.html.

[283] Wolfram Language Documentation. *Weierstrass HalfPeriods*. Wolfram Research, 2013. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/WeierstrassHalfPeriods.html.

[284] Wolfram Language Documentation. *Weierstrass Invariants*. Wolfram Research, 2013. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/WeierstrassInvariants.html.

[285] Wolfram Language Documentation. *Parabolic Cylinder Function D*. Wolfram Research, 2016. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/ParabolicCylinderD.html.

[286] Wolfram Language Documentation. *Regularized Generalized Hypergeometric Function Function 1F2*. Wolfram Research, 2016. [Online; accessed 22-July-2024]. URL: https://reference.wolfram.com/language/ref/HypergeometricPFQRegularized.html.

[287] J.R. Dormand and P.J. Prince. A family of embedded runge-kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19 – 26, 1980. URL: http://www.sciencedirect.com/science/article/pii/0771050X80900133, doi:http://dx.doi.org/10.1016/0771-050X(80)90013-3.

[288] M. Dowell and P. Jarratt. The "pegasus" method for computing the root of an equation. *BIT*, 12:503–508, 1972. doi:https://doi.org/10.1007/BF01932959.

[289] E. Dubinov and A. A. Dubinova. Exact integral-free expressions for the integral debye functions. *Technical Physics Letters*, 34(12):503–508, 2008.

[290] D.G. Duffy. On the numerical inversion of laplace transforms: comparison of three new methods on characteristic problems from applications. *ACM Transactions on Mathematical Software*, 19(3):333–359, 1993.

[291] D.G. Duffy. *Advanced Engineering Mathematics*. CRC Press, 1998.

[292] Charles W. Dunnett. A multiple comparison procedure for comparing several treatments with a control. *Journal of the American Statistical Association*, 50:1096–1121, 1955.

[293] T.M. Dunster. *Legendre and Related Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/14.2.

[294] T.M. Dunster. *Spherical and Spheroidal Harmonics*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/14.30.

[295] C. Dutang, V. Goulet, and N. Langevin. Feller-pareto and related distributions: numerical implementation and actuarial applications. *Journal of Statistical Software*, 12:1–22, 2022.

[296] A. Dzieciol, S. Yngve, and P. O. Fröman. Coulomb wave functions with complex values of the variable and the parameters. *J. Math. Phys.*, 2022. doi:https://doi.org/10.1063/1.533083.

[297] W. Ehrhardt. *The AMath and DAMath Special Functions*. Reference Manual and Implementation notes. Version 2.27, 2018. [Online; accessed 22-July-2024]. URL: https://web.archive.org/web/20181221065629/http://wolfgang-ehrhardt.de/specialfunctions.pdf.

[298] O. Espinosa and V. Moll. A generalized polygamma function. *Integral Transforms and Special Functions*, 2004.

[299] E. Fehlberg. Klassische runge-kutta-formeln vierter und niedrigerer ordnung mit schrittweiten-kontrolle und ihre anwendung auf wärmeleitungsprobleme. *Computing (Arch. Elektron. Rechnen)*, 6:61 – 71, 1980.

[300] S. A. Fellingham and D. J. Stoker. An approximation for the exact distribution of the wilcoxon test for symmetry. *Journal of the American Statistical Association*, 59:899–905, 1964. Article Stable url: http://www.jstor.org/stable/2283109.

[301] C. Ferreira and J. L. López. Asymptotic expansions of the hurwitz–lerch zeta function. *J. Math. Anal. Appl.*, 298:210–224, 2004. Available at https://www.sciencedirect.com/science/article/pii/S0022247X04003920.

[302] R. Ferréol. *Archimedean Spiral*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/archimede/archimede.shtml.

[303] R. Ferréol. *Cardioid*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/cardioid/cardioid.shtml.

[304] R. Ferréol. *Ceva trisectrix and sectrix*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/trisectricedeceva/trisectricedeceva.shtml.

[305] R. Ferréol. *Circle*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/cercle/cercle.shtml.

[306] R. Ferréol. *Cycloid*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/cycloid/cycloid.shtml.

[307] R. Ferréol. *Ellipse*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/ellipse/ellipse.shtml.

[308] R. Ferréol. *Epicycloid*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/epicycloid/epicycloid.shtml.

[309] R. Ferréol. *Epispiral*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/epi/epi.shtml.

[310] R. Ferréol. *Epitrochoid*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/epitrochoid/epitrochoid.shtml.

[311] R. Ferréol. *Fish curve*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/poisson/poisson.shtml.

[312] R. Ferréol. *Freeth's Nephroid*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/freeth/nephroiddefreeth.shtml.

[313] R. Ferréol. *Hyperbolic (or reciprocal) Spiral*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/hyperbolic/hyperbolic.shtml.

[314] R. Ferréol. *Hypocycloid*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/hypocycloid/hypocycloid.shtml.

[315] R. Ferréol. *Hypotrochoid*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; accessed 22-July-2024]. URL: https://mathcurve.com/courbes2d.gb/hypotrochoid/hypotrochoid.shtml.

[316] R. Ferréol. *Lamé curve*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/lame/lame.shtml.

[317] R. Ferréol. *Lemniscate of Bernoulli*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/lemniscate/lemniscate.shtml.

[318] R. Ferréol. *Lemniscate of Gerono, or Eight*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/gerono/gerono.shtml.

[319] R. Ferréol. *Limaçon (or snail) of Pascal*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/limacon/limacon.shtml.

[320] R. Ferréol. *Lissajous curve*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/lissajous/lissajous.shtml.

[321] R. Ferréol. *Lituus*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/lituus/lituus.shtml.

[322] R. Ferréol. *Logarithmic Spiral*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/logarithmic/logarithmic.shtml.

[323] R. Ferréol. *Rose*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/rosace/rosace.shtml.

[324] R. Ferréol. *Sluze Cubic*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/cubicsluze/cubicsluze.shtml.

[325] R. Ferréol. *Strophoid*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/strophoid/strophoid.shtml.

[326] R. Ferréol. *Trochoid*. Encyclopédie des formes mathématiques remarquables, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/trochoid/trochoid.shtml.

[327] R. Ferréol. *Cissoid*. Encyclopédie des formes mathématiques remarquables, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/cissoid/cissoid.shtml.

[328] R. Ferréol. *Cornu spiral or clothoid*. Encyclopédie des formes mathématiques remarquables, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/cornu/cornu.shtml.

[329] R. Ferréol. *Cyclic-harmonic curves*. Encyclopédie des formes mathématiques remarquables, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/conchoidderosace/conchoidderosace.shtml.

[330] R. Ferréol. *Maclaurin trisectrix*. Encyclopédie des formes mathématiques remarquables, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/maclaurin/maclaurin.shtml.

[331] R. Ferréol. *Nodal curve*. Encyclopédie des formes mathématiques remarquables, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/noeud/noeud.shtml.

[332] R. Ferréol. *Poinsot spiral*. Encyclopédie des formes mathématiques remarquables, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/poinsot/poinsot.shtml.

[333] R. Ferréol. *Rational circular cubic*. Encyclopédie des formes mathématiques remarquables, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/cubiccirculairerationnelle/cubiccirculairerationnelle.shtml.

[334] R. Ferréol. *Right rational circular cubic*. Encyclopédie des formes mathématiques remarquables, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/cubiccirculairerationnelle/cubiccirculairerationnelledroite.shtml.

[335] R. Ferréol. *SiCi Spiral*. Encyclopédie des formes mathématiques remarquables, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathcurve.com/courbes2d.gb/sici/sici.shtml.

[336] R. A. Fisher. The general sampling distribution of the multiple correlation coefficient. *Proceedings of the Royal Society of London. Series A*, 121(788):654–673, 1928. Article Stable url: http://www.jstor.org/stable/2283109.

[337] R. A. Fisher and E. A. Cornish. The percentile points of distributions having known cumulants. *Technometrics*, 2:209–225, 1960.

[338] Y. Fujikoshi. Asymptotic expansions of the distributions of test statistics in multivariate analysis. *J. Sci. Hiroshima Univ. Ser. A-I Math*, 34(1):73–144, 1970. Available at http://projecteuclid.org/euclid.hmj/1206138381.

[339] Y. Fujikoshi. Asymptotic formulas for the non-null distributions of three statistics for multivariate linear hypothesis. *Annals of the Institute of Statistical Mathematics*, 25:423–436, 1973.

[340] Y. Fujikoshi. Asymptotic expansions for the non-null distributions of three statistics in gmanova. *Annals of the Institute of Statistical Mathematics*, 26:289–297, 1974.

[341] Cordeiro G. The beta moyal: a useful skew distribution. *International Journal of Research and Reviews in Applied Sciences*, 10:171–192, 2012. Aavailable at http://www.arpapress.com/Volumes/Vol10Issue2/IJRRAS_10_2_02.pdf.

[342] D. Gaspard. Connection formulas between coulomb wave functions. 2018. Available at https://arxiv.org/abs/1804.10976.

[343] Constantine Gatsonis and Allan R. Sampson. Connection formulas between coulomb wave functions. *Psychological Bulletin*, 106(3):516–524, 1989.

[344] L. Gatteschi. Asymptotics and bounds for the zeros of laguerre polynomials: a survey. *Journal of Computational and Applied Mathematics*, 144:7–27, 2002.

[345] S. Geisser. Multivariate analysis of variance for a special covariance case. *Journal of the American Statistical Association*, 58:660–669, 1963.

[346] A. Genz and F. Bretz. *Computation of Multivariate Normal and t Probabilities, Lecture notes in Statistics 195*. Springer-Verlag, New York, 2009.

[347] A. Genz, F. Bretz, T. Miwa, X. Mi, F. Leisch, F. Scheipl, and T. Hothorn. *mvtnorm: Multivariate Normal and t Distributions. R package version 1.0-12*. CRAN R-project, 2020.

[348] Y. Gessner. *Analytische Eigenschaften doppelt nichtzentraler Verteilungen*. Dissertation. Universität Trier., 2014.

[349] P. Ginzberg. Quaternion matrices: statistical properties and applications to signal processing and wavelets. *PhD thesis, Imperial College*, 2013. Available as https://spiral.imperial.ac.uk:8443/bitstream/10044/1/18975/1/Ginzberg-P-2013-PhD-Thesis.pdf.

[350] P. Glasserman and J. Ruiz-Mata. Computing the credit loss distribution in the gaussian copula model: a comparison of methods. *Journal of Credit Risk*, 2(4):33–66, 2006. Extended and edited reprint from http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.22.6768.

[351] David Goldberg. What every computer scientist should know about floating point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991. Extended and edited reprint from http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.22.6768.

[352] P.R. Graves-Morris, D.E. Roberts, and A. Salam. The epsilon algorithm and related topics. *Journal of Computational and Applied Mathematics*, 2000.

[353] A. Gray, E. Abbena, and S. Salamon. *Modern Differential Geometry of Curves and Surfaces with Mathematica, 3rd ed.* CRC Press, Boca Raton, FL, 2006.

[354] F. Grubbs. Procedures for detecting outlying observations in samples. *Technometrics*, 11(1):1–21, 1969.

[355] William C. Guenther. Desk calculation of probabilities for the distribution of the sample correlation coefficient. *The American Statistician*, 31:45–48, 1977.

[356] A.K. Gupta and J. Tang. On a general distribution theory for a class of likelihood criteria. *Austral. J. Statist.*, 30:359–366, 1988.

[357] J. Gurland. A relatively simple form of the distribution of the multiple correlation coefficient. *Journal of the Royal Statistical Society, Series B, Methodological*, 30:276–283, 1968.

[358] J. Gurland and O. Asiribo. On the distribution of the multiple correlation coefficient and the kinked chi-square random variable. *Statistica Sinica*, 1:493–502, 1991. Article Stable url: https://www.jstor.org/stable/24304023?seq=1.

[359] J. Gurland and R. Milton. Further consideration of the distribution of the multiple correlation coefficient. *Journal of the Royal Statistical Society. Series B (Methodological)*, 32(3):381–394, 1970. Article Stable url: http://www.jstor.org/stable/2984364.

[360] M. Haas. A note on the moments of the skew-normal distribution. *Economics Bulletin*, 32(4):3306–3312, 2012.

[361] G. J. Hahn and R. W. Hendrickson. A table of percentage points of the distribution of the largest absolute value of k student t variates and its applications. *Biometrika*, 58(2):323–332, 1971.

[362] E. Hairer, S. P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer, Berlin, 2nd edition, 1993.

[363] B.I. Harley. Relation between the distributions of non-central t and of a transformed correlation coefficient. *Biometrika*, 44(1-2):219–224, 1957.

[364] H. L. Harter. Tables of range and studentized range. *The Annals of Mathematical Statistics*, 31(4):1122–1147, 1960. Article Stable url: http://www.jstor.org/stable/2237810.

[365] N. Hauberg. The non-central nakagami distribution. 2018. Available as http://www2.compute.dtu.dk/Âăsohau/papers/nakagami2018/nakagami.pdf.

[366] Maplesoft Online Help. *EllipticE, EllipticCE*. Maplesoft, 2024. [Online; accessed 22-July-2024]. URL: https://www.maplesoft.com/support/help/maple/view.aspx?path=EllipticE.

[367] Maplesoft Online Help. *EllipticF, EllipticK, EllipticCK*. Maplesoft, 2024. [Online; accessed 22-July-2024]. URL: https://www.maplesoft.com/support/help/maple/view.aspx?path=EllipticF.

[368] Maplesoft Online Help. *EllipticPi, EllipticCPi*. Maplesoft, 2024. [Online; accessed 22-July-2024]. URL: https://www.maplesoft.com/support/help/maple/view.aspx?path=EllipticPi.

[369] Maplesoft Online Help. *KelvinBer, KelvinBei*. Maplesoft, 2024. [Online; accessed 22-July-2024]. URL: https://www.maplesoft.com/support/help/Maple/view.aspx?path=Kelvin.

[370] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 1st edition, 2002. Online resource: http://www.maths.manchester.ac.uk/Âăhigham/asna/.

[371] Nicholas J. Higham. Accuracy and stability of numerical algorithms - presentation given at 3rd many-core and reconfigurable supercomputing network workshop queen's university, belfast, january 15-16, 2009. 2009. Available as http://cpc.cs.qub.ac.uk/MRSN/higham.pdf.

[372] I. D. Hill, R. Hill, and R. L. Holder. Algorithm as 99: fitting johnson curves by moments. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 1976.

[373] Y. Hochberg and A. C. Tamhane. *Multiple comparison procedures*. John Wiley & Sons, Inc., New York, NY, USA, 1987. ISBN 0-471-82222-1.

[374] H.H.H. Homeier. Scalar levin-type sequence transformations. *J. Comp. Appl. Math.*, 122:81–147, 2000. Available as https://arxiv.org/abs/math/0005209.

[375] Harold Hotelling. New light on the correlation coefficient and its transforms. *Journal of the Royal Statistical Society, Series B, Methodological*, 15:193–225, 1953.

[376] S. Hu and M.-S. Kim. On dirichlet's lambda functions. 2018. Available as https://arxiv.org/abs/1806.07762v2.

[377] A.D. Hutson. An alternative skew exponential power distribution formulation. *Commun Stat Theory Methods*, 48(12):3005–3024, 2019. Available as https://doi.org/10.1186/s13662-018-1527-9.

[378] R. James, Langou, J., and B. R. Lowery. On matrix balancing and eigenvector computation. 2014. Available as https://arxiv.org/pdf/1401.5766.pdf.

[379] R. Jaschke. Algorithm as 269: high order cornish-fisher expansion. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 41(11):233–240, 2001.

[380] J.L. Jensen. The modified signed likelihood statistic and saddlepoint approximations. *Annals of Mathematical Statistics*, 79:693–703, 1992.

[381] F. Jiménez and P. Pedro Jodrá. A note on the moments and computer generation of the shifted gompertz distribution. *Communications in Statistics - Theory and Methods*, 38(1):75–89, 2008.

[382] F. Jiménez Torres. Estimation of parameters of the shifted gompertz distribution using least squares, maximum likelihood and moments methods. *Journal of Computational and Applied Mathematics*, 255:867–877, 2014.

[383] Z. Jin. On the lucas polynomials and some of their new identities. *Adv Differ Equ*, 2018. Available as https://doi.org/10.1186/s13662-018-1527-9.

[384] P. Jodrá. Systems of frequency curves generated by methods of translation. *Biometrika*, 36(1/2):149–176, 1949.

[385] P. Jodrá. On order statistics from the gompertz–makeham distribution and the lambert w function. *Mathematical Modelling and Analysis*, 18(3):432–445, 2013.

[386] F. Johansson. Characteristic functions in the prob package. 2018. Available as https://cran.r-project.org/web/packages/prob/vignettes/charfunc.pdf.

[387] N. L. Johnson, S. Kotz, and N. Balakrishnan. *Continuous Univariate Distributions, Volume 1*. Wiley & Sons, Inc., New York-London-Sydney-Toronto, 2nd edition, 1994.

[388] N. L. Johnson, S. Kotz, and N. Balakrishnan. *Continuous Univariate Distributions, Volume 2*. Wiley & Sons, Inc., New York-London-Sydney-Toronto, 2nd edition, 1995.

[389] N. L. Johnson, S. Kotz, and N. Balakrishnan. *Univariate discrete distributions*. Wiley & Sons, Inc., New York-London-Sydney-Toronto, 3rd edition, 1995.

[390] M.C. Jones and M.J. Faddy. A skew extension of the t -distribution, with applications. *J. R. Statist. Soc. B*, 65:159–174, 2003.

[391] M.C. Jones and M.J. Faddy. Sinh-arcsinh distributions. *Biometrika*, 96:761–780, 2009.

[392] K. Kelley. Sample size planning for the squared multiple correlation coefficient: accuracy in parameter estimation via narrow confidence intervals. *Multivariate Behavioral Research*, 43:524–555, 2008.

[393] C. Kleiber and S. Kotz. *Statistical Size Distributions in Economics and Actuarial Sciences*. Wiley & Sons, Inc., New York-London-Sydney-Toronto, 2003.

[394] K. Kocherlakota and S. Kocherlakota. On the doubly noncentral t distribution. *Communications in Statistics - Simulation and Computation*, 20(1):23–31, 1991. URL: http://www.tandfonline.com/doi/abs/10.1080/03610919108812936, arXiv:http://www.tandfonline.com/doi/pdf/10.1080/03610919108812936, doi:10.1080/03610919108812936.

[395] P. Koev and A. Edelman. The efficient evaluation of the hypergeometric function of a matrix argument. *Mathematics of Computation*, 75:833–846, 2006. Article Stable url: http://math.mit.edu/Âăplamen/files/hyper.pdf, Software available at http://math.mit.edu/Âăplamen/software/mhgref.html. arXiv:arXiv:math/0505344.

[396] J. Kolassa and P. McCullagh. Edgeworth series for lattice distributions. *The Annals of Statistics*, 18:981–985, 1990.

[397] T.H. Koornwinder, R. S. C. Wong, R. Koekoek, and R. Swarttouw. *Orthogonal Polynomials*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/18.3.

[398] Kotz, S. and S. Nadarajah. *Extreme Value Distributions. Theory and Applications*. London: Imperial College Press, 2000.

[399] S.N. Krivoshapko and V.N. Ivanov. *Encyclopedia of Analytical Surfaces*. Springer International Publishing Switzerland, 2015.

[400] K.L. Kuhlman. Review of inverse laplace transform algorithms for laplace-space numerical approaches. *Numerical Algorithms*, 62:339–355, 2013.

[401] R. W. Kulp and B. N. Nagarsenker. An asymptotic expansion of the nonnull distribution of wilks criterion for testing the multivariate linear hypothesis. *The Annals of Statistics*, 12(4):1576–1583, 1984. Available at http://http://projecteuclid.org/euclid.aos/1176346816.

[402] D. Kumar. Accurately computing log(1-\exp (-|a|)). assessed from the rmpfr package. *International Journal of System Assurance Engineering and Management*, 2017. Available at https://www.researchgate.net/publication/315464331_The_Singh-Maddala_distribution_properties_and_estimation.

[403] W. Kutta. Beitrag zur näherungsweisen integration totaler differentialgleichungen. *Zeitschrift für Mathematik und Physik*, 46:435–453, 1901.

[404] Y.-S. Lee and T.-K. Lin. Algorithm as 269: high order cornish-fisher expansion. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 41(1):233–240, 1992. Article Stable url: http://www.jstor.org/stable/2347649.

[405] Yoong-Sin Lee. Asymptotic formulae for the distribution of a multivariate test statistic: power comparisons of certain multivariate tests. *Biometrika*, 58:647–651, 1971.

[406] Yoong-Sin Lee. Distribution of the canonical correlations and asymptotic expansions for distributions of certain independent test statistics. *The Annals of Mathematical Statistics*, 42:526–537, 1971. Available at http://projecteuclid.org/euclid.aoms/1177693403.

[407] Yoong-Sin Lee. Some results on the sampling distribution of the multiple correlation coefficient. *Journal of the Royal Statistical Society, Series B, Methodological*, 33:117–130, 1971.

[408] Yoong-Sin Lee. Tables of upper percentage points of the multiple correlation coefficient. *Biometrika*, 59:175–189, 1972.

[409] D.V. Lindley. Fiducial distributions and Bayes' theorem. *Journal of the Royal Statistical Society. Series B (Methodological)*, pages 102.107, 1958.

[410] Robert Lugannani and Stephen Rice. Saddle point approximation for the distribution of the sum of independent random variables. *Advances in Applied Probability*, 12(2):475–490, 1980.

[411] F.J. Marques, C.A. Coelho, and B.C. Arnold. A general near-exact distribution theory for the most common likelihood ratio test statistics used in multivariate analysis. *Test*, 20:180 – 203, 2011.

[412] J.W. Mauchly. Significance test for sphericity of a normal nvariate distribution. *Annals of Mathematical Statistics*, 11:204–209, 1940.

[413] J. Meier. *Tanh Spirale*. Parametrische Flächen und Körper, 2020. [Online; last edited on 13 April 2020; accessed 22-Dec-2024]. URL: http://www.3d-meier.de/tut3/Seite190.html.

[414] N. Michel. Precise coulomb wave functions for a wide range of complex l, eta and z. *Comput. Phys. Comm.*, 2007. Available as http://arxiv.org/abs/physics/0702051v1.

[415] R. C Milton. *Rank Order Probabilities. Two-Sample Normal Shift Alternatives*. Wiley & Sons, Inc., New York-London-Sydney-Toronto, 1970.

[416] M. Mori. *Developments in the double exponential formula for numerical integration, Proceedings of the International Congress of Mathematicians, Kyoto 1990*. Springer-Verlag, Berlin, 1991.

[417] J. J. Moré, B. S. Garbow, and K. E. Hillstrom. User guide for minpack-1, argonne national laboratory report anl-80-74, argonne, ill. 1980. Available as http://www.mcs.anl.gov/Âămore/ANL8074a.pdf (Chapter 1-3) and http://www.mcs.anl.gov/Âămore/ANL8074b.pdf (Chapter 4).

[418] R. J. Muirhead. *Aspects of Multivariate Statistical Theory*. John Wiley and Sons, 1982.

[419] R.J. Muirhead and Y. Chikuse. Approximations for the distributions of the extreme latent roots of three matrices. *Ann. Inst. Statist. Math.*, 27:473–478, 2007. Available as http://arxiv.org/abs/physics/0702051v1.

[420] H. Murakami and T. Kamakura. A saddlepoint approximation to a nonparametric test for ordered alternatives. *Journal of the Japan Statistical Society*, 39(2):143–153, dec 2009. URL: http://ci.nii.ac.jp/naid/10025991989/en/, doi:.

[421] B. N. Nagarsenker and J. Suniaga. Distributions of a class of statistics useful in multivariate analysis. *Journal of the American Statistical Association*, 78(382):472–475, 1983. URL: http://www.tandfonline.com/doi/abs/10.1080/01621459.1983.10477998, arXiv:http://www.tandfonline.com/doi/pdf/10.1080/01621459.1983.10477998, doi:10.1080/01621459.1983.10477998.

[422] S.C. Narula. Orthogonal polynomial regression for unequal spacing and frequencies. *Journal of Quality Technology*, 10:170–179, 1978.

[423] V. Nijimbere. Evaluation of some non-elementary integrals involving sine, cosine, exponential and logarithmic integrals: part 1. *Ural Mathematical Journal*, 2017.

[424] G.E. Noether. *Elements of nonparametric statistics*. SIAM series in applied mathematics. Wiley, 1967. URL: http://books.google.co.uk/books?id=BgFxIPl7tNAC.

[425] R. G. O'Brien and G. Shieh. Pragmatic, unifying algorithm gives power probabilities for common f tests of the multivariate general linear hypothesis. 1992. Poster presented at the American Statistical Association Meetings, Boston, Statistical Computing Section. Also, paper in review, downloadable in PDF form from http://www.bio.ri.ccf.org/UnifyPow.

[426] Robert E. Odeh. *Confidence limits on the correlation coefficient. In: Selected Tables in Mathematical Statistics, Vol. 10*. American Mathematical Society, 1986.

[427] The On-Line Encyclopedia of Integer Sequences. Hyperfactorials: product_k = 1..n k^k. 2000.

[428] The On-Line Encyclopedia of Integer Sequences. Superfactorials: product of first n factorials. 2000.

[429] T. Okayama. Explicit error bound for the tanh rule and the de formula for integrals with logarithmic singularity. *JSIAM Letters*, 6:9 – 11, 2014.

[430] T. Okayama, T. Matsuo, and M. Sugihara. Error estimates with explicit constants for sinc approximation, sinc quadrature and sinc indefinite integration. *Numerische Mathematik*, 124:361 – 394, 2013.

[431] Tomoaki Okayama. Explicit error bound for modified numerical iterated integration by means of sinc methods. In *Revised Selected Papers of the 6th International Conference on Mathematical Aspects of Computer and Information Sciences - Volume 9582*, MACIS 2015, 202–217. New York, NY, USA, 2016. Springer-Verlag New York, Inc. URL: http://dx.doi.org/10.1007/978-3-319-32859-1_17, doi:10.1007/978-3-319-32859-1_17.

[432] I. Olkin and S. J. Press. Testing and estimation for a circular stationary model. *Annals of Mathematical Statistics*, 40:1358–1373, 1969.

[433] F. W. J. Olver. *Airy Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/9.2.

[434] F. W. J. Olver. *Error Functions - Dawson's Integral*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/7.2#E5.

[435] F. W. J. Olver. *Error Functions - Faddeeva function*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/7.2#E3.

[436] F. W. J. Olver. *Error Functions - Goodwin-Staton Integral*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/7.2#E12.

[437] F. W. J. Olver. *Error Functions - Repeated Integrals of the Complementary Error Function*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/7.18.

[438] F. W. J. Olver. *Scorer Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/9.12.

[439] F. W. J. Olver. *Voigt Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/7.19.

[440] F. W. J. Olver and L. Maximon. *Bessel Function of the First Kind*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/10.2#E2.

[441] F. W. J. Olver and L. Maximon. *Bessel Function of the Second Kind*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/10.2#E3.

[442] L. Olver, F. W. J.; Maximon. *Bessel Function Zeros*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/10.25#E2.

[443] L. Olver, F. W. J.; Maximon. *Hankel Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: http://dlmf.nist.gov/10.2.E5.

[444] L. Olver, F. W. J.; Maximon. *Kelvin Functions - Definitions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/10.61.

[445] L. Olver, F. W. J.; Maximon. *Modified Bessel Function of the First Kind*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/10.25#E2.

[446] L. Olver, F. W. J.; Maximon. *Modified Bessel Function of the Second Kind*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/10.25#E3.

[447] L. Olver, F. W. J.; Maximon. *Spherical Bessel Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/10.47.

[448] L. Olver, F. W. J.; Maximon. *Spherical Hankel Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/10.47.

[449] Ooura, T. and Mori. M. The double exponential formula for oscillatory functions over the half infinite interval. *Journal of Computational and Applied Mathematics*, 38:353–360, 1991.

[450] Ooura, T. and Mori. M. A robust double exponential formula for fourier-type integrals. *Journal of computational and applied mathematics*, 112:229–241, 1999.

[451] T. Ooura. A double exponential formula for the fourier transforms. *Publ. RIMS, Kyoto Univ.*, 41:971–977, 2005. Available as https://pdfs.semanticscholar.org/16ec/a5d76fd6b3d7acaaff0b2a6e8a70caa70190.pdf.

[452] E. Orjebin. A recursive formula for the moments of a truncated univariate normal distribution. *The School of Mathematics and Physics, The University of Queensland, Australia*, 2014. URL: https://people.smp.uq.edu.au/YoniNazarathy/teaching_projects/studentWork/EricOrjebin_TruncatedNormalMoments.pdf.

[453] D. B. Owen. Tables for computing bivariate normal probabilities. *Ann. Math. Statist.*, 27:1075–1090, 1956. Available as http://projecteuclid.org/DPubS/Repository/1.0/Disseminate?view=body&id=pdf_1&handle=euclid.aoms/1177728074.

[454] D. B. Owen. A survey of properties and applications of the noncentral `t`-distribution. *Technometrics*, 10:445–478, 1968.

[455] M. S Paolella. *Fundamental Probability: A Computational Approach*. Wiley & Sons, Inc., New York-London-Sydney-Toronto, 2006.

[456] M. S Paolella. *Intermediate Probability: A Computational Approach*. Wiley & Sons, Inc., New York-London-Sydney-Toronto, 2007.

[457] R. B. Paris. *Anger–Weber Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/11.10.

[458] R. B. Paris. *Incomplete beta function*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/8.17.

[459] R. B. Paris. *Incomplete gamma function*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/8.2#i.

[460] R. B. Paris. *Lommel Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/11.9.

[461] R. B. Paris. *Struve and Modified Struve Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/11.2.

[462] R. B. Paros. *Generalized Exponential Integral*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/8.19.

[463] M. Patefield and D. Tand. Fast and accurate calculation of owen's t-function. *Journal of Statistical Software*, 2000. Available as http://www.jstatsoft.org/v05/i05/paper.

[464] P. B. Patnaik. The noncentral `\chi ^2`and `F`-distributions and their applications. *Biometrika*, 36:202—232, 1949.

[465] David B. Peizer and John W. Pratt. A normal approximation for binomial, `F`, beta, and other common, related tail probabilities. i (Ref: p1457-1483). *Journal of the American Statistical Association*, 63:1416–1456, 1968.

[466] S. Penev and T. Raykov. A wiener germ approximation of the noncentral chi square distribution and of its quantiles. *Computational Statistics*, 15:219–228, 2000. Preprint available at http://www.researchgate.net/publication/2648631_A_Wiener_Germ_Approximation_of_the_Noncentral_Chi_Square_Distribution_and_of_Its_Quantiles.

[467] K. Petras. Self-validating integration and approximation of piecewise analytic functions. *Journal of Computational and Applied Mathematics*, 145:345 – 359, 2002.

[468] K. Petras. Principles of verified numerical integration. *Journal of Computational and Applied Mathematics*, 199:317 – 328, 2007.

[469] T. Pham-Gia. Exact distribution of the generalized wilks' statistic and applications. *Journal of Multivariate Analysis*, 99(8):1698–1716, 2008. Available at http://www.sciencedirect.com/science/article/pii/S0047259X08000249. URL: http://www.sciencedirect.com/science/article/pii/S0047259X08000249, doi:http://dx.doi.org/10.1016/j.jmva.2008.01.021.

[470] R. Piessens, E. de Doncker-Kapenga, C.W. Überhuber, and D. Kahaner. *QUADPACK: A subroutine package for automatic integration*. Springer-Verlag, 1983. ISBN 978-3-540-12553-2. Public domain Fortran source: http://www.netlib.org/quadpack/.

[471] K.C.S Pillai and Young. On the exact distribution of hotelling's generalized t2. *Journal of Multivariate Analysis*, 99:90–107, 1971.

[472] J.F. Pires, A. H. M. A. Cysneiros, and F. Cribari-Netob. Improved inference for the generalized pareto distribution. *Brazilian Journal of Probability and Statistics*, 32:69–85, 2018. doi:http://dx.doi.org/10.1016/j.jmva.2008.01.021.

[473] M. J. D. Powell. *A hybrid method for nonlinear equations*. Numerical methods for nonlinear algebraic equations, Philip Rabinowitz, editor, chapter 6, pages 87-114, Gordon and Breach Science Publishers, New York, 1970.

[474] W.H. Press, S.A. Teukolsky, W.T. Vetterling, and B.P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, 3rd edition, 2007. Available from http://www.nr.com/.

[475] J. Reig, V. M. Rodrigo Peñarrocha, L. Rubio, M. T. Martínez-Inglés, and J. M. Molina-García-Pardo. The folded normal distribution: a new model for the small-scale fading in line-of-sight (los) condition. *IEEE Access*, 7:77328–77339, 2019.

[476] P. L. Reinhardt, W. P.; Walker. *Jacobian Elliptic Functions - Definitions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/22.2.

[477] P. L. Reinhardt, W. P.; Walker. *Jacobi's Amplitude(am) Function*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/22.16.

[478] P. L. Reinhardt, W. P.; Walker. *Theta Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/20.2.

[479] M.I. Riffi. A generalized transmuted gompertz-makeham distribution. *Journal of Scientific and Engineering Research*, 5:252–266, 2018.

[480] H. Rinne. *Taschenbuch der Statistik*. Frankfurt, M. : Deutsch, 4 edition, 2008.

[481] M.M. Ristić, B.V. Popović, and S. Nadarajah. Libby and novick's generalized beta exponential distribution. *Journal of Statistical Computation and Simulation*, 2013.

[482] Pierre Robillard. Kendall's S Distribution with Ties in One Ranking. *Journal of the American Statistical Association*, 67(338):453–455, 1972. URL: http://www.tandfonline.com/doi/abs/10.1080/01621459.1972.10482409, arXiv:http://www.tandfonline.com/doi/pdf/10.1080/01621459.1972.10482409, doi:10.1080/01621459.1972.10482409.

[483] R. N. Rodriguez. A guide to singh-maddala distributions. *Biometrika*, 64:129–134, 1977.

[484] F.W.J. Roy, R.; Olver. *Binomial Coefficients*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/1.2#i.

[485] F.W.J. Roy, R.; Olver. *Hyperbolic functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/4.28.

[486] F.W.J. Roy, R.; Olver. *Inverse Hyperbolic Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/4.37.

[487] F.W.J. Roy, R.; Olver. *Inverse Trigonometric Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/4.23.

[488] F.W.J. Roy, R.; Olver. *Lambert W-Function*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/4.13.

[489] F.W.J. Roy, R.; Olver. *Powers with General Bases*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/4.2#iv.

[490] F.W.J. Roy, R.; Olver. *The Exponential Function*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/4.2#iii.

[491] F.W.J. Roy, R.; Olver. *The Logarithm*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/4.2#i.

[492] F.W.J. Roy, R.; Olver. *Trigonometric functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/4.14.

[493] H. Ruben. Some new results on the distribution of the sample correlation coefficient. *Journal of the Royal Statistical Society, Series B, Methodological*, 28:513–525, 1966.

[494] S. M. Rump. Über die numerische auflösung von differentialgleichungen. *Mathematische Annalen, Springer*, 46(2):167–178, 1895.

[495] E. Schröder. Über unendlich viele algorithmen zur auflösung der gleichungen. *Mathematische Annalen, Springer*, 2:317–365, 1870.

[496] G.A.F. Seber. The non-central chi-square and beta distributions. *Biometrika*, 50:542–544, 1963.

[497] R. A. Shorack. Recursive generation of the distribution of the mann-whitney-wilcoxon u-statistic under generalized lehmann alternatives. *Annals of Mathematical Statistics*, 37(1):284–286, 1966. Available at http://projecteuclid.org/euclid.aoms/1177699621.

[498] R. A. Shorack. Tables of the distribution of the mann-whitney-wilcoxon u-statistic under lehmann alternatives. *Technometrics*, 9(4):666–677, 1967.

[499] R.T. Short. Computation of rice and noncentral chi-squared probabilities. 2012. Available as http://www.mcs.anl.gov/Âămore/ANL8074a.pdf.

[500] A. Sidi. *Practical Extrapolation Methods: Theory and Applications*. Cambridge Monographs on Applied and Computational Mathematics). Cambridge: Cambridge University Press., 2003.

[501] J.H. Skillings. On the null distribution of Jonckheere's statistic used in two way models for ordered alternatives. *Technometrics*, 22:431–436, 1980.

[502] Kadry S. Smaili K., Kadri T. Hypoexponential distribution with different parameters. *Applied Mathematics*, 4:431–436, 2013.

[503] B. Snow. The third moment of kendall's tau in normal samples. *Biometrika*, 49(1-2):177–183, 1962.

[504] W.C. Soong. Exact simultaneous confidence intervals for multiple comparisons with the mean. *Computational Statistics & Data Analysis*, 37:33–47, 2001.

[505] H. Stehfest. Algorithm 368: numerical inversion of laplace transforms. *Communications of the ACM*, 13(1):47–49, 1970.

[506] Rod Stephens. *WPF 3d: Three-Dimensional Graphics with WPF and C#*. CreateSpace Independent Publishing Platform, 2018. ISBN 978-1983905964. [Online; accessed 22-July-2024]. URL: https://github.com/WriterRod/WPF-3d-source.

[507] M. R. Stoline and H. K. Ury. Tables of the studentized maximum modulus distribution and an application to multiple comparisons among means. *Technometrics*, 21(1):269–271, 1979. Article Stable url: http://www.jstor.org/stable/1268584.

[508] Kathleen Subrahmaniam and Kocherlakota Subrahmaniam. Some extensions to miss f. n. david's tables of sample correlation coefficient: distribution function and percentiles. *Sankhya, Series B, Indian Journal of Statistics*, 45:75–147, 1983.

[509] R. M. Sundrum. Moments of the rank correlation coefficient tau in the general case. *Biometrika*, 40(3/4):409–420, 1953. Article Stable url: http://www.jstor.org/stable/2333357.

[510] R. M. Sundrum. A further approximation to the distribution of wilcoxon's statistic in the general case. *Journal of the Royal Statistical Society. Series B (Methodological)*, 16(2):255–260, 1954. Article Stable url: http://www.jstor.org/stable/2984051, a draft manuscript as available at http://repository.lib.ncsu.edu/dr/bitstream/1840.4/2253/1/ISMS_1954_89.pdf.

[511] H. Takahasi and M. Mori. Double exponential formulas for numerical integration. *Publications of the Research Institute for Mathematical Sciences, Kyoto University*, 9:721 – 741, 1974.

[512] A. Talbot. The accurate numerical inversion of laplace transforms. *IMA Journal of Applied Mathematics*, 23(1):97, 1979.

[513] J. Tang and A.K. Gupta. On the distribution of the product of independent beta random variables. *Statistics & Probability Letters*, 2:165–168, 1984.

[514] V. E. Tarasov. Some identities with generalized hypergeometric functions. *Appl. Math. Inf. Sci.*, 10:1729–1734, 2016.

[515] R Core Team. *The (non-central) Beta Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Beta.

[516] R Core Team. *The (non-central) Chi-Squared Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Chisquare.

[517] R Core Team. *The (non-central) F Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/FDist.

[518] R Core Team. *The (non-central) Student t Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/TDist.

[519] R Core Team. *The Beta Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Beta.

[520] R Core Team. *The Cauchy Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Cauchy.

[521] R Core Team. *The Chi-Squared Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Chisquare.

[522] R Core Team. *The Exponential Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Exponential.

[523] R Core Team. *The F Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/FDist.

[524] R Core Team. *The Gamma Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/GammaDist.

[525] R Core Team. *The Log Normal Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Lognormal.

[526] R Core Team. *The Normal Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Normal.

[527] R Core Team. *The Student t Distribution*. The R Project for Statistical Computing, 2019. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/TDist.

[528] R Core Team. *Distribution of the Wilcoxon Rank Sum Statistic*. The R Project for Statistical Computing, 2020. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Wilcoxon.

[529] R Core Team. *Distribution of the Wilcoxon Signed Rank Statistic*. The R Project for Statistical Computing, 2020. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/SignRank.

[530] R Core Team. *The Binomial Distribution*. The R Project for Statistical Computing, 2020. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Binomial.

[531] R Core Team. *The Geometric Distribution*. The R Project for Statistical Computing, 2020. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Geometric.

[532] R Core Team. *The Hypergeometric Distribution*. The R Project for Statistical Computing, 2020. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Hypergeometric.

[533] R Core Team. *The Negative Binomial Distribution*. The R Project for Statistical Computing, 2020. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/NegBinomial.

[534] R Core Team. *The Poisson Distribution*. The R Project for Statistical Computing, 2020. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Poisson.

[535] R Core Team. *The Studentized Range Distribution*. The R Project for Statistical Computing, 2020. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Tukey.

[536] R Core Team. *The Uniform Distribution*. The R Project for Statistical Computing, 2020. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Uniform.

[537] R Core Team. *The Weibull Distribution*. The R Project for Statistical Computing, 2020. [Online; accessed 22-July-2024]. URL: https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/Weibull.

[538] The Mpmath Development Team. *Auxiliary function Sincpi*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#sincpi.

[539] The Mpmath Development Team. *CDF of the normal distribution*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#ncdf.

[540] The Mpmath Development Team. *CosPi*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#cospi.

[541] The Mpmath Development Team. *Cosine*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#cos.

[542] The Mpmath Development Team. *Cube root*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#cbrt.

[543] The Mpmath Development Team. *Density of the normal distribution*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#npdf.

[544] The Mpmath Development Team. *Error Function complement erfc*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#erfc.

[545] The Mpmath Development Team. *Error Function erf*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#erf.

[546] The Mpmath Development Team. *Error Function erfi*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#erfi.

[547] The Mpmath Development Team. *Exponential Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#exp.

[548] The Mpmath Development Team. *Exponential Function expm1*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#expm1.

[549] The Mpmath Development Team. *General incomplete beta function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#betainc.

[550] The Mpmath Development Team. *Incomplete gamma function, general form*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#gammainc.

[551] The Mpmath Development Team. *Inverse of the Error Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#erfinv.

[552] The Mpmath Development Team. *Logarithm.function log1p*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#log1p.

[553] The Mpmath Development Team. *Logarithm.function, base 10*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#log10.

[554] The Mpmath Development Team. *Logarithm.function, base e*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#ln.

[555] The Mpmath Development Team. *Power function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#power.

[556] The Mpmath Development Team. *SinPi*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#sinpi.

[557] The Mpmath Development Team. *Sine*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#sin.

[558] The Mpmath Development Team. *Square root*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2008. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#sqrt.

[559] The Mpmath Development Team. *Cosecant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#csc.

[560] The Mpmath Development Team. *Cotangent*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#cot.

[561] The Mpmath Development Team. *Hyperbolic Cosecant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#csch.

[562] The Mpmath Development Team. *Hyperbolic Cosine*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#cosh.

[563] The Mpmath Development Team. *Hyperbolic Cotangent*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#coth.

[564] The Mpmath Development Team. *Hyperbolic Secant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#sech.

[565] The Mpmath Development Team. *Hyperbolic Sine*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#sinh.

[566] The Mpmath Development Team. *Hyperbolic Tangent*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#tanh.

[567] The Mpmath Development Team. *Inverse Cosecant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#acsc.

[568] The Mpmath Development Team. *Inverse Cosine*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#acos.

[569] The Mpmath Development Team. *Inverse Cotangent*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#acot.

[570] The Mpmath Development Team. *Inverse Hyperbolic Cosecant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#acsch.

[571] The Mpmath Development Team. *Inverse Hyperbolic Cosine*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#acosh.

[572] The Mpmath Development Team. *Inverse Hyperbolic Cotangent*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#acoth.

[573] The Mpmath Development Team. *Inverse Hyperbolic Secant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#atanh.

[574] The Mpmath Development Team. *Inverse Hyperbolic Sine*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#asinh.

[575] The Mpmath Development Team. *Inverse Hyperbolic Tangent*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hyperbolic.html#atanh.

[576] The Mpmath Development Team. *Inverse Secant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#asec.

[577] The Mpmath Development Team. *Inverse Sine*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#asin.

[578] The Mpmath Development Team. *Inverse Tangent*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#atan.

[579] The Mpmath Development Team. *Secant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#sec.

[580] The Mpmath Development Team. *Tangent*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2009. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#tan.

[581] The Mpmath Development Team. *Bernoulli number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/numtheory.html#bernoulli.

[582] The Mpmath Development Team. *Bernoulli polynomial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/numtheory.html#bernpoly.

[583] The Mpmath Development Team. *Bessel functions of the first kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#besselj.

[584] The Mpmath Development Team. *Bessel functions of the second kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#besselj.

[585] The Mpmath Development Team. *Beta Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#loggamma.

[586] The Mpmath Development Team. *Binomial coefficient*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#binomial.

[587] The Mpmath Development Team. *Factorial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#factorial-fac.

[588] The Mpmath Development Team. *Falling factorial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#ff.

[589] The Mpmath Development Team. *Gamma Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#gamma.

[590] The Mpmath Development Team. *Hypergeometric function 0F1*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hypergeometric.html#hyp0f1.

[591] The Mpmath Development Team. *Hypergeometric function 1F1 (Kummer)*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hypergeometric.html#hyp1f1.

[592] The Mpmath Development Team. *Hypergeometric function 2F1 (Gauss)*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hypergeometric.html#hyp1f1.

[593] The Mpmath Development Team. *Hypergeometric function U (Tricomi)*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hypergeometric.html#hyp1f1.

[594] The Mpmath Development Team. *Lambert W function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#lambert-w-function.

[595] The Mpmath Development Team. *LogGamma Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#loggamma.

[596] The Mpmath Development Team. *Modified Bessel Function of the First Kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#besselj.

[597] The Mpmath Development Team. *Modified Bessel Function of the Second Kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#besselj.

[598] The Mpmath Development Team. *Polygamma function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#lambert-w-function.

[599] The Mpmath Development Team. *Rising factorial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2010. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#rf.

[600] The Mpmath Development Team. *Apéry's Constant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/constants.html#apery-s-constant-apery.

[601] The Mpmath Development Team. *Arithmetic-Geometric Mean*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#agm.

[602] The Mpmath Development Team. *Base of natural logarithm*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/constants.html#base-of-the-natural-logarithm-e.

[603] The Mpmath Development Team. *Cardinal sine sinc*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#sinc.

[604] The Mpmath Development Team. *Catalan's Constant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/constants.html#catalan-s-constant-catalan.

[605] The Mpmath Development Team. *Degree constant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/constants.html#degree-degree.

[606] The Mpmath Development Team. *Digamma function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#psi-digamma.

[607] The Mpmath Development Team. *DoubleFactorial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#fac2.

[608] The Mpmath Development Team. *Euler Number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/numtheory.html#eulernum.

[609] The Mpmath Development Team. *Euler Polynomial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/numtheory.html#eulerpoly.

[610] The Mpmath Development Team. *Euler-Mascheroni Constant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/constants.html#euler-s-constant-euler.

[611] The Mpmath Development Team. *Glaisher-Kinkelin Constant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/constants.html#glaisher-s-constant-glaisher.

[612] The Mpmath Development Team. *Golden Ratio*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/constants.html#golden-ratio-phi.

[613] The Mpmath Development Team. *Harmonic Number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#harmonic.

[614] The Mpmath Development Team. *Hypotenuse*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#hypot.

[615] The Mpmath Development Team. *Inverse tangent (2 real arguments)*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#atan2.

[616] The Mpmath Development Team. *Khinchin's Constant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/constants.html#khinchin-s-constant-khinchin.

[617] The Mpmath Development Team. *Pi*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/constants.html#pi-pi.

[618] The Mpmath Development Team. *Power function powm1*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#powm1.

[619] The Mpmath Development Team. *Unit roots*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#unitroots.

[620] The Mpmath Development Team. *nth Root*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2011. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#root.

[621] The Mpmath Development Team. *Airy Ai Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#airyai.

[622] The Mpmath Development Team. *Airy Bi Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#airybi.

[623] The Mpmath Development Team. *Associated Legendre Polynomial Plm*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/orthogonal.html#legenp.

[624] The Mpmath Development Team. *Associated Legendre Polynomial Qlm*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/orthogonal.html#legenq.

[625] The Mpmath Development Team. *Barnes G-function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#barnesg.

[626] The Mpmath Development Team. *Chebyshev Polynomial of the First Kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/orthogonal.html#chebyt.

[627] The Mpmath Development Team. *Chebyshev Polynomial of the Second Kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/orthogonal.html#chebyu.

[628] The Mpmath Development Team. *Gegenbauer Polynomial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/orthogonal.html#gegenbauer.

[629] The Mpmath Development Team. *Hankel Function H1*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#hankel1.

[630] The Mpmath Development Team. *Hankel Function H2*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#hankel2.

[631] The Mpmath Development Team. *Hermite Polynomial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/orthogonal.html#hermite.

[632] The Mpmath Development Team. *Hyperfactorial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#hyperfac.

[633] The Mpmath Development Team. *Jacobi Polynomial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/orthogonal.html#jacobi.

[634] The Mpmath Development Team. *Laguerre Polynomial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/orthogonal.html#laguerre-polynomials.

[635] The Mpmath Development Team. *Legendre Polynomial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/orthogonal.html#legendre.

[636] The Mpmath Development Team. *Spherical Harmonic*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/orthogonal.html#spherical-harmonics.

[637] The Mpmath Development Team. *Superfactorial*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/gamma.html#superfac.

[638] The Mpmath Development Team. *Zeros of Airy Ai Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#airyaizero.

[639] The Mpmath Development Team. *Zeros of Airy Bi Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#airybizero.

[640] The Mpmath Development Team. *Zeros of Bessel Functions of the First Kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#besseljzero.

[641] The Mpmath Development Team. *Zeros of Bessel Functions of the Second Kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2012. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#besselyzero.

[642] The Mpmath Development Team. *Carlson Elliptic Integral RC*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#elliprc.

[643] The Mpmath Development Team. *Carlson Elliptic Integral RD*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#elliprd.

[644] The Mpmath Development Team. *Carlson Elliptic Integral RF*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#elliprf.

[645] The Mpmath Development Team. *Carlson Elliptic Integral RG*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#elliprg.

[646] The Mpmath Development Team. *Carlson Elliptic Integral RJ*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#elliprj.

[647] The Mpmath Development Team. *Chi*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#chi.

[648] The Mpmath Development Team. *Complete Elliptic Integral of the First Kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#ellipk.

[649] The Mpmath Development Team. *Complete Elliptic Integral of the Second Kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#ellipf.

[650] The Mpmath Development Team. *Cosine Integral*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#ci.

[651] The Mpmath Development Team. *Exponential Integral E1*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#e1.

[652] The Mpmath Development Team. *Exponential Integral Ei*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#ei.

[653] The Mpmath Development Team. *Exponential Integral En*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#expint.

[654] The Mpmath Development Team. *Fresnel Integral C*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#fresnelc.

[655] The Mpmath Development Team. *Fresnel Integral S*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#fresnels.

[656] The Mpmath Development Team. *General Elliptic Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#ellipfun.

[657] The Mpmath Development Team. *Incomplete Elliptic Integral of the Second Kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#ellipe.

[658] The Mpmath Development Team. *Incomplete Elliptic Integral of the Third Kind*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#ellippi.

[659] The Mpmath Development Team. *Logarithmic integral*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#li.

[660] The Mpmath Development Team. *Polylogarithm*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/zeta.html#polylog.

[661] The Mpmath Development Team. *Riemann and Hurwitz Zeta Function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/zeta.html#zeta.

[662] The Mpmath Development Team. *Shi*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#shi.

[663] The Mpmath Development Team. *Sine Integral*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2013. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/expintegrals.html#si.

[664] The Mpmath Development Team. *Absolute value of a number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#fabs.

[665] The Mpmath Development Team. *Addition of numbers*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#fadd.

[666] The Mpmath Development Team. *Ceiling of a number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#ceil.

[667] The Mpmath Development Team. *Complex argument (phase) of a number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#arg.

[668] The Mpmath Development Team. *Complex conjugate of a number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#conj.

[669] The Mpmath Development Team. *Complex number: conversion from polar coordinates*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#rect.

[670] The Mpmath Development Team. *Division of numbers*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#fdiv.

[671] The Mpmath Development Team. *Dot product of numbers*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#fdot.

[672] The Mpmath Development Team. *Floor of a number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#floor.

[673] The Mpmath Development Team. *Imaginary part of a number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#im.

[674] The Mpmath Development Team. *Modular division*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#fmod.

[675] The Mpmath Development Team. *Multiplication of numbers*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#fmul.

[676] The Mpmath Development Team. *Negation of numbers*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#fneg.

[677] The Mpmath Development Team. *Polar representation of a complex number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#polar.

[678] The Mpmath Development Team. *Product of numbers*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#fprod.

[679] The Mpmath Development Team. *Real part of a number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#re.

[680] The Mpmath Development Team. *Sign of a number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#sign.

[681] The Mpmath Development Team. *Subtraction of numbers*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#fsub.

[682] The Mpmath Development Team. *Summation of numbers*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2014. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#fsum.

[683] The Mpmath Development Team. *Chopping off small real or imaginary parts of a number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#chop.

[684] The Mpmath Development Team. *Frexp*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#frexp.

[685] The Mpmath Development Team. *Generation of a list of evenly spaced real numbers*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#linspace.

[686] The Mpmath Development Team. *Generation of a list of real numbers*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#arange.

[687] The Mpmath Development Team. *Klein j-invariant*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/elliptic.html#kleinj.

[688] The Mpmath Development Team. *Ldexp*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#ldexp.

[689] The Mpmath Development Team. *Random number*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#rand.

[690] The Mpmath Development Team. *Testing for NaN (not-a-number)*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#isnan.

[691] The Mpmath Development Team. *Testing if a number is finite*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#isfinite.

[692] The Mpmath Development Team. *Testing if a number is infinite*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#isinf.

[693] The Mpmath Development Team. *Testing if a number is integer-valued*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#isint.

[694] The Mpmath Development Team. *Testing if a number is normal*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#isnormal.

[695] The Mpmath Development Team. *Testing if numbers are almost equal*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2015. [Online; accessed 22-July-2024]. URL: https://mpmath.org/doc/1.1.0/general.html#almosteq.

[696] The Mpmath Development Team. *Backlund S function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2016. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/zeta.html#siegeltheta.

[697] The Mpmath Development Team. *Dirichlet eta function (Alternating zeta function)*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2016. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/zeta.html#altzeta.

[698] The Mpmath Development Team. *Generalized Clausen cosine function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2016. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/zeta.html#clcos.

[699] The Mpmath Development Team. *Generalized Clausen sine function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2016. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/zeta.html#clsin.

[700] The Mpmath Development Team. *Lerch's transcendent*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2016. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/zeta.html#lerchphi.

[701] The Mpmath Development Team. *Riemann-Siegel Z function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2016. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/zeta.html#siegelz.

[702] The Mpmath Development Team. *Riemann-Siegel theta function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2016. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/zeta.html#backlunds.

[703] The Mpmath Development Team. *Zeros of the Riemann zeta function*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2016. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/zeta.html#zetazero.

[704] The Mpmath Development Team. *Degrees*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2017. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#degrees.

[705] The Mpmath Development Team. *Exponential function expj*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2017. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#expj.

[706] The Mpmath Development Team. *Exponential function expjpi*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2017. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/powers.html#expjpi.

[707] The Mpmath Development Team. *Kelvin function bei*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2017. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#bei.

[708] The Mpmath Development Team. *Kelvin function ber*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2017. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#ber.

[709] The Mpmath Development Team. *Kelvin function kei*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2017. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#kei.

[710] The Mpmath Development Team. *Kelvin function ker*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2017. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#ker.

[711] The Mpmath Development Team. *Radians*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2017. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/trigonometric.html#radians.

[712] The Mpmath Development Team. *Struve function H*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2017. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#struveh.

[713] The Mpmath Development Team. *Struve function L*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2017. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#struvel.

[714] The Mpmath Development Team. *Coulomb wave function F*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#coulombf.

[715] The Mpmath Development Team. *Coulomb wave function G*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#coulombg.

[716] The Mpmath Development Team. *Generalized hypergeometric function 1F2*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/hypergeometric.html#hyp1f2.

[717] The Mpmath Development Team. *Normalizing Gamow constant for Coulomb wave functions*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#coulombc.

[718] The Mpmath Development Team. *Parabolic cylinder function D*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#pcfd.

[719] The Mpmath Development Team. *Parabolic cylinder function U*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#pcfu.

[720] The Mpmath Development Team. *Parabolic cylinder function V*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#pcfv.

[721] The Mpmath Development Team. *Parabolic cylinder function W*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#pcfw.

[722] The Mpmath Development Team. *Scorer function Gi*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#scorergi.

[723] The Mpmath Development Team. *Scorer function Hi*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#scorerhi.

[724] The Mpmath Development Team. *Whittaker function M*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#whitm.

[725] The Mpmath Development Team. *Whittaker function W*. mpmath - a Python library for arbitrary-precision floating-point arithmetic, 2018. [Online; accessed 22-July-2024]. URL: https://mpmath.readthedocs.io/en/latest/functions/bessel.html#whitw.

[726] The FLINT Team. *Acb: Bernoulli polynomial (in: Zeta function)*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb.html#zeta-function.

[727] The FLINT Team. *Acb: Error functions and Fresnel integrals*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_hypgeom.html#error-functions-and-fresnel-integrals.

[728] The FLINT Team. *Acb: Exponentials and logarithms*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb.html#exponentials-and-logarithms.

[729] The FLINT Team. *Acb: Gamma function and factorials*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb.html#gamma-function.

[730] The FLINT Team. *Acb: Hyperbolic functions*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb.html#hyperbolic-functions.

[731] The FLINT Team. *Acb: Incomplete gamma and beta functions*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_hypgeom.html#incomplete-gamma-and-beta-functions.

[732] The FLINT Team. *Acb: Inverse hyperbolic functions*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb.html#inverse-hyperbolic-functions.

[733] The FLINT Team. *Acb: Inverse trigonometric functions*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb.html#inverse-trigonometric-functions.

[734] The FLINT Team. *Acb: Lambert W function*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb.html#lambert-w-function.

[735] The FLINT Team. *Acb: Powers and roots*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb.html#powers-and-roots.

[736] The FLINT Team. *Acb: Trigonometric functions*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb.html#trigonometric-functions.

[737] The FLINT Team. *Arb: Error functions and Fresnel integrals*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb_hypgeom.html#error-functions-and-fresnel-integrals.

[738] The FLINT Team. *Arb: Exponentials and logarithms*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb.html#exponentials-and-logarithms.

[739] The FLINT Team. *Arb: Gamma function and factorials*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb.html#gamma-function-and-factorials.

[740] The FLINT Team. *Arb: Hyperbolic functions*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb.html#hyperbolic-functions.

[741] The FLINT Team. *Arb: Incomplete gamma and beta functions*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb_hypgeom.html#incomplete-gamma-and-beta-functions.

[742] The FLINT Team. *Arb: Inverse hyperbolic functions*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb.html#inverse-hyperbolic-functions.

[743] The FLINT Team. *Arb: Inverse trigonometric functions*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb.html#inverse-trigonometric-functions.

[744] The FLINT Team. *Arb: Lambert W function*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb.html#lambert-w-function.

[745] The FLINT Team. *Arb: Powers and roots*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb.html#powers-and-roots.

[746] The FLINT Team. *Arb: Trigonometric functions*. FLINT: Fast Library for Number Theory, 2019. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb.html#trigonometric-functions.

[747] The FLINT Team. *Acb: Bessel functions*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_hypgeom.html#bessel-functions.

[748] The FLINT Team. *Acb: Carlson symmetric elliptic integrals*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_elliptic.html#carlson-symmetric-elliptic-integrals.

[749] The FLINT Team. *Acb: Complete elliptic integrals*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_elliptic.html#complete-elliptic-integrals.

[750] The FLINT Team. *Acb: Confluent hypergeometric functions*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_hypgeom.html#confluent-hypergeometric-functions.

[751] The FLINT Team. *Acb: Gauss hypergeometric function*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_hypgeom.html#gauss-hypergeometric-function.

[752] The FLINT Team. *Acb: Jacobi theta functions*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_modular.html#jacobi-theta-functions.

[753] The FLINT Team. *Acb: Legendre incomplete elliptic integrals*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_elliptic.html#legendre-incomplete-elliptic-integrals.

[754] The FLINT Team. *Acb: Modified Bessel functions*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_hypgeom.html#bessel-functions.

[755] The FLINT Team. *Acb: Orthogonal polynomials and functions*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_hypgeom.html#orthogonal-polynomials-and-functions.

[756] The FLINT Team. *Arb: Bessel functions*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb_hypgeom.html#bessel-functions.

[757] The FLINT Team. *Arb: Confluent hypergeometric functions*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb_hypgeom.html#confluent-hypergeometric-functions.

[758] The FLINT Team. *Arb: Gauss hypergeometric function*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb_hypgeom.html#gauss-hypergeometric-function.

[759] The FLINT Team. *Arb: Orthogonal polynomials and functions*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb_hypgeom.html#orthogonal-polynomials-and-functions.

[760] The FLINT Team. *Arb: Other special functions: AGM*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb.html#other-special-functions.

[761] The FLINT Team. *Hurwitz zeta function*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. url: https://flintlib.org/doc/arb.html#zeta-function and https://flintlib.org/doc/acb_dirichlet.html#hurwitz-zeta-function.

[762] The FLINT Team. *Riemann zeta function*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_dirichlet.html#riemann-zeta-function.

[763] The FLINT Team. *Zeta function*. FLINT: Fast Library for Number Theory, 2020. [Online; accessed 22-July-2024]. url: https://flintlib.org/doc/arb.html#zeta-function and https://flintlib.org/doc/acb.html#zeta-function.

[764] The FLINT Team. *Acb: Coulomb wave functions*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_hypgeom.html#coulomb-wave-functions.

[765] The FLINT Team. *Acb: Dedekind eta function*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_modular.html#dedekind-eta-function.

[766] The FLINT Team. *Acb: Exponential and trigonometric integrals*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_hypgeom.html#exponential-and-trigonometric-integrals.

[767] The FLINT Team. *Acb: Hardy Z-functions*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_dirichlet.html#hardy-z-functions.

[768] The FLINT Team. *Acb: Lerch transcendent*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_dirichlet.html#lerch-transcendent.

[769] The FLINT Team. *Acb: Modular forms*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_elliptic.html#weierstrass-elliptic-functions.

[770] The FLINT Team. *Acb: Weierstrass elliptic functions*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/acb_elliptic.html#weierstrass-elliptic-functions.

[771] The FLINT Team. *Arb: Coulomb wave functions*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb_hypgeom.html#coulomb-wave-functions.

[772] The FLINT Team. *Arb: Exponential and trigonometric integrals*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. URL: https://flintlib.org/doc/arb_hypgeom.html#exponential-and-trigonometric-integrals.

[773] The FLINT Team. *Dilogarithm*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. url: https://flintlib.org/doc/acb_hypgeom.html#dilogarithm and https://flintlib.org/doc/arb_hypgeom.html#dilogarithm.

[774] The FLINT Team. *Polylogarithms*. FLINT: Fast Library for Number Theory, 2021. [Online; accessed 22-July-2024]. url: https://flintlib.org/doc/arb.html#polylogarithms and https://flintlib.org/doc/acb.html#polylogarithms.

[775] N. M. Temme. *Error Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/7.2#i.

[776] N. M. Temme. *Exponential and Logarithmic Integrals*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/6.2.

[777] N. M. Temme. *Fresnel Integrals*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/7.2#iii.

[778] N. M. Temme. *Hyperbolic Analogs of the Sine and Cosine Integrals*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/6.2#E16.

[779] N. M. Temme. *Logarithmic integral*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/6.2#E8.

[780] N. M. Temme. *Parabolic Cylinder Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/12.2.

[781] N. M. Temme. *Sine and Cosine Integrals*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/6.2#E11.

[782] I. J. Thompson. *Coulomb Functions*. NIST Digital Library of Mathematical Functions, 2010. URL: https://dlmf.nist.gov/33.

[783] I.J. Thompson and A.R. Barnett. Coulomb and bessel functions of complex arguments and order. *J. Comp. Phys.*, 1986.

[784] M.L. Tiku. A note on approximating to the non-central f-distribution. *Biometrika*, 53:606–610, 1966.

[785] Y. L. Tong. *The multivariate normal distribution*. Springer-Verlag, 1990.

[786] A. Townsend, T. Trogdon, and S. Olver. Fast computation of gauss quadrature nodes and weights on the whole real line. *IMA Journal of Numerical Analysis*, 36(1):337–358, 2016. Article Stable url: https://doi.org/10.1093/imanum/drv002.

[787] M. J. Tretter and G. W. Walster. Continued fractions for the incomplete beta function: additions and corrections. *Ann. Stat.*, 7(2):462–465, 1979. Available at http://projecteuclid.org/euclid.aos/1176344629.

[788] M. Tsagris, C. Beneki, and H. Hassani. On the folded normal distribution. *Mathematics*, 2(1):12–28, 2014.

[789] M. A. vandeWiel. Exact distributions of distribution-free test statistics. eindhoven: technische universiteit eindhoven. 2000. Available as http://www.mcs.anl.gov/Âămore/ANL8074a.pdf.

[790] C. Walck. *Handbook on Statistical Distributions for experimentalists*. University of Stockholm, Internal Report SUF-PFY/96–01, 2007. Available as http://www.stat.rice.edu/ÂădobelmanÂă/textfiles/DistributionsHandbook.pdf.

[791] G. W. Walster and M. J. Tretter. Exact noncentral distributions of wilks' `\lambda ` and wilks-lawley u criteria as mixtures of incomplete beta functions: for three tests. *Ann. Stat.*, 8(6):1388–1390, 1980. Available at http://projecteuclid.org/euclid.aos/1176345210.

[792] S. Wang. One-step saddlepoint approximations for quantiles. *Computational Statistics & Data Analysis*, 20:65–74, 1995.

[793] S. Wang and H.L. Gray. Approximating tail probabilities of noncentral distributions. *Computational Statistics & Data Analysis*, 15(3):343 – 352, 1993. Available at: http://www.sciencedirect.com/science/article/pii/016794739390261Q. URL: http://www.sciencedirect.com/science/article/pii/016794739390261Q, doi:http://dx.doi.org/10.1016/0167-9473(93)90261-Q.

[794] E. Wegert. *Visual Complex Functions - An Introduction with Phase Portraits*. Birkhäuser Basel, 2012.

[795] E. W. Weisstein. *Beta Distribution*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BetaDistribution.html.

[796] E. W. Weisstein. *Chi-Squared Distribution*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Chi-SquaredDistribution.html.

[797] E. W. Weisstein. *Correlation Coefficient–Bivariate Normal Distribution*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/CorrelationCoefficientBivariateNormalDistribution.html.

[798] E. W. Weisstein. *Cubic Formula*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/CubicFormula.html.

[799] E. W. Weisstein. *F-Distribution*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/F-Distribution.html.

[800] E. W. Weisstein. *Filon's Integration Formula*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/FilonsIntegrationFormula.html.

[801] E. W. Weisstein. *Gaussian Quadrature*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/GaussianQuadrature.html.

[802] E. W. Weisstein. *Halley's Method*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/HalleysMethod.html.

[803] E. W. Weisstein. *Hermite-Gauss Quadrature*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Hermite-GaussQuadrature.html.

[804] E. W. Weisstein. *Jacobi-Gauss Quadrature*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Jacobi-GaussQuadrature.html.

[805] E. W. Weisstein. *Laguerre-Gauss Quadrature*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Laguerre-GaussQuadrature.html.

[806] E. W. Weisstein. *Legendre-Gauss Quadrature*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Legendre-GaussQuadrature.html.

[807] E. W. Weisstein. *Newton's Method*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/NewtonsMethod.html.

[808] E. W. Weisstein. *Noncentral Chi-Squared Distribution*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/NoncentralChi-SquaredDistribution.html.

[809] E. W. Weisstein. *Noncentral F-Distribution*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/NoncentralF-Distribution.html.

[810] E. W. Weisstein. *Noncentral Student's t-Distribution*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/NoncentralStudentst-Distribution.html.

[811] E. W. Weisstein. *Quadratic Equation.* From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/QuadraticEquation.html.

[812] E. W. Weisstein. *Schröder's Method*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/SchroedersMethod.html.

[813] E. W. Weisstein. *Student's t-Distribution*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Studentst-Distribution.html.

[814] E. W. Weisstein. *Wynn's Epsilon Method*. From MathWorld - A Wolfram Web Resource, 2004. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/WynnsEpsilonMethod.html.

[815] E. W. Weisstein. *Bernoulli Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BernoulliDistribution.html.

[816] E. W. Weisstein. *Binomial Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BinomialDistribution.html.

[817] E. W. Weisstein. *Cauchy Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/CauchyDistribution.html.

[818] E. W. Weisstein. *Chi Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ChiDistribution.html.

[819] E. W. Weisstein. *Exponential Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ExponentialDistribution.html.

[820] E. W. Weisstein. *Gamma Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/GammaDistribution.html.

[821] E. W. Weisstein. *Geometric Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/GeometricDistribution.html.

[822] E. W. Weisstein. *Gumbel Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/GumbelDistribution.html.

[823] E. W. Weisstein. *Half-Normal distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Half-NormalDistribution.html.

[824] E. W. Weisstein. *Hypergeometric Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HypergeometricDistribution.html.

[825] E. W. Weisstein. *Inverse Gaussian (or Wald) Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseGaussianDistribution.html.

[826] E. W. Weisstein. *Laplace Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LaplaceDistribution.html.

[827] E. W. Weisstein. *Log Normal Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LogNormalDistribution.html.

[828] E. W. Weisstein. *Log-Series Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Log-SeriesDistribution.html.

[829] E. W. Weisstein. *Logistic Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LogisticDistribution.html.

[830] E. W. Weisstein. *Lévy Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LevyDistribution.html.

[831] E. W. Weisstein. *Maxwell Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/MaxwellDistribution.html.

[832] E. W. Weisstein. *Negative Binomial Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/NegativeBinomialDistribution.html.

[833] E. W. Weisstein. *Normal Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/NormalDistribution.html.

[834] E. W. Weisstein. *Pareto Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ParetoDistribution.html.

[835] E. W. Weisstein. *Poisson Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/PoissonDistribution.html.

[836] E. W. Weisstein. *Rayleigh Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/RayleighDistribution.html.

[837] E. W. Weisstein. *Triangular Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/TriangularDistribution.html.

[838] E. W. Weisstein. *Uniform Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/UniformDistribution.html.

[839] E. W. Weisstein. *Weibull Distribution*. From MathWorld - A Wolfram Web Resource, 2005. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/WeibullDistribution.html.

[840] E. W. Weisstein. *Binary Logarithm*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BinaryLogarithm.html.

[841] E. W. Weisstein. *Common Logarithm*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/CommonLogarithm.html.

[842] E. W. Weisstein. *Cube Root*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/CubeRoot.html.

[843] E. W. Weisstein. *Erf*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Erf.html.

[844] E. W. Weisstein. *Erfc*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Erfc.html.

[845] E. W. Weisstein. *Erfi*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Erfi.html.

[846] E. W. Weisstein. *Exponential Function*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ExponentialFunction.html.

[847] E. W. Weisstein. *Incomplete Beta Function*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/IncompleteBetaFunction.html.

[848] E. W. Weisstein. *Incomplete Gamma Function*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/IncompleteGammaFunction.html.

[849] E. W. Weisstein. *Inverse Erf*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/InverseErf.html.

[850] E. W. Weisstein. *Inverse Erfc*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/InverseErfc.html.

[851] E. W. Weisstein. *Logarithm*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Logarithm.html.

[852] E. W. Weisstein. *Natural Logarithm*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/NaturalLogarithm.html.

[853] E. W. Weisstein. *Power*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Power.html.

[854] E. W. Weisstein. *Regularized Incomplete Beta Function*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/RegularizedBetaFunction.html.

[855] E. W. Weisstein. *Regularized Incomplete Gamma Function*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/RegularizedGammaFunction.html.

[856] E. W. Weisstein. *Square*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Square.html.

[857] E. W. Weisstein. *Square Root*. From MathWorld - A Wolfram Web Resource, 2006. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/SquareRoot.html.

[858] E. W. Weisstein. *Cosecant*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Cosecant.html.

[859] E. W. Weisstein. *Cosine*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Cosine.html.

[860] E. W. Weisstein. *Cotangent*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Cotangent.html.

[861] E. W. Weisstein. *Hyperbolic Cosecant*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HyperbolicCosecant.html.

[862] E. W. Weisstein. *Hyperbolic Cosine*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HyperbolicCosine.html.

[863] E. W. Weisstein. *Hyperbolic Cotangent*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HyperbolicCotangent.html.

[864] E. W. Weisstein. *Hyperbolic Secant*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HyperbolicSecant.html.

[865] E. W. Weisstein. *Hyperbolic Sine*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HyperbolicSine.html.

[866] E. W. Weisstein. *Hyperbolic Tangent*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HyperbolicTangent.html.

[867] E. W. Weisstein. *Inverse Cosecant*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseCosecant.html.

[868] E. W. Weisstein. *Inverse Cosine*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseCosine.html.

[869] E. W. Weisstein. *Inverse Cotangent*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseCotangent.html.

[870] E. W. Weisstein. *Inverse Secant*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseSecant.html.

[871] E. W. Weisstein. *Inverse Sine*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseSine.html.

[872] E. W. Weisstein. *Inverse Tangent*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseTangent.html.

[873] E. W. Weisstein. *Secant*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Secant.html.

[874] E. W. Weisstein. *Sine*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Sine.html.

[875] E. W. Weisstein. *Tangent*. From MathWorld - A Wolfram Web Resource, 2007. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Tangent.html.

[876] E. W. Weisstein. *Bernoulli Number*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BernoulliNumber.html.

[877] E. W. Weisstein. *Bernoulli Polynomial*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BernoulliPolynomial.html.

[878] E. W. Weisstein. *Bessel Function of the First Kind.* From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BesselFunctionoftheFirstKind.html.

[879] E. W. Weisstein. *Bessel Function of the Second Kind.* From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BesselFunctionoftheSecondKind.html.

[880] E. W. Weisstein. *Beta Function*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BetaFunction.html.

[881] E. W. Weisstein. *Binomial coefficient*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BinomialCoefficient.html.

[882] E. W. Weisstein. *Factorial*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Factorial.html.

[883] E. W. Weisstein. *Falling factorial*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/FallingFactorial.html.

[884] E. W. Weisstein. *Gamma Function*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/GammaFunction.html.

[885] E. W. Weisstein. *Inverse Hyperbolic Cosecant*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseHyperbolicCosecant.html.

[886] E. W. Weisstein. *Inverse Hyperbolic Cosine*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseHyperbolicCosine.html.

[887] E. W. Weisstein. *Inverse Hyperbolic Cotangent*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseHyperbolicCotangent.html.

[888] E. W. Weisstein. *Inverse Hyperbolic Secant*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseHyperbolicSecant.html.

[889] E. W. Weisstein. *Inverse Hyperbolic Sine*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseHyperbolicSine.html.

[890] E. W. Weisstein. *Inverse Hyperbolic Tangent*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InverseHyperbolicTangent.html.

[891] E. W. Weisstein. *Lambert W-Function*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LambertW-Function.html.

[892] E. W. Weisstein. *LogGamma Function*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LogGammaFunction.html.

[893] E. W. Weisstein. *Polygamma Function*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/PolygammaFunction.html.

[894] E. W. Weisstein. *Reciprocal Gamma Function*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/GammaFunction.html.

[895] E. W. Weisstein. *Rising factorial (Pochhammer's Symbol)*. From MathWorld - A Wolfram Web Resource, 2008. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/RisingFactorial.html.

[896] E. W. Weisstein. *(Gauss) Hypergeometric Function*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HypergeometricFunction.html.

[897] E. W. Weisstein. *Apéry's Constant*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/AperysConstant.html.

[898] E. W. Weisstein. *Catalan's Constant*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/CatalansConstant.html.

[899] E. W. Weisstein. *Confluent Hypergeometric Function of the First Kind*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ConfluentHypergeometricFunctionoftheFirstKind.html.

[900] E. W. Weisstein. *Confluent Hypergeometric Function of the Second Kind*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ConfluentHypergeometricFunctionoftheSecondKind.html.

[901] E. W. Weisstein. *Confluent Hypergeometric Limit Function*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ConfluentHypergeometricLimitFunction.html.

[902] E. W. Weisstein. *Degree/Radian*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Radian.html.

[903] E. W. Weisstein. *Euler-Mascheroni Constant*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Euler-MascheroniConstant.html.

[904] E. W. Weisstein. *Glaisher-Kinkelin Constant*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Glaisher-KinkelinConstant.html.

[905] E. W. Weisstein. *Golden Ratio*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/GoldenRatio.html.

[906] E. W. Weisstein. *Khinchin's Constant*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/KhinchinsConstant.html.

[907] E. W. Weisstein. *Modified Bessel Function of the First Kind*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ModifiedBesselFunctionoftheFirstKind.html.

[908] E. W. Weisstein. *Modified Bessel Function of the Second Kind*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ModifiedBesselFunctionoftheSecondKind.html.

[909] E. W. Weisstein. *Natural Logarithm of 10*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/NaturalLogarithmof10.html.

[910] E. W. Weisstein. *Natural Logarithm of 2*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/NaturalLogarithmof2.html.

[911] E. W. Weisstein. *Pi*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Pi.html.

[912] E. W. Weisstein. *e*. From MathWorld - A Wolfram Web Resource, 2009. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/e.html.

[913] E. W. Weisstein. *Arithmetic-Geometric Mean*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Arithmetic-GeometricMean.html.

[914] E. W. Weisstein. *Barnes G-function*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BarnesG-Function.html.

[915] E. W. Weisstein. *Chebyshev Polynomial of the First Kind*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ChebyshevPolynomialoftheFirstKind.html.

[916] E. W. Weisstein. *Chebyshev Polynomial of the Second Kind*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ChebyshevPolynomialoftheFirstKind.html.

[917] E. W. Weisstein. *Digamma Function*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/DigammaFunction.html.

[918] E. W. Weisstein. *DoubleFactorial*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/DoubleFactorial.html.

[919] E. W. Weisstein. *Euler Number*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/EulerNumber.html.

[920] E. W. Weisstein. *Euler Polynomial*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/EulerPolynomial.html.

[921] E. W. Weisstein. *Gegenbauer Polynomial*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/GegenbauerPolynomial.html.

[922] E. W. Weisstein. *Generalized Harmonic Numbers*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HarmonicNumber.html.

[923] E. W. Weisstein. *Harmonic Numbers*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HarmonicNumber.html.

[924] E. W. Weisstein. *Hermite Polynomial*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HermitePolynomial.html.

[925] E. W. Weisstein. *Hyperfactorial*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Hyperfactorial.html.

[926] E. W. Weisstein. *Jacobi Polynomial*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/JacobiPolynomial.html.

[927] E. W. Weisstein. *Laguerre Polynomial*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LaguerrePolynomial.html.

[928] E. W. Weisstein. *Legendre Function of the Second Kind Q*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/LegendreFunctionoftheSecondKind.html.

[929] E. W. Weisstein. *Legendre Polynomial P*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LegendrePolynomial.html.

[930] E. W. Weisstein. *Logarithm of the Barnes G-function*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BarnesG-Function.html.

[931] E. W. Weisstein. *Prime Counting Function*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/PrimeCountingFunction.html.

[932] E. W. Weisstein. *Riemann Prime Counting Function*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/RiemannPrimeCountingFunction.html.

[933] E. W. Weisstein. *Root of Unity*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/RootofUnity.html.

[934] E. W. Weisstein. *Sinc Function*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/SincFunction.html.

[935] E. W. Weisstein. *Spherical Harmonic*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/SphericalHarmonic.html.

[936] E. W. Weisstein. *Superfactorial*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Superfactorial.html.

[937] E. W. Weisstein. *Trigamma Function*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/TrigammaFunction.html.

[938] E. W. Weisstein. *nth Root*. From MathWorld - A Wolfram Web Resource, 2010. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/nthRoot.html.

[939] E. W. Weisstein. *Airy Functions*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/AiryFunctions.html.

[940] E. W. Weisstein. *Hankel Function of the First Kind*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HankelFunctionoftheFirstKind.html.

[941] E. W. Weisstein. *Hankel Function of the Second Kind*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/HankelFunctionoftheSecondKind.html.

[942] E. W. Weisstein. *Modified Spherical Bessel Function of the First Kind*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/ModifiedSphericalBesselFunctionoftheFirstKind.html.

[943] E. W. Weisstein. *Modified Spherical Bessel Function of the Second Kind*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/ModifiedSphericalBesselFunctionoftheSecondKind.html.

[944] E. W. Weisstein. *Spherical Bessel Function of the First Kind*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/SphericalBesselFunctionoftheFirstKind.html.

[945] E. W. Weisstein. *Spherical Bessel Function of the Second Kind*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/SphericalBesselFunctionoftheSecondKind.html.

[946] E. W. Weisstein. *Spherical Hankel Function of the First Kind*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/SphericalHankelFunctionoftheFirstKind.html.

[947] E. W. Weisstein. *Zeros of the Airy function Ai*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/AiryFunctionZeros.html.

[948] E. W. Weisstein. *Zeros of the Airy function Bi*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/AiryFunctionZeros.html.

[949] E. W. Weisstein. *Zeros of the Bessel function of the first kind*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BesselFunctionZeros.html.

[950] E. W. Weisstein. *Zeros of the Bessel function of the second kind*. From MathWorld - A Wolfram Web Resource, 2011. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/BesselFunctionZeros.html.

[951] E. W. Weisstein. *Bose-Einstein Distribution*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Bose-EinsteinDistribution.html.

[952] E. W. Weisstein. *Complete Elliptic Integral of the First Kind*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/CompleteEllipticIntegraloftheFirstKind.html.

[953] E. W. Weisstein. *Complete Elliptic Integral of the Second Kind*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/CompleteEllipticIntegraloftheSecondKind.html.

[954] E. W. Weisstein. *Complete Elliptic Integral of the Third Kind*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/EllipticIntegraloftheThirdKind.html.

[955] E. W. Weisstein. *Dilogarithm*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Dilogarithm.html.

[956] E. W. Weisstein. *Elliptic Function*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/EllipticFunction.html.

[957] E. W. Weisstein. *Fermi-Dirac Distribution*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Fermi-DiracDistribution.html.

[958] E. W. Weisstein. *Heuman Lambda Function*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/JacobiZetaFunction.html.

[959] E. W. Weisstein. *Hurwitz Zeta Function*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/RiemannZetaFunction.html.

[960] E. W. Weisstein. *Incomplete Elliptic Integral of the First Kind*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/EllipticIntegraloftheFirstKind.html.

[961] E. W. Weisstein. *Incomplete Elliptic Integral of the Second Kind*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/EllipticIntegraloftheSecondKind.html.

[962] E. W. Weisstein. *Incomplete Elliptic Integral of the Third Kind*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/EllipticIntegraloftheThirdKind.html.

[963] E. W. Weisstein. *Inverse tangent integral*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/InverseTangentIntegral.html.

[964] E. W. Weisstein. *Jacobi Elliptic Functions*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/JacobiEllipticFunctions.html.

[965] E. W. Weisstein. *Jacobi Theta Functions*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/JacobiThetaFunctions.html.

[966] E. W. Weisstein. *Jacobi Zeta Function*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/JacobiZetaFunction.html.

[967] E. W. Weisstein. *Legendre's chi function*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/LegendresChi-Function.html.

[968] E. W. Weisstein. *Lemniscate Function*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/LemniscateFunction.html.

[969] E. W. Weisstein. *Polylogarithm*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Polylogarithm.html.

[970] E. W. Weisstein. *Riemann Zeta Function*. From MathWorld - A Wolfram Web Resource, 2012. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/RiemannZetaFunction.html.

[971] E. W. Weisstein. *Catalan Number*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/CatalanNumber.html.

[972] E. W. Weisstein. *Cosine Integral*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/CosineIntegral.html.

[973] E. W. Weisstein. *Dawson's Integral*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/DawsonsIntegral.html.

[974] E. W. Weisstein. *Dedekind Eta Function*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/DedekindEtaFunction.html.

[975] E. W. Weisstein. *Dirichlet Beta Function*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/DirichletBetaFunction.html.

[976] E. W. Weisstein. *Elliptic Lambda Function*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/EllipticLambdaFunction.html.

[977] E. W. Weisstein. *Erfc Differential Equation*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/ErfcDifferentialEquation.html.

[978] E. W. Weisstein. *Exponential Integral Ei*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ExponentialIntegral.html.

[979] E. W. Weisstein. *Exponential Integral En*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/En-Function.html.

[980] E. W. Weisstein. *Fresnel Integral C*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/FresnelIntegrals.html.WolframLanguageDocumentation:https://reference.wolfram.com/language/ref/FresnelC.html.

[981] E. W. Weisstein. *Fresnel Integral S*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/FresnelIntegrals.html.WolframLanguageDocumentation:https://reference.wolfram.com/language/ref/FresnelS.html.

[982] E. W. Weisstein. *Gudermannian function*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Gudermannian.html.

[983] E. W. Weisstein. *Haversine*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Haversine.html.

[984] E. W. Weisstein. *Hyperbolic Cosine Integral*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Chi.html.

[985] E. W. Weisstein. *Hyperbolic Sine Integral*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Shi.html.

[986] E. W. Weisstein. *Inverse Gudermannian function*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/InverseGudermannian.html.

[987] E. W. Weisstein. *Inverse Haversine*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/InverseHaversine.html.

[988] E. W. Weisstein. *Inverse of Weierstrass Elliptic Function*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/WeierstrassEllipticFunction.html.

[989] E. W. Weisstein. *Logarithmic integral*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LogarithmicIntegral.html.

[990] E. W. Weisstein. *Modular Discriminant*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/ModularDiscriminant.html.

[991] E. W. Weisstein. *Sine Integral*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/SineIntegral.html.

[992] E. W. Weisstein. *Weierstrass Elliptic Function*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/WeierstrassEllipticFunction.html.

[993] E. W. Weisstein. *Weierstrass Elliptic Function, First derivative*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/WeierstrassEllipticFunction.html.

[994] E. W. Weisstein. *Weierstrass Sigma*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/WeierstrassSigmaFunction.html.

[995] E. W. Weisstein. *Weierstrass Zeta Function*. From MathWorld - A Wolfram Web Resource, 2013. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/WeierstrassZetaFunction.html.

[996] E. W. Weisstein. *Debye Functions*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/DebyeFunctions.html.

[997] E. W. Weisstein. *Dirichlet Eta Function*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/DirichletEtaFunction.html.

[998] E. W. Weisstein. *Dirichlet Lambda function*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/DirichletLambdaFunction.html.

[999] E. W. Weisstein. *Elliptic Modulus k*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/EllipticModulus.html.

[1000] E. W. Weisstein. *Elliptic Nome q*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Nome.html.

[1001] E. W. Weisstein. *Elliptic parameter m*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Parameter.html.

[1002] E. W. Weisstein. *Fibonacci Number*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/FibonacciNumber.html.

[1003] E. W. Weisstein. *Fibonacci Polynomials*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Fibonacci_polynomials.

[1004] E. W. Weisstein. *Half-Period Ratio tau*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Half-PeriodRatio.html.

[1005] E. W. Weisstein. *Jacobi Amplitude*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/JacobiAmplitude.html.

[1006] E. W. Weisstein. *Klein's Absolute Invariant*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/KleinsAbsoluteInvariant.html.

[1007] E. W. Weisstein. *Lerch Transcendent*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LerchTranscendent.html.

[1008] E. W. Weisstein. *Logistic_function (Sigmoid Function)*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/SigmoidFunction.html.

[1009] E. W. Weisstein. *Logit Transformation*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/LogitTransformation.html.

[1010] E. W. Weisstein. *Lucas Number*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/LucasNumber.html.

[1011] E. W. Weisstein. *Lucas Polynomials*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/LucasPolynomial.html.

[1012] E. W. Weisstein. *Marcum Q function*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/MarcumQ-Function.html.

[1013] E. W. Weisstein. *Owen's T function*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/OwenT-Function.html.

[1014] E. W. Weisstein. *Riemann Xi*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Xi-Function.html.

[1015] E. W. Weisstein. *Riemann Zeta Function Zeros*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/RiemannZetaFunctionZeros.html.

[1016] E. W. Weisstein. *Riemann-Siegel Function Theta*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Riemann-SiegelFunctions.html.WolframLanguageDocumentation:https://reference.wolfram.com/language/ref/RiemannSiegelTheta.html.

[1017] E. W. Weisstein. *Riemann-Siegel Function Z*. From MathWorld - A Wolfram Web Resource, 2014. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Riemann-SiegelFunctions.html.WolframLanguageDocumentation:https://reference.wolfram.com/language/ref/RiemannSiegelZ.html.

[1018] E. W. Weisstein. *Anger Function J*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/AngerFunction.html.

[1019] E. W. Weisstein. *Cis*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/Cis.html.

[1020] E. W. Weisstein. *Clausen Function*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ClausenFunction.html.

[1021] E. W. Weisstein. *Kelvin Function Bei*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Bei.html.

[1022] E. W. Weisstein. *Kelvin Function Ber*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Ber.html.

[1023] E. W. Weisstein. *Kelvin Function Kei*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Kei.html.

[1024] E. W. Weisstein. *Kelvin Function Ker*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/Ker.html.

[1025] E. W. Weisstein. *Modified Struve Function L*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ModifiedStruveFunction.html.

[1026] E. W. Weisstein. *Struve Function H*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/StruveFunction.html.

[1027] E. W. Weisstein. *Weber Function E*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/WeberFunctions.html.

[1028] E. W. Weisstein. *q-Pochhammer Symbol*. From MathWorld - A Wolfram Web Resource, 2015. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/q-PochhammerSymbol.html.

[1029] E. W. Weisstein. *Coulomb Wave Function*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/CoulombWaveFunction.html.

[1030] E. W. Weisstein. *Double Series*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/DoubleSeries.html.

[1031] E. W. Weisstein. *Generalized Hypergeometric Function 1F2*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/GeneralizedHypergeometricFunction.html.

[1032] E. W. Weisstein. *Infinite Product*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/InfiniteProduct.html.

[1033] E. W. Weisstein. *Lommel Function*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/LommelFunction.html.

[1034] E. W. Weisstein. *Madelung Constants*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/MadelungConstants.html.

[1035] E. W. Weisstein. *Parabolic Cylinder Functions*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/ParabolicCylinderFunction.html.

[1036] E. W. Weisstein. *Scorer Function Gi*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/AiryFunctions.html.WolframLanguageDocumentation:https://reference.wolfram.com/language/ref/ScorerGi.html.

[1037] E. W. Weisstein. *Scorer Function Hi*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/AiryFunctions.html.WolframLanguageDocumentation:https://reference.wolfram.com/language/ref/ScorerHi.html.

[1038] E. W. Weisstein. *Whittaker Function M*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/WhittakerFunction.html..WolframLanguageDocumentation:https://reference.wolfram.com/language/ref/WhittakerM.html.

[1039] E. W. Weisstein. *Whittaker Function W*. From MathWorld - A Wolfram Web Resource, 2016. [Online; accessed 22-July-2024]. URL: http://mathworld.wolfram.com/WhittakerFunction.html..WolframLanguageDocumentation:https://reference.wolfram.com/language/ref/WhittakerW.html.

[1040] E. W. Weisstein. *Circle*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Circle.html.

[1041] E. W. Weisstein. *Cycloid*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Cycloid.html.

[1042] E. W. Weisstein. *Ellipse*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Ellipse.html.

[1043] E. W. Weisstein. *Epicycloid*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Epicycloid.html.

[1044] E. W. Weisstein. *Epitrochoid*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Epitrochoid.html.

[1045] E. W. Weisstein. *Fish curve*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/FishCurve.html.

[1046] E. W. Weisstein. *Heart curve*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/HeartCurve.html.

[1047] E. W. Weisstein. *Hypocycloid*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Hypocycloid.html.

[1048] E. W. Weisstein. *Hypotrochoid*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Hypotrochoid.html.

[1049] E. W. Weisstein. *Lemniscate of Bernoulli*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Lemniscate.html.

[1050] E. W. Weisstein. *Lemniscate of Gerono*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/EightCurve.html.

[1051] E. W. Weisstein. *Lissajous curve*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/LissajousCurve.html.

[1052] E. W. Weisstein. *Superellipse (Lamé curve)*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Superellipse.html.

[1053] E. W. Weisstein. *Trochoid*. From MathWorld - A Wolfram Web Resource, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Trochoid.html.

[1054] E. W. Weisstein. *Archimedes Spiral*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/ArchimedeanSpiral.html.

[1055] E. W. Weisstein. *Butterfly curve*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/ButterflyCurve.html.

[1056] E. W. Weisstein. *Cardioid*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Cardioid.html.

[1057] E. W. Weisstein. *Conchoid of de Sluze*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/ConchoidofdeSluze.html.

[1058] E. W. Weisstein. *Cycloid of Ceva*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/CycloidofCeva.html.

[1059] E. W. Weisstein. *Epispiral*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Epispiral.html.

[1060] E. W. Weisstein. *Fermat's Spiral*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/FermatsSpiral.html.

[1061] E. W. Weisstein. *Freeth's Nephroid*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/FreethsNephroid.html.

[1062] E. W. Weisstein. *Hyperbolic Spiral*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/HyperbolicSpiral.html.

[1063] E. W. Weisstein. *Limaçon curve*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Limacon.html.

[1064] E. W. Weisstein. *Lituus*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Lituus.html.

[1065] E. W. Weisstein. *Logarithmic Spiral*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Logarithmic_spiral.

[1066] E. W. Weisstein. *Rose curve*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/RoseCurve.html.

[1067] E. W. Weisstein. *Strophoid*. From MathWorld - A Wolfram Web Resource, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Strophoid.html.

[1068] E. W. Weisstein. *One-Sheeted Hyperboloid*. From MathWorld - A Wolfram Web Resource, 2020. [Online; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/One-SheetedHyperboloid.html.

[1069] E. W. Weisstein. *Acnode*. From MathWorld - A Wolfram Web Resource, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Acnode.html.

[1070] E. W. Weisstein. *Cissoid*. From MathWorld - A Wolfram Web Resource, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Cissoid.html.

[1071] E. W. Weisstein. *Cornu Spiral*. From MathWorld - A Wolfram Web Resource, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/CornuSpiral.html.

[1072] E. W. Weisstein. *Cotes' Spiral*. From MathWorld - A Wolfram Web Resource, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/CotesSpiral.html.

[1073] E. W. Weisstein. *Crunode*. From MathWorld - A Wolfram Web Resource, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Crunode.html.

[1074] E. W. Weisstein. *Cusp*. From MathWorld - A Wolfram Web Resource, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/Cusp.html.

[1075] E. W. Weisstein. *Maclaurin Trisectrix*. From MathWorld - A Wolfram Web Resource, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/MaclaurinTrisectrix.html.

[1076] E. W. Weisstein. *Nielsen's_spiral*. From MathWorld - A Wolfram Web Resource, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/NielsensSpiral.html.

[1077] E. W. Weisstein. *Poinsot's Spirals*. From MathWorld - A Wolfram Web Resource, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/PoinsotsSpirals.html.

[1078] E.J. Weniger. Nonlinear sequence transformations for the acceleration of convergence and the summation of divergent series. *Computer Physics Reports*, 10(5-6):189–371, 1989. URL: https://arxiv.org/abs/math/0306302.

[1079] Whittaker and Watson. *A Course of Modern Analysis*. Cambridge University Press, 4th edition, 1927.

[1080] D. Widder. *The Laplace Transform*. Princeton, 1941.

[1081] P. Wieschollek. Cppoptimizationlibrary. 2016. Available as https://github.com/PatWie/CppNumericalSolvers/tree/master.

[1082] Wikipedia. *Adaptive quadrature*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Adaptive_quadrature.

[1083] Wikipedia. *Anderson–Björck algorithm*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Regula_falsi#Anderson%E2%80%93Bj%C3%B6rck_algorithm.

[1084] Wikipedia. *Bell polynomials*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Bell_polynomials.

[1085] Wikipedia. *Bisection method*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Bisection_method.

[1086] Wikipedia. *Brent's method*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Brent%27s_method.

[1087] Wikipedia. *Cornish-Fisher expansion*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Cornish-Fisher_expansion.

[1088] Wikipedia. *Cubic equation*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Cubic_equation.

[1089] Wikipedia. *Durand-Kerner method*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Durand%E2%80%93Kerner_method.

[1090] Wikipedia. *Edgeworth_series*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Edgeworth_series#The_Edgeworth_series.

[1091] Wikipedia. *Halley's method*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Halley%27s_method.

[1092] Wikipedia. *Illinois_algorithm*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Regula_falsi#The_Illinois_algorithm.

[1093] Wikipedia. *Muller's method*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: http://en.wikipedia.org/wiki/Muller's_method.

[1094] Wikipedia. *Newton's method*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Newton%27s_method.

[1095] Wikipedia. *Numerical integration*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Numerical_integration.

[1096] Wikipedia. *QUADPACK*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/QUADPACK.

[1097] Wikipedia. *Quadratic equation*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Quadratic_equation.

[1098] Wikipedia. *Quartic equation*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Quartic_equation.

[1099] Wikipedia. *Ridders' method*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Ridders%27_method.

[1100] Wikipedia. *Secant method*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Secant_method.

[1101] Wikipedia. *Steffensen's method*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Steffensen%27s_method.

[1102] Wikipedia. *Truncation error (numerical integration)*. Wikimedia Foundation, 2004. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Truncation_error_(numerical_integration).

[1103] Wikipedia. *BulirschStoer algorithm.* Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Bulirsch%E2%80%93Stoer_algorithm.

[1104] Wikipedia. *Cash-Karp method.* Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Cash%E2%80%93Karp_method.

[1105] Wikipedia. *Clenshaw-Curtis quadrature*. Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Clenshaw%E2%80%93Curtis_quadrature.

[1106] Wikipedia. *Dormand-Prince method.* Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Dormand%E2%80%93Prince_method.

[1107] Wikipedia. *Families of multistep methods.* Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Linear_multistep_method#Families_of_multistep_methods.

[1108] Wikipedia. *Gauss-Hermite quadrature*. Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Gauss%E2%80%93Hermite_quadrature.

[1109] Wikipedia. *Gauss-Jacobi quadrature*. Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Gauss%E2%80%93Jacobi_quadrature.

[1110] Wikipedia. *Gauss-Kronrod quadrature formula*. Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Gauss%E2%80%93Kronrod_quadrature_formula.

[1111] Wikipedia. *Gauss-Laguerre quadrature*. Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Gauss%E2%80%93Laguerre_quadrature.

[1112] Wikipedia. *Gauss-Legendre quadrature*. Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Gauss%E2%80%93Legendre_quadrature.

[1113] Wikipedia. *Initial value problem.* Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Initial_value_problem.

[1114] Wikipedia. *Richardson extrapolation.* Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Richardson_extrapolation.

[1115] Wikipedia. *Runge-Kutta methods.* Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Runge%E2%80%93Kutta_methods.

[1116] Wikipedia. *Runge-Kutta-Fehlberg method.* Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Runge%E2%80%93Kutta%E2%80%93Fehlberg_method.

[1117] Wikipedia. *Series acceleration.* Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Series_acceleration.

[1118] Wikipedia. *Shanks transformation.* Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Shanks_transformation.

[1119] Wikipedia. *Tanh-sinh quadrature*. Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Tanh-sinh_quadrature.

[1120] Wikipedia. *Trapezoidal rule*. Wikimedia Foundation, 2005. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Trapezoidal_rule.

[1121] Wikipedia. *Characteristic function (probability theory)*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Characteristic_function_(probability_theory).

[1122] Wikipedia. *Cumulant*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Cumulant#Definition.

[1123] Wikipedia. *Cumulative distribution function*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Cumulative_distribution_function.

[1124] Wikipedia. *Excess kurtosis*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Kurtosis#Excess_kurtosis.

[1125] Wikipedia. *Expected value*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Expected_value.

[1126] Wikipedia. *Fat-tailed distribution*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Fat-tailed_distribution.

[1127] Wikipedia. *Hazard function and cumulative hazard function*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Survival_analysis#Hazard_function_and_cumulative_hazard_function.

[1128] Wikipedia. *Heavy-tailed distribution*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Heavy-tailed_distribution.

[1129] Wikipedia. *Inverse functions and differentiation*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Inverse_functions_and_differentiation.

[1130] Wikipedia. *Kurtosis*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Kurtosis.

[1131] Wikipedia. *Median*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Median.

[1132] Wikipedia. *Mode (statistics). Wikipedia*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Mode_(statistics).

[1133] Wikipedia. *Moment (mathematics)*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Saddlepoint_approximation_method.

[1134] Wikipedia. *Moment-generating function*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Moment-generating_function.

[1135] Wikipedia. *Probability density function*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Probability_density_function.

[1136] Wikipedia. *Probability distribution*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Probability_distribution.

[1137] Wikipedia. *Quantile function*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Quantile_function.

[1138] Wikipedia. *Saddlepoint approximation method*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Saddlepoint_approximation_method.

[1139] Wikipedia. *Skewness*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Skewness.

[1140] Wikipedia. *Standard deviation*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Standard_deviation.

[1141] Wikipedia. *Support (mathematics). Wikipedia*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Support_(mathematics).

[1142] Wikipedia. *Survival function*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Survival_function#Parametric_survival_functions.

[1143] Wikipedia. *Symmetric probability distribution*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Symmetric_probability_distribution.

[1144] Wikipedia. *Variance*. Wikimedia Foundation, 2006. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Variance.

[1145] Wikipedia. *Arcsine distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Arcsine_distribution.

[1146] Wikipedia. *Beta distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Beta_distribution.

[1147] Wikipedia. *Cauchy distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Cauchy_distribution.

[1148] Wikipedia. *Chi-squared distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Chi-squared_distribution.

[1149] Wikipedia. *Definition of entropy and differential entropy*. Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Maximum_entropy_probability_distribution#Definition_of_entropy_and_differential_entropy.

[1150] Wikipedia. *Exponential distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Exponential_distribution.

[1151] Wikipedia. *F-distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/F-distribution.

[1152] Wikipedia. *Gamma distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Gamma_distribution.

[1153] Wikipedia. *Gumbel distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Gumbel_distribution.

[1154] Wikipedia. *Hazard function and cumulative hazard function*. Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Likelihood_function.

[1155] Wikipedia. *Inverse Gaussian distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Inverse_Gaussian_distribution.

[1156] Wikipedia. *Inverse-gamma distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Inverse-gamma_distribution.

[1157] Wikipedia. *Laplace distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Laplace_distribution.

[1158] Wikipedia. *Log-normal distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Log-normal_distribution.

[1159] Wikipedia. *Logistic distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Logistic_distribution.

[1160] Wikipedia. *Mixture distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Mixture_distribution.

[1161] Wikipedia. *Mixture distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Convolution_of_probability_distributions.

[1162] Wikipedia. *Noncentral F-distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Noncentral_F-distribution.

[1163] Wikipedia. *Noncentral beta distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Noncentral_beta_distribution.

[1164] Wikipedia. *Noncentral chi-squared distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Noncentral_chi-squared_distribution.

[1165] Wikipedia. *Noncentral t-distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Noncentral_t-distribution.

[1166] Wikipedia. *Normal distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Normal_distribution.

[1167] Wikipedia. *Pareto distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Pareto_distribution.

[1168] Wikipedia. *Pearson correlation coefficient.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Pearson_correlation_coefficient#Using_the_exact_distribution.

[1169] Wikipedia. *Probability mass function.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Probability_mass_function.

[1170] Wikipedia. *Probability-generating function.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Probability-generating_function.

[1171] Wikipedia. *Student's t-distribution.* Wikimedia Foundation, 2007. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Student%27s_t-distribution.

[1172] Wikipedia. *Bernoulli distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Bernoulli_distribution.

[1173] Wikipedia. *Binomial distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Binomial_distribution.

[1174] Wikipedia. *Birnbaum-Saunders distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Birnbaum-Saunders_distribution.

[1175] Wikipedia. *Chi distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/chi_distribution.

[1176] Wikipedia. *Geometric distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Geometric_distribution.

[1177] Wikipedia. *Half-normal distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Half-normal_distribution.

[1178] Wikipedia. *Hyperexponential distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Hyperexponential_distribution.

[1179] Wikipedia. *Hypergeometric distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Hypergeometric_distribution.

[1180] Wikipedia. *Inverse-chi-squared distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Inverse-chi-squared_distribution.

[1181] Wikipedia. *Jonckheere's trend test.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Jonckheere%27s_trend_test.

[1182] Wikipedia. *Kendall rank correlation coefficient.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Kendall_rank_correlation_coefficient#Significance_tests.

[1183] Wikipedia. *Kumaraswamy distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Kumaraswamy_distribution.

[1184] Wikipedia. *Lévy distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/LÃľvy_distribution.

[1185] Wikipedia. *Mann–Whitney U test.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Mann-Whitney_U_test.

[1186] Wikipedia. *Maxwell distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Maxwell_distribution.

[1187] Wikipedia. *Nakagami distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Nakagami_distribution.

[1188] Wikipedia. *Negative binomial distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Negative_binomial_distribution.

[1189] Wikipedia. *Poisson distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Poisson_distribution.

[1190] Wikipedia. *Rayleigh distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Rayleigh_distribution.

[1191] Wikipedia. *Skew normal distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Skew_normal_distribution.

[1192] Wikipedia. *Triangular distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Triangular_distribution.

[1193] Wikipedia. *Uniform (continuous) distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Uniform_distribution_(continuous).

[1194] Wikipedia. *Weibull distribution.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Weibull_distribution.

[1195] Wikipedia. *Wilcoxon signed-rank test.* Wikimedia Foundation, 2008. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Wilcoxon_signed-rank_test.

[1196] Wikipedia. *Beta prime distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Beta_prime_distribution.

[1197] Wikipedia. *Burr distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Burr_distribution.

[1198] Wikipedia. *Dagum distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Dagum_distribution.

[1199] Wikipedia. *Erlang distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Erlang_distribution.

[1200] Wikipedia. *Exponential generalized beta distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Generalized_beta_distribution#Exponential_generalized_beta_distribution.

[1201] Wikipedia. *Exponentially Modified Gaussian distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Exponentially_modified_Gaussian_distribution.

[1202] Wikipedia. *Feller–Pareto distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Generalized_beta_distribution#Exponential_generalized_beta_distribution.

[1203] Wikipedia. *Folded normal distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Folded_normal_distribution.

[1204] Wikipedia. *Fréchet distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Fr%C3%A9chet_distribution.

[1205] Wikipedia. *GEV distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Generalized_extreme_value_distribution.

[1206] Wikipedia. *Generalized Pareto distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Generalized_Pareto_distribution.

[1207] Wikipedia. *Generalized Pareto distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Generalized_Pareto_distribution.

[1208] Wikipedia. *Generalized beta distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Generalized_beta_distribution.

[1209] Wikipedia. *Generalized gamma distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Generalized_gamma_distribution.

[1210] Wikipedia. *Generalized logistic distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Generalized_logistic_distribution.

[1211] Wikipedia. *Generalized normal distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Generalized_normal_distribution#Symmetric_version.

[1212] Wikipedia. *Hypoexponential Distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Hypoexponential_distribution.

[1213] Wikipedia. *Johnson SU and SB distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Johnson%27s_SU-distribution.

[1214] Wikipedia. *Log-logistic (Fisk) distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Log-logistic_distribution.

[1215] Wikipedia. *Lomax distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Lomax_distribution.

[1216] Wikipedia. *Noncentral chi distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Noncentral_chi_distribution.

[1217] Wikipedia. *Shifted Gompertz distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Shifted_Gompertz_distribution.

[1218] Wikipedia. *Studentized range distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Studentized_range_distribution.

[1219] Wikipedia. *Truncated normal distribution.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Truncated_normal_distribution.

[1220] Wikipedia. *Voigt profile.* Wikimedia Foundation, 2009. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Voigt_profile.

[1221] Wikipedia. *Fisher's noncentral hypergeometric distribution.* Wikimedia Foundation, 2010. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Fisher%27s_noncentral_hypergeometric_distribution.

[1222] Wikipedia. *Incomplete beta function.* Wikimedia Foundation, 2010. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Beta_function#Incomplete_beta_function.

[1223] Wikipedia. *Incomplete gamma function.* Wikimedia Foundation, 2010. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Incomplete_gamma_function.

[1224] Wikipedia. *Landau distribution.* Wikimedia Foundation, 2010. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Landau_distribution.

[1225] Wikipedia. *Logarithmic distribution.* Wikimedia Foundation, 2010. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Logarithmic_distribution.

[1226] Wikipedia. *Pearson correlation coefficient, exact distribution.* Wikimedia Foundation, 2010. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Pearson_correlation_coefficient#Using_the_exact_distribution.

[1227] Wikipedia. *Rice distribution.* Wikimedia Foundation, 2010. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Rice_distribution.

[1228] Wikipedia. *Stable distribution.* Wikimedia Foundation, 2010. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Stable_distribution.

[1229] Wikipedia. *Zeta distribution.* Wikimedia Foundation, 2010. [Online; accessed 22-July-2024]. URL: https://mathworld.wolfram.com/ZipfDistribution.html.

[1230] Wikipedia. *Common logarithm.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Common_logarithm.

[1231] Wikipedia. *Complementary error function.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Error_function#Complementary_error_function.

[1232] Wikipedia. *Complex logarithm.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Complex_logarithm.

[1233] Wikipedia. *Cube root.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Cube_root.

[1234] Wikipedia. *Error function.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Error_function.

[1235] Wikipedia. *Exponential function (expm1).* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Exponential_function#expm1.

[1236] Wikipedia. *Exponential function.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Exponential_function.

[1237] Wikipedia. *Exponentiation.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Exponentiation.

[1238] Wikipedia. *Hyperbolic functions.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Hyperbolic_function.

[1239] Wikipedia. *Imaginary error function.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Error_function#Imaginary_error_function.

[1240] Wikipedia. *Natural logarithm (log1p).* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Natural_logarithm#High_precision.

[1241] Wikipedia. *Natural logarithm.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Natural_logarithm.

[1242] Wikipedia. *Power of 10.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Power_of_10.

[1243] Wikipedia. *Power of two.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Power_of_two.

[1244] Wikipedia. *Sine.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Sine.

[1245] Wikipedia. *Square (algebra).* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Square_(algebra).

[1246] Wikipedia. *Square root.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Square_root.

[1247] Wikipedia. *Trigonometric functions.* Wikimedia Foundation, 2011. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Trigonometric_functions.

[1248] Wikipedia. *Bernoulli number.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Bernoulli_number.

[1249] Wikipedia. *Bernoulli polynomials.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Bernoulli_polynomials.

[1250] Wikipedia. *Bessel functions of the first kind.* Wikimedia Foundation, 2012. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Bessel_function#Bessel_functions_of_the_first_kind.

[1251] Wikipedia. *Bessel functions of the second kind*. Wikimedia Foundation, 2012. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Bessel_function#Bessel_functions_of_the_second_kind.

[1252] Wikipedia. *Beta function.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Beta_function.

[1253] Wikipedia. *Binomial coefficient.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Binomial_coefficient.

[1254] Wikipedia. *Confluent Hypergeometric Limit Function*. Wikimedia Foundation, 2012. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Generalized_hypergeometric_function#The_series_0F1.

[1255] Wikipedia. *Confluent hypergeometric function*. Wikimedia Foundation, 2012. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Confluent_hypergeometric_function.

[1256] Wikipedia. *Factorial.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Factorial.

[1257] Wikipedia. *Falling and rising factorials.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Falling_and_rising_factorials.

[1258] Wikipedia. *Gamma function.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Gamma_function.

[1259] Wikipedia. *Hypergeometric (Gauss) function*. Wikimedia Foundation, 2012. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Hypergeometric_function.

[1260] Wikipedia. *Inverse Trigonometric functions.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Inverse_trigonometric_functions.

[1261] Wikipedia. *Inverse hyperbolic functions.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Inverse_hyperbolic_functions.

[1262] Wikipedia. *Lambert W function.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Lambert_W_function.

[1263] Wikipedia. *Machine epsilon*. Wikimedia Foundation, 2012. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Machine_epsilon.

[1264] Wikipedia. *Modified Bessel functions*. Wikimedia Foundation, 2012. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Bessel_function#Modified_Bessel_functions.

[1265] Wikipedia. *Natural logarithm of 2*. Wikimedia Foundation, 2012. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Natural_logarithm_of_2.

[1266] Wikipedia. *Pi*. Wikimedia Foundation, 2012. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Pi.

[1267] Wikipedia. *Polygamma function.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Polygamma_function.

[1268] Wikipedia. *Reciprocal gamma function.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Reciprocal_gamma_function.

[1269] Wikipedia. *The log-gamma function.* Wikimedia Foundation, 2012. [Online; accessed 22-July-2024]. URL: https://en.wikipedia.org/wiki/Gamma_function#The_log-gamma_function.

[1270] Wikipedia. *e (mathematical constant)*. Wikimedia Foundation, 2012. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/E_(mathematical_constant).

[1271] Wikipedia. *Apéry's constant*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/ApÃľry%27s_constant.

[1272] Wikipedia. *Arithmetic-geometric mean*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Arithmetic-geometric_mean.

[1273] Wikipedia. *Atan2*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Atan2.

[1274] Wikipedia. *Conversion between radians and degrees*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Radian#Conversion_between_radians_and_degrees.

[1275] Wikipedia. *Digamma function*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Digamma_function.

[1276] Wikipedia. *Double factorial*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Double_factorial.

[1277] Wikipedia. *Euler number*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Euler_numbers.

[1278] Wikipedia. *Euler polynomials; in: Bernoulli polynomials*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Bernoulli_polynomials.

[1279] Wikipedia. *Euler-Mascheroni constant*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Euler-Mascheroni_constant.

[1280] Wikipedia. *Glaisher-Kinkelin constant*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Glaisher-Kinkelin_constant.

[1281] Wikipedia. *Golden ratio*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Golden_ratio.

[1282] Wikipedia. *Golden ratio*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Golden_ratio.

[1283] Wikipedia. *Group of nth roots of unity*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Root_of_unity#Group_of_nth_roots_of_unity.

[1284] Wikipedia. *Harmonic number*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Harmonic_number.

[1285] Wikipedia. *Hyperfactorial*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Factorial#Hyperfactorial.

[1286] Wikipedia. *Hypotenuse*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Hypotenuse.

[1287] Wikipedia. *Khinchin's constant*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Khinchin%27s_constant.

[1288] Wikipedia. *Prime number theorem*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Prime_number_theorem.

[1289] Wikipedia. *Prime-counting function*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Bernoulli_polynomials.

[1290] Wikipedia. *Prime-counting function*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Prime-counting_function.

[1291] Wikipedia. *Root of unity*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Root_of_unity#Group_of_nth_roots_of_unity.

[1292] Wikipedia. *Sinc function*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Sinc_function.

[1293] Wikipedia. *Superfactorial*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Factorial#Superfactorial.

[1294] Wikipedia. *Trigamma function*. Wikimedia Foundation, 2013. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Trigamma_function.

[1295] Wikipedia. *Airy function*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Airy_function.

[1296] Wikipedia. *Associated Legendre polynomials*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Associated_Legendre_polynomials.

[1297] Wikipedia. *Barnes G-function*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Barnes_G-function.

[1298] Wikipedia. *Bulirsch-Integrale*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://de.wikipedia.org/wiki/Elliptisches_Integral#Bulirsch-Integrale.

[1299] Wikipedia. *Carlson symmetric form*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Carlson_symmetric_form.

[1300] Wikipedia. *Chebyshev polynomials*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Chebyshev_polynomials.

[1301] Wikipedia. *Complete elliptic integral of the first kind*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Elliptic_integral#Complete_elliptic_integral_of_the_first_kind.

[1302] Wikipedia. *Complete elliptic integral of the second kind*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Elliptic_integral#Complete_elliptic_integral_of_the_second_kind.

[1303] Wikipedia. *Complete elliptic integral of the third kind*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Elliptic_integral#Complete_elliptic_integral_of_the_third_kind.

[1304] Wikipedia. *Elliptic function*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Elliptic_function.

[1305] Wikipedia. *Gegenbauer polynomials*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Gegenbauer_polynomials.

[1306] Wikipedia. *Hankel functions*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Bessel_function#Hankel_functions.

[1307] Wikipedia. *Hermite polynomials*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Hermite_polynomials.

[1308] Wikipedia. *Incomplete elliptic integral of the first kind*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Elliptic_integral#Incomplete_elliptic_integral_of_the_first_kind.

[1309] Wikipedia. *Incomplete elliptic integral of the second kind*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Elliptic_integral#Incomplete_elliptic_integral_of_the_second_kind.

[1310] Wikipedia. *Incomplete elliptic integral of the third kind*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Elliptic_integral#Incomplete_elliptic_integral_of_the_third_kind.

[1311] Wikipedia. *Jacobi elliptic functions*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Jacobi_elliptic_functions.

[1312] Wikipedia. *Jacobi polynomials*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Jacobi_polynomials.

[1313] Wikipedia. *Laguerre polynomials*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Laguerre_polynomials.

[1314] Wikipedia. *Legendre polynomials*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Legendre_polynomials.

[1315] Wikipedia. *Spherical Bessel functions*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Bessel_function#Spherical_Bessel_functions:_jn,_yn.

[1316] Wikipedia. *Spherical Hankel functions*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Bessel_function#Spherical_Hankel_functions:_h(1)n,_h(2)n.

[1317] Wikipedia. *Spherical harmonics*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Spherical_harmonics.

[1318] Wikipedia. *Theta function*. Wikimedia Foundation, 2014. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Theta_function.

[1319] Wikipedia. *Complete Fermi-Dirac integral*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Complete_Fermi%E2%80%93Dirac_integral.

[1320] Wikipedia. *Cosine integral*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Trigonometric_integral#Cosine_integral.

[1321] Wikipedia. *Exponential integral*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Exponential_integral.

[1322] Wikipedia. *Faddeeva function*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Faddeeva_function.

[1323] Wikipedia. *Fresnel integral*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Fresnel_integral.

[1324] Wikipedia. *Goodwin-Staton integral*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Goodwin-Staton_integral.

[1325] Wikipedia. *Hurwitz zeta function*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Hurwitz_zeta_function.

[1326] Wikipedia. *Hyperbolic cosine integral*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Trigonometric_integral#Hyperbolic_cosine_integral.

[1327] Wikipedia. *Hyperbolic sine integral*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Trigonometric_integral#Hyperbolic_sine_integral.

[1328] Wikipedia. *Legendre chi function*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Legendre_chi_function.

[1329] Wikipedia. *Lemniscate elliptic functions*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Lemniscate_elliptic_functions.

[1330] Wikipedia. *Logarithmic integral function*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Logarithmic_integral_function.

[1331] Wikipedia. *Neville theta functions*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Neville_theta_functions.

[1332] Wikipedia. *Polylogarithm*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Polylogarithm.

[1333] Wikipedia. *Polylogarithm, inverse tangent integral*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Polylogarithm#Relationship_to_other_functions.

[1334] Wikipedia. *Riemann zeta function*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Riemann_zeta_function.

[1335] Wikipedia. *Sine integral*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Trigonometric_integral#Sine_integral.

[1336] Wikipedia. *Spence's function*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Spence%27s_function.

[1337] Wikipedia. *Voigt profile*. Wikimedia Foundation, 2015. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Voigt_profile.

[1338] Wikipedia. *Catalan number*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Catalan_number.

[1339] Wikipedia. *Dawson function*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Dawson_function.

[1340] Wikipedia. *Debye functions and polylogarithms*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Debye_functionandhttps://en.wikipedia.org/wiki/Polylogarithm.

[1341] Wikipedia. *Dirichlet beta function*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: http://en.wikipedia.org/wiki/Dirichlet_beta_function.

[1342] Wikipedia. *Fibonacci Number*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: http://mathworld.wolfram.com/FibonacciNumber.html.

[1343] Wikipedia. *Fibonacci Polynomials*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Fibonacci_polynomials.

[1344] Wikipedia. *Gudermannian function*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Gudermannian_function.

[1345] Wikipedia. *Haversine*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Versine#Haversineandhttps://en.wikipedia.org/wiki/Haversine_formula.

[1346] Wikipedia. *Inverse Gudermannian function*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Gudermannian_function#Inverse.

[1347] Wikipedia. *Inverse Haversine*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Versine#Inverse_functions.

[1348] Wikipedia. *Logistic function*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Logistic_function.

[1349] Wikipedia. *Logit function*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Logit.

[1350] Wikipedia. *Lucas Number*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Lucas_number.

[1351] Wikipedia. *Lucas Polynomials*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Lucas_sequenceandhttps://en.wikipedia.org/wiki/Fibonacci_polynomials.

[1352] Wikipedia. *Marcum Q-function*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Owen%27s_T_function.

[1353] Wikipedia. *Owen's T function*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Owen%27s_T_function.

[1354] Wikipedia. *Riemann Xi*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Riemann_Xi_function.

[1355] Wikipedia. *Weierstrass elliptic function: Modular discriminant*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Weierstrass_elliptic_function#Modular_discriminant.

[1356] Wikipedia. *Weierstrass elliptic function: The constants e1, e2 and e3*. Wikimedia Foundation, 2016. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Weierstrass_elliptic_function#The_constants_e1,_e2_and_e3.

[1357] Wikipedia. *Clausen function*. Wikimedia Foundation, 2017. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Clausen_function.

[1358] Wikipedia. *Half-period ratio*. Wikimedia Foundation, 2017. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Half-period_ratio.

[1359] Wikipedia. *Lerch's transcendent (Lerch zeta function)*. Wikimedia Foundation, 2017. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Lerch_zeta_function.

[1360] Wikipedia. *Nome (mathematics)*. Wikimedia Foundation, 2017. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Nome_(mathematics).

[1361] Wikipedia. *Riemann hypothesis*. Wikimedia Foundation, 2017. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Riemann_hypothesis.

[1362] Wikipedia. *Riemann-Siegel Z function*. Wikimedia Foundation, 2017. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Z_function.

[1363] Wikipedia. *Riemann-Siegel theta function*. Wikimedia Foundation, 2017. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Riemann-Siegel_theta_function.

[1364] Wikipedia. *j-invariant*. Wikimedia Foundation, 2017. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/J-invariant.

[1365] Wikipedia. *Anger function*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Anger_function.

[1366] Wikipedia. *Circle*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Circle.

[1367] Wikipedia. *Correlation matrix*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Covariance_matrix#Correlation_matrix.

[1368] Wikipedia. *Coulomb wave function*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Coulomb_wave_function.

[1369] Wikipedia. *Covariance matrix*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Covariance_matrix.

[1370] Wikipedia. *Cycloid*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Cycloid.

[1371] Wikipedia. *Ellipse*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Ellipse.

[1372] Wikipedia. *Epicycloid*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Epicycloid.

[1373] Wikipedia. *Epitrochoid*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Epitrochoid.

[1374] Wikipedia. *Euler's formula*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Euler%27s_formula.

[1375] Wikipedia. *Fish curve*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Fish_curve.

[1376] Wikipedia. *Generalized hypergeometric function*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Generalized_hypergeometric_function.

[1377] Wikipedia. *Heart curve*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Heart_symbol#Parametrisation.

[1378] Wikipedia. *Hypocycloid*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Hypocycloid.

[1379] Wikipedia. *Hypotrochoid*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Hypotrochoid.

[1380] Wikipedia. *Kelvin function bei*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Kelvin_functions#bei(x).

[1381] Wikipedia. *Kelvin function ber*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Kelvin_functions#ber(x).

[1382] Wikipedia. *Kelvin function kei*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Kelvin_functions#kei(x).

[1383] Wikipedia. *Kelvin function ker*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Kelvin_functions#ker(x).

[1384] Wikipedia. *Lemniscate of Bernoulli*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Lemniscate_of_Bernoulli.

[1385] Wikipedia. *Lemniscate of Gerono*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Lemniscate_of_Gerono.

[1386] Wikipedia. *Lissajous curve*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Lissajous_curve.

[1387] Wikipedia. *Lommel function*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Lommel_function.

[1388] Wikipedia. *Parabolic cylinder function*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Parabolic_cylinder_function.

[1389] Wikipedia. *Sample mean and covariance*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Sample_mean_and_covariance#Sample_covariance.

[1390] Wikipedia. *Scorer's function*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Scorer%27s_function.

[1391] Wikipedia. *Struve function*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Struve_function.

[1392] Wikipedia. *Superellipse (Lamé curve)*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Superellipse.

[1393] Wikipedia. *Trace (linear algebra)*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Determinant.

[1394] Wikipedia. *Trochoid*. Wikimedia Foundation, 2018. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Trochoid.

[1395] Wikipedia. *Whittaker function*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Whittaker_function.

[1396] Wikipedia. *q-Pochhammer symbol*. Wikimedia Foundation, 2018. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Q-Pochhammer_symbol.

[1397] Wikipedia. *Archimedes Spiral*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Archimedean_spiral.

[1398] Wikipedia. *Butterfly curve*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Butterfly_curve_(transcendental).

[1399] Wikipedia. *Cardioid*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Cardioid.

[1400] Wikipedia. *Cholesky decomposition*. Wikimedia Foundation, 2019. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Cholesky_decomposition.

[1401] Wikipedia. *Conchoid of de Sluze*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Conchoid_of_de_Sluze.

[1402] Wikipedia. *Cycloid of Ceva*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://de.wikipedia.org/wiki/Zykloide_von_Ceva.

[1403] Wikipedia. *Eigendecomposition of a matrix*. Wikimedia Foundation, 2019. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Eigendecomposition_of_a_matrix#Eigendecomposition_of_a_matrix.

[1404] Wikipedia. *Eigendecomposition of a matrix: Generalized_eigenvalue_problem*. Wikimedia Foundation, 2019. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Eigendecomposition_of_a_matrix#Generalized_eigenvalue_problem.

[1405] Wikipedia. *Eigendecomposition of a matrix: Useful facts*. Wikimedia Foundation, 2019. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Eigendecomposition_of_a_matrix#Useful_facts.

[1406] Wikipedia. *Epispiral*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Epispiral.

[1407] Wikipedia. *Fermat's Spiral*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Fermat%27s_spiral.

[1408] Wikipedia. *Freeth's Nephroid*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Nephroid.

[1409] Wikipedia. *Hessenberg matrix*. Wikimedia Foundation, 2019. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Hessenberg_matrix.

[1410] Wikipedia. *Householder transformation: Tridiagonalization*. Wikimedia Foundation, 2019. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Householder_transformation#Tridiagonalization.

[1411] Wikipedia. *Hyperbolic Spiral*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Hyperbolic_spiral.

[1412] Wikipedia. *LU factorization with partial pivoting*. Wikimedia Foundation, 2019. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/LU_decomposition#LU_factorization_with_partial_pivoting.

[1413] Wikipedia. *Limaçon curve*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Lima%C3%A7on.

[1414] Wikipedia. *Lituus*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Lituus_(mathematics).

[1415] Wikipedia. *Logarithmic Spiral*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://mathworld.wolfram.com/LogarithmicSpiral.html.

[1416] Wikipedia. *Moore-Penrose inverse*. Wikimedia Foundation, 2019. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Moore%E2%80%93Penrose_inverse.

[1417] Wikipedia. *Rose curve*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Maurer_rose.

[1418] Wikipedia. *Singular value decomposition*. Wikimedia Foundation, 2019. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Singular_value_decomposition.

[1419] Wikipedia. *Strophoid*. Wikimedia Foundation, 2019. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Strophoid.

[1420] Wikipedia. *Tridiagonal matrix*. Wikimedia Foundation, 2019. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Tridiagonal_matrix.

[1421] Wikipedia. *Analytic function of a matrix*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Analytic_function_of_a_matrix.

[1422] Wikipedia. *Broyden-Fletcher-Goldfarb-Shanno algorithm*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Broyden%E2%80%93Fletcher%E2%80%93Goldfarb%E2%80%93Shanno_algorithm.

[1423] Wikipedia. *CMA-ES*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/CMA-ES.

[1424] Wikipedia. *Generalized Schur decomposition*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Schur_decomposition#Generalized_Schur_decomposition.

[1425] Wikipedia. *LDL decomposition*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Cholesky_decomposition#LDL_decomposition_2.

[1426] Wikipedia. *LU factorization with full pivoting*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/LU_decomposition#LU_factorization_with_full_pivoting.

[1427] Wikipedia. *Levenberg-Marquardt algorithm*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Levenberg%E2%80%93Marquardt_algorithm.

[1428] Wikipedia. *Limited-memory BFGS*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Limited-memory_BFGS.

[1429] Wikipedia. *Logarithm of a matrix*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Logarithm_of_a_matrix.

[1430] Wikipedia. *Matrix decomposition*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Matrix_decomposition.

[1431] Wikipedia. *Matrix exponential*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Matrix_exponential.

[1432] Wikipedia. *Nelder-Mead method*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Nelder-Mead_method.

[1433] Wikipedia. *Powell's dog leg method*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Powell%27s_dog_leg_method.

[1434] Wikipedia. *QR decomposition*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/QR_decomposition.

[1435] Wikipedia. *QR decomposition*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Eigendecomposition_of_a_matrix#Generalized_eigenvalue_problem.

[1436] Wikipedia. *QR decomposition: Column pivoting*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/QR_decomposition#Column_pivoting.

[1437] Wikipedia. *Schur decomposition*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Schur_decomposition.

[1438] Wikipedia. *Square root of a matrix*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Square_root_of_a_matrix.

[1439] Wikipedia. *Trigonometric functions of matrices*. Wikimedia Foundation, 2020. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Trigonometric_functions_of_matrices.

[1440] Wikipedia. *One-sample t-test*. Wikimedia Foundation, 2021. [Online; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Student%27s_t-test#One-sample_t-test.

[1441] Wikipedia. *Acnode*. Wikimedia Foundation, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Acnode.

[1442] Wikipedia. *Cissoid*. Wikimedia Foundation, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Cissoid.

[1443] Wikipedia. *Cotes's spiral*. Wikimedia Foundation, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Cotes%27s_spiral.

[1444] Wikipedia. *Crunode*. Wikimedia Foundation, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Crunode.

[1445] Wikipedia. *Cusp (singularity)*. Wikimedia Foundation, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Cusp_(singularity).

[1446] Wikipedia. *Euler spiral*. Wikimedia Foundation, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Fresnel_integral#Euler_spiral.

[1447] Wikipedia. *Nielsen's_spiral*. Wikimedia Foundation, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Trigonometric_integral#Nielsen's_spiral.

[1448] Wikipedia. *Poinsot's spirals*. Wikimedia Foundation, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Poinsot%27s_spirals.

[1449] Wikipedia. *Singular point of a curve*. Wikimedia Foundation, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Singular_point_of_a_curve.

[1450] Wikipedia. *Trisectrix of Maclaurin*. Wikimedia Foundation, 2022. [Online; last edited on 5 December 2022; accessed 22-Dec-2024]. URL: https://en.wikipedia.org/wiki/Trisectrix_of_Maclaurin.

[1451] S. S. Wilks. Certain generalizations in the analysis of variance. *Biometrika*, 24:471–494, 1932.

[1452] S. S. Wilks. On the independence of k sets of normally distributed statistical variables. *Econometrica*, 3:309–326, 1935.

[1453] S. S. Wilks. Sample criteria for testing equality ofmeans, equality of variances, and equality of covariances in a normal multivariate distribution. *Annals of Mathematical Statistics*, 17:257–281, 1946.

[1454] A. Winterbottom. Cornish-fisher expansions for confidence limits. *Communications in Statistics, Part B – Simulation and Computation*, 41:69–79, 1979.

[1455] A. Winterbottom. Estimation for the bivariate normal correlation coefficient using asymptotic expansions. *Communications in Statistics, Part B – Simulation and Computation [Split from: @J(CommStat)]*, 9:599–609, 1980.

[1456] C. S. Withers and S. Nadarajah. Charlier and edgeworth expansions for distributions and densities in terms of bell polynomials. *Probability and Mathematical Statistics*, 29:271, 2009.

[1457] C. S. Withers and S. Nadarajah. Edgeworth–cornish–fisher–hill–davis expansions for normal and non-normal limits via bell polynomials. *Stochastics: An International Journal of Probability and Stochastic Processes*, 87:794–805, 2015.

[1458] V. Witkovsky. The characteristic functions toolbox. In *The 10th International Conference of the ERCIM Workgroup on Computational and Methodological Statistics (CMStatistics 2017)*. University of London, UK, 2017. Available from https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/.

[1459] V. Witkovský. Computing the distribution of a linear combination of inverted gamma variables. *Kybernetika*, 37(1):79–90, 2001.

[1460] V. Witkovský. On the exact computation of the density and of the quantiles of linear combinations of t and f random variables. *Journal of Statistical Planning and Inference*, 94:1–13, 2001.

[1461] V. Witkovský. A note on Computing Extreme Tail Probabilities of the Noncentral T Distribution with Large Noncentrality Parameter. *ArXiv e-prints*, 2013.

[1462] V. Witkovský. *Arcsine Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfS_Arcsine.m.

[1463] V. Witkovský. *Beta Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_Student.m.

[1464] V. Witkovský. *Chi-Squared Dsitribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_ChiSquare.m.

[1465] V. Witkovský. *Exponential Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfX_Exponential.m.

[1466] V. Witkovský. *F-Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_FisherSnedecor.m.

[1467] V. Witkovský. *Gamma Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfX_Gamma.m.

[1468] V. Witkovský. *Gumbel Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_Gumbel.m.

[1469] V. Witkovský. *Inverse Gamma Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfX_InverseGamma.m.

[1470] V. Witkovský. *Inverse Gaussian Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_InverseGaussian.m.

[1471] V. Witkovský. *Laplace Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_Laplace.m.

[1472] V. Witkovský. *Log Normal Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfX_LogNormal.m.

[1473] V. Witkovský. *Noncentral Beta Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_BetaNC.m.

[1474] V. Witkovský. *Noncentral Chi-Squared Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_ChiSquare.m.

[1475] V. Witkovský. *Noncentral F-Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_FisherSnedecorNC.m.

[1476] V. Witkovský. *Pareto Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfX_Pareto.m.

[1477] V. Witkovský. *Rayleigh Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_Rayleigh.m.

[1478] V. Witkovský. *Rectangular (Uniform) Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfS_Rectangular.m.

[1479] V. Witkovský. *Student's t-Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_Student.m.

[1480] V. Witkovský. *Triangular Distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfS_Triangular.m.

[1481] V. Witkovský. *Weibull distribution*. CharFunTool: The characteristic functions toolbox, 2017. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfX_Weibull.m.

[1482] V. Witkovský. *Binomial distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfN_Binomial.m.

[1483] V. Witkovský. *Canonical Correlation: Noncentral Wilks' Lambda Distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_LogRV_WilksLambdaNC.m.

[1484] V. Witkovský. *Chi Distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_Chi.m.

[1485] V. Witkovský. *Geometric distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfN_Geometric.m.

[1486] V. Witkovský. *Half-normal distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_HalfNormal.m.

[1487] V. Witkovský. *Maxwell Distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_MaxwellBoltzmann.m.

[1488] V. Witkovský. *Nakagami Distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_Nakagami.m.

[1489] V. Witkovský. *Negative Binomial distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfN_NegativeBinomial.m.

[1490] V. Witkovský. *Noncentral Wilks' Lambda Distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_LogRV_WilksLambdaNC.m.

[1491] V. Witkovský. *Noncentral chi distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_ChiNC.m.

[1492] V. Witkovský. *Poisson distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfN_Poisson.m.

[1493] V. Witkovský. *Rice distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_Rice.m.

[1494] V. Witkovský. *SkewNormal Distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_SkewNormal.m.

[1495] V. Witkovský. *Test for Equality of Means*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cfTest_EqualityMeans.m.

[1496] V. Witkovský. *Wilks' Lambda Distribution*. CharFunTool: The characteristic functions toolbox, 2018. [Online; accessed 22-July-2024]. URL: https://github.com/witkovsky/CharFunTool/blob/master/CF_Repository/cf_LogRV_WilksLambda.m.

[1497] Y. Yin. *Multiple Hypothesis Testing and Multiple Outlier Identification Methods*. Thesis. University of Saskatchewan, 2010. Available as https://harvest.usask.ca/server/api/core/bitstreams/30ee94cc-80ae-4939-8362-53b746a319a7/content.

[1498] Y. Yu. The shape of noncentral chi-square density. 2011. Available as https://arxiv.org/pdf/1106.5241.

[1499] H. Zakerzadeh and A. Dolati. Generalized lindley distribution. *Journal of Mathematical Extension*, 3(2):13–25, 2009.

[1500] Z. Zhang. An improvement to the brent's method. *IJEA*, 2:2–26, 2011. Available at: https://www.cscjournals.org/manuscript/Journals/IJEA/Volume2/Issue1/IJEA-7.pdf.

[1501] H. Zimmermann. Exact calculation of permutational distributions for two dependent samples. *Biometrical Journal*, 27:349–352, 1985.

[1502] H. Zimmermann. Exact calculation of permutational distributions for two independent samples. *Biometrical Journal*, 27:431–434, 1985.

[1503] Kaveh R. Ghazi and Vincent Lefèvre and Philippe Théveny and Paul Zimmermann. Why and how to use arbitrary precision. *Computing in Science and Engineering*, 12(3):62–65, 2010. Preprint available at http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.212.6321&rep=rep1&type=pdf.

[1504] W. Hofschuster and W. Krämer. C-xsc 2.0: a c++ library for extended scientific computing. In R. Alt, A. Frommer, R.B. Kearfott, and W. Luther, editors, *Numerical Software with Result Verification, Lecture notes in Computer Science*, pages 15–35. Springer-Verlag, Heidelberg, 2004.

INDEX

A

abs() (*ctx method*), 229
acos() (*ctx method*), 282
acos1m() (*math53 method*), 2158
acosd() (*math53 method*), 2156
acosh() (*ctx method*), 292
acosh1p() (*math53 method*), 2161
acot() (*ctx method*), 286
acotc() (*ctx method*), 2233
acotd() (*math53 method*), 2158
acoth() (*ctx method*), 295
acothc() (*ctx method*), 2233
acsc() (*ctx method*), 287
acsch() (*ctx method*), 297
AdamsBashforthMoultonConst() (*ctxboost method*), 881
add() (*npm method*), 1164
agm() (*ctx method*), 314
airy_ai() (*ctx method*), 1373
airy_ai_prime() (*ctx method*), 1377
airy_ai_zero() (*ctx method*), 1379
airy_bi() (*ctx method*), 1375
airy_bi_prime() (*ctx method*), 1378
airy_bi_zero() (*ctx method*), 1380
all() (*npm method*), 1180
anger_j() (*ctx method*), 1483
any() (*npm method*), 1180
apery (*ctx property*), 227
append() (*npm method*), 1141
arange() (*npm method*), 1104
arccl() (*math53 method*), 2188
arcgd() (*math53 method*), 2162
archav() (*math53 method*), 2158
arcsine_cdf() (*math53 method*), 336
arcsine_pdf() (*math53 method*), 336
arcsine_qtf() (*math53 method*), 336
arcsl() (*math53 method*), 2187
argmax() (*npm method*), 1151
argmin() (*npm method*), 1151
argsort() (*npm method*), 1149
argwhere() (*npm method*), 1152
array() (*npm method*), 1094
array_equal() (*npm method*), 1182
array_equiv() (*npm method*), 1183
array_split() (*npm method*), 1135
asanyarray() (*npm method*), 1096
asarray() (*npm method*), 1095
asec() (*ctx method*), 289
asech() (*ctx method*), 296
asin() (*ctx method*), 281
asind() (*math53 method*), 2156
asinh() (*ctx method*), 291
atan() (*ctx method*), 284
atan2() (*ctx method*), 285
atand() (*math53 method*), 2157
atanh() (*ctx method*), 293
atleast_1d() (*npm method*), 1126
atleast_2d() (*npm method*), 1126
atleast_3d() (*npm method*), 1126
average() (*npm method*), 1168

B

barnes_g() (*ctx method*), 1322
bateman_g() (*math53 method*), 2192
bernoulli() (*ctx method*), 1318
bernoulli_cdf() (*math53 method*), 746
bernoulli_pmf() (*math53 method*), 746
bernoulli_qtf() (*math53 method*), 746
bernpoly() (*ctx method*), 1319
bessel_i0() (*math53 method*), 2205
bessel_i0_int() (*math53 method*), 2208
bessel_i0e() (*math53 method*), 2206
bessel_i1() (*math53 method*), 2205
bessel_i1e() (*math53 method*), 2207
bessel_iv() (*ctx method*), 1349
bessel_iv_prime() (*ctx method*), 1353
bessel_j0() (*math53 method*), 2202
bessel_j0_int() (*math53 method*), 2208
bessel_j1() (*math53 method*), 2202
bessel_jn() (*math53 method*), 2202
bessel_jv() (*ctx method*), 1341
bessel_jv_prime() (*ctx method*), 1345
bessel_jv_zero() (*ctx method*), 1347
bessel_k0() (*math53 method*), 2205
bessel_k0_int() (*math53 method*), 2208

bessel_k0e() (*math53 method*), 2207
bessel_k1() (*math53 method*), 2206
bessel_k1e() (*math53 method*), 2207
bessel_kn() (*math53 method*), 2206
bessel_kv() (*ctx method*), 1351
bessel_kv_prime() (*ctx method*), 1354
bessel_lambda() (*math53 method*), 2168
bessel_y0() (*math53 method*), 2203
bessel_y0_int() (*math53 method*), 2208
bessel_y1() (*math53 method*), 2203
bessel_yn() (*math53 method*), 2203
bessel_yv() (*ctx method*), 1343
bessel_yv_prime() (*ctx method*), 1346
bessel_yv_zero() (*ctx method*), 1348
besselpoly() (*ctx method*), 1367
besseltheta() (*ctx method*), 1368
beta() (*ctx method*), 310
beta_cdf() (*math53 method*), 516
beta_davis_qtf() (*ctx method*), 1546
beta_lower() (*ctx method*), 1460
beta_nc_benton_cdf_sf() (*ctx method*), 1517
beta_nc_cdf() (*math53 method*), 575
beta_nc_pdf() (*math53 method*), 575
beta_nc_qtf() (*math53 method*), 575
beta_nc_seber_cdf() (*ctx method*), 1508
beta_nc_spa() (*ctx method*), 593
beta_nc_spa_inv() (*ctx method*), 593
beta_pdf() (*math53 method*), 516
beta_peizer_cdf_sf_pdf() (*ctx method*), 1514
beta_prod_spa() (*ctx method*), 658
beta_prod_spa_inv() (*ctx method*), 658
beta_product_bd() (*ctx method*), 658
beta_product_bd_cdf_old() (*ctx method*), 658
beta_product_bd_inv() (*ctx method*), 658
beta_product_ecf() (*ctx method*), 654
beta_product_ecf_inv() (*ctx method*), 654
beta_product_mu3_cdf() (*ctx method*), 1569
beta_product_mu3_qtf() (*ctx method*), 1569
beta_qtf() (*math53 method*), 516
beta_recurrence() (*ctx method*), 519
beta_seber_cdf() (*ctx method*), 1506
beta_upper() (*ctx method*), 1464
BFGS() (*ctxboost method*), 936
bfgs_c() (*ctx method*), 839
binomial() (*ctx method*), 311
binomial_cdf() (*math53 method*), 760
binomial_ecf() (*ctx method*), 764
binomial_ecf_inv() (*ctx method*), 764
binomial_fm_pmf() (*ctx method*), 1522
binomial_ft() (*ctx method*), 764
binomial_pmf() (*math53 method*), 760
binomial_pmf_vector() (*ctx method*), 1535
binomial_qtf() (*math53 method*), 760
binomial_spa() (*ctx method*), 764
binomial_spa_inv() (*ctx method*), 764
bisection() (*ctx method*), 832
block() (*npm method*), 1131
bose_einstein() (*ctx method*), 1310
box_cov_bd() (*ctx method*), 692
box_cov_bd_inv() (*ctx method*), 692
box_davis_ecf() (*ctx method*), 692
box_davis_ecf_inv() (*ctx method*), 692
box_davis_expansion() (*ctx method*), 901
box_davis_expansion_inv() (*ctx method*), 902
box_means_cov_bd() (*ctx method*), 697
box_means_cov_bd_inv() (*ctx method*), 697
box_spa() (*ctx method*), 687
box_spa_inv() (*ctx method*), 687
BracketRoot() (*ctxboost method*), 827
brent() (*ctx method*), 836
BrentMinimum() (*ctx method*), 831
bring() (*math53 method*), 2150
broadcast_arrays() (*npm method*), 1127
broadcast_to() (*npm method*), 1127
BulirschStoerAdaptive() (*ctxboost method*), 885
BulirschStoerDenseOutput() (*ctxboost method*), 887

C

c_he_tridiag_0(), 1232
c_x() (*dist_amoroso method*), 498
c_x() (*dist_arcsine method*), 338
c_x() (*dist_bennett method*), 781
c_x() (*dist_beta method*), 518
c_x() (*dist_beta_nc_type_I method*), 577
c_x() (*dist_beta_prime method*), 534
c_x() (*dist_beta_product method*), 656
c_x() (*dist_binomial method*), 762
c_x() (*dist_birnb_saunders method*), 438
c_x() (*dist_box_cov method*), 689
c_x() (*dist_box_mvn method*), 694
c_x() (*dist_box_nsame_cov method*), 685
c_x() (*dist_cauchy method*), 342
c_x() (*dist_chi method*), 470
c_x() (*dist_chi_nc method*), 581
c_x() (*dist_chi_squared method*), 474
c_x() (*dist_chi_squared_nc method*), 563
c_x() (*dist_dagum method*), 388
c_x() (*dist_dunnett1_t method*), 634
c_x() (*dist_dunnett2_t method*), 637
c_x() (*dist_emg method*), 442
c_x() (*dist_exponential method*), 345
c_x() (*dist_feller_pareto method*), 549
c_x() (*dist_fisher_f method*), 523
c_x() (*dist_fisher_f_2nc method*), 607
c_x() (*dist_fisher_f_nc method*), 572
c_x() (*dist_fisher_r2_type_I method*), 595
c_x() (*dist_fisher_z method*), 552
c_x() (*dist_fisk method*), 391

c_x() (*dist_folded_normal method*), 445
c_x() (*dist_frechet method*), 394
c_x() (*dist_friedman method*), 811
c_x() (*dist_gamma method*), 479
c_x() (*dist_gen_beta_exp method*), 546
c_x() (*dist_genbeta1 method*), 537
c_x() (*dist_genbeta2 method*), 540
c_x() (*dist_genlogistic method*), 543
c_x() (*dist_genpareto method*), 400
c_x() (*dist_geometric method*), 752
c_x() (*dist_gev method*), 398
c_x() (*dist_gompertz method*), 403
c_x() (*dist_gumbel method*), 349
c_x() (*dist_half_normal method*), 448
c_x() (*dist_halperin_t method*), 643
c_x() (*dist_hotelling_t2 method*), 713
c_x() (*dist_hyperexponential method*), 353
c_x() (*dist_hypergeometric method*), 772
c_x() (*dist_hypoexponential method*), 504
c_x() (*dist_invchisquared method*), 483
c_x() (*dist_invgamma method*), 487
c_x() (*dist_johnson_sb method*), 451
c_x() (*dist_johnson_su method*), 454
c_x() (*dist_jterpsta_s method*), 801
c_x() (*dist_kendall_tau method*), 796
c_x() (*dist_kruskal_wallis method*), 814
c_x() (*dist_kumaraswamy method*), 357
c_x() (*dist_landau method*), 730
c_x() (*dist_laplace method*), 361
c_x() (*dist_levy method*), 417
c_x() (*dist_lindley method*), 508
c_x() (*dist_logistic method*), 365
c_x() (*dist_lognormal method*), 420
c_x() (*dist_logrv_beta method*), 531
c_x() (*dist_logrv_beta_sum method*), 652
c_x() (*dist_logrv_chisquared method*), 501
c_x() (*dist_lomax method*), 406
c_x() (*dist_lrt_s0 method*), 699
c_x() (*dist_lrt_x0_s0 method*), 703
c_x() (*dist_mann_whitney_u method*), 785
c_x() (*dist_mann_whitney_u_lehmann method*), 789
c_x() (*dist_mann_whitney_u_milton method*), 792
c_x() (*dist_mauchley method*), 681
c_x() (*dist_maxwell method*), 491
c_x() (*dist_moyal method*), 424
c_x() (*dist_nair_t method*), 640
c_x() (*dist_nakagami method*), 495
c_x() (*dist_negbinom method*), 767
c_x() (*dist_nelson_h method*), 646
c_x() (*dist_nmax_0 method*), 457
c_x() (*dist_nmax_rho method*), 613
c_x() (*dist_nmax_rhoij method*), 619
c_x() (*dist_nmm_0 method*), 460
c_x() (*dist_nmm_rho method*), 616
c_x() (*dist_nmm_rhoij method*), 622
c_x() (*dist_normal method*), 428
c_x() (*dist_normal_range method*), 625
c_x() (*dist_page_l method*), 805
c_x() (*dist_page_l_nc_milton method*), 808
c_x() (*dist_pareto method*), 369
c_x() (*dist_pearson_rho method*), 559
c_x() (*dist_pearson_rho_nc method*), 587
c_x() (*dist_pillai_v method*), 710
c_x() (*dist_poisson method*), 757
c_x() (*dist_rayleigh method*), 373
c_x() (*dist_rice method*), 584
c_x() (*dist_roy method*), 707
c_x() (*dist_sasnormal method*), 463
c_x() (*dist_shifted_gompertz method*), 409
c_x() (*dist_singh_maddala method*), 412
c_x() (*dist_skewnormal method*), 432
c_x() (*dist_skewt method*), 556
c_x() (*dist_smax method*), 628
c_x() (*dist_smm method*), 631
c_x() (*dist_stacy method*), 514
c_x() (*dist_student_t method*), 528
c_x() (*dist_student_t_2nc method*), 604
c_x() (*dist_student_t_nc method*), 568
c_x() (*dist_studentized_range method*), 649
c_x() (*dist_triangular method*), 377
c_x() (*dist_truncated_normal method*), 466
c_x() (*dist_uniform method*), 381
c_x() (*dist_wald method*), 436
c_x() (*dist_weibull method*), 385
c_x() (*dist_wilcoxon method*), 777
c_x() (*dist_wilks_cs method*), 665
c_x() (*dist_wilks_iblocks method*), 677
c_x() (*dist_wilks_ip method*), 673
c_x() (*dist_wilks_lambda method*), 661
c_x() (*dist_wilks_lambda_corr method*), 720
c_x() (*dist_wilks_lambda_glm method*), 716
c_x() (*dist_wilks_lcm method*), 669
c_x() (*rv_base method*), 327
c_x() (*skewexponpower method*), 511
c_y() (*dist_logrv_beta_nc_type_II method*), 591
c_y() (*dist_logrv_fisher_1mr2 method*), 600
CashKarpAdaptive() (*ctxboost method*), 882
CashKarpConst() (*ctxboost method*), 878
catalan (*ctx property*), 227
catalan_c() (*math53 method*), 2167
cauchy_cdf() (*math53 method*), 340
cauchy_pdf() (*math53 method*), 340
cauchy_qtf() (*math53 method*), 340
cbrt() (*ctx method*), 235
cdf() (*dist_amoroso method*), 497
cdf() (*dist_arcsine method*), 337
cdf() (*dist_bennett method*), 780
cdf() (*dist_bernoulli method*), 747

cdf() (*dist_beta method*), 517
cdf() (*dist_beta_nc_type_I method*), 576
cdf() (*dist_beta_prime method*), 533
cdf() (*dist_beta_product method*), 655
cdf() (*dist_binomial method*), 761
cdf() (*dist_birnb_saunders method*), 437
cdf() (*dist_box_cov method*), 688
cdf() (*dist_box_mvn method*), 693
cdf() (*dist_box_nsame_cov method*), 684
cdf() (*dist_cauchy method*), 341
cdf() (*dist_chi method*), 469
cdf() (*dist_chi_nc method*), 580
cdf() (*dist_chi_squared method*), 473
cdf() (*dist_chi_squared_nc method*), 562
cdf() (*dist_dagum method*), 387
cdf() (*dist_dunnett1_t method*), 633
cdf() (*dist_dunnett2_t method*), 636
cdf() (*dist_emg method*), 441
cdf() (*dist_exponential method*), 344
cdf() (*dist_feller_pareto method*), 548
cdf() (*dist_fisher_f method*), 522
cdf() (*dist_fisher_f_2nc method*), 606
cdf() (*dist_fisher_f_nc method*), 571
cdf() (*dist_fisher_r2_type_I method*), 594
cdf() (*dist_fisher_z method*), 551
cdf() (*dist_fisk method*), 390
cdf() (*dist_folded_normal method*), 444
cdf() (*dist_frechet method*), 393
cdf() (*dist_friedman method*), 810
cdf() (*dist_gamma method*), 478
cdf() (*dist_gen_beta_exp method*), 545
cdf() (*dist_genbeta1 method*), 536
cdf() (*dist_genbeta2 method*), 539
cdf() (*dist_genlogistic method*), 542
cdf() (*dist_genpareto method*), 399
cdf() (*dist_geometric method*), 751
cdf() (*dist_gev method*), 396
cdf() (*dist_gompertz method*), 402
cdf() (*dist_gumbel method*), 348
cdf() (*dist_half_normal method*), 447
cdf() (*dist_halperin_t method*), 642
cdf() (*dist_holtsmark method*), 732
cdf() (*dist_hotelling_t2 method*), 712
cdf() (*dist_hyperexponential method*), 352
cdf() (*dist_hypergeometric method*), 771
cdf() (*dist_hypoexponential method*), 503
cdf() (*dist_invchisquared method*), 482
cdf() (*dist_invgamma method*), 486
cdf() (*dist_johnson_sb method*), 450
cdf() (*dist_johnson_su method*), 453
cdf() (*dist_jterpsta_s method*), 799
cdf() (*dist_kendall_tau method*), 794
cdf() (*dist_kolmogorov_smirnov method*), 724
cdf() (*dist_kruskal_wallis method*), 813
cdf() (*dist_kumaraswamy method*), 356
cdf() (*dist_landau method*), 729
cdf() (*dist_laplace method*), 360
cdf() (*dist_levy method*), 416
cdf() (*dist_lindley method*), 507
cdf() (*dist_logistic method*), 364
cdf() (*dist_lognormal method*), 419
cdf() (*dist_logrv_beta method*), 530
cdf() (*dist_logrv_beta_nc_type_II method*), 590
cdf() (*dist_logrv_beta_sum method*), 651
cdf() (*dist_logrv_chisquared method*), 500
cdf() (*dist_logrv_fisher_1mr2 method*), 599
cdf() (*dist_lomax method*), 405
cdf() (*dist_lrt_s0 method*), 698
cdf() (*dist_lrt_x0_s0 method*), 702
cdf() (*dist_mann_whitney_u method*), 783
cdf() (*dist_mann_whitney_u_lehmann method*), 788
cdf() (*dist_mann_whitney_u_milton method*), 791
cdf() (*dist_mapairy method*), 737
cdf() (*dist_mauchley method*), 680
cdf() (*dist_maxwell method*), 490
cdf() (*dist_moyal method*), 423
cdf() (*dist_nair_t method*), 639
cdf() (*dist_nakagami method*), 494
cdf() (*dist_negbinom method*), 766
cdf() (*dist_nelson_h method*), 645
cdf() (*dist_nmax_0 method*), 456
cdf() (*dist_nmax_rho method*), 612
cdf() (*dist_nmax_rhoij method*), 618
cdf() (*dist_nmm_0 method*), 459
cdf() (*dist_nmm_rho method*), 615
cdf() (*dist_nmm_rhoij method*), 621
cdf() (*dist_normal method*), 427
cdf() (*dist_normal_range method*), 624
cdf() (*dist_page_l method*), 804
cdf() (*dist_page_l_nc_milton method*), 807
cdf() (*dist_pareto method*), 368
cdf() (*dist_pearson_rho method*), 558
cdf() (*dist_pearson_rho_nc method*), 586
cdf() (*dist_pillai_v method*), 709
cdf() (*dist_poisson method*), 756
cdf() (*dist_rayleigh method*), 372
cdf() (*dist_rice method*), 583
cdf() (*dist_roy method*), 706
cdf() (*dist_sasnormal method*), 462
cdf() (*dist_saspoint5 method*), 742
cdf() (*dist_shifted_gompertz method*), 408
cdf() (*dist_singh_maddala method*), 411
cdf() (*dist_skewnormal method*), 431
cdf() (*dist_skewt method*), 555
cdf() (*dist_smax method*), 627
cdf() (*dist_smm method*), 630
cdf() (*dist_stacy method*), 513
cdf() (*dist_student_t method*), 527

cdf() (*dist_student_t_2nc method*), 603
cdf() (*dist_student_t_nc method*), 567
cdf() (*dist_studentized_range method*), 648
cdf() (*dist_triangular method*), 376
cdf() (*dist_truncated_normal method*), 465
cdf() (*dist_uniform method*), 380
cdf() (*dist_wald method*), 435
cdf() (*dist_weibull method*), 384
cdf() (*dist_wilcoxon method*), 775
cdf() (*dist_wilks_cs method*), 664
cdf() (*dist_wilks_iblocks method*), 676
cdf() (*dist_wilks_ip method*), 672
cdf() (*dist_wilks_lambda method*), 660
cdf() (*dist_wilks_lambda_corr method*), 719
cdf() (*dist_wilks_lambda_glm method*), 715
cdf() (*dist_wilks_lcm method*), 668
cdf() (*rv_base method*), 322
cdf() (*skewexponpower method*), 510
cdf_from_cf_continuous() (*ctx method*), 1490
cdf_from_cf_continuous_boost() (*ctx method*), 855
cdf_from_cf_lattice() (*ctx method*), 1492
cdf_from_pdf() (*ctx method*), 1490
cdf_from_pmf() (*ctx method*), 1490
ceil() (*ctx method*), 213
ceil() (*npm method*), 1186
cel() (*math53 method*), 2174
cel1() (*math53 method*), 2173
cel2() (*math53 method*), 2173
centered() (*mat method*), 1214
central_moments() (*rv_base method*), 329
centralmoments_from_cumulants() (*ctx method*), 1501
centralmoments_from_factorialmoments() (*ctx method*), 1501
centralmoments_from_rawmoments() (*ctx method*), 1501
cf_from_rawmoments() (*ctx method*), 1494
cgf_from_cf() (*ctx method*), 1496
cgf_from_cumulants() (*ctx method*), 1496
cgf_from_mgf() (*ctx method*), 1496
cgf_from_pgf() (*ctx method*), 1496
chebyfit() (*ctx method*), 904
chebyshev_t() (*ctx method*), 1446
chebyshev_u() (*ctx method*), 1447
chebyshev_v() (*ctx method*), 1448
chebyshev_w() (*ctx method*), 1449
chf() (*dist_bernoulli method*), 747
chf() (*dist_holtsmark method*), 732
chf() (*dist_kolmogorov_smirnov method*), 724
chf() (*dist_mapairy method*), 737
chf() (*dist_saspoint5 method*), 742
chf() (*rv_continuous method*), 323
chi2_cdf() (*math53 method*), 472
chi2_cohen_cdf() (*ctx method*), 1504
chi2_nc_benton_cdf_sf() (*ctx method*), 1515
chi2_nc_canal_qtf() (*ctx method*), 1544
chi2_nc_cdf() (*math53 method*), 561
chi2_nc_cohen_cdf() (*ctx method*), 1506
chi2_nc_mu2_cdf() (*ctx method*), 1557
chi2_nc_mu2_qtf() (*ctx method*), 1557
chi2_nc_pdf() (*math53 method*), 561
chi2_nc_penev_cdf() (*ctx method*), 1544
chi2_nc_qtf() (*math53 method*), 561
chi2_nc_wb_cl() (*ctx method*), 1557
chi2_pdf() (*math53 method*), 472
chi2_qtf() (*math53 method*), 472
chi2_variance_1sample_ci() (*ctx method*), 2296
chi2_variance_1sample_power() (*ctx method*), 2297
chi2_variance_1sample_samplesize() (*ctx method*), 2298
chi2_variance_1sample_test() (*ctx method*), 2295
chi_cdf() (*math53 method*), 468
chi_pdf() (*math53 method*), 468
chi_qtf() (*math53 method*), 468
chi_squared_ecf() (*ctx method*), 475
chi_squared_ecf_inv() (*ctx method*), 475
chi_squared_gp() (*ctx method*), 475
chi_squared_nc_ecf() (*ctx method*), 565
chi_squared_nc_ecf_inv() (*ctx method*), 565
chi_squared_nc_gp() (*ctx method*), 565
chi_squared_nc_quad_cdf() (*ctx method*), 1528
chi_squared_nc_quad_cdf_verified() (*ctx method*), 1532
chi_squared_nc_recurrence() (*ctx method*), 564
chi_squared_nc_spa() (*ctx method*), 565
chi_squared_nc_spa_inv() (*ctx method*), 565
chi_squared_recurrence() (*ctx method*), 475
chi_squared_spa() (*ctx method*), 475
chi_squared_spa_inv() (*ctx method*), 476
cholesky() (*ctx method*), 1220
cholesky_solve() (*ctx method*), 1221
cin() (*math53 method*), 2227
cinh() (*math53 method*), 2227
clausen2() (*ctx method*), 1309
clausen_cos() (*ctx method*), 1308
clausen_sin() (*ctx method*), 1306
CMAes() (*ctxboost method*), 934
cochran_s_pmf_vector() (*ctx method*), 1542
cochran_s_pmf_vector() (*math53lib method*), 2288
cohen_alt() (*ctx method*), 923
column_stack() (*npm method*), 1134
compound_si() (*ctx method*), 2154
comprel() (*ctx method*), 2154
concatenate() (*npm method*), 1130
cond() (*ctx method*), 1222
conj() (*ctx method*), 230
conj_grad_c() (*ctx method*), 839
ConjugateGradientDescent() (*ctxboost method*), 942
convolve() (*npm method*), 1178
copysign() (*ctx method*), 215
copyto() (*npm method*), 1120
cornish_fisher() (*ctx method*), 898

correlation() (*mat method*), 1217
cos() (*ctx method*), 260
cos_integral() (*ctx method*), 1441
cosd() (*math53 method*), 2156
cosh() (*ctx method*), 273
cosh_integral() (*ctx method*), 1437
coshm1() (*math53 method*), 2161
cosint() (*math53 method*), 2159
coslemn() (*math53 method*), 2187
cospi() (*ctx method*), 268
cot() (*ctx method*), 262
cotd() (*math53 method*), 2157
coth() (*ctx method*), 276
cotpi() (*ctx method*), 270
coulomb_cl() (*math53 method*), 2217
coulomb_f() (*ctx method*), 1405
coulomb_f_fprime() (*math53 method*), 2219
coulomb_g() (*ctx method*), 1406
coulomb_g() (*math53 method*), 2220
coulomb_hminus() (*ctx method*), 1408
coulomb_hplus() (*ctx method*), 1407
coulomb_sl() (*math53 method*), 2218
covariance_matrix() (*mat method*), 1216
covers() (*math53 method*), 2159
csc() (*ctx method*), 264
csch() (*ctx method*), 277
cscpi() (*ctx method*), 270
ctx.dist_amoroso (*built-in class*), 497
ctx.dist_arcsine (*built-in class*), 336
ctx.dist_bennett (*built-in class*), 780
ctx.dist_beta (*built-in class*), 516
ctx.dist_beta_nc_type_I (*built-in class*), 576
ctx.dist_beta_prime (*built-in class*), 533
ctx.dist_beta_product (*built-in class*), 655
ctx.dist_binomial (*built-in class*), 760
ctx.dist_birnb_saunders (*built-in class*), 437
ctx.dist_box_cov (*built-in class*), 688
ctx.dist_box_mvn (*built-in class*), 693
ctx.dist_box_nsame_cov (*built-in class*), 684
ctx.dist_cauchy (*built-in class*), 340
ctx.dist_chi (*built-in class*), 468
ctx.dist_chi2 (*built-in class*), 472
ctx.dist_chi2_nc (*built-in class*), 562
ctx.dist_chi_nc (*built-in class*), 580
ctx.dist_dagum (*built-in class*), 387
ctx.dist_dunnett1_t (*built-in class*), 633
ctx.dist_dunnett2_t (*built-in class*), 636
ctx.dist_emg (*built-in class*), 441
ctx.dist_exponential (*built-in class*), 343
ctx.dist_feller_pareto (*built-in class*), 548
ctx.dist_fisher_f (*built-in class*), 521
ctx.dist_fisher_f_2nc (*built-in class*), 606
ctx.dist_fisher_f_nc (*built-in class*), 571
ctx.dist_fisher_r2_type_I (*built-in class*), 594
ctx.dist_fisher_z (*built-in class*), 551
ctx.dist_fisk (*built-in class*), 390
ctx.dist_folded_normal (*built-in class*), 444
ctx.dist_frechet (*built-in class*), 393
ctx.dist_friedman (*built-in class*), 810
ctx.dist_gamma (*built-in class*), 477
ctx.dist_gen_beta_exp (*built-in class*), 545
ctx.dist_genbeta1 (*built-in class*), 536
ctx.dist_genbeta2 (*built-in class*), 539
ctx.dist_genlogistic (*built-in class*), 542
ctx.dist_genpareto (*built-in class*), 399
ctx.dist_geometric (*built-in class*), 751
ctx.dist_gev (*built-in class*), 396
ctx.dist_gompertz (*built-in class*), 402
ctx.dist_gumbel (*built-in class*), 347
ctx.dist_half_normal (*built-in class*), 447
ctx.dist_halperin_t (*built-in class*), 642
ctx.dist_hotelling_t2 (*built-in class*), 712
ctx.dist_hyperexponential (*built-in class*), 351
ctx.dist_hypergeometric (*built-in class*), 770
ctx.dist_hypoexponential (*built-in class*), 503
ctx.dist_inverse_chi2 (*built-in class*), 481
ctx.dist_inverse_gamma (*built-in class*), 485
ctx.dist_johnson_sb (*built-in class*), 450
ctx.dist_johnson_su (*built-in class*), 453
ctx.dist_jterpsta_s (*built-in class*), 799
ctx.dist_kendall_tau (*built-in class*), 794
ctx.dist_kruskal_wallis (*built-in class*), 813
ctx.dist_kumaraswamy (*built-in class*), 355
ctx.dist_landau (*built-in class*), 728
ctx.dist_laplace (*built-in class*), 359
ctx.dist_levy (*built-in class*), 415
ctx.dist_lindley (*built-in class*), 507
ctx.dist_logistic (*built-in class*), 363
ctx.dist_lognormal (*built-in class*), 418
ctx.dist_logrv_beta (*built-in class*), 530
ctx.dist_logrv_beta_nc_type_II (*built-in class*), 590
ctx.dist_logrv_beta_sum (*built-in class*), 651
ctx.dist_logrv_chisquared (*built-in class*), 500
ctx.dist_logrv_fisher_1mr2 (*built-in class*), 599
ctx.dist_lomax (*built-in class*), 405
ctx.dist_lrt_s0 (*built-in class*), 697
ctx.dist_lrt_x0_s0 (*built-in class*), 702
ctx.dist_mann_whitney_u (*built-in class*), 783
ctx.dist_mann_whitney_u_lehmann (*built-in class*), 788
ctx.dist_mann_whitney_u_milton (*built-in class*), 791
ctx.dist_mauchley (*built-in class*), 680
ctx.dist_maxwell (*built-in class*), 489
ctx.dist_moyal (*built-in class*), 422
ctx.dist_nair_t (*built-in class*), 639
ctx.dist_nakagami (*built-in class*), 493
ctx.dist_negbinom (*built-in class*), 765
ctx.dist_nelson_h (*built-in class*), 645
ctx.dist_nmax_0 (*built-in class*), 456

ctx.dist_nmax_rho (*built-in class*), 612
ctx.dist_nmax_rhoij (*built-in class*), 618
ctx.dist_nmm_0 (*built-in class*), 459
ctx.dist_nmm_rho (*built-in class*), 615
ctx.dist_nmm_rhoij (*built-in class*), 621
ctx.dist_normal (*built-in class*), 426
ctx.dist_normal_range (*built-in class*), 624
ctx.dist_page_l (*built-in class*), 804
ctx.dist_page_l_nc_milton (*built-in class*), 807
ctx.dist_pareto (*built-in class*), 367
ctx.dist_pearson_rho (*built-in class*), 558
ctx.dist_pearson_rho_nc (*built-in class*), 586
ctx.dist_pillai_v (*built-in class*), 709
ctx.dist_poisson (*built-in class*), 755
ctx.dist_rayleigh (*built-in class*), 371
ctx.dist_rice (*built-in class*), 583
ctx.dist_roy (*built-in class*), 706
ctx.dist_sasnormal (*built-in class*), 462
ctx.dist_shifted_gompertz (*built-in class*), 408
ctx.dist_singh_maddala (*built-in class*), 411
ctx.dist_skewnormal (*built-in class*), 430
ctx.dist_skewt (*built-in class*), 555
ctx.dist_smax (*built-in class*), 627
ctx.dist_smm (*built-in class*), 630
ctx.dist_stacy (*built-in class*), 513
ctx.dist_student_t (*built-in class*), 526
ctx.dist_student_t_2nc (*built-in class*), 603
ctx.dist_student_t_nc (*built-in class*), 567
ctx.dist_studentized_range (*built-in class*), 648
ctx.dist_triangular (*built-in class*), 376
ctx.dist_truncated_normal (*built-in class*), 465
ctx.dist_uniform (*built-in class*), 379
ctx.dist_wald (*built-in class*), 434
ctx.dist_weibull (*built-in class*), 383
ctx.dist_wilcoxon (*built-in class*), 775
ctx.dist_wilks_cs (*built-in class*), 664
ctx.dist_wilks_iblocks (*built-in class*), 676
ctx.dist_wilks_ip (*built-in class*), 672
ctx.dist_wilks_lambda (*built-in class*), 660
ctx.dist_wilks_lambda_corr (*built-in class*), 719
ctx.dist_wilks_lambda_glm (*built-in class*), 715
ctx.dist_wilks_lcm (*built-in class*), 668
ctx.skewexponpower (*built-in class*), 510
ctxlistcplx (*gui property*), 93
ctxlistreal (*gui property*), 93
cube() (*ctx method*), 251
cuberoot() (*math53 method*), 2232
cubic_equation() (*ctx method*), 2265
cubic_equation_monic() (*ctx method*), 2264
cubsolve() (*ctx method*), 820
cumprod() (*npm method*), 1156
cumsum() (*npm method*), 1156
cumulants() (*dist_amoroso method*), 499
cumulants() (*dist_arcsine method*), 339
cumulants() (*dist_bennett method*), 782
cumulants() (*dist_beta method*), 519
cumulants() (*dist_beta_nc_type_I method*), 578
cumulants() (*dist_beta_prime method*), 535
cumulants() (*dist_beta_product method*), 657
cumulants() (*dist_binomial method*), 763
cumulants() (*dist_birnb_saunders method*), 439
cumulants() (*dist_box_cov method*), 690
cumulants() (*dist_box_mvn method*), 695
cumulants() (*dist_box_nsame_cov method*), 686
cumulants() (*dist_cauchy method*), 342
cumulants() (*dist_chi method*), 471
cumulants() (*dist_chi_nc method*), 582
cumulants() (*dist_chi_squared method*), 475
cumulants() (*dist_chi_squared_nc method*), 564
cumulants() (*dist_dagum method*), 388
cumulants() (*dist_dunnett1_t method*), 635
cumulants() (*dist_dunnett2_t method*), 638
cumulants() (*dist_emg method*), 443
cumulants() (*dist_exponential method*), 346
cumulants() (*dist_feller_pareto method*), 550
cumulants() (*dist_fisher_f method*), 524
cumulants() (*dist_fisher_f_2nc method*), 608
cumulants() (*dist_fisher_f_nc method*), 573
cumulants() (*dist_fisher_r2_type_I method*), 596
cumulants() (*dist_fisher_z method*), 553
cumulants() (*dist_fisk method*), 391
cumulants() (*dist_folded_normal method*), 446
cumulants() (*dist_frechet method*), 394
cumulants() (*dist_friedman method*), 812
cumulants() (*dist_gamma method*), 480
cumulants() (*dist_gen_beta_exp method*), 547
cumulants() (*dist_genbeta1 method*), 538
cumulants() (*dist_genbeta2 method*), 541
cumulants() (*dist_genlogistic method*), 544
cumulants() (*dist_genpareto method*), 400
cumulants() (*dist_geometric method*), 753
cumulants() (*dist_gev method*), 398
cumulants() (*dist_gompertz method*), 404
cumulants() (*dist_gumbel method*), 350
cumulants() (*dist_half_normal method*), 449
cumulants() (*dist_halperin_t method*), 644
cumulants() (*dist_hotelling_t2 method*), 714
cumulants() (*dist_hyperexponential method*), 354
cumulants() (*dist_hypergeometric method*), 774
cumulants() (*dist_hypoexponential method*), 505
cumulants() (*dist_invchisquared method*), 484
cumulants() (*dist_invgamma method*), 487
cumulants() (*dist_johnson_sb method*), 452
cumulants() (*dist_johnson_su method*), 455
cumulants() (*dist_jterpsta_s method*), 802
cumulants() (*dist_kendall_tau method*), 797
cumulants() (*dist_kruskal_wallis method*), 815
cumulants() (*dist_kumaraswamy method*), 358

cumulants() (*dist_landau method*), 730
cumulants() (*dist_laplace method*), 362
cumulants() (*dist_levy method*), 417
cumulants() (*dist_lindley method*), 509
cumulants() (*dist_logistic method*), 366
cumulants() (*dist_lognormal method*), 420
cumulants() (*dist_logrv_beta method*), 532
cumulants() (*dist_logrv_beta_nc_type_II method*), 592
cumulants() (*dist_logrv_beta_sum method*), 653
cumulants() (*dist_logrv_chisquared method*), 502
cumulants() (*dist_logrv_fisher_1mr2 method*), 601
cumulants() (*dist_lomax method*), 406
cumulants() (*dist_lrt_s0 method*), 700
cumulants() (*dist_lrt_x0_s0 method*), 704
cumulants() (*dist_mann_whitney_u method*), 786
cumulants() (*dist_mann_whitney_u_lehmann method*), 790
cumulants() (*dist_mann_whitney_u_milton method*), 793
cumulants() (*dist_mauchley method*), 682
cumulants() (*dist_maxwell method*), 492
cumulants() (*dist_moyal method*), 425
cumulants() (*dist_nair_t method*), 641
cumulants() (*dist_nakagami method*), 496
cumulants() (*dist_negbinom method*), 769
cumulants() (*dist_nelson_h method*), 647
cumulants() (*dist_nmax_0 method*), 458
cumulants() (*dist_nmax_rho method*), 614
cumulants() (*dist_nmax_rhoij method*), 620
cumulants() (*dist_nmm_0 method*), 461
cumulants() (*dist_nmm_rho method*), 617
cumulants() (*dist_nmm_rhoij method*), 623
cumulants() (*dist_normal method*), 429
cumulants() (*dist_normal_range method*), 626
cumulants() (*dist_page_l method*), 806
cumulants() (*dist_page_l_nc_milton method*), 809
cumulants() (*dist_pareto method*), 370
cumulants() (*dist_pearson_rho method*), 560
cumulants() (*dist_pearson_rho_nc method*), 588
cumulants() (*dist_pillai_v method*), 711
cumulants() (*dist_poisson method*), 758
cumulants() (*dist_rayleigh method*), 374
cumulants() (*dist_rice method*), 585
cumulants() (*dist_roy method*), 708
cumulants() (*dist_sasnormal method*), 464
cumulants() (*dist_shifted_gompertz method*), 410
cumulants() (*dist_singh_maddala method*), 412
cumulants() (*dist_skewnormal method*), 433
cumulants() (*dist_skewt method*), 557
cumulants() (*dist_smax method*), 629
cumulants() (*dist_smm method*), 632
cumulants() (*dist_stacy method*), 515
cumulants() (*dist_student_t method*), 529
cumulants() (*dist_student_t_2nc method*), 605
cumulants() (*dist_student_t_nc method*), 569
cumulants() (*dist_studentized_range method*), 650
cumulants() (*dist_triangular method*), 378
cumulants() (*dist_truncated_normal method*), 467
cumulants() (*dist_uniform method*), 382
cumulants() (*dist_wald method*), 437
cumulants() (*dist_weibull method*), 386
cumulants() (*dist_wilcoxon method*), 778
cumulants() (*dist_wilks_cs method*), 666
cumulants() (*dist_wilks_iblocks method*), 678
cumulants() (*dist_wilks_ip method*), 674
cumulants() (*dist_wilks_lambda method*), 662
cumulants() (*dist_wilks_lambda_corr method*), 721
cumulants() (*dist_wilks_lambda_glm method*), 717
cumulants() (*dist_wilks_lcm method*), 670
cumulants() (*rv_base method*), 330
cumulants() (*skewexponpower method*), 512
cumulants_from_cdf() (*ctx method*), 1502
cumulants_from_centralmoments() (*ctx method*), 1502
cumulants_from_factorialmoments() (*ctx method*), 1502
cumulants_from_pmfvector(), 1502
cumulants_from_rawmoments() (*ctx method*), 1502
cylinder_d() (*ctx method*), 1411
cylinder_u() (*ctx method*), 1412
cylinder_v() (*ctx method*), 1413
cylinder_w() (*ctx method*), 1414

D

damath_bessel_in() (*math53 method*), 2205
dawson() (*ctx method*), 1424
dawson2() (*math53 method*), 2222
debye() (*math53 method*), 2195
dedekind_eta() (*ctx method*), 1287
degree (*ctx property*), 225
delete() (*npm method*), 1140
deriv1_c() (*ctx method*), 888
deriv2_c() (*ctx method*), 888
det() (*ctx method*), 1221
diag() (*npm method*), 1108
diagflat() (*npm method*), 1111
diagonal() (*npm method*), 1110
diff() (*ctx method*), 889
diff() (*npm method*), 1157
difference() (*ctx method*), 891
differint() (*ctx method*), 893
diffs() (*ctx method*), 891
diffs_exp() (*ctx method*), 892
diffs_prod() (*ctx method*), 892
diffun() (*ctx method*), 891
digamma() (*ctx method*), 1298
dilog() (*ctx method*), 1304
dirichlet_beta() (*ctx method*), 1335
dirichlet_eta() (*ctx method*), 1333
dirichlet_eta_i() (*math53 method*), 2197
dirichlet_eta_m1() (*ctx method*), 1334
dirichlet_lambda() (*ctx method*), 1336

dist_bernoulli() (*CtxBoost method*), 747
dist_holtsmark() (*CtxBoost method*), 731
dist_kolmogorov_smirnov() (*CtxBoost method*), 723
dist_mapairy() (*CtxBoost method*), 737
dist_saspoint5() (*CtxBoost method*), 741
divide() (*npm method*), 1165
DormandPrince5Adaptive() (*ctxboost method*), 883
DormandPrince5Const() (*ctxboost method*), 879
DormandPrince5DenseOutput() (*ctxboost method*), 886
dot() (*npm method*), 1171
double_factorial() (*ctx method*), 306
dsplit() (*npm method*), 1135
dstack() (*npm method*), 1133
dunn_sidak_qtf() (*ctx method*), 1561
Dunnett1Cdf() (*math53lib method*), 2280
Dunnett1Dist() (*math53lib method*), 2280
Dunnett1Pdf() (*math53lib method*), 2280
Dunnett1Qtf() (*math53lib method*), 2280
Dunnett2Cdf() (*math53lib method*), 2281
Dunnett2Dist() (*math53lib method*), 2281
Dunnett2Pdf() (*math53lib method*), 2281
Dunnett2Qtf() (*math53lib method*), 2281

E

e (*ctx property*), 226
edgeworth() (*ctx method*), 897
ediff1d() (*npm method*), 1158
egamma (*ctx property*), 227
ei_inv() (*math53 method*), 2228
eibeta() (*math53 method*), 2229
eig() (*ctx method*), 1233
eig_sort() (*ctx method*), 1234
eigh() (*ctx method*), 1230
eighe() (*ctx method*), 1229
eigsy() (*ctx method*), 1228
ein() (*math53 method*), 2227
einstein() (*math53 method*), 2151
einsum() (*npm method*), 1176
eisx2() (*math53 method*), 2228
el1() (*math53 method*), 2174
el2() (*math53 method*), 2174
el3() (*math53 method*), 2175
elliptic_b() (*math53 method*), 2171
elliptic_b_inc() (*math53 method*), 2171
elliptic_d() (*math53 method*), 2171
elliptic_d_inc() (*math53 method*), 2172
elliptic_e() (*ctx method*), 1250
elliptic_e_inc() (*ctx method*), 1252
elliptic_f() (*ctx method*), 1252
elliptic_invariants_from_roots() (*ctx method*), 1278
elliptic_k() (*ctx method*), 1249
elliptic_modular_delta() (*ctx.ctx method*), 1288
elliptic_modular_lambda() (*ctx.ctx method*), 1287
elliptic_modulus() (*math53 method*), 2169
elliptic_nome() (*math53 method*), 2169
elliptic_pi() (*ctx method*), 1251
elliptic_pi_inc() (*ctx method*), 1253
elliptic_rc() (*ctx method*), 1241
elliptic_rd() (*ctx method*), 1241
elliptic_rf() (*ctx method*), 1239
elliptic_rg() (*ctx method*), 1239
elliptic_rj() (*ctx method*), 1240
elliptic_roots_from_tau() (*ctx method*), 1281
EllipticCE() (*math53 method*), 2177
EllipticCK() (*math53 method*), 2176
EllipticCPi() (*math53 method*), 2178
EllipticE() (*math53 method*), 2179
EllipticEC() (*math53 method*), 2177
EllipticECim() (*math53 method*), 2177
EllipticF() (*math53 method*), 2179
EllipticK() (*math53 method*), 2176
EllipticKim() (*math53 method*), 2176
EllipticPi() (*math53 method*), 2179
EllipticPiC() (*math53 method*), 2178
EllipticPiCim() (*math53 method*), 2178
entropy() (*rv_continuous method*), 332
entropy() (*rv_discrete method*), 334
epsilon() (*ctx method*), 222
equal() (*npm method*), 1184
erf() (*ctx method*), 1415
erf2() (*math53 method*), 2221
erf_inv() (*ctx method*), 1419
erfc() (*ctx method*), 1417
erfc_inv() (*ctx method*), 1420
erfcx() (*math53 method*), 2221
erfcx_inv() (*math53 method*), 2222
erfg() (*math53 method*), 2222
erfh() (*math53 method*), 2221
erfi() (*ctx method*), 1423
erfi_inv() (*math53 method*), 2222
erfq() (*math53 method*), 2221
euler_q() (*math53 method*), 2199
eulernum() (*ctx method*), 1320
eulerpoly() (*ctx method*), 1321
eval_cubic() (*ctx method*), 2265
eval_monic_cubic() (*ctx method*), 2264
eval_quadratic() (*ctx method*), 2263
eval_quartic() (*ctx method*), 2266
excess_kurtosis() (*rv_base method*), 326
exp() (*ctx method*), 238
exp10() (*ctx method*), 240
exp10m1() (*ctx method*), 243
exp2() (*ctx method*), 241
exp2m1() (*ctx method*), 244
exp_integral_e1() (*ctx method*), 1431
exp_integral_e1_scaled() (*math53 method*), 2227
exp_integral_ei() (*ctx method*), 1432
exp_integral_ei_scaled() (*math53 method*), 2228

exp_integral_en() (*ctx method*), 1438
expand_dims() (*npm method*), 1127
expint3() (*math53 method*), 2223
expj() (*ctx method*), 239
expjpi() (*ctx method*), 240
expm1() (*ctx method*), 242
expmx2h() (*math53 method*), 2150
expn() (*math53 method*), 2216
exponential_cdf() (*math53 method*), 343
exponential_pdf() (*math53 method*), 343
exponential_qtf() (*math53 method*), 343
exprel() (*math53 method*), 2150
expreln() (*math53 method*), 2216
ExpSinh() (*ctxboost method*), 853
expx2() (*math53 method*), 2151
eye() (*npm method*), 1088

F

fabs() (*ctx method*), 229
factorial() (*ctx method*), 305
factorial_moments_from_cumulants() (*ctx method*), 1498
factorial_moments_from_rawmoments() (*ctx method*), 1498
fadd() (*ctx method*), 1207
faddeeva() (*ctx method*), 1425
falling_factorial() (*ctx method*), 309
fdim() (*ctx method*), 217
fdiv() (*ctx method*), 1211
fdot() (*ctx method*), 1212
Fehlberg78Adaptive() (*ctxboost method*), 884
Fehlberg78Const() (*ctxboost method*), 880
fermi_dirac() (*ctx method*), 1311
fermi_dirac_i() (*math53 method*), 2193
fermi_dirac_m05() (*math53 method*), 2194
fermi_dirac_p05() (*math53 method*), 2194
fermi_dirac_p15() (*math53 method*), 2194
fermi_dirac_p25() (*math53 method*), 2194
fibfun() (*math53 method*), 2155
fibpoly() (*math53 method*), 2154
findroot() (*ctx method*), 832
fisher_f_cdf() (*math53 method*), 521
fisher_f_davis_qtf() (*ctx method*), 1546
fisher_f_nc2_cdf_recurrence() (*ctx method*), 609
fisher_f_nc2_ecf() (*ctx method*), 609
fisher_f_nc2_ecf_inv() (*ctx method*), 609
fisher_f_nc2_mu2_cdf() (*ctx method*), 1562
fisher_f_nc2_mu2_qtf() (*ctx method*), 1563
fisher_f_nc2_pdf_recurrence() (*ctx method*), 608
fisher_f_nc2_spa() (*ctx method*), 609
fisher_f_nc2_spa_inv() (*ctx method*), 610
fisher_f_nc_benton_cdf_sf() (*ctx method*), 1517
fisher_f_nc_cdf() (*math53 method*), 570
fisher_f_nc_cdf_recurrence() (*ctx method*), 574
fisher_f_nc_ecf() (*ctx method*), 574
fisher_f_nc_ecf_inv() (*ctx method*), 574
fisher_f_nc_mu2_cdf() (*ctx method*), 1561
fisher_f_nc_mu2_cl() (*ctx method*), 1562
fisher_f_nc_mu2_qtf() (*ctx method*), 1562
fisher_f_nc_pdf() (*math53 method*), 570
fisher_f_nc_pdf_recurrence() (*ctx method*), 573
fisher_f_nc_qtf() (*math53 method*), 570
fisher_f_nc_seber_cdf() (*ctx method*), 1507
fisher_f_pdf() (*math53 method*), 521
fisher_f_qtf() (*math53 method*), 521
fisher_f_seber_cdf() (*ctx method*), 1505
fisher_log1mr2_gp() (*ctx method*), 602
fisher_r2_gd1_cdf() (*ctx method*), 1510
fisher_r2_gd2_cdf() (*ctx method*), 1520
fisher_r2_lee_cdf() (*ctx method*), 1564
fisher_r2_lee_cl() (*ctx method*), 1564
fisher_r2_lee_mu3_cdf() (*ctx method*), 1579
fisher_r2_lee_qtf() (*ctx method*), 1564
fisher_r2_spa() (*ctx method*), 602
fisher_r2_spa_inv() (*ctx method*), 602
fisher_z_ecf() (*ctx method*), 554
fisher_z_ecf_inv() (*ctx method*), 554
fisher_z_spa() (*ctx method*), 554
fisher_z_spa_inv() (*ctx method*), 554
fitsint32() (*ctx method*), 211
fitsint64() (*ctx method*), 211
fix() (*npm method*), 1187
flat() (*ndarry method*), 1122
flatten() (*ndarry method*), 1123
flip() (*npm method*), 1144
fliplr() (*npm method*), 1144
flipud() (*npm method*), 1145
floor() (*ctx method*), 213
floor() (*npm method*), 1186
floor_divide() (*npm method*), 1166
fma() (*ctx method*), 203
fmax() (*ctx method*), 204
fmin() (*ctx method*), 204
fmod() (*ctx method*), 219
fmul() (*ctx method*), 1210
fneg() (*ctx method*), 1209
fourier() (*ctx method*), 905
fourierval() (*ctx method*), 906
fprod() (*ctx method*), 1212
fratio_variance_2csamples_ci() (*ctx method*), 2327
fratio_variance_2csamples_power() (*ctx method*), 2329
fratio_variance_2csamples_samplesize() (*ctx method*), 2331
fratio_variance_2csamples_test() (*ctx method*), 2325
fratio_variance_2isamples_ci() (*ctx method*), 2311
fratio_variance_2isamples_power() (*ctx method*), 2313
fratio_variance_2isamples_samplesize() (*ctx method*), 2315
fratio_variance_2isamples_test() (*ctx method*), 2309

frechet_pmf() (*ctx method*), 1521
fresnel_c() (*ctx method*), 1428
fresnel_f() (*math53 method*), 2224
fresnel_g() (*math53 method*), 2224
fresnel_s() (*ctx method*), 1426
frexp() (*ctx method*), 215
friedman_s_pmf_vector() (*ctx method*), 1542
friedman_s_pmf_vector() (*math53lib method*), 2288
fromfile() (*npm method*), 1097
fromfunction() (*npm method*), 1097
fromiter() (*npm method*), 1098
fromstring() (*npm method*), 1099
fsub() (*ctx method*), 1209
fsum() (*ctx method*), 1212
full() (*npm method*), 1091
full_like() (*npm method*), 1092

G

g2g3_from_tau() (*ctx method*), 1280
g_x() (*dist_bennett method*), 781
g_x() (*dist_binomial method*), 762
g_x() (*dist_hypergeometric method*), 772
g_x() (*dist_jterpsta_s method*), 800
g_x() (*dist_kendall_tau method*), 795
g_x() (*dist_mann_whitney_u method*), 784
g_x() (*dist_mann_whitney_u_milton method*), 792
g_x() (*dist_mann_whitney_u_normal method*), 789
g_x() (*dist_negbinom method*), 767
g_x() (*dist_poisson method*), 757
g_x() (*dist_wilcoxon method*), 776
gamma() (*ctx method*), 299
gamma1pm1() (*ctx method*), 300
gamma_canal_qtf() (*ctx method*), 1545
gamma_cdf() (*math53 method*), 477
gamma_delta_ratio() (*ctx method*), 309
gamma_inv() (*math53 method*), 2166
gamma_lower() (*ctx method*), 1400
gamma_p() (*ctx method*), 1397
gamma_p_inv() (*ctx method*), 1402
gamma_p_inva() (*ctx method*), 1403
gamma_p_prime() (*ctx method*), 1402
gamma_paris_cdf_sf() (*ctx method*), 1514
gamma_pdf() (*math53 method*), 477
gamma_peizer_cdf_sf_pdf() (*ctx method*), 1513
gamma_q() (*ctx method*), 1398
gamma_q_inv() (*ctx method*), 1403
gamma_q_inva() (*ctx method*), 1403
gamma_qtf() (*math53 method*), 477
gamma_ratio() (*ctx method*), 309
gamma_tricomi() (*ctx method*), 2215
gamma_upper() (*ctx method*), 1401
gammastar() (*math53 method*), 2167
gauss_quad_rules() (*ctx method*), 866
GaussKronrod() (*ctxboost method*), 850
GaussLegendre() (*ctxboost method*), 849
gd() (*math53 method*), 2162
gegenbauer_c() (*ctx method*), 1450
gei() (*math53 method*), 2228
genfromtxt() (*npm method*), 1102
geometric_cdf() (*math53 method*), 750
geometric_pmf() (*math53 method*), 750
geometric_qtf() (*math53 method*), 751
geomspace() (*npm method*), 1107
get_local_appdata() (*gui method*), 91
get_local_appdata_xlcalcnet() (*gui method*), 91
get_my_documents() (*gui method*), 91
glaisher (*ctx property*), 228
goodwin_staton() (*math53 method*), 2224
gradient() (*npm method*), 1158
gradient_c() (*ctx method*), 888
GradientDescent() (*ctxboost method*), 940
greater() (*npm method*), 1183
greater_equal() (*npm method*), 1183
gumbel_cdf() (*math53 method*), 347
gumbel_pdf() (*math53 method*), 347
gumbel_qtf() (*math53 method*), 347

H

Halley() (*ctxboost method*), 829
halley_c() (*ctx method*), 838
HalperinCdf() (*math53lib method*), 2283
HalperinDist() (*math53lib method*), 2283
HalperinPdf() (*math53lib method*), 2283
HalperinQtf() (*math53lib method*), 2283
hankel_h1() (*ctx method*), 1369
hankel_h2() (*ctx method*), 1370
hardy_theta() (*mathc53 method*), 1329
hardy_z() (*mathc53 method*), 1330
harmonic() (*ctx method*), 1300
harmonic2() (*ctx method*), 1317
has_apm() (*gui method*), 92
has_gpm() (*gui method*), 92
has_xlcalcnet2() (*gui method*), 92
hav() (*math53 method*), 2158
hazard() (*rv_continuous method*), 323
hermite_h() (*ctx method*), 1395
hermite_he() (*ctx method*), 1396
hessenberg() (*ctx method*), 1234
heuman_lambda() (*math53 method*), 2172
hf() (*dist_bernoulli method*), 747
hf() (*dist_holtsmark method*), 732
hf() (*dist_kolmogorov_smirnov method*), 724
hf() (*dist_mapairy method*), 737
hf() (*dist_saspoint5 method*), 742
holtsmark_cdf() (*math53 method*), 731
holtsmark_pdf() (*math53 method*), 731
holtsmark_qtf() (*math53 method*), 731
hotelling_t2_bd() (*ctx method*), 714

hotelling_t2_bd_inv() (*ctx method*), 714
hotelling_t2_ecf() (*ctx method*), 714
hotelling_t2_ecf_inv() (*ctx method*), 714
hotelling_t2_glm_chi2_cdf() (*ctx method*), 1576
hotelling_t2_glm_mu2_cdf() (*ctx method*), 1580
hotelling_t2_ind_chi2_cdf() (*ctx method*), 1577
hotelling_t2_ind_mu2_cdf() (*ctx method*), 1580
hotelling_t2_mu3_cdf() (*ctx method*), 1567
hotelling_t2_mu3_qtf() (*ctx method*), 1567
hsplit() (*npm method*), 1136
hstack() (*npm method*), 1133
hurwitz_zeta() (*ctx method*), 1315
HyperbolicParaboloid() (*User method*), 1968
hyperexponential_cdf() (*math53 method*), 351
hyperexponential_pdf() (*math53 method*), 351
hyperexponential_qtf() (*math53 method*), 351
hyperfactorial() (*ctx method*), 1324
hyperg_0f1() (*ctx method*), 1338
hyperg_0f1r() (*ctx method*), 1340
hyperg_1f1() (*ctx method*), 1389
hyperg_1f1r() (*ctx method*), 1391
hyperg_1f2() (*ctx method*), 1473
hyperg_1f2r() (*ctx method*), 1474
hyperg_2f1() (*ctx method*), 1443
hyperg_2f1r() (*ctx method*), 1445
hyperg_u() (*ctx method*), 1392
hypergeo_ecf() (*ctx method*), 774
hypergeo_ecf_inv() (*ctx method*), 774
hypergeo_fm_pmf() (*ctx method*), 1522
hypergeo_ft() (*ctx method*), 764
hypergeo_nc_pmf_vector() (*ctx method*), 1536
hypergeo_spa() (*ctx method*), 774
hypergeo_spa_inv() (*ctx method*), 774
hypergeometric_cdf() (*math53 method*), 770
hypergeometric_pmf() (*math53 method*), 770
hypergeometric_pmf_vector() (*ctx method*), 1536
hypergeometric_qtf() (*math53 method*), 770
hypermat_1f1_butler() (*ctx method*), 1583
hypermat_2f1_butler() (*ctx method*), 1584
hypot() (*ctx method*), 253
hypot3() (*math53 method*), 2154

I

ibeta() (*ctx method*), 1461
ibeta_inv() (*ctx method*), 1467
ibeta_inva() (*ctx method*), 1469
ibeta_invb() (*ctx method*), 1471
ibeta_prime() (*ctx method*), 1463
ibetac() (*ctx method*), 1465
ibetac_inv() (*ctx method*), 1468
ibetac_inva() (*ctx method*), 1470
ibetac_invb() (*ctx method*), 1472
identity() (*npm method*), 1088
illinois() (*ctx method*), 834
ilogb() (*ctx method*), 216
imag() (*ctx method*), 230
improve_solution(), 1222
inerfc() (*math53 method*), 2223
inf() (*ctx method*), 206
info() (*gui method*), 92
inner() (*npm method*), 1172
insert() (*npm method*), 1140
intde() (*ctx method*), 845
intdei() (*ctx method*), 846
intdeo() (*ctx method*), 847
inverse() (*ctx method*), 1221
inverse_chi2_cdf() (*math53 method*), 481
inverse_chi2_pdf() (*math53 method*), 481
inverse_chi2_qtf() (*math53 method*), 481
inverse_gamma_cdf() (*math53 method*), 485
inverse_gamma_pdf() (*math53 method*), 485
inverse_gamma_qtf() (*math53 method*), 485
inverse_tan_integral() (*ctx method*), 1314
invertlaplace() (*ctx method*), 870
invlapcohen() (*ctx method*), 875
invlapdehoog() (*ctx method*), 874
invlapstehfest() (*ctx method*), 873
invlaptalbot() (*ctx method*), 872
isf() (*dist_amoroso method*), 498
isf() (*dist_arcsine method*), 338
isf() (*dist_bennett method*), 781
isf() (*dist_bernoulli method*), 747
isf() (*dist_beta method*), 518
isf() (*dist_beta_nc_type_I method*), 577
isf() (*dist_beta_prime method*), 534
isf() (*dist_beta_product method*), 656
isf() (*dist_binomial method*), 762
isf() (*dist_birnb_saunders method*), 438
isf() (*dist_box_cov method*), 689
isf() (*dist_box_mvn method*), 694
isf() (*dist_box_nsame_cov method*), 685
isf() (*dist_cauchy method*), 341
isf() (*dist_chi method*), 470
isf() (*dist_chi_nc method*), 581
isf() (*dist_chi_squared method*), 473
isf() (*dist_chi_squared_nc method*), 563
isf() (*dist_dagum method*), 388
isf() (*dist_dunnett1_t method*), 634
isf() (*dist_dunnett2_t method*), 637
isf() (*dist_emg method*), 442
isf() (*dist_exponential method*), 344
isf() (*dist_feller_pareto method*), 549
isf() (*dist_fisher_f method*), 523
isf() (*dist_fisher_f_2nc method*), 607
isf() (*dist_fisher_f_nc method*), 572
isf() (*dist_fisher_r2_type_I method*), 595
isf() (*dist_fisher_z method*), 552
isf() (*dist_fisk method*), 391

isf() (*dist_folded_normal method*), 445
isf() (*dist_frechet method*), 394
isf() (*dist_friedman method*), 811
isf() (*dist_gamma method*), 479
isf() (*dist_gen_beta_exp method*), 546
isf() (*dist_genbeta1 method*), 537
isf() (*dist_genbeta2 method*), 540
isf() (*dist_genlogistic method*), 543
isf() (*dist_genpareto method*), 400
isf() (*dist_geometric method*), 752
isf() (*dist_gev method*), 397
isf() (*dist_gompertz method*), 403
isf() (*dist_gumbel method*), 348
isf() (*dist_half_normal method*), 448
isf() (*dist_halperin_t method*), 643
isf() (*dist_holtsmark method*), 732
isf() (*dist_hotelling_t2 method*), 713
isf() (*dist_hyperexponential method*), 353
isf() (*dist_hypergeometric method*), 772
isf() (*dist_hypoexponential method*), 504
isf() (*dist_invchisquared method*), 483
isf() (*dist_invgamma method*), 487
isf() (*dist_johnson_sb method*), 451
isf() (*dist_johnson_su method*), 454
isf() (*dist_jterpsta_s method*), 800
isf() (*dist_kendall_tau method*), 795
isf() (*dist_kolmogorov_smirnov method*), 724
isf() (*dist_kruskal_wallis method*), 814
isf() (*dist_kumaraswamy method*), 356
isf() (*dist_landau method*), 730
isf() (*dist_laplace method*), 360
isf() (*dist_levy method*), 416
isf() (*dist_lindley method*), 508
isf() (*dist_logistic method*), 364
isf() (*dist_lognormal method*), 419
isf() (*dist_logrv_beta method*), 531
isf() (*dist_logrv_beta_nc_type_II method*), 591
isf() (*dist_logrv_beta_sum method*), 652
isf() (*dist_logrv_chisquared method*), 501
isf() (*dist_logrv_fisher_1mr2 method*), 600
isf() (*dist_lomax method*), 406
isf() (*dist_lrt_s0 method*), 699
isf() (*dist_lrt_x0_s0 method*), 703
isf() (*dist_mann_whitney_u method*), 784
isf() (*dist_mann_whitney_u_lehmann method*), 789
isf() (*dist_mann_whitney_u_milton method*), 792
isf() (*dist_mapairy method*), 737
isf() (*dist_mauchley method*), 681
isf() (*dist_maxwell method*), 491
isf() (*dist_moyal method*), 423
isf() (*dist_nair_t method*), 640
isf() (*dist_nakagami method*), 495
isf() (*dist_negbinom method*), 767
isf() (*dist_nelson_h method*), 646
isf() (*dist_nmax_0 method*), 457
isf() (*dist_nmax_rho method*), 613
isf() (*dist_nmax_rhoij method*), 619
isf() (*dist_nmm_0 method*), 460
isf() (*dist_nmm_rho method*), 616
isf() (*dist_nmm_rhoij method*), 622
isf() (*dist_normal method*), 428
isf() (*dist_normal_range method*), 625
isf() (*dist_page_l method*), 805
isf() (*dist_page_l_nc_milton method*), 808
isf() (*dist_pareto method*), 368
isf() (*dist_pearson_rho method*), 559
isf() (*dist_pearson_rho_nc method*), 587
isf() (*dist_pillai_v method*), 710
isf() (*dist_poisson method*), 757
isf() (*dist_rayleigh method*), 372
isf() (*dist_rice method*), 584
isf() (*dist_roy method*), 707
isf() (*dist_sasnormal method*), 463
isf() (*dist_saspoint5 method*), 742
isf() (*dist_shifted_gompertz method*), 409
isf() (*dist_singh_maddala method*), 412
isf() (*dist_skewnormal method*), 431
isf() (*dist_skewt method*), 556
isf() (*dist_smax method*), 628
isf() (*dist_smm method*), 631
isf() (*dist_stacy method*), 514
isf() (*dist_student_t method*), 528
isf() (*dist_student_t_2nc method*), 604
isf() (*dist_student_t_nc method*), 568
isf() (*dist_studentized_range method*), 649
isf() (*dist_triangular method*), 377
isf() (*dist_truncated_normal method*), 466
isf() (*dist_uniform method*), 380
isf() (*dist_wald method*), 435
isf() (*dist_weibull method*), 384
isf() (*dist_wilcoxon method*), 776
isf() (*dist_wilks_cs method*), 665
isf() (*dist_wilks_iblocks method*), 677
isf() (*dist_wilks_ip method*), 673
isf() (*dist_wilks_lambda method*), 660
isf() (*dist_wilks_lambda_corr method*), 720
isf() (*dist_wilks_lambda_glm method*), 716
isf() (*dist_wilks_lcm method*), 669
isf() (*rv_base method*), 324
isf() (*skewexponpower method*), 511
isfinite() (*ctx method*), 208
isinf() (*ctx method*), 208
isinteger() (*ctx method*), 210
isnan() (*ctx method*), 209
isneginf() (*ctx method*), 209
isnormal() (*ctx method*), 210
isnumber() (*ctx method*), 210
isone() (*ctx method*), 209

isposinf() (*ctx method*), 208
isregular() (*ctx method*), 210
isunordered() (*ctx method*), 211
iszero() (*ctx method*), 209

J

jacobi_amplitude() (*math53 method*), 2170
jacobi_arccd() (*math53 method*), 2183
jacobi_arccn() (*math53 method*), 2182
jacobi_arccs() (*math53 method*), 2185
jacobi_arcdc() (*math53 method*), 2184
jacobi_arcdn() (*math53 method*), 2182
jacobi_arcds() (*math53 method*), 2185
jacobi_arcnc() (*math53 method*), 2184
jacobi_arcnd() (*math53 method*), 2183
jacobi_arcns() (*math53 method*), 2185
jacobi_arcsc() (*math53 method*), 2184
jacobi_arcsd() (*math53 method*), 2183
jacobi_arcsn() (*math53 method*), 2182
jacobi_c() (*ctx method*), 888
jacobi_cd() (*ctx method*), 1266
jacobi_cn() (*ctx method*), 1257
jacobi_cs() (*ctx method*), 1268
jacobi_dc() (*ctx method*), 1263
jacobi_dn() (*ctx method*), 1259
jacobi_ds() (*ctx method*), 1269
jacobi_nc() (*ctx method*), 1261
jacobi_nd() (*ctx method*), 1264
jacobi_ns() (*ctx method*), 1267
jacobi_p() (*ctx method*), 1451
jacobi_sc() (*ctx method*), 1262
jacobi_sd() (*ctx method*), 1265
jacobi_sn() (*ctx method*), 1255
jacobi_theta02() (*math53 method*), 2180
jacobi_theta03() (*math53 method*), 2180
jacobi_theta04() (*math53 method*), 2180
jacobi_theta1() (*ctx method*), 1270
jacobi_theta1p() (*math53 method*), 2180
jacobi_theta2() (*ctx method*), 1272
jacobi_theta3() (*ctx method*), 1274
jacobi_theta4() (*ctx method*), 1276
jacobi_zeta() (*math53 method*), 2172
jensen_inv() (*ctx method*), 901
jterpsta_ecf() (*ctx method*), 802
jterpsta_ecf_inv() (*ctx method*), 802
jterpsta_fm_pmf() (*ctx method*), 1525
jterpsta_ft() (*ctx method*), 802
jterpsta_s_pmf_vector() (*ctx method*), 1539
jterpsta_spa() (*ctx method*), 803
jterpsta_spa_inv() (*ctx method*), 803
JTerpstaSPmfVector() (*math53lib method*), 2285

K

k_x() (*dist_amoroso method*), 498
k_x() (*dist_arcsine method*), 338
k_x() (*dist_bennett method*), 782
k_x() (*dist_beta method*), 518
k_x() (*dist_beta_nc_type_I method*), 578
k_x() (*dist_beta_prime method*), 534
k_x() (*dist_beta_product method*), 656
k_x() (*dist_binomial method*), 763
k_x() (*dist_birnb_saunders method*), 439
k_x() (*dist_box_cov method*), 690
k_x() (*dist_box_mvn method*), 695
k_x() (*dist_box_nsame_cov method*), 686
k_x() (*dist_cauchy method*), 342
k_x() (*dist_chi method*), 470
k_x() (*dist_chi_nc method*), 581
k_x() (*dist_chi_squared method*), 474
k_x() (*dist_chi_squared_nc method*), 564
k_x() (*dist_dagum method*), 388
k_x() (*dist_dunnett1_t method*), 635
k_x() (*dist_dunnett2_t method*), 637
k_x() (*dist_emg method*), 442
k_x() (*dist_exponential method*), 345
k_x() (*dist_feller_pareto method*), 550
k_x() (*dist_fisher_f method*), 523
k_x() (*dist_fisher_f_2nc method*), 607
k_x() (*dist_fisher_f_nc method*), 573
k_x() (*dist_fisher_r2_type_I method*), 596
k_x() (*dist_fisher_z method*), 553
k_x() (*dist_fisk method*), 391
k_x() (*dist_folded_normal method*), 445
k_x() (*dist_frechet method*), 394
k_x() (*dist_friedman method*), 811
k_x() (*dist_gamma method*), 479
k_x() (*dist_gen_beta_exp method*), 546
k_x() (*dist_genbeta1 method*), 537
k_x() (*dist_genbeta2 method*), 540
k_x() (*dist_genlogistic method*), 544
k_x() (*dist_genpareto method*), 400
k_x() (*dist_geometric method*), 753
k_x() (*dist_gev method*), 398
k_x() (*dist_gompertz method*), 403
k_x() (*dist_gumbel method*), 349
k_x() (*dist_half_normal method*), 448
k_x() (*dist_halperin_t method*), 643
k_x() (*dist_hotelling_t2 method*), 713
k_x() (*dist_hyperexponential method*), 353
k_x() (*dist_hypergeometric method*), 773
k_x() (*dist_hypoexponential method*), 505
k_x() (*dist_invchisquared method*), 483
k_x() (*dist_invgamma method*), 487
k_x() (*dist_johnson_sb method*), 451
k_x() (*dist_johnson_su method*), 454
k_x() (*dist_jterpsta_s method*), 801
k_x() (*dist_kendall_tau method*), 796
k_x() (*dist_kruskal_wallis method*), 814

k_x() (*dist_kumaraswamy method*), 357
k_x() (*dist_landau method*), 730
k_x() (*dist_laplace method*), 361
k_x() (*dist_levy method*), 417
k_x() (*dist_lindley method*), 508
k_x() (*dist_logistic method*), 365
k_x() (*dist_lognormal method*), 420
k_x() (*dist_logrv_beta method*), 531
k_x() (*dist_logrv_beta_sum method*), 653
k_x() (*dist_logrv_chisquared method*), 501
k_x() (*dist_lomax method*), 406
k_x() (*dist_lrt_s0 method*), 699
k_x() (*dist_lrt_x0_s0 method*), 704
k_x() (*dist_mann_whitney_u method*), 785
k_x() (*dist_mann_whitney_u_lehmann method*), 790
k_x() (*dist_mann_whitney_u_milton method*), 793
k_x() (*dist_mauchley method*), 682
k_x() (*dist_maxwell method*), 491
k_x() (*dist_moyal method*), 424
k_x() (*dist_nair_t method*), 640
k_x() (*dist_nakagami method*), 495
k_x() (*dist_negbinom method*), 768
k_x() (*dist_nelson_h method*), 646
k_x() (*dist_nmax_0 method*), 458
k_x() (*dist_nmax_rho method*), 613
k_x() (*dist_nmax_rhoij method*), 619
k_x() (*dist_nmm_0 method*), 461
k_x() (*dist_nmm_rho method*), 616
k_x() (*dist_nmm_rhoij method*), 622
k_x() (*dist_normal method*), 428
k_x() (*dist_normal_range method*), 625
k_x() (*dist_page_l method*), 805
k_x() (*dist_page_l_nc_milton method*), 808
k_x() (*dist_pareto method*), 369
k_x() (*dist_pearson_rho method*), 560
k_x() (*dist_pearson_rho_nc method*), 588
k_x() (*dist_pillai_v method*), 710
k_x() (*dist_poisson method*), 758
k_x() (*dist_rayleigh method*), 373
k_x() (*dist_rice method*), 584
k_x() (*dist_roy method*), 707
k_x() (*dist_sasnormal method*), 463
k_x() (*dist_shifted_gompertz method*), 409
k_x() (*dist_singh_maddala method*), 412
k_x() (*dist_skewnormal method*), 432
k_x() (*dist_skewt method*), 556
k_x() (*dist_smax method*), 628
k_x() (*dist_smm method*), 631
k_x() (*dist_stacy method*), 514
k_x() (*dist_student_t method*), 528
k_x() (*dist_student_t_2nc method*), 604
k_x() (*dist_student_t_nc method*), 569
k_x() (*dist_studentized_range method*), 650
k_x() (*dist_triangular method*), 378
k_x() (*dist_truncated_normal method*), 466
k_x() (*dist_uniform method*), 381
k_x() (*dist_wald method*), 436
k_x() (*dist_weibull method*), 385
k_x() (*dist_wilcoxon method*), 778
k_x() (*dist_wilks_cs method*), 666
k_x() (*dist_wilks_iblocks method*), 678
k_x() (*dist_wilks_ip method*), 674
k_x() (*dist_wilks_lambda method*), 661
k_x() (*dist_wilks_lambda_corr method*), 720
k_x() (*dist_wilks_lambda_glm method*), 716
k_x() (*dist_wilks_lcm method*), 670
k_x() (*rv_base method*), 328
k_x() (*skewexponpower method*), 511
k_y() (*dist_logrv_beta_nc_type_II method*), 592
k_y() (*dist_logrv_fisher_1mr2 method*), 601
kelvin_bei() (*ctx method*), 1382
kelvin_bei0() (*math53 method*), 2210
kelvin_bei0_prime() (*math53 method*), 2211
kelvin_bei_prime() (*ctx method*), 1386
kelvin_ber() (*ctx method*), 1381
kelvin_ber0() (*math53 method*), 2210
kelvin_ber0_prime() (*math53 method*), 2211
kelvin_ber_prime() (*ctx method*), 1385
kelvin_kei() (*ctx method*), 1384
kelvin_kei0() (*math53 method*), 2211
kelvin_kei0_prime() (*math53 method*), 2212
kelvin_kei_prime() (*ctx method*), 1388
kelvin_ker() (*ctx method*), 1383
kelvin_ker0() (*math53 method*), 2210
kelvin_ker0_prime() (*math53 method*), 2212
kelvin_ker_prime() (*ctx method*), 1387
kendall_fm_pmf() (*ctx method*), 1525
kendall_ft() (*ctx method*), 797
kendall_tau_ecf() (*ctx method*), 797
kendall_tau_ecf_inv() (*ctx method*), 798
kendall_tau_nc_mu4() (*ctx method*), 1555
kendall_tau_pmf_vector() (*ctx method*), 1538
kendall_tau_spa() (*ctx method*), 798
kendall_tau_spa_inv() (*ctx method*), 798
KendallTauPmfVector() (*math53lib method*), 2285
kepler() (*math53 method*), 2163
khinchin (*ctx property*), 228
klein_j() (*ctx method*), 1289
kolmogorov_smirnov_cdf() (*math53 method*), 723
kolmogorov_smirnov_pdf() (*math53 method*), 723
kolmogorov_smirnov_qtf() (*math53 method*), 723
kron() (*npm method*), 1177
kruskal_wallis_h_pmf_vector() (*ctx method*), 1543
kumaraswamy_cdf() (*math53 method*), 355
kumaraswamy_pdf() (*math53 method*), 355
kumaraswamy_qtf() (*math53 method*), 355
kurtosis() (*dist_bernoulli method*), 749
kurtosis() (*dist_holtsmark method*), 734

kurtosis() (*dist_kolmogorov_smirnov method*), 726
kurtosis() (*dist_mapairy method*), 739
kurtosis() (*dist_saspoint5 method*), 744
kurtosis() (*rv_base method*), 325
kurtosis_excess() (*dist_bernoulli method*), 749
kurtosis_excess() (*dist_holtsmark method*), 734
kurtosis_excess() (*dist_kolmogorov_smirnov method*), 726
kurtosis_excess() (*dist_mapairy method*), 739
kurtosis_excess() (*dist_saspoint5 method*), 744

L

laguerre_l() (*ctx method*), 1394
lambert_w0() (*ctx method*), 312
lambert_wk() (*ctx method*), 313
lambert_wm1() (*ctx method*), 313
landau_cdf() (*math53 method*), 728
landau_pdf() (*math53 method*), 728
landau_qtf() (*math53 method*), 728
langevinl() (*math53 method*), 2163
langevinlinv() (*math53 method*), 2163
laplace_cdf() (*math53 method*), 359
laplace_pdf() (*math53 method*), 359
laplace_qtf() (*math53 method*), 359
laurent_cdf() (*ctx method*), 1521
LBFGS() (*ctxboost method*), 938
lbfgs_c() (*ctx method*), 839
ldexp() (*ctx method*), 217
legendre_chi() (*ctx method*), 1312
legendre_p() (*ctx method*), 1454
legendre_plm() (*ctx method*), 1455
legendre_q() (*ctx method*), 1456
legendre_qlm() (*ctx method*), 1457
lerch_phi() (*ctx method*), 1291
lerch_zeta() (*ctx method*), 1294
less() (*npm method*), 1184
less_equal() (*npm method*), 1184
Levenberg() (*ctxboost method*), 929
levenberg_marquardt_c() (*ctx method*), 839
levin() (*ctx method*), 920
levy_cdf() (*math53 method*), 415
levy_pdf() (*math53 method*), 415
levy_qtf() (*math53 method*), 415
lexsort() (*npm method*), 1148
lgamma() (*ctx method*), 301
lgamma1p() (*math53 method*), 2165
lgamma_inv() (*math53 method*), 2167
lgamma_s() (*math53 method*), 2165
limit() (*ctx method*), 916
linspace() (*npm method*), 1105
llrint() (*ctx method*), 212
llround() (*ctx method*), 214
ln10 (*ctx property*), 226
ln2 (*ctx property*), 225
loadtxt() (*npm method*), 1100
lobachevsky_c() (*math53 method*), 2195
lobachevsky_s() (*math53 method*), 2195
localmin() (*ctx method*), 821
log() (*ctx method*), 245
log10() (*ctx method*), 246
log10p1() (*ctx method*), 249
log1mbeta_nc_gp() (*ctx method*), 592
log1mexp() (*math53 method*), 2152
log1p() (*ctx method*), 248
log1pexp() (*math53 method*), 2152
log1pmx() (*math53 method*), 2152
log2() (*ctx method*), 247
log2p1() (*ctx method*), 249
log_beta_prod_gp() (*ctx method*), 657
log_box_cov_gp() (*ctx method*), 691
log_integral() (*ctx method*), 1434
log_integral_inv() (*math53 method*), 2229
logaddexp() (*math53 method*), 2152
logb() (*ctx method*), 216
logbarnes_g() (*ctx method*), 1323
logbeta() (*math53 method*), 2166
logbinomial() (*math53 method*), 2166
logcosh() (*math53 method*), 2162
logfactorial() (*math53 method*), 2165
logical_and() (*npm method*), 1181
logical_or() (*npm method*), 1181
logical_xor() (*npm method*), 1182
logistic() (*math53 method*), 2151
logistic_cdf() (*math53 method*), 363
logistic_pdf() (*math53 method*), 363
logistic_qtf() (*math53 method*), 363
logit() (*math53 method*), 2153
lognormal_cdf() (*math53 method*), 418
lognormal_pdf() (*math53 method*), 418
lognormal_qtf() (*math53 method*), 418
logpdf() (*rv_continuous method*), 332
logpmf() (*rv_discrete method*), 335
logrv_beta_ecf_pdf() (*ctx method*), 532
logrv_beta_ecf_qtf() (*ctx method*), 532
logrv_chisquared_ecf() (*ctx method*), 502
logrv_chisquared_ecf_inv() (*ctx method*), 502
logseries_cdf() (*math53 method*), 2200
logseries_pmf() (*math53 method*), 2200
logsinh() (*math53 method*), 2162
logspace() (*npm method*), 1106
logsubexp() (*math53 method*), 2153
lommel_s1() (*ctx method*), 1485
lommel_s2() (*ctx method*), 1486
lowestvalue() (*ctx method*), 223
lrint() (*ctx method*), 212
lround() (*ctx method*), 214
lrt_vc0_bd() (*ctx method*), 700
lrt_vc0_bd_inv() (*ctx method*), 701
lrt_x0_vc0_bd() (*ctx method*), 705

lrt_x0_vc0_bd_inv() (*ctx method*), 705
lu() (*ctx method*), 1221
lu_solve() (*ctx method*), 1222
lucpoly() (*math53 method*), 2155
luggannini_rice() (*ctx method*), 900

M

m_elliptic_e() (*ctx method*), 1243
m_elliptic_e_inc() (*ctx method*), 1246
m_elliptic_f() (*ctx method*), 1245
m_elliptic_k() (*ctx method*), 1243
m_elliptic_pi() (*ctx method*), 1244
m_elliptic_pi_inc() (*ctx method*), 1247
m_x() (*dist_amoroso method*), 498
m_x() (*dist_arcsine method*), 338
m_x() (*dist_bennett method*), 781
m_x() (*dist_beta method*), 518
m_x() (*dist_beta_nc_type_I method*), 577
m_x() (*dist_beta_prime method*), 534
m_x() (*dist_beta_product method*), 656
m_x() (*dist_binomial method*), 762
m_x() (*dist_birnb_saunders method*), 439
m_x() (*dist_box_cov method*), 689
m_x() (*dist_box_mvn method*), 694
m_x() (*dist_box_nsame_cov method*), 685
m_x() (*dist_cauchy method*), 342
m_x() (*dist_chi method*), 470
m_x() (*dist_chi_nc method*), 581
m_x() (*dist_chi_squared method*), 474
m_x() (*dist_chi_squared_nc method*), 563
m_x() (*dist_dagum method*), 388
m_x() (*dist_dunnett1_t method*), 634
m_x() (*dist_dunnett2_t method*), 637
m_x() (*dist_emg method*), 442
m_x() (*dist_exponential method*), 345
m_x() (*dist_feller_pareto method*), 549
m_x() (*dist_fisher_f method*), 523
m_x() (*dist_fisher_f_2nc method*), 607
m_x() (*dist_fisher_f_nc method*), 572
m_x() (*dist_fisher_r2_type_I method*), 595
m_x() (*dist_fisher_z method*), 552
m_x() (*dist_fisk method*), 391
m_x() (*dist_folded_normal method*), 445
m_x() (*dist_frechet method*), 394
m_x() (*dist_friedman method*), 811
m_x() (*dist_gamma method*), 479
m_x() (*dist_gen_beta_exp method*), 546
m_x() (*dist_genbeta1 method*), 537
m_x() (*dist_genbeta2 method*), 540
m_x() (*dist_genlogistic method*), 543
m_x() (*dist_genpareto method*), 400
m_x() (*dist_geometric method*), 753
m_x() (*dist_gev method*), 398
m_x() (*dist_gompertz method*), 403
m_x() (*dist_gumbel method*), 349
m_x() (*dist_half_normal method*), 448
m_x() (*dist_halperin_t method*), 643
m_x() (*dist_hotelling_t2 method*), 713
m_x() (*dist_hyperexponential method*), 353
m_x() (*dist_hypergeometric method*), 773
m_x() (*dist_hypoexponential method*), 505
m_x() (*dist_invchisquared method*), 483
m_x() (*dist_invgamma method*), 487
m_x() (*dist_johnson_sb method*), 451
m_x() (*dist_johnson_su method*), 454
m_x() (*dist_jterpsta_s method*), 801
m_x() (*dist_kendall_tau method*), 796
m_x() (*dist_kruskal_wallis method*), 814
m_x() (*dist_kumaraswamy method*), 357
m_x() (*dist_landau method*), 730
m_x() (*dist_laplace method*), 361
m_x() (*dist_levy method*), 417
m_x() (*dist_lindley method*), 508
m_x() (*dist_logistic method*), 365
m_x() (*dist_lognormal method*), 420
m_x() (*dist_logrv_beta method*), 531
m_x() (*dist_logrv_beta_sum method*), 652
m_x() (*dist_logrv_chisquared method*), 501
m_x() (*dist_lomax method*), 406
m_x() (*dist_lrt_s0 method*), 699
m_x() (*dist_lrt_x0_s0 method*), 704
m_x() (*dist_mann_whitney_u method*), 785
m_x() (*dist_mann_whitney_u_lehmann method*), 789
m_x() (*dist_mann_whitney_u_milton method*), 792
m_x() (*dist_mauchley method*), 681
m_x() (*dist_maxwell method*), 491
m_x() (*dist_moyal method*), 424
m_x() (*dist_nair_t method*), 640
m_x() (*dist_nakagami method*), 495
m_x() (*dist_negbinom method*), 768
m_x() (*dist_nelson_h method*), 646
m_x() (*dist_nmax_0 method*), 457
m_x() (*dist_nmax_rho method*), 613
m_x() (*dist_nmax_rhoij method*), 619
m_x() (*dist_nmm_0 method*), 460
m_x() (*dist_nmm_rho method*), 616
m_x() (*dist_nmm_rhoij method*), 622
m_x() (*dist_normal method*), 428
m_x() (*dist_normal_range method*), 625
m_x() (*dist_page_l method*), 805
m_x() (*dist_page_l_nc_milton method*), 808
m_x() (*dist_pareto method*), 369
m_x() (*dist_pearson_rho method*), 559
m_x() (*dist_pearson_rho_nc method*), 587
m_x() (*dist_pillai_v method*), 710
m_x() (*dist_poisson method*), 757
m_x() (*dist_rayleigh method*), 373
m_x() (*dist_rice method*), 584

m_x() (*dist_roy method*), 707
m_x() (*dist_sasnormal method*), 463
m_x() (*dist_shifted_gompertz method*), 409
m_x() (*dist_singh_maddala method*), 412
m_x() (*dist_skewnormal method*), 432
m_x() (*dist_skewt method*), 556
m_x() (*dist_smax method*), 628
m_x() (*dist_smm method*), 631
m_x() (*dist_stacy method*), 514
m_x() (*dist_student_t method*), 528
m_x() (*dist_student_t_2nc method*), 604
m_x() (*dist_student_t_nc method*), 569
m_x() (*dist_studentized_range method*), 649
m_x() (*dist_triangular method*), 377
m_x() (*dist_truncated_normal method*), 466
m_x() (*dist_uniform method*), 381
m_x() (*dist_wald method*), 436
m_x() (*dist_weibull method*), 385
m_x() (*dist_wilcoxon method*), 777
m_x() (*dist_wilks_cs method*), 665
m_x() (*dist_wilks_iblocks method*), 677
m_x() (*dist_wilks_ip method*), 673
m_x() (*dist_wilks_lambda method*), 661
m_x() (*dist_wilks_lambda_corr method*), 720
m_x() (*dist_wilks_lambda_glm method*), 716
m_x() (*dist_wilks_lcm method*), 669
m_x() (*rv_base method*), 328
m_x() (*skewexponpower method*), 511
m_y() (*dist_logrv_beta_nc_type_II method*), 591
m_y() (*dist_logrv_fisher_1mr2 method*), 601
mannwhitney_ecf() (*ctx method*), 786
mannwhitney_ecf_inv() (*ctx method*), 786
mannwhitney_fm_pmf() (*ctx method*), 1524
mannwhitney_ft() (*ctx method*), 786
mannwhitney_nc_mu4() (*ctx method*), 1553
mannwhitney_spa() (*ctx method*), 786
mannwhitney_spa_inv() (*ctx method*), 787
mannwhitney_u_lehmann_pmf_vector() (*ctx method*), 1540
mannwhitney_u_milton_pmf_vector() (*ctx method*), 1541
mannwhitney_u_pmf_vector() (*ctx method*), 1539
MannWhitneyUPmfVector() (*math53lib method*), 2285
mapairy_cdf() (*math53 method*), 736
mapairy_pdf() (*math53 method*), 736
mapairy_qtf() (*math53 method*), 736
marcum_q() (*ctx method*), 1355
marcumq1() (*ctx method*), 1528
marcumq1_verified() (*ctx method*), 1532
mat_adjoint() (*ctx method*), 957
mat_as_diagonal() (*ctx method*), 956
mat_cod_householder_qr() (*ctx method*), 1008
mat_col_piv_householder_qr() (*ctx method*), 996
mat_cols() (*ctx method*), 949
mat_concat_horizontal() (*ctx method*), 961
mat_concat_vertical() (*ctx method*), 962
mat_conjugate() (*ctx method*), 957
mat_conservative_resize() (*ctx method*), 956
mat_correlation() (*ctx method*), 971
mat_coshm() (*ctx method*), 1078
mat_cosm() (*ctx method*), 1076
mat_count() (*ctx method*), 968
mat_covariance() (*ctx method*), 971
mat_eigen_system() (*ctx method*), 1058
mat_eigen_values() (*ctx method*), 1055
mat_expm() (*ctx method*), 1074
mat_fft_fwd() (*ctx method*), 1071
mat_fft_inv() (*ctx method*), 1071
mat_fill_linear() (*ctx method*), 948
mat_full_piv_householder_qr() (*ctx method*), 1002
mat_full_piv_lu() (*ctx method*), 983
mat_gen_eigen_system() (*ctx method*), 1068
mat_gen_eigen_values() (*ctx method*), 1066
mat_generalized_self_adjoint_eigen_system() (*ctx method*), 1040
mat_generalized_self_adjoint_eigen_values() (*ctx method*), 1037
mat_get_block() (*ctx method*), 950
mat_get_col() (*ctx method*), 951
mat_get_diagonal() (*ctx method*), 952
mat_get_row() (*ctx method*), 951
mat_get_triangular_view() (*ctx method*), 953
mat_hessenberg() (*ctx method*), 1050
mat_householder_qr() (*ctx method*), 992
mat_identity() (*ctx method*), 947
mat_jacobi_svd() (*ctx method*), 1015
mat_jacobi_svd_full() (*ctx method*), 1023
mat_jacobi_svd_thin() (*ctx method*), 1018
mat_ldlt() (*ctx method*), 972
mat_llt() (*ctx method*), 988
mat_logm() (*ctx method*), 1082
mat_max_coeff() (*ctx method*), 970
mat_mean() (*ctx method*), 970
mat_min_coeff() (*ctx method*), 970
mat_norm() (*ctx method*), 971
mat_ones() (*ctx method*), 947
mat_partial_piv_lu() (*ctx method*), 978
mat_powm() (*ctx method*), 1084
mat_prod() (*ctx method*), 970
mat_pseudo_eigen_system() (*ctx method*), 1064
mat_random() (*ctx method*), 947
mat_random_selfadjoint() (*ctx method*), 948
mat_random_selfadjoint_posdef() (*ctx method*), 948
mat_random_symmetric() (*ctx method*), 947
mat_real_qz() (*ctx method*), 1062
mat_replicate_col_wise() (*ctx method*), 961
mat_replicate_full() (*ctx method*), 960
mat_replicate_row_wise() (*ctx method*), 960
mat_resize() (*ctx method*), 955
mat_resize_like() (*ctx method*), 955

mat_reverse_col_wise() (*ctx method*), 959
mat_reverse_full() (*ctx method*), 959
mat_reverse_row_wise() (*ctx method*), 959
mat_rows() (*ctx method*), 949
mat_schur() (*ctx method*), 1053
mat_self_adjoint_eigen_system() (*ctx method*), 1032
mat_self_adjoint_eigen_values() (*ctx method*), 1029
mat_set_block() (*ctx method*), 950
mat_set_col() (*ctx method*), 952
mat_set_diagonal() (*ctx method*), 952
mat_set_row() (*ctx method*), 951
mat_set_triangular_view() (*ctx method*), 953
mat_sinhm() (*ctx method*), 1077
mat_sinm() (*ctx method*), 1075
mat_size() (*ctx method*), 949
mat_sort() (*ctx method*), 954
mat_sort_rows_by_col() (*ctx method*), 954
mat_sqrtm() (*ctx method*), 1080
mat_squared_norm() (*ctx method*), 971
mat_stable_norm() (*ctx method*), 971
mat_sum() (*ctx method*), 970
mat_t() (*ctx method*), 946
mat_trace() (*ctx method*), 970
mat_transpose() (*ctx method*), 958
mat_tridiagonalization() (*ctx method*), 1045
mat_zeros() (*ctx method*), 946
matmul() (*npm method*), 1174
mauchley_bd() (*ctx method*), 683
mauchley_bd_inv() (*ctx method*), 683
max() (*npm method*), 1161
maximum() (*npm method*), 1161
maxvalue() (*ctx method*), 222
maxwell_cdf() (*math53 method*), 489
maxwell_pdf() (*math53 method*), 489
maxwell_qtf() (*math53 method*), 489
mbrent() (*ctx method*), 824
md_newton_c() (*ctx method*), 838
mean() (*dist_bernoulli method*), 748
mean() (*dist_holtsmark method*), 733
mean() (*dist_kolmogorov_smirnov method*), 725
mean() (*dist_mapairy method*), 738
mean() (*dist_saspoint5 method*), 743
mean() (*npm method*), 1169
mean() (*rv_base method*), 324
median() (*dist_bernoulli method*), 748
median() (*dist_holtsmark method*), 733
median() (*dist_kolmogorov_smirnov method*), 725
median() (*dist_mapairy method*), 738
median() (*dist_saspoint5 method*), 743
median() (*npm method*), 1168
median() (*rv_base method*), 324
mgf_from_cf() (*ctx method*), 1495
mgf_from_cgf() (*ctx method*), 1495
mgf_from_pdf() (*ctx method*), 1495
mgf_from_pgf() (*ctx method*), 1495
mgf_from_rawmoments() (*ctx method*), 1495
min() (*npm method*), 1162
minimum() (*npm method*), 1161
minposvalue() (*ctx method*), 223
mnewton_c() (*ctx method*), 837
mnorm() (*ctx method*), 1219
mode() (*dist_bernoulli method*), 748
mode() (*dist_holtsmark method*), 733
mode() (*dist_kolmogorov_smirnov method*), 724
mode() (*dist_mapairy method*), 738
mode() (*dist_saspoint5 method*), 743
mode() (*rv_base method*), 324
modf() (*ctx method*), 219
moebius() (*math53 method*), 2199
moments() (*dist_amoroso method*), 498
moments() (*dist_arcsine method*), 339
moments() (*dist_bennett method*), 782
moments() (*dist_beta method*), 519
moments() (*dist_beta_nc_type_I method*), 578
moments() (*dist_beta_prime method*), 534
moments() (*dist_beta_product method*), 657
moments() (*dist_binomial method*), 763
moments() (*dist_birnb_saunders method*), 439
moments() (*dist_box_cov method*), 690
moments() (*dist_box_mvn method*), 695
moments() (*dist_box_nsame_cov method*), 686
moments() (*dist_cauchy method*), 342
moments() (*dist_chi method*), 470
moments() (*dist_chi_nc method*), 581
moments() (*dist_chi_squared method*), 474
moments() (*dist_chi_squared_nc method*), 564
moments() (*dist_dagum method*), 388
moments() (*dist_dunnett1_t method*), 635
moments() (*dist_dunnett2_t method*), 637
moments() (*dist_emg method*), 443
moments() (*dist_exponential method*), 345
moments() (*dist_feller_pareto method*), 550
moments() (*dist_fisher_f method*), 523
moments() (*dist_fisher_f_2nc method*), 607
moments() (*dist_fisher_f_nc method*), 573
moments() (*dist_fisher_r2_type_I method*), 596
moments() (*dist_fisher_z method*), 553
moments() (*dist_fisk method*), 391
moments() (*dist_folded_normal method*), 446
moments() (*dist_frechet method*), 394
moments() (*dist_friedman method*), 812
moments() (*dist_gamma method*), 480
moments() (*dist_gen_beta_exp method*), 547
moments() (*dist_genbeta1 method*), 537
moments() (*dist_genbeta2 method*), 540
moments() (*dist_genlogistic method*), 544
moments() (*dist_genpareto method*), 400
moments() (*dist_geometric method*), 753

moments() (*dist_gev method*), 398
moments() (*dist_gompertz method*), 404
moments() (*dist_gumbel method*), 349
moments() (*dist_half_normal method*), 449
moments() (*dist_halperin_t method*), 643
moments() (*dist_hotelling_t2 method*), 713
moments() (*dist_hyperexponential method*), 354
moments() (*dist_hypergeometric method*), 773
moments() (*dist_hypoexponential method*), 505
moments() (*dist_invchisquared method*), 483
moments() (*dist_invgamma method*), 487
moments() (*dist_johnson_sb method*), 452
moments() (*dist_johnson_su method*), 455
moments() (*dist_jterpsta_s method*), 802
moments() (*dist_kendall_tau method*), 797
moments() (*dist_kruskal_wallis method*), 815
moments() (*dist_kumaraswamy method*), 357
moments() (*dist_landau method*), 730
moments() (*dist_laplace method*), 361
moments() (*dist_levy method*), 417
moments() (*dist_lindley method*), 509
moments() (*dist_logistic method*), 365
moments() (*dist_lognormal method*), 420
moments() (*dist_logrv_beta method*), 532
moments() (*dist_logrv_beta_nc_type_II method*), 592
moments() (*dist_logrv_beta_sum method*), 653
moments() (*dist_logrv_chisquared method*), 502
moments() (*dist_logrv_fisher_1mr2 method*), 601
moments() (*dist_lomax method*), 406
moments() (*dist_lrt_s0 method*), 700
moments() (*dist_lrt_x0_s0 method*), 704
moments() (*dist_mann_whitney_u method*), 786
moments() (*dist_mann_whitney_u_lehmann method*), 790
moments() (*dist_mann_whitney_u_milton method*), 793
moments() (*dist_mauchley method*), 682
moments() (*dist_maxwell method*), 492
moments() (*dist_moyal method*), 424
moments() (*dist_nair_t method*), 640
moments() (*dist_nakagami method*), 496
moments() (*dist_negbinom method*), 768
moments() (*dist_nelson_h method*), 646
moments() (*dist_nmax_0 method*), 458
moments() (*dist_nmax_rho method*), 614
moments() (*dist_nmax_rhoij method*), 620
moments() (*dist_nmm_0 method*), 461
moments() (*dist_nmm_rho method*), 616
moments() (*dist_nmm_rhoij method*), 622
moments() (*dist_normal method*), 429
moments() (*dist_normal_range method*), 626
moments() (*dist_page_l method*), 805
moments() (*dist_page_l_nc_milton method*), 809
moments() (*dist_pareto method*), 369
moments() (*dist_pearson_rho method*), 560
moments() (*dist_pearson_rho_nc method*), 588
moments() (*dist_pillai_v method*), 710
moments() (*dist_poisson method*), 758
moments() (*dist_rayleigh method*), 373
moments() (*dist_rice method*), 584
moments() (*dist_roy method*), 708
moments() (*dist_sasnormal method*), 464
moments() (*dist_shifted_gompertz method*), 410
moments() (*dist_singh_maddala method*), 412
moments() (*dist_skewnormal method*), 432
moments() (*dist_skewt method*), 556
moments() (*dist_smax method*), 628
moments() (*dist_smm method*), 631
moments() (*dist_stacy method*), 514
moments() (*dist_student_t method*), 528
moments() (*dist_student_t_2nc method*), 604
moments() (*dist_student_t_nc method*), 569
moments() (*dist_studentized_range method*), 650
moments() (*dist_triangular method*), 378
moments() (*dist_truncated_normal method*), 467
moments() (*dist_uniform method*), 381
moments() (*dist_wald method*), 436
moments() (*dist_weibull method*), 385
moments() (*dist_wilcoxon method*), 778
moments() (*dist_wilks_cs method*), 666
moments() (*dist_wilks_iblocks method*), 678
moments() (*dist_wilks_ip method*), 674
moments() (*dist_wilks_lambda method*), 661
moments() (*dist_wilks_lambda_corr method*), 721
moments() (*dist_wilks_lambda_glm method*), 717
moments() (*dist_wilks_lcm method*), 670
moments() (*skewexponpower method*), 512
moveaxis() (*npm method*), 1124
moyal_cdf() (*math53 method*), 422
moyal_pdf() (*math53 method*), 422
moyal_qtf() (*math53 method*), 422
muller() (*ctx method*), 835
multiply() (*npm method*), 1165

N

NairCdf() (*math53lib method*), 2282
NairDist() (*math53lib method*), 2282
NairPdf() (*math53lib method*), 2282
NairQtf() (*math53lib method*), 2282
nakagami_cdf() (*math53 method*), 493
nakagami_pdf() (*math53 method*), 493
nakagami_qtf() (*math53 method*), 493
nan() (*ctx method*), 207
nc_ci() (*dist_fisher_f_2nc method*), 608
nc_ci() (*dist_fisher_f_nc method*), 573
ndens() (*ctx method*), 1421
ndis() (*ctx method*), 1422
ndisx() (*math53 method*), 1422
nearbyint() (*ctx method*), 212
negative() (*npm method*), 1164

negative_binomial_pmf_vector() (*ctx method*), 1536
negbinom_ecf() (*ctx method*), 769
negbinom_ecf_inv() (*ctx method*), 769
negbinom_spa() (*ctx method*), 769
negbinom_spa_inv() (*ctx method*), 769
negbinomial_cdf() (*math53 method*), 765
negbinomial_pmf() (*math53 method*), 765
negbinomial_qtf() (*math53 method*), 765
neginf() (*ctx method*), 206
negzero (*ctx property*), 205
NelderMead() (*ctxboost method*), 932
neville_theta_c() (*math53 method*), 2189
neville_theta_d() (*math53 method*), 2189
neville_theta_n() (*math53 method*), 2189
neville_theta_s() (*math53 method*), 2190
newton_c() (*ctx method*), 836
newton_steffenson_c() (*ctx method*), 836
NewtonDescent() (*ctxboost method*), 944
NewtonRaphson() (*ctxboost method*), 828
nextabove() (*ctx method*), 224
nextbelow() (*ctx method*), 224
nexttowards() (*ctx method*), 223
NmaxRhoCdf() (*math53lib method*), 2271
NmaxRhoDist() (*math53lib method*), 2271
NmaxRhoijCdf() (*math53lib method*), 2274
NmaxRhoijDist() (*math53lib method*), 2274
NmaxRhoijPdf() (*math53lib method*), 2274
NmaxRhoijQtf() (*math53lib method*), 2274
NmaxRhoPdf() (*math53lib method*), 2271
NmaxRhoQtf() (*math53lib method*), 2271
NmmRhoCdf() (*math53lib method*), 2273
NmmRhoDist() (*math53lib method*), 2273
NmmRhoijCdf() (*math53lib method*), 2275
NmmRhoijDist() (*math53lib method*), 2275
NmmRhoijPdf() (*math53lib method*), 2275
NmmRhoijQtf() (*math53lib method*), 2275
NmmRhoPdf() (*math53lib method*), 2273
NmmRhoQtf() (*math53lib method*), 2273
nnlf() (*rv_continuous method*), 332
nonzero() (*npm method*), 1153
norm() (*ctx method*), 1218
Norm() (*mat method*), 1216
normal_cdf() (*math53 method*), 426
normal_pdf() (*math53 method*), 426
normal_qtf() (*math53 method*), 426
not_equal() (*npm method*), 1185
nprod() (*ctx method*), 914
NrangeCdf() (*math53lib method*), 2277
NrangeDist() (*math53lib method*), 2277
NrangePdf() (*math53lib method*), 2277
NrangeQtf() (*math53lib method*), 2277
nsum() (*ctx method*), 907

O

odefun() (*ctx method*), 894
one (*Cxt property*), 205
onej (*ctx property*), 206
ones() (*npm method*), 1089
ones_like() (*npm method*), 1089
OouraCos() (*ctxboost method*), 854
OouraSin() (*ctxboost method*), 855
outer() (*npm method*), 1173
owen_t() (*ctx method*), 1430
owent() (*ctx method*), 1529
owent_verified() (*ctx method*), 1533

P

pad() (*npm method*), 1139
pade() (*ctx method*), 904
page_ecf() (*ctx method*), 806
page_ecf_inv() (*ctx method*), 806
page_l_pmf_vector() (*ctx method*), 1540
page_spa() (*ctx method*), 806
page_spa_inv() (*ctx method*), 806
PageLPmfVector() (*math53lib method*), 2286
pareto_cdf() (*math53 method*), 367
pareto_pdf() (*math53 method*), 367
pareto_qtf() (*math53 method*), 367
pdf() (*dist_amoroso method*), 497
pdf() (*dist_arcsine method*), 337
pdf() (*dist_beta method*), 517
pdf() (*dist_beta_nc_type_I method*), 576
pdf() (*dist_beta_prime method*), 533
pdf() (*dist_beta_product method*), 655
pdf() (*dist_birnb_saunders method*), 437
pdf() (*dist_box_cov method*), 688
pdf() (*dist_box_mvn method*), 693
pdf() (*dist_box_nsame_cov method*), 684
pdf() (*dist_cauchy method*), 341
pdf() (*dist_chi method*), 469
pdf() (*dist_chi_nc method*), 580
pdf() (*dist_chi_squared method*), 473
pdf() (*dist_chi_squared_nc method*), 562
pdf() (*dist_dagum method*), 387
pdf() (*dist_dunnett1_t method*), 633
pdf() (*dist_dunnett2_t method*), 636
pdf() (*dist_emg method*), 441
pdf() (*dist_exponential method*), 343
pdf() (*dist_feller_pareto method*), 548
pdf() (*dist_fisher_f method*), 522
pdf() (*dist_fisher_f_2nc method*), 606
pdf() (*dist_fisher_f_nc method*), 571
pdf() (*dist_fisher_r2_type_I method*), 594
pdf() (*dist_fisher_z method*), 551
pdf() (*dist_fisk method*), 390
pdf() (*dist_folded_normal method*), 444
pdf() (*dist_frechet method*), 393

pdf() (*dist_gamma method*), 478
pdf() (*dist_gen_beta_exp method*), 545
pdf() (*dist_genbeta1 method*), 536
pdf() (*dist_genbeta2 method*), 539
pdf() (*dist_genlogistic method*), 542
pdf() (*dist_genpareto method*), 399
pdf() (*dist_gev method*), 396
pdf() (*dist_gompertz method*), 402
pdf() (*dist_gumbel method*), 348
pdf() (*dist_half_normal method*), 447
pdf() (*dist_halperin_t method*), 642
pdf() (*dist_holtsmark method*), 732
pdf() (*dist_hotelling_t2 method*), 712
pdf() (*dist_hyperexponential method*), 352
pdf() (*dist_hypoexponential method*), 503
pdf() (*dist_invchisquared method*), 482
pdf() (*dist_invgamma method*), 486
pdf() (*dist_johnson_sb method*), 450
pdf() (*dist_johnson_su method*), 453
pdf() (*dist_kolmogorov_smirnov method*), 724
pdf() (*dist_kumaraswamy method*), 356
pdf() (*dist_landau method*), 729
pdf() (*dist_laplace method*), 359
pdf() (*dist_levy method*), 416
pdf() (*dist_lindley method*), 507
pdf() (*dist_logistic method*), 363
pdf() (*dist_lognormal method*), 418
pdf() (*dist_logrv_beta method*), 530
pdf() (*dist_logrv_beta_nc_type_II method*), 590
pdf() (*dist_logrv_beta_sum method*), 651
pdf() (*dist_logrv_chisquared method*), 500
pdf() (*dist_logrv_fisher_1mr2 method*), 599
pdf() (*dist_lomax method*), 405
pdf() (*dist_lrt_s0 method*), 697
pdf() (*dist_lrt_x0_s0 method*), 702
pdf() (*dist_mapairy method*), 737
pdf() (*dist_mauchley method*), 680
pdf() (*dist_maxwell method*), 490
pdf() (*dist_moyal method*), 423
pdf() (*dist_nair_t method*), 639
pdf() (*dist_nakagami method*), 494
pdf() (*dist_nelson_h method*), 645
pdf() (*dist_nmax_0 method*), 456
pdf() (*dist_nmax_rho method*), 612
pdf() (*dist_nmax_rhoij method*), 618
pdf() (*dist_nmm_0 method*), 459
pdf() (*dist_nmm_rho method*), 615
pdf() (*dist_nmm_rhoij method*), 621
pdf() (*dist_normal method*), 427
pdf() (*dist_normal_range method*), 624
pdf() (*dist_pareto method*), 367
pdf() (*dist_pearson_rho method*), 558
pdf() (*dist_pearson_rho_nc method*), 586
pdf() (*dist_pillai_v method*), 709
pdf() (*dist_rayleigh method*), 371
pdf() (*dist_rice method*), 583
pdf() (*dist_roy method*), 706
pdf() (*dist_sasnormal method*), 462
pdf() (*dist_saspoint5 method*), 742
pdf() (*dist_shifted_gompertz method*), 408
pdf() (*dist_singh_maddala method*), 411
pdf() (*dist_skewnormal method*), 430
pdf() (*dist_skewt method*), 555
pdf() (*dist_smax method*), 627
pdf() (*dist_smm method*), 630
pdf() (*dist_stacy method*), 513
pdf() (*dist_student_t method*), 527
pdf() (*dist_student_t_2nc method*), 603
pdf() (*dist_student_t_nc method*), 567
pdf() (*dist_studentized_range method*), 648
pdf() (*dist_triangular method*), 376
pdf() (*dist_truncated_normal method*), 465
pdf() (*dist_uniform method*), 379
pdf() (*dist_wald method*), 435
pdf() (*dist_weibull method*), 383
pdf() (*dist_wilks_cs method*), 664
pdf() (*dist_wilks_iblocks method*), 676
pdf() (*dist_wilks_ip method*), 672
pdf() (*dist_wilks_lambda method*), 660
pdf() (*dist_wilks_lambda_corr method*), 719
pdf() (*dist_wilks_lambda_glm method*), 715
pdf() (*dist_wilks_lcm method*), 668
pdf() (*rv_continuous method*), 331
pdf() (*skewexponpower method*), 510
pdf_from_cdf() (*ctx method*), 1488
pdf_from_cf() (*ctx method*), 1488
pdf_from_cf_boost() (*ctx method*), 855
pearson_rho_ci() (*ctx method*), 2335
pearson_rho_nc_gt_cdf() (*ctx method*), 1519
pearson_rho_nc_ht_cdf() (*ctx method*), 1518
pearson_rho_nc_owen_cdf() (*ctx method*), 1509
pearson_rho_nc_owen_pdf() (*ctx method*), 1508
pearson_rho_power() (*ctx method*), 2337
pearson_rho_samplesize() (*ctx method*), 2339
pearson_rho_test() (*ctx method*), 2333
pearson_rho_wb_cdf() (*ctx method*), 1547
pearson_rho_wb_cl() (*ctx method*), 1548
pearson_rho_wb_pdf() (*ctx method*), 1547
pearson_rho_wb_qtf() (*ctx method*), 1547
pentagamma() (*math53 method*), 2191
pgf() (*rv_discrete method*), 333
pgf_from_mgf() (*ctx method*), 1497
pgf_from_pmf_vector() (*ctx method*), 1497
phase() (*ctx method*), 230
phi (*ctx property*), 225
pi (*ctx property*), 226
pillai_v_bd() (*ctx method*), 711
pillai_v_bd_inv() (*ctx method*), 711

pillai_v_ecf() (*ctx method*), 711
pillai_v_ecf_inv() (*ctx method*), 711
pillai_v_glm_chi2_cdf() (*ctx method*), 1574
pillai_v_glm_mu2_cdf() (*ctx method*), 1581
pillai_v_ind_chi2_cdf() (*ctx method*), 1575
pillai_v_ind_mu2_cdf() (*ctx method*), 1581
pillay_v_mu3_cdf() (*ctx method*), 1568
pillay_v_mu3_qtf() (*ctx method*), 1568
plot (*gui property*), 94
pmf() (*dist_bennett method*), 780
pmf() (*dist_bernoulli method*), 747
pmf() (*dist_binomial method*), 761
pmf() (*dist_friedman method*), 810
pmf() (*dist_geometric method*), 751
pmf() (*dist_hypergeometric method*), 771
pmf() (*dist_jterpsta_s method*), 799
pmf() (*dist_kendall_tau method*), 794
pmf() (*dist_kruskal_wallis method*), 813
pmf() (*dist_mann_whitney_u method*), 783
pmf() (*dist_mann_whitney_u_lehmann method*), 788
pmf() (*dist_mann_whitney_u_milton method*), 791
pmf() (*dist_negbinom method*), 766
pmf() (*dist_page_l method*), 804
pmf() (*dist_page_l_nc_milton method*), 807
pmf() (*dist_poisson method*), 756
pmf() (*dist_wilcoxon method*), 775
pmf() (*rv_discrete method*), 333
pmf_from_cdf() (*ctx method*), 1489
pmf_from_cf_lattice() (*ctx method*), 1489
pmf_from_factorialmoments() (*ctx method*), 1489
pmfvec() (*rv_discrete method*), 334
poch1() (*math53 method*), 2167
poisson_cdf() (*math53 method*), 755
poisson_ecf() (*ctx method*), 758
poisson_ecf_inv() (*ctx method*), 758
poisson_pmf() (*math53 method*), 755
poisson_pmf_vector() (*ctx method*), 1535
poisson_qtf() (*math53 method*), 755
poisson_spa() (*ctx method*), 759
poisson_spa_inv() (*ctx method*), 759
polar() (*ctx method*), 231
poly_eval() (*ctx method*), 1070
poly_roots() (*ctx method*), 1070
polygamma() (*ctx method*), 1295
polylog() (*ctx method*), 1301
polylog_i() (*math53 method*), 2193
polyroots() (*ctx method*), 925
polyval() (*ctx method*), 925
positive() (*npm method*), 1164
pow() (*ctx method*), 253
pow1p() (*ctx method*), 255
pow1pm1() (*ctx method*), 256
pow_si() (*ctx method*), 252
PowellHybrid() (*ctxboost method*), 927
powm1() (*ctx method*), 254
primezeta() (*math53 method*), 2198
prod() (*npm method*), 1155
psf() (*rv_continuous method*), 331
psi() (*ctx method*), 1299
psi_inv() (*math53 method*), 2192
psistar() (*math53 method*), 2191

Q

qagi() (*ctx method*), 843
qags() (*ctx method*), 842
qawc() (*ctx method*), 844
qr() (*ctx method*), 1223
qr_solve() (*ctx method*), 1224
qtf() (*dist_amoroso method*), 497
qtf() (*dist_arcsine method*), 337
qtf() (*dist_bennett method*), 780
qtf() (*dist_bernoulli method*), 747
qtf() (*dist_beta method*), 517
qtf() (*dist_beta_nc_type_I method*), 577
qtf() (*dist_beta_prime method*), 534
qtf() (*dist_beta_product method*), 655
qtf() (*dist_binomial method*), 762
qtf() (*dist_birnb_saunders method*), 438
qtf() (*dist_box_cov method*), 689
qtf() (*dist_box_mvn method*), 694
qtf() (*dist_box_nsame_cov method*), 685
qtf() (*dist_cauchy method*), 341
qtf() (*dist_chi method*), 469
qtf() (*dist_chi_nc method*), 580
qtf() (*dist_chi_squared method*), 473
qtf() (*dist_chi_squared_nc method*), 563
qtf() (*dist_dagum method*), 387
qtf() (*dist_dunnett1_t method*), 634
qtf() (*dist_dunnett2_t method*), 637
qtf() (*dist_emg method*), 441
qtf() (*dist_exponential method*), 344
qtf() (*dist_feller_pareto method*), 549
qtf() (*dist_fisher_f method*), 523
qtf() (*dist_fisher_f_2nc method*), 607
qtf() (*dist_fisher_f_nc method*), 572
qtf() (*dist_fisher_r2_type_I method*), 595
qtf() (*dist_fisher_z method*), 552
qtf() (*dist_fisk method*), 390
qtf() (*dist_folded_normal method*), 444
qtf() (*dist_frechet method*), 393
qtf() (*dist_friedman method*), 810
qtf() (*dist_gamma method*), 478
qtf() (*dist_gen_beta_exp method*), 545
qtf() (*dist_genbeta1 method*), 536
qtf() (*dist_genbeta2 method*), 539
qtf() (*dist_genlogistic method*), 543
qtf() (*dist_genpareto method*), 399
qtf() (*dist_geometric method*), 752

qtf() (*dist_gev method*), 397
qtf() (*dist_gompertz method*), 402
qtf() (*dist_gumbel method*), 348
qtf() (*dist_half_normal method*), 447
qtf() (*dist_halperin_t method*), 643
qtf() (*dist_holtsmark method*), 732
qtf() (*dist_hotelling_t2 method*), 712
qtf() (*dist_hyperexponential method*), 352
qtf() (*dist_hypergeometric method*), 772
qtf() (*dist_hypoexponential method*), 504
qtf() (*dist_invchisquared method*), 482
qtf() (*dist_invgamma method*), 486
qtf() (*dist_johnson_sb method*), 450
qtf() (*dist_johnson_su method*), 453
qtf() (*dist_jterpsta_s method*), 800
qtf() (*dist_kendall_tau method*), 795
qtf() (*dist_kolmogorov_smirnov method*), 724
qtf() (*dist_kruskal_wallis method*), 813
qtf() (*dist_kumaraswamy method*), 356
qtf() (*dist_landau method*), 729
qtf() (*dist_laplace method*), 360
qtf() (*dist_levy method*), 416
qtf() (*dist_lindley method*), 507
qtf() (*dist_logistic method*), 364
qtf() (*dist_lognormal method*), 419
qtf() (*dist_logrv_beta method*), 530
qtf() (*dist_logrv_beta_nc_type_II method*), 591
qtf() (*dist_logrv_beta_sum method*), 652
qtf() (*dist_logrv_chisquared method*), 500
qtf() (*dist_logrv_fisher_1mr2 method*), 600
qtf() (*dist_lomax method*), 405
qtf() (*dist_lrt_s0 method*), 698
qtf() (*dist_lrt_x0_s0 method*), 703
qtf() (*dist_mann_whitney_u method*), 784
qtf() (*dist_mann_whitney_u_lehmann method*), 788
qtf() (*dist_mann_whitney_u_milton method*), 791
qtf() (*dist_mapairy method*), 737
qtf() (*dist_mauchley method*), 681
qtf() (*dist_maxwell method*), 490
qtf() (*dist_moyal method*), 423
qtf() (*dist_nair_t method*), 640
qtf() (*dist_nakagami method*), 494
qtf() (*dist_negbinom method*), 767
qtf() (*dist_nelson_h method*), 646
qtf() (*dist_nmax_0 method*), 457
qtf() (*dist_nmax_rho method*), 612
qtf() (*dist_nmax_rhoij method*), 618
qtf() (*dist_nmm_0 method*), 460
qtf() (*dist_nmm_rho method*), 615
qtf() (*dist_nmm_rhoij method*), 621
qtf() (*dist_normal method*), 427
qtf() (*dist_normal_range method*), 624
qtf() (*dist_page_l method*), 804
qtf() (*dist_page_l_nc_milton method*), 807
qtf() (*dist_pareto method*), 368
qtf() (*dist_pearson_rho method*), 558
qtf() (*dist_pearson_rho_nc method*), 587
qtf() (*dist_pillai_v method*), 709
qtf() (*dist_poisson method*), 756
qtf() (*dist_rayleigh method*), 372
qtf() (*dist_rice method*), 583
qtf() (*dist_roy method*), 706
qtf() (*dist_sasnormal method*), 462
qtf() (*dist_saspoint5 method*), 742
qtf() (*dist_shifted_gompertz method*), 408
qtf() (*dist_singh_maddala method*), 411
qtf() (*dist_skewnormal method*), 431
qtf() (*dist_skewt method*), 555
qtf() (*dist_smax method*), 628
qtf() (*dist_smm method*), 631
qtf() (*dist_stacy method*), 513
qtf() (*dist_student_t method*), 527
qtf() (*dist_student_t_2nc method*), 604
qtf() (*dist_student_t_nc method*), 568
qtf() (*dist_studentized_range method*), 649
qtf() (*dist_triangular method*), 377
qtf() (*dist_truncated_normal method*), 465
qtf() (*dist_uniform method*), 380
qtf() (*dist_wald method*), 435
qtf() (*dist_weibull method*), 384
qtf() (*dist_wilcoxon method*), 776
qtf() (*dist_wilks_cs method*), 665
qtf() (*dist_wilks_iblocks method*), 677
qtf() (*dist_wilks_ip method*), 673
qtf() (*dist_wilks_lambda method*), 660
qtf() (*dist_wilks_lambda_corr method*), 719
qtf() (*dist_wilks_lambda_glm method*), 715
qtf() (*dist_wilks_lcm method*), 669
qtf() (*rv_base method*), 323
qtf() (*skewexponpower method*), 510
quad() (*ctx method*), 858
quad_bellshaped(), 1527
quad_erf() (*ctx method*), 1527
quad_erf_verified() (*ctx method*), 1531
quad_gamma_lower() (*ctx method*), 1527
quad_gamma_lower_verified() (*ctx method*), 1531
quad_gamma_upper() (*ctx method*), 1527
quad_gamma_upper_verified() (*ctx method*), 1531
quad_ibeta() (*ctx method*), 1528
quad_ibeta_verified() (*ctx method*), 1531
quad_verified(), 864
quade_l_pmf_vector() (*ctx method*), 1540
quade_s_pmf_vector() (*ctx method*), 1542
quade_s_pmf_vector() (*math53lib method*), 2288
QuadeLPmfVector() (*math53lib method*), 2287
quadosc() (*ctx method*), 862
quadratic_equation() (*ctx method*), 2263
quadsubdiv() (*ctx method*), 861

quanc8() (*ctx method*), 841
quartic_equation() (*ctx method*), 2266

R

r_sy_tridiag(), 1231
range_lower_endpoint() (*Dist method*), 327
range_lower_endpoint() (*dist_bernoulli method*), 749
range_lower_endpoint() (*dist_holtsmark method*), 734
range_lower_endpoint() (*dist_kolmogorov_smirnov method*), 726
range_lower_endpoint() (*dist_mapairy method*), 739
range_lower_endpoint() (*dist_saspoint5 method*), 744
range_upper_endpoint() (*Dist method*), 327
range_upper_endpoint() (*dist_bernoulli method*), 750
range_upper_endpoint() (*dist_holtsmark method*), 735
range_upper_endpoint() (*dist_kolmogorov_smirnov method*), 726
range_upper_endpoint() (*dist_mapairy method*), 740
range_upper_endpoint() (*dist_saspoint5 method*), 745
ravel() (*npm method*), 1121
raw_moments() (*rv_base method*), 329
rawadd() (*ctx method*), 201
rawaddint32() (*ctx method*), 202
rawdiv() (*ctx method*), 202
rawdivint32() (*ctx method*), 203
rawint32div() (*ctx method*), 203
rawint32sub() (*ctx method*), 202
rawmoments_from_centralmoments() (*ctx method*), 1499
rawmoments_from_cf() (*ctx method*), 1500
rawmoments_from_cumulants() (*ctx method*), 1500
rawmoments_from_factorial_moments() (*ctx method*), 1499
rawmoments_from_mgf() (*ctx method*), 1500
rawmoments_from_pdf() (*ctx method*), 1499
rawmoments_from_pgf() (*ctx method*), 1500
rawmoments_from_pmf_vector() (*ctx method*), 1499
rawmul() (*ctx method*), 202
rawmulint32() (*ctx method*), 203
rawsub() (*ctx method*), 201
rawsubint32() (*ctx method*), 202
rayleigh_cdf() (*math53 method*), 371
rayleigh_pdf() (*math53 method*), 371
rayleigh_qtf() (*math53 method*), 371
real() (*ctx method*), 230
reciprocal() (*npm method*), 1167
rect() (*ctx method*), 231
recurrence_cdf() (*dist_fisher_f method*), 524
recurrence_pdf() (*dist_fisher_f method*), 524
remainder() (*ctx method*), 220
remainder() (*npm method*), 1166
remquo() (*ctx method*), 220
repeat() (*npm method*), 1138
reshape() (*npm method*), 1120
residual() (*ctx method*), 1222
rgamma() (*ctx method*), 303
richardson() (*ctx method*), 918
ridder() (*ctx method*), 835
riemann_r() (*math53 method*), 2198
riemann_r_inv() (*math53 method*), 2198
riemann_xi() (*ctx method*), 1332
rint() (*ctx method*), 212
rising_factorial() (*ctx method*), 307
rogers_ramanujan_cf() (*math53 method*), 2198
roll() (*npm method*), 1145
root_si() (*ctx method*), 236
roots_to_monic_poly() (*ctx method*), 1070
rot90() (*npm method*), 1146
round() (*ctx method*), 214
roy_chiani_cdf() (*ctx method*), 1559
roy_chiani_pdf() (*ctx method*), 1558
roy_chiani_qtf() (*ctx method*), 1560
roy_glm_mu2_cdf() (*ctx method*), 1581
roy_ind_mu2_cdf() (*ctx method*), 1582
roy_pdf_cdf_sf() (*ctx method*), 1511
rsqrt() (*ctx method*), 233
RungeKutta4Const() (*ctxboost method*), 877
rv_base (*built-in class*), 322
rv_continuous (*built-in class*), 331
rv_discrete (*built-in class*), 333

S

saspoint5_cdf() (*math53 method*), 741
saspoint5_pdf() (*math53 method*), 741
saspoint5_qtf() (*math53 method*), 741
scalbln() (*ctx method*), 217
scalbn() (*ctx method*), 217
Schroeder() (*ctxboost method*), 830
schur() (*ctx method*), 1235
scorer_gi() (*ctx method*), 1475
scorer_gi_prime() (*ctx method*), 1477
scorer_hi() (*ctx method*), 1476
scorer_hi_prime() (*ctx method*), 1478
sec() (*ctx method*), 265
secant() (*ctx method*), 833
sech() (*ctx method*), 279
secpi() (*ctx method*), 270
select() (*npm method*), 1154
set_cdf_method() (*rv_base method*), 322
set_central_moments_method() (*rv_base method*), 329
set_cf_method() (*rv_base method*), 327
set_cgf_method() (*rv_base method*), 328
set_cumulants_method() (*rv_base method*), 330
set_mgf_method() (*rv_base method*), 328
set_moments_method() (*rv_base method*), 329
set_pdf_method() (*rv_base method*), 331
set_pgf_method() (*rv_base method*), 334
set_pmf_method() (*rv_base method*), 333
set_qtf_method() (*rv_base method*), 323

setdps() (*gui method*), 93
sf() (*dist_amoroso method*), 497
sf() (*dist_arcsine method*), 337
sf() (*dist_bennett method*), 780
sf() (*dist_bernoulli method*), 747
sf() (*dist_beta method*), 517
sf() (*dist_beta_nc_type_I method*), 576
sf() (*dist_beta_prime method*), 533
sf() (*dist_beta_product method*), 655
sf() (*dist_binomial method*), 761
sf() (*dist_birnb_saunders method*), 438
sf() (*dist_box_cov method*), 688
sf() (*dist_box_mvn method*), 693
sf() (*dist_box_nsame_cov method*), 684
sf() (*dist_cauchy method*), 341
sf() (*dist_chi method*), 469
sf() (*dist_chi_nc method*), 580
sf() (*dist_chi_squared method*), 473
sf() (*dist_chi_squared_nc method*), 562
sf() (*dist_dagum method*), 387
sf() (*dist_dunnett1_t method*), 633
sf() (*dist_dunnett2_t method*), 636
sf() (*dist_emg method*), 441
sf() (*dist_exponential method*), 344
sf() (*dist_feller_pareto method*), 548
sf() (*dist_fisher_f method*), 522
sf() (*dist_fisher_f_2nc method*), 606
sf() (*dist_fisher_f_nc method*), 571
sf() (*dist_fisher_r2_type_I method*), 594
sf() (*dist_fisher_z method*), 551
sf() (*dist_fisk method*), 390
sf() (*dist_folded_normal method*), 444
sf() (*dist_frechet method*), 393
sf() (*dist_friedman method*), 810
sf() (*dist_gamma method*), 478
sf() (*dist_gen_beta_exp method*), 545
sf() (*dist_genbeta1 method*), 536
sf() (*dist_genbeta2 method*), 539
sf() (*dist_genlogistic method*), 542
sf() (*dist_genpareto method*), 399
sf() (*dist_geometric method*), 752
sf() (*dist_gev method*), 397
sf() (*dist_gompertz method*), 402
sf() (*dist_gumbel method*), 348
sf() (*dist_half_normal method*), 447
sf() (*dist_halperin_t method*), 642
sf() (*dist_holtsmark method*), 732
sf() (*dist_hotelling_t2 method*), 712
sf() (*dist_hyperexponential method*), 352
sf() (*dist_hypergeometric method*), 771
sf() (*dist_hypoexponential method*), 504
sf() (*dist_invchisquared method*), 482
sf() (*dist_invgamma method*), 486
sf() (*dist_johnson_sb method*), 450
sf() (*dist_johnson_su method*), 453
sf() (*dist_jterpsta_s method*), 800
sf() (*dist_kendall_tau method*), 795
sf() (*dist_kolmogorov_smirnov method*), 724
sf() (*dist_kruskal_wallis method*), 813
sf() (*dist_kumaraswamy method*), 356
sf() (*dist_landau method*), 729
sf() (*dist_laplace method*), 360
sf() (*dist_levy method*), 416
sf() (*dist_lindley method*), 507
sf() (*dist_logistic method*), 364
sf() (*dist_lognormal method*), 419
sf() (*dist_logrv_beta method*), 530
sf() (*dist_logrv_beta_nc_type_II method*), 590
sf() (*dist_logrv_beta_sum method*), 651
sf() (*dist_logrv_chisquared method*), 500
sf() (*dist_logrv_fisher_1mr2 method*), 599
sf() (*dist_lomax method*), 405
sf() (*dist_lrt_s0 method*), 698
sf() (*dist_lrt_x0_s0 method*), 702
sf() (*dist_mann_whitney_u method*), 783
sf() (*dist_mann_whitney_u_lehmann method*), 788
sf() (*dist_mann_whitney_u_milton method*), 791
sf() (*dist_mapairy method*), 737
sf() (*dist_mauchley method*), 680
sf() (*dist_maxwell method*), 490
sf() (*dist_moyal method*), 423
sf() (*dist_nair_t method*), 639
sf() (*dist_nakagami method*), 494
sf() (*dist_negbinom method*), 766
sf() (*dist_nelson_h method*), 645
sf() (*dist_nmax_0 method*), 456
sf() (*dist_nmax_rho method*), 612
sf() (*dist_nmax_rhoij method*), 618
sf() (*dist_nmm_0 method*), 459
sf() (*dist_nmm_rho method*), 615
sf() (*dist_nmm_rhoij method*), 621
sf() (*dist_normal method*), 427
sf() (*dist_normal_range method*), 624
sf() (*dist_page_l method*), 804
sf() (*dist_page_l_nc_milton method*), 807
sf() (*dist_pareto method*), 368
sf() (*dist_pearson_rho method*), 558
sf() (*dist_pearson_rho_nc method*), 586
sf() (*dist_pillai_v method*), 709
sf() (*dist_poisson method*), 756
sf() (*dist_rayleigh method*), 372
sf() (*dist_rice method*), 583
sf() (*dist_roy method*), 706
sf() (*dist_sasnormal method*), 462
sf() (*dist_saspoint5 method*), 742
sf() (*dist_shifted_gompertz method*), 408
sf() (*dist_singh_maddala method*), 411
sf() (*dist_skewnormal method*), 431

sf() (*dist_skewt method*), 555
sf() (*dist_smax method*), 627
sf() (*dist_smm method*), 630
sf() (*dist_stacy method*), 513
sf() (*dist_student_t method*), 527
sf() (*dist_student_t_2nc method*), 603
sf() (*dist_student_t_nc method*), 568
sf() (*dist_studentized_range method*), 648
sf() (*dist_triangular method*), 376
sf() (*dist_truncated_normal method*), 465
sf() (*dist_uniform method*), 380
sf() (*dist_wald method*), 435
sf() (*dist_weibull method*), 384
sf() (*dist_wilcoxon method*), 776
sf() (*dist_wilks_cs method*), 664
sf() (*dist_wilks_iblocks method*), 676
sf() (*dist_wilks_ip method*), 672
sf() (*dist_wilks_lambda method*), 660
sf() (*dist_wilks_lambda_corr method*), 719
sf() (*dist_wilks_lambda_glm method*), 715
sf() (*dist_wilks_lcm method*), 668
sf() (*rv_base method*), 322
sf() (*skewexponpower method*), 510
shanks() (*ctx method*), 918
shape() (*npm method*), 1120
sheppard() (*ctx method*), 899
sheppard_per_cgf() (*ctx method*), 899
shifted_sin_integral() (*math53 method*), 2229
sign() (*ctx method*), 229
signbit() (*ctx method*), 208
signgamma() (*math53 method*), 2165
signtest_pmf_vector() (*ctx method*), 1538
SignтestPmfVector() (*math53lib method*), 2286
sin() (*ctx method*), 258
sin() (*ctxlib method*), 2262
sin_integral() (*ctx method*), 1439
sinc() (*ctx method*), 267
sincpi() (*ctx method*), 270
sind() (*math53 method*), 2156
sinh() (*ctx method*), 272
sinh_integral() (*ctx method*), 1436
sinhc() (*math53 method*), 2161
sinhmx() (*math53 method*), 2161
SinhSinh() (*ctxboost method*), 852
sinint() (*math53 method*), 2160
sinlemn() (*math53 method*), 2187
sinpi() (*ctx method*), 268
skewness() (*dist_bernoulli method*), 749
skewness() (*dist_holtsmark method*), 734
skewness() (*dist_kolmogorov_smirnov method*), 725
skewness() (*dist_mapairy method*), 739
skewness() (*dist_saspoint5 method*), 744
skewness() (*rv_base method*), 325
skewnormal_cdf() (*math53 method*), 430
skewnormal_pdf() (*math53 method*), 430
skewnormal_qtf() (*math53 method*), 430
SmaxCdf() (*math53lib method*), 2278
SmaxDist() (*math53lib method*), 2278
SmaxPdf() (*math53lib method*), 2278
SmaxQtf() (*math53lib method*), 2278
SmmCdf() (*math53lib method*), 2279
SmmDist() (*math53lib method*), 2279
SmmPdf() (*math53lib method*), 2279
SmmQtf() (*math53lib method*), 2279
sort() (*npm method*), 1147
sort_complex() (*npm method*), 1150
spearman_mu8() (*ctx method*), 1552
spearman_rho_pmf_vector() (*ctx method*), 1539
SpearmanRhoPmfVector() (*math53lib method*), 2285
sph_bessel_in() (*ctx method*), 1362
sph_bessel_in_prime() (*ctx method*), 1365
sph_bessel_jn() (*ctx method*), 1356
sph_bessel_jn_prime() (*ctx method*), 1358
sph_bessel_jn_zero() (*ctx method*), 1360
sph_bessel_kn() (*ctx method*), 1364
sph_bessel_kn_prime() (*ctx method*), 1366
sph_bessel_yn() (*ctx method*), 1357
sph_bessel_yn_prime() (*ctx method*), 1359
sph_bessel_yn_zero() (*ctx method*), 1361
sph_hankel_h1() (*ctx method*), 1371
sph_hankel_h2() (*ctx method*), 1372
spherical_harmonic() (*ctx method*), 1458
split() (*npm method*), 1135
sqr() (*ctx method*), 251
sqrt() (*ctx method*), 232
sqrt1mz2() (*math53 method*), 2232
sqrt1pm1() (*ctx method*), 234
sqrt1pmx() (*math53 method*), 2150
squadx() (*ctx method*), 819
square() (*npm method*), 1167
squaredNorm() (*mat method*), 1215
squeeze() (*npm method*), 1128
stack() (*npm method*), 1130
standardized() (*mat method*), 1214
stdev() (*dist_bernoulli method*), 748
stdev() (*dist_holtsmark method*), 733
stdev() (*dist_kolmogorov_smirnov method*), 725
stdev() (*dist_mapairy method*), 738
stdev() (*dist_saspoint5 method*), 743
stdev() (*rv_base method*), 325
StdrangeCdf() (*math53lib method*), 2284
StdrangeDist() (*math53lib method*), 2284
StdrangePdf() (*math53lib method*), 2284
StdrangeQtf() (*math53lib method*), 2284
struve_h() (*ctx method*), 1479
struve_h0() (*math53 method*), 2230
struve_h1() (*math53 method*), 2230
struve_k() (*ctx method*), 1481

struve_l() (*ctx method*), 1480
struve_l0() (*math53 method*), 2230
struve_l1() (*math53 method*), 2230
struve_m() (*ctx method*), 1482
student_t_1sample_power() (*ctx method*), 2292
student_t_1sample_samplesize() (*ctx method*), 2294
student_t_1sample_test() (*ctx method*), 2289
student_t_cdf() (*math53 method*), 526
student_t_nc2_broda_cdf() (*ctx method*), 1551
student_t_nc2_broda_qtf() (*ctx method*), 1552
student_t_nc2_ecf() (*ctx method*), 605
student_t_nc2_ecf_inv() (*ctx method*), 605
student_t_nc_akahira_cl() (*ctx method*), 1550
student_t_nc_benton_cdf_sf() (*ctx method*), 1516
student_t_nc_broda_cdf() (*ctx method*), 1549
student_t_nc_broda_pdf() (*ctx method*), 1549
student_t_nc_cdf() (*math53 method*), 566
student_t_nc_ecf() (*ctx method*), 569
student_t_nc_ecf_inv() (*ctx method*), 569
student_t_nc_harley_qtf() (*ctx method*), 1550
student_t_nc_owen_cdf() (*ctx method*), 1507
student_t_nc_pdf() (*math53 method*), 566
student_t_nc_qtf() (*math53 method*), 566
student_t_owen_cdf() (*ctx method*), 1504
student_t_pdf() (*math53 method*), 526
student_t_qtf() (*math53 method*), 526
studentt_2csamples_ci() (*ctx method*), 2319
studentt_2csamples_power() (*ctx method*), 2321
studentt_2csamples_samplesize() (*ctx method*), 2323
studentt_2csamples_test() (*ctx method*), 2317
studentt_2isamples_ci() (*ctx method*), 2301
studentt_2isamples_equivalence() (*ctx method*), 2308
studentt_2isamples_power() (*ctx method*), 2303
studentt_2isamples_power2() (*ctx method*), 2307
studentt_2isamples_samplesize() (*ctx method*), 2305
studentt_2isamples_test() (*ctx method*), 2299
subtract() (*npm method*), 1165
sum() (*npm method*), 1155
sumap() (*ctx method*), 914
sumem() (*ctx method*), 913
summary() (*mat method*), 1217
superfactorial() (*ctx method*), 1325
support_lower_endpoint() (*Dist method*), 326
support_lower_endpoint() (*dist_bernoulli method*), 749
support_lower_endpoint() (*dist_holtsmark method*), 734
support_lower_endpoint() (*dist_kolmogorov_smirnov method*), 726
support_lower_endpoint() (*dist_mapairy method*), 739
support_lower_endpoint() (*dist_saspoint5 method*), 744
support_upper_endpoint() (*Dist method*), 326
support_upper_endpoint() (*dist_bernoulli method*), 749
support_upper_endpoint() (*dist_holtsmark method*), 734
support_upper_endpoint() (*dist_kolmogorov_smirnov method*), 726
support_upper_endpoint() (*dist_mapairy method*), 739
support_upper_endpoint() (*dist_saspoint5 method*), 744
surd() (*mathc53 method*), 2232
svd() (*ctx method*), 1227
svd_c() (*ctx method*), 1226
svd_r() (*ctx method*), 1225
swapaxes() (*npm method*), 1124
synchrotron_f() (*math53 method*), 2213
synchrotron_g() (*math53 method*), 2213

T

t() (*ctx method*), 96
t() (*npm method*), 1085
take_along_axis() (*npm method*), 1150
tan() (*ctx method*), 261
tand() (*math53 method*), 2157
tangent_int_2() (*math53 method*), 2195
tanh() (*ctx method*), 275
TanhSinh() (*ctxboost method*), 851
tanpi() (*ctx method*), 269
taylor() (*ctx method*), 894
tensordot() (*npm method*), 1175
tetragamma() (*math53 method*), 2191
tile() (*npm method*), 1137
toroidal_plm() (*ctx method*), 1458
toroidal_qlm() (*ctx method*), 1459
trace() (*mat method*), 1215
transport_jn() (*math53 method*), 2196
transpose() (*npm method*), 1124
trapezoid() (*npm method*), 1159
Trapezoidal() (*ctxboost method*), 848
tri() (*npm method*), 1093
triangular_cdf() (*math53 method*), 375
triangular_pdf() (*math53 method*), 375
triangular_qtf() (*math53 method*), 375
tridiag_eigen(), 1232
trigamma() (*ctx method*), 1297
tril() (*npm method*), 1111
trilog() (*ctx method*), 1303
trim_zeros() (*npm method*), 1142
triu() (*npm method*), 1112
trunc() (*ctx method*), 213
trunc() (*npm method*), 1187

U

ulp() (*ctx method*), 222
uniform_cdf() (*math53 method*), 379
uniform_pdf() (*math53 method*), 379
uniform_qtf() (*math53 method*), 379
unique() (*npm method*), 1142
unitroot() (*ctx method*), 237

V

vander() (*npm method*), 1112

var() (*npm method*), 1170
variance() (*dist_bernoulli method*), 748
variance() (*dist_holtsmark method*), 733
variance() (*dist_kolmogorov_smirnov method*), 725
variance() (*dist_mapairy method*), 738
variance() (*dist_saspoint5 method*), 743
variance() (*rv_base method*), 325
vdot() (*npm method*), 1171
vectorize() (*npm method*), 1188
vers() (*math53 method*), 2159
versint() (*math53 method*), 2159
voigt_h() (*math53 method*), 2225
voigt_profile_pdf() (*math53 method*), 2226
voigt_u() (*math53 method*), 2225
voigt_v() (*math53 method*), 2225
vsplit() (*npm method*), 1137
vstack() (*npm method*), 1132

W

wald_cdf() (*math53 method*), 434
wald_pdf() (*math53 method*), 434
wald_qtf() (*math53 method*), 434
weber_e() (*ctx method*), 1484
weibull_cdf() (*math53 method*), 383
weibull_pdf() (*math53 method*), 383
weibull_qtf() (*math53 method*), 383
weierstrass_p_e() (*ctx method*), 2241
weierstrass_p_g() (*ctx method*), 2234
weierstrass_p_inv_e() (*ctx method*), 2242
weierstrass_p_inv_t() (*ctx method*), 1284
weierstrass_p_invG() (*ctx method*), 2240
weierstrass_p_prime_() (*ctx method*), 2236
weierstrass_p_prime_t() (*ctx method*), 1283
weierstrass_p_t() (*ctx method*), 1283
weierstrass_pprime_e() (*ctx method*), 2241
weierstrass_sigma_e() (*ctx method*), 2242
weierstrass_sigma_g() (*ctx method*), 2239
weierstrass_sigma_t() (*ctx method*), 1286
weierstrass_zeta_e() (*ctx method*), 2241
weierstrass_zeta_g() (*ctx method*), 2238
weierstrass_zeta_t() (*ctx method*), 1285
where() (*npm method*), 1153
whittaker_m() (*ctx method*), 1409
whittaker_w() (*ctx method*), 1410
wilcoxon_bennett_pmf_vector() (*ctx method*), 1538
wilcoxon_ecf() (*ctx method*), 779
wilcoxon_ecf_inv() (*ctx method*), 779
wilcoxon_fm_pmf() (*ctx method*), 1523
wilcoxon_ft() (*ctx method*), 779
wilcoxon_pmf_vector() (*ctx method*), 1538
wilcoxon_spa() (*ctx method*), 779
wilcoxon_spa_inv() (*ctx method*), 779
WilcoxonPmfVector() (*math53lib method*), 2286
wilks_iblocks_bd() (*ctx method*), 679
wilks_iblocks_bd_inv() (*ctx method*), 679
wilks_lambda_bd() (*ctx method*), 663
wilks_lambda_bd_inv() (*ctx method*), 663
wilks_lambda_ecf() (*ctx method*), 662
wilks_lambda_ecf_inv() (*ctx method*), 662
wilks_lambda_glm_chi2_cdf() (*ctx method*), 1571
wilks_lambda_glm_gp() (*ctx method*), 717
wilks_lambda_glm_inv() (*ctx method*), 717
wilks_lambda_glm_mu2_cdf() (*ctx method*), 1579
wilks_lambda_glm_spa() (*ctx method*), 717
wilks_lambda_gp() (*ctx method*), 662
wilks_lambda_ind_chi2_cdf() (*ctx method*), 1573
wilks_lambda_ind_gp() (*ctx method*), 721
wilks_lambda_ind_mu2_cdf() (*ctx method*), 1579
wilks_lambda_ind_spa() (*ctx method*), 721
wilks_lambda_ind_spa_inv() (*ctx method*), 721
wilks_lambda_rao_cdf() (*ctx method*), 1566
wilks_lambda_rao_qtf() (*ctx method*), 1566
wilks_lambda_spa() (*ctx method*), 662
wilks_lambda_spa_inv() (*ctx method*), 662
wright_omega() (*math53 method*), 2153

Z

zbrent() (*ctx method*), 825
zernike_r() (*ctx method*), 1453
zero (*ctx property*), 205
zeroin() (*ctx method*), 826
zeros() (*npm method*), 1090
zeros_like() (*npm method*), 1090
zeta() (*ctx method*), 1326
zeta1p() (*math53 method*), 2197
zeta_cdf() (*math53 method*), 2201
zeta_i() (*math53 method*), 2197
zeta_pmf() (*math53 method*), 2200
zeta_zero() (*ctx method*), 1337
zetam1() (*ctx method*), 1328